

Help on package pandas:

NAME

pandas

DESCRIPTION

pandas - a powerful data analysis and manipulation library for Python
=====

****pandas**** is a Python package providing fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, ****real world**** data analysis in Python. Additionally, it has the broader goal of becoming ****the most powerful and flexible open source data analysis / manipulation tool available in any language****. It is already well on its way toward this goal.

Main Features

Here are just a few of the things that pandas does well:

- Easy handling of missing data in floating point as well as non-floating point data.
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects
- Automatic and explicit data alignment: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let `'Series'`, `'DataFrame'`, etc. automatically align the data for you in computations.
- Powerful, flexible group by functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data.
- Make it easy to convert ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects.
- Intelligent label-based slicing, fancy indexing, and subsetting of large data sets.
- Intuitive merging and joining data sets.
- Flexible reshaping and pivoting of data sets.
- Hierarchical labeling of axes (possible to have multiple labels per tick).
- Robust IO tools for loading data from flat files (CSV and delimited), Excel files, databases, and saving/loading data from the ultrafast HDF5 format.
- Time series-specific functionality: date range generation and frequency conversion, moving window statistics, date shifting and lagging.

PACKAGE CONTENTS

`_config (package)`
`_libs (package)`
`_testing (package)`
`_typing`
`_version`
`_version_meson`
`api (package)`
`arrays (package)`
`compat (package)`
`conftest`
`core (package)`
`errors (package)`
`io (package)`
`plotting (package)`
`testing`
`tests (package)`
`tseries (package)`
`util (package)`

SUBMODULES

`offsets`

CLASSES

`builtins.object`
 `ExcelFile`
 `Flags`
 `Grouper`
 `HDFStore`
 `NamedAgg`
`pandas._libs.interval.IntervalMixin(builtins.object)`
 `Interval`
`pandas._libs.tslibs.dtypes.PeriodDtypeBase(builtins.object)`

```

        PeriodDtype(pandas._libs.tslibs.dtypes.PeriodDtypeBase, pandas.core.dtypes.dtypes.PandasExtensionDtype)
pandas._libs.tslibs.offsets.RelativeDeltaOffset(pandas._libs.tslibs.offsets.BaseOffset)
    DateOffset
pandas._libs.tslibs.period._Period(pandas._libs.tslibs.period.PeriodMixin)
    Period
pandas._libs.tslibs.timedeltas._Timedelta(datetime.timedelta)
    Timedelta
pandas._libs.tslibs.timestamps._Timestamp(pandas._libs.tslibs.base.ABCTimestamp)
    Timestamp
pandas.api.extensions.ExtensionDtype(builtins.object)
    CategoricalDtype(pandas.core.dtypes.dtypes.PandasExtensionDtype, pandas.api.extensions.ExtensionDtype)
    SparseDtype
pandas.core.arraylike.OpsMixin(builtins.object)
    DataFrame(pandas.core.generic.NDFrame, pandas.core.arraylike.OpsMixin)
pandas.core.arrays._mixins.NDArrayBackedExtensionArray(pandas._libs.arrays.NDArrayBacked, pandas.core.generic.NDFrame)
    Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.PandasObject)
pandas.core.arrays.floating.FloatingDtype(pandas.core.arrays.numeric.NumericDtype)
    Float32Dtype
    Float64Dtype
pandas.core.arrays.integer.IntegerDtype(pandas.core.arrays.numeric.NumericDtype)
    Int16Dtype
    Int32Dtype
    Int64Dtype
    Int8Dtype
    UInt16Dtype
    UInt32Dtype
    UInt64Dtype
    UInt8Dtype
pandas.core.base.IndexOpsMixin(pandas.core.arraylike.OpsMixin)
    Index(pandas.core.base.IndexOpsMixin, pandas.core.base.PandasObject)
        MultiIndex
        RangeIndex
    Series(pandas.core.base.IndexOpsMixin, pandas.core.generic.NDFrame)
pandas.core.base.PandasObject(pandas.core.accessor.DirNamesMixin)
    Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.PandasObject)
    Index(pandas.core.base.IndexOpsMixin, pandas.core.base.PandasObject)
        MultiIndex
        RangeIndex
pandas.core.dtypes.base.StorageExtensionDtype(pandas.api.extensions.ExtensionDtype)
    ArrowDtype
    StringDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype(pandas.api.extensions.ExtensionDtype)
    BooleanDtype
pandas.core.dtypes.dtypes.PandasExtensionDtype(pandas.api.extensions.ExtensionDtype)
    CategoricalDtype(pandas.core.dtypes.dtypes.PandasExtensionDtype, pandas.api.extensions.ExtensionDtype)
    DatetimeTZDtype
    IntervalDtype
    PeriodDtype(pandas._libs.tslibs.dtypes.PeriodDtypeBase, pandas.core.dtypes.dtypes.PandasExtensionDtype)
pandas.core.generic.NDFrame(pandas.core.base.PandasObject, pandas.core.indexing.IndexingMixin)
    DataFrame(pandas.core.generic.NDFrame, pandas.core.arraylike.OpsMixin)
    Series(pandas.core.base.IndexOpsMixin, pandas.core.generic.NDFrame)
pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin(pandas.core.indexes.extension.NDArrayBackedExtensionArray)
    PeriodIndex
pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin(pandas.core.indexes.datetimelike.DatetimeIndex)
    DatetimeIndex
    TimedeltaIndex
pandas.core.indexes.extension.ExtensionIndex(Index)
    IntervalIndex
pandas.core.indexes.extension.NDArrayBackedExtensionIndex(pandas.core.indexes.extension.ExtensionIndex)
    CategoricalIndex
pandas.core.strings.object_array.ObjectStringArrayMixin(builtins.object)
    Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.PandasObject)
typing.Generic(builtins.object)
    ExcelWriter

class ArrowDtype(pandas.core.dtypes.base.StorageExtensionDtype)
|     ArrowDtype(pyarrow_dtype: pa.DataType) -> None
|
|     An ExtensionDtype for PyArrow data types.
|
|     .. warning::
|
|         ArrowDtype is considered experimental. The implementation and
|         parts of the API may change without warning.
|
|     While most ``dtype`` arguments can accept the "string"
|     constructor, e.g. ``"int64[pyarrow]"``, ArrowDtype is useful

```

if the data type contains parameters like ``pyarrow.timestamp``.

Parameters

pyarrow_dtype : pa.DataType

An instance of a `pyarrow.DataType` <<https://arrow.apache.org/docs/python/api/datatypes.html>>

Attributes

pyarrow_dtype

Methods

None

Returns

ArrowDtype

See Also

DataFrame.convert_dtypes : Convert columns to the best possible dtypes.

Examples

```
>>> import pyarrow as pa
```

```
>>> pd.ArrowDtype(pa.int64())
```

```
int64[pyarrow]
```

Types with parameters must be constructed with ArrowDtype.

```
>>> pd.ArrowDtype(pa.timestamp("s", tz="America/New_York"))
```

```
timestamp[s, tz=America/New_York][pyarrow]
```

```
>>> pd.ArrowDtype(pa.list_(pa.int64()))
```

```
list<item: int64>[pyarrow]
```

Method resolution order:

ArrowDtype

pandas.core.dtypes.base.StorageExtensionDtype

pandas.api.extensions.ExtensionDtype

builtins.object

Methods defined here:

```
__eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.ArrowDtype
    Check whether 'other' is equal to self.
```

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes in ``self.\_metadata`` are equal between `self` and `other`.

Parameters

other : Any

Returns

bool

```
__from_arrow__(self, array: pa.Array | pa.ChunkedArray) -> ArrowExtensionArray from pandas.core.dtypes.dtypes.ArrowDtype
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.
```

```
__hash__(self) -> int from pandas.core.dtypes.dtypes.ArrowDtype
    Return hash(self).
```

```
__init__(self, pyarrow_dtype: pa.DataType) -> None from pandas.core.dtypes.dtypes.ArrowDtype
    Initialize self. See help(type(self)) for accurate signature.
```

```
__repr__(self) -> str from pandas.core.dtypes.dtypes.ArrowDtype
    Return repr(self).
```

```
construct_array_type(self) -> type_t[ArrowExtensionArray] from pandas.core.dtypes.dtypes.ArrowDtype
    Return the array type associated with this dtype.
```

Returns

```

-----
type
-----
Class methods defined here:
construct_from_string(string: str) -> ArrowDtype from pandas.core.dtypes.dtypes.ArrowDtype
    Construct this type from a string.

Parameters
-----
string : str
    string should follow the format f"{pyarrow_type}[pyarrow]"
    e.g. int64[pyarrow]
-----
Readonly properties defined here:

name
    A string identifying the data type.

type
    Returns associated scalar type.
-----
Data descriptors defined here:

itemsize
    Return the number of bytes in this dtype.

    For Arrow-backed dtypes:
    - Returns the fixed-width bit size divided by 8 for standard fixed-width types.
    - For boolean types, returns the NumPy itemsize.
    - Falls back to the NumPy dtype itemsize for variable-width & unsupported types.

Examples
-----
>>> import pyarrow as pa
>>> import pandas as pd
>>> dtype = pd.ArrowDtype(pa.int32())
>>> dtype.itemsize
4

>>> dtype = pd.ArrowDtype(pa.bool_())
>>> dtype.itemsize # falls back to numpy dtype
1

kind

numpy_dtype
    Return an instance of the related numpy dtype
-----
Methods inherited from pandas.core.dtypes.base.StorageExtensionDtype:

__str__(self) -> str
    Return str(self).
-----
Readonly properties inherited from pandas.core.dtypes.base.StorageExtensionDtype:

na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.

storage
-----
Data and other attributes inherited from pandas.core.dtypes.base.StorageExtensionDtype:

__annotations__ = {}
-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

```

```

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

    Parameters
    -----
    dtype : object
        The object to check.

    Returns
    -----
    bool

    Notes
    -----
    The default implementation is True if

    1. ``cls.construct_from_string(dtype)`` is an instance
       of ``cls``.
    2. ``dtype`` is an object and is an instance of ``cls``
    3. ``dtype`` has a ``dtype`` attribute, and any of the above
       conditions is true for ``dtype.dtype``.

-----
Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names
    Ordered list of field names, or None if there are no fields.

    This is for compatibility with NumPy arrays, and may be removed in the
    future.

-----
Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

index_class
    The Index subclass to return from Index.__new__ when this dtype is
    encountered.

```

```

class BooleanDtype(pandas.core.dtypes.dtypes.BaseMaskedDtype)
    Extension dtype for boolean data.

    This is a pandas Extension dtype for boolean data with support for
    missing values. BooleanDtype is the dtype companion to :class:`BooleanArray`,
    which implements Kleene logic (sometimes called three-value logic) for
    logical operations. See :ref:`boolean.kleene` for more.

    .. warning::

        BooleanDtype is considered experimental. The implementation and
        parts of the API may change without warning.

```

Attributes

None

Methods

None

See Also

arrays.BooleanArray : Array of boolean (True/False) data with missing values.
Int64Dtype : Extension dtype for int64 integer data.
StringDtype : Extension dtype for string data.

Examples

>>> pd.BooleanDtype()
BooleanDtype

>>> pd.array([True, False, None], dtype=pd.BooleanDtype())
<BooleanArray>
[True, False, <NA>]
Length: 3, dtype: boolean

>>> pd.array([True, False, None], dtype="boolean")
<BooleanArray>
[True, False, <NA>]
Length: 3, dtype: boolean

Method resolution order:

BooleanDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object

Methods defined here:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BooleanArray from pandas.
Construct BooleanArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str from pandas.core.arrays.boolean.BooleanDtype
Return repr(self).

construct_array_type(self) -> type_t[BooleanArray] from pandas.core.arrays.boolean.BooleanDtype.
Return the array type associated with this dtype.

Returns

type

Readonly properties defined here:

kind

A character code (one of 'biufcmMOSUV'), default 'O'

This should match the NumPy dtype used when the array is converted to an ndarray, which is probably 'O' for object if the extension type cannot be represented as a built-in NumPy type.

See Also

numpy.dtype.kind

numpy_dtype

Return an instance of our numpy dtype

type

The scalar type for the array, e.g. ``int``

It's expected ``ExtensionArray[item]`` returns an instance of ``ExtensionDtype.type`` for scalar ``item``, assuming that value is valid (not NA). NA values do not need to be instances of ``type``.

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'boolean'
```

Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.
```

Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.
```

Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
itemsize
    Return the number of bytes in this dtype
```

Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
base = None
```

Methods inherited from pandas.api.extensions.ExtensionDtype:

```
__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.
```

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes in ``self.\_metadata`` are equal between `self` and `other`.

Parameters

other : Any

Returns

bool

```
__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).
```

```
__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.
```

```
__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).
```

```
empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.
```

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Class methods inherited from pandas.api.extensions.ExtensionDtype:

`construct_from_string(string: str) -> Self` from `pandas.core.dtypes.base.ExtensionDtype`
Construct this type from a string.

This is useful mainly for data types that accept parameters.
For example, a `period` dtype accepts a frequency parameter that can be set as `period[h]` (where H means hourly frequency).

By default, in the abstract class, just the name of the type is expected. But subclasses can overwrite this method to accept parameters.

Parameters

`string : str`
The name of the type, for example `category`.

Returns

`ExtensionDtype`
Instance of the dtype.

Raises

`TypeError`
If a class cannot be constructed from this 'string'.

Examples

For extension dtypes with arguments the following may be an adequate implementation.

```
>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )
```

`is_dtype(dtype: object) -> bool` from `pandas.core.dtypes.base.ExtensionDtype`
Check if we match 'dtype'.

Parameters

`dtype : object`
The object to check.

Returns

`bool`

Notes

The default implementation is True if

1. `cls.construct_from_string(dtype)` is an instance of `cls`.
2. `dtype` is an object and is an instance of `cls`.
3. `dtype` has a `dtype` attribute, and any of the above conditions is true for `dtype.dtype`.

Readonly properties inherited from `pandas.api.extensions.ExtensionDtype`:

`names`

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from `pandas.api.extensions.ExtensionDtype`:


```

|   __dict__
|       dictionary for instance variables
|
|   __weakref__
|       list of weak references to the object
|
|   index_class
|       The Index subclass to return from Index.__new__ when this dtype is
|       encountered.
|
class Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.Panda
    Categorical(
        values,
        categories=None,
        ordered=None,
        dtype: Dtype | None = None,
        copy: bool = True
    ) -> None

    Represent a categorical variable in classic R / S-plus fashion.

    `Categoricals` can only take on a limited, and usually fixed, number
    of possible values (`categories`). In contrast to statistical categorical
    variables, a `Categorical` might have an order, but numerical operations
    (additions, divisions, ...) are not possible.

    All values of the `Categorical` are either in `categories` or `np.nan`.
    Assigning values outside of `categories` will raise a `ValueError`. Order
    is defined by the order of the `categories`, not lexical order of the
    values.

    Parameters
    -----
    values : list-like
        The values of the categorical. If categories are given, values not in
        categories will be replaced with NaN.
    categories : Index-like (unique), optional
        The unique categories for this categorical. If not given, the
        categories are assumed to be the unique values of `values` (sorted, if
        possible, otherwise in the order in which they appear).
    ordered : bool, default False
        Whether or not this categorical is treated as an ordered categorical.
        If True, the resulting categorical will be ordered.
        An ordered categorical respects, when sorted, the order of its
        `categories` attribute (which in turn is the `categories` argument, if
        provided).
    dtype : CategoricalDtype
        An instance of ``CategoricalDtype`` to use for this categorical.
    copy : bool, default True
        Whether to copy if the codes are unchanged.

    Attributes
    -----
    categories : Index
        The categories of this categorical.
    codes : ndarray
        The codes (integer positions, which point to the categories) of this
        categorical, read only.
    ordered : bool
        Whether or not this Categorical is ordered.
    dtype : CategoricalDtype
        The instance of ``CategoricalDtype`` storing the ``categories``
        and ``ordered``.

    Methods
    -----
    from_codes
    as_ordered
    as_unordered
    set_categories
    rename_categories
    reorder_categories
    add_categories
    remove_categories
    remove_unused_categories
    map

```

__array__

Raises

ValueError

If the categories do not validate.

TypeError

If an explicit ``ordered=True`` is given but no `categories` and the `values` are not sortable.

See Also

CategoricalDtype : Type for categorical data.

CategoricalIndex : An Index with an underlying ``Categorical``.

Notes

See the `user guide

<https://pandas.pydata.org/pandas-docs/stable/user_guide/categorical.html>`\_\_` for more.

Examples

```
>>> pd.Categorical([1, 2, 3, 1, 2, 3])
```

```
[1, 2, 3, 1, 2, 3]
```

```
Categories (3, int64): [1, 2, 3]
```

```
>>> pd.Categorical(["a", "b", "c", "a", "b", "c"])
```

```
['a', 'b', 'c', 'a', 'b', 'c']
```

```
Categories (3, str): ['a', 'b', 'c']
```

Missing values are not included as a category.

```
>>> c = pd.Categorical([1, 2, 3, 1, 2, 3, np.nan])
```

```
>>> c
```

```
[1, 2, 3, 1, 2, 3, NaN]
```

```
Categories (3, int64): [1, 2, 3]
```

However, their presence is indicated in the `codes` attribute by code ``-1``.

```
>>> c.codes
```

```
array([ 0,  1,  2,  0,  1,  2, -1], dtype=int8)
```

Ordered `Categoricals` can be sorted according to the custom order of the categories and can have a min and max value.

```
>>> c = pd.Categorical(
```

```
...     ["a", "b", "c", "a", "b", "c"], ordered=True, categories=["c", "b", "a"]
```

```
... )
```

```
>>> c
```

```
['a', 'b', 'c', 'a', 'b', 'c']
```

```
Categories (3, str): ['c' < 'b' < 'a']
```

```
>>> c.min()
```

```
'c'
```

Method resolution order:

Categorical

pandas.core.arrays._mixins.NDArrayBackedExtensionArray

pandas._libs.arrays.NDArrayBacked

pandas.api.extensions.ExtensionArray

pandas.core.base.PandasObject

pandas.core.accessor.DirNamesMixin

pandas.core.strings.object_array.ObjectStringArrayMixin

builtins.object

Methods defined here:

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays._mixins.NDArrayBackedExtensionArray

The numpy array interface.

Users should not call this directly. Rather, it is invoked by :func:`numpy.array` and :func:`numpy.asarray`.

Parameters

dtype : np.dtype or None

```

        Specifies the dtype for the array.

copy : bool or None, optional
    See :func:`numpy.asarray`.

Returns
-----
numpy.array
    A numpy array of either the specified dtype or,
    if dtype==None (default), the same dtype as
    categorical.categories.dtype.

See Also
-----
numpy.asarray : Convert input to numpy.ndarray.

Examples
-----
>>> cat = pd.Categorical(["a", "b"], ordered=True)

The following calls ``cat.__array__``

>>> np.asarray(cat)
array(['a', 'b'], dtype=object)

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.array
__contains__(self, key) -> bool from pandas.core.arrays.categorical.Categorical
    Returns True if `key` is in this Categorical.

__eq__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__ge__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__gt__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>

__init__(
    self,
    values,
    categories=None,
    ordered=None,
    dtype: Dtype | None = None,
    copy: bool = True
) -> None from pandas.core.arrays.categorical.Categorical
    Initialize self. See help(type(self)) for accurate signature.

__iter__(self) -> Iterator from pandas.core.arrays.categorical.Categorical
    Returns an Iterator over the values of this Categorical.

__le__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__lt__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__ne__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>

__repr__(self) -> str from pandas.core.arrays.categorical.Categorical
    String representation.

__setstate__(self, state) -> None from pandas.core.arrays.categorical.Categorical
    Necessary for making this object picklable

add_categories(self, new_categories) -> Self from pandas.core.arrays.categorical.Categorical
    Add new categories.

    `new_categories` will be included at the last/highest place in the
    categories and will be unused directly after this call.

Parameters
-----
new_categories : category or list-like of category
    The new categories to be included.

Returns
-----
Categorical
    Categorical with new categories added.

```

Raises

ValueError

If the new categories include old categories or do not validate as categories

See Also

rename_categories : Rename categories.

reorder_categories : Reorder categories.

remove_categories : Remove the specified categories.

remove_unused_categories : Remove categories which are not used.

set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["c", "b", "c"])
>>> c
```

```
['c', 'b', 'c']
```

```
Categories (2, str): ['b', 'c']
```

```
>>> c.add_categories(["d", "a"])
['c', 'b', 'c']
```

```
Categories (4, str): ['b', 'c', 'd', 'a']
```

```
argsort(self, *, ascending: bool = True, kind: SortKind = 'quicksort', **kwargs) -> npt.NDArray
Return the indices that would sort the Categorical.
```

Missing values are sorted at the end.

Parameters

ascending : bool, default True

Whether the indices should result in an ascending or descending sort.

kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.

**kwargs:

passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]

See Also

numpy.ndarray.argsort

Notes

While an ordering is applied to the category values, arg-sorting in this context refers more to organizing and grouping together based on matching category values. Thus, this function can be called on an unordered Categorical instance unlike the functions 'Categorical.min' and 'Categorical.max'.

Examples

```
>>> pd.Categorical(["b", "b", "a", "c"]).argsort()
array([2, 0, 1, 3])
```

```
>>> cat = pd.Categorical(
...     ["b", "b", "a", "c"], categories=["c", "b", "a"], ordered=True
... )
>>> cat.argsort()
array([3, 0, 1, 2])
```

Missing values are placed at the end

```
>>> cat = pd.Categorical([2, None, 1])
>>> cat.argsort()
array([2, 0, 1])
```

```
as_ordered(self) -> Self from pandas.core.arrays.categorical.Categorical
Set the Categorical to be ordered.
```

```

Returns
-----
Categorical
    Ordered Categorical.

See Also
-----
as_unordered : Set the Categorical to be unordered.

Examples
-----
For :class:`pandas.Series`:

>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False
>>> ser = ser.cat.as_ordered()
>>> ser.cat.ordered
True

For :class:`pandas.CategoricalIndex`:

>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci.ordered
False
>>> ci = ci.as_ordered()
>>> ci.ordered
True

as_unordered(self) -> Self from pandas.core.arrays.categorical.Categorical
    Set the Categorical to be unordered.

Returns
-----
Categorical
    Unordered Categorical.

See Also
-----
as_ordered : Set the Categorical to be ordered.

Examples
-----
For :class:`pandas.Series`:

>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
True
>>> ser = ser.cat.as_unordered()
>>> ser.cat.ordered
False

For :class:`pandas.CategoricalIndex`:

>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"], ordered=True)
>>> ci.ordered
True
>>> ci = ci.as_unordered()
>>> ci.ordered
False

astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike from pandas.core.arrays.categorical.Categorical
    Coerce this type to another dtype

Parameters
-----
dtype : numpy dtype or pandas type
copy : bool, default True
    By default, astype always returns a newly allocated object.
    If copy is set to False and dtype is categorical, the original
    object is returned.

check_for_ordered(self, op) -> None from pandas.core.arrays.categorical.Categorical
    assert that we are ordered

describe(self) -> DataFrame from pandas.core.arrays.categorical.Categorical

```

Describes this Categorical

Returns

description: `DataFrame`

A dataframe with frequency and counts by category.

`equals(self, other: object) -> bool` from `pandas.core.arrays.categorical.Categorical`
Returns True if categorical arrays are equal.

Parameters

other : `Categorical`

Returns

bool

`isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]` from `pandas.core.arrays.categorical.C`
Check whether `values` are contained in Categorical.

Return a boolean NumPy Array showing whether each element in the Categorical matches an element in the passed sequence of `values` exactly.

Parameters

values : np.ndarray or ExtensionArray

The sequence of values to test. Passing in a single string will raise a ``TypeError``. Instead, turn a single string into a list of one element.

Returns

np.ndarray[bool]

Raises

TypeError

* If `values` is not a set or list-like

See Also

`pandas.Series.isin` : Equivalent method on Series.

Examples

```
>>> s = pd.Categorical(["llama", "cow", "llama", "beetle", "llama", "hippo"])
>>> s.isin(["cow", "llama"])
array([ True,  True,  True, False,  True, False])
```

Passing a single string as ``s.isin('llama')`` will raise an error. Use a list of one element instead:

```
>>> s.isin(["llama"])
array([ True, False,  True, False,  True, False])
```

`isna(self) -> npt.NDArray[np.bool_]` from `pandas.core.arrays.categorical.Categorical`
Detect missing values

Missing values (-1 in .codes) are detected.

Returns

np.ndarray[bool] of whether my values are null

See Also

`isna` : Top-level isna.

`isnull` : Alias of isna.

`Categorical.notna` : Boolean inverse of Categorical.isna.

`isnull = isna(self) -> npt.NDArray[np.bool_]`

`map(self, mapper, na_action: Literal['ignore'] | None = None)` from `pandas.core.arrays.categor`
Map categories using an input mapping or function.

Maps the categories to new categories. If the mapping correspondence is one-to-one the result is a `:class:`~pandas.Categorical`` which has the same order property as the original, otherwise a `:class:`~pandas.Index`` is returned. NaN values are unaffected.

If a ``dict`` or `:class:`~pandas.Series`` is used any unmapped category is mapped to ``NaN``. Note that if this happens an `:class:`~pandas.Index`` will be returned.

Parameters

`mapper` : function, dict, or Series
Mapping correspondence.
`na_action` : {None, 'ignore'}, default None
If 'ignore', propagate NaN values, without passing them to the mapping correspondence.

Returns

`pandas.Categorical` or `pandas.Index`
Mapped categorical.

See Also

`CategoricalIndex.map` : Apply a mapping correspondence on a `:class:`~pandas.CategoricalIndex``.
`Index.map` : Apply a mapping correspondence on an `:class:`~pandas.Index``.
`Series.map` : Apply a mapping correspondence on a `:class:`~pandas.Series``.
`Series.apply` : Apply more complex functions on a `:class:`~pandas.Series``.

Examples

```
>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.map(lambda x: x.upper(), na_action=None)
['A', 'B', 'C']
Categories (3, str): ['A', 'B', 'C']
>>> cat.map({"a": "first", "b": "second", "c": "third"}, na_action=None)
['first', 'second', 'third']
Categories (3, str): ['first', 'second', 'third']
```

If the mapping is one-to-one the ordering of the categories is preserved:

```
>>> cat = pd.Categorical(["a", "b", "c"], ordered=True)
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a' < 'b' < 'c']
>>> cat.map({"a": 3, "b": 2, "c": 1}, na_action=None)
[3, 2, 1]
Categories (3, int64): [3 < 2 < 1]
```

If the mapping is not one-to-one an `:class:`~pandas.Index`` is returned:

```
>>> cat.map({"a": "first", "b": "second", "c": "first"}, na_action=None)
Index(['first', 'second', 'first'], dtype='str')
```

If a ``dict`` is used, all unmapped categories are mapped to ``NaN`` and the result is an `:class:`~pandas.Index``:

```
>>> cat.map({"a": "first", "b": "second"}, na_action=None)
Index(['first', 'second', nan], dtype='str')
```

The mapping function is applied to categories, not to each value. It is therefore only called once per unique category, and the result reused for all occurrences:

```
>>> cat = pd.Categorical(["a", "a", "b"])
>>> calls = []
>>> def f(x):
...     calls.append(x)
...     return x.upper()
```

```

>>> result = cat.map(f)
>>> result
['A', 'A', 'B']
Categories (2, str): ['A', 'B']
>>> calls
['a', 'b']

max(self, *, skipna: bool = True, **kwargs) from pandas.core.arrays.categorical.Categorical
The maximum value of the object.

Only ordered `Categoricals` have a maximum!

Raises
-----
TypeError
    If the `Categorical` is not `ordered`.

Returns
-----
max : the maximum of this `Categorical`, NA if array is empty

memory_usage(self, deep: bool = False) -> int from pandas.core.arrays.categorical.Categorical
Memory usage of my values

Parameters
-----
deep : bool
    Introspect the data deeply, interrogate
    `object` dtypes for system-level memory consumption

Returns
-----
bytes used

Notes
-----
Memory usage does not include memory consumed by elements that
are not components of the array if deep=False

See Also
-----
numpy.ndarray.nbytes

min(self, *, skipna: bool = True, **kwargs) from pandas.core.arrays.categorical.Categorical
The minimum value of the object.

Only ordered `Categoricals` have a minimum!

Raises
-----
TypeError
    If the `Categorical` is not `ordered`.

Returns
-----
min : the minimum of this `Categorical`, NA value if empty

notna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.categorical.Categorical
Inverse of isna

Both missing values (-1 in .codes) and NA as a category are detected as
null.

Returns
-----
np.ndarray[bool] of whether my values are not null

See Also
-----
notna : Top-level notna.
notnull : Alias of notna.
Categorical.isna : Boolean inverse of Categorical.notna.

notnull = notna(self) -> npt.NDArray[np.bool_]

remove_categories(self, removals) -> Self from pandas.core.arrays.categorical.Categorical
Remove the specified categories.

```


The ``removals`` argument must be a subset of the current categories.
Any values that were part of the removed categories will be set to NaN.

Parameters

removals : category or list of categories
The categories which should be removed.

Returns

Categorical
Categorical with removed categories.

Raises

ValueError
If the removals are not contained in the categories

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["a", "c", "b", "c", "d"])
>>> c
['a', 'c', 'b', 'c', 'd']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c.remove_categories(["d", "a"])
[NaN, 'c', 'b', 'c', NaN]
Categories (2, str): ['b', 'c']
```

remove_unused_categories(self) -> Self from pandas.core.arrays.categorical.Categorical
Remove categories which are not used.

This method is useful when working with datasets
that undergo dynamic changes where categories may no longer be
relevant, allowing to maintain a clean, efficient data structure.

Returns

Categorical
Categorical with unused categories dropped.

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["a", "c", "b", "c", "d"])
>>> c
['a', 'c', 'b', 'c', 'd']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c[2] = "a"
>>> c[4] = "c"
>>> c
['a', 'c', 'a', 'c', 'c']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c.remove_unused_categories()
['a', 'c', 'a', 'c', 'c']
Categories (2, str): ['a', 'c']
```

rename_categories(self, new_categories) -> Self from pandas.core.arrays.categorical.Categorical
Rename categories.

This method is commonly used to re-label or adjust the category names in categorical data without changing the underlying data. It is useful in situations where you want to modify the labels used for clarity, consistency, or readability.

Parameters

`new_categories` : list-like, dict-like or callable

New categories which will replace old categories.

- * list-like: all items must be unique and the number of items in the new categories must match the existing number of categories.

- * dict-like: specifies a mapping from old categories to new. Categories not contained in the mapping are passed through and extra categories in the mapping are ignored.

- * callable : a callable that is called on all items in the old categories and whose return values comprise the new categories.

Returns

Categorical

Categorical with renamed categories.

Raises

ValueError

If new categories are list-like and do not have the same number of items than the current categories or do not validate as categories

See Also

`reorder_categories` : Reorder categories.

`add_categories` : Add new categories.

`remove_categories` : Remove the specified categories.

`remove_unused_categories` : Remove categories which are not used.

`set_categories` : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["a", "a", "b"])
>>> c.rename_categories([0, 1])
[0, 0, 1]
Categories (2, int64): [0, 1]
```

For dict-like `new_categories`, extra keys are ignored and categories not in the dictionary are passed through

```
>>> c.rename_categories({"a": "A", "c": "C"})
['A', 'A', 'b']
Categories (2, str): ['A', 'b']
```

You may also provide a callable to create the new categories

```
>>> c.rename_categories(lambda x: x.upper())
['A', 'A', 'B']
Categories (2, str): ['A', 'B']
```

`reorder_categories(self, new_categories, ordered=None)` -> Self from `pandas.core.arrays.categorical`
Reorder categories as specified in `new_categories`.

`new_categories` need to include all old categories and no new category items.

Parameters

`new_categories` : Index-like

The categories in new order.

`ordered` : bool, optional

Whether or not the categorical is treated as an ordered categorical.

If not given, do not change the ordered information.

Returns

Categorical

Categorical with reordered categories.

Raises

ValueError

If the new categories do not contain all old category items or any new ones

See Also

`rename_categories` : Rename categories.

`add_categories` : Add new categories.

`remove_categories` : Remove the specified categories.

`remove_unused_categories` : Remove categories which are not used.

`set_categories` : Set the categories to the specified ones.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser = ser.cat.reorder_categories(["c", "b", "a"], ordered=True)
>>> ser
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']
```

```
>>> ser.sort_values()
2    c
1    b
0    a
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['a', 'b', 'c'],
                  ordered=False, dtype='category')
>>> ci.reorder_categories(["c", "b", "a"], ordered=True)
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['c', 'b', 'a'],
                  ordered=True, dtype='category')
```

`set_categories(self, new_categories, ordered=None, rename: bool = False) -> Self` from pandas
Set the categories to the specified new categories.

`new_categories` can include new categories (which will result in unused categories) or remove old categories (which results in values set to `NaN`). If `rename=True`, the categories will simply be renamed (less or more items than in old categories will result in values set to `NaN` or in unused categories respectively).

This method can be used to perform more than one action of adding, removing, and reordering simultaneously and is therefore faster than performing the individual steps via the more specialised methods.

On the other hand this methods does not do checks (e.g., whether the old categories are included in the new categories on a reorder), which can result in surprising changes, for example when using special string dtypes, which do not consider a `S1` string equal to a single char python string.

Parameters

`new_categories` : Index-like

The categories in new order.

`ordered` : bool, default None

Whether or not the categorical is treated as an ordered categorical.

If not given, do not change the ordered information.

rename : bool, default False
Whether or not the new_categories should be considered as a rename of the old categories or as reordered categories.

Returns

Categorical

New categories to be used, with optional ordering changes.

Raises

ValueError

If new_categories does not validate as categories

See Also

rename_categories : Rename categories.

reorder_categories : Reorder categories.

add_categories : Add new categories.

remove_categories : Remove the specified categories.

remove_unused_categories : Remove categories which are not used.

Examples

For :class:`pandas.Series`:

```
>>> raw_cat = pd.Categorical(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ser = pd.Series(raw_cat)
>>> ser
0    a
1    b
2    c
3   NaN
dtype: category
Categories (3, str): ['a' < 'b' < 'c']
```

```
>>> ser.cat.set_categories(["A", "B", "C"], rename=True)
0    A
1    B
2    C
3   NaN
dtype: category
Categories (3, str): ['A' < 'B' < 'C']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ci
CategoricalIndex(['a', 'b', 'c', nan], categories=['a', 'b', 'c'],
                  ordered=True, dtype='category')

>>> ci.set_categories(["A", "B", "C"])
CategoricalIndex([nan, 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
>>> ci.set_categories(["A", "B", "C"], rename=True)
CategoricalIndex(['A', 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
```

set_ordered(self, value: bool) -> Self from pandas.core.arrays.categorical.Categorical
Set the ordered attribute to the boolean value.

Parameters

value : bool

Set whether this categorical is ordered (True) or not (False).

sort_values(
self,
*,
inplace: bool = False,
ascending: bool = True,
na_position: str = 'last'
) -> Self | None from pandas.core.arrays.categorical.Categorical

Sort the Categorical by category value returning a new Categorical by default.

While an ordering is applied to the category values, sorting in this context refers more to organizing and grouping together based on matching category values. Thus, this function can be called on an unordered Categorical instance unlike the functions 'Categorical.min' and 'Categorical.max'.

Parameters

`inplace` : bool, default False
Do operation in place.
`ascending` : bool, default True
Order ascending. Passing False orders descending. The ordering parameter provides the method by which the category values are organized.
`na_position` : {'first', 'last'} (optional, default='last')
'first' puts NaNs at the beginning
'last' puts NaNs at the end

Returns

Categorical or None

See Also

`Categorical.sort`
`Series.sort_values`

Examples

```
>>> c = pd.Categorical([1, 2, 2, 1, 5])
>>> c
[1, 2, 2, 1, 5]
Categories (3, int64): [1, 2, 5]
>>> c.sort_values()
[1, 1, 2, 2, 5]
Categories (3, int64): [1, 2, 5]
>>> c.sort_values(ascending=False)
[5, 2, 2, 1, 1]
Categories (3, int64): [1, 2, 5]

>>> c = pd.Categorical([1, 2, 2, 1, 5])
```

'sort_values' behaviour with NaNs. Note that 'na_position' is independent of the 'ascending' parameter:

```
>>> c = pd.Categorical([np.nan, 2, 2, np.nan, 5])
>>> c
[NaN, 2, 2, NaN, 5]
Categories (2, int64): [2, 5]
>>> c.sort_values()
[2, 2, 5, NaN, NaN]
Categories (2, int64): [2, 5]
>>> c.sort_values(ascending=False)
[5, 2, 2, NaN, NaN]
Categories (2, int64): [2, 5]
>>> c.sort_values(na_position="first")
[NaN, NaN, 2, 2, 5]
Categories (2, int64): [2, 5]
>>> c.sort_values(ascending=False, na_position="first")
[NaN, NaN, 5, 2, 2]
Categories (2, int64): [2, 5]
```

`unique(self)` -> Self from `pandas.core.arrays.categorical.Categorical`
Return the ``Categorical`` which ``categories`` and ``codes`` are unique.

Returns

Categorical

See Also

`pandas.unique`
`CategoricalIndex.unique`

Series.unique : Return unique values of Series object.

Examples

```
-----
>>> pd.Categorical(list("baabc")).unique()
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> pd.Categorical(list("baab"), categories=list("abc"), ordered=True).unique()
['b', 'a']
Categories (3, str): ['a' < 'b' < 'c']
```

value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.categorical.Categorical
Return a Series containing counts of each category.

Every category will have an entry, even those with a count of 0.

Parameters

dropna : bool, default True
Don't include counts of NaN.

Returns

counts : Series

See Also

Series.value_counts

Class methods defined here:

```
from_codes(
    codes,
    categories=None,
    ordered=None,
    dtype: Dtype | None = None,
    validate: bool = True
) -> Self from pandas.core.arrays.categorical.Categorical
Make a Categorical type from codes and categories or dtype.
```

This constructor is useful if you already have codes and categories/dtype and so do not need the (computation intensive) factorization step, which is usually done on the constructor.

If your data does not follow this convention, please use the normal constructor.

Parameters

codes : array-like of int
An integer array, where each integer points to a category in categories or dtype.categories, or else is -1 for NaN.
categories : index-like, optional
The categories for the categorical. Items need to be unique. If the categories are not given here, then they must be provided in `dtype`.
ordered : bool, optional
Whether or not this categorical is treated as an ordered categorical. If not given here or in `dtype`, the resulting categorical will be unordered.
dtype : CategoricalDtype or "category", optional
If :class:`CategoricalDtype`, cannot be used together with `categories` or `ordered`.
validate : bool, default True
If True, validate that the codes are valid for the dtype. If False, don't validate that the codes are valid. Be careful about skipping validation, as invalid codes can lead to severe problems, such as segfaults.

.. versionadded:: 2.1.0

Returns

Categorical

See Also

codes : The category codes of the categorical.
CategoricalIndex : An Index with an underlying ``Categorical``.

Examples

```
-----
>>> dtype = pd.CategoricalDtype(["a", "b"], ordered=True)
>>> pd.Categorical.from_codes(codes=[0, 1, 0, 1], dtype=dtype)
['a', 'b', 'a', 'b']
Categories (2, str): ['a' < 'b']
```

Readonly properties defined here:

categories

The categories of this categorical.

Setting assigns new values to each category (effectively a rename of each individual category).

The assigned value has to be a list-like object. All items must be unique and the number of items in the new categories must be the same as the number of items in the old categories.

Raises

ValueError

If the new categories do not validate as categories or if the number of new categories is unequal the number of old categories

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.categories
Index(['a', 'b', 'c'], dtype='str')

>>> raw_cat = pd.Categorical([None, "b", "c", None], categories=["b", "c", "d"])
>>> ser = pd.Series(raw_cat)
>>> ser.cat.categories
Index(['b', 'c', 'd'], dtype='str')
```

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.categories
Index(['a', 'b'], dtype='str')
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "c", "b", "a", "c", "b"])
>>> ci.categories
Index(['a', 'b', 'c'], dtype='str')

>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
>>> ci.categories
Index(['c', 'b', 'a'], dtype='str')
```

codes

The category codes of this categorical index.

Codes are an array of integers which are the positions of the actual values in the categories array.

There is no setter, use the other categorical methods and the normal item setter to change values in the categorical.

Returns

ndarray[int]
A non-writable view of the ``codes`` array.

See Also

Categorical.from_codes : Make a Categorical from codes.
CategoricalIndex : An Index with an underlying ``Categorical``.

Examples

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.codes
array([0, 1], dtype=int8)
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a", "b", "c"])
>>> ci.codes
array([0, 1, 2, 0, 1, 2], dtype=int8)

>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
>>> ci.codes
array([2, 0], dtype=int8)
```

dtype

The :class:`~pandas.api.types.CategoricalDtype` for this instance.

See Also

astype : Cast argument to a specified dtype.
CategoricalDtype : Type for categorical data.

Examples

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat
['a', 'b']
Categories (2, str): ['a' < 'b']
>>> cat.dtype
CategoricalDtype(categories=['a', 'b'], ordered=True, categories_dtype=str)
```

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> pd.array([1, 2, 3]).nbytes
27
```

ordered

Whether the categories have an ordered relationship.

See Also

set_ordered : Set the ordered attribute.
as_ordered : Set the Categorical to be ordered.
as_unordered : Set the Categorical to be unordered.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False

>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
```


True

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.ordered
True
```

```
>>> cat = pd.Categorical(["a", "b"], ordered=False)
>>> cat.ordered
False
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b"], ordered=True)
>>> ci.ordered
True
```

```
>>> ci = pd.CategoricalIndex(["a", "b"], ordered=False)
>>> ci.ordered
False
```

Data and other attributes defined here:

__annotations__ = {'_dtype': 'CategoricalDtype'}

__array_priority__ = 1000

__hash__ = None

Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

__getitem__(self, key: PositionalIndexer2D) -> Self | Any
Select a subset of self.

Parameters

item : int, slice, or ndarray

* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

__setitem__(self, key, value) -> None
Set one or more values inplace.

This method is not required to satisfy the pandas extension array interface.

Parameters

key : int, ndarray, or slice

When called from, e.g. ``Series.\_\_setitem\_\_``, ``key`` will be one of

* scalar int

```

        * ndarray of integers.
        * boolean ndarray
        * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
       value or values to be set of ``key``.

Returns
-----
None

Raises
-----
ValueError
    If the array is readonly and modification is attempted.

argmax(self, axis: AxisInt = 0, skipna: bool = True)
    Return the index of maximum value.

    In case of multiple occurrences of the maximum value, the index
    corresponding to the first occurrence is returned.

Parameters
-----
skipna : bool, default True

Returns
-----
int

See Also
-----
ExtensionArray.argmin : Return the index of the minimum value.

Examples
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

argmin(self, axis: AxisInt = 0, skipna: bool = True)
    Return the index of minimum value.

    In case of multiple occurrences of the minimum value, the index
    corresponding to the first occurrence is returned.

Parameters
-----
skipna : bool, default True

Returns
-----
int

See Also
-----
ExtensionArray.argmax : Return the index of the maximum value.

Examples
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

fillna(self, value, limit: int | None = None, copy: bool = True) -> Self
    Fill NA/NaN values using the specified method.

Parameters
-----
value : scalar, array-like
    If a scalar value is passed it is used to fill all missing values.
    Alternatively, an array-like "value" can be given. It's expected
    that the array-like have the same length as 'self'.
limit : int, default None
    The maximum number of entries where NA values will be filled.
copy : bool, default True
    Whether to make a copy of the data before filling. If False, then

```

the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
With NA/NaN filled.

See Also

api.extensions.ExtensionArray.dropna : Return ExtensionArray without NA values.
api.extensions.ExtensionArray.isna : A 1-D array indicating if each value is missing.

Examples

>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

insert(self, loc: int, item) -> Self
Make new ExtensionArray inserting new item at location. Follows Python list.append semantics for negative values.

Parameters

loc : int
item : object

Returns

type(self)

searchsorted(
self,
value: NumpyValueArrayLike | ExtensionArray,
side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into `self`.
side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given. If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1
The number of periods to shift. Negative values are allowed
for shifting backwards.

fill_value : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on
this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
returned.

If ``periods > len(self)`` , then an array of size
len(self) is returned, with all values filled with
``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

take(
 self,
 indices: TakeIndexer,
 *,
 allow_fill: bool = False,
 fill_value: Any = None,
 axis: AxisInt = 0
) -> Self
Take elements from an array.

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.
allow_fill : bool, default False
How to handle negative values in ``indices``.

* False: negative values in ``indices`` indicate positional indices
from the right (the default). This is similar to
:func:`numpy.take`.

```

* True: negative values in `indices` indicate
  missing values. These values are set to `fill_value`. Any other
  other negative values raise a ``ValueError``.

fill_value : any, optional
  Fill value to use for NA-indices when `allow_fill` is True.
  This may be ``None``, in which case the default NA value for
  the type, ``self.dtype.na_value``, is used.

  For many ExtensionArrays, there will be two representations of
  `fill_value`: a user-facing "boxed" scalar, and a low-level
  physical NA value. `fill_value` should be the user-facing version,
  and the implementation should handle translating that to the
  physical version for processing the take if necessary.

Returns
-----
ExtensionArray
  An array formed with selected `indices`.

Raises
-----
IndexError
  When the indices are out of bounds for the array.
ValueError
  When `indices` contains negative values other than ``-1``
  and `allow_fill` is True.

See Also
-----
numpy.take : Take elements from an array along an axis.
api.extensions.take : Take elements from an array.

Notes
-----
ExtensionArray.take is called by ``Series.__getitem``, ``.loc``,
``.iloc``, when `indices` is a sequence of values. Additionally,
it's called by :meth:`Series.reindex`, or any other method
that causes realignment, with a `fill_value`.

Examples
-----
Here's an example implementation, which relies on casting the
extension array to object dtype. This uses the helper method
:func:`pandas.api.extensions.take`.

.. code-block:: python

    def take(self, indices, allow_fill=False, fill_value=None):
        from pandas.api.extensions import take

        # If the ExtensionArray is backed by an ndarray, then
        # just pass that here instead of coercing to object.
        data = self.astype(object)

        if allow_fill and fill_value is None:
            fill_value = self.dtype.na_value

        # fill value should always be translated from the scalar
        # type for the array, to the physical storage type for
        # the data, before passing to take.

        result = take(
            data, indices, fill_value=fill_value, allow_fill=allow_fill
        )
        return self._from_sequence(result, dtype=self.dtype)

view(self, dtype: Dtype | None = None) -> ArrayLike
  Return a view on the array.

Parameters
-----
dtype : str, np.dtype, or ExtensionDtype, optional
  Default None.

Returns
-----

```

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Data descriptors inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

Methods inherited from pandas._libs.arrays.NDArrayBacked:

__len__(self, /)
 Return len(self).

__reduce__ = __reduce_cython__(...)
__reduce_cython__(self)
__setstate_cython__(self, __pyx_state)

copy(self, order='C')
delete(self, loc, axis=0)
ravel(self, order='C')
repeat(self, repeats, axis: int | np.integer = 0)
reshape(self, *args, **kwargs)
swapaxes(self, axis1, axis2)
transpose(self, *axes)

Static methods inherited from pandas._libs.arrays.NDArrayBacked:

__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
 Create and return a new object. See help(type) for accurate signature.

Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:

T

ndim

shape

size

Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:

```
__pyx_vtable__ = <capsule object NULL>
```

```
-----  
Methods inherited from pandas.api.extensions.ExtensionArray:
```

```
dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray  
Return ExtensionArray without NA values.
```

```
Returns
```

```
-----
```

```
Self
```

```
An ExtensionArray of the same type as the original but with all  
NA values removed.
```

```
See Also
```

```
-----
```

```
Series.dropna : Remove missing values from a Series.
```

```
DataFrame.dropna : Remove missing values from a DataFrame.
```

```
api.extensions.ExtensionArray.isna : Check for missing values in  
an ExtensionArray.
```

```
Examples
```

```
-----
```

```
>>> pd.array([1, 2, np.nan]).dropna()
```

```
<IntegerArray>
```

```
[1, 2]
```

```
Length: 2, dtype: Int64
```

```
duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] f  
Return boolean ndarray denoting duplicate values.
```

```
Parameters
```

```
-----
```

```
keep : {'first', 'last', False}, default 'first'
```

```
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
```

```
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
```

```
- False : Mark all duplicates as ``True``.
```

```
Returns
```

```
-----
```

```
ndarray[bool]
```

```
With true in indices where elements are duplicated and false otherwise.
```

```
See Also
```

```
-----
```

```
DataFrame.duplicated : Return boolean Series denoting  
duplicate rows.
```

```
Series.duplicated : Indicate duplicate Series values.
```

```
api.extensions.ExtensionArray.unique : Compute the ExtensionArray  
of unique values.
```

```
Examples
```

```
-----
```

```
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
```

```
array([False,  True, False, False,  True])
```

```
factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pand  
Encode the extension array as an enumerated type.
```

```
Parameters
```

```
-----
```

```
use_na_sentinel : bool, default True
```

```
If True, the sentinel -1 will be used for NaN values. If False,
```

```
NaN values will be encoded as non-negative integers and will not drop the
```

```
NaN from the uniques of the values.
```

```
Returns
```

```
-----
```

```
codes : ndarray
```

```
An integer NumPy array that's an indexer into the original  
ExtensionArray.
```

```
uniques : ExtensionArray
```

```
An ExtensionArray containing the unique values of `self`.
```

```
.. note::
```

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

>>> idx1 = pd.PeriodIndex(
... ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
... freq="M",
...)
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

interpolate(
 self,
 *,
 method: InterpolateOptions,
 axis: int,
 index: Index,
 limit,
 limit_direction,
 limit_area,
 copy: bool,
 **kwargs
) -> Self from pandas.core.arrays.base.ExtensionArray
Fill NaN values using an interpolation method.

Parameters

method : str, default 'linear'
Interpolation technique to use. One of:
* 'linear': Ignore the index and treat the values as equally spaced. This is the only method supported on MultiIndexes.
* 'time': Works on daily and higher resolution data to interpolate given length of interval.
* 'index', 'values': use the actual numerical values of the index.
* 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric', 'polynomial': Passed to scipy.interpolate.interp1d, whereas 'spline' is passed to scipy.interpolate.UnivariateSpline. These methods use the numerical values of the index.
Both 'polynomial' and 'spline' require that you also specify an order (int), e.g. arr.interpolate(method='polynomial', order=5).
* 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima', 'cubicspline': Wrappers around the SciPy interpolation methods of similar names. See Notes.
* 'from_derivatives': Refers to scipy.interpolate.BPoly.from_derivatives.
axis : int
Axis to interpolate along. For 1-dimensional data, use 0.
index : Index
Index to use for interpolation.
limit : int or None
Maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction : {'forward', 'backward', 'both'}
Consecutive NaNs will be filled in this direction.
limit_area : {'inside', 'outside'} or None
If limit is specified, consecutive NaNs will be filled with this restriction.
* None: No fill restriction.
* 'inside': Only fill NaNs surrounded by valid values (interpolate).
* 'outside': Only fill NaNs outside valid values (extrapolate).
copy : bool
If True, a copy of the object is returned with interpolated values.
**kwargs : optional
Keyword arguments to pass on to the interpolating function.

Returns

ExtensionArray
An ExtensionArray with interpolated values.

See Also

Series.interpolate : Interpolate values in a Series.
DataFrame.interpolate : Interpolate values in a DataFrame.

Notes

- All parameters must be specified as keyword arguments.
- The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima' methods are wrappers around the respective SciPy implementations of similar names. These use the actual numerical values of the index.

Examples

Interpolating values in a NumPy array:

```
>>> arr = pd.arrays.NumpyExtensionArray(np.array([0, 1, np.nan, 3]))
>>> arr.interpolate(
...     method="linear",
...     limit=3,
...     limit_direction="forward",
...     index=pd.Index(range(len(arr))),
...     fill_value=1,
...     copy=False,
...     axis=0,
...     limit_area="inside",
... )
<NumpyExtensionArray>
[0.0, 1.0, 2.0, 3.0]
Length: 4, dtype: float64
```

Interpolating values in a FloatingArray:

```
>>> arr = pd.array([1.0, pd.NA, 3.0, 4.0, pd.NA, 6.0], dtype="Float64")
>>> arr.interpolate(
...     method="linear",
...     axis=0,
...     index=pd.Index(range(len(arr))),
...     limit=None,
...     limit_direction="both",
...     limit_area=None,
...     copy=True,
... )
<FloatingArray>
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Length: 6, dtype: Float64
```

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar
The element at the specified position.

Raises

ValueError
If no index is provided and the array does not have exactly one element.
IndexError
If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```
-----
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Convert to a NumPy ndarray.
```

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

```
-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is a not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
```

Returns

```
-----
numpy.ndarray
```

```
tolist(self) -> list from pandas.core.arrays.base.ExtensionArray
Return a list of the values.
```

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

```
-----
list
    Python list of values in array.
```

See Also

```
-----
Index.tolist: Return a list of the values in the Index.
Series.tolist: Return a list of the values in the Series.
```

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

```
__pandas_priority__ = 1000
```

Methods inherited from pandas.core.base.PandasObject:

```
__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values
```

Methods inherited from pandas.core.accessor.DirNamesMixin:

```

    __dir__(self) -> list[str]
        Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.
class CategoricalDtype(pandas.core.dtypes.dtypes.PandasExtensionDtype, pandas.api.extensions.ExtensionDtype):
    CategoricalDtype(categories=None, ordered: Ordered = False) -> None

    Type for categorical data with the categories and orderedness.

    It is a dtype representation for categorical data, which allows users to define
    a fixed set of values and optionally impose an ordering. This is particularly
    useful for handling categorical variables efficiently, as it can significantly
    reduce memory usage compared to using object dtypes.

    Parameters
    -----
    categories : sequence, optional
        Must be unique, and must not contain any nulls.
        The categories are stored in an Index,
        and if an index is provided the dtype of that index will be used.
    ordered : bool or None, default False
        Whether or not this categorical is treated as an ordered categorical.
        None can be used to maintain the ordered value of existing categoricals when
        used in operations that combine categoricals, e.g. astype, and will resolve to
        False if there is no existing ordered to maintain.

    Attributes
    -----
    categories
    ordered

    Methods
    -----
    None

    See Also
    -----
    Categorical : Represent a categorical variable in classic R / S-plus fashion.

    Notes
    -----
    This class is useful for specifying the type of a ``Categorical``
    independent of the values. See :ref:`categorical.categoricaldtype`
    for more.

    Examples
    -----
    >>> t = pd.CategoricalDtype(categories=["b", "a"], ordered=True)
    >>> pd.Series(["a", "b", "a", None], dtype=t)
    0      a
    1      b
    2      a
    3     NaN
    dtype: category
    Categories (2, str): ['b' < 'a']

    An empty CategoricalDtype with a specific dtype can be created
    by providing an empty index. As follows,

    >>> pd.CategoricalDtype(pd.DatetimeIndex([])).categories.dtype
    dtype('<M8[s]')

    Method resolution order:
    CategoricalDtype
    pandas.core.dtypes.dtypes.PandasExtensionDtype
    pandas.api.extensions.ExtensionDtype
    builtins.object

    Methods defined here:

    __eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.CategoricalDtype
    Rules for CDT equality:
    1) Any CDT is equal to the string 'category'

```

```

2) Any CDT is equal to itself
3) Any CDT is equal to a CDT with categories=None regardless of ordered
4) A CDT with ordered=True is only equal to another CDT with
   ordered=True and identical categories in the same order
5) A CDT with ordered={False, None} is only equal to another CDT with
   ordered={False, None} and identical categories, but same order is
   not required. There is no distinction between False/None.
6) Any other comparison returns False

__hash__(self) -> int from pandas.core.dtypes.dtypes.CategoricalDtype
    Return hash(self).

__init__(self, categories=None, ordered: Ordered = False) -> None from pandas.core.dtypes.dtypes.CategoricalDtype
    Initialize self. See help(type(self)) for accurate signature.

__repr__(self) -> str_type from pandas.core.dtypes.dtypes.CategoricalDtype
    Return a string representation for a particular object.

__setstate__(self, state: MutableMapping[str_type, Any]) -> None from pandas.core.dtypes.dtypes.CategoricalDtype

construct_array_type(self) -> type_t[Categorical] from pandas.core.dtypes.dtypes.CategoricalDtype
    Return the array type associated with this dtype.

Returns
-----
type

update_dtype(self, dtype: str_type | CategoricalDtype) -> CategoricalDtype from pandas.core.dtypes.dtypes.CategoricalDtype
    Returns a CategoricalDtype with categories and ordered taken from dtype
    if specified, otherwise falling back to self if unspecified

Parameters
-----
dtype : CategoricalDtype

Returns
-----
new_dtype : CategoricalDtype

-----
Class methods defined here:

construct_from_string(string: str_type) -> CategoricalDtype from pandas.core.dtypes.dtypes.CategoricalDtype
    Construct a CategoricalDtype from a string.

Parameters
-----
string : str
    Must be the string "category" in order to be successfully constructed.

Returns
-----
CategoricalDtype
    Instance of the dtype.

Raises
-----
TypeError
    If a CategoricalDtype cannot be constructed from the input.

-----
Static methods defined here:

validate_categories(categories, fastpath: bool = False) -> Index from pandas.core.dtypes.dtypes.CategoricalDtype
    Validates that we have good categories

Parameters
-----
categories : array-like
fastpath : bool
    Whether to skip nan and uniqueness checks

Returns
-----
categories : Index

validate_ordered(ordered: Ordered) -> None from pandas.core.dtypes.dtypes.CategoricalDtype

```

Validates that we have a valid ordered parameter. If it is not a boolean, a `TypeError` will be raised.

Parameters

`ordered` : object
The parameter to be verified.

Raises

`TypeError`
If '`ordered`' is not a boolean.

Readonly properties defined here:

`categories`

An ``Index`` containing the unique categories allowed.

See Also

`ordered` : Whether the categories have an ordered relationship.

Examples

>>> `cat_type = pd.CategoricalDtype(categories=["a", "b"], ordered=True)`
>>> `cat_type.categories`
Index(['a', 'b'], dtype='str')

`ordered`

Whether the categories have an ordered relationship.

See Also

`categories` : An Index containing the unique categories allowed.

Examples

>>> `cat_type = pd.CategoricalDtype(categories=["a", "b"], ordered=True)`
>>> `cat_type.ordered`
True

>>> `cat_type = pd.CategoricalDtype(categories=["a", "b"], ordered=False)`
>>> `cat_type.ordered`
False

Data descriptors defined here:

`index_class`

Data and other attributes defined here:

`__annotations__` = {'_cache_dtypes': 'dict[str_type, PandasExtensionDty...

`base` = `dtype('O')`

`kind` = 'O'

`name` = 'category'

`str` = '|008'

`type` = <class 'pandas.core.dtypes.dtypes.CategoricalDtypeType'>
the type of `CategoricalDtype`, this metaclass determines subclass ability

Methods inherited from `pandas.core.dtypes.dtypes.PandasExtensionDtype`:

`__getstate__(self)` -> dict[str_type, Any]
Helper for pickle.

Class methods inherited from `pandas.core.dtypes.dtypes.PandasExtensionDtype`:

```
reset_cache() -> None
    clear the cache
```

Data and other attributes inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

```
isbuiltin = 0
```

```
isnative = 0
```

```
itemsize = 8
```

```
num = 100
```

```
shape = ()
```

```
subdtype = None
```

Methods inherited from pandas.api.extensions.ExtensionDtype:

```
__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.
```

```
__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).
```

```
empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.
```

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Class methods inherited from pandas.api.extensions.ExtensionDtype:

```
is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.
```

Parameters

dtype : object
 The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

```
na_value
    Default NA value to use for this type.
```

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

```
names
```

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

class CategoricalIndex(pandas.core.indexes.extension.NDArrayBackedExtensionIndex)

CategoricalIndex(
 data=None,
 categories=None,
 ordered=None,
 dtype: Dtype | None = None,
 copy: bool = False,
 name: Hashable | None = None
) -> Self

Index based on an underlying :class:`Categorical`.

CategoricalIndex, like Categorical, can only take on a limited, and usually fixed, number of possible values (`categories`). Also, like Categorical, it might have an order, but numerical operations (additions, divisions, ...) are not possible.

Parameters

data : array-like (1-dimensional)
 The values of the categorical. If `categories` are given, values not in `categories` will be replaced with NaN.
categories : index-like, optional
 The categories for the categorical. Items need to be unique. If the categories are not given here (and also not in `dtype`), they will be inferred from the `data`.
ordered : bool, optional
 Whether or not this categorical is treated as an ordered categorical. If not given here or in `dtype`, the resulting categorical will be unordered.
dtype : CategoricalDtype or "category", optional
 If :class:`CategoricalDtype`, cannot be used together with `categories` or `ordered`.
copy : bool, default False
 Make a copy of input ndarray.
name : object, optional
 Name to be stored in the index.

Attributes

codes
categories
ordered

Methods

rename_categories
reorder_categories
add_categories
remove_categories
remove_unused_categories
set_categories
as_ordered
as_unordered
map

Raises

ValueError
 If the categories do not validate.
TypeError
 If an explicit ``ordered=True`` is given but no `categories` and the `values` are not sortable.

See Also

Index : The base pandas Index type.
Categorical : A categorical array.
CategoricalDtype : Type for categorical data.

Notes

See the `user guide`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/advanced.html#categoricalindex>`\_\_`
for more.

Examples

>>> pd.CategoricalIndex(["a", "b", "c", "a", "b", "c"])
CategoricalIndex(['a', 'b', 'c', 'a', 'b', 'c'],
 categories=['a', 'b', 'c'], ordered=False, dtype='category')

`CategoricalIndex` can also be instantiated from a `Categorical`:

>>> c = pd.Categorical(["a", "b", "c", "a", "b", "c"])
>>> pd.CategoricalIndex(c)
CategoricalIndex(['a', 'b', 'c', 'a', 'b', 'c'],
 categories=['a', 'b', 'c'], ordered=False, dtype='category')

Ordered `CategoricalIndex` can have a min and max value.

>>> ci = pd.CategoricalIndex(
... ["a", "b", "c", "a", "b", "c"], ordered=True, categories=["c", "b", "a"]
...)
>>> ci
CategoricalIndex(['a', 'b', 'c', 'a', 'b', 'c'],
 categories=['c', 'b', 'a'], ordered=True, dtype='category')
>>> ci.min()
'c'

Method resolution order:

CategoricalIndex
pandas.core.indexes.extension.NDArrayBackedExtensionIndex
pandas.core.indexes.extension.ExtensionIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
builtins.object

Methods defined here:

__contains__(self, key: Any) -> bool from pandas.core.indexes.category.CategoricalIndex
Return a boolean indicating whether the provided key is in the index.

Parameters

key : label
The key to check if it is present in the index.

Returns

bool
Whether the key search is in the index.

Raises

TypeError
If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the
list-like key is in the index.

Examples

>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')


```
>>> 2 in idx
True
>>> 6 in idx
False
```

```
add_categories(self, new_categories) -> Self from pandas.core.indexes.extension._inherit_from
Add new categories.
```

`new\_categories` will be included at the last/highest place in the categories and will be unused directly after this call.

Parameters

new_categories : category or list-like of category
The new categories to be included.

Returns

Categorical
Categorical with new categories added.

Raises

ValueError
If the new categories include old categories or do not validate as categories

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["c", "b", "c"])
>>> c
['c', 'b', 'c']
Categories (2, str): ['b', 'c']

>>> c.add_categories(["d", "a"])
['c', 'b', 'c']
Categories (4, str): ['b', 'c', 'd', 'a']
```

```
argsort(self, *, ascending: bool = True, kind: SortKind = 'quicksort', **kwargs) -> npt.NDArray
Return the indices that would sort the Categorical.
```

Missing values are sorted at the end.

Parameters

ascending : bool, default True
Whether the indices should result in an ascending or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.
**kwargs:
passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]

See Also

numpy.ndarray.argsort

Notes

While an ordering is applied to the category values, arg-sorting in this context refers more to organizing and grouping together based on matching category values. Thus, this function can be called on an unordered Categorical instance unlike the functions 'Categorical.min' and 'Categorical.max'.

Examples

```
-----
>>> pd.Categorical(["b", "b", "a", "c"]).argsort()
array([2, 0, 1, 3])

>>> cat = pd.Categorical(
...     ["b", "b", "a", "c"], categories=["c", "b", "a"], ordered=True
... )
>>> cat.argsort()
array([3, 0, 1, 2])
```

Missing values are placed at the end

```
>>> cat = pd.Categorical([2, None, 1])
>>> cat.argsort()
array([2, 0, 1])
```

```
as_ordered(self) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
Set the Categorical to be ordered.
```

Returns

```
-----
Categorical
Ordered Categorical.
```

See Also

```
-----
as_unordered : Set the Categorical to be unordered.
```

Examples

```
-----
For :class:`pandas.Series`:
```

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False
>>> ser = ser.cat.as_ordered()
>>> ser.cat.ordered
True
```

```
For :class:`pandas.CategoricalIndex`:
```

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci.ordered
False
>>> ci = ci.as_ordered()
>>> ci.ordered
True
```

```
as_unordered(self) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
Set the Categorical to be unordered.
```

Returns

```
-----
Categorical
Unordered Categorical.
```

See Also

```
-----
as_ordered : Set the Categorical to be ordered.
```

Examples

```
-----
For :class:`pandas.Series`:
```

```
>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
True
>>> ser = ser.cat.as_unordered()
>>> ser.cat.ordered
False
```

```
For :class:`pandas.CategoricalIndex`:
```

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"], ordered=True)
```

```
>>> ci.ordered
True
>>> ci = ci.as_unordered()
>>> ci.ordered
False
```

`equals(self, other: object) -> bool` from `pandas.core.indexes.category.CategoricalIndex`
Determine if two `CategoricalIndex` objects contain the same elements.

The order and orderedness of elements matters. The categories matter, but the order of the categories matters only when `ordered=True`.

Parameters

other : object
The `CategoricalIndex` object to compare with.

Returns

bool
`True` if two `pandas.CategoricalIndex` objects have equal elements, `False` otherwise.

See Also

`Categorical.equals` : Returns `True` if categorical arrays are equal.

Examples

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a", "b", "c"])
>>> ci2 = pd.CategoricalIndex(pd.Categorical(["a", "b", "c", "a", "b", "c"]))
>>> ci.equals(ci2)
True
```

The order of elements matters.

```
>>> ci3 = pd.CategoricalIndex(["c", "b", "a", "a", "b", "c"])
>>> ci.equals(ci3)
False
```

The orderedness also matters.

```
>>> ci4 = ci.as_ordered()
>>> ci.equals(ci4)
False
```

The categories matter, but the order of the categories matters only when `ordered=True`.

```
>>> ci5 = ci.set_categories(["a", "b", "c", "d"])
>>> ci.equals(ci5)
False
```

```
>>> ci6 = ci.set_categories(["b", "c", "a"])
>>> ci.equals(ci6)
True
```

```
>>> ci_ordered = pd.CategoricalIndex(
...     ["a", "b", "c", "a", "b", "c"], ordered=True
... )
>>> ci2_ordered = ci_ordered.set_categories(["b", "c", "a"])
>>> ci_ordered.equals(ci2_ordered)
False
```

`map(self, mapper, na_action: Literal['ignore'] | None = None)` from `pandas.core.indexes.category.CategoricalIndex`
Map values using input an input mapping or function.

Maps the values (their categories, not the codes) of the index to new categories. If the mapping correspondence is one-to-one the result is a `pandas.CategoricalIndex` which has the same order property as the original, otherwise an `pandas.Index` is returned.

If a `dict` or `pandas.Series` is used any unmapped category is mapped to `NaN`. Note that if this happens an `pandas.Index` will be returned.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}, default 'ignore'
If 'ignore', propagate NaN values, without passing them to
the mapping correspondence.

Returns

pandas.CategoricalIndex or pandas.Index
Mapped index.

See Also

Index.map : Apply a mapping correspondence on an
:class:`pandas.Index`.
Series.map : Apply a mapping correspondence on a
:class:`pandas.Series`.
Series.apply : Apply more complex functions on a
:class:`pandas.Series`.

Examples

>>> idx = pd.CategoricalIndex(["a", "b", "c"])
>>> idx
CategoricalIndex(['a', 'b', 'c'], categories=['a', 'b', 'c'],
ordered=False, dtype='category')
>>> idx.map(lambda x: x.upper())
CategoricalIndex(['A', 'B', 'C'], categories=['A', 'B', 'C'],
ordered=False, dtype='category')
>>> idx.map({"a": "first", "b": "second", "c": "third"})
CategoricalIndex(['first', 'second', 'third'], categories=['first',
'second', 'third'], ordered=False, dtype='category')

If the mapping is one-to-one the ordering of the categories is
preserved:

>>> idx = pd.CategoricalIndex(["a", "b", "c"], ordered=True)
>>> idx
CategoricalIndex(['a', 'b', 'c'], categories=['a', 'b', 'c'],
ordered=True, dtype='category')
>>> idx.map({"a": 3, "b": 2, "c": 1})
CategoricalIndex([3, 2, 1], categories=[3, 2, 1], ordered=True,
dtype='category')

If the mapping is not one-to-one an :class:`pandas.Index` is returned:

>>> idx.map({"a": "first", "b": "second", "c": "first"})
Index(['first', 'second', 'first'], dtype='str')

If a `dict` is used, all unmapped categories are mapped to `NaN` and
the result is an :class:`pandas.Index`:

>>> idx.map({"a": "first", "b": "second"})
Index(['first', 'second', nan], dtype='str')

max(self, *, skipna: bool = True, **kwargs) from pandas.core.indexes.extension._inherit_from.
The maximum value of the object.

Only ordered `Categoricals` have a maximum!

Raises

TypeError
If the `Categorical` is not `ordered`.

Returns

max : the maximum of this `Categorical`, NA if array is empty

min(self, *, skipna: bool = True, **kwargs) from pandas.core.indexes.extension._inherit_from.
The minimum value of the object.

Only ordered `Categoricals` have a minimum!

Raises

TypeError

```

        If the `Categorical` is not `ordered`.

    Returns
    -----
    min : the minimum of this `Categorical`, NA value if empty

reindex(
    self,
    target,
    method=None,
    level=None,
    limit: int | None = None,
    tolerance=None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.category.Categorical
    Create index with target's values (move/add/delete values as necessary)

    Returns
    -----
    new_index : pd.Index
        Resulting index
    indexer : np.ndarray[np.intp] or None
        Indices of output values in original index

remove_categories(self, removals) -> Self from pandas.core.indexes.extension._inherit_from_data
    Remove the specified categories.

    The ``removals`` argument must be a subset of the current categories.
    Any values that were part of the removed categories will be set to NaN.

    Parameters
    -----
    removals : category or list of categories
        The categories which should be removed.

    Returns
    -----
    Categorical
        Categorical with removed categories.

    Raises
    -----
    ValueError
        If the removals are not contained in the categories

    See Also
    -----
    rename_categories : Rename categories.
    reorder_categories : Reorder categories.
    add_categories : Add new categories.
    remove_unused_categories : Remove categories which are not used.
    set_categories : Set the categories to the specified ones.

    Examples
    -----
    >>> c = pd.Categorical(["a", "c", "b", "c", "d"])
    >>> c
    ['a', 'c', 'b', 'c', 'd']
    Categories (4, str): ['a', 'b', 'c', 'd']

    >>> c.remove_categories(["d", "a"])
    [NaN, 'c', 'b', 'c', NaN]
    Categories (2, str): ['b', 'c']

remove_unused_categories(self) -> Self from pandas.core.indexes.extension._inherit_from_data
    Remove categories which are not used.

    This method is useful when working with datasets
    that undergo dynamic changes where categories may no longer be
    relevant, allowing to maintain a clean, efficient data structure.

    Returns
    -----
    Categorical
        Categorical with unused categories dropped.

    See Also
    -----

```

```
rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
set_categories : Set the categories to the specified ones.
```

Examples

```
>>> c = pd.Categorical(["a", "c", "b", "c", "d"])
>>> c
['a', 'c', 'b', 'c', 'd']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c[2] = "a"
>>> c[4] = "c"
>>> c
['a', 'c', 'a', 'c', 'c']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c.remove_unused_categories()
['a', 'c', 'a', 'c', 'c']
Categories (2, str): ['a', 'c']
```

```
rename_categories(self, new_categories) -> Self from pandas.core.indexes.extension._inherited
Rename categories.
```

This method is commonly used to re-label or adjust the category names in categorical data without changing the underlying data. It is useful in situations where you want to modify the labels used for clarity, consistency, or readability.

Parameters

new_categories : list-like, dict-like or callable

New categories which will replace old categories.

- * list-like: all items must be unique and the number of items in the new categories must match the existing number of categories.

- * dict-like: specifies a mapping from old categories to new. Categories not contained in the mapping are passed through and extra categories in the mapping are ignored.

- * callable : a callable that is called on all items in the old categories and whose return values comprise the new categories.

Returns

Categorical
Categorical with renamed categories.

Raises

ValueError

If new categories are list-like and do not have the same number of items than the current categories or do not validate as categories

See Also

```
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.
```

Examples

```
>>> c = pd.Categorical(["a", "a", "b"])
>>> c.rename_categories([0, 1])
[0, 0, 1]
Categories (2, int64): [0, 1]
```

For dict-like ``new\_categories``, extra keys are ignored and categories not in the dictionary are passed through

```
>>> c.rename_categories({"a": "A", "c": "C"})
['A', 'A', 'b']
Categories (2, str): ['A', 'b']
```

You may also provide a callable to create the new categories

```
>>> c.rename_categories(lambda x: x.upper())
['A', 'A', 'B']
Categories (2, str): ['A', 'B']
```

```
reorder_categories(self, new_categories, ordered=None) -> Self from pandas.core.indexes.extension
Reorder categories as specified in new_categories.
```

``new\_categories`` need to include all old categories and no new category items.

Parameters

```
new_categories : Index-like
    The categories in new order.
ordered : bool, optional
    Whether or not the categorical is treated as an ordered categorical.
    If not given, do not change the ordered information.
```

Returns

```
Categorical
    Categorical with reordered categories.
```

Raises

```
ValueError
    If the new categories do not contain all old category items or any
    new ones
```

See Also

```
rename_categories : Rename categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.
```

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser = ser.cat.reorder_categories(["c", "b", "a"], ordered=True)
>>> ser
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']
```

```
>>> ser.sort_values()
2    c
1    b
0    a
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['a', 'b', 'c'],
                  ordered=False, dtype='category')
>>> ci.reorder_categories(["c", "b", "a"], ordered=True)
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['c', 'b', 'a'],
                  ordered=True, dtype='category')
```

searchsorted(

```

self,
value: NumpyValueArrayLike | ExtensionArray,
side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.indexes.extension._inherit_from_data.<1
Find indices where elements should be inserted to maintain order.

```

Find the indices into a sorted array `self` (a) such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```

=====
`side`  returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====

```

Parameters

```

-----
value : array-like, list or scalar
        Value(s) to insert into `self`.
side : {'left', 'right'}, optional
        If 'left', the index of the first suitable location found is given.
        If 'right', return the last such index. If there is no suitable
        index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
        Optional array of integer indices that sort array a into ascending
        order. They are typically the result of argsort.

```

Returns

```

-----
array of ints or int
        If value is array-like, array of insertion points.
        If value is scalar, a single integer.

```

See Also

```

-----
numpy.searchsorted : Similar method from NumPy.

```

Examples

```

-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

```

```

set_categories(self, new_categories, ordered=None, rename: bool = False) -> Self from pandas
Set the categories to the specified new categories.

```

`new\_categories` can include new categories (which will result in unused categories) or remove old categories (which results in values set to `NaN`). If `rename=True`, the categories will simply be renamed (less or more items than in old categories will result in values set to `NaN` or in unused categories respectively).

This method can be used to perform more than one action of adding, removing, and reordering simultaneously and is therefore faster than performing the individual steps via the more specialised methods.

On the other hand this methods does not do checks (e.g., whether the old categories are included in the new categories on a reorder), which can result in surprising changes, for example when using special string dtypes, which do not consider a S1 string equal to a single char python string.

Parameters

```

-----
new_categories : Index-like
        The categories in new order.
ordered : bool, default None
        Whether or not the categorical is treated as an ordered categorical.
        If not given, do not change the ordered information.
rename : bool, default False
        Whether or not the new_categories should be considered as a rename
        of the old categories or as reordered categories.

```


Returns

Categorical

New categories to be used, with optional ordering changes.

Raises

ValueError

If new_categories does not validate as categories

See Also

rename_categories : Rename categories.

reorder_categories : Reorder categories.

add_categories : Add new categories.

remove_categories : Remove the specified categories.

remove_unused_categories : Remove categories which are not used.

Examples

For :class:`pandas.Series`:

```
>>> raw_cat = pd.Categorical(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ser = pd.Series(raw_cat)
>>> ser
0    a
1    b
2    c
3   NaN
dtype: category
Categories (3, str): ['a' < 'b' < 'c']
```

```
>>> ser.cat.set_categories(["A", "B", "C"], rename=True)
```

```
0    A
1    B
2    C
3   NaN
dtype: category
Categories (3, str): ['A' < 'B' < 'C']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ci
CategoricalIndex(['a', 'b', 'c', nan], categories=['a', 'b', 'c'],
                  ordered=True, dtype='category')
```

```
>>> ci.set_categories(["A", "B", "C"])
CategoricalIndex([nan, 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
```

```
>>> ci.set_categories(["A", "B", "C"], rename=True)
CategoricalIndex(['A', 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
```

tolist(self) -> list from pandas.core.indexes.extension._inherit_from_data.<locals>
Return a list of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list

Python list of values in array.

See Also

Index.tolist: Return a list of the values in the Index.

Series.tolist: Return a list of the values in the Series.

Examples

```

-----
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]

```

Static methods defined here:

```

__new__(
    cls,
    data=None,
    categories=None,
    ordered=None,
    dtype: Dtype | None = None,
    copy: bool = False,
    name: Hashable | None = None
) -> Self from pandas.core.indexes.category.CategoricalIndex
    Create and return a new object. See help(type) for accurate signature.

```

Readonly properties defined here:

inferred_type
Return a string of the type inferred from the values.

See Also

Index.dtype : Return the dtype object of the underlying data.

Examples

```

-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.inferred_type
'integer'

```

Data descriptors defined here:

categories

The categories of this categorical.

Setting assigns new values to each category (effectively a rename of each individual category).

The assigned value has to be a list-like object. All items must be unique and the number of items in the new categories must be the same as the number of items in the old categories.

Raises

ValueError

If the new categories do not validate as categories or if the number of new categories is unequal the number of old categories

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples

For :class:`pandas.Series`:

```

>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.categories
Index(['a', 'b', 'c'], dtype='str')

>>> raw_cat = pd.Categorical([None, "b", "c", None], categories=["b", "c", "d"])
>>> ser = pd.Series(raw_cat)
>>> ser.cat.categories
Index(['b', 'c', 'd'], dtype='str')

```

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.categories
Index(['a', 'b'], dtype='str')
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "c", "b", "a", "c", "b"])
>>> ci.categories
Index(['a', 'b', 'c'], dtype='str')

>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
>>> ci.categories
Index(['c', 'b', 'a'], dtype='str')
```

codes

The category codes of this categorical index.

Codes are an array of integers which are the positions of the actual values in the categories array.

There is no setter, use the other categorical methods and the normal item setter to change values in the categorical.

Returns

ndarray[int]

A non-writable view of the ``codes`` array.

See Also

Categorical.from_codes : Make a Categorical from codes.

CategoricalIndex : An Index with an underlying ``Categorical``.

Examples

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.codes
array([0, 1], dtype=int8)
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a", "b", "c"])
>>> ci.codes
array([0, 1, 2, 0, 1, 2], dtype=int8)

>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
>>> ci.codes
array([2, 0], dtype=int8)
```

ordered

Whether the categories have an ordered relationship.

See Also

set_ordered : Set the ordered attribute.

as_ordered : Set the Categorical to be ordered.

as_unordered : Set the Categorical to be unordered.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False
```

```
>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
True
```

For :class:`pandas.Categorical`:

```

>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.ordered
True

>>> cat = pd.Categorical(["a", "b"], ordered=False)
>>> cat.ordered
False

For :class:`pandas.CategoricalIndex`:

>>> ci = pd.CategoricalIndex(["a", "b"], ordered=True)
>>> ci.ordered
True

>>> ci = pd.CategoricalIndex(["a", "b"], ordered=False)
>>> ci.ordered
False

```

Data and other attributes defined here:

```
__annotations__ = {'_data': 'Categorical', '_values': 'Categorical', '...
```

Methods inherited from Index:

```

__abs__(self) -> Index from pandas.core.indexes.base.Index

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
    The array interface, return my values.

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index

__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
    Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
    Parameters
    -----
    memo, default None
        Standard signature. Unused

__getitem__(self, key) from pandas.core.indexes.base.Index
    Override numpy.ndarray's __getitem__ method to work as desired.

    This function adds lists and Series as valid boolean indexers
    (ndarrays only supports ndarray with dtype=bool).

    If resulting ndim != 1, plain ndarray is returned instead of
    corresponding `Index` subclass.

__iadd__(self, other) from pandas.core.indexes.base.Index

__invert__(self) -> Index from pandas.core.indexes.base.Index

__len__(self) -> int from pandas.core.indexes.base.Index
    Return the length of the Index.

__neg__(self) -> Index from pandas.core.indexes.base.Index

__pos__(self) -> Index from pandas.core.indexes.base.Index

__reduce__(self) from pandas.core.indexes.base.Index
    Helper for pickle.

__repr__(self) -> str_t from pandas.core.indexes.base.Index
    Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether all elements are Truthy.

```

Parameters

*args

Required for compatibility with numpy.

**kwargs

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

Index.any : Return whether any element in an Index is True.

Series.any : Return whether any element in a Series is True.

Series.all : Return whether all elements in a Series are True.

Notes

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

Examples

True, because nonzero integers are considered True.

```
>>> pd.Index([1, 2, 3]).all()
```

```
True
```

False, because ``0`` is considered False.

```
>>> pd.Index([0, 1, 2]).all()
```

```
False
```

```
any(self, *args, **kwargs) from pandas.core.indexes.base.Index
```

Return whether any element is Truthy.

Parameters

*args

Required for compatibility with numpy.

**kwargs

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

Index.all : Return whether all elements are True.

Series.all : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
```

```
>>> index.any()
```

```
True
```

```
>>> index = pd.Index([0, 0, 0])
```

```
>>> index.any()
```

```
False
```

```
append(self, other: Index | Sequence[Index]) -> Index from pandas.core.indexes.base.Index
```

Append a collection of Index options together.

Parameters

other : Index or list/tuple of indices

Single Index or a collection of indices, which can be either a list or a tuple.

Returns

Index

Returns a new Index object resulting from appending the provided other indices to the original Index.

See Also

Index.insert : Make new Index inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.append(pd.Index([4]))
Index([1, 2, 3, 4], dtype='int64')
```

argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base

Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations,
the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the maximum value.

See Also

Series.argmax : Return position of the maximum value.

Series.argmin : Return position of the minimum value.

numpy.ndarray.argmax : Equivalent method for numpy arrays.

Series.idxmax : Return index label of the maximum values.

Series.idxmin : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
the minimum cereal calories is the first element,
since index is zero-indexed.

argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base

Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations,
the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``

and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the minimum value.

See Also

Series.argmax : Return position of the minimum value.
Series.argmax : Return position of the maximum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
the minimum cereal calories is the first element,
since index is zero-indexed.

asof(self, label) from pandas.core.indexes.base.Index
Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it
is in the index, or return the previous index label if the passed one
is not in the index.

Parameters

label : object

The label up to which the method returns the latest index label.

Returns

object

The passed label if it is in the index. The previous label if the
passed label is not in the sorted index or `NaN` if there is no
such label.

See Also

Series.asof : Return the latest value in a Series up to the
passed index.
merge_asof : Perform an asof merge (similar to left join but it
matches on nearest key rather than equal key).
Index.get_loc : An `asof` is a thin wrapper around `get\_loc`
with method='pad'.

Examples

`Index.asof` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
>>> idx.asof("2014-01-01")
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label,
NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp]` from `pandas`
Return the locations (indices) of labels in the index.

As in the `:meth:`pandas.Index.asof``, if the label (a particular entry in `where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in `where``, -1 is returned.

`mask`` is used to ignore `NA`` values in the index during calculation.

Parameters

`where` : Index
An Index consisting of an array of timestamps.
`mask` : `np.ndarray[bool]`
Array of booleans denoting where values in the original data are not `NA``.

Returns

`np.ndarray[np.intp]`
An array of locations (indices) of the labels from the index which correspond to the return values of `:meth:`pandas.Index.asof`` for every element in `where``.

See Also

`Index.asof` : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use `mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by `dtype`. When conversion is impossible, a `TypeError` exception is raised.

Parameters

`dtype` : numpy dtype or pandas type
Note that any signed integer `dtype`` is treated as `'int64``, and any unsigned integer `dtype`` is treated as `'uint64``, regardless of the size.
`copy` : bool, default True
By default, `astype` always returns a newly allocated object. If `copy` is set to False and internal requirements on `dtype` are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index

Index with values cast to specified dtype.

See Also

Index.dtype: Return the dtype object of the underlying data.

Index.dtypes: Return the dtype object of the underlying data.

Index.convert_dtypes: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.astype("float")
```

```
Index([1.0, 2.0, 3.0], dtype='float64')
```

copy(self, name: Hashable | None = None, deep: bool = False) -> Self from pandas.core.indexes

Make a copy of this object.

Name is set on the new object.

Parameters

name : Label, optional

Set name for new object.

deep : bool, default False

If True attempts to make a deep copy of the Index.

Else makes a shallow copy.

Returns

Index

Index refer to new object which is a copy of this object.

See Also

Index.delete: Make new Index with passed location(-s) deleted.

Index.drop: Make new Index with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> new_idx = idx.copy()
```

```
>>> idx is new_idx
```

```
False
```

delete(self, loc: int | np.integer | list[int] | npt.NDArray[np.integer]) -> Self from pandas

Make new Index with passed location(-s) deleted.

Parameters

loc : int or list of int

Location of item(-s) which will be deleted.

Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

numpy.delete : Delete any rows and column from NumPy array (ndarray).

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> idx.delete(1)
```

```
Index(['a', 'c'], dtype='str')
```

```

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')

```

`diff(self, periods: int = 1) -> Index from pandas.core.indexes.base.Index`
Computes the difference between consecutive values in the Index object.

If periods is greater than 1, computes the difference between values that are `periods` number of positions apart.

Parameters

periods : int, optional
The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

Index
A new Index object with the computed differences.

Examples

```

>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')

```

`difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index`
Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters

other : Index or array-like
Index object or an array-like object containing elements to be compared with the elements of the original Index.

sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise `TypeError`).

Returns

Index
Returns a new Index object containing elements that are in the original Index but not in the `other` Index.

See Also

`Index.symmetric_difference` : Compute the symmetric difference of two Index objects.
`Index.intersection` : Form the intersection of two Index objects.

Examples

```

>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')

```

`drop(self, labels: Index | np.ndarray | Iterable[Hashable], errors: IgnoreRaise = 'raise')`
-> Index from pandas.core.indexes.base.Index
Make new Index with passed list of labels deleted.

Parameters

labels : array-like or scalar

Array-like object or a scalar value, representing the labels to be removed from the Index.

errors : {'ignore', 'raise'}, default 'raise'

If 'ignore', suppress error and existing labels are dropped.

Returns

Index

Will be same type as self, except for RangeIndex.

Raises

KeyError

If not all of the labels are found in the selected axis

See Also

Index.dropna : Return Index without NA/Nan values.

Index.drop_duplicates : Return Index with duplicate values removed.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> idx.drop(["a"])
```

```
Index(['b', 'c'], dtype='str')
```

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index
Return Index with duplicate values removed.

Parameters

keep : {'first', 'last', ``False``}, default 'first'

- 'first' : Drop duplicates except for the first occurrence.

- 'last' : Drop duplicates except for the last occurrence.

- ``False`` : Drop all duplicates.

Returns

Index

A new Index object with the duplicate values removed.

See Also

Series.drop_duplicates : Equivalent method on Series.

DataFrame.drop_duplicates : Equivalent method on DataFrame.

Index.duplicated : Related method on Index, indicating duplicate Index values.

Examples

Generate a pandas.Index with duplicate values.

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])
```

The `keep` parameter controls which duplicate values are removed. The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of keep is 'first'.

```
>>> idx.drop_duplicates(keep="first")
```

```
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')
```

The value 'last' keeps the last occurrence for each set of duplicated entries.

```
>>> idx.drop_duplicates(keep="last")
```

```
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')
```

The value ``False`` discards all sets of duplicated entries.

```
>>> idx.drop_duplicates(keep=False)
```

```
Index(['cow', 'beetle', 'hippo'], dtype='str')
```

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0
If a string is given, must be the name of a level
If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index after removing the requested level(s).

See Also

Index.dropna : Return Index without NA/NaN values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
```

```
MultiIndex([(1, 3, 5),
             (2, 4, 6)],
           names=['x', 'y', 'z'])
```

```
>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
           names=['y', 'z'])
```

```
>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
           names=['x', 'y'])
```

```
>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
           names=['x', 'y'])
```

```
>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')
```

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/NaN values.

Parameters

how : {'any', 'all'}, default 'any'
If the Index is a MultiIndex, drop the value when any or all levels are NaN.

Returns

Index

Returns an Index object after removing NA/NaN values.

See Also

Index.fillna : Fill NA/NaN values with the specified value.
Index.isna : Detect missing values.

Examples

```
>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')
```

duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting

array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

keep : {'first', 'last', False}, default 'first'
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

np.ndarray[bool]
A numpy array of boolean values indicating duplicate index values.

See Also

Series.duplicated : Equivalent method on pandas.Series.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Index.drop_duplicates : Remove duplicate values from Index.

Examples

By default, for each set of duplicated values, the first occurrence is set to False and all others to True:

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])
```

which is equivalent to

```
>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])
```

fillna(self, value) from pandas.core.indexes.base.Index
Fill NA/NAN values with the specified value.

Parameters

value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index
NA/NAN values replaced with `value`.

See Also

DataFrame.fillna : Fill NaN values of a DataFrame.
Series.fillna : Fill NaN Values of a Series.

Examples

```
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

get_indexer(
self,

```

target,
method: ReindexMethod | None = None,
limit: int | None = None,
tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to ndarray.take to align the
current data to the new index.

Parameters
-----
target : Index
    An iterable containing the values to be used for computing indexer.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
    * default: exact matches only.
    * pad / ffill: find the PREVIOUS index value if no exact match.
    * backfill / bfill: use NEXT index value if no exact match
    * nearest: use the NEAREST index value if no exact match. Tied
      distances are broken by preferring the larger index value.
limit : int, optional
    Maximum number of consecutive labels in ``target`` to match for
    inexact matches.
tolerance : optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

    Tolerance may be a scalar value, which applies the same tolerance
    to all values, or list-like, which applies variable tolerance per
    element. List-like includes list, tuple, array, Series, and must be
    the same size as the index and its dtype must exactly match the
    index's type.

Returns
-----
np.ndarray[np.intp]
    Integers from 0 to n - 1 indicating that the index at these
    positions matches the corresponding target values. Missing values
    in the target are marked by -1.

See Also
-----
Index.get_indexer_for : Returns an indexer even when non-unique.
Index.get_non_unique : Returns indexer and masks for new index given
    the current index.

Notes
-----
Returns -1 for unmatched values, for further explanation see the
example below.

Examples
-----
>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([ 1,  2, -1])

Notice that the return value is an array of locations in ``index``
and ``x`` is marked by -1, as it is not in ``index``.

get_indexer_for(self, target) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Guaranteed return of an indexer even when non-unique.

This dispatches to get_indexer or get_indexer_non_unique
as appropriate.

Parameters
-----
target : Index
    An iterable containing the values to be used for computing indexer.

Returns
-----
np.ndarray[np.intp]
    List of indices.

```

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.

`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Examples

```
>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])
```

`get_indexer_non_unique(self, target) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]` from `pandas.core.indexes.base`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index

An iterable containing the values to be used for computing indexer.

Returns

`indexer` : `np.ndarray[np.intp]`

Integers from 0 to `n - 1` indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

`missing` : `np.ndarray[np.intp]`

An indexer into the target of the values not found. These correspond to the -1 in the indexer array.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.

`Index.get_indexer_for` : Returns an indexer even when non-unique.

Examples

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned `indexer` contains only integers equal to -1. It demonstrates that there's no match between the index and the `target` values at these positions. The mask `[0, 1, 2]` in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in `index`. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

`get_level_values = _get_level_values(self, level) -> Index`

`get_loc(self, key)` from `pandas.core.indexes.base.Index`

Get integer location, slice or boolean mask for requested label.

Parameters

`key` : label

The key to check its location if it is present in the index.

Returns

int if unique index, slice if monotonic index, else mask
Integer location, slice or boolean mask.

See Also

Index.get_slice_bound : Calculate slice bound that corresponds to
given label.
Index.get_indexer : Computes indexer and mask for new index given
the current index.
Index.get_non_unique : Returns indexer and masks for new index given
the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

>>> unique_index = pd.Index(list("abc"))
>>> unique_index.get_loc("b")
1

>>> monotonic_index = pd.Index(list("abbc"))
>>> monotonic_index.get_loc("b")
slice(1, 3, None)

>>> non_monotonic_index = pd.Index(list("abcb"))
>>> non_monotonic_index.get_loc("b")
array([False, True, False, True])

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position
of given label.

Parameters

label : object
The label for which to calculate the slice bound.
side : {'left', 'right'}
if 'left' return leftmost position of given label.
if 'right' return one-past-the-rightmost position of given label.

Returns

int
Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested
label.

Examples

>>> idx = pd.RangeIndex(5)
>>> idx.get_slice_bound(3, "left")
3

>>> idx.get_slice_bound(3, "right")
4

If ``label`` is non-unique in the index, an error will be raised.

```
>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
  KeyError: Cannot get left slice bound for non-unique label: 'a'
```

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
Group the index labels by a given array of values.

Parameters

values : array
Values used to determine the groups.

Returns

```

dict
    {group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.

Parameters
-----
other : Index
    The Index object you want to compare with the current Index object.

Returns
-----
bool
    If two Index objects have equal elements and same type True,
    otherwise False.

See Also
-----
Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
    If we have an object dtype, try to infer a non-object dtype.

Parameters
-----
copy : bool, default True
    Whether to make a copy in cases where no inference occurs.

Returns
-----
Index
    An Index with a new dtype if the dtype was inferred
    or a shallow copy if the dtype could not be inferred.

See Also
-----
Index.inferred_type: Return a string of the type inferred from the values.

Examples
-----
>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')

insert(self, loc: int, item) -> Index from pandas.core.indexes.base.Index
    Make new Index inserting new item at location.

    Follows Python numpy.insert semantics for negative values.

Parameters
-----
loc : int
    The integer location where the new item will be inserted.
item : object
    The new item to be inserted into the Index.

Returns
-----
Index
    Returns a new Index object resulting from inserting the specified item at
    the specified location within the original Index.

```

See Also

Index.append : Append a collection of Indexes together.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

intersection(self, other, sort: bool = False) from pandas.core.indexes.base.Index
Form the intersection of two Index objects.

This returns a new Index with elements common to the index and `other`.

Parameters

other : Index or array-like

An Index or an array-like object containing elements to form the intersection with the original Index.

sort : True, False or None, default False

Whether to sort the resulting index.

* None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.

* False : do not sort the result.

* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.intersection(idx2)
Index([3, 4], dtype='int64')
```

is_(self, other) -> bool from pandas.core.indexes.base.Index

More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks that metadata is also the same.

Parameters

other : object

Other object to compare against.

Returns

bool

True if both have same underlying data, False otherwise.

See Also

Index.identical : Works like ``Index.is\_`` but also checks metadata.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx1.is_(idx1.view())
True
```

```
>>> idx1.is_(idx1.copy())
False
```

```
isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
    Return a boolean array where the index values are in `values`.
```

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like
 Sought values.
level : str or int, optional
 Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

np.ndarray[bool]
 NumPy array of boolean values.

See Also

Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a ``ValueError``.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])  
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(  
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]  
... )  
>>> midx  
MultiIndex([(1, 'red'),  
            (2, 'blue'),  
            (3, 'green')],  
            names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")  
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])  
array([ True, False, False])
```

```
isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index  
    Detect missing values.
```

Return a boolean same-sized object indicating if the values are NA. NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get mapped to ``True`` values.

Everything else get mapped to ``False`` values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

numpy.ndarray[bool]

A boolean array of whether my values are NA.

See Also

Index.notna : Boolean inverse of isna.

Index.dropna : Omit entries with missing values.

isna : Top-level isna.

Series.isna : Detect missing values in Series object.

Examples

Show which entries in a pandas.Index are NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

isnull = isna(self) -> npt.NDArray[np.bool_]

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index

The other index on which join is performed.

how : {'left', 'right', 'inner', 'outer'}

level : int or level name, default None

It is either the integer position or the name of the level.

return_indexers : bool, default False

Whether to return the indexers or not for both the index objects.

sort : bool, default False

Sort the join keys lexicographically in the result Index. If False, the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)

The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index

or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a database-style join.

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.base.Index
Memory usage of the values.

Parameters

deep : bool, default False
Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption.

Returns

bytes used
Returns memory usage of the values in the Index in bytes.

See Also

numpy.ndarray.nbytes : Total bytes consumed by the elements of the array.

Notes

Memory usage does not include memory consumed by elements that are not components of the array if deep=False or if used on PyPy

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24
```

notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to ``False`` values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
-----
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```

>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])

notnull = notna(self) -> npt.NDArray[np.bool_]

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters
-----
mask : array-like of bool
    Array of booleans denoting where values should be replaced.
value : scalar
    Scalar value to use to fill holes (e.g. 0).
    This value cannot be a list-like.

Returns
-----
Index
    A new Index of the values set with the mask.

See Also
-----
numpy.putmask : Changes elements of an array
    based on conditional and input values.

Examples
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters
-----
order : {'K', 'A', 'C', 'F'}, default 'C'
    Specify the memory layout of the view. This parameter is not
    implemented currently.

Returns
-----
Index
    A view on self.

See Also
-----
numpy.ndarray.ravel : Return a flattened array.

Examples
-----
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')

rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
Alter Index or MultiIndex name.

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters
-----
name : Hashable or a sequence of the previous
    Name(s) to set.
inplace : bool, default False
    Modifies the object directly, instead of creating a new Index or
    MultiIndex.

Returns
-----

```

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.set_names : Able to set new names partially and by level.

Examples

```
-----
>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")
Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

>>> idx = pd.MultiIndex.from_product(
...     [ ["python", "cobra"], [2018, 2019] ], names=["kind", "year"]
... )
>>> idx
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.
```

repeat(self, repeats, axis: None = None) -> Self from pandas.core.indexes.base.Index
Repeat elements of an Index.

Returns a new Index where each element of the current Index is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints

The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty Index.

axis : None

Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

Index

Newly created Index with repeated elements.

See Also

Series.repeat : Equivalent function for Series.

numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')
```

round(self, decimals: int = 0) -> Self from pandas.core.indexes.base.Index
Round each value in the Index to the given number of decimals.

Parameters

decimals : int, optional

Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

Returns

Index

A new Index with the rounded values.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10.1234, 20.5678, 30.9123, 40.4567, 50.7890])
>>> idx.round(decimals=2)
Index([10.12, 20.57, 30.91, 40.46, 50.79], dtype='float64')
```

set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core.
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

names : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

level : int, Hashable or a sequence of the previous, optional
If the index is a MultiIndex and names is not dict-like, level(s) to set
(None for all levels). Otherwise level must be None.

inplace : bool, default False
Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            )
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['species', 'year'])
```

When renaming levels with a dict, levels can not be passed.

```
>>> idx.set_names({"kind": "snake"})
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['snake', 'year'])
```

shift(self, periods: int = 1, freq=None) -> Self from pandas.core.indexes.base.Index
Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes
by a specified time increment a given number of times.

Parameters

periods : int, default 1
 Number of periods (or increments) to shift by,
 can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or str, optional
 Frequency increment to shift by.
 If None, the index is shifted by its own `freq` attribute.
 Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

pandas.Index
 Shifted index.

See Also

Series.shift : Shift values of Series.

Notes

This method is only implemented for datetime-like index classes,
i.e., DatetimeIndex, PeriodIndex and TimedeltaIndex.

Examples

Put the first 5 month starts of 2011 into an index.

```
>>> month_starts = pd.date_range("1/1/2011", periods=5, freq="MS")
>>> month_starts
DatetimeIndex(['2011-01-01', '2011-02-01', '2011-03-01', '2011-04-01',
               '2011-05-01'],
              dtype='datetime64[us]', freq='MS')
```

Shift the index by 10 days.

```
>>> month_starts.shift(10, freq="D")
DatetimeIndex(['2011-01-11', '2011-02-11', '2011-03-11', '2011-04-11',
               '2011-05-11'],
              dtype='datetime64[us]', freq=None)
```

The default value of `freq` is the `freq` attribute of the index,
which is 'MS' (month start) in this example.

```
>>> month_starts.shift(10)
DatetimeIndex(['2011-11-01', '2011-12-01', '2012-01-01', '2012-02-01',
               '2012-03-01'],
              dtype='datetime64[us]', freq='MS')
```

```
slice_indexer(
    self,
    start: Hashable | None = None,
    end: Hashable | None = None,
    step: int | None = None
) -> slice from pandas.core.indexes.base.Index
Compute the slice indexer for input labels and step.
```

Index needs to be ordered and unique.

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, default None
 If None, defaults to 1.

Returns

slice
 A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is
 not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)
```

```
slice_locs(
    self,
    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.
```

Parameters

start : label, default None
If None, defaults to the beginning.
end : label, default None
If None, defaults to the end.
step : int, defaults None
If None, defaults to 1.

Returns

tuple[int, int]
Returns a tuple of two integers representing the slice locations for the input labels within the index.

See Also

Index.get_loc : Get location for a single label.

Notes

This method only works if the index is monotonic or unique.

Examples

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)

>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)
```

```
sort_values(
    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

```

-----
return_indexer : bool, default False
    Should the indices that would sort the index be returned.
ascending : bool, default True
    Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.
key : callable, optional
    If not None, apply the key function to the index values
    before sorting. This is similar to the `key` argument in the
    builtin :meth:`sorted` function, with the notable difference that
    this `key` function should be *vectorized*. It should expect an
    ``Index`` and return an ``Index`` of the same shape.

Returns
-----
sorted_index : pandas.Index
    Sorted copy of the index.
indexer : numpy.ndarray, optional
    The indices that the index itself was sorted by.

See Also
-----
Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples
-----
>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')

Sort values in ascending order (default behavior).

>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')

Sort values in descending order, and also get the indices `idx` was
sorted by.

>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))

sortlevel(
    self,
    level=None,
    ascending: bool | list[bool] = True,
    sort_remaining=None,
    na_position: NaPosition = 'first'
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
    For internal compatibility with the Index API.

Sort the Index. This is for compat with MultiIndex

Parameters
-----
ascending : bool, default True
    False to sort in descending order
na_position : {'first' or 'last'}, default 'first'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.

    .. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns
-----
Index

symmetric_difference(
    self,
    other,
    result_name: abc.Hashable | None = None,
    sort: bool | None = None
) from pandas.core.indexes.base.Index

```

Compute the symmetric difference of two Index objects.

Parameters

other : Index or array-like
Index or an array-like object with elements to compute the symmetric difference with the original Index.
result_name : str
A string representing the name of the resulting Index, if desired.
sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any TypeError from incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any TypeErrors from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index
Returns a new Index object containing elements that appear in either the original Index or the `other` Index, but not both.

See Also

Index.difference : Return a new Index with elements of index not in other.
Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes

`symmetric\_difference` contains elements that appear in either `idx1` or `idx2` but not both. Equivalent to the Index created by `idx1.difference(idx2) | idx2.difference(idx1)` with duplicates dropped.

Examples

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')

```
take(  
    self,  
    indices,  
    axis: Axis = 0,  
    allow_fill: bool = True,  
    fill_value=None,  
    **kwargs  
) -> Self from pandas.core.indexes.base.Index
```

Return a new Index of the values selected by the indices.

For internal compatibility with numpy arrays.

Parameters

indices : array-like
Indices to be taken.
axis : {0 or 'index'}, optional
The axis over which to select values, always 0 or 'index'.
allow_fill : bool, default True
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in `indices` indicate missing values. These values are set to `fill\_value`. Any other negative values raise a `ValueError`.

fill_value : scalar, default None
If allow_fill=True and fill_value is not None, indices specified by -1 are regarded as NA. If Index doesn't hold NA, raise ValueError.

```

**kwargs
    Required for compatibility with numpy.

Returns
-----
Index
    An index formed of elements at the given indices. Will be the same
    type as self, except for RangeIndex.

See Also
-----
numpy.ndarray.take: Return an array formed from the
    elements of a at the given indices.

Examples
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.take([2, 2, 1, 2])
Index(['c', 'c', 'b', 'c'], dtype='str')

to_flat_index(self) -> Self from pandas.core.indexes.base.Index
    Identity method.

    This is implemented for compatibility with subclass implementations
    when chaining.

Returns
-----
pd.Index
    Caller.

See Also
-----
MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.core
    Create a DataFrame with a column containing the Index.

Parameters
-----
index : bool, default True
    Set the index of the returned DataFrame as the original Index.

name : object, defaults to index.name
    The passed name should substitute for the index name (if it has
    one).

Returns
-----
DataFrame
    DataFrame containing the original Index data.

See Also
-----
Index.to_series : Convert an Index to a Series.
Series.to_frame : Convert Series to DataFrame.

Examples
-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
      animal
Ant      Ant
Bear    Bear
Cow      Cow

By default, the original Index is reused. To enforce a new Index:

>>> idx.to_frame(index=False)
      animal
0      Ant
1     Bear
2      Cow

To override the name of the resulting column, specify `name`:

```

```
>>> idx.to_frame(index=False, name="zoo")
      zoo
0   Ant
1  Bear
2   Cow
```

`to_series(self, index=None, name: Hashable | None = None) -> Series` from `pandas.core.indexes`
 Create a Series with both index and values equal to the index keys.

Useful with `map` for returning an indexer based on an index.

Parameters

`index` : Index, optional
 Index of resulting Series. If None, defaults to original index.
`name` : str, optional
 Name of resulting Series. If None, defaults to name of original index.

Returns

Series

The dtype will be based on the type of the Index values.

See Also

`Index.to_frame` : Convert an Index to a DataFrame.
`Series.to_frame` : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ```index```:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1     Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ```name```:

```
>>> idx.to_series(name="zoo")
zoo
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
 Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
 Index or an array-like object containing elements to form the union with the original Index.
`sort` : bool or None, default None
 Whether to sort the resulting Index.

* None : Sort the result, except when

1. ```self``` and ```other``` are equal.
2. ```self``` or ```other``` has length 0.

3. Some values in `self` or `other` cannot be compared.
A RuntimeWarning is issued in this case.

* False : do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the `other` Index.

See Also

Index.unique : Return unique values in the index.
Index.intersection : Form the intersection of two Index objects.
Index.difference : Return a new Index with elements of index not in `other`.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')
```

Union mismatched dtypes

```
>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')
```

MultiIndex case

```
>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )
```

unique(self, level: Hashable | None = None) -> Self from pandas.core.indexes.base.Index
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

level : int or hashable, optional

Only return values from specified level (for MultiIndex).

If int, gets the level by integer position, else by level name.

Returns

Index

Unique values in the index.

See Also

unique : Numpy array of unique values in that column.

Series.unique : Return unique values of Series object.

Examples

```
>>> idx = pd.Index([1, 1, 2, 3, 3])
```

```
>>> idx.unique()
```

```
Index([1, 2, 3], dtype='int64')
```

view(self, cls=None) from pandas.core.indexes.base.Index

Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the `cls` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

cls : data-type or ndarray sub-class, optional

Data-type descriptor of the returned view, e.g., float32 or int16.

Omitting it results in the view having the same data-type as `self`.

This argument can also be specified as an ndarray sub-class, e.g., np.int64 or np.float32 which then specifies the type of the returned object.

Returns

Index or ndarray

A view of the Index. If `cls` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric `cls` is specified an ndarray view with the new dtype is returned.

Raises

ValueError

If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

Index.copy : Returns a copy of the Index.

numpy.ndarray.view : Returns a new view of array with the same data.

Examples

```
>>> idx = pd.Index([-1, 0, 1])
```

```
>>> idx.view()
```

```
Index([-1, 0, 1], dtype='int64')
```

```
>>> idx.view(np.uint64)
```

```
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
```

```
array([ nan,   nan, 0.e+00, 0.e+00, 1.e-45, 0.e+00], dtype=float32)
```

where(self, cond, other=None) -> Index from pandas.core.indexes.base.Index

Replace values where the condition is False.

The replacement is taken from other.

Parameters

cond : bool array-like with the same length as self
Condition to select the values on.
other : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index
A copy of self with values replaced from other
where the condition is False.

See Also

Series.where : Same method for Series.
DataFrame.where : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

has_duplicates

Check if the Index has duplicate values.

Returns

bool
Whether or not the Index has duplicate values.

See Also

Index.is_unique : Inverse method that checks if it has unique values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True

>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False
```

is_monotonic_decreasing

Return a boolean if the values are equal or decreasing.

Returns

bool

See Also

Index.is_monotonic_increasing : Check if the values are equal or increasing.

Examples

```

-----
>>> pd.Index([3, 2, 1]).is_monotonic_decreasing
True
>>> pd.Index([3, 2, 2]).is_monotonic_decreasing
True
>>> pd.Index([3, 1, 2]).is_monotonic_decreasing
False

is_monotonic_increasing
    Return a boolean if the values are equal or increasing.

    Returns
    -----
    bool

    See Also
    -----
    Index.is_monotonic_decreasing : Check if the values are equal or decreasing.

    Examples
    -----
    >>> pd.Index([1, 2, 3]).is_monotonic_increasing
    True
    >>> pd.Index([1, 2, 2]).is_monotonic_increasing
    True
    >>> pd.Index([1, 3, 2]).is_monotonic_increasing
    False

nlevels
    Number of levels.

shape
    Return a tuple of the shape of the underlying data.

    See Also
    -----
    Index.size: Return the number of elements in the underlying data.
    Index.ndim: Number of dimensions of the underlying data, by definition 1.
    Index.dtype: Return the dtype object of the underlying data.
    Index.values: Return an array representing the data in the Index.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')
    >>> idx.shape
    (3,)

values
    Return an array representing the data in the Index.

    .. warning::

        We recommend using :attr:`Index.array` or
        :meth:`Index.to_numpy`, depending on whether you need
        a reference to the underlying data or a NumPy array.

    .. versionchanged:: 3.0.0

        The returned array is read-only.

    Returns
    -----
    array: numpy.ndarray or ExtensionArray

    See Also
    -----
    Index.array : Reference to the underlying data.
    Index.to_numpy : A NumPy array representing the underlying data.

    Examples
    -----
    For :class:`pandas.Index`:

    >>> idx = pd.Index([1, 2, 3])
    >>> idx

```

```
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[[0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors inherited from Index:

array

The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.

Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
```

```
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

dtype

Return the dtype object of the underlying data.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.dtype
dtype('int64')
```

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

bool

See Also

Index.isna : Detect missing values.
Index.dropna : Return Index without NA/NaN values.
Index.fillna : Fill NA/NaN values with the specified value.

Examples

```
-----
>>> s = pd.Series([1, 2, 3], index=["a", "b", None])
>>> s
a    1
b    2
None 3
dtype: int64
>>> s.index.hasnans
True
```

is_unique

Return if the index has unique values.

Returns

bool

See Also

Index.has_duplicates : Inverse method that checks if it has duplicate values.

Examples

```
-----
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.is_unique
False

>>> idx = pd.Index([1, 5, 7])
>>> idx.is_unique
True

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.is_unique
False

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.is_unique
True
```

name

Return Index or MultiIndex name.

Returns

label (hashable object)
 The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.
Index.rename: Able to set new names partially and by level.
Series.name: Corresponding Series property.

Examples

>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx
Index([1, 2, 3], dtype='int64', name='x')
>>> idx.name
'x'

names

Get names on index.

This method returns a FrozenList containing the names of the object.
It's primarily intended for internal use.

Returns

FrozenList

 A FrozenList containing the object's names, contains None if the object
 does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx.names
FrozenList(['x'])

>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
>>> idx.names
FrozenList(['x', 'y'])

If the index does not have a name set:

>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])

Data and other attributes inherited from Index:

__pandas_priority__ = 2000

str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method.
Patterned after Python's string methods, with some inspiration from
R's stringr package.

Parameters

data : Series or Index
 The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.
Index.str : Vectorized string functions for Index.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str

>>> s.str.split("_")
0    [A, Str, Series]
dtype: object

>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

`__iter__(self)` -> Iterator
Return an iterator of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

iterator

An iterator yielding scalar values from the Series.

See Also

Series.items : Lazily iterate over (index, value) tuples.

Examples

```
-----
>>> s = pd.Series([1, 2, 3])
>>> for x in s:
...     print(x)
1
2
3
```

`factorize(self, sort: bool = False, use_na_sentinel: bool = True)` -> tuple[npt.NDArray[np.in], npt.NDArray[np.in]]
Encode the object as an enumerated type or categorical variable.

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function :func:`pandas.factorize`, and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

Parameters

`sort` : bool, default False
Sort `uniques` and shuffle `codes` to maintain the relationship.
`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray

An integer ndarray that's an indexer into `uniques`.
`uniques.take(codes)` will have the same values as `values`.

`uniques` : ndarray, Index, or Categorical

The unique valid values. When `values` is Categorical, `uniques` is a Categorical. When `values` is some other pandas object, an `Index` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in `values`, `uniques` will *not* contain an entry for it.

See Also

cut : Discretize continuous-valued array.
unique : Find the unique value in an array.

Notes

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

These examples all show factorize as a top-level method like `pd.factorize(values)`. The results are identical for methods like `meth:Series.factorize`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When `use_na_sentinel=True` (the default), missing values are indicated in the `codes` with the sentinel value `-1` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that `'b'` is in `uniques.categories`, despite not being present in `cat.values`.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting `use_na_sentinel=False`.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
```

```

array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])

```

item(self)
Return the first element of the underlying data as a Python scalar.

Returns

scalar
The first element of Series or Index.

Raises

ValueError
If the data is not length = 1.

See Also

Index.values : Returns an array representing the data in the Index.
Series.head : Returns the first `n` rows.

Examples

>>> s = pd.Series([1])
>>> s.item()
1

For an index:

```

>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'

```

nunique(self, dropna: bool = True) -> int
Return number of unique elements in the object.

Excludes NA values by default.

Parameters

dropna : bool, default True
Don't include NaN in the count.

Returns

int
An integer indicating the number of unique elements in the object.

See Also

DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.

Examples

>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0 1
1 3
2 5
3 7
4 7
dtype: int64

>>> s.nunique()
4

to_list = tolist(self) -> list
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar

(for Timestamp/Timedelta/Interval/Period)

Returns

list

List containing the values as Python or pandas scalars.

See Also

numpy.ndarray.tolist : Return the array as an a.ndim-levels deep nested list of Python scalars.

Examples

For Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

>>> idx.to_list()
[1, 2, 3]
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>,
    **kwargs
) -> np.ndarray
A NumPy ndarray representing the values in this Series or Index.
```

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

**kwargs

Additional keywords passed through to the ``to\_numpy`` method of the underlying array (for extension arrays).

Returns

numpy.ndarray

The NumPy ndarray holding the values from this Series or Index. The dtype of the array may differ. See Notes.

See Also

Series.array : Get the actual data stored within.

Index.array : Get the actual data stored within.

DataFrame.to_numpy : Similar method for DataFrame.

Notes

The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result

in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` *may* require copying data and coercing the result to a NumPy type (possibly object), which may be expensive. When you need a no-copy reference to the underlying data, :attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of ``to\_numpy()`` for various dtypes within pandas.

dtype	array type
category[T]	ndarray[T] (same dtype as input)
period	ndarray[object] (Periods)
interval	ndarray[object] (Intervals)
IntegerNA	ndarray[object]
datetime64[ns]	datetime64[ns]
datetime64[ns, tz]	ndarray[object] (Timestamps)

Examples

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)
```

Specify the ``dtype`` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

```
>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

transpose(self, *args, **kwargs) -> Self
Return the transpose, which is by definition self.

Returns

%(klass)s

value_counts(
 self,
 normalize: bool = False,
 sort: bool = True,
 ascending: bool = False,
 bins=None,
 dropna: bool = True
) -> Series
Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

normalize : bool, default False
If True then the object returned will contain the relative frequencies of the unique values.
sort : bool, default True
Stable sort by frequencies when True. Preserve the order of the data when False.

```

.. versionchanged:: 3.0.0

    Prior to 3.0.0, the sort was unstable.
ascending : bool, default False
    Sort in ascending order.
bins : int, optional
    Rather than count values, group them into half-open bins,
    a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
    Don't include counts of NaN.

Returns
-----
Series
    Series containing counts of unique values.

See Also
-----
Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples
-----
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by
dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64

**bins**

Bins can be useful for going from a continuous variable to a
categorical variable; instead of counting unique
apparitions of values, divide the index in the specified
number of half-open bins.

>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]      2
(3.0, 4.0]      1
Name: count, dtype: int64

**dropna**

With `dropna` set to `False` we can also see NaN index values.

>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64

**Categorical Dtypes**

Rows with categorical type will be counted as one group
if they have same categories and order.
In the example below, even though ``a``, ``c``, and ``d``
all have the same data types of ``category``,
only ``c`` and ``d`` will be counted as one group
since ``a`` doesn't have the same categories.

```

```

>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str          1
Name: count, dtype: int64

```

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```

>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str

```

For Index:

```

>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')

```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False

>>> idx_empty = pd.Index([])

```

```
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.
Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.

Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2       Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose      3.1    801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN      NaN    NaN     NaN
moose  NaN      NaN    NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"])
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN      NaN
elk        1.7      NaN
moose      3.0      NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
```

```

    __radd__(self, other)
    __rand__(self, other)
    __rdivmod__(self, other)
    __rfloordiv__(self, other)
    __rmod__(self, other)
    __rmul__(self, other)
    __ror__(self, other)
        Return value|self.
    __rpow__(self, other)
    __rsub__(self, other)
    __rtruediv__(self, other)
    __rxor__(self, other)
    __sub__(self, other)
    __truediv__(self, other)
    __xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
    __dict__
        dictionary for instance variables
    __weakref__
        list of weak references to the object

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
    __hash__ = None

-----
Methods inherited from pandas.core.base.PandasObject:
    __sizeof__(self) -> int
        Generates the total memory usage for an object that returns
        either a value or Series of values

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
    __dir__(self) -> list[str]
        Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.

```

class DataFrame(pandas.core.generic.NDFrame, pandas.core.arraylike.OpsMixin)

```

    DataFrame(
        data=None,
        index: Axes | None = None,
        columns: Axes | None = None,
        dtype: Dtype | None = None,
        copy: bool | None = None
    ) -> None

```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Parameters

data : ndarray (structured or homogeneous), Iterable, dict, or DataFrame
Dict can contain Series, arrays, constants, dataclass or list-like objects. If data is a dict, column order follows insertion-order. If a dict contains Series which have an index defined, it is aligned by its index. This alignment also occurs if data is a Series or a DataFrame itself. Alignment is done on Series/DataFrame inputs.

If data is a list of dicts, column order follows insertion-order.

index : Index or array-like
Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.
columns : Index or array-like
Column labels to use for resulting frame when data does not have them, defaulting to RangeIndex(0, 1, 2, ..., n). If data contains column labels, will perform column selection instead.
dtype : dtype, default None
Data type to force. Only a single dtype is allowed. If None, infer. If ``data`` is DataFrame then is ignored.
copy : bool or None, default None
Copy data from inputs.
For dict data, the default of None behaves like ``copy=True``. For DataFrame or 2d ndarray input, the default of None behaves like ``copy=False``.
If data is a dict containing one or more Series (possibly of different dtypes), ``copy=False`` will ensure that these inputs are not copied.

See Also

DataFrame.from_records : Constructor from tuples, also record arrays.
DataFrame.from_dict : From dicts of Series, arrays, or dicts.
read_csv : Read a comma-separated values (csv) file into DataFrame.
read_table : Read general delimited file into DataFrame.
read_clipboard : Read text from clipboard into DataFrame.

Notes

Please reference the :ref:`User Guide <basics.dataframe>` for more information.

Examples

Constructing DataFrame from a dictionary.

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df
   col1  col2
0     1     3
1     2     4
```

Notice that the inferred dtype is int64.

```
>>> df.dtypes
col1    int64
col2    int64
dtype: object
```

To enforce a single dtype:

```
>>> df = pd.DataFrame(data=d, dtype=np.int8)
>>> df.dtypes
col1    int8
col2    int8
dtype: object
```

Constructing DataFrame from a dictionary including Series:

```
>>> d = {"col1": [0, 1, 2, 3], "col2": pd.Series([2, 3], index=[2, 3])}
>>> pd.DataFrame(data=d, index=[0, 1, 2, 3])
   col1  col2
0     0   NaN
1     1   NaN
2     2   2.0
3     3   3.0
```

Constructing DataFrame from numpy ndarray:

```
>>> df2 = pd.DataFrame(
...     np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), columns=["a", "b", "c"]
... )
>>> df2
   a  b  c
0  1  2  3
1  4  5  6
2  7  8  9
```

Constructing DataFrame from a numpy ndarray that has labeled columns:

```
>>> data = np.array(
...     [(1, 2, 3), (4, 5, 6), (7, 8, 9)],
...     dtype=[("a", "i4"), ("b", "i4"), ("c", "i4")],
... )
>>> df3 = pd.DataFrame(data, columns=["c", "a"])
>>> df3
   c  a
0  3  1
1  6  4
2  9  7
```

Constructing DataFrame from dataclass:

```
>>> from dataclasses import make_dataclass
>>> Point = make_dataclass("Point", [("x", int), ("y", int)])
>>> pd.DataFrame([Point(0, 0), Point(0, 3), Point(2, 3)])
   x  y
0  0  0
1  0  3
2  2  3
```

Constructing DataFrame from Series/DataFrame:

```
>>> ser = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> df = pd.DataFrame(data=ser, index=["a", "c"])
>>> df
   0
a  1
c  3

>>> df1 = pd.DataFrame([1, 2, 3], index=["a", "b", "c"], columns=["x"])
>>> df2 = pd.DataFrame(data=df1, index=["a", "c"])
>>> df2
   x
a  1
c  3
```

Method resolution order:

```
DataFrame
pandas.core.generic.NDFrame
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
pandas.core.indexing.IndexingMixin
pandas.core.arraylike.OpsMixin
builtins.object
```

Methods defined here:

```
__arrow_c_stream__(self, requested_schema=None) from pandas.core.frame.DataFrame
Export the pandas DataFrame as an Arrow C stream PyCapsule.
```

This relies on pyarrow to convert the pandas DataFrame to the Arrow format (and follows the default behaviour of ``pyarrow.Table.from\_pandas`` in its handling of the index, i.e. store the index as a column except for RangeIndex).
This conversion is not necessarily zero-copy.

Parameters

requested_schema : PyCapsule, default None

The schema to which the dataframe should be casted, passed as a PyCapsule containing a C ArrowSchema representation of the requested schema.

Returns

PyCapsule

```
__dataframe__(self, nan_as_null: bool = False, allow_copy: bool = True) -> DataFrameXchg from pandas.core.frame.DataFrame
Return the dataframe interchange object implementing the interchange protocol.
```

```
.. deprecated:: 3.0.0
```

The DataFrame Interchange Protocol is deprecated.
For dataframe-agnostic code, you may want to look into:

- ``Arrow PyCapsule Interface` <<https://arrow.apache.org/docs/format/CDataInterface/PyCapsuleInterface.html>>
- ``Narwhals` <<https://github.com/narwhals-dev/narwhals>>

```
.. note::
```

For new development, we highly recommend using the Arrow C Data Interface alongside the Arrow PyCapsule Interface instead of the interchange protocol

```
.. warning::
```

Due to severe implementation issues, we recommend only considering using the interchange protocol in the following cases:

- converting to pandas: for pandas >= 2.0.3
- converting from pandas: for pandas >= 3.0.0

Parameters

```
nan_as_null : bool, default False
    ``nan_as_null`` is DEPRECATED and has no effect. Please avoid using
    it; it will be removed in a future release.
allow_copy : bool, default True
    Whether to allow memory copying when exporting. If set to False
    it would cause non-zero-copy exports to fail.
```

Returns

```
DataFrame interchange object
    The object which consuming library can use to ingress the dataframe.
```

See Also

```
DataFrame.from_records : Constructor from tuples, also record arrays.
DataFrame.from_dict : From dicts of Series, arrays, or dicts.
```

Notes

```
Details on the interchange protocol:
https://data-apis.org/dataframe-protocol/latest/index.html
```

Examples

```
>>> df_not_necessarily_pandas = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> interchange_object = df_not_necessarily_pandas.__dataframe__()
>>> interchange_object.column_names()
Index(['A', 'B'], dtype='str')
>>> df_pandas = pd.api.interchange.from_dataframe(
...     interchange_object.select_columns_by_name(["A"])
... )
>>> df_pandas
   A
0  1
1  2
```

These methods (```column_names```, ```select_columns_by_name```) should work for any dataframe library which implements the interchange protocol.

```
__divmod__(self, other) -> tuple[DataFrame, DataFrame] from pandas.core.frame.DataFrame
```

```
__getitem__(self, key) from pandas.core.frame.DataFrame
```

```
__init__(
    self,
    data=None,
    index: Axes | None = None,
    columns: Axes | None = None,
    dtype: Dtype | None = None,
```

```

    copy: bool | None = None
) -> None from pandas.core.frame.DataFrame
    Initialize self. See help(type(self)) for accurate signature.

__len__(self) -> int from pandas.core.frame.DataFrame
    Returns length of info axis, but here we use the index.

__matmul__(self, other: AnyArrayLike | DataFrame) -> DataFrame | Series from pandas.core.frame.DataFrame
    Matrix multiplication using binary `@` operator.

__rdivmod__(self, other) -> tuple[DataFrame, DataFrame] from pandas.core.frame.DataFrame

__repr__(self) -> str from pandas.core.frame.DataFrame
    Return a string representation for a particular DataFrame.

__rmatmul__(self, other) -> DataFrame from pandas.core.frame.DataFrame
    Matrix multiplication using binary `@` operator.

__setitem__(self, key, value) -> None from pandas.core.frame.DataFrame
    Set item(s) in DataFrame by key.

    This method allows you to set the values of one or more columns in the
    DataFrame using a key. If the key does not exist, a new
    column will be created.

    Parameters
    -----
    key : The object(s) in the index which are to be assigned to
          Column label(s) to set. Can be a single column name, list of column names,
          or tuple for MultiIndex columns.
    value : scalar, array-like, Series, or DataFrame
          Value(s) to set for the specified key(s).

    Returns
    -----
    None
    This method does not return a value.

    See Also
    -----
    DataFrame.loc : Access and set values by label-based indexing.
    DataFrame.iloc : Access and set values by position-based indexing.
    DataFrame.assign : Assign new columns to a DataFrame.

    Notes
    -----
    When assigning a Series to a DataFrame column, pandas aligns the Series
    by index labels, not by position. This means:

    * Values from the Series are matched to DataFrame rows by index label
    * If a Series index label doesn't exist in the DataFrame index, it's ignored
    * If a DataFrame index label doesn't exist in the Series index, NaN is assigned
    * The order of values in the Series doesn't matter; only the index labels matter

    Examples
    -----
    Basic column assignment:

    >>> df = pd.DataFrame({"A": [1, 2, 3]})
    >>> df["B"] = [4, 5, 6] # Assigns by position
    >>> df
       A  B
    0  1  4
    1  2  5
    2  3  6

    Series assignment with index alignment:

    >>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
    >>> s = pd.Series([10, 20], index=[1, 3]) # Note: index 3 doesn't exist in df
    >>> df["B"] = s # Assigns by index label, not position
    >>> df
       A  B
    0  1  NaN
    1  2  10.0
    2  3  NaN

```

Series assignment with partial index match:

```
>>> df = pd.DataFrame({"A": [1, 2, 3, 4]}, index=["a", "b", "c", "d"])
>>> s = pd.Series([100, 200], index=["b", "d"])
>>> df["B"] = s
>>> df
   A      B
a  1   NaN
b  2  100.0
c  3   NaN
d  4  200.0
```

Series index labels NOT in DataFrame, ignored:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=["x", "y", "z"])
>>> s = pd.Series([10, 20, 30, 40, 50], index=["x", "y", "a", "b", "z"])
>>> df["B"] = s
>>> df
   A      B
x  1    10
y  2    20
z  3    50
```

`add(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Addition of dataframe and other, element-wise (binary operator ``add``).

Equivalent to ``dataframe + other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``radd``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                    'degrees': [360, 180, 360]},
...                    index=['circle', 'triangle', 'rectangle'])
>>> df
   angles  degrees
circle      0     360
triangle    3     180
```

```
rectangle      4      360
```

Add a scalar with operator version which return the same results.

```
>>> df + 1
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361
```

```
>>> df.add(1)
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361
```

Divide by constant with reverse version.

```
>>> df.div(10)
      angles  degrees
circle     0.0     36.0
triangle   0.3     18.0
rectangle  0.4     36.0
```

```
>>> df.rdiv(10)
      angles  degrees
circle      inf  0.027778
triangle   3.333333 0.055556
rectangle  2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358
```

```
>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1     359
triangle    2     179
rectangle   3     359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0     720
triangle    0     360
rectangle   0     720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6     360
rectangle   12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle     3
rectangle    4

>>> df * other
```

	angles	degrees
circle	0	NaN
triangle	9	NaN
rectangle	16	NaN

```
>>> df.mul(other, fill_value=0)
```

	angles	degrees
circle	0	0.0
triangle	9	0.0
rectangle	16	0.0

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
```

```
>>> df_multindex
```

	angles	degrees
A circle	0	360
triangle	3	180
rectangle	4	360
B square	4	360
pentagon	5	540
hexagon	6	720

```
>>> df.div(df_multindex, level=1, fill_value=0)
```

	angles	degrees
A circle	NaN	1.0
triangle	1.0	1.0
rectangle	1.0	1.0
B square	0.0	0.0
pentagon	0.0	0.0
hexagon	0.0	0.0

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                          'B': [6, 7, 8, 9]})
```

```
>>> df_pow.pow(2)
```

	A	B
0	4	36
1	9	49
2	16	64
3	25	81

```
agg = aggregate(self, func=None, axis: Axis = 0, *args, **kwargs)
```

aggregate(self, func=None, axis: Axis = 0, *args, **kwargs) from pandas.core.frame.DataFrame
Aggregate using one or more operations over the specified axis.

Parameters

func : function, str, list or dict

Function to use for aggregating the data. If a function, must either work when passed a DataFrame or when passed to DataFrame.apply.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. ``[np.sum, 'mean']``
- dict of axis labels -> functions, function names or list of such.

axis : {0 or 'index', 1 or 'columns'}, default 0

If 0 or 'index': apply function to each column.

If 1 or 'columns': apply function to each row.

*args

Positional arguments to pass to `func`.

**kwargs

Keyword arguments to pass to `func`.

Returns

scalar, Series or DataFrame

The return can be:

- * scalar : when Series.agg is called with single function

- * Series : when DataFrame.agg is called with a single function
- * DataFrame : when DataFrame.agg is called with several functions

See Also

DataFrame.apply : Perform any type of operations.
 DataFrame.transform : Perform transformation type operations.
 DataFrame.groupby : Perform operations over groups.
 DataFrame.resample : Perform operations over resampled bins.
 DataFrame.rolling : Perform operations over rolling window.
 DataFrame.expanding : Perform operations over expanding window.
 core.window.ewm.ExponentialMovingWindow : Perform operation over exponential weighted window.

Notes

The aggregation operations are always performed over an axis, either the index (default) or the column axis. This behavior is different from ``numpy`` aggregation functions (``mean``, ``median``, ``prod``, ``sum``, ``std``, ``var``), where the default is to compute the aggregation of the flattened array, e.g., ``numpy.mean(arr_2d)`` as opposed to ``numpy.mean(arr_2d, axis=0)``.

``agg`` is an alias for ``aggregate``. Use the alias.

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

A passed user-defined-function will be passed a Series for evaluation.

If ``func`` defines an index relabeling, ``axis`` must be ``0`` or ``index``.

Examples

```
>>> df = pd.DataFrame(
...     [[1, 2, 3], [4, 5, 6], [7, 8, 9], [np.nan, np.nan, np.nan]],
...     columns=["A", "B", "C"],
... )
```

Aggregate these functions over the rows.

```
>>> df.agg(["sum", "min"])
      A      B      C
sum  12.0  15.0  18.0
min   1.0   2.0   3.0
```

Different aggregations per column.

```
>>> df.agg({"A": ["sum", "min"], "B": ["min", "max"]})
      A      B
sum  12.0  NaN
min   1.0   2.0
max   NaN   8.0
```

Aggregate different functions over the columns and rename the index of the resulting DataFrame.

```
>>> df.agg(x=("A", "max"), y=("B", "min"), z=("C", "mean"))
      A      B      C
x   7.0  NaN  NaN
y   NaN  2.0  NaN
z   NaN  NaN  6.0
```

Aggregate over the columns.

```
>>> df.agg("mean", axis="columns")
0      2.0
1      5.0
2      8.0
3      NaN
dtype: float64
```

```
all(
    self,
    *,
    axis: Axis | None = 0,
```



```

    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> Series | bool from pandas.core.frame.DataFrame
    Return whether all elements are True, potentially over an axis.

    Returns True unless there at least one element within a series or
    along a DataFrame axis that is False or equivalent (e.g. zero or
    empty).

Parameters
-----
axis : {0 or 'index', 1 or 'columns', None}, default 0
    Indicate which axis or axes should be reduced. For `Series` this parameter
    is unused and defaults to 0.

    * 0 / 'index' : reduce the index, return a Series whose index is the
      original column labels.
    * 1 / 'columns' : reduce the columns, return a Series whose index is the
      original index.
    * None : reduce all axes, return a scalar.

bool_only : bool, default False
    Include only boolean columns. Not implemented for Series.
skipna : bool, default True
    Exclude NA/null values. If the entire row/column is NA and skipna is
    True, then the result will be True, as for an empty row/column.
    If skipna is False, then NA are treated as True, because these are not
    equal to zero.
**kwargs : any, default None
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.

Returns
-----
Series or scalar
    If axis=None, then a scalar boolean is returned.
    Otherwise a Series is returned with index matching the index argument.

See Also
-----
Series.all : Return True if all elements are True.
DataFrame.any : Return True if one (or more) elements are True.

Examples
-----
**Series**

>>> pd.Series([True, True]).all()
True
>>> pd.Series([True, False]).all()
False
>>> pd.Series([], dtype="float64").all()
True
>>> pd.Series([np.nan]).all()
True
>>> pd.Series([np.nan]).all(skipna=False)
True

**DataFrames**

Create a DataFrame from a dictionary.

>>> df = pd.DataFrame({"col1": [True, True], "col2": [True, False]})
>>> df
   col1  col2
0  True   True
1  True  False

Default behaviour checks if values in each column all return True.

>>> df.all()
col1    True
col2   False
dtype: bool

Specify ``axis='columns'`` to check if values in each row all return True.

```

```
>>> df.all(axis="columns")
0      True
1      False
dtype: bool
```

Or ``axis=None`` for whether every value is True.

```
>>> df.all(axis=None)
False
```

```
any(
    self,
    *,
    axis: Axis | None = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> Series | bool from pandas.core.frame.DataFrame
Return whether any element is True, potentially over an axis.
```

Returns False unless there is at least one element within a series or along a DataFrame axis that is True or equivalent (e.g. non-zero or non-empty).

Parameters

axis : {0 or 'index', 1 or 'columns', None}, default 0
Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

- * 0 / 'index' : reduce the index, return a Series whose index is the original column labels.
- * 1 / 'columns' : reduce the columns, return a Series whose index is the original index.
- * None : reduce all axes, return a scalar.

bool_only : bool, default False

Include only boolean columns. Not implemented for Series.

skipna : bool, default True

Exclude NA/null values. If the entire row/column is NA and skipna is True, then the result will be False, as for an empty row/column.

If skipna is False, then NA are treated as True, because these are not equal to zero.

****kwargs** : any, default None

Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or scalar

If axis=None, then a scalar boolean is returned.

Otherwise a Series is returned with index matching the index argument.

See Also

numpy.any : Numpy version of this method.

Series.any : Return whether any element is True.

Series.all : Return whether all elements are True.

DataFrame.any : Return whether any element is True over requested axis.

DataFrame.all : Return whether all elements are True over requested axis.

Examples

****Series****

For Series input, the output is a scalar indicating whether any element is True.

```
>>> pd.Series([False, False]).any()
False
>>> pd.Series([True, False]).any()
True
>>> pd.Series([], dtype="float64").any()
False
>>> pd.Series([np.nan]).any()
False
```

```
>>> pd.Series([np.nan]).any(skipna=False)
True
```

****DataFrame****

Whether each column contains at least one True element (the default).

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [0, 2], "C": [0, 0]})
>>> df
   A  B  C
0  1  0  0
1  2  2  0

>>> df.any()
A      True
B      True
C     False
dtype: bool
```

Aggregating over the columns.

```
>>> df = pd.DataFrame({"A": [True, False], "B": [1, 2]})
>>> df
   A  B
0  True  1
1 False  2

>>> df.any(axis="columns")
0      True
1      True
dtype: bool
```

```
>>> df = pd.DataFrame({"A": [True, False], "B": [1, 0]})
>>> df
   A  B
0  True  1
1 False  0

>>> df.any(axis="columns")
0      True
1     False
dtype: bool
```

Aggregating over the entire DataFrame with ``axis=None``.

```
>>> df.any(axis=None)
True
```

``any`` for an empty DataFrame is an empty Series.

```
>>> pd.DataFrame([]).any()
Series([], dtype: bool)
```

```
apply(
    self,
    func: AggFuncType,
    axis: Axis = 0,
    raw: bool = False,
    result_type: Literal['expand', 'reduce', 'broadcast'] | None = None,
    args=(),
    by_row: Literal[False, 'compat'] = 'compat',
    engine: Callable | None | Literal['python', 'numba'] = None,
    engine_kwargs: dict[str, bool] | None = None,
    **kwargs
) from pandas.core.frame.DataFrame
Apply a function along an axis of the DataFrame.

Objects passed to the function are Series objects whose index is
either the DataFrame's index (``axis=0``) or the DataFrame's columns
(``axis=1``). By default (``result_type=None``), the final return type
is inferred from the return type of the applied function. Otherwise,
it depends on the ``result_type`` argument. The return type of the applied
function is inferred based on the first computed result obtained after
applying the function to a Series object.
```

Parameters

```

func : function
    Function to apply to each column or row.
axis : {0 or 'index', 1 or 'columns'}, default 0
    Axis along which the function is applied:

    * 0 or 'index': apply function to each column.
    * 1 or 'columns': apply function to each row.

raw : bool, default False
    Determines if row or column is passed as a Series or ndarray object:

    * ``False`` : passes each row or column as a Series to the
      function.
    * ``True`` : the passed function will receive ndarray objects
      instead.
      If you are just applying a NumPy reduction function this will
      achieve much better performance.

.. note::

    When ``raw=True``, the result dtype is inferred from the first
    returned value.

result_type : {'expand', 'reduce', 'broadcast', None}, default None
    These only act when ``axis=1`` (columns):

    * 'expand' : list-like results will be turned into columns.
    * 'reduce' : returns a Series if possible rather than expanding
      list-like results. This is the opposite of 'expand'.
    * 'broadcast' : results will be broadcast to the original shape
      of the DataFrame, the original index and columns will be
      retained.

    The default behaviour (None) depends on the return value of the
    applied function: list-like results will be returned as a Series
    of those. However if the apply function returns a Series these
    are expanded to columns.
args : tuple
    Positional arguments to pass to `func` in addition to the
    array/series.
by_row : False or "compat", default "compat"
    Only has an effect when ``func`` is a listlike or dictlike of funcs
    and the func isn't a string.
    If "compat", will if possible first translate the func into pandas
    methods (e.g. ``Series().apply(np.sum)`` will be translated to
    ``Series().sum()``). If that doesn't work, will try call to apply again with
    ``by_row=True`` and if that fails, will call apply again with
    ``by_row=False`` (backward compatible).
    If False, the funcs will be passed the whole Series at once.

.. versionadded:: 2.1.0

engine : decorator or {'python', 'numba'}, optional
    Choose the execution engine to use. If not provided the function
    will be executed by the regular Python interpreter.

    Other options include JIT compilers such Numba and Bodo, which in some
    cases can speed up the execution. To use an executor you can provide
    the decorators ``numba.jit``, ``numba.njit`` or ``bodo.jit``. You can
    also provide the decorator with parameters, like ``numba.jit(nogit=True)``.

    Not all functions can be executed with all execution engines. In general,
    JIT compilers will require type stability in the function (no variable
    should change data type during the execution). And not all pandas and
    NumPy APIs are supported. Check the engine documentation [1]_ and [2]_
    for limitations.

.. warning::

    String parameters will stop being supported in a future pandas version.

.. versionadded:: 2.2.0

engine_kwargs : dict
    Pass keyword arguments to the engine.
    This is currently only used by the numba engine,
    see the documentation for the engine argument for more information.

```

```

**kwargs
    Additional keyword arguments to pass as keywords arguments to
    `func`.

Returns
-----
Series or DataFrame
    Result of applying ``func`` along the given axis of the
    DataFrame.

See Also
-----
DataFrame.map: For elementwise operations.
DataFrame.aggregate: Only perform aggregating type operations.
DataFrame.transform: Only perform transforming type operations.

Notes
-----
Functions that mutate the passed object can produce unexpected
behavior or errors and are not supported. See :ref:`gotchas.udf-mutation`
for more details.

References
-----
.. [1] `Numba documentation
    <https://numba.readthedocs.io/en/stable/index.html>`_
.. [2] `Bodo documentation
    <https://docs.bodo.ai/latest/>`_

Examples
-----
>>> df = pd.DataFrame([[4, 9]] * 3, columns=["A", "B"])
>>> df
   A  B
0  4  9
1  4  9
2  4  9

Using a numpy universal function (in this case the same as
`np.sqrt(df)`):

>>> df.apply(np.sqrt)
   A    B
0  2.0  3.0
1  2.0  3.0
2  2.0  3.0

Using a reducing function on either axis

>>> df.apply(np.sum, axis=0)
A    12
B    27
dtype: int64

>>> df.apply(np.sum, axis=1)
0    13
1    13
2    13
dtype: int64

Returning a list-like will result in a Series

>>> df.apply(lambda x: [1, 2], axis=1)
0    [1, 2]
1    [1, 2]
2    [1, 2]
dtype: object

Passing ``result_type='expand'`` will expand list-like results
to columns of a Dataframe

>>> df.apply(lambda x: [1, 2], axis=1, result_type="expand")
   0  1
0  1  2
1  1  2
2  1  2

```

Returning a Series inside the function is similar to passing ``result\_type='expand'``. The resulting column names will be the Series index.

```
>>> df.apply(lambda x: pd.Series([1, 2], index=["foo", "bar"]), axis=1)
   foo bar
0    1   2
1    1   2
2    1   2
```

Passing ``result\_type='broadcast'`` will ensure the same shape result, whether list-like or scalar is returned by the function, and broadcast it along the axis. The resulting column names will be the originals.

```
>>> df.apply(lambda x: [1, 2], axis=1, result_type="broadcast")
   A B
0  1 2
1  1 2
2  1 2
```

Advanced users can speed up their code by using a Just-in-time (JIT) compiler with ``apply``. The main JIT compilers available for pandas are Numba and Bodo. In general, JIT compilation is only possible when the function passed to ``apply`` has type stability (variables in the function do not change their type during the execution).

```
>>> import bodo # doctest: +SKIP
>>> df.apply(lambda x: x.A + x.B, axis=1, engine=bodo.jit) # doctest: +SKIP
```

Note that JIT compilation is only recommended for functions that take a significant amount of time to run. Fast functions are unlikely to run faster with JIT compilation.

`assign(self, **kwargs) -> DataFrame` from `pandas.core.frame.DataFrame`
Assign new columns to a DataFrame.

Returns a new object with all original columns in addition to new ones. Existing columns that are re-assigned will be overwritten.

Parameters

**kwargs : callable or Series
The column names are keywords. If the values are callable, they are computed on the DataFrame and assigned to the new columns. The callable must not change input DataFrame (though pandas doesn't check it). If the values are not callable, (e.g. a Series, scalar, or array), they are simply assigned.

Returns

DataFrame
A new DataFrame with the new columns in addition to all the existing columns.

See Also

`DataFrame.loc` : Select a subset of a DataFrame by labels.
`DataFrame.iloc` : Select a subset of a DataFrame by positions.

Notes

Assigning multiple columns within the same ``assign`` is possible. Later items in ``\*\*kwargs`` may refer to newly created or modified columns in 'df'; items are computed and assigned into 'df' in order.

Examples

```
>>> df = pd.DataFrame({"temp_c": [17.0, 25.0]}, index=["Portland", "Berkeley"])
>>> df
   temp_c
Portland  17.0
Berkeley  25.0
```

Where the value is a callable, evaluated on `df`:

```
>>> df.assign(temp_f=lambda x: x.temp_c * 9 / 5 + 32)
      temp_c  temp_f
Portland   17.0    62.6
Berkeley   25.0    77.0
```

Alternatively, the same behavior can be achieved by directly referencing an existing Series or sequence:

```
>>> df.assign(temp_f=df["temp_c"] * 9 / 5 + 32)
      temp_c  temp_f
Portland   17.0    62.6
Berkeley   25.0    77.0
```

or by using `:meth:`pandas.col``:

```
>>> df.assign(temp_f=pd.col("temp_c") * 9 / 5 + 32)
      temp_c  temp_f
Portland   17.0    62.6
Berkeley   25.0    77.0
```

You can create multiple columns within the same assign where one of the columns depends on another one defined within the same assign:

```
>>> df.assign(
...     temp_f=lambda x: x["temp_c"] * 9 / 5 + 32,
...     temp_k=lambda x: (x["temp_f"] + 459.67) * 5 / 9,
... )
      temp_c  temp_f  temp_k
Portland   17.0    62.6  290.15
Berkeley   25.0    77.0  298.15
```

```
boxplot = boxplot_frame(
    self: DataFrame,
    column=None,
    by=None,
    ax=None,
    fontsize: int | None = None,
    rot: int = 0,
    grid: bool = True,
    figsize: tuple[float, float] | None = None,
    layout=None,
    return_type=None,
    backend=None,
    **kwargs
)
```

from pandas.plotting
Make a box plot from DataFrame columns.

Make a box-and-whisker plot from DataFrame columns, optionally grouped by some other columns. A box plot is a method for graphically depicting groups of numerical data through their quartiles. The box extends from the Q1 to Q3 quartile values of the data, with a line at the median (Q2). The whiskers extend from the edges of box to show the range of the data. By default, they extend no more than `1.5 * IQR (IQR = Q3 - Q1)` from the edges of the box, ending at the farthest data point within that interval. Outliers are plotted as separate dots.

For further details see
Wikipedia's entry for `boxplot` <https://en.wikipedia.org/wiki/Box_plot>`_.

Parameters

```
column : str or list of str, optional
    Column name or list of names, or vector.
    Can be any valid input to :meth:`pandas.DataFrame.groupby`.
by : str or array-like, optional
    Column in the DataFrame to :meth:`pandas.DataFrame.groupby`.
    One box-plot will be done per value of columns in `by`.
ax : object of class matplotlib.axes.Axes, optional
    The matplotlib axes to be used by boxplot.
fontsize : float or str
    Tick label font size in points or as a string (e.g., `large`).
rot : float, default 0
    The rotation angle of labels (in degrees)
    with respect to the screen coordinate system.
grid : bool, default True
    Setting this to True will show the grid.
```

```

figsize : A tuple (width, height) in inches
    The size of the figure to create in matplotlib.
layout : tuple (rows, columns), optional
    For example, (3, 5) will display the subplots
    using 3 rows and 5 columns, starting from the top-left.
return_type : {'axes', 'dict', 'both'} or None, default 'axes'
    The kind of object to return. The default is ``axes``.

    * 'axes' returns the matplotlib axes the boxplot is drawn on.
    * 'dict' returns a dictionary whose values are the matplotlib
      lines of the boxplot.
    * 'both' returns a namedtuple with the axes and dict.
    * when grouping with ``by``, a Series mapping columns to
      ``return_type`` is returned.

    If ``return_type`` is `None`, a NumPy array
    of axes with the same shape as ``layout`` is returned.
backend : str, default None
    Backend to use instead of the backend specified in the option
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
    specify the ``plotting.backend`` for the whole session, set
    ``pd.options.plotting.backend``.

**kwargs
    All other plotting keyword arguments to be passed to
    :func:`matplotlib.pyplot.boxplot`.

Returns
-----
result
    See Notes.

See Also
-----
Series.plot.hist: Make a histogram.
matplotlib.pyplot.boxplot : Matplotlib equivalent plot.

Notes
-----
The return type depends on the `return_type` parameter:

* 'axes' : object of class matplotlib.axes.Axes
* 'dict' : dict of matplotlib.lines.Line2D objects
* 'both' : a namedtuple with structure (ax, lines)

For data grouped with ``by``, return a Series of the above or a numpy
array:

* :class:`~pandas.Series`
* :class:`~numpy.array` (for ``return_type = None``)

Use ``return_type='dict'`` when you want to tweak the appearance
of the lines after plotting. In this case a dict containing the Lines
making up the boxes, caps, fliers, medians, and whiskers is returned.

Examples
-----

Boxplots can be created for every column in the dataframe
by ``df.boxplot()`` or indicating the columns to be used:

.. plot::
   :context: close-figs

   >>> np.random.seed(1234)
   >>> df = pd.DataFrame(
   ...     np.random.randn(10, 4), columns=["Col1", "Col2", "Col3", "Col4"]
   ... )
   >>> boxplot = df.boxplot(column=["Col1", "Col2", "Col3"]) # doctest: +SKIP

Boxplots of variables distributions grouped by the values of a third
variable can be created using the option ``by``. For instance:

.. plot::
   :context: close-figs

   >>> df = pd.DataFrame(np.random.randn(10, 2), columns=["Col1", "Col2"])

```



```
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])
>>> boxplot = df.boxplot(by="X")
```

A list of strings (i.e. ``['X', 'Y']``) can be passed to boxplot in order to group the data by combination of the variables in the x-axis:

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(np.random.randn(10, 3), columns=["Col1", "Col2", "Col3"])
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])
>>> df["Y"] = pd.Series(["A", "B", "A", "B", "A", "B", "A", "B", "A", "B"])
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by=["X", "Y"])
```

The layout of boxplot can be adjusted giving a tuple to ``layout``:

```
.. plot::
:context: close-figs

>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", layout=(2, 1))
```

Additional formatting can be done to the boxplot, like suppressing the grid (``grid=False``), rotating the labels in the x-axis (i.e. ``rot=45``) or changing the fontsize (i.e. ``fontsize=15``):

```
.. plot::
:context: close-figs

>>> boxplot = df.boxplot(grid=False, rot=45, fontsize=15) # doctest: +SKIP
```

The parameter ``return\_type`` can be used to select the type of element returned by 'boxplot'. When ``return\_type='axes'`` is selected, the matplotlib axes on which the boxplot is drawn are returned:

```
.. plot::
:context: close-figs

>>> boxplot = df.boxplot(column=["Col1", "Col2"], return_type="axes")
>>> type(boxplot)
<class 'matplotlib.axes._axes.Axes'>
```

When grouping with ``by``, a Series mapping columns to ``return\_type`` is returned:

```
.. plot::
:context: close-figs

>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type="axes")
>>> type(boxplot)
<class 'pandas.Series'>
```

If ``return\_type`` is ``None``, a NumPy array of axes with the same shape as ``layout`` is returned:

```
.. plot::
:context: close-figs

>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type=None)
>>> type(boxplot)
<class 'numpy.ndarray'>
```

```
combine(
    self,
    other: DataFrame,
    func: Callable[[Series, Series], Series | Hashable],
    fill_value=None,
    overwrite: bool = True
) -> DataFrame from pandas.core.frame.DataFrame
Perform column-wise combine with another DataFrame.
```

Combines a DataFrame with ``other`` DataFrame using ``func`` to element-wise combine columns. The row and column indexes of the resulting DataFrame will be the union of the two.

Parameters

```
-----
other : DataFrame
```

```

    The DataFrame to merge column-wise.
func : function
    Function that takes two series as inputs and return a Series or a
    scalar. Used to merge the two dataframes column by columns.
fill_value : scalar value, default None
    The value to fill NaNs with prior to passing any column to the
    merge func.
overwrite : bool, default True
    If True, columns in `self` that do not exist in `other` will be
    overwritten with NaNs.

Returns
-----
DataFrame
    Combination of the provided DataFrames.

See Also
-----
DataFrame.combine_first : Combine two DataFrame objects and default to
    non-null values in frame calling the method.

Examples
-----
Combine using a simple function that chooses the smaller column.

>>> df1 = pd.DataFrame({"A": [0, 0], "B": [4, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> take_smaller = lambda s1, s2: s1 if s1.sum() < s2.sum() else s2
>>> df1.combine(df2, take_smaller)
   A  B
0  0  3
1  0  3

Example using a true element-wise combine function.

>>> df1 = pd.DataFrame({"A": [5, 0], "B": [2, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine(df2, np.minimum)
   A  B
0  1  2
1  0  3

Using `fill_value` fills Nones prior to passing the column to the
merge function.

>>> df1 = pd.DataFrame({"A": [0, 0], "B": [None, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine(df2, take_smaller, fill_value=-5)
   A  B
0  0 -5.0
1  0  4.0

Example that demonstrates the use of `overwrite` and behavior when
the axis differ between the dataframes.

>>> df1 = pd.DataFrame({"A": [0, 0], "B": [4, 4]})
>>> df2 = pd.DataFrame(
...     {
...         "B": [3, 3],
...         "C": [-10, 1],
...     },
...     index=[1, 2],
... )
>>> df1.combine(df2, take_smaller)
   A  B  C
0 NaN NaN NaN
1 NaN 3.0 -10.0
2 NaN 3.0  1.0

>>> df1.combine(df2, take_smaller, overwrite=False)
   A  B  C
0 0.0 NaN NaN
1 0.0 3.0 -10.0
2 NaN 3.0  1.0

Demonstrating the preference of the passed in dataframe.

```

```

>>> df2 = pd.DataFrame(
...     {
...         "B": [3, 3],
...         "C": [1, 1],
...     },
...     index=[1, 2],
... )
>>> df2.combine(df1, take_smaller)
   B  C  A
0 NaN NaN 0.0
1 3.0 NaN 0.0
2 3.0 NaN NaN

>>> df2.combine(df1, take_smaller, overwrite=False)
   B  C  A
0 NaN NaN 0.0
1 3.0 1.0 0.0
2 3.0 1.0 NaN

```

`combine_first(self, other: DataFrame) -> DataFrame` from `pandas.core.frame.DataFrame`
 Update null elements with value in the same location in `other`.

Combine two DataFrame objects by filling null values in one DataFrame with non-null values from other DataFrame. The row and column indexes of the resulting DataFrame will be the union of the two. The resulting dataframe contains the 'first' dataframe values and overrides the second one values where both `first.loc[index, col]` and `second.loc[index, col]` are not missing values, upon calling `first.combine_first(second)`.

Parameters

`other : DataFrame`
 Provided DataFrame to use to fill null values.

Returns

`DataFrame`
 The result of combining the provided DataFrame with the other object.

See Also

`DataFrame.combine` : Perform series-wise operation on two DataFrames using a given function.

Examples

```

>>> df1 = pd.DataFrame({"A": [None, 0], "B": [None, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine_first(df2)
   A  B
0 1.0 3.0
1 0.0 4.0

```

Null values still persist if the location of that null value does not exist in `other`

```

>>> df1 = pd.DataFrame({"A": [None, 0], "B": [4, None]})
>>> df2 = pd.DataFrame({"B": [3, 3], "C": [1, 1]}, index=[1, 2])
>>> df1.combine_first(df2)
   A  B  C
0 NaN 4.0 NaN
1 0.0 3.0 1.0
2 NaN 3.0 1.0

```

`compare(`
`self,`
`other: DataFrame,`
`align_axis: Axis = 1,`
`keep_shape: bool = False,`
`keep_equal: bool = False,`
`result_names: Suffixes = ('self', 'other')`
`) -> DataFrame` from `pandas.core.frame.DataFrame`
 Compare to another DataFrame and show the differences.

Parameters

other : DataFrame
Object to compare with.

align_axis : {0 or 'index', 1 or 'columns'}, default 1
Determine which axis to align the comparison on.

- * 0, or 'index' : Resulting differences are stacked vertically with rows drawn alternately from self and other.
- * 1, or 'columns' : Resulting differences are aligned horizontally with columns drawn alternately from self and other.

keep_shape : bool, default False
If true, all rows and columns are kept.
Otherwise, only the ones with different values are kept.

keep_equal : bool, default False
If true, the result keeps values that are equal.
Otherwise, equal values are shown as NaNs.

result_names : tuple, default ('self', 'other')
Set the dataframes names in the comparison.

Returns

DataFrame

DataFrame that shows the differences stacked side by side.

The resulting index will be a MultiIndex with 'self' and 'other' stacked alternately at the inner level.

Raises

ValueError

When the two DataFrames don't have identical labels or shape.

See Also

Series.compare : Compare with another Series and show differences.

DataFrame.equals : Test whether two objects contain the same elements.

Notes

Matching NaNs will not appear as a difference.

Can only compare identically-labeled
(i.e. same shape, identical row and column labels) DataFrames

Examples

```
>>> df = pd.DataFrame(
...     {
...         "col1": ["a", "a", "b", "b", "a"],
...         "col2": [1.0, 2.0, 3.0, np.nan, 5.0],
...         "col3": [1.0, 2.0, 3.0, 4.0, 5.0],
...     },
...     columns=["col1", "col2", "col3"],
... )
```

```
>>> df
   col1  col2  col3
0     a   1.0   1.0
1     a   2.0   2.0
2     b   3.0   3.0
3     b   NaN   4.0
4     a   5.0   5.0
```

```
>>> df2 = df.copy()
>>> df2.loc[0, "col1"] = "c"
>>> df2.loc[2, "col3"] = 4.0
>>> df2
   col1  col2  col3
0     c   1.0   1.0
1     a   2.0   2.0
2     b   3.0   4.0
3     b   NaN   4.0
4     a   5.0   5.0
```

Align the differences on columns

```

>>> df.compare(df2)
      col1      col3
self other self other
0      a      c  NaN  NaN
2  NaN  NaN  3.0  4.0

Assign result_names

>>> df.compare(df2, result_names=("left", "right"))
      col1      col3
left right left right
0      a      c  NaN  NaN
2  NaN  NaN  3.0  4.0

Stack the differences on rows

>>> df.compare(df2, align_axis=0)
      col1      col3
0 self      a  NaN
  other      c  NaN
2 self  NaN  3.0
  other  NaN  4.0

Keep the equal values

>>> df.compare(df2, keep_equal=True)
      col1      col3
self other self other
0      a      c  1.0  1.0
2      b      b  3.0  4.0

Keep all original rows and columns

>>> df.compare(df2, keep_shape=True)
      col1      col2      col3
self other self other self other
0      a      c  NaN  NaN  NaN  NaN
1  NaN  NaN  NaN  NaN  NaN  NaN
2  NaN  NaN  NaN  NaN  3.0  4.0
3  NaN  NaN  NaN  NaN  NaN  NaN
4  NaN  NaN  NaN  NaN  NaN  NaN

Keep all original rows and columns and also all original values

>>> df.compare(df2, keep_shape=True, keep_equal=True)
      col1      col2      col3
self other self other self other
0      a      c  1.0  1.0  1.0  1.0
1      a      a  2.0  2.0  2.0  2.0
2      b      b  3.0  3.0  3.0  4.0
3      b      b  NaN  NaN  4.0  4.0
4      a      a  5.0  5.0  5.0  5.0

```

corr(
self,
method: CorrelationMethod = 'pearson',
min_periods: int = 1,
numeric_only: bool = False
) -> DataFrame from pandas.core.frame.DataFrame
Compute pairwise correlation of columns, excluding NA/null values.

Parameters

method : {'pearson', 'kendall', 'spearman'} or callable
Method of correlation:

- * pearson : standard correlation coefficient
- * kendall : Kendall Tau correlation coefficient
- * spearman : Spearman rank correlation
- * callable: callable with input two 1d ndarrays and returning a float. Note that the returned matrix from corr will have 1 along the diagonals and will be symmetric regardless of the callable's behavior.

min_periods : int, optional
Minimum number of observations required per pair of columns to have a valid result. Currently only available for Pearson

and Spearman correlation.
numeric_only : bool, default False
Include only 'float', 'int' or 'boolean' data.
.. versionchanged:: 2.0.0
The default value of 'numeric_only' is now 'False'.

Returns

DataFrame
Correlation matrix.

See Also

DataFrame.corrwith : Compute pairwise correlation with another
DataFrame or Series.
Series.corr : Compute the correlation between two Series.

Notes

Pearson, Kendall and Spearman correlation are currently computed using pairwise complete
* 'Pearson correlation coefficient <https://en.wikipedia.org/wiki/Pearson_correlation_coefficient>
* 'Kendall rank correlation coefficient <https://en.wikipedia.org/wiki/Kendall_rank_correlation_coefficient>
* 'Spearman's rank correlation coefficient <https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient>

Examples

>>> def histogram_intersection(a, b):
... v = np.minimum(a, b).sum().round(decimals=1)
... return v
>>> df = pd.DataFrame(
... [(0.2, 0.3), (0.0, 0.6), (0.6, 0.0), (0.2, 0.1)],
... columns=["dogs", "cats"],
...)
>>> df.corr(method=histogram_intersection)
 dogs cats
dogs 1.0 0.3
cats 0.3 1.0

>>> df = pd.DataFrame(
... [(1, 1), (2, np.nan), (np.nan, 3), (4, 4)], columns=["dogs", "cats"]
...)
>>> df.corr(min_periods=3)
 dogs cats
dogs 1.0 NaN
cats NaN 1.0

corrwith(
 self,
 other: DataFrame | Series,
 axis: Axis = 0,
 drop: bool = False,
 method: CorrelationMethod = 'pearson',
 numeric_only: bool = False,
 min_periods: int | None = None
) -> Series from pandas.core.frame.DataFrame
Compute pairwise correlation.

Pairwise correlation is computed between rows or columns of
DataFrame with rows or columns of Series or DataFrame. DataFrames
are first aligned along both axes before computing the
correlations.

Parameters

other : DataFrame, Series
Object with which to compute correlations.
axis : {0 or 'index', 1 or 'columns'}, default 0
The axis to use. 0 or 'index' to compute row-wise, 1 or 'columns' for
column-wise.
drop : bool, default False
Drop missing indices from result.
method : {'pearson', 'kendall', 'spearman'} or callable
Method of correlation:

* pearson : standard correlation coefficient

```

* kendall : Kendall Tau correlation coefficient
* spearman : Spearman rank correlation
* callable: callable with input two 1d ndarrays
    and returning a float.

numeric_only : bool, default False
    Include only `float`, `int` or `boolean` data.

min_periods : int, optional
    Minimum number of observations needed to have a valid result.

    .. versionchanged:: 2.0.0
        The default value of ``numeric_only`` is now ``False``.

Returns
-----
Series
    Pairwise correlations.

See Also
-----
DataFrame.corr : Compute pairwise correlation of columns.

Examples
-----
>>> index = ["a", "b", "c", "d", "e"]
>>> columns = ["one", "two", "three", "four"]
>>> df1 = pd.DataFrame(
...     np.arange(20).reshape(5, 4), index=index, columns=columns
... )
>>> df2 = pd.DataFrame(
...     np.arange(16).reshape(4, 4), index=index[:4], columns=columns
... )
>>> df1.corrwith(df2)
one      1.0
two      1.0
three    1.0
four     1.0
dtype: float64

>>> df2.corrwith(df1, axis=1)
a      1.0
b      1.0
c      1.0
d      1.0
e      NaN
dtype: float64

count(self, axis: Axis = 0, numeric_only: bool = False) -> Series from pandas.core.frame.DataFrame
    Count non-NA cells for each column or row.

    The values `None`, `NaN`, `NaT`, ``pandas.NA`` are considered NA.

Parameters
-----
axis : {0 or 'index', 1 or 'columns'}, default 0
    If 0 or 'index' counts are generated for each column.
    If 1 or 'columns' counts are generated for each row.
numeric_only : bool, default False
    Include only `float`, `int` or `boolean` data.

Returns
-----
Series
    For each column/row the number of non-NA/null entries.

See Also
-----
Series.count: Number of non-NA elements in a Series.
DataFrame.value_counts: Count unique combinations of columns.
DataFrame.shape: Number of DataFrame rows and columns (including NA
    elements).
DataFrame.isna: Boolean same-sized DataFrame showing places of NA
    elements.

Examples
-----

```

Constructing DataFrame from a dictionary:

```
>>> df = pd.DataFrame(
...     {
...         "Person": ["John", "Myla", "Lewis", "John", "Myla"],
...         "Age": [24.0, np.nan, 21.0, 33, 26],
...         "Single": [False, True, True, True, False],
...     }
... )
>>> df
   Person  Age  Single
0    John  24.0   False
1    Myla   NaN    True
2   Lewis  21.0    True
3    John  33.0    True
4    Myla  26.0   False
```

Notice the uncounted NA values:

```
>>> df.count()
Person    5
Age       4
Single    5
dtype: int64
```

Counts for each **row**:

```
>>> df.count(axis="columns")
0    3
1    2
2    3
3    3
4    3
dtype: int64
```

```
cov(
    self,
    min_periods: int | None = None,
    ddof: int | None = 1,
    numeric_only: bool = False
) -> DataFrame from pandas.core.frame.DataFrame
Compute pairwise covariance of columns, excluding NA/null values.
```

Compute the pairwise covariance among the series of a DataFrame.
The returned data frame is the `covariance matrix`
<https://en.wikipedia.org/wiki/Covariance_matrix>`\_\_` of the columns
of the DataFrame.

Both NA and null values are automatically excluded from the
calculation. (See the note below about bias from missing values.)
A threshold can be set for the minimum number of
observations for each value created. Comparisons with observations
below this threshold will be returned as ``NaN``.

This method is generally used for the analysis of time series data to
understand the relationship between different measures
across time.

Parameters

```
-----
min_periods : int, optional
    Minimum number of observations required per pair of columns
    to have a valid result.

ddof : int, default 1
    Delta degrees of freedom. The divisor used in calculations
    is ``N - ddof``, where ``N`` represents the number of elements.
    This argument is applicable only when no ``nan`` is in the dataframe.

numeric_only : bool, default False
    Include only `float`, `int` or `boolean` data.

.. versionchanged:: 2.0.0
    The default value of ``numeric_only`` is now ``False``.
```

Returns

DataFrame

The covariance matrix of the series of the DataFrame.

See Also

Series.cov : Compute covariance with another Series.

core.window.ewm.ExponentialMovingWindow.cov : Exponential weighted sample covariance.

core.window.expanding.Expanding.cov : Expanding sample covariance.

core.window.rolling.Rolling.cov : Rolling sample covariance.

Notes

Returns the covariance matrix of the DataFrame's time series.
The covariance is normalized by N-ddof.

For DataFrames that have Series that are missing data (assuming that data is `missing at random`

<https://en.wikipedia.org/wiki/Missing_data#Missing_at_random>`\_\_`)
the returned covariance matrix will be an unbiased estimate of the variance and covariance between the member Series.

However, for many applications this estimate may not be acceptable because the estimate covariance matrix is not guaranteed to be positive semi-definite. This could lead to estimate correlations having absolute values which are greater than one, and/or a non-invertible covariance matrix. See `Estimation of covariance matrices` <https://en.wikipedia.org/w/index.php?title=Estimation_of_covariance_matrices>`\_\_` for more details.

Examples

```
>>> df = pd.DataFrame(
...     [(1, 2), (0, 3), (2, 0), (1, 1)], columns=["dogs", "cats"]
... )
>>> df.cov()
           dogs      cats
dogs  0.666667 -1.000000
cats -1.000000  1.666667
```

```
>>> np.random.seed(42)
>>> df = pd.DataFrame(
...     np.random.randn(1000, 5), columns=["a", "b", "c", "d", "e"]
... )
>>> df.cov()
           a          b          c          d          e
a  0.998438 -0.020161  0.059277 -0.008943  0.014144
b -0.020161  1.059352 -0.008543 -0.024738  0.009826
c  0.059277 -0.008543  1.010670 -0.001486 -0.000271
d -0.008943 -0.024738 -0.001486  0.921297 -0.013692
e  0.014144  0.009826 -0.000271 -0.013692  0.977795
```

****Minimum number of periods****

This method also supports an optional ``min\_periods`` keyword that specifies the required minimum number of non-NA observations for each column pair in order to have a valid result:

```
>>> np.random.seed(42)
>>> df = pd.DataFrame(np.random.randn(20, 3), columns=["a", "b", "c"])
>>> df.loc[df.index[:5], "a"] = np.nan
>>> df.loc[df.index[5:10], "b"] = np.nan
>>> df.cov(min_periods=12)
           a          b          c
a  0.316741      NaN -0.150812
b      NaN  1.248003  0.191417
c -0.150812  0.191417  0.895202
```

```
cummax(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
) -> Self from pandas.core.frame.DataFrame
Return cumulative maximum over a DataFrame or Series axis.
```

Returns a DataFrame or Series of the same size containing the cumulative maximum.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
The index or the name of the axis. 0 is equivalent to None or 'index'.
For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
Exclude NA/null values. If an entire row/column is NA, the result will be NA.
numeric_only : bool, default False
Include only float, int, boolean columns.
*args, **kwargs
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or DataFrame
Return cumulative maximum of Series or DataFrame.

See Also

core.window.expanding.Expanding.max : Similar functionality but ignores ``NaN`` values.
DataFrame.max : Return the maximum over DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

Series

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummax()
0    2.0
1    NaN
2    5.0
3    5.0
4    5.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummax(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

DataFrame

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

By default, iterates over rows and finds the maximum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cummax()
      A    B
0  2.0  1.0
1  3.0  NaN
2  3.0  1.0
```

To iterate over columns and find the maximum in each row, use ``axis=1``

```
>>> df.cummax(axis=1)
      A    B
0  2.0  2.0
1  3.0  NaN
2  1.0  1.0
```

```
cummin(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
) -> Self from pandas.core.frame.DataFrame
Return cumulative minimum over a DataFrame or Series axis.
```

Returns a DataFrame or Series of the same size containing the cumulative minimum.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
The index or the name of the axis. 0 is equivalent to None or 'index'.
For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
Exclude NA/null values. If an entire row/column is NA, the result will be NA.
numeric_only : bool, default False
Include only float, int, boolean columns.
*args, **kwargs
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or DataFrame
Return cumulative minimum of Series or DataFrame.

See Also

core.window.expanding.Expanding.min : Similar functionality but ignores ``NaN`` values.
DataFrame.min : Return the minimum over DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

```
**Series**
```

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummin()
0    2.0
1    NaN
2    2.0
3   -1.0
4   -1.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummin(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

****DataFrame****

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

By default, iterates over rows and finds the minimum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cummin()
   A    B
0  2.0  1.0
1  2.0  NaN
2  1.0  0.0
```

To iterate over columns and find the minimum in each row, use ``axis=1``

```
>>> df.cummin(axis=1)
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

```
cumprod(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
) -> Self from pandas.core.frame.DataFrame
Return cumulative product over a DataFrame or Series axis.
```

Returns a DataFrame or Series of the same size containing the cumulative product.

Parameters

```
axis : {0 or 'index', 1 or 'columns'}, default 0
    The index or the name of the axis. 0 is equivalent to None or 'index'.
    For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If an entire row/column is NA, the result
    will be NA.
numeric_only : bool, default False
    Include only float, int, boolean columns.
*args, **kwargs
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.
```

Returns

Series or DataFrame

Return cumulative product of Series or DataFrame.

See Also

core.window.expanding.Expanding.prod : Similar functionality
but ignores ``NaN`` values.
DataFrame.prod : Return the product over
DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

Series

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cumprod()
0    2.0
1    NaN
2   10.0
3  -10.0
4   -0.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cumprod(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

DataFrame

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

By default, iterates over rows and finds the product in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cumprod()
   A    B
0  2.0  1.0
1  6.0  NaN
2  6.0  0.0
```

To iterate over columns and find the product in each row, use ``axis=1``

```
>>> df.cumprod(axis=1)
   A    B
0  2.0  2.0
1  3.0  NaN
2  1.0  0.0
```

```
cumsum(
    self,
```

```

axis: Axis = 0,
skipna: bool = True,
numeric_only: bool = False,
*args,
**kwargs
) -> Self from pandas.core.frame.DataFrame
Return cumulative sum over a DataFrame or Series axis.

Returns a DataFrame or Series of the same size containing the cumulative
sum.

Parameters
-----
axis : {0 or 'index', 1 or 'columns'}, default 0
    The index or the name of the axis. 0 is equivalent to None or 'index'.
    For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If an entire row/column is NA, the result
    will be NA.
numeric_only : bool, default False
    Include only float, int, boolean columns.
*args, **kwargs
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.

Returns
-----
Series or DataFrame
    Return cumulative sum of Series or DataFrame.

See Also
-----
core.window.expanding.Expanding.sum : Similar functionality
    but ignores ``NaN`` values.
DataFrame.sum : Return the sum over
    DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples
-----
**Series**

>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64

By default, NA values are ignored.

>>> s.cumsum()
0    2.0
1    NaN
2    7.0
3    6.0
4    6.0
dtype: float64

To include NA values in the operation, use ``skipna=False``

>>> s.cumsum(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

**DataFrame**

>>> df = pd.DataFrame(

```

```
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A  B
0  2.0 1.0
1  3.0 NaN
2  1.0 0.0
```

By default, iterates over rows and finds the sum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cumsum()
   A  B
0  2.0 1.0
1  5.0 NaN
2  6.0 1.0
```

To iterate over columns and find the sum in each row, use ``axis=1``

```
>>> df.cumsum(axis=1)
   A  B
0  2.0 3.0
1  3.0 NaN
2  1.0 1.0
```

`diff(self, periods: int = 1, axis: Axis = 0) -> DataFrame` from `pandas.core.frame.DataFrame`
First discrete difference of element.

Calculates the difference of a DataFrame element compared with another element in the DataFrame (default is element in previous row).

Parameters

`periods` : int, default 1
Periods to shift for calculating difference, accepts negative values.
`axis` : {0 or 'index', 1 or 'columns'}, default 0
Take difference over rows (0) or columns (1).

Returns

`DataFrame`
First differences of the Series.

See Also

`DataFrame.pct_change`: Percent change over given number of periods.
`DataFrame.shift`: Shift index by desired number of periods with an optional time freq.
`Series.diff`: First discrete difference of object.

Notes

For boolean dtypes, this uses `:meth:`operator.xor`` rather than `:meth:`operator.sub``.
The result is calculated according to current dtype in DataFrame, however dtype of the result is always float64.

Examples

Difference with previous row

```
>>> df = pd.DataFrame(
...     {
...         "a": [1, 2, 3, 4, 5, 6],
...         "b": [1, 1, 2, 3, 5, 8],
...         "c": [1, 4, 9, 16, 25, 36],
...     }
... )
>>> df
   a  b  c
0  1  1  1
1  2  1  4
2  3  2  9
3  4  3 16
```

```

4  5  5  25
5  6  8  36
>>> df.diff()
      a    b    c
0  NaN  NaN  NaN
1  1.0  0.0  3.0
2  1.0  1.0  5.0
3  1.0  1.0  7.0
4  1.0  2.0  9.0
5  1.0  3.0  11.0

```

Difference with previous column

```

>>> df.diff(axis=1)
      a    b    c
0  NaN  0    0
1  NaN -1    3
2  NaN -1    7
3  NaN -1   13
4  NaN  0   20
5  NaN  2   28

```

Difference with 3rd previous row

```

>>> df.diff(periods=3)
      a    b    c
0  NaN  NaN  NaN
1  NaN  NaN  NaN
2  NaN  NaN  NaN
3  3.0  2.0  15.0
4  3.0  4.0  21.0
5  3.0  6.0  27.0

```

Difference with following row

```

>>> df.diff(periods=-1)
      a    b    c
0 -1.0  0.0 -3.0
1 -1.0 -1.0 -5.0
2 -1.0 -1.0 -7.0
3 -1.0 -2.0 -9.0
4 -1.0 -3.0 -11.0
5  NaN  NaN  NaN

```

Overflow in input dtype

```

>>> df = pd.DataFrame({"a": [1, 0]}, dtype=np.uint8)
>>> df.diff()
      a
0  NaN
1  255.0

```

`div = truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`divide = truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`dot(self, other: AnyArrayLike | DataFrame) -> DataFrame | Series` from `pandas.core.frame.DataFrame`
 Compute the matrix multiplication between the DataFrame and other.

This method computes the matrix product between the DataFrame and the values of an other Series, DataFrame or a numpy array.

It can also be called using `self @ other`.

Parameters

`other` : Series, DataFrame or array-like
 The other object to compute the matrix product with.

Returns

Series or DataFrame

If other is a Series, return the matrix product between self and other as a Series. If other is a DataFrame or a numpy.array, return the matrix product of self and other in a DataFrame of a np.array.

See Also

Series.dot: Similar method for Series.

Notes

The dimensions of DataFrame and other must be compatible in order to compute the matrix multiplication. In addition, the column names of DataFrame and the index of other must contain the same values, as they will be aligned prior to the multiplication.

The dot method for Series computes the inner product, instead of the matrix product here.

Examples

Here we multiply a DataFrame with a Series.

```
>>> df = pd.DataFrame([[0, 1, -2, -1], [1, 1, 1, 1]])
>>> s = pd.Series([1, 1, 2, 1])
>>> df.dot(s)
0    -4
1     5
dtype: int64
```

Here we multiply a DataFrame with another DataFrame.

```
>>> other = pd.DataFrame([[0, 1], [1, 2], [-1, -1], [2, 0]])
>>> df.dot(other)
      0    1
0     1    4
1     2    2
```

Note that the dot method give the same result as @

```
>>> df @ other
      0    1
0     1    4
1     2    2
```

The dot method works also if other is an np.array.

```
>>> arr = np.array([[0, 1], [1, 2], [-1, -1], [2, 0]])
>>> df.dot(arr)
      0    1
0     1    4
1     2    2
```

Note how shuffling of the objects does not change the result.

```
>>> s2 = s.reindex([1, 0, 2, 3])
>>> df.dot(s2)
0    -4
1     5
dtype: int64
```

```
drop(
    self,
    labels: IndexLabel | ListLike = None,
    *,
    axis: Axis = 0,
    index: IndexLabel | ListLike = None,
    columns: IndexLabel | ListLike = None,
    level: Level | None = None,
    inplace: bool = False,
    errors: IgnoreRaise = 'raise'
) -> DataFrame | None from pandas.core.frame.DataFrame
Drop specified labels from rows or columns.
```

Remove rows or columns by specifying label names and corresponding axis, or by directly specifying index or column names. When using a multi-index, labels on different levels can be removed by specifying the level. See the :ref:`user guide <advanced.shown\_levels>` for more information about the now unused levels.

Parameters

labels : single label or iterable of labels

Index or column labels to drop. A tuple will be used as a single label and not treated as an iterable.

axis : {0 or 'index', 1 or 'columns'}, default 0
Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns').

index : single label or iterable of labels
Alternative to specifying axis ('labels, axis=0' is equivalent to 'index=labels').

columns : single label or iterable of labels
Alternative to specifying axis ('labels, axis=1' is equivalent to 'columns=labels').

level : int or level name, optional
For MultiIndex, level from which the labels will be removed.

inplace : bool, default False
If False, return a copy. Otherwise, do operation in place and return None.

errors : {'ignore', 'raise'}, default 'raise'
If 'ignore', suppress error and only existing labels are dropped.

Returns

DataFrame or None

Returns DataFrame or None DataFrame with the specified index or column labels removed or None if inplace=True.

Raises

KeyError

If any of the labels is not found in the selected axis.

See Also

DataFrame.loc : Label-location based indexer for selection by label.

DataFrame.dropna : Return DataFrame with labels on given axis omitted where (all or any) data are missing.

DataFrame.drop_duplicates : Return DataFrame with duplicate rows removed, optionally only considering certain columns.

Series.drop : Return Series with specified index labels removed.

Examples

```
>>> df = pd.DataFrame(np.arange(12).reshape(3, 4), columns=["A", "B", "C", "D"])
>>> df
```

```
   A  B  C  D
0  0  1  2  3
1  4  5  6  7
2  8  9 10 11
```

Drop columns

```
>>> df.drop(["B", "C"], axis=1)
```

```
   A  D
0  0  3
1  4  7
2  8 11
```

```
>>> df.drop(columns=["B", "C"])
```

```
   A  D
0  0  3
1  4  7
2  8 11
```

Drop a row by index

```
>>> df.drop([0, 1])
```

```
   A  B  C  D
2  8  9 10 11
```

Drop columns and/or rows of MultiIndex DataFrame

```
>>> midx = pd.MultiIndex(
...     levels=[["llama", "cow", "falcon"], ["speed", "weight", "length"]],
...     codes=[[0, 0, 0, 1, 1, 1, 2, 2, 2], [0, 1, 2, 0, 1, 2, 0, 1, 2]],
... )
>>> df = pd.DataFrame(
...     index=midx,
```

```

...     columns=["big", "small"],
...     data=[
...         [45, 30],
...         [200, 100],
...         [1.5, 1],
...         [30, 20],
...         [250, 150],
...         [1.5, 0.8],
...         [320, 250],
...         [1, 0.8],
...         [0.3, 0.2],
...     ],
... )
>>> df

```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
	length	1.5	1.0
cow	speed	30.0	20.0
	weight	250.0	150.0
	length	1.5	0.8
falcon	speed	320.0	250.0
	weight	1.0	0.8
	length	0.3	0.2

Drop a specific index combination from the MultiIndex DataFrame, i.e., drop the combination ``falcon`` and ``weight``, which deletes only the corresponding row

```

>>> df.drop(index=("falcon", "weight"))

```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
	length	1.5	1.0
cow	speed	30.0	20.0
	weight	250.0	150.0
	length	1.5	0.8
falcon	speed	320.0	250.0
	length	0.3	0.2

```

>>> df.drop(index="cow", columns="small")

```

		big
llama	speed	45.0
	weight	200.0
	length	1.5
falcon	speed	320.0
	weight	1.0
	length	0.3

```

>>> df.drop(index="length", level=1)

```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
cow	speed	30.0	20.0
	weight	250.0	150.0
falcon	speed	320.0	250.0
	weight	1.0	0.8

```

drop_duplicates(
    self,
    subset: Hashable | Iterable[Hashable] | None = None,
    *,
    keep: DropKeep = 'first',
    inplace: bool = False,
    ignore_index: bool = False
) -> DataFrame | None from pandas.core.frame.DataFrame
Return DataFrame with duplicate rows removed.

```

Considering certain columns is optional. Indexes, including time indexes are ignored.

Parameters

```

subset : column label or iterable of labels, optional
    Only consider certain columns for identifying duplicates, by
    default use all of the columns.
keep : {'first', 'last', ``False``}, default 'first'

```

Determines which duplicates (if any) to keep.

- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

`inplace : bool, default ``False```

Whether to modify the DataFrame rather than creating a new one.

`ignore_index : bool, default ``False```

If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

Returns

DataFrame or None

DataFrame with duplicates removed or None if ``inplace=True``.

See Also

`DataFrame.value_counts`: Count unique combinations of columns.

Notes

This method requires columns specified by ``subset`` to be of hashable type. Passing unhashable columns will raise a ``TypeError``.

Examples

Consider dataset containing ramen rating.

```
>>> df = pd.DataFrame(
...     {
...         "brand": ["Yum Yum", "Yum Yum", "Indomie", "Indomie", "Indomie"],
...         "style": ["cup", "cup", "cup", "pack", "pack"],
...         "rating": [4, 4, 3.5, 15, 5],
...     }
... )
>>> df
   brand style rating
0  Yum Yum  cup    4.0
1  Yum Yum  cup    4.0
2  Indomie  cup    3.5
3  Indomie pack   15.0
4  Indomie pack    5.0
```

By default, it removes duplicate rows based on all columns.

```
>>> df.drop_duplicates()
   brand style rating
0  Yum Yum  cup    4.0
2  Indomie  cup    3.5
3  Indomie pack   15.0
4  Indomie pack    5.0
```

To remove duplicates on specific column(s), use ``subset``.

```
>>> df.drop_duplicates(subset=["brand"])
   brand style rating
0  Yum Yum  cup    4.0
2  Indomie  cup    3.5
```

To remove duplicates and keep last occurrences, use ``keep``.

```
>>> df.drop_duplicates(subset=["brand", "style"], keep="last")
   brand style rating
1  Yum Yum  cup    4.0
2  Indomie  cup    3.5
4  Indomie pack    5.0
```

```
dropna(
    self,
    *,
    axis: Axis = 0,
    how: AnyAll | lib.NoDefault = <no_default>,
    thresh: int | lib.NoDefault = <no_default>,
    subset: IndexLabel | AnyArrayLike | None = None,
    inplace: bool = False,
    ignore_index: bool = False
```

```
) -> DataFrame | None from pandas.core.frame.DataFrame
Remove missing values.
```

See the :ref:`User Guide <missing\_data>` for more on which values are considered missing, and how to work with missing data.

Parameters

```
axis : {0 or 'index', 1 or 'columns'}, default 0
    Determine if rows or columns which contain missing values are
    removed.

    * 0, or 'index' : Drop rows which contain missing values.
    * 1, or 'columns' : Drop columns which contain missing value.

    Only a single axis is allowed.

how : {'any', 'all'}, default 'any'
    Determine if row or column is removed from DataFrame, when we have
    at least one NA or all NA.

    * 'any' : If any NA values are present, drop that row or column.
    * 'all' : If all values are NA, drop that row or column.

thresh : int, optional
    Require that many non-NA values. Cannot be combined with how.
subset : column label or iterable of labels, optional
    Labels along other axis to consider, e.g. if you are dropping rows
    these would be a list of columns to include.
inplace : bool, default False
    Whether to modify the DataFrame rather than creating a new one.
ignore_index : bool, default ``False``
    If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

.. versionadded:: 2.0.0
```

Returns

```
DataFrame or None
    DataFrame with NA entries dropped from it or None if ``inplace=True``.
```

See Also

```
DataFrame.isna: Indicate missing values.
DataFrame.notna : Indicate existing (non-missing) values.
DataFrame.fillna : Replace missing values.
Series.dropna : Drop missing values.
Index.dropna : Drop missing indices.
```

Examples

```
>>> df = pd.DataFrame(
...     {
...         "name": ["Alfred", "Batman", "Catwoman"],
...         "toy": [np.nan, "Batmobile", "Bullwhip"],
...         "born": [pd.NaT, pd.Timestamp("1940-04-25"), pd.NaT],
...     }
... )
>>> df
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

Drop the rows where at least one element is missing.

```
>>> df.dropna()
   name      toy      born
1  Batman  Batmobile  1940-04-25
```

Drop the columns where at least one element is missing.

```
>>> df.dropna(axis="columns")
   name
0  Alfred
1  Batman
2  Catwoman
```

Drop the rows where all elements are missing.

```
>>> df.dropna(how="all")
   name      toy      born
0  Alfred    NaN     NaT
1  Batman  Batmobile 1940-04-25
2 Catwoman  Bullwhip     NaT
```

Keep only the rows with at least 2 non-NA values.

```
>>> df.dropna(thresh=2)
   name      toy      born
1  Batman  Batmobile 1940-04-25
2 Catwoman  Bullwhip     NaT
```

Define in which columns to look for missing values.

```
>>> df.dropna(subset=["name", "toy"])
   name      toy      born
1  Batman  Batmobile 1940-04-25
2 Catwoman  Bullwhip     NaT
```

```
duplicated(
    self,
    subset: Hashable | Iterable[Hashable] | None = None,
    keep: DropKeep = 'first'
) -> Series from pandas.core.frame.DataFrame
Return boolean Series denoting duplicate rows.
```

Considering certain columns is optional.

Parameters

subset : column label or iterable of labels, optional
Only consider certain columns for identifying duplicates, by default use all of the columns.
keep : {'first', 'last', False}, default 'first'
Determines which duplicates (if any) to mark.

- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

Returns

Series

Boolean series for each duplicated rows.

See Also

Index.duplicated : Equivalent method on index.
Series.duplicated : Equivalent method on Series.
Series.drop_duplicates : Remove duplicate values from Series.
DataFrame.drop_duplicates : Remove duplicate values from DataFrame.

Examples

Consider dataset containing ramen rating.

```
>>> df = pd.DataFrame(
...     {
...         "brand": ["Yum Yum", "Yum Yum", "Indomie", "Indomie", "Indomie"],
...         "style": ["cup", "cup", "cup", "pack", "pack"],
...         "rating": [4, 4, 3.5, 15, 5],
...     }
... )
>>> df
   brand style  rating
0  Yum Yum   cup    4.0
1  Yum Yum   cup    4.0
2  Indomie   cup    3.5
3  Indomie  pack   15.0
4  Indomie  pack    5.0
```

By default, for each set of duplicated values, the first occurrence is set on False and all others on True.

```
>>> df.duplicated()
0    False
1     True
2    False
3    False
4    False
dtype: bool
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True.

```
>>> df.duplicated(keep="last")
0     True
1    False
2    False
3    False
4    False
dtype: bool
```

By setting ``keep`` on False, all duplicates are True.

```
>>> df.duplicated(keep=False)
0     True
1     True
2    False
3    False
4    False
dtype: bool
```

To find duplicates on specific column(s), use ``subset``.

```
>>> df.duplicated(subset=["brand"])
0    False
1     True
2    False
3     True
4     True
dtype: bool
```

`eq(self, other, axis: Axis = 'columns', level=None) -> DataFrame` from `pandas.core.frame.DataFrame`
Get Not equal to of dataframe and other, element-wise (binary operator ``eq``).

Among flexible wrappers (``eq``, ``ne``, ``le``, ``lt``, ``ge``, ``gt``) to comparison operators.

Equivalent to ``==``, ``!=``, ``<=``, ``<``, ``>=``, ``>`` with support to choose axis (rows or columns) and level for comparison.

Parameters

```
other : scalar, sequence, Series, or DataFrame
        Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
        Whether to compare by the index (0 or 'index') or columns
        (1 or 'columns').
level : int or label
        Broadcast across a level, matching Index values on the passed
        MultiIndex level.
```

Returns

```
-----
DataFrame of bool
        Result of the comparison.
```

See Also

```
-----
DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality
               or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than
               inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality
               or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than
               inequality elementwise.
```

Notes

Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples

```
>>> df = pd.DataFrame(
...     {"cost": [250, 150, 100], "revenue": [100, 250, 300]},
...     index=["A", "B", "C"],
... )
>>> df
```

	cost	revenue
A	250	100
B	150	250
C	100	300

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
```

	cost	revenue
A	False	True
B	False	False
C	True	False

```
>>> df.eq(100)
```

	cost	revenue
A	False	True
B	False	False
C	True	False

When `other` is a `:class:`Series``, the columns of a `DataFrame` are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
```

	cost	revenue
A	True	True
B	True	False
C	False	True

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis="index")
```

	cost	revenue
A	True	False
B	True	True
C	True	True
D	True	True

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
```

	cost	revenue
A	True	True
B	False	False
C	False	False

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis="index")
```

	cost	revenue
A	True	False
B	False	True
C	True	False

Compare to a `DataFrame` of different shape.

```
>>> other = pd.DataFrame(
...     {"revenue": [300, 250, 100, 150]}, index=["A", "B", "C", "D"]
... )
>>> other
```

	revenue
A	300
B	250
C	100

D 150

```
>>> df.gt(other)
      cost  revenue
A  False   False
B  False   False
C  False    True
D  False   False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame(
...     {
...         "cost": [250, 150, 100, 150, 300, 220],
...         "revenue": [100, 250, 300, 200, 175, 225],
...     },
...     index=[
...         ["Q1", "Q1", "Q1", "Q2", "Q2", "Q2"],
...         ["A", "B", "C", "A", "B", "C"],
...     ],
... )
>>> df_multindex
      cost  revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225
```

```
>>> df.le(df_multindex, level=1)
      cost  revenue
Q1 A    True     True
   B    True     True
   C    True     True
Q2 A   False     True
   B    True    False
   C    True    False
```

`eval(self, expr: str, *, inplace: bool = False, **kwargs) -> Any | None` from `pandas.core.frame`
Evaluate a string describing operations on DataFrame columns.

.. warning::

This method can run arbitrary code which can make you vulnerable to code injection if you pass user input to this function.

Operates on columns only, not specific rows or elements. This allows ``eval`` to run arbitrary code, which can make you vulnerable to code injection if you pass user input to this function.

Parameters

`expr` : str

The expression string to evaluate.

You can refer to variables in the environment by prefixing them with an '@' character like ``@a + b``.

You can refer to column names that are not valid Python variable names by surrounding them in backticks. Thus, column names containing spaces or punctuation (besides underscores) or starting with digits must be surrounded by backticks. (For example, a column named "Area (cm^2)" would be referenced as ``Area (cm^2)``). Column names which are Python keywords (like "if", "for", "import", etc) cannot be used.

For example, if one of your columns is called ``a`` and you want to sum it with ``b``, your query should be ``a` + b``.

See the documentation for `:func:`eval`` for full details of supported operations and functions in the expression string.

`inplace` : bool, default False

If the expression contains an assignment, whether to perform the operation inplace and mutate the existing DataFrame. Otherwise, a new DataFrame is returned.

`**kwargs`

See the documentation for :func:`eval` for complete details on the keyword arguments accepted by :meth:`~pandas.DataFrame.eval`.

Returns

ndarray, scalar, pandas object, or None
The result of the evaluation or None if ``inplace=True``.

See Also

`DataFrame.query` : Evaluates a boolean expression to query the columns of a frame.
`DataFrame.assign` : Can evaluate an expression or function to create new values for a column.
`eval` : Evaluate a Python expression as a string using various backends.

Notes

For more details see the API documentation for :func:`~eval`. For detailed examples see :ref:`enhancing performance with eval <enhancingperf.eval>`.

Examples

```
>>> df = pd.DataFrame(
...     {"A": range(1, 6), "B": range(10, 0, -2), "C&C": range(10, 5, -1)}
... )
>>> df
   A  B  C&C
0  1 10   10
1  2  8    9
2  3  6    8
3  4  4    7
4  5  2    6
>>> df.eval("A + B")
0    11
1    10
2     9
3     8
4     7
dtype: int64
```

Assignment is allowed though by default the original DataFrame is not modified.

```
>>> df.eval("D = A + B")
   A  B  C&C  D
0  1 10   10 11
1  2  8    9 10
2  3  6    8  9
3  4  4    7  8
4  5  2    6  7
>>> df
   A  B  C&C
0  1 10   10
1  2  8    9
2  3  6    8
3  4  4    7
4  5  2    6
```

Multiple columns can be assigned to using multi-line expressions:

```
>>> df.eval(
...     '''
...     D = A + B
...     E = A - B
...     '''
... )
   A  B  C&C  D  E
0  1 10   10 11 -9
1  2  8    9 10 -6
2  3  6    8  9 -3
3  4  4    7  8  0
4  5  2    6  7  3
```

For columns with spaces or other disallowed characters in their name, you can use backtick quoting.

```
>>> df.eval("B * `C&C`")
0    100
1     72
2     48
3     28
4     12
dtype: int64
```

Local variables shall be explicitly referenced using ``@`` character in front of the name:

```
>>> local_var = 2
>>> df.eval("@local_var * A")
0     2
1     4
2     6
3     8
4    10
Name: A, dtype: int64
```

`explode(self, column: IndexLabel, ignore_index: bool = False) -> DataFrame` from `pandas.core.`
Transform each element of a list-like to a row, replicating index values.

Parameters

`column` : IndexLabel
Column(s) to explode.
For multiple columns, specify a non-empty list with each element be str or tuple, and all specified columns their list-like data on same row of the frame must have matching length.

`ignore_index` : bool, default False
If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

`DataFrame`
Exploded lists to rows of the subset columns;
index will be duplicated for these rows.

Raises

`ValueError` :
* If columns of the frame are not unique.
* If specified columns to explode is empty list.
* If specified columns to explode have not matching count of elements rowwise in the frame.

See Also

`DataFrame.unstack` : Pivot a level of the (necessarily hierarchical) index labels.
`DataFrame.melt` : Unpivot a `DataFrame` from wide format to long format.
`Series.explode` : Explode a `DataFrame` from list-like columns to long format.

Notes

This routine will explode list-likes including lists, tuples, sets, Series, and `np.ndarray`. The result dtype of the subset rows will be object. Scalars will be returned unchanged, and empty list-likes will result in a `np.nan` for that row. In addition, the ordering of rows in the output will be non-deterministic when exploding sets.

Reference :`ref:`the user guide <reshaping.explode>`` for more examples.

Examples

>>> df = pd.DataFrame(
... {
... "A": [[0, 1, 2], "foo", [], [3, 4]],
... "B": 1,
... "C": ["a", "b", "c"], np.nan, [], ["d", "e"]],
...)

```
>>> df
   A B      C
0 [0, 1, 2] 1 [a, b, c]
1      foo 1      NaN
2      [] 1      []
3 [3, 4] 1 [d, e]
```

Single-column explode.

```
>>> df.explode("A")
   A B      C
0  0 1 [a, b, c]
0  1 1 [a, b, c]
0  2 1 [a, b, c]
1 foo 1      NaN
2 NaN 1      []
3  3 1 [d, e]
3  4 1 [d, e]
```

Multi-column explode.

```
>>> df.explode(list("AC"))
   A B C
0  0 1 a
0  1 1 b
0  2 1 c
1 foo 1 NaN
2 NaN 1 NaN
3  3 1 d
3  4 1 e
```

`floordiv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from
Get Integer division of dataframe and other, element-wise (binary operator `'floordiv'`).

Equivalent to `dataframe // other`, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, `'rfloordiv'`.

Among flexible wrappers (`'add'`, `'sub'`, `'mul'`, `'div'`, `'floordiv'`, `'mod'`, `'pow'`) to arithmetic operators: `'+'`, `'-'`, `'*'`, `'/'`, `'//'`, `'%'`, `'**'`.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```

-----
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
      angles  degrees
circle      0     360
triangle    3     180
rectangle   4     360

```

Add a scalar with operator version which return the same results.

```

>>> df + 1
      angles  degrees
circle      1     361
triangle    4     181
rectangle   5     361

```

```

>>> df.add(1)
      angles  degrees
circle      1     361
triangle    4     181
rectangle   5     361

```

Divide by constant with reverse version.

```

>>> df.div(10)
      angles  degrees
circle    0.0     36.0
triangle  0.3     18.0
rectangle 0.4     36.0

```

```

>>> df.rdiv(10)
      angles  degrees
circle    inf  0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778

```

Subtract a list and Series by axis with operator version.

```

>>> df - [1, 2]
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358

```

```

>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358

```

```

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1     359
triangle    2     179
rectangle   3     359

```

Multiply a dictionary by axis.

```

>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0     720
triangle    0     360
rectangle   0     720

```

```

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6     360
rectangle    12    1080

```

Multiply a DataFrame of different shape with operator version.

```

>>> other = pd.DataFrame({'angles': [0, 3, 4]},

```

```
... index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle    3
rectangle   4
```

```
>>> df * other
      angles  degrees
circle      0     NaN
triangle    9     NaN
rectangle   16     NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0     0.0
triangle    9     0.0
rectangle   16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
```

```
>>> df_multindex
      angles  degrees
A circle      0     360
  triangle    3     180
  rectangle    4     360
B square      4     360
  pentagon    5     540
  hexagon     6     720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle  1.0     1.0
  rectangle  1.0     1.0
B square    0.0     0.0
  pentagon  0.0     0.0
  hexagon   0.0     0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
```

```
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

ge(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame
Get Greater than or equal to of dataframe and other, element-wise (binary operator `ge`)

Among flexible wrappers (`eq`, `ne`, `le`, `lt`, `ge`, `gt`) to comparison operators.

Equivalent to `==`, `!=`, `<=`, `<`, `>=`, `>` with support to choose axis (rows or columns) and level for comparison.

Parameters

```
other : scalar, sequence, Series, or DataFrame
    Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
    Whether to compare by the index (0 or 'index') or columns
    (1 or 'columns').
level : int or label
    Broadcast across a level, matching Index values on the passed
    MultiIndex level.
```

Returns

```
DataFrame of bool
    Result of the comparison.
```

See Also

DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality
or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than
inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality
or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than
inequality elementwise.

Notes

Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples

>>> df = pd.DataFrame({'cost': [250, 150, 100],
... 'revenue': [100, 250, 300]},
... index=['A', 'B', 'C'])
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300

Comparison with a scalar, using either the operator or method:

```
>>> df == 100  
   cost  revenue  
A  False      True  
B  False     False  
C   True     False
```

```
>>> df.eq(100)  
   cost  revenue  
A  False      True  
B  False     False  
C   True     False
```

When `other` is a :class:`Series`, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])  
   cost  revenue  
A   True      True  
B   True     False  
C  False      True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')  
   cost  revenue  
A   True     False  
B   True      True  
C   True      True  
D   True      True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]  
   cost  revenue  
A   True      True  
B  False     False  
C  False     False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')  
   cost  revenue  
A   True     False  
B  False      True
```

```
C    True    False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                        index=['A', 'B', 'C', 'D'])
```

```
>>> other
   revenue
A        300
B        250
C        100
D        150
```

```
>>> df.gt(other)
   cost  revenue
A  False   False
B  False   False
C  False    True
D  False   False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
...                               'revenue': [100, 250, 300, 200, 175, 225]},
...                              index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                     ['A', 'B', 'C', 'A', 'B', 'C']])
```

```
>>> df_multindex
   cost  revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225
```

```
>>> df.le(df_multindex, level=1)
   cost  revenue
Q1 A    True    True
   B    True    True
   C    True    True
Q2 A   False    True
   B    True   False
   C    True   False
```

```
groupby(
    self,
    by=None,
    level: IndexLabel | None = None,
    *,
    as_index: bool = True,
    sort: bool = True,
    group_keys: bool = True,
    observed: bool = True,
    dropna: bool = True
) -> DataFrameGroupBy from pandas.core.frame.DataFrame
Group DataFrame using a mapper or by a Series of columns.
```

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters

by : mapping, function, label, pd.Grouper or list of such
Used to determine the groups for the groupby.
If ``by`` is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see ``.align()`` method). If a list or ndarray of length equal to the number of rows is passed (see the `groupby user guide <https://pandas.pydata.org/pandas-docs/stable/user_guide/groupby.html#splitting-an-object>), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in ``self``.
Notice that a tuple is interpreted as a (single) key.

level : int, level name, or sequence of such, default None
If the axis is a MultiIndex (hierarchical), group by a particular

level or levels. Do not specify both ``by`` and ``level``.

as_index : bool, default True
 Return object with group labels as the index. Only relevant for DataFrame input. as_index=False is effectively "SQL-style" grouped output. This argument has no effect on filtrations (see the `filtrations in the user guide`_<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>_), such as ``head()``, ``tail()``, ``nth()`` and in transformations (see the `transformations in the user guide`_<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>_).

sort : bool, default True
 Sort group keys. Get better performance by turning this off. Note this does not influence the order of observations within each group. Groupby preserves the order of rows within each group. If False, the groups will appear in the same order as they did in the original DataFrame. This argument has no effect on filtrations (see the `filtrations in the user guide`_<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>_), such as ``head()``, ``tail()``, ``nth()`` and in transformations (see the `transformations in the user guide`_<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>_).

.. versionchanged:: 2.0.0

Specifying ``sort=False`` with an ordered categorical grouper will no longer sort the values.

group_keys : bool, default True
 When calling apply and the ``by`` argument produces a like-indexed (i.e. :ref:`a transform <groupby.transform>`) result, add group keys to index to identify pieces. By default group keys are not included when the result's index (and column) labels match the inputs, and are included otherwise.

.. versionchanged:: 2.0.0

``group\_keys`` now defaults to ``True``.

observed : bool, default True
 This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical groupers.

.. versionchanged:: 3.0.0

The default value is now ``True``.

dropna : bool, default True
 If True, and if group keys contain NA values, NA values together with row/column will be dropped. If False, NA values will also be treated as the key in groups.

Returns

 pandas.api.typing.DataFrameGroupBy
 Returns a groupby object that contains information about the groups.

See Also

 resample : Convenience method for frequency conversion and resampling of time series.

Notes

 See the `user guide`_<<https://pandas.pydata.org/pandas-docs/stable/groupby.html>>_ for more detailed usage and examples, including splitting an object into groups, iterating through groups, selecting a group, aggregation, and more.

The implementation of groupby is hash-based, meaning in particular that objects that compare as equal will be considered to be in the same group. An exception to this is that pandas has special handling of NA values: any NA values will be collapsed to a single group, regardless of how they compare. See the user guide linked above for more details.

Examples

```

-----
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
...         "Max Speed": [380.0, 370.0, 24.0, 26.0],
...     }
... )
>>> df
   Animal  Max Speed
0  Falcon      380.0
1  Falcon      370.0
2  Parrot       24.0
3  Parrot       26.0
>>> df.groupby(["Animal"]).mean()
           Max Speed
Animal
Falcon      375.0
Parrot      25.0

**Hierarchical Indexes**

We can groupby different levels of a hierarchical index
using the `level` parameter:

>>> arrays = [
...     ["Falcon", "Falcon", "Parrot", "Parrot"],
...     ["Captive", "Wild", "Captive", "Wild"],
... ]
>>> index = pd.MultiIndex.from_arrays(arrays, names=("Animal", "Type"))
>>> df = pd.DataFrame({"Max Speed": [390.0, 350.0, 30.0, 20.0]}, index=index)
>>> df
           Max Speed
Animal Type
Falcon Captive      390.0
       Wild        350.0
Parrot Captive       30.0
       Wild         20.0
>>> df.groupby(level=0).mean()
           Max Speed
Animal
Falcon      370.0
Parrot      25.0
>>> df.groupby(level="Type").mean()
           Max Speed
Type
Captive      210.0
Wild        185.0

We can also choose to include NA in group keys or not by setting
`dropna` parameter, the default setting is `True`.

>>> arr = [[1, 2, 3], [1, None, 4], [2, 1, 3], [1, 2, 2]]
>>> df = pd.DataFrame(arr, columns=["a", "b", "c"])

>>> df.groupby(by=["b"]).sum()
           a    c
b
1.0 2    3
2.0 2    5

>>> df.groupby(by=["b"], dropna=False).sum()
           a    c
b
1.0 2    3
2.0 2    5
NaN 1    4

>>> arr = [{"a", 12, 12}, [None, 12.3, 33.0], ["b", 12.3, 123], [{"a", 1, 1}]
>>> df = pd.DataFrame(arr, columns=["a", "b", "c"])

>>> df.groupby(by="a").sum()
           b    c
a
a    13.0   13.0
b    12.3  123.0

>>> df.groupby(by="a", dropna=False).sum()

```

	b	c
a		
a	13.0	13.0
b	12.3	123.0
NaN	12.3	33.0

When using `df.apply()`, use `group_keys` to include or exclude the group keys. The `group_keys` argument defaults to `True` (include).

```
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
...         "Max Speed": [380.0, 370.0, 24.0, 26.0],
...     }
... )
>>> df.groupby("Animal", group_keys=True)[["Max Speed"]].apply(lambda x: x)
Max Speed
Animal
Falcon 0      380.0
       1      370.0
Parrot 2       24.0
       3       26.0

>>> df.groupby("Animal", group_keys=False)[["Max Speed"]].apply(lambda x: x)
Max Speed
0      380.0
1      370.0
2       24.0
3       26.0
```

`gt(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame`
Get Greater than of dataframe and other, element-wise (binary operator `'gt'`).

Among flexible wrappers (`'eq'`, `'ne'`, `'le'`, `'lt'`, `'ge'`, `'gt'`) to comparison operators.

Equivalent to `'=='`, `'!='`, `'<='`, `'<'`, `'>='`, `'>'` with support to choose axis (rows or columns) and level for comparison.

Parameters

`other` : scalar, sequence, Series, or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}, default 'columns'
Whether to compare by the index (0 or 'index') or columns (1 or 'columns').
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns

DataFrame of bool
Result of the comparison.

See Also

`DataFrame.eq` : Compare DataFrames for equality elementwise.
`DataFrame.ne` : Compare DataFrames for inequality elementwise.
`DataFrame.le` : Compare DataFrames for less than inequality or equality elementwise.
`DataFrame.lt` : Compare DataFrames for strictly less than inequality elementwise.
`DataFrame.ge` : Compare DataFrames for greater than inequality or equality elementwise.
`DataFrame.gt` : Compare DataFrames for strictly greater than inequality elementwise.

Notes

Mismatched indices will be unioned together.
`'NaN'` values are considered different (i.e. `'NaN' != 'NaN'`).

Examples

```
>>> df = pd.DataFrame({'cost': [250, 150, 100],
...                     'revenue': [100, 250, 300]},
```

```

...                                     index=['A', 'B', 'C'])
>>> df
   cost  revenue
A    250     100
B    150     250
C    100     300

```

Comparison with a scalar, using either the operator or method:

```

>>> df == 100
   cost  revenue
A  False     True
B  False     False
C   True     False

>>> df.eq(100)
   cost  revenue
A  False     True
B  False     False
C   True     False

```

When `other` is a `:class:`Series``, the columns of a `DataFrame` are aligned with the index of `other` and broadcast:

```

>>> df != pd.Series([100, 250], index=["cost", "revenue"])
   cost  revenue
A   True     True
B   True     False
C  False     True

```

Use the method to control the broadcast axis:

```

>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
   cost  revenue
A   True     False
B   True     True
C   True     True
D   True     True

```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```

>>> df == [250, 100]
   cost  revenue
A   True     True
B  False     False
C  False     False

```

Use the method to control the axis:

```

>>> df.eq([250, 250, 100], axis='index')
   cost  revenue
A   True     False
B  False     True
C   True     False

```

Compare to a `DataFrame` of different shape.

```

>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                       index=['A', 'B', 'C', 'D'])
>>> other
   revenue
A       300
B       250
C       100
D       150

>>> df.gt(other)
   cost  revenue
A  False     False
B  False     False
C  False     True
D  False     False

```

Compare to a `MultiIndex` by level.

```

>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],

```

```

...                                     'revenue': [100, 250, 300, 200, 175, 225]],
...                                     index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                             ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
      cost revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225

>>> df.le(df_multindex, level=1)
      cost revenue
Q1 A   True     True
   B   True     True
   C   True     True
Q2 A  False     True
   B   True     False
   C   True     False

hist = hist_frame(
    data: DataFrame,
    column: IndexLabel | None = None,
    by=None,
    grid: bool = True,
    xlabelsize: int | None = None,
    xrot: float | None = None,
    ylabelsize: int | None = None,
    yrot: float | None = None,
    ax=None,
    sharex: bool = False,
    sharey: bool = False,
    figsize: tuple[int, int] | None = None,
    layout: tuple[int, int] | None = None,
    bins: int | Sequence[int] = 10,
    backend: str | None = None,
    legend: bool = False,
    **kwargs
) from pandas.plotting
    Make a histogram of the DataFrame's columns.

A `histogram`_ is a representation of the distribution of data.
This function calls :meth:`matplotlib.pyplot.hist`, on each series in
the DataFrame, resulting in one histogram per column.

.. _histogram: https://en.wikipedia.org/wiki/Histogram

Parameters
-----
data : DataFrame
    The pandas object holding the data.
column : str or sequence, optional
    If passed, will be used to limit data to a subset of columns.
by : object, optional
    If passed, then used to form histograms for separate groups.
grid : bool, default True
    Whether to show axis grid lines.
xlabelsize : int, default None
    If specified changes the x-axis label size.
xrot : float, default None
    Rotation of x axis labels. For example, a value of 90 displays the
    x labels rotated 90 degrees clockwise.
ylabelsize : int, default None
    If specified changes the y-axis label size.
yrot : float, default None
    Rotation of y axis labels. For example, a value of 90 displays the
    y labels rotated 90 degrees clockwise.
ax : Matplotlib axes object, default None
    The axes to plot the histogram on.
sharex : bool, default True if ax is None else False
    In case subplots=True, share x axis and set some x axis labels to
    invisible; defaults to True if ax is None otherwise False if an ax
    is passed in.
    Note that passing in both an ax and sharex=True will alter all x axis
    labels for all subplots in a figure.
sharey : bool, default False

```

In case subplots=True, share y axis and set some y axis labels to invisible.

figsize : tuple, optional
The size in inches of the figure to create. Uses the value in ``matplotlib.rcParams`` by default.

layout : tuple, optional
Tuple of (rows, columns) for the layout of the histograms.

bins : int or sequence, default 10
Number of histogram bins to be used. If an integer is given, bins + 1 bin edges are calculated and returned. If bins is a sequence, gives bin edges, including left edge of first bin and right edge of last bin. In this case, bins is returned unmodified.

backend : str, default None
Backend to use instead of the backend specified in the option ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to specify the ``plotting.backend`` for the whole session, set ``pd.options.plotting.backend``.

legend : bool, default False
Whether to show the legend.

****kwargs**
All other plotting keyword arguments to be passed to `:meth:`matplotlib.pyplot.hist``.

Returns

np.ndarray
2D NumPy Array of `:class:`matplotlib.axes.Axes``.

See Also

`matplotlib.pyplot.hist` : Plot a histogram using matplotlib.

Examples

This example draws a histogram based on the length and width of some animals, displayed in three bins

```
.. plot::
    :context: close-figs

    >>> data = {
    ...     "length": [1.5, 0.5, 1.2, 0.9, 3],
    ...     "width": [0.7, 0.2, 0.15, 0.2, 1.1],
    ... }
    >>> index = ["pig", "rabbit", "duck", "chicken", "horse"]
    >>> df = pd.DataFrame(data, index=index)
    >>> hist = df.hist(bins=3)
```

`idxmax(self, axis: Axis = 0, skipna: bool = True, numeric_only: bool = False) -> Series from`
Return index of first occurrence of maximum over requested axis.

NA/null values are excluded.

Parameters

axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.

skipna : bool, default True
Exclude NA/null values. If the entire DataFrame is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

numeric_only : bool, default False
Include only ``float``, ``int`` or ``boolean`` data.

Returns

Series
Indexes of maxima along the specified axis.

Raises

ValueError
* If the row/column is empty

See Also

Series.idxmax : Return index of the maximum element.

Notes

This method is the DataFrame version of ``ndarray.argmax``.

Examples

Consider a dataset containing food consumption in Argentina.

```
>>> df = pd.DataFrame(
...     {
...         "consumption": [10.51, 103.11, 55.48],
...         "co2_emissions": [37.2, 19.66, 1712],
...     },
...     index=["Pork", "Wheat Products", "Beef"],
... )
```

```
>>> df
          consumption  co2_emissions
Pork              10.51             37.20
Wheat Products    103.11             19.66
Beef              55.48            1712.00
```

By default, it returns the index for the maximum value in each column.

```
>>> df.idxmax()
consumption      Wheat Products
co2_emissions           Beef
dtype: str
```

To return the index for the maximum value in each row, use ``axis="columns"``.

```
>>> df.idxmax(axis="columns")
Pork      co2_emissions
Wheat Products      consumption
Beef      co2_emissions
dtype: str
```

idxmin(self, axis: Axis = 0, skipna: bool = True, numeric_only: bool = False) -> Series from
Return index of first occurrence of minimum over requested axis.

NA/null values are excluded.

Parameters

axis : {{0 or 'index', 1 or 'columns'}}, default 0

The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.

skipna : bool, default True

Exclude NA/null values. If the entire DataFrame is NA,
or if ``skipna=False`` and there is an NA value, this method
will raise a ``ValueError``.

numeric_only : bool, default False

Include only ``float``, ``int`` or ``boolean`` data.

Returns

Series

Indexes of minima along the specified axis.

Raises

ValueError

* If the row/column is empty

See Also

Series.idxmin : Return index of the minimum element.

Notes

This method is the DataFrame version of ``ndarray.argmin``.

Examples

Consider a dataset containing food consumption in Argentina.

```
>>> df = pd.DataFrame(  
...     {  
...         "consumption": [10.51, 103.11, 55.48],  
...         "co2_emissions": [37.2, 19.66, 1712],  
...     },  
...     index=["Pork", "Wheat Products", "Beef"],  
... )
```

```
>>> df  
          consumption  co2_emissions  
Pork              10.51             37.20  
Wheat Products    103.11             19.66  
Beef              55.48            1712.00
```

By default, it returns the index for the minimum value in each column.

```
>>> df.idxmin()  
consumption      Pork  
co2_emissions    Wheat Products  
dtype: str
```

To return the index for the minimum value in each row, use ``axis="columns"``.

```
>>> df.idxmin(axis="columns")  
Pork      consumption  
Wheat Products  co2_emissions  
Beef      consumption  
dtype: str
```

```
info(  
    self,  
    verbose: bool | None = None,  
    buf: WriteBuffer[str] | None = None,  
    max_cols: int | None = None,  
    memory_usage: bool | str | None = None,  
    show_counts: bool | None = None  
) -> None from pandas.core.frame.DataFrame  
Print a concise summary of a DataFrame.
```

This method prints information about a DataFrame including the index dtype and columns, non-NA values and memory usage.

Parameters

verbose : bool, optional
Whether to print the full summary. By default, the setting in ``pandas.options.display.max\_info\_columns`` is followed.
buf : writable buffer, defaults to sys.stdout
Where to send the output. By default, the output is printed to sys.stdout. Pass a writable buffer if you need to further process the output.
max_cols : int, optional
When to switch from the verbose to the truncated output. If the DataFrame has more than `max\_cols` columns, the truncated output is used. By default, the setting in ``pandas.options.display.max\_info\_columns`` is used.
memory_usage : bool, str, optional
Specifies whether total memory usage of the DataFrame elements (including the index) should be displayed. By default, this follows the ``pandas.options.display.memory\_usage`` setting.

True always show memory usage. False never shows memory usage. A value of 'deep' is equivalent to "True with deep introspection". Memory usage is shown in human-readable units (base-2 representation). Without deep introspection a memory estimation is made based in column dtype and number of rows assuming values consume the same memory amount for corresponding dtypes. With deep memory introspection, a real memory usage calculation is performed at the cost of computational resources. See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.
show_counts : bool, optional
Whether to show the non-null counts. By default, this is shown only if the DataFrame is smaller than ``pandas.options.display.max\_info\_rows`` and


```pandas.options.display.max_info_columns```. A value of True always shows the counts, and False never shows the counts.

#### Returns

None

This method prints a summary of a DataFrame and returns None.

#### See Also

`DataFrame.describe`: Generate descriptive statistics of DataFrame columns.

`DataFrame.memory_usage`: Memory usage of DataFrame columns.

#### Examples

```
>>> int_values = [1, 2, 3, 4, 5]
>>> text_values = ["alpha", "beta", "gamma", "delta", "epsilon"]
>>> float_values = [0.0, 0.25, 0.5, 0.75, 1.0]
>>> df = pd.DataFrame(
... {
... "int_col": int_values,
... "text_col": text_values,
... "float_col": float_values,
... }
...)
>>> df
 int_col text_col float_col
0 1 alpha 0.00
1 2 beta 0.25
2 3 gamma 0.50
3 4 delta 0.75
4 5 epsilon 1.00
```

Prints information of all columns:

```
>>> df.info(verbose=True)
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- -
0 int_col 5 non-null int64
1 text_col 5 non-null str
2 float_col 5 non-null float64
dtypes: float64(1), int64(1), str(1)
memory usage: 278.0 bytes
```

Prints a summary of columns count and its dtypes but not per column information:

```
>>> df.info(verbose=False)
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Columns: 3 entries, int_col to float_col
dtypes: float64(1), int64(1), str(1)
memory usage: 278.0 bytes
```

Pipe output of `DataFrame.info` to buffer instead of `sys.stdout`, get buffer content and writes to a text file:

```
>>> import io
>>> buffer = io.StringIO()
>>> df.info(buf=buffer)
>>> s = buffer.getvalue()
>>> with open("df_info.txt", "w", encoding="utf-8") as f: # doctest: +SKIP
... f.write(s)
260
```

The `memory_usage` parameter allows deep introspection mode, specially useful for big DataFrames and fine-tune memory optimization:

```
>>> random_strings_array = np.random.choice(["a", "b", "c"], 10**6)
>>> df = pd.DataFrame(
... {
... "column_1": np.random.choice(["a", "b", "c"], 10**6),
... "column_2": np.random.choice(["a", "b", "c"], 10**6),
... })
```

```
... "column_3": np.random.choice(["a", "b", "c"], 10**6),
... }
...)
```

```
>>> df.info()
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- ---
0 column_1 1000000 non-null str
1 column_2 1000000 non-null str
2 column_3 1000000 non-null str
dtypes: str(3)
memory usage: 25.7 MB
```

```
>>> df.info(memory_usage="deep")
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- ---
0 column_1 1000000 non-null str
1 column_2 1000000 non-null str
2 column_3 1000000 non-null str
dtypes: str(3)
memory usage: 25.7 MB
```

```
insert(
 self,
 loc: int,
 column: Hashable,
 value: object,
 allow_duplicates: bool | lib.NoDefault = <no_default>
) -> None from pandas.core.frame.DataFrame
Insert column into DataFrame at specified location.
```

Raises a ValueError if `column` is already contained in the DataFrame, unless `allow\_duplicates` is set to True.

#### Parameters

```

loc : int
 Insertion index. Must verify 0 <= loc <= len(columns).
column : str, number, or hashable object
 Label of the inserted column.
value : Scalar, Series, or array-like
 Content of the inserted column.
allow_duplicates : bool, optional, default lib.no_default
 Allow duplicate column labels to be created.
```

#### See Also

```

Index.insert : Insert new item by index.
```

#### Examples

```

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df
 col1 col2
0 1 3
1 2 4
>>> df.insert(1, "newcol", [99, 99])
>>> df
 col1 newcol col2
0 1 99 3
1 2 99 4
>>> df.insert(0, "col1", [100, 100], allow_duplicates=True)
>>> df
 col1 col1 newcol col2
0 100 1 99 3
1 100 2 99 4
```

Notice that pandas uses index alignment in case of `value` from type `Series`:

```
>>> df.insert(0, "col0", pd.Series([5, 6], index=[1, 2]))
>>> df
 col0 col1 col1 newcol col2
0 5 1 1 99 3
1 6 2 2 99 4
```

0	NaN	100	1	99	3
1	5.0	100	2	99	4

`isetitem(self, loc, value)` -> None from `pandas.core.frame.DataFrame`  
Set the given value in the column with position `loc`.

This is a positional analogue to `__setitem__`.

Parameters

-----  
`loc` : int or sequence of ints  
 Index position for the column.  
`value` : scalar or arraylike  
 Value(s) for the column.

See Also

-----  
`DataFrame.iloc` : Purely integer-location based indexing for selection by position.

Notes

-----  
`frame.isetitem(loc, value)` is an in-place method as it will modify the DataFrame in place (not returning a new object). In contrast to `frame.iloc[:, i] = value` which will try to update the existing values in place, `frame.isetitem(loc, value)` will not update the values of the column itself in place, it will instead insert a new array.

In cases where `frame.columns` is unique, this is equivalent to `frame[frame.columns[i]] = value`.

Examples

-----  

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.isetitem(1, [5, 6])
>>> df
```

	A	B
0	1	5
1	2	6

`isin(self, values: Series | DataFrame | Sequence | Mapping)` -> DataFrame from `pandas.core.frame.DataFrame`  
Whether each element in the DataFrame is contained in values.

Parameters

-----  
`values` : iterable, Series, DataFrame or dict  
 The result will only be true at a location if all the labels match. If `values` is a Series, that's the index. If `values` is a dict, the keys must be the column names, which must match. If `values` is a DataFrame, then both the index and column labels must match.

Returns

-----  
 DataFrame  
 DataFrame of booleans showing whether each element in the DataFrame is contained in values.

See Also

-----  
`DataFrame.eq`: Equality test for DataFrame.  
`Series.isin`: Equivalent method on Series.  
`Series.str.contains`: Test if pattern or regex is contained within a string of a Series or Index.

Notes

-----  
`__iter__` is used (and not `__contains__`) to iterate over values when checking if it contains the elements in DataFrame.

Examples

-----  

```
>>> df = pd.DataFrame(
... {"num_legs": [2, 4], "num_wings": [2, 0]}, index=["falcon", "dog"]
...)
>>> df
```

	num_legs	num_wings
falcon	2	2
dog	4	0

```
falcon 2 2
dog 4 0
```

When ``values`` is a list check whether every value in the DataFrame is present in the list (which animals have 0 or 2 legs or wings)

```
>>> df.isin([0, 2])
 num_legs num_wings
falcon True True
dog False True
```

To check if ``values`` is *not* in the DataFrame, use the ``~`` operator:

```
>>> ~df.isin([0, 2])
 num_legs num_wings
falcon False False
dog True False
```

When ``values`` is a dict, we can pass values to check for each column separately:

```
>>> df.isin({"num_wings": [0, 3]})
 num_legs num_wings
falcon False False
dog False True
```

When ``values`` is a Series or DataFrame the index and column must match. Note that 'falcon' does not match based on the number of legs in other.

```
>>> other = pd.DataFrame(
... {"num_legs": [8, 3], "num_wings": [0, 2]}, index=["spider", "falcon"]
...)
>>> df.isin(other)
 num_legs num_wings
falcon False True
dog False False
```

`isna(self) -> DataFrame from pandas.core.frame.DataFrame`  
Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as None or :attr:`numpy.NaN`, gets mapped to True values. Everything else gets mapped to False values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

Series/DataFrame  
Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See Also

Series.isnull : Alias of isna.  
DataFrame.isnull : Alias of isna.  
Series.notna : Boolean inverse of isna.  
DataFrame.notna : Boolean inverse of isna.  
Series.dropna : Omit axes labels with missing values.  
DataFrame.dropna : Omit axes labels with missing values.  
isna : Top-level isna.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
... dict(
... age=[5, 6, np.nan],
... born=[
... pd.NaT,
... pd.Timestamp("1939-05-27"),
... pd.Timestamp("1940-04-25"),
...],
... name=["Alfred", "Batman", ""],
... toy=[None, "Batmobile", "Joker"],
...)
...)
```

```

...)
...)
>>> df
 age born name toy
0 5.0 NaT Alfred NaN
1 6.0 1939-05-27 Batman Batmobile
2 NaN 1940-04-25 Joker

```

```

>>> df.isna()
 age born name toy
0 False True False True
1 False False False False
2 True False False False

```

Show which entries in a Series are NA.

```

>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0 5.0
1 6.0
2 NaN
dtype: float64

>>> ser.isna()
0 False
1 False
2 True
dtype: bool

```

`isnull(self)` -> DataFrame from `pandas.core.frame.DataFrame`  
 DataFrame.isnull is an alias for DataFrame.isna.

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as None or `:attr:`numpy.NaN``, gets mapped to True values. Everything else gets mapped to False values. Characters such as empty strings `''` or `:attr:`numpy.inf`` are not considered NA values.

Returns

-----  
 Series/DataFrame  
 Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See Also

-----  
 Series.isnull : Alias of isna.  
 DataFrame.isnull : Alias of isna.  
 Series.notna : Boolean inverse of isna.  
 DataFrame.notna : Boolean inverse of isna.  
 Series.dropna : Omit axes labels with missing values.  
 DataFrame.dropna : Omit axes labels with missing values.  
 isna : Top-level isna.

Examples

-----  
 Show which entries in a DataFrame are NA.

```

>>> df = pd.DataFrame(
... dict(
... age=[5, 6, np.nan],
... born=[
... pd.NaT,
... pd.Timestamp("1939-05-27"),
... pd.Timestamp("1940-04-25"),
...],
... name=["Alfred", "Batman", ""],
... toy=[None, "Batmobile", "Joker"],
...)
...)
>>> df
 age born name toy
0 5.0 NaT Alfred NaN
1 6.0 1939-05-27 Batman Batmobile
2 NaN 1940-04-25 Joker

```

```
>>> df.isna()
 age born name toy
0 False True False True
1 False False False False
2 True False False False
```

Show which entries in a Series are NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0 5.0
1 6.0
2 NaN
dtype: float64
```

```
>>> ser.isna()
0 False
1 False
2 True
dtype: bool
```

`items(self)` -> Iterable[tuple[Hashable, Series]] from `pandas.core.frame.DataFrame`  
Iterate over (column name, Series) pairs.

Iterates over the DataFrame columns, returning a tuple with the column name and the content as a Series.

Yields

```

label : object
 The column names for the DataFrame being iterated over.
content : Series
 The column entries belonging to each label, as a Series.
```

See Also

```

DataFrame.iterrows : Iterate over DataFrame rows as
 (index, Series) pairs.
DataFrame.itertuples : Iterate over DataFrame rows as namedtuples
 of the values.
```

Examples

```

>>> df = pd.DataFrame(
... {
... "species": ["bear", "bear", "marsupial"],
... "population": [1864, 22000, 80000],
... },
... index=["panda", "polar", "koala"],
...)
>>> df
 species population
panda bear 1864
polar bear 22000
koala marsupial 80000
>>> for label, content in df.items():
... print(f"label: {label}")
... print(f"content: {content}", sep="\n")
label: species
content:
panda bear
polar bear
koala marsupial
Name: species, dtype: str
label: population
content:
panda 1864
polar 22000
koala 80000
Name: population, dtype: int64
```

`iterrows(self)` -> Iterable[tuple[Hashable, Series]] from `pandas.core.frame.DataFrame`  
Iterate over DataFrame rows as (index, Series) pairs.

Yields

```

```

index : label or tuple of label  
The index of the row. A tuple for a `MultiIndex`.  
data : Series  
The data of the row as a Series.

See Also

-----  
DataFrame.itertuples : Iterate over DataFrame rows as namedtuples of the values.  
DataFrame.items : Iterate over (column name, Series) pairs.

Notes

- 
1. Because ``iterrows`` returns a Series for each row, it does **not** preserve dtypes across the rows (dtypes are preserved across columns for DataFrames).  
  
To preserve dtypes while iterating over the rows, it is better to use :meth:`itertuples` which returns namedtuples of the values and which is generally faster than ``iterrows``.
  2. You should **never modify** something you are iterating over. This is not guaranteed to work in all cases. Depending on the data types, the iterator returns a copy and not a view, and writing to it will have no effect.

Examples

-----  
>>> df = pd.DataFrame([[1, 1.5]], columns=["int", "float"])  
>>> row = next(df.iterrows())[1]  
>>> row  
int 1.0  
float 1.5  
Name: 0, dtype: float64  
>>> print(row["int"].dtype)  
float64  
>>> print(df["int"].dtype)  
int64

itertuples(self, index: bool = True, name: str | None = 'Pandas') -> Iterable[tuple[Any, ...]]  
Iterate over DataFrame rows as namedtuples.

Parameters

-----  
index : bool, default True  
If True, return the index as the first element of the tuple.  
name : str or None, default "Pandas"  
The name of the returned namedtuples or None to return regular tuples.

Returns

-----  
iterator

An object to iterate over namedtuples for each row in the DataFrame with the first field possibly being the index and following fields being the column values.

See Also

-----  
DataFrame.iterrows : Iterate over DataFrame rows as (index, Series) pairs.  
DataFrame.items : Iterate over (column name, Series) pairs.

Notes

-----  
The column names will be renamed to positional names if they are invalid Python identifiers, repeated, or start with an underscore.

Examples

-----  
>>> df = pd.DataFrame(  
... {"num\_legs": [4, 2], "num\_wings": [0, 2]}, index=["dog", "hawk"]  
... )  
>>> df  
 num\_legs num\_wings  
dog 4 0  
hawk 2 2

```
>>> for row in df.itertuples():
... print(row)
Pandas(Index='dog', num_legs=4, num_wings=0)
Pandas(Index='hawk', num_legs=2, num_wings=2)
```

By setting the `index` parameter to False we can remove the index as the first element of the tuple:

```
>>> for row in df.itertuples(index=False):
... print(row)
Pandas(num_legs=4, num_wings=0)
Pandas(num_legs=2, num_wings=2)
```

With the `name` parameter set we set a custom name for the yielded namedtuples:

```
>>> for row in df.itertuples(name="Animal"):
... print(row)
Animal(Index='dog', num_legs=4, num_wings=0)
Animal(Index='hawk', num_legs=2, num_wings=2)
```

```
join(
 self,
 other: DataFrame | Series | Iterable[DataFrame | Series],
 on: IndexLabel | None = None,
 how: MergeHow = 'left',
 lsuffix: str = '',
 rsuffix: str = '',
 sort: bool = False,
 validate: JoinValidate | None = None
) -> DataFrame from pandas.core.frame.DataFrame
 Join columns of another DataFrame.
```

Join columns with `other` DataFrame either on index or on a key column. Efficiently join multiple DataFrame objects by index at once by passing a list.

Parameters  
-----

**other** : DataFrame, Series, or a list containing any combination of them  
Index should be similar to one of the columns in this one. If a Series is passed, its name attribute must be set, and that will be used as the column name in the resulting joined DataFrame.

**on** : str, list of str, or array-like, optional  
Column or index level name(s) in the caller to join on the index in `other`, otherwise joins index-on-index. If multiple values given, the `other` DataFrame must have a MultiIndex. Can pass an array as the join key if it is not already contained in the calling DataFrame. Like an Excel VLOOKUP operation.

**how** : {'left', 'right', 'outer', 'inner', 'cross', 'left\_anti', 'right\_anti'}, default 'left'  
How to handle the operation of the two objects.

- \* left: use calling frame's index (or column if on is specified)
- \* right: use `other`'s index.
- \* outer: form union of calling frame's index (or column if on is specified) with `other`'s index, and sort it lexicographically.
- \* inner: form intersection of calling frame's index (or column if on is specified) with `other`'s index, preserving the order of the calling's one.
- \* cross: creates the cartesian product from both frames, preserves the order of the left keys.
- \* left\_anti: use set difference of calling frame's index and `other`'s index.
- \* right\_anti: use set difference of `other`'s index and calling frame's index.

**lsuffix** : str, default ''  
Suffix to use from left frame's overlapping columns.

**rsuffix** : str, default ''  
Suffix to use from right frame's overlapping columns.

**sort** : bool, default False  
Order result DataFrame lexicographically by the join key. If False, the order of the join key depends on the join type (how keyword).

**validate** : str, optional  
If specified, checks if join is of specified type.

- \* "one\_to\_one" or "1:1": check if join keys are unique in both left



- and right datasets.
- \* "one\_to\_many" or "1:m": check if join keys are unique in left dataset.
- \* "many\_to\_one" or "m:1": check if join keys are unique in right dataset.
- \* "many\_to\_many" or "m:m": allowed, but does not result in checks.

#### Returns

-----  
DataFrame

A dataframe containing columns from both the caller and `other`.

#### See Also

-----  
DataFrame.merge : For column(s)-on-column(s) operations.

#### Notes

-----  
Parameters `on`, `lsuffix`, and `rsuffix` are not supported when passing a list of `DataFrame` objects.

#### Examples

-----  
>>> df = pd.DataFrame(  
... {  
... "key": ["K0", "K1", "K2", "K3", "K4", "K5"],  
... "A": ["A0", "A1", "A2", "A3", "A4", "A5"],  
... }  
... )

```
>>> df
 key A
0 K0 A0
1 K1 A1
2 K2 A2
3 K3 A3
4 K4 A4
5 K5 A5
```

```
>>> other = pd.DataFrame({"key": ["K0", "K1", "K2"], "B": ["B0", "B1", "B2"]})
```

```
>>> other
 key B
0 K0 B0
1 K1 B1
2 K2 B2
```

Join DataFrames using their indexes.

```
>>> df.join(other, lsuffix="_caller", rsuffix="_other")
 key_caller A key_other B
0 K0 A0 K0 B0
1 K1 A1 K1 B1
2 K2 A2 K2 B2
3 K3 A3 NaN NaN
4 K4 A4 NaN NaN
5 K5 A5 NaN NaN
```

If we want to join using the key columns, we need to set key to be the index in both `df` and `other`. The joined DataFrame will have key as its index.

```
>>> df.set_index("key").join(other.set_index("key"))
 A B
key
K0 A0 B0
K1 A1 B1
K2 A2 B2
K3 A3 NaN
K4 A4 NaN
K5 A5 NaN
```

Another option to join using the key columns is to use the `on` parameter. DataFrame.join always uses `other`'s index but we can use any column in `df`. This method preserves the original DataFrame's index in the result.

```
>>> df.join(other.set_index("key"), on="key")
 key A B
```

```

0 K0 A0 B0
1 K1 A1 B1
2 K2 A2 B2
3 K3 A3 NaN
4 K4 A4 NaN
5 K5 A5 NaN

```

Using non-unique key values shows how they are matched.

```

>>> df = pd.DataFrame(
... {
... "key": ["K0", "K1", "K1", "K3", "K0", "K1"],
... "A": ["A0", "A1", "A2", "A3", "A4", "A5"],
... }
...)

```

```

>>> df
 key A
0 K0 A0
1 K1 A1
2 K1 A2
3 K3 A3
4 K0 A4
5 K1 A5

```

```

>>> df.join(other.set_index("key"), on="key", validate="m:1")
 key A B
0 K0 A0 B0
1 K1 A1 B1
2 K1 A2 B1
3 K3 A3 NaN
4 K0 A4 B0
5 K1 A5 B1

```

```

kurt(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return unbiased kurtosis over requested axis.

```

Kurtosis obtained using Fisher's definition of kurtosis (kurtosis of normal == 0.0). Normalized by N-1.

Parameters

axis : {index (0), columns (1)}

Axis for the function to be applied on.

For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric\_only : bool, default False

Include only float, int, boolean columns.

\*\*kwargs

Additional keyword arguments to be passed to the function.

Returns

Series or scalar

Unbiased kurtosis over requested axis.

See Also

Dataframe.kurtosis : Returns unbiased kurtosis over requested axis.

Examples

```
>>> s = pd.Series([1, 2, 2, 3], index=["cat", "dog", "dog", "mouse"])
>>> s
cat 1
dog 2
dog 2
mouse 3
dtype: int64
>>> s.kurt()
1.5
```

With a DataFrame

```
>>> df = pd.DataFrame(
... {"a": [1, 2, 2, 3], "b": [3, 4, 4, 4]},
... index=["cat", "dog", "dog", "mouse"],
...)
>>> df
 a b
cat 1 3
dog 2 4
dog 2 4
mouse 3 4
>>> df.kurt()
a 1.5
b 4.0
dtype: float64
```

With axis=None

```
>>> df.kurt(axis=None)
-0.9886927196984727
```

Using axis=1

```
>>> df = pd.DataFrame(
... {"a": [1, 2], "b": [3, 4], "c": [3, 4], "d": [1, 2]},
... index=["cat", "dog"],
...)
>>> df.kurt(axis=1)
cat -6.0
dog -6.0
dtype: float64
```

```
kurtosis = kurt(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any
```

`le(self, other, axis: Axis = 'columns', level=None)` -> DataFrame from pandas.core.frame.DataFrame  
Get Greater than or equal to of dataframe and other, element-wise (binary operator '`le`')

Among flexible wrappers (`'eq'`, `'ne'`, `'le'`, `'lt'`, `'ge'`, `'gt'`) to comparison operators.

Equivalent to `'=='`, `'!='`, `'<='`, `'<'`, `'>='`, `'>'` with support to choose axis (rows or columns) and level for comparison.

Parameters

```
other : scalar, sequence, Series, or DataFrame
 Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
 Whether to compare by the index (0 or 'index') or columns
 (1 or 'columns').
level : int or label
 Broadcast across a level, matching Index values on the passed
 MultiIndex level.
```

Returns

```
DataFrame of bool
 Result of the comparison.
```

See Also

-----

DataFrame.eq : Compare DataFrames for equality elementwise.  
DataFrame.ne : Compare DataFrames for inequality elementwise.  
DataFrame.le : Compare DataFrames for less than inequality  
or equality elementwise.  
DataFrame.lt : Compare DataFrames for strictly less than  
inequality elementwise.  
DataFrame.ge : Compare DataFrames for greater than inequality  
or equality elementwise.  
DataFrame.gt : Compare DataFrames for strictly greater than  
inequality elementwise.

Notes

-----

Mismatched indices will be unioned together.  
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples

-----

```
>>> df = pd.DataFrame({'cost': [250, 150, 100],
... 'revenue': [100, 250, 300]},
... index=['A', 'B', 'C'])
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300
```

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
 cost revenue
A False True
B False False
C True False
```

```
>>> df.eq(100)
 cost revenue
A False True
B False False
C True False
```

When `other` is a `:class:`Series``, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
 cost revenue
A True True
B True False
C False True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
 cost revenue
A True False
B True True
C True True
D True True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
 cost revenue
A True True
B False False
C False False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
 cost revenue
A True False
B False True
C True False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
... index=['A', 'B', 'C', 'D'])
>>> other
 revenue
A 300
B 250
C 100
D 150

>>> df.gt(other)
 cost revenue
A False False
B False False
C False True
D False False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
... 'revenue': [100, 250, 300, 200, 175, 225]},
... index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
... ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
 cost revenue
Q1 A 250 100
 B 150 250
 C 100 300
Q2 A 150 200
 B 300 175
 C 220 225

>>> df.le(df_multindex, level=1)
 cost revenue
Q1 A True True
 B True True
 C True True
Q2 A False True
 B True False
 C True False
```

`lt(self, other, axis: Axis = 'columns', level=None) -> DataFrame` from `pandas.core.frame.DataFrame`  
Get Greater than of dataframe and other, element-wise (binary operator ``lt``).

Among flexible wrappers (``eq``, ``ne``, ``le``, ``lt``, ``ge``, ``gt``) to comparison operators.

Equivalent to ``==``, ``!=``, ``<=``, ``<``, ``>=``, ``>`` with support to choose axis (rows or columns) and level for comparison.

Parameters

-----  
`other` : scalar, sequence, Series, or DataFrame  
Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}, default 'columns'  
Whether to compare by the index (0 or 'index') or columns (1 or 'columns').  
`level` : int or label  
Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns

-----  
DataFrame of bool  
Result of the comparison.

See Also

-----  
`DataFrame.eq` : Compare DataFrames for equality elementwise.  
`DataFrame.ne` : Compare DataFrames for inequality elementwise.  
`DataFrame.le` : Compare DataFrames for less than inequality or equality elementwise.  
`DataFrame.lt` : Compare DataFrames for strictly less than inequality elementwise.  
`DataFrame.ge` : Compare DataFrames for greater than inequality

or equality elementwise.  
DataFrame.gt : Compare DataFrames for strictly greater than  
inequality elementwise.

#### Notes

Mismatched indices will be unioned together.  
`NaN` values are considered different (i.e. `NaN` != `NaN`).

#### Examples

```
>>> df = pd.DataFrame({'cost': [250, 150, 100],
... 'revenue': [100, 250, 300]},
... index=['A', 'B', 'C'])
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300
```

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
 cost revenue
A False True
B False False
C True False
```

```
>>> df.eq(100)
 cost revenue
A False True
B False False
C True False
```

When `other` is a `:class:`Series``, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
 cost revenue
A True True
B True False
C False True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
 cost revenue
A True False
B True True
C True True
D True True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
 cost revenue
A True True
B False False
C False False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
 cost revenue
A True False
B False True
C True False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
... index=['A', 'B', 'C', 'D'])
>>> other
 revenue
A 300
B 250
```

```
C 100
D 150
```

```
>>> df.gt(other)
 cost revenue
A False False
B False False
C False True
D False False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
... 'revenue': [100, 250, 300, 200, 175, 225]},
... index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
... ['A', 'B', 'C', 'A', 'B', 'C']])
```

```
>>> df_multindex
 cost revenue
Q1 A 250 100
 B 150 250
 C 100 300
Q2 A 150 200
 B 300 175
 C 220 225
```

```
>>> df.le(df_multindex, level=1)
```

```
 cost revenue
Q1 A True True
 B True True
 C True True
Q2 A False True
 B True False
 C True False
```

```
map(
 self,
 func: PythonFuncType,
 na_action: Literal['ignore'] | None = None,
 **kwargs
) -> DataFrame from pandas.core.frame.DataFrame
Apply a function to a Dataframe elementwise.
```

.. versionadded:: 2.1.0

DataFrame.applymap was deprecated and renamed to DataFrame.map.

This method applies a function that accepts and returns a scalar to every element of a DataFrame.

Parameters

```

func : callable
 Python function, returns a single value from a single value.
na_action : {None, 'ignore'}, default None
 If 'ignore', propagate NaN values, without passing them to func.
**kwargs
 Additional keyword arguments to pass as keywords arguments to `func`.
```

Returns

```

DataFrame
 Transformed DataFrame.
```

See Also

```

DataFrame.apply : Apply a function along input axis of DataFrame.
DataFrame.replace: Replace values given in `to_replace` with `value`.
Series.map : Apply a function elementwise on a Series.
```

Examples

```

>>> df = pd.DataFrame([[1, 2.12], [3.356, 4.567]])
>>> df
 0 1
0 1.000 2.120
1 3.356 4.567
```

```
>>> df.map(lambda x: len(str(x)))
 0 1
0 3 4
1 5 5
```

Like Series.map, NA values can be ignored:

```
>>> df_copy = df.copy()
>>> df_copy.iloc[0, 0] = pd.NA
>>> df_copy.map(lambda x: len(str(x)), na_action="ignore")
 0 1
0 NaN 4
1 5.0 5
```

It is also possible to use `map` with functions that are not `lambda` functions:

```
>>> df.map(round, ndigits=1)
 0 1
0 1.0 2.1
1 3.4 4.6
```

Note that a vectorized version of `func` often exists, which will be much faster. You could square each number elementwise.

```
>>> df.map(lambda x: x**2)
 0 1
0 1.000000 4.494400
1 11.262736 20.857489
```

But it's better to avoid map in that case.

```
>>> df**2
 0 1
0 1.000000 4.494400
1 11.262736 20.857489
```

```
max(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return the maximum of the values over the requested axis.
```

If you want the *index* of the maximum, use ``idxmax``. This is the equivalent of the ``numpy.ndarray`` method ``argmax``.

Parameters

axis : {index (0), columns (1)}  
Axis for the function to be applied on.  
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True  
Exclude NA/null values when computing the result.  
numeric\_only : bool, default False  
Include only float, int, boolean columns.

\*\*kwargs  
Additional keyword arguments to be passed to the function.

Returns

Series or scalar  
Value containing the calculation referenced in the description.

See Also

-----



Series.sum : Return the sum.  
 Series.min : Return the minimum.  
 Series.max : Return the maximum.  
 Series.idxmin : Return the index of the minimum.  
 Series.idxmax : Return the index of the maximum.  
 DataFrame.sum : Return the sum over the requested axis.  
 DataFrame.min : Return the minimum over the requested axis.  
 DataFrame.max : Return the maximum over the requested axis.  
 DataFrame.idxmin : Return the index of the minimum over the requested axis.  
 DataFrame.idxmax : Return the index of the maximum over the requested axis.

#### Examples

```

>>> idx = pd.MultiIndex.from_arrays(
... [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
... names=["blooded", "animal"],
...)
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm dog 4
 falcon 2
cold fish 0
 spider 8
Name: legs, dtype: int64

>>> s.max()
8

```

```

mean(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return the mean of the values over the requested axis.

```

#### Parameters

axis : {index (0), columns (1)}  
 Axis for the function to be applied on.  
 For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True  
 Exclude NA/null values when computing the result.

numeric\_only : bool, default False  
 Include only float, int, boolean columns.

\*\*kwargs  
 Additional keyword arguments to be passed to the function.

#### Returns

Series or scalar  
 Value containing the calculation referenced in the description.

#### See Also

Series.sum : Return the sum.  
 Series.min : Return the minimum.  
 Series.max : Return the maximum.  
 Series.idxmin : Return the index of the minimum.  
 Series.idxmax : Return the index of the maximum.  
 DataFrame.sum : Return the sum over the requested axis.  
 DataFrame.min : Return the minimum over the requested axis.  
 DataFrame.max : Return the maximum over the requested axis.  
 DataFrame.idxmin : Return the index of the minimum over the requested axis.  
 DataFrame.idxmax : Return the index of the maximum over the requested axis.

#### Examples

```

>>> s = pd.Series([1, 2, 3])
>>> s.mean()
2.0

```

With a DataFrame

```

>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
 a b
tiger 1 2
zebra 2 3
>>> df.mean()
a 1.5
b 2.5
dtype: float64

```

Using axis=1

```

>>> df.mean(axis=1)
tiger 1.5
zebra 2.5
dtype: float64

```

In this case, `numeric_only` should be set to `True` to avoid getting an error.

```

>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.mean(numeric_only=True)
a 1.5
dtype: float64

```

```

median(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return the median of the values over the requested axis.

```

Parameters

```

axis : {index (0), columns (1)}
 Axis for the function to be applied on.
 For `Series` this parameter is unused and defaults to 0.

 For DataFrames, specifying ``axis=None`` will apply the aggregation
 across both axes.

 .. versionadded:: 2.0.0

```

```

skipna : bool, default True
 Exclude NA/null values when computing the result.
numeric_only : bool, default False
 Include only float, int, boolean columns.

```

```

**kwargs
 Additional keyword arguments to be passed to the function.

```

Returns

```

Series or scalar
 Value containing the calculation referenced in the description.

```

See Also

```

Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.

```

DataFrame.idxmax : Return the index of the maximum over the requested axis.

#### Examples

```

>>> s = pd.Series([1, 2, 3])
>>> s.median()
2.0
```

With a DataFrame

```
>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
 a b
tiger 1 2
zebra 2 3
>>> df.median()
a 1.5
b 2.5
dtype: float64
```

Using axis=1

```
>>> df.median(axis=1)
tiger 1.5
zebra 2.5
dtype: float64
```

In this case, `numeric\_only` should be set to `True` to avoid getting an error.

```
>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.median(numeric_only=True)
a 1.5
dtype: float64
```

```
melt(
 self,
 id_vars=None,
 value_vars=None,
 var_name=None,
 value_name: Hashable = 'value',
 col_level: Level | None = None,
 ignore_index: bool = True
) -> DataFrame from pandas.core.frame.DataFrame
Unpivot DataFrame from wide to long format, optionally leaving identifiers set.
```

This function is useful to massage a DataFrame into a format where one or more columns are identifier variables (`id\_vars`), while all other columns, considered measured variables (`value\_vars`), are "unpivoted" to the row axis, leaving just two non-identifier columns, 'variable' and 'value'.

#### Parameters

```

id_vars : scalar, tuple, list, or ndarray, optional
 Column(s) to use as identifier variables.
value_vars : scalar, tuple, list, or ndarray, optional
 Column(s) to unpivot. If not specified, uses all columns that
 are not set as `id_vars`.
var_name : scalar, default None
 Name to use for the 'variable' column. If None it uses
 ``frame.columns.name`` or 'variable'.
value_name : scalar, default 'value'
 Name to use for the 'value' column, can't be an existing column label.
col_level : scalar, optional
 If columns are a MultiIndex then use this level to melt.
ignore_index : bool, default True
 If True, original index is ignored. If False, original index is retained.
 Index labels will be repeated as necessary.
```

#### Returns

```

DataFrame
 Unpivoted DataFrame.
```

#### See Also

```

```

melt : Identical method.  
 pivot\_table : Create a spreadsheet-style pivot table as a DataFrame.  
 DataFrame.pivot : Return reshaped DataFrame organized by given index / column values.  
 DataFrame.explode : Explode a DataFrame from list-like columns to long format.

#### Notes

Reference :ref:`the user guide <reshaping.melt>` for more examples.

#### Examples

```

>>> df = pd.DataFrame(
... {
... "A": {0: "a", 1: "b", 2: "c"},
... "B": {0: 1, 1: 3, 2: 5},
... "C": {0: 2, 1: 4, 2: 6},
... }
...)
>>> df
 A B C
0 a 1 2
1 b 3 4
2 c 5 6

>>> df.melt(id_vars=["A"], value_vars=["B"])
 A variable value
0 a B 1
1 b B 3
2 c B 5

>>> df.melt(id_vars=["A"], value_vars=["B", "C"])
 A variable value
0 a B 1
1 b B 3
2 c B 5
3 a C 2
4 b C 4
5 c C 6

```

The names of 'variable' and 'value' columns can be customized:

```

>>> df.melt(
... id_vars=["A"],
... value_vars=["B"],
... var_name="myVarname",
... value_name="myValname",
...)
 A myVarname myValname
0 a B 1
1 b B 3
2 c B 5

```

Original index values can be kept around:

```

>>> df.melt(id_vars=["A"], value_vars=["B", "C"], ignore_index=False)
 A variable value
0 a B 1
1 b B 3
2 c B 5
0 a C 2
1 b C 4
2 c C 6

```

If you have multi-index columns:

```

>>> df.columns = [list("ABC"), list("DEF")]
>>> df
 A B C D E F
0 a 1 2 a 1 2
1 b 3 4 b 3 4
2 c 5 6 c 5 6

>>> df.melt(col_level=0, id_vars=["A"], value_vars=["B"])
 A variable value
0 a B 1

```

```

0 a B 1
1 b B 3
2 c B 5

```

```

>>> df.melt(id_vars=["A", "D"], value_vars=["B", "E"])
(A, D) variable_0 variable_1 value
0 a B E 1
1 b B E 3
2 c B E 5

```

`memory_usage(self, index: bool = True, deep: bool = False) -> Series` from `pandas.core.frame`.  
Return the memory usage of each column in bytes.

The memory usage can optionally include the contribution of the index and elements of `'object'` dtype.

This value is displayed in `'DataFrame.info'` by default. This can be suppressed by setting `'pandas.options.display.memory_usage'` to `False`.

#### Parameters

`index` : bool, default `True`  
Specifies whether to include the memory usage of the DataFrame's index in returned Series. If `'index=True'`, the memory usage of the index is the first item in the output.

`deep` : bool, default `False`  
If `True`, introspect the data deeply by interrogating `'object'` dtypes for system-level memory consumption, and include it in the returned values.

#### Returns

##### Series

A Series whose index is the original column names and whose values is the memory usage of each column in bytes.

#### See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of an ndarray.

`Series.memory_usage` : Bytes consumed by a Series.

`Categorical` : Memory-efficient array for string values with many repeated values.

`DataFrame.info` : Concise summary of a DataFrame.

#### Notes

See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.

#### Examples

```

>>> dtypes = ["int64", "float64", "complex128", "object", "bool"]
>>> data = dict([(t, np.ones(shape=5000, dtype=int).astype(t)) for t in dtypes])
>>> df = pd.DataFrame(data)
>>> df.head()

```

```

 int64 float64 complex128 object bool
0 1 1.0 1.0+0.0j 1 True
1 1 1.0 1.0+0.0j 1 True
2 1 1.0 1.0+0.0j 1 True
3 1 1.0 1.0+0.0j 1 True
4 1 1.0 1.0+0.0j 1 True

```

```

>>> df.memory_usage()
Index 132
int64 40000
float64 40000
complex128 80000
object 40000
bool 5000
dtype: int64

```

```

>>> df.memory_usage(index=False)
int64 40000
float64 40000
complex128 80000
object 40000

```

```
bool 5000
dtype: int64
```

The memory footprint of `object` dtype columns is ignored by default:

```
>>> df.memory_usage(deep=True)
Index 132
int64 40000
float64 40000
complex128 80000
object 180000
bool 5000
dtype: int64
```

Use a Categorical for efficient storage of an object-dtype column with many repeated values.

```
>>> df["object"].astype("category").memory_usage(deep=True)
5140
```

```
merge(
 self,
 right: DataFrame | Series,
 how: MergeHow = 'inner',
 on: IndexLabel | AnyArrayLike | None = None,
 left_on: IndexLabel | AnyArrayLike | None = None,
 right_on: IndexLabel | AnyArrayLike | None = None,
 left_index: bool = False,
 right_index: bool = False,
 sort: bool = False,
 suffixes: Suffixes = ('_x', '_y'),
 copy: bool | lib.NoDefault = <no_default>,
 indicator: str | bool = False,
 validate: MergeValidate | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Merge DataFrame or named Series objects with a database-style join.
```

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes \*will be ignored\*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

.. warning::

If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters

right : DataFrame or named Series  
Object to merge with.

how : {'left', 'right', 'outer', 'inner', 'cross', 'left\_anti', 'right\_anti'},  
default 'inner'  
Type of merge to be performed.

- \* left: use only keys from left frame, similar to a SQL left outer join; preserve key order.
- \* right: use only keys from right frame, similar to a SQL right outer join; preserve key order.
- \* outer: use union of keys from both frames, similar to a SQL full outer join; sort keys lexicographically.
- \* inner: use intersection of keys from both frames, similar to a SQL inner join; preserve the order of the left keys.
- \* cross: creates the cartesian product from both frames, preserves the order of the left keys.
- \* left\_anti: use only keys from left frame that are not in right frame, similar to SQL left anti join; preserve key order.

.. versionadded:: 3.0

- \* right\_anti: use only keys from right frame that are not in left frame, similar to SQL right anti join; preserve key order.

.. versionadded:: 3.0

`on` : Hashable or a sequence of the previous  
 Column or index level names to join on. These must be found in both DataFrames. If ``on`` is None and not merging on indexes then this defaults to the intersection of the columns in both DataFrames.

`left_on` : Hashable or a sequence of the previous, or array-like  
 Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

`right_on` : Hashable or a sequence of the previous, or array-like  
 Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

`left_index` : bool, default False  
 Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

`right_index` : bool, default False  
 Use the index from the right DataFrame as the join key. Same caveats as `left_index`.

`sort` : bool, default False  
 Sort the join keys lexicographically in the result DataFrame. If False, the order of the join keys depends on the join type (how keyword).

`suffixes` : list-like, default is `("_x", "_y")`  
 A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in ``left`` and ``right`` respectively. Pass a value of ``None`` instead of a string to indicate that the column name from ``left`` or ``right`` should be left as-is, with no suffix. At least one of the values must not be None.

`copy` : bool, default False  
 This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0  
  
 This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_ for more details.

`indicator` : bool or str, default False  
 If True, adds a column to the output DataFrame called `"_merge"` with information on the source of each row. The column can be given a different name by providing a string argument. The column will have a Categorical type with the value of `"left_only"` for observations whose merge key only appears in the left DataFrame, `"right_only"` for observations whose merge key only appears in the right DataFrame, and `"both"` if the observation's merge key is found in both DataFrames.

`validate` : str, optional  
 If specified, checks if merge is of specified type.

- \* `"one_to_one"` or `"1:1"`: check if merge keys are unique in both left and right datasets.
- \* `"one_to_many"` or `"1:m"`: check if merge keys are unique in left dataset.
- \* `"many_to_one"` or `"m:1"`: check if merge keys are unique in right dataset.
- \* `"many_to_many"` or `"m:m"`: allowed, but does not result in checks.

Returns  
 -----  
 DataFrame  
 A DataFrame of the two merged objects.

See Also  
 -----  
`merge_ordered` : Merge with optional filling/interpolation.  
`merge_asof` : Merge on nearest keys.  
`DataFrame.join` : Similar method using indices.

Examples  
 -----  

```
>>> df1 = pd.DataFrame({'lkey': ['foo', 'bar', 'baz', 'foo'],
... 'value': [1, 2, 3, 5]})
```

```
>>> df2 = pd.DataFrame({'rkey': ['foo', 'bar', 'baz', 'foo'],
... 'value': [5, 6, 7, 8]})
```

```
>>> df1
 lkey value
0 foo 1
1 bar 2
2 baz 3
3 foo 5
```

```
>>> df2
 rkey value
0 foo 5
1 bar 6
2 baz 7
3 foo 8
```

Merge df1 and df2 on the lkey and rkey columns. The value columns have the default suffixes, \_x and \_y, appended.

```
>>> df1.merge(df2, left_on='lkey', right_on='rkey')
 lkey value_x rkey value_y
0 foo 1 foo 5
1 foo 1 foo 8
2 bar 2 bar 6
3 baz 3 baz 7
4 foo 5 foo 5
5 foo 5 foo 8
```

Merge DataFrames df1 and df2 with specified left and right suffixes appended to any overlapping columns.

```
>>> df1.merge(df2, left_on='lkey', right_on='rkey',
... suffixes=('_left', '_right'))
 lkey value_left rkey value_right
0 foo 1 foo 5
1 foo 1 foo 8
2 bar 2 bar 6
3 baz 3 baz 7
4 foo 5 foo 5
5 foo 5 foo 8
```

Merge DataFrames df1 and df2, but raise an exception if the DataFrames have any overlapping columns.

```
>>> df1.merge(df2, left_on='lkey', right_on='rkey', suffixes=(False, False))
Traceback (most recent call last):
...
ValueError: columns overlap but no suffix specified:
Index(['value'], dtype='object')
```

```
>>> df1 = pd.DataFrame({'a': ['foo', 'bar'], 'b': [1, 2]})
>>> df2 = pd.DataFrame({'a': ['foo', 'baz'], 'c': [3, 4]})
```

```
>>> df1
 a b
0 foo 1
1 bar 2
```

```
>>> df2
 a c
0 foo 3
1 baz 4
```

```
>>> df1.merge(df2, how='inner', on='a')
 a b c
0 foo 1 3
```

```
>>> df1.merge(df2, how='left', on='a')
 a b c
0 foo 1 3.0
1 bar 2 NaN
```

```
>>> df1 = pd.DataFrame({'left': ['foo', 'bar']})
>>> df2 = pd.DataFrame({'right': [7, 8]})
>>> df1
 left
0 foo
1 bar
>>> df2
 right
```



```

0 7
1 8

>>> df1.merge(df2, how='cross')
 left right
0 foo 7
1 foo 8
2 bar 7
3 bar 8

```

```

min(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return the minimum of the values over the requested axis.

If you want the *index* of the minimum, use ``idxmin``.
This is the equivalent of the ``numpy.ndarray`` method ``argmin``.

Parameters

axis : {index (0), columns (1)}
 Axis for the function to be applied on.
 For `Series` this parameter is unused and defaults to 0.

 For DataFrames, specifying ``axis=None`` will apply the aggregation
 across both axes.

 .. versionadded:: 2.0.0

skipna : bool, default True
 Exclude NA/null values when computing the result.
numeric_only : bool, default False
 Include only float, int, boolean columns.

**kwargs
 Additional keyword arguments to be passed to the function.

Returns

Series or scalar
 Value containing the calculation referenced in the description.

See Also

Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

>>> idx = pd.MultiIndex.from_arrays(
... [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
... names=["blooded", "animal"],
...)
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm dog 4
 falcon 2
cold fish 0
 spider 8
Name: legs, dtype: int64

>>> s.min()
0

```

```
mod(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas
Get Modulo of dataframe and other, element-wise (binary operator `mod`).
```

Equivalent to ``dataframe % other``, but with support to substitute a fill\_value for missing data in one of the inputs. With reverse version, `rmod`.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

#### Parameters

-----  
other : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.  
axis : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns.  
(1 or 'columns'). For Series input, axis to match Series index on.  
level : int or label  
Broadcast across a level, matching Index values on the  
passed MultiIndex level.  
fill\_value : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for  
successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing  
the result will be missing.

#### Returns

-----  
DataFrame  
Result of the arithmetic operation.

#### See Also

-----  
DataFrame.add : Add DataFrames.  
DataFrame.sub : Subtract DataFrames.  
DataFrame.mul : Multiply DataFrames.  
DataFrame.div : Divide DataFrames (float division).  
DataFrame.truediv : Divide DataFrames (float division).  
DataFrame.floordiv : Divide DataFrames (integer division).  
DataFrame.mod : Calculate modulo (remainder after division).  
DataFrame.pow : Calculate exponential power.

#### Notes

-----  
Mismatched indices will be unioned together.

#### Examples

-----  
>>> df = pd.DataFrame({'angles': [0, 3, 4],  
... 'degrees': [360, 180, 360]},  
... index=['circle', 'triangle', 'rectangle'])  
>>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0

triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
 angles degrees
circle inf 0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN

>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
```

```

rectangle 4 360
B square 4 360
pentagon 5 540
hexagon 6 720

>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
triangle 1.0 1.0
rectangle 1.0 1.0
B square 0.0 0.0
pentagon 0.0 0.0
hexagon 0.0 0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81

```

`mode(self, axis: Axis = 0, numeric_only: bool = False, dropna: bool = True) -> DataFrame` from pandas  
 Get the mode(s) of each element along the selected axis.

The mode of a set of values is the value that appears most often.  
 It can be multiple values.

#### Parameters

`axis` : {0 or 'index', 1 or 'columns'}, default 0  
 The axis to iterate over while searching for the mode:

- \* 0 or 'index' : get mode of each column
- \* 1 or 'columns' : get mode of each row.

`numeric_only` : bool, default False  
 If True, only apply to numeric columns.  
`dropna` : bool, default True  
 Don't consider counts of NaN/NaT.

#### Returns

`DataFrame`  
 The modes of each column or row.

#### See Also

`Series.mode` : Return the highest frequency value in a Series.  
`Series.value_counts` : Return the counts of values in a Series.

#### Examples

```

>>> df = pd.DataFrame(
... [
... ("bird", 2, 2),
... ("mammal", 4, np.nan),
... ("arthropod", 8, 0),
... ("bird", 2, np.nan),
...],
... index=("falcon", "horse", "spider", "ostrich"),
... columns=("species", "legs", "wings"),
...)
>>> df
 species legs wings
falcon bird 2 2.0
horse mammal 4 NaN
spider arthropod 8 0.0
ostrich bird 2 NaN

```

By default, missing values are not considered, and the mode of wings are both 0 and 2. Because the resulting DataFrame has two rows, the second row of ``species`` and ``legs`` contains ``NaN``.

```

>>> df.mode()
 species legs wings

```

```
0 bird 2.0 0.0
1 NaN NaN 2.0
```

Setting ``dropna=False`` ``NaN`` values are considered and they can be the mode (like for wings).

```
>>> df.mode(dropna=False)
 species legs wings
0 bird 2 NaN
```

Setting ``numeric\_only=True``, only the mode of numeric columns is computed, and columns of other types are ignored.

```
>>> df.mode(numeric_only=True)
 legs wings
0 2.0 0.0
1 NaN 2.0
```

To compute the mode over columns and not rows, use the axis parameter:

```
>>> df.mode(axis="columns", numeric_only=True)
 0 1
falcon 2.0 NaN
horse 4.0 NaN
spider 0.0 8.0
ostrich 2.0 NaN
```

`mul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas`  
Get Multiplication of dataframe and other, element-wise (binary operator ``mul``).

Equivalent to ``dataframe * other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``rmul``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns.  
(1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
Broadcast across a level, matching Index values on the passed MultiIndex level.  
`fill_value` : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing the result will be missing.

#### Returns

DataFrame  
Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

#### Notes

Mismatched indices will be unioned together.

#### Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
```

```

>>> df
 angles degrees
circle 0 360
triangle 3 180
rectangle 4 360

Add a scalar with operator version which return the same
results.

>>> df + 1
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361

>>> df.add(1)
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361

Divide by constant with reverse version.

>>> df.div(10)
 angles degrees
circle 0.0 36.0
triangle 0.3 18.0
rectangle 0.4 36.0

>>> df.rdiv(10)
 angles degrees
circle inf 0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778

Subtract a list and Series by axis with operator version.

>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359

Multiply a dictionary by axis.

>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080

Multiply a DataFrame of different shape with operator version.

>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0

```

```
triangle 3
rectangle 4
```

```
>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
```

```
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

`multiply = mul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`ne(self, other, axis: Axis = 'columns', level=None) -> DataFrame` from `pandas.core.frame.DataFrame`.  
Get Not equal to of dataframe and other, element-wise (binary operator `'ne'`).

Among flexible wrappers (`'eq'`, `'ne'`, `'le'`, `'lt'`, `'ge'`, `'gt'`) to comparison operators.

Equivalent to `'=='`, `'!='`, `'<='`, `'<'`, `'>='`, `'>'` with support to choose axis (rows or columns) and level for comparison.

Parameters

```
other : scalar, sequence, Series, or DataFrame
 Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
 Whether to compare by the index (0 or 'index') or columns
 (1 or 'columns').
level : int or label
 Broadcast across a level, matching Index values on the passed
 MultiIndex level.
```

Returns

```
DataFrame of bool
Result of the comparison.
```

See Also

```

DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality
 or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than
 inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality
 or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than
 inequality elementwise.

```

#### Notes

```

Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

```

#### Examples

```

>>> df = pd.DataFrame(
... {"cost": [250, 150, 100], "revenue": [100, 250, 300]},
... index=["A", "B", "C"],
...)
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300

```

Comparison with a scalar, using either the operator or method:

```

>>> df == 100
 cost revenue
A False True
B False False
C True False

>>> df.eq(100)
 cost revenue
A False True
B False False
C True False

```

When `other` is a `:class:`Series``, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```

>>> df != pd.Series([100, 250], index=["cost", "revenue"])
 cost revenue
A True True
B True False
C False True

```

Use the method to control the broadcast axis:

```

>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis="index")
 cost revenue
A True False
B True True
C True True
D True True

```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```

>>> df == [250, 100]
 cost revenue
A True True
B False False
C False False

```

Use the method to control the axis:

```

>>> df.eq([250, 250, 100], axis="index")
 cost revenue
A True False
B False True
C True False

```



Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame(
... {"revenue": [300, 250, 100, 150]}, index=["A", "B", "C", "D"]
...)
>>> other
 revenue
A 300
B 250
C 100
D 150

>>> df.gt(other)
 cost revenue
A False False
B False False
C False True
D False False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame(
... {
... "cost": [250, 150, 100, 150, 300, 220],
... "revenue": [100, 250, 300, 200, 175, 225],
... },
... index=[
... ["Q1", "Q1", "Q1", "Q2", "Q2", "Q2"],
... ["A", "B", "C", "A", "B", "C"],
...],
...)
>>> df_multindex
 cost revenue
Q1 A 250 100
 B 150 250
 C 100 300
Q2 A 150 200
 B 300 175
 C 220 225

>>> df.le(df_multindex, level=1)
 cost revenue
Q1 A True True
 B True True
 C True True
Q2 A False True
 B True False
 C True False
```

```
nlargest(
 self,
 n: int,
 columns: IndexLabel,
 keep: NsmallestNlargestKeep = 'first'
) -> DataFrame from pandas.core.frame.DataFrame
Return the first `n` rows ordered by `columns` in descending order.
```

Return the first `n` rows with the largest values in `columns`, in descending order. The columns that are not specified are returned as well, but not used for ordering.

This method is equivalent to `df.sort_values(columns, ascending=False).head(n)`, but more performant.

Parameters

-----

`n` : int

Number of rows to return.

`columns` : Hashable or a sequence of the previous

Column label(s) to order by.

`keep` : {'first', 'last', 'all'}, default 'first'

Where there are duplicate values:

- 'first' : prioritize the first occurrence(s)
- 'last' : prioritize the last occurrence(s)

- ``all`` : keep all the ties of the smallest item even if it means selecting more than ``n`` items.

#### Returns

-----  
DataFrame

The first ``n`` rows ordered by the given columns in descending order.

#### See Also

-----

DataFrame.nsmallest : Return the first ``n`` rows ordered by ``columns`` in ascending order.

DataFrame.sort\_values : Sort DataFrame by the values.

DataFrame.head : Return the first ``n`` rows without re-ordering.

#### Notes

-----

This function cannot be used with all column types. For example, when specifying columns with ``object`` or ``category`` dtypes, ``TypeError`` is raised.

#### Examples

-----

```
>>> df = pd.DataFrame(
... {
... "population": [
... 59000000,
... 65000000,
... 434000,
... 434000,
... 434000,
... 337000,
... 11300,
... 11300,
... 11300,
...],
... "GDP": [1937894, 2583560, 12011, 4520, 12128, 17036, 182, 38, 311],
... "alpha-2": ["IT", "FR", "MT", "MV", "BN", "IS", "NR", "TV", "AI"],
... },
... index=[
... "Italy",
... "France",
... "Malta",
... "Maldives",
... "Brunei",
... "Iceland",
... "Nauru",
... "Tuvalu",
... "Anguilla",
...],
...)
>>> df
```

	population	GDP	alpha-2
Italy	59000000	1937894	IT
France	65000000	2583560	FR
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN
Iceland	337000	17036	IS
Nauru	11300	182	NR
Tuvalu	11300	38	TV
Anguilla	11300	311	AI

In the following example, we will use ``nlargest`` to select the three rows having the largest values in column "population".

```
>>> df.nlargest(3, "population")
 population GDP alpha-2
France 65000000 2583560 FR
Italy 59000000 1937894 IT
Malta 434000 12011 MT
```

When using ``keep='last'``, ties are resolved in reverse order:

```
>>> df.nlargest(3, "population", keep="last")
 population GDP alpha-2
France 65000000 2583560 FR
Italy 59000000 1937894 IT
Malta 434000 12011 MT
```

France	65000000	2583560	FR
Italy	59000000	1937894	IT
Brunei	434000	12128	BN

When using ``keep='all'``, the number of element kept can go beyond ``n`` if there are duplicate values for the smallest element, all the ties are kept:

```
>>> df.nlargest(3, "population", keep="all")
```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN

However, ``nlargest`` does not keep ``n`` distinct largest elements:

```
>>> df.nlargest(5, "population", keep="all")
```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN

To order by the largest values in column "population" and then "GDP", we can specify multiple columns like in the next example.

```
>>> df.nlargest(3, ["population", "GDP"])
```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Brunei	434000	12128	BN

`notna(self)` -> DataFrame from pandas.core.frame.DataFrame  
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or `:attr:`numpy.inf`` are not considered NA values. NA values, such as None or `:attr:`numpy.NaN``, get mapped to False values.

Returns

-----

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

-----

`Series.notnull` : Alias of `notna`.

`DataFrame.notnull` : Alias of `notna`.

`Series.isna` : Boolean inverse of `notna`.

`DataFrame.isna` : Boolean inverse of `notna`.

`Series.dropna` : Omit axes labels with missing values.

`DataFrame.dropna` : Omit axes labels with missing values.

`notna` : Top-level `notna`.

Examples

-----

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
... dict(
... age=[5, 6, np.nan],
... born=[
... pd.NaT,
... pd.Timestamp("1939-05-27"),
... pd.Timestamp("1940-04-25"),
...],
... name=["Alfred", "Batman", ""],
... toy=[None, "Batmobile", "Joker"],
...)
...)
>>> df
```

	age	born	name	toy
0	5.0	NaT	Alfred	NaN
1	6.0	1939-05-27	Batman	Batmobile
2	NaN	1940-04-25		Joker

```
>>> df.notna()
 age born name toy
0 True False True False
1 True True True True
2 False True True True
```

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0 5.0
1 6.0
2 NaN
dtype: float64
```

```
>>> ser.notna()
0 True
1 True
2 False
dtype: bool
```

`notnull(self)` -> DataFrame from `pandas.core.frame.DataFrame`  
 DataFrame.notnull is an alias for DataFrame.notna.

Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or `:attr:`numpy.inf`` are not considered NA values. NA values, such as `None` or `:attr:`numpy.NaN``, get mapped to False values.

Returns

-----  
 Series/DataFrame  
 Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

-----  
 Series.notnull : Alias of notna.  
 DataFrame.notnull : Alias of notna.  
 Series.isna : Boolean inverse of notna.  
 DataFrame.isna : Boolean inverse of notna.  
 Series.dropna : Omit axes labels with missing values.  
 DataFrame.dropna : Omit axes labels with missing values.  
 notna : Top-level notna.

Examples

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
... dict(
... age=[5, 6, np.nan],
... born=[
... pd.NaT,
... pd.Timestamp("1939-05-27"),
... pd.Timestamp("1940-04-25"),
...],
... name=["Alfred", "Batman", ""],
... toy=[None, "Batmobile", "Joker"],
...)
...)
>>> df
 age born name toy
0 5.0 NaT Alfred NaN
1 6.0 1939-05-27 Batman Batmobile
2 NaN 1940-04-25 Joker
```

```
>>> df.notnull()
 age born name toy
0 True True True True
1 True True True True
2 False True True True
```

```

0 True False True False
1 True True True True
2 False True True True

```

Show which entries in a Series are not NA.

```

>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0 5.0
1 6.0
2 NaN
dtype: float64

>>> ser.notnull()
0 True
1 True
2 False
dtype: bool

```

```

nsmallest(
 self,
 n: int,
 columns: IndexLabel,
 keep: NsmallestNlargestKeep = 'first'
) -> DataFrame from pandas.core.frame.DataFrame
Return the first `n` rows ordered by `columns` in ascending order.

```

Return the first `n` rows with the smallest values in `columns`, in ascending order. The columns that are not specified are returned as well, but not used for ordering.

This method is equivalent to ``df.sort\_values(columns, ascending=True).head(n)``, but more performant.

Parameters

```

n : int
 Number of items to retrieve.
columns : list or str
 Column name or names to order by.
keep : {'first', 'last', 'all'}, default 'first'
 Where there are duplicate values:

 - ``first`` : take the first occurrence.
 - ``last`` : take the last occurrence.
 - ``all`` : keep all the ties of the largest item even if it means
 selecting more than ``n`` items.

```

Returns

```

DataFrame
 DataFrame with the first `n` rows ordered by `columns` in ascending order.

```

See Also

```

DataFrame.nlargest : Return the first `n` rows ordered by `columns` in
 descending order.
DataFrame.sort_values : Sort DataFrame by the values.
DataFrame.head : Return the first `n` rows without re-ordering.

```

Examples

```

>>> df = pd.DataFrame(
... {
... "population": [
... 59000000,
... 65000000,
... 434000,
... 434000,
... 434000,
... 337000,
... 337000,
... 11300,
... 11300,
...],
... "GDP": [1937894, 2583560, 12011, 4520, 12128, 17036, 182, 38, 311],
... }
...)

```

```

... "alpha-2": ["IT", "FR", "MT", "MV", "BN", "IS", "NR", "TV", "AI"],
... },
... index=[
... "Italy",
... "France",
... "Malta",
... "Maldives",
... "Brunei",
... "Iceland",
... "Nauru",
... "Tuvalu",
... "Anguilla",
...],
...)
>>> df

```

	population	GDP	alpha-2
Italy	59000000	1937894	IT
France	65000000	2583560	FR
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN
Iceland	337000	17036	IS
Nauru	337000	182	NR
Tuvalu	11300	38	TV
Anguilla	11300	311	AI

In the following example, we will use ``nsmallest`` to select the three rows having the smallest values in column "population".

```

>>> df.nsmallest(3, "population")
 population GDP alpha-2
Tuvalu 11300 38 TV
Anguilla 11300 311 AI
Iceland 337000 17036 IS

```

When using ``keep='last'``, ties are resolved in reverse order:

```

>>> df.nsmallest(3, "population", keep="last")
 population GDP alpha-2
Anguilla 11300 311 AI
Tuvalu 11300 38 TV
Nauru 337000 182 NR

```

When using ``keep='all'``, the number of element kept can go beyond ``n`` if there are duplicate values for the largest element, all the ties are kept.

```

>>> df.nsmallest(3, "population", keep="all")
 population GDP alpha-2
Tuvalu 11300 38 TV
Anguilla 11300 311 AI
Iceland 337000 17036 IS
Nauru 337000 182 NR

```

However, ``nsmallest`` does not keep ``n`` distinct smallest elements:

```

>>> df.nsmallest(4, "population", keep="all")
 population GDP alpha-2
Tuvalu 11300 38 TV
Anguilla 11300 311 AI
Iceland 337000 17036 IS
Nauru 337000 182 NR

```

To order by the smallest values in column "population" and then "GDP", we can specify multiple columns like in the next example.

```

>>> df.nsmallest(3, ["population", "GDP"])
 population GDP alpha-2
Tuvalu 11300 38 TV
Anguilla 11300 311 AI
Nauru 337000 182 NR

```

nunique(self, axis: Axis = 0, dropna: bool = True) -> Series from pandas.core.frame.DataFrame  
Count number of distinct elements in specified axis.

Return Series with number of distinct elements. Can ignore NaN

values.

#### Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0  
The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.  
dropna : bool, default True  
Don't include NaN in the counts.

#### Returns

##### Series

Series with counts of unique values per row or column, depending on `axis`.

#### See Also

Series.nunique: Method nunique for Series.  
DataFrame.count: Count non-NA cells for each column or row.

#### Examples

```
>>> df = pd.DataFrame({"A": [4, 5, 6], "B": [4, 1, 1]})
>>> df.nunique()
A 3
B 2
dtype: int64

>>> df.nunique(axis=1)
0 1
1 2
2 2
dtype: int64
```

`pivot(self, *, columns, index=<no_default>, values=<no_default>)` -> DataFrame from pandas.com  
Return reshaped DataFrame organized by given index / column values.

Reshape data (produce a "pivot" table) based on column values. Uses unique values from specified `index` / `columns` to form axes of the resulting DataFrame. This function does not support data aggregation, multiple values will result in a MultiIndex in the columns. See the :ref:`User Guide <reshaping>` for more on reshaping.

#### Parameters

columns : Hashable or a sequence of the previous  
Column to use to make new frame's columns.  
index : Hashable or a sequence of the previous, optional  
Column to use to make new frame's index. If not given, uses existing index.  
values : Hashable or a sequence of the previous, optional  
Column(s) to use for populating new frame's values. If not specified, all remaining columns will be used and the result will have hierarchically indexed columns.

#### Returns

##### DataFrame

Returns reshaped DataFrame.

#### Raises

##### ValueError:

When there are any `index`, `columns` combinations with multiple values. `DataFrame.pivot\_table` when you need to aggregate.

#### See Also

DataFrame.pivot\_table : Generalization of pivot that can handle duplicate values for one index/column pair.  
DataFrame.unstack : Pivot based on the index values instead of a column.  
wide\_to\_long : Wide panel to long format. Less flexible but more user-friendly than melt.

#### Notes

For finer-tuned control, see hierarchical indexing documentation along





```

 foo bar baz
0 one A 1
1 one A 2
2 two B 3
3 two C 4

```

Notice that the first two rows are the same for our `index` and `columns` arguments.

```

>>> df.pivot(index='foo', columns='bar', values='baz')
Traceback (most recent call last):

```

```

...
ValueError: Index contains duplicate entries, cannot reshape

```

```

pivot_table(
 self,
 values=None,
 index=None,
 columns=None,
 aggfunc: AggFuncType = 'mean',
 fill_value=None,
 margins: bool = False,
 dropna: bool = True,
 margins_name: Level = 'All',
 observed: bool = True,
 sort: bool = True,
 **kwargs
)

```

```

-> DataFrame from pandas.core.frame.DataFrame
Create a spreadsheet-style pivot table as a DataFrame.

```

The levels in the pivot table will be stored in MultiIndex objects (hierarchical indexes) on the index and columns of the result DataFrame.

#### Parameters

```

values : list-like or scalar, optional
 Column or columns to aggregate.
index : column, Grouper, array, or sequence of the previous
 Keys to group by on the pivot table index. If a list is passed,
 it can contain any of the other types (except list). If an array is
 passed, it must be the same length as the data and will be used in
 the same manner as column values.
columns : column, Grouper, array, or sequence of the previous
 Keys to group by on the pivot table column. If a list is passed,
 it can contain any of the other types (except list). If an array is
 passed, it must be the same length as the data and will be used in
 the same manner as column values.
aggfunc : function, list of functions, dict, default "mean"
 If a list of functions is passed, the resulting pivot table will have
 hierarchical columns whose top level are the function names
 (inferred from the function objects themselves).
 If a dict is passed, the key is column to aggregate and the value is
 function or list of functions. If ``margin=True``, aggfunc will be
 used to calculate the partial aggregates.
fill_value : scalar, default None
 Value to replace missing values with (in the resulting pivot table,
 after aggregation).
margins : bool, default False
 If ``margins=True``, special ``All`` columns and rows
 will be added with partial group aggregates across the categories
 on the rows and columns.
dropna : bool, default True
 Do not include columns whose entries are all NaN. If True,

 * rows with an NA value in any column will be omitted before computing
 margins,
 * index/column keys containing NA values will be dropped (see ``dropna``
 parameter in :meth:`DataFrame.groupby`).

margins_name : str, default 'All'
 Name of the row / column that will contain the totals
 when margins is True.
observed : bool, default False
 This only applies if any of the groupers are Categoricals.
 If True: only show observed values for categorical groupers.
 If False: show all values for categorical groupers.

```

.. versionchanged:: 3.0.0

The default value is now ``True``.

sort : bool, default True  
Specifies if the result should be sorted.

\*\*kwargs : dict  
Optional keyword arguments to pass to ``aggfunc``.

Returns

-----  
DataFrame  
An Excel style pivot table.

See Also

-----  
DataFrame.pivot : Pivot without aggregation that can handle non-numeric data.  
DataFrame.melt: Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.  
wide\_to\_long : Wide panel to long format. Less flexible but more user-friendly than melt.

Notes

-----  
Reference :ref:`the user guide <reshaping.pivot>` for more examples.

Examples

-----  
>>> df = pd.DataFrame({"A": ["foo", "foo", "foo", "foo", "foo",  
... "bar", "bar", "bar", "bar"],  
... "B": ["one", "one", "one", "two", "two",  
... "one", "one", "two", "two"],  
... "C": ["small", "large", "large", "small",  
... "small", "large", "small", "small",  
... "large"],  
... "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],  
... "E": [2, 4, 5, 5, 6, 6, 8, 9, 9]})  
>>> df  
 A B C D E  
0 foo one small 1 2  
1 foo one large 2 4  
2 foo one large 2 5  
3 foo two small 3 5  
4 foo two small 3 6  
5 bar one large 4 6  
6 bar one small 5 8  
7 bar two small 6 9  
8 bar two large 7 9

This first example aggregates values by taking the sum.

>>> table = pd.pivot\_table(df, values='D', index=['A', 'B'],  
... columns=['C'], aggfunc="sum")  
>>> table  
 C large small  
A B  
bar one 4.0 5.0  
 two 7.0 6.0  
foo one 4.0 1.0  
 two NaN 6.0

We can also fill missing values using the ``fill\_value`` parameter.

>>> table = pd.pivot\_table(df, values='D', index=['A', 'B'],  
... columns=['C'], aggfunc="sum", fill\_value=0)  
>>> table  
 C large small  
A B  
bar one 4 5  
 two 7 6  
foo one 4 1  
 two 0 6

The next example aggregates by taking the mean across multiple columns.

```
>>> table = pd.pivot_table(df, values=['D', 'E'], index=['A', 'C'],
... aggfunc={'D': "mean", 'E': "mean"})
>>> table
```

		D	E
A	C		
bar	large	5.500000	7.500000
	small	5.500000	8.500000
foo	large	2.000000	4.500000
	small	2.333333	4.333333

We can also calculate multiple types of aggregations for any given value column.

```
>>> table = pd.pivot_table(df, values=['D', 'E'], index=['A', 'C'],
... aggfunc={'D': "mean",
... 'E': ["min", "max", "mean"]})
>>> table
```

		D		E	
		mean	max	mean	min
A	C				
bar	large	5.500000	9	7.500000	6
	small	5.500000	9	8.500000	8
foo	large	2.000000	5	4.500000	4
	small	2.333333	6	4.333333	2

`pop(self, item: Hashable) -> Series` from `pandas.core.frame.DataFrame`  
Return item and drop it from DataFrame. Raise `KeyError` if not found.

Parameters

`item` : label  
Label of column to be popped.

Returns

Series  
Series representing the item that is dropped.

See Also

`DataFrame.drop`: Drop specified labels from rows or columns.  
`DataFrame.drop_duplicates`: Return DataFrame with duplicate rows removed.

Examples

```
>>> df = pd.DataFrame(
... [
... ("falcon", "bird", 389.0),
... ("parrot", "bird", 24.0),
... ("lion", "mammal", 80.5),
... ("monkey", "mammal", np.nan),
...],
... columns=("name", "class", "max_speed"),
...)
>>> df
```

	name	class	max_speed
0	falcon	bird	389.0
1	parrot	bird	24.0
2	lion	mammal	80.5
3	monkey	mammal	NaN

```
>>> df.pop("class")
0 bird
1 bird
2 mammal
3 mammal
Name: class, dtype: str
```

```
>>> df
```

	name	max_speed
0	falcon	389.0
1	parrot	24.0
2	lion	80.5
3	monkey	NaN

`pow(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from `pandas`  
Get Exponential power of dataframe and other, element-wise (binary operator ``pow``).

Equivalent to `dataframe ** other`, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, `rpow`.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns.  
(1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
Broadcast across a level, matching Index values on the  
passed MultiIndex level.  
`fill_value` : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for  
successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing  
the result will be missing.

#### Returns

DataFrame  
Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

#### Notes

Mismatched indices will be unioned together.

#### Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
 angles degrees
circle 0 360
triangle 3 180
rectangle 4 360
```

Add a scalar with operator version which return the same results.

```
>>> df + 1
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361

>>> df.add(1)
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361
```

Divide by constant with reverse version.

```
>>> df.div(10)
 angles degrees
circle 0.0 36.0
triangle 0.3 18.0
rectangle 0.4 36.0
```

```
>>> df.rdiv(10)
 angles degrees
circle inf 0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
```

```

 hexagon 6 720

>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81

prod(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 min_count: int = 0,
 **kwargs
) -> Series from pandas.core.frame.DataFrame
 Return the product of the values over the requested axis.

Parameters

axis : {index (0), columns (1)}
 Axis for the function to be applied on.
 For `Series` this parameter is unused and defaults to 0.

 .. warning::

 The behavior of DataFrame.prod with ``axis=None`` is deprecated,
 in a future version this will reduce over both axes and return a scalar
 To retain the old behavior, pass axis=0 (or do not pass axis).

 .. versionadded:: 2.0.0

skipna : bool, default True
 Exclude NA/null values when computing the result.
numeric_only : bool, default False
 Include only float, int, boolean columns. Not implemented for Series.

min_count : int, default 0
 The required number of valid values to perform the operation. If fewer than
 ``min_count`` non-NA values are present the result will be NA.
**kwargs
 Additional keyword arguments to be passed to the function.

Returns

Series or scalar
 The product of the values over the requested axis.

See Also

Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

By default, the product of an empty or all-NA Series is ``1``

```

```
>>> pd.Series([], dtype="float64").prod()
1.0
```

This can be controlled with the ``min\_count`` parameter

```
>>> pd.Series([], dtype="float64").prod(min_count=1)
nan
```

Thanks to the ``skipna`` parameter, ``min\_count`` handles all-NA and empty series identically.

```
>>> pd.Series([np.nan]).prod()
1.0
```

```
>>> pd.Series([np.nan]).prod(min_count=1)
nan
```

```
product = prod(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 min_count: int = 0,
 **kwargs
) -> Series
```

```
quantile(
 self,
 q: float | AnyArrayLike | Sequence[float] = 0.5,
 axis: Axis = 0,
 numeric_only: bool = False,
 interpolation: QuantileInterpolation = 'linear',
 method: Literal['single', 'table'] = 'single'
) -> Series | DataFrame from pandas.core.frame.DataFrame
Return values at the given quantile over requested axis.
```

#### Parameters

```
q : float or array-like, default 0.5 (50% quantile)
 Value between 0 <= q <= 1, the quantile(s) to compute.
axis : {0 or 'index', 1 or 'columns'}, default 0
 Equals 0 or 'index' for row-wise, 1 or 'columns' for column-wise.
numeric_only : bool, default False
 Include only `float`, `int` or `boolean` data.
```

```
.. versionchanged:: 2.0.0
 The default value of ``numeric_only`` is now ``False``.
```

```
interpolation : {'linear', 'lower', 'higher', 'midpoint', 'nearest'}
 This optional parameter specifies the interpolation method to use,
 when the desired quantile lies between two data points `i` and `j`:
```

```
* linear: `(i + (j - i) * fraction)` where `fraction` is the
 fractional part of the index surrounded by `i` and `j`.
* lower: `i`.
* higher: `j`.
* nearest: `i` or `j` whichever is nearest.
* midpoint: `(i + j) / 2.
```

```
method : {'single', 'table'}, default 'single'
 Whether to compute quantiles per-column ('single') or over all columns
 ('table'). When 'table', the only allowed interpolation methods are
 'nearest', 'lower', and 'higher'.
```

#### Returns

```
Series or DataFrame
```

```
If ``q`` is an array, a DataFrame will be returned where the
 index is ``q``, the columns are the columns of self, and the
 values are the quantiles.
```

```
If ``q`` is a float, a Series will be returned where the
 index is the columns of self and the values are the quantiles.
```

#### See Also

```

```

core.window.rolling.Rolling.quantile: Rolling quantile.  
numpy.percentile: Numpy function to compute the percentile.

#### Examples

```

>>> df = pd.DataFrame(
... np.array([[1, 1], [2, 10], [3, 100], [4, 100]]), columns=["a", "b"]
...)
>>> df.quantile(0.1)
a 1.3
b 3.7
Name: 0.1, dtype: float64
>>> df.quantile([0.1, 0.5])
 a b
0.1 1.3 3.7
0.5 2.5 55.0
```

Specifying `method='table'` will compute the quantile over all columns.

```
>>> df.quantile(0.1, method="table", interpolation="nearest")
a 1
b 1
Name: 0.1, dtype: int64
>>> df.quantile([0.1, 0.5], method="table", interpolation="nearest")
 a b
0.1 1 1
0.5 3 100
```

Specifying `numeric\_only=False` will compute the quantiles for all columns.

```
>>> df = pd.DataFrame(
... {
... "A": [1, 2],
... "B": [pd.Timestamp("2010"), pd.Timestamp("2011")],
... "C": [pd.Timedelta("1 days"), pd.Timedelta("2 days")],
... }
...)
>>> df.quantile(0.5, numeric_only=False)
A 1.5
B 2010-07-02 12:00:00
C 1 days 12:00:00
Name: 0.5, dtype: object
```

```
query(
 self,
 expr: str,
 *,
 parser: Literal['pandas', 'python'] = 'pandas',
 engine: Literal['python', 'numexpr'] | None = None,
 local_dict: dict[str, Any] | None = None,
 global_dict: dict[str, Any] | None = None,
 resolvers: list[Mapping] | None = None,
 level: int = 0,
 inplace: bool = False
) -> DataFrame | None from pandas.core.frame.DataFrame
Query the columns of a DataFrame with a boolean expression.
```

.. warning::

This method can run arbitrary code which can make you vulnerable to code injection if you pass user input to this function.

#### Parameters

-----  
expr : str  
The query string to evaluate.

See the documentation for :func:`eval` for details of supported operations and functions in the query string.

See the documentation for :meth:`DataFrame.eval` for details on referring to column names and variables in the query string.

parser : {'pandas', 'python'}, default 'pandas'

The parser to use to construct the syntax tree from the expression. The default of ``'pandas'`` parses code slightly different than standard Python. Alternatively, you can parse an expression using the



``python`` parser to retain strict Python semantics. See the :ref:`enhancing performance <enhancingperf.eval>` documentation for more details.

engine : {'python', 'numexpr'}, default 'numexpr'

The engine used to evaluate the expression. Supported engines are

- None : tries to use ``numexpr``, falls back to ``python``
- ``numexpr`` : This default engine evaluates pandas objects using numexpr for large speed ups in complex expressions with large frames.
- ``python`` : Performs operations as if you had ``eval``'d in top level python. This engine is generally not that useful.

More backends may be available in the future.

local\_dict : dict or None, optional

A dictionary of local variables, taken from locals() by default.

global\_dict : dict or None, optional

A dictionary of global variables, taken from globals() by default.

resolvers : list of dict-like or None, optional

A list of objects implementing the ``\_\_getitem\_\_`` special method that you can use to inject an additional collection of namespaces to use for variable lookup. For example, this is used in the

:meth:`~DataFrame.query` method to inject the

``DataFrame.index`` and ``DataFrame.columns``

variables that refer to their respective :class:`~pandas.DataFrame` instance attributes.

level : int, optional

The number of prior stack frames to traverse and add to the current scope. Most users will **not** need to change this parameter.

inplace : bool

Whether to modify the DataFrame rather than creating a new one.

Returns

-----  
DataFrame or None

DataFrame resulting from the provided query expression or

None if ``inplace=True``.

See Also

-----  
eval : Evaluate a string describing operations on DataFrame columns.

DataFrame.eval : Evaluate a string describing operations on DataFrame columns.

Notes

-----  
The result of the evaluation of this expression is first passed to :attr:`~DataFrame.loc` and if that fails because of a multidimensional key (e.g., a DataFrame) then the result will be passed to :meth:`~DataFrame.\_\_getitem\_\_`.

This method uses the top-level :func:`eval` function to evaluate the passed query.

The :meth:`~pandas.DataFrame.query` method uses a slightly modified Python syntax by default. For example, the ``&`` and ``|`` (bitwise) operators have the precedence of their boolean cousins, :keyword:`and` and :keyword:`or`. This *is* syntactically valid Python, however the semantics are different.

You can change the semantics of the expression by passing the keyword argument ``parser='python'``. This enforces the same semantics as evaluation in Python space. Likewise, you can pass ``engine='python'`` to evaluate an expression using Python itself as a backend. This is not recommended as it is inefficient compared to using ``numexpr`` as the engine.

The :attr:`~DataFrame.index` and

:attr:`~DataFrame.columns` attributes of the

:class:`~pandas.DataFrame` instance are placed in the query namespace by default, which allows you to treat both the index and columns of the frame as a column in the frame.

The identifier ``index`` is used for the frame index; you can also use the name of the index to identify it in a query. Please note that Python keywords may not be used as identifiers.

For further details and examples see the ``query`` documentation in :ref:`indexing <indexing.query>`.

#### \*Backtick quoted variables\*

Backtick quoted variables are parsed as literal Python code and are converted internally to a Python valid identifier. This can lead to the following problems.

During parsing a number of disallowed characters inside the backtick quoted string are replaced by strings that are allowed as a Python identifier. These characters include all operators in Python, the space character, the question mark, the exclamation mark, the dollar sign, and the euro sign.

A backtick can be escaped by double backticks.

See also the `Python documentation about lexical analysis <[https://docs.python.org/3/reference/lexical\\_analysis.html](https://docs.python.org/3/reference/lexical_analysis.html)>`\_\_ in combination with the source code in :mod:`pandas.core.computation.parsing`.

#### Examples

```

>>> df = pd.DataFrame(
... {"A": range(1, 6), "B": range(10, 0, -2), "C&C": range(10, 5, -1)}
...)
>>> df
 A B C&C
0 1 10 10
1 2 8 9
2 3 6 8
3 4 4 7
4 5 2 6
>>> df.query("A > B")
 A B C&C
4 5 2 6
```

The previous expression is equivalent to

```
>>> df[df.A > df.B]
 A B C&C
4 5 2 6
```

For columns with spaces in their name, you can use backtick quoting.

```
>>> df.query("B == `C&C`")
 A B C&C
0 1 10 10
```

The previous expression is equivalent to

```
>>> df[df.B == df["C&C"]]
 A B C&C
0 1 10 10
```

Using local variable:

```
>>> local_var = 2
>>> df.query("A <= @local_var")
 A B C&C
0 1 10 10
1 2 8 9
```

radd(self, other, axis: Axis = 'columns', level=None, fill\_value=None) -> DataFrame from pandas  
Get Addition of dataframe and other, element-wise (binary operator `radd`).

Equivalent to ``other + dataframe``, but with support to substitute a fill\_value for missing data in one of the inputs. With reverse version, `add`.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

#### Parameters

```

other : scalar, sequence, Series, dict or DataFrame
 Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
 Whether to compare by the index (0 or 'index') or columns.
```

(1 or 'columns'). For Series input, axis to match Series index on.  
level : int or label  
Broadcast across a level, matching Index values on the  
passed MultiIndex level.  
fill\_value : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for  
successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing  
the result will be missing.

#### Returns

-----  
DataFrame

Result of the arithmetic operation.

#### See Also

-----  
DataFrame.add : Add DataFrames.  
DataFrame.sub : Subtract DataFrames.  
DataFrame.mul : Multiply DataFrames.  
DataFrame.div : Divide DataFrames (float division).  
DataFrame.truediv : Divide DataFrames (float division).  
DataFrame.floordiv : Divide DataFrames (integer division).  
DataFrame.mod : Calculate modulo (remainder after division).  
DataFrame.pow : Calculate exponential power.

#### Notes

-----  
Mismatched indices will be unioned together.

#### Examples

-----  
>>> df = pd.DataFrame({'angles': [0, 3, 4],  
... 'degrees': [360, 180, 360]},  
... index=['circle', 'triangle', 'rectangle'])  
>>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same  
results.

>>> df + 1

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

>>> df.add(1)

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

>>> df.div(10)

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

>>> df.rdiv(10)

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

>>> df - [1, 2]

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359

Multiply a dictionary by axis.

>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080

Multiply a DataFrame of different shape with operator version.

>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN

>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0

Divide by a MultiIndex by level.

>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720

>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})

```

```
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

```
rdiv = rtruediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame
```

```
reindex(
 self,
 labels=None,
 *,
 index=None,
 columns=None,
 axis: Axis | None = None,
 method: ReindexMethod | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 level: Level | None = None,
 fill_value: Scalar | None = nan,
 limit: int | None = None,
 tolerance=None
)
```

```
-> DataFrame from pandas.core.frame.DataFrame
Conform DataFrame to new index with optional filling logic.
```

Places NA/NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and ``copy=False``.

Parameters  
-----

labels : array-like, optional  
New labels / index to conform the axis specified by 'axis' to.

index : array-like, optional  
New labels for the index. Preferably an Index object to avoid duplicating data.

columns : array-like, optional  
New labels for the columns. Preferably an Index object to avoid duplicating data.

axis : int or str, optional  
Axis to target. Can be either the axis name ('index', 'columns') or number (0, 1).

method : {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}  
Method to use for filling holes in reindexed DataFrame.  
Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

- \* None (default): don't fill gaps
- \* pad / ffill: Propagate last valid observation forward to next valid.
- \* backfill / bfill: Use next valid observation to fill gap.
- \* nearest: Use nearest valid observations to fill gap.

copy : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`\_ for more details.

level : int or name  
Broadcast across a level, matching Index values on the passed MultiIndex level.

fill\_value : scalar, default np.nan  
Value to use for missing values. Defaults to NaN, but can be any "compatible" value.

limit : int, default None  
Maximum number of consecutive elements to forward or backward fill.

tolerance : optional  
Maximum distance between original and new labels for inexact

matches. The values of the index at the matching locations most satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

-----

DataFrame

DataFrame with changed index.

See Also

-----

DataFrame.set\_index : Set row labels.

DataFrame.reset\_index : Remove row labels or move them to new columns.

DataFrame.reindex\_like : Change to same indices as other DataFrame.

Examples

-----

`DataFrame.reindex` supports two calling conventions

\* `DataFrame(index=index_labels, columns=column_labels, ...)`

\* `DataFrame(labels, axis={'index', 'columns'}, ...)`

We *highly* recommend using keyword arguments to clarify your intent.

Create a DataFrame with some fictional data.

```
>>> index = ["Firefox", "Chrome", "Safari", "IE10", "Konqueror"]
>>> columns = ["http_status", "response_time"]
>>> df = pd.DataFrame(
... [[200, 0.04], [200, 0.02], [404, 0.07], [404, 0.08], [301, 1.0]],
... columns=columns,
... index=index,
...)
>>> df
```

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

Create a new index and reindex the DataFrame. By default values in the new index that do not have corresponding records in the DataFrame are assigned `NaN`.

```
>>> new_index = ["Safari", "Iceweasel", "Comodo Dragon", "IE10", "Chrome"]
>>> df.reindex(new_index)
```

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02

We can fill in the missing values by passing a value to the keyword `fill_value`. Because the index is not monotonically increasing or decreasing, we cannot use arguments to the keyword `method` to fill the `NaN` values.

```
>>> df.reindex(new_index, fill_value=0)
```

	http_status	response_time
Safari	404	0.07
Iceweasel	0	0.00
Comodo Dragon	0	0.00
IE10	404	0.08
Chrome	200	0.02

```
>>> df.reindex(new_index, fill_value="missing")
```

	http_status	response_time
Safari	404	0.07
Iceweasel	missing	missing

Comodo Dragon	missing	missing
IE10	404	0.08
Chrome	200	0.02

We can also reindex the columns.

```
>>> df.reindex(columns=["http_status", "user_agent"])
 http_status user_agent
Firefox 200 NaN
Chrome 200 NaN
Safari 404 NaN
IE10 404 NaN
Konqueror 301 NaN
```

Or we can use "axis-style" keyword arguments

```
>>> df.reindex(["http_status", "user_agent"], axis="columns")
 http_status user_agent
Firefox 200 NaN
Chrome 200 NaN
Safari 404 NaN
IE10 404 NaN
Konqueror 301 NaN
```

To further illustrate the filling functionality in ``reindex``, we will create a DataFrame with a monotonically increasing index (for example, a sequence of dates).

```
>>> date_index = pd.date_range("1/1/2010", periods=6, freq="D")
>>> df2 = pd.DataFrame(
... {"prices": [100, 101, np.nan, 100, 89, 88]}, index=date_index
...)
>>> df2
 prices
2010-01-01 100.0
2010-01-02 101.0
2010-01-03 NaN
2010-01-04 100.0
2010-01-05 89.0
2010-01-06 88.0
```

Suppose we decide to expand the DataFrame to cover a wider date range.

```
>>> date_index2 = pd.date_range("12/29/2009", periods=10, freq="D")
>>> df2.reindex(date_index2)
 prices
2009-12-29 NaN
2009-12-30 NaN
2009-12-31 NaN
2010-01-01 100.0
2010-01-02 101.0
2010-01-03 NaN
2010-01-04 100.0
2010-01-05 89.0
2010-01-06 88.0
2010-01-07 NaN
```

The index entries that did not have a value in the original data frame (for example, '2009-12-29') are by default filled with ``NaN``. If desired, we can fill in the missing values using one of several options.

For example, to back-propagate the last valid value to fill the ``NaN`` values, pass ``bfill`` as an argument to the ``method`` keyword.

```
>>> df2.reindex(date_index2, method="bfill")
 prices
2009-12-29 100.0
2009-12-30 100.0
2009-12-31 100.0
2010-01-01 100.0
2010-01-02 101.0
2010-01-03 NaN
2010-01-04 100.0
2010-01-05 89.0
```

```
2010-01-06 88.0
2010-01-07 NaN
```

Please note that the ``NaN`` value present in the original DataFrame (at index value 2010-01-03) will not be filled by any of the value propagation schemes. This is because filling while reindexing does not look at DataFrame values, but only compares the original and desired indexes. If you do want to fill in the ``NaN`` values present in the original DataFrame, use the ``fillna()`` method.

See the :ref:`user guide <basics.reindexing>` for more.

```
rename(
 self,
 mapper: Renamer | None = None,
 *,
 index: Renamer | None = None,
 columns: Renamer | None = None,
 axis: Axis | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 inplace: bool = False,
 level: Level | None = None,
 errors: IgnoreRaise = 'ignore'
) -> DataFrame | None from pandas.core.frame.DataFrame
Rename columns or index labels.
```

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

See the :ref:`user guide <basics.rename>` for more.

#### Parameters

**mapper** : dict-like or function  
Dict-like or function transformations to apply to that axis' values. Use either ``mapper`` and ``axis`` to specify the axis to target with ``mapper``, or ``index`` and ``columns``.

**index** : dict-like or function  
Alternative to specifying axis (``mapper, axis=0`` is equivalent to ``index=mapper``).

**columns** : dict-like or function  
Alternative to specifying axis (``mapper, axis=1`` is equivalent to ``columns=mapper``).

**axis** : {0 or 'index', 1 or 'columns'}, default 0  
Axis to target with ``mapper``. Can be either the axis name ('index', 'columns') or number (0, 1). The default is 'index'.

**copy** : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_ for more details.

**inplace** : bool, default False  
Whether to modify the DataFrame rather than creating a new one. If True then value of copy is ignored.

**level** : int or level name, default None  
In case of a MultiIndex, only rename labels in the specified level.

**errors** : {'ignore', 'raise'}, default 'ignore'  
If 'raise', raise a ``KeyError`` when a dict-like ``mapper``, ``index``, or ``columns`` contains labels that are not present in the Index being transformed.  
If 'ignore', existing keys will be renamed and extra keys will be ignored.

#### Returns

DataFrame or None



DataFrame with the renamed axis labels or None if ``inplace=True``.

Raises

-----

KeyError

If any of the labels is not found in the selected axis and  
"errors='raise'".

See Also

-----

DataFrame.rename\_axis : Set the name of the axis.

Examples

-----

``DataFrame.rename`` supports two calling conventions

\* ``(index=index\_mapper, columns=columns\_mapper, ...)``

\* ``(mapper, axis={'index', 'columns'}, ...)``

We *highly* recommend using keyword arguments to clarify your intent.

Rename columns using a mapping:

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
```

```
>>> df.rename(columns={"A": "a", "B": "c"})
```

```
 a c
0 1 4
1 2 5
2 3 6
```

Rename index using a mapping:

```
>>> df.rename(index={0: "x", 1: "y", 2: "z"})
```

```
 A B
x 1 4
y 2 5
z 3 6
```

Cast index labels to a different type:

```
>>> df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

```
>>> df.rename(index=str).index
```

```
Index(['0', '1', '2'], dtype='str')
```

```
>>> df.rename(columns={"A": "a", "B": "b", "C": "c"}, errors="raise")
```

```
Traceback (most recent call last):
```

```
KeyError: ['C'] not found in axis
```

Using axis-style parameters:

```
>>> df.rename(str.lower, axis="columns")
```

```
 a b
0 1 4
1 2 5
2 3 6
```

```
>>> df.rename({1: 2, 2: 4}, axis="index")
```

```
 A B
0 1 4
2 2 5
4 3 6
```

reorder\_levels(self, order: Sequence[int | str], axis: Axis = 0) -> DataFrame from pandas.core  
Rearrange index or column levels using input ``order``.

May not drop or duplicate levels.

Parameters

-----

order : list of int or list of str

List representing new level order. Reference level by number  
(position) or by key (label).

axis : {0 or 'index', 1 or 'columns'}, default 0  
Where to reorder levels.

Returns

-----

DataFrame

DataFrame with indices or columns with reordered levels.

See Also

-----

DataFrame.swaplevel : Swap levels i and j in a MultiIndex.

Examples

-----

```
>>> data = {
... "class": ["Mammals", "Mammals", "Reptiles"],
... "diet": ["Omnivore", "Carnivore", "Carnivore"],
... "species": ["Humans", "Dogs", "Snakes"],
... }
>>> df = pd.DataFrame(data, columns=["class", "diet", "species"])
>>> df = df.set_index(["class", "diet"])
>>> df
```

		species
class	diet	
Mammals	Omnivore	Humans
	Carnivore	Dogs
Reptiles	Carnivore	Snakes

Let's reorder the levels of the index:

```
>>> df.reorder_levels(["diet", "class"])
>>> df
```

		species
diet	class	
Omnivore	Mammals	Humans
Carnivore	Mammals	Dogs
	Reptiles	Snakes

```
reset_index(
 self,
 level: IndexLabel | None = None,
 *,
 drop: bool = False,
 inplace: bool = False,
 col_level: Hashable = 0,
 col_fill: Hashable = '',
 allow_duplicates: bool | lib.NoDefault = <no_default>,
 names: Hashable | Sequence[Hashable] | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Reset the index, or a level of it.
```

Reset the index of the DataFrame, and use the default one instead.  
If the DataFrame has a MultiIndex, this method can remove one or more levels.

Parameters

-----

level : int, str, tuple, or list, default None

Only remove the given levels from the index. Removes all levels by default.

drop : bool, default False

Do not try to insert index into dataframe columns. This resets the index to the default integer index.

inplace : bool, default False

Whether to modify the DataFrame rather than creating a new one.

col\_level : int or str, default 0

If the columns have multiple levels, determines which level the labels are inserted into. By default it is inserted into the first level.

col\_fill : object, default ''

If the columns have multiple levels, determines how the other levels are named. If None then the index name is repeated.

allow\_duplicates : bool, optional, default lib.no\_default

Allow duplicate column labels to be created.

names : int, str or 1-dimensional list, default None

Using the given string, rename the DataFrame column which contains the index data. If the DataFrame has a MultiIndex, this has to be a list with length equal to the number of levels.

Returns

-----

DataFrame or None  
DataFrame with the new index or None if ``inplace=True``.

See Also

-----  
DataFrame.set\_index : Opposite of reset\_index.  
DataFrame.reindex : Change to new indices or expand indices.  
DataFrame.reindex\_like : Change to same indices as other DataFrame.

Examples

-----  
>>> df = pd.DataFrame(  
... [("bird", 389.0), ("bird", 24.0), ("mammal", 80.5), ("mammal", np.nan)],  
... index=["falcon", "parrot", "lion", "monkey"],  
... columns=("class", "max\_speed"),  
... )  
>>> df

	class	max_speed
falcon	bird	389.0
parrot	bird	24.0
lion	mammal	80.5
monkey	mammal	NaN

When we reset the index, the old index is added as a column, and a new sequential index is used:

>>> df.reset\_index()  
 index class max\_speed  
0 falcon bird 389.0  
1 parrot bird 24.0  
2 lion mammal 80.5  
3 monkey mammal NaN

We can use the `drop` parameter to avoid the old index being added as a column:

>>> df.reset\_index(drop=True)  
 class max\_speed  
0 bird 389.0  
1 bird 24.0  
2 mammal 80.5  
3 mammal NaN

You can also use `reset\_index` with `MultiIndex`.

>>> index = pd.MultiIndex.from\_tuples(  
... [  
... ("bird", "falcon"),  
... ("bird", "parrot"),  
... ("mammal", "lion"),  
... ("mammal", "monkey"),  
... ],  
... names=["class", "name"],  
... )  
>>> columns = pd.MultiIndex.from\_tuples([("speed", "max"), ("species", "type")])  
>>> df = pd.DataFrame(  
... [(389.0, "fly"), (24.0, "fly"), (80.5, "run"), (np.nan, "jump")],  
... index=index,  
... columns=columns,  
... )  
>>> df

		speed		species	
		max		type	
bird	falcon	389.0		fly	
	parrot	24.0		fly	
mammal	lion	80.5		run	
	monkey	NaN		jump	

Using the `names` parameter, choose a name for the index column:

>>> df.reset\_index(names=["classes", "names"])  
 classes names speed species  
 max type  
0 bird falcon 389.0 fly  
1 bird parrot 24.0 fly  
2 mammal lion 80.5 run

```
3 mammal monkey NaN jump
```

If the index has multiple levels, we can reset a subset of them:

```
>>> df.reset_index(level="class")
 class speed species
 max type
name
falcon bird 389.0 fly
parrot bird 24.0 fly
lion mammal 80.5 run
monkey mammal NaN jump
```

If we are not dropping the index, by default, it is placed in the top level. We can place it in another level:

```
>>> df.reset_index(level="class", col_level=1)
 class speed species
 max type
name
falcon bird 389.0 fly
parrot bird 24.0 fly
lion mammal 80.5 run
monkey mammal NaN jump
```

When the index is inserted under another level, we can specify under which one with the parameter ``col_fill``:

```
>>> df.reset_index(level="class", col_level=1, col_fill="species")
 species speed species
 class max type
name
falcon bird 389.0 fly
parrot bird 24.0 fly
lion mammal 80.5 run
monkey mammal NaN jump
```

If we specify a nonexistent level for ``col_fill``, it is created:

```
>>> df.reset_index(level="class", col_level=1, col_fill="genus")
 genus speed species
 class max type
name
falcon bird 389.0 fly
parrot bird 24.0 fly
lion mammal 80.5 run
monkey mammal NaN jump
```

```
rfloordiv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from
Get Integer division of dataframe and other, element-wise (binary operator `rfloordiv`).
```

Equivalent to ``other // dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``floordiv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

#### Parameters

```
other : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
level : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.
fill_value : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.
```

#### Returns

```
DataFrame
Result of the arithmetic operation.
```

See Also

-----

DataFrame.add : Add DataFrames.  
DataFrame.sub : Subtract DataFrames.  
DataFrame.mul : Multiply DataFrames.  
DataFrame.div : Divide DataFrames (float division).  
DataFrame.truediv : Divide DataFrames (float division).  
DataFrame.floordiv : Divide DataFrames (integer division).  
DataFrame.mod : Calculate modulo (remainder after division).  
DataFrame.pow : Calculate exponential power.

Notes

-----

Mismatched indices will be unioned together.

Examples

-----

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN

>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720

>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

`rmod(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas`  
Get Modulo of dataframe and other, element-wise (binary operator ``rmod``).

Equivalent to ``other % dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``mod``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to

arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns.  
(1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
Broadcast across a level, matching Index values on the  
passed MultiIndex level.  
`fill_value` : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for  
successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing  
the result will be missing.

#### Returns

##### DataFrame

Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

#### Notes

Mismatched indices will be unioned together.

#### Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN

>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720

>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
```



triangle	1.0	1.0
rectangle	1.0	1.0
B square	0.0	0.0
pentagon	0.0	0.0
hexagon	0.0	0.0

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
```

```
>>> df_pow.pow(2)
```

	A	B
0	4	36
1	9	49
2	16	64
3	25	81

`rmul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas  
Get Multiplication of dataframe and other, element-wise (binary operator ``rmul``).

Equivalent to ``other * dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``mul``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns.  
(1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
Broadcast across a level, matching Index values on the passed MultiIndex level.  
`fill_value` : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.  
If data in both corresponding DataFrame locations is missing the result will be missing.

#### Returns

DataFrame  
Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

#### Notes

Mismatched indices will be unioned together.

#### Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361

triangle	4	181
rectangle	5	361

```
>>> df.add(1)
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361
```

Divide by constant with reverse version.

```
>>> df.div(10)
 angles degrees
circle 0.0 36.0
triangle 0.3 18.0
rectangle 0.4 36.0
```

```
>>> df.rdiv(10)
 angles degrees
circle inf 0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358
```

```
>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4
```

```
>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
```

```
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

`round(self, decimals: int | dict[IndexLabel, int] | Series = 0, *args, **kwargs) -> DataFrame`  
 Round numeric columns in a DataFrame to a variable number of decimal places.

#### Parameters

**decimals** : int, dict, Series  
 Number of decimal places to round each column to. If an int is given, round each column to the same number of places. Otherwise dict and Series round to variable numbers of places. Column names should be in the keys if `decimals` is a dict-like, or in the index if `decimals` is a Series. Any columns not included in `decimals` will be left as is. Elements of `decimals` which are not columns of the input will be ignored.

#### \*args

Additional keywords have no effect but might be accepted for compatibility with numpy.

#### \*\*kwargs

Additional keywords have no effect but might be accepted for compatibility with numpy.

#### Returns

##### DataFrame

A DataFrame with the affected columns rounded to the specified number of decimal places.

#### See Also

`numpy.around` : Round a numpy array to the given number of decimals.  
`Series.round` : Round a Series to the given number of decimals.

#### Notes

For values exactly halfway between rounded decimal values, pandas rounds to the nearest even value (e.g. -0.5 and 0.5 round to 0.0, 1.5 and 2.5 round to 2.0, etc.).

#### Examples

```
>>> df = pd.DataFrame(
... [(0.21, 0.32), (0.01, 0.67), (0.66, 0.03), (0.21, 0.18)],
... columns=["dogs", "cats"],
...)
>>> df
 dogs cats
0 0.21 0.32
1 0.01 0.67
2 0.66 0.03
3 0.21 0.18
```

By providing an integer each column is rounded to the same number of decimal places

```
>>> df.round(1)
 dogs cats
0 0.2 0.3
1 0.0 0.7
2 0.7 0.0
3 0.2 0.2
```

With a dict, the number of places for specific columns can be specified with the column names as key and the number of decimal places as value

```
>>> df.round({"dogs": 1, "cats": 0})
 dogs cats
0 0.2 0.0
1 0.0 1.0
2 0.7 0.0
3 0.2 0.0
```

Using a Series, the number of places for specific columns can be specified with the column names as index and the number of decimal places as value

```
>>> decimals = pd.Series([0, 1], index=["cats", "dogs"])
>>> df.round(decimals)
 dogs cats
0 0.2 0.0
1 0.0 1.0
2 0.7 0.0
3 0.2 0.0
```

`rpow(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas  
Get Exponential power of dataframe and other, element-wise (binary operator ``rpow``).

Equivalent to ``other ** dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``pow``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
Any single or multiple element data structure, or list-like object.

`axis` : {0 or 'index', 1 or 'columns'}  
Whether to compare by the index (0 or 'index') or columns. (1 or 'columns'). For Series input, axis to match Series index on.

`level` : int or label  
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : float or None, default None  
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation. If data in both corresponding DataFrame locations is missing the result will be missing.

#### Returns

DataFrame  
Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.

```

DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

```

Notes

-----

Mismatched indices will be unioned together.

Examples

-----

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df

```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```

>>> df + 1

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```

>>> df.add(1)

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```

>>> df.div(10)

```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```

>>> df.rdiv(10)

```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```

>>> df - [1, 2]

```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub([1, 2], axis='columns')

```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')

```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```

>>> df.mul({'angles': 0, 'degrees': 2})

```

	angles	degrees
--	--------	---------

circle	0	720
triangle	0	360
rectangle	0	720

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4
```

```
>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

`rsub(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas`  
 Get Subtraction of dataframe and other, element-wise (binary operator ``rsub``).

Equivalent to ``other - dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``sub``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters  
 -----

other : scalar, sequence, Series, dict or DataFrame  
 Any single or multiple element data structure, or list-like object.  
 axis : {0 or 'index', 1 or 'columns'}  
 Whether to compare by the index (0 or 'index') or columns.  
 (1 or 'columns'). For Series input, axis to match Series index on.  
 level : int or label  
 Broadcast across a level, matching Index values on the  
 passed MultiIndex level.  
 fill\_value : float or None, default None  
 Fill existing missing (NaN) values, and any new element needed for  
 successful DataFrame alignment, with this value before computation.  
 If data in both corresponding DataFrame locations is missing  
 the result will be missing.

#### Returns

-----  
 DataFrame

Result of the arithmetic operation.

#### See Also

-----

DataFrame.add : Add DataFrames.  
 DataFrame.sub : Subtract DataFrames.  
 DataFrame.mul : Multiply DataFrames.  
 DataFrame.div : Divide DataFrames (float division).  
 DataFrame.truediv : Divide DataFrames (float division).  
 DataFrame.floordiv : Divide DataFrames (integer division).  
 DataFrame.mod : Calculate modulo (remainder after division).  
 DataFrame.pow : Calculate exponential power.

#### Notes

-----

Mismatched indices will be unioned together.

#### Examples

-----

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4
```

```
>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
```



```

 hexagon 0.0 0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81

```

`rtruediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from`  
 Get Floating division of dataframe and other, element-wise (binary operator ``rtruediv``).

Equivalent to ``other / dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``truediv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

**Parameters**  
 -----  
`other` : scalar, sequence, Series, dict or DataFrame  
 Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
 Whether to compare by the index (0 or 'index') or columns.  
 (1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
 Broadcast across a level, matching Index values on the  
 passed MultiIndex level.  
`fill_value` : float or None, default None  
 Fill existing missing (NaN) values, and any new element needed for  
 successful DataFrame alignment, with this value before computation.  
 If data in both corresponding DataFrame locations is missing  
 the result will be missing.

**Returns**  
 -----  
 DataFrame  
 Result of the arithmetic operation.

**See Also**  
 -----  
`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

**Notes**  
 -----  
 Mismatched indices will be unioned together.

**Examples**  
 -----  
 >>> df = pd.DataFrame({'angles': [0, 3, 4],  
 ... 'degrees': [360, 180, 360]},  
 ... index=['circle', 'triangle', 'rectangle'])  
 >>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```

>>> df + 1
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361

>>> df.add(1)

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
```

	angles	degrees
circle	0	720
triangle	0	360
rectangle	0	720

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
```

	angles	degrees
circle	0	0
triangle	6	360
rectangle	12	1080

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
```

	angles
circle	0
triangle	3
rectangle	4

```
>>> df * other
```

	angles	degrees
circle	0	NaN
triangle	9	NaN
rectangle	16	NaN

```
>>> df.mul(other, fill_value=0)
```

	angles	degrees
circle	0	0.0
triangle	9	0.0
rectangle	16	0.0

Divide by a MultiIndex by level.

```

>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
>>> df_multindex

```

	angles	degrees
A circle	0	360
triangle	3	180
rectangle	4	360
B square	4	360
pentagon	5	540
hexagon	6	720

```

>>> df.div(df_multindex, level=1, fill_value=0)

```

	angles	degrees
A circle	NaN	1.0
triangle	1.0	1.0
rectangle	1.0	1.0
B square	0.0	0.0
pentagon	0.0	0.0
hexagon	0.0	0.0

```

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)

```

	A	B
0	4	36
1	9	49
2	16	64
3	25	81

`select_dtypes(self, include=None, exclude=None) -> DataFrame` from `pandas.core.frame.DataFrame`  
 Return a subset of the DataFrame's columns based on the column dtypes.

This method allows for filtering columns based on their data types. It is useful when working with heterogeneous DataFrames where operations need to be performed on a specific subset of data types.

#### Parameters

`include, exclude` : scalar or list-like  
 A selection of dtypes or strings to be included/excluded. At least one of these parameters must be supplied.

#### Returns

`DataFrame`  
 The subset of the frame including the dtypes in `include` and excluding the dtypes in `exclude`.

#### Raises

`ValueError`  
 \* If both of `include` and `exclude` are empty  
 \* If `include` and `exclude` have overlapping elements  
`TypeError`  
 \* If any kind of string dtype is passed in.

#### See Also

`DataFrame.dtypes`: Return Series with the data type of each column.

#### Notes

\* To select all `numeric` types, use `np.number` or `'number'`  
 \* To select strings you must use the `object` dtype, but note that this will return *all* object dtype columns. With `pd.options.future.infer_string` enabled, using `"str"` will work to select all string columns.  
 \* See the `numpy` dtype hierarchy  
<https://numpy.org/doc/stable/reference/arrays.scalars.html>  
 \* To select datetimes, use `np.datetime64`, `'datetime'` or `'datetime64'`  
 \* To select timedeltas, use `np.timedelta64`, `'timedelta'` or `'timedelta64'`

\* To select Pandas categorical dtypes, use ``'category'``  
 \* To select Pandas datetimetz dtypes, use ``'datetimetz'``  
 or ``'datetime64[ns, tz]'``

#### Examples

```
>>> df = pd.DataFrame(
... {"a": [1, 2] * 3, "b": [True, False] * 3, "c": [1.0, 2.0] * 3}
...)
>>> df
```

```
 a b c
0 1 True 1.0
1 2 False 2.0
2 1 True 1.0
3 2 False 2.0
4 1 True 1.0
5 2 False 2.0
```

```
>>> df.select_dtypes(include="bool")
```

```
 b
0 True
1 False
2 True
3 False
4 True
5 False
```

```
>>> df.select_dtypes(include=["float64"])
```

```
 c
0 1.0
1 2.0
2 1.0
3 2.0
4 1.0
5 2.0
```

```
>>> df.select_dtypes(exclude=["int64"])
```

```
 b c
0 True 1.0
1 False 2.0
2 True 1.0
3 False 2.0
4 True 1.0
5 False 2.0
```

```
sem(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 ddof: int = 1,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return unbiased standard error of the mean over requested axis.
```

Normalized by N-1 by default. This can be changed using the ddof argument

#### Parameters

axis : {index (0), columns (1)}  
 For ``Series`` this parameter is unused and defaults to 0.

.. warning::

The behavior of `DataFrame.sem` with ``axis=None`` is deprecated,  
 in a future version this will reduce over both axes and return a scalar  
 To retain the old behavior, pass `axis=0` (or do not pass axis).

skipna : bool, default True

Exclude NA/null values. If an entire row/column is NA, the result  
 will be NA.

ddof : int, default 1

Delta Degrees of Freedom. The divisor used in calculations is `N - ddof`,  
 where `N` represents the number of elements.

numeric\_only : bool, default False

Include only float, int, boolean columns. Not implemented for Series.

**\*\*kwargs :**  
Additional keywords passed.

**Returns**

-----  
Series or DataFrame (if level specified)  
Unbiased standard error of the mean over requested axis.

**See Also**

-----  
DataFrame.var : Return unbiased variance over requested axis.  
DataFrame.std : Returns sample standard deviation over requested axis.

**Examples**

-----  
>>> s = pd.Series([1, 2, 3])  
>>> round(s.sem(), 6)  
0.57735

**With a DataFrame**

```
>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
 a b
tiger 1 2
zebra 2 3
>>> df.sem()
a 0.5
b 0.5
dtype: float64
```

**Using axis=1**

```
>>> df.sem(axis=1)
tiger 0.5
zebra 0.5
dtype: float64
```

In this case, `numeric_only` should be set to `True` to avoid getting an error.

```
>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.sem(numeric_only=True)
a 0.5
dtype: float64
```

```
set_axis(
 self,
 labels,
 *,
 axis: Axis = 0,
 copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
Assign desired index to given axis.
```

Indexes for column or row labels can be changed by assigning a list-like or Index.

**Parameters**

-----  
**labels :** list-like, Index  
The values for the new index.

**axis :** {0 or 'index', 1 or 'columns'}, default 0  
The axis to update. The value 0 identifies the rows. For `'Series'` this parameter is unused and defaults to 0.

**copy :** bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `'user guide on Copy-on-Write'`

[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_  
for more details.

#### Returns

-----  
DataFrame

An object of type DataFrame.

#### See Also

-----

DataFrame.rename\_axis : Alter the name of the index or columns.

#### Examples

-----

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
```

Change the row labels.

```
>>> df.set_axis(["a", "b", "c"], axis="index")
```

```
 A B
a 1 4
b 2 5
c 3 6
```

Change the column labels.

```
>>> df.set_axis(["I", "II"], axis="columns")
```

```
 I II
0 1 4
1 2 5
2 3 6
```

```
set_index(
 self,
 keys,
 *,
 drop: bool = True,
 append: bool = False,
 inplace: bool = False,
 verify_integrity: bool | lib.NoDefault = <no_default>
) -> DataFrame | None from pandas.core.frame.DataFrame
Set the DataFrame index using existing columns.
```

Set the DataFrame index (row labels) using one or more existing columns or arrays (of the correct length). The index can replace the existing index or expand on it.

#### Parameters

-----

keys : label or array-like or list of labels/arrays

This parameter can be either a single column key, a single array of the same length as the calling DataFrame, or a list containing an arbitrary combination of column keys and arrays. Here, "array" encompasses :class:`Series`, :class:`Index`, ``np.ndarray``, and instances of :class:`~collections.abc.Iterator`.

drop : bool, default True

Delete columns to be used as the new index.

append : bool, default False

Whether to append columns to existing index.

Setting to True will add the new columns to existing index.

When set to False, the current index will be dropped from the DataFrame.

inplace : bool, default False

Whether to modify the DataFrame rather than creating a new one.

verify\_integrity : bool, default False

Check the new index for duplicates. Otherwise defer the check until necessary. Setting to False will improve the performance of this method.

.. deprecated:: 3.0.0

#### Returns

-----

DataFrame or None

Changed row labels or None if ``inplace=True``.

#### See Also

-----

DataFrame.reset\_index : Opposite of set\_index.  
DataFrame.reindex : Change to new indices or expand indices.  
DataFrame.reindex\_like : Change to same indices as other DataFrame.

#### Examples

```
>>> df = pd.DataFrame(
... {
... "month": [1, 4, 7, 10],
... "year": [2012, 2014, 2013, 2014],
... "sale": [55, 40, 84, 31],
... }
...)
>>> df
```

	month	year	sale
0	1	2012	55
1	4	2014	40
2	7	2013	84
3	10	2014	31

Set the index to become the 'month' column:

```
>>> df.set_index("month")
```

	year	sale
month		
1	2012	55
4	2014	40
7	2013	84
10	2014	31

Create a MultiIndex using columns 'year' and 'month':

```
>>> df.set_index(["year", "month"])
```

		sale
year	month	
2012	1	55
2014	4	40
2013	7	84
2014	10	31

Create a MultiIndex using an Index and a column:

```
>>> df.set_index([pd.Index([1, 2, 3, 4]), "year"])
```

		month	sale
	year		
1	2012	1	55
2	2014	4	40
3	2013	7	84
4	2014	10	31

Create a MultiIndex using two Series:

```
>>> s = pd.Series([1, 2, 3, 4])
>>> df.set_index([s, s**2])
```

		month	year	sale
1	1	1	2012	55
2	4	4	2014	40
3	9	7	2013	84
4	16	10	2014	31

Append a column to the existing index:

```
>>> df = df.set_index("month")
>>> df.set_index("year", append=True)
```

		sale
month	year	
1	2012	55
4	2014	40
7	2013	84
10	2014	31

```
>>> df.set_index("year", append=False)
```

	sale
year	
2012	55
2014	40
2013	84

```

shift(
 self,
 periods: int | Sequence[int] = 1,
 freq: Frequency | None = None,
 axis: Axis = 0,
 fill_value: Hashable = <no_default>,
 suffix: str | None = None
) -> DataFrame from pandas.core.frame.DataFrame
 Shift index by desired number of periods with an optional time `freq`.

```

When `freq` is not passed, shift the index without realigning the data. If `freq` is passed (in this case, the index must be date or datetime, or it will raise a `NotImplementedError`), the index will be increased using the periods and the `freq`. `freq` can be inferred when specified as "infer" as long as either freq or inferred\_freq attribute is set in the index.

#### Parameters

```

periods : int or Sequence
 Number of periods to shift. Can be positive or negative.
 If an iterable of ints, the data will be shifted once by each int.
 This is equivalent to shifting by one value at a time and
 concatenating all resulting frames. The resulting columns will have
 the shift suffixed to their column names. For multiple periods,
 axis must not be 1.

freq : DateOffset, tseries.offsets, timedelta, or str, optional
 Offset to use from the tseries module or time rule (e.g. 'EOM').
 If `freq` is specified then the index values are shifted but the
 data is not realigned. That is, use `freq` if you would like to
 extend the index when shifting and preserve the original data.
 If `freq` is specified as "infer" then it will be inferred from
 the freq or inferred_freq attributes of the index. If neither of
 those attributes exist, a ValueError is thrown.

axis : {0 or 'index', 1 or 'columns', None}, default None
 Shift direction. For `Series` this parameter is unused and defaults to 0.

fill_value : object, optional
 The scalar value to use for newly introduced missing values.
 the default depends on the dtype of `self`.
 For Boolean and numeric NumPy data types, ``np.nan`` is used.
 For datetime, timedelta, or period data, etc. :attr:`NaT` is used.
 For extension dtypes, ``self.dtype.na_value`` is used.

suffix : str, optional
 If str and periods is an iterable, this is added after the column
 name and before the shift value for each shifted column name.
 For `Series` this parameter is unused and defaults to `None`.

```

#### Returns

```

DataFrame
 Copy of input object, shifted.

```

#### See Also

```

Index.shift : Shift values of Index.
DatetimeIndex.shift : Shift values of DatetimeIndex.
PeriodIndex.shift : Shift values of PeriodIndex.

```

#### Examples

```

>>> df = pd.DataFrame(
... [[10, 13, 17], [20, 23, 27], [15, 18, 22], [30, 33, 37], [45, 48, 52]],
... columns=["Col1", "Col2", "Col3"],
... index=pd.date_range("2020-01-01", "2020-01-05"),
...)
>>> df

```

	Col1	Col2	Col3
2020-01-01	10	13	17
2020-01-02	20	23	27
2020-01-03	15	18	22
2020-01-04	30	33	37
2020-01-05	45	48	52

```

>>> df.shift(periods=3)

```

	Col1	Col2	Col3
--	------	------	------



2020-01-01	NaN	NaN	NaN
2020-01-02	NaN	NaN	NaN
2020-01-03	NaN	NaN	NaN
2020-01-04	10.0	13.0	17.0
2020-01-05	20.0	23.0	27.0

```
>>> df.shift(periods=1, axis="columns")
```

	Col1	Col2	Col3
2020-01-01	NaN	10	13
2020-01-02	NaN	20	23
2020-01-03	NaN	15	18
2020-01-04	NaN	30	33
2020-01-05	NaN	45	48

```
>>> df.shift(periods=3, fill_value=0)
```

	Col1	Col2	Col3
2020-01-01	0	0	0
2020-01-02	0	0	0
2020-01-03	0	0	0
2020-01-04	10	13	17
2020-01-05	20	23	27

```
>>> df.shift(periods=3, freq="D")
```

	Col1	Col2	Col3
2020-01-04	10	13	17
2020-01-05	20	23	27
2020-01-06	15	18	22
2020-01-07	30	33	37
2020-01-08	45	48	52

```
>>> df.shift(periods=3, freq="infer")
```

	Col1	Col2	Col3
2020-01-04	10	13	17
2020-01-05	20	23	27
2020-01-06	15	18	22
2020-01-07	30	33	37
2020-01-08	45	48	52

```
>>> df["Col1"].shift(periods=[0, 1, 2])
```

	Col1_0	Col1_1	Col1_2
2020-01-01	10	NaN	NaN
2020-01-02	20	10.0	NaN
2020-01-03	15	20.0	10.0
2020-01-04	30	15.0	20.0
2020-01-05	45	30.0	15.0

```
skew(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return unbiased skew over requested axis.
```

Normalized by N-1.

Parameters

axis : {index (0), columns (1)}

Axis for the function to be applied on.

For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric\_only : bool, default False

Include only float, int, boolean columns.

\*\*kwargs

Additional keyword arguments to be passed to the function.

## Returns

-----

Series or scalar

Unbiased skew over requested axis.

## See Also

-----

Dataframe.kurt : Returns unbiased kurtosis over requested axis.

## Examples

-----

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> s.skew()
```

```
0.0
```

With a DataFrame

```
>>> df = pd.DataFrame(
... {"a": [1, 2, 3], "b": [2, 3, 4], "c": [1, 3, 5]},
... index=["tiger", "zebra", "cow"],
...)
>>> df
```

```
 a b c
tiger 1 2 1
zebra 2 3 3
cow 3 4 5
```

```
>>> df.skew()
```

```
a 0.0
```

```
b 0.0
```

```
c 0.0
```

```
dtype: float64
```

Using axis=1

```
>>> df.skew(axis=1)
```

```
tiger 1.732051
```

```
zebra -1.732051
```

```
cow 0.000000
```

```
dtype: float64
```

In this case, `numeric_only` should be set to `True` to avoid getting an error.

```
>>> df = pd.DataFrame(
... {"a": [1, 2, 3], "b": ["T", "Z", "X"]}, index=["tiger", "zebra", "cow"]
...)
```

```
>>> df.skew(numeric_only=True)
```

```
a 0.0
```

```
dtype: float64
```

```
sort_index(
 self,
 *,
 axis: Axis = 0,
 level: IndexLabel | None = None,
 ascending: bool | Sequence[bool] = True,
 inplace: bool = False,
 kind: SortKind = 'quicksort',
 na_position: NaPosition = 'last',
 sort_remaining: bool = True,
 ignore_index: bool = False,
 key: IndexKeyFunc | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Sort object by labels (along an axis).
```

Returns a new DataFrame sorted by label if `inplace` argument is `False`, otherwise updates the original DataFrame and returns None.

## Parameters

-----

axis : {0 or 'index', 1 or 'columns'}, default 0

The axis along which to sort. The value 0 identifies the rows, and 1 identifies the columns.

level : int or level name or list of ints or list of level names

If not None, sort on values in specified index level(s).

ascending : bool or list-like of bools, default True

Sort ascending vs. descending. When the index is a MultiIndex the

sort direction can be controlled for each level individually.

`inplace` : bool, default False  
Whether to modify the DataFrame rather than creating a new one.

`kind` : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'  
Choice of sorting algorithm. See also `:func:`numpy.sort`` for more information. ``mergesort`` and ``stable`` are the only stable algorithms. For DataFrames, this option is only applied when sorting on a single column or label.

`na_position` : {'first', 'last'}, default 'last'  
Puts NaNs at the beginning if ``first``; ``last`` puts NaNs at the end. Not implemented for MultiIndex.

`sort_remaining` : bool, default True  
If True and sorting by level and index is multilevel, sort by other levels too (in order) after sorting by specified level.

`ignore_index` : bool, default False  
If True, the resulting axis will be labeled 0, 1, ..., n - 1.

`key` : callable, optional  
If not None, apply the key function to the index values before sorting. This is similar to the ``key`` argument in the builtin `:meth:`sorted`` function, with the notable difference that this ``key`` function should be *\*vectorized\**. It should expect an ``Index`` and return an ``Index`` of the same shape. For MultiIndex inputs, the key is applied *\*per level\**.

Returns

-----

DataFrame or None

The original DataFrame sorted by the labels or None if ``inplace=True``.

See Also

-----

`Series.sort_index` : Sort Series by the index.

`DataFrame.sort_values` : Sort DataFrame by the value.

`Series.sort_values` : Sort Series by the value.

Examples

-----

```
>>> df = pd.DataFrame(
... [1, 2, 3, 4, 5], index=[100, 29, 234, 1, 150], columns=["A"]
...)
>>> df.sort_index()
 A
1 4
29 2
100 1
150 5
234 3
```

By default, it sorts in ascending order, to sort in descending order, use ``ascending=False``

```
>>> df.sort_index(ascending=False)
 A
234 3
150 5
100 1
29 2
1 4
```

A key function can be specified which is applied to the index before sorting. For a ``MultiIndex`` this is applied to each level separately.

```
>>> df = pd.DataFrame({"a": [1, 2, 3, 4]}, index=["A", "b", "C", "d"])
>>> df.sort_index(key=lambda x: x.str.lower())
 a
A 1
b 2
C 3
d 4
```

```
sort_values(
 self,
 by: IndexLabel,
 *,
 axis: Axis = 0,
 ascending: bool | list[bool] | tuple[bool, ...] = True,
 inplace: bool = False,
```

```

kind: SortKind = 'quicksort',
na_position: str = 'last',
ignore_index: bool = False,
key: ValueKeyFunc | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Sort by the values along either axis.

Parameters

by : str or list of str
 Name or list of names to sort by.

 - if `axis` is 0 or `index` then `by` may contain index
 levels and/or column labels.
 - if `axis` is 1 or `columns` then `by` may contain column
 levels and/or index labels.
axis : "{0 or 'index', 1 or 'columns'}", default 0
 Axis to be sorted.
ascending : bool or list of bool, default True
 Sort ascending vs. descending. Specify list for multiple sort
 orders. If this is a list of bools, must match the length of
 the by.
inplace : bool, default False
 If True, perform operation in-place.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
 Choice of sorting algorithm. See also :func:`numpy.sort` for more
 information. `mergesort` and `stable` are the only stable algorithms. For
 DataFrames, this option is only applied when sorting on a single
 column or label.
na_position : {'first', 'last'}, default 'last'
 Puts NaNs at the beginning if `first`; `last` puts NaNs at the
 end.
ignore_index : bool, default False
 If True, the resulting axis will be labeled 0, 1, ..., n - 1.
key : callable, optional
 Apply the key function to the values
 before sorting. This is similar to the `key` argument in the
 builtin :meth:`sorted` function, with the notable difference that
 this `key` function should be *vectorized*. It should expect a
 ``Series`` and return a Series with the same shape as the input.
 It will be applied to each column in `by` independently. The values in the
 returned Series will be used as the keys for sorting.

Returns

DataFrame or None
 DataFrame with sorted values or None if ``inplace=True``.

See Also

DataFrame.sort_index : Sort a DataFrame by the index.
Series.sort_values : Similar method for a Series.

Examples

>>> df = pd.DataFrame(
... {
... "col1": ["A", "A", "B", np.nan, "D", "C"],
... "col2": [2, 1, 9, 8, 7, 4],
... "col3": [0, 1, 9, 4, 2, 3],
... "col4": ["a", "B", "c", "D", "e", "F"],
... }
...)
>>> df
 col1 col2 col3 col4
0 A 2 0 a
1 A 1 1 B
2 B 9 9 c
3 NaN 8 4 D
4 D 7 2 e
5 C 4 3 F

Sort by a single column

In this case, we are sorting the rows according to values in ``col1``:
>>> df.sort_values(by=["col1"])

```

	col1	col2	col3	col4
0	A	2	0	a
1	A	1	1	B
2	B	9	9	c
5	C	4	3	F
4	D	7	2	e
3	NaN	8	4	D

**\*\*Sort by multiple columns\*\***

You can also provide multiple columns to ``by`` argument, as shown below. In this example, the rows are first sorted according to ``col1``, and then the rows that have an identical value in ``col1`` are sorted according to ``col2``.

```
>>> df.sort_values(by=["col1", "col2"])
 col1 col2 col3 col4
1 A 1 1 B
0 A 2 0 a
2 B 9 9 c
5 C 4 3 F
4 D 7 2 e
3 NaN 8 4 D
```

**\*\*Sort in a descending order\*\***

The sort order can be reversed using ``ascending`` argument, as shown below:

```
>>> df.sort_values(by="col1", ascending=False)
 col1 col2 col3 col4
4 D 7 2 e
5 C 4 3 F
2 B 9 9 c
0 A 2 0 a
1 A 1 1 B
3 NaN 8 4 D
```

**\*\*Placing any ``NaN`` first\*\***

Note that in the above example, the rows that contain an ``NaN`` value in their ``col1`` are placed at the end of the dataframe. This behavior can be modified via ``na\_position`` argument, as shown below:

```
>>> df.sort_values(by="col1", ascending=False, na_position="first")
 col1 col2 col3 col4
3 NaN 8 4 D
4 D 7 2 e
5 C 4 3 F
2 B 9 9 c
0 A 2 0 a
1 A 1 1 B
```

**\*\*Customized sort order\*\***

The ``key`` argument allows for a further customization of sorting behaviour. For example, you may want to ignore the ``letter's`` case <[https://en.wikipedia.org/wiki/Letter\\_case](https://en.wikipedia.org/wiki/Letter_case)>`\_\_ when sorting strings:

```
>>> df.sort_values(by="col4", key=lambda col: col.str.lower())
 col1 col2 col3 col4
0 A 2 0 a
1 A 1 1 B
2 B 9 9 c
3 NaN 8 4 D
4 D 7 2 e
5 C 4 3 F
```

Another typical example is `natural sorting` <[https://en.wikipedia.org/wiki/Natural\\_sort\\_order](https://en.wikipedia.org/wiki/Natural_sort_order)>`\_\_\_. This can be done using ``natsort`` package <<https://github.com/SethMMorton/natsort>>`\_\_, which provides a function to generate a key to sort data in their natural order:

```
>>> df = pd.DataFrame(
... {
```

```

... "hours": ["0hr", "128hr", "0hr", "64hr", "64hr", "128hr"],
... "mins": [
... "10mins",
... "40mins",
... "40mins",
... "40mins",
... "10mins",
... "10mins",
...],
... "value": [10, 20, 30, 40, 50, 60],
... }
...)

```

```
>>> df
```

	hours	mins	value
0	0hr	10mins	10
1	128hr	40mins	20
2	0hr	40mins	30
3	64hr	40mins	40
4	64hr	10mins	50
5	128hr	10mins	60

```
>>> from natsort import natsort_keygen
```

```
>>> df.sort_values(
... by=["hours", "mins"],
... key=natsort_keygen(),
...)
```

	hours	mins	value
0	0hr	10mins	10
2	0hr	40mins	30
4	64hr	10mins	50
3	64hr	40mins	40
5	128hr	10mins	60
1	128hr	40mins	20

```

stack(
 self,
 level: IndexLabel = -1,
 dropna: bool | lib.NoDefault = <no_default>,
 sort: bool | lib.NoDefault = <no_default>,
 future_stack: bool = True
) from pandas.core.frame.DataFrame
Stack the prescribed level(s) from columns to index.

```

Return a reshaped DataFrame or Series having a multi-level index with one or more new inner-most levels compared to the current DataFrame. The new inner-most levels are created by pivoting the columns of the current dataframe:

- if the columns have a single level, the output is a Series;
- if the columns have multiple levels, the new index level(s) is (are) taken from the prescribed level(s) and the output is a DataFrame.

#### Parameters

```

level : int, str, list, default -1
 Level(s) to stack from the column axis onto the index
 axis, defined as one index or label, or a list of indices
 or labels.
dropna : bool, default True
 Whether to drop rows in the resulting Frame/Series with
 missing values. Stacking a column level onto the index
 axis can create combinations of index and column values
 that are missing from the original dataframe. See Examples
 section.
sort : bool, default True
 Whether to sort the levels of the resulting MultiIndex.
future_stack : bool, default True
 Whether to use the new implementation that will replace the current
 implementation in pandas 3.0. When True, dropna and sort have no impact
 on the result and must remain unspecified. See :ref:`pandas 2.1.0 Release
 notes <whatsnew_210.enhancements.new_stack>` for more details.

```

#### Returns

```

Dataframe or Series
 Stacked dataframe or series.

```

See Also

```

DataFrame.unstack : Unstack prescribed level(s) from index axis
 onto column axis.
DataFrame.pivot : Reshape dataframe from long format to wide
 format.
DataFrame.pivot_table : Create a spreadsheet-style pivot table
 as a DataFrame.

```

#### Notes

```

The function is named by analogy with a collection of books
being reorganized from being side by side on a horizontal
position (the columns of the dataframe) to being stacked
vertically on top of each other (in the index of the
dataframe).

```

Reference :ref:`the user guide <reshaping.stacking>` for more examples.

#### Examples

```

Single level columns

```

```

>>> df_single_level_cols = pd.DataFrame(
... [[0, 1], [2, 3]], index=["cat", "dog"], columns=["weight", "height"]
...)

```

Stacking a dataframe with a single level column axis returns a Series:

```

>>> df_single_level_cols
 weight height
cat 0 1
dog 2 3
>>> df_single_level_cols.stack()
cat weight 0
 height 1
dog weight 2
 height 3
dtype: int64

```

```

Multi level columns: simple case

```

```

>>> multicol1 = pd.MultiIndex.from_tuples(
... [("weight", "kg"), ("weight", "pounds")]
...)
>>> df_multi_level_cols1 = pd.DataFrame(
... [[1, 2], [2, 4]], index=["cat", "dog"], columns=multicol1
...)

```

Stacking a dataframe with a multi-level column axis:

```

>>> df_multi_level_cols1
 weight
 kg pounds
cat 1 2
dog 2 4
>>> df_multi_level_cols1.stack()
cat kg 1
 pounds 2
dog kg 2
 pounds 4

```

```

Missing values

```

```

>>> multicol2 = pd.MultiIndex.from_tuples([("weight", "kg"), ("height", "m")])
>>> df_multi_level_cols2 = pd.DataFrame(
... [[1.0, 2.0], [3.0, 4.0]], index=["cat", "dog"], columns=multicol2
...)

```

It is common to have missing values when stacking a dataframe with multi-level columns, as the stacked dataframe typically has more values than the original dataframe. Missing values are filled with NaNs:

```

>>> df_multi_level_cols2
 weight height
 kg m

```

```

cat 1.0 2.0
dog 3.0 4.0
>>> df_multi_level_cols2.stack()
 weight height
cat kg 1.0 NaN
 m NaN 2.0
dog kg 3.0 NaN
 m NaN 4.0

```

**\*\*Prescribing the level(s) to be stacked\*\***

The first parameter controls which level or levels are stacked:

```

>>> df_multi_level_cols2.stack(0)
 kg m
cat weight 1.0 NaN
 height NaN 2.0
dog weight 3.0 NaN
 height NaN 4.0
>>> df_multi_level_cols2.stack([0, 1])
cat weight kg 1.0
 height m 2.0
dog weight kg 3.0
 height m 4.0
dtype: float64

```

```

std(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 ddof: int = 1,
 numeric_only: bool = False,
 **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return sample standard deviation over requested axis.

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

axis : {index (0), columns (1)}
 For `Series` this parameter is unused and defaults to 0.

 .. warning::

 The behavior of DataFrame.std with ``axis=None`` is deprecated,
 in a future version this will reduce over both axes and return a scalar
 To retain the old behavior, pass axis=0 (or do not pass axis).

skipna : bool, default True
 Exclude NA/null values. If an entire row/column is NA, the result
 will be NA.
ddof : int, default 1
 Delta Degrees of Freedom. The divisor used in calculations is N - ddof,
 where N represents the number of elements.
numeric_only : bool, default False
 Include only float, int, boolean columns. Not implemented for Series.
**kwargs : dict
 Additional keyword arguments to be passed to the function.

Returns

Series or scalar
 Standard deviation over requested axis.

See Also

Series.std : Return standard deviation over Series values.
DataFrame.mean : Return the mean of the values over the requested axis.
DataFrame.median : Return the median of the values over the requested axis.
DataFrame.mode : Get the mode(s) of each element along the requested axis.
DataFrame.sum : Return the sum of the values over the requested axis.

Notes

To have the same behaviour as `numpy.std`, use `ddof=0` (instead of the

```



```
default `ddof=1`)
```

#### Examples

```

>>> df = pd.DataFrame(
... {
... "person_id": [0, 1, 2, 3],
... "age": [21, 25, 62, 43],
... "height": [1.61, 1.87, 1.49, 2.01],
... }
...).set_index("person_id")
>>> df
 age height
person_id
0 21 1.61
1 25 1.87
2 62 1.49
3 43 2.01
```

The standard deviation of the columns can be found as follows:

```
>>> df.std()
age 18.786076
height 0.237417
dtype: float64
```

Alternatively, `ddof=0` can be set to normalize by N instead of N-1:

```
>>> df.std(ddof=0)
age 16.269219
height 0.205609
dtype: float64
```

sub(self, other, axis: Axis = 'columns', level=None, fill\_value=None) -> DataFrame from pandas  
Get Subtraction of dataframe and other, element-wise (binary operator `sub`).

Equivalent to ``dataframe - other``, but with support to substitute a fill\_value for missing data in one of the inputs. With reverse version, `rsub`.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

#### Parameters

```

other : scalar, sequence, Series, dict or DataFrame
 Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
 Whether to compare by the index (0 or 'index') or columns.
 (1 or 'columns'). For Series input, axis to match Series index on.
level : int or label
 Broadcast across a level, matching Index values on the
 passed MultiIndex level.
fill_value : float or None, default None
 Fill existing missing (NaN) values, and any new element needed for
 successful DataFrame alignment, with this value before computation.
 If data in both corresponding DataFrame locations is missing
 the result will be missing.
```

#### Returns

```

DataFrame
 Result of the arithmetic operation.
```

#### See Also

```

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.
```

#### Notes

```

Mismatched indices will be unioned together.
```

### Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
```

	angles	degrees
circle	0	720
triangle	0	360
rectangle	0	720

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
```

	angles	degrees
circle	0	0
triangle	6	360
rectangle	12	1080

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
```

```
>>> other
 angles
circle 0
triangle 3
rectangle 4
```

```
>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN
```

```
>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
```

```
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
```

```
>>> df_pow.pow(2)
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

```
subtract = sub(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame
```

```
sum(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 min_count: int = 0,
 **kwargs
```

```
) -> Series from pandas.core.frame.DataFrame
Return the sum of the values over the requested axis.
```

This is equivalent to the method ``numpy.sum``.

Parameters

```
axis : {index (0), columns (1)}
 Axis for the function to be applied on.
 For `Series` this parameter is unused and defaults to 0.
```

```
.. warning::
```

The behavior of `DataFrame.sum` with `axis=None` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass `axis=0` (or do not pass `axis`).

.. versionadded:: 2.0.0

`skipna` : bool, default True  
Exclude NA/null values when computing the result.  
`numeric_only` : bool, default False  
Include only float, int, boolean columns. Not implemented for Series.  
`min_count` : int, default 0  
The required number of valid values to perform the operation. If fewer than `min_count` non-NA values are present the result will be NA.  
**\*\*kwargs**  
Additional keyword arguments to be passed to the function.

Returns

Series or scalar  
Sum over requested axis.

See Also

Series.sum : Return the sum over Series values.  
DataFrame.mean : Return the mean of the values over the requested axis.  
DataFrame.median : Return the median of the values over the requested axis.  
DataFrame.mode : Get the mode(s) of each element along the requested axis.  
DataFrame.std : Return the standard deviation of the values over the requested axis.

Examples

```
>>> idx = pd.MultiIndex.from_arrays(
... [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
... names=["blooded", "animal"],
...)
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm dog 4
 falcon 2
cold fish 0
 spider 8
Name: legs, dtype: int64

>>> s.sum()
14
```

By default, the sum of an empty or all-NA Series is `0`.

```
>>> pd.Series([], dtype="float64").sum() # min_count=0 is the default
0.0
```

This can be controlled with the `min_count` parameter. For example, if you'd like the sum of an empty series to be `NaN`, pass `min_count=1`.

```
>>> pd.Series([], dtype="float64").sum(min_count=1)
nan
```

Thanks to the `skipna` parameter, `min_count` handles all-NA and empty series identically.

```
>>> pd.Series([np.nan]).sum()
0.0
```

```
>>> pd.Series([np.nan]).sum(min_count=1)
nan
```

`swaplevel(self, i: Axis = -2, j: Axis = -1, axis: Axis = 0) -> DataFrame from pandas.core.frame.DataFrame`  
Swap levels `i` and `j` in a `MultiIndex`.

Default is to swap the two innermost levels of the index.

Parameters

`i, j` : int or str

Levels of the indices to be swapped. Can pass level name as string.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
The axis to swap levels on. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.

Returns

-----  
DataFrame

DataFrame with levels swapped in MultiIndex.

See Also

-----  
DataFrame.reorder\_levels: Reorder levels of MultiIndex.  
DataFrame.sort\_index: Sort MultiIndex.

Examples

```
>>> df = pd.DataFrame(
... {"Grade": ["A", "B", "A", "C"]},
... index=[
... ["Final exam", "Final exam", "Coursework", "Coursework"],
... ["History", "Geography", "History", "Geography"],
... ["January", "February", "March", "April"],
...],
...)
>>> df
```

			Grade
Final exam	History	January	A
	Geography	February	B
Coursework	History	March	A
	Geography	April	C

In the following example, we will swap the levels of the indices.  
Here, we will swap the levels column-wise, but levels can be swapped row-wise  
in a similar manner. Note that column-wise is the default behaviour.  
By not supplying any arguments for i and j, we swap the last and second to  
last indices.

```
>>> df.swaplevel()
```

			Grade
Final exam	January	History	A
	February	Geography	B
Coursework	March	History	A
	April	Geography	C

By supplying one argument, we can choose which index to swap the last  
index with. We can for example swap the first index with the last one as  
follows.

```
>>> df.swaplevel(0)
```

			Grade
January	History	Final exam	A
February	Geography	Final exam	B
March	History	Coursework	A
April	Geography	Coursework	C

We can also define explicitly which indices we want to swap by supplying values  
for both i and j. Here, we for example swap the first and second indices.

```
>>> df.swaplevel(0, 1)
```

			Grade
History	Final exam	January	A
Geography	Final exam	February	B
History	Coursework	March	A
Geography	Coursework	April	C

```
to_dict(
 self,
 orient: Literal['dict', 'list', 'series', 'split', 'tight', 'records', 'index'] = 'dict',
 *,
 into: type[MutableMappingT] | MutableMappingT = <class 'dict'>,
 index: bool = True
) -> MutableMappingT | list[MutableMappingT] from pandas.core.frame.DataFrame
Convert the DataFrame to a dictionary.
```

The type of the key-value pairs can be customized with the parameters  
(see below).

## Parameters

**orient** : str {'dict', 'list', 'series', 'split', 'tight', 'records', 'index'}  
Determines the type of the values of the dictionary.

- 'dict' (default) : dict like {column -> {index -> value}}
- 'list' : dict like {column -> [values]}
- 'series' : dict like {column -> Series(values)}
- 'split' : dict like  
{'index' -> [index], 'columns' -> [columns], 'data' -> [values]}
- 'tight' : dict like  
{'index' -> [index], 'columns' -> [columns], 'data' -> [values],  
'index\_names' -> [index.names], 'column\_names' -> [column.names]}
- 'records' : list like  
[{column -> value}, ... , {column -> value}]
- 'index' : dict like {index -> {column -> value}}

**into** : class, default dict

The collections.abc.MutableMapping subclass used for all Mappings in the return value. Can be the actual class or an empty instance of the mapping type you want. If you want a collections.defaultdict, you must pass it initialized.

**index** : bool, default True

Whether to include the index item (and index\_names item if 'orient' is 'tight') in the returned dictionary. Can only be ``False`` when 'orient' is 'split' or 'tight'. Note that when 'orient' is 'records', this parameter does not take effect (index item always not included).

.. versionadded:: 2.0.0

## Returns

dict, list or collections.abc.MutableMapping

Return a collections.abc.MutableMapping object representing the DataFrame. The resulting transformation depends on the 'orient' parameter.

## See Also

DataFrame.from\_dict: Create a DataFrame from a dictionary.

DataFrame.to\_json: Convert a DataFrame to JSON format.

## Examples

```
>>> df = pd.DataFrame(
... {"col1": [1, 2], "col2": [0.5, 0.75]}, index=["row1", "row2"]
...)
>>> df
 col1 col2
row1 1 0.50
row2 2 0.75
>>> df.to_dict()
{'col1': {'row1': 1, 'row2': 2}, 'col2': {'row1': 0.5, 'row2': 0.75}}
```

You can specify the return orientation.

```
>>> df.to_dict("series")
{'col1': row1 1
 row2 2
Name: col1, dtype: int64,
 'col2': row1 0.50
 row2 0.75
Name: col2, dtype: float64}

>>> df.to_dict("split")
{'index': ['row1', 'row2'], 'columns': ['col1', 'col2'],
 'data': [[1, 0.5], [2, 0.75]]}

>>> df.to_dict("records")
[{'col1': 1, 'col2': 0.5}, {'col1': 2, 'col2': 0.75}]

>>> df.to_dict("index")
{'row1': {'col1': 1, 'col2': 0.5}, 'row2': {'col1': 2, 'col2': 0.75}}
```

```
>>> df.to_dict("tight")
{'index': ['row1', 'row2'], 'columns': ['col1', 'col2'],
 'data': [[1, 0.5], [2, 0.75]], 'index_names': [None], 'column_names': [None]}
```

You can also specify the mapping type.

```
>>> from collections import OrderedDict, defaultdict
>>> df.to_dict(into=OrderedDict)
OrderedDict([('col1', OrderedDict([('row1', 1), ('row2', 2)])),
 ('col2', OrderedDict([('row1', 0.5), ('row2', 0.75)]))])
```

If you want a `defaultdict`, you need to initialize it:

```
>>> dd = defaultdict(list)
>>> df.to_dict("records", into=dd)
[defaultdict(<class 'list'>, {'col1': 1, 'col2': 0.5}),
 defaultdict(<class 'list'>, {'col1': 2, 'col2': 0.75})]
```

`to_feather(self, path: FilePath | WriteBuffer[bytes], **kwargs) -> None` from `pandas.core.frame`  
Write a DataFrame to the binary Feather format.

#### Parameters

```
path : str, path object, file-like object
 String, path object (implementing ``os.PathLike[str]``), or file-like
 object implementing a binary ``write()`` function. If a string or a path,
 it will be used as Root Directory path when writing a partitioned dataset.
**kwargs :
 Additional keywords passed to :func:`pyarrow.feather.write_feather`.
 This includes the `compression`, `compression_level`, `chunksize`
 and `version` keywords.
```

#### See Also

```
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.
DataFrame.to_excel : Write object to an Excel sheet.
DataFrame.to_sql : Write to a sql table.
DataFrame.to_csv : Write a csv file.
DataFrame.to_json : Convert the object to a JSON string.
DataFrame.to_html : Render a DataFrame as an HTML table.
DataFrame.to_string : Convert DataFrame to a string.
```

#### Notes

```
This function writes the dataframe as a `feather file
<https://arrow.apache.org/docs/python/feather.html>`. Requires a default
index. For saving the DataFrame with your custom index use a method that
supports custom indices e.g. `to_parquet`.
```

#### Examples

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]])
>>> df.to_feather("file.feather") # doctest: +SKIP
```

```
to_html(
 self,
 buf: FilePath | WriteBuffer[str] | None = None,
 *,
 columns: Axes | None = None,
 col_space: ColspaceArgType | None = None,
 header: bool = True,
 index: bool = True,
 na_rep: str = 'NaN',
 formatters: FormattersType | None = None,
 float_format: FloatFormatType | None = None,
 sparsify: bool | None = None,
 index_names: bool = True,
 justify: str | None = None,
 max_rows: int | None = None,
 max_cols: int | None = None,
 show_dimensions: bool | str = False,
 decimal: str = '.',
 bold_rows: bool = True,
 classes: str | list | tuple | None = None,
 escape: bool = True,
 notebook: bool = False,
 border: int | bool | None = None,
```

```

table_id: str | None = None,
render_links: bool = False,
encoding: str | None = None
) -> str | None from pandas.core.frame.DataFrame
Render a DataFrame as an HTML table.

```

#### Parameters

```

buf : str, Path or StringIO-like, optional, default None
 Buffer to write to. If None, the output is returned as a string.
columns : array-like, optional, default None
 The subset of columns to write. Writes all columns by default.
col_space : str or int, list or dict of int or str, optional
 The minimum width of each column in CSS length units. An int is assumed to
header : bool, optional
 Whether to print column labels, default True.
index : bool, optional, default True
 Whether to print index (row) labels.
na_rep : str, optional, default 'NaN'
 String representation of ``NaN`` to use.
formatters : list, tuple or dict of one-param. functions, optional
 Formatter functions to apply to columns' elements by position or
 name.
 The result of each function must be a unicode string.
 List/tuple must be of length equal to the number of columns.
float_format : one-parameter function, optional, default None
 Formatter function to apply to columns' elements if they are
 floats. This function must return a unicode string and will be
 applied only to the non-``NaN`` elements, with ``NaN`` being
 handled by ``na_rep``.
sparsify : bool, optional, default True
 Set to False for a DataFrame with a hierarchical index to print
 every multiindex key at each row.
index_names : bool, optional, default True
 Prints the names of the indexes.
justify : str, default None
 How to justify the column labels. If None uses the option from
 the print configuration (controlled by set_option), 'right' out
 of the box. Valid values are

 * left
 * right
 * center
 * justify
 * justify-all
 * start
 * end
 * inherit
 * match-parent
 * initial
 * unset.
max_rows : int, optional
 Maximum number of rows to display in the console.
max_cols : int, optional
 Maximum number of columns to display in the console.
show_dimensions : bool, default False
 Display DataFrame dimensions (number of rows by number of columns).
decimal : str, default '.'
 Character recognized as decimal separator, e.g. ',' in Europe.

bold_rows : bool, default True
 Make the row labels bold in the output.
classes : str or list or tuple, default None
 CSS class(es) to apply to the resulting html table.
escape : bool, default True
 Convert the characters <, >, and & to HTML-safe sequences.
notebook : {True, False}, default False
 Whether the generated HTML is for IPython Notebook.
border : int or bool
 When an integer value is provided, it sets the border attribute in
 the opening tag, specifying the thickness of the border.
 If ``False`` or ``0`` is passed, the border attribute will not
 be present in the ``<table>`` tag.
 The default value for this parameter is governed by
 ``pd.options.display.html.border``.
table_id : str, optional
 A css id is included in the opening ``<table>`` tag if specified.

```



render\_links : bool, default False  
Convert URLs to HTML links.  
encoding : str, default "utf-8"  
Set character encoding.

Returns

-----  
str or None

If buf is None, returns the result as a string. Otherwise returns None.

See Also

-----  
to\_string : Convert DataFrame to a string.

Examples

-----  
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})  
>>> html\_string = df.to\_html()  
>>> print(html\_string)  
<table border="1" class="dataframe">  
 <thead>  
 <tr style="text-align: right;">  
 <th></th>  
 <th>col1</th>  
 <th>col2</th>  
 </tr>  
 </thead>  
 <tbody>  
 <tr>  
 <th>0</th>  
 <td>1</td>  
 <td>4</td>  
 </tr>  
 <tr>  
 <th>1</th>  
 <td>2</td>  
 <td>3</td>  
 </tr>  
 </tbody>  
</table>

HTML output

```
+-----+-----+
| |col1 |col2 |
+====+=====+
|0 |1 |4 |
+-----+-----+
|1 |2 |3 |
+-----+-----+
```

>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})  
>>> html\_string = df.to\_html(index=False)  
>>> print(html\_string)  
<table border="1" class="dataframe">  
 <thead>  
 <tr style="text-align: right;">  
 <th>col1</th>  
 <th>col2</th>  
 </tr>  
 </thead>  
 <tbody>  
 <tr>  
 <td>1</td>  
 <td>4</td>  
 </tr>  
 <tr>  
 <td>2</td>  
 <td>3</td>  
 </tr>  
 </tbody>  
</table>

HTML output

```
+-----+-----+
```

col1	col2
1	4
2	3

```

to_iceberg(
 self,
 table_identifier: str,
 catalog_name: str | None = None,
 *,
 catalog_properties: dict[str, Any] | None = None,
 location: str | None = None,
 append: bool = False,
 snapshot_properties: dict[str, str] | None = None
) -> None from pandas.core.frame.DataFrame
 Write a DataFrame to an Apache Iceberg table.

 .. versionadded:: 3.0.0

 .. warning::

 to_iceberg is experimental and may change without warning.

Parameters

table_identifier : str
 Table identifier.
catalog_name : str, optional
 The name of the catalog.
catalog_properties : dict of {str: str}, optional
 The properties that are used next to the catalog configuration.
location : str, optional
 Location for the table.
append : bool, default False
 If ``True``, append data to the table, instead of replacing the content.
snapshot_properties : dict of {str: str}, optional
 Custom properties to be added to the snapshot summary

See Also

read_iceberg : Read an Apache Iceberg table.
DataFrame.to_parquet : Write a DataFrame in Parquet format.

Examples

>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> df.to_iceberg("my_table", catalog_name="my_catalog") # doctest: +SKIP

to_markdown(
 self,
 buf: FilePath | WriteBuffer[str] | None = None,
 *,
 mode: str = 'wt',
 index: bool = True,
 storage_options: StorageOptions | None = None,
 **kwargs
) -> str | None from pandas.core.frame.DataFrame
 Print DataFrame in Markdown-friendly format.

Parameters

buf : str, Path or StringIO-like, optional, default None
 Buffer to write to. If None, the output is returned as a string.
mode : str, optional
 Mode in which file is opened, "wt" by default.
index : bool, optional, default True
 Add index (row) labels.

storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g.
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
 are forwarded to ``urllib.request.Request`` as header options. For other
 URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are
 forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
 details, and for more examples on storage options refer `here`

```

[https://pandas.pydata.org/docs/user\\_guide/io.html?highlight=storage\\_options#reading-writing-remote-files](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files)>`\_.

**\*\*kwargs**

These parameters will be passed to ``tabulate`` <https://pypi.org/project/tabulate>>`\_.

**Returns**

-----  
str

DataFrame in Markdown-friendly format.

**See Also**

-----  
`DataFrame.to_html` : Render DataFrame to HTML-formatted table.  
`DataFrame.to_latex` : Render DataFrame to LaTeX-formatted table.

**Notes**

-----  
Requires the ``tabulate`` <https://pypi.org/project/tabulate>>`\_ package.

**Examples**

-----  
>>> df = pd.DataFrame(  
... data={"animal\_1": ["elk", "pig"], "animal\_2": ["dog", "quetzal"]}  
... )  
>>> print(df.to\_markdown())  
| | animal\_1 | animal\_2 |  
|---|:-----|:-----|  
| 0 | elk | dog |  
| 1 | pig | quetzal |

Output markdown with a tabulate option.

>>> print(df.to\_markdown(tablefmt="grid"))  
+-----+-----+  
| | animal\_1 | animal\_2 |  
+====+=====+=====+  
| 0 | elk | dog |  
+-----+-----+-----+  
| 1 | pig | quetzal |  
+-----+-----+-----+

`to_numpy(`  
    self,  
    dtype: npt.DTypeLike | None = None,  
    copy: bool = False,  
    na\_value: object = <no\_default>  
) -> np.ndarray from pandas.core.frame.DataFrame  
Convert the DataFrame to a NumPy array.

By default, the dtype of the returned array will be the common NumPy dtype of all types in the DataFrame. For example, if the dtypes are ``float16`` and ``float32``, the results dtype will be ``float32``. This may require copying data and coercing values, which may be expensive.

**Parameters**

-----  
`dtype` : str or numpy.dtype, optional  
    The dtype to pass to `:meth:`numpy.asarray``.  
`copy` : bool, default False  
    Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not *\*ensure\** that ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.  
`na_value` : Any, optional  
    The value to use for missing values. The default value depends on ``dtype`` and the dtypes of the DataFrame columns.

**Returns**

-----  
numpy.ndarray  
    The NumPy array representing the values in the DataFrame.

**See Also**

-----  
`Series.to_numpy` : Similar method for Series.

## Examples

```
>>> pd.DataFrame({"A": [1, 2], "B": [3, 4]}).to_numpy()
array([[1, 3],
 [2, 4]])
```

With heterogeneous data, the lowest common type will have to be used.

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3.0, 4.5]})
>>> df.to_numpy()
array([[1. , 3.],
 [2. , 4.5]])
```

For a mix of numeric and non-numeric types, the output array will have object dtype.

```
>>> df["C"] = pd.date_range("2000", periods=2)
>>> df.to_numpy()
array([[1, 3.0, Timestamp('2000-01-01 00:00:00')],
 [2, 4.5, Timestamp('2000-01-02 00:00:00')]], dtype=object)
```

```
to_orc(
 self,
 path: FilePath | WriteBuffer[bytes] | None = None,
 *,
 engine: Literal['pyarrow'] = 'pyarrow',
 index: bool | None = None,
 engine_kwargs: dict[str, Any] | None = None
) -> bytes | None from pandas.core.frame.DataFrame
Write a DataFrame to the Optimized Row Columnar (ORC) format.
```

## Parameters

**path** : str, file-like object or None, default None  
If a string, it will be used as Root Directory path when writing a partitioned dataset. By file-like object, we refer to objects with a write() method, such as a file handle (e.g. via builtin open function). If path is None, a bytes object is returned.

**engine** : {'pyarrow'}, default 'pyarrow'  
ORC library to use.

**index** : bool, optional  
If ``True``, include the dataframe's index(es) in the file output. If ``False``, they will not be written to the file. If ``None``, similar to ``infer`` the dataframe's index(es) will be saved. However, instead of being saved as values, the RangeIndex will be stored as a range in the metadata so it doesn't require much space and is faster. Other indexes will be included as columns in the file output.

**engine\_kwargs** : dict[str, Any] or None, default None  
Additional keyword arguments passed to :func:`pyarrow.orc.write\_table`.

## Returns

bytes if no ``path`` argument is provided else None  
Bytes object with DataFrame data if ``path`` is not specified else None.

## Raises

**NotImplementedError**  
Dtype of one or more columns is category, unsigned integers, interval, period or sparse.

**ValueError**  
engine is not pyarrow.

## See Also

**read\_orc** : Read a ORC file.  
**DataFrame.to\_parquet** : Write a parquet file.  
**DataFrame.to\_csv** : Write a csv file.  
**DataFrame.to\_sql** : Write to a sql table.  
**DataFrame.to\_hdf** : Write to hdf.

## Notes

- \* Find more information on ORC  
`here <[https://en.wikipedia.org/wiki/Apache\\_ORC](https://en.wikipedia.org/wiki/Apache_ORC)>`\_\_.
- \* Before using this function you should read the :ref:`user guide about ORC <io.orc>` and :ref:`install optional dependencies <install.warn\_orc>`.
- \* This function requires `pyarrow <<https://arrow.apache.org/docs/python/>>` library.
- \* For supported dtypes please refer to `supported ORC features in Arrow <<https://arrow.apache.org/docs/cpp/orc.html#data-types>>`\_\_.
- \* Currently timezones in datetime columns are not preserved when a dataframe is converted into ORC files.

#### Examples

```

>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> df.to_orc("df.orc") # doctest: +SKIP
>>> pd.read_orc("df.orc") # doctest: +SKIP
 col1 col2
0 1 4
1 2 3

```

If you want to get a buffer to the orc content you can write it to `io.BytesIO`

```

>>> import io
>>> b = io.BytesIO(df.to_orc()) # doctest: +SKIP
>>> b.seek(0) # doctest: +SKIP
0
>>> content = b.read() # doctest: +SKIP

```

```

to_parquet(
 self,
 path: FilePath | WriteBuffer[bytes] | None = None,
 *,
 engine: Literal['auto', 'pyarrow', 'fastparquet'] = 'auto',
 compression: ParquetCompressionOptions = 'snappy',
 index: bool | None = None,
 partition_cols: list[str] | None = None,
 storage_options: StorageOptions | None = None,
 filesystem: Any = None,
 **kwargs
) -> bytes | None from pandas.core.frame.DataFrame
Write a DataFrame to the binary parquet format.

```

This function writes the dataframe as a `parquet file <<https://parquet.apache.org/>>`\_\_. You can choose different parquet backends, and have the option of compression. See :ref:`the user guide <io.parquet>` for more details.

#### Parameters

```

path : str, path object, file-like object, or None, default None
 String, path object (implementing ``os.PathLike[str]``), or file-like
 object implementing a binary ``write()`` function. If None, the result is
 returned as bytes. If a string or path, it will be used as Root Directory
 path when writing a partitioned dataset.
engine : {'auto', 'pyarrow', 'fastparquet'}, default 'auto'
 Parquet library to use. If 'auto', then the option
 ``io.parquet.engine`` is used. The default ``io.parquet.engine``
 behavior is to try 'pyarrow', falling back to 'fastparquet' if
 'pyarrow' is unavailable.
compression : str or None, default 'snappy'
 Name of the compression to use. Use ``None`` for no compression.
 Supported options: 'snappy', 'gzip', 'brotli', 'lz4', 'zstd'.
index : bool, default None
 If ``True``, include the dataframe's index(es) in the file output.
 If ``False``, they will not be written to the file.
 If ``None``, similar to ``True`` the dataframe's index(es)
 will be saved. However, instead of being saved as values,
 the RangeIndex will be stored as a range in the metadata so it
 doesn't require much space and is faster. Other indexes will
 be included as columns in the file output.
partition_cols : list, optional, default None
 Column names by which to partition the dataset.
 Columns are partitioned in the order they are given.
 Must be None if path is not a string.
storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g.
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs

```

are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`filesystem` : fsspec or pyarrow filesystem, default None  
 Filesystem object to use when reading the parquet file. Only implemented for `engine="pyarrow"`.

`.. versionadded:: 2.1.0`

**\*\*kwargs**  
 Additional arguments passed to the parquet library. See `:ref:`pandas io <io.parquet>` for more details.

#### Returns

bytes if no path argument is provided else None  
 Returns the DataFrame converted to the binary parquet format as bytes if no path argument. Returns None and writes the DataFrame to the specified location in the Parquet format if the path argument is provided.

#### See Also

`read_parquet` : Read a parquet file.  
`DataFrame.to_orc` : Write an orc file.  
`DataFrame.to_csv` : Write a csv file.  
`DataFrame.to_sql` : Write to a sql table.  
`DataFrame.to_hdf` : Write to hdf.

#### Notes

- \* This function requires either the `fastparquet` `<https://pypi.org/project/fastparquet>` or `pyarrow` `<https://arrow.apache.org/docs/python/>` library.
- \* When saving a DataFrame with categorical columns to parquet, the file size may increase due to the inclusion of all possible categories, not just those present in the data. This behavior is expected and consistent with pandas' handling of categorical data. To manage file size and ensure a more predictable roundtrip process, consider using `:meth:`Categorical.remove_unused_categories`` on the DataFrame before saving.

#### Examples

```
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [3, 4]})
>>> df.to_parquet("df.parquet.gz", compression="gzip") # doctest: +SKIP
>>> pd.read_parquet("df.parquet.gz") # doctest: +SKIP
 col1 col2
0 1 3
1 2 4
```

If you want to get a buffer to the parquet content you can use a `io.BytesIO` object, as long as you don't use `partition_cols`, which creates multiple files.

```
>>> import io
>>> f = io.BytesIO()
>>> df.to_parquet(f)
>>> f.seek(0)
0
>>> content = f.read()
```

```
to_period(
 self,
 freq: Frequency | None = None,
 axis: Axis = 0,
 copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
Convert DataFrame from DatetimeIndex to PeriodIndex.
```

Convert DataFrame from DatetimeIndex to PeriodIndex with desired frequency (inferred from index if not passed). Either index or columns can be converted, depending on `axis` argument.

#### Parameters

-----

freq : str, default  
Frequency of the PeriodIndex.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
The axis to convert (the index by default).  
copy : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.  
  
.. deprecated:: 3.0.0  
  
This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_ for more details.

#### Returns

-----  
DataFrame

The DataFrame with the converted PeriodIndex.

#### See Also

-----  
Series.to\_period: Equivalent method for Series.  
Series.dt.to\_period: Convert DateTime column values.

#### Examples

-----  
>>> idx = pd.to\_datetime(  
... [  
... "2001-03-31 00:00:00",  
... "2002-05-31 00:00:00",  
... "2003-08-31 00:00:00",  
... ]  
... )  
  
>>> idx  
DatetimeIndex(['2001-03-31', '2002-05-31', '2003-08-31'],  
 dtype='datetime64[us]', freq=None)  
  
>>> idx.to\_period("M")  
PeriodIndex(['2001-03', '2002-05', '2003-08'], dtype='period[M]')  
  
For the yearly frequency  
  
>>> idx.to\_period("Y")  
PeriodIndex(['2001', '2002', '2003'], dtype='period[Y-DEC]')

to\_records(self, index: bool = True, column\_dtypes=None, index\_dtypes=None) -> np.rec.recarray  
Convert DataFrame to a NumPy record array.

Index will be included as the first field of the record array if requested.

#### Parameters

-----  
index : bool, default True  
Include index in resulting record array, stored in 'index' field or using the index label, if set.  
column\_dtypes : str, type, dict, default None  
If a string or type, the data type to store all columns. If a dictionary, a mapping of column names and indices (zero-indexed) to specific data types.  
index\_dtypes : str, type, dict, default None  
If a string or type, the data type to store all index levels. If a dictionary, a mapping of index level names and indices (zero-indexed) to specific data types.  
  
This mapping is applied only if `index=True`.

#### Returns

-----  
numpy.rec.recarray  
NumPy ndarray with the DataFrame labels as fields and each row of the DataFrame as entries.

See Also

-----

`DataFrame.from_records`: Convert structured or record ndarray to `DataFrame`.

`numpy.rec.recrarray`: An ndarray that allows field access using attributes, analogous to typed columns in a spreadsheet.

Examples

-----

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [0.5, 0.75]}, index=["a", "b"])
>>> df
```

```
 A B
a 1 0.5
b 2 0.75
```

```
>>> df.to_records()
rec.array([(b'a', 1, 0.5), (b'b', 2, 0.75)],
 dtype=[('index', 'O'), ('A', '<i8'), ('B', '<f8')])
```

If the `DataFrame` index has no label then the `recarray` field name is set to `'index'`. If the index has a label then this is used as the field name:

```
>>> df.index = df.index.rename("I")
>>> df.to_records()
rec.array([(b'I', 1, 0.5), (b'b', 2, 0.75)],
 dtype=[('I', 'O'), ('A', '<i8'), ('B', '<f8')])
```

The index can be excluded from the record array:

```
>>> df.to_records(index=False)
rec.array([(1, 0.5), (2, 0.75)],
 dtype=[('A', '<i8'), ('B', '<f8')])
```

Data types can be specified for the columns:

```
>>> df.to_records(column_dtypes={"A": "int32"})
rec.array([(b'a', 1, 0.5), (b'b', 2, 0.75)],
 dtype=[('I', 'O'), ('A', '<i4'), ('B', '<f8')])
```

As well as for the index:

```
>>> df.to_records(index_dtypes="<S2")
rec.array([(b'a', 1, 0.5), (b'b', 2, 0.75)],
 dtype=[('I', 'S2'), ('A', '<i8'), ('B', '<f8')])
```

```
>>> index_dtypes = f"<S{df.index.str.len().max()}"
>>> df.to_records(index_dtypes=index_dtypes)
rec.array([(b'a', 1, 0.5), (b'b', 2, 0.75)],
 dtype=[('I', 'S1'), ('A', '<i8'), ('B', '<f8')])
```

```
to_stata(
 self,
 path: FilePath | WriteBuffer[bytes],
 *,
 convert_dates: dict[Hashable, str] | None = None,
 write_index: bool = True,
 byteorder: ToStataByteorder | None = None,
 time_stamp: datetime.datetime | None = None,
 data_label: str | None = None,
 variable_labels: dict[Hashable, str] | None = None,
 version: int | None = 114,
 convert_strl: Sequence[Hashable] | None = None,
 compression: CompressionOptions = 'infer',
 storage_options: StorageOptions | None = None,
 value_labels: dict[Hashable, dict[float, str]] | None = None
) -> None from pandas.core.frame.DataFrame
Export DataFrame object to Stata dta format.
```

Writes the `DataFrame` to a Stata dataset file.  
"dta" files contain a Stata dataset.

Parameters

-----

`path` : str, path object, or buffer

String, path object (implementing `os.PathLike[str]`), or file-like object implementing a binary `write()` function.



```

convert_dates : dict
 Dictionary mapping columns containing datetime types to stata
 internal format to use when writing the dates. Options are 'tc',
 'td', 'tm', 'tw', 'th', 'tq', 'ty'. Column can be either an integer
 or a name. Datetime columns that do not have a conversion type
 specified will be converted to 'tc'. Raises NotImplementedError if
 a datetime column has timezone information.
write_index : bool
 Write the index to Stata dataset.
byteorder : str
 Can be ">", "<", "little", or "big". default is `sys.byteorder`.
time_stamp : datetime
 A datetime to use as file creation date. Default is the current
 time.
data_label : str, optional
 A label for the data set. Must be 80 characters or smaller.
variable_labels : dict
 Dictionary containing columns as keys and variable labels as
 values. Each label must be 80 characters or smaller.
version : {{114, 117, 118, 119, None}}, default 114
 Version to use in the output dta file. Set to None to let pandas
 decide between 118 or 119 formats depending on the number of
 columns in the frame. Version 114 can be read by Stata 10 and
 later. Version 117 can be read by Stata 13 or later. Version 118
 is supported in Stata 14 and later. Version 119 is supported in
 Stata 15 and later. Version 114 limits string variables to 244
 characters or fewer while versions 117 and later allow strings
 with lengths up to 2,000,000 characters. Versions 118 and 119
 support Unicode characters, and version 119 supports more than
 32,767 variables.

 Version 119 should usually only be used when the number of
 variables exceeds the capacity of dta format 118. Exporting
 smaller datasets in format 119 may have unintended consequences,
 and, as of November 2020, Stata SE cannot read version 119 files.

convert_strl : list, optional
 List of column names to convert to string columns to Stata StrL
 format. Only available if version is 117. Storing strings in the
 StrL format can produce smaller dta files if strings have more than
 8 characters and values are repeated.

compression : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and 'path' is
 path-like, then detect compression from the following extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 Set to ``None`` for no compression.
 Can also be a dict with key ``method`` set to one of
 {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
 other key-value pairs are forwarded to
 ``zipfile.ZipFile``, ``gzip.GzipFile``,
 ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
 ``tarfile.TarFile``, respectively.
 As an example, the following could be passed for faster compression and
 to create a reproducible gzip archive:
 ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g.
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
 are forwarded to ``urllib.request.Request`` as header options. For other
 URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are
 forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
 details, and for more examples on storage options refer `here`
 <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

value_labels : dict of dicts
 Dictionary containing columns as keys and dictionaries of column value
 to labels as values. Labels for a single variable must be 32,000
 characters or smaller.

Raises

NotImplementedError

```

- \* If datetimes contain timezone information
- \* Column dtype is not representable in Stata

ValueError

- \* Columns listed in convert\_dates are neither datetime64[ns] or datetime.datetime
- \* Column listed in convert\_dates is not in DataFrame
- \* Categorical label contains more than 32,000 characters

See Also

-----

read\_stata : Import Stata data files.  
 io.stata.StataWriter : Low-level writer for Stata data files.  
 io.stata.StataWriter117 : Low-level writer for version 117 files.

Examples

-----

```
>>> df = pd.DataFrame(
... [["falcon", 350], ["parrot", 18]], columns=["animal", "parrot"]
...)
>>> df.to_stata("animals.dta") # doctest: +SKIP
```

```
to_string(
 self,
 buf: FilePath | WriteBuffer[str] | None = None,
 *,
 columns: Axes | None = None,
 col_space: int | list[int] | dict[Hashable, int] | None = None,
 header: bool | SequenceNotStr[str] = True,
 index: bool = True,
 na_rep: str = 'NaN',
 formatters: fmt.FormattersType | None = None,
 float_format: fmt.FloatFormatType | None = None,
 sparsify: bool | None = None,
 index_names: bool = True,
 justify: str | None = None,
 max_rows: int | None = None,
 max_cols: int | None = None,
 show_dimensions: bool = False,
 decimal: str = '.',
 line_width: int | None = None,
 min_rows: int | None = None,
 max_colwidth: int | None = None,
 encoding: str | None = None
) -> str | None from pandas.core.frame.DataFrame
 Render a DataFrame to a console-friendly tabular output.
```

Parameters

-----

buf : str, Path or StringIO-like, optional, default None  
 Buffer to write to. If None, the output is returned as a string.

columns : array-like, optional, default None  
 The subset of columns to write. Writes all columns by default.

col\_space : int, list or dict of int, optional  
 The minimum width of each column. If a list of ints is given every integers

header : bool or list of str, optional  
 Write out the column names. If a list of columns is given, it is assumed to

index : bool, optional, default True  
 Whether to print index (row) labels.

na\_rep : str, optional, default 'NaN'  
 String representation of ``NaN`` to use.

formatters : list, tuple or dict of one-param. functions, optional  
 Formatter functions to apply to columns' elements by position or name.  
 The result of each function must be a unicode string.  
 List/tuple must be of length equal to the number of columns.

float\_format : one-parameter function, optional, default None  
 Formatter function to apply to columns' elements if they are floats. This function must return a unicode string and will be applied only to the non-``NaN`` elements, with ``NaN`` being handled by ``na\_rep``.

sparsify : bool, optional, default True  
 Set to False for a DataFrame with a hierarchical index to print every multiindex key at each row.

index\_names : bool, optional, default True  
 Prints the names of the indexes.

justify : str, default None  
 How to justify the column labels. If None uses the option from

the print configuration (controlled by set\_option), 'right' out of the box. Valid values are

```
* left
* right
* center
* justify
* justify-all
* start
* end
* inherit
* match-parent
* initial
* unset.
max_rows : int, optional
 Maximum number of rows to display in the console.
max_cols : int, optional
 Maximum number of columns to display in the console.
show_dimensions : bool, default False
 Display DataFrame dimensions (number of rows by number of columns).
decimal : str, default '.'
 Character recognized as decimal separator, e.g. ',' in Europe.
```

```
line_width : int, optional
 Width to wrap a line in characters.
min_rows : int, optional
 The number of rows to display in the console in a truncated repr
 (when number of rows is above `max_rows`).
max_colwidth : int, optional
 Max width to truncate each column in characters. By default, no limit.
encoding : str, default "utf-8"
 Set character encoding.
```

```
Returns

str or None
 If buf is None, returns the result as a string. Otherwise returns
 None.
```

See Also

-----  
to\_html : Convert DataFrame to HTML.

Examples

```

>>> d = {"col1": [1, 2, 3], "col2": [4, 5, 6]}
>>> df = pd.DataFrame(d)
>>> print(df.to_string())
 col1 col2
0 1 4
1 2 5
2 3 6
```

```
to_timestamp(
 self,
 freq: Frequency | None = None,
 how: ToTimestampHow = 'start',
 axis: Axis = 0,
 copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
 Cast PeriodIndex to DatetimeIndex of timestamps, at *beginning* of period.
```

This can be changed to the \*end\* of the period, by specifying `how="e"`.

Parameters

```

freq : str, default frequency of PeriodIndex
 Desired frequency.
how : {'s', 'e', 'start', 'end'}
 Convention for converting period to timestamp; start of period
 vs. end.
axis : {0 or 'index', 1 or 'columns'}, default 0
 The axis to convert (the index by default).
copy : bool, default False
 This keyword is now ignored; changing its value will have no
 impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_` for more details.

Returns

-----  
DataFrame with DatetimeIndex  
DataFrame with the PeriodIndex cast to DatetimeIndex.

See Also

-----  
DataFrame.to\_period: Inverse method to cast DatetimeIndex to PeriodIndex.  
Series.to\_timestamp: Equivalent method for Series.

Examples

-----  
>>> idx = pd.PeriodIndex(["2023", "2024"], freq="Y")  
>>> d = {"col1": [1, 2], "col2": [3, 4]}  
>>> df1 = pd.DataFrame(data=d, index=idx)  
>>> df1

	col1	col2
2023	1	3
2024	2	4

The resulting timestamps will be at the beginning of the year in this case

>>> df1 = df1.to\_timestamp()  
>>> df1

	col1	col2
2023-01-01	1	3
2024-01-01	2	4

>>> df1.index  
DatetimeIndex(['2023-01-01', '2024-01-01'], dtype='datetime64[us]', freq=None)

Using `freq` which is the offset that the Timestamps will have

>>> df2 = pd.DataFrame(data=d, index=idx)  
>>> df2 = df2.to\_timestamp(freq="M")  
>>> df2

	col1	col2
2023-01-31	1	3
2024-01-31	2	4

>>> df2.index  
DatetimeIndex(['2023-01-31', '2024-01-31'], dtype='datetime64[us]', freq=None)

to\_xml(  
 self,  
 path\_or\_buffer: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,  
 \*,  
 index: bool = True,  
 root\_name: str | None = 'data',  
 row\_name: str | None = 'row',  
 na\_rep: str | None = None,  
 attr\_cols: list[str] | None = None,  
 elem\_cols: list[str] | None = None,  
 namespaces: dict[str | None, str] | None = None,  
 prefix: str | None = None,  
 encoding: str = 'utf-8',  
 xml\_declaration: bool | None = True,  
 pretty\_print: bool | None = True,  
 parser: XMLParsers | None = 'lxml',  
 stylesheet: FilePath | ReadBuffer[str] | ReadBuffer[bytes] | None = None,  
 compression: CompressionOptions = 'infer',  
 storage\_options: StorageOptions | None = None  
) -> str | None from pandas.core.frame.DataFrame  
Render a DataFrame to an XML document.

Parameters

-----  
path\_or\_buffer : str, path object, file-like object, or None, default None  
String, path object (implementing `os.PathLike[str]`), or file-like object implementing a `write()` function. If None, the result is returned

```

 as a string.
index : bool, default True
 Whether to include index in XML document.
root_name : str, default 'data'
 The name of root element in XML document.
row_name : str, default 'row'
 The name of row element in XML document.
na_rep : str, optional
 Missing data representation.
attr_cols : list-like, optional
 List of columns to write as attributes in row element.
 Hierarchical columns will be flattened with underscore
 delimiting the different levels.
elem_cols : list-like, optional
 List of columns to write as children in row element. By default,
 all columns output as children of row element. Hierarchical
 columns will be flattened with underscore delimiting the
 different levels.
namespaces : dict, optional
 All namespaces to be defined in root element. Keys of dict
 should be prefix names and values of dict corresponding URIs.
 Default namespaces should be given empty string key. For
 example, ::

 namespaces = {"": "https://example.com"}

prefix : str, optional
 Namespace prefix to be used for every element and/or attribute
 in document. This should be one of the keys in ``namespaces``
 dict.
encoding : str, default 'utf-8'
 Encoding of the resulting document.
xml_declaration : bool, default True
 Whether to include the XML declaration at start of document.
pretty_print : bool, default True
 Whether output should be pretty printed with indentation and
 line breaks.
parser : {'lxml', 'etree'}, default 'lxml'
 Parser module to use for building of tree. Only 'lxml' and
 'etree' are supported. With 'lxml', the ability to use XSLT
 stylesheet is supported.
stylesheet : str, path object or file-like object, optional
 A URL, file-like object, or a raw string containing an XSLT
 script used to transform the raw XML output. Script should use
 layout of elements and attributes from original output. This
 argument requires ``lxml`` to be installed. Only XSLT 1.0
 scripts and not later versions is currently supported.
compression : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and
 'path_or_buffer' is
 path-like, then detect compression from the following extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 Set to ``None`` for no compression.
 Can also be a dict with key ``'method'`` set to one of
 {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
 other key-value pairs are forwarded to
 ``zipfile.ZipFile``, ``gzip.GzipFile``,
 ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
 ``tarfile.TarFile``, respectively.
 As an example, the following could be passed for faster compression and
 to create a reproducible gzip archive:
 ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.
storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g.
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
 are forwarded to ``urllib.request.Request`` as header options. For other
 URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
 forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
 details, and for more examples on storage options refer `here
 <https://pandas.pydata.org/docs/user_guide/io.html?
 highlight=storage_options#reading-writing-remote-files>`_.

```

#### Returns

-----

None or str

If ``io`` is None, returns the resulting XML format as a

string. Otherwise returns None.

#### See Also

`to_json` : Convert the pandas object to a JSON string.  
`to_html` : Convert DataFrame to a html.

#### Examples

```
>>> df = pd.DataFrame(
... [["square", 360, 4], ["circle", 360, np.nan], ["triangle", 180, 3]],
... columns=["shape", "degrees", "sides"],
...)
```

```
>>> df.to_xml() # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
```

```
<data>
 <row>
 <index>0</index>
 <shape>square</shape>
 <degrees>360</degrees>
 <sides>4.0</sides>
 </row>
 <row>
 <index>1</index>
 <shape>circle</shape>
 <degrees>360</degrees>
 <sides/>
 </row>
 <row>
 <index>2</index>
 <shape>triangle</shape>
 <degrees>180</degrees>
 <sides>3.0</sides>
 </row>
</data>
```

```
>>> df.to_xml(
... attr_cols=["index", "shape", "degrees", "sides"]
...) # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
<data>
 <row index="0" shape="square" degrees="360" sides="4.0"/>
 <row index="1" shape="circle" degrees="360"/>
 <row index="2" shape="triangle" degrees="180" sides="3.0"/>
</data>
```

```
>>> df.to_xml(
... namespaces={"doc": "https://example.com"}, prefix="doc"
...) # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
<doc:data xmlns:doc="https://example.com">
 <doc:row>
 <doc:index>0</doc:index>
 <doc:shape>square</doc:shape>
 <doc:degrees>360</doc:degrees>
 <doc:sides>4.0</doc:sides>
 </doc:row>
 <doc:row>
 <doc:index>1</doc:index>
 <doc:shape>circle</doc:shape>
 <doc:degrees>360</doc:degrees>
 <doc:sides/>
 </doc:row>
 <doc:row>
 <doc:index>2</doc:index>
 <doc:shape>triangle</doc:shape>
 <doc:degrees>180</doc:degrees>
 <doc:sides>3.0</doc:sides>
 </doc:row>
</doc:data>
```

`transform(self, func: AggFuncType, axis: Axis = 0, *args, **kwargs) -> DataFrame` from pandas  
Call ``func`` on self producing a DataFrame with the same axis shape as self.

#### Parameters

```

func : function, str, list-like or dict-like
 Function to use for transforming the data. If a function, must either
 work when passed a DataFrame or when passed to DataFrame.apply. If func
 is both list-like and dict-like, dict-like behavior takes precedence.

 Accepted combinations are:

 - function
 - string function name
 - list-like of functions and/or function names, e.g. ``[np.exp, 'sqrt']``
 - dict-like of axis labels -> functions, function names or list-like
 of such.
axis : {0 or 'index', 1 or 'columns'}, default 0
 If 0 or 'index': apply function to each column.
 If 1 or 'columns': apply function to each row.
*args
 Positional arguments to pass to `func`.
**kwargs
 Keyword arguments to pass to `func`.

Returns

DataFrame
 A DataFrame that must have the same length as self.

Raises

ValueError : If the returned DataFrame has a different length than self.

See Also

DataFrame.agg : Only perform aggregating type operations.
DataFrame.apply : Invoke function on a DataFrame.

Notes

Functions that mutate the passed object can produce unexpected
behavior or errors and are not supported. See :ref:`gotchas.udf-mutation`
for more details.

Examples

>>> df = pd.DataFrame({"A": range(3), "B": range(1, 4)})
>>> df
 A B
0 0 1
1 1 2
2 2 3
>>> df.transform(lambda x: x + 1)
 A B
0 1 2
1 2 3
2 3 4

Even though the resulting DataFrame must have the same length as the
input DataFrame, it is possible to provide several input functions:

>>> s = pd.Series(range(3))
>>> s
0 0
1 1
2 2
dtype: int64
>>> s.transform([np.sqrt, np.exp])
 sqrt exp
0 0.000000 1.000000
1 1.000000 2.718282
2 1.414214 7.389056

You can call transform on a GroupBy object:

>>> df = pd.DataFrame(
... {
... "Date": [
... "2015-05-08",
... "2015-05-07",
... "2015-05-06",

```

```

... "2015-05-05",
... "2015-05-08",
... "2015-05-07",
... "2015-05-06",
... "2015-05-05",
...],
... "Data": [5, 8, 6, 1, 50, 100, 60, 120],
...)
>>> df
 Date Data
0 2015-05-08 5
1 2015-05-07 8
2 2015-05-06 6
3 2015-05-05 1
4 2015-05-08 50
5 2015-05-07 100
6 2015-05-06 60
7 2015-05-05 120
>>> df.groupby("Date")["Data"].transform("sum")
0 55
1 108
2 66
3 121
4 55
5 108
6 66
7 121
Name: Data, dtype: int64

>>> df = pd.DataFrame(
... {
... "c": [1, 1, 1, 2, 2, 2, 2],
... "type": ["m", "n", "o", "m", "m", "n", "n"],
... }
...)
>>> df
 c type
0 1 m
1 1 n
2 1 o
3 2 m
4 2 m
5 2 n
6 2 n
>>> df["size"] = df.groupby("c")["type"].transform(len)
>>> df
 c type size
0 1 m 3
1 1 n 3
2 1 o 3
3 2 m 4
4 2 m 4
5 2 n 4
6 2 n 4

```

`transpose(self, *args, copy: bool | lib.NoDefault = <no_default>) -> DataFrame from pandas.`  
 Transpose index and columns.

Reflect the DataFrame over its main diagonal by writing rows as columns and vice-versa. The property `:attr:'.T'` is an accessor to the method `:meth:'.transpose'`.

Parameters  
 -----

`*args` : tuple, optional  
 Accepted for compatibility with NumPy.  
`copy` : bool, default False  
 This keyword is now ignored; changing its value will have no impact on the method.

Note that a copy is always required for mixed dtype DataFrames, or for DataFrames with any extension types.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since



pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_ for more details.

#### Returns

-----  
DataFrame

The transposed DataFrame.

#### See Also

-----  
numpy.transpose : Permute the dimensions of a given array.

#### Notes

-----  
Transposing a DataFrame with mixed dtypes will result in a homogeneous DataFrame with the `object` dtype. In such a case, a copy of the data is always made.

#### Examples

-----  
\*\*Square DataFrame with homogeneous dtype\*\*

```
>>> d1 = {"col1": [1, 2], "col2": [3, 4]}
>>> df1 = pd.DataFrame(data=d1)
>>> df1
 col1 col2
0 1 3
1 2 4

>>> df1_transposed = df1.T # or df1.transpose()
>>> df1_transposed
 0 1
col1 1 2
col2 3 4
```

When the dtype is homogeneous in the original DataFrame, we get a transposed DataFrame with the same dtype:

```
>>> df1.dtypes
col1 int64
col2 int64
dtype: object
>>> df1_transposed.dtypes
0 int64
1 int64
dtype: object
```

\*\*Non-square DataFrame with mixed dtypes\*\*

```
>>> d2 = {
... "name": ["Alice", "Bob"],
... "score": [9.5, 8],
... "employed": [False, True],
... "kids": [0, 0],
... }
>>> df2 = pd.DataFrame(data=d2)
>>> df2
 name score employed kids
0 Alice 9.5 False 0
1 Bob 8.0 True 0

>>> df2_transposed = df2.T # or df2.transpose()
>>> df2_transposed
 0 1
name Alice Bob
score 9.5 8.0
employed False True
kids 0 0
```

When the DataFrame has mixed dtypes, we get a transposed DataFrame with the `object` dtype:

```
>>> df2.dtypes
name str
```

```

score float64
employed bool
kids int64
dtype: object
>>> df2_transposed.dtypes
0 object
1 object
dtype: object

```

`truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from`  
 Get Floating division of dataframe and other, element-wise (binary operator ``truediv``).

Equivalent to ``dataframe / other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``rtruediv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

#### Parameters

`other` : scalar, sequence, Series, dict or DataFrame  
 Any single or multiple element data structure, or list-like object.  
`axis` : {0 or 'index', 1 or 'columns'}  
 Whether to compare by the index (0 or 'index') or columns.  
 (1 or 'columns'). For Series input, axis to match Series index on.  
`level` : int or label  
 Broadcast across a level, matching Index values on the  
 passed MultiIndex level.  
`fill_value` : float or None, default None  
 Fill existing missing (NaN) values, and any new element needed for  
 successful DataFrame alignment, with this value before computation.  
 If data in both corresponding DataFrame locations is missing  
 the result will be missing.

#### Returns

DataFrame  
 Result of the arithmetic operation.

#### See Also

`DataFrame.add` : Add DataFrames.  
`DataFrame.sub` : Subtract DataFrames.  
`DataFrame.mul` : Multiply DataFrames.  
`DataFrame.div` : Divide DataFrames (float division).  
`DataFrame.truediv` : Divide DataFrames (float division).  
`DataFrame.floordiv` : Divide DataFrames (integer division).  
`DataFrame.mod` : Calculate modulo (remainder after division).  
`DataFrame.pow` : Calculate exponential power.

#### Notes

Mismatched indices will be unioned together.

#### Examples

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df
 angles degrees
circle 0 360
triangle 3 180
rectangle 4 360

```

Add a scalar with operator version which return the same results.

```

>>> df + 1
 angles degrees
circle 1 361
triangle 4 181
rectangle 5 361

>>> df.add(1)
 angles degrees
circle 1 361

```

triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
 angles degrees
circle 0.0 36.0
triangle 0.3 18.0
rectangle 0.4 36.0

>>> df.rdiv(10)
 angles degrees
circle inf 0.027778
triangle 3.333333 0.055556
rectangle 2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub([1, 2], axis='columns')
 angles degrees
circle -1 358
triangle 2 178
rectangle 3 358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')
 angles degrees
circle -1 359
triangle 2 179
rectangle 3 359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
 angles degrees
circle 0 720
triangle 0 360
rectangle 0 720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
 angles degrees
circle 0 0
triangle 6 360
rectangle 12 1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
... index=['circle', 'triangle', 'rectangle'])
>>> other
 angles
circle 0
triangle 3
rectangle 4

>>> df * other
 angles degrees
circle 0 NaN
triangle 9 NaN
rectangle 16 NaN

>>> df.mul(other, fill_value=0)
 angles degrees
circle 0 0.0
triangle 9 0.0
rectangle 16 0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
```

```
... 'degrees': [360, 180, 360, 360, 540, 720]},
... index=[['A', 'A', 'A', 'B', 'B', 'B'],
... ['circle', 'triangle', 'rectangle',
... 'square', 'pentagon', 'hexagon']])
```

```
>>> df_multindex
 angles degrees
A circle 0 360
 triangle 3 180
 rectangle 4 360
B square 4 360
 pentagon 5 540
 hexagon 6 720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
```

```
 angles degrees
A circle NaN 1.0
 triangle 1.0 1.0
 rectangle 1.0 1.0
B square 0.0 0.0
 pentagon 0.0 0.0
 hexagon 0.0 0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
... 'B': [6, 7, 8, 9]})
```

```
>>> df_pow.pow(2)
```

```
 A B
0 4 36
1 9 49
2 16 64
3 25 81
```

unstack(self, level: IndexLabel = -1, fill\_value=None, sort: bool = True) -> DataFrame | Series  
Pivot a level of the (necessarily hierarchical) index labels.

Returns a DataFrame having a new level of column labels whose inner-most level consists of the pivoted index labels.

If the index is not a MultiIndex, the output will be a Series (the analogue of stack when the columns are not a MultiIndex).

#### Parameters

level : int, str, or list of these, default -1 (last level)  
Level(s) of index to unstack, can pass level name.  
fill\_value : scalar  
Replace NaN with this value if the unstack produces missing values.  
sort : bool, default True  
Sort the level(s) in the resulting MultiIndex columns.

#### Returns

Series or DataFrame  
If index is a MultiIndex: DataFrame with pivoted index labels as new inner-most level column labels, else Series.

#### See Also

DataFrame.pivot : Pivot a table based on column values.  
DataFrame.stack : Pivot a level of the column labels (inverse operation from 'unstack').

#### Notes

Reference :ref:`the user guide <reshaping.stacking>` for more examples.

#### Examples

```
>>> index = pd.MultiIndex.from_tuples(
... [("one", "a"), ("one", "b"), ("two", "a"), ("two", "b")])
...)
>>> s = pd.Series(np.arange(1.0, 5.0), index=index)
>>> s
one a 1.0
 b 2.0
two a 3.0
 b 4.0
dtype: float64
```

```

>>> s.unstack(level=-1)
 a b
one 1.0 2.0
two 3.0 4.0

>>> s.unstack(level=0)
 one two
a 1.0 3.0
b 2.0 4.0

>>> df = s.unstack(level=0)
>>> df.unstack()
one a 1.0
 b 2.0
two a 3.0
 b 4.0
dtype: float64

```

```

update(
 self,
 other,
 join: UpdateJoin = 'left',
 overwrite: bool = True,
 filter_func=None,
 errors: IgnoreRaise = 'ignore'
) -> None from pandas.core.frame.DataFrame
 Modify in place using non-NA values from another DataFrame.

```

Aligns on indices. There is no return value.

#### Parameters

```

other : DataFrame, or object coercible into a DataFrame
 Should have at least one matching index/column label
 with the original DataFrame. If a Series is passed,
 its name attribute must be set, and that will be
 used as the column name to align with the original DataFrame.
join : {'left'}, default 'left'
 Only left join is implemented, keeping the index and columns of the
 original object.
overwrite : bool, default True
 How to handle non-NA values for overlapping keys:

 * True: overwrite original DataFrame's values
 with values from `other`.
 * False: only update values that are NA in
 the original DataFrame.

filter_func : callable(1d-array) -> bool 1d-array, optional
 Can choose to replace values other than NA. Return True for values
 that should be updated.
errors : {'raise', 'ignore'}, default 'ignore'
 If 'raise', will raise a ValueError if the DataFrame and `other`
 both contain non-NA data in the same place.

```

#### Returns

```

None
 This method directly changes calling object.

```

#### Raises

```

ValueError
 * When `errors='raise'` and there's overlapping non-NA data.
 * When `errors` is not either ``'ignore'`` or ``'raise'``
NotImplementedError
 * If `join != 'left'`

```

#### See Also

```

dict.update : Similar method for dictionaries.
DataFrame.merge : For column(s)-on-column(s) operations.

```

#### Notes

```

1. Duplicate indices on `other` are not supported and raises `ValueError`.

```

## Examples

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [400, 500, 600]})
>>> new_df = pd.DataFrame({"B": [4, 5, 6], "C": [7, 8, 9]})
>>> df.update(new_df)
>>> df
 A B
0 1 4
1 2 5
2 3 6
```

The DataFrame's length does not increase as a result of the update, only values at matching index/column labels are updated.

```
>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_df = pd.DataFrame({"B": ["d", "e", "f", "g", "h", "i"]})
>>> df.update(new_df)
>>> df
 A B
0 a d
1 b e
2 c f
```

```
>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_df = pd.DataFrame({"B": ["d", "f"]}, index=[0, 2])
>>> df.update(new_df)
>>> df
 A B
0 a d
1 b y
2 c f
```

For Series, its name attribute must be set.

```
>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_column = pd.Series(["d", "e", "f"], name="B")
>>> df.update(new_column)
>>> df
 A B
0 a d
1 b e
2 c f
```

If `other` contains NaNs the corresponding values are not updated in the original dataframe.

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [400.0, 500.0, 600.0]})
>>> new_df = pd.DataFrame({"B": [4, np.nan, 6]})
>>> df.update(new_df)
>>> df
 A B
0 1 4.0
1 2 500.0
2 3 6.0
```

```
value_counts(
 self,
 subset: IndexLabel | None = None,
 normalize: bool = False,
 sort: bool = True,
 ascending: bool = False,
 dropna: bool = True
) -> Series from pandas.core.frame.DataFrame
Return a Series containing the frequency of each distinct row in the DataFrame.
```

## Parameters

subset : Hashable or a sequence of the previous, optional  
Columns to use when counting unique combinations.  
normalize : bool, default False  
Return proportions rather than frequencies.  
sort : bool, default True  
Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, ``sort=False`` would sort by the columns values.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False

Sort in ascending order.

dropna : bool, default True

Do not include counts of rows that contain NA values.

Returns

-----

Series

Series containing the frequency of each distinct row in the DataFrame.

See Also

-----

Series.value\_counts: Equivalent method on Series.

Notes

-----

The returned Series will have a MultiIndex with one level per input column but an Index (non-multi) for a single label. By default, rows that contain any NA values are omitted from the result. By default, the resulting Series will be sorted by frequencies in descending order so that the first element is the most frequently-occurring row.

Examples

-----

```
>>> df = pd.DataFrame(
... {"num_legs": [2, 4, 4, 6], "num_wings": [2, 0, 0, 0]},
... index=["falcon", "dog", "cat", "ant"],
...)
>>> df
```

	num_legs	num_wings
falcon	2	2
dog	4	0
cat	4	0
ant	6	0

```
>>> df.value_counts()
num_legs num_wings
4 0 2
2 2 1
6 0 1
Name: count, dtype: int64
```

```
>>> df.value_counts(sort=False)
num_legs num_wings
2 2 1
4 0 2
6 0 1
Name: count, dtype: int64
```

```
>>> df.value_counts(ascending=True)
num_legs num_wings
2 2 1
6 0 1
4 0 2
Name: count, dtype: int64
```

```
>>> df.value_counts(normalize=True)
num_legs num_wings
4 0 0.50
2 2 0.25
6 0 0.25
Name: proportion, dtype: float64
```

With `dropna` set to `False` we can also count rows with NA values.

```
>>> df = pd.DataFrame(
... {
... "first_name": ["John", "Anne", "John", "Beth"],
... "middle_name": ["Smith", pd.NA, pd.NA, "Louise"],
... }
...)
```

```

>>> df
 first_name middle_name
0 John Smith
1 Anne NaN
2 John NaN
3 Beth Louise

>>> df.value_counts()
first_name middle_name
John Smith 1
Beth Louise 1
Name: count, dtype: int64

>>> df.value_counts(dropna=False)
first_name middle_name
John Smith 1
Anne NaN 1
John NaN 1
Beth Louise 1
Name: count, dtype: int64

>>> df.value_counts("first_name")
first_name
John 2
Anne 1
Beth 1
Name: count, dtype: int64

```

var(  
    self,  
    \*,  
    axis: Axis | None = 0,  
    skipna: bool = True,  
    ddof: int = 1,  
    numeric\_only: bool = False,  
    \*\*kwargs  
) -> Series | Any from pandas.core.frame.DataFrame  
Return unbiased variance over requested axis.

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters  
-----  
axis : {index (0), columns (1)}  
    For `Series` this parameter is unused and defaults to 0.

.. warning::  
    The behavior of DataFrame.var with ``axis=None`` is deprecated,  
    in a future version this will reduce over both axes and return a scalar  
    To retain the old behavior, pass axis=0 (or do not pass axis).

skipna : bool, default True  
    Exclude NA/null values. If an entire row/column is NA, the result  
    will be NA.

ddof : int, default 1  
    Delta Degrees of Freedom. The divisor used in calculations is N - ddof,  
    where N represents the number of elements.

numeric\_only : bool, default False  
    Include only float, int, boolean columns. Not implemented for Series.

\*\*kwargs :  
    Additional keywords passed.

Returns  
-----  
Series or scalar  
    Unbiased variance over requested axis.

See Also  
-----  
numpy.var : Equivalent function in NumPy.  
Series.var : Return unbiased variance over Series values.  
Series.std : Return standard deviation over Series values.  
DataFrame.std : Return standard deviation of the values over  
    the requested axis.

Examples



```

>>> df = pd.DataFrame(
... {
... "person_id": [0, 1, 2, 3],
... "age": [21, 25, 62, 43],
... "height": [1.61, 1.87, 1.49, 2.01],
... }
...).set_index("person_id")
>>> df

```

```

 age height
person_id
0 21 1.61
1 25 1.87
2 62 1.49
3 43 2.01

```

```

>>> df.var()
age 352.916667
height 0.056367
dtype: float64

```

Alternatively, ``ddof=0`` can be set to normalize by N instead of N-1:

```

>>> df.var(ddof=0)
age 264.687500
height 0.042275
dtype: float64

```

-----  
Class methods defined here:

`from_arrow(data: ArrowArrayExportable | ArrowStreamExportable) -> DataFrame` from pandas.core  
Construct a DataFrame from a tabular Arrow object.

This function accepts any Arrow-compatible tabular object implementing the `Arrow PyCapsule Protocol` (i.e. having an `__arrow_c_array__` or `__arrow_c_stream__` method).

This function currently relies on `pyarrow` to convert the tabular object in Arrow format to pandas.

.. `_Arrow PyCapsule Protocol`: <https://arrow.apache.org/docs/format/CDataInterface/PyCapsuleProtocol.html>

.. versionadded:: 3.0

Parameters

`data` : `pyarrow.Table` or Arrow-compatible table  
Any tabular object implementing the `Arrow PyCapsule Protocol` (i.e. has an `__arrow_c_array__` or `__arrow_c_stream__` method).

Returns

-----  
`DataFrame`

See Also

-----  
`Series.from_arrow` : Construct a Series from an Arrow object.

Examples

```

>>> import pyarrow as pa
>>> table = pa.table({"a": [1, 2, 3], "b": ["x", "y", "z"]})
>>> pd.DataFrame.from_arrow(table)
 a b
0 1 x
1 2 y
2 3 z

```

```

from_dict(
 data: dict,
 orient: FromDictOrient = 'columns',
 dtype: Dtype | None = None,
 columns: Axes | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Construct DataFrame from dict of array-like or dicts.

```

Creates DataFrame object from dictionary by columns or by index allowing dtype specification.

#### Parameters

`data` : dict  
Of the form {field : array-like} or {field : dict}.  
`orient` : {'columns', 'index', 'tight'}, default 'columns'  
The "orientation" of the data. If the keys of the passed dict should be the columns of the resulting DataFrame, pass 'columns' (default). Otherwise if the keys should be rows, pass 'index'. If 'tight', assume a dict with keys ['index', 'columns', 'data', 'index\_names', 'column\_names'].  
`dtype` : dtype, default None  
Data type to force after DataFrame construction, otherwise infer.  
`columns` : list, default None  
Column labels to use when ``orient='index'``. Raises a ValueError if used with ``orient='columns'`` or ``orient='tight'``.

#### Returns

DataFrame

#### See Also

`DataFrame.from_records` : DataFrame from structured ndarray, sequence of tuples or dicts, or DataFrame.  
`DataFrame` : DataFrame object creation using constructor.  
`DataFrame.to_dict` : Convert the DataFrame to a dictionary.

#### Examples

By default the keys of the dict become the DataFrame columns:

```
>>> data = {"col_1": [3, 2, 1, 0], "col_2": ["a", "b", "c", "d"]}
>>> pd.DataFrame.from_dict(data)
 col_1 col_2
0 3 a
1 2 b
2 1 c
3 0 d
```

Specify ``orient='index'`` to create the DataFrame using dictionary keys as rows:

```
>>> data = {"row_1": [3, 2, 1, 0], "row_2": ["a", "b", "c", "d"]}
>>> pd.DataFrame.from_dict(data, orient="index")
 row_1 row_2
0 3 a
1 2 b
2 1 c
3 0 d
```

When using the 'index' orientation, the column names can be specified manually:

```
>>> pd.DataFrame.from_dict(data, orient="index", columns=["A", "B", "C", "D"])
 row_1 row_2
0 3 a
1 2 b
2 1 c
3 0 d
```

Specify ``orient='tight'`` to create the DataFrame using a 'tight' format:

```
>>> data = {
... "index": [("a", "b"), ("a", "c")],
... "columns": [("x", 1), ("y", 2)],
... "data": [[1, 3], [2, 4]],
... "index_names": ["n1", "n2"],
... "column_names": ["z1", "z2"],
... }
>>> pd.DataFrame.from_dict(data, orient="tight")
 n1 n2
a z1 z2
b x y
c 1 2
```

```

from_records(
 data,
 index=None,
 exclude=None,
 columns=None,
 coerce_float: bool = False,
 nrows: int | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Convert structured or record ndarray to DataFrame.

Creates a DataFrame object from a structured ndarray, or iterable of
tuples or dicts.

Parameters

data : structured ndarray, iterable of tuples or dicts
 Structured input data.
index : str, list of fields, array-like
 Field of array to use as the index, alternately a specific set of
 input labels to use.
exclude : sequence, default None
 Columns or fields to exclude.
columns : sequence, default None
 Column names to use. If the passed data do not have names
 associated with them, this argument provides names for the
 columns. Otherwise, this argument indicates the order of the columns
 in the result (any names not found in the data will become all-NA
 columns) and limits the data to these columns if not all column names
 are provided.
coerce_float : bool, default False
 Attempt to convert values of non-string, non-numeric objects (like
 decimal.Decimal) to floating point, useful for SQL result sets.
nrows : int, default None
 Number of rows to read if data is an iterator.

Returns

DataFrame

See Also

DataFrame.from_dict : DataFrame from dict of array-like or dicts.
DataFrame : DataFrame object creation using constructor.

Examples

Data can be provided as a structured ndarray:

>>> data = np.array(
... [(3, "a"), (2, "b"), (1, "c"), (0, "d")],
... dtype=[("col_1", "i4"), ("col_2", "U1")],
...)
>>> pd.DataFrame.from_records(data)
 col_1 col_2
0 3 a
1 2 b
2 1 c
3 0 d

Data can be provided as a list of dicts:

>>> data = [
... {"col_1": 3, "col_2": "a"},
... {"col_1": 2, "col_2": "b"},
... {"col_1": 1, "col_2": "c"},
... {"col_1": 0, "col_2": "d"},
...]
>>> pd.DataFrame.from_records(data)
 col_1 col_2
0 3 a
1 2 b
2 1 c
3 0 d

Data can be provided as a list of tuples with corresponding columns:

```

```
>>> data = [(3, "a"), (2, "b"), (1, "c"), (0, "d")]
>>> pd.DataFrame.from_records(data, columns=["col_1", "col_2"])
 col_1 col_2
0 3 a
1 2 b
2 1 c
3 0 d
```

-----  
Readonly properties defined here:

T

The transpose of the DataFrame.

Returns

-----

DataFrame

The transposed DataFrame.

See Also

-----

DataFrame.transpose : Transpose index and columns.

Examples

-----

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
```

```
>>> df
```

```
 col1 col2
0 1 3
1 2 4
```

```
>>> df.T
```

```
 0 1
col1 1 2
col2 3 4
```

axes

Return a list representing the axes of the DataFrame.

It has the row axis labels and column axis labels as the only members.  
They are returned in that order.

See Also

-----

DataFrame.index: The index (row labels) of the DataFrame.

DataFrame.columns: The column labels of the DataFrame.

Examples

-----

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
```

```
>>> df.axes
```

```
[RangeIndex(start=0, stop=2, step=1), Index(['col1', 'col2'], dtype='str')]
```

shape

Return a tuple representing the dimensionality of the DataFrame.

Unlike the `len()` method, which only returns the number of rows, `shape` provides both row and column counts, making it a more informative method for understanding dataset size.

See Also

-----

numpy.ndarray.shape : Tuple of array dimensions.

Examples

-----

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
```

```
>>> df.shape
```

```
(2, 2)
```

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4], "col3": [5, 6]})
```

```
>>> df.shape
```

```
(2, 3)
```

style

Returns a Styler object.

Contains methods for building a styled HTML representation of the DataFrame.

See Also

`io.formats.style.Styler` : Helps style a DataFrame or Series according to the data with HTML and CSS.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2, 3]})
>>> df.style # doctest: +SKIP
```

Please see

``Table Visualization <../../user_guide/style.ipynb>`_` for more examples.

values

Return a Numpy representation of the DataFrame.

.. warning::

We recommend using `:meth:`DataFrame.to_numpy`` instead.

Only the values in the DataFrame will be returned, the axes labels will be removed.

Returns

`numpy.ndarray`  
The values of the DataFrame.

See Also

`DataFrame.to_numpy` : Recommended alternative to this method.

`DataFrame.index` : Retrieve the index labels.

`DataFrame.columns` : Retrieving the column names.

Notes

The dtype will be a lower-common-denominator dtype (implicit upcasting); that is to say if the dtypes (even of numeric types) are mixed, the one that accommodates all will be chosen. Use this with care if you are not dealing with the blocks.

e.g. If the dtypes are float16 and float32, dtype will be upcast to float32. If dtypes are int32 and uint8, dtype will be upcast to int32. By `:func:`numpy.find_common_type`` convention, mixing int64 and uint64 will result in a float64 dtype.

Examples

A DataFrame where all columns are the same type (e.g., int64) results in an array of the same type.

```
>>> df = pd.DataFrame(
... {"age": [3, 29], "height": [94, 170], "weight": [31, 115]}
...)
>>> df
 age height weight
0 3 94 31
1 29 170 115
>>> df.dtypes
age int64
height int64
weight int64
dtype: object
>>> df.values
array([[3, 94, 31],
 [29, 170, 115]])
```

A DataFrame with mixed type columns(e.g., str/object, int64, float32) results in an ndarray of the broadest type that accommodates these mixed types (e.g., object).

```
>>> df2 = pd.DataFrame(
... [
... ("parrot", 24.0, "second"),
... ("lion", 80.5, 1),
...]
...)
```

```

... ("monkey", np.nan, None),
...],
... columns=("name", "max_speed", "rank"),
...)
>>> df2.dtypes
name str
max_speed float64
rank object
dtype: object
>>> df2.values
array([['parrot', 24.0, 'second'],
 ['lion', 80.5, 1],
 ['monkey', nan, None]], dtype=object)

```

-----  
Data descriptors defined here:

columns

The column labels of the DataFrame.

This property holds the column names as a pandas ``Index`` object. It provides an immutable sequence of column labels that can be used for data selection, renaming, and alignment in DataFrame operations.

Returns

-----

pandas.Index

The column labels of the DataFrame.

See Also

-----

DataFrame.index: The index (row labels) of the DataFrame.

DataFrame.axes: Return a list representing the axes of the DataFrame.

Examples

-----

```
>>> df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
```

```
>>> df
```

```

 A B
0 1 3
1 2 4

```

```
>>> df.columns
```

```
Index(['A', 'B'], dtype='str')
```

index

The index (row labels) of the DataFrame.

The index of a DataFrame is a series of labels that identify each row. The labels can be integers, strings, or any other hashable type. The index is used for label-based access and alignment, and can be accessed or modified using this attribute.

Returns

-----

pandas.Index

The index labels of the DataFrame.

See Also

-----

DataFrame.columns : The column labels of the DataFrame.

DataFrame.to\_numpy : Convert the DataFrame to a NumPy array.

Examples

-----

```

>>> df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Aritra'],
... 'Age': [25, 30, 35],
... 'Location': ['Seattle', 'New York', 'Kona']},
... index=[10, 20, 30])

```

```
>>> df.index
```

```
Index([10, 20, 30], dtype='int64')
```

In this example, we create a DataFrame with 3 rows and 3 columns, including Name, Age, and Location information. We set the index labels to be the integers 10, 20, and 30. We then access the ``index`` attribute of the DataFrame, which returns an ``Index`` object containing the index labels.

```
>>> df.index = [100, 200, 300]
```

```
>>> df
 Name Age Location
100 Alice 25 Seattle
200 Bob 30 New York
300 Aritra 35 Kona
```

In this example, we modify the index labels of the DataFrame by assigning a new list of labels to the `index` attribute. The DataFrame is then updated with the new labels, and the output shows the modified DataFrame.

-----  
Data and other attributes defined here:

```
__annotations__ = {'_AXIS_ORDERS': "list[Literal['index', 'columns']]"...
```

```
__pandas_priority__ = 4000
```

```
plot = <class 'pandas.plotting.PlotAccessor'>
 Make plots of Series or DataFrame.
```

Uses the backend specified by the option ``plotting.backend``. By default, matplotlib is used.

Parameters

-----  
data : Series or DataFrame  
      The object for which the method is called.

Attributes

-----  
x : label or position, default None  
   Only used if data is a DataFrame.  
y : label, position or list of label, positions, default None  
   Allows plotting of one column versus another. Only used if data is a DataFrame.

kind : str  
      The kind of plot to produce:

- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

ax : matplotlib axes object, default None  
    An axes of the current figure.

subplots : bool or sequence of iterables, default False  
          Whether to group columns into subplots:

- ``False`` : No subplots will be used
- ``True`` : Make separate subplots for each column.
- sequence of iterables of column labels: Create a subplot for each group of columns. For example ``[('a', 'c'), ('b', 'd')]`` will create 2 subplots: one with columns 'a' and 'c', and one with columns 'b' and 'd'. Remaining columns that aren't specified will be plotted in additional subplots (one per column).

sharex : bool, default True if ax is None else False  
      In case ``subplots=True``, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in; Be aware, that passing in both an ax and ``sharex=True`` will alter all x axis labels for all axis in a figure.

sharey : bool, default False  
      In case ``subplots=True``, share y axis and set some y axis labels to invisible.

layout : tuple, optional  
        (rows, columns) for the layout of subplots.

figsize : a tuple (width, height) in inches  
          Size of a figure object.

use\_index : bool, default True  
          Use index as ticks for x axis.

title : str or list  
      Title to use for the plot. If a string is passed, print the string

at the top of the figure. If a list is passed and `subplots` is True, print each item in the list above the corresponding subplot.

**grid** : bool, default None (matlab style default)  
Axis grid lines.

**legend** : bool or {'reverse'}  
Place legend on axis subplots.

**style** : list or dict  
The matplotlib line style per column.

**logx** : bool or 'sym', default False  
Use log scaling or symlog scaling on x axis.

**logy** : bool or 'sym' default False  
Use log scaling or symlog scaling on y axis.

**loglog** : bool or 'sym', default False  
Use log scaling or symlog scaling on both x and y axes.

**xticks** : sequence  
Values to use for the xticks.

**yticks** : sequence  
Values to use for the yticks.

**xlim** : 2-tuple/list  
Set the x limits of the current axes.

**ylim** : 2-tuple/list  
Set the y limits of the current axes.

**xlabel** : label, optional  
Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.

.. versionchanged:: 2.0.0  
Now applicable to histograms.

**ylabel** : label, optional  
Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.

.. versionchanged:: 2.0.0  
Now applicable to histograms.

**rot** : float, default None  
Rotation for ticks (xticks for vertical, yticks for horizontal plots).

**fontsize** : float, default None  
Font size for xticks and yticks.

**colormap** : str or matplotlib colormap object, default None  
Colormap to select colors from. If string, load colormap with that name from matplotlib.

**colorbar** : bool, optional  
If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).

**position** : float  
Specify relative alignments for bar plot layout.  
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).

**table** : bool, Series or DataFrame, default False  
If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout.  
If a Series or DataFrame is passed, use passed data to draw a table.

**yerr** : DataFrame, Series, array-like, dict and str  
See :ref:`Plotting with Error Bars <visualization.errorbars>` for detail.

**xerr** : DataFrame, Series, array-like, dict and str  
Equivalent to yerr.

**stacked** : bool, default False in line and bar plots, and True in area plot  
If True, create stacked plot.

**secondary\_y** : bool or sequence, default False  
Whether to plot on the secondary y-axis if a list/tuple, which columns to plot on secondary y-axis.

**mark\_right** : bool, default True  
When using a secondary\_y axis, automatically mark the column labels with "(right)" in the legend.

**include\_bool** : bool, default is False  
If True, boolean values can be plotted.

**backend** : str, default None



Backend to use instead of the backend specified in the option  
`plotting.backend`. For instance, 'matplotlib'. Alternatively, to  
specify the `plotting.backend` for the whole session, set  
`pd.options.plotting.backend`.

**\*\*kwargs**

Options to pass to matplotlib plotting method.

**Returns**

-----

:class: `matplotlib.axes.Axes` or numpy.ndarray of them

If the backend is not the default matplotlib one, the return value  
will be the object returned by the backend.

**See Also**

-----

matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.

DataFrame.hist : Make a histogram.

DataFrame.boxplot : Make a box plot.

DataFrame.plot.scatter : Make a scatter plot with varying marker  
point size and color.

DataFrame.plot.hexbin : Make a hexagonal binning plot of  
two variables.

DataFrame.plot.kde : Make Kernel Density Estimate plot using  
Gaussian kernels.

DataFrame.plot.area : Make a stacked area plot.

DataFrame.plot.bar : Make a bar plot.

DataFrame.plot.barh : Make a horizontal bar plot.

**Notes**

-----

- See matplotlib documentation online for more on this subject

- If `kind` = 'bar' or 'barh', you can specify relative alignments  
for bar plot layout by `position` keyword.

From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5  
(center)

**Examples**

-----

For Series:

```
.. plot::
 :context: close-figs

 >>> ser = pd.Series([1, 2, 3, 3])
 >>> plot = ser.plot(kind="hist", title="My plot")
```

For DataFrame:

```
.. plot::
 :context: close-figs

 >>> df = pd.DataFrame(
 ... {
 ... "length": [1.5, 0.5, 1.2, 0.9, 3],
 ... "width": [0.7, 0.2, 0.15, 0.2, 1.1],
 ... },
 ... index=["pig", "rabbit", "duck", "chicken", "horse"],
 ...)
 >>> plot = df.plot(title="DataFrame Plot")
```

For SeriesGroupBy:

```
.. plot::
 :context: close-figs

 >>> lst = [-1, -2, -3, 1, 2, 3]
 >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
 >>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")
```

For DataFrameGroupBy:

```
.. plot::
 :context: close-figs

 >>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})
 >>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")
```

```
sparse = <class 'pandas.core.arrays.sparse.accessor.SparseFrameAccesso...
DataFrame accessor for sparse data.
```

It allows users to interact with a `DataFrame` that contains sparse data types (`SparseDtype`). It provides methods and attributes to efficiently work with sparse storage, reducing memory usage while maintaining compatibility with standard pandas operations.

Parameters

-----  
data : scipy.sparse.spmatrix  
Must be convertible to csc format.

See Also

-----  
DataFrame.sparse.density : Ratio of non-sparse points to total (dense) data points.

Examples

-----  
>>> df = pd.DataFrame({"a": [1, 2, 0, 0], "b": [3, 0, 0, 4]}, dtype="Sparse[int]")  
>>> df.sparse.density  
np.float64(0.5)

-----  
Methods inherited from pandas.core.generic.NDFrame:

\_\_abs\_\_(self) -> Self

\_\_array\_\_(self, dtype: npt.DTypeLike | None = None, copy: bool | None = None) -> np.ndarray

\_\_array\_ufunc\_\_(self, ufunc: np.ufunc, method: str, \*inputs: Any, \*\*kwargs: Any)

\_\_bool\_\_(self) -> NoReturn

\_\_contains\_\_(self, key) -> bool  
True if the key is in the info axis

\_\_copy\_\_(self) -> Self

\_\_deepcopy\_\_(self, memo=None) -> Self  
Parameters

-----  
memo, default None  
Standard signature. Unused

\_\_delitem\_\_(self, key) -> None  
Delete item

\_\_finalize\_\_(self, other, method: str | None = None, \*\*kwargs) -> Self  
Propagate metadata from other to self.

This is the default implementation. Subclasses may override this method to implement their own metadata handling.

Parameters

-----  
other : the object from which to get the attributes that we are going to propagate. If ``other`` has an ``input\_objs`` attribute, then this attribute must contain an iterable of objects, each with an ``attrs`` attribute.

method : str, optional  
A passed method name providing context on where ``\_\_finalize\_\_`` was called.

.. warning::

The value passed as `method` are not currently considered stable across pandas releases.

Notes

-----  
In case ``other`` has an ``input\_objs`` attribute, this method only propagates its metadata if each object in ``input\_objs`` has the exact same metadata as the others.

```

__getattr__(self, name: str)
 After regular attribute access, try looking up the name
 This allows simpler access to columns for interactive use.

__getstate__(self) -> dict[str, Any]
 Helper for pickle.

__iadd__(self, other) -> Self

__iand__(self, other) -> Self

__ifloordiv__(self, other) -> Self

__imod__(self, other) -> Self

__imul__(self, other) -> Self

__invert__(self) -> Self

__ior__(self, other) -> Self

__ipow__(self, other) -> Self

__isub__(self, other) -> Self

__iter__(self) -> Iterator
 Iterate over info axis.

 Returns

 iterator
 Info axis as iterator.

 See Also

 DataFrame.items : Iterate over (column name, Series) pairs.
 DataFrame.itertuples : Iterate over DataFrame rows as namedtuples.

 Examples

 >>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
 >>> for x in df:
 ... print(x)
 A
 B

__itruediv__(self, other) -> Self

__ixor__(self, other) -> Self

__neg__(self) -> Self

__pos__(self) -> Self

__round__(self, decimals: int = 0) -> Self

__setattr__(self, name: str, value) -> None
 After regular attribute access, try setting the name
 This allows simpler access to columns for interactive use.

__setstate__(self, state) -> None

abs(self) -> Self
 Return a Series/DataFrame with absolute numeric value of each element.

 This function only applies to elements that are all numeric.

 Returns

 abs
 Series/DataFrame containing the absolute value of each element.

 See Also

 numpy.absolute : Calculate the absolute value element-wise.

 Notes

```

-----  
For ``complex`` inputs, ``1.2 + 1j``, the absolute value is  
:math:`\sqrt{a^2 + b^2}``.

#### Examples

-----  
Absolute numeric values in a Series.

```
>>> s = pd.Series([-1.10, 2, -3.33, 4])
>>> s.abs()
0 1.10
1 2.00
2 3.33
3 4.00
dtype: float64
```

Absolute numeric values in a Series with complex numbers.

```
>>> s = pd.Series([1.2 + 1j])
>>> s.abs()
0 1.56205
dtype: float64
```

Absolute numeric values in a Series with a Timedelta element.

```
>>> s = pd.Series([pd.Timedelta("1 days")])
>>> s.abs()
0 1 days
dtype: timedelta64[us]
```

Select rows with data closest to certain value using argsort (from  
`StackOverflow` <<https://stackoverflow.com/a/17758115>>`\_`).

```
>>> df = pd.DataFrame(
... {"a": [4, 5, 6, 7], "b": [10, 20, 30, 40], "c": [100, 50, -30, -50]}
...)
>>> df
 a b c
0 4 10 100
1 5 20 50
2 6 30 -30
3 7 40 -50
>>> df.loc[(df.c - 43).abs().argsort()]
 a b c
1 5 20 50
0 4 10 100
2 6 30 -30
3 7 40 -50
```

`add_prefix(self, prefix: str, axis: Axis | None = None) -> Self`  
Prefix labels with string `prefix`.

For Series, the row labels are prefixed.  
For DataFrame, the column labels are prefixed.

#### Parameters

-----  
prefix : str  
The string to add before each label.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
Axis to add prefix on  
  
.. versionadded:: 2.0.0

#### Returns

-----  
Series or DataFrame  
New Series or DataFrame with updated labels.

#### See Also

-----  
Series.add\_suffix: Suffix row labels with string `suffix`.  
DataFrame.add\_suffix: Suffix column labels with string `suffix`.

#### Examples

-----  
>>> s = pd.Series([1, 2, 3, 4])

```

>>> s
0 1
1 2
2 3
3 4
dtype: int64

>>> s.add_prefix("item_")
item_0 1
item_1 2
item_2 3
item_3 4
dtype: int64

>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})
>>> df
 A B
0 1 3
1 2 4
2 3 5
3 4 6

>>> df.add_prefix("col_")
 col_A col_B
0 1 3
1 2 4
2 3 5
3 4 6

```

`add_suffix(self, suffix: str, axis: Axis | None = None) -> Self`  
 Suffix labels with string `suffix`.

For Series, the row labels are suffixed.  
 For DataFrame, the column labels are suffixed.

Parameters

-----

`suffix` : str

The string to add after each label.

`axis` : {0 or 'index', 1 or 'columns', None}, default None

Axis to add suffix on

.. versionadded:: 2.0.0

Returns

-----

Series or DataFrame

New Series or DataFrame with updated labels.

See Also

-----

`Series.add_prefix`: Prefix row labels with string `prefix`.

`DataFrame.add_prefix`: Prefix column labels with string `prefix`.

Examples

-----

```

>>> s = pd.Series([1, 2, 3, 4])

```

```

>>> s
0 1
1 2
2 3
3 4
dtype: int64

```

```

>>> s.add_suffix("_item")

```

```

0_item 1
1_item 2
2_item 3
3_item 4
dtype: int64

```

```

>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})

```

```

>>> df
 A B
0 1 3
1 2 4
2 3 5

```

```
3 4 6
```

```
>>> df.add_suffix("_col")
```

```
 A_col B_col
0 1 3
1 2 4
2 3 5
3 4 6
```

```
align(
 self,
 other: NDFrameT,
 join: AlignJoin = 'outer',
 axis: Axis | None = None,
 level: Level | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 fill_value: Hashable | None = None
) -> tuple[Self, NDFrameT]
```

Align two objects on their axes with the specified join method.

Join method is specified for each axis Index.

#### Parameters

-----

other : DataFrame or Series

The object to align with.

join : {'outer', 'inner', 'left', 'right'}, default 'outer'

Type of alignment to be performed.

\* left: use only keys from left frame, preserve key order.

\* right: use only keys from right frame, preserve key order.

\* outer: use union of keys from both frames, sort keys lexicographically.

\* inner: use intersection of keys from both frames,  
preserve the order of the left keys.

axis : allowed axis of the other object, default None

Align on index (0), columns (1), or both (None).

level : int or level name, default None

Broadcast across a level, matching Index values on the  
passed MultiIndex level.

copy : bool, default False

This keyword is now ignored; changing its value will have no  
impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since  
pandas 3.0, this method always returns a new object using a lazy  
copy mechanism that defers copies until necessary  
(Copy-on-Write). See the `user guide on Copy-on-Write`  
<[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_  
for more details.

fill\_value : scalar, default np.nan

Value to use for missing values. Defaults to NaN, but can be any  
"compatible" value.

#### Returns

-----

tuple of (Series/DataFrame, type of other)

Aligned objects.

#### See Also

-----

Series.align : Align two objects on their axes with specified join method.

DataFrame.align : Align two objects on their axes with specified join method.

#### Examples

-----

```
>>> df = pd.DataFrame(
... [[1, 2, 3, 4], [6, 7, 8, 9]], columns=["D", "B", "E", "A"], index=[1, 2]
...)
>>> other = pd.DataFrame(
... [[10, 20, 30, 40], [60, 70, 80, 90], [600, 700, 800, 900]],
... columns=["A", "B", "C", "D"],
... index=[2, 3, 4],
...)
```

```
>>> df
 D B E A
1 1 2 3 4
2 6 7 8 9
>>> other
 A B C D
2 10 20 30 40
3 60 70 80 90
4 600 700 800 900
```

Align on columns:

```
>>> left, right = df.align(other, join="outer", axis=1)
>>> left
 A B C D E
1 4 2 NaN 1 3
2 9 7 NaN 6 8
>>> right
 A B C D E
2 10 20 30 40 NaN
3 60 70 80 90 NaN
4 600 700 800 900 NaN
```

We can also align on the index:

```
>>> left, right = df.align(other, join="outer", axis=0)
>>> left
 D B E A
1 1.0 2.0 3.0 4.0
2 6.0 7.0 8.0 9.0
3 NaN NaN NaN NaN
4 NaN NaN NaN NaN
>>> right
 A B C D
1 NaN NaN NaN NaN
2 10.0 20.0 30.0 40.0
3 60.0 70.0 80.0 90.0
4 600.0 700.0 800.0 900.0
```

Finally, the default `axis=None` will align on both index and columns:

```
>>> left, right = df.align(other, join="outer", axis=None)
>>> left
 A B C D E
1 4.0 2.0 NaN 1.0 3.0
2 9.0 7.0 NaN 6.0 8.0
3 NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN
>>> right
 A B C D E
1 NaN NaN NaN NaN NaN
2 10.0 20.0 30.0 40.0 NaN
3 60.0 70.0 80.0 90.0 NaN
4 600.0 700.0 800.0 900.0 NaN
```

```
asfreq(
 self,
 freq: Frequency,
 method: FillnaOptions | None = None,
 how: Literal['start', 'end'] | None = None,
 normalize: bool = False,
 fill_value: Hashable | None = None
) -> Self
Convert time series to specified frequency.
```

Returns the original data conformed to a new index with the specified frequency.

If the index of this Series/DataFrame is a `:class:`~pandas.PeriodIndex``, the new index is the result of transforming the original index with `:meth:`PeriodIndex.asfreq`` (`<pandas.PeriodIndex.asfreq>` (so the original index will map one-to-one to the new index)).

Otherwise, the new index will be equivalent to ``pd.date_range(start, end, freq=freq)`` where ``start`` and ``end`` are, respectively, the min and max entries in the original index (see `:func:`pandas.date_range``). The values corresponding to any timesteps in the new index which were not present

in the original index will be null (``NaN``), unless a method for filling such unknowns is provided (see the ``method`` parameter below).

The `:meth:`resample`` method is more appropriate if an operation on each group of timesteps (such as an aggregate) is necessary to represent the data at the new frequency.

#### Parameters

-----  
freq : DateOffset or str  
Frequency DateOffset or string.  
method : {{'backfill'/'bfill', 'pad'/'ffill'}}, default None  
Method to use for filling holes in reindexed Series (note this does not fill NaNs that already were present):  
  
\* 'pad' / 'ffill': propagate last valid observation forward to next valid based on the order of the index  
\* 'backfill' / 'bfill': use NEXT valid observation to fill.  
how : {{'start', 'end'}}, default end  
For PeriodIndex only (see PeriodIndex.asfreq).  
normalize : bool, default False  
Whether to reset output index to midnight.  
fill\_value : scalar, optional  
Value to use for missing values, applied during upsampling (note this does not fill NaNs that already were present).

#### Returns

-----  
Series/DataFrame  
Series/DataFrame object reindexed to the specified frequency.

#### See Also

-----  
`reindex` : Conform DataFrame to new index with optional filling logic.

#### Notes

-----  
To learn more about the frequency strings, please see `:ref:`this link<timeseries.offset_aliases>``.

#### Examples

-----  
Start by creating a series with 4 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=4, freq="min")
>>> series = pd.Series([0.0, None, 2.0, 3.0], index=index)
>>> df = pd.DataFrame({"s": series})
>>> df
```

```
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:01:00 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:03:00 3.0
```

Upsample the series into 30 second bins.

```
>>> df.asfreq(freq="30s")
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 NaN
2000-01-01 00:03:00 3.0
```

Upsample again, providing a ``fill value``.

```
>>> df.asfreq(freq="30s", fill_value=9.0)
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 9.0
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 9.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 9.0
2000-01-01 00:03:00 3.0
```



Upsample again, providing a ``method``.

```
>>> df.upsample(freq="30s", method="bfill")
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 2.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 3.0
2000-01-01 00:03:00 3.0
```

`asof(self, where, subset=None)`

Return the last row(s) without any NaNs before `where`.

The last row (for each element in `where`, if list) without any NaN is taken.

In case of a :class:`~pandas.DataFrame`, the last row without NaN considering only the subset of columns (if not `None`)

If there is no good value, NaN is returned for a Series or a Series of NaN values for a DataFrame

Parameters

-----

`where` : date or array-like of dates

Date(s) before which the last row(s) are returned.

`subset` : str or array-like of str, default `None`

For DataFrame, if not `None`, only use these columns to check for NaNs.

Returns

-----

scalar, Series, or DataFrame

The return can be:

- \* scalar : when `self` is a Series and `where` is a scalar
- \* Series: when `self` is a Series and `where` is an array-like, or when `self` is a DataFrame and `where` is a scalar
- \* DataFrame : when `self` is a DataFrame and `where` is an array-like

See Also

-----

`merge_asof` : Perform an asof merge. Similar to left join.

Notes

-----

Dates are assumed to be sorted. Raises if this is not the case.

Examples

-----

A Series and a scalar `where`.

```
>>> s = pd.Series([1, 2, np.nan, 4], index=[10, 20, 30, 40])
>>> s
10 1.0
20 2.0
30 NaN
40 4.0
dtype: float64
```

```
>>> s.asof(20)
np.float64(2.0)
```

For a sequence `where`, a Series is returned. The first value is NaN, because the first element of `where` is before the first index value.

```
>>> s.asof([5, 20])
5 NaN
20 2.0
dtype: float64
```

Missing values are not considered. The following is ``2.0``, not

NaN, even though NaN is at the index location for ``30``.

```
>>> s.asof(30)
np.float64(2.0)
```

Take all columns into consideration

```
>>> df = pd.DataFrame(
... {
... "a": [10.0, 20.0, 30.0, 40.0, 50.0],
... "b": [None, None, None, None, 500],
... },
... index=pd.DatetimeIndex(
... [
... "2018-02-27 09:01:00",
... "2018-02-27 09:02:00",
... "2018-02-27 09:03:00",
... "2018-02-27 09:04:00",
... "2018-02-27 09:05:00",
...]
...),
...)
>>> df.asof(pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]))
 a b
2018-02-27 09:03:30 NaN NaN
2018-02-27 09:04:30 NaN NaN
```

Take a single column into consideration

```
>>> df.asof(
... pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]),
... subset=["a"],
...)
 a b
2018-02-27 09:03:30 30.0 NaN
2018-02-27 09:04:30 40.0 NaN
```

```
astype(
 self,
 dtype,
 copy: bool | lib.NoDefault = <no_default>,
 errors: IgnoreRaise = 'raise'
) -> Self
Cast a pandas object to a specified dtype ``dtype``.
```

This method allows the conversion of the data types of pandas objects, including DataFrames and Series, to the specified dtype. It supports casting entire objects to a single data type or applying different data types to individual columns using a mapping.

Parameters

**dtype** : str, data type, Series or Mapping of column name -> data type  
Use a str, numpy.dtype, pandas.ExtensionDtype or Python type to cast entire pandas object to the same type. Alternatively, use a mapping, e.g. {col: dtype, ...}, where col is a column label and dtype is a numpy.dtype or Python type to cast one or more of the DataFrame's columns to column-specific types.

**copy** : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the [user guide on Copy-on-Write](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

**errors** : {'raise', 'ignore'}, default 'raise'  
Control raising of exceptions on invalid data for provided dtype.

- ``raise`` : allow exceptions to be raised
- ``ignore`` : suppress exceptions. On error return original object.

## Returns

-----  
same type as caller  
The pandas object casted to the specified ``dtype``.

## See Also

-----  
to\_datetime : Convert argument to datetime.  
to\_timedelta : Convert argument to timedelta.  
to\_numeric : Convert argument to a numeric type.  
numpy.ndarray.astype : Cast a numpy array to a specified type.

## Notes

-----  
.. versionchanged:: 2.0.0  
  
Using ``astype`` to convert from timezone-naive dtype to  
timezone-aware dtype will raise an exception.  
Use :meth:`Series.dt.tz\_localize` instead.

## Examples

-----  
Create a DataFrame:

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df.dtypes
col1 int64
col2 int64
dtype: object
```

Cast all columns to int32:

```
>>> df.astype("int32").dtypes
col1 int32
col2 int32
dtype: object
```

Cast col1 to int32 using a dictionary:

```
>>> df.astype({"col1": "int32"}).dtypes
col1 int32
col2 int64
dtype: object
```

Create a series:

```
>>> ser = pd.Series([1, 2], dtype="int32")
>>> ser
0 1
1 2
dtype: int32
>>> ser.astype("int64")
0 1
1 2
dtype: int64
```

Convert to categorical type:

```
>>> ser.astype("category")
0 1
1 2
dtype: category
Categories (2, int32): [1, 2]
```

Convert to ordered categorical type with custom ordering:

```
>>> from pandas.api.types import CategoricalDtype
>>> cat_dtype = CategoricalDtype(categories=[2, 1], ordered=True)
>>> ser.astype(cat_dtype)
0 1
1 2
dtype: category
Categories (2, int64): [2 < 1]
```

Create a series of dates:

```

>>> ser_date = pd.Series(pd.date_range("20200101", periods=3))
>>> ser_date
0 2020-01-01
1 2020-01-02
2 2020-01-03
dtype: datetime64[us]

```

**at\_time(self, time, asof: bool = False, axis: Axis | None = None) -> Self**  
Select values at particular time of day (e.g., 9:30AM).

**Parameters**  
-----  
time : datetime.time or str  
The values to select.  
asof : bool, default False  
This parameter is currently not supported.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
For `Series` this parameter is unused and defaults to 0.

**Returns**  
-----  
Series or DataFrame  
The values with the specified time.

**Raises**  
-----  
TypeError  
If the index is not a :class:`DatetimeIndex`

**See Also**  
-----  
between\_time : Select values between particular times of the day.  
first : Select initial periods of time series based on a date offset.  
last : Select final periods of time series based on a date offset.  
DatetimeIndex.indexer\_at\_time : Get just the index locations for values at particular time of the day.

**Examples**  
-----  
>>> i = pd.date\_range("2018-04-09", periods=4, freq="12h")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts

	A
2018-04-09 00:00:00	1
2018-04-09 12:00:00	2
2018-04-10 00:00:00	3
2018-04-10 12:00:00	4

```

>>> ts.at_time("12:00")
```

	A
2018-04-09 12:00:00	2
2018-04-10 12:00:00	4

```

between_time(
 self,
 start_time,
 end_time,
 inclusive: IntervalClosedType = 'both',
 axis: Axis | None = None
) -> Self
Select values between particular times of the day (e.g., 9:00-9:30 AM).

By setting ``start_time`` to be later than ``end_time``,
you can get the times that are *not* between the two times.

Parameters

start_time : datetime.time or str
 Initial time as a time filter limit.
end_time : datetime.time or str
 End time as a time filter limit.
inclusive : {"both", "neither", "left", "right"}, default "both"
 Include boundaries; whether to set each bound as closed or open.
axis : {0 or 'index', 1 or 'columns'}, default 0
 Determine range time on index or columns value.
 For `Series` this parameter is unused and defaults to 0.

```

## Returns

-----

Series or DataFrame

Data from the original object filtered to the specified dates range.

## Raises

-----

TypeError

If the index is not a :class:`DatetimeIndex`

## See Also

-----

`at_time` : Select values at a particular time of the day.

`first` : Select initial periods of time series based on a date offset.

`last` : Select final periods of time series based on a date offset.

`DatetimeIndex.indexer_between_time` : Get just the index locations for values between particular times of the day.

## Examples

-----

```
>>> i = pd.date_range("2018-04-09", periods=4, freq="1D20min")
```

```
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
```

```
>>> ts
```

```
 A
2018-04-09 00:00:00 1
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3
2018-04-12 01:00:00 4
```

```
>>> ts.between_time("0:15", "0:45")
```

```
 A
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3
```

You get the times that are *\*not\** between two times by setting  
``start\_time`` later than ``end\_time``:

```
>>> ts.between_time("0:45", "0:15")
```

```
 A
2018-04-09 00:00:00 1
2018-04-12 01:00:00 4
```

```
bfill(
 self,
 *,
 axis: None | Axis = None,
 inplace: bool = False,
 limit: None | int = None,
 limit_area: Literal['inside', 'outside'] | None = None
) -> Self
```

Fill NA/Nan values by using the next valid observation to fill the gap.

This method fills missing values in a backward direction along the specified axis, propagating non-null values from later positions to earlier positions containing NaN.

## Parameters

-----

`axis` : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.

`inplace` : bool, default False

If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

`limit` : int, default None

If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

`limit_area` : {'None', 'inside', 'outside'}, default None

If limit is specified, consecutive NaNs will be filled with this restriction.

\* ``None``: No fill restriction.

```

* 'inside': Only fill NaNs surrounded by valid values
(interpolate).
* 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

```

#### Returns

```

Series/DataFrame
 Object with missing values filled.

```

#### See Also

```

DataFrame.ffill : Fill NA/NAN values by propagating the last valid
observation to next valid.

```

#### Examples

```

For Series:

```

```

>>> s = pd.Series([1, None, None, 2])
>>> s.bfill()
0 1.0
1 2.0
2 2.0
3 2.0
dtype: float64
>>> s.bfill(limit=1)
0 1.0
1 NaN
2 2.0
3 2.0
dtype: float64

```

```

With DataFrame:

```

```

>>> df = pd.DataFrame({"A": [1, None, None, 4], "B": [None, 5, None, 7]})
>>> df
 A B
0 1.0 NaN
1 NaN 5.0
2 NaN NaN
3 4.0 7.0
>>> df.bfill()
 A B
0 1.0 5.0
1 4.0 5.0
2 4.0 7.0
3 4.0 7.0
>>> df.bfill(limit=1)
 A B
0 1.0 5.0
1 NaN 5.0
2 4.0 7.0
3 4.0 7.0

```

```

clip(
 self,
 lower=None,
 upper=None,
 *,
 axis: Axis | None = None,
 inplace: bool = False,
 **kwargs
) -> Self
 Trim values at input threshold(s).

```

Assigns values outside boundary to boundary values. Thresholds can be singular values or array like, and in the latter case the clipping is performed element-wise in the specified axis.

#### Parameters

```

lower : float or array-like, default None
 Minimum threshold value. All values below this
 threshold will be set to it. A missing
 threshold (e.g `NA`) will not clip the value.

```

upper : float or array-like, default None  
 Maximum threshold value. All values above this threshold will be set to it. A missing threshold (e.g. 'NA') will not clip the value.  
 axis : {{0 or 'index', 1 or 'columns', None}}, default None  
 Align object with lower and upper along the given axis.  
 For 'Series' this parameter is unused and defaults to 'None'.  
 inplace : bool, default False  
 Whether to perform the operation in place on the data.  
 \*\*kwargs  
 Additional keywords have no effect but might be accepted for compatibility with numpy.

#### Returns

-----  
 Series or DataFrame  
 Same type as calling object with the values outside the clip boundaries replaced.

#### See Also

-----  
 Series.clip : Trim values at input threshold in series.  
 DataFrame.clip : Trim values at input threshold in DataFrame.  
 numpy.clip : Clip (limit) the values in an array.

#### Examples

-----  
 >>> data = {"col\_0": [9, -3, 0, -1, 5], "col\_1": [-2, -7, 6, 8, -5]}  
 >>> df = pd.DataFrame(data)  
 >>> df

	col_0	col_1
0	9	-2
1	-3	-7
2	0	6
3	-1	8
4	5	-5

Clips per column using lower and upper thresholds:

>>> df.clip(-4, 6)

	col_0	col_1
0	6	-2
1	-3	-4
2	0	6
3	-1	6
4	5	-4

Clips using specific lower and upper thresholds per column:

>>> df.clip([-2, -1], [4, 5])

	col_0	col_1
0	4	-1
1	-2	-1
2	0	5
3	-1	5
4	4	-1

Clips using specific lower and upper thresholds per column element:

>>> t = pd.Series([2, -4, -1, 6, 3])  
 >>> t

0	2
1	-4
2	-1
3	6
4	3

dtype: int64

>>> df.clip(t, t + 4, axis=0)

	col_0	col_1
0	6	2
1	-3	-4
2	0	3
3	6	8
4	5	3

Clips using specific lower threshold per column element, with missing values:

```
>>> t = pd.Series([2, -4, np.nan, 6, 3])
>>> t
0 2.0
1 -4.0
2 NaN
3 6.0
4 3.0
dtype: float64
```

```
>>> df.clip(t, axis=0)
col_0 col_1
0 9.0 2.0
1 -3.0 -4.0
2 0.0 6.0
3 6.0 8.0
4 5.0 3.0
```

```
convert_dtypes(
 self,
 infer_objects: bool = True,
 convert_string: bool = True,
 convert_integer: bool = True,
 convert_boolean: bool = True,
 convert_floating: bool = True,
 dtype_backend: DtypeBackend = 'numpy_nullable'
) -> Self
 Convert columns from numpy dtypes to the best dtypes that support ``pd.NA``.
```

#### Parameters

```
infer_objects : bool, default True
 Whether object dtypes should be converted to the best possible types.
convert_string : bool, default True
 Whether object dtypes should be converted to ``StringDtype()``.
convert_integer : bool, default True
 Whether, if possible, conversion can be done to integer extension types.
convert_boolean : bool, defaults True
 Whether object dtypes should be converted to ``BooleanDtypes()``.
convert_floating : bool, defaults True
 Whether, if possible, conversion can be done to floating extension types.
 If ``convert_integer`` is also True, preference will be give to integer
 dtypes if the floats can be faithfully casted to integers.
dtype_backend : {'numpy_nullable', 'pyarrow'}, default 'numpy_nullable'
 Back-end data type applied to the resultant :class:`DataFrame` or
 :class:`Series` (still experimental). Behaviour is as follows:

 * ``"numpy_nullable"``: returns nullable-dtype-backed
 :class:`DataFrame` or :class:`Series`.
 * ``"pyarrow"``: returns pyarrow-backed nullable :class:`ArrowDtype`
 :class:`DataFrame` or :class:`Series`.

.. versionadded:: 2.0
```

#### Returns

```
Series or DataFrame
 Copy of input object with new dtype.
```

#### See Also

```
infer_objects : Infer dtypes of objects.
to_datetime : Convert argument to datetime.
to_timedelta : Convert argument to timedelta.
to_numeric : Convert argument to a numeric type.
```

#### Notes

```
By default, ``convert_dtypes`` will attempt to convert a Series (or each
Series in a DataFrame) to dtypes that support ``pd.NA``. By using the options
``convert_string``, ``convert_integer``, ``convert_boolean`` and
``convert_floating``, it is possible to turn off individual conversions
to ``StringDtype``, the integer extension types, ``BooleanDtype``
or floating extension types, respectively.
```

```
For object-dtyped columns, if ``infer_objects`` is ``True``, use the inference
rules as during normal Series/DataFrame construction. Then, if possible,
```



convert to ``StringDtype``, ``BooleanDtype`` or an appropriate integer or floating extension type, otherwise leave as ``object``.

If the dtype is integer, convert to an appropriate integer extension type.

If the dtype is numeric, and consists of all integers, convert to an appropriate integer extension type. Otherwise, convert to an appropriate floating extension type.

In the future, as new dtypes are added that support ``pd.NA``, the results of this method will change to support those new dtypes.

#### Examples

```
>>> df = pd.DataFrame(
... {
... "a": pd.Series([1, 2, 3], dtype=np.dtype("int32")),
... "b": pd.Series(["x", "y", "z"], dtype=np.dtype("O")),
... "c": pd.Series([True, False, np.nan], dtype=np.dtype("O")),
... "d": pd.Series(["h", "i", np.nan], dtype=np.dtype("O")),
... "e": pd.Series([10, np.nan, 20], dtype=np.dtype("float")),
... "f": pd.Series([np.nan, 100.5, 200], dtype=np.dtype("float")),
... }
...)
```

Start with a DataFrame with default dtypes.

```
>>> df
 a b c d e f
0 1 x True h 10.0 NaN
1 2 y False i NaN 100.5
2 3 z NaN NaN 20.0 200.0
```

```
>>> df.dtypes
a int32
b object
c object
d object
e float64
f float64
dtype: object
```

Convert the DataFrame to use best possible dtypes.

```
>>> dfn = df.convert_dtypes()
>>> dfn
 a b c d e f
0 1 x True h 10 <NA>
1 2 y False i <NA> 100.5
2 3 z <NA> <NA> 20 200.0
```

```
>>> dfn.dtypes
a Int32
b string
c boolean
d string
e Int64
f Float64
dtype: object
```

Start with a Series of strings and missing data represented by ``np.nan``.

```
>>> s = pd.Series(["a", "b", np.nan])
>>> s
0 a
1 b
2 NaN
dtype: str
```

Obtain a Series with dtype ``StringDtype``.

```
>>> s.convert_dtypes()
0 a
1 b
2 <NA>
dtype: string
```

```

copy(self, deep: bool = True) -> Self
 Make a copy of this object's indices and data.

When ``deep=True`` (default), a new object will be created with a
copy of the calling object's data and indices. Modifications to
the data or indices of the copy will not be reflected in the
original object (see notes below).

When ``deep=False``, a new object will be created without copying
the calling object's data or index (only references to the data
and index are copied). With Copy-on-Write, changes to the original
will *not* be reflected in the shallow copy (and vice versa). The
shallow copy uses a lazy (deferred) copy mechanism that copies the
data only when any changes to the original or shallow copy are made,
ensuring memory efficiency while maintaining data integrity.

.. note::
 In pandas versions prior to 3.0, the default behavior without
 Copy-on-Write was different: changes to the original *were* reflected
 in the shallow copy (and vice versa). See the :ref:`Copy-on-Write
 user guide <copy_on_write>` for more information.

Parameters

deep : bool, default True
 Make a deep copy, including a copy of the data and the indices.
 With ``deep=False`` neither the indices nor the data are copied.

Returns

Series or DataFrame
 Object type matches caller.

See Also

copy.copy : Return a shallow copy of an object.
copy.deepcopy : Return a deep copy of an object.

Notes

When ``deep=True``, data is copied but actual Python objects
will not be copied recursively, only the reference to the object.
This is in contrast to ``copy.deepcopy`` in the Standard Library,
which recursively copies object data (see examples below).

While ``Index`` objects are copied when ``deep=True``, the underlying
numpy array is not copied for performance reasons. Since ``Index`` is
immutable, the underlying data can be safely shared and a copy
is not needed.

Since pandas is not thread safe, see the
:ref:`gotchas <gotchas.thread-safety>` when copying in a threading
environment.

Copy-on-Write protects shallow copies against accidental modifications.
This means that any changes to the copied data would make a new copy
of the data upon write (and vice versa). Changes made to either the
original or copied variable would not be reflected in the counterpart.
See :ref:`Copy-on-Write <copy_on_write>` for more information.

Examples

>>> s = pd.Series([1, 2], index=["a", "b"])
>>> s
a 1
b 2
dtype: int64

>>> s_copy = s.copy(deep=True)
>>> s_copy
a 1
b 2
dtype: int64

Due to Copy-on-Write, shallow copies still protect data modifications.
Note shallow does not get modified below.

```

```
>>> s = pd.Series([1, 2], index=["a", "b"])
>>> shallow = s.copy(deep=False)
>>> s.iloc[1] = 200
>>> shallow
a 1
b 2
dtype: int64
```

When the data has object dtype, even a deep copy does not copy the underlying Python objects. Updating a nested data object will be reflected in the deep copy.

```
>>> s = pd.Series([[1, 2], [3, 4]])
>>> deep = s.copy()
>>> s[0][0] = 10
>>> s
0 [10, 2]
1 [3, 4]
dtype: object
>>> deep
0 [10, 2]
1 [3, 4]
dtype: object
```

`describe(self, percentiles=None, include=None, exclude=None) -> Self`  
Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding ``NaN`` values.

Analyzes both numeric and object series, as well as ``DataFrame`` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

#### Parameters

- 
- `percentiles` : list-like of numbers, optional  
The percentiles to include in the output. All should fall between 0 and 1. The default, ``None``, will automatically return the 25th, 50th, and 75th percentiles.
  - `include` : 'all', list-like of dtypes or None (default), optional  
A white list of data types to include in the result. Ignored for ``Series``. Here are the options:
    - 'all' : All columns of the input will be included in the output.
    - A list-like of dtypes : Limits the results to the provided data types.  
To limit the result to numeric types submit ``numpy.number``. To limit it instead to object columns submit the ``numpy.object`` data type. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(include=['O'])``). To select pandas categorical columns, use ``category``.
    - None (default) : The result will include all numeric columns.
  - `exclude` : list-like of dtypes or None (default), optional  
A black list of data types to omit from the result. Ignored for ``Series``. Here are the options:
    - A list-like of dtypes : Excludes the provided data types from the result. To exclude numeric types submit ``numpy.number``. To exclude object columns submit the data type ``numpy.object``. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(exclude=['O'])``). To exclude pandas categorical columns, use ``category``.
    - None (default) : The result will exclude nothing.

#### Returns

-----  
Series or DataFrame  
Summary statistics of the Series or DataFrame provided.

#### See Also

-----  
`DataFrame.count`: Count number of non-NA/null observations.  
`DataFrame.max`: Maximum of the values in the object.

DataFrame.min: Minimum of the values in the object.  
DataFrame.mean: Mean of the values.  
DataFrame.std: Standard deviation of the observations.  
DataFrame.select\_dtypes: Subset of a DataFrame including/excluding columns based on their dtype.

#### Notes

For numeric data, the result's index will include ``count``, ``mean``, ``std``, ``min``, ``max`` as well as lower, ``50`` and upper percentiles. By default the lower percentile is ``25`` and the upper percentile is ``75``. The ``50`` percentile is the same as the median.

For object data (e.g. strings), the result's index will include ``count``, ``unique``, ``top``, and ``freq``. The ``top`` is the most common value. The ``freq`` is the most common value's frequency.

If multiple object values have the highest count, then the ``count`` and ``top`` results will be arbitrarily chosen from among those with the highest count.

For mixed data types provided via a ``DataFrame``, the default is to return only an analysis of numeric columns. If the DataFrame consists only of object and categorical data without any numeric columns, the default is to return an analysis of both the object and categorical columns. If ``include='all'`` is provided as an option, the result will include a union of attributes of each type.

The ``include`` and ``exclude`` parameters can be used to limit which columns in a ``DataFrame`` are analyzed for the output. The parameters are ignored when analyzing a ``Series``.

#### Examples

Describing a numeric ``Series``.

```
>>> s = pd.Series([1, 2, 3])
>>> s.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
dtype: float64
```

Describing a categorical ``Series``.

```
>>> s = pd.Series(["a", "a", "b", "c"])
>>> s.describe()
count 4
unique 3
top a
freq 2
dtype: object
```

Describing a timestamp ``Series``.

```
>>> s = pd.Series(
... [
... np.datetime64("2000-01-01"),
... np.datetime64("2010-01-01"),
... np.datetime64("2010-01-01"),
...]
...)
>>> s.describe()
count 3
mean 2006-09-01 08:00:00
min 2000-01-01 00:00:00
25% 2004-12-31 12:00:00
50% 2010-01-01 00:00:00
75% 2010-01-01 00:00:00
max 2010-01-01 00:00:00
```

dtype: object

Describing a ``DataFrame``. By default only numeric fields are returned.

```
>>> df = pd.DataFrame(
... {
... "categorical": pd.Categorical(["d", "e", "f"]),
... "numeric": [1, 2, 3],
... "object": ["a", "b", "c"],
... }
...)
>>> df.describe()
 numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Describing all columns of a ``DataFrame`` regardless of data type.

```
>>> df.describe(include="all") # doctest: +SKIP
 categorical numeric object
count 3 3.0 3
unique 3 NaN 3
top f NaN a
freq 1 NaN 1
mean NaN 2.0 NaN
std NaN 1.0 NaN
min NaN 1.0 NaN
25% NaN 1.5 NaN
50% NaN 2.0 NaN
75% NaN 2.5 NaN
max NaN 3.0 NaN
```

Describing a column from a ``DataFrame`` by accessing it as an attribute.

```
>>> df.numeric.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
Name: numeric, dtype: float64
```

Including only numeric columns in a ``DataFrame`` description.

```
>>> df.describe(include=[np.number])
 numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Including only string columns in a ``DataFrame`` description.

```
>>> df.describe(include=[object]) # doctest: +SKIP
 object
count 3
unique 3
top a
freq 1
```

Including only categorical columns from a ``DataFrame`` description.

```
>>> df.describe(include=["category"])
 categorical
count 3
unique 3
top d
freq 1
```

Excluding numeric columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[np.number]) # doctest: +SKIP
 categorical object
count 3 3
unique 3 3
top f a
freq 1 1
```

Excluding object columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[object]) # doctest: +SKIP
 categorical numeric
count 3 3.0
unique 3 NaN
top f NaN
freq 1 NaN
mean NaN 2.0
std NaN 1.0
min NaN 1.0
25% NaN 1.5
50% NaN 2.0
75% NaN 2.5
max NaN 3.0
```

`droplevel(self, level: IndexLabel, axis: Axis = 0) -> Self`  
 Return Series/DataFrame with requested index / column level(s) removed.

Parameters

level : int, str, or list-like  
 If a string is given, must be the name of a level  
 If list-like, elements must be names or positional indexes  
 of levels.

axis : {{0 or 'index', 1 or 'columns'}}, default 0  
 Axis along which the level(s) is removed:

\* 0 or 'index': remove level(s) in column.  
 \* 1 or 'columns': remove level(s) in row.

For ``Series`` this parameter is unused and defaults to 0.

Returns

Series/DataFrame  
 Series/DataFrame with requested index / column level(s) removed.

See Also

`DataFrame.replace` : Replace values given in ``to\_replace`` with ``value``.  
`DataFrame.pivot` : Return reshaped DataFrame organized by given  
 index / column values.

Examples

```
>>> df = (
... pd.DataFrame([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
... .set_index([0, 1])
... .rename_axis(["a", "b"])
...)
```

```
>>> df.columns = pd.MultiIndex.from_tuples(
... [("c", "e"), ("d", "f")], names=["level_1", "level_2"]
...)
```

```
>>> df
level_1 c d
level_2 e f
a b
```

```

1 2 3 4
5 6 7 8
9 10 11 12

```

```
>>> df.droplevel("a")
```

```

level_1 c d
level_2 e f
b
2 3 4
6 7 8
10 11 12

```

```
>>> df.droplevel("level_2", axis=1)
```

```

level_1 c d
a b
1 2 3 4
5 6 7 8
9 10 11 12

```

```
equals(self, other: object) -> bool
```

Test whether two objects contain the same elements.

This function allows two Series or DataFrames to be compared against each other to see if they have the same shape and elements. NaNs in the same location are considered equal.

The row/column index do not need to have the same type, as long as the values are considered equal. Corresponding columns and index must be of the same dtype.

Parameters

-----

other : Series or DataFrame

The other Series or DataFrame to be compared with the first.

Returns

-----

bool

True if all elements are the same in both objects, False otherwise.

See Also

-----

Series.eq : Compare two Series objects of the same length and return a Series where each element is True if the element in each Series is equal, False otherwise.

DataFrame.eq : Compare two DataFrame objects of the same shape and return a DataFrame where each element is True if the respective element in each DataFrame is equal, False otherwise.

testing.assert\_series\_equal : Raises an AssertionError if left and right are not equal. Provides an easy interface to ignore inequality in dtypes, indexes and precision among others.

testing.assert\_frame\_equal : Like assert\_series\_equal, but targets DataFrames.

numpy.array\_equal : Return True if two arrays have the same shape and elements, False otherwise.

Examples

-----

```
>>> df = pd.DataFrame({1: [10], 2: [20]})
```

```
>>> df
```

```

 1 2
0 10 20

```

DataFrames df and exactly\_equal have the same types and values for their elements and column labels, which will return True.

```
>>> exactly_equal = pd.DataFrame({1: [10], 2: [20]})
```

```
>>> exactly_equal
```

```

 1 2
0 10 20

```

```
>>> df.equals(exactly_equal)
```

```
True
```

DataFrames df and different\_column\_type have the same element types and values, but have different types for the column labels, which will still return True.

```
>>> different_column_type = pd.DataFrame({1.0: [10], 2.0: [20]})
>>> different_column_type
 1.0 2.0
0 10 20
>>> df.equals(different_column_type)
True
```

DataFrames `df` and `different_data_type` have different types for the same values for their elements, and will return `False` even though their column labels are the same values and types.

```
>>> different_data_type = pd.DataFrame({1: [10.0], 2: [20.0]})
>>> different_data_type
 1 2
0 10.0 20.0
>>> df.equals(different_data_type)
False
```

DataFrames with NaN in the same locations compare equal.

```
>>> df_nan1 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
>>> df_nan2 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
>>> df_nan1.equals(df_nan2)
True
```

If the NaN values are not in the same locations, they compare unequal.

```
>>> df_nan3 = pd.DataFrame({"a": [1, np.nan], "b": [3, 4]})
>>> df_nan1.equals(df_nan3)
False
```

```
ewm(
 self,
 com: float | None = None,
 span: float | None = None,
 halflife: float | TimedeltaConvertibleTypes | None = None,
 alpha: float | None = None,
 min_periods: int | None = 0,
 adjust: bool = True,
 ignore_na: bool = False,
 times: np.ndarray | DataFrame | Series | None = None,
 method: Literal['single', 'table'] = 'single'
) -> ExponentialMovingWindow
Provide exponentially weighted (EW) calculations.
```

Exactly one of ``com``, ``span``, ``halflife``, or ``alpha`` must be provided if ``times`` is not provided. If ``times`` is provided and ``adjust=True``, ``halflife`` and one of ``com``, ``span`` or ``alpha`` may be provided. If ``times`` is provided and ``adjust=False``, ``halflife`` must be the only provided decay-specification parameter.

#### Parameters

-----  
**com** : float, optional  
 Specify decay in terms of center of mass  

$$\alpha = 1 / (1 + com)$$
, for  $com \geq 0$ .

**span** : float, optional  
 Specify decay in terms of span  

$$\alpha = 2 / (span + 1)$$
, for  $span \geq 1$ .

**halflife** : float, str, timedelta, optional  
 Specify decay in terms of half-life  

$$\alpha = 1 - \exp(-\ln(2) / halflife)$$
, for  $halflife > 0$ .

If ``times`` is specified, a timedelta convertible unit over which an observation decays to half its value. Only applicable to ``mean()``, and halflife value will not apply to the other functions.

**alpha** : float, optional  
 Specify smoothing factor  $\alpha$  directly



```


$$0 < \alpha \leq 1$$

min_periods : int, default 0
 Minimum number of observations in window required to have a value;
 otherwise, result is ``np.nan``.

adjust : bool, default True
 Divide by decaying adjustment factor in beginning periods to account
 for imbalance in relative weightings (viewing EWMA as a moving average).

 - When ``adjust=True`` (default), the EW function is calculated using weights

$$w_i = (1 - \alpha)^i$$

 For example, the EW moving average of the series

$$[x_0, x_1, \dots, x_t]$$

 would be:

 .. math::
 y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots + (1 - \alpha)^t x_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}

 - When ``adjust=False``, the exponentially weighted function is calculated
 recursively:

 .. math::
 \begin{split}
 y_0 &= x_0 \\
 y_t &= (1 - \alpha) y_{t-1} + \alpha x_t,
 \end{split}

ignore_na : bool, default False
 Ignore missing values when calculating weights.

 - When ``ignore_na=False`` (default), weights are based on absolute positions.
 For example, the weights of x_0 and x_2 used in calculating
 the final weighted average of $[x_0, \text{None}, x_2]$ are
 $(1 - \alpha)^2$ and 1 if ``adjust=True``, and
 $(1 - \alpha)^2$ and α if ``adjust=False``.

 - When ``ignore_na=True``, weights are based
 on relative positions. For example, the weights of x_0 and x_2
 used in calculating the final weighted average of
 $[x_0, \text{None}, x_2]$ are $1 - \alpha$ and 1 if
 ``adjust=True``, and $1 - \alpha$ and α if ``adjust=False``.

times : np.ndarray, Series, default None

 Only applicable to ``mean()``.

 Times corresponding to the observations. Must be monotonically increasing and
 ``datetime64[ns]`` dtype.

 If 1-D array like, a sequence with the same shape as the observations.

method : str {'single', 'table'}, default 'single'
 Execute the rolling operation per single column or row (``'single'``)
 or over the entire object (``'table'``).

 This argument is only implemented when specifying ``engine='numba'``
 in the method call.

 Only applicable to ``mean()``

Returns

pandas.api.typing.ExponentialMovingWindow
 An instance of ExponentialMovingWindow for further exponentially weighted (EW)
 calculations, e.g. using the ``mean`` method.

See Also

rolling : Provides rolling window calculations.
expanding : Provides expanding transformations.

Notes

See :ref:`Windowing Operations <window.exponentially_weighted>`
for further usage details and examples.

Examples

```

```

>>> df = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]})
>>> df
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0

>>> df.ewm(com=0.5).mean()
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
>>> df.ewm(alpha=2 / 3).mean()
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213

adjust

>>> df.ewm(com=0.5, adjust=True).mean()
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
>>> df.ewm(com=0.5, adjust=False).mean()
 B
0 0.000000
1 0.666667
2 1.555556
3 1.555556
4 3.650794

ignore_na

>>> df.ewm(com=0.5, ignore_na=True).mean()
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.225000
>>> df.ewm(com=0.5, ignore_na=False).mean()
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213

times

Exponentially weighted mean with weights calculated with a timedelta ``halflife``
relative to ``times``.

>>> times = ['2020-01-01', '2020-01-03', '2020-01-10', '2020-01-15', '2020-01-17']
>>> df.ewm(halflife='4 days', times=pd.DatetimeIndex(times)).mean()
 B
0 0.000000
1 0.585786
2 1.523889
3 1.523889
4 3.233686

expanding(
 self,
 min_periods: int = 1,
 method: Literal['single', 'table'] = 'single'
) -> Expanding

```

Provide expanding window calculations.

An expanding window yields the value of an aggregation statistic with all the data available up to that point in time.

#### Parameters

-----

`min_periods` : int, default 1

Minimum number of observations in window required to have a value; otherwise, result is ``np.nan``.

`method` : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row (``'single'``) or over the entire object (``'table'``).

This argument is only implemented when specifying ``engine='numba'`` in the method call.

#### Returns

-----

`pandas.api.typing.Expanding`

An instance of `Expanding` for further expanding window calculations, e.g. using the ``sum`` method.

#### See Also

-----

`rolling` : Provides rolling window calculations.

`ewm` : Provides exponential weighted functions.

#### Notes

-----

See :ref:`Windowing Operations <window.expanding>` for further usage details and examples.

#### Examples

-----

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
```

```
>>> df
```

```
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

```
min_periods
```

Expanding sum with 1 vs 3 observations needed to calculate a value.

```
>>> df.expanding(1).sum()
```

```
 B
0 0.0
1 1.0
2 3.0
3 3.0
4 7.0
```

```
>>> df.expanding(3).sum()
```

```
 B
0 NaN
1 NaN
2 3.0
3 3.0
4 7.0
```

```
ffill(
```

```
 self,
```

```
 *,
```

```
 axis: None | Axis = None,
```

```
 inplace: bool = False,
```

```
 limit: None | int = None,
```

```
 limit_area: Literal['inside', 'outside'] | None = None
```

```
) -> Self
```

Fill NA/NaN values by propagating the last valid observation to next valid.

#### Parameters

-----

`axis` : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame

Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.

`inplace` : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

`limit` : int, default None  
If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

`limit_area` : {`None`, 'inside', 'outside'}, default None  
If limit is specified, consecutive NaNs will be filled with this restriction.

- \* `None`: No fill restriction.
- \* 'inside': Only fill NaNs surrounded by valid values (interpolate).
- \* 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

Returns

-----  
Series/DataFrame  
Object with missing values filled.

See Also

-----  
`DataFrame.bfill` : Fill NA/NaN values by using the next valid observation to fill the gap.

Examples

```
>>> df = pd.DataFrame(
... [
... [np.nan, 2, np.nan, 0],
... [3, 4, np.nan, 1],
... [np.nan, np.nan, np.nan, np.nan],
... [np.nan, 3, np.nan, 4],
...],
... columns=list("ABCD"),
...)
>>> df
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

```
>>> df.ffill()
 A B C D
0 NaN 2.0 NaN 0.0
1 3.0 4.0 NaN 1.0
2 3.0 4.0 NaN 1.0
3 3.0 3.0 NaN 4.0
```

```
>>> ser = pd.Series([1, np.nan, 2, 3])
>>> ser.ffill()
0 1.0
1 1.0
2 2.0
3 3.0
dtype: float64
```

```
fillna(
 self,
 value: Hashable | Mapping | Series | DataFrame,
 *,
 axis: Axis | None = None,
 inplace: bool = False,
 limit: int | None = None
) -> Self
 Fill NA/NaN values with `value`.
```

## Parameters

-----  
value : scalar, dict, Series, or DataFrame  
Value to use to fill holes (e.g. 0), alternately a dict/Series/DataFrame of values specifying which value to use for each index (for a Series) or column (for a DataFrame). Values not in the dict/Series/DataFrame will not be filled. This value cannot be a list.  
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.  
inplace : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).  
limit : int, default None  
This is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

## Returns

-----  
Series/DataFrame  
Object with missing values filled.

## See Also

-----  
ffill : Fill values by propagating the last valid observation to next valid.  
bfill : Fill values by using the next valid observation to fill the gap.  
interpolate : Fill NaN values using interpolation.  
reindex : Conform object to new index.  
asfreq : Convert TimeSeries to specified frequency.

## Notes

-----  
For non-object dtype, ``value=None`` will use the NA value of the dtype. See more details in the :ref:`Filling missing data<missing\_data.fillna>` section.

## Examples

-----  
>>> df = pd.DataFrame(  
... [  
... [np.nan, 2, np.nan, 0],  
... [3, 4, np.nan, 1],  
... [np.nan, np.nan, np.nan, np.nan],  
... [np.nan, 3, np.nan, 4],  
... ],  
... columns=list("ABCD"),  
... )  
>>> df  
 A B C D  
0 NaN 2.0 NaN 0.0  
1 3.0 4.0 NaN 1.0  
2 NaN NaN NaN NaN  
3 NaN 3.0 NaN 4.0

Replace all NaN elements with 0s.

```
>>> df.fillna(0)
 A B C D
0 0.0 2.0 0.0 0.0
1 3.0 4.0 0.0 1.0
2 0.0 0.0 0.0 0.0
3 0.0 3.0 0.0 4.0
```

Replace all NaN elements in column 'A', 'B', 'C', and 'D', with 0, 1, 2, and 3 respectively.

```
>>> values = {"A": 0, "B": 1, "C": 2, "D": 3}
>>> df.fillna(value=values)
 A B C D
0 0.0 2.0 2.0 0.0
1 3.0 4.0 2.0 1.0
2 0.0 1.0 2.0 3.0
3 0.0 3.0 2.0 4.0
```

Only replace the first NaN element.

```
>>> df.fillna(value=values, limit=1)
```

	A	B	C	D
0	0.0	2.0	2.0	0.0
1	3.0	4.0	NaN	1.0
2	NaN	1.0	NaN	3.0
3	NaN	3.0	NaN	4.0

When filling using a DataFrame, replacement happens along the same column names and same indices

```
>>> df2 = pd.DataFrame(np.zeros((4, 4)), columns=list("ABCE"))
```

```
>>> df.fillna(df2)
```

	A	B	C	D
0	0.0	2.0	0.0	0.0
1	3.0	4.0	0.0	1.0
2	0.0	0.0	0.0	NaN
3	0.0	3.0	0.0	4.0

Note that column D is not affected since it is not present in df2.

```
filter(
 self,
 items=None,
 like: str | None = None,
 regex: str | None = None,
 axis: Axis | None = None
) -> Self
Subset the DataFrame or Series according to the specified index labels.
```

For DataFrame, filter rows or columns depending on ``axis`` argument. Note that this routine does not filter based on content. The filter is applied to the labels of the index.

Parameters

```
items : list-like
 Keep labels from axis which are in items.
like : str
 Keep labels from axis for which "like in label == True".
regex : str (regular expression)
 Keep labels from axis for which re.search(regex, label) == True.
axis : {0 or 'index', 1 or 'columns', None}, default None
 The axis to filter on, expressed either as an index (int)
 or axis name (str). By default this is the info axis, 'columns' for
 ``DataFrame``. For ``Series`` this parameter is unused and defaults to
 ``None``.
```

Returns

```
Same type as caller
The filtered subset of the DataFrame or Series.
```

See Also

```
DataFrame.loc : Access a group of rows and columns
by label(s) or a boolean array.
```

Notes

The ``items``, ``like``, and ``regex`` parameters are enforced to be mutually exclusive.

``axis`` defaults to the info axis that is used when indexing with ``[]``.

Examples

```
>>> df = pd.DataFrame(
... np.array([[1, 2, 3], [4, 5, 6]]),
... index=["mouse", "rabbit"],
... columns=["one", "two", "three"],
...)
>>> df
```

	one	two	three
mouse	1	2	3
rabbit	4	5	6

```

>>> # select columns by name
>>> df.filter(items=["one", "three"])
 one three
mouse 1 3
rabbit 4 6

>>> # select columns by regular expression
>>> df.filter(regex="e$", axis=1)
 one three
mouse 1 3
rabbit 4 6

>>> # select rows containing 'bbi'
>>> df.filter(like="bbi", axis=0)
 one two three
rabbit 4 5 6

```

first\_valid\_index(self) -> Hashable  
Return index for first non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information on which values are considered missing.

Returns  
-----  
type of index  
Index of first non-missing value.

See Also  
-----  
DataFrame.last\_valid\_index : Return index for last non-NA value or None, if no non-NA value is found.  
Series.last\_valid\_index : Return index for last non-NA value or None, if no non-NA value is found.  
DataFrame.isna : Detect missing values.

Examples  
-----  
For Series:

```

>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
1
>>> s.last_valid_index()
2

>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If all elements in Series are NA/null, returns None.

```

>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If Series is empty, returns None.

For DataFrame:

```

>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
 A B
0 NaN NaN
1 NaN 3.0
2 2.0 4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2
>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})

```

```
>>> df
 A B
0 None None
1 None None
2 None None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If DataFrame is empty, returns None.

`get(self, key, default=None)`  
Get item from object for given key (ex: DataFrame column).

Returns ``default`` value if not found.

Parameters

-----  
key : object  
Key for which item should be returned.  
default : object, default None  
Default value to return if key is not found.

Returns

-----  
same type as items contained in object  
Item for given key or ``default`` value, if key is not found.

See Also

-----  
DataFrame.get : Get item from object for given key (ex: DataFrame column).  
Series.get : Get item from object for given key (ex: DataFrame column).

Examples

```
>>> df = pd.DataFrame(
... [
... [24.3, 75.7, "high"],
... [31, 87.8, "high"],
... [22, 71.6, "medium"],
... [35, 95, "medium"],
...],
... columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
... index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
...)
```

```
>>> df
 temp_celsius temp_fahrenheit windspeed
2014-02-12 24.3 75.7 high
2014-02-13 31.0 87.8 high
2014-02-14 22.0 71.6 medium
2014-02-15 35.0 95.0 medium
```

```
>>> df.get(["temp_celsius", "windspeed"])
 temp_celsius windspeed
2014-02-12 24.3 high
2014-02-13 31.0 high
2014-02-14 22.0 medium
2014-02-15 35.0 medium
```

```
>>> ser = df["windspeed"]
>>> ser.get("2014-02-13")
'high'
```



If the key isn't found, the default value will be used.

```
>>> df.get(["temp_celsius", "temp_kelvin"], default="default_value")
'default_value'
```

```
>>> ser.get("2014-02-10", "[unknown]")
'[unknown]'
```

```
head(self, n: int = 5) -> Self
Return the first `n` rows.
```

This function exhibits the same behavior as `df[:n]`, returning the first `n` rows based on position. It is useful for quickly checking if your object has the right type of data in it.

When `n` is positive, it returns the first `n` rows. For `n` equal to 0, it returns an empty object. When `n` is negative, it returns all rows except the last `|n|` rows, mirroring the behavior of `df[:n]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

`n` : int, default 5  
Number of rows to select.

Returns

same type as caller  
The first `n` rows of the caller object.

See Also

`DataFrame.tail`: Returns the last `n` rows.

Examples

```
>>> df = pd.DataFrame(
... {
... "animal": [
... "alligator",
... "bee",
... "falcon",
... "lion",
... "monkey",
... "parrot",
... "shark",
... "whale",
... "zebra",
...]
... }
...)
>>> df
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
5 parrot
6 shark
7 whale
8 zebra
```

Viewing the first 5 lines

```
>>> df.head()
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
```

Viewing the first `n` lines (three in this case)

```
>>> df.head(3)
```

```

 animal
0 alligator
1 bee
2 falcon

```

For negative values of `n`

```

>>> df.head(-3)
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
5 parrot

```

```
infer_objects(self, copy: bool | lib.NoDefault = <no_default>) -> Self
 Attempt to infer better dtypes for object columns.
```

Attempts soft conversion of object-dtyped columns, leaving non-object and unconvertible columns unchanged. The inference rules are the same as during normal Series/DataFrame construction.

Parameters

```

copy : bool, default False
 This keyword is now ignored; changing its value will have no
 impact on the method.

```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`\_ for more details.

Returns

```

same type as input object
 Returns an object of the same type as the input object.

```

See Also

```

to_datetime : Convert argument to datetime.
to_timedelta : Convert argument to timedelta.
to_numeric : Convert argument to numeric type.
convert_dtypes : Convert argument to best possible dtype.

```

Examples

```

>>> df = pd.DataFrame({"A": ["a", 1, 2, 3]})
>>> df = df.iloc[1:]
>>> df
 A
1 1
2 2
3 3

```

```

>>> df.dtypes
A object
dtype: object

```

```

>>> df.infer_objects().dtypes
A int64
dtype: object

```

```

interpolate(
 self,
 method: InterpolateOptions = 'linear',
 *,
 axis: Axis = 0,
 limit: int | None = None,
 inplace: bool = False,
 limit_direction: Literal['forward', 'backward', 'both'] | None = None,

```

```

 limit_area: Literal['inside', 'outside'] | None = None,
 **kwargs
) -> Self
 Fill NaN values using an interpolation method.

Please note that only ``method='linear'`` is supported for
DataFrame/Series with a MultiIndex.

Parameters

method : str, default 'linear'
 Interpolation technique to use. One of:

 * 'linear': Ignore the index and treat the values as equally
 spaced. This is the only method supported on MultiIndexes.
 * 'time': Works on daily and higher resolution data to interpolate
 given length of interval. This interpolates values based on
 time interval between observations.
 * 'index': The interpolation uses the numerical values
 of the DataFrame's index to linearly calculate missing values.
 * 'values': Interpolation based on the numerical values
 in the DataFrame, treating them as equally spaced along the index.
 * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic',
 'barycentric', 'polynomial': Passed to
 `scipy.interpolate.interp1d`, whereas 'spline' is passed to
 `scipy.interpolate.UnivariateSpline`. These methods use the numerical
 values of the index. Both 'polynomial' and 'spline' require that
 you also specify an `order` (int), e.g.
 ``df.interpolate(method='polynomial', order=5)``. Note that,
 `slinear` method in Pandas refers to the SciPy first order `spline`
 instead of Pandas first order `spline`.
 * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima',
 'cubicspline': Wrappers around the SciPy interpolation methods of
 similar names. See `Notes`.
 * 'from_derivatives': Refers to
 `scipy.interpolate.BPoly.from_derivatives`.

axis : {{0 or 'index', 1 or 'columns', None}}, default None
 Axis to interpolate along. For `Series` this parameter is unused
 and defaults to 0.
limit : int, optional
 Maximum number of consecutive NaNs to fill. Must be greater than
 0.
inplace : bool, default False
 Update the data in place if possible.
limit_direction : {{'forward', 'backward', 'both'}}, optional, default 'forward'
 Consecutive NaNs will be filled in this direction.

limit_area : {{None, 'inside', 'outside'}}, default None
 If limit is specified, consecutive NaNs will be filled with this
 restriction.

 * ``None``: No fill restriction.
 * 'inside': Only fill NaNs surrounded by valid values
 (interpolate).
 * 'outside': Only fill NaNs outside valid values (extrapolate).

**kwargs : optional
 Keyword arguments to pass on to the interpolating function.

Returns

Series or DataFrame
 Returns the same object type as the caller, interpolated at
 some or all ``NaN`` values.

See Also

fillna : Fill missing values using different methods.
scipy.interpolate.Akima1DInterpolator : Piecewise cubic polynomials
 (Akima interpolator).
scipy.interpolate.BPoly.from_derivatives : Piecewise polynomial in the
 Bernstein basis.
scipy.interpolate.interp1d : Interpolate a 1-D function.
scipy.interpolate.KroghInterpolator : Interpolate polynomial (Krogh
 interpolator).
scipy.interpolate.PchipInterpolator : PCHIP 1-d monotonic cubic

```

interpolation.  
scipy.interpolate.CubicSpline : Cubic spline data interpolator.

#### Notes

-----

The 'krogh', 'piecewise\_polynomial', 'spline', 'pchip' and 'akima' methods are wrappers around the respective SciPy implementations of similar names. These use the actual numerical values of the index. For more information on their behavior, see the `SciPy documentation`

<<https://docs.scipy.org/doc/scipy/reference/interpolate.html#univariate-interpolation>>`

#### Examples

-----

Filling in ``NaN`` in a :class:`~pandas.Series` via linear interpolation.

```
>>> s = pd.Series([0, 1, np.nan, 3])
>>> s
0 0.0
1 1.0
2 NaN
3 3.0
dtype: float64
>>> s.interpolate()
0 0.0
1 1.0
2 2.0
3 3.0
dtype: float64
```

Filling in ``NaN`` in a Series via polynomial interpolation or splines: Both 'polynomial' and 'spline' methods require that you also specify an ``order`` (int).

```
>>> s = pd.Series([0, 2, np.nan, 8])
>>> s.interpolate(method="polynomial", order=2)
0 0.000000
1 2.000000
2 4.666667
3 8.000000
dtype: float64
```

Fill the DataFrame forward (that is, going down) along each column using linear interpolation.

Note how the last entry in column 'a' is interpolated differently, because there is no entry after it to use for interpolation. Note how the first entry in column 'b' remains ``NaN``, because there is no entry before it to use for interpolation.

```
>>> df = pd.DataFrame(
... [
... (0.0, np.nan, -1.0, 1.0),
... (np.nan, 2.0, np.nan, np.nan),
... (2.0, 3.0, np.nan, 9.0),
... (np.nan, 4.0, -4.0, 16.0),
...],
... columns=list("abcd"),
...)
>>> df
 a b c d
0 0.0 NaN -1.0 1.0
1 NaN 2.0 NaN NaN
2 2.0 3.0 NaN 9.0
3 NaN 4.0 -4.0 16.0
>>> df.interpolate(method="linear", limit_direction="forward", axis=0)
 a b c d
0 0.0 NaN -1.0 1.0
1 1.0 2.0 -2.0 5.0
2 2.0 3.0 -3.0 9.0
3 2.0 4.0 -4.0 16.0
```

Using polynomial interpolation.

```
>>> df["d"].interpolate(method="polynomial", order=2)
0 1.0
```

```

1 4.0
2 9.0
3 16.0
Name: d, dtype: float64

```

`keys(self)` -> Index  
Get the 'info axis' (see Indexing for more).

This is index for Series, columns for DataFrame.

Returns

-----  
Index

Info axis.

See Also

-----  
`DataFrame.index` : The index (row labels) of the DataFrame.  
`DataFrame.columns`: The column labels of the DataFrame.

Examples

```

>>> d = pd.DataFrame(
... data={"A": [1, 2, 3], "B": [0, 4, 8]}, index=["a", "b", "c"]
...)
>>> d
 A B
a 1 0
b 2 4
c 3 8
>>> d.keys()
Index(['A', 'B'], dtype='str')

```

`last_valid_index(self)` -> Hashable  
Return index for last non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information on which values are considered missing.

Returns

-----  
type of index

Index of last non-missing value.

See Also

-----  
`DataFrame.first_valid_index` : Return index for first non-NA value or None, if no non-NA value is found.  
`Series.first_valid_index` : Return index for first non-NA value or None, if no non-NA value is found.  
`DataFrame.isna` : Detect missing values.

Examples

-----  
For Series:

```

>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
1
>>> s.last_valid_index()
2

>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If all elements in Series are NA/null, returns None.

```

>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If Series is empty, returns None.

For DataFrame:

```
>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
 A B
0 NaN NaN
1 NaN 3.0
2 2.0 4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2

>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
 A B
0 None None
1 None None
2 None None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If DataFrame is empty, returns None.

```
mask(
 self,
 cond,
 other=<no_default>,
 *,
 inplace: bool = False,
 axis: Axis | None = None,
 level: Level | None = None
) -> Self
 Replace values where the condition is True.

Parameters

cond : bool Series/DataFrame, array-like, or callable
 Where `cond` is False, keep the original value. Where
 True, replace with corresponding value from `other`.
 If `cond` is callable, it is computed on the Series/DataFrame and
 should return boolean Series/DataFrame or array. The callable must
 not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
 Entries where `cond` is True are replaced with
 corresponding value from `other`.
 If other is callable, it is computed on the Series/DataFrame and
 should return scalar or Series/DataFrame. The callable must not
 change input Series/DataFrame (though pandas doesn't check it).
 If not specified, entries will be filled with the corresponding
 NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension
 dtypes).
inplace : bool, default False
 Whether to perform the operation in place on the data.
axis : int, default None
 Alignment axis if needed. For `Series` this parameter is
 unused and defaults to 0.
level : int, default None
 Alignment level if needed.

Returns

```

## Series or DataFrame

When applied to a Series, the function will return a Series,  
and when applied to a DataFrame, it will return a DataFrame.

## See Also

`:func:DataFrame.where` : Return an object of same shape as caller.  
`:func:Series.where` : Return an object of same shape as caller.

## Notes

The mask method is an application of the if-then idiom. For each element in the caller, if `cond` is `False` the element is used; otherwise the corresponding element from `other` is used. If the axis of `other` does not align with axis of `cond` Series/DataFrame, the values of `cond` on misaligned index positions will be filled with True.

The signature for `:func:Series.where` or `:func:DataFrame.where` differs from `:func:numpy.where`. Roughly `df1.where(m, df2)` is equivalent to `np.where(m, df1, df2)`.

For further details and examples see the `mask` documentation in `:ref:indexing <indexing.where_mask>`.

The dtype of the object takes precedence. The fill value is casted to the object's dtype, if this can be done losslessly.

## Examples

```
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0 NaN
1 1.0
2 2.0
3 3.0
4 4.0
dtype: float64
>>> s.mask(s > 0)
0 0.0
1 NaN
2 NaN
3 NaN
4 NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0 0
1 99
2 99
3 99
4 99
dtype: int64
>>> s.mask(t, 99)
0 99
1 1
2 99
3 99
4 99
dtype: int64

>>> s.where(s > 1, 10)
0 10
1 10
2 2
3 3
4 4
dtype: int64
>>> s.mask(s > 1, 10)
0 0
1 1
2 10
3 10
4 10
dtype: int64
```

```

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
 A B
0 0 1
1 2 3
2 4 5
3 6 7
4 8 9
>>> m = df % 3 == 0
>>> df.where(m, -df)
 A B
0 0 -1
1 -2 3
2 -4 5
3 6 -7
4 -8 9
>>> df.where(m, -df) == np.where(m, df, -df)
 A B
0 True True
1 True True
2 True True
3 True True
4 True True
>>> df.where(m, -df) == df.mask(~m, -df)
 A B
0 True True
1 True True
2 True True
3 True True
4 True True

```

```

pct_change(
 self,
 periods: int = 1,
 fill_method: None = None,
 freq=None,
 **kwargs
) -> Self
 Fractional change between the current and a prior element.

```

Computes the fractional change from the immediately previous row by default. This is useful in comparing the fraction of change in a time series of elements.

.. note::

Despite the name of this method, it calculates fractional change (also known as per unit change or relative change) and not percentage change. If you need the percentage change, multiply these values by 100.

Parameters

```

periods : int, default 1
 Periods to shift for forming percent change.
fill_method : None
 Must be None. This argument will be removed in a future version of pandas.
freq : DateOffset, timedelta, or str, optional
 Increment to use from time series API (e.g. 'ME' or BDay()).
**kwargs
 Additional keyword arguments are passed into
 `DataFrame.shift` or `Series.shift`.

```

Returns

```

Series or DataFrame
 The same type as the calling object.

```

See Also

```

Series.diff : Compute the difference of two elements in a Series.
DataFrame.diff : Compute the difference of two elements in a DataFrame.
Series.shift : Shift the index by some number of periods.
DataFrame.shift : Shift the index by some number of periods.

```

Examples



```

Series
```

```
>>> s = pd.Series([90, 91, 85])
>>> s
0 90
1 91
2 85
dtype: int64
```

```
>>> s.pct_change()
0 NaN
1 0.011111
2 -0.065934
dtype: float64
```

```
>>> s.pct_change(periods=2)
0 NaN
1 NaN
2 -0.055556
dtype: float64
```

See the percentage change in a Series where filling NAs with last valid observation forward to next valid.

```
>>> s = pd.Series([90, 91, None, 85])
>>> s
0 90.0
1 91.0
2 NaN
3 85.0
dtype: float64
```

```
>>> s.ffill().pct_change()
0 NaN
1 0.011111
2 0.000000
3 -0.065934
dtype: float64
```

```
DataFrame
```

Percentage change in French franc, Deutsche Mark, and Italian lira from 1980-01-01 to 1980-03-01.

```
>>> df = pd.DataFrame(
... {
... "FR": [4.0405, 4.0963, 4.3149],
... "GR": [1.7246, 1.7482, 1.8519],
... "IT": [804.74, 810.01, 860.13],
... },
... index=["1980-01-01", "1980-02-01", "1980-03-01"],
...)
>>> df
```

	FR	GR	IT
1980-01-01	4.0405	1.7246	804.74
1980-02-01	4.0963	1.7482	810.01
1980-03-01	4.3149	1.8519	860.13

```
>>> df.pct_change()
 FR GR IT
1980-01-01 NaN NaN NaN
1980-02-01 0.013810 0.013684 0.006549
1980-03-01 0.053365 0.059318 0.061876
```

Percentage of change in GOOG and APPL stock volume. Shows computing the percentage change between columns.

```
>>> df = pd.DataFrame(
... {
... "2016": [1769950, 30586265],
... "2015": [1500923, 40912316],
... "2014": [1371819, 41403351],
... },
... index=["GOOG", "APPL"],
...)
>>> df
```

	2016	2015	2014
GOOG	1769950	1500923	1371819
APPL	30586265	40912316	41403351

```
>>> df.pct_change(axis="columns", periods=-1)
 2016 2015 2014
GOOG 0.179241 0.094112 NaN
APPL -0.252395 -0.011860 NaN
```

```
pipe(
 self,
 func: Callable[[Concatenate[Self, P], T] | tuple[Callable[... , T], str],
 *args: Any,
 **kwargs: Any
) -> T
Apply chainable functions that expect Series or DataFrames.
```

#### Parameters

```
func : function
 Function to apply to the Series/DataFrame.
 ``args``, and ``kwargs`` are passed into ``func``.
 Alternatively a ``(callable, data_keyword)`` tuple where
 ``data_keyword`` is a string indicating the keyword of
 ``callable`` that expects the Series/DataFrame.
args : iterable, optional
 Positional arguments passed into ``func``.
**kwargs : mapping, optional
 A dictionary of keyword arguments passed into ``func``.
```

#### Returns

```
The return type of ``func``.
The result of applying ``func`` to the Series or DataFrame.
```

#### See Also

```
DataFrame.apply : Apply a function along input axis of DataFrame.
DataFrame.map : Apply a function elementwise on a whole DataFrame.
Series.map : Apply a mapping correspondence on a
:class:`~pandas.Series`.
```

#### Notes

```
Use ``.pipe`` when chaining together functions that expect
Series, DataFrames or GroupBy objects.
```

#### Examples

```
Constructing an income DataFrame from a dictionary.
```

```
>>> data = [[8000, 1000], [9500, np.nan], [5000, 2000]]
>>> df = pd.DataFrame(data, columns=["Salary", "Others"])
>>> df
 Salary Others
0 8000 1000.0
1 9500 NaN
2 5000 2000.0
```

```
Functions that perform tax reductions on an income DataFrame.
```

```
>>> def subtract_federal_tax(df):
... return df * 0.9
>>> def subtract_state_tax(df, rate):
... return df * (1 - rate)
>>> def subtract_national_insurance(df, rate, rate_increase):
... new_rate = rate + rate_increase
... return df * (1 - new_rate)
```

```
Instead of writing
```

```
>>> subtract_national_insurance(
... subtract_state_tax(subtract_federal_tax(df), rate=0.12),
... rate=0.05,
... rate_increase=0.02,
...) # doctest: +SKIP
```

You can write

```
>>> (
... df.pipe(subtract_federal_tax)
... .pipe(subtract_state_tax, rate=0.12)
... .pipe(subtract_national_insurance, rate=0.05, rate_increase=0.02)
...)
 Salary Others
0 5892.48 736.56
1 6997.32 NaN
2 3682.80 1473.12
```

If you have a function that takes the data as (say) the second argument, pass a tuple indicating which keyword expects the data. For example, suppose `subtract_national_insurance` takes its data as `df` in the second argument:

```
>>> def subtract_national_insurance(rate, df, rate_increase):
... new_rate = rate + rate_increase
... return df * (1 - new_rate)
>>> (
... df.pipe(subtract_federal_tax)
... .pipe(subtract_state_tax, rate=0.12)
... .pipe(
... (subtract_national_insurance, "df"), rate=0.05, rate_increase=0.02
...)
...)
 Salary Others
0 5892.48 736.56
1 6997.32 NaN
2 3682.80 1473.12
```

```
rank(
 self,
 axis: Axis = 0,
 method: Literal['average', 'min', 'max', 'first', 'dense'] = 'average',
 numeric_only: bool = False,
 na_option: Literal['keep', 'top', 'bottom'] = 'keep',
 ascending: bool = True,
 pct: bool = False
) -> Self
 Compute numerical data ranks (1 through n) along axis.
```

By default, equal values are assigned a rank that is the average of the ranks of those values.

#### Parameters

-----  
axis : {0 or 'index', 1 or 'columns'}, default 0  
Index to direct ranking.  
For `Series` this parameter is unused and defaults to 0.  
method : {'average', 'min', 'max', 'first', 'dense'}, default 'average'  
How to rank the group of records that have the same value (i.e. ties):

- \* average: average rank of the group
- \* min: lowest rank in the group
- \* max: highest rank in the group
- \* first: ranks assigned in order they appear in the array
- \* dense: like 'min', but rank always increases by 1 between groups.

numeric\_only : bool, default False  
For DataFrame objects, rank only numeric columns if set to True.

.. versionchanged:: 2.0.0  
The default value of `numeric_only` is now `False`.

na\_option : {'keep', 'top', 'bottom'}, default 'keep'  
How to rank NaN values:

- \* keep: assign NaN rank to NaN values
- \* top: assign lowest rank to NaN values
- \* bottom: assign highest rank to NaN values

ascending : bool, default True  
Whether or not the elements should be ranked in ascending order.

pct : bool, default False  
Whether or not to display the returned rankings in percentile

form.

Returns

-----

same type as caller

Return a Series or DataFrame with data ranks as values.

See Also

-----

core.groupby.DataFrameGroupBy.rank : Rank of values within each group.

core.groupby.SeriesGroupBy.rank : Rank of values within each group.

Examples

-----

```
>>> df = pd.DataFrame(
... data={
... "Animal": ["cat", "penguin", "dog", "spider", "snake"],
... "Number_legs": [4, 2, 4, 8, np.nan],
... }
...)
>>> df
 Animal Number_legs
0 cat 4.0
1 penguin 2.0
2 dog 4.0
3 spider 8.0
4 snake NaN
```

Ties are assigned the mean of the ranks (by default) for the group.

```
>>> s = pd.Series(range(5), index=list("abcde"))
>>> s["d"] = s["b"]
>>> s.rank()
a 1.0
b 2.5
c 4.0
d 2.5
e 5.0
dtype: float64
```

The following example shows how the method behaves with the above parameters:

- \* default\_rank: this is the default behaviour obtained without using any parameter.
- \* max\_rank: setting ``method = 'max'`` the records that have the same values are ranked using the highest rank (e.g.: since 'cat' and 'dog' are both in the 2nd and 3rd position, rank 3 is assigned.)
- \* NA\_bottom: choosing ``na\_option = 'bottom'``, if there are records with NaN values they are placed at the bottom of the ranking.
- \* pct\_rank: when setting ``pct = True``, the ranking is expressed as percentile rank.

```
>>> df["default_rank"] = df["Number_legs"].rank()
>>> df["max_rank"] = df["Number_legs"].rank(method="max")
>>> df["NA_bottom"] = df["Number_legs"].rank(na_option="bottom")
>>> df["pct_rank"] = df["Number_legs"].rank(pct=True)
>>> df
 Animal Number_legs default_rank max_rank NA_bottom pct_rank
0 cat 4.0 2.5 3.0 2.5 0.625
1 penguin 2.0 1.0 1.0 1.0 0.250
2 dog 4.0 2.5 3.0 2.5 0.625
3 spider 8.0 4.0 4.0 4.0 1.000
4 snake NaN NaN NaN 5.0 NaN
```

```
reindex_like(
 self,
 other,
 method: Literal['backfill', 'bfill', 'pad', 'ffill', 'nearest'] | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 limit: int | None = None,
 tolerance=None
) -> Self
Return an object with matching indices as other object.
```

Conform the object to the same index on all axes. Optional filling logic, placing NaN in locations having no value

in the previous index. A new object is produced unless the new index is equivalent to the current one and copy=False.

#### Parameters

other : Object of the same data type  
Its row and column indices are used to define the new indices of this object.  
method : {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}  
Method to use for filling holes in reindexed DataFrame.  
Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

.. deprecated:: 3.0.0

- \* None (default): don't fill gaps
- \* pad / ffill: propagate last valid observation forward to next valid
- \* backfill / bfill: use next valid observation to fill gap
- \* nearest: use nearest valid observations to fill gap.

copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` [https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

limit : int, default None

Maximum number of consecutive labels to fill for inexact matches.

tolerance : optional

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

#### Returns

Series or DataFrame

Same type as caller, but with changed indices on each axis.

#### See Also

DataFrame.set\_index : Set row labels.

DataFrame.reset\_index : Remove row labels or move them to new columns.

DataFrame.reindex : Change to new indices or expand indices.

#### Notes

Same as calling

```.reindex(index=other.index, columns=other.columns,...)``.

Examples

```
>>> df1 = pd.DataFrame(  
...     [  
...         [24.3, 75.7, "high"],  
...         [31, 87.8, "high"],  
...         [22, 71.6, "medium"],  
...         [35, 95, "medium"],  
...     ],  
...     columns=["temp_celsius", "temp_fahrenheit", "windspeed"],  
...     index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),  
... )  
  
>>> df1
```

```

temp_celsius temp_fahrenheit windspeed
2014-02-12    24.3             75.7      high
2014-02-13    31.0             87.8      high
2014-02-14    22.0             71.6    medium
2014-02-15    35.0             95.0    medium

>>> df2 = pd.DataFrame(
...     [[28, "low"], [30, "low"], [35.1, "medium"]],
...     columns=["temp_celsius", "windspeed"],
...     index=pd.DatetimeIndex(["2014-02-12", "2014-02-13", "2014-02-15"]),
... )

```

```

>>> df2

temp_celsius windspeed
2014-02-12    28.0      low
2014-02-13    30.0      low
2014-02-15    35.1    medium

```

```

>>> df2.reindex_like(df1)

temp_celsius temp_fahrenheit windspeed
2014-02-12    28.0             NaN      low
2014-02-13    30.0             NaN      low
2014-02-14     NaN             NaN      NaN
2014-02-15    35.1             NaN    medium

```

```

rename_axis(
    self,
    mapper: IndexLabel | lib.NoDefault = <no_default>,
    *,
    index=<no_default>,
    columns=<no_default>,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>,
    inplace: bool = False
) -> Self | None
Set the name of the axis for the index or columns.

```

Parameters

mapper : scalar, list-like, optional
Value to set the axis name attribute.

Use either ``mapper`` and ``axis`` to specify the axis to target with ``mapper``, or ``index`` and/or ``columns``.

index : scalar, list-like, dict-like or function, optional
A scalar, list-like, dict-like or functions transformations to apply to that axis' values.

columns : scalar, list-like, dict-like or function, optional
A scalar, list-like, dict-like or functions transformations to apply to that axis' values.

axis : {0 or 'index', 1 or 'columns'}, default 0
The axis to rename.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

inplace : bool, default False
Modifies the object directly, instead of creating a new Series or DataFrame.

Returns

DataFrame, or None
The same type as the caller or None if ``inplace=True``.

See Also

Series.rename : Alter Series index labels or name.
 DataFrame.rename : Alter DataFrame index labels or name.
 Index.rename : Set new names on index.

Notes

 ``DataFrame.rename\_axis`` supports two calling conventions

```
* ``(index=index_mapper, columns=columns_mapper, ...)``
* ``(mapper, axis={'index', 'columns'}, ...)``
```

The first calling convention will only modify the names of the index and/or the names of the Index object that is the columns. In this case, the parameter ``copy`` is ignored.

The second calling convention will modify the names of the corresponding index if mapper is a list or a scalar. However, if mapper is dict-like or a function, it will use the deprecated behavior of modifying the axis *labels*.

We *highly* recommend using keyword arguments to clarify your intent.

Examples

****DataFrame****

```
>>> df = pd.DataFrame(
...     {"num_legs": [4, 4, 2], "num_arms": [0, 0, 2]}, ["dog", "cat", "monkey"]
... )
>>> df
   num_legs  num_arms
dog         4         0
cat         4         0
monkey      2         2
>>> df = df.rename_axis("animal")
>>> df
   num_legs  num_arms
animal
dog         4         0
cat         4         0
monkey      2         2
>>> df = df.rename_axis("limbs", axis="columns")
>>> df
limbs  num_legs  num_arms
animal
dog         4         0
cat         4         0
monkey      2         2
```

****MultiIndex****

```
>>> df.index = pd.MultiIndex.from_product(
...     [ ["mammal"], ["dog", "cat", "monkey"] ], names=["type", "name"]
... )
>>> df
limbs  num_legs  num_arms
type  name
mammal dog         4         0
       cat         4         0
       monkey      2         2

>>> df.rename_axis(index={"type": "class"})
limbs  num_legs  num_arms
class  name
mammal dog         4         0
       cat         4         0
       monkey      2         2

>>> df.rename_axis(columns=str.upper)
LIMBS  num_legs  num_arms
type  name
mammal dog         4         0
       cat         4         0
       monkey      2         2
```

replace(

```

self,
to_replace=None,
value=<no_default>,
*,
inplace: bool = False,
regex: bool = False
) -> Self
Replace values given in `to_replace` with `value`.

Values of the Series/DataFrame are replaced with other values dynamically.
This differs from updating with ``.loc`` or ``.iloc``, which require
you to specify a location to update with some value.

Parameters
-----
to_replace : str, regex, list, dict, Series, int, float, or None
    How to find the values that will be replaced.

    * numeric, str or regex:

        - numeric: numeric values equal to `to_replace` will be
          replaced with `value`
        - str: string exactly matching `to_replace` will be replaced
          with `value`
        - regex: regexes matching `to_replace` will be replaced with
          `value`

    * list of str, regex, or numeric:

        - First, if `to_replace` and `value` are both lists, they
          must be the same length.
        - Second, if ``regex=True`` then all of the strings in both
          lists will be interpreted as regexes otherwise they will match
          directly. This doesn't matter much for `value` since there
          are only a few possible substitution regexes you can use.
        - str, regex and numeric rules apply as above.

    * dict:

        - Dicts can be used to specify different replacement values
          for different existing values. For example,
          ``{'a': 'b', 'y': 'z'}`` replaces the value 'a' with 'b' and
          'y' with 'z'. To use a dict in this way, the optional `value`
          parameter should not be given.
        - For a DataFrame a dict can specify that different values
          should be replaced in different columns. For example,
          ``{'a': 1, 'b': 'z'}`` looks for the value 1 in column 'a'
          and the value 'z' in column 'b' and replaces these values
          with whatever is specified in `value`. The `value` parameter
          should not be ``None`` in this case. You can treat this as a
          special case of passing two lists except that you are
          specifying the column to search in.
        - For a DataFrame nested dictionaries, e.g.,
          ``{'a': {'b': np.nan}}``, are read as follows: look in column
          'a' for the value 'b' and replace it with NaN. The optional `value`
          parameter should not be specified to use a nested dict in this
          way. You can nest regular expressions as well. Note that
          column names (the top-level dictionary keys in a nested
          dictionary) cannot be regular expressions.

    * None:

        - This means that the `regex` argument must be a string,
          compiled regular expression, or list, dict, ndarray or
          Series of such elements. If `value` is also ``None`` then
          this must be a nested dictionary or Series.

    See the examples section for examples of each of these.
value : scalar, dict, list, str, regex, default None
    Value to replace any values matching `to_replace` with.
    For a DataFrame a dict of values can be used to specify which
    value to use for each column (columns not in the dict will not be
    filled). Regular expressions, strings and lists or dicts of such
    objects are also allowed.

inplace : bool, default False
    If True, performs operation inplace.

```


`regex` : bool or same types as ``to_replace``, default False
Whether to interpret ``to_replace`` and/or ``value`` as regular expressions. Alternatively, this could be a regular expression or a list, dict, or array of regular expressions in which case ``to_replace`` must be ``None``.

Returns

Series/DataFrame
Object after replacement.

Raises

AssertionError
* If ``regex`` is not a ``bool`` and ``to_replace`` is not ``None``.

TypeError

- * If ``to_replace`` is not a scalar, array-like, ``dict``, or ``None``
- * If ``to_replace`` is a ``dict`` and ``value`` is not a ``list``, ``dict``, ``ndarray``, or ``Series``
- * If ``to_replace`` is ``None`` and ``regex`` is not compilable into a regular expression or is a list, dict, ndarray, or Series.
- * When replacing multiple ``bool`` or ``datetime64`` objects and the arguments to ``to_replace`` does not match the type of the value being replaced

ValueError

- * If a ``list`` or an ``ndarray`` is passed to ``to_replace`` and ``value`` but they are not the same length.

See Also

`Series.fillna` : Fill NA values.
`DataFrame.fillna` : Fill NA values.
`Series.where` : Replace values based on boolean condition.
`DataFrame.where` : Replace values based on boolean condition.
`DataFrame.map`: Apply a function to a Dataframe elementwise.
`Series.map`: Map values of Series according to an input mapping or function.
`Series.str.replace` : Simple string replacement.

Notes

- * Regex substitution is performed under the hood with ``re.sub``. The rules for substitution for ``re.sub`` are the same.
- * Regular expressions will only substitute on strings, meaning you cannot provide, for example, a regular expression matching floating point numbers and expect the columns in your frame that have a numeric dtype to be matched. However, if those floating point numbers *are* strings, then you can do this.
- * This method has *a lot* of options. You are encouraged to experiment and play with this method to gain intuition about how it works.
- * When dict is used as the ``to_replace`` value, it is like key(s) in the dict are the ``to_replace`` part and value(s) in the dict are the value parameter.

Examples

****Scalar ``to_replace`` and ``value``****

```
>>> s = pd.Series([1, 2, 3, 4, 5])
>>> s.replace(1, 5)
0    5
1    2
2    3
3    4
4    5
dtype: int64
```

```
>>> df = pd.DataFrame(
...     {
...         "A": [0, 1, 2, 3, 4],
...         "B": [5, 6, 7, 8, 9],
...         "C": ["a", "b", "c", "d", "e"],
...     })
```

```

... )
>>> df.replace(0, 5)
   A  B  C
0  5  5  a
1  1  6  b
2  2  7  c
3  3  8  d
4  4  9  e

**List-like `to_replace`**

>>> df.replace([0, 1, 2, 3], 4)
   A  B  C
0  4  5  a
1  4  6  b
2  4  7  c
3  4  8  d
4  4  9  e

>>> df.replace([0, 1, 2, 3], [4, 3, 2, 1])
   A  B  C
0  4  5  a
1  3  6  b
2  2  7  c
3  1  8  d
4  4  9  e

**dict-like `to_replace`**

>>> df.replace({0: 10, 1: 100})
   A  B  C
0  10  5  a
1 100  6  b
2   2  7  c
3   3  8  d
4   4  9  e

>>> df.replace({"A": 0, "B": 5}, 100)
   A  B  C
0 100 100  a
1   1   6  b
2   2   7  c
3   3   8  d
4   4   9  e

>>> df.replace({"A": {0: 100, 4: 400}})
   A  B  C
0 100  5  a
1   1  6  b
2   2  7  c
3   3  8  d
4 400  9  e

**Regular expression `to_replace`**

>>> df = pd.DataFrame({"A": ["bat", "foo", "bait"], "B": ["abc", "bar", "xyz"]})
>>> df.replace(to_replace=r"^ba.$", value="new", regex=True)
   A  B
0  new abc
1  foo new
2  bait xyz

>>> df.replace({"A": r"^ba.$"}, {"A": "new"}, regex=True)
   A  B
0  new abc
1  foo bar
2  bait xyz

>>> df.replace(regex=r"^ba.$", value="new")
   A  B
0  new abc
1  foo new
2  bait xyz

>>> df.replace(regex={"^ba.$": "new", "foo": "xyz"})
   A  B
0  new abc

```

```

1 xyz new
2 bait xyz

>>> df.replace(regex=[r"^ba.$", "foo"], value="new")
   A  B
0  new abc
1  new new
2  bait xyz

```

Compare the behavior of `df.replace({'a': None})` and `df.replace('a', None)` to understand the peculiarities of the `to_replace` parameter:

```
>>> s = pd.Series([10, "a", "a", "b", "a"])
```

When one uses a dict as the `to_replace` value, it is like the value(s) in the dict are equal to the `value` parameter. `df.replace({'a': None})` is equivalent to `df.replace(to_replace={'a': None}, value=None)`:

```
>>> s.replace({'a': None})
0    10
1    None
2    None
3     b
4    None
dtype: object

```

If `None` is explicitly passed for `value`, it will be respected:

```
>>> s.replace("a", None)
0    10
1    None
2    None
3     b
4    None
dtype: object

```

When `regex=True`, `value` is not `None` and `to_replace` is a string, the replacement will be applied in all columns of the DataFrame.

```
>>> df = pd.DataFrame(
...     {
...         "A": [0, 1, 2, 3, 4],
...         "B": ["a", "b", "c", "d", "e"],
...         "C": ["f", "g", "h", "i", "j"],
...     }
... )

>>> df.replace(to_replace="^[a-g]", value="e", regex=True)
   A  B  C
0  0  e  e
1  1  e  e
2  2  e  h
3  3  e  i
4  4  e  j

```

If `value` is not `None` and `to_replace` is a dictionary, the dictionary keys will be the DataFrame columns that the replacement will be applied.

```
>>> df.replace(to_replace={"B": "^[a-c]", "C": "^[h-j]"}, value="e", regex=True)
   A  B  C
0  0  e  f
1  1  e  g
2  2  e  e
3  3  d  e
4  4  e  e

```

```

resample(
    self,
    rule,
    closed: Literal['right', 'left'] | None = None,
    label: Literal['right', 'left'] | None = None,
    convention: Literal['start', 'end', 's', 'e'] = 'start',
    on: Level | None = None,
    level: Level | None = None,
    origin: str | TimestampConvertibleTypes = 'start_day',

```

```
offset: TimedeltaConvertibleTypes | None = None,
group_keys: bool = False
) -> Resampler
Resample time-series data.
```

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (`DatetimeIndex`, `PeriodIndex`, or `TimedeltaIndex`), or the caller must pass the label of a datetime-like series/index to the `on`/`level` keyword parameter.

Parameters

rule : DateOffset, Timedelta or str
The offset string or object representing target conversion.

closed : {'right', 'left'}, default None
Which side of bin interval is closed. The default is 'left' for all frequency offsets except for 'ME', 'YE', 'QE', 'BME', 'BA', 'BQE', and 'W' which all have a default of 'right'.

label : {'right', 'left'}, default None
Which bin edge label to label bucket with. The default is 'left' for all frequency offsets except for 'ME', 'YE', 'QE', 'BME', 'BA', 'BQE', and 'W' which all have a default of 'right'.

convention : {'start', 'end', 's', 'e'}, default 'start'
For `PeriodIndex` only, controls whether to use the start or end of `rule`.

on : str, optional
For a DataFrame, column to use instead of index for resampling. Column must be datetime-like.

level : str or int, optional
For a MultiIndex, level (name or number) to use for resampling. `level` must be datetime-like.

origin : Timestamp or str, default 'start_day'
The timestamp on which to adjust the grouping. The timezone of origin must match the timezone of the index.
If string, must be Timestamp convertible or one of the following:

- 'epoch': `origin` is 1970-01-01
- 'start': `origin` is the first value of the timeseries
- 'start_day': `origin` is the first day at midnight of the timeseries
- 'end': `origin` is the last value of the timeseries
- 'end_day': `origin` is the ceiling midnight of the last day

.. note::

Only takes effect for Tick-frequencies (i.e. fixed frequencies like days, hours, and minutes, rather than months or quarters).

offset : Timedelta or str, default is None
An offset timedelta added to the origin.

group_keys : bool, default False
Whether to include the group keys in the result index when using `.apply()` on the resampled object.

.. versionchanged:: 2.0.0

`group_keys` now defaults to `False`.

Returns

pandas.api.typing.Resampler
:class: ~pandas.core.Resampler object.

See Also

Series.resample : Resample a Series.
DataFrame.resample : Resample a DataFrame.
groupby : Group Series/DataFrame by mapping, function, label, or list of labels.
asfreq : Reindex a Series/DataFrame with the given frequency without grouping.

Notes

See the `user guide`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#resampling>`__
for more.

To learn more about the offset strings, please see `this link`

https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#dateoffset-objects

Examples

Start by creating a series with 9 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=9, freq="min")
>>> series = pd.Series(range(9), index=index)
>>> series
2000-01-01 00:00:00    0
2000-01-01 00:01:00    1
2000-01-01 00:02:00    2
2000-01-01 00:03:00    3
2000-01-01 00:04:00    4
2000-01-01 00:05:00    5
2000-01-01 00:06:00    6
2000-01-01 00:07:00    7
2000-01-01 00:08:00    8
Freq: min, dtype: int64
```

Downsample the series into 3 minute bins and sum the values of the timestamps falling into a bin.

```
>>> series.resample("3min").sum()
2000-01-01 00:00:00    3
2000-01-01 00:03:00   12
2000-01-01 00:06:00   21
Freq: 3min, dtype: int64
```

Downsample the series into 3 minute bins as above, but label each bin using the right edge instead of the left. Please note that the value in the bucket used as the label is not included in the bucket, which it labels. For example, in the original series the bucket ``2000-01-01 00:03:00`` contains the value 3, but the summed value in the resampled bucket with the label ``2000-01-01 00:03:00`` does not include 3 (if it did, the summed value would be 6, not 3).

```
>>> series.resample("3min", label="right").sum()
2000-01-01 00:03:00    3
2000-01-01 00:06:00   12
2000-01-01 00:09:00   21
Freq: 3min, dtype: int64
```

To include this value close the right side of the bin interval, as shown below.

```
>>> series.resample("3min", label="right", closed="right").sum()
2000-01-01 00:00:00    0
2000-01-01 00:03:00    6
2000-01-01 00:06:00   15
2000-01-01 00:09:00   15
Freq: 3min, dtype: int64
```

Upsample the series into 30 second bins.

```
>>> series.resample("30s").asfreq()[0:5] # Select first 5 rows
2000-01-01 00:00:00    0.0
2000-01-01 00:00:30   NaN
2000-01-01 00:01:00    1.0
2000-01-01 00:01:30   NaN
2000-01-01 00:02:00    2.0
Freq: 30s, dtype: float64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``ffill`` method.

```
>>> series.resample("30s").ffill()[0:5]
2000-01-01 00:00:00    0
2000-01-01 00:00:30    0
2000-01-01 00:01:00    1
2000-01-01 00:01:30    1
2000-01-01 00:02:00    2
Freq: 30s, dtype: int64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``bfill`` method.

```
>>> series.resample("30s").bfill()[0:5]
2000-01-01 00:00:00    0
2000-01-01 00:00:30    1
2000-01-01 00:01:00    1
2000-01-01 00:01:30    2
2000-01-01 00:02:00    2
Freq: 30s, dtype: int64
```

Pass a custom function via ``apply``

```
>>> def custom_resampler(arraylike):
...     return np.sum(arraylike) + 5
>>> series.resample("3min").apply(custom_resampler)
2000-01-01 00:00:00     8
2000-01-01 00:03:00    17
2000-01-01 00:06:00    26
Freq: 3min, dtype: int64
```

For a Series with a PeriodIndex, the keyword `convention` can be used to control whether to use the start or end of `rule`.

Resample a year by quarter using 'start' `convention`. Values are assigned to the first quarter of the period.

```
>>> s = pd.Series(
...     [1, 2], index=pd.period_range("2012-01-01", freq="Y", periods=2)
... )
>>> s
2012    1
2013    2
Freq: Y-DEC, dtype: int64
>>> s.resample("Q", convention="start").asfreq()
2012Q1    1.0
2012Q2    NaN
2012Q3    NaN
2012Q4    NaN
2013Q1    2.0
2013Q2    NaN
2013Q3    NaN
2013Q4    NaN
Freq: Q-DEC, dtype: float64
```

Resample quarters by month using 'end' `convention`. Values are assigned to the last month of the period.

```
>>> q = pd.Series(
...     [1, 2, 3, 4], index=pd.period_range("2018-01-01", freq="Q", periods=4)
... )
>>> q
2018Q1    1
2018Q2    2
2018Q3    3
2018Q4    4
Freq: Q-DEC, dtype: int64
>>> q.resample("M", convention="end").asfreq()
2018-03    1.0
2018-04    NaN
2018-05    NaN
2018-06    2.0
2018-07    NaN
2018-08    NaN
2018-09    3.0
2018-10    NaN
2018-11    NaN
2018-12    4.0
Freq: M, dtype: float64
```

For DataFrame objects, the keyword `on` can be used to specify the column instead of the index for resampling.

```
>>> df = pd.DataFrame([10, 11, 9, 13, 14, 18, 17, 19], columns=["price"])
>>> df["volume"] = [50, 60, 40, 100, 50, 100, 40, 50]
>>> df["week_starting"] = pd.date_range("01/01/2018", periods=8, freq="W")
>>> df
   price  volume week_starting
0     10     50  2018-01-07
1     11     60  2018-01-14
```

```

2      9      40    2018-01-21
3     13     100    2018-01-28
4     14      50    2018-02-04
5     18     100    2018-02-11
6     17      40    2018-02-18
7     19      50    2018-02-25
>>> df.resample("ME", on="week_starting").mean()
           price  volume
week_starting
2018-01-31    10.75    62.5
2018-02-28    17.00    60.0

```

For a DataFrame with MultiIndex, the keyword `level` can be used to specify on which level the resampling needs to take place.

```

>>> days = pd.date_range("1/1/2000", periods=4, freq="D")
>>> df2 = pd.DataFrame(
...     [
...         [10, 50],
...         [11, 60],
...         [9, 40],
...         [13, 100],
...         [14, 50],
...         [18, 100],
...         [17, 40],
...         [19, 50],
...     ],
...     columns=["price", "volume"],
...     index=pd.MultiIndex.from_product([days, ["morning", "afternoon"]]),
... )
>>> df2
           price  volume
2000-01-01 morning      10      50
           afternoon    11      60
2000-01-02 morning       9      40
           afternoon    13     100
2000-01-03 morning      14      50
           afternoon    18     100
2000-01-04 morning      17      40
           afternoon    19      50
>>> df2.resample("D", level=0).sum()
           price  volume
2000-01-01     21     110
2000-01-02     22     140
2000-01-03     32     150
2000-01-04     36      90

```

If you want to adjust the start of the bins based on a fixed timestamp:

```

>>> start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
>>> rng = pd.date_range(start, end, freq="7min")
>>> ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
>>> ts
2000-10-01 23:30:00    0
2000-10-01 23:37:00    3
2000-10-01 23:44:00    6
2000-10-01 23:51:00    9
2000-10-01 23:58:00   12
2000-10-02 00:05:00   15
2000-10-02 00:12:00   18
2000-10-02 00:19:00   21
2000-10-02 00:26:00   24
Freq: 7min, dtype: int64

>>> ts.resample("17min").sum()
2000-10-01 23:14:00    0
2000-10-01 23:31:00    9
2000-10-01 23:48:00   21
2000-10-02 00:05:00   54
2000-10-02 00:22:00   24
Freq: 17min, dtype: int64

>>> ts.resample("17min", origin="epoch").sum()
2000-10-01 23:18:00    0
2000-10-01 23:35:00   18
2000-10-01 23:52:00   27
2000-10-02 00:09:00   39

```

```
2000-10-02 00:26:00    24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", origin="2000-01-01").sum()
2000-10-01 23:24:00     3
2000-10-01 23:41:00    15
2000-10-01 23:58:00    45
2000-10-02 00:15:00    45
Freq: 17min, dtype: int64
```

If you want to adjust the start of the bins with an `offset` `Timedelta`, the two following lines are equivalent:

```
>>> ts.resample("17min", origin="start").sum()
2000-10-01 23:30:00     9
2000-10-01 23:47:00    21
2000-10-02 00:04:00    54
2000-10-02 00:21:00    24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", offset="23h30min").sum()
2000-10-01 23:30:00     9
2000-10-01 23:47:00    21
2000-10-02 00:04:00    54
2000-10-02 00:21:00    24
Freq: 17min, dtype: int64
```

If you want to take the largest Timestamp as the end of the bins:

```
>>> ts.resample("17min", origin="end").sum()
2000-10-01 23:35:00     0
2000-10-01 23:52:00    18
2000-10-02 00:09:00    27
2000-10-02 00:26:00    63
Freq: 17min, dtype: int64
```

In contrast with the `start_day`, you can use `end_day` to take the ceiling midnight of the largest Timestamp as the end of the bins and drop the bins not containing data:

```
>>> ts.resample("17min", origin="end_day").sum()
2000-10-01 23:38:00     3
2000-10-01 23:55:00    15
2000-10-02 00:12:00    45
2000-10-02 00:29:00    45
Freq: 17min, dtype: int64
```

```
rolling(
    self,
    window: int | dt.timedelta | str | BaseOffset | BaseIndexer,
    min_periods: int | None = None,
    center: bool = False,
    win_type: str | None = None,
    on: str | None = None,
    closed: IntervalClosedType | None = None,
    step: int | None = None,
    method: str = 'single'
) -> Window | Rolling
```

Provide rolling window calculations.

Parameters

window : int, timedelta, str, offset, or BaseIndexer subclass
Interval of the moving window.

If an integer, the delta between the start and end of each window.
The number of points in the window depends on the `closed` argument.

If a timedelta, str, or offset, the time period of each window. Each window will be a variable sized based on the observations included in the time-period. This is only valid for datetimelike indexes.
To learn more about the offsets & frequency strings, please see [:ref:`this link<timeseries.offset\\_aliases>`](#).

If a BaseIndexer subclass, the window boundaries based on the defined `get_window_bounds` method. Additional rolling keyword arguments, namely `min_periods`, `center`, `closed` and

``step`` will be passed to ``get\_window\_bounds``.

min_periods : int, default None
 Minimum number of observations in window required to have a value; otherwise, result is ``np.nan``.

For a window that is specified by an offset, ``min\_periods`` will default to 1.

For a window that is specified by an integer, ``min\_periods`` will default to the size of the window.

center : bool, default False
 If False, set the window labels as the right edge of the window index.
 If True, set the window labels as the center of the window index.

win_type : str, default None
 If ``None``, all points are evenly weighted.
 If a string, it must be a valid `scipy.signal` window function
<https://docs.scipy.org/doc/scipy/reference/signal.windows.html#module-scipy.signal>.

Certain Scipy window types require additional parameters to be passed in the aggregation function. The additional parameters must match the keywords specified in the Scipy window type method signature.

on : str, optional
 For a DataFrame, a column label or Index level on which to calculate the rolling window, rather than the DataFrame's index.
 Provided integer column is ignored and excluded from result since an integer index is not used to calculate the rolling window.

closed : str, default None
 Determines the inclusivity of points in the window
 If ``'right'``, uses the window (first, last] meaning the last point is included in the calculations.
 If ``'left'``, uses the window [first, last) meaning the first point is included in the calculations.
 If ``'both'``, uses the window [first, last] meaning all points in the window are included in the calculations.
 If ``'neither'``, uses the window (first, last) meaning the first and last points in the window are excluded from calculations.
 () and [] are referencing open and closed set notation respectively.
 Default ``None`` (``'right'``).

step : int, default None
 Evaluate the window at every ``step`` result, equivalent to slicing as ``[::step]``. ``window`` must be an integer. Using a step argument other than None or 1 will produce a result with a different shape than the input.

method : str {'single', 'table'}, default 'single'
 Execute the rolling operation per single column or row (``'single'``) or over the entire object (``'table'``).
 This argument is only implemented when specifying ``engine='numba'`` in the method call.

Returns

 pandas.api.typing.Window or pandas.api.typing.Rolling
 An instance of Window is returned if ``win\_type`` is passed. Otherwise, an instance of Rolling is returned.

See Also

 expanding : Provides expanding transformations.
 ewm : Provides exponential weighted functions.

Notes

See :ref:`Windowing Operations <window.generic>` for further usage details and examples.

Examples

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
```

```
>>> df
   B
0  0.0
1  1.0
2  2.0
3  NaN
4  4.0
```

****window****

Rolling sum with a window length of 2 observations.

```
>>> df.rolling(2).sum()
```

```
   B
0  NaN
1  1.0
2  3.0
3  NaN
4  NaN
```

Rolling sum with a window span of 2 seconds.

```
>>> df_time = pd.DataFrame(
...     {"B": [0, 1, 2, np.nan, 4]},
...     index=[
...         pd.Timestamp("20130101 09:00:00"),
...         pd.Timestamp("20130101 09:00:02"),
...         pd.Timestamp("20130101 09:00:03"),
...         pd.Timestamp("20130101 09:00:05"),
...         pd.Timestamp("20130101 09:00:06"),
...     ],
... )
```

```
>>> df_time
```

```
                B
2013-01-01 09:00:00  0.0
2013-01-01 09:00:02  1.0
2013-01-01 09:00:03  2.0
2013-01-01 09:00:05  NaN
2013-01-01 09:00:06  4.0
```

```
>>> df_time.rolling("2s").sum()
```

```
                B
2013-01-01 09:00:00  0.0
2013-01-01 09:00:02  1.0
2013-01-01 09:00:03  3.0
2013-01-01 09:00:05  NaN
2013-01-01 09:00:06  4.0
```

Rolling sum with forward looking windows with 2 observations.

```
>>> indexer = pd.api.indexers.FixedForwardWindowIndexer(window_size=2)
>>> df.rolling(window=indexer, min_periods=1).sum()
```

```
   B
0  1.0
1  3.0
2  2.0
3  4.0
4  4.0
```

****min_periods****

Rolling sum with a window length of 2 observations, but only needs a minimum of 1 observation to calculate a value.

```
>>> df.rolling(2, min_periods=1).sum()
```

```
   B
0  0.0
```

```

1  1.0
2  3.0
3  2.0
4  4.0

**center**

Rolling sum with the result assigned to the center of the window index.

>>> df.rolling(3, min_periods=1, center=True).sum()
      B
0  1.0
1  3.0
2  3.0
3  6.0
4  4.0

>>> df.rolling(3, min_periods=1, center=False).sum()
      B
0  0.0
1  1.0
2  3.0
3  3.0
4  6.0

**step**

Rolling sum with a window length of 2 observations, minimum of 1 observation to
calculate a value, and a step of 2.

>>> df.rolling(2, min_periods=1, step=2).sum()
      B
0  0.0
2  3.0
4  4.0

**win_type**

Rolling sum with a window length of 2, using the Scipy ``'gaussian'``
window type. ``std`` is required in the aggregation function.

>>> df.rolling(2, win_type="gaussian").sum(std=3)
      B
0      NaN
1  0.986207
2  2.958621
3      NaN
4      NaN

**on**

Rolling sum with a window length of 2 days.

>>> df = pd.DataFrame(
...     {
...         "A": [
...             pd.to_datetime("2020-01-01"),
...             pd.to_datetime("2020-01-01"),
...             pd.to_datetime("2020-01-02"),
...         ],
...         "B": [1, 2, 3],
...     },
...     index=pd.date_range("2020", periods=3),
... )

>>> df
      A  B
2020-01-01 2020-01-01  1
2020-01-02 2020-01-01  2
2020-01-03 2020-01-02  3

>>> df.rolling("2D", on="A").sum()
      A  B
2020-01-01 2020-01-01  1.0
2020-01-02 2020-01-01  3.0
2020-01-03 2020-01-02  6.0

```

```

sample(
    self,
    n: int | None = None,
    frac: float | None = None,
    replace: bool = False,
    weights=None,
    random_state: RandomState | None = None,
    axis: Axis | None = None,
    ignore_index: bool = False
) -> Self
    Return a random sample of items from an axis of object.

    You can use `random_state` for reproducibility.

Parameters
-----
n : int, optional
    Number of items from axis to return. Cannot be used with `frac`.
    Default = 1 if `frac` = None.
frac : float, optional
    Fraction of axis items to return. Cannot be used with `n`.
replace : bool, default False
    Allow or disallow sampling of the same row more than once.
weights : str or ndarray-like, optional
    Default ``None`` results in equal probability weighting.
    If passed a Series, will align with target object on index. Index
    values in weights not found in sampled object will be ignored and
    index values in sampled object not in weights will be assigned
    weights of zero.
    If called on a DataFrame, will accept the name of a column
    when axis = 0.
    Unless weights are a Series, weights must be same length as axis
    being sampled.
    If weights do not sum to 1, they will be normalized to sum to 1.
    Missing values in the weights column will be treated as zero.
    Infinite values not allowed.
    When replace = False will not allow ``(n * max(weights) / sum(weights)) > 1``
    in order to avoid biased results. See the Notes below for more details.
random_state : int, array-like, BitGenerator, np.random.RandomState, np.random.Generator
    If int, array-like, or BitGenerator, seed for random number generator.
    If np.random.RandomState or np.random.Generator, use as given.
    Default ``None`` results in sampling with the current state of np.random.
axis : {0 or 'index', 1 or 'columns', None}, default None
    Axis to sample. Accepts axis number or name. Default is stat axis
    for given data type. For `Series` this parameter is unused and defaults to `None`.
ignore_index : bool, default False
    If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns
-----
Series or DataFrame
    A new object of same type as caller containing `n` items randomly
    sampled from the caller object.

See Also
-----
DataFrameGroupBy.sample: Generates random samples from each group of a
    DataFrame object.
SeriesGroupBy.sample: Generates random samples from each group of a
    Series object.
numpy.random.choice: Generates a random sample from a given 1-D numpy
    array.

Notes
-----
If `frac` > 1, `replacement` should be set to `True`.

When replace = False will not allow ``(n * max(weights) / sum(weights)) > 1``,
since that would cause results to be biased. E.g. sampling 2 items without replacement
with weights [100, 1, 1] would yield two last items in 1/2 of cases, instead of 1/102.
This is similar to specifying `n=4` without replacement on a Series with 3 elements.

Examples
-----
>>> df = pd.DataFrame(
...     {
...         "num_legs": [2, 4, 8, 0],

```

```
...         "num_wings": [2, 0, 0, 0],
...         "num_specimen_seen": [10, 2, 1, 8],
...     },
...     index=["falcon", "dog", "spider", "fish"],
... )
>>> df
   num_legs  num_wings  num_specimen_seen
falcon      2         2                 10
dog          4         0                  2
spider       8         0                  1
fish         0         0                  8
```

Extract 3 random elements from the ``Series`` ``df['num\_legs']``:
Note that we use ``random\_state`` to ensure the reproducibility of the examples.

```
>>> df["num_legs"].sample(n=3, random_state=1)
fish      0
spider    8
falcon    2
Name: num_legs, dtype: int64
```

A random 50% sample of the ``DataFrame`` with replacement:

```
>>> df.sample(frac=0.5, replace=True, random_state=1)
   num_legs  num_wings  num_specimen_seen
dog          4         0                  2
fish         0         0                  8
```

An upsample sample of the ``DataFrame`` with replacement:
Note that ``replace`` parameter has to be ``True`` for ``frac`` parameter > 1.

```
>>> df.sample(frac=2, replace=True, random_state=1)
   num_legs  num_wings  num_specimen_seen
dog          4         0                  2
fish         0         0                  8
falcon        2         2                 10
falcon        2         2                 10
fish         0         0                  8
dog          4         0                  2
fish         0         0                  8
dog          4         0                  2
```

Using a DataFrame column as weights. Rows with larger value in the ``num\_specimen\_seen`` column are more likely to be sampled.

```
>>> df.sample(n=2, weights="num_specimen_seen", random_state=1)
   num_legs  num_wings  num_specimen_seen
falcon        2         2                 10
fish         0         0                  8
```

```
set_flags(
    self,
    *,
    copy: bool | lib.NoDefault = <no_default>,
    allows_duplicate_labels: bool | None = None
) -> Self
```

Return a new object with updated flags.

This method creates a shallow copy of the original object, preserving its underlying data while modifying its global flags. In particular, it allows you to update properties such as whether duplicate labels are permitted. This behavior is especially useful in method chains, where one wishes to adjust DataFrame or Series characteristics without altering the original object.

Parameters

copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`

https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`\_\_`
for more details.

`allows_duplicate_labels` : bool, optional
Whether the returned object allows duplicate labels.

Returns

Series or DataFrame
The same type as the caller.

See Also

`DataFrame.attrs` : Global metadata applying to this dataset.
`DataFrame.flags` : Global flags applying to this object.

Notes

This method returns a new object that's a view on the same data as the input. Mutating the input or the output values will be reflected in the other.

This method is intended to be used in method chains.

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in `:attr: `DataFrame.attrs``.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags.allows_duplicate_labels
True
>>> df2 = df.set_flags(allows_duplicate_labels=False)
>>> df2.flags.allows_duplicate_labels
False
```

`squeeze(self, axis: Axis | None = None) -> Scalar | Series | DataFrame`
Squeeze 1 dimensional axis objects into scalars.

Series or DataFrames with a single element are squeezed to a scalar. DataFrames with a single column or a single row are squeezed to a Series. Otherwise the object is unchanged.

This method is most useful when you don't know if your object is a Series or DataFrame, but you do know it has just a single column. In that case you can safely call ``squeeze`` to ensure you have a Series.

Parameters

`axis` : {0 or 'index', 1 or 'columns', None}, default None
A specific axis to squeeze. By default, all length-1 axes are squeezed. For ``Series`` this parameter is unused and defaults to ``None``.

Returns

DataFrame, Series, or scalar
The projection after squeezing ``axis`` or all the axes.

See Also

`Series.iloc` : Integer-location based indexing for selecting scalars.
`DataFrame.iloc` : Integer-location based indexing for selecting Series.
`Series.to_frame` : Inverse of `DataFrame.squeeze` for a single-column DataFrame.

Examples

```
>>> primes = pd.Series([2, 3, 5, 7])
```

Slicing might produce a Series with a single value:

```
>>> even_primes = primes[primes % 2 == 0]
>>> even_primes
0    2
dtype: int64
```

```
>>> even_primes.squeeze()
np.int64(2)
```

Squeezing objects with more than one value in every axis does nothing:

```
>>> odd_primes = primes[primes % 2 == 1]
>>> odd_primes
1    3
2    5
3    7
dtype: int64
```

```
>>> odd_primes.squeeze()
1    3
2    5
3    7
dtype: int64
```

Squeezing is even more effective when used with DataFrames.

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["a", "b"])
>>> df
   a  b
0  1  2
1  3  4
```

Slicing a single column will produce a DataFrame with the columns having only one value:

```
>>> df_a = df[["a"]]
>>> df_a
   a
0  1
1  3
```

So the columns can be squeezed down, resulting in a Series:

```
>>> df_a.squeeze("columns")
0    1
1    3
Name: a, dtype: int64
```

Slicing a single row from a single column will produce a single scalar DataFrame:

```
>>> df_0a = df.loc[df.index < 1, ["a"]]
>>> df_0a
   a
0  1
```

Squeezing the rows produces a single scalar Series:

```
>>> df_0a.squeeze("rows")
a    1
Name: 0, dtype: int64
```

Squeezing all axes will project directly into a scalar:

```
>>> df_0a.squeeze()
np.int64(1)
```

```
tail(self, n: int = 5) -> Self
Return the last `n` rows.
```

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to `df[|n|:]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

n : int, default 5

Number of rows to select.

Returns

type of caller
The last `n` rows of the caller object.

See Also

DataFrame.head : The first `n` rows of the caller object.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "animal": [
...             "alligator",
...             "bee",
...             "falcon",
...             "lion",
...             "monkey",
...             "parrot",
...             "shark",
...             "whale",
...             "zebra",
...         ]
...     }
... )
>>> df
   animal
0  alligator
1     bee
2   falcon
3     lion
4   monkey
5   parrot
6    shark
7    whale
8    zebra
```

Viewing the last 5 lines

```
>>> df.tail()
   animal
4  monkey
5  parrot
6  shark
7  whale
8  zebra
```

Viewing the last `n` lines (three in this case)

```
>>> df.tail(3)
   animal
6  shark
7  whale
8  zebra
```

For negative values of `n`

```
>>> df.tail(-3)
   animal
3    lion
4  monkey
5  parrot
6  shark
7  whale
8  zebra
```

take(self, indices, axis: Axis = 0, **kwargs) -> Self
Return the elements in the given *positional* indices along an axis.

This means that we are not indexing according to actual values in the index attribute of the object. We are indexing according to the actual position of the element in the object.

Parameters

indices : array-like
An array of ints indicating which positions to take.
axis : {0 or 'index', 1 or 'columns'}, default 0
The axis on which to select elements. ``0`` means that we are selecting rows, ``1`` means that we are selecting columns.
For `Series` this parameter is unused and defaults to 0.
**kwargs
For compatibility with :meth:`numpy.take`. Has no effect on the output.

Returns

same type as caller
An array-like containing the elements taken from the object.

See Also

DataFrame.loc : Select a subset of a DataFrame by labels.
DataFrame.iloc : Select a subset of a DataFrame by positions.
numpy.take : Take elements from an array along an axis.

Examples

>>> df = pd.DataFrame(
... [
... ("falcon", "bird", 389.0),
... ("parrot", "bird", 24.0),
... ("lion", "mammal", 80.5),
... ("monkey", "mammal", np.nan),
...],
... columns=["name", "class", "max_speed"],
... index=[0, 2, 3, 1],
...)
>>> df

	name	class	max_speed
0	falcon	bird	389.0
2	parrot	bird	24.0
3	lion	mammal	80.5
1	monkey	mammal	NaN

Take elements at positions 0 and 3 along the axis 0 (default).

Note how the actual indices selected (0 and 1) do not correspond to our selected indices 0 and 3. That's because we are selecting the 0th and 3rd rows, not rows whose indices equal 0 and 3.

```
>>> df.take([0, 3])
   name  class  max_speed
0  falcon   bird    389.0
1  monkey  mammal     NaN
```

Take elements at indices 1 and 2 along the axis 1 (column selection).

```
>>> df.take([1, 2], axis=1)
   class  max_speed
0   bird    389.0
2   bird    24.0
3  mammal    80.5
1  mammal     NaN
```

We may take elements using negative integers for positive indices, starting from the end of the object, just like with Python lists.

```
>>> df.take([-1, -2])
   name  class  max_speed
1  monkey  mammal     NaN
3   lion  mammal    80.5
```

to_clipboard(self, *, excel: bool = True, sep: str | None = None, **kwargs) -> None
Copy object to the system clipboard.

Write a text representation of object to the system clipboard.
This can be pasted into Excel, for example.

Parameters

excel : bool, default True
Produce output in a csv format for easy pasting into excel.

- True, use the provided separator for csv pasting.
- False, write a string representation of the object to the clipboard.

sep : str, default ``\t``
Field delimiter.

**kwargs
These parameters will be passed to DataFrame.to_csv.

See Also

DataFrame.to_csv : Write a DataFrame to a comma-separated values (csv) file.

read_clipboard : Read text from clipboard and pass to read_csv.

Notes

Requirements for your platform.

- Linux : `xclip`, or `xsel` (with `PyQt4` modules)
- Windows : none
- macOS : none

This method uses the processes developed for the package `pyperclip`. A solution to render any output string format is given in the examples.

Examples

Copy the contents of a DataFrame to the clipboard.

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])
```

```
>>> df.to_clipboard(sep=",") # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # ,A,B,C
... # 0,1,2,3
... # 1,4,5,6
```

We can omit the index by passing the keyword `index` and setting it to false.

```
>>> df.to_clipboard(sep=",", index=False) # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # A,B,C
... # 1,2,3
... # 4,5,6
```

Using the original `pyperclip` package for any string output format.

```
.. code-block:: python
```

```
import pyperclip

html = df.style.to_html()
pyperclip.copy(html)
```

```
to_csv(
    self,
    path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
    *,
    sep: str = ',',
    na_rep: str = '',
    float_format: str | Callable | None = None,
    columns: Sequence[Hashable] | None = None,
    header: bool | list[str] = True,
    index: bool = True,
    index_label: IndexLabel | None = None,
    mode: str = 'w',
    encoding: str | None = None,
    compression: CompressionOptions = 'infer',
    quoting: int | None = None,
    quotechar: str = '"',
    lineterminator: str | None = None,
    chunksize: int | None = None,
    date_format: str | None = None,
```

```

doublequote: bool = True,
escapechar: str | None = None,
decimal: str = '.',
errors: OpenFileErrors = 'strict',
storage_options: StorageOptions | None = None
) -> str | None
    Write object to a comma-separated values (csv) file.

Parameters
-----
path_or_buf : str, path object, file-like object, or None, default None
    String, path object (implementing os.PathLike[str]), or file-like
    object implementing a write() function. If None, the result is
    returned as a string. If a non-binary file object is passed, it should
    be opened with `newline=''`, disabling universal newlines. If a binary
    file object is passed, `mode` might need to contain a `b`.
sep : str, default ','
    String of length 1. Field delimiter for the output file.
na_rep : str, default ''
    Missing data representation.
float_format : str, Callable, default None
    Format string for floating point numbers. If a Callable is given, it takes
    precedence over other numeric formatting parameters, like decimal.
columns : sequence, optional
    Columns to write.
header : bool or list of str, default True
    Write out the column names. If a list of strings is given it is
    assumed to be aliases for the column names.
index : bool, default True
    Write row names (index).
index_label : str or sequence, or False, default None
    Column label for index column(s) if desired. If None is given, and
    `header` and `index` are True, then the index names are used. A
    sequence should be given if the object uses MultiIndex. If
    False do not print fields for index names. Use index_label=False
    for easier importing in R.
mode : {'w', 'x', 'a'}, default 'w'
    Forwarded to either `open(mode=)` or `fsspec.open(mode=)` to control
    the file opening. Typical values include:

    - 'w', truncate the file first.
    - 'x', exclusive creation, failing if the file already exists.
    - 'a', append to the end of file if it exists.

encoding : str, optional
    A string representing the encoding to use in the output file,
    defaults to 'utf-8'. `encoding` is not supported if `path_or_buf`
    is a non-binary file object.

compression : str or dict, default 'infer'
    For on-the-fly compression of the output data. If 'infer' and
    'path_or_buf' is path-like, then detect compression from the following
    extensions: '.gz',
    '.bz2', '.zip', '.xz', '.zstd', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
    (otherwise no compression).
    Set to `None` for no compression.
    Can also be a dict with key `method` set to one of
    {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
    other key-value pairs are forwarded to
    `zipfile.ZipFile`, `gzip.GzipFile`,
    `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or
    `tarfile.TarFile`, respectively.
    As an example, the following could be passed for faster compression and
    to create a reproducible gzip archive:
    `compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}`.

    May be a dict with key 'method' as compression mode
    and other entries as additional compression options if
    compression mode is 'zip'.

    Passing compression options as keys in dict is
    supported for compression modes 'gzip', 'bz2', 'zstd', and 'zip'.
quoting : optional constant from csv module
    Defaults to csv.QUOTE_MINIMAL. If you have set a `float_format`
    then floats are converted to strings and thus csv.QUOTE_NONNUMERIC
    will treat them as non-numeric.
quotechar : str, default '"'

```

String of length 1. Character used to quote fields.

lineterminator : str, optional
The newline character or character sequence to use in the output file. Defaults to `os.linesep`, which depends on the OS in which this method is called (`\n` for linux, `\r\n` for Windows, i.e.).

chunksize : int or None
Rows to write at a time.

date_format : str, default None
Format string for datetime objects.

doublequote : bool, default True
Control quoting of `quotechar` inside a field.

escapechar : str, default None
String of length 1. Character used to escape `sep` and `quotechar` when appropriate.

decimal : str, default '.'
Character recognized as decimal separator. E.g. use ',' for European data.

errors : str, default 'strict'
Specifies how encoding and decoding errors are to be handled. See the errors argument for :func:`open` for a full list of options.

storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

Returns

None or str
If path_or_buf is None, returns the resulting csv format as a string. Otherwise returns None.

See Also

read_csv : Load a CSV file into a DataFrame.
to_excel : Write DataFrame to an Excel file.

Examples

Create 'out.csv' containing 'df' without indices

```
>>> df = pd.DataFrame(
...     [ ["Raphael", "red", "sai"], ["Donatello", "purple", "bo staff"] ],
...     columns=["name", "mask", "weapon"],
... )
>>> df.to_csv("out.csv", index=False) # doctest: +SKIP
```

Create 'out.zip' containing 'out.csv'

```
>>> df.to_csv(index=False)
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
>>> compression_opts = dict(
...     method="zip", archive_name="out.csv"
... ) # doctest: +SKIP
>>> df.to_csv(
...     "out.zip", index=False, compression=compression_opts
... ) # doctest: +SKIP
```

To write a csv file to a new folder or nested folder you will first need to create it using either Pathlib or os:

```
>>> from pathlib import Path # doctest: +SKIP
>>> filepath = Path("folder/subfolder/out.csv") # doctest: +SKIP
>>> filepath.parent.mkdir(parents=True, exist_ok=True) # doctest: +SKIP
>>> df.to_csv(filepath) # doctest: +SKIP
```

```
>>> import os # doctest: +SKIP
>>> os.makedirs("folder/subfolder", exist_ok=True) # doctest: +SKIP
>>> df.to_csv("folder/subfolder/out.csv") # doctest: +SKIP
```

Format floats to two decimal places:

```
>>> df.to_csv("out1.csv", float_format="%.2f") # doctest: +SKIP
```

Format floats using scientific notation:

```
>>> df.to_csv("out2.csv", float_format="{:.2e}").format) # doctest: +SKIP
```

```
to_excel(
    self,
    excel_writer: FilePath | WriteExcelBuffer | ExcelWriter,
    *,
    sheet_name: str = 'Sheet1',
    na_rep: str = '',
    float_format: str | None = None,
    columns: Sequence[Hashable] | None = None,
    header: Sequence[Hashable] | bool = True,
    index: bool = True,
    index_label: IndexLabel | None = None,
    startrow: int = 0,
    startcol: int = 0,
    engine: Literal['openpyxl', 'xlsxwriter'] | None = None,
    merge_cells: bool = True,
    inf_rep: str = 'inf',
    freeze_panes: tuple[int, int] | None = None,
    storage_options: StorageOptions | None = None,
    engine_kwargs: dict[str, Any] | None = None,
    autofilter: bool = False
) -> None
Write object to an Excel sheet.
```

To write a single object to an Excel .xlsx file it is only necessary to specify a target file name. To write to multiple sheets it is necessary to create an `ExcelWriter` object with a target file name, and specify a sheet in the file to write to.

Multiple sheets may be written to by specifying unique `sheet_name`. With all data written to the file it is necessary to save the changes. Note that creating an `ExcelWriter` object with a file name that already exists will overwrite the existing file because the default mode is write.

Parameters

excel_writer : path-like, file-like, or `ExcelWriter` object
File path or existing `ExcelWriter`.

sheet_name : str, default 'Sheet1'
Name of sheet which will contain `DataFrame`.

na_rep : str, default ''
Missing data representation.

float_format : str, optional
Format string for floating point numbers. For example
`float_format="%.2f"` will format 0.1234 to 0.12.

columns : sequence or list of str, optional
Columns to write.

header : bool or list of str, default True
Write out the column names. If a list of string is given it is assumed to be aliases for the column names.

index : bool, default True
Write row names (index).

index_label : str or sequence, optional
Column label for index column(s) if desired. If not specified, and `header` and `index` are True, then the index names are used. A sequence should be given if the `DataFrame` uses `MultiIndex`.

startrow : int, default 0
Upper left cell row to dump data frame.

startcol : int, default 0
Upper left cell column to dump data frame.

engine : str, optional
Write engine to use, 'openpyxl' or 'xlsxwriter'. You can also set this via the options `io.excel.xlsx.writer` or `io.excel.xlsm.writer`.

merge_cells : bool or 'columns', default False
If True, write `MultiIndex` index and columns as merged cells.
If 'columns', merge `MultiIndex` column cells only.

inf_rep : str, default 'inf'
Representation for infinity (there is no native representation for infinity in Excel).

freeze_panes : tuple of int (length 2), optional

Specifies the one-based bottommost row and rightmost column that is to be frozen.

`storage_options` : dict, optional
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with `s3://`, and `gcs://`) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`engine_kwargs` : dict, optional
 Arbitrary keyword arguments passed to excel engine.

`autofilter` : bool, default False
 If True, add automatic filters to all columns.

See Also

`to_csv` : Write DataFrame to a comma-separated values (csv) file.
`ExcelWriter` : Class for writing DataFrame objects into excel sheets.
`read_excel` : Read an Excel file into a pandas DataFrame.
`read_csv` : Read a comma-separated values (csv) file into DataFrame.
`io.formats.style.Styler.to_excel` : Add styles to Excel sheet.

Notes

For compatibility with `DataFrame.to_csv`, `to_excel` serializes lists and dicts to strings before writing.

Once a workbook has been saved it is not possible to write further data without rewriting the whole workbook.

pandas will check the number of rows, columns, and cell character count does not exceed Excel's limitations. All other limitations must be checked by the user.

Examples

Create, write to and save a workbook:

```
>>> df1 = pd.DataFrame(
...     [ ["a", "b"], ["c", "d"] ],
...     index=["row 1", "row 2"],
...     columns=["col 1", "col 2"],
... )
>>> df1.to_excel("output.xlsx") # doctest: +SKIP
```

To specify the sheet name:

```
>>> df1.to_excel("output.xlsx", sheet_name="Sheet_name_1") # doctest: +SKIP
```

If you wish to write to more than one sheet in the workbook, it is necessary to specify an `ExcelWriter` object:

```
>>> df2 = df1.copy()
>>> with pd.ExcelWriter("output.xlsx") as writer: # doctest: +SKIP
...     df1.to_excel(writer, sheet_name="Sheet_name_1")
...     df2.to_excel(writer, sheet_name="Sheet_name_2")
```

`ExcelWriter` can also be used to append to an existing Excel file:

```
>>> with pd.ExcelWriter("output.xlsx", mode="a") as writer: # doctest: +SKIP
...     df1.to_excel(writer, sheet_name="Sheet_name_3")
```

To set the library that is used to write the Excel file, you can pass the `engine` keyword (the default engine is automatically chosen depending on the file extension):

```
>>> df1.to_excel("output1.xlsx", engine="xlsxwriter") # doctest: +SKIP
```

```
to_hdf(
    self,
    path_or_buf: FilePath | HDFStore,
    *,
    key: str,
```

```

mode: Literal['a', 'w', 'r+'] = 'a',
complevel: int | None = None,
complib: Literal['zlib', 'lzo', 'bzip2', 'blosc'] | None = None,
append: bool = False,
format: Literal['fixed', 'table'] | None = None,
index: bool = True,
min_itemsize: int | dict[str, int] | None = None,
nan_rep=None,
dropna: bool | None = None,
data_columns: Literal[True] | list[str] | None = None,
errors: OpenFileErrors = 'strict',
encoding: str = 'UTF-8'
) -> None
Write the contained data to an HDF5 file using HDFStore.

Hierarchical Data Format (HDF) is self-describing, allowing an
application to interpret the structure and contents of a file with
no outside information. One HDF file can hold a mix of related objects
which can be accessed as a group or as individual objects.

In order to add another DataFrame or Series to an existing HDF file
please use append mode and a different a key.

.. warning::

    One can store a subclass of ``DataFrame`` or ``Series`` to HDF5,
    but the type of the subclass is lost upon storing.

For more information see the :ref:`user guide <io.hdf5>`.

Parameters
-----
path_or_buf : str or pandas.HDFStore
    File path or HDFStore object.
key : str
    Identifier for the group in the store.
mode : {'a', 'w', 'r+'}, default 'a'
    Mode to open file:

    - 'w': write, a new file is created (an existing file with
      the same name would be deleted).
    - 'a': append, an existing file is opened for reading and
      writing, and if the file does not exist it is created.
    - 'r+': similar to 'a', but the file must already exist.
complevel : {0-9}, default None
    Specifies a compression level for data.
    A value of 0 or None disables compression.
complib : {'zlib', 'lzo', 'bzip2', 'blosc'}, default 'zlib'
    Specifies the compression library to be used.
    These additional compressors for Blosc are supported
    (default if no compressor specified: 'blosc:blosclz'):
    {'blosc:blosclz', 'blosc:lz4', 'blosc:lz4hc', 'blosc:snappy',
    'blosc:zlib', 'blosc:zstd'}.
    Specifying a compression library which is not available issues
    a ValueError.
append : bool, default False
    For Table formats, append the input data to the existing.
format : {'fixed', 'table', None}, default 'fixed'
    Possible values:

    - 'fixed': Fixed format. Fast writing/reading. Not-appendable,
      nor searchable.
    - 'table': Table format. Write as a PyTables Table structure
      which may perform worse but allow more flexible operations
      like searching / selecting subsets of the data.
    - If None, pd.get_option('io.hdf.default_format') is checked,
      followed by fallback to "fixed".
index : bool, default True
    Write DataFrame index as a column.
min_itemsize : dict or int, optional
    Map column names to minimum string sizes for columns.
nan_rep : Any, optional
    How to represent null values as str.
    Not allowed with append=True.
dropna : bool, default False, optional
    Remove missing values.
data_columns : list of columns or True, optional

```

List of columns to create as indexed data columns for on-disk queries, or True to use all columns. By default only the axes of the object are indexed. See `:ref:`Query via data columns<io.hdf5-query-data-columns>``. for more information.

Applicable only to `format='table'`.

`errors` : str, default 'strict'

Specifies how encoding and decoding errors are to be handled. See the errors argument for `:func:`open`` for a full list of options.

`encoding` : str, default "UTF-8"

Set character encoding.

See Also

`read_hdf` : Read from HDF file.

`DataFrame.to_orc` : Write a DataFrame to the binary orc format.

`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.

`DataFrame.to_sql` : Write to a SQL table.

`DataFrame.to_feather` : Write out feather-format for DataFrames.

`DataFrame.to_csv` : Write out to a csv file.

Examples

```
>>> df = pd.DataFrame(
...     {"A": [1, 2, 3], "B": [4, 5, 6]}, index=["a", "b", "c"]
... ) # doctest: +SKIP
>>> df.to_hdf("data.h5", key="df", mode="w") # doctest: +SKIP
```

We can add another object to the same file:

```
>>> s = pd.Series([1, 2, 3, 4]) # doctest: +SKIP
>>> s.to_hdf("data.h5", key="s") # doctest: +SKIP
```

Reading from HDF file:

```
>>> pd.read_hdf("data.h5", "df") # doctest: +SKIP
A B
a 1 4
b 2 5
c 3 6
>>> pd.read_hdf("data.h5", "s") # doctest: +SKIP
0    1
1    2
2    3
3    4
dtype: int64
```

```
to_json(
    self,
    path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
    *,
    orient: Literal['split', 'records', 'index', 'table', 'columns', 'values'] | None = None,
    date_format: str | None = None,
    double_precision: int = 10,
    force_ascii: bool = True,
    date_unit: TimeUnit = 'ms',
    default_handler: Callable[[Any], JSONSerializable] | None = None,
    lines: bool = False,
    compression: CompressionOptions = 'infer',
    index: bool | None = None,
    indent: int | None = None,
    storage_options: StorageOptions | None = None,
    mode: Literal['a', 'w'] = 'w'
) -> str | None
    Convert the object to a JSON string.
```

Note NaN's and None will be converted to null and datetime objects will be converted to UNIX timestamps.

Parameters

`path_or_buf` : str, path object, file-like object, or None, default None
String, path object (implementing `os.PathLike[str]`), or file-like object implementing a `write()` function. If None, the result is returned as a string.

`orient` : str

Indication of expected JSON string format.

* Series:

- default is 'index'
- allowed values are: {'split', 'records', 'index', 'table'}.

* DataFrame:

- default is 'columns'
- allowed values are: {'split', 'records', 'index', 'columns', 'values', 'table'}.

* The format of the JSON string:

- 'split' : dict like {'index' -> [index], 'columns' -> [columns], 'data' -> [values]}
- 'records' : list like [{column -> value}, ... , {column -> value}]
- 'index' : dict like {index -> {column -> value}}
- 'columns' : dict like {column -> {index -> value}}
- 'values' : just the values array
- 'table' : dict like {'schema': {schema}, 'data': {data}}

Describing the data, where data component is like ``orient='records'``.

date_format : {None, 'epoch', 'iso'}

Type of date conversion. 'epoch' = epoch milliseconds, 'iso' = ISO8601. The default depends on the 'orient'. For ``orient='table'``, the default is 'iso'. For all other orients, the default is 'epoch'.

.. deprecated:: 3.0.0

'epoch' date format is deprecated and will be removed in a future version, please use 'iso' instead.

double_precision : int, default 10

The number of decimal places to use when encoding floating point values. The possible maximal value is 15. Passing double_precision greater than 15 will raise a ValueError.

force_ascii : bool, default True

Force encoded string to be ASCII.

date_unit : str, default 'ms' (milliseconds)

The time unit to encode to, governs timestamp and ISO8601 precision. One of 's', 'ms', 'us', 'ns' for second, millisecond, microsecond, and nanosecond respectively.

default_handler : callable, default None

Handler to call if object cannot otherwise be converted to a suitable format for JSON. Should receive a single argument which is the object to convert and return a serialisable object.

lines : bool, default False

If 'orient' is 'records' write out line-delimited json format. Will throw ValueError if incorrect 'orient' since others are not list-like.

compression : str or dict, default 'infer'

For on-the-fly compression of the output data. If 'infer' and 'path_or_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).

Set to ``None`` for no compression.

Can also be a dict with key ``'method'`` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to ``zipfile.ZipFile``, ``gzip.GzipFile``, ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or ``tarfile.TarFile``, respectively.

As an example, the following could be passed for faster compression and to create a reproducible gzip archive:

``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}```.

index : bool or None, default None

The index is only used when 'orient' is 'split', 'index', 'column', or 'table'. Of these, 'index' and 'column' do not support ``index=False``. The string 'index' as a column name with empty :class:`Index` or if it is 'index' will raise a ``ValueError``.

```

indent : int, optional
    Length of whitespace used to indent each record.

storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

mode : str, default 'w' (writing)
    Specify the IO mode for output when supplying a path_or_buf.
    Accepted args are 'w' (writing) and 'a' (append) only.
    mode='a' is only supported when lines is True and orient is 'records'.

```

Returns

None or str

If path_or_buf is None, returns the resulting json format as a string. Otherwise returns None.

See Also

read_json : Convert a JSON string to pandas object.

Notes

The behavior of ``indent=0`` varies from the stdlib, which does not indent the output but does insert newlines. Currently, ``indent=0`` and the default ``indent=None`` are equivalent in pandas, though this may change in a future release.

``orient='table'`` contains a 'pandas_version' field under 'schema'. This stores the version of `pandas` used in the latest revision of the schema.

Examples

```

>>> from json import loads, dumps
>>> df = pd.DataFrame(
...     [ ["a", "b"], ["c", "d"] ],
...     index=["row 1", "row 2"],
...     columns=["col 1", "col 2"],
... )

>>> result = df.to_json(orient="split")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "columns": [
    "col 1",
    "col 2"
  ],
  "index": [
    "row 1",
    "row 2"
  ],
  "data": [
    [
      "a",
      "b"
    ],
    [
      "c",
      "d"
    ]
  ]
}}

```

Encoding/decoding a Dataframe using ``'records'`` formatted JSON. Note that index labels are not preserved with this encoding.

```

>>> result = df.to_json(orient="records")
>>> parsed = loads(result)

```

```
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
  {{
    "col 1": "a",
    "col 2": "b"
  }},
  {{
    "col 1": "c",
    "col 2": "d"
  }}
]
```

Encoding/decoding a Dataframe using ``'index'`` formatted JSON:

```
>>> result = df.to_json(orient="index")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "row 1": {{
    "col 1": "a",
    "col 2": "b"
  }},
  "row 2": {{
    "col 1": "c",
    "col 2": "d"
  }}
}}
```

Encoding/decoding a Dataframe using ``'columns'`` formatted JSON:

```
>>> result = df.to_json(orient="columns")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "col 1": {{
    "row 1": "a",
    "row 2": "c"
  }},
  "col 2": {{
    "row 1": "b",
    "row 2": "d"
  }}
}}
```

Encoding/decoding a Dataframe using ``'values'`` formatted JSON:

```
>>> result = df.to_json(orient="values")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
  [
    "a",
    "b"
  ],
  [
    "c",
    "d"
  ]
]
```

Encoding with Table Schema:

```
>>> result = df.to_json(orient="table")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "schema": {{
    "fields": [
      {{
        "name": "index",
        "type": "string"
      }},
      {{
        "name": "col 1",
        "type": "string"
      }}
    ]
  }}
}}
```

```

        "name": "col 2",
        "type": "string"
    }},
    ],
    "primaryKey": [
        "index"
    ],
    "pandas_version": "1.4.0"
}},
"data": [
    {{
        "index": "row 1",
        "col 1": "a",
        "col 2": "b"
    }},
    {{
        "index": "row 2",
        "col 1": "c",
        "col 2": "d"
    }}
]
}}

to_latex(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    columns: Sequence[Hashable] | None = None,
    header: bool | SequenceNotStr[str] = True,
    index: bool = True,
    na_rep: str = 'NaN',
    formatters: FormattersType | None = None,
    float_format: FloatFormatType | None = None,
    sparsify: bool | None = None,
    index_names: bool = True,
    bold_rows: bool = False,
    column_format: str | None = None,
    longtable: bool | None = None,
    escape: bool | None = None,
    encoding: str | None = None,
    decimal: str = '.',
    multicolumn: bool | None = None,
    multicolumn_format: str | None = None,
    multirow: bool | None = None,
    caption: str | tuple[str, str] | None = None,
    label: str | None = None,
    position: str | None = None
) -> str | None
    Render object to a LaTeX tabular, longtable, or nested table.

    Requires ``\usepackage{booktabs}``. The output can be copy/pasted
    into a main LaTeX document or read from an external file
    with ``\input{table.tex}``.

    .. versionchanged:: 2.0.0
        Refactored to use the Styler implementation via jinja2 templating.

Parameters
-----
buf : str, Path or StringIO-like, optional, default None
    Buffer to write to. If None, the output is returned as a string.
columns : list of label, optional
    The subset of columns to write. Writes all columns by default.
header : bool or list of str, default True
    Write out the column names. If a list of strings is given,
    it is assumed to be aliases for the column names. Braces must be escaped.
index : bool, default True
    Write row names (index).
na_rep : str, default 'NaN'
    Missing data representation.
formatters : list of functions or dict of {str: function}, optional
    Formatter functions to apply to columns' elements by position or
    name. The result of each function must be a unicode string.
    List must be of length equal to the number of columns.
float_format : one-parameter function or str, optional, default None
    Formatter for floating point numbers. For example
    ``float_format="%0.2f"`` and ``float_format="{:0.2f}".format`` will

```

both result in 0.1234 being formatted as 0.12.

sparsify : bool, optional
Set to False for a DataFrame with a hierarchical index to print every multiindex key at each row. By default, the value will be read from the config module.

index_names : bool, default True
Prints the names of the indexes.

bold_rows : bool, default False
Make the row labels bold in the output.

column_format : str, optional
The columns format as specified in `LaTeX table format` <https://en.wikibooks.org/wiki/LaTeX/Tables> e.g. 'rcl' for 3 columns. By default, 'l' will be used for all columns except columns of numbers, which default to 'r'.

longtable : bool, optional
Use a longtable environment instead of tabular. Requires adding a `\usepackage{longtable}` to your LaTeX preamble. By default, the value will be read from the pandas config module, and set to `True` if the option ```styler.latex.environment``` is `"longtable"`.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed.

escape : bool, optional
By default, the value will be read from the pandas config module and set to `True` if the option ```styler.format.escape``` is `"latex"`. When set to False prevents from escaping latex special characters in column names.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to `False`.

encoding : str, optional
A string representing the encoding to use in the output file, defaults to 'utf-8'.

decimal : str, default '.'
Character recognized as decimal separator, e.g. ',' in Europe.

multicolumn : bool, default True
Use `\multicolumn` to enhance MultiIndex columns. The default will be read from the config module, and is set as the option ```styler.sparse.columns```.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed.

multicolumn_format : str, default 'r'
The alignment for multicolumns, similar to `column_format`. The default will be read from the config module, and is set as the option ```styler.latex.multicol_align```.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to "r".

multirow : bool, default True
Use `\multirow` to enhance MultiIndex rows. Requires adding a `\usepackage{multirow}` to your LaTeX preamble. Will print centered labels (instead of top-aligned) across the contained rows, separating groups via clines. The default will be read from the pandas config module, and is set as the option ```styler.sparse.index```.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to `True`.

caption : str or tuple, optional
Tuple (full_caption, short_caption), which results in ```\caption[short_caption]{full_caption}```; if a single string is passed, no short caption will be set.

label : str, optional
The LaTeX label to be placed inside ```\label{}``` in the output. This is used with ```\ref{}``` in the main ```.tex``` file.

position : str, optional
The LaTeX positional argument for tables, to be placed after ```\begin{}``` in the output.

Returns

str or None
If buf is None, returns the result as a string. Otherwise returns None.

See Also

io.formats.style.Styler.to_latex : Render a DataFrame to LaTeX
with conditional formatting.

DataFrame.to_string : Render a DataFrame to a console-friendly
tabular output.

DataFrame.to_html : Render a DataFrame as an HTML table.

Notes

As of v2.0.0 this method has changed to use the Styler implementation as
part of :meth:`Styler.to\_latex` via ``jinja2`` templating. This means
that ``jinja2`` is a requirement, and needs to be installed, for this method
to function. It is advised that users switch to using Styler, since that
implementation is more frequently updated and contains much more
flexibility with the output.

Examples

Convert a general DataFrame to LaTeX with formatting:

```
>>> df = pd.DataFrame(dict(name=['Raphael', 'Donatello'],
...                          age=[26, 45],
...                          height=[181.23, 177.65]))
>>> print(df.to_latex(index=False,
...                    formatters={"name": str.upper},
...                    float_format="{:.1f}".format,
...                    )) # doctest: +SKIP
\begin{tabular}{lrr}
\toprule
name & age & height \\
\midrule
RAPHAEL & 26 & 181.2 \\
DONATELLO & 45 & 177.7 \\
\bottomrule
\end{tabular}
```

```
to_pickle(
    self,
    path: FilePath | WriteBuffer[bytes],
    *,
    compression: CompressionOptions = 'infer',
    protocol: int = 5,
    storage_options: StorageOptions | None = None
) -> None
Pickle (serialize) object to file.
```

Parameters

path : str, path object, or file-like object
String, path object (implementing ``os.PathLike[str]``), or file-like
object implementing a binary ``write()`` function. File path where
the pickled object will be stored.

compression : str or dict, default 'infer'

For on-the-fly compression of the output data. If 'infer' and
'path_or_buf' is path-like, then detect compression from the following
extensions: '.gz',
'bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
(otherwise no compression).

Set to ``None`` for no compression.

Can also be a dict with key ``'method'`` set to one of
{``'zip'``, ``'gzip'``, ``'bz2'``, ``'zstd'``, ``'xz'``, ``'tar'``} and
other key-value pairs are forwarded to
``zipfile.ZipFile``, ``gzip.GzipFile``,
``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
``tarfile.TarFile``, respectively.

As an example, the following could be passed for faster compression and
to create a reproducible gzip archive:

```
``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.
```

protocol : int

Int which indicates which protocol should be used by the pickler,
default HIGHEST_PROTOCOL (see [1]_ paragraph 12.1.2). The possible

values are 0, 1, 2, 3, 4, 5. A negative value for the protocol parameter is equivalent to setting its value to HIGHEST_PROTOCOL.

.. [1] <https://docs.python.org/3/library/pickle.html>.

`storage_options` : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.`

See Also

`read_pickle` : Load pickled pandas object (or any object) from file.
`DataFrame.to_hdf` : Write DataFrame to an HDF5 file.
`DataFrame.to_sql` : Write DataFrame to a SQL database.
`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.

Examples

```
>>> original_df = pd.DataFrame(
...     {"foo": range(5), "bar": range(5, 10)}
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
>>> original_df.to_pickle("./dummy.pkl") # doctest: +SKIP

>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
```

```
to_sql(
    self,
    name: str,
    con,
    *,
    schema: str | None = None,
    if_exists: Literal['fail', 'replace', 'append', 'delete_rows'] = 'fail',
    index: bool = True,
    index_label: IndexLabel | None = None,
    chunksize: int | None = None,
    dtype: DtypeArg | None = None,
    method: Literal['multi'] | Callable | None = None
) -> int | None
Write records stored in a DataFrame to a SQL database.
```

Databases supported by SQLAlchemy [1]_ are supported. Tables can be newly created, appended to, or overwritten.

.. warning::

The pandas library does not attempt to sanitize inputs provided via a `to_sql` call. Please refer to the documentation for the underlying database driver to see if it will properly prevent injection, or alternatively be advised of a security risk when executing arbitrary commands in a `to_sql` call.

Parameters

`name` : str
Name of SQL table.
`con` : ADBC connection, `sqlalchemy.engine.(Engine or Connection)` or `sqlite3.Connection`
ADBC provides high performance I/O with native type support, where available. Using SQLAlchemy makes it possible to use any DB supported by that library. Legacy support is provided for `sqlite3.Connection` objects. The user

is responsible for engine disposal and connection closure for the SQLAlchemy connectable. See [here](https://docs.sqlalchemy.org/en/20/core/connections.html#engine-disposal) <https://docs.sqlalchemy.org/en/20/core/connections.html#engine-disposal>.
If passing a sqlalchemy.engine.Connection which is already in a transaction, the transaction will not be committed. If passing a sqlite3.Connection, it will not be possible to roll back the record insertion.

schema : str, optional

Specify the schema (if database flavor supports this). If None, use default schema.

if_exists : {'fail', 'replace', 'append', 'delete_rows'}, default 'fail'
How to behave if the table already exists.

- * fail: Raise a ValueError.
- * replace: Drop the table before inserting new values.
- * append: Insert new values to the existing table.
- * delete_rows: If a table exists, delete all records and insert data.

index : bool, default True

Write DataFrame index as a column. Uses `index_label` as the column name in the table. Creates a table index for this column.

index_label : str or sequence, default None

Column label for index column(s). If None is given (default) and `index` is True, then the index names are used.

A sequence should be given if the DataFrame uses MultiIndex.

chunksize : int, optional

Specify the number of rows in each batch to be written to the database connection at a time. By default, all rows will be written at once. Also see the method keyword.

dtype : dict or scalar, optional

Specifying the datatype for columns. If a dictionary is used, the keys should be the column names and the values should be the SQLAlchemy types or strings for the sqlite3 legacy mode. If a scalar is provided, it will be applied to all columns.

method : {None, 'multi', callable}, optional

Controls the SQL insertion clause used:

- * None : Uses standard SQL `INSERT` clause (one per row).
- * 'multi': Pass multiple values in a single `INSERT` clause.
- * callable with signature ```(pd_table, conn, keys, data_iter)```.

Details and a sample callable implementation can be found in the section [:ref: insert method <io.sql.method>](#).

Returns

None or int

Number of rows affected by `to_sql`. None is returned if the callable passed into `method` does not return an integer number of rows.

The number of returned rows affected is the sum of the `rowcount` attribute of `sqlite3.Cursor` or SQLAlchemy connectable which may not reflect the exact number of written rows as stipulated in the `sqlite3` <https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.rowcount> or SQLAlchemy <https://docs.sqlalchemy.org/en/20/core/connections.html#sqlalchemy.engine.Connection.rowcount>.

Raises

ValueError

When the table already exists and `if_exists` is 'fail' (the default).

See Also

`read_sql` : Read a DataFrame from a table.

Notes

Timezone aware datetime columns will be written as `Timestamp with timezone` type with SQLAlchemy if supported by the database. Otherwise, the datetimes will be stored as timezone unaware timestamps local to the original timezone.

Not all datastores support `method="multi"`. Oracle, for example, does not support multi-value insert.

References

.. [1] <https://docs.sqlalchemy.org>

.. [2] <https://www.python.org/dev/peps/pep-0249/>

Examples

Create an in-memory SQLite database.

```
>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite://', echo=False)
```

Create a table from scratch with 3 rows.

```
>>> df = pd.DataFrame({'name' : ['User 1', 'User 2', 'User 3']})
>>> df
   name
0  User 1
1  User 2
2  User 3
```

```
>>> df.to_sql(name='users', con=engine)
3
>>> from sqlalchemy import text
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3')]
```

An `sqlalchemy.engine.Connection` can also be passed to `con`:

```
>>> with engine.begin() as connection:
...     df1 = pd.DataFrame({'name' : ['User 4', 'User 5']})
...     df1.to_sql(name='users', con=connection, if_exists='append')
2
```

This is allowed to support operations that require that the same DBAPI connection is used for the entire operation.

```
>>> df2 = pd.DataFrame({'name' : ['User 6', 'User 7']})
>>> df2.to_sql(name='users', con=engine, if_exists='append')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3'),
 (0, 'User 4'), (1, 'User 5'), (0, 'User 6'),
 (1, 'User 7')]
```

Overwrite the table with just `df2`.

```
>>> df2.to_sql(name='users', con=engine, if_exists='replace',
...             index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 6'), (1, 'User 7')]
```

Delete all rows before inserting new records with `df3`.

```
>>> df3 = pd.DataFrame({"name": ['User 8', 'User 9']})
>>> df3.to_sql(name='users', con=engine, if_exists='delete_rows',
...             index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 8'), (1, 'User 9')]
```

Use `method` to define a callable insertion method to do nothing if there's a primary key conflict on a table in a PostgreSQL database.

```
>>> from sqlalchemy.dialects.postgresql import insert
>>> def insert_on_conflict_nothing(table, conn, keys, data_iter):
...     # "a" is the primary key in "conflict_table"
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = insert(table.table).values(data).on_conflict_do_nothing(index_elements=["a"])
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F82
...                     method=insert_on_conflict_nothing) # doctest: +SKIP
0
```

For MySQL, a callable to update columns ``b`` and ``c`` if there's a conflict on a primary key.

```
>>> from sqlalchemy.dialects.mysql import insert # noqa: F811
>>> def insert_on_conflict_update(table, conn, keys, data_iter):
...     # update columns "b" and "c" on primary key conflict
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = (
...         insert(table.table)
...         .values(data)
...     )
...     stmt = stmt.on_duplicate_key_update(b=stmt.inserted.b, c=stmt.inserted.c)
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F82
...                     method=insert_on_conflict_update) # doctest: +SKIP
2
```

Specify the dtype (especially useful for integers with missing values). Notice that while pandas is forced to store the data as floating point, the database supports nullable integers. When fetching the data with Python, we get back integer scalars.

```
>>> df = pd.DataFrame({"A": [1, None, 2]})
>>> df
   A
0  1.0
1  NaN
2  2.0
```

```
>>> from sqlalchemy.types import Integer
>>> df.to_sql(name='integers', con=engine, index=False,
...          dtype={"A": Integer()})
3
```

```
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM integers")).fetchall()
[(1,), (None,), (2,)]
```

.. versionadded:: 2.2.0

pandas now supports writing via ADBC drivers

```
>>> df = pd.DataFrame({'name' : ['User 10', 'User 11', 'User 12']})
>>> df
   name
0  User 10
1  User 11
2  User 12
```

```
>>> from adbc_driver_sqlite import dbapi # doctest:+SKIP
>>> with dbapi.connect("sqlite://") as conn: # doctest:+SKIP
...     df.to_sql(name="users", con=conn)
3
```

`to_xarray(self)`

Return an xarray object from the pandas object.

Returns

xarray.DataArray or xarray.Dataset

Data in the pandas structure converted to Dataset if the object is a DataFrame, or a DataArray if the object is a Series.

See Also

`DataFrame.to_hdf` : Write DataFrame to an HDF5 file.

`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.

Notes

See the `xarray` docs <<https://xarray.pydata.org/en/stable/>>`__

Examples

```
>>> df = pd.DataFrame(
...     [
```

```

...         ("falcon", "bird", 389.0, 2),
...         ("parrot", "bird", 24.0, 2),
...         ("lion", "mammal", 80.5, 4),
...         ("monkey", "mammal", np.nan, 4),
...     ],
...     columns=["name", "class", "max_speed", "num_legs"],
... )
>>> df
   name  class  max_speed  num_legs
0  falcon   bird     389.0         2
1  parrot   bird      24.0         2
2    lion  mammal     80.5         4
3  monkey  mammal      NaN         4

>>> df.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (index: 4)
Coordinates:
  * index    (index) int64 32B 0 1 2 3
Data variables:
  name      (index) object 32B 'falcon' 'parrot' 'lion' 'monkey'
  class     (index) object 32B 'bird' 'bird' 'mammal' 'mammal'
  max_speed (index) float64 32B 389.0 24.0 80.5 nan
  num_legs  (index) int64 32B 2 2 4 4

>>> df["max_speed"].to_xarray() # doctest: +SKIP
<xarray.DataArray 'max_speed' (index: 4)>
array([389. , 24. , 80.5,  nan])
Coordinates:
  * index    (index) int64 0 1 2 3

>>> dates = pd.to_datetime(
...     ["2018-01-01", "2018-01-01", "2018-01-02", "2018-01-02"]
... )
>>> df_multiindex = pd.DataFrame(
...     {
...         "date": dates,
...         "animal": ["falcon", "parrot", "falcon", "parrot"],
...         "speed": [350, 18, 361, 15],
...     }
... )
>>> df_multiindex = df_multiindex.set_index(["date", "animal"])

>>> df_multiindex
              speed
date  animal
2018-01-01 falcon    350
           parrot     18
2018-01-02 falcon    361
           parrot     15

>>> df_multiindex.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (date: 2, animal: 2)
Coordinates:
  * date      (date) datetime64[s] 2018-01-01 2018-01-02
  * animal    (animal) object 'falcon' 'parrot'
Data variables:
  speed      (date, animal) int64 350 18 361 15

```

```

truncate(
    self,
    before=None,
    after=None,
    axis: Axis | None = None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self
Truncate a Series or DataFrame before and after some index value.

```

This is a useful shorthand for boolean indexing based on index values above or below certain thresholds.

Parameters

```

-----
before : date, str, int
    Truncate all rows before this index value.
after  : date, str, int

```

Truncate all rows after this index value.
axis : {0 or 'index', 1 or 'columns'}, optional
Axis to truncate. Truncates the index (rows) by default.
For `Series` this parameter is unused and defaults to 0.
copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.
.. deprecated:: 3.0.0
This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`\_\_` for more details.

Returns

type of caller
The truncated Series or DataFrame.

See Also

DataFrame.loc : Select a subset of a DataFrame by label.
DataFrame.iloc : Select a subset of a DataFrame by position.

Notes

If the index being truncated contains only datetime values, `before` and `after` may be specified as strings instead of Timestamps.

Examples

>>> df = pd.DataFrame(
... {
... "A": ["a", "b", "c", "d", "e"],
... "B": ["f", "g", "h", "i", "j"],
... "C": ["k", "l", "m", "n", "o"],
... },
... index=[1, 2, 3, 4, 5],
...)
>>> df
 A B C
1 a f k
2 b g l
3 c h m
4 d i n
5 e j o

>>> df.truncate(before=2, after=4)
 A B C
2 b g l
3 c h m
4 d i n

The columns of a DataFrame can be truncated.

>>> df.truncate(before="A", after="B", axis="columns")
 A B
1 a f
2 b g
3 c h
4 d i
5 e j

For Series, only rows can be truncated.

>>> df["A"].truncate(before=2, after=4)
2 b
3 c
4 d
Name: A, dtype: str

The index values in ``truncate`` can be datetimes or string dates.

```
>>> dates = pd.date_range("2016-01-01", "2016-02-01", freq="s")
>>> df = pd.DataFrame(index=dates, data={"A": 1})
>>> df.tail()
                A
2016-01-31 23:59:56  1
2016-01-31 23:59:57  1
2016-01-31 23:59:58  1
2016-01-31 23:59:59  1
2016-02-01 00:00:00  1

>>> df.truncate(
...     before=pd.Timestamp("2016-01-05"), after=pd.Timestamp("2016-01-10")
... ).tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Because the index is a `DatetimeIndex` containing only dates, we can specify ``before`` and ``after`` as strings. They will be coerced to `Timestamps` before truncation.

```
>>> df.truncate("2016-01-05", "2016-01-10").tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Note that ``truncate`` assumes a 0 value for any unspecified time component (midnight). This differs from partial string slicing, which returns any partially matching dates.

```
>>> df.loc["2016-01-05":"2016-01-10", :].tail()
                A
2016-01-10 23:59:55  1
2016-01-10 23:59:56  1
2016-01-10 23:59:57  1
2016-01-10 23:59:58  1
2016-01-10 23:59:59  1
```

```
tz_convert(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self
    Convert tz-aware axis to target time zone.
```

Parameters

```
-----
tz : str or tzinfo object or None
    Target time zone. Passing ``None`` will convert to
    UTC and remove the timezone information.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
    The axis to convert
level : int, str, default None
    If axis is a MultiIndex, convert a specific level. Otherwise
    must be None.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

Series/DataFrame

Object with time zone converted axis.

Raises

TypeError

If the axis is tz-naive.

See Also

DataFrame.tz_localize: Localize tz-naive index of DataFrame to target time zone.

Series.tz_localize: Localize tz-naive index of Series to target time zone.

Examples

Change to another time zone:

```
>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]),
... )
>>> s.tz_convert("Asia/Shanghai")
2018-09-15 07:30:00+08:00    1
dtype: int64
```

Pass None to convert to UTC and get a tz-naive index:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_convert(None)
2018-09-14 23:30:00    1
dtype: int64
```

```
tz_localize(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Localize time zone naive index of a Series or DataFrame to target time zone.
```

This operation localizes the Index. To localize the values in a time zone naive Series, use :meth:`Series.dt.tz\_localize`.

Parameters

tz : str or tzinfo or None

Time zone to localize. Passing ``None`` will remove the time zone information and preserve local time.

axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to localize

level : int, str, default None

If axis is a MultiIndex, localize a specific level. Otherwise must be None.

copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`_ for more details.

ambiguous : 'infer', bool, bool-ndarray, 'NaT', default 'raise'

When clocks moved backward due to DST, ambiguous times may arise.

For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be

```

        handled.

        - 'infer' will attempt to infer fall dst-transition hours based on
          order
        - bool (or bool-ndarray) where True signifies a DST time, False designates
          a non-DST time (note that this flag is only applicable for
          ambiguous times)
        - 'NaT' will return NaT where there are ambiguous times
        - 'raise' will raise a ValueError if there are ambiguous
          times.
    nonexistent : str, default 'raise'
        A nonexistent time does not exist in a particular timezone
        where clocks moved forward due to DST. Valid values are:

        - 'shift_forward' will shift the nonexistent time forward to the
          closest existing time
        - 'shift_backward' will shift the nonexistent time backward to the
          closest existing time
        - 'NaT' will return NaT where there are nonexistent times
        - timedelta objects will shift nonexistent times by the timedelta
        - 'raise' will raise a ValueError if there are
          nonexistent times.

Returns
-----
Series/DataFrame
    Same type as the input, with time zone naive or aware index, depending on
    ``tz``.

Raises
-----
TypeError
    If the TimeSeries is tz-aware and tz is not None.

See Also
-----
Series.dt.tz_localize: Localize the values in a time zone naive Series.
Timestamp.tz_localize: Localize the Timestamp to a timezone.

Examples
-----
Localize local times:

>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00"]),
... )
>>> s.tz_localize("CET")
2018-09-15 01:30:00+02:00    1
dtype: int64

Pass None to convert to tz-naive index and preserve local time:

>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_localize(None)
2018-09-15 01:30:00    1
dtype: int64

Be careful with DST changes. When there is sequential data, pandas
can infer the DST time:

>>> s = pd.Series(
...     range(7),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 03:00:00",
...             "2018-10-28 03:30:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous="infer")
2018-10-28 01:30:00+02:00    0

```

```

2018-10-28 02:00:00+02:00    1
2018-10-28 02:30:00+02:00    2
2018-10-28 02:00:00+01:00    3
2018-10-28 02:30:00+01:00    4
2018-10-28 03:00:00+01:00    5
2018-10-28 03:30:00+01:00    6
dtype: int64

```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```

>>> s = pd.Series(
...     range(3),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:20:00",
...             "2018-10-28 02:36:00",
...             "2018-10-28 03:46:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous=np.array([True, True, False]))
2018-10-28 01:20:00+02:00    0
2018-10-28 02:36:00+02:00    1
2018-10-28 03:46:00+01:00    2
dtype: int64

```

If the DST transition causes nonexistent times, you can shift these dates forward or backward with a timedelta object or `'shift_forward'` or `'shift_backward'`.

```

>>> dti = pd.DatetimeIndex(
...     ["2015-03-29 02:30:00", "2015-03-29 03:30:00"], dtype="M8[ns]"
... )
>>> s = pd.Series(range(2), index=dti)
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_forward")
2015-03-29 03:00:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_backward")
2015-03-29 01:59:59.999999999+01:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent=pd.Timedelta("1h"))
2015-03-29 03:30:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64

```

```

where(
    self,
    cond,
    other=<no_default>,
    *,
    inplace: bool = False,
    axis: Axis | None = None,
    level: Level | None = None
) -> Self

```

Replace values where the condition is False.

This method allows conditional replacement of values. Where the condition evaluates to True, the original values are retained; where it evaluates to False, values are replaced with corresponding entries from `other`.

Parameters

```

cond : bool Series/DataFrame, array-like, or callable
    Where cond is True, keep the original value. Where
    False, replace with corresponding value from other.
    If cond is callable, it is computed on the Series/DataFrame and
    should return boolean Series/DataFrame or array. The callable must
    not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
    Entries where cond is False are replaced with
    corresponding value from other.
    If other is callable, it is computed on the Series/DataFrame and
    should return scalar or Series/DataFrame. The callable must not

```


change input Series/DataFrame (though pandas doesn't check it).
If not specified, entries will be filled with the corresponding
NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension
dtypes).

`inplace` : bool, default False

Whether to perform the operation in place on the data.

`axis` : int, default None

Alignment axis if needed. For `Series` this parameter is
unused and defaults to 0.

`level` : int, default None

Alignment level if needed.

Returns

Series or DataFrame

When applied to a Series, the function will return a Series,
and when applied to a DataFrame, it will return a DataFrame.

See Also

`:func:DataFrame.mask` : Return an object of same shape as caller.

`:func:Series.mask` : Return an object of same shape as caller.

Notes

The `where` method is an application of the if-then idiom. For each
element in the caller, if `cond` is `True` the
element is used; otherwise the corresponding element from
`other` is used. If the axis of `other` does not align with axis of
`cond` Series/DataFrame, the values of `cond` on misaligned index positions
will be filled with False.

The signature for `:func:Series.where` or

`:func:DataFrame.where` differs from `:func:numpy.where`.

Roughly `df1.where(m, df2)` is equivalent to `np.where(m, df1, df2)`.

For further details and examples see the `where` documentation in
`:ref:indexing <indexing.where_mask>`.

The dtype of the object takes precedence. The fill value is casted to
the object's dtype, if this can be done losslessly.

Examples

```
>>> s = pd.Series(range(5))
```

```
>>> s.where(s > 0)
```

```
0    NaN
```

```
1    1.0
```

```
2    2.0
```

```
3    3.0
```

```
4    4.0
```

```
dtype: float64
```

```
>>> s.mask(s > 0)
```

```
0    0.0
```

```
1    NaN
```

```
2    NaN
```

```
3    NaN
```

```
4    NaN
```

```
dtype: float64
```

```
>>> s = pd.Series(range(5))
```

```
>>> t = pd.Series([True, False])
```

```
>>> s.where(t, 99)
```

```
0     0
```

```
1    99
```

```
2    99
```

```
3    99
```

```
4    99
```

```
dtype: int64
```

```
>>> s.mask(t, 99)
```

```
0    99
```

```
1     1
```

```
2    99
```

```
3    99
```

```
4    99
```

```
dtype: int64
```

```

>>> s.where(s > 1, 10)
0    10
1    10
2     2
3     3
4     4
dtype: int64
>>> s.mask(s > 1, 10)
0     0
1     1
2    10
3    10
4    10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
   A  B
0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True

```

```

xs(
    self,
    key: IndexLabel,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    drop_level: bool = True
) -> Self
    Return cross-section from the Series/DataFrame.

    This method takes a `key` argument to select data at a particular
    level of a MultiIndex.

    Parameters
    -----
    key : label or tuple of label
        Label contained in the index, or partially in a MultiIndex.
    axis : {0 or 'index', 1 or 'columns'}, default 0
        Axis to retrieve cross-section on.
    level : object, defaults to first n levels (n=1 or len(key))
        In case of a key partially contained in a MultiIndex, indicate
        which levels are used. Levels can be referred by label or position.
    drop_level : bool, default True
        If False, returns object with same levels as self.

    Returns
    -----
    Series or DataFrame
        Cross-section from the original Series or DataFrame
        corresponding to the selected index levels.

    See Also

```

DataFrame.loc : Access a group of rows and columns
by label(s) or a boolean array.
DataFrame.iloc : Purely integer-location based indexing
for selection by position.

Notes

`xs` can not be used to set values.

MultiIndex Slicers is a generic way to get/set values on
any level or levels.
It is a superset of `xs` functionality, see
:ref:`MultiIndex Slicers <advanced.mi\_slicers>`.

Examples

>>> d = {
... "num_legs": [4, 4, 2, 2],
... "num_wings": [0, 0, 2, 2],
... "class": ["mammal", "mammal", "mammal", "bird"],
... "animal": ["cat", "dog", "bat", "penguin"],
... "locomotion": ["walks", "walks", "flies", "walks"],
... }
>>> df = pd.DataFrame(data=d)
>>> df = df.set_index(["class", "animal", "locomotion"])
>>> df

			num_legs	num_wings
class	animal	locomotion		
mammal	cat	walks	4	0
	dog	walks	4	0
	bat	flies	2	2
bird	penguin	walks	2	2

Get values at specified index

>>> df.xs("mammal")

		num_legs	num_wings
animal	locomotion		
cat	walks	4	0
dog	walks	4	0
bat	flies	2	2

Get values at several indexes

>>> df.xs(("mammal", "dog", "walks"))
num_legs 4
num_wings 0
Name: (mammal, dog, walks), dtype: int64

Get values at specified index and level

>>> df.xs("cat", level=1)

		num_legs	num_wings
class	locomotion		
mammal	walks	4	0

Get values at several indexes and levels

>>> df.xs(("bird", "walks"), level=[0, "locomotion"])

	num_legs	num_wings
animal		
penguin	2	2

Get values at specified column and axis

>>> df.xs("num_wings", axis=1)

class	animal	locomotion	
mammal	cat	walks	0
	dog	walks	0
	bat	flies	2
bird	penguin	walks	2

Name: num_wings, dtype: int64

Readonly properties inherited from pandas.core.generic.NDFrame:

dtypes

Return the dtypes in the DataFrame.

This returns a Series with the data type of each column. The result's index is the original DataFrame's columns. Columns with mixed types are stored with the ``object`` dtype. See :ref:`the User Guide <basics.dtypes>` for more.

Returns

pandas.Series
The data type of each column.

See Also

Series.dtypes : Return the dtype object of the underlying data.

Examples

>>> df = pd.DataFrame(
... {
... "float": [1.0],
... "int": [1],
... "datetime": [pd.Timestamp("20180310")],
... "string": ["foo"],
... }
...)
>>> df.dtypes
float float64
int int64
datetime datetime64[us]
string str
dtype: object

empty

Indicator whether Series/DataFrame is empty.

True if Series/DataFrame is entirely empty (no items), meaning any of the axes are of length 0.

Returns

bool
If Series/DataFrame is empty, return True, if not return False.

See Also

Series.dropna : Return series without null values.
DataFrame.dropna : Return DataFrame with labels on given axis omitted where (all or any) data are missing.

Notes

If Series/DataFrame contains only NaNs, it is still not considered empty. See the example below.

Examples

An example of an actual empty DataFrame. Notice the index is empty:

```
>>> df_empty = pd.DataFrame({"A": []})  
>>> df_empty  
Empty DataFrame  
Columns: [A]  
Index: []  
>>> df_empty.empty  
True
```

If we only have NaNs in our DataFrame, it is not considered empty! We will need to drop the NaNs to make the DataFrame empty:

```
>>> df = pd.DataFrame({"A": [np.nan]})  
>>> df  
   A  
0 NaN  
>>> df.empty  
False
```

```
>>> df.dropna().empty
True

>>> ser_empty = pd.Series({"A": []})
>>> ser_empty
A    []
dtype: object
>>> ser_empty.empty
False
>>> ser_empty = pd.Series()
>>> ser_empty.empty
True
```

flags

Get the properties associated with this pandas object.

The available flags are

```
* :attr:`Flags.allows_duplicate_labels`
```

See Also

Flags : Flags that apply to pandas objects.

DataFrame.attrs : Global metadata applying to this dataset.

Notes

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in :attr:`DataFrame.attrs`.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags
<Flags(allows_duplicate_labels=True)>
```

Flags can be get or set using ``.``

```
>>> df.flags.allows_duplicate_labels
True
>>> df.flags.allows_duplicate_labels = False
```

Or by slicing with a key

```
>>> df.flags["allows_duplicate_labels"]
False
>>> df.flags["allows_duplicate_labels"] = True
```

ndim

Return an int representing the number of axes / array dimensions.

Return 1 if Series. Otherwise return 2 if DataFrame.

See Also

numpy.ndarray.ndim : Number of array dimensions.

Examples

```
>>> s = pd.Series({"a": 1, "b": 2, "c": 3})
>>> s.ndim
1
```

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.ndim
2
```

size

Return an int representing the number of elements in this object.

Return the number of rows if Series. Otherwise return the number of rows times number of columns if DataFrame.

See Also

numpy.ndarray.size : Number of elements in the array.

Examples

```
-----
>>> s = pd.Series({"a": 1, "b": 2, "c": 3})
>>> s.size
3

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.size
4
```

Data descriptors inherited from pandas.core.generic.NDFrame:

attrs

Dictionary of global attributes of this dataset.

.. warning::

attrs is experimental and may change without warning.

See Also

DataFrame.flags : Global flags applying to this object.

Notes

Many operations that create new datasets will copy ``attrs``. Copies are always deep so that changing ``attrs`` will only affect the present dataset. :func:`pandas.concat` and :func:`pandas.merge` will only copy ``attrs`` if all input datasets have the same ``attrs``.

Examples

For Series:

```
>>> ser = pd.Series([1, 2, 3])
>>> ser.attrs = {"A": [10, 20, 30]}
>>> ser.attrs
{'A': [10, 20, 30]}
```

For DataFrame:

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.attrs = {"A": [10, 20, 30]}
>>> df.attrs
{'A': [10, 20, 30]}
```

Data and other attributes inherited from pandas.core.generic.NDFrame:

__array_priority__ = 1000

Methods inherited from pandas.core.base.PandasObject:

__sizeof__(self) -> int
Generates the total memory usage for an object that returns either a value or Series of values

Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]
Provide method name lookup and completion.

Notes

Only provide 'public' methods.

Data descriptors inherited from pandas.core.accessor.DirNamesMixin:

__dict__
dictionary for instance variables

__weakref__

list of weak references to the object

Readonly properties inherited from pandas.core.indexing.IndexingMixin:

at

Access a single value for a row/column label pair.

Similar to ``loc``, in that both provide label-based lookups. Use ``at`` if you only need to get or set a single value in a DataFrame or Series.

Raises

KeyError

If getting a value and 'label' does not exist in a DataFrame or Series.

ValueError

If row/column label pair is not a tuple or if any label from the pair is not a scalar for DataFrame.

If label is list-like (*excluding* NamedTuple) for Series.

See Also

DataFrame.at : Access a single value for a row/column pair by label.

DataFrame.iat : Access a single value for a row/column pair by integer position.

DataFrame.loc : Access a group of rows and columns by label(s).

DataFrame.iloc : Access a group of rows and columns by integer position(s).

Series.at : Access a single value by label.

Series.iat : Access a single value by integer position.

Series.loc : Access a group of rows by label(s).

Series.iloc : Access a group of rows by integer position(s).

Notes

See :ref:`Fast scalar value getting and setting <indexing.basics.get\_value>` for more details.

Examples

```
>>> df = pd.DataFrame(  
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]],  
...     index=[4, 5, 6],  
...     columns=["A", "B", "C"],  
... )  
>>> df  
   A  B  C  
4  0  2  3  
5  0  4  1  
6 10 20 30
```

Get value at specified row/column pair

```
>>> df.at[4, "B"]  
np.int64(2)
```

Set value at specified row/column pair

```
>>> df.at[4, "B"] = 10  
>>> df.at[4, "B"]  
np.int64(10)
```

Get value within a Series

```
>>> df.loc[5].at["B"]  
np.int64(4)
```

iat

Access a single value for a row/column pair by integer position.

Similar to ``iloc``, in that both provide integer-based lookups. Use ``iat`` if you only need to get or set a single value in a DataFrame or Series.

Raises

IndexError
When integer position is out of bounds.

See Also

DataFrame.at : Access a single value for a row/column label pair.
DataFrame.loc : Access a group of rows and columns by label(s).
DataFrame.iloc : Access a group of rows and columns by integer position(s).

Examples

>>> df = pd.DataFrame(
... [[0, 2, 3], [0, 4, 1], [10, 20, 30]], columns=["A", "B", "C"]
...)
>>> df
 A B C
0 0 2 3
1 0 4 1
2 10 20 30

Get value at specified row/column pair

>>> df.iat[1, 2]
np.int64(1)

Set value at specified row/column pair

>>> df.iat[1, 2] = 10
>>> df.iat[1, 2]
np.int64(10)

Get value within a series

>>> df.loc[0].iat[1]
np.int64(2)

iloc

Purely integer-location based indexing for selection by position.

.. versionchanged:: 3.0

Callables which return a tuple are deprecated as input.

``.iloc[]`` is primarily integer position based (from ``0`` to ``length-1`` of the axis), but may also be used with a boolean array.

Allowed inputs are:

- An integer, e.g. ``5``.
- A list or array of integers, e.g. ``[4, 3, 0]``.
- A slice object with ints, e.g. ``1:7``.
- A boolean array.
- A ``callable`` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above). This is useful in method chains, when you don't have a reference to the calling object, but would like to base your selection on some value.
- A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. ``(0, 1)``.

``.iloc`` will raise ``IndexError`` if a requested indexer is out-of-bounds, except *slice* indexers which allow out-of-bounds indexing (this conforms with python/numpy *slice* semantics).

See more at :ref:`Selection by Position <indexing.integer>`.

See Also

DataFrame.iat : Fast integer location scalar accessor.
DataFrame.loc : Purely label-location based indexer for selection by label.
Series.iloc : Purely integer-location based indexing for selection by position.

Examples

```
>>> mydict = [
...     {"a": 1, "b": 2, "c": 3, "d": 4},
...     {"a": 100, "b": 200, "c": 300, "d": 400},
...     {"a": 1000, "b": 2000, "c": 3000, "d": 4000},
... ]
>>> df = pd.DataFrame(mydict)
>>> df
```

	a	b	c	d
0	1	2	3	4
1	100	200	300	400
2	1000	2000	3000	4000

****Indexing just the rows****

With a scalar integer.

```
>>> type(df.iloc[0])
<class 'pandas.Series'>
>>> df.iloc[0]
a    1
b    2
c    3
d    4
Name: 0, dtype: int64
```

With a list of integers.

```
>>> df.iloc[[0]]
   a  b  c  d
0  1  2  3  4
>>> type(df.iloc[[0]])
<class 'pandas.DataFrame'>
```

```
>>> df.iloc[[0, 1]]
   a  b  c  d
0  1  2  3  4
1 100 200 300 400
```

With a `slice` object.

```
>>> df.iloc[:3]
   a  b  c  d
0  1  2  3  4
1 100 200 300 400
2 1000 2000 3000 4000
```

With a boolean mask the same length as the index.

```
>>> df.iloc[[True, False, True]]
   a  b  c  d
0  1  2  3  4
2 1000 2000 3000 4000
```

With a callable, useful in method chains. The `x` passed to the ``lambda`` is the DataFrame being sliced. This selects the rows whose index label even.

```
>>> df.iloc[lambda x: x.index % 2 == 0]
   a  b  c  d
0  1  2  3  4
2 1000 2000 3000 4000
```

****Indexing both axes****

You can mix the indexer types for the index and columns. Use ``:``` to select the entire axis.

With scalar integers.

```
>>> df.iloc[0, 1]
np.int64(2)
```

With lists of integers.

```
>>> df.iloc[[0, 2], [1, 3]]
   b  d
0  2  4
```

```
2 2000 4000
```

With `slice` objects.

```
>>> df.iloc[1:3, 0:3]
   a    b    c
1  100  200  300
2 1000 2000 3000
```

With a boolean array whose length matches the columns.

```
>>> df.iloc[:, [True, False, True, False]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

With a callable function that expects the Series or DataFrame.

```
>>> df.iloc[:, lambda df: [0, 2]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

`loc`

Access a group of rows and columns by label(s) or a boolean array.

`df.loc[]` is primarily label based, but may also be used with a boolean array.

Allowed inputs are:

- A single label, e.g. `df.loc[5]` or `df.loc['a']`, (note that `df.loc[5]` is interpreted as a *label* of the index, and **never** as an integer position along the index).
- A list or array of labels, e.g. `df.loc['a', 'b', 'c']`.
- A slice object with labels, e.g. `df.loc['a':'f']`.

.. warning:: Note that contrary to usual python slices, **both** the start and the stop are included

- A boolean array of the same length as the axis being sliced, e.g. `df.loc[[True, False, True]]`.
- An alignable boolean Series. The index of the key will be aligned before masking.
- An alignable Index. The Index of the returned selection will be the input.
- A `callable` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above)

See more at :ref:`Selection by Label <indexing.label>`.

Raises

KeyError

If any items are not found.

IndexingError

If an indexed key is passed and its index is unalignable to the frame index.

See Also

`DataFrame.at` : Access a single value for a row/column label pair.

`DataFrame.iloc` : Access group of rows and columns by integer position(s).

`DataFrame.xs` : Returns a cross-section (row(s) or column(s)) from the Series/DataFrame.

`Series.loc` : Access group of values using labels.

Examples

****Getting values****

```
>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=["cobra", "viper", "sidewinder"],
...     columns=["max_speed", "shield"],
... )
>>> df
```

	max_speed	shield
cobra	1	2
viper	4	5
sidewinder	7	8

Single label. Note this returns the row as a Series.

```
>>> df.loc["viper"]
max_speed    4
shield       5
Name: viper, dtype: int64
```

List of labels. Note using ``[]`` returns a DataFrame.

```
>>> df.loc[["viper", "sidewinder"]]
      max_speed  shield
viper         4      5
sidewinder    7      8
```

Single label for row and column

```
>>> df.loc["cobra", "shield"]
np.int64(2)
```

Slice with labels for row and single label for column. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc["cobra":"viper", "max_speed"]
cobra    1
viper    4
Name: max_speed, dtype: int64
```

Boolean list with the same length as the row axis

```
>>> df.loc[[False, False, True]]
      max_speed  shield
sidewinder    7      8
```

Alignable boolean Series:

```
>>> df.loc[
...     pd.Series([False, True, False], index=["viper", "sidewinder", "cobra"])
... ]
      max_speed  shield
sidewinder    7      8
```

Index (same behavior as ``df.reindex``)

```
>>> df.loc[pd.Index(["cobra", "viper"], name="foo")]
      max_speed  shield
foo
cobra         1      2
viper         4      5
```

Conditional that returns a boolean Series

```
>>> df.loc[df["shield"] > 6]
      max_speed  shield
sidewinder    7      8
```

Conditional that returns a boolean Series with column labels specified

```
>>> df.loc[df["shield"] > 6, ["max_speed"]]
      max_speed
sidewinder    7
```

Multiple conditional using ``&`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 1) & (df["shield"] < 8)]
      max_speed  shield
viper         4      5
```

Multiple conditional using ``|`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 4) | (df["shield"] < 5)]
      max_speed  shield
cobra         1      2
```

```
sidewinder          7      8
```

Please ensure that each condition is wrapped in parentheses ``()``.
See the :ref:`user guide<indexing.boolean>`
for more details and explanations of Boolean indexing.

.. note::

If you find yourself using 3 or more conditionals in ``.loc[]``,
consider using :ref:`advanced indexing<advanced.advanced\_hierarchical>`.

See below for using ``.loc[]`` on MultiIndex DataFrames.

Callable that returns a boolean Series

```
>>> df.loc[lambda df: df["shield"] == 8]
      max_speed  shield
sidewinder      7      8
```

****Setting values****

Set value for all items matching the list of labels

```
>>> df.loc[["viper", "sidewinder"], ["shield"]] = 50
>>> df
      max_speed  shield
cobra           1      2
viper           4     50
sidewinder      7     50
```

Set value for an entire row

```
>>> df.loc["cobra"] = 10
>>> df
      max_speed  shield
cobra         10     10
viper          4     50
sidewinder      7     50
```

Set value for an entire column

```
>>> df.loc[:, "max_speed"] = 30
>>> df
      max_speed  shield
cobra         30     10
viper         30     50
sidewinder     30     50
```

Set value for rows matching callable condition

```
>>> df.loc[df["shield"] > 35] = 0
>>> df
      max_speed  shield
cobra         30     10
viper          0      0
sidewinder      0      0
```

Add value matching location

```
>>> df.loc["viper", "shield"] += 5
>>> df
      max_speed  shield
cobra         30     10
viper          0      5
sidewinder      0      0
```

Setting using a ``Series`` or a ``DataFrame`` sets the values matching the
index labels, not the index positions.

```
>>> shuffled_df = df.loc[["viper", "cobra", "sidewinder"]]
>>> df.loc[:] += shuffled_df
>>> df
      max_speed  shield
cobra         60     20
viper          0     10
sidewinder      0      0
```

****Getting values on a DataFrame with an index that has integer labels****

Another example using integers for the index

```
>>> df = pd.DataFrame(  
...     [[1, 2], [4, 5], [7, 8]],  
...     index=[7, 8, 9],  
...     columns=["max_speed", "shield"],  
... )  
>>> df
```

	max_speed	shield
7	1	2
8	4	5
9	7	8

Slice with integer labels for rows. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc[7:9]
```

	max_speed	shield
7	1	2
8	4	5
9	7	8

****Getting values with a MultiIndex****

A number of examples using a DataFrame with a MultiIndex

```
>>> tuples = [  
...     ("cobra", "mark i"),  
...     ("cobra", "mark ii"),  
...     ("sidewinder", "mark i"),  
...     ("sidewinder", "mark ii"),  
...     ("viper", "mark ii"),  
...     ("viper", "mark iii"),  
... ]  
>>> index = pd.MultiIndex.from_tuples(tuples)  
>>> values = [[12, 2], [0, 4], [10, 20], [1, 4], [7, 1], [16, 36]]  
>>> df = pd.DataFrame(values, columns=["max_speed", "shield"], index=index)  
>>> df
```

		max_speed	shield
cobra	mark i	12	2
	mark ii	0	4
sidewinder	mark i	10	20
	mark ii	1	4
viper	mark ii	7	1
	mark iii	16	36

Single label. Note this returns a DataFrame with a single index.

```
>>> df.loc["cobra"]
```

	max_speed	shield
mark i	12	2
mark ii	0	4

Single index tuple. Note this returns a Series.

```
>>> df.loc[("cobra", "mark ii")]  
max_speed    0  
shield       4  
Name: (cobra, mark ii), dtype: int64
```

Single label for row and column. Similar to passing in a tuple, this returns a Series.

```
>>> df.loc["cobra", "mark i"]  
max_speed    12  
shield       2  
Name: (cobra, mark i), dtype: int64
```

Single tuple. Note using ``[[]]`` returns a DataFrame.

```
>>> df.loc[ [("cobra", "mark ii")]]
```

	max_speed	shield
cobra mark ii	0	4

Single tuple for the index with a single label for the column

```
>>> df.loc[("cobra", "mark i"), "shield"]
np.int64(2)
```

Slice from index tuple to single label

```
>>> df.loc[("cobra", "mark i") : "viper"]
           max_speed  shield
cobra      mark i      12      2
           mark ii      0      4
sidewinder mark i      10     20
           mark ii      1      4
viper      mark ii      7      1
           mark iii     16     36
```

Slice from index tuple to index tuple

```
>>> df.loc[("cobra", "mark i") : ("viper", "mark ii")]
           max_speed  shield
cobra      mark i      12      2
           mark ii      0      4
sidewinder mark i      10     20
           mark ii      1      4
viper      mark ii      7      1
```

Please see the :ref:`user guide<advanced.advanced\_hierarchical>` for more details and explanations of advanced indexing.

****Assignment with Series****

When assigning a Series to `.loc[row_indexer, col_indexer]`, pandas aligns the Series by index labels, not by order or position.

Series assignment with `.loc` and index alignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
>>> s = pd.Series([10, 20], index=[1, 0]) # Note reversed order
>>> df.loc[:, "B"] = s # Aligns by index, not order
>>> df
   A  B
0  1  20.0
1  2  10.0
2  3   NaN
```

Methods inherited from `pandas.core.arraylike.OpsMixin`:

`__add__(self, other)`

Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ```other``` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
   height  weight
elk     1.5    500
moose    2.6    800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
      height  weight
elk        3.0   501.5
moose       4.1   801.5
```

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0   501.5
moose       3.1   801.5
```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0   501.5
moose       3.1   801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0   500.5
moose       4.1   800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk   NaN      NaN    NaN     NaN
moose NaN      NaN    NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0   500.5
moose       4.1   801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk       1.7     NaN
moose     3.0     NaN
```

`__and__(self, other)`

`__eq__(self, other)`
Return self==value.

`__floordiv__(self, other)`

`__ge__(self, other)`
Return self>=value.

`__gt__(self, other)`
Return self>value.

`__le__(self, other)`
Return self<=value.

`__lt__(self, other)`
Return self<value.

`__mod__(self, other)`

```

__mul__(self, other)

__ne__(self, other)
    Return self!=value.

__or__(self, other)
    Return self|value.

__pow__(self, other)

__radd__(self, other)

__rand__(self, other)

__rfloordiv__(self, other)

__rmod__(self, other)

__rmul__(self, other)

__ror__(self, other)
    Return value|self.

__rpow__(self, other)

__rsub__(self, other)

__rtruediv__(self, other)

__rxor__(self, other)

__sub__(self, other)

__truediv__(self, other)

__xor__(self, other)

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None

```

```

class DateOffset(pandas._libs.tslib.offsets.RelativeDeltaOffset):
    Standard kind of date increment used for a date range.

    Works exactly like the keyword argument form of relativedelta.
    Note that the positional argument form of relativedelta is not
    supported. Use of the keyword n is discouraged-- you would be better
    off specifying n in the keywords you use, but regardless it is
    there for you. n is needed for DateOffset subclasses.

    DateOffset works as follows. Each offset specify a set of dates
    that conform to the DateOffset. For example, Bday defines this
    set to be the set of dates that are weekdays (M-F). To test if a
    date is in the set of a DateOffset dateOffset we can use the
    is_on_offset method: dateOffset.is_on_offset(date).

    If a date is not on a valid date, the rollback and rollforward
    methods can be used to roll the date to the nearest valid date
    before/after the date.

    DateOffsets can be created to move dates forward a given number of
    valid dates. For example, Bday(2) can be added to a date to move
    it two business days forward. If the date does not start on a
    valid date, first it is moved to a valid date. Thus pseudo code
    is::

        def __add__(date):
            date = rollback(date) # does nothing if date is valid
            return date + <n number of periods>

    When a date offset is created for a negative number of periods,
    the date is first rolled forward. The pseudo code is::

        def __add__(date):
            date = rollforward(date) # does nothing if date is valid

```



```
return date + <n number of periods>
```

Zero presents a problem. Should it roll forward or back? We arbitrarily have it rollforward:

```
date + BDay(0) == BDay.rollforward(date)
```

Since 0 is a bit weird, we suggest avoiding its use.

Besides, adding a DateOffsets specified by the singular form of the date component can be used to replace certain component of the timestamp.

Attributes

n : int, default 1

The number of time periods the offset represents.

If specified without a temporal pattern, defaults to n days.

normalize : bool, default False

Whether to round the result of a DateOffset addition down to the previous midnight.

weekday : int {0, 1, ..., 6}, default 0

A specific integer for the day of the week.

- 0 is Monday
- 1 is Tuesday
- 2 is Wednesday
- 3 is Thursday
- 4 is Friday
- 5 is Saturday
- 6 is Sunday

Instead Weekday type from dateutil.relativedelta can be used.

- MO is Monday
- TU is Tuesday
- WE is Wednesday
- TH is Thursday
- FR is Friday
- SA is Saturday
- SU is Sunday.

**kwds

Temporal parameter that add to or replace the offset value.

Parameters that **add** to the offset (like Timedelta):

- years
- months
- weeks
- days
- hours
- minutes
- seconds
- milliseconds
- microseconds
- nanoseconds

Parameters that **replace** the offset value:

- year
- month
- day
- weekday
- hour
- minute
- second
- microsecond
- nanosecond.

See Also

dateutil.relativedelta.relativedelta : The relativedelta type is designed to be applied to an existing datetime and can replace specific components of that datetime, or represents an interval of time.

Examples

```

-----
>>> from pandas.tseries.offsets import DateOffset
>>> ts = pd.Timestamp('2017-01-01 09:10:11')
>>> ts + DateOffset(months=3)
Timestamp('2017-04-01 09:10:11')

>>> ts = pd.Timestamp('2017-01-01 09:10:11')
>>> ts + DateOffset(months=2)
Timestamp('2017-03-01 09:10:11')
>>> ts + DateOffset(day=31)
Timestamp('2017-01-31 09:10:11')

>>> ts + pd.DateOffset(hour=8)
Timestamp('2017-01-01 08:10:11')

Method resolution order:
  DateOffset
  pandas._libs.tslib.offsets.RelativeDeltaOffset
  pandas._libs.tslib.offsets.BaseOffset
  builtins.object

Methods defined here:
  __setattr__(self, name, value)
-----
Data descriptors defined here:
  __dict__
    dictionary for instance variables
  __weakref__
    list of weak references to the object
-----
Methods inherited from pandas._libs.tslib.offsets.RelativeDeltaOffset:
  __getstate__(self)
    Return a picklable state
  __init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.
  __reduce_cython__(self)
  __setstate__(self, state)
    Reconstruct an instance from a pickled state
  __setstate_cython__(self, __pyx_state)
  is_on_offset(self, dt: datetime) -> bool
-----
Static methods inherited from pandas._libs.tslib.offsets.RelativeDeltaOffset:
  __new__(*args, **kwargs) class method of pandas._libs.tslib.offsets.RelativeDeltaOffset
    Create and return a new object. See help(type) for accurate signature.
-----
Methods inherited from pandas._libs.tslib.offsets.BaseOffset:
  __add__(self, object, /)
  __eq__(self, value, /)
    Return self==value.
  __ge__(self, value, /)
    Return self>=value.
  __gt__(self, value, /)
    Return self>value.
  __hash__(self, /)
    Return hash(self).
  __le__(self, value, /)
    Return self<=value.

```

```

__lt__(self, value, /)
    Return self<value.

__mul__(self, object, /)

__ne__(self, value, /)
    Return self!=value.

__neg__(self, /)
    -self

__radd__(self, object, /)

__repr__(self, /)
    Return repr(self).

__rmul__(self, object, /)

__rsub__(self, object, /)

__sub__(self, object, /)

copy(self)
    Return a copy of the frequency.

    See Also
    -----
    tseries.offsets.Week.copy : Return a copy of Week offset.
    tseries.offsets.DateOffset.copy : Return a copy of date offset.
    tseries.offsets.MonthEnd.copy : Return a copy of MonthEnd offset.
    tseries.offsets.YearBegin.copy : Return a copy of YearBegin offset.

    Examples
    -----
    >>> freq = pd.DateOffset(1)
    >>> freq_copy = freq.copy()
    >>> freq is freq_copy
    False

is_month_end(self, ts)
    Return boolean whether a timestamp occurs on the month end.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_month_start : Return boolean whether a timestamp occurs on the month start.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)
    >>> freq = pd.offsets.Hour(5)
    >>> freq.is_month_end(ts)
    False

is_month_start(self, ts)
    Return boolean whether a timestamp occurs on the month start.

    This method determines if a given timestamp falls on the first day
    of a month, considering the frequency of the offset.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_month_end : Return boolean whether a timestamp occurs on the month end.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)

```

```

>>> freq = pd.offsets.Hour(5)
>>> freq.is_month_start(ts)
True

is_quarter_end(self, ts)
    Return boolean whether a timestamp occurs on the quarter end.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_quarter_start : Return boolean whether a timestamp
        occurs on the quarter start.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)
    >>> freq = pd.offsets.Hour(5)
    >>> freq.is_quarter_end(ts)
    False

is_quarter_start(self, ts)
    Return boolean whether a timestamp occurs on the quarter start.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_quarter_end : Return boolean whether a timestamp occurs on the quarter end.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)
    >>> freq = pd.offsets.Hour(5)
    >>> freq.is_quarter_start(ts)
    True

is_year_end(self, ts)
    Return boolean whether a timestamp occurs on the year end.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_year_start : Return boolean whether a timestamp occurs on the year start.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)
    >>> freq = pd.offsets.Hour(5)
    >>> freq.is_year_end(ts)
    False

is_year_start(self, ts)
    Return boolean whether a timestamp occurs on the year start.

    Parameters
    -----
    ts : Timestamp
        The timestamp to check.

    See Also
    -----
    is_year_end : Return boolean whether a timestamp occurs on the year end.

    Examples
    -----
    >>> ts = pd.Timestamp(2022, 1, 1)

```

```

>>> freq = pd.offsets.Hour(5)
>>> freq.is_year_start(ts)
True

```

`rollback(self, dt) -> datetime`
Roll provided date backward to next offset only if not on offset.

If the provided date is already on an offset, it is returned unchanged.
Otherwise, the date is rolled backward to the previous offset boundary.

Parameters

dt : datetime or Timestamp
Timestamp to rollback.

Returns

Timestamp
Rolled timestamp if not on offset, otherwise unchanged timestamp.

See Also

rollforward : Roll provided date forward to next offset only if not on offset.
is_on_offset : Return boolean whether a timestamp intersects with this frequency.

Examples

>>> ts = pd.Timestamp("2025-01-15 09:00:00")
>>> offset = pd.tseries.offsets.MonthEnd()

Timestamp is not on the offset (not a month end), so it rolls backward:

```

>>> offset.rollback(ts)
Timestamp('2024-12-31 00:00:00')

```

If the timestamp is already on the offset, it remains unchanged:

```

>>> ts_on_offset = pd.Timestamp("2025-01-31")
>>> offset.rollback(ts_on_offset)
Timestamp('2025-01-31 00:00:00')

```

`rollforward(self, dt) -> datetime`
Roll provided date forward to next offset only if not on offset.

If the provided date is already on an offset, it is returned unchanged.
Otherwise, the date is rolled forward to the next offset boundary.

Parameters

dt : datetime or Timestamp
Timestamp to roll forward.

Returns

Timestamp
Rolled timestamp if not on offset, otherwise unchanged timestamp.

See Also

rollback : Roll provided date backward to previous offset only if not on offset.
is_on_offset : Return boolean whether a timestamp intersects with this frequency.

Examples

>>> ts = pd.Timestamp("2025-01-15 09:00:00")
>>> offset = pd.tseries.offsets.MonthEnd()

Timestamp is not on the offset (not a month end), so it rolls forward:

```

>>> offset.rollforward(ts)
Timestamp('2025-01-31 00:00:00')

```

If the timestamp is already on the offset, it remains unchanged:

```
>>> ts_on_offset = pd.Timestamp("2025-01-31")
>>> offset.rollforward(ts_on_offset)
Timestamp('2025-01-31 00:00:00')
```

Data descriptors inherited from pandas._libs.tslibs.offsets.BaseOffset:

base

Returns a copy of the calling offset object with n=1 and all other attributes equal.

freqstr

Return a string representing the frequency.

The string representation typically consists of the offset multiplier and a frequency alias (e.g., ``'D'`` for daily, ``'h'`` for hourly).

See Also

tseries.offsets.BusinessDay.freqstr :

Return a string representing an offset frequency in Business Days.

tseries.offsets.BusinessHour.freqstr :

Return a string representing an offset frequency in Business Hours.

tseries.offsets.Week.freqstr :

Return a string representing an offset frequency in Weeks.

tseries.offsets.Hour.freqstr :

Return a string representing an offset frequency in Hours.

Examples

```
>>> pd.DateOffset(5).freqstr
'<5 * DateOffsets>'
```

```
>>> pd.offsets.BusinessHour(2).freqstr
'2bh'
```

```
>>> pd.offsets.Nano().freqstr
'ns'
```

```
>>> pd.offsets.Nano(-3).freqstr
'-3ns'
```

kwds

Return a dict of extra parameters for the offset.

These parameters exclude ``n`` and ``normalize``, which are common to all offsets. The returned dictionary can be passed to the offset constructor to recreate the offset with the same settings.

See Also

tseries.offsets.DateOffset : The base class for all pandas date offsets.

tseries.offsets.WeekOfMonth : Represents the week of the month.

tseries.offsets.LastWeekOfMonth : Represents the last week of the month.

Examples

```
>>> pd.DateOffset(5).kwds
{}
```

```
>>> pd.offsets.FY5253Quarter().kwds
{'weekday': 0,
 'startingMonth': 1,
 'qtr_with_extra_week': 1,
 'variation': 'nearest'}
```

n

Return the count of the number of periods.

This represents how many multiples of the offset frequency should be applied. For example, ``Hour(5)`` has ``n=5``.

See Also

DateOffset.normalize : Return boolean whether the frequency can align with midnight.

Examples

```
>>> pd.offsets.Hour(5).n
5
```

```
>>> pd.offsets.Day(3).n
3
```

name

Return a string representing the base frequency.

See Also

tseries.offsets.Week : Represents a weekly offset.
DateOffset : Base class for all other offset classes.
tseries.offsets.Day : Represents a single day offset.
tseries.offsets.MonthEnd : Represents a monthly offset that snaps to the end of the month.

Examples

```
>>> pd.offsets.Hour().name
'h'
```

```
>>> pd.offsets.Hour(5).name
'h'
```

nanos

Returns an integer of the total number of nanoseconds for fixed frequencies.

Raises

ValueError
If the frequency is non-fixed.

See Also

tseries.offsets.Hour.nanos :
Returns an integer of the total number of nanoseconds.
tseries.offsets.Day.nanos :
Returns an integer of the total number of nanoseconds.

Examples

```
>>> pd.offsets.Week(n=1).nanos
ValueError: Week: weekday=None is a non-fixed frequency
```

normalize

Return boolean whether the frequency can align with midnight.

This is True for offsets that round the result of a DateOffset addition down to the previous midnight.

See Also

tseries.offsets.DateOffset : The standard kind of date increment.

Examples

```
>>> pd.offsets.Hour(5).normalize
False
```

```
>>> pd.offsets.Day(5).normalize
False
```

rule_code

Return a string representing the base frequency.

See Also

tseries.offsets.Hour.rule_code :
Returns a string representing the base frequency of 'h'.
tseries.offsets.Day.rule_code :
Returns a string representing the base frequency of 'D'.

Examples

```

>>> pd.offsets.Hour().rule_code
'h'

>>> pd.offsets.Week(5).rule_code
'W'
-----
Data and other attributes inherited from pandas._libs.tslibs.offsets.BaseOffset:
__array_priority__ = 1000

class DatetimeIndex(pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin)
    DatetimeIndex(
        data=None,
        freq: Frequency | lib.NoDefault = <no_default>,
        tz=<no_default>,
        ambiguous: TimeAmbiguous = 'raise',
        dayfirst: bool = False,
        yearfirst: bool = False,
        dtype: Dtype | None = None,
        copy: bool | None = None,
        name: Hashable | None = None
    ) -> Self

    Immutable ndarray-like of datetime64 data.

    Represented internally as int64, and which can be boxed to Timestamp objects
    that are subclasses of datetime and carry metadata.

    .. versionchanged:: 2.0.0
        The various numeric date/time attributes (:attr:`~DatetimeIndex.day`,
        :attr:`~DatetimeIndex.month`, :attr:`~DatetimeIndex.year` etc.) now have dtype
        ``int32``. Previously they had dtype ``int64``.

    Parameters
    -----
    data : array-like (1-dimensional)
        Datetime-like data to construct index with.
    freq : str or pandas offset object, optional
        One of pandas date offset strings or corresponding objects. The string
        'infer' can be passed in order to set the frequency of the index as the
        inferred frequency upon creation.
    tz : zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, datetime.tzinfo or str
        Set the Timezone of the data.
    ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
        When clocks moved backward due to DST, ambiguous times may arise.
        For example in Central European Time (UTC+01), when going from 03:00
        DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC
        and at 01:30:00 UTC. In such a situation, the 'ambiguous' parameter
        dictates how ambiguous times should be handled.

        - 'infer' will attempt to infer fall dst-transition hours based on
          order
        - bool-ndarray where True signifies a DST time, False signifies a
          non-DST time (note that this flag is only applicable for ambiguous
          times)
        - 'NaT' will return NaT where there are ambiguous times
        - 'raise' will raise a ValueError if there are ambiguous times.
    dayfirst : bool, default False
        If True, parse dates in `data` with the day first order.
    yearfirst : bool, default False
        If True parse dates in `data` with the year first order.
    dtype : numpy.dtype or DatetimeTZDtype or str, default None
        Note that the only NumPy dtype allowed is 'datetime64[ns]'.
    copy : bool, default None
        Whether to copy input data, only relevant for array, Series, and Index
        inputs (for other input, e.g. a list, a new array is created anyway).
        Defaults to True for array input and False for Index/Series.
        Set to False to avoid copying array input at your own risk (if you
        know the input data won't be modified elsewhere).
        Set to True to force copying Series/Index up front.
    name : label, default None
        Name to be stored in the index.

    Attributes
    -----
    year

```


month
day
hour
minute
second
microsecond
nanosecond
date
time
timetz
dayofyear
day_of_year
dayofweek
day_of_week
weekday
quarter
tz
freq
freqstr
is_month_start
is_month_end
is_quarter_start
is_quarter_end
is_year_start
is_year_end
is_leap_year
inferred_freq

Methods

normalize
strftime
snap
tz_convert
tz_localize
round
floor
ceil
to_period
to_pydatetime
to_series
to_frame
to_julian_date
month_name
day_name
mean
std

See Also

Index : The base pandas Index type.
TimedeltaIndex : Index of timedelta64 data.
PeriodIndex : Index of Period data.
to_datetime : Convert argument to datetime.
date_range : Create a fixed-frequency DatetimeIndex.

Notes

To learn more about the frequency strings, please see
:ref:`this link<timeseries.offset\_aliases>`.

Examples

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> idx
DatetimeIndex(['2020-01-01 10:00:00+00:00', '2020-02-01 11:00:00+00:00'],
              dtype='datetime64[us, UTC]', freq=None)
```

Method resolution order:

DatetimeIndex
pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin
pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin
pandas.core.indexes.extension.NDArrayBackedExtensionIndex
pandas.core.indexes.extension.ExtensionIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin

```
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
abc.ABC
builtins.object
```

Methods defined here:

```
__reduce__(self) from pandas.core.indexes.datetimes.DatetimeIndex
    Helper for pickle.
```

```
as_unit(self, unit: TimeUnit, round_ok: bool = True) -> Self from pandas.core.indexes.extensions
    Convert to a dtype with the given unit resolution.
```

The limits of timestamp representation depend on the chosen resolution.
Different resolutions can be converted to each other through `as_unit`.

Parameters

unit : {'s', 'ms', 'us', 'ns'}
round_ok : bool, default True
 If False and the conversion requires rounding, raise `ValueError`.

Returns

same type as self
 Converted to the specified unit.

See Also

`Timestamp.as_unit` : Convert to the given unit.

Examples

For :class:`pandas.DatetimeIndex`:

```
>>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])
>>> idx
DatetimeIndex(['2020-01-02 01:02:03.004005006'],
              dtype='datetime64[ns]', freq=None)
>>> idx.as_unit("s")
DatetimeIndex(['2020-01-02 01:02:03'], dtype='datetime64[s]', freq=None)
```

For :class:`pandas.TimedeltaIndex`:

```
>>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
>>> tdelta_idx
TimedeltaIndex(['1 days 00:03:00.000002042'],
              dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.as_unit("s")
TimedeltaIndex(['1 days 00:03:00'], dtype='timedelta64[s]', freq=None)
```

```
ceil(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.indexes.extensions._inherit_from_data.<locals>
    Perform ceil operation on the data to the specified `freq`.
```

Parameters

freq : str or Offset

The frequency level to ceil the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
 Only relevant for `DatetimeIndex`:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a `ValueError` if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default '
A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series
Index of the same type for a DatetimeIndex or TimedeltaIndex,
or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :
Perform floor operation on the data to the specified `freq`.
DatetimeIndex.snap :
Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

DatetimeIndex

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.ceil('h')
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',
               '2018-01-01 13:00:00'],
              dtype='datetime64[us]', freq=None)
```

Series

```
>>> pd.Series(rng).dt.ceil("h")
0    2018-01-01 12:00:00
1    2018-01-01 12:00:00
2    2018-01-01 13:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 01:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.ceil("h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.ceil("h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

day_name(self, locale=None) -> npt.NDArray[np.object_] from pandas.core.indexes.extension.i
Return the day names with specified locale.

Parameters

locale : str, optional

Locale determining the language in which to return the day name.
Default is English locale (``'en\_US.utf8'``). Use the command
``locale -a`` on your terminal on Unix systems to find your locale
language code.

Returns

Series or Index

Series or Index of day names.

See Also

DatetimeIndex.month_name : Return the month names with specified locale.

Examples

```
>>> s = pd.Series(pd.date_range(start="2018-01-01", freq="D", periods=3))
```

```
>>> s
```

```
0    2018-01-01
```

```
1    2018-01-02
```

```
2    2018-01-03
```

```
dtype: datetime64[us]
```

```
>>> s.dt.day_name()
```

```
0    Monday
```

```
1    Tuesday
```

```
2    Wednesday
```

```
dtype: str
```

```
>>> idx = pd.date_range(start="2018-01-01", freq="D", periods=3)
```

```
>>> idx
```

```
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03'],
```

```
              dtype='datetime64[us]', freq='D')
```

```
>>> idx.day_name()
```

```
Index(['Monday', 'Tuesday', 'Wednesday'], dtype='str')
```

Using the ``locale`` parameter you can set a different locale language,
for example: ``idx.day\_name(locale='pt\_BR.utf8')`` will return day
names in Brazilian Portuguese language.

```
>>> idx = pd.date_range(start="2018-01-01", freq="D", periods=3)
```

```
>>> idx
```

```
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03'],
```

```
              dtype='datetime64[us]', freq='D')
```

```
>>> idx.day_name(locale="pt_BR.utf8") # doctest: +SKIP
```

```
Index(['Segunda', 'Terça', 'Quarta'], dtype='str')
```

```
floor(
```

```
    self,
```

```
    freq,
```

```
    ambiguous: TimeAmbiguous = 'raise',
```

```
    nonexistent: TimeNonexistent = 'raise'
```

```
) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
```

Perform floor operation on the data to the specified ``freq``.

Parameters

freq : str or Offset

The frequency level to floor the index to. Must be a fixed
frequency like 's' (second) not 'ME' (month end). See
:ref:`frequency aliases <timeseries.offset\_aliases>` for
a list of possible ``freq`` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'

Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta,

default 'NaT'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series

Index of the same type for a DatetimeIndex or TimedeltaIndex, or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :

Perform floor operation on the data to the specified `freq`.

DatetimeIndex.snap :

Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, flooring will take place relative to the local ("wall") time and re-localized to the same timezone. When flooring near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
```

```
>>> rng
```

```
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.floor('h')
```

```
DatetimeIndex(['2018-01-01 11:00:00', '2018-01-01 12:00:00',
               '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.floor("h")
```

```
0    2018-01-01 11:00:00
```

```
1    2018-01-01 12:00:00
```

```
2    2018-01-01 12:00:00
```

```
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
```

```
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
```

```
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

get_loc(self, key) from pandas.core.indexes.datetimes.DatetimeIndex
Get integer location for requested label

Returns

```

loc : int

indexer_at_time(self, time, asof: bool = False) -> npt.NDArray[np.intp] from pandas.core.indexes.datetimes.DatetimeIndex
Return index locations of values at particular time of day.

Parameters
-----
time : datetime.time or str
    Time passed in either as object (datetime.time) or as string in
    appropriate format ("%H:%M", "%H%M", "%I:%M%p", "%I%M%p",
    "%H:%M:%S", "%H%M%S", "%I:%M:%S%p", "%I%M%S%p").
asof : bool, default False
    This parameter is currently not supported.

Returns
-----
np.ndarray[np.intp]
    Index locations of values at given `time` of day.

See Also
-----
indexer_between_time : Get index locations of values between particular
    times of day.
DataFrame.at_time : Select values at particular time of day.

Examples
-----
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00", "2/1/2020 11:00", "3/1/2020 10:00"]
... )
>>> idx.indexer_at_time("10:00")
array([0, 2])

indexer_between_time(
    self,
    start_time,
    end_time,
    include_start: bool = True,
    include_end: bool = True
) -> npt.NDArray[np.intp] from pandas.core.indexes.datetimes.DatetimeIndex
Return index locations of values between particular times of day.

Parameters
-----
start_time, end_time : datetime.time, str
    Time passed either as object (datetime.time) or as string in
    appropriate format ("%H:%M", "%H%M", "%I:%M%p", "%I%M%p",
    "%H:%M:%S", "%H%M%S", "%I:%M:%S%p", "%I%M%S%p").
include_start : bool, default True
    Include boundaries; whether to set start bound as closed or open.
include_end : bool, default True
    Include boundaries; whether to set end bound as closed or open.

Returns
-----
np.ndarray[np.intp]
    Index locations of values between particular times of day.

See Also
-----
indexer_at_time : Get index locations of values at particular time of day.
DataFrame.between_time : Select values between particular times of day.

Examples
-----
>>> idx = pd.date_range("2023-01-01", periods=4, freq="h")
>>> idx
DatetimeIndex(['2023-01-01 00:00:00', '2023-01-01 01:00:00',
               '2023-01-01 02:00:00', '2023-01-01 03:00:00'],
              dtype='datetime64[us]', freq='h')
>>> idx.indexer_between_time("00:00", "2:00", include_end=False)
array([0, 1])

isocalendar(self) -> DataFrame from pandas.core.indexes.datetimes.DatetimeIndex
Calculate year, week, and day according to the ISO 8601 standard.
Returns
-----

```

DataFrame
With columns year, week and day.
See Also

Timestamp.isocalendar : Function return a 3-tuple containing ISO year,
week number, and weekday for the given Timestamp object.
datetime.date.isocalendar : Return a named tuple object with
three components: year, week and weekday.

Examples

```
>>> idx = pd.date_range(start="2019-12-29", freq="D", periods=4)
>>> idx.isocalendar()
          year  week  day
2019-12-29  2019   52    7
2019-12-30  2020    1    1
2019-12-31  2020    1    2
2020-01-01  2020    1    3
>>> idx.isocalendar().week
2019-12-29    52
2019-12-30     1
2019-12-31     1
2020-01-01     1
Freq: D, Name: week, dtype: UInt32
```

month_name(self, locale=None) -> npt.NDArray[np.object_] from pandas.core.indexes.extension.
Return the month names with specified locale.

Parameters

locale : str, optional
Locale determining the language in which to return the month name.
Default is English locale (``'en\_US.utf8'``). Use the command
``locale -a`` on your terminal on Unix systems to find your locale
language code.

Returns

Series or Index
Series or Index of month names.

See Also

DatetimeIndex.day_name : Return the day names with specified locale.

Examples

```
>>> s = pd.Series(pd.date_range(start="2018-01", freq="ME", periods=3))
>>> s
0    2018-01-31
1    2018-02-28
2    2018-03-31
dtype: datetime64[us]
>>> s.dt.month_name()
0    January
1    February
2    March
dtype: str
```

```
>>> idx = pd.date_range(start="2018-01", freq="ME", periods=3)
>>> idx
DatetimeIndex(['2018-01-31', '2018-02-28', '2018-03-31'],
              dtype='datetime64[us]', freq='ME')
>>> idx.month_name()
Index(['January', 'February', 'March'], dtype='str')
```

Using the ``locale`` parameter you can set a different locale language,
for example: ``idx.month\_name(locale='pt\_BR.utf8')`` will return month
names in Brazilian Portuguese language.

```
>>> idx = pd.date_range(start="2018-01", freq="ME", periods=3)
>>> idx
DatetimeIndex(['2018-01-31', '2018-02-28', '2018-03-31'],
              dtype='datetime64[us]', freq='ME')
>>> idx.month_name(locale='pt_BR.utf8') # doctest: +SKIP
Index(['Janeiro', 'Fevereiro', 'Março'], dtype='str')
```

normalize(self) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
Convert times to midnight.

The time component of the date-time is converted to midnight i.e. 00:00:00. This is useful in cases, when the time does not matter. Length is unaltered. The timezones are unaffected.

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on Datetime Array/Index.

Returns

DatetimeArray, DatetimeIndex or Series
The same type as the original data. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

floor : Floor the datetimes to the specified freq.
ceil : Ceil the datetimes to the specified freq.
round : Round the datetimes to the specified freq.

Examples

```
>>> idx = pd.date_range(
...     start="2014-08-01 10:00", freq="h", periods=3, tz="Asia/Calcutta"
... )
>>> idx
DatetimeIndex(['2014-08-01 10:00:00+05:30',
                '2014-08-01 11:00:00+05:30',
                '2014-08-01 12:00:00+05:30'],
              dtype='datetime64[us, Asia/Calcutta]', freq='h')
>>> idx.normalize()
DatetimeIndex(['2014-08-01 00:00:00+05:30',
                '2014-08-01 00:00:00+05:30',
                '2014-08-01 00:00:00+05:30'],
              dtype='datetime64[us, Asia/Calcutta]', freq=None)
```

round(
 self,
 freq,
 ambiguous: TimeAmbiguous = 'raise',
 nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
Perform round operation on the data to the specified `freq`.

Parameters

freq : str or Offset

The frequency level to round the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta,
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

default 'NaT'

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series

Index of the same type for a DatetimeIndex or TimedeltaIndex,
or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :

Perform floor operation on the data to the specified `freq`.

DatetimeIndex.snap :

Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, rounding will take place relative to the local ("wall") time and re-localized to the same timezone. When rounding near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
```

```
>>> rng
```

```
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',  
              '2018-01-01 12:01:00'],  
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.round('h')
```

```
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',  
              '2018-01-01 12:00:00'],  
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.round("h")
```

```
0    2018-01-01 12:00:00
```

```
1    2018-01-01 12:00:00
```

```
2    2018-01-01 12:00:00
```

```
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
```

```
DatetimeIndex(['2021-10-31 02:00:00+01:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
```

```
DatetimeIndex(['2021-10-31 02:00:00+02:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

slice_indexer(self, start=None, end=None, step=None) from pandas.core.indexes.datetimes.DatetimeIndex

Return indexer for specified label slice.

Index.slice_indexer, customized to handle time slicing.

In addition to functionality provided by Index.slice_indexer, does the following:

- if both `start` and `end` are instances of `datetime.time`, it invokes `indexer\_between\_time`
- if `start` and `end` are both either string or None perform value-based selection in non-monotonic cases.

snap(self, freq: Frequency = 'S') -> DatetimeIndex from pandas.core.indexes.datetimes.DatetimeIndex

Snap time stamps to nearest occurring frequency.

Parameters

 freq : str, Timedelta, datetime.timedelta, or DateOffset, default 'S'
 Frequency strings can have multiples, e.g. '5h'. See
 :ref:`here <timeseries.offset\_aliases>` for a list of
 frequency aliases.

Returns

 DatetimeIndex
 Time stamps to nearest occurring `freq`.

See Also

 DatetimeIndex.round : Perform round operation on the data to the
 specified `freq`.
 DatetimeIndex.floor : Perform floor operation on the data to the
 specified `freq`.

Examples

 >>> idx = pd.DatetimeIndex(
 ... ["2023-01-01", "2023-01-02", "2023-02-01", "2023-02-02"],
 ... dtype="M8[ns]",
 ...)
 >>> idx
 DatetimeIndex(['2023-01-01', '2023-01-02', '2023-02-01', '2023-02-02'],
 dtype='datetime64[ns]', freq=None)
 >>> idx.snap("MS")
 DatetimeIndex(['2023-01-01', '2023-01-01', '2023-02-01', '2023-02-01'],
 dtype='datetime64[ns]', freq=None)

std(
 self,
 axis=None,
 dtype=None,
 out=None,
 ddof: int = 1,
 keepdims: bool = False,
 skipna: bool = True
) -> Timedelta from pandas.core.indexes.extension._inherit_from_data.<locals>
 Return sample standard deviation over requested axis.

Normalized by `N-1` by default. This can be changed using ``ddof``.

Parameters

 axis : int, optional
 Axis for the function to be applied on. For :class:`pandas.Series`
 this parameter is unused and defaults to ``None``.
 dtype : dtype, optional, default None
 Type to use in computing the standard deviation. For arrays of
 integer type the default is float64, for arrays of float types
 it is the same as the array type.
 out : ndarray, optional, default None
 Alternative output array in which to place the result. It must have
 the same shape as the expected output but the type (of the
 calculated values) will be cast if necessary.
 ddof : int, default 1
 Degrees of Freedom. The divisor used in calculations is `N - ddof`,
 where `N` represents the number of elements.
 keepdims : bool, optional
 If this is set to True, the axes which are reduced are left in the
 result as dimensions with size one. With this option, the result
 will broadcast correctly against the input array. If the default
 value is passed, then keepdims will not be passed through to the
 std method of sub-classes of ndarray, however any non-default value
 will be. If the sub-class method does not implement keepdims any
 exceptions will be raised.
 skipna : bool, default True
 Exclude NA/null values. If an entire row/column is ``NA``, the result
 will be ``NA``.

Returns

 Timedelta
 Standard deviation over requested axis.

See Also

`numpy.ndarray.std` : Returns the standard deviation of the array elements along given axis.

`Series.std` : Return sample standard deviation over requested axis.

Examples

For `:class:`pandas.DatetimeIndex``:

```
>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.std()
Timedelta('1 days 00:00:00')
```

`strftime(self, date_format)` -> Index from `pandas.core.indexes.datetimes.DatetimeIndex`
Convert to Index using specified `date_format`.

Return an Index of formatted strings specified by `date_format`, which supports the same string format as the python standard library. Details of the string format can be found in ``python string format doc <https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>``.

Formats supported by the C ``strftime`` API but not by the python string format doc (such as ``"%R"`, `"%r"```) are not officially supported and should be preferably replaced with their supported equivalents (such as ``"%H:%M"`, `"%I:%M:%S %p"```).
Note that ``PeriodIndex`` support additional directives, detailed in ``Period.strftime``.

Parameters

`date_format` : str
Date format string (e.g. `"%Y-%m-%d"`).

Returns

`ndarray[object]`
NumPy ndarray of formatted strings.

See Also

`to_datetime` : Convert the given argument to datetime.
`DatetimeIndex.normalize` : Return `DatetimeIndex` with times to midnight.
`DatetimeIndex.round` : Round the `DatetimeIndex` to the specified freq.
`DatetimeIndex.floor` : Floor the `DatetimeIndex` to the specified freq.
`Timestamp.strftime` : Format a single Timestamp.
`Period.strftime` : Format a single Period.

Examples

```
>>> rng = pd.date_range(pd.Timestamp("2018-03-10 09:00"), periods=3, freq="s")
>>> rng.strftime("%B %d, %Y, %r")
Index(['March 10, 2018, 09:00:00 AM', 'March 10, 2018, 09:00:01 AM',
      'March 10, 2018, 09:00:02 AM'],
      dtype='str')
```

`to_julian_date(self)` -> Index from `pandas.core.indexes.datetimes.DatetimeIndex`
Convert TimeStamp to a Julian Date.

This method returns the number of days as a float since noon January 1, 4713 BC.

https://en.wikipedia.org/wiki/Julian_day

Returns

`ndarray` or `Index`
Float values that represent each date in Julian Calendar.

See Also

`Timestamp.to_julian_date` : Equivalent method on ``Timestamp`` objects.

Examples

```

>>> idx = pd.DatetimeIndex(["2028-08-12 00:54", "2028-08-12 02:06"])
>>> idx.to_julian_date()
Index([2461995.5375, 2461995.5875], dtype='float64')

```

to_period(self, freq=None) -> PeriodArray from pandas.core.indexes.extension._inherit_from_d
Cast to PeriodArray/PeriodIndex at a particular frequency.

Converts DatetimeArray/Index to PeriodArray/PeriodIndex.

Parameters

freq : str or Period, optional
One of pandas' :ref:`period aliases <timeseries.period\_aliases>`
or a Period object. Will be inferred by default.

Returns

PeriodArray/PeriodIndex
Immutable ndarray holding ordinal values at a particular frequency.

Raises

ValueError
When converting a DatetimeArray/Index with non-regular values,
so that a frequency cannot be inferred.

See Also

PeriodIndex: Immutable ndarray holding ordinal values.
DatetimeIndex.to_pydatetime: Return DatetimeIndex as object.

Examples

>>> df = pd.DataFrame(
... {"y": [1, 2, 3]},
... index=pd.to_datetime(
... [
... "2000-03-31 00:00:00",
... "2000-05-31 00:00:00",
... "2000-08-31 00:00:00",
...]
...),
...)
>>> df.index.to_period("M")
PeriodIndex(['2000-03', '2000-05', '2000-08'],
 dtype='period[M]')

Infer the daily frequency

```

>>> idx = pd.date_range("2017-01-01", periods=2)
>>> idx.to_period()
PeriodIndex(['2017-01-01', '2017-01-02'],
            dtype='period[D]')

```

to_pydatetime(self) -> npt.NDArray[np.object_] from pandas.core.indexes.extension._inherit_f
Return an ndarray of ``datetime.datetime`` objects.

Returns

numpy.ndarray
An ndarray of ``datetime.datetime`` objects.

See Also

DatetimeIndex.to_julian_date : Converts Datetime Array to float64 ndarray
of Julian Dates.

Examples

>>> idx = pd.date_range("2018-02-27", periods=3)
>>> idx.to_pydatetime()
array([datetime.datetime(2018, 2, 27, 0, 0),
 datetime.datetime(2018, 2, 28, 0, 0),
 datetime.datetime(2018, 3, 1, 0, 0)], dtype=object)

tz_convert(self, tz) -> Self from pandas.core.indexes.dates.DatetimeIndex
Convert tz-aware Datetime Array/Index from one time zone to another.

Parameters

`tz` : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, datetime.tzinfo or None
Time zone for time. Corresponding timestamps would be converted to this time zone of the Datetime Array/Index. A `tz` of None will convert to UTC and remove the timezone information.

Returns

Array or Index
Datetime Array/Index with target `tz`.

Raises

TypeError
If Datetime Array/Index is tz-naive.

See Also

`DatetimeIndex.tz` : A timezone that has a variable offset from UTC.
`DatetimeIndex.tz_localize` : Localize tz-naive DatetimeIndex to a given time zone, or remove timezone from a tz-aware DatetimeIndex.

Examples

With the `tz` parameter, we can change the DatetimeIndex to other time zones:

```
>>> dti = pd.date_range(
...     start="2014-08-01 09:00", freq="h", periods=3, tz="Europe/Berlin"
... )
```

```
>>> dti
DatetimeIndex(['2014-08-01 09:00:00+02:00',
               '2014-08-01 10:00:00+02:00',
               '2014-08-01 11:00:00+02:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='h')
```

```
>>> dti.tz_convert("US/Central")
DatetimeIndex(['2014-08-01 02:00:00-05:00',
               '2014-08-01 03:00:00-05:00',
               '2014-08-01 04:00:00-05:00'],
              dtype='datetime64[us, US/Central]', freq='h')
```

With the ``tz=None``, we can remove the timezone (after converting to UTC if necessary):

```
>>> dti = pd.date_range(
...     start="2014-08-01 09:00", freq="h", periods=3, tz="Europe/Berlin"
... )
```

```
>>> dti
DatetimeIndex(['2014-08-01 09:00:00+02:00',
               '2014-08-01 10:00:00+02:00',
               '2014-08-01 11:00:00+02:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='h')
```

```
>>> dti.tz_convert(None)
DatetimeIndex(['2014-08-01 07:00:00',
               '2014-08-01 08:00:00',
               '2014-08-01 09:00:00'],
              dtype='datetime64[us]', freq='h')
```

```
tz_localize(
    self,
    tz,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.indexes.datetimes.DatetimeIndex
Localize tz-naive Datetime Array/Index to tz-aware Datetime Array/Index.
```

This method takes a time zone (`tz`) naive Datetime Array/Index object and makes this time zone aware. It does not move the time to another time zone.

This method can also be used to do the inverse -- to create a time

zone unaware object from an aware object. To that end, pass `tz=None`.

Parameters

`tz` : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, datetime.tzinfo or None
Time zone to convert timestamps to. Passing ``None`` will remove the time zone information preserving local time.

`ambiguous` : 'infer', 'NaT', bool array, default 'raise'
When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False signifies a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

`nonexistent` : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

Same type as self
Array/Index converted to the specified time zone.

Raises

`TypeError`
If the Datetime Array/Index is tz-aware and tz is not None.

See Also

`DatetimeIndex.tz_convert` : Convert tz-aware DatetimeIndex from one time zone to another.

Examples

```
>>> tz_naive = pd.date_range('2018-03-01 09:00', periods=3)
>>> tz_naive
DatetimeIndex(['2018-03-01 09:00:00', '2018-03-02 09:00:00',
               '2018-03-03 09:00:00'],
              dtype='datetime64[us]', freq='D')
```

Localize DatetimeIndex in US/Eastern time zone:

```
>>> tz_aware = tz_naive.tz_localize(tz='US/Eastern')
>>> tz_aware
DatetimeIndex(['2018-03-01 09:00:00-05:00',
               '2018-03-02 09:00:00-05:00',
               '2018-03-03 09:00:00-05:00'],
              dtype='datetime64[us, US/Eastern]', freq=None)
```

With the ``tz=None``, we can remove the time zone information while keeping the local time (not converted to UTC):

```
>>> tz_aware.tz_localize(None)
DatetimeIndex(['2018-03-01 09:00:00', '2018-03-02 09:00:00',
               '2018-03-03 09:00:00'],
              dtype='datetime64[us]', freq=None)
```

Be careful with DST changes. When there is sequential data, pandas can infer the DST time:

```
>>> s = pd.to_datetime(pd.Series(['2018-10-28 01:30:00',
...                               '2018-10-28 02:00:00',
...                               '2018-10-28 02:30:00',
...                               '2018-10-28 02:00:00',
...                               '2018-10-28 02:30:00',
...                               '2018-10-28 03:00:00',
...                               '2018-10-28 03:30:00']))
>>> s.dt.tz_localize('CET', ambiguous='infer')
0    2018-10-28 01:30:00+02:00
1    2018-10-28 02:00:00+02:00
2    2018-10-28 02:30:00+02:00
3    2018-10-28 02:00:00+01:00
4    2018-10-28 02:30:00+01:00
5    2018-10-28 03:00:00+01:00
6    2018-10-28 03:30:00+01:00
dtype: datetime64[us, CET]
```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```
>>> s = pd.to_datetime(pd.Series(['2018-10-28 01:20:00',
...                               '2018-10-28 02:36:00',
...                               '2018-10-28 03:46:00']))
>>> s.dt.tz_localize('CET', ambiguous=np.array([True, True, False]))
0    2018-10-28 01:20:00+02:00
1    2018-10-28 02:36:00+02:00
2    2018-10-28 03:46:00+01:00
dtype: datetime64[us, CET]
```

If the DST transition causes nonexistent times, you can shift these dates forward or backwards with a timedelta object or `'shift_forward'` or `'shift_backwards'`.

```
>>> s = pd.to_datetime(pd.Series(['2015-03-29 02:30:00',
...                               '2015-03-29 03:30:00'], dtype="M8[ns]"))
>>> s.dt.tz_localize('Europe/Warsaw', nonexistent='shift_forward')
0    2015-03-29 03:00:00+02:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]

>>> s.dt.tz_localize('Europe/Warsaw', nonexistent='shift_backward')
0    2015-03-29 01:59:59.999999999+01:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]

>>> s.dt.tz_localize('Europe/Warsaw', nonexistent=pd.Timedelta('1h'))
0    2015-03-29 03:30:00+02:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]
```

Static methods defined here:

```
__new__(
    cls,
    data=None,
    freq: Frequency | lib.NoDefault = <no_default>,
    tz=<no_default>,
    ambiguous: TimeAmbiguous = 'raise',
    dayfirst: bool = False,
    yearfirst: bool = False,
    dtype: Dtype | None = None,
    copy: bool | None = None,
    name: Hashable | None = None
) -> Self from pandas.core.indexes.datetimes.DatetimeIndex
    Create and return a new object. See help(type) for accurate signature.
```

Readonly properties defined here:

`inferred_type`
Return a string of the type inferred from the values.

See Also

Index.dtype : Return the dtype object of the underlying data.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.inferred_type
'integer'

Data descriptors defined here:

date

Returns numpy array of python :class:`datetime.date` objects.

Namely, the date part of Timestamps without time and
timezone information.

See Also

DatetimeIndex.time : Returns numpy array of :class:`datetime.time` objects.

The time part of the Timestamps.

DatetimeIndex.year : The year of the datetime.

DatetimeIndex.month : The month as January=1, December=12.

DatetimeIndex.day : The day of the datetime.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.date
0    2020-01-01
1    2020-02-01
dtype: object
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.date
array([datetime.date(2020, 1, 1), datetime.date(2020, 2, 1)], dtype=object)
```

day

The day of the datetime.

See Also

DatetimeIndex.year: The year of the datetime.

DatetimeIndex.month: The month as January=1, December=12.

DatetimeIndex.hour: The hours of the datetime.

Examples

>>> datetime_series = pd.Series(
... pd.date_range("2000-01-01", periods=3, freq="D")
...)
>>> datetime_series
0 2000-01-01
1 2000-01-02
2 2000-01-03
dtype: datetime64[us]
>>> datetime_series.dt.day
0 1
1 2
2 3
dtype: int32

day_of_week

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

Series.dt.dayofweek : Alias.

Series.dt.weekday : Alias.

Series.dt.day_name : Returns the name of the day of the week.

Examples

```
>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
```

```
>>> s.dt.dayofweek
```

```
2016-12-31    5
```

```
2017-01-01    6
```

```
2017-01-02    0
```

```
2017-01-03    1
```

```
2017-01-04    2
```

```
2017-01-05    3
```

```
2017-01-06    4
```

```
2017-01-07    5
```

```
2017-01-08    6
```

```
Freq: D, dtype: int32
```

day_of_year

The ordinal day of the year.

See Also

DatetimeIndex.dayofweek : The day of the week with Monday=0, Sunday=6.

DatetimeIndex.day : The day of the datetime.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
```

```
>>> s = pd.to_datetime(s)
```

```
>>> s
```

```
0    2020-01-01 10:00:00+00:00
```

```
1    2020-02-01 11:00:00+00:00
```

```
dtype: datetime64[us, UTC]
```

```
>>> s.dt.dayofyear
```

```
0     1
```

```
1    32
```

```
dtype: int32
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",  
...                          "2/1/2020 11:00:00+00:00"])
```

```
>>> idx.dayofyear
```

```
Index([1, 32], dtype='int32')
```

dayofweek

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

```
-----
Series.dt.dayofweek : Alias.
Series.dt.weekday : Alias.
Series.dt.day_name : Returns the name of the day of the week.
```

Examples

```
-----
>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
>>> s.dt.dayofweek
2016-12-31    5
2017-01-01    6
2017-01-02    0
2017-01-03    1
2017-01-04    2
2017-01-05    3
2017-01-06    4
2017-01-07    5
2017-01-08    6
Freq: D, dtype: int32
```

dayofyear

The ordinal day of the year.

See Also

```
-----
DatetimeIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
DatetimeIndex.day : The day of the datetime.
```

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.dayofyear
0     1
1    32
dtype: int32
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",
...                          "2/1/2020 11:00:00+00:00"])
>>> idx.dayofyear
Index([1, 32], dtype='int32')
```

days_in_month

The number of days in the month.

See Also

```
-----
Series.dt.day : Return the day of the month.
Series.dt.is_month_end : Return a boolean indicating if the
                        date is the last day of the month.
Series.dt.is_month_start : Return a boolean indicating if the
                        date is the first day of the month.
Series.dt.month : Return the month as January=1 through December=12.
```

Examples

```
-----
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.daysinmonth
0     31
1     29
dtype: int32
```

daysinmonth

The number of days in the month.

See Also

Series.dt.day : Return the day of the month.

Series.dt.is_month_end : Return a boolean indicating if the date is the last day of the month.

Series.dt.is_month_start : Return a boolean indicating if the date is the first day of the month.

Series.dt.month : Return the month as January=1 through December=12.

Examples

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
```

```
>>> s = pd.to_datetime(s)
```

```
>>> s
```

```
0    2020-01-01 10:00:00+00:00
```

```
1    2020-02-01 11:00:00+00:00
```

```
dtype: datetime64[us, UTC]
```

```
>>> s.dt.daysinmonth
```

```
0     31
```

```
1     29
```

```
dtype: int32
```

dtype

The dtype for the DatetimeArray.

.. warning::

A future version of pandas will change dtype to never be a ``numpy.dtype``. Instead, :attr:`DatetimeArray.dtype` will always be an instance of an ``ExtensionDtype`` subclass.

Returns

numpy.dtype or DatetimeTZDtype

If the values are tz-naive, then ``np.dtype('datetime64[ns]')`` is returned.

If the values are tz-aware, then the ``DatetimeTZDtype`` is returned.

hour

The hours of the datetime.

See Also

DatetimeIndex.day: The day of the datetime.

DatetimeIndex.minute: The minutes of the datetime.

DatetimeIndex.second: The seconds of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="h")
... )
```

```
>>> datetime_series
```

```
0    2000-01-01 00:00:00
```

```
1    2000-01-01 01:00:00
```

```
2    2000-01-01 02:00:00
```

```
dtype: datetime64[us]
```

```
>>> datetime_series.dt.hour
```

```
0     0
```

```
1     1
```

```
2     2
```

```
dtype: int32
```

is_leap_year

Boolean indicator if the date belongs to a leap year.

A leap year is a year, which has 366 days (instead of 365) including 29th of February as an intercalary day.

Leap years are years which are multiples of four with the exception of years divisible by 100 but not by 400.

Returns

Series or ndarray

Booleans indicating if dates belong to a leap year.

See Also

DatetimeIndex.is_year_end : Indicate whether the date is the last day of the year.
DatetimeIndex.is_year_start : Indicate whether the date is the first day of a year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> idx = pd.date_range("2012-01-01", "2015-01-01", freq="YE")
>>> idx
DatetimeIndex(['2012-12-31', '2013-12-31', '2014-12-31'],
              dtype='datetime64[us]', freq='YE-DEC')
>>> idx.is_leap_year
array([ True, False, False])

>>> dates_series = pd.Series(idx)
>>> dates_series
0    2012-12-31
1    2013-12-31
2    2014-12-31
dtype: datetime64[us]
>>> dates_series.dt.is_leap_year
0     True
1    False
2    False
dtype: bool
```

is_month_end

Indicates whether the date is the last day of the month.

Returns

Series or array
For Series, returns a Series with boolean values.
For DatetimeIndex, returns a boolean array.

See Also

is_month_start : Return a boolean indicating whether the date is the first day of the month.
is_month_end : Return a boolean indicating whether the date is the last day of the month.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> s = pd.Series(pd.date_range("2018-02-27", periods=3))
>>> s
0    2018-02-27
1    2018-02-28
2    2018-03-01
dtype: datetime64[us]
>>> s.dt.is_month_start
0    False
1    False
2     True
dtype: bool
>>> s.dt.is_month_end
0    False
1     True
2    False
dtype: bool

>>> idx = pd.date_range("2018-02-27", periods=3)
>>> idx.is_month_start
array([False, False,  True])
>>> idx.is_month_end
array([False,  True, False])
```

```

is_month_start
    Indicates whether the date is the first day of the month.

    Returns
    -----
    Series or array
        For Series, returns a Series with boolean values.
        For DatetimeIndex, returns a boolean array.

    See Also
    -----
    is_month_start : Return a boolean indicating whether the date
        is the first day of the month.
    is_month_end : Return a boolean indicating whether the date
        is the last day of the month.

    Examples
    -----
    This method is available on Series with datetime values under
    the ``.dt`` accessor, and directly on DatetimeIndex.

    >>> s = pd.Series(pd.date_range("2018-02-27", periods=3))
    >>> s
    0    2018-02-27
    1    2018-02-28
    2    2018-03-01
    dtype: datetime64[us]
    >>> s.dt.is_month_start
    0    False
    1    False
    2     True
    dtype: bool
    >>> s.dt.is_month_end
    0    False
    1     True
    2    False
    dtype: bool

    >>> idx = pd.date_range("2018-02-27", periods=3)
    >>> idx.is_month_start
    array([False, False,  True])
    >>> idx.is_month_end
    array([False,  True, False])

```

is_normalized
Returns True if all of the dates are at midnight ("no time")

is_quarter_end
Indicator for whether the date is the last day of a quarter.

```

    Returns
    -----
    is_quarter_end : Series or DatetimeIndex
        The same type as the original data with boolean values. Series will
        have the same name and index. DatetimeIndex will have the same
        name.

    See Also
    -----
    quarter : Return the quarter of the date.
    is_quarter_start : Similar property indicating the quarter start.

    Examples
    -----
    This method is available on Series with datetime values under
    the ``.dt`` accessor, and directly on DatetimeIndex.

    >>> df = pd.DataFrame({'dates': pd.date_range("2017-03-30",
    ...                                           periods=4)})
    >>> df.assign(quarter=df.dates.dt.quarter,
    ...         is_quarter_end=df.dates.dt.is_quarter_end)

```

	dates	quarter	is_quarter_end
0	2017-03-30	1	False
1	2017-03-31	1	True
2	2017-04-01	2	False
3	2017-04-02	2	False

```

>>> idx = pd.date_range('2017-03-30', periods=4)
>>> idx
DatetimeIndex(['2017-03-30', '2017-03-31', '2017-04-01', '2017-04-02'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_quarter_end
array([False,  True,  False,  False])

```

is_quarter_start
Indicator for whether the date is the first day of a quarter.

Returns

is_quarter_start : Series or DatetimeIndex
The same type as the original data with boolean values. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

quarter : Return the quarter of the date.
is_quarter_end : Similar property for indicating the quarter end.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```

>>> df = pd.DataFrame({'dates': pd.date_range("2017-03-30",
...                                           periods=4)})
>>> df.assign(quarter=df.dates.dt.quarter,
...          is_quarter_start=df.dates.dt.is_quarter_start)

```

	dates	quarter	is_quarter_start
0	2017-03-30	1	False
1	2017-03-31	1	False
2	2017-04-01	2	True
3	2017-04-02	2	False

```

>>> idx = pd.date_range('2017-03-30', periods=4)
>>> idx
DatetimeIndex(['2017-03-30', '2017-03-31', '2017-04-01', '2017-04-02'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_quarter_start
array([False,  False,  True,  False])

```

is_year_end
Indicate whether the date is the last day of the year.

Returns

Series or DatetimeIndex
The same type as the original data with boolean values. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

is_year_start : Similar property indicating the start of the year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```

>>> dates = pd.Series(pd.date_range("2017-12-30", periods=3))
>>> dates
0    2017-12-30
1    2017-12-31
2    2018-01-01
dtype: datetime64[us]

>>> dates.dt.is_year_end
0    False
1     True
2    False
dtype: bool

```

```

>>> idx = pd.date_range("2017-12-30", periods=3)
>>> idx
DatetimeIndex(['2017-12-30', '2017-12-31', '2018-01-01'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_year_end
array([False,  True, False])

```

is_year_start
Indicate whether the date is the first day of a year.

Returns

Series or DatetimeIndex
The same type as the original data with boolean values. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

is_year_end : Similar property indicating the last day of the year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```

>>> dates = pd.Series(pd.date_range("2017-12-30", periods=3))
>>> dates
0    2017-12-30
1    2017-12-31
2    2018-01-01
dtype: datetime64[us]

>>> dates.dt.is_year_start
0    False
1    False
2     True
dtype: bool

>>> idx = pd.date_range("2017-12-30", periods=3)
>>> idx
DatetimeIndex(['2017-12-30', '2017-12-31', '2018-01-01'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_year_start
array([False, False,  True])

```

This method, when applied to Series with datetime values under the ``.dt`` accessor, will lose information about Business offsets.

```

>>> dates = pd.Series(pd.date_range("2020-10-30", periods=4, freq="BYS"))
>>> dates
0    2021-01-01
1    2022-01-03
2    2023-01-02
3    2024-01-01
dtype: datetime64[us]

>>> dates.dt.is_year_start
0     True
1    False
2    False
3     True
dtype: bool

>>> idx = pd.date_range("2020-10-30", periods=4, freq="BYS")
>>> idx
DatetimeIndex(['2021-01-01', '2022-01-03', '2023-01-02', '2024-01-01'],
              dtype='datetime64[us]', freq='BYS-JAN')

>>> idx.is_year_start
array([ True,  True,  True,  True])

```

microsecond
The microseconds of the datetime.

See Also

DatetimeIndex.second: The seconds of the datetime.

DatetimeIndex.nanosecond: The nanoseconds of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="us")
... )
>>> datetime_series
0    2000-01-01 00:00:00.000000
1    2000-01-01 00:00:00.000001
2    2000-01-01 00:00:00.000002
dtype: datetime64[us]
>>> datetime_series.dt.microsecond
0      0
1      1
2      2
dtype: int32
```

minute

The minutes of the datetime.

See Also

DatetimeIndex.hour: The hours of the datetime.

DatetimeIndex.second: The seconds of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="min")
... )
>>> datetime_series
0    2000-01-01 00:00:00
1    2000-01-01 00:01:00
2    2000-01-01 00:02:00
dtype: datetime64[us]
>>> datetime_series.dt.minute
0      0
1      1
2      2
dtype: int32
```

month

The month as January=1, December=12.

See Also

DatetimeIndex.year: The year of the datetime.

DatetimeIndex.day: The day of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="ME")
... )
>>> datetime_series
0    2000-01-31
1    2000-02-29
2    2000-03-31
dtype: datetime64[us]
>>> datetime_series.dt.month
0      1
1      2
2      3
dtype: int32
```

nanosecond

The nanoseconds of the datetime.

See Also

DatetimeIndex.second: The seconds of the datetime.

DatetimeIndex.microsecond: The microseconds of the datetime.

Examples

```
-----
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="ns")
... )
>>> datetime_series
0    2000-01-01 00:00:00.000000000
1    2000-01-01 00:00:00.000000001
2    2000-01-01 00:00:00.000000002
dtype: datetime64[ns]
>>> datetime_series.dt.nanosecond
0      0
1      1
2      2
dtype: int32
```

quarter

The quarter of the date.

See Also

DatetimeIndex.snap : Snap time stamps to nearest occurring frequency.
DatetimeIndex.time : Returns numpy array of datetime.time objects.
The time part of the Timestamps.

Examples

```
-----
For Series:

>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "4/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-04-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.quarter
0      1
1      2
dtype: int32
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",
...                          "2/1/2020 11:00:00+00:00"])
>>> idx.quarter
Index([1, 1], dtype='int32')
```

second

The seconds of the datetime.

See Also

DatetimeIndex.minute: The minutes of the datetime.
DatetimeIndex.microsecond: The microseconds of the datetime.
DatetimeIndex.nanosecond: The nanoseconds of the datetime.

Examples

```
-----
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="s")
... )
>>> datetime_series
0    2000-01-01 00:00:00
1    2000-01-01 00:00:01
2    2000-01-01 00:00:02
dtype: datetime64[us]
>>> datetime_series.dt.second
0      0
1      1
2      2
dtype: int32
```

time

Returns numpy array of :class:`datetime.time` objects.

The time part of the Timestamps.

See Also

DatetimeIndex.timetz : Returns numpy array of :class:`datetime.time` objects with timezones. The time part of the Timestamps.
DatetimeIndex.date : Returns numpy array of python :class:`datetime.date` objects. Namely, the date part of Timestamps without time and timezone information.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.time
0    10:00:00
1    11:00:00
dtype: object
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.time
array([datetime.time(10, 0), datetime.time(11, 0)], dtype=object)
```

timetz

Returns numpy array of :class:`datetime.time` objects with timezones.

The time part of the Timestamps.

See Also

DatetimeIndex.time : Returns numpy array of :class:`datetime.time` objects. The time part of the Timestamps.
DatetimeIndex.tz : Return the timezone.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.timetz
0    10:00:00+00:00
1    11:00:00+00:00
dtype: object
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.timetz
array([datetime.time(10, 0, tzinfo=datetime.timezone.utc),
       datetime.time(11, 0, tzinfo=datetime.timezone.utc)], dtype=object)
```

tz

Return the timezone.

Returns

zoneinfo.ZoneInfo,, datetime.tzinfo, pytz.tzinfo.BaseTZInfo, dateutil.tz.tz.tzfile, or None
Returns None when the array is tz-naive.

See Also

DatetimeIndex.tz_localize : Localize tz-naive DatetimeIndex to a given time zone, or remove timezone from a tz-aware DatetimeIndex.
DatetimeIndex.tz_convert : Convert tz-aware DatetimeIndex from one time zone to another.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.tz
datetime.timezone.utc
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.tz
datetime.timezone.utc
```

tzinfo

Alias for tz attribute

weekday

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

Series.dt.dayofweek : Alias.

Series.dt.weekday : Alias.

Series.dt.day_name : Returns the name of the day of the week.

Examples

```
>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
>>> s.dt.dayofweek
2016-12-31    5
2017-01-01    6
2017-01-02    0
2017-01-03    1
2017-01-04    2
2017-01-05    3
2017-01-06    4
2017-01-07    5
2017-01-08    6
Freq: D, dtype: int32
```

year

The year of the datetime.

See Also

DatetimeIndex.month: The month as January=1, December=12.

DatetimeIndex.day: The day of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="YE")
... )
>>> datetime_series
```

```

0    2000-12-31
1    2001-12-31
2    2002-12-31
dtype: datetime64[us]
>>> datetime_series.dt.year
0    2000
1    2001
2    2002
dtype: int32

```

Data and other attributes defined here:

`__abstractmethods__` = frozenset()

`__annotations__` = {'_data': 'DatetimeArray', '_values': 'DatetimeArray...'

Methods inherited from `pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin`:

`delete(self, loc) -> Self`

Make new Index with passed location(-s) deleted.

Parameters

`loc` : int or list of int

Location of item(-s) which will be deleted.

Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

`numpy.delete` : Delete any rows and column from NumPy array (ndarray).

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete(1)
Index(['a', 'c'], dtype='str')
```

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')
```

`insert(self, loc: int, item)`

Make new Index inserting new item at location.

Follows Python `numpy.insert` semantics for negative values.

Parameters

`loc` : int

The integer location where the new item will be inserted.

`item` : object

The new item to be inserted into the Index.

Returns

Index

Returns a new Index object resulting from inserting the specified item at the specified location within the original Index.

See Also

`Index.append` : Append a collection of Indexes together.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

`shift(self, periods: int = 1, freq=None) -> Self`

Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes

by a specified time increment a given number of times.

Parameters

periods : int, default 1
Number of periods (or increments) to shift by,
can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or string, optional
Frequency increment to shift by.
If None, the index is shifted by its own `freq` attribute.
Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

pandas.DatetimeIndex
Shifted index.

See Also

Index.shift : Shift values of Index.
PeriodIndex.shift : Shift values of PeriodIndex.

```
take(  
    self,  
    indices,  
    axis: Axis = 0,  
    allow_fill: bool = True,  
    fill_value=None,  
    **kwargs  
) -> Self
```

Return a new Index of the values selected by the indices.
For internal compatibility with numpy arrays.

Parameters

indices : array-like
Indices to be taken.
axis : {0 or 'index'}, optional
The axis over which to select values, always 0 or 'index'.
allow_fill : bool, default True
How to handle negative values in `indices`.
* False: negative values in `indices` indicate positional indices
from the right (the default). This is similar to
:func:`numpy.take`.
* True: negative values in `indices` indicate
missing values. These values are set to `fill\_value`. Any other
other negative values raise a ``ValueError``.
fill_value : scalar, default None
If allow_fill=True and fill_value is not None, indices specified by
-1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
**kwargs
Required for compatibility with numpy.

Returns

Index
An index formed of elements at the given indices. Will be the same
type as self, except for RangeIndex.

See Also

numpy.ndarray.take: Return an array formed from the
elements of a at the given indices.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])  
>>> idx.take([2, 2, 1, 2])  
Index(['c', 'c', 'b', 'c'], dtype='str')
```

Readonly properties inherited from pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin:

unit

values

Return an array representing the data in the Index.

.. warning::

We recommend using `:attr:`Index.array`` or `:meth:`Index.to_numpy``, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

`Index.array` : Reference to the underlying data.
`Index.to_numpy` : A NumPy array representing the underlying data.

Examples

For `:class:`pandas.Index``:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For `:class:`pandas.IntervalIndex``:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[[0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors inherited from `pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin`:

`inferred_freq`

Return the inferred frequency of the index.

Returns

str or None

A string representing a frequency generated by ```infer_freq```.
Returns ```None``` if the frequency cannot be inferred.

See Also

`DatetimeIndex.freqstr` : Return the frequency object as a string if it's set, otherwise ```None```.

Examples

For ```DatetimeIndex```:

```
>>> idx = pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
>>> idx.inferred_freq
'2D'
```

For ```TimedeltaIndex```:

```
>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
               dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.inferred_freq
'10D'
```

Methods inherited from `pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin`:

`__contains__(self, key: Any) -> bool`

Return a boolean indicating whether the provided key is in the index.

Parameters

key : label

The key to check if it is present in the index.

Returns

bool

Whether the key search is in the index.

Raises

TypeError

If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the list-like key is in the index.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> 2 in idx
True
>>> 6 in idx
False
```

equals(self, other: Any) -> bool

Determines if two Index objects contain the same elements.

mean(self, *, skipna: bool = True, axis: int | None = 0)

Return the mean value of the Array.

Parameters

skipna : bool, default True

Whether to ignore any NaT elements.

axis : int, optional, default 0

Axis for the function to be applied on.

Returns

scalar

Timestamp or Timedelta.

See Also

numpy.ndarray.mean : Returns the average of array elements along a given axis.

Series.mean : Return the mean value in a Series.

Notes

mean is only defined for Datetime and Timedelta dtypes, not for Period.

Examples

For :class:`pandas.DatetimeIndex`:

```
>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.mean()
Timestamp('2001-01-02 00:00:00')
```

For :class:`pandas.TimedeltaIndex`:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
              dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.mean()
Timedelta('2 days 00:00:00')
```

Readonly properties inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

asi8

freqstr

Return the frequency object as a string if it's set, otherwise None.

See Also

DatetimeIndex.inferred_freq : Returns a string representing a frequency generated by infer_freq.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
>>> idx.freqstr
'D'
```

The frequency can be inferred if there are more than 2 points:

```
>>> idx = pd.DatetimeIndex(
...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
... )
>>> idx.freqstr
'2D'
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
>>> idx.freqstr
'M'
```

Data descriptors inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

freq

Return the frequency object if it is set, otherwise None.

To learn more about the frequency strings, please see
:ref:`this link<timeseries.offset\_aliases>`.

See Also

DatetimeIndex.freq : Return the frequency object if it is set, otherwise None.
PeriodIndex.freq : Return the frequency object if it is set, otherwise None.

Examples

```
>>> datetimeindex = pd.date_range(
...     "2022-02-22 02:22:22", periods=10, tz="America/Chicago", freq="h"
... )
>>> datetimeindex
DatetimeIndex(['2022-02-22 02:22:22-06:00', '2022-02-22 03:22:22-06:00',
              '2022-02-22 04:22:22-06:00', '2022-02-22 05:22:22-06:00',
              '2022-02-22 06:22:22-06:00', '2022-02-22 07:22:22-06:00',
              '2022-02-22 08:22:22-06:00', '2022-02-22 09:22:22-06:00',
              '2022-02-22 10:22:22-06:00', '2022-02-22 11:22:22-06:00'],
              dtype='datetime64[us, America/Chicago]', freq='h')
>>> datetimeindex.freq
<Hour>
```

hasnans

resolution

Returns day, hour, minute, second, millisecond or microsecond

Methods inherited from Index:

__abs__(self) -> Index from pandas.core.indexes.base.Index

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
The array interface, return my values.


```

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index
__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
    Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
    Parameters
    -----
    memo, default None
        Standard signature. Unused

__getitem__(self, key) from pandas.core.indexes.base.Index
    Override numpy.ndarray's __getitem__ method to work as desired.

    This function adds lists and Series as valid boolean indexers
    (ndarrays only supports ndarray with dtype=bool).

    If resulting ndim != 1, plain ndarray is returned instead of
    corresponding `Index` subclass.

__iadd__(self, other) from pandas.core.indexes.base.Index

__invert__(self) -> Index from pandas.core.indexes.base.Index

__len__(self) -> int from pandas.core.indexes.base.Index
    Return the length of the Index.

__neg__(self) -> Index from pandas.core.indexes.base.Index

__pos__(self) -> Index from pandas.core.indexes.base.Index

__repr__(self) -> str_t from pandas.core.indexes.base.Index
    Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether all elements are Truthy.

    Parameters
    -----
    *args
        Required for compatibility with numpy.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    bool or array-like (if axis is specified)
        A single element array-like may be converted to bool.

    See Also
    -----
    Index.any : Return whether any element in an Index is True.
    Series.any : Return whether any element in a Series is True.
    Series.all : Return whether all elements in a Series are True.

    Notes
    -----
    Not a Number (NaN), positive infinity and negative infinity
    evaluate to True because these are not equal to zero.

    Examples
    -----
    True, because nonzero integers are considered True.

    >>> pd.Index([1, 2, 3]).all()
    True

    False, because ``0`` is considered False.

    >>> pd.Index([0, 1, 2]).all()

```

False

any(self, *args, **kwargs) from pandas.core.indexes.base.Index
Return whether any element is Truthy.

Parameters

*args

Required for compatibility with numpy.

**kwargs

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

Index.all : Return whether all elements are True.

Series.all : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
```

```
>>> index.any()
```

```
True
```

```
>>> index = pd.Index([0, 0, 0])
```

```
>>> index.any()
```

```
False
```

append(self, other: Index | Sequence[Index]) -> Index from pandas.core.indexes.base.Index
Append a collection of Index options together.

Parameters

other : Index or list/tuple of indices

Single Index or a collection of indices, which can be either a list or a
tuple.

Returns

Index

Returns a new Index object resulting from appending the provided other
indices to the original Index.

See Also

Index.insert : Make new Index inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx.append(pd.Index([4]))
```

```
Index([1, 2, 3, 4], dtype='int64')
```

argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base.Index
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations,
the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the maximum value.

See Also

Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmax : Return index label of the maximum values.
Series.idxmin : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base
Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations, the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the minimum value.

See Also

Series.argmin : Return position of the minimum value.
Series.argmax : Return position of the maximum value.
numpy.ndarray.argmin : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

`argsort(self, *args, **kwargs)` -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Return the integer indices that would sort the index.

Parameters

`*args`

Passed to ``numpy.ndarray.argsort``.

`**kwargs`

Passed to ``numpy.ndarray.argsort``.

Returns

`np.ndarray[np.intp]`

Integer indices that would sort the index if used as an indexer.

See Also

`numpy.argsort` : Similar method for NumPy arrays.

`Index.sort_values` : Return sorted copy of Index.

Examples

```
>>> idx = pd.Index(["b", "a", "d", "c"])
```

```
>>> idx
```

```
Index(['b', 'a', 'd', 'c'], dtype='str')
```

```
>>> order = idx.argsort()
```

```
>>> order
```

```
array([1, 0, 3, 2])
```

```
>>> idx[order]
```

```
Index(['a', 'b', 'c', 'd'], dtype='str')
```

`asof(self, label)` from `pandas.core.indexes.base.Index`

Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

`label` : object

The label up to which the method returns the latest index label.

Returns

object

The passed label if it is in the index. The previous label if the passed label is not in the sorted index or ``NaN`` if there is no such label.

See Also

`Series.asof` : Return the latest value in a Series up to the passed index.

`merge_asof` : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).

`Index.get_loc` : An ``asof`` is a thin wrapper around ``get_loc`` with `method='pad'`.

Examples

``Index.asof`` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
```

```
>>> idx.asof("2014-01-01")
```

```
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
```

```
'2014-01-02'
```

If all of the labels in the index are later than the passed label, `NaN` is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp]` from `pandas`
Return the locations (indices) of labels in the index.

As in the `:meth:`pandas.Index.asof``, if the label (a particular entry in `where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in `where``, -1 is returned.

`mask`` is used to ignore `NA`` values in the index during calculation.

Parameters

`where` : Index
An Index consisting of an array of timestamps.
`mask` : `np.ndarray[bool]`
Array of booleans denoting where values in the original data are not `NA``.

Returns

`np.ndarray[np.intp]`
An array of locations (indices) of the labels from the index which correspond to the return values of `:meth:`pandas.Index.asof`` for every element in `where``.

See Also

`Index.asof` : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use `mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by `dtype`. When conversion is impossible, a `TypeError` exception is raised.

Parameters

`dtype` : numpy dtype or pandas type
Note that any signed integer `dtype`` is treated as `'int64``, and any unsigned integer `dtype`` is treated as `'uint64``, regardless of the size.
`copy` : bool, default True
By default, `astype` always returns a newly allocated object. If `copy` is set to False and internal requirements on `dtype` are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index

Index with values cast to specified dtype.

See Also

`Index.dtype`: Return the dtype object of the underlying data.

`Index.dtypes`: Return the dtype object of the underlying data.

`Index.convert_dtypes`: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.astype("float")
```

```
Index([1.0, 2.0, 3.0], dtype='float64')
```

`copy(self, name: Hashable | None = None, deep: bool = False) -> Self` from `pandas.core.indexes`

Make a copy of this object.

Name is set on the new object.

Parameters

`name` : Label, optional

Set name for new object.

`deep` : bool, default False

If True attempts to make a deep copy of the Index.

Else makes a shallow copy.

Returns

Index

Index refer to new object which is a copy of this object.

See Also

`Index.delete`: Make new Index with passed location(-s) deleted.

`Index.drop`: Make new Index with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ```deep```, but if ```deep``` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> new_idx = idx.copy()
```

```
>>> idx is new_idx
```

```
False
```

`diff(self, periods: int = 1) -> Index` from `pandas.core.indexes.base.Index`

Computes the difference between consecutive values in the Index object.

If `periods` is greater than 1, computes the difference between values that are ```periods``` number of positions apart.

Parameters

`periods` : int, optional

The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

Index

A new Index object with the computed differences.

Examples

```
>>> import pandas as pd
```

```
>>> idx = pd.Index([10, 20, 30, 40, 50])
```

```
>>> idx.diff()
```

```
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')
```

```
difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index
    Return a new Index with elements of index not in `other`.
```

This is the set difference of two Index objects.

Parameters

```
other : Index or array-like
    Index object or an array-like object containing elements to be compared
    with the elements of the original Index.
sort : bool or None, default None
    Whether to sort the resulting index. By default, the
    values are attempted to be sorted, but any TypeError from
    incomparable elements is caught by pandas.

    * None : Attempt to sort the result, but catch any TypeErrors
    from comparing incomparable elements.
    * False : Do not sort the result.
    * True : Sort the result (which may raise TypeError).
```

Returns

Index

Returns a new Index object containing elements that are in the original Index but not in the `other` Index.

See Also

```
Index.symmetric_difference : Compute the symmetric difference of two Index
    objects.
Index.intersection : Form the intersection of two Index objects.
```

Examples

```
>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')
```

```
drop(
    self,
    labels: Index | np.ndarray | Iterable[Hashable],
    errors: IgnoreRaise = 'raise'
) -> Index from pandas.core.indexes.base.Index
    Make new Index with passed list of labels deleted.
```

Parameters

```
labels : array-like or scalar
    Array-like object or a scalar value, representing the labels to be removed
    from the Index.
errors : {'ignore', 'raise'}, default 'raise'
    If 'ignore', suppress error and existing labels are dropped.
```

Returns

Index

Will be same type as self, except for RangeIndex.

Raises

KeyError

If not all of the labels are found in the selected axis

See Also

```
Index.dropna : Return Index without NA/Nan values.
Index.drop_duplicates : Return Index with duplicate values removed.
```

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')
```

`drop_duplicates(self, *, keep: DropKeep = 'first') -> Self` from `pandas.core.indexes.base.Index`
Return Index with duplicate values removed.

Parameters

`keep` : {'first', 'last', ``False``}, default 'first'
- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

Returns

Index

A new Index object with the duplicate values removed.

See Also

`Series.drop_duplicates` : Equivalent method on Series.
`DataFrame.drop_duplicates` : Equivalent method on DataFrame.
`Index.duplicated` : Related method on Index, indicating duplicate Index values.

Examples

Generate a pandas.Index with duplicate values.

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])
```

The `'keep'` parameter controls which duplicate values are removed.
The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of `keep` is 'first'.

```
>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')
```

The value 'last' keeps the last occurrence for each set of duplicated entries.

```
>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')
```

The value ``False`` discards all sets of duplicated entries.

```
>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')
```

`droplevel(self, level: IndexLabel = 0)` from `pandas.core.indexes.base.Index`
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

`level` : int, str, or list-like, default 0
If a string is given, must be the name of a level
If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index after removing the requested level(s).

See Also

`Index.dropna` : Return Index without NA/Nan values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
            (2, 4, 6)],
```



```

        names=['x', 'y', 'z'])

>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
            names=['y', 'z'])

>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')

```

`dropna(self, how: AnyAll = 'any') -> Self` from `pandas.core.indexes.base.Index`
Return Index without NA/NaN values.

Parameters

how : {'any', 'all'}, default 'any'
If the Index is a MultiIndex, drop the value when any or all levels are NaN.

Returns

Index
Returns an Index object after removing NA/NaN values.

See Also

Index.fillna : Fill NA/NaN values with the specified value.
Index.isna : Detect missing values.

Examples

>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')

`duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_]` from `pandas.core.indexes`
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

keep : {'first', 'last', False}, default 'first'
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

np.ndarray[bool]
A numpy array of boolean values indicating duplicate index values.

See Also

Series.duplicated : Equivalent method on `pandas.Series`.
DataFrame.duplicated : Equivalent method on `pandas.DataFrame`.
Index.drop_duplicates : Remove duplicate values from Index.

Examples

By default, for each set of duplicated values, the first occurrence is

set to False and all others to True:

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])
```

which is equivalent to

```
>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])
```

`fillna(self, value)` from `pandas.core.indexes.base.Index`
Fill NA/Nan values with the specified value.

Parameters

value : scalar

Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index

NA/Nan values replaced with `value`.

See Also

`DataFrame.fillna` : Fill NaN values of a `DataFrame`.

`Series.fillna` : Fill NaN Values of a `Series`.

Examples

```
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

```
get_indexer(
    self,
    target,
    method: ReindexMethod | None = None,
    limit: int | None = None,
    tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
```

Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

target : Index

An iterable containing the values to be used for computing indexer.

method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional

* default: exact matches only.

* pad / ffill: find the PREVIOUS index value if no exact match.

* backfill / bfill: use NEXT index value if no exact match

* nearest: use the NEAREST index value if no exact match. Tied

distances are broken by preferring the larger index value.

limit : int, optional

Maximum number of consecutive labels in ``target`` to match for inexact matches.

tolerance : optional

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

`np.ndarray[np.intp]`

Integers from 0 to `n - 1` indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by `-1`.

See Also

`Index.get_indexer_for` : Returns an indexer even when non-unique.

`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Notes

Returns `-1` for unmatched values, for further explanation see the example below.

Examples

```
>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([ 1,  2, -1])
```

Notice that the return value is an array of locations in `index` and `index` is marked by `-1`, as it is not in `index`.

`get_indexer_for(self, target)` -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Guaranteed return of an indexer even when non-unique.

This dispatches to `get_indexer` or `get_indexer_non_unique` as appropriate.

Parameters

`target` : Index

An iterable containing the values to be used for computing indexer.

Returns

`np.ndarray[np.intp]`

List of indices.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.

`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Examples

```
>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])
```

`get_indexer_non_unique(self, target)` -> `tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]` from `pandas.core.indexes.base.Index`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index

An iterable containing the values to be used for computing indexer.

Returns

`indexer` : `np.ndarray[np.intp]`

Integers from 0 to `n - 1` indicating that the index at these

positions matches the corresponding target values. Missing values in the target are marked by -1.
missing : np.ndarray[np.intp]
An indexer into the target of the values not found.
These correspond to the -1 in the indexer array.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned ``indexer`` contains only integers equal to -1. It demonstrates that there's no match between the index and the ``target`` values at these positions. The mask [0, 1, 2] in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in ``index``. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

get_level_values = _get_level_values(self, level) -> Index

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position of given label.

Parameters

label : object
The label for which to calculate the slice bound.
side : {'left', 'right'}
if 'left' return leftmost position of given label.
if 'right' return one-past-the-rightmost position of given label.

Returns

int
Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested label.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.get_slice_bound(3, "left")
3
>>> idx.get_slice_bound(3, "right")
4
```

If ``label`` is non-unique in the index, an error will be raised.

```
>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
```

```

        KeyError: Cannot get left slice bound for non-unique label: 'a'

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
    Group the index labels by a given array of values.

Parameters
-----
values : array
    Values used to determine the groups.

Returns
-----
dict
    {group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.

Parameters
-----
other : Index
    The Index object you want to compare with the current Index object.

Returns
-----
bool
    If two Index objects have equal elements and same type True,
    otherwise False.

See Also
-----
Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
    If we have an object dtype, try to infer a non-object dtype.

Parameters
-----
copy : bool, default True
    Whether to make a copy in cases where no inference occurs.

Returns
-----
Index
    An Index with a new dtype if the dtype was inferred
    or a shallow copy if the dtype could not be inferred.

See Also
-----
Index.inferred_type: Return a string of the type inferred from the values.

Examples
-----
>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')

intersection(self, other, sort: bool = False) from pandas.core.indexes.base.Index
    Form the intersection of two Index objects.

    This returns a new Index with elements common to the index and `other`.

```

Parameters

other : Index or array-like

An Index or an array-like object containing elements to form the intersection with the original Index.

sort : True, False or None, default False

Whether to sort the resulting index.

* None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.

* False : do not sort the result.

* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
```

```
>>> idx2 = pd.Index([3, 4, 5, 6])
```

```
>>> idx1.intersection(idx2)
```

```
Index([3, 4], dtype='int64')
```

is_(self, other) -> bool from pandas.core.indexes.base.Index

More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks that metadata is also the same.

Parameters

other : object

Other object to compare against.

Returns

bool

True if both have same underlying data, False otherwise.

See Also

Index.identical : Works like ``Index.is\_`` but also checks metadata.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
```

```
>>> idx1.is_(idx1.view())
```

```
True
```

```
>>> idx1.is_(idx1.copy())
```

```
False
```

isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_] from pandas.core.indexes

Return a boolean array where the index values are in `values`.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like

Sought values.

level : str or int, optional

Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

```
-----
np.ndarray[bool]
    NumPy array of boolean values.
```

See Also

```
-----
Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.
```

Notes

```
-----
In the case of `MultiIndex` you must either specify `values` as a
list-like object containing tuples that are the same length as the
number of levels, or specify `level`. Otherwise it will raise a
`ValueError`.
```

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]
... )
>>> midx
MultiIndex([(1, 'red'),
            (2, 'blue'),
            (3, 'green')],
           names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])
array([ True, False, False])
```

`isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index`
Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None`, `:attr:`numpy.NaN`` or `:attr:`pd.NaT``, get mapped to `True` values. Everything else get mapped to `False` values. Characters such as empty strings `''` or `:attr:`numpy.inf`` are not considered NA values.

Returns

```
-----
numpy.ndarray[bool]
    A boolean array of whether my values are NA.
```

See Also

```
-----
Index.notna : Boolean inverse of isna.
Index.dropna : Omit entries with missing values.
isna : Top-level isna.
Series.isna : Detect missing values in Series object.
```

Examples

```
-----
Show which entries in a pandas.Index are NA. The result is an
array.
```

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

```
isnull = isna(self) -> npt.NDArray[np.bool_]
```

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas.core.indexes.base
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

```
other : Index
    The other index on which join is performed.
how : {'left', 'right', 'inner', 'outer'}
level : int or level name, default None
    It is either the integer position or the name of the level.
return_indexers : bool, default False
    Whether to return the indexers or not for both the index objects.
sort : bool, default False
    Sort the join keys lexicographically in the result Index. If False,
    the order of the join keys depends on the join type (how keyword).
```

Returns

```
join_index, (left_indexer, right_indexer)
    The new index.
```

See Also

```
DataFrame.join : Join columns with `other` DataFrame either on index
or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a
database-style join.
```

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

```
map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base
Map values using an input mapping or function.
```


Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}
If 'ignore', propagate NA values, without passing them to the mapping correspondence.

Returns

Union[Index, MultiIndex]
The output of the mapping function applied to the index.
If the function returns a tuple with more than one element a MultiIndex will be returned.

See Also

Index.where : Replace values where the condition is False.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')

Using `map` with a function:

>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')

max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base.Index
Return the maximum value of the Index.

Parameters

axis : int, optional
For compatibility with NumPy. Only 0 or None are allowed.
skipna : bool, default True
Exclude NA/null values when showing the result.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Maximum value.

See Also

Index.min : Return the minimum value in an Index.
Series.max : Return the maximum value in a Series.
DataFrame.max : Return the maximum values in a DataFrame.

Examples

>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'

For a MultiIndex, the maximum is determined lexicographically.

>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)

memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.base.Index
Memory usage of the values.

Parameters

`deep` : bool, default False
Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption.

Returns

bytes used
Returns memory usage of the values in the Index in bytes.

See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of the array.

Notes

Memory usage does not include memory consumed by elements that are not components of the array if `deep=False` or if used on PyPy

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24
```

```
min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base.Index
Return the minimum value of the Index.
```

Parameters

`axis` : {None}
Dummy argument for consistency with Series.
`skipna` : bool, default True
Exclude NA/null values when showing the result.
`*args, **kwargs`
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Minimum value.

See Also

`Index.max` : Return the maximum value of the object.
`Series.min` : Return the minimum value in a Series.
`DataFrame.min` : Return the minimum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'
```

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

```
notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect existing (non-missing) values.
```

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or `:attr:`numpy.inf`` are not considered NA values. NA values, such as `None` or `:attr:`numpy.NaN``, get mapped to ``False`` values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

notnull = notna(self) -> npt.NDArray[np.bool_]

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters

mask : array-like of bool
Array of booleans denoting where values should be replaced.
value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-likes.

Returns

Index
A new Index of the values set with the mask.

See Also

numpy.putmask : Changes elements of an array
based on conditional and input values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')
```

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not
implemented currently.

Returns

Index
A view on self.

See Also

numpy.ndarray.ravel : Return a flattened array.

Examples

```
-----
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')
```

```
reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.
```

Parameters

```
-----
target : an iterable
    An iterable containing the values to be used for creating the new index.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
    * default: exact matches only.
    * pad / ffill: find the PREVIOUS index value if no exact match.
    * backfill / bfill: use NEXT index value if no exact match
    * nearest: use the NEAREST index value if no exact match. Tied
      distances are broken by preferring the larger index value.
level : int, optional
    Level of multiindex.
limit : int, optional
    Maximum number of consecutive labels in ``target`` to match for
    inexact matches.
tolerance : int, float, or list-like, optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation ``abs(index[indexer] - target) <= tolerance``.
```

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

```
-----
new_index : pd.Index
    Resulting index.
indexer : np.ndarray[np.intp] or None
    Indices of output values in original index.
```

Raises

```
-----
TypeError
    If ``method`` passed along with ``level``.
ValueError
    If non-unique multi-index
ValueError
    If non-unique index and ``method`` or ``limit`` passed.
```

See Also

```
-----
Series.reindex : Conform Series to new index with optional filling logic.
DataFrame.reindex : Conform DataFrame to new index with optional filling logic.
```

Examples

```
-----
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))
```

```
rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
Alter Index or MultiIndex name.
```

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters

name : Hashable or a sequence of the previous Name(s) to set.
inplace : bool, default False
Modifies the object directly, instead of creating a new Index or MultiIndex.

Returns

Index or None
The same type as the caller or None if ``inplace=True``.

See Also

Index.set_names : Able to set new names partially and by level.

Examples

```
>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")
Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

>>> idx = pd.MultiIndex.from_product(
...     ["python", "cobra"], [2018, 2019], names=["kind", "year"]
... )
>>> idx
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.
```

repeat(self, repeats, axis: None = None) -> Self from pandas.core.indexes.base.Index
Repeat elements of an Index.

Returns a new Index where each element of the current Index is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty Index.
axis : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

Index
Newly created Index with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
```

```

Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')
set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters
-----
names : Hashable or a sequence of the previous or dict-like for MultiIndex
        Name(s) to set.

level : int, Hashable or a sequence of the previous, optional
        If the index is a MultiIndex and names is not dict-like, level(s) to set
        (None for all levels). Otherwise level must be None.

inplace : bool, default False
        Modifies the object directly, instead of creating a new Index or
        MultiIndex.

Returns
-----
Index or None
        The same type as the caller or None if ``inplace=True``.

See Also
-----
Index.rename : Able to set new names without level.

Examples
-----
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            )
>>> idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['species', 'year'])

When renaming levels with a dict, levels can not be passed.

>>> idx.set_names({"kind": "snake"})
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['snake', 'year'])

slice_locs(
    self,
    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.

Parameters
-----
start : label, default None
        If None, defaults to the beginning.
end : label, default None
        If None, defaults to the end.
step : int, defaults None
        If None, defaults to 1.

```

Returns

tuple[int, int]

Returns a tuple of two integers representing the slice locations for the input labels within the index.

See Also

Index.get_loc : Get location for a single label.

Notes

This method only works if the index is monotonic or unique.

Examples

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)
```

```
>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)
```

```
sort_values(
    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

return_indexer : bool, default False
Should the indices that would sort the index be returned.

ascending : bool, default True
Should the index values be sorted in an ascending order.

na_position : {'first' or 'last'}, default 'last'
Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.

key : callable, optional
If not None, apply the key function to the index values before sorting. This is similar to the 'key' argument in the builtin :meth:`sorted` function, with the notable difference that this 'key' function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.

Returns

sorted_index : pandas.Index
Sorted copy of the index.

indexer : numpy.ndarray, optional
The indices that the index itself was sorted by.

See Also

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

```
>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')
```

Sort values in ascending order (default behavior).

```
>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')
```

Sort values in descending order, and also get the indices `idx` was sorted by.

```
>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))
```

```
sortlevel(
    self,
    level=None,
    ascending: bool | list[bool] = True,
    sort_remaining=None,
    na_position: NaPosition = 'first'
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
For internal compatibility with the Index API.

Sort the Index. This is for compat with MultiIndex

Parameters
-----
ascending : bool, default True
    False to sort in descending order
na_position : {'first' or 'last'}, default 'first'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.
```

.. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns

Index

```
symmetric_difference(
    self,
    other,
    result_name: abc.Hashable | None = None,
    sort: bool | None = None
) from pandas.core.indexes.base.Index
Compute the symmetric difference of two Index objects.
```

Parameters

other : Index or array-like
Index or an array-like object with elements to compute the symmetric difference with the original Index.

result_name : str
A string representing the name of the resulting Index, if desired.

sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any TypeError from incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any TypeErrors from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object containing elements that appear in either the original Index or the `other` Index, but not both.

See Also

Index.difference : Return a new Index with elements of index not in other.
Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes

``symmetric\_difference`` contains elements that appear in either ``idx1`` or ``idx2`` but not both. Equivalent to the Index created by ``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates dropped.

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')
```

`to_flat_index(self)` -> Self from `pandas.core.indexes.base.Index`
Identity method.

This is implemented for compatibility with subclass implementations when chaining.

Returns

`pd.Index`
Caller.

See Also

`MultiIndex.to_flat_index` : Subclass implementation.

`to_frame(self, index: bool = True, name: Hashable = <no_default>)` -> `DataFrame` from `pandas.core`
Create a `DataFrame` with a column containing the `Index`.

Parameters

`index` : bool, default True
Set the index of the returned `DataFrame` as the original `Index`.

`name` : object, defaults to `index.name`
The passed name should substitute for the index name (if it has one).

Returns

`DataFrame`
`DataFrame` containing the original `Index` data.

See Also

`Index.to_series` : Convert an `Index` to a `Series`.
`Series.to_frame` : Convert `Series` to `DataFrame`.

Examples

```
-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
      animal
animal
Ant      Ant
Bear    Bear
Cow      Cow
```

By default, the original `Index` is reused. To enforce a new `Index`:

```
>>> idx.to_frame(index=False)
      animal
0    Ant
1  Bear
2    Cow
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_frame(index=False, name="zoo")
      zoo
0    Ant
1  Bear
2    Cow
```

`to_series(self, index=None, name: Hashable | None = None)` -> `Series` from `pandas.core.indexes`
Create a `Series` with both index and values equal to the index keys.

Useful with `map` for returning an indexer based on an index.

Parameters

`index` : Index, optional
Index of resulting Series. If None, defaults to original index.
`name` : str, optional
Name of resulting Series. If None, defaults to name of original index.

Returns

Series

The dtype will be based on the type of the Index values.

See Also

`Index.to_frame` : Convert an Index to a DataFrame.
`Series.to_frame` : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to `index`:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1     Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify `name`:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.

`sort` : bool or None, default None
Whether to sort the resulting Index.

* None : Sort the result, except when

1. `self` and `other` are equal.
2. `self` or `other` has length 0.
3. Some values in `self` or `other` cannot be compared.
A `RuntimeWarning` is issued in this case.

* False : do not sort the result.

* True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the `other` Index.

See Also

```

-----
Index.unique : Return unique values in the index.
Index.intersection : Form the intersection of two Index objects.
Index.difference : Return a new Index with elements of index not in `other`.

```

Examples

Union matching dtypes

```

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')

```

Union mismatched dtypes

```

>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')

```

MultiIndex case

```

>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )

```

```

unique(self, level: Hashable | None = None) -> Self from pandas.core.indexes.base.Index
Return unique values in the index.

```

Unique values are returned in order of appearance, this does NOT sort.

Parameters

```

level : int or hashable, optional
    Only return values from specified level (for MultiIndex).
    If int, gets the level by integer position, else by level name.

```

Returns

```

Index
    Unique values in the index.

```

See Also

unique : Numpy array of unique values in that column.
Series.unique : Return unique values of Series object.

Examples

>>> idx = pd.Index([1, 1, 2, 3, 3])
>>> idx.unique()
Index([1, 2, 3], dtype='int64')

view(self, cls=None) from pandas.core.indexes.base.Index
Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the `cls` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

cls : data-type or ndarray sub-class, optional
Data-type descriptor of the returned view, e.g., float32 or int16. Omitting it results in the view having the same data-type as `self`. This argument can also be specified as an ndarray sub-class, e.g., np.int64 or np.float32 which then specifies the type of the returned object.

Returns

Index or ndarray
A view of the Index. If `cls` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric `cls` is specified an ndarray view with the new dtype is returned.

Raises

ValueError
If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

Index.copy : Returns a copy of the Index.
numpy.ndarray.view : Returns a new view of array with the same data.

Examples

>>> idx = pd.Index([-1, 0, 1])
>>> idx.view()
Index([-1, 0, 1], dtype='int64')

>>> idx.view(np.uint64)
array([18446744073709551615, 0, 1],
 dtype=uint64)

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

>>> idx.view("float32")
array([nan, nan, 0.e+00, 0.e+00, 1.e-45, 0.e+00], dtype=float32)

where(self, cond, other=None) -> Index from pandas.core.indexes.base.Index
Replace values where the condition is False.

The replacement is taken from other.

Parameters

cond : bool array-like with the same length as self
Condition to select the values on.
other : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index
A copy of self with values replaced from other where the condition is False.

See Also

Series.where : Same method for Series.
DataFrame.where : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

has_duplicates
Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

Index.is_unique : Inverse method that checks if it has unique values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False
```

is_monotonic_decreasing
Return a boolean if the values are equal or decreasing.

Returns

bool

See Also

Index.is_monotonic_increasing : Check if the values are equal or increasing.

Examples

```
>>> pd.Index([3, 2, 1]).is_monotonic_decreasing
True
>>> pd.Index([3, 2, 2]).is_monotonic_decreasing
True
>>> pd.Index([3, 1, 2]).is_monotonic_decreasing
False
```

is_monotonic_increasing
Return a boolean if the values are equal or increasing.

Returns

bool

See Also

Index.is_monotonic_decreasing : Check if the values are equal or decreasing.

Examples

```
>>> pd.Index([1, 2, 3]).is_monotonic_increasing
True
>>> pd.Index([1, 2, 2]).is_monotonic_increasing
True
>>> pd.Index([1, 3, 2]).is_monotonic_increasing
False
```

nlevels

Number of levels.

shape

Return a tuple of the shape of the underlying data.

See Also

Index.size: Return the number of elements in the underlying data.
Index.ndim: Number of dimensions of the underlying data, by definition 1.
Index.dtype: Return the dtype object of the underlying data.
Index.values: Return an array representing the data in the Index.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)
```

Data descriptors inherited from Index:

array

The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.
Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

=====

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

is_unique

Return if the index has unique values.

Returns

bool

See Also

Index.has_duplicates : Inverse method that checks if it has duplicate values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.is_unique
False
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.is_unique
True
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.is_unique
False
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.is_unique
True
```

name

Return Index or MultiIndex name.

Returns

label (hashable object)
The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.
Index.rename: Able to set new names partially and by level.
Series.name: Corresponding Series property.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx
```

```
Index([1, 2, 3], dtype='int64', name='x')
>>> idx.name
'x'
```

names

Get names on index.

This method returns a FrozenList containing the names of the object. It's primarily intended for internal use.

Returns

FrozenList

A FrozenList containing the object's names, contains None if the object does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx.names
FrozenList(['x'])
```

```
>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
>>> idx.names
FrozenList(['x', 'y'])
```

If the index does not have a name set:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])
```

Data and other attributes inherited from Index:

__pandas_priority__ = 2000

str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method. Patterned after Python's string methods, with some inspiration from R's stringr package.

Parameters

data : Series or Index

The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.

Index.str : Vectorized string functions for Index.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:


```

__iter__(self) -> Iterator
    Return an iterator of the values.

    These are each a scalar type, which is a Python scalar
    (for str, int, float) or a pandas scalar
    (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    iterator
        An iterator yielding scalar values from the Series.

    See Also
    -----
    Series.items : Lazily iterate over (index, value) tuples.

    Examples
    -----
    >>> s = pd.Series([1, 2, 3])
    >>> for x in s:
    ...     print(x)
    1
    2
    3

factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.intc], npt.NDArray[np.intc]]
    Encode the object as an enumerated type or categorical variable.

    This method is useful for obtaining a numeric representation of an
    array when all that matters is identifying distinct values. factorize
    is available as both a top-level function :func:`pandas.factorize`,
    and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

    Parameters
    -----
    sort : bool, default False
        Sort uniques` and shuffle codes` to maintain the
        relationship.
    use_na_sentinel : bool, default True
        If True, the sentinel -1 will be used for NaN values. If False,
        NaN values will be encoded as non-negative integers and will not drop the
        NaN from the uniques of the values.

    Returns
    -----
    codes : ndarray
        An integer ndarray that's an indexer into uniques`.
        uniques.take(codes)` will have the same values as values`.
    uniques : ndarray, Index, or Categorical
        The unique valid values. When values` is Categorical, uniques`
        is a Categorical. When values` is some other pandas object, an
        Index` is returned. Otherwise, a 1-D ndarray is returned.

    .. note::

        Even if there's a missing value in values`, uniques` will
        *not* contain an entry for it.

    See Also
    -----
    cut : Discretize continuous-valued array.
    unique : Find the unique value in an array.

    Notes
    -----
    Reference :ref:`the user guide <reshaping.factorize>` for more examples.

    Examples
    -----
    These examples all show factorize as a top-level method like
    pd.factorize(values)`. The results are identical for methods like
    :meth:`Series.factorize`.

    >>> codes, uniques = pd.factorize(
    ...     np.array(["b", "b", "a", "c", "b"], dtype="O")
    ... )
    >>> codes

```

```
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With ``sort=True``, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use\_na\_sentinel=True`` (the default), missing values are indicated in the `codes` with the sentinel value ``-1`` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])
```

```
item(self)
    Return the first element of the underlying data as a Python scalar.

    Returns
    -----
    scalar
        The first element of Series or Index.

    Raises
```

```
-----
ValueError
    If the data is not length = 1.
```

See Also

```
-----
Index.values : Returns an array representing the data in the Index.
Series.head : Returns the first `n` rows.
```

Examples

```
-----
>>> s = pd.Series([1])
>>> s.item()
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'
```

```
nunique(self, dropna: bool = True) -> int
Return number of unique elements in the object.
```

Excludes NA values by default.

Parameters

```
-----
dropna : bool, default True
    Don't include NaN in the count.
```

Returns

```
-----
int
    An integer indicating the number of unique elements in the object.
```

See Also

```
-----
DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.
```

Examples

```
-----
>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0    1
1    3
2    5
3    7
4    7
dtype: int64

>>> s.nunique()
4
```

```
searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.
```

Find the indices into a sorted Index `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::

The Index *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

```
-----
value : array-like or scalar
    Values to insert into `self`.
```

side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort `self` into ascending order. They are typically the result of ``np.argsort``.

Returns

int or array of int
A scalar or array of insertion points with the same shape as `value`.

See Also

sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

>>> ser = pd.Series([1, 2, 3])
>>> ser
0 1
1 2
2 3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0 2000-03-11
1 2000-03-12
2 2000-03-13
dtype: datetime64[us]

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
... ["apple", "bread", "bread", "cheese", "milk"], ordered=True
...)
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']

>>> ser.searchsorted("bread")
np.int64(1)

>>> ser.searchsorted(["bread"], side="right")
array([3])

If the values are not monotonically sorted, wrong locations
may be returned:

>>> ser = pd.Series([2, 1, 3])
>>> ser
0 2
1 1
2 3
dtype: int64

```

>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1

to_list = tolist(self) -> list

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>,
    **kwargs
) -> np.ndarray
    A NumPy ndarray representing the values in this Series or Index.

Parameters
-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
**kwargs
    Additional keywords passed through to the ``to_numpy`` method
    of the underlying array (for extension arrays).

Returns
-----
numpy.ndarray
    The NumPy ndarray holding the values from this Series or Index.
    The dtype of the array may differ. See Notes.

See Also
-----
Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

Notes
-----
The returned array will be the same up to equality (values equal
in `self` will be equal in the returned array; likewise for values
that are not equal). When `self` contains an ExtensionArray, the
dtype may be different. For example, for a category-dtype Series,
``to_numpy()`` will return a NumPy array and the categorical dtype
will be lost.

For NumPy dtypes, this will be a reference to the actual data stored
in this Series or Index (assuming ``copy=False``). Modifying the result
in place will modify the data stored in the Series or Index (not that
we recommend doing that).

For extension types, ``to_numpy()`` *may* require copying data and
coercing the result to a NumPy type (possibly object), which may be
expensive. When you need a no-copy reference to the underlying data,
:attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of
``to_numpy()`` for various dtypes within pandas.

=====
dtype          array type
=====
category[T]    ndarray[T] (same dtype as input)
period         ndarray[object] (Periods)
interval       ndarray[object] (Intervals)
IntegerNA      ndarray[object]
datetime64[ns] datetime64[ns]
datetime64[ns, tz] ndarray[object] (Timestamps)
=====

Examples
-----

```

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)
```

Specify the `dtype` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

```
>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

`tolist(self)` -> list
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

```
-----
list
    List containing the values as Python or pandas scalers.
```

See Also

```
-----
numpy.ndarray.tolist : Return the array as an a.ndim-levels deep
    nested list of Python scalars.
```

Examples

```
-----
For Series
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.to_list()
[1, 2, 3]
```

`transpose(self, *args, **kwargs)` -> Self
Return the transpose, which is by definition self.

Returns

```
-----
%(klass)s
```

```
value_counts(
    self,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    bins=None,
    dropna: bool = True
)
```

-> Series
Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

normalize : bool, default False
If True then the object returned will contain the relative frequencies of the unique values.
sort : bool, default True
Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False
Sort in ascending order.
bins : int, optional
Rather than count values, group them into half-open bins, a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
Don't include counts of NaN.

Returns

Series

Series containing counts of unique values.

See Also

Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64
```

With `normalize` set to `True`, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64
```

****bins****

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]     2
(3.0, 4.0]     1
Name: count, dtype: int64
```

****dropna****

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64
```

****Categorical Dtypes****

Rows with categorical type will be counted as one group if they have same categories and order.
In the example below, even though ``a``, ``c``, and ``d`` all have the same data types of ``category``, only ``c`` and ``d`` will be counted as one group since ``a`` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str          1
Name: count, dtype: int64
```

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.
Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
-----
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
      height  weight
elk       1.5     500
moose     2.6     800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
      height  weight
```

```

elk      3.0    501.5
moose    4.1    801.5

```

Each element of a list is added to a column of the DataFrame, in order.

```

>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0    501.5
moose      3.1    801.5

```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose      3.1    801.5

```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5

```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN

```

```

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5

```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```

>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk        1.7     NaN
moose      3.0     NaN

```

`__and__(self, other)`

`__divmod__(self, other)`

`__eq__(self, other)`
Return self==value.

`__floordiv__(self, other)`

`__ge__(self, other)`
Return self>=value.

`__gt__(self, other)`
Return self>value.

`__le__(self, other)`
Return self<=value.

`__lt__(self, other)`
Return self<value.

`__mod__(self, other)`

```

__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
__hash__ = None

-----
Methods inherited from pandas.core.base.PandasObject:
__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
__dir__(self) -> list[str]
    Provide method name lookup and completion.

    Notes
    ----
    Only provide 'public' methods.

```

```

class DatetimeTZDtype(pandas.core.dtypes.dtypes.PandasExtensionDtype)
|     DatetimeTZDtype(unit: TimeUnit | DatetimeTZDtype = 'ns', tz=None) -> None
|
|     An ExtensionDtype for timezone-aware datetime data.
|
|     **This is not an actual numpy dtype**, but a duck type.

```

Parameters

unit : str, default "ns"
 The precision of the datetime data. Valid options are
 ``"s"`` , ``"ms"`` , ``"us"`` , ``"ns"`` .
tz : str, int, or datetime.tzinfo
 The timezone.

Attributes

unit
tz

Methods

None

Raises

ZoneInfoNotFoundError
 When the requested timezone cannot be found.

See Also

numpy.datetime64 : Numpy data type for datetime.
datetime.datetime : Python datetime object.

Examples

>>> from zoneinfo import ZoneInfo
>>> pd.DatetimeTZDtype(tz=ZoneInfo("UTC"))
datetime64[ns, UTC]

>>> pd.DatetimeTZDtype(tz=ZoneInfo("Europe/Paris"))
datetime64[ns, Europe/Paris]

Method resolution order:

DatetimeTZDtype
pandas.core.dtypes.dtypes.PandasExtensionDtype
pandas.api.extensions.ExtensionDtype
builtins.object

Methods defined here:

__eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.DatetimeTZDtype
 Check whether 'other' is equal to self.

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes
 in ``self.\_metadata`` are equal between ``self`` and ``other``.

Parameters

other : Any

Returns

bool

__from_arrow__(self, array: pa.Array | pa.ChunkedArray) -> DatetimeArray from pandas.core.dtypes
 Construct DatetimeArray from pyarrow Array/ChunkedArray.

Note: If the units in the pyarrow Array are the same as this
DatetimeDtype, then values corresponding to the integer representation
of ``NaT`` (e.g. one nanosecond before :attr:`pandas.Timestamp.min`)
are converted to ``NaT``, regardless of the null indicator in the
pyarrow array.

Parameters

array : pyarrow.Array or pyarrow.ChunkedArray
 The Arrow array to convert to DatetimeArray.

Returns

```

extension array : DatetimeArray

__hash__(self) -> int from pandas.core.dtypes.dtypes.DatetimeTZDtype
    Return hash(self).

__init__(self, unit: TimeUnit | DatetimeTZDtype = 'ns', tz=None) -> None from pandas.core.dtypes.dtypes.DatetimeTZDtype
    Initialize self. See help(type(self)) for accurate signature.

__setstate__(self, state) -> None from pandas.core.dtypes.dtypes.DatetimeTZDtype

__str__(self) -> str_type from pandas.core.dtypes.dtypes.DatetimeTZDtype
    Return str(self).

construct_array_type(self) -> type_t[DatetimeArray] from pandas.core.dtypes.dtypes.DatetimeTZDtype
    Return the array type associated with this dtype.

Returns
-----
type
-----

Class methods defined here:

construct_from_string(string: str_type) -> DatetimeTZDtype from pandas.core.dtypes.dtypes.DatetimeTZDtype
    Construct a DatetimeTZDtype from a string.

Parameters
-----
string : str
    The string alias for this DatetimeTZDtype.
    Should be formatted like ``datetime64[ns, <tz>]``,
    where ``<tz>`` is the timezone name.

Examples
-----
>>> DatetimeTZDtype.construct_from_string("datetime64[ns, UTC]")
datetime64[ns, UTC]
-----

Readonly properties defined here:

na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.

name
    A string representation of the dtype.

tz
    The timezone.

See Also
-----
DatetimeTZDtype.unit : Retrieves precision of the datetime data.

Examples
-----
>>> from zoneinfo import ZoneInfo
>>> dtype = pd.DatetimeTZDtype(tz=ZoneInfo("America/Los_Angeles"))
>>> dtype.tz
zoneinfo.ZoneInfo(key='America/Los_Angeles')

unit
    The precision of the datetime data.

See Also
-----
DatetimeTZDtype.tz : Retrieves the timezone.

Examples
-----
>>> from zoneinfo import ZoneInfo
>>> dtype = pd.DatetimeTZDtype(tz=ZoneInfo("America/Los_Angeles"))
>>> dtype.unit

```

'ns'

Data descriptors defined here:

base

index_class

str

Data and other attributes defined here:

__annotations__ = {'_cache_dtypes': 'dict[str_type, PandasExtensionDty...

kind = 'M'

num = 101

type = <class 'pandas.Timestamp'>

Pandas replacement for python datetime.datetime object.

Timestamp is the pandas equivalent of python's Datetime and is interchangeable with it in most cases. It's the type used for the entries that make up a DatetimeIndex, and other timeseries oriented data structures in pandas.

Parameters

ts_input : datetime-like, str, int, float
Value to be converted to Timestamp.

year : int
Value of year.

month : int
Value of month.

day : int
Value of day.

hour : int, optional, default 0
Value of hour.

minute : int, optional, default 0
Value of minute.

second : int, optional, default 0
Value of second.

microsecond : int, optional, default 0
Value of microsecond.

tzinfo : datetime.tzinfo, optional, default None
Timezone info.

nanosecond : int, optional, default 0
Value of nanosecond.

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for time which Timestamp will have.

unit : str
Unit used for conversion if ts_input is of type int or float. The valid values are 'W', 'D', 'h', 'm', 's', 'ms', 'us', and 'ns'. For example, 's' means seconds and 'ms' means milliseconds.

For float inputs, the result will be stored in nanoseconds, and the unit attribute will be set as ``'ns'``.

fold : {0, 1}, default None, keyword-only
Due to daylight saving time, one wall clock time can occur twice when shifting from summer to winter time; fold describes whether the datetime-like corresponds to the first (0) or the second time (1) the wall clock hits the ambiguous time.

See Also

Timedelta : Represents a duration, the difference between two dates or times.
datetime.datetime : Python datetime.datetime object.

Notes

There are essentially three calling conventions for the constructor. The primary form accepts four parameters. They can be passed by position or keyword.

The other two forms mimic the parameters from ``datetime.datetime``. They

can be passed by either position or keyword, but not both mixed together.

Examples

Using the primary calling convention:

This converts a datetime-like string

```
>>> pd.Timestamp('2017-01-01T12')
Timestamp('2017-01-01 12:00:00')
```

This converts a float representing a Unix epoch in units of seconds

```
>>> pd.Timestamp(1513393355.5, unit='s')
Timestamp('2017-12-16 03:02:35.500000')
```

This converts an int representing a Unix-epoch in units of weeks

```
>>> pd.Timestamp(1535, unit='W')
Timestamp('1999-06-03 00:00:00')
```

This converts an int representing a Unix-epoch in units of seconds and for a particular timezone

```
>>> pd.Timestamp(1513393355, unit='s', tz='US/Pacific')
Timestamp('2017-12-15 19:02:35-0800', tz='US/Pacific')
```

Using the other two forms that mimic the API for ``datetime.datetime``:

```
>>> pd.Timestamp(2017, 1, 1, 12)
Timestamp('2017-01-01 12:00:00')
```

```
>>> pd.Timestamp(year=2017, month=1, day=1, hour=12)
Timestamp('2017-01-01 12:00:00')
```

Methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

__getstate__(self) -> dict[str_type, Any]
Helper for pickle.

__repr__(self) -> str_type
Return a string representation for a particular object.

Class methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

reset_cache() -> None
clear the cache

Data and other attributes inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

isbuiltin = 0

isnative = 0

itemsizes = 8

shape = ()

subdtype = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Return self!=value.

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
Construct an ExtensionArray of this dtype with the given shape.

Analogous to numpy.empty.

Parameters

```

        shape : int or tuple[int]

        Returns
        -----
        ExtensionArray

    -----
    Class methods inherited from pandas.api.extensions.ExtensionDtype:

    is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
        Check if we match 'dtype'.

        Parameters
        -----
        dtype : object
            The object to check.

        Returns
        -----
        bool

        Notes
        -----
        The default implementation is True if

        1. ``cls.construct_from_string(dtype)`` is an instance
           of ``cls``.
        2. ``dtype`` is an object and is an instance of ``cls``
        3. ``dtype`` has a ``dtype`` attribute, and any of the above
           conditions is true for ``dtype.dtype``.

    -----
    Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

    names
        Ordered list of field names, or None if there are no fields.

        This is for compatibility with NumPy arrays, and may be removed in the
        future.

    -----
    Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

    __dict__
        dictionary for instance variables

    __weakref__
        list of weak references to the object

```

```

class ExcelFile(builtins.object)
    ExcelFile(
        path_or_buffer,
        engine: str | None = None,
        storage_options: StorageOptions | None = None,
        engine_kwargs: dict | None = None
    ) -> None

    Class for parsing tabular Excel sheets into DataFrame objects.

    See read_excel for more documentation.

    Parameters
    -----
    path_or_buffer : str, bytes, pathlib.Path,
        A file-like object, xlrd workbook or openpyxl workbook.
        If a string or path object, expected to be a path to a
        .xls, .xlsx, .xlsb, .xlsm, .odf, .ods, or .odt file.
    engine : str, default None
        If io is not a buffer or path, this must be set to identify io.
        Supported engines: ``xlrd``, ``openpyxl``, ``odf``, ``pyxlsb``, ``calamine``
        Engine compatibility :

        - ``xlrd`` supports old-style Excel files (.xls).
        - ``openpyxl`` supports newer Excel file formats.
        - ``odf`` supports OpenDocument file formats (.odf, .ods, .odt).
        - ``pyxlsb`` supports Binary Excel files.
        - ``calamine`` supports Excel (.xls, .xlsx, .xlsm, .xlsb)

```

and OpenDocument (.ods) file formats.

The engine `xlrd` <<https://xlrd.readthedocs.io/en/latest/>>`\_` now only supports old-style `xls` files. When `engine=None`, the following logic will be used to determine the engine:

- If `path_or_buffer` is an OpenDocument format (.odf, .ods, .odt), then `odf` <<https://pypi.org/project/odfpy/>>`\_` will be used.
- Otherwise if `path_or_buffer` is an xls format, `xlrd` will be used.
- Otherwise if `path_or_buffer` is in xlsx format, `pyxlsb` <<https://pypi.org/project/pyxlsb/>>`\_` will be used.
- Otherwise if `openpyxl` <<https://pypi.org/project/openpyxl/>>`\_` is installed, then `openpyxl` will be used.
- Otherwise if `xlrd >= 2.0` is installed, a `ValueError` will be raised.

.. warning::

Please do not report issues when using `xlrd` to read `xlsx` files. This is not supported, switch to using `openpyxl` instead.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`\_`.

`engine_kwargs` : dict, optional

Arbitrary keyword arguments passed to excel engine.

See Also

`DataFrame.to_excel` : Write DataFrame to an Excel file.

`DataFrame.to_csv` : Write DataFrame to a comma-separated values (csv) file.

`read_csv` : Read a comma-separated values (csv) file into DataFrame.

`read_fwf` : Read a table of fixed-width formatted lines into DataFrame.

Examples

```
>>> file = pd.ExcelFile("myfile.xlsx") # doctest: +SKIP
>>> with pd.ExcelFile("myfile.xls") as xls: # doctest: +SKIP
...     df1 = pd.read_excel(xls, "Sheet1") # doctest: +SKIP
```

Methods defined here:

`__enter__(self)` -> Self from pandas.io.excel._base.ExcelFile

`__exit__(self, exc_type: type[BaseException] | None, exc_value: BaseException | None, traceback: TracebackType | None)` -> None from pandas.io.excel._base.ExcelFile

`__fspath__(self)` from pandas.io.excel._base.ExcelFile

`__init__(self, path_or_buffer, engine: str | None = None, storage_options: StorageOptions | None = None, engine_kwargs: dict | None = None)` -> None from pandas.io.excel._base.ExcelFile
Initialize self. See help(type(self)) for accurate signature.

`close(self)` -> None from pandas.io.excel._base.ExcelFile
close io if necessary

`parse(self, sheet_name: str | int | list[int] | list[str] | None = 0, header: int | Sequence[int] | None = 0, names: SequenceNotStr[Hashable] | range | None = None, index_col: int | Sequence[int] | None = None,`

```

usecols=None,
converters=None,
true_values: Iterable[Hashable] | None = None,
false_values: Iterable[Hashable] | None = None,
skiprows: Sequence[int] | int | Callable[[int], object] | None = None,
nrows: int | None = None,
na_values=None,
parse_dates: list | dict | bool = False,
date_format: str | dict[Hashable, str] | None = None,
thousands: str | None = None,
comment: str | None = None,
skipfooter: int = 0,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
**kwds
) -> DataFrame | dict[str, DataFrame] | dict[int, DataFrame] from pandas.io.excel._base.ExcelFile
Parse specified sheet(s) into a DataFrame.

```

Equivalent to `read_excel(ExcelFile, ...)` See the `read_excel` docstring for more info on accepted parameters.

Parameters

```

sheet_name : str, int, list, or None, default 0
    Strings are used for sheet names. Integers are used in zero-indexed
    sheet positions (chart sheets do not count as a sheet position).
    Lists of strings/integers are used to request multiple sheets.
    When ``None``, will return a dictionary containing DataFrames for
    each sheet.
header : int, list of int, default 0
    Row (0-indexed) to use for the column labels of the parsed
    DataFrame. If a list of integers is passed those row positions will
    be combined into a ``MultiIndex``. Use None if there is no header.
names : array-like, default None
    List of column names to use. If file contains no header row,
    then you should explicitly pass header=None.
index_col : int, str, list of int, default None
    Column (0-indexed) to use as the row labels of the DataFrame.
    Pass None if there is no such column. If a list is passed,
    those columns will be combined into a ``MultiIndex``. If a
    subset of data is selected with ``usecols``, index_col
    is based on the subset.

    Missing values will be forward filled to allow roundtripping with
    ``to_excel`` for ``merged_cells=True``. To avoid forward filling the
    missing values use ``set_index`` after reading the data instead of
    ``index_col``.
usecols : str, list-like, or callable, default None
    * If None, then parse all columns.
    * If str, then indicates comma separated list of Excel column letters
      and column ranges (e.g. "A:E" or "A,C,E:F"). Ranges are inclusive of
      both sides.
    * If list of int, then indicates list of column numbers to be parsed
      (0-indexed).
    * If list of string, then indicates list of column names to be parsed.
    * If callable, then evaluate each column name against it and parse the
      column if the callable returns ``True``.

    Returns a subset of the columns according to behavior above.
converters : dict, default None
    Dict of functions for converting values in certain columns. Keys can
    either be integers or column labels, values are functions that take one
    input argument, the Excel cell content, and return the transformed
    content.
true_values : list, default None
    Values to consider as True.
false_values : list, default None
    Values to consider as False.
skiprows : list-like, int, or callable, optional
    Line numbers to skip (0-indexed) or number of lines to skip (int) at the
    start of the file. If callable, the callable function will be evaluated
    against the row indices, returning True if the row should be skipped and
    False otherwise. An example of a valid callable argument would be ``lambda
    x: x in [0, 2]``.
nrows : int, default None
    Number of rows to parse.
na_values : scalar, str, list-like, or dict, default None
    Additional strings to recognize as NA/NaN. If dict passed, specific

```

per-column NA values.

`parse_dates` : bool, list-like, or dict, default False
The behavior is as follows:

- * ``bool``. If True -> try parsing the index.
- * ``list`` of int or names. e.g. If [1, 2, 3] -> try parsing columns 1, 2, 3 each as a separate date column.
- * ``list`` of lists. e.g. If [[1, 3]] -> combine columns 1 and 3 and parse as a single date column.
- * ``dict``, e.g. {'foo' : [1, 3]} -> parse columns 1, 3 as date and call result 'foo'

If a column or index contains an unparseable date, the entire column or index will be returned unaltered as an object data type. If you don't want to parse some cells as date just change their type in Excel to "Text". For non-standard datetime parsing, use ``pd.to\_datetime`` after ``pd.read\_excel``.

Note: A fast-path exists for iso8601-formatted dates.

`date_format` : str or dict of column -> format, default ``None``
If used in conjunction with ``parse\_dates``, will parse dates according to this format. For anything more complex, please read in as ``object`` and then apply :func:`to\_datetime` as-needed.

`thousands` : str, default None
Thousands separator for parsing string columns to numeric. Note that this parameter is only necessary for columns stored as TEXT in Excel, any numeric columns will automatically be parsed, regardless of display format.

`comment` : str, default None
Comments out remainder of line. Pass a character or characters to this argument to indicate comments in the input file. Any data between the comment string and the end of the current line is ignored.

`skipfooter` : int, default 0
Rows at the end to skip (0-indexed).

`dtype_backend` : {'numpy_nullable', 'pyarrow'}
Back-end data type applied to the resultant :class:`DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- * ``"numpy\_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
- * ``"pyarrow"``: returns pyarrow-backed nullable :class:`ArrowDtype` :class:`DataFrame`

.. versionadded:: 2.0

**kwds : dict, optional
Arbitrary keyword arguments passed to excel engine.

Returns

DataFrame or dict of DataFrames

DataFrame from the passed in Excel file.

See Also

`read_excel` : Read an Excel sheet values (xlsx) file into DataFrame.
`read_csv` : Read a comma-separated values (csv) file into DataFrame.
`read_fwf` : Read a table of fixed-width formatted lines into DataFrame.

Examples

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])
>>> df.to_excel("myfile.xlsx") # doctest: +SKIP
>>> file = pd.ExcelFile("myfile.xlsx") # doctest: +SKIP
>>> file.parse() # doctest: +SKIP
```

Readonly properties defined here:

book

Gets the Excel workbook.

Workbook is the top-level container for all document information.

Returns

Excel Workbook

The workbook object of the type defined by the engine being used.

See Also

`read_excel` : Read an Excel file into a pandas DataFrame.

Examples

```
>>> file = pd.ExcelFile("myfile.xlsx") # doctest: +SKIP
>>> file.book # doctest: +SKIP
<openpyxl.workbook.workbook.Workbook object at 0x11eb5ad70>
>>> file.book.path # doctest: +SKIP
'/xl/workbook.xml'
>>> file.book.active # doctest: +SKIP
<openpyxl.worksheet._read_only.ReadOnlyWorksheet object at 0x11eb5b370>
>>> file.book.sheetnames # doctest: +SKIP
['Sheet1', 'Sheet2']
```

`sheet_names`

Names of the sheets in the document.

This is particularly useful for loading a specific sheet into a DataFrame when you do not know the sheet names beforehand.

Returns

list of str
List of sheet names in the document.

See Also

`ExcelFile.parse` : Parse a sheet into a DataFrame.

`read_excel` : Read an Excel file into a pandas DataFrame. If you know the sheet names, it may be easier to specify them directly to `read_excel`.

Examples

```
>>> file = pd.ExcelFile("myfile.xlsx") # doctest: +SKIP
>>> file.sheet_names # doctest: +SKIP
["Sheet1", "Sheet2"]
```

Data descriptors defined here:

`__dict__`
dictionary for instance variables

`__weakref__`
list of weak references to the object

Data and other attributes defined here:

`CalamineReader` = <class 'pandas.io.excel._calamine.CalamineReader'>

`ODFReader` = <class 'pandas.io.excel._odfreader.ODFReader'>

`OpenpyxlReader` = <class 'pandas.io.excel._openpyxl.OpenpyxlReader'>

`PyxlsbReader` = <class 'pandas.io.excel._pyxlsb.PyxlsbReader'>

`XlrdReader` = <class 'pandas.io.excel._xlrd.XlrdReader'>

`__annotations__` = {'_engines': 'Mapping[str, Any]'}

class ExcelWriter(typing.Generic)

```
    ExcelWriter(
        path: FilePath | WriteExcelBuffer | ExcelWriter,
        engine: str | None = None,
        date_format: str | None = None,
        datetime_format: str | None = None,
        mode: str = 'w',
        storage_options: StorageOptions | None = None,
        if_sheet_exists: ExcelWriterIfSheetExists | None = None,
        engine_kwargs: dict | None = None
    ) -> Self
```

Class for writing DataFrame objects into excel sheets.

Default is to use:

```
* `xlsxwriter` <https://pypi.org/project/XlsxWriter/>`__ for xlsx files if xlsxwriter
  is installed otherwise `openpyxl` <https://pypi.org/project/openpyxl/>`__
* `odf` <https://pypi.org/project/odfpy/>`__ for ods files
```

See :meth:`DataFrame.to\_excel` for typical usage.

The writer should be used as a context manager. Otherwise, call `close()` to save and close any opened file handles.

Parameters

path : str or typing.BinaryIO
Path to xls or xlsx or ods file.
engine : str (optional)
Engine to use for writing. If None, defaults to
`io.excel.<extension>.writer`. NOTE: can only be passed as a keyword
argument.
date_format : str, default None
Format string for dates written into Excel files (e.g. 'YYYY-MM-DD').
datetime_format : str, default None
Format string for datetime objects written into Excel files.
(e.g. 'YYYY-MM-DD HH:MM:SS').
mode : {'w', 'a'}, default 'w'
File mode to use (write or append). Append does not work with fsspec URLs.
storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g.
host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
are forwarded to `urllib.request.Request` as header options. For other
URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more
details, and for more examples on storage options refer `here`
<[https://pandas.pydata.org/docs/user_guide/io.html?](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files)
highlight=storage_options#reading-writing-remote-files>`_.

if_sheet_exists : {'error', 'new', 'replace', 'overlay'}, default 'error'
How to behave when trying to write to a sheet that already
exists (append mode only).

- * error: raise a ValueError.
- * new: Create a new sheet, with a name determined by the engine.
- * replace: Delete the contents of the sheet before writing to it.
- * overlay: Write contents to the existing sheet without first removing,
but possibly over top of, the existing contents.

engine_kwargs : dict, optional
Keyword arguments to be passed into the engine. These will be passed to
the following functions of the respective engines:

```
* xlsxwriter: `xlsxwriter.Workbook(file, **engine_kwargs)`
* openpyxl (write mode): `openpyxl.Workbook(**engine_kwargs)`
* openpyxl (append mode): `openpyxl.load_workbook(file, **engine_kwargs)`
* odf: `odf.opendocument.OpenDocumentSpreadsheet(**engine_kwargs)`
```

See Also

read_excel : Read an Excel sheet values (xlsx) file into DataFrame.
read_csv : Read a comma-separated values (csv) file into DataFrame.
read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Notes

For compatibility with CSV writers, ExcelWriter serializes lists
and dicts to strings before writing.

Examples

Default usage:

```
>>> df = pd.DataFrame([["ABC", "XYZ"]], columns=["Foo", "Bar"]) # doctest: +SKIP
>>> with pd.ExcelWriter("path_to_file.xlsx") as writer:
...     df.to_excel(writer) # doctest: +SKIP
```

To write to separate sheets in a single file:

```
>>> df1 = pd.DataFrame([["AAA", "BBB"]], columns=["Spam", "Egg"]) # doctest: +SKIP
>>> df2 = pd.DataFrame([["ABC", "XYZ"]], columns=["Foo", "Bar"]) # doctest: +SKIP
>>> with pd.ExcelWriter("path_to_file.xlsx") as writer:
...     df1.to_excel(writer, sheet_name="Sheet1") # doctest: +SKIP
...     df2.to_excel(writer, sheet_name="Sheet2") # doctest: +SKIP
```

You can set the date format or datetime format:

```
>>> from datetime import date, datetime # doctest: +SKIP
>>> df = pd.DataFrame(
...     [
...         [date(2014, 1, 31), date(1999, 9, 24)],
...         [datetime(1998, 5, 26, 23, 33, 4), datetime(2014, 2, 28, 13, 5, 13)],
...     ],
...     index=["Date", "Datetime"],
...     columns=["X", "Y"],
... ) # doctest: +SKIP
>>> with pd.ExcelWriter(
...     "path_to_file.xlsx",
...     date_format="YYYY-MM-DD",
...     datetime_format="YYYY-MM-DD HH:MM:SS",
... ) as writer:
...     df.to_excel(writer) # doctest: +SKIP
```

You can also append to an existing Excel file:

```
>>> with pd.ExcelWriter("path_to_file.xlsx", mode="a", engine="openpyxl") as writer:
...     df.to_excel(writer, sheet_name="Sheet3") # doctest: +SKIP
```

Here, the `if_sheet_exists` parameter can be set to replace a sheet if it already exists:

```
>>> with pd.ExcelWriter(
...     "path_to_file.xlsx",
...     mode="a",
...     engine="openpyxl",
...     if_sheet_exists="replace",
... ) as writer:
...     df.to_excel(writer, sheet_name="Sheet1") # doctest: +SKIP
```

You can also write multiple DataFrames to a single sheet. Note that the `if_sheet_exists` parameter needs to be set to `overlay`:

```
>>> with pd.ExcelWriter(
...     "path_to_file.xlsx",
...     mode="a",
...     engine="openpyxl",
...     if_sheet_exists="overlay",
... ) as writer:
...     df1.to_excel(writer, sheet_name="Sheet1")
...     df2.to_excel(writer, sheet_name="Sheet1", startcol=3) # doctest: +SKIP
```

You can store Excel file in RAM:

```
>>> import io
>>> df = pd.DataFrame([["ABC", "XYZ"]], columns=["Foo", "Bar"])
>>> buffer = io.BytesIO()
>>> with pd.ExcelWriter(buffer) as writer:
...     df.to_excel(writer)
```

You can pack Excel file into zip archive:

```
>>> import zipfile # doctest: +SKIP
>>> df = pd.DataFrame([["ABC", "XYZ"]], columns=["Foo", "Bar"]) # doctest: +SKIP
>>> with zipfile.ZipFile("path_to_file.zip", "w") as zf:
...     with zf.open("filename.xlsx", "w") as buffer:
...         with pd.ExcelWriter(buffer) as writer:
...             df.to_excel(writer) # doctest: +SKIP
```

You can specify additional arguments to the underlying engine:

```
>>> with pd.ExcelWriter(
...     "path_to_file.xlsx",
...     engine="xlsxwriter",
...     engine_kwargs={"options": {"nan_inf_to_errors": True}},
... ) as writer:
```

```

...     df.to_excel(writer) # doctest: +SKIP

In append mode, ``engine_kwargs`` are passed through to
openpyxl's ``load_workbook``:

>>> with pd.ExcelWriter(
...     "path_to_file.xlsx",
...     engine="openpyxl",
...     mode="a",
...     engine_kwargs={"keep_vba": True}},
... ) as writer:
...     df.to_excel(writer, sheet_name="Sheet2") # doctest: +SKIP

Method resolution order:
  ExcelWriter
  typing.Generic
  builtins.object

Methods defined here:

__enter__(self) -> Self from pandas.io.excel._base.ExcelWriter
    # Allow use as a contextmanager

__exit__(
    self,
    exc_type: type[BaseException] | None,
    exc_value: BaseException | None,
    traceback: TracebackType | None
) -> None from pandas.io.excel._base.ExcelWriter

__fspath__(self) -> str from pandas.io.excel._base.ExcelWriter

__init__(
    self,
    path: FilePath | WriteExcelBuffer | ExcelWriter,
    engine: str | None = None,
    date_format: str | None = None,
    datetime_format: str | None = None,
    mode: str = 'w',
    storage_options: StorageOptions | None = None,
    if_sheet_exists: ExcelWriterIfSheetExists | None = None,
    engine_kwargs: dict[str, Any] | None = None
) -> None from pandas.io.excel._base.ExcelWriter
    Initialize self. See help(type(self)) for accurate signature.

close(self) -> None from pandas.io.excel._base.ExcelWriter
    synonym for save, to make it more file-like

-----
Class methods defined here:

check_extension(ext: str) -> Literal[True] from pandas.io.excel._base.ExcelWriter
    checks that path's extension against the Writer's supported
    extensions. If it isn't supported, raises UnsupportedFileTypeError.

-----
Static methods defined here:

__new__(
    cls,
    path: FilePath | WriteExcelBuffer | ExcelWriter,
    engine: str | None = None,
    date_format: str | None = None,
    datetime_format: str | None = None,
    mode: str = 'w',
    storage_options: StorageOptions | None = None,
    if_sheet_exists: ExcelWriterIfSheetExists | None = None,
    engine_kwargs: dict | None = None
) -> Self from pandas.io.excel._base.ExcelWriter
    Create and return a new object. See help(type) for accurate signature.

-----
Readonly properties defined here:

book
    Book instance. Class type will depend on the engine used.

```



```

        This attribute can be used to access engine-specific features.

date_format
    Format string for dates written into Excel files (e.g. 'YYYY-MM-DD').

datetime_format
    Format string for dates written into Excel files (e.g. 'YYYY-MM-DD').

engine
    Name of engine.

if_sheet_exists
    How to behave when writing to a sheet that already exists in append mode.

sheets
    Mapping of sheet names to sheet objects.

supported_extensions
    Extensions that writer engine supports.

-----
Data descriptors defined here:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
-----
Data and other attributes defined here:
__annotations__ = {'_engine': 'str', '_supported_extensions': 'tuple[s...
__orig_bases__ = (typing.Generic[~WorkbookT],)
__parameters__ = (~WorkbookT,)
-----
Class methods inherited from typing.Generic:
__class_getitem__(...)
    Parameterizes a generic class.

    At least, parameterizing a generic class is the *main* thing this
    method does. For example, for some generic class `Foo`, this is called
    when we do `Foo[int]` - there, with `cls=Foo` and `params=int`.

    However, note that this method is also called when defining generic
    classes in the first place with `class Foo[T]: ...`.

__init_subclass__(...)
    Function to initialize subclasses.

class Flags(builtins.object)
    Flags(obj: NDFrame, *, allows_duplicate_labels: bool) -> None

    Flags that apply to pandas objects.

    "Flags" differ from "metadata". Flags reflect properties of the pandas
    object (the Series or DataFrame). Metadata refer to properties of the
    dataset, and should be stored in DataFrame.attrs.

    Parameters
    -----
    obj : Series or DataFrame
        The object these flags are associated with.
    allows_duplicate_labels : bool, default True
        Whether to allow duplicate labels in this object. By default,
        duplicate labels are permitted. Setting this to ``False`` will
        cause an :class:`errors.DuplicateLabelError` to be raised when
        `index` (or columns for DataFrame) is not unique, or any
        subsequent operation on introduces duplicates.
        See :ref:`duplicates.disallow` for more.

        .. warning::

```

This is an experimental feature. Currently, many methods fail to propagate the ``allows\_duplicate\_labels`` value. In future versions it is expected that every method taking or returning one or more DataFrame or Series objects will propagate ``allows\_duplicate\_labels``.

See Also

DataFrame.attrs : Dictionary of global attributes of this dataset.
Series.attrs : Dictionary of global attributes of this dataset.

Examples

Attributes can be set in two ways:

```
>>> df = pd.DataFrame()
>>> df.flags
<Flags(allows_duplicate_labels=True)>
>>> df.flags.allows_duplicate_labels = False
>>> df.flags
<Flags(allows_duplicate_labels=False)>

>>> df.flags["allows_duplicate_labels"] = True
>>> df.flags
<Flags(allows_duplicate_labels=True)>
```

Methods defined here:

__eq__(self, other: object) -> bool from pandas.core.flags.Flags
Return self==value.

__getitem__(self, key: str) from pandas.core.flags.Flags

__init__(self, obj: NDFrame, *, allows_duplicate_labels: bool) -> None from pandas.core.flags.Flags
Initialize self. See help(type(self)) for accurate signature.

__repr__(self) -> str from pandas.core.flags.Flags
Return repr(self).

__setitem__(self, key: str, value) -> None from pandas.core.flags.Flags

Data descriptors defined here:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

allows_duplicate_labels
Whether this object allows duplicate labels.

Setting ``allows\_duplicate\_labels=False`` ensures that the index (and columns of a DataFrame) are unique. Most methods that accept and return a Series or DataFrame will propagate the value of ``allows\_duplicate\_labels``.

See :ref:`duplicates` for more.

See Also

DataFrame.attrs : Set global metadata on this object.
DataFrame.set_flags : Set global flags on this object.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]}, index=["a", "a"])
>>> df.flags.allows_duplicate_labels
True
>>> df.flags.allows_duplicate_labels = False
Traceback (most recent call last):
...
pandas.errors.DuplicateLabelError: Index has duplicates.
positions
label
a      [0, 1]
```

```

-----
Data and other attributes defined here:
__annotations__ = {'_keys': 'set[str]'}
__hash__ = None

class Float32Dtype(pandas.core.arrays.floating.FloatingDtype)
    An ExtensionDtype for float32 data.

    This dtype uses ``pd.NA`` as missing value indicator.

    Attributes
    -----
    None

    Methods
    -----
    None

    See Also
    -----
    CategoricalDtype : Type for categorical data with the categories and orderedness.
    IntegerDtype : An ExtensionDtype to hold a single size & kind of integer dtype.
    StringDtype : An ExtensionDtype for string data.

    Examples
    -----
    For Float32Dtype:

    >>> ser = pd.Series([2.25, pd.NA], dtype=pd.Float32Dtype())
    >>> ser.dtype
    Float32Dtype()

    For Float64Dtype:

    >>> ser = pd.Series([2.25, pd.NA], dtype=pd.Float64Dtype())
    >>> ser.dtype
    Float64Dtype()

    Method resolution order:
    Float32Dtype
    pandas.core.arrays.floating.FloatingDtype
    pandas.core.arrays.numeric.NumericDtype
    pandas.core.dtypes.dtypes.BaseMaskedDtype
    pandas.api.extensions.ExtensionDtype
    builtins.object

    Data and other attributes defined here:
    __annotations__ = {'name': 'ClassVar[str]'}
    name = 'Float32'

    type = <class 'numpy.float32'>
        Single-precision floating-point number type, compatible with C ``float``.

        :Character code: ``'f'``
        :Canonical name: ``numpy.single``
        :Alias on this platform (win32 AMD64): ``numpy.float32``: 32-bit-precision floating-point number

    -----
    Methods inherited from pandas.core.arrays.floating.FloatingDtype:

    construct_array_type(self) -> type[FloatingArray]
        Return the array type associated with this dtype.

        Returns
        -----
        type

    -----
    Methods inherited from pandas.core.arrays.numeric.NumericDtype:

    __from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
        Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

```

```

__repr__(self) -> str
    Return repr(self).

-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:
is_signed_integer
is_unsigned_integer

-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.

-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.

-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
itemsizesize
    Return the number of bytes in this dtype
kind
numpy_dtype
    Return an instance of our numpy dtype

-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
base = None

-----
Methods inherited from pandas.api.extensions.ExtensionDtype:
__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

```

```
Parameters
-----
shape : int or tuple[int]

Returns
-----
ExtensionArray
```

Class methods inherited from pandas.api.extensions.ExtensionDtype:

`construct_from_string(string: str) -> Self` from pandas.core.dtypes.base.ExtensionDtype
Construct this type from a string.

This is useful mainly for data types that accept parameters.
For example, a period dtype accepts a frequency parameter that
can be set as `period[h]` (where H means hourly frequency).

By default, in the abstract class, just the name of the type is
expected. But subclasses can overwrite this method to accept
parameters.

```
Parameters
-----
string : str
    The name of the type, for example category.
```

```
Returns
-----
ExtensionDtype
    Instance of the dtype.
```

```
Raises
-----
TypeError
    If a class cannot be constructed from this 'string'.
```

```
Examples
-----
For extension dtypes with arguments the following may be an  
adequate implementation.
```

```
>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )
```

`is_dtype(dtype: object) -> bool` from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

```
Parameters
-----
dtype : object
    The object to check.
```

```
Returns
-----
bool
```

```
Notes
-----
The default implementation is True if
```

1. `cls.construct_from_string(dtype)` is an instance of `cls`.
 2. `dtype` is an object and is an instance of `cls`.
 3. `dtype` has a `dtype` attribute, and any of the above conditions is true for `dtype.dtype`.
-

```

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:
names
    Ordered list of field names, or None if there are no fields.

    This is for compatibility with NumPy arrays, and may be removed in the
    future.

-----
Data descriptors inherited from pandas.api.extensions.ExtensionDtype:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
index_class
    The Index subclass to return from Index.__new__ when this dtype is
    encountered.

class Float64Dtype(pandas.core.arrays.floating.FloatingDtype)
    An ExtensionDtype for float64 data.

    This dtype uses ``pd.NA`` as missing value indicator.

    Attributes
    -----
    None

    Methods
    -----
    None

    See Also
    -----
    CategoricalDtype : Type for categorical data with the categories and orderedness.
    IntegerDtype : An ExtensionDtype to hold a single size & kind of integer dtype.
    StringDtype : An ExtensionDtype for string data.

    Examples
    -----
    For Float32Dtype:

    >>> ser = pd.Series([2.25, pd.NA], dtype=pd.Float32Dtype())
    >>> ser.dtype
    Float32Dtype()

    For Float64Dtype:

    >>> ser = pd.Series([2.25, pd.NA], dtype=pd.Float64Dtype())
    >>> ser.dtype
    Float64Dtype()

    Method resolution order:
    Float64Dtype
    pandas.core.arrays.floating.FloatingDtype
    pandas.core.arrays.numeric.NumericDtype
    pandas.core.dtypes.dtypes.BaseMaskedDtype
    pandas.api.extensions.ExtensionDtype
    builtins.object

    Data and other attributes defined here:
    __annotations__ = {'name': 'ClassVar[str]'}
    name = 'Float64'

    type = <class 'numpy.float64'>
        Double-precision floating-point number type, compatible with Python :class:`float` and C

        :Character code: ``'d'``
        :Canonical name: `numpy.double`
        :Alias on this platform (win32 AMD64): `numpy.float64`: 64-bit precision floating-point
    -----

```

```

Methods inherited from pandas.core.arrays.floating.FloatingDtype:

construct_array_type(self) -> type[FloatingArray]
    Return the array type associated with this dtype.

    Returns
    -----
    type
-----
Methods inherited from pandas.core.arrays.numeric.NumericDtype:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str
    Return repr(self).
-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

is_signed_integer

is_unsigned_integer
-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.
-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.
-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

itemsizesize
    Return the number of bytes in this dtype

kind

numpy_dtype
    Return an instance of our numpy dtype
-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None
-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

```

```

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray
    -----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

    Parameters
    -----
    string : str
        The name of the type, for example ``category``.

    Returns
    -----
    ExtensionDtype
        Instance of the dtype.

    Raises
    -----
    TypeError
        If a class cannot be constructed from this 'string'.

    Examples
    -----
    For extension dtypes with arguments the following may be an
    adequate implementation.

    >>> import re
    >>> @classmethod
    ... def construct_from_string(cls, string):
    ...     pattern = re.compile(r"^my_type\[({P<arg_name>.+})\]$")
    ...     match = pattern.match(string)
    ...     if match:
    ...         return cls(**match.groupdict())
    ...     else:
    ...         raise TypeError(
    ...             f"Cannot construct a '{cls.__name__}' from '{string}'"
    ...         )

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

    Parameters
    -----
    dtype : object
        The object to check.

    Returns
    -----

```



```

-----
bool

Notes
-----
The default implementation is True if

1. ``cls.construct_from_string(dtype)`` is an instance
   of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above
   conditions is true for ``dtype.dtype``.

-----
Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names
    Ordered list of field names, or None if there are no fields.

    This is for compatibility with NumPy arrays, and may be removed in the
    future.

-----
Data descriptors inherited from pandas.api.extensions.ExtensionDtype:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
index_class
    The Index subclass to return from Index.__new__ when this dtype is
    encountered.

```

class Grouper(builtins.object)

```

    Grouper(*args, **kwargs)

    A Grouper allows the user to specify a groupby instruction for an object.

    This specification will select a column via the key parameter, or if the
    level parameter is given, a level of the index of the target
    object.

    If ``level`` is passed as a keyword to both `Grouper` and
    `groupby`, the values passed to `Grouper` take precedence.

    Parameters
    -----
    *args
        Currently unused, reserved for future use.
    **kwargs
        Dictionary of the keyword arguments to pass to Grouper.

    Attributes
    -----
    key : str, defaults to None
        Groupby key, which selects the grouping column of the target.
    level : name/number, defaults to None
        The level for the target index.
    freq : str / frequency object, defaults to None
        This will groupby the specified frequency if the target selection
        (via key or level) is a datetime-like object. For full specification
        of available frequencies, please see :ref:`here<timeseries.offset_aliases>`.
    sort : bool, default to False
        Whether to sort the resulting labels.
    closed : {'left' or 'right'}
        Closed end of interval. Only when `freq` parameter is passed.
    label : {'left' or 'right'}
        Interval boundary to use for labeling.
        Only when `freq` parameter is passed.
    convention : {'start', 'end', 'e', 's'}
        If grouper is PeriodIndex and `freq` parameter is passed.

    origin : Timestamp or str, default 'start_day'
        The timestamp on which to adjust the grouping. The timezone of origin must
        match the timezone of the index.

```

If string, must be one of the following:

- 'epoch': `origin` is 1970-01-01
- 'start': `origin` is the first value of the timeseries
- 'start_day': `origin` is the first day at midnight of the timeseries
- 'end': `origin` is the last value of the timeseries
- 'end_day': `origin` is the ceiling midnight of the last day

offset : Timedelta or str, default is None
An offset timedelta added to the origin.

dropna : bool, default True
If True, and if group keys contain NA values, NA values together with row/column will be dropped. If False, NA values will also be treated as the key in groups.

Returns

Grouper or pandas.api.typing.TimeGrouper
A TimeGrouper is returned if ``freq`` is not ``None``. Otherwise, a Grouper is returned.

See Also

Series.groupby : Apply a function groupby to a Series.
DataFrame.groupby : Apply a function groupby.

Examples

``df.groupby(pd.Grouper(key="Animal"))`` is equivalent to ``df.groupby('Animal')``

```
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Parrot", "Falcon", "Falcon", "Parrot"],
...         "Speed": [100, 5, 200, 300, 15],
...     }
... )
>>> df
   Animal  Speed
0  Falcon   100
1  Parrot    5
2  Falcon   200
3  Falcon   300
4  Parrot    15
>>> df.groupby(pd.Grouper(key="Animal")).mean()
   Speed
Animal
Falcon  200.0
Parrot   10.0
```

Specify a resample operation on the column 'Publish date'

```
>>> df = pd.DataFrame(
...     {
...         "Publish date": [
...             pd.Timestamp("2000-01-02"),
...             pd.Timestamp("2000-01-02"),
...             pd.Timestamp("2000-01-09"),
...             pd.Timestamp("2000-01-16"),
...         ],
...         "ID": [0, 1, 2, 3],
...         "Price": [10, 20, 30, 40],
...     }
... )
>>> df
   Publish date  ID  Price
0  2000-01-02    0    10
1  2000-01-02    1    20
2  2000-01-09    2    30
3  2000-01-16    3    40
>>> df.groupby(pd.Grouper(key="Publish date", freq="1W")).mean()
   ID  Price
Publish date
2000-01-02    0.5   15.0
2000-01-09    2.0   30.0
2000-01-16    3.0   40.0
```

If you want to adjust the start of the bins based on a fixed timestamp:

```
>>> start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
>>> rng = pd.date_range(start, end, freq="7min")
>>> ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
>>> ts
2000-10-01 23:30:00    0
2000-10-01 23:37:00    3
2000-10-01 23:44:00    6
2000-10-01 23:51:00    9
2000-10-01 23:58:00   12
2000-10-02 00:05:00   15
2000-10-02 00:12:00   18
2000-10-02 00:19:00   21
2000-10-02 00:26:00   24
Freq: 7min, dtype: int64

>>> ts.groupby(pd.Grouper(freq="17min")).sum()
2000-10-01 23:14:00    0
2000-10-01 23:31:00    9
2000-10-01 23:48:00   21
2000-10-02 00:05:00   54
2000-10-02 00:22:00   24
Freq: 17min, dtype: int64

>>> ts.groupby(pd.Grouper(freq="17min", origin="epoch")).sum()
2000-10-01 23:18:00    0
2000-10-01 23:35:00   18
2000-10-01 23:52:00   27
2000-10-02 00:09:00   39
2000-10-02 00:26:00   24
Freq: 17min, dtype: int64

>>> ts.groupby(pd.Grouper(freq="17min", origin="2000-01-01")).sum()
2000-10-01 23:24:00    3
2000-10-01 23:41:00   15
2000-10-01 23:58:00   45
2000-10-02 00:15:00   45
Freq: 17min, dtype: int64
```

If you want to adjust the start of the bins with an `offset` `Timedelta`, the two following lines are equivalent:

```
>>> ts.groupby(pd.Grouper(freq="17min", origin="start")).sum()
2000-10-01 23:30:00    9
2000-10-01 23:47:00   21
2000-10-02 00:04:00   54
2000-10-02 00:21:00   24
Freq: 17min, dtype: int64

>>> ts.groupby(pd.Grouper(freq="17min", offset="23h30min")).sum()
2000-10-01 23:30:00    9
2000-10-01 23:47:00   21
2000-10-02 00:04:00   54
2000-10-02 00:21:00   24
Freq: 17min, dtype: int64
```

To replace the use of the deprecated `base` argument, you can now use `offset`, in this example it is equivalent to have `base=2`:

```
>>> ts.groupby(pd.Grouper(freq="17min", offset="2min")).sum()
2000-10-01 23:16:00    0
2000-10-01 23:33:00    9
2000-10-01 23:50:00   36
2000-10-02 00:07:00   39
2000-10-02 00:24:00   24
Freq: 17min, dtype: int64
```

Methods defined here:

```
__init__(
    self,
    key=None,
    level=None,
    freq=None,
    sort: bool = False,
```

```

        dropna: bool = True
    ) -> None from pandas.core.groupby.grouper.Grouper
        Initialize self. See help(type(self)) for accurate signature.

    __repr__(self) -> str from pandas.core.groupby.grouper.Grouper
        Return repr(self).

-----
Static methods defined here:

    __new__(cls, *args, **kwargs) from pandas.core.groupby.grouper.Grouper
        Create and return a new object. See help(type) for accurate signature.

-----
Data descriptors defined here:

    __dict__
        dictionary for instance variables

    __weakref__
        list of weak references to the object

-----
Data and other attributes defined here:

    __annotations__ = {'_attributes': 'tuple[str, ...]', '_grouper': 'Inde...
class HDFStore(builtins.object)
    HDFStore(
        path,
        mode: str = 'a',
        complevel: int | None = None,
        complib=None,
        fletcher32: bool = False,
        **kwargs
    ) -> None

    Dict-like IO interface for storing pandas objects in PyTables.

    Either Fixed or Table format.

    .. warning::

        Pandas uses PyTables for reading and writing HDF5 files, which allows
        serializing object-dtype data with pickle when using the "fixed" format.
        Loading pickled data received from untrusted sources can be unsafe.

        See: https://docs.python.org/3/library/pickle.html for more.

    Parameters
    -----
    path : str
        File path to HDF5 file.
    mode : {'a', 'w', 'r', 'r+'}, default 'a'

        ``'r'``
            Read-only; no data can be modified.
        ``'w'``
            Write; a new file is created (an existing file with the same
            name would be deleted).
        ``'a'``
            Append; an existing file is opened for reading and writing,
            and if the file does not exist it is created.
        ``'r+'``
            It is similar to ``'a'``, but the file must already exist.
    complevel : int, 0-9, default None
        Specifies a compression level for data.
        A value of 0 or None disables compression.
    complib : {'zlib', 'lzo', 'bzip2', 'blosc'}, default 'zlib'
        Specifies the compression library to be used.
        These additional compressors for Blosc are supported
        (default if no compressor specified: 'blosc:blosclz'):
        {'blosc:blosclz', 'blosc:lz4', 'blosc:lz4hc', 'blosc:snappy',
        'blosc:zlib', 'blosc:zstd'}.
        Specifying a compression library which is not available issues
        a ValueError.
    fletcher32 : bool, default False

```

```

        If applying compression use the fletcher32 checksum.
    **kwargs
        These parameters will be passed to the PyTables open_file method.

Examples
-----
>>> bar = pd.DataFrame(np.random.randn(10, 4))
>>> store = pd.HDFStore("test.h5")
>>> store["foo"] = bar # write to HDF5
>>> bar = store["foo"] # retrieve
>>> store.close()

**Create or load HDF5 file in-memory**

When passing the `driver` option to the PyTables open_file method through
**kwargs, the HDF5 file is loaded or created in-memory and will only be
written when closed:

>>> bar = pd.DataFrame(np.random.randn(10, 4))
>>> store = pd.HDFStore("test.h5", driver="H5FD_CORE")
>>> store["foo"] = bar
>>> store.close() # only now, data is written to disk

Methods defined here:

__contains__(self, key: str) -> bool from pandas.io.pytables.HDFStore
    check for existence of this key
    can match the exact pathname or the pathnm w/o the leading '/'

__delitem__(self, key: str) -> int | None from pandas.io.pytables.HDFStore

__enter__(self) -> Self from pandas.io.pytables.HDFStore

__exit__(
    self,
    exc_type: type[BaseException] | None,
    exc_value: BaseException | None,
    traceback: TracebackType | None
) -> None from pandas.io.pytables.HDFStore

__fspath__(self) -> str from pandas.io.pytables.HDFStore

__getattr__(self, name: str) from pandas.io.pytables.HDFStore
    allow attribute access to get stores

__getitem__(self, key: str) from pandas.io.pytables.HDFStore

__init__(
    self,
    path,
    mode: str = 'a',
    complevel: int | None = None,
    complib=None,
    fletcher32: bool = False,
    **kwargs
) -> None from pandas.io.pytables.HDFStore
    Initialize self. See help(type(self)) for accurate signature.

__iter__(self) -> Iterator[str] from pandas.io.pytables.HDFStore

__len__(self) -> int from pandas.io.pytables.HDFStore

__repr__(self) -> str from pandas.io.pytables.HDFStore
    Return repr(self).

__setitem__(self, key: str, value) -> None from pandas.io.pytables.HDFStore

append(
    self,
    key: str,
    value: DataFrame | Series,
    format=None,
    axes=None,
    index: bool | list[str] = True,
    append: bool = True,
    complib=None,
    complevel: int | None = None,

```

```

columns=None,
min_itemsize: int | dict[str, int] | None = None,
nan_rep=None,
chunksize: int | None = None,
expectedrows=None,
dropna: bool | None = None,
data_columns: Literal[True] | list[str] | None = None,
encoding=None,
errors: str = 'strict'
) -> None from pandas.io.pytables.HDFStore
Append to Table in file.

Node must already exist and be Table format.

Parameters
-----
key : str
    Key of object to append.
value : {Series, DataFrame}
    Value of object to append.
format : 'table' is the default
    Format to use when storing object in HDFStore. Value can be one of:

    ``'table'``
        Table format. Write as a PyTables Table structure which may perform
        worse but allow more flexible operations like searching / selecting
        subsets of the data.
axes : default None
    This parameter is currently not accepted.
index : bool, default True
    Write DataFrame index as a column.
append : bool, default True
    Append the input data to the existing.
complib : default None
    This parameter is currently not accepted.
complevel : int, 0-9, default None
    Specifies a compression level for data.
    A value of 0 or None disables compression.
columns : default None
    This parameter is currently not accepted, try data_columns.
min_itemsize : int, dict, or None
    Dict of columns that specify minimum str sizes.
nan_rep : str
    Str to use as str nan representation.
chunksize : int or None
    Size to chunk the writing.
expectedrows : int
    Expected TOTAL row size of this table.
dropna : bool, default False, optional
    Do not write an ALL nan row to the store settable
    by the option 'io.hdf.dropna_table'.
data_columns : list of columns, or True, default None
    List of columns to create as indexed data columns for on-disk
    queries, or True to use all columns. By default only the axes
    of the object are indexed. See `here
    <https://pandas.pydata.org/pandas-docs/stable/user\_guide/io.html#query-via-data-columns>
encoding : default None
    Provide an encoding for str.
errors : str, default 'strict'
    The error handling scheme to use for encoding errors.
    The default is 'strict' meaning that encoding errors raise a
    UnicodeEncodeError. Other possible values are 'ignore', 'replace' and
    'xmlcharrefreplace' as well as any other name registered with
    codecs.register_error that can handle UnicodeEncodeErrors.

See Also
-----
HDFStore.append_to_multiple : Append to multiple tables.

Notes
-----
Does *not* check if data being appended overlaps with existing
data in the table, so be careful

Examples
-----
>>> df1 = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])

```

```
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data", df1, format="table") # doctest: +SKIP
>>> df2 = pd.DataFrame([[5, 6], [7, 8]], columns=["A", "B"])
>>> store.append("data", df2) # doctest: +SKIP
>>> store.close() # doctest: +SKIP
```

```
   A  B
0  1  2
1  3  4
0  5  6
1  7  8
```

```
append_to_multiple(
    self,
    d: dict,
    value,
    selector,
    data_columns=None,
    axes=None,
    dropna: bool = False,
    **kwargs
```

```
) -> None from pandas.io.pytables.HDFStore
Append to multiple tables
```

Parameters

d : a dict of table_name to table_columns, None is acceptable as the values of one node (this will get all the remaining columns)
value : a pandas object
selector : a string that designates the indexable table; all of its columns will be designed as data_columns, unless data_columns is passed, in which case these are used
data_columns : list of columns to create as data columns, or True to use all columns
dropna : if evaluates to True, drop rows from all tables if any single row in each table has all NaN. Default False.

Notes

axes parameter is currently not accepted

```
close(self) -> None from pandas.io.pytables.HDFStore
Close the PyTables file handle
```

```
copy(
    self,
    file,
    mode: str = 'w',
    propindexes: bool = True,
    keys=None,
    complib=None,
    complevel: int | None = None,
    fletcher32: bool = False,
    overwrite: bool = True
```

```
) -> HDFStore from pandas.io.pytables.HDFStore
Copy the existing store to a new file, updating in place.
```

Parameters

propindexes : bool, default True
Restore indexes in copied file.
keys : list, optional
List of keys to include in the copy (defaults to all).
overwrite : bool, default True
Whether to overwrite (remove and replace) existing nodes in the new store.
mode, complib, complevel, fletcher32 same as in HDFStore.__init__

Returns

open file handle of the new store

```
create_table_index(
    self,
    key: str,
    columns=None,
    optlevel: int | None = None,
    kind: str | None = None
) -> None from pandas.io.pytables.HDFStore
```

Create a pytables index on the table.

Parameters

key : str

columns : None, bool, or listlike[str]

Indicate which columns to create an index on.

* False : Do not create any indexes.

* True : Create indexes on all columns.

* None : Create indexes on all columns.

* listlike : Create indexes on the given columns.

optlevel : int or None, default None

Optimization level, if None, pytables defaults to 6.

kind : str or None, default None

Kind of index, if None, pytables defaults to "medium".

Raises

TypeError: raises if the node is not a table

flush(self, fsync: bool = False) -> None from pandas.io.pytables.HDFStore
Force all buffered modifications to be written to disk.

Parameters

fsync : bool (default False)

call ``os.fsync()`` on the file handle to force writing to disk.

Notes

Without ``fsync=True``, flushing may not guarantee that the OS writes to disk. With fsync, the operation will block until the OS claims the file has been written; however, other caching layers may still interfere.

get(self, key: str) from pandas.io.pytables.HDFStore
Retrieve pandas object stored in file.

Parameters

key : str

Object to retrieve from file. Raises KeyError if not found.

Returns

object

Same type as object stored in file.

See Also

HDFStore.get_node : Returns the node with the key.

HDFStore.get_storer : Returns the storer object for a key.

Examples

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
```

```
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
```

```
>>> store.put("data", df) # doctest: +SKIP
```

```
>>> store.get("data") # doctest: +SKIP
```

```
>>> store.close() # doctest: +SKIP
```

get_node(self, key: str) -> Node | None from pandas.io.pytables.HDFStore
return the node with the key or None if it does not exist

get_storer(self, key: str) -> GenericFixed | Table from pandas.io.pytables.HDFStore
return the storer object for a key, raise if not in the file

groups(self) -> list from pandas.io.pytables.HDFStore
Return a list of all the top-level nodes.

Each node returned is not a pandas storage object.

Returns

list

List of objects.

See Also

HDFStore.get_node : Returns the node with the key.

Examples

>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data", df) # doctest: +SKIP
>>> print(store.groups()) # doctest: +SKIP
>>> store.close() # doctest: +SKIP
[/data (Group) '
 children := ['axis0' (Array), 'axis1' (Array), 'block0_values' (Array),
 'block0_items' (Array)]

info(self) -> str from pandas.io.pytables.HDFStore
Print detailed information on the store.

Returns

str

A String containing the python pandas class name, filepath to the HDF5 file and all the object keys along with their respective dataframe shapes.

See Also

HDFStore.get_storer : Returns the storer object for a key.

Examples

>>> df1 = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
>>> df2 = pd.DataFrame([[5, 6], [7, 8]], columns=["C", "D"])
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data1", df1) # doctest: +SKIP
>>> store.put("data2", df2) # doctest: +SKIP
>>> print(store.info()) # doctest: +SKIP
>>> store.close() # doctest: +SKIP
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5
/data1 frame (shape->[2,2])
/data2 frame (shape->[2,2])

items(self) -> Iterator[tuple[str, list]] from pandas.io.pytables.HDFStore
iterate on key->group

keys(self, include: str = 'pandas') -> list[str] from pandas.io.pytables.HDFStore
Return a list of keys corresponding to objects stored in HDFStore.

Parameters

include : str, default 'pandas'
When kind equals 'pandas' return pandas objects.
When kind equals 'native' return native HDF5 Table objects.

Returns

list

List of ABSOLUTE path-names (e.g. have the leading '/').

Raises

raises ValueError if kind has an illegal value

See Also

HDFStore.info : Prints detailed information on the store.
HDFStore.get_node : Returns the node with the key.
HDFStore.get_storer : Returns the storer object for a key.

Examples

>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data", df) # doctest: +SKIP

```
>>> store.get("data") # doctest: +SKIP
>>> print(store.keys()) # doctest: +SKIP
['/data1', '/data2']
>>> store.close() # doctest: +SKIP

open(self, mode: str = 'a', **kwargs) -> None from pandas.io.pytables.HDFStore
Open the file in the specified mode
```

Parameters

```
-----
mode : {'a', 'w', 'r', 'r+'}, default 'a'
    See HDFStore docstring or tables.open_file for info about modes
**kwargs
    These parameters will be passed to the PyTables open_file method.
```

```
put(
    self,
    key: str,
    value: DataFrame | Series,
    format=None,
    index: bool = True,
    append: bool = False,
    complib=None,
    complevel: int | None = None,
    min_itemsize: int | dict[str, int] | None = None,
    nan_rep=None,
    data_columns: Literal[True] | list[str] | None = None,
    encoding=None,
    errors: str = 'strict',
    track_times: bool = True,
    dropna: bool = False
) -> None from pandas.io.pytables.HDFStore
Store object in HDFStore.
```

This method writes a pandas DataFrame or Series into an HDF5 file using either the fixed or table format. The `table` format allows additional operations like incremental appends and queries but may have performance trade-offs. The `fixed` format provides faster read/write operations but does not support appends or queries.

Parameters

```
-----
key : str
    Key of object to store in file.
value : {Series, DataFrame}
    Value of object to store in file.
format : 'fixed(f)|table(t)', default is 'fixed'
    Format to use when storing object in HDFStore. Value can be one of:

    ``'fixed'``
        Fixed format. Fast writing/reading. Not-appendable, nor searchable.
    ``'table'``
        Table format. Write as a PyTables Table structure which may perform
        worse but allow more flexible operations like searching / selecting
        subsets of the data.
index : bool, default True
    Write DataFrame index as a column.
append : bool, default False
    This will force Table format, append the input data to the existing.
complib : default None
    This parameter is currently not accepted.
complevel : int, 0-9, default None
    Specifies a compression level for data.
    A value of 0 or None disables compression.
min_itemsize : int, dict, or None
    Dict of columns that specify minimum str sizes.
nan_rep : str
    Str to use as str nan representation.
data_columns : list of columns or True, default None
    List of columns to create as data columns, or True to use all columns.
    See `here
    <https://pandas.pydata.org/pandas-docs/stable/user\_guide/io.html#query-via-data-columns>
encoding : str, default None
    Provide an encoding for strings.
errors : str, default 'strict'
    The error handling scheme to use for encoding errors.
    The default is 'strict' meaning that encoding errors raise a
```

UnicodeEncodeError. Other possible values are 'ignore', 'replace' and 'xmlcharrefreplace' as well as any other name registered with codecs.register_error that can handle UnicodeEncodeErrors.

track_times : bool, default True
 Parameter is propagated to 'create_table' method of 'PyTables'.
 If set to False it enables to have the same h5 files (same hashes) independent on creation time.

dropna : bool, default False, optional
 Remove missing values.

See Also

HDFStore.info : Prints detailed information on the store.
 HDFStore.get_storer : Returns the storer object for a key.

Examples

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data", df) # doctest: +SKIP
```

remove(self, key: str, where=None, start=None, stop=None) -> int | None from pandas.io.pytables
 Remove pandas object partially by specifying the where condition

Parameters

key : str
 Node to remove or delete rows from

where : list of Term (or convertible) objects, optional

start : integer (defaults to None), row number to start selection

stop : integer (defaults to None), row number to stop selection

Returns

number of rows removed (or None if not a Table)

Raises

raises KeyError if key is not a valid store

```
select(
    self,
    key: str,
    where=None,
    start=None,
    stop=None,
    columns=None,
    iterator: bool = False,
    chunksize: int | None = None,
    auto_close: bool = False
) from pandas.io.pytables.HDFStore
```

Retrieve pandas object stored in file, optionally based on where criteria.

.. warning::

Pandas uses PyTables for reading and writing HDF5 files, which allows serializing object-dtype data with pickle when using the "fixed" format. Loading pickled data received from untrusted sources can be unsafe.

See: <https://docs.python.org/3/library/pickle.html> for more.

Parameters

key : str
 Object being retrieved from file.

where : list or None
 List of Term (or convertible) objects, optional.

start : int or None
 Row number to start selection.

stop : int, default None
 Row number to stop selection.

columns : list or None
 A list of columns that if not None, will limit the return columns.

iterator : bool or False
 Returns an iterator.

chunksize : int or None
 Number or rows to include in iteration, return an iterator.

`auto_close` : bool or False
Should automatically close the store when finished.

Returns

object
Retrieved object from file.

See Also

`HDFStore.select_as_coordinates` : Returns the selection as an index.

`HDFStore.select_column` : Returns a single column from the table.

`HDFStore.select_as_multiple` : Retrieves pandas objects from multiple tables.

Examples

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
```

```
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
```

```
>>> store.put("data", df) # doctest: +SKIP
```

```
>>> store.get("data") # doctest: +SKIP
```

```
>>> print(store.keys()) # doctest: +SKIP
```

```
['/data1', '/data2']
```

```
>>> store.select("/data1") # doctest: +SKIP
```

```
   A  B
```

```
0  1  2
```

```
1  3  4
```

```
>>> store.select("/data1", where="columns == A") # doctest: +SKIP
```

```
   A
```

```
0  1
```

```
1  3
```

```
>>> store.close() # doctest: +SKIP
```

```
select_as_coordinates(  
    self,  
    key: str,  
    where=None,  
    start: int | None = None,  
    stop: int | None = None  
) from pandas.io.pytables.HDFStore  
    return the selection as an Index
```

.. warning::

Pandas uses PyTables for reading and writing HDF5 files, which allows serializing object-dtype data with pickle when using the "fixed" format. Loading pickled data received from untrusted sources can be unsafe.

See: <https://docs.python.org/3/library/pickle.html> for more.

Parameters

`key` : str

`where` : list of Term (or convertible) objects, optional

`start` : integer (defaults to None), row number to start selection

`stop` : integer (defaults to None), row number to stop selection

```
select_as_multiple(  
    self,  
    keys,  
    where=None,  
    selector=None,  
    columns=None,  
    start=None,  
    stop=None,  
    iterator: bool = False,  
    chunksize: int | None = None,  
    auto_close: bool = False  
) from pandas.io.pytables.HDFStore  
    Retrieve pandas objects from multiple tables.
```

.. warning::

Pandas uses PyTables for reading and writing HDF5 files, which allows serializing object-dtype data with pickle when using the "fixed" format. Loading pickled data received from untrusted sources can be unsafe.

See: <https://docs.python.org/3/library/pickle.html> for more.

Parameters

keys : a list of the tables
selector : the table to apply the where criteria (defaults to keys[0] if not supplied)
columns : the columns I want back
start : integer (defaults to None), row number to start selection
stop : integer (defaults to None), row number to stop selection
iterator : bool, return an iterator, default False
chunksize : n rows to include in iteration, return an iterator
auto_close : bool, default False
Should automatically close the store when finished.

Raises

raises KeyError if keys or selector is not found or keys is empty
raises TypeError if keys is not a list or tuple
raises ValueError if the tables are not ALL THE SAME DIMENSIONS

```
select_column(  
    self,  
    key: str,  
    column: str,  
    start: int | None = None,  
    stop: int | None = None  
) from pandas.io.pytables.HDFStore  
    return a single column from the table. This is generally only useful to  
    select an indexable
```

.. warning::

Pandas uses PyTables for reading and writing HDF5 files, which allows serializing object-dtype data with pickle when using the "fixed" format. Loading pickled data received from untrusted sources can be unsafe.

See: <https://docs.python.org/3/library/pickle.html> for more.

Parameters

key : str
column : str
The column of interest.
start : int or None, default None
stop : int or None, default None

Raises

raises KeyError if the column is not found (or key is not a valid store)
raises ValueError if the column can not be extracted individually (it is part of a data block)

walk(self, where: str = '/') -> Iterator[tuple[str, list[str], list[str]]] from pandas.io.py
Walk the pytables group hierarchy for pandas objects.

This generator will yield the group path, subgroups and pandas object names for each group.

Any non-pandas PyTables objects that are not a group will be ignored.

The `where` group itself is listed first (preorder), then each of its child groups (following an alphanumerical order) is also traversed, following the same procedure.

Parameters

where : str, default "/"
Group where to start walking.

Yields

path : str
Full path to a group (without trailing '/').
groups : list
Names (strings) of the groups contained in `path`.

```
leaves : list
    Names (strings) of the pandas objects contained in `path`.
```

See Also

HDFStore.info : Prints detailed information on the store.

Examples

```
>>> df1 = pd.DataFrame([[1, 2], [3, 4]], columns=["A", "B"])
>>> store = pd.HDFStore("store.h5", "w") # doctest: +SKIP
>>> store.put("data", df1, format="table") # doctest: +SKIP
>>> df2 = pd.DataFrame([[5, 6], [7, 8]], columns=["A", "B"])
>>> store.append("data", df2) # doctest: +SKIP
>>> store.close() # doctest: +SKIP
>>> for group in store.walk(): # doctest: +SKIP
...     print(group) # doctest: +SKIP
>>> store.close() # doctest: +SKIP
```

Readonly properties defined here:

filename

is_open

return a boolean indicating whether the file is open

root

return the root node

Data descriptors defined here:

__dict__

dictionary for instance variables

__weakref__

list of weak references to the object

Data and other attributes defined here:

__annotations__ = {'_handle': 'File | None', '_mode': 'str'}

```
class Index(pandas.core.base.IndexOpsMixin, pandas.core.base.PandasObject)
```

```
    Index(
        data=None,
        dtype=None,
        copy: bool | None = None,
        name=None,
        tupleize_cols: bool = True
    ) -> Self
```

Immutable sequence used for indexing and alignment.

The basic object storing axis labels for all pandas objects.

.. versionchanged:: 2.0.0

Index can hold all numpy numeric dtypes (except float16). Previously only int64/uint64/float64 dtypes were accepted.

Parameters

data : array-like (1-dimensional)

An array-like structure containing the data for the index. This could be a Python list, a NumPy array, or a pandas Series.

dtype : str, numpy.dtype, or ExtensionDtype, optional

Data type for the output Index. If not specified, this will be inferred from `data`.

See the :ref:`user guide <basics.dtypes>` for more usages.

copy : bool, default None

Whether to copy input data, only relevant for array, Series, and Index inputs (for other input, e.g. a list, a new array is created anyway).

Defaults to True for array input and False for Index/Series.

Set to False to avoid copying array input at your own risk (if you know the input data won't be modified elsewhere).

Set to True to force copying Series/Index input up front.
name : object
Name to be stored in the index.
tupleize_cols : bool (default: True)
When True, attempt to create a MultiIndex if possible.

See Also

RangeIndex : Index implementing a monotonic integer range.
CategoricalIndex : Index of :class:`Categorical` s.
MultiIndex : A multi-level, or hierarchical Index.
IntervalIndex : An Index of :class:`Interval` s.
DatetimeIndex : Index of datetime64 data.
TimedeltaIndex : Index of timedelta64 data.
PeriodIndex : Index of Period data.

Notes

An Index instance can **only** contain hashable objects.
An Index instance **can not** hold numpy float16 dtype.

Examples

>>> pd.Index([1, 2, 3])
Index([1, 2, 3], dtype='int64')

>>> pd.Index(list("abc"))
Index(['a', 'b', 'c'], dtype='str')

>>> pd.Index([1, 2, 3], dtype="uint8")
Index([1, 2, 3], dtype='uint8')

Method resolution order:

Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
builtins.object

Methods defined here:

__abs__(self) -> Index from pandas.core.indexes.base.Index

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
The array interface, return my values.

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index

__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__contains__(self, key: Any) -> bool from pandas.core.indexes.base.Index
Return a boolean indicating whether the provided key is in the index.

Parameters

key : label
The key to check if it is present in the index.

Returns

bool
Whether the key search is in the index.

Raises

TypeError
If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the
list-like key is in the index.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')

>>> 2 in idx
True
>>> 6 in idx
False
```

```
__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
Parameters
-----
memo, default None
    Standard signature. Unused

__getitem__(self, key) from pandas.core.indexes.base.Index
    Override numpy.ndarray's __getitem__ method to work as desired.

    This function adds lists and Series as valid boolean indexers
    (ndarrays only supports ndarray with dtype=bool).

    If resulting ndim != 1, plain ndarray is returned instead of
    corresponding `Index` subclass.

__iadd__(self, other) from pandas.core.indexes.base.Index

__invert__(self) -> Index from pandas.core.indexes.base.Index

__len__(self) -> int from pandas.core.indexes.base.Index
    Return the length of the Index.

__neg__(self) -> Index from pandas.core.indexes.base.Index

__pos__(self) -> Index from pandas.core.indexes.base.Index

__reduce__(self) from pandas.core.indexes.base.Index
    Helper for pickle.

__repr__(self) -> str_t from pandas.core.indexes.base.Index
    Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether all elements are Truthy.

Parameters
-----
*args
    Required for compatibility with numpy.
**kwargs
    Required for compatibility with numpy.

Returns
-----
bool or array-like (if axis is specified)
    A single element array-like may be converted to bool.

See Also
-----
Index.any : Return whether any element in an Index is True.
Series.any : Return whether any element in a Series is True.
Series.all : Return whether all elements in a Series are True.

Notes
-----
Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples
-----
True, because nonzero integers are considered True.
```



```
>>> pd.Index([1, 2, 3]).all()
True

False, because ``0`` is considered False.
```

```
>>> pd.Index([0, 1, 2]).all()
False
```

`any(self, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return whether any element is Truthy.

Parameters

`*args`
Required for compatibility with numpy.
`**kwargs`
Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)
A single element array-like may be converted to bool.

See Also

`Index.all` : Return whether all elements are True.
`Series.all` : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
>>> index.any()
True
```

```
>>> index = pd.Index([0, 0, 0])
>>> index.any()
False
```

`append(self, other: Index | Sequence[Index])` -> `Index` from `pandas.core.indexes.base.Index`
Append a collection of `Index` options together.

Parameters

`other` : `Index` or list/tuple of indices
Single `Index` or a collection of indices, which can be either a list or a tuple.

Returns

`Index`

Returns a new `Index` object resulting from appending the provided other indices to the original `Index`.

See Also

`Index.insert` : Make new `Index` inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.append(pd.Index([4]))
Index([1, 2, 3, 4], dtype='int64')
```

`argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` -> `int` from `pandas`
Return int position of the largest value in the `Index`.

If the maximum is achieved in multiple locations,
the first row position is returned.

Parameters

`axis` : `None`

Unused. Parameter needed for compatibility with DataFrame.
 skipna : bool, default True
 Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
 and there is an NA value, this method will raise a ``ValueError``.
 *args, **kwargs
 Additional arguments and keywords for compatibility with NumPy.

Returns

int
 Row position of the maximum value.

See Also

Series.argmax : Return position of the maximum value.
 Series.argmin : Return position of the minimum value.
 numpy.ndarray.argmax : Equivalent method for numpy arrays.
 Series.idxmax : Return index label of the maximum values.
 Series.idxmin : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
 the minimum cereal calories is the first element,
 since index is zero-indexed.

argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from
 Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations,
 the first row position is returned.

Parameters

axis : None
 Unused. Parameter needed for compatibility with DataFrame.
 skipna : bool, default True
 Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
 and there is an NA value, this method will raise a ``ValueError``.
 *args, **kwargs
 Additional arguments and keywords for compatibility with NumPy.

Returns

int
 Row position of the minimum value.

See Also

Series.argmin : Return position of the minimum value.
 Series.argmax : Return position of the maximum value.
 numpy.ndarray.argmin : Equivalent method for numpy arrays.
 Series.idxmin : Return index label of the minimum values.
 Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
```

```
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

```
argsort(self, *args, **kwargs) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
```

Return the integer indices that would sort the index.

Parameters

*args

Passed to ``numpy.ndarray.argsort``.

**kwargs

Passed to ``numpy.ndarray.argsort``.

Returns

`np.ndarray[np.intp]`

Integer indices that would sort the index if used as an indexer.

See Also

`numpy.argsort` : Similar method for NumPy arrays.

`Index.sort_values` : Return sorted copy of Index.

Examples

```
>>> idx = pd.Index(["b", "a", "d", "c"])
```

```
>>> idx
```

```
Index(['b', 'a', 'd', 'c'], dtype='str')
```

```
>>> order = idx.argsort()
```

```
>>> order
```

```
array([1, 0, 3, 2])
```

```
>>> idx[order]
```

```
Index(['a', 'b', 'c', 'd'], dtype='str')
```

```
asof(self, label) from pandas.core.indexes.base.Index
```

Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

label : object

The label up to which the method returns the latest index label.

Returns

object

The passed label if it is in the index. The previous label if the passed label is not in the sorted index or ``NaN`` if there is no such label.

See Also

`Series.asof` : Return the latest value in a Series up to the passed index.

`merge_asof` : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).

`Index.get_loc` : An ``asof`` is a thin wrapper around ``get_loc`` with `method='pad'`.

Examples

``Index.asof`` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
```

```
>>> idx.asof("2014-01-01")
```

```
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label, NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

```
asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp] from pandas
Return the locations (indices) of labels in the index.
```

As in the :meth:`pandas.Index.asof`, if the label (a particular entry in ``where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in ``where``, -1 is returned.

``mask`` is used to ignore ``NA`` values in the index during calculation.

Parameters

```
where : Index
    An Index consisting of an array of timestamps.
mask : np.ndarray[bool]
    Array of booleans denoting where values in the original
    data are not ``NA``.
```

Returns

```
np.ndarray[np.intp]
    An array of locations (indices) of the labels from the index
    which correspond to the return values of :meth:`pandas.Index.asof`
    for every element in ``where``.
```

See Also

Index.asof : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use ``mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

```
astype(self, dtype: Dtype, copy: bool = True) from pandas.core.indexes.base.Index
Create an Index with values cast to dtypes.
```

The class of a new Index is determined by dtype. When conversion is impossible, a TypeError exception is raised.

Parameters

```
dtype : numpy dtype or pandas type
    Note that any signed integer `dtype` is treated as ``'int64'``,
    and any unsigned integer `dtype` is treated as ``'uint64'``,
    regardless of the size.
```

copy : bool, default True
By default, astype always returns a newly allocated object.
If copy is set to False and internal requirements on dtype are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index

Index with values cast to specified dtype.

See Also

Index.dtype: Return the dtype object of the underlying data.

Index.dtypes: Return the dtype object of the underlying data.

Index.convert_dtypes: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.astype("float")
```

```
Index([1.0, 2.0, 3.0], dtype='float64')
```

copy(self, name: Hashable | None = None, deep: bool = False) -> Self from pandas.core.indexes

Make a copy of this object.

Name is set on the new object.

Parameters

name : Label, optional

Set name for new object.

deep : bool, default False

If True attempts to make a deep copy of the Index.

Else makes a shallow copy.

Returns

Index

Index refer to new object which is a copy of this object.

See Also

Index.delete: Make new Index with passed location(-s) deleted.

Index.drop: Make new Index with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> new_idx = idx.copy()
```

```
>>> idx is new_idx
```

```
False
```

delete(self, loc: int | np.integer | list[int] | npt.NDArray[np.integer]) -> Self from pandas

Make new Index with passed location(-s) deleted.

Parameters

loc : int or list of int

Location of item(-s) which will be deleted.

Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

numpy.delete : Delete any rows and column from NumPy array (ndarray).

Examples

```
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete(1)
Index(['a', 'c'], dtype='str')
```

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')
```

`diff(self, periods: int = 1) -> Index` from `pandas.core.indexes.base.Index`
Computes the difference between consecutive values in the Index object.

If `periods` is greater than 1, computes the difference between values that are ``periods`` number of positions apart.

Parameters

`periods` : int, optional
The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

`Index`
A new Index object with the computed differences.

Examples

```
-----
>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')
```

`difference(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Return a new Index with elements of index not in ``other``.

This is the set difference of two Index objects.

Parameters

`other` : Index or array-like
Index object or an array-like object containing elements to be compared with the elements of the original Index.
`sort` : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

- * None : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.
- * False : Do not sort the result.
- * True : Sort the result (which may raise `TypeError`).

Returns

`Index`
Returns a new Index object containing elements that are in the original Index but not in the ``other`` Index.

See Also

`Index.symmetric_difference` : Compute the symmetric difference of two Index objects.
`Index.intersection` : Form the intersection of two Index objects.

Examples

```
-----
>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')
```

`drop(`

```

self,
labels: Index | np.ndarray | Iterable[Hashable],
errors: IgnoreRaise = 'raise'
) -> Index from pandas.core.indexes.base.Index
Make new Index with passed list of labels deleted.

Parameters
-----
labels : array-like or scalar
    Array-like object or a scalar value, representing the labels to be removed
    from the Index.
errors : {'ignore', 'raise'}, default 'raise'
    If 'ignore', suppress error and existing labels are dropped.

Returns
-----
Index
    Will be same type as self, except for RangeIndex.

Raises
-----
KeyError
    If not all of the labels are found in the selected axis

See Also
-----
Index.dropna : Return Index without NA/Nan values.
Index.drop_duplicates : Return Index with duplicate values removed.

Examples
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index
Return Index with duplicate values removed.

Parameters
-----
keep : {'first', 'last', ``False``}, default 'first'
    - 'first' : Drop duplicates except for the first occurrence.
    - 'last' : Drop duplicates except for the last occurrence.
    - ``False`` : Drop all duplicates.

Returns
-----
Index
    A new Index object with the duplicate values removed.

See Also
-----
Series.drop_duplicates : Equivalent method on Series.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Index.duplicated : Related method on Index, indicating duplicate
    Index values.

Examples
-----
Generate a pandas.Index with duplicate values.

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])

The `keep` parameter controls which duplicate values are removed.
The value 'first' keeps the first occurrence for each
set of duplicated entries. The default value of keep is 'first'.

>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')

The value 'last' keeps the last occurrence for each set of duplicated
entries.

>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')

The value ``False`` discards all sets of duplicated entries.

```

```

>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')

```

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0
 If a string is given, must be the name of a level
 If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex
 Returns an Index or MultiIndex object, depending on the resulting index after removing the requested level(s).

See Also

Index.dropna : Return Index without NA/Nan values.

Examples

```

>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
             (2, 4, 6)],
            names=['x', 'y', 'z'])

>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
            names=['y', 'z'])

>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')

```

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/Nan values.

Parameters

how : {'any', 'all'}, default 'any'
 If the Index is a MultiIndex, drop the value when any or all levels are NaN.

Returns

Index
 Returns an Index object after removing NA/Nan values.

See Also

Index.fillna : Fill NA/Nan values with the specified value.
Index.isna : Detect missing values.

Examples

```

>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()

```



```

Index([1.0, 3.0], dtype='float64')
duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes
    Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting
array. Either all duplicates, all except the first, or all except the
last occurrence of duplicates can be indicated.

Parameters
-----
keep : {'first', 'last', False}, default 'first'
    The value or values in a set of duplicates to mark as missing.

    - 'first' : Mark duplicates as ``True`` except for the first
      occurrence.
    - 'last' : Mark duplicates as ``True`` except for the last
      occurrence.
    - ``False`` : Mark all duplicates as ``True``.

Returns
-----
np.ndarray[bool]
    A numpy array of boolean values indicating duplicate index values.

See Also
-----
Series.duplicated : Equivalent method on pandas.Series.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Index.drop_duplicates : Remove duplicate values from Index.

Examples
-----
By default, for each set of duplicated values, the first occurrence is
set to False and all others to True:

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])

which is equivalent to

>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])

By using 'last', the last occurrence of each set of duplicated values
is set on False and all others on True:

>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])

By setting keep on ``False``, all duplicates are True:

>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])

equals(self, other: Any) -> bool from pandas.core.indexes.base.Index
    Determine if two Index object are equal.

    The things that are being compared are:

    * The elements inside the Index object.
    * The order of the elements inside the Index object.

Parameters
-----
other : Any
    The other object to compare against.

Returns
-----
bool
    True if "other" is an Index and it has the same elements and order
    as the calling index; False otherwise.

See Also
-----

```

Index.identical: Checks that object attributes and types are also equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx1
Index([1, 2, 3], dtype='int64')
>>> idx1.equals(pd.Index([1, 2, 3]))
True
```

The elements inside are compared

```
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2
Index(['1', '2', '3'], dtype='str')

>>> idx1.equals(idx2)
False
```

The order is compared

```
>>> ascending_idx = pd.Index([1, 2, 3])
>>> ascending_idx
Index([1, 2, 3], dtype='int64')
>>> descending_idx = pd.Index([3, 2, 1])
>>> descending_idx
Index([3, 2, 1], dtype='int64')
>>> ascending_idx.equals(descending_idx)
False
```

The dtype is *not* compared

```
>>> int64_idx = pd.Index([1, 2, 3], dtype="int64")
>>> int64_idx
Index([1, 2, 3], dtype='int64')
>>> uint64_idx = pd.Index([1, 2, 3], dtype="uint64")
>>> uint64_idx
Index([1, 2, 3], dtype='uint64')
>>> int64_idx.equals(uint64_idx)
True
```

fillna(self, value) from pandas.core.indexes.base.Index
Fill NA/NAN values with the specified value.

Parameters

value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index
NA/NAN values replaced with `value`.

See Also

DataFrame.fillna : Fill NaN values of a DataFrame.
Series.fillna : Fill NaN Values of a Series.

Examples

```
-----
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

```
get_indexer(
    self,
    target,
    method: ReindexMethod | None = None,
    limit: int | None = None,
    tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Compute indexer and mask for new index given the current index.
```

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.
`method` : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.
`limit` : int, optional
Maximum number of consecutive labels in ``target`` to match for inexact matches.
`tolerance` : optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

`np.ndarray[np.intp]`
Integers from 0 to `n - 1` indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

See Also

`Index.get_indexer_for` : Returns an indexer even when non-unique.
`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Notes

Returns -1 for unmatched values, for further explanation see the example below.

Examples

```
>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([ 1,  2, -1])
```

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

`get_indexer_for(self, target) -> npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Guaranteed return of an indexer even when non-unique.

This dispatches to `get_indexer` or `get_indexer_non_unique` as appropriate.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.

Returns

`np.ndarray[np.intp]`
List of indices.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.
`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Examples

```
-----
>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])
```

`get_indexer_non_unique(self, target) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]` from
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.

Returns

`indexer` : `np.ndarray[np.intp]`
Integers from 0 to `n - 1` indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by `-1`.
`missing` : `np.ndarray[np.intp]`
An indexer into the target of the values not found. These correspond to the `-1` in the indexer array.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.
`Index.get_indexer_for` : Returns an indexer even when non-unique.

Examples

```
-----
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned `indexer` contains only integers equal to `-1`. It demonstrates that there's no match between the index and the `target` values at these positions. The mask `[0, 1, 2]` in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in `index`. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

`get_level_values = _get_level_values(self, level) -> Index`

`get_loc(self, key)` from `pandas.core.indexes.base.Index`
Get integer location, slice or boolean mask for requested label.

Parameters

`key` : label
The key to check its location if it is present in the index.

Returns

`int` if unique index, slice if monotonic index, else mask
Integer location, slice or boolean mask.

See Also

`Index.get_slice_bound` : Calculate slice bound that corresponds to given label.

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_non_unique : Returns indexer and masks for new index given the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

```
>>> unique_index = pd.Index(list("abc"))
>>> unique_index.get_loc("b")
1

>>> monotonic_index = pd.Index(list("abbc"))
>>> monotonic_index.get_loc("b")
slice(1, 3, None)

>>> non_monotonic_index = pd.Index(list("abcb"))
>>> non_monotonic_index.get_loc("b")
array([False,  True,  False,  True])
```

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position of given label.

Parameters

label : object
The label for which to calculate the slice bound.
side : {'left', 'right'}
if 'left' return leftmost position of given label.
if 'right' return one-past-the-rightmost position of given label.

Returns

int
Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested label.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.get_slice_bound(3, "left")
3

>>> idx.get_slice_bound(3, "right")
4
```

If ``label`` is non-unique in the index, an error will be raised.

```
>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
KeyError: Cannot get left slice bound for non-unique label: 'a'
```

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
Group the index labels by a given array of values.

Parameters

values : array
Values used to determine the groups.

Returns

dict
{group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
Similar to equals, but checks that object attributes and types are also equal.

Parameters

other : Index
The Index object you want to compare with the current Index object.

Returns

bool
If two Index objects have equal elements and same type True,
otherwise False.

See Also

Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples

>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
If we have an object dtype, try to infer a non-object dtype.

Parameters

copy : bool, default True
Whether to make a copy in cases where no inference occurs.

Returns

Index
An Index with a new dtype if the dtype was inferred
or a shallow copy if the dtype could not be inferred.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')

insert(self, loc: int, item) -> Index from pandas.core.indexes.base.Index
Make new Index inserting new item at location.

Follows Python numpy.insert semantics for negative values.

Parameters

loc : int
The integer location where the new item will be inserted.
item : object
The new item to be inserted into the Index.

Returns

Index
Returns a new Index object resulting from inserting the specified item at
the specified location within the original Index.

See Also

Index.append : Append a collection of Indexes together.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

`intersection(self, other, sort: bool = False)` from `pandas.core.indexes.base.Index`
Form the intersection of two Index objects.

This returns a new Index with elements common to the index and ``other``.

Parameters

```
-----
other : Index or array-like
    An Index or an array-like object containing elements to form the
    intersection with the original Index.
sort : True, False or None, default False
    Whether to sort the resulting index.

    * None : sort the result, except when `self` and `other` are equal
      or when the values cannot be compared.
    * False : do not sort the result.
    * True : Sort the result (which may raise TypeError).
```

Returns

```
-----
Index
    Returns a new Index object with elements common to both the original Index
    and the `other` Index.
```

See Also

```
-----
Index.union : Form the union of two Index objects.
Index.difference : Return a new Index with elements of index not in other.
Index.isin : Return a boolean array where the index values are in values.
```

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.intersection(idx2)
Index([3, 4], dtype='int64')
```

`is_(self, other) -> bool` from `pandas.core.indexes.base.Index`
More flexible, faster check like ``is`` but that works through views.

Note: this is **not** the same as ``Index.identical()``, which checks that metadata is also the same.

Parameters

```
-----
other : object
    Other object to compare against.
```

Returns

```
-----
bool
    True if both have same underlying data, False otherwise.
```

See Also

```
-----
Index.identical : Works like `Index.is_` but also checks metadata.
```

Examples

```
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx1.is_(idx1.view())
True

>>> idx1.is_(idx1.copy())
False
```

`isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_]` from `pandas.core.indexes.base.Index`
Return a boolean array where the index values are in ``values``.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like
 Sought values.
level : str or int, optional
 Name or position of the index level to use (if the index is a
 `MultiIndex`).

Returns

np.ndarray[bool]
 NumPy array of boolean values.

See Also

Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a ``ValueError``.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])  
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(  
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]  
... )  
>>> midx  
MultiIndex([(1, 'red'),  
            (2, 'blue'),  
            (3, 'green')],  
           names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")  
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])  
array([ True, False, False])
```

isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get mapped to ``True`` values. Everything else get mapped to ``False`` values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

numpy.ndarray[bool]
 A boolean array of whether my values are NA.

See Also

Index.notna : Boolean inverse of isna.
Index.dropna : Omit entries with missing values.
isna : Top-level isna.
Series.isna : Detect missing values in Series object.

Examples

Show which entries in a pandas.Index are NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

```
isnull = isna(self) -> npt.NDArray[np.bool_]
```

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index
The other index on which join is performed.
how : {'left', 'right', 'inner', 'outer'}
level : int or level name, default None
It is either the integer position or the name of the level.
return_indexers : bool, default False
Whether to return the indexers or not for both the index objects.
sort : bool, default False
Sort the join keys lexicographically in the result Index. If False, the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)
The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a database-style join.

Examples

>>> idx1 = pd.Index([1, 2, 3])

```
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

`map(self, mapper, na_action: Literal['ignore'] | None = None)` from `pandas.core.indexes.base`.
Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
 Mapping correspondence.
na_action : {None, 'ignore'}
 If 'ignore', propagate NA values, without passing them to the mapping correspondence.

Returns

 Union[Index, MultiIndex]
 The output of the mapping function applied to the index.
 If the function returns a tuple with more than one element a MultiIndex will be returned.

See Also

Index.where : Replace values where the condition is False.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')
```

Using ``map`` with a function:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')
```

`max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` from `pandas.core`.
Return the maximum value of the Index.

Parameters

axis : int, optional
 For compatibility with NumPy. Only 0 or None are allowed.
skipna : bool, default True
 Exclude NA/null values when showing the result.
***args, **kwargs**
 Additional arguments and keywords for compatibility with NumPy.

Returns

 scalar
 Maximum value.

See Also

Index.min : Return the minimum value in an Index.
Series.max : Return the maximum value in a Series.
DataFrame.max : Return the maximum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'
```

For a MultiIndex, the maximum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)
```

`memory_usage(self, deep: bool = False) -> int` from `pandas.core.indexes.base.Index`
Memory usage of the values.

Parameters

`deep` : bool, default False
 Introspect the data deeply, interrogate
 `object` dtypes for system-level memory consumption.

Returns

bytes used
 Returns memory usage of the values in the Index in bytes.

See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of the
 array.

Notes

Memory usage does not include memory consumed by elements that
are not components of the array if `deep=False` or if used on PyPy

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24

`min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return the minimum value of the Index.

Parameters

`axis` : {None}
 Dummy argument for consistency with Series.
`skipna` : bool, default True
 Exclude NA/null values when showing the result.
`*args, **kwargs`
 Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
 Minimum value.

See Also

`Index.max` : Return the maximum value of the object.
`Series.min` : Return the minimum value in a Series.
`DataFrame.min` : Return the minimum values in a DataFrame.

Examples

>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

`notna(self) -> npt.NDArray[np.bool_]` from `pandas.core.indexes.base.Index`

Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to ``False`` values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

notnull = notna(self) -> npt.NDArray[np.bool_]

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters

mask : array-like of bool
Array of booleans denoting where values should be replaced.
value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index
A new Index of the values set with the mask.

See Also

numpy.putmask : Changes elements of an array
based on conditional and input values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')
```

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not implemented currently.

Returns

Index

A view on self.

See Also

`numpy.ndarray.ravel` : Return a flattened array.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')
```

```
reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.
```

Parameters

target : an iterable

An iterable containing the values to be used for creating the new index.

method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional

* default: exact matches only.

* pad / ffill: find the PREVIOUS index value if no exact match.

* backfill / bfill: use NEXT index value if no exact match

* nearest: use the NEAREST index value if no exact match. Tied

distances are broken by preferring the larger index value.

level : int, optional

Level of multiindex.

limit : int, optional

Maximum number of consecutive labels in ``target`` to match for inexact matches.

tolerance : int, float, or list-like, optional

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

new_index : `pd.Index`

Resulting index.

indexer : `np.ndarray[np.intp]` or `None`

Indices of output values in original index.

Raises

TypeError

If ``method`` passed along with ``level``.

ValueError

If non-unique multi-index

ValueError

If non-unique index and ``method`` or ``limit`` passed.

See Also

`Series.reindex` : Conform Series to new index with optional filling logic.

`DataFrame.reindex` : Conform DataFrame to new index with optional filling logic.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
```

```

Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))

```

rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
Alter Index or MultiIndex name.

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters

name : Hashable or a sequence of the previous
 Name(s) to set.
inplace : bool, default False
 Modifies the object directly, instead of creating a new Index or
 MultiIndex.

Returns

Index or None
 The same type as the caller or None if ``inplace=True``.

See Also

Index.set_names : Able to set new names partially and by level.

Examples

>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")
Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

```

>>> idx = pd.MultiIndex.from_product(
...     [ ["python", "cobra"], [2018, 2019] ], names=["kind", "year"]
... )
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.

```

repeat(self, repeats, axis: None = None) -> Self from pandas.core.indexes.base.Index
Repeat elements of an Index.

Returns a new Index where each element of the current Index
is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
 The number of repetitions for each element. This should be a
 non-negative integer. Repeating 0 times will return an empty
 Index.
axis : None
 Must be ``None``. Has no effect but is accepted for compatibility
 with numpy.

Returns

Index
 Newly created Index with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')
```

round(self, decimals: int = 0) -> Self from pandas.core.indexes.base.Index
Round each value in the Index to the given number of decimals.

Parameters

decimals : int, optional
Number of decimal places to round to. If decimals is negative,
it specifies the number of positions to the left of the decimal point.

Returns

Index
A new Index with the rounded values.

Examples

```
-----
>>> import pandas as pd
>>> idx = pd.Index([10.1234, 20.5678, 30.9123, 40.4567, 50.7890])
>>> idx.round(decimals=2)
Index([10.12, 20.57, 30.91, 40.46, 50.79], dtype='float64')
```

set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

names : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

level : int, Hashable or a sequence of the previous, optional
If the index is a MultiIndex and names is not dict-like, level(s) to set
(None for all levels). Otherwise level must be None.

inplace : bool, default False
Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None
The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([( 'python', 2018),
            ( 'python', 2019),
            ( 'cobra', 2018),
            ( 'cobra', 2019)],
           )
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([( 'python', 2018),
```

```

        ('python', 2019),
        ('cobra', 2018),
        ('cobra', 2019)],
        names=['species', 'year'])

```

When renaming levels with a dict, levels can not be passed.

```

>>> idx.set_names({"kind": "snake"})
MultiIndex([( 'python', 2018),
            ( 'python', 2019),
            ( 'cobra', 2018),
            ( 'cobra', 2019)],
            names=['snake', 'year'])

```

`shift(self, periods: int = 1, freq=None) -> Self` from `pandas.core.indexes.base.Index`
Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes by a specified time increment a given number of times.

Parameters

```

periods : int, default 1
    Number of periods (or increments) to shift by,
    can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or str, optional
    Frequency increment to shift by.
    If None, the index is shifted by its own `freq` attribute.
    Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

```

Returns

```

pandas.Index
    Shifted index.

```

See Also

```

Series.shift : Shift values of Series.

```

Notes

This method is only implemented for datetime-like index classes, i.e., `DatetimeIndex`, `PeriodIndex` and `TimedeltaIndex`.

Examples

Put the first 5 month starts of 2011 into an index.

```

>>> month_starts = pd.date_range("1/1/2011", periods=5, freq="MS")
>>> month_starts
DatetimeIndex(['2011-01-01', '2011-02-01', '2011-03-01', '2011-04-01',
              '2011-05-01'],
              dtype='datetime64[us]', freq='MS')

```

Shift the index by 10 days.

```

>>> month_starts.shift(10, freq="D")
DatetimeIndex(['2011-01-11', '2011-02-11', '2011-03-11', '2011-04-11',
              '2011-05-11'],
              dtype='datetime64[us]', freq=None)

```

The default value of `freq` is the `freq` attribute of the index, which is 'MS' (month start) in this example.

```

>>> month_starts.shift(10)
DatetimeIndex(['2011-11-01', '2011-12-01', '2012-01-01', '2012-02-01',
              '2012-03-01'],
              dtype='datetime64[us]', freq='MS')

```

```

slice_indexer(
    self,
    start: Hashable | None = None,
    end: Hashable | None = None,
    step: int | None = None
) -> slice from pandas.core.indexes.base.Index
    Compute the slice indexer for input labels and step.

```


Index needs to be ordered and unique.

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, default None
 If None, defaults to 1.

Returns

slice
 A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)
```

```
slice_locs(
    self,
    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.
```

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, defaults None
 If None, defaults to 1.

Returns

tuple[int, int]
 Returns a tuple of two integers representing the slice locations for the input labels within the index.

See Also

Index.get_loc : Get location for a single label.

Notes

This method only works if the index is monotonic or unique.

Examples

>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")

```

(1, 3)

>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)

```

sort_values

```

self,
*,
return_indexer: bool = False,
ascending: bool = True,
na_position: NaPosition = 'last',
key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.

```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

return_indexer : bool, default False
Should the indices that would sort the index be returned.

ascending : bool, default True
Should the index values be sorted in an ascending order.

na_position : {'first' or 'last'}, default 'last'
Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.

key : callable, optional
If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.

Returns

sorted_index : pandas.Index
Sorted copy of the index.

indexer : numpy.ndarray, optional
The indices that the index itself was sorted by.

See Also

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

```

>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')

```

Sort values in ascending order (default behavior).

```

>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')

```

Sort values in descending order, and also get the indices `idx` was sorted by.

```

>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))

```

sortlevel

```

self,
level=None,
ascending: bool | list[bool] = True,
sort_remaining=None,
na_position: NaPosition = 'first'
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
For internal compatibility with the Index API.

```

Sort the Index. This is for compat with MultiIndex

Parameters

```

    ascending : bool, default True
        False to sort in descending order
    na_position : {'first' or 'last'}, default 'first'
        Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
        the end.

    .. versionadded:: 2.1.0

    level, sort_remaining are compat parameters

    Returns
    -----
    Index

symmetric_difference(
    self,
    other,
    result_name: abc.Hashable | None = None,
    sort: bool | None = None
) from pandas.core.indexes.base.Index
    Compute the symmetric difference of two Index objects.

    Parameters
    -----
    other : Index or array-like
        Index or an array-like object with elements to compute the symmetric
        difference with the original Index.
    result_name : str
        A string representing the name of the resulting Index, if desired.
    sort : bool or None, default None
        Whether to sort the resulting index. By default, the
        values are attempted to be sorted, but any TypeError from
        incomparable elements is caught by pandas.

        * None : Attempt to sort the result, but catch any TypeErrors
          from comparing incomparable elements.
        * False : Do not sort the result.
        * True : Sort the result (which may raise TypeError).

    Returns
    -----
    Index
    Returns a new Index object containing elements that appear in either the
    original Index or the `other` Index, but not both.

    See Also
    -----
    Index.difference : Return a new Index with elements of index not in other.
    Index.union : Form the union of two Index objects.
    Index.intersection : Form the intersection of two Index objects.

    Notes
    -----
    ``symmetric_difference`` contains elements that appear in either
    ``idx1`` or ``idx2`` but not both. Equivalent to the Index created by
    ``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates
    dropped.

    Examples
    -----
    >>> idx1 = pd.Index([1, 2, 3, 4])
    >>> idx2 = pd.Index([2, 3, 4, 5])
    >>> idx1.symmetric_difference(idx2)
    Index([1, 5], dtype='int64')

take(
    self,
    indices,
    axis: Axis = 0,
    allow_fill: bool = True,
    fill_value=None,
    **kwargs
) -> Self from pandas.core.indexes.base.Index
    Return a new Index of the values selected by the indices.

    For internal compatibility with numpy arrays.

```

Parameters

indices : array-like
Indices to be taken.
axis : {0 or 'index'}, optional
The axis over which to select values, always 0 or 'index'.
allow_fill : bool, default True
How to handle negative values in `indices`.

- * False: negative values in `indices` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.
- * True: negative values in `indices` indicate missing values. These values are set to `fill\_value`. Any other negative values raise a ``ValueError``.

fill_value : scalar, default None
If allow_fill=True and fill_value is not None, indices specified by -1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
**kwargs
Required for compatibility with numpy.

Returns

Index
An index formed of elements at the given indices. Will be the same type as self, except for RangeIndex.

See Also

numpy.ndarray.take: Return an array formed from the elements of a at the given indices.

Examples

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.take([2, 2, 1, 2])
Index(['c', 'c', 'b', 'c'], dtype='str')

to_flat_index(self) -> Self from pandas.core.indexes.base.Index
Identity method.

This is implemented for compatibility with subclass implementations when chaining.

Returns

pd.Index
Caller.

See Also

MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.core
Create a DataFrame with a column containing the Index.

Parameters

index : bool, default True
Set the index of the returned DataFrame as the original Index.
name : object, defaults to index.name
The passed name should substitute for the index name (if it has one).

Returns

DataFrame
DataFrame containing the original Index data.

See Also

Index.to_series : Convert an Index to a Series.
Series.to_frame : Convert Series to DataFrame.

Examples

```
-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
      animal
animal
Ant      Ant
Bear    Bear
Cow     Cow
```

By default, the original Index is reused. To enforce a new Index:

```
>>> idx.to_frame(index=False)
      animal
0      Ant
1     Bear
2      Cow
```

To override the name of the resulting column, specify `name`:

```
>>> idx.to_frame(index=False, name="zoo")
      zoo
0      Ant
1     Bear
2      Cow
```

`to_series(self, index=None, name: Hashable | None = None) -> Series` from `pandas.core.indexes`
Create a Series with both index and values equal to the index keys.

Useful with `map` for returning an indexer based on an index.

Parameters

```
-----
index : Index, optional
    Index of resulting Series. If None, defaults to original index.
name : str, optional
    Name of resulting Series. If None, defaults to name of original
    index.
```

Returns

```
-----
Series
    The dtype will be based on the type of the Index values.
```

See Also

```
-----
Index.to_frame : Convert an Index to a DataFrame.
Series.to_frame : Convert Series to DataFrame.
```

Examples

```
-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear    Bear
Cow     Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ``index``:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1     Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear    Bear
Cow     Cow
```

```

Name: zoo, dtype: str

union(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index
    Form the union of two Index objects.

    If the Index objects are incompatible, both Index objects will be
    cast to dtype('object') first.

Parameters
-----
other : Index or array-like
    Index or an array-like object containing elements to form the union
    with the original Index.
sort : bool or None, default None
    Whether to sort the resulting Index.

    * None : Sort the result, except when

        1. `self` and `other` are equal.
        2. `self` or `other` has length 0.
        3. Some values in `self` or `other` cannot be compared.
        A RuntimeWarning is issued in this case.

    * False : do not sort the result.
    * True : Sort the result (which may raise TypeError).

Returns
-----
Index
    Returns a new Index object with all unique elements from both the original
    Index and the `other` Index.

See Also
-----
Index.unique : Return unique values in the index.
Index.intersection : Form the intersection of two Index objects.
Index.difference : Return a new Index with elements of index not in `other`.

Examples
-----
Union matching dtypes

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')

Union mismatched dtypes

>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')

MultiIndex case

>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (3, 'Green')],
           )

```

```

        (2, 'Blue'),
        (2, 'Green'),
        (2, 'Red'),
        (3, 'Green'),
        (3, 'Red')],
    )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )

```

`unique(self, level: Hashable | None = None) -> Self` from `pandas.core.indexes.base.Index`
 Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

`level` : int or hashable, optional
 Only return values from specified level (for MultiIndex).
 If int, gets the level by integer position, else by level name.

Returns

 Index
 Unique values in the index.

See Also

`unique` : Numpy array of unique values in that column.
`Series.unique` : Return unique values of Series object.

Examples

>>> `idx = pd.Index([1, 1, 2, 3, 3])`
>>> `idx.unique()`
Index([1, 2, 3], dtype='int64')

`view(self, cls=None)` from `pandas.core.indexes.base.Index`
 Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the ``cls`` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

`cls` : data-type or ndarray sub-class, optional
 Data-type descriptor of the returned view, e.g., `float32` or `int16`.
 Omitting it results in the view having the same data-type as ``self``.
 This argument can also be specified as an ndarray sub-class, e.g., `np.int64` or `np.float32` which then specifies the type of the returned object.

Returns

 Index or ndarray
 A view of the Index. If ``cls`` is `None`, the returned object is an Index view with the same dtype as the calling object. If a numeric ``cls`` is specified an ndarray view with the new dtype is returned.

Raises

 ValueError
 If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an `'int64'` Index as `'str'`.

See Also

Index.copy : Returns a copy of the Index.
numpy.ndarray.view : Returns a new view of array with the same data.

Examples

```
-----
>>> idx = pd.Index([-1, 0, 1])
>>> idx.view()
Index([-1, 0, 1], dtype='int64')

>>> idx.view(np.uint64)
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
array([  nan,   nan,  0.e+00,  0.e+00,  1.e-45,  0.e+00], dtype=float32)
```

where(self, cond, other=None) -> Index from pandas.core.indexes.base.Index
Replace values where the condition is False.

The replacement is taken from other.

Parameters

cond : bool array-like with the same length as self
Condition to select the values on.
other : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index
A copy of self with values replaced from other
where the condition is False.

See Also

Series.where : Same method for Series.
DataFrame.where : Same method for DataFrame.

Examples

```
-----
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Static methods defined here:

```
__new__(
    cls,
    data=None,
    dtype=None,
    copy: bool | None = None,
    name=None,
    tupleize_cols: bool = True
) -> Self from pandas.core.indexes.base.Index
Create and return a new object. See help(type) for accurate signature.
```

Readonly properties defined here:

has_duplicates
Check if the Index has duplicate values.

Returns

bool
Whether or not the Index has duplicate values.

See Also

Index.is_unique : Inverse method that checks if it has unique values.

Examples

```
-----
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True

>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False
```

is_monotonic_decreasing

Return a boolean if the values are equal or decreasing.

Returns

bool

See Also

Index.is_monotonic_increasing : Check if the values are equal or increasing.

Examples

```
-----
>>> pd.Index([3, 2, 1]).is_monotonic_decreasing
True
>>> pd.Index([3, 2, 2]).is_monotonic_decreasing
True
>>> pd.Index([3, 1, 2]).is_monotonic_decreasing
False
```

is_monotonic_increasing

Return a boolean if the values are equal or increasing.

Returns

bool

See Also

Index.is_monotonic_decreasing : Check if the values are equal or decreasing.

Examples

```
-----
>>> pd.Index([1, 2, 3]).is_monotonic_increasing
True
>>> pd.Index([1, 2, 2]).is_monotonic_increasing
True
>>> pd.Index([1, 3, 2]).is_monotonic_increasing
False
```

nlevels

Number of levels.

shape

Return a tuple of the shape of the underlying data.

See Also

Index.size: Return the number of elements in the underlying data.
Index.ndim: Number of dimensions of the underlying data, by definition 1.
Index.dtype: Return the dtype object of the underlying data.
Index.values: Return an array representing the data in the Index.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)
```

values

Return an array representing the data in the Index.

.. warning::

We recommend using :attr:`Index.array` or :meth:`Index.to\_numpy`, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

Index.array : Reference to the underlying data.

Index.to_numpy : A NumPy array representing the underlying data.

Examples

For :class:`pandas.Index`:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors defined here:

array

The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.

Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

=====

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

dtype

Return the dtype object of the underlying data.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.dtype
dtype('int64')
```

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

bool

See Also

Index.isna : Detect missing values.
Index.dropna : Return Index without NA/NaN values.
Index.fillna : Fill NA/NaN values with the specified value.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", None])
>>> s
a    1
b    2
None 3
dtype: int64
>>> s.index.hasnans
True
```

```

inferred_type
    Return a string of the type inferred from the values.

    See Also
    -----
    Index.dtype : Return the dtype object of the underlying data.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')
    >>> idx.inferred_type
    'integer'

is_unique
    Return if the index has unique values.

    Returns
    -----
    bool

    See Also
    -----
    Index.has_duplicates : Inverse method that checks if it has duplicate values.

    Examples
    -----
    >>> idx = pd.Index([1, 5, 7, 7])
    >>> idx.is_unique
    False

    >>> idx = pd.Index([1, 5, 7])
    >>> idx.is_unique
    True

    >>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
    ...     "category"
    ... )
    >>> idx.is_unique
    False

    >>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
    >>> idx.is_unique
    True

name
    Return Index or MultiIndex name.

    Returns
    -----
    label (hashable object)
        The name of the Index.

    See Also
    -----
    Index.set_names: Able to set new names partially and by level.
    Index.rename: Able to set new names partially and by level.
    Series.name: Corresponding Series property.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3], name="x")
    >>> idx
    Index([1, 2, 3], dtype='int64', name='x')
    >>> idx.name
    'x'

names
    Get names on index.

    This method returns a FrozenList containing the names of the object.
    It's primarily intended for internal use.

    Returns
    -----

```

FrozenList

A FrozenList containing the object's names, contains None if the object does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx.names
FrozenList(['x'])
```

```
>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
>>> idx.names
FrozenList(['x', 'y'])
```

If the index does not have a name set:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])
```

Data and other attributes defined here:

```
__annotations__ = {'__hash__': 'ClassVar[None]', '_attributes': 'list[...]
```

```
__pandas_priority__ = 2000
```

```
str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.
```

NAs stay NA unless handled otherwise by a particular method.
Patterned after Python's string methods, with some inspiration from R's stringr package.

Parameters

data : Series or Index
The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.
Index.str : Vectorized string functions for Index.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

```
__iter__(self) -> Iterator
Return an iterator of the values.
```

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

iterator

An iterator yielding scalar values from the Series.

See Also

Series.items : Lazily iterate over (index, value) tuples.

Examples

```
>>> s = pd.Series([1, 2, 3])
>>> for x in s:
...     print(x)
1
2
3
```

factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.int64], npt.NDArray[np.int64]]
Encode the object as an enumerated type or categorical variable.

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function :func:`pandas.factorize`, and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

Parameters

`sort` : bool, default False
Sort `uniques` and shuffle `codes` to maintain the relationship.
`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray
An integer ndarray that's an indexer into `uniques`.
`uniques.take(codes)` will have the same values as `values`.
`uniques` : ndarray, Index, or Categorical
The unique valid values. When `values` is Categorical, `uniques` is a Categorical. When `values` is some other pandas object, an `Index` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in `values`, `uniques` will *not* contain an entry for it.

See Also

`cut` : Discretize continuous-valued array.
`unique` : Find the unique value in an array.

Notes

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

These examples all show `factorize` as a top-level method like `pd.factorize(values)`. The results are identical for methods like `Series.factorize`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is the maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
```

```
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use\_na\_sentinel=True`` (the default), missing values are indicated in the `codes` with the sentinel value ``-1`` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])
```

`item(self)`

Return the first element of the underlying data as a Python scalar.

Returns

scalar

The first element of Series or Index.

Raises

ValueError

If the data is not length = 1.

See Also

`Index.values` : Returns an array representing the data in the Index.

`Series.head` : Returns the first `n` rows.

Examples

```

-----
>>> s = pd.Series([1])
>>> s.item()
1

```

For an index:

```

>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'

```

nunique(self, dropna: bool = True) -> int
Return number of unique elements in the object.

Excludes NA values by default.

Parameters

```

-----
dropna : bool, default True
        Don't include NaN in the count.

```

Returns

int

An integer indicating the number of unique elements in the object.

See Also

```

-----
DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.

```

Examples

```

-----
>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0    1
1    3
2    5
3    7
4    7
dtype: int64

>>> s.nunique()
4

```

```

searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.

```

Find the indices into a sorted Index `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::

The Index *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

```

-----
value : array-like or scalar
        Values to insert into `self`.
side : {'left', 'right'}, optional
        If 'left', the index of the first suitable location found is given.
        If 'right', return the last such index. If there is no suitable
        index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
        Optional array of integer indices that sort `self` into ascending
        order. They are typically the result of ``np.argsort``.

```

Returns

int or array of int
A scalar or array of insertion points with the same shape as `value`.

See Also

sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

>>> ser = pd.Series([1, 2, 3])
>>> ser
0 1
1 2
2 3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0 2000-03-11
1 2000-03-12
2 2000-03-13
dtype: datetime64[us]

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
... ["apple", "bread", "bread", "cheese", "milk"], ordered=True
...)
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']

>>> ser.searchsorted("bread")
np.int64(1)

>>> ser.searchsorted(["bread"], side="right")
array([3])

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])  
>>> ser  
0    2  
1    1  
2    3  
dtype: int64  
  
>>> ser.searchsorted(1) # doctest: +SKIP  
0 # wrong result, correct would be 1
```

to_list = tolist(self) -> list

to_numpy(
 self,
 dtype: npt.DTypeLike | None = None,
 copy: bool = False,
 na_value: object = <no_default>,
)

```

    **kwargs
) -> np.ndarray
    A NumPy ndarray representing the values in this Series or Index.

Parameters
-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
**kwargs
    Additional keywords passed through to the ``to_numpy`` method
    of the underlying array (for extension arrays).

Returns
-----
numpy.ndarray
    The NumPy ndarray holding the values from this Series or Index.
    The dtype of the array may differ. See Notes.

See Also
-----
Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

Notes
-----
The returned array will be the same up to equality (values equal
in `self` will be equal in the returned array; likewise for values
that are not equal). When `self` contains an ExtensionArray, the
dtype may be different. For example, for a category-dtype Series,
``to_numpy()`` will return a NumPy array and the categorical dtype
will be lost.

For NumPy dtypes, this will be a reference to the actual data stored
in this Series or Index (assuming ``copy=False``). Modifying the result
in place will modify the data stored in the Series or Index (not that
we recommend doing that).

For extension types, ``to_numpy()`` *may* require copying data and
coercing the result to a NumPy type (possibly object), which may be
expensive. When you need a no-copy reference to the underlying data,
:attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of
``to_numpy()`` for various dtypes within pandas.

=====
dtype          array type
=====
category[T]    ndarray[T] (same dtype as input)
period         ndarray[object] (Periods)
interval       ndarray[object] (Intervals)
IntegerNA      ndarray[object]
datetime64[ns] datetime64[ns]
datetime64[ns, tz] ndarray[object] (Timestamps)
=====

Examples
-----
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)

Specify the `dtype` to control how datetime-aware data is represented.
Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp`
objects, each with the correct ``tz``.

>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)

```

```
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

`tolist(self)` -> list

Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list

List containing the values as Python or pandas scalars.

See Also

`numpy.ndarray.tolist` : Return the array as an a.ndim-levels deep nested list of Python scalars.

Examples

For Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.tolist()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.tolist()
[1, 2, 3]
```

`transpose(self, *args, **kwargs)` -> Self

Return the transpose, which is by definition self.

Returns

%(klass)s

`value_counts(`

```
self,
normalize: bool = False,
sort: bool = True,
ascending: bool = False,
bins=None,
dropna: bool = True
```

) -> Series

Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

`normalize` : bool, default False

If True then the object returned will contain the relative frequencies of the unique values.

`sort` : bool, default True

Stable sort by frequencies when True. Preserve the order of the data when False.

```

.. versionchanged:: 3.0.0

    Prior to 3.0.0, the sort was unstable.
ascending : bool, default False
    Sort in ascending order.
bins : int, optional
    Rather than count values, group them into half-open bins,
    a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
    Don't include counts of NaN.

Returns
-----
Series
    Series containing counts of unique values.

See Also
-----
Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples
-----
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by
dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64

**bins**

Bins can be useful for going from a continuous variable to a
categorical variable; instead of counting unique
apparitions of values, divide the index in the specified
number of half-open bins.

>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]     2
(3.0, 4.0]     1
Name: count, dtype: int64

**dropna**

With `dropna` set to `False` we can also see NaN index values.

>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64

**Categorical Dtypes**

Rows with categorical type will be counted as one group
if they have same categories and order.
In the example below, even though ``a``, ``c``, and ``d``
all have the same data types of ``category``,
only ``c`` and ``d`` will be counted as one group
since ``a`` doesn't have the same categories.

```

```

>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64

```

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```

>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str

```

For Index:

```

>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')

```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False

>>> idx_empty = pd.Index([])

```

```
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.
Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.

Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2       Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose      3.1    801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN     NaN     NaN
moose  NaN     NaN     NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk        1.7     NaN
moose      3.0     NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
```



```

    __radd__(self, other)
    __rand__(self, other)
    __rdivmod__(self, other)
    __rfloordiv__(self, other)
    __rmod__(self, other)
    __rmul__(self, other)
    __ror__(self, other)
        Return value|self.
    __rpow__(self, other)
    __rsub__(self, other)
    __rtruediv__(self, other)
    __rxor__(self, other)
    __sub__(self, other)
    __truediv__(self, other)
    __xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
    __dict__
        dictionary for instance variables
    __weakref__
        list of weak references to the object

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
    __hash__ = None

-----
Methods inherited from pandas.core.base.PandasObject:
    __sizeof__(self) -> int
        Generates the total memory usage for an object that returns
        either a value or Series of values

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
    __dir__(self) -> list[str]
        Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.

```

class Int16Dtype(pandas.core.arrays.integer.IntegerDtype)

An ExtensionDtype for int16 integer data.

Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

Attributes

None

Methods

None

See Also

Int8Dtype : 8-bit nullable integer type.
Int16Dtype : 16-bit nullable integer type.

```
Int32Dtype : 32-bit nullable integer type.
Int64Dtype : 64-bit nullable integer type.
```

Examples

For Int8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
>>> ser.dtype
Int8Dtype()
```

For Int16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
>>> ser.dtype
Int16Dtype()
```

For Int32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
>>> ser.dtype
Int32Dtype()
```

For Int64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
>>> ser.dtype
Int64Dtype()
```

For UInt8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
>>> ser.dtype
UInt8Dtype()
```

For UInt16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
>>> ser.dtype
UInt16Dtype()
```

For UInt32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
>>> ser.dtype
UInt32Dtype()
```

For UInt64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
>>> ser.dtype
UInt64Dtype()
```

Method resolution order:

```
Int16Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}

```

```
name = 'Int16'

```

```
type = <class 'numpy.int16'>
Signed integer type, compatible with C ``short``.
```

```
:Character code: ``'h'``
```

```
:Canonical name: `numpy.short`
```

```
:Alias on this platform (win32 AMD64): `numpy.int16`: 16-bit signed integer (``-32_768``
```

```
-----
Methods inherited from pandas.core.arrays.integer.IntegerDtype:
```

```

construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.

    Returns
    -----
    type
-----
Methods inherited from pandas.core.arrays.numeric.NumericDtype:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str
    Return repr(self).
-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:
is_signed_integer
is_unsigned_integer
-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.
-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.
-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

itemsizesize
    Return the number of bytes in this dtype

kind

numpy_dtype
    Return an instance of our numpy dtype
-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None
-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype

```

```

    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

Parameters
-----
shape : int or tuple[int]

Returns
-----
ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

Parameters
-----
string : str
    The name of the type, for example ``category``.

Returns
-----
ExtensionDtype
    Instance of the dtype.

Raises
-----
TypeError
    If a class cannot be constructed from this 'string'.

Examples
-----
For extension dtypes with arguments the following may be an
adequate implementation.

>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

Parameters
-----
dtype : object
    The object to check.

Returns
-----

```

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__

dictionary for instance variables

__weakref__

list of weak references to the object

index_class

The Index subclass to return from Index.__new__ when this dtype is encountered.

class Int32Dtype(pandas.core.arrays.integer.IntegerDtype)

An ExtensionDtype for int32 integer data.

Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

Attributes

None

Methods

None

See Also

Int8Dtype : 8-bit nullable integer type.

Int16Dtype : 16-bit nullable integer type.

Int32Dtype : 32-bit nullable integer type.

Int64Dtype : 64-bit nullable integer type.

Examples

For Int8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
>>> ser.dtype
Int8Dtype()
```

For Int16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
>>> ser.dtype
Int16Dtype()
```

For Int32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
>>> ser.dtype
Int32Dtype()
```

For Int64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
```

```

>>> ser.dtype
Int64Dtype()

For UInt8Dtype:
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
>>> ser.dtype
UInt8Dtype()

For UInt16Dtype:
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
>>> ser.dtype
UInt16Dtype()

For UInt32Dtype:
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
>>> ser.dtype
UInt32Dtype()

For UInt64Dtype:
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
>>> ser.dtype
UInt64Dtype()

Method resolution order:
  Int32Dtype
  pandas.core.arrays.integer.IntegerDtype
  pandas.core.arrays.numeric.NumericDtype
  pandas.core.dtypes.dtypes.BaseMaskedDtype
  pandas.api.extensions.ExtensionDtype
  builtins.object

Data and other attributes defined here:
__annotations__ = {'name': 'ClassVar[str]'}
name = 'Int32'
type = <class 'numpy.int32'>
      Signed integer type, compatible with C ``long``.

      :Character code: ``l``
      :Canonical name: `numpy.long`
      :Alias on this platform (win32 AMD64): `numpy.int32`: 32-bit signed integer (``-2_147_483_648`` to
      2_147_483_647)

-----
Methods inherited from pandas.core.arrays.integer.IntegerDtype:

construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.

    Returns
    -----
    type

-----
Methods inherited from pandas.core.arrays.numeric.NumericDtype:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str
    Return repr(self).

-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

is_signed_integer
is_unsigned_integer

-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```

```

from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.

-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.

-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

itemsizes
    Return the number of bytes in this dtype

kind

numpy_dtype
    Return an instance of our numpy dtype

-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None

-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between 'self' and 'other'.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

```

This is useful mainly for data types that accept parameters. For example, a period dtype accepts a frequency parameter that can be set as ``period[h]`` (where H means hourly frequency).

By default, in the abstract class, just the name of the type is expected. But subclasses can overwrite this method to accept parameters.

Parameters

string : str

The name of the type, for example ``category``.

Returns

ExtensionDtype

Instance of the dtype.

Raises

TypeError

If a class cannot be constructed from this 'string'.

Examples

For extension dtypes with arguments the following may be an adequate implementation.

```
>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )
```

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

Parameters

dtype : object

The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__

dictionary for instance variables


```

|   __weakref__
|       list of weak references to the object
|
|   index_class
|       The Index subclass to return from Index.__new__ when this dtype is
|       encountered.
|
class Int64Dtype(pandas.core.arrays.integer.IntegerDtype)
|   An ExtensionDtype for int64 integer data.
|
|   Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.
|
|   Attributes
|   -----
|   None
|
|   Methods
|   -----
|   None
|
|   See Also
|   -----
|   Int8Dtype : 8-bit nullable integer type.
|   Int16Dtype : 16-bit nullable integer type.
|   Int32Dtype : 32-bit nullable integer type.
|   Int64Dtype : 64-bit nullable integer type.
|
|   Examples
|   -----
|   For Int8Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
|   >>> ser.dtype
|   Int8Dtype()
|
|   For Int16Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
|   >>> ser.dtype
|   Int16Dtype()
|
|   For Int32Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
|   >>> ser.dtype
|   Int32Dtype()
|
|   For Int64Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
|   >>> ser.dtype
|   Int64Dtype()
|
|   For UInt8Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
|   >>> ser.dtype
|   UInt8Dtype()
|
|   For UInt16Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
|   >>> ser.dtype
|   UInt16Dtype()
|
|   For UInt32Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
|   >>> ser.dtype
|   UInt32Dtype()
|
|   For UInt64Dtype:
|
|   >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
|   >>> ser.dtype
|   UInt64Dtype()

```

Method resolution order:

```
Int64Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'Int64'
```

```
type = <class 'numpy.int64'>
```

Default signed integer type, 64bit on 64bit systems and 32bit on 32bit systems.

:Character code: ``'q'``

:Canonical name: `numpy.int\_`

:Alias on this platform (win32 AMD64): `numpy.int64`: 64-bit signed integer (``-9\_223\_372\_036\_854\_775\_807 to 9\_223\_372\_036\_854\_775\_807``)

:Alias on this platform (win32 AMD64): `numpy.intp`: Signed integer large enough to fit platform integers

Methods inherited from pandas.core.arrays.integer.IntegerDtype:

```
construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.
```

Returns

type

Methods inherited from pandas.core.arrays.numeric.NumericDtype:

```
__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.
```

```
__repr__(self) -> str
    Return repr(self).
```

Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

is_signed_integer

is_unsigned_integer

Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.
```

Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
na_value
    Default NA value to use for this type.
```

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
itemsize
    Return the number of bytes in this dtype
```

kind

```
numpy_dtype
    Return an instance of our numpy dtype
```

Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

`__eq__(self, other: object) -> bool` from pandas.core.dtypes.base.ExtensionDtype
Check whether 'other' is equal to self.

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes in `self._metadata` are equal between `self` and `other`.`

Parameters

other : Any

Returns

bool

`__hash__(self) -> int` from pandas.core.dtypes.base.ExtensionDtype
Return hash(self).

`__ne__(self, other: object) -> bool` from pandas.core.dtypes.base.ExtensionDtype
Return self!=value.

`__str__(self) -> str` from pandas.core.dtypes.base.ExtensionDtype
Return str(self).

`empty(self, shape: Shape) -> ExtensionArray` from pandas.core.dtypes.base.ExtensionDtype
Construct an ExtensionArray of this dtype with the given shape.

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Class methods inherited from pandas.api.extensions.ExtensionDtype:

`construct_from_string(string: str) -> Self` from pandas.core.dtypes.base.ExtensionDtype
Construct this type from a string.

This is useful mainly for data types that accept parameters.
For example, a period dtype accepts a frequency parameter that can be set as ``period[h]`` (where H means hourly frequency).

By default, in the abstract class, just the name of the type is expected. But subclasses can overwrite this method to accept parameters.

Parameters

string : str
The name of the type, for example ``category``.

Returns

ExtensionDtype
Instance of the dtype.

Raises

TypeError
If a class cannot be constructed from this 'string'.

Examples

For extension dtypes with arguments the following may be an adequate implementation.

```
>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )
```

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

Parameters

dtype : object
The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

index_class
The Index subclass to return from Index.__new__ when this dtype is encountered.

class Int8Dtype(pandas.core.arrays.integer.IntegerDtype)

An ExtensionDtype for int8 integer data.

Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

Attributes

None

Methods

None

See Also

Int8Dtype : 8-bit nullable integer type.
Int16Dtype : 16-bit nullable integer type.

```
Int32Dtype : 32-bit nullable integer type.  
Int64Dtype : 64-bit nullable integer type.
```

Examples

For Int8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())  
>>> ser.dtype  
Int8Dtype()
```

For Int16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())  
>>> ser.dtype  
Int16Dtype()
```

For Int32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())  
>>> ser.dtype  
Int32Dtype()
```

For Int64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())  
>>> ser.dtype  
Int64Dtype()
```

For UInt8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())  
>>> ser.dtype  
UInt8Dtype()
```

For UInt16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())  
>>> ser.dtype  
UInt16Dtype()
```

For UInt32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())  
>>> ser.dtype  
UInt32Dtype()
```

For UInt64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())  
>>> ser.dtype  
UInt64Dtype()
```

Method resolution order:

```
Int8Dtype  
pandas.core.arrays.integer.IntegerDtype  
pandas.core.arrays.numeric.NumericDtype  
pandas.core.dtypes.dtypes.BaseMaskedDtype  
pandas.api.extensions.ExtensionDtype  
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}  
name = 'Int8'
```

```
type = <class 'numpy.int8'>  
Signed integer type, compatible with C ``char``.
```

```
:Character code: ``b'``  
:Canonical name: `numpy.byte`  
:Alias on this platform (win32 AMD64): `numpy.int8`: 8-bit signed integer (``-128`` to ``127``)
```

Methods inherited from pandas.core.arrays.integer.IntegerDtype:

```

construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.

    Returns
    -----
    type
-----
Methods inherited from pandas.core.arrays.numeric.NumericDtype:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str
    Return repr(self).
-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:
is_signed_integer
is_unsigned_integer
-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
    Construct the MaskedDtype corresponding to the given numpy dtype.
-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
na_value
    Default NA value to use for this type.

    This is used in e.g. ExtensionArray.take. This should be the
    user-facing "boxed" version of the NA value, not the physical NA value
    for storage. e.g. for JSONArray, this is an empty dictionary.
-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
itemsizes
    Return the number of bytes in this dtype
kind
numpy_dtype
    Return an instance of our numpy dtype
-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
base = None
-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype

```

```

        Return hash(self).

    __ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
        Return self!=value.

    __str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
        Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

    Parameters
    -----
    string : str
        The name of the type, for example ``category``.

    Returns
    -----
    ExtensionDtype
        Instance of the dtype.

    Raises
    -----
    TypeError
        If a class cannot be constructed from this 'string'.

    Examples
    -----
    For extension dtypes with arguments the following may be an
    adequate implementation.

    >>> import re
    >>> @classmethod
    ... def construct_from_string(cls, string):
    ...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
    ...     match = pattern.match(string)
    ...     if match:
    ...         return cls(**match.groupdict())
    ...     else:
    ...         raise TypeError(
    ...             f"Cannot construct a '{cls.__name__}' from '{string}'"
    ...         )

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

    Parameters
    -----
    dtype : object
        The object to check.

    Returns
    -----

```

bool

Notes

The default implementation is True if

1. `cls.construct_from_string(dtype)` is an instance of `cls`.
2. `dtype` is an object and is an instance of `cls`
3. `dtype` has a `dtype` attribute, and any of the above conditions is true for `dtype.dtype`.

Readonly properties inherited from `pandas.api.extensions.ExtensionDtype`:

names

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from `pandas.api.extensions.ExtensionDtype`:

`__dict__`

dictionary for instance variables

`__weakref__`

list of weak references to the object

index_class

The Index subclass to return from `Index.__new__` when this dtype is encountered.

class `Interval(pandas._libs.interval.IntervalMixin)`

Immutable object implementing an Interval, a bounded slice-like interval.

Attributes

left : orderable scalar

Left bound for the interval.

right : orderable scalar

Right bound for the interval.

closed : {'right', 'left', 'both', 'neither'}, default 'right'

Whether the interval is closed on the left-side, right-side, both or neither. See the Notes for more detailed explanation.

See Also

`IntervalIndex` : An Index of Interval objects that are all closed on the same side.

`cut` : Convert continuous data into discrete bins (Categorical of Interval objects).

`qcut` : Convert continuous data into bins (Categorical of Interval objects) based on quantiles.

`Period` : Represents a period of time.

Notes

The parameters `left` and `right` must be from the same type, you must be able to compare them and they must satisfy `left <= right`.

A closed interval (in mathematics denoted by square brackets) contains its endpoints, i.e. the closed interval `[0, 5]` is characterized by the conditions `0 <= x <= 5`. This is what `closed='both'` stands for. An open interval (in mathematics denoted by parentheses) does not contain its endpoints, i.e. the open interval `(0, 5)` is characterized by the conditions `0 < x < 5`. This is what `closed='neither'` stands for. Intervals can also be half-open or half-closed, i.e. `[0, 5)` is described by `0 <= x < 5` (`closed='left'`) and `(0, 5]` is described by `0 < x <= 5` (`closed='right'`).

Examples

It is possible to build Intervals of different types, like numeric ones:

```
>>> iv = pd.Interval(left=0, right=5)
```



```

>>> iv
Interval(0, 5, closed='right')

You can check if an element belongs to it, or if it contains another interval:

>>> 2.5 in iv
True
>>> pd.Interval(left=2, right=5, closed='both') in iv
True

You can test the bounds (`closed='right'`, so ` $0 < x \leq 5$ `):

>>> 0 in iv
False
>>> 5 in iv
True
>>> 0.0001 in iv
True

Calculate its length

>>> iv.length
5

You can operate with `+` and `*` over an Interval and the operation
is applied to each of its bounds, so the result depends on the type
of the bound elements

>>> shifted_iv = iv + 3
>>> shifted_iv
Interval(3, 8, closed='right')
>>> extended_iv = iv * 10.0
>>> extended_iv
Interval(0.0, 50.0, closed='right')

To create a time interval you can use Timestamps as the bounds

>>> year_2017 = pd.Interval(pd.Timestamp('2017-01-01 00:00:00'),
...                          pd.Timestamp('2018-01-01 00:00:00'),
...                          closed='left')
>>> pd.Timestamp('2017-01-01 00:00:00') in year_2017
True
>>> year_2017.length
Timedelta('365 days 00:00:00')

Method resolution order:
  Interval
  pandas._libs.interval.IntervalMixin
  builtins.object

Methods defined here:

__add__(self, object, /)

__contains__(self, key, /)
    Return bool(key in self).

__eq__(self, value, /)
    Return self==value.

__floordiv__(self, object, /)

__ge__(self, value, /)
    Return self>=value.

__gt__(self, value, /)
    Return self>value.

__hash__(self, /)
    Return hash(self).

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__le__(self, value, /)
    Return self<=value.

```

```

__lt__(self, value, /)
    Return self<value.

__mul__(self, object, /)

__ne__(self, value, /)
    Return self!=value.

__radd__(self, object, /)

__reduce__(self)

__repr__(self, /)
    Return repr(self).

__rfloordiv__(self, value, /)
    Return value//self.

__rmul__(self, object, /)

__rsub__(self, value, /)
    Return value-self.

__rtruediv__(self, value, /)
    Return value/self.

__str__(self, /)
    Return str(self).

__sub__(self, object, /)

__truediv__(self, object, /)

overlaps(self, other)
    Check whether two Interval objects overlap.

    Two intervals overlap if they share a common point, including closed
    endpoints. Intervals that only have an open endpoint in common do not
    overlap.

    Parameters
    -----
    other : Interval
        Interval to check against for an overlap.

    Returns
    -----
    bool
        True if the two intervals overlap.

    See Also
    -----
    IntervalArray.overlaps : The corresponding method for IntervalArray.
    IntervalIndex.overlaps : The corresponding method for IntervalIndex.

    Examples
    -----
    >>> i1 = pd.Interval(0, 2)
    >>> i2 = pd.Interval(1, 3)
    >>> i1.overlaps(i2)
    True
    >>> i3 = pd.Interval(4, 5)
    >>> i1.overlaps(i3)
    False

    Intervals that share closed endpoints overlap:

    >>> i4 = pd.Interval(0, 1, closed='both')
    >>> i5 = pd.Interval(1, 2, closed='both')
    >>> i4.overlaps(i5)
    True

    Intervals that only have an open endpoint in common do not overlap:

    >>> i6 = pd.Interval(1, 2, closed='neither')
    >>> i4.overlaps(i6)
    False

```

Static methods defined here:

`__new__(*args, **kwargs)`
Create and return a new object. See `help(type)` for accurate signature.

Data descriptors defined here:

`closed`

String describing the inclusive side the intervals.

Either ```left```, ```right```, ```both``` or ```neither```.

See Also

`Interval.closed_left` : Check if the interval is closed on the left side.
`Interval.closed_right` : Check if the interval is closed on the right side.
`Interval.open_left` : Check if the interval is open on the left side.
`Interval.open_right` : Check if the interval is open on the right side.

Examples

```
>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.closed
'left'
```

`left`

Left bound for the interval.

See Also

`Interval.right` : Return the right bound for the interval.
`numpy.ndarray.left` : A similar method in numpy for obtaining the left endpoint(s) of intervals.

Examples

```
>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.left
1
```

`right`

Right bound for the interval.

See Also

`Interval.left` : Return the left bound for the interval.
`numpy.ndarray.right` : A similar method in numpy for obtaining the right endpoint(s) of intervals.

Examples

```
>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.right
2
```

Data and other attributes defined here:

`__array_priority__` = 1000

Methods inherited from `pandas._libs.interval.IntervalMixin`:

`__reduce_cython__(self)`

`__setstate__` = `__setstate_cython__(...)`

`__setstate_cython__(self, __pyx_state)`

Data descriptors inherited from pandas._libs.interval.IntervalMixin:

closed_left

Check if the interval is closed on the left side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is closed on the left-side.

See Also

Interval.closed_right : Check if the interval is closed on the right side.

Interval.open_left : Boolean inverse of closed_left.

Examples

```
>>> iv = pd.Interval(0, 5, closed='left')
```

```
>>> iv.closed_left
```

```
True
```

```
>>> iv = pd.Interval(0, 5, closed='right')
```

```
>>> iv.closed_left
```

```
False
```

closed_right

Check if the interval is closed on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is closed on the left-side.

See Also

Interval.closed_left : Check if the interval is closed on the left side.

Interval.open_right : Boolean inverse of closed_right.

Examples

```
>>> iv = pd.Interval(0, 5, closed='both')
```

```
>>> iv.closed_right
```

```
True
```

```
>>> iv = pd.Interval(0, 5, closed='left')
```

```
>>> iv.closed_right
```

```
False
```

is_empty

Indicates if an interval is empty, meaning it contains no points.

An interval is considered empty if its `left` and `right` endpoints are equal, and it is not closed on both sides. This means that the interval does not include any real points. In the case of an :class:`pandas.arrays.IntervalArray` or :class:`IntervalIndex`, the property returns a boolean array indicating the emptiness of each interval.

Returns

bool or ndarray

A boolean indicating if a scalar :class:`Interval` is empty, or a boolean ``ndarray`` positionally indicating if an ``Interval`` in an :class:`~arrays.IntervalArray` or :class:`IntervalIndex` is empty.

See Also

Interval.length : Return the length of the Interval.

Examples

An `:class:`Interval`` that contains points is not empty:

```
>>> pd.Interval(0, 1, closed='right').is_empty
False
```

An ``Interval`` that does not contain any points is empty:

```
>>> pd.Interval(0, 0, closed='right').is_empty
True
>>> pd.Interval(0, 0, closed='left').is_empty
True
>>> pd.Interval(0, 0, closed='neither').is_empty
True
```

An ``Interval`` that contains a single point is not empty:

```
>>> pd.Interval(0, 0, closed='both').is_empty
False
```

An `:class:`~arrays.IntervalArray`` or `:class:`IntervalIndex`` returns a boolean ``ndarray`` positionally indicating if an ``Interval`` is empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'),
...        pd.Interval(1, 2, closed='neither')]
>>> pd.arrays.IntervalArray(ivs).is_empty
array([ True, False])
```

Missing values are not considered empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'), np.nan]
>>> pd.IntervalIndex(ivs).is_empty
array([ True, False])
```

length

Return the length of the Interval.

See Also

`Interval.is_empty` : Indicates if an interval contains no points.

Examples

```
>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.length
1
```

mid

Return the midpoint of the Interval.

See Also

`Interval.left` : Return the left bound for the interval.
`Interval.right` : Return the right bound for the interval.
`Interval.length` : Return the length of the interval.

Examples

```
>>> iv = pd.Interval(0, 5)
>>> iv.mid
2.5
```

open_left

Check if the interval is open on the left side.

For the meaning of ``closed`` and ``open`` see `:class:`~pandas.Interval``.

Returns

bool

True if the Interval is not closed on the left-side.

See Also

`Interval.open_right` : Check if the interval is open on the right side.

Interval.closed_left : Boolean inverse of open_left.

Examples

```
>>> iv = pd.Interval(0, 5, closed='neither')
>>> iv.open_left
True
```

```
>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.open_left
False
```

open_right

Check if the interval is open on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is not closed on the left-side.

See Also

Interval.open_left : Check if the interval is open on the left side.
Interval.closed_right : Boolean inverse of open_right.

Examples

```
>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.open_right
True
```

```
>>> iv = pd.Interval(0, 5)
>>> iv.open_right
False
```

```
class IntervalDtype(pandas.core.dtypes.dtypes.PandasExtensionDtype)
    IntervalDtype(subtype=None, closed: IntervalClosedType | None = None) -> None

    An ExtensionDtype for Interval data.

    **This is not an actual numpy dtype**, but a duck type.

    Parameters
    -----
    subtype : str, np.dtype
        The dtype of the Interval bounds.
    closed : {'right', 'left', 'both', 'neither'}, default 'right'
        Whether the interval is closed on the left-side, right-side, both or
        neither. See the Notes for more detailed explanation.

    Attributes
    -----
    subtype

    Methods
    -----
    None

    See Also
    -----
    PeriodDtype : An ExtensionDtype for Period data.

    Examples
    -----
    >>> pd.IntervalDtype(subtype="int64", closed="both")
    interval[int64, both]

    Method resolution order:
        IntervalDtype
        pandas.core.dtypes.dtypes.PandasExtensionDtype
        pandas.api.extensions.ExtensionDtype
        builtins.object

    Methods defined here:
```

```

__eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.IntervalDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__from_arrow__(self, array: pa.Array | pa.ChunkedArray) -> IntervalArray from pandas.core.dtypes.dtypes.IntervalArray
    Construct IntervalArray from pyarrow Array/ChunkedArray.

__hash__(self) -> int from pandas.core.dtypes.dtypes.IntervalDtype
    Return hash(self).

__init__(self, subtype=None, closed: IntervalClosedType | None = None) -> None from pandas.core.dtypes.dtypes.IntervalDtype
    Initialize self. See help(type(self)) for accurate signature.

__setstate__(self, state) -> None from pandas.core.dtypes.dtypes.IntervalDtype

__str__(self) -> str_type from pandas.core.dtypes.dtypes.IntervalDtype
    Return str(self).

construct_array_type(self) -> type[IntervalArray] from pandas.core.dtypes.dtypes.IntervalDtype
    Return the array type associated with this dtype.

    Returns
    -----
    type

```

Class methods defined here:

```

construct_from_string(string: str_type) -> IntervalDtype from pandas.core.dtypes.dtypes.IntervalDtype
    attempt to construct this type from a string, raise a TypeError
    if its not possible

is_dtype(dtype: object) -> bool from pandas.core.dtypes.dtypes.IntervalDtype
    Return a boolean if the passed type is an actual dtype that we
    can match (via string or type)

```

Readonly properties defined here:

```

closed

subtype
    The dtype of the Interval bounds.

    See Also
    -----
    IntervalDtype: An ExtensionDtype for Interval data.

    Examples
    -----
    >>> dtype = pd.IntervalDtype(subtype="int64", closed="both")
    >>> dtype.subtype
    dtype('int64')

type
    The scalar type for the array, e.g. ``int``

    It's expected ``ExtensionArray[item]`` returns an instance
    of ``ExtensionDtype.type`` for scalar ``item``, assuming
    that value is valid (not NA). NA values do not need to be
    instances of `type`.

```

Data descriptors defined here:

index_class

Data and other attributes defined here:

__annotations__ = {'_cache_dtypes': 'dict[str_type, PandasExtensionDty...

base = dtype('O')

kind = 'O'

name = 'interval'

num = 103

str = '|008'

Methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

__getstate__(self) -> dict[str_type, Any]
Helper for pickle.

__repr__(self) -> str_type
Return a string representation for a particular object.

Class methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

reset_cache() -> None
clear the cache

Data and other attributes inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

isbuiltin = 0

isnative = 0

itemsize = 8

shape = ()

subdtype = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Return self!=value.

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
Construct an ExtensionArray of this dtype with the given shape.

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

na_value
Default NA value to use for this type.

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

names
Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

class IntervalIndex(pandas.core.indexes.extension.ExtensionIndex)

IntervalIndex(
 data,
 closed: IntervalClosedType | None = None,
 dtype: Dtype | None = None,
 copy: bool | None = None,
 name: Hashable | None = None,
 verify_integrity: bool = True
) -> Self

Immutable index of intervals that are closed on the same side.

Parameters

data : array-like (1-dimensional)
 Array-like (ndarray, :class:`DateTimeArray`, :class:`TimeDeltaArray`) containing
 Interval objects from which to build the IntervalIndex.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
 Whether the intervals are closed on the left-side, right-side, both or
 neither.
dtype : dtype or None, default None
 If None, dtype will be inferred.
copy : bool, default None
 Whether to copy input data, only relevant for array, Series, and Index
 inputs (for other input, e.g. a list, a new array is created anyway).
 Defaults to True for array input and False for Index/Series.
 Set to False to avoid copying array input at your own risk (if you
 know the input data won't be modified elsewhere).
 Set to True to force copying Series/Index input up front.
name : object, optional
 Name to be stored in the index.
verify_integrity : bool, default True
 Verify that the IntervalIndex is valid.

Attributes

left
right
closed
mid
length
is_empty
is_non_overlapping_monotonic
is_overlapping
values

Methods

from_arrays
from_tuples
from_breaks
contains
overlaps
set_closed
to_tuples

See Also

Index : The base pandas Index type.
Interval : A bounded slice-like interval; the elements of an IntervalIndex.
interval_range : Function to create a fixed frequency IntervalIndex.
cut : Bin values into discrete Intervals.
qcut : Bin values into equal-sized Intervals based on rank or sample quantiles.

Notes

See the `user guide

<https://pandas.pydata.org/pandas-docs/stable/user_guide/advanced.html#intervalindex>`__
for more.

Examples

A new ``IntervalIndex`` is typically constructed using

:func:`interval\_range`:

```
>>> pd.interval_range(start=0, end=5)
IntervalIndex([(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]],
              dtype='interval[int64, right]')
```

It may also be constructed using one of the constructor

methods: :meth:`IntervalIndex.from\_arrays`,

:meth:`IntervalIndex.from\_breaks`, and :meth:`IntervalIndex.from\_tuples`.

See further examples in the doc strings of ``interval\_range`` and the
mentioned constructor methods.

Method resolution order:

```
IntervalIndex
pandas.core.indexes.extension.ExtensionIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
builtins.object
```

Methods defined here:

```
__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.indexes.interval.IntervalIndex
    Return the IntervalArray's data as a numpy array of Interval
    objects (with dtype='object')
```

```
__contains__(self, key: Any) -> bool from pandas.core.indexes.interval.IntervalIndex
    return a boolean if this key is IN the index
    We *only* accept an Interval
```

Parameters

key : Interval

Returns

bool

```
__reduce__(self) from pandas.core.indexes.interval.IntervalIndex
    Helper for pickle.
```

```
contains(self, other) from pandas.core.indexes.extension._inherit_from_data.<locals>
    Check elementwise if the Intervals contain the value.
```

Return a boolean mask whether the value is contained in the Intervals
of the IntervalArray.

Parameters

other : scalar

The value to check whether it is contained in the Intervals.

Returns

boolean array

A boolean mask whether the value is contained in the Intervals.

See Also

Interval.contains : Check whether Interval object contains value.

IntervalArray.overlaps : Check if an Interval overlaps the values in the
IntervalArray.

Examples

```

>>> intervals = pd.arrays.IntervalArray.from_tuples([(0, 1), (1, 3), (2, 4)])
>>> intervals
<IntervalArray>
[(0, 1], (1, 3], (2, 4]]
Length: 3, dtype: interval[int64, right]

>>> intervals.contains(0.5)
array([ True, False, False])

```

`get_indexer_non_unique(self, target: Index) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

target : IntervalIndex or list of Intervals
An iterable containing the values to be used for computing indexer.

Returns

indexer : np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.
missing : np.ndarray[np.intp]
An indexer into the target of the values not found. These correspond to the -1 in the indexer array.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))

In the example below there are no matched values.

```

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))

```

For this reason, the returned `indexer` contains only integers equal to -1. It demonstrates that there's no match between the index and the `target` values at these positions. The mask `[0, 1, 2]` in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in `index`. The second item is a mask shows that the first and third elements are missing.

```

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))

```

`get_loc(self, key) -> int | slice | np.ndarray` from `pandas.core.indexes.interval.IntervalIndex`
Get integer location, slice or boolean mask for requested label.

The `get_loc` method is used to retrieve the integer index, a slice for slicing objects, or a boolean mask indicating the presence of the label in the `IntervalIndex`.

Parameters

key : label
The value or range to find in the `IntervalIndex`.

Returns

int if unique index, slice if monotonic index, else mask
The position or positions found. This could be a single

number, a range, or an array of true/false values
indicating the position(s) of the label.

See Also

IntervalIndex.get_indexer_non_unique : Compute indexer and
mask for new index given the current index.
Index.get_loc : Similar method in the base Index class.

Examples

>>> i1, i2 = pd.Interval(0, 1), pd.Interval(1, 2)
>>> index = pd.IntervalIndex([i1, i2])
>>> index.get_loc(1)
0

You can also supply a point inside an interval.

>>> index.get_loc(1.5)
1

If a label is in several intervals, you get the locations of all the
relevant intervals.

>>> i3 = pd.Interval(0, 2)
>>> overlapping_index = pd.IntervalIndex([i1, i2, i3])
>>> overlapping_index.get_loc(0.5)
array([True, False, True])

Only exact matches will be returned if an interval is provided.

>>> index.get_loc(pd.Interval(0, 1))
0

memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.interval.IntervalIndex
Memory usage of the values.

Parameters

deep : bool, default False
Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption.

Returns

bytes used
Returns memory usage of the values in the Index in bytes.

See Also

numpy.ndarray.nbytes : Total bytes consumed by the elements of the
array.

Notes

Memory usage does not include memory consumed by elements that
are not components of the array if deep=False or if used on PyPy

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24

overlaps(self, other) from pandas.core.indexes.extension._inherit_from_data.<locals>
Check elementwise if an Interval overlaps the values in the IntervalArray.

Two intervals overlap if they share a common point, including closed
endpoints. Intervals that only have an open endpoint in common do not
overlap.

Parameters

other : IntervalArray
Interval to check against for an overlap.

Returns

ndarray
Boolean array positionally indicating where an overlap occurs.

See Also

Interval.overlaps : Check whether two Interval objects overlap.

Examples

>>> data = [(0, 1), (1, 3), (2, 4)]
>>> intervals = pd.arrays.IntervalArray.from_tuples(data)
>>> intervals
<IntervalArray>
[[0, 1], (1, 3], (2, 4]]
Length: 3, dtype: interval[int64, right]

>>> intervals.overlaps(pd.Interval(0.5, 1.5))
array([True, True, False])

Intervals that share closed endpoints overlap:

>>> intervals.overlaps(pd.Interval(1, 3, closed="left"))
array([True, True, True])

Intervals that only have an open endpoint in common do not overlap:

>>> intervals.overlaps(pd.Interval(1, 2, closed="right"))
array([False, True, False])

set_closed(self, closed: IntervalClosedType) -> Self from pandas.core.indexes.extension._in
Return an identical IntervalArray closed on the specified side.

Parameters

closed : {'left', 'right', 'both', 'neither'}
Whether the intervals are closed on the left-side, right-side, both
or neither.

Returns

IntervalArray
A new IntervalArray with the specified side closures.

See Also

IntervalArray.closed : Returns inclusive side of the Interval.
arrays.IntervalArray.closed : Returns inclusive side of the IntervalArray.

Examples

>>> index = pd.arrays.IntervalArray.from_breaks(range(4))
>>> index
<IntervalArray>
[[0, 1], (1, 2], (2, 3]]
Length: 3, dtype: interval[int64, right]
>>> index.set_closed("both")
<IntervalArray>
[[0, 1], [1, 2], [2, 3]]
Length: 3, dtype: interval[int64, both]

to_tuples(self, na_tuple: bool = True) -> np.ndarray from pandas.core.indexes.extension._in
Return an ndarray (if self is IntervalArray) or Index (if self is IntervalIndex)

Parameters

na_tuple : bool, default True
If ``True``, return ``NA`` as a tuple ``(nan, nan)``. If ``False``,
just return ``NA`` as ``nan``.

Returns

ndarray or Index
An ndarray of tuples representing the intervals
if `self` is an IntervalArray.
An Index of tuples representing the intervals
if `self` is an IntervalIndex.

See Also

IntervalArray.to_list : Convert IntervalArray to a list of tuples.
IntervalArray.to_numpy : Convert IntervalArray to a numpy array.
IntervalArray.unique : Find unique intervals in an IntervalArray.

Examples

For :class:`pandas.IntervalArray`:

```
>>> idx = pd.arrays.IntervalArray.from_tuples([(0, 1), (1, 2)])
>>> idx
<IntervalArray>
[(0, 1], (1, 2]]
Length: 2, dtype: interval[int64, right]
>>> idx.to_tuples()
array([(np.int64(0), np.int64(1)), (np.int64(1), np.int64(2))],
      dtype=object)
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=2)
>>> idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> idx.to_tuples()
Index([(0, 1), (1, 2)], dtype='object')
```

Class methods defined here:

```
from_arrays(
    left,
    right,
    closed: IntervalClosedType = 'right',
    name: Hashable | None = None,
    copy: bool = False,
    dtype: Dtype | None = None
) -> IntervalIndex from pandas.core.indexes.interval.IntervalIndex
Construct from two arrays defining the left and right bounds.
```

Parameters

left : array-like (1-dimensional)
Left bounds for each interval.
right : array-like (1-dimensional)
Right bounds for each interval.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
Whether the intervals are closed on the left-side, right-side, both
or neither.
name : str, optional
Name of the resulting IntervalIndex.
copy : bool, default False
Copy the data.
dtype : dtype, optional
If None, dtype will be inferred.

Returns

IntervalIndex

Raises

ValueError
When a value is missing in only one of `left` or `right`.
When a value in `left` is greater than the corresponding value
in `right`.

See Also

interval_range : Function to create a fixed frequency IntervalIndex.
IntervalIndex.from_breaks : Construct an IntervalIndex from an array of
splits.
IntervalIndex.from_tuples : Construct an IntervalIndex from an
array-like of tuples.

Notes

Each element of `left` must be less than or equal to the `right` element at the same position. If an element is missing, it must be missing in both `left` and `right`. A `TypeError` is raised when using an unsupported type for `left` or `right`. At the moment, 'category', 'object', and 'string' subtypes are not supported.

Examples

>>> pd.IntervalIndex.from_arrays([0, 1, 2], [1, 2, 3])
IntervalIndex([(0, 1], (1, 2], (2, 3]],
 dtype='interval[int64, right]')

from_breaks(
 breaks,
 closed: IntervalClosedType | None = 'right',
 name: Hashable | None = None,
 copy: bool = False,
 dtype: Dtype | None = None
) -> IntervalIndex from pandas.core.indexes.interval.IntervalIndex
Construct an IntervalIndex from an array of splits.

Parameters

breaks : array-like (1-dimensional)
 Left and right bounds for each interval.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
 Whether the intervals are closed on the left-side, right-side, both or neither.
name : str, optional
 Name of the resulting IntervalIndex.
copy : bool, default False
 Copy the data.
dtype : dtype or None, default None
 If None, dtype will be inferred.

Returns

IntervalIndex

See Also

interval_range : Function to create a fixed frequency IntervalIndex.
IntervalIndex.from_arrays : Construct from a left and right array.
IntervalIndex.from_tuples : Construct from a sequence of tuples.

Examples

>>> pd.IntervalIndex.from_breaks([0, 1, 2, 3])
IntervalIndex([(0, 1], (1, 2], (2, 3]],
 dtype='interval[int64, right]')

from_tuples(
 data,
 closed: IntervalClosedType = 'right',
 name: Hashable | None = None,
 copy: bool = False,
 dtype: Dtype | None = None
) -> IntervalIndex from pandas.core.indexes.interval.IntervalIndex
Construct an IntervalIndex from an array-like of tuples.

Parameters

data : array-like (1-dimensional)
 Array of tuples.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
 Whether the intervals are closed on the left-side, right-side, both or neither.
name : str, optional
 Name of the resulting IntervalIndex.
copy : bool, default False
 By-default copy the data, this is compat only and ignored.
dtype : dtype or None, default None
 If None, dtype will be inferred.

Returns

IntervalIndex

See Also

`interval_range` : Function to create a fixed frequency `IntervalIndex`.
`IntervalIndex.from_arrays` : Construct an `IntervalIndex` from a left and right array.
`IntervalIndex.from_breaks` : Construct an `IntervalIndex` from an array of splits.

Examples

>>> pd.IntervalIndex.from_tuples([(0, 1), (1, 2)])
IntervalIndex([(0, 1], (1, 2]],
 dtype='interval[int64, right]')

Static methods defined here:

```
__new__(  
    cls,  
    data,  
    closed: IntervalClosedType | None = None,  
    dtype: Dtype | None = None,  
    copy: bool | None = None,  
    name: Hashable | None = None,  
    verify_integrity: bool = True  
) -> Self from pandas.core.indexes.interval.IntervalIndex  
    Create and return a new object. See help(type) for accurate signature.
```

Readonly properties defined here:

`inferred_type`
 Return a string of the type inferred from the values

`is_overlapping`
 Return True if the `IntervalIndex` has overlapping intervals, else False.

Two intervals overlap if they share a common point, including closed endpoints. Intervals that only have an open endpoint in common do not overlap.

Returns

`bool`
 Boolean indicating if the `IntervalIndex` has overlapping intervals.

See Also

`Interval.overlaps` : Check whether two `Interval` objects overlap.
`IntervalIndex.overlaps` : Check an `IntervalIndex` elementwise for overlaps.

Examples

>>> index = pd.IntervalIndex.from_tuples([(0, 2), (1, 3), (4, 5)])
>>> index
IntervalIndex([(0, 2], (1, 3], (4, 5]],
 dtype='interval[int64, right]')
>>> index.is_overlapping
True

Intervals that share closed endpoints overlap:

```
>>> index = pd.interval_range(0, 3, closed="both")  
>>> index  
IntervalIndex([[0, 1], [1, 2], [2, 3]],  
              dtype='interval[int64, both]')  
>>> index.is_overlapping  
True
```

Intervals that only have an open endpoint in common do not overlap:

```
>>> index = pd.interval_range(0, 3, closed="left")  
>>> index  
IntervalIndex([[0, 1), [1, 2), [2, 3)],
```



```

        dtype='interval[int64, left]')
>>> index.is_overlapping
False

```

length

Calculate the length of each interval in the IntervalIndex.

This method returns a new Index containing the lengths of each interval in the IntervalIndex. The length of an interval is defined as the difference between its end and its start.

Returns

Index

An Index containing the lengths of each interval.

See Also

Interval.length : Return the length of the Interval.

Examples

```

>>> intervals = pd.IntervalIndex.from_arrays(
...     [1, 2, 3], [4, 5, 6], closed="right"
... )
>>> intervals.length
Index([3, 3, 3], dtype='int64')

```

```

>>> intervals = pd.IntervalIndex.from_tuples([(1, 5), (6, 10), (11, 15)])
>>> intervals.length
Index([4, 4, 4], dtype='int64')

```

Data descriptors defined here:

closed

String describing the inclusive side the intervals.

Either ``left``, ``right``, ``both`` or ``neither``.

See Also

IntervalArray.closed : Returns inclusive side of the IntervalArray.

Interval.closed : Returns inclusive side of the Interval.

IntervalIndex.closed : Returns inclusive side of the IntervalIndex.

Examples

For arrays:

```

>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
>>> interv_arr
<IntervalArray>
[[0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.closed
'right'

```

For Interval Index:

```

>>> interv_idx = pd.interval_range(start=0, end=2)
>>> interv_idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> interv_idx.closed
'right'

```

closed_left

Check if the interval is closed on the left side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is closed on the left-side.

See Also

Interval.closed_right : Check if the interval is closed on the right side.
Interval.open_left : Boolean inverse of closed_left.

Examples

>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.closed_left
True

>>> iv = pd.Interval(0, 5, closed='right')
>>> iv.closed_left
False

closed_right

Check if the interval is closed on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is closed on the left-side.

See Also

Interval.closed_left : Check if the interval is closed on the left side.
Interval.open_right : Boolean inverse of closed_right.

Examples

>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.closed_right
True

>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.closed_right
False

is_empty

Indicates if an interval is empty, meaning it contains no points.

An interval is considered empty if its `left` and `right` endpoints are equal, and it is not closed on both sides. This means that the interval does not include any real points. In the case of an :class:`pandas.arrays.IntervalArray` or :class:`IntervalIndex`, the property returns a boolean array indicating the emptiness of each interval.

Returns

bool or ndarray

A boolean indicating if a scalar :class:`Interval` is empty, or a boolean ``ndarray`` positionally indicating if an ``Interval`` in an :class:`~arrays.IntervalArray` or :class:`IntervalIndex` is empty.

See Also

Interval.length : Return the length of the Interval.

Examples

An :class:`Interval` that contains points is not empty:

>>> pd.Interval(0, 1, closed='right').is_empty
False

An ``Interval`` that does not contain any points is empty:

>>> pd.Interval(0, 0, closed='right').is_empty
True
>>> pd.Interval(0, 0, closed='left').is_empty
True
>>> pd.Interval(0, 0, closed='neither').is_empty
True

An ``Interval`` that contains a single point is not empty:

```
>>> pd.Interval(0, 0, closed='both').is_empty
False
```

An :class:`~arrays.IntervalArray` or :class:`IntervalIndex` returns a boolean ``ndarray`` positionally indicating if an ``Interval`` is empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'),
...         pd.Interval(1, 2, closed='neither')]
>>> pd.arrays.IntervalArray(ivs).is_empty
array([ True, False])
```

Missing values are not considered empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'), np.nan]
>>> pd.IntervalIndex(ivs).is_empty
array([ True, False])
```

is_monotonic_decreasing

Return True if the IntervalIndex is monotonic decreasing (only equal or decreasing values), else False

is_non_overlapping_monotonic

Return a boolean whether the IntervalArray/IntervalIndex is non-overlapping and monotonic

Non-overlapping means (no Intervals share points), and monotonic means either monotonic increasing or monotonic decreasing.

See Also

overlaps : Check if two IntervalIndex objects overlap.

Examples

For arrays:

```
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
>>> interv_arr
<IntervalArray>
[[0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.is_non_overlapping_monotonic
True
```

```
>>> interv_arr = pd.arrays.IntervalArray(
...     [pd.Interval(0, 1), pd.Interval(-1, 0.1)]
... )
>>> interv_arr
<IntervalArray>
[[0.0, 1.0], (-1.0, 0.1]]
Length: 2, dtype: interval[float64, right]
>>> interv_arr.is_non_overlapping_monotonic
False
```

For Interval Index:

```
>>> interv_idx = pd.interval_range(start=0, end=2)
>>> interv_idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> interv_idx.is_non_overlapping_monotonic
True

>>> interv_idx = pd.interval_range(start=0, end=2, closed="both")
>>> interv_idx
IntervalIndex([(0, 1], [1, 2]], dtype='interval[int64, both]')
>>> interv_idx.is_non_overlapping_monotonic
False
```

is_unique

Return True if the IntervalIndex contains unique elements, else False.

left

Return left bounds of the intervals in the IntervalIndex.

The left bounds of each interval in the IntervalIndex are

returned as an Index. The datatype of the left bounds is the same as the datatype of the endpoints of the intervals.

Returns

Index

An Index containing the left bounds of the intervals.

See Also

IntervalIndex.right : Return the right bounds of the intervals in the IntervalIndex.

IntervalIndex.mid : Return the mid-point of the intervals in the IntervalIndex.

IntervalIndex.length : Return the length of the intervals in the IntervalIndex.

Examples

```
>>> iv_idx = pd.IntervalIndex.from_arrays([1, 2, 3], [4, 5, 6], closed="right")
```

```
>>> iv_idx.left
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> iv_idx = pd.IntervalIndex.from_tuples(
```

```
...     [(1, 4), (2, 5), (3, 6)], closed="left"
```

```
... )
```

```
>>> iv_idx.left
```

```
Index([1, 2, 3], dtype='int64')
```

mid

Return the midpoint of each interval in the IntervalIndex as an Index.

Each midpoint is calculated as the average of the left and right bounds of each interval. The midpoints are returned as a pandas Index object.

Returns

pandas.Index

An Index containing the midpoints of each interval.

See Also

IntervalIndex.left : Return the left bounds of the intervals in the IntervalIndex.

IntervalIndex.right : Return the right bounds of the intervals in the IntervalIndex.

IntervalIndex.length : Return the length of the intervals in the IntervalIndex.

Notes

The midpoint is the average of the interval bounds, potentially resulting in a floating-point number even if bounds are integers. The returned Index will have a dtype that accurately holds the midpoints. This computation is the same regardless of whether intervals are open or closed.

Examples

```
>>> iv_idx = pd.IntervalIndex.from_arrays([1, 2, 3], [4, 5, 6])
```

```
>>> iv_idx.mid
```

```
Index([2.5, 3.5, 4.5], dtype='float64')
```

```
>>> iv_idx = pd.IntervalIndex.from_tuples([(1, 4), (2, 5), (3, 6)])
```

```
>>> iv_idx.mid
```

```
Index([2.5, 3.5, 4.5], dtype='float64')
```

open_left

Check if the interval is open on the left side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool

True if the Interval is not closed on the left-side.

See Also

Interval.open_right : Check if the interval is open on the right side.
Interval.closed_left : Boolean inverse of open_left.

Examples

>>> iv = pd.Interval(0, 5, closed='neither')
>>> iv.open_left
True

>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.open_left
False

open_right

Check if the interval is open on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns

bool
True if the Interval is not closed on the left-side.

See Also

Interval.open_left : Check if the interval is open on the left side.
Interval.closed_right : Boolean inverse of open_right.

Examples

>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.open_right
True

>>> iv = pd.Interval(0, 5)
>>> iv.open_right
False

right

Return right bounds of the intervals in the IntervalIndex.

The right bounds of each interval in the IntervalIndex are returned as an Index. The datatype of the right bounds is the same as the datatype of the endpoints of the intervals.

Returns

Index
An Index containing the right bounds of the intervals.

See Also

IntervalIndex.left : Return the left bounds of the intervals in the IntervalIndex.
IntervalIndex.mid : Return the mid-point of the intervals in the IntervalIndex.
IntervalIndex.length : Return the length of the intervals in the IntervalIndex.

Examples

>>> iv_idx = pd.IntervalIndex.from_arrays([1, 2, 3], [4, 5, 6], closed="right")
>>> iv_idx.right
Index([4, 5, 6], dtype='int64')

>>> iv_idx = pd.IntervalIndex.from_tuples(
... [(1, 4), (2, 5), (3, 6)], closed="left"
...)
>>> iv_idx.right
Index([4, 5, 6], dtype='int64')

Data and other attributes defined here:

__annotations__ = {'_data': 'IntervalArray', '_values': 'IntervalArray...

Methods inherited from Index:

`__abs__(self)` -> Index from `pandas.core.indexes.base.Index`

`__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs)` from `pandas.core.indexes.base.Index`

`__array_wrap__(self, result, context=None, return_scalar=False)` from `pandas.core.indexes.base.Index`
Gets called after a ufunc and other functions e.g. `np.split`.

`__bool__(self)` -> NoReturn from `pandas.core.indexes.base.Index`

`__copy__(self)` -> Self from `pandas.core.indexes.base.Index`

`__deepcopy__(self, memo=None)` -> Self from `pandas.core.indexes.base.Index`

Parameters

`memo`, default None

Standard signature. Unused

`__getitem__(self, key)` from `pandas.core.indexes.base.Index`

Override `numpy.ndarray`'s `__getitem__` method to work as desired.

This function adds lists and Series as valid boolean indexers
(`ndarrays` only supports `ndarray` with `dtype=bool`).

If resulting `ndim != 1`, plain `ndarray` is returned instead of
corresponding ``Index`` subclass.

`__iadd__(self, other)` from `pandas.core.indexes.base.Index`

`__invert__(self)` -> Index from `pandas.core.indexes.base.Index`

`__len__(self)` -> int from `pandas.core.indexes.base.Index`
Return the length of the Index.

`__neg__(self)` -> Index from `pandas.core.indexes.base.Index`

`__pos__(self)` -> Index from `pandas.core.indexes.base.Index`

`__repr__(self)` -> str_t from `pandas.core.indexes.base.Index`
Return a string representation for this object.

`__setitem__(self, key, value)` -> None from `pandas.core.indexes.base.Index`

`all(self, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return whether all elements are Truthy.

Parameters

`*args`

Required for compatibility with `numpy`.

`**kwargs`

Required for compatibility with `numpy`.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

`Index.any` : Return whether any element in an Index is True.

`Series.any` : Return whether any element in a Series is True.

`Series.all` : Return whether all elements in a Series are True.

Notes

Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples

True, because nonzero integers are considered True.

```
>>> pd.Index([1, 2, 3]).all()
```

```
True
```

False, because ``0`` is considered False.

```
>>> pd.Index([0, 1, 2]).all()
False
```

`any(self, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return whether any element is Truthy.

Parameters

*args

Required for compatibility with numpy.

**kwargs

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

`Index.all` : Return whether all elements are True.

`Series.all` : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
```

```
>>> index.any()
```

```
True
```

```
>>> index = pd.Index([0, 0, 0])
```

```
>>> index.any()
```

```
False
```

`append(self, other: Index | Sequence[Index])` -> Index from `pandas.core.indexes.base.Index`
Append a collection of Index options together.

Parameters

`other` : Index or list/tuple of indices

Single Index or a collection of indices, which can be either a list or a tuple.

Returns

Index

Returns a new Index object resulting from appending the provided other indices to the original Index.

See Also

`Index.insert` : Make new Index inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx.append(pd.Index([4]))
```

```
Index([1, 2, 3, 4], dtype='int64')
```

`argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` -> int from `pandas.core.indexes.base.Index`
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations, the first row position is returned.

Parameters

`axis` : None

Unused. Parameter needed for compatibility with DataFrame.

`skipna` : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the maximum value.

See Also

Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmax : Return index label of the maximum values.
Series.idxmin : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base

Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations, the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the minimum value.

See Also

Series.argmin : Return position of the minimum value.
Series.argmax : Return position of the maximum value.
numpy.ndarray.argmin : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```


The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

`argsort(self, *args, **kwargs)` -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Return the integer indices that would sort the index.

Parameters

`*args`

Passed to ``numpy.ndarray.argsort``.

`**kwargs`

Passed to ``numpy.ndarray.argsort``.

Returns

`np.ndarray[np.intp]`

Integer indices that would sort the index if used as an indexer.

See Also

`numpy.argsort` : Similar method for NumPy arrays.

`Index.sort_values` : Return sorted copy of Index.

Examples

```
>>> idx = pd.Index(["b", "a", "d", "c"])
```

```
>>> idx
```

```
Index(['b', 'a', 'd', 'c'], dtype='str')
```

```
>>> order = idx.argsort()
```

```
>>> order
```

```
array([1, 0, 3, 2])
```

```
>>> idx[order]
```

```
Index(['a', 'b', 'c', 'd'], dtype='str')
```

`asof(self, label)` from `pandas.core.indexes.base.Index`

Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

`label` : object

The label up to which the method returns the latest index label.

Returns

object

The passed label if it is in the index. The previous label if the passed label is not in the sorted index or ``NaN`` if there is no such label.

See Also

`Series.asof` : Return the latest value in a Series up to the passed index.

`merge_asof` : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).

`Index.get_loc` : An ``asof`` is a thin wrapper around ``get_loc`` with `method='pad'`.

Examples

``Index.asof`` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
```

```
>>> idx.asof("2014-01-01")
```

```
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
```

```
'2014-01-02'
```

If all of the labels in the index are later than the passed label, NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp]` from `pandas`
Return the locations (indices) of labels in the index.

As in the :meth:`pandas.Index.asof`, if the label (a particular entry in ``where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in ``where``, -1 is returned.

``mask`` is used to ignore ``NA`` values in the index during calculation.

Parameters

where : Index
 An Index consisting of an array of timestamps.
mask : np.ndarray[bool]
 Array of booleans denoting where values in the original data are not ``NA``.

Returns

np.ndarray[np.intp]
 An array of locations (indices) of the labels from the index which correspond to the return values of :meth:`pandas.Index.asof` for every element in ``where``.

See Also

Index.asof : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use ``mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by dtype. When conversion is impossible, a `TypeError` exception is raised.

Parameters

dtype : numpy dtype or pandas type
 Note that any signed integer `dtype` is treated as ``'int64'``, and any unsigned integer `dtype` is treated as ``'uint64'``, regardless of the size.
copy : bool, default True
 By default, `astype` always returns a newly allocated object.

If copy is set to False and internal requirements on dtype are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index

Index with values cast to specified dtype.

See Also

Index.dtype: Return the dtype object of the underlying data.

Index.dtypes: Return the dtype object of the underlying data.

Index.convert_dtypes: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.astype("float")
```

```
Index([1.0, 2.0, 3.0], dtype='float64')
```

copy(self, name: Hashable | None = None, deep: bool = False) -> Self from pandas.core.indexes

Make a copy of this object.

Name is set on the new object.

Parameters

name : Label, optional

Set name for new object.

deep : bool, default False

If True attempts to make a deep copy of the Index.

Else makes a shallow copy.

Returns

Index

Index refer to new object which is a copy of this object.

See Also

Index.delete: Make new Index with passed location(-s) deleted.

Index.drop: Make new Index with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> new_idx = idx.copy()
```

```
>>> idx is new_idx
```

```
False
```

delete(self, loc: int | np.integer | list[int] | npt.NDArray[np.integer]) -> Self from pandas

Make new Index with passed location(-s) deleted.

Parameters

loc : int or list of int

Location of item(-s) which will be deleted.

Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

numpy.delete : Delete any rows and column from NumPy array (ndarray).

Examples

```

-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete(1)
Index(['a', 'c'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')

diff(self, periods: int = 1) -> Index from pandas.core.indexes.base.Index
Computes the difference between consecutive values in the Index object.

If periods is greater than 1, computes the difference between values that
are `periods` number of positions apart.

Parameters
-----
periods : int, optional
    The number of positions between the current and previous
    value to compute the difference with. Default is 1.

Returns
-----
Index
    A new Index object with the computed differences.

Examples
-----
>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')

difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index
Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters
-----
other : Index or array-like
    Index object or an array-like object containing elements to be compared
    with the elements of the original Index.
sort : bool or None, default None
    Whether to sort the resulting index. By default, the
    values are attempted to be sorted, but any TypeError from
    incomparable elements is caught by pandas.

    * None : Attempt to sort the result, but catch any TypeErrors
      from comparing incomparable elements.
    * False : Do not sort the result.
    * True : Sort the result (which may raise TypeError).

Returns
-----
Index
    Returns a new Index object containing elements that are in the original
    Index but not in the `other` Index.

See Also
-----
Index.symmetric_difference : Compute the symmetric difference of two Index
    objects.
Index.intersection : Form the intersection of two Index objects.

Examples
-----
>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')

drop(
    self,
    labels: Index | np.ndarray | Iterable[Hashable],

```

```

errors: IgnoreRaise = 'raise'
) -> Index from pandas.core.indexes.base.Index
Make new Index with passed list of labels deleted.

Parameters
-----
labels : array-like or scalar
    Array-like object or a scalar value, representing the labels to be removed
    from the Index.
errors : {'ignore', 'raise'}, default 'raise'
    If 'ignore', suppress error and existing labels are dropped.

Returns
-----
Index
    Will be same type as self, except for RangeIndex.

Raises
-----
KeyError
    If not all of the labels are found in the selected axis

See Also
-----
Index.dropna : Return Index without NA/NaN values.
Index.drop_duplicates : Return Index with duplicate values removed.

Examples
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index
Return Index with duplicate values removed.

Parameters
-----
keep : {'first', 'last', ``False``}, default 'first'
    - 'first' : Drop duplicates except for the first occurrence.
    - 'last' : Drop duplicates except for the last occurrence.
    - ``False`` : Drop all duplicates.

Returns
-----
Index
    A new Index object with the duplicate values removed.

See Also
-----
Series.drop_duplicates : Equivalent method on Series.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Index.duplicated : Related method on Index, indicating duplicate
    Index values.

Examples
-----
Generate a pandas.Index with duplicate values.

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])

The `keep` parameter controls which duplicate values are removed.
The value 'first' keeps the first occurrence for each
set of duplicated entries. The default value of keep is 'first'.

>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')

The value 'last' keeps the last occurrence for each set of duplicated
entries.

>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')

The value ``False`` discards all sets of duplicated entries.

>>> idx.drop_duplicates(keep=False)

```

```

Index(['cow', 'beetle', 'hippo'], dtype='str')

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be
of Index type, not MultiIndex. The original index is not modified inplace.

Parameters
-----
level : int, str, or list-like, default 0
    If a string is given, must be the name of a level
    If list-like, elements must be names or indexes of levels.

Returns
-----
Index or MultiIndex
    Returns an Index or MultiIndex object, depending on the resulting index
    after removing the requested level(s).

See Also
-----
Index.dropna : Return Index without NA/Nan values.

Examples
-----
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
             (2, 4, 6)],
            names=['x', 'y', 'z'])

>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
            names=['y', 'z'])

>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/Nan values.

Parameters
-----
how : {'any', 'all'}, default 'any'
    If the Index is a MultiIndex, drop the value when any or all levels
    are NaN.

Returns
-----
Index
    Returns an Index object after removing NA/Nan values.

See Also
-----
Index.fillna : Fill NA/Nan values with the specified value.
Index.isna : Detect missing values.

Examples
-----
>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')

```

`deduplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes`
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

`keep` : {'first', 'last', False}, default 'first'
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

`np.ndarray[bool]`
A numpy array of boolean values indicating duplicate index values.

See Also

`Series.duplicated` : Equivalent method on `pandas.Series`.
`DataFrame.duplicated` : Equivalent method on `pandas.DataFrame`.
`Index.drop_duplicates` : Remove duplicate values from `Index`.

Examples

By default, for each set of duplicated values, the first occurrence is set to False and all others to True:

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])
```

which is equivalent to

```
>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])
```

By setting `keep` on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])
```

`equals(self, other: Any) -> bool from pandas.core.indexes.base.Index`
Determine if two Index object are equal.

The things that are being compared are:

- * The elements inside the Index object.
- * The order of the elements inside the Index object.

Parameters

`other` : Any
The other object to compare against.

Returns

`bool`
True if "other" is an Index and it has the same elements and order as the calling index; False otherwise.

See Also

`Index.identical`: Checks that object attributes and types are also equal.
`Index.has_duplicates`: Check if the Index has duplicate values.

Index.is_unique: Return if the index has unique values.

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx1
Index([1, 2, 3], dtype='int64')
>>> idx1.equals(pd.Index([1, 2, 3]))
True
```

The elements inside are compared

```
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2
Index(['1', '2', '3'], dtype='str')
```

```
>>> idx1.equals(idx2)
False
```

The order is compared

```
>>> ascending_idx = pd.Index([1, 2, 3])
>>> ascending_idx
Index([1, 2, 3], dtype='int64')
>>> descending_idx = pd.Index([3, 2, 1])
>>> descending_idx
Index([3, 2, 1], dtype='int64')
>>> ascending_idx.equals(descending_idx)
False
```

The dtype is *not* compared

```
>>> int64_idx = pd.Index([1, 2, 3], dtype="int64")
>>> int64_idx
Index([1, 2, 3], dtype='int64')
>>> uint64_idx = pd.Index([1, 2, 3], dtype="uint64")
>>> uint64_idx
Index([1, 2, 3], dtype='uint64')
>>> int64_idx.equals(uint64_idx)
True
```

fillna(self, value) from pandas.core.indexes.base.Index
Fill NA/NaN values with the specified value.

Parameters

```
-----
value : scalar
    Scalar value to use to fill holes (e.g. 0).
    This value cannot be a list-like.
```

Returns

```
-----
Index
    NA/NaN values replaced with `value`.
```

See Also

```
-----
DataFrame.fillna : Fill NaN values of a DataFrame.
Series.fillna : Fill NaN Values of a Series.
```

Examples

```
-----
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

```
get_indexer(
    self,
    target,
    method: ReindexMethod | None = None,
    limit: int | None = None,
    tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
    Compute indexer and mask for new index given the current index.
```

The indexer should be then used as an input to ndarray.take to align the current data to the new index.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.
limit : int, optional
Maximum number of consecutive labels in ``target`` to match for inexact matches.
tolerance : optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

See Also

Index.get_indexer_for : Returns an indexer even when non-unique.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Notes

Returns -1 for unmatched values, for further explanation see the example below.

Examples

>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([1, 2, -1])

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

get_indexer_for(self, target) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Guaranteed return of an indexer even when non-unique.

This dispatches to get_indexer or get_indexer_non_unique as appropriate.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.

Returns

np.ndarray[np.intp]
List of indices.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Examples

```

>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])

get_level_values = _get_level_values(self, level) -> Index

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
    Calculate slice bound that corresponds to given label.

    Returns leftmost (one-past-the-rightmost if ``side=='right'``) position
    of given label.

    Parameters
    -----
    label : object
        The label for which to calculate the slice bound.
    side : {'left', 'right'}
        if 'left' return leftmost position of given label.
        if 'right' return one-past-the-rightmost position of given label.

    Returns
    -----
    int
        Index of label.

    See Also
    -----
    Index.get_loc : Get integer location, slice or boolean mask for requested
        label.

    Examples
    -----
    >>> idx = pd.RangeIndex(5)
    >>> idx.get_slice_bound(3, "left")
    3

    >>> idx.get_slice_bound(3, "right")
    4

    If ``label`` is non-unique in the index, an error will be raised.

    >>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
    >>> idx_duplicate.get_slice_bound("a", "left")
    Traceback (most recent call last):
    KeyError: Cannot get left slice bound for non-unique label: 'a'

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
    Group the index labels by a given array of values.

    Parameters
    -----
    values : array
        Values used to determine the groups.

    Returns
    -----
    dict
        {group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.

    Parameters
    -----
    other : Index
        The Index object you want to compare with the current Index object.

    Returns
    -----
    bool
        If two Index objects have equal elements and same type True,
        otherwise False.

    See Also
    -----
    Index.equals: Determine if two Index object are equal.
    Index.has_duplicates: Check if the Index has duplicate values.

```

`Index.is_unique`: Return if the index has unique values.

Examples

```
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False
```

`infer_objects(self, copy: bool = True)` -> Index from `pandas.core.indexes.base.Index`
If we have an object dtype, try to infer a non-object dtype.

Parameters

`copy` : bool, default True
Whether to make a copy in cases where no inference occurs.

Returns

Index
An Index with a new dtype if the dtype was inferred
or a shallow copy if the dtype could not be inferred.

See Also

`Index.inferred_type`: Return a string of the type inferred from the values.

Examples

```
-----
>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')
```

`insert(self, loc: int, item)` -> Index from `pandas.core.indexes.base.Index`
Make new Index inserting new item at location.

Follows Python numpy.insert semantics for negative values.

Parameters

`loc` : int
The integer location where the new item will be inserted.
`item` : object
The new item to be inserted into the Index.

Returns

Index
Returns a new Index object resulting from inserting the specified item at
the specified location within the original Index.

See Also

`Index.append` : Append a collection of Indexes together.

Examples

```
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

`intersection(self, other, sort: bool = False)` from `pandas.core.indexes.base.Index`
Form the intersection of two Index objects.

This returns a new Index with elements common to the index and ``other``.

Parameters

`other` : Index or array-like
An Index or an array-like object containing elements to form the
intersection with the original Index.

sort : True, False or None, default False
Whether to sort the resulting index.

- * None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.
- * False : do not sort the result.
- * True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
```

```
>>> idx2 = pd.Index([3, 4, 5, 6])
```

```
>>> idx1.intersection(idx2)
```

```
Index([3, 4], dtype='int64')
```

is_(self, other) -> bool from pandas.core.indexes.base.Index

More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks that metadata is also the same.

Parameters

other : object

Other object to compare against.

Returns

bool

True if both have same underlying data, False otherwise.

See Also

Index.identical : Works like ``Index.is\_`` but also checks metadata.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
```

```
>>> idx1.is_(idx1.view())
```

```
True
```

```
>>> idx1.is_(idx1.copy())
```

```
False
```

isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_] from pandas.core.indexes

Return a boolean array where the index values are in `values`.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like

Sought values.

level : str or int, optional

Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

np.ndarray[bool]

NumPy array of boolean values.

See Also

```
-----
Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.
```

Notes

```
-----
In the case of `MultiIndex` you must either specify `values` as a
list-like object containing tuples that are the same length as the
number of levels, or specify `level`. Otherwise it will raise a
`ValueError`.
```

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]
... )
>>> midx
MultiIndex([(1, 'red'),
            (2, 'blue'),
            (3, 'green')],
            names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])
array([ True, False, False])
```

```
isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect missing values.
```

Return a boolean same-sized object indicating if the values are NA. NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get mapped to ``True`` values. Everything else get mapped to ``False`` values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

```
-----
numpy.ndarray[bool]
A boolean array of whether my values are NA.
```

See Also

```
-----
Index.notna : Boolean inverse of isna.
Index.dropna : Omit entries with missing values.
isna : Top-level isna.
Series.isna : Detect missing values in Series object.
```

Examples

```
-----
Show which entries in a pandas.Index are NA. The result is an
array.
```

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
```

```
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

```
isnull = isna(self) -> npt.NDArray[np.bool_]
```

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index

The other index on which join is performed.

how : {'left', 'right', 'inner', 'outer'}

level : int or level name, default None

It is either the integer position or the name of the level.

return_indexers : bool, default False

Whether to return the indexers or not for both the index objects.

sort : bool, default False

Sort the join keys lexicographically in the result Index. If False, the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)
The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index or on a key.

DataFrame.merge : Merge DataFrame or named Series objects with a database-style join.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

```
map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base.Index
Map values using an input mapping or function.
```

Parameters

mapper : function, dict, or Series
Mapping correspondence.

na_action : {None, 'ignore'}
If 'ignore', propagate NA values, without passing them to the mapping correspondence.

Returns

Union[Index, MultiIndex]
The output of the mapping function applied to the index.
If the function returns a tuple with more than one element a MultiIndex will be returned.

See Also

Index.where : Replace values where the condition is False.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')

Using `map` with a function:

>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')

max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base
Return the maximum value of the Index.

Parameters

axis : int, optional
For compatibility with NumPy. Only 0 or None are allowed.
skipna : bool, default True
Exclude NA/null values when showing the result.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Maximum value.

See Also

Index.min : Return the minimum value in an Index.
Series.max : Return the maximum value in a Series.
DataFrame.max : Return the maximum values in a DataFrame.

Examples

>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'

For a MultiIndex, the maximum is determined lexicographically.

>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)

min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base
Return the minimum value of the Index.

Parameters

axis : {None}
Dummy argument for consistency with Series.

skipna : bool, default True
Exclude NA/null values when showing the result.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Minimum value.

See Also

Index.max : Return the maximum value of the object.
Series.min : Return the minimum value in a Series.
DataFrame.min : Return the minimum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1
```

```
>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'
```

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA.
Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.
NA values, such as None or :attr:`numpy.NaN`, get mapped to ``False`` values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

notnull = notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters

mask : array-like of bool
Array of booleans denoting where values should be replaced.
value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index

A new Index of the values set with the mask.

See Also

numpy.putmask : Changes elements of an array
based on conditional and input values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')
```

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not
implemented currently.

Returns

Index

A view on self.

See Also

numpy.ndarray.ravel : Return a flattened array.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')
```

reindex(
self,
target,
method: ReindexMethod | None = None,
level=None,
limit: int | None = None,
tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.

Parameters

target : an iterable
An iterable containing the values to be used for creating the new index.
method : {None, 'pad', 'ffill', 'backfill', 'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied
distances are broken by preferring the larger index value.
level : int, optional
Level of multiindex.
limit : int, optional
Maximum number of consecutive labels in ``target`` to match for
inexact matches.
tolerance : int, float, or list-like, optional
Maximum distance between original and new labels for inexact
matches. The values of the index at the matching locations must

satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

`new_index` : `pd.Index`
Resulting index.
`indexer` : `np.ndarray[np.intp]` or `None`
Indices of output values in original index.

Raises

`TypeError`
If `method` passed along with `level`.
`ValueError`
If non-unique multi-index
`ValueError`
If non-unique index and `method` or `limit` passed.

See Also

`Series.reindex` : Conform Series to new index with optional filling logic.
`DataFrame.reindex` : Conform DataFrame to new index with optional filling logic.

Examples

`>>> idx = pd.Index(["car", "bike", "train", "tractor"])`
`>>> idx`
`Index(['car', 'bike', 'train', 'tractor'], dtype='str')`
`>>> idx.reindex(["car", "bike"])`
`(Index(['car', 'bike'], dtype='str'), array([0, 1]))`

`rename(self, name, *, inplace: bool = False) -> Self | None` from `pandas.core.indexes.base.Index`
Alter Index or MultiIndex name.

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters

`name` : Hashable or a sequence of the previous
Name(s) to set.
`inplace` : bool, default `False`
Modifies the object directly, instead of creating a new Index or MultiIndex.

Returns

`Index` or `None`
The same type as the caller or `None` if `inplace=True`.

See Also

`Index.set_names` : Able to set new names partially and by level.

Examples

`>>> idx = pd.Index(["A", "C", "A", "B"], name="score")`
`>>> idx.rename("grade")`
`Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')`

`>>> idx = pd.MultiIndex.from_product(`
`... [ ["python", "cobra"], [2018, 2019] ], names=["kind", "year"]`
`... )`
`>>> idx`
`MultiIndex([('python', 2018),`
 `('python', 2019),`
 `('cobra', 2018),`
 `('cobra', 2019)],`
 `names=['kind', 'year'])`
`>>> idx.rename(["species", "year"])`
`MultiIndex([('python', 2018),`

```

        ('python', 2019),
        ('cobra', 2018),
        ('cobra', 2019)],
        names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.

```

`repeat(self, repeats, axis: None = None) -> Self` from `pandas.core.indexes.base.Index`
Repeat elements of an Index.

Returns a new Index where each element of the current Index is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty Index.

`axis` : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

Index
Newly created Index with repeated elements.

See Also

`Series.repeat` : Equivalent function for Series.
`numpy.repeat` : Similar method for :class:`numpy.ndarray`.

Examples

```

>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')

```

`round(self, decimals: int = 0) -> Self` from `pandas.core.indexes.base.Index`
Round each value in the Index to the given number of decimals.

Parameters

`decimals` : int, optional
Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

Returns

Index
A new Index with the rounded values.

Examples

```

>>> import pandas as pd
>>> idx = pd.Index([10.1234, 20.5678, 30.9123, 40.4567, 50.7890])
>>> idx.round(decimals=2)
Index([10.12, 20.57, 30.91, 40.46, 50.79], dtype='float64')

```

`set_names(self, names, *, level=None, inplace: bool = False) -> Self | None` from `pandas.core`
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

`names` : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

`level` : int, Hashable or a sequence of the previous, optional
If the index is a MultiIndex and names is not dict-like, level(s) to set

(None for all levels). Otherwise level must be None.

inplace : bool, default False
Modifies the object directly, instead of creating a new Index or MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
```

```
>>> idx
```

```
Index([1, 2, 3, 4], dtype='int64')
```

```
>>> idx.set_names("quarter")
```

```
Index([1, 2, 3, 4], dtype='int64', name='quarter')
```

```
>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
```

```
>>> idx
```

```
MultiIndex([( 'python', 2018),  
            ( 'python', 2019),  
            ( 'cobra', 2018),  
            ( 'cobra', 2019)],  
           )
```

```
>>> idx = idx.set_names(["kind", "year"])
```

```
>>> idx.set_names("species", level=0)
```

```
MultiIndex([( 'python', 2018),  
            ( 'python', 2019),  
            ( 'cobra', 2018),  
            ( 'cobra', 2019)],  
           names=['species', 'year'])
```

When renaming levels with a dict, levels can not be passed.

```
>>> idx.set_names({"kind": "snake"})
```

```
MultiIndex([( 'python', 2018),  
            ( 'python', 2019),  
            ( 'cobra', 2018),  
            ( 'cobra', 2019)],  
           names=['snake', 'year'])
```

shift(self, periods: int = 1, freq=None) -> Self from pandas.core.indexes.base.Index
Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes by a specified time increment a given number of times.

Parameters

periods : int, default 1

Number of periods (or increments) to shift by,
can be positive or negative.

freq : pandas.DateOffset, pandas.Timedelta or str, optional

Frequency increment to shift by.

If None, the index is shifted by its own `freq` attribute.

Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

pandas.Index

Shifted index.

See Also

Series.shift : Shift values of Series.

Notes

This method is only implemented for datetime-like index classes,
i.e., DatetimeIndex, PeriodIndex and TimedeltaIndex.

Examples

Put the first 5 month starts of 2011 into an index.

```
>>> month_starts = pd.date_range("1/1/2011", periods=5, freq="MS")
>>> month_starts
DatetimeIndex(['2011-01-01', '2011-02-01', '2011-03-01', '2011-04-01',
               '2011-05-01'],
              dtype='datetime64[us]', freq='MS')
```

Shift the index by 10 days.

```
>>> month_starts.shift(10, freq="D")
DatetimeIndex(['2011-01-11', '2011-02-11', '2011-03-11', '2011-04-11',
               '2011-05-11'],
              dtype='datetime64[us]', freq=None)
```

The default value of `freq` is the `freq` attribute of the index, which is 'MS' (month start) in this example.

```
>>> month_starts.shift(10)
DatetimeIndex(['2011-11-01', '2011-12-01', '2012-01-01', '2012-02-01',
               '2012-03-01'],
              dtype='datetime64[us]', freq='MS')
```

```
slice_indexer(
    self,
    start: Hashable | None = None,
    end: Hashable | None = None,
    step: int | None = None
) -> slice from pandas.core.indexes.base.Index
    Compute the slice indexer for input labels and step.
```

Index needs to be ordered and unique.

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, default None
 If None, defaults to 1.

Returns

slice
 A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)
```

```
slice_locs(
    self,
```

```

    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.

Parameters
-----
start : label, default None
    If None, defaults to the beginning.
end : label, default None
    If None, defaults to the end.
step : int, defaults None
    If None, defaults to 1.

Returns
-----
tuple[int, int]
    Returns a tuple of two integers representing the slice locations for the
    input labels within the index.

See Also
-----
Index.get_loc : Get location for a single label.

Notes
-----
This method only works if the index is monotonic or unique.

Examples
-----
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)

>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)

```

sort_values(

```

    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.

Return a sorted copy of the index, and optionally return the indices
that sorted the index itself.

Parameters
-----
return_indexer : bool, default False
    Should the indices that would sort the index be returned.
ascending : bool, default True
    Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.
key : callable, optional
    If not None, apply the key function to the index values
    before sorting. This is similar to the `key` argument in the
    builtin :meth:`sorted` function, with the notable difference that
    this `key` function should be *vectorized*. It should expect an
    ``Index`` and return an ``Index`` of the same shape.

Returns
-----
sorted_index : pandas.Index
    Sorted copy of the index.
indexer : numpy.ndarray, optional
    The indices that the index itself was sorted by.

See Also
-----

```

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

```
-----  
>>> idx = pd.Index([10, 100, 1, 1000])  
>>> idx  
Index([10, 100, 1, 1000], dtype='int64')
```

Sort values in ascending order (default behavior).

```
>>> idx.sort_values()  
Index([1, 10, 100, 1000], dtype='int64')
```

Sort values in descending order, and also get the indices `idx` was sorted by.

```
>>> idx.sort_values(ascending=False, return_indexer=True)  
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))
```

```
sortlevel(  
    self,  
    level=None,  
    ascending: bool | list[bool] = True,  
    sort_remaining=None,  
    na_position: NaPosition = 'first'  
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index  
For internal compatibility with the Index API.
```

Sort the Index. This is for compat with MultiIndex

Parameters

```
-----  
ascending : bool, default True  
    False to sort in descending order  
na_position : {'first' or 'last'}, default 'first'  
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at  
    the end.
```

.. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns

Index

```
symmetric_difference(  
    self,  
    other,  
    result_name: abc.Hashable | None = None,  
    sort: bool | None = None  
) from pandas.core.indexes.base.Index  
Compute the symmetric difference of two Index objects.
```

Parameters

```
-----  
other : Index or array-like  
    Index or an array-like object with elements to compute the symmetric  
    difference with the original Index.  
result_name : str  
    A string representing the name of the resulting Index, if desired.  
sort : bool or None, default None  
    Whether to sort the resulting index. By default, the  
    values are attempted to be sorted, but any TypeError from  
    incomparable elements is caught by pandas.
```

* None : Attempt to sort the result, but catch any TypeErrors
from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object containing elements that appear in either the original Index or the `other` Index, but not both.

See Also

Index.difference : Return a new Index with elements of index not in other.
Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes

``symmetric\_difference`` contains elements that appear in either
``idx1`` or ``idx2`` but not both. Equivalent to the Index created by
``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates
dropped.

Examples

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')

```
take(  
    self,  
    indices,  
    axis: Axis = 0,  
    allow_fill: bool = True,  
    fill_value=None,  
    **kwargs  
) -> Self from pandas.core.indexes.base.Index  
Return a new Index of the values selected by the indices.
```

For internal compatibility with numpy arrays.

Parameters

indices : array-like
Indices to be taken.
axis : {0 or 'index'}, optional
The axis over which to select values, always 0 or 'index'.
allow_fill : bool, default True
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices
from the right (the default). This is similar to
:func:`numpy.take`.

* True: negative values in `indices` indicate
missing values. These values are set to `fill\_value`. Any
other negative values raise a ``ValueError``.

fill_value : scalar, default None
If allow_fill=True and fill_value is not None, indices specified by
-1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
**kwargs
Required for compatibility with numpy.

Returns

Index

An index formed of elements at the given indices. Will be the same
type as self, except for RangeIndex.

See Also

numpy.ndarray.take: Return an array formed from the
elements of a at the given indices.

Examples

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.take([2, 2, 1, 2])
Index(['c', 'c', 'b', 'c'], dtype='str')

```
to_flat_index(self) -> Self from pandas.core.indexes.base.Index  
Identity method.
```

This is implemented for compatibility with subclass implementations

when chaining.

Returns

pd.Index
 Caller.

See Also

MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.core.indexes
Create a DataFrame with a column containing the Index.

Parameters

index : bool, default True
 Set the index of the returned DataFrame as the original Index.

name : object, defaults to index.name
 The passed name should substitute for the index name (if it has one).

Returns

DataFrame
 DataFrame containing the original Index data.

See Also

Index.to_series : Convert an Index to a Series.
Series.to_frame : Convert Series to DataFrame.

Examples

>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
 animal
animal
Ant Ant
Bear Bear
Cow Cow

By default, the original Index is reused. To enforce a new Index:

>>> idx.to_frame(index=False)
 animal
0 Ant
1 Bear
2 Cow

To override the name of the resulting column, specify `name`:

>>> idx.to_frame(index=False, name="zoo")
 zoo
0 Ant
1 Bear
2 Cow

to_series(self, index=None, name: Hashable | None = None) -> Series from pandas.core.indexes
Create a Series with both index and values equal to the index keys.

Useful with map for returning an indexer based on an index.

Parameters

index : Index, optional
 Index of resulting Series. If None, defaults to original index.
name : str, optional
 Name of resulting Series. If None, defaults to name of original index.

Returns

Series
 The dtype will be based on the type of the Index values.

See Also

`Index.to_frame` : Convert an Index to a DataFrame.

`Series.to_frame` : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to `index`:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1      Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify `name`:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.

`sort` : bool or None, default None
Whether to sort the resulting Index.

* None : Sort the result, except when

1. `self` and `other` are equal.
2. `self` or `other` has length 0.
3. Some values in `self` or `other` cannot be compared.
A `RuntimeWarning` is issued in this case.

* False : do not sort the result.

* True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the `other` Index.

See Also

`Index.unique` : Return unique values in the index.

`Index.intersection` : Form the intersection of two Index objects.

`Index.difference` : Return a new Index with elements of index not in `other`.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
```

```
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')
```

Union mismatched dtypes

```
>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')
```

MultiIndex case

```
>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )
```

`unique(self, level: Hashable | None = None) -> Self` from `pandas.core.indexes.base.Index`
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

level : int or hashable, optional
Only return values from specified level (for MultiIndex).
If int, gets the level by integer position, else by level name.

Returns

Index
Unique values in the index.

See Also

unique : Numpy array of unique values in that column.
Series.unique : Return unique values of Series object.

Examples

```
>>> idx = pd.Index([1, 1, 2, 3, 3])
>>> idx.unique()
Index([1, 2, 3], dtype='int64')
```

`view(self, cls=None)` from `pandas.core.indexes.base.Index`

Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the ``cls`` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

`cls` : data-type or ndarray sub-class, optional
Data-type descriptor of the returned view, e.g., float32 or int16. Omitting it results in the view having the same data-type as ``self``. This argument can also be specified as an ndarray sub-class, e.g., `np.int64` or `np.float32` which then specifies the type of the returned object.

Returns

Index or ndarray

A view of the Index. If ``cls`` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric ``cls`` is specified an ndarray view with the new dtype is returned.

Raises

ValueError

If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

`Index.copy` : Returns a copy of the Index.

`numpy.ndarray.view` : Returns a new view of array with the same data.

Examples

```
>>> idx = pd.Index([-1, 0, 1])
>>> idx.view()
Index([-1, 0, 1], dtype='int64')
```

```
>>> idx.view(np.uint64)
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
array([ nan,   nan,  0.e+00,  0.e+00,  1.e-45,  0.e+00], dtype=float32)
```

`where(self, cond, other=None)` -> Index from `pandas.core.indexes.base.Index`
Replace values where the condition is False.

The replacement is taken from other.

Parameters

`cond` : bool array-like with the same length as self
Condition to select the values on.
`other` : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index

A copy of self with values replaced from other where the condition is False.

See Also

`Series.where` : Same method for Series.

`DataFrame.where` : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

has_duplicates

Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

`Index.is_unique` : Inverse method that checks if it has unique values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False
```

is_monotonic_increasing

Return a boolean if the values are equal or increasing.

Returns

bool

See Also

`Index.is_monotonic_decreasing` : Check if the values are equal or decreasing.

Examples

```
>>> pd.Index([1, 2, 3]).is_monotonic_increasing
True
>>> pd.Index([1, 2, 2]).is_monotonic_increasing
True
>>> pd.Index([1, 3, 2]).is_monotonic_increasing
False
```

nlevels

Number of levels.

shape

Return a tuple of the shape of the underlying data.

See Also

`Index.size`: Return the number of elements in the underlying data.

`Index.ndim`: Number of dimensions of the underlying data, by definition 1.

`Index.dtype`: Return the dtype object of the underlying data.

`Index.values`: Return an array representing the data in the Index.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)
```

values

Return an array representing the data in the Index.

.. warning::

We recommend using :attr:`Index.array` or :meth:`Index.to\_numpy`, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

Index.array : Reference to the underlying data.

Index.to_numpy : A NumPy array representing the underlying data.

Examples

For :class:`pandas.Index`:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors inherited from Index:

array

The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.

Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

=====

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

dtype

Return the dtype object of the underlying data.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.dtype
dtype('int64')
```

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

bool

See Also

Index.isna : Detect missing values.
Index.dropna : Return Index without NA/NaN values.
Index.fillna : Fill NA/NaN values with the specified value.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", None])
>>> s
a    1
b    2
None 3
dtype: int64
>>> s.index.hasnans
True
```

```

name
    Return Index or MultiIndex name.

    Returns
    -----
    label (hashable object)
        The name of the Index.

    See Also
    -----
    Index.set_names: Able to set new names partially and by level.
    Index.rename: Able to set new names partially and by level.
    Series.name: Corresponding Series property.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3], name="x")
    >>> idx
    Index([1, 2, 3], dtype='int64', name='x')
    >>> idx.name
    'x'

names
    Get names on index.

    This method returns a FrozenList containing the names of the object.
    It's primarily intended for internal use.

    Returns
    -----
    FrozenList
        A FrozenList containing the object's names, contains None if the object
        does not have a name.

    See Also
    -----
    Index.name : Index name as a string, or None for MultiIndex.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3], name="x")
    >>> idx.names
    FrozenList(['x'])

    >>> idx = pd.Index([1, 2, 3], name=("x", "y"))
    >>> idx.names
    FrozenList(['x', 'y'])

    If the index does not have a name set:

    >>> idx = pd.Index([1, 2, 3])
    >>> idx.names
    FrozenList([None])

-----
Data and other attributes inherited from Index:
__pandas_priority__ = 2000

str = <class 'pandas.core.strings.accessor.StringMethods'>
    Vectorized string functions for Series and Index.

    NAs stay NA unless handled otherwise by a particular method.
    Patterned after Python's string methods, with some inspiration from
    R's stringr package.

    Parameters
    -----
    data : Series or Index
        The content of the Series or Index.

    See Also
    -----
    Series.str : Vectorized string functions for Series.
    Index.str : Vectorized string functions for Index.

```


Examples

```
-----
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str

>>> s.str.split("_")
0    [A, Str, Series]
dtype: object

>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

`__iter__(self)` -> Iterator

Return an iterator of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

iterator

An iterator yielding scalar values from the Series.

See Also

`Series.items` : Lazily iterate over (index, value) tuples.

Examples

```
-----
>>> s = pd.Series([1, 2, 3])
>>> for x in s:
...     print(x)
1
2
3
```

`factorize(self, sort: bool = False, use_na_sentinel: bool = True)` -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]
Encode the object as an enumerated type or categorical variable.

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. ``factorize`` is available as both a top-level function `:func:`pandas.factorize``, and as a method `:meth:`Series.factorize`` and `:meth:`Index.factorize``.

Parameters

`sort` : bool, default False

Sort ``uniques`` and shuffle ``codes`` to maintain the relationship.

`use_na_sentinel` : bool, default True

If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray

An integer ndarray that's an indexer into ``uniques``. ``uniques.take(codes)`` will have the same values as ``values``.

`uniques` : ndarray, Index, or Categorical

The unique valid values. When ``values`` is Categorical, ``uniques`` is a Categorical. When ``values`` is some other pandas object, an ``Index`` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in ``values``, ``uniques`` will *not* contain an entry for it.

See Also

cut : Discretize continuous-valued array.
unique : Find the unique value in an array.

Notes

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

These examples all show factorize as a top-level method like
``pd.factorize(values)``. The results are identical for methods like
:meth:`Series.factorize`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With ``sort=True``, the ``uniques`` will be sorted, and ``codes`` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use\_na\_sentinel=True`` (the default), missing values are indicated in the ``codes`` with the sentinel value ``-1`` and missing values are not included in ``uniques``.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of ``uniques`` will differ. For Categoricals, a ``Categorical`` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
```

```

array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])

```

item(self)
Return the first element of the underlying data as a Python scalar.

Returns

scalar
The first element of Series or Index.

Raises

ValueError
If the data is not length = 1.

See Also

Index.values : Returns an array representing the data in the Index.
Series.head : Returns the first `n` rows.

Examples

>>> s = pd.Series([1])
>>> s.item()
1

For an index:

```

>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'

```

nunique(self, dropna: bool = True) -> int
Return number of unique elements in the object.

Excludes NA values by default.

Parameters

dropna : bool, default True
Don't include NaN in the count.

Returns

int
An integer indicating the number of unique elements in the object.

See Also

DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.

Examples

>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0 1
1 3
2 5
3 7
4 7
dtype: int64

```

>>> s.nunique()
4

```

searchsorted(
self,
value: NumpyValueArrayLike | ExtensionArray,

```

side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted Index `self` such that, if the
corresponding elements in `value` were inserted before the indices,
the order of `self` would be preserved.

.. note::

    The Index must be monotonically sorted, otherwise
    wrong locations will likely be returned. Pandas does not
    check this for you.

Parameters
-----
value : array-like or scalar
    Values to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort `self` into ascending
    order. They are typically the result of ``np.argsort``.

Returns
-----
int or array of int
    A scalar or array of insertion points with the
    same shape as `value`.

See Also
-----
sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes
-----
Binary search is used to find the required insertion points.

Examples
-----
>>> ser = pd.Series([1, 2, 3])
>>> ser
0    1
1    2
2    3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0    2000-03-11
1    2000-03-12
2    2000-03-13
dtype: datetime64[us]

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
... )
>>> ser

```

```
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']
```

```
>>> ser.searchsorted("bread")
np.int64(1)
```

```
>>> ser.searchsorted(["bread"], side="right")
array([3])
```

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])
>>> ser
0    2
1    1
2    3
dtype: int64
```

```
>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1
```

to_list = tolist(self) -> list

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>,
    **kwargs
) -> np.ndarray
A NumPy ndarray representing the values in this Series or Index.
```

Parameters

dtype : str or numpy.dtype, optional
The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.
na_value : Any, optional
The value to use for missing values. The default value depends on `dtype` and the type of the array.
**kwargs
Additional keywords passed through to the ``to\_numpy`` method of the underlying array (for extension arrays).

Returns

numpy.ndarray
The NumPy ndarray holding the values from this Series or Index. The dtype of the array may differ. See Notes.

See Also

Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

Notes

The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` *may* require copying data and coercing the result to a NumPy type (possibly object), which may be

expensive. When you need a no-copy reference to the underlying data, `:attr: 'Series.array'` should be used instead.

This table lays out the different dtypes and default return types of `to_numpy()` for various dtypes within pandas.

dtype	array type
category[T]	ndarray[T] (same dtype as input)
period	ndarray[object] (Periods)
interval	ndarray[object] (Intervals)
IntegerNA	ndarray[object]
datetime64[ns]	datetime64[ns]
datetime64[ns, tz]	ndarray[object] (Timestamps)

Examples

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)
```

Specify the `dtype` to control how datetime-aware data is represented. Use `dtype=object` to return an ndarray of pandas `:class: 'Timestamp'` objects, each with the correct `tz`.

```
>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or `dtype='datetime64[ns]'` to return an ndarray of native `datetime64` values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

`tolist(self) -> list`
Return a list of the values.

These are each a scalar type, which is a Python scalar (for `str`, `int`, `float`) or a pandas scalar (for `Timestamp`/`Timedelta`/`Interval`/`Period`)

Returns

`list`
List containing the values as Python or pandas scalars.

See Also

`numpy.ndarray.tolist` : Return the array as an `a.ndim`-levels deep nested list of Python scalars.

Examples

For Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.tolist()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.tolist()
[1, 2, 3]
```

`transpose(self, *args, **kwargs) -> Self`

Return the transpose, which is by definition self.

Returns

%(klass)s

value_counts()

self,
normalize: bool = False,
sort: bool = True,
ascending: bool = False,
bins=None,
dropna: bool = True

) -> Series

Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

normalize : bool, default False

If True then the object returned will contain the relative frequencies of the unique values.

sort : bool, default True

Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False

Sort in ascending order.

bins : int, optional

Rather than count values, group them into half-open bins, a convenience for ``pd.cut``, only works with numeric data.

dropna : bool, default True

Don't include counts of NaN.

Returns

Series

Series containing counts of unique values.

See Also

Series.count: Number of non-NA elements in a Series.

DataFrame.count: Number of non-NA elements in a DataFrame.

DataFrame.value_counts: Equivalent method on DataFrames.

Examples

>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])

>>> index.value_counts()

3.0 2

1.0 1

2.0 1

4.0 1

Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])

>>> s.value_counts(normalize=True)

3.0 0.4

1.0 0.2

2.0 0.2

4.0 0.2

Name: proportion, dtype: float64

****bins****

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified

number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]      2
(3.0, 4.0]      1
Name: count, dtype: int64
```

****dropna****

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64
```

****Categorical Dtypes****

Rows with categorical type will be counted as one group if they have same categories and order.
In the example below, even though `''a''`, `''c''`, and `''d''` all have the same data types of `category`, only `''c''` and `''d''` will be counted as one group since `''a''` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
```

```
   a  b  c  d
0  1  2  3  3
```

```
>>> df.dtypes
a    category
b      str
c    category
d    category
dtype: object
```

```
>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64
```

Readonly properties inherited from `pandas.core.base.IndexOpsMixin`:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
```



```
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.

Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.

Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
-----
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
```

	height	weight
elk	3.0	500.5
moose	4.1	800.5

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
```

	elk	height	moose	weight
elk	NaN	NaN	NaN	NaN
moose	NaN	NaN	NaN	NaN

```
>>> df[["height", "weight"]].add(s2, axis="index")
```

	height	weight
elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
```

	height	weight
deer	NaN	NaN
elk	1.7	NaN
moose	3.0	NaN

__and__(self, other)

__divmod__(self, other)

```

__eq__(self, other)
    Return self==value.

__floordiv__(self, other)

__ge__(self, other)
    Return self>=value.

__gt__(self, other)
    Return self>value.

__le__(self, other)
    Return self<=value.

__lt__(self, other)
    Return self<value.

__mod__(self, other)

__mul__(self, other)

__ne__(self, other)
    Return self!=value.

__or__(self, other)
    Return self|value.

__pow__(self, other)

__radd__(self, other)

__rand__(self, other)

__rdivmod__(self, other)

__rfloordiv__(self, other)

__rmod__(self, other)

__rmul__(self, other)

__ror__(self, other)
    Return value|self.

__rpow__(self, other)

__rsub__(self, other)

__rtruediv__(self, other)

__rxor__(self, other)

__sub__(self, other)

__truediv__(self, other)

__xor__(self, other)

```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

```

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

```

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

```

__hash__ = None

```

Methods inherited from pandas.core.base.PandasObject:

```

__sizeof__(self) -> int
    Generates the total memory usage for an object that returns

```

either a value or Series of values

Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]
Provide method name lookup and completion.

Notes

Only provide 'public' methods.

class MultiIndex(Index)

MultiIndex(
 levels=None,
 codes=None,
 sortorder=None,
 names=None,
 copy: bool = False,
 name=None,
 verify_integrity: bool = True
) -> Self

A multi-level, or hierarchical, index object for pandas objects.

Parameters

levels : sequence of arrays
 The unique labels for each level.
codes : sequence of arrays
 Integers for each level designating which label at each location.
sortorder : optional int
 Level of sortedness (must be lexicographically sorted by that level).
names : optional sequence of objects
 Names for each of the index levels. (name is accepted for compat).
copy : bool, default False
 Copy the meta-data.
name : Label
 Kept for compatibility with 1-dimensional Index. Should not be used.
verify_integrity : bool, default True
 Check that the levels/codes are consistent and valid.

Attributes

names
levels
codes
nlevels
levshape
dtypes

Methods

from_arrays
from_tuples
from_product
from_frame
set_levels
set_codes
to_frame
to_flat_index
sortlevel
droplevel
swaplevel
reorder_levels
remove_unused_levels
get_level_values
get_indexer
get_loc
get_locs
get_loc_level
drop

See Also

MultiIndex.from_arrays : Convert list of arrays to MultiIndex.

MultiIndex.from_product : Create a MultiIndex from the cartesian product of iterables.
MultiIndex.from_tuples : Convert list of tuples to a MultiIndex.
MultiIndex.from_frame : Make a MultiIndex from a DataFrame.
Index : The base pandas Index type.

Notes

See the `user guide`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/advanced.html>`\_\_`
for more.

Examples

A new ``MultiIndex`` is typically constructed using one of the helper methods :meth:`MultiIndex.from\_arrays`, :meth:`MultiIndex.from\_product` and :meth:`MultiIndex.from\_tuples`. For example (using ``.from\_arrays``):

```
>>> arrays = [[1, 1, 2, 2], ["red", "blue", "red", "blue"]]
>>> pd.MultiIndex.from_arrays(arrays, names=("number", "color"))
MultiIndex([(1, 'red'),
            (1, 'blue'),
            (2, 'red'),
            (2, 'blue')],
            names=['number', 'color'])
```

See further examples for how to construct a MultiIndex in the doc strings of the mentioned helper methods.

Method resolution order:

```
MultiIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
builtins.object
```

Methods defined here:

__abs__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_invalid

__add__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_invalid

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.multi.MultiIndex
the array interface, return my values

__contains__(self, key: Any) -> bool from pandas.core.indexes.multi.MultiIndex
Return a boolean indicating whether the provided key is in the index.

Parameters

key : label
The key to check if it is present in the index.

Returns

bool
Whether the key search is in the index.

Raises

TypeError
If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the list-like key is in the index.

Examples

>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"], ["c"]])
>>> mi
MultiIndex([('a', 'b', 'c')],
)

```

>>> "a" in mi
True
>>> "x" in mi
False

__divmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__floordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__getitem__(self, key) from pandas.core.indexes.multi.MultiIndex
    Override numpy.ndarray's __getitem__ method to work as desired.

    This function adds lists and Series as valid boolean indexers
    (ndarrays only supports ndarray with dtype=bool).

    If resulting ndim != 1, plain ndarray is returned instead of
    corresponding `Index` subclass.

__iadd__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__invert__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__isub__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__len__(self) -> int from pandas.core.indexes.multi.MultiIndex
    Return the length of the Index.

__mod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__mul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__neg__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__pos__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__pow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__radd__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rdivmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__reduce__(self) from pandas.core.indexes.multi.MultiIndex
    Necessary for making this object picklable

__rfloordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rmul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rpow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rsub__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__rtruediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__sub__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

__truediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_inval

append(self, other) from pandas.core.indexes.multi.MultiIndex
    Append a collection of Index options together.

    The `append` method is used to combine multiple `Index` objects into a single
    `Index`. This is particularly useful when dealing with multi-level indexing
    (MultiIndex) where you might need to concatenate different levels of indices.
    The method handles the alignment of the levels and codes of the indices being
    appended to ensure consistency in the resulting `MultiIndex`.

Parameters
-----
other : Index or list/tuple of indices
    Index or list/tuple of Index objects to be appended.

Returns
-----
Index

```

The combined index.

See Also

MultiIndex: A multi-level, or hierarchical, index object for pandas objects.
Index.append : Append a collection of Index options together.
concat : Concatenate pandas objects along a particular axis.

Examples

>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"]])
>>> mi
MultiIndex([('a', 'b')],
)
>>> mi.append(mi)
MultiIndex([('a', 'b'), ('a', 'b')],
)

argsort(self, *args, na_position: NaPosition = 'last', **kwargs) -> npt.NDArray[np.intp] from
Return the integer indices that would sort the index.

Parameters

*args
 Passed to `numpy.ndarray.argsort`.
na_position : {'first' or 'last'}, default 'last'
 Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
 the end.
**kwargs
 Passed to `numpy.ndarray.argsort`.

Returns

np.ndarray[np.intp]
 Integer indices that would sort the index if used as
 an indexer.

See Also

numpy.argsort : Similar method for NumPy arrays.
Index.argsort : Similar method for Index.

Examples

>>> midx = pd.MultiIndex.from_arrays([[3, 2], ["e", "c"]])
>>> midx
MultiIndex([(3, 'e'),
 (2, 'c')],
)

>>> order = midx.argsort()
>>> order
array([1, 0])

>>> midx[order]
MultiIndex([(2, 'c'),
 (3, 'e')],
)

>>> midx = pd.MultiIndex.from_arrays([[2, 2], [np.nan, 0]])
>>> midx.argsort(na_position="first")
array([0, 1])

>>> midx.argsort()
array([1, 0])

astype(self, dtype, copy: bool = True) from pandas.core.indexes.multi.MultiIndex
Create an MultiIndex with values cast to dtypes.

The class of a new Index is determined by dtype. When conversion is
impossible, a TypeError exception is raised.

Parameters

dtype : numpy dtype or pandas type
 Note that any signed integer `dtype` is treated as ``'int64'``,
 and any unsigned integer `dtype` is treated as ``'uint64'``,

regardless of the size.
copy : bool, default True
By default, astype always returns a newly allocated object.
If copy is set to False and internal requirements on dtype are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

MultiIndex

MultiIndex with values cast to specified dtype.

See Also

Index.dtype: Return the dtype object of the underlying data.

Index.dtypes: Return the dtype object of the underlying data.

Index.convert_dtypes: Convert columns to the best possible dtypes.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([[1, 2, 3], [4, 5, 6]])
```

```
>>> mi
```

```
MultiIndex([(1, 4),  
            (2, 5),  
            (3, 6)],  
           )
```

```
>>> mi.astype("object")
```

```
MultiIndex([(1, 4),  
            (2, 5),  
            (3, 6)],  
           )
```

```
copy(self, names=None, deep: bool = False, name=None) -> Self from pandas.core.indexes.multi  
Make a copy of this object. Names, dtype, levels and codes can be passed and will
```

The `copy` method provides a mechanism to create a duplicate of an existing MultiIndex object. This is particularly useful in scenarios where modifications are required on an index, but the original MultiIndex should remain unchanged. By specifying the `deep` parameter, users can control whether the copy should be a deep or shallow copy, providing flexibility depending on the size and complexity of the MultiIndex.

Parameters

names : sequence, optional

Names to set on the new MultiIndex object.

deep : bool, default False

If False, the new object will be a shallow copy. If True, a deep copy will be attempted. Deep copying can be potentially expensive for large MultiIndex objects.

name : Label

Kept for compatibility with 1-dimensional Index. Should not be used.

Returns

MultiIndex

A new MultiIndex object with the specified modifications.

See Also

MultiIndex.from_arrays : Convert arrays to MultiIndex.

MultiIndex.from_tuples : Convert list of tuples to MultiIndex.

MultiIndex.from_frame : Convert DataFrame to MultiIndex.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy. This could be potentially expensive on large MultiIndex objects.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"], ["c"]])
```

```
>>> mi
```

```
MultiIndex([( 'a', 'b', 'c')],  
           )
```

```
>>> mi.copy()
```

```

MultiIndex([('a', 'b', 'c')],
)
delete(self, loc) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
    Make new index with passed location deleted

Returns
-----
new_index : MultiIndex

drop(
    self,
    codes,
    level: Index | np.ndarray | Iterable[Hashable] | None = None,
    errors: IgnoreRaise = 'raise'
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
    Make a new :class:`pandas.MultiIndex` with the passed list of codes deleted.

This method allows for the removal of specified labels from a MultiIndex.
The labels to be removed can be provided as a list of tuples if no level
is specified, or as a list of labels from a specific level if the level
parameter is provided. This can be useful for refining the structure of a
MultiIndex to fit specific requirements.

Parameters
-----
codes : array-like
    Must be a list of tuples when ``level`` is not specified.
level : int or level name, default None
    Level from which the labels will be dropped.
errors : str, default 'raise'
    If 'ignore', suppress error and existing labels are dropped.

Returns
-----
MultiIndex
    A new MultiIndex with the specified labels removed.

See Also
-----
MultiIndex.remove_unused_levels : Create new MultiIndex from current that
    removes unused levels.
MultiIndex.reorder_levels : Rearrange levels using input order.
MultiIndex.rename : Rename levels in a MultiIndex.

Examples
-----
>>> idx = pd.MultiIndex.from_product(
...     [(0, 1, 2), ("green", "purple")], names=["number", "color"]
... )
>>> idx
MultiIndex([(0, 'green'),
            (0, 'purple'),
            (1, 'green'),
            (1, 'purple'),
            (2, 'green'),
            (2, 'purple')],
            names=['number', 'color'])
>>> idx.drop([(1, "green"), (2, "purple")])
MultiIndex([(0, 'green'),
            (0, 'purple'),
            (1, 'purple'),
            (2, 'green')],
            names=['number', 'color'])

We can also drop from a specific level.

>>> idx.drop("green", level="color")
MultiIndex([(0, 'purple'),
            (1, 'purple'),
            (2, 'purple')],
            names=['number', 'color'])

>>> idx.drop([1, 2], level=0)
MultiIndex([(0, 'green'),
            (0, 'purple')],
            names=['number', 'color'])

```

`dropna(self, how: AnyAll = 'any') -> MultiIndex` from `pandas.core.indexes.multi.MultiIndex`
Return MultiIndex without NA/Nan values.

Parameters

`how : {'any', 'all'}, default 'any'`
Drop the value when any or all levels are NaN.

Returns

`Index`
Returns an MultiIndex object after removing NA/Nan values.

See Also

`Index.fillna` : Fill NA/Nan values with the specified value.
`Index.isna` : Detect missing values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([[np.nan, np.nan, 2.0], [3.0, np.nan, 4.0]])
>>> mi.dropna()
MultiIndex([(2.0, 4.0)],
            )
>>> mi.dropna(how="all")
MultiIndex([(nan, 3.0),
            (2.0, 4.0)],
            )
```

`duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_]` from `pandas.core.indexes`
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

`keep : {'first', 'last', False}, default 'first'`
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

`np.ndarray[bool]`
A numpy array of boolean values indicating duplicate index values.

See Also

`Series.duplicated` : Equivalent method on `pandas.Series`.
`DataFrame.duplicated` : Equivalent method on `pandas.DataFrame`.
`Index.drop_duplicates` : Remove duplicate values from Index.

Examples

By default, for each set of duplicated values, the first occurrence is set to False and all others to True:

```
>>> mi = pd.MultiIndex.from_arrays((list("abca"), list("defd")))
>>> mi.duplicated()
array([False, False, False,  True])
```

which is equivalent to

```
>>> mi.duplicated(keep="first")
array([False, False, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> mi.duplicated(keep="last")
```

```
array([ True, False, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> mi.duplicated(keep=False)
array([ True, False, False,  True])
```

`equal_levels(self, other: MultiIndex) -> bool` from `pandas.core.indexes.multi.MultiIndex`
Return True if the levels of both MultiIndex objects are the same

`equals(self, other: object) -> bool` from `pandas.core.indexes.multi.MultiIndex`
Determines if two MultiIndex objects have the same labeling information
(the levels themselves do not necessarily have to be the same)

See Also

`equal_levels`

`fillna(self, value)` from `pandas.core.indexes.multi.MultiIndex`
`fillna` is not implemented for MultiIndex

`get_level_values(self, level) -> Index` from `pandas.core.indexes.multi.MultiIndex`
Return vector of label values for requested level.

Length of returned vector is equal to the length of the index.
The `'get_level_values'` method is a crucial utility for extracting specific level values from a `'MultiIndex'`. This function is particularly useful when working with multi-level data, allowing you to isolate and manipulate individual levels without having to deal with the complexity of the entire `'MultiIndex'` structure. It seamlessly handles both integer and string-based level access, providing flexibility in how you can interact with the data. Additionally, this method ensures that the returned `'Index'` maintains the integrity of the original data, even when missing values are present, by appropriately casting the result to a suitable data type.

Parameters

`level : int or str`
 `'level'` is either the integer position of the level in the MultiIndex, or the name of the level.

Returns

`Index`
Values is a level of this MultiIndex converted to a single `:class:'Index'` (or subclass thereof).

See Also

`MultiIndex` : A multi-level, or hierarchical, index object for pandas objects.
`Index` : Immutable sequence used for indexing and alignment.
`MultiIndex.remove_unused_levels` : Create new MultiIndex from current that removes unused levels.

Notes

If the level contains missing values, the result may be casted to `'float'` with missing values specified as `'NaN'`. This is because the level is converted to a regular `'Index'`.

Examples

Create a MultiIndex:

```
>>> mi = pd.MultiIndex.from_arrays((list("abc"), list("def")))
>>> mi.names = ["level_1", "level_2"]
```

Get level values by supplying level as either integer or name:

```
>>> mi.get_level_values(0)
Index(['a', 'b', 'c'], dtype='str', name='level_1')
>>> mi.get_level_values("level_2")
Index(['d', 'e', 'f'], dtype='str', name='level_2')
```

If a level contains missing values, the return type of the level may be cast to `'float'`.

```
>>> pd.MultiIndex.from_arrays([[1, None, 2], [3, 4, 5]]).dtypes
level_0    int64
level_1    int64
dtype: object
>>> pd.MultiIndex.from_arrays([[1, None, 2], [3, 4, 5]]).get_level_values(0)
Index([1.0, nan, 2.0], dtype='float64')
```

`get_loc(self, key)` from `pandas.core.indexes.multi.MultiIndex`
Get location for a label or a tuple of labels. The location is returned as an int

This method returns the integer location, slice object, or boolean mask corresponding to the specified key, which can be a single label or a tuple of labels. The key represents a position in the MultiIndex, and the location indicates where the key is found within the index.

Parameters

`key` : label or tuple of labels (one for each level)
A label or tuple of labels that correspond to the levels of the MultiIndex. The key must match the structure of the MultiIndex.

Returns

int, slice object or boolean mask
If the key is past the lexicographic depth, the return may be a boolean mask array, otherwise it is always a slice or int.

See Also

`Index.get_loc` : The `get_loc` method for (single-level) index.
`MultiIndex.slice_locs` : Get slice location given start label(s) and end label(s).
`MultiIndex.get_locs` : Get location for a label/slice/list/mask or a sequence of such.

Notes

The key cannot be a slice, list of same-level labels, a boolean mask, or a sequence of such. If you want to use those, use `:meth: 'MultiIndex.get_locs'` instead.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([list("abb"), list("def")])

>>> mi.get_loc("b")
slice(1, 3, None)

>>> mi.get_loc(("b", "e"))
1
```

`get_loc_level(self, key, level: IndexLabel = 0, drop_level: bool = True)` from `pandas.core.indexes.multi.MultiIndex`
Get location and sliced index for requested label(s)/level(s).

The `'get_loc_level'` method is a more advanced form of `'get_loc'`, allowing users to specify not just a label or sequence of labels, but also the level(s) in which to search. This method is useful when you need to isolate particular sections of a MultiIndex, either for further analysis or for slicing and dicing the data. The method provides flexibility in terms of maintaining or dropping levels from the resulting index based on the `'drop_level'` parameter.

Parameters

`key` : label or sequence of labels
The label(s) for which to get the location.
`level` : int/level name or list thereof, optional
The level(s) in the MultiIndex to consider. If not provided, defaults to the first level.
`drop_level` : bool, default True
If `'False'`, the resulting index will not drop any level.

Returns

tuple
A 2-tuple where the elements :

Element 0: int, slice object or boolean array.

Element 1: The resulting sliced multiindex/index. If the key contains all levels, this will be ``None``.

See Also

MultiIndex.get_loc : Get location for a label or a tuple of labels.
MultiIndex.get_locs : Get location for a label/slice/list/mask or a sequence of such.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([list("abb"), list("def")], names=["A", "B"])
```

```
>>> mi.get_loc_level("b")  
(slice(1, 3, None), Index(['e', 'f'], dtype='str', name='B'))
```

```
>>> mi.get_loc_level("e", level="B")  
(array([False,  True, False]), Index(['b'], dtype='str', name='A'))
```

```
>>> mi.get_loc_level(["b", "e"])  
(1, None)
```

get_locs(self, seq) -> npt.NDArray[np.intp] from pandas.core.indexes.multi.MultiIndex
Get location for a sequence of labels.

Parameters

seq : label, slice, list, mask or a sequence of such
You should use one of the above for each level.
If a level should not be used, set it to ``slice(None)``.

Returns

numpy.ndarray
NumPy array of integers suitable for passing to iloc.

See Also

MultiIndex.get_loc : Get location for a label or a tuple of labels.
MultiIndex.slice_locs : Get slice location given start label(s) and end label(s).

Examples

```
>>> mi = pd.MultiIndex.from_arrays([list("abb"), list("def")])
```

```
>>> mi.get_locs("b") # doctest: +SKIP  
array([1, 2], dtype=int64)
```

```
>>> mi.get_locs([slice(None), ["e", "f"]]) # doctest: +SKIP  
array([1, 2], dtype=int64)
```

```
>>> mi.get_locs([[True, False, True], slice("e", "f")]) # doctest: +SKIP  
array([2], dtype=int64)
```

get_slice_bound(
self,
label: Hashable | Sequence[Hashable],
side: Literal['left', 'right']
) -> int from pandas.core.indexes.multi.MultiIndex
For an ordered MultiIndex, compute slice bound
that corresponds to given label.

Returns leftmost (one-past-the-rightmost if `side=='right') position
of given label.

Parameters

label : object or tuple of objects
side : {'left', 'right'}

Returns

int

Index of label.

Notes

This method only works if level 0 index of the MultiIndex is lexicographically sorted.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([list("abbc"), list("gefd")])
```

Get the locations from the leftmost 'b' in the first level until the end of the multiindex:

```
>>> mi.get_slice_bound("b", side="left")
1
```

Like above, but if you get the locations from the rightmost 'b' in the first level and 'f' in the second level:

```
>>> mi.get_slice_bound(("b", "f"), side="right")
3
```

See Also

MultiIndex.get_loc : Get location for a label or a tuple of labels.

MultiIndex.get_locs : Get location for a label/slice/list/mask or a sequence of such.

insert(self, loc: int, item) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Make new MultiIndex inserting new item at location

Parameters

loc : int

item : tuple

Must be same length as number of levels in the MultiIndex

Returns

new_index : Index

isin(self, values, level=None) -> npt.NDArray[np.bool_] from pandas.core.indexes.multi.MultiIndex
Return a boolean array where the index values are in `values`.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like

Sought values.

level : str or int, optional

Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

np.ndarray[bool]

NumPy array of boolean values.

See Also

Series.isin : Same for Series.

DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a ``ValueError``.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])
array([ True, False, False])
```

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]
... )
>>> mi
MultiIndex([(1, 'red'),
            (2, 'blue'),
            (3, 'green')],
            names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> mi.isin(["red", "orange", "yellow"], level="color")
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> mi.isin([(1, "red"), (3, "red")])
array([ True, False, False])
```

memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.multi.MultiIndex
Memory usage of the values.

Parameters

deep : bool, default False
Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption.

Returns

bytes used
Returns memory usage of the values in the Index in bytes.

See Also

numpy.ndarray.nbytes : Total bytes consumed by the elements of the
array.

Notes

Memory usage does not include memory consumed by elements that
are not components of the array if deep=False or if used on PyPy

Examples

```
>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"], ["c"]])
>>> mi.memory_usage()
81
```

putmask(self, mask, value: MultiIndex) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Return a new MultiIndex of the values set with the mask.

Parameters

mask : array like
value : MultiIndex
Must either be the same length as self or length one

Returns

MultiIndex

remove_unused_levels(self) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Create new MultiIndex from current that removes unused levels.

Unused level(s) means levels that are not expressed in the labels. The resulting MultiIndex will have the same outward appearance, meaning the same .values and ordering. It will also be .equals() to the original.

The `remove_unused_levels` method is useful in cases where you have a MultiIndex with hierarchical levels, but some of these levels are no longer needed due to filtering or subsetting operations. By removing the unused levels, the resulting MultiIndex becomes more compact and efficient, which can improve performance in subsequent operations.

Returns

MultiIndex

A new MultiIndex with unused levels removed.

See Also

MultiIndex.droplevel : Remove specified levels from a MultiIndex.

MultiIndex.reorder_levels : Rearrange levels of a MultiIndex.

MultiIndex.set_levels : Set new levels on a MultiIndex.

Examples

```
>>> mi = pd.MultiIndex.from_product([range(2), list("ab")])
```

```
>>> mi
```

```
MultiIndex([(0, 'a'),
            (0, 'b'),
            (1, 'a'),
            (1, 'b')],
           )
```

```
>>> mi[2:]
```

```
MultiIndex([(1, 'a'),
            (1, 'b')],
           )
```

The 0 from the first level is not represented and can be removed

```
>>> mi2 = mi[2:].remove_unused_levels()
```

```
>>> mi2.levels
```

```
FrozenList([[1], ['a', 'b']])
```

`rename = set_names(self, names, *, level=None, inplace: bool = False) -> Self | None`

`reorder_levels(self, order) -> MultiIndex` from `pandas.core.indexes.multi.MultiIndex`
Rearrange levels using input order. May not drop or duplicate levels.

`reorder_levels` is useful when you need to change the order of levels in a MultiIndex, such as when reordering levels for hierarchical indexing. It maintains the integrity of the MultiIndex, ensuring that all existing levels are present and no levels are duplicated. This method is helpful for aligning the index structure with other data structures or for optimizing the order for specific data operations.

Parameters

order : list of int or list of str

List representing new level order. Reference level by number (position) or by key (label).

Returns

MultiIndex

A new MultiIndex with levels rearranged according to the specified order.

See Also

MultiIndex.swaplevel : Swap two levels of the MultiIndex.

MultiIndex.set_names : Set names for the MultiIndex levels.

DataFrame.reorder_levels : Reorder levels in a DataFrame with a MultiIndex.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([[1, 2], [3, 4]], names=["x", "y"])
```

```

>>> mi
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.reorder_levels(order=[1, 0])
MultiIndex([(3, 1),
            (4, 2)],
            names=['y', 'x'])

>>> mi.reorder_levels(order=["y", "x"])
MultiIndex([(3, 1),
            (4, 2)],
            names=['y', 'x'])

```

repeat(self, repeats: int, axis=None) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Repeat elements of a MultiIndex.

Returns a new MultiIndex where each element of the current MultiIndex is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty MultiIndex.

axis : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

MultiIndex
Newly created MultiIndex with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```

>>> idx = pd.MultiIndex.from_arrays([["a", "b", "c"], [1, 2, 3]])
>>> idx
MultiIndex([( 'a', 1),
            ( 'b', 2),
            ( 'c', 3)],
            )
>>> idx.repeat(2)
MultiIndex([( 'a', 1),
            ( 'a', 1),
            ( 'b', 2),
            ( 'b', 2),
            ( 'c', 3),
            ( 'c', 3)],
            )
>>> idx.repeat([1, 2, 3])
MultiIndex([( 'a', 1),
            ( 'b', 2),
            ( 'b', 2),
            ( 'c', 3),
            ( 'c', 3),
            ( 'c', 3)],
            )

```

set_codes(self, codes, *, level=None, verify_integrity: bool = True) -> MultiIndex from pandas
Set new codes on MultiIndex. Defaults to returning new index.

Parameters

codes : sequence or list of sequence
New codes to apply.

level : int, level name, or sequence of int/level names (default None)
Level(s) to set (None for all levels).

verify_integrity : bool, default True
If True, checks that levels and codes are compatible.

Returns

new index (of same type and class...etc) or None
The same type as the caller or None if ``inplace=True``.

See Also

MultiIndex.set_levels : Set new levels on MultiIndex.
MultiIndex.codes : Get the codes of the levels in the MultiIndex.
MultiIndex.levels : Get the levels of the MultiIndex.

Examples

>>> idx = pd.MultiIndex.from_tuples(
... [(1, "one"), (1, "two"), (2, "one"), (2, "two")], names=["foo", "bar"]
...)
>>> idx
MultiIndex([(1, 'one'),
 (1, 'two'),
 (2, 'one'),
 (2, 'two')],
 names=['foo', 'bar'])

>>> idx.set_codes([[1, 0, 1, 0], [0, 0, 1, 1]])
MultiIndex([(2, 'one'),
 (1, 'one'),
 (2, 'two'),
 (1, 'two')],
 names=['foo', 'bar'])
>>> idx.set_codes([1, 0, 1, 0], level=0)
MultiIndex([(2, 'one'),
 (1, 'two'),
 (2, 'one'),
 (1, 'two')],
 names=['foo', 'bar'])
>>> idx.set_codes([0, 0, 1, 1], level="bar")
MultiIndex([(1, 'one'),
 (1, 'one'),
 (2, 'two'),
 (2, 'two')],
 names=['foo', 'bar'])
>>> idx.set_codes([[1, 0, 1, 0], [0, 0, 1, 1]], level=[0, 1])
MultiIndex([(2, 'one'),
 (1, 'one'),
 (2, 'two'),
 (1, 'two')],
 names=['foo', 'bar'])

set_levels(self, levels, *, level=None, verify_integrity: bool = True) -> MultiIndex from pandas
Set new levels on MultiIndex. Defaults to returning new index.

The ``set\_levels`` method provides a flexible way to change the levels of a ``MultiIndex``. This is particularly useful when you need to update the index structure of your DataFrame without altering the data. The method returns a new ``MultiIndex`` unless the operation is performed in-place, ensuring that the original index remains unchanged unless explicitly modified.

The method checks the integrity of the new levels against the existing codes by default, but this can be disabled if you are confident that your levels are consistent with the underlying data. This can be useful when you want to perform optimizations or make specific adjustments to the index levels that do not strictly adhere to the original structure.

Parameters

levels : sequence or list of sequence
New level(s) to apply.
level : int, level name, or sequence of int/level names (default None)
Level(s) to set (None for all levels).
verify_integrity : bool, default True
If True, checks that levels and codes are compatible.

Returns

MultiIndex

A new `MultiIndex` with the updated levels.

See Also

MultiIndex.set_codes : Set new codes on the existing `MultiIndex`.
MultiIndex.remove_unused_levels : Create new MultiIndex from current that removes unused levels.
Index.set_names : Set Index or MultiIndex name.

Examples

>>> idx = pd.MultiIndex.from_tuples(
... [
... (1, "one"),
... (1, "two"),
... (2, "one"),
... (2, "two"),
... (3, "one"),
... (3, "two"),
...],
... names=["foo", "bar"],
...)
>>> idx
MultiIndex([(1, 'one'),
 (1, 'two'),
 (2, 'one'),
 (2, 'two'),
 (3, 'one'),
 (3, 'two')],
 names=['foo', 'bar'])

>>> idx.set_levels([["a", "b", "c"], [1, 2]])
MultiIndex([(a', 1),
 (a', 2),
 (b', 1),
 (b', 2),
 (c', 1),
 (c', 2)],
 names=['foo', 'bar'])
>>> idx.set_levels([["a", "b", "c"], level=0)
MultiIndex([(a', 'one'),
 (a', 'two'),
 (b', 'one'),
 (b', 'two'),
 (c', 'one'),
 (c', 'two')],
 names=['foo', 'bar'])
>>> idx.set_levels([["a", "b"], level="bar")
MultiIndex([(1, 'a'),
 (1, 'b'),
 (2, 'a'),
 (2, 'b'),
 (3, 'a'),
 (3, 'b')],
 names=['foo', 'bar'])

If any of the levels passed to ``set\_levels()`` exceeds the existing length, all of the values from that argument will be stored in the MultiIndex levels, though the values will be truncated in the MultiIndex output.

```
>>> idx.set_levels([["a", "b", "c"], [1, 2, 3, 4]], level=[0, 1])
MultiIndex([(a', 1),
            (a', 2),
            (b', 1),
            (b', 2),
            (c', 1),
            (c', 2)],
            names=['foo', 'bar'])
>>> idx.set_levels([["a", "b", "c"], [1, 2, 3, 4]], level=[0, 1]).levels
FrozenList([(a', 'b', 'c'), [1, 2, 3, 4]])
```

slice_locs(self, start=None, end=None, step=None) -> tuple[int, int] from pandas.core.indexes
For an ordered MultiIndex, compute the slice locations for input labels.

The input labels can be tuples representing partial levels, e.g. for a

MultiIndex with 3 levels, you can pass a single value (corresponding to the first level), or a 1-, 2-, or 3-tuple.

Parameters

start : label or tuple, default None
 If None, defaults to the beginning
end : label or tuple
 If None, defaults to the end
step : int or None
 Slice step

Returns

(start, end) : (int, int)

Notes

This method only works if the MultiIndex is properly lexsorted. So, if only the first 2 levels of a 3-level MultiIndex are lexsorted, you can only pass two levels to ``.slice\_locs``.

Examples

>>> mi = pd.MultiIndex.from_arrays(
... [list("abbd"), list("deff")], names=["A", "B"]
...)

Get the slice locations from the beginning of 'b' in the first level until the end of the multiindex:

```
>>> mi.slice_locs(start="b")  
(1, 4)
```

Like above, but stop at the end of 'b' in the first level and 'f' in the second level:

```
>>> mi.slice_locs(start="b", end=("b", "f"))  
(1, 3)
```

See Also

MultiIndex.get_loc : Get location for a label or a tuple of labels.
MultiIndex.get_locs : Get location for a label/slice/list/mask or a sequence of such.

```
sortlevel(  
    self,  
    level: IndexLabel = 0,  
    ascending: bool | list[bool] = True,  
    sort_remaining: bool = True,  
    na_position: str = 'first'  
) -> tuple[MultiIndex, npt.NDArray[np.intp]] from pandas.core.indexes.multi.MultiIndex  
Sort MultiIndex at the requested level.
```

This method is useful when dealing with MultiIndex objects, allowing for sorting at a specific level of the index. The function preserves the relative ordering of data within the same level while sorting the overall MultiIndex. The method provides flexibility with the `ascending` parameter to define the sort order and with the `sort\_remaining` parameter to control whether the remaining levels should also be sorted. Sorting a MultiIndex can be crucial when performing operations that require ordered indices, such as grouping or merging datasets. The `na\_position` argument is important in handling missing values consistently across different levels.

Parameters

level : list-like, int or str, default 0
 If a string is given, must be a name of the level.
 If list-like must be names or ints of levels.
ascending : bool, default True
 False to sort in descending order.
 Can also be a list to specify a directed ordering.
sort_remaining : bool, default True
 If True, sorts by the remaining levels after sorting by the specified `level`.
na_position : {'first' or 'last'}, default 'first'

Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.

.. versionadded:: 2.1.0

Returns

sorted_index : pd.MultiIndex
 Resulting index.
indexer : np.ndarray[np.intp]
 Indices of output values in original index.

See Also

MultiIndex : A multi-level, or hierarchical, index object for pandas objects.
Index.sort_values : Sort Index values.
DataFrame.sort_index : Sort DataFrame by the index.
Series.sort_index : Sort Series by the index.

Examples

>>> mi = pd.MultiIndex.from_arrays([[0, 0], [2, 1]])
>>> mi
MultiIndex([(0, 2),
 (0, 1)],
)

>>> mi.sortlevel()
(MultiIndex([(0, 1),
 (0, 2)],
), array([1, 0]))

>>> mi.sortlevel(sort_remaining=False)
(MultiIndex([(0, 2),
 (0, 1)],
), array([0, 1]))

>>> mi.sortlevel(1)
(MultiIndex([(0, 1),
 (0, 2)],
), array([1, 0]))

>>> mi.sortlevel(1, ascending=False)
(MultiIndex([(0, 2),
 (0, 1)],
), array([0, 1]))

swaplevel(self, i=-2, j=-1) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Swap level i with level j.

Calling this method does not change the ordering of the values.

Default is to swap the last two levels of the MultiIndex.

Parameters

i : int, str, default -2
 First level of index to be swapped. Can pass level name as string.
 Type of parameters can be mixed. If i is a negative int, the first
 level is indexed relative to the end of the MultiIndex.
j : int, str, default -1
 Second level of index to be swapped. Can pass level name as string.
 Type of parameters can be mixed. If j is a negative int, the second
 level is indexed relative to the end of the MultiIndex.

Returns

MultiIndex
 A new MultiIndex.

See Also

Series.swaplevel : Swap levels i and j in a MultiIndex.
DataFrame.swaplevel : Swap levels i and j in a MultiIndex on a
 particular axis.

Examples

```

-----
>>> mi = pd.MultiIndex(
...     levels=["a", "b"], ["bb", "aa"], ["aaa", "bbb"]],
...     codes=[[0, 0, 1, 1], [0, 1, 0, 1], [1, 0, 1, 0]],
... )
>>> mi
MultiIndex([('a', 'bb', 'bbb'),
            ('a', 'aa', 'aaa'),
            ('b', 'bb', 'bbb'),
            ('b', 'aa', 'aaa')],
           )
>>> mi.swaplevel()
MultiIndex([('a', 'bbb', 'bb'),
            ('a', 'aaa', 'aa'),
            ('b', 'bbb', 'bb'),
            ('b', 'aaa', 'aa')],
           )
>>> mi.swaplevel(0)
MultiIndex([('bbb', 'bb', 'a'),
            ('aaa', 'aa', 'a'),
            ('bbb', 'bb', 'b'),
            ('aaa', 'aa', 'b')],
           )
>>> mi.swaplevel(0, 1)
MultiIndex([('bb', 'a', 'bbb'),
            ('aa', 'a', 'aaa'),
            ('bb', 'b', 'bbb'),
            ('aa', 'b', 'aaa')],
           )

take(
    self: MultiIndex,
    indices,
    axis: Axis = 0,
    allow_fill: bool = True,
    fill_value=None,
    **kwargs
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
    Return a new MultiIndex of the values selected by the indices.

    For internal compatibility with numpy arrays.

    Parameters
    -----
    indices : array-like
        Indices to be taken.
    axis : {0 or 'index'}, optional
        The axis over which to select values, always 0 or 'index'.
    allow_fill : bool, default True
        How to handle negative values in `indices`.

        * False: negative values in `indices` indicate positional indices
          from the right (the default). This is similar to
          :func:`numpy.take`.

        * True: negative values in `indices` indicate
          missing values. These values are set to `fill_value`. Any other
          other negative values raise a ``ValueError``.

    fill_value : scalar, default None
        If allow_fill=True and fill_value is not None, indices specified by
        -1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    Index
        An index formed of elements at the given indices. Will be the same
        type as self, except for RangeIndex.

    See Also
    -----
    numpy.ndarray.take: Return an array formed from the
        elements of a at the given indices.

    Examples

```

```

-----
>>> idx = pd.MultiIndex.from_arrays([["a", "b", "c"], [1, 2, 3]])
>>> idx
MultiIndex([(a', 1),
            (b', 2),
            (c', 3)],
            )
>>> idx.take([2, 2, 1, 0])
MultiIndex([(c', 3),
            (c', 3),
            (b', 2),
            (a', 1)],
            )

```

to_flat_index(self) -> Index from pandas.core.indexes.multi.MultiIndex
Convert a MultiIndex to an Index of Tuples containing the level values.

Returns

pd.Index
Index with the MultiIndex data represented in Tuples.

See Also

MultiIndex.from_tuples : Convert flat index back to MultiIndex.

Notes

This method will simply return the caller if called by anything other than a MultiIndex.

Examples

>>> index = pd.MultiIndex.from_product(
... [["foo", "bar"], ["baz", "qux"]], names=["a", "b"]
...)
>>> index.to_flat_index()
Index([(foo', 'baz'), (foo', 'qux'),
 (bar', 'baz'), (bar', 'qux')],
 dtype='object')

to_frame(
self,
index: bool = True,
name=<no_default>,
allow_duplicates: bool = False
) -> DataFrame from pandas.core.indexes.multi.MultiIndex
Create a DataFrame with the levels of the MultiIndex as columns.

Column ordering is determined by the DataFrame constructor with data as a dict.

Parameters

index : bool, default True
Set the index of the returned DataFrame as the original MultiIndex.

name : list / sequence of str, optional
The passed names should substitute index level names.

allow_duplicates : bool, optional default False
Allow duplicate column labels to be created.

Returns

DataFrame
DataFrame representation of the MultiIndex, with levels as columns.

See Also

DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Examples

>>> mi = pd.MultiIndex.from_arrays([["a", "b"], ["c", "d"]])
>>> mi


```
MultiIndex([('a', 'c'),
            ('b', 'd')],
           )
```

```
>>> df = mi.to_frame()
>>> df
   0  1
a c  a  c
b d  b  d
```

```
>>> df = mi.to_frame(index=False)
>>> df
   0  1
0  a  c
1  b  d
```

```
>>> df = mi.to_frame(name=["x", "y"])
>>> df
   x  y
a c  a  c
b d  b  d
```

`truncate(self, before=None, after=None)` -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Slice index between two labels / tuples, return new MultiIndex.

Parameters

`before` : label or tuple, can be partial. Default None
None defaults to start.
`after` : label or tuple, can be partial. Default None
None defaults to end.

Returns

MultiIndex
The truncated MultiIndex.

See Also

`DataFrame.truncate` : Truncate a DataFrame before and after some index values.
`Series.truncate` : Truncate a Series before and after some index values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays([["a", "b", "c"], ["x", "y", "z"]])
>>> mi
MultiIndex([('a', 'x'), ('b', 'y'), ('c', 'z')],
           )
>>> mi.truncate(before="a", after="b")
MultiIndex([('a', 'x'), ('b', 'y')],
           )
```

`unique(self, level=None)` from pandas.core.indexes.multi.MultiIndex
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

`level` : int or hashable, optional
Only return values from specified level (for MultiIndex).
If int, gets the level by integer position, else by level name.

Returns

MultiIndex
Unique values in the MultiIndex.

See Also

`unique` : Numpy array of unique values in that column.
`Series.unique` : Return unique values of Series object.

Examples

```
>>> mi = pd.MultiIndex.from_arrays((list("abca"), list("defd")))
>>> mi
```

```

MultiIndex([(a', 'd'),
            (b', 'e'),
            (c', 'f'),
            (a', 'd')],
           )
>>> mi.unique()
MultiIndex([(a', 'd'),
            (b', 'e'),
            (c', 'f')],
           )

view(self, cls=None) -> Self from pandas.core.indexes.multi.MultiIndex
this is defined as a copy with the same identity
-----
Class methods defined here:

from_arrays(
    arrays,
    sortorder: int | None = None,
    names: Sequence[Hashable] | Hashable | lib.NoDefault = <no_default>
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Convert arrays to MultiIndex.

Parameters
-----
arrays : list / sequence of array-likes
    Each array-like gives one level's value for each data point.
    len(arrays) is the number of levels.
sortorder : int or None
    Level of sortedness (must be lexicographically sorted by that
    level).
names : list / sequence of str, optional
    Names for the levels in the index.

Returns
-----
MultiIndex

See Also
-----
MultiIndex.from_tuples : Convert list of tuples to MultiIndex.
MultiIndex.from_product : Make a MultiIndex from cartesian product
    of iterables.
MultiIndex.from_frame : Make a MultiIndex from a DataFrame.

Examples
-----
>>> arrays = [[1, 1, 2, 2], ["red", "blue", "red", "blue"]]
>>> pd.MultiIndex.from_arrays(arrays, names=("number", "color"))
MultiIndex([(1, 'red'),
            (1, 'blue'),
            (2, 'red'),
            (2, 'blue')],
           names=['number', 'color'])

from_frame(
    df: DataFrame,
    sortorder: int | None = None,
    names: Sequence[Hashable] | Hashable | None = None
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Make a MultiIndex from a DataFrame.

Parameters
-----
df : DataFrame
    DataFrame to be converted to MultiIndex.
sortorder : int, optional
    Level of sortedness (must be lexicographically sorted by that
    level).
names : list-like, optional
    If no names are provided, use the column names, or tuple of column
    names if the columns is a MultiIndex. If a sequence, overwrite
    names with the given sequence.

Returns
-----

```

MultiIndex
The MultiIndex representation of the given DataFrame.

See Also

MultiIndex.from_arrays : Convert list of arrays to MultiIndex.
MultiIndex.from_tuples : Convert list of tuples to MultiIndex.
MultiIndex.from_product : Make a MultiIndex from cartesian product
of iterables.

Examples

>>> df = pd.DataFrame(
... [["HI", "Temp"], ["HI", "Precip"], ["NJ", "Temp"], ["NJ", "Precip"]],
... columns=["a", "b"],
...)
>>> df
 a b
0 HI Temp
1 HI Precip
2 NJ Temp
3 NJ Precip

>>> pd.MultiIndex.from_frame(df)
MultiIndex([('HI', 'Temp'),
 ('HI', 'Precip'),
 ('NJ', 'Temp'),
 ('NJ', 'Precip')],
 names=['a', 'b'])

Using explicit names, instead of the column names

>>> pd.MultiIndex.from_frame(df, names=["state", "observation"])
MultiIndex([('HI', 'Temp'),
 ('HI', 'Precip'),
 ('NJ', 'Temp'),
 ('NJ', 'Precip')],
 names=['state', 'observation'])

from_product(
 iterables: Sequence[Iterable[Hashable]],
 sortorder: int | None = None,
 names: Sequence[Hashable] | Hashable | lib.NoDefault = <no_default>
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Make a MultiIndex from the cartesian product of multiple iterables.

Parameters

iterables : list / sequence of iterables
 Each iterable has unique labels for each level of the index.
sortorder : int or None
 Level of sortedness (must be lexicographically sorted by that
 level).
names : list / sequence of str, optional
 Names for the levels in the index.
 If not explicitly provided, names will be inferred from the
 elements of iterables if an element has a name attribute.

Returns

MultiIndex

See Also

MultiIndex.from_arrays : Convert list of arrays to MultiIndex.
MultiIndex.from_tuples : Convert list of tuples to MultiIndex.
MultiIndex.from_frame : Make a MultiIndex from a DataFrame.

Examples

>>> numbers = [0, 1, 2]
>>> colors = ["green", "purple"]
>>> pd.MultiIndex.from_product([numbers, colors], names=["number", "color"])
MultiIndex([(0, 'green'),
 (0, 'purple'),
 (1, 'green'),
 (1, 'purple')],
 names=['number', 'color'])

```

        (2, 'green'),
        (2, 'purple')],
        names=['number', 'color'])

from_tuples(
    tuples: Iterable[tuple[Hashable, ...]],
    sortorder: int | None = None,
    names: Sequence[Hashable] | Hashable | None = None
) -> MultiIndex from pandas.core.indexes.multi.MultiIndex
Convert list of tuples to MultiIndex.

Parameters
-----
tuples : list / sequence of tuple-likes
    Each tuple is the index of one row/column.
sortorder : int or None
    Level of sortedness (must be lexicographically sorted by that
    level).
names : list / sequence of str, optional
    Names for the levels in the index.

Returns
-----
MultiIndex

See Also
-----
MultiIndex.from_arrays : Convert list of arrays to MultiIndex.
MultiIndex.from_product : Make a MultiIndex from cartesian product
    of iterables.
MultiIndex.from_frame : Make a MultiIndex from a DataFrame.

Examples
-----
>>> tuples = [(1, "red"), (1, "blue"), (2, "red"), (2, "blue")]
>>> pd.MultiIndex.from_tuples(tuples, names=("number", "color"))
MultiIndex([(1, 'red'),
            (1, 'blue'),
            (2, 'red'),
            (2, 'blue')],
            names=['number', 'color'])
-----
Static methods defined here:

__new__(
    cls,
    levels=None,
    codes=None,
    sortorder=None,
    names=None,
    copy: bool = False,
    name=None,
    verify_integrity: bool = True
) -> Self from pandas.core.indexes.multi.MultiIndex
    Create and return a new object. See help(type) for accurate signature.
-----
Readonly properties defined here:

array
    Raises a ValueError for `MultiIndex` because there's no single
    array backing a MultiIndex.

    Raises
    -----
    ValueError

codes
    Codes of the MultiIndex.

    Codes are the position of the index value in the list of level values
    for each level.

Returns
-----
tuple of numpy.ndarray

```

The codes of the MultiIndex. Each array in the tuple corresponds to a level in the MultiIndex.

See Also

MultiIndex.set_codes : Set new codes on MultiIndex.

Examples

>>> arrays = [[1, 1, 2, 2], ["red", "blue", "red", "blue"]]
>>> mi = pd.MultiIndex.from_arrays(arrays, names=("number", "color"))
>>> mi.codes
FrozenList([[0, 0, 1, 1], [1, 0, 1, 0]])

levshape

A tuple representing the length of each level in the MultiIndex.

In a `MultiIndex`, each level can contain multiple unique values. The `levshape` property provides a quick way to assess the size of each level by returning a tuple where each entry represents the number of unique values in that specific level. This is particularly useful in scenarios where you need to understand the structure and distribution of your index levels, such as when working with multidimensional data.

See Also

MultiIndex.shape : Return a tuple of the shape of the MultiIndex.
MultiIndex.levels : Returns the levels of the MultiIndex.

Examples

>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"], ["c"]])
>>> mi
MultiIndex([('a', 'b', 'c')],
)
>>> mi.levshape
(1, 1, 1)

nlevels

Integer number of levels in this MultiIndex.

See Also

MultiIndex.levels : Get the levels of the MultiIndex.
MultiIndex.codes : Get the codes of the MultiIndex.
MultiIndex.from_arrays : Convert arrays to MultiIndex.
MultiIndex.from_tuples : Convert list of tuples to MultiIndex.

Examples

>>> mi = pd.MultiIndex.from_arrays([["a"], ["b"], ["c"]])
>>> mi
MultiIndex([('a', 'b', 'c')],
)
>>> mi.nlevels
3

size

Return the number of elements in the underlying data.

values

Return an array representing the data in the Index.

.. warning::

We recommend using `:attr: `Index.array`` or `:meth: `Index.to_numpy``, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

`Index.array` : Reference to the underlying data.

`Index.to_numpy` : A NumPy array representing the underlying data.

Examples

For `:class:`pandas.Index``:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For `:class:`pandas.IntervalIndex``:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors defined here:

`dtype`

`dtypes`

Return the dtypes as a Series for the underlying MultiIndex.

See Also

`Index.dtype` : Return the dtype object of the underlying data.

`Series.dtypes` : Return the data type of the underlying Series.

Examples

```
>>> idx = pd.MultiIndex.from_product(
...     [(0, 1, 2), ("green", "purple")], names=["number", "color"]
... )
>>> idx
MultiIndex([(0, 'green'),
            (0, 'purple'),
            (1, 'green'),
            (1, 'purple'),
            (2, 'green'),
            (2, 'purple')],
           names=['number', 'color'])
>>> idx.dtypes
number    int64
color     object
dtype: object
```

`inferred_type`

`is_monotonic_decreasing`

Return a boolean if the values are equal or decreasing.

`is_monotonic_increasing`

Return a boolean if the values are equal or increasing.

`levels`

Levels of the MultiIndex.

Levels refer to the different hierarchical levels or layers in a MultiIndex. In a MultiIndex, each level represents a distinct dimension or category of the index.

To access the levels, you can use the `levels` attribute of the MultiIndex, which returns a tuple of Index objects. Each Index object represents a level in the MultiIndex and contains the unique values found in that specific level.

If a MultiIndex is created with levels A, B, C, and the DataFrame using it filters out all rows of the level C, `MultiIndex.levels` will still return A, B, C.

See Also

MultiIndex.codes : The codes of the levels in the MultiIndex.
MultiIndex.get_level_values : Return vector of label values for requested level.

Examples

>>> index = pd.MultiIndex.from_product(
... ["mammal"], ("goat", "human", "cat", "dog")),
... names=["Category", "Animals"],
...)
>>> leg_num = pd.DataFrame(data=(4, 2, 4, 4), index=index, columns=["Legs"])
>>> leg_num

		Legs
Category	Animals	
mammal	goat	4
	human	2
	cat	4
	dog	4

>>> leg_num.index.levels
FrozenList(['mammal'], ['cat', 'dog', 'goat', 'human'])

MultiIndex levels will not change even if the DataFrame using the MultiIndex does not contain all them anymore.
See how "human" is not in the DataFrame, but it is still in levels:

>>> large_leg_num = leg_num[leg_num.Legs > 2]
>>> large_leg_num

		Legs
Category	Animals	
mammal	goat	4
	cat	4
	dog	4

>>> large_leg_num.index.levels
FrozenList(['mammal'], ['cat', 'dog', 'goat', 'human'])

names

Names of levels in MultiIndex.

This attribute provides access to the names of the levels in a `MultiIndex`. The names are stored as a `FrozenList`, which is an immutable list-like container. Each name corresponds to a level in the `MultiIndex`, and can be used to identify or manipulate the levels individually.

See Also

MultiIndex.set_names : Set Index or MultiIndex name.
MultiIndex.rename : Rename specific levels in a MultiIndex.
Index.names : Get names on index.

Examples

>>> mi = pd.MultiIndex.from_arrays(
... [[1, 2], [3, 4], [5, 6]], names=['x', 'y', 'z']
...)
>>> mi
MultiIndex([(1, 3, 5),
 (2, 4, 6)],
 names=['x', 'y', 'z'])
>>> mi.names
FrozenList(['x', 'y', 'z'])

nbytes

return the number of bytes in the underlying data

Data and other attributes defined here:

__annotations__ = {'_names': 'list[Hashable | None]', 'sortorder': 'in...

Methods inherited from Index:

```

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index
__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
    Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
    Parameters
    -----
    memo, default None
        Standard signature. Unused

__repr__(self) -> str_t from pandas.core.indexes.base.Index
    Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether all elements are Truthy.

    Parameters
    -----
    *args
        Required for compatibility with numpy.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    bool or array-like (if axis is specified)
        A single element array-like may be converted to bool.

    See Also
    -----
    Index.any : Return whether any element in an Index is True.
    Series.any : Return whether any element in a Series is True.
    Series.all : Return whether all elements in a Series are True.

    Notes
    -----
    Not a Number (NaN), positive infinity and negative infinity
    evaluate to True because these are not equal to zero.

    Examples
    -----
    True, because nonzero integers are considered True.

    >>> pd.Index([1, 2, 3]).all()
    True

    False, because ``0`` is considered False.

    >>> pd.Index([0, 1, 2]).all()
    False

any(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether any element is Truthy.

    Parameters
    -----
    *args
        Required for compatibility with numpy.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    bool or array-like (if axis is specified)
        A single element array-like may be converted to bool.

    See Also
    -----
    Index.all : Return whether all elements are True.
    Series.all : Return whether all elements are True.

```


Notes

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
>>> index.any()
True
```

```
>>> index = pd.Index([0, 0, 0])
>>> index.any()
False
```

`argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int` from `pandas`
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations,
the first row position is returned.

Parameters

`axis` : None

Unused. Parameter needed for compatibility with DataFrame.

`skipna` : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
and there is an NA value, this method will raise a ``ValueError``.

`*args, **kwargs`

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the maximum value.

See Also

`Series.argmax` : Return position of the maximum value.

`Series.argmin` : Return position of the minimum value.

`numpy.ndarray.argmax` : Equivalent method for numpy arrays.

`Series.idxmax` : Return index label of the maximum values.

`Series.idxmin` : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
the minimum cereal calories is the first element,
since index is zero-indexed.

`argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int` from `pandas`
Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations,
the first row position is returned.

Parameters

`axis` : None

Unused. Parameter needed for compatibility with DataFrame.

`skipna` : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
and there is an NA value, this method will raise a ``ValueError``.

`*args, **kwargs`

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the minimum value.

See Also

Series.argmax : Return position of the minimum value.

Series.argmax : Return position of the maximum value.

numpy.ndarray.argmax : Equivalent method for numpy arrays.

Series.idxmin : Return index label of the minimum values.

Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')
```

```
>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

asof(self, label) from pandas.core.indexes.base.Index

Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

label : object

The label up to which the method returns the latest index label.

Returns

object

The passed label if it is in the index. The previous label if the passed label is not in the sorted index or `NaN` if there is no such label.

See Also

Series.asof : Return the latest value in a Series up to the passed index.

merge_asof : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).

Index.get_loc : An `asof` is a thin wrapper around `get\_loc` with method='pad'.

Examples

`Index.asof` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
>>> idx.asof("2014-01-01")
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label, NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp]` from `pandas`
Return the locations (indices) of labels in the index.

As in the `:meth:`pandas.Index.asof``, if the label (a particular entry in ``where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in ``where``, -1 is returned.

``mask`` is used to ignore ``NA`` values in the index during calculation.

Parameters

`where` : Index
An Index consisting of an array of timestamps.
`mask` : `np.ndarray[bool]`
Array of booleans denoting where values in the original data are not ``NA``.

Returns

`np.ndarray[np.intp]`
An array of locations (indices) of the labels from the index which correspond to the return values of `:meth:`pandas.Index.asof`` for every element in ``where``.

See Also

`Index.asof` : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use ``mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`diff(self, periods: int = 1) -> Index` from `pandas.core.indexes.base.Index`
Computes the difference between consecutive values in the Index object.

If `periods` is greater than 1, computes the difference between values that are ``periods`` number of positions apart.

Parameters

`periods` : int, optional
The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

Index
A new Index object with the computed differences.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
```

```

>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')

```

difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index

Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters

other : Index or array-like
Index object or an array-like object containing elements to be compared with the elements of the original Index.

sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any TypeError from incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any TypeErrors from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index
Returns a new Index object containing elements that are in the original Index but not in the `other` Index.

See Also

Index.symmetric_difference : Compute the symmetric difference of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Examples

```

>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')

```

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index

Return Index with duplicate values removed.

Parameters

keep : {'first', 'last', ``False``}, default 'first'
- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

Returns

Index
A new Index object with the duplicate values removed.

See Also

Series.drop_duplicates : Equivalent method on Series.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Index.duplicated : Related method on Index, indicating duplicate Index values.

Examples

Generate a pandas.Index with duplicate values.

```

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])

```

The `keep` parameter controls which duplicate values are removed. The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of keep is 'first'.

```

>>> idx.drop_duplicates(keep="first")

```

```
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')
```

The value 'last' keeps the last occurrence for each set of duplicated entries.

```
>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')
```

The value ``False`` discards all sets of duplicated entries.

```
>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')
```

`droplevel(self, level: IndexLabel = 0)` from `pandas.core.indexes.base.Index`
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0
If a string is given, must be the name of a level
If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index after removing the requested level(s).

See Also

`Index.dropna` : Return Index without NA/Nan values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
             (2, 4, 6)],
            names=['x', 'y', 'z'])
```

```
>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
            names=['y', 'z'])
```

```
>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])
```

```
>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])
```

```
>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')
```

`get_indexer(`
self,
target,
method: ReindexMethod | None = None,
limit: int | None = None,
tolerance=None
) -> npt.NDArray[np.intp] from `pandas.core.indexes.base.Index`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

target : Index
 An iterable containing the values to be used for computing indexer.
 method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
 * default: exact matches only.
 * pad / ffill: find the PREVIOUS index value if no exact match.
 * backfill / bfill: use NEXT index value if no exact match
 * nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.
 limit : int, optional
 Maximum number of consecutive labels in ``target`` to match for inexact matches.
 tolerance : optional
 Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

 Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

 np.ndarray[np.intp]
 Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

See Also

 Index.get_indexer_for : Returns an indexer even when non-unique.
 Index.get_non_unique : Returns indexer and masks for new index given the current index.

Notes

 Returns -1 for unmatched values, for further explanation see the example below.

Examples

 >>> index = pd.Index(["c", "a", "b"])
 >>> index.get_indexer(["a", "b", "x"])
 array([1, 2, -1])

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

get_indexer_for(self, target) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
 Guaranteed return of an indexer even when non-unique.

This dispatches to get_indexer or get_indexer_non_unique as appropriate.

Parameters

 target : Index
 An iterable containing the values to be used for computing indexer.

Returns

 np.ndarray[np.intp]
 List of indices.

See Also

 Index.get_indexer : Computes indexer and mask for new index given the current index.
 Index.get_non_unique : Returns indexer and masks for new index given the current index.

Examples

 >>> idx = pd.Index([np.nan, "var1", np.nan])
 >>> idx.get_indexer_for([np.nan])
 array([0, 2])

```
get_indexer_non_unique(self, target) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]] from pandas.core.indexes.base.Index
    Compute indexer and mask for new index given the current index.
```

The indexer should be then used as an input to ndarray.take to align the current data to the new index.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.

Returns

indexer : np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.
missing : np.ndarray[np.intp]
An indexer into the target of the values not found. These correspond to the -1 in the indexer array.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned ``indexer`` contains only integers equal to -1. It demonstrates that there's no match between the index and the ``target`` values at these positions. The mask [0, 1, 2] in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in ``index``. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

```
groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
    Group the index labels by a given array of values.
```

Parameters

values : array
Values used to determine the groups.

Returns

dict
{group name -> group labels}

```
identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.
```

Parameters

other : Index
The Index object you want to compare with the current Index object.

Returns

bool

If two Index objects have equal elements and same type True, otherwise False.

See Also

Index.equals: Determine if two Index object are equal.

Index.has_duplicates: Check if the Index has duplicate values.

Index.is_unique: Return if the index has unique values.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
```

```
>>> idx2 = pd.Index(["1", "2", "3"])
```

```
>>> idx2.identical(idx1)
```

```
True
```

```
>>> idx1 = pd.Index(["1", "2", "3"], name="A")
```

```
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
```

```
>>> idx2.identical(idx1)
```

```
False
```

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index

If we have an object dtype, try to infer a non-object dtype.

Parameters

copy : bool, default True

Whether to make a copy in cases where no inference occurs.

Returns

Index

An Index with a new dtype if the dtype was inferred or a shallow copy if the dtype could not be inferred.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
>>> pd.Index(["a", 1]).infer_objects()
```

```
Index(['a', 1], dtype='object')
```

```
>>> pd.Index([1, 2], dtype="object").infer_objects()
```

```
Index([1, 2], dtype='int64')
```

intersection(self, other, sort: bool = False) from pandas.core.indexes.base.Index

Form the intersection of two Index objects.

This returns a new Index with elements common to the index and `other`.

Parameters

other : Index or array-like

An Index or an array-like object containing elements to form the intersection with the original Index.

sort : True, False or None, default False

Whether to sort the resulting index.

* None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.

* False : do not sort the result.

* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples


```

-----
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.intersection(idx2)
Index([3, 4], dtype='int64')

is_(self, other) -> bool from pandas.core.indexes.base.Index
More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks
that metadata is also the same.

Parameters
-----
other : object
    Other object to compare against.

Returns
-----
bool
    True if both have same underlying data, False otherwise.

See Also
-----
Index.identical : Works like ``Index.is_`` but also checks metadata.

Examples
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx1.is_(idx1.view())
True

>>> idx1.is_(idx1.copy())
False

isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect missing values.

Return a boolean same-sized object indicating if the values are NA.
NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get
mapped to ``True`` values.
Everything else get mapped to ``False`` values. Characters such as
empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns
-----
numpy.ndarray[bool]
    A boolean array of whether my values are NA.

See Also
-----
Index.notna : Boolean inverse of isna.
Index.dropna : Omit entries with missing values.
isna : Top-level isna.
Series.isna : Detect missing values in Series object.

Examples
-----
Show which entries in a pandas.Index are NA. The result is an
array.

>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])

Empty strings are not considered NA values. None is considered an NA
value.

>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```

```

>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])

isnull = isna(self) -> npt.NDArray[np.bool_]

join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas.core.indexes.base
Compute join_index and indexers to conform data structures to the new index.

Parameters
-----
other : Index
    The other index on which join is performed.
how : {'left', 'right', 'inner', 'outer'}
    It is either the integer position or the name of the level.
level : int or level name, default None
    Whether to return the indexers or not for both the index objects.
return_indexers : bool, default False
    Whether to return the indexers or not for both the index objects.
sort : bool, default False
    Sort the join keys lexicographically in the result Index. If False,
    the order of the join keys depends on the join type (how keyword).

Returns
-----
join_index, (left_indexer, right_indexer)
    The new index.

See Also
-----
DataFrame.join : Join columns with `other` DataFrame either on index
    or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a
    database-style join.

Examples
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base
Map values using an input mapping or function.

Parameters
-----
mapper : function, dict, or Series
    Mapping correspondence.
na_action : {None, 'ignore'}
    If 'ignore', propagate NA values, without passing them to the
    mapping correspondence.

Returns
-----
Union[Index, MultiIndex]
    The output of the mapping function applied to the index.
    If the function returns a tuple with more than one element
    a MultiIndex will be returned.

See Also

```

Index.where : Replace values where the condition is False.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')

Using `map` with a function:

>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')

max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base
Return the maximum value of the Index.

Parameters

axis : int, optional
For compatibility with NumPy. Only 0 or None are allowed.
skipna : bool, default True
Exclude NA/null values when showing the result.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Maximum value.

See Also

Index.min : Return the minimum value in an Index.
Series.max : Return the maximum value in a Series.
DataFrame.max : Return the maximum values in a DataFrame.

Examples

>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'

For a MultiIndex, the maximum is determined lexicographically.

>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)

min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base
Return the minimum value of the Index.

Parameters

axis : {None}
Dummy argument for consistency with Series.
skipna : bool, default True
Exclude NA/null values when showing the result.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Minimum value.

See Also

Index.max : Return the maximum value of the object.
Series.min : Return the minimum value in a Series.
DataFrame.min : Return the minimum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'
```

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA.
Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.
NA values, such as None or :attr:`numpy.NaN`, get mapped to ``False`` values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

notnull = notna(self) -> npt.NDArray[np.bool_]

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not implemented currently.

Returns

Index
A view on self.

See Also

```

-----
numpy.ndarray.ravel : Return a flattened array.

Examples
-----
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')

reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.

Parameters
-----
target : an iterable
    An iterable containing the values to be used for creating the new index.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
    * default: exact matches only.
    * pad / ffill: find the PREVIOUS index value if no exact match.
    * backfill / bfill: use NEXT index value if no exact match
    * nearest: use the NEAREST index value if no exact match. Tied
      distances are broken by preferring the larger index value.
level : int, optional
    Level of multiindex.
limit : int, optional
    Maximum number of consecutive labels in ``target`` to match for
    inexact matches.
tolerance : int, float, or list-like, optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

    Tolerance may be a scalar value, which applies the same tolerance
    to all values, or list-like, which applies variable tolerance per
    element. List-like includes list, tuple, array, Series, and must be
    the same size as the index and its dtype must exactly match the
    index's type.

Returns
-----
new_index : pd.Index
    Resulting index.
indexer : np.ndarray[np.intp] or None
    Indices of output values in original index.

Raises
-----
TypeError
    If ``method`` passed along with ``level``.
ValueError
    If non-unique multi-index
ValueError
    If non-unique index and ``method`` or ``limit`` passed.

See Also
-----
Series.reindex : Conform Series to new index with optional filling logic.
DataFrame.reindex : Conform DataFrame to new index with optional filling logic.

Examples
-----
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))

round(self, decimals: int = 0) -> Self from pandas.core.indexes.base.Index
Round each value in the Index to the given number of decimals.

```

Parameters

decimals : int, optional

Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

Returns

Index

A new Index with the rounded values.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10.1234, 20.5678, 30.9123, 40.4567, 50.7890])
>>> idx.round(decimals=2)
Index([10.12, 20.57, 30.91, 40.46, 50.79], dtype='float64')
```

set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core.
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

names : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

level : int, Hashable or a sequence of the previous, optional

If the index is a MultiIndex and names is not dict-like, level(s) to set
(None for all levels). Otherwise level must be None.

inplace : bool, default False

Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([( 'python', 2018),
            ( 'python', 2019),
            ( 'cobra', 2018),
            ( 'cobra', 2019)],
           )
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([( 'python', 2018),
            ( 'python', 2019),
            ( 'cobra', 2018),
            ( 'cobra', 2019)],
           names=['species', 'year'])
```

When renaming levels with a dict, levels can not be passed.

```
>>> idx.set_names({"kind": "snake"})
MultiIndex([( 'python', 2018),
            ( 'python', 2019),
            ( 'cobra', 2018),
            ( 'cobra', 2019)],
           names=['snake', 'year'])
```

```
shift(self, periods: int = 1, freq=None) -> Self from pandas.core.indexes.base.Index
Shift index by desired number of time frequency increments.
```

This method is for shifting the values of datetime-like indexes by a specified time increment a given number of times.

Parameters

periods : int, default 1
Number of periods (or increments) to shift by,
can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or str, optional
Frequency increment to shift by.
If None, the index is shifted by its own `freq` attribute.
Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

pandas.Index
Shifted index.

See Also

Series.shift : Shift values of Series.

Notes

This method is only implemented for datetime-like index classes,
i.e., DatetimeIndex, PeriodIndex and TimedeltaIndex.

Examples

Put the first 5 month starts of 2011 into an index.

```
>>> month_starts = pd.date_range("1/1/2011", periods=5, freq="MS")
>>> month_starts
DatetimeIndex(['2011-01-01', '2011-02-01', '2011-03-01', '2011-04-01',
               '2011-05-01'],
              dtype='datetime64[us]', freq='MS')
```

Shift the index by 10 days.

```
>>> month_starts.shift(10, freq="D")
DatetimeIndex(['2011-01-11', '2011-02-11', '2011-03-11', '2011-04-11',
               '2011-05-11'],
              dtype='datetime64[us]', freq=None)
```

The default value of `freq` is the `freq` attribute of the index,
which is 'MS' (month start) in this example.

```
>>> month_starts.shift(10)
DatetimeIndex(['2011-11-01', '2011-12-01', '2012-01-01', '2012-02-01',
               '2012-03-01'],
              dtype='datetime64[us]', freq='MS')
```

```
slice_indexer(
    self,
    start: Hashable | None = None,
    end: Hashable | None = None,
    step: int | None = None
) -> slice from pandas.core.indexes.base.Index
Compute the slice indexer for input labels and step.
```

Index needs to be ordered and unique.

Parameters

start : label, default None
If None, defaults to the beginning.
end : label, default None
If None, defaults to the end.
step : int, default None
If None, defaults to 1.

Returns

slice

A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)
```

```
sort_values(
    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

return_indexer : bool, default False
Should the indices that would sort the index be returned.
ascending : bool, default True
Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
key : callable, optional
If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.

Returns

sorted_index : pandas.Index
Sorted copy of the index.
indexer : numpy.ndarray, optional
The indices that the index itself was sorted by.

See Also

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')

Sort values in ascending order (default behavior).


```

>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')

Sort values in descending order, and also get the indices `idx` was
sorted by.

>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))

```

symmetric_difference

self,
other,
result_name: abc.Hashable | None = None,
sort: bool | None = None
) from pandas.core.indexes.base.Index

Compute the symmetric difference of two Index objects.

Parameters

other : Index or array-like
Index or an array-like object with elements to compute the symmetric difference with the original Index.

result_name : str
A string representing the name of the resulting Index, if desired.

sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

- * None : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.
- * False : Do not sort the result.
- * True : Sort the result (which may raise `TypeError`).

Returns

Index
Returns a new Index object containing elements that appear in either the original Index or the `other` Index, but not both.

See Also

Index.difference : Return a new Index with elements of index not in other.
Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes

``symmetric\_difference`` contains elements that appear in either ``idx1`` or ``idx2`` but not both. Equivalent to the Index created by ``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates dropped.

Examples

```

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')

```

to_series(self, index=None, name: Hashable | None = None) -> Series from pandas.core.indexes

Create a Series with both index and values equal to the index keys.

Useful with map for returning an indexer based on an index.

Parameters

index : Index, optional
Index of resulting Series. If None, defaults to original index.

name : str, optional
Name of resulting Series. If None, defaults to name of original index.

Returns

Series
The dtype will be based on the type of the Index values.

See Also

Index.to_frame : Convert an Index to a DataFrame.
Series.to_frame : Convert Series to DataFrame.

Examples

>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ``index``:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1     Bear
2     Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_series(name="zoo")
zoo
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

union(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index

Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to dtype('object') first.

Parameters

other : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.
sort : bool or None, default None
Whether to sort the resulting Index.

- * None : Sort the result, except when
 1. ``self`` and ``other`` are equal.
 2. ``self`` or ``other`` has length 0.
 3. Some values in ``self`` or ``other`` cannot be compared.A RuntimeWarning is issued in this case.
- * False : do not sort the result.
- * True : Sort the result (which may raise TypeError).

Returns

Index
Returns a new Index object with all unique elements from both the original Index and the ``other`` Index.

See Also

Index.unique : Return unique values in the index.
Index.intersection : Form the intersection of two Index objects.
Index.difference : Return a new Index with elements of index not in ``other``.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
```

```

>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')

Union mismatched dtypes

>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')

MultiIndex case

>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )

```

where(self, cond, other=None) -> Index from pandas.core.indexes.base.Index
 Replace values where the condition is False.

The replacement is taken from other.

Parameters

cond : bool array-like with the same length as self
 Condition to select the values on.
 other : scalar, or array-like, default None
 Replacement if the condition is False.

Returns

pandas.Index
 A copy of self with values replaced from other
 where the condition is False.

See Also

Series.where : Same method for Series.
 DataFrame.where : Same method for DataFrame.

Examples

```

>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx

```

```
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

has_duplicates

Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

Index.is_unique : Inverse method that checks if it has unique values.

Examples

>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True

>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
... "category"
...)
>>> idx.has_duplicates
True

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False

shape

Return a tuple of the shape of the underlying data.

See Also

Index.size: Return the number of elements in the underlying data.
Index.ndim: Number of dimensions of the underlying data, by definition 1.
Index.dtype: Return the dtype object of the underlying data.
Index.values: Return an array representing the data in the Index.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)

Data descriptors inherited from Index:

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

bool

See Also

Index.isna : Detect missing values.
Index.dropna : Return Index without NA/NaN values.
Index.fillna : Fill NA/NaN values with the specified value.

Examples

```

>>> s = pd.Series([1, 2, 3], index=["a", "b", None])
>>> s
a    1
b    2
None 3
dtype: int64
>>> s.index.hasnans
True

```

is_unique
Return if the index has unique values.

Returns

bool

See Also

Index.has_duplicates : Inverse method that checks if it has duplicate values.

Examples

```

>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.is_unique
False

>>> idx = pd.Index([1, 5, 7])
>>> idx.is_unique
True

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.is_unique
False

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.is_unique
True

```

name
Return Index or MultiIndex name.

Returns

label (hashable object)
The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.
Index.rename: Able to set new names partially and by level.
Series.name: Corresponding Series property.

Examples

```

>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx
Index([1, 2, 3], dtype='int64', name='x')
>>> idx.name
'x'

```

Data and other attributes inherited from Index:

```

__pandas_priority__ = 2000

```

str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method.
Patterned after Python's string methods, with some inspiration from
R's stringr package.

Parameters

data : Series or Index

The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.
Index.str : Vectorized string functions for Index.

Examples

>>> s = pd.Series(["A_Str_Series"])
>>> s
0 A_Str_Series
dtype: str

>>> s.str.split("_")
0 [A, Str, Series]
dtype: object

>>> s.str.replace("_", "")
0 AStrSeries
dtype: str

Methods inherited from pandas.core.base.IndexOpsMixin:

__iter__(self) -> Iterator
Return an iterator of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

iterator
An iterator yielding scalar values from the Series.

See Also

Series.items : Lazily iterate over (index, value) tuples.

Examples

>>> s = pd.Series([1, 2, 3])
>>> for x in s:
... print(x)
1
2
3

factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.intc], npt.NDArray[np.intc]]
Encode the object as an enumerated type or categorical variable.

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function `:func:`pandas.factorize``, and as a method `:meth:`Series.factorize`` and `:meth:`Index.factorize``.

Parameters

sort : bool, default False
Sort `uniques` and shuffle `codes` to maintain the relationship.
use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray
An integer ndarray that's an indexer into `uniques`.
`uniques.take(codes)` will have the same values as `values`.
uniques : ndarray, Index, or Categorical
The unique valid values. When `values` is Categorical, `uniques` is a Categorical. When `values` is some other pandas object, an

``Index`` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in ``values``, ``uniques`` will *not* contain an entry for it.

See Also

`cut` : Discretize continuous-valued array.
`unique` : Find the unique value in an array.

Notes

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

These examples all show `factorize` as a top-level method like ``pd.factorize(values)``. The results are identical for methods like `:meth:`Series.factorize``.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With ``sort=True``, the ``uniques`` will be sorted, and ``codes`` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use_na_sentinel=True`` (the default), missing values are indicated in the ``codes`` with the sentinel value ``-1`` and missing values are not included in ``uniques``.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of ``uniques`` will differ. For Categoricals, a ``Categorical`` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])
```

```
item(self)
    Return the first element of the underlying data as a Python scalar.

    Returns
    -----
    scalar
        The first element of Series or Index.

    Raises
    -----
    ValueError
        If the data is not length = 1.

    See Also
    -----
    Index.values : Returns an array representing the data in the Index.
    Series.head : Returns the first ``n`` rows.

    Examples
    -----
    >>> s = pd.Series([1])
    >>> s.item()
    1

    For an index:

    >>> s = pd.Series([1], index=["a"])
    >>> s.index.item()
    'a'

nunique(self, dropna: bool = True) -> int
    Return number of unique elements in the object.

    Excludes NA values by default.

    Parameters
    -----
    dropna : bool, default True
        Don't include NaN in the count.

    Returns
    -----
    int
        An integer indicating the number of unique elements in the object.

    See Also
    -----
    DataFrame.nunique: Method nunique for DataFrame.
    Series.count: Count non-NA/null observations in the Series.

    Examples
    -----
    >>> s = pd.Series([1, 3, 5, 7, 7])
    >>> s
    0    1
    1    3
    2    5
    3    7
    4    7
    dtype: int64
```



```

>>> s.nunique()
4
searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted Index `self` such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    .. note::

        The Index must be monotonically sorted, otherwise
        wrong locations will likely be returned. Pandas does not
        check this for you.

Parameters
-----
value : array-like or scalar
    Values to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort `self` into ascending
    order. They are typically the result of ``np.argsort``.

Returns
-----
int or array of int
    A scalar or array of insertion points with the
    same shape as `value`.

See Also
-----
sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes
-----
Binary search is used to find the required insertion points.

Examples
-----
>>> ser = pd.Series([1, 2, 3])
>>> ser
0    1
1    2
2    3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0    2000-03-11
1    2000-03-12
2    2000-03-13
dtype: datetime64[us]

```

```

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
... )
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']

>>> ser.searchsorted("bread")
np.int64(1)

>>> ser.searchsorted(["bread"], side="right")
array([3])

If the values are not monotonically sorted, wrong locations
may be returned:

>>> ser = pd.Series([2, 1, 3])
>>> ser
0    2
1    1
2    3
dtype: int64

>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1

```

```
to_list = tolist(self) -> list
```

```

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>,
    **kwargs
) -> np.ndarray

```

A NumPy ndarray representing the values in this Series or Index.

Parameters

```

dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
**kwargs
    Additional keywords passed through to the ``to_numpy`` method
    of the underlying array (for extension arrays).

```

Returns

```

numpy.ndarray
    The NumPy ndarray holding the values from this Series or Index.
    The dtype of the array may differ. See Notes.

```

See Also

```

Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

```

Notes

The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` *may* require copying data and coercing the result to a NumPy type (possibly object), which may be expensive. When you need a no-copy reference to the underlying data, :attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of ``to\_numpy()`` for various dtypes within pandas.

dtype	array type
category[T]	ndarray[T] (same dtype as input)
period	ndarray[object] (Periods)
interval	ndarray[object] (Intervals)
IntegerNA	ndarray[object]
datetime64[ns]	datetime64[ns]
datetime64[ns, tz]	ndarray[object] (Timestamps)

Examples

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)
```

Specify the `dtype` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

```
>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

`tolist(self)` -> list
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list
List containing the values as Python or pandas scalars.

See Also

`numpy.ndarray.tolist` : Return the array as an a.ndim-levels deep nested list of Python scalars.

Examples

For Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
```

```

>>> idx
Index([1, 2, 3], dtype='int64')

>>> idx.to_list()
[1, 2, 3]

transpose(self, *args, **kwargs) -> Self
    Return the transpose, which is by definition self.

Returns
-----
%(klass)s

value_counts(
    self,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    bins=None,
    dropna: bool = True
) -> Series
    Return a Series containing counts of unique values.

    The resulting object will be in descending order so that the
    first element is the most frequently-occurring element.
    Excludes NA values by default.

Parameters
-----
normalize : bool, default False
    If True then the object returned will contain the relative
    frequencies of the unique values.
sort : bool, default True
    Stable sort by frequencies when True. Preserve the order of the data
    when False.

    .. versionchanged:: 3.0.0

    Prior to 3.0.0, the sort was unstable.
ascending : bool, default False
    Sort in ascending order.
bins : int, optional
    Rather than count values, group them into half-open bins,
    a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
    Don't include counts of NaN.

Returns
-----
Series
    Series containing counts of unique values.

See Also
-----
Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples
-----
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by
dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2

```

```
Name: proportion, dtype: float64
```

```
**bins**
```

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]      2
(3.0, 4.0]      1
Name: count, dtype: int64
```

```
**dropna**
```

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64
```

```
**Categorical Dtypes**
```

Rows with categorical type will be counted as one group if they have same categories and order.
In the example below, even though `''a''`, `''c''`, and `''d''` all have the same data types of `''category''`, only `''c''` and `''d''` will be counted as one group since `''a''` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64
```

Readonly properties inherited from `pandas.core.base.IndexOpsMixin`:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
```

```
1    Bear
2    Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.

Series.shape: Return a tuple of the shape of the underlying data.

Series.dtype: Return the dtype object of the underlying data.

Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

```

-----
Data and other attributes inherited from pandas.core.base.IndexOpsMixin:
__array_priority__ = 1000
-----

Methods inherited from pandas.core.arraylike.OpsMixin:

__and__(self, other)

__eq__(self, other)
    Return self==value.

__ge__(self, other)
    Return self>=value.

__gt__(self, other)
    Return self>value.

__le__(self, other)
    Return self<=value.

__lt__(self, other)
    Return self<value.

__ne__(self, other)
    Return self!=value.

__or__(self, other)
    Return self|value.

__rand__(self, other)

__ror__(self, other)
    Return value|self.

__rxor__(self, other)

__xor__(self, other)
-----

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object
-----

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None
-----

Methods inherited from pandas.core.base.PandasObject:

__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values
-----

Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]
    Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.

```

```

class NamedAgg(builtins.object)
|     NamedAgg(
|         column: Hashable,
|         aggfunc: Callable[..., Any] | str,
|         *args: Any,

```

```
    **kwargs: Any
) -> None
```

Helper for column specific aggregation with control over output column names.

Parameters

```
column : Hashable
    Column label in the DataFrame to apply aggfunc.
aggfunc : function or str
    Function to apply to the provided column. If string, the name of a built-in
    pandas function.
*args, **kwargs : Any
    Optional positional and keyword arguments passed to ``aggfunc``.
```

See Also

DataFrame.groupby : Group DataFrame using a mapper or by a Series of columns.

Examples

```
>>> df = pd.DataFrame({"key": [1, 1, 2], "a": [-1, 0, 1], 1: [10, 11, 12]})
>>> agg_a = pd.NamedAgg(column="a", aggfunc="min")
>>> agg_1 = pd.NamedAgg(column=1, aggfunc=lambda x: np.mean(x))
>>> df.groupby("key").agg(result_a=agg_a, result_1=agg_1)
```

```
   result_a  result_1
key
1         -1       10.5
2          1       12.0
```

```
>>> def n_between(ser, low, high, **kwargs):
...     return ser.between(low, high, **kwargs).sum()
```

```
>>> agg_between = pd.NamedAgg("a", n_between, 0, 1)
>>> df.groupby("key").agg(count_between=agg_between)
```

```
   count_between
key
1                1
2                1
```

```
>>> agg_between_kw = pd.NamedAgg("a", n_between, 0, 1, inclusive="both")
>>> df.groupby("key").agg(count_between_kw=agg_between_kw)
```

```
   count_between_kw
key
1                   1
2                   1
```

Methods defined here:

```
__eq__(self, other) from pandas.core.groupby.generic.NamedAgg
    Return self==value.
```

```
__getitem__(self, key: int) -> Any from pandas.core.groupby.generic.NamedAgg
    Provide backward-compatible tuple-style access.
```

```
__init__(
    self,
    column: Hashable,
    aggfunc: Callable[..., Any] | str,
    *args: Any,
    **kwargs: Any
) -> None from pandas.core.groupby.generic.NamedAgg
    Initialize self. See help(type(self)) for accurate signature.
```

```
__replace__ = _replace(self, /, **changes) from dataclasses
```

```
__repr__(self) from pandas.core.groupby.generic.NamedAgg
    Return repr(self).
```

Data descriptors defined here:

```
__dict__
    dictionary for instance variables
```

```
__weakref__
    list of weak references to the object
```



```

-----
Data and other attributes defined here:
__annotations__ = {'aggfunc': 'AggScalar', 'args': 'tuple[Any, ...]', ...
__dataclass_fields__ = {'aggfunc': Field(name='aggfunc',type='AggScala...
__dataclass_params__ = _DataclassParams(init=True,repr=True,eq=True,or...
__hash__ = None
__match_args__ = ('column', 'aggfunc', 'args', 'kwargs')
args = ()

class Period(pandas._libs.tslibs.period._Period)
    Period(
        value=None,
        freq=None,
        ordinal=None,
        year=None,
        month=None,
        quarter=None,
        day=None,
        hour=None,
        minute=None,
        second=None
    )

    Represents a period of time.

    A `Period` represents a specific time span rather than a point in time.
    Unlike `Timestamp`, which represents a single instant, a `Period` defines a
    duration, such as a month, quarter, or year. The exact representation is
    determined by the `freq` parameter.

    Parameters
    -----
    value : Period, str, datetime, date or pandas.Timestamp, default None
        The time period represented (e.g., '4Q2005'). This represents neither
        the start or the end of the period, but rather the entire period itself.
    freq : str, default None
        One of pandas period strings or corresponding objects. Accepted
        strings are listed in the
        :ref:`period alias section <timeseries.period_aliases>` in the user docs.
        If value is datetime, freq is required.
    ordinal : int, default None
        The period offset from the proleptic Gregorian epoch.
    year : int, default None
        Year value of the period.
    month : int, default 1
        Month value of the period.
    quarter : int, default None
        Quarter value of the period.
    day : int, default 1
        Day value of the period.
    hour : int, default 0
        Hour value of the period.
    minute : int, default 0
        Minute value of the period.
    second : int, default 0
        Second value of the period.

    See Also
    -----
    Timestamp : Pandas replacement for python datetime.datetime object.
    date_range : Return a fixed frequency DatetimeIndex.
    timedelta_range : Generates a fixed frequency range of timedeltas.

    Examples
    -----
    >>> period = pd.Period('2012-1-1', freq='D')
    >>> period
    Period('2012-01-01', 'D')

    Method resolution order:

```

```
Period
pandas._libs.tslibs.period._Period
pandas._libs.tslibs.period.PeriodMixin
builtins.object
```

Static methods defined here:

```
__new__(
    cls,
    value=None,
    freq=None,
    ordinal=None,
    year=None,
    month=None,
    quarter=None,
    day=None,
    hour=None,
    minute=None,
    second=None
)
```

Data descriptors defined here:

```
__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object
```

Methods inherited from pandas._libs.tslibs.period._Period:

```
__add__(self, object, /)

__eq__(self, value, /)
    Return self==value.

__ge__(self, value, /)
    Return self>=value.

__gt__(self, value, /)
    Return self>value.

__hash__(self, /)
    Return hash(self).

__le__(self, value, /)
    Return self<=value.

__lt__(self, value, /)
    Return self<value.

__ne__(self, value, /)
    Return self!=value.

__radd__(self, object, /)

__reduce__(self)

__repr__(self, /)
    Return repr(self).

__rsub__(self, object, /)

__setstate__(self, state)

__str__(...)
    Return a string representation for a particular DataFrame

__sub__(self, object, /)

asfreq(self, freq, how='E') -> 'Period'
    Convert Period to desired frequency, at the start or end of the interval.

Parameters
-----
```

freq : str, DateOffset
The target frequency to convert the Period object to.
If a string is provided,
it must be a valid :ref:`period alias <timeseries.period\_aliases>`.

how : {'E', 'S', 'end', 'start'}, default 'end'
Specifies whether to align the period to the start or end of the interval:
- 'E' or 'end': Align to the end of the interval.
- 'S' or 'start': Align to the start of the interval.

Returns

Period : Period object with the specified frequency, aligned to the parameter.

See Also

Period.end_time : Return the end Timestamp.
Period.start_time : Return the start Timestamp.
Period.dayofyear : Return the day of the year.
Period.dayofweek : Return the day of the week.

Examples

Convert a daily period to an hourly period, aligning to the end of the day:

```
>>> period = pd.Period('2023-01-01', freq='D')
>>> period.asfreq('h')
Period('2023-01-01 23:00', 'h')
```

Convert a monthly period to a daily period, aligning to the start of the month:

```
>>> period = pd.Period('2023-01', freq='M')
>>> period.asfreq('D', how='start')
Period('2023-01-01', 'D')
```

Convert a yearly period to a monthly period, aligning to the last month:

```
>>> period = pd.Period('2023', freq='Y')
>>> period.asfreq('M', how='end')
Period('2023-12', 'M')
```

Convert a monthly period to an hourly period, aligning to the first day of the month:

```
>>> period = pd.Period('2023-01', freq='M')
>>> period.asfreq('h', how='start')
Period('2023-01-01 00:00', 'H')
```

Convert a weekly period to a daily period, aligning to the last day of the week:

```
>>> period = pd.Period('2023-08-01', freq='W')
>>> period.asfreq('D', how='end')
Period('2023-08-04', 'D')
```

strftime(self, fmt: str | None) -> str
Returns a formatted string representation of the :class:`Period`.

``fmt`` must be ``None`` or a string containing one or several directives.
When ``None``, the format will be determined from the frequency of the Period.
The method recognizes the same directives as the :func:`time.strftime`
function of the standard Python distribution, as well as the specific
additional directives ``%f``, ``%F``, ``%q``, ``%l``, ``%u``, ``%n``.
(formatting & docs originally from scikits.timeries).

Directive	Meaning	Notes
``%a``	Locale's abbreviated weekday name.	
``%A``	Locale's full weekday name.	
``%b``	Locale's abbreviated month name.	
``%B``	Locale's full month name.	

``%c``	Locale's appropriate date and time representation.	
``%d``	Day of the month as a decimal number [01,31].	
``%f``	'Fiscal' year without a century as a decimal number [00,99]	\(1)
``%F``	'Fiscal' year with a century as a decimal number	\(2)
``%H``	Hour (24-hour clock) as a decimal number [00,23].	
``%I``	Hour (12-hour clock) as a decimal number [01,12].	
``%j``	Day of the year as a decimal number [001,366].	
``%m``	Month as a decimal number [01,12].	
``%M``	Minute as a decimal number [00,59].	
``%p``	Locale's equivalent of either AM or PM.	\(3)
``%q``	Quarter as a decimal number [1,4]	
``%S``	Second as a decimal number [00,61].	\(4)
``%l``	Millisecond as a decimal number [000,999].	
``%u``	Microsecond as a decimal number [000000,999999].	
``%n``	Nanosecond as a decimal number [000000000,999999999].	
``%U``	Week number of the year (Sunday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Sunday are considered to be in week 0.	\(5)
``%w``	Weekday as a decimal number [0(Sunday),6].	
``%W``	Week number of the year (Monday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Monday are considered to be in week 0.	\(5)
``%x``	Locale's appropriate date representation.	
``%X``	Locale's appropriate time representation.	
``%y``	Year without century as a decimal number [00,99].	
``%Y``	Year with century as a decimal number.	

``%Z``	Time zone name (no characters		
	if no time zone exists).		
+-----+	+-----+	+-----+	+-----+
``%``	A literal ``%`` character.		
+-----+	+-----+	+-----+	+-----+

The `strftime` method provides a way to represent a `Period` object as a string in a specified format. This is particularly useful when displaying date and time data in different locales or customized formats, suitable for reports or user interfaces. It extends the standard Python string formatting capabilities with additional directives specific to `pandas`, accommodating features like fiscal years and precise sub-second components.

Parameters

`fmt` : str or None
String containing the desired format directives. If `None`, the format is determined based on the `Period`'s frequency.

See Also

`Timestamp.strftime` : Return a formatted string of the `Timestamp`.
`to_datetime` : Convert argument to `datetime`.
`time.strftime` : Format a time object as a string according to a specified format string in the standard Python library.

Notes

- (1) The ```%f``` directive is the same as ```%y``` if the frequency is not quarterly. Otherwise, it corresponds to the 'fiscal' year, as defined by the `:attr:qyear` attribute.
- (2) The ```%F``` directive is the same as ```%Y``` if the frequency is not quarterly. Otherwise, it corresponds to the 'fiscal' year, as defined by the `:attr:qyear` attribute.
- (3) The ```%p``` directive only affects the output hour field if the ```%I``` directive is used to parse the hour.
- (4) The range really is ```0``` to ```61```; this accounts for leap seconds and the (very rare) double leap seconds.
- (5) The ```%U``` and ```%W``` directives are only used in calculations when the day of the week and the year are specified.

Examples

```
>>> from pandas import Period
>>> a = Period(freq='Q-JUL', year=2006, quarter=1)
>>> a.strftime('%F-Q%q')
'2006-Q1'
>>> # Output the last month in the quarter of this date
>>> a.strftime('%b-%Y')
'Oct-2005'
>>>
>>> a = Period(freq='D', year=2001, month=1, day=1)
>>> a.strftime('%d-%b-%Y')
'01-Jan-2001'
>>> a.strftime('%b. %d, %Y was a %A')
'Jan. 01, 2001 was a Monday'
```

`to_timestamp(self, freq=None, how='start')` -> `Timestamp`
Return the `Timestamp` representation of the `Period`.

Uses the target frequency specified at the part of the period specified by `how`, which is either `Start` or `Finish`.

If possible, gives microsecond-unit Timestamp. Otherwise gives nanosecond unit.

Parameters

freq : str or DateOffset
 Target frequency. Default is 'D' if self._freq is week or longer and 'S' otherwise.
how : str, default 'S' (start)
 One of 'S', 'E'. Can be aliased as case insensitive 'Start', 'Finish', 'Begin', 'End'.

Returns

Timestamp

See Also

Timestamp : A class representing a single point in time.
Period : Represents a span of time with a fixed frequency.
PeriodIndex.to_timestamp : Convert a `PeriodIndex` to a `DatetimeIndex`.

Examples

>>> period = pd.Period('2023-1-1', freq='D')
>>> timestamp = period.to_timestamp()
>>> timestamp
Timestamp('2023-01-01 00:00:00')

Class methods inherited from pandas._libs.tslib.period._Period:

now(freq)
 Return the period of now's date.

The `now` method provides a convenient way to generate a period object for the current date and time. This can be particularly useful in financial and economic analysis, where data is often collected and analyzed in regular intervals (e.g., hourly, daily, monthly). By specifying the frequency, users can create periods that match the granularity of their data.

Parameters

freq : str, DateOffset
 Frequency to use for the returned period.

See Also

to_datetime : Convert argument to datetime.
Period : Represents a period of time.
Period.to_timestamp : Return the Timestamp representation of the Period.

Examples

>>> pd.Period.now('h') # doctest: +SKIP
Period('2023-06-12 11:00', 'h')

Data descriptors inherited from pandas._libs.tslib.period._Period:

day
 Get day of the month that a Period falls on.

The `day` property provides a simple way to access the day component of a `Period` object, which represents time spans in various frequencies (e.g., daily, hourly, monthly). If the period's frequency does not include a day component (e.g., yearly or quarterly periods), the returned day corresponds to the first day of that period.

Returns

int

See Also

Period.dayofweek : Get the day of the week.

`Period.dayofyear` : Get the day of the year.

Examples

```
>>> p = pd.Period("2018-03-11", freq='h')
>>> p.day
11
```

`day_of_week`

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

`Period.day_of_week` : Day of the week the period lies in.

`Period.weekday` : Alias of `Period.day_of_week`.

`Period.day` : Day of the month.

`Period.dayofyear` : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
>>> per.day_of_week
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
>>> per.day_of_week
6
>>> per.start_time.day_of_week
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
>>> per.day_of_week
2
>>> per.end_time.day_of_week
2
```

`day_of_year`

Return the day of the year.

This attribute returns the day of the year on which the particular date occurs. The return value ranges between 1 to 365 for regular years and 1 to 366 for leap years.

Returns

int

The day of year.

See Also

`Period.day` : Return the day of the month.

`Period.day_of_week` : Return the day of week.

`PeriodIndex.day_of_year` : Return the day of year of all indexes.

Examples

```
>>> period = pd.Period("2015-10-23", freq='h')
>>> period.day_of_year
296
```

```
>>> period = pd.Period("2012-12-31", freq='D')
>>> period.day_of_year
366
>>> period = pd.Period("2013-01-01", freq='D')
>>> period.day_of_year
1
```

dayofweek

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

Period.day_of_week : Day of the week the period lies in.

Period.weekday : Alias of Period.day_of_week.

Period.day : Day of the month.

Period.dayofyear : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
>>> per.day_of_week
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
>>> per.day_of_week
6
>>> per.start_time.day_of_week
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
>>> per.day_of_week
2
>>> per.end_time.day_of_week
2
```

dayofyear

Return the day of the year.

This attribute returns the day of the year on which the particular date occurs. The return value ranges between 1 to 365 for regular years and 1 to 366 for leap years.

Returns

int

The day of year.

See Also

Period.day : Return the day of the month.

Period.day_of_week : Return the day of week.

PeriodIndex.day_of_year : Return the day of year of all indexes.

Examples

```
>>> period = pd.Period("2015-10-23", freq='h')
>>> period.day_of_year
296
>>> period = pd.Period("2012-12-31", freq='D')
```



```
>>> period.day_of_year
366
>>> period = pd.Period("2013-01-01", freq='D')
>>> period.day_of_year
1
```

days_in_month

Get the total number of days in the month that this period falls on.

Returns

int

See Also

Period.daysinmonth : Gets the number of days in the month.
DatetimeIndex.daysinmonth : Gets the number of days in the month.
calendar.monthrange : Returns a tuple containing weekday
(0-6 ~ Mon-Sun) and number of days (28-31).

Examples

```
>>> p = pd.Period('2018-2-17')
>>> p.days_in_month
28
```

```
>>> pd.Period('2018-03-01').days_in_month
31
```

Handles the leap year case as well:

```
>>> p = pd.Period('2016-2-17')
>>> p.days_in_month
29
```

daysinmonth

Get the total number of days of the month that this period falls on.

Returns

int

See Also

Period.days_in_month : Return the days of the month.
Period.dayofyear : Return the day of the year.

Examples

```
>>> p = pd.Period("2018-03-11", freq='h')
>>> p.daysinmonth
31
```

freq

Return the frequency object for this Period.

The frequency object represents the span of time that this Period covers.
It is a DateOffset that defines the interval type (e.g., daily, monthly).

See Also

Period.freqstr : Return a string representation of the frequency.
Period.asfreq : Convert Period to desired frequency.

Examples

```
>>> period = pd.Period('2020-01', freq='M')
>>> period.freq
<MonthEnd>
```

freqstr

Return a string representation of the frequency.

This property provides the frequency string associated with the `Period` object. The frequency string describes the granularity of the time span represented by the `Period`. Common frequency strings include 'D' for daily, 'M' for monthly, 'Y' for yearly, etc.

See Also

Period.asfreq : Convert Period to desired frequency, at the start or end of the interval.
period_range : Return a fixed frequency PeriodIndex.

Examples

>>> pd.Period('2020-01', 'D').freqstr
'D'

hour

Get the hour of the day component of the Period.

Returns

int

The hour as an integer, between 0 and 23.

See Also

Period.second : Get the second component of the Period.
Period.minute : Get the minute component of the Period.

Examples

>>> p = pd.Period("2018-03-11 13:03:12.050000")
>>> p.hour
13

Period longer than a day

>>> p = pd.Period("2018-03-11", freq="M")
>>> p.hour
0

is_leap_year

Return True if the period's year is in a leap year.

See Also

Timestamp.is_leap_year : Check if the year in a Timestamp is a leap year.
DatetimeIndex.is_leap_year : Boolean indicator if the date belongs to a leap year.

Examples

>>> period = pd.Period('2022-01', 'M')
>>> period.is_leap_year
False

>>> period = pd.Period('2020-01', 'M')
>>> period.is_leap_year
True

minute

Get minute of the hour component of the Period.

Returns

int

The minute as an integer, between 0 and 59.

See Also

Period.hour : Get the hour component of the Period.
Period.second : Get the second component of the Period.

Examples

>>> p = pd.Period("2018-03-11 13:03:12.050000")
>>> p.minute
3

month

Return the month this Period falls on.

Returns

int

See Also

period.week : Get the week of the year on the given Period.

Period.year : Return the year this Period falls on.

Period.day : Return the day of the month this Period falls on.

Notes

The month is based on the `ordinal` and `base` attributes of the Period.

Examples

Create a Period object for January 2022 and get the month:

```
>>> period = pd.Period('2022-01', 'M')
```

```
>>> period.month
```

```
1
```

Period object with no specified frequency, resulting in a default frequency:

```
>>> period = pd.Period('2022', 'Y')
```

```
>>> period.month
```

```
12
```

Create a Period object with a specified frequency but an incomplete date string:

```
>>> period = pd.Period('2022', 'M')
```

```
>>> period.month
```

```
1
```

Handle a case where the Period object is empty, which results in `NaN`:

```
>>> period = pd.Period('nan', 'M')
```

```
>>> period.month
```

```
nan
```

ordinal

Return the integer ordinal for this Period.

The ordinal is the internal integer representation of the Period, representing its position in the sequence of periods of the given frequency. It counts from an epoch (e.g., for daily frequency, ordinal 0 corresponds to January 1, 1970).

See Also

Period.freq : Return the frequency of the Period.

Period.start_time : Return the start time of the Period.

Examples

```
>>> period = pd.Period('2020-01', freq='M')
```

```
>>> period.ordinal
```

```
600
```

```
>>> period = pd.Period('2020-01-01', freq='D')
```

```
>>> period.ordinal
```

```
18262
```

quarter

Return the quarter this Period falls on.

See Also

Timestamp.quarter : Return the quarter of the Timestamp.

Period.year : Return the year of the period.

Period.month : Return the month of the period.

Examples

```
>>> period = pd.Period('2022-04', 'M')
```

```
>>> period.quarter
```

2

qyear

Fiscal year the Period lies in according to its starting-quarter.

The `year` and the `qyear` of the period will be the same if the fiscal and calendar years are the same. When they are not, the fiscal year can be different from the calendar year of the period.

Returns

int

The fiscal year of the period.

See Also

Period.year : Return the calendar year of the period.

Examples

If the natural and fiscal year are the same, `qyear` and `year` will be the same.

```
>>> per = pd.Period('2018Q1', freq='Q')
>>> per.qyear
2018
>>> per.year
2018
```

If the fiscal year starts in April (`Q-MAR`), the first quarter of 2018 will start in April 2017. `year` will then be 2017, but `qyear` will be the fiscal year, 2018.

```
>>> per = pd.Period('2018Q1', freq='Q-MAR')
>>> per.start_time
Timestamp('2017-04-01 00:00:00')
>>> per.qyear
2018
>>> per.year
2017
```

second

Get the second component of the Period.

Returns

int

The second of the Period (ranges from 0 to 59).

See Also

Period.hour : Get the hour component of the Period.

Period.minute : Get the minute component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11 13:03:12.050000")
>>> p.second
12
```

week

Get the week of the year on the given Period.

Returns

int

See Also

Period.dayofweek : Get the day component of the Period.

Period.weekday : Get the day component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11", "h")
>>> p.week
10
```

```
>>> p = pd.Period("2018-02-01", "D")
>>> p.week
5
```

```
>>> p = pd.Period("2018-01-06", "D")
>>> p.week
1
```

weekday

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

Period.dayofweek : Day of the week the period lies in.

Period.weekday : Alias of Period.dayofweek.

Period.day : Day of the month.

Period.dayofyear : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
>>> per.dayofweek
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
>>> per.dayofweek
6
>>> per.start_time.dayofweek
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
>>> per.dayofweek
2
>>> per.end_time.dayofweek
2
```

weekofyear

Get the week of the year on the given Period.

Returns

int

See Also

Period.dayofweek : Get the day component of the Period.

Period.weekday : Get the day component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11", "h")
>>> p.weekofyear
10
```

```
>>> p = pd.Period("2018-02-01", "D")
>>> p.weekofyear
5
```

```
>>> p = pd.Period("2018-01-06", "D")
>>> p.weekofyear
1
```

year

Return the year this Period falls on.

Returns

int

See Also

period.month : Get the month of the year for the given Period.
period.day : Return the day of the month the Period falls on.

Notes

The year is based on the `ordinal` and `base` attributes of the Period.

Examples

Create a Period object for January 2023 and get the year:

```
>>> period = pd.Period('2023-01', 'M')
>>> period.year
2023
```

Create a Period object for 01 January 2023 and get the year:

```
>>> period = pd.Period('2023', 'D')
>>> period.year
2023
```

Get the year for a period representing a quarter:

```
>>> period = pd.Period('2023Q2', 'Q')
>>> period.year
2023
```

Handle a case where the Period object is empty, which results in `NaN`:

```
>>> period = pd.Period('nan', 'M')
>>> period.year
nan
```

Data and other attributes inherited from pandas._libs.tslibs.period._Period:

__array_priority__ = 100

Methods inherited from pandas._libs.tslibs.period.PeriodMixin:

__reduce_cython__(self)

__setstate_cython__(self, __pyx_state)

Data descriptors inherited from pandas._libs.tslibs.period.PeriodMixin:

end_time

Get the Timestamp for the end of the period.

Returns

Timestamp

See Also

Period.start_time : Return the start Timestamp.
Period.dayofyear : Return the day of year.
Period.daysinmonth : Return the days in that month.
Period.dayofweek : Return the day of the week.

Examples

For Period:

```
>>> pd.Period('2020-01', 'D').end_time
Timestamp('2020-01-01 23:59:59.999999')
```

For Series:

```
>>> period_index = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period_index)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.end_time
0    2020-01-31 23:59:59.999999
1    2020-02-29 23:59:59.999999
2    2020-03-31 23:59:59.999999
dtype: datetime64[us]
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.end_time
DatetimeIndex(['2023-01-31 23:59:59.999999',
               '2023-02-28 23:59:59.999999',
               '2023-03-31 23:59:59.999999'],
              dtype='datetime64[us]', freq=None)
```

start_time

Get the Timestamp for the start of the period.

Returns

Timestamp

See Also

Period.end_time : Return the end Timestamp.
Period.dayofyear : Return the day of year.
Period.daysinmonth : Return the days in that month.
Period.dayofweek : Return the day of the week.

Examples

```
>>> period = pd.Period('2012-1-1', freq='D')
>>> period
Period('2012-01-01', 'D')
```

```
>>> period.start_time
Timestamp('2012-01-01 00:00:00')
```

```
>>> period.end_time
Timestamp('2012-01-01 23:59:59.999999')
```

```
class PeriodDtype(pandas._libs.tslibs.dtypes.PeriodDtypeBase, pandas.core.dtypes.dtypes.PandasExtensionDtype):
    PeriodDtype(freq) -> PeriodDtype
```

An ExtensionDtype for Period data.

****This is not an actual numpy dtype**, but a duck type.**

Parameters

freq : str or DateOffset
The frequency of this PeriodDtype.

Attributes

freq

Methods

None

See Also

Period : Represents a single time period.
PeriodIndex : Immutable index for period data.
date_range : Return a fixed frequency DatetimeIndex.
Series : One-dimensional array with axis labels.
DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Examples

```
-----  
>>> pd.PeriodDtype(freq="D")  
period[D]  
  
>>> pd.PeriodDtype(freq=pd.offsets.MonthEnd())  
period[M]
```

Method resolution order:

```
PeriodDtype  
pandas._libs.tslibs.dtypes.PeriodDtypeBase  
pandas.core.dtypes.dtypes.PandasExtensionDtype  
pandas.api.extensions.ExtensionDtype  
builtins.object
```

Methods defined here:

```
__eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.PeriodDtype  
    Return self==value.  
  
__from_arrow__(self, array: pa.Array | pa.ChunkedArray) -> PeriodArray from pandas.core.dtypes.dtypes.PeriodArray  
    Construct PeriodArray from pyarrow Array/ChunkedArray.  
  
__hash__(self, /) from pandas._libs.tslibs.dtypes.PeriodDtypeBase  
    Return hash(self).  
  
__ne__(self, other: object) -> bool from pandas.core.dtypes.dtypes.PeriodDtype  
    Return self!=value.  
  
__reduce__(self) -> tuple[type_t[Self], tuple[str_type]] from pandas.core.dtypes.dtypes.PeriodDtype  
    Helper for pickle.  
  
__str__(self) -> str_type from pandas.core.dtypes.dtypes.PeriodDtype  
    Return str(self).  
  
construct_array_type(self) -> type_t[PeriodArray] from pandas.core.dtypes.dtypes.PeriodDtype  
    Return the array type associated with this dtype.  
  
    Returns  
    -----  
    type
```

Class methods defined here:

```
construct_from_string(string: str_type) -> PeriodDtype from pandas.core.dtypes.dtypes.PeriodDtype  
    Strict construction from a string, raise a TypeError if not  
    possible  
  
is_dtype(dtype: object) -> bool from pandas.core.dtypes.dtypes.PeriodDtype  
    Return a boolean if the passed type is an actual dtype that we  
    can match (via string or type)
```

Static methods defined here:

```
__new__(cls, freq) -> PeriodDtype from pandas.core.dtypes.dtypes.PeriodDtype  
    Parameters  
    -----  
    freq : PeriodDtype, BaseOffset, or string
```

Readonly properties defined here:

```
freq  
    The frequency object of this PeriodDtype.  
  
    The `freq` property returns the `BaseOffset` object that represents the  
    frequency of the PeriodDtype. This frequency specifies the interval (e.g.,  
    daily, monthly, yearly) associated with the Period type. It is essential  
    for operations that depend on time-based calculations within a period index
```


or series.

See Also

Period : Represents a period of time.
PeriodIndex : Immutable ndarray holding ordinal values indicating regular periods.
PeriodDtype : An ExtensionDtype for Period data.
date_range : Return a fixed frequency range of dates.

Examples

>>> dtype = pd.PeriodDtype(freq="D")
>>> dtype.freq
<Day>

na_value

Default NA value to use for this type.

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

name

A string identifying the data type.

Will be used for display in, e.g. ``Series.dtype``

Data descriptors defined here:

__dict__

dictionary for instance variables

__weakref__

list of weak references to the object

index_class

Data and other attributes defined here:

__annotations__ = {'_cache_dtypes': 'dict[BaseOffset, int]', '_freq': ...

base = dtype('O')

kind = 'O'

num = 102

str = '|008'

type = <class 'pandas.Period'>

Represents a period of time.

A `Period` represents a specific time span rather than a point in time. Unlike `Timestamp`, which represents a single instant, a `Period` defines a duration, such as a month, quarter, or year. The exact representation is determined by the `freq` parameter.

Parameters

value : Period, str, datetime, date or pandas.Timestamp, default None
The time period represented (e.g., '4Q2005'). This represents neither the start or the end of the period, but rather the entire period itself.
freq : str, default None
One of pandas period strings or corresponding objects. Accepted strings are listed in the :ref:`period alias section <timeseries.period\_aliases>` in the user docs. If value is datetime, freq is required.
ordinal : int, default None
The period offset from the proleptic Gregorian epoch.
year : int, default None
Year value of the period.
month : int, default 1
Month value of the period.
quarter : int, default None

Quarter value of the period.
day : int, default 1
Day value of the period.
hour : int, default 0
Hour value of the period.
minute : int, default 0
Minute value of the period.
second : int, default 0
Second value of the period.

See Also

Timestamp : Pandas replacement for python datetime.datetime object.
date_range : Return a fixed frequency DatetimeIndex.
timedelta_range : Generates a fixed frequency range of timedeltas.

Examples

>>> period = pd.Period('2012-1-1', freq='D')
>>> period
Period('2012-01-01', 'D')

Methods inherited from pandas._libs.tslib.dtypes.PeriodDtypeBase:

__ge__(self, value, /)
Return self>=value.

__gt__(self, value, /)
Return self>value.

__le__(self, value, /)
Return self<=value.

__lt__(self, value, /)
Return self<value.

__setstate__ = __setstate_cython__(...)

Data and other attributes inherited from pandas._libs.tslib.dtypes.PeriodDtypeBase:

__pyx_vtable__ = <capsule object NULL>

Methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

__getstate__(self) -> dict[str_type, Any]
Helper for pickle.

__repr__(self) -> str_type
Return a string representation for a particular object.

Class methods inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

reset_cache() -> None
clear the cache

Data and other attributes inherited from pandas.core.dtypes.dtypes.PandasExtensionDtype:

isbuiltin = 0

isnative = 0

itemsized = 8

shape = ()

subdtype = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype

```

        Construct an ExtensionArray of this dtype with the given shape.

        Analogous to numpy.empty.

        Parameters
        -----
        shape : int or tuple[int]

        Returns
        -----
        ExtensionArray

    -----
    Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

    names
        Ordered list of field names, or None if there are no fields.

        This is for compatibility with NumPy arrays, and may be removed in the
        future.
class PeriodIndex(pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin)
    PeriodIndex(
        data=None,
        freq=None,
        dtype: Dtype | None = None,
        copy: bool | None = None,
        name: Hashable | None = None
    ) -> Self

    Immutable ndarray holding ordinal values indicating regular periods in time.

    Index keys are boxed to Period objects which carries the metadata (eg,
    frequency information).

    Parameters
    -----
    data : array-like (1d int np.ndarray or PeriodArray), optional
        Optional period-like data to construct index with.
    freq : str or period object, optional
        One of pandas period strings or corresponding objects.
    dtype : str or PeriodDtype, default None
        A dtype from which to extract a freq.
    copy : bool, default None
        Whether to copy input data, only relevant for array, Series, and Index
        inputs (for other input, e.g. a list, a new array is created anyway).
        Defaults to True for array input and False for Index/Series.
        Set to False to avoid copying array input at your own risk (if you
        know the input data won't be modified elsewhere).
        Set to True to force copying Series/Index input up front.
    name : str, default None
        Name of the resulting PeriodIndex.

    Attributes
    -----
    day
    dayofweek
    day_of_week
    dayofyear
    day_of_year
    days_in_month
    daysinmonth
    end_time
    freq
    freqstr
    hour
    is_leap_year
    minute
    month
    quarter
    qyear
    second
    start_time
    week
    weekday
    weekofyear
    year

```

Methods

asfreq
strftime
to_timestamp
from_fields
from_ordinals

Raises

ValueError

Passing the parameter data as a list without specifying either freq or dtype will raise a ValueError: "freq not specified and cannot be inferred"

See Also

Index : The base pandas Index type.
Period : Represents a period of time.
DatetimeIndex : Index with datetime64 data.
TimedeltaIndex : Index of timedelta64 data.
period_range : Create a fixed-frequency PeriodIndex.

Examples

```
>>> idx = pd.PeriodIndex(data=["2000Q1", "2002Q3"], freq="Q")
>>> idx
PeriodIndex(['2000Q1', '2002Q3'], dtype='period[Q-DEC]')
```

Method resolution order:

PeriodIndex
pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin
pandas.core.indexes.extension.NDArrayBackedExtensionIndex
pandas.core.indexes.extension.ExtensionIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
abc.ABC
builtins.object

Methods defined here:

asfreq(self, freq=None, how: str = 'E') -> Self from pandas.core.indexes.period.PeriodIndex
Convert the PeriodArray to the specified frequency `freq`.

Equivalent to applying :meth:`pandas.Period.asfreq` with the given arguments to each :class:`~pandas.Period` in this PeriodArray.

Parameters

freq : str
A frequency.
how : str {'E', 'S'}, default 'E'
Whether the elements should be aligned to the end or start within a period.

* 'E', 'END', or 'FINISH' for end,
* 'S', 'START', or 'BEGIN' for start.

January 31st ('END') vs. January 1st ('START') for example.

Returns

PeriodArray

The transformed PeriodArray with the new frequency.

See Also

arrays.PeriodArray.asfreq: Convert each Period in a PeriodArray to the given frequency.
Period.asfreq : Convert a :class:`~pandas.Period` object to the given frequency.

Examples

```
>>> pidx = pd.period_range("2010-01-01", "2015-01-01", freq="Y")
>>> pidx
```

```

PeriodIndex(['2010', '2011', '2012', '2013', '2014', '2015'],
dtype='period[Y-DEC]')

>>> pidx.asfreq("M")
PeriodIndex(['2010-12', '2011-12', '2012-12', '2013-12', '2014-12',
'2015-12'], dtype='period[M]')

>>> pidx.asfreq("M", how="S")
PeriodIndex(['2010-01', '2011-01', '2012-01', '2013-01', '2014-01',
'2015-01'], dtype='period[M]')

asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> np.ndarray from pandas.core.index
where : array of timestamps
mask : np.ndarray[bool]
      Array of booleans where data is not NA.

get_loc(self, key) from pandas.core.indexes.period.PeriodIndex
Get integer location for requested label.

Parameters
-----
key : Period, NaT, str, or datetime
      String or datetime key must be parsable as Period.

Returns
-----
loc : int or ndarray[int64]

Raises
-----
KeyError
      Key is not present in the index.
TypeError
      If key is listlike or otherwise not hashable.

shift(self, periods: int = 1, freq=None) -> Self from pandas.core.indexes.period.PeriodIndex
Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes
by a specified time increment a given number of times.

Parameters
-----
periods : int, default 1
      Number of periods (or increments) to shift by,
      can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or string, optional
      Frequency increment to shift by.
      If None, the index is shifted by its own `freq` attribute.
      Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns
-----
pandas.DatetimeIndex
      Shifted index.

See Also
-----
Index.shift : Shift values of Index.
PeriodIndex.shift : Shift values of PeriodIndex.

strftime(self, date_format: str) -> npt.NDArray[np.object_] from pandas.core.indexes.extension
Convert to Index using specified date_format.

Return an Index of formatted strings specified by date_format, which
supports the same string format as the python standard library. Details
of the string format can be found in `python string format
doc <https://docs.python.org/3/library/datetime.html
#strftime-and-strptime-behavior>`.

Formats supported by the C `strftime` API but not by the python string format
doc (such as `"%R"`, `"%r"`) are not officially supported and should be
preferably replaced with their supported equivalents (such as `"%H:%M"`,
`"%I:%M:%S %p"`).

Note that `PeriodIndex` support additional directives, detailed in
`Period.strftime`.

```

Parameters

date_format : str
Date format string (e.g. "%Y-%m-%d").

Returns

ndarray[object]
NumPy ndarray of formatted strings.

See Also

to_datetime : Convert the given argument to datetime.
DatetimeIndex.normalize : Return DatetimeIndex with times to midnight.
DatetimeIndex.round : Round the DatetimeIndex to the specified freq.
DatetimeIndex.floor : Floor the DatetimeIndex to the specified freq.
Timestamp.strftime : Format a single Timestamp.
Period.strftime : Format a single Period.

Examples

```
>>> rng = pd.date_range(pd.Timestamp("2018-03-10 09:00"), periods=3, freq="s")
>>> rng.strftime("%B %d, %Y, %r")
Index(['March 10, 2018, 09:00:00 AM', 'March 10, 2018, 09:00:01 AM',
       'March 10, 2018, 09:00:02 AM'],
      dtype='str')
```

to_timestamp(self, freq=None, how: str = 'start') -> DatetimeIndex from pandas.core.indexes.
Cast to DatetimeArray/Index.

If possible, gives microsecond-unit DatetimeArray/Index. Otherwise gives nanosecond unit.

Parameters

freq : str or DateOffset, optional
Target frequency. The default is 'D' for week or longer, 's' otherwise.
how : {'s', 'e', 'start', 'end'}
Whether to use the start or end of the time period being converted.

Returns

DatetimeArray/Index
Timestamp representation of given Period-like object.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.from_fields : Construct a PeriodIndex from fields (year, month, day, etc.).
PeriodIndex.from_ordinals : Construct a PeriodIndex from ordinals.
PeriodIndex.hour : The hour of the period.
PeriodIndex.minute : The minute of the period.
PeriodIndex.month : The month as January=1, December=12.
PeriodIndex.second : The second of the period.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.to_timestamp()
DatetimeIndex(['2023-01-01', '2023-02-01', '2023-03-01'],
              dtype='datetime64[us]', freq='MS')
```

The frequency will not be inferred if the index contains less than three elements, or if the values of index are not strictly monotonic:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02"], freq="M")
>>> idx.to_timestamp()
DatetimeIndex(['2023-01-01', '2023-02-01'], dtype='datetime64[us]', freq=None)
```

```
>>> idx = pd.PeriodIndex(
...     ["2023-01", "2023-02", "2023-02", "2023-03"], freq="2M"
... )
>>> idx.to_timestamp()
```

```
DatetimeIndex(['2023-01-01', '2023-02-01', '2023-02-01', '2023-03-01'],
              dtype='datetime64[us]', freq=None)
```

Class methods defined here:

```
from_fields(
    *,
    year=None,
    quarter=None,
    month=None,
    day=None,
    hour=None,
    minute=None,
    second=None,
    freq=None
) -> Self from pandas.core.indexes.period.PeriodIndex
Construct a PeriodIndex from fields (year, month, day, etc.).
```

Parameters

year : int, array, or Series, default None
Year for the PeriodIndex.
quarter : int, array, or Series, default None
Quarter for the PeriodIndex.
month : int, array, or Series, default None
Month for the PeriodIndex.
day : int, array, or Series, default None
Day for the PeriodIndex.
hour : int, array, or Series, default None
Hour for the PeriodIndex.
minute : int, array, or Series, default None
Minute for the PeriodIndex.
second : int, array, or Series, default None
Second for the PeriodIndex.
freq : str or period object, optional
One of pandas period strings or corresponding objects.

Returns

PeriodIndex

See Also

PeriodIndex.from_ordinals : Construct a PeriodIndex from ordinals.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

```
>>> idx = pd.PeriodIndex.from_fields(year=[2000, 2002], quarter=[1, 3])
>>> idx
PeriodIndex(['2000Q1', '2002Q3'], dtype='period[Q-DEC]')
```

```
from_ordinals(ordinals, *, freq, name=None) -> Self from pandas.core.indexes.period.PeriodIndex
Construct a PeriodIndex from ordinals.
```

Parameters

ordinals : array-like of int
The period offsets from the proleptic Gregorian epoch.
freq : str or period object
One of pandas period strings or corresponding objects.
name : str, default None
Name of the resulting PeriodIndex.

Returns

PeriodIndex

See Also

PeriodIndex.from_fields : Construct a PeriodIndex from fields
(year, month, day, etc.).
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

```
>>> idx = pd.PeriodIndex.from_ordinals([-1, 0, 1], freq="Q")
>>> idx
PeriodIndex(['1969Q4', '1970Q1', '1970Q2'], dtype='period[Q-DEC]')
```

Static methods defined here:

```
__new__(
    cls,
    data=None,
    freq=None,
    dtype: Dtype | None = None,
    copy: bool | None = None,
    name: Hashable | None = None
) -> Self from pandas.core.indexes.period.PeriodIndex
    Create and return a new object. See help(type) for accurate signature.
```

Readonly properties defined here:

inferred_type
Return a string of the type inferred from the values.

See Also

Index.dtype : Return the dtype object of the underlying data.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.inferred_type
'integer'
```

is_full
Returns True if this PeriodIndex is range-like in that all Periods between start and end are present, in order.

values
Return an array representing the data in the Index.

.. warning::

We recommend using :attr:`Index.array` or :meth:`Index.to\_numpy`, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

Index.array : Reference to the underlying data.
Index.to_numpy : A NumPy array representing the underlying data.

Examples

For :class:`pandas.Index`:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
```


Length: 5, dtype: interval[int64, right]

Data descriptors defined here:

day

The days of the period.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.

Examples

>>> idx = pd.PeriodIndex(['2020-01-31', '2020-02-28'], freq='D')
>>> idx.day
Index([31, 28], dtype='int64')

day_of_week

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.

Examples

>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')

day_of_year

The ordinal day of the year.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.
PeriodIndex.year : The year of the period.

Examples

>>> idx = pd.PeriodIndex(["2023-01-10", "2023-02-01", "2023-03-01"], freq="D")
>>> idx.dayofyear
Index([10, 32, 60], dtype='int64')

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')
>>> idx.dayofyear
Index([365, 366, 365], dtype='int64')

dayofweek

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.

PeriodIndex.day_of_year : The ordinal day of the year.
 PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
 PeriodIndex.dayofyear : The ordinal day of the year.
 PeriodIndex.week : The week ordinal of the year.
 PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
 PeriodIndex.weekofyear : The week ordinal of the year.

Examples

```

>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')
  
```

dayofyear

The ordinal day of the year.

See Also

PeriodIndex.day : The days of the period.
 PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
 PeriodIndex.day_of_year : The ordinal day of the year.
 PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
 PeriodIndex.dayofyear : The ordinal day of the year.
 PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
 PeriodIndex.weekofyear : The week ordinal of the year.
 PeriodIndex.year : The year of the period.

Examples

```

>>> idx = pd.PeriodIndex(["2023-01-10", "2023-02-01", "2023-03-01"], freq="D")
>>> idx.dayofyear
Index([10, 32, 60], dtype='int64')

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')
>>> idx.dayofyear
Index([365, 366, 365], dtype='int64')
  
```

days_in_month

The number of days in the month.

See Also

PeriodIndex.day : The days of the period.
 PeriodIndex.days_in_month : The number of days in the month.
 PeriodIndex.daysinmonth : The number of days in the month.
 PeriodIndex.month : The month as January=1, December=12.

Examples

For Series:

```

>>> period = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.days_in_month
0     31
1     29
2     31
dtype: int64
  
```

For PeriodIndex:

```

>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.days_in_month # It can be also entered as `daysinmonth`
Index([31, 28, 31], dtype='int64')
  
```

daysinmonth

The number of days in the month.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.
PeriodIndex.month : The month as January=1, December=12.

Examples

For Series:

```
>>> period = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.days_in_month
0     31
1     29
2     31
dtype: int64
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.days_in_month    # It can be also entered as `daysinmonth`
Index([31, 28, 31], dtype='int64')
```

end_time

Get the Timestamp for the end of the period.

Returns

Timestamp

See Also

Period.start_time : Return the start Timestamp.
Period.dayofyear : Return the day of year.
Period.daysinmonth : Return the days in that month.
Period.dayofweek : Return the day of the week.

Examples

For Period:

```
>>> pd.Period('2020-01', 'D').end_time
Timestamp('2020-01-01 23:59:59.999999')
```

For Series:

```
>>> period_index = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period_index)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.end_time
0    2020-01-31 23:59:59.999999
1    2020-02-29 23:59:59.999999
2    2020-03-31 23:59:59.999999
dtype: datetime64[us]
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.end_time
DatetimeIndex(['2023-01-31 23:59:59.999999',
               '2023-02-28 23:59:59.999999',
               '2023-03-31 23:59:59.999999'],
              dtype='datetime64[us]', freq=None)
```

hour

The hour of the period.

See Also

PeriodIndex.minute : The minute of the period.
PeriodIndex.second : The second of the period.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

>>> idx = pd.PeriodIndex(["2023-01-01 10:00", "2023-01-01 11:00"], freq='h')
>>> idx.hour
Index([10, 11], dtype='int64')

is_leap_year

Logical indicating if the date belongs to a leap year.

See Also

PeriodIndex.qyear : Fiscal year the Period lies in according to its
starting-quarter.
PeriodIndex.year : The year of the period.

Examples

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx.is_leap_year
array([False, True, False])

minute

The minute of the period.

See Also

PeriodIndex.hour : The hour of the period.
PeriodIndex.second : The second of the period.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

>>> idx = pd.PeriodIndex(["2023-01-01 10:30:00",
... "2023-01-01 11:50:00"], freq='min')
>>> idx.minute
Index([30, 50], dtype='int64')

month

The month as January=1, December=12.

See Also

PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.

Examples

>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.month
Index([1, 2, 3], dtype='int64')

quarter

The quarter of the date.

See Also

PeriodIndex.qyear : Fiscal year the Period lies in according to its
starting-quarter.

Examples

>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.quarter
Index([1, 1, 1], dtype='int64')

qyear

Fiscal year the Period lies in according to its starting-quarter.

The `year` and the `qyear` of the period will be the same if the fiscal
and calendar years are the same. When they are not, the fiscal year
can be different from the calendar year of the period.

Returns

int

The fiscal year of the period.

See Also

PeriodIndex.quarter : The quarter of the date.

PeriodIndex.year : The year of the period.

Examples

If the natural and fiscal year are the same, ``qyear`` and ``year`` will be the same.

```
>>> per = pd.Period('2018Q1', freq='Q')
```

```
>>> per.qyear
```

```
2018
```

```
>>> per.year
```

```
2018
```

If the fiscal year starts in April (``Q-MAR``), the first quarter of 2018 will start in April 2017. ``year`` will then be 2017, but ``qyear`` will be the fiscal year, 2018.

```
>>> per = pd.Period('2018Q1', freq='Q-MAR')
```

```
>>> per.start_time
```

```
Timestamp('2017-04-01 00:00:00')
```

```
>>> per.qyear
```

```
2018
```

```
>>> per.year
```

```
2017
```

second

The second of the period.

See Also

PeriodIndex.hour : The hour of the period.

PeriodIndex.minute : The minute of the period.

PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01 10:00:30",
```

```
... "2023-01-01 10:00:31"], freq='s')
```

```
>>> idx.second
```

```
Index([30, 31], dtype='int64')
```

start_time

Get the Timestamp for the start of the period.

Returns

Timestamp

See Also

Period.end_time : Return the end Timestamp.

Period.dayofyear : Return the day of year.

Period.daysinmonth : Return the days in that month.

Period.dayofweek : Return the day of the week.

Examples

```
>>> period = pd.Period('2012-1-1', freq='D')
```

```
>>> period
```

```
Period('2012-01-01', 'D')
```

```
>>> period.start_time
```

```
Timestamp('2012-01-01 00:00:00')
```

```
>>> period.end_time
```

```
Timestamp('2012-01-01 23:59:59.999999')
```

week

The week ordinal of the year.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.week # It can be written `weekofyear`
Index([5, 9, 13], dtype='int64')
```

weekday

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')
```

weekofyear

The week ordinal of the year.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.week # It can be written `weekofyear`
Index([5, 9, 13], dtype='int64')
```

year

The year of the period.

See Also

PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.is_leap_year : Logical indicating if the date belongs to a leap year.
PeriodIndex.weekofyear : The week ordinal of the year.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx.year
Index([2023, 2024, 2025], dtype='int64')
```

Data and other attributes defined here:

__abstractmethods__ = frozenset()

__annotations__ = {'_data': 'PeriodArray', 'dtype': 'PeriodDtype', 'fr...

Methods inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

`__contains__(self, key: Any) -> bool`

Return a boolean indicating whether the provided key is in the index.

Parameters

key : label

The key to check if it is present in the index.

Returns

bool

Whether the key search is in the index.

Raises

TypeError

If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the list-like key is in the index.

Examples

>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> 2 in idx
True
>>> 6 in idx
False

`equals(self, other: Any) -> bool`

Determines if two Index objects contain the same elements.

`mean(self, *, skipna: bool = True, axis: int | None = 0)`

Return the mean value of the Array.

Parameters

skipna : bool, default True

Whether to ignore any NaT elements.

axis : int, optional, default 0

Axis for the function to be applied on.

Returns

scalar

Timestamp or Timedelta.

See Also

numpy.ndarray.mean : Returns the average of array elements along a given axis.
Series.mean : Return the mean value in a Series.

Notes

mean is only defined for Datetime and Timedelta dtypes, not for Period.

Examples

For :class:`pandas.DatetimeIndex`:

>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
 dtype='datetime64[us]', freq='D')
>>> idx.mean()
Timestamp('2001-01-02 00:00:00')

For :class:`pandas.TimedeltaIndex`:

>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")

```
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
                dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.mean()
Timedelta('2 days 00:00:00')
```

Readonly properties inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

asi8

freqstr

Return the frequency object as a string if it's set, otherwise None.

See Also

DatetimeIndex.inferred_freq : Returns a string representing a frequency generated by infer_freq.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
>>> idx.freqstr
'D'
```

The frequency can be inferred if there are more than 2 points:

```
>>> idx = pd.DatetimeIndex(
...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
... )
>>> idx.freqstr
'2D'
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
>>> idx.freqstr
'M'
```

Data descriptors inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

freq

Return the frequency object if it is set, otherwise None.

To learn more about the frequency strings, please see
:ref:`this link<timeseries.offset\_aliases>`.

See Also

DatetimeIndex.freq : Return the frequency object if it is set, otherwise None.
PeriodIndex.freq : Return the frequency object if it is set, otherwise None.

Examples

```
>>> datetimeindex = pd.date_range(
...     "2022-02-22 02:22:22", periods=10, tz="America/Chicago", freq="h"
... )
>>> datetimeindex
DatetimeIndex(['2022-02-22 02:22:22-06:00', '2022-02-22 03:22:22-06:00',
                '2022-02-22 04:22:22-06:00', '2022-02-22 05:22:22-06:00',
                '2022-02-22 06:22:22-06:00', '2022-02-22 07:22:22-06:00',
                '2022-02-22 08:22:22-06:00', '2022-02-22 09:22:22-06:00',
                '2022-02-22 10:22:22-06:00', '2022-02-22 11:22:22-06:00'],
                dtype='datetime64[us, America/Chicago]', freq='h')
>>> datetimeindex.freq
<Hour>
```

hasnans

resolution

Returns day, hour, minute, second, millisecond or microsecond

Methods inherited from Index:


```

__abs__(self) -> Index from pandas.core.indexes.base.Index

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
    The array interface, return my values.

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index

__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
    Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
    Parameters
    -----
    memo, default None
        Standard signature. Unused

__getitem__(self, key) from pandas.core.indexes.base.Index
    Override numpy.ndarray's __getitem__ method to work as desired.

    This function adds lists and Series as valid boolean indexers
    (ndarrays only supports ndarray with dtype=bool).

    If resulting ndim != 1, plain ndarray is returned instead of
    corresponding `Index` subclass.

__iadd__(self, other) from pandas.core.indexes.base.Index

__invert__(self) -> Index from pandas.core.indexes.base.Index

__len__(self) -> int from pandas.core.indexes.base.Index
    Return the length of the Index.

__neg__(self) -> Index from pandas.core.indexes.base.Index

__pos__(self) -> Index from pandas.core.indexes.base.Index

__reduce__(self) from pandas.core.indexes.base.Index
    Helper for pickle.

__repr__(self) -> str_t from pandas.core.indexes.base.Index
    Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
    Return whether all elements are Truthy.

    Parameters
    -----
    *args
        Required for compatibility with numpy.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    bool or array-like (if axis is specified)
        A single element array-like may be converted to bool.

    See Also
    -----
    Index.any : Return whether any element in an Index is True.
    Series.any : Return whether any element in a Series is True.
    Series.all : Return whether all elements in a Series are True.

    Notes
    -----
    Not a Number (NaN), positive infinity and negative infinity
    evaluate to True because these are not equal to zero.

    Examples
    -----

```

True, because nonzero integers are considered True.

```
>>> pd.Index([1, 2, 3]).all()
True
```

False, because ``0`` is considered False.

```
>>> pd.Index([0, 1, 2]).all()
False
```

`any(self, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return whether any element is Truthy.

Parameters

`*args`

Required for compatibility with numpy.

`**kwargs`

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)

A single element array-like may be converted to bool.

See Also

`Index.all` : Return whether all elements are True.

`Series.all` : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
```

```
>>> index.any()
```

```
True
```

```
>>> index = pd.Index([0, 0, 0])
```

```
>>> index.any()
```

```
False
```

`append(self, other: Index | Sequence[Index])` -> Index from `pandas.core.indexes.base.Index`
Append a collection of Index options together.

Parameters

`other` : Index or list/tuple of indices

Single Index or a collection of indices, which can be either a list or a tuple.

Returns

Index

Returns a new Index object resulting from appending the provided other indices to the original Index.

See Also

`Index.insert` : Make new Index inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx.append(pd.Index([4]))
```

```
Index([1, 2, 3, 4], dtype='int64')
```

`argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` -> int from `pandas.core.indexes.base.Index`
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations, the first row position is returned.

Parameters

```

-----
axis : None
    Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
    Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
    and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
    Additional arguments and keywords for compatibility with NumPy.

```

Returns

```
-----
```

```
int
    Row position of the maximum value.
```

See Also

```
-----
```

```
Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmax : Return index label of the maximum values.
Series.idxmin : Return index label of the minimum values.
```

Examples

```
-----
```

Consider dataset containing cereal calories

```

>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)

```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

```

argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from
Return int position of the smallest value in the Index.

```

If the minimum is achieved in multiple locations, the first row position is returned.

Parameters

```
-----
```

```

axis : None
    Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
    Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
    and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
    Additional arguments and keywords for compatibility with NumPy.

```

Returns

```
-----
```

```
int
    Row position of the minimum value.
```

See Also

```
-----
```

```
Series.argmin : Return position of the minimum value.
Series.argmax : Return position of the maximum value.
numpy.ndarray.argmin : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.
```

Examples

```
-----
```

Consider dataset containing cereal calories

```

>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()

```

```
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

`argsort(self, *args, **kwargs)` -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Return the integer indices that would sort the index.

Parameters

`*args`
Passed to ``numpy.ndarray.argsort``.
`**kwargs`
Passed to ``numpy.ndarray.argsort``.

Returns

`np.ndarray[np.intp]`
Integer indices that would sort the index if used as an indexer.

See Also

`numpy.argsort` : Similar method for NumPy arrays.
`Index.sort_values` : Return sorted copy of Index.

Examples

```
>>> idx = pd.Index(["b", "a", "d", "c"])
>>> idx
Index(['b', 'a', 'd', 'c'], dtype='str')

>>> order = idx.argsort()
>>> order
array([1, 0, 3, 2])

>>> idx[order]
Index(['a', 'b', 'c', 'd'], dtype='str')
```

`asof(self, label)` from `pandas.core.indexes.base.Index`
Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

`label` : object
The label up to which the method returns the latest index label.

Returns

object
The passed label if it is in the index. The previous label if the passed label is not in the sorted index or ``NaN`` if there is no such label.

See Also

`Series.asof` : Return the latest value in a Series up to the passed index.
`merge_asof` : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).
`Index.get_loc` : An ``asof`` is a thin wrapper around ``get_loc`` with `method='pad'`.

Examples

``Index.asof`` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
>>> idx.asof("2014-01-01")
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label, NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by dtype. When conversion is impossible, a `TypeError` exception is raised.

Parameters

dtype : numpy dtype or pandas type
Note that any signed integer `dtype` is treated as ``'int64'``, and any unsigned integer `dtype` is treated as ``'uint64'``, regardless of the size.
copy : bool, default True
By default, `astype` always returns a newly allocated object. If copy is set to False and internal requirements on dtype are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index
Index with values cast to specified dtype.

See Also

`Index.dtype`: Return the dtype object of the underlying data.
`Index.dtypes`: Return the dtype object of the underlying data.
`Index.convert_dtypes`: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.astype("float")
Index([1.0, 2.0, 3.0], dtype='float64')
```

`copy(self, name: Hashable | None = None, deep: bool = False) -> Self` from `pandas.core.indexes`
Make a copy of this object.

Name is set on the new object.

Parameters

name : Label, optional
Set name for new object.
deep : bool, default False
If True attempts to make a deep copy of the Index. Else makes a shallow copy.

Returns

Index
Index refer to new object which is a copy of this object.

See Also

`Index.delete`: Make new Index with passed location(-s) deleted.

`Index.drop`: Make new Index with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> new_idx = idx.copy()
>>> idx is new_idx
False
```

`delete(self, loc: int | np.integer | list[int] | npt.NDArray[np.integer]) -> Self` from `pandas`
Make new Index with passed location(-s) deleted.

Parameters

`loc` : int or list of int
Location of item(-s) which will be deleted.
Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

`numpy.delete` : Delete any rows and column from NumPy array (ndarray).

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete(1)
Index(['a', 'c'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')
```

`diff(self, periods: int = 1) -> Index` from `pandas.core.indexes.base.Index`
Computes the difference between consecutive values in the Index object.

If periods is greater than 1, computes the difference between values that are `periods` number of positions apart.

Parameters

`periods` : int, optional
The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

Index

A new Index object with the computed differences.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')
```

`difference(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters

`other` : Index or array-like
Index object or an array-like object containing elements to be compared with the elements of the original Index.
`sort` : bool or None, default None

Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

- * `None` : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.
- * `False` : Do not sort the result.
- * `True` : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object containing elements that are in the original Index but not in the `'other'` Index.

See Also

`Index.symmetric_difference` : Compute the symmetric difference of two Index objects.

`Index.intersection` : Form the intersection of two Index objects.

Examples

>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')

`drop(`
 `self,`
 `labels: Index | np.ndarray | Iterable[Hashable],`
 `errors: IgnoreRaise = 'raise'`
) -> Index from pandas.core.indexes.base.Index
 Make new Index with passed list of labels deleted.

Parameters

`labels` : array-like or scalar
 Array-like object or a scalar value, representing the labels to be removed from the Index.
`errors` : {'ignore', 'raise'}, default 'raise'
 If 'ignore', suppress error and existing labels are dropped.

Returns

Index

Will be same type as `self`, except for `RangeIndex`.

Raises

`KeyError`

If not all of the labels are found in the selected axis

See Also

`Index.dropna` : Return Index without NA/Nan values.
`Index.drop_duplicates` : Return Index with duplicate values removed.

Examples

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')

`drop_duplicates(self, *, keep: DropKeep = 'first')` -> Self from pandas.core.indexes.base.Index
 Return Index with duplicate values removed.

Parameters

`keep` : {'first', 'last', ``False``}, default 'first'
 - 'first' : Drop duplicates except for the first occurrence.
 - 'last' : Drop duplicates except for the last occurrence.
 - ``False`` : Drop all duplicates.

Returns

Index

A new Index object with the duplicate values removed.

See Also

Series.drop_duplicates : Equivalent method on Series.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Index.duplicated : Related method on Index, indicating duplicate
Index values.

Examples

Generate a pandas.Index with duplicate values.

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])
```

The `keep` parameter controls which duplicate values are removed.
The value 'first' keeps the first occurrence for each
set of duplicated entries. The default value of keep is 'first'.

```
>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')
```

The value 'last' keeps the last occurrence for each set of duplicated
entries.

```
>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')
```

The value ``False`` discards all sets of duplicated entries.

```
>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')
```

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be
of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0
If a string is given, must be the name of a level
If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index
after removing the requested level(s).

See Also

Index.dropna : Return Index without NA/Nan values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
              (2, 4, 6)],
            names=['x', 'y', 'z'])
```

```
>>> mi.droplevel()
MultiIndex([(3, 5),
              (4, 6)],
            names=['y', 'z'])
```

```
>>> mi.droplevel(2)
MultiIndex([(1, 3),
              (2, 4)],
            names=['x', 'y'])
```



```

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/Nan values.

Parameters
-----
how : {'any', 'all'}, default 'any'
    If the Index is a MultiIndex, drop the value when any or all levels
    are NaN.

Returns
-----
Index
    Returns an Index object after removing NA/Nan values.

See Also
-----
Index.fillna : Fill NA/Nan values with the specified value.
Index.isna : Detect missing values.

Examples
-----
>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')

duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting
array. Either all duplicates, all except the first, or all except the
last occurrence of duplicates can be indicated.

Parameters
-----
keep : {'first', 'last', False}, default 'first'
    The value or values in a set of duplicates to mark as missing.

    - 'first' : Mark duplicates as ``True`` except for the first
      occurrence.
    - 'last' : Mark duplicates as ``True`` except for the last
      occurrence.
    - ``False`` : Mark all duplicates as ``True``.

Returns
-----
np.ndarray[bool]
    A numpy array of boolean values indicating duplicate index values.

See Also
-----
Series.duplicated : Equivalent method on pandas.Series.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Index.drop_duplicates : Remove duplicate values from Index.

Examples
-----
By default, for each set of duplicated values, the first occurrence is
set to False and all others to True:

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])

which is equivalent to

>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])

By using 'last', the last occurrence of each set of duplicated values

```

is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])
```

`fillna(self, value)` from `pandas.core.indexes.base.Index`
Fill NA/NaN values with the specified value.

Parameters

`value` : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index

NA/NaN values replaced with `value`.

See Also

`DataFrame.fillna` : Fill NaN values of a DataFrame.
`Series.fillna` : Fill NaN Values of a Series.

Examples

```
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

`get_indexer(`
 `self,`
 `target,`
 `method: ReindexMethod | None = None,`
 `limit: int | None = None,`
 `tolerance=None`
) -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.
`method` : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
 * default: exact matches only.

 * pad / ffill: find the PREVIOUS index value if no exact match.

 * backfill / bfill: use NEXT index value if no exact match

 * nearest: use the NEAREST index value if no exact match. Tied
 distances are broken by preferring the larger index value.

`limit` : int, optional

Maximum number of consecutive labels in `target` to match for
inexact matches.

`tolerance` : optional

Maximum distance between original and new labels for inexact
matches. The values of the index at the matching locations must
satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance
to all values, or list-like, which applies variable tolerance per
element. List-like includes list, tuple, array, Series, and must be
the same size as the index and its dtype must exactly match the
index's type.

Returns

`np.ndarray[np.intp]`
Integers from 0 to `n - 1` indicating that the index at these
positions matches the corresponding target values. Missing values
in the target are marked by -1.

See Also

`Index.get_indexer_for` : Returns an indexer even when non-unique.
`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Notes

Returns -1 for unmatched values, for further explanation see the example below.

Examples

```
>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([ 1,  2, -1])
```

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

`get_indexer_for(self, target)` -> `npt.NDArray[np.intp]` from `pandas.core.indexes.base.Index`
Guaranteed return of an indexer even when non-unique.

This dispatches to `get_indexer` or `get_indexer_non_unique` as appropriate.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.

Returns

`np.ndarray[np.intp]`
List of indices.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.
`Index.get_non_unique` : Returns indexer and masks for new index given the current index.

Examples

```
>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])
```

`get_indexer_non_unique(self, target)` -> `tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]]` from `pandas.core.indexes.base.Index`
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

`target` : Index
An iterable containing the values to be used for computing indexer.

Returns

`indexer` : `np.ndarray[np.intp]`
Integers from 0 to `n - 1` indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.
`missing` : `np.ndarray[np.intp]`
An indexer into the target of the values not found. These correspond to the -1 in the indexer array.

See Also

`Index.get_indexer` : Computes indexer and mask for new index given the current index.
`Index.get_indexer_for` : Returns an indexer even when non-unique.

Examples

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned ``indexer`` contains only integers equal to -1. It demonstrates that there's no match between the index and the ``target`` values at these positions. The mask [0, 1, 2] in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in ``index``. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

get_level_values = _get_level_values(self, level) -> Index

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position of given label.

Parameters

label : object

The label for which to calculate the slice bound.

side : {'left', 'right'}

if 'left' return leftmost position of given label.

if 'right' return one-past-the-rightmost position of given label.

Returns

int

Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested label.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.get_slice_bound(3, "left")
3
```

```
>>> idx.get_slice_bound(3, "right")
4
```

If ``label`` is non-unique in the index, an error will be raised.

```
>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
KeyError: Cannot get left slice bound for non-unique label: 'a'
```

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
Group the index labels by a given array of values.

Parameters

values : array

Values used to determine the groups.

Returns

```

dict
    {group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.

Parameters
-----
other : Index
    The Index object you want to compare with the current Index object.

Returns
-----
bool
    If two Index objects have equal elements and same type True,
    otherwise False.

See Also
-----
Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
    If we have an object dtype, try to infer a non-object dtype.

Parameters
-----
copy : bool, default True
    Whether to make a copy in cases where no inference occurs.

Returns
-----
Index
    An Index with a new dtype if the dtype was inferred
    or a shallow copy if the dtype could not be inferred.

See Also
-----
Index.inferred_type: Return a string of the type inferred from the values.

Examples
-----
>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')

insert(self, loc: int, item) -> Index from pandas.core.indexes.base.Index
    Make new Index inserting new item at location.

    Follows Python numpy.insert semantics for negative values.

Parameters
-----
loc : int
    The integer location where the new item will be inserted.
item : object
    The new item to be inserted into the Index.

Returns
-----
Index
    Returns a new Index object resulting from inserting the specified item at
    the specified location within the original Index.

```

See Also

Index.append : Append a collection of Indexes together.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

intersection(self, other, sort: bool = False) from pandas.core.indexes.base.Index
Form the intersection of two Index objects.

This returns a new Index with elements common to the index and `other`.

Parameters

other : Index or array-like

An Index or an array-like object containing elements to form the intersection with the original Index.

sort : True, False or None, default False

Whether to sort the resulting index.

* None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.

* False : do not sort the result.

* True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.intersection(idx2)
Index([3, 4], dtype='int64')
```

is_(self, other) -> bool from pandas.core.indexes.base.Index
More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks that metadata is also the same.

Parameters

other : object

Other object to compare against.

Returns

bool

True if both have same underlying data, False otherwise.

See Also

Index.identical : Works like ``Index.is\_`` but also checks metadata.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx1.is_(idx1.view())
True
```

```
>>> idx1.is_(idx1.copy())
False
```

```
isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
    Return a boolean array where the index values are in `values`.
```

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like
 Sought values.
level : str or int, optional
 Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

np.ndarray[bool]
 NumPy array of boolean values.

See Also

Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a ``ValueError``.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])  
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(  
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]  
... )  
>>> midx  
MultiIndex([(1, 'red'),  
            (2, 'blue'),  
            (3, 'green')],  
            names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")  
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])  
array([ True, False, False])
```

```
isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index  
    Detect missing values.
```

Return a boolean same-sized object indicating if the values are NA. NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get mapped to ``True`` values.

Everything else get mapped to ``False`` values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

numpy.ndarray[bool]

A boolean array of whether my values are NA.

See Also

Index.notna : Boolean inverse of isna.

Index.dropna : Omit entries with missing values.

isna : Top-level isna.

Series.isna : Detect missing values in Series object.

Examples

Show which entries in a pandas.Index are NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

isnull = isna(self) -> npt.NDArray[np.bool_]

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index

The other index on which join is performed.

how : {'left', 'right', 'inner', 'outer'}

level : int or level name, default None

It is either the integer position or the name of the level.

return_indexers : bool, default False

Whether to return the indexers or not for both the index objects.

sort : bool, default False

Sort the join keys lexicographically in the result Index. If False, the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)

The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index

or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a database-style join.

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base.
Map values using an input mapping or function.

Parameters

```
-----
mapper : function, dict, or Series
    Mapping correspondence.
na_action : {None, 'ignore'}
    If 'ignore', propagate NA values, without passing them to the
    mapping correspondence.
```

Returns

```
-----
Union[Index, MultiIndex]
    The output of the mapping function applied to the index.
    If the function returns a tuple with more than one element
    a MultiIndex will be returned.
```

See Also

Index.where : Replace values where the condition is False.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')
```

Using `map` with a function:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')
```

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')
```

max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.
Return the maximum value of the Index.

Parameters

```
-----
axis : int, optional
    For compatibility with NumPy. Only 0 or None are allowed.
skipna : bool, default True
    Exclude NA/null values when showing the result.
*args, **kwargs
    Additional arguments and keywords for compatibility with NumPy.
```

Returns

```
-----
scalar
    Maximum value.
```

See Also

```
-----
Index.min : Return the minimum value in an Index.
Series.max : Return the maximum value in a Series.
DataFrame.max : Return the maximum values in a DataFrame.
```

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'
```

For a MultiIndex, the maximum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)
```

`memory_usage(self, deep: bool = False) -> int` from `pandas.core.indexes.base.Index`
Memory usage of the values.

Parameters

`deep` : bool, default False
Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption.

Returns

bytes used
Returns memory usage of the values in the Index in bytes.

See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of the
array.

Notes

Memory usage does not include memory consumed by elements that
are not components of the array if `deep=False` or if used on PyPy

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24
```

`min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return the minimum value of the Index.

Parameters

`axis` : {None}
Dummy argument for consistency with Series.
`skipna` : bool, default True
Exclude NA/null values when showing the result.
`*args, **kwargs`
Additional arguments and keywords for compatibility with NumPy.

Returns

scalar
Minimum value.

See Also

`Index.max` : Return the maximum value of the object.
`Series.min` : Return the minimum value in a Series.
`DataFrame.min` : Return the minimum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1

>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'
```

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

`notna(self) -> npt.NDArray[np.bool_]` from `pandas.core.indexes.base.Index`
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to ``True``. Characters such as empty strings ``''`` or `:attr:`numpy.inf`` are not considered NA values. NA values, such as `None` or `:attr:`numpy.NaN``, get mapped to ``False`` values.

Returns

`numpy.ndarray[bool]`
Boolean array to indicate which entries are not NA.

See Also

`Index.notnull` : Alias of `notna`.
`Index.isna`: Inverse of `notna`.
`notna` : Top-level `notna`.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. `None` is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

`notnull = notna(self) -> npt.NDArray[np.bool_]`

`putmask(self, mask, value) -> Index` from `pandas.core.indexes.base.Index`
Return a new Index of the values set with the mask.

Parameters

`mask` : array-like of bool
Array of booleans denoting where values should be replaced.
`value` : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

`Index`
A new Index of the values set with the mask.

See Also

`numpy.putmask` : Changes elements of an array based on conditional and input values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')
```

`ravel(self, order: str_t = 'C') -> Self` from `pandas.core.indexes.base.Index`

Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not implemented currently.

Returns

Index
A view on self.

See Also

numpy.ndarray.ravel : Return a flattened array.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')
```

```
reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.
```

Parameters

target : an iterable
An iterable containing the values to be used for creating the new index.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.
level : int, optional
Level of multiindex.
limit : int, optional
Maximum number of consecutive labels in ``target`` to match for inexact matches.
tolerance : int, float, or list-like, optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

new_index : pd.Index
Resulting index.
indexer : np.ndarray[np.intp] or None
Indices of output values in original index.

Raises

TypeError
If ``method`` passed along with ``level``.
ValueError
If non-unique multi-index
ValueError
If non-unique index and ``method`` or ``limit`` passed.

See Also

Series.reindex : Conform Series to new index with optional filling logic.
DataFrame.reindex : Conform DataFrame to new index with optional filling logic.

Examples

```
-----
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))
```

rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
Alter Index or MultiIndex name.

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters

name : Hashable or a sequence of the previous
Name(s) to set.
inplace : bool, default False
Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None
The same type as the caller or None if ``inplace=True``.

See Also

Index.set_names : Able to set new names partially and by level.

Examples

```
-----
>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")
Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

>>> idx = pd.MultiIndex.from_product(
...     [ ["python", "cobra"], [2018, 2019] ], names=["kind", "year"]
... )
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
           names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
           names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.
```

repeat(self, repeats, axis: None = None) -> Self from pandas.core.indexes.base.Index
Repeat elements of an Index.

Returns a new Index where each element of the current Index
is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a
non-negative integer. Repeating 0 times will return an empty
Index.
axis : None
Must be ``None``. Has no effect but is accepted for compatibility
with numpy.

Returns

Index

Newly created Index with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')
```

round(self, decimals: int = 0) -> Self from pandas.core.indexes.base.Index
Round each value in the Index to the given number of decimals.

Parameters

decimals : int, optional
Number of decimal places to round to. If decimals is negative,
it specifies the number of positions to the left of the decimal point.

Returns

Index

A new Index with the rounded values.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10.1234, 20.5678, 30.9123, 40.4567, 50.7890])
>>> idx.round(decimals=2)
Index([10.12, 20.57, 30.91, 40.46, 50.79], dtype='float64')
```

set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core.
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

names : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

level : int, Hashable or a sequence of the previous, optional
If the index is a MultiIndex and names is not dict-like, level(s) to set
(None for all levels). Otherwise level must be None.

inplace : bool, default False
Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([('python', 2018),
```

```

        ('python', 2019),
        ('cobra', 2018),
        ('cobra', 2019)],
    )
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['species', 'year'])

```

When renaming levels with a dict, levels can not be passed.

```

>>> idx.set_names({"kind": "snake"})
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['snake', 'year'])

```

```

slice_indexer(
    self,
    start: Hashable | None = None,
    end: Hashable | None = None,
    step: int | None = None
) -> slice from pandas.core.indexes.base.Index
    Compute the slice indexer for input labels and step.

```

Index needs to be ordered and unique.

Parameters

```

-----
start : label, default None
    If None, defaults to the beginning.
end : label, default None
    If None, defaults to the end.
step : int, default None
    If None, defaults to 1.

```

Returns

```

-----
slice
    A slice object.

```

Raises

```

-----
KeyError : If key does not exist, or key is not unique and index is
            not ordered.

```

See Also

```

-----
Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

```

Notes

```

-----
This function assumes that the data is sorted, so use at your own peril.

```

Examples

```

-----
This is a method on all index types. For example you can do:

```

```

>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)

```

```

slice_locs(
    self,
    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index

```

Compute slice locations for input labels.

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, defaults None
 If None, defaults to 1.

Returns

tuple[int, int]
 Returns a tuple of two integers representing the slice locations for the input labels within the index.

See Also

Index.get_loc : Get location for a single label.

Notes

This method only works if the index is monotonic or unique.

Examples

>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)

>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)

```
sort_values(  
    self,  
    *,  
    return_indexer: bool = False,  
    ascending: bool = True,  
    na_position: NaPosition = 'last',  
    key: Callable | None = None  
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index  
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

return_indexer : bool, default False
 Should the indices that would sort the index be returned.
ascending : bool, default True
 Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
 Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
key : callable, optional
 If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.

Returns

sorted_index : pandas.Index
 Sorted copy of the index.
indexer : numpy.ndarray, optional
 The indices that the index itself was sorted by.

See Also

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples


```

-----
>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')

Sort values in ascending order (default behavior).

>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')

Sort values in descending order, and also get the indices `idx` was
sorted by.

>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))

```

sortlevel(
 self,
 level=None,
 ascending: bool | list[bool] = True,
 sort_remaining=None,
 na_position: NaPosition = 'first'
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
For internal compatibility with the Index API.

Sort the Index. This is for compat with MultiIndex

Parameters

ascending : bool, default True
 False to sort in descending order
na_position : {'first' or 'last'}, default 'first'
 Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
 the end.

.. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns

Index

symmetric_difference(
 self,
 other,
 result_name: abc.Hashable | None = None,
 sort: bool | None = None
) from pandas.core.indexes.base.Index
Compute the symmetric difference of two Index objects.

Parameters

other : Index or array-like
 Index or an array-like object with elements to compute the symmetric
 difference with the original Index.
result_name : str
 A string representing the name of the resulting Index, if desired.
sort : bool or None, default None
 Whether to sort the resulting index. By default, the
 values are attempted to be sorted, but any TypeError from
 incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any TypeErrors
 from comparing incomparable elements.
* False : Do not sort the result.
* True : Sort the result (which may raise TypeError).

Returns

Index
Returns a new Index object containing elements that appear in either the
original Index or the `other` Index, but not both.

See Also

Index.difference : Return a new Index with elements of index not in other.

Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes

``symmetric\_difference`` contains elements that appear in either
``idx1`` or ``idx2`` but not both. Equivalent to the Index created by
``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates
dropped.

Examples

>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')

```
take(  
    self,  
    indices,  
    axis: Axis = 0,  
    allow_fill: bool = True,  
    fill_value=None,  
    **kwargs  
) -> Self from pandas.core.indexes.base.Index  
Return a new Index of the values selected by the indices.
```

For internal compatibility with numpy arrays.

Parameters

indices : array-like
Indices to be taken.
axis : {0 or 'index'}, optional
The axis over which to select values, always 0 or 'index'.
allow_fill : bool, default True
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices
from the right (the default). This is similar to
:func:`numpy.take`.

* True: negative values in `indices` indicate
missing values. These values are set to `fill\_value`. Any
other negative values raise a ``ValueError``.

fill_value : scalar, default None
If allow_fill=True and fill_value is not None, indices specified by
-1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
**kwargs
Required for compatibility with numpy.

Returns

Index
An index formed of elements at the given indices. Will be the same
type as self, except for RangeIndex.

See Also

numpy.ndarray.take: Return an array formed from the
elements of a at the given indices.

Examples

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.take([2, 2, 1, 2])
Index(['c', 'c', 'b', 'c'], dtype='str')

```
to_flat_index(self) -> Self from pandas.core.indexes.base.Index  
Identity method.
```

This is implemented for compatibility with subclass implementations
when chaining.

Returns

pd.Index
Caller.

See Also

MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.core.indexes
Create a DataFrame with a column containing the Index.

Parameters

index : bool, default True
Set the index of the returned DataFrame as the original Index.

name : object, defaults to index.name
The passed name should substitute for the index name (if it has one).

Returns

DataFrame
DataFrame containing the original Index data.

See Also

Index.to_series : Convert an Index to a Series.
Series.to_frame : Convert Series to DataFrame.

Examples

>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
 animal
animal
Ant Ant
Bear Bear
Cow Cow

By default, the original Index is reused. To enforce a new Index:

>>> idx.to_frame(index=False)
 animal
0 Ant
1 Bear
2 Cow

To override the name of the resulting column, specify `name`:

>>> idx.to_frame(index=False, name="zoo")
 zoo
0 Ant
1 Bear
2 Cow

to_series(self, index=None, name: Hashable | None = None) -> Series from pandas.core.indexes
Create a Series with both index and values equal to the index keys.

Useful with map for returning an indexer based on an index.

Parameters

index : Index, optional
Index of resulting Series. If None, defaults to original index.
name : str, optional
Name of resulting Series. If None, defaults to name of original index.

Returns

Series
The dtype will be based on the type of the Index values.

See Also

Index.to_frame : Convert an Index to a DataFrame.
Series.to_frame : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ``index``:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1      Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.

`sort` : bool or None, default None
Whether to sort the resulting Index.

- * None : Sort the result, except when
 1. ``self`` and ``other`` are equal.
 2. ``self`` or ``other`` has length 0.
 3. Some values in ``self`` or ``other`` cannot be compared.
A `RuntimeWarning` is issued in this case.
- * False : do not sort the result.
- * True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the ``other`` Index.

See Also

`Index.unique` : Return unique values in the index.
`Index.intersection` : Form the intersection of two Index objects.
`Index.difference` : Return a new Index with elements of index not in ``other``.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')
```

Union mismatched dtypes

```
>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')
```

MultiIndex case

```
>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )
```

`unique(self, level: Hashable | None = None) -> Self` from `pandas.core.indexes.base.Index`
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

`level` : int or hashable, optional

Only return values from specified level (for MultiIndex).

If int, gets the level by integer position, else by level name.

Returns

Index

Unique values in the index.

See Also

`unique` : Numpy array of unique values in that column.

`Series.unique` : Return unique values of Series object.

Examples

```
>>> idx = pd.Index([1, 1, 2, 3, 3])
```

```
>>> idx.unique()
```

```
Index([1, 2, 3], dtype='int64')
```

`view(self, cls=None)` from `pandas.core.indexes.base.Index`

Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the ``cls`` argument, it attempts

to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

`cls` : data-type or ndarray sub-class, optional
Data-type descriptor of the returned view, e.g., float32 or int16. Omitting it results in the view having the same data-type as ``self``. This argument can also be specified as an ndarray sub-class, e.g., np.int64 or np.float32 which then specifies the type of the returned object.

Returns

Index or ndarray
A view of the Index. If ``cls`` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric ``cls`` is specified an ndarray view with the new dtype is returned.

Raises

ValueError
If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

`Index.copy` : Returns a copy of the Index.
`numpy.ndarray.view` : Returns a new view of array with the same data.

Examples

```
>>> idx = pd.Index([-1, 0, 1])
>>> idx.view()
Index([-1, 0, 1], dtype='int64')
```

```
>>> idx.view(np.uint64)
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
array([ nan,   nan,  0.e+00,  0.e+00,  1.e-45,  0.e+00], dtype=float32)
```

`where(self, cond, other=None)` -> Index from `pandas.core.indexes.base.Index`
Replace values where the condition is False.

The replacement is taken from other.

Parameters

`cond` : bool array-like with the same length as self
Condition to select the values on.
`other` : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index
A copy of self with values replaced from other where the condition is False.

See Also

`Series.where` : Same method for Series.
`DataFrame.where` : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
```

```
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

`has_duplicates`

Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

`Index.is_unique` : Inverse method that checks if it has unique values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
```

```
>>> idx.has_duplicates
```

```
True
```

```
>>> idx = pd.Index([1, 5, 7])
```

```
>>> idx.has_duplicates
```

```
False
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
```

```
... )
```

```
>>> idx.has_duplicates
```

```
True
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
```

```
>>> idx.has_duplicates
```

```
False
```

`is_monotonic_decreasing`

Return a boolean if the values are equal or decreasing.

Returns

bool

See Also

`Index.is_monotonic_increasing` : Check if the values are equal or increasing.

Examples

```
>>> pd.Index([3, 2, 1]).is_monotonic_decreasing
```

```
True
```

```
>>> pd.Index([3, 2, 2]).is_monotonic_decreasing
```

```
True
```

```
>>> pd.Index([3, 1, 2]).is_monotonic_decreasing
```

```
False
```

`is_monotonic_increasing`

Return a boolean if the values are equal or increasing.

Returns

bool

See Also

`Index.is_monotonic_decreasing` : Check if the values are equal or decreasing.

Examples

```
>>> pd.Index([1, 2, 3]).is_monotonic_increasing
```

```
True
```

```
>>> pd.Index([1, 2, 2]).is_monotonic_increasing
```

```
True
```

```
>>> pd.Index([1, 3, 2]).is_monotonic_increasing
```

```
False
```

nlevels
Number of levels.

shape
Return a tuple of the shape of the underlying data.

See Also

Index.size: Return the number of elements in the underlying data.
Index.ndim: Number of dimensions of the underlying data, by definition 1.
Index.dtype: Return the dtype object of the underlying data.
Index.values: Return an array representing the data in the Index.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)

Data descriptors inherited from Index:

array
The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray
An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.
Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.


```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

dtype

Return the dtype object of the underlying data.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.dtype
dtype('int64')
```

is_unique

Return if the index has unique values.

Returns

bool

See Also

Index.has_duplicates : Inverse method that checks if it has duplicate values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.is_unique
False
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.is_unique
True
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.is_unique
False
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.is_unique
True
```

name

Return Index or MultiIndex name.

Returns

label (hashable object)
The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.
Index.rename: Able to set new names partially and by level.
Series.name: Corresponding Series property.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx
```

```
Index([1, 2, 3], dtype='int64', name='x')
>>> idx.name
'x'
```

names

Get names on index.

This method returns a FrozenList containing the names of the object. It's primarily intended for internal use.

Returns

FrozenList

A FrozenList containing the object's names, contains None if the object does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx.names
FrozenList(['x'])
```

```
>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
>>> idx.names
FrozenList(['x', 'y'])
```

If the index does not have a name set:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])
```

Data and other attributes inherited from Index:

__pandas_priority__ = 2000

str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method. Patterned after Python's string methods, with some inspiration from R's stringr package.

Parameters

data : Series or Index

The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.

Index.str : Vectorized string functions for Index.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

```

__iter__(self) -> Iterator
    Return an iterator of the values.

    These are each a scalar type, which is a Python scalar
    (for str, int, float) or a pandas scalar
    (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    iterator
        An iterator yielding scalar values from the Series.

    See Also
    -----
    Series.items : Lazily iterate over (index, value) tuples.

    Examples
    -----
    >>> s = pd.Series([1, 2, 3])
    >>> for x in s:
    ...     print(x)
    1
    2
    3

factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.intc], npt.NDArray[np.intc]]
    Encode the object as an enumerated type or categorical variable.

    This method is useful for obtaining a numeric representation of an
    array when all that matters is identifying distinct values. factorize
    is available as both a top-level function :func:`pandas.factorize`,
    and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

    Parameters
    -----
    sort : bool, default False
        Sort uniques` and shuffle codes` to maintain the
        relationship.
    use_na_sentinel : bool, default True
        If True, the sentinel -1 will be used for NaN values. If False,
        NaN values will be encoded as non-negative integers and will not drop the
        NaN from the uniques of the values.

    Returns
    -----
    codes : ndarray
        An integer ndarray that's an indexer into uniques`.
        uniques.take(codes)` will have the same values as values`.
    uniques : ndarray, Index, or Categorical
        The unique valid values. When values` is Categorical, uniques`
        is a Categorical. When values` is some other pandas object, an
        Index` is returned. Otherwise, a 1-D ndarray is returned.

    .. note::

        Even if there's a missing value in values`, uniques` will
        *not* contain an entry for it.

    See Also
    -----
    cut : Discretize continuous-valued array.
    unique : Find the unique value in an array.

    Notes
    -----
    Reference :ref:`the user guide <reshaping.factorize>` for more examples.

    Examples
    -----
    These examples all show factorize as a top-level method like
    pd.factorize(values)`. The results are identical for methods like
    :meth:`Series.factorize`.

    >>> codes, uniques = pd.factorize(
    ...     np.array(["b", "b", "a", "c", "b"], dtype="O")
    ... )
    >>> codes

```

```
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With ``sort=True``, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use\_na\_sentinel=True`` (the default), missing values are indicated in the `codes` with the sentinel value ``-1`` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([1., 2., nan])
```

```
item(self)
    Return the first element of the underlying data as a Python scalar.

    Returns
    -----
    scalar
        The first element of Series or Index.

    Raises
```

```
-----
ValueError
    If the data is not length = 1.
```

See Also

```
-----
Index.values : Returns an array representing the data in the Index.
Series.head : Returns the first `n` rows.
```

Examples

```
-----
>>> s = pd.Series([1])
>>> s.item()
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'
```

```
nunique(self, dropna: bool = True) -> int
    Return number of unique elements in the object.
```

Excludes NA values by default.

Parameters

```
-----
dropna : bool, default True
    Don't include NaN in the count.
```

Returns

```
-----
int
    An integer indicating the number of unique elements in the object.
```

See Also

```
-----
DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.
```

Examples

```
-----
>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0    1
1    3
2    5
3    7
4    7
dtype: int64

>>> s.nunique()
4
```

```
searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.
```

Find the indices into a sorted Index `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::

The Index *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

```
-----
value : array-like or scalar
    Values to insert into `self`.
```

side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable
index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort `self` into ascending
order. They are typically the result of ``np.argsort``.

Returns

int or array of int
A scalar or array of insertion points with the
same shape as `value`.

See Also

sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

>>> ser = pd.Series([1, 2, 3])
>>> ser
0 1
1 2
2 3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0 2000-03-11
1 2000-03-12
2 2000-03-13
dtype: datetime64[us]

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
... ["apple", "bread", "bread", "cheese", "milk"], ordered=True
...)
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']

>>> ser.searchsorted("bread")
np.int64(1)

>>> ser.searchsorted(["bread"], side="right")
array([3])

If the values are not monotonically sorted, wrong locations
may be returned:

>>> ser = pd.Series([2, 1, 3])
>>> ser
0 2
1 1
2 3
dtype: int64

```

>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1

to_list = tolist(self) -> list

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>,
    **kwargs
) -> np.ndarray
    A NumPy ndarray representing the values in this Series or Index.

Parameters
-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
**kwargs
    Additional keywords passed through to the ``to_numpy`` method
    of the underlying array (for extension arrays).

Returns
-----
numpy.ndarray
    The NumPy ndarray holding the values from this Series or Index.
    The dtype of the array may differ. See Notes.

See Also
-----
Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

Notes
-----
The returned array will be the same up to equality (values equal
in `self` will be equal in the returned array; likewise for values
that are not equal). When `self` contains an ExtensionArray, the
dtype may be different. For example, for a category-dtype Series,
``to_numpy()`` will return a NumPy array and the categorical dtype
will be lost.

For NumPy dtypes, this will be a reference to the actual data stored
in this Series or Index (assuming ``copy=False``). Modifying the result
in place will modify the data stored in the Series or Index (not that
we recommend doing that).

For extension types, ``to_numpy()`` *may* require copying data and
coercing the result to a NumPy type (possibly object), which may be
expensive. When you need a no-copy reference to the underlying data,
:attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of
``to_numpy()`` for various dtypes within pandas.

=====
dtype          array type
=====
category[T]    ndarray[T] (same dtype as input)
period          ndarray[object] (Periods)
interval        ndarray[object] (Intervals)
IntegerNA       ndarray[object]
datetime64[ns]  datetime64[ns]
datetime64[ns, tz] ndarray[object] (Timestamps)
=====

Examples
-----

```

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)
```

Specify the `dtype` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

```
>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

`tolist(self)` -> list
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

```
-----
list
    List containing the values as Python or pandas scalers.
```

See Also

```
-----
numpy.ndarray.tolist : Return the array as an a.ndim-levels deep
    nested list of Python scalars.
```

Examples

```
-----
For Series
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.to_list()
[1, 2, 3]
```

`transpose(self, *args, **kwargs)` -> Self
Return the transpose, which is by definition self.

Returns

```
-----
%(klass)s
```

```
value_counts(
    self,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    bins=None,
    dropna: bool = True
)
```

-> Series
Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

normalize : bool, default False
If True then the object returned will contain the relative frequencies of the unique values.
sort : bool, default True
Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False
Sort in ascending order.
bins : int, optional
Rather than count values, group them into half-open bins, a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
Don't include counts of NaN.

Returns

Series

Series containing counts of unique values.

See Also

Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64
```

With `normalize` set to `True`, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64
```

****bins****

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]     2
(3.0, 4.0]     1
Name: count, dtype: int64
```

****dropna****

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
Name: count, dtype: int64
```

****Categorical Dtypes****

Rows with categorical type will be counted as one group if they have same categories and order.
In the example below, even though ``a``, ``c``, and ``d`` all have the same data types of ``category``, only ``c`` and ``d`` will be counted as one group since ``a`` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64
```

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.
Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
-----
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
      height  weight
elk      1.5     500
moose     2.6     800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
      height  weight
```

```

elk      3.0    501.5
moose    4.1    801.5

```

Each element of a list is added to a column of the DataFrame, in order.

```

>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0    501.5
moose      3.1    801.5

```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose      3.1    801.5

```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5

```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN

```

```

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5

```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```

>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk        1.7     NaN
moose      3.0     NaN

```

```
__and__(self, other)
```

```
__divmod__(self, other)
```

```
__eq__(self, other)
    Return self==value.
```

```
__floordiv__(self, other)
```

```
__ge__(self, other)
    Return self>=value.
```

```
__gt__(self, other)
    Return self>value.
```

```
__le__(self, other)
    Return self<=value.
```

```
__lt__(self, other)
    Return self<value.
```

```
__mod__(self, other)
```

```

    __mul__(self, other)
    __ne__(self, other)
        Return self!=value.
    __or__(self, other)
        Return self|value.
    __pow__(self, other)
    __radd__(self, other)
    __rand__(self, other)
    __rdivmod__(self, other)
    __rfloordiv__(self, other)
    __rmod__(self, other)
    __rmul__(self, other)
    __ror__(self, other)
        Return value|self.
    __rpow__(self, other)
    __rsub__(self, other)
    __rtruediv__(self, other)
    __rxor__(self, other)
    __sub__(self, other)
    __truediv__(self, other)
    __xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
    __dict__
        dictionary for instance variables
    __weakref__
        list of weak references to the object

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
    __hash__ = None

-----
Methods inherited from pandas.core.base.PandasObject:
    __sizeof__(self) -> int
        Generates the total memory usage for an object that returns
        either a value or Series of values

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
    __dir__(self) -> list[str]
        Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.

```

```

class RangeIndex(Index)
    RangeIndex(
        start=None,
        stop=None,
        step=None,
        dtype: Dtype | None = None,
        copy: bool = False,

```

```
name: Hashable | None = None
) -> Self
```

Immutable Index implementing a monotonic integer range.

RangeIndex is a memory-saving special case of an Index limited to representing monotonic ranges with a 64-bit dtype. Using RangeIndex may in some instances improve computing speed.

This is the default index type used by DataFrame and Series when no explicit index is provided by the user.

Parameters

start : int, range, or other RangeIndex instance, default None
 If int and "stop" is not given, interpreted as "stop" instead.
stop : int, default None
 The end value of the range (exclusive).
step : int, default None
 The step size of the range.
dtype : np.int64, default None
 Unused, accepted for homogeneity with other index types.
copy : bool, default False
 Unused, accepted for homogeneity with other index types.
name : object, optional
 Name to be stored in the index.

Attributes

start
stop
step

Methods

from_range

See Also

Index : The base pandas Index type.

Examples

>>> list(pd.RangeIndex(5))
[0, 1, 2, 3, 4]

>>> list(pd.RangeIndex(-2, 4))
[-2, -1, 0, 1, 2, 3]

>>> list(pd.RangeIndex(0, 10, 2))
[0, 2, 4, 6, 8]

>>> list(pd.RangeIndex(2, -10, -3))
[2, -1, -4, -7]

>>> list(pd.RangeIndex(0))
[]

>>> list(pd.RangeIndex(1, 0))
[]

Method resolution order:

RangeIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
builtins.object

Methods defined here:

__abs__(self) -> Self | Index from pandas.core.indexes.range.RangeIndex

__contains__(self, key: Any) -> bool from pandas.core.indexes.range.RangeIndex
 Return a boolean indicating whether the provided key is in the index.

Parameters

key : label

The key to check if it is present in the index.

Returns

bool

Whether the key search is in the index.

Raises

TypeError

If the key is not hashable.

See Also

Index.isin : Returns an ndarray of boolean dtype indicating whether the list-like key is in the index.

Examples

```
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
```

```
>>> 2 in idx
```

```
True
```

```
>>> 6 in idx
```

```
False
```

__floordiv__(self, other) from pandas.core.indexes.range.RangeIndex

__getitem__(self, key) from pandas.core.indexes.range.RangeIndex
Conserve RangeIndex type for scalar and slice keys.

__invert__(self) -> Self from pandas.core.indexes.range.RangeIndex

__iter__(self) -> Iterator[int] from pandas.core.indexes.range.RangeIndex
Return an iterator of the values.

Returns

iterator

An iterator yielding ints from the RangeIndex.

Examples

```
>>> idx = pd.RangeIndex(3)
>>> for x in idx:
...     print(x)
0
1
2
```

__len__(self) -> int from pandas.core.indexes.range.RangeIndex
return the length of the RangeIndex

__neg__(self) -> Self from pandas.core.indexes.range.RangeIndex

__pos__(self) -> Self from pandas.core.indexes.range.RangeIndex

__reduce__(self) from pandas.core.indexes.range.RangeIndex
Helper for pickle.

all(self, *args, **kwargs) -> bool from pandas.core.indexes.range.RangeIndex
Return whether all elements are Truthy.

Parameters

*args

Required for compatibility with numpy.

**kwargs

Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)
A single element array-like may be converted to bool.

See Also

Index.any : Return whether any element in an Index is True.
Series.any : Return whether any element in a Series is True.
Series.all : Return whether all elements in a Series are True.

Notes

Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples

True, because nonzero integers are considered True.

```
>>> pd.Index([1, 2, 3]).all()
True
```

False, because ``0`` is considered False.

```
>>> pd.Index([0, 1, 2]).all()
False
```

any(self, *args, **kwargs) -> bool from pandas.core.indexes.range.RangeIndex
Return whether any element is Truthy.

Parameters

*args
Required for compatibility with numpy.
**kwargs
Required for compatibility with numpy.

Returns

bool or array-like (if axis is specified)
A single element array-like may be converted to bool.

See Also

Index.all : Return whether all elements are True.
Series.all : Return whether all elements are True.

Notes

Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples

```
>>> index = pd.Index([0, 1, 2])
>>> index.any()
True
```

```
>>> index = pd.Index([0, 0, 0])
>>> index.any()
False
```

argmax(self, axis=None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations,
the first row position is returned.

Parameters

axis : None
Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the maximum value.

See Also

Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmax : Return index label of the maximum values.
Series.idxmin : Return index label of the minimum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
the minimum cereal calories is the first element,
since index is zero-indexed.

argmin(self, axis=None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes
Return int position of the smallest value in the Index.

If the minimum is achieved in multiple locations,
the first row position is returned.

Parameters

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

int

Row position of the minimum value.

See Also

Series.argmin : Return position of the minimum value.
Series.argmax : Return position of the maximum value.
numpy.ndarray.argmin : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.

Examples

Consider dataset containing cereal calories

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and
the minimum cereal calories is the first element,
since index is zero-indexed.

argsort(self, *args, **kwargs) -> npt.NDArray[np.intp] from pandas.core.indexes.range.RangeIndex

Returns the indices that would sort the index and its underlying data.

Returns

np.ndarray[np.intp]

See Also

numpy.ndarray.argsort

copy(self, name: Hashable | None = None, deep: bool = False) -> Self from pandas.core.indexes
Make a copy of this object.

Name is set on the new object.

Parameters

name : Label, optional

Set name for new object.

deep : bool, default False

If True attempts to make a deep copy of the RangeIndex.

Else makes a shallow copy.

Returns

RangeIndex

RangeIndex refer to new object which is a copy of this object.

See Also

RangeIndex.delete: Make new RangeIndex with passed location(-s) deleted.

RangeIndex.drop: Make new RangeIndex with passed list of labels deleted.

Notes

In most cases, there should be no functional difference from using ``deep``, but if ``deep`` is passed it will attempt to deepcopy.

Examples

```
>>> idx = pd.RangeIndex(3)
```

```
>>> new_idx = idx.copy()
```

```
>>> idx is new_idx
```

```
False
```

delete(self, loc) -> Index from pandas.core.indexes.range.RangeIndex
Make new Index with passed location(-s) deleted.

Parameters

loc : int or list of int

Location of item(-s) which will be deleted.

Use a list of locations to delete more than one value at the same time.

Returns

Index

Will be same type as self, except for RangeIndex.

See Also

numpy.delete : Delete any rows and column from NumPy array (ndarray).

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> idx.delete(1)
```

```
Index(['a', 'c'], dtype='str')
```

```
>>> idx = pd.Index(["a", "b", "c"])
```

```
>>> idx.delete([0, 2])
```

```
Index(['b'], dtype='str')
```

equals(self, other: object) -> bool from pandas.core.indexes.range.RangeIndex
Determines if two Index objects contain the same elements.

```
factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.in
    Encode the object as an enumerated type or categorical variable.
```

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function `:func:`pandas.factorize``, and as a method `:meth:`Series.factorize`` and `:meth:`Index.factorize``.

Parameters

`sort` : bool, default False
Sort `'uniques'` and shuffle `'codes'` to maintain the relationship.
`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray
An integer ndarray that's an indexer into `'uniques'`.
`'uniques.take(codes)'` will have the same values as `'values'`.
`uniques` : ndarray, Index, or Categorical
The unique valid values. When `'values'` is Categorical, `'uniques'` is a Categorical. When `'values'` is some other pandas object, an `'Index'` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in `'values'`, `'uniques'` will *not* contain an entry for it.

See Also

`cut` : Discretize continuous-valued array.
`unique` : Find the unique value in an array.

Notes

Reference `:ref:`the user guide <reshaping.factorize>`` for more examples.

Examples

These examples all show `factorize` as a top-level method like `pd.factorize(values)`. The results are identical for methods like `:meth:`Series.factorize``.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `'uniques'` will be sorted, and `'codes'` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When `use_na_sentinel=True` (the default), missing values are indicated in the `'codes'` with the sentinel value `-1` and missing values are not included in `'uniques'`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
```

```
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])
```

```
>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])
```

```
get_loc(self, key) -> int from pandas.core.indexes.range.RangeIndex
Get integer location for requested label.
```

Parameters

key : int or float
Label to locate. Integer-like floats (e.g. 3.0) are accepted and treated as the corresponding integer. Non-integer floats and other non-integer labels are not valid and will raise `KeyError` or `InvalidIndexError`.

Returns

int
Integer location of the label within the `RangeIndex`.

Raises

`KeyError`
If the label is not present in the `RangeIndex` or the label is a non-integer value.
`InvalidIndexError`
If the label is of an invalid type for the `RangeIndex`.

See Also

`RangeIndex.get_slice_bound` : Calculate slice bound that corresponds to given label.
`RangeIndex.get_indexer` : Computes indexer and mask for new index given the current index.
`RangeIndex.get_non_unique` : Returns indexer and masks for new index given the current index.
`RangeIndex.get_indexer_for` : Returns an indexer even when non-unique.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.get_loc(3)
3
```

```
>>> idx = pd.RangeIndex(2, 10, 2) # values [2, 4, 6, 8]
>>> idx.get_loc(6)
2
```

`insert(self, loc: int, item) -> Index from pandas.core.indexes.range.RangeIndex`
Make new Index inserting new item at location.

Follows Python `numpy.insert` semantics for negative values.

Parameters

`loc : int`

The integer location where the new item will be inserted.

`item : object`

The new item to be inserted into the Index.

Returns

Index

Returns a new Index object resulting from inserting the specified item at the specified location within the original Index.

See Also

`Index.append` : Append a collection of Indexes together.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')
```

`max(self, axis=None, skipna: bool = True, *args, **kwargs) -> int | float from pandas.core.indexes.range.RangeIndex`
The maximum value of the RangeIndex

`memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.range.RangeIndex`
Memory usage of my values

Parameters

`deep : bool`

Introspect the data deeply, interrogate
`object` dtypes for system-level memory consumption

Returns

bytes used

Notes

Memory usage does not include memory consumed by elements that are not components of the array if `deep=False`

See Also

`numpy.ndarray.nbytes`

`min(self, axis=None, skipna: bool = True, *args, **kwargs) -> int | float from pandas.core.indexes.range.RangeIndex`
The minimum value of the RangeIndex

`round(self, decimals: int = 0) -> Self | Index from pandas.core.indexes.range.RangeIndex`
Round each value in the Index to the given number of decimals.

Parameters

`decimals : int, optional`

Number of decimal places to round to. If `decimals` is negative, it specifies the number of positions to the left of the decimal point
e.g. `round(11.0, -1) == 10.0`.

Returns

Index or RangeIndex
A new Index with the rounded values.

Examples

>>> import pandas as pd
>>> idx = pd.RangeIndex(10, 30, 10)
>>> idx.round(decimals=-1)
RangeIndex(start=10, stop=30, step=10)
>>> idx = pd.RangeIndex(10, 15, 1)
>>> idx.round(decimals=-1)
Index([10, 10, 10, 10, 10], dtype='int64')

```
searchsorted(  
    self,  
    value,  
    side: Literal['left', 'right'] = 'left',  
    sorter: NumpySorter | None = None  
) -> npt.NDArray[np.intp] | np.intp from pandas.core.indexes.range.RangeIndex  
Find indices where elements should be inserted to maintain order.
```

Find the indices into a sorted Index `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::

The Index *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

value : array-like or scalar
Values to insert into `self`.
side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given. If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort `self` into ascending order. They are typically the result of ``np.argsort``.

Returns

int or array of int
A scalar or array of insertion points with the same shape as `value`.

See Also

sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

>>> ser = pd.Series([1, 2, 3])
>>> ser
0 1
1 2
2 3
dtype: int64

>>> ser.searchsorted(4)
np.int64(3)

>>> ser.searchsorted([0, 4])
array([0, 3])

>>> ser.searchsorted([1, 3], side="left")
array([0, 2])

```
>>> ser.searchsorted([1, 3], side="right")
array([1, 3])

>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0    2000-03-11
1    2000-03-12
2    2000-03-13
dtype: datetime64[us]

>>> ser.searchsorted("3/14/2000")
np.int64(3)

>>> ser = pd.Categorical(
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
... )
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']

>>> ser.searchsorted("bread")
np.int64(1)

>>> ser.searchsorted(["bread"], side="right")
array([3])
```

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])
>>> ser
0    2
1    1
2    3
dtype: int64

>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1
```

```
sort_values(
    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray | RangeIndex] from pandas.core.indexes.range.RangeIndex
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

```
-----
return_indexer : bool, default False
    Should the indices that would sort the index be returned.
ascending : bool, default True
    Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
key : callable, optional
    If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.
```

Returns

```
-----
sorted_index : pandas.Index
    Sorted copy of the index.
indexer : numpy.ndarray, optional
    The indices that the index itself was sorted by.
```

See Also

```
-----
```


Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

```
>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')
```

Sort values in ascending order (default behavior).

```
>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')
```

Sort values in descending order, and also get the indices `idx` was sorted by.

```
>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))
```

```
symmetric_difference(
    self,
    other,
    result_name: Hashable | None = None,
    sort=None
) -> Index from pandas.core.indexes.range.RangeIndex
Compute the symmetric difference of two Index objects.
```

Parameters

other : Index or array-like
Index or an array-like object with elements to compute the symmetric difference with the original Index.

result_name : str
A string representing the name of the resulting Index, if desired.

sort : bool or None, default None
Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

- * None : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.
- * False : Do not sort the result.
- * True : Sort the result (which may raise `TypeError`).

Returns

Index
Returns a new Index object containing elements that appear in either the original Index or the `other` Index, but not both.

See Also

`Index.difference` : Return a new Index with elements of index not in other.
`Index.union` : Form the union of two Index objects.
`Index.intersection` : Form the intersection of two Index objects.

Notes

`symmetric_difference` contains elements that appear in either `idx1` or `idx2` but not both. Equivalent to the Index created by `idx1.difference(idx2) | idx2.difference(idx1)` with duplicates dropped.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')
```

```
take(
    self,
    indices,
    axis: Axis = 0,
    allow_fill: bool = True,
    fill_value=None,
```

```

    **kwargs
) -> Self | Index from pandas.core.indexes.range.RangeIndex
    Return a new Index of the values selected by the indices.

    For internal compatibility with numpy arrays.

    Parameters
    -----
    indices : array-like
        Indices to be taken.
    axis : {0 or 'index'}, optional
        The axis over which to select values, always 0 or 'index'.
    allow_fill : bool, default True
        How to handle negative values in `indices`.

        * False: negative values in `indices` indicate positional indices
          from the right (the default). This is similar to
          :func:`numpy.take`.

        * True: negative values in `indices` indicate
          missing values. These values are set to `fill_value`. Any
          other negative values raise a ``ValueError``.

    fill_value : scalar, default None
        If allow_fill=True and fill_value is not None, indices specified by
        -1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
    **kwargs
        Required for compatibility with numpy.

    Returns
    -----
    Index
        An index formed of elements at the given indices. Will be the same
        type as self, except for RangeIndex.

    See Also
    -----
    numpy.ndarray.take: Return an array formed from the
        elements of a at the given indices.

    Examples
    -----
    >>> idx = pd.Index(["a", "b", "c"])
    >>> idx.take([2, 2, 1, 2])
    Index(['c', 'c', 'b', 'c'], dtype='str')

    tolist(self) -> list[int] from pandas.core.indexes.range.RangeIndex
        Return a list of the values.

        These are each a scalar type, which is a Python scalar
        (for str, int, float) or a pandas scalar
        (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    list
        List containing the values as Python or pandas scalars.

    See Also
    -----
    numpy.ndarray.tolist : Return the array as an a.ndim-levels deep
        nested list of Python scalars.

    Examples
    -----
    For Series

    >>> s = pd.Series([1, 2, 3])
    >>> s.tolist()
    [1, 2, 3]

    For Index:

    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')

```

```

>>> idx.to_list()
[1, 2, 3]

value_counts(
    self,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    bins=None,
    dropna: bool = True
) -> Series from pandas.core.indexes.range.RangeIndex
    Return a Series containing counts of unique values.

    The resulting object will be in descending order so that the
    first element is the most frequently-occurring element.
    Excludes NA values by default.

Parameters
-----
normalize : bool, default False
    If True then the object returned will contain the relative
    frequencies of the unique values.
sort : bool, default True
    Stable sort by frequencies when True. Preserve the order of the data
    when False.

    .. versionchanged:: 3.0.0

    Prior to 3.0.0, the sort was unstable.
ascending : bool, default False
    Sort in ascending order.
bins : int, optional
    Rather than count values, group them into half-open bins,
    a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
    Don't include counts of NaN.

Returns
-----
Series
    Series containing counts of unique values.

See Also
-----
Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples
-----
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by
dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64

**bins**

Bins can be useful for going from a continuous variable to a
categorical variable; instead of counting unique
apparitions of values, divide the index in the specified
number of half-open bins.

>>> s.value_counts(bins=3)

```

```
(0.996, 2.0]    2
(2.0, 3.0]     2
(3.0, 4.0]     1
Name: count, dtype: int64
```

****dropna****

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN     1
Name: count, dtype: int64
```

****Categorical Dtypes****

Rows with categorical type will be counted as one group if they have same categories and order. In the example below, even though `a`, `c`, and `d` all have the same data types of `category`, only `c` and `d` will be counted as one group since `a` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
   a  b  c  d
0  1  2  3  3

>>> df.dtypes
a    category
b         str
c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64
```

Class methods defined here:

`from_range(data: range, name=None, dtype: Dtype | None = None) -> Self` from pandas.core.index
Create :class:`pandas.RangeIndex` from a ``range`` object.

This method provides a way to create a :class:`pandas.RangeIndex` directly from a Python ``range`` object. The resulting :class:`RangeIndex` will have the same start, stop, and step values as the input ``range`` object. It is particularly useful for constructing indices in an efficient and memory-friendly manner.

Parameters

`data` : range
The range object to be converted into a RangeIndex.
`name` : str, default None
Name to be stored in the index.
`dtype` : Dtype or None
Data type for the RangeIndex. If None, the default integer type will be used.

Returns

RangeIndex

See Also

`RangeIndex` : Immutable Index implementing a monotonic integer range.
`Index` : Immutable sequence used for indexing and alignment.

Examples

```

-----
>>> pd.RangeIndex.from_range(range(5))
RangeIndex(start=0, stop=5, step=1)

>>> pd.RangeIndex.from_range(range(2, -10, -3))
RangeIndex(start=2, stop=-10, step=-3)

```

Static methods defined here:

```

__new__(
    cls,
    start=None,
    stop=None,
    step=None,
    dtype: Dtype | None = None,
    copy: bool = False,
    name: Hashable | None = None
) -> Self from pandas.core.indexes.range.RangeIndex
    Create and return a new object. See help(type) for accurate signature.

```

Readonly properties defined here:

```

dtype
    Return the dtype object of the underlying data.

    See Also
    -----
    Index.inferred_type: Return a string of the type inferred from the values.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')
    >>> idx.dtype
    dtype('int64')

```

```

inferred_type
    Return a string of the type inferred from the values.

    See Also
    -----
    Index.dtype : Return the dtype object of the underlying data.

    Examples
    -----
    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')
    >>> idx.inferred_type
    'integer'

```

```

is_unique
    return if the index has unique values

```

```

size
    Return the number of elements in the underlying data.

    See Also
    -----
    Series.ndim: Number of dimensions of the underlying data, by definition 1.
    Series.shape: Return a tuple of the shape of the underlying data.
    Series.dtype: Return the dtype object of the underlying data.
    Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

    Examples
    -----
    For Series:

    >>> s = pd.Series(["Ant", "Bear", "Cow"])
    >>> s
    0    Ant
    1    Bear
    2    Cow
    dtype: str

```

```
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

start

The value of the `start` parameter (`0` if this was not supplied).

This property returns the starting value of the `RangeIndex`. If the `start` value is not explicitly provided during the creation of the `RangeIndex`, it defaults to `0`.

See Also

`RangeIndex` : Immutable index implementing a range-based index.
`RangeIndex.stop` : Returns the stop value of the `RangeIndex`.
`RangeIndex.step` : Returns the step value of the `RangeIndex`.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.start
0
```

```
>>> idx = pd.RangeIndex(2, -10, -3)
>>> idx.start
2
```

step

The value of the `step` parameter (`1` if this was not supplied).

The `step` parameter determines the increment (or decrement in the case of negative values) between consecutive elements in the `RangeIndex`.

See Also

`RangeIndex` : Immutable index implementing a range-based index.
`RangeIndex.stop` : Returns the stop value of the `RangeIndex`.
`RangeIndex.start` : Returns the start value of the `RangeIndex`.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.step
1
```

```
>>> idx = pd.RangeIndex(2, -10, -3)
>>> idx.step
-3
```

Even if `class: pandas.RangeIndex` is empty, `step` is still `1` if not supplied.

```
>>> idx = pd.RangeIndex(1, 0)
>>> idx.step
1
```

stop

The value of the `stop` parameter.

This property returns the `stop` value of the `RangeIndex`, which defines the upper (or lower, in case of negative steps) bound of the index range. The `stop` value is exclusive, meaning the `RangeIndex` includes values up to but not including this value.

See Also

`RangeIndex` : Immutable index representing a range of integers.
`RangeIndex.start` : The start value of the `RangeIndex`.
`RangeIndex.step` : The step size between elements in the `RangeIndex`.

Examples

```
>>> idx = pd.RangeIndex(5)
>>> idx.stop
5
```

```
>>> idx = pd.RangeIndex(2, -10, -3)
>>> idx.stop
-10
```

Data descriptors defined here:

is_monotonic_decreasing

is_monotonic_increasing

nbytes

Return the number of bytes in the underlying data.

Data and other attributes defined here:

__annotations__ = {'_range': 'range', '_values': 'np.ndarray'}

Methods inherited from Index:

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
The array interface, return my values.

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index

__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
Gets called after a ufunc and other functions e.g. np.split.

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index

__copy__(self) -> Self from pandas.core.indexes.base.Index

__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index

Parameters

memo, default None

Standard signature. Unused

__iadd__(self, other) from pandas.core.indexes.base.Index

__repr__(self) -> str_t from pandas.core.indexes.base.Index
Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

append(self, other: Index | Sequence[Index]) -> Index from pandas.core.indexes.base.Index
Append a collection of Index options together.

Parameters

other : Index or list/tuple of indices

Single Index or a collection of indices, which can be either a list or a tuple.

Returns

Index

Returns a new Index object resulting from appending the provided other indices to the original Index.

See Also

Index.insert : Make new Index inserting new item at location.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.append(pd.Index([4]))
Index([1, 2, 3, 4], dtype='int64')
```

```
asof(self, label) from pandas.core.indexes.base.Index
    Return the label from the index, or, if not present, the previous one.
```

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

label : object
The label up to which the method returns the latest index label.

Returns

object
The passed label if it is in the index. The previous label if the passed label is not in the sorted index or `NaN` if there is no such label.

See Also

Series.asof : Return the latest value in a Series up to the passed index.
merge_asof : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).
Index.get_loc : An `asof` is a thin wrapper around `get\_loc` with method='pad'.

Examples

`Index.asof` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
>>> idx.asof("2014-01-01")
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label, NaN is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

```
asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp] from pandas
    Return the locations (indices) of labels in the index.
```

As in the :meth:`pandas.Index.asof`, if the label (a particular entry in ``where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in ``where``, -1 is returned.

``mask`` is used to ignore ``NA`` values in the index during calculation.

Parameters

where : Index
An Index consisting of an array of timestamps.
mask : np.ndarray[bool]
Array of booleans denoting where values in the original data are not ``NA``.

Returns

`np.ndarray[np.intp]`
An array of locations (indices) of the labels from the index which correspond to the return values of `:meth:`pandas.Index.asof`` for every element in `where``.

See Also

`Index.asof` : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use `mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by `dtype`. When conversion is impossible, a `TypeError` exception is raised.

Parameters

`dtype` : numpy dtype or pandas type
Note that any signed integer `dtype`` is treated as `int64``, and any unsigned integer `dtype`` is treated as `uint64``, regardless of the size.
`copy` : bool, default True
By default, `astype` always returns a newly allocated object. If `copy` is set to False and internal requirements on `dtype` are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index

Index with values cast to specified dtype.

See Also

`Index.dtype`: Return the dtype object of the underlying data.
`Index.dtypes`: Return the dtype object of the underlying data.
`Index.convert_dtypes`: Convert columns to the best possible dtypes.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.astype("float")
Index([1.0, 2.0, 3.0], dtype='float64')
```

`diff(self, periods: int = 1)` -> Index from `pandas.core.indexes.base.Index`
Computes the difference between consecutive values in the Index object.

If `periods` is greater than 1, computes the difference between values that are `periods`` number of positions apart.

Parameters

`periods` : int, optional
The number of positions between the current and previous value to compute the difference with. Default is 1.

Returns

Index

A new Index object with the computed differences.

Examples

```
>>> import pandas as pd
>>> idx = pd.Index([10, 20, 30, 40, 50])
>>> idx.diff()
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')
```

difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index
Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters

other : Index or array-like

Index object or an array-like object containing elements to be compared with the elements of the original Index.

sort : bool or None, default None

Whether to sort the resulting index. By default, the values are attempted to be sorted, but any `TypeError` from incomparable elements is caught by pandas.

* None : Attempt to sort the result, but catch any `TypeError`s from comparing incomparable elements.

* False : Do not sort the result.

* True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object containing elements that are in the original Index but not in the `other` Index.

See Also

`Index.symmetric_difference` : Compute the symmetric difference of two Index objects.

`Index.intersection` : Form the intersection of two Index objects.

Examples

```
>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')
```

drop()

```
self,
labels: Index | np.ndarray | Iterable[Hashable],
errors: IgnoreRaise = 'raise'
) -> Index from pandas.core.indexes.base.Index
Make new Index with passed list of labels deleted.
```

Parameters

labels : array-like or scalar

Array-like object or a scalar value, representing the labels to be removed from the Index.

errors : {'ignore', 'raise'}, default 'raise'

If 'ignore', suppress error and existing labels are dropped.

Returns

Index

Will be same type as self, except for `RangeIndex`.

Raises

KeyError

If not all of the labels are found in the selected axis

See Also

Index.dropna : Return Index without NA/Nan values.
Index.drop_duplicates : Return Index with duplicate values removed.

Examples

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index
Return Index with duplicate values removed.

Parameters

keep : {'first', 'last', ``False``}, default 'first'
- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

Returns

Index

A new Index object with the duplicate values removed.

See Also

Series.drop_duplicates : Equivalent method on Series.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Index.duplicated : Related method on Index, indicating duplicate
Index values.

Examples

Generate a pandas.Index with duplicate values.

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])

The `keep` parameter controls which duplicate values are removed.
The value 'first' keeps the first occurrence for each
set of duplicated entries. The default value of keep is 'first'.

>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')

The value 'last' keeps the last occurrence for each set of duplicated
entries.

>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')

The value ``False`` discards all sets of duplicated entries.

>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be
of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0
If a string is given, must be the name of a level
If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index
after removing the requested level(s).

See Also

Index.dropna : Return Index without NA/Nan values.

Examples

```
-----
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
            (2, 4, 6)],
            names=['x', 'y', 'z'])

>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
            names=['y', 'z'])

>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')
```

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/Nan values.

Parameters

how : {'any', 'all'}, default 'any'
If the Index is a MultiIndex, drop the value when any or all levels are NaN.

Returns

Index
Returns an Index object after removing NA/Nan values.

See Also

Index.fillna : Fill NA/Nan values with the specified value.
Index.isna : Detect missing values.

Examples

```
-----
>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')
```

duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

keep : {'first', 'last', False}, default 'first'
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

np.ndarray[bool]
A numpy array of boolean values indicating duplicate index values.

See Also

Series.duplicated : Equivalent method on pandas.Series.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Index.drop_duplicates : Remove duplicate values from Index.

Examples

By default, for each set of duplicated values, the first occurrence is set to False and all others to True:

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])
```

which is equivalent to

```
>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([ True, False,  True, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([ True, False,  True, False,  True])
```

fillna(self, value) from pandas.core.indexes.base.Index
Fill NA/NAN values with the specified value.

Parameters

value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-likes.

Returns

Index
NA/NAN values replaced with `value`.

See Also

DataFrame.fillna : Fill NaN values of a DataFrame.
Series.fillna : Fill NaN Values of a Series.

Examples

>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')

```
get_indexer(
    self,
    target,
    method: ReindexMethod | None = None,
    limit: int | None = None,
    tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
```

Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to ndarray.take to align the current data to the new index.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied

distances are broken by preferring the larger index value.

limit : int, optional
Maximum number of consecutive labels in ``target`` to match for inexact matches.

tolerance : optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

See Also

Index.get_indexer_for : Returns an indexer even when non-unique.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Notes

Returns -1 for unmatched values, for further explanation see the example below.

Examples

```
>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([ 1,  2, -1])
```

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

get_indexer_for(self, target) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Guaranteed return of an indexer even when non-unique.

This dispatches to get_indexer or get_indexer_non_unique as appropriate.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.

Returns

np.ndarray[np.intp]
List of indices.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Examples

```
>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])
```

get_indexer_non_unique(self, target) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]] from pandas.core.indexes.base.Index
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to ndarray.take to align the current data to the new index.

Parameters

target : Index

An iterable containing the values to be used for computing indexer.

Returns

indexer : np.ndarray[np.intp]

Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

missing : np.ndarray[np.intp]

An indexer into the target of the values not found.

These correspond to the -1 in the indexer array.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.

Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
```

```
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))
```

In the example below there are no matched values.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
```

```
(array([-1, -1, -1]), array([0, 1, 2]))
```

For this reason, the returned ``indexer`` contains only integers equal to -1. It demonstrates that there's no match between the index and the ``target`` values at these positions. The mask [0, 1, 2] in the return value shows that the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example below the first item is an array of locations in ``index``. The second item is a mask shows that the first and third elements are missing.

```
>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
```

```
(array([-1, 1, 3, 4, -1]), array([0, 2]))
```

```
get_level_values = _get_level_values(self, level) -> Index
```

```
get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.
```

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position of given label.

Parameters

label : object

The label for which to calculate the slice bound.

side : {'left', 'right'}

if 'left' return leftmost position of given label.

if 'right' return one-past-the-rightmost position of given label.

Returns

int

Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested label.

Examples

```
>>> idx = pd.RangeIndex(5)
```

```
>>> idx.get_slice_bound(3, "left")
```

```
3
```

```

>>> idx.get_slice_bound(3, "right")
4

If ``label`` is non-unique in the index, an error will be raised.

>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
  KeyError: Cannot get left slice bound for non-unique label: 'a'

```

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
Group the index labels by a given array of values.

Parameters

values : array
Values used to determine the groups.

Returns

dict
{group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
Similar to equals, but checks that object attributes and types are also equal.

Parameters

other : Index
The Index object you want to compare with the current Index object.

Returns

bool
If two Index objects have equal elements and same type True,
otherwise False.

See Also

Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples

>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

```

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

```

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
If we have an object dtype, try to infer a non-object dtype.

Parameters

copy : bool, default True
Whether to make a copy in cases where no inference occurs.

Returns

Index
An Index with a new dtype if the dtype was inferred
or a shallow copy if the dtype could not be inferred.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

>>> pd.Index(["a", 1]).infer_objects()


```
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')
```

`intersection(self, other, sort: bool = False)` from `pandas.core.indexes.base.Index`
Form the intersection of two Index objects.

This returns a new Index with elements common to the index and ``other``.

Parameters

```
-----
other : Index or array-like
    An Index or an array-like object containing elements to form the
    intersection with the original Index.
sort : True, False or None, default False
    Whether to sort the resulting index.

    * None : sort the result, except when `self` and `other` are equal
      or when the values cannot be compared.
    * False : do not sort the result.
    * True : Sort the result (which may raise TypeError).
```

Returns

```
-----
Index
    Returns a new Index object with elements common to both the original Index
    and the `other` Index.
```

See Also

```
-----
Index.union : Form the union of two Index objects.
Index.difference : Return a new Index with elements of index not in other.
Index.isin : Return a boolean array where the index values are in values.
```

Examples

```
-----
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.intersection(idx2)
Index([3, 4], dtype='int64')
```

`is_(self, other) -> bool` from `pandas.core.indexes.base.Index`
More flexible, faster check like ``is`` but that works through views.

Note: this is **not** the same as ``Index.identical()``, which checks that metadata is also the same.

Parameters

```
-----
other : object
    Other object to compare against.
```

Returns

```
-----
bool
    True if both have same underlying data, False otherwise.
```

See Also

```
-----
Index.identical : Works like `Index.is_` but also checks metadata.
```

Examples

```
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx1.is_(idx1.view())
True

>>> idx1.is_(idx1.copy())
False
```

`isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_]` from `pandas.core.indexes.base.Index`
Return a boolean array where the index values are in ``values``.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like
 Sought values.
level : str or int, optional
 Name or position of the index level to use (if the index is a
 `MultiIndex`).

Returns

np.ndarray[bool]
 NumPy array of boolean values.

See Also

Series.isin : Same for Series.
DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a ``ValueError``.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

Check whether each index value in a list of values.

>>> idx.isin([1, 4])
array([True, False, False])

>>> midx = pd.MultiIndex.from_arrays(
... [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]
...)
>>> midx
MultiIndex([(1, 'red'),
 (2, 'blue'),
 (3, 'green')],
 names=['number', 'color'])

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")  
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])  
array([ True, False, False])
```

```
isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index  
Detect missing values.
```

Return a boolean same-sized object indicating if the values are NA. NA values, such as ``None``, :attr:`numpy.NaN` or :attr:`pd.NaT`, get mapped to ``True`` values. Everything else get mapped to ``False`` values. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns

numpy.ndarray[bool]
 A boolean array of whether my values are NA.

See Also

Index.notna : Boolean inverse of isna.
Index.dropna : Omit entries with missing values.
isna : Top-level isna.
Series.isna : Detect missing values in Series object.

Examples

Show which entries in a pandas.Index are NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

```
isnull = isna(self) -> npt.NDArray[np.bool_]
```

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index
The other index on which join is performed.
how : {'left', 'right', 'inner', 'outer'}
level : int or level name, default None
It is either the integer position or the name of the level.
return_indexers : bool, default False
Whether to return the indexers or not for both the index objects.
sort : bool, default False
Sort the join keys lexicographically in the result Index. If False, the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)
The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a database-style join.

Examples

>>> idx1 = pd.Index([1, 2, 3])

```

>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))

```

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base.
Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}
If 'ignore', propagate NA values, without passing them to the
mapping correspondence.

Returns

Union[Index, MultiIndex]
The output of the mapping function applied to the index.
If the function returns a tuple with more than one element
a MultiIndex will be returned.

See Also

Index.where : Replace values where the condition is False.

Examples

>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')

Using `map` with a function:

```

>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')

```

notna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA.
Non-missing values get mapped to ``True``. Characters such as empty
strings ``''`` or :attr:`numpy.inf` are not considered NA values.
NA values, such as None or :attr:`numpy.NaN`, get mapped to ``False``
values.

Returns

numpy.ndarray[bool]
Boolean array to indicate which entries are not NA.

See Also

Index.notnull : Alias of notna.
Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an
array.

```

>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])

```

Empty strings are not considered NA values. None is considered a NA

```

value.

>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])

notnull = notna(self) -> npt.NDArray[np.bool_]

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters
-----
mask : array-like of bool
    Array of booleans denoting where values should be replaced.
value : scalar
    Scalar value to use to fill holes (e.g. 0).
    This value cannot be a list-like.

Returns
-----
Index
    A new Index of the values set with the mask.

See Also
-----
numpy.putmask : Changes elements of an array
    based on conditional and input values.

Examples
-----
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters
-----
order : {'K', 'A', 'C', 'F'}, default 'C'
    Specify the memory layout of the view. This parameter is not
    implemented currently.

Returns
-----
Index
    A view on self.

See Also
-----
numpy.ndarray.ravel : Return a flattened array.

Examples
-----
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')

reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.

Parameters
-----
target : an iterable
    An iterable containing the values to be used for creating the new index.
method : {None, 'pad', 'ffill', 'backfill', 'bfill', 'nearest'}, optional

```

```

    * default: exact matches only.
    * pad / ffill: find the PREVIOUS index value if no exact match.
    * backfill / bfill: use NEXT index value if no exact match
    * nearest: use the NEAREST index value if no exact match. Tied
      distances are broken by preferring the larger index value.
level : int, optional
    Level of multiindex.
limit : int, optional
    Maximum number of consecutive labels in ``target`` to match for
    inexact matches.
tolerance : int, float, or list-like, optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation abs(index[indexer] - target) <= tolerance.

    Tolerance may be a scalar value, which applies the same tolerance
    to all values, or list-like, which applies variable tolerance per
    element. List-like includes list, tuple, array, Series, and must be
    the same size as the index and its dtype must exactly match the
    index's type.

Returns
-----
new_index : pd.Index
    Resulting index.
indexer : np.ndarray[np.intp] or None
    Indices of output values in original index.

Raises
-----
TypeError
    If ``method`` passed along with ``level``.
ValueError
    If non-unique multi-index
ValueError
    If non-unique index and ``method`` or ``limit`` passed.

See Also
-----
Series.reindex : Conform Series to new index with optional filling logic.
DataFrame.reindex : Conform DataFrame to new index with optional filling logic.

Examples
-----
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))

rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
    Alter Index or MultiIndex name.

    Able to set new names without level. Defaults to returning new index.
    Length of names must match number of levels in MultiIndex.

Parameters
-----
name : Hashable or a sequence of the previous
    Name(s) to set.
inplace : bool, default False
    Modifies the object directly, instead of creating a new Index or
    MultiIndex.

Returns
-----
Index or None
    The same type as the caller or None if ``inplace=True``.

See Also
-----
Index.set_names : Able to set new names partially and by level.

Examples
-----
>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")

```

```

Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

>>> idx = pd.MultiIndex.from_product(
...     ["python", "cobra"], [2018, 2019], names=["kind", "year"]
... )
>>> idx
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([('python', 2018),
            ('python', 2019),
            ('cobra', 2018),
            ('cobra', 2019)],
            names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.

```

`repeat(self, repeats, axis: None = None) -> Self` from `pandas.core.indexes.base.Index`
Repeat elements of an Index.

Returns a new Index where each element of the current Index is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints

The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty Index.

`axis` : None

Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

Index

Newly created Index with repeated elements.

See Also

`Series.repeat` : Equivalent function for Series.

`numpy.repeat` : Similar method for `:class:`numpy.ndarray``.

Examples

```

>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')

```

`set_names(self, names, *, level=None, inplace: bool = False) -> Self | None` from `pandas.core`
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

`names` : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

`level` : int, Hashable or a sequence of the previous, optional

If the index is a MultiIndex and names is not dict-like, level(s) to set (None for all levels). Otherwise level must be None.

`inplace` : bool, default False

Modifies the object directly, instead of creating a new Index or MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            )
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['species', 'year'])
```

When renaming levels with a dict, levels can not be passed.

```
>>> idx.set_names({"kind": "snake"})
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
            names=['snake', 'year'])
```

shift(self, periods: int = 1, freq=None) -> Self from pandas.core.indexes.base.Index
Shift index by desired number of time frequency increments.

This method is for shifting the values of datetime-like indexes by a specified time increment a given number of times.

Parameters

periods : int, default 1
Number of periods (or increments) to shift by,
can be positive or negative.
freq : pandas.DateOffset, pandas.Timedelta or str, optional
Frequency increment to shift by.
If None, the index is shifted by its own `freq` attribute.
Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

pandas.Index
Shifted index.

See Also

Series.shift : Shift values of Series.

Notes

This method is only implemented for datetime-like index classes, i.e., DatetimeIndex, PeriodIndex and TimedeltaIndex.

Examples

Put the first 5 month starts of 2011 into an index.

```
>>> month_starts = pd.date_range("1/1/2011", periods=5, freq="MS")
>>> month_starts
DatetimeIndex(['2011-01-01', '2011-02-01', '2011-03-01', '2011-04-01',
              '2011-05-01'],
              dtype='datetime64[ns]', freq='MS')
```



```

dtype='datetime64[us]', freq='MS')

Shift the index by 10 days.

>>> month_starts.shift(10, freq="D")
DatetimeIndex(['2011-01-11', '2011-02-11', '2011-03-11', '2011-04-11',
               '2011-05-11'],
              dtype='datetime64[us]', freq=None)

The default value of `freq` is the `freq` attribute of the index,
which is 'MS' (month start) in this example.

>>> month_starts.shift(10)
DatetimeIndex(['2011-11-01', '2011-12-01', '2012-01-01', '2012-02-01',
               '2012-03-01'],
              dtype='datetime64[us]', freq='MS')

```

slice_indexer(
 self,
 start: Hashable | None = None,
 end: Hashable | None = None,
 step: int | None = None
) -> slice from pandas.core.indexes.base.Index
Compute the slice indexer for input labels and step.

Index needs to be ordered and unique.

Parameters

start : label, default None
 If None, defaults to the beginning.
end : label, default None
 If None, defaults to the end.
step : int, default None
 If None, defaults to 1.

Returns

slice
 A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.
Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```

>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)

>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)

```

slice_locs(
 self,
 start: SliceType = None,
 end: SliceType = None,
 step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.

Parameters

```

start : label, default None
    If None, defaults to the beginning.
end : label, default None
    If None, defaults to the end.
step : int, defaults None
    If None, defaults to 1.

Returns
-----
tuple[int, int]
    Returns a tuple of two integers representing the slice locations for the
    input labels within the index.

See Also
-----
Index.get_loc : Get location for a single label.

Notes
-----
This method only works if the index is monotonic or unique.

Examples
-----
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)

>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)

sortlevel(
    self,
    level=None,
    ascending: bool | list[bool] = True,
    sort_remaining=None,
    na_position: NaPosition = 'first'
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
    For internal compatibility with the Index API.

    Sort the Index. This is for compat with MultiIndex

Parameters
-----
ascending : bool, default True
    False to sort in descending order
na_position : {'first' or 'last'}, default 'first'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.

    .. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns
-----
Index

to_flat_index(self) -> Self from pandas.core.indexes.base.Index
    Identity method.

    This is implemented for compatibility with subclass implementations
    when chaining.

Returns
-----
pd.Index
    Caller.

See Also
-----
MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.core
    Create a DataFrame with a column containing the Index.

Parameters

```

```

-----
index : bool, default True
    Set the index of the returned DataFrame as the original Index.

name : object, defaults to index.name
    The passed name should substitute for the index name (if it has
    one).

```

Returns

```

-----
DataFrame
    DataFrame containing the original Index data.

```

See Also

```

-----
Index.to_series : Convert an Index to a Series.
Series.to_frame : Convert Series to DataFrame.

```

Examples

```

-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
>>> idx.to_frame()
      animal
animal
Ant      Ant
Bear    Bear
Cow      Cow

```

By default, the original Index is reused. To enforce a new Index:

```

>>> idx.to_frame(index=False)
      animal
0    Ant
1  Bear
2    Cow

```

To override the name of the resulting column, specify `name`:

```

>>> idx.to_frame(index=False, name="zoo")
      zoo
0    Ant
1  Bear
2    Cow

```

```

to_series(self, index=None, name: Hashable | None = None) -> Series from pandas.core.indexes
    Create a Series with both index and values equal to the index keys.

```

Useful with map for returning an indexer based on an index.

Parameters

```

-----
index : Index, optional
    Index of resulting Series. If None, defaults to original index.
name : str, optional
    Name of resulting Series. If None, defaults to name of original
    index.

```

Returns

```

-----
Series
    The dtype will be based on the type of the Index values.

```

See Also

```

-----
Index.to_frame : Convert an Index to a DataFrame.
Series.to_frame : Convert Series to DataFrame.

```

Examples

```

-----
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")

```

By default, the original index and original name is reused.

```

>>> idx.to_series()
      animal
Ant      Ant
Bear    Bear

```

```
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ``index``:

```
>>> idx.to_series(index=[0, 1, 2])
0      Ant
1      Bear
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.
`sort` : bool or None, default None
Whether to sort the resulting Index.

* None : Sort the result, except when

1. ``self`` and ``other`` are equal.
2. ``self`` or ``other`` has length 0.
3. Some values in ``self`` or ``other`` cannot be compared.
A `RuntimeWarning` is issued in this case.

* False : do not sort the result.

* True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the ``other`` Index.

See Also

`Index.unique` : Return unique values in the index.

`Index.intersection` : Form the intersection of two Index objects.

`Index.difference` : Return a new Index with elements of index not in ``other``.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')
```

Union mismatched dtypes

```
>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')
```

MultiIndex case

```
>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
```

```

>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )

```

unique(self, level: Hashable | None = None) -> Self from pandas.core.indexes.base.Index
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

level : int or hashable, optional
Only return values from specified level (for MultiIndex).
If int, gets the level by integer position, else by level name.

Returns

Index
Unique values in the index.

See Also

unique : Numpy array of unique values in that column.
Series.unique : Return unique values of Series object.

Examples

```

>>> idx = pd.Index([1, 1, 2, 3, 3])
>>> idx.unique()
Index([1, 2, 3], dtype='int64')

```

view(self, cls=None) from pandas.core.indexes.base.Index
Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the `cls` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

cls : data-type or ndarray sub-class, optional
Data-type descriptor of the returned view, e.g., float32 or int16.
Omitting it results in the view having the same data-type as `self`.
This argument can also be specified as an ndarray sub-class, e.g., np.int64 or np.float32 which then specifies the type of

the returned object.

Returns

Index or ndarray

A view of the Index. If `cls` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric `cls` is specified an ndarray view with the new dtype is returned.

Raises

ValueError

If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

`Index.copy` : Returns a copy of the Index.

`numpy.ndarray.view` : Returns a new view of array with the same data.

Examples

```
>>> idx = pd.Index([-1, 0, 1])
>>> idx.view()
Index([-1, 0, 1], dtype='int64')
```

```
>>> idx.view(np.uint64)
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
array([ nan,   nan,  0.e+00,  0.e+00,  1.e-45,  0.e+00], dtype=float32)
```

`where(self, cond, other=None)` -> Index from `pandas.core.indexes.base.Index`
Replace values where the condition is False.

The replacement is taken from other.

Parameters

`cond` : bool array-like with the same length as self
Condition to select the values on.

`other` : scalar, or array-like, default None
Replacement if the condition is False.

Returns

pandas.Index

A copy of self with values replaced from other where the condition is False.

See Also

`Series.where` : Same method for Series.

`DataFrame.where` : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
>>> idx.where(idx.isin(["car", "train"]), "other")
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

`has_duplicates`

Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

`Index.is_unique` : Inverse method that checks if it has unique values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False
```

`nlevels`

Number of levels.

`shape`

Return a tuple of the shape of the underlying data.

See Also

`Index.size`: Return the number of elements in the underlying data.
`Index.ndim`: Number of dimensions of the underlying data, by definition 1.
`Index.dtype`: Return the dtype object of the underlying data.
`Index.values`: Return an array representing the data in the Index.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape
(3,)
```

`values`

Return an array representing the data in the Index.

.. warning::

We recommend using `:attr:`Index.array`` or `:meth:`Index.to_numpy``, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: `numpy.ndarray` or `ExtensionArray`

See Also

`Index.array` : Reference to the underlying data.
`Index.to_numpy` : A NumPy array representing the underlying data.

Examples

For `:class:`pandas.Index``:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
```

```
array([1, 2, 3])

For :class:`pandas.IntervalIndex`:

>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors inherited from Index:

array
The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray
An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.
Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
```


Categories (2, str): ['a', 'b']

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

bool

See Also

Index.isna : Detect missing values.

Index.dropna : Return Index without NA/NaN values.

Index.fillna : Fill NA/NaN values with the specified value.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", None])
```

```
>>> s
```

```
a    1
```

```
b    2
```

```
None 3
```

```
dtype: int64
```

```
>>> s.index.hasnans
```

```
True
```

name

Return Index or MultiIndex name.

Returns

label (hashable object)

The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.

Index.rename: Able to set new names partially and by level.

Series.name: Corresponding Series property.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64', name='x')
```

```
>>> idx.name
```

```
'x'
```

names

Get names on index.

This method returns a FrozenList containing the names of the object.
It's primarily intended for internal use.

Returns

FrozenList

A FrozenList containing the object's names, contains None if the object does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
```

```
>>> idx.names
```

```
FrozenList(['x'])
```

```
>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
```

```
>>> idx.names
```

```
FrozenList(['x', 'y'])
```

If the index does not have a name set:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])
```

Data and other attributes inherited from Index:

```
__pandas_priority__ = 2000
```

```
str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.
```

NAs stay NA unless handled otherwise by a particular method.
Patterned after Python's string methods, with some inspiration from
R's stringr package.

Parameters

```
-----
data : Series or Index
      The content of the Series or Index.
```

See Also

```
-----
Series.str : Vectorized string functions for Series.
Index.str : Vectorized string functions for Index.
```

Examples

```
-----
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

```
item(self)
Return the first element of the underlying data as a Python scalar.
```

Returns

```
-----
scalar
      The first element of Series or Index.
```

Raises

```
-----
ValueError
      If the data is not length = 1.
```

See Also

```
-----
Index.values : Returns an array representing the data in the Index.
Series.head : Returns the first `n` rows.
```

Examples

```
-----
>>> s = pd.Series([1])
>>> s.item()
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'
```

```
nunique(self, dropna: bool = True) -> int
```

Return number of unique elements in the object.

Excludes NA values by default.

Parameters

dropna : bool, default True
Don't include NaN in the count.

Returns

int

An integer indicating the number of unique elements in the object.

See Also

DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.

Examples

>>> s = pd.Series([1, 3, 5, 7, 7])

>>> s

0 1

1 3

2 5

3 7

4 7

dtype: int64

>>> s.nunique()

4

to_list = tolist(self) -> list

Return a list of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list

List containing the values as Python or pandas scalars.

See Also

numpy.ndarray.tolist : Return the array as an a.ndim-levels deep
nested list of Python scalars.

Examples

For Series

>>> s = pd.Series([1, 2, 3])

>>> s.to_list()

[1, 2, 3]

For Index:

>>> idx = pd.Index([1, 2, 3])

>>> idx

Index([1, 2, 3], dtype='int64')

>>> idx.to_list()

[1, 2, 3]

to_numpy(

self,

dtype: npt.DTypeLike | None = None,

copy: bool = False,

na_value: object = <no_default>,

**kwargs

) -> np.ndarray

A NumPy ndarray representing the values in this Series or Index.

Parameters

```

-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.
**kwargs
    Additional keywords passed through to the ``to_numpy`` method
    of the underlying array (for extension arrays).

Returns
-----
numpy.ndarray
    The NumPy ndarray holding the values from this Series or Index.
    The dtype of the array may differ. See Notes.

See Also
-----
Series.array : Get the actual data stored within.
Index.array : Get the actual data stored within.
DataFrame.to_numpy : Similar method for DataFrame.

Notes
-----
The returned array will be the same up to equality (values equal
in `self` will be equal in the returned array; likewise for values
that are not equal). When `self` contains an ExtensionArray, the
dtype may be different. For example, for a category-dtype Series,
``to_numpy()`` will return a NumPy array and the categorical dtype
will be lost.

For NumPy dtypes, this will be a reference to the actual data stored
in this Series or Index (assuming ``copy=False``). Modifying the result
in place will modify the data stored in the Series or Index (not that
we recommend doing that).

For extension types, ``to_numpy()`` *may* require copying data and
coercing the result to a NumPy type (possibly object), which may be
expensive. When you need a no-copy reference to the underlying data,
:attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of
``to_numpy()`` for various dtypes within pandas.

=====
dtype          array type
=====
category[T]    ndarray[T] (same dtype as input)
period         ndarray[object] (Periods)
interval       ndarray[object] (Intervals)
IntegerNA      ndarray[object]
datetime64[ns] datetime64[ns]
datetime64[ns, tz] ndarray[object] (Timestamps)
=====

Examples
-----
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)

Specify the `dtype` to control how datetime-aware data is represented.
Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp`
objects, each with the correct ``tz``.

>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)

Or ``dtype='datetime64[ns]'`` to return an ndarray of native

```

datetime64 values. The values are converted to UTC and the timezone info is dropped.

```
>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')
```

transpose(self, *args, **kwargs) -> Self
Return the transpose, which is by definition self.

Returns

%(klass)s

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T
Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.T
0      Ant
1      Bear
2      Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty
Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool
If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.
Series.shape: Return a tuple of the shape of the underlying data.
Series.dtype: Return the dtype object of the underlying data.
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)

Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
   height  weight
elk     1.5    500
moose    2.6    800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
      height  weight
elk        3.0   501.5
moose      4.1   801.5
```

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0   501.5
moose      3.1   801.5
```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0   501.5
moose      3.1   801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0   500.5
moose      4.1   800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk height moose weight
elk   NaN    NaN   NaN    NaN
moose NaN    NaN   NaN    NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0   500.5
moose      4.1   801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN    NaN
elk       1.7    NaN
moose     3.0    NaN
```

```
__and__(self, other)

__divmod__(self, other)

__eq__(self, other)
    Return self==value.

__ge__(self, other)
    Return self>=value.

__gt__(self, other)
    Return self>value.

__le__(self, other)
    Return self<=value.

__lt__(self, other)
    Return self<value.

__mod__(self, other)
```

```

    __mul__(self, other)

    __ne__(self, other)
        Return self!=value.

    __or__(self, other)
        Return self|value.

    __pow__(self, other)

    __radd__(self, other)

    __rand__(self, other)

    __rdivmod__(self, other)

    __rfloordiv__(self, other)

    __rmod__(self, other)

    __rmul__(self, other)

    __ror__(self, other)
        Return value|self.

    __rpow__(self, other)

    __rsub__(self, other)

    __rtruediv__(self, other)

    __rxor__(self, other)

    __sub__(self, other)

    __truediv__(self, other)

    __xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
    __dict__
        dictionary for instance variables
    __weakref__
        list of weak references to the object
-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
    __hash__ = None
-----
Methods inherited from pandas.core.base.PandasObject:
    __sizeof__(self) -> int
        Generates the total memory usage for an object that returns
        either a value or Series of values
-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
    __dir__(self) -> list[str]
        Provide method name lookup and completion.

    Notes
    -----
    Only provide 'public' methods.

```

```

class Series(pandas.core.base.IndexOpsMixin, pandas.core.generic.NDFrame)
    Series(
        data=None,
        index=None,
        dtype: Dtype | None = None,
        name=None,

```



```
copy: bool | None = None
) -> None
```

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude missing data (currently represented as NaN).

Operations between Series (+, -, /, *, **) align values based on their associated index values-- they need not be the same length. The result index will be the sorted union of the two indexes.

Parameters

data : array-like, Iterable, dict, or scalar value
Contains data stored in Series. If data is a dict, argument order is maintained. Unordered sets are not supported.
index : array-like or Index (1d)
Values must be hashable and have the same length as `data`.
Non-unique index values are allowed. Will default to
RangeIndex (0, 1, 2, ..., n) if not provided. If data is dict-like
and index is None, then the keys in the data are used as the index. If the
index is not None, the resulting Series is reindexed with the index values.
dtype : str, numpy.dtype, or ExtensionDtype, optional
Data type for the output Series. If not specified, this will be
inferred from `data`.
See the :ref:`user guide <basics.dtypes>` for more usages.
name : Hashable, default None
The name to give to the Series.
copy : bool, default None
Whether to copy input data, only relevant for array, Series, and Index
inputs (for other input, e.g. a list, a new array is created anyway).
Defaults to True for array input and False for Index/Series.
Even when False for Index/Series, a shallow copy of the data is made.
Set to False to avoid copying array input at your own risk (if you
know the input data won't be modified elsewhere).
Set to True to force copying Series/Index input up front.

See Also

DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.
Index : Immutable sequence used for indexing and alignment.

Notes

Please reference the :ref:`User Guide <basics.series>` for more information.

Examples

Constructing Series from a dictionary with an Index specified

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> ser = pd.Series(data=d, index=["a", "b", "c"])
>>> ser
a    1
b    2
c    3
dtype: int64
```

The keys of the dictionary match with the Index values, hence the Index values have no effect.

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> ser = pd.Series(data=d, index=["x", "y", "z"])
>>> ser
x    NaN
y    NaN
z    NaN
dtype: float64
```

Note that the Index is first built with the keys from the dictionary. After this the Series is reindexed with the given Index values, hence we get all NaN as a result.

Constructing Series from a list with `copy=False`.

```
>>> r = [1, 2]
>>> ser = pd.Series(r, copy=False)
>>> ser.iloc[0] = 999
>>> r
[1, 2]
>>> ser
0    999
1     2
dtype: int64
```

Due to input data type the Series has a `copy` of the original data even though `copy=False`, so the data is unchanged.

Constructing Series from a 1d ndarray with `copy=False`.

```
>>> r = np.array([1, 2])
>>> ser = pd.Series(r, copy=False)
>>> ser.iloc[0] = 999
>>> r
array([999, 2])
>>> ser
0    999
1     2
dtype: int64
```

Due to input data type the Series has a `view` on the original data, so the data is changed as well.

Method resolution order:

```
Series
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.generic.NDFrame
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
pandas.core.indexing.IndexingMixin
builtins.object
```

Methods defined here:

```
__array__(self, dtype: npt.DTypeLike | None = None, copy: bool | None = None) -> np.ndarray
Return the values as a NumPy array.
```

Users should not call this directly. Rather, it is invoked by :func:`numpy.array` and :func:`numpy.asarray`.

Parameters

dtype : str or numpy.dtype, optional

The dtype to use for the resulting NumPy array. By default, the dtype is inferred from the data.

copy : bool or None, optional

See :func:`numpy.asarray`.

Returns

numpy.ndarray

The values in the series converted to a :class:`numpy.ndarray` with the specified `dtype`.

See Also

array : Create a new array from data.

Series.array : Zero-copy view to the array backing the Series.

Series.to_numpy : Series method for similar behavior.

Examples

```
>>> ser = pd.Series([1, 2, 3])
>>> np.asarray(ser)
array([1, 2, 3])
```

For timezone-aware data, the timezones may be retained with
``dtype='object'``

```
>>> tzser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> np.asarray(tzser, dtype="object")
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)
```

Or the values may be localized to UTC and the tzinfo discarded with
``dtype='datetime64[ns]'``

```
>>> np.asarray(tzser, dtype="datetime64[ns]") # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', ...],
      dtype='datetime64[ns]')
```

__arrow_c_stream__(self, requested_schema=None) from pandas.core.series.Series
Export the pandas Series as an Arrow C stream PyCapsule.

This relies on pyarrow to convert the pandas Series to the Arrow format (and follows the default behavior of ``pyarrow.Array.from\_pandas`` in its handling of the index, i.e. to ignore it).
This conversion is not necessarily zero-copy.

Parameters

requested_schema : PyCapsule, default None
The schema to which the dataframe should be casted, passed as a PyCapsule containing a C ArrowSchema representation of the requested schema.

Returns

PyCapsule

__getitem__(self, key) from pandas.core.series.Series

__init__(
self,
data=None,
index=None,
dtype: Dtype | None = None,
name=None,
copy: bool | None = None
) -> None from pandas.core.series.Series
Initialize self. See help(type(self)) for accurate signature.

__len__(self) -> int from pandas.core.series.Series
Return the length of the Series.

__matmul__(self, other) from pandas.core.series.Series
Matrix multiplication using binary ``@`` operator.

__repr__(self) -> str from pandas.core.series.Series
Return a string representation for a particular Series.

__rmatmul__(self, other) from pandas.core.series.Series
Matrix multiplication using binary ``@`` operator.

__setitem__(self, key, value) -> None from pandas.core.series.Series

add(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.ser.
Return Addition of series and other, element-wise (binary operator ``add``).

Equivalent to ``series + other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : Series or scalar value
With which to compute the addition.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.radd : Reverse of the Addition operator, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
```

```
>>> b
```

```
a    1.0
```

```
b    NaN
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.add(b, fill_value=0)
```

```
a    2.0
```

```
b    1.0
```

```
c    1.0
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
agg = aggregate(self, func=None, axis: Axis = 0, *args, **kwargs)
```

aggregate(self, func=None, axis: Axis = 0, *args, **kwargs) from pandas.core.series.Series
Aggregate using one or more operations over the specified axis.

Parameters

func : function, str, list or dict

Function to use for aggregating the data. If a function, must either work when passed a Series or when passed to Series.apply.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. ``[np.sum, 'mean']``
- dict of axis labels -> functions, function names or list of such.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

*args

Positional arguments to pass to `func`.

**kwargs

Keyword arguments to pass to `func`.

Returns

scalar, Series or DataFrame

The return can be:

- * scalar : when Series.agg is called with single function
- * Series : when DataFrame.agg is called with a single function
- * DataFrame : when DataFrame.agg is called with several functions

See Also

Series.apply : Invoke function on a Series.

Series.transform : Transform function producing a Series with like indexes.

Notes

The aggregation operations are always performed over an axis, either the index (default) or the column axis. This behavior is different from `numpy` aggregation functions (`mean`, `median`, `prod`, `sum`, `std`, `var`), where the default is to compute the aggregation of the flattened array, e.g., `numpy.mean(arr_2d)` as opposed to `numpy.mean(arr_2d, axis=0)`.

`agg` is an alias for `aggregate`. Use the alias.

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

A passed user-defined-function will be passed a Series for evaluation.

If `func` defines an index relabeling, `axis` must be `0` or `index`.

Examples

```
>>> s = pd.Series([1, 2, 3, 4])
>>> s
0    1
1    2
2    3
3    4
dtype: int64
```

```
>>> s.agg("min")
1
```

```
>>> s.agg(["min", "max"])
min    1
max    4
dtype: int64
```

```
all(
    self,
    *,
    axis: Axis = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> bool from pandas.core.series.Series
Return whether all elements are True, potentially over an axis.
```

Returns True unless there at least one element within a series or along a Dataframe axis that is False or equivalent (e.g. zero or empty).

Parameters

`axis` : {0 or 'index', 1 or 'columns', None}, default 0
Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

- * 0 / 'index' : reduce the index, return a Series whose index is the original column labels.
- * 1 / 'columns' : reduce the columns, return a Series whose index is the original index.
- * None : reduce all axes, return a scalar.

`bool_only` : bool, default False

Include only boolean columns. Not implemented for Series.

`skipna` : bool, default True

Exclude NA/null values. If the entire row/column is NA and skipna is True, then the result will be True, as for an empty row/column.

If skipna is False, then NA are treated as True, because these are not equal to zero.

`**kwargs` : any, default None

Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or scalar
If axis=None, then a scalar boolean is returned.
Otherwise a Series is returned with index matching the index argument.

See Also

Series.all : Return True if all elements are True.
DataFrame.any : Return True if one (or more) elements are True.

Examples

Series

```
>>> pd.Series([True, True]).all()
True
>>> pd.Series([True, False]).all()
False
>>> pd.Series([], dtype="float64").all()
True
>>> pd.Series([np.nan]).all()
True
>>> pd.Series([np.nan]).all(skipna=False)
True
```

DataFrames

Create a DataFrame from a dictionary.

```
>>> df = pd.DataFrame({"col1": [True, True], "col2": [True, False]})
>>> df
   col1  col2
0  True  True
1  True False
```

Default behaviour checks if values in each column all return True.

```
>>> df.all()
col1    True
col2   False
dtype: bool
```

Specify ``axis='columns'`` to check if values in each row all return True.

```
>>> df.all(axis="columns")
0    True
1   False
dtype: bool
```

Or ``axis=None`` for whether every value is True.

```
>>> df.all(axis=None)
False
```

```
any(
    self,
    *,
    axis: Axis = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> bool from pandas.core.series.Series
Return whether any element is True, potentially over an axis.
```

Returns False unless there is at least one element within a series or along a DataFrame axis that is True or equivalent (e.g. non-zero or non-empty).

Parameters

axis : {0 or 'index', 1 or 'columns', None}, default 0
Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

* 0 / 'index' : reduce the index, return a Series whose index is the original column labels.

* 1 / 'columns' : reduce the columns, return a Series whose index is the

```

        original index.
    * None : reduce all axes, return a scalar.

bool_only : bool, default False
    Include only boolean columns. Not implemented for Series.
skipna : bool, default True
    Exclude NA/null values. If the entire row/column is NA and skipna is
    True, then the result will be False, as for an empty row/column.
    If skipna is False, then NA are treated as True, because these are not
    equal to zero.
**kwargs : any, default None
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.

Returns
-----
Series or scalar
    If axis=None, then a scalar boolean is returned.
    Otherwise a Series is returned with index matching the index argument.

See Also
-----
numpy.any : Numpy version of this method.
Series.any : Return whether any element is True.
Series.all : Return whether all elements are True.
DataFrame.any : Return whether any element is True over requested axis.
DataFrame.all : Return whether all elements are True over requested axis.

Examples
-----
**Series**

For Series input, the output is a scalar indicating whether any element
is True.

>>> pd.Series([False, False]).any()
False
>>> pd.Series([True, False]).any()
True
>>> pd.Series([], dtype="float64").any()
False
>>> pd.Series([np.nan]).any()
False
>>> pd.Series([np.nan]).any(skipna=False)
True

**DataFrame**

Whether each column contains at least one True element (the default).

>>> df = pd.DataFrame({"A": [1, 2], "B": [0, 2], "C": [0, 0]})
>>> df
   A  B  C
0  1  0  0
1  2  2  0

>>> df.any()
A     True
B     True
C     False
dtype: bool

Aggregating over the columns.

>>> df = pd.DataFrame({"A": [True, False], "B": [1, 2]})
>>> df
   A  B
0  True  1
1 False  2

>>> df.any(axis="columns")
0     True
1     True
dtype: bool

>>> df = pd.DataFrame({"A": [True, False], "B": [1, 0]})
>>> df

```

```

      A  B
0   True  1
1  False  0

```

```

>>> df.any(axis="columns")
0    True
1   False
dtype: bool

```

Aggregating over the entire DataFrame with ``axis=None``.

```

>>> df.any(axis=None)
True

```

``any`` for an empty DataFrame is an empty Series.

```

>>> pd.DataFrame([]).any()
Series([], dtype: bool)

```

```

apply(
    self,
    func: AggFuncType,
    args: tuple[Any, ...] = (),
    *,
    by_row: Literal[False, 'compat'] = 'compat',
    **kwargs
) -> DataFrame | Series from pandas.core.series.Series
    Invoke function on values of Series.

```

Can be ufunc (a NumPy function that applies to the entire Series) or a Python function that only works on single values.

Parameters

```

func : function
    Python function or NumPy ufunc to apply.
args : tuple
    Positional arguments passed to func after the series value.
by_row : False or "compat", default "compat"
    If ``"compat"`` and func is a callable, func will be passed each element of
    the Series, like ``Series.map``. If func is a list or dict of
    callables, will first try to translate each func into pandas methods. If
    that doesn't work, will try call to apply again with ``by_row="compat"``
    and if that fails, will call apply again with ``by_row=False``
    (backward compatible).
    If False, the func will be passed the whole Series at once.

    ``by_row`` has no effect when ``func`` is a string.

```

```

.. versionadded:: 2.1.0

```

```

**kwargs
    Additional keyword arguments passed to func.

```

Returns

```

Series or DataFrame
    If func returns a Series object the result will be a DataFrame.

```

See Also

```

Series.map: For element-wise operations.
Series.agg: Only perform aggregating type operations.
Series.transform: Only perform transforming type operations.

```

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

Examples

Create a series with typical summer temperatures for each city.

```

>>> s = pd.Series([20, 21, 12], index=["London", "New York", "Helsinki"])
>>> s
London      20

```



```
New York    21
Helsinki    12
dtype: int64
```

Square the values by defining a function and passing it as an argument to ``apply``.

```
>>> def square(x):
...     return x**2
>>> s.apply(square)
London      400
New York    441
Helsinki    144
dtype: int64
```

Square the values by passing an anonymous function as an argument to ``apply``.

```
>>> s.apply(lambda x: x**2)
London      400
New York    441
Helsinki    144
dtype: int64
```

Define a custom function that needs additional positional arguments and pass these additional arguments using the ``args`` keyword.

```
>>> def subtract_custom_value(x, custom_value):
...     return x - custom_value

>>> s.apply(subtract_custom_value, args=(5,))
London      15
New York    16
Helsinki     7
dtype: int64
```

Define a custom function that takes keyword arguments and pass these arguments to ``apply``.

```
>>> def add_custom_values(x, **kwargs):
...     for month in kwargs:
...         x += kwargs[month]
...     return x

>>> s.apply(add_custom_values, june=30, july=20, august=25)
London      95
New York    96
Helsinki    87
dtype: int64
```

Use a function from the Numpy library.

```
>>> s.apply(np.log)
London      2.995732
New York    3.044522
Helsinki    2.484907
dtype: float64
```

```
argsort(
    self,
    axis: Axis = 0,
    kind: SortKind = 'quicksort',
    order: None = None,
    stable: None = None
) -> Series from pandas.core.series.Series
Return the integer indices that would sort the Series values.
```

Override ndarray.argsort. Arg sorts the value, omitting NA/null values, and places the result in the same locations as the non-NA values.

Parameters

```
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
kind : {'mergesort', 'quicksort', 'heapsort', 'stable'}, default 'quicksort'
    Choice of sorting algorithm. See :func:`numpy.sort` for more
```

information. 'mergesort' and 'stable' are the only stable algorithms.
order : None
Has no effect but is accepted for compatibility with numpy.
stable : None
Has no effect but is accepted for compatibility with numpy.

Returns

Series[np.intp]
Positions of values within the sort order with -1 indicating
nan values.

See Also

numpy.ndarray.argsort : Returns the indices that would sort this array.

Examples

```
>>> s = pd.Series([3, 2, 1])
>>> s.argsort()
0    2
1    1
2    0
dtype: int64
```

autocorr(self, lag: int = 1) -> float from pandas.core.series.Series
Compute the lag-N autocorrelation.

This method computes the Pearson correlation between
the Series and its shifted self.

Parameters

lag : int, default 1
Number of lags to apply before performing autocorrelation.

Returns

float
The Pearson correlation between self and self.shift(lag).

See Also

Series.corr : Compute the correlation between two Series.
Series.shift : Shift index by desired number of periods.
DataFrame.corr : Compute pairwise correlation of columns.
DataFrame.corrwith : Compute pairwise correlation between rows or
columns of two DataFrame objects.

Notes

If the Pearson correlation is not well defined return 'NaN'.

Examples

```
>>> s = pd.Series([0.25, 0.5, 0.2, -0.05])
>>> s.autocorr() # doctest: +ELLIPSIS
0.10355...
>>> s.autocorr(lag=2) # doctest: +ELLIPSIS
-0.99999...
```

If the Pearson correlation is not well defined, then 'NaN' is returned.

```
>>> s = pd.Series([1, 0, 0, 0])
>>> s.autocorr()
nan
```

between(
self,
left,
right,
inclusive: Literal['both', 'neither', 'left', 'right'] = 'both'
) -> Series from pandas.core.series.Series
Return boolean Series equivalent to left <= series <= right.

This function returns a boolean vector containing `True` wherever the
corresponding Series element is between the boundary values `left` and

`right`. NA values are treated as `False`.

Parameters

left : scalar or list-like
Left boundary.
right : scalar or list-like
Right boundary.
inclusive : {"both", "neither", "left", "right"}
Include boundaries. Whether to set each bound as closed or open.

Returns

Series
Series representing whether each element is between left and right (inclusive).

See Also

Series.gt : Greater than of series and other.
Series.lt : Less than of series and other.

Notes

This function is equivalent to ```(left <= ser) & (ser <= right)```

Examples

>>> s = pd.Series([2, 0, 4, 8, np.nan])

Boundary values are included by default:

```
>>> s.between(1, 4)
0    True
1    False
2    True
3    False
4    False
dtype: bool
```

With `inclusive` set to ```"neither"``` boundary values are excluded:

```
>>> s.between(1, 4, inclusive="neither")
0    True
1    False
2    False
3    False
4    False
dtype: bool
```

`left` and `right` can be any scalar value:

```
>>> s = pd.Series(["Alice", "Bob", "Carol", "Eve"])
>>> s.between("Anna", "Daniel")
0    False
1    True
2    True
3    False
dtype: bool
```

```
case_when(
    self,
    caselist: list[tuple[ArrayLike | Callable[[Series], Series | np.ndarray | Sequence[bool]]],
) -> Series from pandas.core.series.Series
Replace values where the conditions are True.
```

.. versionadded:: 2.2.0

Parameters

caselist : A list of tuples of conditions and expected replacements
Takes the form: ```(condition0, replacement0)```,
```(condition1, replacement1)```, ...  
```condition``` should be a 1-D boolean array-like object  
or a callable. If ```condition``` is a callable,
it is computed on the Series
and should return a boolean Series or array.

The callable must not change the input Series (though pandas doesn't check it). ``replacement`` should be a 1-D array-like object, a scalar or a callable. If ``replacement`` is a callable, it is computed on the Series and should return a scalar or Series. The callable must not change the input Series (though pandas doesn't check it).

Returns

Series

A new Series with values replaced based on the provided conditions.

See Also

`Series.mask` : Replace values where the condition is True.

Examples

```
>>> c = pd.Series([6, 7, 8, 9], name="c")
>>> a = pd.Series([0, 0, 1, 2])
>>> b = pd.Series([0, 3, 4, 5])
```

```
>>> c.case_when(
...     caselist=[
...         (a.gt(0), a), # condition, replacement
...         (b.gt(0), b),
...     ]
... )
0    6
1    3
2    1
3    2
Name: c, dtype: int64
```

```
combine(
    self,
    other: Series | Hashable,
    func: Callable[[Hashable, Hashable], Hashable],
    fill_value: Hashable | None = None
) -> Series from pandas.core.series.Series
Combine the Series with a Series or scalar according to `func`.
```

Combine the Series and `other` using `func` to perform elementwise selection for combined Series. `fill\_value` is assumed when value is not present at some index from one of the two Series being combined.

Parameters

`other` : Series or scalar

The value(s) to be combined with the `Series`.

`func` : function

Function that takes two scalars as inputs and returns an element.

`fill_value` : scalar, optional

The value to assume when an index is missing from one Series or the other. The default specifies to use the appropriate NaN value for the underlying dtype of the Series.

Returns

Series

The result of combining the Series with the other object.

See Also

`Series.combine_first` : Combine Series values, choosing the calling Series' values first.

Examples

Consider 2 Datasets ``s1`` and ``s2`` containing highest clocked speeds of different birds.

```
>>> s1 = pd.Series({"falcon": 330.0, "eagle": 160.0})
>>> s1
falcon    330.0
```

```

eagle      160.0
dtype: float64
>>> s2 = pd.Series({"falcon": 345.0, "eagle": 200.0, "duck": 30.0})
>>> s2
falcon      345.0
eagle       200.0
duck        30.0
dtype: float64

```

Now, to combine the two datasets and view the highest speeds of the birds across the two datasets

```

>>> s1.combine(s2, max)
duck      NaN
eagle     200.0
falcon    345.0
dtype: float64

```

In the previous example, the resulting value for duck is missing, because the maximum of a NaN and a float is a NaN. So, in the example, we set ``fill\_value=0``, so the maximum value returned will be the value from some dataset.

```

>>> s1.combine(s2, max, fill_value=0)
duck      30.0
eagle     200.0
falcon    345.0
dtype: float64

```

`combine_first(self, other)` -> Series from `pandas.core.series.Series`
Update null elements with value in the same location in 'other'.

Combine two Series objects by filling null values in one Series with non-null values from the other Series. Result index will be the union of the two indexes.

Parameters

other : Series
The value(s) to be used for filling null values.

Returns

Series
The result of combining the provided Series with the other object.

See Also

`Series.combine` : Perform element-wise operation on two Series using a given function.

Examples

```

>>> s1 = pd.Series([1, np.nan])
>>> s2 = pd.Series([3, 4, 5])
>>> s1.combine_first(s2)
0    1.0
1    4.0
2    5.0
dtype: float64

```

Null values still persist if the location of that null value does not exist in 'other'

```

>>> s1 = pd.Series({"falcon": np.nan, "eagle": 160.0})
>>> s2 = pd.Series({"eagle": 200.0, "duck": 30.0})
>>> s1.combine_first(s2)
duck      30.0
eagle     160.0
falcon     NaN
dtype: float64

```

```

compare(
    self,
    other: Series,
    align_axis: Axis = 1,
    keep_shape: bool = False,

```

```

    keep_equal: bool = False,
    result_names: Suffixes = ('self', 'other')
) -> DataFrame | Series from pandas.core.series.Series
    Compare to another Series and show the differences.

Parameters
-----
other : Series
    Object to compare with.

align_axis : {{0 or 'index', 1 or 'columns'}}, default 1
    Determine which axis to align the comparison on.

    * 0, or 'index' : Resulting differences are stacked vertically
      with rows drawn alternately from self and other.
    * 1, or 'columns' : Resulting differences are aligned horizontally
      with columns drawn alternately from self and other.

keep_shape : bool, default False
    If true, all rows and columns are kept.
    Otherwise, only the ones with different values are kept.

keep_equal : bool, default False
    If true, the result keeps values that are equal.
    Otherwise, equal values are shown as NaNs.

result_names : tuple, default ('self', 'other')
    Set the dataframes names in the comparison.

Returns
-----
Series or DataFrame
    If axis is 0 or 'index' the result will be a Series.
    The resulting index will be a MultiIndex with 'self' and 'other'
    stacked alternately at the inner level.

    If axis is 1 or 'columns' the result will be a DataFrame.
    It will have two columns namely 'self' and 'other'.

See Also
-----
DataFrame.compare : Compare with another DataFrame and show differences.

Notes
-----
Matching NaNs will not appear as a difference.

Examples
-----
>>> s1 = pd.Series(["a", "b", "c", "d", "e"])
>>> s2 = pd.Series(["a", "a", "c", "b", "e"])

Align the differences on columns

>>> s1.compare(s2)
   self other
1    b     a
3    d     b

Stack the differences on indices

>>> s1.compare(s2, align_axis=0)
   self  other
1  self    b
  other    a
3  self    d
  other    b
dtype: str

Keep all original rows

>>> s1.compare(s2, keep_shape=True)
   self  other
0  NaN    NaN
1    b     a
2  NaN    NaN
3    d     b
4  NaN    NaN

```

Keep all original rows and also all original values

```
>>> s1.compare(s2, keep_shape=True, keep_equal=True)
self other
0      a      a
1      b      a
2      c      c
3      d      b
4      e      e
```

```
corr(
    self,
    other: Series,
    method: CorrelationMethod = 'pearson',
    min_periods: int | None = None
) -> float from pandas.core.series.Series
Compute correlation with `other` Series, excluding missing values.
```

The two `Series` objects are not required to be the same length and will be aligned internally before the correlation function is applied.

Parameters

other : Series

Series with which to compute the correlation.

method : {'pearson', 'kendall', 'spearman'} or callable

Method used to compute correlation:

- pearson : Standard correlation coefficient
- kendall : Kendall Tau correlation coefficient
- spearman : Spearman rank correlation
- callable: Callable with input two 1d ndarrays and returning a float.

.. warning::

Note that the returned matrix from corr will have 1 along the diagonals and will be symmetric regardless of the callable's behavior.

min_periods : int, optional

Minimum number of observations needed to have a valid result.

Returns

float

Correlation with other.

See Also

DataFrame.corr : Compute pairwise correlation between columns.

DataFrame.corrwith : Compute pairwise correlation with another DataFrame or Series.

Notes

Pearson, Kendall and Spearman correlation are currently computed using pairwise complete

* `Pearson correlation coefficient <https://en.wikipedia.org/wiki/Pearson_correlation_coefficient

* `Kendall rank correlation coefficient <https://en.wikipedia.org/wiki/Kendall_rank_correlation_coefficient

* `Spearman's rank correlation coefficient <https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient

Automatic data alignment: as with all pandas operations, automatic data alignment is performed.

`corr()` automatically considers values with matching indices.

Examples

```
>>> def histogram_intersection(a, b):
...     v = np.minimum(a, b).sum().round(decimals=1)
...     return v
>>> s1 = pd.Series([0.2, 0.0, 0.6, 0.2])
>>> s2 = pd.Series([0.3, 0.6, 0.0, 0.1])
>>> s1.corr(s2, method=histogram_intersection)
0.3
```

Pandas auto-aligns the values with matching indices

```
>>> s1 = pd.Series([1, 2, 3], index=[0, 1, 2])
>>> s2 = pd.Series([1, 2, 3], index=[2, 1, 0])
```

```
>>> s1.corr(s2)
-1.0
```

If the input is a constant array, the correlation is not defined in this case, and ``np.nan`` is returned.

```
>>> s1 = pd.Series([0.45, 0.45])
>>> s1.corr(s1)
nan
```

`count(self) -> int` from `pandas.core.series.Series`
Return number of non-NA/null observations in the Series.

Returns

int
Number of non-null values in the Series.

See Also

`DataFrame.count` : Count non-NA cells for each column or row.

Examples

```
>>> s = pd.Series([0.0, 1.0, np.nan])
>>> s.count()
2
```

`cov(self, other: Series, min_periods: int | None = None, ddof: int | None = 1) -> float` from
Compute covariance with Series, excluding missing values.

The two `Series` objects are not required to be the same length and will be aligned internally before the covariance is calculated.

Parameters

`other` : Series
Series with which to compute the covariance.
`min_periods` : int, optional
Minimum number of observations needed to have a valid result.
`ddof` : int, default 1
Delta degrees of freedom. The divisor used in calculations is ``N - ddof``, where ``N`` represents the number of elements.

Returns

float
Covariance between Series and other normalized by N-1 (unbiased estimator).

See Also

`DataFrame.cov` : Compute pairwise covariance of columns.

Examples

```
>>> s1 = pd.Series([0.90010907, 0.13484424, 0.62036035])
>>> s2 = pd.Series([0.12528585, 0.26962463, 0.51111198])
>>> s1.cov(s2)
-0.01685762652715874
```

`cummax(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core.`
Return cumulative maximum over a Series.

Returns a Series of the same size containing the cumulative maximum.

Parameters

`axis` : {0 or 'index'}, default 0
This parameter is unused and defaults to 0.
`skipna` : bool, default True
Exclude NA/null values. If the series is NA, the result is NA.
`*args, **kwargs`
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series

Return cumulative maximum of Series.

See Also

core.window.expanding.Expanding.max : Similar functionality
but ignores ``NaN`` values.

Series.max : Return the maximum over a Series.

Series.cummin : Return cumulative minimum.

Series.cumsum : Return cumulative sum.

Series.cumprod : Return cumulative product.

Examples

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
```

```
>>> s
```

```
0    2.0
```

```
1    NaN
```

```
2    5.0
```

```
3   -1.0
```

```
4    0.0
```

```
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummax()
```

```
0    2.0
```

```
1    NaN
```

```
2    5.0
```

```
3    5.0
```

```
4    5.0
```

```
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummax(skipna=False)
```

```
0    2.0
```

```
1    NaN
```

```
2    NaN
```

```
3    NaN
```

```
4    NaN
```

```
dtype: float64
```

cummin(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self from pandas.core.

Return cumulative minimum over a Series.

Returns a Series of the same size containing the cumulative
minimum.

Parameters

axis : {0 or 'index'}, default 0

This parameter is unused and defaults to 0.

skipna : bool, default True

If the entire series is NA, the result will be NA.

*args, **kwargs

Additional keywords have no effect but might be accepted for
compatibility with NumPy.

Returns

Series

Return cumulative minimum of the Series.

See Also

core.window.expanding.Expanding.min : Similar functionality
but ignores ``NaN`` values.

Series.min : Return the minimum value of the Series.

Series.cummax : Return cumulative maximum.

Series.cumsum : Return cumulative sum.

Series.cumprod : Return cumulative product.

Examples

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummin()
0    2.0
1    NaN
2    2.0
3   -1.0
4   -1.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummin(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

`cumprod(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core`
Return cumulative product over a Series.

Returns a Series of the same size containing the cumulative product.

Parameters

```
-----
axis : {0 or 'index'}, default 0
    This parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If entire Series is NA, the result will be NA.
*args, **kwargs
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.
```

Returns

```
-----
Series
    Return cumulative product of Series.
```

See Also

```
-----
core.window.expanding.Expanding.prod : Similar functionality
    but ignores ``NaN`` values.
Series.prod : Return the product over Series.
Series.cummax : Return cumulative maximum.
Series.cummin : Return cumulative minimum.
Series.cumsum : Return cumulative sum.
```

Examples

```
-----
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cumprod()
0    2.0
1    NaN
2   10.0
3  -10.0
```

```
4    -0.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cumprod(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

`cumsum(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core.`
Return cumulative sum over a Series.

Returns a Series of the same size containing the cumulative sum.

Parameters

`axis` : {0 or 'index'}, default 0

This parameter is unused and defaults to 0.

`skipna` : bool, default True

Exclude NA/null values. If entire series is NA, the result will be NA.

`*args, **kwargs`

Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series

Return cumulative sum of Series.

See Also

`core.window.expanding.Expanding.sum` : Similar functionality but ignores ``NaN`` values.

`Series.sum` : Return the sum over Series.

`Series.cummax` : Return cumulative maximum.

`Series.cummin` : Return cumulative minimum.

`Series.cumprod` : Return cumulative product.

Examples

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cumsum()
0    2.0
1    NaN
2    7.0
3    6.0
4    6.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cumsum(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

`diff(self, periods: int = 1) -> Series` from `pandas.core.series.Series`
First discrete difference of Series elements.

Calculates the difference of a Series element compared with another

element in the Series (default is element in previous row).

Parameters

`periods` : int, default 1
Periods to shift for calculating difference, accepts negative values.

Returns

Series
First differences of the Series.

See Also

`Series.pct_change`: Percent change over given number of periods.
`Series.shift`: Shift index by desired number of periods with an optional time freq.
`DataFrame.diff`: First discrete difference of object.

Notes

For boolean dtypes, this uses `:meth:`operator.xor`` rather than `:meth:`operator.sub``.
The result is calculated according to current dtype in Series, however dtype of the result is always float64.

Examples

Difference with previous row

```
>>> s = pd.Series([1, 1, 2, 3, 5, 8])
>>> s.diff()
0    NaN
1    0.0
2    1.0
3    1.0
4    2.0
5    3.0
dtype: float64
```

Difference with 3rd previous row

```
>>> s.diff(periods=3)
0    NaN
1    NaN
2    NaN
3    2.0
4    4.0
5    6.0
dtype: float64
```

Difference with following row

```
>>> s.diff(periods=-1)
0    0.0
1   -1.0
2   -1.0
3   -2.0
4   -3.0
5    NaN
dtype: float64
```

Overflow in input dtype

```
>>> s = pd.Series([1, 0], dtype=np.uint8)
>>> s.diff()
0    NaN
1   255.0
dtype: float64
```

`div = truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

`divide = truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

`divmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.`

Return Integer division and modulo of series and other, element-wise (binary operator ``d``).
Equivalent to ``divmod(series, other)``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
`level` : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

2-Tuple of Series
The result of the operation.

See Also

`Series.rdivmod` : Reverse of the Integer division and modulo operator, see ``Python documentation <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`` for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.divmod(b, fill_value=0)
(a    1.0
 b    inf
 c    inf
 d    0.0
 e    NaN
dtype: float64,
 a    0.0
 b    NaN
 c    NaN
 d    0.0
 e    NaN
dtype: float64)
```

`dot(self, other: AnyArrayLike | DataFrame) -> Series | np.ndarray from pandas.core.series.Series`
Compute the dot product between the Series and the columns of other.

This method computes the dot product between the Series and another one, or the Series and each columns of a DataFrame, or the Series and each columns of an array.

It can also be called using ``self @ other``.

Parameters

`other` : Series, DataFrame or array-like
The other object to compute the dot product with its columns.

Returns

scalar, Series or numpy.ndarray

Return the dot product of the Series and other if other is a Series, the Series of the dot product of Series and each rows of other if other is a DataFrame or a numpy.ndarray between the Series and each columns of the numpy array.

See Also

DataFrame.dot: Compute the matrix product with the DataFrame.

Series.mul: Multiplication of series and other, element-wise.

Notes

The Series and other has to share the same index if other is a Series or a DataFrame.

Examples

```
>>> s = pd.Series([0, 1, 2, 3])
>>> other = pd.Series([-1, 2, -3, 4])
>>> s.dot(other)
8
>>> s @ other
8
>>> df = pd.DataFrame([[0, 1], [-2, 3], [4, -5], [6, 7]])
>>> s.dot(df)
0    24
1    14
dtype: int64
>>> arr = np.array([[0, 1], [-2, 3], [4, -5], [6, 7]])
>>> s.dot(arr)
array([24, 14])
```

```
drop(
    self,
    labels: IndexLabel | ListLike = None,
    *,
    axis: Axis = 0,
    index: IndexLabel | ListLike = None,
    columns: IndexLabel | ListLike = None,
    level: Level | None = None,
    inplace: bool = False,
    errors: IgnoreRaise = 'raise'
) -> Series | None from pandas.core.series.Series
Return Series with specified index labels removed.
```

Remove elements of a Series based on specifying the index labels. When using a multi-index, labels on different levels can be removed by specifying the level.

Parameters

labels : single label or list-like

Index labels to drop.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

index : single label or list-like

Redundant for application on Series, but 'index' can be used instead of 'labels'.

columns : single label or list-like

No change is made to the Series; use 'index' or 'labels' instead.

level : int or level name, optional

For MultiIndex, level for which the labels will be removed.

inplace : bool, default False

If True, do operation inplace and return None.

errors : {'ignore', 'raise'}, default 'raise'

If 'ignore', suppress error and only existing labels are dropped.

Returns

Series or None

Series with specified index labels removed or None if ``inplace=True``.

Raises

```
-----
KeyError
    If none of the labels are found in the index.
```

See Also

```
-----
Series.reindex : Return only specified index labels of Series.
Series.dropna : Return series without null values.
Series.drop_duplicates : Return Series with duplicate values removed.
DataFrame.drop : Drop specified labels from rows or columns.
```

Examples

```
-----
>>> s = pd.Series(data=np.arange(3), index=["A", "B", "C"])
>>> s
A    0
B    1
C    2
dtype: int64
```

Drop labels B and C

```
>>> s.drop(labels=["B", "C"])
A    0
dtype: int64
```

Drop 2nd level label in MultiIndex Series

```
>>> midx = pd.MultiIndex(
...     levels=[["llama", "cow", "falcon"], ["speed", "weight", "length"]],
...     codes=[[0, 0, 0, 1, 1, 1, 2, 2, 2], [0, 1, 2, 0, 1, 2, 0, 1, 2]],
... )
>>> s = pd.Series([45, 200, 1.2, 30, 250, 1.5, 320, 1, 0.3], index=midx)
>>> s
llama  speed    45.0
       weight   200.0
       length    1.2
cow    speed    30.0
       weight   250.0
       length    1.5
falcon speed   320.0
       weight    1.0
       length    0.3
dtype: float64
```

```
>>> s.drop(labels="weight", level=1)
llama  speed    45.0
       length    1.2
cow    speed    30.0
       length    1.5
falcon speed   320.0
       length    0.3
dtype: float64
```

```
drop_duplicates(
    self,
    *,
    keep: DropKeep = 'first',
    inplace: bool = False,
    ignore_index: bool = False
) -> Series | None from pandas.core.series.Series
    Return Series with duplicate values removed.
```

Parameters

```
-----
keep : {'first', 'last', ``False``}, default 'first'
    Method to handle dropping duplicates:
```

- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

```
inplace : bool, default ``False``
    If ``True``, performs operation inplace and returns None.
```

```
ignore_index : bool, default ``False``
    If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.
```

```
.. versionadded:: 2.0.0
```

Returns

Series or None

Series with duplicates dropped or None if ``inplace=True``.

See Also

`Index.drop_duplicates` : Equivalent method on `Index`.

`DataFrame.drop_duplicates` : Equivalent method on `DataFrame`.

`Series.duplicated` : Related method on `Series`, indicating duplicate Series values.

`Series.unique` : Return unique values as an array.

Examples

Generate a Series with duplicated entries.

```
>>> s = pd.Series(
...     ["llama", "cow", "llama", "beetle", "llama", "hippo"], name="animal"
... )
>>> s
0    llama
1      cow
2    llama
3   beetle
4    llama
5    hippo
Name: animal, dtype: str
```

With the 'keep' parameter, the selection behavior of duplicated values can be changed. The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of keep is 'first'.

```
>>> s.drop_duplicates()
0    llama
1      cow
3   beetle
5    hippo
Name: animal, dtype: str
```

The value 'last' for parameter 'keep' keeps the last occurrence for each set of duplicated entries.

```
>>> s.drop_duplicates(keep="last")
1      cow
3   beetle
4    llama
5    hippo
Name: animal, dtype: str
```

The value ``False`` for parameter 'keep' discards all sets of duplicated entries.

```
>>> s.drop_duplicates(keep=False)
1      cow
3   beetle
5    hippo
Name: animal, dtype: str
```

```
dropna(
    self,
    *,
    axis: Axis = 0,
    inplace: bool = False,
    how: AnyAll | None = None,
    ignore_index: bool = False
) -> Series | None from pandas.core.series.Series
Return a new Series with missing values removed.
```

See the :ref:`User Guide <missing\_data>` for more on which values are considered missing, and how to work with missing data.

Parameters

```
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
inplace : bool, default False
    If True, do operation inplace and return None.
how : str, optional
    Not in use. Kept for compatibility.
ignore_index : bool, default ``False``
    If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

.. versionadded:: 2.0.0
```

Returns

```
-----
Series or None
    Series with NA entries dropped from it or None if ``inplace=True``.
```

See Also

```
-----
Series.isna: Indicate missing values.
Series.notna : Indicate existing (non-missing) values.
Series.fillna : Replace missing values.
DataFrame.dropna : Drop rows or columns which contain NA values.
Index.dropna : Drop missing indices.
```

Examples

```
-----
>>> ser = pd.Series([1.0, 2.0, np.nan])
>>> ser
0    1.0
1    2.0
2    NaN
dtype: float64
```

Drop NA values from a Series.

```
>>> ser.dropna()
0    1.0
1    2.0
dtype: float64
```

Empty strings are not considered NA values. ``None`` is considered an NA value.

```
>>> ser = pd.Series([np.nan, 2, pd.NaT, "", None, "I stay"])
>>> ser
0      NaN
1         2
2      NaT
3
4      None
5    I stay
dtype: object
>>> ser.dropna()
1         2
3
5    I stay
dtype: object
```

`duplicated(self, keep: DropKeep = 'first') -> Series` from `pandas.core.series.Series`
Indicate duplicate Series values.

Duplicated values are indicated as ``True`` values in the resulting Series. Either all duplicates, all except the first or all except the last occurrence of duplicates can be indicated.

Parameters

```
-----
keep : {'first', 'last', False}, default 'first'
    Method to handle dropping duplicates:
    - 'first' : Mark duplicates as ``True`` except for the first occurrence.
    - 'last' : Mark duplicates as ``True`` except for the last occurrence.
    - ``False`` : Mark all duplicates as ``True``.
```

Returns

Series[bool]
Series indicating whether each value has occurred in the preceding values.

See Also

Index.duplicated : Equivalent method on pandas.Index.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Series.drop_duplicates : Remove duplicate values from Series.

Examples

By default, for each set of duplicated values, the first occurrence is set on False and all others on True:

```
>>> animals = pd.Series(["llama", "cow", "llama", "beetle", "llama"])
>>> animals.duplicated()
0    False
1    False
2     True
3    False
4     True
dtype: bool
```

which is equivalent to

```
>>> animals.duplicated(keep="first")
0    False
1    False
2     True
3    False
4     True
dtype: bool
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> animals.duplicated(keep="last")
0     True
1    False
2     True
3    False
4    False
dtype: bool
```

By setting keep on ``False``, all duplicates are True:

```
>>> animals.duplicated(keep=False)
0     True
1    False
2     True
3    False
4     True
dtype: bool
```

```
eq(
    self,
    other,
    level: Level | None = None,
    fill_value: float | None = None,
    axis: Axis = 0
) -> Series from pandas.core.series.Series
Return Equal to of series and other, element-wise (binary operator `eq`).

Equivalent to ``series == other``, but with support to substitute a fill_value
for missing data in either one of the inputs.
```

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.
If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.ge` : Return elementwise Greater than or equal to of series and other.

`Series.le` : Return elementwise Less than or equal to of series and other.

`Series.gt` : Return elementwise Greater than of series and other.

`Series.lt` : Return elementwise Less than of series and other.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
```

```
>>> b
```

```
a    1.0
```

```
b    NaN
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.eq(b, fill_value=0)
```

```
a    True
```

```
b    False
```

```
c    False
```

```
d    False
```

```
e    False
```

```
dtype: bool
```

`explode(self, ignore_index: bool = False)` -> Series from `pandas.core.series.Series`
Transform each element of a list-like to a row.

Parameters

`ignore_index` : bool, default False

If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

Series

Exploded lists to rows; index will be duplicated for these rows.

See Also

`Series.str.split` : Split string values on specified separator.

`Series.unstack` : Unstack, a.k.a. pivot, Series with MultiIndex to produce DataFrame.

`DataFrame.melt` : Unpivot a DataFrame from wide format to long format.

`DataFrame.explode` : Explode a DataFrame from list-like columns to long format.

Notes

This routine will explode list-likes including lists, tuples, sets, Series, and np.ndarray. The result dtype of the subset rows will be object. Scalars will be returned unchanged, and empty list-likes will result in an np.nan for that row. In addition, the ordering of elements in the output will be non-deterministic when exploding sets.

Reference :ref:`the user guide <reshaping.explode>` for more examples.

Examples

```

-----
>>> s = pd.Series([[1, 2, 3], "foo", [], [3, 4]])
>>> s
0    [1, 2, 3]
1          foo
2           []
3        [3, 4]
dtype: object

>>> s.explode()
0      1
0      2
0      3
1     foo
2     NaN
3      3
3      4
dtype: object

```

`floordiv(self, other, level=None, fill_value=None, axis: Axis = 0)` -> Series from pandas.core
Return Integer division of series and other, element-wise (binary operator `'floordiv'`).

Equivalent to `'series // other'`, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

```

-----
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as '==' but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

```

Returns

```

-----
Series
    The result of the operation.

```

See Also

```

-----
Series.rfloordiv : Reverse of the Integer division operator, see
    'Python documentation
    <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>'
    for more details.

```

Examples

```

-----
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.floordiv(b, fill_value=0)
a    1.0
b    inf
c    inf
d    0.0
e    NaN
dtype: float64

```

```

ge(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
    Return Greater than or equal to of series and other, element-wise (binary operation)

Equivalent to ``series >= other``, but with support to substitute a
fill_value for missing data in either one of the inputs.

Parameters
-----
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

Returns
-----
Series
    The result of the operation.

See Also
-----
Series.gt : Greater than comparison, element-wise.
Series.le : Less than or equal to comparison, element-wise.
Series.lt : Less than comparison, element-wise.
Series.eq : Equal to comparison, element-wise.
Series.ne : Not equal to comparison, element-wise.

Examples
-----
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=["a", "b", "c", "d", "e"])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
e    1.0
dtype: float64
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=["a", "b", "c", "d", "f"])
>>> b
a    0.0
b    1.0
c    2.0
d    NaN
f    1.0
dtype: float64
>>> a.ge(b, fill_value=0)
a    True
b    True
c    False
d    False
e    True
f    False
dtype: bool

groupby(
    self,
    by=None,
    level: IndexLabel | None = None,
    *,
    as_index: bool = True,
    sort: bool = True,
    group_keys: bool = True,
    observed: bool = True,
    dropna: bool = True
) -> SeriesGroupBy from pandas.core.series.Series
    Group Series using a mapper or by a Series of columns.

```

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters

`by` : mapping, function, label, `pd.Grouper` or list of such
Used to determine the groups for the groupby.
If `by` is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see `align()` method). If a list or ndarray of length equal to the selected axis is passed (see the `groupby` user guide <https://pandas.pydata.org/pandas-docs/stable/user_guide/groupby.html#splitting-an-object>), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in `self`.
Notice that a tuple is interpreted as a (single) key.

`level` : int, level name, or sequence of such, default None
If the axis is a MultiIndex (hierarchical), group by a particular level or levels. Do not specify both `by` and `level`.

`as_index` : bool, default True
Return object with group labels as the index. Only relevant for DataFrame input. `as_index=False` is effectively "SQL-style" grouped output. This argument has no effect on filtrations (see the `filtrations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>), such as `head()`, `tail()`, `nth()` and in transformations (see the `transformations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>).

`sort` : bool, default True
Sort group keys. Get better performance by turning this off.
Note this does not influence the order of observations within each group. Groupby preserves the order of rows within each group. If False, the groups will appear in the same order as they did in the original DataFrame. This argument has no effect on filtrations (see the `filtrations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>), such as `head()`, `tail()`, `nth()` and in transformations (see the `transformations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>).

.. versionchanged:: 2.0.0

Specifying `sort=False` with an ordered categorical grouper will no longer sort the values.

`group_keys` : bool, default True
When calling `apply` and the `by` argument produces a like-indexed (i.e. `:ref:`a transform <groupby.transform>``) result, add group keys to index to identify pieces. By default group keys are not included when the result's index (and column) labels match the inputs, and are included otherwise.

.. versionchanged:: 2.0.0

`group_keys` now defaults to `True`.

`observed` : bool, default True
This only applies if any of the groupers are Categoricals.
If True: only show observed values for categorical groupers.
If False: show all values for categorical groupers.

.. versionchanged:: 3.0.0

The default value is now `True`.

`dropna` : bool, default True
If True, and if group keys contain NA values, NA values together with row/column will be dropped.
If False, NA values will also be treated as the key in groups.

Returns

`pandas.api.typing.SeriesGroupBy`
Returns a groupby object that contains information about the groups.

See Also

resample : Convenience method for frequency conversion and resampling
of time series.

Notes

See the ``user guide`
<<https://pandas.pydata.org/pandas-docs/stable/groupby.html>>`__ for more
detailed usage and examples, including splitting an object into groups,
iterating through groups, selecting a group, aggregation, and more.

The implementation of groupby is hash-based, meaning in particular that
objects that compare as equal will be considered to be in the same group.
An exception to this is that pandas has special handling of NA values:
any NA values will be collapsed to a single group, regardless of how
they compare. See the user guide linked above for more details.

Examples

>>> ser = pd.Series([390., 350., 30., 20.],
... index=['Falcon', 'Falcon', 'Parrot', 'Parrot'],
... name="Max Speed")
>>> ser
Falcon 390.0
Falcon 350.0
Parrot 30.0
Parrot 20.0
Name: Max Speed, dtype: float64

We can pass a list of values to group the Series data by custom labels:

```
>>> ser.groupby(["a", "b", "a", "b"]).mean()
a    210.0
b    185.0
Name: Max Speed, dtype: float64
```

Grouping by numeric labels yields similar results:

```
>>> ser.groupby([0, 1, 0, 1]).mean()
0    210.0
1    185.0
Name: Max Speed, dtype: float64
```

We can group by a level of the index:

```
>>> ser.groupby(level=0).mean()
Falcon    370.0
Parrot     25.0
Name: Max Speed, dtype: float64
```

We can group by a condition applied to the Series values:

```
>>> ser.groupby(ser > 100).mean()
Max Speed
False     25.0
True      370.0
Name: Max Speed, dtype: float64
```

Grouping by Indexes

We can groupby different levels of a hierarchical index
using the ``level`` parameter:

```
>>> arrays = [['Falcon', 'Falcon', 'Parrot', 'Parrot'],
...           ['Captive', 'Wild', 'Captive', 'Wild']]
>>> index = pd.MultiIndex.from_arrays(arrays, names=('Animal', 'Type'))
>>> ser = pd.Series([390., 350., 30., 20.], index=index, name="Max Speed")
>>> ser
Animal Type
Falcon  Captive    390.0
        Wild      350.0
Parrot   Captive    30.0
        Wild       20.0
Name: Max Speed, dtype: float64

>>> ser.groupby(level=0).mean()
Animal
```

```
Falcon    370.0
Parrot    25.0
Name: Max Speed, dtype: float64
```

We can also group by the 'Type' level of the hierarchical index to get the mean speed for each type:

```
>>> ser.groupby(level="Type").mean()
Type
Captive    210.0
Wild      185.0
Name: Max Speed, dtype: float64
```

We can also choose to include `NA` in group keys or not by defining `dropna` parameter, the default setting is `True`.

```
>>> ser = pd.Series([1, 2, 3, 3], index=["a", 'a', 'b', np.nan])
>>> ser.groupby(level=0).sum()
a      3
b      3
dtype: int64
```

To include `NA` values in the group keys, set `dropna=False`:

```
>>> ser.groupby(level=0, dropna=False).sum()
a      3
b      3
NaN    3
dtype: int64
```

We can also group by a custom list with NaN values to handle missing group labels:

```
>>> arrays = ['Falcon', 'Falcon', 'Parrot', 'Parrot']
>>> ser = pd.Series([390., 350., 30., 20.], index=arrays, name="Max Speed")
>>> ser.groupby(["a", "b", "a", np.nan]).mean()
a      210.0
b      350.0
Name: Max Speed, dtype: float64
```

```
>>> ser.groupby(["a", "b", "a", np.nan], dropna=False).mean()
a      210.0
b      350.0
NaN     20.0
Name: Max Speed, dtype: float64
```

`gt(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Greater than of series and other, element-wise (binary operator `gt`).

Equivalent to ``series > other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.lt` : Reverse of the Greater than operator, see

``Python documentation
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`_`
for more details.

Examples

```
-----  
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])  
>>> a  
a    1.0  
b    1.0  
c    1.0  
d    NaN  
e    1.0  
dtype: float64  
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])  
>>> b  
a    0.0  
b    1.0  
c    2.0  
d    NaN  
f    1.0  
dtype: float64  
>>> a.gt(b, fill_value=0)  
a    True  
b    False  
c    False  
d    False  
e    True  
f    False  
dtype: bool
```

```
hist = hist_series(  
    self: Series,  
    by=None,  
    ax=None,  
    grid: bool = True,  
    xlabelsize: int | None = None,  
    xrot: float | None = None,  
    ylabelsize: int | None = None,  
    yrot: float | None = None,  
    figsize: tuple[int, int] | None = None,  
    bins: int | Sequence[int] = 10,  
    backend: str | None = None,  
    legend: bool = False,  
    **kwargs  
) from pandas.plotting  
Draw histogram of the input series using matplotlib.
```

Parameters

```
-----  
by : object, optional  
    If passed, then used to form histograms for separate groups.  
ax : matplotlib axis object  
    If not passed, uses gca().  
grid : bool, default True  
    Whether to show axis grid lines.  
xlabelsize : int, default None  
    If specified changes the x-axis label size.  
xrot : float, default None  
    Rotation of x axis labels.  
ylabelsize : int, default None  
    If specified changes the y-axis label size.  
yrot : float, default None  
    Rotation of y axis labels.  
figsize : tuple, default None  
    Figure size in inches by default.  
bins : int or sequence, default 10  
    Number of histogram bins to be used. If an integer is given, bins + 1  
    bin edges are calculated and returned. If bins is a sequence, gives  
    bin edges, including left edge of first bin and right edge of last  
    bin. In this case, bins is returned unmodified.  
backend : str, default None  
    Backend to use instead of the backend specified in the option  
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to  
    specify the ``plotting.backend`` for the whole session, set  
    ``pd.options.plotting.backend``.  
legend : bool, default False
```

Whether to show the legend.

****kwargs**

To be passed to the actual plotting function.

Returns

matplotlib.axes.Axes
A histogram plot.

See Also

matplotlib.axes.Axes.hist : Plot a histogram using matplotlib.

Examples

For Series:

```
.. plot::
    :context: close-figs

    >>> lst = ["a", "a", "a", "b", "b", "b"]
    >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
    >>> hist = ser.hist()
```

For Groupby:

```
.. plot::
    :context: close-figs

    >>> lst = ["a", "a", "a", "b", "b", "b"]
    >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
    >>> hist = ser.groupby(level=0).hist()
```

idxmax(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Hashable from pandas.c
Return the row label of the maximum value.

If multiple values equal the maximum, the first row label with that value is returned.

Parameters

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Index

Label of the maximum value.

Raises

ValueError

If the Series is empty.

See Also

numpy.argmax : Return indices of the maximum values along the given axis.

DataFrame.idxmax : Return index of first occurrence of maximum over requested axis.

Series.idxmin : Return index *label* of the first occurrence of minimum of values.

Notes

This method is the Series version of ``ndarray.argmax``. This method returns the label of the maximum, while ``ndarray.argmax`` returns the position. To get the position, use ``series.values.argmax()``.

Examples

```

-----
>>> s = pd.Series(data=[1, None, 4, 3, 4], index=["A", "B", "C", "D", "E"])
>>> s
A    1.0
B    NaN
C    4.0
D    3.0
E    4.0
dtype: float64

>>> s.idxmax()
'C'

```

`idxmin(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Hashable from pandas.core.series.Series`
 Return the row label of the minimum value.

If multiple values equal the minimum, the first row label with that value is returned.

Parameters

```

-----
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
    Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
    and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
    Additional arguments and keywords have no effect but might be
    accepted for compatibility with NumPy.

```

Returns

```

-----
Index
    Label of the minimum value.

```

Raises

```

-----
ValueError
    If the Series is empty.

```

See Also

```

-----
numpy.argmax : Return indices of the minimum values
               along the given axis.
DataFrame.idxmin : Return index of first occurrence of minimum
                  over requested axis.
Series.idxmax : Return index *label* of the first occurrence
                of maximum of values.

```

Notes

```

-----
This method is the Series version of ``ndarray.argmin``. This method
returns the label of the minimum, while ``ndarray.argmin`` returns
the position. To get the position, use ``series.values.argmin()``.

```

Examples

```

-----
>>> s = pd.Series(data=[1, None, 4, 1], index=["A", "B", "C", "D"])
>>> s
A    1.0
B    NaN
C    4.0
D    1.0
dtype: float64

>>> s.idxmin()
'A'

```

```

info(
    self,
    verbose: bool | None = None,
    buf: IO[str] | None = None,
    max_cols: int | None = None,
    memory_usage: bool | str | None = None,
    show_counts: bool = True
) -> None from pandas.core.series.Series
Print a concise summary of a Series.

```

This method prints information about a Series including the index dtype, non-NA values and memory usage.

Parameters

`verbose` : bool, optional
Whether to print the full summary. By default, the setting in ```pandas.options.display.max_info_columns``` is followed.

`buf` : writable buffer, defaults to `sys.stdout`
Where to send the output. By default, the output is printed to `sys.stdout`. Pass a writable buffer if you need to further process the output.

`max_cols` : int, optional
Unused, exists only for compatibility with `DataFrame.info`.

`memory_usage` : bool, str, optional
Specifies whether total memory usage of the Series elements (including the index) should be displayed. By default, this follows the ```pandas.options.display.memory_usage``` setting.

True always show memory usage. False never shows memory usage. A value of 'deep' is equivalent to "True with deep introspection". Memory usage is shown in human-readable units (base-2 representation). Without deep introspection a memory estimation is made based in column dtype and number of rows assuming values consume the same memory amount for corresponding dtypes. With deep memory introspection, a real memory usage calculation is performed at the cost of computational resources. See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.

`show_counts` : bool, optional
Whether to show the non-null counts. By default, this is shown only if the DataFrame is smaller than ```pandas.options.display.max_info_rows``` and ```pandas.options.display.max_info_columns```. A value of True always shows the counts, and False never shows the counts.

Returns

None

This method prints a summary of a Series and returns None.

See Also

`Series.describe`: Generate descriptive statistics of Series.
`Series.memory_usage`: Memory usage of Series.

Examples

```
>>> int_values = [1, 2, 3, 4, 5]
>>> text_values = ["alpha", "beta", "gamma", "delta", "epsilon"]
>>> s = pd.Series(text_values, index=int_values)
>>> s.info()
<class 'pandas.Series'>
Index: 5 entries, 1 to 5
Series name: None
Non-Null Count  Dtype
-----
5 non-null      str
dtypes: str(1)
memory usage: 106.0 bytes
```

Prints a summary excluding information about its values:

```
>>> s.info(verbose=False)
<class 'pandas.Series'>
Index: 5 entries, 1 to 5
dtypes: str(1)
memory usage: 106.0 bytes
```

Pipe output of `Series.info` to buffer instead of `sys.stdout`, get buffer content and writes to a text file:

```
>>> import io
>>> buffer = io.StringIO()
>>> s.info(buf=buffer)
>>> s = buffer.getvalue()
```

```
>>> with open("df_info.txt", "w", encoding="utf-8") as f: # doctest: +SKIP
...     f.write(s)
260
```

The `memory_usage` parameter allows deep introspection mode, specially useful for big Series and fine-tune memory optimization:

```
>>> random_strings_array = np.random.choice(["a", "b", "c"], 10**6)
>>> s = pd.Series(np.random.choice(["a", "b", "c"], 10**6))
>>> s.info()
<class 'pandas.Series'>
RangeIndex: 1000000 entries, 0 to 999999
Series name: None
Non-Null Count    Dtype
-----
1000000 non-null  str
dtypes: str(1)
memory usage: 8.6 MB
```

```
>>> s.info(memory_usage="deep")
<class 'pandas.Series'>
RangeIndex: 1000000 entries, 0 to 999999
Series name: None
Non-Null Count    Dtype
-----
1000000 non-null  str
dtypes: str(1)
memory usage: 8.6 MB
```

`isin(self, values) -> Series from pandas.core.series.Series`
Whether elements in Series are contained in `values`.

Return a boolean Series showing whether each element in the Series matches an element in the passed sequence of `values` exactly.

Parameters

`values` : set or list-like
The sequence of values to test. Passing in a single string will raise a `TypeError`. Instead, turn a single string into a list of one element.

Returns

Series
Series of booleans indicating if each element is in values.

Raises

`TypeError`
* If `values` is a string

See Also

`DataFrame.isin` : Equivalent method on `DataFrame`.

Examples

```
>>> s = pd.Series(
...     ["llama", "cow", "llama", "beetle", "llama", "hippo"], name="animal"
... )
>>> s.isin(["cow", "llama"])
0    True
1    True
2    True
3    False
4    True
5    False
Name: animal, dtype: bool
```

To invert the boolean values, use the `~` operator:

```
>>> ~s.isin(["cow", "llama"])
0    False
1    False
2    False
3     True
```

```
4    False
5     True
Name: animal, dtype: bool
```

Passing a single string as `s.isin('llama')` will raise an error. Use a list of one element instead:

```
>>> s.isin(["llama"])
0     True
1    False
2     True
3    False
4     True
5    False
Name: animal, dtype: bool
```

Strings and integers are distinct and are therefore not comparable:

```
>>> pd.Series([1]).isin(["1"])
0    False
dtype: bool
>>> pd.Series([1.1]).isin(["1.1"])
0    False
dtype: bool
```

`isna(self)` -> Series from `pandas.core.series.Series`
Detect missing values.

Return a boolean same-sized Series indicating if the values are NA. NA values, such as `None` or `:attr:'numpy.NaN'`, get mapped to `True` values. Everything else gets mapped to `False` values. Characters such as empty strings `''` or `:attr:'numpy.inf'` are not considered NA values.

Returns

Series

Mask of bool values for each element in Series that indicates whether an element is an NA value.

See Also

`DataFrame.isna` : Detect missing values.

`DataFrame.isnull` : Alias of `isna`.

`Series.notna` : Boolean inverse of `isna`.

`DataFrame.notna` : Boolean inverse of `isna`.

`Series.notnull` : Alias of `notna`.

`DataFrame.notnull` : Alias of `notna`.

`Series.dropna` : Omit axes labels with missing values.

`DataFrame.dropna` : Omit axes labels with missing values.

`isna` : Top-level `isna`.

Examples

Show which entries in a Series are NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
>>> ser.isna()
0    False
1    False
2     True
dtype: bool
```

`isnull(self)` -> Series from `pandas.core.series.Series`
`Series.isnull` is an alias for `Series.isna`.

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None` or `:attr:'numpy.NaN'`, gets mapped to `True` values.

Everything else gets mapped to `False` values. Characters such as empty

strings ```` or :attr:`numpy.inf` are not considered NA values.

Returns

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See Also

Series.isnull : Alias of isna.

DataFrame.isnull : Alias of isna.

Series.notna : Boolean inverse of isna.

DataFrame.notna : Boolean inverse of isna.

Series.dropna : Omit axes labels with missing values.

DataFrame.dropna : Omit axes labels with missing values.

isna : Top-level isna.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born  name      toy
0  5.0      NaT  Alfred    NaN
1  6.0  1939-05-27  Batman  Batmobile
2  NaN  1940-04-25      Joker
```

```
>>> df.isna()
   age  born  name  toy
0  False  True  False  True
1  False  False  False  False
2   True  False  False  False
```

Show which entries in a Series are NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
```

```
>>> ser.isna()
0    False
1    False
2     True
dtype: bool
```

items(self) -> Iterable[tuple[Hashable, Any]] from pandas.core.series.Series
Lazily iterate over (index, value) tuples.

This method returns an iterable tuple (index, value). This is convenient if you want to create a lazy iterator.

Returns

iterable

Iterable of tuples containing the (index, value) pairs from a Series.

See Also

DataFrame.items : Iterate over (column name, Series) pairs.

DataFrame.iterrows : Iterate over DataFrame rows as (index, Series) pairs.

Examples

```
-----
>>> s = pd.Series(["A", "B", "C"])
>>> for index, value in s.items():
...     print(f"Index : {index}, Value : {value}")
Index : 0, Value : A
Index : 1, Value : B
Index : 2, Value : C
```

keys(self) -> Index from pandas.core.series.Series
Return alias for index.

Returns

Index
Index of the Series.

See Also

Series.index : The index (axis labels) of the Series.

Examples

```
-----
>>> s = pd.Series([1, 2, 3], index=[0, 1, 2])
>>> s.keys()
Index([0, 1, 2], dtype='int64')
```

kurt(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 **kwargs
) from pandas.core.series.Series
Return unbiased kurtosis over requested axis.

Kurtosis obtained using Fisher's definition of kurtosis (kurtosis of normal == 0.0). Normalized by N-1.

Parameters

axis : {index (0)}
Axis for the function to be applied on.
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True
Exclude NA/null values when computing the result.
numeric_only : bool, default False
Include only float, int, boolean columns.

**kwargs
Additional keyword arguments to be passed to the function.

Returns

scalar
Unbiased kurtosis.

See Also

Series.skew : Return unbiased skew over requested axis.
Series.var : Return unbiased variance over requested axis.
Series.std : Return unbiased standard deviation over requested axis.

Examples

```
-----
>>> s = pd.Series([1, 2, 2, 3], index=["cat", "dog", "dog", "mouse"])
>>> s
cat    1
dog    2
```



```

dog      2
mouse    3
dtype: int64
>>> s.kurt()
1.5

kurtosis = kurt(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
)

le(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Less than or equal to of series and other, element-wise (binary operator)

Equivalent to ``series <= other``, but with support to substitute a
fill_value for missing data in either one of the inputs.

Parameters
-----
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

Returns
-----
Series
    The result of the operation.

See Also
-----
Series.ge : Return elementwise Greater than or equal to of series and other.
Series.lt : Return elementwise Less than of series and other.
Series.gt : Return elementwise Greater than of series and other.
Series.eq : Return elementwise equal to of series and other.

Examples
-----
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
e    1.0
dtype: float64
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])
>>> b
a    0.0
b    1.0
c    2.0
d    NaN
f    1.0
dtype: float64
>>> a.le(b, fill_value=0)
a    False
b     True
c     True
d    False
e    False
f     True
dtype: bool

```

```
lt(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Greater than of series and other, element-wise (binary operator `lt`).
```

Equivalent to ``series > other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.gt : Element-wise Greater than, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_ for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
e    1.0
```

```
dtype: float64
```

```
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])
```

```
>>> b
```

```
a    0.0
```

```
b    1.0
```

```
c    2.0
```

```
d    NaN
```

```
f    1.0
```

```
dtype: float64
```

```
>>> a.gt(b, fill_value=0)
```

```
a    True
```

```
b    False
```

```
c    False
```

```
d    False
```

```
e    True
```

```
f    False
```

```
dtype: bool
```

```
map(
```

```
self,
```

```
func: Callable | Mapping | Series | None = None,
```

```
na_action: Literal['ignore'] | None = None,
```

```
engine: Callable | None = None,
```

```
**kwargs
```

```
) -> Series from pandas.core.series.Series
```

Map values of Series according to an input mapping or function.

Used for substituting each value in a Series with another value, that may be derived from a function, a ``dict`` or a :class:`Series`.

Parameters

```

func : function, collections.abc.Mapping subclass or Series
      Function or mapping correspondence.
na_action : {None, 'ignore'}, default None
      If 'ignore', propagate NaN values, without passing them to the
      mapping correspondence.
engine : decorator, optional
      Choose the execution engine to use to run the function. Only used for
      functions. If ``map`` is called with a mapping or ``Series``, an
      exception will be raised. If ``engine`` is not provided the function will
      be executed by the regular Python interpreter.

      Options include JIT compilers such as Numba, Bodo or Blosc2, which in some
      cases can speed up the execution. To use an executor you can provide the
      decorators ``numba.jit``, ``numba.njit``, ``bodo.jit`` or ``blosc2.jit``.
      You can also provide the decorator with parameters, like
      ``numba.jit(nogit=True)``.

      Not all functions can be executed with all execution engines. In general,
      JIT compilers will require type stability in the function (no variable
      should change data type during the execution). And not all pandas and
      NumPy APIs are supported. Check the engine documentation for limitations.

      .. versionadded:: 3.0.0

**kwargs
      Additional keyword arguments to pass as keywords arguments to
      `arg`.

      .. versionadded:: 3.0.0

Returns
-----
Series
      Same index as caller.

See Also
-----
Series.apply : For applying more complex functions on a Series.
Series.replace: Replace values given in `to_replace` with `value`.
DataFrame.apply : Apply a function row-/column-wise.
DataFrame.map : Apply a function elementwise on a whole DataFrame.

Notes
-----
When ``arg`` is a dictionary, values in Series that are not in the
dictionary (as keys) are converted to ``NaN``. However, if the
dictionary is a ``dict`` subclass that defines ``__missing__`` (i.e.
provides a method for default values), then this default is used
rather than ``NaN``.

Examples
-----
>>> s = pd.Series(["cat", "dog", np.nan, "rabbit"])
>>> s
0      cat
1      dog
2      NaN
3  rabbit
dtype: str

``map`` accepts a ``dict`` or a ``Series``. Values that are not found
in the ``dict`` are converted to ``NaN``, unless the dict has a default
value (e.g. ``defaultdict``):

>>> s.map({"cat": "kitten", "dog": "puppy"})
0  kitten
1  puppy
2    NaN
3    NaN
dtype: str

It also accepts a function:

>>> s.map("I am a {}".format)
0      I am a cat
1      I am a dog
2      I am a nan

```

```
3    I am a rabbit
dtype: str
```

To avoid applying the function to missing values (and keep them as ``NaN``) ``na\_action='ignore'`` can be used:

```
>>> s.map("I am a {}".format, na_action="ignore")
0    I am a cat
1    I am a dog
2         NaN
3    I am a rabbit
dtype: str
```

For categorical data, the function is only applied to the categories:

```
>>> s = pd.Series(list("cabaa"))
>>> s.map(print)
```

```
c
a
b
a
a
0    None
1    None
2    None
3    None
4    None
dtype: object
```

```
>>> s_cat = s.astype("category")
>>> s_cat.map(print) # function called once per unique category
```

```
a
b
c
0    None
1    None
2    None
3    None
4    None
dtype: object
```

```
max(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
```

Return the maximum of the values over the requested axis.

If you want the *index* of the maximum, use ``idxmax``. This is the equivalent of the ``numpy.ndarray`` method ``argmax``.

Parameters

axis : {index (0)}

Axis for the function to be applied on.
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric_only : bool, default False

Include only float, int, boolean columns.

**kwargs

Additional keyword arguments to be passed to the function.

Returns

scalar or Series (if level specified)

The maximum of the values in the Series.

See Also

numpy.max : Equivalent numpy function for arrays.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

```
>>> idx = pd.MultiIndex.from_arrays(
...     [ ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"] ],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded  animal
warm     dog      4
         falcon   2
cold     fish     0
         spider   8
Name: legs, dtype: int64

>>> s.max()
8
```

```
mean(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Any from pandas.core.series.Series
Return the mean of the values over the requested axis.
```

Parameters

axis : {index (0)}

Axis for the function to be applied on.
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True
Exclude NA/null values when computing the result.

numeric_only : bool, default False
Include only float, int, boolean columns.

**kwargs
Additional keyword arguments to be passed to the function.

Returns

scalar or Series (if level specified)
Mean of the values for the requested axis.

See Also

numpy.median : Equivalent numpy function for computing median.
Series.sum : Sum of the values.
Series.median : Median of the values.
Series.std : Standard deviation of the values.
Series.var : Variance of the values.
Series.min : Minimum value.
Series.max : Maximum value.

Examples

```
>>> s = pd.Series([1, 2, 3])
>>> s.mean()
```

2.0

```
median(  
    self,  
    *,  
    axis: Axis | None = 0,  
    skipna: bool = True,  
    numeric_only: bool = False,  
    **kwargs  
) -> Any from pandas.core.series.Series  
Return the median of the values over the requested axis.
```

Parameters

axis : {index (0)}

Axis for the function to be applied on.
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation
across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric_only : bool, default False

Include only float, int, boolean columns.

**kwargs

Additional keyword arguments to be passed to the function.

Returns

scalar or Series (if level specified)

Median of the values for the requested axis.

See Also

numpy.median : Equivalent numpy function for computing median.

Series.sum : Sum of the values.

Series.median : Median of the values.

Series.std : Standard deviation of the values.

Series.var : Variance of the values.

Series.min : Minimum value.

Series.max : Maximum value.

Examples

>>> s = pd.Series([1, 2, 3])
>>> s.median()
2.0

With a DataFrame

```
>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])  
>>> df
```

```
      a  b  
tiger  1  2  
zebra  2  3  
>>> df.median()  
a    1.5  
b    2.5  
dtype: float64
```

Using axis=1

```
>>> df.median(axis=1)  
tiger    1.5  
zebra    2.5  
dtype: float64
```

In this case, `numeric\_only` should be set to `True`
to avoid getting an error.

```
>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])  
>>> df.median(numeric_only=True)  
a    1.5  
dtype: float64
```

```
memory_usage(self, index: bool = True, deep: bool = False) -> int from pandas.core.series.Series
    Return the memory usage of the Series.
```

The memory usage can optionally include the contribution of the index and of elements of `object` dtype.

Parameters

index : bool, default True
 Specifies whether to include the memory usage of the Series index.
deep : bool, default False
 If True, introspect the data deeply by interrogating `object` dtypes for system-level memory consumption, and include it in the returned value.

Returns

int
 Bytes of memory consumed.

See Also

numpy.ndarray.nbytes : Total bytes consumed by the elements of the array.
DataFrame.memory_usage : Bytes consumed by a DataFrame.

Examples

>>> s = pd.Series(range(3))
>>> s.memory_usage()
156

Not including the index gives the size of the rest of the data, which is necessarily smaller:

```
>>> s.memory_usage(index=False)  
24
```

The memory footprint of `object` values is ignored by default:

```
>>> s = pd.Series(["a", "b"])  
>>> s.values  
<ArrowStringArray>  
['a', 'b']  
Length: 2, dtype: str  
>>> s.memory_usage()  
150  
>>> s.memory_usage(deep=True)  
150
```

```
min(  
    self,  
    *,  
    axis: Axis | None = 0,  
    skipna: bool = True,  
    numeric_only: bool = False,  
    **kwargs  
) from pandas.core.series.Series  
    Return the minimum of the values over the requested axis.
```

If you want the **index** of the minimum, use ``idxmin``. This is the equivalent of the ``numpy.ndarray`` method ``argmin``.

Parameters

axis : {index (0)}
 Axis for the function to be applied on.
 For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True
 Exclude NA/null values when computing the result.

numeric_only : bool, default False
Include only float, int, boolean columns.
**kwargs
Additional keyword arguments to be passed to the function.

Returns

scalar or Series (if level specified)
The minimum of the values in the Series.

See Also

numpy.min : Equivalent numpy function for arrays.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

>>> idx = pd.MultiIndex.from_arrays(
... ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
... names=["blooded", "animal"],
...)
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm dog 4
 falcon 2
cold fish 0
 spider 8
Name: legs, dtype: int64

>>> s.min()
0

mod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Modulo of series and other, element-wise (binary operator `mod`).

Equivalent to ``series % other``, but with support to substitute a
fill_value for missing data in either one of the inputs.

Parameters

other : Series or scalar value
Series with which to compute modulo.
level : int or name
Broadcast across a level, matching Index values on the
passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for
successful Series alignment, with this value before computation.
If data in both corresponding Series locations is missing
the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.rmod : Reverse of the Modulo operator, see
`Python documentation`
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
>>> a


```

a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.mod(b, fill_value=0)
a    0.0
b    NaN
c    NaN
d    0.0
e    NaN
dtype: float64

```

`mode(self, dropna: bool = True) -> Series` from `pandas.core.series.Series`
Return the mode(s) of the Series.

The mode is the value that appears most often. There can be multiple modes.

Always returns Series even if only one value is returned.

Parameters

`dropna` : bool, default True
Don't consider counts of NaN/NaT.

Returns

Series

Modes of the Series in sorted order.

See Also

`numpy.mode` : Equivalent numpy function for computing median.
`Series.sum` : Sum of the values.
`Series.median` : Median of the values.
`Series.std` : Standard deviation of the values.
`Series.var` : Variance of the values.
`Series.min` : Minimum value.
`Series.max` : Maximum value.

Examples

```

>>> s = pd.Series([2, 4, 2, 2, 4, None])
>>> s.mode()
0    2.0
dtype: float64

```

More than one mode:

```

>>> s = pd.Series([2, 4, 8, 2, 4, None])
>>> s.mode()
0    2.0
1    4.0
dtype: float64

```

With and without considering null value:

```

>>> s = pd.Series([2, 4, None, None, 4, None])
>>> s.mode(dropna=False)
0    NaN
dtype: float64
>>> s = pd.Series([2, 4, None, None, 4, None])
>>> s.mode()
0    4.0
dtype: float64

```

```

mul(
    self,
    other,
    level: Level | None = None,

```

```

    fill_value: float | None = None,
    axis: Axis = 0
) -> Series from pandas.core.series.Series
    Return Multiplication of series and other, element-wise (binary operator `mul`).

    Equivalent to ``series * other``, but with support to substitute
    a fill_value for missing data in either one of the inputs.

Parameters
-----
other : Series or scalar value
    With which to compute the multiplication.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

Returns
-----
Series
    The result of the operation.

See Also
-----
Series.rmul : Reverse of the Multiplication operator, see
    `Python documentation
    <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`
    for more details.

Examples
-----
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.multiply(b, fill_value=0)
a    1.0
b    0.0
c    0.0
d    0.0
e    NaN
dtype: float64
>>> a.mul(5, fill_value=0)
a    5.0
b    5.0
c    5.0
d    0.0
dtype: float64

multiply = mul(
    self,
    other,
    level: Level | None = None,
    fill_value: float | None = None,
    axis: Axis = 0
) -> Series

ne(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
    Return Not equal to of series and other, element-wise (binary operator `ne`).

    Equivalent to ``series != other``, but with support to substitute a fill_value for

```

missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.eq : Reverse of the Not equal to operator, see
Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_ for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.ne(b, fill_value=0)
a False
b True
c True
d True
e True
dtype: bool

nlargest(self, n: int = 5, keep: Literal['first', 'last', 'all'] = 'first') -> Series from pandas
Return the largest `n` elements.

Parameters

n : int, default 5
Return this many descending sorted values.
keep : {'first', 'last', 'all'}, default 'first'
When there are duplicate values that cannot all fit in a Series of `n` elements:

- ``first`` : return the first `n` occurrences in order of appearance.
- ``last`` : return the last `n` occurrences in reverse order of appearance.
- ``all`` : keep all occurrences. This can result in a Series of size larger than `n`.

Returns

Series
The `n` largest values in the Series, sorted in decreasing order.

See Also

Series.nsmallest: Get the `n` smallest elements.
Series.sort_values: Sort Series by values.
Series.head: Return the first `n` rows.

Notes

Faster than ``.sort\_values(ascending=False).head(n)`` for small `n` relative to the size of the ``Series`` object.

Examples

```
>>> countries_population = {
...     "Italy": 59000000,
...     "France": 65000000,
...     "Malta": 434000,
...     "Maldives": 434000,
...     "Brunei": 434000,
...     "Iceland": 337000,
...     "Nauru": 11300,
...     "Tuvalu": 11300,
...     "Anguilla": 11300,
...     "Montserrat": 5200,
... }
>>> s = pd.Series(countries_population)
>>> s
Italy          59000000
France         65000000
Malta           434000
Maldives        434000
Brunei          434000
Iceland         337000
Nauru            11300
Tuvalu           11300
Anguilla         11300
Montserrat       5200
dtype: int64
```

The `n` largest elements where ``n=5`` by default.

```
>>> s.nlargest()
France         65000000
Italy          59000000
Malta           434000
Maldives        434000
Brunei          434000
dtype: int64
```

The `n` largest elements where ``n=3``. Default `keep` value is 'first' so Malta will be kept.

```
>>> s.nlargest(3)
France         65000000
Italy          59000000
Malta           434000
dtype: int64
```

The `n` largest elements where ``n=3`` and keeping the last duplicates. Brunei will be kept since it is the last with value 434000 based on the index order.

```
>>> s.nlargest(3, keep="last")
France         65000000
Italy          59000000
Brunei          434000
dtype: int64
```

The `n` largest elements where ``n=3`` with all duplicates kept. Note that the returned Series has five elements due to the three duplicates.

```
>>> s.nlargest(3, keep="all")
France         65000000
Italy          59000000
Malta           434000
Maldives        434000
```

```
Brunei          434000
dtype: int64
```

```
notna(self) -> Series from pandas.core.series.Series
Detect existing (non-missing) values.
```

Return a boolean same-sized Series indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series
Mask of bool values for each element in Series that indicates whether an element is not an NA value.

See Also

Series.isna : Detect missing values.
DataFrame.isna : Detect missing values.
Series.isnull : Alias of isna.
DataFrame.isnull : Alias of isna.
DataFrame.notna : Boolean inverse of isna.
DataFrame.notnull : Alias of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
notna : Top-level notna.

Examples

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
>>> ser.notna()
0     True
1     True
2    False
dtype: bool
```

```
notnull(self) -> Series from pandas.core.series.Series
Series.notnull is an alias for Series.notna.
```

Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series/DataFrame
Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

Series.notnull : Alias of notna.
DataFrame.notnull : Alias of notna.
Series.isna : Boolean inverse of notna.
DataFrame.isna : Boolean inverse of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
notna : Top-level notna.

Examples

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born      name      toy
0  5.0      NaT    Alfred      NaN
1  6.0 1939-05-27    Batman  Batmobile
2  NaN 1940-04-25      Joker
```

```
>>> df.notna()
   age  born  name  toy
0  True False  True False
1  True  True  True  True
2 False  True  True  True
```

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
```

```
>>> ser.notna()
0    True
1    True
2    False
dtype: bool
```

`nsmallest(self, n: int = 5, keep: Literal['first', 'last', 'all'] = 'first') -> Series from`
Return the smallest `n` elements.

Parameters

`n` : int, default 5

Return this many ascending sorted values.

`keep` : {'first', 'last', 'all'}, default 'first'

When there are duplicate values that cannot all fit in a Series of `n` elements:

- ```first``` : return the first `n` occurrences in order of appearance.
- ```last``` : return the last `n` occurrences in reverse order of appearance.
- ```all``` : keep all occurrences. This can result in a Series of size larger than `n`.

Returns

Series

The `n` smallest values in the Series, sorted in increasing order.

See Also

`Series.nlargest`: Get the `n` largest elements.

`Series.sort_values`: Sort Series by values.

`Series.head`: Return the first `n` rows.

Notes

Faster than ```.sort_values().head(n)``` for small `n` relative to the size of the ```Series``` object.

Examples

```
>>> countries_population = {
...     "Italy": 59000000,
```

```

...     "France": 65000000,
...     "Brunei": 434000,
...     "Malta": 434000,
...     "Maldives": 434000,
...     "Iceland": 337000,
...     "Nauru": 11300,
...     "Tuvalu": 11300,
...     "Anguilla": 11300,
...     "Montserrat": 5200,
... }
>>> s = pd.Series(countries_population)
>>> s
Italy          59000000
France         65000000
Brunei         434000
Malta          434000
Maldives       434000
Iceland        337000
Nauru          11300
Tuvalu         11300
Anguilla       11300
Montserrat     5200
dtype: int64

```

The `n` smallest elements where ``n=5`` by default.

```

>>> s.nsmallest()
Montserrat     5200
Nauru         11300
Tuvalu        11300
Anguilla      11300
Iceland       337000
dtype: int64

```

The `n` smallest elements where ``n=3``. Default `keep` value is 'first' so Nauru and Tuvalu will be kept.

```

>>> s.nsmallest(3)
Montserrat     5200
Nauru         11300
Tuvalu        11300
dtype: int64

```

The `n` smallest elements where ``n=3`` and keeping the last duplicates. Anguilla and Tuvalu will be kept since they are the last with value 11300 based on the index order.

```

>>> s.nsmallest(3, keep="last")
Montserrat     5200
Anguilla      11300
Tuvalu        11300
dtype: int64

```

The `n` smallest elements where ``n=3`` with all duplicates kept. Note that the returned Series has four elements due to the three duplicates.

```

>>> s.nsmallest(3, keep="all")
Montserrat     5200
Nauru         11300
Tuvalu        11300
Anguilla      11300
dtype: int64

```

`pop(self, item: Hashable) -> Any` from `pandas.core.series.Series`
Return item and drops from series. Raise `KeyError` if not found.

Parameters

item : label

Index of the element that needs to be removed.

Returns

scalar

Value that is popped from series.

See Also

Series.drop: Drop specified values from Series.
Series.drop_duplicates: Return Series with duplicate values removed.

Examples

>>> ser = pd.Series([1, 2, 3])

>>> ser.pop(0)
1

>>> ser
1 2
2 3
dtype: int64

pow(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Exponential power of series and other, element-wise (binary operator `pow`).

Equivalent to ``series \*\* other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.rpow : Reverse of the Exponential power operator, see [Python documentation](https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types)
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_` for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.pow(b, fill_value=0)
a 1.0
b 1.0
c 1.0
d 0.0
e NaN
dtype: float64

prod(
self,


```

*,
axis: Axis | None = None,
skipna: bool = True,
numeric_only: bool = False,
min_count: int = 0,
**kwargs
) from pandas.core.series.Series
    Return the product of the values over the requested axis.

    By default, missing values are skipped. To include them in the calculation,
    set ``skipna`` parameter to False.

    Parameters
    -----
    axis : {index (0)}
        Axis for the function to be applied on.
        For `Series` this parameter is unused and defaults to 0.

    .. warning::
        The behavior of DataFrame.prod with ``axis=None`` is deprecated,
        in a future version this will reduce over both axes and return a scalar
        To retain the old behavior, pass axis=0 (or do not pass axis).

    .. versionadded:: 2.0.0
    skipna : bool, default True
        Exclude NA/null values when computing the result.
    numeric_only : bool, default False
        Include only float, int, boolean columns. Not implemented for Series.
    min_count : int, default 0
        The required number of valid values to perform the operation. If fewer than
        ``min_count`` non-NA values are present the result will be NA.
    **kwargs
        Additional keyword arguments to be passed to the function.

    Returns
    -----
    scalar
        Value containing the calculation referenced in the description.

    See Also
    -----
    Series.sum : Return the sum.
    Series.min : Return the minimum.
    Series.max : Return the maximum.
    Series.idxmin : Return the index of the minimum.
    Series.idxmax : Return the index of the maximum.

    DataFrame.sum : Return the sum over the requested axis.
    DataFrame.min : Return the minimum over the requested axis.
    DataFrame.max : Return the maximum over the requested axis.
    DataFrame.idxmin : Return the index of the minimum over the requested axis.
    DataFrame.idxmax : Return the index of the maximum over the requested axis.

    Examples
    -----
    By default, the product of an empty or all-NA Series is ``1``

    >>> pd.Series([], dtype="float64").prod()
    1.0

    This can be controlled with the ``min_count`` parameter

    >>> pd.Series([], dtype="float64").prod(min_count=1)
    nan

    Thanks to the ``skipna`` parameter, ``min_count`` handles all-NA and
    empty series identically.

    >>> pd.Series([np.nan]).prod()
    1.0
    >>> pd.Series([np.nan]).prod(min_count=1)
    nan

product = prod(
    self,
    *,
    axis: Axis | None = None,

```

```

        skipna: bool = True,
        numeric_only: bool = False,
        min_count: int = 0,
        **kwargs
    )

quantile(
    self,
    q: float | Sequence[float] | AnyArrayLike = 0.5,
    interpolation: QuantileInterpolation = 'linear'
) -> float | Series from pandas.core.series.Series
    Return value at the given quantile.

Parameters
-----
q : float or array-like, default 0.5 (50% quantile)
    The quantile(s) to compute, which can lie in range: 0 <= q <= 1.
interpolation : {'linear', 'lower', 'higher', 'midpoint', 'nearest'}
    This optional parameter specifies the interpolation method to use,
    when the desired quantile lies between two data points `i` and `j`:

    * linear:  $i + (j - i) * (x - i) / (j - i)$ , where  $(x - i) / (j - i)$  is
      the fractional part of the index surrounded by `i` and `j`.
    * lower: `i`.
    * higher: `j`.
    * nearest: `i` or `j` whichever is nearest.
    * midpoint:  $(i + j) / 2$ .

Returns
-----
float or Series
    If `q` is an array, a Series will be returned where the
    index is `q` and the values are the quantiles, otherwise
    a float will be returned.

See Also
-----
core.window.Rolling.quantile : Calculate the rolling quantile.
numpy.percentile : Returns the q-th percentile(s) of the array elements.

Examples
-----
>>> s = pd.Series([1, 2, 3, 4])
>>> s.quantile(0.5)
2.5
>>> s.quantile([0.25, 0.5, 0.75])
0.25    1.75
0.50    2.50
0.75    3.25
dtype: float64

radd(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series.Series
    Return Addition of series and other, element-wise (binary operator `radd`).

    Equivalent to `other + series`, but with support to substitute a fill_value for
    missing data in either one of the inputs.

Parameters
-----
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as `+=` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

Returns
-----
Series

```

The result of the operation.

See Also

Series.add : Element-wise Addition, see
Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.add(b, fill_value=0)
a    2.0
b    1.0
c    1.0
d    1.0
e    NaN
dtype: float64
```

rdiv = rtruediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series

rdivmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core
Return Integer division and modulo of series and other, element-wise (binary operator `r

Equivalent to ``other divmod series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

2-Tuple of Series
The result of the operation.

See Also

Series.divmod : Element-wise Integer division and modulo, see
Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
```

```

d      NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a      1.0
b      NaN
d      1.0
e      NaN
dtype: float64
>>> a.divmod(b, fill_value=0)
(a      1.0
 b      inf
 c      inf
 d      0.0
 e      NaN
dtype: float64,
 a      0.0
 b      NaN
 c      NaN
 d      0.0
 e      NaN
dtype: float64)

```

```

reindex(
    self,
    index=None,
    *,
    axis: Axis | None = None,
    method: ReindexMethod | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    level: Level | None = None,
    fill_value: Scalar | None = None,
    limit: int | None = None,
    tolerance=None
) -> Series from pandas.core.series.Series
Conform Series to new index with optional filling logic.

```

Places NA/NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and ``copy=False``.

Parameters

index : scalar, list-like, dict-like or function, optional
A scalar, list-like, dict-like or functions transformations to apply to that axis' values.

axis : {0 or 'index'}, default 0
The axis to rename. For `Series` this parameter is unused and defaults to 0.

method : {{None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}}
Method to use for filling holes in reindexed DataFrame.
Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

- * None (default): don't fill gaps
- * pad / ffill: Propagate last valid observation forward to next valid.
- * backfill / bfill: Use next valid observation to fill gap.
- * nearest: Use nearest valid observations to fill gap.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : scalar, default np.nan
Value to use for missing values. Defaults to NaN, but can be any

"compatible" value.
limit : int, default None
Maximum number of consecutive elements to forward or backward fill.
tolerance : optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

Series

Series with changed index.

See Also

DataFrame.set_index : Set row labels.
DataFrame.reset_index : Remove row labels or move them to new columns.
DataFrame.reindex_like : Change to same indices as other DataFrame.

Examples

`DataFrame.reindex` supports two calling conventions

```
* ``(index=index_labels, columns=column_labels, ...)``
* ``(labels, axis={{'index', 'columns'}}, ...)``
```

We *highly* recommend using keyword arguments to clarify your intent.

Create a DataFrame with some fictional data.

```
>>> index = ["Firefox", "Chrome", "Safari", "IE10", "Konqueror"]
>>> columns = ["http_status", "response_time"]
>>> df = pd.DataFrame(
...     [[200, 0.04], [200, 0.02], [404, 0.07], [404, 0.08], [301, 1.0]],
...     columns=columns,
...     index=index,
... )
>>> df
```

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

Create a new index and reindex the DataFrame. By default values in the new index that do not have corresponding records in the DataFrame are assigned `NaN`.

```
>>> new_index = ["Safari", "Iceweasel", "Comodo Dragon", "IE10", "Chrome"]
>>> df.reindex(new_index)
```

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02

We can fill in the missing values by passing a value to the keyword `fill_value`. Because the index is not monotonically increasing or decreasing, we cannot use arguments to the keyword `method` to fill the `NaN` values.

```
>>> df.reindex(new_index, fill_value=0)
```

	http_status	response_time
Safari	404	0.07
Iceweasel	0	0.00
Comodo Dragon	0	0.00
IE10	404	0.08
Chrome	200	0.02

```
>>> df.reindex(new_index, fill_value="missing")
           http_status response_time
Safari             404           0.07
Iceweasel          missing          missing
Comodo Dragon      missing          missing
IE10                404           0.08
Chrome             200           0.02
```

We can also reindex the columns.

```
>>> df.reindex(columns=["http_status", "user_agent"])
           http_status user_agent
Firefox            200         NaN
Chrome             200         NaN
Safari             404         NaN
IE10               404         NaN
Konqueror          301         NaN
```

Or we can use "axis-style" keyword arguments

```
>>> df.reindex(["http_status", "user_agent"], axis="columns")
           http_status user_agent
Firefox            200         NaN
Chrome             200         NaN
Safari             404         NaN
IE10               404         NaN
Konqueror          301         NaN
```

To further illustrate the filling functionality in ``reindex``, we will create a DataFrame with a monotonically increasing index (for example, a sequence of dates).

```
>>> date_index = pd.date_range("1/1/2010", periods=6, freq="D")
>>> df2 = pd.DataFrame(
...     {"prices": [100, 101, np.nan, 100, 89, 88]}, index=date_index
... )
>>> df2
           prices
2010-01-01    100.0
2010-01-02    101.0
2010-01-03     NaN
2010-01-04    100.0
2010-01-05     89.0
2010-01-06     88.0
```

Suppose we decide to expand the DataFrame to cover a wider date range.

```
>>> date_index2 = pd.date_range("12/29/2009", periods=10, freq="D")
>>> df2.reindex(date_index2)
           prices
2009-12-29     NaN
2009-12-30     NaN
2009-12-31     NaN
2010-01-01    100.0
2010-01-02    101.0
2010-01-03     NaN
2010-01-04    100.0
2010-01-05     89.0
2010-01-06     88.0
2010-01-07     NaN
```

The index entries that did not have a value in the original data frame (for example, '2009-12-29') are by default filled with ``NaN``. If desired, we can fill in the missing values using one of several options.

For example, to back-propagate the last valid value to fill the ``NaN`` values, pass ``bfill`` as an argument to the ``method`` keyword.

```
>>> df2.reindex(date_index2, method="bfill")
           prices
2009-12-29    100.0
2009-12-30    100.0
2009-12-31    100.0
```

```

2010-01-01    100.0
2010-01-02    101.0
2010-01-03         NaN
2010-01-04    100.0
2010-01-05     89.0
2010-01-06     88.0
2010-01-07         NaN

```

Please note that the ``NaN`` value present in the original DataFrame (at index value 2010-01-03) will not be filled by any of the value propagation schemes. This is because filling while reindexing does not look at DataFrame values, but only compares the original and desired indexes. If you do want to fill in the ``NaN`` values present in the original DataFrame, use the ``fillna()`` method.

See the :ref:`user guide <basics.reindexing>` for more.

```

rename(
    self,
    index: Renamer | Hashable | None = None,
    *,
    axis: Axis | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    inplace: bool = False,
    level: Level | None = None,
    errors: IgnoreRaise = 'ignore'
) -> Series | None from pandas.core.series.Series
    Alter Series index labels or name.

```

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

Alternatively, change ``Series.name`` with a scalar value.

See the :ref:`user guide <basics.rename>` for more.

Parameters

index : scalar, hashable sequence, dict-like or function optional
 Functions or dict-like are transformations to apply to the index.
 Scalar or hashable sequence-like will alter the ``Series.name`` attribute.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`__ for more details.

inplace : bool, default False

Whether to return a new Series. If True the value of copy is ignored.

level : int or level name, default None

In case of MultiIndex, only rename labels in the specified level.

errors : {'ignore', 'raise'}, default 'ignore'

If 'raise', raise ``KeyError`` when a 'dict-like mapper' or

'index' contains labels that are not present in the index being transformed.

If 'ignore', existing keys will be renamed and extra keys will be ignored.

Returns

Series

A shallow copy with index labels or name altered, or the same object if ``inplace=True`` and index is not a dict or callable else None.

See Also

DataFrame.rename : Corresponding DataFrame method.

Series.rename_axis : Set the name of the axis.

Examples

```
-----
>>> s = pd.Series([1, 2, 3])
>>> s
0    1
1    2
2    3
dtype: int64
>>> s.rename("my_name") # scalar, changes Series.name
0    1
1    2
2    3
Name: my_name, dtype: int64
>>> s.rename(lambda x: x**2) # function, changes labels
0    1
1    2
4    3
dtype: int64
>>> s.rename({1: 3, 2: 5}) # mapping, changes labels
0    1
3    2
5    3
dtype: int64
```

```
rename_axis(
    self,
    mapper: IndexLabel | lib.NoDefault = <no_default>,
    *,
    index=<no_default>,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>,
    inplace: bool = False
) -> Self | None from pandas.core.series.Series
Set the name of the axis for the index.
```

Parameters

mapper : scalar, list-like, optional
Value to set the axis name attribute.

Use either ``mapper`` and ``axis`` to specify the axis to target with ``mapper``, or ``index``.

index : scalar, list-like, dict-like or function, optional
A scalar, list-like, dict-like or functions transformations to apply to that axis' values.

axis : {0 or 'index'}, default 0
The axis to rename. For `Series` this parameter is unused and defaults to 0.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

inplace : bool, default False
Modifies the object directly, instead of creating a new Series or DataFrame.

Returns

Series, or None
The same type as the caller or None if ``inplace=True``.

See Also

Series.rename : Alter Series index labels or name.
DataFrame.rename : Alter DataFrame index labels or name.
Index.rename : Set new names on index.

Examples

```
>>> s = pd.Series(["dog", "cat", "monkey"])
>>> s
0      dog
1      cat
2    monkey
dtype: str
>>> s.rename_axis("animal")
animal
0      dog
1      cat
2    monkey
dtype: str
```

`reorder_levels(self, order: Sequence[Level]) -> Series` from `pandas.core.series.Series`
Rearrange index levels using input order.

May not drop or duplicate levels.

Parameters

`order` : list of int representing new level order
Reference level by number or key.

Returns

Series

Type of caller with index as MultiIndex (new object).

See Also

`DataFrame.reorder_levels` : Rearrange index or column levels using
input ``order``.

Examples

```
>>> arrays = [
...     np.array(["dog", "dog", "cat", "cat", "bird", "bird"]),
...     np.array(["white", "black", "white", "black", "white", "black"]),
... ]
>>> s = pd.Series([1, 2, 3, 3, 5, 2], index=arrays)
>>> s
dog   white    1
      black    2
cat   white    3
      black    3
bird  white    5
      black    2
dtype: int64
>>> s.reorder_levels([1, 0])
white dog    1
black dog    2
white cat    3
black cat    3
white bird   5
black bird   2
dtype: int64
```

`repeat(self, repeats: int | Sequence[int], axis: None = None) -> Series` from `pandas.core.series.Series`
Repeat elements of a Series.

Returns a new Series where each element of the current Series
is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints

The number of repetitions for each element. This should be a
non-negative integer. Repeating 0 times will return an empty
Series.

`axis` : None

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

Newly created Series with repeated elements.

See Also

`Index.repeat` : Equivalent function for Index.
`numpy.repeat` : Similar method for `:class:`numpy.ndarray``.

Examples

```
>>> s = pd.Series(["a", "b", "c"])
>>> s
0    a
1    b
2    c
dtype: str
>>> s.repeat(2)
0    a
0    a
1    b
1    b
2    c
2    c
dtype: str
>>> s.repeat([1, 2, 3])
0    a
1    b
1    b
2    c
2    c
2    c
dtype: str
```

```
reset_index(
    self,
    level: IndexLabel | None = None,
    *,
    drop: bool = False,
    name: Level = <no_default>,
    inplace: bool = False,
    allow_duplicates: bool = False
) -> DataFrame | Series | None from pandas.core.series.Series
Generate a new DataFrame or Series with the index reset.
```

This is useful when the index needs to be treated as a column, or when the index is meaningless and needs to be reset to the default before another operation.

Parameters

`level` : int, str, tuple, or list, default optional
For a Series with a MultiIndex, only remove the specified levels from the index. Removes all levels by default.

`drop` : bool, default False
Just reset the index, without inserting it as a column in the new DataFrame.

`name` : object, optional
The name to use for the column containing the original Series values. Uses `self.name` by default. This argument is ignored when `drop` is True.

`inplace` : bool, default False
Modify the Series in place (do not create a new object).

`allow_duplicates` : bool, default False
Allow duplicate column labels to be created.

Returns

Series or DataFrame or None
When `drop` is False (the default), a DataFrame is returned. The newly created columns will come first in the DataFrame, followed by the original Series values.
When `drop` is True, a `Series` is returned.
In either case, if `inplace=True`, no value is returned.

See Also

DataFrame.reset_index: Analogous function for DataFrame.

Examples

```
>>> s = pd.Series(
...     [1, 2, 3, 4],
...     name="foo",
...     index=pd.Index(["a", "b", "c", "d"], name="idx"),
... )
```

Generate a DataFrame with default index.

```
>>> s.reset_index()
   idx  foo
0    a    1
1    b    2
2    c    3
3    d    4
```

To specify the name of the new column use `name`.

```
>>> s.reset_index(name="values")
   idx  values
0    a        1
1    b        2
2    c        3
3    d        4
```

To generate a new Series with the default set `drop` to True.

```
>>> s.reset_index(drop=True)
0    1
1    2
2    3
3    4
Name: foo, dtype: int64
```

The `level` parameter is interesting for Series with a multi-level index.

```
>>> arrays = [
...     np.array(["bar", "bar", "baz", "baz"]),
...     np.array(["one", "two", "one", "two"]),
... ]
>>> s2 = pd.Series(
...     range(4),
...     name="foo",
...     index=pd.MultiIndex.from_arrays(arrays, names=["a", "b"]),
... )
```

To remove a specific level from the Index, use `level`.

```
>>> s2.reset_index(level="a")
   a  foo
b
one bar    0
two bar    1
one baz    2
two baz    3
```

If `level` is not set, all levels are removed from the Index.

```
>>> s2.reset_index()
   a  b  foo
0  bar one    0
1  bar two    1
2  baz one    2
3  baz two    3
```

`rfloordiv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.com`
Return Integer division of series and other, element-wise (binary operator `rfloordiv`).

Equivalent to ``other // series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
 When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name
 Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)
 Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}
 Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
 The result of the operation.

See Also

Series.floordiv : Element-wise Integer division, see [Python documentation <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>](https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types) for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.floordiv(b, fill_value=0)
a    1.0
b    inf
c    inf
d    0.0
e    NaN
dtype: float64
```

rmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
 Return Modulo of series and other, element-wise (binary operator `rmod`).

Equivalent to ``other % series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
 When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name
 Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)
 Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}
 Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.mod : Element-wise Modulo, see
`Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`
for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.mod(b, fill_value=0)
a 0.0
b NaN
c NaN
d 0.0
e NaN
dtype: float64

rmul(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Multiplication of series and other, element-wise (binary operator `rmul`).

Equivalent to ``other \* series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``\*=`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.mul : Element-wise Multiplication, see
`Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`
for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64

```

>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.multiply(b, fill_value=0)
a    1.0
b    0.0
c    0.0
d    0.0
e    NaN
dtype: float64

```

`round(self, decimals: int = 0, *args, **kwargs) -> Series from pandas.core.series.Series`
 Round each value in a Series to the given number of decimals.

Parameters

`decimals` : int, default 0

Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

`*args, **kwargs`

Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series

Rounded values of the Series.

See Also

`numpy.around` : Round values of an np.array.

`DataFrame.round` : Round values of a DataFrame.

`Series.dt.round` : Round values of data to the specified freq.

Notes

For values exactly halfway between rounded decimal values, pandas rounds to the nearest even value (e.g. -0.5 and 0.5 round to 0.0, 1.5 and 2.5 round to 2.0, etc.).

Examples

```

>>> s = pd.Series([-0.5, 0.1, 2.5, 1.3, 2.7])

```

```

>>> s.round()

```

```

0    -0.0

```

```

1     0.0

```

```

2     2.0

```

```

3     1.0

```

```

4     3.0

```

```

dtype: float64

```

`rpow(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series.Series`
 Return Exponential power of series and other, element-wise (binary operator ``rpow``).

Equivalent to ``other ** series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``**`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.pow : Element-wise Exponential power, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
```

```
>>> b
```

```
a    1.0
```

```
b    NaN
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.pow(b, fill_value=0)
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    0.0
```

```
e    NaN
```

```
dtype: float64
```

rsub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series

Return Subtraction of series and other, element-wise (binary operator `rsub`).

Equivalent to ``other - series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.sub : Element-wise Subtraction, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
```

```
>>> a
```

```
a    1.0
```

```

b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.subtract(b, fill_value=0)
a    0.0
b    1.0
c    1.0
d   -1.0
e    NaN
dtype: float64

```

`rtruediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core`
Return Floating division of series and other, element-wise (binary operator ``rtruediv``).

Equivalent to ``other / series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.truediv : Element-wise Floating division, see ``Python documentation``
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types> for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.divide(b, fill_value=0)
a 1.0
b inf
c inf
d 0.0
e NaN
dtype: float64


```

searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.series.Series
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted Series `self` such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    .. note::
        The Series must be monotonically sorted, otherwise
        wrong locations will likely be returned. Pandas does not
        check this for you.

Parameters
-----
value : array-like or scalar
    Values to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort `self` into ascending
    order. They are typically the result of ``np.argsort``.

Returns
-----
int or array of int
    A scalar or array of insertion points with the
    same shape as `value`.

See Also
-----
sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes
-----
Binary search is used to find the required insertion points.

Examples
-----
>>> ser = pd.Series([1, 2, 3])
>>> ser
0    1
1    2
2    3
dtype: int64
>>> ser.searchsorted(4)
np.int64(3)
>>> ser.searchsorted([0, 4])
array([0, 3])
>>> ser.searchsorted([1, 3], side="left")
array([0, 2])
>>> ser.searchsorted([1, 3], side="right")
array([1, 3])
>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
>>> ser
0    2000-03-11
1    2000-03-12
2    2000-03-13
dtype: datetime64[us]
>>> ser.searchsorted("3/14/2000")
np.int64(3)
>>> ser = pd.Categorical(
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
... )
>>> ser
['apple', 'bread', 'bread', 'cheese', 'milk']
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']
>>> ser.searchsorted("bread")
np.int64(1)

```

```
>>> ser.searchsorted(["bread"], side="right")
array([3])
```

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])
>>> ser
0    2
1    1
2    3
dtype: int64
>>> ser.searchsorted(1) # doctest: +SKIP
0 # wrong result, correct would be 1
```

```
sem(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return unbiased standard error of the mean over requested axis.

Normalized by N-1 by default. This can be changed using the ddof argument

Parameters
-----
axis : {index (0)}
    This parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If an entire row/column is NA, the result
    will be NA.
ddof : int, default 1
    Delta Degrees of Freedom. The divisor used in calculations is N - ddof,
    where N represents the number of elements.
numeric_only : bool, default False
    Include only float, int, boolean columns. Not implemented for Series.
**kwargs :
    Additional keywords have no effect but might be accepted
    for compatibility with NumPy.

Returns
-----
scalar or Series (if level specified)
    Unbiased standard error of the mean over requested axis.

See Also
-----
scipy.stats.sem : Compute standard error of the mean.
Series.std : Return sample standard deviation over requested axis.
Series.var : Return unbiased variance over requested axis.
Series.mean : Return the mean of the values over the requested axis.
Series.median : Return the median of the values over the requested axis.
Series.mode : Return the mode(s) of the Series.

Examples
-----
>>> s = pd.Series([1, 2, 3])
>>> round(s.sem(), 6)
0.57735

set_axis(
    self,
    labels,
    *,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
Assign desired index to given axis.

.. deprecated:: 3.0.0
    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
```

(Copy-on-Write). See the ``user guide on Copy-on-Write``
<https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`\_\_`
for more details.

Indexes for row labels can be changed by assigning a list-like or Index.

Parameters

labels : list-like or Index
 The values for the new index.
axis : {0 or 'index'}, default 0
 The axis to update. The value 0 identifies the rows. For ``Series``
 this parameter is unused and defaults to 0.
copy : bool, default False
 This keyword is now ignored; changing its value will have no
 impact on the method.

Returns

Series

A shallow copy of the object with axis altered to the given index.

See Also

Series.rename_axis : Alter the name of the index.

Examples

>>> s = pd.Series([1, 2, 3])
>>> s
0 1
1 2
2 3
dtype: int64
>>> s.set_axis(["a", "b", "c"], axis=0)
a 1
b 2
c 3
dtype: int64

```
skew(  
    self,  
    *,  
    axis: Axis | None = 0,  
    skipna: bool = True,  
    numeric_only: bool = False,  
    **kwargs  
) from pandas.core.series.Series  
Return unbiased skew over requested axis.
```

Normalized by N-1.

Parameters

axis : {index (0)}
 This parameter is unused and defaults to 0.
skipna : bool, default True
 Exclude NA/null values when computing the result.
numeric_only : bool, default False
 Unused.
**kwargs
 Additional keyword arguments to be passed to the function.

Returns

scalar
 Unbiased skew of the Series.

See Also

Series.var : Return unbiased variance over requested axis.
Series.std : Return unbiased standard deviation over requested axis.

Examples

>>> s = pd.Series([1, 2, 3])

```

>>> s.skew()
0.0

```

sort_index(
self,
*,
axis: Axis = 0,
level: IndexLabel | None = None,
ascending: bool | Sequence[bool] = True,
inplace: bool = False,
kind: SortKind = 'quicksort',
na_position: NaPosition = 'last',
sort_remaining: bool = True,
ignore_index: bool = False,
key: IndexKeyFunc | None = None
) -> Series | None from pandas.core.series.Series
Sort Series by index labels.

Returns a new Series sorted by label if `inplace` argument is
`False`, otherwise updates the original series and returns None.

Parameters

axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.
level : int, optional
If not None, sort on values in specified index level(s).
ascending : bool or list-like of bools, default True
Sort ascending vs. descending. When the index is a MultiIndex the
sort direction can be controlled for each level individually.
inplace : bool, default False
If True, perform operation in-place.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
Choice of sorting algorithm. See also :func:`numpy.sort` for more
information. 'mergesort' and 'stable' are the only stable algorithms. For
DataFrames, this option is only applied when sorting on a single
column or label.
na_position : {'first', 'last'}, default 'last'
If 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
Not implemented for MultiIndex.
sort_remaining : bool, default True
If True and sorting by level and index is multilevel, sort by other
levels too (in order) after sorting by specified level.
ignore_index : bool, default False
If True, the resulting axis will be labeled 0, 1, ..., n - 1.
key : callable, optional
If not None, apply the key function to the index values
before sorting. This is similar to the `key` argument in the
builtin :meth:`sorted` function, with the notable difference that
this `key` function should be *vectorized*. It should expect an
`Index` and return an `Index` of the same shape.

Returns

Series or None
The original Series sorted by the labels or None if `inplace=True`.

See Also

DataFrame.sort_index: Sort DataFrame by the index.
DataFrame.sort_values: Sort DataFrame by the value.
Series.sort_values : Sort Series by the value.

Examples

>>> s = pd.Series(["a", "b", "c", "d"], index=[3, 2, 1, 4])
>>> s.sort_index()
1 c
2 b
3 a
4 d
dtype: str

Sort Descending

```

>>> s.sort_index(ascending=False)
4      d

```

```

3    a
2    b
1    c
dtype: str

```

By default NaNs are put at the end, but use `na\_position` to place them at the beginning

```

>>> s = pd.Series(["a", "b", "c", "d"], index=[3, 2, 1, np.nan])
>>> s.sort_index(na_position="first")
NaN    d
1.0    c
2.0    b
3.0    a
dtype: str

```

Specify index level to sort

```

>>> arrays = [
...     np.array(["qux", "qux", "foo", "foo", "baz", "baz", "bar", "bar"]),
...     np.array(["two", "one", "two", "one", "two", "one", "two", "one"]),
... ]
>>> s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=arrays)
>>> s.sort_index(level=1)
bar one    8
baz one    6
foo one    4
qux one    2
bar two    7
baz two    5
foo two    3
qux two    1
dtype: int64

```

Does not sort by remaining levels when sorting by levels

```

>>> s.sort_index(level=1, sort_remaining=False)
qux one    2
foo one    4
baz one    6
bar one    8
qux two    1
foo two    3
baz two    5
bar two    7
dtype: int64

```

Apply a key function before sorting

```

>>> s = pd.Series([1, 2, 3, 4], index=["A", "b", "C", "d"])
>>> s.sort_index(key=lambda x: x.str.lower())
A    1
b    2
C    3
d    4
dtype: int64

```

```

sort_values(
    self,
    *,
    axis: Axis = 0,
    ascending: bool | Sequence[bool] = True,
    inplace: bool = False,
    kind: SortKind = 'quicksort',
    na_position: NaPosition = 'last',
    ignore_index: bool = False,
    key: ValueKeyFunc | None = None
) -> Series | None from pandas.core.series.Series
Sort by the values.

```

Sort a Series in ascending or descending order by some criterion.

Parameters

```

axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

```

`ascending` : bool or list of bools, default True
 If True, sort values in ascending order, otherwise descending.
`inplace` : bool, default False
 If True, perform operation in-place.
`kind` : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
 Choice of sorting algorithm. See also :func:`numpy.sort` for more information. 'mergesort' and 'stable' are the only stable algorithms.
`na_position` : {'first' or 'last'}, default 'last'
 Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
`ignore_index` : bool, default False
 If True, the resulting axis will be labeled 0, 1, ..., n - 1.
`key` : callable, optional
 If not None, apply the key function to the series values before sorting. This is similar to the 'key' argument in the builtin :meth:`sorted` function, with the notable difference that this 'key' function should be *vectorized*. It should expect a ``Series`` and return an array-like.

Returns

 Series or None
 Series ordered by values or None if ``inplace=True``.

See Also

`Series.sort_index` : Sort by the Series indices.
`DataFrame.sort_values` : Sort DataFrame by the values along either axis.
`DataFrame.sort_index` : Sort DataFrame by indices.

Examples

```

>>> s = pd.Series([np.nan, 1, 3, 10, 5])
>>> s
0      NaN
1      1.0
2      3.0
3     10.0
4      5.0
dtype: float64

```

Sort values ascending order (default behavior)

```

>>> s.sort_values(ascending=True)
1      1.0
2      3.0
4      5.0
3     10.0
0      NaN
dtype: float64

```

Sort values descending order

```

>>> s.sort_values(ascending=False)
3     10.0
4      5.0
2      3.0
1      1.0
0      NaN
dtype: float64

```

Sort values putting NAs first

```

>>> s.sort_values(na_position="first")
0      NaN
1      1.0
2      3.0
4      5.0
3     10.0
dtype: float64

```

Sort a series of strings

```

>>> s = pd.Series(["z", "b", "d", "a", "c"])
>>> s
0      z
1      b

```

```

2    d
3    a
4    c
dtype: str

```

```

>>> s.sort_values()
3    a
1    b
4    c
2    d
0    z
dtype: str

```

Sort using a key function. Your `key` function will be given the `Series` of values and should return an array-like.

```

>>> s = pd.Series(["a", "B", "c", "D", "e"])
>>> s.sort_values()
1    B
3    D
0    a
2    c
4    e
dtype: str
>>> s.sort_values(key=lambda x: x.str.lower())
0    a
1    B
2    c
3    D
4    e
dtype: str

```

NumPy ufuncs work well here. For example, we can sort by the `sin` of the value

```

>>> s = pd.Series([-4, -2, 0, 2, 4])
>>> s.sort_values(key=np.sin)
1    -2
4     4
2     0
0    -4
3     2
dtype: int64

```

More complicated user-defined functions can be used, as long as they expect a Series and return an array-like

```

>>> s.sort_values(key=lambda x: (np.tan(x.cumsum()))))
0    -4
3     2
4     4
1    -2
2     0
dtype: int64

```

```

std(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return sample standard deviation.

```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

```

-----
axis : {index (0)}
    This parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If Series is NA, the result
    will be NA.
ddof : int, default 1
    Delta Degrees of Freedom. The divisor used in calculations is N - ddof,

```

where N represents the number of elements.
numeric_only : bool, default False
Not implemented for Series.
**kwargs :
Additional keywords have no effect but might be accepted
for compatibility with NumPy.

Returns

scalar

Standard deviation over all values in the Series.

See Also

numpy.std : Compute the standard deviation along the specified axis.
Series.var : Return unbiased variance over requested axis.
Series.sem : Return unbiased standard error of the mean over requested axis.
Series.mean : Return the mean of the values over the requested axis.
Series.median : Return the median of the values over the requested axis.
Series.mode : Return the mode(s) of the Series.

Examples

```
>>> s = pd.Series([1, 2, 3])
>>> s.std()
1.0
```

Alternatively, ``ddof=0`` can be set to normalize by N instead of $N-1$:

```
>>> s.std(ddof=0)
0.816496580927726
```

sub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Subtraction of series and other, element-wise (binary operator `sub`).

Equivalent to ``series - other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.rsub : Reverse of the Subtraction operator, see
`Python documentation`
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
```



```
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.subtract(b, fill_value=0)
a    0.0
b    1.0
c    1.0
d   -1.0
e    NaN
dtype: float64
```

`subtract = sub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

```
sum(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
) from pandas.core.series.Series
Return the sum of the values over the requested axis.
```

This is equivalent to the method ``numpy.sum``.

Parameters

`axis : {index (0)}`

Axis for the function to be applied on.

For ``Series`` this parameter is unused and defaults to 0.

.. warning::

The behavior of `DataFrame.sum` with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass `axis=0` (or do not pass `axis`).

.. versionadded:: 2.0.0

`skipna : bool, default True`

Exclude NA/null values when computing the result.

`numeric_only : bool, default False`

Include only float, int, boolean columns. Not implemented for `Series`.

`min_count : int, default 0`

The required number of valid values to perform the operation. If fewer than ``min\_count`` non-NA values are present the result will be NA.

`**kwargs`

Additional keyword arguments to be passed to the function.

Returns

scalar or `Series` (if level specified)

Sum of the values for the requested axis.

See Also

`numpy.sum` : Equivalent numpy function for computing sum.

`Series.mean` : Mean of the values.

`Series.median` : Median of the values.

`Series.std` : Standard deviation of the values.

`Series.var` : Variance of the values.

`Series.min` : Minimum value.

`Series.max` : Maximum value.

Examples

```
>>> idx = pd.MultiIndex.from_arrays(
...     [
...         ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
```

```

blooded  animal
warm     dog      4
         falcon   2
cold     fish     0
         spider   8
Name: legs, dtype: int64

```

```

>>> s.sum()
14

```

By default, the sum of an empty or all-NA Series is ``0``.

```

>>> pd.Series([], dtype="float64").sum() # min_count=0 is the default
0.0

```

This can be controlled with the ``min\_count`` parameter. For example, if you'd like the sum of an empty series to be NaN, pass ``min\_count=1``.

```

>>> pd.Series([], dtype="float64").sum(min_count=1)
nan

```

Thanks to the ``skipna`` parameter, ``min\_count`` handles all-NA and empty series identically.

```

>>> pd.Series([np.nan]).sum()
0.0

```

```

>>> pd.Series([np.nan]).sum(min_count=1)
nan

```

```

swaplevel(
    self,
    i: Level = -2,
    j: Level = -1,
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
    Swap levels i and j in a :class:`MultiIndex`.

```

Default is to swap the two innermost levels of the index.

Parameters

```

i, j : int or str
    Levels of the indices to be swapped. Can pass level name as string.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

Returns

Series

Series with levels swapped in MultiIndex.

See Also

```

DataFrame.swaplevel : Swap levels i and j in a :class:`DataFrame`.
Series.reorder_levels : Rearrange index levels using input order.
MultiIndex.swaplevel : Swap levels i and j in a :class:`MultiIndex`.

```

Examples

```

>>> s = pd.Series(
...     ["A", "B", "A", "C"],
...     index=[
...         ["Final exam", "Final exam", "Coursework", "Coursework"],
...         ["History", "Geography", "History", "Geography"],
...         ["January", "February", "March", "April"],
...     ],

```

```
... )
>>> s
Final exam History January A
          Geography February B
Coursework History March A
          Geography April C
dtype: str
```

In the following example, we will swap the levels of the indices. Here, we will swap the levels column-wise, but levels can be swapped row-wise in a similar manner. Note that column-wise is the default behavior. By not supplying any arguments for *i* and *j*, we swap the last and second to last indices.

```
>>> s.swaplevel()
Final exam January History A
          February Geography B
Coursework March History A
          April Geography C
dtype: str
```

By supplying one argument, we can choose which index to swap the last index with. We can for example swap the first index with the last one as follows.

```
>>> s.swaplevel(0)
January History Final exam A
February Geography Final exam B
March History Coursework A
April Geography Coursework C
dtype: str
```

We can also define explicitly which indices we want to swap by supplying values for both *i* and *j*. Here, we for example swap the first and second indices.

```
>>> s.swaplevel(0, 1)
History Final exam January A
Geography Final exam February B
History Coursework March A
Geography Coursework April C
dtype: str
```

```
to_dict(self, *, into: type[MutableMappingT] | MutableMappingT = <class 'dict'>) -> MutableMapping
Convert Series to {label -> value} dict or dict-like object.
```

Parameters

```
into : class, default dict
    The collections.abc.MutableMapping subclass to use as the return object. Can be the actual class or an empty instance of the mapping type you want. If you want a collections.defaultdict, you must pass it initialized.
```

Returns

```
collections.abc.MutableMapping
    Key-value representation of Series.
```

See Also

```
Series.to_list: Converts Series to a list of the values.
Series.to_numpy: Converts Series to NumPy ndarray.
Series.array: ExtensionArray of the data backing this Series.
```

Examples

```
>>> s = pd.Series([1, 2, 3, 4])
>>> s.to_dict()
{0: 1, 1: 2, 2: 3, 3: 4}
>>> from collections import OrderedDict, defaultdict
>>> s.to_dict(into=OrderedDict)
OrderedDict([(0, 1), (1, 2), (2, 3), (3, 4)])
>>> dd = defaultdict(list)
>>> s.to_dict(into=dd)
defaultdict(<class 'list'>, {0: 1, 1: 2, 2: 3, 3: 4})
```

```
to_frame(self, name: Hashable = <no_default>) -> DataFrame from pandas.core.series.Series
```

Convert Series to DataFrame.

Parameters

name : object, optional
The passed name should substitute for the series name (if it has one).

Returns

DataFrame
DataFrame representation of Series.

See Also

Series.to_dict : Convert Series to dict object.

Examples

>>> s = pd.Series(["a", "b", "c"], name="vals")
>>> s.to_frame()
vals
0 a
1 b
2 c

```
to_markdown(  
    self,  
    buf: IO[str] | None = None,  
    *,  
    mode: str = 'wt',  
    index: bool = True,  
    storage_options: StorageOptions | None = None,  
    **kwargs  
) -> str | None from pandas.core.series.Series  
Print Series in Markdown-friendly format.
```

Parameters

buf : str, Path or StringIO-like, optional, default None
Buffer to write to. If None, the output is returned as a string.
mode : str, optional
Mode in which file is opened, "wt" by default.
index : bool, optional, default True
Add index (row) labels.

storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

**kwargs

These parameters will be passed to ``tabulate``

[<https://pypi.org/project/tabulate/>](https://pypi.org/project/tabulate/)

Returns

str
Series in Markdown-friendly format.

See Also

Series.to_frame : Write a text representation of object to the system clipboard.
Series.to_latex : Render Series to LaTeX-formatted table.

Notes

Requires the ``tabulate`` [<https://pypi.org/project/tabulate/>](https://pypi.org/project/tabulate/) package.

Examples

>>> s = pd.Series(["elk", "pig", "dog", "quetzal"], name="animal")
>>> print(s.to_markdown())

```
|      | animal |
|----:|:-----|
| 0    | elk    |
| 1    | pig    |
| 2    | dog    |
| 3    | quetzal |
```

Output markdown with a tabulate option.

```
>>> print(s.to_markdown(tablefmt="grid"))
+-----+
|      | animal |
+=====+
| 0    | elk    |
+-----+
| 1    | pig    |
+-----+
| 2    | dog    |
+-----+
| 3    | quetzal |
+-----+
```

```
to_period(
    self,
    freq: str | None = None,
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
    Convert Series from DatetimeIndex to PeriodIndex.
```

Parameters

```
-----
freq : str, default None
    Frequency associated with the PeriodIndex.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`_ for more details.

Returns

```
-----
Series
    Series with index converted to PeriodIndex.
```

See Also

```
-----
DataFrame.to_period: Equivalent method for DataFrame.
Series.dt.to_period: Convert DateTime column values.
```

Examples

```
-----
>>> idx = pd.DatetimeIndex(["2023", "2024", "2025"])
>>> s = pd.Series([1, 2, 3], index=idx)
>>> s = s.to_period()
>>> s
2023    1
2024    2
2025    3
Freq: Y-DEC, dtype: int64
```

Viewing the index

```
>>> s.index
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')
```

```
to_string(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    na_rep: str = 'NaN',
```

```

float_format: str | None = None,
header: bool = True,
index: bool = True,
length: bool = False,
dtype: bool = False,
name: bool = False,
max_rows: int | None = None,
min_rows: int | None = None
) -> str | None from pandas.core.series.Series
Render a string representation of the Series.

Parameters
-----
buf : StringIO-like, optional
    Buffer to write to.
na_rep : str, optional
    String representation of NaN to use, default 'NaN'.
float_format : one-parameter function, optional
    Formatter function to apply to columns' elements if they are
    floats, default None.
header : bool, default True
    Add the Series header (index name).
index : bool, optional
    Add index (row) labels, default True.
length : bool, default False
    Add the Series length.
dtype : bool, default False
    Add the Series dtype.
name : bool, default False
    Add the Series name if not None.
max_rows : int, optional
    Maximum number of rows to show before truncating. If None, show
    all.
min_rows : int, optional
    The number of rows to display in a truncated repr (when number
    of rows is above `max_rows`).

Returns
-----
str or None
    String representation of Series if ``buf=None``, otherwise None.

See Also
-----
Series.to_dict : Convert Series to dict object.
Series.to_frame : Convert Series to DataFrame object.
Series.to_markdown : Print Series in Markdown-friendly format.
Series.to_timestamp : Cast to DatetimeIndex of Timestamps.

```

Examples

```

>>> ser = pd.Series([1, 2, 3]).to_string()
>>> ser
'0    1\n1    2\n2    3'

```

```

to_timestamp(
    self,
    freq: Frequency | None = None,
    how: Literal['s', 'e', 'start', 'end'] = 'start',
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
Cast to DatetimeIndex of Timestamps, at *beginning* of period.

```

This can be changed to the *end* of the period, by specifying `how="e"`.

Parameters

```

freq : str, default frequency of PeriodIndex
    Desired frequency.
how : {'s', 'e', 'start', 'end'}
    Convention for converting period to timestamp; start of period
    vs. end.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html for more details.

Returns

Series with DatetimeIndex
Series with the PeriodIndex cast to DatetimeIndex.

See Also

Series.to_period: Inverse method to cast DatetimeIndex to PeriodIndex.
DataFrame.to_timestamp: Equivalent method for DataFrame.

Examples

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> s1 = pd.Series([1, 2, 3], index=idx)
>>> s1
2023 1
2024 2
2025 3
Freq: Y-DEC, dtype: int64

The resulting frequency of the Timestamps is ``YearBegin``

```
>>> s1 = s1.to_timestamp()
>>> s1
2023-01-01    1
2024-01-01    2
2025-01-01    3
Freq: YS-JAN, dtype: int64
```

Using ``freq`` which is the offset that the Timestamps will have

```
>>> s2 = pd.Series([1, 2, 3], index=idx)
>>> s2 = s2.to_timestamp(freq="M")
>>> s2
2023-01-31    1
2024-01-31    2
2025-01-31    3
Freq: YE-JAN, dtype: int64
```

transform(self, func: AggFuncType, axis: Axis = 0, *args, **kwargs) -> DataFrame | Series
Call ``func`` on self producing a Series with the same axis shape as self.

Parameters

func : function, str, list-like or dict-like
Function to use for transforming the data. If a function, must either work when passed a Series or when passed to Series.apply. If func is both list-like and dict-like, dict-like behavior takes precedence.

Accepted combinations are:

- function
- string function name
- list-like of functions and/or function names, e.g. ``[np.exp, 'sqrt']``
- dict-like of axis labels -> functions, function names or list-like of such

axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

*args
Positional arguments to pass to ``func``.
**kwargs
Keyword arguments to pass to ``func``.

Returns

Series
A Series that must have the same length as self.

Raises

ValueError : If the returned Series has a different length than self.

See Also

Series.agg : Only perform aggregating type operations.

Series.apply : Invoke function on a Series.

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

Examples

```
>>> df = pd.DataFrame({"A": range(3), "B": range(1, 4)})
```

```
>>> df
```

```
A  B
0  0  1
1  1  2
2  2  3
```

```
>>> df.transform(lambda x: x + 1)
```

```
A  B
0  1  2
1  2  3
2  3  4
```

Even though the resulting Series must have the same length as the input Series, it is possible to provide several input functions:

```
>>> s = pd.Series(range(3))
```

```
>>> s
```

```
0    0
1    1
2    2
```

```
dtype: int64
```

```
>>> s.transform([np.sqrt, np.exp])
```

```
      sqrt      exp
0  0.000000  1.000000
1  1.000000  2.718282
2  1.414214  7.389056
```

You can call transform on a GroupBy object:

```
>>> df = pd.DataFrame(
...     {
...         "Date": [
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...         ],
...         "Data": [5, 8, 6, 1, 50, 100, 60, 120],
...     }
... )
>>> df
```

```
   Date  Data
0 2015-05-08    5
1 2015-05-07    8
2 2015-05-06    6
3 2015-05-05    1
4 2015-05-08   50
5 2015-05-07  100
6 2015-05-06   60
7 2015-05-05  120
```

```
>>> df.groupby("Date")["Data"].transform("sum")
```

```
0    55
1   108
2    66
3   121
4    55
```



```

5    108
6     66
7    121
Name: Data, dtype: int64

```

```

>>> df = pd.DataFrame(
...     {
...         "c": [1, 1, 1, 2, 2, 2, 2],
...         "type": ["m", "n", "o", "m", "m", "n", "n"],
...     }
... )

```

```

>>> df

```

```

c type
0  1  m
1  1  n
2  1  o
3  2  m
4  2  m
5  2  n
6  2  n

```

```

>>> df["size"] = df.groupby("c")["type"].transform(len)

```

```

>>> df
c type size
0  1  m     3
1  1  n     3
2  1  o     3
3  2  m     4
4  2  m     4
5  2  n     4
6  2  n     4

```

`truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core`
Return Floating division of series and other, element-wise (binary operator ``truediv``)

Equivalent to ``series / other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

other : Series or scalar value
Series with which to compute division.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.rtruediv` : Reverse of the Floating division operator, see ``Python documentation``
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types> for more details.

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])

```

```

>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64

```

```

>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b

```

```

a    1.0
b    NaN

```

```

d    1.0
e    NaN
dtype: float64
>>> a.divide(b, fill_value=0)
a    1.0
b    inf
c    inf
d    0.0
e    NaN
dtype: float64

```

unique(self) -> ArrayLike from pandas.core.series.Series
Return unique values of Series object.

Uniques are returned in order of appearance. Hash table-based unique, therefore does NOT sort.

Returns

ndarray or ExtensionArray

The unique values returned as a NumPy array. See Notes.

See Also

Series.drop_duplicates : Return Series with duplicate values removed.

unique : Top-level unique method for any 1-d array-like object.

Index.unique : Return Index with unique values from an Index object.

Notes

Returns the unique values as a NumPy array. In case of an extension-array backed Series, a new :class:`~api.extensions.ExtensionArray` of that type with just the unique values is returned. This includes

- * Categorical
- * Period
- * Datetime with Timezone
- * Datetime without Timezone
- * Timedelta
- * Interval
- * Sparse
- * IntegerNA

See Examples section.

Examples

```

>>> pd.Series([2, 1, 3, 3], name="A").unique()
array([2, 1, 3])

```

```

>>> pd.Series([pd.Timestamp("2016-01-01") for _ in range(3)]).unique()
<DatetimeArray>
['2016-01-01 00:00:00']
Length: 1, dtype: datetime64[us]

```

```

>>> pd.Series(
...     [pd.Timestamp("2016-01-01", tz="US/Eastern") for _ in range(3)]
... ).unique()
<DatetimeArray>
['2016-01-01 00:00:00-05:00']
Length: 1, dtype: datetime64[us, US/Eastern]

```

A Categorical will return categories in the order of appearance and with the same dtype.

```

>>> pd.Series(pd.Categorical(list("baabc"))).unique()
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> pd.Series(
...     pd.Categorical(list("baabc"), categories=list("abc"), ordered=True)
... ).unique()
['b', 'a', 'c']
Categories (3, str): ['a' < 'b' < 'c']

```

```

unstack(
    self,

```

```

    level: IndexLabel = -1,
    fill_value: Hashable | None = None,
    sort: bool = True
) -> DataFrame from pandas.core.series.Series
Unstack, also known as pivot, Series with MultiIndex to produce DataFrame.

```

Parameters

```

-----
level : int, str, or list of these, default last level
    Level(s) to unstack, can pass level name.
fill_value : scalar value, default None
    Value to use when replacing NaN values.
sort : bool, default True
    Sort the level(s) in the resulting MultiIndex columns.

```

Returns

```

-----
DataFrame
    Unstacked Series.

```

See Also

```

-----
DataFrame.unstack : Pivot the MultiIndex of a DataFrame.

```

Notes

```

-----
Reference :ref:`the user guide <reshaping.stacking>` for more examples.

```

Examples

```

-----
>>> s = pd.Series(
...     [1, 2, 3, 4],
...     index=pd.MultiIndex.from_product([["one", "two"], ["a", "b"]]),
... )
>>> s
one  a    1
     b    2
two  a    3
     b    4
dtype: int64

>>> s.unstack(level=-1)
     a  b
one  1  2
two  3  4

>>> s.unstack(level=0)
     one  two
a      1    3
b      2    4

```

```

update(self, other: Series | Sequence | Mapping) -> None from pandas.core.series.Series
Modify Series in place using values from passed Series.

```

Uses non-NA values from passed Series to make updates. Aligns on index.

Parameters

```

-----
other : Series, or object coercible into Series
    Other Series that provides values to update the current Series.

```

See Also

```

-----
Series.combine : Perform element-wise operation on two Series
    using a given function.
Series.transform: Modify a Series using a function.

```

Examples

```

-----
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, 5, 6]))
>>> s
0    4
1    5
2    6
dtype: int64

```

```
>>> s = pd.Series(["a", "b", "c"])
>>> s.update(pd.Series(["d", "e"], index=[0, 2]))
>>> s
0    d
1    b
2    e
dtype: str
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, 5, 6, 7, 8]))
>>> s
0    4
1    5
2    6
dtype: int64
```

If ``other`` contains NaNs the corresponding values are not updated in the original Series.

```
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, np.nan, 6]))
>>> s
0    4
1    2
2    6
dtype: int64
```

``other`` can also be a non-Series object type that is coercible into a Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.update([4, np.nan, 6])
>>> s
0    4
1    2
2    6
dtype: int64
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.update({1: 9})
>>> s
0    1
1    9
2    3
dtype: int64
```

```
var(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return unbiased variance over requested axis.
```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

axis : {index (0)}
For ``Series`` this parameter is unused and defaults to 0.

.. warning::

The behavior of DataFrame.var with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis).

skipna : bool, default True

Exclude NA/null values. If an entire row/column is NA, the result will be NA.

ddof : int, default 1

Delta Degrees of Freedom. The divisor used in calculations is N - ddof, where N represents the number of elements.

numeric_only : bool, default False
Include only float, int, boolean columns. Not implemented for Series.
**kwargs :
Additional keywords passed.

Returns

scalar or Series (if level specified)
Unbiased variance over requested axis.

See Also

numpy.var : Equivalent function in NumPy.
Series.std : Returns the standard deviation of the Series.
DataFrame.var : Returns the variance of the DataFrame.
DataFrame.std : Return standard deviation of the values over
the requested axis.

Examples

>>> df = pd.DataFrame(
... {
... "person_id": [0, 1, 2, 3],
... "age": [21, 25, 62, 43],
... "height": [1.61, 1.87, 1.49, 2.01],
... }
...).set_index("person_id")
>>> df

	age	height
person_id		
0	21	1.61
1	25	1.87
2	62	1.49
3	43	2.01

>>> df.var()
age 352.916667
height 0.056367
dtype: float64

Alternatively, ``ddof=0`` can be set to normalize by N instead of N-1:

>>> df.var(ddof=0)
age 264.687500
height 0.042275
dtype: float64

Class methods defined here:

from_arrow(data: ArrowArrayExportable | ArrowStreamExportable) -> Series from pandas.core.series
Construct a Series from an array-like Arrow object.

This function accepts any Arrow-compatible array-like object implementing
the `Arrow PyCapsule Protocol` (i.e. having an `__arrow_c_array__`
or `__arrow_c_stream__` method).

This function currently relies on `pyarrow` to convert the object
in Arrow format to pandas.

.. _Arrow PyCapsule Protocol: <https://arrow.apache.org/docs/format/CDataInterface/PyCapsuleProtocol.html>

.. versionadded:: 3.0

Parameters

data : pyarrow.Array or Arrow-compatible object
Any array-like object implementing the Arrow PyCapsule Protocol
(i.e. has an `__arrow_c_array__` or `__arrow_c_stream__`
method).

Returns

Series

See Also

DataFrame.from_arrow : Construct a DataFrame from an Arrow object.

Examples

```
>>> import pyarrow as pa
>>> arrow_array = pa.array([1, 2, 3])
>>> pd.Series.from_arrow(arrow_array)
0    1
1    2
2    3
dtype: int64
```

Readonly properties defined here:

array

The ExtensionArray of the data backing this Series or Index.

This property provides direct access to the underlying array data of a Series or Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

```.array``` differs from ```.values```, which may require converting the data to a different form.

#### See Also

Index.to\_numpy : Similar method that always returns a NumPy array.  
Series.to\_numpy : Similar method that always returns a NumPy array.

#### Notes

-----  
This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ```.array``` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

#### Examples

-----  
For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Series([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
```

```
>>> ser.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

axes

Return a list of the row axis labels.

dtype

Return the dtype object of the underlying data.

See Also

-----

Series.dtypes : Return the dtype object of the underlying data.

Series.astype : Cast a pandas object to a specified dtype dtype.

Series.convert\_dtypes : Convert columns to the best possible dtypes using dtypes supporting pd.NA.

Examples

-----

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> s.dtype
```

```
dtype('int64')
```

dtypes

Return the dtype object of the underlying data.

See Also

-----

DataFrame.dtypes : Return the dtypes in the DataFrame.

Examples

-----

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> s.dtypes
```

```
dtype('int64')
```

hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

-----

bool

See Also

-----

Series.isna : Detect missing values.

Series.notna : Detect existing (non-missing) values.

Examples

-----

```
>>> s = pd.Series([1, 2, 3, None])
```

```
>>> s
```

```
0 1.0
```

```
1 2.0
```

```
2 3.0
```

```
3 NaN
```

```
dtype: float64
```

```
>>> s.hasnans
```

```
True
```

values

Return Series as ndarray or ndarray-like depending on the dtype.

.. warning::

We recommend using :attr:`Series.array` or  
:meth:`Series.to\_numpy`, depending on whether you need  
a reference to the underlying data or a NumPy array.

Returns

-----

numpy.ndarray or ndarray-like

See Also

-----

Series.array : Reference to the underlying data.  
Series.to\_numpy : A NumPy array representing the underlying data.

#### Examples

```
>>> pd.Series([1, 2, 3]).values
array([1, 2, 3])
```

```
>>> pd.Series(list("aabc")).values
<ArrowStringArray>
['a', 'a', 'b', 'c']
Length: 4, dtype: str
```

```
>>> pd.Series(list("aabc")).astype("category").values
['a', 'a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Timezone aware datetime data is converted to UTC:

```
>>> pd.Series(pd.date_range("20130101", periods=3, tz="US/Eastern")).values
array(['2013-01-01T05:00:00.000000',
 '2013-01-02T05:00:00.000000',
 '2013-01-03T05:00:00.000000'], dtype='datetime64[us]')
```

-----  
Data descriptors defined here:

#### index

The index (axis labels) of the Series.

The index of a Series is used to label and identify each element of the underlying data. The index can be thought of as an immutable ordered set (technically a multi-set, as it may contain duplicate labels), and is used to index and align data in pandas.

#### Returns

##### Index

The index labels of the Series.

#### See Also

Series.reindex : Conform Series to new index.  
Index : The base pandas index type.

#### Notes

For more information on pandas indexing, see the ``indexing user guide` <[https://pandas.pydata.org/docs/user\\_guide/indexing.html](https://pandas.pydata.org/docs/user_guide/indexing.html)>`\_\_.

#### Examples

To create a Series with a custom index and view the index labels:

```
>>> cities = ['Kolkata', 'Chicago', 'Toronto', 'Lisbon']
>>> populations = [14.85, 2.71, 2.93, 0.51]
>>> city_series = pd.Series(populations, index=cities)
>>> city_series.index
Index(['Kolkata', 'Chicago', 'Toronto', 'Lisbon'], dtype='object')
```

To change the index labels of an existing Series:

```
>>> city_series.index = ['KOL', 'CHI', 'TOR', 'LIS']
>>> city_series.index
Index(['KOL', 'CHI', 'TOR', 'LIS'], dtype='object')
```

#### name

Return the name of the Series.

The name of a Series becomes its index or column name if it is used to form a DataFrame. It is also used whenever displaying the Series using the interpreter.

#### Returns

label (hashable object)

The name of the Series, also the column name if part of a DataFrame.



See Also

Series.rename : Sets the Series name when given a scalar input.  
Index.name : Corresponding Index property.

Examples

The Series name can be set initially when calling the constructor.

```
>>> s = pd.Series([1, 2, 3], dtype=np.int64, name="Numbers")
```

```
>>> s
0 1
1 2
2 3
Name: Numbers, dtype: int64
```

```
>>> s.name = "Integers"
```

```
>>> s
0 1
1 2
2 3
Name: Integers, dtype: int64
```

The name of a Series within a DataFrame is its column name.

```
>>> df = pd.DataFrame(
... [[1, 2], [3, 4], [5, 6]], columns=["Odd Numbers", "Even Numbers"]
...)
```

```
>>> df
 Odd Numbers Even Numbers
0 1 2
1 3 4
2 5 6
```

```
>>> df["Even Numbers"].name
'Even Numbers'
```

-----  
Data and other attributes defined here:

```
__annotations__ = {'_AXIS_ORDERS': "list[Literal['index', 'columns']]"}...
```

```
__pandas_priority__ = 3000
```

```
cat = <class 'pandas.core.arrays.categorical.CategoricalAccessor'>
Accessor object for categorical properties of the Series values.
```

Parameters

data : Series or CategoricalIndex

The object to which the categorical accessor is attached.

See Also

Series.dt : Accessor object for datetimelike properties of the Series values.  
Series.sparse : Accessor for sparse matrix data types.

Examples

```
>>> s = pd.Series(list("abbccc")).astype("category")
```

```
>>> s
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

```
>>> s.cat.categories
Index(['a', 'b', 'c'], dtype='str')
```

```
>>> s.cat.rename_categories(list("cba"))
```

```
0 c
1 b
2 b
3 a
```

```

4 a
5 a
dtype: category
Categories (3, str): ['c', 'b', 'a']

>>> s.cat.reorder_categories(list("cba"))
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['c', 'b', 'a']

>>> s.cat.add_categories(["d", "e"])
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (5, str): ['a', 'b', 'c', 'd', 'e']

>>> s.cat.remove_categories(["a", "c"])
0 NaN
1 b
2 b
3 NaN
4 NaN
5 NaN
dtype: category
Categories (1, str): ['b']

>>> s1 = s.cat.add_categories(["d", "e"])
>>> s1.cat.remove_unused_categories()
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']

>>> s.cat.set_categories(list("abcde"))
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (5, str): ['a', 'b', 'c', 'd', 'e']

>>> s.cat.as_ordered()
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a' < 'b' < 'c']

>>> s.cat.as_unordered()
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']

```

```
dt = <class 'pandas.core.indexes.accessors.CombinedDatetimelikePropert...
 Accessor object for Series values' datetime-like, timedelta and period properties.
```

See Also

-----  
DatetimeIndex : Index of datetime64 data.

Examples

```

>>> dates = pd.Series(
... ["2024-01-01", "2024-01-15", "2024-02-5"], dtype="datetime64[ns]"
...)
```

```
>>> dates.dt.day
```

```
0 1
```

```
1 15
```

```
2 5
```

```
dtype: int32
```

```
>>> dates.dt.month
```

```
0 1
```

```
1 1
```

```
2 2
```

```
dtype: int32
```

```
>>> dates = pd.Series(
... ["2024-01-01", "2024-01-15", "2024-02-5"], dtype="datetime64[ns, UTC]"
...)
```

```
>>> dates.dt.day
```

```
0 1
```

```
1 15
```

```
2 5
```

```
dtype: int32
```

```
>>> dates.dt.month
```

```
0 1
```

```
1 1
```

```
2 2
```

```
dtype: int32
```

```
list = <class 'pandas.core.arrays.arrow.accessors.ListAccessor'>
 Accessor object for list data properties of the Series values.
```

Parameters

-----  
data : Series  
 Series containing Arrow list data.

```
plot = <class 'pandas.plotting.PlotAccessor'>
 Make plots of Series or DataFrame.
```

Uses the backend specified by the  
option ``plotting.backend``. By default, matplotlib is used.

Parameters

-----  
data : Series or DataFrame  
 The object for which the method is called.

Attributes

-----  
x : label or position, default None  
 Only used if data is a DataFrame.

y : label, position or list of label, positions, default None  
 Allows plotting of one column versus another. Only used if data is a  
 DataFrame.

kind : str  
 The kind of plot to produce:

- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot

- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

ax : matplotlib axes object, default None  
An axes of the current figure.

subplots : bool or sequence of iterables, default False  
Whether to group columns into subplots:

- ``False`` : No subplots will be used
- ``True`` : Make separate subplots for each column.
- sequence of iterables of column labels: Create a subplot for each group of columns. For example ``[('a', 'c'), ('b', 'd')]`` will create 2 subplots: one with columns 'a' and 'c', and one with columns 'b' and 'd'. Remaining columns that aren't specified will be plotted in additional subplots (one per column).

sharex : bool, default True if ax is None else False  
In case ``subplots=True``, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in; Be aware, that passing in both an ax and ``sharex=True`` will alter all x axis labels for all axis in a figure.

sharey : bool, default False  
In case ``subplots=True``, share y axis and set some y axis labels to invisible.

layout : tuple, optional  
(rows, columns) for the layout of subplots.

figsize : a tuple (width, height) in inches  
Size of a figure object.

use\_index : bool, default True  
Use index as ticks for x axis.

title : str or list  
Title to use for the plot. If a string is passed, print the string at the top of the figure. If a list is passed and ``subplots`` is True, print each item in the list above the corresponding subplot.

grid : bool, default None (matlab style default)  
Axis grid lines.

legend : bool or {'reverse'}  
Place legend on axis subplots.

style : list or dict  
The matplotlib line style per column.

logx : bool or 'sym', default False  
Use log scaling or symlog scaling on x axis.

logy : bool or 'sym' default False  
Use log scaling or symlog scaling on y axis.

loglog : bool or 'sym', default False  
Use log scaling or symlog scaling on both x and y axes.

xticks : sequence  
Values to use for the xticks.

yticks : sequence  
Values to use for the yticks.

xlim : 2-tuple/list  
Set the x limits of the current axes.

ylim : 2-tuple/list  
Set the y limits of the current axes.

xlabel : label, optional  
Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.

.. versionchanged:: 2.0.0  
Now applicable to histograms.

ylabel : label, optional  
Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.

.. versionchanged:: 2.0.0  
Now applicable to histograms.

rot : float, default None  
Rotation for ticks (xticks for vertical, yticks for horizontal plots).

fontsize : float, default None  
Font size for xticks and yticks.

colormap : str or matplotlib colormap object, default None

Colormap to select colors from. If string, load colormap with that name from matplotlib.

colorbar : bool, optional  
If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).

position : float  
Specify relative alignments for bar plot layout.  
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).

table : bool, Series or DataFrame, default False  
If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout.  
If a Series or DataFrame is passed, use passed data to draw a table.

yerr : DataFrame, Series, array-like, dict and str  
See :ref:`Plotting with Error Bars <visualization.errorbars>` for detail.

xerr : DataFrame, Series, array-like, dict and str  
Equivalent to yerr.

stacked : bool, default False in line and bar plots, and True in area plot  
If True, create stacked plot.

secondary\_y : bool or sequence, default False  
Whether to plot on the secondary y-axis if a list/tuple, which columns to plot on secondary y-axis.

mark\_right : bool, default True  
When using a secondary\_y axis, automatically mark the column labels with "(right)" in the legend.

include\_bool : bool, default is False  
If True, boolean values can be plotted.

backend : str, default None  
Backend to use instead of the backend specified in the option ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to specify the ``plotting.backend`` for the whole session, set ``pd.options.plotting.backend``.

**\*\*kwargs**  
Options to pass to matplotlib plotting method.

#### Returns

-----  
:class:`matplotlib.axes.Axes` or numpy.ndarray of them  
If the backend is not the default matplotlib one, the return value will be the object returned by the backend.

#### See Also

-----  
matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.  
DataFrame.hist : Make a histogram.  
DataFrame.boxplot : Make a box plot.  
DataFrame.plot.scatter : Make a scatter plot with varying marker point size and color.  
DataFrame.plot.hexbin : Make a hexagonal binning plot of two variables.  
DataFrame.plot.kde : Make Kernel Density Estimate plot using Gaussian kernels.  
DataFrame.plot.area : Make a stacked area plot.  
DataFrame.plot.bar : Make a bar plot.  
DataFrame.plot.barh : Make a horizontal bar plot.

#### Notes

-----  
- See matplotlib documentation online for more on this subject  
- If `kind` = 'bar' or 'barh', you can specify relative alignments for bar plot layout by `position` keyword.  
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center)

#### Examples

-----  
For Series:

```
.. plot::
 :context: close-figs

 >>> ser = pd.Series([1, 2, 3, 3])
 >>> plot = ser.plot(kind="hist", title="My plot")
```

For DataFrame:

```

.. plot::
:context: close-figs

>>> df = pd.DataFrame(
... {
... "length": [1.5, 0.5, 1.2, 0.9, 3],
... "width": [0.7, 0.2, 0.15, 0.2, 1.1],
... },
... index=["pig", "rabbit", "duck", "chicken", "horse"],
...)
>>> plot = df.plot(title="DataFrame Plot")

For SeriesGroupBy:

.. plot::
:context: close-figs

>>> lst = [-1, -2, -3, 1, 2, 3]
>>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
>>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")

For DataFrameGroupBy:

.. plot::
:context: close-figs

>>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})
>>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")

```

sparse = <class 'pandas.core.arrays.sparse.accessor.SparseAccessor'>  
 Accessor for SparseSparse from other sparse matrix data types.

#### Parameters

data : Series or DataFrame  
 The Series or DataFrame to which the SparseAccessor is attached.

#### See Also

Series.sparse.to\_coo : Create a scipy.sparse.coo\_matrix from a Series with MultiIndex.  
 Series.sparse.from\_coo : Create a Series with sparse values from a scipy.sparse.coo\_matrix.

#### Examples

```

>>> ser = pd.Series([0, 0, 2, 2, 2], dtype="Sparse[int]")
>>> ser.sparse.density
0.6
>>> ser.sparse.sp_values
array([2, 2, 2])

```

str = <class 'pandas.core.strings.accessor.StringMethods'>  
 Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method.  
 Patterned after Python's string methods, with some inspiration from R's stringr package.

#### Parameters

data : Series or Index  
 The content of the Series or Index.

#### See Also

Series.str : Vectorized string functions for Series.  
 Index.str : Vectorized string functions for Index.

#### Examples

```

>>> s = pd.Series(["A_Str_Series"])
>>> s
0 A_Str_Series

```

```
dtype: str

>>> s.str.split("_")
0 [A, Str, Series]
dtype: object

>>> s.str.replace("_", "")
0 AStrSeries
dtype: str
```

struct = <class 'pandas.core.arrays.arrow.accessors.StructAccessor'>  
 Accessor object for structured data properties of the Series values.

Parameters

-----  
 data : Series

Series containing Arrow struct data.

-----  
 Methods inherited from pandas.core.base.IndexOpsMixin:

\_\_iter\_\_(self) -> Iterator  
 Return an iterator of the values.

These are each a scalar type, which is a Python scalar  
 (for str, int, float) or a pandas scalar  
 (for Timestamp/Timedelta/Interval/Period)

Returns

-----  
 iterator

An iterator yielding scalar values from the Series.

See Also

-----

Series.items : Lazily iterate over (index, value) tuples.

Examples

-----

```
>>> s = pd.Series([1, 2, 3])
>>> for x in s:
... print(x)
1
2
3
```

argmax(self, axis: AxisInt | None = None, skipna: bool = True, \*args, \*\*kwargs) -> int  
 Return int position of the largest value in the Series.

If the maximum is achieved in multiple locations,  
 the first row position is returned.

Parameters

-----

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``  
 and there is an NA value, this method will raise a ``ValueError``.

\*args, \*\*kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

-----

int

Row position of the maximum value.

See Also

-----

Series.argmax : Return position of the maximum value.

Series.argmin : Return position of the minimum value.

numpy.ndarray.argmax : Equivalent method for numpy arrays.

Series.idxmax : Return index label of the maximum values.

Series.idxmin : Return index label of the minimum values.

## Examples

-----

Consider dataset containing cereal calories

```
>>> s = pd.Series(
... [100.0, 110.0, 120.0, 110.0],
... index=[
... "Corn Flakes",
... "Almond Delight",
... "Cinnamon Toast Crunch",
... "Cocoa Puff",
...],
...)
>>> s
Corn Flakes 100.0
Almond Delight 110.0
Cinnamon Toast Crunch 120.0
Cocoa Puff 110.0
dtype: float64
```

```
>>> s.argmax()
np.int64(2)
>>> s.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since series is zero-indexed.

`argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int`  
Return int position of the smallest value in the Series.

If the minimum is achieved in multiple locations, the first row position is returned.

## Parameters

-----

`axis` : None

Unused. Parameter needed for compatibility with DataFrame.

`skipna` : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

`*args, **kwargs`

Additional arguments and keywords for compatibility with NumPy.

## Returns

-----

int

Row position of the minimum value.

## See Also

-----

`Series.argmax` : Return position of the maximum value.

`Series.argmax` : Return position of the maximum value.

`numpy.ndarray.argmax` : Equivalent method for numpy arrays.

`Series.idxmin` : Return index label of the minimum values.

`Series.idxmax` : Return index label of the maximum values.

## Examples

-----

Consider dataset containing cereal calories

```
>>> s = pd.Series(
... [100.0, 110.0, 120.0, 110.0],
... index=[
... "Corn Flakes",
... "Almond Delight",
... "Cinnamon Toast Crunch",
... "Cocoa Puff",
...],
...)
>>> s
Corn Flakes 100.0
Almond Delight 110.0
Cinnamon Toast Crunch 120.0
Cocoa Puff 110.0
dtype: float64
```



```
>>> s.argmax()
np.int64(2)
>>> s.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since series is zero-indexed.

```
factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.in
Encode the object as an enumerated type or categorical variable.
```

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function `:func:`pandas.factorize``, and as a method `:meth:`Series.factorize`` and `:meth:`Index.factorize``.

#### Parameters

-----

`sort` : bool, default False

Sort `uniques` and shuffle `codes` to maintain the relationship.

`use_na_sentinel` : bool, default True

If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

#### Returns

-----

`codes` : ndarray

An integer ndarray that's an indexer into `uniques`.

`uniques.take(codes)` will have the same values as `values`.

`uniques` : ndarray, Index, or Categorical

The unique valid values. When `values` is Categorical, `uniques` is a Categorical. When `values` is some other pandas object, an `Index` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in `values`, `uniques` will *not* contain an entry for it.

#### See Also

-----

`cut` : Discretize continuous-valued array.

`unique` : Find the unique value in an array.

#### Notes

-----

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

#### Examples

-----

These examples all show `factorize` as a top-level method like `pd.factorize(values)`. The results are identical for methods like `:meth:`Series.factorize``.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", "b", "a", "c", "b"], dtype="O")
...)
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
...)
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When `use_na_sentinel=True` (the default), missing values are indicated in the `codes` with the sentinel value `-1` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", None, "a", "c", "b"], dtype="O")
...)
>>> codes
array([0, -1, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that `'b'` is in `uniques.categories`, despite not being present in `cat.values`.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting `use_na_sentinel=False`.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([0, 1, 0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([1., 2., nan])
```

`item(self)`  
Return the first element of the underlying data as a Python scalar.

Returns

-----

scalar

The first element of Series or Index.

Raises

-----

ValueError

If the data is not length = 1.

See Also

-----

`Index.values` : Returns an array representing the data in the Index.

`Series.head` : Returns the first `n` rows.

Examples

-----

```
>>> s = pd.Series([1])
```

```
>>> s.item()
```

```
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'
```

nunique(self, dropna: bool = True) -> int  
Return number of unique elements in the object.

Excludes NA values by default.

Parameters

-----  
dropna : bool, default True  
Don't include NaN in the count.

Returns

-----  
int

An integer indicating the number of unique elements in the object.

See Also

-----  
DataFrame.nunique: Method nunique for DataFrame.  
Series.count: Count non-NA/null observations in the Series.

Examples

-----  
>>> s = pd.Series([1, 3, 5, 7, 7])  
>>> s  
0 1  
1 3  
2 5  
3 7  
4 7  
dtype: int64

```
>>> s.nunique()
4
```

to\_list = tolist(self) -> list

to\_numpy(  
 self,  
 dtype: npt.DTypeLike | None = None,  
 copy: bool = False,  
 na\_value: object = <no\_default>,  
 \*\*kwargs  
) -> np.ndarray

A NumPy ndarray representing the values in this Series or Index.

Parameters

-----  
dtype : str or numpy.dtype, optional  
The dtype to pass to :meth:`numpy.asarray`.  
copy : bool, default False  
Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not \*ensure\* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.  
na\_value : Any, optional  
The value to use for missing values. The default value depends on `dtype` and the type of the array.  
\*\*kwargs  
Additional keywords passed through to the ``to\_numpy`` method of the underlying array (for extension arrays).

Returns

-----  
numpy.ndarray

The NumPy ndarray holding the values from this Series or Index.  
The dtype of the array may differ. See Notes.

See Also

-----  
Series.array : Get the actual data stored within.  
Index.array : Get the actual data stored within.

DataFrame.to\_numpy : Similar method for DataFrame.

#### Notes

-----  
The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` \*may\* require copying data and coercing the result to a NumPy type (possibly object), which may be expensive. When you need a no-copy reference to the underlying data, :attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of ``to\_numpy()`` for various dtypes within pandas.

dtype	array type
category[T]	ndarray[T] (same dtype as input)
period	ndarray[object] (Periods)
interval	ndarray[object] (Intervals)
IntegerNA	ndarray[object]
datetime64[ns]	datetime64[ns]
datetime64[ns, tz]	ndarray[object] (Timestamps)

#### Examples

-----  
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))  
>>> ser.to\_numpy()  
array(['a', 'b', 'a'], dtype=object)

Specify the `dtype` to control how datetime-aware data is represented. Use `dtype=object` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct `tz`.

>>> ser = pd.Series(pd.date\_range("2000", periods=2, tz="CET"))  
>>> ser.to\_numpy(dtype=object)  
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),  
 Timestamp('2000-01-02 00:00:00+0100', tz='CET')],  
 dtype=object)

Or `dtype='datetime64[ns]'` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

>>> ser.to\_numpy(dtype="datetime64[ns]")  
... # doctest: +ELLIPSIS  
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],  
 dtype='datetime64[ns]')

tolist(self) -> list  
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

#### Returns

-----  
list  
List containing the values as Python or pandas scalars.

#### See Also

-----  
numpy.ndarray.tolist : Return the array as an a.ndim-levels deep nested list of Python scalars.

## Examples

-----

### For Series

```
>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]
```

### For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

>>> idx.to_list()
[1, 2, 3]
```

transpose(self, \*args, \*\*kwargs) -> Self  
Return the transpose, which is by definition self.

## Returns

-----

%(klass)s

### value\_counts()

```
self,
normalize: bool = False,
sort: bool = True,
ascending: bool = False,
bins=None,
dropna: bool = True
```

) -> Series

Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

## Parameters

-----

normalize : bool, default False

If True then the object returned will contain the relative frequencies of the unique values.

sort : bool, default True

Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False

Sort in ascending order.

bins : int, optional

Rather than count values, group them into half-open bins, a convenience for ``pd.cut``, only works with numeric data.

dropna : bool, default True

Don't include counts of NaN.

## Returns

-----

### Series

Series containing counts of unique values.

## See Also

-----

Series.count: Number of non-NA elements in a Series.

DataFrame.count: Number of non-NA elements in a DataFrame.

DataFrame.value\_counts: Equivalent method on DataFrames.

## Examples

-----

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0 2
1.0 1
2.0 1
4.0 1
```

Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0 0.4
1.0 0.2
2.0 0.2
4.0 0.2
Name: proportion, dtype: float64
```

**\*\*bins\*\***

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0] 2
(2.0, 3.0] 2
(3.0, 4.0] 1
Name: count, dtype: int64
```

**\*\*dropna\*\***

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0 2
1.0 1
2.0 1
4.0 1
NaN 1
Name: count, dtype: int64
```

**\*\*Categorical Dtypes\*\***

Rows with categorical type will be counted as one group if they have same categories and order.  
In the example below, even though `a`, `c`, and `d` all have the same data types of `category`, only `c` and `d` will be counted as one group since `a` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
 a b c d
0 1 2 3 3

>>> df.dtypes
a category
b str
c category
d category
dtype: object

>>> df.dtypes.value_counts()
category 2
category 1
str 1
Name: count, dtype: int64
```

-----  
Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

-----  
Index : Immutable sequence used for indexing and alignment.

Examples

```

For Series:

>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0 Ant
1 Bear
2 Cow
dtype: str
>>> s.T
0 Ant
1 Bear
2 Cow
dtype: str

For Index:

>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')

```

#### empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

#### Returns

```

bool
 If Index is empty, return True, if not return False.

```

#### See Also

`Index.size` : Return the number of elements in the underlying data.

#### Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False

```

```

>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True

```

If we only have NaNs in our DataFrame, it is not considered empty!

```

>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False

```

#### is\_monotonic\_decreasing

Return True if values in the object are monotonically decreasing.

#### Returns

```

bool

```

#### See Also

`Series.is_monotonic_increasing` : Return boolean if values in the object are monotonically increasing.

#### Examples

```

>>> s = pd.Series([3, 2, 2, 1])
>>> s.is_monotonic_decreasing
True

```

```

>>> s = pd.Series([1, 2, 3])
>>> s.is_monotonic_decreasing
False
is_monotonic_increasing
 Return True if values in the object are monotonically increasing.

 Returns

 bool

 See Also

 Series.is_monotonic_decreasing : Return boolean if values in the object are
 monotonically decreasing.

 Examples

 >>> s = pd.Series([1, 2, 2])
 >>> s.is_monotonic_increasing
 True

 >>> s = pd.Series([3, 2, 1])
 >>> s.is_monotonic_increasing
 False
is_unique
 Return True if values in the object are unique.

 Returns

 bool

 See Also

 Series.unique : Return unique values of Series object.
 Series.drop_duplicates : Return Series with duplicate values removed.
 Series.duplicated : Indicate duplicate Series values.

 Examples

 >>> s = pd.Series([1, 2, 3])
 >>> s.is_unique
 True

 >>> s = pd.Series([1, 2, 3, 1])
 >>> s.is_unique
 False
nbytes
 Return the number of bytes in the underlying data.

 See Also

 Series.ndim : Number of dimensions of the underlying data.
 Series.size : Return the number of elements in the underlying data.

 Examples

 For Series:

 >>> s = pd.Series(["Ant", "Bear", "Cow"])
 >>> s
 0 Ant
 1 Bear
 2 Cow
 dtype: str
 >>> s.nbytes
 34

 For Index:

 >>> idx = pd.Index([1, 2, 3])
 >>> idx
 Index([1, 2, 3], dtype='int64')
 >>> idx.nbytes

```



**ndim**

Number of dimensions of the underlying data, by definition 1.

See Also

-----

Series.size: Return the number of elements in the underlying data.  
 Series.shape: Return a tuple of the shape of the underlying data.  
 Series.dtype: Return the dtype object of the underlying data.  
 Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

-----

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0 Ant
1 Bear
2 Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

**shape**

Return a tuple of the shape of the underlying data.

See Also

-----

Series.ndim : Number of dimensions of the underlying data.  
 Series.size : Return the number of elements in the underlying data.  
 Series.nbytes : Return the number of bytes in the underlying data.

Examples

-----

```
>>> s = pd.Series([1, 2, 3])
>>> s.shape
(3,)
```

**size**

Return the number of elements in the underlying data.

See Also

-----

Series.ndim: Number of dimensions of the underlying data, by definition 1.  
 Series.shape: Return a tuple of the shape of the underlying data.  
 Series.dtype: Return the dtype object of the underlying data.  
 Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

-----

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0 Ant
1 Bear
2 Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

-----  
Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

`__array_priority__ = 1000`

-----  
Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`

Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

-----  
`other` : scalar, sequence, Series, dict or DataFrame  
Object to be added to the DataFrame.

Returns

-----  
DataFrame

The result of adding ```other``` to DataFrame.

See Also

-----  
`DataFrame.add` : Add a DataFrame and another object, with option for index-  
or column-oriented addition.

Examples

-----  

```
>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names;  
each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

When `other` is a `:class:`Series``, the index of `other` is aligned with the  
columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
```

	height	weight
elk	3.0	500.5
moose	4.1	800.5

Even when the index of `other` is the same as the index of the DataFrame,  
the `:class:`Series`` will not be reoriented. If index-wise alignment is desired,  
`:meth:`DataFrame.add`` should be used with `axis='index'`.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
```

	elk	height	moose	weight
elk	NaN	NaN	NaN	NaN

```
moose NaN NaN NaN NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
```

```
 height weight
elk 2.0 500.5
moose 4.1 801.5
```

When `other` is a :class:`DataFrame`, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
... {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
...)
```

```
>>> df[["height", "weight"]] + other
```

```
 height weight
deer NaN NaN
elk 1.7 NaN
moose 3.0 NaN
```

```
__and__(self, other)
```

```
__divmod__(self, other)
```

```
__eq__(self, other)
Return self==value.
```

```
__floordiv__(self, other)
```

```
__ge__(self, other)
Return self>=value.
```

```
__gt__(self, other)
Return self>value.
```

```
__le__(self, other)
Return self<=value.
```

```
__lt__(self, other)
Return self<value.
```

```
__mod__(self, other)
```

```
__mul__(self, other)
```

```
__ne__(self, other)
Return self!=value.
```

```
__or__(self, other)
Return self|value.
```

```
__pow__(self, other)
```

```
__radd__(self, other)
```

```
__rand__(self, other)
```

```
__rdivmod__(self, other)
```

```
__rfloordiv__(self, other)
```

```
__rmod__(self, other)
```

```
__rmul__(self, other)
```

```
__ror__(self, other)
Return value|self.
```

```
__rpow__(self, other)
```

```
__rsub__(self, other)
```

```
__rtruediv__(self, other)
```

```
__rxor__(self, other)
```

```
__sub__(self, other)
```

```

__truediv__(self, other)

__xor__(self, other)

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None

Methods inherited from pandas.core.generic.NDFrame:

__abs__(self) -> Self

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs: Any, **kwargs: Any)

__bool__(self) -> NoReturn

__contains__(self, key) -> bool
 True if the key is in the info axis

__copy__(self) -> Self

__deepcopy__(self, memo=None) -> Self
 Parameters

 memo, default None
 Standard signature. Unused

__delitem__(self, key) -> None
 Delete item

__finalize__(self, other, method: str | None = None, **kwargs) -> Self
 Propagate metadata from other to self.

 This is the default implementation. Subclasses may override this method to
 implement their own metadata handling.

 Parameters

 other : the object from which to get the attributes that we are going
 to propagate. If ``other`` has an ``input_objs`` attribute, then
 this attribute must contain an iterable of objects, each with an
 ``attrs`` attribute.
 method : str, optional
 A passed method name providing context on where ``__finalize__``
 was called.

 .. warning::

 The value passed as `method` are not currently considered
 stable across pandas releases.

 Notes

 In case ``other`` has an ``input_objs`` attribute, this method only
 propagates its metadata if each object in ``input_objs`` has the exact
 same metadata as the others.

__getattr__(self, name: str)
 After regular attribute access, try looking up the name
 This allows simpler access to columns for interactive use.

__getstate__(self) -> dict[str, Any]
 Helper for pickle.

__iadd__(self, other) -> Self

```

```

__iand__(self, other) -> Self
__ifloordiv__(self, other) -> Self
__imod__(self, other) -> Self
__imul__(self, other) -> Self
__invert__(self) -> Self
__ior__(self, other) -> Self
__ipow__(self, other) -> Self
__isub__(self, other) -> Self
__itruediv__(self, other) -> Self
__ixor__(self, other) -> Self
__neg__(self) -> Self
__pos__(self) -> Self
__round__(self, decimals: int = 0) -> Self
__setattr__(self, name: str, value) -> None
 After regular attribute access, try setting the name
 This allows simpler access to columns for interactive use.
__setstate__(self, state) -> None
abs(self) -> Self
 Return a Series/DataFrame with absolute numeric value of each element.

 This function only applies to elements that are all numeric.

 Returns

 abs
 Series/DataFrame containing the absolute value of each element.

 See Also

 numpy.absolute : Calculate the absolute value element-wise.

 Notes

 For ``complex`` inputs, ``1.2 + 1j``, the absolute value is
 :math:\sqrt{a^2 + b^2}``.

 Examples

 Absolute numeric values in a Series.

 >>> s = pd.Series([-1.10, 2, -3.33, 4])
 >>> s.abs()
 0 1.10
 1 2.00
 2 3.33
 3 4.00
 dtype: float64

 Absolute numeric values in a Series with complex numbers.

 >>> s = pd.Series([1.2 + 1j])
 >>> s.abs()
 0 1.56205
 dtype: float64

 Absolute numeric values in a Series with a Timedelta element.

 >>> s = pd.Series([pd.Timedelta("1 days")])
 >>> s.abs()
 0 1 days
 dtype: timedelta64[us]

```

Select rows with data closest to certain value using argsort (from StackOverflow <<https://stackoverflow.com/a/17758115>>`\_`).

```
>>> df = pd.DataFrame(
... {"a": [4, 5, 6, 7], "b": [10, 20, 30, 40], "c": [100, 50, -30, -50]}
...)
>>> df
 a b c
0 4 10 100
1 5 20 50
2 6 30 -30
3 7 40 -50
>>> df.loc[(df.c - 43).abs().argsort()]
 a b c
1 5 20 50
0 4 10 100
2 6 30 -30
3 7 40 -50
```

`add_prefix(self, prefix: str, axis: Axis | None = None) -> Self`  
Prefix labels with string `prefix`.

For Series, the row labels are prefixed.  
For DataFrame, the column labels are prefixed.

Parameters

-----  
prefix : str  
    The string to add before each label.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
    Axis to add prefix on  
  
    .. versionadded:: 2.0.0

Returns

-----  
Series or DataFrame  
    New Series or DataFrame with updated labels.

See Also

-----  
Series.add\_suffix: Suffix row labels with string `suffix`.  
DataFrame.add\_suffix: Suffix column labels with string `suffix`.

Examples

-----  
>>> s = pd.Series([1, 2, 3, 4])  
>>> s  
0 1  
1 2  
2 3  
3 4  
dtype: int64  
  
>>> s.add\_prefix("item\_")  
item\_0 1  
item\_1 2  
item\_2 3  
item\_3 4  
dtype: int64  
  
>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})  
>>> df  
 A B  
0 1 3  
1 2 4  
2 3 5  
3 4 6  
  
>>> df.add\_prefix("col\_")  
 col\_A col\_B  
0 1 3  
1 2 4  
2 3 5  
3 4 6

`add_suffix(self, suffix: str, axis: Axis | None = None) -> Self`

Suffix labels with string `suffix`.

For Series, the row labels are suffixed.  
For DataFrame, the column labels are suffixed.

#### Parameters

-----  
suffix : str  
    The string to add after each label.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
    Axis to add suffix on  
  
    .. versionadded:: 2.0.0

#### Returns

-----  
Series or DataFrame  
    New Series or DataFrame with updated labels.

#### See Also

-----  
Series.add\_prefix: Prefix row labels with string `prefix`.  
DataFrame.add\_prefix: Prefix column labels with string `prefix`.

#### Examples

-----  
>>> s = pd.Series([1, 2, 3, 4])  
>>> s  
0 1  
1 2  
2 3  
3 4  
dtype: int64  
  
>>> s.add\_suffix("\_item")  
0\_item 1  
1\_item 2  
2\_item 3  
3\_item 4  
dtype: int64  
  
>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})  
>>> df  
   A B  
0 1 3  
1 2 4  
2 3 5  
3 4 6  
  
>>> df.add\_suffix("\_col")  
   A\_col B\_col  
0 1 3  
1 2 4  
2 3 5  
3 4 6

```
align(
 self,
 other: NDFrameT,
 join: AlignJoin = 'outer',
 axis: Axis | None = None,
 level: Level | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 fill_value: Hashable | None = None
) -> tuple[Self, NDFrameT]
Align two objects on their axes with the specified join method.
```

Join method is specified for each axis Index.

#### Parameters

-----  
other : DataFrame or Series  
    The object to align with.  
join : {'outer', 'inner', 'left', 'right'}, default 'outer'  
    Type of alignment to be performed.  
  
    \* left: use only keys from left frame, preserve key order.

- \* right: use only keys from right frame, preserve key order.
- \* outer: use union of keys from both frames, sort keys lexicographically.
- \* inner: use intersection of keys from both frames, preserve the order of the left keys.

axis : allowed axis of the other object, default None  
Align on index (0), columns (1), or both (None).

level : int or level name, default None  
Broadcast across a level, matching Index values on the passed MultiIndex level.

copy : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` [https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

fill\_value : scalar, default np.nan  
Value to use for missing values. Defaults to NaN, but can be any "compatible" value.

Returns

-----  
tuple of (Series/DataFrame, type of other)  
Aligned objects.

See Also

-----  
Series.align : Align two objects on their axes with specified join method.  
DataFrame.align : Align two objects on their axes with specified join method.

Examples

-----  
>>> df = pd.DataFrame(  
... [[1, 2, 3, 4], [6, 7, 8, 9]], columns=["D", "B", "E", "A"], index=[1, 2]  
... )  
>>> other = pd.DataFrame(  
... [[10, 20, 30, 40], [60, 70, 80, 90], [600, 700, 800, 900]],  
... columns=["A", "B", "C", "D"],  
... index=[2, 3, 4],  
... )  
>>> df  
 D B E A  
1 1 2 3 4  
2 6 7 8 9  
>>> other  
 A B C D  
2 10 20 30 40  
3 60 70 80 90  
4 600 700 800 900

Align on columns:

>>> left, right = df.align(other, join="outer", axis=1)  
>>> left  
 A B C D E  
1 4 2 NaN 1 3  
2 9 7 NaN 6 8  
>>> right  
 A B C D E  
2 10 20 30 40 NaN  
3 60 70 80 90 NaN  
4 600 700 800 900 NaN

We can also align on the index:

>>> left, right = df.align(other, join="outer", axis=0)  
>>> left  
 D B E A  
1 1.0 2.0 3.0 4.0  
2 6.0 7.0 8.0 9.0



```

3 NaN NaN NaN NaN
4 NaN NaN NaN NaN
>>> right
 A B C D
1 NaN NaN NaN NaN
2 10.0 20.0 30.0 40.0
3 60.0 70.0 80.0 90.0
4 600.0 700.0 800.0 900.0

```

Finally, the default `axis=None` will align on both index and columns:

```

>>> left, right = df.align(other, join="outer", axis=None)
>>> left
 A B C D E
1 4.0 2.0 NaN 1.0 3.0
2 9.0 7.0 NaN 6.0 8.0
3 NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN
>>> right
 A B C D E
1 NaN NaN NaN NaN NaN
2 10.0 20.0 30.0 40.0 NaN
3 60.0 70.0 80.0 90.0 NaN
4 600.0 700.0 800.0 900.0 NaN

```

```

asfreq(
 self,
 freq: Frequency,
 method: FillnaOptions | None = None,
 how: Literal['start', 'end'] | None = None,
 normalize: bool = False,
 fill_value: Hashable | None = None
) -> Self

```

Convert time series to specified frequency.

Returns the original data conformed to a new index with the specified frequency.

If the index of this Series/DataFrame is a `:class:`~pandas.PeriodIndex``, the new index is the result of transforming the original index with `:meth:`PeriodIndex.asfreq`` (`<pandas.PeriodIndex.asfreq>`) (so the original index will map one-to-one to the new index).

Otherwise, the new index will be equivalent to ``pd.date_range(start, end, freq=freq)`` where ``start`` and ``end`` are, respectively, the min and max entries in the original index (see `:func:`pandas.date_range``). The values corresponding to any timesteps in the new index which were not present in the original index will be null (``NaN``), unless a method for filling such unknowns is provided (see the ``method`` parameter below).

The `:meth:`resample`` method is more appropriate if an operation on each group of timesteps (such as an aggregate) is necessary to represent the data at the new frequency.

#### Parameters

```

freq : DateOffset or str
 Frequency DateOffset or string.
method : {'backfill'/'bfill', 'pad'/'ffill'}, default None
 Method to use for filling holes in reindexed Series (note this
 does not fill NaNs that already were present):

 * 'pad' / 'ffill': propagate last valid observation forward to next
 valid based on the order of the index
 * 'backfill' / 'bfill': use NEXT valid observation to fill.
how : {'start', 'end'}, default end
 For PeriodIndex only (see PeriodIndex.asfreq).
normalize : bool, default False
 Whether to reset output index to midnight.
fill_value : scalar, optional
 Value to use for missing values, applied during upsampling (note
 this does not fill NaNs that already were present).

```

#### Returns

```

Series/DataFrame
Series/DataFrame object reindexed to the specified frequency.

```

See Also

-----  
reindex : Conform DataFrame to new index with optional filling logic.

Notes

-----  
To learn more about the frequency strings, please see  
:ref:`this link<timeseries.offset\_aliases>`.

Examples

-----  
Start by creating a series with 4 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=4, freq="min")
>>> series = pd.Series([0.0, None, 2.0, 3.0], index=index)
>>> df = pd.DataFrame({"s": series})
>>> df
```

```
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:01:00 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:03:00 3.0
```

Upsample the series into 30 second bins.

```
>>> df.asfreq(freq="30s")
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 NaN
2000-01-01 00:03:00 3.0
```

Upsample again, providing a ``fill value``.

```
>>> df.asfreq(freq="30s", fill_value=9.0)
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 9.0
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 9.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 9.0
2000-01-01 00:03:00 3.0
```

Upsample again, providing a ``method``.

```
>>> df.asfreq(freq="30s", method="bfill")
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 2.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 3.0
2000-01-01 00:03:00 3.0
```

asof(self, where, subset=None)  
Return the last row(s) without any NaNs before `where`.

The last row (for each element in `where`, if list) without any NaN is taken.  
In case of a :class:`~pandas.DataFrame`, the last row without NaN considering only the subset of columns (if not `None`)

If there is no good value, NaN is returned for a Series or a Series of NaN values for a DataFrame

Parameters

-----  
where : date or array-like of dates  
Date(s) before which the last row(s) are returned.  
subset : str or array-like of str, default `None`  
For DataFrame, if not `None`, only use these columns to

check for NaNs.

#### Returns

-----  
scalar, Series, or DataFrame

The return can be:

- \* scalar : when `self` is a Series and `where` is a scalar
- \* Series: when `self` is a Series and `where` is an array-like, or when `self` is a DataFrame and `where` is a scalar
- \* DataFrame : when `self` is a DataFrame and `where` is an array-like

#### See Also

-----  
`merge_asof` : Perform an asof merge. Similar to left join.

#### Notes

-----  
Dates are assumed to be sorted. Raises if this is not the case.

#### Examples

-----  
A Series and a scalar `where`.

```
>>> s = pd.Series([1, 2, np.nan, 4], index=[10, 20, 30, 40])
>>> s
10 1.0
20 2.0
30 NaN
40 4.0
dtype: float64
```

```
>>> s.asof(20)
np.float64(2.0)
```

For a sequence `where`, a Series is returned. The first value is NaN, because the first element of `where` is before the first index value.

```
>>> s.asof([5, 20])
5 NaN
20 2.0
dtype: float64
```

Missing values are not considered. The following is ``2.0``, not NaN, even though NaN is at the index location for ``30``.

```
>>> s.asof(30)
np.float64(2.0)
```

Take all columns into consideration

```
>>> df = pd.DataFrame(
... {
... "a": [10.0, 20.0, 30.0, 40.0, 50.0],
... "b": [None, None, None, None, 500],
... },
... index=pd.DatetimeIndex(
... [
... "2018-02-27 09:01:00",
... "2018-02-27 09:02:00",
... "2018-02-27 09:03:00",
... "2018-02-27 09:04:00",
... "2018-02-27 09:05:00",
...]
...),
...)
>>> df.asof(pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]))
 a b
2018-02-27 09:03:30 NaN NaN
2018-02-27 09:04:30 NaN NaN
```

Take a single column into consideration

```
>>> df.asof(
```

```
... pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]),
... subset=["a"],
...)

```

	a	b
2018-02-27 09:03:30	30.0	NaN
2018-02-27 09:04:30	40.0	NaN

```
astype(
 self,
 dtype,
 copy: bool | lib.NoDefault = <no_default>,
 errors: IgnoreRaise = 'raise'
) -> Self
```

Cast a pandas object to a specified dtype ``dtype``.

This method allows the conversion of the data types of pandas objects, including DataFrames and Series, to the specified dtype. It supports casting entire objects to a single data type or applying different data types to individual columns using a mapping.

#### Parameters

**dtype** : str, data type, Series or Mapping of column name -> data type  
Use a str, numpy.dtype, pandas.ExtensionDtype or Python type to cast entire pandas object to the same type. Alternatively, use a mapping, e.g. {col: dtype, ...}, where col is a column label and dtype is a numpy.dtype or Python type to cast one or more of the DataFrame's columns to column-specific types.

**copy** : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`\_ for more details.

**errors** : {'raise', 'ignore'}, default 'raise'  
Control raising of exceptions on invalid data for provided dtype.

- ``raise`` : allow exceptions to be raised
- ``ignore`` : suppress exceptions. On error return original object.

#### Returns

same type as caller  
The pandas object casted to the specified ``dtype``.

#### See Also

**to\_datetime** : Convert argument to datetime.  
**to\_timedelta** : Convert argument to timedelta.  
**to\_numeric** : Convert argument to a numeric type.  
**numpy.ndarray.astype** : Cast a numpy array to a specified type.

#### Notes

.. versionchanged:: 2.0.0

Using ``astype`` to convert from timezone-naive dtype to timezone-aware dtype will raise an exception.  
Use :meth:`Series.dt.tz\_localize` instead.

#### Examples

Create a DataFrame:

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df.dtypes
col1 int64
col2 int64
dtype: object
```

Cast all columns to int32:

```
>>> df.astype("int32").dtypes
col1 int32
col2 int32
dtype: object
```

Cast col1 to int32 using a dictionary:

```
>>> df.astype({"col1": "int32"}).dtypes
col1 int32
col2 int64
dtype: object
```

Create a series:

```
>>> ser = pd.Series([1, 2], dtype="int32")
>>> ser
0 1
1 2
dtype: int32
>>> ser.astype("int64")
0 1
1 2
dtype: int64
```

Convert to categorical type:

```
>>> ser.astype("category")
0 1
1 2
dtype: category
Categories (2, int32): [1, 2]
```

Convert to ordered categorical type with custom ordering:

```
>>> from pandas.api.types import CategoricalDtype
>>> cat_dtype = CategoricalDtype(categories=[2, 1], ordered=True)
>>> ser.astype(cat_dtype)
0 1
1 2
dtype: category
Categories (2, int64): [2 < 1]
```

Create a series of dates:

```
>>> ser_date = pd.Series(pd.date_range("20200101", periods=3))
>>> ser_date
0 2020-01-01
1 2020-01-02
2 2020-01-03
dtype: datetime64[us]
```

`at_time(self, time, asof: bool = False, axis: Axis | None = None) -> Self`  
Select values at particular time of day (e.g., 9:30AM).

Parameters

-----  
`time` : datetime.time or str  
The values to select.  
`asof` : bool, default False  
This parameter is currently not supported.  
`axis` : {0 or 'index', 1 or 'columns'}, default 0  
For `Series` this parameter is unused and defaults to 0.

Returns

-----  
Series or DataFrame  
The values with the specified time.

Raises

-----  
TypeError  
If the index is not a :class:`DatetimeIndex`

See Also

-----  
between\_time : Select values between particular times of the day.  
first : Select initial periods of time series based on a date offset.  
last : Select final periods of time series based on a date offset.  
DatetimeIndex.indexer\_at\_time : Get just the index locations for  
values at particular time of the day.

#### Examples

```

>>> i = pd.date_range("2018-04-09", periods=4, freq="12h")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts
```

```
 A
2018-04-09 00:00:00 1
2018-04-09 12:00:00 2
2018-04-10 00:00:00 3
2018-04-10 12:00:00 4
```

```
>>> ts.at_time("12:00")
 A
2018-04-09 12:00:00 2
2018-04-10 12:00:00 4
```

```
between_time(
 self,
 start_time,
 end_time,
 inclusive: IntervalClosedType = 'both',
 axis: Axis | None = None
) -> Self
Select values between particular times of the day (e.g., 9:00-9:30 AM).
```

By setting ``start\_time`` to be later than ``end\_time``,  
you can get the times that are *not* between the two times.

#### Parameters

-----  
start\_time : datetime.time or str  
Initial time as a time filter limit.  
end\_time : datetime.time or str  
End time as a time filter limit.  
inclusive : {"both", "neither", "left", "right"}, default "both"  
Include boundaries; whether to set each bound as closed or open.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
Determine range time on index or columns value.  
For `Series` this parameter is unused and defaults to 0.

#### Returns

-----  
Series or DataFrame  
Data from the original object filtered to the specified dates range.

#### Raises

-----  
TypeError  
If the index is not a :class:`DatetimeIndex`

#### See Also

-----  
at\_time : Select values at a particular time of the day.  
first : Select initial periods of time series based on a date offset.  
last : Select final periods of time series based on a date offset.  
DatetimeIndex.indexer\_between\_time : Get just the index locations for  
values between particular times of the day.

#### Examples

```

>>> i = pd.date_range("2018-04-09", periods=4, freq="1D20min")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts
```

```
 A
2018-04-09 00:00:00 1
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3
2018-04-12 01:00:00 4
```

```
>>> ts.between_time("0:15", "0:45")
```

```

 A
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3

```

You get the times that are *\*not\** between two times by setting `start_time` later than `end_time``:

```

>>> ts.between_time("0:45", "0:15")
 A
2018-04-09 00:00:00 1
2018-04-12 01:00:00 4

```

```

bfill(
 self,
 *,
 axis: None | Axis = None,
 inplace: bool = False,
 limit: None | int = None,
 limit_area: Literal['inside', 'outside'] | None = None
) -> Self
 Fill NA/NaN values by using the next valid observation to fill the gap.

```

This method fills missing values in a backward direction along the specified axis, propagating non-null values from later positions to earlier positions containing NaN.

#### Parameters

```

axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame
 Axis along which to fill missing values. For `Series`
 this parameter is unused and defaults to 0.
inplace : bool, default False
 If True, fill in-place. Note: this will modify any
 other views on this object (e.g., a no-copy slice for a column in a
 DataFrame).
limit : int, default None
 If method is specified, this is the maximum number of consecutive
 NaN values to forward/backward fill. In other words, if there is
 a gap with more than this number of consecutive NaNs, it will only
 be partially filled. If method is not specified, this is the
 maximum number of entries along the entire axis where NaNs will be
 filled. Must be greater than 0 if not None.
limit_area : {'None', 'inside', 'outside'}, default None
 If limit is specified, consecutive NaNs will be filled with this
 restriction.

 * `None`: No fill restriction.
 * 'inside': Only fill NaNs surrounded by valid values
 (interpolate).
 * 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

```

#### Returns

```

Series/DataFrame
 Object with missing values filled.

```

#### See Also

```

DataFrame.ffill : Fill NA/NaN values by propagating the last valid
 observation to next valid.

```

#### Examples

```

For Series:

```

```

>>> s = pd.Series([1, None, None, 2])
>>> s.bfill()
0 1.0
1 2.0
2 2.0
3 2.0
dtype: float64
>>> s.bfill(limit=1)
0 1.0
1 NaN

```

```

2 2.0
3 2.0
dtype: float64

```

With DataFrame:

```
>>> df = pd.DataFrame({"A": [1, None, None, 4], "B": [None, 5, None, 7]})
```

```
>>> df
```

```

 A B
0 1.0 NaN
1 NaN 5.0
2 NaN NaN
3 4.0 7.0

```

```
>>> df.bfill()
```

```

 A B
0 1.0 5.0
1 4.0 5.0
2 4.0 7.0
3 4.0 7.0

```

```
>>> df.bfill(limit=1)
```

```

 A B
0 1.0 5.0
1 NaN 5.0
2 4.0 7.0
3 4.0 7.0

```

```

clip(
 self,
 lower=None,
 upper=None,
 *,
 axis: Axis | None = None,
 inplace: bool = False,
 **kwargs
) -> Self
 Trim values at input threshold(s).

```

Assigns values outside boundary to boundary values. Thresholds can be singular values or array like, and in the latter case the clipping is performed element-wise in the specified axis.

Parameters

```

lower : float or array-like, default None
 Minimum threshold value. All values below this
 threshold will be set to it. A missing
 threshold (e.g. `NA`) will not clip the value.
upper : float or array-like, default None
 Maximum threshold value. All values above this
 threshold will be set to it. A missing
 threshold (e.g. `NA`) will not clip the value.
axis : {{0 or 'index', 1 or 'columns', None}}, default None
 Align object with lower and upper along the given axis.
 For `Series` this parameter is unused and defaults to `None`.
inplace : bool, default False
 Whether to perform the operation in place on the data.
**kwargs
 Additional keywords have no effect but might be accepted
 for compatibility with numpy.

```

Returns

```

Series or DataFrame
 Same type as calling object with the values outside the
 clip boundaries replaced.

```

See Also

```

Series.clip : Trim values at input threshold in series.
DataFrame.clip : Trim values at input threshold in DataFrame.
numpy.clip : Clip (limit) the values in an array.

```

Examples

```

>>> data = {"col_0": [9, -3, 0, -1, 5], "col_1": [-2, -7, 6, 8, -5]}
>>> df = pd.DataFrame(data)
>>> df

```



	col_0	col_1
0	9	-2
1	-3	-7
2	0	6
3	-1	8
4	5	-5

Clips per column using lower and upper thresholds:

```
>>> df.clip(-4, 6)
 col_0 col_1
0 6 -2
1 -3 -4
2 0 6
3 -1 6
4 5 -4
```

Clips using specific lower and upper thresholds per column:

```
>>> df.clip([-2, -1], [4, 5])
 col_0 col_1
0 4 -1
1 -2 -1
2 0 5
3 -1 5
4 4 -1
```

Clips using specific lower and upper thresholds per column element:

```
>>> t = pd.Series([2, -4, -1, 6, 3])
>>> t
0 2
1 -4
2 -1
3 6
4 3
dtype: int64
```

```
>>> df.clip(t, t + 4, axis=0)
 col_0 col_1
0 6 2
1 -3 -4
2 0 3
3 6 8
4 5 3
```

Clips using specific lower threshold per column element, with missing values:

```
>>> t = pd.Series([2, -4, np.nan, 6, 3])
>>> t
0 2.0
1 -4.0
2 NaN
3 6.0
4 3.0
dtype: float64
```

```
>>> df.clip(t, axis=0)
 col_0 col_1
0 9.0 2.0
1 -3.0 -4.0
2 0.0 6.0
3 6.0 8.0
4 5.0 3.0
```

```
convert_dtypes(
 self,
 infer_objects: bool = True,
 convert_string: bool = True,
 convert_integer: bool = True,
 convert_boolean: bool = True,
 convert_floating: bool = True,
 dtype_backend: DtypeBackend = 'numpy_nullable'
) -> Self
 Convert columns from numpy dtypes to the best dtypes that support ``pd.NA``.

Parameters
```

```

infer_objects : bool, default True
 Whether object dtypes should be converted to the best possible types.
convert_string : bool, default True
 Whether object dtypes should be converted to ``StringDtype()``.
convert_integer : bool, default True
 Whether, if possible, conversion can be done to integer extension types.
convert_boolean : bool, defaults True
 Whether object dtypes should be converted to ``BooleanDtypes()``.
convert_floating : bool, defaults True
 Whether, if possible, conversion can be done to floating extension types.
 If ``convert_integer`` is also True, preference will be give to integer
 dtypes if the floats can be faithfully casted to integers.
dtype_backend : {'numpy_nullable', 'pyarrow'}, default 'numpy_nullable'
 Back-end data type applied to the resultant :class:`DataFrame` or
 :class:`Series` (still experimental). Behaviour is as follows:

 * ``"numpy_nullable"``: returns nullable-dtype-backed
 :class:`DataFrame` or :class:`Series`.
 * ``"pyarrow"``: returns pyarrow-backed nullable :class:`ArrowDtype`
 :class:`DataFrame` or :class:`Series`.

.. versionadded:: 2.0

```

Returns

```

Series or DataFrame
 Copy of input object with new dtype.

```

See Also

```

infer_objects : Infer dtypes of objects.
to_datetime : Convert argument to datetime.
to_timedelta : Convert argument to timedelta.
to_numeric : Convert argument to a numeric type.

```

Notes

```

By default, ``convert_dtypes`` will attempt to convert a Series (or each
Series in a DataFrame) to dtypes that support ``pd.NA``. By using the options
``convert_string``, ``convert_integer``, ``convert_boolean`` and
``convert_floating``, it is possible to turn off individual conversions
to ``StringDtype``, the integer extension types, ``BooleanDtype``
or floating extension types, respectively.

```

For object-dtyped columns, if ``infer\_objects`` is ``True``, use the inference rules as during normal Series/DataFrame construction. Then, if possible, convert to ``StringDtype``, ``BooleanDtype`` or an appropriate integer or floating extension type, otherwise leave as ``object``.

If the dtype is integer, convert to an appropriate integer extension type.

If the dtype is numeric, and consists of all integers, convert to an appropriate integer extension type. Otherwise, convert to an appropriate floating extension type.

In the future, as new dtypes are added that support ``pd.NA``, the results of this method will change to support those new dtypes.

Examples

```

>>> df = pd.DataFrame(
... {
... "a": pd.Series([1, 2, 3], dtype=np.dtype("int32")),
... "b": pd.Series(["x", "y", "z"], dtype=np.dtype("O")),
... "c": pd.Series([True, False, np.nan], dtype=np.dtype("O")),
... "d": pd.Series(["h", "i", np.nan], dtype=np.dtype("O")),
... "e": pd.Series([10, np.nan, 20], dtype=np.dtype("float")),
... "f": pd.Series([np.nan, 100.5, 200], dtype=np.dtype("float")),
... }
...)

```

Start with a DataFrame with default dtypes.

```

>>> df
 a b c d e f
0 1 x True h 10.0 NaN

```

```
1 2 y False i NaN 100.5
2 3 z NaN NaN 20.0 200.0
```

```
>>> df.dtypes
a int32
b object
c object
d object
e float64
f float64
dtype: object
```

Convert the DataFrame to use best possible dtypes.

```
>>> dfn = df.convert_dtypes()
>>> dfn
 a b c d e f
0 1 x True h 10 <NA>
1 2 y False i <NA> 100.5
2 3 z <NA> <NA> 20 200.0
```

```
>>> dfn.dtypes
a Int32
b string
c boolean
d string
e Int64
f Float64
dtype: object
```

Start with a Series of strings and missing data represented by ``np.nan``.

```
>>> s = pd.Series(["a", "b", np.nan])
>>> s
0 a
1 b
2 NaN
dtype: str
```

Obtain a Series with dtype ``StringDtype``.

```
>>> s.convert_dtypes()
0 a
1 b
2 <NA>
dtype: string
```

`copy(self, deep: bool = True) -> Self`  
Make a copy of this object's indices and data.

When ``deep=True`` (default), a new object will be created with a copy of the calling object's data and indices. Modifications to the data or indices of the copy will not be reflected in the original object (see notes below).

When ``deep=False``, a new object will be created without copying the calling object's data or index (only references to the data and index are copied). With Copy-on-Write, changes to the original will *not* be reflected in the shallow copy (and vice versa). The shallow copy uses a lazy (deferred) copy mechanism that copies the data only when any changes to the original or shallow copy are made, ensuring memory efficiency while maintaining data integrity.

.. note::  
In pandas versions prior to 3.0, the default behavior without Copy-on-Write was different: changes to the original *were* reflected in the shallow copy (and vice versa). See the :ref:`Copy-on-Write user guide <copy\_on\_write>` for more information.

Parameters

`deep` : bool, default True  
Make a deep copy, including a copy of the data and the indices.  
With ``deep=False`` neither the indices nor the data are copied.

Returns

-----

Series or DataFrame  
Object type matches caller.

See Also

-----  
copy.copy : Return a shallow copy of an object.  
copy.deepcopy : Return a deep copy of an object.

Notes

-----  
When ``deep=True``, data is copied but actual Python objects will not be copied recursively, only the reference to the object. This is in contrast to `copy.deepcopy` in the Standard Library, which recursively copies object data (see examples below).

While ``Index`` objects are copied when ``deep=True``, the underlying numpy array is not copied for performance reasons. Since ``Index`` is immutable, the underlying data can be safely shared and a copy is not needed.

Since pandas is not thread safe, see the :ref:`gotchas <gotchas.thread-safety>` when copying in a threading environment.

Copy-on-Write protects shallow copies against accidental modifications. This means that any changes to the copied data would make a new copy of the data upon write (and vice versa). Changes made to either the original or copied variable would not be reflected in the counterpart. See :ref:`Copy\_on\_Write <copy\_on\_write>` for more information.

Examples

-----  
>>> s = pd.Series([1, 2], index=["a", "b"])  
>>> s  
a 1  
b 2  
dtype: int64

>>> s\_copy = s.copy(deep=True)  
>>> s\_copy  
a 1  
b 2  
dtype: int64

Due to Copy-on-Write, shallow copies still protect data modifications. Note shallow does not get modified below.

>>> s = pd.Series([1, 2], index=["a", "b"])  
>>> shallow = s.copy(deep=False)  
>>> s.iloc[1] = 200  
>>> shallow  
a 1  
b 2  
dtype: int64

When the data has object dtype, even a deep copy does not copy the underlying Python objects. Updating a nested data object will be reflected in the deep copy.

>>> s = pd.Series([[1, 2], [3, 4]])  
>>> deep = s.copy()  
>>> s[0][0] = 10  
>>> s  
0 [10, 2]  
1 [3, 4]  
dtype: object  
>>> deep  
0 [10, 2]  
1 [3, 4]  
dtype: object

describe(self, percentiles=None, include=None, exclude=None) -> Self  
Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding ``NaN`` values.

Analyzes both numeric and object series, as well as ``DataFrame`` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

#### Parameters

`percentiles` : list-like of numbers, optional  
The percentiles to include in the output. All should fall between 0 and 1. The default, ``None``, will automatically return the 25th, 50th, and 75th percentiles.

`include` : 'all', list-like of dtypes or None (default), optional  
A white list of data types to include in the result. Ignored for ``Series``. Here are the options:

- 'all' : All columns of the input will be included in the output.
- A list-like of dtypes : Limits the results to the provided data types.  
To limit the result to numeric types submit ``numpy.number``. To limit it instead to object columns submit the ``numpy.object`` data type. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(include=['O'])``). To select pandas categorical columns, use ``category``.
- None (default) : The result will include all numeric columns.

`exclude` : list-like of dtypes or None (default), optional  
A black list of data types to omit from the result. Ignored for ``Series``. Here are the options:

- A list-like of dtypes : Excludes the provided data types from the result. To exclude numeric types submit ``numpy.number``. To exclude object columns submit the data type ``numpy.object``. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(exclude=['O'])``). To exclude pandas categorical columns, use ``category``.
- None (default) : The result will exclude nothing.

#### Returns

##### Series or DataFrame

Summary statistics of the Series or DataFrame provided.

#### See Also

`DataFrame.count`: Count number of non-NA/null observations.  
`DataFrame.max`: Maximum of the values in the object.  
`DataFrame.min`: Minimum of the values in the object.  
`DataFrame.mean`: Mean of the values.  
`DataFrame.std`: Standard deviation of the observations.  
`DataFrame.select_dtypes`: Subset of a DataFrame including/excluding columns based on their dtype.

#### Notes

For numeric data, the result's index will include ``count``, ``mean``, ``std``, ``min``, ``max`` as well as lower, ``50`` and upper percentiles. By default the lower percentile is ``25`` and the upper percentile is ``75``. The ``50`` percentile is the same as the median.

For object data (e.g. strings), the result's index will include ``count``, ``unique``, ``top``, and ``freq``. The ``top`` is the most common value. The ``freq`` is the most common value's frequency.

If multiple object values have the highest count, then the ``count`` and ``top`` results will be arbitrarily chosen from among those with the highest count.

For mixed data types provided via a ``DataFrame``, the default is to return only an analysis of numeric columns. If the DataFrame consists only of object and categorical data without any numeric columns, the default is to return an analysis of both the object and categorical columns. If ``include='all'`` is provided as an option, the result will include a union of attributes of each type.

The `include` and `exclude` parameters can be used to limit which columns in a `DataFrame` are analyzed for the output. The parameters are ignored when analyzing a `Series`.

#### Examples

Describing a numeric `Series`.

```
>>> s = pd.Series([1, 2, 3])
>>> s.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
dtype: float64
```

Describing a categorical `Series`.

```
>>> s = pd.Series(["a", "a", "b", "c"])
>>> s.describe()
count 4
unique 3
top a
freq 2
dtype: object
```

Describing a timestamp `Series`.

```
>>> s = pd.Series(
... [
... np.datetime64("2000-01-01"),
... np.datetime64("2010-01-01"),
... np.datetime64("2010-01-01"),
...]
...)
>>> s.describe()
count 3
mean 2006-09-01 08:00:00
min 2000-01-01 00:00:00
25% 2004-12-31 12:00:00
50% 2010-01-01 00:00:00
75% 2010-01-01 00:00:00
max 2010-01-01 00:00:00
dtype: object
```

Describing a `DataFrame`. By default only numeric fields are returned.

```
>>> df = pd.DataFrame(
... {
... "categorical": pd.Categorical(["d", "e", "f"]),
... "numeric": [1, 2, 3],
... "object": ["a", "b", "c"],
... }
...)
>>> df.describe()
numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Describing all columns of a `DataFrame` regardless of data type.

```
>>> df.describe(include="all") # doctest: +SKIP
categorical numeric object
count 3 3.0 3
unique 3 NaN 3
top f NaN a
```

freq	1	NaN	1
mean	NaN	2.0	NaN
std	NaN	1.0	NaN
min	NaN	1.0	NaN
25%	NaN	1.5	NaN
50%	NaN	2.0	NaN
75%	NaN	2.5	NaN
max	NaN	3.0	NaN

Describing a column from a ``DataFrame`` by accessing it as an attribute.

```
>>> df.numeric.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
Name: numeric, dtype: float64
```

Including only numeric columns in a ``DataFrame`` description.

```
>>> df.describe(include=[np.number])
numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Including only string columns in a ``DataFrame`` description.

```
>>> df.describe(include=[object]) # doctest: +SKIP
object
count 3
unique 3
top a
freq 1
```

Including only categorical columns from a ``DataFrame`` description.

```
>>> df.describe(include=["category"])
category
count 3
unique 3
top d
freq 1
```

Excluding numeric columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[np.number]) # doctest: +SKIP
category object
count 3 3
unique 3 3
top f a
freq 1 1
```

Excluding object columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[object]) # doctest: +SKIP
category numeric
count 3 3.0
unique 3 NaN
top f NaN
freq 1 NaN
mean NaN 2.0
std NaN 1.0
min NaN 1.0
25% NaN 1.5
50% NaN 2.0
75% NaN 2.5
```

max NaN 3.0

`droplevel(self, level: IndexLabel, axis: Axis = 0) -> Self`  
Return Series/DataFrame with requested index / column level(s) removed.

#### Parameters

`level` : int, str, or list-like  
If a string is given, must be the name of a level  
If list-like, elements must be names or positional indexes of levels.

`axis` : {{0 or 'index', 1 or 'columns'}}, default 0  
Axis along which the level(s) is removed:

- \* 0 or 'index': remove level(s) in column.
- \* 1 or 'columns': remove level(s) in row.

For `Series` this parameter is unused and defaults to 0.

#### Returns

Series/DataFrame  
Series/DataFrame with requested index / column level(s) removed.

#### See Also

`DataFrame.replace` : Replace values given in `to\_replace` with `value`.  
`DataFrame.pivot` : Return reshaped DataFrame organized by given index / column values.

#### Examples

```
>>> df = (
... pd.DataFrame([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
... .set_index([0, 1])
... .rename_axis(["a", "b"])
...)
```

```
>>> df.columns = pd.MultiIndex.from_tuples(
... [("c", "e"), ("d", "f")], names=["level_1", "level_2"]
...)
```

```
>>> df
level_1 c d
level_2 e f
a b
1 2 3 4
5 6 7 8
9 10 11 12
```

```
>>> df.droplevel("a")
level_1 c d
level_2 e f
b
2 3 4
6 7 8
10 11 12
```

```
>>> df.droplevel("level_2", axis=1)
level_1 c d
a b
1 2 3 4
5 6 7 8
9 10 11 12
```

`equals(self, other: object) -> bool`  
Test whether two objects contain the same elements.

This function allows two Series or DataFrames to be compared against each other to see if they have the same shape and elements. NaNs in the same location are considered equal.

The row/column index do not need to have the same type, as long as the values are considered equal. Corresponding columns and index must be of the same dtype.



## Parameters

-----

other : Series or DataFrame

The other Series or DataFrame to be compared with the first.

## Returns

-----

bool

True if all elements are the same in both objects, False otherwise.

## See Also

-----

Series.eq : Compare two Series objects of the same length and return a Series where each element is True if the element in each Series is equal, False otherwise.

DataFrame.eq : Compare two DataFrame objects of the same shape and return a DataFrame where each element is True if the respective element in each DataFrame is equal, False otherwise.

testing.assert\_series\_equal : Raises an AssertionError if left and right are not equal. Provides an easy interface to ignore inequality in dtypes, indexes and precision among others.

testing.assert\_frame\_equal : Like assert\_series\_equal, but targets DataFrames.

numpy.array\_equal : Return True if two arrays have the same shape and elements, False otherwise.

## Examples

-----

```
>>> df = pd.DataFrame({1: [10], 2: [20]})
```

```
>>> df
 1 2
0 10 20
```

DataFrames df and exactly\_equal have the same types and values for their elements and column labels, which will return True.

```
>>> exactly_equal = pd.DataFrame({1: [10], 2: [20]})
```

```
>>> exactly_equal
```

```
 1 2
0 10 20
```

```
>>> df.equals(exactly_equal)
```

```
True
```

DataFrames df and different\_column\_type have the same element types and values, but have different types for the column labels, which will still return True.

```
>>> different_column_type = pd.DataFrame({1.0: [10], 2.0: [20]})
```

```
>>> different_column_type
```

```
 1.0 2.0
0 10 20
```

```
>>> df.equals(different_column_type)
```

```
True
```

DataFrames df and different\_data\_type have different types for the same values for their elements, and will return False even though their column labels are the same values and types.

```
>>> different_data_type = pd.DataFrame({1: [10.0], 2: [20.0]})
```

```
>>> different_data_type
```

```
 1 2
0 10.0 20.0
```

```
>>> df.equals(different_data_type)
```

```
False
```

DataFrames with NaN in the same locations compare equal.

```
>>> df_nan1 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
```

```
>>> df_nan2 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
```

```
>>> df_nan1.equals(df_nan2)
```

```
True
```

If the NaN values are not in the same locations, they compare unequal.

```
>>> df_nan3 = pd.DataFrame({"a": [1, np.nan], "b": [3, 4]})
```

```
>>> df_nan1.equals(df_nan3)
```

```

False

ewm(
 self,
 com: float | None = None,
 span: float | None = None,
 halflife: float | TimedeltaConvertibleTypes | None = None,
 alpha: float | None = None,
 min_periods: int | None = 0,
 adjust: bool = True,
 ignore_na: bool = False,
 times: np.ndarray | DataFrame | Series | None = None,
 method: Literal['single', 'table'] = 'single'
) -> ExponentialMovingWindow
 Provide exponentially weighted (EW) calculations.

 Exactly one of ``com``, ``span``, ``halflife``, or ``alpha`` must be
 provided if ``times`` is not provided. If ``times`` is provided and ``adjust=True``,
 ``halflife`` and one of ``com``, ``span`` or ``alpha`` may be provided.
 If ``times`` is provided and ``adjust=False``, ``halflife`` must be the only
 provided decay-specification parameter.

 Parameters

 com : float, optional
 Specify decay in terms of center of mass

 :math:\alpha = 1 / (1 + com), for :math:com \geq 0.

 span : float, optional
 Specify decay in terms of span

 :math:\alpha = 2 / (span + 1), for :math:span \geq 1.

 halflife : float, str, timedelta, optional
 Specify decay in terms of half-life

 :math:\alpha = 1 - \exp(-\ln(2) / halflife), for
 :math:halflife > 0.

 If ``times`` is specified, a timedelta convertible unit over which an
 observation decays to half its value. Only applicable to ``mean()``
 and halflife value will not apply to the other functions.

 alpha : float, optional
 Specify smoothing factor :math:\alpha directly

 :math:0 < \alpha \leq 1.

 min_periods : int, default 0
 Minimum number of observations in window required to have a value;
 otherwise, result is ``np.nan``.

 adjust : bool, default True
 Divide by decaying adjustment factor in beginning periods to account
 for imbalance in relative weightings (viewing EWMA as a moving average).

 - When ``adjust=True`` (default), the EW function is calculated using weights
 :math:w_i = (1 - \alpha)^i. For example, the EW moving average of the series
 [:math:x_0, x_1, \dots, x_t] would be:

 .. math::
 y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots + (1 - \alpha)^t x_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}

 - When ``adjust=False``, the exponentially weighted function is calculated
 recursively:

 .. math::
 \begin{split}
 y_0 &= x_0 \\
 y_t &= (1 - \alpha) y_{t-1} + \alpha x_t,
 \end{split}

 ignore_na : bool, default False
 Ignore missing values when calculating weights.

 - When ``ignore_na=False`` (default), weights are based on absolute positions.

```

For example, the weights of  $x_0$  and  $x_2$  used in calculating the final weighted average of  $[x_0, \text{None}, x_2]$  are  $(1-\alpha)^2$  and  $1$  if `adjust=True`, and  $(1-\alpha)^2$  and  $\alpha$  if `adjust=False`.

- When `ignore_na=True`, weights are based on relative positions. For example, the weights of  $x_0$  and  $x_2$  used in calculating the final weighted average of  $[x_0, \text{None}, x_2]$  are  $1-\alpha$  and  $1$  if `adjust=True`, and  $1-\alpha$  and  $\alpha$  if `adjust=False`.

`times` : np.ndarray, Series, default None

Only applicable to `mean()`.

Times corresponding to the observations. Must be monotonically increasing and `datetime64[ns]` dtype.

If 1-D array like, a sequence with the same shape as the observations.

`method` : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row ('single') or over the entire object ('table').

This argument is only implemented when specifying `engine='numba'` in the method call.

Only applicable to `mean()`

Returns

-----  
pandas.api.typing.ExponentialMovingWindow

An instance of ExponentialMovingWindow for further exponentially weighted (EW) calculations, e.g. using the `mean()` method.

See Also

-----  
`rolling` : Provides rolling window calculations.  
`expanding` : Provides expanding transformations.

Notes

-----  
See :ref:`Windowing Operations <window.exponentially\_weighted>` for further usage details and examples.

Examples

-----  
>>> df = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]})

```
>>> df
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

>>> df.ewm(com=0.5).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
```

>>> df.ewm(alpha=2 / 3).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
```

**\*\*adjust\*\***

>>> df.ewm(com=0.5, adjust=True).mean()

```
 B
0 0.000000
1 0.750000
```

```

2 1.615385
3 1.615385
4 3.670213
>>> df.ewm(com=0.5, adjust=False).mean()

```

```

 B
0 0.000000
1 0.666667
2 1.555556
3 1.555556
4 3.650794

```

**\*\*ignore\_na\*\***

```

>>> df.ewm(com=0.5, ignore_na=True).mean()

```

```

 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.225000

```

```

>>> df.ewm(com=0.5, ignore_na=False).mean()

```

```

 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213

```

**\*\*times\*\***

Exponentially weighted mean with weights calculated with a `timedelta` `halflife` relative to `times`.

```

>>> times = ['2020-01-01', '2020-01-03', '2020-01-10', '2020-01-15', '2020-01-17']
>>> df.ewm(halflife='4 days', times=pd.DatetimeIndex(times)).mean()

```

```

 B
0 0.000000
1 0.585786
2 1.523889
3 1.523889
4 3.233686

```

```

expanding(
 self,
 min_periods: int = 1,
 method: Literal['single', 'table'] = 'single'
) -> Expanding
 Provide expanding window calculations.

```

An expanding window yields the value of an aggregation statistic with all the data available up to that point in time.

Parameters

-----

`min_periods` : int, default 1

Minimum number of observations in window required to have a value; otherwise, result is `np.nan`.

`method` : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row (`'single'`) or over the entire object (`'table'`).

This argument is only implemented when specifying `engine='numba'` in the method call.

Returns

-----

`pandas.api.typing.Expanding`

An instance of `Expanding` for further expanding window calculations, e.g. using the `sum` method.

See Also

-----

`rolling` : Provides rolling window calculations.

`ewm` : Provides exponential weighted functions.

Notes

-----  
See :ref:`Windowing Operations <window.expanding>` for further usage details and examples.

#### Examples

-----  
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})

```
>>> df
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

**\*\*min\_periods\*\***

Expanding sum with 1 vs 3 observations needed to calculate a value.

>>> df.expanding(1).sum()

```
 B
0 0.0
1 1.0
2 3.0
3 3.0
4 7.0
```

>>> df.expanding(3).sum()

```
 B
0 NaN
1 NaN
2 3.0
3 3.0
4 7.0
```

```
ffill(
 self,
 *,
 axis: None | Axis = None,
 inplace: bool = False,
 limit: None | int = None,
 limit_area: Literal['inside', 'outside'] | None = None
) -> Self
 Fill NA/Nan values by propagating the last valid observation to next valid.
```

#### Parameters

-----  
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.

inplace : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

limit : int, default None  
If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

limit\_area : {{`None`, 'inside', 'outside'}}, default None  
If limit is specified, consecutive NaNs will be filled with this restriction.

- \* ``None``: No fill restriction.
- \* 'inside': Only fill NaNs surrounded by valid values (interpolate).
- \* 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

#### Returns

-----  
Series/DataFrame  
Object with missing values filled.

See Also

-----  
DataFrame.bfill : Fill NA/NaN values by using the next valid observation to fill the gap.

#### Examples

-----  
>>> df = pd.DataFrame(  
... [  
... [np.nan, 2, np.nan, 0],  
... [3, 4, np.nan, 1],  
... [np.nan, np.nan, np.nan, np.nan],  
... [np.nan, 3, np.nan, 4],  
... ],  
... columns=list("ABCD"),  
... )

>>> df  
 A B C D  
0 NaN 2.0 NaN 0.0  
1 3.0 4.0 NaN 1.0  
2 NaN NaN NaN NaN  
3 NaN 3.0 NaN 4.0

>>> df.ffill()  
 A B C D  
0 NaN 2.0 NaN 0.0  
1 3.0 4.0 NaN 1.0  
2 3.0 4.0 NaN 1.0  
3 3.0 3.0 NaN 4.0

>>> ser = pd.Series([1, np.nan, 2, 3])  
>>> ser.ffill()  
0 1.0  
1 1.0  
2 2.0  
3 3.0  
dtype: float64

fillna(  
 self,  
 value: Hashable | Mapping | Series | DataFrame,  
 \*,  
 axis: Axis | None = None,  
 inplace: bool = False,  
 limit: int | None = None  
) -> Self  
Fill NA/NaN values with `value`.

#### Parameters

-----  
value : scalar, dict, Series, or DataFrame  
Value to use to fill holes (e.g. 0), alternately a dict/Series/DataFrame of values specifying which value to use for each index (for a Series) or column (for a DataFrame). Values not in the dict/Series/DataFrame will not be filled. This value cannot be a list.  
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.  
inplace : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).  
limit : int, default None  
This is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

#### Returns

-----  
Series/DataFrame  
Object with missing values filled.

#### See Also

-----  
ffill : Fill values by propagating the last valid observation to next valid.  
bfill : Fill values by using the next valid observation to fill the gap.  
interpolate : Fill NaN values using interpolation.  
reindex : Conform object to new index.

asfreq : Convert TimeSeries to specified frequency.

#### Notes

For non-object dtype, ``value=None`` will use the NA value of the dtype. See more details in the :ref:`Filling missing data<missing\_data.fillna>` section.

#### Examples

```
>>> df = pd.DataFrame(
... [
... [np.nan, 2, np.nan, 0],
... [3, 4, np.nan, 1],
... [np.nan, np.nan, np.nan, np.nan],
... [np.nan, 3, np.nan, 4],
...],
... columns=list("ABCD"),
...)
>>> df
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

Replace all NaN elements with 0s.

```
>>> df.fillna(0)
```

	A	B	C	D
0	0.0	2.0	0.0	0.0
1	3.0	4.0	0.0	1.0
2	0.0	0.0	0.0	0.0
3	0.0	3.0	0.0	4.0

Replace all NaN elements in column 'A', 'B', 'C', and 'D', with 0, 1, 2, and 3 respectively.

```
>>> values = {"A": 0, "B": 1, "C": 2, "D": 3}
>>> df.fillna(value=values)
```

	A	B	C	D
0	0.0	2.0	2.0	0.0
1	3.0	4.0	2.0	1.0
2	0.0	1.0	2.0	3.0
3	0.0	3.0	2.0	4.0

Only replace the first NaN element.

```
>>> df.fillna(value=values, limit=1)
```

	A	B	C	D
0	0.0	2.0	2.0	0.0
1	3.0	4.0	NaN	1.0
2	NaN	1.0	NaN	3.0
3	NaN	3.0	NaN	4.0

When filling using a DataFrame, replacement happens along the same column names and same indices

```
>>> df2 = pd.DataFrame(np.zeros((4, 4)), columns=list("ABCE"))
>>> df.fillna(df2)
```

	A	B	C	D
0	0.0	2.0	0.0	0.0
1	3.0	4.0	0.0	1.0
2	0.0	0.0	0.0	NaN
3	0.0	3.0	0.0	4.0

Note that column D is not affected since it is not present in df2.

```
filter(
 self,
 items=None,
 like: str | None = None,
 regex: str | None = None,
 axis: Axis | None = None
) -> Self
 Subset the DataFrame or Series according to the specified index labels.
```

For DataFrame, filter rows or columns depending on ``axis`` argument.  
Note that this routine does not filter based on content.  
The filter is applied to the labels of the index.

#### Parameters

-----  
items : list-like  
    Keep labels from axis which are in items.  
like : str  
    Keep labels from axis for which "like in label == True".  
regex : str (regular expression)  
    Keep labels from axis for which re.search(regex, label) == True.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
    The axis to filter on, expressed either as an index (int)  
    or axis name (str). By default this is the info axis, 'columns' for  
    'DataFrame'. For 'Series' this parameter is unused and defaults to  
    'None'.

#### Returns

-----  
Same type as caller  
    The filtered subset of the DataFrame or Series.

#### See Also

-----  
DataFrame.loc : Access a group of rows and columns  
    by label(s) or a boolean array.

#### Notes

-----  
The ``items``, ``like``, and ``regex`` parameters are  
enforced to be mutually exclusive.

``axis`` defaults to the info axis that is used when indexing  
with ``[]``.

#### Examples

-----  
>>> df = pd.DataFrame(  
... np.array([[1, 2, 3], [4, 5, 6]]),  
... index=["mouse", "rabbit"],  
... columns=["one", "two", "three"],  
... )  
>>> df  
          one two three  
mouse      1  2   3  
rabbit      4  5   6  
  
>>> # select columns by name  
>>> df.filter(items=["one", "three"])  
          one three  
mouse      1   3  
rabbit      4   6  
  
>>> # select columns by regular expression  
>>> df.filter(regex="e\$", axis=1)  
          one three  
mouse      1   3  
rabbit      4   6  
  
>>> # select rows containing 'bbi'  
>>> df.filter(like="bbi", axis=0)  
          one two three  
rabbit      4  5   6

first\_valid\_index(self) -> Hashable  
    Return index for first non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information  
on which values are considered missing.

#### Returns

-----  
type of index  
    Index of first non-missing value.

#### See Also



-----  
 DataFrame.last\_valid\_index : Return index for last non-NA value or None, if no non-NA value is found.  
 Series.last\_valid\_index : Return index for last non-NA value or None, if no non-NA value is found.  
 DataFrame.isna : Detect missing values.

#### Examples

-----  
 For Series:

```
>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
1
>>> s.last_valid_index()
2

>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None
```

If all elements in Series are NA/null, returns None.

```
>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None
```

If Series is empty, returns None.

For DataFrame:

```
>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
 A B
0 NaN NaN
1 NaN 3.0
2 2.0 4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2
```

```
>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
 A B
0 None None
1 None None
2 None None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If DataFrame is empty, returns None.

get(self, key, default=None)  
 Get item from object for given key (ex: DataFrame column).

Returns ``default`` value if not found.

Parameters

```

key : object
 Key for which item should be returned.
default : object, default None
 Default value to return if key is not found.

Returns

same type as items contained in object
 Item for given key or ``default`` value, if key is not found.

See Also

DataFrame.get : Get item from object for given key (ex: DataFrame column).
Series.get : Get item from object for given key (ex: DataFrame column).

```

#### Examples

```

>>> df = pd.DataFrame(
... [
... [24.3, 75.7, "high"],
... [31, 87.8, "high"],
... [22, 71.6, "medium"],
... [35, 95, "medium"],
...],
... columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
... index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
...)

```

```

>>> df
 temp_celsius temp_fahrenheit windspeed
2014-02-12 24.3 75.7 high
2014-02-13 31.0 87.8 high
2014-02-14 22.0 71.6 medium
2014-02-15 35.0 95.0 medium

```

```

>>> df.get(["temp_celsius", "windspeed"])
 temp_celsius windspeed
2014-02-12 24.3 high
2014-02-13 31.0 high
2014-02-14 22.0 medium
2014-02-15 35.0 medium

```

```

>>> ser = df["windspeed"]
>>> ser.get("2014-02-13")
'high'

```

If the key isn't found, the default value will be used.

```

>>> df.get(["temp_celsius", "temp_kelvin"], default="default_value")
'default_value'

```

```

>>> ser.get("2014-02-10", "[unknown]")
'[unknown]'

```

```

head(self, n: int = 5) -> Self
 Return the first `n` rows.

```

This function exhibits the same behavior as ``df[:n]``, returning the first ``n`` rows based on position. It is useful for quickly checking if your object has the right type of data in it.

When ``n`` is positive, it returns the first ``n`` rows. For ``n`` equal to 0, it returns an empty object. When ``n`` is negative, it returns all rows except the last ``|n|`` rows, mirroring the behavior of ``df[:n]``.

If ``n`` is larger than the number of rows, this function returns all rows.

#### Parameters

```

n : int, default 5
 Number of rows to select.

```

#### Returns

```

same type as caller
 The first `n` rows of the caller object.

```

See Also

-----  
DataFrame.tail: Returns the last `n` rows.

Examples

-----  
>>> df = pd.DataFrame(  
... {  
... "animal": [  
... "alligator",  
... "bee",  
... "falcon",  
... "lion",  
... "monkey",  
... "parrot",  
... "shark",  
... "whale",  
... "zebra",  
... ]  
... }  
... )  
>>> df  
 animal  
0 alligator  
1 bee  
2 falcon  
3 lion  
4 monkey  
5 parrot  
6 shark  
7 whale  
8 zebra

Viewing the first 5 lines

```
>>> df.head()
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
```

Viewing the first `n` lines (three in this case)

```
>>> df.head(3)
 animal
0 alligator
1 bee
2 falcon
```

For negative values of `n`

```
>>> df.head(-3)
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
5 parrot
```

infer\_objects(self, copy: bool | lib.NoDefault = <no\_default>) -> Self  
Attempt to infer better dtypes for object columns.

Attempts soft conversion of object-dtyped columns, leaving non-object and unconvertible columns unchanged. The inference rules are the same as during normal Series/DataFrame construction.

Parameters

-----  
copy : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_ for more details.

Returns

-----

same type as input object

Returns an object of the same type as the input object.

See Also

-----

`to_datetime` : Convert argument to datetime.

`to_timedelta` : Convert argument to timedelta.

`to_numeric` : Convert argument to numeric type.

`convert_dtypes` : Convert argument to best possible dtype.

Examples

-----

```
>>> df = pd.DataFrame({"A": ["a", 1, 2, 3]})
```

```
>>> df = df.iloc[1:]
```

```
>>> df
```

```
 A
1 1
2 2
3 3
```

```
>>> df.dtypes
```

```
A object
dtype: object
```

```
>>> df.infer_objects().dtypes
```

```
A int64
dtype: object
```

```
interpolate(
 self,
 method: InterpolateOptions = 'linear',
 *,
 axis: Axis = 0,
 limit: int | None = None,
 inplace: bool = False,
 limit_direction: Literal['forward', 'backward', 'both'] | None = None,
 limit_area: Literal['inside', 'outside'] | None = None,
 **kwargs
) -> Self
 Fill NaN values using an interpolation method.
```

Please note that only ``method='linear'`` is supported for DataFrame/Series with a MultiIndex.

Parameters

-----

`method` : str, default 'linear'

Interpolation technique to use. One of:

- \* 'linear': Ignore the index and treat the values as equally spaced. This is the only method supported on MultiIndexes.
- \* 'time': Works on daily and higher resolution data to interpolate given length of interval. This interpolates values based on time interval between observations.
- \* 'index': The interpolation uses the numerical values of the DataFrame's index to linearly calculate missing values.
- \* 'values': Interpolation based on the numerical values in the DataFrame, treating them as equally spaced along the index.
- \* 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric', 'polynomial': Passed to ``scipy.interpolate.interp1d``, whereas 'spline' is passed to ``scipy.interpolate.UnivariateSpline``. These methods use the numerical values of the index. Both 'polynomial' and 'spline' require that you also specify an ``order`` (int), e.g. ``df.interpolate(method='polynomial', order=5)``. Note that, 'slinear' method in Pandas refers to the Scipy first order 'spline'

```

 instead of Pandas first order `spline`.
 * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima',
 'cubicspline': Wrappers around the SciPy interpolation methods of
 similar names. See `Notes`.
 * 'from_derivatives': Refers to
 `scipy.interpolate.BPoly.from_derivatives`.

axis : {{0 or 'index', 1 or 'columns', None}}, default None
 Axis to interpolate along. For `Series` this parameter is unused
 and defaults to 0.
limit : int, optional
 Maximum number of consecutive NaNs to fill. Must be greater than
 0.
inplace : bool, default False
 Update the data in place if possible.
limit_direction : {{'forward', 'backward', 'both'}}, optional, default 'forward'
 Consecutive NaNs will be filled in this direction.

limit_area : {{None, 'inside', 'outside'}}, default None
 If limit is specified, consecutive NaNs will be filled with this
 restriction.

 * ``None``: No fill restriction.
 * 'inside': Only fill NaNs surrounded by valid values
 (interpolate).
 * 'outside': Only fill NaNs outside valid values (extrapolate).

**kwargs : optional
 Keyword arguments to pass on to the interpolating function.

Returns

Series or DataFrame
 Returns the same object type as the caller, interpolated at
 some or all ``NaN`` values.

See Also

fillna : Fill missing values using different methods.
scipy.interpolate.Akima1DInterpolator : Piecewise cubic polynomials
 (Akima interpolator).
scipy.interpolate.BPoly.from_derivatives : Piecewise polynomial in the
 Bernstein basis.
scipy.interpolate.interp1d : Interpolate a 1-D function.
scipy.interpolate.KroghInterpolator : Interpolate polynomial (Krogh
 interpolator).
scipy.interpolate.PchipInterpolator : PCHIP 1-d monotonic cubic
 interpolation.
scipy.interpolate.CubicSpline : Cubic spline data interpolator.

Notes

The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima'
methods are wrappers around the respective SciPy implementations of
similar names. These use the actual numerical values of the index.
For more information on their behavior, see the
`SciPy documentation`
<https://docs.scipy.org/doc/scipy/reference/interpolate.html#univariate-interpolation>`

Examples

Filling in ``NaN`` in a :class:`~pandas.Series` via linear
interpolation.

>>> s = pd.Series([0, 1, np.nan, 3])
>>> s
0 0.0
1 1.0
2 NaN
3 3.0
dtype: float64
>>> s.interpolate()
0 0.0
1 1.0
2 2.0
3 3.0
dtype: float64

```

Filling in ``NaN`` in a Series via polynomial interpolation or splines:  
Both 'polynomial' and 'spline' methods require that you also specify  
an ``order`` (int).

```
>>> s = pd.Series([0, 2, np.nan, 8])
>>> s.interpolate(method="polynomial", order=2)
0 0.000000
1 2.000000
2 4.666667
3 8.000000
dtype: float64
```

Fill the DataFrame forward (that is, going down) along each column  
using linear interpolation.

Note how the last entry in column 'a' is interpolated differently,  
because there is no entry after it to use for interpolation.  
Note how the first entry in column 'b' remains ``NaN``, because there  
is no entry before it to use for interpolation.

```
>>> df = pd.DataFrame(
... [
... (0.0, np.nan, -1.0, 1.0),
... (np.nan, 2.0, np.nan, np.nan),
... (2.0, 3.0, np.nan, 9.0),
... (np.nan, 4.0, -4.0, 16.0),
...],
... columns=list("abcd"),
...)
>>> df
 a b c d
0 0.0 NaN -1.0 1.0
1 NaN 2.0 NaN NaN
2 2.0 3.0 NaN 9.0
3 NaN 4.0 -4.0 16.0
>>> df.interpolate(method="linear", limit_direction="forward", axis=0)
 a b c d
0 0.0 NaN -1.0 1.0
1 1.0 2.0 -2.0 5.0
2 2.0 3.0 -3.0 9.0
3 2.0 4.0 -4.0 16.0
```

Using polynomial interpolation.

```
>>> df["d"].interpolate(method="polynomial", order=2)
0 1.0
1 4.0
2 9.0
3 16.0
Name: d, dtype: float64
```

`last_valid_index(self)` -> Hashable

Return index for last non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information  
on which values are considered missing.

Returns

-----  
type of index  
Index of last non-missing value.

See Also

-----  
`DataFrame.first_valid_index` : Return index for first non-NA value or None, if  
no non-NA value is found.  
`Series.first_valid_index` : Return index for first non-NA value or None, if no  
non-NA value is found.  
`DataFrame.isna` : Detect missing values.

Examples

-----  
For Series:

```
>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
```

```
1
>>> s.last_valid_index()
2
```

```
>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None
```

If all elements in Series are NA/null, returns None.

```
>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None
```

If Series is empty, returns None.

For DataFrame:

```
>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
 A B
0 NaN NaN
1 NaN 3.0
2 2.0 4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2
```

```
>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
 A B
0 None None
1 None None
2 None None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If DataFrame is empty, returns None.

```
mask(
 self,
 cond,
 other=<no_default>,
 *,
 inplace: bool = False,
 axis: Axis | None = None,
 level: Level | None = None
) -> Self
Replace values where the condition is True.
```

Parameters

```

cond : bool Series/DataFrame, array-like, or callable
 Where `cond` is False, keep the original value. Where
 True, replace with corresponding value from `other`.
 If `cond` is callable, it is computed on the Series/DataFrame and
 should return boolean Series/DataFrame or array. The callable must
 not change input Series/DataFrame (though pandas doesn't check it).
```

other : scalar, Series/DataFrame, or callable  
 Entries where `cond` is True are replaced with corresponding value from `other`.  
 If other is callable, it is computed on the Series/DataFrame and should return scalar or Series/DataFrame. The callable must not change input Series/DataFrame (though pandas doesn't check it).  
 If not specified, entries will be filled with the corresponding NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension dtypes).

inplace : bool, default False  
 Whether to perform the operation in place on the data.

axis : int, default None  
 Alignment axis if needed. For `Series` this parameter is unused and defaults to 0.

level : int, default None  
 Alignment level if needed.

#### Returns

##### Series or DataFrame

When applied to a Series, the function will return a Series, and when applied to a DataFrame, it will return a DataFrame.

#### See Also

:func:`DataFrame.where` : Return an object of same shape as caller.  
 :func:`Series.where` : Return an object of same shape as caller.

#### Notes

The mask method is an application of the if-then idiom. For each element in the caller, if ``cond`` is ``False`` the element is used; otherwise the corresponding element from ``other`` is used. If the axis of ``other`` does not align with axis of ``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions will be filled with True.

The signature for :func:`Series.where` or :func:`DataFrame.where` differs from :func:`numpy.where`. Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

For further details and examples see the ``mask`` documentation in :ref:`indexing <indexing.where\_mask>`.

The dtype of the object takes precedence. The fill value is casted to the object's dtype, if this can be done losslessly.

#### Examples

```
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0 NaN
1 1.0
2 2.0
3 3.0
4 4.0
dtype: float64
>>> s.mask(s > 0)
0 0.0
1 NaN
2 NaN
3 NaN
4 NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0 0
1 99
2 99
3 99
4 99
dtype: int64
>>> s.mask(t, 99)
0 99
1 1
```



```

2 99
3 99
4 99
dtype: int64

>>> s.where(s > 1, 10)
0 10
1 10
2 2
3 3
4 4
dtype: int64
>>> s.mask(s > 1, 10)
0 0
1 1
2 10
3 10
4 10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
 A B
0 0 1
1 2 3
2 4 5
3 6 7
4 8 9
>>> m = df % 3 == 0
>>> df.where(m, -df)
 A B
0 0 -1
1 -2 3
2 -4 -5
3 6 -7
4 -8 9
>>> df.where(m, -df) == np.where(m, df, -df)
 A B
0 True True
1 True True
2 True True
3 True True
4 True True
>>> df.where(m, -df) == df.mask(~m, -df)
 A B
0 True True
1 True True
2 True True
3 True True
4 True True

```

```

pct_change(
 self,
 periods: int = 1,
 fill_method: None = None,
 freq=None,
 **kwargs
) -> Self
Fractional change between the current and a prior element.

```

Computes the fractional change from the immediately previous row by default. This is useful in comparing the fraction of change in a time series of elements.

.. note::

Despite the name of this method, it calculates fractional change (also known as per unit change or relative change) and not percentage change. If you need the percentage change, multiply these values by 100.

Parameters

-----  
periods : int, default 1

Periods to shift for forming percent change.

fill\_method : None

Must be None. This argument will be removed in a future version of pandas.

freq : DateOffset, timedelta, or str, optional  
Increment to use from time series API (e.g. 'ME' or BDay()).  
\*\*kwargs  
Additional keyword arguments are passed into  
`DataFrame.shift` or `Series.shift`.

Returns

Series or DataFrame  
The same type as the calling object.

See Also

Series.diff : Compute the difference of two elements in a Series.  
DataFrame.diff : Compute the difference of two elements in a DataFrame.  
Series.shift : Shift the index by some number of periods.  
DataFrame.shift : Shift the index by some number of periods.

Examples

**\*\*Series\*\***

```
>>> s = pd.Series([90, 91, 85])
>>> s
0 90
1 91
2 85
dtype: int64
```

```
>>> s.pct_change()
0 NaN
1 0.011111
2 -0.065934
dtype: float64
```

```
>>> s.pct_change(periods=2)
0 NaN
1 NaN
2 -0.055556
dtype: float64
```

See the percentage change in a Series where filling NAs with last valid observation forward to next valid.

```
>>> s = pd.Series([90, 91, None, 85])
>>> s
0 90.0
1 91.0
2 NaN
3 85.0
dtype: float64
```

```
>>> s.ffill().pct_change()
0 NaN
1 0.011111
2 0.000000
3 -0.065934
dtype: float64
```

**\*\*DataFrame\*\***

Percentage change in French franc, Deutsche Mark, and Italian lira from 1980-01-01 to 1980-03-01.

```
>>> df = pd.DataFrame(
... {
... "FR": [4.0405, 4.0963, 4.3149],
... "GR": [1.7246, 1.7482, 1.8519],
... "IT": [804.74, 810.01, 860.13],
... },
... index=["1980-01-01", "1980-02-01", "1980-03-01"],
...)
>>> df
```

	FR	GR	IT
1980-01-01	4.0405	1.7246	804.74
1980-02-01	4.0963	1.7482	810.01
1980-03-01	4.3149	1.8519	860.13

```
>>> df.pct_change()
 FR GR IT
1980-01-01 NaN NaN NaN
1980-02-01 0.013810 0.013684 0.006549
1980-03-01 0.053365 0.059318 0.061876
```

Percentage of change in GOOG and APPL stock volume. Shows computing the percentage change between columns.

```
>>> df = pd.DataFrame(
... {
... "2016": [1769950, 30586265],
... "2015": [1500923, 40912316],
... "2014": [1371819, 41403351],
... },
... index=["GOOG", "APPL"],
...)
>>> df
```

```

 2016 2015 2014
GOOG 1769950 1500923 1371819
APPL 30586265 40912316 41403351
```

```
>>> df.pct_change(axis="columns", periods=-1)
 2016 2015 2014
GOOG 0.179241 0.094112 NaN
APPL -0.252395 -0.011860 NaN
```

```
pipe(
 self,
 func: Callable[[Concatenate[Self, P], T] | tuple[Callable[... , T], str],
 *args: Any,
 **kwargs: Any
) -> T
```

Apply chainable functions that expect Series or DataFrames.

Parameters

```
func : function
 Function to apply to the Series/DataFrame.
 ``args``, and ``kwargs`` are passed into ``func``.
 Alternatively a ``(callable, data_keyword)`` tuple where
 ``data_keyword`` is a string indicating the keyword of
 ``callable`` that expects the Series/DataFrame.
*args : iterable, optional
 Positional arguments passed into ``func``.
**kwargs : mapping, optional
 A dictionary of keyword arguments passed into ``func``.
```

Returns

```

The return type of ``func``.
The result of applying ``func`` to the Series or DataFrame.
```

See Also

```

DataFrame.apply : Apply a function along input axis of DataFrame.
DataFrame.map : Apply a function elementwise on a whole DataFrame.
Series.map : Apply a mapping correspondence on a
:~pandas.Series`.
```

Notes

```

Use ``.pipe`` when chaining together functions that expect
Series, DataFrames or GroupBy objects.
```

Examples

```

Constructing an income DataFrame from a dictionary.
```

```
>>> data = [[8000, 1000], [9500, np.nan], [5000, 2000]]
>>> df = pd.DataFrame(data, columns=["Salary", "Others"])
>>> df
 Salary Others
0 8000 1000.0
1 9500 NaN
2 5000 2000.0
```

Functions that perform tax reductions on an income DataFrame.

```
>>> def subtract_federal_tax(df):
... return df * 0.9
>>> def subtract_state_tax(df, rate):
... return df * (1 - rate)
>>> def subtract_national_insurance(df, rate, rate_increase):
... new_rate = rate + rate_increase
... return df * (1 - new_rate)
```

Instead of writing

```
>>> subtract_national_insurance(
... subtract_state_tax(subtract_federal_tax(df), rate=0.12),
... rate=0.05,
... rate_increase=0.02,
...) # doctest: +SKIP
```

You can write

```
>>> (
... df.pipe(subtract_federal_tax)
... .pipe(subtract_state_tax, rate=0.12)
... .pipe(subtract_national_insurance, rate=0.05, rate_increase=0.02)
...)
 Salary Others
0 5892.48 736.56
1 6997.32 NaN
2 3682.80 1473.12
```

If you have a function that takes the data as (say) the second argument, pass a tuple indicating which keyword expects the data. For example, suppose ``national\_insurance`` takes its data as ``df`` in the second argument:

```
>>> def subtract_national_insurance(rate, df, rate_increase):
... new_rate = rate + rate_increase
... return df * (1 - new_rate)
>>> (
... df.pipe(subtract_federal_tax)
... .pipe(subtract_state_tax, rate=0.12)
... .pipe(
... (subtract_national_insurance, "df"), rate=0.05, rate_increase=0.02
...)
...)
 Salary Others
0 5892.48 736.56
1 6997.32 NaN
2 3682.80 1473.12
```

```
rank(
 self,
 axis: Axis = 0,
 method: Literal['average', 'min', 'max', 'first', 'dense'] = 'average',
 numeric_only: bool = False,
 na_option: Literal['keep', 'top', 'bottom'] = 'keep',
 ascending: bool = True,
 pct: bool = False
) -> Self
 Compute numerical data ranks (1 through n) along axis.
```

By default, equal values are assigned a rank that is the average of the ranks of those values.

Parameters

-----

axis : {0 or 'index', 1 or 'columns'}, default 0  
Index to direct ranking.  
For ``Series`` this parameter is unused and defaults to 0.

method : {'average', 'min', 'max', 'first', 'dense'}, default 'average'  
How to rank the group of records that have the same value (i.e. ties):

- \* average: average rank of the group
- \* min: lowest rank in the group
- \* max: highest rank in the group
- \* first: ranks assigned in order they appear in the array

```
* dense: like 'min', but rank always increases by 1 between groups.
numeric_only : bool, default False
 For DataFrame objects, rank only numeric columns if set to True.
```

```
.. versionchanged:: 2.0.0
 The default value of ``numeric_only`` is now ``False``.
```

```
na_option : {'keep', 'top', 'bottom'}, default 'keep'
 How to rank NaN values:
```

```
* keep: assign NaN rank to NaN values
* top: assign lowest rank to NaN values
* bottom: assign highest rank to NaN values
```

```
ascending : bool, default True
 Whether or not the elements should be ranked in ascending order.
pct : bool, default False
 Whether or not to display the returned rankings in percentile
 form.
```

Returns

```

same type as caller
 Return a Series or DataFrame with data ranks as values.
```

See Also

```

core.groupby.DataFrameGroupBy.rank : Rank of values within each group.
core.groupby.SeriesGroupBy.rank : Rank of values within each group.
```

Examples

```

>>> df = pd.DataFrame(
... data={
... "Animal": ["cat", "penguin", "dog", "spider", "snake"],
... "Number_legs": [4, 2, 4, 8, np.nan],
... }
...)
>>> df
 Animal Number_legs
0 cat 4.0
1 penguin 2.0
2 dog 4.0
3 spider 8.0
4 snake NaN
```

Ties are assigned the mean of the ranks (by default) for the group.

```
>>> s = pd.Series(range(5), index=list("abcde"))
>>> s["d"] = s["b"]
>>> s.rank()
a 1.0
b 2.5
c 4.0
d 2.5
e 5.0
dtype: float64
```

The following example shows how the method behaves with the above parameters:

```
* default_rank: this is the default behaviour obtained without using
 any parameter.
* max_rank: setting ``method = 'max'`` the records that have the
 same values are ranked using the highest rank (e.g.: since 'cat'
 and 'dog' are both in the 2nd and 3rd position, rank 3 is assigned.)
* NA_bottom: choosing ``na_option = 'bottom'``, if there are records
 with NaN values they are placed at the bottom of the ranking.
* pct_rank: when setting ``pct = True``, the ranking is expressed as
 percentile rank.
```

```
>>> df["default_rank"] = df["Number_legs"].rank()
>>> df["max_rank"] = df["Number_legs"].rank(method="max")
>>> df["NA_bottom"] = df["Number_legs"].rank(na_option="bottom")
>>> df["pct_rank"] = df["Number_legs"].rank(pct=True)
>>> df
```

	Animal	Number_legs	default_rank	max_rank	NA_bottom	pct_rank
0	cat	4.0	2.5	3.0	2.5	0.625
1	penguin	2.0	1.0	1.0	1.0	0.250
2	dog	4.0	2.5	3.0	2.5	0.625
3	spider	8.0	4.0	4.0	4.0	1.000
4	snake	NaN	NaN	NaN	5.0	NaN

```
reindex_like(
 self,
 other,
 method: Literal['backfill', 'bfill', 'pad', 'ffill', 'nearest'] | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 limit: int | None = None,
 tolerance=None
) -> Self
Return an object with matching indices as other object.
```

Conform the object to the same index on all axes. Optional filling logic, placing NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and copy=False.

#### Parameters

-----

**other** : Object of the same data type  
Its row and column indices are used to define the new indices of this object.

**method** : {None, 'backfill', 'bfill', 'pad', 'ffill', 'nearest'}  
Method to use for filling holes in reindexed DataFrame. Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

.. deprecated:: 3.0.0

- \* None (default): don't fill gaps
- \* pad / ffill: propagate last valid observation forward to next valid
- \* backfill / bfill: use next valid observation to fill gap
- \* nearest: use nearest valid observations to fill gap.

**copy** : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` [https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

**limit** : int, default None  
Maximum number of consecutive labels to fill for inexact matches.

**tolerance** : optional  
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

#### Returns

-----

**Series or DataFrame**  
Same type as caller, but with changed indices on each axis.

#### See Also

-----

**DataFrame.set\_index** : Set row labels.  
**DataFrame.reset\_index** : Remove row labels or move them to new columns.  
**DataFrame.reindex** : Change to new indices or expand indices.

## Notes

-----

Same as calling

```
`.reindex(index=other.index, columns=other.columns,...)`.
```

## Examples

-----

```
>>> df1 = pd.DataFrame(
... [
... [24.3, 75.7, "high"],
... [31, 87.8, "high"],
... [22, 71.6, "medium"],
... [35, 95, "medium"],
...],
... columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
... index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
...)
```

```
>>> df1
 temp_celsius temp_fahrenheit windspeed
2014-02-12 24.3 75.7 high
2014-02-13 31.0 87.8 high
2014-02-14 22.0 71.6 medium
2014-02-15 35.0 95.0 medium
```

```
>>> df2 = pd.DataFrame(
... [[28, "low"], [30, "low"], [35.1, "medium"]],
... columns=["temp_celsius", "windspeed"],
... index=pd.DatetimeIndex(["2014-02-12", "2014-02-13", "2014-02-15"]),
...)
```

```
>>> df2
 temp_celsius windspeed
2014-02-12 28.0 low
2014-02-13 30.0 low
2014-02-15 35.1 medium
```

```
>>> df2.reindex_like(df1)
 temp_celsius temp_fahrenheit windspeed
2014-02-12 28.0 NaN low
2014-02-13 30.0 NaN low
2014-02-14 NaN NaN NaN
2014-02-15 35.1 NaN medium
```

```
replace(
 self,
 to_replace=None,
 value=<no_default>,
 *,
 inplace: bool = False,
 regex: bool = False
) -> Self
Replace values given in `to_replace` with `value`.
```

Values of the Series/DataFrame are replaced with other values dynamically. This differs from updating with ``.loc`` or ``.iloc``, which require you to specify a location to update with some value.

## Parameters

-----

`to_replace` : str, regex, list, dict, Series, int, float, or None  
How to find the values that will be replaced.

\* numeric, str or regex:

- numeric: numeric values equal to ``to_replace`` will be replaced with ``value``
- str: string exactly matching ``to_replace`` will be replaced with ``value``
- regex: regexes matching ``to_replace`` will be replaced with ``value``

\* list of str, regex, or numeric:

- First, if ``to_replace`` and ``value`` are both lists, they **must** be the same length.
- Second, if ```regex=True``` then all of the strings in **both**

lists will be interpreted as regexes otherwise they will match directly. This doesn't matter much for `value` since there are only a few possible substitution regexes you can use.

- str, regex and numeric rules apply as above.

\* dict:

- Dicts can be used to specify different replacement values for different existing values. For example, ``{'a': 'b', 'y': 'z'}`` replaces the value 'a' with 'b' and 'y' with 'z'. To use a dict in this way, the optional `value` parameter should not be given.
- For a DataFrame a dict can specify that different values should be replaced in different columns. For example, ``{'a': 1, 'b': 'z'}`` looks for the value 1 in column 'a' and the value 'z' in column 'b' and replaces these values with whatever is specified in `value`. The `value` parameter should not be ``None`` in this case. You can treat this as a special case of passing two lists except that you are specifying the column to search in.
- For a DataFrame nested dictionaries, e.g., ``{'a': {'b': np.nan}}``, are read as follows: look in column 'a' for the value 'b' and replace it with NaN. The optional `value` parameter should not be specified to use a nested dict in this way. You can nest regular expressions as well. Note that column names (the top-level dictionary keys in a nested dictionary) **cannot** be regular expressions.

\* None:

- This means that the `regex` argument must be a string, compiled regular expression, or list, dict, ndarray or Series of such elements. If `value` is also ``None`` then this **must** be a nested dictionary or Series.

See the examples section for examples of each of these.

value : scalar, dict, list, str, regex, default None  
Value to replace any values matching `to\_replace` with.  
For a DataFrame a dict of values can be used to specify which value to use for each column (columns not in the dict will not be filled). Regular expressions, strings and lists or dicts of such objects are also allowed.

inplace : bool, default False

If True, performs operation inplace.

regex : bool or same types as `to\_replace`, default False  
Whether to interpret `to\_replace` and/or `value` as regular expressions. Alternatively, this could be a regular expression or a list, dict, or array of regular expressions in which case `to\_replace` must be ``None``.

Returns

-----

Series/DataFrame  
Object after replacement.

Raises

-----

AssertionError

- \* If `regex` is not a ``bool`` and `to\_replace` is not ``None``.

TypeError

- \* If `to\_replace` is not a scalar, array-like, ``dict``, or ``None``
- \* If `to\_replace` is a ``dict`` and `value` is not a ``list``, ``dict``, ``ndarray``, or ``Series``
- \* If `to\_replace` is ``None`` and `regex` is not compilable into a regular expression or is a list, dict, ndarray, or Series.
- \* When replacing multiple ``bool`` or ``datetime64`` objects and the arguments to `to\_replace` does not match the type of the value being replaced

ValueError

- \* If a ``list`` or an ``ndarray`` is passed to `to\_replace` and `value` but they are not the same length.



See Also

-----

Series.fillna : Fill NA values.

DataFrame.fillna : Fill NA values.

Series.where : Replace values based on boolean condition.

DataFrame.where : Replace values based on boolean condition.

DataFrame.map: Apply a function to a DataFrame elementwise.

Series.map: Map values of Series according to an input mapping or function.

Series.str.replace : Simple string replacement.

Notes

-----

\* Regex substitution is performed under the hood with ``re.sub``. The rules for substitution for ``re.sub`` are the same.

\* Regular expressions will only substitute on strings, meaning you cannot provide, for example, a regular expression matching floating point numbers and expect the columns in your frame that have a numeric dtype to be matched. However, if those floating point numbers *are* strings, then you can do this.

\* This method has *a lot* of options. You are encouraged to experiment and play with this method to gain intuition about how it works.

\* When dict is used as the `to_replace` value, it is like key(s) in the dict are the `to_replace` part and value(s) in the dict are the `value` parameter.

Examples

-----

**Scalar `to_replace` and `value`**

```
>>> s = pd.Series([1, 2, 3, 4, 5])
```

```
>>> s.replace(1, 5)
```

```
0 5
1 2
2 3
3 4
4 5
dtype: int64
```

```
>>> df = pd.DataFrame(
... {
... "A": [0, 1, 2, 3, 4],
... "B": [5, 6, 7, 8, 9],
... "C": ["a", "b", "c", "d", "e"],
... }
...)
```

```
>>> df.replace(0, 5)
```

```
 A B C
0 5 5 a
1 1 6 b
2 2 7 c
3 3 8 d
4 4 9 e
```

**List-like `to_replace`**

```
>>> df.replace([0, 1, 2, 3], 4)
```

```
 A B C
0 4 5 a
1 4 6 b
2 4 7 c
3 4 8 d
4 4 9 e
```

```
>>> df.replace([0, 1, 2, 3], [4, 3, 2, 1])
```

```
 A B C
0 4 5 a
1 3 6 b
2 2 7 c
3 1 8 d
4 4 9 e
```

**dict-like `to_replace`**

```
>>> df.replace({0: 10, 1: 100})
```

```
 A B C
0 10 5 a
```

```

1 100 6 b
2 2 7 c
3 3 8 d
4 4 9 e

```

```
>>> df.replace({"A": 0, "B": 5}, 100)
```

```

 A B C
0 100 100 a
1 1 6 b
2 2 7 c
3 3 8 d
4 4 9 e

```

```
>>> df.replace({"A": {0: 100, 4: 400}})
```

```

 A B C
0 100 5 a
1 1 6 b
2 2 7 c
3 3 8 d
4 400 9 e

```

**\*\*Regular expression `to\_replace`\*\***

```
>>> df = pd.DataFrame({"A": ["bat", "foo", "bait"], "B": ["abc", "bar", "xyz"]})
```

```
>>> df.replace(to_replace=r"^ba.$", value="new", regex=True)
```

```

 A B
0 new abc
1 foo new
2 bait xyz

```

```
>>> df.replace({"A": r"^ba.$"}, {"A": "new"}, regex=True)
```

```

 A B
0 new abc
1 foo bar
2 bait xyz

```

```
>>> df.replace(regex=r"^ba.$", value="new")
```

```

 A B
0 new abc
1 foo new
2 bait xyz

```

```
>>> df.replace(regex={r"^ba.$": "new", "foo": "xyz"})
```

```

 A B
0 new abc
1 xyz new
2 bait xyz

```

```
>>> df.replace(regex=[r"^ba.$", "foo"], value="new")
```

```

 A B
0 new abc
1 new new
2 bait xyz

```

Compare the behavior of `s.replace({'a': None})` and `s.replace('a', None)` to understand the peculiarities of the `to_replace` parameter:

```
>>> s = pd.Series([10, "a", "a", "b", "a"])
```

When one uses a dict as the `to_replace` value, it is like the value(s) in the dict are equal to the `value` parameter.

`s.replace({'a': None})` is equivalent to `s.replace(to_replace={'a': None}, value=None)`:

```
>>> s.replace({"a": None})
```

```

0 10
1 None
2 None
3 b
4 None
dtype: object

```

If `None` is explicitly passed for `value`, it will be respected:

```
>>> s.replace("a", None)
```

```

0 10

```

```

1 None
2 None
3 b
4 None
dtype: object

```

When ``regex=True``, ``value`` is not ``None`` and ``to\_replace`` is a string, the replacement will be applied in all columns of the DataFrame.

```

>>> df = pd.DataFrame(
... {
... "A": [0, 1, 2, 3, 4],
... "B": ["a", "b", "c", "d", "e"],
... "C": ["f", "g", "h", "i", "j"],
... }
...)

>>> df.replace(to_replace="^[a-g]", value="e", regex=True)
 A B C
0 0 e e
1 1 e e
2 2 e h
3 3 e i
4 4 e j

```

If ``value`` is not ``None`` and ``to\_replace`` is a dictionary, the dictionary keys will be the DataFrame columns that the replacement will be applied.

```

>>> df.replace(to_replace={"B": "^[a-c]", "C": "^[h-j]"}, value="e", regex=True)
 A B C
0 0 e f
1 1 e g
2 2 e e
3 3 d e
4 4 e e

```

```

resample(
 self,
 rule,
 closed: Literal['right', 'left'] | None = None,
 label: Literal['right', 'left'] | None = None,
 convention: Literal['start', 'end', 's', 'e'] = 'start',
 on: Level | None = None,
 level: Level | None = None,
 origin: str | TimestampConvertibleTypes = 'start_day',
 offset: TimedeltaConvertibleTypes | None = None,
 group_keys: bool = False
) -> Resampler
Resample time-series data.

```

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (`DatetimeIndex`, `PeriodIndex`, or `TimedeltaIndex`), or the caller must pass the label of a datetime-like series/index to the `on`/`level` keyword parameter.

#### Parameters

```

rule : DateOffset, Timedelta or str
 The offset string or object representing target conversion.
closed : {'right', 'left'}, default None
 Which side of bin interval is closed. The default is 'left'
 for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
 'BA', 'BQE', and 'W' which all have a default of 'right'.
label : {'right', 'left'}, default None
 Which bin edge label to label bucket with. The default is 'left'
 for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
 'BA', 'BQE', and 'W' which all have a default of 'right'.
convention : {'start', 'end', 's', 'e'}, default 'start'
 For 'PeriodIndex' only, controls whether to use the start or
 end of 'rule'.
on : str, optional
 For a DataFrame, column to use instead of index for resampling.
 Column must be datetime-like.
level : str or int, optional
 For a MultiIndex, level (name or number) to use for
 resampling. 'level' must be datetime-like.
origin : Timestamp or str, default 'start_day'

```

The timestamp on which to adjust the grouping. The timezone of origin must match the timezone of the index.  
If string, must be Timestamp convertible or one of the following:

- 'epoch': 'origin' is 1970-01-01
- 'start': 'origin' is the first value of the timeseries
- 'start\_day': 'origin' is the first day at midnight of the timeseries
- 'end': 'origin' is the last value of the timeseries
- 'end\_day': 'origin' is the ceiling midnight of the last day

.. note::

Only takes effect for Tick-frequencies (i.e. fixed frequencies like days, hours, and minutes, rather than months or quarters).  
offset : Timedelta or str, default is None  
An offset timedelta added to the origin.

group\_keys : bool, default False  
Whether to include the group keys in the result index when using ``.apply()`` on the resampled object.

.. versionchanged:: 2.0.0

``group\_keys`` now defaults to ``False``.

Returns

-----  
pandas.api.typing.Resampler  
:class: ~pandas.core.Resampler` object.

See Also

-----  
Series.resample : Resample a Series.  
DataFrame.resample : Resample a DataFrame.  
groupby : Group Series/DataFrame by mapping, function, label, or list of labels.  
asfreq : Reindex a Series/DataFrame with the given frequency without grouping.

Notes

-----  
See the `user guide`  
<[https://pandas.pydata.org/pandas-docs/stable/user\\_guide/timeseries.html#resampling](https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#resampling)>`\_\_  
for more.

To learn more about the offset strings, please see `this link`  
<[https://pandas.pydata.org/pandas-docs/stable/user\\_guide/timeseries.html#dateoffset-objects](https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#dateoffset-objects)>

Examples

-----  
Start by creating a series with 9 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=9, freq="min")
>>> series = pd.Series(range(9), index=index)
>>> series
2000-01-01 00:00:00 0
2000-01-01 00:01:00 1
2000-01-01 00:02:00 2
2000-01-01 00:03:00 3
2000-01-01 00:04:00 4
2000-01-01 00:05:00 5
2000-01-01 00:06:00 6
2000-01-01 00:07:00 7
2000-01-01 00:08:00 8
Freq: min, dtype: int64
```

Downsample the series into 3 minute bins and sum the values of the timestamps falling into a bin.

```
>>> series.resample("3min").sum()
2000-01-01 00:00:00 3
2000-01-01 00:03:00 12
2000-01-01 00:06:00 21
Freq: 3min, dtype: int64
```

Downsample the series into 3 minute bins as above, but label each bin using the right edge instead of the left. Please note that the value in the bucket used as the label is not included in the bucket,

which it labels. For example, in the original series the bucket ``2000-01-01 00:03:00`` contains the value 3, but the summed value in the resampled bucket with the label ``2000-01-01 00:03:00`` does not include 3 (if it did, the summed value would be 6, not 3).

```
>>> series.resample("3min", label="right").sum()
2000-01-01 00:03:00 3
2000-01-01 00:06:00 12
2000-01-01 00:09:00 21
Freq: 3min, dtype: int64
```

To include this value close the right side of the bin interval, as shown below.

```
>>> series.resample("3min", label="right", closed="right").sum()
2000-01-01 00:00:00 0
2000-01-01 00:03:00 6
2000-01-01 00:06:00 15
2000-01-01 00:09:00 15
Freq: 3min, dtype: int64
```

Upsample the series into 30 second bins.

```
>>> series.resample("30s").asfreq()[0:5] # Select first 5 rows
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 1.0
2000-01-01 00:01:30 NaN
2000-01-01 00:02:00 2.0
Freq: 30s, dtype: float64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``ffill`` method.

```
>>> series.resample("30s").ffill()[0:5]
2000-01-01 00:00:00 0
2000-01-01 00:00:30 0
2000-01-01 00:01:00 1
2000-01-01 00:01:30 1
2000-01-01 00:02:00 2
Freq: 30s, dtype: int64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``bfill`` method.

```
>>> series.resample("30s").bfill()[0:5]
2000-01-01 00:00:00 0
2000-01-01 00:00:30 1
2000-01-01 00:01:00 1
2000-01-01 00:01:30 2
2000-01-01 00:02:00 2
Freq: 30s, dtype: int64
```

Pass a custom function via ``apply``

```
>>> def custom_resampler(arraylike):
... return np.sum(arraylike) + 5
>>> series.resample("3min").apply(custom_resampler)
2000-01-01 00:00:00 8
2000-01-01 00:03:00 17
2000-01-01 00:06:00 26
Freq: 3min, dtype: int64
```

For a Series with a PeriodIndex, the keyword ``convention`` can be used to control whether to use the start or end of ``rule``.

Resample a year by quarter using 'start' ``convention``. Values are assigned to the first quarter of the period.

```
>>> s = pd.Series(
... [1, 2], index=pd.period_range("2012-01-01", freq="Y", periods=2)
...)
>>> s
2012 1
2013 2
Freq: Y-DEC, dtype: int64
>>> s.resample("Q", convention="start").asfreq()
```

```

2012Q1 1.0
2012Q2 NaN
2012Q3 NaN
2012Q4 NaN
2013Q1 2.0
2013Q2 NaN
2013Q3 NaN
2013Q4 NaN
Freq: Q-DEC, dtype: float64

```

Resample quarters by month using 'end' `convention`. Values are assigned to the last month of the period.

```

>>> q = pd.Series(
... [1, 2, 3, 4], index=pd.period_range("2018-01-01", freq="Q", periods=4)
...)
>>> q
2018Q1 1
2018Q2 2
2018Q3 3
2018Q4 4
Freq: Q-DEC, dtype: int64
>>> q.resample("M", convention="end").asfreq()
2018-03 1.0
2018-04 NaN
2018-05 NaN
2018-06 2.0
2018-07 NaN
2018-08 NaN
2018-09 3.0
2018-10 NaN
2018-11 NaN
2018-12 4.0
Freq: M, dtype: float64

```

For DataFrame objects, the keyword `on` can be used to specify the column instead of the index for resampling.

```

>>> df = pd.DataFrame([10, 11, 9, 13, 14, 18, 17, 19], columns=["price"])
>>> df["volume"] = [50, 60, 40, 100, 50, 100, 40, 50]
>>> df["week_starting"] = pd.date_range("01/01/2018", periods=8, freq="W")
>>> df
 price volume week_starting
0 10 50 2018-01-07
1 11 60 2018-01-14
2 9 40 2018-01-21
3 13 100 2018-01-28
4 14 50 2018-02-04
5 18 100 2018-02-11
6 17 40 2018-02-18
7 19 50 2018-02-25
>>> df.resample("ME", on="week_starting").mean()
 price volume
week_starting
2018-01-31 10.75 62.5
2018-02-28 17.00 60.0

```

For a DataFrame with MultiIndex, the keyword `level` can be used to specify on which level the resampling needs to take place.

```

>>> days = pd.date_range("1/1/2000", periods=4, freq="D")
>>> df2 = pd.DataFrame(
... [
... [10, 50],
... [11, 60],
... [9, 40],
... [13, 100],
... [14, 50],
... [18, 100],
... [17, 40],
... [19, 50],
...],
... columns=["price", "volume"],
... index=pd.MultiIndex.from_product([days, ["morning", "afternoon"]]),
...)
>>> df2

```

price volume

2000-01-01	morning	10	50
	afternoon	11	60
2000-01-02	morning	9	40
	afternoon	13	100
2000-01-03	morning	14	50
	afternoon	18	100
2000-01-04	morning	17	40
	afternoon	19	50

```
>>> df2.resample("D", level=0).sum()
 price volume
2000-01-01 21 110
2000-01-02 22 140
2000-01-03 32 150
2000-01-04 36 90
```

If you want to adjust the start of the bins based on a fixed timestamp:

```
>>> start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
>>> rng = pd.date_range(start, end, freq="7min")
>>> ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
>>> ts
2000-10-01 23:30:00 0
2000-10-01 23:37:00 3
2000-10-01 23:44:00 6
2000-10-01 23:51:00 9
2000-10-01 23:58:00 12
2000-10-02 00:05:00 15
2000-10-02 00:12:00 18
2000-10-02 00:19:00 21
2000-10-02 00:26:00 24
Freq: 7min, dtype: int64
```

```
>>> ts.resample("17min").sum()
2000-10-01 23:14:00 0
2000-10-01 23:31:00 9
2000-10-01 23:48:00 21
2000-10-02 00:05:00 54
2000-10-02 00:22:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", origin="epoch").sum()
2000-10-01 23:18:00 0
2000-10-01 23:35:00 18
2000-10-01 23:52:00 27
2000-10-02 00:09:00 39
2000-10-02 00:26:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", origin="2000-01-01").sum()
2000-10-01 23:24:00 3
2000-10-01 23:41:00 15
2000-10-01 23:58:00 45
2000-10-02 00:15:00 45
Freq: 17min, dtype: int64
```

If you want to adjust the start of the bins with an `offset` Timedelta, the two following lines are equivalent:

```
>>> ts.resample("17min", origin="start").sum()
2000-10-01 23:30:00 9
2000-10-01 23:47:00 21
2000-10-02 00:04:00 54
2000-10-02 00:21:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", offset="23h30min").sum()
2000-10-01 23:30:00 9
2000-10-01 23:47:00 21
2000-10-02 00:04:00 54
2000-10-02 00:21:00 24
Freq: 17min, dtype: int64
```

If you want to take the largest Timestamp as the end of the bins:

```
>>> ts.resample("17min", origin="end").sum()
2000-10-01 23:35:00 0
2000-10-01 23:52:00 18
```

```

2000-10-02 00:09:00 27
2000-10-02 00:26:00 63
Freq: 17min, dtype: int64

```

In contrast with the ``start_day``, you can use ``end_day`` to take the ceiling midnight of the largest Timestamp as the end of the bins and drop the bins not containing data:

```

>>> ts.resample("17min", origin="end_day").sum()
2000-10-01 23:38:00 3
2000-10-01 23:55:00 15
2000-10-02 00:12:00 45
2000-10-02 00:29:00 45
Freq: 17min, dtype: int64

```

```

rolling(
 self,
 window: int | dt.timedelta | str | BaseOffset | BaseIndexer,
 min_periods: int | None = None,
 center: bool = False,
 win_type: str | None = None,
 on: str | None = None,
 closed: IntervalClosedType | None = None,
 step: int | None = None,
 method: str = 'single'
) -> Window | Rolling
Provide rolling window calculations.

```

#### Parameters

**window** : int, timedelta, str, offset, or BaseIndexer subclass  
Interval of the moving window.

If an integer, the delta between the start and end of each window.  
The number of points in the window depends on the ``closed`` argument.

If a timedelta, str, or offset, the time period of each window. Each window will be a variable sized based on the observations included in the time-period. This is only valid for datetimelike indexes.  
To learn more about the offsets & frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

If a BaseIndexer subclass, the window boundaries based on the defined ``get_window_bounds`` method. Additional rolling keyword arguments, namely ``min_periods``, ``center``, ``closed`` and ``step`` will be passed to ``get_window_bounds``.

**min\_periods** : int, default None  
Minimum number of observations in window required to have a value; otherwise, result is ``np.nan``.

For a window that is specified by an offset, ``min_periods`` will default to 1.

For a window that is specified by an integer, ``min_periods`` will default to the size of the window.

**center** : bool, default False  
If False, set the window labels as the right edge of the window index.

If True, set the window labels as the center of the window index.

**win\_type** : str, default None  
If ``None``, all points are evenly weighted.

If a string, it must be a valid ``scipy.signal`` window function  
<<https://docs.scipy.org/doc/scipy/reference/signal.windows.html#module-scipy.signal>>.

Certain Scipy window types require additional parameters to be passed in the aggregation function. The additional parameters must match the keywords specified in the Scipy window type method signature.

**on** : str, optional  
For a DataFrame, a column label or Index level on which to calculate the rolling window, rather than the DataFrame's index.

Provided integer column is ignored and excluded from result since



an integer index is not used to calculate the rolling window.

**closed** : str, default None  
Determines the inclusivity of points in the window

If `''right''`, uses the window (first, last] meaning the last point is included in the calculations.

If `''left''`, uses the window [first, last) meaning the first point is included in the calculations.

If `''both''`, uses the window [first, last] meaning all points in the window are included in the calculations.

If `''neither''`, uses the window (first, last) meaning the first and last points in the window are excluded from calculations.

( ) and [ ] are referencing open and closed set notation respectively.

Default `''None''` (`''right''`).

**step** : int, default None  
Evaluate the window at every `''step''` result, equivalent to slicing as `''[:step]''`. `''window''` must be an integer. Using a step argument other than None or 1 will produce a result with a different shape than the input.

**method** : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row (`''single''`) or over the entire object (`''table''`).

This argument is only implemented when specifying `''engine='numba''` in the method call.

**Returns**  
-----  
pandas.api.typing.Window or pandas.api.typing.Rolling  
An instance of Window is returned if `''win_type''` is passed. Otherwise, an instance of Rolling is returned.

**See Also**  
-----  
expanding : Provides expanding transformations.  
ewm : Provides exponential weighted functions.

**Notes**  
-----  
See :ref:`Windowing Operations <window.generic>` for further usage details and examples.

**Examples**  
-----

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
>>> df
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

**\*\*window\*\***

Rolling sum with a window length of 2 observations.

```
>>> df.rolling(2).sum()
 B
0 NaN
1 1.0
2 3.0
3 NaN
4 NaN
```

Rolling sum with a window span of 2 seconds.

```
>>> df_time = pd.DataFrame(
```

```
... {"B": [0, 1, 2, np.nan, 4]},
... index=[
... pd.Timestamp("20130101 09:00:00"),
... pd.Timestamp("20130101 09:00:02"),
... pd.Timestamp("20130101 09:00:03"),
... pd.Timestamp("20130101 09:00:05"),
... pd.Timestamp("20130101 09:00:06"),
...],
...)
```

```
>>> df_time
 B
2013-01-01 09:00:00 0.0
2013-01-01 09:00:02 1.0
2013-01-01 09:00:03 2.0
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0
```

```
>>> df_time.rolling("2s").sum()
 B
2013-01-01 09:00:00 0.0
2013-01-01 09:00:02 1.0
2013-01-01 09:00:03 3.0
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0
```

Rolling sum with forward looking windows with 2 observations.

```
>>> indexer = pd.api.indexers.FixedForwardWindowIndexer(window_size=2)
>>> df.rolling(window=indexer, min_periods=1).sum()
 B
0 1.0
1 3.0
2 2.0
3 4.0
4 4.0
```

**\*\*min\_periods\*\***

Rolling sum with a window length of 2 observations, but only needs a minimum of 1 observation to calculate a value.

```
>>> df.rolling(2, min_periods=1).sum()
 B
0 0.0
1 1.0
2 3.0
3 2.0
4 4.0
```

**\*\*center\*\***

Rolling sum with the result assigned to the center of the window index.

```
>>> df.rolling(3, min_periods=1, center=True).sum()
 B
0 1.0
1 3.0
2 3.0
3 6.0
4 4.0
```

```
>>> df.rolling(3, min_periods=1, center=False).sum()
 B
0 0.0
1 1.0
2 3.0
3 3.0
4 6.0
```

**\*\*step\*\***

Rolling sum with a window length of 2 observations, minimum of 1 observation to calculate a value, and a step of 2.

```
>>> df.rolling(2, min_periods=1, step=2).sum()
 B
```

```
0 0.0
2 3.0
4 4.0
```

**\*\*win\_type\*\***

Rolling sum with a window length of 2, using the Scipy `'gaussian'` window type. `'std'` is required in the aggregation function.

```
>>> df.rolling(2, win_type="gaussian").sum(std=3)
 B
0 NaN
1 0.986207
2 2.958621
3 NaN
4 NaN
```

**\*\*on\*\***

Rolling sum with a window length of 2 days.

```
>>> df = pd.DataFrame(
... {
... "A": [
... pd.to_datetime("2020-01-01"),
... pd.to_datetime("2020-01-01"),
... pd.to_datetime("2020-01-02"),
...],
... "B": [1, 2, 3],
... },
... index=pd.date_range("2020", periods=3),
...)
```

```
>>> df
 A B
2020-01-01 2020-01-01 1
2020-01-02 2020-01-01 2
2020-01-03 2020-01-02 3
```

```
>>> df.rolling("2D", on="A").sum()
 A B
2020-01-01 2020-01-01 1.0
2020-01-02 2020-01-01 3.0
2020-01-03 2020-01-02 6.0
```

```
sample(
 self,
 n: int | None = None,
 frac: float | None = None,
 replace: bool = False,
 weights=None,
 random_state: RandomState | None = None,
 axis: Axis | None = None,
 ignore_index: bool = False
) -> Self
Return a random sample of items from an axis of object.
```

You can use `'random_state'` for reproducibility.

**Parameters**

```

n : int, optional
 Number of items from axis to return. Cannot be used with `frac`.
 Default = 1 if `frac` = None.
frac : float, optional
 Fraction of axis items to return. Cannot be used with `n`.
replace : bool, default False
 Allow or disallow sampling of the same row more than once.
weights : str or ndarray-like, optional
 Default 'None' results in equal probability weighting.
 If passed a Series, will align with target object on index. Index
 values in weights not found in sampled object will be ignored and
 index values in sampled object not in weights will be assigned
 weights of zero.
 If called on a DataFrame, will accept the name of a column
 when axis = 0.
 Unless weights are a Series, weights must be same length as axis
```

being sampled.  
 If weights do not sum to 1, they will be normalized to sum to 1.  
 Missing values in the weights column will be treated as zero.  
 Infinite values not allowed.  
 When `replace = False` will not allow `((n * max(weights) / sum(weights)) > 1)` in order to avoid biased results. See the Notes below for more details.  
`random_state` : int, array-like, BitGenerator, np.random.RandomState, np.random.Generator  
 If int, array-like, or BitGenerator, seed for random number generator.  
 If np.random.RandomState or np.random.Generator, use as given.  
 Default `None` results in sampling with the current state of np.random.  
`axis` : {0 or 'index', 1 or 'columns', None}, default None  
 Axis to sample. Accepts axis number or name. Default is stat axis for given data type. For `Series` this parameter is unused and defaults to `None`.  
`ignore_index` : bool, default False  
 If True, the resulting index will be labeled 0, 1, ..., n - 1.

#### Returns

Series or DataFrame  
 A new object of same type as caller containing `n` items randomly sampled from the caller object.

#### See Also

`DataFrameGroupBy.sample`: Generates random samples from each group of a DataFrame object.  
`SeriesGroupBy.sample`: Generates random samples from each group of a Series object.  
`numpy.random.choice`: Generates a random sample from a given 1-D numpy array.

#### Notes

If `frac > 1`, `replacement` should be set to `True`.

When `replace = False` will not allow `((n * max(weights) / sum(weights)) > 1)`, since that would cause results to be biased. E.g. sampling 2 items without replacement with weights `[100, 1, 1]` would yield two last items in 1/2 of cases, instead of 1/102. This is similar to specifying `n=4` without replacement on a Series with 3 elements.

#### Examples

```
>>> df = pd.DataFrame(
... {
... "num_legs": [2, 4, 8, 0],
... "num_wings": [2, 0, 0, 0],
... "num_specimen_seen": [10, 2, 1, 8],
... },
... index=["falcon", "dog", "spider", "fish"],
...)
>>> df
```

	num_legs	num_wings	num_specimen_seen
falcon	2	2	10
dog	4	0	2
spider	8	0	1
fish	0	0	8

Extract 3 random elements from the `Series` `df['num_legs']`:  
 Note that we use `random_state` to ensure the reproducibility of the examples.

```
>>> df["num_legs"].sample(n=3, random_state=1)
fish 0
spider 8
falcon 2
Name: num_legs, dtype: int64
```

A random 50% sample of the `DataFrame` with replacement:

```
>>> df.sample(frac=0.5, replace=True, random_state=1)
```

	num_legs	num_wings	num_specimen_seen
dog	4	0	2
fish	0	0	8

An upsample sample of the `DataFrame` with replacement:

Note that `replace` parameter has to be `True` for `frac` parameter `> 1`.

```
>>> df.sample(frac=2, replace=True, random_state=1)
 num_legs num_wings num_specimen_seen
dog 4 0 2
fish 0 0 8
falcon 2 2 10
falcon 2 2 10
fish 0 0 8
dog 4 0 2
fish 0 0 8
dog 4 0 2
```

Using a DataFrame column as weights. Rows with larger value in the 'num\_specimen\_seen' column are more likely to be sampled.

```
>>> df.sample(n=2, weights="num_specimen_seen", random_state=1)
 num_legs num_wings num_specimen_seen
falcon 2 2 10
fish 0 0 8
```

```
set_flags(
 self,
 *,
 copy: bool | lib.NoDefault = <no_default>,
 allows_duplicate_labels: bool | None = None
) -> Self
Return a new object with updated flags.
```

This method creates a shallow copy of the original object, preserving its underlying data while modifying its global flags. In particular, it allows you to update properties such as whether duplicate labels are permitted. This behavior is especially useful in method chains, where one wishes to adjust DataFrame or Series characteristics without altering the original object.

#### Parameters

`copy` : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the 'user guide on Copy-on-Write' [https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

`allows_duplicate_labels` : bool, optional  
Whether the returned object allows duplicate labels.

#### Returns

Series or DataFrame  
The same type as the caller.

#### See Also

`DataFrame.attrs` : Global metadata applying to this dataset.  
`DataFrame.flags` : Global flags applying to this object.

#### Notes

This method returns a new object that's a view on the same data as the input. Mutating the input or the output values will be reflected in the other.

This method is intended to be used in method chains.

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in `:attr:`DataFrame.attrs``.

#### Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags.allows_duplicate_labels
```

```

True
>>> df2 = df.set_flags(allows_duplicate_labels=False)
>>> df2.flags.allows_duplicate_labels
False

```

shift(  
self,  
periods: int | Sequence[int] = 1,  
freq=None,  
axis: Axis = 0,  
fill\_value: Hashable = <no\_default>,  
suffix: str | None = None  
) -> Self | DataFrame

Shift index by desired number of periods with an optional time `freq`.

When `freq` is not passed, shift the index without realigning the data. If `freq` is passed (in this case, the index must be date or datetime, or it will raise a `NotImplementedError`), the index will be increased using the periods and the `freq`. `freq` can be inferred when specified as "infer" as long as either freq or inferred\_freq attribute is set in the index.

Parameters  
-----

periods : int or Sequence  
Number of periods to shift. Can be positive or negative. If an iterable of ints, the data will be shifted once by each int. This is equivalent to shifting by one value at a time and concatenating all resulting frames. The resulting columns will have the shift suffixed to their column names. For multiple periods, axis must not be 1.

freq : DateOffset, tseries.offsets, timedelta, or str, optional  
Offset to use from the tseries module or time rule (e.g. 'EOM'). If `freq` is specified then the index values are shifted but the data is not realigned. That is, use `freq` if you would like to extend the index when shifting and preserve the original data. If `freq` is specified as "infer" then it will be inferred from the freq or inferred\_freq attributes of the index. If neither of those attributes exist, a ValueError is thrown.

axis : {{0 or 'index', 1 or 'columns', None}}, default None  
Shift direction. For `Series` this parameter is unused and defaults to 0.

fill\_value : object, optional  
The scalar value to use for newly introduced missing values. the default depends on the dtype of `self`. For Boolean and numeric NumPy data types, ``np.nan`` is used. For datetime, timedelta, or period data, etc. :attr:`NaT` is used. For extension dtypes, ``self.dtype.na\_value`` is used.

suffix : str, optional  
If str and periods is an iterable, this is added after the column name and before the shift value for each shifted column name. For `Series` this parameter is unused and defaults to `None`.

Returns  
-----

Series/DataFrame  
Copy of input object, shifted.

See Also  
-----

Index.shift : Shift values of Index.  
DatetimeIndex.shift : Shift values of DatetimeIndex.  
PeriodIndex.shift : Shift values of PeriodIndex.

Examples  
-----

```

>>> df = pd.DataFrame(
... [[10, 13, 17], [20, 23, 27], [15, 18, 22], [30, 33, 37], [45, 48, 52]],
... columns=["Col1", "Col2", "Col3"],
... index=pd.date_range("2020-01-01", "2020-01-05"),
...)
>>> df

```

	Col1	Col2	Col3
2020-01-01	10	13	17
2020-01-02	20	23	27
2020-01-03	15	18	22
2020-01-04	30	33	37
2020-01-05	45	48	52

```

>>> df.shift(periods=3)
 Col1 Col2 Col3
2020-01-01 NaN NaN NaN
2020-01-02 NaN NaN NaN
2020-01-03 NaN NaN NaN
2020-01-04 10.0 13.0 17.0
2020-01-05 20.0 23.0 27.0

>>> df.shift(periods=1, axis="columns")
 Col1 Col2 Col3
2020-01-01 NaN 10 13
2020-01-02 NaN 20 23
2020-01-03 NaN 15 18
2020-01-04 NaN 30 33
2020-01-05 NaN 45 48

>>> df.shift(periods=3, fill_value=0)
 Col1 Col2 Col3
2020-01-01 0 0 0
2020-01-02 0 0 0
2020-01-03 0 0 0
2020-01-04 10 13 17
2020-01-05 20 23 27

>>> df.shift(periods=3, freq="D")
 Col1 Col2 Col3
2020-01-04 10 13 17
2020-01-05 20 23 27
2020-01-06 15 18 22
2020-01-07 30 33 37
2020-01-08 45 48 52

>>> df.shift(periods=3, freq="infer")
 Col1 Col2 Col3
2020-01-04 10 13 17
2020-01-05 20 23 27
2020-01-06 15 18 22
2020-01-07 30 33 37
2020-01-08 45 48 52

>>> df["Col1"].shift(periods=[0, 1, 2])
 Col1_0 Col1_1 Col1_2
2020-01-01 10 NaN NaN
2020-01-02 20 10.0 NaN
2020-01-03 15 20.0 10.0
2020-01-04 30 15.0 20.0
2020-01-05 45 30.0 15.0

```

`squeeze(self, axis: Axis | None = None) -> Scalar | Series | DataFrame`  
 Squeeze 1 dimensional axis objects into scalars.

Series or DataFrames with a single element are squeezed to a scalar.  
 DataFrames with a single column or a single row are squeezed to a Series. Otherwise the object is unchanged.

This method is most useful when you don't know if your object is a Series or DataFrame, but you do know it has just a single column. In that case you can safely call ``squeeze`` to ensure you have a Series.

#### Parameters

`axis` : {0 or 'index', 1 or 'columns', None}, default None  
 A specific axis to squeeze. By default, all length-1 axes are squeezed. For ``Series`` this parameter is unused and defaults to ``None``.

#### Returns

DataFrame, Series, or scalar  
 The projection after squeezing ``axis`` or all the axes.

#### See Also

`Series.iloc` : Integer-location based indexing for selecting scalars.  
`DataFrame.iloc` : Integer-location based indexing for selecting Series.  
`Series.to_frame` : Inverse of `DataFrame.squeeze` for a

single-column DataFrame.

#### Examples

```
>>> primes = pd.Series([2, 3, 5, 7])
```

Slicing might produce a Series with a single value:

```
>>> even_primes = primes[primes % 2 == 0]
>>> even_primes
0 2
dtype: int64
```

```
>>> even_primes.squeeze()
np.int64(2)
```

Squeezing objects with more than one value in every axis does nothing:

```
>>> odd_primes = primes[primes % 2 == 1]
>>> odd_primes
1 3
2 5
3 7
dtype: int64
```

```
>>> odd_primes.squeeze()
1 3
2 5
3 7
dtype: int64
```

Squeezing is even more effective when used with DataFrames.

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["a", "b"])
>>> df
 a b
0 1 2
1 3 4
```

Slicing a single column will produce a DataFrame with the columns having only one value:

```
>>> df_a = df[["a"]]
>>> df_a
 a
0 1
1 3
```

So the columns can be squeezed down, resulting in a Series:

```
>>> df_a.squeeze("columns")
0 1
1 3
Name: a, dtype: int64
```

Slicing a single row from a single column will produce a single scalar DataFrame:

```
>>> df_0a = df.loc[df.index < 1, ["a"]]
>>> df_0a
 a
0 1
```

Squeezing the rows produces a single scalar Series:

```
>>> df_0a.squeeze("rows")
a 1
Name: 0, dtype: int64
```

Squeezing all axes will project directly into a scalar:

```
>>> df_0a.squeeze()
np.int64(1)
```

```
tail(self, n: int = 5) -> Self
Return the last `n` rows.
```



This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to `df[|n|:]`.

If `n` is larger than the number of rows, this function returns all rows.

#### Parameters

`n` : int, default 5  
Number of rows to select.

#### Returns

type of caller  
The last `n` rows of the caller object.

#### See Also

`DataFrame.head` : The first `n` rows of the caller object.

#### Examples

```
>>> df = pd.DataFrame(
... {
... "animal": [
... "alligator",
... "bee",
... "falcon",
... "lion",
... "monkey",
... "parrot",
... "shark",
... "whale",
... "zebra",
...]
... }
...)
>>> df
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
5 parrot
6 shark
7 whale
8 zebra
```

Viewing the last 5 lines

```
>>> df.tail()
 animal
4 monkey
5 parrot
6 shark
7 whale
8 zebra
```

Viewing the last `n` lines (three in this case)

```
>>> df.tail(3)
 animal
6 shark
7 whale
8 zebra
```

For negative values of `n`

```
>>> df.tail(-3)
 animal
3 lion
4 monkey
5 parrot
```

```

6 shark
7 whale
8 zebra

```

```

take(self, indices, axis: Axis = 0, **kwargs) -> Self
Return the elements in the given *positional* indices along an axis.

```

This means that we are not indexing according to actual values in the index attribute of the object. We are indexing according to the actual position of the element in the object.

Parameters  
-----

indices : array-like  
An array of ints indicating which positions to take.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
The axis on which to select elements. ``0`` means that we are selecting rows, ``1`` means that we are selecting columns.  
For `Series` this parameter is unused and defaults to 0.  
\*\*kwargs  
For compatibility with :meth:`numpy.take`. Has no effect on the output.

Returns  
-----

same type as caller  
An array-like containing the elements taken from the object.

See Also  
-----

DataFrame.loc : Select a subset of a DataFrame by labels.  
DataFrame.iloc : Select a subset of a DataFrame by positions.  
numpy.take : Take elements from an array along an axis.

Examples  
-----

```

>>> df = pd.DataFrame(
... [
... ("falcon", "bird", 389.0),
... ("parrot", "bird", 24.0),
... ("lion", "mammal", 80.5),
... ("monkey", "mammal", np.nan),
...],
... columns=["name", "class", "max_speed"],
... index=[0, 2, 3, 1],
...)
>>> df
 name class max_speed
0 falcon bird 389.0
2 parrot bird 24.0
3 lion mammal 80.5
1 monkey mammal NaN

```

Take elements at positions 0 and 3 along the axis 0 (default).

Note how the actual indices selected (0 and 1) do not correspond to our selected indices 0 and 3. That's because we are selecting the 0th and 3rd rows, not rows whose indices equal 0 and 3.

```

>>> df.take([0, 3])
 name class max_speed
0 falcon bird 389.0
1 monkey mammal NaN

```

Take elements at indices 1 and 2 along the axis 1 (column selection).

```

>>> df.take([1, 2], axis=1)
 class max_speed
0 bird 389.0
2 bird 24.0
3 mammal 80.5
1 mammal NaN

```

We may take elements using negative integers for positive indices, starting from the end of the object, just like with Python lists.

```

>>> df.take([-1, -2])

```

	name	class	max_speed
1	monkey	mammal	NaN
3	lion	mammal	80.5

`to_clipboard(self, *, excel: bool = True, sep: str | None = None, **kwargs) -> None`  
Copy object to the system clipboard.

Write a text representation of object to the system clipboard.  
This can be pasted into Excel, for example.

#### Parameters

-----

`excel : bool, default True`

Produce output in a csv format for easy pasting into excel.

- True, use the provided separator for csv pasting.
- False, write a string representation of the object to the clipboard.

`sep : str, default ``\t```

Field delimiter.

`**kwargs`

These parameters will be passed to `DataFrame.to_csv`.

#### See Also

-----

`DataFrame.to_csv` : Write a `DataFrame` to a comma-separated values (csv) file.

`read_clipboard` : Read text from clipboard and pass to `read_csv`.

#### Notes

-----

Requirements for your platform.

- Linux : ``xclip``, or ``xsel`` (with ``PyQt4`` modules)
- Windows : none
- macOS : none

This method uses the processes developed for the package ``pyperclip``. A solution to render any output string format is given in the examples.

#### Examples

-----

Copy the contents of a `DataFrame` to the clipboard.

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])
```

```
>>> df.to_clipboard(sep=",") # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # ,A,B,C
... # 0,1,2,3
... # 1,4,5,6
```

We can omit the index by passing the keyword ``index`` and setting it to false.

```
>>> df.to_clipboard(sep=",", index=False) # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # A,B,C
... # 1,2,3
... # 4,5,6
```

Using the original ``pyperclip`` package for any string output format.

```
.. code-block:: python
```

```
import pyperclip

html = df.style.to_html()
pyperclip.copy(html)
```

```
to_csv(
 self,
 path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
 *,
 sep: str = ',',
 na_rep: str = '',
 float_format: str | Callable | None = None,
```

```

columns: Sequence[Hashable] | None = None,
header: bool | list[str] = True,
index: bool = True,
index_label: IndexLabel | None = None,
mode: str = 'w',
encoding: str | None = None,
compression: CompressionOptions = 'infer',
quoting: int | None = None,
quotechar: str = '"',
lineterminator: str | None = None,
chunksize: int | None = None,
date_format: str | None = None,
doublequote: bool = True,
escapechar: str | None = None,
decimal: str = '.',
errors: OpenFileErrors = 'strict',
storage_options: StorageOptions | None = None
) -> str | None
 Write object to a comma-separated values (csv) file.

Parameters

path_or_buf : str, path object, file-like object, or None, default None
 String, path object (implementing os.PathLike[str]), or file-like
 object implementing a write() function. If None, the result is
 returned as a string. If a non-binary file object is passed, it should
 be opened with `newline=''`, disabling universal newlines. If a binary
 file object is passed, `mode` might need to contain a `b`.
sep : str, default ','
 String of length 1. Field delimiter for the output file.
na_rep : str, default ''
 Missing data representation.
float_format : str, Callable, default None
 Format string for floating point numbers. If a Callable is given, it takes
 precedence over other numeric formatting parameters, like decimal.
columns : sequence, optional
 Columns to write.
header : bool or list of str, default True
 Write out the column names. If a list of strings is given it is
 assumed to be aliases for the column names.
index : bool, default True
 Write row names (index).
index_label : str or sequence, or False, default None
 Column label for index column(s) if desired. If None is given, and
 `header` and `index` are True, then the index names are used. A
 sequence should be given if the object uses MultiIndex. If
 False do not print fields for index names. Use index_label=False
 for easier importing in R.
mode : {'w', 'x', 'a'}, default 'w'
 Forwarded to either `open(mode=)` or `fsspec.open(mode=)` to control
 the file opening. Typical values include:

 - 'w', truncate the file first.
 - 'x', exclusive creation, failing if the file already exists.
 - 'a', append to the end of file if it exists.

encoding : str, optional
 A string representing the encoding to use in the output file,
 defaults to 'utf-8'. `encoding` is not supported if `path_or_buf`
 is a non-binary file object.

compression : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and
 'path_or_buf' is path-like, then detect compression from the following
 extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 Set to `None` for no compression.
 Can also be a dict with key `method` set to one of
 {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
 other key-value pairs are forwarded to
 `zipfile.ZipFile`, `gzip.GzipFile`,
 `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or
 `tarfile.TarFile`, respectively.
 As an example, the following could be passed for faster compression and
 to create a reproducible gzip archive:
 `compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}`.

```

May be a dict with key 'method' as compression mode and other entries as additional compression options if compression mode is 'zip'.

Passing compression options as keys in dict is supported for compression modes 'gzip', 'bz2', 'zstd', and 'zip'.  
quoting : optional constant from csv module  
Defaults to csv.QUOTE\_MINIMAL. If you have set a 'float\_format' then floats are converted to strings and thus csv.QUOTE\_NONNUMERIC will treat them as non-numeric.  
quotechar : str, default '\"'  
String of length 1. Character used to quote fields.  
lineterminator : str, optional  
The newline character or character sequence to use in the output file. Defaults to 'os.linesep', which depends on the OS in which this method is called ('\\n' for linux, '\\r\\n' for Windows, i.e.).  
chunksize : int or None  
Rows to write at a time.  
date\_format : str, default None  
Format string for datetime objects.  
doublequote : bool, default True  
Control quoting of 'quotechar' inside a field.  
escapechar : str, default None  
String of length 1. Character used to escape 'sep' and 'quotechar' when appropriate.  
decimal : str, default '.'  
Character recognized as decimal separator. E.g. use ',' for European data.  
errors : str, default 'strict'  
Specifies how encoding and decoding errors are to be handled. See the errors argument for 'func:open' for a full list of options.  
storage\_options : dict, optional  
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to 'urllib.request.Request' as header options. For other URLs (e.g. starting with 's3://', and 'gcs://') the key-value pairs are forwarded to 'fsspec.open'. Please see 'fsspec' and 'urllib' for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>_.`

#### Returns

-----  
None or str

If path\_or\_buf is None, returns the resulting csv format as a string. Otherwise returns None.

#### See Also

-----  
read\_csv : Load a CSV file into a DataFrame.  
to\_excel : Write DataFrame to an Excel file.

#### Examples

-----  
Create 'out.csv' containing 'df' without indices

```
>>> df = pd.DataFrame(
... [["Raphael", "red", "sai"], ["Donatello", "purple", "bo staff"]],
... columns=["name", "mask", "weapon"],
...)
>>> df.to_csv("out.csv", index=False) # doctest: +SKIP
```

Create 'out.zip' containing 'out.csv'

```
>>> df.to_csv(index=False)
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
>>> compression_opts = dict(
... method="zip", archive_name="out.csv"
...) # doctest: +SKIP
>>> df.to_csv(
... "out.zip", index=False, compression=compression_opts
...) # doctest: +SKIP
```

To write a csv file to a new folder or nested folder you will first

need to create it using either Pathlib or os:

```
>>> from pathlib import Path # doctest: +SKIP
>>> filepath = Path("folder/subfolder/out.csv") # doctest: +SKIP
>>> filepath.parent.mkdir(parents=True, exist_ok=True) # doctest: +SKIP
>>> df.to_csv(filepath) # doctest: +SKIP
```

```
>>> import os # doctest: +SKIP
>>> os.makedirs("folder/subfolder", exist_ok=True) # doctest: +SKIP
>>> df.to_csv("folder/subfolder/out.csv") # doctest: +SKIP
```

Format floats to two decimal places:

```
>>> df.to_csv("out1.csv", float_format="%.2f") # doctest: +SKIP
```

Format floats using scientific notation:

```
>>> df.to_csv("out2.csv", float_format="{:.2e}").format) # doctest: +SKIP
```

```
to_excel(
 self,
 excel_writer: FilePath | WriteExcelBuffer | ExcelWriter,
 *,
 sheet_name: str = 'Sheet1',
 na_rep: str = '',
 float_format: str | None = None,
 columns: Sequence[Hashable] | None = None,
 header: Sequence[Hashable] | bool = True,
 index: bool = True,
 index_label: IndexLabel | None = None,
 startrow: int = 0,
 startcol: int = 0,
 engine: Literal['openpyxl', 'xlsxwriter'] | None = None,
 merge_cells: bool = True,
 inf_rep: str = 'inf',
 freeze_panes: tuple[int, int] | None = None,
 storage_options: StorageOptions | None = None,
 engine_kwargs: dict[str, Any] | None = None,
 autofilter: bool = False
) -> None
 Write object to an Excel sheet.
```

To write a single object to an Excel .xlsx file it is only necessary to specify a target file name. To write to multiple sheets it is necessary to create an `ExcelWriter` object with a target file name, and specify a sheet in the file to write to.

Multiple sheets may be written to by specifying unique `sheet_name`. With all data written to the file it is necessary to save the changes. Note that creating an `ExcelWriter` object with a file name that already exists will overwrite the existing file because the default mode is write.

#### Parameters

```

excel_writer : path-like, file-like, or ExcelWriter object
 File path or existing ExcelWriter.
sheet_name : str, default 'Sheet1'
 Name of sheet which will contain DataFrame.
na_rep : str, default ''
 Missing data representation.
float_format : str, optional
 Format string for floating point numbers. For example
 ``float_format="%.2f"`` will format 0.1234 to 0.12.
columns : sequence or list of str, optional
 Columns to write.
header : bool or list of str, default True
 Write out the column names. If a list of string is given it is
 assumed to be aliases for the column names.
index : bool, default True
 Write row names (index).
index_label : str or sequence, optional
 Column label for index column(s) if desired. If not specified, and
 `header` and `index` are True, then the index names are used. A
 sequence should be given if the DataFrame uses MultiIndex.
startrow : int, default 0
 Upper left cell row to dump data frame.
startcol : int, default 0
```

Upper left cell column to dump data frame.

`engine` : str, optional  
 Write engine to use, 'openpyxl' or 'xlsxwriter'. You can also set this via the options ``io.excel.xlsx.writer`` or ``io.excel.xlsm.writer``.

`merge_cells` : bool or 'columns', default False  
 If True, write MultiIndex index and columns as merged cells.  
 If 'columns', merge MultiIndex column cells only.

`inf_rep` : str, default 'inf'  
 Representation for infinity (there is no native representation for infinity in Excel).

`freeze_panes` : tuple of int (length 2), optional  
 Specifies the one-based bottommost row and rightmost column that is to be frozen.

`storage_options` : dict, optional  
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`engine_kwargs` : dict, optional  
 Arbitrary keyword arguments passed to excel engine.

`autofilter` : bool, default False  
 If True, add automatic filters to all columns.

#### See Also

-----

`to_csv` : Write DataFrame to a comma-separated values (csv) file.  
`ExcelWriter` : Class for writing DataFrame objects into excel sheets.  
`read_excel` : Read an Excel file into a pandas DataFrame.  
`read_csv` : Read a comma-separated values (csv) file into DataFrame.  
`io.formats.style.Styler.to_excel` : Add styles to Excel sheet.

#### Notes

-----

For compatibility with :meth:`~DataFrame.to\_csv`,  
`to_excel` serializes lists and dicts to strings before writing.

Once a workbook has been saved it is not possible to write further data without rewriting the whole workbook.

pandas will check the number of rows, columns,  
 and cell character count does not exceed Excel's limitations.  
 All other limitations must be checked by the user.

#### Examples

-----

Create, write to and save a workbook:

```
>>> df1 = pd.DataFrame(
... [
... ["a", "b"], ["c", "d"]],
... index=["row 1", "row 2"],
... columns=["col 1", "col 2"],
...)
>>> df1.to_excel("output.xlsx") # doctest: +SKIP
```

To specify the sheet name:

```
>>> df1.to_excel("output.xlsx", sheet_name="Sheet_name_1") # doctest: +SKIP
```

If you wish to write to more than one sheet in the workbook, it is necessary to specify an `ExcelWriter` object:

```
>>> df2 = df1.copy()
>>> with pd.ExcelWriter("output.xlsx") as writer: # doctest: +SKIP
... df1.to_excel(writer, sheet_name="Sheet_name_1")
... df2.to_excel(writer, sheet_name="Sheet_name_2")
```

`ExcelWriter` can also be used to append to an existing Excel file:

```
>>> with pd.ExcelWriter("output.xlsx", mode="a") as writer: # doctest: +SKIP
... df1.to_excel(writer, sheet_name="Sheet_name_3")
```

To set the library that is used to write the Excel file, you can pass the `engine` keyword (the default engine is automatically chosen depending on the file extension):

```
>>> df1.to_excel("output1.xlsx", engine="xlsxwriter") # doctest: +SKIP
```

```
to_hdf(
 self,
 path_or_buf: FilePath | HDFStore,
 *,
 key: str,
 mode: Literal['a', 'w', 'r+'] = 'a',
 complevel: int | None = None,
 complib: Literal['zlib', 'lzo', 'bzip2', 'blosc'] | None = None,
 append: bool = False,
 format: Literal['fixed', 'table'] | None = None,
 index: bool = True,
 min_itemsize: int | dict[str, int] | None = None,
 nan_rep=None,
 dropna: bool | None = None,
 data_columns: Literal[True] | list[str] | None = None,
 errors: OpenFileErrors = 'strict',
 encoding: str = 'UTF-8'
) -> None
Write the contained data to an HDF5 file using HDFStore.

Hierarchical Data Format (HDF) is self-describing, allowing an
application to interpret the structure and contents of a file with
no outside information. One HDF file can hold a mix of related objects
which can be accessed as a group or as individual objects.
```

In order to add another DataFrame or Series to an existing HDF file please use append mode and a different a key.

.. warning::

One can store a subclass of ``DataFrame`` or ``Series`` to HDF5, but the type of the subclass is lost upon storing.

For more information see the :ref:`user guide <io.hdf5>`.

#### Parameters

-----

**path\_or\_buf** : str or pandas.HDFStore  
File path or HDFStore object.

**key** : str  
Identifier for the group in the store.

**mode** : {'a', 'w', 'r+'}, default 'a'  
Mode to open file:

- 'w': write, a new file is created (an existing file with the same name would be deleted).
- 'a': append, an existing file is opened for reading and writing, and if the file does not exist it is created.
- 'r+': similar to 'a', but the file must already exist.

**complevel** : {0-9}, default None  
Specifies a compression level for data.  
A value of 0 or None disables compression.

**complib** : {'zlib', 'lzo', 'bzip2', 'blosc'}, default 'zlib'  
Specifies the compression library to be used.  
These additional compressors for Blosc are supported (default if no compressor specified: 'blosc:blosclz'):  
{'blosc:blosclz', 'blosc:lz4', 'blosc:lz4hc', 'blosc:snappy', 'blosc:zlib', 'blosc:zstd'}.  
Specifying a compression library which is not available issues a ValueError.

**append** : bool, default False  
For Table formats, append the input data to the existing.

**format** : {'fixed', 'table', None}, default 'fixed'  
Possible values:

- 'fixed': Fixed format. Fast writing/reading. Not-appendable, nor searchable.
- 'table': Table format. Write as a PyTables Table structure which may perform worse but allow more flexible operations like searching / selecting subsets of the data.



- If None, `pd.get_option('io.hdf.default_format')` is checked, followed by fallback to "fixed".

`index` : bool, default True  
 Write DataFrame index as a column.

`min_itemsize` : dict or int, optional  
 Map column names to minimum string sizes for columns.

`nan_rep` : Any, optional  
 How to represent null values as str.  
 Not allowed with `append=True`.

`dropna` : bool, default False, optional  
 Remove missing values.

`data_columns` : list of columns or True, optional  
 List of columns to create as indexed data columns for on-disk queries, or True to use all columns. By default only the axes of the object are indexed. See `:ref:`Query via data columns<io.hdf5-query-data-columns>`` for more information.  
 Applicable only to `format='table'`.

`errors` : str, default 'strict'  
 Specifies how encoding and decoding errors are to be handled. See the `errors` argument for `:func:`open`` for a full list of options.

`encoding` : str, default "UTF-8"  
 Set character encoding.

See Also

-----

`read_hdf` : Read from HDF file.  
`DataFrame.to_orc` : Write a DataFrame to the binary orc format.  
`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.  
`DataFrame.to_sql` : Write to a SQL table.  
`DataFrame.to_feather` : Write out feather-format for DataFrames.  
`DataFrame.to_csv` : Write out to a csv file.

Examples

-----

```

>>> df = pd.DataFrame(
... {"A": [1, 2, 3], "B": [4, 5, 6]}, index=["a", "b", "c"]
...) # doctest: +SKIP
>>> df.to_hdf("data.h5", key="df", mode="w") # doctest: +SKIP

```

We can add another object to the same file:

```

>>> s = pd.Series([1, 2, 3, 4]) # doctest: +SKIP
>>> s.to_hdf("data.h5", key="s") # doctest: +SKIP

```

Reading from HDF file:

```

>>> pd.read_hdf("data.h5", "df") # doctest: +SKIP
A B
a 1 4
b 2 5
c 3 6
>>> pd.read_hdf("data.h5", "s") # doctest: +SKIP
0 1
1 2
2 3
3 4
dtype: int64

```

```

to_json(
 self,
 path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
 *,
 orient: Literal['split', 'records', 'index', 'table', 'columns', 'values'] | None = None,
 date_format: str | None = None,
 double_precision: int = 10,
 force_ascii: bool = True,
 date_unit: TimeUnit = 'ms',
 default_handler: Callable[[Any], JSONSerializable] | None = None,
 lines: bool = False,
 compression: CompressionOptions = 'infer',
 index: bool | None = None,
 indent: int | None = None,
 storage_options: StorageOptions | None = None,
 mode: Literal['a', 'w'] = 'w'
) -> str | None

```

Convert the object to a JSON string.

Note NaN's and None will be converted to null and datetime objects will be converted to UNIX timestamps.

#### Parameters

`path_or_buf` : str, path object, file-like object, or None, default None  
String, path object (implementing `os.PathLike[str]`), or file-like object implementing a `write()` function. If None, the result is returned as a string.

`orient` : str  
Indication of expected JSON string format.

##### \* Series:

- default is 'index'
- allowed values are: {'split', 'records', 'index', 'table'}.

##### \* DataFrame:

- default is 'columns'
- allowed values are: {'split', 'records', 'index', 'columns', 'values', 'table'}.

##### \* The format of the JSON string:

- 'split' : dict like {'index' -> [index], 'columns' -> [columns], 'data' -> [values]}
- 'records' : list like [{column -> value}, ... , {column -> value}]
- 'index' : dict like {'index' -> {column -> value}}
- 'columns' : dict like {'column' -> {index -> value}}
- 'values' : just the values array
- 'table' : dict like {'schema': {schema}, 'data': {data}}

Describing the data, where data component is like ``orient='records'``.

`date_format` : {None, 'epoch', 'iso'}

Type of date conversion. 'epoch' = epoch milliseconds, 'iso' = ISO8601. The default depends on the `orient`. For `orient='table'`, the default is 'iso'. For all other orients, the default is 'epoch'.

.. deprecated:: 3.0.0

'epoch' date format is deprecated and will be removed in a future version, please use 'iso' instead.

`double_precision` : int, default 10

The number of decimal places to use when encoding floating point values. The possible maximal value is 15. Passing `double_precision` greater than 15 will raise a `ValueError`.

`force_ascii` : bool, default True

Force encoded string to be ASCII.

`date_unit` : str, default 'ms' (milliseconds)

The time unit to encode to, governs timestamp and ISO8601 precision. One of 's', 'ms', 'us', 'ns' for second, millisecond, microsecond, and nanosecond respectively.

`default_handler` : callable, default None

Handler to call if object cannot otherwise be converted to a suitable format for JSON. Should receive a single argument which is the object to convert and return a serialisable object.

`lines` : bool, default False

If `orient` is 'records' write out line-delimited json format. Will throw `ValueError` if incorrect `orient` since others are not list-like.

`compression` : str or dict, default 'infer'

For on-the-fly compression of the output data. If 'infer' and `path_or_buf` is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). Set to `None` for no compression.

Can also be a dict with key `method` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`,

```bz2.BZ2File```, ```zstandard.ZstdCompressor```, ```lzma.LZMAFile``` or ```tarfile.TarFile```, respectively.

As an example, the following could be passed for faster compression and to create a reproducible gzip archive:

```compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}```.

`index` : bool or None, default None

The index is only used when 'orient' is 'split', 'index', 'column', or 'table'. Of these, 'index' and 'column' do not support

`index=False`. The string 'index' as a column name with empty `:class:`Index`` or if it is 'index' will raise a ```ValueError```.

`indent` : int, optional

Length of whitespace used to indent each record.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ```urllib.request.Request``` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ```fsspec.open```. Please see ```fsspec``` and ```urllib``` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`mode` : str, default 'w' (writing)

Specify the IO mode for output when supplying a `path_or_buf`.

Accepted args are 'w' (writing) and 'a' (append) only.

`mode='a'` is only supported when `lines` is True and `orient` is 'records'.

Returns

-----

None or str

If `path_or_buf` is None, returns the resulting json format as a string. Otherwise returns None.

See Also

-----

`read_json` : Convert a JSON string to pandas object.

Notes

-----

The behavior of ```indent=0``` varies from the `stdlib`, which does not indent the output but does insert newlines. Currently, ```indent=0``` and the default ```indent=None``` are equivalent in pandas, though this may change in a future release.

```orient='table'``` contains a 'pandas\_version' field under 'schema'.

This stores the version of 'pandas' used in the latest revision of the schema.

Examples

```
>>> from json import loads, dumps
```

```
>>> df = pd.DataFrame(
```

```
...     [{"a", "b"}, {"c", "d"}],
```

```
...     index=["row 1", "row 2"],
```

```
...     columns=["col 1", "col 2"],
```

```
... )
```

```
>>> result = df.to_json(orient="split")
```

```
>>> parsed = loads(result)
```

```
>>> dumps(parsed, indent=4) # doctest: +SKIP
```

```
{{
```

```
    "columns": [
        "col 1",
        "col 2"
```

```
    ],
```

```
    "index": [
        "row 1",
        "row 2"
```

```
    ],
```

```
    "data": [
```

```
        [
            "a",
            "b"
```

```
        ],
```

```

        [
            "c",
            "d"
        ]
    ]
}
}

```

Encoding/decoding a Dataframe using ``'records'`` formatted JSON.
Note that index labels are not preserved with this encoding.

```

>>> result = df.to_json(orient="records")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
    {
        "col 1": "a",
        "col 2": "b"
    },
    {
        "col 1": "c",
        "col 2": "d"
    }
]

```

Encoding/decoding a Dataframe using ``'index'`` formatted JSON:

```

>>> result = df.to_json(orient="index")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{
    "row 1": {
        "col 1": "a",
        "col 2": "b"
    },
    "row 2": {
        "col 1": "c",
        "col 2": "d"
    }
}

```

Encoding/decoding a Dataframe using ``'columns'`` formatted JSON:

```

>>> result = df.to_json(orient="columns")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{
    "col 1": {
        "row 1": "a",
        "row 2": "c"
    },
    "col 2": {
        "row 1": "b",
        "row 2": "d"
    }
}

```

Encoding/decoding a Dataframe using ``'values'`` formatted JSON:

```

>>> result = df.to_json(orient="values")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
    [
        "a",
        "b"
    ],
    [
        "c",
        "d"
    ]
]

```

Encoding with Table Schema:

```

>>> result = df.to_json(orient="table")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP

```

```

{{
  "schema": {{
    "fields": [
      {{
        "name": "index",
        "type": "string"
      }},
      {{
        "name": "col 1",
        "type": "string"
      }},
      {{
        "name": "col 2",
        "type": "string"
      }}
    ],
    "primaryKey": [
      "index"
    ],
    "pandas_version": "1.4.0"
  }},
  "data": [
    {{
      "index": "row 1",
      "col 1": "a",
      "col 2": "b"
    }},
    {{
      "index": "row 2",
      "col 1": "c",
      "col 2": "d"
    }}
  ]
}}

```

```

to_latex(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    columns: Sequence[Hashable] | None = None,
    header: bool | SequenceNotStr[str] = True,
    index: bool = True,
    na_rep: str = 'NaN',
    formatters: FormattersType | None = None,
    float_format: FloatFormatType | None = None,
    sparsify: bool | None = None,
    index_names: bool = True,
    bold_rows: bool = False,
    column_format: str | None = None,
    longtable: bool | None = None,
    escape: bool | None = None,
    encoding: str | None = None,
    decimal: str = '.',
    multicolumn: bool | None = None,
    multicolumn_format: str | None = None,
    multirow: bool | None = None,
    caption: str | tuple[str, str] | None = None,
    label: str | None = None,
    position: str | None = None
) -> str | None

```

Render object to a LaTeX tabular, longtable, or nested table.

Requires ``\usepackage{booktabs}``. The output can be copy/pasted into a main LaTeX document or read from an external file with ``\input{table.tex}``.

.. versionchanged:: 2.0.0

Refactored to use the Styler implementation via jinja2 templating.

Parameters

buf : str, Path or StringIO-like, optional, default None
 Buffer to write to. If None, the output is returned as a string.

columns : list of label, optional
 The subset of columns to write. Writes all columns by default.

header : bool or list of str, default True
 Write out the column names. If a list of strings is given,

it is assumed to be aliases for the column names. Braces must be escaped.

`index` : bool, default True
Write row names (index).

`na_rep` : str, default 'NaN'
Missing data representation.

`formatters` : list of functions or dict of {str: function}, optional
Formatter functions to apply to columns' elements by position or name. The result of each function must be a unicode string. List must be of length equal to the number of columns.

`float_format` : one-parameter function or str, optional, default None
Formatter for floating point numbers. For example
``float\_format="%.2f"`` and ``float\_format="{:0.2f}"`` will both result in 0.1234 being formatted as 0.12.

`sparsify` : bool, optional
Set to False for a DataFrame with a hierarchical index to print every multiindex key at each row. By default, the value will be read from the config module.

`index_names` : bool, default True
Prints the names of the indexes.

`bold_rows` : bool, default False
Make the row labels bold in the output.

`column_format` : str, optional
The columns format as specified in `LaTeX table format` <<https://en.wikibooks.org/wiki/LaTeX/Tables>>__ e.g. 'rcl' for 3 columns. By default, 'l' will be used for all columns except columns of numbers, which default to 'r'.

`longtable` : bool, optional
Use a longtable environment instead of tabular. Requires adding a `\usepackage{longtable}` to your LaTeX preamble. By default, the value will be read from the pandas config module, and set to `True` if the option ``styler.latex.environment`` is ``"longtable"``.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed.

`escape` : bool, optional
By default, the value will be read from the pandas config module and set to `True` if the option ``styler.format.escape`` is ``"latex"``. When set to False prevents from escaping latex special characters in column names.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to `False`.

`encoding` : str, optional
A string representing the encoding to use in the output file, defaults to 'utf-8'.

`decimal` : str, default '.'
Character recognized as decimal separator, e.g. ',' in Europe.

`multicolumn` : bool, default True
Use `\multicolumn` to enhance MultiIndex columns. The default will be read from the config module, and is set as the option ``styler.sparse.columns``.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed.

`multicolumn_format` : str, default 'r'
The alignment for multicolumns, similar to `column_format`. The default will be read from the config module, and is set as the option ``styler.latex.multicol\_align``.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to "r".

`multirow` : bool, default True
Use `\multirow` to enhance MultiIndex rows. Requires adding a `\usepackage{multirow}` to your LaTeX preamble. Will print centered labels (instead of top-aligned) across the contained rows, separating groups via clines. The default will be read from the pandas config module, and is set as the option ``styler.sparse.index``.

.. versionchanged:: 2.0.0
The pandas option affecting this argument has changed, as has the default value to `True`.

`caption` : str or tuple, optional
Tuple (full_caption, short_caption),

which results in `\\caption[short_caption]{full_caption}`;
 if a single string is passed, no short caption will be set.
 label : str, optional
 The LaTeX label to be placed inside `\\label{}` in the output.
 This is used with `\\ref{}` in the main `\\.tex` file.

position : str, optional
 The LaTeX positional argument for tables, to be placed after
`\\begin{}` in the output.

Returns

 str or None
 If buf is None, returns the result as a string. Otherwise returns None.

See Also

 io.formats.style.Styler.to_latex : Render a DataFrame to LaTeX
 with conditional formatting.
 DataFrame.to_string : Render a DataFrame to a console-friendly
 tabular output.
 DataFrame.to_html : Render a DataFrame as an HTML table.

Notes

 As of v2.0.0 this method has changed to use the Styler implementation as
 part of :meth:`.Styler.to\_latex` via `\\jinja2` templating. This means
 that `\\jinja2` is a requirement, and needs to be installed, for this method
 to function. It is advised that users switch to using Styler, since that
 implementation is more frequently updated and contains much more
 flexibility with the output.

Examples

 Convert a general DataFrame to LaTeX with formatting:

```
>>> df = pd.DataFrame(dict(name=['Raphael', 'Donatello'],
...                          age=[26, 45],
...                          height=[181.23, 177.65]))
>>> print(df.to_latex(index=False,
...                    formatters={"name": str.upper,
...                                float_format="{:.1f}".format,
...                                }) # doctest: +SKIP
\\begin{tabular}{lrr}
\\toprule
name & age & height \\
\\midrule
RAPHAEL & 26 & 181.2 \\
DONATELLO & 45 & 177.7 \\
\\bottomrule
\\end{tabular}
```

```
to_pickle(
    self,
    path: FilePath | WriteBuffer[bytes],
    *,
    compression: CompressionOptions = 'infer',
    protocol: int = 5,
    storage_options: StorageOptions | None = None
) -> None
Pickle (serialize) object to file.
```

Parameters

 path : str, path object, or file-like object
 String, path object (implementing `\\os.PathLike[str]`), or file-like
 object implementing a binary `\\write()` function. File path where
 the pickled object will be stored.

compression : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and
 'path_or_buf' is path-like, then detect compression from the following
 extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 Set to `None` for no compression.
 Can also be a dict with key `'method'` set to one of

{`'zip'`, `'gzip'`, `'bz2'`, `'zstd'`, `'xz'`, `'tar'`} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively.

As an example, the following could be passed for faster compression and to create a reproducible gzip archive:

```
compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}
```

`protocol` : int

Int which indicates which protocol should be used by the pickler, default `HIGHEST_PROTOCOL` (see [1]_ paragraph 12.1.2). The possible values are 0, 1, 2, 3, 4, 5. A negative value for the protocol parameter is equivalent to setting its value to `HIGHEST_PROTOCOL`.

.. [1] <https://docs.python.org/3/library/pickle.html>.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with `s3://`, and `gcs://`) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

See Also

`read_pickle` : Load pickled pandas object (or any object) from file.
`DataFrame.to_hdf` : Write DataFrame to an HDF5 file.
`DataFrame.to_sql` : Write DataFrame to a SQL database.
`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.

Examples

```
>>> original_df = pd.DataFrame(
...     {"foo": range(5), "bar": range(5, 10)}
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
>>> original_df.to_pickle("./dummy.pkl") # doctest: +SKIP

>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
```

```
to_sql(
    self,
    name: str,
    con,
    *,
    schema: str | None = None,
    if_exists: Literal['fail', 'replace', 'append', 'delete_rows'] = 'fail',
    index: bool = True,
    index_label: IndexLabel | None = None,
    chunksize: int | None = None,
    dtype: DtypeArg | None = None,
    method: Literal['multi'] | Callable | None = None
) -> int | None
Write records stored in a DataFrame to a SQL database.
```

Databases supported by SQLAlchemy [1]_ are supported. Tables can be newly created, appended to, or overwritten.

.. warning::

The pandas library does not attempt to sanitize inputs provided via a `to_sql` call.

Please refer to the documentation for the underlying database driver to see if it will properly prevent injection, or alternatively be advised of a security risk when executing arbitrary commands in a `to_sql` call.

Parameters

`name` : str

Name of SQL table.

`con` : ADBC connection, `sqlalchemy.engine.(Engine or Connection)` or `sqlite3.Connection`
ADBC provides high performance I/O with native type support, where available. Using SQLAlchemy makes it possible to use any DB supported by that library. Legacy support is provided for `sqlite3.Connection` objects. The user is responsible for engine disposal and connection closure for the SQLAlchemy connectable. See [here](https://docs.sqlalchemy.org/en/20/core/connections.html) `<https://docs.sqlalchemy.org/en/20/core/connections.html>`
If passing a `sqlalchemy.engine.Connection` which is already in a transaction, the transaction will not be committed. If passing a `sqlite3.Connection`, it will not be possible to roll back the record insertion.

`schema` : str, optional

Specify the schema (if database flavor supports this). If None, use default schema.

`if_exists` : {'fail', 'replace', 'append', 'delete_rows'}, default 'fail'
How to behave if the table already exists.

* fail: Raise a `ValueError`.

* replace: Drop the table before inserting new values.

* append: Insert new values to the existing table.

* delete_rows: If a table exists, delete all records and insert data.

`index` : bool, default True

Write DataFrame index as a column. Uses `'index_label'` as the column name in the table. Creates a table index for this column.

`index_label` : str or sequence, default None

Column label for index column(s). If None is given (default) and `'index'` is True, then the index names are used.

A sequence should be given if the DataFrame uses `MultiIndex`.

`chunksize` : int, optional

Specify the number of rows in each batch to be written to the database connection at a time. By default, all rows will be written at once. Also see the `method` keyword.

`dtype` : dict or scalar, optional

Specifying the datatype for columns. If a dictionary is used, the keys should be the column names and the values should be the SQLAlchemy types or strings for the `sqlite3` legacy mode. If a scalar is provided, it will be applied to all columns.

`method` : {None, 'multi', callable}, optional

Controls the SQL insertion clause used:

* None : Uses standard SQL ```INSERT``` clause (one per row).

* 'multi': Pass multiple values in a single ```INSERT``` clause.

* callable with signature ```(pd_table, conn, keys, data_iter)```.

Details and a sample callable implementation can be found in the section `:ref:`insert method <io.sql.method>`.

Returns

None or int

Number of rows affected by `to_sql`. None is returned if the callable passed into ```method``` does not return an integer number of rows.

The number of returned rows affected is the sum of the ```rowcount``` attribute of ```sqlite3.Cursor``` or SQLAlchemy connectable which may not reflect the exact number of written rows as stipulated in the ```sqlite3``` <https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.rowcount> or SQLAlchemy <https://docs.sqlalchemy.org/en/20/core/connections.html#sqlalchemy.engine.Connection.rowcount>.

Raises

`ValueError`

When the table already exists and `'if_exists'` is 'fail' (the default).

See Also

`read_sql` : Read a DataFrame from a table.

Notes

Timezone aware datetime columns will be written as
`Timestamp with timezone` type with SQLAlchemy if supported by the
database. Otherwise, the datetimes will be stored as timezone unaware
timestamps local to the original timezone.

Not all datastores support `method="multi"`. Oracle, for example,
does not support multi-value insert.

References

.. [1] <https://docs.sqlalchemy.org>
.. [2] <https://www.python.org/dev/peps/pep-0249/>

Examples

Create an in-memory SQLite database.

```
>>> from sqlalchemy import create_engine  
>>> engine = create_engine('sqlite://', echo=False)
```

Create a table from scratch with 3 rows.

```
>>> df = pd.DataFrame({'name' : ['User 1', 'User 2', 'User 3']})  
>>> df  
      name  
0  User 1  
1  User 2  
2  User 3
```

```
>>> df.to_sql(name='users', con=engine)  
3  
>>> from sqlalchemy import text  
>>> with engine.connect() as conn:  
...     conn.execute(text("SELECT * FROM users")).fetchall()  
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3')]
```

An `sqlalchemy.engine.Connection` can also be passed to `con`:

```
>>> with engine.begin() as connection:  
...     df1 = pd.DataFrame({'name' : ['User 4', 'User 5']})  
...     df1.to_sql(name='users', con=connection, if_exists='append')  
2
```

This is allowed to support operations that require that the same
DBAPI connection is used for the entire operation.

```
>>> df2 = pd.DataFrame({'name' : ['User 6', 'User 7']})  
>>> df2.to_sql(name='users', con=engine, if_exists='append')  
2  
>>> with engine.connect() as conn:  
...     conn.execute(text("SELECT * FROM users")).fetchall()  
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3'),  
(0, 'User 4'), (1, 'User 5'), (0, 'User 6'),  
(1, 'User 7')]
```

Overwrite the table with just `df2`.

```
>>> df2.to_sql(name='users', con=engine, if_exists='replace',  
...           index_label='id')  
2  
>>> with engine.connect() as conn:  
...     conn.execute(text("SELECT * FROM users")).fetchall()  
[(0, 'User 6'), (1, 'User 7')]
```

Delete all rows before inserting new records with `df3`

```
>>> df3 = pd.DataFrame({"name": ['User 8', 'User 9']})  
>>> df3.to_sql(name='users', con=engine, if_exists='delete_rows',  
...           index_label='id')  
2  
>>> with engine.connect() as conn:  
...     conn.execute(text("SELECT * FROM users")).fetchall()  
[(0, 'User 8'), (1, 'User 9')]
```

Use `method` to define a callable insertion method to do nothing
if there's a primary key conflict on a table in a PostgreSQL database.

```

>>> from sqlalchemy.dialects.postgresql import insert
>>> def insert_on_conflict_nothing(table, conn, keys, data_iter):
...     # "a" is the primary key in "conflict_table"
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = insert(table.table).values(data).on_conflict_do_nothing(index_elements=["a"])
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F821
...                     method=insert_on_conflict_nothing) # doctest: +SKIP
0

```

For MySQL, a callable to update columns ``b`` and ``c`` if there's a conflict on a primary key.

```

>>> from sqlalchemy.dialects.mysql import insert # noqa: F811
>>> def insert_on_conflict_update(table, conn, keys, data_iter):
...     # update columns "b" and "c" on primary key conflict
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = (
...         insert(table.table)
...         .values(data)
...         )
...     stmt = stmt.on_duplicate_key_update(b=stmt.inserted.b, c=stmt.inserted.c)
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F821
...                     method=insert_on_conflict_update) # doctest: +SKIP
2

```

Specify the dtype (especially useful for integers with missing values). Notice that while pandas is forced to store the data as floating point, the database supports nullable integers. When fetching the data with Python, we get back integer scalars.

```

>>> df = pd.DataFrame({"A": [1, None, 2]})
>>> df
   A
0  1.0
1  NaN
2  2.0

>>> from sqlalchemy.types import Integer
>>> df.to_sql(name='integers', con=engine, index=False,
...           dtype={"A": Integer()})
3

>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM integers")).fetchall()
[(1,), (None,), (2,)]

.. versionadded:: 2.2.0

```

pandas now supports writing via ADBC drivers

```

>>> df = pd.DataFrame({'name' : ['User 10', 'User 11', 'User 12']})
>>> df
   name
0  User 10
1  User 11
2  User 12

>>> from adbc_driver_sqlite import dbapi # doctest:+SKIP
>>> with dbapi.connect("sqlite://") as conn: # doctest:+SKIP
...     df.to_sql(name="users", con=conn)
3

```

`to_xarray(self)`

Return an xarray object from the pandas object.

Returns

xarray.DataArray or xarray.Dataset

Data in the pandas structure converted to Dataset if the object is a DataFrame, or a DataArray if the object is a Series.

See Also

```
-----
DataFrame.to_hdf : Write DataFrame to an HDF5 file.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.
```

Notes

```
-----
See the `xarray docs <https://xarray.pydata.org/en/stable/>`__
```

Examples

```
-----
>>> df = pd.DataFrame(
...     [
...         ("falcon", "bird", 389.0, 2),
...         ("parrot", "bird", 24.0, 2),
...         ("lion", "mammal", 80.5, 4),
...         ("monkey", "mammal", np.nan, 4),
...     ],
...     columns=["name", "class", "max_speed", "num_legs"],
... )
>>> df
   name  class  max_speed  num_legs
0  falcon   bird     389.0         2
1  parrot   bird     24.0         2
2    lion  mammal     80.5         4
3  monkey  mammal      NaN         4

>>> df.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (index: 4)
Coordinates:
  * index    (index) int64 32B 0 1 2 3
Data variables:
  name      (index) object 32B 'falcon' 'parrot' 'lion' 'monkey'
  class     (index) object 32B 'bird' 'bird' 'mammal' 'mammal'
  max_speed (index) float64 32B 389.0 24.0 80.5 nan
  num_legs  (index) int64 32B 2 2 4 4

>>> df["max_speed"].to_xarray() # doctest: +SKIP
<xarray.DataArray 'max_speed' (index: 4)>
array([389. , 24. , 80.5,  nan])
Coordinates:
  * index    (index) int64 0 1 2 3

>>> dates = pd.to_datetime(
...     ["2018-01-01", "2018-01-01", "2018-01-02", "2018-01-02"]
... )
>>> df_multiindex = pd.DataFrame(
...     {
...         "date": dates,
...         "animal": ["falcon", "parrot", "falcon", "parrot"],
...         "speed": [350, 18, 361, 15],
...     }
... )
>>> df_multiindex = df_multiindex.set_index(["date", "animal"])

>>> df_multiindex
              speed
date  animal
2018-01-01 falcon    350
           parrot    18
2018-01-02 falcon   361
           parrot    15

>>> df_multiindex.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (date: 2, animal: 2)
Coordinates:
  * date     (date) datetime64[s] 2018-01-01 2018-01-02
  * animal   (animal) object 'falcon' 'parrot'
Data variables:
  speed     (date, animal) int64 350 18 361 15

truncate(
    self,
    before=None,
    after=None,
    axis: Axis | None = None,
```

```

copy: bool | lib.NoDefault = <no_default>
) -> Self
    Truncate a Series or DataFrame before and after some index value.

This is a useful shorthand for boolean indexing based on index
values above or below certain thresholds.

Parameters
-----
before : date, str, int
    Truncate all rows before this index value.
after : date, str, int
    Truncate all rows after this index value.
axis : {0 or 'index', 1 or 'columns'}, optional
    Axis to truncate. Truncates the index (rows) by default.
    For `Series` this parameter is unused and defaults to 0.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

.. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
    for more details.

Returns
-----
type of caller
    The truncated Series or DataFrame.

See Also
-----
DataFrame.loc : Select a subset of a DataFrame by label.
DataFrame.iloc : Select a subset of a DataFrame by position.

Notes
-----
If the index being truncated contains only datetime values,
`before` and `after` may be specified as strings instead of
Timestamps.

Examples
-----
>>> df = pd.DataFrame(
...     {
...         "A": ["a", "b", "c", "d", "e"],
...         "B": ["f", "g", "h", "i", "j"],
...         "C": ["k", "l", "m", "n", "o"],
...     },
...     index=[1, 2, 3, 4, 5],
... )
>>> df
   A B C
1  a f k
2  b g l
3  c h m
4  d i n
5  e j o

>>> df.truncate(before=2, after=4)
   A B C
2  b g l
3  c h m
4  d i n

The columns of a DataFrame can be truncated.

>>> df.truncate(before="A", after="B", axis="columns")
   A B
1  a f
2  b g
3  c h
4  d i

```

5 e j

For Series, only rows can be truncated.

```
>>> df["A"].truncate(before=2, after=4)
2      b
3      c
4      d
Name: A, dtype: str
```

The index values in ``truncate`` can be datetimes or string dates.

```
>>> dates = pd.date_range("2016-01-01", "2016-02-01", freq="s")
>>> df = pd.DataFrame(index=dates, data={"A": 1})
>>> df.tail()
                A
2016-01-31 23:59:56  1
2016-01-31 23:59:57  1
2016-01-31 23:59:58  1
2016-01-31 23:59:59  1
2016-02-01 00:00:00  1

>>> df.truncate(
...     before=pd.Timestamp("2016-01-05"), after=pd.Timestamp("2016-01-10")
... ).tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Because the index is a DatetimeIndex containing only dates, we can specify `before` and `after` as strings. They will be coerced to Timestamps before truncation.

```
>>> df.truncate("2016-01-05", "2016-01-10").tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Note that ``truncate`` assumes a 0 value for any unspecified time component (midnight). This differs from partial string slicing, which returns any partially matching dates.

```
>>> df.loc["2016-01-05":"2016-01-10", :].tail()
                A
2016-01-10 23:59:55  1
2016-01-10 23:59:56  1
2016-01-10 23:59:57  1
2016-01-10 23:59:58  1
2016-01-10 23:59:59  1
```

```
tz_convert(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self
    Convert tz-aware axis to target time zone.
```

Parameters

tz : str or tzinfo object or None
Target time zone. Passing ``None`` will convert to UTC and remove the timezone information.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to convert
level : int, str, default None
If axis is a MultiIndex, convert a specific level. Otherwise must be None.
copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

Series/DataFrame
Object with time zone converted axis.

Raises

TypeError
If the axis is tz-naive.

See Also

DataFrame.tz_localize: Localize tz-naive index of DataFrame to target time zone.
Series.tz_localize: Localize tz-naive index of Series to target time zone.

Examples

Change to another time zone:

```
>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]),
... )
>>> s.tz_convert("Asia/Shanghai")
2018-09-15 07:30:00+08:00    1
dtype: int64
```

Pass None to convert to UTC and get a tz-naive index:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_convert(None)
2018-09-14 23:30:00    1
dtype: int64
```

```
tz_localize(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Localize time zone naive index of a Series or DataFrame to target time zone.
```

This operation localizes the Index. To localize the values in a time zone naive Series, use :meth:`Series.dt.tz\_localize`.

Parameters

tz : str or tzinfo or None
Time zone to localize. Passing ``None`` will remove the time zone information and preserve local time.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to localize
level : int, str, default None
If axis is a MultiIndex, localize a specific level. Otherwise must be None.
copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since

pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`\_\_` for more details.

`ambiguous` : 'infer', bool, bool-ndarray, 'NaT', default 'raise'
When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `'ambiguous'` parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool (or bool-ndarray) where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

`nonexistent` : str, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST. Valid values are:

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

Series/DataFrame

Same type as the input, with time zone naive or aware index, depending on `'tz'`.

Raises

TypeError

If the `TimeSeries` is `tz-aware` and `tz` is not `None`.

See Also

`Series.dt.tz_localize`: Localize the values in a time zone naive Series.

`Timestamp.tz_localize`: Localize the `Timestamp` to a timezone.

Examples

Localize local times:

```
>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00"]),
... )
>>> s.tz_localize("CET")
2018-09-15 01:30:00+02:00    1
dtype: int64
```

Pass `None` to convert to `tz-naive` index and preserve local time:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_localize(None)
2018-09-15 01:30:00    1
dtype: int64
```

Be careful with DST changes. When there is sequential data, pandas can infer the DST time:

```
>>> s = pd.Series(
...     range(7),
...     index=pd.DatetimeIndex(
...         [
```



```

...         "2018-10-28 01:30:00",
...         "2018-10-28 02:00:00",
...         "2018-10-28 02:30:00",
...         "2018-10-28 02:00:00",
...         "2018-10-28 02:30:00",
...         "2018-10-28 03:00:00",
...         "2018-10-28 03:30:00",
...     ]
... ),
... )
>>> s.tz_localize("CET", ambiguous="infer")
2018-10-28 01:30:00+02:00    0
2018-10-28 02:00:00+02:00    1
2018-10-28 02:30:00+02:00    2
2018-10-28 02:00:00+01:00    3
2018-10-28 02:30:00+01:00    4
2018-10-28 03:00:00+01:00    5
2018-10-28 03:30:00+01:00    6
dtype: int64

```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```

>>> s = pd.Series(
...     range(3),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:20:00",
...             "2018-10-28 02:36:00",
...             "2018-10-28 03:46:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous=np.array([True, True, False]))
2018-10-28 01:20:00+02:00    0
2018-10-28 02:36:00+02:00    1
2018-10-28 03:46:00+01:00    2
dtype: int64

```

If the DST transition causes nonexistent times, you can shift these dates forward or backward with a timedelta object or `'shift_forward'` or `'shift_backward'`.

```

>>> dti = pd.DatetimeIndex(
...     ["2015-03-29 02:30:00", "2015-03-29 03:30:00"], dtype="M8[ns]"
... )
>>> s = pd.Series(range(2), index=dti)
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_forward")
2015-03-29 03:00:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_backward")
2015-03-29 01:59:59.999999999+01:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent=pd.Timedelta("1h"))
2015-03-29 03:30:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64

```

```

where(
    self,
    cond,
    other=<no_default>,
    *,
    inplace: bool = False,
    axis: Axis | None = None,
    level: Level | None = None
) -> Self
Replace values where the condition is False.

```

This method allows conditional replacement of values. Where the condition evaluates to True, the original values are retained; where it evaluates to False, values are replaced with corresponding entries from `other`.

Parameters

```

-----
cond : bool Series/DataFrame, array-like, or callable
      Where `cond` is True, keep the original value. Where
      False, replace with corresponding value from `other`.
      If `cond` is callable, it is computed on the Series/DataFrame and
      should return boolean Series/DataFrame or array. The callable must
      not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
      Entries where `cond` is False are replaced with
      corresponding value from `other`.
      If other is callable, it is computed on the Series/DataFrame and
      should return scalar or Series/DataFrame. The callable must not
      change input Series/DataFrame (though pandas doesn't check it).
      If not specified, entries will be filled with the corresponding
      NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension
      dtypes).
inplace : bool, default False
      Whether to perform the operation in place on the data.
axis : int, default None
      Alignment axis if needed. For `Series` this parameter is
      unused and defaults to 0.
level : int, default None
      Alignment level if needed.

```

Returns

```

-----
Series or DataFrame
      When applied to a Series, the function will return a Series,
      and when applied to a DataFrame, it will return a DataFrame.

```

See Also

```

-----
:func:`DataFrame.mask` : Return an object of same shape as caller.
:func:`Series.mask` : Return an object of same shape as caller.

```

Notes

```

-----
The where method is an application of the if-then idiom. For each
element in the caller, if ``cond`` is ``True`` the
element is used; otherwise the corresponding element from
``other`` is used. If the axis of ``other`` does not align with axis of
``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions
will be filled with False.

```

```

The signature for :func:`Series.where` or
:func:`DataFrame.where` differs from :func:`numpy.where`.
Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

```

```

For further details and examples see the ``where`` documentation in
:ref:`indexing <indexing.where_mask>`.

```

```

The dtype of the object takes precedence. The fill value is casted to
the object's dtype, if this can be done losslessly.

```

Examples

```

-----
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0    NaN
1    1.0
2    2.0
3    3.0
4    4.0
dtype: float64
>>> s.mask(s > 0)
0    0.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0    0
1   99

```

```

2    99
3    99
4    99
dtype: int64
>>> s.mask(t, 99)
0    99
1     1
2    99
3    99
4    99
dtype: int64

>>> s.where(s > 1, 10)
0    10
1    10
2     2
3     3
4     4
dtype: int64
>>> s.mask(s > 1, 10)
0     0
1     1
2    10
3    10
4    10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
   A  B
0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A      B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A      B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True

```

xs(

```

    self,
    key: IndexLabel,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    drop_level: bool = True
) -> Self

```

Return cross-section from the Series/DataFrame.

This method takes a `key` argument to select data at a particular level of a MultiIndex.

Parameters

key : label or tuple of label

Label contained in the index, or partially in a MultiIndex.

axis : {0 or 'index', 1 or 'columns'}, default 0

Axis to retrieve cross-section on.

level : object, defaults to first n levels (n=1 or len(key))

In case of a key partially contained in a MultiIndex, indicate which levels are used. Levels can be referred by label or position.
 drop_level : bool, default True
 If False, returns object with same levels as self.

Returns

 Series or DataFrame
 Cross-section from the original Series or DataFrame corresponding to the selected index levels.

See Also

 DataFrame.loc : Access a group of rows and columns by label(s) or a boolean array.
 DataFrame.iloc : Purely integer-location based indexing for selection by position.

Notes

 `xs` can not be used to set values.

MultiIndex Slicers is a generic way to get/set values on any level or levels.

It is a superset of `xs` functionality, see :ref:MultiIndex Slicers <advanced.mi_slicers>.

Examples

 >>> d = {
 ... "num_legs": [4, 4, 2, 2],
 ... "num_wings": [0, 0, 2, 2],
 ... "class": ["mammal", "mammal", "mammal", "bird"],
 ... "animal": ["cat", "dog", "bat", "penguin"],
 ... "locomotion": ["walks", "walks", "flies", "walks"],
 ... }
 >>> df = pd.DataFrame(data=d)
 >>> df = df.set_index(["class", "animal", "locomotion"])
 >>> df

			num_legs	num_wings
class	animal	locomotion		
mammal	cat	walks	4	0
	dog	walks	4	0
	bat	flies	2	2
bird	penguin	walks	2	2

Get values at specified index

```
>>> df.xs("mammal")
```

		num_legs	num_wings
animal	locomotion		
cat	walks	4	0
dog	walks	4	0
bat	flies	2	2

Get values at several indexes

```
>>> df.xs(("mammal", "dog", "walks"))
```

```
num_legs    4
num_wings    0
Name: (mammal, dog, walks), dtype: int64
```

Get values at specified index and level

```
>>> df.xs("cat", level=1)
```

		num_legs	num_wings
class	locomotion		
mammal	walks	4	0

Get values at several indexes and levels

```
>>> df.xs(("bird", "walks"), level=[0, "locomotion"])
```

		num_legs	num_wings
animal			
penguin	walks	2	2

Get values at specified column and axis

```
>>> df.xs("num_wings", axis=1)
class    animal    locomotion
mammal   cat       walks      0
         dog       walks      0
         bat       flies      2
bird     penguin   walks      2
Name: num_wings, dtype: int64
```

Readonly properties inherited from pandas.core.generic.NDFrame:

flags

Get the properties associated with this pandas object.

The available flags are

* :attr:`Flags.allows\_duplicate\_labels`

See Also

Flags : Flags that apply to pandas objects.

DataFrame.attrs : Global metadata applying to this dataset.

Notes

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in :attr:`DataFrame.attrs`.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags
<Flags(allows_duplicate_labels=True)>
```

Flags can be get or set using ````

```
>>> df.flags.allows_duplicate_labels
True
>>> df.flags.allows_duplicate_labels = False
```

Or by slicing with a key

```
>>> df.flags["allows_duplicate_labels"]
False
>>> df.flags["allows_duplicate_labels"] = True
```

Data descriptors inherited from pandas.core.generic.NDFrame:

attrs

Dictionary of global attributes of this dataset.

.. warning::

attrs is experimental and may change without warning.

See Also

DataFrame.flags : Global flags applying to this object.

Notes

Many operations that create new datasets will copy ``attrs``. Copies are always deep so that changing ``attrs`` will only affect the present dataset. :func:`pandas.concat` and :func:`pandas.merge` will only copy ``attrs`` if all input datasets have the same ``attrs``.

Examples

For Series:

```
>>> ser = pd.Series([1, 2, 3])
>>> ser.attrs = {"A": [10, 20, 30]}
>>> ser.attrs
{'A': [10, 20, 30]}
```

For DataFrame:

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.attrs = {"A": [10, 20, 30]}
>>> df.attrs
{'A': [10, 20, 30]}
```

Methods inherited from pandas.core.base.PandasObject:

```
__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values
```

Methods inherited from pandas.core.accessor.DirNamesMixin:

```
__dir__(self) -> list[str]
    Provide method name lookup and completion.
```

Notes

Only provide 'public' methods.

Readonly properties inherited from pandas.core.indexing.IndexingMixin:

at

Access a single value for a row/column label pair.

Similar to ``loc``, in that both provide label-based lookups. Use ``at`` if you only need to get or set a single value in a DataFrame or Series.

Raises

KeyError

If getting a value and 'label' does not exist in a DataFrame or Series.

ValueError

If row/column label pair is not a tuple or if any label from the pair is not a scalar for DataFrame.

If label is list-like (*excluding* NamedTuple) for Series.

See Also

DataFrame.at : Access a single value for a row/column pair by label.

DataFrame.iat : Access a single value for a row/column pair by integer position.

DataFrame.loc : Access a group of rows and columns by label(s).

DataFrame.iloc : Access a group of rows and columns by integer position(s).

Series.at : Access a single value by label.

Series.iat : Access a single value by integer position.

Series.loc : Access a group of rows by label(s).

Series.iloc : Access a group of rows by integer position(s).

Notes

See :ref:`Fast scalar value getting and setting <indexing.basics.get\_value>` for more details.

Examples

```
>>> df = pd.DataFrame(
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]],
...     index=[4, 5, 6],
...     columns=["A", "B", "C"],
... )
>>> df
   A  B  C
4  0  2  3
5  0  4  1
6 10 20 30
```

Get value at specified row/column pair

```
>>> df.at[4, "B"]
np.int64(2)
```

Set value at specified row/column pair

```
>>> df.at[4, "B"] = 10
>>> df.at[4, "B"]
np.int64(10)
```

Get value within a Series

```
>>> df.loc[5].at["B"]
np.int64(4)
```

iat

Access a single value for a row/column pair by integer position.

Similar to ``iloc``, in that both provide integer-based lookups. Use ``iat`` if you only need to get or set a single value in a DataFrame or Series.

Raises

IndexError

When integer position is out of bounds.

See Also

DataFrame.at : Access a single value for a row/column label pair.

DataFrame.loc : Access a group of rows and columns by label(s).

DataFrame.iloc : Access a group of rows and columns by integer position(s).

Examples

```
>>> df = pd.DataFrame(
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]], columns=["A", "B", "C"]
... )
>>> df
   A  B  C
0  0  2  3
1  0  4  1
2 10 20 30
```

Get value at specified row/column pair

```
>>> df.iat[1, 2]
np.int64(1)
```

Set value at specified row/column pair

```
>>> df.iat[1, 2] = 10
>>> df.iat[1, 2]
np.int64(10)
```

Get value within a series

```
>>> df.loc[0].iat[1]
np.int64(2)
```

iloc

Purely integer-location based indexing for selection by position.

.. versionchanged:: 3.0

Callables which return a tuple are deprecated as input.

``.iloc[]`` is primarily integer position based (from ``0`` to ``length-1`` of the axis), but may also be used with a boolean array.

Allowed inputs are:

- An integer, e.g. ``5``.
- A list or array of integers, e.g. ``[4, 3, 0]``.
- A slice object with ints, e.g. ``1:7``.
- A boolean array.

- A ``callable`` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above). This is useful in method chains, when you don't have a reference to the calling object, but would like to base your selection on some value.
- A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. ``(0, 1)``.

``.iloc`` will raise ``IndexError`` if a requested indexer is out-of-bounds, except *slice* indexers which allow out-of-bounds indexing (this conforms with python/numpy *slice* semantics).

See more at :ref:`Selection by Position <indexing.integer>`.

See Also

 DataFrame.iat : Fast integer location scalar accessor.
 DataFrame.loc : Purely label-location based indexer for selection by label.
 Series.iloc : Purely integer-location based indexing for selection by position.

Examples

```
>>> mydict = [
...     {"a": 1, "b": 2, "c": 3, "d": 4},
...     {"a": 100, "b": 200, "c": 300, "d": 400},
...     {"a": 1000, "b": 2000, "c": 3000, "d": 4000},
... ]
>>> df = pd.DataFrame(mydict)
>>> df
```

	a	b	c	d
0	1	2	3	4
1	100	200	300	400
2	1000	2000	3000	4000

****Indexing just the rows****

With a scalar integer.

```
>>> type(df.iloc[0])
<class 'pandas.Series'>
>>> df.iloc[0]
a    1
b    2
c    3
d    4
Name: 0, dtype: int64
```

With a list of integers.

```
>>> df.iloc[[0]]
   a  b  c  d
0  1  2  3  4
>>> type(df.iloc[[0]])
<class 'pandas.DataFrame'>
```

```
>>> df.iloc[[0, 1]]
   a  b  c  d
0  1  2  3  4
1 100 200 300 400
```

With a `slice` object.

```
>>> df.iloc[:3]
   a  b  c  d
0  1  2  3  4
1 100 200 300 400
2 1000 2000 3000 4000
```

With a boolean mask the same length as the index.

```
>>> df.iloc[[True, False, True]]
   a  b  c  d
0  1  2  3  4
2 1000 2000 3000 4000
```

With a callable, useful in method chains. The `x` passed

to the ``lambda`` is the DataFrame being sliced. This selects the rows whose index label even.

```
>>> df.iloc[lambda x: x.index % 2 == 0]
   a    b    c    d
0   1    2    3    4
2 1000 2000 3000 4000
```

****Indexing both axes****

You can mix the indexer types for the index and columns. Use ``:``` to select the entire axis.

With scalar integers.

```
>>> df.iloc[0, 1]
np.int64(2)
```

With lists of integers.

```
>>> df.iloc[[0, 2], [1, 3]]
   b    d
0   2    4
2 2000 4000
```

With ``slice`` objects.

```
>>> df.iloc[1:3, 0:3]
   a    b    c
1  100  200  300
2 1000 2000 3000
```

With a boolean array whose length matches the columns.

```
>>> df.iloc[:, [True, False, True, False]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

With a callable function that expects the Series or DataFrame.

```
>>> df.iloc[:, lambda df: [0, 2]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

loc

Access a group of rows and columns by label(s) or a boolean array.

``.loc[]`` is primarily label based, but may also be used with a boolean array.

Allowed inputs are:

- A single label, e.g. ``5`` or ``a``, (note that ``5`` is interpreted as a *label* of the index, and ****never**** as an integer position along the index).
- A list or array of labels, e.g. ``['a', 'b', 'c']``.
- A slice object with labels, e.g. ``a:f``.

.. warning:: Note that contrary to usual python slices, ****both**** the start and the stop are included

- A boolean array of the same length as the axis being sliced, e.g. ``[True, False, True]``.
- An alignable boolean Series. The index of the key will be aligned before masking.
- An alignable Index. The Index of the returned selection will be the input.
- A ``callable`` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above)

See more at :ref:`Selection by Label <indexing.label>`.

Raises

KeyError
If any items are not found.
IndexingError
If an indexed key is passed and its index is unalignable to the frame index.

See Also

DataFrame.at : Access a single value for a row/column label pair.
DataFrame.iloc : Access group of rows and columns by integer position(s).
DataFrame.xs : Returns a cross-section (row(s) or column(s)) from the Series/DataFrame.
Series.loc : Access group of values using labels.

Examples

Getting values

```
>>> df = pd.DataFrame(  
...     [[1, 2], [4, 5], [7, 8]],  
...     index=["cobra", "viper", "sidewinder"],  
...     columns=["max_speed", "shield"],  
... )  
>>> df
```

	max_speed	shield
cobra	1	2
viper	4	5
sidewinder	7	8

Single label. Note this returns the row as a Series.

```
>>> df.loc["viper"]  
max_speed    4  
shield       5  
Name: viper, dtype: int64
```

List of labels. Note using ``[]`` returns a DataFrame.

```
>>> df.loc[["viper", "sidewinder"]]  
           max_speed  shield  
viper           4       5  
sidewinder      7       8
```

Single label for row and column

```
>>> df.loc["cobra", "shield"]  
np.int64(2)
```

Slice with labels for row and single label for column. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc["cobra":"viper", "max_speed"]  
cobra    1  
viper    4  
Name: max_speed, dtype: int64
```

Boolean list with the same length as the row axis

```
>>> df.loc[[False, False, True]]  
           max_speed  shield  
sidewinder      7       8
```

Alignable boolean Series:

```
>>> df.loc[  
...     pd.Series([False, True, False], index=["viper", "sidewinder", "cobra"])  
... ]  
           max_speed  shield  
sidewinder      7       8
```

Index (same behavior as ``df.reindex``)

```
>>> df.loc[pd.Index(["cobra", "viper"], name="foo")]  
           max_speed  shield  
foo  
cobra          1       2  
viper          4       5
```

Conditional that returns a boolean Series

```
>>> df.loc[df["shield"] > 6]
           max_speed  shield
sidewinder         7       8
```

Conditional that returns a boolean Series with column labels specified

```
>>> df.loc[df["shield"] > 6, ["max_speed"]]
           max_speed
sidewinder         7
```

Multiple conditional using ``&`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 1) & (df["shield"] < 8)]
           max_speed  shield
viper             4       5
```

Multiple conditional using ``|`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 4) | (df["shield"] < 5)]
           max_speed  shield
cobra              1       2
sidewinder         7       8
```

Please ensure that each condition is wrapped in parentheses ``()``.
See the :ref:`user guide<indexing.boolean>`
for more details and explanations of Boolean indexing.

```
.. note::
    If you find yourself using 3 or more conditionals in ``.loc[]``,
    consider using :ref:`advanced indexing<advanced.advanced_hierarchical>`.
```

See below for using ``.loc[]`` on MultiIndex DataFrames.

Callable that returns a boolean Series

```
>>> df.loc[lambda df: df["shield"] == 8]
           max_speed  shield
sidewinder         7       8
```

****Setting values****

Set value for all items matching the list of labels

```
>>> df.loc[["viper", "sidewinder"], ["shield"]] = 50
>>> df
           max_speed  shield
cobra              1       2
viper             4      50
sidewinder         7      50
```

Set value for an entire row

```
>>> df.loc["cobra"] = 10
>>> df
           max_speed  shield
cobra            10      10
viper             4      50
sidewinder         7      50
```

Set value for an entire column

```
>>> df.loc[:, "max_speed"] = 30
>>> df
           max_speed  shield
cobra             30      10
viper             30      50
sidewinder        30      50
```

Set value for rows matching callable condition

```
>>> df.loc[df["shield"] > 35] = 0
>>> df
           max_speed  shield
cobra             30      10
viper             0       0
```

```
sidewinder          0      0
```

Add value matching location

```
>>> df.loc["viper", "shield"] += 5
>>> df
```

```
      max_speed  shield
cobra          30     10
viper           0      5
sidewinder       0      0
```

Setting using a ``Series`` or a ``DataFrame`` sets the values matching the index labels, not the index positions.

```
>>> shuffled_df = df.loc[["viper", "cobra", "sidewinder"]]
>>> df.loc[:] += shuffled_df
>>> df
```

```
      max_speed  shield
cobra          60     20
viper           0     10
sidewinder       0      0
```

****Getting values on a DataFrame with an index that has integer labels****

Another example using integers for the index

```
>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=[7, 8, 9],
...     columns=["max_speed", "shield"],
... )
>>> df
```

	max_speed	shield
7	1	2
8	4	5
9	7	8

Slice with integer labels for rows. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc[7:9]
      max_speed  shield
7           1      2
8           4      5
9           7      8
```

****Getting values with a MultiIndex****

A number of examples using a DataFrame with a MultiIndex

```
>>> tuples = [
...     ("cobra", "mark i"),
...     ("cobra", "mark ii"),
...     ("sidewinder", "mark i"),
...     ("sidewinder", "mark ii"),
...     ("viper", "mark ii"),
...     ("viper", "mark iii"),
... ]
>>> index = pd.MultiIndex.from_tuples(tuples)
>>> values = [[12, 2], [0, 4], [10, 20], [1, 4], [7, 1], [16, 36]]
>>> df = pd.DataFrame(values, columns=["max_speed", "shield"], index=index)
>>> df
```

		max_speed	shield
cobra	mark i	12	2
	mark ii	0	4
sidewinder	mark i	10	20
	mark ii	1	4
viper	mark ii	7	1
	mark iii	16	36

Single label. Note this returns a DataFrame with a single index.

```
>>> df.loc["cobra"]
      max_speed  shield
mark i         12      2
mark ii         0      4
```

Single index tuple. Note this returns a Series.

```
>>> df.loc[("cobra", "mark ii")]
max_speed    0
shield       4
Name: (cobra, mark ii), dtype: int64
```

Single label for row and column. Similar to passing in a tuple, this returns a Series.

```
>>> df.loc["cobra", "mark i"]
max_speed    12
shield       2
Name: (cobra, mark i), dtype: int64
```

Single tuple. Note using ``[]`` returns a DataFrame.

```
>>> df.loc[["cobra", "mark ii"]]
      max_speed  shield
cobra mark ii      0      4
```

Single tuple for the index with a single label for the column

```
>>> df.loc[("cobra", "mark i"), "shield"]
np.int64(2)
```

Slice from index tuple to single label

```
>>> df.loc[("cobra", "mark i") : "viper"]
      max_speed  shield
cobra      mark i      12      2
      mark ii      0      4
sidewinder mark i      10     20
      mark ii      1      4
viper      mark ii      7      1
      mark iii     16     36
```

Slice from index tuple to index tuple

```
>>> df.loc[("cobra", "mark i") : ("viper", "mark ii")]
      max_speed  shield
cobra      mark i      12      2
      mark ii      0      4
sidewinder mark i      10     20
      mark ii      1      4
viper      mark ii      7      1
```

Please see the :ref:`user guide<advanced.advanced\_hierarchical>` for more details and explanations of advanced indexing.

****Assignment with Series****

When assigning a Series to `.loc[row_indexer, col_indexer]`, pandas aligns the Series by index labels, not by order or position.

Series assignment with `.loc` and index alignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
>>> s = pd.Series([10, 20], index=[1, 0]) # Note reversed order
>>> df.loc[:, "B"] = s # Aligns by index, not order
>>> df
   A  B
0  1  20.0
1  2  10.0
2  3   NaN
```

```
class SparseDtype(pandas.api.extensions.ExtensionDtype)
    SparseDtype(dtype: Dtype = <class 'numpy.float64'>, fill_value: Any = None) -> None
```

Dtype for data stored in :class:`SparseArray`.

`SparseDtype` is used as the data type for :class:`SparseArray`, enabling more efficient storage of data that contains a significant number of repetitive values typically represented by a fill value. It supports any scalar dtype as the underlying data type of the non-fill values.`

Parameters

```

-----
dtype : str, ExtensionDtype, numpy.dtype, type, default numpy.float64
    The dtype of the underlying array storing the non-fill value values.
fill_value : scalar, optional
    The scalar value not stored in the SparseArray. By default, this
    depends on ``dtype``.

```

```

=====
dtype      na_value
=====
float      ``np.nan``
complex    ``np.nan``
int        ``0``
bool       ``False``
datetime64 ``pd.NaT``
timedelta64 ``pd.NaT``
=====

```

The default value may be overridden by specifying a ``fill\_value``.

Attributes

```
-----
```

None

Methods

```
-----
```

None

See Also

```
-----
```

arrays.SparseArray : The array structure that uses SparseDtype for data representation.

Examples

```
-----
```

```

>>> ser = pd.Series([1, 0, 0], dtype=pd.SparseDtype(dtype=int, fill_value=0))
>>> ser
0    1
1    0
2    0
dtype: Sparse[int64, 0]
>>> ser.sparse.density
0.3333333333333333

```

Method resolution order:

```

SparseDtype
pandas.api.extensions.ExtensionDtype
builtins.object

```

Methods defined here:

```

__eq__(self, other: object) -> bool from pandas.core.dtypes.dtypes.SparseDtype
    Check whether 'other' is equal to self.

```

By default, 'other' is considered equal if either

```

* it's a string matching 'self.name'.
* it's an instance of this type and all of the attributes
  in ``self._metadata`` are equal between `self` and `other`.

```

Parameters

```
-----
```

other : Any

Returns

```
-----
```

bool

```

__hash__(self) -> int from pandas.core.dtypes.dtypes.SparseDtype
    Return hash(self).

```

```

__init__(self, dtype: Dtype = <class 'numpy.float64'>, fill_value: Any = None) -> None from pandas.core.dtypes.dtypes.SparseDtype
    Initialize self. See help(type(self)) for accurate signature.

```

```

__repr__(self) -> str from pandas.core.dtypes.dtypes.SparseDtype
    Return repr(self).

```

`construct_array_type(self) -> type_t[SparseArray]` from `pandas.core.dtypes.dtypes.SparseDtype`
Return the array type associated with this dtype.

Returns

type

`update_dtype(self, dtype) -> SparseDtype` from `pandas.core.dtypes.dtypes.SparseDtype`
Convert the SparseDtype to a new dtype.

This takes care of converting the ``fill\_value``.

Parameters

`dtype : Union[str, numpy.dtype, SparseDtype]`
The new dtype to use.

- * For a SparseDtype, it is simply returned
- * For a NumPy dtype (or str), the current fill value is converted to the new dtype, and a SparseDtype with `dtype` and the new fill value is returned.

Returns

SparseDtype
A new SparseDtype with the correct `dtype` and fill value for that `dtype`.

Raises

ValueError
When the current fill value cannot be converted to the new `dtype` (e.g. trying to convert ``np.nan`` to an integer dtype).

Examples

```
>>> SparseDtype(int, 0).update_dtype(float)
Sparse[float64, np.float64(0.0)]
```

```
>>> SparseDtype(int, 1).update_dtype(SparseDtype(float, np.nan))
Sparse[float64, nan]
```

Class methods defined here:

`construct_from_string(string: str) -> SparseDtype` from `pandas.core.dtypes.dtypes.SparseDtype`
Construct a SparseDtype from a string form.

Parameters

`string : str`
Can take the following forms.

string	dtype
'int'	SparseDtype[np.int64, 0]
'Sparse'	SparseDtype[np.float64, nan]
'Sparse[int]'	SparseDtype[np.int64, 0]
'Sparse[int, 0]'	SparseDtype[np.int64, 0]

It is not possible to specify non-default fill values with a string. An argument like ``'Sparse[int, 1]`` will raise a ``TypeError`` because the default fill value for integers is 0.

Returns

SparseDtype

`is_dtype(dtype: object) -> bool` from `pandas.core.dtypes.dtypes.SparseDtype`
Check if we match 'dtype'.

Parameters

dtype : object
The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties defined here:

fill_value
The fill value of the array.

Converting the SparseArray to a dense ndarray will fill the array with this value.

.. warning::

It's possible to end up with a SparseArray that has ``fill\_value`` values in ``sp\_values``. This can occur, for example, when setting ``SparseArray.fill\_value`` directly.

kind
The sparse kind. Either 'integer', or 'block'.

name
A string identifying the data type.

Will be used for display in, e.g. ``Series.dtype``

subtype

type
The scalar type for the array, e.g. ``int``

It's expected ``ExtensionArray[item]`` returns an instance of ``ExtensionDtype.type`` for scalar ``item``, assuming that value is valid (not NA). NA values do not need to be instances of ``type``.

Methods inherited from pandas.api.extensions.ExtensionDtype:

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
Construct an ExtensionArray of this dtype with the given shape.

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

na_value
Default NA value to use for this type.

This is used in e.g. `ExtensionArray.take`. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for `JSONArray`, this is an empty dictionary.

names

Ordered list of field names, or `None` if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from `pandas.api.extensions.ExtensionDtype`:

`__dict__`

dictionary for instance variables

`__weakref__`

list of weak references to the object

`index_class`

The Index subclass to return from `Index.__new__` when this dtype is encountered.

Data and other attributes inherited from `pandas.api.extensions.ExtensionDtype`:

`__annotations__` = {}

`class StringDtype(pandas.core.dtypes.base.StorageExtensionDtype)`

`StringDtype(`

`storage: str | None = None,`

`na_value: libmissing.NAType | float = <NA>`

`) -> None`

Extension dtype for string data.

.. warning::

`StringDtype` is considered experimental. The implementation and parts of the API may change without warning.

Parameters

`storage` : {"python", "pyarrow"}, optional

If not given, the value of `pd.options.mode.string_storage`.

`na_value` : {np.nan, pd.NA}, default pd.NA

Whether the dtype follows NaN or NA missing value semantics.

Attributes

`storage`

`na_value`

Methods

`None`

See Also

`BooleanDtype` : Extension dtype for boolean data.

Examples

`>>> pd.StringDtype()`
`<StringDtype(na_value=<NA>)>`

`>>> pd.StringDtype(storage="python")`
`<StringDtype(storage='python', na_value=<NA>)>`

Method resolution order:

`StringDtype`

`pandas.core.dtypes.base.StorageExtensionDtype`

`pandas.api.extensions.ExtensionDtype`

`builtins.object`

Methods defined here:

```

__eq__(self, other: object) -> bool from pandas.core.arrays.string_.StringDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

Parameters
-----
other : Any

Returns
-----
bool

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseStringArray from pandas.core.arrays.string_.StringDtype
    Construct StringArray from pyarrow Array/ChunkedArray.

__hash__(self) -> int from pandas.core.arrays.string_.StringDtype
    Return hash(self).

__init__(
    self,
    storage: str | None = None,
    na_value: libmissing.NAType | float = <NA>
) -> None from pandas.core.arrays.string_.StringDtype
    Initialize self. See help(type(self)) for accurate signature.

__reduce__(self) from pandas.core.arrays.string_.StringDtype
    Helper for pickle.

__repr__(self) -> str from pandas.core.arrays.string_.StringDtype
    Return repr(self).

__setstate__(self, state: MutableMapping[str, Any]) -> None from pandas.core.arrays.string_.StringDtype

construct_array_type(self) -> type_t[BaseStringArray] from pandas.core.arrays.string_.StringDtype
    Return the array type associated with this dtype.

Returns
-----
type

-----
Class methods defined here:

construct_from_string(string) -> Self from pandas.core.arrays.string_.StringDtype
    Construct a StringDtype from a string.

Parameters
-----
string : str
    The type of the name. The storage type will be taking from `string`.
    Valid options and their storage types are

=====
string                                result storage
=====
`'string'`                             pd.options.mode.string_storage, default python
`'string[python]'`                     python
`'string[pyarrow]'`                   pyarrow
=====

Returns
-----
StringDtype

Raise
-----
TypeError
    If the string is not a valid option.

-----
Readonly properties defined here:

```

`na_value`

The missing value representation for this dtype.

This value indicates which missing value semantics are used by this dtype. Returns ``np.nan`` for the default string dtype with NumPy semantics, and ``pd.NA`` for the opt-in string dtype with pandas NA semantics.

See Also

`isna` : Detect missing values.
`NA` : Missing value indicator for nullable dtypes.

Examples

>>> ser = pd.Series(["a", "b"])
>>> ser.dtype
<StringDtype(na_value=nan)>
>>> ser.dtype.na_value
nan

`name`

A string identifying the data type.

Will be used for display in, e.g. ``Series.dtype``

`storage`

The storage backend for this dtype.

Can be either "pyarrow" or "python".

See Also

`StringDtype.na_value` : The missing value for this dtype.

Examples

>>> ser = pd.Series(["a", "b"])
>>> ser.dtype
<StringDtype(na_value=nan)>
>>> ser.dtype.storage
'pyarrow'

`type`

The scalar type for the array, e.g. ``int``

It's expected ``ExtensionArray[item]`` returns an instance of ``ExtensionDtype.type`` for scalar ``item``, assuming that value is valid (not NA). NA values do not need to be instances of ``type``.

Methods inherited from `pandas.core.dtypes.base.StorageExtensionDtype`:

`__str__(self)` -> str
Return `str(self)`.

Data and other attributes inherited from `pandas.core.dtypes.base.StorageExtensionDtype`:

`__annotations__` = {}

Methods inherited from `pandas.api.extensions.ExtensionDtype`:

`__ne__(self, other: object)` -> bool from `pandas.core.dtypes.base.ExtensionDtype`
Return `self!=value`.

`empty(self, shape: Shape)` -> `ExtensionArray` from `pandas.core.dtypes.base.ExtensionDtype`
Construct an `ExtensionArray` of this dtype with the given shape.

Analogous to `numpy.empty`.

Parameters

`shape` : int or tuple[int]

```

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

    Parameters
    -----
    dtype : object
        The object to check.

    Returns
    -----
    bool

    Notes
    -----
    The default implementation is True if

    1. ``cls.construct_from_string(dtype)`` is an instance
       of ``cls``.
    2. ``dtype`` is an object and is an instance of ``cls``
    3. ``dtype`` has a ``dtype`` attribute, and any of the above
       conditions is true for ``dtype.dtype``.

-----
Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

kind
    A character code (one of 'biufcmMOSUV'), default 'O'

    This should match the NumPy dtype used when the array is
    converted to an ndarray, which is probably 'O' for object if
    the extension type cannot be represented as a built-in NumPy
    type.

    See Also
    -----
    numpy.dtype.kind

names
    Ordered list of field names, or None if there are no fields.

    This is for compatibility with NumPy arrays, and may be removed in the
    future.

-----
Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

index_class
    The Index subclass to return from Index.__new__ when this dtype is
    encountered.
class Timedelta(pandas._libs.tslib.timedeltaas.Timedelta)
    Timedelta(value=<object object at 0x000001A9511B16C0>, unit=None, **kwargs)

    Represents a duration, the difference between two dates or times.

    Timedelta is the pandas equivalent of python's ``datetime.timedelta``
    and is interchangeable with it in most cases.

    Parameters
    -----
    value : Timedelta, timedelta, np.timedelta64, str, int or float
        Input value.
    unit : str, default 'ns'
        If input is an integer, denote the unit of the input.

```

If input is a float, denote the unit of the integer parts.
The decimal parts with resolution lower than 1 nanosecond are ignored.

Possible values:

- * 'W', or 'D'
- * 'days', or 'day'
- * 'hours', 'hour', 'hr', or 'h'
- * 'minutes', 'minute', 'min', or 'm'
- * 'seconds', 'second', 'sec', or 's'
- * 'milliseconds', 'millisecond', 'millis', 'milli', or 'ms'
- * 'microseconds', 'microsecond', 'micros', 'micro', or 'us'
- * 'nanoseconds', 'nanosecond', 'nanos', 'nano', or 'ns'.

.. deprecated:: 3.0.0

Allowing the values `w`, `d`, `MIN`, `MS`, `US` and `NS` to denote units
are deprecated in favour of the values `W`, `D`, `min`, `ms`, `us` and `ns`.

****kwargs**

Available kwargs: {days, seconds, microseconds,
milliseconds, minutes, hours, weeks}.

Values for construction in compat with datetime.timedelta.

Numpy ints and floats will be coerced to python ints and floats.

See Also

Timestamp : Represents a single timestamp in time.
TimedeltaIndex : Immutable Index of timedelta64 data.
DateOffset : Standard kind of date increment used for a date range.
to_timedelta : Convert argument to timedelta.
datetime.timedelta : Represents a duration in the datetime module.
numpy.timedelta64 : Represents a duration compatible with NumPy.

Notes

The constructor may take in either both values of value and unit or
kwargs as above. Either one of them must be used during initialization

The ``.value`` attribute is always in ns.

If the precision is higher than nanoseconds, the precision of the duration is
truncated to nanoseconds.

Examples

Here we initialize Timedelta object with both value and unit

```
>>> td = pd.Timedelta(1, "D")
>>> td
Timedelta('1 days 00:00:00')
```

Here we initialize the Timedelta object with kwargs

```
>>> td2 = pd.Timedelta(days=1)
>>> td2
Timedelta('1 days 00:00:00')
```

We see that either way we get the same result

Method resolution order:

```
Timedelta
pandas._libs.tslibs.timedeltas._Timedelta
datetime.timedelta
builtins.object
```

Methods defined here:

```
__abs__(self)
__add__(self, other)
__divmod__(self, other)
__floordiv__(self, other)
__mod__(self, other)
```

```

__mul__(self, other)
__neg__(self)
__pos__(self)
__radd__(self, other)
__rdivmod__(self, other)
__reduce__(self)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__ = __mul__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__setstate__(self, state)
__sub__(self, other)
__truediv__(self, other)

ceil(self, freq)
    Return a new Timedelta ceiled to this resolution.

    Parameters
    -----
    freq : str
        Frequency string indicating the ceiling resolution. Must be a fixed
        frequency like 's' (second) not 'ME' (month end). See
        :ref:`frequency aliases <timeseries.offset_aliases>` for
        a list of possible `freq` values.

    Returns
    -----
    Timedelta
        A new Timedelta object ceiled to the specified resolution.

    See Also
    -----
    Timedelta.floor : Floor the Timedelta to the specified resolution.
    Timedelta.round : Round the Timedelta to the nearest specified resolution.

    Examples
    -----
    >>> td = pd.Timedelta('1001ms')
    >>> td
    Timedelta('0 days 00:00:01.001000')
    >>> td.ceil('s')
    Timedelta('0 days 00:00:02')

floor(self, freq)
    Return a new Timedelta floored to this resolution.

    Parameters
    -----
    freq : str
        Frequency string indicating the flooring resolution.
        It uses the same units as class constructor :class:`~pandas.Timedelta`.

    Returns
    -----
    Timedelta
        A new Timedelta object floored to the specified resolution.

    See Also
    -----
    Timestamp.ceil : Round the Timestamp up to the nearest specified resolution.
    Timestamp.round : Round the Timestamp to the nearest specified resolution.

```

Examples

```
-----
>>> td = pd.Timedelta('1001ms')
>>> td
Timedelta('0 days 00:00:01.001000')
>>> td.floor('s')
Timedelta('0 days 00:00:01')
```

round(self, freq)
Round the Timedelta to the specified resolution.

Parameters

```
-----
freq : str
    Frequency string indicating the rounding resolution.
    It uses the same units as class constructor :class:`~pandas.Timedelta`.
```

Returns

```
-----
a new Timedelta rounded to the given resolution of `freq`
```

Raises

```
-----
ValueError if the freq cannot be converted
```

See Also

```
-----
Timedelta.floor : Floor the Timedelta to the specified resolution.
Timedelta.round : Round the Timedelta to the nearest specified resolution.
Timestamp.ceil : Similar method for Timestamp objects.
```

Examples

```
-----
>>> td = pd.Timedelta('1001ms')
>>> td
Timedelta('0 days 00:00:01.001000')
>>> td.round('s')
Timedelta('0 days 00:00:01')
```

Static methods defined here:

```
__new__(cls, value=<object object at 0x000001A9511B16C0>, unit=None, **kwargs)
```

Data descriptors defined here:

```
__dict__
    dictionary for instance variables
```

```
__weakref__
    list of weak references to the object
```

Methods inherited from pandas._libs.tslibs.timedeltas._Timedelta:

```
__bool__(self, /)
    True if self else False
```

```
__eq__(self, value, /)
    Return self==value.
```

```
__ge__(self, value, /)
    Return self>=value.
```

```
__gt__(self, value, /)
    Return self>value.
```

```
__hash__(self, /)
    Return hash(self).
```

```
__le__(self, value, /)
    Return self<=value.
```

```
__lt__(self, value, /)
    Return self<value.
```

```

__ne__(self, value, /)
    Return self!=value.

__reduce_cython__(self)

__repr__(self, /)
    Return repr(self).

__setstate_cython__(self, __pyx_state)

__str__(self, /)
    Return str(self).

as_unit(self, unit, round_ok=True)
    Convert the underlying int64 representation to the given unit.

    Parameters
    -----
    unit : {"ns", "us", "ms", "s"}
    round_ok : bool, default True
        If False and the conversion requires rounding, raise.

    Returns
    -----
    Timedelta

    See Also
    -----
    Timedelta : Represents a duration, the difference between two dates or times.
    to_timedelta : Convert argument to timedelta.
    Timedelta.asm8 : Return a numpy timedelta64 array scalar view.

    Examples
    -----
    >>> td = pd.Timedelta('1001ms')
    >>> td
    Timedelta('0 days 00:00:01.001000')
    >>> td.as_unit('s')
    Timedelta('0 days 00:00:01')

isoformat(self) -> str
    Format the Timedelta as ISO 8601 Duration.

    ``P[n]Y[n]M[n]DT[n]H[n]M[n]S`` , where the ``[n]`` s are replaced by the
    values. See Wikipedia:
    `ISO 8601 § Durations <https://en.wikipedia.org/wiki/ISO_8601#Durations>`_.

    Returns
    -----
    str

    See Also
    -----
    Timestamp.isoformat : Function is used to convert the given
        Timestamp object into the ISO format.

    Notes
    -----
    The longest component is days, whose value may be larger than
    365.
    Every component is always included, even if its value is 0.
    Pandas uses nanosecond precision, so up to 9 decimal places may
    be included in the seconds component.
    Trailing 0's are removed from the seconds component after the decimal.
    We do not 0 pad components, so it's `...T5H...`, not `...T05H...`

    Examples
    -----
    >>> td = pd.Timedelta(days=6, minutes=50, seconds=3,
    ...                    milliseconds=10, microseconds=10, nanoseconds=12)

    >>> td.isoformat()
    'P6DT0H50M3.010010012S'
    >>> pd.Timedelta(hours=1, seconds=10).isoformat()
    'P0DT1H0M10S'
    >>> pd.Timedelta(days=500.5).isoformat()
    'P500DT12H0M0S'

```



```

to_numpy(self, dtype=None, copy=False) -> np.timedelta64
    Convert the Timedelta to a NumPy timedelta64.

    This is an alias method for `Timedelta.to_timedelta64()`.

Parameters
-----
dtype : NoneType
    It is available here only for compatibility. Its value will not
    affect the return value.
copy : bool, default False
    It is available here only for compatibility. Its value will not
    affect the return value.

Returns
-----
numpy.timedelta64

See Also
-----
Series.to_numpy : Similar method for Series.

Examples
-----
>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.to_numpy()
numpy.timedelta64(259200000000000, 'ns')

```

to_pytimedelta(self)
 Convert a pandas Timedelta object into a python ``datetime.timedelta`` object.

Timedelta objects are internally saved as numpy datetime64[ns] dtype.
 Use to_pytimedelta() to convert to object dtype.

Returns

 datetime.timedelta or numpy.array of datetime.timedelta

See Also

 to_timedelta : Convert argument to Timedelta type.

Notes

 Any nanosecond resolution will be lost.

Examples

 >>> td = pd.Timedelta('3D')
 >>> td
 Timedelta('3 days 00:00:00')
 >>> td.to_pytimedelta()
 datetime.timedelta(days=3)

to_timedelta64(self) -> np.timedelta64
 Return a numpy.timedelta64 object with 'ns' precision.

Since NumPy uses ``timedelta64`` objects for its time operations, converting
 a pandas ``Timedelta`` into a NumPy ``timedelta64`` provides seamless
 integration between the two libraries, especially when working in environments
 that heavily rely on NumPy for array-based calculations.

See Also

 to_timedelta : Convert argument to timedelta.
 numpy.timedelta64 : A NumPy object for time duration.
 Timedelta : Represents a duration, the difference between two dates
 or times.

Examples

 >>> td = pd.Timedelta('3D')
 >>> td
 Timedelta('3 days 00:00:00')

```

>>> td.to_timedelta64()
numpy.timedelta64(259200000000000, 'ns')

```

total_seconds(self) -> float
Total seconds in the duration.

This method calculates the total duration in seconds by combining the days, seconds, and microseconds of the `Timedelta` object.

See Also

`to_timedelta` : Convert argument to `timedelta`.
`Timedelta` : Represents a duration, the difference between two dates or times.
`Timedelta.seconds` : Returns the seconds component of the `timedelta`.
`Timedelta.microseconds` : Returns the microseconds component of the `timedelta`.

Examples

```

>>> td = pd.Timedelta('1min')
>>> td
Timedelta('0 days 00:01:00')
>>> td.total_seconds()
60.0

```

view(self, dtype)
Array view compatibility.

This method allows you to reinterpret the underlying data of a `Timedelta` object as a different dtype. The `view` method provides a way to reinterpret the internal representation of the `Timedelta` object without modifying its data. This is particularly useful when you need to work with the underlying data directly, such as for performance optimizations or interfacing with low-level APIs. The returned value is typically the number of nanoseconds since the epoch, represented as an integer or another specified dtype.

Parameters

`dtype` : str or dtype
The dtype to view the underlying data as.

See Also

`Timedelta.asm8` : Return a numpy `timedelta64` array scalar view.
`numpy.ndarray.view` : Returns a view of an array with the same data.
`Timedelta.to_numpy` : Converts the `Timedelta` to a NumPy `timedelta64`.
`Timedelta.total_seconds` : Returns the total duration of the `Timedelta` object in seconds.

Examples

```

>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.view(int)
259200000000000

```

Data descriptors inherited from `pandas._libs.tslibs.timedeltas._Timedelta`:

`asm8`
Return a numpy `timedelta64` array scalar view.

Provides access to the array scalar view (i.e. a combination of the value and the units) associated with the `numpy.timedelta64().view()`, including a 64-bit integer representation of the `timedelta` in nanoseconds (Python `int` compatible).

Returns

numpy `timedelta64` array scalar view
Array scalar view of the `timedelta` in nanoseconds.

See Also

`Timedelta.total_seconds` : Return the total seconds in the duration.
`Timedelta.components` : Return a namedtuple of the `Timedelta`'s components.
`Timedelta.to_timedelta64` : Convert the `Timedelta` to a `numpy.timedelta64`.

Examples

```
-----
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
>>> td.asm8
numpy.timedelta64(86520000003042, 'ns')

>>> td = pd.Timedelta('2 min 3 s')
>>> td.asm8
numpy.timedelta64(123000000000, 'ns')

>>> td = pd.Timedelta('3 ms 5 us')
>>> td.asm8
numpy.timedelta64(3005000, 'ns')

>>> td = pd.Timedelta(42, unit='ns')
>>> td.asm8
numpy.timedelta64(42, 'ns')
```

components

Return a components namedtuple-like.

Each component represents a different time unit, allowing you to access the breakdown of the total duration in terms of days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds.

See Also

```
-----
Timedelta.total_seconds : Returns the total duration of the Timedelta in
seconds.
to_timedelta : Convert argument to Timedelta.
Timedelta : Represents a duration, the difference between two dates or times.
```

Examples

```
-----
>>> td = pd.Timedelta('2 day 4 min 3 us 42 ns')
>>> td.components
Components(days=2, hours=0, minutes=4, seconds=0, milliseconds=0,
microseconds=3, nanoseconds=42)
```

days

Returns the days of the timedelta.

The ``days`` attribute of a ``pandas.Timedelta`` object provides the number of days represented by the ``Timedelta``. This is useful for extracting the day component from a ``Timedelta`` that may also include hours, minutes, seconds, and smaller time units. This attribute simplifies the process of working with durations where only the day component is of interest.

Returns

```
-----
int
```

See Also

```
-----
Timedelta.seconds : Returns the seconds component of the timedelta.
Timedelta.microseconds : Returns the microseconds component of the timedelta.
Timedelta.total_seconds : Returns the total duration in seconds.
```

Examples

```
-----
>>> td = pd.Timedelta(1, "d")
>>> td.days
1

>>> td = pd.Timedelta('4 min 3 us 42 ns')
>>> td.days
0
```

microseconds

Return the number of microseconds (n), where $0 \leq n < 1$ millisecond.

`Timedelta.microseconds = milliseconds * 1000 + microseconds.`

Returns

```
-----
int
```

Number of microseconds.

See Also

Timedelta.components : Return all attributes with assigned values
(i.e. days, hours, minutes, seconds, milliseconds, microseconds,
nanoseconds).

Examples

Using string input

```
>>> td = pd.Timedelta('1 days 2 min 3 us')
```

```
>>> td.microseconds
3
```

Using integer input

```
>>> td = pd.Timedelta(42, unit='us')
>>> td.microseconds
42
```

nanoseconds

Return the number of nanoseconds (n), where $0 \leq n < 1$ microsecond.

Returns

int
Number of nanoseconds.

See Also

Timedelta.components : Return all attributes with assigned values
(i.e. days, hours, minutes, seconds, milliseconds, microseconds,
nanoseconds).

Examples

Using string input

```
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
```

```
>>> td.nanoseconds
42
```

Using integer input

```
>>> td = pd.Timedelta(42, unit='ns')
>>> td.nanoseconds
42
```

resolution_string

Return a string representing the lowest timedelta resolution.

Each timedelta has a defined resolution that represents the lowest OR
most granular level of precision. Each level of resolution is
represented by a short string as defined below:

Resolution:	Return value
-------------	--------------

* Days:	'D'
* Hours:	'h'
* Minutes:	'min'
* Seconds:	's'
* Milliseconds:	'ms'
* Microseconds:	'us'
* Nanoseconds:	'ns'

Returns

str
Timedelta resolution.

Examples

>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')

```

>>> td.resolution_string
'ns'

>>> td = pd.Timedelta('1 days 2 min 3 us')
>>> td.resolution_string
'us'

>>> td = pd.Timedelta('2 min 3 s')
>>> td.resolution_string
's'

>>> td = pd.Timedelta(36, unit='us')
>>> td.resolution_string
'us'

```

seconds

Return the total hours, minutes, and seconds of the timedelta as seconds.

`Timedelta.seconds = hours * 3600 + minutes * 60 + seconds.`

Returns

int

Number of seconds.

See Also

`Timedelta.components` : Return all attributes with assigned values (i.e. days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds).

`Timedelta.total_seconds` : Express the Timedelta as total number of seconds.

Examples

****Using string input****

```

>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
>>> td.seconds
120

```

****Using integer input****

```

>>> td = pd.Timedelta(42, unit='s')
>>> td.seconds
42

```

unit

Return the unit of Timedelta object.

The unit of Timedelta object is nanosecond, i.e., 'ns' by default.

Returns

str

See Also

`Timedelta.value` : Return the value of Timedelta object in nanoseconds.

`Timedelta.as_unit` : Convert the underlying int64 representation to the given unit.

Examples

```

>>> td = pd.Timedelta(42, unit='us')
'ns'

```

value

Return the value of Timedelta object in nanoseconds.

Return the total seconds, milliseconds and microseconds of the timedelta as nanoseconds.

Returns

int

See Also

Timedelta.unit : Return the unit of Timedelta object.

Examples

>>> pd.Timedelta(1, "us").value
1000

Data and other attributes inherited from pandas._libs.tslibs.timedeltas._Timedelta:

__array_priority__ = 100

__pyx_vtable__ = <capsule object NULL>

max = Timedelta('106751 days 23:47:16.854775807')
Returns the maximum bound possible for Timedelta.

This property provides access to the largest possible value that can be represented by a Timedelta object.

Returns

Timedelta

See Also

Timedelta.min: Returns the minimum bound possible for Timedelta.
Timedelta.resolution: Returns the smallest possible difference between non-equal Timedelta objects.

Examples

>>> pd.Timedelta.max
106751 days 23:47:16.854775807

min = Timedelta('-106752 days +00:12:43.145224193')
Returns the minimum bound possible for Timedelta.

This property provides access to the smallest possible value that can be represented by a Timedelta object.

Returns

Timedelta

See Also

Timedelta.max: Returns the maximum bound possible for Timedelta.
Timedelta.resolution: Returns the smallest possible difference between non-equal Timedelta objects.

Examples

>>> pd.Timedelta.min
-106752 days +00:12:43.145224193

resolution = Timedelta('0 days 00:00:00.000000001')
Returns the smallest possible difference between non-equal Timedelta objects.

The resolution value is determined by the underlying representation of time units and is equivalent to Timedelta(nanoseconds=1).

Returns

Timedelta

See Also

Timedelta.max: Returns the maximum bound possible for Timedelta.
Timedelta.min: Returns the minimum bound possible for Timedelta.

Examples

>>> pd.Timedelta.resolution

```
|      0 days 00:00:00.000000001
```

```
class TimedeltaIndex(pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin)
|   TimedeltaIndex(
|       data=None,
|       freq=<no_default>,
|       dtype=None,
|       copy: bool | None = None,
|       name=None
|   )
|
|   Immutable Index of timedelta64 data.
|
|   Represented internally as int64, and scalars returned Timedelta objects.
|
|   Parameters
|   -----
|   data : array-like (1-dimensional), optional
|       Optional timedelta-like data to construct index with.
|   freq : str or pandas offset object, optional
|       One of pandas date offset strings or corresponding objects. The string
|       ``'infer'`` can be passed in order to set the frequency of the index as
|       the inferred frequency upon creation.
|   dtype : numpy.dtype or str, default None
|       Valid ``numpy`` dtypes are ``timedelta64[ns]``, ``timedelta64[us]``,
|       ``timedelta64[ms]``, and ``timedelta64[s]``.
|   copy : bool, default None
|       Whether to copy input data, only relevant for array, Series, and Index
|       inputs (for other input, e.g. a list, a new array is created anyway).
|       Defaults to True for array input and False for Index/Series.
|       Set to False to avoid copying array input at your own risk (if you
|       know the input data won't be modified elsewhere).
|       Set to True to force copying Series/Index input up front.
|   name : object
|       Name to be stored in the index.
|
|   Attributes
|   -----
|   days
|   seconds
|   microseconds
|   nanoseconds
|   components
|   inferred_freq
|
|   Methods
|   -----
|   to_pytimedelta
|   to_series
|   round
|   floor
|   ceil
|   to_frame
|   mean
|
|   See Also
|   -----
|   Index : The base pandas Index type.
|   Timedelta : Represents a duration between two dates or times.
|   DatetimeIndex : Index of datetime64 data.
|   PeriodIndex : Index of Period data.
|   timedelta_range : Create a fixed-frequency TimedeltaIndex.
|
|   Notes
|   ----
|   To learn more about the frequency strings, please see
|   :ref:`this link<timeseries.offset_aliases>`.
|
|   Examples
|   -----
|   >>> pd.TimedeltaIndex(["0 days", "1 days", "2 days", "3 days", "4 days"])
|   TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'],
|                   dtype='timedelta64[us]', freq=None)
|
|   We can also let pandas infer the frequency when possible.
|
|   >>> pd.TimedeltaIndex(np.arange(5) * 24 * 3600 * 1e9, freq="infer")
```

```
TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'],
               dtype='timedelta64[ns]', freq='D')
```

Method resolution order:

```
TimedeltaIndex
pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin
pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin
pandas.core.indexes.extension.NDArrayBackedExtensionIndex
pandas.core.indexes.extension.ExtensionIndex
Index
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
abc.ABC
builtins.object
```

Methods defined here:

```
__abs__(self) -> TimedeltaArray from pandas.core.indexes.extension._inherit_from_data.<locals>
# error: Incompatible redefinition (redefinition with type "Callable[[Any,
# VarArg(Any), KwArg(Any)], Any]", original type "property")
```

```
__neg__(self) -> TimedeltaArray from pandas.core.indexes.extension._inherit_from_data.<locals>
# error: Incompatible redefinition (redefinition with type "Callable[[Any,
# VarArg(Any), KwArg(Any)], Any]", original type "property")
```

```
__pos__(self) -> TimedeltaArray from pandas.core.indexes.extension._inherit_from_data.<locals>
# error: Incompatible redefinition (redefinition with type "Callable[[Any,
# VarArg(Any), KwArg(Any)], Any]", original type "property")
```

```
ceil(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
Perform ceil operation on the data to the specified `freq`.
```

Parameters

freq : str or Offset

The frequency level to ceil the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series

Index of the same type for a DatetimeIndex or TimedeltaIndex, or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :

Perform floor operation on the data to the specified `freq`.

DatetimeIndex.snap :

Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
```

```
>>> rng
```

```
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',  
               '2018-01-01 12:01:00'],  
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.ceil('h')
```

```
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',  
               '2018-01-01 13:00:00'],  
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.ceil("h")
```

```
0    2018-01-01 12:00:00
```

```
1    2018-01-01 12:00:00
```

```
2    2018-01-01 13:00:00
```

```
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 01:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.ceil("h", ambiguous=False)
```

```
DatetimeIndex(['2021-10-31 02:00:00+01:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.ceil("h", ambiguous=True)
```

```
DatetimeIndex(['2021-10-31 02:00:00+02:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

floor(

self,

freq,

ambiguous: TimeAmbiguous = 'raise',

nonexistent: TimeNonexistent = 'raise'

) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>

Perform floor operation on the data to the specified `freq`.

Parameters

freq : str or Offset

The frequency level to floor the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'

Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)

- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta,
A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

default '

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series

Index of the same type for a DatetimeIndex or TimedeltaIndex,
or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :

Perform floor operation on the data to the specified `freq`.

DatetimeIndex.snap :

Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, flooring will take place relative to the local ("wall") time and re-localized to the same timezone. When flooring near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
```

```
>>> rng
```

```
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.floor('h')
```

```
DatetimeIndex(['2018-01-01 11:00:00', '2018-01-01 12:00:00',
               '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.floor("h")
```

```
0    2018-01-01 11:00:00
1    2018-01-01 12:00:00
2    2018-01-01 12:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
```

```
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
```

```
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```

get_loc(self, key) from pandas.core.indexes.timedeltas.TimedeltaIndex
    Get integer location for requested label

Returns
-----
loc : int, slice, or ndarray[int]

median(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs) from pandas.core
    # error: Incompatible redefinition (redefinition with type "Callable[[Any,
    # VarArg(Any), KwArg(Any)], Any]", original type "property")

round(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.indexes.extension._inherit_from_data.<locals>
    Perform round operation on the data to the specified `freq`.

Parameters
-----
freq : str or Offset
    The frequency level to round the index to. Must be a fixed
    frequency like 's' (second) not 'ME' (month end). See
    :ref:`frequency aliases <timeseries.offset_aliases>` for
    a list of possible `freq` values.
ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
    Only relevant for DatetimeIndex:

    - 'infer' will attempt to infer fall dst-transition hours based on
      order
    - bool-ndarray where True signifies a DST time, False designates
      a non-DST time (note that this flag is only applicable for
      ambiguous times)
    - 'NaT' will return NaT where there are ambiguous times
    - 'raise' will raise a ValueError if there are ambiguous
      times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default '
    A nonexistent time does not exist in a particular timezone
    where clocks moved forward due to DST.

    - 'shift_forward' will shift the nonexistent time forward to the
      closest existing time
    - 'shift_backward' will shift the nonexistent time backward to the
      closest existing time
    - 'NaT' will return NaT where there are nonexistent times
    - timedelta objects will shift nonexistent times by the timedelta
    - 'raise' will raise a ValueError if there are
      nonexistent times.

Returns
-----
DatetimeIndex, TimedeltaIndex, or Series
    Index of the same type for a DatetimeIndex or TimedeltaIndex,
    or a Series with the same index for a Series.

Raises
-----
ValueError if the `freq` cannot be converted.

See Also
-----
DatetimeIndex.floor :
    Perform floor operation on the data to the specified `freq`.
DatetimeIndex.snap :
    Snap time stamps to nearest occurring frequency.

Notes
-----
If the timestamps have a timezone, rounding will take place relative to the
local ("wall") time and re-localized to the same timezone. When rounding
near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
control the re-localization behavior.

Examples
-----

```

```
**DatetimeIndex**
```

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
```

```
>>> rng
```

```
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',  
               '2018-01-01 12:01:00'],  
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.round('h')
```

```
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',  
               '2018-01-01 12:00:00'],  
              dtype='datetime64[us]', freq=None)
```

```
**Series**
```

```
>>> pd.Series(rng).dt.round("h")
```

```
0    2018-01-01 12:00:00
```

```
1    2018-01-01 12:00:00
```

```
2    2018-01-01 12:00:00
```

```
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
```

```
DatetimeIndex(['2021-10-31 02:00:00+01:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
```

```
DatetimeIndex(['2021-10-31 02:00:00+02:00'],  
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
std(
```

```
    self,
```

```
    *,
```

```
    axis: AxisInt | None = None,
```

```
    dtype: NpDtype | None = None,
```

```
    out=None,
```

```
    ddof: int = 1,
```

```
    keepdims: bool = False,
```

```
    skipna: bool = True
```

```
) from pandas.core.indexes.extension._inherit_from_data.<locals>
```

```
# error: Incompatible redefinition (redefinition with type "Callable[[Any,  
# VarArg(Any), KwArg(Any)], Any]", original type "property")
```

```
sum(
```

```
    self,
```

```
    *,
```

```
    axis: AxisInt | None = None,
```

```
    dtype: NpDtype | None = None,
```

```
    out=None,
```

```
    keepdims: bool = False,
```

```
    initial=None,
```

```
    skipna: bool = True,
```

```
    min_count: int = 0
```

```
) from pandas.core.indexes.extension._inherit_from_data.<locals>
```

```
# error: Incompatible redefinition (redefinition with type "Callable[[Any,  
# VarArg(Any), KwArg(Any)], Any]", original type "property")
```

```
to_pytimedelta(self) -> npt.NDArray[np.object_] from pandas.core.indexes.extension._inherit_  
Return an ndarray of datetime.timedelta objects.
```

```
Returns
```

```
-----
```

```
numpy.ndarray
```

A NumPy ``timedelta64`` object representing the same duration as the original pandas ``Timedelta`` object. The precision of the resulting object is in nanoseconds, which is the default time resolution used by pandas for ``Timedelta`` objects, ensuring high precision for time-based calculations.

```
See Also
```

```
-----
```

```
to_timedelta : Convert argument to timedelta format.
```

```
Timedelta : Represents a duration between two dates or times.
```

DatetimeIndex: Index of datetime64 data.
Timedelta.components : Return a components namedtuple-like
of a single timedelta.

Examples

```
-----
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
               dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.to_pytimedelta()
array([datetime.timedelta(days=1), datetime.timedelta(days=2),
       datetime.timedelta(days=3)], dtype=object)

>>> tidx = pd.TimedeltaIndex(data=["1 days 02:30:45", "3 days 04:15:10"])
>>> tidx
TimedeltaIndex(['1 days 02:30:45', '3 days 04:15:10'],
               dtype='timedelta64[us]', freq=None)
>>> tidx.to_pytimedelta()
array([datetime.timedelta(days=1, seconds=9045),
       datetime.timedelta(days=3, seconds=15310)], dtype=object)
```

total_seconds(self) -> npt.NDArray[np.float64] from pandas.core.indexes.extension._inherit_f
Return total duration of each element expressed in seconds.

This method is available directly on TimedeltaArray, TimedeltaIndex
and on Series containing timedelta values under the ``.dt`` namespace.

Returns

ndarray, Index or Series
When the calling object is a TimedeltaArray, the return type
is ndarray. When the calling object is a TimedeltaIndex,
the return type is an Index with a float64 dtype. When the calling object
is a Series, the return type is Series of type `float64` whose
index is the same as the original.

See Also

datetime.timedelta.total_seconds : Standard library version
of this method.
TimedeltaIndex.components : Return a DataFrame with components of
each Timedelta.

Examples

```
-----
**Series**

>>> s = pd.Series(pd.to_timedelta(np.arange(5), unit="D"))
>>> s
0    0 days
1    1 days
2    2 days
3    3 days
4    4 days
dtype: timedelta64[s]

>>> s.dt.total_seconds()
0         0.0
1    86400.0
2   172800.0
3   259200.0
4   345600.0
dtype: float64

**TimedeltaIndex**

>>> idx = pd.to_timedelta(np.arange(5), unit="D")
>>> idx
TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'],
               dtype='timedelta64[s]', freq=None)

>>> idx.total_seconds()
Index([0.0, 86400.0, 172800.0, 259200.0, 345600.0], dtype='float64')
```

Static methods defined here:

```

__new__(
    cls,
    data=None,
    freq=<no_default>,
    dtype=None,
    copy: bool | None = None,
    name=None
) from pandas.core.indexes.timedeltas.TimedeltaIndex
    Create and return a new object. See help(type) for accurate signature.

```

Readonly properties defined here:

inferred_type
Return a string of the type inferred from the values.

See Also

Index.dtype : Return the dtype object of the underlying data.

Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.inferred_type
'integer'

```

Data descriptors defined here:

components
Return a DataFrame of the individual resolution components of the Timedeltas.

The components (days, hours, minutes seconds, milliseconds, microseconds, nanoseconds) are returned as columns in a DataFrame.

Returns

DataFrame

See Also

TimedeltaIndex.total_seconds : Return total duration expressed in seconds.
TimedeltaIndex.components : Return a components namedtuple-like of a single
timedelta.

Examples

```

>>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
>>> tdelta_idx
TimedeltaIndex(['1 days 00:03:00.000002042'],
                dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.components
   days  hours  minutes  seconds  milliseconds  microseconds  nanoseconds
0      1      0         3         0             0              2             42

```

days

Number of days for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.
Series.dt.microseconds : Return number of microseconds for each element.
Series.dt.nanoseconds : Return number of nanoseconds for each element.

Examples

For Series:

```

>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='D'))
>>> ser
0    1 days
1    2 days
2    3 days
dtype: timedelta64[s]

```

```
>>> ser.dt.days
```

```
0    1
1    2
2    3
dtype: int64
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
```

```
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
               dtype='timedelta64[us]', freq=None)
```

```
>>> tdelta_idx.days
Index([0, 10, 20], dtype='int64')
```

microseconds

Number of microseconds (≥ 0 and less than 1 second) for each element.

See Also

pd.Timedelta.microseconds : Number of microseconds (≥ 0 and less than 1 second).

pd.Timedelta.to_pytimedelta.microseconds : Number of microseconds (≥ 0 and less than 1 second) of a datetime.timedelta.

Examples

For Series:

```
>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='us'))
```

```
>>> ser
0    0 days 00:00:00.000001
1    0 days 00:00:00.000002
2    0 days 00:00:00.000003
dtype: timedelta64[us]
```

```
>>> ser.dt.microseconds
```

```
0    1
1    2
2    3
dtype: int32
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='us')
```

```
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:00.000001', '0 days 00:00:00.000002',
               '0 days 00:00:00.000003'],
               dtype='timedelta64[us]', freq=None)
```

```
>>> tdelta_idx.microseconds
Index([1, 2, 3], dtype='int32')
```

nanoseconds

Number of nanoseconds (≥ 0 and less than 1 microsecond) for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.

Series.dt.microseconds : Return number of nanoseconds for each element.

Examples

For Series:

```
>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='ns'))
```

```
>>> ser
0    0 days 00:00:00.000000001
1    0 days 00:00:00.000000002
2    0 days 00:00:00.000000003
dtype: timedelta64[ns]
```

```
>>> ser.dt.nanoseconds
```

```
0    1
1    2
2    3
dtype: int32
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='ns')
```

```
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:00.000000001', '0 days 00:00:00.000000002',
               '0 days 00:00:00.000000003'],
              dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.nanoseconds
Index([1, 2, 3], dtype='int32')
```

seconds

Number of seconds (≥ 0 and less than 1 day) for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.
Series.dt.nanoseconds : Return number of nanoseconds for each element.

Examples

For Series:

```
>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='s'))
>>> ser
0    0 days 00:00:01
1    0 days 00:00:02
2    0 days 00:00:03
dtype: timedelta64[s]
>>> ser.dt.seconds
0     1
1     2
2     3
dtype: int32
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='s')
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:01', '0 days 00:00:02', '0 days 00:00:03'],
              dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.seconds
Index([1, 2, 3], dtype='int32')
```

Data and other attributes defined here:

```
__abstractmethods__ = frozenset()
__annotations__ = {'_data': 'TimedeltaArray'}
```

Methods inherited from pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin:

as_unit(self, unit: TimeUnit) -> Self
Convert to a dtype with the given unit resolution.

This method is for converting the dtype of a ``DatetimeIndex`` or
``TimedeltaIndex`` to a new dtype with the given unit
resolution/precision.

Parameters

unit : {'s', 'ms', 'us', 'ns'}

Returns

same type as self
Converted to the specified unit.

See Also

Timestamp.as_unit : Convert to the given unit.
Timedelta.as_unit : Convert to the given unit.
DatetimeIndex.as_unit : Convert to the given unit.
TimedeltaIndex.as_unit : Convert to the given unit.

Examples

For :class:`pandas.DatetimeIndex`:


```

>>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])
>>> idx
DatetimeIndex(['2020-01-02 01:02:03.004005006'],
              dtype='datetime64[ns]', freq=None)
>>> idx.as_unit("s")
DatetimeIndex(['2020-01-02 01:02:03'], dtype='datetime64[s]', freq=None)

For :class:`pandas.TimedeltaIndex`:

>>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
>>> tdelta_idx
TimedeltaIndex(['1 days 00:03:00.000002042'],
              dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.as_unit("s")
TimedeltaIndex(['1 days 00:03:00'], dtype='timedelta64[s]', freq=None)

delete(self, loc) -> Self
    Make new Index with passed location(-s) deleted.

Parameters
-----
loc : int or list of int
    Location of item(-s) which will be deleted.
    Use a list of locations to delete more than one value at the same time.

Returns
-----
Index
    Will be same type as self, except for RangeIndex.

See Also
-----
numpy.delete : Delete any rows and column from NumPy array (ndarray).

Examples
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete(1)
Index(['a', 'c'], dtype='str')
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.delete([0, 2])
Index(['b'], dtype='str')

insert(self, loc: int, item)
    Make new Index inserting new item at location.
    Follows Python numpy.insert semantics for negative values.

Parameters
-----
loc : int
    The integer location where the new item will be inserted.
item : object
    The new item to be inserted into the Index.

Returns
-----
Index
    Returns a new Index object resulting from inserting the specified item at
    the specified location within the original Index.

See Also
-----
Index.append : Append a collection of Indexes together.

Examples
-----
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.insert(1, "x")
Index(['a', 'x', 'b', 'c'], dtype='str')

shift(self, periods: int = 1, freq=None) -> Self
    Shift index by desired number of time frequency increments.
    This method is for shifting the values of datetime-like indexes
    by a specified time increment a given number of times.

Parameters
-----

```

periods : int, default 1
 Number of periods (or increments) to shift by,
 can be positive or negative.
 freq : pandas.DateOffset, pandas.Timedelta or string, optional
 Frequency increment to shift by.
 If None, the index is shifted by its own `freq` attribute.
 Offset aliases are valid strings, e.g., 'D', 'W', 'M' etc.

Returns

 pandas.DatetimeIndex
 Shifted index.

See Also

 Index.shift : Shift values of Index.
 PeriodIndex.shift : Shift values of PeriodIndex.

```
take(
    self,
    indices,
    axis: Axis = 0,
    allow_fill: bool = True,
    fill_value=None,
    **kwargs
) -> Self
```

Return a new Index of the values selected by the indices.
 For internal compatibility with numpy arrays.

Parameters

 indices : array-like
 Indices to be taken.
 axis : {0 or 'index'}, optional
 The axis over which to select values, always 0 or 'index'.
 allow_fill : bool, default True
 How to handle negative values in `indices`.
 * False: negative values in `indices` indicate positional indices
 from the right (the default). This is similar to
 :func:`numpy.take`.
 * True: negative values in `indices` indicate
 missing values. These values are set to `fill\_value`. Any other
 other negative values raise a ``ValueError``.
 fill_value : scalar, default None
 If allow_fill=True and fill_value is not None, indices specified by
 -1 are regarded as NA. If Index doesn't hold NA, raise ValueError.
 **kwargs
 Required for compatibility with numpy.

Returns

 Index
 An index formed of elements at the given indices. Will be the same
 type as self, except for RangeIndex.

See Also

 numpy.ndarray.take: Return an array formed from the
 elements of a at the given indices.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.take([2, 2, 1, 2])
Index(['c', 'c', 'b', 'c'], dtype='str')
```

 Readonly properties inherited from pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin:

unit

values

Return an array representing the data in the Index.

.. warning::

We recommend using :attr:`Index.array` or

:meth:`Index.to\_numpy`, depending on whether you need a reference to the underlying data or a NumPy array.

.. versionchanged:: 3.0.0

The returned array is read-only.

Returns

array: numpy.ndarray or ExtensionArray

See Also

Index.array : Reference to the underlying data.
Index.to_numpy : A NumPy array representing the underlying data.

Examples

For :class:`pandas.Index`:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.values
array([1, 2, 3])
```

For :class:`pandas.IntervalIndex`:

```
>>> idx = pd.interval_range(start=0, end=5)
>>> idx.values
<IntervalArray>
[(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]]
Length: 5, dtype: interval[int64, right]
```

Data descriptors inherited from pandas.core.indexes.datetimelike.DatetimeTimedeltaMixin:

inferred_freq

Return the inferred frequency of the index.

Returns

str or None

A string representing a frequency generated by ``infer\_freq``.
Returns ``None`` if the frequency cannot be inferred.

See Also

DatetimeIndex.freqstr : Return the frequency object as a string if it's set, otherwise ``None``.

Examples

For ``DatetimeIndex``:

```
>>> idx = pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
>>> idx.inferred_freq
'2D'
```

For ``TimedeltaIndex``:

```
>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
               dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.inferred_freq
'10D'
```

Methods inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

__contains__(self, key: Any) -> bool

Return a boolean indicating whether the provided key is in the index.

Parameters

key : label

```

        The key to check if it is present in the index.

Returns
-----
bool
    Whether the key search is in the index.

Raises
-----
TypeError
    If the key is not hashable.

See Also
-----
Index.isin : Returns an ndarray of boolean dtype indicating whether the
    list-like key is in the index.

Examples
-----
>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> 2 in idx
True
>>> 6 in idx
False

equals(self, other: Any) -> bool
    Determines if two Index objects contain the same elements.

mean(self, *, skipna: bool = True, axis: int | None = 0)
    Return the mean value of the Array.

Parameters
-----
skipna : bool, default True
    Whether to ignore any NaT elements.
axis : int, optional, default 0
    Axis for the function to be applied on.

Returns
-----
scalar
    Timestamp or Timedelta.

See Also
-----
numpy.ndarray.mean : Returns the average of array elements along a given axis.
Series.mean : Return the mean value in a Series.

Notes
-----
mean is only defined for Datetime and Timedelta dtypes, not for Period.

Examples
-----
For :class:`pandas.DatetimeIndex`:

>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.mean()
Timestamp('2001-01-02 00:00:00')

For :class:`pandas.TimedeltaIndex`:

>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
              dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.mean()
Timedelta('2 days 00:00:00')
-----
Readonly properties inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

```

asi8

freqstr

Return the frequency object as a string if it's set, otherwise None.

See Also

DatetimeIndex.inferred_freq : Returns a string representing a frequency generated by infer_freq.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
>>> idx.freqstr
'D'
```

The frequency can be inferred if there are more than 2 points:

```
>>> idx = pd.DatetimeIndex(
...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
... )
>>> idx.freqstr
'2D'
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
>>> idx.freqstr
'M'
```

Data descriptors inherited from pandas.core.indexes.datetimelike.DatetimeIndexOpsMixin:

freq

Return the frequency object if it is set, otherwise None.

To learn more about the frequency strings, please see
:ref:`this link<timeseries.offset\_aliases>`.

See Also

DatetimeIndex.freq : Return the frequency object if it is set, otherwise None.
PeriodIndex.freq : Return the frequency object if it is set, otherwise None.

Examples

```
>>> datetimeindex = pd.date_range(
...     "2022-02-22 02:22:22", periods=10, tz="America/Chicago", freq="h"
... )
>>> datetimeindex
DatetimeIndex(['2022-02-22 02:22:22-06:00', '2022-02-22 03:22:22-06:00',
              '2022-02-22 04:22:22-06:00', '2022-02-22 05:22:22-06:00',
              '2022-02-22 06:22:22-06:00', '2022-02-22 07:22:22-06:00',
              '2022-02-22 08:22:22-06:00', '2022-02-22 09:22:22-06:00',
              '2022-02-22 10:22:22-06:00', '2022-02-22 11:22:22-06:00'],
              dtype='datetime64[us, America/Chicago]', freq='h')
>>> datetimeindex.freq
<Hour>
```

hasnans

resolution

Returns day, hour, minute, second, millisecond or microsecond

Methods inherited from Index:

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.indexes.base.Index
The array interface, return my values.

__array_ufunc__(self, ufunc: np.ufunc, method: str_t, *inputs, **kwargs) from pandas.core.indexes.base.Index

__array_wrap__(self, result, context=None, return_scalar=False) from pandas.core.indexes.base.Index
Gets called after a ufunc and other functions e.g. np.split.

```

__bool__(self) -> NoReturn from pandas.core.indexes.base.Index
__copy__(self) -> Self from pandas.core.indexes.base.Index
__deepcopy__(self, memo=None) -> Self from pandas.core.indexes.base.Index
Parameters
-----
memo, default None
Standard signature. Unused

__getitem__(self, key) from pandas.core.indexes.base.Index
Override numpy.ndarray's __getitem__ method to work as desired.

This function adds lists and Series as valid boolean indexers
(ndarrays only supports ndarray with dtype=bool).

If resulting ndim != 1, plain ndarray is returned instead of
corresponding `Index` subclass.

__iadd__(self, other) from pandas.core.indexes.base.Index
__invert__(self) -> Index from pandas.core.indexes.base.Index
__len__(self) -> int from pandas.core.indexes.base.Index
Return the length of the Index.

__reduce__(self) from pandas.core.indexes.base.Index
Helper for pickle.

__repr__(self) -> str_t from pandas.core.indexes.base.Index
Return a string representation for this object.

__setitem__(self, key, value) -> None from pandas.core.indexes.base.Index

all(self, *args, **kwargs) from pandas.core.indexes.base.Index
Return whether all elements are Truthy.

Parameters
-----
*args
    Required for compatibility with numpy.
**kwargs
    Required for compatibility with numpy.

Returns
-----
bool or array-like (if axis is specified)
A single element array-like may be converted to bool.

See Also
-----
Index.any : Return whether any element in an Index is True.
Series.any : Return whether any element in a Series is True.
Series.all : Return whether all elements in a Series are True.

Notes
-----
Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples
-----
True, because nonzero integers are considered True.

>>> pd.Index([1, 2, 3]).all()
True

False, because ``0`` is considered False.

>>> pd.Index([0, 1, 2]).all()
False

any(self, *args, **kwargs) from pandas.core.indexes.base.Index
Return whether any element is Truthy.

Parameters
-----

```

```

*args
    Required for compatibility with numpy.
**kwargs
    Required for compatibility with numpy.

Returns
-----
bool or array-like (if axis is specified)
    A single element array-like may be converted to bool.

See Also
-----
Index.all : Return whether all elements are True.
Series.all : Return whether all elements are True.

Notes
-----
Not a Number (NaN), positive infinity and negative infinity
evaluate to True because these are not equal to zero.

Examples
-----
>>> index = pd.Index([0, 1, 2])
>>> index.any()
True

>>> index = pd.Index([0, 0, 0])
>>> index.any()
False

```

append(self, other: Index | Sequence[Index]) -> Index from pandas.core.indexes.base.Index
Append a collection of Index options together.

```

Parameters
-----
other : Index or list/tuple of indices
    Single Index or a collection of indices, which can be either a list or a
    tuple.

Returns
-----
Index
    Returns a new Index object resulting from appending the provided other
    indices to the original Index.

```

```

See Also
-----
Index.insert : Make new Index inserting new item at location.

```

```

Examples
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx.append(pd.Index([4]))
Index([1, 2, 3, 4], dtype='int64')

```

argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base.Index
Return int position of the largest value in the Index.

If the maximum is achieved in multiple locations,
the first row position is returned.

```

Parameters
-----
axis : None
    Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
    Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
    and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
    Additional arguments and keywords for compatibility with NumPy.

```

```

Returns
-----
int
    Row position of the maximum value.

```

See Also

```
-----
Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmax : Return index label of the maximum values.
Series.idxmin : Return index label of the minimum values.
```

Examples

```
-----
Consider dataset containing cereal calories
```

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

```
argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int from pandas.core.indexes.base.Index
Return int position of the smallest value in the Index.
```

If the minimum is achieved in multiple locations, the first row position is returned.

Parameters

```
-----
axis : None
    Unused. Parameter needed for compatibility with DataFrame.
skipna : bool, default True
    Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.
*args, **kwargs
    Additional arguments and keywords for compatibility with NumPy.
```

Returns

```
-----
int
    Row position of the minimum value.
```

See Also

```
-----
Series.argmax : Return position of the maximum value.
Series.argmin : Return position of the minimum value.
numpy.ndarray.argmax : Equivalent method for numpy arrays.
Series.idxmin : Return index label of the minimum values.
Series.idxmax : Return index label of the maximum values.
```

Examples

```
-----
Consider dataset containing cereal calories
```

```
>>> idx = pd.Index([100.0, 110.0, 120.0, 110.0])
>>> idx
Index([100.0, 110.0, 120.0, 110.0], dtype='float64')

>>> idx.argmax()
np.int64(2)
>>> idx.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since index is zero-indexed.

```
argsort(self, *args, **kwargs) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Return the integer indices that would sort the index.
```

Parameters

```
-----
*args
    Passed to `numpy.ndarray.argsort`.
```


****kwargs**
Passed to ``numpy.ndarray.argsort``.

Returns

`np.ndarray[np.intp]`
Integer indices that would sort the index if used as an indexer.

See Also

`numpy.argsort` : Similar method for NumPy arrays.
`Index.sort_values` : Return sorted copy of Index.

Examples

```
>>> idx = pd.Index(["b", "a", "d", "c"])
>>> idx
Index(['b', 'a', 'd', 'c'], dtype='str')

>>> order = idx.argsort()
>>> order
array([1, 0, 3, 2])

>>> idx[order]
Index(['a', 'b', 'c', 'd'], dtype='str')
```

`asof(self, label)` from `pandas.core.indexes.base.Index`
Return the label from the index, or, if not present, the previous one.

Assuming that the index is sorted, return the passed index label if it is in the index, or return the previous index label if the passed one is not in the index.

Parameters

`label` : object
The label up to which the method returns the latest index label.

Returns

object
The passed label if it is in the index. The previous label if the passed label is not in the sorted index or ``NaN`` if there is no such label.

See Also

`Series.asof` : Return the latest value in a Series up to the passed index.
`merge_asof` : Perform an asof merge (similar to left join but it matches on nearest key rather than equal key).
`Index.get_loc` : An ``asof`` is a thin wrapper around ``get_loc`` with `method='pad'`.

Examples

``Index.asof`` returns the latest index label up to the passed label.

```
>>> idx = pd.Index(["2013-12-31", "2014-01-02", "2014-01-03"])
>>> idx.asof("2014-01-01")
'2013-12-31'
```

If the label is in the index, the method returns the passed label.

```
>>> idx.asof("2014-01-02")
'2014-01-02'
```

If all of the labels in the index are later than the passed label, `NaN` is returned.

```
>>> idx.asof("1999-01-02")
nan
```

If the index is not sorted, an error is raised.

```
>>> idx_not_sorted = pd.Index(["2013-12-31", "2015-01-02", "2014-01-03"])
```

```
>>> idx_not_sorted.asof("2013-12-31")
Traceback (most recent call last):
ValueError: index must be monotonic increasing or decreasing
```

`asof_locs(self, where: Index, mask: npt.NDArray[np.bool_]) -> npt.NDArray[np.intp]` from `pandas`
Return the locations (indices) of labels in the index.

As in the `:meth:`pandas.Index.asof``, if the label (a particular entry in ``where``) is not in the index, the latest index label up to the passed label is chosen and its index returned.

If all of the labels in the index are later than a label in ``where``, `-1` is returned.

``mask`` is used to ignore ``NA`` values in the index during calculation.

Parameters

`where` : Index
An Index consisting of an array of timestamps.
`mask` : `np.ndarray[bool]`
Array of booleans denoting where values in the original data are not ``NA``.

Returns

`np.ndarray[np.intp]`
An array of locations (indices) of the labels from the index which correspond to the return values of `:meth:`pandas.Index.asof`` for every element in ``where``.

See Also

`Index.asof` : Return the label from the index, or, if not present, the previous one.

Examples

```
>>> idx = pd.date_range("2023-06-01", periods=3, freq="D")
>>> where = pd.DatetimeIndex(
...     ["2023-05-30 00:12:00", "2023-06-01 00:00:00", "2023-06-02 23:59:59"]
... )
>>> mask = np.ones(3, dtype=bool)
>>> idx.asof_locs(where, mask)
array([-1,  0,  1])
```

We can use ``mask`` to ignore certain values in the index during calculation.

```
>>> mask[1] = False
>>> idx.asof_locs(where, mask)
array([-1,  0,  0])
```

`astype(self, dtype: Dtype, copy: bool = True)` from `pandas.core.indexes.base.Index`
Create an Index with values cast to dtypes.

The class of a new Index is determined by `dtype`. When conversion is impossible, a `TypeError` exception is raised.

Parameters

`dtype` : numpy dtype or pandas type
Note that any signed integer ``dtype`` is treated as ``int64``, and any unsigned integer ``dtype`` is treated as ``uint64``, regardless of the size.
`copy` : bool, default True
By default, `astype` always returns a newly allocated object. If `copy` is set to False and internal requirements on `dtype` are satisfied, the original data is used to create a new Index or the original Index is returned.

Returns

Index
Index with values cast to specified dtype.

See Also

Index.dtype: Return the dtype object of the underlying data.
Index.dtypes: Return the dtype object of the underlying data.
Index.convert_dtypes: Convert columns to the best possible dtypes.

Examples

```
-----  
>>> idx = pd.Index([1, 2, 3])  
>>> idx  
Index([1, 2, 3], dtype='int64')  
>>> idx.astype("float")  
Index([1.0, 2.0, 3.0], dtype='float64')
```

copy(self, name: Hashable | None = None, deep: bool = False) -> Self from pandas.core.indexes
Make a copy of this object.

Name is set on the new object.

Parameters

```
-----  
name : Label, optional  
    Set name for new object.  
deep : bool, default False  
    If True attempts to make a deep copy of the Index.  
    Else makes a shallow copy.
```

Returns

```
-----  
Index  
    Index refer to new object which is a copy of this object.
```

See Also

```
-----  
Index.delete: Make new Index with passed location(-s) deleted.  
Index.drop: Make new Index with passed list of labels deleted.
```

Notes

```
-----  
In most cases, there should be no functional difference from using  
``deep``, but if ``deep`` is passed it will attempt to deepcopy.
```

Examples

```
-----  
>>> idx = pd.Index(["a", "b", "c"])  
>>> new_idx = idx.copy()  
>>> idx is new_idx  
False
```

diff(self, periods: int = 1) -> Index from pandas.core.indexes.base.Index
Computes the difference between consecutive values in the Index object.

If periods is greater than 1, computes the difference between values that are `periods` number of positions apart.

Parameters

```
-----  
periods : int, optional  
    The number of positions between the current and previous  
    value to compute the difference with. Default is 1.
```

Returns

```
-----  
Index  
    A new Index object with the computed differences.
```

Examples

```
-----  
>>> import pandas as pd  
>>> idx = pd.Index([10, 20, 30, 40, 50])  
>>> idx.diff()  
Index([nan, 10.0, 10.0, 10.0, 10.0], dtype='float64')
```

difference(self, other, sort: bool | None = None) from pandas.core.indexes.base.Index
Return a new Index with elements of index not in `other`.

This is the set difference of two Index objects.

Parameters

```

-----
other : Index or array-like
    Index object or an array-like object containing elements to be compared
    with the elements of the original Index.
sort : bool or None, default None
    Whether to sort the resulting index. By default, the
    values are attempted to be sorted, but any TypeError from
    incomparable elements is caught by pandas.

    * None : Attempt to sort the result, but catch any TypeErrors
      from comparing incomparable elements.
    * False : Do not sort the result.
    * True : Sort the result (which may raise TypeError).

```

Returns

Index

Returns a new Index object containing elements that are in the original Index but not in the `other` Index.

See Also

Index.symmetric_difference : Compute the symmetric difference of two Index objects.

Index.intersection : Form the intersection of two Index objects.

Examples

```

>>> idx1 = pd.Index([2, 1, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.difference(idx2)
Index([1, 2], dtype='int64')
>>> idx1.difference(idx2, sort=False)
Index([2, 1], dtype='int64')

```

```

drop(
    self,
    labels: Index | np.ndarray | Iterable[Hashable],
    errors: IgnoreRaise = 'raise'
) -> Index from pandas.core.indexes.base.Index
    Make new Index with passed list of labels deleted.

```

Parameters

labels : array-like or scalar

Array-like object or a scalar value, representing the labels to be removed from the Index.

errors : {'ignore', 'raise'}, default 'raise'

If 'ignore', suppress error and existing labels are dropped.

Returns

Index

Will be same type as self, except for RangeIndex.

Raises

KeyError

If not all of the labels are found in the selected axis

See Also

Index.dropna : Return Index without NA/NAN values.

Index.drop_duplicates : Return Index with duplicate values removed.

Examples

```

>>> idx = pd.Index(["a", "b", "c"])
>>> idx.drop(["a"])
Index(['b', 'c'], dtype='str')

```

```

drop_duplicates(self, *, keep: DropKeep = 'first') -> Self from pandas.core.indexes.base.Index
    Return Index with duplicate values removed.

```

Parameters

keep : {'first', 'last', ``False``}, default 'first'

- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

Returns

Index

A new Index object with the duplicate values removed.

See Also

Series.drop_duplicates : Equivalent method on Series.

DataFrame.drop_duplicates : Equivalent method on DataFrame.

Index.duplicated : Related method on Index, indicating duplicate Index values.

Examples

Generate a pandas.Index with duplicate values.

```
>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama", "hippo"])
```

The `keep` parameter controls which duplicate values are removed.

The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of keep is 'first'.

```
>>> idx.drop_duplicates(keep="first")
Index(['llama', 'cow', 'beetle', 'hippo'], dtype='str')
```

The value 'last' keeps the last occurrence for each set of duplicated entries.

```
>>> idx.drop_duplicates(keep="last")
Index(['cow', 'beetle', 'llama', 'hippo'], dtype='str')
```

The value ``False`` discards all sets of duplicated entries.

```
>>> idx.drop_duplicates(keep=False)
Index(['cow', 'beetle', 'hippo'], dtype='str')
```

droplevel(self, level: IndexLabel = 0) from pandas.core.indexes.base.Index
Return index with requested level(s) removed.

If resulting index has only 1 level left, the result will be of Index type, not MultiIndex. The original index is not modified inplace.

Parameters

level : int, str, or list-like, default 0

If a string is given, must be the name of a level

If list-like, elements must be names or indexes of levels.

Returns

Index or MultiIndex

Returns an Index or MultiIndex object, depending on the resulting index after removing the requested level(s).

See Also

Index.dropna : Return Index without NA/NaN values.

Examples

```
>>> mi = pd.MultiIndex.from_arrays(
...     [[1, 2], [3, 4], [5, 6]], names=["x", "y", "z"]
... )
>>> mi
MultiIndex([(1, 3, 5),
             (2, 4, 6)],
           names=['x', 'y', 'z'])
```

```
>>> mi.droplevel()
MultiIndex([(3, 5),
            (4, 6)],
           names=['y', 'z'])
```

```

>>> mi.droplevel(2)
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel("z")
MultiIndex([(1, 3),
            (2, 4)],
            names=['x', 'y'])

>>> mi.droplevel(["x", "y"])
Index([5, 6], dtype='int64', name='z')

```

dropna(self, how: AnyAll = 'any') -> Self from pandas.core.indexes.base.Index
Return Index without NA/NaN values.

Parameters

how : {'any', 'all'}, default 'any'
If the Index is a MultiIndex, drop the value when any or all levels are NaN.

Returns

Index
Returns an Index object after removing NA/NaN values.

See Also

Index.fillna : Fill NA/NaN values with the specified value.
Index.isna : Detect missing values.

Examples

>>> idx = pd.Index([1, np.nan, 3])
>>> idx.dropna()
Index([1.0, 3.0], dtype='float64')

duplicated(self, keep: DropKeep = 'first') -> npt.NDArray[np.bool_] from pandas.core.indexes
Indicate duplicate index values.

Duplicated values are indicated as ``True`` values in the resulting array. Either all duplicates, all except the first, or all except the last occurrence of duplicates can be indicated.

Parameters

keep : {'first', 'last', False}, default 'first'
The value or values in a set of duplicates to mark as missing.

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

np.ndarray[bool]
A numpy array of boolean values indicating duplicate index values.

See Also

Series.duplicated : Equivalent method on pandas.Series.
DataFrame.duplicated : Equivalent method on pandas.DataFrame.
Index.drop_duplicates : Remove duplicate values from Index.

Examples

By default, for each set of duplicated values, the first occurrence is set to False and all others to True:

```

>>> idx = pd.Index(["llama", "cow", "llama", "beetle", "llama"])
>>> idx.duplicated()
array([False, False,  True, False,  True])

```

which is equivalent to

```
>>> idx.duplicated(keep="first")
array([False, False,  True, False,  True])
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> idx.duplicated(keep="last")
array([  True, False,  True, False, False])
```

By setting keep on ``False``, all duplicates are True:

```
>>> idx.duplicated(keep=False)
array([  True, False,  True, False,  True])
```

`fillna(self, value)` from `pandas.core.indexes.base.Index`
Fill NA/Nan values with the specified value.

Parameters

value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index
NA/Nan values replaced with `value`.

See Also

`DataFrame.fillna` : Fill NaN values of a `DataFrame`.
`Series.fillna` : Fill NaN Values of a `Series`.

Examples

```
>>> idx = pd.Index([np.nan, np.nan, 3])
>>> idx.fillna(0)
Index([0.0, 0.0, 3.0], dtype='float64')
```

```
get_indexer(
    self,
    target,
    method: ReindexMethod | None = None,
    limit: int | None = None,
    tolerance=None
) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
```

Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to `ndarray.take` to align the current data to the new index.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.
method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional
* default: exact matches only.
* pad / ffill: find the PREVIOUS index value if no exact match.
* backfill / bfill: use NEXT index value if no exact match
* nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.
limit : int, optional
Maximum number of consecutive labels in ``target`` to match for inexact matches.
tolerance : optional
Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, `Series`, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.

See Also

Index.get_indexer_for : Returns an indexer even when non-unique.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Notes

Returns -1 for unmatched values, for further explanation see the example below.

Examples

>>> index = pd.Index(["c", "a", "b"])
>>> index.get_indexer(["a", "b", "x"])
array([1, 2, -1])

Notice that the return value is an array of locations in ``index`` and ``x`` is marked by -1, as it is not in ``index``.

get_indexer_for(self, target) -> npt.NDArray[np.intp] from pandas.core.indexes.base.Index
Guaranteed return of an indexer even when non-unique.

This dispatches to get_indexer or get_indexer_non_unique as appropriate.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.

Returns

np.ndarray[np.intp]
List of indices.

See Also

Index.get_indexer : Computes indexer and mask for new index given the current index.
Index.get_non_unique : Returns indexer and masks for new index given the current index.

Examples

>>> idx = pd.Index([np.nan, "var1", np.nan])
>>> idx.get_indexer_for([np.nan])
array([0, 2])

get_indexer_non_unique(self, target) -> tuple[npt.NDArray[np.intp], npt.NDArray[np.intp]] from pandas.core.indexes.base.Index
Compute indexer and mask for new index given the current index.

The indexer should be then used as an input to ndarray.take to align the current data to the new index.

Parameters

target : Index
An iterable containing the values to be used for computing indexer.

Returns

indexer : np.ndarray[np.intp]
Integers from 0 to n - 1 indicating that the index at these positions matches the corresponding target values. Missing values in the target are marked by -1.
missing : np.ndarray[np.intp]
An indexer into the target of the values not found. These correspond to the -1 in the indexer array.

See Also

Index.get_indexer : Computes indexer and mask for new index given
the current index.
Index.get_indexer_for : Returns an indexer even when non-unique.

Examples

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["b", "b"])
(array([1, 3, 4, 1, 3, 4]), array([], dtype=int64))

In the example below there are no matched values.

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["q", "r", "t"])
(array([-1, -1, -1]), array([0, 1, 2]))

For this reason, the returned ``indexer`` contains only integers equal to -1.
It demonstrates that there's no match between the index and the ``target``
values at these positions. The mask [0, 1, 2] in the return value shows that
the first, second, and third elements are missing.

Notice that the return value is a tuple contains two items. In the example
below the first item is an array of locations in ``index``. The second
item is a mask shows that the first and third elements are missing.

>>> index = pd.Index(["c", "b", "a", "b", "b"])
>>> index.get_indexer_non_unique(["f", "b", "s"])
(array([-1, 1, 3, 4, -1]), array([0, 2]))

get_level_values = _get_level_values(self, level) -> Index

get_slice_bound(self, label, side: Literal['left', 'right']) -> int from pandas.core.indexes
Calculate slice bound that corresponds to given label.

Returns leftmost (one-past-the-rightmost if ``side=='right'``) position
of given label.

Parameters

label : object
The label for which to calculate the slice bound.
side : {'left', 'right'}
if 'left' return leftmost position of given label.
if 'right' return one-past-the-rightmost position of given label.

Returns

int
Index of label.

See Also

Index.get_loc : Get integer location, slice or boolean mask for requested
label.

Examples

>>> idx = pd.RangeIndex(5)
>>> idx.get_slice_bound(3, "left")
3

>>> idx.get_slice_bound(3, "right")
4

If ``label`` is non-unique in the index, an error will be raised.

>>> idx_duplicate = pd.Index(["a", "b", "a", "c", "d"])
>>> idx_duplicate.get_slice_bound("a", "left")
Traceback (most recent call last):
KeyError: Cannot get left slice bound for non-unique label: 'a'

groupby(self, values) -> PrettyDict[Hashable, Index] from pandas.core.indexes.base.Index
Group the index labels by a given array of values.

Parameters

```

values : array
    Values used to determine the groups.

Returns
-----
dict
    {group name -> group labels}

identical(self, other) -> bool from pandas.core.indexes.base.Index
    Similar to equals, but checks that object attributes and types are also equal.

Parameters
-----
other : Index
    The Index object you want to compare with the current Index object.

Returns
-----
bool
    If two Index objects have equal elements and same type True,
    otherwise False.

See Also
-----
Index.equals: Determine if two Index object are equal.
Index.has_duplicates: Check if the Index has duplicate values.
Index.is_unique: Return if the index has unique values.

Examples
-----
>>> idx1 = pd.Index(["1", "2", "3"])
>>> idx2 = pd.Index(["1", "2", "3"])
>>> idx2.identical(idx1)
True

>>> idx1 = pd.Index(["1", "2", "3"], name="A")
>>> idx2 = pd.Index(["1", "2", "3"], name="B")
>>> idx2.identical(idx1)
False

infer_objects(self, copy: bool = True) -> Index from pandas.core.indexes.base.Index
    If we have an object dtype, try to infer a non-object dtype.

Parameters
-----
copy : bool, default True
    Whether to make a copy in cases where no inference occurs.

Returns
-----
Index
    An Index with a new dtype if the dtype was inferred
    or a shallow copy if the dtype could not be inferred.

See Also
-----
Index.inferred_type: Return a string of the type inferred from the values.

Examples
-----
>>> pd.Index(["a", 1]).infer_objects()
Index(['a', 1], dtype='object')
>>> pd.Index([1, 2], dtype="object").infer_objects()
Index([1, 2], dtype='int64')

intersection(self, other, sort: bool = False) from pandas.core.indexes.base.Index
    Form the intersection of two Index objects.

    This returns a new Index with elements common to the index and `other`.

Parameters
-----
other : Index or array-like
    An Index or an array-like object containing elements to form the
    intersection with the original Index.
sort : True, False or None, default False
    Whether to sort the resulting index.

```

- * None : sort the result, except when `self` and `other` are equal or when the values cannot be compared.
- * False : do not sort the result.
- * True : Sort the result (which may raise TypeError).

Returns

Index

Returns a new Index object with elements common to both the original Index and the `other` Index.

See Also

Index.union : Form the union of two Index objects.

Index.difference : Return a new Index with elements of index not in other.

Index.isin : Return a boolean array where the index values are in values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3, 4])
```

```
>>> idx2 = pd.Index([3, 4, 5, 6])
```

```
>>> idx1.intersection(idx2)
```

```
Index([3, 4], dtype='int64')
```

is_(self, other) -> bool from pandas.core.indexes.base.Index

More flexible, faster check like ``is`` but that works through views.

Note: this is *not* the same as ``Index.identical()`` , which checks that metadata is also the same.

Parameters

other : object

Other object to compare against.

Returns

bool

True if both have same underlying data, False otherwise.

See Also

Index.identical : Works like ``Index.is\_`` but also checks metadata.

Examples

```
>>> idx1 = pd.Index(["1", "2", "3"])
```

```
>>> idx1.is_(idx1.view())
```

```
True
```

```
>>> idx1.is_(idx1.copy())
```

```
False
```

isin(self, values, level: str_t | int | None = None) -> npt.NDArray[np.bool_] from pandas.core.indexes

Return a boolean array where the index values are in `values`.

Compute boolean array of whether each index value is found in the passed set of values. The length of the returned boolean array matches the length of the index.

Parameters

values : set or list-like

Sought values.

level : str or int, optional

Name or position of the index level to use (if the index is a `MultiIndex`).

Returns

np.ndarray[bool]

NumPy array of boolean values.

See Also

Series.isin : Same for Series.

DataFrame.isin : Same method for DataFrames.

Notes

In the case of `MultiIndex` you must either specify `values` as a list-like object containing tuples that are the same length as the number of levels, or specify `level`. Otherwise it will raise a `ValueError`.

If `level` is specified:

- if it is the name of one *and only one* index level, use that level;
- otherwise it should be a number indicating level position.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
```

Check whether each index value in a list of values.

```
>>> idx.isin([1, 4])
array([ True, False, False])
```

```
>>> midx = pd.MultiIndex.from_arrays(
...     [[1, 2, 3], ["red", "blue", "green"]], names=["number", "color"]
... )
>>> midx
MultiIndex([(1, 'red'),
            (2, 'blue'),
            (3, 'green')],
           names=['number', 'color'])
```

Check whether the strings in the 'color' level of the MultiIndex are in a list of colors.

```
>>> midx.isin(["red", "orange", "yellow"], level="color")
array([ True, False, False])
```

To check across the levels of a MultiIndex, pass a list of tuples:

```
>>> midx.isin([(1, "red"), (3, "red")])
array([ True, False, False])
```

`isna(self) -> npt.NDArray[np.bool_] from pandas.core.indexes.base.Index`
Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None`, `:attr:`numpy.NaN`` or `:attr:`pd.NaT``, get mapped to `True` values. Everything else get mapped to `False` values. Characters such as empty strings `''` or `:attr:`numpy.inf`` are not considered NA values.

Returns

`numpy.ndarray[bool]`
A boolean array of whether my values are NA.

See Also

`Index.notna` : Boolean inverse of `isna`.
`Index.dropna` : Omit entries with missing values.
`isna` : Top-level `isna`.
`Series.isna` : Detect missing values in Series object.

Examples

Show which entries in a pandas.Index are NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.isna()
array([False, False,  True])
```

Empty strings are not considered NA values. None is considered an NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.isna()
array([False, False, False,  True])
```

For datetimes, `NaT` (Not a Time) is considered as an NA value.

```
>>> idx = pd.DatetimeIndex(
...     [pd.Timestamp("1940-04-25"), pd.Timestamp(""), None, pd.NaT]
... )
>>> idx
DatetimeIndex(['1940-04-25', 'NaT', 'NaT', 'NaT'],
              dtype='datetime64[us]', freq=None)
>>> idx.isna()
array([False,  True,  True,  True])
```

isnull = isna(self) -> npt.NDArray[np.bool_]

```
join(
    self,
    other: Index,
    *,
    how: JoinHow = 'left',
    level: Level | None = None,
    return_indexers: bool = False,
    sort: bool = False
) -> Index | tuple[Index, npt.NDArray[np.intp] | None, npt.NDArray[np.intp] | None] from pandas.core.indexes.base
Compute join_index and indexers to conform data structures to the new index.
```

Parameters

other : Index
The other index on which join is performed.
how : {'left', 'right', 'inner', 'outer'}
level : int or level name, default None
It is either the integer position or the name of the level.
return_indexers : bool, default False
Whether to return the indexers or not for both the index objects.
sort : bool, default False
Sort the join keys lexicographically in the result Index. If False,
the order of the join keys depends on the join type (how keyword).

Returns

join_index, (left_indexer, right_indexer)
The new index.

See Also

DataFrame.join : Join columns with `other` DataFrame either on index
or on a key.
DataFrame.merge : Merge DataFrame or named Series objects with a
database-style join.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([4, 5, 6])
>>> idx1.join(idx2, how="outer")
Index([1, 2, 3, 4, 5, 6], dtype='int64')
>>> idx1.join(other=idx2, how="outer", return_indexers=True)
(Index([1, 2, 3, 4, 5, 6], dtype='int64'),
 array([ 0,  1,  2, -1, -1, -1]), array([-1, -1, -1,  0,  1,  2]))
```

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.indexes.base.
Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}
If 'ignore', propagate NA values, without passing them to the

mapping correspondence.

Returns

Union[Index, MultiIndex]

The output of the mapping function applied to the index.

If the function returns a tuple with more than one element a MultiIndex will be returned.

See Also

Index.where : Replace values where the condition is False.

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.map({1: "a", 2: "b", 3: "c"})
Index(['a', 'b', 'c'], dtype='str')
```

Using `map` with a function:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.map("I am a {}".format)
Index(['I am a 1', 'I am a 2', 'I am a 3'], dtype='str')
```

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx.map(lambda x: x.upper())
Index(['A', 'B', 'C'], dtype='str')
```

max(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) from pandas.core.indexes.base.Index
Return the maximum value of the Index.

Parameters

axis : int, optional

For compatibility with NumPy. Only 0 or None are allowed.

skipna : bool, default True

Exclude NA/null values when showing the result.

*args, **kwargs

Additional arguments and keywords for compatibility with NumPy.

Returns

scalar

Maximum value.

See Also

Index.min : Return the minimum value in an Index.

Series.max : Return the maximum value in a Series.

DataFrame.max : Return the maximum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.max()
3
```

```
>>> idx = pd.Index(["c", "b", "a"])
>>> idx.max()
'c'
```

For a MultiIndex, the maximum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.max()
('b', 2)
```

memory_usage(self, deep: bool = False) -> int from pandas.core.indexes.base.Index
Memory usage of the values.

Parameters

deep : bool, default False

Introspect the data deeply, interrogate

`object` dtypes for system-level memory consumption.

Returns

bytes used

Returns memory usage of the values in the Index in bytes.

See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of the array.

Notes

Memory usage does not include memory consumed by elements that are not components of the array if `deep=False` or if used on PyPy

Examples

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.memory_usage()
24
```

`min(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs)` from `pandas.core.indexes.base.Index`
Return the minimum value of the Index.

Parameters

`axis` : {None}

Dummy argument for consistency with Series.

`skipna` : bool, default True

Exclude NA/null values when showing the result.

`*args, **kwargs`

Additional arguments and keywords for compatibility with NumPy.

Returns

scalar

Minimum value.

See Also

`Index.max` : Return the maximum value of the object.

`Series.min` : Return the minimum value in a Series.

`DataFrame.min` : Return the minimum values in a DataFrame.

Examples

```
>>> idx = pd.Index([3, 2, 1])
>>> idx.min()
1
```

```
>>> idx = pd.Index(["c", "b", "a"])
>>> idx.min()
'a'
```

For a MultiIndex, the minimum is determined lexicographically.

```
>>> idx = pd.MultiIndex.from_product([("a", "b"), (2, 1)])
>>> idx.min()
('a', 1)
```

`notna(self) -> npt.NDArray[np.bool_]` from `pandas.core.indexes.base.Index`
Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to `True`. Characters such as empty strings `''` or `:attr:'numpy.inf'` are not considered NA values. NA values, such as `None` or `:attr:'numpy.NaN'`, get mapped to `False` values.

Returns

`numpy.ndarray[bool]`

Boolean array to indicate which entries are not NA.

See Also

`Index.notnull` : Alias of `notna`.

Index.isna: Inverse of notna.
notna : Top-level notna.

Examples

Show which entries in an Index are not NA. The result is an array.

```
>>> idx = pd.Index([5.2, 6.0, np.nan])
>>> idx
Index([5.2, 6.0, nan], dtype='float64')
>>> idx.notna()
array([ True,  True, False])
```

Empty strings are not considered NA values. None is considered a NA value.

```
>>> idx = pd.Index(["black", "", "red", None])
>>> idx
Index(['black', '', 'red', nan], dtype='str')
>>> idx.notna()
array([ True,  True,  True, False])
```

notnull = notna(self) -> npt.NDArray[np.bool_]

putmask(self, mask, value) -> Index from pandas.core.indexes.base.Index
Return a new Index of the values set with the mask.

Parameters

mask : array-like of bool
Array of booleans denoting where values should be replaced.
value : scalar
Scalar value to use to fill holes (e.g. 0).
This value cannot be a list-like.

Returns

Index
A new Index of the values set with the mask.

See Also

numpy.putmask : Changes elements of an array
based on conditional and input values.

Examples

```
>>> idx1 = pd.Index([1, 2, 3])
>>> idx2 = pd.Index([5, 6, 7])
>>> idx1.putmask([True, False, False], idx2)
Index([5, 2, 3], dtype='int64')
```

ravel(self, order: str_t = 'C') -> Self from pandas.core.indexes.base.Index
Return a view on self.

Parameters

order : {'K', 'A', 'C', 'F'}, default 'C'
Specify the memory layout of the view. This parameter is not
implemented currently.

Returns

Index
A view on self.

See Also

numpy.ndarray.ravel : Return a flattened array.

Examples

```
>>> s = pd.Series([1, 2, 3], index=["a", "b", "c"])
>>> s.index.ravel()
Index(['a', 'b', 'c'], dtype='str')
```



```

reindex(
    self,
    target,
    method: ReindexMethod | None = None,
    level=None,
    limit: int | None = None,
    tolerance: float | None = None
) -> tuple[Index, npt.NDArray[np.intp] | None] from pandas.core.indexes.base.Index
Create index with target's values.

```

Parameters

target : an iterable

An iterable containing the values to be used for creating the new index.

method : {None, 'pad'/'ffill', 'backfill'/'bfill', 'nearest'}, optional

* default: exact matches only.

* pad / ffill: find the PREVIOUS index value if no exact match.

* backfill / bfill: use NEXT index value if no exact match

* nearest: use the NEAREST index value if no exact match. Tied distances are broken by preferring the larger index value.

level : int, optional

Level of multiindex.

limit : int, optional

Maximum number of consecutive labels in ``target`` to match for inexact matches.

tolerance : int, float, or list-like, optional

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation `abs(index[indexer] - target) <= tolerance`.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

new_index : pd.Index

Resulting index.

indexer : np.ndarray[np.intp] or None

Indices of output values in original index.

Raises

TypeError

If ``method`` passed along with ``level``.

ValueError

If non-unique multi-index

ValueError

If non-unique index and ``method`` or ``limit`` passed.

See Also

Series.reindex : Conform Series to new index with optional filling logic.

DataFrame.reindex : Conform DataFrame to new index with optional filling logic.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
>>> idx
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
```

```
>>> idx.reindex(["car", "bike"])
(Index(['car', 'bike'], dtype='str'), array([0, 1]))
```

```

rename(self, name, *, inplace: bool = False) -> Self | None from pandas.core.indexes.base.Index
Alter Index or MultiIndex name.

```

Able to set new names without level. Defaults to returning new index.
Length of names must match number of levels in MultiIndex.

Parameters

name : Hashable or a sequence of the previous

Name(s) to set.

inplace : bool, default False

Modifies the object directly, instead of creating a new Index or

MultiIndex.

Returns

Index or None

The same type as the caller or None if ``inplace=True``.

See Also

Index.set_names : Able to set new names partially and by level.

Examples

```
>>> idx = pd.Index(["A", "C", "A", "B"], name="score")
>>> idx.rename("grade")
Index(['A', 'C', 'A', 'B'], dtype='str', name='grade')

>>> idx = pd.MultiIndex.from_product(
...     [ ["python", "cobra"], [2018, 2019] ], names=["kind", "year"]
... )
>>> idx
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
           names=['kind', 'year'])
>>> idx.rename(["species", "year"])
MultiIndex([( 'python', 2018),
             ( 'python', 2019),
             ( 'cobra', 2018),
             ( 'cobra', 2019)],
           names=['species', 'year'])
>>> idx.rename("species")
Traceback (most recent call last):
TypeError: Must pass list-like as `names`.
```

repeat(self, repeats, axis: None = None) -> Self from pandas.core.indexes.base.Index
Repeat elements of an Index.

Returns a new Index where each element of the current Index
is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints

The number of repetitions for each element. This should be a
non-negative integer. Repeating 0 times will return an empty
Index.

axis : None

Must be ``None``. Has no effect but is accepted for compatibility
with numpy.

Returns

Index

Newly created Index with repeated elements.

See Also

Series.repeat : Equivalent function for Series.

numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

```
>>> idx = pd.Index(["a", "b", "c"])
>>> idx
Index(['a', 'b', 'c'], dtype='str')
>>> idx.repeat(2)
Index(['a', 'a', 'b', 'b', 'c', 'c'], dtype='str')
>>> idx.repeat([1, 2, 3])
Index(['a', 'b', 'b', 'c', 'c', 'c'], dtype='str')
```

set_names(self, names, *, level=None, inplace: bool = False) -> Self | None from pandas.core
Set Index or MultiIndex name.

Able to set new names partially and by level.

Parameters

names : Hashable or a sequence of the previous or dict-like for MultiIndex
Name(s) to set.

level : int, Hashable or a sequence of the previous, optional
If the index is a MultiIndex and names is not dict-like, level(s) to set
(None for all levels). Otherwise level must be None.

inplace : bool, default False
Modifies the object directly, instead of creating a new Index or
MultiIndex.

Returns

Index or None
The same type as the caller or None if ``inplace=True``.

See Also

Index.rename : Able to set new names without level.

Examples

>>> idx = pd.Index([1, 2, 3, 4])
>>> idx
Index([1, 2, 3, 4], dtype='int64')
>>> idx.set_names("quarter")
Index([1, 2, 3, 4], dtype='int64', name='quarter')

>>> idx = pd.MultiIndex.from_product([["python", "cobra"], [2018, 2019]])
>>> idx
MultiIndex([('python', 2018),
 ('python', 2019),
 ('cobra', 2018),
 ('cobra', 2019)],
)
>>> idx = idx.set_names(["kind", "year"])
>>> idx.set_names("species", level=0)
MultiIndex([('python', 2018),
 ('python', 2019),
 ('cobra', 2018),
 ('cobra', 2019)],
 names=['species', 'year'])

When renaming levels with a dict, levels can not be passed.

```
>>> idx.set_names({"kind": "snake"})  
MultiIndex([('python', 2018),  
            ('python', 2019),  
            ('cobra', 2018),  
            ('cobra', 2019)],  
            names=['snake', 'year'])
```

```
slice_indexer(  
    self,  
    start: Hashable | None = None,  
    end: Hashable | None = None,  
    step: int | None = None  
) -> slice from pandas.core.indexes.base.Index  
Compute the slice indexer for input labels and step.
```

Index needs to be ordered and unique.

Parameters

start : label, default None
If None, defaults to the beginning.
end : label, default None
If None, defaults to the end.
step : int, default None
If None, defaults to 1.

Returns

slice
A slice object.

Raises

KeyError : If key does not exist, or key is not unique and index is not ordered.

See Also

Index.slice_locs : Computes slice locations for input labels.

Index.get_slice_bound : Retrieves slice bound that corresponds to given label.

Notes

This function assumes that the data is sorted, so use at your own peril.

Examples

This is a method on all index types. For example you can do:

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_indexer(start="b", end="c")
slice(1, 3, None)
```

```
>>> idx = pd.MultiIndex.from_arrays([list("abcd"), list("efgh")])
>>> idx.slice_indexer(start="b", end=("c", "g"))
slice(1, 3, None)
```

```
slice_locs(
    self,
    start: SliceType = None,
    end: SliceType = None,
    step: int | None = None
) -> tuple[int, int] from pandas.core.indexes.base.Index
Compute slice locations for input labels.
```

Parameters

start : label, default None

If None, defaults to the beginning.

end : label, default None

If None, defaults to the end.

step : int, defaults None

If None, defaults to 1.

Returns

tuple[int, int]

Returns a tuple of two integers representing the slice locations for the input labels within the index.

See Also

Index.get_loc : Get location for a single label.

Notes

This method only works if the index is monotonic or unique.

Examples

```
>>> idx = pd.Index(list("abcd"))
>>> idx.slice_locs(start="b", end="c")
(1, 3)
```

```
>>> idx = pd.Index(list("bcde"))
>>> idx.slice_locs(start="a", end="c")
(0, 2)
```

```
sort_values(
    self,
    *,
    return_indexer: bool = False,
    ascending: bool = True,
    na_position: NaPosition = 'last',
    key: Callable | None = None
) -> Self | tuple[Self, np.ndarray] from pandas.core.indexes.base.Index
Return a sorted copy of the index.
```

Return a sorted copy of the index, and optionally return the indices that sorted the index itself.

Parameters

return_indexer : bool, default False
Should the indices that would sort the index be returned.
ascending : bool, default True
Should the index values be sorted in an ascending order.
na_position : {'first' or 'last'}, default 'last'
Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
key : callable, optional
If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an ``Index`` and return an ``Index`` of the same shape.

Returns

sorted_index : pandas.Index
Sorted copy of the index.
indexer : numpy.ndarray, optional
The indices that the index itself was sorted by.

See Also

Series.sort_values : Sort values of a Series.
DataFrame.sort_values : Sort values in a DataFrame.

Examples

>>> idx = pd.Index([10, 100, 1, 1000])
>>> idx
Index([10, 100, 1, 1000], dtype='int64')

Sort values in ascending order (default behavior).

>>> idx.sort_values()
Index([1, 10, 100, 1000], dtype='int64')

Sort values in descending order, and also get the indices `idx` was sorted by.

>>> idx.sort_values(ascending=False, return_indexer=True)
(Index([1000, 100, 10, 1], dtype='int64'), array([3, 1, 0, 2]))

```
sortlevel(  
    self,  
    level=None,  
    ascending: bool | list[bool] = True,  
    sort_remaining=None,  
    na_position: NaPosition = 'first'  
) -> tuple[Self, np.ndarray] from pandas.core.indexes.base.Index  
For internal compatibility with the Index API.
```

Sort the Index. This is for compat with MultiIndex

Parameters

ascending : bool, default True
False to sort in descending order
na_position : {'first' or 'last'}, default 'first'
Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.

.. versionadded:: 2.1.0

level, sort_remaining are compat parameters

Returns

Index

```
symmetric_difference(  

```

```

self,
other,
result_name: abc.Hashable | None = None,
sort: bool | None = None
) from pandas.core.indexes.base.Index
Compute the symmetric difference of two Index objects.

Parameters
-----
other : Index or array-like
    Index or an array-like object with elements to compute the symmetric
    difference with the original Index.
result_name : str
    A string representing the name of the resulting Index, if desired.
sort : bool or None, default None
    Whether to sort the resulting index. By default, the
    values are attempted to be sorted, but any TypeError from
    incomparable elements is caught by pandas.

    * None : Attempt to sort the result, but catch any TypeErrors
      from comparing incomparable elements.
    * False : Do not sort the result.
    * True : Sort the result (which may raise TypeError).

Returns
-----
Index
    Returns a new Index object containing elements that appear in either the
    original Index or the `other` Index, but not both.

See Also
-----
Index.difference : Return a new Index with elements of index not in other.
Index.union : Form the union of two Index objects.
Index.intersection : Form the intersection of two Index objects.

Notes
-----
``symmetric_difference`` contains elements that appear in either
``idx1`` or ``idx2`` but not both. Equivalent to the Index created by
``idx1.difference(idx2) | idx2.difference(idx1)`` with duplicates
dropped.

Examples
-----
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([2, 3, 4, 5])
>>> idx1.symmetric_difference(idx2)
Index([1, 5], dtype='int64')

to_flat_index(self) -> Self from pandas.core.indexes.base.Index
Identity method.

This is implemented for compatibility with subclass implementations
when chaining.

Returns
-----
pd.Index
    Caller.

See Also
-----
MultiIndex.to_flat_index : Subclass implementation.

to_frame(self, index: bool = True, name: Hashable = <no_default>) -> DataFrame from pandas.c
Create a DataFrame with a column containing the Index.

Parameters
-----
index : bool, default True
    Set the index of the returned DataFrame as the original Index.

name : object, defaults to index.name
    The passed name should substitute for the index name (if it has
    one).

```

Returns

DataFrame

DataFrame containing the original Index data.

See Also

`Index.to_series` : Convert an Index to a Series.

`Series.to_frame` : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

```
>>> idx.to_frame()
```

```
      animal
Ant      Ant
Bear    Bear
Cow      Cow
```

By default, the original Index is reused. To enforce a new Index:

```
>>> idx.to_frame(index=False)
```

```
      animal
0      Ant
1     Bear
2      Cow
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_frame(index=False, name="zoo")
```

```
      zoo
0      Ant
1     Bear
2      Cow
```

`to_series(self, index=None, name: Hashable | None = None) -> Series` from `pandas.core.indexes`
Create a Series with both index and values equal to the index keys.

Useful with `map` for returning an indexer based on an index.

Parameters

`index` : Index, optional

Index of resulting Series. If None, defaults to original index.

`name` : str, optional

Name of resulting Series. If None, defaults to name of original index.

Returns

Series

The dtype will be based on the type of the Index values.

See Also

`Index.to_frame` : Convert an Index to a DataFrame.

`Series.to_frame` : Convert Series to DataFrame.

Examples

```
>>> idx = pd.Index(["Ant", "Bear", "Cow"], name="animal")
```

By default, the original index and original name is reused.

```
>>> idx.to_series()
```

```
animal
Ant      Ant
Bear    Bear
Cow      Cow
Name: animal, dtype: str
```

To enforce a new index, specify new labels to ``index``:

```
>>> idx.to_series(index=[0, 1, 2])
```

```
0      Ant
1     Bear
```

```
2      Cow
Name: animal, dtype: str
```

To override the name of the resulting column, specify ``name``:

```
>>> idx.to_series(name="zoo")
animal
Ant      Ant
Bear     Bear
Cow      Cow
Name: zoo, dtype: str
```

`union(self, other, sort: bool | None = None)` from `pandas.core.indexes.base.Index`
Form the union of two Index objects.

If the Index objects are incompatible, both Index objects will be cast to `dtype('object')` first.

Parameters

`other` : Index or array-like
Index or an array-like object containing elements to form the union with the original Index.

`sort` : bool or None, default None
Whether to sort the resulting Index.

* None : Sort the result, except when

1. ``self`` and ``other`` are equal.
2. ``self`` or ``other`` has length 0.
3. Some values in ``self`` or ``other`` cannot be compared.
A `RuntimeWarning` is issued in this case.

* False : do not sort the result.

* True : Sort the result (which may raise `TypeError`).

Returns

Index

Returns a new Index object with all unique elements from both the original Index and the ``other`` Index.

See Also

`Index.unique` : Return unique values in the index.

`Index.intersection` : Form the intersection of two Index objects.

`Index.difference` : Return a new Index with elements of index not in ``other``.

Examples

Union matching dtypes

```
>>> idx1 = pd.Index([1, 2, 3, 4])
>>> idx2 = pd.Index([3, 4, 5, 6])
>>> idx1.union(idx2)
Index([1, 2, 3, 4, 5, 6], dtype='int64')
```

Union mismatched dtypes

```
>>> idx1 = pd.Index(["a", "b", "c", "d"])
>>> idx2 = pd.Index([1, 2, 3, 4])
>>> idx1.union(idx2)
Index(['a', 'b', 'c', 'd', 1, 2, 3, 4], dtype='object')
```

MultiIndex case

```
>>> idx1 = pd.MultiIndex.from_arrays(
...     [[1, 1, 2, 2], ["Red", "Blue", "Red", "Blue"]]
... )
>>> idx1
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue')],
           )
>>> idx2 = pd.MultiIndex.from_arrays(
...     [[3, 3, 2, 2], ["Red", "Green", "Red", "Green"]]
... )
```



```

... )
>>> idx2
MultiIndex([(3, 'Red'),
            (3, 'Green'),
            (2, 'Red'),
            (2, 'Green')],
           )
>>> idx1.union(idx2)
MultiIndex([(1, 'Blue'),
            (1, 'Red'),
            (2, 'Blue'),
            (2, 'Green'),
            (2, 'Red'),
            (3, 'Green'),
            (3, 'Red')],
           )
>>> idx1.union(idx2, sort=False)
MultiIndex([(1, 'Red'),
            (1, 'Blue'),
            (2, 'Red'),
            (2, 'Blue'),
            (3, 'Red'),
            (3, 'Green'),
            (2, 'Green')],
           )

```

`unique(self, level: Hashable | None = None) -> Self` from `pandas.core.indexes.base.Index`
Return unique values in the index.

Unique values are returned in order of appearance, this does NOT sort.

Parameters

level : int or hashable, optional
Only return values from specified level (for MultiIndex).
If int, gets the level by integer position, else by level name.

Returns

Index
Unique values in the index.

See Also

unique : Numpy array of unique values in that column.
Series.unique : Return unique values of Series object.

Examples

>>> idx = pd.Index([1, 1, 2, 3, 3])
>>> idx.unique()
Index([1, 2, 3], dtype='int64')

`view(self, cls=None)` from `pandas.core.indexes.base.Index`
Return a view of the Index with the specified dtype or a new Index instance.

This method returns a view of the calling Index object if no arguments are provided. If a dtype is specified through the `cls` argument, it attempts to return a view of the Index with the specified dtype. Note that viewing the Index as a different dtype reinterprets the underlying data, which can lead to unexpected results for non-numeric or incompatible dtype conversions.

Parameters

cls : data-type or ndarray sub-class, optional
Data-type descriptor of the returned view, e.g., `float32` or `int16`.
Omitting it results in the view having the same data-type as `self`.
This argument can also be specified as an ndarray sub-class, e.g., `np.int64` or `np.float32` which then specifies the type of the returned object.

Returns

Index or ndarray
A view of the Index. If `cls` is None, the returned object is an Index view with the same dtype as the calling object. If a numeric `cls` is specified an ndarray view with the new dtype is returned.

Raises

ValueError

If attempting to change to a dtype in a way that is not compatible with the original dtype's memory layout, for example, viewing an 'int64' Index as 'str'.

See Also

Index.copy : Returns a copy of the Index.

numpy.ndarray.view : Returns a new view of array with the same data.

Examples

```
>>> idx = pd.Index([-1, 0, 1])
```

```
>>> idx.view()
```

```
Index([-1, 0, 1], dtype='int64')
```

```
>>> idx.view(np.uint64)
```

```
array([18446744073709551615,           0,           1],
      dtype=uint64)
```

Viewing as 'int32' or 'float32' reinterprets the memory, which may lead to unexpected behavior:

```
>>> idx.view("float32")
```

```
array([ nan,   nan,  0.e+00,  0.e+00,  1.e-45,  0.e+00], dtype=float32)
```

where(self, cond, other=None) -> Index from pandas.core.indexes.base.Index
Replace values where the condition is False.

The replacement is taken from other.

Parameters

cond : bool array-like with the same length as self

Condition to select the values on.

other : scalar, or array-like, default None

Replacement if the condition is False.

Returns

pandas.Index

A copy of self with values replaced from other where the condition is False.

See Also

Series.where : Same method for Series.

DataFrame.where : Same method for DataFrame.

Examples

```
>>> idx = pd.Index(["car", "bike", "train", "tractor"])
```

```
>>> idx
```

```
Index(['car', 'bike', 'train', 'tractor'], dtype='str')
```

```
>>> idx.where(idx.isin(["car", "train"]), "other")
```

```
Index(['car', 'other', 'train', 'other'], dtype='str')
```

Readonly properties inherited from Index:

has_duplicates

Check if the Index has duplicate values.

Returns

bool

Whether or not the Index has duplicate values.

See Also

Index.is_unique : Inverse method that checks if it has unique values.

Examples

```

>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.has_duplicates
True

>>> idx = pd.Index([1, 5, 7])
>>> idx.has_duplicates
False

>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.has_duplicates
True

>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.has_duplicates
False

```

is_monotonic_decreasing

Return a boolean if the values are equal or decreasing.

Returns

bool

See Also

Index.is_monotonic_increasing : Check if the values are equal or increasing.

Examples

```

>>> pd.Index([3, 2, 1]).is_monotonic_decreasing
True
>>> pd.Index([3, 2, 2]).is_monotonic_decreasing
True
>>> pd.Index([3, 1, 2]).is_monotonic_decreasing
False

```

is_monotonic_increasing

Return a boolean if the values are equal or increasing.

Returns

bool

See Also

Index.is_monotonic_decreasing : Check if the values are equal or decreasing.

Examples

```

>>> pd.Index([1, 2, 3]).is_monotonic_increasing
True
>>> pd.Index([1, 2, 2]).is_monotonic_increasing
True
>>> pd.Index([1, 3, 2]).is_monotonic_increasing
False

```

nlevels

Number of levels.

shape

Return a tuple of the shape of the underlying data.

See Also

Index.size: Return the number of elements in the underlying data.
Index.ndim: Number of dimensions of the underlying data, by definition 1.
Index.dtype: Return the dtype object of the underlying data.
Index.values: Return an array representing the data in the Index.

Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.shape

```

(3,)

Data descriptors inherited from Index:

array

The ExtensionArray of the data backing this Index.

This property provides direct access to the underlying array data of an Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

``.array`` differs from ``.values``, which may require converting the data to a different form.

See Also

Index.to_numpy : Similar method that always returns a NumPy array.
Series.to_numpy : Similar method that always returns a NumPy array.

Notes

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ``.array`` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

Examples

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Index([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> idx = pd.Index(pd.Categorical(["a", "b", "a"]))
>>> idx.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

dtype

Return the dtype object of the underlying data.

See Also

Index.inferred_type: Return a string of the type inferred from the values.

Examples

```
-----
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.dtype
dtype('int64')
```

is_unique

Return if the index has unique values.

Returns

bool

See Also

Index.has_duplicates : Inverse method that checks if it has duplicate values.

Examples

```
>>> idx = pd.Index([1, 5, 7, 7])
>>> idx.is_unique
False
```

```
>>> idx = pd.Index([1, 5, 7])
>>> idx.is_unique
True
```

```
>>> idx = pd.Index(["Watermelon", "Orange", "Apple", "Watermelon"]).astype(
...     "category"
... )
>>> idx.is_unique
False
```

```
>>> idx = pd.Index(["Orange", "Apple", "Watermelon"]).astype("category")
>>> idx.is_unique
True
```

name

Return Index or MultiIndex name.

Returns

label (hashable object)
The name of the Index.

See Also

Index.set_names: Able to set new names partially and by level.

Index.rename: Able to set new names partially and by level.

Series.name: Corresponding Series property.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx
Index([1, 2, 3], dtype='int64', name='x')
>>> idx.name
'x'
```

names

Get names on index.

This method returns a FrozenList containing the names of the object.
It's primarily intended for internal use.

Returns

FrozenList

A FrozenList containing the object's names, contains None if the object does not have a name.

See Also

Index.name : Index name as a string, or None for MultiIndex.

Examples

```
>>> idx = pd.Index([1, 2, 3], name="x")
>>> idx.names
FrozenList(['x'])
```

```
>>> idx = pd.Index([1, 2, 3], name=("x", "y"))
>>> idx.names
FrozenList(['x', 'y'])
```

If the index does not have a name set:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.names
FrozenList([None])
```

Data and other attributes inherited from Index:

__pandas_priority__ = 2000

str = <class 'pandas.core.strings.accessor.StringMethods'>
Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method.
Patterned after Python's string methods, with some inspiration from
R's stringr package.

Parameters

data : Series or Index
The content of the Series or Index.

See Also

Series.str : Vectorized string functions for Series.
Index.str : Vectorized string functions for Index.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: str
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: str
```

Methods inherited from pandas.core.base.IndexOpsMixin:

__iter__(self) -> Iterator
Return an iterator of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

iterator
An iterator yielding scalar values from the Series.

See Also

Series.items : Lazily iterate over (index, value) tuples.

Examples

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> for x in s:
...     print(x)
1
2
3
```

```
factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.in
Encode the object as an enumerated type or categorical variable.
```

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. `factorize` is available as both a top-level function `:func:`pandas.factorize``, and as a method `:meth:`Series.factorize`` and `:meth:`Index.factorize``.

Parameters

```
-----
sort : bool, default False
    Sort `uniques` and shuffle `codes` to maintain the
    relationship.
use_na_sentinel : bool, default True
    If True, the sentinel -1 will be used for NaN values. If False,
    NaN values will be encoded as non-negative integers and will not drop the
    NaN from the uniques of the values.
```

Returns

```
-----
codes : ndarray
    An integer ndarray that's an indexer into `uniques`.
    ``uniques.take(codes)`` will have the same values as `values`.
uniques : ndarray, Index, or Categorical
    The unique valid values. When `values` is Categorical, `uniques`
    is a Categorical. When `values` is some other pandas object, an
    `Index` is returned. Otherwise, a 1-D ndarray is returned.
```

.. note::

Even if there's a missing value in `values`, `uniques` will
not contain an entry for it.

See Also

```
-----
cut : Discretize continuous-valued array.
unique : Find the unique value in an array.
```

Notes

```
-----
Reference :ref:`the user guide <reshaping.factorize>` for more examples.
```

Examples

```
-----
These examples all show factorize as a top-level method like
``pd.factorize(values)``. The results are identical for methods like
:meth:`Series.factorize`.
```

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O")
... )
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is the maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When `use_na_sentinel=True` (the default), missing values are indicated in the `codes` with the sentinel value `-1` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", None, "a", "c", "b"], dtype="O")
... )
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([ 1.,  2., nan])
```

`item(self)`
Return the first element of the underlying data as a Python scalar.

Returns

scalar

The first element of Series or Index.

Raises

ValueError

If the data is not length = 1.

See Also

`Index.values` : Returns an array representing the data in the Index.

`Series.head` : Returns the first `n` rows.

Examples

```
>>> s = pd.Series([1])
>>> s.item()
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
```



```

'a'

nunique(self, dropna: bool = True) -> int
    Return number of unique elements in the object.

    Excludes NA values by default.

Parameters
-----
dropna : bool, default True
    Don't include NaN in the count.

Returns
-----
int
    An integer indicating the number of unique elements in the object.

See Also
-----
DataFrame.nunique: Method nunique for DataFrame.
Series.count: Count non-NA/null observations in the Series.

Examples
-----
>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0    1
1    3
2    5
3    7
4    7
dtype: int64

>>> s.nunique()
4

```

searchsorted(
 self,
 value: NumpyValueArrayLike | ExtensionArray,
 side: Literal['left', 'right'] = 'left',
 sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted Index `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::

The Index *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

value : array-like or scalar
 Values to insert into `self`.
side : {'left', 'right'}, optional
 If 'left', the index of the first suitable location found is given. If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
 Optional array of integer indices that sort `self` into ascending order. They are typically the result of ``np.argsort``.

Returns

int or array of int
 A scalar or array of insertion points with the same shape as `value`.

See Also

sort_values : Sort by the values along either axis.
numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

```
>>> ser = pd.Series([1, 2, 3])
```

```
>>> ser
```

```
0    1
```

```
1    2
```

```
2    3
```

```
dtype: int64
```

```
>>> ser.searchsorted(4)
```

```
np.int64(3)
```

```
>>> ser.searchsorted([0, 4])
```

```
array([0, 3])
```

```
>>> ser.searchsorted([1, 3], side="left")
```

```
array([0, 2])
```

```
>>> ser.searchsorted([1, 3], side="right")
```

```
array([1, 3])
```

```
>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
```

```
>>> ser
```

```
0    2000-03-11
```

```
1    2000-03-12
```

```
2    2000-03-13
```

```
dtype: datetime64[us]
```

```
>>> ser.searchsorted("3/14/2000")
```

```
np.int64(3)
```

```
>>> ser = pd.Categorical(
```

```
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
```

```
... )
```

```
>>> ser
```

```
['apple', 'bread', 'bread', 'cheese', 'milk']
```

```
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']
```

```
>>> ser.searchsorted("bread")
```

```
np.int64(1)
```

```
>>> ser.searchsorted(["bread"], side="right")
```

```
array([3])
```

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])
```

```
>>> ser
```

```
0    2
```

```
1    1
```

```
2    3
```

```
dtype: int64
```

```
>>> ser.searchsorted(1) # doctest: +SKIP
```

```
0 # wrong result, correct would be 1
```

```
to_list = tolist(self) -> list
```

```
to_numpy(
```

```
    self,
```

```
    dtype: npt.DTypeLike | None = None,
```

```
    copy: bool = False,
```

```
    na_value: object = <no_default>,
```

```
    **kwargs
```

```
) -> np.ndarray
```

A NumPy ndarray representing the values in this Series or Index.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

**kwargs

Additional keywords passed through to the ``to\_numpy`` method of the underlying array (for extension arrays).

Returns

numpy.ndarray

The NumPy ndarray holding the values from this Series or Index. The dtype of the array may differ. See Notes.

See Also

Series.array : Get the actual data stored within.

Index.array : Get the actual data stored within.

DataFrame.to_numpy : Similar method for DataFrame.

Notes

The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` *may* require copying data and coercing the result to a NumPy type (possibly object), which may be expensive. When you need a no-copy reference to the underlying data, :attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of ``to\_numpy()`` for various dtypes within pandas.

dtype	array type
category[T]	ndarray[T] (same dtype as input)
period	ndarray[object] (Periods)
interval	ndarray[object] (Intervals)
IntegerNA	ndarray[object]
datetime64[ns]	datetime64[ns]
datetime64[ns, tz]	ndarray[object] (Timestamps)

Examples

>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)

Specify the `dtype` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
 Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
 dtype=object)

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

>>> ser.to_numpy(dtype="datetime64[ns]")

```

... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
      dtype='datetime64[ns]')

tolist(self) -> list
    Return a list of the values.

    These are each a scalar type, which is a Python scalar
    (for str, int, float) or a pandas scalar
    (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    list
        List containing the values as Python or pandas scalars.

    See Also
    -----
    numpy.ndarray.tolist : Return the array as an a.ndim-levels deep
        nested list of Python scalars.

    Examples
    -----
    For Series

    >>> s = pd.Series([1, 2, 3])
    >>> s.tolist()
    [1, 2, 3]

    For Index:

    >>> idx = pd.Index([1, 2, 3])
    >>> idx
    Index([1, 2, 3], dtype='int64')

    >>> idx.tolist()
    [1, 2, 3]

transpose(self, *args, **kwargs) -> Self
    Return the transpose, which is by definition self.

    Returns
    -----
    %(klass)s

value_counts(
    self,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    bins=None,
    dropna: bool = True
) -> Series
    Return a Series containing counts of unique values.

    The resulting object will be in descending order so that the
    first element is the most frequently-occurring element.
    Excludes NA values by default.

    Parameters
    -----
    normalize : bool, default False
        If True then the object returned will contain the relative
        frequencies of the unique values.
    sort : bool, default True
        Stable sort by frequencies when True. Preserve the order of the data
        when False.

    .. versionchanged:: 3.0.0

        Prior to 3.0.0, the sort was unstable.
    ascending : bool, default False
        Sort in ascending order.
    bins : int, optional
        Rather than count values, group them into half-open bins,
        a convenience for ``pd.cut``, only works with numeric data.
    dropna : bool, default True

```

Don't include counts of NaN.

Returns

Series

Series containing counts of unique values.

See Also

Series.count: Number of non-NA elements in a Series.

DataFrame.count: Number of non-NA elements in a DataFrame.

DataFrame.value_counts: Equivalent method on DataFrames.

Examples

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
```

```
>>> index.value_counts()
```

```
3.0    2
```

```
1.0    1
```

```
2.0    1
```

```
4.0    1
```

```
Name: count, dtype: int64
```

With `normalize` set to `True`, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
```

```
>>> s.value_counts(normalize=True)
```

```
3.0    0.4
```

```
1.0    0.2
```

```
2.0    0.2
```

```
4.0    0.2
```

```
Name: proportion, dtype: float64
```

bins

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
```

```
(0.996, 2.0]    2
```

```
(2.0, 3.0]    2
```

```
(3.0, 4.0]    1
```

```
Name: count, dtype: int64
```

dropna

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
```

```
3.0    2
```

```
1.0    1
```

```
2.0    1
```

```
4.0    1
```

```
NaN    1
```

```
Name: count, dtype: int64
```

Categorical Dtypes

Rows with categorical type will be counted as one group if they have same categories and order.

In the example below, even though `a`, `c`, and `d`

all have the same data types of `category`,

only `c` and `d` will be counted as one group

since `a` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
```

```
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
```

```
>>> df
```

```
   a  b  c  d
```

```
0  1  2  3  3
```

```
>>> df.dtypes
```

```
a    category
```

```
b         str
```

```

c    category
d    category
dtype: object

>>> df.dtypes.value_counts()
category    2
category    1
str         1
Name: count, dtype: int64

```

Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

Index : Immutable sequence used for indexing and alignment.

Examples

For Series:

```

>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.T
0    Ant
1    Bear
2    Cow
dtype: str

```

For Index:

```

>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')

```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

bool

If Index is empty, return True, if not return False.

See Also

Index.size : Return the number of elements in the underlying data.

Examples

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False

```

```

>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True

```

If we only have NaNs in our DataFrame, it is not considered empty!

```

>>> idx = pd.Index([np.nan, np.nan])
>>> idx

```

```
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

Series.ndim : Number of dimensions of the underlying data.

Series.size : Return the number of elements in the underlying data.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.nbytes
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.nbytes
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

Series.size: Return the number of elements in the underlying data.

Series.shape: Return a tuple of the shape of the underlying data.

Series.dtype: Return the dtype object of the underlying data.

Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0      Ant
1      Bear
2      Cow
dtype: str
>>> s.ndim
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

size

Return the number of elements in the underlying data.

See Also

Series.ndim: Number of dimensions of the underlying data, by definition 1.

Series.shape: Return a tuple of the shape of the underlying data.

Series.dtype: Return the dtype object of the underlying data.

Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
```

```
>>> s
0    Ant
1    Bear
2    Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

`__array_priority__` = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`
Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ```other``` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
   height  weight
elk     1.5    500
moose    2.6    800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
   height  weight
elk     3.0   501.5
moose    4.1   801.5
```

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
   height  weight
elk     2.0   501.5
moose    3.1   801.5
```

Keys of a dictionary are aligned to the DataFrame, based on column names;
each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
   height  weight
elk     2.0   501.5
moose    3.1   801.5
```

When ```other``` is a `:class:`Series``, the index of ```other``` is aligned with the

columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN      NaN    NaN     NaN
moose  NaN      NaN    NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"])
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN      NaN
elk        1.7      NaN
moose      3.0      NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
```

```

__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)

-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
__hash__ = None

-----
Methods inherited from pandas.core.base.PandasObject:
__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:
__dir__(self) -> list[str]
    Provide method name lookup and completion.

Notes
-----
    Only provide 'public' methods.

```

class Timestamp(pandas._libs.tslibs.timestamps.Timestamp)

```

    Timestamp(
        ts_input=<object object at 0x000001A9511B16D0>,
        year=None,
        month=None,
        day=None,
        hour=None,
        minute=None,
        second=None,
        microsecond=None,
        tzinfo=None,
        *,
        nanosecond=None,
        tz=<object object at 0x000001A9511B16D0>,
        unit=None,
        fold=None
    )

```

Pandas replacement for python datetime.datetime object.

Timestamp is the pandas equivalent of python's Datetime and is interchangeable with it in most cases. It's the type used for the entries that make up a DatetimeIndex, and other timeseries oriented data structures in pandas.

Parameters

```

-----
ts_input : datetime-like, str, int, float
    Value to be converted to Timestamp.
year : int
    Value of year.
month : int
    Value of month.
day : int
    Value of day.
hour : int, optional, default 0
    Value of hour.
minute : int, optional, default 0
    Value of minute.
second : int, optional, default 0
    Value of second.
microsecond : int, optional, default 0
    Value of microsecond.
tzinfo : datetime.tzinfo, optional, default None
    Timezone info.
nanosecond : int, optional, default 0
    Value of nanosecond.
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
    Time zone for time which Timestamp will have.
unit : str
    Unit used for conversion if ts_input is of type int or float. The
    valid values are 'W', 'D', 'h', 'm', 's', 'ms', 'us', and 'ns'. For
    example, 's' means seconds and 'ms' means milliseconds.

    For float inputs, the result will be stored in nanoseconds, and
    the unit attribute will be set as ``'ns'``.
fold : {0, 1}, default None, keyword-only
    Due to daylight saving time, one wall clock time can occur twice
    when shifting from summer to winter time; fold describes whether the
    datetime-like corresponds to the first (0) or the second time (1)
    the wall clock hits the ambiguous time.

See Also
-----
Timedelta : Represents a duration, the difference between two dates or times.
datetime.datetime : Python datetime.datetime object.

Notes
-----
There are essentially three calling conventions for the constructor. The
primary form accepts four parameters. They can be passed by position or
keyword.

The other two forms mimic the parameters from ``datetime.datetime``. They
can be passed by either position or keyword, but not both mixed together.

Examples
-----
Using the primary calling convention:

This converts a datetime-like string

>>> pd.Timestamp('2017-01-01T12')
Timestamp('2017-01-01 12:00:00')

This converts a float representing a Unix epoch in units of seconds

>>> pd.Timestamp(1513393355.5, unit='s')
Timestamp('2017-12-16 03:02:35.500000')

This converts an int representing a Unix-epoch in units of weeks

>>> pd.Timestamp(1535, unit='W')
Timestamp('1999-06-03 00:00:00')

This converts an int representing a Unix-epoch in units of seconds
and for a particular timezone

>>> pd.Timestamp(1513393355, unit='s', tz='US/Pacific')
Timestamp('2017-12-15 19:02:35-0800', tz='US/Pacific')

Using the other two forms that mimic the API for ``datetime.datetime``:

```

```

>>> pd.Timestamp(2017, 1, 1, 12)
Timestamp('2017-01-01 12:00:00')

>>> pd.Timestamp(year=2017, month=1, day=1, hour=12)
Timestamp('2017-01-01 12:00:00')

Method resolution order:
  Timestamp
  pandas._libs.tslibs.timestamps._Timestamp
  pandas._libs.tslibs.base.ABCTimestamp
  datetime.datetime
  datetime.date
  builtins.object

Methods defined here:

astimezone = tz_convert(self, tz)

ceil(self, freq, ambiguous='raise', nonexistent='raise')
    Return a new Timestamp ceiled to this resolution.

    Parameters
    -----
    freq : str
        Frequency string indicating the ceiling resolution.
    ambiguous : bool or {'raise', 'NaT'}, default 'raise'
        The behavior is as follows:

        * bool contains flags to determine if time is dst or not (note
          that this flag is only applicable for ambiguous fall dst dates).
        * 'NaT' will return NaT for an ambiguous time.
        * 'raise' will raise a ValueError for an ambiguous time.

    nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'
        A nonexistent time does not exist in a particular timezone
        where clocks moved forward due to DST.

        * 'shift_forward' will shift the nonexistent time forward to the
          closest existing time.
        * 'shift_backward' will shift the nonexistent time backward to the
          closest existing time.
        * 'NaT' will return NaT where there are nonexistent times.
        * timedelta objects will shift nonexistent times by the timedelta.
        * 'raise' will raise a ValueError if there are
          nonexistent times.

    Raises
    -----
    ValueError if the freq cannot be converted.

    See Also
    -----
    Timestamp.floor : Round down a Timestamp to the specified resolution.
    Timestamp.round : Round a Timestamp to the specified resolution.
    Series.dt.ceil : Ceil the datetime values in a Series.

    Notes
    -----
    If the Timestamp has a timezone, ceiling will take place relative to the
    local ("wall") time and re-localized to the same timezone. When ceiling
    near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
    control the re-localization behavior.

    Examples
    -----
    Create a timestamp object:

    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')

    A timestamp can be ceiled using multiple frequency units:

    >>> ts.ceil(freq='h') # hour
    Timestamp('2020-03-14 16:00:00')

    >>> ts.ceil(freq='min') # minute
    Timestamp('2020-03-14 15:33:00')

```

```
>>> ts.ceil(freq='s') # seconds
Timestamp('2020-03-14 15:32:53')

>>> ts.ceil(freq='us') # microseconds
Timestamp('2020-03-14 15:32:52.192549')
```

`freq` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):`

```
>>> ts.ceil(freq='5min')
Timestamp('2020-03-14 15:35:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.ceil(freq='1h30min')
Timestamp('2020-03-14 16:30:00')
```

Analogous for `pd.NaT`:`

```
>>> pd.NaT.ceil()
NaT
```

When rounding near a daylight savings time transition, use `ambiguous` or nonexistent` to control how the timestamp should be re-localized.`

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.ceil("h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.ceil("h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

`ctime(self)`

Return a `ctime()` style string representing the `Timestamp`.

This method returns a string representing the date and time in the format returned by the standard library's `time.ctime()` function, which is typically in the form 'Day Mon DD HH:MM:SS YYYY'.

If the `Timestamp` is outside the range supported by Python's standard library, a NotImplementedError` is raised.`

Returns

str

A string representing the `Timestamp` in `ctime` format.

See Also

`time.ctime` : Return a string representing time in `ctime` format.

`Timestamp` : Represents a single timestamp, similar to `datetime`.`

`datetime.datetime.ctime` : Return a `ctime` style string from a `datetime` object.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00')
```

```
>>> ts.ctime()
```

```
'Sun Jan  1 10:00:00 2023'
```

`date(self)`

Returns `datetime.date` with the same year, month, and day.`

This method extracts the date component from the `Timestamp` and returns it as a datetime.date` object, discarding the time information.`

Returns

`datetime.date`

The date part of the `Timestamp`.`

See Also

`Timestamp` : Represents a single timestamp, similar to `datetime`.`

`datetime.datetime.date` : Extract the date component from a `datetime` object.`

Examples

Extract the date from a Timestamp:

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.date()
datetime.date(2023, 1, 1)
```

dst(self)

Return the daylight saving time (DST) adjustment.

This method returns the DST adjustment as a `datetime.timedelta` object if the Timestamp is timezone-aware and DST is applicable.

See Also

`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2000-06-01 00:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2000-06-01 00:00:00+0200', tz='Europe/Brussels')
>>> ts.dst()
datetime.timedelta(seconds=3600)
```

floor(self, freq, ambiguous='raise', nonexistent='raise')

Return a new Timestamp floored to this resolution.

Parameters

`freq` : str

Frequency string indicating the flooring resolution.

`ambiguous` : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

`nonexistent` : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * `timedelta` objects will shift nonexistent times by the `timedelta`.
- * 'raise' will raise a ValueError if there are nonexistent times.

Raises

ValueError if the `freq` cannot be converted.

See Also

`Timestamp.ceil` : Round up a Timestamp to the specified resolution.

`Timestamp.round` : Round a Timestamp to the specified resolution.

`Series.dt.floor` : Round down the datetime values in a Series.

Notes

If the Timestamp has a timezone, flooring will take place relative to the local ("wall") time and re-localized to the same timezone. When flooring near daylight savings time, use `nonexistent` and `ambiguous` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be floored using multiple frequency units:

```
>>> ts.floor(freq='h') # hour
Timestamp('2020-03-14 15:00:00')
```

```
>>> ts.floor(freq='min') # minute
Timestamp('2020-03-14 15:32:00')
```

```
>>> ts.floor(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')
```

```
>>> ts.floor(freq='ns') # nanoseconds
Timestamp('2020-03-14 15:32:52.192548651')
```

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

```
>>> ts.floor(freq='5min')
Timestamp('2020-03-14 15:30:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.floor(freq='1h30min')
Timestamp('2020-03-14 15:00:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.floor()
NaT
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 03:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.floor("2h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.floor("2h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

`isocalendar(self)`

Return a named tuple containing ISO year, week number, and weekday.

See Also

`DatetimeIndex.isocalendar` : Return a 3-tuple containing ISO year, week number, and weekday for the given `DatetimeIndex` object.

`datetime.date.isocalendar` : The equivalent method for `'datetime.date'` objects.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00')
```

```
>>> ts.isocalendar()
```

```
datetime.ISoCalendarDate(year=2022, week=52, weekday=7)
```

`isoweekday(self)`

Return the day of the week represented by the date.

Monday == 1 ... Sunday == 7.

See Also

`Timestamp.weekday` : Return the day of the week with Monday=0, Sunday=6.

`Timestamp.isocalendar` : Return a tuple containing ISO year, week number and weekday.

`datetime.date.isoweekday` : Equivalent method in `datetime` module.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00')
```

```

>>> ts.isoweekday()
7

```

replace(
self,
year=None,
month=None,
day=None,
hour=None,
minute=None,
second=None,
microsecond=None,
nanosecond=None,
tzinfo=<class 'object'>,
fold=None
)

Implements datetime.replace, handles nanoseconds.

This method creates a new ``Timestamp`` object by replacing the specified fields with new values. The new ``Timestamp`` retains the original fields that are not explicitly replaced. This method handles nanoseconds, and the ``tzinfo`` parameter allows for timezone replacement without conversion.

Parameters

year : int, optional
The year to replace. If ``None``, the year is not changed.

month : int, optional
The month to replace. If ``None``, the month is not changed.

day : int, optional
The day to replace. If ``None``, the day is not changed.

hour : int, optional
The hour to replace. If ``None``, the hour is not changed.

minute : int, optional
The minute to replace. If ``None``, the minute is not changed.

second : int, optional
The second to replace. If ``None``, the second is not changed.

microsecond : int, optional
The microsecond to replace. If ``None``, the microsecond is not changed.

nanosecond : int, optional
The nanosecond to replace. If ``None``, the nanosecond is not changed.

tzinfo : tz-convertible, optional
The timezone information to replace. If ``None``, the timezone is not changed.

fold : int, optional
The fold information to replace. If ``None``, the fold is not changed.

Returns

`Timestamp``
A new ``Timestamp`` object with the specified fields replaced.

See Also

`Timestamp`` : Represents a single timestamp, similar to ``datetime``.
`to_datetime`` : Converts various types of data to datetime.

Notes

The ``replace`` method does not perform timezone conversions. If you need to convert the timezone, use the ``tz_convert`` method instead.

Examples

Create a timestamp object:

```

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')

```

Replace year and the hour:

```

>>> ts.replace(year=1999, hour=10)
Timestamp('1999-03-14 10:32:52.192548651+0000', tz='UTC')

```

Replace timezone (not a conversion):

```

>>> import zoneinfo

```



```
>>> ts.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
Timestamp('2020-03-14 15:32:52.192548651-0700', tz='US/Pacific')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
NaT
```

```
round(self, freq, ambiguous='raise', nonexistent='raise')
Round the Timestamp to the specified resolution.
```

This method rounds the given Timestamp down to a specified frequency level. It is particularly useful in data analysis to normalize timestamps to regular frequency intervals. For instance, rounding to the nearest minute, hour, or day can help in time series comparisons or resampling operations.

Parameters

freq : str

Frequency string indicating the rounding resolution.

ambiguous : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Returns

a new Timestamp rounded to the given resolution of `freq`

Raises

ValueError if the freq cannot be converted

See Also

datetime.round : Similar behavior in native Python datetime module.

Timestamp.floor : Round the Timestamp downward to the nearest multiple of the specified frequency.

Timestamp.ceil : Round the Timestamp upward to the nearest multiple of the specified frequency.

Notes

If the Timestamp has a timezone, rounding will take place relative to the local ("wall") time and re-localized to the same timezone. When rounding near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be rounded using multiple frequency units:

```
>>> ts.round(freq='h') # hour
Timestamp('2020-03-14 16:00:00')
```

```
>>> ts.round(freq='min') # minute
```

```
Timestamp('2020-03-14 15:33:00')

>>> ts.round(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')

>>> ts.round(freq='ms') # milliseconds
Timestamp('2020-03-14 15:32:52.193000')

`freq` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

>>> ts.round(freq='5min')
Timestamp('2020-03-14 15:35:00')

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

>>> ts.round(freq='1h30min')
Timestamp('2020-03-14 15:00:00')
```

Analogous for `pd.NaT`:

```
>>> pd.NaT.round()
NaT
```

When rounding near a daylight savings time transition, use `ambiguous` or `nonexistent` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")

>>> ts_tz.round("h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')

>>> ts_tz.round("h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

`strftime(self, format)`

Return a formatted string of the Timestamp.

Parameters

`format` : str

Format string to convert Timestamp to string.

See `strftime` documentation for more information on the format string:

<https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>.

See Also

`Timestamp.isoformat` : Return the time formatted according to ISO 8601.

`pd.to_datetime` : Convert argument to datetime.

`Period.strftime` : Format a single Period.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.strftime('%Y-%m-%d %X')
'2020-03-14 15:32:52'
```

`time(self)`

Return time object with same time but with `tzinfo=None`.

This method extracts the time part of the `Timestamp` object, excluding any timezone information. It returns a `datetime.time` object which only represents the time (hours, minutes, seconds, and microseconds).

See Also

`Timestamp.date` : Return date object with same year, month and day.

`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.

`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.time()
datetime.time(10, 0)
```

```

timetuple(self)
    Return time tuple, compatible with time.localtime().

    This method converts the `Timestamp` into a time tuple, which is compatible
    with functions like `time.localtime()`. The time tuple is a named tuple with
    attributes such as year, month, day, hour, minute, second, weekday,
    day of the year, and daylight savings indicator.

    See Also
    -----
    time.localtime : Converts a POSIX timestamp into a time tuple.
    Timestamp : The `Timestamp` that represents a specific point in time.
    datetime.datetime.timetuple : Equivalent method in the `datetime` module.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00')
    >>> ts
    Timestamp('2023-01-01 10:00:00')
    >>> ts.timetuple()
    time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1,
    tm_hour=10, tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=-1)

timetz(self)
    Return time object with same time and tzinfo.

    This method returns a datetime.time object with
    the time and tzinfo corresponding to the pd.Timestamp
    object, ignoring any information about the day/date.

    See Also
    -----
    datetime.datetime.timetz : Return datetime.time object with the
    same time attributes as the datetime object.
    datetime.time : Class to represent the time of day, independent
    of any particular day.
    datetime.datetime.tzinfo : Attribute of datetime.datetime objects
    representing the timezone of the datetime object.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
    >>> ts
    Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
    >>> ts.timetz()
    datetime.time(10, 0, tzinfo=<DstTzInfo 'Europe/Brussels' CET+1:00:00 STD>)

to_julian_date(self) -> np.float64
    Convert Timestamp to a Julian Date.

    This method returns the number of days as a float since
    0 Julian date, which is noon January 1, 4713 BC.

    See Also
    -----
    Timestamp.toordinal : Return proleptic Gregorian ordinal.
    Timestamp.timestamp : Return POSIX timestamp as float.
    Timestamp : Represents a single timestamp.

    Examples
    -----
    >>> ts = pd.Timestamp('2020-03-14T15:32:52')
    >>> ts.to_julian_date()
    2458923.147824074

toordinal(self)
    Return proleptic Gregorian ordinal. January 1 of year 1 is day 1.

    The proleptic Gregorian ordinal is a continuous count of days since
    January 1 of year 1, which is considered day 1. This method converts
    the `Timestamp` to its equivalent ordinal number, useful for date arithmetic
    and comparison operations.

    See Also
    -----
    datetime.datetime.toordinal : Equivalent method in the `datetime` module.
    Timestamp : The `Timestamp` that represents a specific point in time.

```

Timestamp.fromordinal : Create a `Timestamp` from an ordinal.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:50')
>>> ts
Timestamp('2023-01-01 10:00:50')
>>> ts.toordinal()
738521
```

tz_convert(self, tz)

Convert timezone-aware Timestamp to another time zone.

This method is used to convert a timezone-aware Timestamp object to a different time zone. The original UTC time remains the same; only the time zone information is changed. If the Timestamp is timezone-naive, a TypeError is raised.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for time which Timestamp will be converted to.
None will remove timezone holding UTC time.

Returns

converted : Timestamp

Raises

TypeError
If Timestamp is tz-naive.

See Also

Timestamp.tz_localize : Localize the Timestamp to a timezone.
DatetimeIndex.tz_convert : Convert a DatetimeIndex to another time zone.
DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.
datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a timestamp object with UTC timezone:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Change to Tokyo timezone:

```
>>> ts.tz_convert(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Can also use ``astimezone``:

```
>>> ts.astimezone(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_convert(tz='Asia/Tokyo')
NaT
```

tz_localize(self, tz, ambiguous='raise', nonexistent='raise')

Localize the Timestamp to a timezone.

Convert naive Timestamp to local time zone or remove timezone from timezone-aware Timestamp.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for time which Timestamp will be converted to.
None will remove timezone holding local time.

ambiguous : bool, 'NaT', default 'raise'
When clocks moved backward due to DST, ambiguous times may arise.

For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be handled.

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

The behavior is as follows:

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Returns

localized : Timestamp

Raises

TypeError

If the Timestamp is tz-aware and tz is not None.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.

Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.

datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a naive timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

```
>>> ts
```

```
Timestamp('2020-03-14 15:32:52.192548651')
```

Add 'Europe/Stockholm' as timezone:

```
>>> ts.tz_localize(tz='Europe/Stockholm')
```

```
Timestamp('2020-03-14 15:32:52.192548651+0100', tz='Europe/Stockholm')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_localize()
```

```
NaT
```

tzname(self)

Return time zone name.

This method returns the name of the Timestamp's time zone as a string.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.

Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
```

```

>>> ts.tzname()
'CET'

utcoffset(self)
    Return utc offset.

    This method returns the difference between UTC and the local time
    as a `timedelta` object. It is useful for understanding the time
    difference between the current timezone and UTC.

    Returns
    -----
    timedelta
        The difference between UTC and the local time as a `timedelta` object.

    See Also
    -----
    datetime.datetime.utcnow :
        Standard library method to get the UTC offset of a datetime object.
    Timestamp.tzname : Return the name of the timezone.
    Timestamp.dst : Return the daylight saving time (DST) adjustment.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
    >>> ts
    Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
    >>> ts.utcoffset()
    datetime.timedelta(seconds=3600)

utctimetuple(self)
    Return UTC time tuple, compatible with `time.localtime()`.

    This method converts the Timestamp to UTC and returns a time tuple
    containing 9 components: year, month, day, hour, minute, second,
    weekday, day of year, and DST flag. This is particularly useful for
    converting a Timestamp to a format compatible with time module functions.

    Returns
    -----
    time.struct_time
        A time.struct_time object representing the UTC time.

    See Also
    -----
    datetime.datetime.utctimetuple :
        Return UTC time tuple, compatible with time.localtime().
    Timestamp.timetuple : Return time tuple of local time.
    time.struct_time : Time tuple structure used by time functions.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
    >>> ts
    Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
    >>> ts.utctimetuple()
    time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1, tm_hour=9,
    tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=0)

weekday(self)
    Return the day of the week represented by the date.

    Monday == 0 ... Sunday == 6.

    See Also
    -----
    Timestamp.dayofweek : Return the day of the week with Monday=0, Sunday=6.
    Timestamp.isoweekday : Return the day of the week with Monday=1, Sunday=7.
    datetime.date.weekday : Equivalent method in datetime module.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01')
    >>> ts
    Timestamp('2023-01-01 00:00:00')
    >>> ts.weekday()
    6

```

Class methods defined here:

```
combine(date, time)
    Timestamp.combine(date, time)
```

Combine a date and time into a single Timestamp object.

This method takes a `date` object and a `time` object and combines them into a single `Timestamp` that has the same date and time fields.

Parameters

date : datetime.date
 The date part of the Timestamp.
time : datetime.time
 The time part of the Timestamp.

Returns

Timestamp
 A new `Timestamp` object representing the combined date and time.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Examples

>>> from datetime import date, time
>>> pd.Timestamp.combine(date(2020, 3, 14), time(15, 30, 15))
Timestamp('2020-03-14 15:30:15')

```
fromordinal(ordinal, tz=None)
    Construct a timestamp from a proleptic Gregorian ordinal.
```

This method creates a `Timestamp` object corresponding to the given proleptic Gregorian ordinal, which is a count of days from January 1, 0001 (using the proleptic Gregorian calendar). The time part of the `Timestamp` is set to midnight (00:00:00) by default.

Parameters

ordinal : int
 Date corresponding to a proleptic Gregorian ordinal.
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
 Time zone for the Timestamp.

Returns

Timestamp
 A `Timestamp` object representing the specified ordinal date.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Notes

By definition there cannot be any tz info on the ordinal itself.

Examples

Convert an ordinal to a `Timestamp`:

```
>>> pd.Timestamp.fromordinal(737425)
Timestamp('2020-01-01 00:00:00')
```

Create a `Timestamp` from an ordinal with timezone information:

```
>>> pd.Timestamp.fromordinal(737425, tz='UTC')
Timestamp('2020-01-01 00:00:00+0000', tz='UTC')
```

```

fromtimestamp(ts, tz=None)
    Create a `Timestamp` object from a POSIX timestamp.

    This method converts a POSIX timestamp (the number of seconds since
    January 1, 1970, 00:00:00 UTC) into a `Timestamp` object. The resulting
    `Timestamp` can be localized to a specific time zone if provided.

    Parameters
    -----
    ts : float
        The POSIX timestamp to convert, representing seconds since
        the epoch (1970-01-01 00:00:00 UTC).
    tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, optional
        Time zone for the `Timestamp`. If not provided, the `Timestamp` will
        be timezone-naive (i.e., without time zone information).

    Returns
    -----
    Timestamp
        A `Timestamp` object representing the given POSIX timestamp.

    See Also
    -----
    Timestamp : Represents a single timestamp, similar to `datetime`.
    to_datetime : Converts various types of data to datetime.
    datetime.datetime.fromtimestamp : Returns a datetime from a POSIX timestamp.

    Examples
    -----
    Convert a POSIX timestamp to a `Timestamp`:

    >>> pd.Timestamp.fromtimestamp(1584199972) # doctest: +SKIP
    Timestamp('2020-03-14 15:32:52')

    Note that the output may change depending on your local time and time zone:

    >>> pd.Timestamp.fromtimestamp(1584199972, tz='UTC') # doctest: +SKIP
    Timestamp('2020-03-14 15:32:52+0000', tz='UTC')

now(tz=None)
    Return new Timestamp object representing current time local to tz.

    This method returns a new `Timestamp` object that represents the current time.
    If a timezone is provided, either through a timezone object or an IANA
    standard timezone identifier, the current time will be localized to that
    timezone.
    Otherwise, it returns the current local time.

    Parameters
    -----
    tz : str or timezone object, default None
        Timezone to localize to.

    See Also
    -----
    to_datetime : Convert argument to datetime.
    Timestamp.utcnow : Return a new Timestamp representing UTC day and time.
    Timestamp.today : Return the current time in the local timezone.

    Examples
    -----
    >>> pd.Timestamp.now() # doctest: +SKIP
    Timestamp('2020-11-16 22:06:16.378782')

    If you want a specific timezone, in this case 'Brazil/East':

    >>> pd.Timestamp.now('Brazil/East') # doctest: +SKIP
    Timestamp('2025-11-11 22:17:59.609943-03:00')

    Analogous for ``pd.NaT``:

    >>> pd.NaT.now()
    NaT

strptime(date_string, format)
    Convert string argument to datetime.

```


This method is not implemented; calling it will raise `NotImplementedError`. Use `pd.to_datetime()` instead.

Parameters

`date_string` : str
String to convert to a datetime.
`format` : str, default None
The format string to parse time, e.g. "%d/%m/%Y".

See Also

`pd.to_datetime` : Convert argument to datetime.
`datetime.datetime.strptime` : Return a datetime corresponding to a string representing a date and time, parsed according to a separate format string.
`datetime.datetime.strftime` : Return a string representing the date and time, controlled by an explicit format string.
`Timestamp.isoformat` : Return the time formatted according to ISO 8601.

Examples

>>> pd.Timestamp.strptime("2023-01-01", "%d/%m/%Y")
Traceback (most recent call last):
NotImplementedError

`today(tz=None)`
Return the current time in the local timezone.

This differs from `datetime.today()` in that it can be localized to a passed timezone.

Parameters

`tz` : str or timezone object, default None
Timezone to localize to.

See Also

`datetime.datetime.today` : Returns the current local date.
`Timestamp.now` : Returns current time with optional timezone.
`Timestamp` : A class representing a specific timestamp.

Examples

>>> pd.Timestamp.today() # doctest: +SKIP
Timestamp('2020-11-16 22:37:39.969883')

Analogous for ``pd.NaT``:

>>> pd.NaT.today()
NaT

`utcfromtimestamp(ts)`
`Timestamp.utcfromtimestamp(ts)`

Construct a timezone-aware UTC datetime from a POSIX timestamp.

This method creates a datetime object from a POSIX timestamp, keeping the Timestamp object's timezone.

Parameters

`ts` : float
POSIX timestamp.

See Also

`Timezone.tzname` : Return time zone name.
`Timestamp.utcnow` : Return a new Timestamp representing UTC day and time.
`Timestamp.fromtimestamp` : Transform timestamp[, tz] to tz's local time from POSIX timestamp.

Notes

`Timestamp.utcfromtimestamp` behavior differs from `datetime.utcfromtimestamp` in returning a timezone-aware object.

Examples

```
>>> pd.Timestamp.utctimestamp(1584199972)
Timestamp('2020-03-14 15:32:52+0000', tz='UTC')
```

```
utcnow()
Timestamp.utcnow()
```

Return a new Timestamp representing UTC day and time.

See Also

Timestamp : Constructs an arbitrary datetime.
Timestamp.now : Return the current local date and time, which can be timezone-aware.
Timestamp.today : Return the current local date and time with timezone information set to None.
to_datetime : Convert argument to datetime.
date_range : Return a fixed frequency DatetimeIndex.
Timestamp.utctimetuple : Return UTC time tuple, compatible with time.localtime().

Examples

```
>>> pd.Timestamp.utcnow() # doctest: +SKIP
Timestamp('2020-11-16 22:50:18.092888+0000', tz='UTC')
```

Static methods defined here:

```
__new__(
    cls,
    ts_input=<object object at 0x000001A9511B16D0>,
    year=None,
    month=None,
    day=None,
    hour=None,
    minute=None,
    second=None,
    microsecond=None,
    tzinfo=None,
    *,
    nanosecond=None,
    tz=<object object at 0x000001A9511B16D0>,
    unit=None,
    fold=None
)
```

Readonly properties defined here:

tzinfo

Returns the timezone info of the Timestamp.

This property returns a `datetime.tzinfo` object if the Timestamp is timezone-aware. If the Timestamp has no timezone, it returns `None`. If the Timestamp is in UTC or a fixed-offset timezone, it returns `datetime.timezone`. If the Timestamp uses an IANA timezone (e.g., "America/New_York"), it returns `zoneinfo.ZoneInfo`.

See Also

Timestamp.tz : Alias for `tzinfo`, may return a `zoneinfo.ZoneInfo` object.
Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.
Timestamp.tz_localize : Localize the Timestamp to a specific timezone.

Examples

```
>>> ts = pd.Timestamp("2023-01-01 12:00:00", tz="UTC")
>>> ts.tzinfo
datetime.timezone.utc
```

```
>>> ts_naive = pd.Timestamp("2023-01-01 12:00:00")
>>> ts_naive.tzinfo
```

Data descriptors defined here:

`__dict__`
dictionary for instance variables

`__weakref__`
list of weak references to the object

`daysinmonth`
Return the number of days in the month.

Returns

int

See Also

`Timestamp.month_name` : Return the month name of the Timestamp with specified locale.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.days_in_month
31

`tz`
Alias for `tzinfo`.

The `'tz'` property provides a simple and direct way to retrieve the timezone information of a `'Timestamp'` object. It is particularly useful when working with time series data that includes timezone information, allowing for easy access and manipulation of the timezone context.

See Also

`Timestamp.tzinfo` : Returns the timezone information of the Timestamp.
`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.
`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

Examples

>>> ts = pd.Timestamp(1584226800, unit='s', tz='Europe/Stockholm')
>>> ts.tz
zoneinfo.ZoneInfo(key='Europe/Stockholm')

`weekofyear`
Return the week number of the year.

Returns

int

See Also

`Timestamp.weekday` : Return the day of the week.
`Timestamp.quarter` : Return the quarter of the year.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.week
11

Methods inherited from `pandas._libs.tslibs.timestamps.Timestamp`:

`__add__(self, object, /)`
Return self+value.

`__eq__(self, value, /)`
Return self==value.

`__ge__(self, value, /)`
Return self>=value.

`__gt__(self, value, /)`

```

        Return self>value.

__hash__(self, /)
    Return hash(self).

__le__(self, value, /)
    Return self<=value.

__lt__(self, value, /)
    Return self<value.

__ne__(self, value, /)
    Return self!=value.

__radd__(self, object, /)
    Return value+self.

__reduce__(self)

__reduce_ex__(self, protocol)

__repr__(self, /)
    Return repr(self).

__rsub__(self, object, /)
    Return value-self.

__setstate__(self, state)

__sub__(self, object, /)
    Return self-value.

as_unit(self, unit, round_ok=True)
    Convert the underlying int64 representation to the given unit.

    Parameters
    -----
    unit : {"ns", "us", "ms", "s"}
    round_ok : bool, default True
        If False and the conversion requires rounding, raise.

    Returns
    -----
    Timestamp

    See Also
    -----
    Timestamp.asm8 : Return numpy datetime64 format with same precision.
    Timestamp.to_pydatetime : Convert Timestamp object to a native
        Python datetime object.
    to_timedelta : Convert argument into timedelta object,
        which can represent differences in times.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 00:00:00.01')
    >>> ts
    Timestamp('2023-01-01 00:00:00.010000')
    >>> ts.unit
    'ms'
    >>> ts = ts.as_unit('s')
    >>> ts
    Timestamp('2023-01-01 00:00:00')
    >>> ts.unit
    's'

day_name(self, locale=None) -> str
    Return the day name of the Timestamp with specified locale.

    Parameters
    -----
    locale : str, default None (English locale)
        Locale determining the language in which to return the day name.

    Returns
    -----
    str

```

See Also

Timestamp.day_of_week : Return day of the week.
Timestamp.day_of_year : Return day of the year.

Examples

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.day_name()
'Saturday'

Analogous for ``pd.NaT``:

>>> pd.NaT.day_name()
nan

isoformat(self, sep: str = 'T', timespec: str = 'auto') -> str
Return the time formatted according to ISO 8601.

The full format looks like 'YYYY-MM-DD HH:MM:SS.mmmmmnnn'.
By default, the fractional part is omitted if self.microsecond == 0
and self._nanosecond == 0.

If self.tzinfo is not None, the UTC offset is also attached,
giving a full format of 'YYYY-MM-DD HH:MM:SS.mmmmmnnn+HH:MM'.

Parameters

sep : str, default 'T'
String used as the separator between the date and time.

timespec : str, default 'auto'
Specifies the number of additional terms of the time to include.
The valid values are 'auto', 'hours', 'minutes', 'seconds',
'milliseconds', 'microseconds', and 'nanoseconds'.

Returns

str

See Also

Timestamp.strftime : Return a formatted string.
Timestamp.isocalendar : Return a tuple containing ISO year, week number and
weekday.

Examples

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.isoformat()
'2020-03-14T15:32:52.192548651'
>>> ts.isoformat(timespec='microseconds')
'2020-03-14T15:32:52.192548'

month_name(self, locale=None) -> str
Return the month name of the Timestamp with specified locale.

This method returns the full name of the month corresponding to the
`Timestamp`, such as 'January', 'February', etc. The month name can
be returned in a specified locale if provided; otherwise, it defaults
to the English locale.

Parameters

locale : str, default None (English locale)
Locale determining the language in which to return the month name.

Returns

str
The full month name as a string.

See Also

Timestamp.day_name : Returns the name of the day of the week.
Timestamp.strftime : Returns a formatted string of the Timestamp.

`datetime.datetime.strftime` : Returns a string representing the date and time.

Examples

Get the month name in English (default):

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.month_name()
'March'
```

Analogous for `pd.NaT`:

```
>>> pd.NaT.month_name()
nan
```

`normalize(self)` -> 'Timestamp'

Normalize Timestamp to midnight, preserving tz information.

This method sets the time component of the `'Timestamp'` to midnight (00:00:00), while preserving the date and time zone information. It is useful when you need to standardize the time across different `'Timestamp'` objects without altering the time zone or the date.

Returns

Timestamp

See Also

`Timestamp.floor` : Rounds `'Timestamp'` down to the nearest frequency.
`Timestamp.ceil` : Rounds `'Timestamp'` up to the nearest frequency.
`Timestamp.round` : Rounds `'Timestamp'` to the nearest frequency.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14, 15, 30)
>>> ts.normalize()
Timestamp('2020-03-14 00:00:00')
```

`timestamp(self)`

Return POSIX timestamp as float.

This method converts the `'Timestamp'` object to a POSIX timestamp, which is the number of seconds since the Unix epoch (January 1, 1970). The returned value is a floating-point number, where the integer part represents the seconds, and the fractional part represents the microseconds.

Returns

float
The POSIX timestamp representation of the `'Timestamp'` object.

See Also

`Timestamp.fromtimestamp` : Construct a `'Timestamp'` from a POSIX timestamp.
`datetime.datetime.timestamp` : Equivalent method from the `'datetime'` module.
`Timestamp.to_pydatetime` : Convert the `'Timestamp'` to a `'datetime'` object.
`Timestamp.to_datetime64` : Converts `'Timestamp'` to `'numpy.datetime64'`.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
>>> ts.timestamp()
1584199972.192548
```

`to_datetime64(self)`

Return a NumPy datetime64 object with same precision.

This method returns a `numpy.datetime64` object with the same date and time information and precision as the `pd.Timestamp` object.

See Also

`numpy.datetime64` : Class to represent dates and times with high precision.
`Timestamp.to_numpy` : Alias for this method.
`Timestamp.asm8` : Alias for this method.
`pd.to_datetime` : Convert argument to datetime.

Examples

```
-----
>>> ts = pd.Timestamp(year=2023, month=1, day=1,
...                     hour=10, second=15)
>>> ts
Timestamp('2023-01-01 10:00:15')
>>> ts.to_datetime64()
numpy.datetime64('2023-01-01T10:00:15.000000')
```

`to_numpy(self, dtype=None, copy=False) -> np.datetime64`
Convert the Timestamp to a NumPy datetime64.

This is an alias method for ``Timestamp.to_datetime64()``. The `dtype` and `copy` parameters are available here only for compatibility. Their values will not affect the return value.

Parameters

```
-----
dtype : dtype, optional
    Data type of the output, ignored in this method as the return type
    is always `numpy.datetime64`.
copy : bool, default False
    Whether to ensure that the returned value is a new object. This
    parameter is also ignored as the method does not support copying.
```

Returns

```
-----
numpy.datetime64
```

See Also

```
-----
DatetimeIndex.to_numpy : Similar method for DatetimeIndex.
```

Examples

```
-----
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.to_numpy()
numpy.datetime64('2020-03-14T15:32:52.192548651')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.to_numpy()
numpy.datetime64('NaT')
```

`to_period(self, freq=None)`
Return a period of which this timestamp is an observation.

This method converts the given Timestamp to a Period object, which represents a span of time, such as a year, month, etc., based on the specified frequency.

Parameters

```
-----
freq : str, optional
    Frequency string for the period (e.g., 'Y', 'M', 'W'). Defaults to `None`.
```

See Also

```
-----
Timestamp : Represents a specific timestamp.
Period : Represents a span of time.
to_period : Converts an object to a Period.
```

Examples

```
-----
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> # Year end frequency
>>> ts.to_period(freq='Y')
Period('2020', 'Y-DEC')
```

```
>>> # Month end frequency
>>> ts.to_period(freq='M')
Period('2020-03', 'M')
```

```
>>> # Weekly frequency
>>> ts.to_period(freq='W')
Period('2020-03-09/2020-03-15', 'W-SUN')
```

```

>>> # Quarter end frequency
>>> ts.to_period(freq='Q')
Period('2020Q1', 'Q-DEC')

```

to_pydatetime(self, warn=True)
Convert a Timestamp object to a native Python datetime object.

This method is useful for when you need to utilize a pandas Timestamp object in contexts where native Python datetime objects are expected or required. The conversion discards the nanoseconds component, and a warning can be issued in such cases if desired.

Parameters

warn : bool, default True
If True, issues a warning when the timestamp includes nonzero nanoseconds, as these will be discarded during the conversion.

Returns

datetime.datetime or NaT
Returns a datetime.datetime object representing the timestamp, with year, month, day, hour, minute, second, and microsecond components. If the timestamp is NaT (Not a Time), returns NaT.

See Also

datetime.datetime : The standard Python datetime class that this method returns.
Timestamp.timestamp : Convert a Timestamp object to POSIX timestamp.
Timestamp.to_datetime64 : Convert a Timestamp object to numpy.datetime64.

Examples

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
>>> ts.to_pydatetime()
datetime.datetime(2020, 3, 14, 15, 32, 52, 192548)

Analogous for ``pd.NaT``:

```

>>> pd.NaT.to_pydatetime()
NaT

```

Data descriptors inherited from pandas._libs.tslibs.timestamps.Timestamp:

asm8
Return numpy datetime64 format with same precision.

See Also

numpy.datetime64 : Numpy datatype for dates and times with high precision.
Timestamp.to_numpy : Convert the Timestamp to a NumPy datetime64.
to_datetime : Convert argument to datetime.

Examples

>>> ts = pd.Timestamp(2020, 3, 14, 15)
>>> ts.asm8
numpy.datetime64('2020-03-14T15:00:00.000000')

day
Return the day of the Timestamp.

Returns

int
The day of the Timestamp.

See Also

Timestamp.week : Return the week number of the year.
Timestamp.weekday : Return the day of the week.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.day
31
```

day_of_week

Return day of the week.

Returns

int

See Also

Timestamp.isoweekday : Return the ISO day of the week represented by the date.

Timestamp.weekday : Return the day of the week represented by the date.

Timestamp.day_of_year : Return day of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_week
```

```
5
```

day_of_year

Return the day of the year.

Returns

int

See Also

Timestamp.day_of_week : Return day of the week.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_year
```

```
74
```

dayofweek

Return day of the week.

Returns

int

See Also

Timestamp.isoweekday : Return the ISO day of the week represented by the date.

Timestamp.weekday : Return the day of the week represented by the date.

Timestamp.day_of_year : Return day of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_week
```

```
5
```

dayofyear

Return the day of the year.

Returns

int

See Also

Timestamp.day_of_week : Return day of the week.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_year
```

```
74
```

days_in_month

Return the number of days in the month.

Returns

int

See Also

Timestamp.month_name : Return the month name of the Timestamp with specified locale.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.days_in_month
31
```

fold

Return the fold value of the Timestamp.

Returns

int

The fold value of the Timestamp, where 0 indicates the first occurrence of the ambiguous time, and 1 indicates the second.

See Also

Timestamp.dst : Return the daylight saving time (DST) adjustment.

Timestamp.tzinfo : Return the timezone information associated.

Examples

```
>>> ts = pd.Timestamp("2024-11-03 01:30:00")
>>> ts.fold
0
```

hour

Return the hour of the Timestamp.

Returns

int

The hour of the Timestamp.

See Also

Timestamp.minute : Return the minute of the Timestamp.

Timestamp.second : Return the second of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.hour
16
```

is_leap_year

Return True if year is a leap year.

A leap year is a year, which has 366 days (instead of 365) including 29th of February as an intercalary day. Leap years are years which are multiples of four with the exception of years divisible by 100 but not by 400.

Returns

bool

True if year is a leap year, else False

See Also

Period.is_leap_year : Return True if the period's year is in a leap year.

DatetimeIndex.is_leap_year : Boolean indicator if the date belongs to a leap year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```

>>> ts.is_leap_year
True

is_month_end
Check if the date is the last day of the month.

Returns
-----
bool
    True if the date is the last day of the month.

See Also
-----
Timestamp.is_month_start : Similar property indicating month start.

Examples
-----
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_month_end
False

>>> ts = pd.Timestamp(2020, 12, 31)
>>> ts.is_month_end
True

is_month_start
Check if the date is the first day of the month.

Returns
-----
bool
    True if the date is the first day of the month.

See Also
-----
Timestamp.is_month_end : Similar property indicating the last day of the month.

Examples
-----
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_month_start
False

>>> ts = pd.Timestamp(2020, 1, 1)
>>> ts.is_month_start
True

is_quarter_end
Check if date is last day of the quarter.

Returns
-----
bool
    True if date is last day of the quarter.

See Also
-----
Timestamp.is_quarter_start : Similar property indicating the quarter start.
Timestamp.quarter : Return the quarter of the date.

Examples
-----
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_quarter_end
False

>>> ts = pd.Timestamp(2020, 3, 31)
>>> ts.is_quarter_end
True

is_quarter_start
Check if the date is the first day of the quarter.

Returns
-----
bool
    True if date is first day of the quarter.

```

See Also

Timestamp.is_quarter_end : Similar property indicating the quarter end.
Timestamp.quarter : Return the quarter of the date.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_quarter_start
False

>>> ts = pd.Timestamp(2020, 4, 1)
>>> ts.is_quarter_start
True

is_year_end

Return True if date is last day of the year.

Returns

bool

See Also

Timestamp.is_year_start : Similar property indicating the start of the year.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_year_end
False

>>> ts = pd.Timestamp(2020, 12, 31)
>>> ts.is_year_end
True

is_year_start

Return True if date is first day of the year.

Returns

bool

See Also

Timestamp.is_year_end : Similar property indicating the end of the year.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_year_start
False

>>> ts = pd.Timestamp(2020, 1, 1)
>>> ts.is_year_start
True

microsecond

Return the microsecond of the Timestamp.

Returns

int

The microsecond of the Timestamp.

See Also

Timestamp.second : Return the second of the Timestamp.
Timestamp.minute : Return the minute of the Timestamp.

Examples

>>> ts = pd.Timestamp("2024-08-31 16:16:30.2304")
>>> ts.microsecond
230400

minute

Return the minute of the Timestamp.

Returns

int

The minute of the Timestamp.

See Also

Timestamp.hour : Return the hour of the Timestamp.

Timestamp.second : Return the second of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.minute
```

```
16
```

month

Return the month of the Timestamp.

Returns

int

The month of the Timestamp.

See Also

Timestamp.day : Return the day of the Timestamp.

Timestamp.year : Return the year of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.month
```

```
8
```

nanosecond

Return the nanosecond of the Timestamp.

Returns

int

The nanosecond of the Timestamp.

See Also

Timestamp.second : Return the second of the Timestamp.

Timestamp.microsecond : Return the microsecond of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30.230400015")
```

```
>>> ts.nanosecond
```

```
15
```

quarter

Return the quarter of the year for the `Timestamp`.

This property returns an integer representing the quarter of the year in which the `Timestamp` falls. The quarters are defined as follows:

- Q1: January 1 to March 31
- Q2: April 1 to June 30
- Q3: July 1 to September 30
- Q4: October 1 to December 31

Returns

int

The quarter of the year (1 through 4).

See Also

Timestamp.month : Returns the month of the `Timestamp`.

Timestamp.year : Returns the year of the `Timestamp`.

Examples

Get the quarter for a `Timestamp`:

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.quarter
1
```

For a `Timestamp` in the fourth quarter:

```
>>> ts = pd.Timestamp(2020, 10, 14)
>>> ts.quarter
4
```

second

Return the second of the Timestamp.

Returns

int

The second of the Timestamp.

See Also

`Timestamp.microsecond` : Return the microsecond of the Timestamp.
`Timestamp.minute` : Return the minute of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.second
30
```

unit

The abbreviation associated with `self._creso`.

This property returns a string representing the time unit of the Timestamp's resolution. It corresponds to the smallest time unit that can be represented by this Timestamp object. The possible values are:

- 's' (second)
- 'ms' (millisecond)
- 'us' (microsecond)
- 'ns' (nanosecond)

Returns

str

A string abbreviation of the Timestamp's resolution unit:

- 's' for second
- 'ms' for millisecond
- 'us' for microsecond
- 'ns' for nanosecond

See Also

`Timestamp.resolution` : Return resolution of the Timestamp.
`Timedelta` : A duration expressing the difference between two dates or times.

Examples

```
>>> pd.Timestamp("2020-01-01 12:34:56").unit
's'

>>> pd.Timestamp("2020-01-01 12:34:56.123").unit
'ms'

>>> pd.Timestamp("2020-01-01 12:34:56.123456").unit
'us'

>>> pd.Timestamp("2020-01-01 12:34:56.123456789").unit
'ns'
```

value

Return the value of the Timestamp.

Returns

`int`
The integer representation of the Timestamp object in nanoseconds since the Unix epoch (1970-01-01 00:00:00 UTC).

See Also

`Timestamp.second` : Return the second of the Timestamp.
`Timestamp.minute` : Return the minute of the Timestamp.

Examples

>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.value
1725120990000000000

`week`
Return the week number of the year.

Returns

`int`

See Also

`Timestamp.weekday` : Return the day of the week.
`Timestamp.quarter` : Return the quarter of the year.

Examples

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.week
11

`year`
Return the year of the Timestamp.

Returns

`int`
The year of the Timestamp.

See Also

`Timestamp.month` : Return the month of the Timestamp.
`Timestamp.day` : Return the day of the Timestamp.

Examples

>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.year
2024

Data and other attributes inherited from `pandas._libs.tslibs.timestamps._Timestamp`:

`__array_priority__` = 100

`__pyx_vtable__` = <capsule object NULL>

`max` = `Timestamp('2262-04-11 23:47:16.854775807')`
Returns the maximum bound possible for Timestamp.

This property provides access to the largest possible value that can be represented by a Timestamp object.

Returns

`Timestamp`

See Also

`Timestamp.min`: Returns the minimum bound possible for Timestamp.
`Timestamp.resolution`: Returns the smallest possible difference between non-equal Timestamp objects.

Examples

```

>>> pd.Timestamp.max
Timestamp('2262-04-11 23:47:16.854775807')

min = Timestamp('1677-09-21 00:12:43.145224193')
Returns the minimum bound possible for Timestamp.

This property provides access to the smallest possible value that
can be represented by a Timestamp object.

Returns
-----
Timestamp

See Also
-----
Timestamp.max: Returns the maximum bound possible for Timestamp.
Timestamp.resolution: Returns the smallest possible difference between
non-equal Timestamp objects.

Examples
-----
>>> pd.Timestamp.min
Timestamp('1677-09-21 00:12:43.145224193')

resolution = Timedelta('0 days 00:00:00.000000001')
Returns the smallest possible difference between non-equal Timestamp objects.

The resolution value is determined by the underlying representation of time
units and is equivalent to Timedelta(nanoseconds=1).

Returns
-----
Timedelta

See Also
-----
Timestamp.max: Returns the maximum bound possible for Timestamp.
Timestamp.min: Returns the minimum bound possible for Timestamp.

Examples
-----
>>> pd.Timestamp.resolution
Timedelta('0 days 00:00:00.000000001')

```

```

Methods inherited from pandas._libs.tslibs.base.ABCTimestamp:
__reduce_cython__(self)
__setstate_cython__(self, __pyx_state)

```

```

Methods inherited from datetime.datetime:
__replace__(self, /, **changes)
    The same as replace().
__str__(self, /)
    Return str(self).

```

```

Class methods inherited from datetime.datetime:
fromisoformat(object, /)
    string -> datetime from a string in most ISO 8601 formats

```

```

Methods inherited from datetime.date:
__format__(...)
    Formats self with strftime.

```

```

Class methods inherited from datetime.date:

```



```

fromisocalendar(...)
    int, int, int -> Construct a date from the ISO year, week number and weekday.

    This is the inverse of the date.isocalendar() function

```

```

class UInt16Dtype(pandas.core.arrays.integer.IntegerDtype)
    An ExtensionDtype for uint16 integer data.

    Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

    Attributes
    -----
    None

    Methods
    -----
    None

    See Also
    -----
    Int8Dtype : 8-bit nullable integer type.
    Int16Dtype : 16-bit nullable integer type.
    Int32Dtype : 32-bit nullable integer type.
    Int64Dtype : 64-bit nullable integer type.

    Examples
    -----
    For Int8Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
    >>> ser.dtype
    Int8Dtype()

    For Int16Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
    >>> ser.dtype
    Int16Dtype()

    For Int32Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
    >>> ser.dtype
    Int32Dtype()

    For Int64Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
    >>> ser.dtype
    Int64Dtype()

    For UInt8Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
    >>> ser.dtype
    UInt8Dtype()

    For UInt16Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
    >>> ser.dtype
    UInt16Dtype()

    For UInt32Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
    >>> ser.dtype
    UInt32Dtype()

    For UInt64Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
    >>> ser.dtype
    UInt64Dtype()

    Method resolution order:

```

```
UInt16Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'UInt16'
```

```
type = <class 'numpy.uint16'>
      Unsigned integer type, compatible with C ``unsigned short``.
```

```
      :Character code: ``'H'``
```

```
      :Canonical name: `numpy.ushort`
```

```
      :Alias on this platform (win32 AMD64): `numpy.uint16`: 16-bit unsigned integer (``0`` to
```

```
-----
Methods inherited from pandas.core.arrays.integer.IntegerDtype:
```

```
construct_array_type(self) -> type[IntegerArray]
      Return the array type associated with this dtype.
```

```
      Returns
```

```
      -----
      type
```

```
-----
Methods inherited from pandas.core.arrays.numeric.NumericDtype:
```

```
__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
      Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.
```

```
__repr__(self) -> str
      Return repr(self).
```

```
-----
Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:
```

```
is_signed_integer
```

```
is_unsigned_integer
```

```
-----
Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
```

```
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
      Construct the MaskedDtype corresponding to the given numpy dtype.
```

```
-----
Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
```

```
na_value
      Default NA value to use for this type.
```

```
      This is used in e.g. ExtensionArray.take. This should be the
      user-facing "boxed" version of the NA value, not the physical NA value
      for storage. e.g. for JSONArray, this is an empty dictionary.
```

```
-----
Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
```

```
itemsizesize
      Return the number of bytes in this dtype
```

```
kind
```

```
numpy_dtype
      Return an instance of our numpy dtype
```

```
-----
Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:
```

```

base = None

-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

    Parameters
    -----
    string : str
        The name of the type, for example ``category``.

    Returns
    -----
    ExtensionDtype
        Instance of the dtype.

    Raises
    -----
    TypeError
        If a class cannot be constructed from this 'string'.

    Examples
    -----
    For extension dtypes with arguments the following may be an
    adequate implementation.

```

```

>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )

```

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

Parameters

dtype : object
The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names

Ordered list of field names, or None if there are no fields.

This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__

dictionary for instance variables

__weakref__

list of weak references to the object

index_class

The Index subclass to return from Index.__new__ when this dtype is encountered.

class UInt32Dtype(pandas.core.arrays.integer.IntegerDtype)

An ExtensionDtype for uint32 integer data.

Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

Attributes

None

Methods

None

See Also

Int8Dtype : 8-bit nullable integer type.
Int16Dtype : 16-bit nullable integer type.
Int32Dtype : 32-bit nullable integer type.
Int64Dtype : 64-bit nullable integer type.

Examples

For Int8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
>>> ser.dtype
Int8Dtype()
```

For Int16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
>>> ser.dtype
Int16Dtype()
```

For Int32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
>>> ser.dtype
Int32Dtype()
```

For Int64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
>>> ser.dtype
Int64Dtype()
```

For UInt8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
>>> ser.dtype
UInt8Dtype()
```

For UInt16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
>>> ser.dtype
UInt16Dtype()
```

For UInt32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
>>> ser.dtype
UInt32Dtype()
```

For UInt64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
>>> ser.dtype
UInt64Dtype()
```

Method resolution order:

```
UInt32Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'UInt32'
```

```
type = <class 'numpy.uint32'>
```

Unsigned integer type, compatible with C ``unsigned long``.

:Character code: ``'L'``

:Canonical name: ``numpy.ulong``

:Alias on this platform (win32 AMD64): ``numpy.uint32``: 32-bit unsigned integer (``0`` to

Methods inherited from pandas.core.arrays.integer.IntegerDtype:

```
construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.
```

Returns

type

Methods inherited from pandas.core.arrays.numeric.NumericDtype:

__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.

__repr__(self) -> str
Return repr(self).

Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

is_signed_integer

is_unsigned_integer

Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
Construct the MaskedDtype corresponding to the given numpy dtype.

Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

na_value
Default NA value to use for this type.

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

itemsize
Return the number of bytes in this dtype

kind

numpy_dtype
Return an instance of our numpy dtype

Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check whether 'other' is equal to self.

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes in ``self.\_metadata`` are equal between `self` and `other`.

Parameters

other : Any

Returns

bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype

```

        Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

Parameters
-----
shape : int or tuple[int]

Returns
-----
ExtensionArray
-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

Parameters
-----
string : str
    The name of the type, for example ``category``.

Returns
-----
ExtensionDtype
    Instance of the dtype.

Raises
-----
TypeError
    If a class cannot be constructed from this 'string'.

Examples
-----
For extension dtypes with arguments the following may be an
adequate implementation.

>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check if we match 'dtype'.

Parameters
-----
dtype : object
    The object to check.

Returns
-----
bool

Notes

```

```

-----
The default implementation is True if

1. ``cls.construct_from_string(dtype)`` is an instance
   of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above
   conditions is true for ``dtype.dtype``.

-----
Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names
    Ordered list of field names, or None if there are no fields.

    This is for compatibility with NumPy arrays, and may be removed in the
    future.

-----
Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

index_class
    The Index subclass to return from Index.__new__ when this dtype is
    encountered.

class UInt64Dtype(pandas.core.arrays.integer.IntegerDtype)
    An ExtensionDtype for uint64 integer data.

    Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.

    Attributes
    -----
    None

    Methods
    -----
    None

    See Also
    -----
    Int8Dtype : 8-bit nullable integer type.
    Int16Dtype : 16-bit nullable integer type.
    Int32Dtype : 32-bit nullable integer type.
    Int64Dtype : 64-bit nullable integer type.

    Examples
    -----
    For Int8Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
    >>> ser.dtype
    Int8Dtype()

    For Int16Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
    >>> ser.dtype
    Int16Dtype()

    For Int32Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
    >>> ser.dtype
    Int32Dtype()

    For Int64Dtype:

    >>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
    >>> ser.dtype
    Int64Dtype()

```


For UInt8Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
>>> ser.dtype
UInt8Dtype()
```

For UInt16Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
>>> ser.dtype
UInt16Dtype()
```

For UInt32Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
>>> ser.dtype
UInt32Dtype()
```

For UInt64Dtype:

```
>>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
>>> ser.dtype
UInt64Dtype()
```

Method resolution order:

```
UInt64Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'UInt64'
```

```
type = <class 'numpy.uint64'>
```

Unsigned signed integer type, 64bit on 64bit systems and 32bit on 32bit systems.

:Character code: ``'Q'``

:Canonical name: `numpy.uint`

:Alias on this platform (win32 AMD64): `numpy.uint64`: 64-bit unsigned integer (``0`` to

:Alias on this platform (win32 AMD64): `numpy.uintp`: Unsigned integer large enough to f

Methods inherited from pandas.core.arrays.integer.IntegerDtype:

```
construct_array_type(self) -> type[IntegerArray]
    Return the array type associated with this dtype.
```

Returns

type

Methods inherited from pandas.core.arrays.numeric.NumericDtype:

```
__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
    Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.
```

```
__repr__(self) -> str
    Return repr(self).
```

Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

is_signed_integer

is_unsigned_integer

Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
```

Construct the MaskedDtype corresponding to the given numpy dtype.

Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

na_value

Default NA value to use for this type.

This is used in e.g. ExtensionArray.take. This should be the user-facing "boxed" version of the NA value, not the physical NA value for storage. e.g. for JSONArray, this is an empty dictionary.

Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

itemsizes

Return the number of bytes in this dtype

kind

numpy_dtype

Return an instance of our numpy dtype

Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

base = None

Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check whether 'other' is equal to self.

By default, 'other' is considered equal if either

- * it's a string matching 'self.name'.
- * it's an instance of this type and all of the attributes in ``self.\_metadata`` are equal between `self` and `other`.

Parameters

other : Any

Returns

bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
Construct an ExtensionArray of this dtype with the given shape.

Analogous to numpy.empty.

Parameters

shape : int or tuple[int]

Returns

ExtensionArray

Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
Construct this type from a string.

This is useful mainly for data types that accept parameters.

For example, a period dtype accepts a frequency parameter that can be set as ``period[h]`` (where H means hourly frequency).

By default, in the abstract class, just the name of the type is expected. But subclasses can overwrite this method to accept parameters.

Parameters

string : str
 The name of the type, for example ``category``.

Returns

ExtensionDtype
 Instance of the dtype.

Raises

TypeError
 If a class cannot be constructed from this 'string'.

Examples

For extension dtypes with arguments the following may be an adequate implementation.

```
>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )
```

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

Parameters

dtype : object
 The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names

 Ordered list of field names, or None if there are no fields.

 This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__

 dictionary for instance variables

__weakref__

```

|         list of weak references to the object
|
| index_class
|     The Index subclass to return from Index.__new__ when this dtype is
|     encountered.
|
class UInt8Dtype(pandas.core.arrays.integer.IntegerDtype)
|     An ExtensionDtype for uint8 integer data.
|
|     Uses :attr:`pandas.NA` as its missing value, rather than :attr:`numpy.nan`.
|
| Attributes
| -----
| None
|
| Methods
| -----
| None
|
| See Also
| -----
| Int8Dtype : 8-bit nullable integer type.
| Int16Dtype : 16-bit nullable integer type.
| Int32Dtype : 32-bit nullable integer type.
| Int64Dtype : 64-bit nullable integer type.
|
| Examples
| -----
| For Int8Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.Int8Dtype())
| >>> ser.dtype
| Int8Dtype()
|
| For Int16Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.Int16Dtype())
| >>> ser.dtype
| Int16Dtype()
|
| For Int32Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.Int32Dtype())
| >>> ser.dtype
| Int32Dtype()
|
| For Int64Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.Int64Dtype())
| >>> ser.dtype
| Int64Dtype()
|
| For UInt8Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt8Dtype())
| >>> ser.dtype
| UInt8Dtype()
|
| For UInt16Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt16Dtype())
| >>> ser.dtype
| UInt16Dtype()
|
| For UInt32Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt32Dtype())
| >>> ser.dtype
| UInt32Dtype()
|
| For UInt64Dtype:
|
| >>> ser = pd.Series([2, pd.NA], dtype=pd.UInt64Dtype())
| >>> ser.dtype
| UInt64Dtype()
|
| Method resolution order:

```

```
UInt8Dtype
pandas.core.arrays.integer.IntegerDtype
pandas.core.arrays.numeric.NumericDtype
pandas.core.dtypes.dtypes.BaseMaskedDtype
pandas.api.extensions.ExtensionDtype
builtins.object
```

Data and other attributes defined here:

```
__annotations__ = {'name': 'ClassVar[str]'}
name = 'UInt8'
```

```
type = <class 'numpy.uint8'>
      Unsigned integer type, compatible with C ``unsigned char``.
```

```
      :Character code: ``'B'``
```

```
      :Canonical name: `numpy.ubyte`
```

```
      :Alias on this platform (win32 AMD64): `numpy.uint8`: 8-bit unsigned integer (``0`` to ``255``)
```

Methods inherited from pandas.core.arrays.integer.IntegerDtype:

```
construct_array_type(self) -> type[IntegerArray]
      Return the array type associated with this dtype.
```

```
      Returns
```

```
      -----
      type
```

Methods inherited from pandas.core.arrays.numeric.NumericDtype:

```
__from_arrow__(self, array: pyarrow.Array | pyarrow.ChunkedArray) -> BaseMaskedArray
      Construct IntegerArray/FloatingArray from pyarrow Array/ChunkedArray.
```

```
__repr__(self) -> str
      Return repr(self).
```

Data descriptors inherited from pandas.core.arrays.numeric.NumericDtype:

```
is_signed_integer
```

```
is_unsigned_integer
```

Class methods inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
from_numpy_dtype(dtype: np.dtype) -> BaseMaskedDtype
      Construct the MaskedDtype corresponding to the given numpy dtype.
```

Readonly properties inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
na_value
      Default NA value to use for this type.
```

```
      This is used in e.g. ExtensionArray.take. This should be the
      user-facing "boxed" version of the NA value, not the physical NA value
      for storage. e.g. for JSONArray, this is an empty dictionary.
```

Data descriptors inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```
itemsizesize
      Return the number of bytes in this dtype
```

```
kind
```

```
numpy_dtype
      Return an instance of our numpy dtype
```

Data and other attributes inherited from pandas.core.dtypes.dtypes.BaseMaskedDtype:

```

base = None

-----
Methods inherited from pandas.api.extensions.ExtensionDtype:

__eq__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Check whether 'other' is equal to self.

    By default, 'other' is considered equal if either

    * it's a string matching 'self.name'.
    * it's an instance of this type and all of the attributes
      in ``self._metadata`` are equal between `self` and `other`.

    Parameters
    -----
    other : Any

    Returns
    -----
    bool

__hash__(self) -> int from pandas.core.dtypes.base.ExtensionDtype
    Return hash(self).

__ne__(self, other: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
    Return self!=value.

__str__(self) -> str from pandas.core.dtypes.base.ExtensionDtype
    Return str(self).

empty(self, shape: Shape) -> ExtensionArray from pandas.core.dtypes.base.ExtensionDtype
    Construct an ExtensionArray of this dtype with the given shape.

    Analogous to numpy.empty.

    Parameters
    -----
    shape : int or tuple[int]

    Returns
    -----
    ExtensionArray

-----
Class methods inherited from pandas.api.extensions.ExtensionDtype:

construct_from_string(string: str) -> Self from pandas.core.dtypes.base.ExtensionDtype
    Construct this type from a string.

    This is useful mainly for data types that accept parameters.
    For example, a period dtype accepts a frequency parameter that
    can be set as ``period[h]`` (where H means hourly frequency).

    By default, in the abstract class, just the name of the type is
    expected. But subclasses can overwrite this method to accept
    parameters.

    Parameters
    -----
    string : str
        The name of the type, for example ``category``.

    Returns
    -----
    ExtensionDtype
        Instance of the dtype.

    Raises
    -----
    TypeError
        If a class cannot be constructed from this 'string'.

    Examples
    -----
    For extension dtypes with arguments the following may be an
    adequate implementation.

```

```

>>> import re
>>> @classmethod
... def construct_from_string(cls, string):
...     pattern = re.compile(r"^my_type\[ (?P<arg_name>.+)\]$")
...     match = pattern.match(string)
...     if match:
...         return cls(**match.groupdict())
...     else:
...         raise TypeError(
...             f"Cannot construct a '{cls.__name__}' from '{string}'"
...         )

```

is_dtype(dtype: object) -> bool from pandas.core.dtypes.base.ExtensionDtype
Check if we match 'dtype'.

Parameters

dtype : object
 The object to check.

Returns

bool

Notes

The default implementation is True if

1. ``cls.construct\_from\_string(dtype)`` is an instance of ``cls``.
2. ``dtype`` is an object and is an instance of ``cls``
3. ``dtype`` has a ``dtype`` attribute, and any of the above conditions is true for ``dtype.dtype``.

Readonly properties inherited from pandas.api.extensions.ExtensionDtype:

names
 Ordered list of field names, or None if there are no fields.

 This is for compatibility with NumPy arrays, and may be removed in the future.

Data descriptors inherited from pandas.api.extensions.ExtensionDtype:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

index_class
 The Index subclass to return from Index.__new__ when this dtype is encountered.

FUNCTIONS

```

array(
    data: Sequence[object] | AnyArrayLike,
    dtype: Dtype | None = None,
    copy: bool = True
) -> ExtensionArray
Create an array.

```

This method constructs an array using pandas extension types when possible. If `dtype` is specified, it determines the type of array returned. Otherwise, pandas attempts to infer the appropriate dtype based on `data`.

Parameters

data : Sequence of objects
 The scalars inside `data` should be instances of the scalar type for `dtype`. It's expected that `data` represents a 1-dimensional array of data.

 When `data` is an Index or Series, the underlying array

will be extracted from ``data``.

`dtype` : str, np.dtype, or ExtensionDtype, optional
The dtype to use for the array. This may be a NumPy dtype or an extension type registered with pandas using `:meth:`pandas.api.extensions.register_extension_dtype``.

If not specified, there are two possibilities:

1. When ``data`` is a `:class:`Series``, `:class:`Index``, or `:class:`ExtensionArray``, the ``dtype`` will be taken from the data.
2. Otherwise, pandas will attempt to infer the ``dtype`` from the data.

Note that when ``data`` is a NumPy array, ``data.dtype`` is *not* used for inferring the array type. This is because NumPy cannot represent all the types of data that can be held in extension arrays.

Currently, pandas will infer an extension dtype for sequences of

Scalar Type	Array Type
<code>:class:`pandas.Interval`</code>	<code>:class:`pandas.arrays.IntervalArray`</code>
<code>:class:`pandas.Period`</code>	<code>:class:`pandas.arrays.PeriodArray`</code>
<code>:class:`datetime.datetime`</code>	<code>:class:`pandas.arrays.DatetimeArray`</code>
<code>:class:`datetime.timedelta`</code>	<code>:class:`pandas.arrays.TimedeltaArray`</code>
<code>:class:`int`</code>	<code>:class:`pandas.arrays.IntegerArray`</code>
<code>:class:`float`</code>	<code>:class:`pandas.arrays.FloatingArray`</code>
<code>:class:`str`</code>	<code>:class:`pandas.arrays.StringArray`</code> or <code>:class:`pandas.arrays.ArrowStringArray`</code>
<code>:class:`bool`</code>	<code>:class:`pandas.arrays.BooleanArray`</code>

The `ExtensionArray` created when the scalar type is `:class:`str`` is determined by ``pd.options.mode.string_storage`` if the dtype is not explicitly given.

For all other cases, NumPy's usual inference rules will be used.

`copy` : bool, default True
Whether to copy the data, even if not necessary. Depending on the type of ``data``, creating the new array may require copying data, even if ``copy=False``.

Returns

`ExtensionArray`
The newly created array.

Raises

`ValueError`
When ``data`` is not 1-dimensional.

See Also

`numpy.array` : Construct a NumPy array.
`Series` : Construct a pandas Series.
`Index` : Construct a pandas Index.
`arrays.NumpyExtensionArray` : `ExtensionArray` wrapping a NumPy array.
`Series.array` : Extract the array stored within a Series.

Notes

Omitting the ``dtype`` argument means pandas will attempt to infer the best array type from the values in the data. As new array types are added by pandas and 3rd party libraries, the "best" array type may change. We recommend specifying ``dtype`` to ensure that

1. the correct array type for the data is returned
2. the returned array type doesn't change as new extension types are added by pandas and third-party libraries

Additionally, if the underlying memory representation of the returned array matters, we recommend specifying the ``dtype`` as a concrete object rather than a string alias or allowing it to be inferred. For example,

a future version of pandas or a 3rd-party library may include a dedicated ExtensionArray for string data. In this event, the following would no longer return a `:class:`arrays.NumpyExtensionArray`` backed by a NumPy array.

```
>>> pd.array(["a", "b"], dtype=str)
<ArrowStringArray>
['a', 'b']
Length: 2, dtype: str
```

This would instead return the new ExtensionArray dedicated for string data. If you really need the new array to be backed by a NumPy array, specify that in the dtype.

```
>>> pd.array(["a", "b"], dtype=np.dtype("<U1"))
<NumpyExtensionArray>
['a', 'b']
Length: 2, dtype: str32
```

Finally, Pandas has arrays that mostly overlap with NumPy

```
* :class:`arrays.DatetimeArray`
* :class:`arrays.TimedeltaArray`
```

When data with a `datetime64[ns]` or `timedelta64[ns]` dtype is passed, pandas will always return a `DatetimeArray` or `TimedeltaArray` rather than a `NumpyExtensionArray`. This is for symmetry with the case of timezone-aware data, which NumPy does not natively support.

```
>>> pd.array(["2015", "2016"], dtype="datetime64[ns]")
<DatetimeArray>
['2015-01-01 00:00:00', '2016-01-01 00:00:00']
Length: 2, dtype: datetime64[ns]
```

```
>>> pd.array(["1h", "2h"], dtype="timedelta64[ns]")
<TimedeltaArray>
['0 days 01:00:00', '0 days 02:00:00']
Length: 2, dtype: timedelta64[ns]
```

Examples

If a dtype is not specified, pandas will infer the best dtype from the values. See the description of `dtype` for the types pandas infers for.

```
>>> pd.array([1, 2])
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64
```

```
>>> pd.array([1, 2, np.nan])
<IntegerArray>
[1, 2, <NA>]
Length: 3, dtype: Int64
```

```
>>> pd.array([1.1, 2.2])
<FloatingArray>
[1.1, 2.2]
Length: 2, dtype: Float64
```

```
>>> pd.array(["a", None, "c"])
<ArrowStringArray>
['a', <NA>, 'c']
Length: 3, dtype: string
```

```
>>> with pd.option_context("string_storage", "python"):
...     arr = pd.array(["a", None, "c"])
>>> arr
<StringArray>
['a', <NA>, 'c']
Length: 3, dtype: string
```

```
>>> pd.array([pd.Period("2000", freq="D"), pd.Period("2000", freq="D")])
<PeriodArray>
['2000-01-01', '2000-01-01']
Length: 2, dtype: period[D]
```

You can use the string alias for `dtype`

```
>>> pd.array(["a", "b", "a"], dtype="category")
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

Or specify the actual dtype

```
>>> pd.array(
...     ["a", "b", "a"], dtype=pd.CategoricalDtype(["a", "b", "c"], ordered=True)
... )
['a', 'b', 'a']
Categories (3, str): ['a' < 'b' < 'c']
```

If pandas does not infer a dedicated extension type a
:class:`arrays.NumpyExtensionArray` is returned.

```
>>> pd.array([1 + 1j, 3 + 2j])
<NumpyExtensionArray>
[(1+1j), (3+2j)]
Length: 2, dtype: complex128
```

As mentioned in the "Notes" section, new extension types may be added in the future (by pandas or 3rd party libraries), causing the return value to no longer be a :class:`arrays.NumpyExtensionArray`. Specify the `dtype` as a NumPy dtype if you need to ensure there's no future change in behavior.

```
>>> pd.array([1, 2], dtype=np.dtype("int32"))
<NumpyExtensionArray>
[1, 2]
Length: 2, dtype: int32
```

`data` must be 1-dimensional. A ValueError is raised when the input has the wrong dimensionality.

```
>>> pd.array(1)
Traceback (most recent call last):
...
ValueError: Cannot pass scalar '1' to 'pandas.array'.
```

```
bdate_range(
    start=None,
    end=None,
    periods: int | None = None,
    freq: Frequency | dt.timedelta = 'B',
    tz=None,
    normalize: bool = True,
    name: Hashable | None = None,
    weekmask=None,
    holidays=None,
    inclusive: IntervalClosedType = 'both',
    **kwargs
) -> DatetimeIndex
Return a fixed frequency DatetimeIndex with business day as the default.
```

Parameters

```
-----
start : str or datetime-like, default None
    Left bound for generating dates.
end : str or datetime-like, default None
    Right bound for generating dates.
periods : int, default None
    Number of periods to generate.
freq : str, Timedelta, datetime.timedelta, or DateOffset, default 'B'
    Frequency strings can have multiples, e.g. '5h'. The default is
    business daily ('B').
tz : str or None
    Time zone name for returning localized DatetimeIndex, for example
    Asia/Beijing.
normalize : bool, default False
    Normalize start/end dates to midnight before generating date range.
name : Hashable, default None
    Name of the resulting DatetimeIndex.
weekmask : str or None, default None
    Weekmask of valid business days, passed to ``numpy.busdaycalendar``,
    only used when custom frequency strings are passed. The default
    value None is equivalent to 'Mon Tue Wed Thu Fri'.
holidays : list-like or None, default None
```

Dates to exclude from the set of valid business days, passed to ``numpy.busdaycalendar``, only used when custom frequency strings are passed.

`inclusive` : {"both", "neither", "left", "right"}, default "both"

Include boundaries; Whether to set each bound as closed or open.

`**kwargs`

For compatibility. Has no effect on the result.

Returns

DatetimeIndex

Fixed frequency DatetimeIndex.

See Also

`date_range` : Return a fixed frequency DatetimeIndex.

`period_range` : Return a fixed frequency PeriodIndex.

`timedelta_range` : Return a fixed frequency TimedeltaIndex.

Notes

Of the four parameters: ``start``, ``end``, ``periods``, and ``freq``, exactly three must be specified. Specifying ``freq`` is a requirement for ``bdate_range``. Use ``date_range`` if specifying ``freq`` is not desired.

To learn more about the frequency strings, please see [:ref: this link<timeseries.offset_aliases>](#).

Examples

Note how the two weekend days are skipped in the result.

```
>>> pd.bdate_range(start="1/1/2018", end="1/08/2018")
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04',
               '2018-01-05', '2018-01-08'],
              dtype='datetime64[us]', freq='B')
```

`col(col_name: Hashable) -> Expression`

Generate deferred object representing a column of a DataFrame.

Any place which accepts ``lambda df: df[col_name]``, such as `:meth:`DataFrame.assign`` or `:meth:`DataFrame.loc``, can also accept ``pd.col(col_name)``.

.. versionadded:: 3.0.0

Parameters

`col_name` : Hashable

Column name.

Returns

``pandas.api.typing.Expression``

A deferred object representing a column of a DataFrame.

See Also

`DataFrame.query` : Query columns of a dataframe using string expressions.

Examples

You can use ``col`` in ``assign``.

```
>>> df = pd.DataFrame({"name": ["beluga", "narwhal"], "speed": [100, 110]})
>>> df.assign(name_titlecase=pd.col("name").str.title())
   name  speed name_titlecase
0  beluga   100         Beluga
1  narwhal  110         Narwhal
```

You can also use it for filtering.

```
>>> df.loc[pd.col("speed") > 105]
   name  speed
1  narwhal   110
```

```
concat(
    objs: Iterable[Series | DataFrame] | Mapping[HashableT, Series | DataFrame],
    *,
    axis: Axis = 0,
    join: str = 'outer',
    ignore_index: bool = False,
    keys: Iterable[Hashable] | None = None,
    levels=None,
    names: list[HashableT] | None = None,
    verify_integrity: bool = False,
    sort: bool | lib.NoDefault = <no_default>,
    copy: bool | lib.NoDefault = <no_default>
) -> DataFrame | Series
Concatenate pandas objects along a particular axis.
```

Allows optional set logic along the other axes.

Can also add a layer of hierarchical indexing on the concatenation axis, which may be useful if the labels are the same (or overlapping) on the passed axis number.

Parameters

```
-----
objs : an iterable or mapping of Series or DataFrame objects
      If a mapping is passed, the keys will be used as the `keys`
      argument, unless it is passed, in which case the values will be
      selected (see below). Any None objects will be dropped silently unless
      they are all None in which case a ValueError will be raised.
axis : {0/'index', 1/'columns'}, default 0
      The axis to concatenate along.
join : {'inner', 'outer'}, default 'outer'
      How to handle indexes on other axis (or axes).
ignore_index : bool, default False
      If True, do not use the index values along the concatenation axis. The
      resulting axis will be labeled 0, ..., n - 1. This is useful if you are
      concatenating objects where the concatenation axis does not have
      meaningful indexing information. Note the index values on the other
      axes are still respected in the join.
keys : sequence, default None
      If multiple levels passed, should contain tuples. Construct
      hierarchical index using the passed keys as the outermost level.
levels : list of sequences, default None
      Specific levels (unique values) to use for constructing a
      MultiIndex. Otherwise they will be inferred from the keys.
names : list, default None
      Names for the levels in the resulting hierarchical index.
verify_integrity : bool, default False
      Check whether the new concatenated axis contains duplicates. This can
      be very expensive relative to the actual data concatenation.
sort : bool, default False
      Sort non-concatenation axis. One exception to this is when the
      non-concatenation axis is a DatetimeIndex and join='outer' and the axis is
      not already aligned. In that case, the non-concatenation axis is always
      sorted lexicographically.
copy : bool, default False
      This keyword is now ignored; changing its value will have no
      impact on the method.

.. deprecated:: 3.0.0

      This keyword is ignored and will be removed in pandas 4.0. Since
      pandas 3.0, this method always returns a new object using a lazy
      copy mechanism that defers copies until necessary
      (Copy-on-Write). See the `user guide on Copy-on-Write
      <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
      for more details.
```

Returns

```
-----
object, type of objs
      When concatenating all ``Series`` along the index (axis=0), a
      ``Series`` is returned. When ``objs`` contains at least one
      ``DataFrame``, a ``DataFrame`` is returned. When concatenating along
      the columns (axis=1), a ``DataFrame`` is returned.
```

See Also

```
-----
```

DataFrame.join : Join DataFrames using indexes.
DataFrame.merge : Merge DataFrames by indexes or columns.

Notes

The keys, levels, and names arguments are all optional.

A walkthrough of how this method fits in with other tools for combining pandas objects can be found [here](https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html)
<https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html>`__.

It is not recommended to build DataFrames by adding single rows in a for loop. Build a list of rows and make a DataFrame in a single concat.

Examples

Combine two ``Series``.

```
>>> s1 = pd.Series(["a", "b"])
>>> s2 = pd.Series(["c", "d"])
>>> pd.concat([s1, s2])
0    a
1    b
0    c
1    d
dtype: str
```

Clear the existing index and reset it in the result by setting the ``ignore\_index`` option to ``True``.

```
>>> pd.concat([s1, s2], ignore_index=True)
0    a
1    b
2    c
3    d
dtype: str
```

Add a hierarchical index at the outermost level of the data with the ``keys`` option.

```
>>> pd.concat([s1, s2], keys=["s1", "s2"])
s1 0    a
   1    b
s2 0    c
   1    d
dtype: str
```

Label the index keys you create with the ``names`` option.

```
>>> pd.concat([s1, s2], keys=["s1", "s2"], names=["Series name", "Row ID"])
Series name  Row ID
s1          0      a
           1      b
s2          0      c
           1      d
dtype: str
```

Combine two ``DataFrame`` objects with identical columns.

```
>>> df1 = pd.DataFrame([["a", 1], ["b", 2]], columns=["letter", "number"])
>>> df1
  letter  number
0      a        1
1      b        2
>>> df2 = pd.DataFrame([["c", 3], ["d", 4]], columns=["letter", "number"])
>>> df2
  letter  number
0      c        3
1      d        4
>>> pd.concat([df1, df2])
  letter  number
0      a        1
1      b        2
0      c        3
1      d        4
```

Combine ``DataFrame`` objects with overlapping columns

and return everything. Columns outside the intersection will be filled with ``NaN`` values.

```
>>> df3 = pd.DataFrame(
...     [{"c", 3, "cat"}, {"d", 4, "dog"}], columns=["letter", "number", "animal"]
... )
>>> df3
  letter  number animal
0      c        3    cat
1      d        4    dog
>>> pd.concat([df1, df3], sort=False)
  letter  number animal
0      a        1   NaN
1      b        2   NaN
0      c        3    cat
1      d        4    dog
```

Combine ``DataFrame`` objects with overlapping columns and return only those that are shared by passing ``inner`` to the ``join`` keyword argument.

```
>>> pd.concat([df1, df3], join="inner")
  letter  number
0      a        1
1      b        2
0      c        3
1      d        4
```

Combine ``DataFrame`` objects horizontally along the x axis by passing in ``axis=1``.

```
>>> df4 = pd.DataFrame(
...     [{"bird", "polly"}, {"monkey", "george"}], columns=["animal", "name"]
... )
>>> pd.concat([df1, df4], axis=1)
  letter  number  animal  name
0      a        1    bird  polly
1      b        2  monkey  george
```

Prevent the result from including duplicate index values with the ``verify\_integrity`` option.

```
>>> df5 = pd.DataFrame([1], index=["a"])
>>> df5
  0
a  1
>>> df6 = pd.DataFrame([2], index=["a"])
>>> df6
  0
a  2
>>> pd.concat([df5, df6], verify_integrity=True)
Traceback (most recent call last):
```

```
ValueError: Indexes have overlapping values: ['a']
```

Append a single row to the end of a ``DataFrame`` object.

```
>>> df7 = pd.DataFrame({"a": 1, "b": 2}, index=[0])
>>> df7
   a  b
0  1  2
>>> new_row = pd.Series({"a": 3, "b": 4})
>>> new_row
a    3
b    4
dtype: int64
>>> pd.concat([df7, new_row.to_frame().T], ignore_index=True)
   a  b
0  1  2
1  3  4
```

```
crosstab(
    index,
    columns,
    values=None,
    rownames=None,
    colnames=None,
```

```

aggfunc=None,
margins: bool = False,
margins_name: Hashable = 'All',
dropna: bool = True,
normalize: bool | Literal[0, 1, 'all', 'index', 'columns'] = False
) -> DataFrame
    Compute a simple cross tabulation of two (or more) factors.

```

By default, computes a frequency table of the factors unless an array of values and an aggregation function are passed.

Parameters

```

-----
index : array-like, Series, or list of arrays/Series
    Values to group by in the rows.
columns : array-like, Series, or list of arrays/Series
    Values to group by in the columns.
values : array-like, optional
    Array of values to aggregate according to the factors.
    Requires `aggfunc` be specified.
rownames : sequence, default None
    If passed, must match number of row arrays passed.
colnames : sequence, default None
    If passed, must match number of column arrays passed.
aggfunc : function, optional
    If specified, requires `values` be specified as well.
margins : bool, default False
    Add row/column margins (subtotals).
margins_name : str, default 'All'
    Name of the row/column that will contain the totals
    when margins is True.
dropna : bool, default True
    Do not include columns whose entries are all NaN.
normalize : bool, {'all', 'index', 'columns'}, or {0,1}, default False
    Normalize by dividing all values by the sum of values.

    - If passed 'all' or `True`, will normalize over all values.
    - If passed 'index' will normalize over each row.
    - If passed 'columns' will normalize over each column.
    - If margins is `True`, will also normalize margin values.

```

Returns

```

-----
DataFrame
    Cross tabulation of the data.

```

See Also

```

-----
DataFrame.pivot : Reshape data based on column values.
pivot_table : Create a pivot table as a DataFrame.

```

Notes

```

-----
Any Series passed will have their name attributes used unless row or column
names for the cross-tabulation are specified.

```

Any input passed containing Categorical data will have **all** of its categories included in the cross-tabulation, even if the actual data does not contain any instances of a particular category.

In the event that there aren't overlapping indexes an empty DataFrame will be returned.

Reference :ref:`the user guide <reshaping.crosstabulations>` for more examples.

Examples

```

-----
>>> a = np.array(
...     [
...         "foo",
...         "foo",
...         "foo",
...         "foo",
...         "bar",
...         "bar",
...         "bar",
...         "bar",
...     ]

```

```

...     "foo",
...     "foo",
...     "foo",
... ],
... dtype=object,
... )
>>> b = np.array(
...     [
...         "one",
...         "one",
...         "one",
...         "two",
...         "one",
...         "one",
...         "one",
...         "two",
...         "two",
...         "two",
...         "one",
...     ],
...     dtype=object,
... )
>>> c = np.array(
...     [
...         "dull",
...         "dull",
...         "shiny",
...         "dull",
...         "dull",
...         "shiny",
...         "shiny",
...         "dull",
...         "shiny",
...         "shiny",
...         "shiny",
...     ],
...     dtype=object,
... )
>>> pd.crosstab(a, [b, c], rownames=["a"], colnames=["b", "c"])
b one two
c dull shiny dull shiny
a
bar 1 2 1 0
foo 2 2 1 2

```

Here 'c' and 'f' are not represented in the data and will not be shown in the output because dropna is True by default. Set dropna=False to preserve categories with no data.

```

>>> foo = pd.Categorical(["a", "b"], categories=["a", "b", "c"])
>>> bar = pd.Categorical(["d", "e"], categories=["d", "e", "f"])
>>> pd.crosstab(foo, bar)
col_0 d e
row_0
a 1 0
b 0 1
>>> pd.crosstab(foo, bar, dropna=False)
col_0 d e f
row_0
a 1 0 0
b 0 1 0
c 0 0 0

```

```

cut(
    x,
    bins,
    right: bool = True,
    labels=None,
    retbins: bool = False,
    precision: int = 3,
    include_lowest: bool = False,
    duplicates: str = 'raise',
    ordered: bool = True
)

```

Bin values into discrete intervals.

Use `cut` when you need to segment and sort data values into bins. This

function is also useful for going from a continuous variable to a categorical variable. For example, `'cut'` could convert ages to groups of age ranges. Supports binning into an equal number of bins, or a pre-specified array of bins.

Parameters

`x` : 1d ndarray or Series

The input array to be binned. Must be 1-dimensional.

`bins` : int, sequence of scalars, or IntervalIndex

The criteria to bin by.

- * int : Defines the number of equal-width bins in the range of `'x'`. The range of `'x'` is extended by .1% on each side to include the minimum and maximum values of `'x'`.

- * sequence of scalars : Defines the bin edges allowing for non-uniform width. No extension of the range of `'x'` is done.

- * IntervalIndex : Defines the exact bins to be used. Note that IntervalIndex for `'bins'` must be non-overlapping.

`right` : bool, default True

Indicates whether `'bins'` includes the rightmost edge or not. If `'right == True'` (the default), then the `'bins'` `'[1, 2, 3, 4]'` indicate `(1,2]`, `(2,3]`, `(3,4]`. This argument is ignored when `'bins'` is an IntervalIndex.

`labels` : array or False, default None

Specifies the labels for the returned bins. Must be the same length as the resulting bins. If False, returns only integer indicators of the bins. This affects the type of the output container (see below). This argument is ignored when `'bins'` is an IntervalIndex. If True, raises an error. When `'ordered=False'`, labels must be provided.

`retbins` : bool, default False

Whether to return the bins or not. Useful when bins is provided as a scalar.

`precision` : int, default 3

The precision at which to store and display the bins labels.

`include_lowest` : bool, default False

Whether the first interval should be left-inclusive or not.

`duplicates` : {'raise', 'drop'}, default 'raise'

If bin edges are not unique, raise ValueError or drop non-uniques.

`ordered` : bool, default True

Whether the labels are ordered or not. Applies to returned types Categorical and Series (with Categorical dtype). If True, the resulting categorical will be ordered. If False, the resulting categorical will be unordered (labels must be provided).

Returns

`out` : Categorical, Series, or ndarray

An array-like object representing the respective bin for each value of `'x'`. The type depends on the value of `'labels'`.

- * None (default) : returns a Series for Series `'x'` or a Categorical for all other inputs. The values stored within are Interval dtype.

- * sequence of scalars : returns a Series for Series `'x'` or a Categorical for all other inputs. The values stored within are whatever the type in the sequence is.

- * False : returns a 1d ndarray or Series of integers.

`bins` : numpy.ndarray or IntervalIndex.

The computed or specified bins. Only returned when `'retbins=True'`. For scalar or sequence `'bins'`, this is an ndarray with the computed bins. If set `'duplicates=drop'`, `'bins'` will drop non-unique bin. For an IntervalIndex `'bins'`, this is equal to `'bins'`.

See Also

`qcut` : Discretize variable into equal-sized buckets based on rank or based on sample quantiles.

Categorical : Array type for storing data that come from a fixed set of values.

Series : One-dimensional array with axis labels (including time series).

IntervalIndex : Immutable Index implementing an ordered, sliceable set.

numpy.histogram_bin_edges: Function to calculate only the edges of the bins

used by the histogram function.

Notes

Any NA values will be NA in the result. Out of bounds values will be NA in the resulting Series or Categorical object.

`numpy.histogram_bin_edges` can be used along with `cut` to calculate bins according to some predefined methods.

Reference :ref:`the user guide <reshaping.tile.cut>` for more examples.

Examples

Discretize into three equal-sized bins.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3)
... # doctest: +ELLIPSIS
[(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
```

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, retbins=True)
... # doctest: +ELLIPSIS
([(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
array([0.994, 3.    , 5.    , 7.    ])
```

Discovers the same bins, but assign them specific labels. Notice that the returned Categorical's categories are `labels` and is ordered.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, labels=["bad", "medium", "good"])
['bad', 'good', 'medium', 'medium', 'good', 'bad']
Categories (3, str): ['bad' < 'medium' < 'good']
```

`ordered=False` will result in unordered categories when labels are passed. This parameter can be used to allow non-unique labels:

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, labels=["B", "A", "B"], ordered=False)
['B', 'B', 'A', 'A', 'B', 'B']
Categories (2, str): ['A', 'B']
```

`labels=False` implies you just want the bins back.

```
>>> pd.cut([0, 1, 1, 2], bins=4, labels=False)
array([0, 1, 1, 3])
```

Passing a Series as an input returns a Series with categorical dtype:

```
>>> s = pd.Series(np.array([2, 4, 6, 8, 10]), index=["a", "b", "c", "d", "e"])
>>> pd.cut(s, 3)
... # doctest: +ELLIPSIS
a    (1.992, 4.667]
b    (1.992, 4.667]
c    (4.667, 7.333]
d    (7.333, 10.0]
e    (7.333, 10.0]
dtype: category
Categories (3, interval[float64, right]): [(1.992, 4.667] < (4.667, ...
```

Passing a Series as an input returns a Series with mapping value. It is used to map numerically to intervals based on bins.

```
>>> s = pd.Series(np.array([2, 4, 6, 8, 10]), index=["a", "b", "c", "d", "e"])
>>> pd.cut(s, [0, 2, 4, 6, 8, 10], labels=False, retbins=True, right=False)
... # doctest: +ELLIPSIS
(a    1.0
 b    2.0
 c    3.0
 d    4.0
 e    NaN
 dtype: float64,
 array([ 0,  2,  4,  6,  8, 10]))
```

Use `drop` optional when bins is not unique

```
>>> pd.cut(
...     s,
```

```

...     [0, 2, 4, 6, 10, 10],
...     labels=False,
...     retbins=True,
...     right=False,
...     duplicates="drop",
... )
... # doctest: +ELLIPSIS
(a      1.0
 b      2.0
 c      3.0
 d      3.0
 e      NaN
 dtype: float64,
 array([ 0,  2,  4,  6, 10]))

```

Passing an IntervalIndex for `bins` results in those categories exactly. Notice that values not covered by the IntervalIndex are set to NaN. 0 is to the left of the first bin (which is closed on the right), and 1.5 falls between two bins.

```

>>> bins = pd.IntervalIndex.from_tuples([(0, 1), (2, 3), (4, 5)])
>>> pd.cut([0, 0.5, 1.5, 2.5, 4.5], bins)
[NaN, (0.0, 1.0], NaN, (2.0, 3.0], (4.0, 5.0]]
Categories (3, interval[int64, right]): [(0, 1] < (2, 3] < (4, 5]]

```

Using np.histogram_bin_edges with cut

```

>>> pd.cut(
...     np.array([1, 7, 5, 4]),
...     bins=np.histogram_bin_edges(np.array([1, 7, 5, 4]), bins="auto"),
... )
... # doctest: +ELLIPSIS
[NaN, (5.0, 7.0], (3.0, 5.0], (3.0, 5.0]]
Categories (3, interval[float64, right]): [(1.0, 3.0] < (3.0, 5.0] < (5.0, 7.0]]

```

```

date_range(
    start=None,
    end=None,
    periods=None,
    freq=None,
    tz=None,
    normalize: bool = False,
    name: Hashable | None = None,
    inclusive: IntervalClosedType = 'both',
    *,
    unit: TimeUnit | None = None,
    **kwargs
) -> DatetimeIndex

```

Return a fixed frequency DatetimeIndex.

Returns the range of equally spaced time points (where the difference between any two adjacent points is specified by the given frequency) such that they fall in the range `[start, end]`, where the first one and the last one are, resp., the first and last time points in that range that fall on the boundary of `freq` (if given as a frequency string) or that are valid for `freq` (if given as a :class:`pandas.tseries.offsets.DateOffset`). If `freq` is positive, the points satisfy `start <=[] x <=[] end`, and if `freq` is negative, the points satisfy `end <=[] x <=[] start`. (If exactly one of `start`, `end`, or `freq` is *not* specified, this missing parameter can be computed given `periods`, the number of timesteps in the range. See the note below.)

Parameters

```

-----
start : str or datetime-like, optional
    Left bound for generating dates.
end : str or datetime-like, optional
    Right bound for generating dates.
periods : int, optional
    Number of periods to generate.
freq : str, Timedelta, datetime.timedelta, or DateOffset, default 'D'
    Frequency strings can have multiples, e.g. '5h'. See
    :ref:`here <tseries.offset_aliases>` for a list of
    frequency aliases.
tz : str or tzinfo, optional
    Time zone name for returning localized DatetimeIndex, for example
    'Asia/Hong_Kong'. By default, the resulting DatetimeIndex is
    timezone-naive unless timezone-aware datetime-likes are passed.

```

```

normalize : bool, default False
    Normalize start/end dates to midnight before generating date range.
name : Hashable, default None
    Name of the resulting DatetimeIndex.
inclusive : {"both", "neither", "left", "right"}, default "both"
    Include boundaries; Whether to set each bound as closed or open.
unit : {'s', 'ms', 'us', 'ns', None}, default None
    Specify the desired resolution of the result.
    If not specified, this is inferred from the 'start', 'end', and 'freq'
    using the same inference as :class:`Timestamp` taking the highest
    resolution of the three that are provided.

.. versionadded:: 2.0.0
**kwargs
    For compatibility. Has no effect on the result.

```

Returns

DatetimeIndex

A DatetimeIndex object of the generated dates.

See Also

DatetimeIndex : An immutable container for datetimes.

timedelta_range : Return a fixed frequency TimedeltaIndex.

period_range : Return a fixed frequency PeriodIndex.

interval_range : Return a fixed frequency IntervalIndex.

Notes

Of the four parameters ``start``, ``end``, ``periods``, and ``freq``, a maximum of three can be specified at once. Of the three parameters ``start``, ``end``, and ``periods``, at least two must be specified. If ``freq`` is omitted, the resulting ``DatetimeIndex`` will have ``periods`` linearly spaced elements between ``start`` and ``end`` (closed on both sides).

To learn more about the frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

Examples

****Specifying the values****

The next four examples generate the same ``DatetimeIndex``, but vary the combination of ``start``, ``end`` and ``periods``.

Specify ``start`` and ``end``, with the default daily frequency.

```

>>> pd.date_range(start="1/1/2018", end="1/08/2018")
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04',
              '2018-01-05', '2018-01-06', '2018-01-07', '2018-01-08'],
              dtype='datetime64[us]', freq='D')

```

Specify timezone-aware ``start`` and ``end``, with the default daily frequency.

```

>>> pd.date_range(
...     start=pd.to_datetime("1/1/2018").tz_localize("Europe/Berlin"),
...     end=pd.to_datetime("1/08/2018").tz_localize("Europe/Berlin"),
... )
DatetimeIndex(['2018-01-01 00:00:00+01:00', '2018-01-02 00:00:00+01:00',
              '2018-01-03 00:00:00+01:00', '2018-01-04 00:00:00+01:00',
              '2018-01-05 00:00:00+01:00', '2018-01-06 00:00:00+01:00',
              '2018-01-07 00:00:00+01:00', '2018-01-08 00:00:00+01:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='D')

```

Specify ``start`` and ``periods``, the number of periods (days).

```

>>> pd.date_range(start="1/1/2018", periods=8)
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04',
              '2018-01-05', '2018-01-06', '2018-01-07', '2018-01-08'],
              dtype='datetime64[us]', freq='D')

```

Specify ``end`` and ``periods``, the number of periods (days).

```

>>> pd.date_range(end="1/1/2018", periods=8)
DatetimeIndex(['2017-12-25', '2017-12-26', '2017-12-27', '2017-12-28',

```

```

        '2017-12-29', '2017-12-30', '2017-12-31', '2018-01-01'],
dtype='datetime64[us]', freq='D')

```

Specify `start`, `end`, and `periods`; the frequency is generated automatically (linearly spaced).

```

>>> pd.date_range(start="2018-04-24", end="2018-04-27", periods=3)
DatetimeIndex(['2018-04-24 00:00:00', '2018-04-25 12:00:00',
               '2018-04-27 00:00:00'],
              dtype='datetime64[us]', freq=None)

```

****Other Parameters****

Changed the `freq` (frequency) to ``'ME'`` (month end frequency).

```

>>> pd.date_range(start="1/1/2018", periods=5, freq="ME")
DatetimeIndex(['2018-01-31', '2018-02-28', '2018-03-31', '2018-04-30',
               '2018-05-31'],
              dtype='datetime64[us]', freq='ME')

```

Multiples are allowed

```

>>> pd.date_range(start="1/1/2018", periods=5, freq="3ME")
DatetimeIndex(['2018-01-31', '2018-04-30', '2018-07-31', '2018-10-31',
               '2019-01-31'],
              dtype='datetime64[us]', freq='3ME')

```

`freq` can also be specified as an Offset object.

```

>>> pd.date_range(start="1/1/2018", periods=5, freq=pd.offsets.MonthEnd(3))
DatetimeIndex(['2018-01-31', '2018-04-30', '2018-07-31', '2018-10-31',
               '2019-01-31'],
              dtype='datetime64[us]', freq='3ME')

```

Specify `tz` to set the timezone.

```

>>> pd.date_range(start="1/1/2018", periods=5, tz="Asia/Tokyo")
DatetimeIndex(['2018-01-01 00:00:00+09:00', '2018-01-02 00:00:00+09:00',
               '2018-01-03 00:00:00+09:00', '2018-01-04 00:00:00+09:00',
               '2018-01-05 00:00:00+09:00'],
              dtype='datetime64[us, Asia/Tokyo]', freq='D')

```

`inclusive` controls whether to include `start` and `end` that are on the boundary. The default, "both", includes boundary points on either end.

```

>>> pd.date_range(start="2017-01-01", end="2017-01-04", inclusive="both")
DatetimeIndex(['2017-01-01', '2017-01-02', '2017-01-03', '2017-01-04'],
              dtype='datetime64[us]', freq='D')

```

Use ``inclusive='left'`` to exclude `end` if it falls on the boundary.

```

>>> pd.date_range(start="2017-01-01", end="2017-01-04", inclusive="left")
DatetimeIndex(['2017-01-01', '2017-01-02', '2017-01-03'],
              dtype='datetime64[us]', freq='D')

```

Use ``inclusive='right'`` to exclude `start` if it falls on the boundary, and similarly ``inclusive='neither'`` will exclude both `start` and `end`.

```

>>> pd.date_range(start="2017-01-01", end="2017-01-04", inclusive="right")
DatetimeIndex(['2017-01-02', '2017-01-03', '2017-01-04'],
              dtype='datetime64[us]', freq='D')

```

****Specify a unit****

```

>>> pd.date_range(start="2017-01-01", periods=10, freq="100YS", unit="s")
DatetimeIndex(['2017-01-01', '2117-01-01', '2217-01-01', '2317-01-01',
               '2417-01-01', '2517-01-01', '2617-01-01', '2717-01-01',
               '2817-01-01', '2917-01-01'],
              dtype='datetime64[s]', freq='100YS-JAN')

```

`describe_option(pat: str = '', _print_desc: bool = True) -> str | None`
 Print the description for one or more registered options.

Call with no arguments to get a listing for all registered options.

Parameters

```

pat : str, default ""
    String or string regexp pattern.
    Empty string will return all options.
    For regexp strings, all matching keys will have their description displayed.
_print_desc : bool, default True
    If True (default) the description(s) will be printed to stdout.
    Otherwise, the description(s) will be returned as a string
    (for testing).

Returns
-----
None
    If ``_print_desc=True``.
str
    If the description(s) as a string if ``_print_desc=False``.

See Also
-----
get_option : Retrieve the value of the specified option.
set_option : Set the value of the specified option or options.
reset_option : Reset one or more options to their default value.

Notes
-----
For all available options, please view the
:ref:`User Guide <options.available>`.

Examples
-----
>>> pd.describe_option("display.max_columns") # doctest: +SKIP
display.max_columns : int
    If max_cols is exceeded, switch to truncate view...

eval(
    expr: str | BinOp,
    parser: str = 'pandas',
    engine: str | None = None,
    local_dict=None,
    global_dict=None,
    resolvers=(),
    level: int = 0,
    target=None,
    inplace: bool = False
) -> Any
    Evaluate a Python expression as a string using various backends.

.. warning::

    This function can run arbitrary code which can make you vulnerable to code
    injection if you pass user input to this function.

Parameters
-----
expr : str
    The expression to evaluate. This string cannot contain any Python
    `statements`
    <https://docs.python.org/3/reference/simple\_stmts.html#simple-statements>`,
    only Python `expressions`
    <https://docs.python.org/3/reference/simple\_stmts.html#expression-statements>`.

    By default, with the numexpr engine, the following operations are supported:

    - Arithmetic operations: ``+``, ``-``, ``*``, ``/``, ``**``, ``%``
    - Boolean operations: ``|`` (or), ``&`` (and), and ``~`` (not)
    - Comparison operators: ``<``, ``<=``, ``==``, ``!=``, ``>=``, ``>``

    Furthermore, the following mathematical functions are supported:

    - Trigonometric: ``sin``, ``cos``, ``tan``, ``arcsin``, ``arccos``, ``arctan``
    - Logarithms: ``log`` natural, ``log10`` base 10, ``log1p`` log(1+x)
    - Absolute Value ``abs``
    - Square root ``sqrt``
    - Exponential ``exp`` and Exponential minus one ``expm1``

    See the numexpr engine `documentation`
    <https://numexpr.readthedocs.io/en/latest/user\_guide.html#supported-functions>`
    for further function support details.

```

Using the `python` engine allows the use of native Python operators such as floor division `//`, in addition to built-in and user-defined Python functions.

Additionally, the `pandas` parser allows the use of `:keyword:and`, `:keyword:or`, and `:keyword:not` with the same semantics as the corresponding bitwise operators.

`parser` : {'pandas', 'python'}, default 'pandas'

The parser to use to construct the syntax tree from the expression. The default of `python` parses code slightly different than standard Python. Alternatively, you can parse an expression using the `python` parser to retain strict Python semantics. See the [:ref:enhancing performance <enhancingperf.eval>](#) documentation for more details.

`engine` : {'python', 'numexpr'}, optional, default None

The engine used to evaluate the expression. Supported engines are

- None : tries to use `numexpr`, falls back to `python`
- `numexpr` : This is the default engine when `numexpr` is installed. Evaluates pandas objects using `numexpr` for large speed ups in complex expressions with large frames.
- `python` : Performs operations as if you had `eval`'d in top level python. This engine is generally not that useful.

More backends may be available in the future.

`local_dict` : dict or None, optional

A dictionary of local variables, taken from `locals()` by default.

`global_dict` : dict or None, optional

A dictionary of global variables, taken from `globals()` by default.

`resolvers` : list of dict-like or None, optional

A list of objects implementing the `__getitem__` special method that you can use to inject an additional collection of namespaces to use for variable lookup. For example, this is used in the `DataFrame.query` method to inject the `DataFrame.index` and `DataFrame.columns` variables that refer to their respective `class:~pandas.DataFrame` instance attributes.

`level` : int, optional

The number of prior stack frames to traverse and add to the current scope. Most users will **not** need to change this parameter.

`target` : object, optional, default None

This is the target object for assignment. It is used when there is variable assignment in the expression. If so, then `target` must support item assignment with string keys, and if a copy is being returned, it must also support `.copy()`.

`inplace` : bool, default False

If `target` is provided, and the expression mutates `target`, whether to modify `target` inplace. Otherwise, return a copy of `target` with the mutation.

Returns

ndarray, numeric scalar, DataFrame, Series, or None

The completion value of evaluating the given code or None if `inplace=True`.

Raises

ValueError

There are many instances where such an error can be raised:

- `target=None`, but the expression is multiline.
- The expression is multiline, but not all them have item assignment. An example of such an arrangement is this:

```
a = b + 1
a + 2
```

Here, there are expressions on different lines, making it multiline, but the last line has no variable assigned to the output of `a + 2`.

- `inplace=True`, but the expression is missing item assignment.
- Item assignment is provided, but the `target` does not support string item assignment.
- Item assignment is provided and `inplace=False`, but the `target` does not support the `.copy()` method

See Also

```
-----
DataFrame.query : Evaluates a boolean expression to query the columns
                  of a frame.
DataFrame.eval : Evaluate a string describing operations on
                DataFrame columns.
```

Notes

```
-----
The ``dtype`` of any objects involved in an arithmetic ``%`` operation are
recursively cast to ``float64``.
```

See the :ref:`enhancing performance <enhancingperf.eval>` documentation for more details.

Examples

```
-----
>>> df = pd.DataFrame({"animal": ["dog", "pig"], "age": [10, 20]})
>>> df
   animal  age
0     dog   10
1     pig   20
```

We can add a new column using ``pd.eval``:

```
>>> pd.eval("double_age = df.age * 2", target=df)
   animal  age  double_age
0     dog   10           20
1     pig   20           40
```

```
factorize(
    values,
    sort: bool = False,
    use_na_sentinel: bool = True,
    size_hint: int | None = None
) -> tuple[np.ndarray, np.ndarray | Index]
Encode the object as an enumerated type or categorical variable.
```

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. ``factorize`` is available as both a top-level function :func:`pandas.factorize`, and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

Parameters

```
-----
values : sequence
    A 1-D sequence. Sequences that aren't pandas objects are
    coerced to ndarrays before factorization.
sort : bool, default False
    Sort `uniques` and shuffle `codes` to maintain the
    relationship.
use_na_sentinel : bool, default True
    If True, the sentinel -1 will be used for NaN values. If False,
    NaN values will be encoded as non-negative integers and will not drop the
    NaN from the uniques of the values.
size_hint : int, optional
    Hint to the hashtable sizer.
```

Returns

```
-----
codes : ndarray
    An integer ndarray that's an indexer into `uniques`.
    ``uniques.take(codes)`` will have the same values as `values`.
uniques : ndarray, Index, or Categorical
    The unique valid values. When `values` is Categorical, `uniques`
    is a Categorical. When `values` is some other pandas object, an
    `Index` is returned. Otherwise, a 1-D ndarray is returned.

    .. note::

        Even if there's a missing value in `values`, `uniques` will
        *not* contain an entry for it.
```

See Also

```
-----
cut : Discretize continuous-valued array.
unique : Find the unique value in an array.
Notes
```

Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

These examples all show factorize as a top-level method like
`pd.factorize(values)`. The results are identical for methods like
:meth:`Series.factorize`.

```
>>> codes, uniques = pd.factorize(np.array(["b", "b", "a", "c", "b"], dtype="O"))
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With `sort=True`, the `uniques` will be sorted, and `codes` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
...     np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
... )
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When `use\_na\_sentinel=True` (the default), missing values are indicated in the `codes` with the sentinel value `-1` and missing values are not included in `uniques`.

```
>>> codes, uniques = pd.factorize(np.array(["b", None, "a", "c", "b"], dtype="O"))
>>> codes
array([ 0, -1,  1,  2,  0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of `uniques` will differ. For Categoricals, a `Categorical` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Notice that `b` is in `uniques.categories`, despite not being present in `cat.values`.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting `use\_na\_sentinel=False`.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([ 0,  1,  0, -1])
>>> uniques
array([1., 2.])
```

```
>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([1., 2., nan])
```

from_dummies(

```

data: DataFrame,
sep: None | str = None,
default_category: None | Hashable | dict[str, Hashable] = None
) -> DataFrame
Create a categorical ``DataFrame`` from a ``DataFrame`` of dummy variables.

Inverts the operation performed by :func:`~pandas.get_dummies`.

Parameters
-----
data : DataFrame
    Data which contains dummy-coded variables in form of integer columns of
    1's and 0's.
sep : str, default None
    Separator used in the column names of the dummy categories they are
    character indicating the separation of the categorical names from the prefixes.
    For example, if your column names are 'prefix_A' and 'prefix_B',
    you can strip the underscore by specifying sep='_'.
default_category : None, Hashable or dict of Hashables, default None
    The default category is the implied category when a value has none of the
    listed categories specified with a one, i.e. if all dummies in a row are
    zero. Can be a single value for all variables or a dict directly mapping
    the default categories to a prefix of a variable. The default category
    will be coerced to the dtype of ``data.columns`` if such coercion is
    lossless, and will raise otherwise.

Returns
-----
DataFrame
    Categorical data decoded from the dummy input-data.

Raises
-----
ValueError
    * When the input ``DataFrame`` ``data`` contains NA values.
    * When the input ``DataFrame`` ``data`` contains column names with separators
      that do not match the separator specified with ``sep``.
    * When a ``dict`` passed to ``default_category`` does not include an implied
      category for each prefix.
    * When a value in ``data`` has more than one category assigned to it.
    * When ``default_category=None`` and a value in ``data`` has no category
      assigned to it.
TypeError
    * When the input ``data`` is not of type ``DataFrame``.
    * When the input ``DataFrame`` ``data`` contains non-dummy data.
    * When the passed ``sep`` is of a wrong data type.
    * When the passed ``default_category`` is of a wrong data type.

See Also
-----
:func:`~pandas.get_dummies` : Convert ``Series`` or ``DataFrame`` to dummy codes.
:class:`~pandas.Categorical` : Represent a categorical variable in classic.

Notes
-----
The columns of the passed dummy data should only include 1's and 0's,
or boolean values.

Examples
-----
>>> df = pd.DataFrame({"a": [1, 0, 0, 1], "b": [0, 1, 0, 0], "c": [0, 0, 1, 0]})
>>> df
   a  b  c
0  1  0  0
1  0  1  0
2  0  0  1
3  1  0  0

>>> pd.from_dummies(df)
   a
0  a
1  b
2  c
3  a

>>> df = pd.DataFrame(
...     {

```

```
...     "col1_a": [1, 0, 1],
...     "col1_b": [0, 1, 0],
...     "col2_a": [0, 1, 0],
...     "col2_b": [1, 0, 0],
...     "col2_c": [0, 0, 1],
... }
... )
```

```
>>> df
   col1_a  col1_b  col2_a  col2_b  col2_c
0        1        0        0        1        0
1        0        1        1        0        0
2        1        0        0        0        1
```

```
>>> pd.from_dummies(df, sep="_")
   col1  col2
0     a     b
1     b     a
2     a     c
```

```
>>> df = pd.DataFrame(
...     {
...         "col1_a": [1, 0, 0],
...         "col1_b": [0, 1, 0],
...         "col2_a": [0, 1, 0],
...         "col2_b": [1, 0, 0],
...         "col2_c": [0, 0, 0],
...     }
... )
```

```
>>> df
   col1_a  col1_b  col2_a  col2_b  col2_c
0        1        0        0        1        0
1        0        1        1        0        0
2        0        0        0        0        0
```

```
>>> pd.from_dummies(df, sep="_", default_category={"col1": "d", "col2": "e"})
   col1  col2
0     a     b
1     b     a
2     d     e
```

```
get_dummies(
    data,
    prefix=None,
    prefix_sep: str | Iterable[str] | dict[str, str] = '_',
    dummy_na: bool = False,
    columns=None,
    sparse: bool = False,
    drop_first: bool = False,
    dtype: NpDtype | None = None
) -> DataFrame
Convert categorical variable into dummy/indicator variables.
```

Each variable is converted in as many 0/1 variables as there are different values. Columns in the output are each named after a value; if the input is a DataFrame, the name of the original variable is prepended to the value.

Parameters

```
-----
data : array-like, Series, or DataFrame
    Data of which to get dummy indicators.
prefix : str, list of str, or dict of str, default None
    A string to be prepended to DataFrame column names.
    Pass a list with length equal to the number of columns
    when calling get_dummies on a DataFrame. Alternatively, `prefix`
    can be a dictionary mapping column names to prefixes.
prefix_sep : str, list of str, or dict of str, default '_'
    Should you choose to prepend DataFrame column names with a prefix, this
    is the separator/delimiter to use between the two. Alternatively,
    `prefix_sep` can be a list with length equal to the number of columns,
    or a dictionary mapping column names to separators.
dummy_na : bool, default False
    If True, a NaN indicator column will be added even if no NaN values are present.
    If False, NA values are encoded as all zero.
columns : list-like, default None
    Column names in the DataFrame to be encoded.
```

```

    If `columns` is None then all the columns with
    `object`, `string`, or `category` dtype will be converted.
sparse : bool, default False
    Whether the dummy-encoded columns should be backed by
    a :class:`SparseArray` (True) or a regular NumPy array (False).
drop_first : bool, default False
    Whether to get k-1 dummies out of k categorical levels by removing the
    first level.
dtype : dtype, default bool
    Data type for new columns. Only a single dtype is allowed.

Returns
-----
DataFrame
    Dummy-coded data. If `data` contains other columns than the
    dummy-coded one(s), these will be prepended, unaltered, to the result.

See Also
-----
Series.str.get_dummies : Convert Series of strings to dummy codes.
:func:`pandas.from_dummies` : Convert dummy codes to categorical ``DataFrame``.

Notes
-----
Reference :ref:`the user guide <reshaping.dummies>` for more examples.

Examples
-----
>>> s = pd.Series(list("abca"))

>>> pd.get_dummies(s)
      a      b      c
0  True  False  False
1  False  True  False
2  False  False  True
3  True  False  False

>>> s1 = ["a", "b", np.nan]

>>> pd.get_dummies(s1)
      a      b
0  True  False
1  False  True
2  False  False

>>> pd.get_dummies(s1, dummy_na=True)
      a      b    NaN
0  True  False  False
1  False  True  False
2  False  False  True

>>> df = pd.DataFrame({"A": ["a", "b", "a"], "B": ["b", "a", "c"], "C": [1, 2, 3]})

>>> pd.get_dummies(df, prefix=["col1", "col2"])
   C  col1_a  col1_b  col2_a  col2_b  col2_c
0  1     True   False   False    True   False
1  2     False  True    True    False  False
2  3     True   False   False   False    True

>>> pd.get_dummies(pd.Series(list("abcaa")))
      a      b      c
0  True  False  False
1  False  True  False
2  False  False  True
3  True  False  False
4  True  False  False

>>> pd.get_dummies(pd.Series(list("abcaa")), drop_first=True)
      b      c
0  False  False
1   True  False
2  False  True
3  False  False
4  False  False

>>> pd.get_dummies(pd.Series(list("abc")), dtype=float)
      a      b      c

```

```

0  1.0  0.0  0.0
1  0.0  1.0  0.0
2  0.0  0.0  1.0

```

`get_option(pat: str) -> Any`

Retrieve the value of the specified option.

This method allows users to query the current value of a given option in the pandas configuration system. Options control various display, performance, and behavior-related settings within pandas.

Parameters

`pat : str`

Regex which should match a single option.

.. warning::

Partial matches are supported for convenience, but unless you use the full option name (e.g. `x.y.z.option_name`), your code may break in future versions if new options with similar names are introduced.

Returns

Any

The value of the option.

Raises

`OptionError` : if no such option exists

See Also

`set_option` : Set the value of the specified option or options.

`reset_option` : Reset one or more options to their default value.

`describe_option` : Print the description for one or more registered options.

Notes

For all available options, please view the :ref:`User Guide <options.available>` or use ``pandas.describe_option()``.

Examples

```
>>> pd.get_option("display.max_columns") # doctest: +SKIP
```

```
4
```

`infer_freq(`

`index: DatetimeIndex | TimedeltaIndex | Series | DatetimeLikeArrayMixin`

`) -> str | None`

Infer the most likely frequency given the input index.

This method attempts to deduce the most probable frequency (e.g., 'D' for daily, 'H' for hourly) from a sequence of datetime-like objects. It is particularly useful when the frequency of a time series is not explicitly set or known but can be inferred from its values.

Parameters

`index : DatetimeIndex, TimedeltaIndex, Series or array-like`

If passed a Series will use the values of the series (NOT THE INDEX).

Returns

`str or None`

None if no discernible frequency.

Raises

`TypeError`

If the index is not datetime-like.

`ValueError`

If there are fewer than three values.

See Also

`date_range` : Return a fixed frequency `DatetimeIndex`.

timedelta_range : Return a fixed frequency TimedeltaIndex with day as the default.
period_range : Return a fixed frequency PeriodIndex.
DatetimeIndex.freq : Return the frequency object if it is set, otherwise None.

Examples

```
-----
>>> idx = pd.date_range(start="2020/12/01", end="2020/12/30", periods=30)
>>> pd.infer_freq(idx)
'D'
```

```
interval_range(
    start=None,
    end=None,
    periods=None,
    freq=None,
    name: Hashable | None = None,
    closed: IntervalClosedType = 'right'
) -> IntervalIndex
Return a fixed frequency IntervalIndex.
```

Parameters

```
-----
start : numeric or datetime-like, default None
    Left bound for generating intervals.
end : numeric or datetime-like, default None
    Right bound for generating intervals.
periods : int, default None
    Number of periods to generate.
freq : numeric, str, Timedelta, datetime.timedelta, or DateOffset, default None
    The length of each interval. Must be consistent with the type of start
    and end, e.g. 2 for numeric, or '5H' for datetime-like. Default is 1
    for numeric and 'D' for datetime-like.
name : str, default None
    Name of the resulting IntervalIndex.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
    Whether the intervals are closed on the left-side, right-side, both
    or neither.
```

Returns

```
-----
IntervalIndex
    Object with a fixed frequency.
```

See Also

```
-----
IntervalIndex : An Index of intervals that are all closed on the same side.
```

Notes

```
-----
Of the four parameters ``start``, ``end``, ``periods``, and ``freq``,
exactly three must be specified. If ``freq`` is omitted, the resulting
``IntervalIndex`` will have ``periods`` linearly spaced elements between
``start`` and ``end``, inclusively.
```

To learn more about datetime-like frequency strings, please see
:ref: this [link<timeseries.offset_aliases>](#).

Examples

```
-----
Numeric ``start`` and ``end`` is supported.
```

```
>>> pd.interval_range(start=0, end=5)
IntervalIndex([(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]],
              dtype='interval[int64, right]')
```

Additionally, datetime-like input is also supported.

```
>>> pd.interval_range(
...     start=pd.Timestamp("2017-01-01"), end=pd.Timestamp("2017-01-04")
... )
IntervalIndex([(2017-01-01 00:00:00, 2017-01-02 00:00:00],
              (2017-01-02 00:00:00, 2017-01-03 00:00:00],
              (2017-01-03 00:00:00, 2017-01-04 00:00:00]],
              dtype='interval[datetime64[us], right]')
```

The ``freq`` parameter specifies the frequency between the left and right endpoints of the individual intervals within the ``IntervalIndex``. For

numeric ``start`` and ``end``, the frequency must also be numeric.

```
>>> pd.interval_range(start=0, periods=4, freq=1.5)
IntervalIndex([(0.0, 1.5], (1.5, 3.0], (3.0, 4.5], (4.5, 6.0]],
              dtype='interval[float64, right]')
```

Similarly, for datetime-like ``start`` and ``end``, the frequency must be convertible to a DateOffset.

```
>>> pd.interval_range(start=pd.Timestamp("2017-01-01"), periods=3, freq="MS")
IntervalIndex([(2017-01-01 00:00:00, 2017-02-01 00:00:00],
              (2017-02-01 00:00:00, 2017-03-01 00:00:00],
              (2017-03-01 00:00:00, 2017-04-01 00:00:00]],
              dtype='interval[datetime64[us], right]')
```

Specify ``start``, ``end``, and ``periods``; the frequency is generated automatically (linearly spaced).

```
>>> pd.interval_range(start=0, end=6, periods=4)
IntervalIndex([(0.0, 1.5], (1.5, 3.0], (3.0, 4.5], (4.5, 6.0]],
              dtype='interval[float64, right]')
```

The ``closed`` parameter specifies which endpoints of the individual intervals within the ``IntervalIndex`` are closed.

```
>>> pd.interval_range(end=5, periods=4, closed="both")
IntervalIndex([[1, 2], [2, 3], [3, 4], [4, 5]],
              dtype='interval[int64, both]')
```

```
isna(obj: object) -> bool | npt.NDArray[np.bool_] | NDFrame
Detect missing values for an array-like object.
```

This function takes a scalar or array-like object and indicates whether values are missing (``NaN`` in numeric arrays, ``None`` or ``Na`` in object arrays, ``NaT`` in datetimelike).

Parameters

obj : scalar or array-like
Object to check for null or missing values.

Returns

bool or array-like of bool
For scalar input, returns a scalar boolean.
For array input, returns an array of boolean indicating whether each corresponding element is missing.

See Also

notna : Boolean inverse of pandas.isna.
Series.isna : Detect missing values in a Series.
DataFrame.isna : Detect missing values in a DataFrame.
Index.isna : Detect missing values in an Index.

Examples

Scalar arguments (including strings) result in a scalar boolean.

```
>>> pd.isna("dog")
False
```

```
>>> pd.isna(pd.NA)
True
```

```
>>> pd.isna(np.nan)
True
```

ndarrays result in an ndarray of booleans.

```
>>> array = np.array([[1, np.nan, 3], [4, 5, np.nan]])
>>> array
array([[ 1., nan,  3.],
       [ 4.,  5., nan]])
>>> pd.isna(array)
array([[False,  True, False],
       [False, False,  True]])
```

For indexes, an ndarray of booleans is returned.

```
>>> index = pd.DatetimeIndex(["2017-07-05", "2017-07-06", None, "2017-07-08"])
>>> index
DatetimeIndex(['2017-07-05', '2017-07-06', 'NaT', '2017-07-08'],
              dtype='datetime64[us]', freq=None)
>>> pd.isna(index)
array([False, False,  True, False])
```

For Series and DataFrame, the same type is returned, containing booleans.

```
>>> df = pd.DataFrame([["ant", "bee", "cat"], ["dog", None, "fly"]])
>>> df
   0    1    2
0 ant  bee  cat
1 dog NaN  fly
>>> pd.isna(df)
   0    1    2
0 False False False
1 False  True False

>>> pd.isna(df[1])
0    False
1     True
Name: 1, dtype: bool
```

`isnull = isna(obj: object) -> bool | npt.NDArray[np.bool_] | NDFrame`
Detect missing values for an array-like object.

This function takes a scalar or array-like object and indicates whether values are missing (``NaN`` in numeric arrays, ``None`` or ``NaT`` in object arrays, ``NaT`` in datetimelike).

Parameters

obj : scalar or array-like
Object to check for null or missing values.

Returns

bool or array-like of bool
For scalar input, returns a scalar boolean.
For array input, returns an array of boolean indicating whether each corresponding element is missing.

See Also

notna : Boolean inverse of pandas.isna.
Series.isna : Detect missing values in a Series.
DataFrame.isna : Detect missing values in a DataFrame.
Index.isna : Detect missing values in an Index.

Examples

Scalar arguments (including strings) result in a scalar boolean.

```
>>> pd.isna("dog")
False

>>> pd.isna(pd.NA)
True

>>> pd.isna(np.nan)
True
```

ndarrays result in an ndarray of booleans.

```
>>> array = np.array([[1, np.nan, 3], [4, 5, np.nan]])
>>> array
array([[ 1., nan,  3.],
       [ 4.,  5., nan]])
>>> pd.isna(array)
array([[False,  True, False],
       [False, False,  True]])
```

For indexes, an ndarray of booleans is returned.

```
>>> index = pd.DatetimeIndex(["2017-07-05", "2017-07-06", None, "2017-07-08"])
```



```
>>> index
DatetimeIndex(['2017-07-05', '2017-07-06', 'NaT', '2017-07-08'],
              dtype='datetime64[us]', freq=None)
>>> pd.isna(index)
array([False, False,  True, False])
```

For Series and DataFrame, the same type is returned, containing booleans.

```
>>> df = pd.DataFrame([["ant", "bee", "cat"], ["dog", None, "fly"]])
>>> df
   0  1  2
0 ant bee cat
1 dog NaN fly
>>> pd.isna(df)
   0  1  2
0 False False False
1 False  True False

>>> pd.isna(df[1])
0    False
1     True
Name: 1, dtype: bool
```

```
json_normalize(
    data: dict | list[dict] | Series,
    record_path: str | list | None = None,
    meta: str | list[str | list[str]] | None = None,
    meta_prefix: str | None = None,
    record_prefix: str | None = None,
    errors: IgnoreRaise = 'raise',
    sep: str = '.',
    max_level: int | None = None
) -> DataFrame
Normalize semi-structured JSON data into a flat table.
```

This method is designed to transform semi-structured JSON data, such as nested dictionaries or lists, into a flat table. This is particularly useful when handling JSON-like data structures that contain deeply nested fields.

Parameters

```
-----
data : dict, list of dicts, or Series of dicts
      Unserialized JSON objects.
record_path : str or list of str, default None
      Path in each object to list of records. If not passed, data will be
      assumed to be an array of records.
meta : list of paths (str or list of str), default None
      Fields to use as metadata for each record in resulting table.
meta_prefix : str, default None
      String to prefix records with dotted path, e.g. foo.bar.field if
      meta is ['foo', 'bar'].
record_prefix : str, default None
      String to prefix records with dotted path, e.g. foo.bar.field if
      path to records is ['foo', 'bar'].
errors : {'raise', 'ignore'}, default 'raise'
      Configures error handling.

      * 'ignore' : will ignore KeyError if keys listed in meta are not
        always present.
      * 'raise' : will raise KeyError if keys listed in meta are not
        always present.
sep : str, default '.'
      Nested records will generate names separated by sep.
      e.g., for sep='.', {'foo': {'bar': 0}} -> foo.bar.
max_level : int, default None
      Max number of levels(depth of dict) to normalize.
      if None, normalizes all levels.
```

Returns

```
-----
DataFrame
    The normalized data, represented as a pandas DataFrame.
```

See Also

```
-----
DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.
Series : One-dimensional ndarray with axis labels (including time series).
```

Examples

```
-----
>>> data = [
...     {"id": 1, "name": {"first": "Coleen", "last": "Volk"}},
...     {"name": {"given": "Mark", "family": "Regner"}},
...     {"id": 2, "name": "Faye Raker"},
... ]
>>> pd.json_normalize(data)
   id name.first name.last name.given name.family      name
0  1.0      Coleen      Volk        NaN        NaN        NaN
1  NaN         NaN         NaN        Mark      Regner        NaN
2  2.0         NaN         NaN        NaN        NaN  Faye Raker

>>> data = [
...     {
...         "id": 1,
...         "name": "Cole Volk",
...         "fitness": {"height": 130, "weight": 60},
...     },
...     {"name": "Mark Reg", "fitness": {"height": 130, "weight": 60}},
...     {
...         "id": 2,
...         "name": "Faye Raker",
...         "fitness": {"height": 130, "weight": 60},
...     },
... ]
>>> pd.json_normalize(data, max_level=0)
   id      name      fitness
0  1.0  Cole Volk  {'height': 130, 'weight': 60}
1  NaN  Mark Reg  {'height': 130, 'weight': 60}
2  2.0  Faye Raker {'height': 130, 'weight': 60}
```

Normalizes nested data up to level 1.

```
>>> data = [
...     {
...         "id": 1,
...         "name": "Cole Volk",
...         "fitness": {"height": 130, "weight": 60},
...     },
...     {"name": "Mark Reg", "fitness": {"height": 130, "weight": 60}},
...     {
...         "id": 2,
...         "name": "Faye Raker",
...         "fitness": {"height": 130, "weight": 60},
...     },
... ]
>>> pd.json_normalize(data, max_level=1)
   id      name  fitness.height  fitness.weight
0  1.0  Cole Volk             130             60
1  NaN  Mark Reg             130             60
2  2.0  Faye Raker           130             60

>>> data = [
...     {
...         "id": 1,
...         "name": "Cole Volk",
...         "fitness": {"height": 130, "weight": 60},
...     },
...     {"name": "Mark Reg", "fitness": {"height": 130, "weight": 60}},
...     {
...         "id": 2,
...         "name": "Faye Raker",
...         "fitness": {"height": 130, "weight": 60},
...     },
... ]
>>> series = pd.Series(data, index=pd.Index(["a", "b", "c"]))
>>> pd.json_normalize(series)
   id      name  fitness.height  fitness.weight
a  1.0  Cole Volk             130             60
b  NaN  Mark Reg             130             60
c  2.0  Faye Raker           130             60
```

```
>>> data = [
...     {
...         "state": "Florida",
...         "shortname": "FL",
...     },
... ]
```

```

...     "info": {"governor": "Rick Scott"},
...     "counties": [
...         {"name": "Dade", "population": 12345},
...         {"name": "Broward", "population": 40000},
...         {"name": "Palm Beach", "population": 60000},
...     ],
... },
... {
...     "state": "Ohio",
...     "shortname": "OH",
...     "info": {"governor": "John Kasich"},
...     "counties": [
...         {"name": "Summit", "population": 1234},
...         {"name": "Cuyahoga", "population": 1337},
...     ],
... },
... ]
>>> result = pd.json_normalize(
...     data, "counties", ["state", "shortname", ["info", "governor"]]
... )
>>> result
   name  population  state shortname info.governor
0   Dade      12345  Florida      FL    Rick Scott
1  Broward     40000  Florida      FL    Rick Scott
2 Palm Beach     60000  Florida      FL    Rick Scott
3   Summit      1234   Ohio      OH    John Kasich
4  Cuyahoga     1337   Ohio      OH    John Kasich

>>> data = {"A": [1, 2]}
>>> pd.json_normalize(data, "A", record_prefix="Prefix.")
   Prefix.0
0         1
1         2

```

Returns normalized data with columns prefixed with the given string.

`lreshape(data: DataFrame, groups: dict, dropna: bool = True) -> DataFrame`
 Reshape wide-format data to long. Generalized inverse of `DataFrame.pivot`.

Accepts a dictionary, ``groups``, in which each key is a new column name and each value is a list of old column names that will be "melted" under the new column name as part of the reshape.

Parameters

`data` : DataFrame
 The wide-format DataFrame.
`groups` : dict
 {new_name : list_of_columns}.
`dropna` : bool, default True
 Do not include columns whose entries are all NaN.

Returns

DataFrame
 Reshaped DataFrame.

See Also

`melt` : Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.
`pivot` : Create a spreadsheet-style pivot table as a DataFrame.
`DataFrame.pivot` : Pivot without aggregation that can handle non-numeric data.
`DataFrame.pivot_table` : Generalization of pivot that can handle duplicate values for one index/column pair.
`DataFrame.unstack` : Pivot based on the index values instead of a column.
`wide_to_long` : Wide panel to long format. Less flexible but more user-friendly than melt.

Examples

```

>>> data = pd.DataFrame(
...     {
...         "hr1": [514, 573],
...         "hr2": [545, 526],
...     }
... )

```

```

...         "team": ["Red Sox", "Yankees"],
...         "year1": [2007, 2007],
...         "year2": [2008, 2008],
...     }
... )
>>> data
   hr1  hr2   team  year1  year2
0  514  545  Red Sox   2007   2008
1  573  526  Yankees   2007   2008

>>> pd.lreshape(data, {"year": ["year1", "year2"], "hr": ["hr1", "hr2"]})
   team  year  hr
0  Red Sox  2007  514
1  Yankees  2007  573
2  Red Sox  2008  545
3  Yankees  2008  526

```

`melt()` -> DataFrame

Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.

This function is useful to reshape a DataFrame into a format where one or more columns are identifier variables (`'id_vars'`), while all other columns are considered measured variables (`'value_vars'`), and are "unpivoted" to the row axis, leaving just two non-identifier columns, `'variable'` and `'value'`.

Parameters

- `frame` : DataFrame
The DataFrame to unpivot.
- `id_vars` : scalar, tuple, list, or ndarray, optional
Column(s) to use as identifier variables.
- `value_vars` : scalar, tuple, list, or ndarray, optional
Column(s) to unpivot. If not specified, uses all columns that are not set as `'id_vars'`.
- `var_name` : scalar, tuple, list, or ndarray, optional
Name to use for the `'variable'` column. If None it uses `'frame.columns.name'` or `'variable'`. Must be a scalar if columns are a MultiIndex.
- `value_name` : scalar, default `'value'`
Name to use for the `'value'` column, can't be an existing column label.
- `col_level` : scalar, optional
If columns are a MultiIndex then use this level to melt.
- `ignore_index` : bool, default True
If True, original index is ignored. If False, the original index is retained. Index labels will be repeated as necessary.

Returns

- DataFrame
Unpivoted DataFrame.

See Also

- `DataFrame.melt` : Identical method.
- `pivot_table` : Create a spreadsheet-style pivot table as a DataFrame.
- `DataFrame.pivot` : Return reshaped DataFrame organized by given index / column values.
- `DataFrame.explode` : Explode a DataFrame from list-like columns to long format.

Notes

- Reference :ref:`the user guide <reshaping.melt>` for more examples.

Examples

```

>>> df = pd.DataFrame(
...     {

```

```

...     "A": {0: "a", 1: "b", 2: "c"},
...     "B": {0: 1, 1: 3, 2: 5},
...     "C": {0: 2, 1: 4, 2: 6},
... }
... )
>>> df
  A B C
0 a 1 2
1 b 3 4
2 c 5 6

>>> pd.melt(df, id_vars=["A"], value_vars=["B"])
  A variable value
0 a         B     1
1 b         B     3
2 c         B     5

```

```

>>> pd.melt(df, id_vars=["A"], value_vars=["B", "C"])
  A variable value
0 a         B     1
1 b         B     3
2 c         B     5
3 a         C     2
4 b         C     4
5 c         C     6

```

The names of 'variable' and 'value' columns can be customized:

```

>>> pd.melt(
...     df,
...     id_vars=["A"],
...     value_vars=["B"],
...     var_name="myVarname",
...     value_name="myValname",
... )
  A myVarname myValname
0 a         B         1
1 b         B         3
2 c         B         5

```

Original index values can be kept around:

```

>>> pd.melt(df, id_vars=["A"], value_vars=["B", "C"], ignore_index=False)
  A variable value
0 a         B     1
1 b         B     3
2 c         B     5
0 a         C     2
1 b         C     4
2 c         C     6

```

If you have multi-index columns:

```

>>> df.columns = [list("ABC"), list("DEF")]
>>> df
  A B C
D E F
0 a 1 2
1 b 3 4
2 c 5 6

>>> pd.melt(df, col_level=0, id_vars=["A"], value_vars=["B"])
  A variable value
0 a         B     1
1 b         B     3
2 c         B     5

>>> pd.melt(df, id_vars=[("A", "D")], value_vars=[("B", "E")])
(A, D) variable_0 variable_1 value
0 a         B         E     1
1 b         B         E     3
2 c         B         E     5

```

```

merge(
    left: DataFrame | Series,
    right: DataFrame | Series,
    how: MergeHow = 'inner',

```

```

on: IndexLabel | AnyArrayLike | None = None,
left_on: IndexLabel | AnyArrayLike | None = None,
right_on: IndexLabel | AnyArrayLike | None = None,
left_index: bool = False,
right_index: bool = False,
sort: bool = False,
suffixes: Suffixes = ('_x', '_y'),
copy: bool | lib.NoDefault = <no_default>,
indicator: str | bool = False,
validate: str | None = None
) -> DataFrame

```

Merge DataFrame or named Series objects with a database-style join.

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

.. warning::

If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters

```

-----
left : DataFrame or named Series
    First pandas object to merge.
right : DataFrame or named Series
    Second pandas object to merge.
how : {'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti'},
    default 'inner'
    Type of merge to be performed.

    * left: use only keys from left frame, similar to a SQL left outer join;
      preserve key order.
    * right: use only keys from right frame, similar to a SQL right outer join;
      preserve key order.
    * outer: use union of keys from both frames, similar to a SQL full outer
      join; sort keys lexicographically.
    * inner: use intersection of keys from both frames, similar to a SQL inner
      join; preserve the order of the left keys.
    * cross: creates the cartesian product from both frames, preserves the order
      of the left keys.
    * left_anti: use only keys from left frame that are not in right frame, similar
      to SQL left anti join; preserve key order.
    * right_anti: use only keys from right frame that are not in left frame, similar
      to SQL right anti join; preserve key order.
on : Hashable or a sequence of the previous
    Column or index level names to join on. These must be found in both
    DataFrames. If `on` is None and not merging on indexes then this defaults
    to the intersection of the columns in both DataFrames.
left_on : Hashable or a sequence of the previous, or array-like
    Column or index level names to join on in the left DataFrame. Can also
    be an array or list of arrays of the length of the left DataFrame.
    These arrays are treated as if they are columns.
right_on : Hashable or a sequence of the previous, or array-like
    Column or index level names to join on in the right DataFrame. Can also
    be an array or list of arrays of the length of the right DataFrame.
    These arrays are treated as if they are columns.
left_index : bool, default False
    Use the index from the left DataFrame as the join key(s). If it is a
    MultiIndex, the number of keys in the other DataFrame (either the index
    or a number of columns) must match the number of levels.
right_index : bool, default False
    Use the index from the right DataFrame as the join key. Same caveats as
    left_index.
sort : bool, default False
    Sort the join keys lexicographically in the result DataFrame. If False,
    the order of the join keys depends on the join type (how keyword).
suffixes : list-like, default is ("_x", "_y")
    A length-2 sequence where each element is optionally a string
    indicating the suffix to add to overlapping column names in
    `left` and `right` respectively. Pass a value of `None` instead

```

of a string to indicate that the column name from `left` or `right` should be left as-is, with no suffix. At least one of the values must not be None.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`\_\_` for more details.

indicator : bool or str, default False
If True, adds a column to the output DataFrame called "_merge" with information on the source of each row. The column can be given a different name by providing a string argument. The column will have a Categorical type with the value of "left_only" for observations whose merge key only appears in the left DataFrame, "right_only" for observations whose merge key only appears in the right DataFrame, and "both" if the observation's merge key is found in both DataFrames.

validate : str, optional
If specified, checks if merge is of specified type.

- * "one_to_one" or "1:1": check if merge keys are unique in both left and right datasets.
- * "one_to_many" or "1:m": check if merge keys are unique in left dataset.
- * "many_to_one" or "m:1": check if merge keys are unique in right dataset.
- * "many_to_many" or "m:m": allowed, but does not result in checks.

Returns

DataFrame

A DataFrame of the two merged objects.

See Also

merge_ordered : Merge with optional filling/interpolation.
merge_asof : Merge on nearest keys.
DataFrame.join : Similar method using indices.

Examples

```
>>> df1 = pd.DataFrame(
...     {"lkey": ["foo", "bar", "baz", "foo"], "value": [1, 2, 3, 5]}
... )
>>> df2 = pd.DataFrame(
...     {"rkey": ["foo", "bar", "baz", "foo"], "value": [5, 6, 7, 8]}
... )
>>> df1
   lkey value
0  foo     1
1  bar     2
2  baz     3
3  foo     5
>>> df2
   rkey value
0  foo     5
1  bar     6
2  baz     7
3  foo     8
```

Merge df1 and df2 on the lkey and rkey columns. The value columns have the default suffixes, _x and _y, appended.

```
>>> df1.merge(df2, left_on="lkey", right_on="rkey")
   lkey  value_x rkey  value_y
0  foo         1  foo         5
1  foo         1  foo         8
2  bar         2  bar         6
3  baz         3  baz         7
```

4	foo	5	foo	5
5	foo	5	foo	8

Merge DataFrames df1 and df2 with specified left and right suffixes appended to any overlapping columns.

```
>>> df1.merge(df2, left_on="lkey", right_on="rkey", suffixes=("_left", "_right"))
```

	lkey	value_left	rkey	value_right
0	foo	1	foo	5
1	foo	1	foo	8
2	bar	2	bar	6
3	baz	3	baz	7
4	foo	5	foo	5
5	foo	5	foo	8

Merge DataFrames df1 and df2, but raise an exception if the DataFrames have any overlapping columns.

```
>>> df1.merge(df2, left_on="lkey", right_on="rkey", suffixes=(False, False))
```

Traceback (most recent call last):

```
...
ValueError: columns overlap but no suffix specified:
Index(['value'], dtype='str')
```

```
>>> df1 = pd.DataFrame({"a": ["foo", "bar"], "b": [1, 2]})
>>> df2 = pd.DataFrame({"a": ["foo", "baz"], "c": [3, 4]})
```

```
>>> df1
   a  b
0  foo  1
1  bar  2
>>> df2
   a  c
0  foo  3
1  baz  4
```

```
>>> df1.merge(df2, how="inner", on="a")
   a  b  c
0  foo  1  3
```

```
>>> df1.merge(df2, how="left", on="a")
   a  b  c
0  foo  1  3.0
1  bar  2  NaN
```

```
>>> df1 = pd.DataFrame({"left": ["foo", "bar"]})
>>> df2 = pd.DataFrame({"right": [7, 8]})
>>> df1
  left
0  foo
1  bar
>>> df2
  right
0     7
1     8
```

```
>>> df1.merge(df2, how="cross")
  left  right
0  foo     7
1  foo     8
2  bar     7
3  bar     8
```

```
merge_asof(
    left: DataFrame | Series,
    right: DataFrame | Series,
    on: IndexLabel | None = None,
    left_on: IndexLabel | None = None,
    right_on: IndexLabel | None = None,
    left_index: bool = False,
    right_index: bool = False,
    by=None,
    left_by=None,
    right_by=None,
    suffixes: Suffixes = ('_x', '_y'),
    tolerance: int | datetime.timedelta | None = None,
    allow_exact_matches: bool = True,
    direction: str = 'backward'
```


) -> DataFrame

Perform a merge by key distance.

This is similar to a left-join except that we match on nearest key rather than equal keys. Both DataFrames must be first sorted by the merge key in ascending order before calling this function. Sorting by any additional 'by' grouping columns is not required.

For each row in the left DataFrame:

- A "backward" search selects the last row in the right DataFrame whose 'on' key is less than or equal to the left's key.
- A "forward" search selects the first row in the right DataFrame whose 'on' key is greater than or equal to the left's key.
- A "nearest" search selects the row in the right DataFrame whose 'on' key is closest in absolute distance to the left's key.

Optionally match on equivalent keys with 'by' before searching with 'on'.

Parameters

left : DataFrame or named Series

First pandas object to merge.

right : DataFrame or named Series

Second pandas object to merge.

on : label

Field name to join on. Must be found in both DataFrames.

The data MUST be in ascending order. Furthermore this must be a numeric column, such as datetimelike, integer, or float. ``on`` or ``left\_on`` / ``right\_on`` must be given.

left_on : label

Field name to join on in left DataFrame. If specified, sort the left DataFrame by this column in ascending order before merging.

right_on : label

Field name to join on in right DataFrame. If specified, sort the right DataFrame by this column in ascending order before merging.

left_index : bool

Use the index of the left DataFrame as the join key.

right_index : bool

Use the index of the right DataFrame as the join key.

by : column name or list of column names

Match on these columns before performing merge operation. It is not required to sort by these columns.

left_by : column name

Field names to match on in the left DataFrame.

right_by : column name

Field names to match on in the right DataFrame.

suffixes : 2-length sequence (tuple, list, ...)

Suffix to apply to overlapping column names in the left and right side, respectively.

tolerance : int or timedelta, optional, default None

Select asof tolerance within this range; must be compatible with the merge index.

allow_exact_matches : bool, default True

- If True, allow matching with the same 'on' value (i.e. less-than-or-equal-to / greater-than-or-equal-to)
- If False, don't match the same 'on' value (i.e., strictly less-than / strictly greater-than).

direction : 'backward' (default), 'forward', or 'nearest'

Whether to search for prior, subsequent, or closest matches.

Returns

DataFrame

A DataFrame of the two merged objects, containing all rows from the left DataFrame and the nearest matches from the right DataFrame.

See Also

merge : Merge with a database-style join.

merge_ordered : Merge with optional filling/interpolation.

Examples

```

-----
>>> left = pd.DataFrame({"a": [1, 5, 10], "left_val": ["a", "b", "c"]})
>>> left
   a left_val
0  1         a
1  5         b
2 10         c

>>> right = pd.DataFrame({"a": [1, 2, 3, 6, 7], "right_val": [1, 2, 3, 6, 7]})
>>> right
   a right_val
0  1          1
1  2          2
2  3          3
3  6          6
4  7          7

>>> pd.merge_asof(left, right, on="a")
   a left_val right_val
0  1         a         1
1  5         b         3
2 10         c         7

>>> pd.merge_asof(left, right, on="a", allow_exact_matches=False)
   a left_val right_val
0  1         a        NaN
1  5         b         3.0
2 10         c         7.0

>>> pd.merge_asof(left, right, on="a", direction="forward")
   a left_val right_val
0  1         a         1.0
1  5         b         6.0
2 10         c         NaN

>>> pd.merge_asof(left, right, on="a", direction="nearest")
   a left_val right_val
0  1         a         1
1  5         b         6
2 10         c         7

```

We can use indexed DataFrames as well.

```

>>> left = pd.DataFrame({"left_val": ["a", "b", "c"], index=[1, 5, 10]})
>>> left
   left_val
1         a
5         b
10        c

>>> right = pd.DataFrame({"right_val": [1, 2, 3, 6, 7], index=[1, 2, 3, 6, 7]})
>>> right
   right_val
1          1
2          2
3          3
6          6
7          7

>>> pd.merge_asof(left, right, left_index=True, right_index=True)
   left_val right_val
1         a         1
5         b         3
10        c         7

```

Here is a real-world times-series example

```

>>> quotes = pd.DataFrame(
...     {
...         "time": [
...             pd.Timestamp("2016-05-25 13:30:00.023"),
...             pd.Timestamp("2016-05-25 13:30:00.023"),
...             pd.Timestamp("2016-05-25 13:30:00.030"),
...             pd.Timestamp("2016-05-25 13:30:00.041"),
...             pd.Timestamp("2016-05-25 13:30:00.048"),
...             pd.Timestamp("2016-05-25 13:30:00.049"),
...             pd.Timestamp("2016-05-25 13:30:00.072"),

```

```

...         pd.Timestamp("2016-05-25 13:30:00.075"),
...     ],
...     "ticker": [
...         "GOOG",
...         "MSFT",
...         "MSFT",
...         "MSFT",
...         "GOOG",
...         "AAPL",
...         "GOOG",
...         "MSFT",
...     ],
...     "bid": [720.50, 51.95, 51.97, 51.99, 720.50, 97.99, 720.50, 52.01],
...     "ask": [720.93, 51.96, 51.98, 52.00, 720.93, 98.01, 720.88, 52.03],
... }
... )
>>> quotes

```

		time	ticker	bid	ask
0	2016-05-25	13:30:00.023	GOOG	720.50	720.93
1	2016-05-25	13:30:00.023	MSFT	51.95	51.96
2	2016-05-25	13:30:00.030	MSFT	51.97	51.98
3	2016-05-25	13:30:00.041	MSFT	51.99	52.00
4	2016-05-25	13:30:00.048	GOOG	720.50	720.93
5	2016-05-25	13:30:00.049	AAPL	97.99	98.01
6	2016-05-25	13:30:00.072	GOOG	720.50	720.88
7	2016-05-25	13:30:00.075	MSFT	52.01	52.03

```

>>> trades = pd.DataFrame(
...     {
...         "time": [
...             pd.Timestamp("2016-05-25 13:30:00.023"),
...             pd.Timestamp("2016-05-25 13:30:00.038"),
...             pd.Timestamp("2016-05-25 13:30:00.048"),
...             pd.Timestamp("2016-05-25 13:30:00.048"),
...             pd.Timestamp("2016-05-25 13:30:00.048"),
...         ],
...         "ticker": ["MSFT", "MSFT", "GOOG", "GOOG", "AAPL"],
...         "price": [51.95, 51.95, 720.77, 720.92, 98.0],
...         "quantity": [75, 155, 100, 100, 100],
...     }
... )
>>> trades

```

		time	ticker	price	quantity
0	2016-05-25	13:30:00.023	MSFT	51.95	75
1	2016-05-25	13:30:00.038	MSFT	51.95	155
2	2016-05-25	13:30:00.048	GOOG	720.77	100
3	2016-05-25	13:30:00.048	GOOG	720.92	100
4	2016-05-25	13:30:00.048	AAPL	98.00	100

By default we are taking the asof of the quotes

```

>>> pd.merge_asof(trades, quotes, on="time", by="ticker")

```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	51.95	51.96
1	2016-05-25	13:30:00.038	MSFT	51.95	155	51.97	51.98
2	2016-05-25	13:30:00.048	GOOG	720.77	100	720.50	720.93
3	2016-05-25	13:30:00.048	GOOG	720.92	100	720.50	720.93
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

We only asof within 2ms between the quote time and the trade time

```

>>> pd.merge_asof(
...     trades, quotes, on="time", by="ticker", tolerance=pd.Timedelta("2ms")
... )

```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	51.95	51.96
1	2016-05-25	13:30:00.038	MSFT	51.95	155	NaN	NaN
2	2016-05-25	13:30:00.048	GOOG	720.77	100	720.50	720.93
3	2016-05-25	13:30:00.048	GOOG	720.92	100	720.50	720.93
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

We only asof within 10ms between the quote time and the trade time and we exclude exact matches on time. However *prior* data will propagate forward

```

>>> pd.merge_asof(
...     trades,

```

```

...     quotes,
...     on="time",
...     by="ticker",
...     tolerance=pd.Timedelta("10ms"),
...     allow_exact_matches=False,
... )

```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	NaN	NaN
1	2016-05-25	13:30:00.038	MSFT	51.95	155	51.97	51.98
2	2016-05-25	13:30:00.048	GOOG	720.77	100	NaN	NaN
3	2016-05-25	13:30:00.048	GOOG	720.92	100	NaN	NaN
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

```

merge_ordered(
    left: DataFrame | Series,
    right: DataFrame | Series,
    on: IndexLabel | None = None,
    left_on: IndexLabel | None = None,
    right_on: IndexLabel | None = None,
    left_by=None,
    right_by=None,
    fill_method: str | None = None,
    suffixes: Suffixes = ('_x', '_y'),
    how: JoinHow = 'outer'
) -> DataFrame

```

Perform a merge for ordered data with optional filling/interpolation.

Designed for ordered data like time series data. Optionally perform group-wise merge (see examples).

Parameters

left : DataFrame or named Series
First pandas object to merge.

right : DataFrame or named Series
Second pandas object to merge.

on : Hashable or a sequence of the previous
Field names to join on. Must be found in both DataFrames.

left_on : Hashable or a sequence of the previous, or array-like
Field names to join on in left DataFrame. Can be a vector or list of vectors of the length of the DataFrame to use a particular vector as the join key instead of columns.

right_on : Hashable or a sequence of the previous, or array-like
Field names to join on in right DataFrame or vector/list of vectors per left_on docs.

left_by : column name or list of column names
Group left DataFrame by group columns and merge piece by piece with right DataFrame. Must be None if either left or right are a Series.

right_by : column name or list of column names
Group right DataFrame by group columns and merge piece by piece with left DataFrame. Must be None if either left or right are a Series.

fill_method : {'ffill', None}, default None
Interpolation method for data.

suffixes : list-like, default is ("_x", "_y")
A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in `left` and `right` respectively. Pass a value of `None` instead of a string to indicate that the column name from `left` or `right` should be left as-is, with no suffix. At least one of the values must not be None.

how : {'left', 'right', 'outer', 'inner'}, default 'outer'

- * left: use only keys from left frame (SQL: left outer join)
- * right: use only keys from right frame (SQL: right outer join)
- * outer: use union of keys from both frames (SQL: full outer join)
- * inner: use intersection of keys from both frames (SQL: inner join).

Returns

DataFrame

The merged DataFrame output type will be the same as 'left', if it is a subclass of DataFrame.

See Also

merge : Merge with a database-style join.
merge_asof : Merge on nearest keys.

Examples

```
-----
>>> from pandas import merge_ordered
>>> df1 = pd.DataFrame(
...     {
...         "key": ["a", "c", "e", "a", "c", "e"],
...         "lvalue": [1, 2, 3, 1, 2, 3],
...         "group": ["a", "a", "a", "b", "b", "b"],
...     }
... )
>>> df1
   key  lvalue group
0    a        1    a
1    c        2    a
2    e        3    a
3    a        1    b
4    c        2    b
5    e        3    b

>>> df2 = pd.DataFrame({"key": ["b", "c", "d"], "rvalue": [1, 2, 3]})
>>> df2
   key  rvalue
0    b        1
1    c        2
2    d        3
```

```
>>> merge_ordered(df1, df2, fill_method="ffill", left_by="group")
   key  lvalue group  rvalue
0    a        1    a     NaN
1    b        1    a     1.0
2    c        2    a     2.0
3    d        2    a     3.0
4    e        3    a     3.0
5    a        1    b     NaN
6    b        1    b     1.0
7    c        2    b     2.0
8    d        2    b     3.0
9    e        3    b     3.0
```

`notna(obj: object) -> bool | npt.NDArray[np.bool_] | NDFrame`
Detect non-missing values for an array-like object.

This function takes a scalar or array-like object and indicates whether values are valid (not missing, which is ``NaN`` in numeric arrays, ``None`` or ``NaN`` in object arrays, ``NaT`` in datetimelike).

Parameters

`obj` : array-like or object value
Object to check for *not* null or *non*-missing values.

Returns

`bool` or array-like of `bool`
For scalar input, returns a scalar boolean.
For array input, returns an array of boolean indicating whether each corresponding element is valid.

See Also

`isna` : Boolean inverse of `pandas.notna`.
`Series.notna` : Detect valid values in a `Series`.
`DataFrame.notna` : Detect valid values in a `DataFrame`.
`Index.notna` : Detect valid values in an `Index`.

Examples

Scalar arguments (including strings) result in a scalar boolean.

```
>>> pd.notna("dog")
True

>>> pd.notna(pd.NA)
False

>>> pd.notna(np.nan)
False
```

ndarrays result in an ndarray of booleans.

```
>>> array = np.array([[1, np.nan, 3], [4, 5, np.nan]])
>>> array
array([[ 1., nan,  3.],
       [ 4.,  5., nan]])
>>> pd.notna(array)
array([[ True, False,  True],
       [ True,  True, False]])
```

For indexes, an ndarray of booleans is returned.

```
>>> index = pd.DatetimeIndex(["2017-07-05", "2017-07-06", None, "2017-07-08"])
>>> index
DatetimeIndex(['2017-07-05', '2017-07-06', 'NaT', '2017-07-08'],
              dtype='datetime64[us]', freq=None)
>>> pd.notna(index)
array([ True,  True, False,  True])
```

For Series and DataFrame, the same type is returned, containing booleans.

```
>>> df = pd.DataFrame([["ant", "bee", "cat"], ["dog", None, "fly"]])
>>> df
   0    1    2
0 ant bee cat
1 dog NaN fly
>>> pd.notna(df)
   0    1    2
0 True True True
1 True False True

>>> pd.notna(df[1])
0    True
1   False
Name: 1, dtype: bool
```

```
notnull = notna(obj: object) -> bool | npt.NDArray[np.bool_] | NDFrame
Detect non-missing values for an array-like object.
```

This function takes a scalar or array-like object and indicates whether values are valid (not missing, which is ``NaN`` in numeric arrays, ``None`` or ``NaN`` in object arrays, ``NaT`` in datetimelike).

Parameters

obj : array-like or object value
Object to check for *not* null or *non*-missing values.

Returns

bool or array-like of bool
For scalar input, returns a scalar boolean.
For array input, returns an array of boolean indicating whether each corresponding element is valid.

See Also

isna : Boolean inverse of pandas.notna.
Series.notna : Detect valid values in a Series.
DataFrame.notna : Detect valid values in a DataFrame.
Index.notna : Detect valid values in an Index.

Examples

Scalar arguments (including strings) result in a scalar boolean.

```
>>> pd.notna("dog")
True

>>> pd.notna(pd.NA)
False

>>> pd.notna(np.nan)
False
```

ndarrays result in an ndarray of booleans.

```
>>> array = np.array([[1, np.nan, 3], [4, 5, np.nan]])
```

```
>>> array
array([[ 1., nan,  3.],
       [ 4.,  5., nan]])
>>> pd.notna(array)
array([[ True, False,  True],
       [ True,  True, False]])
```

For indexes, an ndarray of booleans is returned.

```
>>> index = pd.DatetimeIndex(["2017-07-05", "2017-07-06", None, "2017-07-08"])
>>> index
DatetimeIndex(['2017-07-05', '2017-07-06', 'NaT', '2017-07-08'],
              dtype='datetime64[us]', freq=None)
>>> pd.notna(index)
array([ True,  True, False,  True])
```

For Series and DataFrame, the same type is returned, containing booleans.

```
>>> df = pd.DataFrame([["ant", "bee", "cat"], ["dog", None, "fly"]])
>>> df
   0    1    2
0 ant bee cat
1 dog NaN fly
>>> pd.notna(df)
   0    1    2
0 True True True
1 True False True

>>> pd.notna(df[1])
0     True
1    False
Name: 1, dtype: bool
```

`option_context(*args)` -> Generator[None]

Context manager to temporarily set options in a ``with`` statement.

This method allows users to set one or more pandas options temporarily within a controlled block. The previous options' values are restored once the block is exited. This is useful when making temporary adjustments to pandas' behavior without affecting the global state.

Parameters

`*args` : str | object | dict
 An even amount of arguments provided in pairs which will be interpreted as (pattern, value) pairs. Alternatively, a single dictionary of {pattern: value} may be provided.

Returns

 None
 No return value.

Yields

 None
 No yield value.

See Also

`get_option` : Retrieve the value of the specified option.
`set_option` : Set the value of the specified option.
`reset_option` : Reset one or more options to their default value.
`describe_option` : Print the description for one or more registered options.

Notes

 For all available options, please view the :ref:`User Guide <options.available>` or use ``pandas.describe\_option()``.

Examples

```
>>> from pandas import option_context
>>> with option_context("display.max_rows", 10, "display.max_columns", 5):
...     pass
>>> with option_context({"display.max_rows": 10, "display.max_columns": 5}):
...     pass
```

```

period_range(
    start=None,
    end=None,
    periods: int | None = None,
    freq=None,
    name: Hashable | None = None
) -> PeriodIndex
    Return a fixed frequency PeriodIndex.

    The day (calendar) is the default frequency.

Parameters
-----
start : str, datetime, date, pandas.Timestamp, or period-like, default None
    Left bound for generating periods.
end : str, datetime, date, pandas.Timestamp, or period-like, default None
    Right bound for generating periods.
periods : int, default None
    Number of periods to generate.
freq : str or DateOffset, optional
    Frequency alias. By default the freq is taken from `start` or `end`
    if those are Period objects. Otherwise, the default is ``"D"`` for
    daily frequency.
name : str, default None
    Name of the resulting PeriodIndex.

Returns
-----
PeriodIndex
    A PeriodIndex of fixed frequency periods.

See Also
-----
date_range : Returns a fixed frequency DatetimeIndex.
Period : Represents a period of time.
PeriodIndex : Immutable ndarray holding ordinal values indicating regular periods
    in time.

Notes
-----
Of the three parameters: ``start``, ``end``, and ``periods``, exactly two
must be specified.

To learn more about the frequency strings, please see
:ref:`this link<timeseries.offset_aliases>`.

Examples
-----
>>> pd.period_range(start="2017-01-01", end="2018-01-01", freq="M")
PeriodIndex(['2017-01', '2017-02', '2017-03', '2017-04', '2017-05', '2017-06',
            '2017-07', '2017-08', '2017-09', '2017-10', '2017-11', '2017-12',
            '2018-01'],
            dtype='period[M]')

If ``start`` or ``end`` are ``Period`` objects, they will be used as anchor
endpoints for a ``PeriodIndex`` with frequency matching that of the
``period_range`` constructor.

>>> pd.period_range(
...     start=pd.Period("2017Q1", freq="Q"),
...     end=pd.Period("2017Q2", freq="Q"),
...     freq="M",
... )
PeriodIndex(['2017-03', '2017-04', '2017-05', '2017-06'],
            dtype='period[M]')

pivot(
    data: DataFrame,
    *,
    columns: IndexLabel,
    index: IndexLabel | lib.NoDefault = <no_default>,
    values: IndexLabel | lib.NoDefault = <no_default>
) -> DataFrame
    Return reshaped DataFrame organized by given index / column values.

    Reshape data (produce a "pivot" table) based on column values. Uses
    unique values from specified `index` / `columns` to form axes of the

```


resulting DataFrame. This function does not support data aggregation, multiple values will result in a MultiIndex in the columns. See the :ref:`User Guide <reshaping>` for more on reshaping.

Parameters

data : DataFrame
 Input pandas DataFrame object.
columns : Hashable or a sequence of the previous
 Column to use to make new frame's columns.
index : Hashable or a sequence of the previous, optional
 Column to use to make new frame's index. If not given, uses existing index.
values : Hashable or a sequence of the previous, optional
 Column(s) to use for populating new frame's values. If not specified, all remaining columns will be used and the result will have hierarchically indexed columns.

Returns

DataFrame
 Returns reshaped DataFrame.

Raises

ValueError:
 When there are any `index`, `columns` combinations with multiple values. `DataFrame.pivot\_table` when you need to aggregate.

See Also

DataFrame.pivot_table : Generalization of pivot that can handle duplicate values for one index/column pair.
DataFrame.unstack : Pivot based on the index values instead of a column.
wide_to_long : Wide panel to long format. Less flexible but more user-friendly than melt.

Notes

For finer-tuned control, see hierarchical indexing documentation along with the related stack/unstack methods.

Reference :ref:`the user guide <reshaping.pivot>` for more examples.

Examples

>>> df = pd.DataFrame(
... {
... "foo": ["one", "one", "one", "two", "two", "two"],
... "bar": ["A", "B", "C", "A", "B", "C"],
... "baz": [1, 2, 3, 4, 5, 6],
... "zoo": ["x", "y", "z", "q", "w", "t"],
... }
...)
>>> df
 foo bar baz zoo
0 one A 1 x
1 one B 2 y
2 one C 3 z
3 two A 4 q
4 two B 5 w
5 two C 6 t

>>> df.pivot(index="foo", columns="bar", values="baz")
bar A B C
foo
one 1 2 3
two 4 5 6

>>> df.pivot(index="foo", columns="bar")["baz"]
bar A B C
foo
one 1 2 3
two 4 5 6

>>> df.pivot(index="foo", columns="bar", values=["baz", "zoo"])
 baz zoo
foo
one 1 2 3
two 4 5 6

```

bar   A  B  C   A  B  C
foo
one   1  2  3   x  y  z
two   4  5  6   q  w  t

```

You could also assign a list of column names or a list of index names.

```

>>> df = pd.DataFrame(
...     {
...         "lev1": [1, 1, 1, 2, 2, 2],
...         "lev2": [1, 1, 2, 1, 1, 2],
...         "lev3": [1, 2, 1, 2, 1, 2],
...         "lev4": [1, 2, 3, 4, 5, 6],
...         "values": [0, 1, 2, 3, 4, 5],
...     }
... )
>>> df

```

```

      lev1 lev2 lev3 lev4 values
0      1     1     1     1      0
1      1     1     2     2      1
2      1     2     1     3      2
3      2     1     2     4      3
4      2     1     1     5      4
5      2     2     2     6      5

```

```

>>> df.pivot(index="lev1", columns=["lev2", "lev3"], values="values")

```

```

lev2      1      2
lev3      1      2
lev1
1      0.0  1.0  2.0  NaN
2      4.0  3.0  NaN  5.0

```

```

>>> df.pivot(index=["lev1", "lev2"], columns=["lev3"], values="values")

```

```

      lev3      1      2
lev1  lev2
1      1      0.0  1.0
      2      2.0  NaN
2      1      4.0  3.0
      2      NaN  5.0

```

A `ValueError` is raised if there are any duplicates.

```

>>> df = pd.DataFrame(
...     {
...         "foo": ["one", "one", "two", "two"],
...         "bar": ["A", "A", "B", "C"],
...         "baz": [1, 2, 3, 4],
...     }
... )
>>> df

```

```

      foo bar  baz
0    one  A     1
1    one  A     2
2    two  B     3
3    two  C     4

```

Notice that the first two rows are the same for our ``index`` and ``columns`` arguments.

```

>>> df.pivot(index="foo", columns="bar", values="baz")
Traceback (most recent call last):

```

```

...
ValueError: Index contains duplicate entries, cannot reshape

```

```

pivot_table(
    data: DataFrame,
    values=None,
    index=None,
    columns=None,
    aggfunc: AggFuncType = 'mean',
    fill_value=None,
    margins: bool = False,
    dropna: bool = True,
    margins_name: Hashable = 'All',
    observed: bool = True,
    sort: bool = True,
    **kwargs
)

```

) -> DataFrame

Create a spreadsheet-style pivot table as a DataFrame.

The levels in the pivot table will be stored in MultiIndex objects (hierarchical indexes) on the index and columns of the result DataFrame.

Parameters

data : DataFrame

Input pandas DataFrame object.

values : list-like or scalar, optional

Column or columns to aggregate.

index : column, Grouper, array, or sequence of the previous

Keys to group by on the pivot table index. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

columns : column, Grouper, array, or sequence of the previous

Keys to group by on the pivot table column. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

aggfunc : function, list of functions, dict, default "mean"

If a list of functions is passed, the resulting pivot table will have hierarchical columns whose top level are the function names (inferred from the function objects themselves).

If a dict is passed, the key is column to aggregate and the value is function or list of functions. If ``margins=True``, aggfunc will be used to calculate the partial aggregates.

fill_value : scalar, default None

Value to replace missing values with (in the resulting pivot table, after aggregation).

margins : bool, default False

If ``margins=True``, special ``All`` columns and rows will be added with partial group aggregates across the categories on the rows and columns.

dropna : bool, default True

Do not include columns whose entries are all NaN. If True,

* rows with an NA value in any column will be omitted before computing margins,
* index/column keys containing NA values will be dropped (see ``dropna`` parameter in :meth:`DataFrame.groupby`).

margins_name : str, default 'All'

Name of the row / column that will contain the totals when margins is True.

observed : bool, default False

This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical groupers.

.. versionchanged:: 3.0.0

The default value is now ``True``.

sort : bool, default True

Specifies if the result should be sorted.

**kwargs : dict

Optional keyword arguments to pass to ``aggfunc``.

.. versionadded:: 3.0.0

Returns

DataFrame

An Excel style pivot table.

See Also

DataFrame.pivot : Pivot without aggregation that can handle non-numeric data.

DataFrame.melt: Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.

wide_to_long : Wide panel to long format. Less flexible but more user-friendly than melt.

Notes

Reference :ref:`the user guide <reshaping.pivot>` for more examples.

Examples

```
-----  
>>> df = pd.DataFrame(  
...     {  
...         "A": ["foo", "foo", "foo", "foo", "foo", "bar", "bar", "bar", "bar"],  
...         "B": ["one", "one", "one", "two", "two", "one", "one", "two", "two"],  
...         "C": [  
...             "small",  
...             "large",  
...             "large",  
...             "small",  
...             "small",  
...             "large",  
...             "small",  
...             "small",  
...             "large",  
...         ],  
...         "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],  
...         "E": [2, 4, 5, 5, 6, 6, 8, 9, 9],  
...     }  
... )  
>>> df  
   A  B  C  D  E  
0  foo one small 1 2  
1  foo one large 2 4  
2  foo one large 2 5  
3  foo two small 3 5  
4  foo two small 3 6  
5  bar one large 4 6  
6  bar one small 5 8  
7  bar two small 6 9  
8  bar two large 7 9
```

This first example aggregates values by taking the sum.

```
>>> table = pd.pivot_table(  
...     df, values="D", index=["A", "B"], columns=["C"], aggfunc="sum"  
... )  
>>> table  
C      large  small  
A  B  
bar one    4.0    5.0  
   two    7.0    6.0  
foo one    4.0    1.0  
   two    NaN    6.0
```

We can also fill missing values using the `fill\_value` parameter.

```
>>> table = pd.pivot_table(  
...     df, values="D", index=["A", "B"], columns=["C"], aggfunc="sum", fill_value=0  
... )  
>>> table  
C      large  small  
A  B  
bar one     4     5  
   two     7     6  
foo one     4     1  
   two     0     6
```

The next example aggregates by taking the mean across multiple columns.

```
>>> table = pd.pivot_table(  
...     df, values=["D", "E"], index=["A", "C"], aggfunc={"D": "mean", "E": "mean"}  
... )  
>>> table  
      D      E  
A  C  
bar large  5.500000  7.500000  
   small  5.500000  8.500000  
foo large  2.000000  4.500000  
   small  2.333333  4.333333
```

We can also calculate multiple types of aggregations for any given value column.

```
>>> table = pd.pivot_table(
...     df,
...     values=["D", "E"],
...     index=["A", "C"],
...     aggfunc={"D": "mean", "E": ["min", "max", "mean"]},
... )
>>> table
```

			D	E		
			mean	max	mean	min
A	C					
bar	large		5.500000	9	7.500000	6
	small		5.500000	9	8.500000	8
foo	large		2.000000	5	4.500000	4
	small		2.333333	6	4.333333	2

```
qcut(
    x,
    q,
    labels=None,
    retbins: bool = False,
    precision: int = 3,
    duplicates: str = 'raise'
)
```

Quantile-based discretization function.

Discretize variable into equal-sized buckets based on rank or based on sample quantiles. For example 1000 values for 10 quantiles would produce a Categorical object indicating quantile membership for each data point.

Parameters

x : 1d ndarray or Series
Input Numpy array or pandas Series object to be discretized.

q : int or list-like of float
Number of quantiles. 10 for deciles, 4 for quartiles, etc. Alternately array of quantiles, e.g. [0, .25, .5, .75, 1.] for quartiles.

labels : array or False, default None
Used as labels for the resulting bins. Must be of the same length as the resulting bins. If False, return only integer indicators of the bins. If True, raises an error.

retbins : bool, optional
Whether to return the (bins, labels) or not. Can be useful if bins is given as a scalar.

precision : int, optional
The precision at which to store and display the bins labels.

duplicates : {default 'raise', 'drop'}, optional
If bin edges are not unique, raise ValueError or drop non-uniques.

Returns

out : Categorical or Series or array of integers if labels is False
The return type (Categorical or Series) depends on the input: a Series of type category if input is a Series else Categorical. Bins are represented as categories when categorical data is returned.

bins : ndarray of floats
Returned only if `retbins` is True.

See Also

cut : Bin values into discrete intervals.
Series.quantile : Return value at the given quantile.

Notes

Out of bounds values will be NA in the resulting Categorical object

Examples

```
>>> pd.qcut(range(5), 4)
... # doctest: +ELLIPSIS
[(-0.001, 1.0], (-0.001, 1.0], (1.0, 2.0], (2.0, 3.0], (3.0, 4.0]]
Categories (4, interval[float64, right]): [(-0.001, 1.0] < (1.0, 2.0] ...

>>> pd.qcut(range(5), 3, labels=["good", "medium", "bad"])
... # doctest: +SKIP
[good, good, medium, bad, bad]
Categories (3, str): [good < medium < bad]
```

```

>>> pd.qcut(range(5), 4, labels=False)
array([0, 0, 1, 2, 3])

read_clipboard(
    sep: str = '\\s+',
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
    **kwargs
)
    Read text from clipboard and pass to :func:`~pandas.read_csv`.

    Parses clipboard contents similar to how CSV files are parsed
    using :func:`~pandas.read_csv`.

    Parameters
    -----
    sep : str, default '\\s+'
        A string or regex delimiter. The default of ``\\s+`` denotes
        one or more whitespace characters.

    dtype_backend : {'numpy_nullable', 'pyarrow'}
        Back-end data type applied to the resultant :class:`DataFrame`
        (still experimental). If not specified, the default behavior
        is to not use nullable data types. If specified, the behavior
        is as follows:

        * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
        * ``"pyarrow"``: returns pyarrow-backed nullable
          :class:`ArrowDtype` :class:`DataFrame`

        .. versionadded:: 2.0

    **kwargs
        See :func:`~pandas.read_csv` for the full argument list.

    Returns
    -----
    DataFrame
        A parsed :class:`~pandas.DataFrame` object.

    See Also
    -----
    DataFrame.to_clipboard : Copy object to the system clipboard.
    read_csv : Read a comma-separated values (csv) file into DataFrame.
    read_fwf : Read a table of fixed-width formatted lines into DataFrame.

    Examples
    -----
    >>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])
    >>> df.to_clipboard() # doctest: +SKIP
    >>> pd.read_clipboard() # doctest: +SKIP
       A  B  C
    0   1  2  3
    1   4  5  6

read_csv(
    filepath_or_buffer: FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str],
    *,
    sep: str | None | lib.NoDefault = <no_default>,
    delimiter: str | None | lib.NoDefault = None,
    header: int | Sequence[int] | None | Literal['infer'] = 'infer',
    names: Sequence[Hashable] | None | lib.NoDefault = <no_default>,
    index_col: IndexLabel | Literal[False] | None = None,
    usecols: UsecolsArgType = None,
    dtype: DtypeArg | None = None,
    engine: CSVEngine | None = None,
    converters: Mapping[HashableT, Callable] | None = None,
    true_values: list | None = None,
    false_values: list | None = None,
    skipinitialspace: bool = False,
    skiprows: list[int] | int | Callable[[Hashable], bool] | None = None,
    skipfooter: int = 0,
    nrows: int | None = None,
    na_values: Hashable | Iterable[Hashable] | Mapping[Hashable, Iterable[Hashable]] | None = None,
    keep_default_na: bool = True,
    na_filter: bool = True,
    skip_blank_lines: bool = True,
    parse_dates: bool | Sequence[Hashable] | None = None,

```

```

date_format: str | dict[Hashable, str] | None = None,
dayfirst: bool = False,
cache_dates: bool = True,
iterator: bool = False,
chunksize: int | None = None,
compression: CompressionOptions = 'infer',
thousands: str | None = None,
decimal: str = '.',
lineterminator: str | None = None,
quotechar: str = '"',
quoting: int = 0,
doublequote: bool = True,
escapechar: str | None = None,
comment: str | None = None,
encoding: str | None = None,
encoding_errors: str | None = 'strict',
dialect: str | csv.Dialect | None = None,
on_bad_lines: str = 'error',
low_memory: bool = True,
memory_map: bool = False,
float_precision: Literal['high', 'legacy', 'round_trip'] | None = None,
storage_options: StorageOptions | None = None,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame | TextFileReader
Read a comma-separated values (csv) file into DataFrame.

```

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for
`IO Tools` <https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html>`_.

Parameters

```

-----
filepath_or_buffer : str, path object or file-like object
    Any valid string path is acceptable. The string could be a URL. Valid
    URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is
    expected. A local file could be: file://localhost/path/to/table.csv.

    If you want to pass in a path object, pandas accepts any ``os.PathLike``.

    By file-like object, we refer to objects with a ``read()`` method, such as
    a file handle (e.g. via builtin ``open`` function) or ``StringIO``.
sep : str, default ','
    Character or regex pattern to treat as the delimiter. If ``sep=None``, the
    C engine cannot automatically detect
    the separator, but the Python parsing engine can, meaning the latter will
    be used and automatically detect the separator from only the first valid
    row of the file by Python's builtin sniffer tool, ``csv.Sniffer``.
    In addition, separators longer than 1 character and different from
    ``'\s+'`` will be interpreted as regular expressions and will also force
    the use of the Python parsing engine. Note that regex delimiters are prone
    to ignoring quoted data. Regex example: ``'\r\t'``.
delimiter : str, optional
    Alias for ``sep``.
header : int, Sequence of int, 'infer' or None, default 'infer'
    Row number(s) containing column labels and marking the start of the
    data (zero-indexed). Default behavior is to infer the column names:
    if no ``names``
    are passed the behavior is identical to ``header=0`` and column
    names are inferred from the first line of the file, if column
    names are passed explicitly to ``names`` then the behavior is identical to
    ``header=None``. Explicitly pass ``header=0`` to be able to
    replace existing names. The header can be a list of integers that
    specify row locations for a :class:`~pandas.MultiIndex` on the columns
    e.g. ``[0, 1, 3]``. Intervening rows that are not specified will be
    skipped (e.g. 2 in this example is skipped). Note that this
    parameter ignores commented lines and empty lines if
    ``skip_blank_lines=True``, so ``header=0`` denotes the first line of
    data rather than the first line of the file.

```

When inferred from the file contents, headers are kept distinct from each other by renaming duplicate names with a numeric suffix of the form ``".{count}"`` starting from 1, e.g. ``"foo"`` and ``"foo.1"``.

Empty headers are named ``"Unnamed: {i}"`` or ``"Unnamed: {i}\_level\_{level}"`` in the case of MultiIndex columns.

names : Sequence of Hashable, optional
Sequence of column labels to apply. If the file contains a header row, then you should explicitly pass ``header=0`` to override the column names. Duplicates in this list are not allowed.

index_col : Hashable, Sequence of Hashable or False, optional
Column(s) to use as row label(s), denoted either by column labels or column indices. If a sequence of labels or indices is given, :class:`~pandas.MultiIndex` will be formed for the row labels.

Note: ``index\_col=False`` can be used to force pandas to *not* use the first column as the index, e.g., when you have a malformed file with delimiters at the end of each line.

usecols : Sequence of Hashable or Callable, optional
Subset of columns to select, denoted either by column labels or column indices. If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in ``names`` or inferred from the document header row(s). If ``names`` are given, the document header row(s) are not taken into account. For example, a valid list-like ``usecols`` parameter would be ``[0, 1, 2]`` or ``['foo', 'bar', 'baz']``. Element order is ignored, so ``usecols=[0, 1]`` is the same as ``[1, 0]``. To instantiate a :class:`~pandas.DataFrame` from ``data`` with element order preserved use ``pd.read\_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]`` for columns in ``['foo', 'bar']`` order or ``pd.read\_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]`` for ``['bar', 'foo']`` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to ``True``. An example of a valid callable argument would be ``lambda x: x.upper() in ['AAA', 'BBB', 'DDD']``. Using this parameter results in much faster parsing time and lower memory usage.

dtype : dtype or dict of {{Hashable : dtype}}, optional
Data type(s) to apply to either the whole dataset or individual columns. E.g., ``{'a': np.float64, 'b': np.int32, 'c': 'Int64'}``. Use ``str`` or ``object`` together with suitable ``na\_values`` settings to preserve and not interpret ``dtype``. If ``converters`` are specified, they will be applied INSTEAD of ``dtype`` conversion. Specify a ``defaultdict`` as input where the default determines the ``dtype`` of the columns which are not explicitly listed.

engine : {'c', 'python', 'pyarrow'}, optional
Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. Multithreading is currently only supported by the pyarrow engine. Some features of the "pyarrow" engine are unsupported or may not work correctly.

converters : dict of {{Hashable : Callable}}, optional
Functions for converting values in specified columns. Keys can either be column labels or column indices.

true_values : list, optional
Values to consider as ``True`` in addition to case-insensitive variants of 'True'.

false_values : list, optional
Values to consider as ``False`` in addition to case-insensitive variants of 'False'.

skipinitialspace : bool, default False
Skip spaces after delimiter.

skiprows : int, list of int or Callable, optional
Line numbers to skip (0-indexed) or number of lines to skip (``int``) at the start of the file.

If callable, the callable function will be evaluated against the row indices, returning ``True`` if the row should be skipped and ``False`` otherwise. An example of a valid callable argument would be ``lambda x: x in [0, 2]``.

skipfooter : int, default 0
Number of lines at bottom of file to skip (Unsupported with ``engine='c'``).

nrows : int, optional
Number of rows of file to read. Useful for reading pieces of large files. Refers to the number of data rows in the returned DataFrame, excluding:
* The header row containing column names.

* Rows before the header row, if ``header=1`` or larger.

Example usage:

* To read the first 999,999 (non-header) rows:

```
``read_csv(..., nrows=999999)``
```

* To read rows 1,000,000 through 1,999,999:

```
``read_csv(..., skiprows=1000000, nrows=999999)``
```

`na_values` : Hashable, Iterable of Hashable or dict of `{Hashable : Iterable}`,

optional

Additional strings to recognize as ``NA``/``NaN``. If ``dict``

passed, specific

per-column ``NA`` values. By default the following values

are interpreted as

``NaN``: empty string, "NaN", "N/A", "NULL", and other common

representations of missing data.

`keep_default_na` : bool, default True

Whether or not to include the default ``NaN`` values when parsing the data.

Depending on whether ``na\_values`` is passed in, the behavior is as follows:

* If ``keep\_default\_na`` is ``True``, and ``na\_values``

are specified, ``na\_values``

is appended to the default ``NaN`` values used for parsing.

* If ``keep\_default\_na`` is ``True``, and ``na\_values`` are not specified, only the default ``NaN`` values are used for parsing.

* If ``keep\_default\_na`` is ``False``, and ``na\_values`` are specified, only the ``NaN`` values specified ``na\_values`` are used for parsing.

* If ``keep\_default\_na`` is ``False``, and ``na\_values`` are not specified, no strings will be parsed as ``NaN``.

Note that if ``na\_filter`` is passed in as ``False``,

the ``keep\_default\_na`` and

``na\_values`` parameters will be ignored.

`na_filter` : bool, default True

Detect missing value markers (empty strings and the value of ``na\_values``). In

data without any ``NA`` values, passing ``na\_filter=False`` can improve the

performance of reading a large file.

`skip_blank_lines` : bool, default True

If ``True``, skip over blank lines rather than interpreting as ``NaN`` values.

`parse_dates` : bool, None, list of Hashable, default None

The behavior is as follows:

* ``bool``. If ``True`` -> try parsing the index.

* ``None``. Behaves like ``True`` if ``date\_format`` is specified.

* ``list`` of ``int`` or names.

e.g. If `[1, 2, 3]` -> try parsing columns 1, 2, 3

each as a separate date column.

If a column or index cannot be represented as an array of ``datetime``,

say because of an unparseable value or a mixture of timezones, the column

or index will be returned unaltered as an ``object`` data type. For

non-standard ``datetime`` parsing, use `:func:`~pandas.to_datetime`` after

`:func:`~pandas.read_csv``.

Note: A fast-path exists for iso8601-formatted dates.

`date_format` : str or dict of column -> format, optional

Format to use for parsing dates and/or times when

used in conjunction with ``parse\_dates``.

The strftime to parse time, e.g. `:const: "%d/%m/%Y"`. See

`strftime` documentation

<<https://docs.python.org/3/library/datetime.html>

`#strftime-and-strptime-behavior`> for more information on choices, though

note that `:const: "%f"` will parse all the way up to nanoseconds.

You can also pass:

- "ISO8601", to parse any ISO8601 <https://en.wikipedia.org/wiki/ISO_8601> time string (not necessarily in exactly the same format);

- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with `dayfirst`.

.. versionadded:: 2.0.0

`dayfirst` : bool, default False

DD/MM format dates, international and European format.

`cache_dates` : bool, default True

If ``True``, use a cache of unique, converted dates to apply the ``datetime``

conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets.

iterator : bool, default False
Return ``TextFileReader`` object for iteration or getting chunks with
``get\_chunk()``.

chunksize : int, optional
Number of lines to read from the file per chunk. Passing a value will cause the
function to return a ``TextFileReader`` object for iteration.
See the `IO Tools docs`
<<https://pandas.pydata.org/pandas-docs/stable/io.html#io-chunking>>`\_`
for more information on ``iterator`` and ``chunksize``.

compression : str or dict, default 'infer'
For on-the-fly decompression of on-disk data.
If 'infer' and 'filepath_or_buffer' is
path-like, then detect compression from the following extensions: '.gz',
'bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
(otherwise no compression).
If using 'zip' or 'tar', the ZIP file must contain only
one data file to be read in.
Set to ``None`` for no decompression.
Can also be a dict with key ``'method'`` set
to one of {``'zip'``, ``'gzip'``, ``'bz2'``,
``'zstd'``, ``'xz'``, ``'tar'``} and
other key-value pairs are forwarded to
``zipfile.ZipFile``, ``gzip.GzipFile``,
``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
``tarfile.TarFile``, respectively.
As an example, the following could be passed for
Zstandard decompression using a
custom compression dictionary:
``compression={'method': 'zstd', 'dict\_data': my\_compression\_dict}``.

thousands : str (length 1), optional
Character acting as the thousands separator in numerical values.

decimal : str (length 1), default '.'
Character to recognize as decimal point (e.g., use ',' for European data).

lineterminator : str (length 1), optional
Character used to denote a line break. Only valid with C parser.

quotechar : str (length 1), optional
Character used to denote the start and end of a quoted item. Quoted
items can include the ``delimiter`` and it will be ignored.

quoting : {0 or csv.QUOTE_MINIMAL, 1 or csv.QUOTE_ALL,
2 or csv.QUOTE_NONNUMERIC, 3 or csv.QUOTE_NONE}, default csv.QUOTE_MINIMAL
Control field quoting behavior per ``csv.QUOTE\_\*`` constants. Default is
``csv.QUOTE\_MINIMAL`` (i.e., 0) which implies that
only fields containing special
characters are quoted (e.g., characters defined
in ``quotechar``, ``delimiter``,
or ``lineterminator``).

doublequote : bool, default True
When ``quotechar`` is specified and ``quoting`` is not ``QUOTE\_NONE``, indicate
whether or not to interpret two consecutive ``quotechar`` elements INSIDE a
field as a single ``quotechar`` element.

escapechar : str (length 1), optional
Character used to escape other characters.

comment : str (length 1), optional
Character indicating that the remainder of line should not be parsed.
If found at the beginning
of a line, the line will be ignored altogether. This parameter must be a
single character. Like empty lines (as long as ``skip\_blank\_lines=True``),
fully commented lines are ignored by the parameter ``header`` but not by
``skiprows``. For example, if ``comment='#'``, parsing
``#empty\na,b,c\n1,2,3`` with ``header=0`` will result in ``a,b,c`` being
treated as the header.

encoding : str, optional, default 'utf-8'
Encoding to use for UTF when reading/writing (ex. ``'utf-8'``). `List of Python
standard encodings`
<<https://docs.python.org/3/library/codecs.html#standard-encodings>>`\_` .

encoding_errors : str, optional, default 'strict'
How encoding errors are treated. `List of possible values`
<<https://docs.python.org/3/library/codecs.html#error-handlers>>`\_` .

dialect : str or csv.Dialect, optional
If provided, this parameter will override values (default or not) for the

following parameters: ``delimiter``, ``doublequote``, ``escapechar``, ``skipinitialspace``, ``quotechar``, and ``quoting``. If it is necessary to override values, a ``ParserWarning`` will be issued. See ``csv.Dialect`` documentation for more details.

`on_bad_lines` : `{{'error', 'warn', 'skip'}}` or Callable, default 'error'
 Specifies what to do upon encountering a bad line (a line with too many fields). Allowed values are:

- ``'error'``, raise an Exception when a bad line is encountered.
- ``'warn'``, raise a warning when a bad line is encountered and skip that line.
- ``'skip'``, skip bad lines without raising or warning when they are encountered.
- Callable, function that will process a single bad line.
 - With ``engine='python'``, function with signature `((bad_line: list[str]) -> list[str] | None)`.
 ``bad\_line`` is a list of strings split by the ``sep``.
 If the function returns ``None``, the bad line will be ignored.
 If the function returns a new ``list`` of strings with more elements than expected, a ``ParserWarning`` will be emitted while dropping extra elements.
 - With ``engine='pyarrow'``, function with signature as described in pyarrow documentation: `invalid_row_handler`
https://arrow.apache.org/docs/python/generated/pyarrow.csv.ParseOptions.html#pyarrow.csv.ParseOptions.invalid_row_handler>`_.

.. versionchanged:: 2.2.0

Callable for ``engine='pyarrow'``

`low_memory` : bool, default True
 Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. To ensure no mixed types either set ``False``, or specify the type with the ``dtype`` parameter. Note that the entire file is read into a single `:class:`~pandas.DataFrame`` regardless, use the ``chunksize`` or ``iterator`` parameter to return the data in chunks. (Only valid with C parser).

`memory_map` : bool, default False
 If a filepath is provided for ``filepath\_or\_buffer``, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

`float_precision` : `{{'high', 'legacy', 'round_trip'}}`, optional
 Specifies which converter the C engine should use for floating-point values. The options are ``None`` or ``'high'`` for the ordinary converter, ``'legacy'`` for the original lower precision pandas converter, and ``'round\_trip'`` for the round-trip converter.

`storage_options` : dict, optional
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.`

`dtype_backend` : `{{'numpy_nullable', 'pyarrow'}}`
 Back-end data type applied to the resultant `:class:`DataFrame`` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- * ``"numpy\_nullable"``: returns nullable-dtype-backed `:class:`DataFrame``
- * ``"pyarrow"``: returns pyarrow-backed nullable `:class:`ArrowDtype`` `:class:`DataFrame``

.. versionadded:: 2.0

Returns

`DataFrame` or `TextFileReader`

A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

See Also

DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.
read_table : Read general delimited file into DataFrame.
read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Examples

>>> pd.read_csv("data.csv") # doctest: +SKIP
 Name Value
0 foo 1
1 bar 2
2 #baz 3

Index and header can be specified via the `index\_col` and `header` arguments.

>>> pd.read_csv("data.csv", header=None) # doctest: +SKIP
 0 1
0 Name Value
1 foo 1
2 bar 2
3 #baz 3

>>> pd.read_csv("data.csv", index_col="Value") # doctest: +SKIP
 Name
Value
1 foo
2 bar
3 #baz

Column types are inferred but can be explicitly specified using the dtype argument.

>>> pd.read_csv("data.csv", dtype={"Value": float}) # doctest: +SKIP
 Name Value
0 foo 1.0
1 bar 2.0
2 #baz 3.0

True, False, and NA values, and thousands separators have defaults, but can be explicitly specified, too. Supply the values you would like as strings or lists of strings!

>>> pd.read_csv("data.csv", na_values=["foo", "bar"]) # doctest: +SKIP
 Name Value
0 NaN 1
1 NaN 2
2 #baz 3

Comment lines in the input file can be skipped using the `comment` argument.

>>> pd.read_csv("data.csv", comment="#") # doctest: +SKIP
 Name Value
0 foo 1
1 bar 2

By default, columns with dates will be read as ``object`` rather than ``datetime``.

>>> df = pd.read_csv("tmp.csv") # doctest: +SKIP

>>> df # doctest: +SKIP
 col 1 col 2 col 3
0 10 10/04/2018 Sun 15 Jan 2023
1 20 15/04/2018 Fri 12 May 2023

>>> df.dtypes # doctest: +SKIP
col 1 int64
col 2 object
col 3 object
dtype: object

Specific columns can be parsed as dates by using the `parse\_dates` and `date\_format` arguments.

>>> df = pd.read_csv(
... "tmp.csv",
... parse_dates=[1, 2],
... date_format={"col 2": "%d/%m/%Y", "col 3": "%a %d %b %Y"}},

```

... ) # doctest: +SKIP

>>> df.dtypes # doctest: +SKIP
col 1      int64
col 2    datetime64[ns]
col 3    datetime64[ns]
dtype: object

read_excel(
    io,
    sheet_name: str | int | list[IntStrT] | None = 0,
    *,
    header: int | Sequence[int] | None = 0,
    names: SequenceNotStr[Hashable] | range | None = None,
    index_col: int | str | Sequence[int] | None = None,
    usecols: int | str | Sequence[int] | Sequence[str] | Callable[[HashableT], bool] | None = None,
    dtype: DtypeArg | None = None,
    engine: Literal['xlrd', 'openpyxl', 'odf', 'pyxlsb', 'calamine'] | None = None,
    converters: dict[str, Callable] | dict[int, Callable] | None = None,
    true_values: Iterable[Hashable] | None = None,
    false_values: Iterable[Hashable] | None = None,
    skiprows: Sequence[int] | int | Callable[[int], object] | None = None,
    nrows: int | None = None,
    na_values=None,
    keep_default_na: bool = True,
    na_filter: bool = True,
    verbose: bool = False,
    parse_dates: list | dict | bool = False,
    date_format: dict[Hashable, str] | str | None = None,
    thousands: str | None = None,
    decimal: str = '.',
    comment: str | None = None,
    skipfooter: int = 0,
    storage_options: StorageOptions | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
    engine_kwargs: dict | None = None
) -> DataFrame | dict[IntStrT, DataFrame]
    Read an Excel file into a ``DataFrame``.

    Supports ``xls``, ``xlsx``, ``xslm``, ``xlsb``, ``odf``, ``ods`` and ``odt`` file extensions
    read from a local filesystem or URL. Supports an option to read
    a single sheet or a list of sheets.

    Parameters
    -----
    io : str, ExcelFile, xlrd.Book, path object, or file-like object
        Any valid string path is acceptable. The string could be a URL. Valid
        URL schemes include http, ftp, s3, and file. For file URLs, a host is
        expected. A local file could be: ``file://localhost/path/to/table.xlsx``.

        If you want to pass in a path object, pandas accepts any ``os.PathLike``.

        By file-like object, we refer to objects with a ``read()`` method,
        such as a file handle (e.g. via builtin ``open`` function)
        or ``StringIO``.

    sheet_name : str, int, list, or None, default 0
        Strings are used for sheet names. Integers are used in zero-indexed
        sheet positions (chart sheets do not count as a sheet position).
        Lists of strings/integers are used to request multiple sheets.
        When ``None``, will return a dictionary containing DataFrames for each sheet.

    Available cases:

    * Defaults to ``0``: 1st sheet as a ``DataFrame``
    * ``1``: 2nd sheet as a ``DataFrame``
    * ``"Sheet1"``: Load sheet with name "Sheet1"
    * ``[0, 1, "Sheet5"]``: Load first, second and sheet named "Sheet5"
      as a dict of ``DataFrame``
    * ``None``: Returns a dictionary containing DataFrames for each sheet.

    header : int, list of int, default 0
        Row (0-indexed) to use for the column labels of the parsed
        DataFrame. If a list of integers is passed those row positions will
        be combined into a ``MultiIndex``. Use None if there is no header.
    names : array-like, default None
        List of column names to use. If file contains no header row,

```

then you should explicitly pass header=None.

index_col : int, str, list of int, default None
 Column (0-indexed) to use as the row labels of the DataFrame.
 Pass None if there is no such column. If a list is passed,
 those columns will be combined into a ``MultiIndex``. If a
 subset of data is selected with ``usecols``, index_col
 is based on the subset.

Missing values will be forward filled to allow roundtripping with
 ``to\_excel`` for ``merged\_cells=True``. To avoid forward filling the
 missing values use ``set\_index`` after reading the data instead of
 ``index\_col``.

usecols : str, list-like, or callable, default None
 * If None, then parse all columns.
 * If str, then indicates comma separated list of Excel column letters
 and column ranges (e.g. "A:E" or "A,C,E:F"). Ranges are inclusive of
 both sides.
 * If list of int, then indicates list of column numbers to be parsed
 (0-indexed).
 * If list of string, then indicates list of column names to be parsed.
 * If callable, then evaluate each column name against it and parse the
 column if the callable returns ``True``.

Returns a subset of the columns according to behavior above.

dtype : Type name or dict of column -> type, default None
 Data type for data or columns. E.g. {'a': np.float64, 'b': np.int32}
 Use ``object`` to preserve data as stored in Excel and not interpret dtype,
 which will necessarily result in ``object`` dtype.
 If converters are specified, they will be applied INSTEAD
 of dtype conversion.
 If you use ``None``, it will infer the dtype of each column based on the data.

engine : {'openpyxl', 'calamine', 'odf', 'pyxlsb', 'xlrd'}, default None
 If io is not a buffer or path, this must be set to identify io.
 Engine compatibility :

- ``openpyxl`` supports newer Excel file formats.
- ``calamine`` supports Excel (.xls, .xlsx, .xslm, .xlsb)
 and OpenDocument (.ods) file formats.
- ``odf`` supports OpenDocument file formats (.odf, .ods, .odt).
- ``pyxlsb`` supports Binary Excel files.
- ``xlrd`` supports old-style Excel files (.xls).

When ``engine=None``, the following logic will be used to determine the engine:

- If ``path\_or\_buffer`` is an OpenDocument format (.odf, .ods, .odt),
 then ``odf`` <<https://pypi.org/project/odfpy/>> will be used.
- Otherwise if ``path\_or\_buffer`` is an xls format, ``xlrd`` will be used.
- Otherwise if ``path\_or\_buffer`` is in xlsb format, ``pyxlsb`` will be used.
- Otherwise ``openpyxl`` will be used.

converters : dict, default None
 Dict of functions for converting values in certain columns. Keys can
 either be integers or column labels, values are functions that take one
 input argument, the Excel cell content, and return the transformed
 content.

true_values : list, default None
 Values to consider as True.

false_values : list, default None
 Values to consider as False.

skiprows : list-like, int, or callable, optional
 Line numbers to skip (0-indexed) or number of lines to skip (int) at the
 start of the file. If callable, the callable function will be evaluated
 against the row indices, returning True if the row should be skipped and
 False otherwise. An example of a valid callable argument would be ``lambda
 x: x in [0, 2]``.

nrows : int, default None
 Number of rows to parse. Does not include header rows.

na_values : scalar, str, list-like, or dict, default None
 Additional strings to recognize as NA/NaN. If dict passed, specific
 per-column NA values. By default the following values are interpreted
 as NaN: '', '#N/A', '#N/A N/A', '#NA', '-1.#IND', '-1.#QNAN', '-NaN', '-nan',
 '1.#IND', '1.#QNAN', '<NA>', 'N/A', 'NA', 'NULL', 'NaN', 'None',
 'n/a', 'nan', 'null'.

keep_default_na : bool, default True
 Whether or not to include the default NaN values when parsing the data.
 Depending on whether ``na\_values`` is passed in, the behavior is as follows:
 * If ``keep\_default\_na`` is True, and ``na\_values`` are specified,

```

    ``na_values`` is appended to the default NaN values used for parsing.
* If ``keep_default_na`` is True, and ``na_values`` are not specified, only
  the default NaN values are used for parsing.
* If ``keep_default_na`` is False, and ``na_values`` are specified, only
  the NaN values specified ``na_values`` are used for parsing.
* If ``keep_default_na`` is False, and ``na_values`` are not specified, no
  strings will be parsed as NaN.

Note that if ``na_filter`` is passed in as False, the ``keep_default_na`` and
``na_values`` parameters will be ignored.
na_filter : bool, default True
  Detect missing value markers (empty strings and the value of na_values). In
  data without any NAs, passing ``na_filter=False`` can improve the
  performance of reading a large file.
verbose : bool, default False
  Indicate number of NA values placed in non-numeric columns.
parse_dates : bool, list-like, or dict, default False
  The behavior is as follows:

  * ``bool``. If True -> try parsing the index.
  * ``list`` of int or names. e.g. If [1, 2, 3] -> try parsing columns 1, 2, 3
    each as a separate date column.
  * ``list`` of lists. e.g. If [[1, 3]] -> combine columns 1 and 3 and parse as
    a single date column.
  * ``dict``, e.g. {'foo' : [1, 3]} -> parse columns 1, 3 as date and call
    result 'foo'

If a column or index contains an unparseable date, the entire column or
index will be returned unaltered as an object data type. If you don't want to
parse some cells as date just change their type in Excel to "Text".
For non-standard datetime parsing, use ``pd.to_datetime`` after
``pd.read_excel``.

Note: A fast-path exists for iso8601-formatted dates.
date_format : str or dict of column -> format, default ``None``
  If used in conjunction with ``parse_dates``, will parse dates according to this
  format. For anything more complex,
  please read in as ``object`` and then apply :func:`to_datetime` as-needed.

.. versionadded:: 2.0.0

thousands : str, default None
  Thousands separator for parsing string columns to numeric. Note that
  this parameter is only necessary for columns stored as TEXT in Excel,
  any numeric columns will automatically be parsed, regardless of display
  format.
decimal : str, default '.'
  Character to recognize as decimal point for parsing string columns to numeric.
  Note that this parameter is only necessary for columns stored as TEXT in Excel,
  any numeric columns will automatically be parsed, regardless of display
  format.(e.g. use ',' for European data).
comment : str, default None
  Comments out remainder of line. Pass a character or characters to this
  argument to indicate comments in the input file. Any data between the
  comment string and the end of the current line is ignored.
skipfooter : int, default 0
  Rows at the end to skip (0-indexed).
storage_options : dict, optional
  Extra options that make sense for a particular storage connection, e.g.
  host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
  are forwarded to ``urllib.request.Request`` as header options. For other
  URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
  forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
  details, and for more examples on storage options refer `here`
  <https://pandas.pydata.org/docs/user\_guide/io.html?
  highlight=storage_options#reading-writing-remote-files>`_.

dtype_backend : {'numpy_nullable', 'pyarrow'}
  Back-end data type applied to the resultant :class:`DataFrame`
  (still experimental). If not specified, the default behavior
  is to not use nullable data types. If specified, the behavior
  is as follows:

  * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
  * ``"pyarrow"``: returns pyarrow-backed nullable
    :class:`ArrowDtype` :class:`DataFrame`

```

```

.. versionadded:: 2.0

engine_kwargs : dict, optional
    Arbitrary keyword arguments passed to excel engine.

Returns
-----
DataFrame or dict of DataFrames
    DataFrame from the passed in Excel file. See notes in sheet_name
    argument for more information on when a dict of DataFrames is returned.

See Also
-----
DataFrame.to_excel : Write DataFrame to an Excel file.
DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.
read_csv : Read a comma-separated values (csv) file into DataFrame.
read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Notes
-----
For specific information on the methods used for each Excel engine, refer to the
pandas
:ref:`user guide <io.excel_reader>`

Examples
-----
The file can be read using the file name as string or an open file object:

>>> pd.read_excel("tmp.xlsx", index_col=0) # doctest: +SKIP
   Name  Value
0  string1    1
1  string2    2
2  #Comment    3

>>> pd.read_excel(open("tmp.xlsx", "rb"), sheet_name="Sheet3") # doctest: +SKIP
   Unnamed: 0  Name  Value
0           0  string1    1
1           1  string2    2
2           2  #Comment    3

Index and header can be specified via the `index_col` and `header` arguments

>>> pd.read_excel("tmp.xlsx", index_col=None, header=None) # doctest: +SKIP
   0      1      2
0  NaN    Name  Value
1  0.0  string1    1
2  1.0  string2    2
3  2.0  #Comment    3

Column types are inferred but can be explicitly specified

>>> pd.read_excel(
...     "tmp.xlsx", index_col=0, dtype={"Name": str, "Value": float}
... ) # doctest: +SKIP
   Name  Value
0  string1  1.0
1  string2  2.0
2  #Comment  3.0

True, False, and NA values, and thousands separators have defaults,
but can be explicitly specified, too. Supply the values you would like
as strings or lists of strings!

>>> pd.read_excel(
...     "tmp.xlsx", index_col=0, na_values=["string1", "string2"]
... ) # doctest: +SKIP
   Name  Value
0     NaN    1
1     NaN    2
2  #Comment    3

Comment lines in the excel input file can be skipped using the
`comment` kwarg.

>>> pd.read_excel("tmp.xlsx", index_col=0, comment="#") # doctest: +SKIP
   Name  Value
0  string1  1.0

```



```

1 string2    2.0
2      None    NaN

```

`read_feather()`

path: FilePath | ReadBuffer[bytes],
columns: Sequence[Hashable] | None = None,
use_threads: bool = True,
storage_options: StorageOptions | None = None,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame

Load a feather-format object from the file path.

Feather is particularly useful for scenarios that require efficient serialization and deserialization of tabular data. It supports schema preservation, making it a reliable choice for use cases such as sharing data between Python and R, or persisting intermediate results during data processing pipelines. This method provides additional flexibility with options for selective column reading, thread parallelism, and choosing the backend for data types.

Parameters

path : str, path object, or file-like object
String, path object (implementing `os.PathLike[str]`), or file-like object implementing a binary `read()` function. The string could be a URL. Valid URL schemes include http, ftp, s3, gs and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.feather`.

columns : sequence, default None
If not provided, all columns are read.

use_threads : bool, default True
Whether to parallelize reading using multiple threads.

storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

dtype_backend : {'numpy_nullable', 'pyarrow'}
Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- * `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`.
- * `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`.

.. versionadded:: 2.0

Returns

type of object stored in file
DataFrame object stored in the file.

See Also

`read_csv` : Read a comma-separated values (csv) file into a pandas DataFrame.
`read_excel` : Read an Excel file into a pandas DataFrame.
`read_spss` : Read an SPSS file into a pandas DataFrame.
`read_orc` : Load an ORC object into a pandas DataFrame.
`read_sas` : Read SAS file into a pandas DataFrame.

Examples

```

>>> df = pd.read_feather("path/to/file.feather") # doctest: +SKIP

```

`read_fwf()`

filepath_or_buffer: FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str],
*,
colspecs: Sequence[tuple[int, int]] | str | None = 'infer',
widths: Sequence[int] | None = None,
infer_nrows: int = 100,
iterator: bool = False,

```

    chunksize: int | None = None,
    **kwargs: Unpack[_read_shared[HashableT]]
) -> DataFrame | TextFileReader
    Read a table of fixed-width formatted lines into DataFrame.

    Also supports optionally iterating or breaking of the file
    into chunks.

    Additional help can be found in the `online docs for IO Tools
    <https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html>`.

Parameters
-----
filepath_or_buffer : str, path object, or file-like object
    String, path object (implementing `os.PathLike[str]`), or file-like
    object implementing a text `read()` function. The string could be a URL.
    Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is
    expected. A local file could be:
    `file://localhost/path/to/table.csv`.
colspecs : list of tuple (int, int) or 'infer'. optional
    A list of tuples giving the extents of the fixed-width
    fields of each line as half-open intervals (i.e., [from, to] ).
    String value 'infer' can be used to instruct the parser to try
    detecting the column specifications from the first 100 rows of
    the data which are not being skipped via skiprows (default='infer').
widths : list of int, optional
    A list of field widths which can be used instead of 'colspecs' if
    the intervals are contiguous.
infer_nrows : int, default 100
    The number of rows to consider when letting the parser determine the
    `colspecs`.
iterator : bool, default False
    Return `TextFileReader` object for iteration or getting chunks with
    `get_chunk()`.
chunksize : int, optional
    Number of lines to read from the file per chunk.
**kwargs : optional
    Optional keyword arguments can be passed to `TextFileReader`.

```

Returns

```

-----
DataFrame or TextFileReader
    A comma-separated values (csv) file is returned as two-dimensional
    data structure with labeled axes.

```

See Also

```

-----
DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.
read_csv : Read a comma-separated values (csv) file into DataFrame.

```

Examples

```

-----
>>> pd.read_fwf("data.csv") # doctest: +SKIP

```

```

read_hdf(
    path_or_buf: FilePath | HDFStore,
    key=None,
    mode: str = 'r',
    errors: str = 'strict',
    where: str | list | None = None,
    start: int | None = None,
    stop: int | None = None,
    columns: list[str] | None = None,
    iterator: bool = False,
    chunksize: int | None = None,
    **kwargs
)
    Read from the store, close it if we opened it.

    Retrieve pandas object stored in file, optionally based on where
    criteria.

    .. warning::

```

Pandas uses PyTables for reading and writing HDF5 files, which allows serializing object-dtype data with pickle when using the "fixed" format. Loading pickled data received from untrusted sources can be unsafe.

See: <https://docs.python.org/3/library/pickle.html> for more.

Parameters

`path_or_buf` : str, path object, pandas.HDFStore
Any valid string path is acceptable. Only supports the local file system, remote URLs and file-like objects are not supported.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

Alternatively, pandas accepts an open `:class:`pandas.HDFStore`` object.

`key` : object, optional
The group identifier in the store. Can be omitted if the HDF file contains a single pandas object.

`mode` : {'r', 'r+', 'a'}, default 'r'
Mode to use when opening the file. Ignored if `path_or_buf` is a `:class:`pandas.HDFStore``. Default is 'r'.

`errors` : str, default 'strict'
Specifies how encoding and decoding errors are to be handled. See the `errors` argument for `:func:`open`` for a full list of options.

`where` : list, optional
A list of Term (or convertible) objects.

`start` : int, optional
Row number to start selection.

`stop` : int, optional
Row number to stop selection.

`columns` : list, optional
A list of columns names to return.

`iterator` : bool, optional
Return an iterator object.

`chunksize` : int, optional
Number of rows to include in an iteration when using an iterator.

`**kwargs`
Additional keyword arguments passed to `HDFStore`.

Returns

object
The selected object. Return type depends on the object stored.

See Also

`DataFrame.to_hdf` : Write an HDF file from a DataFrame.

`HDFStore` : Low-level access to HDF files.

Notes

When `errors="surrogatepass"`, `pd.options.future.infer_string` is true, and PyArrow is installed, if a UTF-16 surrogate is encountered when decoding to UTF-8, the resulting dtype will be `pd.StringDtype(storage="python", na_value=np.nan)`.

Examples

```
>>> df = pd.DataFrame([[1, 1.0, "a"]], columns=["x", "y", "z"]) # doctest: +SKIP
>>> df.to_hdf("./store.h5", "data") # doctest: +SKIP
>>> reread = pd.read_hdf("./store.h5") # doctest: +SKIP
```

```
read_html(
    io: FilePath | ReadBuffer[str],
    *,
    match: str | Pattern = '.+',
    flavor: HTMLFlavors | Sequence[HTMLFlavors] | None = None,
    header: int | Sequence[int] | None = None,
    index_col: int | Sequence[int] | None = None,
    skiprows: int | Sequence[int] | slice | None = None,
    attrs: dict[str, str] | None = None,
    parse_dates: bool = False,
    thousands: str | None = ',',
    encoding: str | None = None,
    decimal: str = '.',
    converters: dict | None = None,
    na_values: Iterable[object] | None = None,
    keep_default_na: bool = True,
```

```

displayed_only: bool = True,
extract_links: Literal['header', 'footer', 'body', 'all'] | None = None,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
storage_options: StorageOptions = None
) -> list[DataFrame]
Read HTML tables into a ``list`` of ``DataFrame`` objects.

Parameters
-----
io : str, path object, or file-like object
    String path, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a string ``read()`` function.
    The string can represent a URL. Note that
    lxml only accepts the http, ftp and file url protocols. If you have a
    URL that starts with ``'https'`` you might try removing the ``'s'``.

match : str or compiled regular expression, optional
    The set of tables containing text matching this regex or string will be
    returned. Unless the HTML is extremely simple you will probably need to
    pass a non-empty string here. Defaults to ``.+`` (match any non-empty
    string). The default value will return all tables contained on a page.
    This value is converted to a regular expression so that there is
    consistent behavior between BeautifulSoup and lxml.

flavor : [{"lxml", "html5lib", "bs4"}] or list-like, optional
    The parsing engine (or list of parsing engines) to use. 'bs4' and
    'html5lib' are synonymous with each other, they are both there for
    backwards compatibility. The default of ``None`` tries to use ``lxml``
    to parse and if that fails it falls back on ``bs4`` + ``html5lib``.

header : int or list-like, optional
    The row (or list of rows for a :class:`~pandas.MultiIndex`) to use to
    make the columns headers.

index_col : int or list-like, optional
    The column (or list of columns) to use to create the index.

skiprows : int, list-like or slice, optional
    Number of rows to skip after parsing the column integer. 0-based. If a
    sequence of integers or a slice is given, will skip the rows indexed by
    that sequence. Note that a single element sequence means 'skip the nth
    row' whereas an integer means 'skip n rows'.

attrs : dict, optional
    This is a dictionary of attributes that you can pass to use to identify
    the table in the HTML. These are not checked for validity before being
    passed to lxml or BeautifulSoup. However, these attributes must be
    valid HTML table attributes to work correctly. For example, ::

        attrs = {"id": "table"}

    is a valid attribute dictionary because the 'id' HTML tag attribute is
    a valid HTML attribute for *any* HTML tag as per `this document
    <https://html.spec.whatwg.org/multipage/dom.html#global-attributes>`_. ::

        attrs = {"asdf": "table"}

    is *not* a valid attribute dictionary because 'asdf' is not a valid
    HTML attribute even if it is a valid XML attribute. Valid HTML 4.01
    table attributes can be found `here
    <http://www.w3.org/TR/REC-html40/struct/tables.html#h-11.2>`_. A
    working draft of the HTML 5 spec can be found `here
    <https://html.spec.whatwg.org/multipage/tables.html>`_. It contains the
    latest information on table attributes for the modern web.

parse_dates : bool, optional
    See :func:`~read_csv` for more details.

thousands : str, optional
    Separator to use to parse thousands. Defaults to ``','``.

encoding : str, optional
    The encoding used to decode the web page. Defaults to ``None``. ``None``
    preserves the previous encoding behavior, which depends on the
    underlying parser library (e.g., the parser library will try to use
    the encoding provided by the document).

decimal : str, default '.'

```

Character to recognize as decimal point (e.g. use ',' for European data).

converters : dict, default None

Dict of functions for converting values in certain columns. Keys can either be integers or column labels, values are functions that take one input argument, the cell (not column) content, and return the transformed content.

na_values : iterable, default None

Custom NA values.

keep_default_na : bool, default True

If na_values are specified and keep_default_na is False the default NaN values are overridden, otherwise they're appended to.

displayed_only : bool, default True

Whether elements with "display: none" should be parsed.

extract_links : {{None, "all", "header", "body", "footer"}}

Table elements in the specified section(s) with <a> tags will have their href extracted.

dtype_backend : {{'numpy_nullable', 'pyarrow'}}

Back-end data type applied to the resultant :class:`DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

```
* ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
* ``"pyarrow"``: returns pyarrow-backed nullable
  :class:`ArrowDtype` :class:`DataFrame`
```

.. versionadded:: 2.0

storage_options : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

.. versionadded:: 2.1.0

Returns

dfs

A list of DataFrames.

See Also

read_csv : Read a comma-separated values (csv) file into DataFrame.

Notes

Before using this function you should read the :ref:`gotchas` about the HTML parsing libraries <io.html.gotchas>`.

Expect to do some cleanup after you call this function. For example, you might need to manually assign column names if the column names are converted to NaN when you pass the `header=0` argument. We try to assume as little as possible about the structure of the table and push the idiosyncrasies of the HTML contained in the table to the user.

This function searches for ``<table>`` elements and only for ``<tr>`` and ``<th>`` rows and ``<td>`` elements within each ``<tr>`` or ``<th>`` element in the table. ``<td>`` stands for "table data". This function attempts to properly handle ``colspan`` and ``rowspan`` attributes. If the function has a ``<thead>`` argument, it is used to construct the header, otherwise the function attempts to find the header within the body (by putting rows with only ``<th>`` elements into the header).

Similar to :func:`~read\_csv` the `header` argument is applied **after** `skiprows` is applied.

This function will **always** return a list of `:class:`DataFrame`` **or** it will fail, i.e., it will **not** return an empty list, save for some rare cases.
 It might return an empty list in case of inputs with single row and `<td>` containing only whitespaces.

Examples

See the `:ref:`read_html`` documentation in the IO section of the docs `<io.read_html>` for some examples of reading in HTML tables.

```
read_iceberg(
    table_identifier: str,
    catalog_name: str | None = None,
    *,
    catalog_properties: dict[str, Any] | None = None,
    columns: list[str] | None = None,
    row_filter: str | None = None,
    case_sensitive: bool = True,
    snapshot_id: int | None = None,
    limit: int | None = None,
    scan_properties: dict[str, Any] | None = None
) -> DataFrame
    Read an Apache Iceberg table into a pandas DataFrame.

    .. versionadded:: 3.0.0

    .. warning::

        read_iceberg is experimental and may change without warning.

Parameters
-----
table_identifier : str
    Table identifier.
catalog_name : str, optional
    The name of the catalog.
catalog_properties : dict of {str: str}, optional
    The properties that are used next to the catalog configuration.
columns : list of str, optional
    A list of strings representing the column names to return in the output
    dataframe.
row_filter : str, optional
    A string that describes the desired rows.
case_sensitive : bool, default True
    If True column matching is case sensitive.
snapshot_id : int, optional
    Snapshot ID to time travel to. By default the table will be scanned as of the
    current snapshot ID.
limit : int, optional
    An integer representing the number of rows to return in the scan result.
    By default all matching rows will be fetched.
scan_properties : dict of {str: obj}, optional
    Additional Table properties as a dictionary of string key value pairs to use
    for this scan.
```

Returns

`DataFrame`
 DataFrame based on the Iceberg table.

See Also

`read_parquet` : Read a Parquet file.

Examples

```
>>> df = pd.read_iceberg(
...     table_identifier="my_table",
...     catalog_name="my_catalog",
...     catalog_properties={"s3.secret-access-key": "my-secret"},
...     row_filter="trip_distance >= 10.0",
...     columns=["VendorID", "tpep_pickup_datetime"],
... ) # doctest: +SKIP
```

```
read_json(
    path_or_buf: FilePath | ReadBuffer[str] | ReadBuffer[bytes],
```

```

*,
orient: str | None = None,
typ: Literal['frame', 'series'] = 'frame',
dtype: DtypeArg | None = None,
convert_axes: bool | None = None,
convert_dates: bool | list[str] = True,
keep_default_dates: bool = True,
precise_float: bool = False,
date_unit: str | None = None,
encoding: str | None = None,
encoding_errors: str | None = 'strict',
lines: bool = False,
chunksize: int | None = None,
compression: CompressionOptions = 'infer',
nrows: int | None = None,
storage_options: StorageOptions | None = None,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
engine: JSONEngine = 'ujson'
) -> DataFrame | Series | JsonReader
Convert a JSON string to pandas object.

```

This method reads JSON files or JSON-like data and converts them into pandas objects. It supports a variety of input formats, including line-delimited JSON, compressed files, and various data representations (table, records, index-based, etc.). When `chunksize` is specified, an iterator is returned instead of loading the entire data into memory.

Parameters

path_or_buf : a str path, path object or file-like object
Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be:
`file://localhost/path/to/table.json`.

If you want to pass in a path object, pandas accepts any
`os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

orient : str, optional
Indication of expected JSON string format.
Compatible JSON strings can be produced by `to\_json()` with a corresponding orient value.
The set of possible orients is:

- `split` : dict like
`{index -> [index], columns -> [columns], data -> [values]}`
- `records` : list like
`[{column -> value}, ..., {column -> value}]`
- `index` : dict like `{index -> {column -> value}}`
- `columns` : dict like `{column -> {index -> value}}`
- `values` : just the values array
- `table` : dict like `{'schema': {schema}, 'data': {data}}`

The allowed and default values depend on the value of the `typ` parameter.

* when `typ == 'series'`,

- allowed orients are `{'split', 'records', 'index'}`
- default is `index`
- The Series index must be unique for orient `index`.

* when `typ == 'frame'`,

- allowed orients are `{'split', 'records', 'index', 'columns', 'values', 'table'}`
- default is `columns`
- The DataFrame index must be unique for orients `index` and `columns`.
- The DataFrame columns must be unique for orients `index`, `columns`, and `records`.

typ : `{'frame', 'series'}`, default 'frame'

The type of object to recover.

`dtype` : bool or dict, default None

If True, infer dtypes; if a dict of column to dtype, then use those;
if False, then don't infer dtypes at all, applies only to the data.

For all `orient` values except `'table'`, default is True.

`convert_axes` : bool, default None

Try to convert the axes to the proper dtypes.

For all `orient` values except `'table'`, default is True.

`convert_dates` : bool or list of str, default True

If True then default datelike columns may be converted (depending on
`keep_default_dates`).

If False, no dates will be converted.

If a list of column names, then those columns will be converted and
default datelike columns may also be converted (depending on
`keep_default_dates`).

`keep_default_dates` : bool, default True

If parsing dates (`convert_dates` is not False), then try to parse the
default datelike columns.

A column label is datelike if

- * it ends with `'_at'`,
- * it ends with `'_time'`,
- * it begins with `'timestamp'`,
- * it is `'modified'`, or
- * it is `'date'`.

`precise_float` : bool, default False

Set to enable usage of higher precision (`strtod`) function when
decoding string to double values. Default (False) is to use fast but
less precise builtin functionality.

`date_unit` : str, default None

The timestamp unit to detect if converting dates. The default behaviour
is to try and detect the correct precision, but if this is not desired
then pass one of 's', 'ms', 'us' or 'ns' to force parsing only seconds,
milliseconds, microseconds or nanoseconds respectively.

`encoding` : str, default is 'utf-8'

The encoding to use to decode py3 bytes.

`encoding_errors` : str, optional, default "strict"

How encoding errors are treated. List of possible values

<<https://docs.python.org/3/library/codecs.html#error-handlers>>`\_` .

`lines` : bool, default False

Read the file as a json object per line.

`chunksize` : int, optional

Return `JsonReader` object for iteration.

See the `'line-delimited json'` docs

<https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html#line-delimited-json>`\_`
for more information on `chunksize`.

This can only be passed if `lines=True`.

If this is None, the file will be read into memory all at once.

`compression` : str or dict, default 'infer'

For on-the-fly decompression of on-disk data. If 'infer' and `'path_or_buf'` is
path-like, then detect compression from the following extensions: '.gz',
'bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
(otherwise no compression).

If using 'zip' or 'tar', the ZIP file must contain only one data file to be
read in.

Set to `None` for no decompression.

Can also be a dict with key `'method'` set

to one of `{'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'}`
and other key-value pairs are forwarded to

`zipfile.ZipFile`, `gzip.GzipFile`,


```
``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or  
``tarfile.TarFile``, respectively.
```

As an example, the following could be passed for Zstandard decompression using a custom compression dictionary:

```
``compression={'method': 'zstd', 'dict_data': my_compression_dict}``.
```

`nrows` : int, optional

The number of lines from the line-delimited jsonfile that has to be read.
This can only be passed if `lines=True`.

If this is None, all the rows will be returned.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with `s3://`, and `gcs://`) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`dtype_backend` : `{'numpy_nullable', 'pyarrow'}`

Back-end data type applied to the resultant `:class:`DataFrame`` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

```
* ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`  
* ``"pyarrow"``: returns pyarrow-backed nullable  
  :class:`ArrowDtype` :class:`DataFrame`
```

.. versionadded:: 2.0

`engine` : `{'ujson', 'pyarrow'}`, default "ujson"

Parser engine to use. The `"pyarrow"` engine is only available when `lines=True`.

.. versionadded:: 2.0

Returns

Series, DataFrame, or `pandas.api.typing.JsonReader`

A `JsonReader` is returned when `chunksize` is not `0` or `None`.
Otherwise, the type returned depends on the value of `typ`.

See Also

`DataFrame.to_json` : Convert a DataFrame to a JSON string.

`Series.to_json` : Convert a Series to a JSON string.

`json_normalize` : Normalize semi-structured JSON data into a flat table.

Notes

Specific to `orient='table'`, if a `:class:`DataFrame`` with a literal `:class:`Index`` name of `index` gets written with `:func:`to_json``, the subsequent read operation will incorrectly set the `:class:`Index`` name to `None`. This is because `index` is also used by `:func:`DataFrame.to_json`` to denote a missing `:class:`Index`` name, and the subsequent `:func:`read_json`` operation cannot distinguish between the two. The same limitation is encountered with a `:class:`MultiIndex`` and any names beginning with `'level_'`.

Examples

```
>>> from io import StringIO  
>>> df = pd.DataFrame(  
...     [{"a", "b"}, {"c", "d"}],  
...     index=["row 1", "row 2"],  
...     columns=["col 1", "col 2"],  
... )
```

Encoding/decoding a Dataframe using `'split'` formatted JSON:

```
>>> df.to_json(orient="split")  
'{"columns":["col 1","col 2"],"index":["row 1","row 2"],"data":[["a","b"],["c","d"]}]'  
  
>>> pd.read_json(StringIO(_), orient="split") # noqa: F821
```

```

      col 1 col 2
row 1      a      b
row 2      c      d

```

Encoding/decoding a Dataframe using ``'index'`` formatted JSON:

```

>>> df.to_json(orient="index")
'{"row 1":{"col 1":"a","col 2":"b"},"row 2":{"col 1":"c","col 2":"d"}}'

>>> pd.read_json(StringIO(_), orient="index") # noqa: F821
      col 1 col 2
row 1      a      b
row 2      c      d

```

Encoding/decoding a Dataframe using ``'records'`` formatted JSON.
Note that index labels are not preserved with this encoding.

```

>>> df.to_json(orient="records")
'[{"col 1":"a","col 2":"b"}, {"col 1":"c","col 2":"d"}]'

>>> pd.read_json(StringIO(_), orient="records") # noqa: F821
      col 1 col 2
0          a      b
1          c      d

```

Encoding with Table Schema

```

>>> df.to_json(orient="table")
'{"schema":{"fields":[{"name":"index","type":"string","extDtype":"str"}, {"name":"col 1","type":

```

The following example uses ``dtype\_backend="numpy\_nullable"``

```

>>> data = '{"index": {"0": 0, "1": 1},
...         "a": {"0": 1, "1": null},
...         "b": {"0": 2.5, "1": 4.5},
...         "c": {"0": true, "1": false},
...         "d": {"0": "a", "1": "b"},
...         "e": {"0": 1577.2, "1": 1577.1}}'
>>> pd.read_json(StringIO(data), dtype_backend="numpy_nullable")
      index      a      b      c      d      e
0         0      1  2.5  True  a  1577.2
1         1  <NA>  4.5  False b  1577.1

```

```

read_orc(
    path: FilePath | ReadBuffer[bytes],
    columns: list[str] | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
    filesystem: pyarrow.fs.FileSystem | fsspec.spec.AbstractFileSystem | None = None,
    **kwargs: Any
) -> DataFrame
Load an ORC object from the file path, returning a DataFrame.

```

This method reads an ORC (Optimized Row Columnar) file into a pandas DataFrame using the `pyarrow.orc` library. ORC is a columnar storage format that provides efficient compression and fast retrieval for analytical workloads. It allows reading specific columns, handling different filesystem types (such as local storage, cloud storage via fsspec, or pyarrow filesystem), and supports different data type backends, including `numpy\_nullable` and `pyarrow`.

Parameters

```

-----
path : str, path object, or file-like object
      String, path object (implementing ``os.PathLike[str]``), or file-like
      object implementing a binary ``read()`` function. The string could be a URL.
      Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is
      expected. A local file could be:
      ``file://localhost/path/to/table.orc``.
columns : list, default None
      If not None, only these columns will be read from the file.
      Output always follows the ordering of the file and not the columns list.
      This mirrors the original behaviour of
      :external+pyarrow:py:meth:`pyarrow.orc.ORCFile.read`.
dtype_backend : {'numpy_nullable', 'pyarrow'}
      Back-end data type applied to the resultant :class:`DataFrame`
      (still experimental). If not specified, the default behavior
      is to not use nullable data types. If specified, the behavior
      is as follows:

```

```

* ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
* ``"pyarrow"``: returns pyarrow-backed nullable
  :class:`ArrowDtype` :class:`DataFrame`

.. versionadded:: 2.0

filesystem : fsspec or pyarrow filesystem, default None
  Filesystem object to use when reading the orc file.

.. versionadded:: 2.1.0

**kwargs
  Any additional kwargs are passed to pyarrow.

Returns
-----
DataFrame
  DataFrame based on the ORC file.

See Also
-----
read_csv : Read a comma-separated values (csv) file into a pandas DataFrame.
read_excel : Read an Excel file into a pandas DataFrame.
read_spss : Read an SPSS file into a pandas DataFrame.
read_sas : Load a SAS file into a pandas DataFrame.
read_feather : Load a feather-format object into a pandas DataFrame.

Notes
-----
Before using this function you should read the :ref:`user guide about ORC <io.orc>`
and :ref:`install optional dependencies <install.warn_orc>`.

If ``path`` is a URI scheme pointing to a local or remote file (e.g. "s3://"),
a ``pyarrow.fs`` filesystem will be attempted to read the file. You can also pass a
pyarrow or fsspec filesystem object into the filesystem keyword to override this
behavior.

Examples
-----
>>> result = pd.read_orc("example_pa.orc") # doctest: +SKIP

read_parquet(
    path: FilePath | ReadBuffer[bytes],
    engine: str = 'auto',
    columns: list[str] | None = None,
    storage_options: StorageOptions | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,
    filesystem: Any = None,
    filters: list[tuple] | list[list[tuple]] | None = None,
    to_pandas_kwargs: dict | None = None,
    **kwargs
) -> DataFrame
  Load a parquet object from the file path, returning a DataFrame.

The function automatically handles reading the data from a parquet file
and creates a DataFrame with the appropriate structure.

Parameters
-----
path : str, path object or file-like object
  String, path object (implementing ``os.PathLike[str]``), or file-like
  object implementing a binary ``read()`` function.
  The string could be a URL. Valid URL schemes include http, ftp, s3,
  gs, and file. For file URLs, a host is expected. A local file could be:
  ``file://localhost/path/to/table.parquet``.
  A file URL can also be a path to a directory that contains multiple
  partitioned parquet files. Both pyarrow and fastparquet support
  paths to directories as well as file URLs. A directory path could be:
  ``file://localhost/path/to/tables`` or ``s3://bucket/partition_dir``.
engine : {'auto', 'pyarrow', 'fastparquet'}, default 'auto'
  Parquet library to use. If 'auto', then the option
  ``io.parquet.engine`` is used. The default ``io.parquet.engine``
  behavior is to try 'pyarrow', falling back to 'fastparquet' if
  'pyarrow' is unavailable.

When using the ``'pyarrow'`` engine and no storage options are provided
and a filesystem is implemented by both ``pyarrow.fs`` and ``fsspec``

```

```

    (e.g. "s3://"), then the ``pyarrow.fs`` filesystem is attempted first.
    Use the filesystem keyword with an instantiated fsspec filesystem
    if you wish to use its implementation.
columns : list, default=None
    If not None, only these columns will be read from the file.
storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value
    pairs are forwarded to ``urllib.request.Request`` as header options.
    For other URLs (e.g. starting with "s3://", and "gcs://") the
    key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec``
    and ``urllib`` for more details, and for more examples on storage
    options refer `here <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`_.
dtype_backend : {'numpy_nullable', 'pyarrow'}
    Back-end data type applied to the resultant :class:`DataFrame`
    (still experimental). If not specified, the default behavior
    is to not use nullable data types. If specified, the behavior
    is as follows:

    * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
    * ``"pyarrow"``: returns pyarrow-backed nullable
      :class:`ArrowDtype` :class:`DataFrame`

    .. versionadded:: 2.0

filesystem : fsspec or pyarrow filesystem, default None
    Filesystem object to use when reading the parquet file. Only implemented
    for ``engine="pyarrow"``.

    .. versionadded:: 2.1.0

filters : List[Tuple] or List[List[Tuple]], default None
    To filter out data.
    Filter syntax: [[(column, op, val), ...],...]
    where op is [==, =, >, >=, <, <=, !=, in, not in]
    The innermost tuples are transposed into a set of filters applied
    through an `AND` operation.
    The outer list combines these sets of filters through an `OR`
    operation.
    A single list of tuples can also be used, meaning that no `OR`
    operation between set of filters is to be conducted.

    Using this argument will NOT result in row-wise filtering of the final
    partitions unless ``engine="pyarrow"`` is also specified. For
    other engines, filtering is only performed at the partition level, that is,
    to prevent the loading of some row-groups and/or files.

    .. versionadded:: 2.1.0

to_pandas_kwargs : dict | None, default None
    Keyword arguments to pass through to :func:`pyarrow.Table.to_pandas`
    when ``engine="pyarrow"``.

    .. versionadded:: 3.0.0

**kwargs
    Additional keyword arguments passed to the engine:

    * For ``engine="pyarrow"``: passed to :func:`pyarrow.parquet.read_table`
    * For ``engine="fastparquet"``: passed to
      :meth:`fastparquet.ParquetFile.to_pandas`

Returns
-----
DataFrame
    DataFrame based on parquet file.

See Also
-----
DataFrame.to_parquet : Create a parquet object that serializes a DataFrame.

Examples
-----
>>> original_df = pd.DataFrame({"foo": range(5), "bar": range(5, 10)})
>>> original_df
   foo  bar
0    0    5
1    1    6
2    2    7
3    3    8
4    4    9

```

```

0    0    5
1    1    6
2    2    7
3    3    8
4    4    9
>>> df_parquet_bytes = original_df.to_parquet()
>>> from io import BytesIO
>>> restored_df = pd.read_parquet(BytesIO(df_parquet_bytes))
>>> restored_df
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
>>> restored_df.equals(original_df)
True
>>> restored_bar = pd.read_parquet(BytesIO(df_parquet_bytes), columns=["bar"])
>>> restored_bar
   bar
0     5
1     6
2     7
3     8
4     9
>>> restored_bar.equals(original_df[["bar"]])
True

```

The function uses `kwargs` that are passed directly to the engine. In the following example, we use the `filters` argument of the pyarrow engine to filter the rows of the DataFrame.

Since `pyarrow` is the default engine, we can omit the `engine` argument. Note that the `filters` argument is implemented by the `pyarrow` engine, which can benefit from multithreading and also potentially be more economical in terms of memory.

```

>>> sel = [("foo", ">", 2)]
>>> restored_part = pd.read_parquet(BytesIO(df_parquet_bytes), filters=sel)
>>> restored_part
   foo  bar
0     3    8
1     4    9

```

```

read_pickle(
    filepath_or_buffer: FilePath | ReadPickleBuffer,
    compression: CompressionOptions = 'infer',
    storage_options: StorageOptions | None = None
) -> DataFrame | Series
Load pickled pandas object (or any object) from file and return unpickled object.

```

.. warning::

Loading pickled data received from untrusted sources can be unsafe. See [here](https://docs.python.org/3/library/pickle.html) <<https://docs.python.org/3/library/pickle.html>>`__.

Parameters

```

-----
filepath_or_buffer : str, path object, or file-like object
    String, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a binary ``readlines()`` function.
    Also accepts URL. URL is not limited to S3 and GCS.
compression : str or dict, default 'infer'
    For on-the-fly decompression of on-disk data. If 'infer' and
    'filepath_or_buffer' is path-like, then detect compression from the
    following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar',
    '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).
    If using 'zip' or 'tar', the ZIP file must contain only one data file
    to be read in.
    Set to ``None`` for no decompression.
    Can also be a dict with key ``'method'`` set
    to one of {``'zip'``, ``'gzip'``, ``'bz2'``, ``'zstd'``, ``'xz'``,
    ``'tar'``} and other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for Zstandard decompression

```

using a custom compression dictionary:
``compression={'method': 'zstd', 'dict_data': my_compression_dict}``.`
storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_`.`

Returns

object
 The unpickled pandas object (or any object) that was stored in file.

See Also

DataFrame.to_pickle : Pickle (serialize) DataFrame object to file.
Series.to_pickle : Pickle (serialize) Series object to file.
read_hdf : Read HDF5 file into a DataFrame.
read_sql : Read SQL query or database table into a DataFrame.
read_parquet : Load a parquet object, returning a DataFrame.

Notes

read_pickle is only guaranteed to be backwards compatible to pandas 1.0 provided the object was serialized with **to_pickle**.

Examples

```
>>> original_df = pd.DataFrame(
...     [{"foo": range(5), "bar": range(5, 10)}]
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
>>> pd.to_pickle(original_df, "./dummy.pkl") # doctest: +SKIP

>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
```

```
read_sas(
    filepath_or_buffer: FilePath | ReadBuffer[bytes],
    *,
    format: str | None = None,
    index: Hashable | None = None,
    encoding: str | None = None,
    chunksize: int | None = None,
    iterator: bool = False,
    compression: CompressionOptions = 'infer'
) -> DataFrame | SASReader
Read SAS files stored as either XPORT or SAS7BDAT format files.
```

Parameters

filepath_or_buffer : str, path object, or file-like object
 String, path object (implementing ``os.PathLike[str]``), or file-like object implementing a binary ``read()`` function. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be:
``file://localhost/path/to/table.sas7bdat``.
format : str `{'xport', 'sas7bdat'}` or None
 If None, file format is inferred from file extension. If 'xport' or 'sas7bdat', uses the corresponding format.
index : identifier of index column, defaults to None

Identifier of column that should be used as index of the DataFrame.
encoding : str, default is None
Encoding for text data. If None, text data are stored as raw bytes.
chunksize : int
Read file `chunksize` lines at a time, returns iterator.
iterator : bool, defaults to False
If True, returns an iterator for reading the file incrementally.
compression : str or dict, default 'infer'
For on-the-fly decompression of on-disk data. If 'infer' and 'filepath_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). Set to `None` for no decompression. Can also be a dict with key `method` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively. As an example, the following could be passed for faster compression and to create a reproducible gzip archive:
`compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}`.

Returns

DataFrame, SAS7BDATReader, or XportReader

DataFrame if iterator=False and chunksize=None, else SAS7BDATReader or XportReader, file format is inferred from file extension.

See Also

read_csv : Read a comma-separated values (csv) file into a DataFrame.

read_excel : Read an Excel file into a pandas DataFrame.

read_spss : Read an SPSS file into a pandas DataFrame.

read_orc : Load an ORC object into a pandas DataFrame.

read_feather : Load a feather-format object into a pandas DataFrame.

Examples

```
>>> df = pd.read_sas("sas_data.sas7bdat") # doctest: +SKIP
```

read_spss(

path: str | Path,

usecols: Sequence[str] | None = None,

convert_categoricals: bool = True,

dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,

**kwargs: Any

) -> DataFrame

Load an SPSS file from the file path, returning a DataFrame.

Parameters

path : str or Path

File path.

usecols : list-like, optional

Return a subset of the columns. If None, return all columns.

convert_categoricals : bool, default is True

Convert categorical columns into pd.Categorical.

dtype_backend : {'numpy_nullable', 'pyarrow'}

Back-end data type applied to the resultant :class:`DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

* ``"numpy\_nullable"``: returns nullable-dtype-backed :class:`DataFrame`

* ``"pyarrow"``: returns pyarrow-backed

nullable :class:`ArrowDtype` :class:`DataFrame`

.. versionadded:: 2.0

**kwargs

Additional keyword arguments that can be passed to :func:`pyreadstat.read\_sav`.

.. versionadded:: 3.0

Returns

DataFrame

DataFrame based on the SPSS file.

See Also

`read_csv` : Read a comma-separated values (csv) file into a pandas DataFrame.
`read_excel` : Read an Excel file into a pandas DataFrame.
`read_sas` : Read an SAS file into a pandas DataFrame.
`read_orc` : Load an ORC object into a pandas DataFrame.
`read_feather` : Load a feather-format object into a pandas DataFrame.

Examples

```
>>> df = pd.read_spss("spss_data.sav") # doctest: +SKIP
```

```
read_sql(  
    sql,  
    con,  
    index_col: str | list[str] | None = None,  
    coerce_float: bool = True,  
    params=None,  
    parse_dates=None,  
    columns: list[str] | None = None,  
    chunksize: int | None = None,  
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>,  
    dtype: DtypeArg | None = None  
) -> DataFrame | Iterator[DataFrame]  
Read SQL query or database table into a DataFrame.
```

This function is a convenience wrapper around `read_sql_table` and `read_sql_query` (for backward compatibility). It will delegate to the specific function depending on the provided input. A SQL query will be routed to `read_sql_query`, while a database table name will be routed to `read_sql_table`. Note that the delegated function might have more specific notes about their functionality not listed here.

Parameters

`sql` : str or SQLAlchemy Selectable (select or text object)
SQL query to be executed or a table name.
`con` : ADBC Connection, SQLAlchemy connectable, str, or sqlite3 connection
ADBC provides high performance I/O with native type support, where available. Using SQLAlchemy makes it possible to use any DB supported by that library. If a DBAPI2 object, only sqlite3 is supported. The user is responsible for engine disposal and connection closure for the ADBC connection and SQLAlchemy connectable; str connections are closed automatically. See [here <https://docs.sqlalchemy.org/en/20/core/connections.html>](https://docs.sqlalchemy.org/en/20/core/connections.html).
`index_col` : str or list of str, optional, default: None
Column(s) to set as index(MultiIndex).
`coerce_float` : bool, default True
Attempts to convert values of non-string, non-numeric objects (like decimal.Decimal) to floating point, useful for SQL result sets.
`params` : list, tuple or dict, optional, default: None
List of parameters to pass to execute method. The syntax used to pass parameters is database driver dependent. Check your database driver documentation for which of the five syntax styles, described in PEP 249's paramstyle, is supported.
Eg. for psycopg2, uses %(name)s so use params={'name' : 'value'}.
`parse_dates` : list or dict, default: None
- List of column names to parse as dates.
- Dict of {column_name: format string} where format string is strftime compatible in case of parsing string times, or is one of (D, s, ns, ms, us) in case of parsing integer timestamps.
- Dict of {column_name: arg dict}, where the arg dict corresponds to the keyword arguments of :func:`pandas.to\_datetime`
Especially useful with databases without native Datetime support, such as SQLite.
`columns` : list, default: None
List of column names to select from SQL table (only used when reading a table).
`chunksize` : int, default None
If specified, return an iterator where `chunksize` is the number of rows to include in each chunk.
`dtype_backend` : {'numpy_nullable', 'pyarrow'}
Back-end data type applied to the resultant :class:`DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

* ``"numpy\_nullable"``: returns nullable-dtype-backed :class:`DataFrame`


```

* ``"pyarrow"``: returns pyarrow-backed nullable
: class:`ArrowDtype` : class:`DataFrame`

.. versionadded:: 2.0
dtype : Type name or dict of columns
Data type for data or columns. E.g. np.float64 or
{'a': np.float64, 'b': np.int32, 'c': 'Int64'}.
The argument is ignored if a table is passed instead of a query.

.. versionadded:: 2.0.0

```

Returns

DataFrame or Iterator[DataFrame]

Returns a DataFrame object that contains the result set of the executed SQL query or an SQL Table based on the provided input, in relation to the specified database connection.

See Also

read_sql_table : Read SQL database table into a DataFrame.

read_sql_query : Read SQL query into a DataFrame.

Notes

``pandas`` does not attempt to sanitize SQL statements; instead it simply forwards the statement you are executing to the underlying driver, which may or may not sanitize from there. Please refer to the underlying driver documentation for any details. Generally, be wary when accepting statements from arbitrary sources.

Examples

Read data from SQL via either a SQL query or a SQL tablename.

When using a SQLite database only SQL queries are accepted, providing only the SQL tablename will result in an error.

```

>>> from sqlite3 import connect
>>> conn = connect(":memory:")
>>> df = pd.DataFrame(
...     data=[[0, "10/11/12"], [1, "12/11/10"]],
...     columns=["int_column", "date_column"],
... )
>>> df.to_sql(name="test_data", con=conn)
2

>>> pd.read_sql("SELECT int_column, date_column FROM test_data", conn)
   int_column date_column
0           0    10/11/12
1           1    12/11/10

```

```

>>> pd.read_sql("test_data", "postgres:///db_name") # doctest:+SKIP

```

For parameterized query, using ``params`` is recommended over string interpolation.

```

>>> from sqlalchemy import text
>>> sql = text(
...     "SELECT int_column, date_column FROM test_data WHERE int_column=:int_val"
... )
>>> pd.read_sql(sql, conn, params={"int_val": 1}) # doctest:+SKIP
   int_column date_column
0           1    12/11/10

```

Apply date parsing to columns through the ``parse\_dates`` argument

The ``parse\_dates`` argument calls ``pd.to\_datetime`` on the provided columns. Custom argument values for applying ``pd.to\_datetime`` on a column are specified via a dictionary format:

```

>>> pd.read_sql(
...     "SELECT int_column, date_column FROM test_data",
...     conn,
...     parse_dates={"date_column": {"format": "%d/%m/%y"}},
... )
   int_column date_column
0           0  2012-11-10
1           1  2010-11-12
.. versionadded:: 2.2.0

```

pandas now supports reading via ADBC drivers

```
>>> from adbc_driver_postgresql import dbapi # doctest:+SKIP
>>> with dbapi.connect("postgres:///db_name") as conn: # doctest:+SKIP
...     pd.read_sql("SELECT int_column FROM test_data", conn)
      int_column
0              0
1              1

read_sql_query(
    sql,
    con,
    index_col: str | list[str] | None = None,
    coerce_float: bool = True,
    params: list[Any] | Mapping[str, Any] | None = None,
    parse_dates: list[str] | dict[str, str] | dict[str, dict[str, Any]] | None = None,
    chunksize: int | None = None,
    dtype: DtypeArg | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame | Iterator[DataFrame]
Read SQL query into a DataFrame.
```

Returns a DataFrame corresponding to the result set of the query string. Optionally provide an `index\_col` parameter to use one of the columns as the index, otherwise default integer index will be used.

Parameters

sql : str SQL query or SQLAlchemy Selectable (select or text object)
SQL query to be executed.

con : SQLAlchemy connectable, str, or sqlite3 connection
Using SQLAlchemy makes it possible to use any DB supported by that library. If a DBAPI2 object, only sqlite3 is supported.

index_col : str or list of str, optional, default: None
Column(s) to set as index(MultiIndex).

coerce_float : bool, default True
Attempts to convert values of non-string, non-numeric objects (like decimal.Decimal) to floating point. Useful for SQL result sets.

params : list, tuple or mapping, optional, default: None
List of parameters to pass to execute method. The syntax used to pass parameters is database driver dependent. Check your database driver documentation for which of the five syntax styles, described in PEP 249's paramstyle, is supported.
Eg. for psycopg2, uses %(name)s so use params={'name' : 'value'}.

parse_dates : list or dict, default: None

- List of column names to parse as dates.
- Dict of `{column\_name: format string}` where format string is strftime compatible in case of parsing string times, or is one of (D, s, ns, ms, us) in case of parsing integer timestamps.
- Dict of `{column\_name: arg dict}`, where the arg dict corresponds to the keyword arguments of :func:`pandas.to\_datetime`
Especially useful with databases without native Datetime support, such as SQLite.

chunksize : int, default None
If specified, return an iterator where `chunksize` is the number of rows to include in each chunk.

dtype : Type name or dict of columns
Data type for data or columns. E.g. np.float64 or {'a': np.float64, 'b': np.int32, 'c': 'Int64'}.

dtype_backend : {'numpy_nullable', 'pyarrow'}
Back-end data type applied to the resultant :class:`DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- * ``"numpy\_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
- * ``"pyarrow"``: returns pyarrow-backed nullable :class:`ArrowDtype` :class:`DataFrame`

.. versionadded:: 2.0

Returns

DataFrame or Iterator[DataFrame]
Returns a DataFrame object that contains the result set of the executed SQL query, in relation to the specified database connection.

See Also

```

-----
read_sql_table : Read SQL database table into a DataFrame.
read_sql : Read SQL query or database table into a DataFrame.

```

Notes

```
-----
```

Any datetime values with time zone information parsed via the ``parse_dates`` parameter will be converted to UTC.

Examples

```
-----
```

```

>>> from sqlalchemy import create_engine # doctest: +SKIP
>>> engine = create_engine("sqlite:///database.db") # doctest: +SKIP
>>> sql_query = "SELECT int_column FROM test_data" # doctest: +SKIP
>>> with engine.connect() as conn, conn.begin(): # doctest: +SKIP
...     data = pd.read_sql_query(sql_query, conn) # doctest: +SKIP

```

```

read_sql_table(
    table_name: str,
    con,
    schema: str | None = None,
    index_col: str | list[str] | None = None,
    coerce_float: bool = True,
    parse_dates: list[str] | dict[str, str] | dict[str, dict[str, Any]] | None = None,
    columns: list[str] | None = None,
    chunksize: int | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame | Iterator[DataFrame]
    Read SQL database table into a DataFrame.

```

Given a table name and a SQLAlchemy connectable, returns a DataFrame. This function does not support DBAPI connections.

Parameters

```
-----
```

```

table_name : str
    Name of SQL table in database.
con : SQLAlchemy connectable or str
    A database URI could be provided as str.
    SQLite DBAPI connection mode not supported.
schema : str, default None
    Name of SQL schema in database to query (if database flavor
    supports this). Uses default schema if None (default).
index_col : str or list of str, optional, default: None
    Column(s) to set as index(MultiIndex).
coerce_float : bool, default True
    Attempts to convert values of non-string, non-numeric objects (like
    decimal.Decimal) to floating point. Can result in loss of Precision.
parse_dates : list or dict, default None
    - List of column names to parse as dates.
    - Dict of ``{column_name: format string}`` where format string is
      strftime compatible in case of parsing string times or is one of
      (D, s, ns, ms, us) in case of parsing integer timestamps.
    - Dict of ``{column_name: arg dict}``, where the arg dict corresponds
      to the keyword arguments of :func:`pandas.to_datetime`
      Especially useful with databases without native Datetime support,
      such as SQLite.
columns : list, default None
    List of column names to select from SQL table.
chunksize : int, default None
    If specified, returns an iterator where `chunksize` is the number of
    rows to include in each chunk.
dtype_backend : {'numpy_nullable', 'pyarrow'}
    Back-end data type applied to the resultant :class:`DataFrame`
    (still experimental). If not specified, the default behavior
    is to not use nullable data types. If specified, the behavior
    is as follows:

    * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
    * ``"pyarrow"``: returns pyarrow-backed nullable
      :class:`ArrowDtype` :class:`DataFrame`

    .. versionadded:: 2.0

```

Returns

```
-----
```

DataFrame or Iterator[DataFrame]

A SQL table is returned as two-dimensional data structure with labeled axes.

See Also

read_sql_query : Read SQL query into a DataFrame.
read_sql : Read SQL query or database table into a DataFrame.

Notes

Any datetime values with time zone information will be converted to UTC.

Examples

>>> pd.read_sql_table("table_name", "postgres:///db_name") # doctest:+SKIP

```
read_stata(  
    filepath_or_buffer: FilePath | ReadBuffer[bytes],  
    *,  
    convert_dates: bool = True,  
    convert_categoricals: bool = True,  
    index_col: str | None = None,  
    convert_missing: bool = False,  
    preserve_dtypes: bool = True,  
    columns: Sequence[str] | None = None,  
    order_categoricals: bool = True,  
    chunksize: int | None = None,  
    iterator: bool = False,  
    compression: CompressionOptions = 'infer',  
    storage_options: StorageOptions | None = None  
) -> DataFrame | StataReader  
Read Stata file into DataFrame.
```

Parameters

filepath_or_buffer : str, path object or file-like object
Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be: ``file://localhost/path/to/table.dta``.

If you want to pass in a path object, pandas accepts any ``os.PathLike``.

By file-like object, we refer to objects with a ``read()`` method, such as a file handle (e.g. via builtin ``open`` function) or ``StringIO``.
convert_dates : bool, default True
Convert date variables to DataFrame time values.
convert_categoricals : bool, default True
Read value labels and convert columns to Categorical/Factor variables.
index_col : str, optional
Column to set as index.
convert_missing : bool, default False
Flag indicating whether to convert missing values to their Stata representations. If False, missing values are replaced with nan. If True, columns containing missing values are returned with object data types and missing values are represented by StataMissingValue objects.
preserve_dtypes : bool, default True
Preserve Stata datatypes. If False, numeric data are upcast to pandas default types for foreign data (float64 or int64).
columns : list or None
Columns to retain. Columns will be returned in the given order. None returns all columns.
order_categoricals : bool, default True
Flag indicating whether converted categorical data are ordered.
chunksize : int, default None
Return StataReader object for iterations, returns chunks with given number of lines.
iterator : bool, default False
Return StataReader object.
compression : str or dict, default 'infer'
For on-the-fly decompression of on-disk data. If 'infer' and 'filepath_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in. Set to ``None`` for no decompression.

Can also be a dict with key ``method`` set to one of
 {``zip``, ``gzip``, ``bz2``, ``zstd``, ``xz``, ``tar``} and
 other key-value pairs are forwarded to
 ``zipfile.ZipFile``, ``gzip.GzipFile``,
 ``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
 ``tarfile.TarFile``, respectively.

As an example, the following could be passed for Zstandard decompression using a
 custom compression dictionary:

```
``compression={'method': 'zstd', 'dict_data': my_compression_dict}``.
storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`_.
```

Returns

DataFrame, pandas.api.typing.StataReader
 If iterator or chunksize, returns StataReader, else DataFrame.

See Also

io.stata.StataReader : Low-level reader for Stata data files.
 DataFrame.to_stata: Export Stata data files.

Notes

Categorical variables read through an iterator may not have the same
 categories and dtype. This occurs when a variable stored in a DTA
 file is associated to an incomplete set of value labels that only
 label a strict subset of the values.

Examples

Creating a dummy stata for this example

```
>>> df = pd.DataFrame(
...     {
...         "animal": ["falcon", "parrot", "falcon", "parrot"],
...         "speed": [350, 18, 361, 15],
...     }
... ) # doctest: +SKIP
>>> df.to_stata("animals.dta") # doctest: +SKIP
```

Read a Stata dta file:

```
>>> df = pd.read_stata("animals.dta") # doctest: +SKIP
```

Read a Stata dta file in 10,000 line chunks:

```
>>> values = np.random.randint(
...     0, 10, size=(20_000, 1), dtype="uint8"
... ) # doctest: +SKIP
>>> df = pd.DataFrame(values, columns=["i"]) # doctest: +SKIP
>>> df.to_stata("filename.dta") # doctest: +SKIP

>>> with pd.read_stata('filename.dta', chunksize=10000) as itr: # doctest: +SKIP
>>>     for chunk in itr:
...     # Operate on a single chunk, e.g., chunk.mean()
...     pass # doctest: +SKIP
```

```
read_table(
    filepath_or_buffer: FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str],
    *,
    sep: str | None | lib.NoDefault = <no_default>,
    delimiter: str | None | lib.NoDefault = None,
    header: int | Sequence[int] | None | Literal['infer'] = 'infer',
    names: Sequence[Hashable] | None | lib.NoDefault = <no_default>,
    index_col: IndexLabel | Literal[False] | None = None,
    usecols: UsecolsArgType = None,
    dtype: DtypeArg | None = None,
    engine: CSVEngine | None = None,
```

```

converters: Mapping[HashableT, Callable] | None = None,
true_values: list | None = None,
false_values: list | None = None,
skipinitialspace: bool = False,
skiprows: list[int] | int | Callable[[Hashable], bool] | None = None,
skipfooter: int = 0,
nrows: int | None = None,
na_values: Hashable | Iterable[Hashable] | Mapping[Hashable, Iterable[Hashable]] | None = None,
keep_default_na: bool = True,
na_filter: bool = True,
skip_blank_lines: bool = True,
parse_dates: bool | Sequence[Hashable] | None = None,
date_format: str | dict[Hashable, str] | None = None,
dayfirst: bool = False,
cache_dates: bool = True,
iterator: bool = False,
chunksize: int | None = None,
compression: CompressionOptions = 'infer',
thousands: str | None = None,
decimal: str = '.',
lineterminator: str | None = None,
quotechar: str = '"',
quoting: int = 0,
doublequote: bool = True,
escapechar: str | None = None,
comment: str | None = None,
encoding: str | None = None,
encoding_errors: str | None = 'strict',
dialect: str | csv.Dialect | None = None,
on_bad_lines: str = 'error',
low_memory: bool = True,
memory_map: bool = False,
float_precision: Literal['high', 'legacy', 'round_trip'] | None = None,
storage_options: StorageOptions | None = None,
dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame | TextFileReader
Read general delimited file into DataFrame.

```

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for
`IO Tools` <https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html>`_.

Parameters

```

-----
filepath_or_buffer : str, path object or file-like object
    Any valid string path is acceptable. The string could be a URL. Valid
    URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is
    expected. A local file could be: file://localhost/path/to/table.csv.

    If you want to pass in a path object, pandas accepts any ``os.PathLike``.

    By file-like object, we refer to objects with a ``read()`` method, such as
    a file handle (e.g. via builtin ``open`` function) or ``StringIO``.
sep : str, default '\t' (tab-stop)
    Character or regex pattern to treat as the delimiter. If ``sep=None``, the
    C engine cannot automatically detect
    the separator, but the Python parsing engine can, meaning the latter will
    be used and automatically detect the separator from only the first valid
    row of the file by Python's builtin sniffer tool, ``csv.Sniffer``.
    In addition, separators longer than 1 character and different from
    ``'\s+'`` will be interpreted as regular expressions and will also force
    the use of the Python parsing engine. Note that regex delimiters are prone
    to ignoring quoted data. Regex example: ``'\r\t'``.
delimiter : str, optional
    Alias for ``sep``.
header : int, Sequence of int, 'infer' or None, default 'infer'
    Row number(s) containing column labels and marking the start of the
    data (zero-indexed). Default behavior
    is to infer the column names: if no ``names``
    are passed the behavior is identical to ``header=0`` and column
    names are inferred from the first line of the file, if column
    names are passed explicitly to ``names`` then the behavior is identical to
    ``header=None``. Explicitly pass ``header=0`` to be able to
    replace existing names. The header can be a list of integers that
    specify row locations for a :class:`~pandas.MultiIndex` on the columns

```

e.g. ``[0, 1, 3]``. Intervening rows that are not specified will be skipped (e.g. 2 in this example is skipped). Note that this parameter ignores commented lines and empty lines if ``skip\_blank\_lines=True``, so ``header=0`` denotes the first line of data rather than the first line of the file.

When inferred from the file contents, headers are kept distinct from each other by renaming duplicate names with a numeric suffix of the form ``".{count}"`` starting from 1, e.g. ``"foo"`` and ``"foo.1"``.

Empty headers are named ``"Unnamed: {i}"`` or ``"Unnamed: {i}\_level\_{{level}}"`` in the case of MultiIndex columns.

names : Sequence of Hashable, optional
Sequence of column labels to apply. If the file contains a header row, then you should explicitly pass ``header=0`` to override the column names. Duplicates in this list are not allowed.

index_col : Hashable, Sequence of Hashable or False, optional
Column(s) to use as row label(s), denoted either by column labels or column indices. If a sequence of labels or indices is given, :class:`~pandas.MultiIndex` will be formed for the row labels.

Note: ``index\_col=False`` can be used to force pandas to *not* use the first column as the index, e.g., when you have a malformed file with delimiters at the end of each line.

usecols : Sequence of Hashable or Callable, optional
Subset of columns to select, denoted either by column labels or column indices. If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in ``names`` or inferred from the document header row(s). If ``names`` are given, the document header row(s) are not taken into account. For example, a valid list-like ``usecols`` parameter would be ``[0, 1, 2]`` or ``['foo', 'bar', 'baz']``. Element order is ignored, so ``usecols=[0, 1]`` is the same as ``[1, 0]``. To instantiate a :class:`~pandas.DataFrame` from ``data`` with element order preserved use ``pd.read\_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]`` for columns in ``['foo', 'bar']`` order or ``pd.read\_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]`` for ``['bar', 'foo']`` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to ``True``. An example of a valid callable argument would be ``lambda x: x.upper() in ['AAA', 'BBB', 'DDD']``. Using this parameter results in much faster parsing time and lower memory usage.

dtype : dtype or dict of {{Hashable : dtype}}, optional
Data type(s) to apply to either the whole dataset or individual columns. E.g., ``{'a': np.float64, 'b': np.int32, 'c': 'Int64'}``. Use ``str`` or ``object`` together with suitable ``na\_values`` settings to preserve and not interpret ``dtype``. If ``converters`` are specified, they will be applied INSTEAD of ``dtype`` conversion. Specify a ``defaultdict`` as input where the default determines the ``dtype`` of the columns which are not explicitly listed.

engine : {{'c', 'python', 'pyarrow'}}, optional
Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. Multithreading is currently only supported by the pyarrow engine. The 'pyarrow' engine is an *experimental* engine, and some features are unsupported, or may not work correctly, with this engine.

converters : dict of {{Hashable : Callable}}, optional
Functions for converting values in specified columns. Keys can either be column labels or column indices.

true_values : list, optional
Values to consider as ``True`` in addition to case-insensitive variants of 'True'.

false_values : list, optional
Values to consider as ``False`` in addition to case-insensitive variants of 'False'.

skipinitialspace : bool, default False
Skip spaces after delimiter.

skiprows : int, list of int or Callable, optional
Line numbers to skip (0-indexed) or number of lines to skip (``int``) at the start of the file.

If callable, the callable function will be evaluated against the row

indices, returning ``True`` if the row should be skipped and ``False`` otherwise.
An example of a valid callable argument would be ``lambda x: x in [0, 2]``.

skipfooter : int, default 0
Number of lines at bottom of file to skip (Unsupported with ``engine='c'``).

nrows : int, optional
Number of rows of file to read. Useful for reading pieces of large files.
Refers to the number of data rows in the returned DataFrame, excluding:

- * The header row containing column names.
- * Rows before the header row, if ``header=1`` or larger.

Example usage:

- * To read the first 999,999 (non-header) rows:
``read\_csv(..., nrows=999999)``
- * To read rows 1,000,000 through 1,999,999:
``read\_csv(..., skiprows=1000000, nrows=999999)``

na_values : Hashable, Iterable of Hashable or dict of {{Hashable : Iterable}}, optional
Additional strings to recognize as ``NA``/``NaN``.
If ``dict`` passed, specific per-column ``NA`` values. By default the following values are interpreted as ``NaN``: empty string, "NaN", "N/A", "NULL", and other common representations of missing data.

keep_default_na : bool, default True
Whether or not to include the default ``NaN`` values when parsing the data.
Depending on whether ``na\_values`` is passed in, the behavior is as follows:

- * If ``keep\_default\_na`` is ``True``, and ``na\_values`` are specified, ``na\_values`` is appended to the default ``NaN`` values used for parsing.
- * If ``keep\_default\_na`` is ``True``, and ``na\_values`` are not specified, only the default ``NaN`` values are used for parsing.
- * If ``keep\_default\_na`` is ``False``, and ``na\_values`` are specified, only the ``NaN`` values specified in ``na\_values`` are used for parsing.
- * If ``keep\_default\_na`` is ``False``, and ``na\_values`` are not specified, no strings will be parsed as ``NaN``.

Note that if ``na\_filter`` is passed in as ``False``, the ``keep\_default\_na`` and ``na\_values`` parameters will be ignored.

na_filter : bool, default True
Detect missing value markers (empty strings and the value of ``na\_values``). In data without any ``NA`` values, passing ``na\_filter=False`` can improve the performance of reading a large file.

skip_blank_lines : bool, default True
If ``True``, skip over blank lines rather than interpreting as ``NaN`` values.

parse_dates : bool, None, list of Hashable, default None
The behavior is as follows:

- * ``bool``. If ``True`` -> try parsing the index.
- * ``None``. Behaves like ``True`` if ``date\_format`` is specified.
- * ``list`` of ``int`` or names.
e.g. If ``[1, 2, 3]`` -> try parsing columns 1, 2, 3 each as a separate date column.

If a column or index cannot be represented as an array of ``datetime``, say because of an unparsable value or a mixture of timezones, the column or index will be returned unaltered as an ``object`` data type. For non-standard ``datetime`` parsing, use :func:`~pandas.to\_datetime` after :func:`~pandas.read\_csv`.

Note: A fast-path exists for iso8601-formatted dates.

date_format : str or dict of column -> format, optional
Format to use for parsing dates and/or times when used in conjunction with ``parse\_dates``.
The strftime to parse time, e.g. :const:`"%d/%m/%Y"`. See <https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior> for more information on choices, though note that :const:`"%f"`` will parse all the way up to nanoseconds.
You can also pass:

- "ISO8601", to parse any ISO8601 https://en.wikipedia.org/wiki/ISO_8601


```

        time string (not necessarily in exactly the same format);
        - "mixed", to infer the format for each element individually. This is risky,
          and you should probably use it along with `dayfirst`.

    .. versionadded:: 2.0.0
dayfirst : bool, default False
    DD/MM format dates, international and European format.
cache_dates : bool, default True
    If ``True``, use a cache of unique, converted dates to apply the ``datetime``
    conversion. May produce significant speed-up when parsing duplicate
    date strings, especially ones with timezone offsets.

iterator : bool, default False
    Return ``TextFileReader`` object for iteration or getting chunks with
    ``get_chunk()``.
chunksize : int, optional
    Number of lines to read from the file per chunk. Passing a value will cause the
    function to return a ``TextFileReader`` object for iteration.
    See the `IO Tools docs
    <https://pandas.pydata.org/pandas-docs/stable/io.html#io-chunking>`_
    for more information on ``iterator`` and ``chunksize``.

compression : str or dict, default 'infer'
    For on-the-fly decompression of on-disk data. If 'infer'
    and 'filepath_or_buffer' is
    path-like, then detect compression from the following extensions: '.gz',
    '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
    (otherwise no compression).
    If using 'zip' or 'tar', the ZIP file must contain
    only one data file to be read in.
    Set to ``None`` for no decompression.
    Can also be a dict with key ``'method'`` set
    to one of {``'zip'``, ``'gzip'``, ``'bz2'``,
    ``'zstd'``, ``'xz'``, ``'tar'``} and
    other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for
    Zstandard decompression using a
    custom compression dictionary:
    ``compression={'method': 'zstd', 'dict_data': my_compression_dict}``.

thousands : str (length 1), optional
    Character acting as the thousands separator in numerical values.
decimal : str (length 1), default '.'
    Character to recognize as decimal point (e.g., use ',' for European data).
lineterminator : str (length 1), optional
    Character used to denote a line break. Only valid with C parser.
quotechar : str (length 1), optional
    Character used to denote the start and end of a quoted item. Quoted
    items can include the ``delimiter`` and it will be ignored.
quoting : {0 or csv.QUOTE_MINIMAL, 1 or csv.QUOTE_ALL, 2 or
    csv.QUOTE_NONNUMERIC, 3 or csv.QUOTE_NONE}}, default csv.QUOTE_MINIMAL
    Control field quoting behavior per ``csv.QUOTE_*`` constants. Default is
    ``csv.QUOTE_MINIMAL`` (i.e., 0) which
    implies that only fields containing special
    characters are quoted (e.g., characters defined
    in ``quotechar``, ``delimiter``,
    or ``lineterminator``.
doublequote : bool, default True
    When ``quotechar`` is specified and ``quoting`` is not ``QUOTE_NONE``, indicate
    whether or not to interpret two consecutive ``quotechar`` elements INSIDE a
    field as a single ``quotechar`` element.
escapechar : str (length 1), optional
    Character used to escape other characters.
comment : str (length 1), optional
    Character indicating that the remainder of line should not be parsed.
    If found at the beginning
    of a line, the line will be ignored altogether. This parameter must be a
    single character. Like empty lines (as long as ``skip_blank_lines=True``),
    fully commented lines are ignored by the parameter ``header`` but not by
    ``skiprows``. For example, if ``comment='#'``, parsing
    ``#empty\na,b,c\n1,2,3`` with ``header=0`` will result in ``'a,b,c'`` being
    treated as the header.
encoding : str, optional, default 'utf-8'
    Encoding to use for UTF when reading/writing (ex. ``'utf-8'``). `List of Python

```

```

standard_encodings
<https://docs.python.org/3/library/codecs.html#standard-encodings>`_ .

encoding_errors : str, optional, default 'strict'
    How encoding errors are treated. `List of possible values
    <https://docs.python.org/3/library/codecs.html#error-handlers>`_ .

dialect : str or csv.Dialect, optional
    If provided, this parameter will override values (default or not) for the
    following parameters: ``delimiter``, ``doublequote``, ``escapechar``,
    ``skipinitialspace``, ``quotechar``, and ``quoting``. If it is necessary to
    override values, a ``ParserWarning`` will be issued. See ``csv.Dialect``
    documentation for more details.
on_bad_lines : {'error', 'warn', 'skip'} or Callable, default 'error'
    Specifies what to do upon encountering a bad
    line (a line with too many fields).
    Allowed values are:

    - ``'error'``, raise an Exception when a bad line is encountered.
    - ``'warn'``, raise a warning when a bad line is encountered and
      skip that line.
    - ``'skip'``, skip bad lines without raising or warning when they
      are encountered.
    - Callable, function that will process a single bad line.
      - With ``engine='python'``, function with signature
        ``(bad_line: list[str]) -> list[str] | None``.
        ``bad_line`` is a list of strings split by the ``sep``.
        If the function returns ``None``, the bad line will be ignored.
        If the function returns a new ``list`` of strings with more elements than
        expected, a ``ParserWarning`` will be emitted while
        dropping extra elements.
      - With ``engine='pyarrow'``, function with signature
        as described in pyarrow documentation: ``invalid_row_handler``
        <https://arrow.apache.org/docs/
        python/generated/pyarrow.csv.ParseOptions.html
        #pyarrow.csv.ParseOptions.invalid\_row\_handler>`_ .

.. versionadded:: 2.2.0

    Callable for ``engine='pyarrow'``

low_memory : bool, default True
    Internally process the file in chunks, resulting in lower memory use
    while parsing, but possibly mixed type inference. To ensure no mixed
    types either set ``False``, or specify the type with the ``dtype`` parameter.
    Note that the entire file is read into a single :class:`~pandas.DataFrame`
    regardless, use the ``chunksize`` or ``iterator`` parameter
    to return the data in
    chunks. (Only valid with C parser).
memory_map : bool, default False
    If a filepath is provided for ``filepath_or_buffer``, map the file object
    directly onto memory and access the data directly from there. Using this
    option can improve performance because there is no longer any I/O overhead.
float_precision : {'high', 'legacy', 'round_trip'}, optional
    Specifies which converter the C engine should use for floating-point
    values. The options are ``None`` or ``'high'`` for the ordinary converter,
    ``'legacy'`` for the original lower precision pandas converter, and
    ``'round_trip'`` for the round-trip converter.

storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user\_guide/io.html?
    highlight=storage\_options#reading-writing-remote-files>`_ .

dtype_backend : {'numpy_nullable', 'pyarrow'}
    Back-end data type applied to the resultant :class:`DataFrame`
    (still experimental). If not specified, the default behavior
    is to not use nullable data types. If specified, the behavior
    is as follows:

    * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
    * ``"pyarrow"``: returns pyarrow-backed nullable

```

```

        :class:`ArrowDtype` :class:`DataFrame`

    .. versionadded:: 2.0

Returns
-----
DataFrame or TextFileReader
    A comma-separated values (csv) file is returned as two-dimensional
    data structure with labeled axes.

See Also
-----
DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.
read_csv : Read a comma-separated values (csv) file into DataFrame.
read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Examples
-----
>>> pd.read_table("data.csv") # doctest: +SKIP
   Name  Value
0   foo      1
1   bar      2
2  #baz      3

Index and header can be specified via the `index_col` and `header` arguments.

>>> pd.read_table("data.csv", header=None) # doctest: +SKIP
   0      1
0  Name  Value
1   foo      1
2   bar      2
3  #baz      3

>>> pd.read_table("data.csv", index_col="Value") # doctest: +SKIP
   Name
Value
1     foo
2     bar
3    #baz

Column types are inferred but can be explicitly specified using the dtype argument.

>>> pd.read_table("data.csv", dtype={"Value": float}) # doctest: +SKIP
   Name  Value
0   foo    1.0
1   bar    2.0
2  #baz    3.0

True, False, and NA values, and thousands separators have defaults,
but can be explicitly specified, too. Supply the values you would like
as strings or lists of strings!

>>> pd.read_table("data.csv", na_values=["foo", "bar"]) # doctest: +SKIP
   Name  Value
0   NaN      1
1   NaN      2
2  #baz      3

Comment lines in the input file can be skipped using the `comment` argument.

>>> pd.read_table("data.csv", comment="#") # doctest: +SKIP
   Name  Value
0   foo      1
1   bar      2

By default, columns with dates will be read as ``object`` rather than ``datetime``.

>>> df = pd.read_table("tmp.csv") # doctest: +SKIP

>>> df # doctest: +SKIP
   col 1      col 2      col 3
0      10  10/04/2018  Sun 15 Jan 2023
1      20  15/04/2018  Fri 12 May 2023

>>> df.dtypes # doctest: +SKIP
col 1    int64
col 2    object

```

```
col 3    object
dtype: object
```

Specific columns can be parsed as dates by using the `'parse_dates'` and `'date_format'` arguments.

```
>>> df = pd.read_table(
...     "tmp.csv",
...     parse_dates=[1, 2],
...     date_format={"col 2": "%d/%m/%Y", "col 3": "%a %d %b %Y"}},
... ) # doctest: +SKIP

>>> df.dtypes # doctest: +SKIP
col 1          int64
col 2    datetime64[ns]
col 3    datetime64[ns]
dtype: object
```

```
read_xml(
    path_or_buffer: FilePath | ReadBuffer[bytes] | ReadBuffer[str],
    *,
    xpath: str = './*',
    namespaces: dict[str, str] | None = None,
    elems_only: bool = False,
    attrs_only: bool = False,
    names: Sequence[str] | None = None,
    dtype: DtypeArg | None = None,
    converters: ConvertersArg | None = None,
    parse_dates: ParseDatesArg | None = None,
    encoding: str | None = 'utf-8',
    parser: XMLParsers = 'lxml',
    stylesheet: FilePath | ReadBuffer[bytes] | ReadBuffer[str] | None = None,
    iterparse: dict[str, list[str]] | None = None,
    compression: CompressionOptions = 'infer',
    storage_options: StorageOptions | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
) -> DataFrame
Read XML document into a :class:`~pandas.DataFrame` object.

Parameters
-----
path_or_buffer : str, path object, or file-like object
    String path, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a ``read()`` function. The string can be a path.
    The string can further be a URL. Valid URL schemes
    include http, ftp, s3, and file.

xpath : str, optional, default './\*'
    The ``XPath`` to parse required set of nodes for migration to
    :class:`~pandas.DataFrame`. ``XPath`` should return a collection of elements
    and not a single element. Note: The ``etree`` parser supports limited ``XPath``
    expressions. For more complex ``XPath``, use ``lxml`` which requires
    installation.

namespaces : dict, optional
    The namespaces defined in XML document as dicts with key being
    namespace prefix and value the URI. There is no need to include all
    namespaces in XML, only the ones used in ``xpath`` expression.
    Note: if XML document uses default namespace denoted as
    ``xmlns='<URI>'`` without a prefix, you must assign any temporary
    namespace prefix such as 'doc' to the URI in order to parse
    underlying nodes and/or attributes.

elems_only : bool, optional, default False
    Parse only the child elements at the specified ``xpath``. By default,
    all child elements and non-empty text nodes are returned.

attrs_only : bool, optional, default False
    Parse only the attributes at the specified ``xpath``.
    By default, all attributes are returned.

names : list-like, optional
    Column names for DataFrame of parsed XML data. Use this parameter to
    rename original element names and distinguish same named elements and
    attributes.

dtype : Type name or dict of column -> type, optional
```

Data type for data or columns. E.g. `{'a': np.float64, 'b': np.int32, 'c': 'Int64'}`
Use `'str'` or `'object'` together with suitable `'na_values'` settings to preserve and not interpret dtype.
If converters are specified, they will be applied INSTEAD of dtype conversion.

`converters` : dict, optional
Dict of functions for converting values in certain columns. Keys can either be integers or column labels.

`parse_dates` : bool or list of int or names or list of lists or dict, default False
Identifiers to parse index or columns to datetime. The behavior is as follows:

- * boolean. If True -> try parsing the index.
- * list of int or names. e.g. If [1, 2, 3] -> try parsing columns 1, 2, 3 each as a separate date column.
- * list of lists. e.g. If [[1, 3]] -> combine columns 1 and 3 and parse as a single date column.
- * dict, e.g. `{'foo' : [1, 3]}` -> parse columns 1, 3 as date and call result 'foo'

`encoding` : str, optional, default 'utf-8'
Encoding of XML document.

`parser` : `{'lxml', 'etree'}`, default 'lxml'
Parser module to use for retrieval of data. Only 'lxml' and 'etree' are supported. With 'lxml' more complex `'XPath'` searches and ability to use XSLT stylesheet are supported.

`stylesheet` : str, path object or file-like object
A URL, file-like object, or a string path containing an XSLT script. This stylesheet should flatten complex, deeply nested XML documents for easier parsing. To use this feature you must have `'lxml'` module installed and specify 'lxml' as `'parser'`. The `'xpath'` must reference nodes of transformed XML document generated after XSLT transformation and not the original XML document. Only XSLT 1.0 scripts and not later versions is currently supported.

`iterparse` : dict, optional
The nodes or attributes to retrieve in iterparsing of XML document as a dict with key being the name of repeating element and value being list of elements or attribute names that are descendants of the repeated element. Note: If this option is used, it will replace `'xpath'` parsing and unlike `'xpath'`, descendants do not need to relate to each other but can exist any where in document under the repeating element. This memory-efficient method should be used for very large XML files (500MB, 1GB, or 5GB+). For example, `{'row_element': ['child_elem', 'attr', 'grandchild_elem']}`.

`compression` : str or dict, default 'infer'
For on-the-fly decompression of on-disk data. If 'infer' and 'path_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in. Set to `'None'` for no decompression. Can also be a dict with key `'method'` set to one of `{'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'}` and other key-value pairs are forwarded to `'zipfile.ZipFile'`, `'gzip.GzipFile'`, `'bz2.BZ2File'`, `'zstandard.ZstdDecompressor'`, `'lzma.LZMAFile'` or `'tarfile.TarFile'`, respectively. As an example, the following could be passed for Zstandard decompression using a custom compression dictionary:
`'compression={'method': 'zstd', 'dict_data': my_compression_dict}'`.

`storage_options` : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `'urllib.request.Request'` as header options. For other URLs (e.g. starting with `'s3://'`, and `'gcs://'`) the key-value pairs are forwarded to `'fsspec.open'`. Please see `'fsspec'` and `'urllib'` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`dtype_backend` : `{'numpy_nullable', 'pyarrow'}`

Back-end data type applied to the resultant `:class:`DataFrame`` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

```
* ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
* ``"pyarrow"``: returns pyarrow-backed nullable
  :class:`ArrowDtype` :class:`DataFrame`

.. versionadded:: 2.0
```

Returns

`df`

A `DataFrame`.

See Also

`read_json` : Convert a JSON string to pandas object.

`read_html` : Read HTML tables into a list of `DataFrame` objects.

Notes

This method is best designed to import shallow XML documents in following format which is the ideal fit for the two-dimensions of a ``DataFrame`` (row by column). ::

```
<root>
  <row>
    <column1>data</column1>
    <column2>data</column2>
    <column3>data</column3>
    ...
  </row>
  <row>
    ...
  </row>
  ...
</root>
```

As a file format, XML documents can be designed any way including layout of elements and attributes as long as it conforms to W3C specifications. Therefore, this method is a convenience handler for a specific flatter design and not all possible XML structures.

However, for more complex XML documents, ``stylesheet`` allows you to temporarily redesign original document with XSLT (a special purpose language) for a flatter version for migration to a `DataFrame`.

This function will **always** return a single `:class:`DataFrame`` or raise exceptions due to issues with XML document, ``xpath``, or other parameters.

See the `:ref:`read_xml`` documentation in the IO section of the docs `<io.read_xml>` for more information in using this method to parse XML files to `DataFrames`.

Examples

```
>>> from io import StringIO
>>> xml = '''<?xml version='1.0' encoding='utf-8'?>
... <data xmlns="http://example.com">
...   <row>
...     <shape>square</shape>
...     <degrees>360</degrees>
...     <sides>4.0</sides>
...   </row>
...   <row>
...     <shape>circle</shape>
...     <degrees>360</degrees>
...     <sides/>
...   </row>
...   <row>
...     <shape>triangle</shape>
...     <degrees>180</degrees>
...     <sides>3.0</sides>
...   </row>
```

```

... </data>'''

>>> df = pd.read_xml(StringIO(xml))
>>> df
   shape  degrees  sides
0  square     360    4.0
1  circle     360    NaN
2 triangle     180    3.0

>>> xml = '''<?xml version='1.0' encoding='utf-8'?>
... <data>
...   <row shape="square" degrees="360" sides="4.0"/>
...   <row shape="circle" degrees="360"/>
...   <row shape="triangle" degrees="180" sides="3.0"/>
... </data>'''

>>> df = pd.read_xml(StringIO(xml), xpath="//row")
>>> df
   shape  degrees  sides
0  square     360    4.0
1  circle     360    NaN
2 triangle     180    3.0

>>> xml = '''<?xml version='1.0' encoding='utf-8'?>
... <doc:data xmlns:doc="https://example.com">
...   <doc:row>
...     <doc:shape>square</doc:shape>
...     <doc:degrees>360</doc:degrees>
...     <doc:sides>4.0</doc:sides>
...   </doc:row>
...   <doc:row>
...     <doc:shape>circle</doc:shape>
...     <doc:degrees>360</doc:degrees>
...     <doc:sides/>
...   </doc:row>
...   <doc:row>
...     <doc:shape>triangle</doc:shape>
...     <doc:degrees>180</doc:degrees>
...     <doc:sides>3.0</doc:sides>
...   </doc:row>
... </doc:data>'''

>>> df = pd.read_xml(
...     StringIO(xml),
...     xpath="//doc:row",
...     namespaces={"doc": "https://example.com"},
... )
>>> df
   shape  degrees  sides
0  square     360    4.0
1  circle     360    NaN
2 triangle     180    3.0

>>> xml_data = '''
...     <data>
...       <row>
...         <index>0</index>
...         <a>1</a>
...         <b>2.5</b>
...         <c>True</c>
...         <d>a</d>
...         <e>2019-12-31 00:00:00</e>
...       </row>
...       <row>
...         <index>1</index>
...         <b>4.5</b>
...         <c>False</c>
...         <d>b</d>
...         <e>2019-12-31 00:00:00</e>
...       </row>
...     </data>
... '''

>>> df = pd.read_xml(
...     StringIO(xml_data), dtype_backend="numpy_nullable", parse_dates=["e"]
... )
>>> df

```

	index	a	b	c	d	e
0	0	1	2.5	True	a	2019-12-31
1	1	<NA>	4.5	False	b	2019-12-31

```
reset_option(pat: str) -> None
```

Reset one or more options to their default value.

This method resets the specified pandas option(s) back to their default values. It allows partial string matching for convenience, but users should exercise caution to avoid unintended resets due to changes in option names in future versions.

Parameters

pat : str/regex

If specified only options matching ``pat`` will be reset.

Pass ``"all"`` as argument to reset all options.

.. warning::

Partial matches are supported for convenience, but unless you use the full option name (e.g. x.y.z.option_name), your code may break in future versions if new options with similar names are introduced.

Returns

None

No return value.

See Also

get_option : Retrieve the value of the specified option.

set_option : Set the value of the specified option or options.

describe_option : Print the description for one or more registered options.

Notes

For all available options, please view the

:ref:`User Guide <options.available>`.

Examples

```
>>> pd.reset_option("display.max_columns") # doctest: +SKIP
```

```
set_eng_float_format(accuracy: int = 3, use_eng_prefix: bool = False) -> None
```

Format float representation in DataFrame with SI notation.

Sets the floating-point display format for ``DataFrame`` objects using engineering notation (SI units), allowing easier readability of values across wide ranges.

Parameters

accuracy : int, default 3

Number of decimal digits after the floating point.

use_eng_prefix : bool, default False

Whether to represent a value with SI prefixes.

Returns

None

This method does not return a value. it updates the global display format for floats in DataFrames.

See Also

set_option : Set the value of the specified option or options.

reset_option : Reset one or more options to their default value.

Examples

```
>>> df = pd.DataFrame([1e-9, 1e-3, 1, 1e3, 1e6])
```

```
>>> df
```

	0
0	1.000000e-09
1	1.000000e-03
2	1.000000e+00
3	1.000000e+03


```

4  1.000000e+06

>>> pd.set_eng_float_format(accuracy=1)
>>> df
      0
0  1.0E-09
1  1.0E-03
2  1.0E+00
3  1.0E+03
4  1.0E+06

>>> pd.set_eng_float_format(use_eng_prefix=True)
>>> df
      0
0  1.000n
1  1.000m
2   1.000
3  1.000k
4  1.000M

>>> pd.set_eng_float_format(accuracy=1, use_eng_prefix=True)
>>> df
      0
0  1.0n
1  1.0m
2   1.0
3  1.0k
4  1.0M

>>> pd.set_option("display.float_format", None) # unset option

set_option(*args) -> None
Set the value of the specified option or options.

This method allows fine-grained control over the behavior and display settings
of pandas. Options affect various functionalities such as output formatting,
display limits, and operational behavior. Settings can be modified at runtime
without requiring changes to global configurations or environment variables.

Parameters
-----
*args : str | object | dict
    Arguments provided in pairs, which will be interpreted as (pattern, value),
    or as a single dictionary containing multiple option-value pairs.
    pattern: str
        Regexp which should match a single option
    value: object
        New value of option

    .. warning::

        Partial pattern matches are supported for convenience, but unless you
        use the full option name (e.g. x.y.z.option_name), your code may break in
        future versions if new options with similar names are introduced.

Returns
-----
None
    No return value.

Raises
-----
ValueError if odd numbers of non-keyword arguments are provided
TypeError if keyword arguments are provided
OptionError if no such option exists

See Also
-----
get_option : Retrieve the value of the specified option.
reset_option : Reset one or more options to their default value.
describe_option : Print the description for one or more registered options.
option_context : Context manager to temporarily set options in a ``with``
    statement.

Notes
-----
For all available options, please view the :ref:`User Guide <options.available>`

```

or use ``pandas.describe\_option()``.

Examples

Option-Value Pair Input:

```
>>> pd.set_option("display.max_columns", 4)
>>> df = pd.DataFrame([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
>>> df
0 1 ... 3 4
0 1 2 ... 4 5
1 6 7 ... 9 10
[2 rows x 5 columns]
>>> pd.reset_option("display.max_columns")
```

Dictionary Input:

```
>>> pd.set_option({"display.max_columns": 4, "display.precision": 1})
>>> df = pd.DataFrame([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
>>> df
0 1 ... 3 4
0 1 2 ... 4 5
1 6 7 ... 9 10
[2 rows x 5 columns]
>>> pd.reset_option("display.max_columns")
>>> pd.reset_option("display.precision")
```

show_versions(as_json: str | bool = False) -> None
Provide useful information, important for bug reports.

It comprises info about hosting operation system, pandas version, and versions of other installed relative packages.

Parameters

as_json : str or bool, default False

- * If False, outputs info in a human readable form to the console.
- * If str, it will be considered as a path to a file. Info will be written to that file in JSON format.
- * If True, outputs info in JSON format to the console.

See Also

get_option : Retrieve the value of the specified option.
set_option : Set the value of the specified option or options.

Examples

```
>>> pd.show_versions() # doctest: +SKIP
Your output may look something like this:
INSTALLED VERSIONS
-----
commit           : 37ea63d540fd27274cad6585082c91b1283f963d
python           : 3.10.6.final.0
python-bits      : 64
OS               : Linux
OS-release       : 5.10.102.1-microsoft-standard-WSL2
Version          : #1 SMP Wed Mar 2 00:30:59 UTC 2022
machine          : x86_64
processor        : x86_64
byteorder        : little
LC_ALL           : None
LANG             : en_GB.UTF-8
LOCALE           : en_GB.UTF-8
pandas           : 2.0.1
numpy            : 1.24.3
...
```

test(extra_args: list[str] | None = None, run_doctests: bool = False) -> None
Run the pandas test suite using pytest.

By default, runs with the marks -m "not slow and not network and not db"

Parameters

extra_args : list[str], default None
Extra marks to run the tests.

run_doctests : bool, default False
Whether to only run the Python and Cython doctests. If you would like to run both doctests/regular tests, just append "--doctest-modules"/"--doctest-cython" to extra_args.

See Also

pytest.main : The main entry point for pytest testing framework.

Examples

>>> pd.test() # doctest: +SKIP
running: pytest...

```
timedelta_range(  
    start=None,  
    end=None,  
    periods: int | None = None,  
    freq=None,  
    name=None,  
    closed=None,  
    *,  
    unit: TimeUnit | None = None  
) -> TimedeltaIndex  
Return a fixed frequency TimedeltaIndex with day as the default.
```

Parameters

start : str or timedelta-like, default None
Left bound for generating timedeltas.
end : str or timedelta-like, default None
Right bound for generating timedeltas.
periods : int, default None
Number of periods to generate.
freq : str, Timedelta, datetime.timedelta, or DateOffset, default 'D'
Frequency strings can have multiples, e.g. '5h'.
name : Hashable, default None
Name of the resulting TimedeltaIndex.
closed : str, default None
Make the interval closed with respect to the given frequency to the 'left', 'right', or both sides (None).
unit : {'s', 'ms', 'us', 'ns', None}, default None
Specify the desired resolution of the result.
If not specified, this is inferred from the 'start', 'end', and 'freq' using the same inference as :class:`Timedelta` taking the highest resolution of the three that are provided.

.. versionadded:: 2.0.0

Returns

TimedeltaIndex
Fixed frequency, with day as the default.

See Also

date_range : Return a fixed frequency DatetimeIndex.
period_range : Return a fixed frequency PeriodIndex.

Notes

Of the four parameters ``start``, ``end``, ``periods``, and ``freq``, a maximum of three can be specified at once. Of the three parameters ``start``, ``end``, and ``periods``, at least two must be specified. If ``freq`` is omitted, the resulting ``DatetimeIndex`` will have ``periods`` linearly spaced elements between ``start`` and ``end`` (closed on both sides).

To learn more about the frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

Examples

>>> pd.timedelta_range(start="1 day", periods=4)
TimedeltaIndex(['1 days', '2 days', '3 days', '4 days'],
 dtype='timedelta64[us]', freq='D')
The ``closed`` parameter specifies which endpoint is included. The default

behavior is to include both endpoints.

```
>>> pd.timedelta_range(start="1 day", periods=4, closed="right")
TimedeltaIndex(['1 days', '2 days', '3 days', '4 days'],
                dtype='timedelta64[us]', freq='D')
```

The ``freq`` parameter specifies the frequency of the TimedeltaIndex. Only fixed frequencies can be passed, non-fixed frequencies such as 'M' (month end) will raise.

```
>>> pd.timedelta_range(start="1 day", end="2 days", freq="6h")
TimedeltaIndex(['1 days 00:00:00', '1 days 06:00:00', '1 days 12:00:00',
                '1 days 18:00:00', '2 days 00:00:00'],
                dtype='timedelta64[us]', freq='6h')
```

Specify ``start``, ``end``, and ``periods``; the frequency is generated automatically (linearly spaced).

```
>>> pd.timedelta_range(start="1 day", end="5 days", periods=4)
TimedeltaIndex(['1 days 00:00:00', '2 days 08:00:00', '3 days 16:00:00',
                '5 days 00:00:00'],
                dtype='timedelta64[us]', freq=None)
```

****Specify a unit****

```
>>> pd.timedelta_range("1 Day", periods=3, freq="100000D", unit="s")
TimedeltaIndex(['1 days', '100001 days', '200001 days'],
                dtype='timedelta64[s]', freq='100000D')
```

```
to_datetime(
    arg: DatetimeScalarOrArrayConvertible | DictConvertible,
    errors: DateTimeErrorChoices = 'raise',
    dayfirst: bool = False,
    yearfirst: bool = False,
    utc: bool = False,
    format: str | None = None,
    exact: bool | lib.NoDefault = <no_default>,
    unit: str | None = None,
    origin: str = 'unix',
    cache: bool = True
) -> DatetimeIndex | Series | DatetimeScalar | NaTType
Convert argument to datetime.
```

This function converts a scalar, array-like, :class:`Series` or :class:`DataFrame`/dict-like to a pandas datetime object.

Parameters

arg : int, float, str, datetime, list, tuple, 1-d array, Series, DataFrame/dict-like
The object to convert to a datetime. If a :class:`DataFrame` is provided, the method expects minimally the following columns: :const:`"year"`, :const:`"month"`, :const:`"day"`. The column "year" must be specified in 4-digit format.

errors : {'raise', 'coerce'}, default 'raise'
- If :const:`"raise"`, then invalid parsing will raise an exception.
- If :const:`"coerce"`, then invalid parsing will be set as :const:`NaT`.

dayfirst : bool, default False
Specify a date parse order if `arg` is str or is list-like.
If :const:`True`, parses dates with the day first, e.g. :const:`"10/11/12"` is parsed as :const:`"2012-11-10"`.

.. warning::
 `"dayfirst=True"` is not strict, but will prefer to parse with day first.

yearfirst : bool, default False
Specify a date parse order if `arg` is str or is list-like.

- If :const:`True` parses dates with the year first, e.g. :const:`"10/11/12"` is parsed as :const:`"2010-11-12"`.

- If both `dayfirst` and `yearfirst` are :const:`True`, `yearfirst` is preceded (same as :mod:`dateutil`).

.. warning::
 `"yearfirst=True"` is not strict, but will prefer to parse

with year first.

utc : bool, default False
Control timezone-related parsing, localization and conversion.

- If :const:`True`, the function **always** returns a timezone-aware UTC-localized :class:`Timestamp`, :class:`Series` or :class:`DatetimeIndex`. To do this, timezone-naive inputs are **localized** as UTC, while timezone-aware inputs are **converted** to UTC.
- If :const:`False` (default), inputs will not be coerced to UTC. Timezone-naive inputs will remain naive, while timezone-aware ones will keep their time offsets. Limitations exist for mixed offsets (typically, daylight savings), see :ref:`Examples <to\_datetime\_tz\_examples>` section for details.

See also: pandas general documentation about `timezone conversion and localization`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#time-zone-handling>`_.

format : str, default None
The strftime to parse time, e.g. :const:`"%d/%m/%Y"`. See `strftime documentation`
<<https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>>`\_ for more information on choices, though note that :const:`"%f"` will parse all the way up to nanoseconds. You can also pass:

- "ISO8601", to parse any `ISO8601` <https://en.wikipedia.org/wiki/ISO_8601>`_ time string (not necessarily in exactly the same format);
- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with `dayfirst`.

.. note::

If a :class:`DataFrame` is passed, then `format` has no effect.

exact : bool, default True
Control how `format` is used:

- If :const:`True`, require an exact `format` match.
- If :const:`False`, allow the `format` to match anywhere in the target string.

Cannot be used alongside ``format='ISO8601'`` or ``format='mixed'``.

unit : str, default 'ns'
The unit of the arg (D,s,ms,us,ns) denote the unit, which is an integer or float number. This will be based off the origin. Example, with ``unit='ms'`` and ``origin='unix'``, this would calculate the number of milliseconds to the unix epoch start.

origin : scalar, default 'unix'
Define the reference date. The numeric values would be parsed as number of units (defined by `unit`) since this reference date.

- If :const:`'unix'` (or POSIX) time; origin is set to 1970-01-01.
- If :const:`'julian'`, unit must be :const:`'D'`, and origin is set to beginning of Julian Calendar. Julian day number :const:`0` is assigned to the day starting at noon on January 1, 4713 BC.
- If Timestamp convertible (Timestamp, dt.datetime, np.datetime64 or date string), origin is set to Timestamp identified by origin.
- If a float or integer, origin is the difference (in units determined by the ``unit`` argument) relative to 1970-01-01.

cache : bool, default True
If :const:`True`, use a cache of unique, converted dates to apply the datetime conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets. The cache is only used when there are at least 50 values. The presence of out-of-bounds values will render the cache unusable and may slow down parsing.

Returns

datetime
If parsing succeeded.
Return type depends on input (types in parenthesis correspond to fallback in case of unsuccessful timezone or out-of-range timestamp

parsing):

- scalar: :class:`Timestamp` (or :class:`datetime.datetime`)
- array-like: :class:`DatetimeIndex` (or :class:`Series` with :class:`object` dtype containing :class:`datetime.datetime`)
- Series: :class:`Series` of :class:`datetime64` dtype (or :class:`Series` of :class:`object` dtype containing :class:`datetime.datetime`)
- DataFrame: :class:`Series` of :class:`datetime64` dtype (or :class:`Series` of :class:`object` dtype containing :class:`datetime.datetime`)

Raises

ParserError

When parsing a date from string fails.

ValueError

When another datetime conversion error happens. For example when one of 'year', 'month', 'day' columns is missing in a :class:`DataFrame`, or when a Timezone-aware :class:`datetime.datetime` is found in an array-like of mixed time offsets, and ``utc=False``, or when parsing datetimes with mixed time zones unless ``utc=True``. If parsing datetimes with mixed time zones, please specify ``utc=True``.

See Also

DataFrame.astype : Cast argument to a specified dtype.

to_timedelta : Convert argument to timedelta.

convert_dtypes : Convert dtypes.

Notes

Many input types are supported, and lead to different output types:

- ****scalars**** can be int, float, str, datetime object (from stdlib :mod:`datetime` module or :mod:`numpy`). They are converted to :class:`Timestamp` when possible, otherwise they are converted to :class:`datetime.datetime`. None/NaN/null scalars are converted to :const:`NaT`.
- ****array-like**** can contain int, float, str, datetime objects. They are converted to :class:`DatetimeIndex` when possible, otherwise they are converted to :class:`Index` with :class:`object` dtype, containing :class:`datetime.datetime`. None/NaN/null entries are converted to :const:`NaT` in both cases.
- ****Series**** are converted to :class:`Series` with :class:`datetime64` dtype when possible, otherwise they are converted to :class:`Series` with :class:`object` dtype, containing :class:`datetime.datetime`. None/NaN/null entries are converted to :const:`NaT` in both cases.
- ****DataFrame/dict-like**** are converted to :class:`Series` with :class:`datetime64` dtype. For each row a datetime is created from assembling the various dataframe columns. Column keys can be common abbreviations like ['year', 'month', 'day', 'minute', 'second', 'ms', 'us', 'ns']) or plurals of the same.

The following causes are responsible for :class:`datetime.datetime` objects being returned (possibly inside an :class:`Index` or a :class:`Series` with :class:`object` dtype) instead of a proper pandas designated type (:class:`Timestamp`, :class:`DatetimeIndex` or :class:`Series` with :class:`datetime64` dtype):

- when any input element is before :const:`Timestamp.min` or after :const:`Timestamp.max`, see `timestamp limitations` <https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#timeseries-timestamp-limits>`\_`.
- when ``utc=False`` (default) and the input is an array-like or :class:`Series` containing mixed naive/aware datetime, or aware with mixed time offsets. Note that this happens in the (quite frequent) situation when the timezone has a daylight savings policy. In that case you may wish to use ``utc=True``.

Examples

****Handling various input formats****

Assembling a datetime from multiple columns of a `:class:`DataFrame``. The keys can be common abbreviations like `['year', 'month', 'day', 'minute', 'second', 'ms', 'us', 'ns']` or plurals of the same

```
>>> df = pd.DataFrame({"year": [2015, 2016], "month": [2, 3], "day": [4, 5]})
>>> pd.to_datetime(df)
0    2015-02-04
1    2016-03-05
dtype: datetime64[us]
```

Using a unix epoch time

```
>>> pd.to_datetime(1490195805, unit="s")
Timestamp('2017-03-22 15:16:45')
>>> pd.to_datetime(1490195805433502912, unit="ns")
Timestamp('2017-03-22 15:16:45.433502912')
```

.. warning:: For float arg, precision rounding might happen. To prevent unexpected behavior use a fixed-width exact type.

Using a non-unix epoch origin

```
>>> pd.to_datetime([1, 2, 3], unit="D", origin=pd.Timestamp("1960-01-01"))
DatetimeIndex(['1960-01-02', '1960-01-03', '1960-01-04'],
              dtype='datetime64[s]', freq=None)
```

****Differences with `strptime` behavior****

`:const:`"%f"`` will parse all the way up to nanoseconds.

```
>>> pd.to_datetime("2018-10-26 12:00:00.000000001", format="%Y-%m-%d %H:%M:%S.%f")
Timestamp('2018-10-26 12:00:00.000000001')
```

****Non-convertible date/times****

Passing `errors='coerce'`` will force an out-of-bounds date to `:const:`NaT``, in addition to forcing non-dates (or non-parseable dates) to `:const:`NaT``.

```
>>> pd.to_datetime("invalid for Ymd", format="%Y%m%d", errors="coerce")
NaT
```

.. `_to_datetime_tz` examples:

****Timezones and time offsets****

The default behaviour (`utc=False``) is as follows:

- Timezone-naive inputs are converted to timezone-naive `:class:`DatetimeIndex``:

```
>>> pd.to_datetime(["2018-10-26 12:00:00", "2018-10-26 13:00:15"])
DatetimeIndex(['2018-10-26 12:00:00', '2018-10-26 13:00:15'],
              dtype='datetime64[us]', freq=None)
```

- Timezone-aware inputs *with constant time offset* are converted to timezone-aware `:class:`DatetimeIndex``:

```
>>> pd.to_datetime(["2018-10-26 12:00 -0500", "2018-10-26 13:00 -0500"])
DatetimeIndex(['2018-10-26 12:00:00-05:00', '2018-10-26 13:00:00-05:00'],
              dtype='datetime64[us, UTC-05:00]', freq=None)
```

- However, timezone-aware inputs *with mixed time offsets* (for example issued from a timezone with daylight savings, such as Europe/Paris) are ****not successfully converted**** to a `:class:`DatetimeIndex``. Parsing datetimes with mixed time zones will raise a `ValueError` unless `utc=True``:

```
>>> pd.to_datetime(
...     ["2020-10-25 02:00 +0200", "2020-10-25 04:00 +0100"]
... ) # doctest: +SKIP
ValueError: Mixed timezones detected. Pass utc=True in to_datetime
or tz='UTC' in DatetimeIndex to convert to a common timezone.
```

- To create a `:class:`Series`` with mixed offsets and `object`` dtype, please use `:meth:`Series.apply`` and `:func:`datetime.datetime.strptime``:

```
>>> import datetime as dt
>>> ser = pd.Series(["2020-10-25 02:00 +0200", "2020-10-25 04:00 +0100"])
```

```
>>> ser.apply(lambda x: dt.datetime.strptime(x, "%Y-%m-%d %H:%M %Z"))
0    2020-10-25 02:00:00+02:00
1    2020-10-25 04:00:00+01:00
dtype: object
```

- A mix of timezone-aware and timezone-naive inputs will also raise a ValueError unless ``utc=True``:

```
>>> from datetime import datetime
>>> pd.to_datetime(
...     ["2020-01-01 01:00:00-01:00", datetime(2020, 1, 1, 3, 0)]
... ) # doctest: +SKIP
ValueError: Mixed timezones detected. Pass utc=True in to_datetime
or tz='UTC' in DatetimeIndex to convert to a common timezone.
```

|

Setting ``utc=True`` solves most of the above issues:

- Timezone-naive inputs are *localized* as UTC

```
>>> pd.to_datetime(["2018-10-26 12:00", "2018-10-26 13:00"], utc=True)
DatetimeIndex(['2018-10-26 12:00:00+00:00', '2018-10-26 13:00:00+00:00'],
              dtype='datetime64[us, UTC]', freq=None)
```

- Timezone-aware inputs are *converted* to UTC (the output represents the exact same datetime, but viewed from the UTC time offset ``+00:00``).

```
>>> pd.to_datetime(["2018-10-26 12:00 -0530", "2018-10-26 12:00 -0500"], utc=True)
DatetimeIndex(['2018-10-26 17:30:00+00:00', '2018-10-26 17:00:00+00:00'],
              dtype='datetime64[us, UTC]', freq=None)
```

- Inputs can contain both string or datetime, the above rules still apply

```
>>> pd.to_datetime(["2018-10-26 12:00", datetime(2020, 1, 1, 18)], utc=True)
DatetimeIndex(['2018-10-26 12:00:00+00:00', '2020-01-01 18:00:00+00:00'],
              dtype='datetime64[us, UTC]', freq=None)
```

```
to_numeric(
    arg,
    errors: DateTImeErrorChoices = 'raise',
    downcast: Literal['integer', 'signed', 'unsigned', 'float'] | None = None,
    dtype_backend: DtypeBackend | lib.NoDefault = <no_default>
)
```

Convert argument to a numeric type.

If the input is already of a numeric dtype, the dtype will be preserved. For non-numeric inputs, the default return dtype is ``float64`` or ``int64`` depending on the data supplied. Use the ``downcast`` parameter to obtain other dtypes.

Please note that precision loss may occur if really large numbers are passed in. Due to the internal limitations of ``ndarray``, if numbers smaller than ``-9223372036854775808`` (``np.iinfo(np.int64).min``) or larger than ``18446744073709551615`` (``np.iinfo(np.uint64).max``) are passed in, it is very likely they will be converted to float so that they can be stored in an ``ndarray``. These warnings apply similarly to ``Series`` since it internally leverages ``ndarray``.

Parameters

arg : scalar, list, tuple, 1-d array, or Series
Argument to be converted.

errors : {'raise', 'coerce'}, default 'raise'
- If 'raise', then invalid parsing will raise an exception.
- If 'coerce', then invalid parsing will be set as NaN.

downcast : str, default None
Can be 'integer', 'signed', 'unsigned', or 'float'.
If not None, and if the data has been successfully cast to a numerical dtype (or if the data was numeric to begin with), downcast that resulting data to the smallest numerical dtype possible according to the following rules:

- 'integer' or 'signed': smallest signed int dtype (min.: np.int8)

- 'unsigned': smallest unsigned int dtype (min.: np.uint8)
- 'float': smallest float dtype (min.: np.float32)

As this behaviour is separate from the core conversion to numeric values, any errors raised during the downcasting will be surfaced regardless of the value of the 'errors' input.

In addition, downcasting will only occur if the size of the resulting data's dtype is strictly larger than the dtype it is to be cast to, so if none of the dtypes checked satisfy that specification, no downcasting will be performed on the data.

```
dtype_backend : {'numpy_nullable', 'pyarrow'}
Back-end data type applied to the resultant :class:`DataFrame`
(still experimental). If not specified, the default behavior
is to not use nullable data types. If specified, the behavior
is as follows:

* ``"numpy_nullable"``: returns nullable-dtype-backed object
* ``"pyarrow"``: returns with pyarrow-backed nullable object

.. versionadded:: 2.0

Returns
-----
ret
    Numeric if parsing succeeded.
    Return type depends on input. Series if Series, otherwise ndarray.

Raises
-----
ValueError
    If the input contains non-numeric values and `errors='raise'`.
TypeError
    If the input is not list-like, 1D, or scalar convertible to numeric,
    such as nested lists or unsupported input types (e.g., dict).
```

See Also

```
-----
DataFrame.astype : Cast argument to a specified dtype.
to_datetime : Convert argument to datetime.
to_timedelta : Convert argument to timedelta.
numpy.ndarray.astype : Cast a numpy array to a specified type.
DataFrame.convert_dtypes : Convert dtypes.
```

Examples

Take separate series and convert to numeric, coercing when told to

```
>>> s = pd.Series(["1.0", "2", -3])
>>> pd.to_numeric(s)
0    1.0
1    2.0
2   -3.0
dtype: float64
>>> pd.to_numeric(s, downcast="float")
0    1.0
1    2.0
2   -3.0
dtype: float32
>>> pd.to_numeric(s, downcast="signed")
0    1
1    2
2   -3
dtype: int8
>>> s = pd.Series(["apple", "1.0", "2", -3])
>>> pd.to_numeric(s, errors="coerce")
0    NaN
1    1.0
2    2.0
3   -3.0
dtype: float64
```

Downcasting of nullable integer and floating dtypes is supported:

```
>>> s = pd.Series([1, 2, 3], dtype="Int64")
```

```

>>> pd.to_numeric(s, downcast="integer")
0    1
1    2
2    3
dtype: Int8
>>> s = pd.Series([1.0, 2.1, 3.0], dtype="Float64")
>>> pd.to_numeric(s, downcast="float")
0    1.0
1    2.1
2    3.0
dtype: Float32

to_pickle(
    obj: Any,
    filepath_or_buffer: FilePath | WriteBuffer[bytes],
    compression: CompressionOptions = 'infer',
    protocol: int = 5,
    storage_options: StorageOptions | None = None
) -> None
Pickle (serialize) object to file.

Parameters
-----
obj : any object
    Any python object.
filepath_or_buffer : str, path object, or file-like object
    String, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a binary ``write()`` function.
    Also accepts URL. URL has to be of S3 or GCS.
compression : str or dict, default 'infer'
    For on-the-fly compression of the output data. If 'infer' and
    'filepath_or_buffer' is path-like, then detect compression from the
    following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar',
    '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).
    Set to ``None`` for no compression.
    Can also be a dict with key ``'method'`` set
    to one of {``'zip'``, ``'gzip'``, ``'bz2'``, ``'zstd'``, ``'xz'``,
    ``'tar'``} and other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for faster compression
    and to create a reproducible gzip archive:
    ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.
protocol : int
    Int which indicates which protocol should be used by the pickler,
    default HIGHEST_PROTOCOL (see [1], paragraph 12.1.2). The possible
    values for this parameter depend on the version of Python. For Python
    2.x, possible values are 0, 1, 2. For Python>=3.0, 3 is a valid value.
    For Python >= 3.4, 4 is a valid value. A negative value for the
    protocol parameter is equivalent to setting its value to
    HIGHEST_PROTOCOL.
storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`.

.. [1] https://docs.python.org/3/library/pickle.html

See Also
-----
read_pickle : Load pickled pandas object (or any object) from file.
DataFrame.to_hdf : Write DataFrame to an HDF5 file.
DataFrame.to_sql : Write DataFrame to a SQL database.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.

Examples
-----
>>> original_df = pd.DataFrame(
...     {"foo": range(5), "bar": range(5, 10)}
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP

```

```

    foo  bar
0      0   5
1      1   6
2      2   7
3      3   8
4      4   9
>>> pd.to_pickle(original_df, "./dummy.pkl") # doctest: +SKIP

>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
    foo  bar
0      0   5
1      1   6
2      2   7
3      3   8
4      4   9

```

to_timedelta(
 arg: str | int | float | timedelta | list | tuple | range | ArrayLike | Index | Series,
 unit: UnitChoices | None = None,
 errors: DateTimeErrorChoices = 'raise'
) -> Timedelta | TimedeltaIndex | Series | NaTType | Any
 Convert argument to timedelta.

Timedeltas are absolute differences in times, expressed in difference units (e.g. days, hours, minutes, seconds). This method converts an argument from a recognized timedelta format / value into a Timedelta type.

Parameters

 arg : str, timedelta, list-like or Series
 The data to be converted to timedelta.

.. versionchanged:: 2.0
 Strings with units 'M', 'Y' and 'y' do not represent unambiguous timedelta values and will raise an exception.

unit : str, optional
 Denotes the unit of the arg for numeric `arg`. Defaults to ``"ns"``.

Possible values:

```

* 'W'
* 'D' / 'days' / 'day'
* 'hours' / 'hour' / 'hr' / 'h'
* 'm' / 'minute' / 'min' / 'minutes'
* 's' / 'seconds' / 'sec' / 'second'
* 'ms' / 'milliseconds' / 'millisecond' / 'milli' / 'millis'
* 'us' / 'microseconds' / 'microsecond' / 'micro' / 'micros'
* 'ns' / 'nanoseconds' / 'nano' / 'nanos' / 'nanosecond'

```

Must not be specified when `arg` contains strings and ``errors="raise"``.

errors : {'raise', 'coerce'}, default 'raise'
 - If 'raise', then invalid parsing will raise an exception.
 - If 'coerce', then invalid parsing will be set as NaT.

Returns

 timedelta
 If parsing succeeded.
 Return type depends on input:

- list-like: TimedeltaIndex of timedelta64 dtype
- Series: Series of timedelta64 dtype
- scalar: Timedelta

See Also

 DataFrame.astype : Cast argument to a specified dtype.
 to_datetime : Convert argument to datetime.
 convert_dtypes : Convert dtypes.

Notes

 If the precision is higher than nanoseconds, the precision of the duration is

truncated to nanoseconds for string inputs.

Examples

Parsing a single string to a Timedelta:

```
>>> pd.to_timedelta("1 days 06:05:01.00003")
Timedelta('1 days 06:05:01.000030')
>>> pd.to_timedelta("15.5us")
Timedelta('0 days 00:00:00.000015500')
```

Parsing a list or array of strings:

```
>>> pd.to_timedelta(["1 days 06:05:01.00003", "15.5us", "nan"])
TimedeltaIndex(['1 days 06:05:01.000030', '0 days 00:00:00.000015500', NaT],
                dtype='timedelta64[ns]', freq=None)
```

Converting numbers by specifying the `unit` keyword argument:

```
>>> pd.to_timedelta(np.arange(5), unit="s")
TimedeltaIndex(['0 days 00:00:00', '0 days 00:00:01', '0 days 00:00:02',
                '0 days 00:00:03', '0 days 00:00:04'],
                dtype='timedelta64[s]', freq=None)
>>> pd.to_timedelta(np.arange(5), unit="D")
TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'],
                dtype='timedelta64[s]', freq=None)
```

`unique(values)`

Return unique values based on a hash table.

Uniques are returned in order of appearance. This does NOT sort.

Significantly faster than `numpy.unique` for long enough sequences.
Includes NA values.

Parameters

`values` : 1d array-like

The input array-like object containing values from which to extract unique values.

Returns

`numpy.ndarray`, `ExtensionArray` or `NumpyExtensionArray`

The return can be:

- * Index : when the input is an Index
- * Categorical : when the input is a Categorical dtype
- * ndarray : when the input is a Series/ndarray

Return `numpy.ndarray`, `ExtensionArray` or `NumpyExtensionArray`.

See Also

`Index.unique` : Return unique values from an Index.

`Series.unique` : Return unique values of Series object.

Examples

```
>>> pd.unique(pd.Series([2, 1, 3, 3]))
array([2, 1, 3])
```

```
>>> pd.unique(pd.Series([2] + [1] * 5))
array([2, 1])
```

```
>>> pd.unique(pd.Series([pd.Timestamp("20160101"), pd.Timestamp("20160101")]))
array(['2016-01-01T00:00:00.000000'], dtype='datetime64[us]')
```

```
>>> pd.unique(
...     pd.Series(
...         [
...             pd.Timestamp("20160101", tz="US/Eastern"),
...             pd.Timestamp("20160101", tz="US/Eastern"),
...         ],
...         dtype="M8[ns, US/Eastern]",
...     )
... )
```

```
... )
<DatetimeArray>
['2016-01-01 00:00:00-05:00']
Length: 1, dtype: datetime64[ns, US/Eastern]
```

```
>>> pd.unique(
...     pd.Index(
...         [
...             pd.Timestamp("20160101", tz="US/Eastern"),
...             pd.Timestamp("20160101", tz="US/Eastern"),
...         ],
...         dtype="M8[ns, US/Eastern]",
...     )
... )
DatetimeIndex(['2016-01-01 00:00:00-05:00'],
              dtype='datetime64[ns, US/Eastern]',
              freq=None)
```

```
>>> pd.unique(np.array(list("baabc"), dtype="O"))
array(['b', 'a', 'c'], dtype=object)
```

An unordered Categorical will return categories in the order of appearance.

```
>>> pd.unique(pd.Series(pd.Categorical(list("baabc"))))
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

```
>>> pd.unique(pd.Series(pd.Categorical(list("baabc"), categories=list("abc"))))
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
```

An ordered Categorical preserves the category ordering.

```
>>> pd.unique(
...     pd.Series(
...         pd.Categorical(list("baabc"), categories=list("abc"), ordered=True)
...     )
... )
['b', 'a', 'c']
Categories (3, str): ['a' < 'b' < 'c']
```

An array of tuples

```
>>> pd.unique(pd.Series([("a", "b"), ("b", "a"), ("a", "c"), ("b", "a")]).values)
array([('a', 'b'), ('b', 'a'), ('a', 'c')], dtype=object)
```

A NumpyExtensionArray of complex

```
>>> pd.unique(pd.array([1 + 1j, 2, 3]))
<NumpyExtensionArray>
[(1+1j), (2+0j), (3+0j)]
Length: 3, dtype: complex128
```

```
wide_to_long(
    df: DataFrame,
    stubnames,
    i,
    j,
    sep: str = '',
    suffix: str = '\\d+'
) -> DataFrame
```

Unpivot a DataFrame from wide to long format.

Less flexible but more user-friendly than melt.

With stubnames ['A', 'B'], this function expects to find one or more group of columns with format A-suffix1, A-suffix2, ..., B-suffix1, B-suffix2, ... You specify what you want to call this suffix in the resulting long format with `j` (for example `j='year'`)

Each row of these wide variables are assumed to be uniquely identified by `i` (can be a single column name or a list of column names)

All remaining variables in the data frame are left intact.
Parameters

```

-----
df : DataFrame
    The wide-format DataFrame.
stubnames : str or list-like
    The stub name(s). The wide format variables are assumed to
    start with the stub names.
i : str or list-like
    Column(s) to use as id variable(s).
j : str
    The name of the sub-observation variable. What you wish to name your
    suffix in the long format.
sep : str, default ""
    A character indicating the separation of the variable names
    in the wide format, to be stripped from the names in the long format.
    For example, if your column names are A-suffix1, A-suffix2, you
    can strip the hyphen by specifying `sep='-'`.
suffix : str, default '\\d+'
    A regular expression capturing the wanted suffixes. '\\d+' captures
    numeric suffixes. Suffixes with no numbers could be specified with the
    negated character class '\\D+'. You can also further disambiguate
    suffixes, for example, if your wide variables are of the form A-one,
    B-two, ..., and you have an unrelated column A-rating, you can ignore the
    last one by specifying `suffix='(!?one|two)'`. When all suffixes are
    numeric, they are cast to int64/float64.

```

Returns

```

-----
DataFrame
    A DataFrame that contains each stub name as a variable, with new index
    (i, j).

```

See Also

```

-----
melt : Unpivot a DataFrame from wide to long format, optionally leaving
    identifiers set.
pivot : Create a spreadsheet-style pivot table as a DataFrame.
DataFrame.pivot : Pivot without aggregation that can handle
    non-numeric data.
DataFrame.pivot_table : Generalization of pivot that can handle
    duplicate values for one index/column pair.
DataFrame.unstack : Pivot based on the index values instead of a
    column.

```

Notes

```

-----
All extra variables are left untouched. This simply uses
`pandas.melt` under the hood, but is hard-coded to "do the right thing"
in a typical case.

```

Examples

```

-----
>>> np.random.seed(123)
>>> df = pd.DataFrame(
...     {
...         "A1970": {0: "a", 1: "b", 2: "c"},
...         "A1980": {0: "d", 1: "e", 2: "f"},
...         "B1970": {0: 2.5, 1: 1.2, 2: 0.7},
...         "B1980": {0: 3.2, 1: 1.3, 2: 0.1},
...         "X": dict(zip(range(3), np.random.randn(3), strict=True)),
...     }
... )
>>> df["id"] = df.index
>>> df
   A1970 A1980 B1970 B1980      X id
0      a      d   2.5   3.2 -1.085631  0
1      b      e   1.2   1.3  0.997345  1
2      c      f   0.7   0.1  0.282978  2
>>> pd.wide_to_long(df, ["A", "B"], i="id", j="year")
... # doctest: +NORMALIZE_WHITESPACE
      X  A  B
id year
0  1970 -1.085631  a  2.5
1  1970  0.997345  b  1.2
2  1970  0.282978  c  0.7
0  1980 -1.085631  d  3.2
1  1980  0.997345  e  1.3
2  1980  0.282978  f  0.1

```

With multiple id columns

```
>>> df = pd.DataFrame(
...     {
...         "famid": [1, 1, 1, 2, 2, 2, 3, 3, 3],
...         "birth": [1, 2, 3, 1, 2, 3, 1, 2, 3],
...         "ht1": [2.8, 2.9, 2.2, 2, 1.8, 1.9, 2.2, 2.3, 2.1],
...         "ht2": [3.4, 3.8, 2.9, 3.2, 2.8, 2.4, 3.3, 3.4, 2.9],
...     }
... )
>>> df
   famid  birth  ht1  ht2
0      1      1   2.8  3.4
1      1      2   2.9  3.8
2      1      3   2.2  2.9
3      2      1   2.0  3.2
4      2      2   1.8  2.8
5      2      3   1.9  2.4
6      3      1   2.2  3.3
7      3      2   2.3  3.4
8      3      3   2.1  2.9
>>> long_format = pd.wide_to_long(df, stubnames="ht", i=["famid", "birth"], j="age")
>>> long_format
... # doctest: +NORMALIZE_WHITESPACE
           ht
famid birth age
1      1      1   2.8
      1      2   3.4
      2      1   2.9
      2      2   3.8
      3      1   2.2
      3      2   2.9
2      1      1   2.0
      2      2   3.2
      2      1   1.8
      2      2   2.8
      3      1   1.9
      3      2   2.4
3      1      1   2.2
      2      2   3.3
      2      1   2.3
      2      2   3.4
      3      1   2.1
      3      2   2.9
```

Going from long back to wide just takes some creative use of `unstack`

```
>>> wide_format = long_format.unstack()
>>> wide_format.columns = wide_format.columns.map("{0[0]}{0[1]}".format)
>>> wide_format.reset_index()
   famid  birth  ht1  ht2
0      1      1   2.8  3.4
1      1      2   2.9  3.8
2      1      3   2.2  2.9
3      2      1   2.0  3.2
4      2      2   1.8  2.8
5      2      3   1.9  2.4
6      3      1   2.2  3.3
7      3      2   2.3  3.4
8      3      3   2.1  2.9
```

Less wieldy column names are also handled

```
>>> np.random.seed(0)
>>> df = pd.DataFrame(
...     {
...         "A(weekly)-2010": np.random.rand(3),
...         "A(weekly)-2011": np.random.rand(3),
...         "B(weekly)-2010": np.random.rand(3),
...         "B(weekly)-2011": np.random.rand(3),
...         "x": np.random.randint(3, size=3),
...     }
... )
>>> df["id"] = df.index
>>> df # doctest: +NORMALIZE_WHITESPACE, +ELLIPSIS
A(weekly)-2010  A(weekly)-2011  B(weekly)-2010  B(weekly)-2011  x  id
0      0.548814      0.544883      0.437587      0.383442  0  0
```

1	0.715189	0.423655	0.891773	0.791725	1	1
2	0.602763	0.645894	0.963663	0.528895	1	2

```
>>> pd.wide_to_long(df, ["A(weekly)", "B(weekly)"], i="id", j="year", sep="-")
... # doctest: +NORMALIZE_WHITESPACE
      X  A(weekly)  B(weekly)
id year
0  2010  0    0.548814  0.437587
1  2010  1    0.715189  0.891773
2  2010  1    0.602763  0.963663
0  2011  0    0.544883  0.383442
1  2011  1    0.423655  0.791725
2  2011  1    0.645894  0.528895
```

If we have many columns, we could also use a regex to find our stubnames and pass that list on to wide_to_long

```
>>> stubnames = sorted(
...     set(
...         [
...             match[0]
...             for match in df.columns.str.findall(r"[A-B]\(.*\)").values
...             if match != []
...         ]
...     )
... )
>>> list(stubnames)
['A(weekly)', 'B(weekly)']
```

All of the above examples have integers as suffixes. It is possible to have non-integers as suffixes.

```
>>> df = pd.DataFrame(
...     {
...         "famid": [1, 1, 1, 2, 2, 2, 3, 3, 3],
...         "birth": [1, 2, 3, 1, 2, 3, 1, 2, 3],
...         "ht_one": [2.8, 2.9, 2.2, 2, 1.8, 1.9, 2.2, 2.3, 2.1],
...         "ht_two": [3.4, 3.8, 2.9, 3.2, 2.8, 2.4, 3.3, 3.4, 2.9],
...     }
... )
>>> df
   famid  birth  ht_one  ht_two
0      1      1      2.8      3.4
1      1      2      2.9      3.8
2      1      3      2.2      2.9
3      2      1      2.0      3.2
4      2      2      1.8      2.8
5      2      3      1.9      2.4
6      3      1      2.2      3.3
7      3      2      2.3      3.4
8      3      3      2.1      2.9
```

```
>>> long_format = pd.wide_to_long(
...     df, stubnames="ht", i=["famid", "birth"], j="age", sep="_", suffix=r"\w+"
... )
>>> long_format
... # doctest: +NORMALIZE_WHITESPACE
      ht
famid birth age
1      1     one  2.8
      1     two  3.4
      2     one  2.9
      2     two  3.8
      3     one  2.2
      3     two  2.9
2      1     one  2.0
      2     two  3.2
      2     one  1.8
      2     two  2.8
      3     one  1.9
      3     two  2.4
3      1     one  2.2
      2     two  3.3
      2     one  2.3
      2     two  3.4
      3     one  2.1
      3     two  2.9
```



```
DATA
    IndexSlice = <pandas.core.indexing._IndexSlice object>
    NA = <NA>
    NaT = NaT
    __all__ = ['NA', 'ArrowDtype', 'BooleanDtype', 'Categorical', 'Categor...
    __docformat__ = 'restructuredtext'
    __git_version__ = 'e04b26f375035e5106cb913e47b6db612f4ebb11'
    options = <pandas._config.config.DictWrapper object>
```

```
VERSION
    3.0.1
```

```
FILE
    c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\__init__.py
```

Help on package pandas.api in pandas:

```
NAME
    pandas.api - public toolkit API
```

```
PACKAGE CONTENTS
    executors (package)
    extensions (package)
    indexers (package)
    interchange (package)
    internals
    types (package)
    typing (package)
```

```
DATA
    __all__ = ['executors', 'extensions', 'indexers', 'interchange', 'type...
```

```
FILE
    c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\api\__init__.py
```

Help on package pandas.arrays in pandas:

```
NAME
    pandas.arrays - All of pandas' ExtensionArrays.
```

```
DESCRIPTION
    See :ref:`extending.extension-types` for more.
```

```
PACKAGE CONTENTS
```

```
CLASSES
```

```
    pandas._libs.interval.IntervalMixin(builtins.object)
        IntervalArray(pandas._libs.interval.IntervalMixin, pandas.api.extensions.ExtensionArray)
    pandas._libs.tslibs.period.PeriodMixin(builtins.object)
        PeriodArray(pandas.core.arrays.datetimelike.DatelikeOps, pandas._libs.tslibs.period.PeriodMi
    pandas.api.extensions.ExtensionArray(builtins.object)
        IntervalArray(pandas._libs.interval.IntervalMixin, pandas.api.extensions.ExtensionArray)
        SparseArray(pandas.core.arraylike.OpsMixin, pandas.core.base.PandasObject, pandas.api.extens
    pandas.core.arraylike.OpsMixin(builtins.object)
        ArrowExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays.base.ExtensionArraySup
        ArrowStringArray(pandas.core.strings.object_array.ObjectStringArrayMixin, ArrowExtension
        NumpyExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays._mixins.NDArrayBacked
        StringArray(pandas.core.arrays.string_.BaseStringArray, NumpyExtensionArray)
        SparseArray(pandas.core.arraylike.OpsMixin, pandas.core.base.PandasObject, pandas.api.extens
    pandas.core.arrays._arrow_string_mixins.ArrowStringArrayMixin(builtins.object)
        ArrowExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays.base.ExtensionArraySup
        ArrowStringArray(pandas.core.strings.object_array.ObjectStringArrayMixin, ArrowExtension
    pandas.core.arrays._mixins.NDArrayBackedExtensionArray(pandas._libs.arrays.NDArrayBacked, pandas
    pandas.Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.
        NumpyExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays._mixins.NDArrayBacked
        StringArray(pandas.core.arrays.string_.BaseStringArray, NumpyExtensionArray)
    pandas.core.arrays.base.ExtensionArraySupportsAnyAll(pandas.api.extensions.ExtensionArray)
        ArrowExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays.base.ExtensionArraySup
        ArrowStringArray(pandas.core.strings.object_array.ObjectStringArrayMixin, ArrowExtension
    pandas.core.arrays.datetimelike.DatelikeOps(pandas.core.arrays.datetimelike.DatetimeLikeArrayMix
        DatetimeArray(pandas.core.arrays.datetimelike.TimelikeOps, pandas.core.arrays.datetimelike.D
        PeriodArray(pandas.core.arrays.datetimelike.DatelikeOps, pandas._libs.tslibs.period.PeriodMi
    pandas.core.arrays.datetimelike.TimelikeOps(pandas.core.arrays.datetimelike.DatetimeLikeArrayMix
        DatetimeArray(pandas.core.arrays.datetimelike.TimelikeOps, pandas.core.arrays.datetimelike.D
```

```

    TimedeltaArray
pandas.core.arrays.masked.BaseMaskedArray(pandas.core.arraylike.OpsMixin, pandas.api.extensions.
    BooleanArray
pandas.core.arrays.numeric.NumericArray(pandas.core.arrays.masked.BaseMaskedArray)
    FloatingArray
    IntegerArray
pandas.core.arrays.string_.BaseStringArray(pandas.api.extensions.ExtensionArray)
    StringArray(pandas.core.arrays.string_.BaseStringArray, NumpyExtensionArray)
pandas.core.base.PandasObject(pandas.core.accessor.DirNamesMixin)
    pandas.Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.
    SparseArray(pandas.core.arraylike.OpsMixin, pandas.core.base.PandasObject, pandas.api.extensions.
pandas.core.strings.object_array.ObjectStringArrayMixin(builtins.object)
    pandas.Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.
    ArrowStringArray(pandas.core.strings.object_array.ObjectStringArrayMixin, ArrowExtensionArray)
    NumpyExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays._mixins.NDArrayBacked
    StringArray(pandas.core.arrays.string_.BaseStringArray, NumpyExtensionArray)

class ArrowExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays.base.ExtensionArray):
    ArrowExtensionArray(values: pa.Array | pa.ChunkedArray) -> None

    Pandas ExtensionArray backed by a PyArrow ChunkedArray.

    .. warning::

        ArrowExtensionArray is considered experimental. The implementation and
        parts of the API may change without warning.

    Parameters
    -----
    values : pyarrow.Array or pyarrow.ChunkedArray
        The input data to initialize the ArrowExtensionArray.

    Attributes
    -----
    None

    Methods
    -----
    None

    Returns
    -----
    ArrowExtensionArray

    See Also
    -----
    array : Create a Pandas array with a specified dtype.
    DataFrame.to_feather : Write a DataFrame to the binary Feather format.
    read_feather : Load a feather-format object from the file path.

    Notes
    -----
    Most methods are implemented using `pyarrow` compute functions. <https://arrow.apache.org/docs>
    Some methods may either raise an exception or raise a ``PerformanceWarning`` if an
    associated compute function is not available based on the installed version of PyArrow.

    Please install the latest version of PyArrow to enable the best functionality and avoid
    potential bugs in prior versions of PyArrow.

    Examples
    -----
    Create an ArrowExtensionArray with :func:`pandas.array`:

    >>> pd.array([1, 1, None], dtype="int64[pyarrow]")
    <ArrowExtensionArray>
    [1, 1, <NA>]
    Length: 3, dtype: int64[pyarrow]

    Method resolution order:
        ArrowExtensionArray
        pandas.core.arraylike.OpsMixin
        pandas.core.arrays.base.ExtensionArraySupportsAnyAll
        pandas.api.extensions.ExtensionArray
        pandas.core.arrays._arrow_string_mixins.ArrowStringArrayMixin
        builtins.object

    Methods defined here:

```

```

__abs__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Correctly construct numpy arrays when passed to `np.asarray()`.

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.arrays.arrow.array.ArrowExtensionArray

__arrow_array__(self, type=None) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Convert myself to a pyarrow ChunkedArray.

__contains__(self, key) -> bool from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Return for `item` in `self`.

__getitem__(self, item: PositionalIndexer) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Select a subset of `self`.

    Parameters
    -----
    item : int, slice, or ndarray
        * int: The position in `self` to get.
        * slice: A slice object, where `start`, `stop`, and `step` are
          integers or None
        * ndarray: A 1-d boolean NumPy ndarray the same length as `self`

    Returns
    -----
    item : scalar or ExtensionArray

    Notes
    -----
    For scalar ``item``, return a scalar value suitable for the array's
    type. This should be an instance of ``self.dtype.type``.
    For slice ``key``, return an instance of ``ExtensionArray``, even
    if the slice is length 0 or 1.
    For a boolean mask, return an instance of ``ExtensionArray``, filtered
    to the values where ``item`` is True.

__getstate__(self) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Helper for pickle.

__init__(self, values: pa.Array | pa.ChunkedArray) -> None from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Initialize self. See help(type(self)) for accurate signature.

__invert__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__iter__(self) -> Iterator[Any] from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Iterate over elements of the array.

__len__(self) -> int from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Length of this array.

    Returns
    -----
    length : int

__neg__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__pos__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__setitem__(self, key, value) -> None from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Set one or more values inplace.

    Parameters
    -----
    key : int, ndarray, or slice
        When called from, e.g. ``Series.__setitem__``, ``key`` will be
        one of

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

    value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
        value or values to be set of ``key``.

```

Returns

None

`__setstate__(self, state) -> None` from `pandas.core.arrays.arrow.array.ArrowExtensionArray`

`all(self, *, skipna: bool = True, **kwargs) -> bool | NAType` from `pandas.core.arrays.arrow.array.ArrowExtensionArray`
Return whether all elements are truthy.

Returns True unless there is at least one element that is falsey.
By default, NAs are skipped. If `skipna=False` is specified and missing values are present, similar `:ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

`skipna` : bool, default True

Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be True, as for an empty array.

If `skipna` is False, the result will still be False if there is at least one element that is falsey, otherwise NA will be returned if there are NA's present.

Returns

bool or `:attr:`pandas.NA``

See Also

`ArrowExtensionArray.any` : Return whether any element is truthy.

Examples

The result indicates whether all elements are truthy (and by default skips NAs):

```
>>> pd.array([True, True, pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([1, 1, pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").all()
False
>>> pd.array([], dtype="boolean[pyarrow]").all()
True
>>> pd.array([pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([pd.NA], dtype="float64[pyarrow]").all()
True
```

With `skipna=False`, the result can be NA if this is logically required (whether `pd.NA` is True or False influences the result):

```
>>> pd.array([True, True, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
<NA>
>>> pd.array([1, 1, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
<NA>
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
False
>>> pd.array([1, 0, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
False
```

`any(self, *, skipna: bool = True, **kwargs) -> bool | NAType` from `pandas.core.arrays.arrow.array.ArrowExtensionArray`
Return whether any element is truthy.

Returns False unless there is at least one element that is truthy.
By default, NAs are skipped. If `skipna=False` is specified and missing values are present, similar `:ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

`skipna` : bool, default True

Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be False, as for an empty array.

If `skipna` is False, the result will still be True if there is at least one element that is truthy, otherwise NA will be returned if there are NA's present.

Returns

bool or :attr:`pandas.NA`

See Also

ArrowExtensionArray.all : Return whether all elements are truthy.

Examples

The result indicates whether any element is truthy (and by default skips NAs):

```
>>> pd.array([True, False, True], dtype="boolean[pyarrow]").any()
True
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").any()
True
>>> pd.array([False, False, pd.NA], dtype="boolean[pyarrow]").any()
False
>>> pd.array([], dtype="boolean[pyarrow]").any()
False
>>> pd.array([pd.NA], dtype="boolean[pyarrow]").any()
False
>>> pd.array([pd.NA], dtype="float64[pyarrow]").any()
False
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
True
>>> pd.array([1, 0, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
True
>>> pd.array([False, False, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
<NA>
>>> pd.array([0, 0, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
<NA>
```

argmax(self, skipna: bool = True) -> int from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmin : Return the index of the minimum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

argmin(self, skipna: bool = True) -> int from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

argsort(
 self,
 *,
 ascending: bool = True,
 kind: SortKind = 'quicksort',
 na_position: str = 'last',
 **kwargs
) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the indices that would sort this array.

Parameters

ascending : bool, default True
Whether the indices should result in an ascending
or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
If ``'first'``, put ``NaN`` values at the beginning.
If ``'last'``, put ``NaN`` values at the end.
**kwargs
Passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]
Array of indices that sort ``self``. If NaN values are contained,
NaN values are placed at the end.

See Also

numpy.argsort : Sorting implementation used internally.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

copy(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return a shallow copy of the array.

Underlying ChunkedArray is immutable, so a deep copy is unnecessary.

Returns

type(self)

dropna(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return ArrowExtensionArray without NA values.

Returns

ArrowExtensionArray

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] f
Return boolean ndarray denoting duplicate values.

Parameters

keep : {'first', 'last', False}, default 'first'
- ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
- ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

Returns

ndarray[bool]
With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False, True, False, False, True])

equals(self, other) -> bool from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.
Series.equals : Equivalent method for Series.
DataFrame.equals : Equivalent method for DataFrame.

Examples

>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True

>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M'])
```

```
fillna(
    self,
    value: object | ArrayLike,
    limit: int | None = None,
    copy: bool = True
) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Fill NA/NaN values using the specified method.
```

Parameters

value : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.

limit : int, default None
The maximum number of entries where NA values will be filled.

copy : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
With NA/NaN filled.

See Also

api.extensions.ExtensionArray.dropna : Return ExtensionArray without NA values.
api.extensions.ExtensionArray.isna : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64
```

```
interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
See NDFrame.interpolate.__doc__.
```

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_] from pandas.core.arrays.arrow.array.ArrowExtensionArray
Pointwise comparison for set containment in the given values.

Roughly equivalent to ``np.array([x in values for x in self])``

Parameters

`values` : `np.ndarray` or `ExtensionArray`
Values to compare every element in the array against.

Returns

`np.ndarray[bool]`
With true at indices where value is in ``values``.

See Also

`DataFrame.isin`: Whether each element in the `DataFrame` is contained in values.
`Index.isin`: Return a boolean array where the index values are in values.
`Series.isin`: Whether elements in `Series` are contained in values.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean
```

`isna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.arrow.array.ArrowExtensionArray`
Boolean NumPy array indicating if each value is missing.

This should return a 1-D array the same length as `'self'`.

`map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.arrays.arrow.`
Map values using an input mapping or function.

Parameters

`mapper` : function, dict, or `Series`
Mapping correspondence.
`na_action` : `{None, 'ignore'}`, default `None`
If `'ignore'`, propagate NA values, without passing them to the mapping correspondence. If `'ignore'` is not supported, a ``NotImplementedError`` should be raised.

Returns

`Union[ndarray, Index, ExtensionArray]`
The output of the mapping function applied to the array.
If the function returns a tuple with more than one element a `MultiIndex` will be returned.

`reshape(self, *args, **kwargs) from pandas.core.arrays.arrow.array.ArrowExtensionArray`

`round(self, decimals: int = 0, *args, **kwargs) -> Self from pandas.core.arrays.arrow.array.`
Round each value in the array `a` to the given number of decimals.

Parameters

`decimals` : int, default 0
Number of decimal places to round to. If `decimals` is negative, it specifies the number of positions to the left of the decimal point.
`*args, **kwargs`
Additional arguments and keywords have no effect.

Returns

`ArrowExtensionArray`
Rounded values of the `ArrowExtensionArray`.

See Also

`DataFrame.round` : Round values of a `DataFrame`.
`Series.round` : Round values of a `Series`.

`searchsorted(`
 `self,`
 `value: NumpyValueArrayLike | ExtensionArray,`
 `side: Literal['left', 'right'] = 'left',`

```

    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.arrays.arrow.array.ArrowExtensionArray
Find indices where elements should be inserted to maintain order.

```

Find the indices into a sorted array ``self`` (`a`) such that, if the corresponding elements in ``value`` were inserted before the indices, the order of ``self`` would be preserved.

Assuming that ``self`` is sorted:

```

=====
`side` returned index `i` satisfies
=====
left  ``self[i-1] < value <= self[i]``
right ``self[i-1] <= value < self[i]``
=====

```

Parameters

```

value : array-like, list or scalar
    Value(s) to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort array a into ascending
    order. They are typically the result of argsort.

```

Returns

```

array of ints or int
    If value is array-like, array of insertion points.
    If value is scalar, a single integer.

```

See Also

```

numpy.searchsorted : Similar method from NumPy.

```

Examples

```

>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

```

```

take(
    self,
    indices: TakeIndexer,
    allow_fill: bool = False,
    fill_value: Any = None
) -> ArrowExtensionArray from pandas.core.arrays.arrow.array.ArrowExtensionArray
Take elements from an array.

```

Parameters

```

indices : sequence of int or one-dimensional np.ndarray of int
    Indices to be taken.
allow_fill : bool, default False
    How to handle negative values in `indices`.

    * False: negative values in `indices` indicate positional indices
      from the right (the default). This is similar to
      :func:`numpy.take`.

    * True: negative values in `indices` indicate
      missing values. These values are set to `fill_value`. Any other
      other negative values raise a ``ValueError``.

```

```

fill_value : any, optional
    Fill value to use for NA-indices when `allow_fill` is True.
    This may be ``None``, in which case the default NA value for
    the type, ``self.dtype.na_value``, is used.

```

For many ExtensionArrays, there will be two representations of ``fill_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill_value`` should be the user-facing version, and the implementation should handle translating that to the

physical version for processing the take if necessary.

Returns

ExtensionArray

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When `indices` contains negative values other than ``-1`` and `allow\_fill` is True.

See Also

numpy.take

api.extensions.take

Notes

ExtensionArray.take is called by ``Series.\_\_getitem\_\_``, ``.loc``, ``iloc``, when `indices` is a sequence of values. Additionally, it's called by :meth:`Series.reindex`, or any other method that causes realignment, with a `fill\_value`.

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
Convert to a NumPy ndarray.
```

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

```
unique(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Compute the ArrowExtensionArray of unique values.
```

Returns

ArrowExtensionArray

```
value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return a Series containing counts of each unique value.
```

Parameters

dropna : bool, default True

Don't include counts of missing values.

Returns

counts : Series

See Also

Series.value_counts

Readonly properties defined here:

`dtype`
An instance of 'ExtensionDtype'.

`nbytes`
The number of bytes needed to store this object in memory.

Data and other attributes defined here:

`__annotations__` = {'_dtype': 'ArrowDtype', '_pa_array': 'pa.ChunkedArr...'

Methods inherited from `pandas.core.arraylike.OpsMixin`:

`__add__(self, other)`
Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ```other``` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
 height weight
elk 1.5 500
moose 2.6 800

Adding a scalar affects all rows and columns.

>>> df[["height", "weight"]] + 1.5
 height weight
elk 3.0 501.5
moose 4.1 801.5

Each element of a list is added to a column of the DataFrame, in order.

>>> df[["height", "weight"]] + [0.5, 1.5]
 height weight
elk 2.0 501.5
moose 3.1 801.5

Keys of a dictionary are aligned to the DataFrame, based on column names;
each value in the dictionary is added to the corresponding column.

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
 height weight
elk 2.0 501.5
moose 3.1 801.5

When ```other``` is a `:class:`Series``, the index of ```other``` is aligned with the
columns of the DataFrame.

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
 height weight
elk 3.0 500.5
moose 4.1 800.5

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk height moose weight
elk   NaN    NaN   NaN    NaN
moose NaN    NaN   NaN    NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height weight
elk      2.0   500.5
moose     4.1   801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height weight
deer      NaN    NaN
elk       1.7    NaN
moose     3.0    NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
```

```

__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)
-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
__hash__ = None
-----
Methods inherited from pandas.api.extensions.ExtensionArray:
__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).

astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike from pandas.core.arrays.base.
    Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters
-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also
-----
Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
a sequence.

Examples
-----
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype

```

```
Float64Dtype()
```

Otherwise, we will get a Numpy ndarray:

```
>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')
```

```
delete(self, loc: PositionalIndexer) -> Self from pandas.core.arrays.base.ExtensionArray
```

```
insert(self, loc: int, item) -> Self from pandas.core.arrays.base.ExtensionArray
    Insert an item at the given position.
```

Parameters

loc : int

Index where the `item` needs to be inserted.

item : scalar-like

Value to be inserted.

Returns

ExtensionArray

With `item` inserted at `loc`.

See Also

Index.insert: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item cannot be held in an array of this type/dtype, either ValueError or TypeError should be raised.

The default implementation relies on `_from_sequence` to raise on invalid items.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)
<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64
```

```
item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
    Return the array element at the specified position as a Python scalar.
```

Parameters

index : int, optional

Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar

The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

IndexError

If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
```

```

>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)

```

`ravel(self, order: Literal['C', 'F', 'A', 'K'] | None = 'C') -> Self` from `pandas.core.arrays`
Return a flattened view on this array.

Parameters

order : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

ExtensionArray
A flattened view on the array.

See Also

ExtensionArray.tolist: Return a list of the values.

Notes

- Because ExtensionArrays are 1D-only, this is a no-op.
- The "order" argument is ignored, is for compatibility with NumPy.

Examples

>>> pd.array([1, 2, 3]).ravel()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

`repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self` from `pandas`.
Repeat elements of an ExtensionArray.

Returns a new ExtensionArray where each element of the current ExtensionArray is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty ExtensionArray.
axis : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

ExtensionArray
Newly created ExtensionArray with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
Index.repeat : Equivalent function for Index.
numpy.repeat : Similar method for :class:`numpy.ndarray`.
ExtensionArray.take : Take arbitrary positions.

Examples

>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']

`shift(self, periods: int = 1, fill_value: object = None) -> ExtensionArray from pandas.core.`
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

`periods` : int, default 1
The number of periods to shift. Negative values are allowed
for shifting backwards.

`fill_value` : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

`api.extensions.ExtensionArray.transpose` : Return a transposed view on
this array.
`api.extensions.ExtensionArray.factorize` : Encode the extension array as an
enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
returned.

If ``periods > len(self)`` , then an array of size
`len(self)` is returned, with all values filled with
``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along `axis=0`.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64
```

`tolist(self) -> list from pandas.core.arrays.base.ExtensionArray`
Return a list of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list
Python list of values in array.

See Also

`Index.tolist`: Return a list of the values in the Index.
`Series.tolist`: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

`transpose(self, *axes: int) -> Self from pandas.core.arrays.base.ExtensionArray`
Return a transposed view on this array.

Because ExtensionArrays are always 1D, this is a no-op. It is included
for compatibility with `np.ndarray`.

Returns

ExtensionArray

Examples

```
>>> pd.array([1, 2, 3]).transpose()
```

```
<IntegerArray>
```

```
[1, 2, 3]
```

```
Length: 3, dtype: Int64
```

```
view(self, dtype: Dtype | None = None) -> ArrayLike from pandas.core.arrays.base.ExtensionArray
Return a view on the array.
```

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional

Default None.

Returns

ExtensionArray or np.ndarray

A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.

Index.view: Equivalent function for Index.

ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr2 = arr.view()
```

```
>>> arr[0] = 2
```

```
>>> arr2
```

```
<IntegerArray>
```

```
[2, 2, 3]
```

```
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.api.extensions.ExtensionArray:

T

ndim

Extension Arrays are only allowed to be 1-dimensional.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.

ExtensionArray.size: The number of elements in the array.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr.ndim
```

```
1
```

shape

Return a tuple of the array dimensions.

See Also

numpy.ndarray.shape : Similar attribute which returns the shape of an array.

DataFrame.shape : Return a tuple representing the dimensionality of the DataFrame.

Series.shape : Return a tuple representing the dimensionality of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr.shape
```

```

        (3,)
    size
        The number of elements in the array.
    -----
    Data and other attributes inherited from pandas.api.extensions.ExtensionArray:
    __pandas_priority__ = 1000

class ArrowStringArray(pandas.core.strings.object_array.ObjectStringArrayMixin, ArrowExtensionArray):
    ArrowStringArray(values, *, dtype: StringDtype | None = None) -> None

    Extension array for string data in a ``pyarrow.ChunkedArray``.

    .. warning::

        ArrowStringArray is considered experimental. The implementation and
        parts of the API may change without warning.

    Parameters
    -----
    values : pyarrow.Array or pyarrow.ChunkedArray
        The array of data.
    dtype : StringDtype
        The dtype for the array.

    Attributes
    -----
    None

    Methods
    -----
    None

    See Also
    -----
    :func:`array`
        The recommended function for creating an ArrowStringArray.
    Series.str
        The string methods are available on Series backed by
        an ArrowStringArray.

    Notes
    -----
    ArrowStringArray returns a BooleanArray for comparison methods.

    Examples
    -----
    >>> pd.array(["This is", "some text", None, "data."], dtype="string[pyarrow]")
    <ArrowStringArray>
    ['This is', 'some text', <NA>, 'data.']
    Length: 4, dtype: string

    Method resolution order:
    ArrowStringArray
    pandas.core.strings.object_array.ObjectStringArrayMixin
    ArrowExtensionArray
    pandas.core.arraylike.OpsMixin
    pandas.core.arrays.base.ExtensionArraySupportsAnyAll
    pandas.core.arrays.string_.BaseStringArray
    pandas.api.extensions.ExtensionArray
    pandas.core.arrays._arrow_string_mixins.ArrowStringArrayMixin
    builtins.object

    Methods defined here:

    __init__(self, values, *, dtype: StringDtype | None = None) -> None from pandas.core.arrays.string_.BaseStringArray
        Initialize self. See help(type(self)) for accurate signature.

    __len__(self) -> int from pandas.core.arrays.string_arrow.ArrowStringArray
        Length of this array.

    Returns
    -----
    length : int

```

`__pos__(self) -> Self` from `pandas.core.arrays.string_arrow.ArrowStringArray`

`astype(self, dtype, copy: bool = True)` from `pandas.core.arrays.string_arrow.ArrowStringArray`
Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters

`dtype` : str or dtype

Typecode or data-type to which the array is cast.

`copy` : bool, default True

Whether to copy the data, even if not necessary. If False, a copy is made only if the old dtype does not match the new dtype.

Returns

`np.ndarray` or `pandas.api.extensions.ExtensionArray`

An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``, otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also

`Series.astype` : Cast a Series to a different dtype.

`DataFrame.astype` : Cast a DataFrame to a different dtype.

`api.extensions.ExtensionArray` : Base class for ExtensionArray objects.

`core.arrays.DatetimeArray._from_sequence` : Create a DatetimeArray from a sequence.

`core.arrays.TimedeltaArray._from_sequence` : Create a TimedeltaArray from a sequence.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr
```

```
<IntegerArray>
```

```
[1, 2, 3]
```

```
Length: 3, dtype: Int64
```

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

```
>>> arr1 = arr.astype("Float64")
```

```
>>> arr1
```

```
<FloatingArray>
```

```
[1.0, 2.0, 3.0]
```

```
Length: 3, dtype: Float64
```

```
>>> arr1.dtype
```

```
Float64Dtype()
```

Otherwise, we will get a Numpy ndarray:

```
>>> arr2 = arr.astype("float64")
```

```
>>> arr2
```

```
array([1., 2., 3.])
```

```
>>> arr2.dtype
```

```
dtype('float64')
```

`insert(self, loc: int, item) -> ArrowStringArray` from `pandas.core.arrays.string_arrow.ArrowStringArray`
Insert an item at the given position.

Parameters

`loc` : int

Index where the `item` needs to be inserted.

`item` : scalar-like

Value to be inserted.

Returns

`ExtensionArray`

With `item` inserted at `loc`.

See Also

`Index.insert`: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item cannot be held in an array of this type/dtype, either `ValueError` or `TypeError` should be raised.

The default implementation relies on `_from_sequence` to raise on invalid items.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)
<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64
```

`isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]` from `pandas.core.arrays.string_arrow.`
Pointwise comparison for set containment in the given values.

Roughly equivalent to ``np.array([x in values for x in self])``

Parameters

`values` : `np.ndarray` or `ExtensionArray`
Values to compare every element in the array against.

Returns

`np.ndarray[bool]`
With true at indices where value is in ``values``.

See Also

`DataFrame.isin`: Whether each element in the `DataFrame` is contained in values.
`Index.isin`: Return a boolean array where the index values are in values.
`Series.isin`: Whether elements in `Series` are contained in values.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean
```

`value_counts(self, dropna: bool = True) -> Series` from `pandas.core.arrays.string_arrow.ArrowStringArray`
Return a `Series` containing counts of each unique value.

Parameters

`dropna` : `bool`, default `True`
Don't include counts of missing values.

Returns

`counts` : `Series`

See Also

`Series.value_counts`

Readonly properties defined here:

`dtype`
An instance of `'string[pyarrow]'`.

Data and other attributes defined here:

`__annotations__` = `{'_dtype': 'StringDtype'}`

Data descriptors inherited from `pandas.core.strings.object_array.ObjectStringArrayMixin`:

`__dict__`
dictionary for instance variables

```

__weakref__
    list of weak references to the object

-----
Methods inherited from ArrowExtensionArray:

__abs__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Correctly construct numpy arrays when passed to `np.asarray()`.

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.arrays.arrow.array.ArrowExtensionArray

__arrow_array__(self, type=None) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Convert myself to a pyarrow ChunkedArray.

__contains__(self, key) -> bool from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Return for `item in self`.

__getitem__(self, item: PositionalIndexer) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Select a subset of self.

Parameters
-----
item : int, slice, or ndarray
    * int: The position in 'self' to get.
    * slice: A slice object, where 'start', 'stop', and 'step' are
      integers or None
    * ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

Returns
-----
item : scalar or ExtensionArray

Notes
-----
For scalar ``item``, return a scalar value suitable for the array's
type. This should be an instance of ``self.dtype.type``.
For slice ``key``, return an instance of ``ExtensionArray``, even
if the slice is length 0 or 1.
For a boolean mask, return an instance of ``ExtensionArray``, filtered
to the values where ``item`` is True.

__getstate__(self) from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Helper for pickle.

__invert__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__iter__(self) -> Iterator[Any] from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Iterate over elements of the array.

__neg__(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray

__setitem__(self, key, value) -> None from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Set one or more values inplace.

Parameters
-----
key : int, ndarray, or slice
    When called from, e.g. ``Series.__setitem__``, ``key`` will be
    one of

    * scalar int
    * ndarray of integers.
    * boolean ndarray
    * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
    value or values to be set of ``key``.

Returns
-----
None

__setstate__(self, state) -> None from pandas.core.arrays.arrow.array.ArrowExtensionArray

all(self, *, skipna: bool = True, **kwargs) -> bool | NAType from pandas.core.arrays.arrow.array.ArrowExtensionArray

```

Return whether all elements are truthy.

Returns True unless there is at least one element that is falsey.
By default, NAs are skipped. If ``skipna=False`` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

skipna : bool, default True
Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be True, as for an empty array. If `skipna` is False, the result will still be False if there is at least one element that is falsey, otherwise NA will be returned if there are NA's present.

Returns

bool or :attr:`pandas.NA`

See Also

ArrowExtensionArray.any : Return whether any element is truthy.

Examples

The result indicates whether all elements are truthy (and by default skips NAs):

```
>>> pd.array([True, True, pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([1, 1, pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").all()
False
>>> pd.array([], dtype="boolean[pyarrow]").all()
True
>>> pd.array([pd.NA], dtype="boolean[pyarrow]").all()
True
>>> pd.array([pd.NA], dtype="float64[pyarrow]").all()
True
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, True, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
<NA>
>>> pd.array([1, 1, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
<NA>
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
False
>>> pd.array([1, 0, pd.NA], dtype="boolean[pyarrow]").all(skipna=False)
False
```

```
any(self, *, skipna: bool = True, **kwargs) -> bool | NAType from pandas.core.arrays.arrow.array
Return whether any element is truthy.
```

Returns False unless there is at least one element that is truthy.
By default, NAs are skipped. If ``skipna=False`` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

skipna : bool, default True
Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be False, as for an empty array. If `skipna` is False, the result will still be True if there is at least one element that is truthy, otherwise NA will be returned if there are NA's present.

Returns

bool or :attr:`pandas.NA`

See Also

ArrowExtensionArray.all : Return whether all elements are truthy.

Examples

The result indicates whether any element is truthy (and by default skips NAs):

```
>>> pd.array([True, False, True], dtype="boolean[pyarrow]").any()
True
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").any()
True
>>> pd.array([False, False, pd.NA], dtype="boolean[pyarrow]").any()
False
>>> pd.array([], dtype="boolean[pyarrow]").any()
False
>>> pd.array([pd.NA], dtype="boolean[pyarrow]").any()
False
>>> pd.array([pd.NA], dtype="float64[pyarrow]").any()
False
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, False, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
True
>>> pd.array([1, 0, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
True
>>> pd.array([False, False, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
<NA>
>>> pd.array([0, 0, pd.NA], dtype="boolean[pyarrow]").any(skipna=False)
<NA>
```

argmax(self, skipna: bool = True) -> int from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the minimum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

argmin(self, skipna: bool = True) -> int from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
```



```

>>> arr.argmax()
np.int64(1)

argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs
) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return the indices that would sort this array.

Parameters
-----
ascending : bool, default True
    Whether the indices should result in an ascending
    or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
    Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
    If ``'first'``, put ``NaN`` values at the beginning.
    If ``'last'``, put ``NaN`` values at the end.
**kwargs
    Passed through to :func:`numpy.argsort`.

Returns
-----
np.ndarray[np.intp]
    Array of indices that sort ``self``. If NaN values are contained,
    NaN values are placed at the end.

See Also
-----
numpy.argsort : Sorting implementation used internally.

Examples
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

copy(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return a shallow copy of the array.

Underlying ChunkedArray is immutable, so a deep copy is unnecessary.

Returns
-----
type(self)

dropna(self) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return ArrowExtensionArray without NA values.

Returns
-----
ArrowExtensionArray

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]
Return boolean ndarray denoting duplicate values.

Parameters
-----
keep : {'first', 'last', False}, default 'first'
    - ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
    - ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
    - False : Mark all duplicates as ``True``.

Returns
-----
ndarray[bool]
    With true in indices where elements are duplicated and false otherwise.

See Also
-----
DataFrame.duplicated : Return boolean Series denoting
    duplicate rows.

```

Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

```
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])
```

equals(self, other) -> bool from pandas.core.arrays.arrow.array.ArrowExtensionArray
Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.
Series.equals : Equivalent method for Series.
DataFrame.equals : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```

-----
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

fillna(
    self,
    value: object | ArrayLike,
    limit: int | None = None,
    copy: bool = True
) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Fill NA/NaN values using the specified method.

Parameters
-----
value : scalar, array-like
    If a scalar value is passed it is used to fill all missing values.
    Alternatively, an array-like "value" can be given. It's expected
    that the array-like have the same length as 'self'.
limit : int, default None
    The maximum number of entries where NA values will be filled.
copy : bool, default True
    Whether to make a copy of the data before filling. If False, then
    the original should be modified and no new memory should be allocated.
    For ExtensionArray subclasses that cannot do this, it is at the
    author's discretion whether to ignore "copy=False" or to raise.

Returns
-----
ExtensionArray
    With NA/NaN filled.

See Also
-----
api.extensions.ExtensionArray.dropna : Return ExtensionArray without
    NA values.
api.extensions.ExtensionArray.isna : A 1-D array indicating if
    each value is missing.

Examples
-----
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
See NDFrame.interpolate.__doc__.

isna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.arrow.array.ArrowExtensionArray
Boolean NumPy array indicating if each value is missing.

This should return a 1-D array the same length as 'self'.

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.arrays.arrow.array.ArrowExtensionArray
Map values using an input mapping or function.

Parameters
-----

```

```

mapper : function, dict, or Series
    Mapping correspondence.
na_action : {None, 'ignore'}, default None
    If 'ignore', propagate NA values, without passing them to the
    mapping correspondence. If 'ignore' is not supported, a
    ``NotImplementedError`` should be raised.

Returns
-----
Union[ndarray, Index, ExtensionArray]
    The output of the mapping function applied to the array.
    If the function returns a tuple with more than one element
    a MultiIndex will be returned.

reshape(self, *args, **kwargs) from pandas.core.arrays.arrow.array.ArrowExtensionArray

round(self, decimals: int = 0, *args, **kwargs) -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Round each value in the array a to the given number of decimals.

Parameters
-----
decimals : int, default 0
    Number of decimal places to round to. If decimals is negative,
    it specifies the number of positions to the left of the decimal point.
*args, **kwargs
    Additional arguments and keywords have no effect.

Returns
-----
ArrowExtensionArray
    Rounded values of the ArrowExtensionArray.

See Also
-----
DataFrame.round : Round values of a DataFrame.
Series.round : Round values of a Series.

searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted array `self` (a) such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    Assuming that `self` is sorted:

    =====
    `side` returned index `i` satisfies
    =====
    left    ``self[i-1] < value <= self[i]``
    right   ``self[i-1] <= value < self[i]``
    =====

Parameters
-----
value : array-like, list or scalar
    Value(s) to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort array a into ascending
    order. They are typically the result of argsort.

Returns
-----
array of ints or int
    If value is array-like, array of insertion points.
    If value is scalar, a single integer.

See Also

```

```

-----
numpy.searchsorted : Similar method from NumPy.

Examples
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

take(
    self,
    indices: TakeIndexer,
    allow_fill: bool = False,
    fill_value: Any = None
) -> ArrowExtensionArray from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Take elements from an array.

Parameters
-----
indices : sequence of int or one-dimensional np.ndarray of int
    Indices to be taken.
allow_fill : bool, default False
    How to handle negative values in `indices`.

    * False: negative values in `indices` indicate positional indices
      from the right (the default). This is similar to
      :func:`numpy.take`.

    * True: negative values in `indices` indicate
      missing values. These values are set to `fill_value`. Any other
      other negative values raise a ``ValueError``.

fill_value : any, optional
    Fill value to use for NA-indices when `allow_fill` is True.
    This may be ``None``, in which case the default NA value for
    the type, ``self.dtype.na_value``, is used.

    For many ExtensionArrays, there will be two representations of
    `fill_value`: a user-facing "boxed" scalar, and a low-level
    physical NA value. `fill_value` should be the user-facing version,
    and the implementation should handle translating that to the
    physical version for processing the take if necessary.

Returns
-----
ExtensionArray

Raises
-----
IndexError
    When the indices are out of bounds for the array.
ValueError
    When `indices` contains negative values other than ``-1``
    and `allow_fill` is True.

See Also
-----
numpy.take
api.extensions.take

Notes
-----
ExtensionArray.take is called by ``Series.__getitem``, ``.loc``,
``.iloc``, when `indices` is a sequence of values. Additionally,
it's called by :meth:`Series.reindex`, or any other method
that causes realignment, with a `fill_value`.

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.arrow.array.ArrowExtensionArray
    Convert to a NumPy ndarray.

    This is similar to :meth:`numpy.asarray`, but may provide additional control
    over how the conversion is done.

```

Parameters

`dtype` : str or numpy.dtype, optional
The dtype to pass to :meth:`numpy.asarray`.
`copy` : bool, default False
Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.
`na_value` : Any, optional
The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

`unique(self)` -> Self from pandas.core.arrays.arrow.array.ArrowExtensionArray
Compute the ArrowExtensionArray of unique values.

Returns

ArrowExtensionArray

Readonly properties inherited from ArrowExtensionArray:

`nbytes`
The number of bytes needed to store this object in memory.

Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`
Get Addition of DataFrame and other, column-wise.
Equivalent to ``DataFrame.add(other)``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0   501.5
moose      3.1   801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0   500.5
moose      4.1   800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0   500.5
moose      4.1   801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk       1.7     NaN
moose     3.0     NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
```

```

__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
__hash__ = None

-----
Methods inherited from pandas.core.arrays.string_.BaseStringArray:
tolist(self) -> list
    Return a list of the value.

    These are each a scalar type, which is a Python scalar
    (for str, int, float) or pandas scalar
    (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    list

    Examples
    -----
    >>> arr = pd.array(["a", "b", "c"])
    >>> arr.tolist()
    ['a', 'b', 'c']

view(self, dtype: Dtype | None = None) -> Self
    Return a view on the array.

    Parameters
    -----
    dtype : str, np.dtype, or ExtensionDtype, optional
        Default None.

    Returns
    -----
    ExtensionArray or np.ndarray
        A view on the :class:`ExtensionArray`'s data.

    See Also
    -----
    api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
    Index.view: Equivalent function for Index.
    ndarray.view: New view of array with the same data.

    Examples
    -----
    This gives view on the underlying data of an ``ExtensionArray`` and is not a

```


copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Methods inherited from pandas.api.extensions.ExtensionArray:

__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
Return repr(self).

delete(self, loc: PositionalIndexer) -> Self from pandas.core.arrays.base.ExtensionArray

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional

Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar

The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

IndexError

If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

ravel(self, order: Literal['C', 'F', 'A', 'K'] | None = 'C') -> Self from pandas.core.arrays
Return a flattened view on this array.

Parameters

order : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

ExtensionArray

A flattened view on the array.

See Also

ExtensionArray.tolist: Return a list of the values.

Notes

- Because ExtensionArrays are 1D-only, this is a no-op.

- The "order" argument is ignored, is for compatibility with NumPy.

Examples

```
-----
>>> pd.array([1, 2, 3]).ravel()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

`repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self` from pandas.
Repeat elements of an ExtensionArray.

Returns a new ExtensionArray where each element of the current ExtensionArray is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty ExtensionArray.
`axis` : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

ExtensionArray
Newly created ExtensionArray with repeated elements.

See Also

`Series.repeat` : Equivalent function for Series.
`Index.repeat` : Equivalent function for Index.
`numpy.repeat` : Similar method for :class:`numpy.ndarray`.
`ExtensionArray.take` : Take arbitrary positions.

Examples

```
-----
>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
```

`shift(self, periods: int = 1, fill_value: object = None) -> ExtensionArray` from pandas.core.
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

`periods` : int, default 1
The number of periods to shift. Negative values are allowed for shifting backwards.
`fill_value` : object, optional
The scalar value to use for newly introduced missing values. The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

`api.extensions.ExtensionArray.transpose` : Return a transposed view on this array.
`api.extensions.ExtensionArray.factorize` : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)``, then an array of size `len(self)` is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

`transpose(self, *axes: int) -> Self` from `pandas.core.arrays.base.ExtensionArray`
Return a transposed view on this array.

Because ExtensionArrays are always 1D, this is a no-op. It is included for compatibility with `np.ndarray`.

Returns

ExtensionArray

Examples

>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Readonly properties inherited from `pandas.api.extensions.ExtensionArray`:

T

`ndim`
Extension Arrays are only allowed to be 1-dimensional.

See Also

`ExtensionArray.shape`: Return a tuple of the array dimensions.
`ExtensionArray.size`: The number of elements in the array.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.ndim
1

shape

Return a tuple of the array dimensions.

See Also

`numpy.ndarray.shape` : Similar attribute which returns the shape of an array.
`DataFrame.shape` : Return a tuple representing the dimensionality of the `DataFrame`.
`Series.shape` : Return a tuple representing the dimensionality of the `Series`.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shape
(3,)

size

The number of elements in the array.

Data and other attributes inherited from `pandas.api.extensions.ExtensionArray`:

```

|   __pandas_priority__ = 1000
class BooleanArray(pandas.core.arrays.masked.BaseMaskedArray)
|   BooleanArray(values: np.ndarray, mask: np.ndarray, copy: bool = False) -> None
|
|   Array of boolean (True/False) data with missing values.
|
|   This is a pandas Extension array for boolean data, under the hood
|   represented by 2 numpy arrays: a boolean array with the data and
|   a boolean array with the mask (True indicating missing).
|
|   BooleanArray implements Kleene logic (sometimes called three-value
|   logic) for logical operations. See :ref:`boolean.kleene` for more.
|
|   To construct a BooleanArray from generic array-like input, use
|   :func:`pandas.array` specifying ``dtype="boolean"`` (see examples
|   below).
|
|   .. warning::
|
|       BooleanArray is considered experimental. The implementation and
|       parts of the API may change without warning.
|
|   Parameters
|   -----
|   values : numpy.ndarray
|       A 1-d boolean-dtype array with the data.
|   mask : numpy.ndarray
|       A 1-d boolean-dtype array indicating missing values (True
|       indicates missing).
|   copy : bool, default False
|       Whether to copy the `values` and `mask` arrays.
|
|   Attributes
|   -----
|   None
|
|   Methods
|   -----
|   None
|
|   Returns
|   -----
|   BooleanArray
|
|   See Also
|   -----
|   array : Create an array from data with the appropriate dtype.
|   BooleanDtype : Extension dtype for boolean data.
|   Series : One-dimensional ndarray with axis labels (including time series).
|   DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.
|
|   Examples
|   -----
|   Create a BooleanArray with :func:`pandas.array`:
|
|   >>> pd.array([True, False, None], dtype="boolean")
|   <BooleanArray>
|   [True, False, <NA>]
|   Length: 3, dtype: boolean
|
|   Method resolution order:
|       BooleanArray
|       pandas.core.arrays.masked.BaseMaskedArray
|       pandas.core.arraylike.OpsMixin
|       pandas.api.extensions.ExtensionArray
|       builtins.object
|
|   Methods defined here:
|
|   __init__(self, values: np.ndarray, mask: np.ndarray, copy: bool = False) -> None from pandas
|       Initialize self. See help(type(self)) for accurate signature.
|
|   -----
|   Readonly properties defined here:
|
|   dtype

```

An instance of ExtensionDtype.

See Also

api.extensions.ExtensionDtype : Base class for extension dtypes.
api.extensions.ExtensionArray : Base class for extension array types.
api.extensions.ExtensionArray.dtype : The dtype of an ExtensionArray.
Series.dtype : The dtype of a Series.
DataFrame.dtype : The dtype of a DataFrame.

Examples

>>> pd.array([1, 2, 3]).dtype
Int64Dtype()

Methods inherited from pandas.core.arrays.masked.BaseMaskedArray:

__abs__(self) -> Self

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray
the array interface, return my values
We return an object array here to preserve our scalar values

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs)

__arrow_array__(self, type=None)
Convert myself into a pyarrow Array.

__contains__(self, key) -> bool
Return for `item in self`.

__getitem__(self, item: PositionalIndexer) -> Self | Any
Select a subset of self.

Parameters

item : int, slice, or ndarray
* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

__invert__(self) -> Self

__iter__(self) -> Iterator
Iterate over elements of the array.

__len__(self) -> int
Length of this array

Returns

length : int

__neg__(self) -> Self

__pos__(self) -> Self

```

__setitem__(self, key, value) -> None
    Set one or more values inplace.

    This method is not required to satisfy the pandas extension array
    interface.

    Parameters
    -----
    key : int, ndarray, or slice
        When called from, e.g. ``Series.__setitem__``, ``key`` will be
        one of

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

    value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
        value or values to be set of ``key``.

    Returns
    -----
    None

    Raises
    -----
    ValueError
        If the array is readonly and modification is attempted.

all(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType
    Return whether all elements are truthy.

    Returns True unless there is at least one element that is falsey.
    By default, NAs are skipped. If ``skipna=False`` is specified and
    missing values are present, similar :ref:`Kleene logic <boolean.kleene>`
    is used as for logical operations.

    Parameters
    -----
    skipna : bool, default True
        Exclude NA values. If the entire array is NA and ``skipna`` is
        True, then the result will be True, as for an empty array.
        If ``skipna`` is False, the result will still be False if there is
        at least one element that is falsey, otherwise NA will be returned
        if there are NA's present.
    axis : int, optional, default 0
    **kwargs : any, default None
        Additional keywords have no effect but might be accepted for
        compatibility with NumPy.

    Returns
    -----
    bool or :attr:`pandas.NA`

    See Also
    -----
    numpy.all : Numpy version of this method.
    BooleanArray.any : Return whether any element is truthy.

    Examples
    -----
    The result indicates whether all elements are truthy (and by default
    skips NAs):

    >>> pd.array([True, True, pd.NA]).all()
    np.True_
    >>> pd.array([1, 1, pd.NA]).all()
    np.True_
    >>> pd.array([True, False, pd.NA]).all()
    np.False_
    >>> pd.array([], dtype="boolean").all()
    np.True_
    >>> pd.array([pd.NA], dtype="boolean").all()
    np.True_
    >>> pd.array([pd.NA], dtype="Float64").all()
    np.True_

```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, True, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([1, 1, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([True, False, pd.NA]).all(skipna=False)
np.False_
>>> pd.array([1, 0, pd.NA]).all(skipna=False)
np.False_
```

`any(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType`
Return whether any element is truthy.

Returns False unless there is at least one element that is truthy. By default, NAs are skipped. If ``skipna=False`` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

`skipna` : bool, default True
Exclude NA values. If the entire array is NA and ``skipna`` is True, then the result will be False, as for an empty array. If ``skipna`` is False, the result will still be True if there is at least one element that is truthy, otherwise NA will be returned if there are NA's present.
`axis` : int, optional, default 0
`**kwargs` : any, default None
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

bool or :attr:`pandas.NA`

See Also

`numpy.any` : Numpy version of this method.
`BaseMaskedArray.all` : Return whether all elements are truthy.

Examples

The result indicates whether any element is truthy (and by default skips NAs):

```
>>> pd.array([True, False, True]).any()
np.True_
>>> pd.array([True, False, pd.NA]).any()
np.True_
>>> pd.array([False, False, pd.NA]).any()
np.False_
>>> pd.array([], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="Float64").any()
np.False_
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, False, pd.NA]).any(skipna=False)
np.True_
>>> pd.array([1, 0, pd.NA]).any(skipna=False)
np.True_
>>> pd.array([False, False, pd.NA]).any(skipna=False)
<NA>
>>> pd.array([0, 0, pd.NA]).any(skipna=False)
<NA>
```

`astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike`
Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters

```

-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also
-----
Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
    sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
    a sequence.

```

Examples

```

-----
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

```

```

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()

```

Otherwise, we will get a Numpy ndarray:

```

>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')

```

```

copy(self) -> Self
    Return a copy of the array.

```

This method creates a copy of the `ExtensionArray` where modifying the data in the copy will not affect the original array. This is useful when you want to manipulate data without altering the original dataset.

Returns

```

-----
ExtensionArray
    A new ExtensionArray object that is a copy of the current instance.

```

See Also

```

-----
DataFrame.copy : Return a copy of the DataFrame.
Series.copy : Return a copy of the Series.

```

Examples

```

-----
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

```



```

delete(self, loc, axis: AxisInt = 0) -> Self

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]
    Return boolean ndarray denoting duplicate values.

    Parameters
    -----
    keep : {'first', 'last', False}, default 'first'
        - ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
        - ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
        - False : Mark all duplicates as ``True``.

    Returns
    -----
    ndarray[bool]
        With true in indices where elements are duplicated and false otherwise.

    See Also
    -----
    DataFrame.duplicated : Return boolean Series denoting
        duplicate rows.
    Series.duplicated : Indicate duplicate Series values.
    api.extensions.ExtensionArray.unique : Compute the ExtensionArray
        of unique values.

    Examples
    -----
    >>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
    array([False,  True, False, False,  True])

```

equals(self, other) -> bool
 Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

```

    Parameters
    -----
    other : ExtensionArray
        Array to compare to this Array.

    Returns
    -----
    boolean
        whether the arrays are equivalent.

    See Also
    -----
    numpy.array_equal : Equivalent method for numpy array.
    Series.equals : Equivalent method for Series.
    DataFrame.equals : Equivalent method for DataFrame.

    Examples
    -----
    >>> arr1 = pd.array([1, 2, np.nan])
    >>> arr2 = pd.array([1, 2, np.nan])
    >>> arr1.equals(arr2)
    True

    >>> arr1 = pd.array([1, 3, np.nan])
    >>> arr2 = pd.array([1, 2, np.nan])
    >>> arr1.equals(arr2)
    False

```

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray]
 Encode the extension array as an enumerated type.

```

    Parameters
    -----
    use_na_sentinel : bool, default True
        If True, the sentinel -1 will be used for NaN values. If False,
        NaN values will be encoded as non-negative integers and will not drop the
        NaN from the uniques of the values.

    Returns
    -----

```

```

codes : ndarray
    An integer NumPy array that's an indexer into the original
    ExtensionArray.
uniques : ExtensionArray
    An ExtensionArray containing the unique values of `self`.

    .. note::

        uniques will not contain an entry for the NA value of
        the ExtensionArray if there are any missing values present
        in `self`.

```

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```

-----
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

```

```

fillna(self, value, limit: int | None = None, copy: bool = True) -> Self
    Fill NA/NaN values using the specified method.

```

Parameters

value : scalar, array-like
 If a scalar value is passed it is used to fill all missing values.
 Alternatively, an array-like "value" can be given. It's expected
 that the array-like have the same length as 'self'.

limit : int, default None
 The maximum number of entries where NA values will be filled.

copy : bool, default True
 Whether to make a copy of the data before filling. If False, then
 the original should be modified and no new memory should be allocated.
 For ExtensionArray subclasses that cannot do this, it is at the
 author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
 With NA/NaN filled.

See Also

api.extensions.ExtensionArray.dropna : Return ExtensionArray without
 NA values.

api.extensions.ExtensionArray.isna : A 1-D array indicating if
 each value is missing.

Examples

```

-----
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

```

```

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index,
    limit,
    limit_direction,

```

```

        limit_area,
        copy: bool,
        **kwargs
    ) -> FloatingArray
        See NDFrame.interpolate.__doc__.

isin(self, values: ArrayLike) -> BooleanArray
    Pointwise comparison for set containment in the given values.

    Roughly equivalent to `np.array([x in values for x in self])`

    Parameters
    -----
    values : np.ndarray or ExtensionArray
        Values to compare every element in the array against.

    Returns
    -----
    np.ndarray[bool]
        With true at indices where value is in `values`.

    See Also
    -----
    DataFrame.isin: Whether each element in the DataFrame is contained in values.
    Index.isin: Return a boolean array where the index values are in values.
    Series.isin: Whether elements in Series are contained in values.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.isin([1])
    <BooleanArray>
    [True, False, False]
    Length: 3, dtype: boolean

isna(self) -> np.ndarray
    A 1-D array indicating if each value is missing.

    Returns
    -----
    numpy.ndarray or pandas.api.extensions.ExtensionArray
        In most cases, this should return a NumPy ndarray. For
        exceptional cases like ``SparseArray``, where returning
        an ndarray would be expensive, an ExtensionArray may be
        returned.

    See Also
    -----
    ExtensionArray.dropna: Return ExtensionArray without NA values.
    ExtensionArray.fillna: Fill NA/NaN values using the specified method.

    Notes
    -----
    If returning an ExtensionArray, then

    * ``na_values._is_boolean`` should be True
    * ``na_values`` should implement :func:`ExtensionArray._reduce`
    * ``na_values`` should implement :func:`ExtensionArray._accumulate`
    * ``na_values.any`` and ``na_values.all`` should be implemented

    Examples
    -----
    >>> arr = pd.array([1, 2, np.nan, np.nan])
    >>> arr.isna()
    array([False, False,  True,  True])

map(self, mapper, na_action: Literal['ignore'] | None = None)
    Map values using an input mapping or function.

    Parameters
    -----
    mapper : function, dict, or Series
        Mapping correspondence.
    na_action : {None, 'ignore'}, default None
        If 'ignore', propagate NA values, without passing them to the
        mapping correspondence. If 'ignore' is not supported, a
        ``NotImplementedError`` should be raised.

```

Returns

Union[ndarray, Index, ExtensionArray]

The output of the mapping function applied to the array.

If the function returns a tuple with more than one element a MultiIndex will be returned.

max(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

min(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

```
prod(
    self,
    *,
    skipna: bool = True,
    min_count: int = 0,
    axis: AxisInt | None = 0,
    **kwargs
)
```

ravel(self, *args, **kwargs) -> Self
Return a flattened view on this array.

Parameters

order : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

ExtensionArray

A flattened view on the array.

See Also

ExtensionArray.tolist: Return a list of the values.

Notes

- Because ExtensionArrays are 1D-only, this is a no-op.

- The "order" argument is ignored, is for compatibility with NumPy.

Examples

```
>>> pd.array([1, 2, 3]).ravel()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

reshape(self, *args, **kwargs) -> Self

round(self, decimals: int = 0, *args, **kwargs)
Round each value in the array a to the given number of decimals.

Parameters

decimals : int, default 0

Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

*args, **kwargs

Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

NumericArray

Rounded values of the NumericArray.

See Also

numpy.around : Round values of an np.array.

DataFrame.round : Round values of a DataFrame.

Series.round : Round values of a Series.

searchsorted(

```

self,
value: NumpyValueArrayLike | ExtensionArray,
side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the
corresponding elements in `value` were inserted before the indices,
the order of `self` would be preserved.

Assuming that `self` is sorted:

=====
`side` returned index `i` satisfies
=====
left  ``self[i-1] < value <= self[i]``
right ``self[i-1] <= value < self[i]``
=====

Parameters
-----
value : array-like, list or scalar
    Value(s) to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort array a into ascending
    order. They are typically the result of argsort.

Returns
-----
array of ints or int
    If value is array-like, array of insertion points.
    If value is scalar, a single integer.

See Also
-----
numpy.searchsorted : Similar method from NumPy.

Examples
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na_value``.

Parameters
-----
periods : int, default 1
    The number of periods to shift. Negative values are allowed
    for shifting backwards.

fill_value : object, optional
    The scalar value to use for newly introduced missing values.
    The default is ``self.dtype.na_value``.

Returns
-----
ExtensionArray
    Shifted.

See Also
-----
api.extensions.ExtensionArray.transpose : Return a transposed view on
    this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
    enumerated type.

Notes

```

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)``, then an array of size `len(self)` is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

```
std(  
    self,  
    *,  
    skipna: bool = True,  
    axis: AxisInt | None = 0,  
    ddof: int = 1,  
    **kwargs  
)  
  
sum(  
    self,  
    *,  
    skipna: bool = True,  
    min_count: int = 0,  
    axis: AxisInt | None = 0,  
    **kwargs  
)  
  
swapaxes(self, axis1, axis2) -> Self  
  
take(  
    self,  
    indexer,  
    *,  
    allow_fill: bool = False,  
    fill_value: Scalar | None = None,  
    axis: AxisInt = 0  
) -> Self  
    Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
 Indices to be taken.
allow_fill : bool, default False
 How to handle negative values in ``indices``.

* False: negative values in ``indices`` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in ``indices`` indicate missing values. These values are set to ``fill\_value``. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
 Fill value to use for NA-indices when ``allow\_fill`` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of ``fill\_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill\_value`` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray

An array formed with selected `indices`.

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When `indices` contains negative values other than ``-1`` and `allow\_fill` is True.

See Also

numpy.take : Take elements from an array along an axis.

api.extensions.take : Take elements from an array.

Notes

ExtensionArray.take is called by ``Series.\_\_getitem``, ``.loc``, ``.iloc``, when `indices` is a sequence of values. Additionally, it's called by :meth:`Series.reindex`, or any other method that causes realignment, with a `fill\_value`.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method :func:`pandas.api.extensions.take`.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray
Convert to a NumPy Array.
```

By default converts to an object-dtype NumPy array. Specify the `dtype` and `na\_value` keywords to customize the conversion.

Parameters

dtype : dtype, default object

The numpy dtype to convert to.

copy : bool, default False

Whether to ensure that the returned value is a not a view on the array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary. This is typically only possible when no missing values are present and `dtype` is the equivalent numpy dtype.

na_value : scalar, optional

Scalar missing value indicator to use in numpy array. Defaults to the native missing value indicator of this array (pd.NA).

Returns

numpy.ndarray

Examples

An object-dtype is the default result

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a.to_numpy()
array([True, False, <NA>], dtype=object)
```

When no missing values are present, an equivalent dtype can be used.

```
>>> pd.array([True, False], dtype="boolean").to_numpy(dtype="bool")
array([ True, False])
>>> pd.array([1, 2], dtype="Int64").to_numpy("int64")
array([1, 2])
```

However, requesting such dtype will raise a ValueError if missing values are present and the default missing value :attr:`NA` is used.

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a
<BooleanArray>
[True, False, <NA>]
Length: 3, dtype: boolean
```

```
>>> a.to_numpy(dtype="bool")
Traceback (most recent call last):
...
```

ValueError: cannot convert to bool numpy array in presence of missing values

Specify a valid `na\_value` instead

```
>>> a.to_numpy(dtype="bool", na_value=False)
array([ True, False, False])
```

tolist(self) -> list

Return a list of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list

Python list of values in array.

See Also

Index.to_list: Return a list of the values in the Index.
Series.to_list: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

unique(self) -> Self

Compute the BaseMaskedArray of unique values.

Returns

uniques : BaseMaskedArray

value_counts(self, dropna: bool = True) -> Series

Returns a Series containing counts of each unique value.

Parameters

dropna : bool, default True
Don't include counts of missing values.

Returns

counts : Series

See Also

Series.value_counts

```
var(
    self,
    *,
    skipna: bool = True,
    axis: AxisInt | None = 0,
    ddof: int = 1,
    **kwargs
)
```

Readonly properties inherited from pandas.core.arrays.masked.BaseMaskedArray:

T

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> pd.array([1, 2, 3]).nbytes
27
```

ndim

Extension Arrays are only allowed to be 1-dimensional.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.ndim
1
```

shape

Return a tuple of the array dimensions.

See Also

numpy.ndarray.shape : Similar attribute which returns the shape of an array.
DataFrame.shape : Return a tuple representing the dimensionality of the
DataFrame.
Series.shape : Return a tuple representing the dimensionality of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shape
(3,)
```

Data and other attributes inherited from pandas.core.arrays.masked.BaseMaskedArray:

__annotations__ = {}

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ``other`` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
```

	height	weight
elk	3.0	500.5
moose	4.1	800.5

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
```

	elk	height	moose	weight
elk	NaN	NaN	NaN	NaN
moose	NaN	NaN	NaN	NaN

```
>>> df[["height", "weight"]].add(s2, axis="index")
```

	height	weight
elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
```

```

... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk       1.7     NaN
moose     3.0     NaN

```

```
__and__(self, other)
```

```
__divmod__(self, other)
```

```
__eq__(self, other)
Return self==value.
```

```
__floordiv__(self, other)
```

```
__ge__(self, other)
Return self>=value.
```

```
__gt__(self, other)
Return self>value.
```

```
__le__(self, other)
Return self<=value.
```

```
__lt__(self, other)
Return self<value.
```

```
__mod__(self, other)
```

```
__mul__(self, other)
```

```
__ne__(self, other)
Return self!=value.
```

```
__or__(self, other)
Return self|value.
```

```
__pow__(self, other)
```

```
__radd__(self, other)
```

```
__rand__(self, other)
```

```
__rdivmod__(self, other)
```

```
__rfloordiv__(self, other)
```

```
__rmod__(self, other)
```

```
__rmul__(self, other)
```

```
__ror__(self, other)
Return value|self.
```

```
__rpow__(self, other)
```

```
__rsub__(self, other)
```

```
__rtruediv__(self, other)
```

```
__rxor__(self, other)
```

```
__sub__(self, other)
```

```
__truediv__(self, other)
```

```
__xor__(self, other)
```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

```
__dict__
dictionary for instance variables
```

```
__weakref__
list of weak references to the object
```

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None

Methods inherited from pandas.api.extensions.ExtensionArray:

__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
Return repr(self).

argmax(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmin : Return the index of the minimum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

argmin(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

argsort(
 self,
 *,
 ascending: bool = True,
 kind: SortKind = 'quicksort',
 na_position: str = 'last',
 **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Return the indices that would sort this array.

Parameters

ascending : bool, default True

Whether the indices should result in an ascending
or descending sort.

kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.

na_position : {'first', 'last'}, default 'last'
If ``'first'``, put ``NaN`` values at the beginning.
If ``'last'``, put ``NaN`` values at the end.
**kwargs
Passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]
Array of indices that sort ``self``. If NaN values are contained,
NaN values are placed at the end.

See Also

numpy.argsort : Sorting implementation used internally.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self
An ExtensionArray of the same type as the original but with all
NA values removed.

See Also

Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in
an ExtensionArray.

Examples

>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64

insert(self, loc: int, item) -> Self from pandas.core.arrays.base.ExtensionArray
Insert an item at the given position.

Parameters

loc : int
Index where the `item` needs to be inserted.
item : scalar-like
Value to be inserted.

Returns

ExtensionArray
With `item` inserted at `loc`.

See Also

Index.insert: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item
cannot be held in an array of this type/dtype, either ValueError or
TypeError should be raised.

The default implementation relies on _from_sequence to raise on invalid
items.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)

```

<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
    Return the array element at the specified position as a Python scalar.

Parameters
-----
index : int, optional
    Position of the element. If not provided, the array must contain
    exactly one element.

Returns
-----
scalar
    The element at the specified position.

Raises
-----
ValueError
    If no index is provided and the array does not have exactly
    one element.
IndexError
    If the specified position is out of bounds.

See Also
-----
numpy.ndarray.item : Return the item of an array as a scalar.

Examples
-----
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)

repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self from pandas.
    Repeat elements of an ExtensionArray.

Returns a new ExtensionArray where each element of the current ExtensionArray
is repeated consecutively a given number of times.

Parameters
-----
repeats : int or array of ints
    The number of repetitions for each element. This should be a
    non-negative integer. Repeating 0 times will return an empty
    ExtensionArray.
axis : None
    Must be ``None``. Has no effect but is accepted for compatibility
    with numpy.

Returns
-----
ExtensionArray
    Newly created ExtensionArray with repeated elements.

See Also
-----
Series.repeat : Equivalent function for Series.
Index.repeat : Equivalent function for Index.
numpy.repeat : Similar method for :class:`numpy.ndarray`.
ExtensionArray.take : Take arbitrary positions.

Examples
-----
>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)

```

```
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
```

`transpose(self, *axes: int) -> Self` from `pandas.core.arrays.base.ExtensionArray`
Return a transposed view on this array.

Because `ExtensionArrays` are always 1D, this is a no-op. It is included for compatibility with `np.ndarray`.

Returns

`ExtensionArray`

Examples

```
>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

`view(self, dtype: Dtype | None = None) -> ArrayLike` from `pandas.core.arrays.base.ExtensionArray`
Return a view on the array.

Parameters

`dtype` : str, np.dtype, or `ExtensionDtype`, optional
Default None.

Returns

`ExtensionArray` or `np.ndarray`
A view on the `:class:`ExtensionArray``'s data.

See Also

`api.extensions.ExtensionArray.ravel`: Return a flattened view on input array.
`Index.view`: Equivalent function for `Index`.
`ndarray.view`: New view of array with the same data.

Examples

This gives view on the underlying data of an ```ExtensionArray``` and is not a copy. Modifications on either the view or the original ```ExtensionArray``` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from `pandas.api.extensions.ExtensionArray`:

`size`

The number of elements in the array.

Data and other attributes inherited from `pandas.api.extensions.ExtensionArray`:

`__pandas_priority__` = 1000

`class Categorical(pandas.core.arrays._mixins.NDArrayBackedExtensionArray, pandas.core.base.PandasObject)`

```
    Categorical(
        values,
        categories=None,
        ordered=None,
        dtype: Dtype | None = None,
        copy: bool = True
    ) -> None
```

Represent a categorical variable in classic R / S-plus fashion.

`Categoricals` can only take on a limited, and usually fixed, number of possible values (`categories`). In contrast to statistical categorical variables, a `Categorical` might have an order, but numerical operations (additions, divisions, ...) are not possible.

All values of the `Categorical` are either in `categories` or `np.nan`. Assigning values outside of `categories` will raise a `ValueError`. Order is defined by the order of the `categories`, not lexical order of the values.

Parameters

values : list-like
The values of the categorical. If categories are given, values not in categories will be replaced with NaN.
categories : Index-like (unique), optional
The unique categories for this categorical. If not given, the categories are assumed to be the unique values of `values` (sorted, if possible, otherwise in the order in which they appear).
ordered : bool, default False
Whether or not this categorical is treated as an ordered categorical. If True, the resulting categorical will be ordered.
An ordered categorical respects, when sorted, the order of its `categories` attribute (which in turn is the `categories` argument, if provided).
dtype : CategoricalDtype
An instance of ``CategoricalDtype`` to use for this categorical.
copy : bool, default True
Whether to copy if the codes are unchanged.

Attributes

categories : Index
The categories of this categorical.
codes : ndarray
The codes (integer positions, which point to the categories) of this categorical, read only.
ordered : bool
Whether or not this Categorical is ordered.
dtype : CategoricalDtype
The instance of ``CategoricalDtype`` storing the ``categories`` and ``ordered``.

Methods

from_codes
as_ordered
as_unordered
set_categories
rename_categories
reorder_categories
add_categories
remove_categories
remove_unused_categories
map
__array__

Raises

ValueError
If the categories do not validate.
TypeError
If an explicit ``ordered=True`` is given but no `categories` and the `values` are not sortable.

See Also

CategoricalDtype : Type for categorical data.
CategoricalIndex : An Index with an underlying ``Categorical``.

Notes

See the `user guide`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/categorical.html>`\_\_`
for more.

Examples

```
-----
>>> pd.Categorical([1, 2, 3, 1, 2, 3])
[1, 2, 3, 1, 2, 3]
Categories (3, int64): [1, 2, 3]
```

```
>>> pd.Categorical(["a", "b", "c", "a", "b", "c"])
['a', 'b', 'c', 'a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
```

Missing values are not included as a category.

```
>>> c = pd.Categorical([1, 2, 3, 1, 2, 3, np.nan])
>>> c
[1, 2, 3, 1, 2, 3, NaN]
Categories (3, int64): [1, 2, 3]
```

However, their presence is indicated in the `codes` attribute by code `-1`.

```
>>> c.codes
array([ 0,  1,  2,  0,  1,  2, -1], dtype=int8)
```

Ordered `Categoricals` can be sorted according to the custom order of the categories and can have a min and max value.

```
>>> c = pd.Categorical(
...     ["a", "b", "c", "a", "b", "c"], ordered=True, categories=["c", "b", "a"]
... )
>>> c
['a', 'b', 'c', 'a', 'b', 'c']
Categories (3, str): ['c' < 'b' < 'a']
>>> c.min()
'c'
```

Method resolution order:

```
Categorical
pandas.core.arrays._mixins.NDArrayBackedExtensionArray
pandas._libs.arrays.NDArrayBacked
pandas.api.extensions.ExtensionArray
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
pandas.core.strings.object_array.ObjectStringArrayMixin
builtins.object
```

Methods defined here:

```
__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas
The numpy array interface.
```

Users should not call this directly. Rather, it is invoked by `:func:`numpy.array`` and `:func:`numpy.asarray``.

Parameters

`dtype` : np.dtype or None
Specifies the dtype for the array.

`copy` : bool or None, optional
See `:func:`numpy.asarray``.

Returns

```
-----
numpy.array
A numpy array of either the specified dtype or,
if dtype==None (default), the same dtype as
categorical.categories.dtype.
```

See Also

`numpy.asarray` : Convert input to numpy.ndarray.

Examples

```
-----
>>> cat = pd.Categorical(["a", "b"], ordered=True)
```

```

The following calls ``cat.__array__``

>>> np.asarray(cat)
array(['a', 'b'], dtype=object)

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.array
__contains__(self, key) -> bool from pandas.core.arrays.categorical.Categorical
Returns True if `key` is in this Categorical.

__eq__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__ge__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__gt__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>

__init__(
    self,
    values,
    categories=None,
    ordered=None,
    dtype: Dtype | None = None,
    copy: bool = True
) -> None from pandas.core.arrays.categorical.Categorical
Initialize self. See help(type(self)) for accurate signature.

__iter__(self) -> Iterator from pandas.core.arrays.categorical.Categorical
Returns an Iterator over the values of this Categorical.

__le__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__lt__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>
__ne__(self, other) from pandas.core.arrays.categorical._cat_compare_op.<locals>

__repr__(self) -> str from pandas.core.arrays.categorical.Categorical
String representation.

__setstate__(self, state) -> None from pandas.core.arrays.categorical.Categorical
Necessary for making this object picklable

add_categories(self, new_categories) -> Self from pandas.core.arrays.categorical.Categorical
Add new categories.

`new_categories` will be included at the last/highest place in the
categories and will be unused directly after this call.

Parameters
-----
new_categories : category or list-like of category
    The new categories to be included.

Returns
-----
Categorical
    Categorical with new categories added.

Raises
-----
ValueError
    If the new categories include old categories or do not validate as
    categories

See Also
-----
rename_categories : Rename categories.
reorder_categories : Reorder categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples
-----
>>> c = pd.Categorical(["c", "b", "c"])
>>> c
['c', 'b', 'c']
Categories (2, str): ['b', 'c']

```

```

>>> c.add_categories(["d", "a"])
['c', 'b', 'c']
Categories (4, str): ['b', 'c', 'd', 'a']

argsort(self, *, ascending: bool = True, kind: SortKind = 'quicksort', **kwargs) -> npt.NDArray
Return the indices that would sort the Categorical.

Missing values are sorted at the end.

Parameters
-----
ascending : bool, default True
    Whether the indices should result in an ascending
    or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
    Sorting algorithm.
**kwargs:
    passed through to :func:`numpy.argsort`.

Returns
-----
np.ndarray[np.intp]

See Also
-----
numpy.ndarray.argsort

Notes
-----
While an ordering is applied to the category values, arg-sorting
in this context refers more to organizing and grouping together
based on matching category values. Thus, this function can be
called on an unordered Categorical instance unlike the functions
'Categorical.min' and 'Categorical.max'.

Examples
-----
>>> pd.Categorical(["b", "b", "a", "c"]).argsort()
array([2, 0, 1, 3])

>>> cat = pd.Categorical(
...     ["b", "b", "a", "c"], categories=["c", "b", "a"], ordered=True
... )
>>> cat.argsort()
array([3, 0, 1, 2])

Missing values are placed at the end

>>> cat = pd.Categorical([2, None, 1])
>>> cat.argsort()
array([2, 0, 1])

as_ordered(self) -> Self from pandas.core.arrays.categorical.Categorical
Set the Categorical to be ordered.

Returns
-----
Categorical
    Ordered Categorical.

See Also
-----
as_unordered : Set the Categorical to be unordered.

Examples
-----
For :class:`pandas.Series`:

>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False
>>> ser = ser.cat.as_ordered()
>>> ser.cat.ordered
True

For :class:`pandas.CategoricalIndex`:

```

```

>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci.ordered
False
>>> ci = ci.as_ordered()
>>> ci.ordered
True

as_unordered(self) -> Self from pandas.core.arrays.categorical.Categorical
Set the Categorical to be unordered.

Returns
-----
Categorical
    Unordered Categorical.

See Also
-----
as_ordered : Set the Categorical to be ordered.

Examples
-----
For :class:`pandas.Series`:

>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
True
>>> ser = ser.cat.as_unordered()
>>> ser.cat.ordered
False

For :class:`pandas.CategoricalIndex`:

>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"], ordered=True)
>>> ci.ordered
True
>>> ci = ci.as_unordered()
>>> ci.ordered
False

astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike from pandas.core.arrays.categorical.Categorical
Coerce this type to another dtype

Parameters
-----
dtype : numpy dtype or pandas type
copy : bool, default True
    By default, astype always returns a newly allocated object.
    If copy is set to False and dtype is categorical, the original
    object is returned.

check_for_ordered(self, op) -> None from pandas.core.arrays.categorical.Categorical
assert that we are ordered

describe(self) -> DataFrame from pandas.core.arrays.categorical.Categorical
Describes this Categorical

Returns
-----
description: `DataFrame`
    A dataframe with frequency and counts by category.

equals(self, other: object) -> bool from pandas.core.arrays.categorical.Categorical
Returns True if categorical arrays are equal.

Parameters
-----
other : `Categorical`

Returns
-----
bool

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_] from pandas.core.arrays.categorical.Categorical
Check whether `values` are contained in Categorical.

```

Return a boolean NumPy Array showing whether each element in the Categorical matches an element in the passed sequence of `values` exactly.

Parameters

`values` : np.ndarray or ExtensionArray
The sequence of values to test. Passing in a single string will raise a ``TypeError``. Instead, turn a single string into a list of one element.

Returns

np.ndarray[bool]

Raises

`TypeError`
* If `values` is not a set or list-like

See Also

`pandas.Series.isin` : Equivalent method on Series.

Examples

```
>>> s = pd.Categorical(["llama", "cow", "llama", "beetle", "llama", "hippo"])
>>> s.isin(["cow", "llama"])
array([ True,  True,  True, False,  True, False])
```

Passing a single string as ``s.isin('llama')`` will raise an error. Use a list of one element instead:

```
>>> s.isin(["llama"])
array([ True, False,  True, False,  True, False])
```

`isna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.categorical.Categorical`
Detect missing values

Missing values (-1 in .codes) are detected.

Returns

np.ndarray[bool] of whether my values are null

See Also

`isna` : Top-level isna.
`isnull` : Alias of isna.
`Categorical.notna` : Boolean inverse of Categorical.isna.

`isnull = isna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.categorical.Categorical`

`map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.arrays.categorical.Categorical`
Map categories using an input mapping or function.

Maps the categories to new categories. If the mapping correspondence is one-to-one the result is a :class:`~pandas.Categorical` which has the same order property as the original, otherwise a :class:`~pandas.Index` is returned. NaN values are unaffected.

If a `dict` or :class:`~pandas.Series` is used any unmapped category is mapped to `NaN`. Note that if this happens an :class:`~pandas.Index` will be returned.

Parameters

`mapper` : function, dict, or Series
Mapping correspondence.
`na_action` : {None, 'ignore'}, default None
If 'ignore', propagate NaN values, without passing them to the mapping correspondence.

Returns

pandas.Categorical or pandas.Index
Mapped categorical.

See Also

CategoricalIndex.map : Apply a mapping correspondence on a
:class:`~pandas.CategoricalIndex`.
Index.map : Apply a mapping correspondence on an
:class:`~pandas.Index`.
Series.map : Apply a mapping correspondence on a
:class:`~pandas.Series`.
Series.apply : Apply more complex functions on a
:class:`~pandas.Series`.

Examples

>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.map(lambda x: x.upper(), na_action=None)
['A', 'B', 'C']
Categories (3, str): ['A', 'B', 'C']
>>> cat.map({"a": "first", "b": "second", "c": "third"}, na_action=None)
['first', 'second', 'third']
Categories (3, str): ['first', 'second', 'third']

If the mapping is one-to-one the ordering of the categories is preserved:

>>> cat = pd.Categorical(["a", "b", "c"], ordered=True)
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a' < 'b' < 'c']
>>> cat.map({"a": 3, "b": 2, "c": 1}, na_action=None)
[3, 2, 1]
Categories (3, int64): [3 < 2 < 1]

If the mapping is not one-to-one an :class:`~pandas.Index` is returned:

>>> cat.map({"a": "first", "b": "second", "c": "first"}, na_action=None)
Index(['first', 'second', 'first'], dtype='str')

If a `dict` is used, all unmapped categories are mapped to `NaN` and the result is an :class:`~pandas.Index`:

>>> cat.map({"a": "first", "b": "second"}, na_action=None)
Index(['first', 'second', nan], dtype='str')

The mapping function is applied to categories, not to each value. It is therefore only called once per unique category, and the result reused for all occurrences:

>>> cat = pd.Categorical(["a", "a", "b"])
>>> calls = []
>>> def f(x):
... calls.append(x)
... return x.upper()
>>> result = cat.map(f)
>>> result
['A', 'A', 'B']
Categories (2, str): ['A', 'B']
>>> calls
['a', 'b']

max(self, *, skipna: bool = True, **kwargs) from pandas.core.arrays.categorical.Categorical
The maximum value of the object.

Only ordered `Categoricals` have a maximum!

Raises

TypeError

If the `Categorical` is not `ordered`.

Returns

max : the maximum of this `Categorical`, NA if array is empty

```

memory_usage(self, deep: bool = False) -> int from pandas.core.arrays.categorical.Categorical
    Memory usage of my values

    Parameters
    -----
    deep : bool
        Introspect the data deeply, interrogate
        `object` dtypes for system-level memory consumption

    Returns
    -----
    bytes used

    Notes
    -----
    Memory usage does not include memory consumed by elements that
    are not components of the array if deep=False

    See Also
    -----
    numpy.ndarray.nbytes

min(self, *, skipna: bool = True, **kwargs) from pandas.core.arrays.categorical.Categorical
    The minimum value of the object.

    Only ordered `Categoricals` have a minimum!

    Raises
    -----
    TypeError
        If the `Categorical` is not `ordered`.

    Returns
    -----
    min : the minimum of this `Categorical`, NA value if empty

notna(self) -> npt.NDArray[np.bool_] from pandas.core.arrays.categorical.Categorical
    Inverse of isna

    Both missing values (-1 in .codes) and NA as a category are detected as
    null.

    Returns
    -----
    np.ndarray[bool] of whether my values are not null

    See Also
    -----
    notna : Top-level notna.
    notnull : Alias of notna.
    Categorical.isna : Boolean inverse of Categorical.notna.

notnull = notna(self) -> npt.NDArray[np.bool_]

remove_categories(self, removals) -> Self from pandas.core.arrays.categorical.Categorical
    Remove the specified categories.

    The ``removals`` argument must be a subset of the current categories.
    Any values that were part of the removed categories will be set to NaN.

    Parameters
    -----
    removals : category or list of categories
        The categories which should be removed.

    Returns
    -----
    Categorical
        Categorical with removed categories.

    Raises
    -----
    ValueError
        If the removals are not contained in the categories

    See Also
    -----

```

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["a", "c", "b", "c", "d"])
>>> c
['a', 'c', 'b', 'c', 'd']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c.remove_categories(["d", "a"])
[NaN, 'c', 'b', 'c', NaN]
Categories (2, str): ['b', 'c']
```

remove_unused_categories(self) -> Self from pandas.core.arrays.categorical.Categorical
Remove categories which are not used.

This method is useful when working with datasets that undergo dynamic changes where categories may no longer be relevant, allowing to maintain a clean, efficient data structure.

Returns

Categorical
Categorical with unused categories dropped.

See Also

rename_categories : Rename categories.
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
set_categories : Set the categories to the specified ones.

Examples

```
>>> c = pd.Categorical(["a", "c", "b", "c", "d"])
>>> c
['a', 'c', 'b', 'c', 'd']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c[2] = "a"
>>> c[4] = "c"
>>> c
['a', 'c', 'a', 'c', 'c']
Categories (4, str): ['a', 'b', 'c', 'd']

>>> c.remove_unused_categories()
['a', 'c', 'a', 'c', 'c']
Categories (2, str): ['a', 'c']
```

rename_categories(self, new_categories) -> Self from pandas.core.arrays.categorical.Categorical
Rename categories.

This method is commonly used to re-label or adjust the category names in categorical data without changing the underlying data. It is useful in situations where you want to modify the labels used for clarity, consistency, or readability.

Parameters

new_categories : list-like, dict-like or callable

New categories which will replace old categories.

* list-like: all items must be unique and the number of items in the new categories must match the existing number of categories.

* dict-like: specifies a mapping from old categories to new. Categories not contained in the mapping are passed through and extra categories in the mapping are ignored.


```

        * callable : a callable that is called on all items in the old
                      categories and whose return values comprise the new categories.

Returns
-----
Categorical
    Categorical with renamed categories.

Raises
-----
ValueError
    If new categories are list-like and do not have the same number of
    items than the current categories or do not validate as categories

See Also
-----
reorder_categories : Reorder categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples
-----
>>> c = pd.Categorical(["a", "a", "b"])
>>> c.rename_categories([0, 1])
[0, 0, 1]
Categories (2, int64): [0, 1]

For dict-like ``new_categories``, extra keys are ignored and
categories not in the dictionary are passed through

>>> c.rename_categories({"a": "A", "c": "C"})
['A', 'A', 'b']
Categories (2, str): ['A', 'b']

You may also provide a callable to create the new categories

>>> c.rename_categories(lambda x: x.upper())
['A', 'A', 'B']
Categories (2, str): ['A', 'B']

reorder_categories(self, new_categories, ordered=None) -> Self from pandas.core.arrays.categorical
Reorder categories as specified in new_categories.

``new_categories`` need to include all old categories and no new category
items.

Parameters
-----
new_categories : Index-like
    The categories in new order.
ordered : bool, optional
    Whether or not the categorical is treated as an ordered categorical.
    If not given, do not change the ordered information.

Returns
-----
Categorical
    Categorical with reordered categories.

Raises
-----
ValueError
    If the new categories do not contain all old category items or any
    new ones

See Also
-----
rename_categories : Rename categories.
add_categories : Add new categories.
remove_categories : Remove the specified categories.
remove_unused_categories : Remove categories which are not used.
set_categories : Set the categories to the specified ones.

Examples
-----

```

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser = ser.cat.reorder_categories(["c", "b", "a"], ordered=True)
>>> ser
0    a
1    b
2    c
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']

>>> ser.sort_values()
2    c
1    b
0    a
3    a
dtype: category
Categories (3, str): ['c' < 'b' < 'a']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a"])
>>> ci
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['a', 'b', 'c'],
                 ordered=False, dtype='category')
>>> ci.reorder_categories(["c", "b", "a"], ordered=True)
CategoricalIndex(['a', 'b', 'c', 'a'], categories=['c', 'b', 'a'],
                 ordered=True, dtype='category')
```

set_categories(self, new_categories, ordered=None, rename: bool = False) -> Self from pandas
Set the categories to the specified new categories.

`new_categories` can include new categories (which will result in unused categories) or remove old categories (which results in values set to `NaN`). If `rename=True`, the categories will simply be renamed (less or more items than in old categories will result in values set to `NaN` or in unused categories respectively).

This method can be used to perform more than one action of adding, removing, and reordering simultaneously and is therefore faster than performing the individual steps via the more specialised methods.

On the other hand this methods does not do checks (e.g., whether the old categories are included in the new categories on a reorder), which can result in surprising changes, for example when using special string dtypes, which do not consider a \$1 string equal to a single char python string.

Parameters

`new_categories` : Index-like
The categories in new order.
`ordered` : bool, default None
Whether or not the categorical is treated as an ordered categorical.
If not given, do not change the ordered information.
`rename` : bool, default False
Whether or not the new_categories should be considered as a rename of the old categories or as reordered categories.

Returns

Categorical
New categories to be used, with optional ordering changes.

Raises

ValueError
If new_categories does not validate as categories

See Also

`rename_categories` : Rename categories.
`reorder_categories` : Reorder categories.
`add_categories` : Add new categories.
`remove_categories` : Remove the specified categories.
`remove_unused_categories` : Remove categories which are not used.

Examples

For :class:`pandas.Series`:

```
>>> raw_cat = pd.Categorical(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ser = pd.Series(raw_cat)
>>> ser
0    a
1    b
2    c
3   NaN
dtype: category
Categories (3, str): ['a' < 'b' < 'c']
```

```
>>> ser.cat.set_categories(["A", "B", "C"], rename=True)
0    A
1    B
2    C
3   NaN
dtype: category
Categories (3, str): ['A' < 'B' < 'C']
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(
...     ["a", "b", "c", None], categories=["a", "b", "c"], ordered=True
... )
>>> ci
CategoricalIndex(['a', 'b', 'c', nan], categories=['a', 'b', 'c'],
                  ordered=True, dtype='category')

>>> ci.set_categories(["A", "B", "C"])
CategoricalIndex([nan, 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
>>> ci.set_categories(["A", "b", "c"], rename=True)
CategoricalIndex(['A', 'b', 'c', nan], categories=['A', 'b', 'c'],
                  ordered=True, dtype='category')
```

set_ordered(self, value: bool) -> Self from pandas.core.arrays.categorical.Categorical
Set the ordered attribute to the boolean value.

Parameters

value : bool

Set whether this categorical is ordered (True) or not (False).

```
sort_values(
    self,
    *,
    inplace: bool = False,
    ascending: bool = True,
    na_position: str = 'last'
) -> Self | None from pandas.core.arrays.categorical.Categorical
Sort the Categorical by category value returning a new
Categorical by default.
```

While an ordering is applied to the category values, sorting in this context refers more to organizing and grouping together based on matching category values. Thus, this function can be called on an unordered Categorical instance unlike the functions 'Categorical.min' and 'Categorical.max'.

Parameters

inplace : bool, default False

Do operation in place.

ascending : bool, default True

Order ascending. Passing False orders descending. The ordering parameter provides the method by which the category values are organized.

na_position : {'first', 'last'} (optional, default='last')

'first' puts NaNs at the beginning

'last' puts NaNs at the end

Returns

Categorical or None

See Also

Categorical.sort
Series.sort_values

Examples

```
>>> c = pd.Categorical([1, 2, 2, 1, 5])
>>> c
[1, 2, 2, 1, 5]
Categories (3, int64): [1, 2, 5]
>>> c.sort_values()
[1, 1, 2, 2, 5]
Categories (3, int64): [1, 2, 5]
>>> c.sort_values(ascending=False)
[5, 2, 2, 1, 1]
Categories (3, int64): [1, 2, 5]

>>> c = pd.Categorical([1, 2, 2, 1, 5])

'sort_values' behaviour with NaNs. Note that 'na_position'
is independent of the 'ascending' parameter:

>>> c = pd.Categorical([np.nan, 2, 2, np.nan, 5])
>>> c
[NaN, 2, 2, NaN, 5]
Categories (2, int64): [2, 5]
>>> c.sort_values()
[2, 2, 5, NaN, NaN]
Categories (2, int64): [2, 5]
>>> c.sort_values(ascending=False)
[5, 2, 2, NaN, NaN]
Categories (2, int64): [2, 5]
>>> c.sort_values(na_position="first")
[NaN, NaN, 2, 2, 5]
Categories (2, int64): [2, 5]
>>> c.sort_values(ascending=False, na_position="first")
[NaN, NaN, 5, 2, 2]
Categories (2, int64): [2, 5]
```

unique(self) -> Self from pandas.core.arrays.categorical.Categorical
Return the ``Categorical`` which ``categories`` and ``codes`` are
unique.

Returns

Categorical

See Also

pandas.unique
CategoricalIndex.unique
Series.unique : Return unique values of Series object.

Examples

```
>>> pd.Categorical(list("baabc")).unique()
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> pd.Categorical(list("baab"), categories=list("abc"), ordered=True).unique()
['b', 'a']
Categories (3, str): ['a' < 'b' < 'c']
```

value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.categorical.Categorical
Return a Series containing counts of each category.

Every category will have an entry, even those with a count of 0.

Parameters

dropna : bool, default True
Don't include counts of NaN.

Returns

counts : Series

See Also

Series.value_counts

Class methods defined here:

```
from_codes(  
    codes,  
    categories=None,  
    ordered=None,  
    dtype: Dtype | None = None,  
    validate: bool = True  
) -> Self from pandas.core.arrays.categorical.Categorical  
    Make a Categorical type from codes and categories or dtype.  
  
    This constructor is useful if you already have codes and  
    categories/dtype and so do not need the (computation intensive)  
    factorization step, which is usually done on the constructor.  
  
    If your data does not follow this convention, please use the normal  
    constructor.  
  
Parameters  
-----  
codes : array-like of int  
    An integer array, where each integer points to a category in  
    categories or dtype.categories, or else is -1 for NaN.  
categories : index-like, optional  
    The categories for the categorical. Items need to be unique.  
    If the categories are not given here, then they must be provided  
    in `dtype`.  
ordered : bool, optional  
    Whether or not this categorical is treated as an ordered  
    categorical. If not given here or in `dtype`, the resulting  
    categorical will be unordered.  
dtype : CategoricalDtype or "category", optional  
    If :class:`CategoricalDtype`, cannot be used together with  
    `categories` or `ordered`.  
validate : bool, default True  
    If True, validate that the codes are valid for the dtype.  
    If False, don't validate that the codes are valid. Be careful about skipping  
    validation, as invalid codes can lead to severe problems, such as segfaults.  
  
.. versionadded:: 2.1.0
```

Returns

Categorical

See Also

codes : The category codes of the categorical.
CategoricalIndex : An Index with an underlying ``Categorical``.

Examples

>>> dtype = pd.CategoricalDtype(["a", "b"], ordered=True)
>>> pd.Categorical.from_codes(codes=[0, 1, 0, 1], dtype=dtype)
['a', 'b', 'a', 'b']
Categories (2, str): ['a' < 'b']

Readonly properties defined here:

categories
 The categories of this categorical.

Setting assigns new values to each category (effectively a rename of
each individual category).

The assigned value has to be a list-like object. All items must be
unique and the number of items in the new categories must be the same

as the number of items in the old categories.

Raises

ValueError

If the new categories do not validate as categories or if the number of new categories is unequal the number of old categories

See Also

rename_categories : Rename categories.

reorder_categories : Reorder categories.

add_categories : Add new categories.

remove_categories : Remove the specified categories.

remove_unused_categories : Remove categories which are not used.

set_categories : Set the categories to the specified ones.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
```

```
>>> ser.cat.categories
```

```
Index(['a', 'b', 'c'], dtype='str')
```

```
>>> raw_cat = pd.Categorical([None, "b", "c", None], categories=["b", "c", "d"])
```

```
>>> ser = pd.Series(raw_cat)
```

```
>>> ser.cat.categories
```

```
Index(['b', 'c', 'd'], dtype='str')
```

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
```

```
>>> cat.categories
```

```
Index(['a', 'b'], dtype='str')
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "c", "b", "a", "c", "b"])
```

```
>>> ci.categories
```

```
Index(['a', 'b', 'c'], dtype='str')
```

```
>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
```

```
>>> ci.categories
```

```
Index(['c', 'b', 'a'], dtype='str')
```

codes

The category codes of this categorical index.

Codes are an array of integers which are the positions of the actual values in the categories array.

There is no setter, use the other categorical methods and the normal item setter to change values in the categorical.

Returns

ndarray[int]

A non-writable view of the ``codes`` array.

See Also

Categorical.from_codes : Make a Categorical from codes.

CategoricalIndex : An Index with an underlying ``Categorical``.

Examples

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
```

```
>>> cat.codes
```

```
array([0, 1], dtype=int8)
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b", "c", "a", "b", "c"])
```

```
>>> ci.codes
```

```
array([0, 1, 2, 0, 1, 2], dtype=int8)

>>> ci = pd.CategoricalIndex(["a", "c"], categories=["c", "b", "a"])
>>> ci.codes
array([2, 0], dtype=int8)
```

dtype

The :class:`pandas.api.types.CategoricalDtype` for this instance.

See Also

astype : Cast argument to a specified dtype.
CategoricalDtype : Type for categorical data.

Examples

>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat
['a', 'b']
Categories (2, str): ['a' < 'b']
>>> cat.dtype
CategoricalDtype(categories=['a', 'b'], ordered=True, categories_dtype=str)

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

>>> pd.array([1, 2, 3]).nbytes
27

ordered

Whether the categories have an ordered relationship.

See Also

set_ordered : Set the ordered attribute.
as_ordered : Set the Categorical to be ordered.
as_unordered : Set the Categorical to be unordered.

Examples

For :class:`pandas.Series`:

```
>>> ser = pd.Series(["a", "b", "c", "a"], dtype="category")
>>> ser.cat.ordered
False

>>> raw_cat = pd.Categorical(["a", "b", "c", "a"], ordered=True)
>>> ser = pd.Series(raw_cat)
>>> ser.cat.ordered
True
```

For :class:`pandas.Categorical`:

```
>>> cat = pd.Categorical(["a", "b"], ordered=True)
>>> cat.ordered
True

>>> cat = pd.Categorical(["a", "b"], ordered=False)
>>> cat.ordered
False
```

For :class:`pandas.CategoricalIndex`:

```
>>> ci = pd.CategoricalIndex(["a", "b"], ordered=True)
>>> ci.ordered
True

>>> ci = pd.CategoricalIndex(["a", "b"], ordered=False)
>>> ci.ordered
False
```

Data and other attributes defined here:

`__annotations__ = {'_dtype': 'CategoricalDtype'}`

`__array_priority__ = 1000`

`__hash__ = None`

Methods inherited from `pandas.core.arrays._mixins.NDArrayBackedExtensionArray`:

`__getitem__(self, key: PositionalIndexer2D) -> Self | Any`
Select a subset of self.

Parameters

`item : int, slice, or ndarray`

* `int`: The position in 'self' to get.

* `slice`: A slice object, where 'start', 'stop', and 'step' are integers or None

* `ndarray`: A 1-d boolean NumPy ndarray the same length as 'self'

* `list[int]`: A list of int

Returns

`item : scalar or ExtensionArray`

Notes

For scalar ```item```, return a scalar value suitable for the array's type. This should be an instance of ```self.dtype.type```.

For slice ```key```, return an instance of ```ExtensionArray```, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ```ExtensionArray```, filtered to the values where ```item``` is True.

`__setitem__(self, key, value) -> None`
Set one or more values inplace.

This method is not required to satisfy the pandas extension array interface.

Parameters

`key : int, ndarray, or slice`

When called from, e.g. ```Series.__setitem__```, ```key``` will be one of

* scalar int
* ndarray of integers.
* boolean ndarray
* slice object

`value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object`
value or values to be set of ```key```.

Returns

`None`

Raises

`ValueError`

If the array is readonly and modification is attempted.

`argmax(self, axis: AxisInt = 0, skipna: bool = True)`
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the minimum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

argmin(self, axis: AxisInt = 0, skipna: bool = True)

Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)
```

fillna(self, value, limit: int | None = None, copy: bool = True) -> Self

Fill NA/NaN values using the specified method.

Parameters

value : scalar, array-like

If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.

limit : int, default None

The maximum number of entries where NA values will be filled.

copy : bool, default True

Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray

With NA/NaN filled.

See Also

api.extensions.ExtensionArray.dropna : Return ExtensionArray without NA values.

api.extensions.ExtensionArray.isna : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
```

```

[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

insert(self, loc: int, item) -> Self
    Make new ExtensionArray inserting new item at location. Follows
    Python list.append semantics for negative values.

Parameters
-----
loc : int
item : object

Returns
-----
type(self)

searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted array `self` (a) such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    Assuming that `self` is sorted:

    =====
    `side` returned index `i` satisfies
    =====
    left  ``self[i-1] < value <= self[i]``
    right ``self[i-1] <= value < self[i]``
    =====

Parameters
-----
value : array-like, list or scalar
    Value(s) to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort array a into ascending
    order. They are typically the result of argsort.

Returns
-----
array of ints or int
    If value is array-like, array of insertion points.
    If value is scalar, a single integer.

See Also
-----
numpy.searchsorted : Similar method from NumPy.

Examples
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
    Shift values by desired number.

    Newly introduced missing values are filled with
    ``self.dtype.na_value``.

Parameters
-----
periods : int, default 1
    The number of periods to shift. Negative values are allowed
    for shifting backwards.

```

`fill_value` : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

`api.extensions.ExtensionArray.transpose` : Return a transposed view on this array.
`api.extensions.ExtensionArray.factorize` : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`` , then an array of size `len(self)` is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along `axis=0`.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64
```

```
take(
    self,
    indices: TakeIndexer,
    *,
    allow_fill: bool = False,
    fill_value: Any = None,
    axis: AxisInt = 0
) -> Self
Take elements from an array.
```

Parameters

`indices` : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.
`allow_fill` : bool, default False
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices from the right (the default). This is similar to `:func:`numpy.take``.

* True: negative values in `indices` indicate missing values. These values are set to `fill_value`. Any other other negative values raise a `ValueError`.

`fill_value` : any, optional
Fill value to use for NA-indices when `allow_fill` is True.
This may be `None`, in which case the default NA value for the type, `self.dtype.na_value`, is used.

For many ExtensionArrays, there will be two representations of `fill_value`: a user-facing "boxed" scalar, and a low-level physical NA value. `fill_value` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray
An array formed with selected `indices`.

Raises

`IndexError`

When the indices are out of bounds for the array.

`ValueError`

When ``indices`` contains negative values other than ``-1`` and ``allow_fill`` is `True`.

See Also

`numpy.take` : Take elements from an array along an axis.

`api.extensions.take` : Take elements from an array.

Notes

`ExtensionArray.take` is called by ``Series.__getitem__``, ``loc``, ``iloc``, when ``indices`` is a sequence of values. Additionally, it's called by `:meth:`Series.reindex``, or any other method that causes realignment, with a ``fill_value``.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method `:func:`pandas.api.extensions.take``.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

`view(self, dtype: Dtype | None = None) -> ArrayLike`
Return a view on the array.

Parameters

`dtype` : `str`, `np.dtype`, or `ExtensionDtype`, optional
Default `None`.

Returns

`ExtensionArray` or `np.ndarray`

A view on the `:class:`ExtensionArray``'s data.

See Also

`api.extensions.ExtensionArray.ravel`: Return a flattened view on input array.

`Index.view`: Equivalent function for `Index`.

`ndarray.view`: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
```

Length: 3, dtype: Int64

Data descriptors inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

Methods inherited from pandas._libs.arrays.NDArrayBacked:

__len__(self, /)
Return len(self).

__reduce__ = __reduce_cython__(...)

__reduce_cython__(self)

__setstate_cython__(self, __pyx_state)

copy(self, order='C')

delete(self, loc, axis=0)

ravel(self, order='C')

repeat(self, repeats, axis: int | np.integer = 0)

reshape(self, *args, **kwargs)

swapaxes(self, axis1, axis2)

transpose(self, *axes)

Static methods inherited from pandas._libs.arrays.NDArrayBacked:

__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
Create and return a new object. See help(type) for accurate signature.

Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:

T

ndim

shape

size

Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:

__pyx_vtable__ = <capsule object NULL>

Methods inherited from pandas.api.extensions.ExtensionArray:

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all NA values removed.

See Also

Series.dropna : Remove missing values from a Series.

DataFrame.dropna : Remove missing values from a DataFrame.

api.extensions.ExtensionArray.isna : Check for missing values in an ExtensionArray.

Examples

```
-----
>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64
```

```

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]
Return boolean ndarray denoting duplicate values.
```

Parameters

```
-----
keep : {'first', 'last', False}, default 'first'
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.
```

Returns

```
-----
ndarray[bool]
With true in indices where elements are duplicated and false otherwise.
```

See Also

```
-----
DataFrame.duplicated : Return boolean Series denoting
duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray
of unique values.
```

Examples

```
-----
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])
```

```

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas
Encode the extension array as an enumerated type.
```

Parameters

```
-----
use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False,
NaN values will be encoded as non-negative integers and will not drop the
NaN from the uniques of the values.
```

Returns

```
-----
codes : ndarray
An integer NumPy array that's an indexer into the original
ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

    uniques will *not* contain an entry for the NA value of
    the ExtensionArray if there are any missing values present
    in `self`.
```

See Also

```
-----
factorize : Top-level factorize method that dispatches here.
```

Notes

```
-----
:meth:`pandas.factorize` offers a `sort` keyword as well.
```

Examples

```
-----
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
```

```

>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.base.ExtensionArray
Fill NaN values using an interpolation method.

Parameters
-----
method : str, default 'linear'
    Interpolation technique to use. One of:
    * 'linear': Ignore the index and treat the values as equally spaced.
      This is the only method supported on MultiIndexes.
    * 'time': Works on daily and higher resolution data to interpolate
      given length of interval.
    * 'index', 'values': use the actual numerical values of the index.
    * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric',
      'polynomial': Passed to scipy.interpolate.interp1d, whereas 'spline'
      is passed to scipy.interpolate.UnivariateSpline. These methods use
      the numerical values of the index.
    Both 'polynomial' and 'spline' require that you also specify an
    order (int), e.g. arr.interpolate(method='polynomial', order=5).
    * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima',
      'cubicspline': Wrappers around the SciPy interpolation methods
      of similar names. See Notes.
    * 'from_derivatives': Refers to scipy.interpolate.BPoly.from_derivatives.
axis : int
    Axis to interpolate along. For 1-dimensional data, use 0.
index : Index
    Index to use for interpolation.
limit : int or None
    Maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction : {'forward', 'backward', 'both'}
    Consecutive NaNs will be filled in this direction.
limit_area : {'inside', 'outside'} or None
    If limit is specified, consecutive NaNs will be filled with this
    restriction.
    * None: No fill restriction.
    * 'inside': Only fill NaNs surrounded by valid values (interpolate).
    * 'outside': Only fill NaNs outside valid values (extrapolate).
copy : bool
    If True, a copy of the object is returned with interpolated values.
**kwargs : optional
    Keyword arguments to pass on to the interpolating function.

Returns
-----
ExtensionArray
    An ExtensionArray with interpolated values.

See Also
-----
Series.interpolate : Interpolate values in a Series.
DataFrame.interpolate : Interpolate values in a DataFrame.

Notes
-----
- All parameters must be specified as keyword arguments.
- The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima'
  methods are wrappers around the respective SciPy implementations of
  similar names. These use the actual numerical values of the index.

Examples
-----
Interpolating values in a NumPy array:

>>> arr = pd.arrays.NumpyExtensionArray(np.array([0, 1, np.nan, 3]))

```

```
>>> arr.interpolate(
...     method="linear",
...     limit=3,
...     limit_direction="forward",
...     index=pd.Index(range(len(arr))),
...     fill_value=1,
...     copy=False,
...     axis=0,
...     limit_area="inside",
... )
<NumpyExtensionArray>
[0.0, 1.0, 2.0, 3.0]
Length: 4, dtype: float64
```

Interpolating values in a FloatingArray:

```
>>> arr = pd.array([1.0, pd.NA, 3.0, 4.0, pd.NA, 6.0], dtype="Float64")
>>> arr.interpolate(
...     method="linear",
...     axis=0,
...     index=pd.Index(range(len(arr))),
...     limit=None,
...     limit_direction="both",
...     limit_area=None,
...     copy=True,
... )
<FloatingArray>
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Length: 6, dtype: Float64
```

`item(self, index: int | None = None)` from `pandas.core.arrays.base.ExtensionArray`
Return the array element at the specified position as a Python scalar.

Parameters

`index` : int, optional

Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar

The element at the specified position.

Raises

`ValueError`

If no `index` is provided and the array does not have exactly one element.

`IndexError`

If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

`to_numpy(`

`self,`

`dtype: npt.DTypeLike | None = None,`

`copy: bool = False,`

`na_value: object = <no_default>`

`) -> np.ndarray` from `pandas.core.arrays.base.ExtensionArray`

Convert to a NumPy ndarray.

This is similar to `:meth:`numpy.asarray``, but may provide additional control

over how the conversion is done.

Parameters

`dtype` : str or numpy.dtype, optional
The dtype to pass to :meth:`numpy.asarray`.
`copy` : bool, default False
Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.
`na_value` : Any, optional
The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

`tolist(self)` -> list from pandas.core.arrays.base.ExtensionArray
Return a list of the values.

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list
Python list of values in array.

See Also

`Index.to_list`: Return a list of the values in the Index.
`Series.to_list`: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

`__pandas_priority__` = 1000

Methods inherited from pandas.core.base.PandasObject:

`__sizeof__(self)` -> int
Generates the total memory usage for an object that returns either a value or Series of values

Methods inherited from pandas.core.accessor.DirNamesMixin:

`__dir__(self)` -> list[str]
Provide method name lookup and completion.

Notes

Only provide 'public' methods.

```
class DatetimeArray(pandas.core.arrays.datetimelike.TimelikeOps, pandas.core.arrays.datetimelike.DatetimeArray(data, dtype: Dtype | None = None, freq=None, copy: bool = False) -> None
```

Pandas ExtensionArray for tz-naive or tz-aware datetime data.

.. warning::

DatetimeArray is currently experimental, and its API may change without warning. In particular, `attr: 'DatetimeArray.dtype'` is expected to change to always be an instance of an `ExtensionDtype` subclass.

Parameters

```

-----
data : Series, Index, DatetimeArray, ndarray
    The datetime data.

    For DatetimeArray `values` (or a Series or Index boxing one),
    `dtype` and `freq` will be extracted from `values`.

dtype : numpy.dtype or DatetimeTZDtype
    Note that the only NumPy dtype allowed is 'datetime64[ns]'.
freq : str or Offset, optional
    The frequency.
copy : bool, default False
    Whether to copy the underlying array of values.

Attributes
-----
None

Methods
-----
None

See Also
-----
DatetimeIndex : Immutable Index for datetime-like data.
Series : One-dimensional labeled array capable of holding datetime-like data.
Timestamp : Pandas replacement for python datetime.datetime object.
to_datetime : Convert argument to datetime.
period_range : Return a fixed frequency PeriodIndex.

Examples
-----
>>> pd.arrays.DatetimeArray._from_sequence(
...     pd.DatetimeIndex(["2023-01-01", "2023-01-02"], freq="D")
... )
<DatetimeArray>
['2023-01-01 00:00:00', '2023-01-02 00:00:00']
Length: 2, dtype: datetime64[us]

Method resolution order:
DatetimeArray
pandas.core.arrays.datetimelike.TimelikeOps
pandas.core.arrays.datetimelike.DatelikeOps
pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin
pandas.core.arraylike.OpsMixin
pandas.core.arrays._mixins.NDArrayBackedExtensionArray
pandas._libs.arrays.NDArrayBacked
pandas.api.extensions.ExtensionArray
builtins.object

Methods defined here:

__array__(self, dtype=None, copy=None) -> np.ndarray from pandas.core.arrays.datetimes.DatetimeArray
__iter__(self) -> Iterator from pandas.core.arrays.datetimes.DatetimeArray
    Return an iterator over the boxed values

Yields
-----
timestamp : Timestamp

astype(self, dtype, copy: bool = True) from pandas.core.arrays.datetimes.DatetimeArray
    Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters
-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,

```

otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also

Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from a sequence.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()

Otherwise, we will get a Numpy ndarray:

>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')

day_name(self, locale=None) -> npt.NDArray[np.object_] from pandas.core.arrays.datetimes.DatetimeArray
Return the day names with specified locale.

Parameters

locale : str, optional
Locale determining the language in which to return the day name.
Default is English locale (``'en\_US.utf8'``). Use the command
``locale -a`` on your terminal on Unix systems to find your locale
language code.

Returns

Series or Index
Series or Index of day names.

See Also

DatetimeIndex.month_name : Return the month names with specified locale.

Examples

>>> s = pd.Series(pd.date_range(start="2018-01-01", freq="D", periods=3))
>>> s
0 2018-01-01
1 2018-01-02
2 2018-01-03
dtype: datetime64[us]
>>> s.dt.day_name()
0 Monday
1 Tuesday
2 Wednesday
dtype: str

>>> idx = pd.date_range(start="2018-01-01", freq="D", periods=3)
>>> idx
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03'],
 dtype='datetime64[us]', freq='D')
>>> idx.day_name()

```
Index(['Monday', 'Tuesday', 'Wednesday'], dtype='str')
```

Using the ``locale`` parameter you can set a different locale language, for example: ``idx.day\_name(locale='pt\_BR.utf8')`` will return day names in Brazilian Portuguese language.

```
>>> idx = pd.date_range(start="2018-01-01", freq="D", periods=3)
>>> idx
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.day_name(locale="pt_BR.utf8") # doctest: +SKIP
Index(['Segunda', 'Terça', 'Quarta'], dtype='str')
```

`isocalendar(self)` -> DataFrame from pandas.core.arrays.datetimes.DatetimeArray
Calculate year, week, and day according to the ISO 8601 standard.

Returns

DataFrame

With columns year, week and day.

See Also

Timestamp.isocalendar : Function return a 3-tuple containing ISO year, week number, and weekday for the given Timestamp object.

datetime.date.isocalendar : Return a named tuple object with three components: year, week and weekday.

Examples

```
>>> idx = pd.date_range(start="2019-12-29", freq="D", periods=4)
>>> idx.isocalendar()
      year  week  day
2019-12-29  2019   52    7
2019-12-30  2020    1    1
2019-12-31  2020    1    2
2020-01-01  2020    1    3
>>> idx.isocalendar().week
2019-12-29    52
2019-12-30     1
2019-12-31     1
2020-01-01     1
Freq: D, Name: week, dtype: UInt32
```

`month_name(self, locale=None)` -> npt.NDArray[np.object_] from pandas.core.arrays.datetimes.DatetimeArray
Return the month names with specified locale.

Parameters

locale : str, optional

Locale determining the language in which to return the month name. Default is English locale (``'en\_US.utf8'``). Use the command ``locale -a`` on your terminal on Unix systems to find your locale language code.

Returns

Series or Index

Series or Index of month names.

See Also

DatetimeIndex.day_name : Return the day names with specified locale.

Examples

```
>>> s = pd.Series(pd.date_range(start="2018-01", freq="ME", periods=3))
>>> s
0    2018-01-31
1    2018-02-28
2    2018-03-31
dtype: datetime64[us]
>>> s.dt.month_name()
0    January
1    February
2     March
dtype: str
```

```
>>> idx = pd.date_range(start="2018-01", freq="ME", periods=3)
>>> idx
DatetimeIndex(['2018-01-31', '2018-02-28', '2018-03-31'],
              dtype='datetime64[us]', freq='ME')
>>> idx.month_name()
Index(['January', 'February', 'March'], dtype='str')
```

Using the ``locale`` parameter you can set a different locale language, for example: ``idx.month\_name(locale='pt\_BR.utf8')`` will return month names in Brazilian Portuguese language.

```
>>> idx = pd.date_range(start="2018-01", freq="ME", periods=3)
>>> idx
DatetimeIndex(['2018-01-31', '2018-02-28', '2018-03-31'],
              dtype='datetime64[us]', freq='ME')
>>> idx.month_name(locale="pt_BR.utf8") # doctest: +SKIP
Index(['Janeiro', 'Fevereiro', 'Março'], dtype='str')
```

normalize(self) -> Self from pandas.core.arrays.datetimes.DatetimeArray
Convert times to midnight.

The time component of the date-time is converted to midnight i.e. 00:00:00. This is useful in cases, when the time does not matter. Length is unaltered. The timezones are unaffected.

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on Datetime Array/Index.

Returns

DatetimeArray, DatetimeIndex or Series
The same type as the original data. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

floor : Floor the datetimes to the specified freq.
ceil : Ceil the datetimes to the specified freq.
round : Round the datetimes to the specified freq.

Examples

```
>>> idx = pd.date_range(
...     start="2014-08-01 10:00", freq="h", periods=3, tz="Asia/Calcutta"
... )
>>> idx
DatetimeIndex(['2014-08-01 10:00:00+05:30',
              '2014-08-01 11:00:00+05:30',
              '2014-08-01 12:00:00+05:30'],
              dtype='datetime64[us, Asia/Calcutta]', freq='h')
>>> idx.normalize()
DatetimeIndex(['2014-08-01 00:00:00+05:30',
              '2014-08-01 00:00:00+05:30',
              '2014-08-01 00:00:00+05:30'],
              dtype='datetime64[us, Asia/Calcutta]', freq=None)
```

std(
self,
axis=None,
dtype=None,
out=None,
ddof: int = 1,
keepdims: bool = False,
skipna: bool = True
) -> Timedelta from pandas.core.arrays.datetimes.DatetimeArray
Return sample standard deviation over requested axis.

Normalized by ``N-1`` by default. This can be changed using ``ddof``.

Parameters

axis : int, optional
Axis for the function to be applied on. For :class:`pandas.Series` this parameter is unused and defaults to ``None``.
dtype : dtype, optional, default None
Type to use in computing the standard deviation. For arrays of

integer type the default is float64, for arrays of float types it is the same as the array type.

out : ndarray, optional, default None
Alternative output array in which to place the result. It must have the same shape as the expected output but the type (of the calculated values) will be cast if necessary.

ddof : int, default 1
Degrees of Freedom. The divisor used in calculations is `'N - ddof'`, where `'N'` represents the number of elements.

keepdims : bool, optional
If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array. If the default value is passed, then keepdims will not be passed through to the std method of sub-classes of ndarray, however any non-default value will be. If the sub-class method does not implement keepdims any exceptions will be raised.

skipna : bool, default True
Exclude NA/null values. If an entire row/column is `'NA'`, the result will be `'NA'`.

Returns

Timedelta

Standard deviation over requested axis.

See Also

numpy.ndarray.std : Returns the standard deviation of the array elements along given axis.

Series.std : Return sample standard deviation over requested axis.

Examples

For :class:`pandas.DatetimeIndex`:

```
>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.std()
Timedelta('1 days 00:00:00')
```

to_julian_date(self) -> npt.NDArray[np.float64] from pandas.core.arrays.datetimes.DatetimeArray
Convert Timestamp to a Julian Date.

This method returns the number of days as a float since noon January 1, 4713 BC.

https://en.wikipedia.org/wiki/Julian_day

Returns

ndarray or Index

Float values that represent each date in Julian Calendar.

See Also

Timestamp.to_julian_date : Equivalent method on `'Timestamp'` objects.

Examples

```
>>> idx = pd.DatetimeIndex(["2028-08-12 00:54", "2028-08-12 02:06"])
>>> idx.to_julian_date()
Index([2461995.5375, 2461995.5875], dtype='float64')
```

to_period(self, freq=None) -> PeriodArray from pandas.core.arrays.datetimes.DatetimeArray
Cast to PeriodArray/PeriodIndex at a particular frequency.

Converts DatetimeArray/Index to PeriodArray/PeriodIndex.

Parameters

freq : str or Period, optional

One of pandas' :ref:`period aliases <timeseries.period\_aliases>` or a Period object. Will be inferred by default.

Returns

PeriodArray/PeriodIndex
Immutable ndarray holding ordinal values at a particular frequency.

Raises

ValueError
When converting a DatetimeArray/Index with non-regular values,
so that a frequency cannot be inferred.

See Also

PeriodIndex: Immutable ndarray holding ordinal values.
DatetimeIndex.to_pydatetime: Return DatetimeIndex as object.

Examples

>>> df = pd.DataFrame(
... {"y": [1, 2, 3]},
... index=pd.to_datetime(
... [
... "2000-03-31 00:00:00",
... "2000-05-31 00:00:00",
... "2000-08-31 00:00:00",
...]
...),
...)
>>> df.index.to_period("M")
PeriodIndex(['2000-03', '2000-05', '2000-08'],
 dtype='period[M]')

Infer the daily frequency

>>> idx = pd.date_range("2017-01-01", periods=2)
>>> idx.to_period()
PeriodIndex(['2017-01-01', '2017-01-02'],
 dtype='period[D]')

to_pydatetime(self) -> npt.NDArray[np.object_] from pandas.core.arrays.datetimes.DatetimeArray
Return an ndarray of ``datetime.datetime`` objects.

Returns

numpy.ndarray
An ndarray of ``datetime.datetime`` objects.

See Also

DatetimeIndex.to_julian_date : Converts Datetime Array to float64 ndarray
of Julian Dates.

Examples

>>> idx = pd.date_range("2018-02-27", periods=3)
>>> idx.to_pydatetime()
array([datetime.datetime(2018, 2, 27, 0, 0),
 datetime.datetime(2018, 2, 28, 0, 0),
 datetime.datetime(2018, 3, 1, 0, 0)], dtype=object)

tz_convert(self, tz) -> Self from pandas.core.arrays.datetimes.DatetimeArray
Convert tz-aware Datetime Array/Index from one time zone to another.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, datetime.tzinfo or None
Time zone for time. Corresponding timestamps would be converted
to this time zone of the Datetime Array/Index. A ``tz`` of None will
convert to UTC and remove the timezone information.

Returns

Array or Index
Datetime Array/Index with target ``tz``.

Raises

TypeError

If Datetime Array/Index is tz-naive.

See Also

DatetimeIndex.tz : A timezone that has a variable offset from UTC.
DatetimeIndex.tz_localize : Localize tz-naive DatetimeIndex to a given time zone, or remove timezone from a tz-aware DatetimeIndex.

Examples

With the `tz` parameter, we can change the DatetimeIndex to other time zones:

```
>>> dti = pd.date_range(
...     start="2014-08-01 09:00", freq="h", periods=3, tz="Europe/Berlin"
... )
```

```
>>> dti
DatetimeIndex(['2014-08-01 09:00:00+02:00',
               '2014-08-01 10:00:00+02:00',
               '2014-08-01 11:00:00+02:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='h')
```

```
>>> dti.tz_convert("US/Central")
DatetimeIndex(['2014-08-01 02:00:00-05:00',
               '2014-08-01 03:00:00-05:00',
               '2014-08-01 04:00:00-05:00'],
              dtype='datetime64[us, US/Central]', freq='h')
```

With the ``tz=None``, we can remove the timezone (after converting to UTC if necessary):

```
>>> dti = pd.date_range(
...     start="2014-08-01 09:00", freq="h", periods=3, tz="Europe/Berlin"
... )
```

```
>>> dti
DatetimeIndex(['2014-08-01 09:00:00+02:00',
               '2014-08-01 10:00:00+02:00',
               '2014-08-01 11:00:00+02:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='h')
```

```
>>> dti.tz_convert(None)
DatetimeIndex(['2014-08-01 07:00:00',
               '2014-08-01 08:00:00',
               '2014-08-01 09:00:00'],
              dtype='datetime64[us]', freq='h')
```

```
tz_localize(
    self,
    tz,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self from pandas.core.arrays.datetimes.DatetimeArray
Localize tz-naive Datetime Array/Index to tz-aware Datetime Array/Index.
```

This method takes a time zone (tz) naive Datetime Array/Index object and makes this time zone aware. It does not move the time to another time zone.

This method can also be used to do the inverse -- to create a time zone unaware object from an aware object. To that end, pass `tz=None`.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, datetime.tzinfo or None
Time zone to convert timestamps to. Passing ``None`` will remove the time zone information preserving local time.
ambiguous : 'infer', 'NaT', bool array, default 'raise'
When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on


```

order
- bool-ndarray where True signifies a DST time, False signifies a
  non-DST time (note that this flag is only applicable for
  ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous
  times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the
  closest existing time
- 'shift_backward' will shift the nonexistent time backward to the
  closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are
  nonexistent times.

Returns
-----
Same type as self
Array/Index converted to the specified time zone.

Raises
-----
TypeError
    If the Datetime Array/Index is tz-aware and tz is not None.

See Also
-----
DatetimeIndex.tz_convert : Convert tz-aware DatetimeIndex from
    one time zone to another.

Examples
-----
>>> tz_naive = pd.date_range('2018-03-01 09:00', periods=3)
>>> tz_naive
DatetimeIndex(['2018-03-01 09:00:00', '2018-03-02 09:00:00',
               '2018-03-03 09:00:00'],
              dtype='datetime64[us]', freq='D')

Localize DatetimeIndex in US/Eastern time zone:

>>> tz_aware = tz_naive.tz_localize(tz='US/Eastern')
>>> tz_aware
DatetimeIndex(['2018-03-01 09:00:00-05:00',
               '2018-03-02 09:00:00-05:00',
               '2018-03-03 09:00:00-05:00'],
              dtype='datetime64[us, US/Eastern]', freq=None)

With the ``tz=None``, we can remove the time zone information
while keeping the local time (not converted to UTC):

>>> tz_aware.tz_localize(None)
DatetimeIndex(['2018-03-01 09:00:00', '2018-03-02 09:00:00',
               '2018-03-03 09:00:00'],
              dtype='datetime64[us]', freq=None)

Be careful with DST changes. When there is sequential data, pandas can
infer the DST time:

>>> s = pd.to_datetime(pd.Series(['2018-10-28 01:30:00',
...                               '2018-10-28 02:00:00',
...                               '2018-10-28 02:30:00',
...                               '2018-10-28 02:00:00',
...                               '2018-10-28 02:30:00',
...                               '2018-10-28 03:00:00',
...                               '2018-10-28 03:30:00']))
>>> s.dt.tz_localize('CET', ambiguous='infer')
0    2018-10-28 01:30:00+02:00
1    2018-10-28 02:00:00+02:00
2    2018-10-28 02:30:00+02:00
3    2018-10-28 02:00:00+01:00
4    2018-10-28 02:30:00+01:00

```

```

5    2018-10-28 03:00:00+01:00
6    2018-10-28 03:30:00+01:00
dtype: datetime64[us, CET]

```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```

>>> s = pd.to_datetime(pd.Series(['2018-10-28 01:20:00',
...                               '2018-10-28 02:36:00',
...                               '2018-10-28 03:46:00']))
>>> s.dt.tz_localize('CET', ambiguous=np.array([True, True, False]))
0    2018-10-28 01:20:00+02:00
1    2018-10-28 02:36:00+02:00
2    2018-10-28 03:46:00+01:00
dtype: datetime64[us, CET]

```

If the DST transition causes nonexistent times, you can shift these dates forward or backwards with a timedelta object or `'shift_forward'` or `'shift_backwards'`.

```

>>> s = pd.to_datetime(pd.Series(['2015-03-29 02:30:00',
...                               '2015-03-29 03:30:00'], dtype="M8[ns]"))
>>> s.dt.tz_localize('Europe/Warsaw', nonexistent='shift_forward')
0    2015-03-29 03:00:00+02:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]

>>> s.dt.tz_localize('Europe/Warsaw', nonexistent='shift_backward')
0    2015-03-29 01:59:59.999999999+01:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]

>>> s.dt.tz_localize('Europe/Warsaw', nonexistent=pd.Timedelta('1h'))
0    2015-03-29 03:30:00+02:00
1    2015-03-29 03:30:00+02:00
dtype: datetime64[ns, Europe/Warsaw]

```

Readonly properties defined here:

`date`

Returns numpy array of python `:class:`datetime.date`` objects.

Namely, the date part of Timestamps without time and timezone information.

See Also

`DatetimeIndex.time` : Returns numpy array of `:class:`datetime.time`` objects.
The time part of the Timestamps.
`DatetimeIndex.year` : The year of the datetime.
`DatetimeIndex.month` : The month as January=1, December=12.
`DatetimeIndex.day` : The day of the datetime.

Examples

For Series:

```

>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.date
0    2020-01-01
1    2020-02-01
dtype: object

```

For DatetimeIndex:

```

>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.date
array([datetime.date(2020, 1, 1), datetime.date(2020, 2, 1)], dtype=object)

```

day

The day of the datetime.

See Also

DatetimeIndex.year: The year of the datetime.

DatetimeIndex.month: The month as January=1, December=12.

DatetimeIndex.hour: The hours of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="D")
... )
>>> datetime_series
0    2000-01-01
1    2000-01-02
2    2000-01-03
dtype: datetime64[us]
>>> datetime_series.dt.day
0     1
1     2
2     3
dtype: int32
```

day_of_week

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

Series.dt.dayofweek : Alias.

Series.dt.weekday : Alias.

Series.dt.day_name : Returns the name of the day of the week.

Examples

```
>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
>>> s.dt.dayofweek
2016-12-31    5
2017-01-01    6
2017-01-02    0
2017-01-03    1
2017-01-04    2
2017-01-05    3
2017-01-06    4
2017-01-07    5
2017-01-08    6
Freq: D, dtype: int32
```

day_of_year

The ordinal day of the year.

See Also

DatetimeIndex.dayofweek : The day of the week with Monday=0, Sunday=6.

DatetimeIndex.day : The day of the datetime.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
```

```
>>> s.dt.dayofyear
0      1
1     32
dtype: int32

For DatetimeIndex:

>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",
...                          "2/1/2020 11:00:00+00:00"])
>>> idx.dayofyear
Index([1, 32], dtype='int32')
```

dayofweek

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

Series.dt.dayofweek : Alias.

Series.dt.weekday : Alias.

Series.dt.day_name : Returns the name of the day of the week.

Examples

```
>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
>>> s.dt.dayofweek
2016-12-31      5
2017-01-01      6
2017-01-02      0
2017-01-03      1
2017-01-04      2
2017-01-05      3
2017-01-06      4
2017-01-07      5
2017-01-08      6
Freq: D, dtype: int32
```

dayofyear

The ordinal day of the year.

See Also

DatetimeIndex.dayofweek : The day of the week with Monday=0, Sunday=6.

DatetimeIndex.day : The day of the datetime.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.dayofyear
0      1
1     32
dtype: int32
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",
...                          "2/1/2020 11:00:00+00:00"])
>>> idx.dayofyear
Index([1, 32], dtype='int32')
```

days_in_month

The number of days in the month.

See Also

`Series.dt.day` : Return the day of the month.

`Series.dt.is_month_end` : Return a boolean indicating if the date is the last day of the month.

`Series.dt.is_month_start` : Return a boolean indicating if the date is the first day of the month.

`Series.dt.month` : Return the month as January=1 through December=12.

Examples

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
```

```
>>> s = pd.to_datetime(s)
```

```
>>> s
```

```
0    2020-01-01 10:00:00+00:00
```

```
1    2020-02-01 11:00:00+00:00
```

```
dtype: datetime64[us, UTC]
```

```
>>> s.dt.daysinmonth
```

```
0     31
```

```
1     29
```

```
dtype: int32
```

`daysinmonth`

The number of days in the month.

See Also

`Series.dt.day` : Return the day of the month.

`Series.dt.is_month_end` : Return a boolean indicating if the date is the last day of the month.

`Series.dt.is_month_start` : Return a boolean indicating if the date is the first day of the month.

`Series.dt.month` : Return the month as January=1 through December=12.

Examples

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
```

```
>>> s = pd.to_datetime(s)
```

```
>>> s
```

```
0    2020-01-01 10:00:00+00:00
```

```
1    2020-02-01 11:00:00+00:00
```

```
dtype: datetime64[us, UTC]
```

```
>>> s.dt.daysinmonth
```

```
0     31
```

```
1     29
```

```
dtype: int32
```

`dtype`

The dtype for the DatetimeArray.

.. warning::

A future version of pandas will change dtype to never be a ``numpy.dtype``. Instead, `:attr:`DatetimeArray.dtype`` will always be an instance of an ``ExtensionDtype`` subclass.

Returns

`numpy.dtype` or `DatetimeTZDtype`

If the values are tz-naive, then ``np.dtype('datetime64[ns]')`` is returned.

If the values are tz-aware, then the ``DatetimeTZDtype`` is returned.

`hour`

The hours of the datetime.

See Also

`DatetimeIndex.day`: The day of the datetime.

`DatetimeIndex.minute`: The minutes of the datetime.

`DatetimeIndex.second`: The seconds of the datetime.

Examples

```

-----
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="h")
... )
>>> datetime_series
0    2000-01-01 00:00:00
1    2000-01-01 01:00:00
2    2000-01-01 02:00:00
dtype: datetime64[us]
>>> datetime_series.dt.hour
0      0
1      1
2      2
dtype: int32

```

is_leap_year

Boolean indicator if the date belongs to a leap year.

A leap year is a year, which has 366 days (instead of 365) including 29th of February as an intercalary day. Leap years are years which are multiples of four with the exception of years divisible by 100 but not by 400.

Returns

Series or ndarray
Booleans indicating if dates belong to a leap year.

See Also

DatetimeIndex.is_year_end : Indicate whether the date is the last day of the year.
DatetimeIndex.is_year_start : Indicate whether the date is the first day of a year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```

>>> idx = pd.date_range("2012-01-01", "2015-01-01", freq="YE")
>>> idx
DatetimeIndex(['2012-12-31', '2013-12-31', '2014-12-31'],
              dtype='datetime64[us]', freq='YE-DEC')
>>> idx.is_leap_year
array([ True, False, False])

```

```

>>> dates_series = pd.Series(idx)
>>> dates_series
0    2012-12-31
1    2013-12-31
2    2014-12-31
dtype: datetime64[us]
>>> dates_series.dt.is_leap_year
0      True
1     False
2     False
dtype: bool

```

is_month_end

Indicates whether the date is the last day of the month.

Returns

Series or array
For Series, returns a Series with boolean values.
For DatetimeIndex, returns a boolean array.

See Also

is_month_start : Return a boolean indicating whether the date is the first day of the month.
is_month_end : Return a boolean indicating whether the date is the last day of the month.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> s = pd.Series(pd.date_range("2018-02-27", periods=3))
>>> s
0    2018-02-27
1    2018-02-28
2    2018-03-01
dtype: datetime64[us]
>>> s.dt.is_month_start
0    False
1    False
2     True
dtype: bool
>>> s.dt.is_month_end
0    False
1     True
2    False
dtype: bool

>>> idx = pd.date_range("2018-02-27", periods=3)
>>> idx.is_month_start
array([False, False,  True])
>>> idx.is_month_end
array([False,  True, False])
```

is_month_start

Indicates whether the date is the first day of the month.

Returns

Series or array

For Series, returns a Series with boolean values.
For DatetimeIndex, returns a boolean array.

See Also

is_month_start : Return a boolean indicating whether the date is the first day of the month.

is_month_end : Return a boolean indicating whether the date is the last day of the month.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> s = pd.Series(pd.date_range("2018-02-27", periods=3))
>>> s
0    2018-02-27
1    2018-02-28
2    2018-03-01
dtype: datetime64[us]
>>> s.dt.is_month_start
0    False
1    False
2     True
dtype: bool
>>> s.dt.is_month_end
0    False
1     True
2    False
dtype: bool

>>> idx = pd.date_range("2018-02-27", periods=3)
>>> idx.is_month_start
array([False, False,  True])
>>> idx.is_month_end
array([False,  True, False])
```

is_normalized

Returns True if all of the dates are at midnight ("no time")

is_quarter_end

Indicator for whether the date is the last day of a quarter.

Returns

```

-----
is_quarter_end : Series or DatetimeIndex
    The same type as the original data with boolean values. Series will
    have the same name and index. DatetimeIndex will have the same
    name.

```

See Also

```

-----
quarter : Return the quarter of the date.
is_quarter_start : Similar property indicating the quarter start.

```

Examples

```

-----
This method is available on Series with datetime values under
the ``.dt`` accessor, and directly on DatetimeIndex.

```

```

>>> df = pd.DataFrame({'dates': pd.date_range("2017-03-30",
...                                           periods=4)})
>>> df.assign(quarter=df.dates.dt.quarter,
...           is_quarter_end=df.dates.dt.is_quarter_end)
   dates      quarter  is_quarter_end
0 2017-03-30         1             False
1 2017-03-31         1              True
2 2017-04-01         2             False
3 2017-04-02         2             False

>>> idx = pd.date_range('2017-03-30', periods=4)
>>> idx
DatetimeIndex(['2017-03-30', '2017-03-31', '2017-04-01', '2017-04-02'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_quarter_end
array([False,  True,  False,  False])

```

```

is_quarter_start
Indicator for whether the date is the first day of a quarter.

```

Returns

```

-----
is_quarter_start : Series or DatetimeIndex
    The same type as the original data with boolean values. Series will
    have the same name and index. DatetimeIndex will have the same
    name.

```

See Also

```

-----
quarter : Return the quarter of the date.
is_quarter_end : Similar property for indicating the quarter end.

```

Examples

```

-----
This method is available on Series with datetime values under
the ``.dt`` accessor, and directly on DatetimeIndex.

```

```

>>> df = pd.DataFrame({'dates': pd.date_range("2017-03-30",
...                                           periods=4)})
>>> df.assign(quarter=df.dates.dt.quarter,
...           is_quarter_start=df.dates.dt.is_quarter_start)
   dates      quarter  is_quarter_start
0 2017-03-30         1             False
1 2017-03-31         1             False
2 2017-04-01         2              True
3 2017-04-02         2             False

>>> idx = pd.date_range('2017-03-30', periods=4)
>>> idx
DatetimeIndex(['2017-03-30', '2017-03-31', '2017-04-01', '2017-04-02'],
              dtype='datetime64[us]', freq='D')

>>> idx.is_quarter_start
array([False,  False,  True,  False])

```

```

is_year_end
Indicate whether the date is the last day of the year.

```

Returns

```

-----

```


Series or DatetimeIndex

The same type as the original data with boolean values. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

`is_year_start` : Similar property indicating the start of the year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> dates = pd.Series(pd.date_range("2017-12-30", periods=3))
>>> dates
0    2017-12-30
1    2017-12-31
2    2018-01-01
dtype: datetime64[us]
```

```
>>> dates.dt.is_year_end
0    False
1     True
2    False
dtype: bool
```

```
>>> idx = pd.date_range("2017-12-30", periods=3)
>>> idx
DatetimeIndex(['2017-12-30', '2017-12-31', '2018-01-01'],
              dtype='datetime64[us]', freq='D')
```

```
>>> idx.is_year_end
array([False,  True, False])
```

`is_year_start`

Indicate whether the date is the first day of a year.

Returns

Series or DatetimeIndex

The same type as the original data with boolean values. Series will have the same name and index. DatetimeIndex will have the same name.

See Also

`is_year_end` : Similar property indicating the last day of the year.

Examples

This method is available on Series with datetime values under the ``.dt`` accessor, and directly on DatetimeIndex.

```
>>> dates = pd.Series(pd.date_range("2017-12-30", periods=3))
>>> dates
0    2017-12-30
1    2017-12-31
2    2018-01-01
dtype: datetime64[us]
```

```
>>> dates.dt.is_year_start
0    False
1    False
2     True
dtype: bool
```

```
>>> idx = pd.date_range("2017-12-30", periods=3)
>>> idx
DatetimeIndex(['2017-12-30', '2017-12-31', '2018-01-01'],
              dtype='datetime64[us]', freq='D')
```

```
>>> idx.is_year_start
array([False, False,  True])
```

This method, when applied to Series with datetime values under the ``.dt`` accessor, will lose information about Business offsets.

```

>>> dates = pd.Series(pd.date_range("2020-10-30", periods=4, freq="BYS"))
>>> dates
0    2021-01-01
1    2022-01-03
2    2023-01-02
3    2024-01-01
dtype: datetime64[us]

>>> dates.dt.is_year_start
0     True
1    False
2    False
3     True
dtype: bool

>>> idx = pd.date_range("2020-10-30", periods=4, freq="BYS")
>>> idx
DatetimeIndex(['2021-01-01', '2022-01-03', '2023-01-02', '2024-01-01'],
              dtype='datetime64[us]', freq='BYS-JAN')

>>> idx.is_year_start
array([ True,  True,  True,  True])

```

microsecond

The microseconds of the datetime.

See Also

DatetimeIndex.second: The seconds of the datetime.
 DatetimeIndex.nanosecond: The nanoseconds of the datetime.

Examples

```

>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="us")
... )
>>> datetime_series
0    2000-01-01 00:00:00.000000
1    2000-01-01 00:00:00.000001
2    2000-01-01 00:00:00.000002
dtype: datetime64[us]
>>> datetime_series.dt.microsecond
0      0
1      1
2      2
dtype: int32

```

minute

The minutes of the datetime.

See Also

DatetimeIndex.hour: The hours of the datetime.
 DatetimeIndex.second: The seconds of the datetime.

Examples

```

>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="min")
... )
>>> datetime_series
0    2000-01-01 00:00:00
1    2000-01-01 00:01:00
2    2000-01-01 00:02:00
dtype: datetime64[us]
>>> datetime_series.dt.minute
0      0
1      1
2      2
dtype: int32

```

month

The month as January=1, December=12.

See Also

DatetimeIndex.year: The year of the datetime.
DatetimeIndex.day: The day of the datetime.

Examples

```
-----
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="ME")
... )
>>> datetime_series
0    2000-01-31
1    2000-02-29
2    2000-03-31
dtype: datetime64[us]
>>> datetime_series.dt.month
0     1
1     2
2     3
dtype: int32
```

nanosecond

The nanoseconds of the datetime.

See Also

DatetimeIndex.second: The seconds of the datetime.
DatetimeIndex.microsecond: The microseconds of the datetime.

Examples

```
-----
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="ns")
... )
>>> datetime_series
0    2000-01-01 00:00:00.000000000
1    2000-01-01 00:00:00.000000001
2    2000-01-01 00:00:00.000000002
dtype: datetime64[ns]
>>> datetime_series.dt.nanosecond
0     0
1     1
2     2
dtype: int32
```

quarter

The quarter of the date.

See Also

DatetimeIndex.snap : Snap time stamps to nearest occurring frequency.
DatetimeIndex.time : Returns numpy array of datetime.time objects.
The time part of the Timestamps.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "4/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-04-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.quarter
0     1
1     2
dtype: int32
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00",
...                          "2/1/2020 11:00:00+00:00"])
>>> idx.quarter
Index([1, 1], dtype='int32')
```

second

The seconds of the datetime.

See Also

DatetimeIndex.minute: The minutes of the datetime.
DatetimeIndex.microsecond: The microseconds of the datetime.
DatetimeIndex.nanosecond: The nanoseconds of the datetime.

Examples

```
>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="s")
... )
>>> datetime_series
0    2000-01-01 00:00:00
1    2000-01-01 00:00:01
2    2000-01-01 00:00:02
dtype: datetime64[us]
>>> datetime_series.dt.second
0    0
1    1
2    2
dtype: int32
```

time

Returns numpy array of :class:`datetime.time` objects.

The time part of the Timestamps.

See Also

DatetimeIndex.timetz : Returns numpy array of :class:`datetime.time` objects with timezones. The time part of the Timestamps.
DatetimeIndex.date : Returns numpy array of python :class:`datetime.date` objects. Namely, the date part of Timestamps without time and timezone information.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.time
0    10:00:00
1    11:00:00
dtype: object
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.time
array([datetime.time(10, 0), datetime.time(11, 0)], dtype=object)
```

timetz

Returns numpy array of :class:`datetime.time` objects with timezones.

The time part of the Timestamps.

See Also

DatetimeIndex.time : Returns numpy array of :class:`datetime.time` objects. The time part of the Timestamps.
DatetimeIndex.tz : Return the timezone.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
```

```

1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.tz
0    10:00:00+00:00
1    11:00:00+00:00
dtype: object

For DatetimeIndex:

>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.tz
array([datetime.time(10, 0, tzinfo=datetime.timezone.utc),
datetime.time(11, 0, tzinfo=datetime.timezone.utc)], dtype=object)

```

tzinfo

Alias for tz attribute

weekday

The day of the week with Monday=0, Sunday=6.

Return the day of the week. It is assumed the week starts on Monday, which is denoted by 0 and ends on Sunday which is denoted by 6. This method is available on both Series with datetime values (using the `dt` accessor) or DatetimeIndex.

Returns

Series or Index

Containing integers indicating the day number.

See Also

Series.dt.dayofweek : Alias.

Series.dt.weekday : Alias.

Series.dt.day_name : Returns the name of the day of the week.

Examples

```

>>> s = pd.date_range('2016-12-31', '2017-01-08', freq='D').to_series()
>>> s.dt.dayofweek
2016-12-31    5
2017-01-01    6
2017-01-02    0
2017-01-03    1
2017-01-04    2
2017-01-05    3
2017-01-06    4
2017-01-07    5
2017-01-08    6
Freq: D, dtype: int32

```

year

The year of the datetime.

See Also

DatetimeIndex.month: The month as January=1, December=12.

DatetimeIndex.day: The day of the datetime.

Examples

```

>>> datetime_series = pd.Series(
...     pd.date_range("2000-01-01", periods=3, freq="YE")
... )
>>> datetime_series
0    2000-12-31
1    2001-12-31
2    2002-12-31
dtype: datetime64[us]
>>> datetime_series.dt.year
0    2000
1    2001
2    2002
dtype: int32

```

Data descriptors defined here:

tz

Return the timezone.

Returns

zoneinfo.ZoneInfo,, datetime.tzinfo, pytz.tzinfo.BaseTZInfo, dateutil.tz.tz.tzfile, or None
Returns None when the array is tz-naive.

See Also

DatetimeIndex.tz_localize : Localize tz-naive DatetimeIndex to a given time zone, or remove timezone from a tz-aware DatetimeIndex.
DatetimeIndex.tz_convert : Convert tz-aware DatetimeIndex from one time zone to another.

Examples

For Series:

```
>>> s = pd.Series(["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"])
>>> s = pd.to_datetime(s)
>>> s
0    2020-01-01 10:00:00+00:00
1    2020-02-01 11:00:00+00:00
dtype: datetime64[us, UTC]
>>> s.dt.tz
datetime.timezone.utc
```

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(
...     ["1/1/2020 10:00:00+00:00", "2/1/2020 11:00:00+00:00"]
... )
>>> idx.tz
datetime.timezone.utc
```

Data and other attributes defined here:

__annotations__ = {'_bool_ops': 'list[str]', '_datetimelike_methods': ...

__array_priority__ = 1000

Methods inherited from pandas.core.arrays.datetimelike.TimelikeOps:

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs)

all(self, *, axis: AxisInt | None = None, skipna: bool = True) -> bool

any(self, *, axis: AxisInt | None = None, skipna: bool = True) -> bool

as_unit(self, unit: TimeUnit, round_ok: bool = True) -> Self
Convert to a dtype with the given unit resolution.

The limits of timestamp representation depend on the chosen resolution.
Different resolutions can be converted to each other through as_unit.

Parameters

unit : {'s', 'ms', 'us', 'ns'}
round_ok : bool, default True
If False and the conversion requires rounding, raise ValueError.

Returns

same type as self
Converted to the specified unit.

See Also

Timestamp.as_unit : Convert to the given unit.

Examples

```

-----
For :class:`pandas.DatetimeIndex`:

>>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])
>>> idx
DatetimeIndex(['2020-01-02 01:02:03.004005006'],
              dtype='datetime64[ns]', freq=None)
>>> idx.as_unit("s")
DatetimeIndex(['2020-01-02 01:02:03'], dtype='datetime64[s]', freq=None)

For :class:`pandas.TimedeltaIndex`:

>>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
>>> tdelta_idx
TimedeltaIndex(['1 days 00:03:00.000002042'],
              dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.as_unit("s")
TimedeltaIndex(['1 days 00:03:00'], dtype='timedelta64[s]', freq=None)

ceil(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Perform ceil operation on the data to the specified `freq`.

Parameters
-----
freq : str or Offset
    The frequency level to ceil the index to. Must be a fixed
    frequency like 's' (second) not 'ME' (month end). See
    :ref:`frequency aliases <timeseries.offset_aliases>` for
    a list of possible `freq` values.
ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
    Only relevant for DatetimeIndex:

    - 'infer' will attempt to infer fall dst-transition hours based on
      order
    - bool-ndarray where True signifies a DST time, False designates
      a non-DST time (note that this flag is only applicable for
      ambiguous times)
    - 'NaT' will return NaT where there are ambiguous times
    - 'raise' will raise a ValueError if there are ambiguous
      times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
    A nonexistent time does not exist in a particular timezone
    where clocks moved forward due to DST.

    - 'shift_forward' will shift the nonexistent time forward to the
      closest existing time
    - 'shift_backward' will shift the nonexistent time backward to the
      closest existing time
    - 'NaT' will return NaT where there are nonexistent times
    - timedelta objects will shift nonexistent times by the timedelta
    - 'raise' will raise a ValueError if there are
      nonexistent times.

Returns
-----
DatetimeIndex, TimedeltaIndex, or Series
    Index of the same type for a DatetimeIndex or TimedeltaIndex,
    or a Series with the same index for a Series.

Raises
-----
ValueError if the `freq` cannot be converted.

See Also
-----
DatetimeIndex.floor :
    Perform floor operation on the data to the specified `freq`.
DatetimeIndex.snap :
    Snap time stamps to nearest occurring frequency.

Notes

```

If the timestamps have a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.ceil('h')
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',
               '2018-01-01 13:00:00'],
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.ceil("h")
0    2018-01-01 12:00:00
1    2018-01-01 12:00:00
2    2018-01-01 13:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 01:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.ceil("h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.ceil("h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

copy(self, order: str = 'C') -> Self
Return a copy of the array.

This method creates a copy of the `ExtensionArray` where modifying the data in the copy will not affect the original array. This is useful when you want to manipulate data without altering the original dataset.

Returns

ExtensionArray

A new `ExtensionArray` object that is a copy of the current instance.

See Also

DataFrame.copy : Return a copy of the DataFrame.

Series.copy : Return a copy of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

factorize(self, use_na_sentinel: bool = True, sort: bool = False)
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True

If True, the sentinel -1 will be used for NaN values. If False,

NaN values will be encoded as non-negative integers and will not drop the

NaN from the uniques of the values.

Returns

codes : ndarray

An integer NumPy array that's an indexer into the original ExtensionArray.

uniques : ExtensionArray

An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')
```

```
floor(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
```

Perform floor operation on the data to the specified `freq`.

Parameters

freq : str or Offset

The frequency level to floor the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'

Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

 DatetimeIndex, TimedeltaIndex, or Series
 Index of the same type for a DatetimeIndex or TimedeltaIndex,
 or a Series with the same index for a Series.

Raises

 ValueError if the `freq` cannot be converted.

See Also

 DatetimeIndex.floor :
 Perform floor operation on the data to the specified `freq`.
 DatetimeIndex.snap :
 Snap time stamps to nearest occurring frequency.

Notes

 If the timestamps have a timezone, flooring will take place relative to the
 local ("wall") time and re-localized to the same timezone. When flooring
 near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
 control the re-localization behavior.

Examples

 DatetimeIndex

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.floor('h')
DatetimeIndex(['2018-01-01 11:00:00', '2018-01-01 12:00:00',
               '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)
```

Series

```
>>> pd.Series(rng).dt.floor("h")
0    2018-01-01 11:00:00
1    2018-01-01 12:00:00
2    2018-01-01 12:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or
 ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self
    See NDFrame.interpolate.__doc__.
```

```
round(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
```

```

    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Perform round operation on the data to the specified `freq`.

Parameters
-----
freq : str or Offset
    The frequency level to round the index to. Must be a fixed
    frequency like 's' (second) not 'ME' (month end). See
    :ref:`frequency aliases <timeseries.offset_aliases>` for
    a list of possible `freq` values.
ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
    Only relevant for DatetimeIndex:

    - 'infer' will attempt to infer fall dst-transition hours based on
      order
    - bool-ndarray where True signifies a DST time, False designates
      a non-DST time (note that this flag is only applicable for
      ambiguous times)
    - 'NaT' will return NaT where there are ambiguous times
    - 'raise' will raise a ValueError if there are ambiguous
      times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
    A nonexistent time does not exist in a particular timezone
    where clocks moved forward due to DST.

    - 'shift_forward' will shift the nonexistent time forward to the
      closest existing time
    - 'shift_backward' will shift the nonexistent time backward to the
      closest existing time
    - 'NaT' will return NaT where there are nonexistent times
    - timedelta objects will shift nonexistent times by the timedelta
    - 'raise' will raise a ValueError if there are
      nonexistent times.

Returns
-----
DatetimeIndex, TimedeltaIndex, or Series
    Index of the same type for a DatetimeIndex or TimedeltaIndex,
    or a Series with the same index for a Series.

Raises
-----
ValueError if the `freq` cannot be converted.

See Also
-----
DatetimeIndex.floor :
    Perform floor operation on the data to the specified `freq`.
DatetimeIndex.snap :
    Snap time stamps to nearest occurring frequency.

Notes
-----
If the timestamps have a timezone, rounding will take place relative to the
local ("wall") time and re-localized to the same timezone. When rounding
near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
control the re-localization behavior.

Examples
-----
**DatetimeIndex**

>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
              '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')

>>> rng.round('h')
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',
              '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)

**Series**

```

```
>>> pd.Series(rng).dt.round("h")
0    2018-01-01 12:00:00
1    2018-01-01 12:00:00
2    2018-01-01 12:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
take(
    self,
    indices: TakeIndexer,
    *,
    allow_fill: bool = False,
    fill_value: Any = None,
    axis: AxisInt = 0
) -> Self
    Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.

allow_fill : bool, default False
How to handle negative values in ``indices``.

* False: negative values in ``indices`` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in ``indices`` indicate missing values. These values are set to ``fill\_value``. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
Fill value to use for NA-indices when ``allow\_fill`` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of ``fill\_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill\_value`` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray
An array formed with selected ``indices``.

Raises

IndexError
When the indices are out of bounds for the array.

ValueError
When ``indices`` contains negative values other than ``-1`` and ``allow\_fill`` is True.

See Also

numpy.take : Take elements from an array along an axis.
api.extensions.take : Take elements from an array.

Notes

ExtensionArray.take is called by ``Series.\_\_getitem\_\_``, ``.loc``, ``iloc``, when ``indices`` is a sequence of values. Additionally,

it's called by :meth:`Series.reindex`, or any other method that causes realignment, with a `fill\_value`.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method :func:`pandas.api.extensions.take`.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

Data descriptors inherited from pandas.core.arrays.datetimelike.TimelikeOps:

freq

Return the frequency object if it is set, otherwise None.

To learn more about the frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

See Also

DatetimeIndex.freq : Return the frequency object if it is set, otherwise None.
PeriodIndex.freq : Return the frequency object if it is set, otherwise None.

Examples

```
>>> datetimeindex = pd.date_range(
...     "2022-02-22 02:22:22", periods=10, tz="America/Chicago", freq="h"
... )
>>> datetimeindex
DatetimeIndex(['2022-02-22 02:22:22-06:00', '2022-02-22 03:22:22-06:00',
              '2022-02-22 04:22:22-06:00', '2022-02-22 05:22:22-06:00',
              '2022-02-22 06:22:22-06:00', '2022-02-22 07:22:22-06:00',
              '2022-02-22 08:22:22-06:00', '2022-02-22 09:22:22-06:00',
              '2022-02-22 10:22:22-06:00', '2022-02-22 11:22:22-06:00'],
              dtype='datetime64[us, America/Chicago]', freq='h')
>>> datetimeindex.freq
<Hour>
```

unit

The precision unit of the datetime data.

Returns the precision unit for the dtype.
It means the smallest time frame that can be stored within this dtype.

Returns

str

Unit string representation (e.g. "ns").

See Also

TimelikeOps.as_unit : Converts to a specific unit.

Examples

>>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])

```
>>> idx.unit
'ns'
>>> idx.as_unit("s").unit
's'
```

Methods inherited from pandas.core.arrays.datetimelike.DatetimeOps:

`strftime(self, date_format: str) -> npt.NDArray[np.object_]`
Convert to Index using specified date_format.

Return an Index of formatted strings specified by date_format, which supports the same string format as the python standard library. Details of the string format can be found in ``python string format doc <https://docs.python.org/3/library/datetime.html #strftime-and-strptime-behavior>``.

Formats supported by the C ``strftime`` API but not by the python string format doc (such as ``"%R"`, `"%r"``) are not officially supported and should be preferably replaced with their supported equivalents (such as `"%H:%M"`, `"%I:%M:%S %p"``).`

Note that ``PeriodIndex`` support additional directives, detailed in ``Period.strftime``.

Parameters

date_format : str
Date format string (e.g. `"%Y-%m-%d"`).

Returns

ndarray[object]
NumPy ndarray of formatted strings.

See Also

to_datetime : Convert the given argument to datetime.
DatetimeIndex.normalize : Return DatetimeIndex with times to midnight.
DatetimeIndex.round : Round the DatetimeIndex to the specified freq.
DatetimeIndex.floor : Floor the DatetimeIndex to the specified freq.
Timestamp.strftime : Format a single Timestamp.
Period.strftime : Format a single Period.

Examples

>>> rng = pd.date_range(pd.Timestamp("2018-03-10 09:00"), periods=3, freq="s")
>>> rng.strftime("%B %d, %Y, %r")
Index(['March 10, 2018, 09:00:00 AM', 'March 10, 2018, 09:00:01 AM',
 'March 10, 2018, 09:00:02 AM'],
 dtype='str')

Methods inherited from pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin:

`__add__(self, other)`
Get Addition of DataFrame and other, column-wise.

Equivalent to ``Dataframe.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
```

	height	weight
elk	3.0	500.5
moose	4.1	800.5

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
```

	height	weight
elk	NaN	NaN
moose	NaN	NaN

```
>>> df[["height", "weight"]].add(s2, axis="index")
```

	height	weight
elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
```

	height	weight
deer	NaN	NaN
elk	1.7	NaN
moose	3.0	NaN

```
__divmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make
```

```
__floordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mal
```

```
__getitem__(self, key: PositionalIndexer2D) -> Self | DTScalarOrNaT
    This getitem defers to the underlying array, which by-definition can
    only handle list-likes, slices, and integer scalars
```

```
__iadd__(self, other) -> Self
```

```
__init__(self, data, dtype: Dtype | None = None, freq=None, copy: bool = False) -> None
```

```

        Initialize self. See help(type(self)) for accurate signature.

    __isub__(self, other) -> Self

    __mod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in

    __mul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in

    __pow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in

    __radd__(self, other)

    __rdivmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak

    __rfloordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.m

    __rmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_i

    __rmul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_i

    __rpow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_i

    __rsub__(self, other)

    __rtruediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak

    __setitem__(
        self,
        key: int | Sequence[int] | Sequence[bool] | slice,
        value: NaTType | Any | Sequence[Any]
    ) -> None
        Set one or more values inplace.

        This method is not required to satisfy the pandas extension array
        interface.

        Parameters
        -----
        key : int, ndarray, or slice
            When called from, e.g. ``Series.__setitem__``, ``key`` will be
            one of

            * scalar int
            * ndarray of integers.
            * boolean ndarray
            * slice object

        value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
            value or values to be set of ``key``.

        Returns
        -----
        None

        Raises
        -----
        ValueError
            If the array is readonly and modification is attempted.

    __sub__(self, other)

    __truediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak

    isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]
        Compute boolean array of whether each value is found in the
        passed set of values.

        Parameters
        -----
        values : np.ndarray or ExtensionArray

        Returns
        -----
        ndarray[bool]

    isna(self) -> npt.NDArray[np.bool_]
        A 1-D array indicating if each value is missing.

```


Returns

numpy.ndarray or pandas.api.extensions.ExtensionArray
In most cases, this should return a NumPy ndarray. For exceptional cases like ``SparseArray``, where returning an ndarray would be expensive, an ExtensionArray may be returned.

See Also

ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.

Notes

If returning an ExtensionArray, then

- * ``na\_values.\_is\_boolean`` should be True
- * ``na\_values`` should implement :func:`ExtensionArray.\_reduce`
- * ``na\_values`` should implement :func:`ExtensionArray.\_accumulate`
- * ``na\_values.any`` and ``na\_values.all`` should be implemented

Examples

>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False, True, True])

map(self, mapper, na_action: Literal['ignore'] | None = None)
Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}, default None
If 'ignore', propagate NA values, without passing them to the mapping correspondence. If 'ignore' is not supported, a ``NotImplementedError`` should be raised.

Returns

Union[ndarray, Index, ExtensionArray]
The output of the mapping function applied to the array.
If the function returns a tuple with more than one element a MultiIndex will be returned.

max(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)
Return the maximum value of the Array or maximum along an axis.

See Also

numpy.ndarray.max
Index.max : Return the maximum value in an Index.
Series.max : Return the maximum value in a Series.

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0)
Return the mean value of the Array.

Parameters

skipna : bool, default True
Whether to ignore any NaT elements.
axis : int, optional, default 0
Axis for the function to be applied on.

Returns

scalar
Timestamp or Timedelta.

See Also

numpy.ndarray.mean : Returns the average of array elements along a given axis.
Series.mean : Return the mean value in a Series.

Notes

mean is only defined for Datetime and Timedelta dtypes, not for Period.

Examples

For :class:`pandas.DatetimeIndex`:

```
>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.mean()
Timestamp('2001-01-02 00:00:00')
```

For :class:`pandas.TimedeltaIndex`:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
              dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.mean()
Timedelta('2 days 00:00:00')
```

median(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)

min(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)
Return the minimum value of the Array or minimum along an axis.

See Also

numpy.ndarray.min

Index.min : Return the minimum value in an Index.

Series.min : Return the minimum value in a Series.

view(self, dtype: Dtype | None = None) -> ArrayLike
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray

A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.

Index.view: Equivalent function for Index.

ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin:

asi8

Integer representation of the values.

Returns

```

-----
ndarray
    An ndarray with int64 dtype.

freqstr
    Return the frequency object as a string if it's set, otherwise None.

    See Also
    -----
    DatetimeIndex.inferred_freq : Returns a string representing a frequency
        generated by infer_freq.

    Examples
    -----
    For DatetimeIndex:

    >>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
    >>> idx.freqstr
    'D'

    The frequency can be inferred if there are more than 2 points:

    >>> idx = pd.DatetimeIndex(
    ...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
    ... )
    >>> idx.freqstr
    '2D'

    For PeriodIndex:

    >>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
    >>> idx.freqstr
    'M'

inferred_freq
    Tries to return a string representing a frequency generated by infer_freq.

    Returns None if it can't autodetect the frequency.

    See Also
    -----
    DatetimeIndex.freqstr : Return the frequency object as a string if it's set,
        otherwise None.

    Examples
    -----
    For DatetimeIndex:

    >>> idx = pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
    >>> idx.inferred_freq
    '2D'

    For TimedeltaIndex:

    >>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
    >>> tdelta_idx
    TimedeltaIndex(['0 days', '10 days', '20 days'],
                    dtype='timedelta64[us]', freq=None)
    >>> tdelta_idx.inferred_freq
    '10D'

resolution
    Returns day, hour, minute, second, millisecond or microsecond
    -----
    Methods inherited from pandas.core.arraylike.OpsMixin:

    __and__(self, other)

    __eq__(self, other)
        Return self==value.

    __ge__(self, other)
        Return self>=value.

    __gt__(self, other)
        Return self>value.

```

```
__le__(self, other)
    Return self<=value.
```

```
__lt__(self, other)
    Return self<value.
```

```
__ne__(self, other)
    Return self!=value.
```

```
__or__(self, other)
    Return self|value.
```

```
__rand__(self, other)
```

```
__ror__(self, other)
    Return value|self.
```

```
__rxor__(self, other)
```

```
__xor__(self, other)
```

```
-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
```

```
__dict__
    dictionary for instance variables
```

```
__weakref__
    list of weak references to the object
```

```
-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
```

```
__hash__ = None
```

```
-----
Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:
```

```
argmax(self, axis: AxisInt = 0, skipna: bool = True)
    Return the index of maximum value.
```

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the minimum value.

Examples

```
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

```
argmin(self, axis: AxisInt = 0, skipna: bool = True)
    Return the index of minimum value.
```

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

`ExtensionArray.argmax` : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)
```

`equals(self, other) -> bool`

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

`other` : `ExtensionArray`
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

`numpy.array_equal` : Equivalent method for numpy array.

`Series.equals` : Equivalent method for Series.

`DataFrame.equals` : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.

`limit` : int, default None
The maximum number of entries where NA values will be filled.

`copy` : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For `ExtensionArray` subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

`ExtensionArray`
With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return `ExtensionArray` without NA values.

`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
```

```

>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

insert(self, loc: int, item) -> Self
    Make new ExtensionArray inserting new item at location. Follows
    Python list.append semantics for negative values.

    Parameters
    -----
    loc : int
    item : object

    Returns
    -----
    type(self)

searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted array `self` (a) such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    Assuming that `self` is sorted:

    =====
    `side` returned index `i` satisfies
    =====
    left    ``self[i-1] < value <= self[i]``
    right   ``self[i-1] <= value < self[i]``
    =====

    Parameters
    -----
    value : array-like, list or scalar
        Value(s) to insert into `self`.
    side : {'left', 'right'}, optional
        If 'left', the index of the first suitable location found is given.
        If 'right', return the last such index. If there is no suitable
        index, return either 0 or N (where N is the length of `self`).
    sorter : 1-D array-like, optional
        Optional array of integer indices that sort array a into ascending
        order. They are typically the result of argsort.

    Returns
    -----
    array of ints or int
        If value is array-like, array of insertion points.
        If value is scalar, a single integer.

    See Also
    -----
    numpy.searchsorted : Similar method from NumPy.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3, 5])
    >>> arr.searchsorted([4])
    array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
    Shift values by desired number.

    Newly introduced missing values are filled with
    ``self.dtype.na_value``.

    Parameters
    -----
    periods : int, default 1
        The number of periods to shift. Negative values are allowed

```

```

        for shifting backwards.

fill_value : object, optional
    The scalar value to use for newly introduced missing values.
    The default is ``self.dtype.na_value``.

Returns
-----
ExtensionArray
    Shifted.

See Also
-----
api.extensions.ExtensionArray.transpose : Return a transposed view on
    this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
    enumerated type.

Notes
-----
If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
returned.

If ``periods > len(self)`` , then an array of size
len(self) is returned, with all values filled with
``self.dtype.na_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

unique(self) -> Self
    Compute the ExtensionArray of unique values.

Returns
-----
pandas.api.extensions.ExtensionArray
    With unique values from the input array.

See Also
-----
Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples
-----
>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

value_counts(self, dropna: bool = True) -> Series
    Return a Series containing counts of unique values.

Parameters
-----
dropna : bool, default True
    Don't include counts of NA values.

Returns
-----
Series

```

```

Methods inherited from pandas._libs.arrays.NDArrayBacked:

__len__(self, /)
    Return len(self).

```

```

__reduce__ = __reduce_cython__(...)
__reduce_cython__(self)
__setstate__(self, state)
__setstate_cython__(self, __pyx_state)
delete(self, loc, axis=0)
ravel(self, order='C')
repeat(self, repeats, axis: int | np.integer = 0)
reshape(self, *args, **kwargs)
swapaxes(self, axis1, axis2)
transpose(self, *axes)

-----
Static methods inherited from pandas._libs.arrays.NDArrayBacked:
__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
    Create and return a new object.  See help(type) for accurate signature.

-----
Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:
T
nbytes
ndim
shape
size

-----
Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:
__pyx_vtable__ = <capsule object NULL>

-----
Methods inherited from pandas.api.extensions.ExtensionArray:
__contains__(self, item: object) -> bool | np.bool_ from pandas.core.arrays.base.ExtensionArray
    Return for `item in self`.
__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).

argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
    Return the indices that would sort this array.

    Parameters
    -----
    ascending : bool, default True
        Whether the indices should result in an ascending
        or descending sort.
    kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
        Sorting algorithm.
    na_position : {'first', 'last'}, default 'last'
        If ``'first'``, put ``NaN`` values at the beginning.
        If ``'last'``, put ``NaN`` values at the end.
    **kwargs
        Passed through to :func:`numpy.argsort`.

    Returns

```



```
-----
np.ndarray[np.intp]
    Array of indices that sort ``self``. If NaN values are contained,
    NaN values are placed at the end.
```

See Also

```
-----
numpy.argsort : Sorting implementation used internally.
```

Examples

```
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])
```

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all
NA values removed.

See Also

```
-----
Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in
an ExtensionArray.
```

Examples

```
-----
>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64
```

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] f
Return boolean ndarray denoting duplicate values.

Parameters

```
-----
keep : {'first', 'last', False}, default 'first'
- ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
- ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.
```

Returns

ndarray[bool]

With true in indices where elements are duplicated and false otherwise.

See Also

```
-----
DataFrame.duplicated : Return boolean Series denoting
duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray
of unique values.
```

Examples

```
-----
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True,  False,  False,  True])
```

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional

Position of the element. If not provided, the array must contain
exactly one element.

Returns

scalar
The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

IndexError

If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
```

```
>>> arr.item()
```

```
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
```

```
>>> arr.item(0)
```

```
np.int64(1)
```

```
>>> arr.item(2)
```

```
np.int64(3)
```

to_numpy(
self,

dtype: npt.DTypeLike | None = None,
copy: bool = False,

na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray

Convert to a NumPy ndarray.

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

tolist(self) -> list from pandas.core.arrays.base.ExtensionArray

Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list

Python list of values in array.

See Also

Index.tolist: Return a list of the values in the Index.

Series.tolist: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr.tolist()
```

```
[1, 2, 3]
```

```

-----
Data and other attributes inherited from pandas.api.extensions.ExtensionArray:
__pandas_priority__ = 1000

class FloatingArray(pandas.core.arrays.numeric.NumericArray)
    FloatingArray(
        values: np.ndarray,
        mask: npt.NDArray[np.bool_],
        copy: bool = False
    ) -> None

    Array of floating (optional missing) values.

    .. warning::

        FloatingArray is currently experimental, and its API or internal
        implementation may change without warning. Especially the behaviour
        regarding NaN (distinct from NA missing values) is subject to change.

    We represent a FloatingArray with 2 numpy arrays:

    - data: contains a numpy float array of the appropriate dtype
    - mask: a boolean array holding a mask on the data, True is missing

    To construct a FloatingArray from generic array-like input, use
    :func:`pandas.array` with one of the float dtypes (see examples).

    See :ref:`integer_na` for more.

    Parameters
    -----
    values : numpy.ndarray
        A 1-d float-dtype array.
    mask : numpy.ndarray
        A 1-d boolean-dtype array indicating missing values.
    copy : bool, default False
        Whether to copy the `values` and `mask`.

    Attributes
    -----
    None

    Methods
    -----
    None

    Returns
    -----
    FloatingArray

    See Also
    -----
    array : Create an array.
    Float32Dtype : Float32 dtype for FloatingArray.
    Float64Dtype : Float64 dtype for FloatingArray.
    Series : One-dimensional labeled array capable of holding data.
    DataFrame : Two-dimensional, size-mutable, potentially heterogeneous tabular data.

    Examples
    -----
    Create a FloatingArray with :func:`pandas.array`:

    >>> pd.array([0.1, None, 0.3], dtype=pd.Float32Dtype())
    <FloatingArray>
    [0.1, <NA>, 0.3]
    Length: 3, dtype: Float32

    String aliases for the dtypes are also available. They are capitalized.

    >>> pd.array([0.1, None, 0.3], dtype="Float32")
    <FloatingArray>
    [0.1, <NA>, 0.3]
    Length: 3, dtype: Float32

    Method resolution order:

```

```
FloatingArray
pandas.core.arrays.numeric.NumericArray
pandas.core.arrays.masked.BaseMaskedArray
pandas.core.arraylike.OpsMixin
pandas.api.extensions.ExtensionArray
builtins.object
```

Methods inherited from pandas.core.arrays.numeric.NumericArray:

```
__init__(
    self,
    values: np.ndarray,
    mask: npt.NDArray[np.bool_],
    copy: bool = False
) -> None
    Initialize self. See help(type(self)) for accurate signature.
```

Data descriptors inherited from pandas.core.arrays.numeric.NumericArray:

dtype

Data and other attributes inherited from pandas.core.arrays.numeric.NumericArray:

```
__annotations__ = {}
```

Methods inherited from pandas.core.arrays.masked.BaseMaskedArray:

```
__abs__(self) -> Self
```

```
__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray
    the array interface, return my values
    We return an object array here to preserve our scalar values
```

```
__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs)
```

```
__arrow_array__(self, type=None)
    Convert myself into a pyarrow Array.
```

```
__contains__(self, key) -> bool
    Return for `item in self`.
```

```
__getitem__(self, item: PositionalIndexer) -> Self | Any
    Select a subset of self.
```

Parameters

item : int, slice, or ndarray

* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

```
__invert__(self) -> Self
```

```
__iter__(self) -> Iterator
```

Iterate over elements of the array.

`__len__(self) -> int`
Length of this array

Returns

length : int

`__neg__(self) -> Self`

`__pos__(self) -> Self`

`__setitem__(self, key, value) -> None`
Set one or more values inplace.

This method is not required to satisfy the pandas extension array interface.

Parameters

key : int, ndarray, or slice
When called from, e.g. `Series.__setitem__`, `key` will be one of

- * scalar int
- * ndarray of integers.
- * boolean ndarray
- * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
value or values to be set of `key`.

Returns

None

Raises

ValueError

If the array is readonly and modification is attempted.

`all(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType`
Return whether all elements are truthy.

Returns True unless there is at least one element that is falsey.
By default, NAs are skipped. If `skipna=False` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

skipna : bool, default True
Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be True, as for an empty array. If `skipna` is False, the result will still be False if there is at least one element that is falsey, otherwise NA will be returned if there are NA's present.
axis : int, optional, default 0
**kwargs : any, default None
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

bool or :attr:`pandas.NA`

See Also

numpy.all : Numpy version of this method.
BooleanArray.any : Return whether any element is truthy.

Examples

The result indicates whether all elements are truthy (and by default skips NAs):

```
>>> pd.array([True, True, pd.NA]).all()
np.True_
>>> pd.array([1, 1, pd.NA]).all()
np.True_
>>> pd.array([True, False, pd.NA]).all()
np.False_
>>> pd.array([], dtype="boolean").all()
np.True_
>>> pd.array([pd.NA], dtype="boolean").all()
np.True_
>>> pd.array([pd.NA], dtype="Float64").all()
np.True_
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, True, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([1, 1, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([True, False, pd.NA]).all(skipna=False)
np.False_
>>> pd.array([1, 0, pd.NA]).all(skipna=False)
np.False_
```

`any(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType`
Return whether any element is truthy.

Returns False unless there is at least one element that is truthy. By default, NAs are skipped. If ``skipna=False`` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

skipna : bool, default True
 Exclude NA values. If the entire array is NA and ``skipna`` is True, then the result will be False, as for an empty array. If ``skipna`` is False, the result will still be True if there is at least one element that is truthy, otherwise NA will be returned if there are NA's present.
axis : int, optional, default 0
****kwargs** : any, default None
 Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

 bool or :attr:`pandas.NA`

See Also

numpy.any : Numpy version of this method.
BaseMaskedArray.all : Return whether all elements are truthy.

Examples

 The result indicates whether any element is truthy (and by default skips NAs):

```
>>> pd.array([True, False, True]).any()
np.True_
>>> pd.array([True, False, pd.NA]).any()
np.True_
>>> pd.array([False, False, pd.NA]).any()
np.False_
>>> pd.array([], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="Float64").any()
np.False_
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, False, pd.NA]).any(skipna=False)
```

```

np.True_
>>> pd.array([1, 0, pd.NA]).any(skipna=False)
np.True_
>>> pd.array([False, False, pd.NA]).any(skipna=False)
<NA>
>>> pd.array([0, 0, pd.NA]).any(skipna=False)
<NA>

astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike
Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters
-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also
-----
Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
a sequence.

Examples
-----
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()

Otherwise, we will get a Numpy ndarray:

>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')

copy(self) -> Self
Return a copy of the array.

This method creates a copy of the ``ExtensionArray`` where modifying the
data in the copy will not affect the original array. This is useful when
you want to manipulate data without altering the original dataset.

Returns
-----
ExtensionArray
    A new ``ExtensionArray`` object that is a copy of the current instance.

See Also
-----
DataFrame.copy : Return a copy of the DataFrame.

```

Series.copy : Return a copy of the Series.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

delete(self, loc, axis: AxisInt = 0) -> Self

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]
Return boolean ndarray denoting duplicate values.

Parameters

```
-----
keep : {'first', 'last', False}, default 'first'
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.
```

Returns

```
-----
ndarray[bool]
    With true in indices where elements are duplicated and false otherwise.
```

See Also

```
-----
DataFrame.duplicated : Return boolean Series denoting
    duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray
    of unique values.
```

Examples

```
-----
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])
```

equals(self, other) -> bool

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

```
-----
other : ExtensionArray
    Array to compare to this Array.
```

Returns

```
-----
boolean
    Whether the arrays are equivalent.
```

See Also

```
-----
numpy.array_equal : Equivalent method for numpy array.
Series.equals : Equivalent method for Series.
DataFrame.equals : Equivalent method for DataFrame.
```

Examples

```
-----
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True

>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```



```
factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray]
    Encode the extension array as an enumerated type.
```

Parameters

`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.

`uniques` : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

`factorize` : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M'])
```

```
fillna(self, value, limit: int | None = None, copy: bool = True) -> Self
    Fill NA/NaN values using the specified method.
```

Parameters

`value` : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.

`limit` : int, default None
The maximum number of entries where NA values will be filled.

`copy` : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.

`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
```

```

<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> FloatingArray
    See NDFrame.interpolate.__doc__.

isin(self, values: ArrayLike) -> BooleanArray
    Pointwise comparison for set containment in the given values.

    Roughly equivalent to `np.array([x in values for x in self])`

    Parameters
    -----
    values : np.ndarray or ExtensionArray
        Values to compare every element in the array against.

    Returns
    -----
    np.ndarray[bool]
        With true at indices where value is in `values`.

    See Also
    -----
    DataFrame.isin: Whether each element in the DataFrame is contained in values.
    Index.isin: Return a boolean array where the index values are in values.
    Series.isin: Whether elements in Series are contained in values.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.isin([1])
    <BooleanArray>
    [True, False, False]
    Length: 3, dtype: boolean

isna(self) -> np.ndarray
    A 1-D array indicating if each value is missing.

    Returns
    -----
    numpy.ndarray or pandas.api.extensions.ExtensionArray
        In most cases, this should return a NumPy ndarray. For
        exceptional cases like ``SparseArray``, where returning
        an ndarray would be expensive, an ExtensionArray may be
        returned.

    See Also
    -----
    ExtensionArray.dropna: Return ExtensionArray without NA values.
    ExtensionArray.fillna: Fill NA/NaN values using the specified method.

    Notes
    -----
    If returning an ExtensionArray, then

    * ``na_values._is_boolean`` should be True
    * ``na_values`` should implement :func:`ExtensionArray._reduce`
    * ``na_values`` should implement :func:`ExtensionArray._accumulate`
    * ``na_values.any`` and ``na_values.all`` should be implemented

    Examples
    -----
    >>> arr = pd.array([1, 2, np.nan, np.nan])
    >>> arr.isna()
    array([False, False,  True,  True])

```

```

map(self, mapper, na_action: Literal['ignore'] | None = None)
    Map values using an input mapping or function.

    Parameters
    -----
    mapper : function, dict, or Series
        Mapping correspondence.
    na_action : {None, 'ignore'}, default None
        If 'ignore', propagate NA values, without passing them to the
        mapping correspondence. If 'ignore' is not supported, a
        ``NotImplementedError`` should be raised.

    Returns
    -----
    Union[ndarray, Index, ExtensionArray]
        The output of the mapping function applied to the array.
        If the function returns a tuple with more than one element
        a MultiIndex will be returned.

max(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

min(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

prod(
    self,
    *,
    skipna: bool = True,
    min_count: int = 0,
    axis: AxisInt | None = 0,
    **kwargs
)

ravel(self, *args, **kwargs) -> Self
    Return a flattened view on this array.

    Parameters
    -----
    order : {None, 'C', 'F', 'A', 'K'}, default 'C'

    Returns
    -----
    ExtensionArray
        A flattened view on the array.

    See Also
    -----
    ExtensionArray.tolist: Return a list of the values.

    Notes
    -----
    - Because ExtensionArrays are 1D-only, this is a no-op.
    - The "order" argument is ignored, is for compatibility with NumPy.

    Examples
    -----
    >>> pd.array([1, 2, 3]).ravel()
    <IntegerArray>
    [1, 2, 3]
    Length: 3, dtype: Int64

reshape(self, *args, **kwargs) -> Self

round(self, decimals: int = 0, *args, **kwargs)
    Round each value in the array a to the given number of decimals.

    Parameters
    -----
    decimals : int, default 0
        Number of decimal places to round to. If decimals is negative,
        it specifies the number of positions to the left of the decimal point.
    *args, **kwargs
        Additional arguments and keywords have no effect but might be
        accepted for compatibility with NumPy.

```

Returns

NumericArray

Rounded values of the NumericArray.

See Also

numpy.around : Round values of an np.array.

DataFrame.round : Round values of a DataFrame.

Series.round : Round values of a Series.

```
searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
    Find indices where elements should be inserted to maintain order.

    Find the indices into a sorted array `self` (a) such that, if the
    corresponding elements in `value` were inserted before the indices,
    the order of `self` would be preserved.

    Assuming that `self` is sorted:

    =====
    `side` returned index `i` satisfies
    =====
    left    ``self[i-1] < value <= self[i]``
    right   ``self[i-1] <= value < self[i]``
    =====

Parameters
-----
value : array-like, list or scalar
    Value(s) to insert into `self`.
side : {'left', 'right'}, optional
    If 'left', the index of the first suitable location found is given.
    If 'right', return the last such index. If there is no suitable
    index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
    Optional array of integer indices that sort array a into ascending
    order. They are typically the result of argsort.

Returns
-----
array of ints or int
    If value is array-like, array of insertion points.
    If value is scalar, a single integer.

See Also
-----
numpy.searchsorted : Similar method from NumPy.

Examples
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])
```

```
shift(self, periods: int = 1, fill_value=None) -> Self
    Shift values by desired number.
```

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1

The number of periods to shift. Negative values are allowed
for shifting backwards.

fill_value : object, optional

The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`` , then an array of size len(self) is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

```
std(  
    self,  
    *,  
    skipna: bool = True,  
    axis: AxisInt | None = 0,  
    ddof: int = 1,  
    **kwargs  
)  
  
sum(  
    self,  
    *,  
    skipna: bool = True,  
    min_count: int = 0,  
    axis: AxisInt | None = 0,  
    **kwargs  
)  
  
swapaxes(self, axis1, axis2) -> Self  
  
take(  
    self,  
    indexer,  
    *,  
    allow_fill: bool = False,  
    fill_value: Scalar | None = None,  
    axis: AxisInt = 0  
) -> Self  
Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.

allow_fill : bool, default False
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in `indices` indicate missing values. These values are set to `fill\_value`. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
Fill value to use for NA-indices when `allow\_fill` is True.

This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of ``fill\_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill\_value`` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray

An array formed with selected ``indices``.

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When ``indices`` contains negative values other than ``-1`` and ``allow\_fill`` is True.

See Also

numpy.take : Take elements from an array along an axis.

api.extensions.take : Take elements from an array.

Notes

ExtensionArray.take is called by ``Series.\_\_getitem\_\_``, ``.loc``, ``.iloc``, when ``indices`` is a sequence of values. Additionally, it's called by :meth:`Series.reindex`, or any other method that causes realignment, with a ``fill\_value``.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method :func:`pandas.api.extensions.take`.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray
Convert to a NumPy Array.
```

By default converts to an object-dtype NumPy array. Specify the ``dtype`` and ``na\_value`` keywords to customize the conversion.

Parameters

dtype : dtype, default object

The numpy dtype to convert to.

copy : bool, default False

Whether to ensure that the returned value is a not a view on the array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary. This is typically only possible when no missing values are present and `dtype` is the equivalent numpy dtype.

na_value : scalar, optional
 Scalar missing value indicator to use in numpy array. Defaults to the native missing value indicator of this array (pd.NA).

Returns

 numpy.ndarray

Examples

 An object-dtype is the default result

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a.to_numpy()
array([True, False, <NA>], dtype=object)
```

When no missing values are present, an equivalent dtype can be used.

```
>>> pd.array([True, False], dtype="boolean").to_numpy(dtype="bool")
array([ True, False])
>>> pd.array([1, 2], dtype="Int64").to_numpy("int64")
array([1, 2])
```

However, requesting such dtype will raise a ValueError if missing values are present and the default missing value :attr:`NA` is used.

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a
<BooleanArray>
[True, False, <NA>]
Length: 3, dtype: boolean
```

```
>>> a.to_numpy(dtype="bool")
Traceback (most recent call last):
```

```
...
ValueError: cannot convert to bool numpy array in presence of missing values
```

Specify a valid `na\_value` instead

```
>>> a.to_numpy(dtype="bool", na_value=False)
array([ True, False, False])
```

tolist(self) -> list
 Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

 list
 Python list of values in array.

See Also

 Index.to_list: Return a list of the values in the Index.
 Series.to_list: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

unique(self) -> Self
 Compute the BaseMaskedArray of unique values.

Returns

```

        uniques : BaseMaskedArray

value_counts(self, dropna: bool = True) -> Series
    Returns a Series containing counts of each unique value.

    Parameters
    -----
    dropna : bool, default True
        Don't include counts of missing values.

    Returns
    -----
    counts : Series

    See Also
    -----
    Series.value_counts

var(
    self,
    *,
    skipna: bool = True,
    axis: AxisInt | None = 0,
    ddof: int = 1,
    **kwargs
)

-----
Readonly properties inherited from pandas.core.arrays.masked.BaseMaskedArray:
T

nbytes
    The number of bytes needed to store this object in memory.

    See Also
    -----
    ExtensionArray.shape: Return a tuple of the array dimensions.
    ExtensionArray.size: The number of elements in the array.

    Examples
    -----
    >>> pd.array([1, 2, 3]).nbytes
    27

ndim
    Extension Arrays are only allowed to be 1-dimensional.

    See Also
    -----
    ExtensionArray.shape: Return a tuple of the array dimensions.
    ExtensionArray.size: The number of elements in the array.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.ndim
    1

shape
    Return a tuple of the array dimensions.

    See Also
    -----
    numpy.ndarray.shape : Similar attribute which returns the shape of an array.
    DataFrame.shape : Return a tuple representing the dimensionality of the
        DataFrame.
    Series.shape : Return a tuple representing the dimensionality of the Series.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.shape
    (3,)

-----
Data and other attributes inherited from pandas.core.arrays.masked.BaseMaskedArray:

```



```
__array_priority__ = 1000
```

```
-----  
Methods inherited from pandas.core.arraylike.OpsMixin:
```

```
__add__(self, other)
```

```
Get Addition of DataFrame and other, column-wise.
```

```
Equivalent to ``DataFrame.add(other)``.
```

```
Parameters
```

```
-----  
other : scalar, sequence, Series, dict or DataFrame  
Object to be added to the DataFrame.
```

```
Returns
```

```
-----  
DataFrame
```

```
The result of adding ``other`` to DataFrame.
```

```
See Also
```

```
-----  
DataFrame.add : Add a DataFrame and another object, with option for index-  
or column-oriented addition.
```

```
Examples
```

```
-----  
>>> df = pd.DataFrame(  
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]  
... )  
>>> df  
      height  weight  
elk        1.5    500  
moose       2.6    800
```

```
Adding a scalar affects all rows and columns.
```

```
>>> df[["height", "weight"]] + 1.5  
      height  weight  
elk        3.0   501.5  
moose       4.1   801.5
```

```
Each element of a list is added to a column of the DataFrame, in order.
```

```
>>> df[["height", "weight"]] + [0.5, 1.5]  
      height  weight  
elk        2.0   501.5  
moose       3.1   801.5
```

```
Keys of a dictionary are aligned to the DataFrame, based on column names;  
each value in the dictionary is added to the corresponding column.
```

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}  
      height  weight  
elk        2.0   501.5  
moose       3.1   801.5
```

```
When `other` is a :class:`Series`, the index of `other` is aligned with the  
columns of the DataFrame.
```

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])  
>>> df[["height", "weight"]] + s1  
      height  weight  
elk        3.0   500.5  
moose       4.1   800.5
```

```
Even when the index of `other` is the same as the index of the DataFrame,  
the :class:`Series` will not be reoriented. If index-wise alignment is desired,  
:meth:`DataFrame.add` should be used with `axis='index'`.
```

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])  
>>> df[["height", "weight"]] + s2  
      elk  height  moose  weight  
elk    NaN     NaN    NaN     NaN  
moose  NaN     NaN    NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose       4.1    801.5
```

When `other` is a :class:`DataFrame`, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer       NaN     NaN
elk         1.7     NaN
moose       3.0     NaN
```

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
```

```

__xor__(self, other)
-----
Data descriptors inherited from pandas.core.arraylike.OpsMixin:
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
__hash__ = None
-----
Methods inherited from pandas.api.extensions.ExtensionArray:
__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).
argmax(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
    Return the index of maximum value.

    In case of multiple occurrences of the maximum value, the index
    corresponding to the first occurrence is returned.

    Parameters
    -----
    skipna : bool, default True

    Returns
    -----
    int

    See Also
    -----
    ExtensionArray.argmax : Return the index of the minimum value.

    Examples
    -----
    >>> arr = pd.array([3, 1, 2, 5, 4])
    >>> arr.argmax()
    np.int64(3)
argmin(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
    Return the index of minimum value.

    In case of multiple occurrences of the minimum value, the index
    corresponding to the first occurrence is returned.

    Parameters
    -----
    skipna : bool, default True

    Returns
    -----
    int

    See Also
    -----
    ExtensionArray.argmax : Return the index of the maximum value.

    Examples
    -----
    >>> arr = pd.array([3, 1, 2, 5, 4])
    >>> arr.argmin()
    np.int64(1)
argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs

```

) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Return the indices that would sort this array.

Parameters

ascending : bool, default True
Whether the indices should result in an ascending
or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
If 'first', put 'NaN' values at the beginning.
If 'last', put 'NaN' values at the end.
**kwargs
Passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]
Array of indices that sort 'self'. If NaN values are contained,
NaN values are placed at the end.

See Also

numpy.argsort : Sorting implementation used internally.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])
```

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all
NA values removed.

See Also

Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in
an ExtensionArray.

Examples

```
>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64
```

insert(self, loc: int, item) -> Self from pandas.core.arrays.base.ExtensionArray
Insert an item at the given position.

Parameters

loc : int
Index where the 'item' needs to be inserted.
item : scalar-like
Value to be inserted.

Returns

ExtensionArray
With 'item' inserted at 'loc'.

See Also

Index.insert: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item

cannot be held in an array of this type/dtype, either ValueError or TypeError should be raised.

The default implementation relies on `_from_sequence` to raise on invalid items.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)
<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64
```

`item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray`
Return the array element at the specified position as a Python scalar.

Parameters

`index` : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

`scalar`
The element at the specified position.

Raises

`ValueError`
If no index is provided and the array does not have exactly one element.
`IndexError`
If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
-----
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

`repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self from pandas.`
Repeat elements of an ExtensionArray.

Returns a new ExtensionArray where each element of the current ExtensionArray is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty ExtensionArray.
`axis` : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

`ExtensionArray`
Newly created ExtensionArray with repeated elements.

See Also

`Series.repeat` : Equivalent function for Series.
`Index.repeat` : Equivalent function for Index.

numpy.repeat : Similar method for :class:`numpy.ndarray`.
ExtensionArray.take : Take arbitrary positions.

Examples

```
-----
>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
```

transpose(self, *axes: int) -> Self from pandas.core.arrays.base.ExtensionArray
Return a transposed view on this array.

Because ExtensionArrays are always 1D, this is a no-op. It is included for compatibility with np.ndarray.

Returns

ExtensionArray

Examples

```
-----
>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

view(self, dtype: Dtype | None = None) -> ArrayLike from pandas.core.arrays.base.ExtensionArray
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.api.extensions.ExtensionArray:

size

The number of elements in the array.

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

__pandas_priority__ = 1000

class IntegerArray(pandas.core.arrays.numeric.NumericArray)

```
IntegerArray(  
    values: np.ndarray,  
    mask: npt.NDArray[np.bool_],  
    copy: bool = False  
) -> None
```

Array of integer (optional missing) values.

Uses :attr:`pandas.NA` as the missing value.

.. warning::

IntegerArray is currently experimental, and its API or internal implementation may change without warning.

We represent an IntegerArray with 2 numpy arrays:

- data: contains a numpy integer array of the appropriate dtype
- mask: a boolean array holding a mask on the data, True is missing

To construct an IntegerArray from generic array-like input, use :func:`pandas.array` with one of the integer dtypes (see examples).

See :ref:`integer\_na` for more.

Parameters

values : numpy.ndarray
 A 1-d integer-dtype array.
mask : numpy.ndarray
 A 1-d boolean-dtype array indicating missing values.
copy : bool, default False
 Whether to copy the `values` and `mask`.

Attributes

None

Methods

None

Returns

IntegerArray

See Also

array : Create an array using the appropriate dtype, including ``IntegerArray``.
Int32Dtype : An ExtensionDtype for int32 integer data.
UInt16Dtype : An ExtensionDtype for uint16 integer data.

Examples

Create an IntegerArray with :func:`pandas.array`.

```
>>> int_array = pd.array([1, None, 3], dtype=pd.Int32Dtype())  
>>> int_array  
<IntegerArray>  
[1, <NA>, 3]  
Length: 3, dtype: Int32
```

String aliases for the dtypes are also available. They are capitalized.

```
>>> pd.array([1, None, 3], dtype="Int32")  
<IntegerArray>  
[1, <NA>, 3]  
Length: 3, dtype: Int32
```

```
>>> pd.array([1, None, 3], dtype="UInt16")  
<IntegerArray>  
[1, <NA>, 3]  
Length: 3, dtype: UInt16
```

Method resolution order:

IntegerArray
pandas.core.arrays.numeric.NumericArray

```
pandas.core.arrays.masked.BaseMaskedArray
pandas.core.arraylike.OpsMixin
pandas.api.extensions.ExtensionArray
builtins.object
```

Methods inherited from pandas.core.arrays.numeric.NumericArray:

```
__init__(
    self,
    values: np.ndarray,
    mask: npt.NDArray[np.bool_],
    copy: bool = False
) -> None
    Initialize self. See help(type(self)) for accurate signature.
```

Data descriptors inherited from pandas.core.arrays.numeric.NumericArray:

dtype

Data and other attributes inherited from pandas.core.arrays.numeric.NumericArray:

```
__annotations__ = {}
```

Methods inherited from pandas.core.arrays.masked.BaseMaskedArray:

```
__abs__(self) -> Self

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray
    the array interface, return my values
    We return an object array here to preserve our scalar values

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs)

__arrow_array__(self, type=None)
    Convert myself into a pyarrow Array.

__contains__(self, key) -> bool
    Return for `item in self`.

__getitem__(self, item: PositionalIndexer) -> Self | Any
    Select a subset of self.
```

Parameters

```
-----
item : int, slice, or ndarray
    * int: The position in 'self' to get.

    * slice: A slice object, where 'start', 'stop', and 'step' are
      integers or None

    * ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

    * list[int]: A list of int
```

Returns

```
-----
item : scalar or ExtensionArray
```

Notes

```
-----
For scalar ``item``, return a scalar value suitable for the array's
type. This should be an instance of ``self.dtype.type``.
```

```
For slice ``key``, return an instance of ``ExtensionArray``, even
if the slice is length 0 or 1.
```

```
For a boolean mask, return an instance of ``ExtensionArray``, filtered
to the values where ``item`` is True.
```

```
__invert__(self) -> Self
```

```
__iter__(self) -> Iterator
    Iterate over elements of the array.
```



```

__len__(self) -> int
    Length of this array

    Returns
    -----
    length : int

__neg__(self) -> Self

__pos__(self) -> Self

__setitem__(self, key, value) -> None
    Set one or more values inplace.

    This method is not required to satisfy the pandas extension array
    interface.

    Parameters
    -----
    key : int, ndarray, or slice
        When called from, e.g. ``Series.__setitem__``, ``key`` will be
        one of

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

    value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
        value or values to be set of ``key``.

    Returns
    -----
    None

    Raises
    -----
    ValueError
        If the array is readonly and modification is attempted.

all(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType
    Return whether all elements are truthy.

    Returns True unless there is at least one element that is falsey.
    By default, NAs are skipped. If ``skipna=False`` is specified and
    missing values are present, similar :ref:`Kleene logic <boolean.kleene>`
    is used as for logical operations.

    Parameters
    -----
    skipna : bool, default True
        Exclude NA values. If the entire array is NA and ``skipna`` is
        True, then the result will be True, as for an empty array.
        If ``skipna`` is False, the result will still be False if there is
        at least one element that is falsey, otherwise NA will be returned
        if there are NA's present.
    axis : int, optional, default 0
    **kwargs : any, default None
        Additional keywords have no effect but might be accepted for
        compatibility with NumPy.

    Returns
    -----
    bool or :attr:`pandas.NA`

    See Also
    -----
    numpy.all : Numpy version of this method.
    BooleanArray.any : Return whether any element is truthy.

    Examples
    -----
    The result indicates whether all elements are truthy (and by default
    skips NAs):

    >>> pd.array([True, True, pd.NA]).all()
    np.True_

```

```
>>> pd.array([1, 1, pd.NA]).all()
np.True_
>>> pd.array([True, False, pd.NA]).all()
np.False_
>>> pd.array([], dtype="boolean").all()
np.True_
>>> pd.array([pd.NA], dtype="boolean").all()
np.True_
>>> pd.array([pd.NA], dtype="Float64").all()
np.True_
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, True, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([1, 1, pd.NA]).all(skipna=False)
<NA>
>>> pd.array([True, False, pd.NA]).all(skipna=False)
np.False_
>>> pd.array([1, 0, pd.NA]).all(skipna=False)
np.False_
```

```
any(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs) -> np.bool_ | NAType
Return whether any element is truthy.
```

Returns False unless there is at least one element that is truthy. By default, NAs are skipped. If ``skipna=False`` is specified and missing values are present, similar :ref:`Kleene logic <boolean.kleene>` is used as for logical operations.

Parameters

```
skipna : bool, default True
    Exclude NA values. If the entire array is NA and `skipna` is True, then the result will be False, as for an empty array. If `skipna` is False, the result will still be True if there is at least one element that is truthy, otherwise NA will be returned if there are NA's present.
axis : int, optional, default 0
**kwargs : any, default None
    Additional keywords have no effect but might be accepted for compatibility with NumPy.
```

Returns

```
bool or :attr:`pandas.NA`
```

See Also

```
numpy.any : Numpy version of this method.
BaseMaskedArray.all : Return whether all elements are truthy.
```

Examples

The result indicates whether any element is truthy (and by default skips NAs):

```
>>> pd.array([True, False, True]).any()
np.True_
>>> pd.array([True, False, pd.NA]).any()
np.True_
>>> pd.array([False, False, pd.NA]).any()
np.False_
>>> pd.array([], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="boolean").any()
np.False_
>>> pd.array([pd.NA], dtype="Float64").any()
np.False_
```

With ``skipna=False``, the result can be NA if this is logically required (whether ``pd.NA`` is True or False influences the result):

```
>>> pd.array([True, False, pd.NA]).any(skipna=False)
np.True_
>>> pd.array([1, 0, pd.NA]).any(skipna=False)
```

```

np.True_
>>> pd.array([False, False, pd.NA]).any(skipna=False)
<NA>
>>> pd.array([0, 0, pd.NA]).any(skipna=False)
<NA>

```

astype(self, dtype: AstypeArg, copy: bool = True) -> ArrayLike
Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters

```

dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

```

Returns

```

np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

```

See Also

```

Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
a sequence.

```

Examples

```

>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

```

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

```

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()

```

Otherwise, we will get a Numpy ndarray:

```

>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')

```

copy(self) -> Self
Return a copy of the array.

This method creates a copy of the ``ExtensionArray`` where modifying the data in the copy will not affect the original array. This is useful when you want to manipulate data without altering the original dataset.

Returns

```

ExtensionArray
    A new ``ExtensionArray`` object that is a copy of the current instance.

```

See Also

```

DataFrame.copy : Return a copy of the DataFrame.
Series.copy : Return a copy of the Series.

```

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

`delete(self, loc, axis: AxisInt = 0) -> Self`

`duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]`
Return boolean ndarray denoting duplicate values.

Parameters

```
-----
keep : {'first', 'last', False}, default 'first'
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.
```

Returns

```
-----
ndarray[bool]
```

With true in indices where elements are duplicated and false otherwise.

See Also

```
-----
DataFrame.duplicated : Return boolean Series denoting
duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray
of unique values.
```

Examples

```
-----
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])
```

`equals(self, other) -> bool`

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

```
-----
other : ExtensionArray
Array to compare to this Array.
```

Returns

```
-----
boolean
Whether the arrays are equivalent.
```

See Also

```
-----
numpy.array_equal : Equivalent method for numpy array.
Series.equals : Equivalent method for Series.
DataFrame.equals : Equivalent method for DataFrame.
```

Examples

```
-----
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True

>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

`factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray]`
Encode the extension array as an enumerated type.

Parameters

`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
`uniques` : ExtensionArray
An ExtensionArray containing the unique values of ``self``.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in ``self``.

See Also

`factorize` : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a ``sort`` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M'])
```

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.
`limit` : int, default None
The maximum number of entries where NA values will be filled.
`copy` : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.
`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
```

```

        Length: 6, dtype: Int64

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> FloatingArray
    See NDFrame.interpolate.__doc__.

isin(self, values: ArrayLike) -> BooleanArray
    Pointwise comparison for set containment in the given values.

    Roughly equivalent to `np.array([x in values for x in self])`

    Parameters
    -----
    values : np.ndarray or ExtensionArray
        Values to compare every element in the array against.

    Returns
    -----
    np.ndarray[bool]
        With true at indices where value is in `values`.

    See Also
    -----
    DataFrame.isin: Whether each element in the DataFrame is contained in values.
    Index.isin: Return a boolean array where the index values are in values.
    Series.isin: Whether elements in Series are contained in values.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.isin([1])
    <BooleanArray>
    [True, False, False]
    Length: 3, dtype: boolean

isna(self) -> np.ndarray
    A 1-D array indicating if each value is missing.

    Returns
    -----
    numpy.ndarray or pandas.api.extensions.ExtensionArray
        In most cases, this should return a NumPy ndarray. For
        exceptional cases like ``SparseArray``, where returning
        an ndarray would be expensive, an ExtensionArray may be
        returned.

    See Also
    -----
    ExtensionArray.dropna: Return ExtensionArray without NA values.
    ExtensionArray.fillna: Fill NA/NaN values using the specified method.

    Notes
    -----
    If returning an ExtensionArray, then

    * ``na_values._is_boolean`` should be True
    * ``na_values`` should implement :func:`ExtensionArray._reduce`
    * ``na_values`` should implement :func:`ExtensionArray._accumulate`
    * ``na_values.any`` and ``na_values.all`` should be implemented

    Examples
    -----
    >>> arr = pd.array([1, 2, np.nan, np.nan])
    >>> arr.isna()
    array([False, False,  True,  True])

    map(self, mapper, na_action: Literal['ignore'] | None = None)

```

Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}, default None
If 'ignore', propagate NA values, without passing them to the
mapping correspondence. If 'ignore' is not supported, a
``NotImplementedError`` should be raised.

Returns

Union[ndarray, Index, ExtensionArray]
The output of the mapping function applied to the array.
If the function returns a tuple with more than one element
a MultiIndex will be returned.

max(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

min(self, *, skipna: bool = True, axis: AxisInt | None = 0, **kwargs)

prod(
 self,
 *,
 skipna: bool = True,
 min_count: int = 0,
 axis: AxisInt | None = 0,
 **kwargs
)

ravel(self, *args, **kwargs) -> Self
Return a flattened view on this array.

Parameters

order : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

ExtensionArray
A flattened view on the array.

See Also

ExtensionArray.tolist: Return a list of the values.

Notes

- Because ExtensionArrays are 1D-only, this is a no-op.
- The "order" argument is ignored, is for compatibility with NumPy.

Examples

>>> pd.array([1, 2, 3]).ravel()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

reshape(self, *args, **kwargs) -> Self

round(self, decimals: int = 0, *args, **kwargs)
Round each value in the array a to the given number of decimals.

Parameters

decimals : int, default 0
Number of decimal places to round to. If decimals is negative,
it specifies the number of positions to the left of the decimal point.
*args, **kwargs
Additional arguments and keywords have no effect but might be
accepted for compatibility with NumPy.

Returns

NumericArray
Rounded values of the NumericArray.

See Also

numpy.around : Round values of an np.array.
DataFrame.round : Round values of a DataFrame.
Series.round : Round values of a Series.

searchsorted(
self,
value: NumpyValueArrayLike | ExtensionArray,
side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the
corresponding elements in `value` were inserted before the indices,
the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into `self`.
side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable
index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending
order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1
The number of periods to shift. Negative values are allowed
for shifting backwards.

fill_value : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray

Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`` , then an array of size len(self) is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

```
std(  
    self,  
    *,  
    skipna: bool = True,  
    axis: AxisInt | None = 0,  
    ddof: int = 1,  
    **kwargs  
)  
  
sum(  
    self,  
    *,  
    skipna: bool = True,  
    min_count: int = 0,  
    axis: AxisInt | None = 0,  
    **kwargs  
)  
  
swapaxes(self, axis1, axis2) -> Self  
  
take(  
    self,  
    indexer,  
    *,  
    allow_fill: bool = False,  
    fill_value: Scalar | None = None,  
    axis: AxisInt = 0  
) -> Self  
    Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
 Indices to be taken.
allow_fill : bool, default False
 How to handle negative values in ``indices``.

 * False: negative values in ``indices`` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

 * True: negative values in ``indices`` indicate missing values. These values are set to ``fill\_value``. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
 Fill value to use for NA-indices when ``allow\_fill`` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of ``fill_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill_value`` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray

An array formed with selected ``indices``.

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When ``indices`` contains negative values other than ``-1`` and ``allow_fill`` is True.

See Also

`numpy.take` : Take elements from an array along an axis.

`api.extensions.take` : Take elements from an array.

Notes

`ExtensionArray.take` is called by ``Series.__getitem__``, ``loc``, ``iloc``, when ``indices`` is a sequence of values. Additionally, it's called by `:meth:`Series.reindex``, or any other method that causes realignment, with a ``fill_value``.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method `:func:`pandas.api.extensions.take``.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray
Convert to a NumPy Array.
```

By default converts to an object-dtype NumPy array. Specify the ``dtype`` and ``na_value`` keywords to customize the conversion.

Parameters

`dtype` : dtype, default object

The numpy dtype to convert to.

`copy` : bool, default False

Whether to ensure that the returned value is a not a view on the array. Note that ``copy=False`` does not *ensure* that

`to_numpy()` is no-copy. Rather, `copy=True` ensure that a copy is made, even if not strictly necessary. This is typically only possible when no missing values are present and `dtype` is the equivalent numpy dtype.

`na_value` : scalar, optional
Scalar missing value indicator to use in numpy array. Defaults to the native missing value indicator of this array (`pd.NA`).

Returns

`numpy.ndarray`

Examples

An object-dtype is the default result

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a.to_numpy()
array([True, False, <NA>], dtype=object)
```

When no missing values are present, an equivalent dtype can be used.

```
>>> pd.array([True, False], dtype="boolean").to_numpy(dtype="bool")
array([ True, False])
>>> pd.array([1, 2], dtype="Int64").to_numpy("int64")
array([1, 2])
```

However, requesting such dtype will raise a `ValueError` if missing values are present and the default missing value `:attr:'NA'` is used.

```
>>> a = pd.array([True, False, pd.NA], dtype="boolean")
>>> a
<BooleanArray>
[True, False, <NA>]
Length: 3, dtype: boolean
```

```
>>> a.to_numpy(dtype="bool")
Traceback (most recent call last):
```

```
...
ValueError: cannot convert to bool numpy array in presence of missing values
```

Specify a valid `na_value` instead

```
>>> a.to_numpy(dtype="bool", na_value=False)
array([ True, False, False])
```

`tolist(self)` -> list
Return a list of the values.

These are each a scalar type, which is a Python scalar (for `str`, `int`, `float`) or a pandas scalar (for `Timestamp`/`Timedelta`/`Interval`/`Period`)

Returns

list
Python list of values in array.

See Also

`Index.tolist`: Return a list of the values in the Index.
`Series.tolist`: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

`unique(self)` -> Self
Compute the `BaseMaskedArray` of unique values.

Returns

uniques : `BaseMaskedArray`

value_counts(self, dropna: bool = True) -> Series
Returns a Series containing counts of each unique value.

Parameters

dropna : bool, default True
Don't include counts of missing values.

Returns

counts : Series

See Also

Series.value_counts

```
var(
    self,
    *,
    skipna: bool = True,
    axis: AxisInt | None = 0,
    ddof: int = 1,
    **kwargs
)
```

Readonly properties inherited from pandas.core.arrays.masked.BaseMaskedArray:

T

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> pd.array([1, 2, 3]).nbytes
27
```

ndim

Extension Arrays are only allowed to be 1-dimensional.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.ndim
1
```

shape

Return a tuple of the array dimensions.

See Also

numpy.ndarray.shape : Similar attribute which returns the shape of an array.
DataFrame.shape : Return a tuple representing the dimensionality of the
DataFrame.
Series.shape : Return a tuple representing the dimensionality of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shape
(3,)
```

Data and other attributes inherited from pandas.core.arrays.masked.BaseMaskedArray:

__array_priority__ = 1000

Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`

Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ```other``` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
 height weight
elk 1.5 500
moose 2.6 800

Adding a scalar affects all rows and columns.

>>> df[["height", "weight"]] + 1.5
 height weight
elk 3.0 501.5
moose 4.1 801.5

Each element of a list is added to a column of the DataFrame, in order.

>>> df[["height", "weight"]] + [0.5, 1.5]
 height weight
elk 2.0 501.5
moose 3.1 801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
 height weight
elk 2.0 501.5
moose 3.1 801.5

When ```other``` is a `:class:`Series``, the index of ```other``` is aligned with the columns of the DataFrame.

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
 height weight
elk 3.0 500.5
moose 4.1 800.5

Even when the index of ```other``` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ```axis='index'```.

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
 height weight
elk NaN NaN
moose NaN NaN

>>> df[["height", "weight"]].add(s2, axis="index")
 height weight

elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
```

	height	weight
deer	NaN	NaN
elk	1.7	NaN
moose	3.0	NaN

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)
```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

`__dict__`
dictionary for instance variables
`__weakref__`
list of weak references to the object

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

`__hash__` = None

Methods inherited from pandas.api.extensions.ExtensionArray:

`__repr__(self)` -> str from pandas.core.arrays.base.ExtensionArray
Return repr(self).

`argmax(self, skipna: bool = True)` -> int from pandas.core.arrays.base.ExtensionArray
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmin : Return the index of the minimum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

`argmin(self, skipna: bool = True)` -> int from pandas.core.arrays.base.ExtensionArray
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

`argsort(`
 self,
 *,
 ascending: bool = True,
 kind: SortKind = 'quicksort',
 na_position: str = 'last',
 **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Return the indices that would sort this array.

Parameters

ascending : bool, default True
Whether the indices should result in an ascending or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
If 'first', put 'NaN' values at the beginning.
If 'last', put 'NaN' values at the end.
**kwargs
Passed through to :func:`numpy.argsort`.

Returns

np.ndarray[np.intp]
Array of indices that sort 'self'. If NaN values are contained, NaN values are placed at the end.

See Also

numpy.argsort : Sorting implementation used internally.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self
An ExtensionArray of the same type as the original but with all NA values removed.

See Also

Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in an ExtensionArray.

Examples

>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64

insert(self, loc: int, item) -> Self from pandas.core.arrays.base.ExtensionArray
Insert an item at the given position.

Parameters

loc : int
Index where the 'item' needs to be inserted.
item : scalar-like
Value to be inserted.

Returns

ExtensionArray
With 'item' inserted at 'loc'.

See Also

Index.insert: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item cannot be held in an array of this type/dtype, either ValueError or TypeError should be raised.

The default implementation relies on `_from_sequence` to raise on invalid items.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)
<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64
```

`item(self, index: int | None = None)` from `pandas.core.arrays.base.ExtensionArray`
Return the array element at the specified position as a Python scalar.

Parameters

`index` : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar
The element at the specified position.

Raises

`ValueError`
If no index is provided and the array does not have exactly one element.
`IndexError`
If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
-----
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

`repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self` from `pandas`.
Repeat elements of an `ExtensionArray`.

Returns a new `ExtensionArray` where each element of the current `ExtensionArray` is repeated consecutively a given number of times.

Parameters

`repeats` : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty `ExtensionArray`.
`axis` : None
Must be `None`. Has no effect but is accepted for compatibility with numpy.

Returns

`ExtensionArray`
Newly created `ExtensionArray` with repeated elements.

See Also

`Series.repeat` : Equivalent function for `Series`.
`Index.repeat` : Equivalent function for `Index`.
`numpy.repeat` : Similar method for `numpy.ndarray`.
`ExtensionArray.take` : Take arbitrary positions.

Examples

```
>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
```

transpose(self, *axes: int) -> Self from pandas.core.arrays.base.ExtensionArray
Return a transposed view on this array.

Because ExtensionArrays are always 1D, this is a no-op. It is included for compatibility with np.ndarray.

Returns

ExtensionArray

Examples

```
>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

view(self, dtype: Dtype | None = None) -> ArrayLike from pandas.core.arrays.base.ExtensionArray
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.api.extensions.ExtensionArray:

size

The number of elements in the array.

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

__pandas_priority__ = 1000

```
class IntervalArray(pandas._libs.interval.IntervalMixin, pandas.api.extensions.ExtensionArray)
| IntervalArray(
```

```

        data,
        closed: IntervalClosedType | None = None,
        dtype: Dtype | None = None,
        copy: bool = False,
        verify_integrity: bool = True
    ) -> Self

```

Pandas array for interval data that are closed on the same side.

Parameters

```

data : array-like (1-dimensional)
    Array-like (ndarray, :class:`DateTimeArray`, :class:`TimeDeltaArray`) containing
    Interval objects from which to build the IntervalArray.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
    Whether the intervals are closed on the left-side, right-side, both or
    neither.
dtype : dtype or None, default None
    If None, dtype will be inferred.
copy : bool, default False
    Copy the input data.
verify_integrity : bool, default True
    Verify that the IntervalArray is valid.

```

Attributes

```

left
right
closed
mid
length
is_empty
is_non_overlapping_monotonic

```

Methods

```

from_arrays
from_tuples
from_breaks
contains
overlaps
set_closed
to_tuples

```

See Also

```

Index : The base pandas Index type.
Interval : A bounded slice-like interval; the elements of an IntervalArray.
interval_range : Function to create a fixed frequency IntervalIndex.
cut : Bin values into discrete Intervals.
qcut : Bin values into equal-sized Intervals based on rank or sample quantiles.

```

Notes

```

See the `user guide
<https://pandas.pydata.org/pandas-docs/stable/user\_guide/advanced.html#intervalindex>`
for more.

```

Examples

```

A new ``IntervalArray`` can be constructed directly from an array-like of
``Interval`` objects:
>>> pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
<IntervalArray>
[[0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]

```

```

It may also be constructed using one of the constructor
methods: :meth:`IntervalArray.from_arrays`,
:meth:`IntervalArray.from_breaks`, and :meth:`IntervalArray.from_tuples`.

```

Method resolution order:

```

IntervalArray
pandas._libs.interval.IntervalMixin
pandas.api.extensions.ExtensionArray
builtins.object

```

Methods defined here:

`__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray` from pandas.core.arrays.interval.IntervalArray
Return the IntervalArray's data as a numpy array of Interval objects (with dtype='object')

`__arrow_array__(self, type=None)` from pandas.core.arrays.interval.IntervalArray
Convert myself into a pyarrow Array.

`__eq__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self==value.

`__ge__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self>=value.

`__getitem__(self, key: PositionalIndexer) -> Self | IntervalOrNA` from pandas.core.arrays.interval.IntervalArray
Select a subset of self.

Parameters

item : int, slice, or ndarray

* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

`__gt__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self>value.

`__iter__(self) -> Iterator` from pandas.core.arrays.interval.IntervalArray
Iterate over elements of the array.

`__le__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self<=value.

`__len__(self) -> int` from pandas.core.arrays.interval.IntervalArray
Length of this array

Returns

length : int

`__lt__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self<value.

`__ne__(self, other)` from pandas.core.arrays.interval.IntervalArray
Return self!=value.

`__setitem__(self, key, value) -> None` from pandas.core.arrays.interval.IntervalArray
Set one or more values inplace.

This method is not required to satisfy the pandas extension array interface.

Parameters

key : int, ndarray, or slice

When called from, e.g. ``Series.\_\_setitem\_\_``, ``key`` will be

```

        one of

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

    value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
           value or values to be set of ``key``.

    Returns
    -----
    None

    Raises
    -----
    ValueError
        If the array is readonly and modification is attempted.

    argsort(
        self,
        *,
        ascending: bool = True,
        kind: SortKind = 'quicksort',
        na_position: str = 'last',
        **kwargs
    ) -> np.ndarray from pandas.core.arrays.interval.IntervalArray
        Return the indices that would sort this array.

    Parameters
    -----
    ascending : bool, default True
        Whether the indices should result in an ascending
        or descending sort.
    kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
        Sorting algorithm.
    na_position : {'first', 'last'}, default 'last'
        If ``'first'``, put ``NaN`` values at the beginning.
        If ``'last'``, put ``NaN`` values at the end.
    **kwargs
        Passed through to :func:`numpy.argsort`.

    Returns
    -----
    np.ndarray[np.intp]
        Array of indices that sort ``self``. If NaN values are contained,
        NaN values are placed at the end.

    See Also
    -----
    numpy.argsort : Sorting implementation used internally.

    Examples
    -----
    >>> arr = pd.array([3, 1, 2, 5, 4])
    >>> arr.argsort()
    array([1, 2, 0, 4, 3])

    astype(self, dtype, copy: bool = True) from pandas.core.arrays.interval.IntervalArray
        Cast to an ExtensionArray or NumPy array with dtype 'dtype'.

    Parameters
    -----
    dtype : str or dtype
        Typecode or data-type to which the array is cast.

    copy : bool, default True
        Whether to copy the data, even if not necessary. If False,
        a copy is made only if the old dtype does not match the
        new dtype.

    Returns
    -----
    array : ExtensionArray or ndarray
        ExtensionArray or NumPy ndarray with 'dtype' for its dtype.

    contains(self, other) from pandas.core.arrays.interval.IntervalArray

```

Check elementwise if the Intervals contain the value.

Return a boolean mask whether the value is contained in the Intervals of the IntervalArray.

Parameters

other : scalar

The value to check whether it is contained in the Intervals.

Returns

boolean array

A boolean mask whether the value is contained in the Intervals.

See Also

Interval.contains : Check whether Interval object contains value.

IntervalArray.overlaps : Check if an Interval overlaps the values in the IntervalArray.

Examples

```
>>> intervals = pd.arrays.IntervalArray.from_tuples([(0, 1), (1, 3), (2, 4)])
```

```
>>> intervals
```

```
<IntervalArray>
```

```
[[0, 1], (1, 3], (2, 4]]
```

```
Length: 3, dtype: interval[int64, right]
```

```
>>> intervals.contains(0.5)
```

```
array([ True, False, False])
```

copy(self) -> Self from pandas.core.arrays.interval.IntervalArray

Return a copy of the array.

Returns

IntervalArray

delete(self, loc) -> Self from pandas.core.arrays.interval.IntervalArray

equals(self, other) -> bool from pandas.core.arrays.interval.IntervalArray

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray

Array to compare to this Array.

Returns

boolean

Whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.

Series.equals : Equivalent method for Series.

DataFrame.equals : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
```

```
>>> arr2 = pd.array([1, 2, np.nan])
```

```
>>> arr1.equals(arr2)
```

```
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
```

```
>>> arr2 = pd.array([1, 2, np.nan])
```

```
>>> arr1.equals(arr2)
```

```
False
```

fillna(self, value, limit: int | None = None, copy: bool = True) -> Self from pandas.core.ar

Fill NA/NaN values using the specified method.

Parameters

value : scalar, dict, Series
If a scalar value is passed it is used to fill all missing values.
Alternatively, a Series or dict can be used to fill in different values for each index. The value should not be a list. The value(s) passed should be either Interval objects or NA/NaN.
limit : int, default None
(Not implemented yet for IntervalArray)
The maximum number of entries where NA values will be filled.
copy : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated.
For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

filled : IntervalArray with NA/NaN filled

insert(self, loc: int, item: Interval) -> Self from pandas.core.arrays.interval.IntervalArray
Return a new IntervalArray inserting new item at location. Follows Python numpy.insert semantics for negative values. Only Interval objects and NA can be inserted into an IntervalIndex

Parameters

loc : int
item : Interval

Returns

IntervalArray

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_] from pandas.core.arrays.interval.IntervalArray
Pointwise comparison for set containment in the given values.

Roughly equivalent to `np.array([x in values for x in self])`

Parameters

values : np.ndarray or ExtensionArray
Values to compare every element in the array against.

Returns

np.ndarray[bool]
With true at indices where value is in `values`.

See Also

DataFrame.isin: Whether each element in the DataFrame is contained in values.
Index.isin: Return a boolean array where the index values are in values.
Series.isin: Whether elements in Series are contained in values.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean

isna(self) -> np.ndarray from pandas.core.arrays.interval.IntervalArray
A 1-D array indicating if each value is missing.

Returns

numpy.ndarray or pandas.api.extensions.ExtensionArray
In most cases, this should return a NumPy ndarray. For exceptional cases like ``SparseArray``, where returning an ndarray would be expensive, an ExtensionArray may be returned.

See Also

ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.

Notes

If returning an ExtensionArray, then

- * ``na\_values.\_is\_boolean`` should be True
- * ``na\_values`` should implement :func:`ExtensionArray.\_reduce`
- * ``na\_values`` should implement :func:`ExtensionArray.\_accumulate`
- * ``na\_values.any`` and ``na\_values.all`` should be implemented

Examples

>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False, True, True])

max(self, *, axis: AxisInt | None = None, skipna: bool = True) -> IntervalOrNA from pandas.core.arrays.interval.IntervalArray

min(self, *, axis: AxisInt | None = None, skipna: bool = True) -> IntervalOrNA from pandas.core.arrays.interval.IntervalArray

overlaps(self, other) from pandas.core.arrays.interval.IntervalArray
Check elementwise if an Interval overlaps the values in the IntervalArray.

Two intervals overlap if they share a common point, including closed endpoints. Intervals that only have an open endpoint in common do not overlap.

Parameters

other : IntervalArray
Interval to check against for an overlap.

Returns

ndarray
Boolean array positionally indicating where an overlap occurs.

See Also

Interval.overlaps : Check whether two Interval objects overlap.

Examples

>>> data = [(0, 1), (1, 3), (2, 4)]
>>> intervals = pd.arrays.IntervalArray.from_tuples(data)
>>> intervals
<IntervalArray>
[(0, 1], (1, 3], (2, 4]]
Length: 3, dtype: interval[int64, right]

>>> intervals.overlaps(pd.Interval(0.5, 1.5))
array([True, True, False])

Intervals that share closed endpoints overlap:

>>> intervals.overlaps(pd.Interval(1, 3, closed="left"))
array([True, True, True])

Intervals that only have an open endpoint in common do not overlap:

>>> intervals.overlaps(pd.Interval(1, 2, closed="right"))
array([False, True, False])

repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self from pandas.core.arrays.interval.IntervalArray
Repeat elements of an IntervalArray.

Returns a new IntervalArray where each element of the current IntervalArray is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty

IntervalArray.
axis : None
Must be ``None``. Has no effect but is accepted for compatibility
with numpy.

Returns

IntervalArray
Newly created IntervalArray with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
Index.repeat : Equivalent function for Index.
numpy.repeat : Similar method for :class:`numpy.ndarray`.
ExtensionArray.take : Take arbitrary positions.

Examples

>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']

set_closed(self, closed: IntervalClosedType) -> Self from pandas.core.arrays.interval.IntervalArray
Return an identical IntervalArray closed on the specified side.

Parameters

closed : {'left', 'right', 'both', 'neither'}
Whether the intervals are closed on the left-side, right-side, both
or neither.

Returns

IntervalArray
A new IntervalArray with the specified side closures.

See Also

IntervalArray.closed : Returns inclusive side of the Interval.
arrays.IntervalArray.closed : Returns inclusive side of the IntervalArray.

Examples

>>> index = pd.arrays.IntervalArray.from_breaks(range(4))
>>> index
<IntervalArray>
[[0, 1], (1, 2], (2, 3]]
Length: 3, dtype: interval[int64, right]
>>> index.set_closed("both")
<IntervalArray>
[[0, 1], [1, 2], [2, 3]]
Length: 3, dtype: interval[int64, both]

shift(self, periods: int = 1, fill_value: object = None) -> IntervalArray from pandas.core.arrays.interval.IntervalArray
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1
The number of periods to shift. Negative values are allowed
for shifting backwards.

fill_value : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on this array.

api.extensions.ExtensionArray.factorize : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`, then an array of size len(self) is returned, with all values filled with ``self.dtype.na_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr.shift(2)
```

```
<IntegerArray>
```

```
[<NA>, <NA>, 1]
```

```
Length: 3, dtype: Int64
```

```
take(  
    self,  
    indices,  
    *,  
    allow_fill: bool = False,  
    fill_value=None,  
    axis=None,  
    **kwargs  
) -> Self from pandas.core.arrays.interval.IntervalArray  
Take elements from the IntervalArray.
```

Parameters

indices : sequence of integers
Indices to be taken.

allow_fill : bool, default False
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in `indices` indicate missing values. These values are set to `fill\_value`. Any other other negative values raise a ``ValueError``.

fill_value : Interval or NA, optional
Fill value to use for NA-indices when `allow\_fill` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of `fill\_value`: a user-facing "boxed" scalar, and a low-level physical NA value. `fill\_value` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

axis : any, default None
Present for compat with IntervalIndex; does nothing.

Returns

IntervalArray

Raises

```

-----
IndexError
    When the indices are out of bounds for the array.
ValueError
    When `indices` contains negative values other than ``-1``
    and `allow_fill` is True.

to_tuples(self, na_tuple: bool = True) -> np.ndarray from pandas.core.arrays.interval.IntervalArray
    Return an ndarray (if self is IntervalArray) or Index (if self is IntervalIndex)

Parameters
-----
na_tuple : bool, default True
    If ``True``, return ``NA`` as a tuple ``(nan, nan)``. If ``False``,
    just return ``NA`` as ``nan``.

Returns
-----
ndarray or Index
    An ndarray of tuples representing the intervals
    if `self` is an IntervalArray.
    An Index of tuples representing the intervals
    if `self` is an IntervalIndex.

See Also
-----
IntervalArray.to_list : Convert IntervalArray to a list of tuples.
IntervalArray.to_numpy : Convert IntervalArray to a numpy array.
IntervalArray.unique : Find unique intervals in an IntervalArray.

Examples
-----
For :class:`pandas.IntervalArray`:

>>> idx = pd.arrays.IntervalArray.from_tuples([(0, 1), (1, 2)])
>>> idx
<IntervalArray>
[[0, 1], (1, 2]]
Length: 2, dtype: interval[int64, right]
>>> idx.to_tuples()
array([(np.int64(0), np.int64(1)), (np.int64(1), np.int64(2))],
      dtype=object)

For :class:`pandas.IntervalIndex`:

>>> idx = pd.interval_range(start=0, end=2)
>>> idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> idx.to_tuples()
Index([(0, 1), (1, 2)], dtype='object')

unique(self) -> IntervalArray from pandas.core.arrays.interval.IntervalArray
    Compute the ExtensionArray of unique values.

Returns
-----
pandas.api.extensions.ExtensionArray
    With unique values from the input array.

See Also
-----
Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples
-----
>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
-----
Class methods defined here:

from_arrays(

```

```

    left,
    right,
    closed: IntervalClosedType | None = 'right',
    copy: bool = False,
    dtype: Dtype | None = None
) -> Self from pandas.core.arrays.interval.IntervalArray
Construct from two arrays defining the left and right bounds.

Parameters
-----
left : array-like (1-dimensional)
    Left bounds for each interval.
right : array-like (1-dimensional)
    Right bounds for each interval.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
    Whether the intervals are closed on the left-side, right-side, both
    or neither.
copy : bool, default False
    Copy the data.
dtype : dtype, optional
    If None, dtype will be inferred.

Returns
-----
IntervalArray

Raises
-----
ValueError
    When a value is missing in only one of `left` or `right`.
    When a value in `left` is greater than the corresponding value
    in `right`.

See Also
-----
interval_range : Function to create a fixed frequency IntervalIndex.
IntervalArray.from_breaks : Construct an IntervalArray from an array of
    splits.
IntervalArray.from_tuples : Construct an IntervalArray from an
    array-like of tuples.

Notes
-----
Each element of `left` must be less than or equal to the `right`
element at the same position. If an element is missing, it must be
missing in both `left` and `right`. A TypeError is raised when
using an unsupported type for `left` or `right`. At the moment,
'category', 'object', and 'string' subtypes are not supported.

Examples
-----
>>> pd.arrays.IntervalArray.from_arrays([0, 1, 2], [1, 2, 3])
<IntervalArray>
[[0, 1], (1, 2], (2, 3]]
Length: 3, dtype: interval[int64, right]

from_breaks(
    breaks,
    closed: IntervalClosedType | None = 'right',
    copy: bool = False,
    dtype: Dtype | None = None
) -> Self from pandas.core.arrays.interval.IntervalArray
Construct an IntervalArray from an array of splits.

Parameters
-----
breaks : array-like (1-dimensional)
    Left and right bounds for each interval.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
    Whether the intervals are closed on the left-side, right-side, both
    or neither.
copy : bool, default False
    Copy the data.
dtype : dtype or None, default None
    If None, dtype will be inferred.

Returns

```

IntervalArray

See Also

interval_range : Function to create a fixed frequency IntervalIndex.
IntervalArray.from_arrays : Construct from a left and right array.
IntervalArray.from_tuples : Construct from a sequence of tuples.

Examples

>>> pd.arrays.IntervalArray.from_breaks([0, 1, 2, 3])
<IntervalArray>
[[0, 1], (1, 2], (2, 3]]
Length: 3, dtype: interval[int64, right]

from_tuples(
 data,
 closed: IntervalClosedType | None = 'right',
 copy: bool = False,
 dtype: Dtype | None = None
) -> Self from pandas.core.arrays.interval.IntervalArray
Construct an IntervalArray from an array-like of tuples.

Parameters

data : array-like (1-dimensional)
 Array of tuples.
closed : {'left', 'right', 'both', 'neither'}, default 'right'
 Whether the intervals are closed on the left-side, right-side, both
 or neither.
copy : bool, default False
 By-default copy the data, this is compat only and ignored.
dtype : dtype or None, default None
 If None, dtype will be inferred.

Returns

IntervalArray

See Also

interval_range : Function to create a fixed frequency IntervalIndex.
IntervalArray.from_arrays : Construct an IntervalArray from a left and
 right array.
IntervalArray.from_breaks : Construct an IntervalArray from an array of
 splits.

Examples

>>> pd.arrays.IntervalArray.from_tuples([(0, 1), (1, 2)])
<IntervalArray>
[[0, 1], (1, 2]]
Length: 2, dtype: interval[int64, right]

Static methods defined here:

__new__(
 cls,
 data,
 closed: IntervalClosedType | None = None,
 dtype: Dtype | None = None,
 copy: bool = False,
 verify_integrity: bool = True
) -> Self from pandas.core.arrays.interval.IntervalArray
Create and return a new object. See help(type) for accurate signature.

Readonly properties defined here:

closed
 String describing the inclusive side the intervals.

 Either ``left``, ``right``, ``both`` or ``neither``.

See Also

IntervalArray.closed : Returns inclusive side of the IntervalArray.
Interval.closed : Returns inclusive side of the Interval.
IntervalIndex.closed : Returns inclusive side of the IntervalIndex.

Examples

For arrays:

```
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
>>> interv_arr
<IntervalArray>
[[0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.closed
'right'
```

For Interval Index:

```
>>> interv_idx = pd.interval_range(start=0, end=2)
>>> interv_idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> interv_idx.closed
'right'
```

dtype

An instance of ExtensionDtype.

See Also

api.extensions.ExtensionDtype : Base class for extension dtypes.
api.extensions.ExtensionArray : Base class for extension array types.
api.extensions.ExtensionArray.dtype : The dtype of an ExtensionArray.
Series.dtype : The dtype of a Series.
DataFrame.dtype : The dtype of a DataFrame.

Examples

```
>>> pd.array([1, 2, 3]).dtype
Int64Dtype()
```

is_non_overlapping_monotonic

Return a boolean whether the IntervalArray/IntervalIndex is non-overlapping and monotonic.

Non-overlapping means (no Intervals share points), and monotonic means either monotonic increasing or monotonic decreasing.

See Also

overlaps : Check if two IntervalIndex objects overlap.

Examples

For arrays:

```
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
>>> interv_arr
<IntervalArray>
[[0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.is_non_overlapping_monotonic
True
```

```
>>> interv_arr = pd.arrays.IntervalArray(
...     [pd.Interval(0, 1), pd.Interval(-1, 0.1)]
... )
>>> interv_arr
<IntervalArray>
[[0.0, 1.0], (-1.0, 0.1]]
Length: 2, dtype: interval[float64, right]
>>> interv_arr.is_non_overlapping_monotonic
False
```

For Interval Index:

```
>>> interv_idx = pd.interval_range(start=0, end=2)
```

```
>>> interv_idx
IntervalIndex([(0, 1], (1, 2]], dtype='interval[int64, right]')
>>> interv_idx.is_non_overlapping_monotonic
True

>>> interv_idx = pd.interval_range(start=0, end=2, closed="both")
>>> interv_idx
IntervalIndex([(0, 1], [1, 2]], dtype='interval[int64, both]')
>>> interv_idx.is_non_overlapping_monotonic
False
```

left

Return the left endpoints of each Interval in the IntervalArray as an Index.

This property provides access to the left endpoints of the intervals contained within the IntervalArray. This can be useful for analyses where the starting point of each interval is of interest, such as in histogram creation, data aggregation, or any scenario requiring the identification of the beginning of defined ranges. This property returns a ``pandas.Index`` object containing the midpoint for each interval.

See Also

```
-----
arrays.IntervalArray.right : Return the right endpoints of each Interval in
the IntervalArray as an Index.
arrays.IntervalArray.mid : Return the midpoint of each Interval in the
IntervalArray as an Index.
arrays.IntervalArray.contains : Check elementwise if the Intervals contain
the value.
```

Examples

```
-----
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(2, 5)])
>>> interv_arr
<IntervalArray>
[(0, 1], (2, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.left
Index([0, 2], dtype='int64')
```

length

Return an Index with entries denoting the length of each Interval.

The length of an interval is calculated as the difference between its ``right`` and ``left`` bounds. This property is particularly useful when working with intervals where the size of the interval is an important attribute, such as in time-series analysis or spatial data analysis.

See Also

```
-----
arrays.IntervalArray.left : Return the left endpoints of each Interval in
the IntervalArray as an Index.
arrays.IntervalArray.right : Return the right endpoints of each Interval in
the IntervalArray as an Index.
arrays.IntervalArray.mid : Return the midpoint of each Interval in the
IntervalArray as an Index.
```

Examples

```
-----
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])
>>> interv_arr
<IntervalArray>
[(0, 1], (1, 5]]
Length: 2, dtype: interval[int64, right]
>>> interv_arr.length
Index([1, 4], dtype='int64')
```

mid

Return the midpoint of each Interval in the IntervalArray as an Index.

The midpoint of an interval is calculated as the average of its ``left`` and ``right`` bounds. This property returns a ``pandas.Index`` object containing the midpoint for each interval.

See Also

Interval.left : Return left bound for the interval.
Interval.right : Return right bound for the interval.
Interval.length : Return the length of each interval.

Examples

```
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(1, 5)])  
>>> interv_arr  
<IntervalArray>  
[[0, 1], (1, 5]]  
Length: 2, dtype: interval[int64, right]  
>>> interv_arr.mid  
Index([0.5, 3.0], dtype='float64')
```

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> pd.array([1, 2, 3]).nbytes  
27
```

ndim

Extension Arrays are only allowed to be 1-dimensional.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> arr = pd.array([1, 2, 3])  
>>> arr.ndim  
1
```

right

Return the right endpoints of each Interval in the IntervalArray as an Index.

This property extracts the right endpoints from each interval contained within the IntervalArray. This can be helpful in use cases where you need to work with or compare only the upper bounds of intervals, such as when performing range-based filtering, determining interval overlaps, or visualizing the end boundaries of data segments.

See Also

arrays.IntervalArray.left : Return the left endpoints of each Interval in the IntervalArray as an Index.
arrays.IntervalArray.mid : Return the midpoint of each Interval in the IntervalArray as an Index.
arrays.IntervalArray.contains : Check elementwise if the Intervals contain the value.

Examples

```
>>> interv_arr = pd.arrays.IntervalArray([pd.Interval(0, 1), pd.Interval(2, 5)])  
>>> interv_arr  
<IntervalArray>  
[[0, 1], (2, 5]]  
Length: 2, dtype: interval[int64, right]  
>>> interv_arr.right  
Index([1, 5], dtype='int64')
```

size

The number of elements in the array.

Data descriptors defined here:


```

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object
-----
Data and other attributes defined here:

__annotations__ = {'_dtype': 'IntervalDtype', '_left': 'IntervalSide', ...
__hash__ = None
can_hold_na = True
-----
Methods inherited from pandas._libs.interval.IntervalMixin:

__reduce__ = __reduce_cython__(...)
__reduce_cython__(self)
__setstate__ = __setstate_cython__(...)
__setstate_cython__(self, __pyx_state)
-----
Data descriptors inherited from pandas._libs.interval.IntervalMixin:

closed_left
    Check if the interval is closed on the left side.

    For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

    Returns
    -----
    bool
        True if the Interval is closed on the left-side.

    See Also
    -----
    Interval.closed_right : Check if the interval is closed on the right side.
    Interval.open_left : Boolean inverse of closed_left.

    Examples
    -----
    >>> iv = pd.Interval(0, 5, closed='left')
    >>> iv.closed_left
    True

    >>> iv = pd.Interval(0, 5, closed='right')
    >>> iv.closed_left
    False

closed_right
    Check if the interval is closed on the right side.

    For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

    Returns
    -----
    bool
        True if the Interval is closed on the left-side.

    See Also
    -----
    Interval.closed_left : Check if the interval is closed on the left side.
    Interval.open_right : Boolean inverse of closed_right.

    Examples
    -----
    >>> iv = pd.Interval(0, 5, closed='both')
    >>> iv.closed_right
    True

    >>> iv = pd.Interval(0, 5, closed='left')

```

```
>>> iv.closed_right
False
```

is_empty

Indicates if an interval is empty, meaning it contains no points.

An interval is considered empty if its ``left`` and ``right`` endpoints are equal, and it is not closed on both sides. This means that the interval does not include any real points. In the case of an `:class:`pandas.arrays.IntervalArray`` or `:class:`IntervalIndex``, the property returns a boolean array indicating the emptiness of each interval.

Returns

bool or ndarray

A boolean indicating if a scalar `:class:`Interval`` is empty, or a boolean ``ndarray`` positionally indicating if an ``Interval`` in an `:class:`~arrays.IntervalArray`` or `:class:`IntervalIndex`` is empty.

See Also

`Interval.length` : Return the length of the Interval.

Examples

An `:class:`Interval`` that contains points is not empty:

```
>>> pd.Interval(0, 1, closed='right').is_empty
False
```

An ``Interval`` that does not contain any points is empty:

```
>>> pd.Interval(0, 0, closed='right').is_empty
True
>>> pd.Interval(0, 0, closed='left').is_empty
True
>>> pd.Interval(0, 0, closed='neither').is_empty
True
```

An ``Interval`` that contains a single point is not empty:

```
>>> pd.Interval(0, 0, closed='both').is_empty
False
```

An `:class:`~arrays.IntervalArray`` or `:class:`IntervalIndex`` returns a boolean ``ndarray`` positionally indicating if an ``Interval`` is empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'),
...        pd.Interval(1, 2, closed='neither')]
>>> pd.arrays.IntervalArray(ivs).is_empty
array([ True, False])
```

Missing values are not considered empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'), np.nan]
>>> pd.IntervalIndex(ivs).is_empty
array([ True, False])
```

open_left

Check if the interval is open on the left side.

For the meaning of ``closed`` and ``open`` see `:class:`~pandas.Interval``.

Returns

bool

True if the Interval is not closed on the left-side.

See Also

`Interval.open_right` : Check if the interval is open on the right side.

`Interval.closed_left` : Boolean inverse of `open_left`.

Examples

```

>>> iv = pd.Interval(0, 5, closed='neither')
>>> iv.open_left
True

>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.open_left
False

open_right
Check if the interval is open on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns
-----
bool
    True if the Interval is not closed on the left-side.

See Also
-----
Interval.open_left : Check if the interval is open on the left side.
Interval.closed_right : Boolean inverse of open_right.

Examples
-----
>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.open_right
True

>>> iv = pd.Interval(0, 5)
>>> iv.open_right
False
-----
Methods inherited from pandas.api.extensions.ExtensionArray:

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.array
__contains__(self, item: object) -> bool | np.bool_ from pandas.core.arrays.base.ExtensionAr
    Return for `item in self`.

__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).

argmax(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
    Return the index of maximum value.

    In case of multiple occurrences of the maximum value, the index
    corresponding to the first occurrence is returned.

Parameters
-----
skipna : bool, default True

Returns
-----
int

See Also
-----
ExtensionArray.argmin : Return the index of the minimum value.

Examples
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

argmin(self, skipna: bool = True) -> int from pandas.core.arrays.base.ExtensionArray
    Return the index of minimum value.

    In case of multiple occurrences of the minimum value, the index
    corresponding to the first occurrence is returned.

Parameters
-----
skipna : bool, default True

```

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)
```

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all NA values removed.

See Also

Series.dropna : Remove missing values from a Series.

DataFrame.dropna : Remove missing values from a DataFrame.

api.extensions.ExtensionArray.isna : Check for missing values in an ExtensionArray.

Examples

```
>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64
```

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] from pandas.core.arrays.base.ExtensionArray
Return boolean ndarray denoting duplicate values.

Parameters

keep : {'first', 'last', False}, default 'first'

- ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
- ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

Returns

ndarray[bool]

With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.

Series.duplicated : Indicate duplicate Series values.

api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

```
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])
```

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas.core.arrays.base.ExtensionArray
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True

If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

```

-----
codes : ndarray
    An integer NumPy array that's an indexer into the original
    ExtensionArray.
uniques : ExtensionArray
    An ExtensionArray containing the unique values of `self`.

    .. note::

        uniques will not contain an entry for the NA value of
        the ExtensionArray if there are any missing values present
        in `self`.

```

See Also

```

-----
factorize : Top-level factorize method that dispatches here.

```

Notes

```

-----
:meth:`pandas.factorize` offers a `sort` keyword as well.

```

Examples

```

-----
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

```

```

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.base.ExtensionArray
Fill NaN values using an interpolation method.

```

Parameters

```

-----
method : str, default 'linear'
    Interpolation technique to use. One of:
    * 'linear': Ignore the index and treat the values as equally spaced.
      This is the only method supported on MultiIndexes.
    * 'time': Works on daily and higher resolution data to interpolate
      given length of interval.
    * 'index', 'values': use the actual numerical values of the index.
    * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric',
      'polynomial': Passed to scipy.interpolate.interp1d, whereas 'spline'
      is passed to scipy.interpolate.UnivariateSpline. These methods use
      the numerical values of the index.
      Both 'polynomial' and 'spline' require that you also specify an
      order (int), e.g. arr.interpolate(method='polynomial', order=5).
    * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima',
      'cubicspline': Wrappers around the SciPy interpolation methods
      of similar names. See Notes.
    * 'from_derivatives': Refers to scipy.interpolate.BPoly.from_derivatives.
axis : int
    Axis to interpolate along. For 1-dimensional data, use 0.
index : Index
    Index to use for interpolation.
limit : int or None
    Maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction : {'forward', 'backward', 'both'}
    Consecutive NaNs will be filled in this direction.
limit_area : {'inside', 'outside'} or None
    If limit is specified, consecutive NaNs will be filled with this
    restriction.

```

- * None: No fill restriction.
- * 'inside': Only fill NaNs surrounded by valid values (interpolate).
- * 'outside': Only fill NaNs outside valid values (extrapolate).

copy : bool
If True, a copy of the object is returned with interpolated values.

**kwargs : optional
Keyword arguments to pass on to the interpolating function.

Returns

ExtensionArray
An ExtensionArray with interpolated values.

See Also

Series.interpolate : Interpolate values in a Series.
DataFrame.interpolate : Interpolate values in a DataFrame.

Notes

- All parameters must be specified as keyword arguments.
- The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima' methods are wrappers around the respective SciPy implementations of similar names. These use the actual numerical values of the index.

Examples

Interpolating values in a NumPy array:

```
>>> arr = pd.arrays.NumpyExtensionArray(np.array([0, 1, np.nan, 3]))
>>> arr.interpolate(
...     method="linear",
...     limit=3,
...     limit_direction="forward",
...     index=pd.Index(range(len(arr))),
...     fill_value=1,
...     copy=False,
...     axis=0,
...     limit_area="inside",
... )
<NumpyExtensionArray>
[0.0, 1.0, 2.0, 3.0]
Length: 4, dtype: float64
```

Interpolating values in a FloatingArray:

```
>>> arr = pd.array([1.0, pd.NA, 3.0, 4.0, pd.NA, 6.0], dtype="Float64")
>>> arr.interpolate(
...     method="linear",
...     axis=0,
...     index=pd.Index(range(len(arr))),
...     limit=None,
...     limit_direction="both",
...     limit_area=None,
...     copy=True,
... )
<FloatingArray>
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Length: 6, dtype: Float64
```

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar
The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

`IndexError`

If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
```

```
>>> arr.item()
```

```
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
```

```
>>> arr.item(0)
```

```
np.int64(1)
```

```
>>> arr.item(2)
```

```
np.int64(3)
```

`map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.arrays.base.ExtensionArray`
Map values using an input mapping or function.

Parameters

`mapper` : function, dict, or Series

Mapping correspondence.

`na_action` : {None, 'ignore'}, default None

If 'ignore', propagate NA values, without passing them to the mapping correspondence. If 'ignore' is not supported, a `NotImplementedError` should be raised.

Returns

`Union[ndarray, Index, ExtensionArray]`

The output of the mapping function applied to the array.

If the function returns a tuple with more than one element a `MultiIndex` will be returned.

`ravel(self, order: Literal['C', 'F', 'A', 'K'] | None = 'C') -> Self from pandas.core.arrays.base.ExtensionArray`
Return a flattened view on this array.

Parameters

`order` : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

`ExtensionArray`

A flattened view on the array.

See Also

`ExtensionArray.tolist`: Return a list of the values.

Notes

- Because `ExtensionArrays` are 1D-only, this is a no-op.

- The "order" argument is ignored, is for compatibility with NumPy.

Examples

```
>>> pd.array([1, 2, 3]).ravel()
```

```
<IntegerArray>
```

```
[1, 2, 3]
```

```
Length: 3, dtype: Int64
```

`searchsorted(`

`self,`

`value: NumpyValueArrayLike | ExtensionArray,`

`side: Literal['left', 'right'] = 'left',`

`sorter: NumpySorter | None = None`

`) -> npt.NDArray[np.intp] | np.intp from pandas.core.arrays.base.ExtensionArray`

Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the`

corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left  ``self[i-1] < value <= self[i]``
right ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into `self`.
side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

```
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])
```

```
to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Convert to a NumPy ndarray.
```

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional
The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.
na_value : Any, optional
The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

```
tolist(self) -> list from pandas.core.arrays.base.ExtensionArray
Return a list of the values.
```

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list
Python list of values in array.

See Also

Index.to_list: Return a list of the values in the Index.
Series.to_list: Return a list of the values in the Series.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]

transpose(self, *axes: int) -> Self from pandas.core.arrays.base.ExtensionArray
Return a transposed view on this array.

Because ExtensionArrays are always 1D, this is a no-op. It is included for compatibility with np.ndarray.

Returns

ExtensionArray

Examples

>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.base.ExtensionArray
Return a Series containing counts of unique values.

This method returns a Series with unique values as the index and their counts as the values.

Parameters

dropna : bool, default True
Don't include counts of NA values.

Returns

Series

A Series with unique values as index and counts as values.

See Also

Series.value_counts : Equivalent method on Series.
DataFrame.value_counts : Equivalent method on DataFrame.
Index.value_counts : Equivalent method on Index.

Examples

>>> from pandas.core.arrays import IntegerArray
>>> arr = IntegerArray._from_sequence([3, 3, 3, 1, 2, 2])
>>> arr.value_counts()
3 3
1 1
2 2
Name: count, dtype: Int64

view(self, dtype: Dtype | None = None) -> ArrayLike from pandas.core.arrays.base.ExtensionArray
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.api.extensions.ExtensionArray:

T

shape

Return a tuple of the array dimensions.

See Also

numpy.ndarray.shape : Similar attribute which returns the shape of an array.
DataFrame.shape : Return a tuple representing the dimensionality of the DataFrame.
Series.shape : Return a tuple representing the dimensionality of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shape
(3,)
```

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

__pandas_priority__ = 1000

class NumpyExtensionArray(pandas.core.arraylike.OpsMixin, pandas.core.arrays._mixins.NDArrayBacked)

```
    NumpyExtensionArray(
        values: np.ndarray | NumpyExtensionArray,
        copy: bool = False
    ) -> None
```

A pandas ExtensionArray for NumPy data.

This is mostly for internal compatibility, and is not especially useful on its own.

Parameters

values : ndarray
The NumPy ndarray to wrap. Must be 1-dimensional.
copy : bool, default False
Whether to copy ``values``.

Attributes

None

Methods

None

See Also

array : Create an array.
Series.to_numpy : Convert a Series to a NumPy array.

Examples

```

-----
>>> pd.arrays.NumpyExtensionArray(np.array([0, 1, 2, 3]))
<NumpyExtensionArray>
[0, 1, 2, 3]
Length: 4, dtype: int64

Method resolution order:
  NumpyExtensionArray
  pandas.core.arraylike.OpsMixin
  pandas.core.arrays._mixins.NDArrayBackedExtensionArray
  pandas._libs.arrays.NDArrayBacked
  pandas.api.extensions.ExtensionArray
  pandas.core.strings.object_array.ObjectStringArrayMixin
  builtins.object

Methods defined here:

__abs__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray
__array__(self, dtype: np.dtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray
__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.arrays.numpy_.NumpyExtensionArray
__init__(self, values: np.ndarray | NumpyExtensionArray, copy: bool = False) -> None from pandas.core.arrays.numpy_.NumpyExtensionArray
    Initialize self.  See help(type(self)) for accurate signature.
__invert__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray
__neg__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray
__pos__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray

all(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

any(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

astype(self, dtype, copy: bool = True) from pandas.core.arrays.numpy_.NumpyExtensionArray
    Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters
-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also
-----
Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
    sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
    a sequence.

```

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

```
>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()
```

Otherwise, we will get a Numpy ndarray:

```
>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')
```

```
interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.numpy_.NumpyExtensionArray
    See NDFrame.interpolate.__doc__.
```

```
isna(self) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray
    A 1-D array indicating if each value is missing.
```

Returns

```
-----
numpy.ndarray or pandas.api.extensions.ExtensionArray
    In most cases, this should return a NumPy ndarray. For
    exceptional cases like ``SparseArray``, where returning
    an ndarray would be expensive, an ExtensionArray may be
    returned.
```

See Also

```
-----
ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.
```

Notes

If returning an ExtensionArray, then

- * ``na\_values.\_is\_boolean`` should be True
- * ``na\_values`` should implement :func:`ExtensionArray.\_reduce`
- * ``na\_values`` should implement :func:`ExtensionArray.\_accumulate`
- * ``na\_values.any`` and ``na\_values.all`` should be implemented

Examples

```
-----
>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False,  True,  True])
```

```
kurt(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
```

```

        out=None,
        keepdims: bool = False,
        skipna: bool = True
    ) from pandas.core.arrays.numpy_.NumpyExtensionArray

max(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs) -> Scalar from pandas

mean(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

median(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    overwrite_input: bool = False,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

min(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs) -> Scalar from pandas

prod(
    self,
    *,
    axis: AxisInt | None = None,
    skipna: bool = True,
    min_count: int = 0,
    **kwargs
) -> Scalar from pandas.core.arrays.numpy_.NumpyExtensionArray

sem(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

skew(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

std(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

sum(
    self,
    *,
    axis: AxisInt | None = None,
    skipna: bool = True,
    min_count: int = 0,
    **kwargs

```

```

) -> Scalar from pandas.core.arrays.numpy_.NumpyExtensionArray

take(
    self,
    indices: TakeIndexer,
    *,
    allow_fill: bool = False,
    fill_value: Any = None,
    axis: AxisInt = 0
) -> Self from pandas.core.arrays.numpy_.NumpyExtensionArray
    Take entries from this array at each index in a list of indices,
    producing an array containing only those entries.

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray
    Convert to a NumPy ndarray.

    This is similar to :meth:`numpy.asarray`, but may provide additional control
    over how the conversion is done.

    Parameters
    -----
    dtype : str or numpy.dtype, optional
        The dtype to pass to :meth:`numpy.asarray`.
    copy : bool, default False
        Whether to ensure that the returned value is a not a view on
        another array. Note that ``copy=False`` does not *ensure* that
        ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
        a copy is made, even if not strictly necessary.
    na_value : Any, optional
        The value to use for missing values. The default value depends
        on `dtype` and the type of the array.

    Returns
    -----
    numpy.ndarray

var(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

-----
Readonly properties defined here:

dtype
    An instance of ExtensionDtype.

    See Also
    -----
    api.extensions.ExtensionDtype : Base class for extension dtypes.
    api.extensions.ExtensionArray : Base class for extension array types.
    api.extensions.ExtensionArray.dtype : The dtype of an ExtensionArray.
    Series.dtype : The dtype of a Series.
    DataFrame.dtype : The dtype of a DataFrame.

    Examples
    -----
    >>> pd.array([1, 2, 3]).dtype
    Int64Dtype()

-----
Data and other attributes defined here:

__annotations__ = {'_dtype': 'NumpyEADtype', '_ndarray': 'np.ndarray'}
__array_priority__ = 1000

```

Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`

Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ```other``` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
 height weight
elk 1.5 500
moose 2.6 800

Adding a scalar affects all rows and columns.

>>> df[["height", "weight"]] + 1.5
 height weight
elk 3.0 501.5
moose 4.1 801.5

Each element of a list is added to a column of the DataFrame, in order.

>>> df[["height", "weight"]] + [0.5, 1.5]
 height weight
elk 2.0 501.5
moose 3.1 801.5

Keys of a dictionary are aligned to the DataFrame, based on column names;
each value in the dictionary is added to the corresponding column.

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
 height weight
elk 2.0 501.5
moose 3.1 801.5

When ```other``` is a `:class:`Series``, the index of ```other``` is aligned with the
columns of the DataFrame.

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
 height weight
elk 3.0 500.5
moose 4.1 800.5

Even when the index of ```other``` is the same as the index of the DataFrame,
the `:class:`Series`` will not be reoriented. If index-wise alignment is desired,
`:meth:`DataFrame.add`` should be used with ```axis='index'```.

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
 elk height moose weight
elk NaN NaN NaN NaN
moose NaN NaN NaN NaN

>>> df[["height", "weight"]].add(s2, axis="index")
 height weight

elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
```

	height	weight
deer	NaN	NaN
elk	1.7	NaN
moose	3.0	NaN

```
__and__(self, other)
__divmod__(self, other)
__eq__(self, other)
    Return self==value.
__floordiv__(self, other)
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__mod__(self, other)
__mul__(self, other)
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__pow__(self, other)
__radd__(self, other)
__rand__(self, other)
__rdivmod__(self, other)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__(self, other)
__ror__(self, other)
    Return value|self.
__rpow__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__rxor__(self, other)
__sub__(self, other)
__truediv__(self, other)
__xor__(self, other)
```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None

Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

__getitem__(self, key: PositionalIndexer2D) -> Self | Any
 Select a subset of self.

Parameters

item : int, slice, or ndarray
 * int: The position in 'self' to get.

 * slice: A slice object, where 'start', 'stop', and 'step' are integers or None

 * ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

 * list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

__setitem__(self, key, value) -> None
 Set one or more values inplace.

This method is not required to satisfy the pandas extension array interface.

Parameters

key : int, ndarray, or slice
 When called from, e.g. ``Series.\_\_setitem\_\_``, ``key`` will be one of

 * scalar int
 * ndarray of integers.
 * boolean ndarray
 * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
 value or values to be set of ``key``.

Returns

None

Raises

ValueError

 If the array is readonly and modification is attempted.

argmax(self, axis: AxisInt = 0, skipna: bool = True)

Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the minimum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

argmin(self, axis: AxisInt = 0, skipna: bool = True)

Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)
```

equals(self, other) -> bool

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean

whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.

Series.equals : Equivalent method for Series.

DataFrame.equals : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
 Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
 If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.
`limit` : int, default None
 The maximum number of entries where NA values will be filled.
`copy` : bool, default True
 Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

 ExtensionArray
 With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.
`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64
```

`insert(self, loc: int, item) -> Self`
 Make new ExtensionArray inserting new item at location. Follows Python list.append semantics for negative values.

Parameters

`loc` : int
`item` : object

Returns

`type(self)`

`searchsorted(self, value: NumpyValueArrayLike | ExtensionArray, side: Literal['left', 'right'] = 'left', sorter: NumpySorter | None = None) -> npt.NDArray[np.intp] | np.intp`
 Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side`  returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

```

-----
value : array-like, list or scalar
        Value(s) to insert into `self`.
side : {'left', 'right'}, optional
        If 'left', the index of the first suitable location found is given.
        If 'right', return the last such index. If there is no suitable
        index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
        Optional array of integer indices that sort array a into ascending
        order. They are typically the result of argsort.

Returns
-----
array of ints or int
        If value is array-like, array of insertion points.
        If value is scalar, a single integer.

See Also
-----
numpy.searchsorted : Similar method from NumPy.

Examples
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
    Shift values by desired number.

    Newly introduced missing values are filled with
    ``self.dtype.na_value``.

Parameters
-----
periods : int, default 1
    The number of periods to shift. Negative values are allowed
    for shifting backwards.

fill_value : object, optional
    The scalar value to use for newly introduced missing values.
    The default is ``self.dtype.na_value``.

Returns
-----
ExtensionArray
    Shifted.

See Also
-----
api.extensions.ExtensionArray.transpose : Return a transposed view on
    this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
    enumerated type.

Notes
-----
If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
    returned.

If ``periods > len(self)`` , then an array of size
    len(self) is returned, with all values filled with
    ``self.dtype.na_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

unique(self) -> Self
    Compute the ExtensionArray of unique values.

```

Returns

pandas.api.extensions.ExtensionArray
With unique values from the input array.

See Also

Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples

```
>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

value_counts(self, dropna: bool = True) -> Series
Return a Series containing counts of unique values.

Parameters

dropna : bool, default True
Don't include counts of NA values.

Returns

Series

view(self, dtype: Dtype | None = None) -> ArrayLike
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Methods inherited from pandas._libs.arrays.NDArrayBacked:

```
__len__(self, /)
    Return len(self).

__reduce__ = __reduce_cython__(...)

__reduce_cython__(self)

__setstate__(self, state)

__setstate_cython__(self, __pyx_state)
```

```

copy(self, order='C')
delete(self, loc, axis=0)
ravel(self, order='C')
repeat(self, repeats, axis: int | np.integer = 0)
reshape(self, *args, **kwargs)
swapaxes(self, axis1, axis2)
transpose(self, *axes)

-----
Static methods inherited from pandas._libs.arrays.NDArrayBacked:
__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
    Create and return a new object. See help(type) for accurate signature.
-----
Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:
T
nbytes
ndim
shape
size
-----
Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:
__pyx_vtable__ = <capsule object NULL>
-----
Methods inherited from pandas.api.extensions.ExtensionArray:
__contains__(self, item: object) -> bool | np.bool_ from pandas.core.arrays.base.ExtensionArray
    Return for `item in self`.
__iter__(self) -> Iterator[Any] from pandas.core.arrays.base.ExtensionArray
    Iterate over elements of the array.
__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).
argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
    Return the indices that would sort this array.

Parameters
-----
ascending : bool, default True
    Whether the indices should result in an ascending
    or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
    Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
    If ``'first'``, put ``NaN`` values at the beginning.
    If ``'last'``, put ``NaN`` values at the end.
**kwargs
    Passed through to :func:`numpy.argsort`.

Returns
-----
np.ndarray[np.intp]

```

Array of indices that sort ``self``. If NaN values are contained, NaN values are placed at the end.

See Also

numpy.argsort : Sorting implementation used internally.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all NA values removed.

See Also

Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in an ExtensionArray.

Examples

>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64

deduplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] from pandas.core.arrays.base.ExtensionArray
Return boolean ndarray denoting duplicate values.

Parameters

keep : {'first', 'last', False}, default 'first'
- ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
- ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

Returns

ndarray[bool]
With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").deduplicated()
array([False, True, False, False, True])

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas.core.arrays.base.ExtensionArray
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray

An integer NumPy array that's an indexer into the original ExtensionArray.

uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

>>> idx1 = pd.PeriodIndex(
... ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
... freq="M",
...)
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_] from pandas.core.arrays.base.ExtensionArray
Pointwise comparison for set containment in the given values.

Roughly equivalent to `np.array([x in values for x in self])`

Parameters

values : np.ndarray or ExtensionArray
Values to compare every element in the array against.

Returns

np.ndarray[bool]
With true at indices where value is in `values`.

See Also

DataFrame.isin: Whether each element in the DataFrame is contained in values.
Index.isin: Return a boolean array where the index values are in values.
Series.isin: Whether elements in Series are contained in values.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar
The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

`IndexError`

If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)
```

`map(self, mapper, na_action: Literal['ignore'] | None = None)` from `pandas.core.arrays.base.ExtensionArray`
Map values using an input mapping or function.

Parameters

`mapper` : function, dict, or Series

Mapping correspondence.

`na_action` : {None, 'ignore'}, default None

If 'ignore', propagate NA values, without passing them to the mapping correspondence. If 'ignore' is not supported, a `NotImplementedError` should be raised.

Returns

`Union[ndarray, Index, ExtensionArray]`

The output of the mapping function applied to the array.

If the function returns a tuple with more than one element a `MultiIndex` will be returned.

`tolist(self)` -> list from `pandas.core.arrays.base.ExtensionArray`
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list

Python list of values in array.

See Also

`Index.tolist`: Return a list of the values in the Index.

`Series.tolist`: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]
```

Data and other attributes inherited from `pandas.api.extensions.ExtensionArray`:

`__pandas_priority__` = 1000

`class PeriodArray(pandas.core.arrays.datetimelike.DatelikeOps, pandas._libs.tslibs.period.PeriodArray)`
`PeriodArray(values, dtype: Dtype | None = None, copy: bool = False) -> None`

Pandas `ExtensionArray` for storing Period data.

Users should use `:func:`~pandas.array`` to create new instances.

Parameters

values : Union[PeriodArray, Series[period], ndarray[int], PeriodIndex]
The data to store. These should be arrays that can be directly converted to ordinals without inference or copy (PeriodArray, ndarray[int64]), or a box around such an array (Series[period], PeriodIndex).

dtype : PeriodDtype, optional
A PeriodDtype instance from which to extract a `freq`. If both `freq` and `dtype` are specified, then the frequencies must match.

copy : bool, default False
Whether to copy the ordinals before storing.

Attributes

None

Methods

None

See Also

Period: Represents a period of time.
PeriodIndex : Immutable Index for period data.
period_range: Create a fixed-frequency PeriodArray.
array: Construct a pandas array.

Notes

There are two components to a PeriodArray

- ordinals : integer ndarray
- freq : pd.tseries.offsets.Offset

The values are physically stored as a 1-D ndarray of integers. These are called "ordinals" and represent some kind of offset from a base.

The `freq` indicates the span covered by each element of the array. All elements in the PeriodArray have the same `freq`.

Examples

>>> pd.arrays.PeriodArray(pd.PeriodIndex(["2023-01-01", "2023-01-02"], freq="D"))
<PeriodArray>
['2023-01-01', '2023-01-02']
Length: 2, dtype: period[D]

Method resolution order:

PeriodArray
pandas.core.arrays.datetimelike.DatelikeOps
pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin
pandas.core.arraylike.OpsMixin
pandas.core.arrays._mixins.NDArrayBackedExtensionArray
pandas._libs.arrays.NDArrayBacked
pandas.api.extensions.ExtensionArray
pandas._libs.tslibs.period.PeriodMixin
builtins.object

Methods defined here:

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.period.PeriodArray

__arrow_array__(self, type=None) from pandas.core.arrays.period.PeriodArray
Convert myself into a pyarrow Array.

__init__(self, values, dtype: Dtype | None = None, copy: bool = False) -> None from pandas.core.arrays.period.PeriodArray
Initialize self. See help(type(self)) for accurate signature.

asfreq(self, freq=None, how: str = 'E') -> Self from pandas.core.arrays.period.PeriodArray
Convert the PeriodArray to the specified frequency `freq`.

Equivalent to applying :meth:`pandas.Period.asfreq` with the given arguments to each :class:`~pandas.Period` in this PeriodArray.

Parameters

freq : str
A frequency.

how : str {'E', 'S'}, default 'E'
Whether the elements should be aligned to the end
or start within a period.

* 'E', 'END', or 'FINISH' for end,
* 'S', 'START', or 'BEGIN' for start.

January 31st ('END') vs. January 1st ('START') for example.

Returns

PeriodArray
The transformed PeriodArray with the new frequency.

See Also

PeriodIndex.asfreq: Convert each Period in a PeriodIndex to the given frequency.
Period.asfreq : Convert a :class:`~pandas.Period` object to the given frequency.

Examples

>>> pidx = pd.period_range("2010-01-01", "2015-01-01", freq="Y")
>>> pidx
PeriodIndex(['2010', '2011', '2012', '2013', '2014', '2015'],
dtype='period[Y-DEC]')

>>> pidx.asfreq("M")
PeriodIndex(['2010-12', '2011-12', '2012-12', '2013-12', '2014-12',
'2015-12'], dtype='period[M]')

>>> pidx.asfreq("M", how="S")
PeriodIndex(['2010-01', '2011-01', '2012-01', '2013-01', '2014-01',
'2015-01'], dtype='period[M]')

astype(self, dtype, copy: bool = True) from pandas.core.arrays.period.PeriodArray
Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters

dtype : str or dtype
Typecode or data-type to which the array is cast.
copy : bool, default True
Whether to copy the data, even if not necessary. If False,
a copy is made only if the old dtype does not match the
new dtype.

Returns

np.ndarray or pandas.api.extensions.ExtensionArray
An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also

Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
a sequence.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64

```
>>> arr1.dtype
Float64Dtype()
```

Otherwise, we will get a Numpy ndarray:

```
>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')
```

```
searchsorted(
    self,
    value: NumpyValueArrayLike | ExtensionArray,
    side: Literal['left', 'right'] = 'left',
    sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.arrays.period.PeriodArray
Find indices where elements should be inserted to maintain order.
```

Find the indices into a sorted array `self` (a) such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into `self`.

side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given. If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).

sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

```
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])
```

to_timestamp(self, freq=None, how: str = 'start') -> DatetimeArray from pandas.core.arrays.p
Cast to DatetimeArray/Index.

If possible, gives microsecond-unit DatetimeArray/Index. Otherwise gives nanosecond unit.

Parameters

freq : str or DateOffset, optional
Target frequency. The default is 'D' for week or longer, 's' otherwise.

how : {'s', 'e', 'start', 'end'}
Whether to use the start or end of the time period being converted.

Returns

DatetimeArray/Index

Timestamp representation of given Period-like object.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.from_fields : Construct a PeriodIndex from fields
(year, month, day, etc.).
PeriodIndex.from_ordinals : Construct a PeriodIndex from ordinals.
PeriodIndex.hour : The hour of the period.
PeriodIndex.minute : The minute of the period.
PeriodIndex.month : The month as January=1, December=12.
PeriodIndex.second : The second of the period.
PeriodIndex.year : The year of the period.

Examples

>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.to_timestamp()
DatetimeIndex(['2023-01-01', '2023-02-01', '2023-03-01'],
dtype='datetime64[us]', freq='MS')

The frequency will not be inferred if the index contains less than
three elements, or if the values of index are not strictly monotonic:

>>> idx = pd.PeriodIndex(["2023-01", "2023-02"], freq="M")
>>> idx.to_timestamp()
DatetimeIndex(['2023-01-01', '2023-02-01'], dtype='datetime64[us]', freq=None)

>>> idx = pd.PeriodIndex(
... ["2023-01", "2023-02", "2023-02", "2023-03"], freq="2M"
...)
>>> idx.to_timestamp()
DatetimeIndex(['2023-01-01', '2023-02-01', '2023-02-01', '2023-03-01'],
dtype='datetime64[us]', freq=None)

Readonly properties defined here:

day

The days of the period.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.

Examples

>>> idx = pd.PeriodIndex(['2020-01-31', '2020-02-28'], freq='D')
>>> idx.day
Index([31, 28], dtype='int64')

day_of_week

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.

Examples

>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')

day_of_year

The ordinal day of the year.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-10", "2023-02-01", "2023-03-01"], freq="D")
>>> idx.dayofyear
Index([10, 32, 60], dtype='int64')

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')
>>> idx.dayofyear
Index([365, 366, 365], dtype='int64')
```

dayofweek

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')
```

dayofyear

The ordinal day of the year.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-10", "2023-02-01", "2023-03-01"], freq="D")
>>> idx.dayofyear
Index([10, 32, 60], dtype='int64')

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')
>>> idx.dayofyear
Index([365, 366, 365], dtype='int64')
```

days_in_month

The number of days in the month.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.
PeriodIndex.month : The month as January=1, December=12.

Examples

For Series:

```
>>> period = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.days_in_month
0     31
1     29
2     31
dtype: int64
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.days_in_month # It can be also entered as `daysinmonth`
Index([31, 28, 31], dtype='int64')
```

daysinmonth

The number of days in the month.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.
PeriodIndex.month : The month as January=1, December=12.

Examples

For Series:

```
>>> period = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.days_in_month
0     31
1     29
2     31
dtype: int64
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.days_in_month # It can be also entered as `daysinmonth`
Index([31, 28, 31], dtype='int64')
```

freq

Return the frequency object for this PeriodArray.

freqstr

Return the frequency object as a string if it's set, otherwise None.

See Also

DatetimeIndex.inferred_freq : Returns a string representing a frequency generated by infer_freq.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
```

```
>>> idx.freqstr
'D'
```

The frequency can be inferred if there are more than 2 points:

```
>>> idx = pd.DatetimeIndex(
...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
... )
>>> idx.freqstr
'2D'
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
>>> idx.freqstr
'M'
```

hour

The hour of the period.

See Also

PeriodIndex.minute : The minute of the period.
PeriodIndex.second : The second of the period.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01 10:00", "2023-01-01 11:00"], freq='h')
>>> idx.hour
Index([10, 11], dtype='int64')
```

is_leap_year

Logical indicating if the date belongs to a leap year.

See Also

PeriodIndex.qyear : Fiscal year the Period lies in according to its
starting-quarter.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx.is_leap_year
array([False,  True, False])
```

minute

The minute of the period.

See Also

PeriodIndex.hour : The hour of the period.
PeriodIndex.second : The second of the period.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01 10:30:00",
...                       "2023-01-01 11:50:00"], freq='min')
>>> idx.minute
Index([30, 50], dtype='int64')
```

month

The month as January=1, December=12.

See Also

PeriodIndex.days_in_month : The number of days in the month.
PeriodIndex.daysinmonth : The number of days in the month.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.month
Index([1, 2, 3], dtype='int64')
```


quarter

The quarter of the date.

See Also

PeriodIndex.qyear : Fiscal year the Period lies in according to its starting-quarter.

Examples

>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.quarter
Index([1, 1, 1], dtype='int64')

qyear

Fiscal year the Period lies in according to its starting-quarter.

The `year` and the `qyear` of the period will be the same if the fiscal and calendar years are the same. When they are not, the fiscal year can be different from the calendar year of the period.

Returns

int

The fiscal year of the period.

See Also

PeriodIndex.quarter : The quarter of the date.
PeriodIndex.year : The year of the period.

Examples

If the natural and fiscal year are the same, `qyear` and `year` will be the same.

```
>>> per = pd.Period('2018Q1', freq='Q')
>>> per.qyear
2018
>>> per.year
2018
```

If the fiscal year starts in April (`Q-MAR`), the first quarter of 2018 will start in April 2017. `year` will then be 2017, but `qyear` will be the fiscal year, 2018.

```
>>> per = pd.Period('2018Q1', freq='Q-MAR')
>>> per.start_time
Timestamp('2017-04-01 00:00:00')
>>> per.qyear
2018
>>> per.year
2017
```

second

The second of the period.

See Also

PeriodIndex.hour : The hour of the period.
PeriodIndex.minute : The minute of the period.
PeriodIndex.to_timestamp : Cast to DatetimeArray/Index.

Examples

>>> idx = pd.PeriodIndex(["2023-01-01 10:00:30",
... "2023-01-01 10:00:31"], freq='s')
>>> idx.second
Index([30, 31], dtype='int64')

week

The week ordinal of the year.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.

PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.week # It can be written `weekofyear`
Index([5, 9, 13], dtype='int64')
```

weekday

The day of the week with Monday=0, Sunday=6.

See Also

PeriodIndex.day : The days of the period.
PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.weekofyear : The week ordinal of the year.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01-01", "2023-01-02", "2023-01-03"], freq="D")
>>> idx.weekday
Index([6, 0, 1], dtype='int64')
```

weekofyear

The week ordinal of the year.

See Also

PeriodIndex.day_of_week : The day of the week with Monday=0, Sunday=6.
PeriodIndex.dayofweek : The day of the week with Monday=0, Sunday=6.
PeriodIndex.week : The week ordinal of the year.
PeriodIndex.weekday : The day of the week with Monday=0, Sunday=6.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.week # It can be written `weekofyear`
Index([5, 9, 13], dtype='int64')
```

year

The year of the period.

See Also

PeriodIndex.day_of_year : The ordinal day of the year.
PeriodIndex.dayofyear : The ordinal day of the year.
PeriodIndex.is_leap_year : Logical indicating if the date belongs to a leap year.
PeriodIndex.weekofyear : The week ordinal of the year.
PeriodIndex.year : The year of the period.

Examples

```
>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> idx.year
Index([2023, 2024, 2025], dtype='int64')
```

Data descriptors defined here:

dtype

Data and other attributes defined here:

__annotations__ = {'_bool_ops': 'list[str]', '_datetimelike_methods': ...

__array_priority__ = 1000

Methods inherited from pandas.core.arrays.datetimelike.DatelikeOps:

`strftime(self, date_format: str) -> npt.NDArray[np.object_]`
Convert to Index using specified date_format.

Return an Index of formatted strings specified by date_format, which supports the same string format as the python standard library. Details of the string format can be found in ``python string format doc <https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>``.

Formats supported by the C ``strftime`` API but not by the python string format doc (such as ``"%R"`, `"%r"```) are not officially supported and should be preferably replaced with their supported equivalents (such as ``"%H:%M"`, `"%I:%M:%S %p"```).

Note that ``PeriodIndex`` support additional directives, detailed in ``Period.strftime``.

Parameters

`date_format : str`
Date format string (e.g. `"%Y-%m-%d"`).

Returns

`ndarray[object]`
NumPy ndarray of formatted strings.

See Also

`to_datetime` : Convert the given argument to datetime.
`DatetimeIndex.normalize` : Return DatetimeIndex with times to midnight.
`DatetimeIndex.round` : Round the DatetimeIndex to the specified freq.
`DatetimeIndex.floor` : Floor the DatetimeIndex to the specified freq.
`Timestamp.strftime` : Format a single Timestamp.
`Period.strftime` : Format a single Period.

Examples

```
>>> rng = pd.date_range(pd.Timestamp("2018-03-10 09:00"), periods=3, freq="s")
>>> rng.strftime("%B %d, %Y, %r")
Index(['March 10, 2018, 09:00:00 AM', 'March 10, 2018, 09:00:01 AM',
       'March 10, 2018, 09:00:02 AM'],
      dtype='str')
```

Methods inherited from pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin:

`__add__(self, other)`
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

`other : scalar, sequence, Series, dict or DataFrame`
Object to be added to the DataFrame.

Returns

`DataFrame`
The result of adding ``other`` to DataFrame.

See Also

`DataFrame.add` : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
... )
>>> df
      height  weight
elk      1.5    500
moose    2.6    800
```

```

elk      1.5      500
moose    2.6      800

```

Adding a scalar affects all rows and columns.

```

>>> df[["height", "weight"]] + 1.5
      height  weight
elk        3.0   501.5
moose      4.1   801.5

```

Each element of a list is added to a column of the DataFrame, in order.

```

>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0   501.5
moose      3.1   801.5

```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0   501.5
moose      3.1   801.5

```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0   500.5
moose      4.1   800.5

```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN      NaN    NaN     NaN
moose  NaN      NaN    NaN     NaN

```

```

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0   500.5
moose      4.1   801.5

```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```

>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN      NaN
elk        1.7      NaN
moose      3.0      NaN

```

`__divmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make`

`__floordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.ma`

`__getitem__(self, key: PositionalIndexer2D) -> Self | DTScalarOrNaT`
This `getitem` defers to the underlying array, which by-definition can only handle list-likes, slices, and integer scalars

`__iadd__(self, other) -> Self`

`__isub__(self, other) -> Self`

`__iter__(self) -> Iterator`
Iterate over elements of the array.

`__mod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in`

```

__mul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__pow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__radd__(self, other)
__rdivmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak
__rfloordiv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.m
__rmod__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__rmul__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__rpow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__rsub__(self, other)
__rtruediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak

__setitem__(
    self,
    key: int | Sequence[int] | Sequence[bool] | slice,
    value: NaTType | Any | Sequence[Any]
) -> None
    Set one or more values inplace.

    This method is not required to satisfy the pandas extension array
    interface.

    Parameters
    -----
    key : int, ndarray, or slice
        When called from, e.g. ``Series.__setitem__``, ``key`` will be
        one of

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

    value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
        value or values to be set of ``key``.

    Returns
    -----
    None

    Raises
    -----
    ValueError
        If the array is readonly and modification is attempted.

__sub__(self, other)

__truediv__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.mak

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]
    Compute boolean array of whether each value is found in the
    passed set of values.

    Parameters
    -----
    values : np.ndarray or ExtensionArray

    Returns
    -----
    ndarray[bool]

isna(self) -> npt.NDArray[np.bool_]
    A 1-D array indicating if each value is missing.

    Returns
    -----
    numpy.ndarray or pandas.api.extensions.ExtensionArray
        In most cases, this should return a NumPy ndarray. For

```

exceptional cases like ``SparseArray``, where returning an ndarray would be expensive, an ExtensionArray may be returned.

See Also

ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.

Notes

If returning an ExtensionArray, then

- * ``na\_values.\_is\_boolean`` should be True
- * ``na\_values`` should implement :func:`ExtensionArray.\_reduce`
- * ``na\_values`` should implement :func:`ExtensionArray.\_accumulate`
- * ``na\_values.any`` and ``na\_values.all`` should be implemented

Examples

>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False, True, True])

map(self, mapper, na_action: Literal['ignore'] | None = None)
Map values using an input mapping or function.

Parameters

mapper : function, dict, or Series
Mapping correspondence.
na_action : {None, 'ignore'}, default None
If 'ignore', propagate NA values, without passing them to the mapping correspondence. If 'ignore' is not supported, a ``NotImplementedError`` should be raised.

Returns

Union[ndarray, Index, ExtensionArray]
The output of the mapping function applied to the array.
If the function returns a tuple with more than one element a MultiIndex will be returned.

max(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)
Return the maximum value of the Array or maximum along an axis.

See Also

numpy.ndarray.max
Index.max : Return the maximum value in an Index.
Series.max : Return the maximum value in a Series.

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0)
Return the mean value of the Array.

Parameters

skipna : bool, default True
Whether to ignore any NaT elements.
axis : int, optional, default 0
Axis for the function to be applied on.

Returns

scalar
Timestamp or Timedelta.

See Also

numpy.ndarray.mean : Returns the average of array elements along a given axis.
Series.mean : Return the mean value in a Series.

Notes

mean is only defined for Datetime and Timedelta dtypes, not for Period.

Examples

For :class:`pandas.DatetimeIndex`:

```
>>> idx = pd.date_range("2001-01-01 00:00", periods=3)
>>> idx
DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
              dtype='datetime64[us]', freq='D')
>>> idx.mean()
Timestamp('2001-01-02 00:00:00')
```

For :class:`pandas.TimedeltaIndex`:

```
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
              dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.mean()
Timedelta('2 days 00:00:00')
```

`median(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)`

`min(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)`
Return the minimum value of the Array or minimum along an axis.

See Also

`numpy.ndarray.min`
`Index.min` : Return the minimum value in an Index.
`Series.min` : Return the minimum value in a Series.

`view(self, dtype: Dtype | None = None) -> ArrayLike`
Return a view on the array.

Parameters

`dtype` : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

`ExtensionArray` or `np.ndarray`
A view on the :class:`ExtensionArray`'s data.

See Also

`api.extensions.ExtensionArray.ravel`: Return a flattened view on input array.
`Index.view`: Equivalent function for Index.
`ndarray.view`: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from `pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin`:

asi8

Integer representation of the values.

Returns

`ndarray`
An ndarray with int64 dtype.

`inferred_freq`

Tries to return a string representing a frequency generated by infer_freq.

Returns None if it can't autodetect the frequency.

See Also

DatetimeIndex.freqstr : Return the frequency object as a string if it's set, otherwise None.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
>>> idx.inferred_freq
'2D'
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
                dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.inferred_freq
'10D'
```

resolution

Returns day, hour, minute, second, millisecond or microsecond

Methods inherited from pandas.core.arraylike.OpsMixin:

__and__(self, other)

__eq__(self, other)
Return self==value.

__ge__(self, other)
Return self>=value.

__gt__(self, other)
Return self>value.

__le__(self, other)
Return self<=value.

__lt__(self, other)
Return self<value.

__ne__(self, other)
Return self!=value.

__or__(self, other)
Return self|value.

__rand__(self, other)

__ror__(self, other)
Return value|self.

__rxor__(self, other)

__xor__(self, other)

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

__hash__ = None

Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

`argmax(self, axis: AxisInt = 0, skipna: bool = True)`
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

`skipna` : bool, default True

Returns

int

See Also

`ExtensionArray.argmax` : Return the index of the minimum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

`argmin(self, axis: AxisInt = 0, skipna: bool = True)`
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

`skipna` : bool, default True

Returns

int

See Also

`ExtensionArray.argmin` : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

`equals(self, other) -> bool`

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

`other` : ExtensionArray
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

`numpy.array_equal` : Equivalent method for numpy array.
`Series.equals` : Equivalent method for Series.
`DataFrame.equals` : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
 Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
 If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.
`limit` : int, default None
 The maximum number of entries where NA values will be filled.
`copy` : bool, default True
 Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

 ExtensionArray
 With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.
`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64
```

`insert(self, loc: int, item) -> Self`
 Make new ExtensionArray inserting new item at location. Follows Python list.append semantics for negative values.

Parameters

`loc` : int
`item` : object

Returns

`type(self)`

`shift(self, periods: int = 1, fill_value=None) -> Self`
 Shift values by desired number.

Newly introduced missing values are filled with
 ``self.dtype.na\_value``.

Parameters

`periods` : int, default 1
 The number of periods to shift. Negative values are allowed for shifting backwards.
`fill_value` : object, optional
 The scalar value to use for newly introduced missing values. The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`` , then an array of size len(self) is returned, with all values filled with ``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

```
take(  
    self,  
    indices: TakeIndexer,  
    *,  
    allow_fill: bool = False,  
    fill_value: Any = None,  
    axis: AxisInt = 0  
) -> Self  
    Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.

allow_fill : bool, default False
How to handle negative values in `indices`.

* False: negative values in `indices` indicate positional indices from the right (the default). This is similar to :func:`numpy.take`.

* True: negative values in `indices` indicate missing values. These values are set to `fill\_value`. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
Fill value to use for NA-indices when `allow\_fill` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of `fill\_value`: a user-facing "boxed" scalar, and a low-level physical NA value. `fill\_value` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray
An array formed with selected `indices`.

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When `indices` contains negative values other than ``-1``

and `allow\_fill` is True.

See Also

numpy.take : Take elements from an array along an axis.
api.extensions.take : Take elements from an array.

Notes

ExtensionArray.take is called by ``Series.\_\_getitem\_\_``, ``.loc``,
``.iloc``, when `indices` is a sequence of values. Additionally,
it's called by :meth:`Series.reindex`, or any other method
that causes realignment, with a `fill\_value`.

Examples

Here's an example implementation, which relies on casting the
extension array to object dtype. This uses the helper method
:func:`pandas.api.extensions.take`.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

unique(self) -> Self
Compute the ExtensionArray of unique values.

Returns

pandas.api.extensions.ExtensionArray
With unique values from the input array.

See Also

Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples

>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

value_counts(self, dropna: bool = True) -> Series
Return a Series containing counts of unique values.

Parameters

dropna : bool, default True
Don't include counts of NA values.

Returns

Series

Methods inherited from pandas._libs.arrays.NDArrayBacked:

```

__len__(self, /)
    Return len(self).

__reduce__ = __reduce_cython__(...)
__reduce_cython__(self)
__setstate__(self, state)
__setstate_cython__(self, __pyx_state)

copy(self, order='C')
delete(self, loc, axis=0)
ravel(self, order='C')
repeat(self, repeats, axis: int | np.integer = 0)
reshape(self, *args, **kwargs)
swapaxes(self, axis1, axis2)
transpose(self, *axes)

-----
Static methods inherited from pandas._libs.arrays.NDArrayBacked:

__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
    Create and return a new object.  See help(type) for accurate signature.

-----
Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:

T
nbytes
ndim
shape
size

-----
Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:

__pyx_vtable__ = <capsule object NULL>

-----
Methods inherited from pandas.api.extensions.ExtensionArray:

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.array
__contains__(self, item: object) -> bool | np.bool_ from pandas.core.arrays.base.ExtensionAr
    Return for `item in self`.

__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).

argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
    Return the indices that would sort this array.

Parameters
-----
ascending : bool, default True
    Whether the indices should result in an ascending
    or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional

```

```

        Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
    If ``'first'``, put ``NaN`` values at the beginning.
    If ``'last'``, put ``NaN`` values at the end.
**kwargs
    Passed through to :func:`numpy.argsort`.

Returns
-----
np.ndarray[np.intp]
    Array of indices that sort ``self``. If NaN values are contained,
    NaN values are placed at the end.

See Also
-----
numpy.argsort : Sorting implementation used internally.

Examples
-----
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
    Return ExtensionArray without NA values.

Returns
-----
Self
    An ExtensionArray of the same type as the original but with all
    NA values removed.

See Also
-----
Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in
    an ExtensionArray.

Examples
-----
>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]
    Return boolean ndarray denoting duplicate values.

Parameters
-----
keep : {'first', 'last', False}, default 'first'
    - ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
    - ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
    - False : Mark all duplicates as ``True``.

Returns
-----
ndarray[bool]
    With true in indices where elements are duplicated and false otherwise.

See Also
-----
DataFrame.duplicated : Return boolean Series denoting
    duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray
    of unique values.

Examples
-----
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas
    Encode the extension array as an enumerated type.

```

Parameters

`use_na_sentinel` : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

`codes` : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
`uniques` : ExtensionArray
An ExtensionArray containing the unique values of ``self``.

.. note::

uniques will **not** contain an entry for the NA value of the ExtensionArray if there are any missing values present in ``self``.

See Also

`factorize` : Top-level factorize method that dispatches here.

Notes

`:meth:`pandas.factorize`` offers a ``sort`` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')
```

```
interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.base.ExtensionArray
Fill NaN values using an interpolation method.
```

Parameters

`method` : str, default 'linear'
Interpolation technique to use. One of:
* 'linear': Ignore the index and treat the values as equally spaced. This is the only method supported on MultiIndexes.
* 'time': Works on daily and higher resolution data to interpolate given length of interval.
* 'index', 'values': use the actual numerical values of the index.
* 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric', 'polynomial': Passed to `scipy.interpolate.interp1d`, whereas 'spline' is passed to `scipy.interpolate.UnivariateSpline`. These methods use the numerical values of the index.
Both 'polynomial' and 'spline' require that you also specify an order (int), e.g. `arr.interpolate(method='polynomial', order=5)`.
* 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima', 'cubicspline': Wrappers around the SciPy interpolation methods of similar names. See Notes.
* 'from_derivatives': Refers to `scipy.interpolate.BPoly.from_derivatives`.

`axis` : int
Axis to interpolate along. For 1-dimensional data, use 0.

`index` : Index

Index to use for interpolation.
limit : int or None
Maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction : {'forward', 'backward', 'both'}
Consecutive NaNs will be filled in this direction.
limit_area : {'inside', 'outside'} or None
If limit is specified, consecutive NaNs will be filled with this restriction.
* None: No fill restriction.
* 'inside': Only fill NaNs surrounded by valid values (interpolate).
* 'outside': Only fill NaNs outside valid values (extrapolate).
copy : bool
If True, a copy of the object is returned with interpolated values.
**kwargs : optional
Keyword arguments to pass on to the interpolating function.

Returns

ExtensionArray
An ExtensionArray with interpolated values.

See Also

Series.interpolate : Interpolate values in a Series.
DataFrame.interpolate : Interpolate values in a DataFrame.

Notes

- All parameters must be specified as keyword arguments.
- The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima' methods are wrappers around the respective SciPy implementations of similar names. These use the actual numerical values of the index.

Examples

Interpolating values in a NumPy array:

```
>>> arr = pd.arrays.NumpyExtensionArray(np.array([0, 1, np.nan, 3]))
>>> arr.interpolate(
...     method="linear",
...     limit=3,
...     limit_direction="forward",
...     index=pd.Index(range(len(arr))),
...     fill_value=1,
...     copy=False,
...     axis=0,
...     limit_area="inside",
... )
<NumpyExtensionArray>
[0.0, 1.0, 2.0, 3.0]
Length: 4, dtype: float64
```

Interpolating values in a FloatingArray:

```
>>> arr = pd.array([1.0, pd.NA, 3.0, 4.0, pd.NA, 6.0], dtype="Float64")
>>> arr.interpolate(
...     method="linear",
...     axis=0,
...     index=pd.Index(range(len(arr))),
...     limit=None,
...     limit_direction="both",
...     limit_area=None,
...     copy=True,
... )
<FloatingArray>
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Length: 6, dtype: Float64
```

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar

The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

IndexError

If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
```

```
>>> arr.item()
```

```
np.int64(1)
```

```
>>> arr = pd.array([1, 2, 3], dtype="Int64")
```

```
>>> arr.item(0)
```

```
np.int64(1)
```

```
>>> arr.item(2)
```

```
np.int64(3)
```

to_numpy(
self,

dtype: npt.DTypeLike | None = None,
copy: bool = False,

na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray

Convert to a NumPy ndarray.

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

numpy.ndarray

tolist(self) -> list from pandas.core.arrays.base.ExtensionArray
Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

Returns

list

Python list of values in array.

See Also

Index.tolist: Return a list of the values in the Index.

Series.tolist: Return a list of the values in the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
```

```
>>> arr.tolist()
[1, 2, 3]
```

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

```
__pandas_priority__ = 1000
```

Data descriptors inherited from pandas._libs.tslibs.period.PeriodMixin:

end_time

Get the Timestamp for the end of the period.

Returns

Timestamp

See Also

Period.start_time : Return the start Timestamp.

Period.dayofyear : Return the day of year.

Period.daysinmonth : Return the days in that month.

Period.dayofweek : Return the day of the week.

Examples

For Period:

```
>>> pd.Period('2020-01', 'D').end_time
Timestamp('2020-01-01 23:59:59.999999')
```

For Series:

```
>>> period_index = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period_index)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.end_time
0    2020-01-31 23:59:59.999999
1    2020-02-29 23:59:59.999999
2    2020-03-31 23:59:59.999999
dtype: datetime64[us]
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.end_time
DatetimeIndex(['2023-01-31 23:59:59.999999',
               '2023-02-28 23:59:59.999999',
               '2023-03-31 23:59:59.999999'],
              dtype='datetime64[us]', freq=None)
```

start_time

Get the Timestamp for the start of the period.

Returns

Timestamp

See Also

Period.end_time : Return the end Timestamp.

Period.dayofyear : Return the day of year.

Period.daysinmonth : Return the days in that month.

Period.dayofweek : Return the day of the week.

Examples

```
>>> period = pd.Period('2012-1-1', freq='D')
>>> period
Period('2012-01-01', 'D')
```

```
>>> period.start_time
```

```

|         Timestamp('2012-01-01 00:00:00')
|
|         >>> period.end_time
|         Timestamp('2012-01-01 23:59:59.999999')
|
class SparseArray(pandas.core.arraylike.OpsMixin, pandas.core.base.PandasObject, pandas.api.extensions.SparseArray(
|     data,
|     sparse_index=None,
|     fill_value=None,
|     kind: SparseIndexKind = 'integer',
|     dtype: Dtype | None = None,
|     copy: bool = False
| ) -> None

| An ExtensionArray for storing sparse data.
|
| SparseArray efficiently stores data with a high frequency of a
| specific fill value (e.g., zeros), saving memory by only retaining
| non-fill elements and their indices. This class is particularly
| useful for large datasets where most values are redundant.
|
| Parameters
| -----
| data : array-like or scalar
|     A dense array of values to store in the SparseArray. This may contain
|     ``fill_value``.
| sparse_index : SparseIndex, optional
|     Index indicating the locations of sparse elements.
| fill_value : scalar, optional
|     Elements in data that are ``fill_value`` are not stored in the
|     SparseArray. For memory savings, this should be the most common value
|     in ``data``. By default, ``fill_value`` depends on the dtype of ``data``:
|
|     =====
|     data.dtype  na_value
|     =====
|     float       ``np.nan``
|     int         ``0``
|     bool        False
|     datetime64  ``pd.NaT``
|     timedelta64 ``pd.NaT``
|     =====
|
|     The fill value is potentially specified in three ways. In order of
|     precedence, these are
|
|     1. The ``fill_value`` argument
|     2. ``dtype.fill_value`` if ``fill_value`` is None and ``dtype`` is
|        a ``SparseDtype``
|     3. ``data.dtype.fill_value`` if ``fill_value`` is None and ``dtype``
|        is not a ``SparseDtype`` and ``data`` is a ``SparseArray``.
|
| kind : str
|     Can be 'integer' or 'block', default is 'integer'.
|     The type of storage for sparse locations.
|
|     * 'block': Stores a ``block`` and ``block_length`` for each
|       contiguous *span* of sparse values. This is best when
|       sparse data tends to be clumped together, with large
|       regions of ``fill-value`` values between sparse values.
|     * 'integer': uses an integer to store the location of
|       each sparse value.
|
| dtype : np.dtype or SparseDtype, optional
|     The dtype to use for the SparseArray. For numpy dtypes, this
|     determines the dtype of ``self.sp_values``. For SparseDtype,
|     this determines ``self.sp_values`` and ``self.fill_value``.
| copy : bool, default False
|     Whether to explicitly copy the incoming ``data`` array.
|
| Attributes
| -----
| None
|
| Methods
| -----

```

None

See Also

SparseDtype : Dtype for sparse data.

Examples

>>> from pandas.arrays import SparseArray
>>> arr = SparseArray([0, 0, 1, 2])
>>> arr
[0, 0, 1, 2]
Fill: 0
IntIndex
Indices: array([2, 3], dtype=int32)

Method resolution order:

SparseArray
pandas.core.arraylike.OpsMixin
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
pandas.api.extensions.ExtensionArray
builtins.object

Methods defined here:

__abs__(self) -> SparseArray from pandas.core.arrays.sparse.array.SparseArray

__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.sparse.array.SparseArray

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.arrays.sparse.array.SparseArray

__getitem__(self, key: PositionalIndexer | tuple[int | EllipsisType, ...]) -> Self | Any from pandas.core.arrays.sparse.array.SparseArray
Select a subset of self.

Parameters

item : int, slice, or ndarray
* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

item : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

__init__(
self,
data,
sparse_index=None,
fill_value=None,
kind: SparseIndexKind = 'integer',
dtype: Dtype | None = None,
copy: bool = False
) -> None from pandas.core.arrays.sparse.array.SparseArray
Initialize self. See help(type(self)) for accurate signature.

__invert__(self) -> SparseArray from pandas.core.arrays.sparse.array.SparseArray

__len__(self) -> int from pandas.core.arrays.sparse.array.SparseArray
Length of this array

```

Returns
-----
length : int

__neg__(self) -> SparseArray from pandas.core.arrays.sparse.array.SparseArray
__pos__(self) -> SparseArray from pandas.core.arrays.sparse.array.SparseArray
__repr__(self) -> str from pandas.core.arrays.sparse.array.SparseArray
Return repr(self).

__setitem__(self, key, value) -> None from pandas.core.arrays.sparse.array.SparseArray
Set one or more values inplace.

This method is not required to satisfy the pandas extension array
interface.

Parameters
-----
key : int, ndarray, or slice
    When called from, e.g. ``Series.__setitem__``, ``key`` will be
    one of

    * scalar int
    * ndarray of integers.
    * boolean ndarray
    * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
    value or values to be set of ``key``.

Returns
-----
None

Raises
-----
ValueError
    If the array is readonly and modification is attempted.

__setstate__(self, state) -> None from pandas.core.arrays.sparse.array.SparseArray
Necessary for making this object picklable

all(self, axis=None, *args, **kwargs) from pandas.core.arrays.sparse.array.SparseArray
Tests whether all elements evaluate True

Returns
-----
all : bool

See Also
-----
numpy.all

any(self, axis: AxisInt = 0, *args, **kwargs) -> bool from pandas.core.arrays.sparse.array.S
Tests whether at least one of elements evaluate True

Returns
-----
any : bool

See Also
-----
numpy.any

argmax(self, skipna: bool = True) -> int from pandas.core.arrays.sparse.array.SparseArray
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index
corresponding to the first occurrence is returned.

Parameters
-----
skipna : bool, default True

Returns

```

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

argmin(self, skipna: bool = True) -> int from pandas.core.arrays.sparse.array.SparseArray
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

astype(self, dtype: AstypeArg | None = None, copy: bool = True) from pandas.core.arrays.sparse.array.SparseArray
Change the dtype of a SparseArray.

The output will always be a SparseArray. To convert to a dense ndarray with a certain dtype, use :meth:`numpy.asarray`.

Parameters

dtype : np.dtype or ExtensionDtype
For SparseDtype, this changes the dtype of
``self.sp\_values`` and the ``self.fill\_value``.

For other dtypes, this only changes the dtype of
``self.sp\_values``.

copy : bool, default True

Whether to ensure a copy is made, even if not necessary.

Returns

SparseArray

Examples

>>> arr = pd.arrays.SparseArray([0, 0, 1, 2])
>>> arr
[0, 0, 1, 2]
Fill: 0
IntIndex
Indices: array([2, 3], dtype=int32)

>>> arr.astype(pd.SparseDtype(np.dtype("int32")))
[0, 0, 1, 2]
Fill: 0
IntIndex
Indices: array([2, 3], dtype=int32)

Using a NumPy dtype with a different kind (e.g. float) will coerce just ``self.sp\_values``.

```
>>> arr.astype(pd.SparseDtype(np.dtype("float64")))
... # doctest: +NORMALIZE_WHITESPACE
[nan, nan, 1.0, 2.0]
Fill: nan
IntIndex
Indices: array([2, 3], dtype=int32)
```

Using a SparseDtype, you can also change the fill value as well.

```
>>> arr.astype(pd.SparseDtype("float64", fill_value=0.0))
... # doctest: +NORMALIZE_WHITESPACE
[0.0, 0.0, 1.0, 2.0]
Fill: 0.0
IntIndex
Indices: array([2, 3], dtype=int32)
```

`copy(self)` -> Self from `pandas.core.arrays.sparse.array.SparseArray`
Return a copy of the array.

This method creates a copy of the ``ExtensionArray`` where modifying the data in the copy will not affect the original array. This is useful when you want to manipulate data without altering the original dataset.

Returns

`ExtensionArray`
A new ``ExtensionArray`` object that is a copy of the current instance.

See Also

`DataFrame.copy` : Return a copy of the DataFrame.
`Series.copy` : Return a copy of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

`cumsum(self, axis: AxisInt = 0, *args, **kwargs)` -> `SparseArray` from `pandas.core.arrays.sparse`
Cumulative sum of non-NA/null values.

When performing the cumulative summation, any non-NA/null values will be skipped. The resulting `SparseArray` will preserve the locations of NaN values, but the fill value will be ``np.nan`` regardless.

Parameters

`axis` : int or None
Axis over which to perform the cumulative summation. If None, perform cumulative summation over flattened array.

Returns

`cumsum` : `SparseArray`

`duplicated(self, keep: Literal['first', 'last', False] = 'first')` -> `npt.NDArray[np.bool_]` from `pandas.core.arrays.sparse`
Return boolean ndarray denoting duplicate values.

Parameters

`keep` : {'first', 'last', False}, default 'first'
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- `False` : Mark all duplicates as ``True``.

Returns

`ndarray[bool]`
With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

```
>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()  
array([False,  True, False, False,  True])
```

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, SparseArray] from pandas.
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

```
>>> idx1 = pd.PeriodIndex(  
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],  
...     freq="M",  
... )  
>>> arr, idx = idx1.factorize()  
>>> arr  
array([0, 0, 1, 1, 2, 2])  
>>> idx  
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')
```

fillna(self, value, limit: int | None = None, copy: bool = True) -> Self from pandas.core.array.
Fill missing values with `value`.

Parameters

value : scalar
limit : int, optional
Not supported for SparseArray, must be None.
copy: bool, default True
Ignored for SparseArray.

Returns

SparseArray

Notes

When `value` is specified, the result's ``fill\_value`` depends on ``self.fill\_value``. The goal is to maintain low-memory use.

If ``self.fill\_value`` is NA, the result dtype will be ``SparseDtype(self.dtype, fill\_value=value)``. This will preserve

amount of memory used before and after filling.

When ``self.fill\_value`` is not NA, the result dtype will be ``self.dtype``. Again, this preserves the amount of memory used.

`isna(self)` -> Self from `pandas.core.arrays.sparse.array.SparseArray`
A 1-D array indicating if each value is missing.

Returns

`numpy.ndarray` or `pandas.api.extensions.ExtensionArray`
In most cases, this should return a NumPy ndarray. For exceptional cases like ``SparseArray``, where returning an ndarray would be expensive, an `ExtensionArray` may be returned.

See Also

`ExtensionArray.dropna`: Return `ExtensionArray` without NA values.
`ExtensionArray.fillna`: Fill NA/NaN values using the specified method.

Notes

If returning an `ExtensionArray`, then

- * ``na\_values.\_is\_boolean`` should be True
- * ``na\_values`` should implement `:func:`ExtensionArray._reduce``
- * ``na\_values`` should implement `:func:`ExtensionArray._accumulate``
- * ``na\_values.any`` and ``na\_values.all`` should be implemented

Examples

```
>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False,  True,  True])
```

`map(self, mapper, na_action: Literal['ignore'] | None = None)` -> Self from `pandas.core.array`
Map categories using an input mapping or function.

Parameters

`mapper` : dict, Series, callable
The correspondence from old values to new.
`na_action` : {None, 'ignore'}, default None
If 'ignore', propagate NA values, without passing them to the mapping correspondence.

Returns

`SparseArray`
The output array will have the same density as the input.
The output fill value will be the result of applying the mapping to ``self.fill\_value``

Examples

```
>>> arr = pd.arrays.SparseArray([0, 1, 2])
>>> arr.map(lambda x: x + 10)
[10, 11, 12]
Fill: 10
IntIndex
Indices: array([1, 2], dtype=int32)

>>> arr.map({0: 10, 1: 11, 2: 12})
[10, 11, 12]
Fill: 10
IntIndex
Indices: array([1, 2], dtype=int32)

>>> arr.map(pd.Series([10, 11, 12], index=[0, 1, 2]))
[10, 11, 12]
Fill: 10
IntIndex
Indices: array([1, 2], dtype=int32)
```

`max(self, *, axis: AxisInt | None = None, skipna: bool = True)` from `pandas.core.arrays.sparse`
Max of array values, ignoring NA values if specified.

Parameters

axis : int, default 0
Not Used. NumPy compatibility.
skipna : bool, default True
Whether to ignore NA values.

Returns

scalar

mean(self, axis: Axis = 0, *args, **kwargs) from pandas.core.arrays.sparse.array.SparseArray
Mean of non-NA/null values

Returns

mean : float

min(self, *, axis: AxisInt | None = None, skipna: bool = True) from pandas.core.arrays.sparse.array.SparseArray
Min of array values, ignoring NA values if specified.

Parameters

axis : int, default 0
Not Used. NumPy compatibility.
skipna : bool, default True
Whether to ignore NA values.

Returns

scalar

nonzero(self) -> tuple[npt.NDArray[np.int32]] from pandas.core.arrays.sparse.array.SparseArray

searchsorted(
self,
v: ArrayLike | object,
side: Literal['left', 'right'] = 'left',
sorter: NumpySorter | None = None
) -> npt.NDArray[np.intp] | np.intp from pandas.core.arrays.sparse.array.SparseArray
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array `self` (a) such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

Assuming that `self` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into `self`.
side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

```
-----
>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])
```

shift(self, periods: int = 1, fill_value=None) -> Self from pandas.core.arrays.sparse.array.
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1
The number of periods to shift. Negative values are allowed
for shifting backwards.

fill_value : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on
this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
returned.

If ``periods > len(self)`` , then an array of size
len(self) is returned, with all values filled with
``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

```
-----
>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64
```

```
sum(
    self,
    axis: AxisInt = 0,
    min_count: int = 0,
    skipna: bool = True,
    *args,
    **kwargs
) -> Scalar from pandas.core.arrays.sparse.array.SparseArray
Sum of non-NA/null values
```

Parameters

axis : int, default 0
Not Used. NumPy compatibility.
min_count : int, default 0
The required number of valid values to perform the summation. If fewer
than ``min\_count`` valid values are present, the result will be the missing
value indicator for subarray type.
*args, **kwargs
Not Used. NumPy compatibility.

Returns

scalar

```
take(self, indices, *, allow_fill: bool = False, fill_value=None) -> Self from pandas.core.a
Take elements from an array.
```

Parameters

`indices` : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.

`allow_fill` : bool, default False
How to handle negative values in `indices`.

- * False: negative values in `indices` indicate positional indices from the right (the default). This is similar to `:func:'numpy.take'`.

- * True: negative values in `indices` indicate missing values. These values are set to `fill_value`. Any other other negative values raise a `ValueError`.

`fill_value` : any, optional
Fill value to use for NA-indices when `allow_fill` is True. This may be `None`, in which case the default NA value for the type, `self.dtype.na_value`, is used.

For many ExtensionArrays, there will be two representations of `fill_value`: a user-facing "boxed" scalar, and a low-level physical NA value. `fill_value` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray
An array formed with selected `indices`.

Raises

IndexError
When the indices are out of bounds for the array.

ValueError
When `indices` contains negative values other than `-1` and `allow_fill` is True.

See Also

`numpy.take` : Take elements from an array along an axis.
`api.extensions.take` : Take elements from an array.

Notes

ExtensionArray.take is called by `Series.__getitem__`, `Series.iloc`, `Series.iat`, when `indices` is a sequence of values. Additionally, it's called by `Series.reindex`, or any other method that causes realignment, with a `fill_value`.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method `pandas.api.extensions.take`.

```
.. code-block:: python
```

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.
```

```

        result = take(
            data, indices, fill_value=fill_value, allow_fill=allow_fill
        )
        return self._from_sequence(result, dtype=self.dtype)

to_dense(self) -> np.ndarray from pandas.core.arrays.sparse.array.SparseArray
Convert SparseArray to a NumPy array.

Returns
-----
arr : NumPy array

unique(self) -> Self from pandas.core.arrays.sparse.array.SparseArray
Compute the ExtensionArray of unique values.

Returns
-----
pandas.api.extensions.ExtensionArray
    With unique values from the input array.

See Also
-----
Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples
-----
>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.sparse.array.SparseArray
Returns a Series containing counts of unique values.

Parameters
-----
dropna : bool, default True
    Don't include counts of NaN, even if NaN is in sp_values.

Returns
-----
counts : Series

```

Class methods defined here:

```

from_spmatrix(data: _SparseMatrixLike) -> Self from pandas.core.arrays.sparse.array.SparseArray
Create a SparseArray from a scipy.sparse matrix.

Parameters
-----
data : scipy.sparse.sp_matrix
    This should be a SciPy sparse matrix where the size
    of the second dimension is 1. In other words, a
    sparse matrix with a single column.

Returns
-----
SparseArray

Examples
-----
>>> import scipy.sparse
>>> mat = scipy.sparse.coo_matrix((4, 1))
>>> pd.arrays.SparseArray.from_spmatrix(mat)
[0.0, 0.0, 0.0, 0.0]
Fill: 0.0
IntIndex
Indices: array([], dtype=int32)

```

Readonly properties defined here:

density

The percent of non- ``fill\_value`` points, as decimal.

See Also

DataFrame.sparse.from_spmatrix : Create a new DataFrame from a
scipy sparse matrix.

Examples

>>> from pandas.arrays import SparseArray
>>> s = SparseArray([0, 0, 1, 1, 1], fill_value=0)
>>> s.density
0.6

dtype

An instance of ExtensionDtype.

See Also

api.extensions.ExtensionDtype : Base class for extension dtypes.
api.extensions.ExtensionArray : Base class for extension array types.
api.extensions.ExtensionArray.dtype : The dtype of an ExtensionArray.
Series.dtype : The dtype of a Series.
DataFrame.dtype : The dtype of a DataFrame.

Examples

>>> pd.array([1, 2, 3]).dtype
Int64Dtype()

kind

The kind of sparse index for this array. One of {'integer', 'block'}.

nbytes

The number of bytes needed to store this object in memory.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

>>> pd.array([1, 2, 3]).nbytes
27

npoints

The number of non- ``fill\_value`` points.

This property returns the number of elements in the sparse series that are not equal to the ``fill\_value``. Sparse data structures store only the non-``fill\_value`` elements, reducing memory usage when the majority of values are the same.

See Also

Series.sparse.to_dense : Convert a Series from sparse values to dense.
Series.sparse.fill_value : Elements in ``data`` that are ``fill\_value`` are not stored.
Series.sparse.density : The percent of non- ``fill\_value`` points, as decimal.

Examples

>>> from pandas.arrays import SparseArray
>>> s = SparseArray([0, 0, 1, 1, 1], fill_value=0)
>>> s.npoints
3

sp_index

The SparseIndex containing the location of non- ``fill\_value`` points.

sp_values

An ndarray containing the non- ``fill\_value`` values.

This property returns the actual data values stored in the sparse representation, excluding the values that are equal to the ``fill\_value``.

The result is an ndarray of the underlying values, preserving the sparse structure by omitting the default ``fill\_value`` entries.

See Also

Series.sparse.to_dense : Convert a Series from sparse values to dense.
Series.sparse.fill_value : Elements in `data` that are `fill\_value` are not stored.
Series.sparse.density : The percent of non- ``fill\_value`` points, as decimal.

Examples

>>> from pandas.arrays import SparseArray
>>> s = SparseArray([0, 0, 1, 0, 2], fill_value=0)
>>> s.sp_values
array([1, 2])

Data descriptors defined here:

fill_value

Elements in `data` that are `fill\_value` are not stored.

For memory savings, this should be the most common value in the array.

See Also

SparseDtype : Dtype for data stored in :class:`SparseArray`.
Series.value_counts : Return a Series containing counts of unique values.
Series.fillna : Fill NA/NAN in a Series with a specified value.

Examples

>>> ser = pd.Series([0, 0, 2, 2, 2], dtype="Sparse[int]")
>>> ser.sparse.fill_value
0
>>> spa_dtype = pd.SparseDtype(dtype=np.int32, fill_value=2)
>>> ser = pd.Series([0, 0, 2, 2, 2], dtype=spa_dtype)
>>> ser.sparse.fill_value
2

Data and other attributes defined here:

__annotations__ = {'_dtype': 'SparseDtype', '_sparse_index': 'SparseIn...

Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)

Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
 height weight
elk 1.5 500

```
moose      2.6      800
```

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
      height  weight
elk        3.0   501.5
moose       4.1   801.5
```

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0   501.5
moose       3.1   801.5
```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0   501.5
moose       3.1   801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0   500.5
moose       4.1   800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN
```

```
>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0   500.5
moose       4.1   801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer       NaN     NaN
elk        1.7     NaN
moose       3.0     NaN
```

```
__and__(self, other)
```

```
__divmod__(self, other)
```

```
__eq__(self, other)
    Return self==value.
```

```
__floordiv__(self, other)
```

```
__ge__(self, other)
    Return self>=value.
```

```
__gt__(self, other)
    Return self>value.
```

```
__le__(self, other)
    Return self<=value.
```



```

__lt__(self, other)
    Return self<value.

__mod__(self, other)

__mul__(self, other)

__ne__(self, other)
    Return self!=value.

__or__(self, other)
    Return self|value.

__pow__(self, other)

__radd__(self, other)

__rand__(self, other)

__rdivmod__(self, other)

__rfloordiv__(self, other)

__rmod__(self, other)

__rmul__(self, other)

__ror__(self, other)
    Return value|self.

__rpow__(self, other)

__rsub__(self, other)

__rtruediv__(self, other)

__rxor__(self, other)

__sub__(self, other)

__truediv__(self, other)

__xor__(self, other)

```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

```

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

```

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

```

__hash__ = None

```

Methods inherited from pandas.core.base.PandasObject:

```

__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values

```

Methods inherited from pandas.core.accessor.DirNamesMixin:

```

__dir__(self) -> list[str]
    Provide method name lookup and completion.

```

```

    Notes
    -----
    Only provide 'public' methods.

```

Methods inherited from pandas.api.extensions.ExtensionArray:

`__contains__(self, item: object) -> bool` | `np.bool_` from pandas.core.arrays.base.ExtensionArray
Return for ``item`` in `self``.

`__iter__(self) -> Iterator[Any]` from pandas.core.arrays.base.ExtensionArray
Iterate over elements of the array.

`argsort(self, *, ascending: bool = True, kind: SortKind = 'quicksort', na_position: str = 'last', **kwargs)`
-> `np.ndarray` from pandas.core.arrays.base.ExtensionArray
Return the indices that would sort this array.

Parameters

`ascending` : bool, default True

Whether the indices should result in an ascending
or descending sort.

`kind` : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.

`na_position` : {'first', 'last'}, default 'last'

If ```first```, put ```NaN``` values at the beginning.

If ```last```, put ```NaN``` values at the end.

`**kwargs`

Passed through to `:func:`numpy.argsort``.

Returns

`np.ndarray[np.intp]`

Array of indices that sort ```self```. If NaN values are contained,
NaN values are placed at the end.

See Also

`numpy.argsort` : Sorting implementation used internally.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
```

```
>>> arr.argsort()
```

```
array([1, 2, 0, 4, 3])
```

`delete(self, loc: PositionalIndexer) -> Self` from pandas.core.arrays.base.ExtensionArray

`dropna(self) -> Self` from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all
NA values removed.

See Also

`Series.dropna` : Remove missing values from a Series.

`DataFrame.dropna` : Remove missing values from a DataFrame.

`api.extensions.ExtensionArray.isna` : Check for missing values in
an ExtensionArray.

Examples

```
>>> pd.array([1, 2, np.nan]).dropna()
```

```
<IntegerArray>
```

```
[1, 2]
```

```
Length: 2, dtype: Int64
```

`equals(self, other: object) -> bool` from pandas.core.arrays.base.ExtensionArray
Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and
all values compare equal. Missing values in the same location are

considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.
Series.equals : Equivalent method for Series.
DataFrame.equals : Equivalent method for DataFrame.

Examples

>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True

>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False

insert(self, loc: int, item) -> Self from pandas.core.arrays.base.ExtensionArray
Insert an item at the given position.

Parameters

loc : int
Index where the `item` needs to be inserted.
item : scalar-like
Value to be inserted.

Returns

ExtensionArray
With `item` inserted at `loc`.

See Also

Index.insert: Make new Index inserting new item at location.

Notes

This method should be both type and dtype-preserving. If the item cannot be held in an array of this type/dtype, either ValueError or TypeError should be raised.

The default implementation relies on `_from_sequence` to raise on invalid items.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.insert(2, -1)
<IntegerArray>
[1, 2, -1, 3]
Length: 4, dtype: Int64

interpolate(
 self,
 *,
 method: InterpolateOptions,
 axis: int,
 index: Index,
 limit,
 limit_direction,
 limit_area,
 copy: bool,
 **kwargs

```

) -> Self from pandas.core.arrays.base.ExtensionArray
    Fill NaN values using an interpolation method.

Parameters
-----
method : str, default 'linear'
    Interpolation technique to use. One of:
    * 'linear': Ignore the index and treat the values as equally spaced.
      This is the only method supported on MultiIndexes.
    * 'time': Works on daily and higher resolution data to interpolate
      given length of interval.
    * 'index', 'values': use the actual numerical values of the index.
    * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric',
      'polynomial': Passed to scipy.interpolate.interp1d, whereas 'spline'
      is passed to scipy.interpolate.UnivariateSpline. These methods use
      the numerical values of the index.
      Both 'polynomial' and 'spline' require that you also specify an
      order (int), e.g. arr.interpolate(method='polynomial', order=5).
    * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima',
      'cubicspline': Wrappers around the SciPy interpolation methods
      of similar names. See Notes.
    * 'from_derivatives': Refers to scipy.interpolate.BPoly.from_derivatives.
axis : int
    Axis to interpolate along. For 1-dimensional data, use 0.
index : Index
    Index to use for interpolation.
limit : int or None
    Maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction : {'forward', 'backward', 'both'}
    Consecutive NaNs will be filled in this direction.
limit_area : {'inside', 'outside'} or None
    If limit is specified, consecutive NaNs will be filled with this
    restriction.
    * None: No fill restriction.
    * 'inside': Only fill NaNs surrounded by valid values (interpolate).
    * 'outside': Only fill NaNs outside valid values (extrapolate).
copy : bool
    If True, a copy of the object is returned with interpolated values.
**kwargs : optional
    Keyword arguments to pass on to the interpolating function.

Returns
-----
ExtensionArray
    An ExtensionArray with interpolated values.

See Also
-----
Series.interpolate : Interpolate values in a Series.
DataFrame.interpolate : Interpolate values in a DataFrame.

Notes
-----
- All parameters must be specified as keyword arguments.
- The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima'
  methods are wrappers around the respective SciPy implementations of
  similar names. These use the actual numerical values of the index.

Examples
-----
Interpolating values in a NumPy array:

>>> arr = pd.arrays.NumpyExtensionArray(np.array([0, 1, np.nan, 3]))
>>> arr.interpolate(
...     method="linear",
...     limit=3,
...     limit_direction="forward",
...     index=pd.Index(range(len(arr))),
...     fill_value=1,
...     copy=False,
...     axis=0,
...     limit_area="inside",
... )
<NumpyExtensionArray>
[0.0, 1.0, 2.0, 3.0]
Length: 4, dtype: float64

```

Interpolating values in a FloatingArray:

```
>>> arr = pd.array([1.0, pd.NA, 3.0, 4.0, pd.NA, 6.0], dtype="Float64")
>>> arr.interpolate(
...     method="linear",
...     axis=0,
...     index=pd.Index(range(len(arr))),
...     limit=None,
...     limit_direction="both",
...     limit_area=None,
...     copy=True,
... )
<FloatingArray>
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Length: 6, dtype: Float64
```

`isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]` from `pandas.core.arrays.base.ExtensionArray`
Pointwise comparison for set containment in the given values.

Roughly equivalent to ``np.array([x in values for x in self])``

Parameters

values : np.ndarray or ExtensionArray
Values to compare every element in the array against.

Returns

np.ndarray[bool]
With true at indices where value is in ``values``.

See Also

`DataFrame.isin`: Whether each element in the DataFrame is contained in values.
`Index.isin`: Return a boolean array where the index values are in values.
`Series.isin`: Whether elements in Series are contained in values.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean
```

`item(self, index: int | None = None)` from `pandas.core.arrays.base.ExtensionArray`
Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional
Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar
The element at the specified position.

Raises

`ValueError`
If no index is provided and the array does not have exactly one element.
`IndexError`
If the specified position is out of bounds.

See Also

`numpy.ndarray.item` : Return the item of an array as a scalar.

Examples

```
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)
```

```

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)

```

`ravel(self, order: Literal['C', 'F', 'A', 'K'] | None = 'C') -> Self` from `pandas.core.arrays`
Return a flattened view on this array.

Parameters

order : {None, 'C', 'F', 'A', 'K'}, default 'C'

Returns

ExtensionArray
A flattened view on the array.

See Also

ExtensionArray.tolist: Return a list of the values.

Notes

- Because ExtensionArrays are 1D-only, this is a no-op.
- The "order" argument is ignored, is for compatibility with NumPy.

Examples

>>> pd.array([1, 2, 3]).ravel()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

`repeat(self, repeats: int | Sequence[int], axis: AxisInt | None = None) -> Self` from `pandas`.
Repeat elements of an ExtensionArray.

Returns a new ExtensionArray where each element of the current ExtensionArray is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints
The number of repetitions for each element. This should be a non-negative integer. Repeating 0 times will return an empty ExtensionArray.
axis : None
Must be ``None``. Has no effect but is accepted for compatibility with numpy.

Returns

ExtensionArray
Newly created ExtensionArray with repeated elements.

See Also

Series.repeat : Equivalent function for Series.
Index.repeat : Equivalent function for Index.
numpy.repeat : Similar method for :class:`numpy.ndarray`.
ExtensionArray.take : Take arbitrary positions.

Examples

>>> cat = pd.Categorical(["a", "b", "c"])
>>> cat
['a', 'b', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat(2)
['a', 'a', 'b', 'b', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> cat.repeat([1, 2, 3])
['a', 'b', 'b', 'c', 'c', 'c']
Categories (3, str): ['a', 'b', 'c']

`to_numpy(self,`

```

dtype: npt.DTypeLike | None = None,
copy: bool = False,
na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
Convert to a NumPy ndarray.

```

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

```

dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is a not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the type of the array.

```

Returns

```

numpy.ndarray

```

```

tolist(self) -> list from pandas.core.arrays.base.ExtensionArray
Return a list of the values.

```

These are each a scalar type, which is a Python scalar
(for str, int, float) or a pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

```

list
    Python list of values in array.

```

See Also

```

Index.tolist: Return a list of the values in the Index.
Series.tolist: Return a list of the values in the Series.

```

Examples

```

>>> arr = pd.array([1, 2, 3])
>>> arr.tolist()
[1, 2, 3]

```

```

transpose(self, *axes: int) -> Self from pandas.core.arrays.base.ExtensionArray
Return a transposed view on this array.

```

Because ExtensionArrays are always 1D, this is a no-op. It is included for compatibility with np.ndarray.

Returns

```

ExtensionArray

```

Examples

```

>>> pd.array([1, 2, 3]).transpose()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

```

```

view(self, dtype: Dtype | None = None) -> ArrayLike from pandas.core.arrays.base.ExtensionArray
Return a view on the array.

```

Parameters

```

dtype : str, np.dtype, or ExtensionDtype, optional
    Default None.

```

Returns

```

ExtensionArray or np.ndarray

```

A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.api.extensions.ExtensionArray:

T

ndim

Extension Arrays are only allowed to be 1-dimensional.

See Also

ExtensionArray.shape: Return a tuple of the array dimensions.
ExtensionArray.size: The number of elements in the array.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.ndim
1
```

shape

Return a tuple of the array dimensions.

See Also

numpy.ndarray.shape : Similar attribute which returns the shape of an array.
DataFrame.shape : Return a tuple representing the dimensionality of the DataFrame.
Series.shape : Return a tuple representing the dimensionality of the Series.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.shape
(3,)
```

size

The number of elements in the array.

Data and other attributes inherited from pandas.api.extensions.ExtensionArray:

__pandas_priority__ = 1000

```
class StringArray(pandas.core.arrays.string_.BaseStringArray, NumpyExtensionArray)
    StringArray(values, *, dtype: StringDtype | None = None, copy: bool = False) -> None
```

Extension array for string data.

.. warning::

StringArray is considered experimental. The implementation and parts of the API may change without warning.

Parameters

values : array-like
 The array of data.

.. warning::

Currently, this expects an object-dtype ndarray where the elements are Python strings or nan-likes (``None``, ``np.nan``, ``NA``). This may change without warning in the future. Use :meth:`pandas.array` with ``dtype="string"`` for a stable way of creating a `StringArray` from any sequence.

`StringArray` accepts array-likes containing nan-likes (``None``, ``np.nan``) for the ``values`` parameter in addition to strings and :attr:`pandas.NA`

dtype : StringDtype
 Dtype for the array.
 copy : bool, default False
 Whether to copy the array of data.

Attributes

None

Methods

None

See Also

:func:`array`

The recommended function for creating a `StringArray`.

`Series.str`

The string methods are available on `Series` backed by a `StringArray`.

Notes

`StringArray` returns a `BooleanArray` for comparison methods.

Examples

```
>>> pd.array(["This is", "some text", None, "data."], dtype="string")
<ArrowStringArray>
['This is', 'some text', <NA>, 'data.']
Length: 4, dtype: string
```

Unlike arrays instantiated with ``dtype="object"`, ``StringArray`` will convert the values to strings.

```
>>> pd.array(["1", 1], dtype="object")
<NumpyExtensionArray>
['1', 1]
Length: 2, dtype: object
>>> pd.array(["1", 1], dtype="string")
<ArrowStringArray>
['1', '1']
Length: 2, dtype: string
```

However, instantiating `StringArrays` directly with non-strings will raise an error.

For comparison methods, `StringArray` returns a :class:`pandas.BooleanArray`:

```
>>> pd.array(["a", None, "c"], dtype="string[python]") == "a"
<BooleanArray>
[True, <NA>, False]
Length: 3, dtype: boolean
```

Method resolution order:

```
StringArray
pandas.core.arrays.string_.BaseStringArray
NumpyExtensionArray
pandas.core.arraylike.OpsMixin
pandas.core.arrays._mixins.NDArrayBackedExtensionArray
pandas._libs.arrays.NDArrayBacked
pandas.api.extensions.ExtensionArray
```

```
pandas.core.strings.object_array.ObjectStringArrayMixin
builtins.object
```

Methods defined here:

```
__arrow_array__(self, type=None) from pandas.core.arrays.string_.StringArray
    Convert myself into a pyarrow Array.
```

```
__init__(self, values, *, dtype: StringDtype | None = None, copy: bool = False) -> None from
    Initialize self. See help(type(self)) for accurate signature.
```

```
__setitem__(self, key, value) -> None from pandas.core.arrays.string_.StringArray
    Set one or more values inplace.
```

This method is not required to satisfy the pandas extension array interface.

Parameters

key : int, ndarray, or slice
When called from, e.g. ``Series.\_\_setitem\_\_``, ``key`` will be one of

- * scalar int
- * ndarray of integers.
- * boolean ndarray
- * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
value or values to be set of ``key``.

Returns

None

Raises

ValueError

If the array is readonly and modification is attempted.

```
astype(self, dtype, copy: bool = True) from pandas.core.arrays.string_.StringArray
    Cast to a NumPy array or ExtensionArray with 'dtype'.
```

Parameters

dtype : str or dtype
Typecode or data-type to which the array is cast.
copy : bool, default True
Whether to copy the data, even if not necessary. If False, a copy is made only if the old dtype does not match the new dtype.

Returns

np.ndarray or pandas.api.extensions.ExtensionArray
An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``, otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also

Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from a sequence.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64
```

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

```
>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()
```

Otherwise, we will get a Numpy ndarray:

```
>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')
```

`isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]` from `pandas.core.arrays.string_.StringArray`
Pointwise comparison for set containment in the given values.

Roughly equivalent to ``np.array([x in values for x in self])``

Parameters

values : np.ndarray or ExtensionArray
Values to compare every element in the array against.

Returns

`np.ndarray[bool]`
With true at indices where value is in ``values``.

See Also

`DataFrame.isin`: Whether each element in the DataFrame is contained in values.
`Index.isin`: Return a boolean array where the index values are in values.
`Series.isin`: Whether elements in Series are contained in values.

Examples

```
>>> arr = pd.array([1, 2, 3])
>>> arr.isin([1])
<BooleanArray>
[True, False, False]
Length: 3, dtype: boolean
```

`max(self, axis=None, skipna: bool = True, **kwargs) -> Scalar` from `pandas.core.arrays.string_.StringArray`

`memory_usage(self, deep: bool = False) -> int` from `pandas.core.arrays.string_.StringArray`

`min(self, axis=None, skipna: bool = True, **kwargs) -> Scalar` from `pandas.core.arrays.string_.StringArray`

`searchsorted(`

`self,`
`value: NumpyValueArrayLike | ExtensionArray,`
`side: Literal['left', 'right'] = 'left',`
`sorter: NumpySorter | None = None`

`) -> npt.NDArray[np.intp] | np.intp` from `pandas.core.arrays.string_.StringArray`
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array ``self`` (a) such that, if the corresponding elements in ``value`` were inserted before the indices, the order of ``self`` would be preserved.

Assuming that ``self`` is sorted:

```
=====
`side` returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

value : array-like, list or scalar
Value(s) to insert into ``self``.

side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable
index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending
order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

sum(
 self,
 *,
 axis: AxisInt | None = None,
 skipna: bool = True,
 min_count: int = 0,
 **kwargs
) -> Scalar from pandas.core.arrays.string_.StringArray

value_counts(self, dropna: bool = True) -> Series from pandas.core.arrays.string_.StringArray
Return a Series containing counts of unique values.

This method returns a Series with unique values as the index and their
counts as the values.

Parameters

dropna : bool, default True
Don't include counts of NA values.

Returns

Series
A Series with unique values as index and counts as values.

See Also

Series.value_counts : Equivalent method on Series.
DataFrame.value_counts : Equivalent method on DataFrame.
Index.value_counts : Equivalent method on Index.

Examples

>>> from pandas.core.arrays import IntegerArray
>>> arr = IntegerArray._from_sequence([3, 3, 3, 1, 2, 2])
>>> arr.value_counts()
3 3
1 1
2 2
Name: count, dtype: Int64

Methods inherited from pandas.core.arrays.string_.BaseStringArray:

tolist(self) -> list
Return a list of the value.

These are each a scalar type, which is a Python scalar
(for str, int, float) or pandas scalar
(for Timestamp/Timedelta/Interval/Period)

Returns

list

Examples

```
-----
>>> arr = pd.array(["a", "b", "c"])
>>> arr.tolist()
['a', 'b', 'c']
```

view(self, dtype: Dtype | None = None) -> Self
Return a view on the array.

Parameters

dtype : str, np.dtype, or ExtensionDtype, optional
Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Data and other attributes inherited from pandas.core.arrays.string_.BaseStringArray:

__annotations__ = {}

Methods inherited from NumpyExtensionArray:

__abs__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray

__array__(self, dtype: np.dtype | None = None, copy: bool | None = None) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs) from pandas.core.arrays.numpy_.NumpyExtensionArray

__invert__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray

__neg__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray

__pos__(self) -> NumpyExtensionArray from pandas.core.arrays.numpy_.NumpyExtensionArray

```
all(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray
```

```
any(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
)
```

```

) from pandas.core.arrays.numpy_.NumpyExtensionArray

interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self from pandas.core.arrays.numpy_.NumpyExtensionArray
    See NDFrame.interpolate.__doc__.

isna(self) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray
    A 1-D array indicating if each value is missing.

Returns
-----
numpy.ndarray or pandas.api.extensions.ExtensionArray
    In most cases, this should return a NumPy ndarray. For
    exceptional cases like ``SparseArray``, where returning
    an ndarray would be expensive, an ExtensionArray may be
    returned.

See Also
-----
ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.

Notes
-----
If returning an ExtensionArray, then

* ``na_values._is_boolean`` should be True
* ``na_values`` should implement :func:`ExtensionArray._reduce`
* ``na_values`` should implement :func:`ExtensionArray._accumulate`
* ``na_values.any`` and ``na_values.all`` should be implemented

Examples
-----
>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False,  True,  True])

kurt(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

mean(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

median(
    self,
    *,
    axis: AxisInt | None = None,
    out=None,
    overwrite_input: bool = False,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

```

```

prod(
    self,
    *,
    axis: AxisInt | None = None,
    skipna: bool = True,
    min_count: int = 0,
    **kwargs
) -> Scalar from pandas.core.arrays.numpy_.NumpyExtensionArray

```

```

sem(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

```

```

skew(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

```

```

std(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray

```

```

take(
    self,
    indices: TakeIndexer,
    *,
    allow_fill: bool = False,
    fill_value: Any = None,
    axis: AxisInt = 0
) -> Self from pandas.core.arrays.numpy_.NumpyExtensionArray
Take entries from this array at each index in a list of indices,
producing an array containing only those entries.

```

```

to_numpy(
    self,
    dtype: npt.DTypeLike | None = None,
    copy: bool = False,
    na_value: object = <no_default>
) -> np.ndarray from pandas.core.arrays.numpy_.NumpyExtensionArray
Convert to a NumPy ndarray.

```

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

Whether to ensure that the returned value is a not a view on another array. Note that ``copy=False`` does not *ensure* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.

na_value : Any, optional

The value to use for missing values. The default value depends on `dtype` and the type of the array.

Returns

```

-----
numpy.ndarray

var(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NdDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,
    skipna: bool = True
) from pandas.core.arrays.numpy_.NumpyExtensionArray
-----
Readonly properties inherited from NumpyExtensionArray:

dtype
    An instance of ExtensionDtype.

    See Also
    -----
    api.extensions.ExtensionDtype : Base class for extension dtypes.
    api.extensions.ExtensionArray : Base class for extension array types.
    api.extensions.ExtensionArray.dtype : The dtype of an ExtensionArray.
    Series.dtype : The dtype of a Series.
    DataFrame.dtype : The dtype of a DataFrame.

    Examples
    -----
    >>> pd.array([1, 2, 3]).dtype
    Int64Dtype()
-----
Data and other attributes inherited from NumpyExtensionArray:

__array_priority__ = 1000
-----
Methods inherited from pandas.core.arraylike.OpsMixin:

__add__(self, other)
    Get Addition of DataFrame and other, column-wise.

    Equivalent to ``DataFrame.add(other)``.

    Parameters
    -----
    other : scalar, sequence, Series, dict or DataFrame
        Object to be added to the DataFrame.

    Returns
    -----
    DataFrame
        The result of adding ``other`` to DataFrame.

    See Also
    -----
    DataFrame.add : Add a DataFrame and another object, with option for index-
    or column-oriented addition.

    Examples
    -----
    >>> df = pd.DataFrame(
    ...     {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
    ... )
    >>> df
           height  weight
    elk         1.5     500
    moose        2.6     800

    Adding a scalar affects all rows and columns.

    >>> df[["height", "weight"]] + 1.5
           height  weight
    elk         3.0    501.5
    moose        4.1    801.5

```


Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
      height  weight
elk        2.0    501.5
moose      3.1    801.5
```

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose      3.1    801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose      4.1    800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose      4.1    801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk        1.7     NaN
moose      3.0     NaN
```

`__and__(self, other)`

`__divmod__(self, other)`

`__eq__(self, other)`
Return self==value.

`__floordiv__(self, other)`

`__ge__(self, other)`
Return self>=value.

`__gt__(self, other)`
Return self>value.

`__le__(self, other)`
Return self<=value.

`__lt__(self, other)`
Return self<value.

`__mod__(self, other)`

`__mul__(self, other)`

`__ne__(self, other)`
Return self!=value.

`__or__(self, other)`
Return self|value.

`__pow__(self, other)`

`__radd__(self, other)`

`__rand__(self, other)`

`__rdivmod__(self, other)`

`__rfloordiv__(self, other)`

`__rmod__(self, other)`

`__rmul__(self, other)`

`__ror__(self, other)`
Return value|self.

`__rpow__(self, other)`

`__rsub__(self, other)`

`__rtruediv__(self, other)`

`__rxor__(self, other)`

`__sub__(self, other)`

`__truediv__(self, other)`

`__xor__(self, other)`

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

`__dict__`
dictionary for instance variables

`__weakref__`
list of weak references to the object

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

`__hash__` = None

Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

`__getitem__(self, key: PositionalIndexer2D) -> Self | Any`
Select a subset of self.

Parameters

`item` : int, slice, or ndarray

* int: The position in 'self' to get.

* slice: A slice object, where 'start', 'stop', and 'step' are integers or None

* ndarray: A 1-d boolean NumPy ndarray the same length as 'self'

* list[int]: A list of int

Returns

`item` : scalar or ExtensionArray

Notes

For scalar ``item``, return a scalar value suitable for the array's type. This should be an instance of ``self.dtype.type``.

For slice ``key``, return an instance of ``ExtensionArray``, even if the slice is length 0 or 1.

For a boolean mask, return an instance of ``ExtensionArray``, filtered to the values where ``item`` is True.

`argmax(self, axis: AxisInt = 0, skipna: bool = True)`
Return the index of maximum value.

In case of multiple occurrences of the maximum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

`ExtensionArray.argmin` : Return the index of the minimum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)

`argmin(self, axis: AxisInt = 0, skipna: bool = True)`
Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

`ExtensionArray.argmax` : Return the index of the maximum value.

Examples

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)

`equals(self, other) -> bool`
Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean
Whether the arrays are equivalent.

See Also

`numpy.array_equal` : Equivalent method for numpy array.
`Series.equals` : Equivalent method for Series.
`DataFrame.equals` : Equivalent method for DataFrame.

Examples

```
-----
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
False
```

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.
`limit` : int, default None
The maximum number of entries where NA values will be filled.
`copy` : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

`ExtensionArray`
With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.
`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
-----
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64
```

`insert(self, loc: int, item) -> Self`
Make new ExtensionArray inserting new item at location. Follows Python list.append semantics for negative values.

Parameters

`loc` : int
`item` : object

Returns

`type(self)`

`shift(self, periods: int = 1, fill_value=None) -> Self`
Shift values by desired number.

Newly introduced missing values are filled with `self.dtype.na_value` `.`

Parameters

`periods` : int, default 1
The number of periods to shift. Negative values are allowed for shifting backwards.

`fill_value` : object, optional
The scalar value to use for newly introduced missing values.

The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on
this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an
enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is
returned.

If ``periods > len(self)`` , then an array of size
len(self) is returned, with all values filled with
``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

unique(self) -> Self
Compute the ExtensionArray of unique values.

Returns

pandas.api.extensions.ExtensionArray
With unique values from the input array.

See Also

Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples

>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Methods inherited from pandas._libs.arrays.NDArrayBacked:

__len__(self, /)
Return len(self).

__reduce__ = __reduce_cython__(...)

__reduce_cython__(self)

__setstate__(self, state)

__setstate_cython__(self, __pyx_state)

copy(self, order='C')

delete(self, loc, axis=0)

ravel(self, order='C')

repeat(self, repeats, axis: int | np.integer = 0)

```

reshape(self, *args, **kwargs)

swapaxes(self, axis1, axis2)

transpose(self, *axes)

-----
Static methods inherited from pandas._libs.arrays.NDArrayBacked:

__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
    Create and return a new object. See help(type) for accurate signature.

-----
Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:

T
nbytes
ndim
shape
size

-----
Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:

__pyx_vtable__ = <capsule object NULL>

-----
Methods inherited from pandas.api.extensions.ExtensionArray:

__contains__(self, item: object) -> bool | np.bool_ from pandas.core.arrays.base.ExtensionArray
    Return for `item in self`.

__iter__(self) -> Iterator[Any] from pandas.core.arrays.base.ExtensionArray
    Iterate over elements of the array.

__repr__(self) -> str from pandas.core.arrays.base.ExtensionArray
    Return repr(self).

argsort(
    self,
    *,
    ascending: bool = True,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    **kwargs
) -> np.ndarray from pandas.core.arrays.base.ExtensionArray
    Return the indices that would sort this array.

Parameters
-----
ascending : bool, default True
    Whether the indices should result in an ascending
    or descending sort.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
    Sorting algorithm.
na_position : {'first', 'last'}, default 'last'
    If ``'first'``, put ``NaN`` values at the beginning.
    If ``'last'``, put ``NaN`` values at the end.
**kwargs
    Passed through to :func:`numpy.argsort`.

Returns
-----
np.ndarray[np.intp]
    Array of indices that sort ``self``. If NaN values are contained,
    NaN values are placed at the end.

See Also
-----
numpy.argsort : Sorting implementation used internally.

Examples
-----

```

```

>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argsort()
array([1, 2, 0, 4, 3])

```

dropna(self) -> Self from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self
An ExtensionArray of the same type as the original but with all NA values removed.

See Also

Series.dropna : Remove missing values from a Series.
DataFrame.dropna : Remove missing values from a DataFrame.
api.extensions.ExtensionArray.isna : Check for missing values in an ExtensionArray.

Examples

>>> pd.array([1, 2, np.nan]).dropna()
<IntegerArray>
[1, 2]
Length: 2, dtype: Int64

duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_] f
Return boolean ndarray denoting duplicate values.

Parameters

keep : {'first', 'last', False}, default 'first'
- ``'first'`` : Mark duplicates as ``True`` except for the first occurrence.
- ``'last'`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

Returns

ndarray[bool]
With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.
Series.duplicated : Indicate duplicate Series values.
api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False, True, False, False, True])

factorize(self, use_na_sentinel: bool = True) -> tuple[np.ndarray, ExtensionArray] from pandas
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

codes : ndarray
An integer NumPy array that's an indexer into the original ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of the ExtensionArray if there are any missing values present

```

        in `self`.

See Also
-----
factorize : Top-level factorize method that dispatches here.

Notes
-----
:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples
-----
>>> idx1 = pd.PeriodIndex(
...     ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
...     freq="M",
... )
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray
Return the array element at the specified position as a Python scalar.

Parameters
-----
index : int, optional
    Position of the element. If not provided, the array must contain
    exactly one element.

Returns
-----
scalar
    The element at the specified position.

Raises
-----
ValueError
    If no index is provided and the array does not have exactly
    one element.
IndexError
    If the specified position is out of bounds.

See Also
-----
numpy.ndarray.item : Return the item of an array as a scalar.

Examples
-----
>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)

map(self, mapper, na_action: Literal['ignore'] | None = None) from pandas.core.arrays.base.E
Map values using an input mapping or function.

Parameters
-----
mapper : function, dict, or Series
    Mapping correspondence.
na_action : {None, 'ignore'}, default None
    If 'ignore', propagate NA values, without passing them to the
    mapping correspondence. If 'ignore' is not supported, a
    ``NotImplementedError`` should be raised.

Returns
-----
Union[ndarray, Index, ExtensionArray]
    The output of the mapping function applied to the array.
    If the function returns a tuple with more than one element

```



```

        a MultiIndex will be returned.
-----
Data and other attributes inherited from pandas.api.extensions.ExtensionArray:
    __pandas_priority__ = 1000
class TimedeltaArray(pandas.core.arrays.datetimelike.TimelikeOps)
    TimedeltaArray(data, dtype: Dtype | None = None, freq=None, copy: bool = False) -> None
    Pandas ExtensionArray for timedelta data.
    .. warning::
        TimedeltaArray is currently experimental, and its API may change
        without warning. In particular, :attr:`TimedeltaArray.dtype` is
        expected to change to be an instance of an ``ExtensionDtype``
        subclass.
    Parameters
    -----
    data : array-like
        The timedelta data.
    dtype : numpy.dtype
        Currently, only ``numpy.dtype("timedelta64[ns]")`` is accepted.
    freq : Offset, optional
        Frequency of the data.
    copy : bool, default False
        Whether to copy the underlying array of data.
    Attributes
    -----
    None
    Methods
    -----
    None
    See Also
    -----
    Timedelta : Represents a duration, the difference between two dates or times.
    TimedeltaIndex : Immutable Index of timedelta64 data.
    to_timedelta : Convert argument to timedelta.
    Examples
    -----
    >>> pd.arrays.TimedeltaArray._from_sequence(pd.TimedeltaIndex(["1h", "2h"]))
    <TimedeltaArray>
    ['0 days 01:00:00', '0 days 02:00:00']
    Length: 2, dtype: timedelta64[us]
    Method resolution order:
        TimedeltaArray
        pandas.core.arrays.datetimelike.TimelikeOps
        pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin
        pandas.core.arraylike.OpsMixin
        pandas.core.arrays._mixins.NDArrayBackedExtensionArray
        pandas._libs.arrays.NDArrayBacked
        pandas.api.extensions.ExtensionArray
        builtins.object
    Methods defined here:
        __abs__(self) -> TimedeltaArray from pandas.core.arrays.timedeltas.TimedeltaArray
        __divmod__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
        __floordiv__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
        __iter__(self) -> Iterator from pandas.core.arrays.timedeltas.TimedeltaArray
            Iterate over elements of the array.
        __mod__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
        __mul__(self, other) -> Self from pandas.core.arrays.timedeltas.TimedeltaArray
        __neg__(self) -> TimedeltaArray from pandas.core.arrays.timedeltas.TimedeltaArray

```

```

__pos__(self) -> TimedeltaArray from pandas.core.arrays.timedeltas.TimedeltaArray
__rdivmod__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
__rfloordiv__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
__rmod__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
__rmul__ = __mul__(self, other) -> Self
__rtruediv__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
__truediv__(self, other) from pandas.core.arrays.timedeltas.TimedeltaArray
astype(self, dtype, copy: bool = True) from pandas.core.arrays.timedeltas.TimedeltaArray
    Cast to a NumPy array or ExtensionArray with 'dtype'.

Parameters
-----
dtype : str or dtype
    Typecode or data-type to which the array is cast.
copy : bool, default True
    Whether to copy the data, even if not necessary. If False,
    a copy is made only if the old dtype does not match the
    new dtype.

Returns
-----
np.ndarray or pandas.api.extensions.ExtensionArray
    An ``ExtensionArray`` if ``dtype`` is ``ExtensionDtype``,
    otherwise a Numpy ndarray with ``dtype`` for its dtype.

See Also
-----
Series.astype : Cast a Series to a different dtype.
DataFrame.astype : Cast a DataFrame to a different dtype.
api.extensions.ExtensionArray : Base class for ExtensionArray objects.
core.arrays.DatetimeArray._from_sequence : Create a DatetimeArray from a
    sequence.
core.arrays.TimedeltaArray._from_sequence : Create a TimedeltaArray from
    a sequence.

Examples
-----
>>> arr = pd.array([1, 2, 3])
>>> arr
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

Casting to another ``ExtensionDtype`` returns an ``ExtensionArray``:

>>> arr1 = arr.astype("Float64")
>>> arr1
<FloatingArray>
[1.0, 2.0, 3.0]
Length: 3, dtype: Float64
>>> arr1.dtype
Float64Dtype()

Otherwise, we will get a Numpy ndarray:

>>> arr2 = arr.astype("float64")
>>> arr2
array([1., 2., 3.])
>>> arr2.dtype
dtype('float64')

std(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    ddof: int = 1,
    keepdims: bool = False,

```

```

        skipna: bool = True
    ) from pandas.core.arrays.timedeltas.TimedeltaArray

sum(
    self,
    *,
    axis: AxisInt | None = None,
    dtype: NpDtype | None = None,
    out=None,
    keepdims: bool = False,
    initial=None,
    skipna: bool = True,
    min_count: int = 0
) from pandas.core.arrays.timedeltas.TimedeltaArray

to_pytimedelta(self) -> npt.NDArray[np.object_] from pandas.core.arrays.timedeltas.TimedeltaArray
    Return an ndarray of datetime.timedelta objects.

Returns
-----
numpy.ndarray
    A NumPy ``timedelta64`` object representing the same duration as the
    original pandas ``Timedelta`` object. The precision of the resulting
    object is in nanoseconds, which is the default
    time resolution used by pandas for ``Timedelta`` objects, ensuring
    high precision for time-based calculations.

See Also
-----
to_timedelta : Convert argument to timedelta format.
Timedelta : Represents a duration between two dates or times.
DatetimeIndex: Index of datetime64 data.
Timedelta.components : Return a components namedtuple-like
                        of a single timedelta.

Examples
-----
>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
>>> tdelta_idx
TimedeltaIndex(['1 days', '2 days', '3 days'],
                dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.to_pytimedelta()
array([datetime.timedelta(days=1), datetime.timedelta(days=2),
       datetime.timedelta(days=3)], dtype=object)

>>> tidx = pd.TimedeltaIndex(data=["1 days 02:30:45", "3 days 04:15:10"])
>>> tidx
TimedeltaIndex(['1 days 02:30:45', '3 days 04:15:10'],
                dtype='timedelta64[us]', freq=None)
>>> tidx.to_pytimedelta()
array([datetime.timedelta(days=1, seconds=9045),
       datetime.timedelta(days=3, seconds=15310)], dtype=object)

total_seconds(self) -> npt.NDArray[np.float64] from pandas.core.arrays.timedeltas.TimedeltaArray
    Return total duration of each element expressed in seconds.

This method is available directly on TimedeltaArray, TimedeltaIndex
and on Series containing timedelta values under the ``.dt`` namespace.

Returns
-----
ndarray, Index or Series
    When the calling object is a TimedeltaArray, the return type
    is ndarray. When the calling object is a TimedeltaIndex,
    the return type is an Index with a float64 dtype. When the calling object
    is a Series, the return type is Series of type ``float64`` whose
    index is the same as the original.

See Also
-----
datetime.timedelta.total_seconds : Standard library version
    of this method.
TimedeltaIndex.components : Return a DataFrame with components of
    each Timedelta.

Examples
-----

```

```

**Series**

>>> s = pd.Series(pd.to_timedelta(np.arange(5), unit="D"))
>>> s
0    0 days
1    1 days
2    2 days
3    3 days
4    4 days
dtype: timedelta64[s]

>>> s.dt.total_seconds()
0         0.0
1    86400.0
2   172800.0
3   259200.0
4   345600.0
dtype: float64

**TimedeltaIndex**

>>> idx = pd.to_timedelta(np.arange(5), unit="D")
>>> idx
TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'],
               dtype='timedelta64[s]', freq=None)

>>> idx.total_seconds()
Index([0.0, 86400.0, 172800.0, 259200.0, 345600.0], dtype='float64')

```

Readonly properties defined here:

components

Return a DataFrame of the individual resolution components of the Timedeltas.

The components (days, hours, minutes seconds, milliseconds, microseconds, nanoseconds) are returned as columns in a DataFrame.

Returns

DataFrame

See Also

TimedeltaIndex.total_seconds : Return total duration expressed in seconds.
TimedeltaIndex.components : Return a components namedtuple-like of a single
timedelta.

Examples

```

>>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
>>> tdelta_idx
TimedeltaIndex(['1 days 00:03:00.000002042'],
               dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.components
days hours minutes seconds milliseconds microseconds nanoseconds
0      1      0      3      0      0      2      42

```

days

Number of days for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.
Series.dt.microseconds : Return number of microseconds for each element.
Series.dt.nanoseconds : Return number of nanoseconds for each element.

Examples

For Series:

```

>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='D'))
>>> ser
0    1 days
1    2 days
2    3 days
dtype: timedelta64[s]

```

```

>>> ser.dt.days
0    1
1    2
2    3
dtype: int64

For TimedeltaIndex:

>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
               dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.days
Index([0, 10, 20], dtype='int64')

```

dtype
The dtype for the TimedeltaArray.

.. warning::

A future version of pandas will change dtype to be an instance of a :class:`pandas.api.extensions.ExtensionDtype` subclass, not a ``numpy.dtype``.

Returns

numpy.dtype

microseconds
Number of microseconds (≥ 0 and less than 1 second) for each element.

See Also

pd.Timedelta.microseconds : Number of microseconds (≥ 0 and less than 1 second).
pd.Timedelta.to_pytimedelta.microseconds : Number of microseconds (≥ 0 and less than 1 second) of a datetime.timedelta.

Examples

For Series:

```

>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='us'))
>>> ser
0    0 days 00:00:00.000001
1    0 days 00:00:00.000002
2    0 days 00:00:00.000003
dtype: timedelta64[us]
>>> ser.dt.microseconds
0    1
1    2
2    3
dtype: int32

For TimedeltaIndex:

>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='us')
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:00.000001', '0 days 00:00:00.000002',
               '0 days 00:00:00.000003'],
               dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.microseconds
Index([1, 2, 3], dtype='int32')

```

nanoseconds
Number of nanoseconds (≥ 0 and less than 1 microsecond) for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.
Series.dt.microseconds : Return number of nanoseconds for each element.

Examples

For Series:

```

>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='ns'))
>>> ser

```

```

0    0 days 00:00:00.000000001
1    0 days 00:00:00.000000002
2    0 days 00:00:00.000000003
dtype: timedelta64[ns]
>>> ser.dt.nanoseconds
0      1
1      2
2      3
dtype: int32

For TimedeltaIndex:

>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='ns')
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:00.000000001', '0 days 00:00:00.000000002',
                '0 days 00:00:00.000000003'],
                dtype='timedelta64[ns]', freq=None)
>>> tdelta_idx.nanoseconds
Index([1, 2, 3], dtype='int32')
```

seconds

Number of seconds (≥ 0 and less than 1 day) for each element.

See Also

Series.dt.seconds : Return number of seconds for each element.

Series.dt.nanoseconds : Return number of nanoseconds for each element.

Examples

For Series:

```

>>> ser = pd.Series(pd.to_timedelta([1, 2, 3], unit='s'))
>>> ser
0    0 days 00:00:01
1    0 days 00:00:02
2    0 days 00:00:03
dtype: timedelta64[s]
>>> ser.dt.seconds
0      1
1      2
2      3
dtype: int32
```

For TimedeltaIndex:

```

>>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit='s')
>>> tdelta_idx
TimedeltaIndex(['0 days 00:00:01', '0 days 00:00:02', '0 days 00:00:03'],
                dtype='timedelta64[s]', freq=None)
>>> tdelta_idx.seconds
Index([1, 2, 3], dtype='int32')
```

Data and other attributes defined here:

__annotations__ = {'_bool_ops': 'list[str]', '_datetimelike_methods': ...

__array_priority__ = 1000

days_docstring = "Number of days for each element.\n\n See Also\n

microseconds_docstring = "Number of microseconds (≥ 0 and less than 1...

nanoseconds_docstring = "Number of nanoseconds (≥ 0 and less than 1 m...

seconds_docstring = "Number of seconds (≥ 0 and less than 1 day) for....

Methods inherited from pandas.core.arrays.datetimelike.TimelikeOps:

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs, **kwargs)

all(self, *, axis: AxisInt | None = None, skipna: bool = True) -> bool

any(self, *, axis: AxisInt | None = None, skipna: bool = True) -> bool

```

as_unit(self, unit: TimeUnit, round_ok: bool = True) -> Self
    Convert to a dtype with the given unit resolution.

    The limits of timestamp representation depend on the chosen resolution.
    Different resolutions can be converted to each other through as_unit.

    Parameters
    -----
    unit : {'s', 'ms', 'us', 'ns'}
    round_ok : bool, default True
        If False and the conversion requires rounding, raise ValueError.

    Returns
    -----
    same type as self
        Converted to the specified unit.

    See Also
    -----
    Timestamp.as_unit : Convert to the given unit.

    Examples
    -----
    For :class:`pandas.DatetimeIndex`:

    >>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])
    >>> idx
    DatetimeIndex(['2020-01-02 01:02:03.004005006'],
                  dtype='datetime64[ns]', freq=None)
    >>> idx.as_unit("s")
    DatetimeIndex(['2020-01-02 01:02:03'], dtype='datetime64[s]', freq=None)

    For :class:`pandas.TimedeltaIndex`:

    >>> tdelta_idx = pd.to_timedelta(["1 day 3 min 2 us 42 ns"])
    >>> tdelta_idx
    TimedeltaIndex(['1 days 00:03:00.000002042'],
                  dtype='timedelta64[ns]', freq=None)
    >>> tdelta_idx.as_unit("s")
    TimedeltaIndex(['1 days 00:03:00'], dtype='timedelta64[s]', freq=None)

ceil(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Perform ceil operation on the data to the specified `freq`.

    Parameters
    -----
    freq : str or Offset
        The frequency level to ceil the index to. Must be a fixed
        frequency like 's' (second) not 'ME' (month end). See
        :ref:`frequency aliases <timeseries.offset_aliases>` for
        a list of possible `freq` values.
    ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
        Only relevant for DatetimeIndex:

        - 'infer' will attempt to infer fall dst-transition hours based on
          order
        - bool-ndarray where True signifies a DST time, False designates
          a non-DST time (note that this flag is only applicable for
          ambiguous times)
        - 'NaT' will return NaT where there are ambiguous times
        - 'raise' will raise a ValueError if there are ambiguous
          times.

    nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default '
        A nonexistent time does not exist in a particular timezone
        where clocks moved forward due to DST.

        - 'shift_forward' will shift the nonexistent time forward to the
          closest existing time
        - 'shift_backward' will shift the nonexistent time backward to the
          closest existing time
        - 'NaT' will return NaT where there are nonexistent times

```

- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

 DatetimeIndex, TimedeltaIndex, or Series
 Index of the same type for a DatetimeIndex or TimedeltaIndex,
 or a Series with the same index for a Series.

Raises

 ValueError if the `freq` cannot be converted.

See Also

 DatetimeIndex.floor :
 Perform floor operation on the data to the specified `freq`.
 DatetimeIndex.snap :
 Snap time stamps to nearest occurring frequency.

Notes

 If the timestamps have a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.ceil('h')
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',
               '2018-01-01 13:00:00'],
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.ceil("h")
0    2018-01-01 12:00:00
1    2018-01-01 12:00:00
2    2018-01-01 13:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 01:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.ceil("h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.ceil("h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

copy(self, order: str = 'C') -> Self
 Return a copy of the array.

This method creates a copy of the `ExtensionArray` where modifying the data in the copy will not affect the original array. This is useful when you want to manipulate data without altering the original dataset.

Returns

 ExtensionArray
 A new `ExtensionArray` object that is a copy of the current instance.

See Also

DataFrame.copy : Return a copy of the DataFrame.
Series.copy : Return a copy of the Series.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.copy()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

factorize(self, use_na_sentinel: bool = True, sort: bool = False)
Encode the extension array as an enumerated type.

Parameters

use_na_sentinel : bool, default True
If True, the sentinel -1 will be used for NaN values. If False,
NaN values will be encoded as non-negative integers and will not drop the
NaN from the uniques of the values.

Returns

codes : ndarray
An integer NumPy array that's an indexer into the original
ExtensionArray.
uniques : ExtensionArray
An ExtensionArray containing the unique values of `self`.

.. note::

uniques will *not* contain an entry for the NA value of
the ExtensionArray if there are any missing values present
in `self`.

See Also

factorize : Top-level factorize method that dispatches here.

Notes

:meth:`pandas.factorize` offers a `sort` keyword as well.

Examples

>>> idx1 = pd.PeriodIndex(
... ["2014-01", "2014-01", "2014-02", "2014-02", "2014-03", "2014-03"],
... freq="M",
...)
>>> arr, idx = idx1.factorize()
>>> arr
array([0, 0, 1, 1, 2, 2])
>>> idx
PeriodIndex(['2014-01', '2014-02', '2014-03'], dtype='period[M]')

floor(
 self,
 freq,
 ambiguous: TimeAmbiguous = 'raise',
 nonexistent: TimeNonexistent = 'raise'
) -> Self
Perform floor operation on the data to the specified `freq`.

Parameters

freq : str or Offset
The frequency level to floor the index to. Must be a fixed
frequency like 's' (second) not 'ME' (month end). See
:ref:`frequency aliases <timeseries.offset\_aliases>` for
a list of possible `freq` values.
ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'
Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on

```

        order
    - bool-ndarray where True signifies a DST time, False designates
      a non-DST time (note that this flag is only applicable for
      ambiguous times)
    - 'NaT' will return NaT where there are ambiguous times
    - 'raise' will raise a ValueError if there are ambiguous
      times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta,          default '
A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

    - 'shift_forward' will shift the nonexistent time forward to the
      closest existing time
    - 'shift_backward' will shift the nonexistent time backward to the
      closest existing time
    - 'NaT' will return NaT where there are nonexistent times
    - timedelta objects will shift nonexistent times by the timedelta
    - 'raise' will raise a ValueError if there are
      nonexistent times.

Returns
-----
DatetimeIndex, TimedeltaIndex, or Series
    Index of the same type for a DatetimeIndex or TimedeltaIndex,
    or a Series with the same index for a Series.

Raises
-----
ValueError if the `freq` cannot be converted.

See Also
-----
DatetimeIndex.floor :
    Perform floor operation on the data to the specified `freq`.
DatetimeIndex.snap :
    Snap time stamps to nearest occurring frequency.

Notes
-----
If the timestamps have a timezone, flooring will take place relative to the
local ("wall") time and re-localized to the same timezone. When flooring
near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
control the re-localization behavior.

Examples
-----
**DatetimeIndex**

>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')

>>> rng.floor('h')
DatetimeIndex(['2018-01-01 11:00:00', '2018-01-01 12:00:00',
               '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)

**Series**

>>> pd.Series(rng).dt.floor("h")
0    2018-01-01 11:00:00
1    2018-01-01 12:00:00
2    2018-01-01 12:00:00
dtype: datetime64[us]

When rounding near a daylight savings time transition, use ``ambiguous`` or
``nonexistent`` to control how the timestamp should be re-localized.

>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")

>>> rng_tz.floor("2h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)

```

```
>>> rng_tz.floor("2h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
interpolate(
    self,
    *,
    method: InterpolateOptions,
    axis: int,
    index: Index,
    limit,
    limit_direction,
    limit_area,
    copy: bool,
    **kwargs
) -> Self
    See NDFrame.interpolate.__doc__.
```

```
round(
    self,
    freq,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
    Perform round operation on the data to the specified `freq`.
```

Parameters

freq : str or Offset

The frequency level to round the index to. Must be a fixed frequency like 's' (second) not 'ME' (month end). See :ref:`frequency aliases <timeseries.offset\_aliases>` for a list of possible `freq` values.

ambiguous : 'infer', bool-ndarray, 'NaT', default 'raise'

Only relevant for DatetimeIndex:

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool-ndarray where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

DatetimeIndex, TimedeltaIndex, or Series

Index of the same type for a DatetimeIndex or TimedeltaIndex, or a Series with the same index for a Series.

Raises

ValueError if the `freq` cannot be converted.

See Also

DatetimeIndex.floor :

Perform floor operation on the data to the specified `freq`.

DatetimeIndex.snap :

Snap time stamps to nearest occurring frequency.

Notes

If the timestamps have a timezone, rounding will take place relative to the local ("wall") time and re-localized to the same timezone. When rounding near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

****DatetimeIndex****

```
>>> rng = pd.date_range("1/1/2018 11:59:00", periods=3, freq="min")
>>> rng
DatetimeIndex(['2018-01-01 11:59:00', '2018-01-01 12:00:00',
               '2018-01-01 12:01:00'],
              dtype='datetime64[us]', freq='min')
```

```
>>> rng.round('h')
DatetimeIndex(['2018-01-01 12:00:00', '2018-01-01 12:00:00',
               '2018-01-01 12:00:00'],
              dtype='datetime64[us]', freq=None)
```

****Series****

```
>>> pd.Series(rng).dt.round("h")
0    2018-01-01 12:00:00
1    2018-01-01 12:00:00
2    2018-01-01 12:00:00
dtype: datetime64[us]
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> rng_tz = pd.DatetimeIndex(["2021-10-31 03:30:00"], tz="Europe/Amsterdam")
```

```
>>> rng_tz.floor("2h", ambiguous=False)
DatetimeIndex(['2021-10-31 02:00:00+01:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
>>> rng_tz.floor("2h", ambiguous=True)
DatetimeIndex(['2021-10-31 02:00:00+02:00'],
              dtype='datetime64[us, Europe/Amsterdam]', freq=None)
```

```
take(
    self,
    indices: TakeIndexer,
    *,
    allow_fill: bool = False,
    fill_value: Any = None,
    axis: AxisInt = 0
) -> Self
    Take elements from an array.
```

Parameters

indices : sequence of int or one-dimensional np.ndarray of int
Indices to be taken.

allow_fill : bool, default False
How to handle negative values in ``indices``.

* False: negative values in ``indices`` indicate positional indices from the right (the default). This is similar to
:func:`numpy.take`.

* True: negative values in ``indices`` indicate missing values. These values are set to ``fill\_value``. Any other other negative values raise a ``ValueError``.

fill_value : any, optional
Fill value to use for NA-indices when ``allow\_fill`` is True. This may be ``None``, in which case the default NA value for the type, ``self.dtype.na\_value``, is used.

For many ExtensionArrays, there will be two representations of ``fill\_value``: a user-facing "boxed" scalar, and a low-level physical NA value. ``fill\_value`` should be the user-facing version, and the implementation should handle translating that to the physical version for processing the take if necessary.

Returns

ExtensionArray

An array formed with selected `indices`.

Raises

IndexError

When the indices are out of bounds for the array.

ValueError

When `indices` contains negative values other than ``-1`` and `allow\_fill` is True.

See Also

`numpy.take` : Take elements from an array along an axis.

`api.extensions.take` : Take elements from an array.

Notes

`ExtensionArray.take` is called by ``Series.\_\_getitem\_\_``, ``.loc``, ``.iloc``, when `indices` is a sequence of values. Additionally, it's called by :meth:`Series.reindex`, or any other method that causes realignment, with a `fill\_value`.

Examples

Here's an example implementation, which relies on casting the extension array to object dtype. This uses the helper method :func:`pandas.api.extensions.take`.

.. code-block:: python

```
def take(self, indices, allow_fill=False, fill_value=None):
    from pandas.api.extensions import take

    # If the ExtensionArray is backed by an ndarray, then
    # just pass that here instead of coercing to object.
    data = self.astype(object)

    if allow_fill and fill_value is None:
        fill_value = self.dtype.na_value

    # fill value should always be translated from the scalar
    # type for the array, to the physical storage type for
    # the data, before passing to take.

    result = take(
        data, indices, fill_value=fill_value, allow_fill=allow_fill
    )
    return self._from_sequence(result, dtype=self.dtype)
```

Data descriptors inherited from `pandas.core.arrays.datetimelike.TimelikeOps`:

freq

Return the frequency object if it is set, otherwise None.

To learn more about the frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

See Also

`DatetimeIndex.freq` : Return the frequency object if it is set, otherwise None.

`PeriodIndex.freq` : Return the frequency object if it is set, otherwise None.

Examples

```
>>> datetimeindex = pd.date_range(
...     "2022-02-22 02:22:22", periods=10, tz="America/Chicago", freq="h"
... )
>>> datetimeindex
DatetimeIndex(['2022-02-22 02:22:22-06:00', '2022-02-22 03:22:22-06:00',
              '2022-02-22 04:22:22-06:00', '2022-02-22 05:22:22-06:00',
              '2022-02-22 06:22:22-06:00', '2022-02-22 07:22:22-06:00',
              '2022-02-22 08:22:22-06:00', '2022-02-22 09:22:22-06:00',
              '2022-02-22 10:22:22-06:00', '2022-02-22 11:22:22-06:00'],
              dtype='datetime64[ns]', freq='h')
```

```

dtype='datetime64[us, America/Chicago]', freq='h')
>>> datetimeindex.freq
<Hour>

```

unit
The precision unit of the datetime data.

Returns the precision unit for the dtype.
It means the smallest time frame that can be stored within this dtype.

Returns

str
Unit string representation (e.g. "ns").

See Also

TimelikeOps.as_unit : Converts to a specific unit.

Examples

>>> idx = pd.DatetimeIndex(["2020-01-02 01:02:03.004005006"])
>>> idx.unit
'ns'
>>> idx.as_unit("s").unit
's'

Methods inherited from pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin:

__add__(self, other)
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame
The result of adding ``other`` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```

>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```

>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names;
each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
      height  weight
elk        2.0    501.5
moose       3.1    801.5
```

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
      height  weight
elk        3.0    500.5
moose       4.1    800.5
```

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
      elk  height  moose  weight
elk    NaN     NaN   NaN     NaN
moose  NaN     NaN   NaN     NaN

>>> df[["height", "weight"]].add(s2, axis="index")
      height  weight
elk        2.0    500.5
moose       4.1    801.5
```

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer       NaN     NaN
elk        1.7     NaN
moose       3.0     NaN
```

```
__array__(self, dtype: NpDtype | None = None, copy: bool | None = None) -> np.ndarray
__getitem__(self, key: PositionalIndexer2D) -> Self | DTScalarOrNaT
    This getitem defers to the underlying array, which by-definition can
    only handle list-likes, slices, and integer scalars
__iadd__(self, other) -> Self
__init__(self, data, dtype: Dtype | None = None, freq=None, copy: bool = False) -> None
    Initialize self. See help(type(self)) for accurate signature.
__isub__(self, other) -> Self
__pow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__radd__(self, other)
__rpow__(self: object, other: object = None) -> NoReturn from pandas.core.ops.invalid.make_in
__rsub__(self, other)
__setitem__(
    self,
    key: int | Sequence[int] | Sequence[bool] | slice,
    value: NaTType | Any | Sequence[Any]
) -> None
    Set one or more values inplace.

    This method is not required to satisfy the pandas extension array
    interface.

    Parameters
    -----
    key : int, ndarray, or slice
        When called from, e.g. ``Series.__setitem__``, ``key`` will be
        one of
```

```

        * scalar int
        * ndarray of integers.
        * boolean ndarray
        * slice object

value : ExtensionDtype.type, Sequence[ExtensionDtype.type], or object
value or values to be set of ``key``.

Returns
-----
None

Raises
-----
ValueError
    If the array is readonly and modification is attempted.

__sub__(self, other)

isin(self, values: ArrayLike) -> npt.NDArray[np.bool_]
    Compute boolean array of whether each value is found in the
    passed set of values.

Parameters
-----
values : np.ndarray or ExtensionArray

Returns
-----
ndarray[bool]

isna(self) -> npt.NDArray[np.bool_]
    A 1-D array indicating if each value is missing.

Returns
-----
numpy.ndarray or pandas.api.extensions.ExtensionArray
    In most cases, this should return a NumPy ndarray. For
    exceptional cases like ``SparseArray``, where returning
    an ndarray would be expensive, an ExtensionArray may be
    returned.

See Also
-----
ExtensionArray.dropna: Return ExtensionArray without NA values.
ExtensionArray.fillna: Fill NA/NaN values using the specified method.

Notes
-----
If returning an ExtensionArray, then

* ``na_values._is_boolean`` should be True
* ``na_values`` should implement :func:`ExtensionArray._reduce`
* ``na_values`` should implement :func:`ExtensionArray._accumulate`
* ``na_values.any`` and ``na_values.all`` should be implemented

Examples
-----
>>> arr = pd.array([1, 2, np.nan, np.nan])
>>> arr.isna()
array([False, False,  True,  True])

map(self, mapper, na_action: Literal['ignore'] | None = None)
    Map values using an input mapping or function.

Parameters
-----
mapper : function, dict, or Series
    Mapping correspondence.
na_action : {None, 'ignore'}, default None
    If 'ignore', propagate NA values, without passing them to the
    mapping correspondence. If 'ignore' is not supported, a
    ``NotImplementedError`` should be raised.

Returns
-----

```



```

Union[ndarray, Index, ExtensionArray]
    The output of the mapping function applied to the array.
    If the function returns a tuple with more than one element
    a MultiIndex will be returned.

max(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)
    Return the maximum value of the Array or maximum along
    an axis.

    See Also
    -----
    numpy.ndarray.max
    Index.max : Return the maximum value in an Index.
    Series.max : Return the maximum value in a Series.

mean(self, *, skipna: bool = True, axis: AxisInt | None = 0)
    Return the mean value of the Array.

    Parameters
    -----
    skipna : bool, default True
        Whether to ignore any NaT elements.
    axis : int, optional, default 0
        Axis for the function to be applied on.

    Returns
    -----
    scalar
        Timestamp or Timedelta.

    See Also
    -----
    numpy.ndarray.mean : Returns the average of array elements along a given axis.
    Series.mean : Return the mean value in a Series.

    Notes
    -----
    mean is only defined for Datetime and Timedelta dtypes, not for Period.

    Examples
    -----
    For :class:`pandas.DatetimeIndex`:

    >>> idx = pd.date_range("2001-01-01 00:00", periods=3)
    >>> idx
    DatetimeIndex(['2001-01-01', '2001-01-02', '2001-01-03'],
                  dtype='datetime64[us]', freq='D')
    >>> idx.mean()
    Timestamp('2001-01-02 00:00:00')

    For :class:`pandas.TimedeltaIndex`:

    >>> tdelta_idx = pd.to_timedelta([1, 2, 3], unit="D")
    >>> tdelta_idx
    TimedeltaIndex(['1 days', '2 days', '3 days'],
                  dtype='timedelta64[s]', freq=None)
    >>> tdelta_idx.mean()
    Timedelta('2 days 00:00:00')

median(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)

min(self, *, axis: AxisInt | None = None, skipna: bool = True, **kwargs)
    Return the minimum value of the Array or minimum along
    an axis.

    See Also
    -----
    numpy.ndarray.min
    Index.min : Return the minimum value in an Index.
    Series.min : Return the minimum value in a Series.

view(self, dtype: Dtype | None = None) -> ArrayLike
    Return a view on the array.

    Parameters
    -----
    dtype : str, np.dtype, or ExtensionDtype, optional

```

Default None.

Returns

ExtensionArray or np.ndarray
A view on the :class:`ExtensionArray`'s data.

See Also

api.extensions.ExtensionArray.ravel: Return a flattened view on input array.
Index.view: Equivalent function for Index.
ndarray.view: New view of array with the same data.

Examples

This gives view on the underlying data of an ``ExtensionArray`` and is not a copy. Modifications on either the view or the original ``ExtensionArray`` will be reflected on the underlying data:

```
>>> arr = pd.array([1, 2, 3])
>>> arr2 = arr.view()
>>> arr[0] = 2
>>> arr2
<IntegerArray>
[2, 2, 3]
Length: 3, dtype: Int64
```

Readonly properties inherited from pandas.core.arrays.datetimelike.DatetimeLikeArrayMixin:

asi8
Integer representation of the values.

Returns

ndarray
An ndarray with int64 dtype.

freqstr
Return the frequency object as a string if it's set, otherwise None.

See Also

DatetimeIndex.inferred_freq : Returns a string representing a frequency generated by infer_freq.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["1/1/2020 10:00:00+00:00"], freq="D")
>>> idx.freqstr
'D'
```

The frequency can be inferred if there are more than 2 points:

```
>>> idx = pd.DatetimeIndex(
...     ["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer"
... )
>>> idx.freqstr
'2D'
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-1", "2023-2", "2023-3"], freq="M")
>>> idx.freqstr
'M'
```

inferred_freq
Tries to return a string representing a frequency generated by infer_freq.
Returns None if it can't autodetect the frequency.

See Also

DatetimeIndex.freqstr : Return the frequency object as a string if it's set, otherwise None.

Examples

For DatetimeIndex:

```
>>> idx = pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
>>> idx.inferred_freq
'2D'
```

For TimedeltaIndex:

```
>>> tdelta_idx = pd.to_timedelta(["0 days", "10 days", "20 days"])
>>> tdelta_idx
TimedeltaIndex(['0 days', '10 days', '20 days'],
                dtype='timedelta64[us]', freq=None)
>>> tdelta_idx.inferred_freq
'10D'
```

resolution

Returns day, hour, minute, second, millisecond or microsecond

Methods inherited from pandas.core.arraylike.OpsMixin:

```
__and__(self, other)
__eq__(self, other)
    Return self==value.
__ge__(self, other)
    Return self>=value.
__gt__(self, other)
    Return self>value.
__le__(self, other)
    Return self<=value.
__lt__(self, other)
    Return self<value.
__ne__(self, other)
    Return self!=value.
__or__(self, other)
    Return self|value.
__rand__(self, other)
__ror__(self, other)
    Return value|self.
__rxor__(self, other)
__xor__(self, other)
```

Data descriptors inherited from pandas.core.arraylike.OpsMixin:

```
__dict__
    dictionary for instance variables
__weakref__
    list of weak references to the object
```

Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

```
__hash__ = None
```

Methods inherited from pandas.core.arrays._mixins.NDArrayBackedExtensionArray:

```
argmax(self, axis: AxisInt = 0, skipna: bool = True)
    Return the index of maximum value.
```

In case of multiple occurrences of the maximum value, the index

corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the minimum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmax()
np.int64(3)
```

argmin(self, axis: AxisInt = 0, skipna: bool = True)

Return the index of minimum value.

In case of multiple occurrences of the minimum value, the index corresponding to the first occurrence is returned.

Parameters

skipna : bool, default True

Returns

int

See Also

ExtensionArray.argmax : Return the index of the maximum value.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
>>> arr.argmin()
np.int64(1)
```

equals(self, other) -> bool

Return if another array is equivalent to this array.

Equivalent means that both arrays have the same shape and dtype, and all values compare equal. Missing values in the same location are considered equal (in contrast with normal equality).

Parameters

other : ExtensionArray
Array to compare to this Array.

Returns

boolean

Whether the arrays are equivalent.

See Also

numpy.array_equal : Equivalent method for numpy array.

Series.equals : Equivalent method for Series.

DataFrame.equals : Equivalent method for DataFrame.

Examples

```
>>> arr1 = pd.array([1, 2, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
True
```

```
>>> arr1 = pd.array([1, 3, np.nan])
>>> arr2 = pd.array([1, 2, np.nan])
>>> arr1.equals(arr2)
```

False

`fillna(self, value, limit: int | None = None, copy: bool = True) -> Self`
Fill NA/NaN values using the specified method.

Parameters

`value` : scalar, array-like
If a scalar value is passed it is used to fill all missing values. Alternatively, an array-like "value" can be given. It's expected that the array-like have the same length as 'self'.
`limit` : int, default None
The maximum number of entries where NA values will be filled.
`copy` : bool, default True
Whether to make a copy of the data before filling. If False, then the original should be modified and no new memory should be allocated. For ExtensionArray subclasses that cannot do this, it is at the author's discretion whether to ignore "copy=False" or to raise.

Returns

ExtensionArray
With NA/NaN filled.

See Also

`api.extensions.ExtensionArray.dropna` : Return ExtensionArray without NA values.
`api.extensions.ExtensionArray.isna` : A 1-D array indicating if each value is missing.

Examples

```
>>> arr = pd.array([np.nan, np.nan, 2, 3, np.nan, np.nan])
>>> arr.fillna(0)
<IntegerArray>
[0, 0, 2, 3, 0, 0]
Length: 6, dtype: Int64
```

`insert(self, loc: int, item) -> Self`
Make new ExtensionArray inserting new item at location. Follows Python list.append semantics for negative values.

Parameters

`loc` : int
`item` : object

Returns

`type(self)`

`searchsorted(`
 `self,`
 `value: NumpyValueArrayLike | ExtensionArray,`
 `side: Literal['left', 'right'] = 'left',`
 `sorter: NumpySorter | None = None`
`) -> npt.NDArray[np.intp] | np.intp`
Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array ``self`` (a) such that, if the corresponding elements in ``value`` were inserted before the indices, the order of ``self`` would be preserved.

Assuming that ``self`` is sorted:

```
=====
`side`  returned index `i` satisfies
=====
left    ``self[i-1] < value <= self[i]``
right   ``self[i-1] <= value < self[i]``
=====
```

Parameters

`value` : array-like, list or scalar
Value(s) to insert into ``self``.

side : {'left', 'right'}, optional
If 'left', the index of the first suitable location found is given.
If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
sorter : 1-D array-like, optional
Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

array of ints or int
If value is array-like, array of insertion points.
If value is scalar, a single integer.

See Also

numpy.searchsorted : Similar method from NumPy.

Examples

>>> arr = pd.array([1, 2, 3, 5])
>>> arr.searchsorted([4])
array([3])

shift(self, periods: int = 1, fill_value=None) -> Self
Shift values by desired number.

Newly introduced missing values are filled with
``self.dtype.na\_value``.

Parameters

periods : int, default 1
The number of periods to shift. Negative values are allowed for shifting backwards.

fill_value : object, optional
The scalar value to use for newly introduced missing values.
The default is ``self.dtype.na\_value``.

Returns

ExtensionArray
Shifted.

See Also

api.extensions.ExtensionArray.transpose : Return a transposed view on this array.
api.extensions.ExtensionArray.factorize : Encode the extension array as an enumerated type.

Notes

If ``self`` is empty or ``periods`` is 0, a copy of ``self`` is returned.

If ``periods > len(self)`` , then an array of size len(self) is returned, with all values filled with
``self.dtype.na\_value``.

For 2-dimensional ExtensionArrays, we are always shifting along axis=0.

Examples

>>> arr = pd.array([1, 2, 3])
>>> arr.shift(2)
<IntegerArray>
[<NA>, <NA>, 1]
Length: 3, dtype: Int64

unique(self) -> Self
Compute the ExtensionArray of unique values.

Returns

pandas.api.extensions.ExtensionArray

With unique values from the input array.

See Also

Index.unique: Return unique values in the index.
Series.unique: Return unique values of Series object.
unique: Return unique values based on a hash table.

Examples

>>> arr = pd.array([1, 2, 3, 1, 2, 3])
>>> arr.unique()
<IntegerArray>
[1, 2, 3]
Length: 3, dtype: Int64

value_counts(self, dropna: bool = True) -> Series
Return a Series containing counts of unique values.

Parameters

dropna : bool, default True
Don't include counts of NA values.

Returns

Series

Methods inherited from pandas._libs.arrays.NDArrayBacked:

__len__(self, /)
Return len(self).

__reduce__ = __reduce_cython__(...)
__reduce_cython__(self)
__setstate__(self, state)
__setstate_cython__(self, __pyx_state)
delete(self, loc, axis=0)
ravel(self, order='C')
repeat(self, repeats, axis: int | np.integer = 0)
reshape(self, *args, **kwargs)
swapaxes(self, axis1, axis2)
transpose(self, *axes)

Static methods inherited from pandas._libs.arrays.NDArrayBacked:

__new__(*args, **kwargs) class method of pandas._libs.arrays.NDArrayBacked
Create and return a new object. See help(type) for accurate signature.

Data descriptors inherited from pandas._libs.arrays.NDArrayBacked:

T
nbytes
ndim
shape
size

Data and other attributes inherited from pandas._libs.arrays.NDArrayBacked:

__pyx_vtable__ = <capsule object NULL>

Methods inherited from pandas.api.extensions.ExtensionArray:

`__contains__(self, item: object) -> bool` | `np.bool_` from pandas.core.arrays.base.ExtensionArray
Return for ``item`` in `self``.

`__repr__(self) -> str` from pandas.core.arrays.base.ExtensionArray
Return `repr(self)`.

`argsort(self, *, ascending: bool = True, kind: SortKind = 'quicksort', na_position: str = 'last', **kwargs)`
-> `np.ndarray` from pandas.core.arrays.base.ExtensionArray
Return the indices that would sort this array.

Parameters

`ascending` : bool, default True

Whether the indices should result in an ascending
or descending sort.

`kind` : {'quicksort', 'mergesort', 'heapsort', 'stable'}, optional
Sorting algorithm.

`na_position` : {'first', 'last'}, default 'last'

If ```'first'```, put ```NaN``` values at the beginning.

If ```'last'```, put ```NaN``` values at the end.

`**kwargs`

Passed through to `:func:`numpy.argsort``.

Returns

`np.ndarray[np.intp]`

Array of indices that sort ```self```. If NaN values are contained,
NaN values are placed at the end.

See Also

`numpy.argsort` : Sorting implementation used internally.

Examples

```
>>> arr = pd.array([3, 1, 2, 5, 4])
```

```
>>> arr.argsort()
```

```
array([1, 2, 0, 4, 3])
```

`dropna(self) -> Self` from pandas.core.arrays.base.ExtensionArray
Return ExtensionArray without NA values.

Returns

Self

An ExtensionArray of the same type as the original but with all
NA values removed.

See Also

`Series.dropna` : Remove missing values from a Series.

`DataFrame.dropna` : Remove missing values from a DataFrame.

`api.extensions.ExtensionArray.isna` : Check for missing values in
an ExtensionArray.

Examples

```
>>> pd.array([1, 2, np.nan]).dropna()
```

```
<IntegerArray>
```

```
[1, 2]
```

```
Length: 2, dtype: Int64
```

`duplicated(self, keep: Literal['first', 'last', False] = 'first') -> npt.NDArray[np.bool_]` from pandas.core.arrays.base.ExtensionArray
Return boolean ndarray denoting duplicate values.

Parameters

```

keep : {'first', 'last', False}, default 'first'
- ``first`` : Mark duplicates as ``True`` except for the first occurrence.
- ``last`` : Mark duplicates as ``True`` except for the last occurrence.
- False : Mark all duplicates as ``True``.

```

Returns

ndarray[bool]

With true in indices where elements are duplicated and false otherwise.

See Also

DataFrame.duplicated : Return boolean Series denoting duplicate rows.

Series.duplicated : Indicate duplicate Series values.

api.extensions.ExtensionArray.unique : Compute the ExtensionArray of unique values.

Examples

```

>>> pd.array([1, 1, 2, 3, 3], dtype="Int64").duplicated()
array([False,  True, False, False,  True])

```

item(self, index: int | None = None) from pandas.core.arrays.base.ExtensionArray

Return the array element at the specified position as a Python scalar.

Parameters

index : int, optional

Position of the element. If not provided, the array must contain exactly one element.

Returns

scalar

The element at the specified position.

Raises

ValueError

If no index is provided and the array does not have exactly one element.

IndexError

If the specified position is out of bounds.

See Also

numpy.ndarray.item : Return the item of an array as a scalar.

Examples

```

>>> arr = pd.array([1], dtype="Int64")
>>> arr.item()
np.int64(1)

```

```

>>> arr = pd.array([1, 2, 3], dtype="Int64")
>>> arr.item(0)
np.int64(1)
>>> arr.item(2)
np.int64(3)

```

to_numpy(

self,

dtype: npt.DTypeLike | None = None,

copy: bool = False,

na_value: object = <no_default>

) -> np.ndarray from pandas.core.arrays.base.ExtensionArray

Convert to a NumPy ndarray.

This is similar to :meth:`numpy.asarray`, but may provide additional control over how the conversion is done.

Parameters

dtype : str or numpy.dtype, optional

The dtype to pass to :meth:`numpy.asarray`.

copy : bool, default False

```

    Whether to ensure that the returned value is a not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
    na_value : Any, optional
        The value to use for missing values. The default value depends
        on `dtype` and the type of the array.

    Returns
    -----
    numpy.ndarray

    tolist(self) -> list from pandas.core.arrays.base.ExtensionArray
    Return a list of the values.

    These are each a scalar type, which is a Python scalar
    (for str, int, float) or a pandas scalar
    (for Timestamp/Timedelta/Interval/Period)

    Returns
    -----
    list
        Python list of values in array.

    See Also
    -----
    Index.tolist: Return a list of the values in the Index.
    Series.tolist: Return a list of the values in the Series.

    Examples
    -----
    >>> arr = pd.array([1, 2, 3])
    >>> arr.tolist()
    [1, 2, 3]

    -----
    Data and other attributes inherited from pandas.api.extensions.ExtensionArray:
    __pandas_priority__ = 1000

```

```

DATA
__all__ = ['ArrowExtensionArray', 'ArrowStringArray', 'BooleanArray', ...

```

```

FILE
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\arrays\__init__

```

Help on package pandas.compat in pandas:

```

NAME
pandas.compat

```

```

DESCRIPTION
compat
=====

Cross-compatible functions for different versions of Python.

Other items:
* platform checker

```

```

PACKAGE CONTENTS
_constants
_optional
numpy (package)
pickle_compat
pyarrow

```

```

DATA
CHAINED_WARNING_DISABLED = False
HAS_PYARROW = True
IS64 = True
ISMUSL = False
PY312 = True
PY314 = True
PYARROW_MIN_VERSION = '13.0.0'
PYPY = False

```

```
WASM = False
__all__ = ['CHAINED_WARNING_DISABLED', 'HAS_PYARROW', 'IS64', 'ISMUSL'...
is_numpy_dev = False
pa_version_under14p0 = False
pa_version_under14p1 = False
pa_version_under16p0 = False
pa_version_under17p0 = False
pa_version_under18p0 = False
pa_version_under19p0 = False
pa_version_under20p0 = False
pa_version_under21p0 = False
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\compat\__init__
```

Help on package pandas.core in pandas:

NAME

pandas.core

PACKAGE CONTENTS

```
_numba (package)
accessor
algorithms
api
apply
array_algos (package)
arraylike
arrays (package)
base
col
common
computation (package)
config_init
construction
dtypes (package)
flags
frame
generic
groupby (package)
indexers (package)
indexes (package)
indexing
interchange (package)
internals (package)
methods (package)
missing
nanops
ops (package)
resample
reshape (package)
roperator
sample
series
shared_docs
sorting
sparse (package)
strings (package)
tools (package)
util (package)
window (package)
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\core\__init__.py
```

Help on package pandas.errors in pandas:

NAME

pandas.errors - Expose public exceptions & warnings

PACKAGE CONTENTS

cow

CLASSES

```
builtins.AttributeError(builtins.Exception)
```

```

        OptionError(builtins.AttributeError, builtins.KeyError)
builtins.Exception(builtins.BaseException)
    ClosedFileError
    DataError
    IndexingError
    InvalidComparison
    InvalidIndexError
    LossySetitemError
    NoBufferPresent
    NumbaUtilError
    PossibleDataLossError
    SpecificationError
builtins.KeyError(builtins.LookupError)
    OptionError(builtins.AttributeError, builtins.KeyError)
    UnsortedIndexError
builtins.NameError(builtins.Exception)
    NumExprClobberingError
    UndefinedVariableError
builtins.NotImplementedError(builtins.RuntimeError)
    AbstractMethodError
builtins OSError(builtins.Exception)
    DatabaseError
builtins.RuntimeError(builtins.Exception)
    PyperclipException
        PyperclipWindowsException
builtins.TypeError(builtins.Exception)
    IncompatibleFrequency
builtins.UserWarning(builtins.Warning)
    CSSWarning
builtins.ValueError(builtins.Exception)
    DuplicateLabelError
    EmptyDataError
    IntCastingNaNError
    InvalidVersion
    MergeError
    NullFrequencyError
    OutOfBoundsDatetime
    OutOfBoundsTimedelta
    ParserError
    UnsupportedFunctionCall
builtins.Warning(builtins.Exception)
    AttributeConflictWarning
    CategoricalConversionWarning
    ChainedAssignmentError
    DtypeWarning
    IncompatibilityWarning
    InvalidColumnName
    PandasChangeWarning
        PandasDeprecationWarning(PandasChangeWarning, builtins.DeprecationWarning)
        Pandas4Warning
        PandasFutureWarning(PandasChangeWarning, builtins.FutureWarning)
        PandasPendingDeprecationWarning(PandasChangeWarning, builtins.PendingDeprecationWarning)
        Pandas5Warning
    ParserWarning
    PerformanceWarning
    PossiblePrecisionLoss
    ValueLabelTypeMismatch

```

```

class AbstractMethodError(builtins.NotImplementedError)
|     AbstractMethodError(class_instance, methodtype: str = 'method') -> None
|
|     Raise this error instead of NotImplementedError for abstract methods.
|
|     The `AbstractMethodError` is designed for use in classes that follow an abstract
|     base class pattern. By raising this error in the method, it ensures that a subclass
|     must implement the method to provide specific functionality. This is useful in a
|     framework or library where certain methods must be implemented by the user to
|     ensure correct behavior.
|
|     Parameters
|     -----
|     class_instance : object
|         The instance of the class where the abstract method is being called.
|     methodtype : str, default "method"
|         A string indicating the type of method that is abstract.
|         Must be one of {"method", "classmethod", "staticmethod", "property"}.
|

```

See Also

`api.extensions.ExtensionArray`

An example of a pandas extension mechanism that requires implementing specific abstract methods.

`NotImplementedError`

A built-in exception that can also be used for abstract methods but lacks the specificity of `AbstractMethodError` in indicating the need for subclass implementation.

Examples

```
>>> class Foo:
```

```
...     @classmethod
```

```
...     def classmethod(cls):
```

```
...         raise pd.errors.AbstractMethodError(cls, methodtype="classmethod")
```

```
...
```

```
...     def method(self):
```

```
...         raise pd.errors.AbstractMethodError(self)
```

```
>>> test = Foo.classmethod()
```

```
Traceback (most recent call last):
```

```
AbstractMethodError: This classmethod must be defined in the concrete class Foo
```

```
>>> test2 = Foo().method()
```

```
Traceback (most recent call last):
```

```
AbstractMethodError: This classmethod must be defined in the concrete class Foo
```

Method resolution order:

```
AbstractMethodError
builtins.NotImplementedError
builtins.RuntimeError
builtins.Exception
builtins.BaseException
builtins.object
```

Methods defined here:

```
__init__(self, class_instance, methodtype: str = 'method') -> None
    Initialize self. See help(type(self)) for accurate signature.
```

```
__str__(self) -> str
    Return str(self).
```

Data descriptors defined here:

```
__weakref__
    list of weak references to the object
```

Static methods inherited from `builtins.NotImplementedError`:

```
__new__(*args, **kwargs) class method of builtins.NotImplementedError
    Create and return a new object. See help(type) for accurate signature.
```

Methods inherited from `builtins.BaseException`:

```
__reduce__(self, /)
    Helper for pickle.
```

```
__repr__(self, /)
    Return repr(self).
```

```
__setstate__(self, state, /)
```

```
add_note(self, note, /)
    Add a note to the exception
```

```
with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
```

Data descriptors inherited from `builtins.BaseException`:

```
__cause__
```

```

|   __context__
|
|   __dict__
|
|   __suppress_context__
|
|   __traceback__
|
|   args
|
class AttributeConflictWarning(builtins.Warning)
|   Warning raised when index attributes conflict when using HDFStore.
|
|   Occurs when attempting to append an index with a different
|   name than the existing index on an HDFStore or attempting to append an index with a
|   different frequency than the existing index on an HDFStore.
|
|   See Also
|   -----
|   HDFStore : Dict-like IO interface for storing pandas objects in PyTables.
|   DataFrame.to_hdf : Write the contained data to an HDF5 file using HDFStore.
|   read_hdf : Read from an HDF5 file into a DataFrame.
|
|   Examples
|   -----
|   >>> idx1 = pd.Index(["a", "b"], name="name1")
|   >>> df1 = pd.DataFrame([[1, 2], [3, 4]], index=idx1)
|   >>> df1.to_hdf("file", "data", "w", append=True) # doctest: +SKIP
|   >>> idx2 = pd.Index(["c", "d"], name="name2")
|   >>> df2 = pd.DataFrame([[5, 6], [7, 8]], index=idx2)
|   >>> df2.to_hdf("file", "data", "a", append=True) # doctest: +SKIP
|   AttributeConflictWarning: the [index_name] attribute of the existing index is
|   [name1] which conflicts with the new [name2]...
|
|   Method resolution order:
|       AttributeConflictWarning
|       builtins.Warning
|       builtins.Exception
|       builtins.BaseException
|       builtins.object
|
|   Data descriptors defined here:
|
|   __weakref__
|       list of weak references to the object
|
|   -----
|   Static methods inherited from builtins.Warning:
|
|   __new__(*args, **kwargs) class method of builtins.Warning
|       Create and return a new object.  See help(type) for accurate signature.
|
|   -----
|   Methods inherited from builtins.BaseException:
|
|   __init__(self, /, *args, **kwargs)
|       Initialize self.  See help(type(self)) for accurate signature.
|
|   __reduce__(self, /)
|       Helper for pickle.
|
|   __repr__(self, /)
|       Return repr(self).
|
|   __setstate__(self, state, /)
|
|   __str__(self, /)
|       Return str(self).
|
|   add_note(self, note, /)
|       Add a note to the exception
|
|   with_traceback(self, tb, /)
|       Set self.__traceback__ to tb and return self.
|
|   -----
|   Data descriptors inherited from builtins.BaseException:

```

```

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class CSSWarning(builtins.UserWarning)
    Warning is raised when converting css styling fails.

    This can be due to the styling not having an equivalent value or because the
    styling isn't properly formatted.

    See Also
    -----
    DataFrame.style : Returns a Styler object for applying CSS-like styles.
    io.formats.style.Styler : Helps style a DataFrame or Series according to the
        data with HTML and CSS.
    io.formats.style.Styler.to_excel : Export styled DataFrame to Excel.
    io.formats.style.Styler.to_html : Export styled DataFrame to HTML.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 1, 1]})
    >>> df.style.map(lambda x: "background-color: blueGreenRed;").to_excel(
    ...     "styled.xlsx"
    ... ) # doctest: +SKIP
    CSSWarning: Unhandled color format: 'blueGreenRed'
    >>> df.style.map(lambda x: "border: 1px solid red red;").to_excel(
    ...     "styled.xlsx"
    ... ) # doctest: +SKIP
    CSSWarning: Unhandled color format: 'blueGreenRed'

    Method resolution order:
        CSSWarning
        builtins.UserWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.UserWarning:

    __new__(*args, **kwargs) class method of builtins.UserWarning
        Create and return a new object. See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

```

```

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class CategoricalConversionWarning(builtins.Warning)
    Warning is raised when reading a partial labeled Stata file using an iterator.

    This warning helps ensure data integrity and alerts users to potential issues
    during the incremental reading of Stata files with labeled data, allowing for
    additional checks and adjustments as necessary.

    See Also
    -----
    read_stata : Read a Stata file into a DataFrame.
    Categorical : Represents a categorical variable in pandas.

    Examples
    -----
    >>> from pandas.io.stata import StataReader
    >>> with StataReader("dta_file", chunksize=2) as reader: # doctest: +SKIP
    ...     for i, block in enumerate(reader):
    ...         print(i, block)
    ... # CategoricalConversionWarning: One or more series with value labels...

    Method resolution order:
    CategoricalConversionWarning
    builtins.Warning
    builtins.Exception
    builtins.BaseException
    builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object
    -----
    Static methods inherited from builtins.Warning:

    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object. See help(type) for accurate signature.
    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

```



```

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class ChainedAssignmentError(builtins.Warning)
    Warning raised when trying to set using chained assignment.

    With Copy-on-Write now always enabled, chained assignment can
    never work. In such a situation, we are always setting into a temporary
    object that is the result of an indexing operation (getitem), which under
    Copy-on-Write always behaves as a copy. Thus, assigning through a chain
    can never update the original Series or DataFrame.

    For more information on Copy-on-Write,
    see :ref:`the user guide<copy_on_write>`.

    See Also
    -----
    DataFrame.loc : Access a group of rows and columns by label(s) or a boolean array.
    DataFrame.iloc : Purely integer-location based indexing for selection by position.
    Series.loc : Access a group of rows by label(s) or a boolean array.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 1, 1, 2, 2]}, columns=["A"])
    >>> df["A"][0:3] = 10 # doctest: +SKIP
    ... # ChainedAssignmentError: ...

    Method resolution order:
        ChainedAssignmentError
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.Warning:

    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object. See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

```

```

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

```

Data descriptors inherited from builtins.BaseException:

```

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

```

```

class ClosedFileError(builtins.Exception)
    Exception is raised when trying to perform an operation on a closed HDFStore file.

    ``ClosedFileError`` is specific to operations on ``HDFStore`` objects. Once an
    HDFStore is closed, its resources are no longer available, and any further attempt
    to access data or perform file operations will raise this exception.

    See Also
    -----
    HDFStore.close : Closes the PyTables file handle.
    HDFStore.open : Opens the file in the specified mode.
    HDFStore.is_open : Returns a boolean indicating whether the file is open.

    Examples
    -----
    >>> store = pd.HDFStore("my-store", "a") # doctest: +SKIP
    >>> store.close() # doctest: +SKIP
    >>> store.keys() # doctest: +SKIP
    ... # ClosedFileError: my-store file is not open!

    Method resolution order:
        ClosedFileError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
        __weakref__
            list of weak references to the object

```

Static methods inherited from builtins.Exception:

```

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object. See help(type) for accurate signature.

```

Methods inherited from builtins.BaseException:

```

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

```

```

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class DataError(builtins.Exception)
    Exception raised when performing an operation on non-numerical data.

    For example, calling ``ohlc`` on a non-numerical column or a function
    on a rolling window.

    See Also
    -----
    Series.rolling : Provide rolling window calculations on Series object.
    DataFrame.rolling : Provide rolling window calculations on DataFrame object.

    Examples
    -----
    >>> ser = pd.Series(["a", "b", "c"])
    >>> ser.rolling(2).sum()
    Traceback (most recent call last):
    DataError: No numeric types to aggregate

    Method resolution order:
        DataError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.Exception:

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object.  See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

```

```

-----
Data descriptors inherited from builtins.BaseException:
    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class DatabaseError(builtins.OSError)
    Error is raised when executing SQL with bad syntax or SQL that throws an error.

    Raised by :func:`pandas.read_sql` when a bad SQL statement is passed in.

    See Also
    -----
    read_sql : Read SQL query or database table into a DataFrame.

    Examples
    -----
    >>> from sqlite3 import connect
    >>> conn = connect(":memory:")
    >>> pd.read_sql("select * test", conn) # doctest: +SKIP

    Method resolution order:
        DatabaseError
        builtins.OSError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
        __weakref__
            list of weak references to the object

    -----
    Methods inherited from builtins.OSError:
        __init__(self, /, *args, **kwargs)
            Initialize self. See help(type(self)) for accurate signature.

        __reduce__(self, /)
            Helper for pickle.

        __str__(self, /)
            Return str(self).

    -----
    Static methods inherited from builtins.OSError:
        __new__(*args, **kwargs) class method of builtins.OSError
            Create and return a new object. See help(type) for accurate signature.

    -----
    Data descriptors inherited from builtins.OSError:
    characters_written
    errno
        POSIX exception code
    filename
        exception filename
    filename2
        second exception filename
    strerror
        exception strerror

```

```

winerror
    Win32 exception code

-----
Methods inherited from builtins.BaseException:

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class DtypeWarning(builtins.Warning)
    Warning raised when reading different dtypes in a column from a file.

    Raised for a dtype incompatibility. This can happen whenever `read_csv`
    or `read_table` encounter non-uniform dtypes in a column(s) of a given
    CSV file.

    See Also
    -----
    read_csv : Read CSV (comma-separated) file into a DataFrame.
    read_table : Read general delimited file into a DataFrame.

    Notes
    -----
    This warning is issued when dealing with larger files because the dtype
    checking happens per chunk read.

    Despite the warning, the CSV file is read with mixed types in a single
    column which will be an object type. See the examples below to better
    understand this issue.

    Examples
    -----
    This example creates and reads a large CSV file with a column that contains
    `int` and `str`.

    >>> df = pd.DataFrame(
    ...     {
    ...         "a": (["1"] * 100000 + ["X"] * 100000 + ["1"] * 100000),
    ...         "b": ["b"] * 300000,
    ...     }
    ... ) # doctest: +SKIP
    >>> df.to_csv("test.csv", index=False) # doctest: +SKIP
    >>> df2 = pd.read_csv("test.csv") # doctest: +SKIP
    ... # DtypeWarning: Columns (0: a) have mixed types

    Important to notice that `df2` will contain both `str` and `int` for the
    same input, '1'.

    >>> df2.iloc[262140, 0] # doctest: +SKIP
    '1'
    >>> type(df2.iloc[262140, 0]) # doctest: +SKIP
    <class 'str'>
    >>> df2.iloc[262150, 0] # doctest: +SKIP
    1

```

```

>>> type(df2.iloc[262150, 0]) # doctest: +SKIP
<class 'int'>

One way to solve this issue is using the `dtype` parameter in the
`read_csv` and `read_table` functions to explicit the conversion:

>>> df2 = pd.read_csv("test.csv", sep=",", dtype={"a": str}) # doctest: +SKIP

No warning was issued.

Method resolution order:
    DtypeWarning
    builtins.Warning
    builtins.Exception
    builtins.BaseException
    builtins.object

Data descriptors defined here:
    __weakref__
        list of weak references to the object
-----
Static methods inherited from builtins.Warning:
    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object.  See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__

args

class DuplicateLabelError(builtins.ValueError)
    Error raised when an operation would introduce duplicate labels.

    This error is typically encountered when performing operations on objects
    with `allows_duplicate_labels=False` and the operation would result in
    duplicate labels in the index. Duplicate labels can lead to ambiguities
    in indexing and reduce data integrity.

    See Also
    -----
    Series.set_flags : Return a new ``Series`` object with updated flags.

```

DataFrame.set_flags : Return a new ``DataFrame`` object with updated flags.
Series.reindex : Conform ``Series`` object to new index with optional filling logic.
DataFrame.reindex : Conform ``DataFrame`` object to new index with optional filling logic.

Examples

```
>>> s = pd.Series([0, 1, 2], index=["a", "b", "c"]).set_flags(
...     allows_duplicate_labels=False
... )
>>> s.reindex(["a", "a", "b"])
Traceback (most recent call last):
...
DuplicateLabelError: Index has duplicates.
      positions
label
a      [0, 1]
```

Method resolution order:
DuplicateLabelError
builtins.ValueError
builtins.Exception
builtins.BaseException
builtins.object

Data descriptors defined here:

`__weakref__`
list of weak references to the object

Static methods inherited from builtins.ValueError:

`__new__(*args, **kwargs)` class method of builtins.ValueError
Create and return a new object. See help(type) for accurate signature.

Methods inherited from builtins.BaseException:

`__init__(self, /, *args, **kwargs)`
Initialize self. See help(type(self)) for accurate signature.

`__reduce__(self, /)`
Helper for pickle.

`__repr__(self, /)`
Return repr(self).

`__setstate__(self, state, /)`

`__str__(self, /)`
Return str(self).

`add_note(self, note, /)`
Add a note to the exception

`with_traceback(self, tb, /)`
Set self.__traceback__ to tb and return self.

Data descriptors inherited from builtins.BaseException:

`__cause__`

`__context__`

`__dict__`

`__suppress_context__`

`__traceback__`

args

```
class EmptyDataError(builtins.ValueError)
| Exception raised in ``pd.read_csv`` when empty data or header is encountered.
```

This error is typically encountered when attempting to read an empty file or an invalid file where no data or headers are present.

See Also

read_csv : Read a comma-separated values (CSV) file into DataFrame.
errors.ParserError : Exception that is raised by an error encountered in parsing file contents.
errors.DtypeWarning : Warning raised when reading different dtypes in a column from a file.

Examples

>>> from io import StringIO
>>> empty = StringIO()
>>> pd.read_csv(empty)
Traceback (most recent call last):
EmptyDataError: No columns to parse from file

Method resolution order:
EmptyDataError
builtins.ValueError
builtins.Exception
builtins.BaseException
builtins.object

Data descriptors defined here:

__weakref__
list of weak references to the object

Static methods inherited from builtins.ValueError:

__new__(*args, **kwargs) class method of builtins.ValueError
Create and return a new object. See help(type) for accurate signature.

Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
Helper for pickle.

__repr__(self, /)
Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
Return str(self).

add_note(self, note, /)
Add a note to the exception

with_traceback(self, tb, /)
Set self.__traceback__ to tb and return self.

Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class IncompatibilityWarning(builtins.Warning)
| Warning raised when trying to use where criteria on an incompatible HDF5 file.


```

Method resolution order:
    IncompatibilityWarning
    builtins.Warning
    builtins.Exception
    builtins.BaseException
    builtins.object

Data descriptors defined here:
    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.Warning:
    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:
    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:
    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__

args

class IncompatibleFrequency(builtins.TypeError)
    Raised when trying to compare or operate between Periods with different frequencies.

    This error occurs when performing operations between Period objects or
    PeriodArrays that have different frequencies that cannot be aligned,
    such as comparing or doing arithmetic on periods with mismatched frequencies.

    See Also
    -----
    Period : Represents a period of time.
    PeriodIndex : Immutable ndarray holding ordinal values.
    PeriodDtype : An ExtensionDtype for Period data.

    Examples
    -----
    Trying to compare Period objects with different frequencies:

    >>> pd.Period("2024-01", freq="M") - pd.Period("2024-01-01", freq="D")
    Traceback (most recent call last):
    IncompatibleFrequency: Input has different freq=D from Period(freq=M)

```

```

Method resolution order:
  IncompatibleFrequency
  builtins.TypeError
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:

__weakref__
    list of weak references to the object

-----
Static methods inherited from builtins.TypeError:

__new__(*args, **kwargs) class method of builtins.TypeError
    Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

```

```

class IndexingError(builtins.Exception)
    Exception is raised when trying to index and there is a mismatch in dimensions.

    Raised by properties like :attr:`.pandas.DataFrame.iloc` when
    an indexer is out of bounds or :attr:`.pandas.DataFrame.loc` when its index is
    unalignable to the frame index.

    See Also
    -----
    DataFrame.iloc : Purely integer-location based indexing for      selection by position.
    DataFrame.loc : Access a group of rows and columns by label(s)      or a boolean array.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 1, 1]})
    >>> df.loc[..., ..., "A"] # doctest: +SKIP
    ... # IndexingError: indexer may only contain one '...' entry
    >>> df = pd.DataFrame({"A": [1, 1, 1]})
    >>> df.loc[1, ..., ...] # doctest: +SKIP
    ... # IndexingError: Too many indexers

```

```

>>> df[pd.Series([True], dtype=bool)] # doctest: +SKIP
... # IndexingError: Unalignable boolean Series provided as indexer...
>>> s = pd.Series(range(2), index=pd.MultiIndex.from_product([["a", "b"], ["c"]]))
>>> s.loc["a", "c", "d"] # doctest: +SKIP
... # IndexingError: Too many indexers

Method resolution order:
  IndexingError
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:

__weakref__
    list of weak references to the object

-----
Static methods inherited from builtins.Exception:

__new__(*args, **kwargs) class method of builtins.Exception
    Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class IntCastingNaNError(builtins.ValueError)
    Exception raised when converting (`astype`) an array with NaN to an integer type.

    This error occurs when attempting to cast a data structure containing non-finite
    values (such as NaN or infinity) to an integer data type. Integer types do not
    support non-finite values, so such conversions are explicitly disallowed to
    prevent silent data corruption or unexpected behavior.

    See Also
    -----
    DataFrame.astype : Method to cast a pandas DataFrame object to a specified dtype.
    Series.astype : Method to cast a pandas Series object to a specified dtype.

    Examples
    -----
    >>> pd.DataFrame(np.array([[1, np.nan], [2, 3]]), dtype="i8")

```

```

Traceback (most recent call last):
IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer

Method resolution order:
  IntCastingNaNError
  builtins.ValueError
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:
  __weakref__
      list of weak references to the object
-----
Static methods inherited from builtins.ValueError:
  __new__(*args, **kwargs) class method of builtins.ValueError
      Create and return a new object.  See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
  __init__(self, /, *args, **kwargs)
      Initialize self.  See help(type(self)) for accurate signature.
  __reduce__(self, /)
      Helper for pickle.
  __repr__(self, /)
      Return repr(self).
  __setstate__(self, state, /)
  __str__(self, /)
      Return str(self).
  add_note(self, note, /)
      Add a note to the exception
  with_traceback(self, tb, /)
      Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
  __cause__
  __context__
  __dict__
  __suppress_context__
  __traceback__
  args

class InvalidColumnName(builtins.Warning)
    Warning raised by to_stata the column contains a non-valid stata name.

    Because the column name is an invalid Stata variable, the name needs to be
    converted.

    See Also
    -----
    DataFrame.to_stata : Export DataFrame object to Stata dta format.

    Examples
    -----
    >>> df = pd.DataFrame({"0categories": pd.Series([2, 2])})
    >>> df.to_stata("test") # doctest: +SKIP

    Method resolution order:
      InvalidColumnName
      builtins.Warning

```

```

    builtins.Exception
    builtins.BaseException
    builtins.object

Data descriptors defined here:
    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.Warning:
    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:
    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:
    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__

args

class InvalidComparison(builtins.Exception)
    Exception is raised by _validate_comparison_value to indicate an invalid comparison.

    Notes
    ----
    This is an internal error.

    Method resolution order:
        InvalidComparison
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
        __weakref__
            list of weak references to the object

    -----
    Static methods inherited from builtins.Exception:
        __new__(*args, **kwargs) class method of builtins.Exception
            Create and return a new object.  See help(type) for accurate signature.

```

```

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class InvalidIndexError(builtins.Exception)
    Exception raised when attempting to use an invalid index key.

    This exception is triggered when a user attempts to access or manipulate
    data in a pandas DataFrame or Series using an index key that is not valid
    for the given object. This may occur in cases such as using a malformed
    slice, a mismatched key for a ``MultiIndex``, or attempting to access an index
    element that does not exist.

    See Also
    -----
    MultiIndex : A multi-level, or hierarchical, index object for pandas objects.

    Examples
    -----
    >>> idx = pd.MultiIndex.from_product([["x", "y"], [0, 1]])
    >>> df = pd.DataFrame([[1, 1, 2, 2], [3, 3, 4, 4]], columns=idx)
    >>> df
           x      y
    0  1    1    0    1
    0  1    1    2    2
    1  3    3    4    4
    >>> df[:, 0]
    Traceback (most recent call last):
    InvalidIndexError: (slice(None, None, None), 0)

    Method resolution order:
        InvalidIndexError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.Exception:

```

```

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object. See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
args

class InvalidVersion(builtins.ValueError)
    An invalid version was found, users should refer to PEP 440.

    The ``InvalidVersion`` exception is raised when a version string is
    improperly formatted. Pandas uses this exception to ensure that all
    version strings are PEP 440 compliant.

    See Also
    -----
    util.version.Version : Class for handling and parsing version strings.

    Examples
    -----
    >>> pd.util.version.Version("1.")
    Traceback (most recent call last):
    InvalidVersion: Invalid version: '1.'

    Method resolution order:
        InvalidVersion
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object
-----
Static methods inherited from builtins.ValueError:

    __new__(*args, **kwargs) class method of builtins.ValueError
        Create and return a new object. See help(type) for accurate signature.

```

```

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class LossySetitemError(builtins.Exception)
    Raised when trying to do a __setitem__ on an np.ndarray that is not lossless.

    Notes
    ----
    This is an internal error.

    Method resolution order:
        LossySetitemError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.Exception:

__new__(*args, **kwargs) class method of builtins.Exception
    Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)

```



```

        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class MergeError(builtins.ValueError)
    Exception raised when merging data.

    Subclass of ``ValueError``.

    See Also
    -----
    DataFrame.join : For joining DataFrames on their indexes.
    merge : For merging two DataFrames on a common set of keys.

    Examples
    -----
    >>> left = pd.DataFrame(
    ...     {"a": ["a", "b", "b", "d"], "b": ["cat", "dog", "weasel", "horse"]},
    ...     index=range(4),
    ... )
    >>> right = pd.DataFrame(
    ...     {"a": ["a", "b", "c", "d"], "c": ["meow", "bark", "chirp", "nay"]},
    ...     index=range(4),
    ...     ).set_index("a")
    >>> left.join(
    ...     right,
    ...     on="a",
    ...     validate="one_to_one",
    ... )
    Traceback (most recent call last):
    MergeError: Merge keys are not unique in left dataset; not a one-to-one merge

    Method resolution order:
        MergeError
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.ValueError:

    __new__(*args, **kwargs) class method of builtins.ValueError
        Create and return a new object. See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

```

```

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class NoBufferPresent(builtins.Exception)
    Exception is raised in _get_data_buffer to signal that there is no requested buffer.

    Method resolution order:
        NoBufferPresent
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.Exception:

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__

```

```

    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class NullFrequencyError(builtins.ValueError)
    Exception raised when a ``freq`` cannot be null.

    Particularly ``DatetimeIndex.shift``, ``TimedeltaIndex.shift``,
    ``PeriodIndex.shift``.

    See Also
    -----
    Index.shift : Shift values of Index.
    Series.shift : Shift values of Series.

    Examples
    -----
    >>> df = pd.DatetimeIndex(["2011-01-01 10:00", "2011-01-01"], freq=None)
    >>> df.shift(2)
    Traceback (most recent call last):
    NullFrequencyError: Cannot shift with no freq

    Method resolution order:
        NullFrequencyError
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
        __weakref__
            list of weak references to the object

    -----
    Static methods inherited from builtins.ValueError:
        __new__(*args, **kwargs) class method of builtins.ValueError
            Create and return a new object.  See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:
        __init__(self, /, *args, **kwargs)
            Initialize self.  See help(type(self)) for accurate signature.

        __reduce__(self, /)
            Helper for pickle.

        __repr__(self, /)
            Return repr(self).

        __setstate__(self, state, /)

        __str__(self, /)
            Return str(self).

        add_note(self, note, /)
            Add a note to the exception

        with_traceback(self, tb, /)
            Set self.__traceback__ to tb and return self.

    -----
    Data descriptors inherited from builtins.BaseException:
        __cause__
        __context__

```

```

    __dict__
    __suppress_context__
    __traceback__
    args

class NumExprClobberingError(builtins.NameError)
    Exception raised when trying to use a built-in numexpr name as a variable name.

    ``eval`` or ``query`` will throw the error if the engine is set
    to 'numexpr'. 'numexpr' is the default engine value for these methods if the
    numexpr package is installed.

    See Also
    -----
    eval : Evaluate a Python expression as a string using various backends.
    DataFrame.query : Query the columns of a DataFrame with a boolean expression.

    Examples
    -----
    >>> df = pd.DataFrame({"abs": [1, 1, 1]})
    >>> df.query("abs > 2") # doctest: +SKIP
    ... # NumExprClobberingError: Variables in expression "(abs) > (2)" overlap...
    >>> sin, a = 1, 2
    >>> pd.eval("sin + a", engine="numexpr") # doctest: +SKIP
    ... # NumExprClobberingError: Variables in expression "(sin) + (a)" overlap...

    Method resolution order:
        NumExprClobberingError
        builtins.NameError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

    -----
    Methods inherited from builtins.NameError:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    -----
    Data descriptors inherited from builtins.NameError:

    name
        name

    -----
    Static methods inherited from builtins.Exception:

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object. See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)

```

```

        Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class NumbaUtilError(builtins.Exception)
    Error raised for unsupported Numba engine routines.

    See Also
    -----
    DataFrame.groupby : Group DataFrame using a mapper or by a Series of columns.
    Series.groupby : Group Series using a mapper or by a Series of columns.
    DataFrame.agg : Aggregate using one or more operations over the specified axis.
    Series.agg : Aggregate using one or more operations over the specified axis.

    Examples
    -----
    >>> df = pd.DataFrame(
    ...     {"key": ["a", "a", "b", "b"], "data": [1, 2, 3, 4]}, columns=["key", "data"]
    ... )
    >>> def incorrect_function(x):
    ...     return sum(x) * 2.7
    >>> df.groupby("key").agg(incorrect_function, engine="numba")
    Traceback (most recent call last):
    NumbaUtilError: The first 2 arguments to incorrect_function
    must be ['values', 'index']

    Method resolution order:
        NumbaUtilError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
    __weakref__
        list of weak references to the object
-----
Static methods inherited from builtins.Exception:
__new__(*args, **kwargs) class method of builtins.Exception
    Create and return a new object.  See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)

```

```

        Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class OptionError(builtins.AttributeError, builtins.KeyError)
    Exception raised for pandas.options.

    Backwards compatible with KeyError checks.

    See Also
    -----
    options : Access and modify global pandas settings.

    Examples
    -----
    >>> pd.options.context
    Traceback (most recent call last):
    OptionError: No such option

    Method resolution order:
        OptionError
        builtins.AttributeError
        builtins.KeyError
        builtins.LookupError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
    __weakref__
        list of weak references to the object
-----
Methods inherited from builtins.AttributeError:
    __getstate__(self, /)
        Helper for pickle.

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.
-----
Data descriptors inherited from builtins.AttributeError:
    name
        attribute name

    obj
        object
-----
Methods inherited from builtins.KeyError:
    __str__(self, /)
        Return str(self).
-----
Static methods inherited from builtins.LookupError:
    __new__(*args, **kwargs) class method of builtins.LookupError

```

```

        Create and return a new object. See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class OutOfBoundsDatetime(builtins.ValueError)
    Raised when the datetime is outside the range that can be represented.

    This error occurs when attempting to convert or parse a datetime value
    that exceeds the bounds supported by pandas' internal datetime
    representation.

    See Also
    -----
    to_datetime : Convert argument to datetime.
    Timestamp : Pandas replacement for python ``datetime.datetime`` object.

    Examples
    -----
    >>> pd.to_datetime("08335394550")
    Traceback (most recent call last):
    OutOfBoundsDatetime: Parsing "08335394550" to datetime overflows,
    at position 0

    Method resolution order:
        OutOfBoundsDatetime
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object
    -----
    Static methods inherited from builtins.ValueError:

    __new__(*args, **kwargs) class method of builtins.ValueError
        Create and return a new object. See help(type) for accurate signature.
    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

```

```

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class OutOfBoundsTimedelta(builtins.ValueError)
    Raised when encountering a timedelta value that cannot be represented.

    Representation should be within a timedelta64[ns].

    See Also
    -----
    date_range : Return a fixed frequency DatetimeIndex.

    Examples
    -----
    >>> pd.date_range(start="1/1/1700", freq="B", periods=100000, unit="ns")
    Traceback (most recent call last):
    OutOfBoundsTimedelta: Cannot cast 139999 days 00:00:00
    to unit='ns' without overflow.

    Method resolution order:
        OutOfBoundsTimedelta
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.ValueError:

    __new__(*args, **kwargs) class method of builtins.ValueError
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

```



```

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class Pandas4Warning(PandasDeprecationWarning)
    Warning raised for an upcoming change that will be enforced in pandas 4.0.

    See Also
    -----
    errors.PandasChangeWarning: Class for deprecations that will raise any warning.
    errors.PandasPendingDeprecationWarning : Class for deprecations that will raise a
        PendingDeprecationWarning.
    errors.PandasDeprecationWarning : Class for deprecations that will raise a
        DeprecationWarning.
    errors.PandasFutureWarning : Class for deprecations that will raise a FutureWarning.

    Examples
    -----
    >>> pd.errors.Pandas4Warning
    <class 'pandas.errors.Pandas4Warning'>

    Method resolution order:
        Pandas4Warning
        PandasDeprecationWarning
        PandasChangeWarning
        builtins.DeprecationWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods defined here:

    version() -> str
        Version where change will be enforced.

-----
Data descriptors inherited from PandasChangeWarning:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.DeprecationWarning:

    __new__(*args, **kwargs) class method of builtins.DeprecationWarning
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

```

```

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class Pandas5Warning(PandasPendingDeprecationWarning)
    Warning raised for an upcoming change that will be enforced in pandas 5.0.

    See Also
    -----
    errors.PandasChangeWarning: Class for deprecations that will raise any warning.
    errors.PandasPendingDeprecationWarning : Class for deprecations that will raise a
        PendingDeprecationWarning.
    errors.PandasDeprecationWarning : Class for deprecations that will raise a
        DeprecationWarning.
    errors.PandasFutureWarning : Class for deprecations that will raise a FutureWarning.

    Examples
    -----
    >>> pd.errors.Pandas5Warning
    <class 'pandas.errors.Pandas5Warning'>

    Method resolution order:
        Pandas5Warning
        PandasPendingDeprecationWarning
        PandasChangeWarning
        builtins.PendingDeprecationWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods defined here:

    version() -> str
        Version where change will be enforced.

    -----
    Data descriptors inherited from PandasChangeWarning:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.PendingDeprecationWarning:

    __new__(*args, **kwargs) class method of builtins.PendingDeprecationWarning
        Create and return a new object.  See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)

```

```

        Initialize self. See help(type(self)) for accurate signature.
__reduce__(self, /)
    Helper for pickle.
__repr__(self, /)
    Return repr(self).
__setstate__(self, state, /)
__str__(self, /)
    Return str(self).
add_note(self, note, /)
    Add a note to the exception
with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class PandasChangeWarning(builtins.Warning)
    Warning raised for any upcoming change.

    See Also
    -----
    errors.PandasPendingDeprecationWarning : Class for deprecations that will raise a
        PendingDeprecationWarning.
    errors.PandasDeprecationWarning : Class for deprecations that will raise a
        DeprecationWarning.
    errors.PandasFutureWarning : Class for deprecations that will raise a FutureWarning.

    Examples
    -----
    >>> pd.errors.PandasChangeWarning
    <class 'pandas.errors.PandasChangeWarning'>

    Method resolution order:
        PandasChangeWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods defined here:

    version() -> str
        Version where change will be enforced.
    -----
    Data descriptors defined here:
    __weakref__
        list of weak references to the object
    -----
    Static methods inherited from builtins.Warning:
    __new__(*args, **kwargs) class method of builtins.Warning
        Create and return a new object. See help(type) for accurate signature.
    -----
    Methods inherited from builtins.BaseException:

```

```

    __init__(self, /, *args, **kwargs)
        Initialize self.  See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class PandasDeprecationWarning(PandasChangeWarning, builtins.DeprecationWarning)
    Warning raised for an upcoming change that is a DeprecationWarning.

    See Also
    -----
    errors.PandasChangeWarning: Class for deprecations that will raise any warning.
    errors.PandasPendingDeprecationWarning : Class for deprecations that will raise a
        PendingDeprecationWarning.
    errors.PandasFutureWarning : Class for deprecations that will raise a FutureWarning.

    Examples
    -----
    >>> pd.errors.PandasDeprecationWarning
    <class 'pandas.errors.PandasDeprecationWarning'>

    Method resolution order:
        PandasDeprecationWarning
        PandasChangeWarning
        builtins.DeprecationWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods inherited from PandasChangeWarning:

    version() -> str
        Version where change will be enforced.

-----
Data descriptors inherited from PandasChangeWarning:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.DeprecationWarning:

    __new__(*args, **kwargs) class method of builtins.DeprecationWarning
        Create and return a new object.  See help(type) for accurate signature.

-----

```

```

Methods inherited from builtins.BaseException:
__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.
__reduce__(self, /)
    Helper for pickle.
__repr__(self, /)
    Return repr(self).
__setstate__(self, state, /)
__str__(self, /)
    Return str(self).
add_note(self, note, /)
    Add a note to the exception
with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class PandasFutureWarning(PandasChangeWarning, builtins.FutureWarning)
    Warning raised for an upcoming change that is a FutureWarning.

    See Also
    -----
    errors.PandasChangeWarning: Class for deprecations that will raise any warning.
    errors.PandasPendingDeprecationWarning : Class for deprecations that will raise a
        PendingDeprecationWarning.
    errors.PandasDeprecationWarning : Class for deprecations that will raise a
        DeprecationWarning.

    Examples
    -----
    >>> pd.errors.PandasFutureWarning
    <class 'pandas.errors.PandasFutureWarning'>

    Method resolution order:
        PandasFutureWarning
        PandasChangeWarning
        builtins.FutureWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods inherited from PandasChangeWarning:

    version() -> str
        Version where change will be enforced.
    -----
    Data descriptors inherited from PandasChangeWarning:

    __weakref__
        list of weak references to the object
    -----
    Static methods inherited from builtins.FutureWarning:

    __new__(*args, **kwargs) class method of builtins.FutureWarning

```

```

        Create and return a new object. See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class PandasPendingDeprecationWarning(PandasChangeWarning, builtins.PendingDeprecationWarning)
    Warning raised for an upcoming change that is a PendingDeprecationWarning.

    See Also
    -----
    errors.PandasChangeWarning: Class for deprecations that will raise any warning.
    errors.PandasDeprecationWarning : Class for deprecations that will raise a
        DeprecationWarning.
    errors.PandasFutureWarning : Class for deprecations that will raise a FutureWarning.

    Examples
    -----
    >>> pd.errors.PandasPendingDeprecationWarning
    <class 'pandas.errors.PandasPendingDeprecationWarning'>

    Method resolution order:
        PandasPendingDeprecationWarning
        PandasChangeWarning
        builtins.PendingDeprecationWarning
        builtins.Warning
        builtins.Exception
        builtins.BaseException
        builtins.object

    Class methods inherited from PandasChangeWarning:

    version() -> str
        Version where change will be enforced.
-----
Data descriptors inherited from PandasChangeWarning:
__weakref__
    list of weak references to the object
-----
Static methods inherited from builtins.PendingDeprecationWarning:

```

```

    __new__(*args, **kwargs) class method of builtins.PendingDeprecationWarning
        Create and return a new object. See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

    __init__(self, /, *args, **kwargs)
        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__

args

class ParserError(builtins.ValueError)
    Exception that is raised by an error encountered in parsing file contents.

    This is a generic error raised for errors encountered when functions like
    `read_csv` or `read_html` are parsing contents of a file.

    See Also
    -----
    read_csv : Read CSV (comma-separated) file into a DataFrame.
    read_html : Read HTML table into a DataFrame.

    Examples
    -----
    >>> data = '''a,b,c
    ... cat,foo,bar
    ... dog,foo,"baz'''
    >>> from io import StringIO
    >>> pd.read_csv(StringIO(data), skipfooter=1, engine="python")
    Traceback (most recent call last):
    ParserError: ',' expected after '""'. Error could possibly be due
    to parsing errors in the skipped footer rows

    Method resolution order:
        ParserError
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object
    -----

```

```

Static methods inherited from builtins.ValueError:
__new__(*args, **kwargs) class method of builtins.ValueError
    Create and return a new object.  See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__
args

class ParserWarning(builtins.Warning)
    Warning raised when reading a file that doesn't use the default 'c' parser.

    Raised by `pd.read_csv` and `pd.read_table` when it is necessary to change
    parsers, generally from the default 'c' parser to 'python'.

    It happens due to a lack of support or functionality for parsing a
    particular attribute of a CSV file with the requested engine.

    Currently, 'c' unsupported options include the following parameters:

    1. `sep` other than a single character (e.g. regex separators)
    2. `skipfooter` higher than 0

    The warning can be avoided by adding `engine='python'` as a parameter in
    `pd.read_csv` and `pd.read_table` methods.

    See Also
    -----
    pd.read_csv : Read CSV (comma-separated) file into DataFrame.
    pd.read_table : Read general delimited file into DataFrame.

    Examples
    -----
    Using a `sep` in `pd.read_csv` other than a single character:

    >>> import io
    >>> csv = '''a;b;c
    ...      1;1,8
    ...      1;2,1'''
    >>> df = pd.read_csv(io.StringIO(csv), sep="[:,]") # doctest: +SKIP
    ... # ParserWarning: Falling back to the 'python' engine...

    Adding `engine='python'` to `pd.read_csv` removes the Warning:

```



```

>>> df = pd.read_csv(io.StringIO(csv), sep=";", engine="python")

Method resolution order:
  ParserWarning
  builtins.Warning
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:
  __weakref__
    list of weak references to the object

-----
Static methods inherited from builtins.Warning:
  __new__(*args, **kwargs) class method of builtins.Warning
    Create and return a new object.  See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
  __init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

  __reduce__(self, /)
    Helper for pickle.

  __repr__(self, /)
    Return repr(self).

  __setstate__(self, state, /)

  __str__(self, /)
    Return str(self).

  add_note(self, note, /)
    Add a note to the exception

  with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
  __cause__
  __context__
  __dict__
  __suppress_context__
  __traceback__

args

class PerformanceWarning(builtins.Warning)
    Warning raised when there is a possible performance impact.

    See Also
    -----
    DataFrame.set_index : Set the DataFrame index using existing columns.
    DataFrame.loc : Access a group of rows and columns by label(s) or a boolean array.

    Examples
    -----
    >>> df = pd.DataFrame(
    ...     {"jim": [0, 0, 1, 1], "joe": ["x", "x", "z", "y"], "jolie": [1, 2, 3, 4]}
    ... )
    >>> df = df.set_index(["jim", "joe"])
    >>> df
           jolie
jim joe
0  x  1

```

```

      x      2
1      z      3
      y      4
>>> df.loc[(1, "z")] # doctest: +SKIP
# PerformanceWarning: indexing past lexsort depth may impact performance.
df.loc[(1, 'z')]
      jolie
jim  joe
1    z      3

Method resolution order:
  PerformanceWarning
  builtins.Warning
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:
  __weakref__
      list of weak references to the object

-----
Static methods inherited from builtins.Warning:
  __new__(*args, **kwargs) class method of builtins.Warning
      Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:
  __init__(self, /, *args, **kwargs)
      Initialize self.  See help(type(self)) for accurate signature.

  __reduce__(self, /)
      Helper for pickle.

  __repr__(self, /)
      Return repr(self).

  __setstate__(self, state, /)

  __str__(self, /)
      Return str(self).

  add_note(self, note, /)
      Add a note to the exception

  with_traceback(self, tb, /)
      Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:
  __cause__
  __context__
  __dict__
  __suppress_context__
  __traceback__

args

class PossibleDataLossError(builtins.Exception)
    Exception raised when trying to open an HDFStore file when already opened.

    This error is triggered when there is a potential risk of data loss due to
    conflicting operations on an HDFStore file. It serves to prevent unintended
    overwrites or data corruption by enforcing exclusive access to the file.

    See Also
    -----
    HDFStore : Dict-like IO interface for storing pandas objects in PyTables.
    HDFStore.open : Open an HDFStore file in the specified mode.

```

Examples

```
>>> store = pd.HDFStore("my-store", "a") # doctest: +SKIP
>>> store.open("w") # doctest: +SKIP
```

Method resolution order:

- PossibleDataLossError
- builtins.Exception
- builtins.BaseException
- builtins.object

Data descriptors defined here:

`__weakref__`
list of weak references to the object

Static methods inherited from builtins.Exception:

`__new__(*args, **kwargs)` class method of builtins.Exception
Create and return a new object. See help(type) for accurate signature.

Methods inherited from builtins.BaseException:

`__init__(self, /, *args, **kwargs)`
Initialize self. See help(type(self)) for accurate signature.

`__reduce__(self, /)`
Helper for pickle.

`__repr__(self, /)`
Return repr(self).

`__setstate__(self, state, /)`

`__str__(self, /)`
Return str(self).

`add_note(self, note, /)`
Add a note to the exception

`with_traceback(self, tb, /)`
Set self.__traceback__ to tb and return self.

Data descriptors inherited from builtins.BaseException:

`__cause__`

`__context__`

`__dict__`

`__suppress_context__`

`__traceback__`

args

```
class PossiblePrecisionLoss(builtins.Warning)
    Warning raised by to_stata on a column with a value outside or equal to int64.
```

When the column value is outside or equal to the int64 value the column is converted to a float64 dtype.

See Also

`DataFrame.to_stata` : Export DataFrame object to Stata dta format.

Examples

```
>>> df = pd.DataFrame({"s": pd.Series([1, 2**53], dtype=np.int64)})
>>> df.to_stata("test") # doctest: +SKIP
```

Method resolution order:

```

PossiblePrecisionLoss
builtins.Warning
builtins.Exception
builtins.BaseException
builtins.object

Data descriptors defined here:
__weakref__
    list of weak references to the object

-----
Static methods inherited from builtins.Warning:
__new__(*args, **kwargs) class method of builtins.Warning
    Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:
__init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:
__cause__
__context__
__dict__
__suppress_context__
__traceback__

args

class PyperclipException(builtins.RuntimeError)
    Exception raised when clipboard functionality is unsupported.

    Raised by ``to_clipboard()`` and ``read_clipboard()``.

    Method resolution order:
        PyperclipException
        builtins.RuntimeError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.RuntimeError:
    __new__(*args, **kwargs) class method of builtins.RuntimeError
        Create and return a new object.  See help(type) for accurate signature.

```

```

-----
Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class PyperclipWindowsException(PyperclipException)
    PyperclipWindowsException(message: str) -> None

    Exception raised when clipboard functionality is unsupported by Windows.

    Access to the clipboard handle would be denied due to some other
    window process is accessing it.

    Method resolution order:
        PyperclipWindowsException
        PyperclipException
        builtins.RuntimeError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Methods defined here:

    __init__(self, message: str) -> None
        Initialize self. See help(type(self)) for accurate signature.

    -----
    Data descriptors inherited from PyperclipException:

    __weakref__
        list of weak references to the object

    -----
    Static methods inherited from builtins.RuntimeError:

    __new__(*args, **kwargs) class method of builtins.RuntimeError
        Create and return a new object. See help(type) for accurate signature.

    -----
    Methods inherited from builtins.BaseException:

    __reduce__(self, /)
        Helper for pickle.

```

```

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class SpecificationError(builtins.Exception)
    Exception raised by ``agg`` when the functions are ill-specified.

    The exception raised in two scenarios.

    The first way is calling ``agg`` on a
    Dataframe or Series using a nested renamer (dict-of-dict).

    The second way is calling ``agg`` on a Dataframe with duplicated functions
    names without assigning column name.

    See Also
    -----
    DataFrame.agg : Aggregate using one or more operations over the specified axis.
    Series.agg : Aggregate using one or more operations over the specified axis.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 1, 1, 2, 2], "B": range(5), "C": range(5)})
    >>> df.groupby("A").B.agg({"foo": "count"}) # doctest: +SKIP
    ... # SpecificationError: nested renamer is not supported

    >>> df.groupby("A").agg({"B": {"foo": ["sum", "max"]}}) # doctest: +SKIP
    ... # SpecificationError: nested renamer is not supported

    >>> df.groupby("A").agg(["min", "min"]) # doctest: +SKIP
    ... # SpecificationError: nested renamer is not supported

    Method resolution order:
        SpecificationError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:

    __weakref__
        list of weak references to the object

-----
Static methods inherited from builtins.Exception:

    __new__(*args, **kwargs) class method of builtins.Exception
        Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

```

```

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

__cause__
__context__
__dict__
__suppress_context__
__traceback__

args

class UndefinedVariableError(builtins.NameError)
    UndefinedVariableError(name: str, is_local: bool | None = None) -> None

    Exception raised by ``query`` or ``eval`` when using an undefined variable name.

    It will also specify whether the undefined variable is local or not.

    Parameters
    -----
    name : str
        The name of the undefined variable.
    is_local : bool or None, optional
        Indicates whether the undefined variable is considered a local variable.
        If ``True``, the error message specifies it as a local variable.
        If ``False`` or ``None``, the variable is treated as a non-local name.

    See Also
    -----
    DataFrame.query : Query the columns of a DataFrame with a boolean expression.
    DataFrame.eval : Evaluate a string describing operations on DataFrame columns.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 1, 1]})
    >>> df.query("A > x") # doctest: +SKIP
    ... # UndefinedVariableError: name 'x' is not defined
    >>> df.query("A > @y") # doctest: +SKIP
    ... # UndefinedVariableError: local variable 'y' is not defined
    >>> pd.eval("x + 1") # doctest: +SKIP
    ... # UndefinedVariableError: name 'x' is not defined

    Method resolution order:
        UndefinedVariableError
        builtins.NameError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Methods defined here:

    __init__(self, name: str, is_local: bool | None = None) -> None

```

```

        Initialize self. See help(type(self)) for accurate signature.
-----
Data descriptors defined here:
__weakref__
    list of weak references to the object
-----
Data descriptors inherited from builtins.NameError:
name
    name
-----
Static methods inherited from builtins.Exception:
__new__(*args, **kwargs) class method of builtins.Exception
    Create and return a new object. See help(type) for accurate signature.
-----
Methods inherited from builtins.BaseException:
__reduce__(self, /)
    Helper for pickle.

__repr__(self, /)
    Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
    Return str(self).

add_note(self, note, /)
    Add a note to the exception

with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.
-----
Data descriptors inherited from builtins.BaseException:
__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

class UnsortedIndexError(builtins.KeyError)
    Error raised when slicing a MultiIndex which has not been lexsorted.

    Subclass of `KeyError`.

    See Also
    -----
    DataFrame.sort_index : Sort a DataFrame by its index.
    DataFrame.set_index : Set the DataFrame index using existing columns.

    Examples
    -----
    >>> df = pd.DataFrame(
    ...     {
    ...         "cat": [0, 0, 1, 1],
    ...         "color": ["white", "white", "brown", "black"],
    ...         "lives": [4, 4, 3, 7],
    ...     },
    ... )
    >>> df = df.set_index(["cat", "color"])
    >>> df
           lives
cat color
0  white      4
0  white      4
1  brown      3
1  black      7

```



```

cat  color
0    white    4
    white    4
1    brown    3
    black    7
>>> df.loc[(0, "black") : (1, "white")]
Traceback (most recent call last):
UnsortedIndexError: 'Key length (2) was greater
than MultiIndex lexsort depth (1)'

Method resolution order:
  UnsortedIndexError
  builtins.KeyError
  builtins.LookupError
  builtins.Exception
  builtins.BaseException
  builtins.object

Data descriptors defined here:
  __weakref__
    list of weak references to the object

-----
Methods inherited from builtins.KeyError:

  __str__(self, /)
    Return str(self).

-----
Static methods inherited from builtins.LookupError:

  __new__(*args, **kwargs) class method of builtins.LookupError
    Create and return a new object.  See help(type) for accurate signature.

-----
Methods inherited from builtins.BaseException:

  __init__(self, /, *args, **kwargs)
    Initialize self.  See help(type(self)) for accurate signature.

  __reduce__(self, /)
    Helper for pickle.

  __repr__(self, /)
    Return repr(self).

  __setstate__(self, state, /)

  add_note(self, note, /)
    Add a note to the exception

  with_traceback(self, tb, /)
    Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

  __cause__
  __context__
  __dict__
  __suppress_context__
  __traceback__

args

class UnsupportedFunctionCall(builtins.ValueError)
  Exception raised when attempting to call a unsupported numpy function.

  For example, ``np.cumsum(groupby_object)``.

  See Also
  -----

```

DataFrame.groupby : Group DataFrame using a mapper or by a Series of columns.
Series.groupby : Group Series using a mapper or by a Series of columns.
core.groupby.GroupBy.cumsum : Compute cumulative sum for each group.

Examples

```
>>> df = pd.DataFrame(
...     {"A": [0, 0, 1, 1], "B": ["x", "x", "z", "y"], "C": [1, 2, 3, 4]}
... )
>>> np.cumsum(df.groupby(["A"]))
Traceback (most recent call last):
UnsupportedFunctionCall: numpy operations are not valid with groupby.
Use .groupby(...).cumsum() instead
```

Method resolution order:
UnsupportedFunctionCall
builtins.ValueError
builtins.Exception
builtins.BaseException
builtins.object

Data descriptors defined here:

__weakref__
list of weak references to the object

Static methods inherited from builtins.ValueError:

__new__(*args, **kwargs) class method of builtins.ValueError
Create and return a new object. See help(type) for accurate signature.

Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
Helper for pickle.

__repr__(self, /)
Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
Return str(self).

add_note(self, note, /)
Add a note to the exception

with_traceback(self, tb, /)
Set self.__traceback__ to tb and return self.

Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

```
class ValueLabelTypeMismatch(builtins.Warning)
| Warning raised by to_stata on a category column that contains non-string values.
|
| When exporting data to Stata format using the `to_stata` method, category columns
| must have string values as labels. If a category column contains non-string values
| (e.g., integers, floats, or other types), this warning is raised to indicate that
| the Stata file may not correctly represent the data.
```

See Also

DataFrame.to_stata : Export DataFrame object to Stata dta format.
Series.cat : Accessor for categorical properties of the Series values.

Examples

>>> df = pd.DataFrame({"categories": pd.Series(["a", 2], dtype="category")})
>>> df.to_stata("test") # doctest: +SKIP

Method resolution order:
ValueLabelTypeMismatch
builtins.Warning
builtins.Exception
builtins.BaseException
builtins.object

Data descriptors defined here:

__weakref__
list of weak references to the object

Static methods inherited from builtins.Warning:

__new__(*args, **kwargs) class method of builtins.Warning
Create and return a new object. See help(type) for accurate signature.

Methods inherited from builtins.BaseException:

__init__(self, /, *args, **kwargs)
Initialize self. See help(type(self)) for accurate signature.

__reduce__(self, /)
Helper for pickle.

__repr__(self, /)
Return repr(self).

__setstate__(self, state, /)

__str__(self, /)
Return str(self).

add_note(self, note, /)
Add a note to the exception

with_traceback(self, tb, /)
Set self.__traceback__ to tb and return self.

Data descriptors inherited from builtins.BaseException:

__cause__

__context__

__dict__

__suppress_context__

__traceback__

args

DATA
__all__ = ['AbstractMethodError', 'AttributeConflictWarning', 'CSSWarn...

FILE
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\errors__init__

Help on package pandas.io in pandas:

NAME

pandas.io - # ruff: noqa: TC004

PACKAGE CONTENTS

- _util
- api
- clipboard (package)
- clipboards
- common
- excel (package)
- feather_format
- formats (package)
- html
- iceberg
- json (package)
- orc
- parquet
- parsers (package)
- pickle
- pytables
- sas (package)
- spss
- sql
- stata
- xml

DATA

TYPE_CHECKING = False

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\io__init__.py

Help on package pandas.plotting in pandas:

NAME

pandas.plotting - Plotting public API.

DESCRIPTION

Authors of third-party plotting backends should implement a module with a public ``plot(data, kind, \*\*kwargs)``. The parameter `data` will contain the data structure and can be a `Series` or a `DataFrame`. For example, for ``df.plot()`` the parameter `data` will contain the `DataFrame` `df`. In some cases, the data structure is transformed before being sent to the backend (see `PlotAccessor.__call__` in `pandas/plotting/_core.py` for the exact transformations).

The parameter `kind` will be one of:

- line
- bar
- barh
- box
- hist
- kde
- area
- pie
- scatter
- hexbin

See the pandas API reference for documentation on each kind of plot.

Any other keyword argument is currently assumed to be backend specific, but some parameters may be unified and added to the signature in the future (e.g. `title` which should be useful for any backend).

Currently, all the Matplotlib functions in pandas are accessed through the selected backend. For example, `pandas.plotting.boxplot` (equivalent to `DataFrame.boxplot`) is also accessed in the selected backend. This is expected to change, and the exact API is under discussion. But with the current version, backends are expected to implement the next functions:

- plot (describe above, used for `Series.plot` and `DataFrame.plot`)
- hist_series and hist_frame (for `Series.hist` and `DataFrame.hist`)
- boxplot (`pandas.plotting.boxplot(df)` equivalent to `DataFrame.boxplot`)
- boxplot_frame and boxplot_frame_groupby
- register and deregister (register converters for the tick formats)
- Plots not called as `Series` and `DataFrame` methods:

- table
- andrews_curves
- autocorrelation_plot
- bootstrap_plot
- lag_plot
- parallel_coordinates
- radviz
- scatter_matrix

Use the code in `pandas/plotting/_matplotlib.py` and <https://github.com/pyviz/hvplot> as a reference on how to write a backend.

For the discussion about the API see <https://github.com/pandas-dev/pandas/issues/26747>.

PACKAGE CONTENTS

```
_core
_matplotlib (package)
_misc
```

CLASSES

```
pandas.core.base.PandasObject(pandas.core.accessor.DirNamesMixin)
PlotAccessor
```

```
class PlotAccessor(pandas.core.base.PandasObject)
|   PlotAccessor(data: Series | DataFrame) -> None
|
|   Make plots of Series or DataFrame.
|
|   Uses the backend specified by the
|   option ``plotting.backend``. By default, matplotlib is used.
|
|   Parameters
|   -----
|   data : Series or DataFrame
|       The object for which the method is called.
|
|   Attributes
|   -----
|   x : label or position, default None
|       Only used if data is a DataFrame.
|   y : label, position or list of label, positions, default None
|       Allows plotting of one column versus another. Only used if data is a
|       DataFrame.
|   kind : str
|       The kind of plot to produce:
|
|       - 'line' : line plot (default)
|       - 'bar' : vertical bar plot
|       - 'barh' : horizontal bar plot
|       - 'hist' : histogram
|       - 'box' : boxplot
|       - 'kde' : Kernel Density Estimation plot
|       - 'density' : same as 'kde'
|       - 'area' : area plot
|       - 'pie' : pie plot
|       - 'scatter' : scatter plot (DataFrame only)
|       - 'hexbin' : hexbin plot (DataFrame only)
|   ax : matplotlib axes object, default None
|       An axes of the current figure.
|   subplots : bool or sequence of iterables, default False
|       Whether to group columns into subplots:
|
|       - ``False`` : No subplots will be used
|       - ``True`` : Make separate subplots for each column.
|       - sequence of iterables of column labels: Create a subplot for each
|         group of columns. For example ``[('a', 'c'), ('b', 'd')])`` will
|         create 2 subplots: one with columns 'a' and 'c', and one
|         with columns 'b' and 'd'. Remaining columns that aren't specified
|         will be plotted in additional subplots (one per column).
|
|   sharex : bool, default True if ax is None else False
|       In case ``subplots=True``, share x axis and set some x axis labels
|       to invisible; defaults to True if ax is None otherwise False if
|       an ax is passed in; Be aware, that passing in both an ax and
|       ``sharex=True`` will alter all x axis labels for all axis in a figure.
|   sharey : bool, default False
```

```

    In case ``subplots=True``, share y axis and set some y axis labels to invisible.
layout : tuple, optional
    (rows, columns) for the layout of subplots.
figsize : a tuple (width, height) in inches
    Size of a figure object.
use_index : bool, default True
    Use index as ticks for x axis.
title : str or list
    Title to use for the plot. If a string is passed, print the string
    at the top of the figure. If a list is passed and `subplots` is
    True, print each item in the list above the corresponding subplot.
grid : bool, default None (matlab style default)
    Axis grid lines.
legend : bool or {'reverse'}
    Place legend on axis subplots.
style : list or dict
    The matplotlib line style per column.
logx : bool or 'sym', default False
    Use log scaling or symlog scaling on x axis.

logy : bool or 'sym' default False
    Use log scaling or symlog scaling on y axis.

loglog : bool or 'sym', default False
    Use log scaling or symlog scaling on both x and y axes.

xticks : sequence
    Values to use for the xticks.
yticks : sequence
    Values to use for the yticks.
xlim : 2-tuple/list
    Set the x limits of the current axes.
ylim : 2-tuple/list
    Set the y limits of the current axes.
xlabel : label, optional
    Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the
    x-column name for planar plots.

    .. versionchanged:: 2.0.0

        Now applicable to histograms.

ylabel : label, optional
    Name to use for the ylabel on y-axis. Default will show no ylabel, or the
    y-column name for planar plots.

    .. versionchanged:: 2.0.0

        Now applicable to histograms.

rot : float, default None
    Rotation for ticks (xticks for vertical, yticks for horizontal
    plots).
fontsize : float, default None
    Font size for xticks and yticks.
colormap : str or matplotlib colormap object, default None
    Colormap to select colors from. If string, load colormap with that
    name from matplotlib.
colorbar : bool, optional
    If True, plot colorbar (only relevant for 'scatter' and 'hexbin'
    plots).
position : float
    Specify relative alignments for bar plot layout.
    From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5
    (center).
table : bool, Series or DataFrame, default False
    If True, draw a table using the data in the DataFrame and the data
    will be transposed to meet matplotlib's default layout.
    If a Series or DataFrame is passed, use passed data to draw a
    table.
yerr : DataFrame, Series, array-like, dict and str
    See :ref:`Plotting with Error Bars <visualization.errorbars>` for
    detail.
xerr : DataFrame, Series, array-like, dict and str
    Equivalent to yerr.
stacked : bool, default False in line and bar plots, and True in area plot
    If True, create stacked plot.

```

```

secondary_y : bool or sequence, default False
    Whether to plot on the secondary y-axis if a list/tuple, which
    columns to plot on secondary y-axis.
mark_right : bool, default True
    When using a secondary_y axis, automatically mark the column
    labels with "(right)" in the legend.
include_bool : bool, default is False
    If True, boolean values can be plotted.
backend : str, default None
    Backend to use instead of the backend specified in the option
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
    specify the ``plotting.backend`` for the whole session, set
    ``pd.options.plotting.backend``.
**kwargs
    Options to pass to matplotlib plotting method.

Returns
-----
:class:`matplotlib.axes.Axes` or numpy.ndarray of them
    If the backend is not the default matplotlib one, the return value
    will be the object returned by the backend.

See Also
-----
matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.
DataFrame.hist : Make a histogram.
DataFrame.boxplot : Make a box plot.
DataFrame.plot.scatter : Make a scatter plot with varying marker
    point size and color.
DataFrame.plot.hexbin : Make a hexagonal binning plot of
    two variables.
DataFrame.plot.kde : Make Kernel Density Estimate plot using
    Gaussian kernels.
DataFrame.plot.area : Make a stacked area plot.
DataFrame.plot.bar : Make a bar plot.
DataFrame.plot.barh : Make a horizontal bar plot.

Notes
-----
- See matplotlib documentation online for more on this subject
- If `kind` = 'bar' or 'barh', you can specify relative alignments
  for bar plot layout by `position` keyword.
  From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5
  (center)

Examples
-----
For Series:

.. plot::
   :context: close-figs

   >>> ser = pd.Series([1, 2, 3, 3])
   >>> plot = ser.plot(kind="hist", title="My plot")

For DataFrame:

.. plot::
   :context: close-figs

   >>> df = pd.DataFrame(
   ...     {
   ...         "length": [1.5, 0.5, 1.2, 0.9, 3],
   ...         "width": [0.7, 0.2, 0.15, 0.2, 1.1],
   ...     },
   ...     index=["pig", "rabbit", "duck", "chicken", "horse"],
   ... )
   >>> plot = df.plot(title="DataFrame Plot")

For SeriesGroupBy:

.. plot::
   :context: close-figs

   >>> lst = [-1, -2, -3, 1, 2, 3]
   >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
   >>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")

```

For DataFrameGroupBy:

```
.. plot::  
    :context: close-figs  
  
    >>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})  
    >>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")
```

Method resolution order:

```
PlotAccessor  
pandas.core.base.PandasObject  
pandas.core.accessor.DirNamesMixin  
builtins.object
```

Methods defined here:

```
__call__(self, *args, **kwargs) from pandas.plotting._core.PlotAccessor  
    Make plots of Series or DataFrame.
```

Uses the backend specified by the option ``plotting.backend``. By default, matplotlib is used.

Parameters

data : Series or DataFrame
 The object for which the method is called.

Attributes

x : label or position, default None
 Only used if data is a DataFrame.
y : label, position or list of label, positions, default None
 Allows plotting of one column versus another. Only used if data is a DataFrame.

kind : str
 The kind of plot to produce:

- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

ax : matplotlib axes object, default None
 An axes of the current figure.

subplots : bool or sequence of iterables, default False
 Whether to group columns into subplots:

- ``False`` : No subplots will be used
- ``True`` : Make separate subplots for each column.
- sequence of iterables of column labels: Create a subplot for each group of columns. For example ``[('a', 'c'), ('b', 'd')]`` will create 2 subplots: one with columns 'a' and 'c', and one with columns 'b' and 'd'. Remaining columns that aren't specified will be plotted in additional subplots (one per column).

sharex : bool, default True if ax is None else False
 In case ``subplots=True``, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in; Be aware, that passing in both an ax and ``sharex=True`` will alter all x axis labels for all axis in a figure.

sharey : bool, default False
 In case ``subplots=True``, share y axis and set some y axis labels to invisible.

layout : tuple, optional
 (rows, columns) for the layout of subplots.

figsize : a tuple (width, height) in inches
 Size of a figure object.

use_index : bool, default True
 Use index as ticks for x axis.

title : str or list
 Title to use for the plot. If a string is passed, print the string

at the top of the figure. If a list is passed and `subplots` is True, print each item in the list above the corresponding subplot.

grid : bool, default None (matlab style default)
Axis grid lines.

legend : bool or {'reverse'}
Place legend on axis subplots.

style : list or dict
The matplotlib line style per column.

logx : bool or 'sym', default False
Use log scaling or symlog scaling on x axis.

logy : bool or 'sym' default False
Use log scaling or symlog scaling on y axis.

loglog : bool or 'sym', default False
Use log scaling or symlog scaling on both x and y axes.

xticks : sequence
Values to use for the xticks.

yticks : sequence
Values to use for the yticks.

xlim : 2-tuple/list
Set the x limits of the current axes.

ylim : 2-tuple/list
Set the y limits of the current axes.

xlabel : label, optional
Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.

.. versionchanged:: 2.0.0
Now applicable to histograms.

ylabel : label, optional
Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.

.. versionchanged:: 2.0.0
Now applicable to histograms.

rot : float, default None
Rotation for ticks (xticks for vertical, yticks for horizontal plots).

fontsize : float, default None
Font size for xticks and yticks.

colormap : str or matplotlib colormap object, default None
Colormap to select colors from. If string, load colormap with that name from matplotlib.

colorbar : bool, optional
If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).

position : float
Specify relative alignments for bar plot layout.
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).

table : bool, Series or DataFrame, default False
If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout.
If a Series or DataFrame is passed, use passed data to draw a table.

yerr : DataFrame, Series, array-like, dict and str
See :ref:`Plotting with Error Bars <visualization.errorbars>` for detail.

xerr : DataFrame, Series, array-like, dict and str
Equivalent to yerr.

stacked : bool, default False in line and bar plots, and True in area plot
If True, create stacked plot.

secondary_y : bool or sequence, default False
Whether to plot on the secondary y-axis if a list/tuple, which columns to plot on secondary y-axis.

mark_right : bool, default True
When using a secondary_y axis, automatically mark the column labels with "(right)" in the legend.

include_bool : bool, default is False
If True, boolean values can be plotted.

backend : str, default None

Backend to use instead of the backend specified in the option
`plotting.backend`. For instance, 'matplotlib'. Alternatively, to
specify the `plotting.backend` for the whole session, set
`pd.options.plotting.backend`.

****kwargs**

Options to pass to matplotlib plotting method.

Returns

:class:`matplotlib.axes.Axes` or numpy.ndarray of them
If the backend is not the default matplotlib one, the return value
will be the object returned by the backend.

See Also

matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.

DataFrame.hist : Make a histogram.

DataFrame.boxplot : Make a box plot.

DataFrame.plot.scatter : Make a scatter plot with varying marker
point size and color.

DataFrame.plot.hexbin : Make a hexagonal binning plot of
two variables.

DataFrame.plot.kde : Make Kernel Density Estimate plot using
Gaussian kernels.

DataFrame.plot.area : Make a stacked area plot.

DataFrame.plot.bar : Make a bar plot.

DataFrame.plot.barh : Make a horizontal bar plot.

Notes

- See matplotlib documentation online for more on this subject
- If `kind` = 'bar' or 'barh', you can specify relative alignments
for bar plot layout by `position` keyword.
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5
(center)

Examples

For Series:

```
.. plot::
    :context: close-figs

    >>> ser = pd.Series([1, 2, 3, 3])
    >>> plot = ser.plot(kind="hist", title="My plot")
```

For DataFrame:

```
.. plot::
    :context: close-figs

    >>> df = pd.DataFrame(
    ...     {
    ...         "length": [1.5, 0.5, 1.2, 0.9, 3],
    ...         "width": [0.7, 0.2, 0.15, 0.2, 1.1],
    ...     },
    ...     index=["pig", "rabbit", "duck", "chicken", "horse"],
    ... )
    >>> plot = df.plot(title="DataFrame Plot")
```

For SeriesGroupBy:

```
.. plot::
    :context: close-figs

    >>> lst = [-1, -2, -3, 1, 2, 3]
    >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
    >>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")
```

For DataFrameGroupBy:

```
.. plot::
    :context: close-figs

    >>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})
    >>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")
```

```
__init__(self, data: Series | DataFrame) -> None from pandas.plotting._core.PlotAccessor
Initialize self. See help(type(self)) for accurate signature.
```

```
area(
    self,
    x: Hashable | None = None,
    y: Hashable | None = None,
    stacked: bool = True,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Draw a stacked area plot.
```

An area plot displays quantitative data visually.
This function wraps the matplotlib area function.

Parameters

x : label or position, optional
Coordinates for the X axis. By default uses the index.
y : label or position, optional
Column to plot. By default uses all columns.
stacked : bool, default True
Area plots are stacked by default. Set to False to create a
unstacked plot.
**kwargs
Additional keyword arguments are documented in
:meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or numpy.ndarray
Area plot, or array of area plots if subplots is True.

See Also

DataFrame.plot : Make plots of DataFrame using matplotlib.

Examples

Draw an area plot based on basic business metrics:

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(
...     {
...         "sales": [3, 2, 3, 9, 10, 6],
...         "signups": [5, 5, 6, 12, 14, 13],
...         "visits": [20, 42, 28, 62, 81, 50],
...     },
...     index=pd.date_range(
...         start="2018/01/01", end="2018/07/01", freq="ME"
...     ),
... )
>>> ax = df.plot.area()
```

Area plots are stacked by default. To produce an unstacked plot,
pass ``stacked=False``:

```
.. plot::
:context: close-figs

>>> ax = df.plot.area(stacked=False)
```

Draw an area plot for a single column:

```
.. plot::
:context: close-figs

>>> ax = df.plot.area(y="sales")
```

Draw with a different ``x``:

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(
```

```

...     {
...         "sales": [3, 2, 3],
...         "visits": [20, 42, 28],
...         "day": [1, 2, 3],
...     }
... )
>>> ax = df.plot.area(x="day")

```

```

bar(
    self,
    x: Hashable | None = None,
    y: Hashable | None = None,
    color: str | Sequence[str] | dict | None = None,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Vertical bar plot.

```

A bar plot is a plot that presents categorical data with rectangular bars with lengths proportional to the values that they represent. A bar plot shows comparisons among discrete categories. One axis of the plot shows the specific categories being compared, and the other axis represents a measured value.

Parameters

x : label or position, optional
 Allows plotting of one column versus another. If not specified, the index of the DataFrame is used.

y : label or position, optional
 Allows plotting of one column versus another. If not specified, all numerical columns are used.

color : str, array-like, or dict, optional
 The color for each of the DataFrame's columns. Possible values are:

- A single color string referred to by name, RGB or RGBA code, for instance 'red' or '#a98d19'.
- A sequence of color strings referred to by name, RGB or RGBA code, which will be used for each column recursively. For instance ['green', 'yellow'] each column's bar will be filled in green or yellow, alternatively. If there is only a single column to be plotted, then only the first color from the color list will be used.
- A dict of the form {column name : color}, so that each column will be colored accordingly. For example, if your columns are called 'a' and 'b', then passing {'a': 'green', 'b': 'red'} will color bars for column 'a' in green and bars for column 'b' in red.

**kwargs

Additional keyword arguments are documented in :meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or np.ndarray of them
 An ndarray is returned with one :class:`matplotlib.axes.Axes` per column when ``subplots=True``.

See Also

`DataFrame.plot.barh` : Horizontal bar plot.
`DataFrame.plot` : Make plots of a DataFrame.
`matplotlib.pyplot.bar` : Make a bar plot with matplotlib.

Examples

Basic plot.

```

.. plot::
    :context: close-figs

    >>> df = pd.DataFrame({"lab": ["A", "B", "C"], "val": [10, 30, 20]})
    >>> ax = df.plot.bar(x="lab", y="val", rot=0)

```

Plot a whole dataframe to a bar plot. Each column is assigned a distinct color, and each row is nested in a group along the

horizontal axis.

```
.. plot::
:context: close-figs

>>> speed = [0.1, 17.5, 40, 48, 52, 69, 88]
>>> lifespan = [2, 8, 70, 1.5, 25, 12, 28]
>>> index = [
...     "snail",
...     "pig",
...     "elephant",
...     "rabbit",
...     "giraffe",
...     "coyote",
...     "horse",
... ]
>>> df = pd.DataFrame({"speed": speed, "lifespan": lifespan}, index=index)
>>> ax = df.plot.bar(rot=0)
```

Plot stacked bar charts for the DataFrame

```
.. plot::
:context: close-figs

>>> ax = df.plot.bar(stacked=True)
```

Instead of nesting, the figure can be split by column with ``subplots=True``. In this case, a :class:`numpy.ndarray` of :class:`matplotlib.axes.Axes` are returned.

```
.. plot::
:context: close-figs

>>> axes = df.plot.bar(rot=0, subplots=True)
>>> axes[1].legend(loc=2) # doctest: +SKIP
```

If you don't like the default colours, you can specify how you'd like each column to be colored.

```
.. plot::
:context: close-figs

>>> axes = df.plot.bar(
...     rot=0,
...     subplots=True,
...     color={"speed": "red", "lifespan": "green"},
... )
>>> axes[1].legend(loc=2) # doctest: +SKIP
```

Plot a single column.

```
.. plot::
:context: close-figs

>>> ax = df.plot.bar(y="speed", rot=0)
```

Plot only selected categories for the DataFrame.

```
.. plot::
:context: close-figs

>>> ax = df.plot.bar(x="lifespan", rot=0)
```

```
barh(
    self,
    x: Hashable | None = None,
    y: Hashable | None = None,
    color: str | Sequence[str] | dict | None = None,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Make a horizontal bar plot.
```

A horizontal bar plot is a plot that presents quantitative data with rectangular bars with lengths proportional to the values that they represent. A bar plot shows comparisons among discrete categories. One axis of the plot shows the specific categories being compared, and the other axis represents a measured value.

Parameters

`x` : label or position, optional
Allows plotting of one column versus another. If not specified, the index of the DataFrame is used.

`y` : label or position, optional
Allows plotting of one column versus another. If not specified, all numerical columns are used.

`color` : str, array-like, or dict, optional
The color for each of the DataFrame's columns. Possible values are:

- A single color string referred to by name, RGB or RGBA code, for instance 'red' or '#a98d19'.
- A sequence of color strings referred to by name, RGB or RGBA code, which will be used for each column recursively. For instance ['green','yellow'] each column's bar will be filled in green or yellow, alternatively. If there is only a single column to be plotted, then only the first color from the color list will be used.
- A dict of the form {column name : color}, so that each column will be colored accordingly. For example, if your columns are called 'a' and 'b', then passing {'a': 'green', 'b': 'red'} will color bars for column 'a' in green and bars for column 'b' in red.

**kwargs

Additional keyword arguments are documented in :meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or np.ndarray of them
An ndarray is returned with one :class:`matplotlib.axes.Axes` per column when ``subplots=True``.

See Also

`DataFrame.plot.bar` : Vertical bar plot.
`DataFrame.plot` : Make plots of DataFrame using matplotlib.
`matplotlib.axes.Axes.bar` : Plot a vertical bar plot using matplotlib.

Examples

Basic example

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame({"lab": ["A", "B", "C"], "val": [10, 30, 20]})
>>> ax = df.plot.barh(x="lab", y="val")
```

Plot a whole DataFrame to a horizontal bar plot

```
.. plot::
:context: close-figs

>>> speed = [0.1, 17.5, 40, 48, 52, 69, 88]
>>> lifespan = [2, 8, 70, 1.5, 25, 12, 28]
>>> index = [
...     "snail",
...     "pig",
...     "elephant",
...     "rabbit",
...     "giraffe",
...     "coyote",
...     "horse",
... ]
>>> df = pd.DataFrame({"speed": speed, "lifespan": lifespan}, index=index)
>>> ax = df.plot.barh()
```

Plot stacked barh charts for the DataFrame

```
.. plot::
:context: close-figs
```

```
>>> ax = df.plot.barh(stacked=True)
```

We can specify colors for each column

```
.. plot::  
:context: close-figs
```

```
>>> ax = df.plot.barh(color={"speed": "red", "lifespan": "green"})
```

Plot a column of the DataFrame to a horizontal bar plot

```
.. plot::  
:context: close-figs
```

```
>>> speed = [0.1, 17.5, 40, 48, 52, 69, 88]
```

```
>>> lifespan = [2, 8, 70, 1.5, 25, 12, 28]
```

```
>>> index = [
```

```
...     "snail",
```

```
...     "pig",
```

```
...     "elephant",
```

```
...     "rabbit",
```

```
...     "giraffe",
```

```
...     "coyote",
```

```
...     "horse",
```

```
... ]
```

```
>>> df = pd.DataFrame({"speed": speed, "lifespan": lifespan}, index=index)
```

```
>>> ax = df.plot.barh(y="speed")
```

Plot DataFrame versus the desired column

```
.. plot::  
:context: close-figs
```

```
>>> speed = [0.1, 17.5, 40, 48, 52, 69, 88]
```

```
>>> lifespan = [2, 8, 70, 1.5, 25, 12, 28]
```

```
>>> index = [
```

```
...     "snail",
```

```
...     "pig",
```

```
...     "elephant",
```

```
...     "rabbit",
```

```
...     "giraffe",
```

```
...     "coyote",
```

```
...     "horse",
```

```
... ]
```

```
>>> df = pd.DataFrame({"speed": speed, "lifespan": lifespan}, index=index)
```

```
>>> ax = df.plot.barh(x="lifespan")
```

box(self, by: IndexLabel | None = None, **kwargs) -> PlotAccessor from pandas.plotting._core
Make a box plot of the DataFrame columns.

A box plot is a method for graphically depicting groups of numerical data through their quartiles.

The box extends from the Q1 to Q3 quartile values of the data, with a line at the median (Q2). The whiskers extend from the edges of box to show the range of the data. The position of the whiskers is set by default to 1.5*IQR (IQR = Q3 - Q1) from the edges of the box. Outlier points are those past the end of the whiskers.

For further details see Wikipedia's

entry for ``boxplot`` <https://en.wikipedia.org/wiki/Box_plot>`__.

A consideration when using this chart is that the box and the whiskers can overlap, which is very common when plotting small sets of data.

Parameters

by : str or sequence

Column in the DataFrame to group by.

**kwargs

Additional keywords are documented in

:meth:`DataFrame.plot`.

Returns

:class:`matplotlib.axes.Axes` or numpy.ndarray of them

The matplotlib axes containing the box plot.

See Also

DataFrame.boxplot: Another method to draw a box plot.
Series.plot.box: Draw a box plot from a Series object.
matplotlib.pyplot.boxplot: Draw a box plot in matplotlib.

Examples

Draw a box plot from a DataFrame with four columns of randomly generated data.

```
.. plot::  
:context: close-figs  
  
>>> data = np.random.randn(25, 4)  
>>> df = pd.DataFrame(data, columns=list("ABCD"))  
>>> ax = df.plot.box()
```

You can also generate groupings if you specify the `by` parameter (which can take a column name, or a list or tuple of column names):

```
.. plot::  
:context: close-figs  
  
>>> age_list = [8, 10, 12, 14, 72, 74, 76, 78, 20, 25, 30, 35, 60, 85]  
>>> df = pd.DataFrame({"gender": list("MMMMMMMMMMMMMMMM"), "age": age_list})  
>>> ax = df.plot.box(column="age", by="gender", figsize=(10, 8))
```

```
density = kde(  
    self,  
    bw_method: Literal['scott', 'silverman'] | float | Callable | None = None,  
    ind: np.ndarray | int | None = None,  
    weights: np.ndarray | None = None,  
    **kwargs  
) -> PlotAccessor
```

```
hexbin(  
    self,  
    x: Hashable,  
    y: Hashable,  
    C: Hashable | None = None,  
    reduce_C_function: Callable | None = None,  
    gridsize: int | tuple[int, int] | None = None,  
    **kwargs  
) -> PlotAccessor from pandas.plotting._core.PlotAccessor  
Generate a hexagonal binning plot.
```

Generate a hexagonal binning plot of `x` versus `y`. If `C` is `None` (the default), this is a histogram of the number of occurrences of the observations at `(x[i], y[i])`.

If `C` is specified, specifies values at given coordinates `(x[i], y[i])`. These values are accumulated for each hexagonal bin and then reduced according to `reduce\_C\_function`, having as default the NumPy's mean function (:meth:`numpy.mean`). (If `C` is specified, it must also be a 1-D sequence of the same length as `x` and `y`, or a column label.)

Parameters

x : int or str
The column label or position for x points.
y : int or str
The column label or position for y points.
C : int or str, optional
The column label or position for the value of `(x, y)` point.
reduce_C_function : callable, default `np.mean`
Function of one argument that reduces all the values in a bin to a single number (e.g. `np.mean`, `np.max`, `np.sum`, `np.std`).
gridsize : int or tuple of (int, int), default 100
The number of hexagons in the x-direction.
The corresponding number of hexagons in the y-direction is chosen in a way that the hexagons are approximately regular. Alternatively, gridsize can be a tuple with two elements specifying the number of hexagons in the x-direction and the y-direction.

****kwargs**
Additional keyword arguments are documented in
:meth:`DataFrame.plot`.

Returns

matplotlib.Axes
The matplotlib ``Axes`` on which the hexbin is plotted.

See Also

DataFrame.plot : Make plots of a DataFrame.
matplotlib.pyplot.hexbin : Hexagonal binning plot using matplotlib,
the matplotlib function that is used under the hood.

Examples

The following examples are generated with random data from
a normal distribution.

```
.. plot::
    :context: close-figs

    >>> n = 10000
    >>> df = pd.DataFrame({"x": np.random.randn(n), "y": np.random.randn(n)})
    >>> ax = df.plot.hexbin(x="x", y="y", gridsize=20)
```

The next example uses ``C`` and ``np.sum`` as ``reduce\_C\_function``.
Note that ``'observations'`` values ranges from 1 to 5 but the result
plot shows values up to more than 25. This is because of the
``reduce\_C\_function``.

```
.. plot::
    :context: close-figs

    >>> n = 500
    >>> df = pd.DataFrame(
    ...     {
    ...         "coord_x": np.random.uniform(-3, 3, size=n),
    ...         "coord_y": np.random.uniform(30, 50, size=n),
    ...         "observations": np.random.randint(1, 5, size=n),
    ...     }
    ... )
    >>> ax = df.plot.hexbin(
    ...     x="coord_x",
    ...     y="coord_y",
    ...     C="observations",
    ...     reduce_C_function=np.sum,
    ...     gridsize=10,
    ...     cmap="viridis",
    ... )
```

hist(self, by: IndexLabel | None = None, bins: int = 10, **kwargs) -> PlotAccessor from pandas
Draw one histogram of the DataFrame's columns.

A histogram is a representation of the distribution of data.
This function groups the values of all given Series in the DataFrame
into bins and draws all bins in one :class:`matplotlib.axes.Axes`.
This is useful when the DataFrame's Series are in a similar scale.

Parameters

by : str or sequence, optional
Column in the DataFrame to group by.
bins : int, default 10
Number of histogram bins to be used.

****kwargs**
Additional keyword arguments are documented in
:meth:`DataFrame.plot`.

Returns

:class:`matplotlib.axes.Axes`
Return a histogram plot.

See Also

DataFrame.hist : Draw histograms per DataFrame's Series.
Series.hist : Draw a histogram with Series' data.

Examples

When we roll a die 6000 times, we expect to get each value around 1000 times. But when we roll two dice and sum the result, the distribution is going to be quite different. A histogram illustrates those distributions.

```
.. plot::
   :context: close-figs

   >>> df = pd.DataFrame(np.random.randint(1, 7, 6000), columns=["one"])
   >>> df["two"] = df["one"] + np.random.randint(1, 7, 6000)
   >>> ax = df.plot.hist(bins=12, alpha=0.5)
```

A grouped histogram can be generated by providing the parameter `by` (which can be a column name, or a list of column names):

```
.. plot::
   :context: close-figs

   >>> age_list = [8, 10, 12, 14, 72, 74, 76, 78, 20, 25, 30, 35, 60, 85]
   >>> df = pd.DataFrame({"gender": list("MMMMMMMMMMMMMMMM"), "age": age_list})
   >>> ax = df.plot.hist(column=["age"], by="gender", figsize=(10, 8))
```

```
kde(
    self,
    bw_method: Literal['scott', 'silverman'] | float | Callable | None = None,
    ind: np.ndarray | int | None = None,
    weights: np.ndarray | None = None,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Generate Kernel Density Estimate plot using Gaussian kernels.
```

In statistics, `kernel density estimation`_ (KDE) is a non-parametric way to estimate the probability density function (PDF) of a random variable. This function uses Gaussian kernels and includes automatic bandwidth determination.

.. _kernel density estimation:
https://en.wikipedia.org/wiki/Kernel_density_estimation

Parameters

bw_method : str, scalar or callable, optional
The method used to calculate the estimator bandwidth. This can be 'scott', 'silverman', a scalar constant or a callable. If None (default), 'scott' is used. See :class:`scipy.stats.gaussian\_kde` for more information.
ind : NumPy array or int, optional
Evaluation points for the estimated PDF. If None (default), 1000 equally spaced points are used. If `ind` is a NumPy array, the KDE is evaluated at the points passed. If `ind` is an integer, `ind` number of equally spaced points are used.
weights : NumPy array, optional
Weights of datapoints. This must be the same shape as datapoints. If None (default), the samples are assumed to be equally weighted.
****kwargs**
Additional keyword arguments are documented in :meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or numpy.ndarray of them
The matplotlib axes containing the KDE plot.

See Also

scipy.stats.gaussian_kde : Representation of a kernel-density estimate using Gaussian kernels. This is the function used internally to estimate the PDF.

Examples

Given a Series of points randomly sampled from an unknown

distribution, estimate its PDF using KDE with automatic bandwidth determination and plot the results, evaluating them at 1000 equally spaced points (default):

```
.. plot::
   :context: close-figs

   >>> s = pd.Series([1, 2, 2.5, 3, 3.5, 4, 5])
   >>> ax = s.plot.kde()
```

A scalar bandwidth can be specified. Using a small bandwidth value can lead to over-fitting, while using a large bandwidth value may result in under-fitting:

```
.. plot::
   :context: close-figs

   >>> ax = s.plot.kde(bw_method=0.3)
```

```
.. plot::
   :context: close-figs

   >>> ax = s.plot.kde(bw_method=3)
```

Finally, the `ind` parameter determines the evaluation points for the plot of the estimated PDF:

```
.. plot::
   :context: close-figs

   >>> ax = s.plot.kde(ind=[1, 2, 3, 4, 5])
```

For DataFrame, it works in the same way:

```
.. plot::
   :context: close-figs

   >>> df = pd.DataFrame(
   ...     {
   ...         "x": [1, 2, 2.5, 3, 3.5, 4, 5],
   ...         "y": [4, 4, 4.5, 5, 5.5, 6, 6],
   ...     }
   ... )
   >>> ax = df.plot.kde()
```

A scalar bandwidth can be specified. Using a small bandwidth value can lead to over-fitting, while using a large bandwidth value may result in under-fitting:

```
.. plot::
   :context: close-figs

   >>> ax = df.plot.kde(bw_method=0.3)
```

```
.. plot::
   :context: close-figs

   >>> ax = df.plot.kde(bw_method=3)
```

Finally, the `ind` parameter determines the evaluation points for the plot of the estimated PDF:

```
.. plot::
   :context: close-figs

   >>> ax = df.plot.kde(ind=[1, 2, 3, 4, 5, 6])
```

```
line(
    self,
    x: Hashable | None = None,
    y: Hashable | None = None,
    color: str | Sequence[str] | dict | None = None,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Plot Series or DataFrame as lines.
```

This function is useful to plot lines using DataFrame's values

as coordinates.

Parameters

x : label or position, optional
Allows plotting of one column versus another. If not specified, the index of the DataFrame is used.

y : label or position, optional
Allows plotting of one column versus another. If not specified, all numerical columns are used.

color : str, array-like, or dict, optional
The color for each of the DataFrame's columns. Possible values are:

- A single color string referred to by name, RGB or RGBA code, for instance 'red' or '#a98d19'.
- A sequence of color strings referred to by name, RGB or RGBA code, which will be used for each column recursively. For instance ['green', 'yellow'] each column's line will be filled in green or yellow, alternatively. If there is only a single column to be plotted, then only the first color from the color list will be used.
- A dict of the form {column name : color}, so that each column will be colored accordingly. For example, if your columns are called 'a' and 'b', then passing {'a': 'green', 'b': 'red'} will color lines for column 'a' in green and lines for column 'b' in red.

**kwargs

Additional keyword arguments are documented in
:meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or np.ndarray of them
An ndarray is returned with one :class:`matplotlib.axes.Axes` per column when ``subplots=True``.

See Also

matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.

Examples

```
.. plot::
:context: close-figs

>>> s = pd.Series([1, 3, 2])
>>> s.plot.line() # doctest: +SKIP
```

```
.. plot::
:context: close-figs

The following example shows the populations for some animals
over the years.
```

```
>>> df = pd.DataFrame(
...     {
...         "pig": [20, 18, 489, 675, 1776],
...         "horse": [4, 25, 281, 600, 1900],
...     },
...     index=[1990, 1997, 2003, 2009, 2014],
... )
>>> lines = df.plot.line()
```

```
.. plot::
:context: close-figs

An example with subplots, so an array of axes is returned.

>>> axes = df.plot.line(subplots=True)
>>> type(axes)
<class 'numpy.ndarray'>
```

```
.. plot::
:context: close-figs
```

Let's repeat the same example, but specifying colors for each column (in this case, for each animal).

```
>>> axes = df.plot.line(
...     subplots=True, color={"pig": "pink", "horse": "#742802"}
... )
```

```
.. plot::
:context: close-figs
```

The following example shows the relationship between both populations.

```
>>> lines = df.plot.line(x="pig", y="horse")
```

```
pie(self, y: IndexLabel | None = None, **kwargs) -> PlotAccessor from pandas.plotting._core.
Generate a pie plot.
```

A pie plot is a proportional representation of the numerical data in a column. This function wraps :meth:`matplotlib.pyplot.pie` for the specified column. If no column reference is passed and ``subplots=True`` a pie plot is drawn for each numerical column independently.

Parameters

y : int or label, optional
Label or position of the column to plot.
If not provided, ``subplots=True`` argument must be passed.
**kwargs
Keyword arguments to pass on to :meth:`DataFrame.plot`.

Returns

matplotlib.axes.Axes or np.ndarray of them
A NumPy array is returned when ``subplots`` is True.

See Also

Series.plot.pie : Generate a pie plot for a Series.
DataFrame.plot : Make plots of a DataFrame.

Examples

In the example below we have a DataFrame with the information about planet's mass and radius. We pass the 'mass' column to the pie function to get a pie plot.

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(
...     {"mass": [0.330, 4.87, 5.97], "radius": [2439.7, 6051.8, 6378.1]},
...     index=["Mercury", "Venus", "Earth"],
... )
>>> plot = df.plot.pie(y="mass", figsize=(5, 5))
```

```
.. plot::
:context: close-figs

>>> plot = df.plot.pie(subplots=True, figsize=(11, 6))
```

```
scatter(
    self,
    x: Hashable,
    y: Hashable,
    s: Hashable | Sequence[Hashable] | None = None,
    c: Hashable | Sequence[Hashable] | None = None,
    **kwargs
) -> PlotAccessor from pandas.plotting._core.PlotAccessor
Create a scatter plot with varying marker point size and color.
```

The coordinates of each point are defined by two dataframe columns and filled circles are used to represent each point. This kind of plot is useful to see complex correlations between two variables. Points could be for instance natural 2D coordinates like longitude and latitude in

a map or, in general, any pair of metrics that can be plotted against each other.

Parameters

x : int or str
The column name or column position to be used as horizontal coordinates for each point.

y : int or str
The column name or column position to be used as vertical coordinates for each point.

s : str, scalar or array-like, optional
The size of each point. Possible values are:

- A string with the name of the column to be used for marker's size.
- A single scalar so all points have the same size.
- A sequence of scalars, which will be used for each point's size recursively. For instance, when passing [2,14] all points size will be either 2 or 14, alternatively.

c : str, int or array-like, optional
The color of each point. Possible values are:

- A single color string referred to by name, RGB or RGBA code, for instance 'red' or '#a98d19'.
- A sequence of color strings referred to by name, RGB or RGBA code, which will be used for each point's color recursively. For instance ['green','yellow'] all points will be filled in green or yellow, alternatively.
- A column name or position whose values will be used to color the marker points according to a colormap.

**kwargs

Keyword arguments to pass on to :meth:`DataFrame.plot`.

Returns

:class:`matplotlib.axes.Axes` or numpy.ndarray of them
The matplotlib axes containing the scatter plot.

See Also

matplotlib.pyplot.scatter : Scatter plot using multiple input data formats.

Examples

Let's see how to draw a scatter plot using coordinates from the values in a DataFrame's columns.

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(
...     [
...         [5.1, 3.5, 0],
...         [4.9, 3.0, 0],
...         [7.0, 3.2, 1],
...         [6.4, 3.2, 1],
...         [5.9, 3.0, 2],
...     ],
...     columns=["length", "width", "species"],
... )
>>> ax1 = df.plot.scatter(x="length", y="width", c="DarkBlue")
```

And now with the color determined by a column as well.

```
.. plot::
:context: close-figs

>>> ax2 = df.plot.scatter(
...     x="length", y="width", c="species", colormap="viridis"
... )
```

```

-----
Methods inherited from pandas.core.base.PandasObject:

__repr__(self) -> str
    Return a string representation for a particular object.

__sizeof__(self) -> int
    Generates the total memory usage for an object that returns
    either a value or Series of values

-----
Data and other attributes inherited from pandas.core.base.PandasObject:

__annotations__ = {}

-----
Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]
    Provide method name lookup and completion.

    Notes
    ----
    Only provide 'public' methods.

-----
Data descriptors inherited from pandas.core.accessor.DirNamesMixin:

__dict__
    dictionary for instance variables

__weakref__
    list of weak references to the object

```

FUNCTIONS

```

andrews_curves(
    frame: DataFrame,
    class_column: str,
    ax: Axes | None = None,
    samples: int = 200,
    color: list[str] | tuple[str, ...] | None = None,
    colormap: Colormap | str | None = None,
    **kwargs
) -> Axes
    Generate a matplotlib plot for visualizing clusters of multivariate data.

```

Andrews curves have the functional form:

```

.. math::
    f(t) = \frac{x_1}{\sqrt{2}} + x_2 \sin(t) + x_3 \cos(t) +
    x_4 \sin(2t) + x_5 \cos(2t) + \cdots

```

Where x_i coefficients correspond to the values of each dimension and t is linearly spaced between $-\pi$ and $+\pi$. Each row of frame then corresponds to a single curve.

Parameters

```

-----
frame : DataFrame
    Data to be plotted, preferably normalized to (0.0, 1.0).
class_column : label
    Name of the column containing class names.
ax : axes object, default None
    Axes to use.
samples : int
    Number of points to plot in each curve.
color : str, list[str] or tuple[str], optional
    Colors to use for the different classes. Colors can be strings
    or 3-element floating point RGB values.
colormap : str or matplotlib colormap object, default None
    Colormap to select colors from. If a string, load colormap with that
    name from matplotlib.
**kwargs
    Options to pass to matplotlib plotting method.

```

Returns

```
-----
:class:`matplotlib.axes.Axes`
    The matplotlib Axes object with the plot.
```

See Also

```
-----
plotting.parallel_coordinates : Plot parallel coordinates chart.
DataFrame.plot : Make plots of Series or DataFrame.
```

Examples

```
-----
.. plot::
    :context: close-figs

    >>> df = pd.read_csv(
    ...     "https://raw.githubusercontent.com/pandas-dev/"
    ...     "pandas/main/pandas/tests/io/data/csv/iris.csv"
    ... ) # doctest: +SKIP
    >>> pd.plotting.andrews_curves(df, "Name") # doctest: +SKIP
```

```
autocorrelation_plot(series: Series, ax: Axes | None = None, **kwargs) -> Axes
Autocorrelation plot for time series.
```

This method generates an autocorrelation plot for a given time series, which helps to identify any periodic structure or correlation within the data across various lags. It shows the correlation of a time series with a delayed copy of itself as a function of delay. Autocorrelation plots are useful for checking randomness in a data set. If the data are random, the autocorrelations should be near zero for any and all time-lag separations. If the data are not random, then one or more of the autocorrelations will be significantly non-zero.

Parameters

```
-----
series : Series
    The time series to visualize.
ax : Matplotlib axis object, optional
    The matplotlib axis object to use.
**kwargs
    Options to pass to matplotlib plotting method.
```

Returns

```
-----
matplotlib.axes.Axes
    The matplotlib axes containing the autocorrelation plot.
```

See Also

```
-----
Series.autocorr : Compute the lag-N autocorrelation for a Series.
plotting.lag_plot : Lag plot for time series.
```

Examples

```
-----
The horizontal lines in the plot correspond to 95% and 99% confidence bands.
```

The dashed line is 99% confidence band.

```
.. plot::
    :context: close-figs

    >>> spacing = np.linspace(-9 * np.pi, 9 * np.pi, num=1000)
    >>> s = pd.Series(0.7 * np.random.rand(1000) + 0.3 * np.sin(spacing))
    >>> pd.plotting.autocorrelation_plot(s) # doctest: +SKIP
```

```
bootstrap_plot(
    series: Series,
    fig: Figure | None = None,
    size: int = 50,
    samples: int = 500,
    **kwargs
) -> Figure
Bootstrap plot on mean, median and mid-range statistics.
```

The bootstrap plot is used to estimate the uncertainty of a statistic by relying on random sampling with replacement [1]_. This function will generate bootstrapping plots for mean, median and mid-range statistics

for the given number of samples of the given size.

```
.. [1] "Bootstrapping (statistics)" in      https://en.wikipedia.org/wiki/Bootstrapping\_%28statistics%29
```

Parameters

```
-----
series : pandas.Series
    Series from where to get the samplings for the bootstrapping.
fig : matplotlib.figure.Figure, default None
    If given, it will use the `fig` reference for plotting instead of
    creating a new one with default parameters.
size : int, default 50
    Number of data points to consider during each sampling. It must be
    less than or equal to the length of the `series`.
samples : int, default 500
    Number of times the bootstrap procedure is performed.
**kwargs
    Options to pass to matplotlib plotting method.
```

Returns

```
-----
matplotlib.figure.Figure
    Matplotlib figure.
```

See Also

```
-----
DataFrame.plot : Basic plotting for DataFrame objects.
Series.plot : Basic plotting for Series objects.
```

Examples

```
-----
This example draws a basic bootstrap plot for a Series.
```

```
.. plot::
    :context: close-figs

    >>> s = pd.Series(np.random.uniform(size=100))
    >>> pd.plotting.bootstrap_plot(s) # doctest: +SKIP
    <Figure size 640x480 with 6 Axes>
```

```
boxplot(
    data: DataFrame,
    column: str | list[str] | None = None,
    by: str | list[str] | None = None,
    ax: Axes | None = None,
    fontsize: float | str | None = None,
    rot: int = 0,
    grid: bool = True,
    figsize: tuple[float, float] | None = None,
    layout: tuple[int, int] | None = None,
    return_type: str | None = None,
    **kwargs
)
```

Make a box plot from DataFrame columns.

Make a box-and-whisker plot from DataFrame columns, optionally grouped by some other columns. A box plot is a method for graphically depicting groups of numerical data through their quartiles. The box extends from the Q1 to Q3 quartile values of the data, with a line at the median (Q2). The whiskers extend from the edges of box to show the range of the data. By default, they extend no more than $1.5 * IQR$ ($IQR = Q3 - Q1$) from the edges of the box, ending at the farthest data point within that interval. Outliers are plotted as separate dots.

For further details see Wikipedia's entry for `boxplot` <https://en.wikipedia.org/wiki/Box_plot>`_.

Parameters

```
-----
data : DataFrame
    The data to visualize.
column : str or list of str, optional
    Column name or list of names, or vector.
    Can be any valid input to :meth:`pandas.DataFrame.groupby`.
by : str or array-like, optional
    Column in the DataFrame to :meth:`pandas.DataFrame.groupby`.
    One box-plot will be done per value of columns in `by`.
```

```

ax : object of class matplotlib.axes.Axes, optional
    The matplotlib axes to be used by boxplot.
fontsize : float or str
    Tick label font size in points or as a string (e.g., `large`).
rot : float, default 0
    The rotation angle of labels (in degrees)
    with respect to the screen coordinate system.
grid : bool, default True
    Setting this to True will show the grid.
figsize : A tuple (width, height) in inches
    The size of the figure to create in matplotlib.
layout : tuple (rows, columns), optional
    For example, (3, 5) will display the subplots
    using 3 rows and 5 columns, starting from the top-left.
return_type : {'axes', 'dict', 'both'} or None, default 'axes'
    The kind of object to return. The default is ``axes``.

    * 'axes' returns the matplotlib axes the boxplot is drawn on.
    * 'dict' returns a dictionary whose values are the matplotlib
      lines of the boxplot.
    * 'both' returns a namedtuple with the axes and dict.
    * when grouping with ``by``, a Series mapping columns to
      ``return_type`` is returned.

    If ``return_type`` is `None`, a NumPy array
    of axes with the same shape as ``layout`` is returned.

**kwargs
    All other plotting keyword arguments to be passed to
    :func:`matplotlib.pyplot.boxplot`.

Returns
-----
result
    See Notes.

See Also
-----
Series.plot.hist: Make a histogram.
matplotlib.pyplot.boxplot : Matplotlib equivalent plot.

Notes
-----
The return type depends on the `return_type` parameter:

* 'axes' : object of class matplotlib.axes.Axes
* 'dict' : dict of matplotlib.lines.Line2D objects
* 'both' : a namedtuple with structure (ax, lines)

For data grouped with ``by``, return a Series of the above or a numpy
array:

* :class:`~pandas.Series`
* :class:`~numpy.array` (for ``return_type = None``)

Use ``return_type='dict'`` when you want to tweak the appearance
of the lines after plotting. In this case a dict containing the Lines
making up the boxes, caps, fliers, medians, and whiskers is returned.

Examples
-----

Boxplots can be created for every column in the dataframe
by ``df.boxplot()`` or indicating the columns to be used:

.. plot::
   :context: close-figs

   >>> np.random.seed(1234)
   >>> df = pd.DataFrame(
   ...     np.random.randn(10, 4), columns=["Col1", "Col2", "Col3", "Col4"]
   ... )
   >>> boxplot = df.boxplot(column=["Col1", "Col2", "Col3"]) # doctest: +SKIP

Boxplots of variables distributions grouped by the values of a third
variable can be created using the option ``by``. For instance:
.. plot::

```

```
:context: close-figs
```

```
>>> df = pd.DataFrame(np.random.randn(10, 2), columns=["Col1", "Col2"])
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])
>>> boxplot = df.boxplot(by="X")
```

A list of strings (i.e. ``['X', 'Y']``) can be passed to boxplot in order to group the data by combination of the variables in the x-axis:

```
.. plot::
```

```
:context: close-figs
```

```
>>> df = pd.DataFrame(np.random.randn(10, 3), columns=["Col1", "Col2", "Col3"])
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])
>>> df["Y"] = pd.Series(["A", "B", "A", "B", "A", "B", "A", "B", "A", "B"])
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by=["X", "Y"])
```

The layout of boxplot can be adjusted giving a tuple to ``layout``:

```
.. plot::
```

```
:context: close-figs
```

```
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", layout=(2, 1))
```

Additional formatting can be done to the boxplot, like suppressing the grid (``grid=False``), rotating the labels in the x-axis (i.e. ``rot=45``) or changing the fontsize (i.e. ``fontsize=15``):

```
.. plot::
```

```
:context: close-figs
```

```
>>> boxplot = df.boxplot(grid=False, rot=45, fontsize=15) # doctest: +SKIP
```

The parameter ``return\_type`` can be used to select the type of element returned by ``boxplot``. When ``return\_type='axes'`` is selected, the matplotlib axes on which the boxplot is drawn are returned:

```
>>> boxplot = df.boxplot(column=["Col1", "Col2"], return_type="axes")
>>> type(boxplot)
<class 'matplotlib.axes._axes.Axes'>
```

When grouping with ``by``, a Series mapping columns to ``return\_type`` is returned:

```
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type="axes")
>>> type(boxplot)
<class 'pandas.Series'>
```

If ``return\_type`` is ``None``, a NumPy array of axes with the same shape as ``layout`` is returned:

```
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type=None)
>>> type(boxplot)
<class 'numpy.ndarray'>
```

```
boxplot_frame(
    self: DataFrame,
    column=None,
    by=None,
    ax=None,
    fontsize: int | None = None,
    rot: int = 0,
    grid: bool = True,
    figsize: tuple[float, float] | None = None,
    layout=None,
    return_type=None,
    backend=None,
    **kwargs
)
    Make a box plot from DataFrame columns.
```

Make a box-and-whisker plot from DataFrame columns, optionally grouped by some other columns. A box plot is a method for graphically depicting groups of numerical data through their quartiles. The box extends from the Q1 to Q3 quartile values of the data, with a line at the median (Q2). The whiskers extend from the edges of box to show the range of the data. By default, they extend no more than

$1.5 * IQR$ ($IQR = Q3 - Q1$)` from the edges of the box, ending at the farthest data point within that interval. Outliers are plotted as separate dots.

For further details see

Wikipedia's entry for `boxplot` <https://en.wikipedia.org/wiki/Box_plot>`_.

Parameters

column : str or list of str, optional
 Column name or list of names, or vector.
 Can be any valid input to :meth:`pandas.DataFrame.groupby`.
by : str or array-like, optional
 Column in the DataFrame to :meth:`pandas.DataFrame.groupby`.
 One box-plot will be done per value of columns in `by`.
ax : object of class matplotlib.axes.Axes, optional
 The matplotlib axes to be used by boxplot.
fontsize : float or str
 Tick label font size in points or as a string (e.g., `large`).
rot : float, default 0
 The rotation angle of labels (in degrees)
 with respect to the screen coordinate system.
grid : bool, default True
 Setting this to True will show the grid.
figsize : A tuple (width, height) in inches
 The size of the figure to create in matplotlib.
layout : tuple (rows, columns), optional
 For example, (3, 5) will display the subplots
 using 3 rows and 5 columns, starting from the top-left.
return_type : {'axes', 'dict', 'both'} or None, default 'axes'
 The kind of object to return. The default is ``axes``.

 * 'axes' returns the matplotlib axes the boxplot is drawn on.
 * 'dict' returns a dictionary whose values are the matplotlib
 lines of the boxplot.
 * 'both' returns a namedtuple with the axes and dict.
 * when grouping with ``by``, a Series mapping columns to
 ``return\_type`` is returned.

 If ``return\_type`` is `None`, a NumPy array
 of axes with the same shape as ``layout`` is returned.
backend : str, default None
 Backend to use instead of the backend specified in the option
 `plotting.backend`. For instance, 'matplotlib'. Alternatively, to
 specify the `plotting.backend` for the whole session, set
 `pd.options.plotting.backend`.

**kwargs
 All other plotting keyword arguments to be passed to
 :func:`matplotlib.pyplot.boxplot`.

Returns

result
 See Notes.

See Also

Series.plot.hist: Make a histogram.
matplotlib.pyplot.boxplot : Matplotlib equivalent plot.

Notes

The return type depends on the `return\_type` parameter:

- * 'axes' : object of class matplotlib.axes.Axes
- * 'dict' : dict of matplotlib.lines.Line2D objects
- * 'both' : a namedtuple with structure (ax, lines)

For data grouped with ``by``, return a Series of the above or a numpy array:

- * :class:`~pandas.Series`
- * :class:`~numpy.array` (for ``return\_type = None``)

Use ``return\_type='dict'`` when you want to tweak the appearance of the lines after plotting. In this case a dict containing the Lines making up the boxes, caps, fliers, medians, and whiskers is returned.

Examples

Boxplots can be created for every column in the dataframe by `df.boxplot()` or indicating the columns to be used:

```
.. plot::
    :context: close-figs

    >>> np.random.seed(1234)
    >>> df = pd.DataFrame(
    ...     np.random.randn(10, 4), columns=["Col1", "Col2", "Col3", "Col4"]
    ... )
    >>> boxplot = df.boxplot(column=["Col1", "Col2", "Col3"]) # doctest: +SKIP
```

Boxplots of variables distributions grouped by the values of a third variable can be created using the option `by`. For instance:

```
.. plot::
    :context: close-figs

    >>> df = pd.DataFrame(np.random.randn(10, 2), columns=["Col1", "Col2"])
    >>> df["X"] = pd.Series(["A", "A", "A", "A", "B", "B", "B", "B", "B"])
    >>> boxplot = df.boxplot(by="X")
```

A list of strings (i.e. `['X', 'Y']`) can be passed to `boxplot` in order to group the data by combination of the variables in the x-axis:

```
.. plot::
    :context: close-figs

    >>> df = pd.DataFrame(np.random.randn(10, 3), columns=["Col1", "Col2", "Col3"])
    >>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])
    >>> df["Y"] = pd.Series(["A", "B", "A", "B", "A", "B", "A", "B", "A", "B"])
    >>> boxplot = df.boxplot(column=["Col1", "Col2"], by=["X", "Y"])
```

The layout of boxplot can be adjusted giving a tuple to `layout`:

```
.. plot::
    :context: close-figs

    >>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", layout=(2, 1))
```

Additional formatting can be done to the boxplot, like suppressing the grid (`grid=False`), rotating the labels in the x-axis (i.e. `rot=45`) or changing the fontsize (i.e. `fontsize=15`):

```
.. plot::
    :context: close-figs

    >>> boxplot = df.boxplot(grid=False, rot=45, fontsize=15) # doctest: +SKIP
```

The parameter `return_type` can be used to select the type of element returned by `boxplot`. When `return_type='axes'` is selected, the matplotlib axes on which the boxplot is drawn are returned:

```
.. plot::
    :context: close-figs

    >>> boxplot = df.boxplot(column=["Col1", "Col2"], return_type="axes")
    >>> type(boxplot)
    <class 'matplotlib.axes._axes.Axes'>
```

When grouping with `by`, a Series mapping columns to `return_type` is returned:

```
.. plot::
    :context: close-figs

    >>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type="axes")
    >>> type(boxplot)
    <class 'pandas.Series'>
```

If `return_type` is `None`, a NumPy array of axes with the same shape as `layout` is returned:

```
.. plot::
```

```

:context: close-figs

>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type=None)
>>> type(boxplot)
<class 'numpy.ndarray'>

boxplot_frame_groupby(
    grouped: DataFrameGroupBy,
    subplots: bool = True,
    column=None,
    fontsize: int | None = None,
    rot: int = 0,
    grid: bool = True,
    ax=None,
    figsize: tuple[float, float] | None = None,
    layout=None,
    sharex: bool = False,
    sharey: bool = True,
    backend=None,
    **kwargs
)

```

Make box plots from DataFrameGroupBy data.

Parameters

```

-----
grouped : DataFrameGroupBy
    The grouped DataFrame object over which to create the box plots.
subplots : bool
    * ``False`` - no subplots will be used
    * ``True`` - create a subplot for each group.
column : column name or list of names, or vector
    Can be any valid input to groupby.
fontsize : float or str
    Font size for the labels.
rot : float
    Rotation angle of labels (in degrees) on the x-axis.
grid : bool
    Whether to show grid lines on the plot.
ax : Matplotlib axis object, default None
    The axes on which to draw the plots. If None, uses the current axes.
figsize : tuple of (float, float)
    The figure size in inches (width, height).
layout : tuple (optional)
    The layout of the plot: (rows, columns).
sharex : bool, default False
    Whether x-axes will be shared among subplots.
sharey : bool, default True
    Whether y-axes will be shared among subplots.
backend : str, default None
    Backend to use instead of the backend specified in the option
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
    specify the ``plotting.backend`` for the whole session, set
    ``pd.options.plotting.backend``.
**kwargs
    All other plotting keyword arguments to be passed to
    matplotlib's boxplot function.

```

Returns

```

-----
dict or DataFrame.boxplot return value
    If ``subplots=True``, returns a dictionary of group keys to the boxplot
    return values. If ``subplots=False``, returns the boxplot return value
    of a single DataFrame.

```

See Also

```

-----
DataFrame.boxplot : Create a box plot from a DataFrame.
Series.plot : Plot a Series.

```

Examples

```

-----
You can create boxplots for grouped data and show them as separate subplots:

```

```

.. plot:
    :context: close-figs

    >>> import itertools

```

```
>>> tuples = [t for t in itertools.product(range(1000), range(4))]
>>> index = pd.MultiIndex.from_tuples(tuples, names=["lvl0", "lvl1"])
>>> data = np.random.randn(len(index), 4)
>>> df = pd.DataFrame(data, columns=list("ABCD"), index=index)
>>> grouped = df.groupby(level="lvl1")
>>> grouped.boxplot(rot=45, fontsize=12, figsize=(8, 10)) # doctest: +SKIP
```

The ``subplots=False`` option shows the boxplots in a single figure.

```
.. plot::
   :context: close-figs
```

```
>>> grouped.boxplot(subplots=False, rot=45, fontsize=12) # doctest: +SKIP
```

```
deregister_matplotlib_converters = deregister() -> None
Remove pandas formatters and converters.
```

Removes the custom converters added by :func:`register`. This attempts to set the state of the registry back to the state before pandas registered its own units. Converters for pandas' own types like Timestamp and Period are removed completely. Converters for types pandas overwrites, like ``datetime.datetime``, are restored to their original value.

See Also

```
-----
register_matplotlib_converters : Register pandas formatters and converters
                                with matplotlib.
```

Examples

```
-----
.. plot::
   :context: close-figs
```

The following line is done automatically by pandas so the plot can be rendered:

```
>>> pd.plotting.register_matplotlib_converters()

>>> df = pd.DataFrame(
...     {"ts": pd.period_range("2020", periods=2, freq="M"), "y": [1, 2]}
... )
>>> plot = df.plot.line(x="ts", y="y")
```

Unsetting the register manually an error will be raised:

```
>>> pd.set_option(
...     "plotting.matplotlib.register_converters", False
... ) # doctest: +SKIP
>>> df.plot.line(x="ts", y="y") # doctest: +SKIP
Traceback (most recent call last):
TypeError: float() argument must be a string or a real number, not 'Period'
```

```
hist_frame(
    data: DataFrame,
    column: IndexLabel | None = None,
    by=None,
    grid: bool = True,
    xlabelsize: int | None = None,
    xrot: float | None = None,
    ylabelsize: int | None = None,
    yrot: float | None = None,
    ax=None,
    sharex: bool = False,
    sharey: bool = False,
    figsize: tuple[int, int] | None = None,
    layout: tuple[int, int] | None = None,
    bins: int | Sequence[int] = 10,
    backend: str | None = None,
    legend: bool = False,
    **kwargs
)
```

Make a histogram of the DataFrame's columns.

A ``histogram`` is a representation of the distribution of data. This function calls :meth:`matplotlib.pyplot.hist`, on each series in the DataFrame, resulting in one histogram per column.

.. _histogram: <https://en.wikipedia.org/wiki/Histogram>

Parameters

data : DataFrame

The pandas object holding the data.

column : str or sequence, optional

If passed, will be used to limit data to a subset of columns.

by : object, optional

If passed, then used to form histograms for separate groups.

grid : bool, default True

Whether to show axis grid lines.

xlabelsize : int, default None

If specified changes the x-axis label size.

xrot : float, default None

Rotation of x axis labels. For example, a value of 90 displays the x labels rotated 90 degrees clockwise.

ylabelsize : int, default None

If specified changes the y-axis label size.

yrot : float, default None

Rotation of y axis labels. For example, a value of 90 displays the y labels rotated 90 degrees clockwise.

ax : Matplotlib axes object, default None

The axes to plot the histogram on.

sharex : bool, default True if ax is None else False

In case subplots=True, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in.

Note that passing in both an ax and sharex=True will alter all x axis labels for all subplots in a figure.

sharey : bool, default False

In case subplots=True, share y axis and set some y axis labels to invisible.

figsize : tuple, optional

The size in inches of the figure to create. Uses the value in `matplotlib.rcParams` by default.

layout : tuple, optional

Tuple of (rows, columns) for the layout of the histograms.

bins : int or sequence, default 10

Number of histogram bins to be used. If an integer is given, bins + 1 bin edges are calculated and returned. If bins is a sequence, gives bin edges, including left edge of first bin and right edge of last bin. In this case, bins is returned unmodified.

backend : str, default None

Backend to use instead of the backend specified in the option ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to specify the ``plotting.backend`` for the whole session, set ``pd.options.plotting.backend``.

legend : bool, default False

Whether to show the legend.

**kwargs

All other plotting keyword arguments to be passed to :meth:`matplotlib.pyplot.hist`.

Returns

np.ndarray

2D NumPy Array of :class:`matplotlib.axes.Axes`.

See Also

matplotlib.pyplot.hist : Plot a histogram using matplotlib.

Examples

This example draws a histogram based on the length and width of some animals, displayed in three bins

.. plot::

:context: close-figs

```
>>> data = {
...     "length": [1.5, 0.5, 1.2, 0.9, 3],
...     "width": [0.7, 0.2, 0.15, 0.2, 1.1],
```



```

... }
>>> index = ["pig", "rabbit", "duck", "chicken", "horse"]
>>> df = pd.DataFrame(data, index=index)
>>> hist = df.hist(bins=3)

hist_series(
    self: Series,
    by=None,
    ax=None,
    grid: bool = True,
    xlabelsize: int | None = None,
    xrot: float | None = None,
    ylabelsize: int | None = None,
    yrot: float | None = None,
    figsize: tuple[int, int] | None = None,
    bins: int | Sequence[int] = 10,
    backend: str | None = None,
    legend: bool = False,
    **kwargs
)

Draw histogram of the input series using matplotlib.

Parameters
-----
by : object, optional
    If passed, then used to form histograms for separate groups.
ax : matplotlib axis object
    If not passed, uses gca().
grid : bool, default True
    Whether to show axis grid lines.
xlabelsize : int, default None
    If specified changes the x-axis label size.
xrot : float, default None
    Rotation of x axis labels.
ylabelsize : int, default None
    If specified changes the y-axis label size.
yrot : float, default None
    Rotation of y axis labels.
figsize : tuple, default None
    Figure size in inches by default.
bins : int or sequence, default 10
    Number of histogram bins to be used. If an integer is given, bins + 1
    bin edges are calculated and returned. If bins is a sequence, gives
    bin edges, including left edge of first bin and right edge of last
    bin. In this case, bins is returned unmodified.
backend : str, default None
    Backend to use instead of the backend specified in the option
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
    specify the ``plotting.backend`` for the whole session, set
    ``pd.options.plotting.backend``.
legend : bool, default False
    Whether to show the legend.

**kwargs
    To be passed to the actual plotting function.

Returns
-----
matplotlib.axes.Axes
    A histogram plot.

See Also
-----
matplotlib.axes.Axes.hist : Plot a histogram using matplotlib.

Examples
-----
For Series:

.. plot::
   :context: close-figs

   >>> lst = ["a", "a", "a", "b", "b", "b"]
   >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
   >>> hist = ser.hist()

For Groupby:

```

```
.. plot::
    :context: close-figs

    >>> lst = ["a", "a", "a", "b", "b", "b"]
    >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
    >>> hist = ser.groupby(level=0).hist()
```

`lag_plot(series: Series, lag: int = 1, ax: Axes | None = None, **kwargs) -> Axes`
 Lag plot for time series.

A lag plot is a scatter plot of a time series against a lag of itself. It helps in visualizing the temporal dependence between observations by plotting the values at time `t` on the x-axis and the values at time `t + lag` on the y-axis.

Parameters

```
-----
series : Series
    The time series to visualize.
lag : int, default 1
    Lag length of the scatter plot.
ax : Matplotlib axis object, optional
    The matplotlib axis object to use.
**kwargs
    Matplotlib scatter method keyword arguments.
```

Returns

```
-----
matplotlib.axes.Axes
    The matplotlib Axes object containing the lag plot.
```

See Also

```
-----
plotting.autocorrelation_plot : Autocorrelation plot for time series.
matplotlib.pyplot.scatter : A scatter plot of y vs. x with varying marker size
    and/or color in Matplotlib.
```

Examples

```
-----
Lag plots are most commonly used to look for patterns in time series data.
```

Given the following time series

```
.. plot::
    :context: close-figs

    >>> np.random.seed(5)
    >>> x = np.cumsum(np.random.normal(loc=1, scale=5, size=50))
    >>> s = pd.Series(x)
    >>> s.plot() # doctest: +SKIP
```

A lag plot with `lag=1` returns

```
.. plot::
    :context: close-figs

    >>> _ = pd.plotting.lag_plot(s, lag=1)
```

```
parallel_coordinates(
    frame: DataFrame,
    class_column: str,
    cols: list[str] | None = None,
    ax: Axes | None = None,
    color: list[str] | tuple[str, ...] | None = None,
    use_columns: bool = False,
    xticks: list | tuple | None = None,
    colormap: Colormap | str | None = None,
    axvlines: bool = True,
    axvlines_kws: Mapping[str, Any] | None = None,
    sort_labels: bool = False,
    **kwargs
) -> Axes
Parallel coordinates plotting.
```

Parameters

```
-----
frame : DataFrame
    The DataFrame to be plotted.
```

```

class_column : str
    Column name containing class names.
cols : list, optional
    A list of column names to use.
ax : matplotlib.axis, optional
    Matplotlib axis object.
color : list or tuple, optional
    Colors to use for the different classes.
use_columns : bool, optional
    If true, columns will be used as xticks.
xticks : list or tuple, optional
    A list of values to use for xticks.
colormap : str or matplotlib colormap, default None
    Colormap to use for line colors.
axvlines : bool, optional
    If true, vertical lines will be added at each xtick.
axvlines_kwds : keywords, optional
    Options to be passed to axvline method for vertical lines.
sort_labels : bool, default False
    Sort class_column labels, useful when assigning colors.
**kwargs
    Options to pass to matplotlib plotting method.

```

Returns

```

-----
matplotlib.axes.Axes
    The matplotlib axes containing the parallel coordinates plot.

```

See Also

```

-----
plotting.andrews_curves : Generate a matplotlib plot for visualizing clusters
    of multivariate data.
plotting.radviz : Plot a multidimensional dataset in 2D.

```

Examples

```

-----
.. plot::
    :context: close-figs

    >>> df = pd.read_csv(
    ...     "https://raw.githubusercontent.com/pandas-dev/"
    ...     "pandas/main/pandas/tests/io/data/csv/iris.csv"
    ... ) # doctest: +SKIP
    >>> pd.plotting.parallel_coordinates(
    ...     df, "Name", color=("#556270", "#4ECDC4", "#C7F464")
    ... ) # doctest: +SKIP

```

```

radviz(
    frame: DataFrame,
    class_column: str,
    ax: Axes | None = None,
    color: list[str] | tuple[str, ...] | None = None,
    colormap: Colormap | str | None = None,
    **kwds
) -> Axes
    Plot a multidimensional dataset in 2D.

```

Each Series in the DataFrame is represented as an evenly distributed slice on a circle. Each data point is rendered in the circle according to the value on each Series. Highly correlated `Series` in the `DataFrame` are placed closer on the unit circle.

RadViz allow to project an N-dimensional data set into a 2D space where the influence of each dimension can be interpreted as a balance between the influence of all dimensions.

More info available at the `original article`
<https://doi.org/10.1145/331770.331775> describing RadViz.

Parameters

```

-----
frame : `DataFrame`
    Object holding the data.
class_column : str
    Column name containing the name of the data point category.

```

```

ax : :class:`matplotlib.axes.Axes`, optional
    A plot instance to which to add the information.
color : list[str] or tuple[str], optional
    Assign a color to each category. Example: ['blue', 'green'].
colormap : str or :class:`matplotlib.colors.Colormap`, default None
    Colormap to select colors from. If string, load colormap with that
    name from matplotlib.
**kwds
    Options to pass to matplotlib scatter plotting method.

```

Returns

```

-----
:class:`matplotlib.axes.Axes`
    The Axes object from Matplotlib.

```

See Also

```

-----
plotting.andrews_curves : Plot clustering visualization.

```

Examples

```

-----
.. plot::
   :context: close-figs

   >>> df = pd.DataFrame(
   ...     {
   ...         "SepalLength": [6.5, 7.7, 5.1, 5.8, 7.6, 5.0, 5.4, 4.6, 6.7, 4.6],
   ...         "SepalWidth": [3.0, 3.8, 3.8, 2.7, 3.0, 2.3, 3.0, 3.2, 3.3, 3.6],
   ...         "PetalLength": [5.5, 6.7, 1.9, 5.1, 6.6, 3.3, 4.5, 1.4, 5.7, 1.0],
   ...         "PetalWidth": [1.8, 2.2, 0.4, 1.9, 2.1, 1.0, 1.5, 0.2, 2.1, 0.2],
   ...         "Category": [
   ...             "virginica",
   ...             "virginica",
   ...             "setosa",
   ...             "virginica",
   ...             "virginica",
   ...             "versicolor",
   ...             "versicolor",
   ...             "setosa",
   ...             "virginica",
   ...             "setosa",
   ...         ],
   ...     }
   ... )
   >>> pd.plotting.radviz(df, "Category") # doctest: +SKIP

```

```

register_matplotlib_converters = register() -> None
Register pandas formatters and converters with matplotlib.

```

This function modifies the global ``matplotlib.units.registry`` dictionary. pandas adds custom converters for

```

* pd.Timestamp
* pd.Period
* np.datetime64
* datetime.datetime
* datetime.date
* datetime.time

```

See Also

```

-----
deregister_matplotlib_converters : Remove pandas formatters and converters.

```

Examples

```

-----
.. plot::
   :context: close-figs

   The following line is done automatically by pandas so
   the plot can be rendered:

   >>> pd.plotting.register_matplotlib_converters()

   >>> df = pd.DataFrame(
   ...     {"ts": pd.period_range("2020", periods=2, freq="M"), "y": [1, 2]}
   ... )

```

```
>>> plot = df.plot.line(x="ts", y="y")
```

Unsetting the register manually an error will be raised:

```
>>> pd.set_option(
...     "plotting.matplotlib.register_converters", False
... ) # doctest: +SKIP
>>> df.plot.line(x="ts", y="y") # doctest: +SKIP
Traceback (most recent call last):
TypeError: float() argument must be a string or a real number, not 'Period'
```

```
scatter_matrix(
    frame: DataFrame,
    alpha: float = 0.5,
    figsize: tuple[float, float] | None = None,
    ax: Axes | None = None,
    grid: bool = False,
    diagonal: str = 'hist',
    marker: str = '.',
    density_kwds: Mapping[str, Any] | None = None,
    hist_kwds: Mapping[str, Any] | None = None,
    range_padding: float = 0.05,
    **kwargs
) -> np.ndarray
Draw a matrix of scatter plots.
```

Each pair of numeric columns in the DataFrame is plotted against each other, resulting in a matrix of scatter plots. The diagonal plots can display either histograms or Kernel Density Estimation (KDE) plots for each variable.

Parameters

```
-----
frame : DataFrame
    The data to be plotted.
alpha : float, optional
    Amount of transparency applied.
figsize : (float,float), optional
    A tuple (width, height) in inches.
ax : Matplotlib axis object, optional
    An existing Matplotlib axis object for the plots. If None, a new axis is
    created.
grid : bool, optional
    Setting this to True will show the grid.
diagonal : {'hist', 'kde'}
    Pick between 'kde' and 'hist' for either Kernel Density Estimation or
    Histogram plot in the diagonal.
marker : str, optional
    Matplotlib marker type, default '.'.
density_kwds : keywords
    Keyword arguments to be passed to kernel density estimate plot.
hist_kwds : keywords
    Keyword arguments to be passed to hist function.
range_padding : float, default 0.05
    Relative extension of axis range in x and y with respect to
    (x_max - x_min) or (y_max - y_min).
**kwargs
    Keyword arguments to be passed to scatter function.
```

Returns

```
-----
numpy.ndarray
    A matrix of scatter plots.
```

See Also

```
-----
plotting.parallel_coordinates : Plots parallel coordinates for multivariate data.
plotting.andrews_curves : Generates Andrews curves for visualizing clusters of
    multivariate data.
plotting.radviz : Creates a RadViz visualization.
plotting.bootstrap_plot : Visualizes uncertainty in data via bootstrap sampling.
```

Examples

```
-----
.. plot::
    :context: close-figs
    >>> df = pd.DataFrame(np.random.randn(1000, 4), columns=["A", "B", "C", "D"])
```

```
>>> pd.plotting.scatter_matrix(df, alpha=0.2)
array([[<Axes: xlabel='A', ylabel='A'>, <Axes: xlabel='B', ylabel='A'>,
       <Axes: xlabel='C', ylabel='A'>, <Axes: xlabel='D', ylabel='A'>],
       [<Axes: xlabel='A', ylabel='B'>, <Axes: xlabel='B', ylabel='B'>,
       <Axes: xlabel='C', ylabel='B'>, <Axes: xlabel='D', ylabel='B'>],
       [<Axes: xlabel='A', ylabel='C'>, <Axes: xlabel='B', ylabel='C'>,
       <Axes: xlabel='C', ylabel='C'>, <Axes: xlabel='D', ylabel='C'>],
       [<Axes: xlabel='A', ylabel='D'>, <Axes: xlabel='B', ylabel='D'>,
       <Axes: xlabel='C', ylabel='D'>, <Axes: xlabel='D', ylabel='D'>]],
      dtype=object)
```

```
table(ax: Axes, data: DataFrame | Series, **kwargs) -> Table
    Helper function to convert DataFrame and Series to matplotlib.table.
```

This method provides an easy way to visualize tabular data within a Matplotlib figure. It automatically extracts index and column labels from the DataFrame or Series, unless explicitly specified. This function is particularly useful when displaying summary tables alongside other plots or when creating static reports. It utilizes the `matplotlib.pyplot.table` backend and allows customization through various styling options available in Matplotlib.

Parameters

```
-----
ax : Matplotlib axes object
    The axes on which to draw the table.
data : DataFrame or Series
    Data for table contents.
**kwargs
    Keyword arguments to be passed to matplotlib.table.table.
    If 'rowLabels' or 'colLabels' is not specified, data index or column
    names will be used.
```

Returns

```
-----
matplotlib.table object
    The created table as a matplotlib Table object.
```

See Also

```
-----
DataFrame.plot : Make plots of DataFrame using matplotlib.
matplotlib.pyplot.table : Create a table from data in a Matplotlib plot.
```

Examples

```
-----
.. plot::
    :context: close-figs

    >>> import matplotlib.pyplot as plt
    >>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
    >>> fig, ax = plt.subplots()
    >>> ax.axis("off")
    (np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
    >>> table = pd.plotting.table(
    ...     ax, df, loc="center", cellLoc="center", colWidths=[0.2, 0.2]
    ... )
```

DATA

```
__all__ = ['PlotAccessor', 'andrews_curves', 'autocorrelation_plot', '...
plot_params = {'xaxis.compat': False}
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\plotting\__init__
```

Help on module pandas.testing in pandas:

NAME

```
pandas.testing - Public testing utility functions.
```

FUNCTIONS

```
assert_extension_array_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = True,
    index_values=None,
    check_exact: bool | lib.NoDefault = <no_default>,
```

```

    rtol: float | lib.NoDefault = <no_default>,
    atol: float | lib.NoDefault = <no_default>,
    obj: str = 'ExtensionArray'
) -> None
    Check that left and right ExtensionArrays are equal.

This method compares two ``ExtensionArray`` instances for equality,
including checks for missing values, the dtype of the arrays, and
the exactness of the comparison (or tolerance when comparing floats).

Parameters
-----
left, right : ExtensionArray
    The two arrays to compare.
check_dtype : bool, default True
    Whether to check if the ExtensionArray dtypes are identical.
index_values : Index | numpy.ndarray, default None
    Optional index (shared by both left and right), used in output.
check_exact : bool, default False
    Whether to compare number exactly.

.. versionchanged:: 2.2.0

    Defaults to True for integer dtypes if none of
    ``check_exact``, ``rtol`` and ``atol`` are specified.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'ExtensionArray'
    Specify object name being compared, internally used to show appropriate
    assertion message.

.. versionadded:: 2.0.0

See Also
-----
testing.assert_series_equal : Check that left and right ``Series`` are equal.
testing.assert_frame_equal : Check that left and right ``DataFrame`` are equal.
testing.assert_index_equal : Check that left and right ``Index`` are equal.

Notes
-----
Missing values are checked separately from valid values.
A mask of missing values is computed for each and checked to match.
The remaining all-valid values are cast to object dtype and checked.

Examples
-----
>>> from pandas import testing as tm
>>> a = pd.Series([1, 2, 3, 4])
>>> b, c = a.array, a.array
>>> tm.assert_extension_array_equal(b, c)

assert_frame_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = True,
    check_index_type: bool | Literal['equiv'] = 'equiv',
    check_column_type: bool | Literal['equiv'] = 'equiv',
    check_frame_type: bool = True,
    check_names: bool = True,
    by_blocks: bool = False,
    check_exact: bool | lib.NoDefault = <no_default>,
    check_datetimelike_compat: bool = False,
    check_categorical: bool = True,
    check_like: bool = False,
    check_freq: bool = True,
    check_flags: bool = True,
    rtol: float | lib.NoDefault = <no_default>,
    atol: float | lib.NoDefault = <no_default>,
    obj: str = 'DataFrame'
) -> None
    Check that left and right DataFrame are equal.

This function is intended to compare two DataFrames and output any
differences. It is mostly intended for use in unit tests.

```

Additional parameters allow varying the strictness of the equality checks performed.

Parameters

`left` : DataFrame
First DataFrame to compare.

`right` : DataFrame
Second DataFrame to compare.

`check_dtype` : bool, default True
Whether to check the DataFrame dtype is identical.

`check_index_type` : bool or {'equiv'}, default 'equiv'
Whether to check the Index class, dtype and inferred_type are identical.

`check_column_type` : bool or {'equiv'}, default 'equiv'
Whether to check the columns class, dtype and inferred_type are identical. Is passed as the ``exact`` argument of :func:`assert\_index\_equal`.

`check_frame_type` : bool, default True
Whether to check the DataFrame class is identical.

`check_names` : bool, default True
Whether to check that the `names` attribute for both the `index` and `column` attributes of the DataFrame is identical.

`by_blocks` : bool, default False
Specify how to compare internal data. If False, compare by columns. If True, compare by blocks.

`check_exact` : bool, default False
Whether to compare number exactly. If False, the comparison uses the relative tolerance (``rtol``) and absolute tolerance (``atol``) parameters to determine if two values are considered close, according to the formula: $|a - b| \leq (atol + rtol * |b|)$.

.. versionchanged:: 2.2.0

Defaults to True for integer dtypes if none of ``check\_exact``, ``rtol`` and ``atol`` are specified.

`check_datetimelike_compat` : bool, default False
Compare datetime-like which is comparable ignoring dtype.

.. deprecated:: 3.0

`check_categorical` : bool, default True
Whether to compare internal Categorical exactly.

`check_like` : bool, default False
If True, ignore the order of index & columns.
Note: index labels must match their respective rows (same as in columns) - same labels must be with the same data.

`check_freq` : bool, default True
Whether to check the `freq` attribute on a DatetimeIndex or TimedeltaIndex.

`check_flags` : bool, default True
Whether to check the `flags` attribute.

`rtol` : float, default 1e-5
Relative tolerance. Only used when check_exact is False.

`atol` : float, default 1e-8
Absolute tolerance. Only used when check_exact is False.

`obj` : str, default 'DataFrame'
Specify object name being compared, internally used to show appropriate assertion message.

See Also

`assert_series_equal` : Equivalent method for asserting Series equality.
`DataFrame.equals` : Check DataFrame equality.

Examples

This example shows comparing two DataFrames that are equal but with columns of differing dtypes.

```
>>> from pandas.testing import assert_frame_equal
>>> df1 = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
>>> df2 = pd.DataFrame({"a": [1, 2], "b": [3.0, 4.0]})
```

```
df1.equals(df2)
```

```
>>> assert_frame_equal(df1, df2)
df1 differs from df2 as column 'b' is of a different type.
```



```

>>> assert_frame_equal(df1, df2)
Traceback (most recent call last):
...
AssertionError: Attributes of DataFrame.iloc[:, 1] (column name="b") are different

Attribute "dtype" are different
[left]: int64
[right]: float64

Ignore differing dtypes in columns with check_dtype.

>>> assert_frame_equal(df1, df2, check_dtype=False)

assert_index_equal(
    left: Index,
    right: Index,
    exact: bool | str = 'equiv',
    check_names: bool = True,
    check_exact: bool = True,
    check_categorical: bool = True,
    check_order: bool = True,
    rtol: float = 1e-05,
    atol: float = 1e-08,
    obj: str | None = None
) -> None
Check that left and right Index are equal.

Parameters
-----
left : Index
    The first index to compare.
right : Index
    The second index to compare.
exact : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical. If 'equiv', then RangeIndex can be substituted for
    Index with an int64 dtype as well.
check_names : bool, default True
    Whether to check the names attribute.
check_exact : bool, default True
    Whether to compare number exactly.
check_categorical : bool, default True
    Whether to compare internal Categorical exactly.
check_order : bool, default True
    Whether to compare the order of index entries as well as their values.
    If True, both indexes must contain the same elements, in the same order.
    If False, both indexes must contain the same elements, but in any order.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'Index' or 'MultiIndex'
    Specify object name being compared, internally used to show appropriate
    assertion message.

See Also
-----
testing.assert_series_equal : Check that two Series are equal.
testing.assert_frame_equal : Check that two DataFrames are equal.

Examples
-----
>>> from pandas import testing as tm
>>> a = pd.Index([1, 2, 3])
>>> b = pd.Index([1, 2, 3])
>>> tm.assert_index_equal(a, b)

assert_series_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = True,
    check_index_type: bool | Literal['equiv'] = 'equiv',
    check_series_type: bool = True,
    check_names: bool = True,
    check_exact: bool | lib.NoDefault = <no_default>,
    check_datetimelike_compat: bool = False,
    check_categorical: bool = True,

```

```

    check_category_order: bool = True,
    check_freq: bool = True,
    check_flags: bool = True,
    rtol: float | lib.NoDefault = <no_default>,
    atol: float | lib.NoDefault = <no_default>,
    obj: str = 'Series',
    *,
    check_index: bool = True,
    check_like: bool = False
) -> None
    Check that left and right Series are equal.

Parameters
-----
left : Series
    First Series to compare.
right : Series
    Second Series to compare.
check_dtype : bool, default True
    Whether to check the Series dtype is identical.
check_index_type : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical.
check_series_type : bool, default True
    Whether to check the Series class is identical.
check_names : bool, default True
    Whether to check the Series and Index names attribute.
check_exact : bool, default False
    Whether to compare number exactly. This also applies when checking
    Index equivalence.

.. versionchanged:: 2.2.0

    Defaults to True for integer dtypes if none of
    ``check_exact``, ``rtol`` and ``atol`` are specified.

.. versionchanged:: 3.0.0

    check_exact for comparing the Indexes defaults to True by
    checking if an Index is of integer dtypes.

check_datetimelike_compat : bool, default False
    Compare datetime-like which is comparable ignoring dtype.

.. deprecated:: 3.0

check_categorical : bool, default True
    Whether to compare internal Categorical exactly.
check_category_order : bool, default True
    Whether to compare category order of internal Categoricals.
check_freq : bool, default True
    Whether to check the `freq` attribute on a DatetimeIndex or TimedeltaIndex.
check_flags : bool, default True
    Whether to check the `flags` attribute.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'Series'
    Specify object name being compared, internally used to show appropriate
    assertion message.
check_index : bool, default True
    Whether to check index equivalence. If False, then compare only values.
check_like : bool, default False
    If True, ignore the order of the index. Must be False if check_index is False.
    Note: same labels must be with the same data.

See Also
-----
testing.assert_index_equal : Check that two Indexes are equal.
testing.assert_frame_equal : Check that two DataFrames are equal.

Examples
-----
>>> from pandas import testing as tm
>>> a = pd.Series([1, 2, 3, 4])
>>> b = pd.Series([1, 2, 3, 4])

```

```

>>> tm.assert_series_equal(a, b)

DATA
__all__ = ['assert_extension_array_equal', 'assert_frame_equal', 'asse...

FILE
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\testing.py

Help on package pandas.tseries in pandas:

NAME
pandas.tseries - # ruff: noqa: TC004

PACKAGE CONTENTS
api
frequencies
holiday
offsets

DATA
TYPE_CHECKING = False

FILE
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\tseries\__init__

Help on package pandas.util in pandas:

NAME
pandas.util

PACKAGE CONTENTS
_decorators
_doctools
_exceptions
_print_versions
_test_decorators
_tester
_validators
version (package)

FUNCTIONS
__dir__() -> list[str]

__getattr__(key: str)

FILE
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\util\__init__.py

Help on package pandas._config in pandas:

NAME
pandas._config

DESCRIPTION
pandas._config is considered explicitly upstream of everything else in pandas,
should have no intra-pandas dependencies.

importing `dates` and `display` ensures that keys needed by _libs
are initialized.

PACKAGE CONTENTS
config
dates
display
localization

FUNCTIONS
describe_option(pat: str = '', _print_desc: bool = True) -> str | None
    Print the description for one or more registered options.

    Call with no arguments to get a listing for all registered options.

Parameters
-----

```

```

pat : str, default ""
    String or string regexp pattern.
    Empty string will return all options.
    For regexp strings, all matching keys will have their description displayed.
_print_desc : bool, default True
    If True (default) the description(s) will be printed to stdout.
    Otherwise, the description(s) will be returned as a string
    (for testing).

Returns
-----
None
    If ``_print_desc=True``.
str
    If the description(s) as a string if ``_print_desc=False``.

See Also
-----
get_option : Retrieve the value of the specified option.
set_option : Set the value of the specified option or options.
reset_option : Reset one or more options to their default value.

Notes
-----
For all available options, please view the
:ref:`User Guide <options.available>`.

Examples
-----
>>> pd.describe_option("display.max_columns") # doctest: +SKIP
display.max_columns : int
    If max_cols is exceeded, switch to truncate view...

detect_console_encoding() -> str
    Try to find the most capable encoding supported by the console.
    slightly modified from the way IPython handles the same issue.

get_option(pat: str) -> Any
    Retrieve the value of the specified option.

    This method allows users to query the current value of a given option
    in the pandas configuration system. Options control various display,
    performance, and behavior-related settings within pandas.

Parameters
-----
pat : str
    Regexp which should match a single option.

    .. warning::

        Partial matches are supported for convenience, but unless you use the
        full option name (e.g. x.y.z.option_name), your code may break in future
        versions if new options with similar names are introduced.

Returns
-----
Any
    The value of the option.

Raises
-----
OptionError : if no such option exists

See Also
-----
set_option : Set the value of the specified option or options.
reset_option : Reset one or more options to their default value.
describe_option : Print the description for one or more registered options.

Notes
-----
For all available options, please view the :ref:`User Guide <options.available>`
or use ``pandas.describe_option()``.

Examples
-----

```

```
>>> pd.get_option("display.max_columns") # doctest: +SKIP
4
```

`option_context(*args)` -> Generator[None]
Context manager to temporarily set options in a ``with`` statement.

This method allows users to set one or more pandas options temporarily within a controlled block. The previous options' values are restored once the block is exited. This is useful when making temporary adjustments to pandas' behavior without affecting the global state.

Parameters

*args : str | object | dict
An even amount of arguments provided in pairs which will be interpreted as (pattern, value) pairs. Alternatively, a single dictionary of {pattern: value} may be provided.

Returns

None
No return value.

Yields

None
No yield value.

See Also

`get_option` : Retrieve the value of the specified option.
`set_option` : Set the value of the specified option.
`reset_option` : Reset one or more options to their default value.
`describe_option` : Print the description for one or more registered options.

Notes

For all available options, please view the :ref:`User Guide <options.available>` or use ``pandas.describe\_option()``.

Examples

>>> from pandas import option_context
>>> with option_context("display.max_rows", 10, "display.max_columns", 5):
... pass
>>> with option_context({"display.max_rows": 10, "display.max_columns": 5}):
... pass

`reset_option(pat: str)` -> None
Reset one or more options to their default value.

This method resets the specified pandas option(s) back to their default values. It allows partial string matching for convenience, but users should exercise caution to avoid unintended resets due to changes in option names in future versions.

Parameters

pat : str/regex
If specified only options matching ``pat\*`` will be reset.
Pass ``"all"`` as argument to reset all options.

.. warning::

Partial matches are supported for convenience, but unless you use the full option name (e.g. x.y.z.option_name), your code may break in future versions if new options with similar names are introduced.

Returns

None
No return value.

See Also

`get_option` : Retrieve the value of the specified option.
`set_option` : Set the value of the specified option or options.

describe_option : Print the description for one or more registered options.

Notes

For all available options, please view the :ref:`User Guide <options.available>`.

Examples

```
>>> pd.reset_option("display.max_columns") # doctest: +SKIP
```

set_option(*args) -> None

Set the value of the specified option or options.

This method allows fine-grained control over the behavior and display settings of pandas. Options affect various functionalities such as output formatting, display limits, and operational behavior. Settings can be modified at runtime without requiring changes to global configurations or environment variables.

Parameters

*args : str | object | dict

Arguments provided in pairs, which will be interpreted as (pattern, value), or as a single dictionary containing multiple option-value pairs.

pattern: str

Regex which should match a single option

value: object

New value of option

.. warning::

Partial pattern matches are supported for convenience, but unless you use the full option name (e.g. x.y.z.option_name), your code may break in future versions if new options with similar names are introduced.

Returns

None

No return value.

Raises

ValueError if odd numbers of non-keyword arguments are provided

TypeError if keyword arguments are provided

OptionError if no such option exists

See Also

get_option : Retrieve the value of the specified option.

reset_option : Reset one or more options to their default value.

describe_option : Print the description for one or more registered options.

option_context : Context manager to temporarily set options in a ``with`` statement.

Notes

For all available options, please view the :ref:`User Guide <options.available>` or use ``pandas.describe\_option()``.

Examples

Option-Value Pair Input:

```
>>> pd.set_option("display.max_columns", 4)
>>> df = pd.DataFrame([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
>>> df
0 1 ... 3 4
0 1 2 ... 4 5
1 6 7 ... 9 10
[2 rows x 5 columns]
>>> pd.reset_option("display.max_columns")
```

Dictionary Input:

```
>>> pd.set_option({"display.max_columns": 4, "display.precision": 1})
>>> df = pd.DataFrame([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
>>> df
```

```

0 1 ... 3 4
0 1 2 ... 4 5
1 6 7 ... 9 10
[2 rows x 5 columns]
>>> pd.reset_option("display.max_columns")
>>> pd.reset_option("display.precision")

```

DATA

```

__all__ = ['config', 'describe_option', 'detect_console_encoding', 'ge...
options = <pandas._config.config.DictWrapper object>

```

FILE

```

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\_config\__init__

```

Help on package pandas._libs in pandas:

NAME

```

pandas._libs

```

PACKAGE CONTENTS

```

_cyutility
algos
arrays
byteswap
groupby
hashing
hashtable
index
indexing
internals
interval
join
json
lib
missing
ops
ops_dispatch
pandas_datetime
pandas_parser
parsers
properties
reshape
sas
sparse
testing
tslib
tslibs (package)
window (package)
writers

```

CLASSES

```

builtins.ValueError(builtins.Exception)
pandas.errors.OutOfBoundsDatetime
pandas._libs.interval.IntervalMixin(builtins.object)
pandas.Interval
pandas._libs.tslibs.nattypes._NaT(datetime.datetime)
pandas.api.typing.NaTType
pandas._libs.tslibs.period._Period(pandas._libs.tslibs.period.PeriodMixin)
pandas.Period
pandas._libs.tslibs.timedeltas._Timedelta(datetime.timedelta)
pandas.Timedelta
pandas._libs.tslibs.timestamps._Timestamp(pandas._libs.tslibs.base.ABCTimestamp)
pandas.Timestamp

```

```

class Interval(pandas._libs.interval.IntervalMixin)
|   Immutable object implementing an Interval, a bounded slice-like interval.
|
|   Attributes
|   -----
|   left : orderable scalar
|       Left bound for the interval.
|   right : orderable scalar
|       Right bound for the interval.
|   closed : {'right', 'left', 'both', 'neither'}, default 'right'
|       Whether the interval is closed on the left-side, right-side, both or
|       neither. See the Notes for more detailed explanation.

```

See Also

IntervalIndex : An Index of Interval objects that are all closed on the same side.
cut : Convert continuous data into discrete bins (Categorical of Interval objects).
qcut : Convert continuous data into bins (Categorical of Interval objects) based on quantiles.
Period : Represents a period of time.

Notes

The parameters `left` and `right` must be from the same type, you must be able to compare them and they must satisfy `left <= right`.

A closed interval (in mathematics denoted by square brackets) contains its endpoints, i.e. the closed interval `[0, 5]` is characterized by the conditions `0 <= x <= 5`. This is what `closed='both'` stands for. An open interval (in mathematics denoted by parentheses) does not contain its endpoints, i.e. the open interval `(0, 5)` is characterized by the conditions `0 < x < 5`. This is what `closed='neither'` stands for. Intervals can also be half-open or half-closed, i.e. `[0, 5)` is described by `0 <= x < 5` (`closed='left'`) and `(0, 5]` is described by `0 < x <= 5` (`closed='right'`).

Examples

It is possible to build Intervals of different types, like numeric ones:

```
>>> iv = pd.Interval(left=0, right=5)
>>> iv
Interval(0, 5, closed='right')
```

You can check if an element belongs to it, or if it contains another interval:

```
>>> 2.5 in iv
True
>>> pd.Interval(left=2, right=5, closed='both') in iv
True
```

You can test the bounds (`closed='right'`, so `0 < x <= 5`):

```
>>> 0 in iv
False
>>> 5 in iv
True
>>> 0.0001 in iv
True
```

Calculate its length

```
>>> iv.length
5
```

You can operate with `+` and `*` over an Interval and the operation is applied to each of its bounds, so the result depends on the type of the bound elements

```
>>> shifted_iv = iv + 3
>>> shifted_iv
Interval(3, 8, closed='right')
>>> extended_iv = iv * 10.0
>>> extended_iv
Interval(0.0, 50.0, closed='right')
```

To create a time interval you can use Timestamps as the bounds

```
>>> year_2017 = pd.Interval(pd.Timestamp('2017-01-01 00:00:00'),
...                          pd.Timestamp('2018-01-01 00:00:00'),
...                          closed='left')
>>> pd.Timestamp('2017-01-01 00:00:00') in year_2017
True
>>> year_2017.length
Timedelta('365 days 00:00:00')
```

Method resolution order:


```
Interval
pandas._libs.interval.IntervalMixin
builtins.object
```

Methods defined here:

```
__add__(self, object, /)

__contains__(self, key, /)
    Return bool(key in self).

__eq__(self, value, /)
    Return self==value.

__floordiv__(self, object, /)

__ge__(self, value, /)
    Return self>=value.

__gt__(self, value, /)
    Return self>value.

__hash__(self, /)
    Return hash(self).

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__le__(self, value, /)
    Return self<=value.

__lt__(self, value, /)
    Return self<value.

__mul__(self, object, /)

__ne__(self, value, /)
    Return self!=value.

__radd__(self, object, /)

__reduce__(self)

__repr__(self, /)
    Return repr(self).

__rfloordiv__(self, value, /)
    Return value//self.

__rmul__(self, object, /)

__rsub__(self, value, /)
    Return value-self.

__rtruediv__(self, value, /)
    Return value/self.

__str__(self, /)
    Return str(self).

__sub__(self, object, /)

__truediv__(self, object, /)
```

```
overlaps(self, other)
    Check whether two Interval objects overlap.
```

Two intervals overlap if they share a common point, including closed endpoints. Intervals that only have an open endpoint in common do not overlap.

Parameters

other : Interval
Interval to check against for an overlap.

Returns

bool
True if the two intervals overlap.

See Also

IntervalArray.overlaps : The corresponding method for IntervalArray.
IntervalIndex.overlaps : The corresponding method for IntervalIndex.

Examples

>>> i1 = pd.Interval(0, 2)
>>> i2 = pd.Interval(1, 3)
>>> i1.overlaps(i2)
True
>>> i3 = pd.Interval(4, 5)
>>> i1.overlaps(i3)
False

Intervals that share closed endpoints overlap:

>>> i4 = pd.Interval(0, 1, closed='both')
>>> i5 = pd.Interval(1, 2, closed='both')
>>> i4.overlaps(i5)
True

Intervals that only have an open endpoint in common do not overlap:

>>> i6 = pd.Interval(1, 2, closed='neither')
>>> i4.overlaps(i6)
False

Static methods defined here:

__new__(*args, **kwargs)
Create and return a new object. See help(type) for accurate signature.

Data descriptors defined here:

closed
String describing the inclusive side the intervals.

Either ``left``, ``right``, ``both`` or ``neither``.

See Also

Interval.closed_left : Check if the interval is closed on the left side.
Interval.closed_right : Check if the interval is closed on the right side.
Interval.open_left : Check if the interval is open on the left side.
Interval.open_right : Check if the interval is open on the right side.

Examples

>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.closed
'left'

left
Left bound for the interval.

See Also

Interval.right : Return the right bound for the interval.
numpy.ndarray.left : A similar method in numpy for obtaining
the left endpoint(s) of intervals.

Examples

>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
>>> interval.left
1

```

right
    Right bound for the interval.

    See Also
    -----
    Interval.left : Return the left bound for the interval.
    numpy.ndarray.right : A similar method in numpy for obtaining
        the right endpoint(s) of intervals.

    Examples
    -----
    >>> interval = pd.Interval(left=1, right=2, closed='left')
    >>> interval
    Interval(1, 2, closed='left')
    >>> interval.right
    2

```

Data and other attributes defined here:

```

__array_priority__ = 1000

```

Methods inherited from pandas._libs.interval.IntervalMixin:

```

__reduce_cython__(self)
__setstate__ = __setstate_cython__(...)
__setstate_cython__(self, __pyx_state)

```

Data descriptors inherited from pandas._libs.interval.IntervalMixin:

```

closed_left
    Check if the interval is closed on the left side.

    For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

    Returns
    -----
    bool
        True if the Interval is closed on the left-side.

    See Also
    -----
    Interval.closed_right : Check if the interval is closed on the right side.
    Interval.open_left : Boolean inverse of closed_left.

    Examples
    -----
    >>> iv = pd.Interval(0, 5, closed='left')
    >>> iv.closed_left
    True

    >>> iv = pd.Interval(0, 5, closed='right')
    >>> iv.closed_left
    False

```

```

closed_right
    Check if the interval is closed on the right side.

    For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

    Returns
    -----
    bool
        True if the Interval is closed on the left-side.

    See Also
    -----
    Interval.closed_left : Check if the interval is closed on the left side.
    Interval.open_right : Boolean inverse of closed_right.

    Examples
    -----

```

```
>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.closed_right
True
```

```
>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.closed_right
False
```

is_empty

Indicates if an interval is empty, meaning it contains no points.

An interval is considered empty if its `left` and `right` endpoints are equal, and it is not closed on both sides. This means that the interval does not include any real points. In the case of an `:class:`pandas.arrays.IntervalArray`` or `:class:`IntervalIndex``, the property returns a boolean array indicating the emptiness of each interval.

Returns

bool or ndarray

A boolean indicating if a scalar `:class:`Interval`` is empty, or a boolean `ndarray` positionally indicating if an `Interval` in an `:class:`~arrays.IntervalArray`` or `:class:`IntervalIndex`` is empty.

See Also

`Interval.length` : Return the length of the Interval.

Examples

An `:class:`Interval`` that contains points is not empty:

```
>>> pd.Interval(0, 1, closed='right').is_empty
False
```

An `Interval` that does not contain any points is empty:

```
>>> pd.Interval(0, 0, closed='right').is_empty
True
>>> pd.Interval(0, 0, closed='left').is_empty
True
>>> pd.Interval(0, 0, closed='neither').is_empty
True
```

An `Interval` that contains a single point is not empty:

```
>>> pd.Interval(0, 0, closed='both').is_empty
False
```

An `:class:`~arrays.IntervalArray`` or `:class:`IntervalIndex`` returns a boolean `ndarray` positionally indicating if an `Interval` is empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'),
...         pd.Interval(1, 2, closed='neither')]
>>> pd.arrays.IntervalArray(ivs).is_empty
array([ True, False])
```

Missing values are not considered empty:

```
>>> ivs = [pd.Interval(0, 0, closed='neither'), np.nan]
>>> pd.IntervalIndex(ivs).is_empty
array([ True, False])
```

length

Return the length of the Interval.

See Also

`Interval.is_empty` : Indicates if an interval contains no points.

Examples

```
>>> interval = pd.Interval(left=1, right=2, closed='left')
>>> interval
Interval(1, 2, closed='left')
```

```

>>> interval.length
1

mid
Return the midpoint of the Interval.

See Also
-----
Interval.left : Return the left bound for the interval.
Interval.right : Return the right bound for the interval.
Interval.length : Return the length of the interval.

Examples
-----
>>> iv = pd.Interval(0, 5)
>>> iv.mid
2.5

open_left
Check if the interval is open on the left side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns
-----
bool
    True if the Interval is not closed on the left-side.

See Also
-----
Interval.open_right : Check if the interval is open on the right side.
Interval.closed_left : Boolean inverse of open_left.

Examples
-----
>>> iv = pd.Interval(0, 5, closed='neither')
>>> iv.open_left
True

>>> iv = pd.Interval(0, 5, closed='both')
>>> iv.open_left
False

open_right
Check if the interval is open on the right side.

For the meaning of `closed` and `open` see :class:`~pandas.Interval`.

Returns
-----
bool
    True if the Interval is not closed on the left-side.

See Also
-----
Interval.open_left : Check if the interval is open on the left side.
Interval.closed_right : Boolean inverse of open_right.

Examples
-----
>>> iv = pd.Interval(0, 5, closed='left')
>>> iv.open_right
True

>>> iv = pd.Interval(0, 5)
>>> iv.open_right
False

class NaTType(pandas._libs.tslibs.nattype._NaT)
    (N)ot-(A)-(T)ime, the time equivalent of NaN.

    NaT is used to denote missing or null values in datetime and timedelta objects
    in pandas. It functions similarly to how NaN is used for numerical data.
    Operations with NaT will generally propagate the NaT value, similar to NaN.
    NaT can be used in pandas data structures like Series and DataFrame
    to represent missing datetime values. It is useful in data analysis
    and time series analysis when working with incomplete or sparse

```

time-based data. Pandas provides robust handling of NaT to ensure consistency and reliability in computations involving datetime objects.

See Also

NA : NA ("not available") missing value indicator.
isna : Detect missing values (NaN or NaT) in an array-like object.
notna : Detect non-missing values.
numpy.nan : Floating point representation of Not a Number (NaN) for numerical data.

Examples

>>> pd.DataFrame([pd.Timestamp("2023"), np.nan], columns=["col_1"])
 col_1
0 2023-01-01
1 NaT

Method resolution order:

NaTType
pandas._libs.tslibs.nattypes.NaT
datetime.datetime
datetime.date
builtins.object

Methods defined here:

__reduce__(self)
__reduce_ex__(self, protocol)
__rfloordiv__(self, other)
__rmul__(self, other)
__rtruediv__(self, other)

as_unit(self, unit, round_ok=True) -> 'NaTType'
Convert the underlying int64 representation to the given unit.

Parameters

unit : {"ns", "us", "ms", "s"}
round_ok : bool, default True
If False and the conversion requires rounding, raise.

Returns

Timestamp

See Also

Timestamp.asm8 : Return numpy datetime64 format with same precision.
Timestamp.to_pydatetime : Convert Timestamp object to a native
Python datetime object.
to_timedelta : Convert argument into timedelta object,
which can represent differences in times.

Examples

>>> ts = pd.Timestamp('2023-01-01 00:00:00.01')
>>> ts
Timestamp('2023-01-01 00:00:00.010000')
>>> ts.unit
'ms'
>>> ts = ts.as_unit('s')
>>> ts
Timestamp('2023-01-01 00:00:00')
>>> ts.unit
's'

astimezone(*args, **kwargs)

Convert timezone-aware Timestamp to another time zone.

This method is used to convert a timezone-aware Timestamp object to a different time zone. The original UTC time remains the same; only the time zone information is changed. If the Timestamp is timezone-naive, a TypeError is raised.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for time which Timestamp will be converted to.
None will remove timezone holding UTC time.

Returns

converted : Timestamp

Raises

TypeError
If Timestamp is tz-naive.

See Also

Timestamp.tz_localize : Localize the Timestamp to a timezone.
DatetimeIndex.tz_convert : Convert a DatetimeIndex to another time zone.
DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.
datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a timestamp object with UTC timezone:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Change to Tokyo timezone:

```
>>> ts.tz_convert(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Can also use ``astimezone``:

```
>>> ts.astimezone(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_convert(tz='Asia/Tokyo')
NaT
```

ceil(*args, **kwargs)

Return a new Timestamp ceiled to this resolution.

Parameters

freq : str

Frequency string indicating the ceiling resolution.

ambiguous : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Raises

ValueError if the freq cannot be converted.

See Also

Timestamp.floor : Round down a Timestamp to the specified resolution.
Timestamp.round : Round a Timestamp to the specified resolution.
Series.dt.ceil : Ceil the datetime values in a Series.

Notes

If the Timestamp has a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be ceiled using multiple frequency units:

```
>>> ts.ceil(freq='h') # hour  
Timestamp('2020-03-14 16:00:00')
```

```
>>> ts.ceil(freq='min') # minute  
Timestamp('2020-03-14 15:33:00')
```

```
>>> ts.ceil(freq='s') # seconds  
Timestamp('2020-03-14 15:32:53')
```

```
>>> ts.ceil(freq='us') # microseconds  
Timestamp('2020-03-14 15:32:52.192549')
```

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

```
>>> ts.ceil(freq='5min')  
Timestamp('2020-03-14 15:35:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.ceil(freq='1h30min')  
Timestamp('2020-03-14 16:30:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.ceil()  
NaT
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.ceil("h", ambiguous=False)  
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.ceil("h", ambiguous=True)  
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

```
combine(*args, **kwargs)  
Timestamp.combine(date, time)
```

Combine a date and time into a single Timestamp object.

This method takes a `date` object and a `time` object and combines them into a single `Timestamp` that has the same date and time fields.

Parameters

date : datetime.date
 The date part of the Timestamp.
time : datetime.time
 The time part of the Timestamp.

Returns

Timestamp
A new `Timestamp` object representing the combined date and time.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Examples

```
>>> from datetime import date, time
>>> pd.Timestamp.combine(date(2020, 3, 14), time(15, 30, 15))
Timestamp('2020-03-14 15:30:15')
```

ctime(*args, **kwargs)
Return a ctime() style string representing the Timestamp.

This method returns a string representing the date and time in the format returned by the standard library's `time.ctime()` function, which is typically in the form 'Day Mon DD HH:MM:SS YYYY'.

If the `Timestamp` is outside the range supported by Python's standard library, a `NotImplementedError` is raised.

Returns

str
A string representing the Timestamp in ctime format.

See Also

time.ctime : Return a string representing time in ctime format.
Timestamp : Represents a single timestamp, similar to `datetime`.
datetime.datetime.ctime : Return a ctime style string from a datetime object.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.ctime()
'Sun Jan  1 10:00:00 2023'
```

date(*args, **kwargs)
Returns `datetime.date` with the same year, month, and day.

This method extracts the date component from the `Timestamp` and returns it as a `datetime.date` object, discarding the time information.

Returns

datetime.date
The date part of the `Timestamp`.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
datetime.datetime.date : Extract the date component from a `datetime` object.

Examples

Extract the date from a Timestamp:

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.date()
datetime.date(2023, 1, 1)
```

day_name(*args, **kwargs)
Return the day name of the Timestamp with specified locale.

Parameters

locale : str, default None (English locale)
Locale determining the language in which to return the day name.

Returns

str

See Also

Timestamp.day_of_week : Return day of the week.

Timestamp.day_of_year : Return day of the year.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

```
>>> ts.day_name()
```

```
'Saturday'
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.day_name()
```

```
nan
```

dst(*args, **kwargs)

Return the daylight saving time (DST) adjustment.

This method returns the DST adjustment as a `datetime.timedelta` object if the Timestamp is timezone-aware and DST is applicable.

See Also

Timestamp.tz_localize : Localize the Timestamp to a timezone.

Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2000-06-01 00:00:00', tz='Europe/Brussels')
```

```
>>> ts
```

```
Timestamp('2000-06-01 00:00:00+0200', tz='Europe/Brussels')
```

```
>>> ts.dst()
```

```
datetime.timedelta(seconds=3600)
```

floor(*args, **kwargs)

Return a new Timestamp floored to this resolution.

Parameters

freq : str

Frequency string indicating the flooring resolution.

ambiguous : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Raises

ValueError if the freq cannot be converted.

See Also

Timestamp.ceil : Round up a Timestamp to the specified resolution.

Timestamp.round : Round a Timestamp to the specified resolution.

Series.dt.floor : Round down the datetime values in a Series.

Notes

If the Timestamp has a timezone, flooring will take place relative to the local ("wall") time and re-localized to the same timezone. When flooring near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be floored using multiple frequency units:

```
>>> ts.floor(freq='h') # hour
Timestamp('2020-03-14 15:00:00')
```

```
>>> ts.floor(freq='min') # minute
Timestamp('2020-03-14 15:32:00')
```

```
>>> ts.floor(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')
```

```
>>> ts.floor(freq='ns') # nanoseconds
Timestamp('2020-03-14 15:32:52.192548651')
```

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

```
>>> ts.floor(freq='5min')
Timestamp('2020-03-14 15:30:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.floor(freq='1h30min')
Timestamp('2020-03-14 15:00:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.floor()
NaT
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 03:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.floor("2h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.floor("2h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

`fromisocalendar(*args, **kwargs)`

int, int, int -> Construct a date from the ISO year, week number and weekday.

This is the inverse of the `date.isocalendar()` function

`fromordinal(*args, **kwargs)`

Construct a timestamp from a proleptic Gregorian ordinal.

This method creates a `Timestamp` object corresponding to the given proleptic Gregorian ordinal, which is a count of days from January 1, 0001 (using the proleptic Gregorian calendar). The time part of the `Timestamp` is set to midnight (00:00:00) by default.

Parameters

ordinal : int

Date corresponding to a proleptic Gregorian ordinal.

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for the Timestamp.

Returns

Timestamp

```

        A `Timestamp` object representing the specified ordinal date.

See Also
-----
Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Notes
-----
By definition there cannot be any tz info on the ordinal itself.

Examples
-----
Convert an ordinal to a `Timestamp`:

>>> pd.Timestamp.fromordinal(737425)
Timestamp('2020-01-01 00:00:00')

Create a `Timestamp` from an ordinal with timezone information:

>>> pd.Timestamp.fromordinal(737425, tz='UTC')
Timestamp('2020-01-01 00:00:00+0000', tz='UTC')

fromtimestamp(*args, **kwargs)
    Create a `Timestamp` object from a POSIX timestamp.

    This method converts a POSIX timestamp (the number of seconds since
    January 1, 1970, 00:00:00 UTC) into a `Timestamp` object. The resulting
    `Timestamp` can be localized to a specific time zone if provided.

Parameters
-----
ts : float
    The POSIX timestamp to convert, representing seconds since
    the epoch (1970-01-01 00:00:00 UTC).
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, optional
    Time zone for the `Timestamp`. If not provided, the `Timestamp` will
    be timezone-naive (i.e., without time zone information).

Returns
-----
Timestamp
    A `Timestamp` object representing the given POSIX timestamp.

See Also
-----
Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.
datetime.datetime.fromtimestamp : Returns a datetime from a POSIX timestamp.

Examples
-----
Convert a POSIX timestamp to a `Timestamp`:

>>> pd.Timestamp.fromtimestamp(1584199972) # doctest: +SKIP
Timestamp('2020-03-14 15:32:52')

Note that the output may change depending on your local time and time zone:

>>> pd.Timestamp.fromtimestamp(1584199972, tz='UTC') # doctest: +SKIP
Timestamp('2020-03-14 15:32:52+0000', tz='UTC')

isocalendar(*args, **kwargs)
    Return a named tuple containing ISO year, week number, and weekday.

See Also
-----
DatetimeIndex.isocalendar : Return a 3-tuple containing ISO year,
    week number, and weekday for the given DatetimeIndex object.
datetime.date.isocalendar : The equivalent method for `datetime.date` objects.

Examples
-----
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.isocalendar()

```

```

datetime.IsoCalendarDate(year=2022, week=52, weekday=7)

isoweekday(*args, **kwargs)
    Return the day of the week represented by the date.

    Monday == 1 ... Sunday == 7.

    See Also
    -----
    Timestamp.weekday : Return the day of the week with Monday=0, Sunday=6.
    Timestamp.isocalendar : Return a tuple containing ISO year, week number
        and weekday.
    datetime.date.isoweekday : Equivalent method in datetime module.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00')
    >>> ts
    Timestamp('2023-01-01 10:00:00')
    >>> ts.isoweekday()
    7

month_name(*args, **kwargs)
    Return the month name of the Timestamp with specified locale.

    This method returns the full name of the month corresponding to the
    `Timestamp`, such as 'January', 'February', etc. The month name can
    be returned in a specified locale if provided; otherwise, it defaults
    to the English locale.

    Parameters
    -----
    locale : str, default None (English locale)
        Locale determining the language in which to return the month name.

    Returns
    -----
    str
        The full month name as a string.

    See Also
    -----
    Timestamp.day_name : Returns the name of the day of the week.
    Timestamp.strftime : Returns a formatted string of the Timestamp.
    datetime.datetime.strftime : Returns a string representing the date and time.

    Examples
    -----
    Get the month name in English (default):

    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
    >>> ts.month_name()
    'March'

    Analogous for ``pd.NaT``:

    >>> pd.NaT.month_name()
    nan

now(*args, **kwargs)
    Return new Timestamp object representing current time local to tz.

    This method returns a new `Timestamp` object that represents the current time.
    If a timezone is provided, either through a timezone object or an IANA
    standard timezone identifier, the current time will be localized to that
    timezone.
    Otherwise, it returns the current local time.

    Parameters
    -----
    tz : str or timezone object, default None
        Timezone to localize to.

    See Also
    -----
    to_datetime : Convert argument to datetime.
    Timestamp.utcnow : Return a new Timestamp representing UTC day and time.

```

Timestamp.today : Return the current time in the local timezone.

Examples

```
>>> pd.Timestamp.now() # doctest: +SKIP
Timestamp('2020-11-16 22:06:16.378782')
```

If you want a specific timezone, in this case 'Brazil/East':

```
>>> pd.Timestamp.now('Brazil/East') # doctest: +SKIP
Timestamp('2025-11-11 22:17:59.609943-03:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.now()
NaT
```

replace(*args, **kwargs)
Implements datetime.replace, handles nanoseconds.

This method creates a new `Timestamp` object by replacing the specified fields with new values. The new `Timestamp` retains the original fields that are not explicitly replaced. This method handles nanoseconds, and the `tzinfo` parameter allows for timezone replacement without conversion.

Parameters

year : int, optional
The year to replace. If `None`, the year is not changed.
month : int, optional
The month to replace. If `None`, the month is not changed.
day : int, optional
The day to replace. If `None`, the day is not changed.
hour : int, optional
The hour to replace. If `None`, the hour is not changed.
minute : int, optional
The minute to replace. If `None`, the minute is not changed.
second : int, optional
The second to replace. If `None`, the second is not changed.
microsecond : int, optional
The microsecond to replace. If `None`, the microsecond is not changed.
nanosecond : int, optional
The nanosecond to replace. If `None`, the nanosecond is not changed.
tzinfo : tz-convertible, optional
The timezone information to replace. If `None`, the timezone is not changed.
fold : int, optional
The fold information to replace. If `None`, the fold is not changed.

Returns

Timestamp
A new `Timestamp` object with the specified fields replaced.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Notes

The `replace` method does not perform timezone conversions. If you need to convert the timezone, use the `tz\_convert` method instead.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Replace year and the hour:

```
>>> ts.replace(year=1999, hour=10)
Timestamp('1999-03-14 10:32:52.192548651+0000', tz='UTC')
```

Replace timezone (not a conversion):

```

>>> import zoneinfo
>>> ts.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
Timestamp('2020-03-14 15:32:52.192548651-0700', tz='US/Pacific')

Analogous for ``pd.NaT``:

>>> pd.NaT.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
NaT

round(*args, **kwargs)
Round the Timestamp to the specified resolution.

This method rounds the given Timestamp down to a specified frequency
level. It is particularly useful in data analysis to normalize timestamps
to regular frequency intervals. For instance, rounding to the nearest
minute, hour, or day can help in time series comparisons or resampling
operations.

Parameters
-----
freq : str
    Frequency string indicating the rounding resolution.
ambiguous : bool or {'raise', 'NaT'}, default 'raise'
    The behavior is as follows:

    * bool contains flags to determine if time is dst or not (note
      that this flag is only applicable for ambiguous fall dst dates).
    * 'NaT' will return NaT for an ambiguous time.
    * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'
    A nonexistent time does not exist in a particular timezone
    where clocks moved forward due to DST.

    * 'shift_forward' will shift the nonexistent time forward to the
      closest existing time.
    * 'shift_backward' will shift the nonexistent time backward to the
      closest existing time.
    * 'NaT' will return NaT where there are nonexistent times.
    * timedelta objects will shift nonexistent times by the timedelta.
    * 'raise' will raise a ValueError if there are
      nonexistent times.

Returns
-----
a new Timestamp rounded to the given resolution of `freq`

Raises
-----
ValueError if the freq cannot be converted

See Also
-----
datetime.round : Similar behavior in native Python datetime module.
Timestamp.floor : Round the Timestamp downward to the nearest multiple
of the specified frequency.
Timestamp.ceil : Round the Timestamp upward to the nearest multiple of
the specified frequency.

Notes
-----
If the Timestamp has a timezone, rounding will take place relative to the
local ("wall") time and re-localized to the same timezone. When rounding
near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
control the re-localization behavior.

Examples
-----
Create a timestamp object:

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')

A timestamp can be rounded using multiple frequency units:

>>> ts.round(freq='h') # hour
Timestamp('2020-03-14 16:00:00')

```

```
>>> ts.round(freq='min') # minute
Timestamp('2020-03-14 15:33:00')

>>> ts.round(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')

>>> ts.round(freq='ms') # milliseconds
Timestamp('2020-03-14 15:32:52.193000')
```

`freq` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):`

```
>>> ts.round(freq='5min')
Timestamp('2020-03-14 15:35:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.round(freq='1h30min')
Timestamp('2020-03-14 15:00:00')
```

Analogous for `pd.NaT`:`

```
>>> pd.NaT.round()
NaT
```

When rounding near a daylight savings time transition, use `ambiguous` or nonexistent` to control how the timestamp should be re-localized.`

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")

>>> ts_tz.round("h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')

>>> ts_tz.round("h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

`strftime(*args, **kwargs)`

Return a formatted string of the Timestamp.

Parameters

`format : str`

Format string to convert Timestamp to string.

See `strftime` documentation for more information on the format string:

<https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>.

See Also

`Timestamp.isoformat` : Return the time formatted according to ISO 8601.

`pd.to_datetime` : Convert argument to datetime.

`Period.strftime` : Format a single Period.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.strftime('%Y-%m-%d %X')
'2020-03-14 15:32:52'
```

`strptime(*args, **kwargs)`

Convert string argument to datetime.

This method is not implemented; calling it will raise `NotImplementedError`. Use `pd.to_datetime()` instead.

Parameters

`date_string : str`

String to convert to a datetime.

`format : str, default None`

The format string to parse time, e.g. `"%d/%m/%Y"`.

See Also

`pd.to_datetime` : Convert argument to datetime.

`datetime.datetime.strptime` : Return a datetime corresponding to a string representing a date and time, parsed according to a separate format string.

`datetime.datetime.strptime` : Return a string representing the date and time, controlled by an explicit format string.
`Timestamp.isoformat` : Return the time formatted according to ISO 8601.

Examples

```
>>> pd.Timestamp.strptime("2023-01-01", "%d/%m/%y")
Traceback (most recent call last):
NotImplementedError
```

`time(*args, **kwargs)`
Return time object with same time but with `tzinfo=None`.

This method extracts the time part of the ``Timestamp`` object, excluding any timezone information. It returns a ``datetime.time`` object which only represents the time (hours, minutes, seconds, and microseconds).

See Also

`Timestamp.date` : Return date object with same year, month and day.
`Timestamp.tz_convert` : Convert timezone-aware `Timestamp` to another time zone.
`Timestamp.tz_localize` : Localize the `Timestamp` to a timezone.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.time()
datetime.time(10, 0)
```

`timestamp(*args, **kwargs)`
Return POSIX timestamp as float.

This method converts the ``Timestamp`` object to a POSIX timestamp, which is the number of seconds since the Unix epoch (January 1, 1970). The returned value is a floating-point number, where the integer part represents the seconds, and the fractional part represents the microseconds.

Returns

float

The POSIX timestamp representation of the ``Timestamp`` object.

See Also

`Timestamp.fromtimestamp` : Construct a ``Timestamp`` from a POSIX timestamp.
`datetime.datetime.timestamp` : Equivalent method from the ``datetime`` module.
`Timestamp.to_pydatetime` : Convert the ``Timestamp`` to a ``datetime`` object.
`Timestamp.to_datetime64` : Converts ``Timestamp`` to ``numpy.datetime64``.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
>>> ts.timestamp()
1584199972.192548
```

`timetuple(*args, **kwargs)`
Return time tuple, compatible with `time.localtime()`.

This method converts the ``Timestamp`` into a time tuple, which is compatible with functions like ``time.localtime()``. The time tuple is a named tuple with attributes such as year, month, day, hour, minute, second, weekday, day of the year, and daylight savings indicator.

See Also

`time.localtime` : Converts a POSIX timestamp into a time tuple.
`Timestamp` : The ``Timestamp`` that represents a specific point in time.
`datetime.datetime.timetuple` : Equivalent method in the ``datetime`` module.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.timetuple()
```

```

time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1,
tm_hour=10, tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=-1)

timetz(*args, **kwargs)
    Return time object with same time and tzinfo.

    This method returns a datetime.time object with
    the time and tzinfo corresponding to the pd.Timestamp
    object, ignoring any information about the day/date.

    See Also
    -----
    datetime.datetime.timetz : Return datetime.time object with the
        same time attributes as the datetime object.
    datetime.time : Class to represent the time of day, independent
        of any particular day.
    datetime.datetime.tzinfo : Attribute of datetime.datetime objects
        representing the timezone of the datetime object.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
    >>> ts
    Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
    >>> ts.timetz()
    datetime.time(10, 0, tzinfo=<DstTzInfo 'Europe/Brussels' CET+1:00:00 STD>)

to_pydatetime(*args, **kwargs)
    Convert a Timestamp object to a native Python datetime object.

    This method is useful for when you need to utilize a pandas Timestamp
    object in contexts where native Python datetime objects are expected
    or required. The conversion discards the nanoseconds component, and a
    warning can be issued in such cases if desired.

    Parameters
    -----
    warn : bool, default True
        If True, issues a warning when the timestamp includes nonzero
        nanoseconds, as these will be discarded during the conversion.

    Returns
    -----
    datetime.datetime or NaT
        Returns a datetime.datetime object representing the timestamp,
        with year, month, day, hour, minute, second, and microsecond components.
        If the timestamp is NaT (Not a Time), returns NaT.

    See Also
    -----
    datetime.datetime : The standard Python datetime class that this method
        returns.
    Timestamp.timestamp : Convert a Timestamp object to POSIX timestamp.
    Timestamp.to_datetime64 : Convert a Timestamp object to numpy.datetime64.

    Examples
    -----
    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
    >>> ts.to_pydatetime()
    datetime.datetime(2020, 3, 14, 15, 32, 52, 192548)

    Analogous for ``pd.NaT``:

    >>> pd.NaT.to_pydatetime()
    NaT

today(*args, **kwargs)
    Return the current time in the local timezone.

    This differs from datetime.today() in that it can be localized to a
    passed timezone.

    Parameters
    -----
    tz : str or timezone object, default None
        Timezone to localize to.

```

See Also

`datetime.datetime.today` : Returns the current local date.
`Timestamp.now` : Returns current time with optional timezone.
`Timestamp` : A class representing a specific timestamp.

Examples

```
>>> pd.Timestamp.today()      # doctest: +SKIP
Timestamp('2020-11-16 22:37:39.969883')
```

Analogous for `pd.NaT`:

```
>>> pd.NaT.today()
NaT
```

`toordinal(*args, **kwargs)`

Return proleptic Gregorian ordinal. January 1 of year 1 is day 1.

The proleptic Gregorian ordinal is a continuous count of days since January 1 of year 1, which is considered day 1. This method converts the `Timestamp` to its equivalent ordinal number, useful for date arithmetic and comparison operations.

See Also

`datetime.datetime.toordinal` : Equivalent method in the `datetime` module.
`Timestamp` : The `Timestamp` that represents a specific point in time.
`Timestamp.fromordinal` : Create a `Timestamp` from an ordinal.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:50')
>>> ts
Timestamp('2023-01-01 10:00:50')
>>> ts.toordinal()
738521
```

`total_seconds(*args, **kwargs)`

Total seconds in the duration.

This method calculates the total duration in seconds by combining the days, seconds, and microseconds of the `Timedelta` object.

See Also

`to_timedelta` : Convert argument to `timedelta`.
`Timedelta` : Represents a duration, the difference between two dates or times.
`Timedelta.seconds` : Returns the seconds component of the `timedelta`.
`Timedelta.microseconds` : Returns the microseconds component of the `timedelta`.

Examples

```
>>> td = pd.Timedelta('1min')
>>> td
Timedelta('0 days 00:01:00')
>>> td.total_seconds()
60.0
```

`tz_convert(*args, **kwargs)`

Convert timezone-aware `Timestamp` to another time zone.

This method is used to convert a timezone-aware `Timestamp` object to a different time zone. The original UTC time remains the same; only the time zone information is changed. If the `Timestamp` is timezone-naive, a `TypeError` is raised.

Parameters

`tz` : str, `zoneinfo.ZoneInfo`, `pytz.timezone`, `dateutil.tz.tzfile` or None
Time zone for time which `Timestamp` will be converted to.
None will remove timezone holding UTC time.

Returns

converted : `Timestamp`

Raises

TypeError

If Timestamp is tz-naive.

See Also

Timestamp.tz_localize : Localize the Timestamp to a timezone.

DatetimeIndex.tz_convert : Convert a DatetimeIndex to another time zone.

DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.

datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a timestamp object with UTC timezone:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
```

```
>>> ts
```

```
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Change to Tokyo timezone:

```
>>> ts.tz_convert(tz='Asia/Tokyo')
```

```
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Can also use ``astimezone``:

```
>>> ts.astimezone(tz='Asia/Tokyo')
```

```
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_convert(tz='Asia/Tokyo')
```

```
NaT
```

tz_localize(*args, **kwargs)

Localize the Timestamp to a timezone.

Convert naive Timestamp to local time zone or remove
timezone from timezone-aware Timestamp.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None

Time zone for time which Timestamp will be converted to.

None will remove timezone holding local time.

ambiguous : bool, 'NaT', default 'raise'

When clocks moved backward due to DST, ambiguous times may arise.

For example in Central European Time (UTC+01), when going from

03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at

00:30:00 UTC and at 01:30:00 UTC. In such a situation, the

`ambiguous` parameter dictates how ambiguous times should be
handled.

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'

A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

The behavior is as follows:

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Returns

localized : Timestamp

Raises

TypeError

If the Timestamp is tz-aware and tz is not None.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.

Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.

datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a naive timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

```
>>> ts
```

```
Timestamp('2020-03-14 15:32:52.192548651')
```

Add 'Europe/Stockholm' as timezone:

```
>>> ts.tz_localize(tz='Europe/Stockholm')
```

```
Timestamp('2020-03-14 15:32:52.192548651+0100', tz='Europe/Stockholm')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_localize()
```

```
NaT
```

tzname(*args, **kwargs)

Return time zone name.

This method returns the name of the Timestamp's time zone as a string.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.

Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
```

```
>>> ts.tzname()
```

```
'CET'
```

utcfromtimestamp(*args, **kwargs)

Timestamp.utcfromtimestamp(ts)

Construct a timezone-aware UTC datetime from a POSIX timestamp.

This method creates a datetime object from a POSIX timestamp, keeping the Timestamp object's timezone.

Parameters

ts : float

POSIX timestamp.

See Also

Timezone.tzname : Return time zone name.

Timestamp.utcnow : Return a new Timestamp representing UTC day and time.

Timestamp.fromtimestamp : Transform timestamp[, tz] to tz's local time from POSIX timestamp.

Notes

Timestamp.utcfromtimestamp behavior differs from datetime.utcfromtimestamp in returning a timezone-aware object.

Examples

```
-----
>>> pd.Timestamp.utcnow(1584199972)
Timestamp('2020-03-14 15:32:52+0000', tz='UTC')
```

```
utcnow(*args, **kwargs)
Timestamp.utcnow()
```

Return a new Timestamp representing UTC day and time.

See Also

```
-----
Timestamp : Constructs an arbitrary datetime.
Timestamp.now : Return the current local date and time, which
                can be timezone-aware.
Timestamp.today : Return the current local date and time with
                timezone information set to None.
to_datetime : Convert argument to datetime.
date_range : Return a fixed frequency DatetimeIndex.
Timestamp.utctimetuple : Return UTC time tuple, compatible with
                time.localtime().
```

Examples

```
-----
>>> pd.Timestamp.utcnow() # doctest: +SKIP
Timestamp('2020-11-16 22:50:18.092888+0000', tz='UTC')
```

```
utcoffset(*args, **kwargs)
Return utc offset.
```

This method returns the difference between UTC and the local time as a `timedelta` object. It is useful for understanding the time difference between the current timezone and UTC.

Returns

```
-----
timedelta
    The difference between UTC and the local time as a timedelta object.
```

See Also

```
-----
datetime.datetime.utcoffset :
    Standard library method to get the UTC offset of a datetime object.
Timestamp.tzname : Return the name of the timezone.
Timestamp.dst : Return the daylight saving time (DST) adjustment.
```

Examples

```
-----
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.utcoffset()
datetime.timedelta(seconds=3600)
```

```
utctimetuple(*args, **kwargs)
Return UTC time tuple, compatible with time.localtime().
```

This method converts the Timestamp to UTC and returns a time tuple containing 9 components: year, month, day, hour, minute, second, weekday, day of year, and DST flag. This is particularly useful for converting a Timestamp to a format compatible with time module functions.

Returns

```
-----
time.struct_time
    A time.struct_time object representing the UTC time.
```

See Also

```
-----
datetime.datetime.utctimetuple :
    Return UTC time tuple, compatible with time.localtime().
Timestamp.timetuple : Return time tuple of local time.
time.struct_time : Time tuple structure used by time functions.
```

Examples

```
-----
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
```

```

>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.utctimetuple()
time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1, tm_hour=9,
tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=0)
weekday(*args, **kwargs)
    Return the day of the week represented by the date.

    Monday == 0 ... Sunday == 6.

    See Also
    -----
    Timestamp.dayofweek : Return the day of the week with Monday=0, Sunday=6.
    Timestamp.isoweekday : Return the day of the week with Monday=1, Sunday=7.
    datetime.date.weekday : Equivalent method in datetime module.

    Examples
    -----
    >>> ts = pd.Timestamp('2023-01-01')
    >>> ts
    Timestamp('2023-01-01 00:00:00')
    >>> ts.weekday()
    6

```

Static methods defined here:

```

__new__(cls)

```

Readonly properties defined here:

```

day
day_of_week
day_of_year
dayofweek
dayofyear
days
days_in_month
daysinmonth
hour
microsecond
microseconds
millisecond
minute
month
nanosecond
nanoseconds
quarter
qyear
second
seconds
tz
tzinfo

```

value
week
weekofyear
year

Data descriptors defined here:

__dict__
 dictionary for instance variables
__weakref__
 list of weak references to the object

Methods inherited from pandas._libs.tslibs.nattype._NaT:

__add__(self, object, /)
 Return self+value.
__eq__(self, value, /)
 Return self==value.
__floordiv__(self, object, /)
__ge__(self, value, /)
 Return self>=value.
__gt__(self, value, /)
 Return self>value.
__hash__(self, /)
 Return hash(self).
__le__(self, value, /)
 Return self<=value.
__lt__(self, value, /)
 Return self<value.
__mul__(self, object, /)
__ne__(self, value, /)
 Return self!=value.
__neg__(self, /)
 -self
__pos__(self, /)
 +self
__radd__(self, object, /)
 Return value+self.
__reduce_cython__(self)
__repr__(self, /)
 Return repr(self).
__rsub__(self, object, /)
 Return value-self.
__setstate_cython__(self, __pyx_state)
__str__(self, /)
 Return str(self).
__sub__(self, object, /)
 Return self-value.
__truediv__(self, object, /)
isoformat(self, sep: str = 'T', timespec: str = 'auto') -> str


```

to_datetime64(self) -> np.datetime64
    Return a NumPy datetime64 object with same precision.

    This method returns a numpy.datetime64 object with the same
    date and time information and precision as the pd.Timestamp object.

    See Also
    -----
    numpy.datetime64 : Class to represent dates and times with high precision.
    Timestamp.to_numpy : Alias for this method.
    Timestamp.asm8 : Alias for this method.
    pd.to_datetime : Convert argument to datetime.

    Examples
    -----
    >>> ts = pd.Timestamp(year=2023, month=1, day=1,
    ...                    hour=10, second=15)
    >>> ts
    Timestamp('2023-01-01 10:00:15')
    >>> ts.to_datetime64()
    numpy.datetime64('2023-01-01T10:00:15.000000')

to_numpy(self, dtype=None, copy=False) -> np.datetime64 | np.timedelta64
    Convert the Timestamp to a NumPy datetime64.

    This is an alias method for `Timestamp.to_datetime64()`. The dtype and
    copy parameters are available here only for compatibility. Their values
    will not affect the return value.

    Parameters
    -----
    dtype : dtype, optional
        Data type of the output, ignored in this method as the return type
        is always `numpy.datetime64`.
    copy : bool, default False
        Whether to ensure that the returned value is a new object. This
        parameter is also ignored as the method does not support copying.

    Returns
    -----
    numpy.datetime64

    See Also
    -----
    DatetimeIndex.to_numpy : Similar method for DatetimeIndex.

    Examples
    -----
    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
    >>> ts.to_numpy()
    numpy.datetime64('2020-03-14T15:32:52.192548651')

    Analogous for ``pd.NaT``:

    >>> pd.NaT.to_numpy()
    numpy.datetime64('NaT')

-----
Data descriptors inherited from pandas._libs.tslibs.nattype._NaT:
asm8
is_leap_year
is_month_end
is_month_start
is_quarter_end
is_quarter_start
is_year_end
is_year_start
-----

```

```

Data and other attributes inherited from pandas._libs.tslibs.nattype._NaT:
__array_priority__ = 100
-----
Methods inherited from datetime.datetime:
__replace__(self, /, **changes)
    The same as replace().
-----
Class methods inherited from datetime.datetime:
fromisoformat(object, /)
    string -> datetime from a string in most ISO 8601 formats
-----
Data descriptors inherited from datetime.datetime:
fold
-----
Data and other attributes inherited from datetime.datetime:
max = datetime.datetime(9999, 12, 31, 23, 59, 59, 999999)
min = datetime.datetime(1, 1, 1, 0, 0)
resolution = datetime.timedelta(microseconds=1)
-----
Methods inherited from datetime.date:
__format__(...)
    Formats self with strftime.
-----
class OutOfBoundsDatetime(builtins.ValueError)
    Raised when the datetime is outside the range that can be represented.

    This error occurs when attempting to convert or parse a datetime value
    that exceeds the bounds supported by pandas' internal datetime
    representation.

    See Also
    -----
    to_datetime : Convert argument to datetime.
    Timestamp : Pandas replacement for python ``datetime.datetime`` object.

    Examples
    -----
    >>> pd.to_datetime("08335394550")
    Traceback (most recent call last):
    OutOfBoundsDatetime: Parsing "08335394550" to datetime overflows,
    at position 0

    Method resolution order:
        OutOfBoundsDatetime
        builtins.ValueError
        builtins.Exception
        builtins.BaseException
        builtins.object

    Data descriptors defined here:
    __weakref__
        list of weak references to the object
    -----
    Static methods inherited from builtins.ValueError:
    __new__(*args, **kwargs) class method of builtins.ValueError
        Create and return a new object. See help(type) for accurate signature.
    -----
    Methods inherited from builtins.BaseException:
    __init__(self, /, *args, **kwargs)

```

```

        Initialize self. See help(type(self)) for accurate signature.

    __reduce__(self, /)
        Helper for pickle.

    __repr__(self, /)
        Return repr(self).

    __setstate__(self, state, /)

    __str__(self, /)
        Return str(self).

    add_note(self, note, /)
        Add a note to the exception

    with_traceback(self, tb, /)
        Set self.__traceback__ to tb and return self.

-----
Data descriptors inherited from builtins.BaseException:

    __cause__
    __context__
    __dict__
    __suppress_context__
    __traceback__
    args

class Period(pandas._libs.tslibs.period._Period)
    Period(
        value=None,
        freq=None,
        ordinal=None,
        year=None,
        month=None,
        quarter=None,
        day=None,
        hour=None,
        minute=None,
        second=None
    )

    Represents a period of time.

    A `Period` represents a specific time span rather than a point in time.
    Unlike `Timestamp`, which represents a single instant, a `Period` defines a
    duration, such as a month, quarter, or year. The exact representation is
    determined by the `freq` parameter.

    Parameters
    -----
    value : Period, str, datetime, date or pandas.Timestamp, default None
        The time period represented (e.g., '4Q2005'). This represents neither
        the start or the end of the period, but rather the entire period itself.
    freq : str, default None
        One of pandas period strings or corresponding objects. Accepted
        strings are listed in the
        :ref:`period alias section <timeseries.period_aliases>` in the user docs.
        If value is datetime, freq is required.
    ordinal : int, default None
        The period offset from the proleptic Gregorian epoch.
    year : int, default None
        Year value of the period.
    month : int, default 1
        Month value of the period.
    quarter : int, default None
        Quarter value of the period.
    day : int, default 1
        Day value of the period.
    hour : int, default 0
        Hour value of the period.

```

minute : int, default 0
Minute value of the period.
second : int, default 0
Second value of the period.

See Also

Timestamp : Pandas replacement for python datetime.datetime object.
date_range : Return a fixed frequency DatetimeIndex.
timedelta_range : Generates a fixed frequency range of timedeltas.

Examples

>>> period = pd.Period('2012-1-1', freq='D')
>>> period
Period('2012-01-01', 'D')

Method resolution order:

Period
pandas._libs.tslibs.period._Period
pandas._libs.tslibs.period.PeriodMixin
builtins.object

Static methods defined here:

```
__new__(  
    cls,  
    value=None,  
    freq=None,  
    ordinal=None,  
    year=None,  
    month=None,  
    quarter=None,  
    day=None,  
    hour=None,  
    minute=None,  
    second=None  
)
```

Data descriptors defined here:

```
__dict__  
    dictionary for instance variables  
  
__weakref__  
    list of weak references to the object
```

Methods inherited from pandas._libs.tslibs.period._Period:

```
__add__(self, object, /)  
  
__eq__(self, value, /)  
    Return self==value.  
  
__ge__(self, value, /)  
    Return self>=value.  
  
__gt__(self, value, /)  
    Return self>value.  
  
__hash__(self, /)  
    Return hash(self).  
  
__le__(self, value, /)  
    Return self<=value.  
  
__lt__(self, value, /)  
    Return self<value.  
  
__ne__(self, value, /)  
    Return self!=value.  
  
__radd__(self, object, /)  
  
__reduce__(self)
```

```

__repr__(self, /)
    Return repr(self).

__rsub__(self, object, /)

__setstate__(self, state)

__str__(...)
    Return a string representation for a particular DataFrame

__sub__(self, object, /)

asfreq(self, freq, how='E') -> 'Period'
    Convert Period to desired frequency, at the start or end of the interval.

    Parameters
    -----
    freq : str, DateOffset
        The target frequency to convert the Period object to.
        If a string is provided,
        it must be a valid :ref:`period alias <timeseries.period_aliases>`.

    how : {'E', 'S', 'end', 'start'}, default 'end'
        Specifies whether to align the period to the start or end of the interval:
        - 'E' or 'end': Align to the end of the interval.
        - 'S' or 'start': Align to the start of the interval.

    Returns
    -----
    Period : Period object with the specified frequency, aligned to the parameter.

    See Also
    -----
    Period.end_time : Return the end Timestamp.
    Period.start_time : Return the start Timestamp.
    Period.dayofyear : Return the day of the year.
    Period.dayofweek : Return the day of the week.

    Examples
    -----
    Convert a daily period to an hourly period, aligning to the end of the day:

    >>> period = pd.Period('2023-01-01', freq='D')
    >>> period.asfreq('h')
    Period('2023-01-01 23:00', 'h')

    Convert a monthly period to a daily period, aligning to the start of the month:

    >>> period = pd.Period('2023-01', freq='M')
    >>> period.asfreq('D', how='start')
    Period('2023-01-01', 'D')

    Convert a yearly period to a monthly period, aligning to the last month:

    >>> period = pd.Period('2023', freq='Y')
    >>> period.asfreq('M', how='end')
    Period('2023-12', 'M')

    Convert a monthly period to an hourly period,
    aligning to the first day of the month:

    >>> period = pd.Period('2023-01', freq='M')
    >>> period.asfreq('h', how='start')
    Period('2023-01-01 00:00', 'H')

    Convert a weekly period to a daily period, aligning to the last day of the week:

    >>> period = pd.Period('2023-08-01', freq='W')
    >>> period.asfreq('D', how='end')
    Period('2023-08-04', 'D')

    strftime(self, fmt: str | None) -> str
    Returns a formatted string representation of the :class:`Period`.

    ``fmt`` must be ``None`` or a string containing one or several directives.
    When ``None``, the format will be determined from the frequency of the Period.

```

The method recognizes the same directives as the `:func:`time.strftime`` function of the standard Python distribution, as well as the specific additional directives ```%f```, ```%F```, ```%q```, ```%l```, ```%u```, ```%n```. (formatting & docs originally from `scikits.timeries`).

Directive	Meaning	Notes
<code>``%a``</code>	Locale's abbreviated weekday name.	
<code>``%A``</code>	Locale's full weekday name.	
<code>``%b``</code>	Locale's abbreviated month name.	
<code>``%B``</code>	Locale's full month name.	
<code>``%c``</code>	Locale's appropriate date and time representation.	
<code>``%d``</code>	Day of the month as a decimal number [01,31].	
<code>``%f``</code>	'Fiscal' year without a century as a decimal number [00,99]	\(1)
<code>``%F``</code>	'Fiscal' year with a century as a decimal number	\(2)
<code>``%H``</code>	Hour (24-hour clock) as a decimal number [00,23].	
<code>``%I``</code>	Hour (12-hour clock) as a decimal number [01,12].	
<code>``%j``</code>	Day of the year as a decimal number [001,366].	
<code>``%m``</code>	Month as a decimal number [01,12].	
<code>``%M``</code>	Minute as a decimal number [00,59].	
<code>``%p``</code>	Locale's equivalent of either AM or PM.	\(3)
<code>``%q``</code>	Quarter as a decimal number [1,4]	
<code>``%S``</code>	Second as a decimal number [00,61].	\(4)
<code>``%l``</code>	Millisecond as a decimal number [000,999].	
<code>``%u``</code>	Microsecond as a decimal number [000000,999999].	
<code>``%n``</code>	Nanosecond as a decimal number [000000000,999999999].	
<code>``%U``</code>	Week number of the year (Sunday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Sunday are considered to be in week 0.	\(5)
<code>``%w``</code>	Weekday as a decimal number [0(Sunday),6].	
<code>``%W``</code>	Week number of the year (Monday as the first day of	\(5)

	the week) as a decimal number [00,53]. All days in a new year preceding the first Monday are considered to be in week 0.		
``%x``	Locale's appropriate date representation.		
``%X``	Locale's appropriate time representation.		
``%y``	Year without century as a decimal number [00,99].		
``%Y``	Year with century as a decimal number.		
``%Z``	Time zone name (no characters if no time zone exists).		
``%%``	A literal ``%`` character.		

The `strftime` method provides a way to represent a `:class:`Period`` object as a string in a specified format. This is particularly useful when displaying date and time data in different locales or customized formats, suitable for reports or user interfaces. It extends the standard Python string formatting capabilities with additional directives specific to `pandas`, accommodating features like fiscal years and precise sub-second components.

Parameters

`fmt` : str or None
String containing the desired format directives. If `None`, the format is determined based on the `Period`'s frequency.

See Also

`Timestamp.strftime` : Return a formatted string of the `Timestamp`.
`to_datetime` : Convert argument to `datetime`.
`time.strftime` : Format a time object as a string according to a specified format string in the standard Python library.

Notes

- (1) The ```%f``` directive is the same as ```%y``` if the frequency is not quarterly. Otherwise, it corresponds to the 'fiscal' year, as defined by the `:attr:`qyear`` attribute.
- (2) The ```%F``` directive is the same as ```%Y``` if the frequency is not quarterly. Otherwise, it corresponds to the 'fiscal' year, as defined by the `:attr:`qyear`` attribute.
- (3) The ```%p``` directive only affects the output hour field if the ```%I``` directive is used to parse the hour.
- (4) The range really is ```0``` to ```61```; this accounts for leap seconds and the (very rare) double leap seconds.
- (5) The ```%U``` and ```%W``` directives are only used in calculations when the day of the week and the year are specified.

Examples

```
>>> from pandas import Period
>>> a = Period(freq='Q-JUL', year=2006, quarter=1)
```

```

>>> a.strftime('%F-Q%q')
'2006-Q1'
>>> # Output the last month in the quarter of this date
>>> a.strftime('%b-%Y')
'Oct-2005'
>>>
>>> a = Period(freq='D', year=2001, month=1, day=1)
>>> a.strftime('%d-%b-%Y')
'01-Jan-2001'
>>> a.strftime('%b. %d, %Y was a %A')
'Jan. 01, 2001 was a Monday'

```

to_timestamp(self, freq=None, how='start') -> Timestamp
Return the Timestamp representation of the Period.

Uses the target frequency specified at the part of the period specified by `how`, which is either `Start` or `Finish`.

If possible, gives microsecond-unit Timestamp. Otherwise gives nanosecond unit.

Parameters

freq : str or DateOffset
Target frequency. Default is 'D' if self._freq is week or longer and 'S' otherwise.
how : str, default 'S' (start)
One of 'S', 'E'. Can be aliased as case insensitive 'Start', 'Finish', 'Begin', 'End'.

Returns

Timestamp

See Also

Timestamp : A class representing a single point in time.
Period : Represents a span of time with a fixed frequency.
PeriodIndex.to_timestamp : Convert a `PeriodIndex` to a `DatetimeIndex`.

Examples

>>> period = pd.Period('2023-1-1', freq='D')
>>> timestamp = period.to_timestamp()
>>> timestamp
Timestamp('2023-01-01 00:00:00')

Class methods inherited from pandas._libs.tslibs.period._Period:

now(freq)
Return the period of now's date.

The `now` method provides a convenient way to generate a period object for the current date and time. This can be particularly useful in financial and economic analysis, where data is often collected and analyzed in regular intervals (e.g., hourly, daily, monthly). By specifying the frequency, users can create periods that match the granularity of their data.

Parameters

freq : str, DateOffset
Frequency to use for the returned period.

See Also

to_datetime : Convert argument to datetime.
Period : Represents a period of time.
Period.to_timestamp : Return the Timestamp representation of the Period.

Examples

>>> pd.Period.now('h') # doctest: +SKIP
Period('2023-06-12 11:00', 'h')

Data descriptors inherited from pandas._libs.tslibs.period._Period:

day

Get day of the month that a Period falls on.

The ``day`` property provides a simple way to access the day component of a ``Period`` object, which represents time spans in various frequencies (e.g., daily, hourly, monthly). If the period's frequency does not include a day component (e.g., yearly or quarterly periods), the returned day corresponds to the first day of that period.

Returns

int

See Also

`Period.dayofweek` : Get the day of the week.

`Period.dayofyear` : Get the day of the year.

Examples

```
>>> p = pd.Period("2018-03-11", freq='h')
```

```
>>> p.day
```

```
11
```

day_of_week

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

`Period.day_of_week` : Day of the week the period lies in.

`Period.weekday` : Alias of `Period.day_of_week`.

`Period.day` : Day of the month.

`Period.dayofyear` : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
```

```
>>> per.day_of_week
```

```
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
```

```
>>> per.day_of_week
```

```
6
```

```
>>> per.start_time.day_of_week
```

```
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
```

```
>>> per.day_of_week
```

```
2
```

```
>>> per.end_time.day_of_week
```

```
2
```

day_of_year

Return the day of the year.

This attribute returns the day of the year on which the particular date occurs. The return value ranges between 1 to 365 for regular

years and 1 to 366 for leap years.

Returns

int

The day of year.

See Also

Period.day : Return the day of the month.

Period.day_of_week : Return the day of week.

PeriodIndex.day_of_year : Return the day of year of all indexes.

Examples

```
>>> period = pd.Period("2015-10-23", freq='h')
```

```
>>> period.day_of_year
```

```
296
```

```
>>> period = pd.Period("2012-12-31", freq='D')
```

```
>>> period.day_of_year
```

```
366
```

```
>>> period = pd.Period("2013-01-01", freq='D')
```

```
>>> period.day_of_year
```

```
1
```

dayofweek

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

Period.day_of_week : Day of the week the period lies in.

Period.weekday : Alias of Period.day_of_week.

Period.day : Day of the month.

Period.dayofyear : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
```

```
>>> per.day_of_week
```

```
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
```

```
>>> per.day_of_week
```

```
6
```

```
>>> per.start_time.day_of_week
```

```
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
```

```
>>> per.day_of_week
```

```
2
```

```
>>> per.end_time.day_of_week
```

```
2
```

dayofyear

Return the day of the year.

This attribute returns the day of the year on which the particular date occurs. The return value ranges between 1 to 365 for regular years and 1 to 366 for leap years.

Returns

int

The day of year.

See Also

Period.day : Return the day of the month.

Period.day_of_week : Return the day of week.

PeriodIndex.day_of_year : Return the day of year of all indexes.

Examples

```
>>> period = pd.Period("2015-10-23", freq='h')
```

```
>>> period.day_of_year
```

```
296
```

```
>>> period = pd.Period("2012-12-31", freq='D')
```

```
>>> period.day_of_year
```

```
366
```

```
>>> period = pd.Period("2013-01-01", freq='D')
```

```
>>> period.day_of_year
```

```
1
```

days_in_month

Get the total number of days in the month that this period falls on.

Returns

int

See Also

Period.daysinmonth : Gets the number of days in the month.

DatetimeIndex.daysinmonth : Gets the number of days in the month.

calendar.monthrange : Returns a tuple containing weekday

(0-6 ~ Mon-Sun) and number of days (28-31).

Examples

```
>>> p = pd.Period('2018-2-17')
```

```
>>> p.days_in_month
```

```
28
```

```
>>> pd.Period('2018-03-01').days_in_month
```

```
31
```

Handles the leap year case as well:

```
>>> p = pd.Period('2016-2-17')
```

```
>>> p.days_in_month
```

```
29
```

daysinmonth

Get the total number of days of the month that this period falls on.

Returns

int

See Also

Period.days_in_month : Return the days of the month.

Period.dayofyear : Return the day of the year.

Examples

```
>>> p = pd.Period("2018-03-11", freq='h')
```

```
>>> p.daysinmonth
```

```
31
```

freq

Return the frequency object for this Period.

The frequency object represents the span of time that this Period covers. It is a DateOffset that defines the interval type (e.g., daily, monthly).

See Also

`Period.freqstr` : Return a string representation of the frequency.

`Period.asfreq` : Convert Period to desired frequency.

Examples

```
>>> period = pd.Period('2020-01', freq='M')
```

```
>>> period.freq
```

```
<MonthEnd>
```

`freqstr`

Return a string representation of the frequency.

This property provides the frequency string associated with the `Period` object. The frequency string describes the granularity of the time span represented by the `Period`. Common frequency strings include 'D' for daily, 'M' for monthly, 'Y' for yearly, etc.

See Also

`Period.asfreq` : Convert Period to desired frequency, at the start or end of the interval.

`period_range` : Return a fixed frequency `PeriodIndex`.

Examples

```
>>> pd.Period('2020-01', 'D').freqstr
```

```
'D'
```

`hour`

Get the hour of the day component of the Period.

Returns

`int`

The hour as an integer, between 0 and 23.

See Also

`Period.second` : Get the second component of the Period.

`Period.minute` : Get the minute component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11 13:03:12.050000")
```

```
>>> p.hour
```

```
13
```

Period longer than a day

```
>>> p = pd.Period("2018-03-11", freq="M")
```

```
>>> p.hour
```

```
0
```

`is_leap_year`

Return True if the period's year is in a leap year.

See Also

`Timestamp.is_leap_year` : Check if the year in a Timestamp is a leap year.

`DatetimeIndex.is_leap_year` : Boolean indicator if the date belongs to a leap year.

Examples

```
>>> period = pd.Period('2022-01', 'M')
```

```
>>> period.is_leap_year
```

```
False
```

```
>>> period = pd.Period('2020-01', 'M')
```

```
>>> period.is_leap_year
```

```
True
```

`minute`

Get minute of the hour component of the Period.

Returns

int

The minute as an integer, between 0 and 59.

See Also

Period.hour : Get the hour component of the Period.

Period.second : Get the second component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11 13:03:12.050000")
```

```
>>> p.minute
```

```
3
```

month

Return the month this Period falls on.

Returns

int

See Also

period.week : Get the week of the year on the given Period.

Period.year : Return the year this Period falls on.

Period.day : Return the day of the month this Period falls on.

Notes

The month is based on the `ordinal` and `base` attributes of the Period.

Examples

Create a Period object for January 2022 and get the month:

```
>>> period = pd.Period('2022-01', 'M')
```

```
>>> period.month
```

```
1
```

Period object with no specified frequency, resulting in a default frequency:

```
>>> period = pd.Period('2022', 'Y')
```

```
>>> period.month
```

```
12
```

Create a Period object with a specified frequency but an incomplete date string:

```
>>> period = pd.Period('2022', 'M')
```

```
>>> period.month
```

```
1
```

Handle a case where the Period object is empty, which results in `NaN`:

```
>>> period = pd.Period('nan', 'M')
```

```
>>> period.month
```

```
nan
```

ordinal

Return the integer ordinal for this Period.

The ordinal is the internal integer representation of the Period, representing its position in the sequence of periods of the given frequency. It counts from an epoch (e.g., for daily frequency, ordinal 0 corresponds to January 1, 1970).

See Also

Period.freq : Return the frequency of the Period.

Period.start_time : Return the start time of the Period.

Examples

```
>>> period = pd.Period('2020-01', freq='M')
```

```
>>> period.ordinal
```

```
600
```

```
>>> period = pd.Period('2020-01-01', freq='D')
>>> period.ordinal
18262
```

quarter

Return the quarter this Period falls on.

See Also

Timestamp.quarter : Return the quarter of the Timestamp.
Period.year : Return the year of the period.
Period.month : Return the month of the period.

Examples

>>> period = pd.Period('2022-04', 'M')
>>> period.quarter
2

qyear

Fiscal year the Period lies in according to its starting-quarter.

The `year` and the `qyear` of the period will be the same if the fiscal and calendar years are the same. When they are not, the fiscal year can be different from the calendar year of the period.

Returns

int
The fiscal year of the period.

See Also

Period.year : Return the calendar year of the period.

Examples

If the natural and fiscal year are the same, `qyear` and `year` will be the same.

```
>>> per = pd.Period('2018Q1', freq='Q')
>>> per.qyear
2018
>>> per.year
2018
```

If the fiscal year starts in April (`Q-MAR`), the first quarter of 2018 will start in April 2017. `year` will then be 2017, but `qyear` will be the fiscal year, 2018.

```
>>> per = pd.Period('2018Q1', freq='Q-MAR')
>>> per.start_time
Timestamp('2017-04-01 00:00:00')
>>> per.qyear
2018
>>> per.year
2017
```

second

Get the second component of the Period.

Returns

int
The second of the Period (ranges from 0 to 59).

See Also

Period.hour : Get the hour component of the Period.
Period.minute : Get the minute component of the Period.

Examples

>>> p = pd.Period("2018-03-11 13:03:12.050000")
>>> p.second
12

week

Get the week of the year on the given Period.

Returns

int

See Also

Period.dayofweek : Get the day component of the Period.

Period.weekday : Get the day component of the Period.

Examples

```
>>> p = pd.Period("2018-03-11", "h")
```

```
>>> p.week
```

```
10
```

```
>>> p = pd.Period("2018-02-01", "D")
```

```
>>> p.week
```

```
5
```

```
>>> p = pd.Period("2018-01-06", "D")
```

```
>>> p.week
```

```
1
```

weekday

Day of the week the period lies in, with Monday=0 and Sunday=6.

If the period frequency is lower than daily (e.g. hourly), and the period spans over multiple days, the day at the start of the period is used.

If the frequency is higher than daily (e.g. monthly), the last day of the period is used.

Returns

int

Day of the week.

See Also

Period.dayofweek : Day of the week the period lies in.

Period.weekday : Alias of Period.dayofweek.

Period.day : Day of the month.

Period.dayofyear : Day of the year.

Examples

```
>>> per = pd.Period('2017-12-31 22:00', 'h')
```

```
>>> per.dayofweek
```

```
6
```

For periods that span over multiple days, the day at the beginning of the period is returned.

```
>>> per = pd.Period('2017-12-31 22:00', '4h')
```

```
>>> per.dayofweek
```

```
6
```

```
>>> per.start_time.dayofweek
```

```
6
```

For periods with a frequency higher than days, the last day of the period is returned.

```
>>> per = pd.Period('2018-01', 'M')
```

```
>>> per.dayofweek
```

```
2
```

```
>>> per.end_time.dayofweek
```

```
2
```

weekofyear

Get the week of the year on the given Period.

Returns

int

See Also

Period.dayofweek : Get the day component of the Period.
Period.weekday : Get the day component of the Period.

Examples

>>> p = pd.Period("2018-03-11", "h")
>>> p.weekofyear
10

>>> p = pd.Period("2018-02-01", "D")
>>> p.weekofyear
5

>>> p = pd.Period("2018-01-06", "D")
>>> p.weekofyear
1

year

Return the year this Period falls on.

Returns

int

See Also

period.month : Get the month of the year for the given Period.
period.day : Return the day of the month the Period falls on.

Notes

The year is based on the `ordinal` and `base` attributes of the Period.

Examples

Create a Period object for January 2023 and get the year:

>>> period = pd.Period('2023-01', 'M')
>>> period.year
2023

Create a Period object for 01 January 2023 and get the year:

>>> period = pd.Period('2023', 'D')
>>> period.year
2023

Get the year for a period representing a quarter:

>>> period = pd.Period('2023Q2', 'Q')
>>> period.year
2023

Handle a case where the Period object is empty, which results in `NaN`:

>>> period = pd.Period('nan', 'M')
>>> period.year
nan

Data and other attributes inherited from pandas._libs.tslibs.period._Period:

__array_priority__ = 100

Methods inherited from pandas._libs.tslibs.period.PeriodMixin:

__reduce_cython__(self)

__setstate_cython__(self, __pyx_state)

Data descriptors inherited from pandas._libs.tslibs.period.PeriodMixin:

`end_time`

Get the Timestamp for the end of the period.

Returns

Timestamp

See Also

`Period.start_time` : Return the start Timestamp.

`Period.dayofyear` : Return the day of year.

`Period.daysinmonth` : Return the days in that month.

`Period.dayofweek` : Return the day of the week.

Examples

For Period:

```
>>> pd.Period('2020-01', 'D').end_time
Timestamp('2020-01-01 23:59:59.999999')
```

For Series:

```
>>> period_index = pd.period_range('2020-1-1 00:00', '2020-3-1 00:00', freq='M')
>>> s = pd.Series(period_index)
>>> s
0    2020-01
1    2020-02
2    2020-03
dtype: period[M]
>>> s.dt.end_time
0    2020-01-31 23:59:59.999999
1    2020-02-29 23:59:59.999999
2    2020-03-31 23:59:59.999999
dtype: datetime64[us]
```

For PeriodIndex:

```
>>> idx = pd.PeriodIndex(["2023-01", "2023-02", "2023-03"], freq="M")
>>> idx.end_time
DatetimeIndex(['2023-01-31 23:59:59.999999',
               '2023-02-28 23:59:59.999999',
               '2023-03-31 23:59:59.999999'],
              dtype='datetime64[us]', freq=None)
```

`start_time`

Get the Timestamp for the start of the period.

Returns

Timestamp

See Also

`Period.end_time` : Return the end Timestamp.

`Period.dayofyear` : Return the day of year.

`Period.daysinmonth` : Return the days in that month.

`Period.dayofweek` : Return the day of the week.

Examples

```
>>> period = pd.Period('2012-1-1', freq='D')
>>> period
Period('2012-01-01', 'D')
```

```
>>> period.start_time
Timestamp('2012-01-01 00:00:00')
```

```
>>> period.end_time
Timestamp('2012-01-01 23:59:59.999999')
```

`class Timedelta(pandas._libs.tslibs.timedeltas._Timedelta)`

`Timedelta(value=<object object at 0x000001A9511B16C0>, unit=None, **kwargs)`

Represents a duration, the difference between two dates or times.

Timedelta is the pandas equivalent of python's ``datetime.timedelta`` and is interchangeable with it in most cases.

Parameters

value : Timedelta, timedelta, np.timedelta64, str, int or float
 Input value.
unit : str, default 'ns'
 If input is an integer, denote the unit of the input.
 If input is a float, denote the unit of the integer parts.
 The decimal parts with resolution lower than 1 nanosecond are ignored.

Possible values:

- * 'W', or 'D'
- * 'days', or 'day'
- * 'hours', 'hour', 'hr', or 'h'
- * 'minutes', 'minute', 'min', or 'm'
- * 'seconds', 'second', 'sec', or 's'
- * 'milliseconds', 'millisecond', 'millis', 'milli', or 'ms'
- * 'microseconds', 'microsecond', 'micros', 'micro', or 'us'
- * 'nanoseconds', 'nanosecond', 'nanos', 'nano', or 'ns'.

.. deprecated:: 3.0.0

 Allowing the values ``w``, ``d``, ``MIN``, ``MS``, ``US`` and ``NS`` to denote units are deprecated in favour of the values ``W``, ``D``, ``min``, ``ms``, ``us`` and ``ns``.

**kwargs

 Available kwargs: {days, seconds, microseconds, milliseconds, minutes, hours, weeks}.
 Values for construction in compat with datetime.timedelta.
 Numpy ints and floats will be coerced to python ints and floats.

See Also

Timestamp : Represents a single timestamp in time.
TimedeltaIndex : Immutable Index of timedelta64 data.
DateOffset : Standard kind of date increment used for a date range.
to_timedelta : Convert argument to timedelta.
datetime.timedelta : Represents a duration in the datetime module.
numpy.timedelta64 : Represents a duration compatible with NumPy.

Notes

The constructor may take in either both values of value and unit or kwargs as above. Either one of them must be used during initialization

The ``.value`` attribute is always in ns.

If the precision is higher than nanoseconds, the precision of the duration is truncated to nanoseconds.

Examples

Here we initialize Timedelta object with both value and unit

```
>>> td = pd.Timedelta(1, "D")
>>> td
Timedelta('1 days 00:00:00')
```

Here we initialize the Timedelta object with kwargs

```
>>> td2 = pd.Timedelta(days=1)
>>> td2
Timedelta('1 days 00:00:00')
```

We see that either way we get the same result

Method resolution order:

```
Timedelta
pandas._libs.tslibs.timedeltas._Timedelta
datetime.timedelta
builtins.object
```

Methods defined here:

```

__abs__(self)
__add__(self, other)
__divmod__(self, other)
__floordiv__(self, other)
__mod__(self, other)
__mul__(self, other)
__neg__(self)
__pos__(self)
__radd__(self, other)
__rdivmod__(self, other)
__reduce__(self)
__rfloordiv__(self, other)
__rmod__(self, other)
__rmul__ = __mul__(self, other)
__rsub__(self, other)
__rtruediv__(self, other)
__setstate__(self, state)
__sub__(self, other)
__truediv__(self, other)

ceil(self, freq)
    Return a new Timedelta ceiled to this resolution.

    Parameters
    -----
    freq : str
        Frequency string indicating the ceiling resolution. Must be a fixed
        frequency like 's' (second) not 'ME' (month end). See
        :ref:`frequency aliases <timeseries.offset_aliases>` for
        a list of possible `freq` values.

    Returns
    -----
    Timedelta
        A new Timedelta object ceiled to the specified resolution.

    See Also
    -----
        Timedelta.floor : Floor the Timedelta to the specified resolution.
        Timedelta.round : Round the Timedelta to the nearest specified resolution.

    Examples
    -----
    >>> td = pd.Timedelta('1001ms')
    >>> td
    Timedelta('0 days 00:00:01.001000')
    >>> td.ceil('s')
    Timedelta('0 days 00:00:02')

floor(self, freq)
    Return a new Timedelta floored to this resolution.

    Parameters
    -----
    freq : str
        Frequency string indicating the flooring resolution.
        It uses the same units as class constructor :class:`~pandas.Timedelta`.

```

Returns

Timedelta

A new Timedelta object floored to the specified resolution.

See Also

Timestamp.ceil : Round the Timestamp up to the nearest specified resolution.
Timestamp.round : Round the Timestamp to the nearest specified resolution.

Examples

```
>>> td = pd.Timedelta('1001ms')
>>> td
Timedelta('0 days 00:00:01.001000')
>>> td.floor('s')
Timedelta('0 days 00:00:01')
```

round(self, freq)

Round the Timedelta to the specified resolution.

Parameters

freq : str

Frequency string indicating the rounding resolution.
It uses the same units as class constructor :class:`~pandas.Timedelta`.

Returns

a new Timedelta rounded to the given resolution of `freq`

Raises

ValueError if the freq cannot be converted

See Also

Timedelta.floor : Floor the Timedelta to the specified resolution.
Timedelta.round : Round the Timedelta to the nearest specified resolution.
Timestamp.ceil : Similar method for Timestamp objects.

Examples

```
>>> td = pd.Timedelta('1001ms')
>>> td
Timedelta('0 days 00:00:01.001000')
>>> td.round('s')
Timedelta('0 days 00:00:01')
```

Static methods defined here:

__new__(cls, value=<object object at 0x0000001A9511B16C0>, unit=None, **kwargs)

Data descriptors defined here:

__dict__

dictionary for instance variables

__weakref__

list of weak references to the object

Methods inherited from pandas._libs.tslibs.timedeltas._Timedelta:

__bool__(self, /)

True if self else False

__eq__(self, value, /)

Return self==value.

__ge__(self, value, /)

Return self>=value.

__gt__(self, value, /)

Return self>value.

```

__hash__(self, /)
    Return hash(self).

__le__(self, value, /)
    Return self<=value.

__lt__(self, value, /)
    Return self<value.

__ne__(self, value, /)
    Return self!=value.

__reduce_cython__(self)

__repr__(self, /)
    Return repr(self).

__setstate_cython__(self, __pyx_state)

__str__(self, /)
    Return str(self).

as_unit(self, unit, round_ok=True)
    Convert the underlying int64 representation to the given unit.

    Parameters
    -----
    unit : {"ns", "us", "ms", "s"}
    round_ok : bool, default True
        If False and the conversion requires rounding, raise.

    Returns
    -----
    Timedelta

    See Also
    -----
    Timedelta : Represents a duration, the difference between two dates or times.
    to_timedelta : Convert argument to timedelta.
    Timedelta.asm8 : Return a numpy timedelta64 array scalar view.

    Examples
    -----
    >>> td = pd.Timedelta('1001ms')
    >>> td
    Timedelta('0 days 00:00:01.001000')
    >>> td.as_unit('s')
    Timedelta('0 days 00:00:01')

isoformat(self) -> str
    Format the Timedelta as ISO 8601 Duration.

    ``P[n]Y[n]M[n]DT[n]H[n]M[n]S``, where the ``[n]`` s are replaced by the
    values. See Wikipedia:
    `ISO 8601 § Durations <https://en.wikipedia.org/wiki/ISO_8601#Durations>`_.

    Returns
    -----
    str

    See Also
    -----
    Timestamp.isoformat : Function is used to convert the given
        Timestamp object into the ISO format.

    Notes
    -----
    The longest component is days, whose value may be larger than
    365.
    Every component is always included, even if its value is 0.
    Pandas uses nanosecond precision, so up to 9 decimal places may
    be included in the seconds component.
    Trailing 0's are removed from the seconds component after the decimal.
    We do not 0 pad components, so it's `...T5H...`, not `...T05H...`

    Examples

```

```

-----
>>> td = pd.Timedelta(days=6, minutes=50, seconds=3,
...                     milliseconds=10, microseconds=10, nanoseconds=12)

>>> td.isoformat()
'P6DT0H50M3.010010012S'
>>> pd.Timedelta(hours=1, seconds=10).isoformat()
'P0DT1H0M10S'
>>> pd.Timedelta(days=500.5).isoformat()
'P500DT12H0M0S'

to_numpy(self, dtype=None, copy=False) -> np.timedelta64
Convert the Timedelta to a NumPy timedelta64.

This is an alias method for `Timedelta.to_timedelta64()`.

Parameters
-----
dtype : NoneType
    It is available here only for compatibility. Its value will not
    affect the return value.
copy : bool, default False
    It is available here only for compatibility. Its value will not
    affect the return value.

Returns
-----
numpy.timedelta64

See Also
-----
Series.to_numpy : Similar method for Series.

Examples
-----
>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.to_numpy()
numpy.timedelta64(259200000000000, 'ns')

to_pytimedelta(self)
Convert a pandas Timedelta object into a python ``datetime.timedelta`` object.

Timedelta objects are internally saved as numpy datetime64[ns] dtype.
Use to_pytimedelta() to convert to object dtype.

Returns
-----
datetime.timedelta or numpy.array of datetime.timedelta

See Also
-----
to_timedelta : Convert argument to Timedelta type.

Notes
-----
Any nanosecond resolution will be lost.

Examples
-----
>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.to_pytimedelta()
datetime.timedelta(days=3)

to_timedelta64(self) -> np.timedelta64
Return a numpy.timedelta64 object with 'ns' precision.

Since NumPy uses ``timedelta64`` objects for its time operations, converting
a pandas ``Timedelta`` into a NumPy ``timedelta64`` provides seamless
integration between the two libraries, especially when working in environments
that heavily rely on NumPy for array-based calculations.

See Also
-----

```

`to_timedelta` : Convert argument to `timedelta`.
`numpy.timedelta64` : A NumPy object for time duration.
`Timedelta` : Represents a duration, the difference between two dates or times.

Examples

```
>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.to_timedelta64()
numpy.timedelta64(259200000000000,'ns')
```

`total_seconds(self)` -> float
Total seconds in the duration.

This method calculates the total duration in seconds by combining the days, seconds, and microseconds of the `Timedelta` object.

See Also

`to_timedelta` : Convert argument to `timedelta`.
`Timedelta` : Represents a duration, the difference between two dates or times.
`Timedelta.seconds` : Returns the seconds component of the `timedelta`.
`Timedelta.microseconds` : Returns the microseconds component of the `timedelta`.

Examples

```
>>> td = pd.Timedelta('1min')
>>> td
Timedelta('0 days 00:01:00')
>>> td.total_seconds()
60.0
```

`view(self, dtype)`
Array view compatibility.

This method allows you to reinterpret the underlying data of a `Timedelta` object as a different dtype. The `view` method provides a way to reinterpret the internal representation of the `Timedelta` object without modifying its data. This is particularly useful when you need to work with the underlying data directly, such as for performance optimizations or interfacing with low-level APIs. The returned value is typically the number of nanoseconds since the epoch, represented as an integer or another specified dtype.

Parameters

`dtype` : str or dtype
The dtype to view the underlying data as.

See Also

`Timedelta.asm8` : Return a numpy `timedelta64` array scalar view.
`numpy.ndarray.view` : Returns a view of an array with the same data.
`Timedelta.to_numpy` : Converts the `Timedelta` to a NumPy `timedelta64`.
`Timedelta.total_seconds` : Returns the total duration of the `Timedelta` object in seconds.

Examples

```
>>> td = pd.Timedelta('3D')
>>> td
Timedelta('3 days 00:00:00')
>>> td.view(int)
2592000000000000
```

Data descriptors inherited from `pandas._libs.tslibs.timedeltas._Timedelta`:

`asm8`

Return a numpy `timedelta64` array scalar view.

Provides access to the array scalar view (i.e. a combination of the value and the units) associated with the `numpy.timedelta64().view()`, including a 64-bit integer representation of the `timedelta` in nanoseconds (Python int compatible).

Returns

numpy.timedelta64 array scalar view
Array scalar view of the timedelta in nanoseconds.

See Also

Timedelta.total_seconds : Return the total seconds in the duration.
Timedelta.components : Return a namedtuple of the Timedelta's components.
Timedelta.to_timedelta64 : Convert the Timedelta to a numpy.timedelta64.

Examples

```
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
>>> td.asm8
numpy.timedelta64(86520000003042,'ns')
```

```
>>> td = pd.Timedelta('2 min 3 s')
>>> td.asm8
numpy.timedelta64(123000000000,'ns')
```

```
>>> td = pd.Timedelta('3 ms 5 us')
>>> td.asm8
numpy.timedelta64(3005000,'ns')
```

```
>>> td = pd.Timedelta(42, unit='ns')
>>> td.asm8
numpy.timedelta64(42,'ns')
```

components

Return a components namedtuple-like.

Each component represents a different time unit, allowing you to access the breakdown of the total duration in terms of days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds.

See Also

Timedelta.total_seconds : Returns the total duration of the Timedelta in seconds.
to_timedelta : Convert argument to Timedelta.
Timedelta : Represents a duration, the difference between two dates or times.

Examples

```
>>> td = pd.Timedelta('2 day 4 min 3 us 42 ns')
>>> td.components
Components(days=2, hours=0, minutes=4, seconds=0, milliseconds=0,
           microseconds=3, nanoseconds=42)
```

days

Returns the days of the timedelta.

The `days` attribute of a `pandas.Timedelta` object provides the number of days represented by the `Timedelta`. This is useful for extracting the day component from a `Timedelta` that may also include hours, minutes, seconds, and smaller time units. This attribute simplifies the process of working with durations where only the day component is of interest.

Returns

int

See Also

Timedelta.seconds : Returns the seconds component of the timedelta.
Timedelta.microseconds : Returns the microseconds component of the timedelta.
Timedelta.total_seconds : Returns the total duration in seconds.

Examples

```
>>> td = pd.Timedelta(1, "d")
>>> td.days
1
```

```
>>> td = pd.Timedelta('4 min 3 us 42 ns')
>>> td.days
```


0

microseconds

Return the number of microseconds (n), where $0 \leq n < 1$ millisecond.

`Timedelta.microseconds = milliseconds * 1000 + microseconds.`

Returns

int

Number of microseconds.

See Also

`Timedelta.components` : Return all attributes with assigned values (i.e. days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds).

Examples

****Using string input****

```
>>> td = pd.Timedelta('1 days 2 min 3 us')
```

```
>>> td.microseconds
```

```
3
```

****Using integer input****

```
>>> td = pd.Timedelta(42, unit='us')
```

```
>>> td.microseconds
```

```
42
```

nanoseconds

Return the number of nanoseconds (n), where $0 \leq n < 1$ microsecond.

Returns

int

Number of nanoseconds.

See Also

`Timedelta.components` : Return all attributes with assigned values (i.e. days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds).

Examples

****Using string input****

```
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
```

```
>>> td.nanoseconds
```

```
42
```

****Using integer input****

```
>>> td = pd.Timedelta(42, unit='ns')
```

```
>>> td.nanoseconds
```

```
42
```

resolution_string

Return a string representing the lowest timedelta resolution.

Each timedelta has a defined resolution that represents the lowest OR most granular level of precision. Each level of resolution is represented by a short string as defined below:

Resolution:	Return value
-------------	--------------

* Days:	'D'
* Hours:	'h'
* Minutes:	'min'
* Seconds:	's'
* Milliseconds:	'ms'
* Microseconds:	'us'

* Nanoseconds: 'ns'

Returns

str

Timedelta resolution.

Examples

```
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
>>> td.resolution_string
'ns'
```

```
>>> td = pd.Timedelta('1 days 2 min 3 us')
>>> td.resolution_string
'us'
```

```
>>> td = pd.Timedelta('2 min 3 s')
>>> td.resolution_string
's'
```

```
>>> td = pd.Timedelta(36, unit='us')
>>> td.resolution_string
'us'
```

seconds

Return the total hours, minutes, and seconds of the timedelta as seconds.

Timedelta.seconds = hours * 3600 + minutes * 60 + seconds.

Returns

int

Number of seconds.

See Also

Timedelta.components : Return all attributes with assigned values
(i.e. days, hours, minutes, seconds, milliseconds, microseconds,
nanoseconds).

Timedelta.total_seconds : Express the Timedelta as total number of seconds.

Examples

****Using string input****

```
>>> td = pd.Timedelta('1 days 2 min 3 us 42 ns')
>>> td.seconds
120
```

****Using integer input****

```
>>> td = pd.Timedelta(42, unit='s')
>>> td.seconds
42
```

unit

Return the unit of Timedelta object.

The unit of Timedelta object is nanosecond, i.e., 'ns' by default.

Returns

str

See Also

Timedelta.value : Return the value of Timedelta object in nanoseconds.

Timedelta.as_unit : Convert the underlying int64 representation to
the given unit.

Examples

```
>>> td = pd.Timedelta(42, unit='us')
'ns'
```

value

Return the value of Timedelta object in nanoseconds.

Return the total seconds, milliseconds and microseconds of the timedelta as nanoseconds.

Returns

int

See Also

Timedelta.unit : Return the unit of Timedelta object.

Examples

```
>>> pd.Timedelta(1, "us").value
1000
```

Data and other attributes inherited from pandas._libs.tslibs.timedeltas._Timedelta:

__array_priority__ = 100

__pyx_vtable__ = <capsule object NULL>

max = Timedelta('106751 days 23:47:16.854775807')

Returns the maximum bound possible for Timedelta.

This property provides access to the largest possible value that can be represented by a Timedelta object.

Returns

Timedelta

See Also

Timedelta.min: Returns the minimum bound possible for Timedelta.

Timedelta.resolution: Returns the smallest possible difference between non-equal Timedelta objects.

Examples

```
>>> pd.Timedelta.max
106751 days 23:47:16.854775807
```

min = Timedelta('-106752 days +00:12:43.145224193')

Returns the minimum bound possible for Timedelta.

This property provides access to the smallest possible value that can be represented by a Timedelta object.

Returns

Timedelta

See Also

Timedelta.max: Returns the maximum bound possible for Timedelta.

Timedelta.resolution: Returns the smallest possible difference between non-equal Timedelta objects.

Examples

```
>>> pd.Timedelta.min
-106752 days +00:12:43.145224193
```

resolution = Timedelta('0 days 00:00:00.000000001')

Returns the smallest possible difference between non-equal Timedelta objects.

The resolution value is determined by the underlying representation of time units and is equivalent to Timedelta(nanoseconds=1).

Returns

Timedelta

See Also

Timedelta.max: Returns the maximum bound possible for Timedelta.
Timedelta.min: Returns the minimum bound possible for Timedelta.

Examples

>>> pd.Timedelta.resolution
0 days 00:00:00.000000001

class Timestamp(pandas._libs.tslibs.timestamps._Timestamp)

Timestamp(
 ts_input=<object object at 0x000001A9511B16D0>,
 year=None,
 month=None,
 day=None,
 hour=None,
 minute=None,
 second=None,
 microsecond=None,
 tzinfo=None,
 *,
 nanosecond=None,
 tz=<object object at 0x000001A9511B16D0>,
 unit=None,
 fold=None
)

Pandas replacement for python datetime.datetime object.

Timestamp is the pandas equivalent of python's Datetime and is interchangeable with it in most cases. It's the type used for the entries that make up a DatetimeIndex, and other timeseries oriented data structures in pandas.

Parameters

ts_input : datetime-like, str, int, float
 Value to be converted to Timestamp.
year : int
 Value of year.
month : int
 Value of month.
day : int
 Value of day.
hour : int, optional, default 0
 Value of hour.
minute : int, optional, default 0
 Value of minute.
second : int, optional, default 0
 Value of second.
microsecond : int, optional, default 0
 Value of microsecond.
tzinfo : datetime.tzinfo, optional, default None
 Timezone info.
nanosecond : int, optional, default 0
 Value of nanosecond.
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
 Time zone for time which Timestamp will have.
unit : str
 Unit used for conversion if ts_input is of type int or float. The valid values are 'W', 'D', 'h', 'm', 's', 'ms', 'us', and 'ns'. For example, 's' means seconds and 'ms' means milliseconds.

For float inputs, the result will be stored in nanoseconds, and the unit attribute will be set as ``'ns'``.
fold : {0, 1}, default None, keyword-only
 Due to daylight saving time, one wall clock time can occur twice when shifting from summer to winter time; fold describes whether the datetime-like corresponds to the first (0) or the second time (1) the wall clock hits the ambiguous time.

See Also

Timedelta : Represents a duration, the difference between two dates or times.

`datetime.datetime` : Python `datetime.datetime` object.

Notes

There are essentially three calling conventions for the constructor. The primary form accepts four parameters. They can be passed by position or keyword.

The other two forms mimic the parameters from `datetime.datetime`. They can be passed by either position or keyword, but not both mixed together.

Examples

Using the primary calling convention:

This converts a datetime-like string

```
>>> pd.Timestamp('2017-01-01T12')
Timestamp('2017-01-01 12:00:00')
```

This converts a float representing a Unix epoch in units of seconds

```
>>> pd.Timestamp(1513393355.5, unit='s')
Timestamp('2017-12-16 03:02:35.500000')
```

This converts an int representing a Unix-epoch in units of weeks

```
>>> pd.Timestamp(1535, unit='W')
Timestamp('1999-06-03 00:00:00')
```

This converts an int representing a Unix-epoch in units of seconds and for a particular timezone

```
>>> pd.Timestamp(1513393355, unit='s', tz='US/Pacific')
Timestamp('2017-12-15 19:02:35-0800', tz='US/Pacific')
```

Using the other two forms that mimic the API for `datetime.datetime`:

```
>>> pd.Timestamp(2017, 1, 1, 12)
Timestamp('2017-01-01 12:00:00')
```

```
>>> pd.Timestamp(year=2017, month=1, day=1, hour=12)
Timestamp('2017-01-01 12:00:00')
```

Method resolution order:

```
Timestamp
pandas._libs.tslib.timestamps._Timestamp
pandas._libs.tslib.base.ABCTimestamp
datetime.datetime
datetime.date
builtins.object
```

Methods defined here:

```
astimezone = tz_convert(self, tz)
```

```
ceil(self, freq, ambiguous='raise', nonexistent='raise')
    Return a new Timestamp ceiled to this resolution.
```

Parameters

`freq` : str

Frequency string indicating the ceiling resolution.

`ambiguous` : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a `ValueError` for an ambiguous time.

`nonexistent` : {'raise', 'shift_forward', 'shift_backward', 'NaT', `timedelta`}, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.

- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Raises

ValueError if the freq cannot be converted.

See Also

Timestamp.floor : Round down a Timestamp to the specified resolution.

Timestamp.round : Round a Timestamp to the specified resolution.

Series.dt.ceil : Ceil the datetime values in a Series.

Notes

If the Timestamp has a timezone, ceiling will take place relative to the local ("wall") time and re-localized to the same timezone. When ceiling near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be ceiled using multiple frequency units:

```
>>> ts.ceil(freq='h') # hour
Timestamp('2020-03-14 16:00:00')
```

```
>>> ts.ceil(freq='min') # minute
Timestamp('2020-03-14 15:33:00')
```

```
>>> ts.ceil(freq='s') # seconds
Timestamp('2020-03-14 15:32:53')
```

```
>>> ts.ceil(freq='us') # microseconds
Timestamp('2020-03-14 15:32:52.192549')
```

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

```
>>> ts.ceil(freq='5min')
Timestamp('2020-03-14 15:35:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.ceil(freq='1h30min')
Timestamp('2020-03-14 16:30:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.ceil()
NaT
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.ceil("h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.ceil("h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

ctime(self)

Return a ctime() style string representing the Timestamp.

This method returns a string representing the date and time in the format returned by the standard library's ``time.ctime()`` function, which is typically in the form 'Day Mon DD HH:MM:SS YYYY'.

If the `Timestamp` is outside the range supported by Python's standard library, a `NotImplementedError` is raised.

Returns

str

A string representing the Timestamp in ctime format.

See Also

`time.ctime` : Return a string representing time in ctime format.

`Timestamp` : Represents a single timestamp, similar to `datetime`.

`datetime.datetime.ctime` : Return a ctime style string from a datetime object.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00')
```

```
>>> ts.ctime()
```

```
'Sun Jan  1 10:00:00 2023'
```

`date(self)`

Returns `datetime.date` with the same year, month, and day.

This method extracts the date component from the `Timestamp` and returns it as a `datetime.date` object, discarding the time information.

Returns

`datetime.date`

The date part of the `Timestamp`.

See Also

`Timestamp` : Represents a single timestamp, similar to `datetime`.

`datetime.datetime.date` : Extract the date component from a `datetime` object.

Examples

Extract the date from a Timestamp:

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00.00')
```

```
>>> ts
```

```
Timestamp('2023-01-01 10:00:00')
```

```
>>> ts.date()
```

```
datetime.date(2023, 1, 1)
```

`dst(self)`

Return the daylight saving time (DST) adjustment.

This method returns the DST adjustment as a `datetime.timedelta` object if the Timestamp is timezone-aware and DST is applicable.

See Also

`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2000-06-01 00:00:00', tz='Europe/Brussels')
```

```
>>> ts
```

```
Timestamp('2000-06-01 00:00:00+0200', tz='Europe/Brussels')
```

```
>>> ts.dst()
```

```
datetime.timedelta(seconds=3600)
```

`floor(self, freq, ambiguous='raise', nonexistent='raise')`

Return a new Timestamp floored to this resolution.

Parameters

`freq` : str

Frequency string indicating the flooring resolution.

`ambiguous` : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

```

* bool contains flags to determine if time is dst or not (note
  that this flag is only applicable for ambiguous fall dst dates).
* 'NaT' will return NaT for an ambiguous time.
* 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'
A nonexistent time does not exist in a particular timezone
where clocks moved forward due to DST.

* 'shift_forward' will shift the nonexistent time forward to the
  closest existing time.
* 'shift_backward' will shift the nonexistent time backward to the
  closest existing time.
* 'NaT' will return NaT where there are nonexistent times.
* timedelta objects will shift nonexistent times by the timedelta.
* 'raise' will raise a ValueError if there are
  nonexistent times.

Raises
-----
ValueError if the freq cannot be converted.

See Also
-----
Timestamp.ceil : Round up a Timestamp to the specified resolution.
Timestamp.round : Round a Timestamp to the specified resolution.
Series.dt.floor : Round down the datetime values in a Series.

Notes
-----
If the Timestamp has a timezone, flooring will take place relative to the
local ("wall") time and re-localized to the same timezone. When flooring
near daylight savings time, use ``nonexistent`` and ``ambiguous`` to
control the re-localization behavior.

Examples
-----
Create a timestamp object:

>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')

A timestamp can be floored using multiple frequency units:

>>> ts.floor(freq='h') # hour
Timestamp('2020-03-14 15:00:00')

>>> ts.floor(freq='min') # minute
Timestamp('2020-03-14 15:32:00')

>>> ts.floor(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')

>>> ts.floor(freq='ns') # nanoseconds
Timestamp('2020-03-14 15:32:52.192548651')

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

>>> ts.floor(freq='5min')
Timestamp('2020-03-14 15:30:00')

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

>>> ts.floor(freq='1h30min')
Timestamp('2020-03-14 15:00:00')

Analogous for ``pd.NaT``:

>>> pd.NaT.floor()
NaT

When rounding near a daylight savings time transition, use ``ambiguous`` or
``nonexistent`` to control how the timestamp should be re-localized.

>>> ts_tz = pd.Timestamp("2021-10-31 03:30:00").tz_localize("Europe/Amsterdam")

>>> ts_tz.floor("2h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')

```



```

>>> ts_tz.floor("2h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')

```

isocalendar(self)
Return a named tuple containing ISO year, week number, and weekday.

See Also

DatetimeIndex.isocalendar : Return a 3-tuple containing ISO year, week number, and weekday for the given `DatetimeIndex` object.
datetime.date.isocalendar : The equivalent method for ``datetime.date`` objects.

Examples

```

>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.isocalendar()
datetime.ISOCalendarDate(year=2022, week=52, weekday=7)

```

isoweekday(self)
Return the day of the week represented by the date.

Monday == 1 ... Sunday == 7.

See Also

Timestamp.weekday : Return the day of the week with Monday=0, Sunday=6.
Timestamp.isocalendar : Return a tuple containing ISO year, week number and weekday.
datetime.date.isoweekday : Equivalent method in `datetime` module.

Examples

```

>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.isoweekday()
7

```

replace(self, year=None, month=None, day=None, hour=None, minute=None, second=None, microsecond=None, nanosecond=None, tzinfo=<class 'object'>, fold=None)
Implements `datetime.replace`, handles nanoseconds.

This method creates a new ``Timestamp`` object by replacing the specified fields with new values. The new ``Timestamp`` retains the original fields that are not explicitly replaced. This method handles nanoseconds, and the ``tzinfo`` parameter allows for timezone replacement without conversion.

Parameters

year : int, optional
The year to replace. If ``None``, the year is not changed.
month : int, optional
The month to replace. If ``None``, the month is not changed.
day : int, optional
The day to replace. If ``None``, the day is not changed.
hour : int, optional
The hour to replace. If ``None``, the hour is not changed.
minute : int, optional
The minute to replace. If ``None``, the minute is not changed.
second : int, optional
The second to replace. If ``None``, the second is not changed.
microsecond : int, optional
The microsecond to replace. If ``None``, the microsecond is not changed.

nanosecond : int, optional
The nanosecond to replace. If `None`, the nanosecond is not changed.
tzinfo : tz-convertible, optional
The timezone information to replace. If `None`, the timezone is not changed.
fold : int, optional
The fold information to replace. If `None`, the fold is not changed.

Returns

Timestamp

A new `Timestamp` object with the specified fields replaced.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Notes

The `replace` method does not perform timezone conversions. If you need to convert the timezone, use the `tz\_convert` method instead.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Replace year and the hour:

```
>>> ts.replace(year=1999, hour=10)
Timestamp('1999-03-14 10:32:52.192548651+0000', tz='UTC')
```

Replace timezone (not a conversion):

```
>>> import zoneinfo
>>> ts.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
Timestamp('2020-03-14 15:32:52.192548651-0700', tz='US/Pacific')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.replace(tzinfo=zoneinfo.ZoneInfo('US/Pacific'))
NaT
```

round(self, freq, ambiguous='raise', nonexistent='raise')

Round the Timestamp to the specified resolution.

This method rounds the given Timestamp down to a specified frequency level. It is particularly useful in data analysis to normalize timestamps to regular frequency intervals. For instance, rounding to the nearest minute, hour, or day can help in time series comparisons or resampling operations.

Parameters

freq : str

Frequency string indicating the rounding resolution.

ambiguous : bool or {'raise', 'NaT'}, default 'raise'

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : {'raise', 'shift_forward', 'shift_backward', 'NaT', timedelta}, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.

* 'raise' will raise a ValueError if there are nonexistent times.

Returns

a new Timestamp rounded to the given resolution of `freq`

Raises

ValueError if the freq cannot be converted

See Also

datetime.round : Similar behavior in native Python datetime module.
Timestamp.floor : Round the Timestamp downward to the nearest multiple of the specified frequency.
Timestamp.ceil : Round the Timestamp upward to the nearest multiple of the specified frequency.

Notes

If the Timestamp has a timezone, rounding will take place relative to the local ("wall") time and re-localized to the same timezone. When rounding near daylight savings time, use ``nonexistent`` and ``ambiguous`` to control the re-localization behavior.

Examples

Create a timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

A timestamp can be rounded using multiple frequency units:

```
>>> ts.round(freq='h') # hour
Timestamp('2020-03-14 16:00:00')
```

```
>>> ts.round(freq='min') # minute
Timestamp('2020-03-14 15:33:00')
```

```
>>> ts.round(freq='s') # seconds
Timestamp('2020-03-14 15:32:52')
```

```
>>> ts.round(freq='ms') # milliseconds
Timestamp('2020-03-14 15:32:52.193000')
```

``freq`` can also be a multiple of a single unit, like '5min' (i.e. 5 minutes):

```
>>> ts.round(freq='5min')
Timestamp('2020-03-14 15:35:00')
```

or a combination of multiple units, like '1h30min' (i.e. 1 hour and 30 minutes):

```
>>> ts.round(freq='1h30min')
Timestamp('2020-03-14 15:00:00')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.round()
NaT
```

When rounding near a daylight savings time transition, use ``ambiguous`` or ``nonexistent`` to control how the timestamp should be re-localized.

```
>>> ts_tz = pd.Timestamp("2021-10-31 01:30:00").tz_localize("Europe/Amsterdam")
```

```
>>> ts_tz.round("h", ambiguous=False)
Timestamp('2021-10-31 02:00:00+0100', tz='Europe/Amsterdam')
```

```
>>> ts_tz.round("h", ambiguous=True)
Timestamp('2021-10-31 02:00:00+0200', tz='Europe/Amsterdam')
```

strftime(self, format)
Return a formatted string of the Timestamp.

Parameters

```
format : str
    Format string to convert Timestamp to string.
    See strftime documentation for more information on the format string:
    https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior.
```

See Also

```
-----
Timestamp.isoformat : Return the time formatted according to ISO 8601.
pd.to_datetime : Convert argument to datetime.
Period.strftime : Format a single Period.
```

Examples

```
-----
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.strftime('%Y-%m-%d %X')
'2020-03-14 15:32:52'
```

time(self)

Return time object with same time but with tzinfo=None.

This method extracts the time part of the `Timestamp` object, excluding any timezone information. It returns a `datetime.time` object which only represents the time (hours, minutes, seconds, and microseconds).

See Also

```
-----
Timestamp.date : Return date object with same year, month and day.
Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.
Timestamp.tz_localize : Localize the Timestamp to a timezone.
```

Examples

```
-----
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.time()
datetime.time(10, 0)
```

timetuple(self)

Return time tuple, compatible with `time.localtime()`.

This method converts the `Timestamp` into a time tuple, which is compatible with functions like `time.localtime()`. The time tuple is a named tuple with attributes such as year, month, day, hour, minute, second, weekday, day of the year, and daylight savings indicator.

See Also

```
-----
time.localtime : Converts a POSIX timestamp into a time tuple.
Timestamp : The Timestamp that represents a specific point in time.
datetime.datetime.timetuple : Equivalent method in the datetime module.
```

Examples

```
-----
>>> ts = pd.Timestamp('2023-01-01 10:00:00')
>>> ts
Timestamp('2023-01-01 10:00:00')
>>> ts.timetuple()
time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1,
tm_hour=10, tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=-1)
```

timetz(self)

Return time object with same time and tzinfo.

This method returns a `datetime.time` object with the time and tzinfo corresponding to the `pd.Timestamp` object, ignoring any information about the day/date.

See Also

```
-----
datetime.datetime.timetz : Return datetime.time object with the
    same time attributes as the datetime object.
datetime.time : Class to represent the time of day, independent
    of any particular day.
datetime.datetime.tzinfo : Attribute of datetime.datetime objects
    representing the timezone of the datetime object.
```

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.tzinfo
datetime.time(10, 0, tzinfo=<DstTzInfo 'Europe/Brussels' CET+1:00:00 STD>)
```

`to_julian_date(self)` -> np.float64

Convert Timestamp to a Julian Date.

This method returns the number of days as a float since 0 Julian date, which is noon January 1, 4713 BC.

See Also

`Timestamp.toordinal` : Return proleptic Gregorian ordinal.

`Timestamp.timestamp` : Return POSIX timestamp as float.

`Timestamp` : Represents a single timestamp.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52')
>>> ts.to_julian_date()
2458923.147824074
```

`toordinal(self)`

Return proleptic Gregorian ordinal. January 1 of year 1 is day 1.

The proleptic Gregorian ordinal is a continuous count of days since January 1 of year 1, which is considered day 1. This method converts the `Timestamp` to its equivalent ordinal number, useful for date arithmetic and comparison operations.

See Also

`datetime.datetime.toordinal` : Equivalent method in the `datetime` module.

`Timestamp` : The `Timestamp` that represents a specific point in time.

`Timestamp.fromordinal` : Create a `Timestamp` from an ordinal.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:50')
>>> ts
Timestamp('2023-01-01 10:00:50')
>>> ts.toordinal()
738521
```

`tz_convert(self, tz)`

Convert timezone-aware Timestamp to another time zone.

This method is used to convert a timezone-aware Timestamp object to a different time zone. The original UTC time remains the same; only the time zone information is changed. If the Timestamp is timezone-naive, a `TypeError` is raised.

Parameters

`tz` : str, `zoneinfo.ZoneInfo`, `pytz.timezone`, `dateutil.tz.tzfile` or None

Time zone for time which Timestamp will be converted to.

None will remove timezone holding UTC time.

Returns

converted : Timestamp

Raises

`TypeError`

If Timestamp is tz-naive.

See Also

`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

`DatetimeIndex.tz_convert` : Convert a `DatetimeIndex` to another time zone.

`DatetimeIndex.tz_localize` : Localize a `DatetimeIndex` to a specific time zone.

`datetime.datetime.astimezone` : Convert a `datetime` object to another time zone.

Examples

Create a timestamp object with UTC timezone:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651', tz='UTC')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651+0000', tz='UTC')
```

Change to Tokyo timezone:

```
>>> ts.tz_convert(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Can also use ``astimezone``:

```
>>> ts.astimezone(tz='Asia/Tokyo')
Timestamp('2020-03-15 00:32:52.192548651+0900', tz='Asia/Tokyo')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_convert(tz='Asia/Tokyo')
NaT
```

```
tz_localize(self, tz, ambiguous='raise', nonexistent='raise')
    Localize the Timestamp to a timezone.
```

Convert naive Timestamp to local time zone or remove timezone from timezone-aware Timestamp.

Parameters

tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
Time zone for time which Timestamp will be converted to.
None will remove timezone holding local time.

ambiguous : bool, 'NaT', default 'raise'
When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the 'ambiguous' parameter dictates how ambiguous times should be handled.

The behavior is as follows:

- * bool contains flags to determine if time is dst or not (note that this flag is only applicable for ambiguous fall dst dates).
- * 'NaT' will return NaT for an ambiguous time.
- * 'raise' will raise a ValueError for an ambiguous time.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

The behavior is as follows:

- * 'shift_forward' will shift the nonexistent time forward to the closest existing time.
- * 'shift_backward' will shift the nonexistent time backward to the closest existing time.
- * 'NaT' will return NaT where there are nonexistent times.
- * timedelta objects will shift nonexistent times by the timedelta.
- * 'raise' will raise a ValueError if there are nonexistent times.

Returns

localized : Timestamp

Raises

TypeError

If the Timestamp is tz-aware and tz is not None.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.
Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.
DatetimeIndex.tz_localize : Localize a DatetimeIndex to a specific time zone.
datetime.datetime.astimezone : Convert a datetime object to another time zone.

Examples

Create a naive timestamp object:

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts
Timestamp('2020-03-14 15:32:52.192548651')
```

Add 'Europe/Stockholm' as timezone:

```
>>> ts.tz_localize(tz='Europe/Stockholm')
Timestamp('2020-03-14 15:32:52.192548651+0100', tz='Europe/Stockholm')
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.tz_localize()
NaT
```

tzname(self)
Return time zone name.

This method returns the name of the Timestamp's time zone as a string.

See Also

Timestamp.tzinfo : Returns the timezone information of the Timestamp.
Timestamp.tz_convert : Convert timezone-aware Timestamp to another time zone.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.tzname()
'CET'
```

utcoffset(self)
Return utc offset.

This method returns the difference between UTC and the local time as a `timedelta` object. It is useful for understanding the time difference between the current timezone and UTC.

Returns

timedelta
The difference between UTC and the local time as a `timedelta` object.

See Also

datetime.datetime.utcoffset :
Standard library method to get the UTC offset of a datetime object.
Timestamp.tzname : Return the name of the timezone.
Timestamp.dst : Return the daylight saving time (DST) adjustment.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.utcoffset()
datetime.timedelta(seconds=3600)
```

utctimetuple(self)
Return UTC time tuple, compatible with `time.localtime()`.

This method converts the Timestamp to UTC and returns a time tuple containing 9 components: year, month, day, hour, minute, second, weekday, day of year, and DST flag. This is particularly useful for converting a Timestamp to a format compatible with time module functions.

Returns

time.struct_time
A time.struct_time object representing the UTC time.

See Also

datetime.datetime.utctimetuple :
Return UTC time tuple, compatible with time.localtime().
Timestamp.timetuple : Return time tuple of local time.
time.struct_time : Time tuple structure used by time functions.

Examples

>>> ts = pd.Timestamp('2023-01-01 10:00:00', tz='Europe/Brussels')
>>> ts
Timestamp('2023-01-01 10:00:00+0100', tz='Europe/Brussels')
>>> ts.utctimetuple()
time.struct_time(tm_year=2023, tm_mon=1, tm_mday=1, tm_hour=9,
tm_min=0, tm_sec=0, tm_wday=6, tm_yday=1, tm_isdst=0)

weekday(self)

Return the day of the week represented by the date.

Monday == 0 ... Sunday == 6.

See Also

Timestamp.dayofweek : Return the day of the week with Monday=0, Sunday=6.
Timestamp.isoweekday : Return the day of the week with Monday=1, Sunday=7.
datetime.date.weekday : Equivalent method in datetime module.

Examples

>>> ts = pd.Timestamp('2023-01-01')
>>> ts
Timestamp('2023-01-01 00:00:00')
>>> ts.weekday()
6

Class methods defined here:

combine(date, time)

Timestamp.combine(date, time)

Combine a date and time into a single Timestamp object.

This method takes a `date` object and a `time` object
and combines them into a single `Timestamp`
that has the same date and time fields.

Parameters

date : datetime.date
The date part of the Timestamp.
time : datetime.time
The time part of the Timestamp.

Returns

Timestamp

A new `Timestamp` object representing the combined date and time.

See Also

Timestamp : Represents a single timestamp, similar to `datetime`.
to_datetime : Converts various types of data to datetime.

Examples

>>> from datetime import date, time
>>> pd.Timestamp.combine(date(2020, 3, 14), time(15, 30, 15))
Timestamp('2020-03-14 15:30:15')

fromordinal(ordinal, tz=None)

Construct a timestamp from a proleptic Gregorian ordinal.

This method creates a ``Timestamp`` object corresponding to the given proleptic Gregorian ordinal, which is a count of days from January 1, 0001 (using the proleptic Gregorian calendar). The time part of the ``Timestamp`` is set to midnight (00:00:00) by default.

Parameters

ordinal : int
 Date corresponding to a proleptic Gregorian ordinal.
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile or None
 Time zone for the Timestamp.

Returns

Timestamp
 A ``Timestamp`` object representing the specified ordinal date.

See Also

Timestamp : Represents a single timestamp, similar to ``datetime``.
to_datetime : Converts various types of data to datetime.

Notes

By definition there cannot be any tz info on the ordinal itself.

Examples

Convert an ordinal to a ``Timestamp``:

```
>>> pd.Timestamp.fromordinal(737425)
Timestamp('2020-01-01 00:00:00')
```

Create a ``Timestamp`` from an ordinal with timezone information:

```
>>> pd.Timestamp.fromordinal(737425, tz='UTC')
Timestamp('2020-01-01 00:00:00+0000', tz='UTC')
```

fromtimestamp(ts, tz=None)

Create a ``Timestamp`` object from a POSIX timestamp.

This method converts a POSIX timestamp (the number of seconds since January 1, 1970, 00:00:00 UTC) into a ``Timestamp`` object. The resulting ``Timestamp`` can be localized to a specific time zone if provided.

Parameters

ts : float
 The POSIX timestamp to convert, representing seconds since the epoch (1970-01-01 00:00:00 UTC).
tz : str, zoneinfo.ZoneInfo, pytz.timezone, dateutil.tz.tzfile, optional
 Time zone for the ``Timestamp``. If not provided, the ``Timestamp`` will be timezone-naive (i.e., without time zone information).

Returns

Timestamp
 A ``Timestamp`` object representing the given POSIX timestamp.

See Also

Timestamp : Represents a single timestamp, similar to ``datetime``.
to_datetime : Converts various types of data to datetime.
datetime.datetime.fromtimestamp : Returns a datetime from a POSIX timestamp.

Examples

Convert a POSIX timestamp to a ``Timestamp``:

```
>>> pd.Timestamp.fromtimestamp(1584199972) # doctest: +SKIP
Timestamp('2020-03-14 15:32:52')
```

Note that the output may change depending on your local time and time zone:

```
>>> pd.Timestamp.fromtimestamp(1584199972, tz='UTC') # doctest: +SKIP
Timestamp('2020-03-14 15:32:52+0000', tz='UTC')
```

```

now(tz=None)
    Return new Timestamp object representing current time local to tz.

    This method returns a new `Timestamp` object that represents the current time.
    If a timezone is provided, either through a timezone object or an IANA
    standard timezone identifier, the current time will be localized to that
    timezone.
    Otherwise, it returns the current local time.

    Parameters
    -----
    tz : str or timezone object, default None
        Timezone to localize to.

    See Also
    -----
    to_datetime : Convert argument to datetime.
    Timestamp.utcnow : Return a new Timestamp representing UTC day and time.
    Timestamp.today : Return the current time in the local timezone.

    Examples
    -----
    >>> pd.Timestamp.now() # doctest: +SKIP
    Timestamp('2020-11-16 22:06:16.378782')

    If you want a specific timezone, in this case 'Brazil/East':

    >>> pd.Timestamp.now('Brazil/East') # doctest: +SKIP
    Timestamp('2025-11-11 22:17:59.609943-03:00')

    Analogous for ``pd.NaT``:

    >>> pd.NaT.now()
    NaT

strptime(date_string, format)
    Convert string argument to datetime.

    This method is not implemented; calling it will raise NotImplementedError.
    Use pd.to_datetime() instead.

    Parameters
    -----
    date_string : str
        String to convert to a datetime.
    format : str, default None
        The format string to parse time, e.g. "%d/%m/%Y".

    See Also
    -----
    pd.to_datetime : Convert argument to datetime.
    datetime.datetime.strptime : Return a datetime corresponding to a string
        representing a date and time, parsed according to a separate
        format string.
    datetime.datetime.strftime : Return a string representing the date and
        time, controlled by an explicit format string.
    Timestamp.isoformat : Return the time formatted according to ISO 8601.

    Examples
    -----
    >>> pd.Timestamp.strptime("2023-01-01", "%d/%m/%Y")
    Traceback (most recent call last):
    NotImplementedError

today(tz=None)
    Return the current time in the local timezone.

    This differs from datetime.today() in that it can be localized to a
    passed timezone.

    Parameters
    -----
    tz : str or timezone object, default None
        Timezone to localize to.

    See Also
    -----

```

`datetime.datetime.today` : Returns the current local date.
`Timestamp.now` : Returns current time with optional timezone.
`Timestamp` : A class representing a specific timestamp.

Examples

```
>>> pd.Timestamp.today() # doctest: +SKIP
Timestamp('2020-11-16 22:37:39.969883')
```

Analogous for `pd.NaT`:

```
>>> pd.NaT.today()
NaT
```

`utcfromtimestamp(ts)`
`Timestamp.utcfromtimestamp(ts)`

Construct a timezone-aware UTC `datetime` from a POSIX timestamp.

This method creates a `datetime` object from a POSIX timestamp, keeping the `Timestamp` object's timezone.

Parameters

`ts` : float
POSIX timestamp.

See Also

`Timezone.tzname` : Return time zone name.
`Timestamp.utcnow` : Return a new `Timestamp` representing UTC day and time.
`Timestamp.fromtimestamp` : Transform `timestamp[, tz]` to `tz`'s local time from POSIX timestamp.

Notes

`Timestamp.utcfromtimestamp` behavior differs from `datetime.utcfromtimestamp` in returning a timezone-aware object.

Examples

```
>>> pd.Timestamp.utcfromtimestamp(1584199972)
Timestamp('2020-03-14 15:32:52+0000', tz='UTC')
```

`utcnow()`
`Timestamp.utcnow()`

Return a new `Timestamp` representing UTC day and time.

See Also

`Timestamp` : Constructs an arbitrary `datetime`.
`Timestamp.now` : Return the current local date and time, which can be timezone-aware.
`Timestamp.today` : Return the current local date and time with timezone information set to `None`.
`to_datetime` : Convert argument to `datetime`.
`date_range` : Return a fixed frequency `DatetimeIndex`.
`Timestamp.utctimetuple` : Return UTC time tuple, compatible with `time.localtime()`.

Examples

```
>>> pd.Timestamp.utcnow() # doctest: +SKIP
Timestamp('2020-11-16 22:50:18.092888+0000', tz='UTC')
```

Static methods defined here:

```
__new__(
    cls,
    ts_input=<object object at 0x000001A9511B16D0>,
    year=None,
    month=None,
    day=None,
    hour=None,
    minute=None,
```

```

        second=None,
        microsecond=None,
        tzinfo=None,
        *,
        nanosecond=None,
        tz=<object object at 0x000001A9511B16D0>,
        unit=None,
        fold=None
    )

```

Readonly properties defined here:

tzinfo

Returns the timezone info of the Timestamp.

This property returns a ``datetime.tzinfo`` object if the Timestamp is timezone-aware. If the Timestamp has no timezone, it returns ``None``. If the Timestamp is in UTC or a fixed-offset timezone, it returns ``datetime.timezone``. If the Timestamp uses an IANA timezone (e.g., "America/New_York"), it returns ``zoneinfo.ZoneInfo``.

See Also

`Timestamp.tz` : Alias for ``tzinfo``, may return a ``zoneinfo.ZoneInfo`` object.
`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.
`Timestamp.tz_localize` : Localize the Timestamp to a specific timezone.

Examples

```

>>> ts = pd.Timestamp("2023-01-01 12:00:00", tz="UTC")
>>> ts.tzinfo
datetime.timezone.utc

```

```

>>> ts_naive = pd.Timestamp("2023-01-01 12:00:00")
>>> ts_naive.tzinfo

```

Data descriptors defined here:

`__dict__`

dictionary for instance variables

`__weakref__`

list of weak references to the object

daysinmonth

Return the number of days in the month.

Returns

int

See Also

`Timestamp.month_name` : Return the month name of the Timestamp with specified locale.

Examples

```

>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.days_in_month
31

```

tz

Alias for `tzinfo`.

The ``tz`` property provides a simple and direct way to retrieve the timezone information of a ``Timestamp`` object. It is particularly useful when working with time series data that includes timezone information, allowing for easy access and manipulation of the timezone context.

See Also

`Timestamp.tzinfo` : Returns the timezone information of the Timestamp.
`Timestamp.tz_convert` : Convert timezone-aware Timestamp to another time zone.
`Timestamp.tz_localize` : Localize the Timestamp to a timezone.

Examples

```
>>> ts = pd.Timestamp(1584226800, unit='s', tz='Europe/Stockholm')
>>> ts.tz
zoneinfo.ZoneInfo(key='Europe/Stockholm')
```

weekofyear

Return the week number of the year.

Returns

int

See Also

Timestamp.weekday : Return the day of the week.

Timestamp.quarter : Return the quarter of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.week
11
```

Methods inherited from pandas._libs.tslibs.timestamps._Timestamp:

__add__(self, object, /)
Return self+value.

__eq__(self, value, /)
Return self==value.

__ge__(self, value, /)
Return self>=value.

__gt__(self, value, /)
Return self>value.

__hash__(self, /)
Return hash(self).

__le__(self, value, /)
Return self<=value.

__lt__(self, value, /)
Return self<value.

__ne__(self, value, /)
Return self!=value.

__radd__(self, object, /)
Return value+self.

__reduce__(self)

__reduce_ex__(self, protocol)

__repr__(self, /)
Return repr(self).

__rsub__(self, object, /)
Return value-self.

__setstate__(self, state)

__sub__(self, object, /)
Return self-value.

as_unit(self, unit, round_ok=True)
Convert the underlying int64 representation to the given unit.

Parameters

unit : {"ns", "us", "ms", "s"}
round_ok : bool, default True

If False and the conversion requires rounding, raise.

Returns

Timestamp

See Also

Timestamp.asm8 : Return numpy datetime64 format with same precision.

Timestamp.to_pydatetime : Convert Timestamp object to a native Python datetime object.

to_timedelta : Convert argument into timedelta object, which can represent differences in times.

Examples

```
>>> ts = pd.Timestamp('2023-01-01 00:00:00.01')
```

```
>>> ts
```

```
Timestamp('2023-01-01 00:00:00.010000')
```

```
>>> ts.unit
```

```
'ms'
```

```
>>> ts = ts.as_unit('s')
```

```
>>> ts
```

```
Timestamp('2023-01-01 00:00:00')
```

```
>>> ts.unit
```

```
's'
```

day_name(self, locale=None) -> str

Return the day name of the Timestamp with specified locale.

Parameters

locale : str, default None (English locale)

Locale determining the language in which to return the day name.

Returns

str

See Also

Timestamp.day_of_week : Return day of the week.

Timestamp.day_of_year : Return day of the year.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

```
>>> ts.day_name()
```

```
'Saturday'
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.day_name()
```

```
nan
```

isoformat(self, sep: str = 'T', timespec: str = 'auto') -> str

Return the time formatted according to ISO 8601.

The full format looks like 'YYYY-MM-DD HH:MM:SS.mmmmmnnn'.

By default, the fractional part is omitted if self.microsecond == 0 and self._nanosecond == 0.

If self.tzinfo is not None, the UTC offset is also attached, giving a full format of 'YYYY-MM-DD HH:MM:SS.mmmmmnnn+HH:MM'.

Parameters

sep : str, default 'T'

String used as the separator between the date and time.

timespec : str, default 'auto'

Specifies the number of additional terms of the time to include.

The valid values are 'auto', 'hours', 'minutes', 'seconds', 'milliseconds', 'microseconds', and 'nanoseconds'.

Returns

str

See Also

Timestamp.strftime : Return a formatted string.

Timestamp.isocalendar : Return a tuple containing ISO year, week number and weekday.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.isoformat()
'2020-03-14T15:32:52.192548651'
>>> ts.isoformat(timespec='microseconds')
'2020-03-14T15:32:52.192548'
```

month_name(self, locale=None) -> str

Return the month name of the Timestamp with specified locale.

This method returns the full name of the month corresponding to the `Timestamp`, such as 'January', 'February', etc. The month name can be returned in a specified locale if provided; otherwise, it defaults to the English locale.

Parameters

locale : str, default None (English locale)

Locale determining the language in which to return the month name.

Returns

str

The full month name as a string.

See Also

Timestamp.day_name : Returns the name of the day of the week.

Timestamp.strftime : Returns a formatted string of the Timestamp.

datetime.datetime.strftime : Returns a string representing the date and time.

Examples

Get the month name in English (default):

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
>>> ts.month_name()
'March'
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.month_name()
nan
```

normalize(self) -> 'Timestamp'

Normalize Timestamp to midnight, preserving tz information.

This method sets the time component of the `Timestamp` to midnight (00:00:00), while preserving the date and time zone information. It is useful when you need to standardize the time across different `Timestamp` objects without altering the time zone or the date.

Returns

Timestamp

See Also

Timestamp.floor : Rounds `Timestamp` down to the nearest frequency.

Timestamp.ceil : Rounds `Timestamp` up to the nearest frequency.

Timestamp.round : Rounds `Timestamp` to the nearest frequency.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14, 15, 30)
>>> ts.normalize()
Timestamp('2020-03-14 00:00:00')
```

```

timestamp(self)
    Return POSIX timestamp as float.

    This method converts the `Timestamp` object to a POSIX timestamp, which is
    the number of seconds since the Unix epoch (January 1, 1970). The returned
    value is a floating-point number, where the integer part represents the
    seconds, and the fractional part represents the microseconds.

    Returns
    -----
    float
        The POSIX timestamp representation of the `Timestamp` object.

    See Also
    -----
    Timestamp.fromtimestamp : Construct a `Timestamp` from a POSIX timestamp.
    datetime.datetime.timestamp : Equivalent method from the `datetime` module.
    Timestamp.to_pydatetime : Convert the `Timestamp` to a `datetime` object.
    Timestamp.to_datetime64 : Converts `Timestamp` to `numpy.datetime64`.

    Examples
    -----
    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
    >>> ts.timestamp()
    1584199972.192548

to_datetime64(self)
    Return a NumPy datetime64 object with same precision.

    This method returns a numpy.datetime64 object with the same
    date and time information and precision as the pd.Timestamp object.

    See Also
    -----
    numpy.datetime64 : Class to represent dates and times with high precision.
    Timestamp.to_numpy : Alias for this method.
    Timestamp.asm8 : Alias for this method.
    pd.to_datetime : Convert argument to datetime.

    Examples
    -----
    >>> ts = pd.Timestamp(year=2023, month=1, day=1,
    ...                   hour=10, second=15)
    >>> ts
    Timestamp('2023-01-01 10:00:15')
    >>> ts.to_datetime64()
    numpy.datetime64('2023-01-01T10:00:15.000000')

to_numpy(self, dtype=None, copy=False) -> np.datetime64
    Convert the Timestamp to a NumPy datetime64.

    This is an alias method for `Timestamp.to_datetime64()`. The dtype and
    copy parameters are available here only for compatibility. Their values
    will not affect the return value.

    Parameters
    -----
    dtype : dtype, optional
        Data type of the output, ignored in this method as the return type
        is always `numpy.datetime64`.
    copy : bool, default False
        Whether to ensure that the returned value is a new object. This
        parameter is also ignored as the method does not support copying.

    Returns
    -----
    numpy.datetime64

    See Also
    -----
    DatetimeIndex.to_numpy : Similar method for DatetimeIndex.

    Examples
    -----
    >>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
    >>> ts.to_numpy()
    numpy.datetime64('2020-03-14T15:32:52.192548651')

```


Analogous for ``pd.NaT``:

```
>>> pd.NaT.to_numpy()  
numpy.datetime64('NaT')
```

`to_period(self, freq=None)`

Return a period of which this timestamp is an observation.

This method converts the given Timestamp to a Period object, which represents a span of time, such as a year, month, etc., based on the specified frequency.

Parameters

`freq` : str, optional

Frequency string for the period (e.g., 'Y', 'M', 'W'). Defaults to 'None'.

See Also

`Timestamp` : Represents a specific timestamp.

`Period` : Represents a span of time.

`to_period` : Converts an object to a Period.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548651')
```

```
>>> # Year end frequency
```

```
>>> ts.to_period(freq='Y')
```

```
Period('2020', 'Y-DEC')
```

```
>>> # Month end frequency
```

```
>>> ts.to_period(freq='M')
```

```
Period('2020-03', 'M')
```

```
>>> # Weekly frequency
```

```
>>> ts.to_period(freq='W')
```

```
Period('2020-03-09/2020-03-15', 'W-SUN')
```

```
>>> # Quarter end frequency
```

```
>>> ts.to_period(freq='Q')
```

```
Period('2020Q1', 'Q-DEC')
```

`to_pydatetime(self, warn=True)`

Convert a Timestamp object to a native Python datetime object.

This method is useful for when you need to utilize a pandas Timestamp object in contexts where native Python datetime objects are expected or required. The conversion discards the nanoseconds component, and a warning can be issued in such cases if desired.

Parameters

`warn` : bool, default True

If True, issues a warning when the timestamp includes nonzero nanoseconds, as these will be discarded during the conversion.

Returns

`datetime.datetime` or `NaT`

Returns a `datetime.datetime` object representing the timestamp, with year, month, day, hour, minute, second, and microsecond components. If the timestamp is `NaT` (Not a Time), returns `NaT`.

See Also

`datetime.datetime` : The standard Python datetime class that this method returns.

`Timestamp.timestamp` : Convert a Timestamp object to POSIX timestamp.

`Timestamp.to_datetime64` : Convert a Timestamp object to `numpy.datetime64`.

Examples

```
>>> ts = pd.Timestamp('2020-03-14T15:32:52.192548')
```

```
>>> ts.to_pydatetime()
```

```
datetime.datetime(2020, 3, 14, 15, 32, 52, 192548)
```

Analogous for ``pd.NaT``:

```
>>> pd.NaT.to_pydatetime()
NaT
```

Data descriptors inherited from pandas._libs.tslibs.timestamps._Timestamp:

asm8

Return numpy datetime64 format with same precision.

See Also

numpy.datetime64 : Numpy datatype for dates and times with high precision.

Timestamp.to_numpy : Convert the Timestamp to a NumPy datetime64.

to_datetime : Convert argument to datetime.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14, 15)
```

```
>>> ts.asm8
```

```
numpy.datetime64('2020-03-14T15:00:00.000000')
```

day

Return the day of the Timestamp.

Returns

int

The day of the Timestamp.

See Also

Timestamp.week : Return the week number of the year.

Timestamp.weekday : Return the day of the week.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.day
```

```
31
```

day_of_week

Return day of the week.

Returns

int

See Also

Timestamp.isoweekday : Return the ISO day of the week represented by the date.

Timestamp.weekday : Return the day of the week represented by the date.

Timestamp.day_of_year : Return day of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_week
```

```
5
```

day_of_year

Return the day of the year.

Returns

int

See Also

Timestamp.day_of_week : Return day of the week.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.day_of_year
```

```
74
```

```

dayofweek
    Return day of the week.

    Returns
    -----
    int

    See Also
    -----
    Timestamp.isoweekday : Return the ISO day of the week represented by the date.
    Timestamp.weekday : Return the day of the week represented by the date.
    Timestamp.day_of_year : Return day of the year.

    Examples
    -----
    >>> ts = pd.Timestamp(2020, 3, 14)
    >>> ts.day_of_week
    5

dayofyear
    Return the day of the year.

    Returns
    -----
    int

    See Also
    -----
    Timestamp.day_of_week : Return day of the week.

    Examples
    -----
    >>> ts = pd.Timestamp(2020, 3, 14)
    >>> ts.day_of_year
    74

days_in_month
    Return the number of days in the month.

    Returns
    -----
    int

    See Also
    -----
    Timestamp.month_name : Return the month name of the Timestamp with
        specified locale.

    Examples
    -----
    >>> ts = pd.Timestamp(2020, 3, 14)
    >>> ts.days_in_month
    31

fold
    Return the fold value of the Timestamp.

    Returns
    -----
    int
    The fold value of the Timestamp, where 0 indicates the first occurrence
    of the ambiguous time, and 1 indicates the second.

    See Also
    -----
    Timestamp.dst : Return the daylight saving time (DST) adjustment.
    Timestamp.tzinfo : Return the timezone information associated.

    Examples
    -----
    >>> ts = pd.Timestamp("2024-11-03 01:30:00")
    >>> ts.fold
    0

hour
    Return the hour of the Timestamp.

```

Returns

int

The hour of the Timestamp.

See Also

Timestamp.minute : Return the minute of the Timestamp.

Timestamp.second : Return the second of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.hour
```

```
16
```

is_leap_year

Return True if year is a leap year.

A leap year is a year, which has 366 days (instead of 365) including 29th of February as an intercalary day. Leap years are years which are multiples of four with the exception of years divisible by 100 but not by 400.

Returns

bool

True if year is a leap year, else False

See Also

Period.is_leap_year : Return True if the period's year is in a leap year.

DatetimeIndex.is_leap_year : Boolean indicator if the date belongs to a leap year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.is_leap_year
```

```
True
```

is_month_end

Check if the date is the last day of the month.

Returns

bool

True if the date is the last day of the month.

See Also

Timestamp.is_month_start : Similar property indicating month start.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.is_month_end
```

```
False
```

```
>>> ts = pd.Timestamp(2020, 12, 31)
```

```
>>> ts.is_month_end
```

```
True
```

is_month_start

Check if the date is the first day of the month.

Returns

bool

True if the date is the first day of the month.

See Also

Timestamp.is_month_end : Similar property indicating the last day of the month.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_month_start
False
```

```
>>> ts = pd.Timestamp(2020, 1, 1)
>>> ts.is_month_start
True
```

is_quarter_end

Check if date is last day of the quarter.

Returns

bool

True if date is last day of the quarter.

See Also

Timestamp.is_quarter_start : Similar property indicating the quarter start.

Timestamp.quarter : Return the quarter of the date.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_quarter_end
False
```

```
>>> ts = pd.Timestamp(2020, 3, 31)
>>> ts.is_quarter_end
True
```

is_quarter_start

Check if the date is the first day of the quarter.

Returns

bool

True if date is first day of the quarter.

See Also

Timestamp.is_quarter_end : Similar property indicating the quarter end.

Timestamp.quarter : Return the quarter of the date.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_quarter_start
False
```

```
>>> ts = pd.Timestamp(2020, 4, 1)
>>> ts.is_quarter_start
True
```

is_year_end

Return True if date is last day of the year.

Returns

bool

See Also

Timestamp.is_year_start : Similar property indicating the start of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.is_year_end
False
```

```
>>> ts = pd.Timestamp(2020, 12, 31)
>>> ts.is_year_end
True
```

is_year_start

Return True if date is first day of the year.

Returns

bool

See Also

Timestamp.is_year_end : Similar property indicating the end of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.is_year_start
```

```
False
```

```
>>> ts = pd.Timestamp(2020, 1, 1)
```

```
>>> ts.is_year_start
```

```
True
```

microsecond

Return the microsecond of the Timestamp.

Returns

int

The microsecond of the Timestamp.

See Also

Timestamp.second : Return the second of the Timestamp.

Timestamp.minute : Return the minute of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30.2304")
```

```
>>> ts.microsecond
```

```
230400
```

minute

Return the minute of the Timestamp.

Returns

int

The minute of the Timestamp.

See Also

Timestamp.hour : Return the hour of the Timestamp.

Timestamp.second : Return the second of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.minute
```

```
16
```

month

Return the month of the Timestamp.

Returns

int

The month of the Timestamp.

See Also

Timestamp.day : Return the day of the Timestamp.

Timestamp.year : Return the year of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.month
```

```
8
```

nanosecond

Return the nanosecond of the Timestamp.

Returns

int

The nanosecond of the Timestamp.

See Also

Timestamp.second : Return the second of the Timestamp.

Timestamp.microsecond : Return the microsecond of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30.230400015")
```

```
>>> ts.nanosecond
```

```
15
```

quarter

Return the quarter of the year for the `Timestamp`.

This property returns an integer representing the quarter of the year in which the `Timestamp` falls. The quarters are defined as follows:

- Q1: January 1 to March 31
- Q2: April 1 to June 30
- Q3: July 1 to September 30
- Q4: October 1 to December 31

Returns

int

The quarter of the year (1 through 4).

See Also

Timestamp.month : Returns the month of the `Timestamp`.

Timestamp.year : Returns the year of the `Timestamp`.

Examples

Get the quarter for a `Timestamp`:

```
>>> ts = pd.Timestamp(2020, 3, 14)
```

```
>>> ts.quarter
```

```
1
```

For a `Timestamp` in the fourth quarter:

```
>>> ts = pd.Timestamp(2020, 10, 14)
```

```
>>> ts.quarter
```

```
4
```

second

Return the second of the Timestamp.

Returns

int

The second of the Timestamp.

See Also

Timestamp.microsecond : Return the microsecond of the Timestamp.

Timestamp.minute : Return the minute of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
```

```
>>> ts.second
```

```
30
```

unit

The abbreviation associated with self._creso.

This property returns a string representing the time unit of the Timestamp's resolution. It corresponds to the smallest time unit that can be represented by this Timestamp object. The possible values are:

```
- 's' (second)
- 'ms' (millisecond)
- 'us' (microsecond)
- 'ns' (nanosecond)
```

Returns

str

A string abbreviation of the Timestamp's resolution unit:

```
- 's' for second
- 'ms' for millisecond
- 'us' for microsecond
- 'ns' for nanosecond
```

See Also

Timestamp.resolution : Return resolution of the Timestamp.

Timedelta : A duration expressing the difference between two dates or times.

Examples

```
>>> pd.Timestamp("2020-01-01 12:34:56").unit
's'
```

```
>>> pd.Timestamp("2020-01-01 12:34:56.123").unit
'ms'
```

```
>>> pd.Timestamp("2020-01-01 12:34:56.123456").unit
'us'
```

```
>>> pd.Timestamp("2020-01-01 12:34:56.123456789").unit
'ns'
```

value

Return the value of the Timestamp.

Returns

int

The integer representation of the Timestamp object in nanoseconds since the Unix epoch (1970-01-01 00:00:00 UTC).

See Also

Timestamp.second : Return the second of the Timestamp.

Timestamp.minute : Return the minute of the Timestamp.

Examples

```
>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.value
1725120990000000000
```

week

Return the week number of the year.

Returns

int

See Also

Timestamp.weekday : Return the day of the week.

Timestamp.quarter : Return the quarter of the year.

Examples

```
>>> ts = pd.Timestamp(2020, 3, 14)
>>> ts.week
11
```

year

Return the year of the Timestamp.

Returns

int

The year of the Timestamp.

See Also

Timestamp.month : Return the month of the Timestamp.
Timestamp.day : Return the day of the Timestamp.

Examples

>>> ts = pd.Timestamp("2024-08-31 16:16:30")
>>> ts.year
2024

Data and other attributes inherited from pandas._libs.tslibs.timestamps._Timestamp:

__array_priority__ = 100

__pyx_vtable__ = <capsule object NULL>

max = Timestamp('2262-04-11 23:47:16.854775807')
Returns the maximum bound possible for Timestamp.

This property provides access to the largest possible value that can be represented by a Timestamp object.

Returns

Timestamp

See Also

Timestamp.min: Returns the minimum bound possible for Timestamp.
Timestamp.resolution: Returns the smallest possible difference between non-equal Timestamp objects.

Examples

>>> pd.Timestamp.max
Timestamp('2262-04-11 23:47:16.854775807')

min = Timestamp('1677-09-21 00:12:43.145224193')
Returns the minimum bound possible for Timestamp.

This property provides access to the smallest possible value that can be represented by a Timestamp object.

Returns

Timestamp

See Also

Timestamp.max: Returns the maximum bound possible for Timestamp.
Timestamp.resolution: Returns the smallest possible difference between non-equal Timestamp objects.

Examples

>>> pd.Timestamp.min
Timestamp('1677-09-21 00:12:43.145224193')

resolution = Timedelta('0 days 00:00:00.000000001')
Returns the smallest possible difference between non-equal Timestamp objects.

The resolution value is determined by the underlying representation of time units and is equivalent to Timedelta(nanoseconds=1).

Returns

Timedelta

See Also

Timestamp.max: Returns the maximum bound possible for Timestamp.

Timestamp.min: Returns the minimum bound possible for Timestamp.

Examples

```
>>> pd.Timestamp.resolution
Timedelta('0 days 00:00:00.000000001')
```

Methods inherited from pandas._libs.tslibs.base.ABCTimestamp:

__reduce_cython__(self)

__setstate_cython__(self, __pyx_state)

Methods inherited from datetime.datetime:

__replace__(self, /, **changes)
The same as replace().

__str__(self, /)
Return str(self).

Class methods inherited from datetime.datetime:

fromisoformat(object, /)
string -> datetime from a string in most ISO 8601 formats

Methods inherited from datetime.date:

__format__(...)
Formats self with strftime.

Class methods inherited from datetime.date:

fromisocalendar(...)
int, int, int -> Construct a date from the ISO year, week number and weekday.

This is the inverse of the date.isocalendar() function

DATA

```
NaT = NaT
__all__ = ['Interval', 'NaT', 'NaTType', 'OutOfBoundsDatetime', 'Perio...
iNaT = -9223372036854775808
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas_libs__init__.

Help on package pandas._testing in pandas:

NAME

pandas._testing

PACKAGE CONTENTS

```
_hypothesis
_io
_warnings
asserters
compat
contexts
```

CLASSES

```
pandas.DataFrame(pandas.core.generic.NDFrame, pandas.core.arraylike.OpsMixin)
  SubclassedDataFrame
pandas.Series(pandas.core.base.IndexOpsMixin, pandas.core.generic.NDFrame)
  SubclassedSeries
```

```
class SubclassedDataFrame(pandas.DataFrame)
|   SubclassedDataFrame(
|       data=None,
|       index: Axes | None = None,
|       columns: Axes | None = None,
```

```
dtype: Dtype | None = None,  
copy: bool | None = None  
) -> None
```

Method resolution order:

```
SubclassedDataFrame  
pandas.DataFrame  
pandas.core.generic.NDFrame  
pandas.core.base.PandasObject  
pandas.core.accessor.DirNamesMixin  
pandas.core.indexing.IndexingMixin  
pandas.core.arraylike.OpsMixin  
builtins.object
```

Methods inherited from pandas.DataFrame:

```
__arrow_c_stream__(self, requested_schema=None) from pandas.core.frame.DataFrame  
Export the pandas DataFrame as an Arrow C stream PyCapsule.
```

This relies on pyarrow to convert the pandas DataFrame to the Arrow format (and follows the default behaviour of ``pyarrow.Table.from\_pandas`` in its handling of the index, i.e. store the index as a column except for RangeIndex).
This conversion is not necessarily zero-copy.

Parameters

requested_schema : PyCapsule, default None
The schema to which the dataframe should be casted, passed as a PyCapsule containing a C ArrowSchema representation of the requested schema.

Returns

PyCapsule

```
__dataframe__(self, nan_as_null: bool = False, allow_copy: bool = True) -> DataFrameXchg from  
Return the dataframe interchange object implementing the interchange protocol.
```

.. deprecated:: 3.0.0

The Dataframe Interchange Protocol is deprecated.
For dataframe-agnostic code, you may want to look into:

- `Arrow PyCapsule Interface <<https://arrow.apache.org/docs/format/CDataInterface/Py>>
- `Narwhals <<https://github.com/narwhals-dev/narwhals>>`

.. note::

For new development, we highly recommend using the Arrow C Data Interface alongside the Arrow PyCapsule Interface instead of the interchange protocol

.. warning::

Due to severe implementation issues, we recommend only considering using the interchange protocol in the following cases:

- converting to pandas: for pandas >= 2.0.3
- converting from pandas: for pandas >= 3.0.0

Parameters

nan_as_null : bool, default False
`nan\_as\_null` is DEPRECATED and has no effect. Please avoid using it; it will be removed in a future release.
allow_copy : bool, default True
Whether to allow memory copying when exporting. If set to False it would cause non-zero-copy exports to fail.

Returns

DataFrame interchange object
The object which consuming library can use to ingress the dataframe.

See Also

DataFrame.from_records : Constructor from tuples, also record arrays.

`DataFrame.from_dict` : From dicts of Series, arrays, or dicts.

Notes

Details on the interchange protocol:
<https://data-apis.org/dataframe-protocol/latest/index.html>

Examples

>>> df_not_necessarily_pandas = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> interchange_object = df_not_necessarily_pandas.__dataframe__()
>>> interchange_object.column_names()
Index(['A', 'B'], dtype='str')
>>> df_pandas = pd.api.interchange.from_dataframe(
... interchange_object.select_columns_by_name(["A"])
...)

>>> df_pandas

```
      A
0     1
1     2
```

These methods (`__column_names__`, `__select_columns_by_name__`) should work for any dataframe library which implements the interchange protocol.

`__divmod__(self, other)` -> tuple[Dataframe, Dataframe] from pandas.core.frame.DataFrame

`__getitem__(self, key)` from pandas.core.frame.DataFrame

`__init__(
 self,
 data=None,
 index: Axes | None = None,
 columns: Axes | None = None,
 dtype: Dtype | None = None,
 copy: bool | None = None
)` -> None from pandas.core.frame.DataFrame
Initialize self. See help(type(self)) for accurate signature.

`__len__(self)` -> int from pandas.core.frame.DataFrame
Returns length of info axis, but here we use the index.

`__matmul__(self, other: AnyArrayLike | Dataframe)` -> Dataframe | Series from pandas.core.frame.DataFrame
Matrix multiplication using binary `@` operator.

`__rdivmod__(self, other)` -> tuple[Dataframe, Dataframe] from pandas.core.frame.DataFrame

`__repr__(self)` -> str from pandas.core.frame.DataFrame
Return a string representation for a particular Dataframe.

`__rmatmul__(self, other)` -> Dataframe from pandas.core.frame.DataFrame
Matrix multiplication using binary `@` operator.

`__setitem__(self, key, value)` -> None from pandas.core.frame.DataFrame
Set item(s) in Dataframe by key.

This method allows you to set the values of one or more columns in the Dataframe using a key. If the key does not exist, a new column will be created.

Parameters

key : The object(s) in the index which are to be assigned to
 Column label(s) to set. Can be a single column name, list of column names,
 or tuple for MultiIndex columns.
value : scalar, array-like, Series, or Dataframe
 Value(s) to set for the specified key(s).

Returns

None
This method does not return a value.

See Also

`Dataframe.loc` : Access and set values by label-based indexing.
`Dataframe.iloc` : Access and set values by position-based indexing.
`Dataframe.assign` : Assign new columns to a Dataframe.

Notes

When assigning a Series to a DataFrame column, pandas aligns the Series by index labels, not by position. This means:

- * Values from the Series are matched to DataFrame rows by index label
- * If a Series index label doesn't exist in the DataFrame index, it's ignored
- * If a DataFrame index label doesn't exist in the Series index, NaN is assigned
- * The order of values in the Series doesn't matter; only the index labels matter

Examples

Basic column assignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]})
>>> df["B"] = [4, 5, 6] # Assigns by position
>>> df
   A  B
0  1  4
1  2  5
2  3  6
```

Series assignment with index alignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
>>> s = pd.Series([10, 20], index=[1, 3]) # Note: index 3 doesn't exist in df
>>> df["B"] = s # Assigns by index label, not position
>>> df
   A  B
0  1  NaN
1  2  10.0
2  3  NaN
```

Series assignment with partial index match:

```
>>> df = pd.DataFrame({"A": [1, 2, 3, 4]}, index=["a", "b", "c", "d"])
>>> s = pd.Series([100, 200], index=["b", "d"])
>>> df["B"] = s
>>> df
   A  B
a  1  NaN
b  2  100.0
c  3  NaN
d  4  200.0
```

Series index labels NOT in DataFrame, ignored:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=["x", "y", "z"])
>>> s = pd.Series([10, 20, 30, 40, 50], index=["x", "y", "a", "b", "z"])
>>> df["B"] = s
>>> df
   A  B
x  1  10
y  2  20
z  3  50
```

`add(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas`
Get Addition of dataframe and other, element-wise (binary operator ``add``).

Equivalent to ``dataframe + other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``radd``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None

Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation. If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178

```
rectangle      3      358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...         axis='index')
      angles  degrees
circle     -1     359
triangle     2     179
rectangle     3     359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle       0     720
triangle     0     360
rectangle     0     720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle       0         0
triangle      6     360
rectangle     12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle       0
triangle      3
rectangle     4
```

```
>>> df * other
      angles  degrees
circle       0      NaN
triangle      9      NaN
rectangle    16      NaN

>>> df.mul(other, fill_value=0)
      angles  degrees
circle       0      0.0
triangle      9      0.0
rectangle    16      0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle       0     360
  triangle      3     180
  rectangle      4     360
B square       4     360
  pentagon      5     540
  hexagon       6     720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle     NaN      1.0
  triangle  1.0      1.0
  rectangle  1.0      1.0
B square     0.0      0.0
  pentagon   0.0      0.0
  hexagon    0.0      0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
```

3 25 81

```
agg = aggregate(self, func=None, axis: Axis = 0, *args, **kwargs)
```

aggregate(self, func=None, axis: Axis = 0, *args, **kwargs) from pandas.core.frame.DataFrame
Aggregate using one or more operations over the specified axis.

Parameters

func : function, str, list or dict

Function to use for aggregating the data. If a function, must either work when passed a DataFrame or when passed to DataFrame.apply.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. ``[np.sum, 'mean']``
- dict of axis labels -> functions, function names or list of such.

axis : {0 or 'index', 1 or 'columns'}, default 0

If 0 or 'index': apply function to each column.

If 1 or 'columns': apply function to each row.

*args

Positional arguments to pass to `func`.

**kwargs

Keyword arguments to pass to `func`.

Returns

scalar, Series or DataFrame

The return can be:

- * scalar : when Series.agg is called with single function
- * Series : when DataFrame.agg is called with a single function
- * DataFrame : when DataFrame.agg is called with several functions

See Also

DataFrame.apply : Perform any type of operations.

DataFrame.transform : Perform transformation type operations.

DataFrame.groupby : Perform operations over groups.

DataFrame.resample : Perform operations over resampled bins.

DataFrame.rolling : Perform operations over rolling window.

DataFrame.expanding : Perform operations over expanding window.

core.window.ewm.ExponentialMovingWindow : Perform operation over exponential weighted window.

Notes

The aggregation operations are always performed over an axis, either the index (default) or the column axis. This behavior is different from `numpy` aggregation functions (`mean`, `median`, `prod`, `sum`, `std`, `var`), where the default is to compute the aggregation of the flattened array, e.g., ``numpy.mean(arr\_2d)`` as opposed to ``numpy.mean(arr\_2d, axis=0)``.

`agg` is an alias for `aggregate`. Use the alias.

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

A passed user-defined-function will be passed a Series for evaluation.

If ``func`` defines an index relabeling, ``axis`` must be ``0`` or ``index``.

Examples

```
>>> df = pd.DataFrame(  
...     [[1, 2, 3], [4, 5, 6], [7, 8, 9], [np.nan, np.nan, np.nan]],  
...     columns=["A", "B", "C"],  
... )
```

Aggregate these functions over the rows.

```
>>> df.agg(["sum", "min"])
```



```

      A      B      C
sum 12.0  15.0  18.0
min  1.0   2.0   3.0

```

Different aggregations per column.

```

>>> df.agg({"A": ["sum", "min"], "B": ["min", "max"]})
      A      B
sum 12.0  NaN
min  1.0   2.0
max   NaN   8.0

```

Aggregate different functions over the columns and rename the index of the resulting DataFrame.

```

>>> df.agg(x=("A", "max"), y=("B", "min"), z=("C", "mean"))
      A      B      C
x  7.0  NaN  NaN
y  NaN  2.0  NaN
z  NaN  NaN  6.0

```

Aggregate over the columns.

```

>>> df.agg("mean", axis="columns")
0      2.0
1      5.0
2      8.0
3      NaN
dtype: float64

```

```

all(
    self,
    *,
    axis: Axis | None = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> Series | bool from pandas.core.frame.DataFrame
Return whether all elements are True, potentially over an axis.

Returns True unless there at least one element within a series or
along a DataFrame axis that is False or equivalent (e.g. zero or
empty).

Parameters
-----
axis : {0 or 'index', 1 or 'columns', None}, default 0
    Indicate which axis or axes should be reduced. For `Series` this parameter
    is unused and defaults to 0.

    * 0 / 'index' : reduce the index, return a Series whose index is the
      original column labels.
    * 1 / 'columns' : reduce the columns, return a Series whose index is the
      original index.
    * None : reduce all axes, return a scalar.

bool_only : bool, default False
    Include only boolean columns. Not implemented for Series.
skipna : bool, default True
    Exclude NA/null values. If the entire row/column is NA and skipna is
    True, then the result will be True, as for an empty row/column.
    If skipna is False, then NA are treated as True, because these are not
    equal to zero.
**kwargs : any, default None
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.

Returns
-----
Series or scalar
    If axis=None, then a scalar boolean is returned.
    Otherwise a Series is returned with index matching the index argument.

See Also
-----
Series.all : Return True if all elements are True.
DataFrame.any : Return True if one (or more) elements are True.

```

Examples

****Series****

```
>>> pd.Series([True, True]).all()
True
>>> pd.Series([True, False]).all()
False
>>> pd.Series([], dtype="float64").all()
True
>>> pd.Series([np.nan]).all()
True
>>> pd.Series([np.nan]).all(skipna=False)
True
```

****DataFrames****

Create a DataFrame from a dictionary.

```
>>> df = pd.DataFrame({"col1": [True, True], "col2": [True, False]})
>>> df
   col1  col2
0   True   True
1   True  False
```

Default behaviour checks if values in each column all return True.

```
>>> df.all()
col1    True
col2   False
dtype: bool
```

Specify ``axis='columns'`` to check if values in each row all return True.

```
>>> df.all(axis="columns")
0    True
1   False
dtype: bool
```

Or ``axis=None`` for whether every value is True.

```
>>> df.all(axis=None)
False
```

```
any(
    self,
    *,
    axis: Axis | None = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> Series | bool from pandas.core.frame.DataFrame
Return whether any element is True, potentially over an axis.
```

Returns False unless there is at least one element within a series or along a DataFrame axis that is True or equivalent (e.g. non-zero or non-empty).

Parameters

axis : {0 or 'index', 1 or 'columns', None}, default 0
Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

- * 0 / 'index' : reduce the index, return a Series whose index is the original column labels.
- * 1 / 'columns' : reduce the columns, return a Series whose index is the original index.
- * None : reduce all axes, return a scalar.

bool_only : bool, default False

Include only boolean columns. Not implemented for Series.

skipna : bool, default True

Exclude NA/null values. If the entire row/column is NA and skipna is True, then the result will be False, as for an empty row/column.

If skipna is False, then NA are treated as True, because these are not

```

        equal to zero.
    **kwargs : any, default None
        Additional keywords have no effect but might be accepted for
        compatibility with NumPy.

Returns
-----
Series or scalar
    If axis=None, then a scalar boolean is returned.
    Otherwise a Series is returned with index matching the index argument.

See Also
-----
numpy.any : Numpy version of this method.
Series.any : Return whether any element is True.
Series.all : Return whether all elements are True.
DataFrame.any : Return whether any element is True over requested axis.
DataFrame.all : Return whether all elements are True over requested axis.

Examples
-----
**Series**

For Series input, the output is a scalar indicating whether any element
is True.

>>> pd.Series([False, False]).any()
False
>>> pd.Series([True, False]).any()
True
>>> pd.Series([], dtype="float64").any()
False
>>> pd.Series([np.nan]).any()
False
>>> pd.Series([np.nan]).any(skipna=False)
True

**DataFrame**

Whether each column contains at least one True element (the default).

>>> df = pd.DataFrame({"A": [1, 2], "B": [0, 2], "C": [0, 0]})
>>> df
   A  B  C
0  1  0  0
1  2  2  0

>>> df.any()
A     True
B     True
C    False
dtype: bool

Aggregating over the columns.

>>> df = pd.DataFrame({"A": [True, False], "B": [1, 2]})
>>> df
   A  B
0  True  1
1 False  2

>>> df.any(axis="columns")
0     True
1     True
dtype: bool

>>> df = pd.DataFrame({"A": [True, False], "B": [1, 0]})
>>> df
   A  B
0  True  1
1 False  0

>>> df.any(axis="columns")
0     True
1    False
dtype: bool

```

Aggregating over the entire DataFrame with ``axis=None``.

```
>>> df.any(axis=None)
True
```

``any`` for an empty DataFrame is an empty Series.

```
>>> pd.DataFrame([]).any()
Series([], dtype: bool)
```

```
apply(
    self,
    func: AggFuncType,
    axis: Axis = 0,
    raw: bool = False,
    result_type: Literal['expand', 'reduce', 'broadcast'] | None = None,
    args=(),
    by_row: Literal[False, 'compat'] = 'compat',
    engine: Callable | None | Literal['python', 'numba'] = None,
    engine_kwargs: dict[str, bool] | None = None,
    **kwargs
) from pandas.core.frame.DataFrame
Apply a function along an axis of the DataFrame.

Objects passed to the function are Series objects whose index is
either the DataFrame's index (``axis=0``) or the DataFrame's columns
(``axis=1``). By default (``result_type=None``), the final return type
is inferred from the return type of the applied function. Otherwise,
it depends on the ``result_type`` argument. The return type of the applied
function is inferred based on the first computed result obtained after
applying the function to a Series object.
```

Parameters

```
func : function
    Function to apply to each column or row.
axis : {0 or 'index', 1 or 'columns'}, default 0
    Axis along which the function is applied:

    * 0 or 'index': apply function to each column.
    * 1 or 'columns': apply function to each row.

raw : bool, default False
    Determines if row or column is passed as a Series or ndarray object:

    * ``False`` : passes each row or column as a Series to the
      function.
    * ``True`` : the passed function will receive ndarray objects
      instead.
    If you are just applying a NumPy reduction function this will
    achieve much better performance.
```

.. note::

When ``raw=True``, the result dtype is inferred from the **first** returned value.

```
result_type : {'expand', 'reduce', 'broadcast', None}, default None
    These only act when ``axis=1`` (columns):
```

- * 'expand' : list-like results will be turned into columns.
- * 'reduce' : returns a Series if possible rather than expanding list-like results. This is the opposite of 'expand'.
- * 'broadcast' : results will be broadcast to the original shape of the DataFrame, the original index and columns will be retained.

The default behaviour (None) depends on the return value of the applied function: list-like results will be returned as a Series of those. However if the apply function returns a Series these are expanded to columns.

```
args : tuple
    Positional arguments to pass to ``func`` in addition to the
    array/series.
by_row : False or "compat", default "compat"
    Only has an effect when ``func`` is a listlike or dictlike of funcs
    and the func isn't a string.
```

If "compat", will if possible first translate the func into pandas methods (e.g. ``Series().apply(np.sum)`` will be translated to ``Series().sum()``). If that doesn't work, will try call to apply again with ``by\_row=True`` and if that fails, will call apply again with ``by\_row=False`` (backward compatible).
If False, the func will be passed the whole Series at once.

.. versionadded:: 2.1.0

engine : decorator or {'python', 'numba'}, optional
Choose the execution engine to use. If not provided the function will be executed by the regular Python interpreter.

Other options include JIT compilers such Numba and Bodo, which in some cases can speed up the execution. To use an executor you can provide the decorators ``numba.jit``, ``numba.njit`` or ``bodo.jit``. You can also provide the decorator with parameters, like ``numba.jit(nogit=True)``.

Not all functions can be executed with all execution engines. In general, JIT compilers will require type stability in the function (no variable should change data type during the execution). And not all pandas and NumPy APIs are supported. Check the engine documentation [1]_ and [2]_ for limitations.

.. warning::

String parameters will stop being supported in a future pandas version.

.. versionadded:: 2.2.0

engine_kwargs : dict
Pass keyword arguments to the engine.
This is currently only used by the numba engine,
see the documentation for the engine argument for more information.

**kwargs
Additional keyword arguments to pass as keywords arguments to
`func`.

Returns

Series or DataFrame
Result of applying ``func`` along the given axis of the
DataFrame.

See Also

DataFrame.map: For elementwise operations.
DataFrame.aggregate: Only perform aggregating type operations.
DataFrame.transform: Only perform transforming type operations.

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

References

.. [1] `Numba documentation
<<https://numba.readthedocs.io/en/stable/index.html>>`_
.. [2] `Bodo documentation
<<https://docs.bodo.ai/latest/>>`/

Examples

>>> df = pd.DataFrame([[4, 9]] * 3, columns=["A", "B"])
>>> df
 A B
0 4 9
1 4 9
2 4 9

Using a numpy universal function (in this case the same as
``np.sqrt(df)``):

>>> df.apply(np.sqrt)

```

      A    B
0  2.0  3.0
1  2.0  3.0
2  2.0  3.0

```

Using a reducing function on either axis

```

>>> df.apply(np.sum, axis=0)
A    12
B    27
dtype: int64

```

```

>>> df.apply(np.sum, axis=1)
0    13
1    13
2    13
dtype: int64

```

Returning a list-like will result in a Series

```

>>> df.apply(lambda x: [1, 2], axis=1)
0    [1, 2]
1    [1, 2]
2    [1, 2]
dtype: object

```

Passing ``result\_type='expand'`` will expand list-like results to columns of a DataFrame

```

>>> df.apply(lambda x: [1, 2], axis=1, result_type="expand")
      0  1
0  1  2
1  1  2
2  1  2

```

Returning a Series inside the function is similar to passing ``result\_type='expand'``. The resulting column names will be the Series index.

```

>>> df.apply(lambda x: pd.Series([1, 2], index=["foo", "bar"]), axis=1)
      foo  bar
0      1    2
1      1    2
2      1    2

```

Passing ``result\_type='broadcast'`` will ensure the same shape result, whether list-like or scalar is returned by the function, and broadcast it along the axis. The resulting column names will be the originals.

```

>>> df.apply(lambda x: [1, 2], axis=1, result_type="broadcast")
      A  B
0  1  2
1  1  2
2  1  2

```

Advanced users can speed up their code by using a Just-in-time (JIT) compiler with ``apply``. The main JIT compilers available for pandas are Numba and Bodo. In general, JIT compilation is only possible when the function passed to ``apply`` has type stability (variables in the function do not change their type during the execution).

```

>>> import bodo # doctest: +SKIP
>>> df.apply(lambda x: x.A + x.B, axis=1, engine=bodo.jit) # doctest: +SKIP

```

Note that JIT compilation is only recommended for functions that take a significant amount of time to run. Fast functions are unlikely to run faster with JIT compilation.

```

assign(self, **kwargs) -> DataFrame from pandas.core.frame.DataFrame
Assign new columns to a DataFrame.

```

Returns a new object with all original columns in addition to new ones. Existing columns that are re-assigned will be overwritten.

Parameters

****kwargs** : callable or Series
The column names are keywords. If the values are callable, they are computed on the DataFrame and assigned to the new columns. The callable must not change input DataFrame (though pandas doesn't check it). If the values are not callable, (e.g. a Series, scalar, or array), they are simply assigned.

Returns

DataFrame

A new DataFrame with the new columns in addition to all the existing columns.

See Also

DataFrame.loc : Select a subset of a DataFrame by labels.

DataFrame.iloc : Select a subset of a DataFrame by positions.

Notes

Assigning multiple columns within the same ``assign`` is possible. Later items in ``\\*\*kwargs`` may refer to newly created or modified columns in 'df'; items are computed and assigned into 'df' in order.

Examples

```
>>> df = pd.DataFrame({"temp_c": [17.0, 25.0]}, index=["Portland", "Berkeley"])
>>> df
```

	temp_c
Portland	17.0
Berkeley	25.0

Where the value is a callable, evaluated on `df`:

```
>>> df.assign(temp_f=lambda x: x.temp_c * 9 / 5 + 32)
```

	temp_c	temp_f
Portland	17.0	62.6
Berkeley	25.0	77.0

Alternatively, the same behavior can be achieved by directly referencing an existing Series or sequence:

```
>>> df.assign(temp_f=df["temp_c"] * 9 / 5 + 32)
```

	temp_c	temp_f
Portland	17.0	62.6
Berkeley	25.0	77.0

or by using :meth:`pandas.col`:

```
>>> df.assign(temp_f=pd.col("temp_c") * 9 / 5 + 32)
```

	temp_c	temp_f
Portland	17.0	62.6
Berkeley	25.0	77.0

You can create multiple columns within the same assign where one of the columns depends on another one defined within the same assign:

```
>>> df.assign(
...     temp_f=lambda x: x["temp_c"] * 9 / 5 + 32,
...     temp_k=lambda x: (x["temp_f"] + 459.67) * 5 / 9,
... )
```

	temp_c	temp_f	temp_k
Portland	17.0	62.6	290.15
Berkeley	25.0	77.0	298.15

```
boxplot = boxplot_frame(
    self: DataFrame,
    column=None,
    by=None,
    ax=None,
    fontsize: int | None = None,
    rot: int = 0,
    grid: bool = True,
    figsize: tuple[float, float] | None = None,
    layout=None,
    return_type=None,
```

```

        backend=None,
        **kwargs
    ) from pandas.plotting
        Make a box plot from DataFrame columns.

        Make a box-and-whisker plot from DataFrame columns, optionally grouped
        by some other columns. A box plot is a method for graphically depicting
        groups of numerical data through their quartiles.
        The box extends from the Q1 to Q3 quartile values of the data,
        with a line at the median (Q2). The whiskers extend from the edges
        of box to show the range of the data. By default, they extend no more than
         $1.5 * IQR$  ( $IQR = Q3 - Q1$ ) from the edges of the box, ending at the farthest
        data point within that interval. Outliers are plotted as separate dots.

        For further details see
        Wikipedia's entry for boxplot <https://en.wikipedia.org/wiki/Box\_plot>`_.

Parameters
-----
column : str or list of str, optional
    Column name or list of names, or vector.
    Can be any valid input to :meth:`pandas.DataFrame.groupby`.
by : str or array-like, optional
    Column in the DataFrame to :meth:`pandas.DataFrame.groupby`.
    One box-plot will be done per value of columns in `by`.
ax : object of class matplotlib.axes.Axes, optional
    The matplotlib axes to be used by boxplot.
fontsize : float or str
    Tick label font size in points or as a string (e.g., `large`).
rot : float, default 0
    The rotation angle of labels (in degrees)
    with respect to the screen coordinate system.
grid : bool, default True
    Setting this to True will show the grid.
figsize : A tuple (width, height) in inches
    The size of the figure to create in matplotlib.
layout : tuple (rows, columns), optional
    For example, (3, 5) will display the subplots
    using 3 rows and 5 columns, starting from the top-left.
return_type : {'axes', 'dict', 'both'} or None, default 'axes'
    The kind of object to return. The default is `axes`.

    * 'axes' returns the matplotlib axes the boxplot is drawn on.
    * 'dict' returns a dictionary whose values are the matplotlib
      lines of the boxplot.
    * 'both' returns a namedtuple with the axes and dict.
    * when grouping with `by`, a Series mapping columns to
      `return_type` is returned.

    If `return_type` is `None`, a NumPy array
    of axes with the same shape as `layout` is returned.
backend : str, default None
    Backend to use instead of the backend specified in the option
    `plotting.backend`. For instance, 'matplotlib'. Alternatively, to
    specify the `plotting.backend` for the whole session, set
    `pd.options.plotting.backend`.

**kwargs
    All other plotting keyword arguments to be passed to
    :func:`matplotlib.pyplot.boxplot`.

Returns
-----
result
    See Notes.

See Also
-----
Series.plot.hist: Make a histogram.
matplotlib.pyplot.boxplot : Matplotlib equivalent plot.

Notes
-----
The return type depends on the `return_type` parameter:

* 'axes' : object of class matplotlib.axes.Axes
* 'dict' : dict of matplotlib.lines.Line2D objects

```


* 'both' : a namedtuple with structure (ax, lines)

For data grouped with ``by``, return a Series of the above or a numpy array:

```
* :class:`~pandas.Series`  
* :class:`~numpy.array` (for ``return_type = None``)
```

Use ``return\_type='dict'`` when you want to tweak the appearance of the lines after plotting. In this case a dict containing the Lines making up the boxes, caps, fliers, medians, and whiskers is returned.

Examples

Boxplots can be created for every column in the dataframe by ``df.boxplot()`` or indicating the columns to be used:

```
.. plot::  
:context: close-figs  
  
>>> np.random.seed(1234)  
>>> df = pd.DataFrame(  
...     np.random.randn(10, 4), columns=["Col1", "Col2", "Col3", "Col4"]  
... )  
>>> boxplot = df.boxplot(column=["Col1", "Col2", "Col3"]) # doctest: +SKIP
```

Boxplots of variables distributions grouped by the values of a third variable can be created using the option ``by``. For instance:

```
.. plot::  
:context: close-figs  
  
>>> df = pd.DataFrame(np.random.randn(10, 2), columns=["Col1", "Col2"])  
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])  
>>> boxplot = df.boxplot(by="X")
```

A list of strings (i.e. ``['X', 'Y']``) can be passed to boxplot in order to group the data by combination of the variables in the x-axis:

```
.. plot::  
:context: close-figs  
  
>>> df = pd.DataFrame(np.random.randn(10, 3), columns=["Col1", "Col2", "Col3"])  
>>> df["X"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B", "B", "B"])  
>>> df["Y"] = pd.Series(["A", "B", "A", "B", "A", "B", "A", "B", "A", "B"])  
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by=["X", "Y"])
```

The layout of boxplot can be adjusted giving a tuple to ``layout``:

```
.. plot::  
:context: close-figs  
  
>>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", layout=(2, 1))
```

Additional formatting can be done to the boxplot, like suppressing the grid (``grid=False``), rotating the labels in the x-axis (i.e. ``rot=45``) or changing the fontsize (i.e. ``fontsize=15``):

```
.. plot::  
:context: close-figs  
  
>>> boxplot = df.boxplot(grid=False, rot=45, fontsize=15) # doctest: +SKIP
```

The parameter ``return\_type`` can be used to select the type of element returned by ``boxplot``. When ``return\_type='axes'`` is selected, the matplotlib axes on which the boxplot is drawn are returned:

```
.. plot::  
:context: close-figs  
  
>>> boxplot = df.boxplot(column=["Col1", "Col2"], return_type="axes")  
>>> type(boxplot)  
<class 'matplotlib.axes._axes.Axes'>
```

When grouping with ``by``, a Series mapping columns to ``return\_type`` is returned:

```

.. plot::
   :context: close-figs

   >>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type="axes")
   >>> type(boxplot)
   <class 'pandas.Series'>

```

If ``return\_type`` is `None`, a NumPy array of axes with the same shape as ``layout`` is returned:

```

.. plot::
   :context: close-figs

   >>> boxplot = df.boxplot(column=["Col1", "Col2"], by="X", return_type=None)
   >>> type(boxplot)
   <class 'numpy.ndarray'>

```

```

combine(
    self,
    other: DataFrame,
    func: Callable[[Series, Series], Series | Hashable],
    fill_value=None,
    overwrite: bool = True
) -> DataFrame from pandas.core.frame.DataFrame
Perform column-wise combine with another DataFrame.

```

Combines a DataFrame with `other` DataFrame using `func` to element-wise combine columns. The row and column indexes of the resulting DataFrame will be the union of the two.

Parameters

```

other : DataFrame
    The DataFrame to merge column-wise.
func : function
    Function that takes two series as inputs and return a Series or a
    scalar. Used to merge the two dataframes column by columns.
fill_value : scalar value, default None
    The value to fill NaNs with prior to passing any column to the
    merge func.
overwrite : bool, default True
    If True, columns in `self` that do not exist in `other` will be
    overwritten with NaNs.

```

Returns

```

DataFrame
    Combination of the provided DataFrames.

```

See Also

```

DataFrame.combine_first : Combine two DataFrame objects and default to
non-null values in frame calling the method.

```

Examples

Combine using a simple function that chooses the smaller column.

```

>>> df1 = pd.DataFrame({"A": [0, 0], "B": [4, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> take_smaller = lambda s1, s2: s1 if s1.sum() < s2.sum() else s2
>>> df1.combine(df2, take_smaller)
   A  B
0  0  3
1  0  3

```

Example using a true element-wise combine function.

```

>>> df1 = pd.DataFrame({"A": [5, 0], "B": [2, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine(df2, np.minimum)
   A  B
0  1  2
1  0  3

```

Using `fill\_value` fills Nones prior to passing the column to the

merge function.

```
>>> df1 = pd.DataFrame({"A": [0, 0], "B": [None, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine(df2, take_smaller, fill_value=-5)
```

```
   A  B
0  0 -5.0
1  0  4.0
```

Example that demonstrates the use of `overwrite` and behavior when the axis differ between the dataframes.

```
>>> df1 = pd.DataFrame({"A": [0, 0], "B": [4, 4]})
>>> df2 = pd.DataFrame(
...     {
...         "B": [3, 3],
...         "C": [-10, 1],
...     },
...     index=[1, 2],
... )
>>> df1.combine(df2, take_smaller)
```

```
   A  B  C
0 NaN NaN NaN
1 NaN 3.0 -10.0
2 NaN 3.0  1.0
```

```
>>> df1.combine(df2, take_smaller, overwrite=False)
```

```
   A  B  C
0 0.0 NaN NaN
1 0.0 3.0 -10.0
2 NaN 3.0  1.0
```

Demonstrating the preference of the passed in dataframe.

```
>>> df2 = pd.DataFrame(
...     {
...         "B": [3, 3],
...         "C": [1, 1],
...     },
...     index=[1, 2],
... )
>>> df2.combine(df1, take_smaller)
```

```
   B  C  A
0 NaN NaN 0.0
1 3.0 NaN 0.0
2 3.0 NaN NaN
```

```
>>> df2.combine(df1, take_smaller, overwrite=False)
```

```
   B  C  A
0 NaN NaN 0.0
1 3.0 1.0 0.0
2 3.0 1.0 NaN
```

`combine_first(self, other: DataFrame) -> DataFrame` from `pandas.core.frame.DataFrame`
Update null elements with value in the same location in `other`.

Combine two DataFrame objects by filling null values in one DataFrame with non-null values from other DataFrame. The row and column indexes of the resulting DataFrame will be the union of the two. The resulting dataframe contains the 'first' dataframe values and overrides the second one values where both `first.loc[index, col]` and `second.loc[index, col]` are not missing values, upon calling `first.combine_first(second)`.

Parameters

`other` : DataFrame

Provided DataFrame to use to fill null values.

Returns

DataFrame

The result of combining the provided DataFrame with the other object.

See Also

`DataFrame.combine` : Perform series-wise operation on two DataFrames

using a given function.

Examples

```
>>> df1 = pd.DataFrame({"A": [None, 0], "B": [None, 4]})
>>> df2 = pd.DataFrame({"A": [1, 1], "B": [3, 3]})
>>> df1.combine_first(df2)
```

```
   A  B
0  1.0 3.0
1  0.0 4.0
```

Null values still persist if the location of that null value does not exist in `other`

```
>>> df1 = pd.DataFrame({"A": [None, 0], "B": [4, None]})
>>> df2 = pd.DataFrame({"B": [3, 3], "C": [1, 1]}, index=[1, 2])
>>> df1.combine_first(df2)
```

```
   A  B  C
0 NaN 4.0 NaN
1 0.0 3.0 1.0
2 NaN 3.0 1.0
```

```
compare(
    self,
    other: DataFrame,
    align_axis: Axis = 1,
    keep_shape: bool = False,
    keep_equal: bool = False,
    result_names: Suffixes = ('self', 'other')
) -> DataFrame from pandas.core.frame.DataFrame
Compare to another DataFrame and show the differences.
```

Parameters

other : DataFrame
Object to compare with.

align_axis : {0 or 'index', 1 or 'columns'}, default 1
Determine which axis to align the comparison on.

- * 0, or 'index' : Resulting differences are stacked vertically with rows drawn alternately from self and other.
- * 1, or 'columns' : Resulting differences are aligned horizontally with columns drawn alternately from self and other.

keep_shape : bool, default False
If true, all rows and columns are kept.
Otherwise, only the ones with different values are kept.

keep_equal : bool, default False
If true, the result keeps values that are equal.
Otherwise, equal values are shown as NaNs.

result_names : tuple, default ('self', 'other')
Set the dataframes names in the comparison.

Returns

DataFrame

DataFrame that shows the differences stacked side by side.

The resulting index will be a MultiIndex with 'self' and 'other' stacked alternately at the inner level.

Raises

ValueError

When the two DataFrames don't have identical labels or shape.

See Also

Series.compare : Compare with another Series and show differences.
DataFrame.equals : Test whether two objects contain the same elements.

Notes

Matching NaNs will not appear as a difference.

Can only compare identically-labeled
(i.e. same shape, identical row and column labels) DataFrames

Examples

```
>>> df = pd.DataFrame(
...     {
...         "col1": ["a", "a", "b", "b", "a"],
...         "col2": [1.0, 2.0, 3.0, np.nan, 5.0],
...         "col3": [1.0, 2.0, 3.0, 4.0, 5.0],
...     },
...     columns=["col1", "col2", "col3"],
... )
>>> df
   col1  col2  col3
0     a   1.0   1.0
1     a   2.0   2.0
2     b   3.0   3.0
3     b   NaN   4.0
4     a   5.0   5.0
```

```
>>> df2 = df.copy()
>>> df2.loc[0, "col1"] = "c"
>>> df2.loc[2, "col3"] = 4.0
>>> df2
   col1  col2  col3
0     c   1.0   1.0
1     a   2.0   2.0
2     b   3.0   4.0
3     b   NaN   4.0
4     a   5.0   5.0
```

Align the differences on columns

```
>>> df.compare(df2)
   col1  col3
self other self other
0     a     c  NaN  NaN
2  NaN  NaN  3.0  4.0
```

Assign result_names

```
>>> df.compare(df2, result_names=("left", "right"))
   col1  col3
left right left right
0     a     c  NaN  NaN
2  NaN  NaN  3.0  4.0
```

Stack the differences on rows

```
>>> df.compare(df2, align_axis=0)
   col1  col3
0 self    a  NaN
  other   c  NaN
2 self  NaN  3.0
  other  NaN  4.0
```

Keep the equal values

```
>>> df.compare(df2, keep_equal=True)
   col1  col3
self other self other
0     a     c   1.0   1.0
2     b     b   3.0   4.0
```

Keep all original rows and columns

```
>>> df.compare(df2, keep_shape=True)
   col1  col2  col3
self other self other self other
0     a     c  NaN  NaN  NaN  NaN
1  NaN  NaN  NaN  NaN  NaN  NaN
2  NaN  NaN  NaN  NaN   3.0   4.0
3  NaN  NaN  NaN  NaN  NaN  NaN
4  NaN  NaN  NaN  NaN  NaN  NaN
```

Keep all original rows and columns and also all original values

```
>>> df.compare(df2, keep_shape=True, keep_equal=True)
```

	col1	col2	col3
	self	other	self
0	a	c	1.0
1	a	a	2.0
2	b	b	3.0
3	b	b	NaN
4	a	a	5.0

```
corr(  
    self,  
    method: CorrelationMethod = 'pearson',  
    min_periods: int = 1,  
    numeric_only: bool = False  
) -> DataFrame from pandas.core.frame.DataFrame  
Compute pairwise correlation of columns, excluding NA/null values.
```

Parameters

method : {'pearson', 'kendall', 'spearman'} or callable
Method of correlation:

- * pearson : standard correlation coefficient
- * kendall : Kendall Tau correlation coefficient
- * spearman : Spearman rank correlation
- * callable: callable with input two 1d ndarrays and returning a float. Note that the returned matrix from corr will have 1 along the diagonals and will be symmetric regardless of the callable's behavior.

min_periods : int, optional
Minimum number of observations required per pair of columns to have a valid result. Currently only available for Pearson and Spearman correlation.

numeric_only : bool, default False
Include only 'float', 'int' or 'boolean' data.

.. versionchanged:: 2.0.0
The default value of 'numeric_only' is now 'False'.

Returns

DataFrame
Correlation matrix.

See Also

DataFrame.corrwith : Compute pairwise correlation with another DataFrame or Series.
Series.corr : Compute the correlation between two Series.

Notes

Pearson, Kendall and Spearman correlation are currently computed using pairwise complete

- * `Pearson correlation coefficient <https://en.wikipedia.org/wiki/Pearson_correlation_coefficient
- * `Kendall rank correlation coefficient <https://en.wikipedia.org/wiki/Kendall_rank_correlation_coefficient
- * `Spearman's rank correlation coefficient <https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient

Examples

```
>>> def histogram_intersection(a, b):  
...     v = np.minimum(a, b).sum().round(decimals=1)  
...     return v  
>>> df = pd.DataFrame(  
...     [(0.2, 0.3), (0.0, 0.6), (0.6, 0.0), (0.2, 0.1)],  
...     columns=["dogs", "cats"],  
... )  
>>> df.corr(method=histogram_intersection)  
      dogs  cats  
dogs    1.0  0.3  
cats    0.3  1.0
```

```
>>> df = pd.DataFrame(  
...     [(1, 1), (2, np.nan), (np.nan, 3), (4, 4)], columns=["dogs", "cats"]  
... )
```

```

>>> df.corr(min_periods=3)
      dogs  cats
dogs    1.0   NaN
cats    NaN   1.0

```

corrwith(
self,
other: DataFrame | Series,
axis: Axis = 0,
drop: bool = False,
method: CorrelationMethod = 'pearson',
numeric_only: bool = False,
min_periods: int | None = None
) -> Series from pandas.core.frame.DataFrame
Compute pairwise correlation.

Pairwise correlation is computed between rows or columns of
DataFrame with rows or columns of Series or DataFrame. DataFrames
are first aligned along both axes before computing the
correlations.

Parameters

other : DataFrame, Series
Object with which to compute correlations.
axis : {0 or 'index', 1 or 'columns'}, default 0
The axis to use. 0 or 'index' to compute row-wise, 1 or 'columns' for
column-wise.
drop : bool, default False
Drop missing indices from result.
method : {'pearson', 'kendall', 'spearman'} or callable
Method of correlation:

- * pearson : standard correlation coefficient
- * kendall : Kendall Tau correlation coefficient
- * spearman : Spearman rank correlation
- * callable: callable with input two 1d ndarrays
and returning a float.

numeric_only : bool, default False
Include only 'float', 'int' or 'boolean' data.

min_periods : int, optional
Minimum number of observations needed to have a valid result.

.. versionchanged:: 2.0.0
The default value of 'numeric_only' is now 'False'.

Returns

Series
Pairwise correlations.

See Also

DataFrame.corr : Compute pairwise correlation of columns.

Examples

>>> index = ["a", "b", "c", "d", "e"]
>>> columns = ["one", "two", "three", "four"]
>>> df1 = pd.DataFrame(
... np.arange(20).reshape(5, 4), index=index, columns=columns
...)
>>> df2 = pd.DataFrame(
... np.arange(16).reshape(4, 4), index=index[:4], columns=columns
...)
>>> df1.corrwith(df2)
one 1.0
two 1.0
three 1.0
four 1.0
dtype: float64

>>> df2.corrwith(df1, axis=1)
a 1.0
b 1.0

```

c    1.0
d    1.0
e    NaN
dtype: float64

```

count(self, axis: Axis = 0, numeric_only: bool = False) -> Series from pandas.core.frame.DataFrame
 Count non-NA cells for each column or row.

The values `None`, `NaN`, `NaT`, ``pandas.NA`` are considered NA.

Parameters

```

axis : {0 or 'index', 1 or 'columns'}, default 0
    If 0 or 'index' counts are generated for each column.
    If 1 or 'columns' counts are generated for each row.
numeric_only : bool, default False
    Include only `float`, `int` or `boolean` data.

```

Returns

Series

For each column/row the number of non-NA/null entries.

See Also

```

Series.count: Number of non-NA elements in a Series.
DataFrame.value_counts: Count unique combinations of columns.
DataFrame.shape: Number of DataFrame rows and columns (including NA
elements).
DataFrame.isna: Boolean same-sized DataFrame showing places of NA
elements.

```

Examples

Constructing DataFrame from a dictionary:

```

>>> df = pd.DataFrame(
...     {
...         "Person": ["John", "Myla", "Lewis", "John", "Myla"],
...         "Age": [24.0, np.nan, 21.0, 33, 26],
...         "Single": [False, True, True, True, False],
...     }
... )
>>> df
   Person  Age  Single
0   John  24.0   False
1   Myla   NaN    True
2  Lewis  21.0    True
3   John  33.0    True
4   Myla  26.0   False

```

Notice the uncounted NA values:

```

>>> df.count()
Person    5
Age       4
Single    5
dtype: int64

```

Counts for each **row**:

```

>>> df.count(axis="columns")
0    3
1    2
2    3
3    3
4    3
dtype: int64

```

```

cov(
    self,
    min_periods: int | None = None,
    ddof: int | None = 1,
    numeric_only: bool = False
) -> DataFrame from pandas.core.frame.DataFrame
    Compute pairwise covariance of columns, excluding NA/null values.

```


Compute the pairwise covariance among the series of a DataFrame. The returned data frame is the 'covariance matrix' https://en.wikipedia.org/wiki/Covariance_matrix of the columns of the DataFrame.

Both NA and null values are automatically excluded from the calculation. (See the note below about bias from missing values.) A threshold can be set for the minimum number of observations for each value created. Comparisons with observations below this threshold will be returned as ``NaN``.

This method is generally used for the analysis of time series data to understand the relationship between different measures across time.

Parameters

min_periods : int, optional
 Minimum number of observations required per pair of columns to have a valid result.

ddof : int, default 1
 Delta degrees of freedom. The divisor used in calculations is ``N - ddof``, where ``N`` represents the number of elements. This argument is applicable only when no ``nan`` is in the dataframe.

numeric_only : bool, default False
 Include only 'float', 'int' or 'boolean' data.

 .. versionchanged:: 2.0.0
 The default value of ``numeric\_only`` is now ``False``.

Returns

DataFrame
 The covariance matrix of the series of the DataFrame.

See Also

Series.cov : Compute covariance with another Series.
core.window.ewm.ExponentialMovingWindow.cov : Exponential weighted sample covariance.
core.window.expanding.Expanding.cov : Expanding sample covariance.
core.window.rolling.Rolling.cov : Rolling sample covariance.

Notes

Returns the covariance matrix of the DataFrame's time series. The covariance is normalized by N-ddof.

For DataFrames that have Series that are missing data (assuming that data is 'missing at random' https://en.wikipedia.org/wiki/Missing_data#Missing_at_random) the returned covariance matrix will be an unbiased estimate of the variance and covariance between the member Series.

However, for many applications this estimate may not be acceptable because the estimate covariance matrix is not guaranteed to be positive semi-definite. This could lead to estimate correlations having absolute values which are greater than one, and/or a non-invertible covariance matrix. See 'Estimation of covariance matrices' https://en.wikipedia.org/w/index.php?title=Estimation_of_covariance_matrices for more details.

Examples

>>> df = pd.DataFrame(
... [(1, 2), (0, 3), (2, 0), (1, 1)], columns=["dogs", "cats"]
...)
>>> df.cov()
 dogs cats
dogs 0.666667 -1.000000
cats -1.000000 1.666667

>>> np.random.seed(42)
>>> df = pd.DataFrame(
... np.random.randn(1000, 5), columns=["a", "b", "c", "d", "e"]

```

... )
>>> df.cov()
          a          b          c          d          e
a  0.998438 -0.020161  0.059277 -0.008943  0.014144
b -0.020161  1.059352 -0.008543 -0.024738  0.009826
c  0.059277 -0.008543  1.010670 -0.001486 -0.000271
d -0.008943 -0.024738 -0.001486  0.921297 -0.013692
e  0.014144  0.009826 -0.000271 -0.013692  0.977795

**Minimum number of periods**

This method also supports an optional ``min_periods`` keyword
that specifies the required minimum number of non-NA observations for
each column pair in order to have a valid result:

>>> np.random.seed(42)
>>> df = pd.DataFrame(np.random.randn(20, 3), columns=["a", "b", "c"])
>>> df.loc[df.index[:5], "a"] = np.nan
>>> df.loc[df.index[5:10], "b"] = np.nan
>>> df.cov(min_periods=12)
          a          b          c
a  0.316741      NaN -0.150812
b      NaN  1.248003  0.191417
c -0.150812  0.191417  0.895202

```

cummax(
 self,
 axis: Axis = 0,
 skipna: bool = True,
 numeric_only: bool = False,
 *args,
 **kwargs
) -> Self from pandas.core.frame.DataFrame
Return cumulative maximum over a DataFrame or Series axis.

Returns a DataFrame or Series of the same size containing the cumulative maximum.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
 The index or the name of the axis. 0 is equivalent to None or 'index'.
 For 'Series' this parameter is unused and defaults to 0.
skipna : bool, default True
 Exclude NA/null values. If an entire row/column is NA, the result
 will be NA.
numeric_only : bool, default False
 Include only float, int, boolean columns.
*args, **kwargs
 Additional keywords have no effect but might be accepted for
 compatibility with NumPy.

Returns

Series or DataFrame
 Return cumulative maximum of Series or DataFrame.

See Also

pandas.window.expanding.Expanding.max : Similar functionality
 but ignores ``NaN`` values.
DataFrame.max : Return the maximum over
 DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

****Series****

```

>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0

```

```

3    -1.0
4     0.0
dtype: float64

```

By default, NA values are ignored.

```

>>> s.cummax()
0     2.0
1    NaN
2     5.0
3     5.0
4     5.0
dtype: float64

```

To include NA values in the operation, use ``skipna=False``

```

>>> s.cummax(skipna=False)
0     2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

```

****DataFrame****

```

>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0

```

By default, iterates over rows and finds the maximum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```

>>> df.cummax()
   A    B
0  2.0  1.0
1  3.0  NaN
2  3.0  1.0

```

To iterate over columns and find the maximum in each row, use ``axis=1``

```

>>> df.cummax(axis=1)
   A    B
0  2.0  2.0
1  3.0  NaN
2  1.0  1.0

```

```

cummin(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
) -> Self from pandas.core.frame.DataFrame

```

Return cumulative minimum over a DataFrame or Series axis.

Returns a DataFrame or Series of the same size containing the cumulative minimum.

Parameters

```

-----
axis : {0 or 'index', 1 or 'columns'}, default 0
    The index or the name of the axis. 0 is equivalent to None or 'index'.
    For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If an entire row/column is NA, the result
    will be NA.
numeric_only : bool, default False
    Include only float, int, boolean columns.
*args, **kwargs

```

Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or DataFrame

Return cumulative minimum of Series or DataFrame.

See Also

core.window.expanding.Expanding.min : Similar functionality but ignores ``NaN`` values.

DataFrame.min : Return the minimum over DataFrame axis.

DataFrame.cummax : Return cumulative maximum over DataFrame axis.

DataFrame.cummin : Return cumulative minimum over DataFrame axis.

DataFrame.cumsum : Return cumulative sum over DataFrame axis.

DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

****Series****

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
```

```
>>> s
```

```
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummin()
```

```
0    2.0
1    NaN
2    2.0
3   -1.0
4   -1.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummin(skipna=False)
```

```
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

****DataFrame****

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
```

```
>>> df
   A  B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

By default, iterates over rows and finds the minimum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cummin()
```

```
   A  B
0  2.0  1.0
1  2.0  NaN
2  1.0  0.0
```

To iterate over columns and find the minimum in each row, use ``axis=1``

```
>>> df.cummin(axis=1)
```

```
   A  B
```

```

0  2.0  1.0
1  3.0  NaN
2  1.0  0.0

```

```

cumprod(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
)

```

-> Self from pandas.core.frame.DataFrame
Return cumulative product over a DataFrame or Series axis.

Returns a DataFrame or Series of the same size containing the cumulative product.

Parameters

```

axis : {0 or 'index', 1 or 'columns'}, default 0
    The index or the name of the axis. 0 is equivalent to None or 'index'.
    For `Series` this parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If an entire row/column is NA, the result
    will be NA.
numeric_only : bool, default False
    Include only float, int, boolean columns.
*args, **kwargs
    Additional keywords have no effect but might be accepted for
    compatibility with NumPy.

```

Returns

```

Series or DataFrame
    Return cumulative product of Series or DataFrame.

```

See Also

```

core.window.expanding.Expanding.prod : Similar functionality
    but ignores ``NaN`` values.
DataFrame.prod : Return the product over
    DataFrame axis.
DataFrame.cummax : Return cumulative maximum over DataFrame axis.
DataFrame.cummin : Return cumulative minimum over DataFrame axis.
DataFrame.cumsum : Return cumulative sum over DataFrame axis.
DataFrame.cumprod : Return cumulative product over DataFrame axis.

```

Examples

```

**Series**

```

```

>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64

```

By default, NA values are ignored.

```

>>> s.cumprod()
0    2.0
1    NaN
2   10.0
3  -10.0
4   -0.0
dtype: float64

```

To include NA values in the operation, use ``skipna=False``

```

>>> s.cumprod(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN

```

```
4      NaN
dtype: float64
```

```
**DataFrame**
```

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
>>> df
   A    B
0  2.0  1.0
1  3.0  NaN
2  1.0  0.0
```

By default, iterates over rows and finds the product in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cumprod()
   A    B
0  2.0  1.0
1  6.0  NaN
2  6.0  0.0
```

To iterate over columns and find the product in each row, use ``axis=1``

```
>>> df.cumprod(axis=1)
   A    B
0  2.0  2.0
1  3.0  NaN
2  1.0  0.0
```

```
cumsum(
    self,
    axis: Axis = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    *args,
    **kwargs
) -> Self from pandas.core.frame.DataFrame
```

Return cumulative sum over a DataFrame or Series axis.

Returns a DataFrame or Series of the same size containing the cumulative sum.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
The index or the name of the axis. 0 is equivalent to None or 'index'.
For `Series` this parameter is unused and defaults to 0.

skipna : bool, default True
Exclude NA/null values. If an entire row/column is NA, the result will be NA.

numeric_only : bool, default False
Include only float, int, boolean columns.

*args, **kwargs
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or DataFrame
Return cumulative sum of Series or DataFrame.

See Also

core.window.expanding.Expanding.sum : Similar functionality but ignores ``NaN`` values.

DataFrame.sum : Return the sum over DataFrame axis.

DataFrame.cummax : Return cumulative maximum over DataFrame axis.

DataFrame.cummin : Return cumulative minimum over DataFrame axis.

DataFrame.cumsum : Return cumulative sum over DataFrame axis.

DataFrame.cumprod : Return cumulative product over DataFrame axis.

Examples

****Series****

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
```

```
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cumsum()
```

```
0    2.0
1    NaN
2    7.0
3    6.0
4    6.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cumsum(skipna=False)
```

```
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

****DataFrame****

```
>>> df = pd.DataFrame(
...     [[2.0, 1.0], [3.0, np.nan], [1.0, 0.0]], columns=list("AB")
... )
```

```
>>> df
   A  B
0  2.0 1.0
1  3.0 NaN
2  1.0 0.0
```

By default, iterates over rows and finds the sum in each column. This is equivalent to ``axis=None`` or ``axis='index'``.

```
>>> df.cumsum()
```

```
   A  B
0  2.0 1.0
1  5.0 NaN
2  6.0 1.0
```

To iterate over columns and find the sum in each row, use ``axis=1``

```
>>> df.cumsum(axis=1)
```

```
   A  B
0  2.0 3.0
1  3.0 NaN
2  1.0 1.0
```

`diff(self, periods: int = 1, axis: Axis = 0) -> DataFrame` from `pandas.core.frame.DataFrame`
First discrete difference of element.

Calculates the difference of a DataFrame element compared with another element in the DataFrame (default is element in previous row).

Parameters

`periods` : int, default 1

Periods to shift for calculating difference, accepts negative values.

`axis` : {0 or 'index', 1 or 'columns'}, default 0

Take difference over rows (0) or columns (1).

Returns

DataFrame

First differences of the Series.

See Also

DataFrame.pct_change: Percent change over given number of periods.
DataFrame.shift: Shift index by desired number of periods with an optional time freq.
Series.diff: First discrete difference of object.

Notes

For boolean dtypes, this uses :meth:`operator.xor` rather than :meth:`operator.sub`.
The result is calculated according to current dtype in DataFrame, however dtype of the result is always float64.

Examples

Difference with previous row

```
>>> df = pd.DataFrame(
...     {
...         "a": [1, 2, 3, 4, 5, 6],
...         "b": [1, 1, 2, 3, 5, 8],
...         "c": [1, 4, 9, 16, 25, 36],
...     }
... )
>>> df
   a  b  c
0  1  1  1
1  2  1  4
2  3  2  9
3  4  3 16
4  5  5 25
5  6  8 36
>>> df.diff()
   a  b  c
0 NaN NaN NaN
1 1.0 0.0 3.0
2 1.0 1.0 5.0
3 1.0 1.0 7.0
4 1.0 2.0 9.0
5 1.0 3.0 11.0
```

Difference with previous column

```
>>> df.diff(axis=1)
   a  b  c
0 NaN 0  0
1 NaN -1 3
2 NaN -1 7
3 NaN -1 13
4 NaN  0 20
5 NaN  2 28
```

Difference with 3rd previous row

```
>>> df.diff(periods=3)
   a  b  c
0 NaN NaN NaN
1 NaN NaN NaN
2 NaN NaN NaN
3 3.0 2.0 15.0
4 3.0 4.0 21.0
5 3.0 6.0 27.0
```

Difference with following row

```
>>> df.diff(periods=-1)
   a  b  c
0 -1.0 0.0 -3.0
1 -1.0 -1.0 -5.0
2 -1.0 -1.0 -7.0
3 -1.0 -2.0 -9.0
4 -1.0 -3.0 -11.0
5 NaN NaN NaN
```


Overflow in input dtype

```
>>> df = pd.DataFrame({"a": [1, 0]}, dtype=np.uint8)
>>> df.diff()
      a
0   NaN
1  255.0
```

`div = truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`divide = truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`dot(self, other: AnyArrayLike | DataFrame) -> DataFrame | Series` from `pandas.core.frame.DataFrame`
Compute the matrix multiplication between the DataFrame and other.

This method computes the matrix product between the DataFrame and the values of an other Series, DataFrame or a numpy array.

It can also be called using `self @ other`.

Parameters

`other` : Series, DataFrame or array-like
The other object to compute the matrix product with.

Returns

Series or DataFrame
If other is a Series, return the matrix product between self and other as a Series. If other is a DataFrame or a numpy.array, return the matrix product of self and other in a DataFrame of a np.array.

See Also

`Series.dot`: Similar method for Series.

Notes

The dimensions of DataFrame and other must be compatible in order to compute the matrix multiplication. In addition, the column names of DataFrame and the index of other must contain the same values, as they will be aligned prior to the multiplication.

The dot method for Series computes the inner product, instead of the matrix product here.

Examples

Here we multiply a DataFrame with a Series.

```
>>> df = pd.DataFrame([[0, 1, -2, -1], [1, 1, 1, 1]])
>>> s = pd.Series([1, 1, 2, 1])
>>> df.dot(s)
0    -4
1     5
dtype: int64
```

Here we multiply a DataFrame with another DataFrame.

```
>>> other = pd.DataFrame([[0, 1], [1, 2], [-1, -1], [2, 0]])
>>> df.dot(other)
      0    1
0     1    4
1     2    2
```

Note that the dot method give the same result as @

```
>>> df @ other
      0    1
0     1    4
1     2    2
```

The dot method works also if other is an np.array.

```
>>> arr = np.array([[0, 1], [1, 2], [-1, -1], [2, 0]])
>>> df.dot(arr)
```

```

      0  1
0    1  4
1    2  2

```

Note how shuffling of the objects does not change the result.

```

>>> s2 = s.reindex([1, 0, 2, 3])
>>> df.dot(s2)
0    -4
1     5
dtype: int64

```

```

drop(
    self,
    labels: IndexLabel | ListLike = None,
    *,
    axis: Axis = 0,
    index: IndexLabel | ListLike = None,
    columns: IndexLabel | ListLike = None,
    level: Level | None = None,
    inplace: bool = False,
    errors: IgnoreRaise = 'raise'
) -> DataFrame | None from pandas.core.frame.DataFrame
Drop specified labels from rows or columns.

```

Remove rows or columns by specifying label names and corresponding axis, or by directly specifying index or column names. When using a multi-index, labels on different levels can be removed by specifying the level. See the :ref:`user guide <advanced.shown\_levels>` for more information about the now unused levels.

Parameters

```

labels : single label or iterable of labels
    Index or column labels to drop. A tuple will be used as a single
    label and not treated as an iterable.
axis : {0 or 'index', 1 or 'columns'}, default 0
    Whether to drop labels from the index (0 or 'index') or
    columns (1 or 'columns').
index : single label or iterable of labels
    Alternative to specifying axis (`labels, axis=0`
    is equivalent to `index=labels`).
columns : single label or iterable of labels
    Alternative to specifying axis (`labels, axis=1`
    is equivalent to `columns=labels`).
level : int or level name, optional
    For MultiIndex, level from which the labels will be removed.
inplace : bool, default False
    If False, return a copy. Otherwise, do operation
    in place and return None.
errors : {'ignore', 'raise'}, default 'raise'
    If 'ignore', suppress error and only existing labels are
    dropped.

```

Returns

```

Dataframe or None
    Returns DataFrame or None DataFrame with the specified
    index or column labels removed or None if inplace=True.

```

Raises

```

KeyError
    If any of the labels is not found in the selected axis.

```

See Also

```

DataFrame.loc : Label-location based indexer for selection by label.
DataFrame.dropna : Return DataFrame with labels on given axis omitted
    where (all or any) data are missing.
DataFrame.drop_duplicates : Return DataFrame with duplicate rows
    removed, optionally only considering certain columns.
Series.drop : Return Series with specified index labels removed.

```

Examples

```

>>> df = pd.DataFrame(np.arange(12).reshape(3, 4), columns=["A", "B", "C", "D"])

```

```
>>> df
   A  B  C  D
0  0  1  2  3
1  4  5  6  7
2  8  9 10 11
```

Drop columns

```
>>> df.drop(["B", "C"], axis=1)
   A  D
0  0  3
1  4  7
2  8 11
```

```
>>> df.drop(columns=["B", "C"])
   A  D
0  0  3
1  4  7
2  8 11
```

Drop a row by index

```
>>> df.drop([0, 1])
   A  B  C  D
2  8  9 10 11
```

Drop columns and/or rows of MultiIndex DataFrame

```
>>> midx = pd.MultiIndex(
...     levels=[["llama", "cow", "falcon"], ["speed", "weight", "length"]],
...     codes=[[0, 0, 0, 1, 1, 1, 2, 2, 2], [0, 1, 2, 0, 1, 2, 0, 1, 2]],
... )
>>> df = pd.DataFrame(
...     index=midx,
...     columns=["big", "small"],
...     data=[
...         [45, 30],
...         [200, 100],
...         [1.5, 1],
...         [30, 20],
...         [250, 150],
...         [1.5, 0.8],
...         [320, 250],
...         [1, 0.8],
...         [0.3, 0.2],
...     ],
... )
>>> df
```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
	length	1.5	1.0
cow	speed	30.0	20.0
	weight	250.0	150.0
	length	1.5	0.8
falcon	speed	320.0	250.0
	weight	1.0	0.8
	length	0.3	0.2

Drop a specific index combination from the MultiIndex DataFrame, i.e., drop the combination ``'falcon'`` and ``'weight'``, which deletes only the corresponding row

```
>>> df.drop(index=("falcon", "weight"))
```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
	length	1.5	1.0
cow	speed	30.0	20.0
	weight	250.0	150.0
	length	1.5	0.8
falcon	speed	320.0	250.0
	length	0.3	0.2

```
>>> df.drop(index="cow", columns="small")
```

		big
llama	speed	45.0

```

        weight 200.0
        length 1.5
falcon speed 320.0
        weight 1.0
        length 0.3

>>> df.drop(index="length", level=1)

```

		big	small
llama	speed	45.0	30.0
	weight	200.0	100.0
cow	speed	30.0	20.0
	weight	250.0	150.0
falcon	speed	320.0	250.0
	weight	1.0	0.8

```

drop_duplicates(
    self,
    subset: Hashable | Iterable[Hashable] | None = None,
    *,
    keep: DropKeep = 'first',
    inplace: bool = False,
    ignore_index: bool = False
) -> DataFrame | None from pandas.core.frame.DataFrame
Return DataFrame with duplicate rows removed.

```

Considering certain columns is optional. Indexes, including time indexes are ignored.

Parameters

subset : column label or iterable of labels, optional
Only consider certain columns for identifying duplicates, by default use all of the columns.

keep : {'first', 'last', ``False``}, default 'first'
Determines which duplicates (if any) to keep.

- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- ``False`` : Drop all duplicates.

inplace : bool, default ``False``

Whether to modify the DataFrame rather than creating a new one.

ignore_index : bool, default ``False``

If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

Returns

DataFrame or None

DataFrame with duplicates removed or None if ``inplace=True``.

See Also

DataFrame.value_counts: Count unique combinations of columns.

Notes

This method requires columns specified by ``subset`` to be of hashable type. Passing unhashable columns will raise a ``TypeError``.

Examples

Consider dataset containing ramen rating.

```

>>> df = pd.DataFrame(
...     {
...         "brand": ["Yum Yum", "Yum Yum", "Indomie", "Indomie", "Indomie"],
...         "style": ["cup", "cup", "cup", "pack", "pack"],
...         "rating": [4, 4, 3.5, 15, 5],
...     }
... )
>>> df

```

	brand	style	rating
0	Yum Yum	cup	4.0
1	Yum Yum	cup	4.0
2	Indomie	cup	3.5
3	Indomie	pack	15.0
4	Indomie	pack	5.0

By default, it removes duplicate rows based on all columns.

```
>>> df.drop_duplicates()
  brand style rating
0  Yum Yum   cup    4.0
2  Indomie  cup    3.5
3  Indomie pack   15.0
4  Indomie pack    5.0
```

To remove duplicates on specific column(s), use ``subset``.

```
>>> df.drop_duplicates(subset=["brand"])
  brand style rating
0  Yum Yum   cup    4.0
2  Indomie  cup    3.5
```

To remove duplicates and keep last occurrences, use ``keep``.

```
>>> df.drop_duplicates(subset=["brand", "style"], keep="last")
  brand style rating
1  Yum Yum   cup    4.0
2  Indomie  cup    3.5
4  Indomie pack    5.0
```

```
dropna(
    self,
    *,
    axis: Axis = 0,
    how: AnyAll | lib.NoDefault = <no_default>,
    thresh: int | lib.NoDefault = <no_default>,
    subset: IndexLabel | AnyArrayLike | None = None,
    inplace: bool = False,
    ignore_index: bool = False
) -> DataFrame | None from pandas.core.frame.DataFrame
Remove missing values.
```

See the :ref:`User Guide <missing\_data>` for more on which values are considered missing, and how to work with missing data.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
Determine if rows or columns which contain missing values are removed.

- * 0, or 'index' : Drop rows which contain missing values.
- * 1, or 'columns' : Drop columns which contain missing value.

Only a single axis is allowed.

how : {'any', 'all'}, default 'any'
Determine if row or column is removed from DataFrame, when we have at least one NA or all NA.

- * 'any' : If any NA values are present, drop that row or column.
- * 'all' : If all values are NA, drop that row or column.

thresh : int, optional
Require that many non-NA values. Cannot be combined with how.
subset : column label or iterable of labels, optional
Labels along other axis to consider, e.g. if you are dropping rows these would be a list of columns to include.

inplace : bool, default False
Whether to modify the DataFrame rather than creating a new one.

ignore_index : bool, default ``False``
If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

.. versionadded:: 2.0.0

Returns

DataFrame or None
DataFrame with NA entries dropped from it or None if ``inplace=True``.

See Also

DataFrame.isna: Indicate missing values.
 DataFrame.notna : Indicate existing (non-missing) values.
 DataFrame.fillna : Replace missing values.
 Series.dropna : Drop missing values.
 Index.dropna : Drop missing indices.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "name": ["Alfred", "Batman", "Catwoman"],
...         "toy": [np.nan, "Batmobile", "Bullwhip"],
...         "born": [pd.NaT, pd.Timestamp("1940-04-25"), pd.NaT],
...     }
... )
>>> df
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

Drop the rows where at least one element is missing.

```
>>> df.dropna()
      name      toy      born
1  Batman  Batmobile 1940-04-25
```

Drop the columns where at least one element is missing.

```
>>> df.dropna(axis="columns")
      name
0  Alfred
1  Batman
2  Catwoman
```

Drop the rows where all elements are missing.

```
>>> df.dropna(how="all")
      name      toy      born
0  Alfred      NaN      NaT
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip      NaT
```

Keep only the rows with at least 2 non-NA values.

```
>>> df.dropna(thresh=2)
      name      toy      born
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip      NaT
```

Define in which columns to look for missing values.

```
>>> df.dropna(subset=["name", "toy"])
      name      toy      born
1  Batman  Batmobile 1940-04-25
2  Catwoman  Bullwhip      NaT
```

```
def duplicated(
    self,
    subset: Hashable | Iterable[Hashable] | None = None,
    keep: DropKeep = 'first'
) -> Series from pandas.core.frame.DataFrame
    Return boolean Series denoting duplicate rows.
```

Considering certain columns is optional.

Parameters

subset : column label or iterable of labels, optional
 Only consider certain columns for identifying duplicates, by default use all of the columns.
 keep : {'first', 'last', False}, default 'first'
 Determines which duplicates (if any) to mark.

- 'first' : Mark duplicates as 'True' except for the first occurrence.
- 'last' : Mark duplicates as 'True' except for the last occurrence.
- False : Mark all duplicates as 'True'.

Returns

Series

Boolean series for each duplicated rows.

See Also

`Index.duplicated` : Equivalent method on index.

`Series.duplicated` : Equivalent method on Series.

`Series.drop_duplicates` : Remove duplicate values from Series.

`DataFrame.drop_duplicates` : Remove duplicate values from DataFrame.

Examples

Consider dataset containing ramen rating.

```
>>> df = pd.DataFrame(
...     {
...         "brand": ["Yum Yum", "Yum Yum", "Indomie", "Indomie", "Indomie"],
...         "style": ["cup", "cup", "cup", "pack", "pack"],
...         "rating": [4, 4, 3.5, 15, 5],
...     }
... )
>>> df
   brand style  rating
0  Yum Yum   cup    4.0
1  Yum Yum   cup    4.0
2  Indomie   cup    3.5
3  Indomie  pack   15.0
4  Indomie  pack    5.0
```

By default, for each set of duplicated values, the first occurrence is set on False and all others on True.

```
>>> df.duplicated()
0    False
1     True
2    False
3    False
4    False
dtype: bool
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True.

```
>>> df.duplicated(keep="last")
0     True
1    False
2    False
3    False
4    False
dtype: bool
```

By setting ``keep`` on False, all duplicates are True.

```
>>> df.duplicated(keep=False)
0     True
1     True
2    False
3    False
4    False
dtype: bool
```

To find duplicates on specific column(s), use ``subset``.

```
>>> df.duplicated(subset=["brand"])
0    False
1     True
2    False
3     True
4     True
dtype: bool
```

`eq(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame`
Get Not equal to of dataframe and other, element-wise (binary operator ``eq``).

Among flexible wrappers (``eq``, ``ne``, ``le``, ``lt``, ``ge``, ``gt``) to comparison operators.

Equivalent to ``==``, ``!=``, ``<=``, ``<``, ``>=``, ``>`` with support to choose axis (rows or columns) and level for comparison.

Parameters

`other` : scalar, sequence, Series, or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}, default 'columns'
Whether to compare by the index (0 or 'index') or columns (1 or 'columns').
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns

DataFrame of bool
Result of the comparison.

See Also

`DataFrame.eq` : Compare DataFrames for equality elementwise.
`DataFrame.ne` : Compare DataFrames for inequality elementwise.
`DataFrame.le` : Compare DataFrames for less than inequality or equality elementwise.
`DataFrame.lt` : Compare DataFrames for strictly less than inequality elementwise.
`DataFrame.ge` : Compare DataFrames for greater than inequality or equality elementwise.
`DataFrame.gt` : Compare DataFrames for strictly greater than inequality elementwise.

Notes

Mismatched indices will be unioned together.
``NaN`` values are considered different (i.e. ``NaN` != `NaN``).

Examples

```
>>> df = pd.DataFrame(
...     {"cost": [250, 150, 100], "revenue": [100, 250, 300]},
...     index=["A", "B", "C"],
... )
>>> df
   cost  revenue
A   250     100
B   150     250
C   100     300
```

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
   cost  revenue
A  False     True
B  False     False
C   True     False

>>> df.eq(100)
   cost  revenue
A  False     True
B  False     False
C   True     False
```

When ``other`` is a `:class:`Series``, the columns of a DataFrame are aligned with the index of ``other`` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
   cost  revenue
A   True     True
B   True     False
C  False     True
```

Use the method to control the broadcast axis:


```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis="index")
      cost  revenue
A   True   False
B   True    True
C   True    True
D   True    True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
      cost  revenue
A   True    True
B  False   False
C  False   False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis="index")
      cost  revenue
A   True   False
B  False    True
C   True   False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame(
...     {"revenue": [300, 250, 100, 150]}, index=["A", "B", "C", "D"]
... )
>>> other
      revenue
A         300
B         250
C         100
D         150
```

```
>>> df.gt(other)
      cost  revenue
A  False   False
B  False   False
C  False    True
D  False   False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame(
...     {
...         "cost": [250, 150, 100, 150, 300, 220],
...         "revenue": [100, 250, 300, 200, 175, 225],
...     },
...     index=[
...         ["Q1", "Q1", "Q1", "Q2", "Q2", "Q2"],
...         ["A", "B", "C", "A", "B", "C"],
...     ],
... )
>>> df_multindex
      cost  revenue
Q1 A   250     100
   B   150     250
   C   100     300
Q2 A   150     200
   B   300     175
   C   220     225
```

```
>>> df.le(df_multindex, level=1)
      cost  revenue
Q1 A   True    True
   B   True    True
   C   True    True
Q2 A  False    True
   B   True   False
   C   True   False
```

```
eval(self, expr: str, *, inplace: bool = False, **kwargs) -> Any | None from pandas.core.frame
Evaluate a string describing operations on DataFrame columns.
```

```
.. warning::
```

This method can run arbitrary code which can make you vulnerable to code injection if you pass user input to this function.

Operates on columns only, not specific rows or elements. This allows ``eval`` to run arbitrary code, which can make you vulnerable to code injection if you pass user input to this function.

Parameters

`expr` : str

The expression string to evaluate.

You can refer to variables

in the environment by prefixing them with an '@' character like

``@a + b``.

You can refer to column names that are not valid Python variable names by surrounding them in backticks. Thus, column names containing spaces or punctuation (besides underscores) or starting with digits must be surrounded by backticks. (For example, a column named "Area (cm^2)" would be referenced as ``Area (cm^2)``). Column names which are Python keywords (like "if", "for", "import", etc) cannot be used.

For example, if one of your columns is called ``a a`` and you want to sum it with ``b``, your query should be ``a a` + b``.

See the documentation for `:func:`eval`` for full details of supported operations and functions in the expression string.

`inplace` : bool, default False

If the expression contains an assignment, whether to perform the operation inplace and mutate the existing DataFrame. Otherwise, a new DataFrame is returned.

`**kwargs`

See the documentation for `:func:`eval`` for complete details on the keyword arguments accepted by `:meth:`~pandas.DataFrame.eval``.

Returns

ndarray, scalar, pandas object, or None

The result of the evaluation or None if ``inplace=True``.

See Also

`DataFrame.query` : Evaluates a boolean expression to query the columns of a frame.

`DataFrame.assign` : Can evaluate an expression or function to create new values for a column.

`eval` : Evaluate a Python expression as a string using various backends.

Notes

For more details see the API documentation for `:func:`~eval``.

For detailed examples see `:ref:`enhancing performance with eval <enhancingperf.eval>``.

Examples

```
>>> df = pd.DataFrame(
...     {"A": range(1, 6), "B": range(10, 0, -2), "C&C": range(10, 5, -1)}
... )
```

```
>>> df
```

```
   A  B  C&C
0  1 10   10
1  2  8    9
2  3  6    8
3  4  4    7
4  5  2    6
```

```
>>> df.eval("A + B")
```

```
0    11
1    10
2     9
3     8
4     7
```

```
dtype: int64
```

Assignment is allowed though by default the original DataFrame is not modified.

```
>>> df.eval("D = A + B")
   A  B  C&C  D
0  1 10   10 11
1  2  8    9 10
2  3  6    8  9
3  4  4    7  8
4  5  2    6  7
>>> df
   A  B  C&C
0  1 10   10
1  2  8    9
2  3  6    8
3  4  4    7
4  5  2    6
```

Multiple columns can be assigned to using multi-line expressions:

```
>>> df.eval(
...     '''
...     D = A + B
...     E = A - B
...     '''
... )
   A  B  C&C  D  E
0  1 10   10 11 -9
1  2  8    9 10 -6
2  3  6    8  9 -3
3  4  4    7  8  0
4  5  2    6  7  3
```

For columns with spaces or other disallowed characters in their name, you can use backtick quoting.

```
>>> df.eval("B * `C&C`")
0    100
1     72
2     48
3     28
4     12
dtype: int64
```

Local variables shall be explicitly referenced using ``@`` character in front of the name:

```
>>> local_var = 2
>>> df.eval("@local_var * A")
0     2
1     4
2     6
3     8
4    10
Name: A, dtype: int64
```

`explode(self, column: IndexLabel, ignore_index: bool = False) -> DataFrame` from `pandas.core.`
Transform each element of a list-like to a row, replicating index values.

Parameters

`column` : IndexLabel

Column(s) to explode.

For multiple columns, specify a non-empty list with each element be str or tuple, and all specified columns their list-like data on same row of the frame must have matching length.

`ignore_index` : bool, default False

If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

DataFrame

Exploded lists to rows of the subset columns;
index will be duplicated for these rows.

Raises

ValueError :

- * If columns of the frame are not unique.
- * If specified columns to explode is empty list.
- * If specified columns to explode have not matching count of elements rowwise in the frame.

See Also

DataFrame.unstack : Pivot a level of the (necessarily hierarchical) index labels.

DataFrame.melt : Unpivot a DataFrame from wide format to long format.

Series.explode : Explode a DataFrame from list-like columns to long format.

Notes

This routine will explode list-likes including lists, tuples, sets, Series, and np.ndarray. The result dtype of the subset rows will be object. Scalars will be returned unchanged, and empty list-likes will result in a np.nan for that row. In addition, the ordering of rows in the output will be non-deterministic when exploding sets.

Reference :ref:`the user guide <reshaping.explode>` for more examples.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "A": [[0, 1, 2], "foo", [], [3, 4]],
...         "B": 1,
...         "C": ["a", "b", "c"], np.nan, [], ["d", "e"]],
...     )
>>> df
```

	A	B	C
0	[0, 1, 2]	1	[a, b, c]
1	foo	1	NaN
2	[]	1	[]
3	[3, 4]	1	[d, e]

Single-column explode.

```
>>> df.explode("A")
```

	A	B	C
0	0	1	[a, b, c]
0	1	1	[a, b, c]
0	2	1	[a, b, c]
1	foo	1	NaN
2	NaN	1	[]
3	3	1	[d, e]
3	4	1	[d, e]

Multi-column explode.

```
>>> df.explode(list("AC"))
```

	A	B	C
0	0	1	a
0	1	1	b
0	2	1	c
1	foo	1	NaN
2	NaN	1	NaN
3	3	1	d
3	4	1	e

`floordiv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from
Get Integer division of dataframe and other, element-wise (binary operator ``floordiv``).

Equivalent to ``dataframe // other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``rfloordiv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

other : scalar, sequence, Series, dict or DataFrame

Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
level : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.
fill_value : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

DataFrame.add : Add DataFrames.

DataFrame.sub : Subtract DataFrames.

DataFrame.mul : Multiply DataFrames.

DataFrame.div : Divide DataFrames (float division).

DataFrame.truediv : Divide DataFrames (float division).

DataFrame.floordiv : Divide DataFrames (integer division).

DataFrame.mod : Calculate modulo (remainder after division).

DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
--	--------	---------

circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle    -1    358
triangle     2    178
rectangle     3    358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle    -1    359
triangle     2    179
rectangle     3    359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0    720
triangle     0    360
rectangle     0    720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0      0
triangle     6    360
rectangle    12   1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle     3
rectangle     4

>>> df * other
      angles  degrees
circle      0    NaN
triangle     9    NaN
rectangle    16    NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0    0.0
triangle     9    0.0
rectangle    16    0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                              'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0    360
  triangle     3    180
  rectangle     4    360
B square      4    360
  pentagon     5    540
  hexagon      6    720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN    1.0
  triangle  1.0    1.0
  rectangle  1.0    1.0
B square     0.0    0.0
  pentagon   0.0    0.0
  hexagon    0.0    0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4 36
1  9 49
2 16 64
3 25 81
```

`ge(self, other, axis: Axis = 'columns', level=None)` -> DataFrame from pandas.core.frame.DataFrame
Get Greater than or equal to of dataframe and other, element-wise (binary operator ``ge``)

Among flexible wrappers (``eq``, ``ne``, ``le``, ``lt``, ``ge``, ``gt``) to comparison operators.

Equivalent to ``==``, ``!=``, ``<=``, ``<``, ``>=``, ``>`` with support to choose axis (rows or columns) and level for comparison.

Parameters

`other` : scalar, sequence, Series, or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}, default 'columns'
Whether to compare by the index (0 or 'index') or columns (1 or 'columns').
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns

DataFrame of bool
Result of the comparison.

See Also

`DataFrame.eq` : Compare DataFrames for equality elementwise.
`DataFrame.ne` : Compare DataFrames for inequality elementwise.
`DataFrame.le` : Compare DataFrames for less than inequality or equality elementwise.
`DataFrame.lt` : Compare DataFrames for strictly less than inequality elementwise.
`DataFrame.ge` : Compare DataFrames for greater than inequality or equality elementwise.
`DataFrame.gt` : Compare DataFrames for strictly greater than inequality elementwise.

Notes

Mismatched indices will be unioned together.
``NaN`` values are considered different (i.e. ``NaN` != `NaN``).

Examples

```
>>> df = pd.DataFrame({'cost': [250, 150, 100],
...                   'revenue': [100, 250, 300]},
...                   index=['A', 'B', 'C'])
>>> df
   cost  revenue
A   250     100
B   150     250
C   100     300
```

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
   cost  revenue
A  False     True
B  False     False
C   True     False

>>> df.eq(100)
   cost  revenue
A  False     True
B  False     False
C   True     False
```

When `other` is a `:class:`Series``, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
      cost  revenue
A    True    True
B    True    False
C   False    True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
      cost  revenue
A    True    False
B    True    True
C    True    True
D    True    True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
      cost  revenue
A    True    True
B   False   False
C   False   False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
      cost  revenue
A    True    False
B   False    True
C    True    False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                       index=['A', 'B', 'C', 'D'])
>>> other
      revenue
A         300
B         250
C         100
D         150
```

```
>>> df.gt(other)
      cost  revenue
A   False   False
B   False   False
C   False    True
D   False   False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
...                              'revenue': [100, 250, 300, 200, 175, 225]},
...                              index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                    ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
      cost  revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225

>>> df.le(df_multindex, level=1)
      cost  revenue
Q1 A    True    True
   B    True    True
   C    True    True
Q2 A   False    True
   B    True   False
   C    True   False
```



```

groupby(
    self,
    by=None,
    level: IndexLabel | None = None,
    *,
    as_index: bool = True,
    sort: bool = True,
    group_keys: bool = True,
    observed: bool = True,
    dropna: bool = True
) -> DataFrameGroupBy from pandas.core.frame.DataFrame
    Group DataFrame using a mapper or by a Series of columns.

```

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters

by : mapping, function, label, pd.Grouper or list of such

Used to determine the groups for the groupby.

If ``by`` is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see ``.align()`` method). If a list or ndarray of length equal to the number of rows is passed (see the `groupby` user guide <https://pandas.pydata.org/pandas-docs/stable/user_guide/groupby.html#splitting-an-object>), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in ``self``.

Notice that a tuple is interpreted as a (single) key.

level : int, level name, or sequence of such, default None

If the axis is a MultiIndex (hierarchical), group by a particular level or levels. Do not specify both ``by`` and ``level``.

as_index : bool, default True

Return object with group labels as the

index. Only relevant for DataFrame input. as_index=False is effectively "SQL-style" grouped output. This argument has no effect on filtrations (see the `filtrations` in the user guide

<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>`_), such as ``head()`, ``tail()`, ``nth()`` and in transformations (see the `transformations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>`_).

sort : bool, default True

Sort group keys. Get better performance by turning this off.

Note this does not influence the order of observations within each group. Groupby preserves the order of rows within each group. If False, the groups will appear in the same order as they did in the original DataFrame.

This argument has no effect on filtrations (see the `filtrations` in the user guide

<https://pandas.pydata.org/docs/dev/user_guide/groupby.html#filtration>`_), such as ``head()`, ``tail()`, ``nth()`` and in transformations (see the `transformations` in the user guide <https://pandas.pydata.org/docs/dev/user_guide/groupby.html#transformation>`_).

.. versionchanged:: 2.0.0

Specifying ``sort=False`` with an ordered categorical grouper will no longer sort the values.

group_keys : bool, default True

When calling apply and the ``by`` argument produces a like-indexed (i.e. :ref:`a transform <groupby.transform>` result, add group keys to index to identify pieces. By default group keys are not included when the result's index (and column) labels match the inputs, and are included otherwise.

.. versionchanged:: 2.0.0

``group\_keys`` now defaults to ``True``.

observed : bool, default True

This only applies if any of the groupers are Categoricals.

If True: only show observed values for categorical groupers.

If False: show all values for categorical groupers.

```
.. versionchanged:: 3.0.0
```

The default value is now ``True``.

dropna : bool, default True

If True, and if group keys contain NA values, NA values together with row/column will be dropped.

If False, NA values will also be treated as the key in groups.

Returns

pandas.api.typing.DataFrameGroupBy

Returns a groupby object that contains information about the groups.

See Also

resample : Convenience method for frequency conversion and resampling of time series.

Notes

See the `user guide

<<https://pandas.pydata.org/pandas-docs/stable/groupby.html>>`__ for more detailed usage and examples, including splitting an object into groups, iterating through groups, selecting a group, aggregation, and more.

The implementation of groupby is hash-based, meaning in particular that objects that compare as equal will be considered to be in the same group. An exception to this is that pandas has special handling of NA values: any NA values will be collapsed to a single group, regardless of how they compare. See the user guide linked above for more details.

Examples

>>> df = pd.DataFrame(
... {
... "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
... "Max Speed": [380.0, 370.0, 24.0, 26.0],
... }
...)
>>> df

	Animal	Max Speed
0	Falcon	380.0
1	Falcon	370.0
2	Parrot	24.0
3	Parrot	26.0

>>> df.groupby(["Animal"]).mean()
Max Speed

Animal	Max Speed
Falcon	375.0
Parrot	25.0

****Hierarchical Indexes****

We can groupby different levels of a hierarchical index using the `level` parameter:

>>> arrays = [
... ["Falcon", "Falcon", "Parrot", "Parrot"],
... ["Captive", "Wild", "Captive", "Wild"],
...]
>>> index = pd.MultiIndex.from_arrays(arrays, names=("Animal", "Type"))
>>> df = pd.DataFrame({"Max Speed": [390.0, 350.0, 30.0, 20.0]}, index=index)
>>> df

		Max Speed
Animal	Type	
Falcon	Captive	390.0
	Wild	350.0
Parrot	Captive	30.0
	Wild	20.0

>>> df.groupby(level=0).mean()
Max Speed

Animal	Max Speed
Falcon	370.0
Parrot	25.0

>>> df.groupby(level="Type").mean()

	Max Speed
Type	
Captive	210.0
Wild	185.0

We can also choose to include NA in group keys or not by setting `dropna` parameter, the default setting is `True`.

```
>>> arr = [[1, 2, 3], [1, None, 4], [2, 1, 3], [1, 2, 2]]
>>> df = pd.DataFrame(arr, columns=["a", "b", "c"])
```

```
>>> df.groupby(by=["b"]).sum()
a      c
b
1.0 2    3
2.0 2    5
```

```
>>> df.groupby(by=["b"], dropna=False).sum()
a      c
b
1.0 2    3
2.0 2    5
NaN 1    4
```

```
>>> arr = [["a", 12, 12], [None, 12.3, 33.0], ["b", 12.3, 123], ["a", 1, 1]]
>>> df = pd.DataFrame(arr, columns=["a", "b", "c"])
```

```
>>> df.groupby(by="a").sum()
b      c
a
a   13.0  13.0
b   12.3 123.0
```

```
>>> df.groupby(by="a", dropna=False).sum()
b      c
a
a   13.0  13.0
b   12.3 123.0
NaN 12.3  33.0
```

When using ``.apply()`, use ``group_keys`` to include or exclude the group keys. The ``group_keys`` argument defaults to ``True`` (include).

```
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
...         "Max Speed": [380.0, 370.0, 24.0, 26.0],
...     }
... )
>>> df.groupby("Animal", group_keys=True)[["Max Speed"]].apply(lambda x: x)
Max Speed
Animal
Falcon 0    380.0
      1    370.0
Parrot 2     24.0
      3     26.0
```

```
>>> df.groupby("Animal", group_keys=False)[["Max Speed"]].apply(lambda x: x)
Max Speed
0    380.0
1    370.0
2     24.0
3     26.0
```

`gt(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame`
Get Greater than of dataframe and other, element-wise (binary operator `gt`).

Among flexible wrappers (`eq`, `ne`, `le`, `lt`, `ge`, `gt`) to comparison operators.

Equivalent to `==`, `!=`, `<=`, `<`, `>=`, `>` with support to choose axis (rows or columns) and level for comparison.

Parameters

`other` : scalar, sequence, Series, or DataFrame

Any single or multiple element data structure, or list-like object.

axis : {0 or 'index', 1 or 'columns'}, default 'columns'
Whether to compare by the index (0 or 'index') or columns
(1 or 'columns').
level : int or label
Broadcast across a level, matching Index values on the passed
MultiIndex level.

Returns

DataFrame of bool
Result of the comparison.

See Also

DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality
or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than
inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality
or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than
inequality elementwise.

Notes

Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples

>>> df = pd.DataFrame({'cost': [250, 150, 100],
... 'revenue': [100, 250, 300]},
... index=['A', 'B', 'C'])
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300

Comparison with a scalar, using either the operator or method:

```
>>> df == 100  
   cost  revenue  
A  False      True  
B  False      False  
C   True      False  
  
>>> df.eq(100)  
   cost  revenue  
A  False      True  
B  False      False  
C   True      False
```

When `other` is a :class:`Series`, the columns of a DataFrame are aligned
with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])  
   cost  revenue  
A   True      True  
B   True      False  
C  False      True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')  
   cost  revenue  
A   True      False  
B   True      True  
C   True      True  
D   True      True
```

When comparing to an arbitrary sequence, the number of columns must
match the number elements in `other`:

```
>>> df == [250, 100]
```

	cost	revenue
A	True	True
B	False	False
C	False	False

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
      cost revenue
A    True    False
B   False     True
C    True    False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                        index=['A', 'B', 'C', 'D'])
>>> other
      revenue
A         300
B         250
C         100
D         150

>>> df.gt(other)
      cost revenue
A   False    False
B   False    False
C   False     True
D   False    False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
...                               'revenue': [100, 250, 300, 200, 175, 225]},
...                               index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                       ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
      cost revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225

>>> df.le(df_multindex, level=1)
      cost revenue
Q1 A    True     True
   B    True     True
   C    True     True
Q2 A   False     True
   B    True     False
   C    True     False
```

```
hist = hist_frame(
    data: DataFrame,
    column: IndexLabel | None = None,
    by=None,
    grid: bool = True,
    xlabelsize: int | None = None,
    xrot: float | None = None,
    ylabelsize: int | None = None,
    yrot: float | None = None,
    ax=None,
    sharex: bool = False,
    sharey: bool = False,
    figsize: tuple[int, int] | None = None,
    layout: tuple[int, int] | None = None,
    bins: int | Sequence[int] = 10,
    backend: str | None = None,
    legend: bool = False,
    **kwargs
) from pandas.plotting
    Make a histogram of the DataFrame's columns.
```

A `'histogram'` is a representation of the distribution of data.

This function calls `:meth:`matplotlib.pyplot.hist``, on each series in the DataFrame, resulting in one histogram per column.

.. _histogram: <https://en.wikipedia.org/wiki/Histogram>

Parameters

`data` : DataFrame
The pandas object holding the data.

`column` : str or sequence, optional
If passed, will be used to limit data to a subset of columns.

`by` : object, optional
If passed, then used to form histograms for separate groups.

`grid` : bool, default True
Whether to show axis grid lines.

`xlabelsize` : int, default None
If specified changes the x-axis label size.

`xrot` : float, default None
Rotation of x axis labels. For example, a value of 90 displays the x labels rotated 90 degrees clockwise.

`ylabelsize` : int, default None
If specified changes the y-axis label size.

`yrot` : float, default None
Rotation of y axis labels. For example, a value of 90 displays the y labels rotated 90 degrees clockwise.

`ax` : Matplotlib axes object, default None
The axes to plot the histogram on.

`sharex` : bool, default True if ax is None else False
In case subplots=True, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in.
Note that passing in both an ax and sharex=True will alter all x axis labels for all subplots in a figure.

`sharey` : bool, default False
In case subplots=True, share y axis and set some y axis labels to invisible.

`figsize` : tuple, optional
The size in inches of the figure to create. Uses the value in ``matplotlib.rcParams`` by default.

`layout` : tuple, optional
Tuple of (rows, columns) for the layout of the histograms.

`bins` : int or sequence, default 10
Number of histogram bins to be used. If an integer is given, `bins + 1` bin edges are calculated and returned. If bins is a sequence, gives bin edges, including left edge of first bin and right edge of last bin. In this case, bins is returned unmodified.

`backend` : str, default None
Backend to use instead of the backend specified in the option ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to specify the ``plotting.backend`` for the whole session, set ``pd.options.plotting.backend``.

`legend` : bool, default False
Whether to show the legend.

****kwargs**
All other plotting keyword arguments to be passed to `:meth:`matplotlib.pyplot.hist``.

Returns

np.ndarray
2D NumPy Array of `:class:`matplotlib.axes.Axes``.

See Also

`matplotlib.pyplot.hist` : Plot a histogram using matplotlib.

Examples

This example draws a histogram based on the length and width of some animals, displayed in three bins

```
.. plot::  
    :context: close-figs
```

```
>>> data = {
...     "length": [1.5, 0.5, 1.2, 0.9, 3],
...     "width": [0.7, 0.2, 0.15, 0.2, 1.1],
... }
>>> index = ["pig", "rabbit", "duck", "chicken", "horse"]
>>> df = pd.DataFrame(data, index=index)
>>> hist = df.hist(bins=3)
```

`idxmax(self, axis: Axis = 0, skipna: bool = True, numeric_only: bool = False) -> Series from`
Return index of first occurrence of maximum over requested axis.

NA/null values are excluded.

Parameters

axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.
skipna : bool, default True
Exclude NA/null values. If the entire DataFrame is NA,
or if ``skipna=False`` and there is an NA value, this method
will raise a ``ValueError``.
numeric_only : bool, default False
Include only ``float``, ``int`` or ``boolean`` data.

Returns

Series

Indexes of maxima along the specified axis.

Raises

ValueError

* If the row/column is empty

See Also

Series.idxmax : Return index of the maximum element.

Notes

This method is the DataFrame version of ``ndarray.argmax``.

Examples

Consider a dataset containing food consumption in Argentina.

```
>>> df = pd.DataFrame(
...     {
...         "consumption": [10.51, 103.11, 55.48],
...         "co2_emissions": [37.2, 19.66, 1712],
...     },
...     index=["Pork", "Wheat Products", "Beef"],
... )
```

```
>>> df
          consumption  co2_emissions
Pork              10.51             37.20
Wheat Products    103.11             19.66
Beef              55.48            1712.00
```

By default, it returns the index for the maximum value in each column.

```
>>> df.idxmax()
consumption    Wheat Products
co2_emissions           Beef
dtype: str
```

To return the index for the maximum value in each row, use ``axis="columns"``.

```
>>> df.idxmax(axis="columns")
Pork          co2_emissions
Wheat Products    consumption
Beef          co2_emissions
dtype: str
```

`idxmin(self, axis: Axis = 0, skipna: bool = True, numeric_only: bool = False) -> Series from`
Return index of first occurrence of minimum over requested axis.

NA/null values are excluded.

Parameters

axis : {{0 or 'index', 1 or 'columns'}}, default 0
The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.
skipna : bool, default True
Exclude NA/null values. If the entire DataFrame is NA,
or if ``skipna=False`` and there is an NA value, this method
will raise a ``ValueError``.
numeric_only : bool, default False
Include only ``float``, ``int`` or ``boolean`` data.

Returns

Series

Indexes of minima along the specified axis.

Raises

ValueError

* If the row/column is empty

See Also

Series.idxmin : Return index of the minimum element.

Notes

This method is the DataFrame version of ``ndarray.argmin``.

Examples

Consider a dataset containing food consumption in Argentina.

```
>>> df = pd.DataFrame(  
...     {  
...         "consumption": [10.51, 103.11, 55.48],  
...         "co2_emissions": [37.2, 19.66, 1712],  
...     },  
...     index=["Pork", "Wheat Products", "Beef"],  
... )
```

```
>>> df  
          consumption  co2_emissions  
Pork              10.51           37.20  
Wheat Products    103.11           19.66  
Beef              55.48          1712.00
```

By default, it returns the index for the minimum value in each column.

```
>>> df.idxmin()  
consumption      Pork  
co2_emissions   Wheat Products  
dtype: str
```

To return the index for the minimum value in each row, use ``axis="columns"``.

```
>>> df.idxmin(axis="columns")  
Pork      consumption  
Wheat Products  co2_emissions  
Beef      consumption  
dtype: str
```

```
info(  
    self,  
    verbose: bool | None = None,  
    buf: WriteBuffer[str] | None = None,  
    max_cols: int | None = None,  
    memory_usage: bool | str | None = None,  
    show_counts: bool | None = None  
) -> None from pandas.core.frame.DataFrame  
Print a concise summary of a DataFrame.
```

This method prints information about a DataFrame including the index dtype and columns, non-NA values and memory usage.

Parameters

`verbose` : bool, optional
Whether to print the full summary. By default, the setting in ```pandas.options.display.max_info_columns``` is followed.

`buf` : writable buffer, defaults to `sys.stdout`
Where to send the output. By default, the output is printed to `sys.stdout`. Pass a writable buffer if you need to further process the output.

`max_cols` : int, optional
When to switch from the verbose to the truncated output. If the DataFrame has more than `max_cols` columns, the truncated output is used. By default, the setting in ```pandas.options.display.max_info_columns``` is used.

`memory_usage` : bool, str, optional
Specifies whether total memory usage of the DataFrame elements (including the index) should be displayed. By default, this follows the ```pandas.options.display.memory_usage``` setting.

True always show memory usage. False never shows memory usage. A value of 'deep' is equivalent to "True with deep introspection". Memory usage is shown in human-readable units (base-2 representation). Without deep introspection a memory estimation is made based in column dtype and number of rows assuming values consume the same memory amount for corresponding dtypes. With deep memory introspection, a real memory usage calculation is performed at the cost of computational resources. See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.

`show_counts` : bool, optional
Whether to show the non-null counts. By default, this is shown only if the DataFrame is smaller than ```pandas.options.display.max_info_rows``` and ```pandas.options.display.max_info_columns```. A value of True always shows the counts, and False never shows the counts.

Returns

None
This method prints a summary of a DataFrame and returns None.

See Also

`DataFrame.describe`: Generate descriptive statistics of DataFrame columns.
`DataFrame.memory_usage`: Memory usage of DataFrame columns.

Examples

```
>>> int_values = [1, 2, 3, 4, 5]
>>> text_values = ["alpha", "beta", "gamma", "delta", "epsilon"]
>>> float_values = [0.0, 0.25, 0.5, 0.75, 1.0]
>>> df = pd.DataFrame(
...     {
...         "int_col": int_values,
...         "text_col": text_values,
...         "float_col": float_values,
...     }
... )
>>> df
   int_col text_col  float_col
0         1   alpha        0.00
1         2   beta         0.25
2         3  gamma         0.50
3         4  delta         0.75
4         5 epsilon         1.00
```

Prints information of all columns:

```
>>> df.info(verbose=True)
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   int_col      5 non-null        int64
```

```

1  text_col  5 non-null    str
2  float_col 5 non-null   float64
dtypes: float64(1), int64(1), str(1)
memory usage: 278.0 bytes

```

Prints a summary of columns count and its dtypes but not per column information:

```

>>> df.info(verbose=False)
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Columns: 3 entries, int_col to float_col
dtypes: float64(1), int64(1), str(1)
memory usage: 278.0 bytes

```

Pipe output of DataFrame.info to buffer instead of sys.stdout, get buffer content and writes to a text file:

```

>>> import io
>>> buffer = io.StringIO()
>>> df.info(buf=buffer)
>>> s = buffer.getvalue()
>>> with open("df_info.txt", "w", encoding="utf-8") as f: # doctest: +SKIP
...     f.write(s)
260

```

The `memory\_usage` parameter allows deep introspection mode, specially useful for big DataFrames and fine-tune memory optimization:

```

>>> random_strings_array = np.random.choice(["a", "b", "c"], 10**6)
>>> df = pd.DataFrame(
...     {
...         "column_1": np.random.choice(["a", "b", "c"], 10**6),
...         "column_2": np.random.choice(["a", "b", "c"], 10**6),
...         "column_3": np.random.choice(["a", "b", "c"], 10**6),
...     }
... )
>>> df.info()
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   column_1    1000000 non-null   str
1   column_2    1000000 non-null   str
2   column_3    1000000 non-null   str
dtypes: str(3)
memory usage: 25.7 MB

```

```

>>> df.info(memory_usage="deep")
<class 'pandas.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   column_1    1000000 non-null   str
1   column_2    1000000 non-null   str
2   column_3    1000000 non-null   str
dtypes: str(3)
memory usage: 25.7 MB

```

```

insert(
    self,
    loc: int,
    column: Hashable,
    value: object,
    allow_duplicates: bool | lib.NoDefault = <no_default>
) -> None from pandas.core.frame.DataFrame
Insert column into DataFrame at specified location.

```

Raises a ValueError if `column` is already contained in the DataFrame, unless `allow\_duplicates` is set to True.

Parameters

```

-----
loc : int
    Insertion index. Must verify 0 <= loc <= len(columns).

```

column : str, number, or hashable object
Label of the inserted column.
value : Scalar, Series, or array-like
Content of the inserted column.
allow_duplicates : bool, optional, default lib.no_default
Allow duplicate column labels to be created.

See Also

Index.insert : Insert new item by index.

Examples

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df
 col1 col2
0 1 3
1 2 4
>>> df.insert(1, "newcol", [99, 99])
>>> df
 col1 newcol col2
0 1 99 3
1 2 99 4
>>> df.insert(0, "col1", [100, 100], allow_duplicates=True)
>>> df
 col1 col1 newcol col2
0 100 1 99 3
1 100 2 99 4

Notice that pandas uses index alignment in case of `value` from type `Series`:

>>> df.insert(0, "col0", pd.Series([5, 6], index=[1, 2]))
>>> df
 col0 col1 col1 newcol col2
0 NaN 100 1 99 3
1 5.0 100 2 99 4

isetitem(self, loc, value) -> None from pandas.core.frame.DataFrame
Set the given value in the column with position `loc`.

This is a positional analogue to ``\_\_setitem\_\_``.

Parameters

loc : int or sequence of ints
Index position for the column.
value : scalar or arraylike
Value(s) for the column.

See Also

DataFrame.iloc : Purely integer-location based indexing for selection by position.

Notes

``frame.isetitem(loc, value)`` is an in-place method as it will modify the DataFrame in place (not returning a new object). In contrast to ``frame.iloc[:, i] = value`` which will try to update the existing values in place, ``frame.isetitem(loc, value)`` will not update the values of the column itself in place, it will instead insert a new array.

In cases where ``frame.columns`` is unique, this is equivalent to
``frame[frame.columns[i]] = value``.

Examples

>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.isetitem(1, [5, 6])
>>> df
 A B
0 1 5
1 2 6

isin(self, values: Series | DataFrame | Sequence | Mapping) -> DataFrame from pandas.core.frame.DataFrame
Whether each element in the DataFrame is contained in values.

Parameters

values : iterable, Series, DataFrame or dict

The result will only be true at a location if all the labels match. If `values` is a Series, that's the index. If `values` is a dict, the keys must be the column names, which must match. If `values` is a DataFrame, then both the index and column labels must match.

Returns

DataFrame

DataFrame of booleans showing whether each element in the DataFrame is contained in values.

See Also

DataFrame.eq: Equality test for DataFrame.

Series.isin: Equivalent method on Series.

Series.str.contains: Test if pattern or regex is contained within a string of a Series or Index.

Notes

`\_\_iter\_\_` is used (and not `\_\_contains\_\_`) to iterate over values when checking if it contains the elements in DataFrame.

Examples

```
>>> df = pd.DataFrame(
...     {"num_legs": [2, 4], "num_wings": [2, 0]}, index=["falcon", "dog"]
... )
>>> df
```

	num_legs	num_wings
falcon	2	2
dog	4	0

When `values` is a list check whether every value in the DataFrame is present in the list (which animals have 0 or 2 legs or wings)

```
>>> df.isin([0, 2])
```

	num_legs	num_wings
falcon	True	True
dog	False	True

To check if `values` is *not* in the DataFrame, use the `~` operator:

```
>>> ~df.isin([0, 2])
```

	num_legs	num_wings
falcon	False	False
dog	True	False

When `values` is a dict, we can pass values to check for each column separately:

```
>>> df.isin({"num_wings": [0, 3]})
```

	num_legs	num_wings
falcon	False	False
dog	False	True

When `values` is a Series or DataFrame the index and column must match. Note that 'falcon' does not match based on the number of legs in other.

```
>>> other = pd.DataFrame(
...     {"num_legs": [8, 3], "num_wings": [0, 2]}, index=["spider", "falcon"]
... )
>>> df.isin(other)
```

	num_legs	num_wings
falcon	False	True
dog	False	False

isna(self) -> DataFrame from pandas.core.frame.DataFrame
Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as None or :attr:`numpy.NaN`, gets mapped to True

values.
Everything else gets mapped to False values. Characters such as empty strings `` or :attr:`numpy.inf` are not considered NA values.

Returns

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See Also

Series.isnull : Alias of isna.

DataFrame.isnull : Alias of isna.

Series.notna : Boolean inverse of isna.

DataFrame.notna : Boolean inverse of isna.

Series.dropna : Omit axes labels with missing values.

DataFrame.dropna : Omit axes labels with missing values.

isna : Top-level isna.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
```

	age	born	name	toy
0	5.0	NaT	Alfred	NaN
1	6.0	1939-05-27	Batman	Batmobile
2	NaN	1940-04-25		Joker

```
>>> df.isna()
```

	age	born	name	toy
0	False	True	False	True
1	False	False	False	False
2	True	False	False	False

Show which entries in a Series are NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
```

0	5.0
1	6.0
2	NaN

dtype: float64

```
>>> ser.isna()
```

0	False
1	False
2	True

dtype: bool

isnull(self) -> DataFrame from pandas.core.frame.DataFrame
DataFrame.isnull is an alias for DataFrame.isna.

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as None or :attr:`numpy.NaN`, gets mapped to True values. Everything else gets mapped to False values. Characters such as empty strings `` or :attr:`numpy.inf` are not considered NA values.

Returns

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See Also

Series.isnull : Alias of isna.
DataFrame.isnull : Alias of isna.
Series.notna : Boolean inverse of isna.
DataFrame.notna : Boolean inverse of isna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
isna : Top-level isna.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born      name      toy
0  5.0       NaT  Alfred      NaN
1  6.0  1939-05-27  Batman  Batmobile
2  NaN  1940-04-25      Joker

>>> df.isna()
   age  born  name  toy
0 False  True False  True
1 False False False False
2  True False False False
```

Show which entries in a Series are NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64

>>> ser.isna()
0    False
1    False
2     True
dtype: bool
```

items(self) -> Iterable[tuple[Hashable, Series]] from pandas.core.frame.DataFrame
Iterate over (column name, Series) pairs.

Iterates over the DataFrame columns, returning a tuple with the column name and the content as a Series.

Yields

label : object
The column names for the DataFrame being iterated over.
content : Series
The column entries belonging to each label, as a Series.

See Also

DataFrame.iterrows : Iterate over DataFrame rows as (index, Series) pairs.
DataFrame.itertuples : Iterate over DataFrame rows as namedtuples of the values.

Examples

```

-----
>>> df = pd.DataFrame(
...     {
...         "species": ["bear", "bear", "marsupial"],
...         "population": [1864, 22000, 80000],
...     },
...     index=["panda", "polar", "koala"],
... )
>>> df
   species  population
panda   bear       1864
polar   bear      22000
koala  marsupial  80000
>>> for label, content in df.items():
...     print(f"label: {label}")
...     print(f"content: {content}", sep="\n")
label: species
content:
panda      bear
polar      bear
koala  marsupial
Name: species, dtype: str
label: population
content:
panda      1864
polar     22000
koala     80000
Name: population, dtype: int64

```

`iterrows(self)` -> `Iterable[tuple[Hashable, Series]]` from `pandas.core.frame.DataFrame`
Iterate over `DataFrame` rows as (index, `Series`) pairs.

Yields

```

-----
index : label or tuple of label
    The index of the row. A tuple for a `MultiIndex`.
data : Series
    The data of the row as a Series.

```

See Also

```

-----
DataFrame.itertuples : Iterate over DataFrame rows as namedtuples of the values.
DataFrame.items : Iterate over (column name, Series) pairs.

```

Notes

1. Because `iterrows` returns a `Series` for each row, it does **not** preserve dtypes across the rows (dtypes are preserved across columns for `DataFrames`).

To preserve dtypes while iterating over the rows, it is better to use `itertuples` which returns `namedtuples` of the values and which is generally faster than `iterrows`.
2. You should **never modify** something you are iterating over. This is not guaranteed to work in all cases. Depending on the data types, the iterator returns a copy and not a view, and writing to it will have no effect.

Examples

```

-----
>>> df = pd.DataFrame([[1, 1.5]], columns=["int", "float"])
>>> row = next(df.iterrows())[1]
>>> row
int      1.0
float    1.5
Name: 0, dtype: float64
>>> print(row["int"].dtype)
float64
>>> print(df["int"].dtype)
int64

```

`itertuples(self, index: bool = True, name: str | None = 'Pandas')` -> `Iterable[tuple[Any, ...]]`
Iterate over `DataFrame` rows as `namedtuples`.

Parameters

index : bool, default True
If True, return the index as the first element of the tuple.
name : str or None, default "Pandas"
The name of the returned namedtuples or None to return regular tuples.

Returns

iterator

An object to iterate over namedtuples for each row in the DataFrame with the first field possibly being the index and following fields being the column values.

See Also

DataFrame.iterrows : Iterate over DataFrame rows as (index, Series) pairs.

DataFrame.items : Iterate over (column name, Series) pairs.

Notes

The column names will be renamed to positional names if they are invalid Python identifiers, repeated, or start with an underscore.

Examples

>>> df = pd.DataFrame(
... {"num_legs": [4, 2], "num_wings": [0, 2]}, index=["dog", "hawk"]
...)
>>> df

```

      num_legs  num_wings
dog           4          0
hawk          2          2

```

>>> for row in df.itertuples():

... print(row)

Pandas(Index='dog', num_legs=4, num_wings=0)

Pandas(Index='hawk', num_legs=2, num_wings=2)

By setting the `index` parameter to False we can remove the index as the first element of the tuple:

>>> for row in df.itertuples(index=False):

... print(row)

Pandas(num_legs=4, num_wings=0)

Pandas(num_legs=2, num_wings=2)

With the `name` parameter set we set a custom name for the yielded namedtuples:

>>> for row in df.itertuples(name="Animal"):

... print(row)

Animal(Index='dog', num_legs=4, num_wings=0)

Animal(Index='hawk', num_legs=2, num_wings=2)

```

join(
    self,
    other: DataFrame | Series | Iterable[DataFrame | Series],
    on: IndexLabel | None = None,
    how: MergeHow = 'left',
    lsuffix: str = '',
    rsuffix: str = '',
    sort: bool = False,
    validate: JoinValidate | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Join columns of another DataFrame.

```

Join columns with `other` DataFrame either on index or on a key column. Efficiently join multiple DataFrame objects by index at once by passing a list.

Parameters

other : DataFrame, Series, or a list containing any combination of them
Index should be similar to one of the columns in this one. If a Series is passed, its name attribute must be set, and that will be used as the column name in the resulting joined DataFrame.


```

on : str, list of str, or array-like, optional
    Column or index level name(s) in the caller to join on the index
    in `other`, otherwise joins index-on-index. If multiple
    values given, the `other` DataFrame must have a MultiIndex. Can
    pass an array as the join key if it is not already contained in
    the calling DataFrame. Like an Excel VLOOKUP operation.
how : {'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti'},
    default 'left'
    How to handle the operation of the two objects.

    * left: use calling frame's index (or column if on is specified)
    * right: use `other`'s index.
    * outer: form union of calling frame's index (or column if on is
      specified) with `other`'s index, and sort it lexicographically.
    * inner: form intersection of calling frame's index (or column if
      on is specified) with `other`'s index, preserving the order
      of the calling's one.
    * cross: creates the cartesian product from both frames, preserves the order
      of the left keys.
    * left_anti: use set difference of calling frame's index and `other`'s
      index.
    * right_anti: use set difference of `other`'s index and calling frame's
      index.
lsuffix : str, default ''
    Suffix to use from left frame's overlapping columns.
rsuffix : str, default ''
    Suffix to use from right frame's overlapping columns.
sort : bool, default False
    Order result DataFrame lexicographically by the join key. If False,
    the order of the join key depends on the join type (how keyword).
validate : str, optional
    If specified, checks if join is of specified type.

    * "one_to_one" or "1:1": check if join keys are unique in both left
      and right datasets.
    * "one_to_many" or "1:m": check if join keys are unique in left dataset.
    * "many_to_one" or "m:1": check if join keys are unique in right dataset.
    * "many_to_many" or "m:m": allowed, but does not result in checks.

```

Returns

DataFrame

A dataframe containing columns from both the caller and `other`.

See Also

DataFrame.merge : For column(s)-on-column(s) operations.

Notes

Parameters `on`, `lsuffix`, and `rsuffix` are not supported when passing a list of `DataFrame` objects.

Examples

```

>>> df = pd.DataFrame(
...     {
...         "key": ["K0", "K1", "K2", "K3", "K4", "K5"],
...         "A": ["A0", "A1", "A2", "A3", "A4", "A5"],
...     }
... )

```

```

>>> df
   key  A
0  K0  A0
1  K1  A1
2  K2  A2
3  K3  A3
4  K4  A4
5  K5  A5

```

```

>>> other = pd.DataFrame({"key": ["K0", "K1", "K2"], "B": ["B0", "B1", "B2"]})

```

```

>>> other
   key  B
0  K0  B0
1  K1  B1

```

```
2 K2 B2
```

Join DataFrames using their indexes.

```
>>> df.join(other, lsuffix="_caller", rsuffix="_other")
  key_caller  A key_other  B
0         K0  A0         K0  B0
1         K1  A1         K1  B1
2         K2  A2         K2  B2
3         K3  A3         NaN NaN
4         K4  A4         NaN NaN
5         K5  A5         NaN NaN
```

If we want to join using the key columns, we need to set key to be the index in both `df` and `other`. The joined DataFrame will have key as its index.

```
>>> df.set_index("key").join(other.set_index("key"))
      A      B
key
K0    A0    B0
K1    A1    B1
K2    A2    B2
K3    A3    NaN
K4    A4    NaN
K5    A5    NaN
```

Another option to join using the key columns is to use the `on` parameter. DataFrame.join always uses `other`'s index but we can use any column in `df`. This method preserves the original DataFrame's index in the result.

```
>>> df.join(other.set_index("key"), on="key")
  key  A      B
0  K0  A0    B0
1  K1  A1    B1
2  K2  A2    B2
3  K3  A3    NaN
4  K4  A4    NaN
5  K5  A5    NaN
```

Using non-unique key values shows how they are matched.

```
>>> df = pd.DataFrame(
...     {
...         "key": ["K0", "K1", "K1", "K3", "K0", "K1"],
...         "A": ["A0", "A1", "A2", "A3", "A4", "A5"],
...     }
... )
```

```
>>> df
  key  A
0  K0  A0
1  K1  A1
2  K1  A2
3  K3  A3
4  K0  A4
5  K1  A5
```

```
>>> df.join(other.set_index("key"), on="key", validate="m:1")
  key  A      B
0  K0  A0    B0
1  K1  A1    B1
2  K1  A2    B1
3  K3  A3    NaN
4  K0  A4    B0
5  K1  A5    B1
```

```
kurt(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
    Return unbiased kurtosis over requested axis.
```

Kurtosis obtained using Fisher's definition of kurtosis (kurtosis of normal == 0.0). Normalized by N-1.

Parameters

axis : {index (0), columns (1)}

Axis for the function to be applied on.

For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric_only : bool, default False

Include only float, int, boolean columns.

**kwargs

Additional keyword arguments to be passed to the function.

Returns

Series or scalar

Unbiased kurtosis over requested axis.

See Also

DataFrame.kurtosis : Returns unbiased kurtosis over requested axis.

Examples

```
>>> s = pd.Series([1, 2, 2, 3], index=["cat", "dog", "dog", "mouse"])
>>> s
cat    1
dog    2
dog    2
mouse  3
dtype: int64
>>> s.kurt()
1.5
```

With a DataFrame

```
>>> df = pd.DataFrame(
...     {"a": [1, 2, 2, 3], "b": [3, 4, 4, 4]},
...     index=["cat", "dog", "dog", "mouse"],
... )
>>> df
      a  b
cat  1  3
dog  2  4
dog  2  4
mouse 3  4
>>> df.kurt()
a    1.5
b    4.0
dtype: float64
```

With axis=None

```
>>> df.kurt(axis=None)
-0.9886927196984727
```

Using axis=1

```
>>> df = pd.DataFrame(
...     {"a": [1, 2], "b": [3, 4], "c": [3, 4], "d": [1, 2]},
...     index=["cat", "dog"],
... )
>>> df.kurt(axis=1)
cat    -6.0
dog    -6.0
dtype: float64
```

```

kurtosis = kurt(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any

le(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame
Get Greater than or equal to of dataframe and other, element-wise (binary operator `le`)

Among flexible wrappers (`eq`, `ne`, `le`, `lt`, `ge`, `gt`) to comparison operators.

Equivalent to `==`, `!=`, `<=`, `<`, `>=`, `>` with support to choose axis (rows or columns) and level for comparison.

Parameters
-----
other : scalar, sequence, Series, or DataFrame
    Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
    Whether to compare by the index (0 or 'index') or columns (1 or 'columns').
level : int or label
    Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns
-----
DataFrame of bool
    Result of the comparison.

See Also
-----
DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than inequality elementwise.

Notes
-----
Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples
-----
>>> df = pd.DataFrame({'cost': [250, 150, 100],
...                     'revenue': [100, 250, 300]},
...                     index=['A', 'B', 'C'])
>>> df
   cost  revenue
A   250     100
B   150     250
C   100     300

Comparison with a scalar, using either the operator or method:

>>> df == 100
   cost  revenue
A  False     True
B  False     False
C   True     False

>>> df.eq(100)
   cost  revenue
A  False     True
B  False     False
C   True     False

```

When `other` is a `:class:`Series``, the columns of a `DataFrame` are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
      cost  revenue
A    True     True
B    True    False
C   False     True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
      cost  revenue
A    True    False
B    True     True
C    True     True
D    True     True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
      cost  revenue
A    True     True
B   False    False
C   False    False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
      cost  revenue
A    True    False
B   False     True
C    True    False
```

Compare to a `DataFrame` of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                       index=['A', 'B', 'C', 'D'])
>>> other
      revenue
A         300
B         250
C         100
D         150
```

```
>>> df.gt(other)
      cost  revenue
A   False    False
B   False    False
C   False     True
D   False    False
```

Compare to a `MultiIndex` by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
...                              'revenue': [100, 250, 300, 200, 175, 225]},
...                              index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                    ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
      cost  revenue
Q1 A    250     100
   B    150     250
   C    100     300
Q2 A    150     200
   B    300     175
   C    220     225

>>> df.le(df_multindex, level=1)
      cost  revenue
Q1 A    True     True
   B    True     True
   C    True     True
Q2 A   False     True
   B    True    False
   C    True    False
```

```
lt(self, other, axis: Axis = 'columns', level=None) -> DataFrame from pandas.core.frame.DataFrame
    Get Greater than of dataframe and other, element-wise (binary operator `lt`).
```

Among flexible wrappers (`eq`, `ne`, `le`, `lt`, `ge`, `gt`) to comparison operators.

Equivalent to `==`, `!=`, `<=`, `<`, `>=`, `>` with support to choose axis (rows or columns) and level for comparison.

Parameters

other : scalar, sequence, Series, or DataFrame
 Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}, default 'columns'
 Whether to compare by the index (0 or 'index') or columns
 (1 or 'columns').
level : int or label
 Broadcast across a level, matching Index values on the passed
 MultiIndex level.

Returns

DataFrame of bool
 Result of the comparison.

See Also

DataFrame.eq : Compare DataFrames for equality elementwise.
DataFrame.ne : Compare DataFrames for inequality elementwise.
DataFrame.le : Compare DataFrames for less than inequality
 or equality elementwise.
DataFrame.lt : Compare DataFrames for strictly less than
 inequality elementwise.
DataFrame.ge : Compare DataFrames for greater than inequality
 or equality elementwise.
DataFrame.gt : Compare DataFrames for strictly greater than
 inequality elementwise.

Notes

Mismatched indices will be unioned together.
`NaN` values are considered different (i.e. `NaN` != `NaN`).

Examples

>>> df = pd.DataFrame({'cost': [250, 150, 100],
... 'revenue': [100, 250, 300]},
... index=['A', 'B', 'C'])
>>> df
 cost revenue
A 250 100
B 150 250
C 100 300

Comparison with a scalar, using either the operator or method:

```
>>> df == 100  
   cost  revenue  
A  False      True  
B  False     False  
C   True     False  
  
>>> df.eq(100)  
   cost  revenue  
A  False      True  
B  False     False  
C   True     False
```

When `other` is a :class:`Series`, the columns of a DataFrame are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])  
   cost  revenue  
A   True      True  
B   True     False  
C  False      True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis='index')
      cost  revenue
A   True   False
B   True    True
C   True    True
D   True    True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
      cost  revenue
A   True    True
B  False   False
C  False   False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis='index')
      cost  revenue
A   True   False
B  False    True
C   True   False
```

Compare to a DataFrame of different shape.

```
>>> other = pd.DataFrame({'revenue': [300, 250, 100, 150]},
...                       index=['A', 'B', 'C', 'D'])
>>> other
      revenue
A         300
B         250
C         100
D         150

>>> df.gt(other)
      cost  revenue
A  False   False
B  False   False
C  False    True
D  False   False
```

Compare to a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'cost': [250, 150, 100, 150, 300, 220],
...                              'revenue': [100, 250, 300, 200, 175, 225]},
...                              index=[['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2'],
...                                     ['A', 'B', 'C', 'A', 'B', 'C']])
>>> df_multindex
      cost  revenue
Q1 A   250     100
   B   150     250
   C   100     300
Q2 A   150     200
   B   300     175
   C   220     225

>>> df.le(df_multindex, level=1)
      cost  revenue
Q1 A   True    True
   B   True    True
   C   True    True
Q2 A  False    True
   B   True   False
   C   True   False
```

```
map(
    self,
    func: PythonFuncType,
    na_action: Literal['ignore'] | None = None,
    **kwargs
) -> DataFrame from pandas.core.frame.DataFrame
Apply a function to a Dataframe elementwise.

.. versionadded:: 2.1.0
```

DataFrame.applymap was deprecated and renamed to DataFrame.map.

This method applies a function that accepts and returns a scalar to every element of a DataFrame.

Parameters

func : callable
 Python function, returns a single value from a single value.
na_action : {None, 'ignore'}, default None
 If 'ignore', propagate NaN values, without passing them to func.
**kwargs
 Additional keyword arguments to pass as keywords arguments to `func`.

Returns

DataFrame
 Transformed DataFrame.

See Also

DataFrame.apply : Apply a function along input axis of DataFrame.
DataFrame.replace: Replace values given in `to\_replace` with `value`.
Series.map : Apply a function elementwise on a Series.

Examples

>>> df = pd.DataFrame([[1, 2.12], [3.356, 4.567]])
>>> df
 0 1
0 1.000 2.120
1 3.356 4.567

>>> df.map(lambda x: len(str(x)))
 0 1
0 3 4
1 5 5

Like Series.map, NA values can be ignored:

>>> df_copy = df.copy()
>>> df_copy.iloc[0, 0] = pd.NA
>>> df_copy.map(lambda x: len(str(x)), na_action="ignore")
 0 1
0 NaN 4
1 5.0 5

It is also possible to use `map` with functions that are not `lambda` functions:

>>> df.map(round, ndigits=1)
 0 1
0 1.0 2.1
1 3.4 4.6

Note that a vectorized version of `func` often exists, which will be much faster. You could square each number elementwise.

>>> df.map(lambda x: x**2)
 0 1
0 1.000000 4.494400
1 11.262736 20.857489

But it's better to avoid map in that case.

>>> df**2
 0 1
0 1.000000 4.494400
1 11.262736 20.857489

max(
 self,
 *,
 axis: Axis | None = 0,
 skipna: bool = True,


```

    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
    Return the maximum of the values over the requested axis.

    If you want the *index* of the maximum, use ``idxmax``.
    This is the equivalent of the ``numpy.ndarray`` method ``argmax``.

Parameters
-----
axis : {index (0), columns (1)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.

    .. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.

**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
Series or scalar
    Value containing the calculation referenced in the description.

See Also
-----
Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
>>> idx = pd.MultiIndex.from_arrays(
...     [ ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"] ],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded  animal
warm     dog      4
         falcon   2
cold     fish     0
         spider   8
Name: legs, dtype: int64

>>> s.max()
8

mean(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
    Return the mean of the values over the requested axis.

Parameters
-----
axis : {index (0), columns (1)}
    Axis for the function to be applied on.

```

```

For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation
across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.

**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
Series or scalar
    Value containing the calculation referenced in the description.

See Also
-----
Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
>>> s = pd.Series([1, 2, 3])
>>> s.mean()
2.0

With a DataFrame
>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
      a  b
tiger 1  2
zebra 2  3
>>> df.mean()
a    1.5
b    2.5
dtype: float64

Using axis=1
>>> df.mean(axis=1)
tiger    1.5
zebra    2.5
dtype: float64

In this case, `numeric_only` should be set to `True` to avoid
getting an error.

>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.mean(numeric_only=True)
a    1.5
dtype: float64

median(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
    Return the median of the values over the requested axis.

Parameters

```

```

-----
axis : {index (0), columns (1)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.

    .. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.

**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
Series or scalar
    Value containing the calculation referenced in the description.

See Also
-----
Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
>>> s = pd.Series([1, 2, 3])
>>> s.median()
2.0

With a DataFrame

>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
      a  b
tiger 1  2
zebra 2  3
>>> df.median()
a    1.5
b    2.5
dtype: float64

Using axis=1

>>> df.median(axis=1)
tiger    1.5
zebra    2.5
dtype: float64

In this case, `numeric_only` should be set to `True`
to avoid getting an error.

>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.median(numeric_only=True)
a    1.5
dtype: float64

melt(
    self,
    id_vars=None,
    value_vars=None,
    var_name=None,
    value_name: Hashable = 'value',
    col_level: Level | None = None,
    ignore_index: bool = True

```

```
) -> DataFrame from pandas.core.frame.DataFrame
Unpivot DataFrame from wide to long format, optionally leaving identifiers set.
```

This function is useful to massage a DataFrame into a format where one or more columns are identifier variables (``id_vars``), while all other columns, considered measured variables (``value_vars``), are "unpivoted" to the row axis, leaving just two non-identifier columns, 'variable' and 'value'.

Parameters

`id_vars` : scalar, tuple, list, or ndarray, optional
Column(s) to use as identifier variables.
`value_vars` : scalar, tuple, list, or ndarray, optional
Column(s) to unpivot. If not specified, uses all columns that are not set as ``id_vars``.
`var_name` : scalar, default None
Name to use for the 'variable' column. If None it uses ``frame.columns.name`` or 'variable'.
`value_name` : scalar, default 'value'
Name to use for the 'value' column, can't be an existing column label.
`col_level` : scalar, optional
If columns are a MultiIndex then use this level to melt.
`ignore_index` : bool, default True
If True, original index is ignored. If False, original index is retained. Index labels will be repeated as necessary.

Returns

DataFrame
Unpivoted DataFrame.

See Also

`melt` : Identical method.
`pivot_table` : Create a spreadsheet-style pivot table as a DataFrame.
`DataFrame.pivot` : Return reshaped DataFrame organized by given index / column values.
`DataFrame.explode` : Explode a DataFrame from list-like columns to long format.

Notes

Reference :ref:`the user guide <reshaping.melt>` for more examples.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "A": {0: "a", 1: "b", 2: "c"},
...         "B": {0: 1, 1: 3, 2: 5},
...         "C": {0: 2, 1: 4, 2: 6},
...     }
... )
>>> df
   A  B  C
0  a  1  2
1  b  3  4
2  c  5  6

>>> df.melt(id_vars=["A"], value_vars=["B"])
   A variable  value
0  a         B      1
1  b         B      3
2  c         B      5

>>> df.melt(id_vars=["A"], value_vars=["B", "C"])
   A variable  value
0  a         B      1
1  b         B      3
2  c         B      5
3  a         C      2
4  b         C      4
5  c         C      6
```

The names of 'variable' and 'value' columns can be customized:

```
>>> df.melt(
...     id_vars=["A"],
...     value_vars=["B"],
...     var_name="myVarname",
...     value_name="myValname",
... )
A myVarname myValname
0 a B 1
1 b B 3
2 c B 5
```

Original index values can be kept around:

```
>>> df.melt(id_vars=["A"], value_vars=["B", "C"], ignore_index=False)
A variable value
0 a B 1
1 b B 3
2 c B 5
0 a C 2
1 b C 4
2 c C 6
```

If you have multi-index columns:

```
>>> df.columns = [list("ABC"), list("DEF")]
>>> df
A B C
D E F
0 a 1 2
1 b 3 4
2 c 5 6

>>> df.melt(col_level=0, id_vars=["A"], value_vars=["B"])
A variable value
0 a B 1
1 b B 3
2 c B 5

>>> df.melt(id_vars=[("A", "D")], value_vars=[("B", "E")])
(A, D) variable_0 variable_1 value
0 a B E 1
1 b B E 3
2 c B E 5
```

`memory_usage(self, index: bool = True, deep: bool = False) -> Series from pandas.core.frame.DataFrame`
Return the memory usage of each column in bytes.

The memory usage can optionally include the contribution of the index and elements of `'object'` dtype.

This value is displayed in `'DataFrame.info'` by default. This can be suppressed by setting `'pandas.options.display.memory_usage'` to `False`.

Parameters

`index` : bool, default True
Specifies whether to include the memory usage of the DataFrame's index in returned Series. If `'index=True'`, the memory usage of the index is the first item in the output.

`deep` : bool, default False
If True, introspect the data deeply by interrogating `'object'` dtypes for system-level memory consumption, and include it in the returned values.

Returns

Series

A Series whose index is the original column names and whose values is the memory usage of each column in bytes.

See Also

`numpy.ndarray.nbytes` : Total bytes consumed by the elements of an ndarray.

`Series.memory_usage` : Bytes consumed by a Series.

`Categorical` : Memory-efficient array for string values with many repeated values.

DataFrame.info : Concise summary of a DataFrame.

Notes

See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.

Examples

```
>>> dtypes = ["int64", "float64", "complex128", "object", "bool"]
>>> data = dict([(t, np.ones(shape=5000, dtype=int).astype(t)) for t in dtypes])
>>> df = pd.DataFrame(data)
>>> df.head()
```

	int64	float64	complex128	object	bool
0	1	1.0	1.0+0.0j	1	True
1	1	1.0	1.0+0.0j	1	True
2	1	1.0	1.0+0.0j	1	True
3	1	1.0	1.0+0.0j	1	True
4	1	1.0	1.0+0.0j	1	True

```
>>> df.memory_usage()
Index      132
int64      40000
float64     40000
complex128  80000
object     40000
bool       5000
dtype: int64
```

```
>>> df.memory_usage(index=False)
int64      40000
float64     40000
complex128  80000
object     40000
bool       5000
dtype: int64
```

The memory footprint of `object` dtype columns is ignored by default:

```
>>> df.memory_usage(deep=True)
Index      132
int64      40000
float64     40000
complex128  80000
object     180000
bool       5000
dtype: int64
```

Use a Categorical for efficient storage of an object-dtype column with many repeated values.

```
>>> df["object"].astype("category").memory_usage(deep=True)
5140
```

```
merge(
    self,
    right: DataFrame | Series,
    how: MergeHow = 'inner',
    on: IndexLabel | AnyArrayLike | None = None,
    left_on: IndexLabel | AnyArrayLike | None = None,
    right_on: IndexLabel | AnyArrayLike | None = None,
    left_index: bool = False,
    right_index: bool = False,
    sort: bool = False,
    suffixes: Suffixes = ('_x', '_y'),
    copy: bool | lib.NoDefault = <no_default>,
    indicator: str | bool = False,
    validate: MergeValidate | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Merge DataFrame or named Series objects with a database-style join.
```

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are

allowed.

.. warning::

If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters

right : DataFrame or named Series
Object to merge with.
how : {'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti'},
default 'inner'
Type of merge to be performed.

- * left: use only keys from left frame, similar to a SQL left outer join; preserve key order.
- * right: use only keys from right frame, similar to a SQL right outer join; preserve key order.
- * outer: use union of keys from both frames, similar to a SQL full outer join; sort keys lexicographically.
- * inner: use intersection of keys from both frames, similar to a SQL inner join; preserve the order of the left keys.
- * cross: creates the cartesian product from both frames, preserves the order of the left keys.
- * left_anti: use only keys from left frame that are not in right frame, similar to SQL left anti join; preserve key order.

.. versionadded:: 3.0

- * right_anti: use only keys from right frame that are not in left frame, similar to SQL right anti join; preserve key order.

.. versionadded:: 3.0

on : Hashable or a sequence of the previous
Column or index level names to join on. These must be found in both DataFrames. If `on` is None and not merging on indexes then this defaults to the intersection of the columns in both DataFrames.

left_on : Hashable or a sequence of the previous, or array-like
Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

right_on : Hashable or a sequence of the previous, or array-like
Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

left_index : bool, default False
Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

right_index : bool, default False
Use the index from the right DataFrame as the join key. Same caveats as left_index.

sort : bool, default False
Sort the join keys lexicographically in the result DataFrame. If False, the order of the join keys depends on the join type (how keyword).

suffixes : list-like, default is ("_x", "_y")
A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in `left` and `right` respectively. Pass a value of `None` instead of a string to indicate that the column name from `left` or `right` should be left as-is, with no suffix. At least one of the values must not be None.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>` for more details.

indicator : bool or str, default False

If True, adds a column to the output DataFrame called "_merge" with information on the source of each row. The column can be given a different name by providing a string argument. The column will have a Categorical type with the value of "left_only" for observations whose merge key only appears in the left DataFrame, "right_only" for observations whose merge key only appears in the right DataFrame, and "both" if the observation's merge key is found in both DataFrames.

validate : str, optional

If specified, checks if merge is of specified type.

- * "one_to_one" or "1:1": check if merge keys are unique in both left and right datasets.
- * "one_to_many" or "1:m": check if merge keys are unique in left dataset.
- * "many_to_one" or "m:1": check if merge keys are unique in right dataset.
- * "many_to_many" or "m:m": allowed, but does not result in checks.

Returns

DataFrame

A DataFrame of the two merged objects.

See Also

merge_ordered : Merge with optional filling/interpolation.

merge_asof : Merge on nearest keys.

DataFrame.join : Similar method using indices.

Examples

```
>>> df1 = pd.DataFrame({'lkey': ['foo', 'bar', 'baz', 'foo'],
...                     'value': [1, 2, 3, 5]})
>>> df2 = pd.DataFrame({'rkey': ['foo', 'bar', 'baz', 'foo'],
...                     'value': [5, 6, 7, 8]})
```

```
>>> df1
   lkey value
0  foo     1
1  bar     2
2  baz     3
3  foo     5
>>> df2
   rkey value
0  foo     5
1  bar     6
2  baz     7
3  foo     8
```

Merge df1 and df2 on the lkey and rkey columns. The value columns have the default suffixes, _x and _y, appended.

```
>>> df1.merge(df2, left_on='lkey', right_on='rkey')
   lkey  value_x rkey  value_y
0  foo         1  foo         5
1  foo         1  foo         8
2  bar         2  bar         6
3  baz         3  baz         7
4  foo         5  foo         5
5  foo         5  foo         8
```

Merge DataFrames df1 and df2 with specified left and right suffixes appended to any overlapping columns.

```
>>> df1.merge(df2, left_on='lkey', right_on='rkey',
...           suffixes=('_left', '_right'))
   lkey  value_left rkey  value_right
0  foo         1  foo         5
1  foo         1  foo         8
2  bar         2  bar         6
3  baz         3  baz         7
4  foo         5  foo         5
5  foo         5  foo         8
```

Merge DataFrames df1 and df2, but raise an exception if the DataFrames have any overlapping columns.


```
>>> df1.merge(df2, left_on='lkey', right_on='rkey', suffixes=(False, False))
Traceback (most recent call last):
```

```
...
ValueError: columns overlap but no suffix specified:
Index(['value'], dtype='object')
```

```
>>> df1 = pd.DataFrame({'a': ['foo', 'bar'], 'b': [1, 2]})
>>> df2 = pd.DataFrame({'a': ['foo', 'baz'], 'c': [3, 4]})
>>> df1
```

```
   a  b
0  foo  1
1  bar  2
```

```
>>> df2
   a  c
0  foo  3
1  baz  4
```

```
>>> df1.merge(df2, how='inner', on='a')
```

```
   a  b  c
0  foo  1  3
```

```
>>> df1.merge(df2, how='left', on='a')
```

```
   a  b  c
0  foo  1  3.0
1  bar  2  NaN
```

```
>>> df1 = pd.DataFrame({'left': ['foo', 'bar']})
```

```
>>> df2 = pd.DataFrame({'right': [7, 8]})
```

```
>>> df1
  left
0  foo
1  bar
```

```
>>> df2
  right
0     7
1     8
```

```
>>> df1.merge(df2, how='cross')
```

```
  left  right
0  foo     7
1  foo     8
2  bar     7
3  bar     8
```

```
min(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return the minimum of the values over the requested axis.
```

If you want the `*index*` of the minimum, use ```idxmin```.
This is the equivalent of the ```numpy.ndarray``` method ```argmin```.

Parameters

`axis : {index (0), columns (1)}`

Axis for the function to be applied on.

For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ```axis=None``` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

`skipna : bool, default True`

Exclude NA/null values when computing the result.

`numeric_only : bool, default False`

Include only float, int, boolean columns.

`**kwargs`

Additional keyword arguments to be passed to the function.

Returns

Series or scalar
Value containing the calculation referenced in the description.

See Also

Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

>>> idx = pd.MultiIndex.from_arrays(
... [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
... names=["blooded", "animal"],
...)
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm dog 4
 falcon 2
cold fish 0
 spider 8
Name: legs, dtype: int64

>>> s.min()
0

mod(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas
Get Modulo of dataframe and other, element-wise (binary operator `mod`).

Equivalent to ``dataframe % other``, but with support to substitute a fill_value for missing data in one of the inputs. With reverse version, `rmod`.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
level : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.
fill_value : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

>>> df + 1

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

>>> df.add(1)

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

>>> df.div(10)

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

>>> df.rdiv(10)

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

>>> df - [1, 2]

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

>>> df.sub([1, 2], axis='columns')

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

>>> df.mul({'angles': 0, 'degrees': 2})

	angles	degrees
circle	0	720
triangle	0	360
rectangle	0	720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')

	angles	degrees
circle	0	0
triangle	6	360
rectangle	12	1080

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                        index=['circle', 'triangle', 'rectangle'])
>>> other
```

```
      angles
circle      0
triangle    3
rectangle   4
```

```
>>> df * other
      angles  degrees
circle      0     NaN
triangle    9     NaN
rectangle   16     NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0     0.0
triangle    9     0.0
rectangle   16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
```

```
      angles  degrees
A circle      0     360
  triangle    3     180
  rectangle    4     360
B square      4     360
  pentagon    5     540
  hexagon     6     720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle  1.0     1.0
  rectangle  1.0     1.0
B square    0.0     0.0
  pentagon  0.0     0.0
  hexagon   0.0     0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
```

```
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

mode(self, axis: Axis = 0, numeric_only: bool = False, dropna: bool = True) -> DataFrame from
Get the mode(s) of each element along the selected axis.

The mode of a set of values is the value that appears most often.
It can be multiple values.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0

The axis to iterate over while searching for the mode:

- * 0 or 'index' : get mode of each column
- * 1 or 'columns' : get mode of each row.

numeric_only : bool, default False

If True, only apply to numeric columns.

dropna : bool, default True

Don't consider counts of NaN/NaT.

Returns

DataFrame
The modes of each column or row.

See Also

Series.mode : Return the highest frequency value in a Series.
Series.value_counts : Return the counts of values in a Series.

Examples

>>> df = pd.DataFrame(
... [
... ("bird", 2, 2),
... ("mammal", 4, np.nan),
... ("arthropod", 8, 0),
... ("bird", 2, np.nan),
...],
... index=("falcon", "horse", "spider", "ostrich"),
... columns=("species", "legs", "wings"),
...)
>>> df

	species	legs	wings
falcon	bird	2	2.0
horse	mammal	4	NaN
spider	arthropod	8	0.0
ostrich	bird	2	NaN

By default, missing values are not considered, and the mode of wings are both 0 and 2. Because the resulting DataFrame has two rows, the second row of ``species`` and ``legs`` contains ``NaN``.

```
>>> df.mode()  
   species  legs  wings  
0    bird    2.0    0.0  
1    NaN    NaN    2.0
```

Setting ``dropna=False`` ``NaN`` values are considered and they can be the mode (like for wings).

```
>>> df.mode(dropna=False)  
   species  legs  wings  
0    bird    2    NaN
```

Setting ``numeric\_only=True``, only the mode of numeric columns is computed, and columns of other types are ignored.

```
>>> df.mode(numeric_only=True)  
   legs  wings  
0    2.0    0.0  
1    NaN    2.0
```

To compute the mode over columns and not rows, use the axis parameter:

```
>>> df.mode(axis="columns", numeric_only=True)  
   0    1  
falcon    2.0    NaN  
horse     4.0    NaN  
spider    0.0    8.0  
ostrich   2.0    NaN
```

mul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas
Get Multiplication of dataframe and other, element-wise (binary operator `mul`).

Equivalent to ``dataframe \* other``, but with support to substitute a fill_value for missing data in one of the inputs. With reverse version, ``rmul``.

Among flexible wrappers (`add`, `sub`, `mul`, `div`, `floordiv`, `mod`, `pow`) to arithmetic operators: `+`, `-`, `\*`, `/`, `//`, `%`, `\*\*`.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.

level : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.
fill_value : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

DataFrame.add : Add DataFrames.

DataFrame.sub : Subtract DataFrames.

DataFrame.mul : Multiply DataFrames.

DataFrame.div : Divide DataFrames (float division).

DataFrame.truediv : Divide DataFrames (float division).

DataFrame.floordiv : Divide DataFrames (integer division).

DataFrame.mod : Calculate modulo (remainder after division).

DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1     358
triangle     2     178
rectangle     3     358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1     359
triangle     2     179
rectangle     3     359

Multiply a dictionary by axis.

>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle         0     720
triangle        0     360
rectangle        0     720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle         0         0
triangle         6     360
rectangle        12    1080

Multiply a DataFrame of different shape with operator version.

>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle         0
triangle         3
rectangle         4

>>> df * other
      angles  degrees
circle         0     NaN
triangle         9     NaN
rectangle        16     NaN

>>> df.mul(other, fill_value=0)
      angles  degrees
circle         0     0.0
triangle         9     0.0
rectangle        16     0.0

Divide by a MultiIndex by level.

>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle         0     360
  triangle         3     180
  rectangle         4     360
B square          4     360
  pentagon         5     540
  hexagon          6     720

>>> df_multindex.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle     NaN     1.0
  triangle     1.0     1.0
  rectangle     1.0     1.0
B square      0.0     0.0
  pentagon      0.0     0.0
  hexagon      0.0     0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)

```

	A	B
0	4	36
1	9	49
2	16	64
3	25	81

`multiply = mul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame`

`ne(self, other, axis: Axis = 'columns', level=None) -> DataFrame` from `pandas.core.frame.DataFrame`
Get Not equal to of dataframe and other, element-wise (binary operator ``ne``).

Among flexible wrappers (``eq``, ``ne``, ``le``, ``lt``, ``ge``, ``gt``) to comparison operators.

Equivalent to ``==``, ``!=``, ``<=``, ``<``, ``>=``, ``>`` with support to choose axis (rows or columns) and level for comparison.

Parameters

`other` : scalar, sequence, Series, or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}, default 'columns'
Whether to compare by the index (0 or 'index') or columns (1 or 'columns').
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

Returns

DataFrame of bool
Result of the comparison.

See Also

`DataFrame.eq` : Compare DataFrames for equality elementwise.
`DataFrame.ne` : Compare DataFrames for inequality elementwise.
`DataFrame.le` : Compare DataFrames for less than inequality or equality elementwise.
`DataFrame.lt` : Compare DataFrames for strictly less than inequality elementwise.
`DataFrame.ge` : Compare DataFrames for greater than inequality or equality elementwise.
`DataFrame.gt` : Compare DataFrames for strictly greater than inequality elementwise.

Notes

Mismatched indices will be unioned together.
``NaN`` values are considered different (i.e. ``NaN` != `NaN``).

Examples

```
>>> df = pd.DataFrame(
...     {"cost": [250, 150, 100], "revenue": [100, 250, 300]},
...     index=["A", "B", "C"],
... )
>>> df
   cost  revenue
A   250      100
B   150      250
C   100      300
```

Comparison with a scalar, using either the operator or method:

```
>>> df == 100
   cost  revenue
A  False      True
B  False      False
C   True      False

>>> df.eq(100)
   cost  revenue
A  False      True
B  False      False
C   True      False
```


When `other` is a `:class:`Series``, the columns of a `DataFrame` are aligned with the index of `other` and broadcast:

```
>>> df != pd.Series([100, 250], index=["cost", "revenue"])
cost revenue
A   True   True
B   True  False
C  False   True
```

Use the method to control the broadcast axis:

```
>>> df.ne(pd.Series([100, 300], index=["A", "D"]), axis="index")
cost revenue
A   True   False
B   True    True
C   True    True
D   True    True
```

When comparing to an arbitrary sequence, the number of columns must match the number elements in `other`:

```
>>> df == [250, 100]
cost revenue
A   True   True
B  False  False
C  False  False
```

Use the method to control the axis:

```
>>> df.eq([250, 250, 100], axis="index")
cost revenue
A   True   False
B  False    True
C   True   False
```

Compare to a `DataFrame` of different shape.

```
>>> other = pd.DataFrame(
...     {"revenue": [300, 250, 100, 150]}, index=["A", "B", "C", "D"]
... )
>>> other
revenue
A      300
B      250
C      100
D      150

>>> df.gt(other)
cost revenue
A  False  False
B  False  False
C  False   True
D  False  False
```

Compare to a `MultiIndex` by level.

```
>>> df_multindex = pd.DataFrame(
...     {
...         "cost": [250, 150, 100, 150, 300, 220],
...         "revenue": [100, 250, 300, 200, 175, 225],
...     },
...     index=[
...         ["Q1", "Q1", "Q1", "Q2", "Q2", "Q2"],
...         ["A", "B", "C", "A", "B", "C"],
...     ],
... )
>>> df_multindex
cost revenue
Q1 A   250    100
   B   150    250
   C   100    300
Q2 A   150    200
   B   300    175
   C   220    225

>>> df.le(df_multindex, level=1)
cost revenue
```

Q1	A	True	True
	B	True	True
	C	True	True
Q2	A	False	True
	B	True	False
	C	True	False

```

nlargest(
    self,
    n: int,
    columns: IndexLabel,
    keep: NsmallestNlargestKeep = 'first'
) -> DataFrame from pandas.core.frame.DataFrame
    Return the first `n` rows ordered by `columns` in descending order.

    Return the first `n` rows with the largest values in `columns`, in
    descending order. The columns that are not specified are returned as
    well, but not used for ordering.

    This method is equivalent to
    ``df.sort_values(columns, ascending=False).head(n)`` , but more
    performant.

Parameters
-----
n : int
    Number of rows to return.
columns : Hashable or a sequence of the previous
    Column label(s) to order by.
keep : {'first', 'last', 'all'}, default 'first'
    Where there are duplicate values:
    - ``first`` : prioritize the first occurrence(s)
    - ``last`` : prioritize the last occurrence(s)
    - ``all`` : keep all the ties of the smallest item even if it means
        selecting more than ``n`` items.

Returns
-----
DataFrame
    The first `n` rows ordered by the given columns in descending
    order.

See Also
-----
DataFrame.nsmallest : Return the first `n` rows ordered by `columns` in
    ascending order.
DataFrame.sort_values : Sort DataFrame by the values.
DataFrame.head : Return the first `n` rows without re-ordering.

Notes
-----
This function cannot be used with all column types. For example, when
specifying columns with `object` or `category` dtypes, ``TypeError`` is
raised.

Examples
-----
>>> df = pd.DataFrame(
...     {
...         "population": [
...             59000000,
...             65000000,
...             434000,
...             434000,
...             434000,
...             337000,
...             11300,
...             11300,
...             11300,
...         ],
...         "GDP": [1937894, 2583560, 12011, 4520, 12128, 17036, 182, 38, 311],
...         "alpha-2": ["IT", "FR", "MT", "MV", "BN", "IS", "NR", "TV", "AI"],
...     },
...     index=[
...         "Italy",
...         "France",
    
```

```

...     "Malta",
...     "Maldives",
...     "Brunei",
...     "Iceland",
...     "Nauru",
...     "Tuvalu",
...     "Anguilla",
... ],
... )
>>> df

```

	population	GDP	alpha-2
Italy	59000000	1937894	IT
France	65000000	2583560	FR
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN
Iceland	337000	17036	IS
Nauru	11300	182	NR
Tuvalu	11300	38	TV
Anguilla	11300	311	AI

In the following example, we will use ``nlargest`` to select the three rows having the largest values in column "population".

```

>>> df.nlargest(3, "population")

```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Malta	434000	12011	MT

When using ``keep='last'``, ties are resolved in reverse order:

```

>>> df.nlargest(3, "population", keep="last")

```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Brunei	434000	12128	BN

When using ``keep='all'``, the number of element kept can go beyond ``n`` if there are duplicate values for the smallest element, all the ties are kept:

```

>>> df.nlargest(3, "population", keep="all")

```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN

However, ``nlargest`` does not keep ``n`` distinct largest elements:

```

>>> df.nlargest(5, "population", keep="all")

```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN

To order by the largest values in column "population" and then "GDP", we can specify multiple columns like in the next example.

```

>>> df.nlargest(3, ["population", "GDP"])

```

	population	GDP	alpha-2
France	65000000	2583560	FR
Italy	59000000	1937894	IT
Brunei	434000	12128	BN

```

notna(self) -> DataFrame from pandas.core.frame.DataFrame
Detect existing (non-missing) values.

```

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

Series.notnull : Alias of notna.

DataFrame.notnull : Alias of notna.

Series.isna : Boolean inverse of notna.

DataFrame.isna : Boolean inverse of notna.

Series.dropna : Omit axes labels with missing values.

DataFrame.dropna : Omit axes labels with missing values.

notna : Top-level notna.

Examples

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born    name      toy
0  5.0      NaT  Alfred      NaN
1  6.0  1939-05-27  Batman  Batmobile
2  NaN  1940-04-25      Joker
```

```
>>> df.notna()
   age  born  name  toy
0  True False  True False
1  True  True  True  True
2 False  True  True  True
```

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64

>>> ser.notna()
0     True
1     True
2    False
dtype: bool
```

notnull(self) -> DataFrame from pandas.core.frame.DataFrame
DataFrame.notnull is an alias for DataFrame.notna.

Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series/DataFrame

Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

Series.notnull : Alias of notna.
DataFrame.notnull : Alias of notna.
Series.isna : Boolean inverse of notna.
DataFrame.isna : Boolean inverse of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
notna : Top-level notna.

Examples

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born  name      toy
0  5.0      NaT  Alfred     NaN
1  6.0 1939-05-27  Batman  Batmobile
2  NaN 1940-04-25      Joker

>>> df.notnull()
   age  born  name  toy
0  True False  True False
1  True  True  True  True
2  False  True  True  True
```

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64

>>> ser.notnull()
0    True
1    True
2   False
dtype: bool
```

```
nsmallest(
    self,
    n: int,
    columns: IndexLabel,
    keep: NsmallestNlargestKeep = 'first'
) -> DataFrame from pandas.core.frame.DataFrame
Return the first `n` rows ordered by `columns` in ascending order.
```

Return the first `n` rows with the smallest values in `columns`, in ascending order. The columns that are not specified are returned as well, but not used for ordering.

This method is equivalent to
`df.sort\_values(columns, ascending=True).head(n)` , but more performant.

Parameters

n : int
Number of items to retrieve.
columns : list or str
Column name or names to order by.
keep : {'first', 'last', 'all'}, default 'first'
Where there are duplicate values:

- ``first`` : take the first occurrence.
- ``last`` : take the last occurrence.
- ``all`` : keep all the ties of the largest item even if it means selecting more than ``n`` items.

Returns

DataFrame

DataFrame with the first ``n`` rows ordered by ``columns`` in ascending order.

See Also

DataFrame.nlargest : Return the first ``n`` rows ordered by ``columns`` in descending order.

DataFrame.sort_values : Sort DataFrame by the values.

DataFrame.head : Return the first ``n`` rows without re-ordering.

Examples

```

>>> df = pd.DataFrame(
...     {
...         "population": [
...             59000000,
...             65000000,
...             434000,
...             434000,
...             434000,
...             337000,
...             337000,
...             11300,
...             11300,
...         ],
...         "GDP": [1937894, 2583560, 12011, 4520, 12128, 17036, 182, 38, 311],
...         "alpha-2": ["IT", "FR", "MT", "MV", "BN", "IS", "NR", "TV", "AI"],
...     },
...     index=[
...         "Italy",
...         "France",
...         "Malta",
...         "Maldives",
...         "Brunei",
...         "Iceland",
...         "Nauru",
...         "Tuvalu",
...         "Anguilla",
...     ],
... )
>>> df

```

	population	GDP	alpha-2
Italy	59000000	1937894	IT
France	65000000	2583560	FR
Malta	434000	12011	MT
Maldives	434000	4520	MV
Brunei	434000	12128	BN
Iceland	337000	17036	IS
Nauru	337000	182	NR
Tuvalu	11300	38	TV
Anguilla	11300	311	AI

In the following example, we will use ``nsmallest`` to select the three rows having the smallest values in column "population".

```

>>> df.nsmallest(3, "population")

```

	population	GDP	alpha-2
Tuvalu	11300	38	TV
Anguilla	11300	311	AI
Iceland	337000	17036	IS

When using ``keep='last'``, ties are resolved in reverse order:

```

>>> df.nsmallest(3, "population", keep="last")

```

	population	GDP	alpha-2
Anguilla	11300	311	AI
Tuvalu	11300	38	TV
Nauru	337000	182	NR

When using ``keep='all'``, the number of element kept can go beyond ``n`` if there are duplicate values for the largest element, all the ties are kept.

```
>>> df.nsmallest(3, "population", keep="all")
      population  GDP alpha-2
Tuvalu         11300      38    TV
Anguilla        11300     311    AI
Iceland        337000  17036    IS
Nauru          337000     182    NR
```

However, ``nsmallest`` does not keep ``n`` distinct smallest elements:

```
>>> df.nsmallest(4, "population", keep="all")
      population  GDP alpha-2
Tuvalu         11300      38    TV
Anguilla        11300     311    AI
Iceland        337000  17036    IS
Nauru          337000     182    NR
```

To order by the smallest values in column "population" and then "GDP", we can specify multiple columns like in the next example.

```
>>> df.nsmallest(3, ["population", "GDP"])
      population  GDP alpha-2
Tuvalu         11300      38    TV
Anguilla        11300     311    AI
Nauru          337000     182    NR
```

`unique(self, axis: Axis = 0, dropna: bool = True) -> Series from pandas.core.frame.DataFrame`
Count number of distinct elements in specified axis.

Return Series with number of distinct elements. Can ignore NaN values.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.
dropna : bool, default True
Don't include NaN in the counts.

Returns

Series

Series with counts of unique values per row or column, depending on `axis`.

See Also

Series.nunique: Method nunique for Series.
DataFrame.count: Count non-NA cells for each column or row.

Examples

```
>>> df = pd.DataFrame({"A": [4, 5, 6], "B": [4, 1, 1]})
>>> df.nunique()
A      3
B      2
dtype: int64

>>> df.nunique(axis=1)
0      1
1      2
2      2
dtype: int64
```

`pivot(self, *, columns, index=<no_default>, values=<no_default>) -> DataFrame from pandas.core`
Return reshaped DataFrame organized by given index / column values.

Reshape data (produce a "pivot" table) based on column values. Uses unique values from specified `index` / `columns` to form axes of the resulting DataFrame. This function does not support data aggregation, multiple values will result in a MultiIndex in the columns. See the :ref:`User Guide <reshaping>` for more on reshaping.

Parameters

columns : Hashable or a sequence of the previous
 Column to use to make new frame's columns.
index : Hashable or a sequence of the previous, optional
 Column to use to make new frame's index. If not given, uses existing index.
values : Hashable or a sequence of the previous, optional
 Column(s) to use for populating new frame's values. If not
 specified, all remaining columns will be used and the result will
 have hierarchically indexed columns.

Returns

DataFrame
 Returns reshaped DataFrame.

Raises

ValueError:
 When there are any `index`, `columns` combinations with multiple
 values. `DataFrame.pivot\_table` when you need to aggregate.

See Also

DataFrame.pivot_table : Generalization of pivot that can handle
 duplicate values for one index/column pair.
DataFrame.unstack : Pivot based on the index values instead of a
 column.
wide_to_long : Wide panel to long format. Less flexible but more
 user-friendly than melt.

Notes

For finer-tuned control, see hierarchical indexing documentation along
with the related stack/unstack methods.

Reference :ref:`the user guide <reshaping.pivot>` for more examples.

Examples

>>> df = pd.DataFrame({'foo': ['one', 'one', 'one', 'two', 'two',
 ... 'two'],
 ... 'bar': ['A', 'B', 'C', 'A', 'B', 'C'],
 ... 'baz': [1, 2, 3, 4, 5, 6],
 ... 'zoo': ['x', 'y', 'z', 'q', 'w', 't']})

```
>>> df
   foo bar baz zoo
0  one  A   1   x
1  one  B   2   y
2  one  C   3   z
3  two  A   4   q
4  two  B   5   w
5  two  C   6   t
```

```
>>> df.pivot(index='foo', columns='bar', values='baz')
bar A  B  C
foo
one  1  2  3
two  4  5  6
```

```
>>> df.pivot(index='foo', columns='bar')['baz']
bar A  B  C
foo
one  1  2  3
two  4  5  6
```

```
>>> df.pivot(index='foo', columns='bar', values=['baz', 'zoo'])
      baz      zoo
bar A  B  C  A  B  C
foo
one  1  2  3  x  y  z
two  4  5  6  q  w  t
```

You could also assign a list of column names or a list of index names.

```
>>> df = pd.DataFrame({
...     "lev1": [1, 1, 1, 2, 2, 2],
```



```

...         "lev2": [1, 1, 2, 1, 1, 2],
...         "lev3": [1, 2, 1, 2, 1, 2],
...         "lev4": [1, 2, 3, 4, 5, 6],
...         "values": [0, 1, 2, 3, 4, 5])
>>> df
   lev1 lev2 lev3 lev4 values
0     1     1     1     1     0
1     1     1     2     2     1
2     1     2     1     3     2
3     2     1     2     4     3
4     2     1     1     5     4
5     2     2     2     6     5

>>> df.pivot(index="lev1", columns=["lev2", "lev3"], values="values")
lev2      1      2
lev3      1      2      1      2
lev1
1      0.0  1.0  2.0  NaN
2      4.0  3.0  NaN  5.0

>>> df.pivot(index=["lev1", "lev2"], columns=["lev3"], values="values")
lev3      1      2
lev1 lev2
1      1  0.0  1.0
      2  2.0  NaN
2      1  4.0  3.0
      2  NaN  5.0

```

A `ValueError` is raised if there are any duplicates.

```

>>> df = pd.DataFrame({"foo": ['one', 'one', 'two', 'two'],
...                     "bar": ['A', 'A', 'B', 'C'],
...                     "baz": [1, 2, 3, 4]})
>>> df
   foo bar baz
0  one  A   1
1  one  A   2
2  two  B   3
3  two  C   4

```

Notice that the first two rows are the same for our `index` and `columns` arguments.

```

>>> df.pivot(index='foo', columns='bar', values='baz')
Traceback (most recent call last):
...
ValueError: Index contains duplicate entries, cannot reshape

```

```

pivot_table(
    self,
    values=None,
    index=None,
    columns=None,
    aggfunc: AggFuncType = 'mean',
    fill_value=None,
    margins: bool = False,
    dropna: bool = True,
    margins_name: Level = 'All',
    observed: bool = True,
    sort: bool = True,
    **kwargs
)

```

-> `DataFrame` from `pandas.core.frame.DataFrame`
Create a spreadsheet-style pivot table as a `DataFrame`.

The levels in the pivot table will be stored in `MultiIndex` objects (hierarchical indexes) on the index and columns of the result `DataFrame`.

Parameters

`values` : list-like or scalar, optional

Column or columns to aggregate.

`index` : column, Grouper, array, or sequence of the previous

Keys to group by on the pivot table index. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

`columns` : column, Grouper, array, or sequence of the previous

Keys to group by on the pivot table column. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

aggfunc : function, list of functions, dict, default "mean"
 If a list of functions is passed, the resulting pivot table will have hierarchical columns whose top level are the function names (inferred from the function objects themselves).
 If a dict is passed, the key is column to aggregate and the value is function or list of functions. If ``margin=True``, aggfunc will be used to calculate the partial aggregates.

fill_value : scalar, default None
 Value to replace missing values with (in the resulting pivot table, after aggregation).

margins : bool, default False
 If ``margins=True``, special ``All`` columns and rows will be added with partial group aggregates across the categories on the rows and columns.

dropna : bool, default True
 Do not include columns whose entries are all NaN. If True,

- * rows with an NA value in any column will be omitted before computing margins,
- * index/column keys containing NA values will be dropped (see ``dropna`` parameter in :meth:`DataFrame.groupby`).

margins_name : str, default 'All'
 Name of the row / column that will contain the totals when margins is True.

observed : bool, default False
 This only applies if any of the groupers are Categoricals.
 If True: only show observed values for categorical groupers.
 If False: show all values for categorical groupers.

.. versionchanged:: 3.0.0

The default value is now ``True``.

sort : bool, default True
 Specifies if the result should be sorted.

****kwargs** : dict
 Optional keyword arguments to pass to ``aggfunc``.

Returns

DataFrame
 An Excel style pivot table.

See Also

DataFrame.pivot : Pivot without aggregation that can handle non-numeric data.
DataFrame.melt: Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.
wide_to_long : Wide panel to long format. Less flexible but more user-friendly than melt.

Notes

 Reference :ref:`the user guide <reshaping.pivot>` for more examples.

Examples

```
>>> df = pd.DataFrame({"A": ["foo", "foo", "foo", "foo", "foo",
...                           "bar", "bar", "bar", "bar"],
...                    "B": ["one", "one", "one", "two", "two",
...                           "one", "one", "two", "two"],
...                    "C": ["small", "large", "large", "small", "small",
...                           "small", "large", "small", "small",
...                           "large"],
...                    "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],
...                    "E": [2, 4, 5, 5, 6, 6, 8, 9, 9]})
>>> df
```

	A	B	C	D	E
0	foo	one	small	1	2
1	foo	one	large	2	4

```

2  foo  one  large  2  5
3  foo  two  small  3  5
4  foo  two  small  3  6
5  bar  one  large  4  6
6  bar  one  small  5  8
7  bar  two  small  6  9
8  bar  two  large  7  9

```

This first example aggregates values by taking the sum.

```

>>> table = pd.pivot_table(df, values='D', index=['A', 'B'],
...                          columns=['C'], aggfunc="sum")
>>> table
C      large  small
A  B
bar one    4.0    5.0
   two    7.0    6.0
foo one    4.0    1.0
   two    NaN    6.0

```

We can also fill missing values using the `fill\_value` parameter.

```

>>> table = pd.pivot_table(df, values='D', index=['A', 'B'],
...                          columns=['C'], aggfunc="sum", fill_value=0)
>>> table
C      large  small
A  B
bar one     4     5
   two     7     6
foo one     4     1
   two     0     6

```

The next example aggregates by taking the mean across multiple columns.

```

>>> table = pd.pivot_table(df, values=['D', 'E'], index=['A', 'C'],
...                          aggfunc={'D': "mean", 'E': "mean"})
>>> table
A  C      D      E
bar large  5.500000  7.500000
   small  5.500000  8.500000
foo large  2.000000  4.500000
   small  2.333333  4.333333

```

We can also calculate multiple types of aggregations for any given value column.

```

>>> table = pd.pivot_table(df, values=['D', 'E'], index=['A', 'C'],
...                          aggfunc={'D': ["mean", "max"],
...                                    'E': ["min", "max", "mean"]})
>>> table
A  C      D      E      mean  min
   large  mean max      mean  min
bar large  5.500000  9  7.500000  6
   small  5.500000  9  8.500000  8
foo large  2.000000  5  4.500000  4
   small  2.333333  6  4.333333  2

```

pop(self, item: Hashable) -> Series from pandas.core.frame.DataFrame
Return item and drop it from DataFrame. Raise KeyError if not found.

Parameters

item : label
Label of column to be popped.

Returns

Series
Series representing the item that is dropped.

See Also

DataFrame.drop: Drop specified labels from rows or columns.
DataFrame.drop_duplicates: Return DataFrame with duplicate rows removed.

Examples

```
>>> df = pd.DataFrame(
...     [
...         ("falcon", "bird", 389.0),
...         ("parrot", "bird", 24.0),
...         ("lion", "mammal", 80.5),
...         ("monkey", "mammal", np.nan),
...     ],
...     columns=("name", "class", "max_speed"),
... )
>>> df
   name  class  max_speed
0  falcon   bird     389.0
1  parrot   bird      24.0
2   lion  mammal     80.5
3 monkey  mammal      NaN
```

```
>>> df.pop("class")
0    bird
1    bird
2  mammal
3  mammal
Name: class, dtype: str
```

```
>>> df
   name  max_speed
0  falcon     389.0
1  parrot      24.0
2   lion      80.5
3 monkey      NaN
```

`pow(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Exponential power of dataframe and other, element-wise (binary operator `'pow'`).

Equivalent to `'dataframe ** other'`, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, `'rpow'`.

Among flexible wrappers (`'add'`, `'sub'`, `'mul'`, `'div'`, `'floordiv'`, `'mod'`, `'pow'`) to arithmetic operators: `'+'`, `'-'`, `'*'`, `'/'`, `'//'`, `'%'`, `'**'`.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
-----
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
```

	angles	degrees
circle	0	720
triangle	0	360
rectangle	0	720

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
```

	angles	degrees
circle	0	0
triangle	6	360
rectangle	12	1080

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                        index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle    3
rectangle   4
```

```
>>> df * other
      angles  degrees
circle      0     NaN
triangle    9     NaN
rectangle   16     NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0     0.0
triangle    9     0.0
rectangle   16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0     360
  triangle    3     180
  rectangle    4     360
B square      4     360
  pentagon    5     540
  hexagon     6     720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle  1.0     1.0
  rectangle  1.0     1.0
B square    0.0     0.0
  pentagon  0.0     0.0
  hexagon   0.0     0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

```
prod(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
) -> Series from pandas.core.frame.DataFrame
Return the product of the values over the requested axis.

Parameters
-----
axis : {index (0), columns (1)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

.. warning::
```

The behavior of DataFrame.prod with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis).

```

.. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns. Not implemented for Series.

min_count : int, default 0
    The required number of valid values to perform the operation. If fewer than
    ``min_count`` non-NA values are present the result will be NA.
**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
Series or scalar
    The product of the values over the requested axis.

See Also
-----
Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
By default, the product of an empty or all-NA Series is ``1``

>>> pd.Series([], dtype="float64").prod()
1.0

This can be controlled with the ``min_count`` parameter

>>> pd.Series([], dtype="float64").prod(min_count=1)
nan

Thanks to the ``skipna`` parameter, ``min_count`` handles all-NA and
empty series identically.

>>> pd.Series([np.nan]).prod()
1.0

>>> pd.Series([np.nan]).prod(min_count=1)
nan

product = prod(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
) -> Series

quantile(
    self,
    q: float | AnyArrayLike | Sequence[float] = 0.5,
    axis: Axis = 0,
    numeric_only: bool = False,
    interpolation: QuantileInterpolation = 'linear',
    method: Literal['single', 'table'] = 'single'
) -> Series | DataFrame from pandas.core.frame.DataFrame
    Return values at the given quantile over requested axis.

Parameters
-----
q : float or array-like, default 0.5 (50% quantile)
    Value between 0 <= q <= 1, the quantile(s) to compute.
axis : {0 or 'index', 1 or 'columns'}, default 0

```

```

    Equals 0 or 'index' for row-wise, 1 or 'columns' for column-wise.
numeric_only : bool, default False
    Include only `float`, `int` or `boolean` data.

.. versionchanged:: 2.0.0
    The default value of ``numeric_only`` is now ``False``.

interpolation : {'linear', 'lower', 'higher', 'midpoint', 'nearest'}
    This optional parameter specifies the interpolation method to use,
    when the desired quantile lies between two data points `i` and `j`:

    * linear: `(i + (j - i) * fraction)` where `fraction` is the
      fractional part of the index surrounded by `i` and `j`.
    * lower: `i`.
    * higher: `j`.
    * nearest: `i` or `j` whichever is nearest.
    * midpoint: `(i + j) / 2`.
method : {'single', 'table'}, default 'single'
    Whether to compute quantiles per-column ('single') or over all columns
    ('table'). When 'table', the only allowed interpolation methods are
    'nearest', 'lower', and 'higher'.

```

Returns

Series or DataFrame

```

    If ``q`` is an array, a DataFrame will be returned where the
    index is ``q``, the columns are the columns of self, and the
    values are the quantiles.
    If ``q`` is a float, a Series will be returned where the
    index is the columns of self and the values are the quantiles.

```

See Also

```

core.window.rolling.Rolling.quantile: Rolling quantile.
numpy.percentile: Numpy function to compute the percentile.

```

Examples

```

>>> df = pd.DataFrame(
...     np.array([[1, 1], [2, 10], [3, 100], [4, 100]]), columns=["a", "b"]
... )
>>> df.quantile(0.1)
a    1.3
b    3.7
Name: 0.1, dtype: float64
>>> df.quantile([0.1, 0.5])
      a    b
0.1  1.3  3.7
0.5  2.5 55.0

```

Specifying `method='table'` will compute the quantile over all columns.

```

>>> df.quantile(0.1, method="table", interpolation="nearest")
a    1
b    1
Name: 0.1, dtype: int64
>>> df.quantile([0.1, 0.5], method="table", interpolation="nearest")
      a    b
0.1  1    1
0.5  3  100

```

Specifying `numeric\_only=False` will compute the quantiles for all columns.

```

>>> df = pd.DataFrame(
...     {
...         "A": [1, 2],
...         "B": [pd.Timestamp("2010"), pd.Timestamp("2011")],
...         "C": [pd.Timedelta("1 days"), pd.Timedelta("2 days")],
...     }
... )
>>> df.quantile(0.5, numeric_only=False)
A          1.5
B    2010-07-02 12:00:00
C      1 days 12:00:00
Name: 0.5, dtype: object

```



```

query(
    self,
    expr: str,
    *,
    parser: Literal['pandas', 'python'] = 'pandas',
    engine: Literal['python', 'numexpr'] | None = None,
    local_dict: dict[str, Any] | None = None,
    global_dict: dict[str, Any] | None = None,
    resolvers: list[Mapping] | None = None,
    level: int = 0,
    inplace: bool = False
) -> DataFrame | None from pandas.core.frame.DataFrame
Query the columns of a DataFrame with a boolean expression.

.. warning::

    This method can run arbitrary code which can make you vulnerable to code
    injection if you pass user input to this function.

Parameters
-----
expr : str
    The query string to evaluate.

    See the documentation for :func:`eval` for details of
    supported operations and functions in the query string.

    See the documentation for :meth:`DataFrame.eval` for details on
    referring to column names and variables in the query string.
parser : {'pandas', 'python'}, default 'pandas'
    The parser to use to construct the syntax tree from the expression. The
    default of ``'pandas'`` parses code slightly different than standard
    Python. Alternatively, you can parse an expression using the
    ``'python'`` parser to retain strict Python semantics. See the
    :ref:`enhancing performance <enhancingperf.eval>` documentation for
    more details.
engine : {'python', 'numexpr'}, default 'numexpr'

    The engine used to evaluate the expression. Supported engines are

    - None : tries to use ``numexpr``, falls back to ``python``
    - ``'numexpr'`` : This default engine evaluates pandas objects using
      numexpr for large speed ups in complex expressions with large frames.
    - ``'python'`` : Performs operations as if you had ``eval``'d in top
      level python. This engine is generally not that useful.

    More backends may be available in the future.
local_dict : dict or None, optional
    A dictionary of local variables, taken from locals() by default.
global_dict : dict or None, optional
    A dictionary of global variables, taken from globals() by default.
resolvers : list of dict-like or None, optional
    A list of objects implementing the ``__getitem__`` special method that
    you can use to inject an additional collection of namespaces to use for
    variable lookup. For example, this is used in the
    :meth:`~DataFrame.query` method to inject the
    ``DataFrame.index`` and ``DataFrame.columns``
    variables that refer to their respective :class:`~pandas.DataFrame`
    instance attributes.
level : int, optional
    The number of prior stack frames to traverse and add to the current
    scope. Most users will not need to change this parameter.
inplace : bool
    Whether to modify the DataFrame rather than creating a new one.

Returns
-----
DataFrame or None
    DataFrame resulting from the provided query expression or
    None if ``inplace=True``.

See Also
-----
eval : Evaluate a string describing operations on
    DataFrame columns.
DataFrame.eval : Evaluate a string describing operations on

```

DataFrame columns.

Notes

The result of the evaluation of this expression is first passed to :attr:`DataFrame.loc` and if that fails because of a multidimensional key (e.g., a DataFrame) then the result will be passed to :meth:`DataFrame.\_\_getitem\_\_`.

This method uses the top-level :func:`eval` function to evaluate the passed query.

The :meth:`~pandas.DataFrame.query` method uses a slightly modified Python syntax by default. For example, the ``&`` and ``|`` (bitwise) operators have the precedence of their boolean cousins, :keyword:`and` and :keyword:`or`. This *is* syntactically valid Python, however the semantics are different.

You can change the semantics of the expression by passing the keyword argument ``parser='python'``. This enforces the same semantics as evaluation in Python space. Likewise, you can pass ``engine='python'`` to evaluate an expression using Python itself as a backend. This is not recommended as it is inefficient compared to using ``numexpr`` as the engine.

The :attr:`DataFrame.index` and :attr:`DataFrame.columns` attributes of the :class:`~pandas.DataFrame` instance are placed in the query namespace by default, which allows you to treat both the index and columns of the frame as a column in the frame.

The identifier ``index`` is used for the frame index; you can also use the name of the index to identify it in a query. Please note that Python keywords may not be used as identifiers.

For further details and examples see the ``query`` documentation in :ref:`indexing <indexing.query>`.

Backtick quoted variables

Backtick quoted variables are parsed as literal Python code and are converted internally to a Python valid identifier. This can lead to the following problems.

During parsing a number of disallowed characters inside the backtick quoted string are replaced by strings that are allowed as a Python identifier. These characters include all operators in Python, the space character, the question mark, the exclamation mark, the dollar sign, and the euro sign.

A backtick can be escaped by double backticks.

See also the `Python documentation about lexical analysis <[https://docs.python.org/3/reference/lexical\\_analysis.html](https://docs.python.org/3/reference/lexical_analysis.html)>` in combination with the source code in :mod:`pandas.core.computation.parsing`.

Examples

```
>>> df = pd.DataFrame(
...     {"A": range(1, 6), "B": range(10, 0, -2), "C&C": range(10, 5, -1)}
... )
>>> df
   A  B  C&C
0  1 10    9
1  2  8    8
2  3  6    7
3  4  4    6
4  5  2    5
>>> df.query("A > B")
   A  B  C&C
4  5  2    5
```

The previous expression is equivalent to

```
>>> df[df.A > df.B]
   A  B  C&C
4  5  2    5
```

For columns with spaces in their name, you can use backtick quoting.

```
>>> df.query("B == `C&C`")
   A  B  C&C
0  1  10   10
```

The previous expression is equivalent to

```
>>> df[df.B == df["C&C"]]
   A  B  C&C
0  1  10   10
```

Using local variable:

```
>>> local_var = 2
>>> df.query("A <= @local_var")
   A  B  C&C
0  1  10   10
1  2   8    9
```

`radd(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Addition of dataframe and other, element-wise (binary operator ``radd``).

Equivalent to ``other + dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``add``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                    'degrees': [360, 180, 360]},
...                    index=['circle', 'triangle', 'rectangle'])
>>> df
      angles  degrees
circle      0     360
triangle    3     180
rectangle   4     360
```

Add a scalar with operator version which return the same results.

```
>>> df + 1
      angles  degrees
circle      1     361
triangle    4     181
rectangle   5     361
```

```
>>> df.add(1)
      angles  degrees
circle      1     361
triangle    4     181
rectangle   5     361
```

Divide by constant with reverse version.

```
>>> df.div(10)
      angles  degrees
circle    0.0     36.0
triangle  0.3     18.0
rectangle 0.4     36.0
```

```
>>> df.rdiv(10)
      angles  degrees
circle      inf  0.027778
triangle  3.333333 0.055556
rectangle 2.500000 0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
      angles  degrees
circle     -1     358
triangle    2     178
rectangle    3     358
```

```
>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1     358
triangle    2     178
rectangle    3     358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1     359
triangle    2     179
rectangle    3     359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0     720
triangle    0     360
rectangle    0     720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6     360
rectangle    12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle     3
rectangle     4
```

```
>>> df * other
      angles  degrees
circle      0      NaN
triangle     9      NaN
rectangle    16      NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0      0.0
triangle    9      0.0
rectangle   16      0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
>>> df_multindex
```

```
      angles  degrees
A circle      0      360
  triangle      3      180
  rectangle      4      360
B square       4      360
  pentagon      5      540
  hexagon       6      720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
```

```
      angles  degrees
A circle   NaN      1.0
  triangle  1.0      1.0
  rectangle 1.0      1.0
B square   0.0      0.0
  pentagon 0.0      0.0
  hexagon  0.0      0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                         'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
```

```
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

```
rdiv = rtruediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame
```

```
reindex(
    self,
    labels=None,
    *,
    index=None,
    columns=None,
    axis: Axis | None = None,
    method: ReindexMethod | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    level: Level | None = None,
    fill_value: Scalar | None = nan,
    limit: int | None = None,
    tolerance=None
)
```

-> DataFrame from pandas.core.frame.DataFrame
Conform DataFrame to new index with optional filling logic.

Places NA/NAN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and ``copy=False``.

Parameters

labels : array-like, optional
New labels / index to conform the axis specified by 'axis' to.
index : array-like, optional
New labels for the index. Preferably an Index object to avoid duplicating data.
columns : array-like, optional
New labels for the columns. Preferably an Index object to avoid duplicating data.
axis : int or str, optional
Axis to target. Can be either the axis name ('index', 'columns') or number (0, 1).

```

method : {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}
    Method to use for filling holes in reindexed DataFrame.
    Please note: this is only applicable to DataFrames/Series with a
    monotonically increasing/decreasing index.

    * None (default): don't fill gaps
    * pad / ffill: Propagate last valid observation forward to next
      valid.
    * backfill / bfill: Use next valid observation to fill gap.
    * nearest: Use nearest valid observations to fill gap.

copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user\_guide/copy\_on\_write.html>`__
    for more details.

level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : scalar, default np.nan
    Value to use for missing values. Defaults to NaN, but can be any
    "compatible" value.
limit : int, default None
    Maximum number of consecutive elements to forward or backward fill.
tolerance : optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

    Tolerance may be a scalar value, which applies the same tolerance
    to all values, or list-like, which applies variable tolerance per
    element. List-like includes list, tuple, array, Series, and must be
    the same size as the index and its dtype must exactly match the
    index's type.

Returns
-----
DataFrame
    DataFrame with changed index.

See Also
-----
DataFrame.set_index : Set row labels.
DataFrame.reset_index : Remove row labels or move them to new columns.
DataFrame.reindex_like : Change to same indices as other DataFrame.

Examples
-----
``DataFrame.reindex`` supports two calling conventions

* ``(index=index_labels, columns=column_labels, ...)``
* ``(labels, axis={'index', 'columns'}, ...)``

We *highly* recommend using keyword arguments to clarify your
intent.

Create a DataFrame with some fictional data.

>>> index = ["Firefox", "Chrome", "Safari", "IE10", "Konqueror"]
>>> columns = ["http_status", "response_time"]
>>> df = pd.DataFrame(
...     [[200, 0.04], [200, 0.02], [404, 0.07], [404, 0.08], [301, 1.0]],
...     columns=columns,
...     index=index,
... )
>>> df

```

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02

Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

Create a new index and reindex the DataFrame. By default values in the new index that do not have corresponding records in the DataFrame are assigned ``NaN``.

```
>>> new_index = ["Safari", "Iceweasel", "Comodo Dragon", "IE10", "Chrome"]
>>> df.reindex(new_index)
```

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02

We can fill in the missing values by passing a value to the keyword ``fill\_value``. Because the index is not monotonically increasing or decreasing, we cannot use arguments to the keyword ``method`` to fill the ``NaN`` values.

```
>>> df.reindex(new_index, fill_value=0)
```

	http_status	response_time
Safari	404	0.07
Iceweasel	0	0.00
Comodo Dragon	0	0.00
IE10	404	0.08
Chrome	200	0.02

```
>>> df.reindex(new_index, fill_value="missing")
```

	http_status	response_time
Safari	404	0.07
Iceweasel	missing	missing
Comodo Dragon	missing	missing
IE10	404	0.08
Chrome	200	0.02

We can also reindex the columns.

```
>>> df.reindex(columns=["http_status", "user_agent"])
```

	http_status	user_agent
Firefox	200	NaN
Chrome	200	NaN
Safari	404	NaN
IE10	404	NaN
Konqueror	301	NaN

Or we can use "axis-style" keyword arguments

```
>>> df.reindex(["http_status", "user_agent"], axis="columns")
```

	http_status	user_agent
Firefox	200	NaN
Chrome	200	NaN
Safari	404	NaN
IE10	404	NaN
Konqueror	301	NaN

To further illustrate the filling functionality in ``reindex``, we will create a DataFrame with a monotonically increasing index (for example, a sequence of dates).

```
>>> date_index = pd.date_range("1/1/2010", periods=6, freq="D")
>>> df2 = pd.DataFrame(
...     {"prices": [100, 101, np.nan, 100, 89, 88]}, index=date_index
... )
>>> df2
```

	prices
2010-01-01	100.0
2010-01-02	101.0
2010-01-03	NaN
2010-01-04	100.0
2010-01-05	89.0
2010-01-06	88.0

Suppose we decide to expand the DataFrame to cover a wider

date range.

```
>>> date_index2 = pd.date_range("12/29/2009", periods=10, freq="D")
>>> df2.reindex(date_index2)
```

```
prices
2009-12-29    NaN
2009-12-30    NaN
2009-12-31    NaN
2010-01-01    100.0
2010-01-02    101.0
2010-01-03    NaN
2010-01-04    100.0
2010-01-05     89.0
2010-01-06     88.0
2010-01-07    NaN
```

The index entries that did not have a value in the original data frame (for example, '2009-12-29') are by default filled with ``NaN``. If desired, we can fill in the missing values using one of several options.

For example, to back-propagate the last valid value to fill the ``NaN`` values, pass ``bfill`` as an argument to the ``method`` keyword.

```
>>> df2.reindex(date_index2, method="bfill")
```

```
prices
2009-12-29    100.0
2009-12-30    100.0
2009-12-31    100.0
2010-01-01    100.0
2010-01-02    101.0
2010-01-03     NaN
2010-01-04    100.0
2010-01-05     89.0
2010-01-06     88.0
2010-01-07     NaN
```

Please note that the ``NaN`` value present in the original DataFrame (at index value 2010-01-03) will not be filled by any of the value propagation schemes. This is because filling while reindexing does not look at DataFrame values, but only compares the original and desired indexes. If you do want to fill in the ``NaN`` values present in the original DataFrame, use the ``fillna()`` method.

See the :ref:`user guide <basics.reindexing>` for more.

```
rename(
    self,
    mapper: Renamer | None = None,
    *,
    index: Renamer | None = None,
    columns: Renamer | None = None,
    axis: Axis | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    inplace: bool = False,
    level: Level | None = None,
    errors: IgnoreRaise = 'ignore'
) -> DataFrame | None from pandas.core.frame.DataFrame
Rename columns or index labels.
```

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

See the :ref:`user guide <basics.rename>` for more.

Parameters

mapper : dict-like or function

Dict-like or function transformations to apply to that axis' values. Use either ``mapper`` and ``axis`` to specify the axis to target with ``mapper``, or ``index`` and ``columns``.

index : dict-like or function

Alternative to specifying axis (``mapper, axis=0`` is equivalent to ``index=mapper``).

columns : dict-like or function

Alternative to specifying axis (`mapper, axis=1`) is equivalent to `columns=mapper`).

axis : {0 or 'index', 1 or 'columns'}, default 0
Axis to target with `mapper`. Can be either the axis name ('index', 'columns') or number (0, 1). The default is 'index'.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` (https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

inplace : bool, default False
Whether to modify the DataFrame rather than creating a new one. If True then value of copy is ignored.

level : int or level name, default None
In case of a MultiIndex, only rename labels in the specified level.

errors : {'ignore', 'raise'}, default 'ignore'
If 'raise', raise a `KeyError` when a dict-like `mapper`, `index`, or `columns` contains labels that are not present in the Index being transformed.
If 'ignore', existing keys will be renamed and extra keys will be ignored.

Returns

DataFrame or None
DataFrame with the renamed axis labels or None if `inplace=True`.

Raises

KeyError
If any of the labels is not found in the selected axis and `errors='raise'`.

See Also

DataFrame.rename_axis : Set the name of the axis.

Examples

`DataFrame.rename` supports two calling conventions

```
* ``(index=index_mapper, columns=columns_mapper, ...)``
* ``(mapper, axis={'index', 'columns'}, ...)``
```

We *highly* recommend using keyword arguments to clarify your intent.

Rename columns using a mapping:

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
>>> df.rename(columns={"A": "a", "B": "c"})
   a  c
0  1  4
1  2  5
2  3  6
```

Rename index using a mapping:

```
>>> df.rename(index={0: "x", 1: "y", 2: "z"})
   A  B
x  1  4
y  2  5
z  3  6
```

Cast index labels to a different type:

```
>>> df.index
RangeIndex(start=0, stop=3, step=1)
```

```
>>> df.rename(index=str).index
Index(['0', '1', '2'], dtype='str')

>>> df.rename(columns={"A": "a", "B": "b", "C": "c"}, errors="raise")
Traceback (most recent call last):
  File "<ipython-input-1-1>:1", line 1, in <module>
    df.rename(columns={"A": "a", "B": "b", "C": "c"}, errors="raise")
  File "C:\Users\user\AppData\Local\Programs\Python\Python38-32\lib\site-packages\pandas\core\frame.py", line 2922, in rename
    raise KeyError(msg)
KeyError: ["C"] not found in axis
```

Using axis-style parameters:

```
>>> df.rename(str.lower, axis="columns")
   a  b
0  1  4
1  2  5
2  3  6
```

```
>>> df.rename({1: 2, 2: 4}, axis="index")
   A  B
0  1  4
2  2  5
4  3  6
```

reorder_levels(self, order: Sequence[int | str], axis: Axis = 0) -> DataFrame from pandas.com
Rearrange index or column levels using input ``order``.

May not drop or duplicate levels.

Parameters

order : list of int or list of str
List representing new level order. Reference level by number (position) or by key (label).
axis : {0 or 'index', 1 or 'columns'}, default 0
Where to reorder levels.

Returns

DataFrame
DataFrame with indices or columns with reordered levels.

See Also

DataFrame.swaplevel : Swap levels i and j in a MultiIndex.

Examples

```
>>> data = {
...     "class": ["Mammals", "Mammals", "Reptiles"],
...     "diet": ["Omnivore", "Carnivore", "Carnivore"],
...     "species": ["Humans", "Dogs", "Snakes"],
... }
>>> df = pd.DataFrame(data, columns=["class", "diet", "species"])
>>> df = df.set_index(["class", "diet"])
>>> df
```

	class	diet	species
Mammals	Omnivore		Humans
	Carnivore		Dogs
Reptiles	Carnivore		Snakes

Let's reorder the levels of the index:

```
>>> df.reorder_levels(["diet", "class"])
   diet  class  species
Omnivore Mammals  Humans
Carnivore Mammals   Dogs
          Reptiles  Snakes
```

```
reset_index(
    self,
    level: IndexLabel | None = None,
    *,
    drop: bool = False,
    inplace: bool = False,
    col_level: Hashable = 0,
    col_fill: Hashable = '',
    allow_duplicates: bool | lib.NoDefault = <no_default>,
```

```
names: Hashable | Sequence[Hashable] | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Reset the index, or a level of it.
```

Reset the index of the DataFrame, and use the default one instead.
If the DataFrame has a MultiIndex, this method can remove one or more levels.

Parameters

```
level : int, str, tuple, or list, default None
    Only remove the given levels from the index. Removes all levels by
    default.
drop : bool, default False
    Do not try to insert index into dataframe columns. This resets
    the index to the default integer index.
inplace : bool, default False
    Whether to modify the DataFrame rather than creating a new one.
col_level : int or str, default 0
    If the columns have multiple levels, determines which level the
    labels are inserted into. By default it is inserted into the first
    level.
col_fill : object, default ''
    If the columns have multiple levels, determines how the other
    levels are named. If None then the index name is repeated.
allow_duplicates : bool, optional, default lib.no_default
    Allow duplicate column labels to be created.
names : int, str or 1-dimensional list, default None
    Using the given string, rename the DataFrame column which contains the
    index data. If the DataFrame has a MultiIndex, this has to be a list
    with length equal to the number of levels.
```

Returns

```
DataFrame or None
    DataFrame with the new index or None if ``inplace=True``.
```

See Also

```
DataFrame.set_index : Opposite of reset_index.
DataFrame.reindex : Change to new indices or expand indices.
DataFrame.reindex_like : Change to same indices as other DataFrame.
```

Examples

```
>>> df = pd.DataFrame(
...     [("bird", 389.0), ("bird", 24.0), ("mammal", 80.5), ("mammal", np.nan)],
...     index=["falcon", "parrot", "lion", "monkey"],
...     columns=("class", "max_speed"),
... )
>>> df
```

	class	max_speed
falcon	bird	389.0
parrot	bird	24.0
lion	mammal	80.5
monkey	mammal	NaN

When we reset the index, the old index is added as a column, and a new sequential index is used:

```
>>> df.reset_index()
   index  class  max_speed
0  falcon   bird    389.0
1  parrot   bird     24.0
2    lion  mammal     80.5
3  monkey  mammal     NaN
```

We can use the `drop` parameter to avoid the old index being added as a column:

```
>>> df.reset_index(drop=True)
   class  max_speed
0    bird    389.0
1    bird     24.0
2  mammal     80.5
3  mammal     NaN
```

You can also use `reset\_index` with `MultiIndex`.

```
>>> index = pd.MultiIndex.from_tuples(
...     [
...         ("bird", "falcon"),
...         ("bird", "parrot"),
...         ("mammal", "lion"),
...         ("mammal", "monkey"),
...     ],
...     names=["class", "name"],
... )
>>> columns = pd.MultiIndex.from_tuples([("speed", "max"), ("species", "type")])
>>> df = pd.DataFrame(
...     [(389.0, "fly"), (24.0, "fly"), (80.5, "run"), (np.nan, "jump")],
...     index=index,
...     columns=columns,
... )
>>> df
```

		speed	species
		max	type
class	name		
bird	falcon	389.0	fly
	parrot	24.0	fly
mammal	lion	80.5	run
	monkey	NaN	jump

Using the `names` parameter, choose a name for the index column:

```
>>> df.reset_index(names=["classes", "names"])
   classes  names  speed species
   max      type
0    bird  falcon  389.0    fly
1    bird  parrot   24.0    fly
2  mammal   lion   80.5    run
3  mammal  monkey   NaN    jump
```

If the index has multiple levels, we can reset a subset of them:

```
>>> df.reset_index(level="class")
   class  speed species
   max      type
name
falcon   bird  389.0    fly
parrot   bird   24.0    fly
lion    mammal   80.5    run
monkey  mammal   NaN    jump
```

If we are not dropping the index, by default, it is placed in the top level. We can place it in another level:

```
>>> df.reset_index(level="class", col_level=1)
   class  speed species
   max      type
name
falcon   bird  389.0    fly
parrot   bird   24.0    fly
lion    mammal   80.5    run
monkey  mammal   NaN    jump
```

When the index is inserted under another level, we can specify under which one with the parameter `col\_fill`:

```
>>> df.reset_index(level="class", col_level=1, col_fill="species")
   species  speed species
   class    max      type
name
falcon     bird  389.0    fly
parrot     bird   24.0    fly
lion      mammal   80.5    run
monkey     mammal   NaN    jump
```

If we specify a nonexistent level for `col\_fill`, it is created:

```
>>> df.reset_index(level="class", col_level=1, col_fill="genus")
   genus  speed species
   class    max      type
name
```

falcon	bird	389.0	fly
parrot	bird	24.0	fly
lion	mammal	80.5	run
monkey	mammal	NaN	jump

`rfloordiv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from
Get Integer division of dataframe and other, element-wise (binary operator ``rfloordiv``).

Equivalent to ``other // dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``floordiv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

Subtract a list and Series by axis with operator version.

Multiply a dictionary by axis.

Multiply a DataFrame of different shape with operator version.

Divide by a MultiIndex by level.

[illegible]

```

>>> df_multindex
      angles  degrees
A circle      0     360
  triangle      3     180
 rectangle      4     360
B square      4     360
  pentagon      5     540
  hexagon      6     720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle   NaN     1.0
  triangle  1.0     1.0
 rectangle  1.0     1.0
B square   0.0     0.0
  pentagon  0.0     0.0
  hexagon   0.0     0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81

```

`rmod(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from pandas`
 Get Modulo of dataframe and other, element-wise (binary operator ``rmod``).

Equivalent to ``other % dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``mod``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
 Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
 Whether to compare by the index (0 or 'index') or columns.
 (1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
 Broadcast across a level, matching Index values on the
 passed MultiIndex level.
`fill_value` : float or None, default None
 Fill existing missing (NaN) values, and any new element needed for
 successful DataFrame alignment, with this value before computation.
 If data in both corresponding DataFrame locations is missing
 the result will be missing.

Returns

DataFrame
 Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                   'degrees': [360, 180, 360]},
...                   index=['circle', 'triangle', 'rectangle'])

```

```

>>> df
      angles  degrees
circle      0      360
triangle    3      180
rectangle   4      360

Add a scalar with operator version which return the same
results.

>>> df + 1
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361

>>> df.add(1)
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361

Divide by constant with reverse version.

>>> df.div(10)
      angles  degrees
circle    0.0      36.0
triangle  0.3      18.0
rectangle 0.4      36.0

>>> df.rdiv(10)
      angles  degrees
circle      inf  0.027778
triangle  3.333333  0.055556
rectangle  2.500000  0.027778

Subtract a list and Series by axis with operator version.

>>> df - [1, 2]
      angles  degrees
circle     -1      358
triangle    2      178
rectangle   3      358

>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1      358
triangle    2      178
rectangle   3      358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1      359
triangle    2      179
rectangle   3      359

Multiply a dictionary by axis.

>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0      720
triangle    0      360
rectangle   0      720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6      360
rectangle    12     1080

Multiply a DataFrame of different shape with operator version.

>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0

```



```
triangle      3
rectangle     4
```

```
>>> df * other
      angles  degrees
circle      0     NaN
triangle    9     NaN
rectangle   16     NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0     0.0
triangle    9     0.0
rectangle   16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0     360
  triangle    3     180
  rectangle    4     360
B square      4     360
  pentagon    5     540
  hexagon     6     720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle  1.0     1.0
  rectangle  1.0     1.0
B square    0.0     0.0
  pentagon  0.0     0.0
  hexagon   0.0     0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                         'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

`rmul(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Multiplication of dataframe and other, element-wise (binary operator ``rmul``).

Equivalent to ``other * dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``mul``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

>>> df = pd.DataFrame({'angles': [0, 3, 4],
... 'degrees': [360, 180, 360]},
... index=['circle', 'triangle', 'rectangle'])
>>> df

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

>>> df + 1

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

>>> df.add(1)

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

>>> df.div(10)

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

>>> df.rdiv(10)

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

>>> df - [1, 2]

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

>>> df.sub([1, 2], axis='columns')

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
... axis='index')

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0     720
triangle    0     360
rectangle   0     720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6     360
rectangle    12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle     3
rectangle     4

>>> df * other
      angles  degrees
circle      0     NaN
triangle     9     NaN
rectangle    16     NaN

>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0     0.0
triangle     9     0.0
rectangle    16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                              'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                    ['circle', 'triangle', 'rectangle',
...                                     'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0     360
  triangle     3     180
  rectangle     4     360
B square      4     360
  pentagon     5     540
  hexagon      6     720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle  1.0     1.0
  rectangle  1.0     1.0
B square     0.0     0.0
  pentagon   0.0     0.0
  hexagon    0.0     0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

`round(self, decimals: int | dict[IndexLabel, int] | Series = 0, *args, **kwargs) -> DataFrame`
Round numeric columns in a DataFrame to a variable number of decimal places.

Parameters

decimals : int, dict, Series

Number of decimal places to round each column to. If an int is given, round each column to the same number of places. Otherwise dict and Series round to variable numbers of places. Column names should be in the keys if `decimals` is a dict-like, or in the index if `decimals` is a Series. Any columns not included in `decimals` will be left as is. Elements of `decimals` which are not columns of the input will be ignored.

***args**

Additional keywords have no effect but might be accepted for compatibility with numpy.

****kwargs**

Additional keywords have no effect but might be accepted for compatibility with numpy.

Returns

DataFrame

A DataFrame with the affected columns rounded to the specified number of decimal places.

See Also

`numpy.around` : Round a numpy array to the given number of decimals.

`Series.round` : Round a Series to the given number of decimals.

Notes

For values exactly halfway between rounded decimal values, pandas rounds to the nearest even value (e.g. -0.5 and 0.5 round to 0.0, 1.5 and 2.5 round to 2.0, etc.).

Examples

```
>>> df = pd.DataFrame(
...     [(0.21, 0.32), (0.01, 0.67), (0.66, 0.03), (0.21, 0.18)],
...     columns=["dogs", "cats"],
... )
>>> df
   dogs  cats
0  0.21  0.32
1  0.01  0.67
2  0.66  0.03
3  0.21  0.18
```

By providing an integer each column is rounded to the same number of decimal places

```
>>> df.round(1)
   dogs  cats
0  0.2  0.3
1  0.0  0.7
2  0.7  0.0
3  0.2  0.2
```

With a dict, the number of places for specific columns can be specified with the column names as key and the number of decimal places as value

```
>>> df.round({"dogs": 1, "cats": 0})
   dogs  cats
0  0.2  0.0
1  0.0  1.0
2  0.7  0.0
3  0.2  0.0
```

Using a Series, the number of places for specific columns can be specified with the column names as index and the number of decimal places as value

```
>>> decimals = pd.Series([0, 1], index=["cats", "dogs"])
>>> df.round(decimals)
   dogs  cats
0  0.2  0.0
1  0.0  1.0
2  0.7  0.0
3  0.2  0.0
```

`rpow(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Exponential power of dataframe and other, element-wise (binary operator ``rpow``).

Equivalent to ``other ** dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``pow``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.
`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.
`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0

triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
      angles  degrees
circle      inf  0.027778
triangle  3.333333  0.055556
rectangle  2.500000  0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
      angles  degrees
circle      -1    358
triangle     2    178
rectangle     3    358

>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle      -1    358
triangle     2    178
rectangle     3    358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle      -1    359
triangle     2    179
rectangle     3    359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle         0    720
triangle        0    360
rectangle        0    720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle         0         0
triangle         6    360
rectangle        12   1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle         0
triangle         3
rectangle         4

>>> df * other
      angles  degrees
circle         0     NaN
triangle         9     NaN
rectangle        16     NaN

>>> df.mul(other, fill_value=0)
      angles  degrees
circle         0     0.0
triangle         9     0.0
rectangle        16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                      ['circle', 'triangle', 'rectangle',
...                                       'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle         0    360
  triangle         3    180
```

```

rectangle      4      360
B square       4      360
pentagon       5      540
hexagon        6      720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN     1.0
  triangle   1.0     1.0
  rectangle   1.0     1.0
B square     0.0     0.0
  pentagon   0.0     0.0
  hexagon    0.0     0.0

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81

```

`rsub(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Subtraction of dataframe and other, element-wise (binary operator ``rsub``).

Equivalent to ``other - dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``sub``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.

`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.

`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

DataFrame
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.
`DataFrame.mul` : Multiply DataFrames.
`DataFrame.div` : Divide DataFrames (float division).
`DataFrame.truediv` : Divide DataFrames (float division).
`DataFrame.floordiv` : Divide DataFrames (integer division).
`DataFrame.mod` : Calculate modulo (remainder after division).
`DataFrame.pow` : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                    'degrees': [360, 180, 360]},
...                    index=['circle', 'triangle', 'rectangle'])
>>> df
      angles  degrees
circle      0     360
triangle    3     180

```

```
rectangle      4      360
```

Add a scalar with operator version which return the same results.

```
>>> df + 1
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361
```

```
>>> df.add(1)
      angles  degrees
circle      1      361
triangle    4      181
rectangle   5      361
```

Divide by constant with reverse version.

```
>>> df.div(10)
      angles  degrees
circle     0.0     36.0
triangle   0.3     18.0
rectangle  0.4     36.0
```

```
>>> df.rdiv(10)
      angles  degrees
circle      inf  0.027778
triangle   3.333333  0.055556
rectangle  2.500000  0.027778
```

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358
```

```
>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle     -1     358
triangle    2     178
rectangle   3     358
```

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle     -1     359
triangle    2     179
rectangle   3     359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle      0     720
triangle    0     360
rectangle   0     720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0         0
triangle     6     360
rectangle   12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle     3
rectangle    4

>>> df * other
```


	angles	degrees
circle	0	NaN
triangle	9	NaN
rectangle	16	NaN

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0      0.0
triangle    9      0.0
rectangle  16      0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0      360
  triangle      3      180
  rectangle      4      360
B square       4      360
  pentagon      5      540
  hexagon       6      720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle   NaN      1.0
  triangle  1.0      1.0
  rectangle 1.0      1.0
B square    0.0      0.0
  pentagon  0.0      0.0
  hexagon   0.0      0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                         'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

`rtruediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame from`
`Get Floating division of dataframe and other, element-wise (binary operator `rtruediv`).`

Equivalent to ``other / dataframe``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``truediv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.

`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns.
(1 or 'columns'). For Series input, axis to match Series index on.

`level` : int or label
Broadcast across a level, matching Index values on the
passed MultiIndex level.

`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for
successful DataFrame alignment, with this value before computation.
If data in both corresponding DataFrame locations is missing
the result will be missing.

Returns

DataFrame

Result of the arithmetic operation.

See Also

```

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

```

Notes

Mismatched indices will be unioned together.

Examples

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df

```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```

>>> df + 1

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```

>>> df.add(1)

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```

>>> df.div(10)

```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```

>>> df.rdiv(10)

```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```

>>> df - [1, 2]

```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub([1, 2], axis='columns')

```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')

```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```

>>> df.mul({'angles': 0, 'degrees': 2})

```

	angles	degrees
circle	0	720
triangle	0	360
rectangle	0	720

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0      0
triangle    6     360
rectangle   12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle    3
rectangle    4
```

```
>>> df * other
      angles  degrees
circle      0      NaN
triangle     9      NaN
rectangle   16      NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0      0.0
triangle     9      0.0
rectangle   16      0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                              'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0     360
  triangle    3     180
  rectangle    4     360
B square      4     360
  pentagon    5     540
  hexagon     6     720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN      1.0
  triangle  1.0      1.0
  rectangle  1.0      1.0
B square    0.0      0.0
  pentagon  0.0      0.0
  hexagon   0.0      0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

`select_dtypes(self, include=None, exclude=None) -> DataFrame` from `pandas.core.frame.DataFrame`
Return a subset of the DataFrame's columns based on the column dtypes.

This method allows for filtering columns based on their data types. It is useful when working with heterogeneous DataFrames where operations need to be performed on a specific subset of data types.

Parameters

include, exclude : scalar or list-like

A selection of dtypes or strings to be included/excluded. At least one of these parameters must be supplied.

Returns

DataFrame

The subset of the frame including the dtypes in ``include`` and excluding the dtypes in ``exclude``.

Raises

ValueError

* If both of ``include`` and ``exclude`` are empty

* If ``include`` and ``exclude`` have overlapping elements

TypeError

* If any kind of string dtype is passed in.

See Also

DataFrame.dtypes: Return Series with the data type of each column.

Notes

* To select all *numeric* types, use ``np.number`` or ``'number'``

* To select strings you must use the ``object`` dtype, but note that this will return *all* object dtype columns. With ``pd.options.future.infer\_string`` enabled, using ``"str"`` will work to select all string columns.

* See the `numpy dtype hierarchy`_

<<https://numpy.org/doc/stable/reference/arrays.scalars.html>>__

* To select datetimes, use ``np.datetime64``, ``'datetime'`` or ``'datetime64'``

* To select timedeltas, use ``np.timedelta64``, ``'timedelta'`` or ``'timedelta64'``

* To select Pandas categorical dtypes, use ``'category'``

* To select Pandas datetimetz dtypes, use ``'datetimetz'`` or ``'datetime64[ns, tz]'``

Examples

```
>>> df = pd.DataFrame(
...     {"a": [1, 2] * 3, "b": [True, False] * 3, "c": [1.0, 2.0] * 3}
... )
>>> df
```

```
   a  b  c
0  1  True  1.0
1  2  False  2.0
2  1  True  1.0
3  2  False  2.0
4  1  True  1.0
5  2  False  2.0
```

```
>>> df.select_dtypes(include="bool")
```

```
   b
0  True
1  False
2  True
3  False
4  True
5  False
```

```
>>> df.select_dtypes(include=["float64"])
```

```
   c
0  1.0
1  2.0
2  1.0
3  2.0
4  1.0
5  2.0
```

```
>>> df.select_dtypes(exclude=["int64"])
```

```
   b  c
0  True  1.0
1  False  2.0
2  True  1.0
3  False  2.0
4  True  1.0
```

```

5 False 2.0

sem(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
    Return unbiased standard error of the mean over requested axis.

    Normalized by N-1 by default. This can be changed using the ddof argument

    Parameters
    -----
    axis : {index (0), columns (1)}
        For `Series` this parameter is unused and defaults to 0.

        .. warning::

            The behavior of DataFrame.sem with ``axis=None`` is deprecated,
            in a future version this will reduce over both axes and return a scalar
            To retain the old behavior, pass axis=0 (or do not pass axis).

    skipna : bool, default True
        Exclude NA/null values. If an entire row/column is NA, the result
        will be NA.
    ddof : int, default 1
        Delta Degrees of Freedom. The divisor used in calculations is N - ddof,
        where N represents the number of elements.
    numeric_only : bool, default False
        Include only float, int, boolean columns. Not implemented for Series.
    **kwargs :
        Additional keywords passed.

    Returns
    -----
    Series or DataFrame (if level specified)
        Unbiased standard error of the mean over requested axis.

    See Also
    -----
    DataFrame.var : Return unbiased variance over requested axis.
    DataFrame.std : Returns sample standard deviation over requested axis.

    Examples
    -----
    >>> s = pd.Series([1, 2, 3])
    >>> round(s.sem(), 6)
    0.57735

    With a DataFrame

    >>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
    >>> df
           a  b
    tiger  1  2
    zebra  2  3
    >>> df.sem()
    a    0.5
    b    0.5
    dtype: float64

    Using axis=1

    >>> df.sem(axis=1)
    tiger    0.5
    zebra    0.5
    dtype: float64

    In this case, `numeric_only` should be set to `True`
    to avoid getting an error.

    >>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
    >>> df.sem(numeric_only=True)

```

```

a    0.5
dtype: float64

set_axis(
    self,
    labels,
    *,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
    Assign desired index to given axis.

Indexes for column or row labels can be changed by assigning
a list-like or Index.

Parameters
-----
labels : list-like, Index
    The values for the new index.

axis : {0 or 'index', 1 or 'columns'}, default 0
    The axis to update. The value 0 identifies the rows. For `Series`
    this parameter is unused and defaults to 0.

copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
    for more details.

Returns
-----
DataFrame
    An object of type DataFrame.

See Also
-----
DataFrame.rename_axis : Alter the name of the index or columns.

Examples
-----
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})

Change the row labels.

>>> df.set_axis(["a", "b", "c"], axis="index")
   A  B
a  1  4
b  2  5
c  3  6

Change the column labels.

>>> df.set_axis(["I", "II"], axis="columns")
   I  II
0  1   4
1  2   5
2  3   6

set_index(
    self,
    keys,
    *,
    drop: bool = True,
    append: bool = False,
    inplace: bool = False,
    verify_integrity: bool | lib.NoDefault = <no_default>
) -> DataFrame | None from pandas.core.frame.DataFrame
    Set the DataFrame index using existing columns.

```

Set the DataFrame index (row labels) using one or more existing columns or arrays (of the correct length). The index can replace the existing index or expand on it.

Parameters

keys : label or array-like or list of labels/arrays
This parameter can be either a single column key, a single array of the same length as the calling DataFrame, or a list containing an arbitrary combination of column keys and arrays. Here, "array" encompasses `:class:`Series``, `:class:`Index``, ``np.ndarray``, and instances of `:class:`~collections.abc.Iterator``.

drop : bool, default True
Delete columns to be used as the new index.

append : bool, default False
Whether to append columns to existing index.
Setting to True will add the new columns to existing index.
When set to False, the current index will be dropped from the DataFrame.

inplace : bool, default False
Whether to modify the DataFrame rather than creating a new one.

verify_integrity : bool, default False
Check the new index for duplicates. Otherwise defer the check until necessary. Setting to False will improve the performance of this method.

.. deprecated:: 3.0.0

Returns

DataFrame or None
Changed row labels or None if `inplace=True`.

See Also

`DataFrame.reset_index` : Opposite of `set_index`.
`DataFrame.reindex` : Change to new indices or expand indices.
`DataFrame.reindex_like` : Change to same indices as other DataFrame.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "month": [1, 4, 7, 10],
...         "year": [2012, 2014, 2013, 2014],
...         "sale": [55, 40, 84, 31],
...     }
... )
>>> df
   month  year  sale
0      1  2012    55
1      4  2014    40
2      7  2013    84
3     10  2014    31
```

Set the index to become the 'month' column:

```
>>> df.set_index("month")
   year  sale
month
1     2012    55
4     2014    40
7     2013    84
10    2014    31
```

Create a MultiIndex using columns 'year' and 'month':

```
>>> df.set_index(["year", "month"])
   sale
year month
2012  1     55
2014  4     40
2013  7     84
2014  10    31
```

Create a MultiIndex using an Index and a column:

```
>>> df.set_index([pd.Index([1, 2, 3, 4]), "year"])
```

		month	sale
	year		
1	2012	1	55
2	2014	4	40
3	2013	7	84
4	2014	10	31

Create a MultiIndex using two Series:

```
>>> s = pd.Series([1, 2, 3, 4])
>>> df.set_index([s, s**2])
```

	month	year	sale	
1	1	1	2012	55
2	4	4	2014	40
3	9	7	2013	84
4	16	10	2014	31

Append a column to the existing index:

```
>>> df = df.set_index("month")
>>> df.set_index("year", append=True)
```

	month	year	sale
1	1	2012	55
4	4	2014	40
7	7	2013	84
10	10	2014	31

```
>>> df.set_index("year", append=False)
```

	month	year	sale
1	1	2012	55
4	4	2014	40
7	7	2013	84
10	10	2014	31

```
>>> df.set_index("year", append=False)
```

	year	sale
2012	55	
2014	40	
2013	84	
2014	31	

```
shift(
    self,
    periods: int | Sequence[int] = 1,
    freq: Frequency | None = None,
    axis: Axis = 0,
    fill_value: Hashable = <no_default>,
    suffix: str | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Shift index by desired number of periods with an optional time `freq`.
```

When `freq` is not passed, shift the index without realigning the data. If `freq` is passed (in this case, the index must be date or datetime, or it will raise a `NotImplementedError`), the index will be increased using the periods and the `freq`. `freq` can be inferred when specified as "infer" as long as either freq or inferred_freq attribute is set in the index.

Parameters

periods : int or Sequence
Number of periods to shift. Can be positive or negative. If an iterable of ints, the data will be shifted once by each int. This is equivalent to shifting by one value at a time and concatenating all resulting frames. The resulting columns will have the shift suffixed to their column names. For multiple periods, axis must not be 1.

freq : DateOffset, tseries.offsets, timedelta, or str, optional
Offset to use from the tseries module or time rule (e.g. 'EOM'). If `freq` is specified then the index values are shifted but the data is not realigned. That is, use `freq` if you would like to extend the index when shifting and preserve the original data. If `freq` is specified as "infer" then it will be inferred from the freq or inferred_freq attributes of the index. If neither of those attributes exist, a ValueError is thrown.

axis : {0 or 'index', 1 or 'columns', None}, default None
Shift direction. For `Series` this parameter is unused and defaults to 0.

fill_value : object, optional
The scalar value to use for newly introduced missing values. the default depends on the dtype of `self`. For Boolean and numeric NumPy data types, ``np.nan`` is used. For datetime, timedelta, or period data, etc. :attr:`NaT` is used.

For extension dtypes, ``self.dtype.na\_value`` is used.
suffix : str, optional
If str and periods is an iterable, this is added after the column name and before the shift value for each shifted column name.
For `Series` this parameter is unused and defaults to `None`.

Returns

DataFrame

Copy of input object, shifted.

See Also

Index.shift : Shift values of Index.
DatetimeIndex.shift : Shift values of DatetimeIndex.
PeriodIndex.shift : Shift values of PeriodIndex.

Examples

>>> df = pd.DataFrame(
... [[10, 13, 17], [20, 23, 27], [15, 18, 22], [30, 33, 37], [45, 48, 52]],
... columns=["Col1", "Col2", "Col3"],
... index=pd.date_range("2020-01-01", "2020-01-05"),
...)
>>> df

	Col1	Col2	Col3
2020-01-01	10	13	17
2020-01-02	20	23	27
2020-01-03	15	18	22
2020-01-04	30	33	37
2020-01-05	45	48	52

>>> df.shift(periods=3)

	Col1	Col2	Col3
2020-01-01	NaN	NaN	NaN
2020-01-02	NaN	NaN	NaN
2020-01-03	NaN	NaN	NaN
2020-01-04	10.0	13.0	17.0
2020-01-05	20.0	23.0	27.0

>>> df.shift(periods=1, axis="columns")

	Col1	Col2	Col3
2020-01-01	NaN	10	13
2020-01-02	NaN	20	23
2020-01-03	NaN	15	18
2020-01-04	NaN	30	33
2020-01-05	NaN	45	48

>>> df.shift(periods=3, fill_value=0)

	Col1	Col2	Col3
2020-01-01	0	0	0
2020-01-02	0	0	0
2020-01-03	0	0	0
2020-01-04	10	13	17
2020-01-05	20	23	27

>>> df.shift(periods=3, freq="D")

	Col1	Col2	Col3
2020-01-04	10	13	17
2020-01-05	20	23	27
2020-01-06	15	18	22
2020-01-07	30	33	37
2020-01-08	45	48	52

>>> df.shift(periods=3, freq="infer")

	Col1	Col2	Col3
2020-01-04	10	13	17
2020-01-05	20	23	27
2020-01-06	15	18	22
2020-01-07	30	33	37
2020-01-08	45	48	52

>>> df["Col1"].shift(periods=[0, 1, 2])

	Col1_0	Col1_1	Col1_2
2020-01-01	10	NaN	NaN
2020-01-02	20	10.0	NaN
2020-01-03	15	20.0	10.0

2020-01-04	30	15.0	20.0
2020-01-05	45	30.0	15.0

```
skew(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return unbiased skew over requested axis.

Normalized by N-1.

Parameters
-----
axis : {index (0), columns (1)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.

    .. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.

**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
Series or scalar
    Unbiased skew over requested axis.

See Also
-----
DataFrame.kurt : Returns unbiased kurtosis over requested axis.

Examples
-----
>>> s = pd.Series([1, 2, 3])
>>> s.skew()
0.0

With a DataFrame

>>> df = pd.DataFrame(
...     {"a": [1, 2, 3], "b": [2, 3, 4], "c": [1, 3, 5]},
...     index=["tiger", "zebra", "cow"],
... )
>>> df
   a  b  c
tiger  1  2  1
zebra  2  3  3
cow    3  4  5
>>> df.skew()
a    0.0
b    0.0
c    0.0
dtype: float64

Using axis=1

>>> df.skew(axis=1)
tiger    1.732051
zebra   -1.732051
cow      0.000000
dtype: float64

In this case, `numeric_only` should be set to `True` to avoid
getting an error.
```

```
>>> df = pd.DataFrame(
...     {"a": [1, 2, 3], "b": ["T", "Z", "X"]}, index=["tiger", "zebra", "cow"]
... )
>>> df.skew(numeric_only=True)
a    0.0
dtype: float64
```

```
sort_index(
    self,
    *,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    ascending: bool | Sequence[bool] = True,
    inplace: bool = False,
    kind: SortKind = 'quicksort',
    na_position: NaPosition = 'last',
    sort_remaining: bool = True,
    ignore_index: bool = False,
    key: IndexKeyFunc | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Sort object by labels (along an axis).
```

Returns a new DataFrame sorted by label if `inplace` argument is `False`, otherwise updates the original DataFrame and returns None.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
The axis along which to sort. The value 0 identifies the rows, and 1 identifies the columns.

level : int or level name or list of ints or list of level names
If not None, sort on values in specified index level(s).

ascending : bool or list-like of bools, default True
Sort ascending vs. descending. When the index is a MultiIndex the sort direction can be controlled for each level individually.

inplace : bool, default False
Whether to modify the DataFrame rather than creating a new one.

kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
Choice of sorting algorithm. See also :func:`numpy.sort` for more information. `mergesort` and `stable` are the only stable algorithms. For DataFrames, this option is only applied when sorting on a single column or label.

na_position : {'first', 'last'}, default 'last'
Puts NaNs at the beginning if `first`; `last` puts NaNs at the end. Not implemented for MultiIndex.

sort_remaining : bool, default True
If True and sorting by level and index is multilevel, sort by other levels too (in order) after sorting by specified level.

ignore_index : bool, default False
If True, the resulting axis will be labeled 0, 1, ..., n - 1.

key : callable, optional
If not None, apply the key function to the index values before sorting. This is similar to the `key` argument in the builtin :meth:`sorted` function, with the notable difference that this `key` function should be *vectorized*. It should expect an `Index` and return an `Index` of the same shape. For MultiIndex inputs, the key is applied *per level*.

Returns

DataFrame or None

The original DataFrame sorted by the labels or None if `inplace=True`.

See Also

Series.sort_index : Sort Series by the index.
DataFrame.sort_values : Sort DataFrame by the value.
Series.sort_values : Sort Series by the value.

Examples

```
>>> df = pd.DataFrame(
...     [1, 2, 3, 4, 5], index=[100, 29, 234, 1, 150], columns=["A"]
... )
>>> df.sort_index()
   A
1  4
```

```

29    2
100   1
150   5
234   3

```

By default, it sorts in ascending order, to sort in descending order, use ``ascending=False``

```

>>> df.sort_index(ascending=False)
      A
234    3
150    5
100    1
29     2
1      4

```

A key function can be specified which is applied to the index before sorting. For a ``MultiIndex`` this is applied to each level separately.

```

>>> df = pd.DataFrame({"a": [1, 2, 3, 4]}, index=["A", "b", "C", "d"])
>>> df.sort_index(key=lambda x: x.str.lower())
      a
A     1
b     2
C     3
d     4

```

```

sort_values(
    self,
    by: IndexLabel,
    *,
    axis: Axis = 0,
    ascending: bool | list[bool] | tuple[bool, ...] = True,
    inplace: bool = False,
    kind: SortKind = 'quicksort',
    na_position: str = 'last',
    ignore_index: bool = False,
    key: ValueKeyFunc | None = None
) -> DataFrame | None from pandas.core.frame.DataFrame
Sort by the values along either axis.

Parameters
-----
by : str or list of str
    Name or list of names to sort by.

    - if `axis` is 0 or `index` then `by` may contain index
      levels and/or column labels.
    - if `axis` is 1 or `columns` then `by` may contain column
      levels and/or index labels.
axis : "{0 or 'index', 1 or 'columns'}", default 0
    Axis to be sorted.
ascending : bool or list of bool, default True
    Sort ascending vs. descending. Specify list for multiple sort
    orders. If this is a list of bools, must match the length of
    the by.
inplace : bool, default False
    If True, perform operation in-place.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
    Choice of sorting algorithm. See also :func:`numpy.sort` for more
    information. `mergesort` and `stable` are the only stable algorithms. For
    DataFrames, this option is only applied when sorting on a single
    column or label.
na_position : {'first', 'last'}, default 'last'
    Puts NaNs at the beginning if `first`; `last` puts NaNs at the
    end.
ignore_index : bool, default False
    If True, the resulting axis will be labeled 0, 1, ..., n - 1.
key : callable, optional
    Apply the key function to the values
    before sorting. This is similar to the `key` argument in the
    builtin :meth:`sorted` function, with the notable difference that
    this `key` function should be *vectorized*. It should expect a
    ``Series`` and return a Series with the same shape as the input.
    It will be applied to each column in `by` independently. The values in the
    returned Series will be used as the keys for sorting.

```

Returns

DataFrame or None

DataFrame with sorted values or None if ``inplace=True``.

See Also

DataFrame.sort_index : Sort a DataFrame by the index.

Series.sort_values : Similar method for a Series.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "col1": ["A", "A", "B", np.nan, "D", "C"],
...         "col2": [2, 1, 9, 8, 7, 4],
...         "col3": [0, 1, 9, 4, 2, 3],
...         "col4": ["a", "B", "c", "D", "e", "F"],
...     }
... )
>>> df
   col1  col2  col3 col4
0     A     2     0    a
1     A     1     1    B
2     B     9     9    c
3  NaN     8     4    D
4     D     7     2    e
5     C     4     3    F
```

****Sort by a single column****

In this case, we are sorting the rows according to values in ``col1``:

```
>>> df.sort_values(by=["col1"])
   col1  col2  col3 col4
0     A     2     0    a
1     A     1     1    B
2     B     9     9    c
5     C     4     3    F
4     D     7     2    e
3  NaN     8     4    D
```

****Sort by multiple columns****

You can also provide multiple columns to ``by`` argument, as shown below. In this example, the rows are first sorted according to ``col1``, and then the rows that have an identical value in ``col1`` are sorted according to ``col2``.

```
>>> df.sort_values(by=["col1", "col2"])
   col1  col2  col3 col4
1     A     1     1    B
0     A     2     0    a
2     B     9     9    c
5     C     4     3    F
4     D     7     2    e
3  NaN     8     4    D
```

****Sort in a descending order****

The sort order can be reversed using ``ascending=False`` argument, as shown below:

```
>>> df.sort_values(by="col1", ascending=False)
   col1  col2  col3 col4
4     D     7     2    e
5     C     4     3    F
2     B     9     9    c
0     A     2     0    a
1     A     1     1    B
3  NaN     8     4    D
```

****Placing any ``NA`` **first****

Note that in the above example, the rows that contain an ``NA`` value in their ``col1`` are placed at the end of the dataframe. This behavior can be modified via ``na\_position`` argument, as shown below:

```
>>> df.sort_values(by="col1", ascending=False, na_position="first")
   col1 col2 col3 col4
3  NaN    8    4    D
4    D    7    2    e
5    C    4    3    F
2    B    9    9    c
0    A    2    0    a
1    A    1    1    B
```

****Customized sort order****

The `key` argument allows for a further customization of sorting behaviour. For example, you may want to ignore the letter's case <https://en.wikipedia.org/wiki/Letter_case> when sorting strings:

```
>>> df.sort_values(by="col4", key=lambda col: col.str.lower())
   col1 col2 col3 col4
0    A    2    0    a
1    A    1    1    B
2    B    9    9    c
3  NaN    8    4    D
4    D    7    2    e
5    C    4    3    F
```

Another typical example is natural sorting <https://en.wikipedia.org/wiki/Natural_sort_order>. This can be done using `natsort` package <<https://github.com/SethMMorton/natsort>>, which provides a function to generate a key to sort data in their natural order:

```
>>> df = pd.DataFrame(
...     {
...         "hours": ["0hr", "128hr", "0hr", "64hr", "64hr", "128hr"],
...         "mins": [
...             "10mins",
...             "40mins",
...             "40mins",
...             "40mins",
...             "10mins",
...             "10mins",
...         ],
...         "value": [10, 20, 30, 40, 50, 60],
...     }
... )
>>> df
   hours  mins  value
0   0hr  10mins    10
1 128hr  40mins    20
2   0hr  40mins    30
3   64hr  40mins    40
4   64hr  10mins    50
5 128hr  10mins    60
>>> from natsort import natsort_keygen
>>> df.sort_values(
...     by=["hours", "mins"],
...     key=natsort_keygen(),
... )
   hours  mins  value
0   0hr  10mins    10
2   0hr  40mins    30
4   64hr  10mins    50
3   64hr  40mins    40
5 128hr  10mins    60
1 128hr  40mins    20
```

```
stack(
    self,
    level: IndexLabel = -1,
    dropna: bool | lib.NoDefault = <no_default>,
    sort: bool | lib.NoDefault = <no_default>,
    future_stack: bool = True
) from pandas.core.frame.DataFrame
Stack the prescribed level(s) from columns to index.
```

Return a reshaped DataFrame or Series having a multi-level

index with one or more new inner-most levels compared to the current DataFrame. The new inner-most levels are created by pivoting the columns of the current dataframe:

- if the columns have a single level, the output is a Series;
- if the columns have multiple levels, the new index level(s) is (are) taken from the prescribed level(s) and the output is a DataFrame.

Parameters

level : int, str, list, default -1
Level(s) to stack from the column axis onto the index axis, defined as one index or label, or a list of indices or labels.
dropna : bool, default True
Whether to drop rows in the resulting Frame/Series with missing values. Stacking a column level onto the index axis can create combinations of index and column values that are missing from the original dataframe. See Examples section.
sort : bool, default True
Whether to sort the levels of the resulting MultiIndex.
future_stack : bool, default True
Whether to use the new implementation that will replace the current implementation in pandas 3.0. When True, dropna and sort have no impact on the result and must remain unspecified. See :ref:`pandas 2.1.0 Release notes <whatsnew\_210.enhancements.new\_stack>` for more details.

Returns

DataFrame or Series
Stacked dataframe or series.

See Also

DataFrame.unstack : Unstack prescribed level(s) from index axis onto column axis.
DataFrame.pivot : Reshape dataframe from long format to wide format.
DataFrame.pivot_table : Create a spreadsheet-style pivot table as a DataFrame.

Notes

The function is named by analogy with a collection of books being reorganized from being side by side on a horizontal position (the columns of the dataframe) to being stacked vertically on top of each other (in the index of the dataframe).

Reference :ref:`the user guide <reshaping.stacking>` for more examples.

Examples

Single level columns

```
>>> df_single_level_cols = pd.DataFrame(  
...     [[0, 1], [2, 3]], index=["cat", "dog"], columns=["weight", "height"]  
... )
```

Stacking a dataframe with a single level column axis returns a Series:

```
>>> df_single_level_cols  
      weight height  
cat        0      1  
dog         2      3  
>>> df_single_level_cols.stack()  
cat weight    0  
   height    1  
dog weight    2  
   height    3  
dtype: int64
```

Multi level columns: simple case

```
>>> multicol1 = pd.MultiIndex.from_tuples(  
...     [("weight", "kg"), ("weight", "pounds")]
```

```
... )
>>> df_multi_level_cols1 = pd.DataFrame(
...     [[1, 2], [2, 4]], index=["cat", "dog"], columns=multicol1
... )
```

Stacking a dataframe with a multi-level column axis:

```
>>> df_multi_level_cols1
      weight
      kg    pounds
cat      1      2
dog      2      4
>>> df_multi_level_cols1.stack()
      weight
cat kg      1
   pounds  2
dog kg      2
   pounds  4
```

****Missing values****

```
>>> multicol2 = pd.MultiIndex.from_tuples([("weight", "kg"), ("height", "m")])
>>> df_multi_level_cols2 = pd.DataFrame(
...     [[1.0, 2.0], [3.0, 4.0]], index=["cat", "dog"], columns=multicol2
... )
```

It is common to have missing values when stacking a dataframe with multi-level columns, as the stacked dataframe typically has more values than the original dataframe. Missing values are filled with NaNs:

```
>>> df_multi_level_cols2
      weight height
      kg      m
cat   1.0    2.0
dog   3.0    4.0
>>> df_multi_level_cols2.stack()
      weight height
cat kg   1.0    NaN
   m     NaN    2.0
dog kg   3.0    NaN
   m     NaN    4.0
```

****Prescribing the level(s) to be stacked****

The first parameter controls which level or levels are stacked:

```
>>> df_multi_level_cols2.stack(0)
      kg      m
cat weight 1.0 NaN
   height NaN 2.0
dog weight 3.0 NaN
   height NaN 4.0
>>> df_multi_level_cols2.stack([0, 1])
cat weight kg   1.0
   height m    2.0
dog weight kg   3.0
   height m    4.0
dtype: float64
```

```
std(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return sample standard deviation over requested axis.
```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

```
-----
axis : {index (0), columns (1)}
    For `Series` this parameter is unused and defaults to 0.
```


.. warning::

The behavior of `DataFrame.std` with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass `axis=0` (or do not pass `axis`).

`skipna` : bool, default True
Exclude NA/null values. If an entire row/column is NA, the result will be NA.
`ddof` : int, default 1
Delta Degrees of Freedom. The divisor used in calculations is $N - \text{ddof}$, where N represents the number of elements.
`numeric_only` : bool, default False
Include only float, int, boolean columns. Not implemented for Series.
`**kwargs` : dict
Additional keyword arguments to be passed to the function.

Returns

Series or scalar
Standard deviation over requested axis.

See Also

`Series.std` : Return standard deviation over Series values.
`DataFrame.mean` : Return the mean of the values over the requested axis.
`DataFrame.median` : Return the median of the values over the requested axis.
`DataFrame.mode` : Get the mode(s) of each element along the requested axis.
`DataFrame.sum` : Return the sum of the values over the requested axis.

Notes

To have the same behaviour as ``numpy.std``, use ``ddof=0`` (instead of the default ``ddof=1``)

Examples

```
>>> df = pd.DataFrame(
...     {
...         "person_id": [0, 1, 2, 3],
...         "age": [21, 25, 62, 43],
...         "height": [1.61, 1.87, 1.49, 2.01],
...     }
... ).set_index("person_id")
>>> df
      age  height
person_id
0       21    1.61
1       25    1.87
2       62    1.49
3       43    2.01
```

The standard deviation of the columns can be found as follows:

```
>>> df.std()
age      18.786076
height   0.237417
dtype: float64
```

Alternatively, ``ddof=0`` can be set to normalize by N instead of $N-1$:

```
>>> df.std(ddof=0)
age      16.269219
height   0.205609
dtype: float64
```

`sub(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from pandas
Get Subtraction of dataframe and other, element-wise (binary operator ``sub``).

Equivalent to ``dataframe - other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``rsub``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

```

other : scalar, sequence, Series, dict or DataFrame
    Any single or multiple element data structure, or list-like object.
axis : {0 or 'index', 1 or 'columns'}
    Whether to compare by the index (0 or 'index') or columns.
    (1 or 'columns'). For Series input, axis to match Series index on.
level : int or label
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : float or None, default None
    Fill existing missing (NaN) values, and any new element needed for
    successful DataFrame alignment, with this value before computation.
    If data in both corresponding DataFrame locations is missing
    the result will be missing.

```

Returns

```

DataFrame
    Result of the arithmetic operation.

```

See Also

```

DataFrame.add : Add DataFrames.
DataFrame.sub : Subtract DataFrames.
DataFrame.mul : Multiply DataFrames.
DataFrame.div : Divide DataFrames (float division).
DataFrame.truediv : Divide DataFrames (float division).
DataFrame.floordiv : Divide DataFrames (integer division).
DataFrame.mod : Calculate modulo (remainder after division).
DataFrame.pow : Calculate exponential power.

```

Notes

```

Mismatched indices will be unioned together.

```

Examples

```

>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df

```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```

>>> df + 1

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```

>>> df.add(1)

```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```

>>> df.div(10)

```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```

>>> df.rdiv(10)

```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
      angles  degrees
circle      -1     358
triangle     2     178
rectangle    3     358

>>> df.sub([1, 2], axis='columns')
      angles  degrees
circle      -1     358
triangle     2     178
rectangle    3     358

>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
      angles  degrees
circle      -1     359
triangle     2     179
rectangle    3     359
```

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
      angles  degrees
circle         0     720
triangle        0     360
rectangle        0     720

>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle         0         0
triangle         6     360
rectangle        12    1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle         0
triangle         3
rectangle         4

>>> df * other
      angles  degrees
circle         0      NaN
triangle         9      NaN
rectangle        16      NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle         0     0.0
triangle         9     0.0
rectangle        16     0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                               index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                       ['circle', 'triangle', 'rectangle',
...                                        'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle         0     360
  triangle         3     180
  rectangle         4     360
B square         4     360
  pentagon         5     540
  hexagon         6     720

>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle      NaN     1.0
  triangle     1.0     1.0
  rectangle     1.0     1.0
B square       0.0     0.0
```

```

    pentagon      0.0      0.0
    hexagon       0.0      0.0

```

```

>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                          'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)

```

```

   A  B
0  4 36
1  9 49
2 16 64
3 25 81

```

```

subtract = sub(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame

```

```

sum(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
) -> Series from pandas.core.frame.DataFrame

```

Return the sum of the values over the requested axis.

This is equivalent to the method ``numpy.sum``.

Parameters

axis : {index (0), columns (1)}

Axis for the function to be applied on.

For ``Series`` this parameter is unused and defaults to 0.

.. warning::

The behavior of DataFrame.sum with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis).

.. versionadded:: 2.0.0

skipna : bool, default True

Exclude NA/null values when computing the result.

numeric_only : bool, default False

Include only float, int, boolean columns. Not implemented for Series.

min_count : int, default 0

The required number of valid values to perform the operation. If fewer than ``min\_count`` non-NA values are present the result will be NA.

**kwargs

Additional keyword arguments to be passed to the function.

Returns

Series or scalar

Sum over requested axis.

See Also

Series.sum : Return the sum over Series values.

DataFrame.mean : Return the mean of the values over the requested axis.

DataFrame.median : Return the median of the values over the requested axis.

DataFrame.mode : Get the mode(s) of each element along the requested axis.

DataFrame.std : Return the standard deviation of the values over the requested axis.

Examples

```

>>> idx = pd.MultiIndex.from_arrays(
...     [
...         ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"],
...         names=["blooded", "animal"],
...     ]
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded  animal
warm     dog      4
         falcon   2
cold     fish     0

```

```
spider      8
Name: legs, dtype: int64
```

```
>>> s.sum()
14
```

By default, the sum of an empty or all-NA Series is ``0``.

```
>>> pd.Series([], dtype="float64").sum() # min_count=0 is the default
0.0
```

This can be controlled with the ``min\_count`` parameter. For example, if you'd like the sum of an empty series to be NaN, pass ``min\_count=1``.

```
>>> pd.Series([], dtype="float64").sum(min_count=1)
nan
```

Thanks to the ``skipna`` parameter, ``min\_count`` handles all-NA and empty series identically.

```
>>> pd.Series([np.nan]).sum()
0.0
```

```
>>> pd.Series([np.nan]).sum(min_count=1)
nan
```

swaplevel(self, i: Axis = -2, j: Axis = -1, axis: Axis = 0) -> DataFrame from pandas.core.frame
Swap levels i and j in a :class: MultiIndex`.

Default is to swap the two innermost levels of the index.

Parameters

i, j : int or str
Levels of the indices to be swapped. Can pass level name as string.
axis : {0 or 'index', 1 or 'columns'}, default 0
The axis to swap levels on. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.

Returns

DataFrame
DataFrame with levels swapped in MultiIndex.

See Also

DataFrame.reorder_levels: Reorder levels of MultiIndex.
DataFrame.sort_index: Sort MultiIndex.

Examples

```
>>> df = pd.DataFrame(
...     {"Grade": ["A", "B", "A", "C"]},
...     index=[
...         ["Final exam", "Final exam", "Coursework", "Coursework"],
...         ["History", "Geography", "History", "Geography"],
...         ["January", "February", "March", "April"],
...     ],
... )
>>> df
```

			Grade
Final exam	History	January	A
	Geography	February	B
Coursework	History	March	A
	Geography	April	C

In the following example, we will swap the levels of the indices. Here, we will swap the levels column-wise, but levels can be swapped row-wise in a similar manner. Note that column-wise is the default behaviour. By not supplying any arguments for i and j, we swap the last and second to last indices.

```
>>> df.swaplevel()

Final exam  January  History  Grade
            February Geography    B
Coursework  March    History    A
```

April	Geography	C
-------	-----------	---

By supplying one argument, we can choose which index to swap the last index with. We can for example swap the first index with the last one as follows.

```
>>> df.swaplevel(0)
```

			Grade
January	History	Final exam	A
February	Geography	Final exam	B
March	History	Coursework	A
April	Geography	Coursework	C

We can also define explicitly which indices we want to swap by supplying values for both i and j. Here, we for example swap the first and second indices.

```
>>> df.swaplevel(0, 1)
```

			Grade
History	Final exam	January	A
Geography	Final exam	February	B
History	Coursework	March	A
Geography	Coursework	April	C

```
to_dict(
    self,
    orient: Literal['dict', 'list', 'series', 'split', 'tight', 'records', 'index'] = 'dict',
    *,
    into: type[MutableMappingT] | MutableMappingT = <class 'dict'>,
    index: bool = True
) -> MutableMappingT | list[MutableMappingT] from pandas.core.frame.DataFrame
Convert the DataFrame to a dictionary.
```

The type of the key-value pairs can be customized with the parameters (see below).

Parameters

orient : str {'dict', 'list', 'series', 'split', 'tight', 'records', 'index'}
Determines the type of the values of the dictionary.

- 'dict' (default) : dict like {column -> {index -> value}}
- 'list' : dict like {column -> [values]}
- 'series' : dict like {column -> Series(values)}
- 'split' : dict like
{'index' -> [index], 'columns' -> [columns], 'data' -> [values]}
- 'tight' : dict like
{'index' -> [index], 'columns' -> [columns], 'data' -> [values],
'index_names' -> [index.names], 'column_names' -> [column.names]}
- 'records' : list like
[{column -> value}, ... , {column -> value}]
- 'index' : dict like {index -> {column -> value}}

into : class, default dict

The collections.abc.MutableMapping subclass used for all Mappings in the return value. Can be the actual class or an empty instance of the mapping type you want. If you want a collections.defaultdict, you must pass it initialized.

index : bool, default True

Whether to include the index item (and index_names item if 'orient' is 'tight') in the returned dictionary. Can only be ``False`` when 'orient' is 'split' or 'tight'. Note that when 'orient' is 'records', this parameter does not take effect (index item always not included).

.. versionadded:: 2.0.0

Returns

dict, list or collections.abc.MutableMapping
Return a collections.abc.MutableMapping object representing the DataFrame. The resulting transformation depends on the 'orient' parameter.

See Also

DataFrame.from_dict: Create a DataFrame from a dictionary.

`DataFrame.to_json`: Convert a `DataFrame` to JSON format.

Examples

```
-----
>>> df = pd.DataFrame(
...     {"col1": [1, 2], "col2": [0.5, 0.75]}, index=["row1", "row2"]
... )
>>> df
   col1  col2
row1    1  0.50
row2    2  0.75
>>> df.to_dict()
{'col1': {'row1': 1, 'row2': 2}, 'col2': {'row1': 0.5, 'row2': 0.75}}
```

You can specify the return orientation.

```
>>> df.to_dict("series")
{'col1': row1    1
          row2    2
Name: col1, dtype: int64,
 'col2': row1    0.50
          row2    0.75
Name: col2, dtype: float64}

>>> df.to_dict("split")
{'index': ['row1', 'row2'], 'columns': ['col1', 'col2'],
 'data': [[1, 0.5], [2, 0.75]]}

>>> df.to_dict("records")
[{'col1': 1, 'col2': 0.5}, {'col1': 2, 'col2': 0.75}]

>>> df.to_dict("index")
{'row1': {'col1': 1, 'col2': 0.5}, 'row2': {'col1': 2, 'col2': 0.75}}

>>> df.to_dict("tight")
{'index': ['row1', 'row2'], 'columns': ['col1', 'col2'],
 'data': [[1, 0.5], [2, 0.75]], 'index_names': [None], 'column_names': [None]}
```

You can also specify the mapping type.

```
>>> from collections import OrderedDict, defaultdict
>>> df.to_dict(into=OrderedDict)
OrderedDict([('col1', OrderedDict([('row1', 1), ('row2', 2)])),
            ('col2', OrderedDict([('row1', 0.5), ('row2', 0.75)])])])
```

If you want a `defaultdict`, you need to initialize it:

```
>>> dd = defaultdict(list)
>>> df.to_dict("records", into=dd)
[defaultdict(<class 'list'>, {'col1': 1, 'col2': 0.5}),
 defaultdict(<class 'list'>, {'col1': 2, 'col2': 0.75})]
```

`to_feather(self, path: FilePath | WriteBuffer[bytes], **kwargs) -> None` from `pandas.core.frame`
Write a `DataFrame` to the binary Feather format.

Parameters

```
-----
path : str, path object, file-like object
      String, path object (implementing os.PathLike[str]), or file-like
      object implementing a binary write() function. If a string or a path,
      it will be used as Root Directory path when writing a partitioned dataset.
**kwargs :
      Additional keywords passed to :func:pyarrow.feather.write_feather.
      This includes the compression, compression_level, chunksize
      and version keywords.
```

See Also

```
-----
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.
DataFrame.to_excel : Write object to an Excel sheet.
DataFrame.to_sql : Write to a sql table.
DataFrame.to_csv : Write a csv file.
DataFrame.to_json : Convert the object to a JSON string.
DataFrame.to_html : Render a DataFrame as an HTML table.
DataFrame.to_string : Convert DataFrame to a string.
```

Notes

 This function writes the dataframe as a `feather file`
<https://arrow.apache.org/docs/python/feather.html>`. Requires a default index. For saving the DataFrame with your custom index use a method that supports custom indices e.g. `to_parquet`.

Examples

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]])
>>> df.to_feather("file.feather") # doctest: +SKIP
```

```
to_html(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    columns: Axes | None = None,
    col_space: ColspaceArgType | None = None,
    header: bool = True,
    index: bool = True,
    na_rep: str = 'NaN',
    formatters: FormattersType | None = None,
    float_format: FloatFormatType | None = None,
    sparsify: bool | None = None,
    index_names: bool = True,
    justify: str | None = None,
    max_rows: int | None = None,
    max_cols: int | None = None,
    show_dimensions: bool | str = False,
    decimal: str = '.',
    bold_rows: bool = True,
    classes: str | list | tuple | None = None,
    escape: bool = True,
    notebook: bool = False,
    border: int | bool | None = None,
    table_id: str | None = None,
    render_links: bool = False,
    encoding: str | None = None
) -> str | None from pandas.core.frame.DataFrame
    Render a DataFrame as an HTML table.
```

Parameters

```
-----
buf : str, Path or StringIO-like, optional, default None
    Buffer to write to. If None, the output is returned as a string.
columns : array-like, optional, default None
    The subset of columns to write. Writes all columns by default.
col_space : str or int, list or dict of int or str, optional
    The minimum width of each column in CSS length units. An int is assumed to
header : bool, optional
    Whether to print column labels, default True.
index : bool, optional, default True
    Whether to print index (row) labels.
na_rep : str, optional, default 'NaN'
    String representation of ``NaN`` to use.
formatters : list, tuple or dict of one-param. functions, optional
    Formatter functions to apply to columns' elements by position or
    name.
    The result of each function must be a unicode string.
    List/tuple must be of length equal to the number of columns.
float_format : one-parameter function, optional, default None
    Formatter function to apply to columns' elements if they are
    floats. This function must return a unicode string and will be
    applied only to the non-``NaN`` elements, with ``NaN`` being
    handled by ``na_rep``.
sparsify : bool, optional, default True
    Set to False for a DataFrame with a hierarchical index to print
    every multiindex key at each row.
index_names : bool, optional, default True
    Prints the names of the indexes.
justify : str, default None
    How to justify the column labels. If None uses the option from
    the print configuration (controlled by set_option), 'right' out
    of the box. Valid values are

    * left
    * right
    * center
```



```

        * justify
        * justify-all
        * start
        * end
        * inherit
        * match-parent
        * initial
        * unset.
    max_rows : int, optional
        Maximum number of rows to display in the console.
    max_cols : int, optional
        Maximum number of columns to display in the console.
    show_dimensions : bool, default False
        Display DataFrame dimensions (number of rows by number of columns).
    decimal : str, default '.'
        Character recognized as decimal separator, e.g. ',' in Europe.

    bold_rows : bool, default True
        Make the row labels bold in the output.
    classes : str or list or tuple, default None
        CSS class(es) to apply to the resulting html table.
    escape : bool, default True
        Convert the characters <, >, and & to HTML-safe sequences.
    notebook : {True, False}, default False
        Whether the generated HTML is for IPython Notebook.
    border : int or bool
        When an integer value is provided, it sets the border attribute in
        the opening tag, specifying the thickness of the border.
        If ``False`` or ``0`` is passed, the border attribute will not
        be present in the ``<table>`` tag.
        The default value for this parameter is governed by
        ``pd.options.display.html.border``.
    table_id : str, optional
        A css id is included in the opening ``<table>`` tag if specified.
    render_links : bool, default False
        Convert URLs to HTML links.
    encoding : str, default "utf-8"
        Set character encoding.

    Returns
    -----
    str or None
        If buf is None, returns the result as a string. Otherwise returns
        None.

```

See Also

```

to_string : Convert DataFrame to a string.

```

Examples

```

>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> html_string = df.to_html()
>>> print(html_string)
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>col1</th>
      <th>col2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>1</td>
      <td>4</td>
    </tr>
    <tr>
      <th>1</th>
      <td>2</td>
      <td>3</td>
    </tr>
  </tbody>
</table>

```

HTML output

```
+-----+-----+-----+
|      |col1 |col2 |
+=====+=====+
|0      |1      |4      |
+-----+-----+-----+
|1      |2      |3      |
+-----+-----+-----+
```

```
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> html_string = df.to_html(index=False)
>>> print(html_string)
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th>col1</th>
      <th>col2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>4</td>
    </tr>
    <tr>
      <td>2</td>
      <td>3</td>
    </tr>
  </tbody>
</table>
```

HTML output

```
+-----+-----+
|col1 |col2 |
+=====+=====+
|1      |4      |
+-----+-----+
|2      |3      |
+-----+-----+
```

```
to_iceberg(
    self,
    table_identifier: str,
    catalog_name: str | None = None,
    *,
    catalog_properties: dict[str, Any] | None = None,
    location: str | None = None,
    append: bool = False,
    snapshot_properties: dict[str, str] | None = None
) -> None from pandas.core.frame.DataFrame
Write a DataFrame to an Apache Iceberg table.
```

.. versionadded:: 3.0.0

.. warning::

to_iceberg is experimental and may change without warning.

Parameters

```
-----
table_identifier : str
    Table identifier.
catalog_name : str, optional
    The name of the catalog.
catalog_properties : dict of {str: str}, optional
    The properties that are used next to the catalog configuration.
location : str, optional
    Location for the table.
append : bool, default False
    If ``True``, append data to the table, instead of replacing the content.
snapshot_properties : dict of {str: str}, optional
    Custom properties to be added to the snapshot summary
```

See Also

```
-----
read_iceberg : Read an Apache Iceberg table.
```

`DataFrame.to_parquet` : Write a DataFrame in Parquet format.

Examples

```
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> df.to_iceberg("my_table", catalog_name="my_catalog") # doctest: +SKIP
```

```
to_markdown(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    mode: str = 'wt',
    index: bool = True,
    storage_options: StorageOptions | None = None,
    **kwargs
) -> str | None from pandas.core.frame.DataFrame
Print DataFrame in Markdown-friendly format.
```

Parameters

`buf` : str, Path or StringIO-like, optional, default None
Buffer to write to. If None, the output is returned as a string.

`mode` : str, optional
Mode in which file is opened, "wt" by default.

`index` : bool, optional, default True
Add index (row) labels.

`storage_options` : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`**kwargs`
These parameters will be passed to `tabulate` <https://pypi.org/project/tabulate>.

Returns

str
DataFrame in Markdown-friendly format.

See Also

`DataFrame.to_html` : Render DataFrame to HTML-formatted table.
`DataFrame.to_latex` : Render DataFrame to LaTeX-formatted table.

Notes

Requires the `tabulate` <https://pypi.org/project/tabulate> package.

Examples

```
>>> df = pd.DataFrame(
...     data={"animal_1": ["elk", "pig"], "animal_2": ["dog", "quetzal"]}
... )
>>> print(df.to_markdown())
|   | animal_1 | animal_2 |
|---|:-----|:-----|
| 0 | elk      | dog      |
| 1 | pig      | quetzal  |
```

Output markdown with a tabulate option.

```
>>> print(df.to_markdown(tablefmt="grid"))
+-----+
|   | animal_1 | animal_2 |
+=====+
| 0 | elk      | dog      |
+-----+
| 1 | pig      | quetzal  |
+-----+
```

`to_numpy`(

```

self,
dtype: npt.DTypeLike | None = None,
copy: bool = False,
na_value: object = <no_default>
) -> np.ndarray from pandas.core.frame.DataFrame
Convert the DataFrame to a NumPy array.

By default, the dtype of the returned array will be the common NumPy
dtype of all types in the DataFrame. For example, if the dtypes are
`float16` and `float32`, the results dtype will be `float32`.
This may require copying data and coercing values, which may be
expensive.

Parameters
-----
dtype : str or numpy.dtype, optional
    The dtype to pass to :meth:`numpy.asarray`.
copy : bool, default False
    Whether to ensure that the returned value is not a view on
    another array. Note that ``copy=False`` does not *ensure* that
    ``to_numpy()`` is no-copy. Rather, ``copy=True`` ensure that
    a copy is made, even if not strictly necessary.
na_value : Any, optional
    The value to use for missing values. The default value depends
    on `dtype` and the dtypes of the DataFrame columns.

Returns
-----
numpy.ndarray
    The NumPy array representing the values in the DataFrame.

See Also
-----
Series.to_numpy : Similar method for Series.

Examples
-----
>>> pd.DataFrame({"A": [1, 2], "B": [3, 4]}).to_numpy()
array([[1, 3],
       [2, 4]])

With heterogeneous data, the lowest common type will have to
be used.

>>> df = pd.DataFrame({"A": [1, 2], "B": [3.0, 4.5]})
>>> df.to_numpy()
array([[1. , 3. ],
       [2. , 4.5]])

For a mix of numeric and non-numeric types, the output array will
have object dtype.

>>> df["C"] = pd.date_range("2000", periods=2)
>>> df.to_numpy()
array([[1, 3.0, Timestamp('2000-01-01 00:00:00')],
       [2, 4.5, Timestamp('2000-01-02 00:00:00')]], dtype=object)

to_orc(
self,
path: FilePath | WriteBuffer[bytes] | None = None,
*,
engine: Literal['pyarrow'] = 'pyarrow',
index: bool | None = None,
engine_kwargs: dict[str, Any] | None = None
) -> bytes | None from pandas.core.frame.DataFrame
Write a DataFrame to the Optimized Row Columnar (ORC) format.

Parameters
-----
path : str, file-like object or None, default None
    If a string, it will be used as Root Directory path
    when writing a partitioned dataset. By file-like object,
    we refer to objects with a write() method, such as a file handle
    (e.g. via builtin open function). If path is None,
    a bytes object is returned.
engine : {'pyarrow'}, default 'pyarrow'
    ORC library to use.

```

index : bool, optional
 If ``True``, include the dataframe's index(es) in the file output.
 If ``False``, they will not be written to the file.
 If ``None``, similar to ``infer`` the dataframe's index(es) will be saved. However, instead of being saved as values, the RangeIndex will be stored as a range in the metadata so it doesn't require much space and is faster. Other indexes will be included as columns in the file output.
 engine_kwarg : dict[str, Any] or None, default None
 Additional keyword arguments passed to :func:`pyarrow.orc.write\_table`.

Returns

bytes if no ``path`` argument is provided else None
 Bytes object with DataFrame data if ``path`` is not specified else None.

Raises

NotImplementedError
 Dtype of one or more columns is category, unsigned integers, interval, period or sparse.
 ValueError
 engine is not pyarrow.

See Also

read_orc : Read a ORC file.
 DataFrame.to_parquet : Write a parquet file.
 DataFrame.to_csv : Write a csv file.
 DataFrame.to_sql : Write to a sql table.
 DataFrame.to_hdf : Write to hdf.

Notes

- * Find more information on ORC
 `here <[https://en.wikipedia.org/wiki/Apache\\_ORC](https://en.wikipedia.org/wiki/Apache_ORC)>`__.
- * Before using this function you should read the :ref:`user guide about ORC <io.orc>` and :ref:`install optional dependencies <install.warn\_orc>`.
- * This function requires `pyarrow <<https://arrow.apache.org/docs/python/>>` library.
- * For supported dtypes please refer to `supported ORC features in Arrow <<https://arrow.apache.org/docs/cpp/orc.html#data-types>>`__.
- * Currently timezones in datetime columns are not preserved when a dataframe is converted into ORC files.

Examples

```
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [4, 3]})
>>> df.to_orc("df.orc") # doctest: +SKIP
>>> pd.read_orc("df.orc") # doctest: +SKIP
   col1  col2
0      1     4
1      2     3
```

If you want to get a buffer to the orc content you can write it to io.BytesIO

```
>>> import io
>>> b = io.BytesIO(df.to_orc()) # doctest: +SKIP
>>> b.seek(0) # doctest: +SKIP
0
>>> content = b.read() # doctest: +SKIP
```

```
to_parquet(
    self,
    path: FilePath | WriteBuffer[bytes] | None = None,
    *,
    engine: Literal['auto', 'pyarrow', 'fastparquet'] = 'auto',
    compression: ParquetCompressionOptions = 'snappy',
    index: bool | None = None,
    partition_cols: list[str] | None = None,
    storage_options: StorageOptions | None = None,
    filesystem: Any = None,
    **kwargs
) -> bytes | None from pandas.core.frame.DataFrame
Write a DataFrame to the binary parquet format.
```

This function writes the dataframe as a `parquet` file

<<https://parquet.apache.org/>>`. You can choose different parquet backends, and have the option of compression. See :ref:`the user guide <io.parquet>` for more details.

Parameters

path : str, path object, file-like object, or None, default None
String, path object (implementing ``os.PathLike[str]``), or file-like object implementing a binary ``write()`` function. If None, the result is returned as bytes. If a string or path, it will be used as Root Directory path when writing a partitioned dataset.
engine : {'auto', 'pyarrow', 'fastparquet'}, default 'auto'
Parquet library to use. If 'auto', then the option ``io.parquet.engine`` is used. The default ``io.parquet.engine`` behavior is to try 'pyarrow', falling back to 'fastparquet' if 'pyarrow' is unavailable.
compression : str or None, default 'snappy'
Name of the compression to use. Use ``None`` for no compression. Supported options: 'snappy', 'gzip', 'brotli', 'lz4', 'zstd'.
index : bool, default None
If ``True``, include the dataframe's index(es) in the file output. If ``False``, they will not be written to the file. If ``None``, similar to ``True`` the dataframe's index(es) will be saved. However, instead of being saved as values, the RangeIndex will be stored as a range in the metadata so it doesn't require much space and is faster. Other indexes will be included as columns in the file output.
partition_cols : list, optional, default None
Column names by which to partition the dataset. Columns are partitioned in the order they are given. Must be None if path is not a string.
storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files.
filesystem : fsspec or pyarrow filesystem, default None
Filesystem object to use when reading the parquet file. Only implemented for ``engine="pyarrow"``.

.. versionadded:: 2.1.0

**kwargs

Additional arguments passed to the parquet library. See :ref:`pandas io <io.parquet>` for more details.

Returns

bytes if no path argument is provided else None
Returns the DataFrame converted to the binary parquet format as bytes if no path argument. Returns None and writes the DataFrame to the specified location in the Parquet format if the path argument is provided.

See Also

read_parquet : Read a parquet file.
DataFrame.to_orc : Write an orc file.
DataFrame.to_csv : Write a csv file.
DataFrame.to_sql : Write to a sql table.
DataFrame.to_hdf : Write to hdf.

Notes

-
- * This function requires either the `fastparquet` <https://pypi.org/project/fastparquet/> or `pyarrow` <https://arrow.apache.org/docs/python/> library.
 - * When saving a DataFrame with categorical columns to parquet, the file size may increase due to the inclusion of all possible categories, not just those present in the data. This behavior is expected and consistent with pandas' handling of categorical data. To manage file size and ensure a more predictable roundtrip process, consider using :meth:`Categorical.remove\_unused\_categories` on the DataFrame before saving.

Examples

```
-----
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [3, 4]})
>>> df.to_parquet("df.parquet.gzip", compression="gzip") # doctest: +SKIP
>>> pd.read_parquet("df.parquet.gzip") # doctest: +SKIP
   col1  col2
0      1     3
1      2     4
```

If you want to get a buffer to the parquet content you can use a `io.BytesIO` object, as long as you don't use `partition_cols`, which creates multiple files.

```
>>> import io
>>> f = io.BytesIO()
>>> df.to_parquet(f)
>>> f.seek(0)
0
>>> content = f.read()
```

```
to_period(
    self,
    freq: Frequency | None = None,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
    Convert DataFrame from DatetimeIndex to PeriodIndex.
```

Convert DataFrame from DatetimeIndex to PeriodIndex with desired frequency (inferred from index if not passed). Either index or columns can be converted, depending on `axis` argument.

Parameters

```
-----
freq : str, default
    Frequency of the PeriodIndex.
axis : {0 or 'index', 1 or 'columns'}, default 0
    The axis to convert (the index by default).
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html> for more details.

Returns

```
-----
DataFrame
    The DataFrame with the converted PeriodIndex.
```

See Also

```
-----
Series.to_period: Equivalent method for Series.
Series.dt.to_period: Convert DateTime column values.
```

Examples

```
-----
>>> idx = pd.to_datetime(
...     [
...         "2001-03-31 00:00:00",
...         "2002-05-31 00:00:00",
...         "2003-08-31 00:00:00",
...     ]
... )

>>> idx
DatetimeIndex(['2001-03-31', '2002-05-31', '2003-08-31'],
              dtype='datetime64[us]', freq=None)

>>> idx.to_period("M")
PeriodIndex(['2001-03', '2002-05', '2003-08'], dtype='period[M]')
```

For the yearly frequency

```
>>> idx.to_period("Y")
PeriodIndex(['2001', '2002', '2003'], dtype='period[Y-DEC]')
```

`to_records(self, index: bool = True, column_dtypes=None, index_dtypes=None) -> np.rec.recarray`
Convert DataFrame to a NumPy record array.

Index will be included as the first field of the record array if requested.

Parameters

`index` : bool, default True
Include index in resulting record array, stored in 'index' field or using the index label, if set.
`column_dtypes` : str, type, dict, default None
If a string or type, the data type to store all columns. If a dictionary, a mapping of column names and indices (zero-indexed) to specific data types.
`index_dtypes` : str, type, dict, default None
If a string or type, the data type to store all index levels. If a dictionary, a mapping of index level names and indices (zero-indexed) to specific data types.

This mapping is applied only if `index=True`.

Returns

`numpy.rec.recarray`
NumPy ndarray with the DataFrame labels as fields and each row of the DataFrame as entries.

See Also

`DataFrame.from_records`: Convert structured or record ndarray to DataFrame.
`numpy.rec.recarray`: An ndarray that allows field access using attributes, analogous to typed columns in a spreadsheet.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [0.5, 0.75]}, index=["a", "b"])
>>> df
   A    B
a  1  0.5
b  2  0.75
>>> df.to_records()
rec.array([( 'a', 1, 0.5 ), ( 'b', 2, 0.75)],
          dtype=[('index', 'O'), ('A', '<i8'), ('B', '<f8')])
```

If the DataFrame index has no label then the recarray field name is set to 'index'. If the index has a label then this is used as the field name:

```
>>> df.index = df.index.rename("I")
>>> df.to_records()
rec.array([( 'a', 1, 0.5 ), ( 'b', 2, 0.75)],
          dtype=[('I', 'O'), ('A', '<i8'), ('B', '<f8')])
```

The index can be excluded from the record array:

```
>>> df.to_records(index=False)
rec.array([(1, 0.5 ), (2, 0.75)],
          dtype=[('A', '<i8'), ('B', '<f8')])
```

Data types can be specified for the columns:

```
>>> df.to_records(column_dtypes={"A": "int32"})
rec.array([( 'a', 1, 0.5 ), ( 'b', 2, 0.75)],
          dtype=[('I', 'O'), ('A', '<i4'), ('B', '<f8')])
```

As well as for the index:

```
>>> df.to_records(index_dtypes="<S2")
```



```

rec.array([(b'a', 1, 0.5 ), (b'b', 2, 0.75)],
          dtype=[('I', 'S2'), ('A', '<i8'), ('B', '<f8')])

>>> index_dtypes = f"<S{df.index.str.len().max()}"
>>> df.to_records(index_dtypes=index_dtypes)
rec.array([(b'a', 1, 0.5 ), (b'b', 2, 0.75)],
          dtype=[('I', 'S1'), ('A', '<i8'), ('B', '<f8')])

```

to_stata(
self,
path: FilePath | WriteBuffer[bytes],
*,
convert_dates: dict[Hashable, str] | None = None,
write_index: bool = True,
byteorder: ToStataByteorder | None = None,
time_stamp: datetime.datetime | None = None,
data_label: str | None = None,
variable_labels: dict[Hashable, str] | None = None,
version: int | None = 114,
convert_strl: Sequence[Hashable] | None = None,
compression: CompressionOptions = 'infer',
storage_options: StorageOptions | None = None,
value_labels: dict[Hashable, dict[float, str]] | None = None
) -> None from pandas.core.frame.DataFrame
Export DataFrame object to Stata dta format.

Writes the DataFrame to a Stata dataset file.
"dta" files contain a Stata dataset.

Parameters

path : str, path object, or buffer
String, path object (implementing ``os.PathLike[str]``), or file-like
object implementing a binary ``write()`` function.

convert_dates : dict
Dictionary mapping columns containing datetime types to stata
internal format to use when writing the dates. Options are 'tc',
'td', 'tm', 'tw', 'th', 'tq', 'ty'. Column can be either an integer
or a name. Datetime columns that do not have a conversion type
specified will be converted to 'tc'. Raises NotImplementedError if
a datetime column has timezone information.

write_index : bool
Write the index to Stata dataset.

byteorder : str
Can be ">", "<", "<", "little", or "big". default is `sys.byteorder`.

time_stamp : datetime
A datetime to use as file creation date. Default is the current
time.

data_label : str, optional
A label for the data set. Must be 80 characters or smaller.

variable_labels : dict
Dictionary containing columns as keys and variable labels as
values. Each label must be 80 characters or smaller.

version : {{114, 117, 118, 119, None}}, default 114
Version to use in the output dta file. Set to None to let pandas
decide between 118 or 119 formats depending on the number of
columns in the frame. Version 114 can be read by Stata 10 and
later. Version 117 can be read by Stata 13 or later. Version 118
is supported in Stata 14 and later. Version 119 is supported in
Stata 15 and later. Version 114 limits string variables to 244
characters or fewer while versions 117 and later allow strings
with lengths up to 2,000,000 characters. Versions 118 and 119
support Unicode characters, and version 119 supports more than
32,767 variables.

Version 119 should usually only be used when the number of
variables exceeds the capacity of dta format 118. Exporting
smaller datasets in format 119 may have unintended consequences,
and, as of November 2020, Stata SE cannot read version 119 files.

convert_strl : list, optional
List of column names to convert to string columns to Stata StrL
format. Only available if version is 117. Storing strings in the
StrL format can produce smaller dta files if strings have more than
8 characters and values are repeated.

```

compression : str or dict, default 'infer'
    For on-the-fly compression of the output data. If 'infer' and 'path' is
    path-like, then detect compression from the following extensions: '.gz',
    '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
    (otherwise no compression).
    Set to ``None`` for no compression.
    Can also be a dict with key ``'method'`` set to one of
    {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
    other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for faster compression and
    to create a reproducible gzip archive:
    ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`_.

value_labels : dict of dicts
    Dictionary containing columns as keys and dictionaries of column value
    to labels as values. Labels for a single variable must be 32,000
    characters or smaller.

Raises
-----
NotImplementedError
    * If datetimes contain timezone information
    * Column dtype is not representable in Stata
ValueError
    * Columns listed in convert_dates are neither datetime64[ns]
      or datetime.datetime
    * Column listed in convert_dates is not in DataFrame
    * Categorical label contains more than 32,000 characters

See Also
-----
read_stata : Import Stata data files.
io.stata.StataWriter : Low-level writer for Stata data files.
io.stata.StataWriter117 : Low-level writer for version 117 files.

Examples
-----
>>> df = pd.DataFrame(
...     [ ["falcon", 350], ["parrot", 18]], columns=["animal", "parrot"]
... )
>>> df.to_stata("animals.dta") # doctest: +SKIP

to_string(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    columns: Axes | None = None,
    col_space: int | list[int] | dict[Hashable, int] | None = None,
    header: bool | SequenceNotStr[str] = True,
    index: bool = True,
    na_rep: str = 'NaN',
    formatters: fmt.FormatterType | None = None,
    float_format: fmt.FloatFormatType | None = None,
    sparsify: bool | None = None,
    index_names: bool = True,
    justify: str | None = None,
    max_rows: int | None = None,
    max_cols: int | None = None,
    show_dimensions: bool = False,
    decimal: str = '.',
    line_width: int | None = None,
    min_rows: int | None = None,
    max_colwidth: int | None = None,
    encoding: str | None = None

```

```
) -> str | None from pandas.core.frame.DataFrame
Render a DataFrame to a console-friendly tabular output.
```

Parameters

buf : str, Path or StringIO-like, optional, default None
Buffer to write to. If None, the output is returned as a string.

columns : array-like, optional, default None
The subset of columns to write. Writes all columns by default.

col_space : int, list or dict of int, optional
The minimum width of each column. If a list of ints is given every integers

header : bool or list of str, optional
Write out the column names. If a list of columns is given, it is assumed to

index : bool, optional, default True
Whether to print index (row) labels.

na_rep : str, optional, default 'NaN'
String representation of ``NaN`` to use.

formatters : list, tuple or dict of one-param. functions, optional
Formatter functions to apply to columns' elements by position or name.
The result of each function must be a unicode string.
List/tuple must be of length equal to the number of columns.

float_format : one-parameter function, optional, default None
Formatter function to apply to columns' elements if they are floats. This function must return a unicode string and will be applied only to the non-``NaN`` elements, with ``NaN`` being handled by ``na\_rep``.

sparsify : bool, optional, default True
Set to False for a DataFrame with a hierarchical index to print every multiindex key at each row.

index_names : bool, optional, default True
Prints the names of the indexes.

justify : str, default None
How to justify the column labels. If None uses the option from the print configuration (controlled by set_option), 'right' out of the box. Valid values are

- * left
- * right
- * center
- * justify
- * justify-all
- * start
- * end
- * inherit
- * match-parent
- * initial
- * unset.

max_rows : int, optional
Maximum number of rows to display in the console.

max_cols : int, optional
Maximum number of columns to display in the console.

show_dimensions : bool, default False
Display DataFrame dimensions (number of rows by number of columns).

decimal : str, default '.'
Character recognized as decimal separator, e.g. ',' in Europe.

line_width : int, optional
Width to wrap a line in characters.

min_rows : int, optional
The number of rows to display in the console in a truncated repr (when number of rows is above ``max\_rows``).

max_colwidth : int, optional
Max width to truncate each column in characters. By default, no limit.

encoding : str, default "utf-8"
Set character encoding.

Returns

str or None
If buf is None, returns the result as a string. Otherwise returns None.

See Also

to_html : Convert DataFrame to HTML.

Examples

```
-----
>>> d = {"col1": [1, 2, 3], "col2": [4, 5, 6]}
>>> df = pd.DataFrame(d)
>>> print(df.to_string())
   col1  col2
0      1     4
1      2     5
2      3     6
```

```
to_timestamp(
    self,
    freq: Frequency | None = None,
    how: ToTimestampHow = 'start',
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>
) -> DataFrame from pandas.core.frame.DataFrame
    Cast PeriodIndex to DatetimeIndex of timestamps, at *beginning* of period.
```

This can be changed to the *end* of the period, by specifying `how="e"`.

Parameters

```
-----
freq : str, default frequency of PeriodIndex
    Desired frequency.
how : {'s', 'e', 'start', 'end'}
    Convention for converting period to timestamp; start of period
    vs. end.
axis : {0 or 'index', 1 or 'columns'}, default 0
    The axis to convert (the index by default).
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

```
-----
DataFrame with DatetimeIndex
    DataFrame with the PeriodIndex cast to DatetimeIndex.
```

See Also

```
-----
DataFrame.to_period: Inverse method to cast DatetimeIndex to PeriodIndex.
Series.to_timestamp: Equivalent method for Series.
```

Examples

```
-----
>>> idx = pd.PeriodIndex(["2023", "2024"], freq="Y")
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df1 = pd.DataFrame(data=d, index=idx)
>>> df1
   col1  col2
2023    1     3
2024    2     4
```

The resulting timestamps will be at the beginning of the year in this case

```
>>> df1 = df1.to_timestamp()
>>> df1
   col1  col2
2023-01-01    1     3
2024-01-01    2     4
>>> df1.index
DatetimeIndex(['2023-01-01', '2024-01-01'], dtype='datetime64[us]', freq=None)
```

Using `freq` which is the offset that the Timestamps will have

```
>>> df2 = pd.DataFrame(data=d, index=idx)
>>> df2 = df2.to_timestamp(freq="M")
```

```

>>> df2
      col1  col2
2023-01-31    1    3
2024-01-31    2    4
>>> df2.index
DatetimeIndex(['2023-01-31', '2024-01-31'], dtype='datetime64[us]', freq=None)

to_xml(
    self,
    path_or_buffer: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
    *,
    index: bool = True,
    root_name: str | None = 'data',
    row_name: str | None = 'row',
    na_rep: str | None = None,
    attr_cols: list[str] | None = None,
    elem_cols: list[str] | None = None,
    namespaces: dict[str | None, str] | None = None,
    prefix: str | None = None,
    encoding: str = 'utf-8',
    xml_declaration: bool | None = True,
    pretty_print: bool | None = True,
    parser: XMLParsers | None = 'lxml',
    stylesheet: FilePath | ReadBuffer[str] | ReadBuffer[bytes] | None = None,
    compression: CompressionOptions = 'infer',
    storage_options: StorageOptions | None = None
) -> str | None from pandas.core.frame.DataFrame
    Render a DataFrame to an XML document.

Parameters
-----
path_or_buffer : str, path object, file-like object, or None, default None
    String, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a ``write()`` function. If None, the result is returned
    as a string.
index : bool, default True
    Whether to include index in XML document.
root_name : str, default 'data'
    The name of root element in XML document.
row_name : str, default 'row'
    The name of row element in XML document.
na_rep : str, optional
    Missing data representation.
attr_cols : list-like, optional
    List of columns to write as attributes in row element.
    Hierarchical columns will be flattened with underscore
    delimiting the different levels.
elem_cols : list-like, optional
    List of columns to write as children in row element. By default,
    all columns output as children of row element. Hierarchical
    columns will be flattened with underscore delimiting the
    different levels.
namespaces : dict, optional
    All namespaces to be defined in root element. Keys of dict
    should be prefix names and values of dict corresponding URIs.
    Default namespaces should be given empty string key. For
    example, ::

        namespaces = {"": "https://example.com"}

prefix : str, optional
    Namespace prefix to be used for every element and/or attribute
    in document. This should be one of the keys in ``namespaces``
    dict.
encoding : str, default 'utf-8'
    Encoding of the resulting document.
xml_declaration : bool, default True
    Whether to include the XML declaration at start of document.
pretty_print : bool, default True
    Whether output should be pretty printed with indentation and
    line breaks.
parser : {'lxml', 'etree'}, default 'lxml'
    Parser module to use for building of tree. Only 'lxml' and
    'etree' are supported. With 'lxml', the ability to use XSLT
    stylesheet is supported.
stylesheet : str, path object or file-like object, optional
    A URL, file-like object, or a raw string containing an XSLT

```

script used to transform the raw XML output. Script should use layout of elements and attributes from original output. This argument requires ``lxml`` to be installed. Only XSLT 1.0 scripts and not later versions is currently supported.

`compression` : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and 'path_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). Set to ``None`` for no compression. Can also be a dict with key ``'method'`` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to ``zipfile.ZipFile``, ``gzip.GzipFile``, ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or ``tarfile.TarFile``, respectively. As an example, the following could be passed for faster compression and to create a reproducible gzip archive:
 ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

`storage_options` : dict, optional
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

Returns

 None or str
 If ``io`` is None, returns the resulting XML format as a string. Otherwise returns None.

See Also

`to_json` : Convert the pandas object to a JSON string.
`to_html` : Convert DataFrame to a html.

Examples

```
>>> df = pd.DataFrame(
...     [{"square", 360, 4}, {"circle", 360, np.nan}, {"triangle", 180, 3}],
...     columns=["shape", "degrees", "sides"],
... )

>>> df.to_xml() # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
<data>
  <row>
    <index>0</index>
    <shape>square</shape>
    <degrees>360</degrees>
    <sides>4.0</sides>
  </row>
  <row>
    <index>1</index>
    <shape>circle</shape>
    <degrees>360</degrees>
    <sides/>
  </row>
  <row>
    <index>2</index>
    <shape>triangle</shape>
    <degrees>180</degrees>
    <sides>3.0</sides>
  </row>
</data>

>>> df.to_xml(
...     attr_cols=["index", "shape", "degrees", "sides"]
... ) # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
<data>
  <row index="0" shape="square" degrees="360" sides="4.0"/>
```

```

    <row index="1" shape="circle" degrees="360"/>
    <row index="2" shape="triangle" degrees="180" sides="3.0"/>
</data>

>>> df.to_xml(
...     namespaces={"doc": "https://example.com"}, prefix="doc"
... ) # doctest: +SKIP
<?xml version='1.0' encoding='utf-8'?>
<doc:data xmlns:doc="https://example.com">
  <doc:row>
    <doc:index>0</doc:index>
    <doc:shape>square</doc:shape>
    <doc:degrees>360</doc:degrees>
    <doc:sides>4.0</doc:sides>
  </doc:row>
  <doc:row>
    <doc:index>1</doc:index>
    <doc:shape>circle</doc:shape>
    <doc:degrees>360</doc:degrees>
    <doc:sides/>
  </doc:row>
  <doc:row>
    <doc:index>2</doc:index>
    <doc:shape>triangle</doc:shape>
    <doc:degrees>180</doc:degrees>
    <doc:sides>3.0</doc:sides>
  </doc:row>
</doc:data>

```

transform(self, func: AggFuncType, axis: Axis = 0, *args, **kwargs) -> DataFrame from pandas
 Call ``func`` on self producing a DataFrame with the same axis shape as self.

Parameters

func : function, str, list-like or dict-like
 Function to use for transforming the data. If a function, must either work when passed a DataFrame or when passed to DataFrame.apply. If func is both list-like and dict-like, dict-like behavior takes precedence.

Accepted combinations are:

- function
- string function name
- list-like of functions and/or function names, e.g. ``[np.exp, 'sqrt']``
- dict-like of axis labels -> functions, function names or list-like of such.

axis : {0 or 'index', 1 or 'columns'}, default 0
 If 0 or 'index': apply function to each column.
 If 1 or 'columns': apply function to each row.

***args**
 Positional arguments to pass to `func`.

****kwargs**
 Keyword arguments to pass to `func`.

Returns

DataFrame
 A DataFrame that must have the same length as self.

Raises

ValueError : If the returned DataFrame has a different length than self.

See Also

DataFrame.agg : Only perform aggregating type operations.
DataFrame.apply : Invoke function on a DataFrame.

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

Examples

```
>>> df = pd.DataFrame({"A": range(3), "B": range(1, 4)})
```

```
>>> df
   A  B
0  0  1
1  1  2
2  2  3
>>> df.transform(lambda x: x + 1)
   A  B
0  1  2
1  2  3
2  3  4
```

Even though the resulting DataFrame must have the same length as the input DataFrame, it is possible to provide several input functions:

```
>>> s = pd.Series(range(3))
>>> s
0    0
1    1
2    2
dtype: int64
>>> s.transform([np.sqrt, np.exp])
      sqrt      exp
0  0.000000  1.000000
1  1.000000  2.718282
2  1.414214  7.389056
```

You can call transform on a GroupBy object:

```
>>> df = pd.DataFrame(
...     {
...         "Date": [
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...         ],
...         "Data": [5, 8, 6, 1, 50, 100, 60, 120],
...     }
... )
>>> df
   Date  Data
0 2015-05-08    5
1 2015-05-07    8
2 2015-05-06    6
3 2015-05-05    1
4 2015-05-08   50
5 2015-05-07  100
6 2015-05-06   60
7 2015-05-05  120
>>> df.groupby("Date")["Data"].transform("sum")
0    55
1   108
2    66
3   121
4    55
5   108
6    66
7   121
Name: Data, dtype: int64

>>> df = pd.DataFrame(
...     {
...         "c": [1, 1, 1, 2, 2, 2, 2],
...         "type": ["m", "n", "o", "m", "m", "n", "n"],
...     }
... )
>>> df
   c type
0  1   m
1  1   n
2  1   o
3  2   m
4  2   m
```



```

5 2 n
6 2 n
>>> df["size"] = df.groupby("c")["type"].transform(len)
>>> df
   c type size
0  1  m     3
1  1  n     3
2  1  o     3
3  2  m     4
4  2  m     4
5  2  n     4
6  2  n     4

```

`transpose(self, *args, copy: bool | lib.NoDefault = <no_default>) -> DataFrame from pandas.c`
 Transpose index and columns.

Reflect the DataFrame over its main diagonal by writing rows as columns and vice-versa. The property `:attr:'.T'` is an accessor to the method `:meth:'.transpose'`.

Parameters

`*args` : tuple, optional
 Accepted for compatibility with NumPy.
`copy` : bool, default False
 This keyword is now ignored; changing its value will have no impact on the method.

Note that a copy is always required for mixed dtype DataFrames, or for DataFrames with any extensiontypes.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write` (https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) for more details.

Returns

DataFrame
 The transposed DataFrame.

See Also

`numpy.transpose` : Permute the dimensions of a given array.

Notes

Transposing a DataFrame with mixed dtypes will result in a homogeneous DataFrame with the `'object'` dtype. In such a case, a copy of the data is always made.

Examples

****Square DataFrame with homogeneous dtype****

```

>>> d1 = {"col1": [1, 2], "col2": [3, 4]}
>>> df1 = pd.DataFrame(data=d1)
>>> df1
   col1  col2
0     1     3
1     2     4

>>> df1_transposed = df1.T # or df1.transpose()
>>> df1_transposed
   0  1
col1 1 2
col2 3 4

```

When the dtype is homogeneous in the original DataFrame, we get a transposed DataFrame with the same dtype:

```

>>> df1.dtypes
col1    int64

```

```
col2    int64
dtype: object
>>> df1_transposed.dtypes
0    int64
1    int64
dtype: object
```

****Non-square DataFrame with mixed dtypes****

```
>>> d2 = {
...     "name": ["Alice", "Bob"],
...     "score": [9.5, 8],
...     "employed": [False, True],
...     "kids": [0, 0],
... }
>>> df2 = pd.DataFrame(data=d2)
>>> df2
   name  score  employed  kids
0  Alice   9.5     False    0
1   Bob    8.0      True    0

>>> df2_transposed = df2.T # or df2.transpose()
>>> df2_transposed
      0      1
name  Alice  Bob
score   9.5   8.0
employed False  True
kids      0      0
```

When the DataFrame has mixed dtypes, we get a transposed DataFrame with the `object` dtype:

```
>>> df2.dtypes
name      str
score    float64
employed   bool
kids      int64
dtype: object
>>> df2_transposed.dtypes
0    object
1    object
dtype: object
```

`truediv(self, other, axis: Axis = 'columns', level=None, fill_value=None) -> DataFrame` from `pd.DataFrame`.
Get Floating division of dataframe and other, element-wise (binary operator ``truediv``).

Equivalent to ``dataframe / other``, but with support to substitute a `fill_value` for missing data in one of the inputs. With reverse version, ``rtruediv``.

Among flexible wrappers (``add``, ``sub``, ``mul``, ``div``, ``floordiv``, ``mod``, ``pow``) to arithmetic operators: ``+``, ``-``, ``*``, ``/``, ``//``, ``%``, ``**``.

Parameters

`other` : scalar, sequence, Series, dict or DataFrame
Any single or multiple element data structure, or list-like object.

`axis` : {0 or 'index', 1 or 'columns'}
Whether to compare by the index (0 or 'index') or columns. (1 or 'columns'). For Series input, axis to match Series index on.

`level` : int or label
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : float or None, default None
Fill existing missing (NaN) values, and any new element needed for successful DataFrame alignment, with this value before computation. If data in both corresponding DataFrame locations is missing the result will be missing.

Returns

`DataFrame`
Result of the arithmetic operation.

See Also

`DataFrame.add` : Add DataFrames.
`DataFrame.sub` : Subtract DataFrames.

DataFrame.mul : Multiply DataFrames.
 DataFrame.div : Divide DataFrames (float division).
 DataFrame.truediv : Divide DataFrames (float division).
 DataFrame.floordiv : Divide DataFrames (integer division).
 DataFrame.mod : Calculate modulo (remainder after division).
 DataFrame.pow : Calculate exponential power.

Notes

Mismatched indices will be unioned together.

Examples

```
>>> df = pd.DataFrame({'angles': [0, 3, 4],
...                     'degrees': [360, 180, 360]},
...                     index=['circle', 'triangle', 'rectangle'])
>>> df
```

	angles	degrees
circle	0	360
triangle	3	180
rectangle	4	360

Add a scalar with operator version which return the same results.

```
>>> df + 1
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

```
>>> df.add(1)
```

	angles	degrees
circle	1	361
triangle	4	181
rectangle	5	361

Divide by constant with reverse version.

```
>>> df.div(10)
```

	angles	degrees
circle	0.0	36.0
triangle	0.3	18.0
rectangle	0.4	36.0

```
>>> df.rdiv(10)
```

	angles	degrees
circle	inf	0.027778
triangle	3.333333	0.055556
rectangle	2.500000	0.027778

Subtract a list and Series by axis with operator version.

```
>>> df - [1, 2]
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub([1, 2], axis='columns')
```

	angles	degrees
circle	-1	358
triangle	2	178
rectangle	3	358

```
>>> df.sub(pd.Series([1, 1, 1], index=['circle', 'triangle', 'rectangle']),
...        axis='index')
```

	angles	degrees
circle	-1	359
triangle	2	179
rectangle	3	359

Multiply a dictionary by axis.

```
>>> df.mul({'angles': 0, 'degrees': 2})
```

	angles	degrees
circle	0	720

```
triangle      0      360
rectangle     0      720
```

```
>>> df.mul({'circle': 0, 'triangle': 2, 'rectangle': 3}, axis='index')
      angles  degrees
circle      0        0
triangle    6      360
rectangle   12     1080
```

Multiply a DataFrame of different shape with operator version.

```
>>> other = pd.DataFrame({'angles': [0, 3, 4]},
...                       index=['circle', 'triangle', 'rectangle'])
>>> other
      angles
circle      0
triangle    3
rectangle    4
```

```
>>> df * other
      angles  degrees
circle      0      NaN
triangle    9      NaN
rectangle   16      NaN
```

```
>>> df.mul(other, fill_value=0)
      angles  degrees
circle      0      0.0
triangle    9      0.0
rectangle   16      0.0
```

Divide by a MultiIndex by level.

```
>>> df_multindex = pd.DataFrame({'angles': [0, 3, 4, 4, 5, 6],
...                               'degrees': [360, 180, 360, 360, 540, 720]},
...                              index=[['A', 'A', 'A', 'B', 'B', 'B'],
...                                     ['circle', 'triangle', 'rectangle',
...                                      'square', 'pentagon', 'hexagon']])
>>> df_multindex
      angles  degrees
A circle      0      360
  triangle    3      180
  rectangle    4      360
B square      4      360
  pentagon    5      540
  hexagon     6      720
```

```
>>> df.div(df_multindex, level=1, fill_value=0)
      angles  degrees
A circle    NaN      1.0
  triangle  1.0      1.0
  rectangle  1.0      1.0
B square     0.0      0.0
  pentagon  0.0      0.0
  hexagon   0.0      0.0
```

```
>>> df_pow = pd.DataFrame({'A': [2, 3, 4, 5],
...                        'B': [6, 7, 8, 9]})
>>> df_pow.pow(2)
   A  B
0  4  36
1  9  49
2 16  64
3 25  81
```

`unstack(self, level: IndexLabel = -1, fill_value=None, sort: bool = True) -> DataFrame | Series`
Pivot a level of the (necessarily hierarchical) index labels.

Returns a DataFrame having a new level of column labels whose inner-most level consists of the pivoted index labels.

If the index is not a MultiIndex, the output will be a Series (the analogue of stack when the columns are not a MultiIndex).

Parameters

level : int, str, or list of these, default -1 (last level)

Level(s) of index to unstack, can pass level name.
 fill_value : scalar
 Replace NaN with this value if the unstack produces missing values.
 sort : bool, default True
 Sort the level(s) in the resulting MultiIndex columns.

Returns

Series or DataFrame
 If index is a MultiIndex: DataFrame with pivoted index labels as new inner-most level column labels, else Series.

See Also

DataFrame.pivot : Pivot a table based on column values.
 DataFrame.stack : Pivot a level of the column labels (inverse operation from 'unstack').

Notes

Reference :ref:`the user guide <reshaping.stackings>` for more examples.

Examples

```
>>> index = pd.MultiIndex.from_tuples(
...     [("one", "a"), ("one", "b"), ("two", "a"), ("two", "b")]
... )
>>> s = pd.Series(np.arange(1.0, 5.0), index=index)
>>> s
one  a    1.0
     b    2.0
two  a    3.0
     b    4.0
dtype: float64

>>> s.unstack(level=-1)
      a    b
one  1.0  2.0
two  3.0  4.0

>>> s.unstack(level=0)
      one  two
a    1.0   3.0
b    2.0   4.0

>>> df = s.unstack(level=0)
>>> df.unstack()
one  a    1.0
     b    2.0
two  a    3.0
     b    4.0
dtype: float64
```

```
update(
    self,
    other,
    join: UpdateJoin = 'left',
    overwrite: bool = True,
    filter_func=None,
    errors: IgnoreRaise = 'ignore'
) -> None from pandas.core.frame.DataFrame
Modify in place using non-NA values from another DataFrame.
```

Aligns on indices. There is no return value.

Parameters

other : DataFrame, or object coercible into a DataFrame
 Should have at least one matching index/column label with the original DataFrame. If a Series is passed, its name attribute must be set, and that will be used as the column name to align with the original DataFrame.
 join : {'left'}, default 'left'
 Only left join is implemented, keeping the index and columns of the original object.
 overwrite : bool, default True
 How to handle non-NA values for overlapping keys:

```

    * True: overwrite original DataFrame's values
      with values from `other`.
    * False: only update values that are NA in
      the original DataFrame.

filter_func : callable(1d-array) -> bool 1d-array, optional
    Can choose to replace values other than NA. Return True for values
    that should be updated.
errors : {'raise', 'ignore'}, default 'ignore'
    If 'raise', will raise a ValueError if the DataFrame and `other`
    both contain non-NA data in the same place.

Returns
-----
None
    This method directly changes calling object.

Raises
-----
ValueError
    * When `errors='raise'` and there's overlapping non-NA data.
    * When `errors` is not either `ignore` or `raise`
NotImplementedError
    * If `join != 'left'`

See Also
-----
dict.update : Similar method for dictionaries.
DataFrame.merge : For column(s)-on-column(s) operations.

Notes
-----
1. Duplicate indices on `other` are not supported and raises `ValueError`.

Examples
-----
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [400, 500, 600]})
>>> new_df = pd.DataFrame({"B": [4, 5, 6], "C": [7, 8, 9]})
>>> df.update(new_df)
>>> df
   A  B
0  1  4
1  2  5
2  3  6

The DataFrame's length does not increase as a result of the update,
only values at matching index/column labels are updated.

>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_df = pd.DataFrame({"B": ["d", "e", "f", "g", "h", "i"]})
>>> df.update(new_df)
>>> df
   A  B
0  a  d
1  b  e
2  c  f

>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_df = pd.DataFrame({"B": ["d", "f"]}, index=[0, 2])
>>> df.update(new_df)
>>> df
   A  B
0  a  d
1  b  y
2  c  f

For Series, its name attribute must be set.

>>> df = pd.DataFrame({"A": ["a", "b", "c"], "B": ["x", "y", "z"]})
>>> new_column = pd.Series(["d", "e", "f"], name="B")
>>> df.update(new_column)
>>> df
   A  B
0  a  d
1  b  e
2  c  f

```

If `other` contains NaNs the corresponding values are not updated in the original dataframe.

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [400.0, 500.0, 600.0]})
>>> new_df = pd.DataFrame({"B": [4, np.nan, 6]})
>>> df.update(new_df)
>>> df
```

	A	B
0	1	4.0
1	2	500.0
2	3	6.0

```
value_counts(
    self,
    subset: IndexLabel | None = None,
    normalize: bool = False,
    sort: bool = True,
    ascending: bool = False,
    dropna: bool = True
) -> Series from pandas.core.frame.DataFrame
Return a Series containing the frequency of each distinct row in the DataFrame.
```

Parameters

subset : Hashable or a sequence of the previous, optional
Columns to use when counting unique combinations.

normalize : bool, default False
Return proportions rather than frequencies.

sort : bool, default True
Stable sort by frequencies when True. Preserve the order of the data when False.

.. versionchanged:: 3.0.0

Prior to 3.0.0, ``sort=False`` would sort by the columns values.

.. versionchanged:: 3.0.0

Prior to 3.0.0, the sort was unstable.

ascending : bool, default False
Sort in ascending order.

dropna : bool, default True
Do not include counts of rows that contain NA values.

Returns

Series

Series containing the frequency of each distinct row in the DataFrame.

See Also

Series.value_counts: Equivalent method on Series.

Notes

The returned Series will have a MultiIndex with one level per input column but an Index (non-multi) for a single label. By default, rows that contain any NA values are omitted from the result. By default, the resulting Series will be sorted by frequencies in descending order so that the first element is the most frequently-occurring row.

Examples

```
>>> df = pd.DataFrame(
...     {"num_legs": [2, 4, 4, 6], "num_wings": [2, 0, 0, 0]},
...     index=["falcon", "dog", "cat", "ant"],
... )
>>> df
```

	num_legs	num_wings
falcon	2	2
dog	4	0
cat	4	0
ant	6	0

```
>>> df.value_counts()
num_legs  num_wings
```

```

4          0          2
2          2          1
6          0          1
Name: count, dtype: int64

```

```

>>> df.value_counts(sort=False)
num_legs  num_wings
2          2          1
4          0          2
6          0          1
Name: count, dtype: int64

```

```

>>> df.value_counts(ascending=True)
num_legs  num_wings
2          2          1
6          0          1
4          0          2
Name: count, dtype: int64

```

```

>>> df.value_counts(normalize=True)
num_legs  num_wings
4          0          0.50
2          2          0.25
6          0          0.25
Name: proportion, dtype: float64

```

With `dropna` set to `False` we can also count rows with NA values.

```

>>> df = pd.DataFrame(
...     {
...         "first_name": ["John", "Anne", "John", "Beth"],
...         "middle_name": ["Smith", pd.NA, pd.NA, "Louise"],
...     }
... )
>>> df
   first_name middle_name
0        John        Smith
1        Anne         NaN
2        John         NaN
3        Beth        Louise

```

```

>>> df.value_counts()
first_name  middle_name
John        Smith      1
Beth        Louise      1
Name: count, dtype: int64

```

```

>>> df.value_counts(dropna=False)
first_name  middle_name
John        Smith      1
Anne        NaN        1
John        NaN        1
Beth        Louise      1
Name: count, dtype: int64

```

```

>>> df.value_counts("first_name")
first_name
John      2
Anne      1
Beth      1
Name: count, dtype: int64

```

```

var(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) -> Series | Any from pandas.core.frame.DataFrame
Return unbiased variance over requested axis.

```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

axis : {index (0), columns (1)}
 For `Series` this parameter is unused and defaults to 0.

.. warning::

The behavior of DataFrame.var with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis).

skipna : bool, default True
 Exclude NA/null values. If an entire row/column is NA, the result will be NA.

ddof : int, default 1
 Delta Degrees of Freedom. The divisor used in calculations is N - ddof, where N represents the number of elements.

numeric_only : bool, default False
 Include only float, int, boolean columns. Not implemented for Series.

**kwargs :
 Additional keywords passed.

Returns

Series or scalar
 Unbiased variance over requested axis.

See Also

numpy.var : Equivalent function in NumPy.
 Series.var : Return unbiased variance over Series values.
 Series.std : Return standard deviation over Series values.
 DataFrame.std : Return standard deviation of the values over the requested axis.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "person_id": [0, 1, 2, 3],
...         "age": [21, 25, 62, 43],
...         "height": [1.61, 1.87, 1.49, 2.01],
...     })
... df.set_index("person_id")
>>> df
```

	age	height
person_id		
0	21	1.61
1	25	1.87
2	62	1.49
3	43	2.01

```
>>> df.var()
age      352.916667
height    0.056367
dtype: float64
```

Alternatively, ``ddof=0`` can be set to normalize by N instead of N-1:

```
>>> df.var(ddof=0)
age      264.687500
height    0.042275
dtype: float64
```

Class methods inherited from pandas.DataFrame:

from_arrow(data: ArrowArrayExportable | ArrowStreamExportable) -> DataFrame from pandas.core
 Construct a DataFrame from a tabular Arrow object.

This function accepts any Arrow-compatible tabular object implementing the `Arrow PyCapsule Protocol`_ (i.e. having an ``\_\_arrow\_c\_array\_\_`` or ``\_\_arrow\_c\_stream\_\_`` method).

This function currently relies on ``pyarrow`` to convert the tabular object in Arrow format to pandas.

.. _Arrow PyCapsule Protocol: <https://arrow.apache.org/docs/format/CDataInterface/PyCapsule>

.. versionadded:: 3.0

Parameters

data : pyarrow.Table or Arrow-compatible table
Any tabular object implementing the Arrow PyCapsule Protocol (i.e. has an ``\_\_arrow\_c\_array\_\_`` or ``\_\_arrow\_c\_stream\_\_`` method).

Returns

DataFrame

See Also

Series.from_arrow : Construct a Series from an Arrow object.

Examples

>>> import pyarrow as pa
>>> table = pa.table({"a": [1, 2, 3], "b": ["x", "y", "z"]})
>>> pd.DataFrame.from_arrow(table)
 a b
0 1 x
1 2 y
2 3 z

from_dict(
 data: dict,
 orient: FromDictOrient = 'columns',
 dtype: Dtype | None = None,
 columns: Axes | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Construct DataFrame from dict of array-like or dicts.

Creates DataFrame object from dictionary by columns or by index
allowing dtype specification.

Parameters

data : dict
Of the form {field : array-like} or {field : dict}.
orient : {'columns', 'index', 'tight'}, default 'columns'
The "orientation" of the data. If the keys of the passed dict should be the columns of the resulting DataFrame, pass 'columns' (default). Otherwise if the keys should be rows, pass 'index'. If 'tight', assume a dict with keys ['index', 'columns', 'data', 'index_names', 'column_names'].

dtype : dtype, default None
Data type to force after DataFrame construction, otherwise infer.
columns : list, default None
Column labels to use when ``orient='index'``. Raises a ValueError if used with ``orient='columns'`` or ``orient='tight'``.

Returns

DataFrame

See Also

DataFrame.from_records : DataFrame from structured ndarray, sequence of tuples or dicts, or DataFrame.
DataFrame : DataFrame object creation using constructor.
DataFrame.to_dict : Convert the DataFrame to a dictionary.

Examples

By default the keys of the dict become the DataFrame columns:

>>> data = {"col_1": [3, 2, 1, 0], "col_2": ["a", "b", "c", "d"]}
>>> pd.DataFrame.from_dict(data)
 col_1 col_2
0 3 a
1 2 b
2 1 c
3 0 d

Specify ``orient='index'`` to create the DataFrame using dictionary keys as rows:

```
>>> data = {"row_1": [3, 2, 1, 0], "row_2": ["a", "b", "c", "d"]}
>>> pd.DataFrame.from_dict(data, orient="index")
      0  1  2  3
row_1  3  2  1  0
row_2  a  b  c  d
```

When using the 'index' orientation, the column names can be specified manually:

```
>>> pd.DataFrame.from_dict(data, orient="index", columns=["A", "B", "C", "D"])
      A  B  C  D
row_1  3  2  1  0
row_2  a  b  c  d
```

Specify ``orient='tight'`` to create the DataFrame using a 'tight' format:

```
>>> data = {
...     "index": [("a", "b"), ("a", "c")],
...     "columns": [("x", 1), ("y", 2)],
...     "data": [[1, 3], [2, 4]],
...     "index_names": ["n1", "n2"],
...     "column_names": ["z1", "z2"],
... }
>>> pd.DataFrame.from_dict(data, orient="tight")
z1      x  y
z2      1  2
n1 n2
a  b      1  3
   c      2  4
```

```
from_records(
    data,
    index=None,
    exclude=None,
    columns=None,
    coerce_float: bool = False,
    nrows: int | None = None
) -> DataFrame from pandas.core.frame.DataFrame
Convert structured or record ndarray to DataFrame.
```

Creates a DataFrame object from a structured ndarray, or iterable of tuples or dicts.

Parameters

data : structured ndarray, iterable of tuples or dicts
Structured input data.

index : str, list of fields, array-like
Field of array to use as the index, alternately a specific set of input labels to use.

exclude : sequence, default None
Columns or fields to exclude.

columns : sequence, default None
Column names to use. If the passed data do not have names associated with them, this argument provides names for the columns. Otherwise, this argument indicates the order of the columns in the result (any names not found in the data will become all-NA columns) and limits the data to these columns if not all column names are provided.

coerce_float : bool, default False
Attempt to convert values of non-string, non-numeric objects (like decimal.Decimal) to floating point, useful for SQL result sets.

nrows : int, default None
Number of rows to read if data is an iterator.

Returns

DataFrame

See Also

DataFrame.from_dict : DataFrame from dict of array-like or dicts.

DataFrame : DataFrame object creation using constructor.

Examples

Data can be provided as a structured ndarray:

```
>>> data = np.array(
...     [(3, "a"), (2, "b"), (1, "c"), (0, "d")],
...     dtype=[("col_1", "i4"), ("col_2", "U1")],
... )
>>> pd.DataFrame.from_records(data)
   col_1 col_2
0       3    a
1       2    b
2       1    c
3       0    d
```

Data can be provided as a list of dicts:

```
>>> data = [
...     {"col_1": 3, "col_2": "a"},
...     {"col_1": 2, "col_2": "b"},
...     {"col_1": 1, "col_2": "c"},
...     {"col_1": 0, "col_2": "d"},
... ]
>>> pd.DataFrame.from_records(data)
   col_1 col_2
0       3    a
1       2    b
2       1    c
3       0    d
```

Data can be provided as a list of tuples with corresponding columns:

```
>>> data = [(3, "a"), (2, "b"), (1, "c"), (0, "d")]
>>> pd.DataFrame.from_records(data, columns=["col_1", "col_2"])
   col_1 col_2
0       3    a
1       2    b
2       1    c
3       0    d
```

Readonly properties inherited from pandas.DataFrame:

T

The transpose of the DataFrame.

Returns

DataFrame

The transposed DataFrame.

See Also

DataFrame.transpose : Transpose index and columns.

Examples

```
>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df
   col1  col2
0     1     3
1     2     4

>>> df.T
      0  1
col1  1  2
col2  3  4
```

axes

Return a list representing the axes of the DataFrame.

It has the row axis labels and column axis labels as the only members.
They are returned in that order.

See Also

DataFrame.index: The index (row labels) of the DataFrame.
DataFrame.columns: The column labels of the DataFrame.

Examples

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.axes
[RangeIndex(start=0, stop=2, step=1), Index(['col1', 'col2'], dtype='str')]

shape

Return a tuple representing the dimensionality of the DataFrame.

Unlike the `len()` method, which only returns the number of rows, `shape` provides both row and column counts, making it a more informative method for understanding dataset size.

See Also

`numpy.ndarray.shape` : Tuple of array dimensions.

Examples

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.shape
(2, 2)

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4], "col3": [5, 6]})
>>> df.shape
(2, 3)

style

Returns a Styler object.

Contains methods for building a styled HTML representation of the DataFrame.

See Also

`io.formats.style.Styler` : Helps style a DataFrame or Series according to the data with HTML and CSS.

Examples

>>> df = pd.DataFrame({"A": [1, 2, 3]})
>>> df.style # doctest: +SKIP

Please see

`Table Visualization <../user_guide/style.ipynb>` for more examples.

values

Return a Numpy representation of the DataFrame.

.. warning::

We recommend using `:meth:`DataFrame.to_numpy`` instead.

Only the values in the DataFrame will be returned, the axes labels will be removed.

Returns

`numpy.ndarray`
The values of the DataFrame.

See Also

`DataFrame.to_numpy` : Recommended alternative to this method.
`DataFrame.index` : Retrieve the index labels.
`DataFrame.columns` : Retrieving the column names.

Notes

The dtype will be a lower-common-denominator dtype (implicit upcasting); that is to say if the dtypes (even of numeric types) are mixed, the one that accommodates all will be chosen. Use this with care if you are not dealing with the blocks.

e.g. If the dtypes are float16 and float32, dtype will be upcast to float32. If dtypes are int32 and uint8, dtype will be upcast to int32. By `:func:`numpy.find_common_type`` convention, mixing int64 and uint64 will result in a float64 dtype.

Examples

A DataFrame where all columns are the same type (e.g., int64) results in an array of the same type.

```
>>> df = pd.DataFrame(
...     {"age": [3, 29], "height": [94, 170], "weight": [31, 115]}
... )
>>> df
   age  height  weight
0    3     94     31
1   29    170    115
>>> df.dtypes
age          int64
height       int64
weight       int64
dtype: object
>>> df.values
array([[ 3, 94, 31],
       [29, 170, 115]])
```

A DataFrame with mixed type columns (e.g., str/object, int64, float32) results in an ndarray of the broadest type that accommodates these mixed types (e.g., object).

```
>>> df2 = pd.DataFrame(
...     [
...         ("parrot", 24.0, "second"),
...         ("lion", 80.5, 1),
...         ("monkey", np.nan, None),
...     ],
...     columns=("name", "max_speed", "rank"),
... )
>>> df2.dtypes
name          str
max_speed     float64
rank          object
dtype: object
>>> df2.values
array(['parrot', 24.0, 'second'],
      ['lion', 80.5, 1],
      ['monkey', nan, None]], dtype=object)
```

Data descriptors inherited from `pandas.DataFrame`:

columns

The column labels of the DataFrame.

This property holds the column names as a pandas `Index` object. It provides an immutable sequence of column labels that can be used for data selection, renaming, and alignment in DataFrame operations.

Returns

`pandas.Index`

The column labels of the DataFrame.

See Also

`DataFrame.index`: The index (row labels) of the DataFrame.

`DataFrame.axes`: Return a list representing the axes of the DataFrame.

Examples

```
>>> df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
>>> df
   A  B
0  1  3
1  2  4
>>> df.columns
Index(['A', 'B'], dtype='str')
```

index

The index (row labels) of the DataFrame.

The index of a DataFrame is a series of labels that identify each row. The labels can be integers, strings, or any other hashable type. The index is used for label-based access and alignment, and can be accessed or modified using this attribute.

Returns

pandas.Index

The index labels of the DataFrame.

See Also

DataFrame.columns : The column labels of the DataFrame.

DataFrame.to_numpy : Convert the DataFrame to a NumPy array.

Examples

```
>>> df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Aritra'],
...                    'Age': [25, 30, 35],
...                    'Location': ['Seattle', 'New York', 'Kona']},
...                    index=[10, 20, 30])
>>> df.index
Index([10, 20, 30], dtype='int64')
```

In this example, we create a DataFrame with 3 rows and 3 columns, including Name, Age, and Location information. We set the index labels to be the integers 10, 20, and 30. We then access the `index` attribute of the DataFrame, which returns an `Index` object containing the index labels.

```
>>> df.index = [100, 200, 300]
>>> df
```

```
   Name  Age Location
100  Alice   25  Seattle
200   Bob   30  New York
300  Aritra  35    Kona
```

In this example, we modify the index labels of the DataFrame by assigning a new list of labels to the `index` attribute. The DataFrame is then updated with the new labels, and the output shows the modified DataFrame.

Data and other attributes inherited from pandas.DataFrame:

__annotations__ = {}

__pandas_priority__ = 4000

plot = <class 'pandas.plotting.PlotAccessor'>
Make plots of Series or DataFrame.

Uses the backend specified by the option ``plotting.backend``. By default, matplotlib is used.

Parameters

data : Series or DataFrame
The object for which the method is called.

Attributes

x : label or position, default None
Only used if data is a DataFrame.

y : label, position or list of label, positions, default None
Allows plotting of one column versus another. Only used if data is a DataFrame.

kind : str
The kind of plot to produce:

- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot

- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

ax : matplotlib axes object, default None
An axes of the current figure.

subplots : bool or sequence of iterables, default False
Whether to group columns into subplots:

- ``False`` : No subplots will be used
- ``True`` : Make separate subplots for each column.
- sequence of iterables of column labels: Create a subplot for each group of columns. For example ``[( 'a', 'c'), ('b', 'd')]`` will create 2 subplots: one with columns 'a' and 'c', and one with columns 'b' and 'd'. Remaining columns that aren't specified will be plotted in additional subplots (one per column).

sharex : bool, default True if ax is None else False
In case ``subplots=True``, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in; Be aware, that passing in both an ax and ``sharex=True`` will alter all x axis labels for all axis in a figure.

sharey : bool, default False
In case ``subplots=True``, share y axis and set some y axis labels to invisible.

layout : tuple, optional
(rows, columns) for the layout of subplots.

figsize : a tuple (width, height) in inches
Size of a figure object.

use_index : bool, default True
Use index as ticks for x axis.

title : str or list
Title to use for the plot. If a string is passed, print the string at the top of the figure. If a list is passed and ``subplots`` is True, print each item in the list above the corresponding subplot.

grid : bool, default None (matlab style default)
Axis grid lines.

legend : bool or {'reverse'}
Place legend on axis subplots.

style : list or dict
The matplotlib line style per column.

logx : bool or 'sym', default False
Use log scaling or symlog scaling on x axis.

logy : bool or 'sym' default False
Use log scaling or symlog scaling on y axis.

loglog : bool or 'sym', default False
Use log scaling or symlog scaling on both x and y axes.

xticks : sequence
Values to use for the xticks.

yticks : sequence
Values to use for the yticks.

xlim : 2-tuple/list
Set the x limits of the current axes.

ylim : 2-tuple/list
Set the y limits of the current axes.

xlabel : label, optional
Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.

.. versionchanged:: 2.0.0
Now applicable to histograms.

ylabel : label, optional
Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.

.. versionchanged:: 2.0.0
Now applicable to histograms.

rot : float, default None
Rotation for ticks (xticks for vertical, yticks for horizontal)

plots).

fontsize : float, default None
Font size for xticks and yticks.

colormap : str or matplotlib colormap object, default None
Colormap to select colors from. If string, load colormap with that name from matplotlib.

colorbar : bool, optional
If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).

position : float
Specify relative alignments for bar plot layout.
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).

table : bool, Series or DataFrame, default False
If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout.
If a Series or DataFrame is passed, use passed data to draw a table.

yerr : DataFrame, Series, array-like, dict and str
See :ref:`Plotting with Error Bars <visualization.errorbars>` for detail.

xerr : DataFrame, Series, array-like, dict and str
Equivalent to yerr.

stacked : bool, default False in line and bar plots, and True in area plot
If True, create stacked plot.

secondary_y : bool or sequence, default False
Whether to plot on the secondary y-axis if a list/tuple, which columns to plot on secondary y-axis.

mark_right : bool, default True
When using a secondary_y axis, automatically mark the column labels with "(right)" in the legend.

include_bool : bool, default is False
If True, boolean values can be plotted.

backend : str, default None
Backend to use instead of the backend specified in the option ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to specify the ``plotting.backend`` for the whole session, set ``pd.options.plotting.backend``.

****kwargs**
Options to pass to matplotlib plotting method.

Returns

:class:`matplotlib.axes.Axes` or numpy.ndarray of them
If the backend is not the default matplotlib one, the return value will be the object returned by the backend.

See Also

matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.
DataFrame.hist : Make a histogram.
DataFrame.boxplot : Make a box plot.
DataFrame.plot.scatter : Make a scatter plot with varying marker point size and color.
DataFrame.plot.hexbin : Make a hexagonal binning plot of two variables.
DataFrame.plot.kde : Make Kernel Density Estimate plot using Gaussian kernels.
DataFrame.plot.area : Make a stacked area plot.
DataFrame.plot.bar : Make a bar plot.
DataFrame.plot.barh : Make a horizontal bar plot.

Notes

- See matplotlib documentation online for more on this subject
- If `kind` = 'bar' or 'barh', you can specify relative alignments for bar plot layout by `position` keyword.
From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center)

Examples

For Series:

```
.. plot::
    :context: close-figs
```

```
>>> ser = pd.Series([1, 2, 3, 3])
>>> plot = ser.plot(kind="hist", title="My plot")
```

For DataFrame:

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame(
...     {
...         "length": [1.5, 0.5, 1.2, 0.9, 3],
...         "width": [0.7, 0.2, 0.15, 0.2, 1.1],
...     },
...     index=["pig", "rabbit", "duck", "chicken", "horse"],
... )
>>> plot = df.plot(title="DataFrame Plot")
```

For SeriesGroupBy:

```
.. plot::
:context: close-figs

>>> lst = [-1, -2, -3, 1, 2, 3]
>>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
>>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")
```

For DataFrameGroupBy:

```
.. plot::
:context: close-figs

>>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})
>>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")
```

sparse = <class 'pandas.core.arrays.sparse.accessor.SparseFrameAccesso...
DataFrame accessor for sparse data.

It allows users to interact with a `DataFrame` that contains sparse data types (`SparseDtype`). It provides methods and attributes to efficiently work with sparse storage, reducing memory usage while maintaining compatibility with standard pandas operations.

Parameters

data : scipy.sparse.spmatrix
Must be convertible to csc format.

See Also

DataFrame.sparse.density : Ratio of non-sparse points to total (dense) data points.

Examples

>>> df = pd.DataFrame({"a": [1, 2, 0, 0], "b": [3, 0, 0, 4]}, dtype="Sparse[int]")
>>> df.sparse.density
np.float64(0.5)

Methods inherited from pandas.core.generic.NDFrame:

__abs__(self) -> Self

__array__(self, dtype: npt.DTypeLike | None = None, copy: bool | None = None) -> np.ndarray

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs: Any, **kwargs: Any)

__bool__(self) -> NoReturn

__contains__(self, key) -> bool
True if the key is in the info axis

__copy__(self) -> Self

__deepcopy__(self, memo=None) -> Self
Parameters

```

-----
memo, default None
    Standard signature. Unused
__delitem__(self, key) -> None
    Delete item
__finalize__(self, other, method: str | None = None, **kwargs) -> Self
    Propagate metadata from other to self.

    This is the default implementation. Subclasses may override this method to
    implement their own metadata handling.

    Parameters
    -----
    other : the object from which to get the attributes that we are going
        to propagate. If ``other`` has an ``input_objs`` attribute, then
        this attribute must contain an iterable of objects, each with an
        ``attrs`` attribute.
    method : str, optional
        A passed method name providing context on where ``__finalize__``
        was called.

    .. warning::

        The value passed as `method` are not currently considered
        stable across pandas releases.

    Notes
    -----
    In case ``other`` has an ``input_objs`` attribute, this method only
    propagates its metadata if each object in ``input_objs`` has the exact
    same metadata as the others.
__getattr__(self, name: str)
    After regular attribute access, try looking up the name
    This allows simpler access to columns for interactive use.
__getstate__(self) -> dict[str, Any]
    Helper for pickle.
__iadd__(self, other) -> Self
__iand__(self, other) -> Self
__ifloordiv__(self, other) -> Self
__imod__(self, other) -> Self
__imul__(self, other) -> Self
__invert__(self) -> Self
__ior__(self, other) -> Self
__ipow__(self, other) -> Self
__isub__(self, other) -> Self
__iter__(self) -> Iterator
    Iterate over info axis.

    Returns
    -----
    iterator
        Info axis as iterator.

    See Also
    -----
    DataFrame.items : Iterate over (column name, Series) pairs.
    DataFrame.itertuples : Iterate over DataFrame rows as namedtuples.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
    >>> for x in df:
    ...     print(x)

```

```

A
B
__itruediv__(self, other) -> Self
__ixor__(self, other) -> Self
__neg__(self) -> Self
__pos__(self) -> Self
__round__(self, decimals: int = 0) -> Self
__setattr__(self, name: str, value) -> None
    After regular attribute access, try setting the name
    This allows simpler access to columns for interactive use.
__setstate__(self, state) -> None

abs(self) -> Self
    Return a Series/DataFrame with absolute numeric value of each element.

    This function only applies to elements that are all numeric.

Returns
-----
abs
    Series/DataFrame containing the absolute value of each element.

See Also
-----
numpy.absolute : Calculate the absolute value element-wise.

Notes
-----
For ``complex`` inputs, ``1.2 + 1j``, the absolute value is
:math:\sqrt{a^2 + b^2}``.

Examples
-----
Absolute numeric values in a Series.

>>> s = pd.Series([-1.10, 2, -3.33, 4])
>>> s.abs()
0    1.10
1    2.00
2    3.33
3    4.00
dtype: float64

Absolute numeric values in a Series with complex numbers.

>>> s = pd.Series([1.2 + 1j])
>>> s.abs()
0    1.56205
dtype: float64

Absolute numeric values in a Series with a Timedelta element.

>>> s = pd.Series([pd.Timedelta("1 days")])
>>> s.abs()
0    1 days
dtype: timedelta64[us]

Select rows with data closest to certain value using argsort (from
`StackOverflow <https://stackoverflow.com/a/17758115>`__).

>>> df = pd.DataFrame(
...     {"a": [4, 5, 6, 7], "b": [10, 20, 30, 40], "c": [100, 50, -30, -50]}
... )
>>> df
   a  b  c
0  4  10 100
1  5  20  50
2  6  30 -30
3  7  40 -50
>>> df.loc[(df.c - 43).abs().argsort()]

```

	a	b	c
1	5	20	50
0	4	10	100
2	6	30	-30
3	7	40	-50

```
add_prefix(self, prefix: str, axis: Axis | None = None) -> Self
    Prefix labels with string `prefix`.
```

For Series, the row labels are prefixed.
For DataFrame, the column labels are prefixed.

Parameters

prefix : str
 The string to add before each label.
axis : {0 or 'index', 1 or 'columns', None}, default None
 Axis to add prefix on

.. versionadded:: 2.0.0

Returns

Series or DataFrame
 New Series or DataFrame with updated labels.

See Also

Series.add_suffix: Suffix row labels with string `suffix`.
DataFrame.add_suffix: Suffix column labels with string `suffix`.

Examples

>>> s = pd.Series([1, 2, 3, 4])
>>> s
0 1
1 2
2 3
3 4
dtype: int64

>>> s.add_prefix("item_")
item_0 1
item_1 2
item_2 3
item_3 4
dtype: int64

>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})
>>> df
 A B
0 1 3
1 2 4
2 3 5
3 4 6

>>> df.add_prefix("col_")
 col_A col_B
0 1 3
1 2 4
2 3 5
3 4 6

```
add_suffix(self, suffix: str, axis: Axis | None = None) -> Self
    Suffix labels with string `suffix`.
```

For Series, the row labels are suffixed.
For DataFrame, the column labels are suffixed.

Parameters

suffix : str
 The string to add after each label.
axis : {0 or 'index', 1 or 'columns', None}, default None
 Axis to add suffix on

.. versionadded:: 2.0.0

Returns

Series or DataFrame

New Series or DataFrame with updated labels.

See Also

Series.add_prefix: Prefix row labels with string `prefix`.

DataFrame.add_prefix: Prefix column labels with string `prefix`.

Examples

```
>>> s = pd.Series([1, 2, 3, 4])
```

```
>>> s
```

```
0    1
```

```
1    2
```

```
2    3
```

```
3    4
```

```
dtype: int64
```

```
>>> s.add_suffix("_item")
```

```
0_item    1
```

```
1_item    2
```

```
2_item    3
```

```
3_item    4
```

```
dtype: int64
```

```
>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})
```

```
>>> df
```

```
   A  B
```

```
0  1  3
```

```
1  2  4
```

```
2  3  5
```

```
3  4  6
```

```
>>> df.add_suffix("_col")
```

```
   A_col  B_col
```

```
0        1        3
```

```
1        2        4
```

```
2        3        5
```

```
3        4        6
```

```
align(  
    self,  
    other: NDFrameT,  
    join: AlignJoin = 'outer',  
    axis: Axis | None = None,  
    level: Level | None = None,  
    copy: bool | lib.NoDefault = <no_default>,  
    fill_value: Hashable | None = None  
) -> tuple[Self, NDFrameT]  
Align two objects on their axes with the specified join method.
```

Join method is specified for each axis Index.

Parameters

other : DataFrame or Series

The object to align with.

join : {'outer', 'inner', 'left', 'right'}, default 'outer'

Type of alignment to be performed.

* left: use only keys from left frame, preserve key order.

* right: use only keys from right frame, preserve key order.

* outer: use union of keys from both frames, sort keys lexicographically.

* inner: use intersection of keys from both frames,
preserve the order of the left keys.

axis : allowed axis of the other object, default None

Align on index (0), columns (1), or both (None).

level : int or level name, default None

Broadcast across a level, matching Index values on the
passed MultiIndex level.

copy : bool, default False

This keyword is now ignored; changing its value will have no
impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the ``user guide on Copy-on-Write`` https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html for more details.

`fill_value` : scalar, default `np.nan`
Value to use for missing values. Defaults to `NaN`, but can be any "compatible" value.

Returns

tuple of (Series/DataFrame, type of other)
Aligned objects.

See Also

`Series.align` : Align two objects on their axes with specified join method.
`DataFrame.align` : Align two objects on their axes with specified join method.

Examples

>>> df = pd.DataFrame(
... [[1, 2, 3, 4], [6, 7, 8, 9]], columns=["D", "B", "E", "A"], index=[1, 2]
...)
>>> other = pd.DataFrame(
... [[10, 20, 30, 40], [60, 70, 80, 90], [600, 700, 800, 900]],
... columns=["A", "B", "C", "D"],
... index=[2, 3, 4],
...)
>>> df
 D B E A
1 1 2 3 4
2 6 7 8 9
>>> other
 A B C D
2 10 20 30 40
3 60 70 80 90
4 600 700 800 900

Align on columns:

>>> left, right = df.align(other, join="outer", axis=1)
>>> left
 A B C D E
1 4 2 NaN 1 3
2 9 7 NaN 6 8
>>> right
 A B C D E
2 10 20 30 40 NaN
3 60 70 80 90 NaN
4 600 700 800 900 NaN

We can also align on the index:

>>> left, right = df.align(other, join="outer", axis=0)
>>> left
 D B E A
1 1.0 2.0 3.0 4.0
2 6.0 7.0 8.0 9.0
3 NaN NaN NaN NaN
4 NaN NaN NaN NaN
>>> right
 A B C D
1 NaN NaN NaN NaN
2 10.0 20.0 30.0 40.0
3 60.0 70.0 80.0 90.0
4 600.0 700.0 800.0 900.0

Finally, the default ``axis=None`` will align on both index and columns:

>>> left, right = df.align(other, join="outer", axis=None)
>>> left

	A	B	C	D	E
1	4.0	2.0	NaN	1.0	3.0
2	9.0	7.0	NaN	6.0	8.0
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN

```
>>> right
```

	A	B	C	D	E
1	NaN	NaN	NaN	NaN	NaN
2	10.0	20.0	30.0	40.0	NaN
3	60.0	70.0	80.0	90.0	NaN
4	600.0	700.0	800.0	900.0	NaN

```
asfreq(
    self,
    freq: Frequency,
    method: FillnaOptions | None = None,
    how: Literal['start', 'end'] | None = None,
    normalize: bool = False,
    fill_value: Hashable | None = None
) -> Self
Convert time series to specified frequency.
```

Returns the original data conformed to a new index with the specified frequency.

If the index of this Series/DataFrame is a :class:`~pandas.PeriodIndex`, the new index is the result of transforming the original index with :meth:`PeriodIndex.asfreq` <pandas.PeriodIndex.asfreq> (so the original index will map one-to-one to the new index).

Otherwise, the new index will be equivalent to ``pd.date\_range(start, end, freq=freq)`` where ``start`` and ``end`` are, respectively, the min and max entries in the original index (see :func:`pandas.date\_range`). The values corresponding to any timesteps in the new index which were not present in the original index will be null (``NaN``), unless a method for filling such unknowns is provided (see the ``method`` parameter below).

The :meth:`resample` method is more appropriate if an operation on each group of timesteps (such as an aggregate) is necessary to represent the data at the new frequency.

Parameters

```
freq : DateOffset or str
    Frequency DateOffset or string.
method : {'backfill'/'bfill', 'pad'/'ffill'}, default None
    Method to use for filling holes in reindexed Series (note this
    does not fill NaNs that already were present):

    * 'pad' / 'ffill': propagate last valid observation forward to next
      valid based on the order of the index
    * 'backfill' / 'bfill': use NEXT valid observation to fill.
how : {'start', 'end'}, default end
    For PeriodIndex only (see PeriodIndex.asfreq).
normalize : bool, default False
    Whether to reset output index to midnight.
fill_value : scalar, optional
    Value to use for missing values, applied during upsampling (note
    this does not fill NaNs that already were present).
```

Returns

```
Series/DataFrame
Series/DataFrame object reindexed to the specified frequency.
```

See Also

```
reindex : Conform DataFrame to new index with optional filling logic.
```

Notes

To learn more about the frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

Examples

```
Start by creating a series with 4 one minute timestamps.
```



```
>>> index = pd.date_range("1/1/2000", periods=4, freq="min")
>>> series = pd.Series([0.0, None, 2.0, 3.0], index=index)
>>> df = pd.DataFrame({"s": series})
>>> df
```

```
      s
2000-01-01 00:00:00    0.0
2000-01-01 00:01:00   NaN
2000-01-01 00:02:00    2.0
2000-01-01 00:03:00    3.0
```

Upsample the series into 30 second bins.

```
>>> df.asfreq(freq="30s")
      s
2000-01-01 00:00:00    0.0
2000-01-01 00:00:30   NaN
2000-01-01 00:01:00   NaN
2000-01-01 00:01:30   NaN
2000-01-01 00:02:00    2.0
2000-01-01 00:02:30   NaN
2000-01-01 00:03:00    3.0
```

Upsample again, providing a ``fill value``.

```
>>> df.asfreq(freq="30s", fill_value=9.0)
      s
2000-01-01 00:00:00    0.0
2000-01-01 00:00:30    9.0
2000-01-01 00:01:00   NaN
2000-01-01 00:01:30    9.0
2000-01-01 00:02:00    2.0
2000-01-01 00:02:30    9.0
2000-01-01 00:03:00    3.0
```

Upsample again, providing a ``method``.

```
>>> df.asfreq(freq="30s", method="bfill")
      s
2000-01-01 00:00:00    0.0
2000-01-01 00:00:30   NaN
2000-01-01 00:01:00   NaN
2000-01-01 00:01:30    2.0
2000-01-01 00:02:00    2.0
2000-01-01 00:02:30    3.0
2000-01-01 00:03:00    3.0
```

`asof(self, where, subset=None)`
Return the last row(s) without any NaNs before ``where``.

The last row (for each element in ``where``, if list) without any NaN is taken.

In case of a `:class:`~pandas.DataFrame``, the last row without NaN considering only the subset of columns (if not ``None``)

If there is no good value, NaN is returned for a Series or a Series of NaN values for a DataFrame

Parameters

`where` : date or array-like of dates
Date(s) before which the last row(s) are returned.

`subset` : str or array-like of str, default ``None``
For DataFrame, if not ``None``, only use these columns to check for NaNs.

Returns

scalar, Series, or DataFrame

The return can be:

- * scalar : when ``self`` is a Series and ``where`` is a scalar
- * Series: when ``self`` is a Series and ``where`` is an array-like, or when ``self`` is a DataFrame and ``where`` is a scalar
- * DataFrame : when ``self`` is a DataFrame and ``where`` is an array-like

See Also

`merge_asof` : Perform an asof merge. Similar to left join.

Notes

Dates are assumed to be sorted. Raises if this is not the case.

Examples

A Series and a scalar ``where``.

```
>>> s = pd.Series([1, 2, np.nan, 4], index=[10, 20, 30, 40])
>>> s
10    1.0
20    2.0
30    NaN
40    4.0
dtype: float64
```

```
>>> s.asof(20)
np.float64(2.0)
```

For a sequence ``where``, a Series is returned. The first value is NaN, because the first element of ``where`` is before the first index value.

```
>>> s.asof([5, 20])
5      NaN
20     2.0
dtype: float64
```

Missing values are not considered. The following is ``2.0``, not NaN, even though NaN is at the index location for ``30``.

```
>>> s.asof(30)
np.float64(2.0)
```

Take all columns into consideration

```
>>> df = pd.DataFrame(
...     {
...         "a": [10.0, 20.0, 30.0, 40.0, 50.0],
...         "b": [None, None, None, None, 500],
...     },
...     index=pd.DatetimeIndex(
...         [
...             "2018-02-27 09:01:00",
...             "2018-02-27 09:02:00",
...             "2018-02-27 09:03:00",
...             "2018-02-27 09:04:00",
...             "2018-02-27 09:05:00",
...         ]
...     ),
... )
>>> df.asof(pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]))
              a      b
2018-02-27 09:03:30 NaN NaN
2018-02-27 09:04:30 NaN NaN
```

Take a single column into consideration

```
>>> df.asof(
...     pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]),
...     subset=["a"],
... )
              a      b
2018-02-27 09:03:30  30.0 NaN
2018-02-27 09:04:30  40.0 NaN
```

```
astype(
    self,
    dtype,
    copy: bool | lib.NoDefault = <no_default>,
    errors: IgnoreRaise = 'raise'
) -> Self
```

Cast a pandas object to a specified dtype ``dtype``.

This method allows the conversion of the data types of pandas objects, including DataFrames and Series, to the specified dtype. It supports casting entire objects to a single data type or applying different data types to individual columns using a mapping.

Parameters

`dtype` : str, data type, Series or Mapping of column name -> data type
Use a str, numpy.dtype, pandas.ExtensionDtype or Python type to cast entire pandas object to the same type. Alternatively, use a mapping, e.g. {col: dtype, ...}, where col is a column label and dtype is a numpy.dtype or Python type to cast one or more of the DataFrame's columns to column-specific types.

`copy` : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`_ for more details.

`errors` : {'raise', 'ignore'}, default 'raise'
Control raising of exceptions on invalid data for provided dtype.

- ``raise`` : allow exceptions to be raised
- ``ignore`` : suppress exceptions. On error return original object.

Returns

same type as caller
The pandas object casted to the specified ``dtype``.

See Also

`to_datetime` : Convert argument to datetime.
`to_timedelta` : Convert argument to timedelta.
`to_numeric` : Convert argument to a numeric type.
`numpy.ndarray.astype` : Cast a numpy array to a specified type.

Notes

.. versionchanged:: 2.0.0

Using ``astype`` to convert from timezone-naive dtype to timezone-aware dtype will raise an exception.
Use `:meth:`Series.dt.tz_localize`` instead.

Examples

Create a DataFrame:

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df.dtypes
col1    int64
col2    int64
dtype: object
```

Cast all columns to int32:

```
>>> df.astype("int32").dtypes
col1    int32
col2    int32
dtype: object
```

Cast col1 to int32 using a dictionary:

```
>>> df.astype({"col1": "int32"}).dtypes
col1    int32
col2    int64
```

```
dtype: object
```

Create a series:

```
>>> ser = pd.Series([1, 2], dtype="int32")
>>> ser
0    1
1    2
dtype: int32
>>> ser.astype("int64")
0    1
1    2
dtype: int64
```

Convert to categorical type:

```
>>> ser.astype("category")
0    1
1    2
dtype: category
Categories (2, int32): [1, 2]
```

Convert to ordered categorical type with custom ordering:

```
>>> from pandas.api.types import CategoricalDtype
>>> cat_dtype = CategoricalDtype(categories=[2, 1], ordered=True)
>>> ser.astype(cat_dtype)
0    1
1    2
dtype: category
Categories (2, int64): [2 < 1]
```

Create a series of dates:

```
>>> ser_date = pd.Series(pd.date_range("20200101", periods=3))
>>> ser_date
0    2020-01-01
1    2020-01-02
2    2020-01-03
dtype: datetime64[us]
```

`at_time(self, time, asof: bool = False, axis: Axis | None = None) -> Self`
Select values at particular time of day (e.g., 9:30AM).

Parameters

```
-----
time : datetime.time or str
    The values to select.
asof : bool, default False
    This parameter is currently not supported.
axis : {0 or 'index', 1 or 'columns'}, default 0
    For `Series` this parameter is unused and defaults to 0.
```

Returns

```
-----
Series or DataFrame
    The values with the specified time.
```

Raises

```
-----
TypeError
    If the index is not a :class:`DatetimeIndex`
```

See Also

```
-----
between_time : Select values between particular times of the day.
first : Select initial periods of time series based on a date offset.
last : Select final periods of time series based on a date offset.
DatetimeIndex.indexer_at_time : Get just the index locations for
    values at particular time of the day.
```

Examples

```
-----
>>> i = pd.date_range("2018-04-09", periods=4, freq="12h")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts
```

```

2018-04-09 00:00:00 1
2018-04-09 12:00:00 2
2018-04-10 00:00:00 3
2018-04-10 12:00:00 4

```

```

>>> ts.at_time("12:00")
A
2018-04-09 12:00:00 2
2018-04-10 12:00:00 4

```

```

between_time(
    self,
    start_time,
    end_time,
    inclusive: IntervalClosedType = 'both',
    axis: Axis | None = None
) -> Self
    Select values between particular times of the day (e.g., 9:00-9:30 AM).

```

By setting ``start\_time`` to be later than ``end\_time``, you can get the times that are *not* between the two times.

Parameters

```

-----
start_time : datetime.time or str
    Initial time as a time filter limit.
end_time : datetime.time or str
    End time as a time filter limit.
inclusive : {"both", "neither", "left", "right"}, default "both"
    Include boundaries; whether to set each bound as closed or open.
axis : {0 or 'index', 1 or 'columns'}, default 0
    Determine range time on index or columns value.
    For `Series` this parameter is unused and defaults to 0.

```

Returns

```

-----
Series or DataFrame
    Data from the original object filtered to the specified dates range.

```

Raises

```

-----
TypeError
    If the index is not a :class:`DatetimeIndex`

```

See Also

```

-----
at_time : Select values at a particular time of the day.
first : Select initial periods of time series based on a date offset.
last : Select final periods of time series based on a date offset.
DatetimeIndex.indexer_between_time : Get just the index locations for
    values between particular times of the day.

```

Examples

```

>>> i = pd.date_range("2018-04-09", periods=4, freq="1D20min")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts

```

```

A
2018-04-09 00:00:00 1
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3
2018-04-12 01:00:00 4

```

```

>>> ts.between_time("0:15", "0:45")
A
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3

```

You get the times that are *not* between two times by setting ``start\_time`` later than ``end\_time``:

```

>>> ts.between_time("0:45", "0:15")
A
2018-04-09 00:00:00 1
2018-04-12 01:00:00 4

```

```

bfill(

```

```

self,
*,
axis: None | Axis = None,
inplace: bool = False,
limit: None | int = None,
limit_area: Literal['inside', 'outside'] | None = None
) -> Self
    Fill NA/Nan values by using the next valid observation to fill the gap.

    This method fills missing values in a backward direction along the
    specified axis, propagating non-null values from later positions to
    earlier positions containing NaN.

    Parameters
    -----
    axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame
        Axis along which to fill missing values. For `Series`
        this parameter is unused and defaults to 0.
    inplace : bool, default False
        If True, fill in-place. Note: this will modify any
        other views on this object (e.g., a no-copy slice for a column in a
        DataFrame).
    limit : int, default None
        If method is specified, this is the maximum number of consecutive
        NaN values to forward/backward fill. In other words, if there is
        a gap with more than this number of consecutive NaNs, it will only
        be partially filled. If method is not specified, this is the
        maximum number of entries along the entire axis where NaNs will be
        filled. Must be greater than 0 if not None.
    limit_area : {'None', 'inside', 'outside'}, default None
        If limit is specified, consecutive NaNs will be filled with this
        restriction.

        * ``None``: No fill restriction.
        * 'inside': Only fill NaNs surrounded by valid values
          (interpolate).
        * 'outside': Only fill NaNs outside valid values (extrapolate).

    .. versionadded:: 2.2.0

    Returns
    -----
    Series/DataFrame
        Object with missing values filled.

    See Also
    -----
    DataFrame.bfill : Fill NA/Nan values by propagating the last valid
        observation to next valid.

    Examples
    -----
    For Series:

    >>> s = pd.Series([1, None, None, 2])
    >>> s.bfill()
    0    1.0
    1    2.0
    2    2.0
    3    2.0
    dtype: float64
    >>> s.bfill(limit=1)
    0    1.0
    1    NaN
    2    2.0
    3    2.0
    dtype: float64

    With DataFrame:

    >>> df = pd.DataFrame({"A": [1, None, None, 4], "B": [None, 5, None, 7]})
    >>> df
       A    B
    0  1.0  NaN
    1  NaN  5.0
    2  NaN  NaN
    3  4.0  7.0

```

```
>>> df.bfill()
   A    B
0  1.0  5.0
1  4.0  5.0
2  4.0  7.0
3  4.0  7.0
>>> df.bfill(limit=1)
   A    B
0  1.0  5.0
1  NaN  5.0
2  4.0  7.0
3  4.0  7.0
```

```
clip(
    self,
    lower=None,
    upper=None,
    *,
    axis: Axis | None = None,
    inplace: bool = False,
    **kwargs
) -> Self
    Trim values at input threshold(s).
```

Assigns values outside boundary to boundary values. Thresholds can be singular values or array like, and in the latter case the clipping is performed element-wise in the specified axis.

Parameters

```
lower : float or array-like, default None
    Minimum threshold value. All values below this
    threshold will be set to it. A missing
    threshold (e.g. `NA`) will not clip the value.
upper : float or array-like, default None
    Maximum threshold value. All values above this
    threshold will be set to it. A missing
    threshold (e.g. `NA`) will not clip the value.
axis : {{0 or 'index', 1 or 'columns', None}}, default None
    Align object with lower and upper along the given axis.
    For `Series` this parameter is unused and defaults to `None`.
inplace : bool, default False
    Whether to perform the operation in place on the data.
**kwargs
    Additional keywords have no effect but might be accepted
    for compatibility with numpy.
```

Returns

```
Series or DataFrame
    Same type as calling object with the values outside the
    clip boundaries replaced.
```

See Also

```
Series.clip : Trim values at input threshold in series.
DataFrame.clip : Trim values at input threshold in DataFrame.
numpy.clip : Clip (limit) the values in an array.
```

Examples

```
>>> data = {"col_0": [9, -3, 0, -1, 5], "col_1": [-2, -7, 6, 8, -5]}
>>> df = pd.DataFrame(data)
>>> df
   col_0  col_1
0      9     -2
1     -3     -7
2      0      6
3     -1      8
4      5     -5
```

Clips per column using lower and upper thresholds:

```
>>> df.clip(-4, 6)
   col_0  col_1
0      6     -2
1     -3     -4
```

```

2      0      6
3     -1      6
4      5     -4

```

Clips using specific lower and upper thresholds per column:

```

>>> df.clip([-2, -1], [4, 5])
   col_0  col_1
0      4     -1
1     -2     -1
2      0      5
3     -1      5
4      4     -1

```

Clips using specific lower and upper thresholds per column element:

```

>>> t = pd.Series([2, -4, -1, 6, 3])
>>> t
0      2
1     -4
2     -1
3      6
4      3
dtype: int64

```

```

>>> df.clip(t, t + 4, axis=0)
   col_0  col_1
0      6      2
1     -3     -4
2      0      3
3      6      8
4      5      3

```

Clips using specific lower threshold per column element, with missing values:

```

>>> t = pd.Series([2, -4, np.nan, 6, 3])
>>> t
0      2.0
1     -4.0
2      NaN
3      6.0
4      3.0
dtype: float64

```

```

>>> df.clip(t, axis=0)
   col_0  col_1
0      9.0    2.0
1     -3.0   -4.0
2      0.0    6.0
3      6.0    8.0
4      5.0    3.0

```

```

convert_dtypes(
    self,
    infer_objects: bool = True,
    convert_string: bool = True,
    convert_integer: bool = True,
    convert_boolean: bool = True,
    convert_floating: bool = True,
    dtype_backend: DtypeBackend = 'numpy_nullable'
) -> Self
    Convert columns from numpy dtypes to the best dtypes that support ``pd.NA``.

Parameters
-----
infer_objects : bool, default True
    Whether object dtypes should be converted to the best possible types.
convert_string : bool, default True
    Whether object dtypes should be converted to ``StringDtype()``.
convert_integer : bool, default True
    Whether, if possible, conversion can be done to integer extension types.
convert_boolean : bool, defaults True
    Whether object dtypes should be converted to ``BooleanDtypes()``.
convert_floating : bool, defaults True
    Whether, if possible, conversion can be done to floating extension types.
    If ``convert_integer`` is also True, preference will be give to integer
    dtypes if the floats can be faithfully casted to integers.

```


`dtype_backend` : {'numpy_nullable', 'pyarrow'}, default 'numpy_nullable'
Back-end data type applied to the resultant `:class:`DataFrame`` or `:class:`Series`` (still experimental). Behaviour is as follows:

- * ```"numpy_nullable"```: returns nullable-dtype-backed `:class:`DataFrame`` or `:class:`Series``.
- * ```"pyarrow"```: returns pyarrow-backed nullable `:class:`ArrowDtype`` `:class:`DataFrame`` or `:class:`Series``.

.. versionadded:: 2.0

Returns

Series or DataFrame
Copy of input object with new dtype.

See Also

`infer_objects` : Infer dtypes of objects.
`to_datetime` : Convert argument to datetime.
`to_timedelta` : Convert argument to timedelta.
`to_numeric` : Convert argument to a numeric type.

Notes

By default, ```convert_dtypes``` will attempt to convert a Series (or each Series in a DataFrame) to dtypes that support ```pd.NA```. By using the options ```convert_string```, ```convert_integer```, ```convert_boolean``` and ```convert_floating```, it is possible to turn off individual conversions to ```StringDtype```, the integer extension types, ```BooleanDtype``` or floating extension types, respectively.

For object-dtyped columns, if ```infer_objects``` is ```True```, use the inference rules as during normal Series/DataFrame construction. Then, if possible, convert to ```StringDtype```, ```BooleanDtype``` or an appropriate integer or floating extension type, otherwise leave as ```object```.

If the dtype is integer, convert to an appropriate integer extension type.

If the dtype is numeric, and consists of all integers, convert to an appropriate integer extension type. Otherwise, convert to an appropriate floating extension type.

In the future, as new dtypes are added that support ```pd.NA```, the results of this method will change to support those new dtypes.

Examples

```
>>> df = pd.DataFrame(  
...     {  
...         "a": pd.Series([1, 2, 3], dtype=np.dtype("int32")),  
...         "b": pd.Series(["x", "y", "z"], dtype=np.dtype("O")),  
...         "c": pd.Series([True, False, np.nan], dtype=np.dtype("O")),  
...         "d": pd.Series(["h", "i", np.nan], dtype=np.dtype("O")),  
...         "e": pd.Series([10, np.nan, 20], dtype=np.dtype("float")),  
...         "f": pd.Series([np.nan, 100.5, 200], dtype=np.dtype("float")),  
...     }  
... )
```

Start with a DataFrame with default dtypes.

```
>>> df  
   a  b    c    d    e    f  
0  1  x  True    h  10.0  NaN  
1  2  y False    i   NaN 100.5  
2  3  z   NaN   NaN 20.0 200.0
```

```
>>> df.dtypes  
a      int32  
b      object  
c      object  
d      object  
e     float64  
f     float64  
dtype: object
```

Convert the DataFrame to use best possible dtypes.

```
>>> dfn = df.convert_dtypes()
>>> dfn
   a  b      c      d      e      f
0  1  x   True      h    10  <NA>
1  2  y  False      i  <NA>  100.5
2  3  z   <NA>  <NA>    20  200.0
```

```
>>> dfn.dtypes
a      Int32
b      string
c      boolean
d      string
e      Int64
f      Float64
dtype: object
```

Start with a Series of strings and missing data represented by ``np.nan``.

```
>>> s = pd.Series(["a", "b", np.nan])
>>> s
0      a
1      b
2     NaN
dtype: str
```

Obtain a Series with dtype ``StringDtype``.

```
>>> s.convert_dtypes()
0      a
1      b
2     <NA>
dtype: string
```

`copy(self, deep: bool = True) -> Self`
Make a copy of this object's indices and data.

When ``deep=True`` (default), a new object will be created with a copy of the calling object's data and indices. Modifications to the data or indices of the copy will not be reflected in the original object (see notes below).

When ``deep=False``, a new object will be created without copying the calling object's data or index (only references to the data and index are copied). With Copy-on-Write, changes to the original will *not* be reflected in the shallow copy (and vice versa). The shallow copy uses a lazy (deferred) copy mechanism that copies the data only when any changes to the original or shallow copy are made, ensuring memory efficiency while maintaining data integrity.

.. note::
In pandas versions prior to 3.0, the default behavior without Copy-on-Write was different: changes to the original *were* reflected in the shallow copy (and vice versa). See the :ref:`Copy-on-Write user guide <copy\_on\_write>` for more information.

Parameters

deep : bool, default True
Make a deep copy, including a copy of the data and the indices.
With ``deep=False`` neither the indices nor the data are copied.

Returns

Series or DataFrame
Object type matches caller.

See Also

copy.copy : Return a shallow copy of an object.
copy.deepcopy : Return a deep copy of an object.

Notes

When ``deep=True``, data is copied but actual Python objects will not be copied recursively, only the reference to the object. This is in contrast to `copy.deepcopy` in the Standard Library,

which recursively copies object data (see examples below).

While ``Index`` objects are copied when ``deep=True``, the underlying numpy array is not copied for performance reasons. Since ``Index`` is immutable, the underlying data can be safely shared and a copy is not needed.

Since pandas is not thread safe, see the :ref:`gotchas <gotchas.thread-safety>` when copying in a threading environment.

Copy-on-Write protects shallow copies against accidental modifications. This means that any changes to the copied data would make a new copy of the data upon write (and vice versa). Changes made to either the original or copied variable would not be reflected in the counterpart. See :ref:`Copy\_on\_Write <copy\_on\_write>` for more information.

Examples

```
>>> s = pd.Series([1, 2], index=["a", "b"])
>>> s
a    1
b    2
dtype: int64
```

```
>>> s_copy = s.copy(deep=True)
>>> s_copy
a    1
b    2
dtype: int64
```

Due to Copy-on-Write, shallow copies still protect data modifications. Note shallow does not get modified below.

```
>>> s = pd.Series([1, 2], index=["a", "b"])
>>> shallow = s.copy(deep=False)
>>> s.iloc[1] = 200
>>> shallow
a    1
b    2
dtype: int64
```

When the data has object dtype, even a deep copy does not copy the underlying Python objects. Updating a nested data object will be reflected in the deep copy.

```
>>> s = pd.Series([[1, 2], [3, 4]])
>>> deep = s.copy()
>>> s[0][0] = 10
>>> s
0    [10, 2]
1     [3, 4]
dtype: object
>>> deep
0    [10, 2]
1     [3, 4]
dtype: object
```

describe(self, percentiles=None, include=None, exclude=None) -> Self
Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding ``NaN`` values.

Analyzes both numeric and object series, as well as ``DataFrame`` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

Parameters

percentiles : list-like of numbers, optional
The percentiles to include in the output. All should fall between 0 and 1. The default, ``None``, will automatically return the 25th, 50th, and 75th percentiles.
include : 'all', list-like of dtypes or None (default), optional

A white list of data types to include in the result. Ignored for ``Series``. Here are the options:

- 'all' : All columns of the input will be included in the output.
 - A list-like of dtypes : Limits the results to the provided data types.
To limit the result to numeric types submit ``numpy.number``. To limit it instead to object columns submit the ``numpy.object`` data type. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(include=['O'])``). To select pandas categorical columns, use ``category``.
 - None (default) : The result will include all numeric columns.
- exclude : list-like of dtypes or None (default), optional,
A black list of data types to omit from the result. Ignored for ``Series``. Here are the options:
- A list-like of dtypes : Excludes the provided data types from the result. To exclude numeric types submit ``numpy.number``. To exclude object columns submit the data type ``numpy.object``. Strings can also be used in the style of ``select\_dtypes`` (e.g. ``df.describe(exclude=['O'])``). To exclude pandas categorical columns, use ``category``.
 - None (default) : The result will exclude nothing.

Returns

Series or DataFrame

Summary statistics of the Series or DataFrame provided.

See Also

DataFrame.count: Count number of non-NA/null observations.

DataFrame.max: Maximum of the values in the object.

DataFrame.min: Minimum of the values in the object.

DataFrame.mean: Mean of the values.

DataFrame.std: Standard deviation of the observations.

DataFrame.select_dtypes: Subset of a DataFrame including/excluding columns based on their dtype.

Notes

For numeric data, the result's index will include ``count``, ``mean``, ``std``, ``min``, ``max`` as well as lower, ``50`` and upper percentiles. By default the lower percentile is ``25`` and the upper percentile is ``75``. The ``50`` percentile is the same as the median.

For object data (e.g. strings), the result's index will include ``count``, ``unique``, ``top``, and ``freq``. The ``top`` is the most common value. The ``freq`` is the most common value's frequency.

If multiple object values have the highest count, then the ``count`` and ``top`` results will be arbitrarily chosen from among those with the highest count.

For mixed data types provided via a ``DataFrame``, the default is to return only an analysis of numeric columns. If the DataFrame consists only of object and categorical data without any numeric columns, the default is to return an analysis of both the object and categorical columns. If ``include='all'`` is provided as an option, the result will include a union of attributes of each type.

The ``include`` and ``exclude`` parameters can be used to limit which columns in a ``DataFrame`` are analyzed for the output. The parameters are ignored when analyzing a ``Series``.

Examples

Describing a numeric ``Series``.

```
>>> s = pd.Series([1, 2, 3])
>>> s.describe()
count      3.0
mean       2.0
std        1.0
```

```
min      1.0
25%      1.5
50%      2.0
75%      2.5
max      3.0
dtype: float64
```

Describing a categorical ``Series``.

```
>>> s = pd.Series(["a", "a", "b", "c"])
>>> s.describe()
count      4
unique      3
top         a
freq        2
dtype: object
```

Describing a timestamp ``Series``.

```
>>> s = pd.Series(
...     [
...         np.datetime64("2000-01-01"),
...         np.datetime64("2010-01-01"),
...         np.datetime64("2010-01-01"),
...     ]
... )
>>> s.describe()
count      3
mean      2006-09-01 08:00:00
min       2000-01-01 00:00:00
25%       2004-12-31 12:00:00
50%       2010-01-01 00:00:00
75%       2010-01-01 00:00:00
max       2010-01-01 00:00:00
dtype: object
```

Describing a ``DataFrame``. By default only numeric fields are returned.

```
>>> df = pd.DataFrame(
...     {
...         "categorical": pd.Categorical(["d", "e", "f"]),
...         "numeric": [1, 2, 3],
...         "object": ["a", "b", "c"],
...     }
... )
>>> df.describe()
           numeric
count          3.0
mean           2.0
std            1.0
min            1.0
25%            1.5
50%            2.0
75%            2.5
max            3.0
```

Describing all columns of a ``DataFrame`` regardless of data type.

```
>>> df.describe(include="all") # doctest: +SKIP
           categorical  numeric  object
count              3         3.0      3
unique             3         NaN      3
top                f         NaN      a
freq              1         NaN      1
mean              NaN         2.0     NaN
std              NaN         1.0     NaN
min              NaN         1.0     NaN
25%              NaN         1.5     NaN
50%              NaN         2.0     NaN
75%              NaN         2.5     NaN
max              NaN         3.0     NaN
```

Describing a column from a ``DataFrame`` by accessing it as an attribute.

```
>>> df.numeric.describe()
```

```

count      3.0
mean       2.0
std        1.0
min        1.0
25%        1.5
50%        2.0
75%        2.5
max        3.0
Name: numeric, dtype: float64

```

Including only numeric columns in a ``DataFrame`` description.

```

>>> df.describe(include=[np.number])
      numeric
count      3.0
mean       2.0
std        1.0
min        1.0
25%        1.5
50%        2.0
75%        2.5
max        3.0

```

Including only string columns in a ``DataFrame`` description.

```

>>> df.describe(include=[object]) # doctest: +SKIP
      object
count      3
unique     3
top        a
freq       1

```

Including only categorical columns from a ``DataFrame`` description.

```

>>> df.describe(include=["category"])
      categorical
count          3
unique         3
top            d
freq           1

```

Excluding numeric columns from a ``DataFrame`` description.

```

>>> df.describe(exclude=[np.number]) # doctest: +SKIP
      categorical object
count          3      3
unique         3      3
top            f      a
freq           1      1

```

Excluding object columns from a ``DataFrame`` description.

```

>>> df.describe(exclude=[object]) # doctest: +SKIP
      categorical  numeric
count          3      3.0
unique         3      NaN
top            f      NaN
freq           1      NaN
mean          NaN      2.0
std           NaN      1.0
min           NaN      1.0
25%           NaN      1.5
50%           NaN      2.0
75%           NaN      2.5
max           NaN      3.0

```

`droplevel(self, level: IndexLabel, axis: Axis = 0) -> Self`
 Return Series/DataFrame with requested index / column level(s) removed.

Parameters

 level : int, str, or list-like
 If a string is given, must be the name of a level
 If list-like, elements must be names or positional indexes
 of levels.

axis : {{0 or 'index', 1 or 'columns'}}, default 0

Axis along which the level(s) is removed:

- * 0 or 'index': remove level(s) in column.
- * 1 or 'columns': remove level(s) in row.

For `Series` this parameter is unused and defaults to 0.

Returns

Series/DataFrame

Series/DataFrame with requested index / column level(s) removed.

See Also

`DataFrame.replace` : Replace values given in `to\_replace` with `value`.
`DataFrame.pivot` : Return reshaped DataFrame organized by given index / column values.

Examples

```
>>> df = (  
...     pd.DataFrame([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])  
...     .set_index([0, 1])  
...     .rename_axis(["a", "b"])  
... )
```

```
>>> df.columns = pd.MultiIndex.from_tuples(  
...     [("c", "e"), ("d", "f")], names=["level_1", "level_2"]  
... )
```

```
>>> df  
level_1  c  d  
level_2  e  f  
a b  
1 2      3  4  
5 6      7  8  
9 10     11 12
```

```
>>> df.droplevel("a")  
level_1  c  d  
level_2  e  f  
b  
2      3  4  
6      7  8  
10     11 12
```

```
>>> df.droplevel("level_2", axis=1)  
level_1  c  d  
a b  
1 2      3  4  
5 6      7  8  
9 10     11 12
```

`equals(self, other: object) -> bool`

Test whether two objects contain the same elements.

This function allows two Series or DataFrames to be compared against each other to see if they have the same shape and elements. NaNs in the same location are considered equal.

The row/column index do not need to have the same type, as long as the values are considered equal. Corresponding columns and index must be of the same dtype.

Parameters

other : Series or DataFrame

The other Series or DataFrame to be compared with the first.

Returns

bool

True if all elements are the same in both objects, False otherwise.

See Also

Series.eq : Compare two Series objects of the same length and return a Series where each element is True if the element in each Series is equal, False otherwise.

DataFrame.eq : Compare two DataFrame objects of the same shape and return a DataFrame where each element is True if the respective element in each DataFrame is equal, False otherwise.

testing.assert_series_equal : Raises an AssertionError if left and right are not equal. Provides an easy interface to ignore inequality in dtypes, indexes and precision among others.

testing.assert_frame_equal : Like assert_series_equal, but targets DataFrames.

numpy.array_equal : Return True if two arrays have the same shape and elements, False otherwise.

Examples

```

>>> df = pd.DataFrame({1: [10], 2: [20]})
>>> df
   1  2
0 10 20

```

DataFrames df and exactly_equal have the same types and values for their elements and column labels, which will return True.

```

>>> exactly_equal = pd.DataFrame({1: [10], 2: [20]})
>>> exactly_equal
   1  2
0 10 20
>>> df.equals(exactly_equal)
True

```

DataFrames df and different_column_type have the same element types and values, but have different types for the column labels, which will still return True.

```

>>> different_column_type = pd.DataFrame({1.0: [10], 2.0: [20]})
>>> different_column_type
   1.0  2.0
0   10   20
>>> df.equals(different_column_type)
True

```

DataFrames df and different_data_type have different types for the same values for their elements, and will return False even though their column labels are the same values and types.

```

>>> different_data_type = pd.DataFrame({1: [10.0], 2: [20.0]})
>>> different_data_type
   1  2
0 10.0 20.0
>>> df.equals(different_data_type)
False

```

DataFrames with NaN in the same locations compare equal.

```

>>> df_nan1 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
>>> df_nan2 = pd.DataFrame({"a": [1, np.nan], "b": [3, np.nan]})
>>> df_nan1.equals(df_nan2)
True

```

If the NaN values are not in the same locations, they compare unequal.

```

>>> df_nan3 = pd.DataFrame({"a": [1, np.nan], "b": [3, 4]})
>>> df_nan1.equals(df_nan3)
False

```

```

ewm(
    self,
    com: float | None = None,
    span: float | None = None,
    halflife: float | TimedeltaConvertibleTypes | None = None,
    alpha: float | None = None,
    min_periods: int | None = 0,
    adjust: bool = True,
    ignore_na: bool = False,
    times: np.ndarray | DataFrame | Series | None = None,
    method: Literal['single', 'table'] = 'single'
)

```



```

) -> ExponentialMovingWindow
Provide exponentially weighted (EW) calculations.

Exactly one of ``com``, ``span``, ``halflife``, or ``alpha`` must be
provided if ``times`` is not provided. If ``times`` is provided and ``adjust=True``,
``halflife`` and one of ``com``, ``span`` or ``alpha`` may be provided.
If ``times`` is provided and ``adjust=False``, ``halflife`` must be the only
provided decay-specification parameter.

Parameters
-----
com : float, optional
    Specify decay in terms of center of mass

    :math:\alpha = 1 / (1 + com), for :math:com \geq 0.

span : float, optional
    Specify decay in terms of span

    :math:\alpha = 2 / (span + 1), for :math:span \geq 1.

halflife : float, str, timedelta, optional
    Specify decay in terms of half-life

    :math:\alpha = 1 - \exp(-\ln(2) / halflife), for
    :math:halflife > 0.

    If ``times`` is specified, a timedelta convertible unit over which an
    observation decays to half its value. Only applicable to ``mean()``,
    and halflife value will not apply to the other functions.

alpha : float, optional
    Specify smoothing factor :math:\alpha directly

    :math:0 < \alpha \leq 1.

min_periods : int, default 0
    Minimum number of observations in window required to have a value;
    otherwise, result is ``np.nan``.

adjust : bool, default True
    Divide by decaying adjustment factor in beginning periods to account
    for imbalance in relative weightings (viewing EWMA as a moving average).

    - When ``adjust=True`` (default), the EW function is calculated using weights
      :math:w_i = (1 - \alpha)^i. For example, the EW moving average of the series
      [:math:x_0, x_1, \dots, x_t] would be:

      .. math::
         y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots + (1 - \alpha)^t x_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}

    - When ``adjust=False``, the exponentially weighted function is calculated
      recursively:

      .. math::
         \begin{split}
         y_0 &= x_0 \\
         y_t &= (1 - \alpha) y_{t-1} + \alpha x_t,
         \end{split}

ignore_na : bool, default False
    Ignore missing values when calculating weights.

    - When ``ignore_na=False`` (default), weights are based on absolute positions.
      For example, the weights of :math:x_0 and :math:x_2 used in calculating
      the final weighted average of [:math>x_0, None, x_2] are
      :math:(1-\alpha)^2 and :math:1 if ``adjust=True``, and
      :math:(1-\alpha)^2 and :math:\alpha if ``adjust=False``.

    - When ``ignore_na=True``, weights are based
      on relative positions. For example, the weights of :math>x_0 and :math>x_2
      used in calculating the final weighted average of
      [:math>x_0, None, x_2] are :math>1-\alpha and :math:1 if
      ``adjust=True``, and :math>1-\alpha and :math:\alpha if ``adjust=False``.

times : np.ndarray, Series, default None

```

Only applicable to ``mean()``.

Times corresponding to the observations. Must be monotonically increasing and ``datetime64[ns]`` dtype.

If 1-D array like, a sequence with the same shape as the observations.

method : str {'single', 'table'}, default 'single'
Execute the rolling operation per single column or row (``'single'``)
or over the entire object (``'table'``).

This argument is only implemented when specifying ``engine='numba'``
in the method call.

Only applicable to ``mean()``

Returns

pandas.api.typing.ExponentialMovingWindow
An instance of ExponentialMovingWindow for further exponentially weighted (EW)
calculations, e.g. using the ``mean`` method.

See Also

rolling : Provides rolling window calculations.
expanding : Provides expanding transformations.

Notes

See :ref:`Windowing Operations <window.exponentially\_weighted>`
for further usage details and examples.

Examples

>>> df = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]})

>>> df

```
   B
0  0.0
1  1.0
2  2.0
3  NaN
4  4.0
```

>>> df.ewm(com=0.5).mean()

```
   B
0  0.000000
1  0.750000
2  1.615385
3  1.615385
4  3.670213
```

>>> df.ewm(alpha=2 / 3).mean()

```
   B
0  0.000000
1  0.750000
2  1.615385
3  1.615385
4  3.670213
```

adjust

>>> df.ewm(com=0.5, adjust=True).mean()

```
   B
0  0.000000
1  0.750000
2  1.615385
3  1.615385
4  3.670213
```

>>> df.ewm(com=0.5, adjust=False).mean()

```
   B
0  0.000000
1  0.666667
2  1.555556
3  1.555556
4  3.650794
```

ignore_na

```
>>> df.ewm(com=0.5, ignore_na=True).mean()
      B
0  0.000000
1  0.750000
2  1.615385
3  1.615385
4  3.225000
>>> df.ewm(com=0.5, ignore_na=False).mean()
      B
0  0.000000
1  0.750000
2  1.615385
3  1.615385
4  3.670213

**times**

Exponentially weighted mean with weights calculated with a timedelta ``halflife``
relative to ``times``.

>>> times = ['2020-01-01', '2020-01-03', '2020-01-10', '2020-01-15', '2020-01-17']
>>> df.ewm(halflife='4 days', times=pd.DatetimeIndex(times)).mean()
      B
0  0.000000
1  0.585786
2  1.523889
3  1.523889
4  3.233686
```

```
expanding(
    self,
    min_periods: int = 1,
    method: Literal['single', 'table'] = 'single'
) -> Expanding
    Provide expanding window calculations.
```

An expanding window yields the value of an aggregation statistic with all the data available up to that point in time.

Parameters

```
min_periods : int, default 1
    Minimum number of observations in window required to have a value;
    otherwise, result is ``np.nan``.

method : str {'single', 'table'}, default 'single'
    Execute the rolling operation per single column or row (``'single'``)
    or over the entire object (``'table'``).

    This argument is only implemented when specifying ``engine='numba'``
    in the method call.
```

Returns

```
pandas.api.typing.Expanding
    An instance of Expanding for further expanding window calculations,
    e.g. using the ``sum`` method.
```

See Also

```
rolling : Provides rolling window calculations.
ewm : Provides exponential weighted functions.
```

Notes

See :ref:`Windowing Operations <window.expanding>` for further usage details and examples.

Examples

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
>>> df
      B
0  0.0
1  1.0
2  2.0
3  NaN
```

```
4 4.0
```

```
**min_periods**
```

Expanding sum with 1 vs 3 observations needed to calculate a value.

```
>>> df.expanding(1).sum()
```

```
   B
0  0.0
1  1.0
2  3.0
3  3.0
4  7.0
```

```
>>> df.expanding(3).sum()
```

```
   B
0 NaN
1 NaN
2  3.0
3  3.0
4  7.0
```

```
ffill(
    self,
    *,
    axis: None | Axis = None,
    inplace: bool = False,
    limit: None | int = None,
    limit_area: Literal['inside', 'outside'] | None = None
) -> Self
    Fill NA/NAN values by propagating the last valid observation to next valid.

Parameters
-----
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame
    Axis along which to fill missing values. For `Series`
    this parameter is unused and defaults to 0.
inplace : bool, default False
    If True, fill in-place. Note: this will modify any
    other views on this object (e.g., a no-copy slice for a column in a
    DataFrame).
limit : int, default None
    If method is specified, this is the maximum number of consecutive
    NaN values to forward/backward fill. In other words, if there is
    a gap with more than this number of consecutive NaNs, it will only
    be partially filled. If method is not specified, this is the
    maximum number of entries along the entire axis where NaNs will be
    filled. Must be greater than 0 if not None.
limit_area : {'None', 'inside', 'outside'}, default None
    If limit is specified, consecutive NaNs will be filled with this
    restriction.

    * ``None``: No fill restriction.
    * 'inside': Only fill NaNs surrounded by valid values
    (interpolate).
    * 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0
```

Returns

```
Series/DataFrame
    Object with missing values filled.
```

See Also

```
DataFrame.bfill : Fill NA/NAN values by using the next valid observation
to fill the gap.
```

Examples

```
>>> df = pd.DataFrame(
...     [
...         [np.nan, 2, np.nan, 0],
...         [3, 4, np.nan, 1],
...         [np.nan, np.nan, np.nan, np.nan],
...         [np.nan, 3, np.nan, 4],
...     ],
```

```

...     columns=list("ABCD"),
... )
>>> df
   A    B    C    D
0 NaN  2.0 NaN  0.0
1 3.0  4.0 NaN  1.0
2 NaN  NaN NaN  NaN
3 NaN  3.0 NaN  4.0

>>> df.ffmpeg()
   A    B    C    D
0 NaN  2.0 NaN  0.0
1 3.0  4.0 NaN  1.0
2 3.0  4.0 NaN  1.0
3 3.0  3.0 NaN  4.0

>>> ser = pd.Series([1, np.nan, 2, 3])
>>> ser.ffmpeg()
0    1.0
1    1.0
2    2.0
3    3.0
dtype: float64

```

```

fillna(
    self,
    value: Hashable | Mapping | Series | DataFrame,
    *,
    axis: Axis | None = None,
    inplace: bool = False,
    limit: int | None = None
) -> Self
Fill NA/NaN values with `value`.

Parameters
-----
value : scalar, dict, Series, or DataFrame
    Value to use to fill holes (e.g. 0), alternately a
    dict/Series/DataFrame of values specifying which value to use for
    each index (for a Series) or column (for a DataFrame). Values not
    in the dict/Series/DataFrame will not be filled. This value cannot
    be a list.
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame
    Axis along which to fill missing values. For `Series`
    this parameter is unused and defaults to 0.
inplace : bool, default False
    If True, fill in-place. Note: this will modify any
    other views on this object (e.g., a no-copy slice for a column in a
    DataFrame).
limit : int, default None
    This is the maximum number of entries along the entire axis
    where NaNs will be filled. Must be greater than 0 if not None.

```

Returns

```

Series/DataFrame
    Object with missing values filled.

```

See Also

```

ffill : Fill values by propagating the last valid observation to next valid.
bfill : Fill values by using the next valid observation to fill the gap.
interpolate : Fill NaN values using interpolation.
reindex : Conform object to new index.
asfreq : Convert TimeSeries to specified frequency.

```

Notes

```

For non-object dtype, ``value=None`` will use the NA value of the dtype.
See more details in the :ref:`Filling missing data<missing_data.fillna>`
section.

```

Examples

```

>>> df = pd.DataFrame(
...     [
...         [np.nan, 2, np.nan, 0],

```

```

...         [3, 4, np.nan, 1],
...         [np.nan, np.nan, np.nan, np.nan],
...         [np.nan, 3, np.nan, 4],
...     ],
...     columns=list("ABCD"),
... )
>>> df

```

```

      A      B      C      D
0  NaN  2.0  NaN  0.0
1  3.0  4.0  NaN  1.0
2  NaN  NaN  NaN  NaN
3  NaN  3.0  NaN  4.0

```

Replace all NaN elements with 0s.

```

>>> df.fillna(0)
      A      B      C      D
0  0.0  2.0  0.0  0.0
1  3.0  4.0  0.0  1.0
2  0.0  0.0  0.0  0.0
3  0.0  3.0  0.0  4.0

```

Replace all NaN elements in column 'A', 'B', 'C', and 'D', with 0, 1, 2, and 3 respectively.

```

>>> values = {"A": 0, "B": 1, "C": 2, "D": 3}
>>> df.fillna(value=values)
      A      B      C      D
0  0.0  2.0  2.0  0.0
1  3.0  4.0  2.0  1.0
2  0.0  1.0  2.0  3.0
3  0.0  3.0  2.0  4.0

```

Only replace the first NaN element.

```

>>> df.fillna(value=values, limit=1)
      A      B      C      D
0  0.0  2.0  2.0  0.0
1  3.0  4.0  NaN  1.0
2  NaN  1.0  NaN  3.0
3  NaN  3.0  NaN  4.0

```

When filling using a DataFrame, replacement happens along the same column names and same indices

```

>>> df2 = pd.DataFrame(np.zeros((4, 4)), columns=list("ABCE"))
>>> df.fillna(df2)
      A      B      C      D
0  0.0  2.0  0.0  0.0
1  3.0  4.0  0.0  1.0
2  0.0  0.0  0.0  NaN
3  0.0  3.0  0.0  4.0

```

Note that column D is not affected since it is not present in df2.

```

filter(
    self,
    items=None,
    like: str | None = None,
    regex: str | None = None,
    axis: Axis | None = None
) -> Self

```

Subset the DataFrame or Series according to the specified index labels.

For DataFrame, filter rows or columns depending on ``axis`` argument.
Note that this routine does not filter based on content.
The filter is applied to the labels of the index.

Parameters

```

-----
items : list-like
    Keep labels from axis which are in items.
like : str
    Keep labels from axis for which "like in label == True".
regex : str (regular expression)
    Keep labels from axis for which re.search(regex, label) == True.
axis : {0 or 'index', 1 or 'columns', None}, default None

```

The axis to filter on, expressed either as an index (int) or axis name (str). By default this is the info axis, 'columns' for ``DataFrame``. For ``Series`` this parameter is unused and defaults to ``None``.

Returns

Same type as caller

The filtered subset of the DataFrame or Series.

See Also

`DataFrame.loc` : Access a group of rows and columns by label(s) or a boolean array.

Notes

The ``items``, ``like``, and ``regex`` parameters are enforced to be mutually exclusive.

``axis`` defaults to the info axis that is used when indexing with ``[]``.

Examples

```
>>> df = pd.DataFrame(
...     np.array([[1, 2, 3], [4, 5, 6]]),
...     index=["mouse", "rabbit"],
...     columns=["one", "two", "three"],
... )
>>> df
   one  two  three
mouse   1   2     3
rabbit   4   5     6

>>> # select columns by name
>>> df.filter(items=["one", "three"])
   one  three
mouse   1     3
rabbit   4     6

>>> # select columns by regular expression
>>> df.filter(regex="e$", axis=1)
   one  three
mouse   1     3
rabbit   4     6

>>> # select rows containing 'bbi'
>>> df.filter(like="bbi", axis=0)
   one  two  three
rabbit   4   5     6
```

`first_valid_index(self)` -> Hashable

Return index for first non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information on which values are considered missing.

Returns

type of index

Index of first non-missing value.

See Also

`DataFrame.last_valid_index` : Return index for last non-NA value or None, if no non-NA value is found.

`Series.last_valid_index` : Return index for last non-NA value or None, if no non-NA value is found.

`DataFrame.isna` : Detect missing values.

Examples

For Series:

```
>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
```

```

1
>>> s.last_valid_index()
2

>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

If all elements in Series are NA/null, returns None.

>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

If Series is empty, returns None.

For DataFrame:

>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
   A      B
0 NaN    NaN
1 NaN    3.0
2 2.0    4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2

>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
   A      B
0 None  None
1 None  None
2 None  None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None

If all elements in DataFrame are NA/null, returns None.

>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None

If DataFrame is empty, returns None.

```

get(self, key, default=None)
 Get item from object for given key (ex: DataFrame column).
 Returns ``default`` value if not found.

Parameters

 key : object
 Key for which item should be returned.
 default : object, default None
 Default value to return if key is not found.

Returns

 same type as items contained in object
 Item for given key or ``default`` value, if key is not found.

See Also

DataFrame.get : Get item from object for given key (ex: DataFrame column).
Series.get : Get item from object for given key (ex: DataFrame column).

Examples

```
>>> df = pd.DataFrame(  
...     [  
...         [24.3, 75.7, "high"],  
...         [31, 87.8, "high"],  
...         [22, 71.6, "medium"],  
...         [35, 95, "medium"],  
...     ],  
...     columns=["temp_celsius", "temp_fahrenheit", "windspeed"],  
...     index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),  
... )
```

```
>>> df  
                temp_celsius  temp_fahrenheit  windspeed  
2014-02-12             24.3             75.7         high  
2014-02-13             31.0             87.8         high  
2014-02-14             22.0             71.6        medium  
2014-02-15             35.0             95.0        medium
```

```
>>> df.get(["temp_celsius", "windspeed"])  
                temp_celsius  windspeed  
2014-02-12             24.3         high  
2014-02-13             31.0         high  
2014-02-14             22.0        medium  
2014-02-15             35.0        medium
```

```
>>> ser = df["windspeed"]  
>>> ser.get("2014-02-13")  
'high'
```

If the key isn't found, the default value will be used.

```
>>> df.get(["temp_celsius", "temp_kelvin"], default="default_value")  
'default_value'
```

```
>>> ser.get("2014-02-10", "[unknown]")  
'[unknown]'
```

head(self, n: int = 5) -> Self
Return the first `n` rows.

This function exhibits the same behavior as ``df[:n]``, returning the first ``n`` rows based on position. It is useful for quickly checking if your object has the right type of data in it.

When ``n`` is positive, it returns the first ``n`` rows. For ``n`` equal to 0, it returns an empty object. When ``n`` is negative, it returns all rows except the last ``|n|`` rows, mirroring the behavior of ``df[:n]``.

If ``n`` is larger than the number of rows, this function returns all rows.

Parameters

n : int, default 5
Number of rows to select.

Returns

same type as caller
The first `n` rows of the caller object.

See Also

DataFrame.tail: Returns the last `n` rows.

Examples

```
>>> df = pd.DataFrame(  
...     {  
...         "animal": [  
...             "alligator",  
...             "bee",  
...             "falcon",
```

```

...         "lion",
...         "monkey",
...         "parrot",
...         "shark",
...         "whale",
...         "zebra",
...     ]
... }
... )
>>> df
   animal
0  alligator
1      bee
2    falcon
3      lion
4    monkey
5    parrot
6      shark
7      whale
8      zebra

```

Viewing the first 5 lines

```

>>> df.head()
   animal
0  alligator
1      bee
2    falcon
3      lion
4    monkey

```

Viewing the first `n` lines (three in this case)

```

>>> df.head(3)
   animal
0  alligator
1      bee
2    falcon

```

For negative values of `n`

```

>>> df.head(-3)
   animal
0  alligator
1      bee
2    falcon
3      lion
4    monkey
5    parrot

```

`infer_objects(self, copy: bool | lib.NoDefault = <no_default>) -> Self`
 Attempt to infer better dtypes for object columns.

Attempts soft conversion of object-dtyped columns, leaving non-object and unconvertible columns unchanged. The inference rules are the same as during normal Series/DataFrame construction.

Parameters

`copy` : bool, default False
 This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

same type as input object
 Returns an object of the same type as the input object.

See Also

to_datetime : Convert argument to datetime.
to_timedelta : Convert argument to timedelta.
to_numeric : Convert argument to numeric type.
convert_dtypes : Convert argument to best possible dtype.

Examples

>>> df = pd.DataFrame({"A": ["a", 1, 2, 3]})
>>> df = df.iloc[1:]
>>> df
 A
1 1
2 2
3 3

>>> df.dtypes
A object
dtype: object

>>> df.infer_objects().dtypes
A int64
dtype: object

```
interpolate(  
    self,  
    method: InterpolateOptions = 'linear',  
    *,  
    axis: Axis = 0,  
    limit: int | None = None,  
    inplace: bool = False,  
    limit_direction: Literal['forward', 'backward', 'both'] | None = None,  
    limit_area: Literal['inside', 'outside'] | None = None,  
    **kwargs  
) -> Self  
    Fill NaN values using an interpolation method.
```

Please note that only ``method='linear'`` is supported for DataFrame/Series with a MultiIndex.

Parameters

method : str, default 'linear'

Interpolation technique to use. One of:

- * 'linear': Ignore the index and treat the values as equally spaced. This is the only method supported on MultiIndexes.
- * 'time': Works on daily and higher resolution data to interpolate given length of interval. This interpolates values based on time interval between observations.
- * 'index': The interpolation uses the numerical values of the DataFrame's index to linearly calculate missing values.
- * 'values': Interpolation based on the numerical values in the DataFrame, treating them as equally spaced along the index.
- * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric', 'polynomial': Passed to `scipy.interpolate.interp1d`, whereas 'spline' is passed to `scipy.interpolate.UnivariateSpline`. These methods use the numerical values of the index. Both 'polynomial' and 'spline' require that you also specify an `order` (int), e.g. `df.interpolate(method='polynomial', order=5)`. Note that, `slinear` method in Pandas refers to the SciPy first order `spline` instead of Pandas first order `spline`.
- * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima', 'cubicspline': Wrappers around the SciPy interpolation methods of similar names. See `Notes`.
- * 'from_derivatives': Refers to `scipy.interpolate.BPoly.from\_derivatives`.

axis : {{0 or 'index', 1 or 'columns', None}}, default None
Axis to interpolate along. For `Series` this parameter is unused and defaults to 0.

limit : int, optional
Maximum number of consecutive NaNs to fill. Must be greater than 0.

```

inplace : bool, default False
    Update the data in place if possible.
limit_direction : {'forward', 'backward', 'both'}, optional, default 'forward'
    Consecutive NaNs will be filled in this direction.

limit_area : {'None', 'inside', 'outside'}, default None
    If limit is specified, consecutive NaNs will be filled with this
    restriction.

    * ``None``: No fill restriction.
    * 'inside': Only fill NaNs surrounded by valid values
      (interpolate).
    * 'outside': Only fill NaNs outside valid values (extrapolate).

**kwargs : optional
    Keyword arguments to pass on to the interpolating function.

```

Returns

```

-----
Series or DataFrame
    Returns the same object type as the caller, interpolated at
    some or all ``NaN`` values.

```

See Also

```

-----
fillna : Fill missing values using different methods.
scipy.interpolate.Akima1DInterpolator : Piecewise cubic polynomials
    (Akima interpolator).
scipy.interpolate.BPoly.from_derivatives : Piecewise polynomial in the
    Bernstein basis.
scipy.interpolate.interp1d : Interpolate a 1-D function.
scipy.interpolate.KroghInterpolator : Interpolate polynomial (Krogh
    interpolator).
scipy.interpolate.PchipInterpolator : PCHIP 1-d monotonic cubic
    interpolation.
scipy.interpolate.CubicSpline : Cubic spline data interpolator.

```

Notes

```

-----
The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima'
methods are wrappers around the respective SciPy implementations of
similar names. These use the actual numerical values of the index.
For more information on their behavior, see the
`SciPy documentation
<https://docs.scipy.org/doc/scipy/reference/interpolate.html#univariate-interpolation>`

```

Examples

```

-----
Filling in ``NaN`` in a :class:`~pandas.Series` via linear
interpolation.

```

```

>>> s = pd.Series([0, 1, np.nan, 3])
>>> s
0    0.0
1    1.0
2    NaN
3    3.0
dtype: float64
>>> s.interpolate()
0    0.0
1    1.0
2    2.0
3    3.0
dtype: float64

```

```

Filling in ``NaN`` in a Series via polynomial interpolation or splines:
Both 'polynomial' and 'spline' methods require that you also specify
an ``order`` (int).

```

```

>>> s = pd.Series([0, 2, np.nan, 8])
>>> s.interpolate(method="polynomial", order=2)
0    0.000000
1    2.000000
2    4.666667
3    8.000000
dtype: float64

```

Fill the DataFrame forward (that is, going down) along each column using linear interpolation.

Note how the last entry in column 'a' is interpolated differently, because there is no entry after it to use for interpolation. Note how the first entry in column 'b' remains ``NaN``, because there is no entry before it to use for interpolation.

```
>>> df = pd.DataFrame(
...     [
...         (0.0, np.nan, -1.0, 1.0),
...         (np.nan, 2.0, np.nan, np.nan),
...         (2.0, 3.0, np.nan, 9.0),
...         (np.nan, 4.0, -4.0, 16.0),
...     ],
...     columns=list("abcd"),
... )
>>> df
   a    b    c    d
0  0.0 NaN -1.0  1.0
1  NaN  2.0 NaN  NaN
2  2.0  3.0 NaN  9.0
3  NaN  4.0 -4.0 16.0
>>> df.interpolate(method="linear", limit_direction="forward", axis=0)
   a    b    c    d
0  0.0 NaN -1.0  1.0
1  1.0  2.0 -2.0  5.0
2  2.0  3.0 -3.0  9.0
3  2.0  4.0 -4.0 16.0
```

Using polynomial interpolation.

```
>>> df["d"].interpolate(method="polynomial", order=2)
0    1.0
1    4.0
2    9.0
3   16.0
Name: d, dtype: float64
```

keys(self) -> Index

Get the 'info axis' (see Indexing for more).

This is index for Series, columns for DataFrame.

Returns

Index

Info axis.

See Also

DataFrame.index : The index (row labels) of the DataFrame.

DataFrame.columns: The column labels of the DataFrame.

Examples

```
>>> d = pd.DataFrame(
...     data={"A": [1, 2, 3], "B": [0, 4, 8]}, index=["a", "b", "c"]
... )
>>> d
   A  B
a  1  0
b  2  4
c  3  8
>>> d.keys()
Index(['A', 'B'], dtype='str')
```

last_valid_index(self) -> Hashable

Return index for last non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information on which values are considered missing.

Returns

type of index

Index of last non-missing value.

See Also

DataFrame.first_valid_index : Return index for first non-NA value or None, if no non-NA value is found.

Series.first_valid_index : Return index for first non-NA value or None, if no non-NA value is found.

DataFrame.isna : Detect missing values.

Examples

For Series:

```
>>> s = pd.Series([None, 3, 4])
```

```
>>> s.first_valid_index()
```

```
1
```

```
>>> s.last_valid_index()
```

```
2
```

```
>>> s = pd.Series([None, None])
```

```
>>> print(s.first_valid_index())
```

```
None
```

```
>>> print(s.last_valid_index())
```

```
None
```

If all elements in Series are NA/null, returns None.

```
>>> s = pd.Series()
```

```
>>> print(s.first_valid_index())
```

```
None
```

```
>>> print(s.last_valid_index())
```

```
None
```

If Series is empty, returns None.

For DataFrame:

```
>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
```

```
>>> df
```

```
      A      B
```

```
0  NaN    NaN
```

```
1  NaN    3.0
```

```
2  2.0    4.0
```

```
>>> df.first_valid_index()
```

```
1
```

```
>>> df.last_valid_index()
```

```
2
```

```
>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
```

```
>>> df
```

```
      A      B
```

```
0  None  None
```

```
1  None  None
```

```
2  None  None
```

```
>>> print(df.first_valid_index())
```

```
None
```

```
>>> print(df.last_valid_index())
```

```
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
```

```
>>> df
```

```
Empty DataFrame
```

```
Columns: []
```

```
Index: []
```

```
>>> print(df.first_valid_index())
```

```
None
```

```
>>> print(df.last_valid_index())
```

```
None
```

If DataFrame is empty, returns None.

```
mask(  
    self,  
    cond,  
    other=<no_default>,  
)
```

```

*,
inplace: bool = False,
axis: Axis | None = None,
level: Level | None = None
) -> Self
    Replace values where the condition is True.

Parameters
-----
cond : bool Series/DataFrame, array-like, or callable
    Where `cond` is False, keep the original value. Where
    True, replace with corresponding value from `other`.
    If `cond` is callable, it is computed on the Series/DataFrame and
    should return boolean Series/DataFrame or array. The callable must
    not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
    Entries where `cond` is True are replaced with
    corresponding value from `other`.
    If other is callable, it is computed on the Series/DataFrame and
    should return scalar or Series/DataFrame. The callable must not
    change input Series/DataFrame (though pandas doesn't check it).
    If not specified, entries will be filled with the corresponding
    NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension
    dtypes).
inplace : bool, default False
    Whether to perform the operation in place on the data.
axis : int, default None
    Alignment axis if needed. For `Series` this parameter is
    unused and defaults to 0.
level : int, default None
    Alignment level if needed.

Returns
-----
Series or DataFrame
    When applied to a Series, the function will return a Series,
    and when applied to a DataFrame, it will return a DataFrame.

See Also
-----
:func:`DataFrame.where` : Return an object of same shape as caller.
:func:`Series.where` : Return an object of same shape as caller.

Notes
-----
The mask method is an application of the if-then idiom. For each
element in the caller, if ``cond`` is ``False`` the
element is used; otherwise the corresponding element from
``other`` is used. If the axis of ``other`` does not align with axis of
``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions
will be filled with True.

The signature for :func:`Series.where` or
:func:`DataFrame.where` differs from :func:`numpy.where`.
Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

For further details and examples see the ``mask`` documentation in
:ref:`indexing <indexing.where_mask>`.

The dtype of the object takes precedence. The fill value is casted to
the object's dtype, if this can be done losslessly.

Examples
-----
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0    NaN
1    1.0
2    2.0
3    3.0
4    4.0
dtype: float64
>>> s.mask(s > 0)
0    0.0
1    NaN
2    NaN
3    NaN

```

```

4      NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0      0
1     99
2     99
3     99
4     99
dtype: int64
>>> s.mask(t, 99)
0     99
1      1
2     99
3     99
4     99
dtype: int64

>>> s.where(s > 1, 10)
0     10
1     10
2      2
3      3
4      4
dtype: int64
>>> s.mask(s > 1, 10)
0      0
1      1
2     10
3     10
4     10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
   A  B
0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True

```

```

pct_change(
    self,
    periods: int = 1,
    fill_method: None = None,
    freq=None,
    **kwargs
) -> Self

```

Fractional change between the current and a prior element.

Computes the fractional change from the immediately previous row by default. This is useful in comparing the fraction of change in a time

series of elements.

.. note::

Despite the name of this method, it calculates fractional change (also known as per unit change or relative change) and not percentage change. If you need the percentage change, multiply these values by 100.

Parameters

periods : int, default 1

Periods to shift for forming percent change.

fill_method : None

Must be None. This argument will be removed in a future version of pandas.

freq : DateOffset, timedelta, or str, optional

Increment to use from time series API (e.g. 'ME' or BDay()).

**kwargs

Additional keyword arguments are passed into

`DataFrame.shift` or `Series.shift`.

Returns

Series or DataFrame

The same type as the calling object.

See Also

Series.diff : Compute the difference of two elements in a Series.

DataFrame.diff : Compute the difference of two elements in a DataFrame.

Series.shift : Shift the index by some number of periods.

DataFrame.shift : Shift the index by some number of periods.

Examples

****Series****

```
>>> s = pd.Series([90, 91, 85])
```

```
>>> s
```

```
0    90
```

```
1    91
```

```
2    85
```

```
dtype: int64
```

```
>>> s.pct_change()
```

```
0      NaN
```

```
1    0.011111
```

```
2   -0.065934
```

```
dtype: float64
```

```
>>> s.pct_change(periods=2)
```

```
0      NaN
```

```
1      NaN
```

```
2   -0.055556
```

```
dtype: float64
```

See the percentage change in a Series where filling NAs with last valid observation forward to next valid.

```
>>> s = pd.Series([90, 91, None, 85])
```

```
>>> s
```

```
0    90.0
```

```
1    91.0
```

```
2     NaN
```

```
3    85.0
```

```
dtype: float64
```

```
>>> s.ffill().pct_change()
```

```
0      NaN
```

```
1    0.011111
```

```
2    0.000000
```

```
3   -0.065934
```

```
dtype: float64
```

****DataFrame****

Percentage change in French franc, Deutsche Mark, and Italian lira from

1980-01-01 to 1980-03-01.

```
>>> df = pd.DataFrame(
...     {
...         "FR": [4.0405, 4.0963, 4.3149],
...         "GR": [1.7246, 1.7482, 1.8519],
...         "IT": [804.74, 810.01, 860.13],
...     },
...     index=["1980-01-01", "1980-02-01", "1980-03-01"],
... )
>>> df
```

	FR	GR	IT
1980-01-01	4.0405	1.7246	804.74
1980-02-01	4.0963	1.7482	810.01
1980-03-01	4.3149	1.8519	860.13

```
>>> df.pct_change()
```

	FR	GR	IT
1980-01-01	NaN	NaN	NaN
1980-02-01	0.013810	0.013684	0.006549
1980-03-01	0.053365	0.059318	0.061876

Percentage of change in GOOG and APPL stock volume. Shows computing the percentage change between columns.

```
>>> df = pd.DataFrame(
...     {
...         "2016": [1769950, 30586265],
...         "2015": [1500923, 40912316],
...         "2014": [1371819, 41403351],
...     },
...     index=["GOOG", "APPL"],
... )
>>> df
```

	2016	2015	2014
GOOG	1769950	1500923	1371819
APPL	30586265	40912316	41403351

```
>>> df.pct_change(axis="columns", periods=-1)
```

	2016	2015	2014
GOOG	0.179241	0.094112	NaN
APPL	-0.252395	-0.011860	NaN

```
pipe(
    self,
    func: Callable[[Concatenate[Self, P], T] | tuple[Callable[... , T], str],
    *args: Any,
    **kwargs: Any
) -> T
```

Apply chainable functions that expect Series or DataFrames.

Parameters

func : function

Function to apply to the Series/DataFrame. ``args``, and ``kwargs`` are passed into ``func``. Alternatively a ``(callable, data_keyword)`` tuple where ``data_keyword`` is a string indicating the keyword of ``callable`` that expects the Series/DataFrame.

***args** : iterable, optional

Positional arguments passed into ``func``.

****kwargs** : mapping, optional

A dictionary of keyword arguments passed into ``func``.

Returns

The return type of ``func``.
The result of applying ``func`` to the Series or DataFrame.

See Also

`DataFrame.apply` : Apply a function along input axis of DataFrame.
`DataFrame.map` : Apply a function elementwise on a whole DataFrame.
`Series.map` : Apply a mapping correspondence on a `:class:`~pandas.Series``.

Notes

Use ``.pipe`` when chaining together functions that expect Series, DataFrames or GroupBy objects.

Examples

Constructing an income DataFrame from a dictionary.

```
>>> data = [[8000, 1000], [9500, np.nan], [5000, 2000]]
>>> df = pd.DataFrame(data, columns=["Salary", "Others"])
>>> df
```

	Salary	Others
0	8000	1000.0
1	9500	NaN
2	5000	2000.0

Functions that perform tax reductions on an income DataFrame.

```
>>> def subtract_federal_tax(df):
...     return df * 0.9
>>> def subtract_state_tax(df, rate):
...     return df * (1 - rate)
>>> def subtract_national_insurance(df, rate, rate_increase):
...     new_rate = rate + rate_increase
...     return df * (1 - new_rate)
```

Instead of writing

```
>>> subtract_national_insurance(
...     subtract_state_tax(subtract_federal_tax(df), rate=0.12),
...     rate=0.05,
...     rate_increase=0.02,
... ) # doctest: +SKIP
```

You can write

```
>>> (
...     df.pipe(subtract_federal_tax)
...     .pipe(subtract_state_tax, rate=0.12)
...     .pipe(subtract_national_insurance, rate=0.05, rate_increase=0.02)
... )
```

	Salary	Others
0	5892.48	736.56
1	6997.32	NaN
2	3682.80	1473.12

If you have a function that takes the data as (say) the second argument, pass a tuple indicating which keyword expects the data. For example, suppose ``national\_insurance`` takes its data as ``df`` in the second argument:

```
>>> def subtract_national_insurance(rate, df, rate_increase):
...     new_rate = rate + rate_increase
...     return df * (1 - new_rate)
>>> (
...     df.pipe(subtract_federal_tax)
...     .pipe(subtract_state_tax, rate=0.12)
...     .pipe(
...         (subtract_national_insurance, "df"), rate=0.05, rate_increase=0.02
...     )
... )
```

	Salary	Others
0	5892.48	736.56
1	6997.32	NaN
2	3682.80	1473.12

```
rank(
    self,
    axis: Axis = 0,
    method: Literal['average', 'min', 'max', 'first', 'dense'] = 'average',
    numeric_only: bool = False,
    na_option: Literal['keep', 'top', 'bottom'] = 'keep',
    ascending: bool = True,
    pct: bool = False
) -> Self
    Compute numerical data ranks (1 through n) along axis.
```

By default, equal values are assigned a rank that is the average of the ranks of those values.

Parameters

axis : {0 or 'index', 1 or 'columns'}, default 0
Index to direct ranking.
For `Series` this parameter is unused and defaults to 0.
method : {'average', 'min', 'max', 'first', 'dense'}, default 'average'
How to rank the group of records that have the same value (i.e. ties):

- * average: average rank of the group
- * min: lowest rank in the group
- * max: highest rank in the group
- * first: ranks assigned in order they appear in the array
- * dense: like 'min', but rank always increases by 1 between groups.

numeric_only : bool, default False
For DataFrame objects, rank only numeric columns if set to True.

.. versionchanged:: 2.0.0
The default value of ``numeric\_only`` is now ``False``.

na_option : {'keep', 'top', 'bottom'}, default 'keep'
How to rank NaN values:

- * keep: assign NaN rank to NaN values
- * top: assign lowest rank to NaN values
- * bottom: assign highest rank to NaN values

ascending : bool, default True
Whether or not the elements should be ranked in ascending order.

pct : bool, default False
Whether or not to display the returned rankings in percentile form.

Returns

same type as caller
Return a Series or DataFrame with data ranks as values.

See Also

core.groupby.DataFrameGroupBy.rank : Rank of values within each group.
core.groupby.SeriesGroupBy.rank : Rank of values within each group.

Examples

>>> df = pd.DataFrame(
... data={
... "Animal": ["cat", "penguin", "dog", "spider", "snake"],
... "Number_legs": [4, 2, 4, 8, np.nan],
... }
...)
>>> df

	Animal	Number_legs
0	cat	4.0
1	penguin	2.0
2	dog	4.0
3	spider	8.0
4	snake	NaN

Ties are assigned the mean of the ranks (by default) for the group.

```
>>> s = pd.Series(range(5), index=list("abcde"))  
>>> s["d"] = s["b"]  
>>> s.rank()  
a    1.0  
b    2.5  
c    4.0  
d    2.5  
e    5.0  
dtype: float64
```

The following example shows how the method behaves with the above parameters:

- * default_rank: this is the default behaviour obtained without using any parameter.
- * max_rank: setting ``method = 'max'`` the records that have the same values are ranked using the highest rank (e.g.: since 'cat' and 'dog' are both in the 2nd and 3rd position, rank 3 is assigned.)
- * NA_bottom: choosing ``na\_option = 'bottom'``, if there are records with NaN values they are placed at the bottom of the ranking.
- * pct_rank: when setting ``pct = True``, the ranking is expressed as percentile rank.

```
>>> df["default_rank"] = df["Number_legs"].rank()
>>> df["max_rank"] = df["Number_legs"].rank(method="max")
>>> df["NA_bottom"] = df["Number_legs"].rank(na_option="bottom")
>>> df["pct_rank"] = df["Number_legs"].rank(pct=True)
>>> df
```

	Animal	Number_legs	default_rank	max_rank	NA_bottom	pct_rank
0	cat	4.0	2.5	3.0	2.5	0.625
1	penguin	2.0	1.0	1.0	1.0	0.250
2	dog	4.0	2.5	3.0	2.5	0.625
3	spider	8.0	4.0	4.0	4.0	1.000
4	snake	NaN	NaN	NaN	5.0	NaN

```
reindex_like(
    self,
    other,
    method: Literal['backfill', 'bfill', 'pad', 'ffill', 'nearest'] | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    limit: int | None = None,
    tolerance=None
) -> Self
```

Return an object with matching indices as other object.

Conform the object to the same index on all axes. Optional filling logic, placing NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and copy=False.

Parameters

other : Object of the same data type
Its row and column indices are used to define the new indices of this object.

method : {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}
Method to use for filling holes in reindexed DataFrame.
Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

.. deprecated:: 3.0.0

- * None (default): don't fill gaps
- * pad / ffill: propagate last valid observation forward to next valid
- * backfill / bfill: use next valid observation to fill gap
- * nearest: use nearest valid observations to fill gap.

copy : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>` for more details.

limit : int, default None

Maximum number of consecutive labels to fill for inexact matches.

tolerance : optional

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per

element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns

Series or DataFrame

Same type as caller, but with changed indices on each axis.

See Also

DataFrame.set_index : Set row labels.

DataFrame.reset_index : Remove row labels or move them to new columns.

DataFrame.reindex : Change to new indices or expand indices.

Notes

Same as calling

```.reindex(index=other.index, columns=other.columns,...)```.

#### Examples

```
>>> df1 = pd.DataFrame(
... [
... [24.3, 75.7, "high"],
... [31, 87.8, "high"],
... [22, 71.6, "medium"],
... [35, 95, "medium"],
...],
... columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
... index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
...)
```

```
>>> df1
 temp_celsius temp_fahrenheit windspeed
2014-02-12 24.3 75.7 high
2014-02-13 31.0 87.8 high
2014-02-14 22.0 71.6 medium
2014-02-15 35.0 95.0 medium
```

```
>>> df2 = pd.DataFrame(
... [[28, "low"], [30, "low"], [35.1, "medium"]],
... columns=["temp_celsius", "windspeed"],
... index=pd.DatetimeIndex(["2014-02-12", "2014-02-13", "2014-02-15"]),
...)
```

```
>>> df2
 temp_celsius windspeed
2014-02-12 28.0 low
2014-02-13 30.0 low
2014-02-15 35.1 medium
```

```
>>> df2.reindex_like(df1)
 temp_celsius temp_fahrenheit windspeed
2014-02-12 28.0 NaN low
2014-02-13 30.0 NaN low
2014-02-14 NaN NaN NaN
2014-02-15 35.1 NaN medium
```

```
rename_axis(
 self,
 mapper: IndexLabel | lib.NoDefault = <no_default>,
 *,
 index=<no_default>,
 columns=<no_default>,
 axis: Axis = 0,
 copy: bool | lib.NoDefault = <no_default>,
 inplace: bool = False
) -> Self | None
Set the name of the axis for the index or columns.
```

#### Parameters

mapper : scalar, list-like, optional  
Value to set the axis name attribute.

Use either ```mapper``` and ```axis``` to

```

 specify the axis to target with ``mapper``, or ``index``
 and/or ``columns``.
index : scalar, list-like, dict-like or function, optional
 A scalar, list-like, dict-like or functions transformations to
 apply to that axis' values.
columns : scalar, list-like, dict-like or function, optional
 A scalar, list-like, dict-like or functions transformations to
 apply to that axis' values.
axis : {0 or 'index', 1 or 'columns'}, default 0
 The axis to rename.
copy : bool, default False
 This keyword is now ignored; changing its value will have no
 impact on the method.

.. deprecated:: 3.0.0

 This keyword is ignored and will be removed in pandas 4.0. Since
 pandas 3.0, this method always returns a new object using a lazy
 copy mechanism that defers copies until necessary
 (Copy-on-Write). See the `user guide on Copy-on-Write
 <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
 for more details.

inplace : bool, default False
 Modifies the object directly, instead of creating a new Series
 or DataFrame.

Returns

DataFrame, or None
 The same type as the caller or None if ``inplace=True``.

See Also

Series.rename : Alter Series index labels or name.
DataFrame.rename : Alter DataFrame index labels or name.
Index.rename : Set new names on index.

Notes

``DataFrame.rename_axis`` supports two calling conventions

* ``(index=index_mapper, columns=columns_mapper, ...)``
* ``(mapper, axis={'index', 'columns'}, ...)``

The first calling convention will only modify the names of
the index and/or the names of the Index object that is the columns.
In this case, the parameter ``copy`` is ignored.

The second calling convention will modify the names of the
corresponding index if mapper is a list or a scalar.
However, if mapper is dict-like or a function, it will use the
deprecated behavior of modifying the axis *labels*.

We *highly* recommend using keyword arguments to clarify your
intent.

Examples

DataFrame

>>> df = pd.DataFrame(
... {"num_legs": [4, 4, 2], "num_arms": [0, 0, 2]}, ["dog", "cat", "monkey"]
...)
>>> df
 num_legs num_arms
dog 4 0
cat 4 0
monkey 2 2
>>> df = df.rename_axis("animal")
>>> df
 num_legs num_arms
animal
dog 4 0
cat 4 0
monkey 2 2
>>> df = df.rename_axis("limbs", axis="columns")

```

```

>>> df
limbs num_legs num_arms
animal
dog 4 0
cat 4 0
monkey 2 2

MultiIndex

>>> df.index = pd.MultiIndex.from_product(
... [["mammal"], ["dog", "cat", "monkey"]], names=["type", "name"]
...)
>>> df
limbs num_legs num_arms
type name
mammal dog 4 0
 cat 4 0
 monkey 2 2

>>> df.rename_axis(index={"type": "class"})
limbs num_legs num_arms
class name
mammal dog 4 0
 cat 4 0
 monkey 2 2

>>> df.rename_axis(columns=str.upper)
LIMBS num_legs num_arms
type name
mammal dog 4 0
 cat 4 0
 monkey 2 2

```

```

replace(
 self,
 to_replace=None,
 value=<no_default>,
 *,
 inplace: bool = False,
 regex: bool = False
) -> Self
Replace values given in `to_replace` with `value`.

```

Values of the Series/DataFrame are replaced with other values dynamically. This differs from updating with ``.loc`` or ``.iloc``, which require you to specify a location to update with some value.

#### Parameters

**to\_replace** : str, regex, list, dict, Series, int, float, or None  
How to find the values that will be replaced.

\* numeric, str or regex:

- numeric: numeric values equal to `to\_replace` will be replaced with `value`
- str: string exactly matching `to\_replace` will be replaced with `value`
- regex: regexes matching `to\_replace` will be replaced with `value`

\* list of str, regex, or numeric:

- First, if `to\_replace` and `value` are both lists, they **must** be the same length.
- Second, if ``regex=True`` then all of the strings in **both** lists will be interpreted as regexes otherwise they will match directly. This doesn't matter much for `value` since there are only a few possible substitution regexes you can use.
- str, regex and numeric rules apply as above.

\* dict:

- Dicts can be used to specify different replacement values for different existing values. For example, ``{'a': 'b', 'y': 'z'}`` replaces the value 'a' with 'b' and 'y' with 'z'. To use a dict in this way, the optional `value`



```

parameter should not be given.
- For a DataFrame a dict can specify that different values
 should be replaced in different columns. For example,
 ``{'a': 1, 'b': 'z'}`` looks for the value 1 in column 'a'
 and the value 'z' in column 'b' and replaces these values
 with whatever is specified in `value`. The `value` parameter
 should not be ``None`` in this case. You can treat this as a
 special case of passing two lists except that you are
 specifying the column to search in.
- For a DataFrame nested dictionaries, e.g.,
 ``{'a': {'b': np.nan}}``, are read as follows: look in column
 'a' for the value 'b' and replace it with NaN. The optional `value`
 parameter should not be specified to use a nested dict in this
 way. You can nest regular expressions as well. Note that
 column names (the top-level dictionary keys in a nested
 dictionary) cannot be regular expressions.

* None:

- This means that the `regex` argument must be a string,
 compiled regular expression, or list, dict, ndarray or
 Series of such elements. If `value` is also ``None`` then
 this must be a nested dictionary or Series.

See the examples section for examples of each of these.
value : scalar, dict, list, str, regex, default None
Value to replace any values matching `to_replace` with.
For a DataFrame a dict of values can be used to specify which
value to use for each column (columns not in the dict will not be
filled). Regular expressions, strings and lists or dicts of such
objects are also allowed.

inplace : bool, default False
If True, performs operation inplace.
regex : bool or same types as `to_replace`, default False
Whether to interpret `to_replace` and/or `value` as regular
expressions. Alternatively, this could be a regular expression or a
list, dict, or array of regular expressions in which case
`to_replace` must be ``None``.

Returns

Series/DataFrame
Object after replacement.

Raises

AssertionError
* If `regex` is not a ``bool`` and `to_replace` is not
 ``None``.

TypeError
* If `to_replace` is not a scalar, array-like, ``dict``, or ``None``
* If `to_replace` is a ``dict`` and `value` is not a ``list``,
 ``dict``, ``ndarray``, or ``Series``
* If `to_replace` is ``None`` and `regex` is not compilable
 into a regular expression or is a list, dict, ndarray, or
 Series.
* When replacing multiple ``bool`` or ``datetime64`` objects and
 the arguments to `to_replace` does not match the type of the
 value being replaced

ValueError
* If a ``list`` or an ``ndarray`` is passed to `to_replace` and
 `value` but they are not the same length.

See Also

Series.fillna : Fill NA values.
DataFrame.fillna : Fill NA values.
Series.where : Replace values based on boolean condition.
DataFrame.where : Replace values based on boolean condition.
DataFrame.map: Apply a function to a Dataframe elementwise.
Series.map: Map values of Series according to an input mapping or function.
Series.str.replace : Simple string replacement.

Notes

```

```

* Regex substitution is performed under the hood with ``re.sub``. The
 rules for substitution for ``re.sub`` are the same.
* Regular expressions will only substitute on strings, meaning you
 cannot provide, for example, a regular expression matching floating
 point numbers and expect the columns in your frame that have a
 numeric dtype to be matched. However, if those floating point
 numbers are strings, then you can do this.
* This method has a lot of options. You are encouraged to experiment
 and play with this method to gain intuition about how it works.
* When dict is used as the to_replace value, it is like
 key(s) in the dict are the to_replace part and
 value(s) in the dict are the value parameter.

```

#### Examples

**\*\*Scalar `to_replace` and `value`\*\***

```

>>> s = pd.Series([1, 2, 3, 4, 5])
>>> s.replace(1, 5)
0 5
1 2
2 3
3 4
4 5
dtype: int64

```

```

>>> df = pd.DataFrame(
... {
... "A": [0, 1, 2, 3, 4],
... "B": [5, 6, 7, 8, 9],
... "C": ["a", "b", "c", "d", "e"],
... }
...)
>>> df.replace(0, 5)
 A B C
0 5 5 a
1 1 6 b
2 2 7 c
3 3 8 d
4 4 9 e

```

**\*\*List-like `to_replace`\*\***

```

>>> df.replace([0, 1, 2, 3], 4)
 A B C
0 4 5 a
1 4 6 b
2 4 7 c
3 4 8 d
4 4 9 e

>>> df.replace([0, 1, 2, 3], [4, 3, 2, 1])
 A B C
0 4 5 a
1 3 6 b
2 2 7 c
3 1 8 d
4 4 9 e

```

**\*\*dict-like `to_replace`\*\***

```

>>> df.replace({0: 10, 1: 100})
 A B C
0 10 5 a
1 100 6 b
2 2 7 c
3 3 8 d
4 4 9 e

>>> df.replace({"A": 0, "B": 5}, 100)
 A B C
0 100 100 a
1 1 6 b
2 2 7 c
3 3 8 d

```

```

4 4 9 e

>>> df.replace({"A": {0: 100, 4: 400}})
 A B C
0 100 5 a
1 1 6 b
2 2 7 c
3 3 8 d
4 400 9 e

Regular expression `to_replace`

>>> df = pd.DataFrame({"A": ["bat", "foo", "bait"], "B": ["abc", "bar", "xyz"]})
>>> df.replace(to_replace=r"^ba.$", value="new", regex=True)
 A B
0 new abc
1 foo new
2 bait xyz

>>> df.replace({"A": r"^ba.$"}, {"A": "new"}, regex=True)
 A B
0 new abc
1 foo bar
2 bait xyz

>>> df.replace(regex=r"^ba.$", value="new")
 A B
0 new abc
1 foo new
2 bait xyz

>>> df.replace(regex={r"^ba.$": "new", "foo": "xyz"})
 A B
0 new abc
1 xyz new
2 bait xyz

>>> df.replace(regex=[r"^ba.$", "foo"], value="new")
 A B
0 new abc
1 new new
2 bait xyz

Compare the behavior of ``s.replace({'a': None})`` and
``s.replace('a', None)`` to understand the peculiarities
of the `to_replace` parameter:

>>> s = pd.Series([10, "a", "a", "b", "a"])

When one uses a dict as the `to_replace` value, it is like the
value(s) in the dict are equal to the `value` parameter.
``s.replace({'a': None})`` is equivalent to
``s.replace(to_replace={'a': None}, value=None)``:

>>> s.replace({"a": None})
0 10
1 None
2 None
3 b
4 None
dtype: object

If ``None`` is explicitly passed for ``value``, it will be respected:

>>> s.replace("a", None)
0 10
1 None
2 None
3 b
4 None
dtype: object

When ``regex=True``, ``value`` is not ``None`` and `to_replace` is a string,
the replacement will be applied in all columns of the DataFrame.

>>> df = pd.DataFrame(
... {

```

```
... "A": [0, 1, 2, 3, 4],
... "B": ["a", "b", "c", "d", "e"],
... "C": ["f", "g", "h", "i", "j"],
... }
...)
```

```
>>> df.replace(to_replace="^[a-g]", value="e", regex=True)
```

```
 A B C
0 0 e e
1 1 e e
2 2 e h
3 3 e i
4 4 e j
```

If ``value`` is not ``None`` and ``to\_replace`` is a dictionary, the dictionary keys will be the DataFrame columns that the replacement will be applied.

```
>>> df.replace(to_replace={"B": "^[a-c]", "C": "^[h-j]"}, value="e", regex=True)
```

```
 A B C
0 0 e f
1 1 e g
2 2 e e
3 3 d e
4 4 e e
```

```
resample(
 self,
 rule,
 closed: Literal['right', 'left'] | None = None,
 label: Literal['right', 'left'] | None = None,
 convention: Literal['start', 'end', 's', 'e'] = 'start',
 on: Level | None = None,
 level: Level | None = None,
 origin: str | TimestampConvertibleTypes = 'start_day',
 offset: TimedeltaConvertibleTypes | None = None,
 group_keys: bool = False
) -> Resampler
Resample time-series data.
```

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (`DatetimeIndex`, `PeriodIndex`, or `TimedeltaIndex`), or the caller must pass the label of a datetime-like series/index to the `on`/`level` keyword parameter.

#### Parameters

```
rule : DateOffset, Timedelta or str
 The offset string or object representing target conversion.
closed : {'right', 'left'}, default None
 Which side of bin interval is closed. The default is 'left'
 for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
 'BA', 'BQE', and 'W' which all have a default of 'right'.
label : {'right', 'left'}, default None
 Which bin edge label to label bucket with. The default is 'left'
 for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
 'BA', 'BQE', and 'W' which all have a default of 'right'.
convention : {'start', 'end', 's', 'e'}, default 'start'
 For 'PeriodIndex' only, controls whether to use the start or
 end of 'rule'.
on : str, optional
 For a DataFrame, column to use instead of index for resampling.
 Column must be datetime-like.
level : str or int, optional
 For a MultiIndex, level (name or number) to use for
 resampling. 'level' must be datetime-like.
origin : Timestamp or str, default 'start_day'
 The timestamp on which to adjust the grouping. The timezone of origin
 must match the timezone of the index.
 If string, must be Timestamp convertible or one of the following:

 - 'epoch': 'origin' is 1970-01-01
 - 'start': 'origin' is the first value of the timeseries
 - 'start_day': 'origin' is the first day at midnight of the timeseries

 - 'end': 'origin' is the last value of the timeseries
 - 'end_day': 'origin' is the ceiling midnight of the last day
```

```
.. note::
```

```
 Only takes effect for Tick-frequencies (i.e. fixed frequencies like
 days, hours, and minutes, rather than months or quarters).
offset : Timedelta or str, default is None
 An offset timedelta added to the origin.
```

```
group_keys : bool, default False
 Whether to include the group keys in the result index when using
 ``.apply()`` on the resampled object.
```

```
.. versionchanged:: 2.0.0
```

```
 ``group_keys`` now defaults to ``False``.
```

```
Returns
```

```

pandas.api.typing.Resampler
: class: ~pandas.core.Resampler` object.
```

```
See Also
```

```

Series.resample : Resample a Series.
DataFrame.resample : Resample a DataFrame.
groupby : Group Series/DataFrame by mapping, function, label, or list of labels.
asfreq : Reindex a Series/DataFrame with the given frequency without grouping.
```

```
Notes
```

```

See the `user guide
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#resampling>`__
for more.
```

```
To learn more about the offset strings, please see `this link
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#dateoffset-objects>`__
```

```
Examples
```

```

Start by creating a series with 9 one minute timestamps.
```

```
>>> index = pd.date_range("1/1/2000", periods=9, freq="min")
>>> series = pd.Series(range(9), index=index)
>>> series
2000-01-01 00:00:00 0
2000-01-01 00:01:00 1
2000-01-01 00:02:00 2
2000-01-01 00:03:00 3
2000-01-01 00:04:00 4
2000-01-01 00:05:00 5
2000-01-01 00:06:00 6
2000-01-01 00:07:00 7
2000-01-01 00:08:00 8
Freq: min, dtype: int64
```

```
Downsample the series into 3 minute bins and sum the values
of the timestamps falling into a bin.
```

```
>>> series.resample("3min").sum()
2000-01-01 00:00:00 3
2000-01-01 00:03:00 12
2000-01-01 00:06:00 21
Freq: 3min, dtype: int64
```

```
Downsample the series into 3 minute bins as above, but label each
bin using the right edge instead of the left. Please note that the
value in the bucket used as the label is not included in the bucket,
which it labels. For example, in the original series the
bucket ``2000-01-01 00:03:00`` contains the value 3, but the summed
value in the resampled bucket with the label ``2000-01-01 00:03:00``
does not include 3 (if it did, the summed value would be 6, not 3).
```

```
>>> series.resample("3min", label="right").sum()
2000-01-01 00:03:00 3
2000-01-01 00:06:00 12
2000-01-01 00:09:00 21
Freq: 3min, dtype: int64
```

To include this value close the right side of the bin interval, as shown below.

```
>>> series.resample("3min", label="right", closed="right").sum()
2000-01-01 00:00:00 0
2000-01-01 00:03:00 6
2000-01-01 00:06:00 15
2000-01-01 00:09:00 15
Freq: 3min, dtype: int64
```

Upsample the series into 30 second bins.

```
>>> series.resample("30s").asfreq()[0:5] # Select first 5 rows
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 1.0
2000-01-01 00:01:30 NaN
2000-01-01 00:02:00 2.0
Freq: 30s, dtype: float64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``ffill`` method.

```
>>> series.resample("30s").ffill()[0:5]
2000-01-01 00:00:00 0
2000-01-01 00:00:30 0
2000-01-01 00:01:00 1
2000-01-01 00:01:30 1
2000-01-01 00:02:00 2
Freq: 30s, dtype: int64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``bfill`` method.

```
>>> series.resample("30s").bfill()[0:5]
2000-01-01 00:00:00 0
2000-01-01 00:00:30 1
2000-01-01 00:01:00 1
2000-01-01 00:01:30 2
2000-01-01 00:02:00 2
Freq: 30s, dtype: int64
```

Pass a custom function via ``apply``

```
>>> def custom_resampler(arraylike):
... return np.sum(arraylike) + 5
>>> series.resample("3min").apply(custom_resampler)
2000-01-01 00:00:00 8
2000-01-01 00:03:00 17
2000-01-01 00:06:00 26
Freq: 3min, dtype: int64
```

For a Series with a PeriodIndex, the keyword ``convention`` can be used to control whether to use the start or end of ``rule``.

Resample a year by quarter using 'start' ``convention``. Values are assigned to the first quarter of the period.

```
>>> s = pd.Series(
... [1, 2], index=pd.period_range("2012-01-01", freq="Y", periods=2)
...)
>>> s
2012 1
2013 2
Freq: Y-DEC, dtype: int64
>>> s.resample("Q", convention="start").asfreq()
2012Q1 1.0
2012Q2 NaN
2012Q3 NaN
2012Q4 NaN
2013Q1 2.0
2013Q2 NaN
2013Q3 NaN
2013Q4 NaN
Freq: Q-DEC, dtype: float64
```

Resample quarters by month using 'end' ``convention``. Values are

assigned to the last month of the period.

```
>>> q = pd.Series(
... [1, 2, 3, 4], index=pd.period_range("2018-01-01", freq="Q", periods=4)
...)
>>> q
2018Q1 1
2018Q2 2
2018Q3 3
2018Q4 4
Freq: Q-DEC, dtype: int64
>>> q.resample("M", convention="end").asfreq()
2018-03 1.0
2018-04 NaN
2018-05 NaN
2018-06 2.0
2018-07 NaN
2018-08 NaN
2018-09 3.0
2018-10 NaN
2018-11 NaN
2018-12 4.0
Freq: M, dtype: float64
```

For DataFrame objects, the keyword `on` can be used to specify the column instead of the index for resampling.

```
>>> df = pd.DataFrame([10, 11, 9, 13, 14, 18, 17, 19], columns=["price"])
>>> df["volume"] = [50, 60, 40, 100, 50, 100, 40, 50]
>>> df["week_starting"] = pd.date_range("01/01/2018", periods=8, freq="W")
>>> df
 price volume week_starting
0 10 50 2018-01-07
1 11 60 2018-01-14
2 9 40 2018-01-21
3 13 100 2018-01-28
4 14 50 2018-02-04
5 18 100 2018-02-11
6 17 40 2018-02-18
7 19 50 2018-02-25
>>> df.resample("ME", on="week_starting").mean()
 price volume
week_starting
2018-01-31 10.75 62.5
2018-02-28 17.00 60.0
```

For a DataFrame with MultiIndex, the keyword `level` can be used to specify on which level the resampling needs to take place.

```
>>> days = pd.date_range("1/1/2000", periods=4, freq="D")
>>> df2 = pd.DataFrame(
... [
... [10, 50],
... [11, 60],
... [9, 40],
... [13, 100],
... [14, 50],
... [18, 100],
... [17, 40],
... [19, 50],
...],
... columns=["price", "volume"],
... index=pd.MultiIndex.from_product([days, ["morning", "afternoon"]]),
...)
>>> df2
 price volume
2000-01-01 morning 10 50
 afternoon 11 60
2000-01-02 morning 9 40
 afternoon 13 100
2000-01-03 morning 14 50
 afternoon 18 100
2000-01-04 morning 17 40
 afternoon 19 50
>>> df2.resample("D", level=0).sum()
 price volume
2000-01-01 21 110
```

2000-01-02	22	140
2000-01-03	32	150
2000-01-04	36	90

If you want to adjust the start of the bins based on a fixed timestamp:

```
>>> start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
>>> rng = pd.date_range(start, end, freq="7min")
>>> ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
>>> ts
2000-10-01 23:30:00 0
2000-10-01 23:37:00 3
2000-10-01 23:44:00 6
2000-10-01 23:51:00 9
2000-10-01 23:58:00 12
2000-10-02 00:05:00 15
2000-10-02 00:12:00 18
2000-10-02 00:19:00 21
2000-10-02 00:26:00 24
Freq: 7min, dtype: int64
```

```
>>> ts.resample("17min").sum()
2000-10-01 23:14:00 0
2000-10-01 23:31:00 9
2000-10-01 23:48:00 21
2000-10-02 00:05:00 54
2000-10-02 00:22:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", origin="epoch").sum()
2000-10-01 23:18:00 0
2000-10-01 23:35:00 18
2000-10-01 23:52:00 27
2000-10-02 00:09:00 39
2000-10-02 00:26:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", origin="2000-01-01").sum()
2000-10-01 23:24:00 3
2000-10-01 23:41:00 15
2000-10-01 23:58:00 45
2000-10-02 00:15:00 45
Freq: 17min, dtype: int64
```

If you want to adjust the start of the bins with an `offset` Timedelta, the two following lines are equivalent:

```
>>> ts.resample("17min", origin="start").sum()
2000-10-01 23:30:00 9
2000-10-01 23:47:00 21
2000-10-02 00:04:00 54
2000-10-02 00:21:00 24
Freq: 17min, dtype: int64
```

```
>>> ts.resample("17min", offset="23h30min").sum()
2000-10-01 23:30:00 9
2000-10-01 23:47:00 21
2000-10-02 00:04:00 54
2000-10-02 00:21:00 24
Freq: 17min, dtype: int64
```

If you want to take the largest Timestamp as the end of the bins:

```
>>> ts.resample("17min", origin="end").sum()
2000-10-01 23:35:00 0
2000-10-01 23:52:00 18
2000-10-02 00:09:00 27
2000-10-02 00:26:00 63
Freq: 17min, dtype: int64
```

In contrast with the `start\_day`, you can use `end\_day` to take the ceiling midnight of the largest Timestamp as the end of the bins and drop the bins not containing data:

```
>>> ts.resample("17min", origin="end_day").sum()
2000-10-01 23:38:00 3
2000-10-01 23:55:00 15
```



```
2000-10-02 00:12:00 45
2000-10-02 00:29:00 45
Freq: 17min, dtype: int64
```

```
rolling(
 self,
 window: int | dt.timedelta | str | BaseOffset | BaseIndexer,
 min_periods: int | None = None,
 center: bool = False,
 win_type: str | None = None,
 on: str | None = None,
 closed: IntervalClosedType | None = None,
 step: int | None = None,
 method: str = 'single'
) -> Window | Rolling
Provide rolling window calculations.
```

#### Parameters

**window** : int, timedelta, str, offset, or BaseIndexer subclass  
Interval of the moving window.

If an integer, the delta between the start and end of each window.  
The number of points in the window depends on the ``closed`` argument.

If a timedelta, str, or offset, the time period of each window. Each window will be a variable sized based on the observations included in the time-period. This is only valid for datetimelike indexes.  
To learn more about the offsets & frequency strings, please see :ref:`this link<timeseries.offset\_aliases>`.

If a BaseIndexer subclass, the window boundaries based on the defined ``get\_window\_bounds`` method. Additional rolling keyword arguments, namely ``min\_periods``, ``center``, ``closed`` and ``step`` will be passed to ``get\_window\_bounds``.

**min\_periods** : int, default None  
Minimum number of observations in window required to have a value; otherwise, result is ``np.nan``.

For a window that is specified by an offset, ``min\_periods`` will default to 1.

For a window that is specified by an integer, ``min\_periods`` will default to the size of the window.

**center** : bool, default False  
If False, set the window labels as the right edge of the window index.  
If True, set the window labels as the center of the window index.

**win\_type** : str, default None  
If ``None``, all points are evenly weighted.

If a string, it must be a valid `scipy.signal` window function  
<<https://docs.scipy.org/doc/scipy/reference/signal.windows.html#module-scipy.signal>>.

Certain Scipy window types require additional parameters to be passed in the aggregation function. The additional parameters must match the keywords specified in the Scipy window type method signature.

**on** : str, optional  
For a DataFrame, a column label or Index level on which to calculate the rolling window, rather than the DataFrame's index.

Provided integer column is ignored and excluded from result since an integer index is not used to calculate the rolling window.

**closed** : str, default None  
Determines the inclusivity of points in the window

If ``'right'``, uses the window (first, last] meaning the last point is included in the calculations.

If ``'left'``, uses the window [first, last) meaning the first point is included in the calculations.

If ``'both'``, uses the window [first, last] meaning all points in the window are included in the calculations.

If ``'neither'``, uses the window (first, last) meaning the first and last points in the window are excluded from calculations.

() and [] are referencing open and closed set notation respectively.

Default ``None`` (``'right'``).

step : int, default None

Evaluate the window at every ``step`` result, equivalent to slicing as ``[:step]``. ``window`` must be an integer. Using a step argument other than None or 1 will produce a result with a different shape than the input.

method : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row (``'single'``) or over the entire object (``'table'``).

This argument is only implemented when specifying ``engine='numba'`` in the method call.

Returns

-----

pandas.api.typing.Window or pandas.api.typing.Rolling

An instance of Window is returned if ``win\_type`` is passed. Otherwise, an instance of Rolling is returned.

See Also

-----

expanding : Provides expanding transformations.

ewm : Provides exponential weighted functions.

Notes

-----

See :ref:`Windowing Operations <window.generic>` for further usage details and examples.

Examples

-----

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
```

```
>>> df
```

```
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

**\*\*window\*\***

Rolling sum with a window length of 2 observations.

```
>>> df.rolling(2).sum()
```

```
 B
0 NaN
1 1.0
2 3.0
3 NaN
4 NaN
```

Rolling sum with a window span of 2 seconds.

```
>>> df_time = pd.DataFrame(
... {"B": [0, 1, 2, np.nan, 4]},
... index=[
... pd.Timestamp("20130101 09:00:00"),
... pd.Timestamp("20130101 09:00:02"),
... pd.Timestamp("20130101 09:00:03"),
... pd.Timestamp("20130101 09:00:05"),
... pd.Timestamp("20130101 09:00:06"),
...],
...)
```

```
>>> df_time
```

```

 B
2013-01-01 09:00:00 0.0
2013-01-01 09:00:02 1.0
2013-01-01 09:00:03 2.0
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0

```

```
>>> df_time.rolling("2s").sum()
```

```

 B
2013-01-01 09:00:00 0.0
2013-01-01 09:00:02 1.0
2013-01-01 09:00:03 3.0
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0

```

Rolling sum with forward looking windows with 2 observations.

```
>>> indexer = pd.api.indexers.FixedForwardWindowIndexer(window_size=2)
```

```
>>> df.rolling(window=indexer, min_periods=1).sum()
```

```

 B
0 1.0
1 3.0
2 2.0
3 4.0
4 4.0

```

**\*\*min\_periods\*\***

Rolling sum with a window length of 2 observations, but only needs a minimum of 1 observation to calculate a value.

```
>>> df.rolling(2, min_periods=1).sum()
```

```

 B
0 0.0
1 1.0
2 3.0
3 2.0
4 4.0

```

**\*\*center\*\***

Rolling sum with the result assigned to the center of the window index.

```
>>> df.rolling(3, min_periods=1, center=True).sum()
```

```

 B
0 1.0
1 3.0
2 3.0
3 6.0
4 4.0

```

```
>>> df.rolling(3, min_periods=1, center=False).sum()
```

```

 B
0 0.0
1 1.0
2 3.0
3 3.0
4 6.0

```

**\*\*step\*\***

Rolling sum with a window length of 2 observations, minimum of 1 observation to calculate a value, and a step of 2.

```
>>> df.rolling(2, min_periods=1, step=2).sum()
```

```

 B
0 0.0
2 3.0
4 4.0

```

**\*\*win\_type\*\***

Rolling sum with a window length of 2, using the Scipy `''gaussian''` window type. `''std''` is required in the aggregation function.

```
>>> df.rolling(2, win_type="gaussian").sum(std=3)
```

```

 B

```

```

0 NaN
1 0.986207
2 2.958621
3 NaN
4 NaN

on

Rolling sum with a window length of 2 days.

>>> df = pd.DataFrame(
... {
... "A": [
... pd.to_datetime("2020-01-01"),
... pd.to_datetime("2020-01-01"),
... pd.to_datetime("2020-01-02"),
...],
... "B": [1, 2, 3],
... },
... index=pd.date_range("2020", periods=3),
...)

>>> df
 A B
2020-01-01 2020-01-01 1
2020-01-02 2020-01-01 2
2020-01-03 2020-01-02 3

>>> df.rolling("2D", on="A").sum()
 A B
2020-01-01 2020-01-01 1.0
2020-01-02 2020-01-01 3.0
2020-01-03 2020-01-02 6.0

```

```

sample(
 self,
 n: int | None = None,
 frac: float | None = None,
 replace: bool = False,
 weights=None,
 random_state: RandomState | None = None,
 axis: Axis | None = None,
 ignore_index: bool = False
) -> Self

```

Return a random sample of items from an axis of object.

You can use `random\_state` for reproducibility.

#### Parameters

-----

**n** : int, optional

Number of items from axis to return. Cannot be used with `frac`.  
Default = 1 if `frac` = None.

**frac** : float, optional

Fraction of axis items to return. Cannot be used with `n`.

**replace** : bool, default False

Allow or disallow sampling of the same row more than once.

**weights** : str or ndarray-like, optional

Default ``None`` results in equal probability weighting.

If passed a Series, will align with target object on index. Index values in weights not found in sampled object will be ignored and index values in sampled object not in weights will be assigned weights of zero.

If called on a DataFrame, will accept the name of a column when axis = 0.

Unless weights are a Series, weights must be same length as axis being sampled.

If weights do not sum to 1, they will be normalized to sum to 1.

Missing values in the weights column will be treated as zero.

Infinite values not allowed.

When replace = False will not allow ``(n \* max(weights) / sum(weights)) > 1`` in order to avoid biased results. See the Notes below for more details.

**random\_state** : int, array-like, BitGenerator, np.random.RandomState, np.random.Generator

If int, array-like, or BitGenerator, seed for random number generator.

If np.random.RandomState or np.random.Generator, use as given.

Default ``None`` results in sampling with the current state of np.random.

**axis** : {0 or 'index', 1 or 'columns', None}, default None

Axis to sample. Accepts axis number or name. Default is stat axis for given data type. For `Series` this parameter is unused and defaults to `None`.  
 ignore\_index : bool, default False  
 If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

-----  
 Series or DataFrame  
 A new object of same type as caller containing `n` items randomly sampled from the caller object.

See Also

-----  
 DataFrameGroupBy.sample: Generates random samples from each group of a DataFrame object.  
 SeriesGroupBy.sample: Generates random samples from each group of a Series object.  
 numpy.random.choice: Generates a random sample from a given 1-D numpy array.

Notes

-----  
 If `frac` > 1, `replacement` should be set to `True`.

When replace = False will not allow `(n \* max(weights) / sum(weights)) > 1`, since that would cause results to be biased. E.g. sampling 2 items without replacement with weights [100, 1, 1] would yield two last items in 1/2 of cases, instead of 1/102. This is similar to specifying `n=4` without replacement on a Series with 3 elements.

Examples

-----  
 >>> df = pd.DataFrame(  
 ... {  
 ... "num\_legs": [2, 4, 8, 0],  
 ... "num\_wings": [2, 0, 0, 0],  
 ... "num\_specimen\_seen": [10, 2, 1, 8],  
 ... },  
 ... index=["falcon", "dog", "spider", "fish"],  
 ... )  
 >>> df

	num_legs	num_wings	num_specimen_seen
falcon	2	2	10
dog	4	0	2
spider	8	0	1
fish	0	0	8

Extract 3 random elements from the ``Series`` ``df['num\_legs']``:  
 Note that we use `random\_state` to ensure the reproducibility of the examples.

```
>>> df["num_legs"].sample(n=3, random_state=1)
fish 0
spider 8
falcon 2
Name: num_legs, dtype: int64
```

A random 50% sample of the ``DataFrame`` with replacement:

```
>>> df.sample(frac=0.5, replace=True, random_state=1)
 num_legs num_wings num_specimen_seen
dog 4 0 2
fish 0 0 8
```

An upsample sample of the ``DataFrame`` with replacement:  
 Note that `replace` parameter has to be `True` for `frac` parameter > 1.

```
>>> df.sample(frac=2, replace=True, random_state=1)
 num_legs num_wings num_specimen_seen
dog 4 0 2
fish 0 0 8
falcon 2 2 10
falcon 2 2 10
fish 0 0 8
dog 4 0 2
fish 0 0 8
dog 4 0 2
```

Using a DataFrame column as weights. Rows with larger value in the `num\_specimen\_seen` column are more likely to be sampled.

```
>>> df.sample(n=2, weights="num_specimen_seen", random_state=1)
 num_legs num_wings num_specimen_seen
falcon 2 2 10
fish 0 0 8
```

```
set_flags(
 self,
 *,
 copy: bool | lib.NoDefault = <no_default>,
 allows_duplicate_labels: bool | None = None
) -> Self
Return a new object with updated flags.
```

This method creates a shallow copy of the original object, preserving its underlying data while modifying its global flags. In particular, it allows you to update properties such as whether duplicate labels are permitted. This behavior is especially useful in method chains, where one wishes to adjust DataFrame or Series characteristics without altering the original object.

#### Parameters

-----

`copy` : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`\_ for more details.

`allows_duplicate_labels` : bool, optional  
Whether the returned object allows duplicate labels.

#### Returns

-----

Series or DataFrame  
The same type as the caller.

#### See Also

-----

`DataFrame.attrs` : Global metadata applying to this dataset.  
`DataFrame.flags` : Global flags applying to this object.

#### Notes

-----

This method returns a new object that's a view on the same data as the input. Mutating the input or the output values will be reflected in the other.

This method is intended to be used in method chains.

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in `:attr:`DataFrame.attrs``.

#### Examples

-----

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags.allows_duplicate_labels
True
>>> df2 = df.set_flags(allows_duplicate_labels=False)
>>> df2.flags.allows_duplicate_labels
False
```

```
squeeze(self, axis: Axis | None = None) -> Scalar | Series | DataFrame
Squeeze 1 dimensional axis objects into scalars.
```

Series or DataFrames with a single element are squeezed to a scalar. DataFrames with a single column or a single row are squeezed to a Series. Otherwise the object is unchanged.

This method is most useful when you don't know if your object is a Series or DataFrame, but you do know it has just a single column. In that case you can safely call `'squeeze'` to ensure you have a Series.

#### Parameters

`axis` : {0 or 'index', 1 or 'columns', None}, default None  
A specific axis to squeeze. By default, all length-1 axes are squeezed. For `'Series'` this parameter is unused and defaults to `'None'`.

#### Returns

DataFrame, Series, or scalar  
The projection after squeezing `'axis'` or all the axes.

#### See Also

`Series.iloc` : Integer-location based indexing for selecting scalars.  
`DataFrame.iloc` : Integer-location based indexing for selecting Series.  
`Series.to_frame` : Inverse of `DataFrame.squeeze` for a single-column DataFrame.

#### Examples

```
>>> primes = pd.Series([2, 3, 5, 7])
```

Slicing might produce a Series with a single value:

```
>>> even_primes = primes[primes % 2 == 0]
>>> even_primes
0 2
dtype: int64
```

```
>>> even_primes.squeeze()
np.int64(2)
```

Squeezing objects with more than one value in every axis does nothing:

```
>>> odd_primes = primes[primes % 2 == 1]
>>> odd_primes
1 3
2 5
3 7
dtype: int64
```

```
>>> odd_primes.squeeze()
1 3
2 5
3 7
dtype: int64
```

Squeezing is even more effective when used with DataFrames.

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["a", "b"])
>>> df
 a b
0 1 2
1 3 4
```

Slicing a single column will produce a DataFrame with the columns having only one value:

```
>>> df_a = df[["a"]]
>>> df_a
 a
0 1
1 3
```

So the columns can be squeezed down, resulting in a Series:

```
>>> df_a.squeeze("columns")
0 1
1 3
Name: a, dtype: int64
```

Slicing a single row from a single column will produce a single scalar DataFrame:

```
>>> df_0a = df.loc[df.index < 1, ["a"]]
>>> df_0a
 a
0 1
```

Squeezing the rows produces a single scalar Series:

```
>>> df_0a.squeeze("rows")
a 1
Name: 0, dtype: int64
```

Squeezing all axes will project directly into a scalar:

```
>>> df_0a.squeeze()
np.int64(1)
```

`tail(self, n: int = 5) -> Self`  
Return the last `n` rows.

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to `df[|n|:]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

-----  
`n` : int, default 5  
Number of rows to select.

Returns

-----  
type of caller  
The last `n` rows of the caller object.

See Also

-----  
`DataFrame.head` : The first `n` rows of the caller object.

Examples

```
>>> df = pd.DataFrame(
... {
... "animal": [
... "alligator",
... "bee",
... "falcon",
... "lion",
... "monkey",
... "parrot",
... "shark",
... "whale",
... "zebra",
...]
... }
...)
>>> df
 animal
0 alligator
1 bee
2 falcon
3 lion
4 monkey
5 parrot
6 shark
7 whale
8 zebra
```

Viewing the last 5 lines

```
>>> df.tail()
```



```

 animal
4 monkey
5 parrot
6 shark
7 whale
8 zebra

```

Viewing the last `n` lines (three in this case)

```

>>> df.tail(3)
 animal
6 shark
7 whale
8 zebra

```

For negative values of `n`

```

>>> df.tail(-3)
 animal
3 lion
4 monkey
5 parrot
6 shark
7 whale
8 zebra

```

`take(self, indices, axis: Axis = 0, **kwargs) -> Self`  
 Return the elements in the given \*positional\* indices along an axis.

This means that we are not indexing according to actual values in the index attribute of the object. We are indexing according to the actual position of the element in the object.

#### Parameters

-----  
**indices** : array-like  
 An array of ints indicating which positions to take.  
**axis** : {0 or 'index', 1 or 'columns'}, default 0  
 The axis on which to select elements. ``0`` means that we are selecting rows, ``1`` means that we are selecting columns.  
 For `Series` this parameter is unused and defaults to 0.  
**\*\*kwargs**  
 For compatibility with :meth:`numpy.take`. Has no effect on the output.

#### Returns

-----  
 same type as caller  
 An array-like containing the elements taken from the object.

#### See Also

-----  
**DataFrame.loc** : Select a subset of a DataFrame by labels.  
**DataFrame.iloc** : Select a subset of a DataFrame by positions.  
**numpy.take** : Take elements from an array along an axis.

#### Examples

```

>>> df = pd.DataFrame(
... [
... ("falcon", "bird", 389.0),
... ("parrot", "bird", 24.0),
... ("lion", "mammal", 80.5),
... ("monkey", "mammal", np.nan),
...],
... columns=["name", "class", "max_speed"],
... index=[0, 2, 3, 1],
...)
>>> df
 name class max_speed
0 falcon bird 389.0
2 parrot bird 24.0
3 lion mammal 80.5
1 monkey mammal NaN

```

Take elements at positions 0 and 3 along the axis 0 (default).

Note how the actual indices selected (0 and 1) do not correspond to our selected indices 0 and 3. That's because we are selecting the 0th and 3rd rows, not rows whose indices equal 0 and 3.

```
>>> df.take([0, 3])
 name class max_speed
0 falcon bird 389.0
1 monkey mammal NaN
```

Take elements at indices 1 and 2 along the axis 1 (column selection).

```
>>> df.take([1, 2], axis=1)
 class max_speed
0 bird 389.0
2 bird 24.0
3 mammal 80.5
1 mammal NaN
```

We may take elements using negative integers for positive indices, starting from the end of the object, just like with Python lists.

```
>>> df.take([-1, -2])
 name class max_speed
1 monkey mammal NaN
3 lion mammal 80.5
```

`to_clipboard(self, *, excel: bool = True, sep: str | None = None, **kwargs) -> None`  
Copy object to the system clipboard.

Write a text representation of object to the system clipboard.  
This can be pasted into Excel, for example.

#### Parameters

`excel` : bool, default True  
Produce output in a csv format for easy pasting into excel.

- True, use the provided separator for csv pasting.
- False, write a string representation of the object to the clipboard.

`sep` : str, default ``\t``  
Field delimiter.

`**kwargs`  
These parameters will be passed to `DataFrame.to_csv`.

#### See Also

`DataFrame.to_csv` : Write a `DataFrame` to a comma-separated values (csv) file.  
`read_clipboard` : Read text from clipboard and pass to `read_csv`.

#### Notes

Requirements for your platform.

- Linux : ``xclip``, or ``xsel`` (with ``PyQt4`` modules)
- Windows : none
- macOS : none

This method uses the processes developed for the package ``pyperclip``. A solution to render any output string format is given in the examples.

#### Examples

Copy the contents of a `DataFrame` to the clipboard.

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])

>>> df.to_clipboard(sep=",") # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # ,A,B,C
... # 0,1,2,3
... # 1,4,5,6
```

We can omit the index by passing the keyword ``index`` and setting it to false.

```
>>> df.to_clipboard(sep="," index=False) # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # A,B,C
... # 1,2,3
... # 4,5,6
```

Using the original `pyperclip` package for any string output format.

```
.. code-block:: python
```

```
import pyperclip

html = df.style.to_html()
pyperclip.copy(html)
```

```
to_csv(
 self,
 path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
 *,
 sep: str = ',',
 na_rep: str = '',
 float_format: str | Callable | None = None,
 columns: Sequence[Hashable] | None = None,
 header: bool | list[str] = True,
 index: bool = True,
 index_label: IndexLabel | None = None,
 mode: str = 'w',
 encoding: str | None = None,
 compression: CompressionOptions = 'infer',
 quoting: int | None = None,
 quotechar: str = '"',
 lineterminator: str | None = None,
 chunksize: int | None = None,
 date_format: str | None = None,
 doublequote: bool = True,
 escapechar: str | None = None,
 decimal: str = '.',
 errors: OpenFileErrors = 'strict',
 storage_options: StorageOptions | None = None
) -> str | None
Write object to a comma-separated values (csv) file.
```

#### Parameters

```

path_or_buf : str, path object, file-like object, or None, default None
 String, path object (implementing os.PathLike[str]), or file-like
 object implementing a write() function. If None, the result is
 returned as a string. If a non-binary file object is passed, it should
 be opened with `newline=''`, disabling universal newlines. If a binary
 file object is passed, `mode` might need to contain a `b`.
sep : str, default ','
 String of length 1. Field delimiter for the output file.
na_rep : str, default ''
 Missing data representation.
float_format : str, Callable, default None
 Format string for floating point numbers. If a Callable is given, it takes
 precedence over other numeric formatting parameters, like decimal.
columns : sequence, optional
 Columns to write.
header : bool or list of str, default True
 Write out the column names. If a list of strings is given it is
 assumed to be aliases for the column names.
index : bool, default True
 Write row names (index).
index_label : str or sequence, or False, default None
 Column label for index column(s) if desired. If None is given, and
 `header` and `index` are True, then the index names are used. A
 sequence should be given if the object uses MultiIndex. If
 False do not print fields for index names. Use index_label=False
 for easier importing in R.
mode : {{'w', 'x', 'a'}}, default 'w'
 Forwarded to either `open(mode=)` or `fsspec.open(mode=)` to control
 the file opening. Typical values include:

 - 'w', truncate the file first.
 - 'x', exclusive creation, failing if the file already exists.
 - 'a', append to the end of file if it exists.
```

encoding : str, optional  
A string representing the encoding to use in the output file, defaults to 'utf-8'. `encoding` is not supported if `path\_or\_buf` is a non-binary file object.

compression : str or dict, default 'infer'  
For on-the-fly compression of the output data. If 'infer' and 'path\_or\_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). Set to `None` for no compression. Can also be a dict with key `method` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively. As an example, the following could be passed for faster compression and to create a reproducible gzip archive:  
`compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}`.

May be a dict with key 'method' as compression mode and other entries as additional compression options if compression mode is 'zip'.

Passing compression options as keys in dict is supported for compression modes 'gzip', 'bz2', 'zstd', and 'zip'.

quoting : optional constant from csv module  
Defaults to csv.QUOTE\_MINIMAL. If you have set a `float\_format` then floats are converted to strings and thus csv.QUOTE\_NONNUMERIC will treat them as non-numeric.

quotechar : str, default '\"'  
String of length 1. Character used to quote fields.

lineterminator : str, optional  
The newline character or character sequence to use in the output file. Defaults to `os.linesep`, which depends on the OS in which this method is called ('\\n' for linux, '\\r\\n' for Windows, i.e.).

chunksize : int or None  
Rows to write at a time.

date\_format : str, default None  
Format string for datetime objects.

doublequote : bool, default True  
Control quoting of `quotechar` inside a field.

escapechar : str, default None  
String of length 1. Character used to escape `sep` and `quotechar` when appropriate.

decimal : str, default '.'  
Character recognized as decimal separator. E.g. use ',' for European data.

errors : str, default 'strict'  
Specifies how encoding and decoding errors are to be handled. See the errors argument for :func:`open` for a full list of options.

storage\_options : dict, optional  
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) <https://pandas.pydata.org/docs/user\_guide/io.html?highlight=storage\_options#reading-writing-remote-files>`\_.

Returns  
-----  
None or str  
If path\_or\_buf is None, returns the resulting csv format as a string. Otherwise returns None.

See Also  
-----  
read\_csv : Load a CSV file into a DataFrame.  
to\_excel : Write DataFrame to an Excel file.

## Examples

-----

Create 'out.csv' containing 'df' without indices

```
>>> df = pd.DataFrame(
... [["Raphael", "red", "sai"], ["Donatello", "purple", "bo staff"]],
... columns=["name", "mask", "weapon"],
...)
>>> df.to_csv("out.csv", index=False) # doctest: +SKIP
```

Create 'out.zip' containing 'out.csv'

```
>>> df.to_csv(index=False)
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
>>> compression_opts = dict(
... method="zip", archive_name="out.csv"
...) # doctest: +SKIP
>>> df.to_csv(
... "out.zip", index=False, compression=compression_opts
...) # doctest: +SKIP
```

To write a csv file to a new folder or nested folder you will first need to create it using either Pathlib or os:

```
>>> from pathlib import Path # doctest: +SKIP
>>> filepath = Path("folder/subfolder/out.csv") # doctest: +SKIP
>>> filepath.parent.mkdir(parents=True, exist_ok=True) # doctest: +SKIP
>>> df.to_csv(filepath) # doctest: +SKIP
```

```
>>> import os # doctest: +SKIP
>>> os.makedirs("folder/subfolder", exist_ok=True) # doctest: +SKIP
>>> df.to_csv("folder/subfolder/out.csv") # doctest: +SKIP
```

Format floats to two decimal places:

```
>>> df.to_csv("out1.csv", float_format="%.2f") # doctest: +SKIP
```

Format floats using scientific notation:

```
>>> df.to_csv("out2.csv", float_format="{:.2e}").format) # doctest: +SKIP
```

```
to_excel(
 self,
 excel_writer: FilePath | WriteExcelBuffer | ExcelWriter,
 *,
 sheet_name: str = 'Sheet1',
 na_rep: str = '',
 float_format: str | None = None,
 columns: Sequence[Hashable] | None = None,
 header: Sequence[Hashable] | bool = True,
 index: bool = True,
 index_label: IndexLabel | None = None,
 startrow: int = 0,
 startcol: int = 0,
 engine: Literal['openpyxl', 'xlsxwriter'] | None = None,
 merge_cells: bool = True,
 inf_rep: str = 'inf',
 freeze_panes: tuple[int, int] | None = None,
 storage_options: StorageOptions | None = None,
 engine_kwargs: dict[str, Any] | None = None,
 autofilter: bool = False
) -> None
 Write object to an Excel sheet.
```

To write a single object to an Excel .xlsx file it is only necessary to specify a target file name. To write to multiple sheets it is necessary to create an 'ExcelWriter' object with a target file name, and specify a sheet in the file to write to.

Multiple sheets may be written to by specifying unique 'sheet\_name'. With all data written to the file it is necessary to save the changes. Note that creating an 'ExcelWriter' object with a file name that already exists will overwrite the existing file because the default mode is write.

## Parameters

-----

excel\_writer : path-like, file-like, or ExcelWriter object

File path or existing ExcelWriter.  
 sheet\_name : str, default 'Sheet1'  
 Name of sheet which will contain DataFrame.  
 na\_rep : str, default ''  
 Missing data representation.  
 float\_format : str, optional  
 Format string for floating point numbers. For example  
 ``float\_format="%.2f"`` will format 0.1234 to 0.12.  
 columns : sequence or list of str, optional  
 Columns to write.  
 header : bool or list of str, default True  
 Write out the column names. If a list of string is given it is  
 assumed to be aliases for the column names.  
 index : bool, default True  
 Write row names (index).  
 index\_label : str or sequence, optional  
 Column label for index column(s) if desired. If not specified, and  
 `header` and `index` are True, then the index names are used. A  
 sequence should be given if the DataFrame uses MultiIndex.  
 startrow : int, default 0  
 Upper left cell row to dump data frame.  
 startcol : int, default 0  
 Upper left cell column to dump data frame.  
 engine : str, optional  
 Write engine to use, 'openpyxl' or 'xlsxwriter'. You can also set this  
 via the options ``io.excel.xlsx.writer`` or  
 ``io.excel.xlsm.writer``.  
 merge\_cells : bool or 'columns', default False  
 If True, write MultiIndex index and columns as merged cells.  
 If 'columns', merge MultiIndex column cells only.  
 inf\_rep : str, default 'inf'  
 Representation for infinity (there is no native representation for  
 infinity in Excel).  
 freeze\_panes : tuple of int (length 2), optional  
 Specifies the one-based bottommost row and rightmost column that  
 is to be frozen.  
 storage\_options : dict, optional  
 Extra options that make sense for a particular storage connection, e.g.  
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs  
 are forwarded to ``urllib.request.Request`` as header options. For other  
 URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are  
 forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more  
 details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files)  
 <https://pandas.pydata.org/docs/user\_guide/io.html?  
 highlight=storage\_options#reading-writing-remote-files>`\_.  
 engine\_kwargs : dict, optional  
 Arbitrary keyword arguments passed to excel engine.  
 autofilter : bool, default False  
 If True, add automatic filters to all columns.

#### See Also

-----  
 to\_csv : Write DataFrame to a comma-separated values (csv) file.  
 ExcelWriter : Class for writing DataFrame objects into excel sheets.  
 read\_excel : Read an Excel file into a pandas DataFrame.  
 read\_csv : Read a comma-separated values (csv) file into DataFrame.  
 io.formats.style.Styler.to\_excel : Add styles to Excel sheet.

#### Notes

-----  
 For compatibility with :meth:`~DataFrame.to\_csv`,  
 to\_excel serializes lists and dicts to strings before writing.

Once a workbook has been saved it is not possible to write further  
 data without rewriting the whole workbook.

pandas will check the number of rows, columns,  
 and cell character count does not exceed Excel's limitations.  
 All other limitations must be checked by the user.

#### Examples

-----

Create, write to and save a workbook:

```
>>> df1 = pd.DataFrame(
```

```
... [["a", "b"], ["c", "d"]],
... index=["row 1", "row 2"],
... columns=["col 1", "col 2"],
...)
>>> df1.to_excel("output.xlsx") # doctest: +SKIP
```

To specify the sheet name:

```
>>> df1.to_excel("output.xlsx", sheet_name="Sheet_name_1") # doctest: +SKIP
```

If you wish to write to more than one sheet in the workbook, it is necessary to specify an ExcelWriter object:

```
>>> df2 = df1.copy()
>>> with pd.ExcelWriter("output.xlsx") as writer: # doctest: +SKIP
... df1.to_excel(writer, sheet_name="Sheet_name_1")
... df2.to_excel(writer, sheet_name="Sheet_name_2")
```

ExcelWriter can also be used to append to an existing Excel file:

```
>>> with pd.ExcelWriter("output.xlsx", mode="a") as writer: # doctest: +SKIP
... df1.to_excel(writer, sheet_name="Sheet_name_3")
```

To set the library that is used to write the Excel file, you can pass the `engine` keyword (the default engine is automatically chosen depending on the file extension):

```
>>> df1.to_excel("output1.xlsx", engine="xlsxwriter") # doctest: +SKIP
```

```
to_hdf(
 self,
 path_or_buf: FilePath | HDFStore,
 *,
 key: str,
 mode: Literal['a', 'w', 'r+'] = 'a',
 complevel: int | None = None,
 complib: Literal['zlib', 'lzo', 'bzip2', 'blosc'] | None = None,
 append: bool = False,
 format: Literal['fixed', 'table'] | None = None,
 index: bool = True,
 min_itemsize: int | dict[str, int] | None = None,
 nan_rep=None,
 dropna: bool | None = None,
 data_columns: Literal[True] | list[str] | None = None,
 errors: OpenFileErrors = 'strict',
 encoding: str = 'UTF-8'
) -> None
```

Write the contained data to an HDF5 file using HDFStore.

Hierarchical Data Format (HDF) is self-describing, allowing an application to interpret the structure and contents of a file with no outside information. One HDF file can hold a mix of related objects which can be accessed as a group or as individual objects.

In order to add another DataFrame or Series to an existing HDF file please use append mode and a different a key.

.. warning::

One can store a subclass of `DataFrame` or `Series` to HDF5, but the type of the subclass is lost upon storing.

For more information see the :ref:`user guide <io.hdf5>`.

Parameters

```

path_or_buf : str or pandas.HDFStore
 File path or HDFStore object.
key : str
 Identifier for the group in the store.
mode : {'a', 'w', 'r+'}, default 'a'
 Mode to open file:
```

- 'w': write, a new file is created (an existing file with the same name would be deleted).
- 'a': append, an existing file is opened for reading and writing, and if the file does not exist it is created.

```

 - 'r+': similar to 'a', but the file must already exist.
complevel : {0-9}, default None
 Specifies a compression level for data.
 A value of 0 or None disables compression.
complib : {'zlib', 'lzo', 'bzip2', 'blosc'}, default 'zlib'
 Specifies the compression library to be used.
 These additional compressors for Blosc are supported
 (default if no compressor specified: 'blosc:blosclz'):
 {'blosc:blosclz', 'blosc:lz4', 'blosc:lz4hc', 'blosc:snappy',
 'blosc:zlib', 'blosc:zstd'}.
 Specifying a compression library which is not available issues
 a ValueError.
append : bool, default False
 For Table formats, append the input data to the existing.
format : {'fixed', 'table', None}, default 'fixed'
 Possible values:

 - 'fixed': Fixed format. Fast writing/reading. Not-appendable,
 nor searchable.
 - 'table': Table format. Write as a PyTables Table structure
 which may perform worse but allow more flexible operations
 like searching / selecting subsets of the data.
 - If None, pd.get_option('io.hdf.default_format') is checked,
 followed by fallback to "fixed".
index : bool, default True
 Write DataFrame index as a column.
min_itemsize : dict or int, optional
 Map column names to minimum string sizes for columns.
nan_rep : Any, optional
 How to represent null values as str.
 Not allowed with append=True.
dropna : bool, default False, optional
 Remove missing values.
data_columns : list of columns or True, optional
 List of columns to create as indexed data columns for on-disk
 queries, or True to use all columns. By default only the axes
 of the object are indexed. See
 :ref:`Query via data columns<io.hdf5-query-data-columns>`. for
 more information.
 Applicable only to format='table'.
errors : str, default 'strict'
 Specifies how encoding and decoding errors are to be handled.
 See the errors argument for :func:`open` for a full list
 of options.
encoding : str, default "UTF-8"
 Set character encoding.

See Also

read_hdf : Read from HDF file.
DataFrame.to_orc : Write a DataFrame to the binary orc format.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.
DataFrame.to_sql : Write to a SQL table.
DataFrame.to_feather : Write out feather-format for DataFrames.
DataFrame.to_csv : Write out to a csv file.

```

#### Examples

```

>>> df = pd.DataFrame(
... {"A": [1, 2, 3], "B": [4, 5, 6]}, index=["a", "b", "c"]
...) # doctest: +SKIP
>>> df.to_hdf("data.h5", key="df", mode="w") # doctest: +SKIP

```

We can add another object to the same file:

```

>>> s = pd.Series([1, 2, 3, 4]) # doctest: +SKIP
>>> s.to_hdf("data.h5", key="s") # doctest: +SKIP

```

Reading from HDF file:

```

>>> pd.read_hdf("data.h5", "df") # doctest: +SKIP
A B
a 1 4
b 2 5
c 3 6
>>> pd.read_hdf("data.h5", "s") # doctest: +SKIP
0 1

```



```

1 2
2 3
3 4
dtype: int64

to_json(
 self,
 path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
 *,
 orient: Literal['split', 'records', 'index', 'table', 'columns', 'values'] | None = None,
 date_format: str | None = None,
 double_precision: int = 10,
 force_ascii: bool = True,
 date_unit: TimeUnit = 'ms',
 default_handler: Callable[[Any], JSONSerializable] | None = None,
 lines: bool = False,
 compression: CompressionOptions = 'infer',
 index: bool | None = None,
 indent: int | None = None,
 storage_options: StorageOptions | None = None,
 mode: Literal['a', 'w'] = 'w'
) -> str | None
 Convert the object to a JSON string.

 Note NaN's and None will be converted to null and datetime objects
 will be converted to UNIX timestamps.

Parameters

path_or_buf : str, path object, file-like object, or None, default None
 String, path object (implementing os.PathLike[str]), or file-like
 object implementing a write() function. If None, the result is
 returned as a string.
orient : str
 Indication of expected JSON string format.

 * Series:

 - default is 'index'
 - allowed values are: {'split', 'records', 'index', 'table'}.

 * DataFrame:

 - default is 'columns'
 - allowed values are: {'split', 'records', 'index', 'columns',
 'values', 'table'}.

 * The format of the JSON string:

 - 'split' : dict like {'index' -> [index], 'columns' -> [columns],
 'data' -> [values]}
 - 'records' : list like [{column -> value}, ... , {column -> value}]
 - 'index' : dict like {index -> [{column -> value}]}
 - 'columns' : dict like {column -> [{index -> value}]}
 - 'values' : just the values array
 - 'table' : dict like {'schema': [{schema}], 'data': [{data}]}}

 Describing the data, where data component is like ``orient='records'``.

date_format : {{None, 'epoch', 'iso'}}
 Type of date conversion. 'epoch' = epoch milliseconds,
 'iso' = ISO8601. The default depends on the `orient`. For
 ``orient='table'``, the default is 'iso'. For all other orients,
 the default is 'epoch'.

.. deprecated:: 3.0.0
 'epoch' date format is deprecated and will be removed in a future
 version, please use 'iso' instead.

double_precision : int, default 10
 The number of decimal places to use when encoding
 floating point values. The possible maximal value is 15.
 Passing double_precision greater than 15 will raise a ValueError.
force_ascii : bool, default True
 Force encoded string to be ASCII.
date_unit : str, default 'ms' (milliseconds)
 The time unit to encode to, governs timestamp and ISO8601

```

```

precision. One of 's', 'ms', 'us', 'ns' for second, millisecond,
microsecond, and nanosecond respectively.
default_handler : callable, default None
 Handler to call if object cannot otherwise be converted to a
 suitable format for JSON. Should receive a single argument which is
 the object to convert and return a serialisable object.
lines : bool, default False
 If 'orient' is 'records' write out line-delimited json format. Will
 throw ValueError if incorrect 'orient' since others are not
 list-like.

compression : str or dict, default 'infer'
 For on-the-fly compression of the output data. If 'infer' and
 'path_or_buf' is path-like, then detect compression from the following
 extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 Set to ``None`` for no compression.
 Can also be a dict with key ``'method'`` set to one of
 {``'zip'``, ``'gzip'``, ``'bz2'``, ``'zstd'``, ``'xz'``, ``'tar'``} and
 other key-value pairs are forwarded to
 ``zipfile.ZipFile``, ``gzip.GzipFile``,
 ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
 ``tarfile.TarFile``, respectively.
 As an example, the following could be passed for faster compression and
 to create a reproducible gzip archive:
 ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

index : bool or None, default None
 The index is only used when 'orient' is 'split', 'index', 'column',
 or 'table'. Of these, 'index' and 'column' do not support
 ``index=False``. The string 'index' as a column name with empty :class:`Index`
 or if it is 'index' will raise a ``ValueError``.

indent : int, optional
 Length of whitespace used to indent each record.

storage_options : dict, optional
 Extra options that make sense for a particular storage connection, e.g.
 host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
 are forwarded to ``urllib.request.Request`` as header options. For other
 URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are
 forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
 details, and for more examples on storage options refer here
 <https://pandas.pydata.org/docs/user_guide/io.html?
 highlight=storage_options#reading-writing-remote-files>`_.

mode : str, default 'w' (writing)
 Specify the IO mode for output when supplying a path_or_buf.
 Accepted args are 'w' (writing) and 'a' (append) only.
 mode='a' is only supported when lines is True and orient is 'records'.

Returns

None or str
 If path_or_buf is None, returns the resulting json format as a
 string. Otherwise returns None.

See Also

read_json : Convert a JSON string to pandas object.

Notes

The behavior of ``indent=0`` varies from the stdlib, which does not
indent the output but does insert newlines. Currently, ``indent=0``
and the default ``indent=None`` are equivalent in pandas, though this
may change in a future release.

``orient='table'`` contains a 'pandas_version' field under 'schema'.
This stores the version of 'pandas' used in the latest revision of the
schema.

Examples

>>> from json import loads, dumps
>>> df = pd.DataFrame(

```

```

... ["a", "b"], ["c", "d"]],
... index=["row 1", "row 2"],
... columns=["col 1", "col 2"],
...)

>>> result = df.to_json(orient="split")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
 "columns": [
 "col 1",
 "col 2"
],
 "index": [
 "row 1",
 "row 2"
],
 "data": [
 [
 "a",
 "b"
],
 [
 "c",
 "d"
]
]
}}

```

Encoding/decoding a Dataframe using ``'records'`` formatted JSON.  
Note that index labels are not preserved with this encoding.

```

>>> result = df.to_json(orient="records")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
 {{
 "col 1": "a",
 "col 2": "b"
 }},
 {{
 "col 1": "c",
 "col 2": "d"
 }}
]

```

Encoding/decoding a Dataframe using ``'index'`` formatted JSON:

```

>>> result = df.to_json(orient="index")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
 "row 1": {{
 "col 1": "a",
 "col 2": "b"
 }},
 "row 2": {{
 "col 1": "c",
 "col 2": "d"
 }}
}}

```

Encoding/decoding a Dataframe using ``'columns'`` formatted JSON:

```

>>> result = df.to_json(orient="columns")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
 "col 1": {{
 "row 1": "a",
 "row 2": "c"
 }},
 "col 2": {{
 "row 1": "b",
 "row 2": "d"
 }}
}}

```

Encoding/decoding a Dataframe using ``'values'`` formatted JSON:

```
>>> result = df.to_json(orient="values")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
 [
 "a",
 "b"
],
 [
 "c",
 "d"
]
]
```

Encoding with Table Schema:

```
>>> result = df.to_json(orient="table")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
 "schema": {{
 "fields": [
 {{
 "name": "index",
 "type": "string"
 }},
 {{
 "name": "col 1",
 "type": "string"
 }},
 {{
 "name": "col 2",
 "type": "string"
 }}
],
 "primaryKey": [
 "index"
],
 "pandas_version": "1.4.0"
 }},
 "data": [
 {{
 "index": "row 1",
 "col 1": "a",
 "col 2": "b"
 }},
 {{
 "index": "row 2",
 "col 1": "c",
 "col 2": "d"
 }}
]
}}
```

```
to_latex(
 self,
 buf: FilePath | WriteBuffer[str] | None = None,
 *,
 columns: Sequence[Hashable] | None = None,
 header: bool | SequenceNotStr[str] = True,
 index: bool = True,
 na_rep: str = 'NaN',
 formatters: FormattersType | None = None,
 float_format: FloatFormatType | None = None,
 sparsify: bool | None = None,
 index_names: bool = True,
 bold_rows: bool = False,
 column_format: str | None = None,
 longtable: bool | None = None,
 escape: bool | None = None,
 encoding: str | None = None,
 decimal: str = '.',
 multicolumn: bool | None = None,
 multicolumn_format: str | None = None,
```

```

multirow: bool | None = None,
caption: str | tuple[str, str] | None = None,
label: str | None = None,
position: str | None = None
) -> str | None
 Render object to a LaTeX tabular, longtable, or nested table.

Requires ``\usepackage{booktabs}``. The output can be copy/pasted
into a main LaTeX document or read from an external file
with ``\input{table.tex}``.

.. versionchanged:: 2.0.0
 Refactored to use the Styler implementation via jinja2 templating.

Parameters

buf : str, Path or StringIO-like, optional, default None
 Buffer to write to. If None, the output is returned as a string.
columns : list of label, optional
 The subset of columns to write. Writes all columns by default.
header : bool or list of str, default True
 Write out the column names. If a list of strings is given,
 it is assumed to be aliases for the column names. Braces must be escaped.
index : bool, default True
 Write row names (index).
na_rep : str, default 'NaN'
 Missing data representation.
formatters : list of functions or dict of {str: function}, optional
 Formatter functions to apply to columns' elements by position or
 name. The result of each function must be a unicode string.
 List must be of length equal to the number of columns.
float_format : one-parameter function or str, optional, default None
 Formatter for floating point numbers. For example
 ``float_format="%.2f"`` and ``float_format="{:0.2f}".format`` will
 both result in 0.1234 being formatted as 0.12.
sparsify : bool, optional
 Set to False for a DataFrame with a hierarchical index to print
 every multiindex key at each row. By default, the value will be
 read from the config module.
index_names : bool, default True
 Prints the names of the indexes.
bold_rows : bool, default False
 Make the row labels bold in the output.
column_format : str, optional
 The columns format as specified in `LaTeX table format`
 <https://en.wikibooks.org/wiki/LaTeX/Tables>`__ e.g. 'rcl' for 3
 columns. By default, 'l' will be used for all columns except
 columns of numbers, which default to 'r'.
longtable : bool, optional
 Use a longtable environment instead of tabular. Requires
 adding a \usepackage{longtable} to your LaTeX preamble.
 By default, the value will be read from the pandas config
 module, and set to `True` if the option ``styler.latex.environment`` is
 ``"longtable"``.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed.
escape : bool, optional
 By default, the value will be read from the pandas config
 module and set to `True` if the option ``styler.format.escape`` is
 ``"latex"``. When set to False prevents from escaping latex special
 characters in column names.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed, as has the
 default value to `False`.
encoding : str, optional
 A string representing the encoding to use in the output file,
 defaults to 'utf-8'.
decimal : str, default '.'
 Character recognized as decimal separator, e.g. ',' in Europe.
multicolumn : bool, default True
 Use \multicolumn to enhance MultiIndex columns.
 The default will be read from the config module, and is set
 as the option ``styler.sparse.columns``.

.. versionchanged:: 2.0.0

```

The pandas option affecting this argument has changed.  
`multicolumn_format` : str, default 'r'  
 The alignment for multicolumns, similar to `column_format`  
 The default will be read from the config module, and is set as the option  
```styler.latex.multicol_align```.

.. versionchanged:: 2.0.0

The pandas option affecting this argument has changed, as has the
 default value to "r".

`multirow` : bool, default True

Use `\multirow` to enhance MultiIndex rows. Requires adding a
`\usepackage{multirow}` to your LaTeX preamble. Will print
 centered labels (instead of top-aligned) across the contained
 rows, separating groups via clines. The default will be read
 from the pandas config module, and is set as the option
```styler.sparse.index```.

.. versionchanged:: 2.0.0

The pandas option affecting this argument has changed, as has the  
 default value to `True`.

`caption` : str or tuple, optional

Tuple (`full_caption`, `short_caption`),  
 which results in ```\caption[short_caption]{full_caption}```;  
 if a single string is passed, no short caption will be set.

`label` : str, optional

The LaTeX label to be placed inside ```\label{}``` in the output.  
 This is used with ```\ref{}``` in the main ```.tex``` file.

`position` : str, optional

The LaTeX positional argument for tables, to be placed after  
```\begin{}``` in the output.

Returns

str or None

If `buf` is None, returns the result as a string. Otherwise returns None.

See Also

`io.formats.style.Styler.to_latex` : Render a DataFrame to LaTeX
 with conditional formatting.

`DataFrame.to_string` : Render a DataFrame to a console-friendly
 tabular output.

`DataFrame.to_html` : Render a DataFrame as an HTML table.

Notes

As of v2.0.0 this method has changed to use the Styler implementation as
 part of `:meth:`.Styler.to_latex`` via ```jinja2``` templating. This means
 that ```jinja2``` is a requirement, and needs to be installed, for this method
 to function. It is advised that users switch to using Styler, since that
 implementation is more frequently updated and contains much more
 flexibility with the output.

Examples

Convert a general DataFrame to LaTeX with formatting:

```
>>> df = pd.DataFrame(dict(name=['Raphael', 'Donatello'],
...                          age=[26, 45],
...                          height=[181.23, 177.65]))
>>> print(df.to_latex(index=False,
...                    formatters={"name": str.upper,
...                                float_format="{:.1f}".format,
...                                }) # doctest: +SKIP
\begin{tabular}{lrr}
\toprule
name & age & height \\
\midrule
RAPHAEL & 26 & 181.2 \\
DONATELLO & 45 & 177.7 \\
\bottomrule
\end{tabular}
```

```
to_pickle(
    self,
    path: FilePath | WriteBuffer[bytes],
```

```

*,
compression: CompressionOptions = 'infer',
protocol: int = 5,
storage_options: StorageOptions | None = None
) -> None
Pickle (serialize) object to file.

Parameters
-----
path : str, path object, or file-like object
    String, path object (implementing ``os.PathLike[str]``), or file-like
    object implementing a binary ``write()`` function. File path where
    the pickled object will be stored.

compression : str or dict, default 'infer'
    For on-the-fly compression of the output data. If 'infer' and
    'path_or_buf' is path-like, then detect compression from the following
    extensions: '.gz',
    '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
    (otherwise no compression).
    Set to ``None`` for no compression.
    Can also be a dict with key ``'method'`` set to one of
    {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and
    other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for faster compression and
    to create a reproducible gzip archive:
    ``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

protocol : int
    Int which indicates which protocol should be used by the pickler,
    default HIGHEST_PROTOCOL (see [1]_ paragraph 12.1.2). The possible
    values are 0, 1, 2, 3, 4, 5. A negative value for the protocol
    parameter is equivalent to setting its value to HIGHEST_PROTOCOL.

    .. [1] https://docs.python.org/3/library/pickle.html.

storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://" and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user\_guide/io.html?highlight=storage\_options#reading-writing-remote-files>`_.

See Also
-----
read_pickle : Load pickled pandas object (or any object) from file.
DataFrame.to_hdf : Write DataFrame to an HDF5 file.
DataFrame.to_sql : Write DataFrame to a SQL database.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.

Examples
-----
>>> original_df = pd.DataFrame(
...     {"foo": range(5), "bar": range(5, 10)}
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8
4     4    9
>>> original_df.to_pickle("./dummy.pkl") # doctest: +SKIP

>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
   foo  bar
0     0    5
1     1    6
2     2    7
3     3    8

```

```

to_sql(
    self,
    name: str,
    con,
    *,
    schema: str | None = None,
    if_exists: Literal['fail', 'replace', 'append', 'delete_rows'] = 'fail',
    index: bool = True,
    index_label: IndexLabel | None = None,
    chunksize: int | None = None,
    dtype: DtypeArg | None = None,
    method: Literal['multi'] | Callable | None = None
) -> int | None

```

Write records stored in a DataFrame to a SQL database.

Databases supported by SQLAlchemy [1]_ are supported. Tables can be newly created, appended to, or overwritten.

.. warning::

The pandas library does not attempt to sanitize inputs provided via a `to_sql` call. Please refer to the documentation for the underlying database driver to see if it will properly prevent injection, or alternatively be advised of a security risk when executing arbitrary commands in a `to_sql` call.

Parameters

`name` : str

Name of SQL table.

`con` : ADBC connection, `sqlalchemy.engine.(Engine or Connection)` or `sqlite3.Connection`
 ADBC provides high performance I/O with native type support, where available.

Using SQLAlchemy makes it possible to use any DB supported by that library. Legacy support is provided for `sqlite3.Connection` objects. The user is responsible for engine disposal and connection closure for the SQLAlchemy connectable. See [here](https://docs.sqlalchemy.org/en/20/core/connections.html) <https://docs.sqlalchemy.org/en/20/core/connections.html>

If passing a `sqlalchemy.engine.Connection` which is already in a transaction, the transaction will not be committed. If passing a `sqlite3.Connection`, it will not be possible to roll back the record insertion.

`schema` : str, optional

Specify the schema (if database flavor supports this). If None, use default schema.

`if_exists` : {'fail', 'replace', 'append', 'delete_rows'}, default 'fail'
 How to behave if the table already exists.

* fail: Raise a `ValueError`.

* replace: Drop the table before inserting new values.

* append: Insert new values to the existing table.

* delete_rows: If a table exists, delete all records and insert data.

`index` : bool, default True

Write DataFrame index as a column. Uses `index_label` as the column name in the table. Creates a table index for this column.

`index_label` : str or sequence, default None

Column label for index column(s). If None is given (default) and `index` is True, then the index names are used.

A sequence should be given if the DataFrame uses `MultiIndex`.

`chunksize` : int, optional

Specify the number of rows in each batch to be written to the database connection at a time. By default, all rows will be written at once. Also see the `method` keyword.

`dtype` : dict or scalar, optional

Specifying the datatype for columns. If a dictionary is used, the keys should be the column names and the values should be the SQLAlchemy types or strings for the `sqlite3` legacy mode. If a scalar is provided, it will be applied to all columns.

`method` : {None, 'multi', callable}, optional

Controls the SQL insertion clause used:

* None : Uses standard SQL `INSERT` clause (one per row).

* 'multi': Pass multiple values in a single `INSERT` clause.

* callable with signature ```(pd_table, conn, keys, data_iter)```.

Details and a sample callable implementation can be found in the [section :ref:`insert method <io.sql.method>`](#).

Returns


```

-----
None or int
    Number of rows affected by to_sql. None is returned if the callable
    passed into ``method`` does not return an integer number of rows.

    The number of returned rows affected is the sum of the ``rowcount``
    attribute of ``sqlite3.Cursor`` or SQLAlchemy connectable which may not
    reflect the exact number of written rows as stipulated in the
    `sqlite3 <https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.rowcount>`
    `SQLAlchemy <https://docs.sqlalchemy.org/en/20/core/connections.html#sqlalchemy.engine`

Raises
-----
ValueError
    When the table already exists and `if_exists` is 'fail' (the
    default).

See Also
-----
read_sql : Read a DataFrame from a table.

Notes
-----
Timezone aware datetime columns will be written as
`Timestamp with timezone` type with SQLAlchemy if supported by the
database. Otherwise, the datetimes will be stored as timezone unaware
timestamps local to the original timezone.

Not all datastores support ``method="multi"``. Oracle, for example,
does not support multi-value insert.

References
-----
.. [1] https://docs.sqlalchemy.org
.. [2] https://www.python.org/dev/peps/pep-0249/

Examples
-----
Create an in-memory SQLite database.

>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite://', echo=False)

Create a table from scratch with 3 rows.

>>> df = pd.DataFrame({'name' : ['User 1', 'User 2', 'User 3']})
>>> df
   name
0  User 1
1  User 2
2  User 3

>>> df.to_sql(name='users', con=engine)
3
>>> from sqlalchemy import text
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3')]

An `sqlalchemy.engine.Connection` can also be passed to `con`:

>>> with engine.begin() as connection:
...     df1 = pd.DataFrame({'name' : ['User 4', 'User 5']})
...     df1.to_sql(name='users', con=connection, if_exists='append')
2

This is allowed to support operations that require that the same
DBAPI connection is used for the entire operation.

>>> df2 = pd.DataFrame({'name' : ['User 6', 'User 7']})
>>> df2.to_sql(name='users', con=engine, if_exists='append')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3'),
 (0, 'User 4'), (1, 'User 5'), (0, 'User 6'),
 (1, 'User 7')]

```

Overwrite the table with just ``df2``.

```
>>> df2.to_sql(name='users', con=engine, if_exists='replace',
...           index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 6'), (1, 'User 7')]
```

Delete all rows before inserting new records with ``df3``

```
>>> df3 = pd.DataFrame({"name": ['User 8', 'User 9']})
>>> df3.to_sql(name='users', con=engine, if_exists='delete_rows',
...           index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 8'), (1, 'User 9')]
```

Use ``method`` to define a callable insertion method to do nothing if there's a primary key conflict on a table in a PostgreSQL database.

```
>>> from sqlalchemy.dialects.postgresql import insert
>>> def insert_on_conflict_nothing(table, conn, keys, data_iter):
...     # "a" is the primary key in "conflict_table"
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = insert(table.table).values(data).on_conflict_do_nothing(index_elements=["a"])
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F821
...                   method=insert_on_conflict_nothing) # doctest: +SKIP
0
```

For MySQL, a callable to update columns ``b`` and ``c`` if there's a conflict on a primary key.

```
>>> from sqlalchemy.dialects.mysql import insert # noqa: F811
>>> def insert_on_conflict_update(table, conn, keys, data_iter):
...     # update columns "b" and "c" on primary key conflict
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = (
...         insert(table.table)
...         .values(data)
...     )
...     stmt = stmt.on_duplicate_key_update(b=stmt.inserted.b, c=stmt.inserted.c)
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F821
...                   method=insert_on_conflict_update) # doctest: +SKIP
2
```

Specify the dtype (especially useful for integers with missing values). Notice that while pandas is forced to store the data as floating point, the database supports nullable integers. When fetching the data with Python, we get back integer scalars.

```
>>> df = pd.DataFrame({"A": [1, None, 2]})
>>> df
   A
0  1.0
1  NaN
2  2.0
```

```
>>> from sqlalchemy.types import Integer
>>> df.to_sql(name='integers', con=engine, index=False,
...         dtype={"A": Integer()})
3
```

```
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM integers")).fetchall()
[(1,), (None,), (2,)]
```

.. versionadded:: 2.2.0

pandas now supports writing via ADBC drivers

```

>>> df = pd.DataFrame({'name' : ['User 10', 'User 11', 'User 12']})
>>> df
   name
0  User 10
1  User 11
2  User 12

>>> from adbc_driver_sqlite import dbapi # doctest:+SKIP
>>> with dbapi.connect("sqlite://") as conn: # doctest:+SKIP
...     df.to_sql(name="users", con=conn)
3

```

`to_xarray(self)`
Return an xarray object from the pandas object.

Returns

xarray.DataArray or xarray.Dataset
Data in the pandas structure converted to Dataset if the object is a DataFrame, or a DataArray if the object is a Series.

See Also

DataFrame.to_hdf : Write DataFrame to an HDF5 file.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.

Notes

See the `xarray docs <<https://xarray.pydata.org/en/stable/>>`__

Examples

```

>>> df = pd.DataFrame(
...     [
...         ("falcon", "bird", 389.0, 2),
...         ("parrot", "bird", 24.0, 2),
...         ("lion", "mammal", 80.5, 4),
...         ("monkey", "mammal", np.nan, 4),
...     ],
...     columns=["name", "class", "max_speed", "num_legs"],
... )
>>> df
   name  class  max_speed  num_legs
0  falcon   bird     389.0         2
1  parrot   bird     24.0         2
2    lion  mammal     80.5         4
3  monkey  mammal      NaN         4

```

```

>>> df.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (index: 4)
Coordinates:
  * index    (index) int64 32B 0 1 2 3
Data variables:
  name      (index) object 32B 'falcon' 'parrot' 'lion' 'monkey'
  class     (index) object 32B 'bird' 'bird' 'mammal' 'mammal'
  max_speed (index) float64 32B 389.0 24.0 80.5 nan
  num_legs  (index) int64 32B 2 2 4 4

```

```

>>> df["max_speed"].to_xarray() # doctest: +SKIP
<xarray.DataArray 'max_speed' (index: 4)>
array([389. , 24. , 80.5,  nan])
Coordinates:
  * index    (index) int64 0 1 2 3

```

```

>>> dates = pd.to_datetime(
...     ["2018-01-01", "2018-01-01", "2018-01-02", "2018-01-02"]
... )
>>> df_multiindex = pd.DataFrame(
...     {
...         "date": dates,
...         "animal": ["falcon", "parrot", "falcon", "parrot"],
...         "speed": [350, 18, 361, 15],
...     }
... )
>>> df_multiindex = df_multiindex.set_index(["date", "animal"])

```

```

>>> df_multiindex
      date      animal  speed
2018-01-01  falcon    350
            parrot     18
2018-01-02  falcon    361
            parrot     15

>>> df_multiindex.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:  (date: 2, animal: 2)
Coordinates:
  * date      (date) datetime64[s] 2018-01-01 2018-01-02
  * animal    (animal) object 'falcon' 'parrot'
Data variables:
  speed      (date, animal) int64 350 18 361 15

```

truncate(
self,
before=None,
after=None,
axis: Axis | None = None,
copy: bool | lib.NoDefault = <no_default>
) -> Self

Truncate a Series or DataFrame before and after some index value.

This is a useful shorthand for boolean indexing based on index values above or below certain thresholds.

Parameters

before : date, str, int
Truncate all rows before this index value.

after : date, str, int
Truncate all rows after this index value.

axis : {0 or 'index', 1 or 'columns'}, optional
Axis to truncate. Truncates the index (rows) by default. For `Series` this parameter is unused and defaults to 0.

copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

type of caller
The truncated Series or DataFrame.

See Also

DataFrame.loc : Select a subset of a DataFrame by label.
DataFrame.iloc : Select a subset of a DataFrame by position.

Notes

If the index being truncated contains only datetime values, `before` and `after` may be specified as strings instead of Timestamps.

Examples

```

>>> df = pd.DataFrame(
...     {
...         "A": ["a", "b", "c", "d", "e"],
...         "B": ["f", "g", "h", "i", "j"],
...         "C": ["k", "l", "m", "n", "o"],
...     },
...     index=[1, 2, 3, 4, 5],
... )

```

```
>>> df
   A B C
1  a f k
2  b g l
3  c h m
4  d i n
5  e j o
```

```
>>> df.truncate(before=2, after=4)
   A B C
2  b g l
3  c h m
4  d i n
```

The columns of a DataFrame can be truncated.

```
>>> df.truncate(before="A", after="B", axis="columns")
   A B
1  a f
2  b g
3  c h
4  d i
5  e j
```

For Series, only rows can be truncated.

```
>>> df["A"].truncate(before=2, after=4)
2    b
3    c
4    d
Name: A, dtype: str
```

The index values in ``truncate`` can be datetimes or string dates.

```
>>> dates = pd.date_range("2016-01-01", "2016-02-01", freq="s")
>>> df = pd.DataFrame(index=dates, data={"A": 1})
>>> df.tail()
                A
2016-01-31 23:59:56  1
2016-01-31 23:59:57  1
2016-01-31 23:59:58  1
2016-01-31 23:59:59  1
2016-02-01 00:00:00  1

>>> df.truncate(
...     before=pd.Timestamp("2016-01-05"), after=pd.Timestamp("2016-01-10")
... ).tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Because the index is a DatetimeIndex containing only dates, we can specify `before` and `after` as strings. They will be coerced to Timestamps before truncation.

```
>>> df.truncate("2016-01-05", "2016-01-10").tail()
                A
2016-01-09 23:59:56  1
2016-01-09 23:59:57  1
2016-01-09 23:59:58  1
2016-01-09 23:59:59  1
2016-01-10 00:00:00  1
```

Note that ``truncate`` assumes a 0 value for any unspecified time component (midnight). This differs from partial string slicing, which returns any partially matching dates.

```
>>> df.loc["2016-01-05":"2016-01-10", :].tail()
                A
2016-01-10 23:59:55  1
2016-01-10 23:59:56  1
2016-01-10 23:59:57  1
2016-01-10 23:59:58  1
```

2016-01-10 23:59:59 1

```
tz_convert(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self
    Convert tz-aware axis to target time zone.

Parameters
-----
tz : str or tzinfo object or None
    Target time zone. Passing ``None`` will convert to
    UTC and remove the timezone information.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
    The axis to convert
level : int, str, default None
    If axis is a MultiIndex, convert a specific level. Otherwise
    must be None.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
    for more details.

Returns
-----
Series/DataFrame
    Object with time zone converted axis.

Raises
-----
TypeError
    If the axis is tz-naive.

See Also
-----
DataFrame.tz_localize: Localize tz-naive index of DataFrame to target time zone.
Series.tz_localize: Localize tz-naive index of Series to target time zone.

Examples
-----
Change to another time zone:

>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]),
... )
>>> s.tz_convert("Asia/Shanghai")
2018-09-15 07:30:00+08:00    1
dtype: int64

Pass None to convert to UTC and get a tz-naive index:

>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_convert(None)
2018-09-14 23:30:00    1
dtype: int64

tz_localize(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>,
    ambiguous: TimeAmbiguous = 'raise',
    nonexistent: TimeNonexistent = 'raise'
) -> Self
```

Localize time zone naive index of a Series or DataFrame to target time zone.

This operation localizes the Index. To localize the values in a time zone naive Series, use :meth:`Series.dt.tz\_localize`.

Parameters

`tz` : str or tzinfo or None
Time zone to localize. Passing ``None`` will remove the time zone information and preserve local time.

`axis` : {{0 or 'index', 1 or 'columns'}} , default 0
The axis to localize

`level` : int, str, default None
If axis is a MultiIndex, localize a specific level. Otherwise must be None.

`copy` : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`__ for more details.

`ambiguous` : 'infer', bool, bool-ndarray, 'NaT', default 'raise'
When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool (or bool-ndarray) where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

`nonexistent` : str, default 'raise'
A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST. Valid values are:

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

Series/DataFrame
Same type as the input, with time zone naive or aware index, depending on ``tz``.

Raises

TypeError
If the TimeSeries is tz-aware and tz is not None.

See Also

`Series.dt.tz_localize`: Localize the values in a time zone naive Series.
`Timestamp.tz_localize`: Localize the Timestamp to a timezone.

Examples

Localize local times:

```
>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00"]),
... )
>>> s.tz_localize("CET")
2018-09-15 01:30:00+02:00    1
dtype: int64
```

Pass None to convert to tz-naive index and preserve local time:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_localize(None)
2018-09-15 01:30:00    1
dtype: int64
```

Be careful with DST changes. When there is sequential data, pandas can infer the DST time:

```
>>> s = pd.Series(
...     range(7),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 03:00:00",
...             "2018-10-28 03:30:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous="infer")
2018-10-28 01:30:00+02:00    0
2018-10-28 02:00:00+02:00    1
2018-10-28 02:30:00+02:00    2
2018-10-28 02:00:00+01:00    3
2018-10-28 02:30:00+01:00    4
2018-10-28 03:00:00+01:00    5
2018-10-28 03:30:00+01:00    6
dtype: int64
```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```
>>> s = pd.Series(
...     range(3),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:20:00",
...             "2018-10-28 02:36:00",
...             "2018-10-28 03:46:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous=np.array([True, True, False]))
2018-10-28 01:20:00+02:00    0
2018-10-28 02:36:00+02:00    1
2018-10-28 03:46:00+01:00    2
dtype: int64
```

If the DST transition causes nonexistent times, you can shift these dates forward or backward with a timedelta object or `'shift_forward'` or `'shift_backward'`.

```
>>> dti = pd.DatetimeIndex(
...     ["2015-03-29 02:30:00", "2015-03-29 03:30:00"], dtype="M8[ns]"
... )
>>> s = pd.Series(range(2), index=dti)
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_forward")
2015-03-29 03:00:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_backward")
2015-03-29 01:59:59.999999999+01:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
```



```
>>> s.tz_localize("Europe/Warsaw", nonexistent=pd.Timedelta("1h"))
2015-03-29 03:30:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
```

```
where(
    self,
    cond,
    other=<no_default>,
    *,
    inplace: bool = False,
    axis: Axis | None = None,
    level: Level | None = None
) -> Self
Replace values where the condition is False.
```

This method allows conditional replacement of values. Where the condition evaluates to True, the original values are retained; where it evaluates to False, values are replaced with corresponding entries from ``other``.

Parameters

```
cond : bool Series/DataFrame, array-like, or callable
    Where ``cond`` is True, keep the original value. Where
    False, replace with corresponding value from ``other``.
    If ``cond`` is callable, it is computed on the Series/DataFrame and
    should return boolean Series/DataFrame or array. The callable must
    not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
    Entries where ``cond`` is False are replaced with
    corresponding value from ``other``.
    If other is callable, it is computed on the Series/DataFrame and
    should return scalar or Series/DataFrame. The callable must not
    change input Series/DataFrame (though pandas doesn't check it).
    If not specified, entries will be filled with the corresponding
    NULL value (``np.nan`` for numpy dtypes, ``pd.NA`` for extension
    dtypes).
inplace : bool, default False
    Whether to perform the operation in place on the data.
axis : int, default None
    Alignment axis if needed. For ``Series`` this parameter is
    unused and defaults to 0.
level : int, default None
    Alignment level if needed.
```

Returns

```
Series or DataFrame
    When applied to a Series, the function will return a Series,
    and when applied to a DataFrame, it will return a DataFrame.
```

See Also

```
:func: `DataFrame.mask` : Return an object of same shape as caller.
:func: `Series.mask` : Return an object of same shape as caller.
```

Notes

The where method is an application of the if-then idiom. For each element in the caller, if ``cond`` is ``True`` the element is used; otherwise the corresponding element from ``other`` is used. If the axis of ``other`` does not align with axis of ``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions will be filled with False.

The signature for :func: ``Series.where`` or :func: ``DataFrame.where`` differs from :func: ``numpy.where``. Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

For further details and examples see the ``where`` documentation in :ref: `indexing <indexing.where\_mask>`.

The dtype of the object takes precedence. The fill value is casted to the object's dtype, if this can be done losslessly.

Examples

```

-----
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0    NaN
1    1.0
2    2.0
3    3.0
4    4.0
dtype: float64
>>> s.mask(s > 0)
0    0.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0    0
1    99
2    99
3    99
4    99
dtype: int64
>>> s.mask(t, 99)
0    99
1    1
2    99
3    99
4    99
dtype: int64

>>> s.where(s > 1, 10)
0    10
1    10
2    2
3    3
4    4
dtype: int64
>>> s.mask(s > 1, 10)
0    0
1    1
2    10
3    10
4    10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
   A  B
0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A  B
0  True  True
1  True  True
2  True  True

```

```
3 True True
4 True True
```

```
xs(
    self,
    key: IndexLabel,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    drop_level: bool = True
) -> Self
Return cross-section from the Series/DataFrame.

This method takes a `key` argument to select data at a particular
level of a MultiIndex.

Parameters
-----
key : label or tuple of label
    Label contained in the index, or partially in a MultiIndex.
axis : {0 or 'index', 1 or 'columns'}, default 0
    Axis to retrieve cross-section on.
level : object, defaults to first n levels (n=1 or len(key))
    In case of a key partially contained in a MultiIndex, indicate
    which levels are used. Levels can be referred by label or position.
drop_level : bool, default True
    If False, returns object with same levels as self.
```

Returns

```
-----
Series or DataFrame
Cross-section from the original Series or DataFrame
corresponding to the selected index levels.
```

See Also

```
-----
DataFrame.loc : Access a group of rows and columns
    by label(s) or a boolean array.
DataFrame.iloc : Purely integer-location based indexing
    for selection by position.
```

Notes

```
-----
`xs` can not be used to set values.
```

MultiIndex Slicers is a generic way to get/set values on any level or levels. It is a superset of `xs` functionality, see :ref:`MultiIndex Slicers <advanced.mi\_slicers>`.

Examples

```
-----
>>> d = {
...     "num_legs": [4, 4, 2, 2],
...     "num_wings": [0, 0, 2, 2],
...     "class": ["mammal", "mammal", "mammal", "bird"],
...     "animal": ["cat", "dog", "bat", "penguin"],
...     "locomotion": ["walks", "walks", "flies", "walks"],
... }
>>> df = pd.DataFrame(data=d)
>>> df = df.set_index(["class", "animal", "locomotion"])
>>> df
```

			num_legs	num_wings
class	animal	locomotion		
mammal	cat	walks	4	0
	dog	walks	4	0
	bat	flies	2	2
bird	penguin	walks	2	2

Get values at specified index

```
>>> df.xs("mammal")
          num_legs  num_wings
animal locomotion
cat      walks      4         0
dog      walks      4         0
bat      flies      2         2
```

Get values at several indexes

```
>>> df.xs(("mammal", "dog", "walks"))
num_legs    4
num_wings    0
Name: (mammal, dog, walks), dtype: int64
```

Get values at specified index and level

```
>>> df.xs("cat", level=1)
               num_legs  num_wings
class locomotion
mammal walks           4          0
```

Get values at several indexes and levels

```
>>> df.xs(("bird", "walks"), level=[0, "locomotion"])
               num_legs  num_wings
animal
penguin           2          2
```

Get values at specified column and axis

```
>>> df.xs("num_wings", axis=1)
class  animal  locomotion
mammal  cat    walks      0
         dog    walks      0
         bat    flies      2
bird    penguin walks      2
Name: num_wings, dtype: int64
```

Readonly properties inherited from pandas.core.generic.NDFrame:

dtypes

Return the dtypes in the DataFrame.

This returns a Series with the data type of each column. The result's index is the original DataFrame's columns. Columns with mixed types are stored with the ``object`` dtype. See :ref:`the User Guide <basics.dtypes>` for more.

Returns

pandas.Series

The data type of each column.

See Also

Series.dtypes : Return the dtype object of the underlying data.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "float": [1.0],
...         "int": [1],
...         "datetime": [pd.Timestamp("20180310")],
...         "string": ["foo"],
...     }
... )
>>> df.dtypes
float          float64
int            int64
datetime      datetime64[us]
string         str
dtype: object
```

empty

Indicator whether Series/DataFrame is empty.

True if Series/DataFrame is entirely empty (no items), meaning any of the axes are of length 0.

Returns

bool

If Series/DataFrame is empty, return True, if not return False.

See Also

Series.dropna : Return series without null values.
DataFrame.dropna : Return DataFrame with labels on given axis omitted where (all or any) data are missing.

Notes

If Series/DataFrame contains only NaNs, it is still not considered empty. See the example below.

Examples

An example of an actual empty DataFrame. Notice the index is empty:

```
>>> df_empty = pd.DataFrame({"A": []})
>>> df_empty
Empty DataFrame
Columns: [A]
Index: []
>>> df_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty! We will need to drop the NaNs to make the DataFrame empty:

```
>>> df = pd.DataFrame({"A": [np.nan]})
>>> df
   A
0 NaN
>>> df.empty
False
>>> df.dropna().empty
True
```

```
>>> ser_empty = pd.Series({"A": []})
>>> ser_empty
A    []
dtype: object
>>> ser_empty.empty
False
>>> ser_empty = pd.Series()
>>> ser_empty.empty
True
```

flags

Get the properties associated with this pandas object.

The available flags are

* :attr:`Flags.allows\_duplicate\_labels`

See Also

Flags : Flags that apply to pandas objects.
DataFrame.attrs : Global metadata applying to this dataset.

Notes

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in :attr:`DataFrame.attrs`.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags
<Flags(allows_duplicate_labels=True)>
```

Flags can be get or set using ``.``

```
>>> df.flags.allows_duplicate_labels
True
>>> df.flags.allows_duplicate_labels = False
```

Or by slicing with a key

```
>>> df.flags["allows_duplicate_labels"]
False
>>> df.flags["allows_duplicate_labels"] = True
```

ndim

Return an int representing the number of axes / array dimensions.

Return 1 if Series. Otherwise return 2 if DataFrame.

See Also

numpy.ndarray.ndim : Number of array dimensions.

Examples

```
>>> s = pd.Series({"a": 1, "b": 2, "c": 3})
>>> s.ndim
1

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.ndim
2
```

size

Return an int representing the number of elements in this object.

Return the number of rows if Series. Otherwise return the number of rows times number of columns if DataFrame.

See Also

numpy.ndarray.size : Number of elements in the array.

Examples

```
>>> s = pd.Series({"a": 1, "b": 2, "c": 3})
>>> s.size
3

>>> df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
>>> df.size
4
```

Data descriptors inherited from pandas.core.generic.NDFrame:

attrs

Dictionary of global attributes of this dataset.

.. warning::

attrs is experimental and may change without warning.

See Also

DataFrame.flags : Global flags applying to this object.

Notes

Many operations that create new datasets will copy ``attrs``. Copies are always deep so that changing ``attrs`` will only affect the present dataset. :func:`pandas.concat` and :func:`pandas.merge` will only copy ``attrs`` if all input datasets have the same ``attrs``.

Examples

For Series:

```
>>> ser = pd.Series([1, 2, 3])
>>> ser.attrs = {"A": [10, 20, 30]}
>>> ser.attrs
{'A': [10, 20, 30]}
```

For DataFrame:

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.attrs = {"A": [10, 20, 30]}
>>> df.attrs
{'A': [10, 20, 30]}
```

Data and other attributes inherited from pandas.core.generic.NDFrame:

__array_priority__ = 1000

Methods inherited from pandas.core.base.PandasObject:

__sizeof__(self) -> int
Generates the total memory usage for an object that returns
either a value or Series of values

Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]
Provide method name lookup and completion.

Notes

Only provide 'public' methods.

Data descriptors inherited from pandas.core.accessor.DirNamesMixin:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

Readonly properties inherited from pandas.core.indexing.IndexingMixin:

at

Access a single value for a row/column label pair.

Similar to ``loc``, in that both provide label-based lookups. Use
``at`` if you only need to get or set a single value in a DataFrame
or Series.

Raises

KeyError

If getting a value and 'label' does not exist in a DataFrame or Series.

ValueError

If row/column label pair is not a tuple or if any label
from the pair is not a scalar for DataFrame.
If label is list-like (*excluding* NamedTuple) for Series.

See Also

DataFrame.at : Access a single value for a row/column pair by label.
DataFrame.iat : Access a single value for a row/column pair by integer
position.

DataFrame.loc : Access a group of rows and columns by label(s).
DataFrame.iloc : Access a group of rows and columns by integer
position(s).

Series.at : Access a single value by label.
Series.iat : Access a single value by integer position.
Series.loc : Access a group of rows by label(s).
Series.iloc : Access a group of rows by integer position(s).

Notes

See :ref:`Fast scalar value getting and setting <indexing.basics.get\_value>`
for more details.

Examples

>>> df = pd.DataFrame(

```
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]],
...     index=[4, 5, 6],
...     columns=["A", "B", "C"],
... )
>>> df
   A  B  C
4  0  2  3
5  0  4  1
6 10 20 30
```

Get value at specified row/column pair

```
>>> df.at[4, "B"]
np.int64(2)
```

Set value at specified row/column pair

```
>>> df.at[4, "B"] = 10
>>> df.at[4, "B"]
np.int64(10)
```

Get value within a Series

```
>>> df.loc[5].at["B"]
np.int64(4)
```

iat

Access a single value for a row/column pair by integer position.

Similar to ``iloc``, in that both provide integer-based lookups. Use ``iat`` if you only need to get or set a single value in a DataFrame or Series.

Raises

IndexError

When integer position is out of bounds.

See Also

DataFrame.at : Access a single value for a row/column label pair.

DataFrame.loc : Access a group of rows and columns by label(s).

DataFrame.iloc : Access a group of rows and columns by integer position(s).

Examples

```
>>> df = pd.DataFrame(
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]], columns=["A", "B", "C"]
... )
>>> df
   A  B  C
0  0  2  3
1  0  4  1
2 10 20 30
```

Get value at specified row/column pair

```
>>> df.iat[1, 2]
np.int64(1)
```

Set value at specified row/column pair

```
>>> df.iat[1, 2] = 10
>>> df.iat[1, 2]
np.int64(10)
```

Get value within a series

```
>>> df.loc[0].iat[1]
np.int64(2)
```

iloc

Purely integer-location based indexing for selection by position.

.. versionchanged:: 3.0

Callables which return a tuple are deprecated as input.

`df.iloc[]` is primarily integer position based (from `0` to `length-1` of the axis), but may also be used with a boolean array.

Allowed inputs are:

- An integer, e.g. `5`.
- A list or array of integers, e.g. `[4, 3, 0]`.
- A slice object with ints, e.g. `1:7`.
- A boolean array.
- A callable function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above). This is useful in method chains, when you don't have a reference to the calling object, but would like to base your selection on some value.
- A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. `(0, 1)`.

`df.iloc` will raise `IndexError` if a requested indexer is out-of-bounds, except `*slice*` indexers which allow out-of-bounds indexing (this conforms with python/numpy `*slice*` semantics).

See more at :ref:`Selection by Position <indexing.integer>`.

See Also

DataFrame.iat : Fast integer location scalar accessor.
DataFrame.loc : Purely label-location based indexer for selection by label.
Series.iloc : Purely integer-location based indexing for selection by position.

Examples

>>> mydict = [
... {"a": 1, "b": 2, "c": 3, "d": 4},
... {"a": 100, "b": 200, "c": 300, "d": 400},
... {"a": 1000, "b": 2000, "c": 3000, "d": 4000},
...]
>>> df = pd.DataFrame(mydict)
>>> df
 a b c d
0 1 2 3 4
1 100 200 300 400
2 1000 2000 3000 4000

****Indexing just the rows****

With a scalar integer.

```
>>> type(df.iloc[0])  
<class 'pandas.Series'>  
>>> df.iloc[0]  
a    1  
b    2  
c    3  
d    4  
Name: 0, dtype: int64
```

With a list of integers.

```
>>> df.iloc[[0]]  
   a  b  c  d  
0  1  2  3  4  
>>> type(df.iloc[[0]])  
<class 'pandas.DataFrame'>
```

```
>>> df.iloc[[0, 1]]  
   a  b  c  d  
0  1  2  3  4  
1 100 200 300 400
```

With a `'slice'` object.

```
>>> df.iloc[:3]  
   a  b  c  d  
0  1  2  3  4
```

```
1  100   200   300   400
2 1000  2000  3000  4000
```

With a boolean mask the same length as the index.

```
>>> df.iloc[[True, False, True]]
      a      b      c      d
0      1      2      3      4
2 1000  2000  3000  4000
```

With a callable, useful in method chains. The `x` passed to the `lambda` is the DataFrame being sliced. This selects the rows whose index label even.

```
>>> df.iloc[lambda x: x.index % 2 == 0]
      a      b      c      d
0      1      2      3      4
2 1000  2000  3000  4000
```

****Indexing both axes****

You can mix the indexer types for the index and columns. Use `:` to select the entire axis.

With scalar integers.

```
>>> df.iloc[0, 1]
np.int64(2)
```

With lists of integers.

```
>>> df.iloc[[0, 2], [1, 3]]
      b      d
0      2      4
2 2000  4000
```

With `slice` objects.

```
>>> df.iloc[1:3, 0:3]
      a      b      c
1  100   200   300
2 1000  2000  3000
```

With a boolean array whose length matches the columns.

```
>>> df.iloc[:, [True, False, True, False]]
      a      c
0      1      3
1  100   300
2 1000  3000
```

With a callable function that expects the Series or DataFrame.

```
>>> df.iloc[:, lambda df: [0, 2]]
      a      c
0      1      3
1  100   300
2 1000  3000
```

loc

Access a group of rows and columns by label(s) or a boolean array.

`loc[]` is primarily label based, but may also be used with a boolean array.

Allowed inputs are:

- A single label, e.g. `5` or `'a'`, (note that `5` is interpreted as a *label* of the index, and ****never**** as an integer position along the index).
- A list or array of labels, e.g. `['a', 'b', 'c']`.
- A slice object with labels, e.g. `'a':'f'`.

.. warning:: Note that contrary to usual python slices, ****both**** the start and the stop are included

- A boolean array of the same length as the axis being sliced,

```

    e.g. ``[True, False, True]``.
- An alignable boolean Series. The index of the key will be aligned before
  masking.
- An alignable Index. The Index of the returned selection will be the input.
- A ``callable`` function with one argument (the calling Series or
  DataFrame) and that returns valid output for indexing (one of the above)

See more at :ref:`Selection by Label <indexing.label>`.

Raises
-----
KeyError
    If any items are not found.
IndexingError
    If an indexed key is passed and its index is unalignable to the frame index.

See Also
-----
DataFrame.at : Access a single value for a row/column label pair.
DataFrame.iloc : Access group of rows and columns by integer position(s).
DataFrame.xs : Returns a cross-section (row(s) or column(s)) from the
               Series/DataFrame.
Series.loc : Access group of values using labels.

Examples
-----
**Getting values**

>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=["cobra", "viper", "sidewinder"],
...     columns=["max_speed", "shield"],
... )
>>> df
           max_speed  shield
cobra              1       2
viper              4       5
sidewinder         7       8

Single label. Note this returns the row as a Series.

>>> df.loc["viper"]
max_speed    4
shield       5
Name: viper, dtype: int64

List of labels. Note using ``[]`` returns a DataFrame.

>>> df.loc[["viper", "sidewinder"]]
           max_speed  shield
viper              4       5
sidewinder         7       8

Single label for row and column

>>> df.loc["cobra", "shield"]
np.int64(2)

Slice with labels for row and single label for column. As mentioned
above, note that both the start and stop of the slice are included.

>>> df.loc["cobra":"viper", "max_speed"]
cobra      1
viper      4
Name: max_speed, dtype: int64

Boolean list with the same length as the row axis

>>> df.loc[[False, False, True]]
           max_speed  shield
sidewinder         7       8

Alignable boolean Series:

>>> df.loc[
...     pd.Series([False, True, False], index=["viper", "sidewinder", "cobra"])
... ]

```

	max_speed	shield
sidewinder	7	8

Index (same behavior as ``df.reindex``)

```
>>> df.loc[pd.Index(["cobra", "viper"], name="foo")]
      max_speed  shield
foo
cobra         1      2
viper         4      5
```

Conditional that returns a boolean Series

```
>>> df.loc[df["shield"] > 6]
      max_speed  shield
sidewinder     7      8
```

Conditional that returns a boolean Series with column labels specified

```
>>> df.loc[df["shield"] > 6, ["max_speed"]]
      max_speed
sidewinder     7
```

Multiple conditional using ``&`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 1) & (df["shield"] < 8)]
      max_speed  shield
viper         4      5
```

Multiple conditional using ``|`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 4) | (df["shield"] < 5)]
      max_speed  shield
cobra         1      2
sidewinder     7      8
```

Please ensure that each condition is wrapped in parentheses ``()``.

See the :ref:`user guide<indexing.boolean>`

for more details and explanations of Boolean indexing.

.. note::

If you find yourself using 3 or more conditionals in ``.loc[]``, consider using :ref:`advanced indexing<advanced.advanced\_hierarchical>`.

See below for using ``.loc[]`` on MultiIndex DataFrames.

Callable that returns a boolean Series

```
>>> df.loc[lambda df: df["shield"] == 8]
      max_speed  shield
sidewinder     7      8
```

****Setting values****

Set value for all items matching the list of labels

```
>>> df.loc[["viper", "sidewinder"], ["shield"]] = 50
>>> df
      max_speed  shield
cobra         1      2
viper         4     50
sidewinder     7     50
```

Set value for an entire row

```
>>> df.loc["cobra"] = 10
>>> df
      max_speed  shield
cobra        10     10
viper         4     50
sidewinder     7     50
```

Set value for an entire column

```
>>> df.loc[:, "max_speed"] = 30
>>> df
      max_speed  shield
```

cobra	30	10
viper	30	50
sidewinder	30	50

Set value for rows matching callable condition

```
>>> df.loc[df["shield"] > 35] = 0
>>> df
```

	max_speed	shield
cobra	30	10
viper	0	0
sidewinder	0	0

Add value matching location

```
>>> df.loc["viper", "shield"] += 5
>>> df
```

	max_speed	shield
cobra	30	10
viper	0	5
sidewinder	0	0

Setting using a ``Series`` or a ``DataFrame`` sets the values matching the index labels, not the index positions.

```
>>> shuffled_df = df.loc[["viper", "cobra", "sidewinder"]]
>>> df.loc[:] += shuffled_df
>>> df
```

	max_speed	shield
cobra	60	20
viper	0	10
sidewinder	0	0

****Getting values on a DataFrame with an index that has integer labels****

Another example using integers for the index

```
>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=[7, 8, 9],
...     columns=["max_speed", "shield"],
... )
>>> df
```

	max_speed	shield
7	1	2
8	4	5
9	7	8

Slice with integer labels for rows. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc[7:9]
max_speed  shield
7          1      2
8          4      5
9          7      8
```

****Getting values with a MultiIndex****

A number of examples using a DataFrame with a MultiIndex

```
>>> tuples = [
...     ("cobra", "mark i"),
...     ("cobra", "mark ii"),
...     ("sidewinder", "mark i"),
...     ("sidewinder", "mark ii"),
...     ("viper", "mark ii"),
...     ("viper", "mark iii"),
... ]
>>> index = pd.MultiIndex.from_tuples(tuples)
>>> values = [[12, 2], [0, 4], [10, 20], [1, 4], [7, 1], [16, 36]]
>>> df = pd.DataFrame(values, columns=["max_speed", "shield"], index=index)
>>> df
```

		max_speed	shield
cobra	mark i	12	2
	mark ii	0	4
sidewinder	mark i	10	20

	mark ii	1	4
viper	mark ii	7	1
	mark iii	16	36

Single label. Note this returns a DataFrame with a single index.

```
>>> df.loc["cobra"]
      max_speed  shield
mark i         12      2
mark ii         0      4
```

Single index tuple. Note this returns a Series.

```
>>> df.loc[("cobra", "mark ii")]
max_speed    0
shield       4
Name: (cobra, mark ii), dtype: int64
```

Single label for row and column. Similar to passing in a tuple, this returns a Series.

```
>>> df.loc["cobra", "mark i"]
max_speed    12
shield        2
Name: (cobra, mark i), dtype: int64
```

Single tuple. Note using ``[[]]`` returns a DataFrame.

```
>>> df.loc[["cobra", "mark ii"]]
      max_speed  shield
cobra mark ii         0      4
```

Single tuple for the index with a single label for the column

```
>>> df.loc[("cobra", "mark i"), "shield"]
np.int64(2)
```

Slice from index tuple to single label

```
>>> df.loc[("cobra", "mark i") : "viper"]
      max_speed  shield
cobra      mark i         12      2
      mark ii          0      4
sidewinder mark i         10     20
      mark ii          1      4
viper      mark ii          7      1
      mark iii         16     36
```

Slice from index tuple to index tuple

```
>>> df.loc[("cobra", "mark i") : ("viper", "mark ii")]
      max_speed  shield
cobra      mark i         12      2
      mark ii          0      4
sidewinder mark i         10     20
      mark ii          1      4
viper      mark ii          7      1
```

Please see the :ref:`user guide<advanced.advanced\_hierarchical>` for more details and explanations of advanced indexing.

****Assignment with Series****

When assigning a Series to `.loc[row_indexer, col_indexer]`, pandas aligns the Series by index labels, not by order or position.

Series assignment with `.loc` and index alignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
>>> s = pd.Series([10, 20], index=[1, 0]) # Note reversed order
>>> df.loc[:, "B"] = s # Aligns by index, not order
>>> df
   A  B
0  1  20.0
1  2  10.0
2  3  NaN
```

Methods inherited from pandas.core.arraylike.OpsMixin:

`__add__(self, other)`

Get Addition of DataFrame and other, column-wise.

Equivalent to ```DataFrame.add(other)```.

Parameters

other : scalar, sequence, Series, dict or DataFrame
Object to be added to the DataFrame.

Returns

DataFrame

The result of adding ```other``` to DataFrame.

See Also

DataFrame.add : Add a DataFrame and another object, with option for index-
or column-oriented addition.

Examples

>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
 height weight
elk 1.5 500
moose 2.6 800

Adding a scalar affects all rows and columns.

>>> df[["height", "weight"]] + 1.5
 height weight
elk 3.0 501.5
moose 4.1 801.5

Each element of a list is added to a column of the DataFrame, in order.

>>> df[["height", "weight"]] + [0.5, 1.5]
 height weight
elk 2.0 501.5
moose 3.1 801.5

Keys of a dictionary are aligned to the DataFrame, based on column names;
each value in the dictionary is added to the corresponding column.

>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
 height weight
elk 2.0 501.5
moose 3.1 801.5

When ```other``` is a `:class:`Series``, the index of ```other``` is aligned with the
columns of the DataFrame.

>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
 height weight
elk 3.0 500.5
moose 4.1 800.5

Even when the index of ```other``` is the same as the index of the DataFrame,
the `:class:`Series`` will not be reoriented. If index-wise alignment is desired,
`:meth:`DataFrame.add`` should be used with ```axis='index'```.

>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
 elk height moose weight
elk NaN NaN NaN NaN
moose NaN NaN NaN NaN

>>> df[["height", "weight"]].add(s2, axis="index")
 height weight
elk 2.0 500.5

```

    moose      4.1    801.5

When `other` is a :class:`DataFrame`, both columns names and the
index are aligned.

>>> other = pd.DataFrame(
...     {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
... )
>>> df[["height", "weight"]] + other
      height  weight
deer      NaN     NaN
elk       1.7     NaN
moose     3.0     NaN

__and__(self, other)

__eq__(self, other)
    Return self==value.

__floordiv__(self, other)

__ge__(self, other)
    Return self>=value.

__gt__(self, other)
    Return self>value.

__le__(self, other)
    Return self<=value.

__lt__(self, other)
    Return self<value.

__mod__(self, other)

__mul__(self, other)

__ne__(self, other)
    Return self!=value.

__or__(self, other)
    Return self|value.

__pow__(self, other)

__radd__(self, other)

__rand__(self, other)

__rfloordiv__(self, other)

__rmod__(self, other)

__rmul__(self, other)

__ror__(self, other)
    Return value|self.

__rpow__(self, other)

__rsub__(self, other)

__rtruediv__(self, other)

__rxor__(self, other)

__sub__(self, other)

__truediv__(self, other)

__xor__(self, other)

-----
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:
    __hash__ = None
class SubClassedSeries(pandas.Series)

```



```

SubclassedSeries(
    data=None,
    index=None,
    dtype: Dtype | None = None,
    name=None,
    copy: bool | None = None
) -> None

Method resolution order:
SubclassedSeries
pandas.Series
pandas.core.base.IndexOpsMixin
pandas.core.arraylike.OpsMixin
pandas.core.generic.NDFrame
pandas.core.base.PandasObject
pandas.core.accessor.DirNamesMixin
pandas.core.indexing.IndexingMixin
builtins.object

Methods inherited from pandas.Series:

__array__(self, dtype: npt.DTypeLike | None = None, copy: bool | None = None) -> np.ndarray
    Return the values as a NumPy array.

    Users should not call this directly. Rather, it is invoked by
    :func:`numpy.array` and :func:`numpy.asarray`.

Parameters
-----
dtype : str or numpy.dtype, optional
    The dtype to use for the resulting NumPy array. By default,
    the dtype is inferred from the data.

copy : bool or None, optional
    See :func:`numpy.asarray`.

Returns
-----
numpy.ndarray
    The values in the series converted to a :class:`numpy.ndarray`
    with the specified `dtype`.

See Also
-----
array : Create a new array from data.
Series.array : Zero-copy view to the array backing the Series.
Series.to_numpy : Series method for similar behavior.

Examples
-----
>>> ser = pd.Series([1, 2, 3])
>>> np.asarray(ser)
array([1, 2, 3])

For timezone-aware data, the timezones may be retained with
`dtype='object'`

>>> tzser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> np.asarray(tzser, dtype="object")
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
      dtype=object)

Or the values may be localized to UTC and the tzinfo discarded with
`dtype='datetime64[ns]'`

>>> np.asarray(tzser, dtype="datetime64[ns]") # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', ...],
      dtype='datetime64[ns]')

__arrow_c_stream__(self, requested_schema=None) from pandas.core.series.Series
    Export the pandas Series as an Arrow C stream PyCapsule.

    This relies on pyarrow to convert the pandas Series to the Arrow
    format (and follows the default behavior of ``pyarrow.Array.from_pandas``
    in its handling of the index, i.e. to ignore it).
    This conversion is not necessarily zero-copy.

```

Parameters

requested_schema : PyCapsule, default None
The schema to which the dataframe should be casted, passed as a PyCapsule containing a C ArrowSchema representation of the requested schema.

Returns

PyCapsule

__getitem__(self, key) from pandas.core.series.Series

__init__(
self,
data=None,
index=None,
dtype: Dtype | None = None,
name=None,
copy: bool | None = None
) -> None from pandas.core.series.Series
Initialize self. See help(type(self)) for accurate signature.

__len__(self) -> int from pandas.core.series.Series
Return the length of the Series.

__matmul__(self, other) from pandas.core.series.Series
Matrix multiplication using binary '@' operator.

__repr__(self) -> str from pandas.core.series.Series
Return a string representation for a particular Series.

__rmatmul__(self, other) from pandas.core.series.Series
Matrix multiplication using binary '@' operator.

__setitem__(self, key, value) -> None from pandas.core.series.Series

add(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series.Series
Return Addition of series and other, element-wise (binary operator 'add').

Equivalent to ``series + other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : Series or scalar value
With which to compute the addition.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.radd : Reverse of the Addition operator, see
`Python documentation`
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
>>> a
a    1.0
b    1.0
```

```

c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.add(b, fill_value=0)
a    2.0
b    1.0
c    1.0
d    1.0
e    NaN
dtype: float64

```

`agg = aggregate(self, func=None, axis: Axis = 0, *args, **kwargs)`

`aggregate(self, func=None, axis: Axis = 0, *args, **kwargs)` from `pandas.core.series.Series`
Aggregate using one or more operations over the specified axis.

Parameters

`func` : function, str, list or dict
Function to use for aggregating the data. If a function, must either work when passed a Series or when passed to `Series.apply`.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. `[[np.sum, 'mean']]`
- dict of axis labels -> functions, function names or list of such.

`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with `DataFrame`.

`*args`
Positional arguments to pass to ``func``.

`**kwargs`
Keyword arguments to pass to ``func``.

Returns

scalar, Series or DataFrame
The return can be:

- * scalar : when `Series.agg` is called with single function
- * Series : when `DataFrame.agg` is called with a single function
- * DataFrame : when `DataFrame.agg` is called with several functions

See Also

`Series.apply` : Invoke function on a Series.
`Series.transform` : Transform function producing a Series with like indexes.

Notes

The aggregation operations are always performed over an axis, either the index (default) or the column axis. This behavior is different from ``numpy`` aggregation functions (``mean``, ``median``, ``prod``, ``sum``, ``std``, ``var``), where the default is to compute the aggregation of the flattened array, e.g., ``numpy.mean(arr_2d)`` as opposed to ``numpy.mean(arr_2d, axis=0)``.

``agg`` is an alias for ``aggregate``. Use the alias.

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See [:ref:`gotchas.udf-mutation`](#) for more details.

A passed user-defined-function will be passed a Series for evaluation.

If ``func`` defines an index relabeling, ``axis`` must be ``0`` or ``index``.

Examples

```

>>> s = pd.Series([1, 2, 3, 4])
>>> s
0    1
1    2
2    3
3    4
dtype: int64

>>> s.agg("min")
1

>>> s.agg(["min", "max"])
min    1
max    4
dtype: int64

```

all(
 self,
 *,
 axis: Axis = 0,
 bool_only: bool = False,
 skipna: bool = True,
 **kwargs
) -> bool from pandas.core.series.Series

Return whether all elements are True, potentially over an axis.

Returns True unless there at least one element within a series or along a Dataframe axis that is False or equivalent (e.g. zero or empty).

Parameters

axis : {0 or 'index', 1 or 'columns', None}, default 0
 Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

 * 0 / 'index' : reduce the index, return a Series whose index is the original column labels.
 * 1 / 'columns' : reduce the columns, return a Series whose index is the original index.
 * None : reduce all axes, return a scalar.

bool_only : bool, default False
 Include only boolean columns. Not implemented for Series.

skipna : bool, default True
 Exclude NA/null values. If the entire row/column is NA and skipna is True, then the result will be True, as for an empty row/column. If skipna is False, then NA are treated as True, because these are not equal to zero.

**kwargs : any, default None
 Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or scalar
 If axis=None, then a scalar boolean is returned.
 Otherwise a Series is returned with index matching the index argument.

See Also

Series.all : Return True if all elements are True.
DataFrame.any : Return True if one (or more) elements are True.

Examples

```

**Series**

>>> pd.Series([True, True]).all()
True
>>> pd.Series([True, False]).all()
False
>>> pd.Series([], dtype="float64").all()
True
>>> pd.Series([np.nan]).all()
True
>>> pd.Series([np.nan]).all(skipna=False)

```

True

****DataFrames****

Create a DataFrame from a dictionary.

```
>>> df = pd.DataFrame({"col1": [True, True], "col2": [True, False]})
>>> df
   col1  col2
0  True   True
1  True  False
```

Default behaviour checks if values in each column all return True.

```
>>> df.all()
col1    True
col2   False
dtype: bool
```

Specify ``axis='columns'`` to check if values in each row all return True.

```
>>> df.all(axis="columns")
0     True
1    False
dtype: bool
```

Or ``axis=None`` for whether every value is True.

```
>>> df.all(axis=None)
False
```

```
any(
    self,
    *,
    axis: Axis = 0,
    bool_only: bool = False,
    skipna: bool = True,
    **kwargs
) -> bool from pandas.core.series.Series
Return whether any element is True, potentially over an axis.
```

Returns False unless there is at least one element within a series or along a Dataframe axis that is True or equivalent (e.g. non-zero or non-empty).

Parameters

axis : {0 or 'index', 1 or 'columns', None}, default 0
Indicate which axis or axes should be reduced. For `Series` this parameter is unused and defaults to 0.

- * 0 / 'index' : reduce the index, return a Series whose index is the original column labels.
- * 1 / 'columns' : reduce the columns, return a Series whose index is the original index.
- * None : reduce all axes, return a scalar.

bool_only : bool, default False

Include only boolean columns. Not implemented for Series.

skipna : bool, default True

Exclude NA/null values. If the entire row/column is NA and skipna is True, then the result will be False, as for an empty row/column.

If skipna is False, then NA are treated as True, because these are not equal to zero.

**kwargs : any, default None

Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series or scalar

If axis=None, then a scalar boolean is returned.

Otherwise a Series is returned with index matching the index argument.

See Also

numpy.any : Numpy version of this method.

Series.any : Return whether any element is True.
 Series.all : Return whether all elements are True.
 DataFrame.any : Return whether any element is True over requested axis.
 DataFrame.all : Return whether all elements are True over requested axis.

Examples

****Series****

For Series input, the output is a scalar indicating whether any element is True.

```
>>> pd.Series([False, False]).any()
False
>>> pd.Series([True, False]).any()
True
>>> pd.Series([], dtype="float64").any()
False
>>> pd.Series([np.nan]).any()
False
>>> pd.Series([np.nan]).any(skipna=False)
True
```

****DataFrame****

Whether each column contains at least one True element (the default).

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [0, 2], "C": [0, 0]})
>>> df
   A  B  C
0  1  0  0
1  2  2  0
```

```
>>> df.any()
A      True
B      True
C     False
dtype: bool
```

Aggregating over the columns.

```
>>> df = pd.DataFrame({"A": [True, False], "B": [1, 2]})
>>> df
   A  B
0  True  1
1 False  2
```

```
>>> df.any(axis="columns")
0      True
1      True
dtype: bool
```

```
>>> df = pd.DataFrame({"A": [True, False], "B": [1, 0]})
>>> df
   A  B
0  True  1
1 False  0
```

```
>>> df.any(axis="columns")
0      True
1     False
dtype: bool
```

Aggregating over the entire DataFrame with ``axis=None``.

```
>>> df.any(axis=None)
True
```

``any`` for an empty DataFrame is an empty Series.

```
>>> pd.DataFrame([]).any()
Series([], dtype: bool)
```

```
apply(
    self,
    func: AggFuncType,
    args: tuple[Any, ...] = (),
```

```

*,
by_row: Literal[False, 'compat'] = 'compat',
**kwargs
) -> DataFrame | Series from pandas.core.series.Series
Invoke function on values of Series.

Can be ufunc (a NumPy function that applies to the entire Series)
or a Python function that only works on single values.

Parameters
-----
func : function
    Python function or NumPy ufunc to apply.
args : tuple
    Positional arguments passed to func after the series value.
by_row : False or "compat", default "compat"
    If ``"compat"`` and func is a callable, func will be passed each element of
    the Series, like ``Series.map``. If func is a list or dict of
    callables, will first try to translate each func into pandas methods. If
    that doesn't work, will try call to apply again with ``by_row="compat"``
    and if that fails, will call apply again with ``by_row=False``
    (backward compatible).
    If False, the func will be passed the whole Series at once.

    ``by_row`` has no effect when ``func`` is a string.

.. versionadded:: 2.1.0
**kwargs
    Additional keyword arguments passed to func.

Returns
-----
Series or DataFrame
    If func returns a Series object the result will be a DataFrame.

See Also
-----
Series.map: For element-wise operations.
Series.agg: Only perform aggregating type operations.
Series.transform: Only perform transforming type operations.

Notes
-----
Functions that mutate the passed object can produce unexpected
behavior or errors and are not supported. See :ref:`gotchas.udf-mutation`
for more details.

Examples
-----
Create a series with typical summer temperatures for each city.

>>> s = pd.Series([20, 21, 12], index=["London", "New York", "Helsinki"])
>>> s
London      20
New York    21
Helsinki    12
dtype: int64

Square the values by defining a function and passing it as an
argument to ``apply()``.

>>> def square(x):
...     return x**2
>>> s.apply(square)
London      400
New York    441
Helsinki    144
dtype: int64

Square the values by passing an anonymous function as an
argument to ``apply()``.

>>> s.apply(lambda x: x**2)
London      400
New York    441
Helsinki    144
dtype: int64

```

Define a custom function that needs additional positional arguments and pass these additional arguments using the ``args`` keyword.

```
>>> def subtract_custom_value(x, custom_value):
...     return x - custom_value
```

```
>>> s.apply(subtract_custom_value, args=(5,))
London      15
New York    16
Helsinki     7
dtype: int64
```

Define a custom function that takes keyword arguments and pass these arguments to ``apply``.

```
>>> def add_custom_values(x, **kwargs):
...     for month in kwargs:
...         x += kwargs[month]
...     return x
```

```
>>> s.apply(add_custom_values, june=30, july=20, august=25)
London      95
New York    96
Helsinki    87
dtype: int64
```

Use a function from the Numpy library.

```
>>> s.apply(np.log)
London      2.995732
New York    3.044522
Helsinki    2.484907
dtype: float64
```

```
argsort(
    self,
    axis: Axis = 0,
    kind: SortKind = 'quicksort',
    order: None = None,
    stable: None = None
) -> Series from pandas.core.series.Series
Return the integer indices that would sort the Series values.
```

Override ndarray.argsort. Argsorts the value, omitting NA/null values, and places the result in the same locations as the non-NA values.

Parameters

axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.
kind : {'mergesort', 'quicksort', 'heapsort', 'stable'}, default 'quicksort'
Choice of sorting algorithm. See :func:`numpy.sort` for more information. 'mergesort' and 'stable' are the only stable algorithms.
order : None
Has no effect but is accepted for compatibility with numpy.
stable : None
Has no effect but is accepted for compatibility with numpy.

Returns

Series[np.intp]
Positions of values within the sort order with -1 indicating nan values.

See Also

numpy.ndarray.argsort : Returns the indices that would sort this array.

Examples

```
>>> s = pd.Series([3, 2, 1])
>>> s.argsort()
0    2
1    1
2    0
```



```

dtype: int64

autocorr(self, lag: int = 1) -> float from pandas.core.series.Series
    Compute the lag-N autocorrelation.

    This method computes the Pearson correlation between
    the Series and its shifted self.

Parameters
-----
lag : int, default 1
    Number of lags to apply before performing autocorrelation.

Returns
-----
float
    The Pearson correlation between self and self.shift(lag).

See Also
-----
Series.corr : Compute the correlation between two Series.
Series.shift : Shift index by desired number of periods.
DataFrame.corr : Compute pairwise correlation of columns.
DataFrame.corrwith : Compute pairwise correlation between rows or
    columns of two DataFrame objects.

Notes
-----
If the Pearson correlation is not well defined return 'NaN'.

Examples
-----
>>> s = pd.Series([0.25, 0.5, 0.2, -0.05])
>>> s.autocorr() # doctest: +ELLIPSIS
0.10355...
>>> s.autocorr(lag=2) # doctest: +ELLIPSIS
-0.99999...

If the Pearson correlation is not well defined, then 'NaN' is returned.

>>> s = pd.Series([1, 0, 0, 0])
>>> s.autocorr()
nan

```

between(
 self,
 left,
 right,
 inclusive: Literal['both', 'neither', 'left', 'right'] = 'both'
) -> Series from pandas.core.series.Series
Return boolean Series equivalent to left <= series <= right.

This function returns a boolean vector containing `True` wherever the corresponding Series element is between the boundary values `left` and `right`. NA values are treated as `False`.

Parameters

left : scalar or list-like
 Left boundary.
right : scalar or list-like
 Right boundary.
inclusive : {"both", "neither", "left", "right"}
 Include boundaries. Whether to set each bound as closed or open.

Returns

Series
 Series representing whether each element is between left and right (inclusive).

See Also

Series.gt : Greater than of series and other.
Series.lt : Less than of series and other.

Notes

This function is equivalent to `((left <= ser) & (ser <= right))`

Examples

>>> s = pd.Series([2, 0, 4, 8, np.nan])

Boundary values are included by default:

```
>>> s.between(1, 4)
0    True
1    False
2    True
3    False
4    False
dtype: bool
```

With `inclusive` set to `"neither"` boundary values are excluded:

```
>>> s.between(1, 4, inclusive="neither")
0    True
1    False
2    False
3    False
4    False
dtype: bool
```

`left` and `right` can be any scalar value:

```
>>> s = pd.Series(["Alice", "Bob", "Carol", "Eve"])
>>> s.between("Anna", "Daniel")
0    False
1    True
2    True
3    False
dtype: bool
```

```
case_when(
    self,
    caselist: list[tuple[ArrayLike | Callable[[Series], Series | np.ndarray | Sequence[bool]]],
) -> Series from pandas.core.series.Series
Replace values where the conditions are True.
```

.. versionadded:: 2.2.0

Parameters

caselist : A list of tuples of conditions and expected replacements
Takes the form: `((condition0, replacement0), (condition1, replacement1), ...)`
`condition` should be a 1-D boolean array-like object
or a callable. If `condition` is a callable,
it is computed on the Series
and should return a boolean Series or array.
The callable must not change the input Series
(though pandas doesn't check it). `replacement` should be a
1-D array-like object, a scalar or a callable.
If `replacement` is a callable, it is computed on the Series
and should return a scalar or Series. The callable
must not change the input Series
(though pandas doesn't check it).

Returns

Series

A new Series with values replaced based on the provided conditions.

See Also

`Series.mask` : Replace values where the condition is True.

Examples

>>> c = pd.Series([6, 7, 8, 9], name="c")
>>> a = pd.Series([0, 0, 1, 2])
>>> b = pd.Series([0, 3, 4, 5])

```
>>> c.case_when(
...     caselist=[
...         (a.gt(0), a), # condition, replacement
...         (b.gt(0), b),
...     ]
... )
0    6
1    3
2    1
3    2
Name: c, dtype: int64
```

```
combine(
    self,
    other: Series | Hashable,
    func: Callable[[Hashable, Hashable], Hashable],
    fill_value: Hashable | None = None
) -> Series from pandas.core.series.Series
Combine the Series with a Series or scalar according to `func`.

Combine the Series and `other` using `func` to perform elementwise
selection for combined Series.
`fill_value` is assumed when value is not present at some index
from one of the two Series being combined.

Parameters
-----
other : Series or scalar
    The value(s) to be combined with the `Series`.
func : function
    Function that takes two scalars as inputs and returns an element.
fill_value : scalar, optional
    The value to assume when an index is missing from
    one Series or the other. The default specifies to use the
    appropriate NaN value for the underlying dtype of the Series.

Returns
-----
Series
    The result of combining the Series with the other object.

See Also
-----
Series.combine_first : Combine Series values, choosing the calling
    Series' values first.
```

Examples

Consider 2 Datasets ``s1`` and ``s2`` containing
highest clocked speeds of different birds.

```
>>> s1 = pd.Series({"falcon": 330.0, "eagle": 160.0})
>>> s1
falcon    330.0
eagle     160.0
dtype: float64
>>> s2 = pd.Series({"falcon": 345.0, "eagle": 200.0, "duck": 30.0})
>>> s2
falcon    345.0
eagle     200.0
duck       30.0
dtype: float64
```

Now, to combine the two datasets and view the highest speeds
of the birds across the two datasets

```
>>> s1.combine(s2, max)
duck      NaN
eagle     200.0
falcon    345.0
dtype: float64
```

In the previous example, the resulting value for duck is missing,
because the maximum of a NaN and a float is a NaN.
So, in the example, we set ``fill\_value=0``,
so the maximum value returned will be the value from some dataset.

```

>>> s1.combine(s2, max, fill_value=0)
duck      30.0
eagle     200.0
falcon    345.0
dtype: float64

```

combine_first(self, other) -> Series from pandas.core.series.Series
Update null elements with value in the same location in 'other'.

Combine two Series objects by filling null values in one Series with non-null values from the other Series. Result index will be the union of the two indexes.

Parameters

other : Series
The value(s) to be used for filling null values.

Returns

Series
The result of combining the provided Series with the other object.

See Also

Series.combine : Perform element-wise operation on two Series using a given function.

Examples

>>> s1 = pd.Series([1, np.nan])
>>> s2 = pd.Series([3, 4, 5])
>>> s1.combine_first(s2)
0 1.0
1 4.0
2 5.0
dtype: float64

Null values still persist if the location of that null value does not exist in `other`

```

>>> s1 = pd.Series({"falcon": np.nan, "eagle": 160.0})
>>> s2 = pd.Series({"eagle": 200.0, "duck": 30.0})
>>> s1.combine_first(s2)
duck      30.0
eagle     160.0
falcon     NaN
dtype: float64

```

compare(
self,
other: Series,
align_axis: Axis = 1,
keep_shape: bool = False,
keep_equal: bool = False,
result_names: Suffixes = ('self', 'other')
) -> DataFrame | Series from pandas.core.series.Series
Compare to another Series and show the differences.

Parameters

other : Series
Object to compare with.

align_axis : {{0 or 'index', 1 or 'columns'}}, default 1
Determine which axis to align the comparison on.

- * 0, or 'index' : Resulting differences are stacked vertically with rows drawn alternately from self and other.
- * 1, or 'columns' : Resulting differences are aligned horizontally with columns drawn alternately from self and other.

keep_shape : bool, default False
If true, all rows and columns are kept.
Otherwise, only the ones with different values are kept.

keep_equal : bool, default False

If true, the result keeps values that are equal.
Otherwise, equal values are shown as NaNs.

result_names : tuple, default ('self', 'other')
Set the dataframes names in the comparison.

Returns

Series or DataFrame

If axis is 0 or 'index' the result will be a Series.
The resulting index will be a MultiIndex with 'self' and 'other'
stacked alternately at the inner level.

If axis is 1 or 'columns' the result will be a DataFrame.
It will have two columns namely 'self' and 'other'.

See Also

DataFrame.compare : Compare with another DataFrame and show differences.

Notes

Matching NaNs will not appear as a difference.

Examples

>>> s1 = pd.Series(["a", "b", "c", "d", "e"])
>>> s2 = pd.Series(["a", "a", "c", "b", "e"])

Align the differences on columns

```
>>> s1.compare(s2)
      self other
1      b      a
3      d      b
```

Stack the differences on indices

```
>>> s1.compare(s2, align_axis=0)
1  self      b
   other      a
3  self      d
   other      b
dtype: str
```

Keep all original rows

```
>>> s1.compare(s2, keep_shape=True)
      self other
0  NaN  NaN
1    b    a
2  NaN  NaN
3    d    b
4  NaN  NaN
```

Keep all original rows and also all original values

```
>>> s1.compare(s2, keep_shape=True, keep_equal=True)
      self other
0    a      a
1    b      a
2    c      c
3    d      b
4    e      e
```

```
corr(
    self,
    other: Series,
    method: CorrelationMethod = 'pearson',
    min_periods: int | None = None
) -> float from pandas.core.series.Series
Compute correlation with `other` Series, excluding missing values.
```

The two `Series` objects are not required to be the same length and will be aligned internally before the correlation function is applied.

Parameters

```

-----
other : Series
    Series with which to compute the correlation.
method : {'pearson', 'kendall', 'spearman'} or callable
    Method used to compute correlation:

    - pearson : Standard correlation coefficient
    - kendall : Kendall Tau correlation coefficient
    - spearman : Spearman rank correlation
    - callable: Callable with input two 1d ndarrays and returning a float.

    .. warning::
        Note that the returned matrix from corr will have 1 along the
        diagonals and will be symmetric regardless of the callable's
        behavior.
min_periods : int, optional
    Minimum number of observations needed to have a valid result.

Returns
-----
float
    Correlation with other.

See Also
-----
DataFrame.corr : Compute pairwise correlation between columns.
DataFrame.corrwith : Compute pairwise correlation with another
    DataFrame or Series.

Notes
-----
Pearson, Kendall and Spearman correlation are currently computed using pairwise complete

* `Pearson correlation coefficient <https://en.wikipedia.org/wiki/Pearson\_correlation\_coefficient>`
* `Kendall rank correlation coefficient <https://en.wikipedia.org/wiki/Kendall\_rank\_correlation\_coefficient>`
* `Spearman's rank correlation coefficient <https://en.wikipedia.org/wiki/Spearman%27s\_rank\_correlation\_coefficient>`

Automatic data alignment: as with all pandas operations, automatic data alignment is performed.
`corr()` automatically considers values with matching indices.

Examples
-----
>>> def histogram_intersection(a, b):
...     v = np.minimum(a, b).sum().round(decimals=1)
...     return v
>>> s1 = pd.Series([0.2, 0.0, 0.6, 0.2])
>>> s2 = pd.Series([0.3, 0.6, 0.0, 0.1])
>>> s1.corr(s2, method=histogram_intersection)
0.3

Pandas auto-aligns the values with matching indices

>>> s1 = pd.Series([1, 2, 3], index=[0, 1, 2])
>>> s2 = pd.Series([1, 2, 3], index=[2, 1, 0])
>>> s1.corr(s2)
-1.0

If the input is a constant array, the correlation is not defined in this case,
and `np.nan` is returned.

>>> s1 = pd.Series([0.45, 0.45])
>>> s1.corr(s1)
nan

count(self) -> int from pandas.core.series.Series
    Return number of non-NA/null observations in the Series.

Returns
-----
int
    Number of non-null values in the Series.

See Also
-----
DataFrame.count : Count non-NA cells for each column or row.

Examples

```

```

-----
>>> s = pd.Series([0.0, 1.0, np.nan])
>>> s.count()
2

```

`cov(self, other: Series, min_periods: int | None = None, ddof: int | None = 1) -> float from`
 Compute covariance with Series, excluding missing values.

The two `Series` objects are not required to be the same length and will be aligned internally before the covariance is calculated.

Parameters

`other` : Series
 Series with which to compute the covariance.
`min_periods` : int, optional
 Minimum number of observations needed to have a valid result.
`ddof` : int, default 1
 Delta degrees of freedom. The divisor used in calculations is ```N - ddof```, where ```N``` represents the number of elements.

Returns

float
 Covariance between Series and other normalized by N-1 (unbiased estimator).

See Also

`DataFrame.cov` : Compute pairwise covariance of columns.

Examples

```

>>> s1 = pd.Series([0.90010907, 0.13484424, 0.62036035])
>>> s2 = pd.Series([0.12528585, 0.26962463, 0.51111198])
>>> s1.cov(s2)
-0.01685762652715874

```

`cummax(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self from pandas.core.`
 Return cumulative maximum over a Series.

Returns a Series of the same size containing the cumulative maximum.

Parameters

`axis` : {0 or 'index'}, default 0
 This parameter is unused and defaults to 0.
`skipna` : bool, default True
 Exclude NA/null values. If the series is NA, the result is NA.
`*args, **kwargs`
 Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series
 Return cumulative maximum of Series.

See Also

`core.window.expanding.Expanding.max` : Similar functionality but ignores ```NaN``` values.
`Series.max` : Return the maximum over a Series.
`Series.cummin` : Return cumulative minimum.
`Series.cumsum` : Return cumulative sum.
`Series.cumprod` : Return cumulative product.

Examples

```

>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0

```

```
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummax()
0    2.0
1    NaN
2    5.0
3    5.0
4    5.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummax(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64
```

`cummin(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core.`
Return cumulative minimum over a Series.

Returns a Series of the same size containing the cumulative minimum.

Parameters

`axis` : {0 or 'index'}, default 0
This parameter is unused and defaults to 0.
`skipna` : bool, default True
If the entire series is NA, the result will be NA.
`*args, **kwargs`
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series
Return cumulative minimum of the Series.

See Also

`core.window.expanding.Expanding.min` : Similar functionality but ignores ``NaN`` values.
`Series.min` : Return the minimum value of the Series.
`Series.cummax` : Return cumulative maximum.
`Series.cumsum` : Return cumulative sum.
`Series.cumprod` : Return cumulative product.

Examples

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cummin()
0    2.0
1    NaN
2    2.0
3   -1.0
4   -1.0
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cummin(skipna=False)
0    2.0
```



```

1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

```

`cumprod(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core`
 Return cumulative product over a Series.

Returns a Series of the same size containing the cumulative product.

Parameters

```

-----
axis : {0 or 'index'}, default 0
      This parameter is unused and defaults to 0.
skipna : bool, default True
        Exclude NA/null values. If entire Series is NA, the result will be NA.
*args, **kwargs
      Additional keywords have no effect but might be accepted for
      compatibility with NumPy.

```

Returns

```

-----
Series
      Return cumulative product of Series.

```

See Also

```

-----
core.window.expanding.Expanding.prod : Similar functionality
      but ignores ``NaN`` values.
Series.prod : Return the product over Series.
Series.cummax : Return cumulative maximum.
Series.cummin : Return cumulative minimum.
Series.cumsum : Return cumulative sum.

```

Examples

```

-----
>>> s = pd.Series([2, np.nan, 5, -1, 0])
>>> s
0    2.0
1    NaN
2    5.0
3   -1.0
4    0.0
dtype: float64

```

By default, NA values are ignored.

```

>>> s.cumprod()
0    2.0
1    NaN
2   10.0
3  -10.0
4   -0.0
dtype: float64

```

To include NA values in the operation, use ``skipna=False``

```

>>> s.cumprod(skipna=False)
0    2.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

```

`cumsum(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Self` from `pandas.core`
 Return cumulative sum over a Series.

Returns a Series of the same size containing the cumulative sum.

Parameters

```

-----
axis : {0 or 'index'}, default 0
      This parameter is unused and defaults to 0.
skipna : bool, default True

```

Exclude NA/null values. If entire series is NA, the result will be NA.
*args, **kwargs
Additional keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series

Return cumulative sum of Series.

See Also

core.window.expanding.Expanding.sum : Similar functionality but ignores ``NaN`` values.

Series.sum : Return the sum over Series.

Series.cummax : Return cumulative maximum.

Series.cummin : Return cumulative minimum.

Series.cumprod : Return cumulative product.

Examples

```
>>> s = pd.Series([2, np.nan, 5, -1, 0])
```

```
>>> s
```

```
0    2.0
```

```
1    NaN
```

```
2    5.0
```

```
3   -1.0
```

```
4    0.0
```

```
dtype: float64
```

By default, NA values are ignored.

```
>>> s.cumsum()
```

```
0    2.0
```

```
1    NaN
```

```
2    7.0
```

```
3    6.0
```

```
4    6.0
```

```
dtype: float64
```

To include NA values in the operation, use ``skipna=False``

```
>>> s.cumsum(skipna=False)
```

```
0    2.0
```

```
1    NaN
```

```
2    NaN
```

```
3    NaN
```

```
4    NaN
```

```
dtype: float64
```

diff(self, periods: int = 1) -> Series from pandas.core.series.Series
First discrete difference of Series elements.

Calculates the difference of a Series element compared with another element in the Series (default is element in previous row).

Parameters

periods : int, default 1

Periods to shift for calculating difference, accepts negative values.

Returns

Series

First differences of the Series.

See Also

Series.pct_change: Percent change over given number of periods.

Series.shift: Shift index by desired number of periods with an optional time freq.

DataFrame.diff: First discrete difference of object.

Notes

For boolean dtypes, this uses :meth:`operator.xor` rather than

:meth:`operator.sub`.
The result is calculated according to current dtype in Series,
however dtype of the result is always float64.

Examples

Difference with previous row

```
>>> s = pd.Series([1, 1, 2, 3, 5, 8])
>>> s.diff()
0    NaN
1    0.0
2    1.0
3    1.0
4    2.0
5    3.0
dtype: float64
```

Difference with 3rd previous row

```
>>> s.diff(periods=3)
0    NaN
1    NaN
2    NaN
3    2.0
4    4.0
5    6.0
dtype: float64
```

Difference with following row

```
>>> s.diff(periods=-1)
0    0.0
1   -1.0
2   -1.0
3   -2.0
4   -3.0
5    NaN
dtype: float64
```

Overflow in input dtype

```
>>> s = pd.Series([1, 0], dtype=np.uint8)
>>> s.diff()
0    NaN
1   255.0
dtype: float64
```

`div = truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

`divide = truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

`divmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series` from pandas.core.
Return Integer division and modulo of series and other, element-wise (binary operator ``d`

Equivalent to ```divmod(series, other)```, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ```==``` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

2-Tuple of Series
The result of the operation.

See Also

Series.rdivmod : Reverse of the Integer division and modulo operator, see
`Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`
for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.divmod(b, fill_value=0)
(a 1.0
b inf
c inf
d 0.0
e NaN
dtype: float64,
a 0.0
b NaN
c NaN
d 0.0
e NaN
dtype: float64)

dot(self, other: AnyArrayLike | DataFrame) -> Series | np.ndarray from pandas.core.series.Series
Compute the dot product between the Series and the columns of other.

This method computes the dot product between the Series and another one, or the Series and each columns of a DataFrame, or the Series and each columns of an array.

It can also be called using `self @ other`.

Parameters

other : Series, DataFrame or array-like
The other object to compute the dot product with its columns.

Returns

scalar, Series or numpy.ndarray
Return the dot product of the Series and other if other is a Series, the Series of the dot product of Series and each rows of other if other is a DataFrame or a numpy.ndarray between the Series and each columns of the numpy array.

See Also

DataFrame.dot: Compute the matrix product with the DataFrame.
Series.mul: Multiplication of series and other, element-wise.

Notes

The Series and other has to share the same index if other is a Series or a DataFrame.

Examples

>>> s = pd.Series([0, 1, 2, 3])
>>> other = pd.Series([-1, 2, -3, 4])

```

>>> s.dot(other)
8
>>> s @ other
8
>>> df = pd.DataFrame([[0, 1], [-2, 3], [4, -5], [6, 7]])
>>> s.dot(df)
0    24
1    14
dtype: int64
>>> arr = np.array([[0, 1], [-2, 3], [4, -5], [6, 7]])
>>> s.dot(arr)
array([24, 14])

```

drop(
self,
labels: IndexLabel | ListLike = None,
*,
axis: Axis = 0,
index: IndexLabel | ListLike = None,
columns: IndexLabel | ListLike = None,
level: Level | None = None,
inplace: bool = False,
errors: IgnoreRaise = 'raise'
) -> Series | None from pandas.core.series.Series
Return Series with specified index labels removed.

Remove elements of a Series based on specifying the index labels.
When using a multi-index, labels on different levels can be removed
by specifying the level.

Parameters

labels : single label or list-like
Index labels to drop.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.
index : single label or list-like
Redundant for application on Series, but 'index' can be used instead
of 'labels'.
columns : single label or list-like
No change is made to the Series; use 'index' or 'labels' instead.
level : int or level name, optional
For MultiIndex, level for which the labels will be removed.
inplace : bool, default False
If True, do operation inplace and return None.
errors : {'ignore', 'raise'}, default 'raise'
If 'ignore', suppress error and only existing labels are dropped.

Returns

Series or None
Series with specified index labels removed or None if ``inplace=True``.

Raises

KeyError
If none of the labels are found in the index.

See Also

Series.reindex : Return only specified index labels of Series.
Series.dropna : Return series without null values.
Series.drop_duplicates : Return Series with duplicate values removed.
DataFrame.drop : Drop specified labels from rows or columns.

Examples

>>> s = pd.Series(data=np.arange(3), index=["A", "B", "C"])
>>> s
A 0
B 1
C 2
dtype: int64

Drop labels B and C

```

>>> s.drop(labels=["B", "C"])

```

```

A 0
dtype: int64

Drop 2nd level label in MultiIndex Series

>>> midx = pd.MultiIndex(
...     levels=[["llama", "cow", "falcon"], ["speed", "weight", "length"]],
...     codes=[[0, 0, 0, 1, 1, 1, 2, 2, 2], [0, 1, 2, 0, 1, 2, 0, 1, 2]],
... )
>>> s = pd.Series([45, 200, 1.2, 30, 250, 1.5, 320, 1, 0.3], index=midx)
>>> s
llama  speed      45.0
       weight    200.0
       length      1.2
cow    speed      30.0
       weight    250.0
       length      1.5
falcon speed     320.0
       weight      1.0
       length      0.3
dtype: float64

>>> s.drop(labels="weight", level=1)
llama  speed      45.0
       length      1.2
cow    speed      30.0
       length      1.5
falcon speed     320.0
       length      0.3
dtype: float64

drop_duplicates(
    self,
    *,
    keep: DropKeep = 'first',
    inplace: bool = False,
    ignore_index: bool = False
) -> Series | None from pandas.core.series.Series
Return Series with duplicate values removed.

Parameters
-----
keep : {'first', 'last', ``False``}, default 'first'
    Method to handle dropping duplicates:

    - 'first' : Drop duplicates except for the first occurrence.
    - 'last' : Drop duplicates except for the last occurrence.
    - ``False`` : Drop all duplicates.

inplace : bool, default ``False``
    If ``True``, performs operation inplace and returns None.

ignore_index : bool, default ``False``
    If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

.. versionadded:: 2.0.0

Returns
-----
Series or None
    Series with duplicates dropped or None if ``inplace=True``.

See Also
-----
Index.drop_duplicates : Equivalent method on Index.
DataFrame.drop_duplicates : Equivalent method on DataFrame.
Series.duplicated : Related method on Series, indicating duplicate
    Series values.
Series.unique : Return unique values as an array.

Examples
-----
Generate a Series with duplicated entries.

>>> s = pd.Series(
...     ["llama", "cow", "llama", "beetle", "llama", "hippo"], name="animal"
... )

```

```
>>> s
0    llama
1      cow
2    llama
3   beetle
4    llama
5     hippo
Name: animal, dtype: str
```

With the 'keep' parameter, the selection behavior of duplicated values can be changed. The value 'first' keeps the first occurrence for each set of duplicated entries. The default value of keep is 'first'.

```
>>> s.drop_duplicates()
0    llama
1      cow
3   beetle
5     hippo
Name: animal, dtype: str
```

The value 'last' for parameter 'keep' keeps the last occurrence for each set of duplicated entries.

```
>>> s.drop_duplicates(keep="last")
1      cow
3   beetle
4    llama
5     hippo
Name: animal, dtype: str
```

The value ``False`` for parameter 'keep' discards all sets of duplicated entries.

```
>>> s.drop_duplicates(keep=False)
1      cow
3   beetle
5     hippo
Name: animal, dtype: str
```

```
dropna(
    self,
    *,
    axis: Axis = 0,
    inplace: bool = False,
    how: AnyAll | None = None,
    ignore_index: bool = False
) -> Series | None from pandas.core.series.Series
Return a new Series with missing values removed.
```

See the :ref:`User Guide <missing\_data>` for more on which values are considered missing, and how to work with missing data.

Parameters

```
-----
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
inplace : bool, default False
    If True, do operation inplace and return None.
how : str, optional
    Not in use. Kept for compatibility.
ignore_index : bool, default ``False``
    If ``True``, the resulting axis will be labeled 0, 1, ..., n - 1.

.. versionadded:: 2.0.0
```

Returns

```
-----
Series or None
    Series with NA entries dropped from it or None if ``inplace=True``.
```

See Also

```
-----
Series.isna: Indicate missing values.
Series.notna : Indicate existing (non-missing) values.
Series.fillna : Replace missing values.
DataFrame.dropna : Drop rows or columns which contain NA values.
Index.dropna : Drop missing indices.
```

Examples

```
-----
>>> ser = pd.Series([1.0, 2.0, np.nan])
>>> ser
0    1.0
1    2.0
2    NaN
dtype: float64
```

Drop NA values from a Series.

```
>>> ser.dropna()
0    1.0
1    2.0
dtype: float64
```

Empty strings are not considered NA values. ``None`` is considered an NA value.

```
>>> ser = pd.Series([np.nan, 2, pd.NaT, "", None, "I stay"])
>>> ser
0      NaN
1         2
2      NaT
3
4      None
5    I stay
dtype: object
>>> ser.dropna()
1         2
3
5    I stay
dtype: object
```

`Series.duplicated(self, keep: DropKeep = 'first') -> Series` from `pandas.core.series.Series`
Indicate duplicate Series values.

Duplicated values are indicated as ``True`` values in the resulting Series. Either all duplicates, all except the first or all except the last occurrence of duplicates can be indicated.

Parameters

`keep : {'first', 'last', False}`, default 'first'
Method to handle dropping duplicates:

- 'first' : Mark duplicates as ``True`` except for the first occurrence.
- 'last' : Mark duplicates as ``True`` except for the last occurrence.
- ``False`` : Mark all duplicates as ``True``.

Returns

`Series[bool]`
Series indicating whether each value has occurred in the preceding values.

See Also

`Index.duplicated` : Equivalent method on `pandas.Index`.
`DataFrame.duplicated` : Equivalent method on `pandas.DataFrame`.
`Series.drop_duplicates` : Remove duplicate values from Series.

Examples

By default, for each set of duplicated values, the first occurrence is set on False and all others on True:

```
>>> animals = pd.Series(["llama", "cow", "llama", "beetle", "llama"])
>>> animals.duplicated()
0    False
1    False
2     True
3    False
4     True
```



```
dtype: bool
```

which is equivalent to

```
>>> animals.duplicated(keep="first")
0    False
1    False
2     True
3    False
4     True
dtype: bool
```

By using 'last', the last occurrence of each set of duplicated values is set on False and all others on True:

```
>>> animals.duplicated(keep="last")
0     True
1    False
2     True
3    False
4    False
dtype: bool
```

By setting keep on ``False``, all duplicates are True:

```
>>> animals.duplicated(keep=False)
0     True
1    False
2     True
3    False
4     True
dtype: bool
```

```
eq(
    self,
    other,
    level: Level | None = None,
    fill_value: float | None = None,
    axis: Axis = 0
) -> Series from pandas.core.series.Series
Return Equal to of series and other, element-wise (binary operator `eq`).

Equivalent to ``series == other``, but with support to substitute a fill_value
for missing data in either one of the inputs.
```

Parameters

```
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
```

Returns

```
Series
    The result of the operation.
```

See Also

```
Series.ge : Return elementwise Greater than or equal to of series and other.
Series.le : Return elementwise Less than or equal to of series and other.
Series.gt : Return elementwise Greater than of series and other.
Series.lt : Return elementwise Less than of series and other.
```

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
```

```

>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.eq(b, fill_value=0)
a    True
b    False
c    False
d    False
e    False
dtype: bool

```

`explode(self, ignore_index: bool = False) -> Series from pandas.core.series.Series`
 Transform each element of a list-like to a row.

Parameters

`ignore_index` : bool, default False
 If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

Series

Exploded lists to rows; index will be duplicated for these rows.

See Also

`Series.str.split` : Split string values on specified separator.
`Series.unstack` : Unstack, a.k.a. pivot, Series with MultiIndex to produce DataFrame.
`DataFrame.melt` : Unpivot a DataFrame from wide format to long format.
`DataFrame.explode` : Explode a DataFrame from list-like columns to long format.

Notes

This routine will explode list-likes including lists, tuples, sets, Series, and np.ndarray. The result dtype of the subset rows will be object. Scalars will be returned unchanged, and empty list-likes will result in an np.nan for that row. In addition, the ordering of elements in the output will be non-deterministic when exploding sets.

Reference :ref:`the user guide <reshaping.explode>` for more examples.

Examples

```

>>> s = pd.Series([[1, 2, 3], "foo", [], [3, 4]])
>>> s
0    [1, 2, 3]
1         foo
2          []
3    [3, 4]
dtype: object

>>> s.explode()
0    1
0    2
0    3
1    foo
2    NaN
3    3
3    4
dtype: object

```

`floordiv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core`
 Return Integer division of series and other, element-wise (binary operator `'floordiv'`).

Equivalent to `'series // other'`, but with support to substitute a `fill_value` for

missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.rfloordiv : Reverse of the Integer division operator, see [Python documentation <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>](https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types) for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.floordiv(b, fill_value=0)
a 1.0
b inf
c inf
d 0.0
e NaN
dtype: float64

ge(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Greater than or equal to of series and other, element-wise (binary operation)

Equivalent to ``series >= other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.gt : Greater than comparison, element-wise.

Series.le : Less than or equal to comparison, element-wise.

Series.lt : Less than comparison, element-wise.

Series.eq : Equal to comparison, element-wise.

Series.ne : Not equal to comparison, element-wise.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=["a", "b", "c", "d", "e"])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
e    1.0
```

```
dtype: float64
```

```
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=["a", "b", "c", "d", "f"])
```

```
>>> b
```

```
a    0.0
```

```
b    1.0
```

```
c    2.0
```

```
d    NaN
```

```
f    1.0
```

```
dtype: float64
```

```
>>> a.ge(b, fill_value=0)
```

```
a    True
```

```
b    True
```

```
c    False
```

```
d    False
```

```
e    True
```

```
f    False
```

```
dtype: bool
```

```
groupby(  
    self,  
    by=None,  
    level: IndexLabel | None = None,  
    *,  
    as_index: bool = True,  
    sort: bool = True,  
    group_keys: bool = True,  
    observed: bool = True,  
    dropna: bool = True  
) -> SeriesGroupBy from pandas.core.series.Series  
Group Series using a mapper or by a Series of columns.
```

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters

by : mapping, function, label, pd.Grouper or list of such

Used to determine the groups for the groupby.

If ``by`` is a function, it's called on each value of the object's

index. If a dict or Series is passed, the Series or dict VALUES

will be used to determine the groups (the Series' values are first

aligned; see ``.align()`` method). If a list or ndarray of length

equal to the selected axis is passed (see the `groupby user guide

<https://pandas.pydata.org/pandas-docs/stable/user_guide/groupby.html#splitting-an-object

the values are used as-is to determine the groups. A label or list

of labels may be passed to group by the columns in ``self``.

Notice that a tuple is interpreted as a (single) key.

level : int, level name, or sequence of such, default None

If the axis is a MultiIndex (hierarchical), group by a particular

level or levels. Do not specify both ``by`` and ``level``.

as_index : bool, default True

Return object with group labels as the

```

index. Only relevant for DataFrame input. as_index=False is
effectively "SQL-style" grouped output. This argument has no effect
on filtrations (see the filtrations in the user guide
<https://pandas.pydata.org/docs/dev/user\_guide/groupby.html#filtration>`_),
such as head(), tail(), nth() and in transformations
(see the transformations in the user guide
<https://pandas.pydata.org/docs/dev/user\_guide/groupby.html#transformation>`_).
sort : bool, default True
Sort group keys. Get better performance by turning this off.
Note this does not influence the order of observations within each
group. Groupby preserves the order of rows within each group. If False,
the groups will appear in the same order as they did in the original DataFrame.
This argument has no effect on filtrations (see the filtrations in the user guide
<https://pandas.pydata.org/docs/dev/user\_guide/groupby.html#filtration>`_),
such as head(), tail(), nth() and in transformations
(see the transformations in the user guide
<https://pandas.pydata.org/docs/dev/user\_guide/groupby.html#transformation>`_).

.. versionchanged:: 2.0.0

    Specifying sort=False with an ordered categorical grouper will no
    longer sort the values.

group_keys : bool, default True
When calling apply and the by argument produces a like-indexed
(i.e. ref: a transform <groupby.transform>) result, add group keys to
index to identify pieces. By default group keys are not included
when the result's index (and column) labels match the inputs, and
are included otherwise.

.. versionchanged:: 2.0.0

    group_keys now defaults to True.

observed : bool, default True
This only applies if any of the groupers are Categoricals.
If True: only show observed values for categorical groupers.
If False: show all values for categorical groupers.

.. versionchanged:: 3.0.0

    The default value is now True.

dropna : bool, default True
If True, and if group keys contain NA values, NA values together
with row/column will be dropped.
If False, NA values will also be treated as the key in groups.

Returns
-----
pandas.api.typing.SeriesGroupBy
Returns a groupby object that contains information about the groups.

See Also
-----
resample : Convenience method for frequency conversion and resampling
of time series.

Notes
-----
See the user guide
<https://pandas.pydata.org/pandas-docs/stable/groupby.html>`__ for more
detailed usage and examples, including splitting an object into groups,
iterating through groups, selecting a group, aggregation, and more.

The implementation of groupby is hash-based, meaning in particular that
objects that compare as equal will be considered to be in the same group.
An exception to this is that pandas has special handling of NA values:
any NA values will be collapsed to a single group, regardless of how
they compare. See the user guide linked above for more details.

Examples
-----
>>> ser = pd.Series([390., 350., 30., 20.],
...                  index=['Falcon', 'Falcon', 'Parrot', 'Parrot'],
...                  name="Max Speed")
>>> ser

```

```
Falcon    390.0
Falcon    350.0
Parrot     30.0
Parrot     20.0
Name: Max Speed, dtype: float64
```

We can pass a list of values to group the Series data by custom labels:

```
>>> ser.groupby(["a", "b", "a", "b"]).mean()
a    210.0
b    185.0
Name: Max Speed, dtype: float64
```

Grouping by numeric labels yields similar results:

```
>>> ser.groupby([0, 1, 0, 1]).mean()
0    210.0
1    185.0
Name: Max Speed, dtype: float64
```

We can group by a level of the index:

```
>>> ser.groupby(level=0).mean()
Falcon    370.0
Parrot     25.0
Name: Max Speed, dtype: float64
```

We can group by a condition applied to the Series values:

```
>>> ser.groupby(ser > 100).mean()
Max Speed
False     25.0
True      370.0
Name: Max Speed, dtype: float64
```

****Grouping by Indexes****

We can groupby different levels of a hierarchical index using the `level` parameter:

```
>>> arrays = [['Falcon', 'Falcon', 'Parrot', 'Parrot'],
...           ['Captive', 'Wild', 'Captive', 'Wild']]
>>> index = pd.MultiIndex.from_arrays(arrays, names=('Animal', 'Type'))
>>> ser = pd.Series([390., 350., 30., 20.], index=index, name="Max Speed")
>>> ser
Animal Type
Falcon  Captive    390.0
        Wild      350.0
Parrot   Captive    30.0
        Wild      20.0
Name: Max Speed, dtype: float64
```

```
>>> ser.groupby(level=0).mean()
Animal
Falcon    370.0
Parrot     25.0
Name: Max Speed, dtype: float64
```

We can also group by the 'Type' level of the hierarchical index to get the mean speed for each type:

```
>>> ser.groupby(level="Type").mean()
Type
Captive    210.0
Wild      185.0
Name: Max Speed, dtype: float64
```

We can also choose to include `NA` in group keys or not by defining `dropna` parameter, the default setting is `True`.

```
>>> ser = pd.Series([1, 2, 3, 3], index=["a", 'a', 'b', np.nan])
>>> ser.groupby(level=0).sum()
a     3
b     3
dtype: int64
```

To include `NA` values in the group keys, set `dropna=False`:

```
>>> ser.groupby(level=0, dropna=False).sum()
a      3
b      3
NaN     3
dtype: int64
```

We can also group by a custom list with NaN values to handle missing group labels:

```
>>> arrays = ['Falcon', 'Falcon', 'Parrot', 'Parrot']
>>> ser = pd.Series([390., 350., 30., 20.], index=arrays, name="Max Speed")
>>> ser.groupby(["a", "b", "a", np.nan]).mean()
a      210.0
b      350.0
Name: Max Speed, dtype: float64
```

```
>>> ser.groupby(["a", "b", "a", np.nan], dropna=False).mean()
a      210.0
b      350.0
NaN      20.0
Name: Max Speed, dtype: float64
```

`gt(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Greater than of series and other, element-wise (binary operator ``gt``).

Equivalent to ``series > other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

`Series`
The result of the operation.

See Also

`Series.lt` : Reverse of the Greater than operator, see ``Python documentation``
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_ for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])
>>> a
a      1.0
b      1.0
c      1.0
d      NaN
e      1.0
dtype: float64
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])
>>> b
a      0.0
b      1.0
c      2.0
d      NaN
f      1.0
dtype: float64
>>> a.gt(b, fill_value=0)
```

```

a      True
b      False
c      False
d      False
e      True
f      False
dtype: bool

```

```

hist = hist_series(
    self: Series,
    by=None,
    ax=None,
    grid: bool = True,
    xlabelsize: int | None = None,
    xrot: float | None = None,
    ylabelsize: int | None = None,
    yrot: float | None = None,
    figsize: tuple[int, int] | None = None,
    bins: int | Sequence[int] = 10,
    backend: str | None = None,
    legend: bool = False,
    **kwargs
) from pandas.plotting
    Draw histogram of the input series using matplotlib.

```

Parameters

```

by : object, optional
    If passed, then used to form histograms for separate groups.
ax : matplotlib axis object
    If not passed, uses gca().
grid : bool, default True
    Whether to show axis grid lines.
xlabelsize : int, default None
    If specified changes the x-axis label size.
xrot : float, default None
    Rotation of x axis labels.
ylabelsize : int, default None
    If specified changes the y-axis label size.
yrot : float, default None
    Rotation of y axis labels.
figsize : tuple, default None
    Figure size in inches by default.
bins : int or sequence, default 10
    Number of histogram bins to be used. If an integer is given, bins + 1
    bin edges are calculated and returned. If bins is a sequence, gives
    bin edges, including left edge of first bin and right edge of last
    bin. In this case, bins is returned unmodified.
backend : str, default None
    Backend to use instead of the backend specified in the option
    ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
    specify the ``plotting.backend`` for the whole session, set
    ``pd.options.plotting.backend``.
legend : bool, default False
    Whether to show the legend.

```

****kwargs**

To be passed to the actual plotting function.

Returns

```

matplotlib.axes.Axes
    A histogram plot.

```

See Also

```

matplotlib.axes.Axes.hist : Plot a histogram using matplotlib.

```

Examples

For Series:

```

.. plot::
    :context: close-figs

    >>> lst = ["a", "a", "a", "b", "b", "b"]
    >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)

```



```

>>> hist = ser.hist()

For Groupby:

.. plot::
   :context: close-figs

>>> lst = ["a", "a", "a", "b", "b", "b"]
>>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
>>> hist = ser.groupby(level=0).hist()

```

`idxmax`(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Hashable from pandas.core.indexes.common

Return the row label of the maximum value.

If multiple values equal the maximum, the first row label with that value is returned.

Parameters

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs

Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Index

Label of the maximum value.

Raises

ValueError

If the Series is empty.

See Also

`numpy.argmax` : Return indices of the maximum values along the given axis.

`DataFrame.idxmax` : Return index of first occurrence of maximum over requested axis.

`Series.idxmin` : Return index *label* of the first occurrence of minimum of values.

Notes

This method is the Series version of ``ndarray.argmax``. This method returns the label of the maximum, while ``ndarray.argmax`` returns the position. To get the position, use ``series.values.argmax()``.

Examples

```

>>> s = pd.Series(data=[1, None, 4, 3, 4], index=["A", "B", "C", "D", "E"])
>>> s
A    1.0
B    NaN
C    4.0
D    3.0
E    4.0
dtype: float64

>>> s.idxmax()
'C'

```

`idxmin`(self, axis: Axis = 0, skipna: bool = True, *args, **kwargs) -> Hashable from pandas.core.indexes.common

Return the row label of the minimum value.

If multiple values equal the minimum, the first row label with that value is returned.

Parameters

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True
 Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

*args, **kwargs
 Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Index
 Label of the minimum value.

Raises

ValueError
 If the Series is empty.

See Also

numpy.argmax : Return indices of the minimum values along the given axis.
 DataFrame.idxmin : Return index of first occurrence of minimum over requested axis.
 Series.idxmax : Return index *label* of the first occurrence of maximum of values.

Notes

This method is the Series version of ``ndarray.argmax``. This method returns the label of the minimum, while ``ndarray.argmax`` returns the position. To get the position, use ``series.values.argmax()``.

Examples

```
>>> s = pd.Series(data=[1, None, 4, 1], index=["A", "B", "C", "D"])
>>> s
A    1.0
B    NaN
C    4.0
D    1.0
dtype: float64

>>> s.idxmin()
'A'
```

```
info(
    self,
    verbose: bool | None = None,
    buf: IO[str] | None = None,
    max_cols: int | None = None,
    memory_usage: bool | str | None = None,
    show_counts: bool = True
) -> None from pandas.core.series.Series
Print a concise summary of a Series.
```

This method prints information about a Series including the index dtype, non-NA values and memory usage.

Parameters

verbose : bool, optional
 Whether to print the full summary. By default, the setting in ``pandas.options.display.max\_info\_columns`` is followed.

buf : writable buffer, defaults to sys.stdout
 Where to send the output. By default, the output is printed to sys.stdout. Pass a writable buffer if you need to further process the output.

max_cols : int, optional
 Unused, exists only for compatibility with DataFrame.info.

memory_usage : bool, str, optional
 Specifies whether total memory usage of the Series elements (including the index) should be displayed. By default, this follows the ``pandas.options.display.memory\_usage`` setting.

True always show memory usage. False never shows memory usage. A value of 'deep' is equivalent to "True with deep introspection". Memory usage is shown in human-readable units (base-2

representation). Without deep introspection a memory estimation is made based in column dtype and number of rows assuming values consume the same memory amount for corresponding dtypes. With deep memory introspection, a real memory usage calculation is performed at the cost of computational resources. See the :ref:`Frequently Asked Questions <df-memory-usage>` for more details.

`show_counts` : bool, optional
Whether to show the non-null counts. By default, this is shown only if the DataFrame is smaller than ``pandas.options.display.max\_info\_rows`` and ``pandas.options.display.max\_info\_columns``. A value of True always shows the counts, and False never shows the counts.

Returns

None
This method prints a summary of a Series and returns None.

See Also

`Series.describe`: Generate descriptive statistics of Series.
`Series.memory_usage`: Memory usage of Series.

Examples

>>> int_values = [1, 2, 3, 4, 5]
>>> text_values = ["alpha", "beta", "gamma", "delta", "epsilon"]
>>> s = pd.Series(text_values, index=int_values)
>>> s.info()
<class 'pandas.Series'>
Index: 5 entries, 1 to 5
Series name: None
Non-Null Count Dtype
----- ----
5 non-null str
dtypes: str(1)
memory usage: 106.0 bytes

Prints a summary excluding information about its values:

>>> s.info(verbose=False)
<class 'pandas.Series'>
Index: 5 entries, 1 to 5
dtypes: str(1)
memory usage: 106.0 bytes

Pipe output of `Series.info` to buffer instead of `sys.stdout`, get buffer content and writes to a text file:

>>> import io
>>> buffer = io.StringIO()
>>> s.info(buf=buffer)
>>> s = buffer.getvalue()
>>> with open("df_info.txt", "w", encoding="utf-8") as f: # doctest: +SKIP
... f.write(s)
260

The `memory_usage` parameter allows deep introspection mode, specially useful for big Series and fine-tune memory optimization:

>>> random_strings_array = np.random.choice(["a", "b", "c"], 10**6)
>>> s = pd.Series(np.random.choice(["a", "b", "c"], 10**6))
>>> s.info()
<class 'pandas.Series'>
RangeIndex: 1000000 entries, 0 to 999999
Series name: None
Non-Null Count Dtype
----- ----
1000000 non-null str
dtypes: str(1)
memory usage: 8.6 MB

>>> s.info(memory_usage="deep")
<class 'pandas.Series'>
RangeIndex: 1000000 entries, 0 to 999999
Series name: None

```

Non-Null Count    Dtype
-----
1000000 non-null  str
dtypes: str(1)
memory usage: 8.6 MB

```

`isin(self, values) -> Series` from `pandas.core.series.Series`
Whether elements in Series are contained in `values`.

Return a boolean Series showing whether each element in the Series matches an element in the passed sequence of `values` exactly.

Parameters

`values` : set or list-like
The sequence of values to test. Passing in a single string will raise a `TypeError`. Instead, turn a single string into a list of one element.

Returns

Series

Series of booleans indicating if each element is in values.

Raises

`TypeError`
* If `values` is a string

See Also

`DataFrame.isin` : Equivalent method on `DataFrame`.

Examples

```

>>> s = pd.Series(
...     ["llama", "cow", "llama", "beetle", "llama", "hippo"], name="animal"
... )
>>> s.isin(["cow", "llama"])
0    True
1    True
2    True
3    False
4    True
5    False
Name: animal, dtype: bool

```

To invert the boolean values, use the `~` operator:

```

>>> ~s.isin(["cow", "llama"])
0    False
1    False
2    False
3     True
4    False
5     True
Name: animal, dtype: bool

```

Passing a single string as `s.isin('llama')` will raise an error. Use a list of one element instead:

```

>>> s.isin(["llama"])
0    True
1    False
2    True
3    False
4    True
5    False
Name: animal, dtype: bool

```

Strings and integers are distinct and are therefore not comparable:

```

>>> pd.Series([1]).isin(["1"])
0    False
dtype: bool
>>> pd.Series([1.1]).isin(["1.1"])
0    False

```

```

dtype: bool

isna(self) -> Series from pandas.core.series.Series
Detect missing values.

Return a boolean same-sized Series indicating if the values are NA.
NA values, such as None or :attr:`numpy.NaN`, get mapped to True
values.
Everything else gets mapped to False values. Characters such as empty
strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns
-----
Series
    Mask of bool values for each element in Series that
    indicates whether an element is an NA value.

See Also
-----
DataFrame.isna : Detect missing values.
DataFrame.isnull : Alias of isna.
Series.notna : Boolean inverse of isna.
DataFrame.notna : Boolean inverse of isna.
Series.notnull : Alias of notna.
DataFrame.notnull : Alias of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
isna : Top-level isna.

Examples
-----
Show which entries in a Series are NA.

>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
>>> ser.isna()
0    False
1    False
2     True
dtype: bool

isnull(self) -> Series from pandas.core.series.Series
Series.isnull is an alias for Series.isna.

Detect missing values.

Return a boolean same-sized object indicating if the values are NA.
NA values, such as None or :attr:`numpy.NaN`, gets mapped to True
values.
Everything else gets mapped to False values. Characters such as empty
strings ``''`` or :attr:`numpy.inf` are not considered NA values.

Returns
-----
Series/DataFrame
    Mask of bool values for each element in Series/DataFrame
    that indicates whether an element is an NA value.

See Also
-----
Series.isnull : Alias of isna.
DataFrame.isnull : Alias of isna.
Series.notna : Boolean inverse of isna.
DataFrame.notna : Boolean inverse of isna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
isna : Top-level isna.

Examples
-----
Show which entries in a DataFrame are NA.

>>> df = pd.DataFrame(

```

```

...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born      name      toy
0  5.0      NaT    Alfred      NaN
1  6.0 1939-05-27    Batman  Batmobile
2  NaN 1940-04-25      Joker

```

```

>>> df.isna()
   age      born      name      toy
0  False    True    False    True
1  False   False    False   False
2    True   False    False   False

```

Show which entries in a Series are NA.

```

>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2     NaN
dtype: float64

>>> ser.isna()
0    False
1    False
2     True
dtype: bool

```

`items(self)` -> Iterable[tuple[Hashable, Any]] from `pandas.core.series.Series`
 Lazily iterate over (index, value) tuples.

This method returns an iterable tuple (index, value). This is convenient if you want to create a lazy iterator.

Returns

 iterable

Iterable of tuples containing the (index, value) pairs from a Series.

See Also

`DataFrame.items` : Iterate over (column name, Series) pairs.

`DataFrame.iterrows` : Iterate over DataFrame rows as (index, Series) pairs.

Examples

```

>>> s = pd.Series(["A", "B", "C"])
>>> for index, value in s.items():
...     print(f"Index : {index}, Value : {value}")
Index : 0, Value : A
Index : 1, Value : B
Index : 2, Value : C

```

`keys(self)` -> Index from `pandas.core.series.Series`
 Return alias for index.

Returns

Index

Index of the Series.

See Also

`Series.index` : The index (axis labels) of the Series.

Examples

```

-----
>>> s = pd.Series([1, 2, 3], index=[0, 1, 2])
>>> s.keys()
Index([0, 1, 2], dtype='int64')

kurt(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
    Return unbiased kurtosis over requested axis.

    Kurtosis obtained using Fisher's definition of
    kurtosis (kurtosis of normal == 0.0). Normalized by N-1.

    Parameters
    -----
    axis : {index (0)}
        Axis for the function to be applied on.
        For `Series` this parameter is unused and defaults to 0.

        For DataFrames, specifying ``axis=None`` will apply the aggregation
        across both axes.

        .. versionadded:: 2.0.0

    skipna : bool, default True
        Exclude NA/null values when computing the result.
    numeric_only : bool, default False
        Include only float, int, boolean columns.

    **kwargs
        Additional keyword arguments to be passed to the function.

    Returns
    -----
    scalar
        Unbiased kurtosis.

    See Also
    -----
    Series.skew : Return unbiased skew over requested axis.
    Series.var : Return unbiased variance over requested axis.
    Series.std : Return unbiased standard deviation over requested axis.

    Examples
    -----
    >>> s = pd.Series([1, 2, 2, 3], index=["cat", "dog", "dog", "mouse"])
    >>> s
    cat    1
    dog    2
    dog    2
    mouse  3
    dtype: int64
    >>> s.kurt()
    1.5

    kurtosis = kurt(
        self,
        *,
        axis: Axis | None = 0,
        skipna: bool = True,
        numeric_only: bool = False,
        **kwargs
    )

    le(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
        Return Less than or equal to of series and other, element-wise (binary operator

    Equivalent to ``series <= other``, but with support to substitute a
    fill_value for missing data in either one of the inputs.

    Parameters
    -----

```

```

other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

```

Returns

```

Series
    The result of the operation.

```

See Also

```

Series.ge : Return elementwise Greater than or equal to of series and other.
Series.lt : Return elementwise Less than of series and other.
Series.gt : Return elementwise Greater than of series and other.
Series.eq : Return elementwise equal to of series and other.

```

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
e    1.0
dtype: float64
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])
>>> b
a    0.0
b    1.0
c    2.0
d    NaN
f    1.0
dtype: float64
>>> a.le(b, fill_value=0)
a    False
b     True
c     True
d    False
e    False
f     True
dtype: bool

```

```

lt(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
    Return Greater than of series and other, element-wise (binary operator `lt`).

```

Equivalent to ``series > other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

```

other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

```


Returns

Series

The result of the operation.

See Also

Series.gt : Element-wise Greater than, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_

for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan, 1], index=['a', 'b', 'c', 'd', 'e'])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
e    1.0
```

```
dtype: float64
```

```
>>> b = pd.Series([0, 1, 2, np.nan, 1], index=['a', 'b', 'c', 'd', 'f'])
```

```
>>> b
```

```
a    0.0
```

```
b    1.0
```

```
c    2.0
```

```
d    NaN
```

```
f    1.0
```

```
dtype: float64
```

```
>>> a.gt(b, fill_value=0)
```

```
a    True
```

```
b    False
```

```
c    False
```

```
d    False
```

```
e    True
```

```
f    False
```

```
dtype: bool
```

map(

self,

func: Callable | Mapping | Series | None = None,

na_action: Literal['ignore'] | None = None,

engine: Callable | None = None,

**kwargs

) -> Series from pandas.core.series.Series

Map values of Series according to an input mapping or function.

Used for substituting each value in a Series with another value, that may be derived from a function, a ``dict`` or a :class:`Series`.

Parameters

func : function, collections.abc.Mapping subclass or Series

Function or mapping correspondence.

na_action : {None, 'ignore'}, default None

If 'ignore', propagate NaN values, without passing them to the mapping correspondence.

engine : decorator, optional

Choose the execution engine to use to run the function. Only used for functions. If ``map`` is called with a mapping or ``Series``, an exception will be raised. If ``engine`` is not provided the function will be executed by the regular Python interpreter.

Options include JIT compilers such as Numba, Bodo or Blosc2, which in some cases can speed up the execution. To use an executor you can provide the decorators ``numba.jit``, ``numba.njit``, ``bodo.jit`` or ``blosc2.jit``. You can also provide the decorator with parameters, like ``numba.jit(nogit=True)``.

Not all functions can be executed with all execution engines. In general, JIT compilers will require type stability in the function (no variable should change data type during the execution). And not all pandas and NumPy APIs are supported. Check the engine documentation for limitations.

.. versionadded:: 3.0.0

```

**kwargs
    Additional keyword arguments to pass as keywords arguments to
    `arg`.

    .. versionadded:: 3.0.0

Returns
-----
Series
    Same index as caller.

See Also
-----
Series.apply : For applying more complex functions on a Series.
Series.replace: Replace values given in `to_replace` with `value`.
DataFrame.apply : Apply a function row-/column-wise.
DataFrame.map : Apply a function elementwise on a whole DataFrame.

Notes
-----
When ``arg`` is a dictionary, values in Series that are not in the
dictionary (as keys) are converted to ``NaN``. However, if the
dictionary is a ``dict`` subclass that defines ``__missing__`` (i.e.
provides a method for default values), then this default is used
rather than ``NaN``.

Examples
-----
>>> s = pd.Series(["cat", "dog", np.nan, "rabbit"])
>>> s
0      cat
1      dog
2      NaN
3  rabbit
dtype: str

``map`` accepts a ``dict`` or a ``Series``. Values that are not found
in the ``dict`` are converted to ``NaN``, unless the dict has a default
value (e.g. ``defaultdict``):

>>> s.map({"cat": "kitten", "dog": "puppy"})
0  kitten
1  puppy
2    NaN
3    NaN
dtype: str

It also accepts a function:

>>> s.map("I am a {}".format)
0      I am a cat
1      I am a dog
2      I am a nan
3  I am a rabbit
dtype: str

To avoid applying the function to missing values (and keep them as
``NaN``) ``na_action='ignore'`` can be used:

>>> s.map("I am a {}".format, na_action="ignore")
0      I am a cat
1      I am a dog
2          NaN
3  I am a rabbit
dtype: str

For categorical data, the function is only applied to the categories:

>>> s = pd.Series(list("cabaa"))
>>> s.map(print)
c
a
b
a
a
0  None

```

```

1    None
2    None
3    None
4    None
dtype: object

```

```

>>> s_cat = s.astype("category")
>>> s_cat.map(print) # function called once per unique category
a
b
c
0    None
1    None
2    None
3    None
4    None
dtype: object

```

```

max(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return the maximum of the values over the requested axis.

If you want the *index* of the maximum, use ``idxmax``.
This is the equivalent of the ``numpy.ndarray`` method ``argmax``.

Parameters
-----
axis : {index (0)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.

    .. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.
**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
scalar or Series (if level specified)
    The maximum of the values in the Series.

See Also
-----
numpy.max : Equivalent numpy function for arrays.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
>>> idx = pd.MultiIndex.from_arrays(
...     [
...         ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm      dog      4
         falcon    2

```

```
cold    fish    0
        spider   8
Name: legs, dtype: int64
```

```
>>> s.max()
8
```

```
mean(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Any from pandas.core.series.Series
Return the mean of the values over the requested axis.
```

Parameters

```
axis : {index (0)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.
```

```
.. versionadded:: 2.0.0
```

```
skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns.
**kwargs
    Additional keyword arguments to be passed to the function.
```

Returns

```
scalar or Series (if level specified)
    Mean of the values for the requested axis.
```

See Also

```
numpy.median : Equivalent numpy function for computing median.
Series.sum : Sum of the values.
Series.median : Median of the values.
Series.std : Standard deviation of the values.
Series.var : Variance of the values.
Series.min : Minimum value.
Series.max : Maximum value.
```

Examples

```
>>> s = pd.Series([1, 2, 3])
>>> s.mean()
2.0
```

```
median(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) -> Any from pandas.core.series.Series
Return the median of the values over the requested axis.
```

Parameters

```
axis : {index (0)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

    For DataFrames, specifying ``axis=None`` will apply the aggregation
    across both axes.
```

```
.. versionadded:: 2.0.0
```

skipna : bool, default True
 Exclude NA/null values when computing the result.
 numeric_only : bool, default False
 Include only float, int, boolean columns.
 **kwargs
 Additional keyword arguments to be passed to the function.

Returns

 scalar or Series (if level specified)
 Median of the values for the requested axis.

See Also

 numpy.median : Equivalent numpy function for computing median.
 Series.sum : Sum of the values.
 Series.median : Median of the values.
 Series.std : Standard deviation of the values.
 Series.var : Variance of the values.
 Series.min : Minimum value.
 Series.max : Maximum value.

Examples

 >>> s = pd.Series([1, 2, 3])
 >>> s.median()
 2.0

With a DataFrame

```
>>> df = pd.DataFrame({"a": [1, 2], "b": [2, 3]}, index=["tiger", "zebra"])
>>> df
      a  b
tiger 1  2
zebra 2  3
>>> df.median()
a    1.5
b    2.5
dtype: float64
```

Using axis=1

```
>>> df.median(axis=1)
tiger    1.5
zebra    2.5
dtype: float64
```

In this case, `numeric\_only` should be set to `True`
 to avoid getting an error.

```
>>> df = pd.DataFrame({"a": [1, 2], "b": ["T", "Z"]}, index=["tiger", "zebra"])
>>> df.median(numeric_only=True)
a    1.5
dtype: float64
```

memory_usage(self, index: bool = True, deep: bool = False) -> int from pandas.core.series.Series
 Return the memory usage of the Series.

The memory usage can optionally include the contribution of
 the index and of elements of `object` dtype.

Parameters

 index : bool, default True
 Specifies whether to include the memory usage of the Series index.
 deep : bool, default False
 If True, introspect the data deeply by interrogating
 `object` dtypes for system-level memory consumption, and include
 it in the returned value.

Returns

 int
 Bytes of memory consumed.

See Also

numpy.ndarray.nbytes : Total bytes consumed by the elements of the array.
DataFrame.memory_usage : Bytes consumed by a DataFrame.

Examples

```
>>> s = pd.Series(range(3))
>>> s.memory_usage()
156
```

Not including the index gives the size of the rest of the data, which is necessarily smaller:

```
>>> s.memory_usage(index=False)
24
```

The memory footprint of `object` values is ignored by default:

```
>>> s = pd.Series(["a", "b"])
>>> s.values
<ArrowStringArray>
['a', 'b']
Length: 2, dtype: str
>>> s.memory_usage()
150
>>> s.memory_usage(deep=True)
150
```

```
min(
    self,
    *,
    axis: Axis | None = 0,
    skipna: bool = True,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return the minimum of the values over the requested axis.
```

If you want the *index* of the minimum, use ``idxmin``.
This is the equivalent of the ``numpy.ndarray`` method ``argmin``.

Parameters

axis : {index (0)}

Axis for the function to be applied on.
For `Series` this parameter is unused and defaults to 0.

For DataFrames, specifying ``axis=None`` will apply the aggregation across both axes.

.. versionadded:: 2.0.0

skipna : bool, default True
Exclude NA/null values when computing the result.

numeric_only : bool, default False
Include only float, int, boolean columns.

****kwargs**
Additional keyword arguments to be passed to the function.

Returns

scalar or Series (if level specified)
The minimum of the values in the Series.

See Also

numpy.min : Equivalent numpy function for arrays.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples

```

-----
>>> idx = pd.MultiIndex.from_arrays(
...     [["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"]],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded  animal
warm     dog      4
         falcon   2
cold     fish     0
         spider   8
Name: legs, dtype: int64

>>> s.min()
0

```

`mod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Modulo of series and other, element-wise (binary operator ``mod``).

Equivalent to ``series % other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

```

-----
other : Series or scalar value
        Series with which to compute modulo.
level : int or name
        Broadcast across a level, matching Index values on the
        passed MultiIndex level.
fill_value : None or float value, default None (NaN)
        Fill existing missing (NaN) values, and any new element needed for
        successful Series alignment, with this value before computation.
        If data in both corresponding Series locations is missing
        the result of filling (at that location) will be missing.
axis : {0 or 'index'}
        Unused. Parameter needed for compatibility with DataFrame.

```

Returns

```

-----
Series
    The result of the operation.

```

See Also

```

-----
Series.rmod : Reverse of the Modulo operator, see
`Python documentation
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`
for more details.

```

Examples

```

-----
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.mod(b, fill_value=0)
a    0.0
b    NaN
c    NaN
d    0.0
e    NaN
dtype: float64

```

`mode(self, dropna: bool = True) -> Series from pandas.core.series.Series`
Return the mode(s) of the Series.

The mode is the value that appears most often. There can be multiple modes.

Always returns Series even if only one value is returned.

Parameters

dropna : bool, default True
Don't consider counts of NaN/NaT.

Returns

Series
Modes of the Series in sorted order.

See Also

numpy.mode : Equivalent numpy function for computing median.
Series.sum : Sum of the values.
Series.median : Median of the values.
Series.std : Standard deviation of the values.
Series.var : Variance of the values.
Series.min : Minimum value.
Series.max : Maximum value.

Examples

```
>>> s = pd.Series([2, 4, 2, 2, 4, None])
>>> s.mode()
0    2.0
dtype: float64
```

More than one mode:

```
>>> s = pd.Series([2, 4, 8, 2, 4, None])
>>> s.mode()
0    2.0
1    4.0
dtype: float64
```

With and without considering null value:

```
>>> s = pd.Series([2, 4, None, None, 4, None])
>>> s.mode(dropna=False)
0    NaN
dtype: float64
>>> s = pd.Series([2, 4, None, None, 4, None])
>>> s.mode()
0    4.0
dtype: float64
```

```
mul(
    self,
    other,
    level: Level | None = None,
    fill_value: float | None = None,
    axis: Axis = 0
) -> Series from pandas.core.series.Series
Return Multiplication of series and other, element-wise (binary operator `mul`).
```

Equivalent to ``series \* other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : Series or scalar value
With which to compute the multiplication.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

Series.rmul : Reverse of the Multiplication operator, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
```

```
>>> b
```

```
a    1.0
```

```
b    NaN
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.multiply(b, fill_value=0)
```

```
a    1.0
```

```
b    0.0
```

```
c    0.0
```

```
d    0.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.mul(5, fill_value=0)
```

```
a    5.0
```

```
b    5.0
```

```
c    5.0
```

```
d    0.0
```

```
dtype: float64
```

```
multiply = mul(  
    self,  
    other,  
    level: Level | None = None,  
    fill_value: float | None = None,  
    axis: Axis = 0  
) -> Series
```

ne(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series
Return Not equal to of series and other, element-wise (binary operator `ne`).

Equivalent to ``series != other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.eq` : Reverse of the Not equal to operator, see
`Python documentation`
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`
for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.ne(b, fill_value=0)
a    False
b     True
c     True
d     True
e     True
dtype: bool
```

`nlargest(self, n: int = 5, keep: Literal['first', 'last', 'all'] = 'first') -> Series from pandas`
Return the largest `n` elements.

Parameters

`n` : int, default 5
Return this many descending sorted values.
`keep` : {'first', 'last', 'all'}, default 'first'
When there are duplicate values that cannot all fit in a
Series of `n` elements:

- ``first`` : return the first `n` occurrences in order of appearance.
- ``last`` : return the last `n` occurrences in reverse order of appearance.
- ``all`` : keep all occurrences. This can result in a Series of size larger than `n`.

Returns

Series

The `n` largest values in the Series, sorted in decreasing order.

See Also

`Series.nsmallest`: Get the `n` smallest elements.
`Series.sort_values`: Sort Series by values.
`Series.head`: Return the first `n` rows.

Notes

Faster than ``.sort\_values(ascending=False).head(n)`` for small `n` relative to the size of the ``Series`` object.

Examples

```
>>> countries_population = {
...     "Italy": 59000000,
...     "France": 65000000,
...     "Malta": 434000,
...     "Maldives": 434000,
...     "Brunei": 434000,
...     "Iceland": 337000,
...     "Nauru": 11300,
...     "Tuvalu": 11300,
```

```

...     "Anguilla": 11300,
...     "Montserrat": 5200,
... }
>>> s = pd.Series(countries_population)
>>> s
Italy          59000000
France         65000000
Malta           434000
Maldives        434000
Brunei          434000
Iceland         337000
Nauru           11300
Tuvalu          11300
Anguilla        11300
Montserrat      5200
dtype: int64

```

The `n` largest elements where ``n=5`` by default.

```

>>> s.nlargest()
France         65000000
Italy          59000000
Malta           434000
Maldives        434000
Brunei          434000
dtype: int64

```

The `n` largest elements where ``n=3``. Default `keep` value is 'first' so Malta will be kept.

```

>>> s.nlargest(3)
France         65000000
Italy          59000000
Malta           434000
dtype: int64

```

The `n` largest elements where ``n=3`` and keeping the last duplicates. Brunei will be kept since it is the last with value 434000 based on the index order.

```

>>> s.nlargest(3, keep="last")
France         65000000
Italy          59000000
Brunei          434000
dtype: int64

```

The `n` largest elements where ``n=3`` with all duplicates kept. Note that the returned Series has five elements due to the three duplicates.

```

>>> s.nlargest(3, keep="all")
France         65000000
Italy          59000000
Malta           434000
Maldives        434000
Brunei          434000
dtype: int64

```

```

notna(self) -> Series from pandas.core.series.Series
Detect existing (non-missing) values.

```

Return a boolean same-sized Series indicating if the values are not NA. Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values. NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series

Mask of bool values for each element in Series that indicates whether an element is not an NA value.

See Also

Series.isna : Detect missing values.
DataFrame.isna : Detect missing values.
Series.isnull : Alias of isna.

DataFrame.isnull : Alias of isna.
DataFrame.notna : Boolean inverse of isna.
DataFrame.notnull : Alias of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
notna : Top-level notna.

Examples

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
>>> ser.notna()
0     True
1     True
2    False
dtype: bool
```

notnull(self) -> Series from pandas.core.series.Series
Series.notnull is an alias for Series.notna.

Detect existing (non-missing) values.

Return a boolean same-sized object indicating if the values are not NA.
Non-missing values get mapped to True. Characters such as empty strings ``''`` or :attr:`numpy.inf` are not considered NA values.
NA values, such as None or :attr:`numpy.NaN`, get mapped to False values.

Returns

Series/DataFrame
Mask of bool values for each element in Series/DataFrame that indicates whether an element is not an NA value.

See Also

Series.notnull : Alias of notna.
DataFrame.notnull : Alias of notna.
Series.isna : Boolean inverse of notna.
DataFrame.isna : Boolean inverse of notna.
Series.dropna : Omit axes labels with missing values.
DataFrame.dropna : Omit axes labels with missing values.
notna : Top-level notna.

Examples

Show which entries in a DataFrame are not NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
>>> df
   age      born      name      toy
0  5.0      NaT  Alfred      NaN
1  6.0  1939-05-27  Batman  Batmobile
2  NaN  1940-04-25      Joker

>>> df.notna()
   age      born      name      toy
0  True    False    True    False
1  True     True     True     True
2  False     True     True     True
```

Show which entries in a Series are not NA.

```
>>> ser = pd.Series([5, 6, np.nan])
>>> ser
0    5.0
1    6.0
2    NaN
dtype: float64
```

```
>>> ser.notna()
0     True
1     True
2    False
dtype: bool
```

`nsmallest(self, n: int = 5, keep: Literal['first', 'last', 'all'] = 'first') -> Series` from `pandas`
Return the smallest `n` elements.

Parameters

`n` : int, default 5
Return this many ascending sorted values.
`keep` : {'first', 'last', 'all'}, default 'first'
When there are duplicate values that cannot all fit in a Series of `n` elements:

- `'first'` : return the first `n` occurrences in order of appearance.
- `'last'` : return the last `n` occurrences in reverse order of appearance.
- `'all'` : keep all occurrences. This can result in a Series of size larger than `n`.

Returns

Series

The `n` smallest values in the Series, sorted in increasing order.

See Also

`Series.nlargest`: Get the `n` largest elements.
`Series.sort_values`: Sort Series by values.
`Series.head`: Return the first `n` rows.

Notes

Faster than ``.sort_values().head(n)`` for small `n` relative to the size of the `Series` object.

Examples

```
>>> countries_population = {
...     "Italy": 59000000,
...     "France": 65000000,
...     "Brunei": 434000,
...     "Malta": 434000,
...     "Maldives": 434000,
...     "Iceland": 337000,
...     "Nauru": 11300,
...     "Tuvalu": 11300,
...     "Anguilla": 11300,
...     "Montserrat": 5200,
... }
>>> s = pd.Series(countries_population)
>>> s
Italy          59000000
France         65000000
Brunei          434000
Malta           434000
Maldives        434000
Iceland         337000
Nauru            11300
Tuvalu           11300
Anguilla         11300
Montserrat        5200
dtype: int64
```

The `n` smallest elements where ``n=5`` by default.

```
>>> s.nsmallest()
Montserrat    5200
Nauru          11300
Tuvalu         11300
Anguilla       11300
Iceland        337000
dtype: int64
```

The `n` smallest elements where ``n=3``. Default `keep` value is 'first' so Nauru and Tuvalu will be kept.

```
>>> s.nsmallest(3)
Montserrat    5200
Nauru          11300
Tuvalu         11300
dtype: int64
```

The `n` smallest elements where ``n=3`` and keeping the last duplicates. Anguilla and Tuvalu will be kept since they are the last with value 11300 based on the index order.

```
>>> s.nsmallest(3, keep="last")
Montserrat    5200
Anguilla       11300
Tuvalu         11300
dtype: int64
```

The `n` smallest elements where ``n=3`` with all duplicates kept. Note that the returned Series has four elements due to the three duplicates.

```
>>> s.nsmallest(3, keep="all")
Montserrat    5200
Nauru          11300
Tuvalu         11300
Anguilla       11300
dtype: int64
```

pop(self, item: Hashable) -> Any from pandas.core.series.Series
Return item and drops from series. Raise KeyError if not found.

Parameters

item : label
Index of the element that needs to be removed.

Returns

scalar
Value that is popped from series.

See Also

Series.drop: Drop specified values from Series.
Series.drop_duplicates: Return Series with duplicate values removed.

Examples

```
>>> ser = pd.Series([1, 2, 3])

>>> ser.pop(0)
1

>>> ser
1    2
2    3
dtype: int64
```

pow(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.ser.
Return Exponential power of series and other, element-wise (binary operator `pow`).

Equivalent to ``series \*\* other``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

```

-----
other : object
    When a Series is provided, will align on indexes. For all other types,
    will behave the same as ``==`` but with possibly different results due
    to the other arguments.
level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : None or float value, default None (NaN)
    Fill existing missing (NaN) values, and any new element needed for
    successful Series alignment, with this value before computation.
    If data in both corresponding Series locations is missing
    the result of filling (at that location) will be missing.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.

Returns
-----
Series
    The result of the operation.

See Also
-----
Series.rpow : Reverse of the Exponential power operator, see
    `Python documentation
    <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`
    for more details.

```

Examples

```

-----
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.pow(b, fill_value=0)
a    1.0
b    1.0
c    1.0
d    0.0
e    NaN
dtype: float64

```

```

prod(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
) from pandas.core.series.Series
    Return the product of the values over the requested axis.

```

By default, missing values are skipped. To include them in the calculation, set ``skipna`` parameter to False.

Parameters

```

axis : {index (0)}
    Axis for the function to be applied on.
    For `Series` this parameter is unused and defaults to 0.

.. warning::
    The behavior of DataFrame.prod with ``axis=None`` is deprecated,
    in a future version this will reduce over both axes and return a scalar
    To retain the old behavior, pass axis=0 (or do not pass axis).

```

```

        .. versionadded:: 2.0.0
    skipna : bool, default True
        Exclude NA/null values when computing the result.
    numeric_only : bool, default False
        Include only float, int, boolean columns. Not implemented for Series.
    min_count : int, default 0
        The required number of valid values to perform the operation. If fewer than
        ``min_count`` non-NA values are present the result will be NA.
    **kwargs
        Additional keyword arguments to be passed to the function.

Returns
-----
scalar
    Value containing the calculation referenced in the description.

See Also
-----
Series.sum : Return the sum.
Series.min : Return the minimum.
Series.max : Return the maximum.
Series.idxmin : Return the index of the minimum.
Series.idxmax : Return the index of the maximum.

DataFrame.sum : Return the sum over the requested axis.
DataFrame.min : Return the minimum over the requested axis.
DataFrame.max : Return the maximum over the requested axis.
DataFrame.idxmin : Return the index of the minimum over the requested axis.
DataFrame.idxmax : Return the index of the maximum over the requested axis.

Examples
-----
By default, the product of an empty or all-NA Series is ``1``

>>> pd.Series([], dtype="float64").prod()
1.0

This can be controlled with the ``min_count`` parameter

>>> pd.Series([], dtype="float64").prod(min_count=1)
nan

Thanks to the ``skipna`` parameter, ``min_count`` handles all-NA and
empty series identically.

>>> pd.Series([np.nan]).prod()
1.0
>>> pd.Series([np.nan]).prod(min_count=1)
nan

product = prod(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
    **kwargs
)

quantile(
    self,
    q: float | Sequence[float] | AnyArrayLike = 0.5,
    interpolation: QuantileInterpolation = 'linear'
) -> float | Series from pandas.core.series.Series
    Return value at the given quantile.

Parameters
-----
q : float or array-like, default 0.5 (50% quantile)
    The quantile(s) to compute, which can lie in range: 0 <= q <= 1.
interpolation : {'linear', 'lower', 'higher', 'midpoint', 'nearest'}
    This optional parameter specifies the interpolation method to use,
    when the desired quantile lies between two data points ``i`` and ``j``:

    * linear: ``i + (j - i) * (x-i)/(j-i)``, where ``(x-i)/(j-i)`` is
      the fractional part of the index surrounded by ``i > j``.

```


- * lower: ``i``.
- * higher: ``j``.
- * nearest: ``i`` or ``j`` whichever is nearest.
- * midpoint: $(`i` + `j`) / 2$.

Returns

float or Series

If ``q`` is an array, a Series will be returned where the index is ``q`` and the values are the quantiles, otherwise a float will be returned.

See Also

`core.window.Rolling.quantile` : Calculate the rolling quantile.
`numpy.percentile` : Returns the q-th percentile(s) of the array elements.

Examples

```

>>> s = pd.Series([1, 2, 3, 4])
>>> s.quantile(0.5)
2.5
>>> s.quantile([0.25, 0.5, 0.75])
0.25    1.75
0.50    2.50
0.75    3.25
dtype: float64

```

`radd(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Addition of series and other, element-wise (binary operator ``radd``).

Equivalent to ``other + series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
`level` : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.add` : Element-wise Addition, see
``Python documentation``
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>
for more details.

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0

```

```

e    NaN
dtype: float64
>>> a.add(b, fill_value=0)
a    2.0
b    1.0
c    1.0
d    1.0
e    NaN
dtype: float64

```

`rdiv = rtruediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

`rdivmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series` from `pandas.core`
Return Integer division and modulo of series and other, element-wise (binary operator ``r`

Equivalent to ``other divmod series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

2-Tuple of Series

The result of the operation.

See Also

`Series.divmod` : Element-wise Integer division and modulo, see

Python documentation

<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_ for more details.

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a

```

```

a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64

```

```

>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b

```

```

a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64

```

```

>>> a.divmod(b, fill_value=0)

```

```

(a    1.0
 b    inf
 c    inf
 d    0.0
 e    NaN
 dtype: float64,

```

```

a    0.0
 b    NaN
 c    NaN
 d    0.0
 e    NaN
 dtype: float64)

```

```

reindex(
    self,
    index=None,
    *,
    axis: Axis | None = None,
    method: ReindexMethod | None = None,
    copy: bool | lib.NoDefault = <no_default>,
    level: Level | None = None,
    fill_value: Scalar | None = None,
    limit: int | None = None,
    tolerance=None
) -> Series from pandas.core.series.Series
    Conform Series to new index with optional filling logic.

Places NA/NaN in locations having no value in the previous index. A new object
is produced unless the new index is equivalent to the current one and
`copy=False`.

Parameters
-----
index : scalar, list-like, dict-like or function, optional
    A scalar, list-like, dict-like or functions transformations to
    apply to that axis' values.
axis : {0 or 'index'}, default 0
    The axis to rename. For `Series` this parameter is unused and defaults to 0.
method : {{None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}}
    Method to use for filling holes in reindexed DataFrame.
    Please note: this is only applicable to DataFrames/Series with a
    monotonically increasing/decreasing index.

    * None (default): don't fill gaps
    * pad / ffill: Propagate last valid observation forward to next
      valid.
    * backfill / bfill: Use next valid observation to fill gap.
    * nearest: Use nearest valid observations to fill gap.

copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
    for more details.

level : int or name
    Broadcast across a level, matching Index values on the
    passed MultiIndex level.
fill_value : scalar, default np.nan
    Value to use for missing values. Defaults to NaN, but can be any
    "compatible" value.
limit : int, default None
    Maximum number of consecutive elements to forward or backward fill.
tolerance : optional
    Maximum distance between original and new labels for inexact
    matches. The values of the index at the matching locations must
    satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

    Tolerance may be a scalar value, which applies the same tolerance
    to all values, or list-like, which applies variable tolerance per
    element. List-like includes list, tuple, array, Series, and must be
    the same size as the index and its dtype must exactly match the
    index's type.

Returns
-----
Series
    Series with changed index.

See Also
-----
DataFrame.set_index : Set row labels.
DataFrame.reset_index : Remove row labels or move them to new columns.

```

DataFrame.reindex_like : Change to same indices as other DataFrame.

Examples

``DataFrame.reindex`` supports two calling conventions

```
* ``(index=index_labels, columns=column_labels, ...)``  
* ``(labels, axis={{'index', 'columns'}}, ...)``
```

We *highly* recommend using keyword arguments to clarify your intent.

Create a DataFrame with some fictional data.

```
>>> index = ["Firefox", "Chrome", "Safari", "IE10", "Konqueror"]  
>>> columns = ["http_status", "response_time"]  
>>> df = pd.DataFrame(  
...     [[200, 0.04], [200, 0.02], [404, 0.07], [404, 0.08], [301, 1.0]],  
...     columns=columns,  
...     index=index,  
... )  
>>> df
```

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

Create a new index and reindex the DataFrame. By default values in the new index that do not have corresponding records in the DataFrame are assigned ``NaN``.

```
>>> new_index = ["Safari", "Iceweasel", "Comodo Dragon", "IE10", "Chrome"]  
>>> df.reindex(new_index)
```

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02

We can fill in the missing values by passing a value to the keyword ``fill\_value``. Because the index is not monotonically increasing or decreasing, we cannot use arguments to the keyword ``method`` to fill the ``NaN`` values.

```
>>> df.reindex(new_index, fill_value=0)
```

	http_status	response_time
Safari	404	0.07
Iceweasel	0	0.00
Comodo Dragon	0	0.00
IE10	404	0.08
Chrome	200	0.02

```
>>> df.reindex(new_index, fill_value="missing")
```

	http_status	response_time
Safari	404	0.07
Iceweasel	missing	missing
Comodo Dragon	missing	missing
IE10	404	0.08
Chrome	200	0.02

We can also reindex the columns.

```
>>> df.reindex(columns=["http_status", "user_agent"])
```

	http_status	user_agent
Firefox	200	NaN
Chrome	200	NaN
Safari	404	NaN
IE10	404	NaN
Konqueror	301	NaN

Or we can use "axis-style" keyword arguments

```
>>> df.reindex(["http_status", "user_agent"], axis="columns")
```

	http_status	user_agent
--	-------------	------------

Firefox	200	NaN
Chrome	200	NaN
Safari	404	NaN
IE10	404	NaN
Konqueror	301	NaN

To further illustrate the filling functionality in ``reindex``, we will create a DataFrame with a monotonically increasing index (for example, a sequence of dates).

```
>>> date_index = pd.date_range("1/1/2010", periods=6, freq="D")
>>> df2 = pd.DataFrame(
...     {"prices": [100, 101, np.nan, 100, 89, 88]}, index=date_index
... )
>>> df2
```

	prices
2010-01-01	100.0
2010-01-02	101.0
2010-01-03	NaN
2010-01-04	100.0
2010-01-05	89.0
2010-01-06	88.0

Suppose we decide to expand the DataFrame to cover a wider date range.

```
>>> date_index2 = pd.date_range("12/29/2009", periods=10, freq="D")
>>> df2.reindex(date_index2)
```

	prices
2009-12-29	NaN
2009-12-30	NaN
2009-12-31	NaN
2010-01-01	100.0
2010-01-02	101.0
2010-01-03	NaN
2010-01-04	100.0
2010-01-05	89.0
2010-01-06	88.0
2010-01-07	NaN

The index entries that did not have a value in the original data frame (for example, '2009-12-29') are by default filled with ``NaN``. If desired, we can fill in the missing values using one of several options.

For example, to back-propagate the last valid value to fill the ``NaN`` values, pass ``bfill`` as an argument to the ``method`` keyword.

```
>>> df2.reindex(date_index2, method="bfill")
```

	prices
2009-12-29	100.0
2009-12-30	100.0
2009-12-31	100.0
2010-01-01	100.0
2010-01-02	101.0
2010-01-03	NaN
2010-01-04	100.0
2010-01-05	89.0
2010-01-06	88.0
2010-01-07	NaN

Please note that the ``NaN`` value present in the original DataFrame (at index value 2010-01-03) will not be filled by any of the value propagation schemes. This is because filling while reindexing does not look at DataFrame values, but only compares the original and desired indexes. If you do want to fill in the ``NaN`` values present in the original DataFrame, use the ``fillna()`` method.

See the :ref:`user guide <basics.reindexing>` for more.

```
rename(
    self,
    index: Renamer | Hashable | None = None,
    *,
    axis: Axis | None = None,
    copy: bool | lib.NoDefault = <no_default>,

```

```

    inplace: bool = False,
    level: Level | None = None,
    errors: IgnoreRaise = 'ignore'
) -> Series | None from pandas.core.series.Series
    Alter Series index labels or name.

Function / dict values must be unique (1-to-1). Labels not contained in
a dict / Series will be left as-is. Extra labels listed don't throw an
error.

Alternatively, change ``Series.name`` with a scalar value.

See the :ref:`user guide <basics.rename>` for more.

Parameters
-----
index : scalar, hashable sequence, dict-like or function optional
    Functions or dict-like are transformations to apply to
    the index.
    Scalar or hashable sequence-like will alter the ``Series.name``
    attribute.
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

        This keyword is ignored and will be removed in pandas 4.0. Since
        pandas 3.0, this method always returns a new object using a lazy
        copy mechanism that defers copies until necessary
        (Copy-on-Write). See the `user guide on Copy-on-Write
        <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
        for more details.

inplace : bool, default False
    Whether to return a new Series. If True the value of copy is ignored.
level : int or level name, default None
    In case of MultiIndex, only rename labels in the specified level.
errors : {'ignore', 'raise'}, default 'ignore'
    If 'raise', raise `KeyError` when a `dict-like mapper` or
    `index` contains labels that are not present in the index being transformed.
    If 'ignore', existing keys will be renamed and extra keys will be ignored.

Returns
-----
Series
    A shallow copy with index labels or name altered, or the same object
    if ``inplace=True`` and index is not a dict or callable else None.

See Also
-----
DataFrame.rename : Corresponding DataFrame method.
Series.rename_axis : Set the name of the axis.

Examples
-----
>>> s = pd.Series([1, 2, 3])
>>> s
0    1
1    2
2    3
dtype: int64
>>> s.rename("my_name") # scalar, changes Series.name
0    1
1    2
2    3
Name: my_name, dtype: int64
>>> s.rename(lambda x: x**2) # function, changes labels
0    1
1    2
4    3
dtype: int64
>>> s.rename({1: 3, 2: 5}) # mapping, changes labels
0    1
3    2

```

```

5      3
dtype: int64

rename_axis(
    self,
    mapper: IndexLabel | lib.NoDefault = <no_default>,
    *,
    index=<no_default>,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>,
    inplace: bool = False
) -> Self | None from pandas.core.series.Series
    Set the name of the axis for the index.

Parameters
-----
mapper : scalar, list-like, optional
    Value to set the axis name attribute.

    Use either ``mapper`` and ``axis`` to
    specify the axis to target with ``mapper``, or ``index``.

index : scalar, list-like, dict-like or function, optional
    A scalar, list-like, dict-like or functions transformations to
    apply to that axis' values.
axis : {0 or 'index'}, default 0
    The axis to rename. For `Series` this parameter is unused and defaults to 0.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
    for more details.

inplace : bool, default False
    Modifies the object directly, instead of creating a new Series
    or DataFrame.

Returns
-----
Series, or None
    The same type as the caller or None if ``inplace=True``.

See Also
-----
Series.rename : Alter Series index labels or name.
DataFrame.rename : Alter DataFrame index labels or name.
Index.rename : Set new names on index.

Examples
-----
>>> s = pd.Series(["dog", "cat", "monkey"])
>>> s
0      dog
1      cat
2    monkey
dtype: str
>>> s.rename_axis("animal")
animal
0      dog
1      cat
2    monkey
dtype: str

reorder_levels(self, order: Sequence[Level]) -> Series from pandas.core.series.Series
    Rearrange index levels using input order.

    May not drop or duplicate levels.

Parameters

```

order : list of int representing new level order
Reference level by number or key.

Returns

Series

Type of caller with index as MultiIndex (new object).

See Also

DataFrame.reorder_levels : Rearrange index or column levels using
input ``order``.

Examples

>>> arrays = [
... np.array(["dog", "dog", "cat", "cat", "bird", "bird"]),
... np.array(["white", "black", "white", "black", "white", "black"]),
...]
>>> s = pd.Series([1, 2, 3, 3, 5, 2], index=arrays)
>>> s
dog white 1
 black 2
cat white 3
 black 3
bird white 5
 black 2
dtype: int64
>>> s.reorder_levels([1, 0])
white dog 1
black dog 2
white cat 3
black cat 3
white bird 5
black bird 2
dtype: int64

repeat(self, repeats: int | Sequence[int], axis: None = None) -> Series from pandas.core.series
Repeat elements of a Series.

Returns a new Series where each element of the current Series
is repeated consecutively a given number of times.

Parameters

repeats : int or array of ints

The number of repetitions for each element. This should be a
non-negative integer. Repeating 0 times will return an empty
Series.

axis : None

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

Newly created Series with repeated elements.

See Also

Index.repeat : Equivalent function for Index.
numpy.repeat : Similar method for :class:`numpy.ndarray`.

Examples

>>> s = pd.Series(["a", "b", "c"])
>>> s
0 a
1 b
2 c
dtype: str
>>> s.repeat(2)
0 a
0 a
1 b
1 b
2 c


```

2    c
dtype: str
>>> s.repeat([1, 2, 3])
0    a
1    b
1    b
2    c
2    c
2    c
dtype: str

```

```

reset_index(
    self,
    level: IndexLabel | None = None,
    *,
    drop: bool = False,
    name: Level = <no_default>,
    inplace: bool = False,
    allow_duplicates: bool = False
) -> DataFrame | Series | None from pandas.core.series.Series
Generate a new DataFrame or Series with the index reset.

```

This is useful when the index needs to be treated as a column, or when the index is meaningless and needs to be reset to the default before another operation.

Parameters

```

-----
level : int, str, tuple, or list, default optional
    For a Series with a MultiIndex, only remove the specified levels
    from the index. Removes all levels by default.
drop : bool, default False
    Just reset the index, without inserting it as a column in
    the new DataFrame.
name : object, optional
    The name to use for the column containing the original Series
    values. Uses ``self.name`` by default. This argument is ignored
    when `drop` is True.
inplace : bool, default False
    Modify the Series in place (do not create a new object).
allow_duplicates : bool, default False
    Allow duplicate column labels to be created.

```

Returns

```

-----
Series or DataFrame or None
    When `drop` is False (the default), a DataFrame is returned.
    The newly created columns will come first in the DataFrame,
    followed by the original Series values.
    When `drop` is True, a `Series` is returned.
    In either case, if ``inplace=True``, no value is returned.

```

See Also

```

-----
DataFrame.reset_index: Analogous function for DataFrame.

```

Examples

```

>>> s = pd.Series(
...     [1, 2, 3, 4],
...     name="foo",
...     index=pd.Index(["a", "b", "c", "d"], name="idx"),
... )

```

Generate a DataFrame with default index.

```

>>> s.reset_index()
   idx  foo
0    a    1
1    b    2
2    c    3
3    d    4

```

To specify the name of the new column use `name`.

```

>>> s.reset_index(name="values")
   idx  values
0    a       1
1    b       2
2    c       3
3    d       4

```

```

0    a    1
1    b    2
2    c    3
3    d    4

```

To generate a new Series with the default set `drop` to True.

```

>>> s.reset_index(drop=True)
0    1
1    2
2    3
3    4
Name: foo, dtype: int64

```

The `level` parameter is interesting for Series with a multi-level index.

```

>>> arrays = [
...     np.array(["bar", "bar", "baz", "baz"]),
...     np.array(["one", "two", "one", "two"]),
... ]
>>> s2 = pd.Series(
...     range(4),
...     name="foo",
...     index=pd.MultiIndex.from_arrays(arrays, names=["a", "b"]),
... )

```

To remove a specific level from the Index, use `level`.

```

>>> s2.reset_index(level="a")
      a  foo
b
one  bar    0
two  bar    1
one  baz    2
two  baz    3

```

If `level` is not set, all levels are removed from the Index.

```

>>> s2.reset_index()
      a  b  foo
0  bar  one    0
1  bar  two    1
2  baz  one    2
3  baz  two    3

```

`rfloordiv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.com`
Return Integer division of series and other, element-wise (binary operator `rfloordiv`).

Equivalent to ``other // series``, but with support to substitute a fill_value for missing data in either one of the inputs.

Parameters

other : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

level : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

axis : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.floordiv` : Element-wise Integer division, see

``Python documentation
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`_`
for more details.

Examples

```
-----  
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])  
>>> a  
a    1.0  
b    1.0  
c    1.0  
d    NaN  
dtype: float64  
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])  
>>> b  
a    1.0  
b    NaN  
d    1.0  
e    NaN  
dtype: float64  
>>> a.floordiv(b, fill_value=0)  
a    1.0  
b    inf  
c    inf  
d    0.0  
e    NaN  
dtype: float64
```

`rmod(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Modulo of series and other, element-wise (binary operator ``rmod``).

Equivalent to ``other % series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

```
-----  
other : object  
    When a Series is provided, will align on indexes. For all other types,  
    will behave the same as `==` but with possibly different results due  
    to the other arguments.  
level : int or name  
    Broadcast across a level, matching Index values on the  
    passed MultiIndex level.  
fill_value : None or float value, default None (NaN)  
    Fill existing missing (NaN) values, and any new element needed for  
    successful Series alignment, with this value before computation.  
    If data in both corresponding Series locations is missing  
    the result of filling (at that location) will be missing.  
axis : {0 or 'index'}  
    Unused. Parameter needed for compatibility with DataFrame.
```

Returns

```
-----  
Series  
    The result of the operation.
```

See Also

```
-----  
Series.mod : Element-wise Modulo, see  
    `Python documentation  
    <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`_  
    for more details.
```

Examples

```
-----  
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])  
>>> a  
a    1.0  
b    1.0  
c    1.0  
d    NaN  
dtype: float64  
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])  
>>> b  
a    1.0  
b    NaN  
d    1.0
```

```

e    NaN
dtype: float64
>>> a.mod(b, fill_value=0)
a    0.0
b    NaN
c    NaN
d    0.0
e    NaN
dtype: float64

```

`rmul(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
 Return Multiplication of series and other, element-wise (binary operator ``rmul``).

Equivalent to ``other * series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation.

If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.mul` : Element-wise Multiplication, see

``Python documentation`

`<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>` for more details.

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.multiply(b, fill_value=0)
a    1.0
b    0.0
c    0.0
d    0.0
e    NaN
dtype: float64

```

`round(self, decimals: int = 0, *args, **kwargs) -> Series from pandas.core.series.Series`
 Round each value in a Series to the given number of decimals.

Parameters

`decimals` : int, default 0

Number of decimal places to round to. If decimals is negative, it specifies the number of positions to the left of the decimal point.

`*args, **kwargs`
Additional arguments and keywords have no effect but might be accepted for compatibility with NumPy.

Returns

Series

Rounded values of the Series.

See Also

`numpy.around` : Round values of an `np.array`.

`DataFrame.round` : Round values of a `DataFrame`.

`Series.dt.round` : Round values of data to the specified freq.

Notes

For values exactly halfway between rounded decimal values, pandas rounds to the nearest even value (e.g. -0.5 and 0.5 round to 0.0, 1.5 and 2.5 round to 2.0, etc.).

Examples

```
>>> s = pd.Series([-0.5, 0.1, 2.5, 1.3, 2.7])
```

```
>>> s.round()
```

```
0    -0.0
```

```
1     0.0
```

```
2     2.0
```

```
3     1.0
```

```
4     3.0
```

```
dtype: float64
```

`rpow(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Exponential power of series and other, element-wise (binary operator ``rpow``).

Equivalent to ``other ** series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.pow` : Element-wise Exponential power, see

``Python documentation`

`<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>`` for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
```

```
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.pow(b, fill_value=0)
a    1.0
b    1.0
c    1.0
d    0.0
e    NaN
dtype: float64
```

`rsub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.series`
Return Subtraction of series and other, element-wise (binary operator ``rsub``).

Equivalent to ``other - series``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object

When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name

Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)

Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.sub` : Element-wise Subtraction, see

``Python documentation`

`<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>` for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
```

```
>>> a
```

```
a    1.0
```

```
b    1.0
```

```
c    1.0
```

```
d    NaN
```

```
dtype: float64
```

```
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
```

```
>>> b
```

```
a    1.0
```

```
b    NaN
```

```
d    1.0
```

```
e    NaN
```

```
dtype: float64
```

```
>>> a.subtract(b, fill_value=0)
```

```
a    0.0
```

```
b    1.0
```

```
c    1.0
```

```
d   -1.0
```

```
e    NaN
```

```
dtype: float64
```

`rtruediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core`
Return Floating division of series and other, element-wise (binary operator ``rtruediv``).

Equivalent to ``other / series``, but with support to substitute a `fill_value` for

missing data in either one of the inputs.

Parameters

other : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.
level : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.
fill_value : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
axis : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series
The result of the operation.

See Also

Series.truediv : Element-wise Floating division, see
Python documentation
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`\_`
for more details.

Examples

>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a 1.0
b 1.0
c 1.0
d NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a 1.0
b NaN
d 1.0
e NaN
dtype: float64
>>> a.divide(b, fill_value=0)
a 1.0
b inf
c inf
d 0.0
e NaN
dtype: float64

```
searchsorted(  
    self,  
    value: NumpyValueArrayLike | ExtensionArray,  
    side: Literal['left', 'right'] = 'left',  
    sorter: NumpySorter | None = None  
) -> npt.NDArray[np.intp] | np.intp from pandas.core.series.Series  
Find indices where elements should be inserted to maintain order.
```

Find the indices into a sorted Series `self` such that, if the corresponding elements in `value` were inserted before the indices, the order of `self` would be preserved.

.. note::
The Series *must* be monotonically sorted, otherwise wrong locations will likely be returned. Pandas does *not* check this for you.

Parameters

value : array-like or scalar
Values to insert into `self`.
side : {'left', 'right'}, optional

If 'left', the index of the first suitable location found is given.
 If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of `self`).
 sorter : 1-D array-like, optional
 Optional array of integer indices that sort `self` into ascending order. They are typically the result of ``np.argsort``.

Returns

int or array of int

A scalar or array of insertion points with the same shape as `value`.

See Also

sort_values : Sort by the values along either axis.

numpy.searchsorted : Similar method from NumPy.

Notes

Binary search is used to find the required insertion points.

Examples

```
>>> ser = pd.Series([1, 2, 3])
```

```
>>> ser
```

```
0    1
```

```
1    2
```

```
2    3
```

```
dtype: int64
```

```
>>> ser.searchsorted(4)
```

```
np.int64(3)
```

```
>>> ser.searchsorted([0, 4])
```

```
array([0, 3])
```

```
>>> ser.searchsorted([1, 3], side="left")
```

```
array([0, 2])
```

```
>>> ser.searchsorted([1, 3], side="right")
```

```
array([1, 3])
```

```
>>> ser = pd.Series(pd.to_datetime(["3/11/2000", "3/12/2000", "3/13/2000"]))
```

```
>>> ser
```

```
0    2000-03-11
```

```
1    2000-03-12
```

```
2    2000-03-13
```

```
dtype: datetime64[us]
```

```
>>> ser.searchsorted("3/14/2000")
```

```
np.int64(3)
```

```
>>> ser = pd.Categorical(
```

```
...     ["apple", "bread", "bread", "cheese", "milk"], ordered=True
```

```
... )
```

```
>>> ser
```

```
['apple', 'bread', 'bread', 'cheese', 'milk']
```

```
Categories (4, str): ['apple' < 'bread' < 'cheese' < 'milk']
```

```
>>> ser.searchsorted("bread")
```

```
np.int64(1)
```

```
>>> ser.searchsorted(["bread"], side="right")
```

```
array([3])
```

If the values are not monotonically sorted, wrong locations may be returned:

```
>>> ser = pd.Series([2, 1, 3])
```

```
>>> ser
```

```
0    2
```

```
1    1
```

```
2    3
```

```
dtype: int64
```

```
>>> ser.searchsorted(1) # doctest: +SKIP
```

```
0 # wrong result, correct would be 1
```

sem(

self,

*,

axis: Axis | None = None,

skipna: bool = True,

ddof: int = 1,

numeric_only: bool = False,

**kwargs


```

) from pandas.core.series.Series
    Return unbiased standard error of the mean over requested axis.

    Normalized by N-1 by default. This can be changed using the ddof argument

    Parameters
    -----
    axis : {index (0)}
        This parameter is unused and defaults to 0.
    skipna : bool, default True
        Exclude NA/null values. If an entire row/column is NA, the result
        will be NA.
    ddof : int, default 1
        Delta Degrees of Freedom. The divisor used in calculations is N - ddof,
        where N represents the number of elements.
    numeric_only : bool, default False
        Include only float, int, boolean columns. Not implemented for Series.
    **kwargs :
        Additional keywords have no effect but might be accepted
        for compatibility with NumPy.

    Returns
    -----
    scalar or Series (if level specified)
        Unbiased standard error of the mean over requested axis.

    See Also
    -----
    scipy.stats.sem : Compute standard error of the mean.
    Series.std : Return sample standard deviation over requested axis.
    Series.var : Return unbiased variance over requested axis.
    Series.mean : Return the mean of the values over the requested axis.
    Series.median : Return the median of the values over the requested axis.
    Series.mode : Return the mode(s) of the Series.

    Examples
    -----
    >>> s = pd.Series([1, 2, 3])
    >>> round(s.sem(), 6)
    0.57735

set_axis(
    self,
    labels,
    *,
    axis: Axis = 0,
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
    Assign desired index to given axis.

.. deprecated:: 3.0.0
    This keyword is ignored and will be removed in pandas 4.0. Since
    pandas 3.0, this method always returns a new object using a lazy
    copy mechanism that defers copies until necessary
    (Copy-on-Write). See the `user guide on Copy-on-Write
    <https://pandas.pydata.org/docs/dev/user\_guide/copy\_on\_write.html>`__
    for more details.

    Indexes for row labels can be changed by assigning a list-like or Index.

    Parameters
    -----
    labels : list-like or Index
        The values for the new index.
    axis : {0 or 'index'}, default 0
        The axis to update. The value 0 identifies the rows. For `Series`
        this parameter is unused and defaults to 0.
    copy : bool, default False
        This keyword is now ignored; changing its value will have no
        impact on the method.

    Returns
    -----
    Series
        A shallow copy of the object with axis altered to the given index.

    See Also

```

Series.rename_axis : Alter the name of the index.

Examples

>>> s = pd.Series([1, 2, 3])
>>> s
0 1
1 2
2 3
dtype: int64
>>> s.set_axis(["a", "b", "c"], axis=0)
a 1
b 2
c 3
dtype: int64

```
skew(  
    self,  
    *,  
    axis: Axis | None = 0,  
    skipna: bool = True,  
    numeric_only: bool = False,  
    **kwargs  
) from pandas.core.series.Series  
Return unbiased skew over requested axis.
```

Normalized by N-1.

Parameters

axis : {index (0)}
This parameter is unused and defaults to 0.
skipna : bool, default True
Exclude NA/null values when computing the result.
numeric_only : bool, default False
Unused.
**kwargs
Additional keyword arguments to be passed to the function.

Returns

scalar
Unbiased skew of the Series.

See Also

Series.var : Return unbiased variance over requested axis.
Series.std : Return unbiased standard deviation over requested axis.

Examples

>>> s = pd.Series([1, 2, 3])
>>> s.skew()
0.0

```
sort_index(  
    self,  
    *,  
    axis: Axis = 0,  
    level: IndexLabel | None = None,  
    ascending: bool | Sequence[bool] = True,  
    inplace: bool = False,  
    kind: SortKind = 'quicksort',  
    na_position: NaPosition = 'last',  
    sort_remaining: bool = True,  
    ignore_index: bool = False,  
    key: IndexKeyFunc | None = None  
) -> Series | None from pandas.core.series.Series  
Sort Series by index labels.
```

Returns a new Series sorted by label if `inplace` argument is
`False`, otherwise updates the original series and returns None.

Parameters

axis : {0 or 'index'}
 Unused. Parameter needed for compatibility with DataFrame.
 level : int, optional
 If not None, sort on values in specified index level(s).
 ascending : bool or list-like of bools, default True
 Sort ascending vs. descending. When the index is a MultiIndex the
 sort direction can be controlled for each level individually.
 inplace : bool, default False
 If True, perform operation in-place.
 kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
 Choice of sorting algorithm. See also :func:`numpy.sort` for more
 information. 'mergesort' and 'stable' are the only stable algorithms. For
 DataFrames, this option is only applied when sorting on a single
 column or label.
 na_position : {'first', 'last'}, default 'last'
 If 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.
 Not implemented for MultiIndex.
 sort_remaining : bool, default True
 If True and sorting by level and index is multilevel, sort by other
 levels too (in order) after sorting by specified level.
 ignore_index : bool, default False
 If True, the resulting axis will be labeled 0, 1, ..., n - 1.
 key : callable, optional
 If not None, apply the key function to the index values
 before sorting. This is similar to the `key` argument in the
 builtin :meth:`sorted` function, with the notable difference that
 this `key` function should be *vectorized*. It should expect an
 ``Index`` and return an ``Index`` of the same shape.

Returns

Series or None

The original Series sorted by the labels or None if ``inplace=True``.

See Also

DataFrame.sort_index: Sort DataFrame by the index.
 DataFrame.sort_values: Sort DataFrame by the value.
 Series.sort_values : Sort Series by the value.

Examples

```
>>> s = pd.Series(["a", "b", "c", "d"], index=[3, 2, 1, 4])
>>> s.sort_index()
1    c
2    b
3    a
4    d
dtype: str
```

Sort Descending

```
>>> s.sort_index(ascending=False)
4    d
3    a
2    b
1    c
dtype: str
```

By default NaNs are put at the end, but use `na\_position` to place them at the beginning

```
>>> s = pd.Series(["a", "b", "c", "d"], index=[3, 2, 1, np.nan])
>>> s.sort_index(na_position="first")
NaN    d
1.0    c
2.0    b
3.0    a
dtype: str
```

Specify index level to sort

```
>>> arrays = [
...     np.array(["qux", "qux", "foo", "foo", "baz", "baz", "bar", "bar"]),
...     np.array(["two", "one", "two", "one", "two", "one", "two", "one"]),
... ]
>>> s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=arrays)
```

```
>>> s.sort_index(level=1)
bar one      8
baz one      6
foo one      4
qux one      2
bar two      7
baz two      5
foo two      3
qux two      1
dtype: int64
```

Does not sort by remaining levels when sorting by levels

```
>>> s.sort_index(level=1, sort_remaining=False)
qux one      2
foo one      4
baz one      6
bar one      8
qux two      1
foo two      3
baz two      5
bar two      7
dtype: int64
```

Apply a key function before sorting

```
>>> s = pd.Series([1, 2, 3, 4], index=["A", "b", "C", "d"])
>>> s.sort_index(key=lambda x: x.str.lower())
A      1
b      2
C      3
d      4
dtype: int64
```

```
sort_values(
    self,
    *,
    axis: Axis = 0,
    ascending: bool | Sequence[bool] = True,
    inplace: bool = False,
    kind: SortKind = 'quicksort',
    na_position: NaPosition = 'last',
    ignore_index: bool = False,
    key: ValueKeyFunc | None = None
) -> Series | None from pandas.core.series.Series
Sort by the values.
```

Sort a Series in ascending or descending order by some criterion.

Parameters

```
-----
axis : {0 or 'index'}
    Unused. Parameter needed for compatibility with DataFrame.
ascending : bool or list of bools, default True
    If True, sort values in ascending order, otherwise descending.
inplace : bool, default False
    If True, perform operation in-place.
kind : {'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'
    Choice of sorting algorithm. See also :func:`numpy.sort` for more
    information. 'mergesort' and 'stable' are the only stable algorithms.
na_position : {'first' or 'last'}, default 'last'
    Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at
    the end.
ignore_index : bool, default False
    If True, the resulting axis will be labeled 0, 1, ..., n - 1.
key : callable, optional
    If not None, apply the key function to the series values
    before sorting. This is similar to the `key` argument in the
    builtin :meth:`sorted` function, with the notable difference that
    this `key` function should be *vectorized*. It should expect a
    ``Series`` and return an array-like.
```

Returns

```
-----
Series or None
    Series ordered by values or None if ``inplace=True``.
```

See Also

Series.sort_index : Sort by the Series indices.
DataFrame.sort_values : Sort DataFrame by the values along either axis.
DataFrame.sort_index : Sort DataFrame by indices.

Examples

>>> s = pd.Series([np.nan, 1, 3, 10, 5])
>>> s
0 NaN
1 1.0
2 3.0
3 10.0
4 5.0
dtype: float64

Sort values ascending order (default behavior)

>>> s.sort_values(ascending=True)
1 1.0
2 3.0
4 5.0
3 10.0
0 NaN
dtype: float64

Sort values descending order

>>> s.sort_values(ascending=False)
3 10.0
4 5.0
2 3.0
1 1.0
0 NaN
dtype: float64

Sort values putting NAs first

>>> s.sort_values(na_position="first")
0 NaN
1 1.0
2 3.0
4 5.0
3 10.0
dtype: float64

Sort a series of strings

>>> s = pd.Series(["z", "b", "d", "a", "c"])
>>> s
0 z
1 b
2 d
3 a
4 c
dtype: str

>>> s.sort_values()
3 a
1 b
4 c
2 d
0 z
dtype: str

Sort using a key function. Your `key` function will be given the `Series` of values and should return an array-like.

>>> s = pd.Series(["a", "B", "c", "D", "e"])
>>> s.sort_values()
1 B
3 D
0 a
2 c
4 e

```
dtype: str
>>> s.sort_values(key=lambda x: x.str.lower())
0    a
1    B
2    c
3    D
4    e
dtype: str
```

NumPy ufuncs work well here. For example, we can sort by the ``sin`` of the value

```
>>> s = pd.Series([-4, -2, 0, 2, 4])
>>> s.sort_values(key=np.sin)
1    -2
4     4
2     0
0    -4
3     2
dtype: int64
```

More complicated user-defined functions can be used, as long as they expect a Series and return an array-like

```
>>> s.sort_values(key=lambda x: (np.tan(x.cumsum()))))
0    -4
3     2
4     4
1    -2
2     0
dtype: int64
```

```
std(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return sample standard deviation.
```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

```
-----
axis : {index (0)}
    This parameter is unused and defaults to 0.
skipna : bool, default True
    Exclude NA/null values. If Series is NA, the result
    will be NA.
ddof : int, default 1
    Delta Degrees of Freedom. The divisor used in calculations is N - ddof,
    where N represents the number of elements.
numeric_only : bool, default False
    Not implemented for Series.
**kwargs :
    Additional keywords have no effect but might be accepted
    for compatibility with NumPy.
```

Returns

```
-----
scalar
    Standard deviation over all values in the Series.
```

See Also

```
-----
numpy.std : Compute the standard deviation along the specified axis.
Series.var : Return unbiased variance over requested axis.
Series.sem : Return unbiased standard error of the mean over requested axis.
Series.mean : Return the mean of the values over the requested axis.
Series.median : Return the median of the values over the requested axis.
Series.mode : Return the mode(s) of the Series.
```

Examples

```
-----
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.std()
1.0
```

Alternatively, ``ddof=0`` can be set to normalize by N instead of $N-1$:

```
>>> s.std(ddof=0)
0.816496580927726
```

`sub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core.ser`
Return Subtraction of series and other, element-wise (binary operator ``sub``).

Equivalent to ``series - other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : object
When a Series is provided, will align on indexes. For all other types, will behave the same as ``==`` but with possibly different results due to the other arguments.

`level` : int or name
Broadcast across a level, matching Index values on the passed MultiIndex level.

`fill_value` : None or float value, default None (NaN)
Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.

`axis` : {0 or 'index'}
Unused. Parameter needed for compatibility with DataFrame.

Returns

Series

The result of the operation.

See Also

`Series.rsub` : Reverse of the Subtraction operator, see [Python documentation](https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types)
<<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>>`_ for more details.

Examples

```
>>> a = pd.Series([1, 1, 1, np.nan], index=['a', 'b', 'c', 'd'])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=['a', 'b', 'd', 'e'])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.subtract(b, fill_value=0)
a    0.0
b    1.0
c    1.0
d   -1.0
e    NaN
dtype: float64
```

`subtract = sub(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series`

```
sum(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    numeric_only: bool = False,
    min_count: int = 0,
```

```

    **kwargs
) from pandas.core.series.Series
    Return the sum of the values over the requested axis.

    This is equivalent to the method ``numpy.sum``.

Parameters
-----
axis : {index (0)}
    Axis for the function to be applied on.
    For ``Series`` this parameter is unused and defaults to 0.

    .. warning::

        The behavior of DataFrame.sum with ``axis=None`` is deprecated,
        in a future version this will reduce over both axes and return a scalar
        To retain the old behavior, pass axis=0 (or do not pass axis).

    .. versionadded:: 2.0.0

skipna : bool, default True
    Exclude NA/null values when computing the result.
numeric_only : bool, default False
    Include only float, int, boolean columns. Not implemented for Series.

min_count : int, default 0
    The required number of valid values to perform the operation. If fewer than
    ``min_count`` non-NA values are present the result will be NA.
**kwargs
    Additional keyword arguments to be passed to the function.

Returns
-----
scalar or Series (if level specified)
    Sum of the values for the requested axis.

See Also
-----
numpy.sum : Equivalent numpy function for computing sum.
Series.mean : Mean of the values.
Series.median : Median of the values.
Series.std : Standard deviation of the values.
Series.var : Variance of the values.
Series.min : Minimum value.
Series.max : Maximum value.

Examples
-----
>>> idx = pd.MultiIndex.from_arrays(
...     [ ["warm", "warm", "cold", "cold"], ["dog", "falcon", "fish", "spider"] ],
...     names=["blooded", "animal"],
... )
>>> s = pd.Series([4, 2, 0, 8], name="legs", index=idx)
>>> s
blooded animal
warm      dog      4
          falcon   2
cold     fish      0
          spider   8
Name: legs, dtype: int64

>>> s.sum()
14

By default, the sum of an empty or all-NA Series is ``0``.

>>> pd.Series([], dtype="float64").sum() # min_count=0 is the default
0.0

This can be controlled with the ``min_count`` parameter. For example, if
you'd like the sum of an empty series to be NaN, pass ``min_count=1``.

>>> pd.Series([], dtype="float64").sum(min_count=1)
nan

Thanks to the ``skipna`` parameter, ``min_count`` handles all-NA and
empty series identically.

```



```

>>> pd.Series([np.nan]).sum()
0.0

>>> pd.Series([np.nan]).sum(min_count=1)
nan

```

swaplevel(
 self,
 i: Level = -2,
 j: Level = -1,
 copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
Swap levels i and j in a :class:`MultiIndex`.

Default is to swap the two innermost levels of the index.

Parameters

i, j : int or str
 Levels of the indices to be swapped. Can pass level name as string.
copy : bool, default False
 This keyword is now ignored; changing its value will have no
 impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since
pandas 3.0, this method always returns a new object using a lazy
copy mechanism that defers copies until necessary
(Copy-on-Write). See the `user guide on Copy-on-Write`
https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html for
more details.

Returns

Series
Series with levels swapped in MultiIndex.

See Also

DataFrame.swaplevel : Swap levels i and j in a :class:`DataFrame`.
Series.reorder_levels : Rearrange index levels using input order.
MultiIndex.swaplevel : Swap levels i and j in a :class:`MultiIndex`.

Examples

```

>>> s = pd.Series(
...     ["A", "B", "A", "C"],
...     index=[
...         ["Final exam", "Final exam", "Coursework", "Coursework"],
...         ["History", "Geography", "History", "Geography"],
...         ["January", "February", "March", "April"],
...     ],
... )
>>> s

```

Final exam	History	January	A
	Geography	February	B
Coursework	History	March	A
	Geography	April	C

```

dtype: str

```

In the following example, we will swap the levels of the indices.
Here, we will swap the levels column-wise, but levels can be swapped row-wise
in a similar manner. Note that column-wise is the default behavior.
By not supplying any arguments for i and j, we swap the last and second to
last indices.

```

>>> s.swaplevel()

```

Final exam	January	History	A
	February	Geography	B
Coursework	March	History	A
	April	Geography	C

```

dtype: str

```

By supplying one argument, we can choose which index to swap the last
index with. We can for example swap the first index with the last one as

follows.

```
>>> s.swaplevel(0)
January      History      Final exam      A
February     Geography    Final exam      B
March         History      Coursework      A
April         Geography    Coursework      C
dtype: str
```

We can also define explicitly which indices we want to swap by supplying values for both i and j. Here, we for example swap the first and second indices.

```
>>> s.swaplevel(0, 1)
History      Final exam      January      A
Geography     Final exam     February     B
History       Coursework      March        A
Geography     Coursework      April        C
dtype: str
```

`to_dict(self, *, into: type[MutableMappingT] | MutableMappingT = <class 'dict'>) -> MutableMappingT`
Convert Series to {label -> value} dict or dict-like object.

Parameters

`into` : class, default dict
The `collections.abc.MutableMapping` subclass to use as the return object. Can be the actual class or an empty instance of the mapping type you want. If you want a `collections.defaultdict`, you must pass it initialized.

Returns

`collections.abc.MutableMapping`
Key-value representation of Series.

See Also

`Series.to_list`: Converts Series to a list of the values.
`Series.to_numpy`: Converts Series to NumPy ndarray.
`Series.array`: ExtensionArray of the data backing this Series.

Examples

```
>>> s = pd.Series([1, 2, 3, 4])
>>> s.to_dict()
{0: 1, 1: 2, 2: 3, 3: 4}
>>> from collections import OrderedDict, defaultdict
>>> s.to_dict(into=OrderedDict)
OrderedDict([(0, 1), (1, 2), (2, 3), (3, 4)])
>>> dd = defaultdict(list)
>>> s.to_dict(into=dd)
defaultdict(<class 'list'>, {0: 1, 1: 2, 2: 3, 3: 4})
```

`to_frame(self, name: Hashable = <no_default>) -> DataFrame` from `pandas.core.series.Series`
Convert Series to DataFrame.

Parameters

`name` : object, optional
The passed name should substitute for the series name (if it has one).

Returns

`DataFrame`
DataFrame representation of Series.

See Also

`Series.to_dict` : Convert Series to dict object.

Examples

```
>>> s = pd.Series(["a", "b", "c"], name="vals")
>>> s.to_frame()
   vals
0     a
```

```

1      b
2      c

to_markdown(
    self,
    buf: IO[str] | None = None,
    *,
    mode: str = 'wt',
    index: bool = True,
    storage_options: StorageOptions | None = None,
    **kwargs
) -> str | None from pandas.core.series.Series
    Print Series in Markdown-friendly format.

Parameters
-----
buf : str, Path or StringIO-like, optional, default None
    Buffer to write to. If None, the output is returned as a string.
mode : str, optional
    Mode in which file is opened, "wt" by default.
index : bool, optional, default True
    Add index (row) labels.

storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`_.

**kwargs
    These parameters will be passed to `tabulate` <https://pypi.org/project/

Returns
-----
str
    Series in Markdown-friendly format.

See Also
-----
Series.to_frame : Write a text representation of object to the system clipboard.
Series.to_latex : Render Series to LaTeX-formatted table.

Notes
-----
Requires the `tabulate` <https://pypi.org/project/tabulate>`_ package.

Examples
-----
>>> s = pd.Series(["elk", "pig", "dog", "quetzal"], name="animal")
>>> print(s.to_markdown())
|   | animal |
|---:|:-----|
| 0 | elk      |
| 1 | pig      |
| 2 | dog      |
| 3 | quetzal  |

Output markdown with a tabulate option.

>>> print(s.to_markdown(tablefmt="grid"))
+-----+
|   | animal |
+=====+
| 0 | elk      |
+-----+
| 1 | pig      |
+-----+
| 2 | dog      |
+-----+
| 3 | quetzal  |
+-----+

to_period(

```

```

self,
    freq: str | None = None,
    copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
    Convert Series from DatetimeIndex to PeriodIndex.

Parameters
-----
freq : str, default None
    Frequency associated with the PeriodIndex.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

    .. deprecated:: 3.0.0

        This keyword is ignored and will be removed in pandas 4.0. Since
        pandas 3.0, this method always returns a new object using a lazy
        copy mechanism that defers copies until necessary
        (Copy-on-Write). See the `user guide on Copy-on-Write
        <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
        for more details.

Returns
-----
Series
    Series with index converted to PeriodIndex.

See Also
-----
DataFrame.to_period: Equivalent method for DataFrame.
Series.dt.to_period: Convert DateTime column values.

Examples
-----
>>> idx = pd.DatetimeIndex(["2023", "2024", "2025"])
>>> s = pd.Series([1, 2, 3], index=idx)
>>> s = s.to_period()
>>> s
2023    1
2024    2
2025    3
Freq: Y-DEC, dtype: int64

Viewing the index

>>> s.index
PeriodIndex(['2023', '2024', '2025'], dtype='period[Y-DEC]')

to_string(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    na_rep: str = 'NaN',
    float_format: str | None = None,
    header: bool = True,
    index: bool = True,
    length: bool = False,
    dtype: bool = False,
    name: bool = False,
    max_rows: int | None = None,
    min_rows: int | None = None
) -> str | None from pandas.core.series.Series
    Render a string representation of the Series.

Parameters
-----
buf : StringIO-like, optional
    Buffer to write to.
na_rep : str, optional
    String representation of NaN to use, default 'NaN'.
float_format : one-parameter function, optional
    Formatter function to apply to columns' elements if they are
    floats, default None.
header : bool, default True
    Add the Series header (index name).
index : bool, optional

```

Add index (row) labels, default True.
length : bool, default False
Add the Series length.
dtype : bool, default False
Add the Series dtype.
name : bool, default False
Add the Series name if not None.
max_rows : int, optional
Maximum number of rows to show before truncating. If None, show all.
min_rows : int, optional
The number of rows to display in a truncated repr (when number of rows is above `max\_rows`).

Returns

str or None
String representation of Series if ``buf=None``, otherwise None.

See Also

Series.to_dict : Convert Series to dict object.
Series.to_frame : Convert Series to DataFrame object.
Series.to_markdown : Print Series in Markdown-friendly format.
Series.to_timestamp : Cast to DatetimeIndex of Timestamps.

Examples

>>> ser = pd.Series([1, 2, 3]).to_string()
>>> ser
'0 1\n1 2\n2 3'

to_timestamp()

self,
freq: Frequency | None = None,
how: Literal['s', 'e', 'start', 'end'] = 'start',
copy: bool | lib.NoDefault = <no_default>
) -> Series from pandas.core.series.Series
Cast to DatetimeIndex of Timestamps, at *beginning* of period.

This can be changed to the *end* of the period, by specifying `how="e"`.

Parameters

freq : str, default frequency of PeriodIndex
Desired frequency.
how : {'s', 'e', 'start', 'end'}
Convention for converting period to timestamp; start of period vs. end.
copy : bool, default False
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`__ for more details.

Returns

Series with DatetimeIndex
Series with the PeriodIndex cast to DatetimeIndex.

See Also

Series.to_period: Inverse method to cast DatetimeIndex to PeriodIndex.
DataFrame.to_timestamp: Equivalent method for DataFrame.

Examples

>>> idx = pd.PeriodIndex(["2023", "2024", "2025"], freq="Y")
>>> s1 = pd.Series([1, 2, 3], index=idx)
>>> s1

```

2023    1
2024    2
2025    3
Freq: Y-DEC, dtype: int64

```

The resulting frequency of the Timestamps is `YearBegin`

```

>>> s1 = s1.to_timestamp()
>>> s1
2023-01-01    1
2024-01-01    2
2025-01-01    3
Freq: YS-JAN, dtype: int64

```

Using `freq` which is the offset that the Timestamps will have

```

>>> s2 = pd.Series([1, 2, 3], index=idx)
>>> s2 = s2.to_timestamp(freq="M")
>>> s2
2023-01-31    1
2024-01-31    2
2025-01-31    3
Freq: YE-JAN, dtype: int64

```

`transform(self, func: AggFuncType, axis: Axis = 0, *args, **kwargs) -> DataFrame | Series` from
Call `func` on self producing a Series with the same axis shape as self.

Parameters

`func` : function, str, list-like or dict-like

Function to use for transforming the data. If a function, must either work when passed a Series or when passed to Series.apply. If func is both list-like and dict-like, dict-like behavior takes precedence.

Accepted combinations are:

- function
- string function name
- list-like of functions and/or function names, e.g. `[np.exp, 'sqrt']`
- dict-like of axis labels -> functions, function names or list-like of such

`axis` : {0 or 'index'}

Unused. Parameter needed for compatibility with DataFrame.

`*args`

Positional arguments to pass to `func`.

`**kwargs`

Keyword arguments to pass to `func`.

Returns

Series

A Series that must have the same length as self.

Raises

`ValueError` : If the returned Series has a different length than self.

See Also

`Series.agg` : Only perform aggregating type operations.

`Series.apply` : Invoke function on a Series.

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See :ref:`gotchas.udf-mutation` for more details.

Examples

```

>>> df = pd.DataFrame({"A": range(3), "B": range(1, 4)})
>>> df
   A  B
0  0  1
1  1  2
2  2  3

```

```
>>> df.transform(lambda x: x + 1)
A  B
0  1  2
1  2  3
2  3  4
```

Even though the resulting Series must have the same length as the input Series, it is possible to provide several input functions:

```
>>> s = pd.Series(range(3))
>>> s
0    0
1    1
2    2
dtype: int64
>>> s.transform([np.sqrt, np.exp])
      sqrt      exp
0  0.000000  1.000000
1  1.000000  2.718282
2  1.414214  7.389056
```

You can call transform on a GroupBy object:

```
>>> df = pd.DataFrame(
...     {
...         "Date": [
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...             "2015-05-08",
...             "2015-05-07",
...             "2015-05-06",
...             "2015-05-05",
...         ],
...         "Data": [5, 8, 6, 1, 50, 100, 60, 120],
...     }
... )
>>> df
   Date  Data
0 2015-05-08    5
1 2015-05-07    8
2 2015-05-06    6
3 2015-05-05    1
4 2015-05-08   50
5 2015-05-07  100
6 2015-05-06   60
7 2015-05-05  120
>>> df.groupby("Date")["Data"].transform("sum")
0    55
1   108
2    66
3   121
4    55
5   108
6    66
7   121
Name: Data, dtype: int64
```

```
>>> df = pd.DataFrame(
...     {
...         "c": [1, 1, 1, 2, 2, 2, 2],
...         "type": ["m", "n", "o", "m", "m", "n", "n"],
...     }
... )
>>> df
   c  type
0  1    m
1  1    n
2  1    o
3  2    m
4  2    m
5  2    n
6  2    n
>>> df["size"] = df.groupby("c")["type"].transform(len)
>>> df
   c  type  size
0  1    m     1
1  1    n     1
2  1    o     1
3  2    m     2
4  2    m     2
5  2    n     2
6  2    n     2
```

```

0 1 m 3
1 1 n 3
2 1 o 3
3 2 m 4
4 2 m 4
5 2 n 4
6 2 n 4

```

`truediv(self, other, level=None, fill_value=None, axis: Axis = 0) -> Series from pandas.core`
 Return Floating division of series and other, element-wise (binary operator ``truediv``)

Equivalent to ``series / other``, but with support to substitute a `fill_value` for missing data in either one of the inputs.

Parameters

`other` : Series or scalar value
 Series with which to compute division.
`level` : int or name
 Broadcast across a level, matching Index values on the passed MultiIndex level.
`fill_value` : None or float value, default None (NaN)
 Fill existing missing (NaN) values, and any new element needed for successful Series alignment, with this value before computation. If data in both corresponding Series locations is missing the result of filling (at that location) will be missing.
`axis` : {0 or 'index'}
 Unused. Parameter needed for compatibility with DataFrame.

Returns

 Series
 The result of the operation.

See Also

`Series.rtruediv` : Reverse of the Floating division operator, see ``Python documentation``
<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types> for more details.

Examples

```

>>> a = pd.Series([1, 1, 1, np.nan], index=["a", "b", "c", "d"])
>>> a
a    1.0
b    1.0
c    1.0
d    NaN
dtype: float64
>>> b = pd.Series([1, np.nan, 1, np.nan], index=["a", "b", "d", "e"])
>>> b
a    1.0
b    NaN
d    1.0
e    NaN
dtype: float64
>>> a.divide(b, fill_value=0)
a    1.0
b    inf
c    inf
d    0.0
e    NaN
dtype: float64

```

`unique(self) -> ArrayLike from pandas.core.series.Series`
 Return unique values of Series object.

Uniques are returned in order of appearance. Hash table-based unique, therefore does NOT sort.

Returns

 ndarray or ExtensionArray
 The unique values returned as a NumPy array. See Notes.

See Also

Series.drop_duplicates : Return Series with duplicate values removed.
unique : Top-level unique method for any 1-d array-like object.
Index.unique : Return Index with unique values from an Index object.

Notes

Returns the unique values as a NumPy array. In case of an extension-array backed Series, a new :class:`~api.extensions.ExtensionArray` of that type with just the unique values is returned. This includes

- * Categorical
- * Period
- * Datetime with Timezone
- * Datetime without Timezone
- * Timedelta
- * Interval
- * Sparse
- * IntegerNA

See Examples section.

Examples

>>> pd.Series([2, 1, 3, 3], name="A").unique()
array([2, 1, 3])

>>> pd.Series([pd.Timestamp("2016-01-01") for _ in range(3)]).unique()
<DatetimeArray>
['2016-01-01 00:00:00']
Length: 1, dtype: datetime64[us]

>>> pd.Series(
... [pd.Timestamp("2016-01-01", tz="US/Eastern") for _ in range(3)]
...).unique()
<DatetimeArray>
['2016-01-01 00:00:00-05:00']
Length: 1, dtype: datetime64[us, US/Eastern]

A Categorical will return categories in the order of appearance and with the same dtype.

>>> pd.Series(pd.Categorical(list("baabc"))).unique()
['b', 'a', 'c']
Categories (3, str): ['a', 'b', 'c']
>>> pd.Series(
... pd.Categorical(list("baabc"), categories=list("abc"), ordered=True)
...).unique()
['b', 'a', 'c']
Categories (3, str): ['a' < 'b' < 'c']

unstack(
 self,
 level: IndexLabel = -1,
 fill_value: Hashable | None = None,
 sort: bool = True
) -> DataFrame from pandas.core.series.Series
Unstack, also known as pivot, Series with MultiIndex to produce DataFrame.

Parameters

level : int, str, or list of these, default last level
 Level(s) to unstack, can pass level name.
fill_value : scalar value, default None
 Value to use when replacing NaN values.
sort : bool, default True
 Sort the level(s) in the resulting MultiIndex columns.

Returns

DataFrame
 Unstacked Series.

See Also

DataFrame.unstack : Pivot the MultiIndex of a DataFrame.

Notes

Reference :ref:`the user guide <reshaping.stacking>` for more examples.

Examples

```
>>> s = pd.Series(
...     [1, 2, 3, 4],
...     index=pd.MultiIndex.from_product([["one", "two"], ["a", "b"]]),
... )
>>> s
one  a    1
     b    2
two  a    3
     b    4
dtype: int64
```

```
>>> s.unstack(level=-1)
      a  b
one  1  2
two  3  4
```

```
>>> s.unstack(level=0)
      one  two
a       1    3
b       2    4
```

update(self, other: Series | Sequence | Mapping) -> None from pandas.core.series.Series
Modify Series in place using values from passed Series.

Uses non-NA values from passed Series to make updates. Aligns on index.

Parameters

other : Series, or object coercible into Series
Other Series that provides values to update the current Series.

See Also

Series.combine : Perform element-wise operation on two Series using a given function.
Series.transform: Modify a Series using a function.

Examples

```
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, 5, 6]))
>>> s
0    4
1    5
2    6
dtype: int64
```

```
>>> s = pd.Series(["a", "b", "c"])
>>> s.update(pd.Series(["d", "e"], index=[0, 2]))
>>> s
0    d
1    b
2    e
dtype: str
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, 5, 6, 7, 8]))
>>> s
0    4
1    5
2    6
dtype: int64
```

If ``other`` contains NaNs the corresponding values are not updated in the original Series.

```
>>> s = pd.Series([1, 2, 3])
>>> s.update(pd.Series([4, np.nan, 6]))
>>> s
```

```

0    4
1    2
2    6
dtype: int64

```

``other`` can also be a non-Series object type that is coercible into a Series

```

>>> s = pd.Series([1, 2, 3])
>>> s.update([4, np.nan, 6])
>>> s
0    4
1    2
2    6
dtype: int64

```

```

>>> s = pd.Series([1, 2, 3])
>>> s.update({1: 9})
>>> s
0    1
1    9
2    3
dtype: int64

```

```

var(
    self,
    *,
    axis: Axis | None = None,
    skipna: bool = True,
    ddof: int = 1,
    numeric_only: bool = False,
    **kwargs
) from pandas.core.series.Series
Return unbiased variance over requested axis.

```

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters

axis : {index (0)}

For ``Series`` this parameter is unused and defaults to 0.

.. warning::

The behavior of DataFrame.var with ``axis=None`` is deprecated, in a future version this will reduce over both axes and return a scalar To retain the old behavior, pass axis=0 (or do not pass axis).

skipna : bool, default True

Exclude NA/null values. If an entire row/column is NA, the result will be NA.

ddof : int, default 1

Delta Degrees of Freedom. The divisor used in calculations is N - ddof, where N represents the number of elements.

numeric_only : bool, default False

Include only float, int, boolean columns. Not implemented for Series.

**kwargs :

Additional keywords passed.

Returns

scalar or Series (if level specified)

Unbiased variance over requested axis.

See Also

numpy.var : Equivalent function in NumPy.

Series.std : Returns the standard deviation of the Series.

DataFrame.var : Returns the variance of the DataFrame.

DataFrame.std : Return standard deviation of the values over the requested axis.

Examples

```

>>> df = pd.DataFrame(
...     {
...         "person_id": [0, 1, 2, 3],

```

```
...         "age": [21, 25, 62, 43],
...         "height": [1.61, 1.87, 1.49, 2.01],
...     }
... ).set_index("person_id")
>>> df
```

```
      age  height
person_id
0        21    1.61
1        25    1.87
2        62    1.49
3        43    2.01
```

```
>>> df.var()
age      352.916667
height    0.056367
dtype: float64
```

Alternatively, ``ddof=0`` can be set to normalize by N instead of N-1:

```
>>> df.var(ddof=0)
age      264.687500
height    0.042275
dtype: float64
```

Class methods inherited from pandas.Series:

`from_arrow(data: ArrowArrayExportable | ArrowStreamExportable) -> Series` from pandas.core.series
Construct a Series from an array-like Arrow object.

This function accepts any Arrow-compatible array-like object implementing the `Arrow PyCapsule Protocol` (i.e. having an `__arrow_c_array__` or `__arrow_c_stream__` method).

This function currently relies on `pyarrow` to convert the object in Arrow format to pandas.

.. `_Arrow PyCapsule Protocol`: <https://arrow.apache.org/docs/format/CDataInterface/PyCapsuleProtocol.html>

.. versionadded:: 3.0

Parameters

`data` : pyarrow.Array or Arrow-compatible object
Any array-like object implementing the Arrow PyCapsule Protocol (i.e. has an `__arrow_c_array__` or `__arrow_c_stream__` method).

Returns

Series

See Also

`DataFrame.from_arrow` : Construct a DataFrame from an Arrow object.

Examples

```
>>> import pyarrow as pa
>>> arrow_array = pa.array([1, 2, 3])
>>> pd.Series.from_arrow(arrow_array)
0    1
1    2
2    3
dtype: int64
```

Readonly properties inherited from pandas.Series:

`array`

The ExtensionArray of the data backing this Series or Index.

This property provides direct access to the underlying array data of a Series or Index without requiring conversion to a NumPy array. It returns an ExtensionArray, which is the native storage format for pandas extension dtypes.

Returns

ExtensionArray

An ExtensionArray of the values stored within. For extension types, this is the actual array. For NumPy native types, this is a thin (no copy) wrapper around :class:`numpy.ndarray`.

```.array``` differs from ```.values```, which may require converting the data to a different form.

## See Also

-----

`Index.to_numpy` : Similar method that always returns a NumPy array.

`Series.to_numpy` : Similar method that always returns a NumPy array.

## Notes

-----

This table lays out the different array types for each extension dtype within pandas.

dtype	array type
category	Categorical
period	PeriodArray
interval	IntervalArray
IntegerNA	IntegerArray
string	StringArray
boolean	BooleanArray
datetime64[ns, tz]	DatetimeArray

For any 3rd-party extension types, the array type will be an ExtensionArray.

For all remaining dtypes ```.array``` will be a :class:`arrays.NumpyExtensionArray` wrapping the actual ndarray stored within. If you absolutely need a NumPy array (possibly with copying / coercing data), then use :meth:`Series.to\_numpy` instead.

## Examples

-----

For regular NumPy types like int, and float, a NumpyExtensionArray is returned.

```
>>> pd.Series([1, 2, 3]).array
<NumpyExtensionArray>
[1, 2, 3]
Length: 3, dtype: int64
```

For extension types, like Categorical, the actual ExtensionArray is returned

```
>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.array
['a', 'b', 'a']
Categories (2, str): ['a', 'b']
```

## axes

Return a list of the row axis labels.

## dtype

Return the dtype object of the underlying data.

## See Also

-----

`Series.dtypes` : Return the dtype object of the underlying data.

`Series.astype` : Cast a pandas object to a specified dtype dtype.

`Series.convert_dtypes` : Convert columns to the best possible dtypes using dtypes supporting pd.NA.

## Examples

-----

```
>>> s = pd.Series([1, 2, 3])
>>> s.dtype
dtype('int64')
```

## dtypes

Return the dtype object of the underlying data.

See Also

-----  
DataFrame.dtypes : Return the dtypes in the DataFrame.

Examples

-----  
>>> s = pd.Series([1, 2, 3])  
>>> s.dtypes  
dtype('int64')

## hasnans

Return True if there are any NaNs.

Enables various performance speedups.

Returns

-----  
bool

See Also

-----  
Series.isna : Detect missing values.  
Series.notna : Detect existing (non-missing) values.

Examples

-----  
>>> s = pd.Series([1, 2, 3, None])  
>>> s  
0 1.0  
1 2.0  
2 3.0  
3 NaN  
dtype: float64  
>>> s.hasnans  
True

## values

Return Series as ndarray or ndarray-like depending on the dtype.

.. warning::

We recommend using :attr:`Series.array` or  
:meth:`Series.to\_numpy`, depending on whether you need  
a reference to the underlying data or a NumPy array.

Returns

-----  
numpy.ndarray or ndarray-like

See Also

-----  
Series.array : Reference to the underlying data.  
Series.to\_numpy : A NumPy array representing the underlying data.

Examples

-----  
>>> pd.Series([1, 2, 3]).values  
array([1, 2, 3])

>>> pd.Series(list("aabc")).values  
<ArrowStringArray>  
['a', 'a', 'b', 'c']  
Length: 4, dtype: str

>>> pd.Series(list("aabc")).astype("category").values  
['a', 'a', 'b', 'c']  
Categories (3, str): ['a', 'b', 'c']

Timezone aware datetime data is converted to UTC:

>>> pd.Series(pd.date\_range("20130101", periods=3, tz="US/Eastern")).values  
array(['2013-01-01T05:00:00.000000',  
 '2013-01-02T05:00:00.000000',  
 '2013-01-03T05:00:00.000000'], dtype='datetime64[us]')

-----  
Data descriptors inherited from pandas.Series:

index

The index (axis labels) of the Series.

The index of a Series is used to label and identify each element of the underlying data. The index can be thought of as an immutable ordered set (technically a multi-set, as it may contain duplicate labels), and is used to index and align data in pandas.

Returns

-----

Index

The index labels of the Series.

See Also

-----

Series.reindex : Conform Series to new index.

Index : The base pandas index type.

Notes

-----

For more information on pandas indexing, see the `indexing user guide` <[https://pandas.pydata.org/docs/user\\_guide/indexing.html](https://pandas.pydata.org/docs/user_guide/indexing.html)> `\_\_`.

Examples

-----

To create a Series with a custom index and view the index labels:

```
>>> cities = ['Kolkata', 'Chicago', 'Toronto', 'Lisbon']
>>> populations = [14.85, 2.71, 2.93, 0.51]
>>> city_series = pd.Series(populations, index=cities)
>>> city_series.index
Index(['Kolkata', 'Chicago', 'Toronto', 'Lisbon'], dtype='object')
```

To change the index labels of an existing Series:

```
>>> city_series.index = ['KOL', 'CHI', 'TOR', 'LIS']
>>> city_series.index
Index(['KOL', 'CHI', 'TOR', 'LIS'], dtype='object')
```

name

Return the name of the Series.

The name of a Series becomes its index or column name if it is used to form a DataFrame. It is also used whenever displaying the Series using the interpreter.

Returns

-----

label (hashable object)

The name of the Series, also the column name if part of a DataFrame.

See Also

-----

Series.rename : Sets the Series name when given a scalar input.

Index.name : Corresponding Index property.

Examples

-----

The Series name can be set initially when calling the constructor.

```
>>> s = pd.Series([1, 2, 3], dtype=np.int64, name="Numbers")
>>> s
0 1
1 2
2 3
Name: Numbers, dtype: int64
>>> s.name = "Integers"
>>> s
0 1
1 2
2 3
Name: Integers, dtype: int64
```

The name of a Series within a DataFrame is its column name.

```
>>> df = pd.DataFrame(
... [[1, 2], [3, 4], [5, 6]], columns=["Odd Numbers", "Even Numbers"]
...)
>>> df
 Odd Numbers Even Numbers
0 1 2
1 3 4
2 5 6
>>> df["Even Numbers"].name
'Even Numbers'
```

-----  
Data and other attributes inherited from pandas.Series:

\_\_annotations\_\_ = {}

\_\_pandas\_priority\_\_ = 3000

cat = <class 'pandas.core.arrays.categorical.CategoricalAccessor'>  
Accessor object for categorical properties of the Series values.

Parameters

-----  
data : Series or CategoricalIndex  
The object to which the categorical accessor is attached.

See Also

-----  
Series.dt : Accessor object for datetimelike properties of the Series values.  
Series.sparse : Accessor for sparse matrix data types.

Examples

-----  
>>> s = pd.Series(list("abbccc")).astype("category")

```
>>> s
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

```
>>> s.cat.categories
Index(['a', 'b', 'c'], dtype='str')
```

```
>>> s.cat.rename_categories(list("cba"))
```

```
0 c
1 b
2 b
3 a
4 a
5 a
dtype: category
Categories (3, str): ['c', 'b', 'a']
```

```
>>> s.cat.reorder_categories(list("cba"))
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['c', 'b', 'a']
```

```
>>> s.cat.add_categories(["d", "e"])
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
```



```
Categories (5, str): ['a', 'b', 'c', 'd', 'e']
```

```
>>> s.cat.remove_categories(["a", "c"])
```

```
0 NaN
1 b
2 b
3 NaN
4 NaN
5 NaN
dtype: category
Categories (1, str): ['b']
```

```
>>> s1 = s.cat.add_categories(["d", "e"])
```

```
>>> s1.cat.remove_unused_categories()
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

```
>>> s.cat.set_categories(list("abcde"))
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (5, str): ['a', 'b', 'c', 'd', 'e']
```

```
>>> s.cat.as_ordered()
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a' < 'b' < 'c']
```

```
>>> s.cat.as_unordered()
```

```
0 a
1 b
2 b
3 c
4 c
5 c
dtype: category
Categories (3, str): ['a', 'b', 'c']
```

```
dt = <class 'pandas.core.indexes.accessors.CombinedDatetimelikePropert...
Accessor object for Series values' datetime-like, timedelta and period properties.
```

See Also

DatetimeIndex : Index of datetime64 data.

Examples

```
>>> dates = pd.Series(
... ["2024-01-01", "2024-01-15", "2024-02-5"], dtype="datetime64[ns]"
...)
>>> dates.dt.day
0 1
1 15
2 5
dtype: int32
>>> dates.dt.month
0 1
1 1
2 2
dtype: int32
```

```
>>> dates = pd.Series(
... ["2024-01-01", "2024-01-15", "2024-02-5"], dtype="datetime64[ns, UTC]"
...)
>>> dates.dt.day
0 1
1 15
2 5
dtype: int32
>>> dates.dt.month
0 1
1 1
2 2
dtype: int32
```

```
list = <class 'pandas.core.arrays.arrow.accessors.ListAccessor'>
Accessor object for list data properties of the Series values.
```

Parameters

-----

```
data : Series
Series containing Arrow list data.
```

```
plot = <class 'pandas.plotting.PlotAccessor'>
Make plots of Series or DataFrame.
```

Uses the backend specified by the option ``plotting.backend``. By default, matplotlib is used.

Parameters

-----

```
data : Series or DataFrame
The object for which the method is called.
```

Attributes

-----

```
x : label or position, default None
Only used if data is a DataFrame.
y : label, position or list of label, positions, default None
Allows plotting of one column versus another. Only used if data is a
DataFrame.
```

```
kind : str
The kind of plot to produce:
```

- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)

```
ax : matplotlib axes object, default None
An axes of the current figure.
```

```
subplots : bool or sequence of iterables, default False
Whether to group columns into subplots:
```

- ``False`` : No subplots will be used
- ``True`` : Make separate subplots for each column.
- sequence of iterables of column labels: Create a subplot for each group of columns. For example ``[['a', 'c'), ('b', 'd')])`` will create 2 subplots: one with columns 'a' and 'c', and one with columns 'b' and 'd'. Remaining columns that aren't specified will be plotted in additional subplots (one per column).

```
sharex : bool, default True if ax is None else False
In case ``subplots=True``, share x axis and set some x axis labels
to invisible; defaults to True if ax is None otherwise False if
an ax is passed in; Be aware, that passing in both an ax and
``sharex=True`` will alter all x axis labels for all axis in a figure.
```

```
sharey : bool, default False
```

```
In case ``subplots=True``, share y axis and set some y axis labels to invisible.
layout : tuple, optional
```

(rows, columns) for the layout of subplots.  
 figsize : a tuple (width, height) in inches  
 Size of a figure object.  
 use\_index : bool, default True  
 Use index as ticks for x axis.  
 title : str or list  
 Title to use for the plot. If a string is passed, print the string at the top of the figure. If a list is passed and `subplots` is True, print each item in the list above the corresponding subplot.  
 grid : bool, default None (matlab style default)  
 Axis grid lines.  
 legend : bool or {'reverse'}  
 Place legend on axis subplots.  
 style : list or dict  
 The matplotlib line style per column.  
 logx : bool or 'sym', default False  
 Use log scaling or symlog scaling on x axis.  
  
 logy : bool or 'sym' default False  
 Use log scaling or symlog scaling on y axis.  
  
 loglog : bool or 'sym', default False  
 Use log scaling or symlog scaling on both x and y axes.  
  
 xticks : sequence  
 Values to use for the xticks.  
 yticks : sequence  
 Values to use for the yticks.  
 xlim : 2-tuple/list  
 Set the x limits of the current axes.  
 ylim : 2-tuple/list  
 Set the y limits of the current axes.  
 xlabel : label, optional  
 Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.  
  
 .. versionchanged:: 2.0.0  
  
 Now applicable to histograms.  
  
 ylabel : label, optional  
 Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.  
  
 .. versionchanged:: 2.0.0  
  
 Now applicable to histograms.  
  
 rot : float, default None  
 Rotation for ticks (xticks for vertical, yticks for horizontal plots).  
 fontsize : float, default None  
 Font size for xticks and yticks.  
 colormap : str or matplotlib colormap object, default None  
 Colormap to select colors from. If string, load colormap with that name from matplotlib.  
 colorbar : bool, optional  
 If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).  
 position : float  
 Specify relative alignments for bar plot layout.  
 From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).  
 table : bool, Series or DataFrame, default False  
 If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout.  
 If a Series or DataFrame is passed, use passed data to draw a table.  
 yerr : DataFrame, Series, array-like, dict and str  
 See :ref:`Plotting with Error Bars <visualization.errorbars>` for detail.  
 xerr : DataFrame, Series, array-like, dict and str  
 Equivalent to yerr.  
 stacked : bool, default False in line and bar plots, and True in area plot  
 If True, create stacked plot.  
 secondary\_y : bool or sequence, default False  
 Whether to plot on the secondary y-axis if a list/tuple, which

```

 columns to plot on secondary y-axis.
mark_right : bool, default True
 When using a secondary_y axis, automatically mark the column
 labels with "(right)" in the legend.
include_bool : bool, default is False
 If True, boolean values can be plotted.
backend : str, default None
 Backend to use instead of the backend specified in the option
 ``plotting.backend``. For instance, 'matplotlib'. Alternatively, to
 specify the ``plotting.backend`` for the whole session, set
 ``pd.options.plotting.backend``.
**kwargs
 Options to pass to matplotlib plotting method.

Returns

:class:`matplotlib.axes.Axes` or numpy.ndarray of them
 If the backend is not the default matplotlib one, the return value
 will be the object returned by the backend.

See Also

matplotlib.pyplot.plot : Plot y versus x as lines and/or markers.
DataFrame.hist : Make a histogram.
DataFrame.boxplot : Make a box plot.
DataFrame.plot.scatter : Make a scatter plot with varying marker
 point size and color.
DataFrame.plot.hexbin : Make a hexagonal binning plot of
 two variables.
DataFrame.plot.kde : Make Kernel Density Estimate plot using
 Gaussian kernels.
DataFrame.plot.area : Make a stacked area plot.
DataFrame.plot.bar : Make a bar plot.
DataFrame.plot.barh : Make a horizontal bar plot.

Notes

- See matplotlib documentation online for more on this subject
- If `kind` = 'bar' or 'barh', you can specify relative alignments
 for bar plot layout by `position` keyword.
 From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5
 (center)

Examples

For Series:

.. plot::
 :context: close-figs

 >>> ser = pd.Series([1, 2, 3, 3])
 >>> plot = ser.plot(kind="hist", title="My plot")

For DataFrame:

.. plot::
 :context: close-figs

 >>> df = pd.DataFrame(
 ... {
 ... "length": [1.5, 0.5, 1.2, 0.9, 3],
 ... "width": [0.7, 0.2, 0.15, 0.2, 1.1],
 ... },
 ... index=["pig", "rabbit", "duck", "chicken", "horse"],
 ...)
 >>> plot = df.plot(title="DataFrame Plot")

For SeriesGroupBy:

.. plot::
 :context: close-figs

 >>> lst = [-1, -2, -3, 1, 2, 3]
 >>> ser = pd.Series([1, 2, 2, 4, 6, 6], index=lst)
 >>> plot = ser.groupby(lambda x: x > 0).plot(title="SeriesGroupBy Plot")

For DataFrameGroupBy:

```

```

.. plot::
:context: close-figs

>>> df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": ["A", "B", "A", "B"]})
>>> plot = df.groupby("col2").plot(kind="bar", title="DataFrameGroupBy Plot")

```

sparse = <class 'pandas.core.arrays.sparse.accessor.SparseAccessor'>  
 Accessor for SparseSparse from other sparse matrix data types.

Parameters  
 -----

data : Series or DataFrame  
 The Series or DataFrame to which the SparseAccessor is attached.

See Also  
 -----

Series.sparse.to\_coo : Create a scipy.sparse.coo\_matrix from a Series with MultiIndex.  
 Series.sparse.from\_coo : Create a Series with sparse values from a scipy.sparse.coo\_matrix.

Examples  
 -----

```

>>> ser = pd.Series([0, 0, 2, 2, 2], dtype="Sparse[int]")
>>> ser.sparse.density
0.6
>>> ser.sparse.sp_values
array([2, 2, 2])

```

str = <class 'pandas.core.strings.accessor.StringMethods'>  
 Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method.  
 Patterned after Python's string methods, with some inspiration from R's stringr package.

Parameters  
 -----

data : Series or Index  
 The content of the Series or Index.

See Also  
 -----

Series.str : Vectorized string functions for Series.  
 Index.str : Vectorized string functions for Index.

Examples  
 -----

```

>>> s = pd.Series(["A_Str_Series"])
>>> s
0 A_Str_Series
dtype: str

>>> s.str.split("_")
0 [A, Str, Series]
dtype: object

>>> s.str.replace("_", "")
0 AStrSeries
dtype: str

```

struct = <class 'pandas.core.arrays.arrow.accessors.StructAccessor'>  
 Accessor object for structured data properties of the Series values.

Parameters  
 -----

data : Series  
 Series containing Arrow struct data.

-----  
 Methods inherited from pandas.core.base.IndexOpsMixin:

```

__iter__(self) -> Iterator
 Return an iterator of the values.

 These are each a scalar type, which is a Python scalar
 (for str, int, float) or a pandas scalar
 (for Timestamp/Timedelta/Interval/Period)

 Returns

 iterator
 An iterator yielding scalar values from the Series.

 See Also

 Series.items : Lazily iterate over (index, value) tuples.

 Examples

 >>> s = pd.Series([1, 2, 3])
 >>> for x in s:
 ... print(x)
 1
 2
 3

argmax(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int
 Return int position of the largest value in the Series.

 If the maximum is achieved in multiple locations,
 the first row position is returned.

 Parameters

 axis : None
 Unused. Parameter needed for compatibility with DataFrame.
 skipna : bool, default True
 Exclude NA/null values. If the entire Series is NA, or if ``skipna=False``
 and there is an NA value, this method will raise a ``ValueError``.
 *args, **kwargs
 Additional arguments and keywords for compatibility with NumPy.

 Returns

 int
 Row position of the maximum value.

 See Also

 Series.argmax : Return position of the maximum value.
 Series.argmin : Return position of the minimum value.
 numpy.ndarray.argmax : Equivalent method for numpy arrays.
 Series.idxmax : Return index label of the maximum values.
 Series.idxmin : Return index label of the minimum values.

 Examples

 Consider dataset containing cereal calories

 >>> s = pd.Series(
 ... [100.0, 110.0, 120.0, 110.0],
 ... index=[
 ... "Corn Flakes",
 ... "Almond Delight",
 ... "Cinnamon Toast Crunch",
 ... "Cocoa Puff",
 ...],
 ...)
 >>> s
 Corn Flakes 100.0
 Almond Delight 110.0
 Cinnamon Toast Crunch 120.0
 Cocoa Puff 110.0
 dtype: float64

 >>> s.argmax()
 np.int64(2)
 >>> s.argmin()

```

```
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since series is zero-indexed.

```
argmin(self, axis: AxisInt | None = None, skipna: bool = True, *args, **kwargs) -> int
Return int position of the smallest value in the Series.
```

If the minimum is achieved in multiple locations, the first row position is returned.

#### Parameters

-----

axis : None

Unused. Parameter needed for compatibility with DataFrame.

skipna : bool, default True

Exclude NA/null values. If the entire Series is NA, or if ``skipna=False`` and there is an NA value, this method will raise a ``ValueError``.

\*args, \*\*kwargs

Additional arguments and keywords for compatibility with NumPy.

#### Returns

-----

int

Row position of the minimum value.

#### See Also

-----

Series.argmax : Return position of the maximum value.

Series.argmax : Return position of the maximum value.

numpy.ndarray.argmax : Equivalent method for numpy arrays.

Series.idxmin : Return index label of the minimum values.

Series.idxmax : Return index label of the maximum values.

#### Examples

-----

Consider dataset containing cereal calories

```
>>> s = pd.Series(
... [100.0, 110.0, 120.0, 110.0],
... index=[
... "Corn Flakes",
... "Almond Delight",
... "Cinnamon Toast Crunch",
... "Cocoa Puff",
...],
...)
>>> s
Corn Flakes 100.0
Almond Delight 110.0
Cinnamon Toast Crunch 120.0
Cocoa Puff 110.0
dtype: float64
```

```
>>> s.argmax()
np.int64(2)
>>> s.argmin()
np.int64(0)
```

The maximum cereal calories is the third element and the minimum cereal calories is the first element, since series is zero-indexed.

```
factorize(self, sort: bool = False, use_na_sentinel: bool = True) -> tuple[npt.NDArray[np.in
Encode the object as an enumerated type or categorical variable.
```

This method is useful for obtaining a numeric representation of an array when all that matters is identifying distinct values. ``factorize`` is available as both a top-level function :func:`pandas.factorize`, and as a method :meth:`Series.factorize` and :meth:`Index.factorize`.

#### Parameters

-----

sort : bool, default False

Sort ``uniques`` and shuffle ``codes`` to maintain the relationship.

`use_na_sentinel` : bool, default True  
If True, the sentinel -1 will be used for NaN values. If False, NaN values will be encoded as non-negative integers and will not drop the NaN from the uniques of the values.

Returns

-----  
`codes` : ndarray  
An integer ndarray that's an indexer into ``uniques``.  
``uniques.take(codes)`` will have the same values as ``values``.  
`uniques` : ndarray, Index, or Categorical  
The unique valid values. When ``values`` is Categorical, ``uniques`` is a Categorical. When ``values`` is some other pandas object, an ``Index`` is returned. Otherwise, a 1-D ndarray is returned.

.. note::

Even if there's a missing value in ``values``, ``uniques`` will *not* contain an entry for it.

See Also

-----  
`cut` : Discretize continuous-valued array.  
`unique` : Find the unique value in an array.

Notes

-----  
Reference :ref:`the user guide <reshaping.factorize>` for more examples.

Examples

-----  
These examples all show `factorize` as a top-level method like ``pd.factorize(values)``. The results are identical for methods like `:meth:`Series.factorize``.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", "b", "a", "c", "b"], dtype="O")
...)
>>> codes
array([0, 0, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

With ``sort=True``, the ``uniques`` will be sorted, and ``codes`` will be shuffled so that the relationship is maintained.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", "b", "a", "c", "b"], dtype="O"), sort=True
...)
>>> codes
array([1, 1, 0, 2, 1])
>>> uniques
array(['a', 'b', 'c'], dtype=object)
```

When ``use_na_sentinel=True`` (the default), missing values are indicated in the ``codes`` with the sentinel value ``-1`` and missing values are not included in ``uniques``.

```
>>> codes, uniques = pd.factorize(
... np.array(["b", None, "a", "c", "b"], dtype="O")
...)
>>> codes
array([0, -1, 1, 2, 0])
>>> uniques
array(['b', 'a', 'c'], dtype=object)
```

Thus far, we've only factorized lists (which are internally coerced to NumPy arrays). When factorizing pandas objects, the type of ``uniques`` will differ. For Categoricals, a ``Categorical`` is returned.

```
>>> cat = pd.Categorical(["a", "a", "c"], categories=["a", "b", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
['a', 'c']
Categories (3, str): ['a', 'b', 'c']
```



Notice that ``'b'`` is in ``uniques.categories``, despite not being present in ``cat.values``.

For all other pandas objects, an Index of the appropriate type is returned.

```
>>> cat = pd.Series(["a", "a", "c"])
>>> codes, uniques = pd.factorize(cat)
>>> codes
array([0, 0, 1])
>>> uniques
Index(['a', 'c'], dtype='str')
```

If NaN is in the values, and we want to include NaN in the uniques of the values, it can be achieved by setting ``use\_na\_sentinel=False``.

```
>>> values = np.array([1, 2, 1, np.nan])
>>> codes, uniques = pd.factorize(values) # default: use_na_sentinel=True
>>> codes
array([0, 1, 0, -1])
>>> uniques
array([1., 2.])

>>> codes, uniques = pd.factorize(values, use_na_sentinel=False)
>>> codes
array([0, 1, 0, 2])
>>> uniques
array([1., 2., nan])
```

`item(self)`  
Return the first element of the underlying data as a Python scalar.

Returns

-----

scalar

The first element of Series or Index.

Raises

-----

ValueError

If the data is not length = 1.

See Also

-----

`Index.values` : Returns an array representing the data in the Index.

`Series.head` : Returns the first ``n`` rows.

Examples

-----

```
>>> s = pd.Series([1])
>>> s.item()
1
```

For an index:

```
>>> s = pd.Series([1], index=["a"])
>>> s.index.item()
'a'
```

`nunique(self, dropna: bool = True) -> int`  
Return number of unique elements in the object.

Excludes NA values by default.

Parameters

-----

`dropna` : bool, default True

Don't include NaN in the count.

Returns

-----

int

An integer indicating the number of unique elements in the object.

See Also

-----

DataFrame.nunique: Method nunique for DataFrame.  
Series.count: Count non-NA/null observations in the Series.

#### Examples

```

>>> s = pd.Series([1, 3, 5, 7, 7])
>>> s
0 1
1 3
2 5
3 7
4 7
dtype: int64

>>> s.nunique()
4
```

to\_list = tolist(self) -> list

to\_numpy(  
 self,  
 dtype: npt.DTypeLike | None = None,  
 copy: bool = False,  
 na\_value: object = <no\_default>,  
 \*\*kwargs  
) -> np.ndarray  
A NumPy ndarray representing the values in this Series or Index.

#### Parameters

-----  
dtype : str or numpy.dtype, optional  
The dtype to pass to :meth:`numpy.asarray`.  
copy : bool, default False  
Whether to ensure that the returned value is not a view on another array. Note that ``copy=False`` does not \*ensure\* that ``to\_numpy()`` is no-copy. Rather, ``copy=True`` ensure that a copy is made, even if not strictly necessary.  
na\_value : Any, optional  
The value to use for missing values. The default value depends on `dtype` and the type of the array.  
\*\*kwargs  
Additional keywords passed through to the ``to\_numpy`` method of the underlying array (for extension arrays).

#### Returns

-----  
numpy.ndarray  
The NumPy ndarray holding the values from this Series or Index. The dtype of the array may differ. See Notes.

#### See Also

-----  
Series.array : Get the actual data stored within.  
Index.array : Get the actual data stored within.  
DataFrame.to\_numpy : Similar method for DataFrame.

#### Notes

-----  
The returned array will be the same up to equality (values equal in `self` will be equal in the returned array; likewise for values that are not equal). When `self` contains an ExtensionArray, the dtype may be different. For example, for a category-dtype Series, ``to\_numpy()`` will return a NumPy array and the categorical dtype will be lost.

For NumPy dtypes, this will be a reference to the actual data stored in this Series or Index (assuming ``copy=False``). Modifying the result in place will modify the data stored in the Series or Index (not that we recommend doing that).

For extension types, ``to\_numpy()`` \*may\* require copying data and coercing the result to a NumPy type (possibly object), which may be expensive. When you need a no-copy reference to the underlying data, :attr:`Series.array` should be used instead.

This table lays out the different dtypes and default return types of ``to\_numpy()`` for various dtypes within pandas.

```

=====
dtype array type
=====
category[T] ndarray[T] (same dtype as input)
period ndarray[object] (Periods)
interval ndarray[object] (Intervals)
IntegerNA ndarray[object]
datetime64[ns] datetime64[ns]
datetime64[ns, tz] ndarray[object] (Timestamps)
=====

```

#### Examples

```

>>> ser = pd.Series(pd.Categorical(["a", "b", "a"]))
>>> ser.to_numpy()
array(['a', 'b', 'a'], dtype=object)

```

Specify the `dtype` to control how datetime-aware data is represented. Use ``dtype=object`` to return an ndarray of pandas :class:`Timestamp` objects, each with the correct ``tz``.

```

>>> ser = pd.Series(pd.date_range("2000", periods=2, tz="CET"))
>>> ser.to_numpy(dtype=object)
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET'),
 Timestamp('2000-01-02 00:00:00+0100', tz='CET')],
 dtype=object)

```

Or ``dtype='datetime64[ns]'`` to return an ndarray of native datetime64 values. The values are converted to UTC and the timezone info is dropped.

```

>>> ser.to_numpy(dtype="datetime64[ns]")
... # doctest: +ELLIPSIS
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00...'],
 dtype='datetime64[ns]')

```

`tolist(self)` -> list

Return a list of the values.

These are each a scalar type, which is a Python scalar (for str, int, float) or a pandas scalar (for Timestamp/Timedelta/Interval/Period)

#### Returns

list

List containing the values as Python or pandas scalars.

#### See Also

`numpy.ndarray.tolist` : Return the array as an a.ndim-levels deep nested list of Python scalars.

#### Examples

##### For Series

```

>>> s = pd.Series([1, 2, 3])
>>> s.to_list()
[1, 2, 3]

```

##### For Index:

```

>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')

```

```

>>> idx.to_list()
[1, 2, 3]

```

`transpose(self, *args, **kwargs)` -> Self

Return the transpose, which is by definition self.

#### Returns

%(klass)s

```

value_counts(
 self,
 normalize: bool = False,
 sort: bool = True,
 ascending: bool = False,
 bins=None,
 dropna: bool = True
) -> Series
 Return a Series containing counts of unique values.

 The resulting object will be in descending order so that the
 first element is the most frequently-occurring element.
 Excludes NA values by default.

Parameters

normalize : bool, default False
 If True then the object returned will contain the relative
 frequencies of the unique values.
sort : bool, default True
 Stable sort by frequencies when True. Preserve the order of the data
 when False.

 .. versionchanged:: 3.0.0

 Prior to 3.0.0, the sort was unstable.
ascending : bool, default False
 Sort in ascending order.
bins : int, optional
 Rather than count values, group them into half-open bins,
 a convenience for ``pd.cut``, only works with numeric data.
dropna : bool, default True
 Don't include counts of NaN.

Returns

Series
 Series containing counts of unique values.

See Also

Series.count: Number of non-NA elements in a Series.
DataFrame.count: Number of non-NA elements in a DataFrame.
DataFrame.value_counts: Equivalent method on DataFrames.

Examples

>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0 2
1.0 1
2.0 1
4.0 1
Name: count, dtype: int64

With `normalize` set to `True`, returns the relative frequency by
dividing all values by the sum of values.

>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0 0.4
1.0 0.2
2.0 0.2
4.0 0.2
Name: proportion, dtype: float64

bins

Bins can be useful for going from a continuous variable to a
categorical variable; instead of counting unique
apparitions of values, divide the index in the specified
number of half-open bins.

>>> s.value_counts(bins=3)
(0.996, 2.0] 2
(2.0, 3.0] 2

```

```
(3.0, 4.0] 1
Name: count, dtype: int64
```

**\*\*dropna\*\***

With `dropna` set to `False` we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0 2
1.0 1
2.0 1
4.0 1
NaN 1
Name: count, dtype: int64
```

**\*\*Categorical Dtypes\*\***

Rows with categorical type will be counted as one group if they have same categories and order.  
In the example below, even though `a`, `c`, and `d` all have the same data types of `category`, only `c` and `d` will be counted as one group since `a` doesn't have the same categories.

```
>>> df = pd.DataFrame({"a": [1], "b": ["2"], "c": [3], "d": [3]})
>>> df = df.astype({"a": "category", "c": "category", "d": "category"})
>>> df
 a b c d
0 1 2 3 3

>>> df.dtypes
a category
b str
c category
d category
dtype: object

>>> df.dtypes.value_counts()
category 2
category 1
str 1
Name: count, dtype: int64
```

-----  
Readonly properties inherited from pandas.core.base.IndexOpsMixin:

T

Return the transpose, which is by definition self.

See Also

-----  
Index : Immutable sequence used for indexing and alignment.

Examples

-----  
For Series:

```
>>> s = pd.Series(['Ant', 'Bear', 'Cow'])
>>> s
0 Ant
1 Bear
2 Cow
dtype: str
>>> s.T
0 Ant
1 Bear
2 Cow
dtype: str
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx.T
Index([1, 2, 3], dtype='int64')
```

empty

Indicator whether Index is empty.

An Index is considered empty if it has no elements. This property can be useful for quickly checking the state of an Index, especially in data processing and analysis workflows where handling of empty datasets might be required.

Returns

-----

bool

If Index is empty, return True, if not return False.

See Also

-----

Index.size : Return the number of elements in the underlying data.

Examples

-----

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.empty
False
```

```
>>> idx_empty = pd.Index([])
>>> idx_empty
Index([], dtype='object')
>>> idx_empty.empty
True
```

If we only have NaNs in our DataFrame, it is not considered empty!

```
>>> idx = pd.Index([np.nan, np.nan])
>>> idx
Index([nan, nan], dtype='float64')
>>> idx.empty
False
```

is\_monotonic\_decreasing

Return True if values in the object are monotonically decreasing.

Returns

-----

bool

See Also

-----

Series.is\_monotonic\_increasing : Return boolean if values in the object are monotonically increasing.

Examples

-----

```
>>> s = pd.Series([3, 2, 2, 1])
>>> s.is_monotonic_decreasing
True
```

```
>>> s = pd.Series([1, 2, 3])
>>> s.is_monotonic_decreasing
False
```

is\_monotonic\_increasing

Return True if values in the object are monotonically increasing.

Returns

-----

bool

See Also

-----

Series.is\_monotonic\_decreasing : Return boolean if values in the object are monotonically decreasing.

Examples

-----

```
>>> s = pd.Series([1, 2, 2])
>>> s.is_monotonic_increasing
True
```

```
>>> s = pd.Series([3, 2, 1])
>>> s.is_monotonic_increasing
False
```

is\_unique

Return True if values in the object are unique.

Returns

-----  
bool

See Also

-----

Series.unique : Return unique values of Series object.

Series.drop\_duplicates : Return Series with duplicate values removed.

Series.duplicated : Indicate duplicate Series values.

Examples

-----

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> s.is_unique
```

```
True
```

```
>>> s = pd.Series([1, 2, 3, 1])
```

```
>>> s.is_unique
```

```
False
```

nbytes

Return the number of bytes in the underlying data.

See Also

-----

Series.ndim : Number of dimensions of the underlying data.

Series.size : Return the number of elements in the underlying data.

Examples

-----

For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
```

```
>>> s
```

```
0 Ant
```

```
1 Bear
```

```
2 Cow
```

```
dtype: str
```

```
>>> s.nbytes
```

```
34
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
```

```
>>> idx
```

```
Index([1, 2, 3], dtype='int64')
```

```
>>> idx.nbytes
```

```
24
```

ndim

Number of dimensions of the underlying data, by definition 1.

See Also

-----

Series.size: Return the number of elements in the underlying data.

Series.shape: Return a tuple of the shape of the underlying data.

Series.dtype: Return the dtype object of the underlying data.

Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

-----

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
```

```
>>> s
```

```
0 Ant
```

```
1 Bear
```

```
2 Cow
```

```
dtype: str
```

```
>>> s.ndim
```

```
1
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.ndim
1
```

shape

Return a tuple of the shape of the underlying data.

See Also

-----  
Series.ndim : Number of dimensions of the underlying data.  
Series.size : Return the number of elements in the underlying data.  
Series.nbytes : Return the number of bytes in the underlying data.

Examples

-----  
>>> s = pd.Series([1, 2, 3])  
>>> s.shape  
(3,)

size

Return the number of elements in the underlying data.

See Also

-----  
Series.ndim: Number of dimensions of the underlying data, by definition 1.  
Series.shape: Return a tuple of the shape of the underlying data.  
Series.dtype: Return the dtype object of the underlying data.  
Series.values: Return Series as ndarray or ndarray-like depending on the dtype.

Examples

-----  
For Series:

```
>>> s = pd.Series(["Ant", "Bear", "Cow"])
>>> s
0 Ant
1 Bear
2 Cow
dtype: str
>>> s.size
3
```

For Index:

```
>>> idx = pd.Index([1, 2, 3])
>>> idx
Index([1, 2, 3], dtype='int64')
>>> idx.size
3
```

-----  
Data and other attributes inherited from pandas.core.base.IndexOpsMixin:

\_\_array\_priority\_\_ = 1000

-----  
Methods inherited from pandas.core.arraylike.OpsMixin:

\_\_add\_\_(self, other)  
Get Addition of DataFrame and other, column-wise.

Equivalent to ``DataFrame.add(other)``.

Parameters

-----  
other : scalar, sequence, Series, dict or DataFrame  
Object to be added to the DataFrame.

Returns

-----  
DataFrame

The result of adding ``other`` to DataFrame.



See Also

-----  
DataFrame.add : Add a DataFrame and another object, with option for index- or column-oriented addition.

Examples

```
>>> df = pd.DataFrame(
... {"height": [1.5, 2.6], "weight": [500, 800]}, index=["elk", "moose"]
...)
>>> df
```

	height	weight
elk	1.5	500
moose	2.6	800

Adding a scalar affects all rows and columns.

```
>>> df[["height", "weight"]] + 1.5
```

	height	weight
elk	3.0	501.5
moose	4.1	801.5

Each element of a list is added to a column of the DataFrame, in order.

```
>>> df[["height", "weight"]] + [0.5, 1.5]
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

Keys of a dictionary are aligned to the DataFrame, based on column names; each value in the dictionary is added to the corresponding column.

```
>>> df[["height", "weight"]] + {"height": 0.5, "weight": 1.5}
```

	height	weight
elk	2.0	501.5
moose	3.1	801.5

When `other` is a `:class:`Series``, the index of `other` is aligned with the columns of the DataFrame.

```
>>> s1 = pd.Series([0.5, 1.5], index=["weight", "height"])
>>> df[["height", "weight"]] + s1
```

	height	weight
elk	3.0	500.5
moose	4.1	800.5

Even when the index of `other` is the same as the index of the DataFrame, the `:class:`Series`` will not be reoriented. If index-wise alignment is desired, `:meth:`DataFrame.add`` should be used with ``axis='index'``.

```
>>> s2 = pd.Series([0.5, 1.5], index=["elk", "moose"])
>>> df[["height", "weight"]] + s2
```

	elk	height	moose	weight
elk	NaN	NaN	NaN	NaN
moose	NaN	NaN	NaN	NaN

```
>>> df[["height", "weight"]].add(s2, axis="index")
```

	height	weight
elk	2.0	500.5
moose	4.1	801.5

When `other` is a `:class:`DataFrame``, both columns names and the index are aligned.

```
>>> other = pd.DataFrame(
... {"height": [0.2, 0.4, 0.6]}, index=["elk", "moose", "deer"]
...)
>>> df[["height", "weight"]] + other
```

	height	weight
deer	NaN	NaN
elk	1.7	NaN
moose	3.0	NaN

\_\_and\_\_(self, other)

\_\_divmod\_\_(self, other)

```

__eq__(self, other)
 Return self==value.

__floordiv__(self, other)

__ge__(self, other)
 Return self>=value.

__gt__(self, other)
 Return self>value.

__le__(self, other)
 Return self<=value.

__lt__(self, other)
 Return self<value.

__mod__(self, other)

__mul__(self, other)

__ne__(self, other)
 Return self!=value.

__or__(self, other)
 Return self|value.

__pow__(self, other)

__radd__(self, other)

__rand__(self, other)

__rdivmod__(self, other)

__rfloordiv__(self, other)

__rmod__(self, other)

__rmul__(self, other)

__ror__(self, other)
 Return value|self.

__rpow__(self, other)

__rsub__(self, other)

__rtruediv__(self, other)

__rxor__(self, other)

__sub__(self, other)

__truediv__(self, other)

__xor__(self, other)

```

-----  
Data descriptors inherited from pandas.core.arraylike.OpsMixin:

```

__dict__
 dictionary for instance variables

__weakref__
 list of weak references to the object

```

-----  
Data and other attributes inherited from pandas.core.arraylike.OpsMixin:

```

__hash__ = None

```

-----  
Methods inherited from pandas.core.generic.NDFrame:

```

__abs__(self) -> Self

```

```

__array_ufunc__(self, ufunc: np.ufunc, method: str, *inputs: Any, **kwargs: Any)

__bool__(self) -> NoReturn

__contains__(self, key) -> bool
 True if the key is in the info axis

__copy__(self) -> Self

__deepcopy__(self, memo=None) -> Self
 Parameters

 memo, default None
 Standard signature. Unused

__delitem__(self, key) -> None
 Delete item

__finalize__(self, other, method: str | None = None, **kwargs) -> Self
 Propagate metadata from other to self.

 This is the default implementation. Subclasses may override this method to
 implement their own metadata handling.

 Parameters

 other : the object from which to get the attributes that we are going
 to propagate. If ``other`` has an ``input_objs`` attribute, then
 this attribute must contain an iterable of objects, each with an
 ``attrs`` attribute.
 method : str, optional
 A passed method name providing context on where ``__finalize__``
 was called.

 .. warning::

 The value passed as `method` are not currently considered
 stable across pandas releases.

 Notes

 In case ``other`` has an ``input_objs`` attribute, this method only
 propagates its metadata if each object in ``input_objs`` has the exact
 same metadata as the others.

__getattr__(self, name: str)
 After regular attribute access, try looking up the name
 This allows simpler access to columns for interactive use.

__getstate__(self) -> dict[str, Any]
 Helper for pickle.

__iadd__(self, other) -> Self

__iand__(self, other) -> Self

__ifloordiv__(self, other) -> Self

__imod__(self, other) -> Self

__imul__(self, other) -> Self

__invert__(self) -> Self

__ior__(self, other) -> Self

__ipow__(self, other) -> Self

__isub__(self, other) -> Self

__itruediv__(self, other) -> Self

__ixor__(self, other) -> Self

__neg__(self) -> Self

__pos__(self) -> Self

```

```

__round__(self, decimals: int = 0) -> Self

__setattr__(self, name: str, value) -> None
 After regular attribute access, try setting the name
 This allows simpler access to columns for interactive use.

__setstate__(self, state) -> None

abs(self) -> Self
 Return a Series/DataFrame with absolute numeric value of each element.

 This function only applies to elements that are all numeric.

 Returns

 abs
 Series/DataFrame containing the absolute value of each element.

 See Also

 numpy.absolute : Calculate the absolute value element-wise.

 Notes

 For ``complex`` inputs, ``1.2 + 1j``, the absolute value is
 :math:\sqrt{a^2 + b^2}``.

 Examples

 Absolute numeric values in a Series.

 >>> s = pd.Series([-1.10, 2, -3.33, 4])
 >>> s.abs()
 0 1.10
 1 2.00
 2 3.33
 3 4.00
 dtype: float64

 Absolute numeric values in a Series with complex numbers.

 >>> s = pd.Series([1.2 + 1j])
 >>> s.abs()
 0 1.56205
 dtype: float64

 Absolute numeric values in a Series with a Timedelta element.

 >>> s = pd.Series([pd.Timedelta("1 days")])
 >>> s.abs()
 0 1 days
 dtype: timedelta64[us]

 Select rows with data closest to certain value using argsort (from
 `StackOverflow <https://stackoverflow.com/a/17758115>`__).

 >>> df = pd.DataFrame(
 ... {"a": [4, 5, 6, 7], "b": [10, 20, 30, 40], "c": [100, 50, -30, -50]}
 ...)
 >>> df
 a b c
 0 4 10 100
 1 5 20 50
 2 6 30 -30
 3 7 40 -50
 >>> df.loc[(df.c - 43).abs().argsort()]
 a b c
 1 5 20 50
 0 4 10 100
 2 6 30 -30
 3 7 40 -50

add_prefix(self, prefix: str, axis: Axis | None = None) -> Self
 Prefix labels with string `prefix`.

 For Series, the row labels are prefixed.

```

For DataFrame, the column labels are prefixed.

#### Parameters

-----  
prefix : str  
    The string to add before each label.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
    Axis to add prefix on  
  
    .. versionadded:: 2.0.0

#### Returns

-----  
Series or DataFrame  
    New Series or DataFrame with updated labels.

#### See Also

-----  
Series.add\_suffix: Suffix row labels with string `suffix`.  
DataFrame.add\_suffix: Suffix column labels with string `suffix`.

#### Examples

-----  
>>> s = pd.Series([1, 2, 3, 4])  
>>> s  
0 1  
1 2  
2 3  
3 4  
dtype: int64  
  
>>> s.add\_prefix("item\_")  
item\_0 1  
item\_1 2  
item\_2 3  
item\_3 4  
dtype: int64  
  
>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})  
>>> df  
   A B  
0 1 3  
1 2 4  
2 3 5  
3 4 6  
  
>>> df.add\_prefix("col\_")  
     col\_A col\_B  
0 1 3  
1 2 4  
2 3 5  
3 4 6

add\_suffix(self, suffix: str, axis: Axis | None = None) -> Self  
    Suffix labels with string `suffix`.

For Series, the row labels are suffixed.  
For DataFrame, the column labels are suffixed.

#### Parameters

-----  
suffix : str  
    The string to add after each label.  
axis : {0 or 'index', 1 or 'columns', None}, default None  
    Axis to add suffix on  
  
    .. versionadded:: 2.0.0

#### Returns

-----  
Series or DataFrame  
    New Series or DataFrame with updated labels.

#### See Also

-----  
Series.add\_prefix: Prefix row labels with string `prefix`.  
DataFrame.add\_prefix: Prefix column labels with string `prefix`.

## Examples

```
>>> s = pd.Series([1, 2, 3, 4])
>>> s
0 1
1 2
2 3
3 4
dtype: int64

>>> s.add_suffix("_item")
0_item 1
1_item 2
2_item 3
3_item 4
dtype: int64

>>> df = pd.DataFrame({"A": [1, 2, 3, 4], "B": [3, 4, 5, 6]})
>>> df
 A B
0 1 3
1 2 4
2 3 5
3 4 6

>>> df.add_suffix("_col")
 A_col B_col
0 1 3
1 2 4
2 3 5
3 4 6
```

```
align(
 self,
 other: NDFrameT,
 join: AlignJoin = 'outer',
 axis: Axis | None = None,
 level: Level | None = None,
 copy: bool | lib.NoDefault = <no_default>,
 fill_value: Hashable | None = None
) -> tuple[Self, NDFrameT]
```

Align two objects on their axes with the specified join method.

Join method is specified for each axis Index.

## Parameters

**other** : DataFrame or Series

The object to align with.

**join** : {'outer', 'inner', 'left', 'right'}, default 'outer'

Type of alignment to be performed.

- \* left: use only keys from left frame, preserve key order.
- \* right: use only keys from right frame, preserve key order.
- \* outer: use union of keys from both frames, sort keys lexicographically.
- \* inner: use intersection of keys from both frames, preserve the order of the left keys.

**axis** : allowed axis of the other object, default None

Align on index (0), columns (1), or both (None).

**level** : int or level name, default None

Broadcast across a level, matching Index values on the passed MultiIndex level.

**copy** : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the [user guide on Copy-on-Write](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html) <[https://pandas.pydata.org/docs/dev/user\\_guide/copy\\_on\\_write.html](https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html)>`\_\_ for more details.

fill\_value : scalar, default np.nan  
Value to use for missing values. Defaults to NaN, but can be any "compatible" value.

Returns

-----  
tuple of (Series/DataFrame, type of other)  
Aligned objects.

See Also

-----  
Series.align : Align two objects on their axes with specified join method.  
DataFrame.align : Align two objects on their axes with specified join method.

Examples

-----  
>>> df = pd.DataFrame(  
... [[1, 2, 3, 4], [6, 7, 8, 9]], columns=["D", "B", "E", "A"], index=[1, 2]  
... )  
>>> other = pd.DataFrame(  
... [[10, 20, 30, 40], [60, 70, 80, 90], [600, 700, 800, 900]],  
... columns=["A", "B", "C", "D"],  
... index=[2, 3, 4],  
... )  
>>> df  
 D B E A  
1 1 2 3 4  
2 6 7 8 9  
>>> other  
 A B C D  
2 10 20 30 40  
3 60 70 80 90  
4 600 700 800 900

Align on columns:

>>> left, right = df.align(other, join="outer", axis=1)  
>>> left  
 A B C D E  
1 4 2 NaN 1 3  
2 9 7 NaN 6 8  
>>> right  
 A B C D E  
2 10 20 30 40 NaN  
3 60 70 80 90 NaN  
4 600 700 800 900 NaN

We can also align on the index:

>>> left, right = df.align(other, join="outer", axis=0)  
>>> left  
 D B E A  
1 1.0 2.0 3.0 4.0  
2 6.0 7.0 8.0 9.0  
3 NaN NaN NaN NaN  
4 NaN NaN NaN NaN  
>>> right  
 A B C D  
1 NaN NaN NaN NaN  
2 10.0 20.0 30.0 40.0  
3 60.0 70.0 80.0 90.0  
4 600.0 700.0 800.0 900.0

Finally, the default `axis=None` will align on both index and columns:

>>> left, right = df.align(other, join="outer", axis=None)  
>>> left  
 A B C D E  
1 4.0 2.0 NaN 1.0 3.0  
2 9.0 7.0 NaN 6.0 8.0  
3 NaN NaN NaN NaN NaN  
4 NaN NaN NaN NaN NaN  
>>> right  
 A B C D E  
1 NaN NaN NaN NaN NaN  
2 10.0 20.0 30.0 40.0 NaN  
3 60.0 70.0 80.0 90.0 NaN

```
4 600.0 700.0 800.0 900.0 NaN
```

```
asfreq(
 self,
 freq: Frequency,
 method: FillnaOptions | None = None,
 how: Literal['start', 'end'] | None = None,
 normalize: bool = False,
 fill_value: Hashable | None = None
) -> Self
```

Convert time series to specified frequency.

Returns the original data conformed to a new index with the specified frequency.

If the index of this Series/DataFrame is a `:class:`~pandas.PeriodIndex``, the new index is the result of transforming the original index with `:meth:`~PeriodIndex.asfreq`` `<pandas.PeriodIndex.asfreq>` (so the original index will map one-to-one to the new index).

Otherwise, the new index will be equivalent to ``pd.date_range(start, end, freq=freq)`` where ``start`` and ``end`` are, respectively, the min and max entries in the original index (see `:func:`~pandas.date_range``). The values corresponding to any timesteps in the new index which were not present in the original index will be null (``NaN``), unless a method for filling such unknowns is provided (see the ``method`` parameter below).

The `:meth:`~resample`` method is more appropriate if an operation on each group of timesteps (such as an aggregate) is necessary to represent the data at the new frequency.

#### Parameters

`freq` : DateOffset or str

Frequency DateOffset or string.

`method` : `{{'backfill'/'bfill', 'pad'/'ffill'}}`, default None

Method to use for filling holes in reindexed Series (note this does not fill NaNs that already were present):

- \* `'pad' / 'ffill'`: propagate last valid observation forward to next valid based on the order of the index

- \* `'backfill' / 'bfill'`: use NEXT valid observation to fill.

`how` : `{{'start', 'end'}}`, default end

For PeriodIndex only (see PeriodIndex.asfreq).

`normalize` : bool, default False

Whether to reset output index to midnight.

`fill_value` : scalar, optional

Value to use for missing values, applied during upsampling (note this does not fill NaNs that already were present).

#### Returns

Series/DataFrame

Series/DataFrame object reindexed to the specified frequency.

#### See Also

`reindex` : Conform DataFrame to new index with optional filling logic.

#### Notes

To learn more about the frequency strings, please see `:ref:`this link<timeseries.offset_aliases>``.

#### Examples

Start by creating a series with 4 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=4, freq="min")
>>> series = pd.Series([0.0, None, 2.0, 3.0], index=index)
>>> df = pd.DataFrame({"s": series})
>>> df
```

```
 s
2000-01-01 00:00:00 0.0
2000-01-01 00:01:00 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:03:00 3.0
```



Upsample the series into 30 second bins.

```
>>> df.asfreq(freq="30s")
 S
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 NaN
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 NaN
2000-01-01 00:03:00 3.0
```

Upsample again, providing a ``fill value``.

```
>>> df.asfreq(freq="30s", fill_value=9.0)
 S
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 9.0
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 9.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 9.0
2000-01-01 00:03:00 3.0
```

Upsample again, providing a ``method``.

```
>>> df.asfreq(freq="30s", method="bfill")
 S
2000-01-01 00:00:00 0.0
2000-01-01 00:00:30 NaN
2000-01-01 00:01:00 NaN
2000-01-01 00:01:30 2.0
2000-01-01 00:02:00 2.0
2000-01-01 00:02:30 3.0
2000-01-01 00:03:00 3.0
```

`asof(self, where, subset=None)`

Return the last row(s) without any NaNs before `where`.

The last row (for each element in `where`, if list) without any NaN is taken.

In case of a :class:`~pandas.DataFrame`, the last row without NaN considering only the subset of columns (if not `None`)

If there is no good value, NaN is returned for a Series or a Series of NaN values for a DataFrame

Parameters

-----

where : date or array-like of dates

        Date(s) before which the last row(s) are returned.

subset : str or array-like of str, default `None`

        For DataFrame, if not `None`, only use these columns to check for NaNs.

Returns

-----

scalar, Series, or DataFrame

The return can be:

- \* scalar : when `self` is a Series and `where` is a scalar
- \* Series: when `self` is a Series and `where` is an array-like, or when `self` is a DataFrame and `where` is a scalar
- \* DataFrame : when `self` is a DataFrame and `where` is an array-like

See Also

-----

`merge_asof` : Perform an asof merge. Similar to left join.

Notes

-----

Dates are assumed to be sorted. Raises if this is not the case.

Examples

-----  
A Series and a scalar `where`.

```
>>> s = pd.Series([1, 2, np.nan, 4], index=[10, 20, 30, 40])
```

```
>>> s
10 1.0
20 2.0
30 NaN
40 4.0
dtype: float64
```

```
>>> s.asof(20)
np.float64(2.0)
```

For a sequence `where`, a Series is returned. The first value is NaN, because the first element of `where` is before the first index value.

```
>>> s.asof([5, 20])
5 NaN
20 2.0
dtype: float64
```

Missing values are not considered. The following is ``2.0``, not NaN, even though NaN is at the index location for ``30``.

```
>>> s.asof(30)
np.float64(2.0)
```

Take all columns into consideration

```
>>> df = pd.DataFrame(
... {
... "a": [10.0, 20.0, 30.0, 40.0, 50.0],
... "b": [None, None, None, None, 500],
... },
... index=pd.DatetimeIndex(
... [
... "2018-02-27 09:01:00",
... "2018-02-27 09:02:00",
... "2018-02-27 09:03:00",
... "2018-02-27 09:04:00",
... "2018-02-27 09:05:00",
...]
...),
...)
>>> df.asof(pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]))
 a b
2018-02-27 09:03:30 NaN NaN
2018-02-27 09:04:30 NaN NaN
```

Take a single column into consideration

```
>>> df.asof(
... pd.DatetimeIndex(["2018-02-27 09:03:30", "2018-02-27 09:04:30"]),
... subset=["a"],
...)
 a b
2018-02-27 09:03:30 30.0 NaN
2018-02-27 09:04:30 40.0 NaN
```

```
astype(
 self,
 dtype,
 copy: bool | lib.NoDefault = <no_default>,
 errors: IgnoreRaise = 'raise'
) -> Self
Cast a pandas object to a specified dtype ``dtype``.
```

This method allows the conversion of the data types of pandas objects, including DataFrames and Series, to the specified dtype. It supports casting entire objects to a single data type or applying different data types to individual columns using a mapping.

Parameters

-----  
dtype : str, data type, Series or Mapping of column name -> data type

Use a str, numpy.dtype, pandas.ExtensionDtype or Python type to cast entire pandas object to the same type. Alternatively, use a mapping, e.g. {col: dtype, ...}, where col is a column label and dtype is a numpy.dtype or Python type to cast one or more of the DataFrame's columns to column-specific types.

copy : bool, default False  
This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`\_ for more details.

errors : {'raise', 'ignore'}, default 'raise'  
Control raising of exceptions on invalid data for provided dtype.

- ``raise`` : allow exceptions to be raised
- ``ignore`` : suppress exceptions. On error return original object.

Returns

-----

same type as caller  
The pandas object casted to the specified ``dtype``.

See Also

-----

to\_datetime : Convert argument to datetime.  
to\_timedelta : Convert argument to timedelta.  
to\_numeric : Convert argument to a numeric type.  
numpy.ndarray.astype : Cast a numpy array to a specified type.

Notes

-----

.. versionchanged:: 2.0.0

Using ``astype`` to convert from timezone-naive dtype to timezone-aware dtype will raise an exception.  
Use :meth:`Series.dt.tz\_localize` instead.

Examples

-----

Create a DataFrame:

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df.dtypes
col1 int64
col2 int64
dtype: object
```

Cast all columns to int32:

```
>>> df.astype("int32").dtypes
col1 int32
col2 int32
dtype: object
```

Cast col1 to int32 using a dictionary:

```
>>> df.astype({"col1": "int32"}).dtypes
col1 int32
col2 int64
dtype: object
```

Create a series:

```
>>> ser = pd.Series([1, 2], dtype="int32")
>>> ser
0 1
1 2
dtype: int32
>>> ser.astype("int64")
```

```
0 1
1 2
dtype: int64
```

Convert to categorical type:

```
>>> ser.astype("category")
0 1
1 2
dtype: category
Categories (2, int32): [1, 2]
```

Convert to ordered categorical type with custom ordering:

```
>>> from pandas.api.types import CategoricalDtype
>>> cat_dtype = CategoricalDtype(categories=[2, 1], ordered=True)
>>> ser.astype(cat_dtype)
0 1
1 2
dtype: category
Categories (2, int64): [2 < 1]
```

Create a series of dates:

```
>>> ser_date = pd.Series(pd.date_range("20200101", periods=3))
>>> ser_date
0 2020-01-01
1 2020-01-02
2 2020-01-03
dtype: datetime64[us]
```

`at_time(self, time, asof: bool = False, axis: Axis | None = None) -> Self`  
Select values at particular time of day (e.g., 9:30AM).

Parameters

time : datetime.time or str  
The values to select.  
asof : bool, default False  
This parameter is currently not supported.  
axis : {0 or 'index', 1 or 'columns'}, default 0  
For `Series` this parameter is unused and defaults to 0.

Returns

Series or DataFrame  
The values with the specified time.

Raises

TypeError  
If the index is not a :class:`DatetimeIndex`

See Also

between\_time : Select values between particular times of the day.  
first : Select initial periods of time series based on a date offset.  
last : Select final periods of time series based on a date offset.  
DatetimeIndex.indexer\_at\_time : Get just the index locations for values at particular time of the day.

Examples

```
>>> i = pd.date_range("2018-04-09", periods=4, freq="12h")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts
 A
2018-04-09 00:00:00 1
2018-04-09 12:00:00 2
2018-04-10 00:00:00 3
2018-04-10 12:00:00 4

>>> ts.at_time("12:00")
 A
2018-04-09 12:00:00 2
2018-04-10 12:00:00 4
```

```

between_time(
 self,
 start_time,
 end_time,
 inclusive: IntervalClosedType = 'both',
 axis: Axis | None = None
) -> Self
 Select values between particular times of the day (e.g., 9:00-9:30 AM).

 By setting ``start_time`` to be later than ``end_time``,
 you can get the times that are *not* between the two times.

Parameters

start_time : datetime.time or str
 Initial time as a time filter limit.
end_time : datetime.time or str
 End time as a time filter limit.
inclusive : {"both", "neither", "left", "right"}, default "both"
 Include boundaries; whether to set each bound as closed or open.
axis : {0 or 'index', 1 or 'columns'}, default 0
 Determine range time on index or columns value.
 For `Series` this parameter is unused and defaults to 0.

Returns

Series or DataFrame
 Data from the original object filtered to the specified dates range.

Raises

TypeError
 If the index is not a :class:`DatetimeIndex`

See Also

at_time : Select values at a particular time of the day.
first : Select initial periods of time series based on a date offset.
last : Select final periods of time series based on a date offset.
DatetimeIndex.indexer_between_time : Get just the index locations for
 values between particular times of the day.

Examples

>>> i = pd.date_range("2018-04-09", periods=4, freq="1D20min")
>>> ts = pd.DataFrame({"A": [1, 2, 3, 4]}, index=i)
>>> ts
 A
2018-04-09 00:00:00 1
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3
2018-04-12 01:00:00 4

>>> ts.between_time("0:15", "0:45")
 A
2018-04-10 00:20:00 2
2018-04-11 00:40:00 3

You get the times that are *not* between two times by setting
``start_time`` later than ``end_time``:

>>> ts.between_time("0:45", "0:15")
 A
2018-04-09 00:00:00 1
2018-04-12 01:00:00 4

bfill(
 self,
 *,
 axis: None | Axis = None,
 inplace: bool = False,
 limit: None | int = None,
 limit_area: Literal['inside', 'outside'] | None = None
) -> Self
 Fill NA/NaN values by using the next valid observation to fill the gap.

 This method fills missing values in a backward direction along the

```

specified axis, propagating non-null values from later positions to earlier positions containing NaN.

#### Parameters

`axis` : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.

`inplace` : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

`limit` : int, default None  
If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

`limit_area` : {'None', 'inside', 'outside'}, default None  
If limit is specified, consecutive NaNs will be filled with this restriction.

- \* ``None``: No fill restriction.
- \* 'inside': Only fill NaNs surrounded by valid values (interpolate).
- \* 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

#### Returns

Series/DataFrame  
Object with missing values filled.

#### See Also

`DataFrame.ffill` : Fill NA/NAN values by propagating the last valid observation to next valid.

#### Examples

For Series:

```
>>> s = pd.Series([1, None, None, 2])
>>> s.bfill()
0 1.0
1 2.0
2 2.0
3 2.0
dtype: float64
>>> s.bfill(limit=1)
0 1.0
1 NaN
2 2.0
3 2.0
dtype: float64
```

With DataFrame:

```
>>> df = pd.DataFrame({"A": [1, None, None, 4], "B": [None, 5, None, 7]})
>>> df
 A B
0 1.0 NaN
1 NaN 5.0
2 NaN NaN
3 4.0 7.0
>>> df.bfill()
 A B
0 1.0 5.0
1 4.0 5.0
2 4.0 7.0
3 4.0 7.0
>>> df.bfill(limit=1)
 A B
0 1.0 5.0
1 NaN 5.0
```

```

2 4.0 7.0
3 4.0 7.0

```

```

clip(
 self,
 lower=None,
 upper=None,
 *,
 axis: Axis | None = None,
 inplace: bool = False,
 **kwargs
) -> Self
 Trim values at input threshold(s).

```

Assigns values outside boundary to boundary values. Thresholds can be singular values or array like, and in the latter case the clipping is performed element-wise in the specified axis.

#### Parameters

```

lower : float or array-like, default None
 Minimum threshold value. All values below this
 threshold will be set to it. A missing
 threshold (e.g. `NA`) will not clip the value.
upper : float or array-like, default None
 Maximum threshold value. All values above this
 threshold will be set to it. A missing
 threshold (e.g. `NA`) will not clip the value.
axis : {{0 or 'index', 1 or 'columns', None}}, default None
 Align object with lower and upper along the given axis.
 For `Series` this parameter is unused and defaults to `None`.
inplace : bool, default False
 Whether to perform the operation in place on the data.
**kwargs
 Additional keywords have no effect but might be accepted
 for compatibility with numpy.

```

#### Returns

```

Series or DataFrame
 Same type as calling object with the values outside the
 clip boundaries replaced.

```

#### See Also

```

Series.clip : Trim values at input threshold in series.
DataFrame.clip : Trim values at input threshold in DataFrame.
numpy.clip : Clip (limit) the values in an array.

```

#### Examples

```

>>> data = {"col_0": [9, -3, 0, -1, 5], "col_1": [-2, -7, 6, 8, -5]}
>>> df = pd.DataFrame(data)
>>> df
 col_0 col_1
0 9 -2
1 -3 -7
2 0 6
3 -1 8
4 5 -5

```

Clips per column using lower and upper thresholds:

```

>>> df.clip(-4, 6)
 col_0 col_1
0 6 -2
1 -3 -4
2 0 6
3 -1 6
4 5 -4

```

Clips using specific lower and upper thresholds per column:

```

>>> df.clip([-2, -1], [4, 5])
 col_0 col_1
0 4 -1
1 -2 -1

```

```

2 0 5
3 -1 5
4 4 -1

```

Clips using specific lower and upper thresholds per column element:

```

>>> t = pd.Series([2, -4, -1, 6, 3])
>>> t
0 2
1 -4
2 -1
3 6
4 3
dtype: int64

```

```

>>> df.clip(t, t + 4, axis=0)
 col_0 col_1
0 6 2
1 -3 -4
2 0 3
3 6 8
4 5 3

```

Clips using specific lower threshold per column element, with missing values:

```

>>> t = pd.Series([2, -4, np.nan, 6, 3])
>>> t
0 2.0
1 -4.0
2 NaN
3 6.0
4 3.0
dtype: float64

```

```

>>> df.clip(t, axis=0)
 col_0 col_1
0 9.0 2.0
1 -3.0 -4.0
2 0.0 6.0
3 6.0 8.0
4 5.0 3.0

```

```

convert_dtypes(
 self,
 infer_objects: bool = True,
 convert_string: bool = True,
 convert_integer: bool = True,
 convert_boolean: bool = True,
 convert_floating: bool = True,
 dtype_backend: DtypeBackend = 'numpy_nullable'
) -> Self
 Convert columns from numpy dtypes to the best dtypes that support ``pd.NA``.

Parameters

infer_objects : bool, default True
 Whether object dtypes should be converted to the best possible types.
convert_string : bool, default True
 Whether object dtypes should be converted to ``StringDtype()``.
convert_integer : bool, default True
 Whether, if possible, conversion can be done to integer extension types.
convert_boolean : bool, defaults True
 Whether object dtypes should be converted to ``BooleanDtypes()``.
convert_floating : bool, defaults True
 Whether, if possible, conversion can be done to floating extension types.
 If ``convert_integer`` is also True, preference will be give to integer
 dtypes if the floats can be faithfully casted to integers.
dtype_backend : {'numpy_nullable', 'pyarrow'}, default 'numpy_nullable'
 Back-end data type applied to the resultant :class:`DataFrame` or
 :class:`Series` (still experimental). Behaviour is as follows:

 * ``"numpy_nullable"``: returns nullable-dtype-backed
 :class:`DataFrame` or :class:`Series`.
 * ``"pyarrow"``: returns pyarrow-backed nullable :class:`ArrowDtype`
 :class:`DataFrame` or :class:`Series`.

.. versionadded:: 2.0

```



## Returns

Series or DataFrame  
Copy of input object with new dtype.

## See Also

`infer_objects` : Infer dtypes of objects.  
`to_datetime` : Convert argument to datetime.  
`to_timedelta` : Convert argument to timedelta.  
`to_numeric` : Convert argument to a numeric type.

## Notes

By default, `convert_dtypes` will attempt to convert a Series (or each Series in a DataFrame) to dtypes that support `pd.NA`. By using the options `convert_string`, `convert_integer`, `convert_boolean` and `convert_floating`, it is possible to turn off individual conversions to `StringDtype`, the integer extension types, `BooleanDtype` or floating extension types, respectively.

For object-dtyped columns, if `infer_objects` is `True`, use the inference rules as during normal Series/DataFrame construction. Then, if possible, convert to `StringDtype`, `BooleanDtype` or an appropriate integer or floating extension type, otherwise leave as `object`.

If the dtype is integer, convert to an appropriate integer extension type.

If the dtype is numeric, and consists of all integers, convert to an appropriate integer extension type. Otherwise, convert to an appropriate floating extension type.

In the future, as new dtypes are added that support `pd.NA`, the results of this method will change to support those new dtypes.

## Examples

```
>>> df = pd.DataFrame(
... {
... "a": pd.Series([1, 2, 3], dtype=np.dtype("int32")),
... "b": pd.Series(["x", "y", "z"], dtype=np.dtype("O")),
... "c": pd.Series([True, False, np.nan], dtype=np.dtype("O")),
... "d": pd.Series(["h", "i", np.nan], dtype=np.dtype("O")),
... "e": pd.Series([10, np.nan, 20], dtype=np.dtype("float")),
... "f": pd.Series([np.nan, 100.5, 200], dtype=np.dtype("float")),
... }
...)
```

Start with a DataFrame with default dtypes.

```
>>> df
 a b c d e f
0 1 x True h 10.0 NaN
1 2 y False i NaN 100.5
2 3 z NaN NaN 20.0 200.0
```

```
>>> df.dtypes
a int32
b object
c object
d object
e float64
f float64
dtype: object
```

Convert the DataFrame to use best possible dtypes.

```
>>> dfn = df.convert_dtypes()
>>> dfn
 a b c d e f
0 1 x True h 10 <NA>
1 2 y False i <NA> 100.5
2 3 z <NA> <NA> 20 200.0
```

```
>>> dfn.dtypes
a Int32
```

```

b string
c boolean
d string
e Int64
f Float64
dtype: object

```

Start with a Series of strings and missing data represented by ``np.nan``.

```

>>> s = pd.Series(["a", "b", np.nan])
>>> s
0 a
1 b
2 NaN
dtype: str

```

Obtain a Series with dtype ``StringDtype``.

```

>>> s.convert_dtypes()
0 a
1 b
2 <NA>
dtype: string

```

`copy(self, deep: bool = True) -> Self`  
 Make a copy of this object's indices and data.

When ``deep=True`` (default), a new object will be created with a copy of the calling object's data and indices. Modifications to the data or indices of the copy will not be reflected in the original object (see notes below).

When ``deep=False``, a new object will be created without copying the calling object's data or index (only references to the data and index are copied). With Copy-on-Write, changes to the original will *not* be reflected in the shallow copy (and vice versa). The shallow copy uses a lazy (deferred) copy mechanism that copies the data only when any changes to the original or shallow copy are made, ensuring memory efficiency while maintaining data integrity.

.. note::  
 In pandas versions prior to 3.0, the default behavior without Copy-on-Write was different: changes to the original *were* reflected in the shallow copy (and vice versa). See the :ref:`Copy-on-Write user guide <copy\_on\_write>` for more information.

#### Parameters

`deep` : bool, default True  
 Make a deep copy, including a copy of the data and the indices. With ``deep=False`` neither the indices nor the data are copied.

#### Returns

Series or DataFrame  
 Object type matches caller.

#### See Also

`copy.copy` : Return a shallow copy of an object.  
`copy.deepcopy` : Return a deep copy of an object.

#### Notes

When ``deep=True``, data is copied but actual Python objects will not be copied recursively, only the reference to the object. This is in contrast to `copy.deepcopy` in the Standard Library, which recursively copies object data (see examples below).

While ``Index`` objects are copied when ``deep=True``, the underlying numpy array is not copied for performance reasons. Since ``Index`` is immutable, the underlying data can be safely shared and a copy is not needed.

Since pandas is not thread safe, see the :ref:`gotchas <gotchas.thread-safety>` when copying in a threading environment.

Copy-on-Write protects shallow copies against accidental modifications. This means that any changes to the copied data would make a new copy of the data upon write (and vice versa). Changes made to either the original or copied variable would not be reflected in the counterpart. See :ref:`Copy\_on\_Write <copy\_on\_write>` for more information.

#### Examples

```

>>> s = pd.Series([1, 2], index=["a", "b"])
>>> s
a 1
b 2
dtype: int64
```

```
>>> s_copy = s.copy(deep=True)
>>> s_copy
a 1
b 2
dtype: int64
```

Due to Copy-on-Write, shallow copies still protect data modifications. Note shallow does not get modified below.

```
>>> s = pd.Series([1, 2], index=["a", "b"])
>>> shallow = s.copy(deep=False)
>>> s.iloc[1] = 200
>>> shallow
a 1
b 2
dtype: int64
```

When the data has object dtype, even a deep copy does not copy the underlying Python objects. Updating a nested data object will be reflected in the deep copy.

```
>>> s = pd.Series([[1, 2], [3, 4]])
>>> deep = s.copy()
>>> s[0][0] = 10
>>> s
0 [10, 2]
1 [3, 4]
dtype: object
>>> deep
0 [10, 2]
1 [3, 4]
dtype: object
```

`describe(self, percentiles=None, include=None, exclude=None) -> Self`  
Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding ``NaN`` values.

Analyzes both numeric and object series, as well as ``DataFrame`` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

#### Parameters

```

percentiles : list-like of numbers, optional
 The percentiles to include in the output. All should fall between 0 and 1. The default, ``None``, will automatically return the 25th, 50th, and 75th percentiles.
include : 'all', list-like of dtypes or None (default), optional
 A white list of data types to include in the result. Ignored for ``Series``. Here are the options:

 - 'all' : All columns of the input will be included in the output.
 - A list-like of dtypes : Limits the results to the provided data types.
 To limit the result to numeric types submit ``numpy.number``. To limit it instead to object columns submit the ``numpy.object`` data type. Strings can also be used in the style of
```

```

 ``select_dtypes`` (e.g. ``df.describe(include=['O'])``). To
 select pandas categorical columns, use ``'category'``.
- None (default) : The result will include all numeric columns.
exclude : list-like of dtypes or None (default), optional,
A black list of data types to omit from the result. Ignored
for ``Series``. Here are the options:

- A list-like of dtypes : Excludes the provided data types
from the result. To exclude numeric types submit
``numpy.number``. To exclude object columns submit the data
type ``numpy.object``. Strings can also be used in the style of
``select_dtypes`` (e.g. ``df.describe(exclude=['O'])``). To
exclude pandas categorical columns, use ``'category'``.
- None (default) : The result will exclude nothing.

```

#### Returns

```

Series or DataFrame
Summary statistics of the Series or Dataframe provided.

```

#### See Also

```

DataFrame.count: Count number of non-NA/null observations.
DataFrame.max: Maximum of the values in the object.
DataFrame.min: Minimum of the values in the object.
DataFrame.mean: Mean of the values.
DataFrame.std: Standard deviation of the observations.
DataFrame.select_dtypes: Subset of a DataFrame including/excluding
columns based on their dtype.

```

#### Notes

```

For numeric data, the result's index will include ``count``,
``mean``, ``std``, ``min``, ``max`` as well as lower, ``50`` and
upper percentiles. By default the lower percentile is ``25`` and the
upper percentile is ``75``. The ``50`` percentile is the
same as the median.

For object data (e.g. strings), the result's index
will include ``count``, ``unique``, ``top``, and ``freq``. The ``top``
is the most common value. The ``freq`` is the most common value's
frequency.

If multiple object values have the highest count, then the
``count`` and ``top`` results will be arbitrarily chosen from
among those with the highest count.

For mixed data types provided via a ``DataFrame``, the default is to
return only an analysis of numeric columns. If the DataFrame consists
only of object and categorical data without any numeric columns, the
default is to return an analysis of both the object and categorical
columns. If ``include='all'`` is provided as an option, the result
will include a union of attributes of each type.

The ``include`` and ``exclude`` parameters can be used to limit
which columns in a ``DataFrame`` are analyzed for the output.
The parameters are ignored when analyzing a ``Series``.

```

#### Examples

```

Describing a numeric ``Series``.

```

```

>>> s = pd.Series([1, 2, 3])
>>> s.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
dtype: float64

```

```

Describing a categorical ``Series``.

```

```

>>> s = pd.Series(["a", "a", "b", "c"])

```

```
>>> s.describe()
count 4
unique 3
top a
freq 2
dtype: object
```

Describing a timestamp ``Series``.

```
>>> s = pd.Series(
... [
... np.datetime64("2000-01-01"),
... np.datetime64("2010-01-01"),
... np.datetime64("2010-01-01"),
...]
...)
>>> s.describe()
count 3
mean 2006-09-01 08:00:00
min 2000-01-01 00:00:00
25% 2004-12-31 12:00:00
50% 2010-01-01 00:00:00
75% 2010-01-01 00:00:00
max 2010-01-01 00:00:00
dtype: object
```

Describing a ``DataFrame``. By default only numeric fields are returned.

```
>>> df = pd.DataFrame(
... {
... "categorical": pd.Categorical(["d", "e", "f"]),
... "numeric": [1, 2, 3],
... "object": ["a", "b", "c"],
... }
...)
>>> df.describe()
numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Describing all columns of a ``DataFrame`` regardless of data type.

```
>>> df.describe(include="all") # doctest: +SKIP
categorical numeric object
count 3 3.0 3
unique 3 NaN 3
top f NaN a
freq 1 NaN 1
mean NaN 2.0 NaN
std NaN 1.0 NaN
min NaN 1.0 NaN
25% NaN 1.5 NaN
50% NaN 2.0 NaN
75% NaN 2.5 NaN
max NaN 3.0 NaN
```

Describing a column from a ``DataFrame`` by accessing it as an attribute.

```
>>> df.numeric.describe()
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
Name: numeric, dtype: float64
```

Including only numeric columns in a ``DataFrame`` description.

```
>>> df.describe(include=[np.number])
numeric
count 3.0
mean 2.0
std 1.0
min 1.0
25% 1.5
50% 2.0
75% 2.5
max 3.0
```

Including only string columns in a ``DataFrame`` description.

```
>>> df.describe(include=[object]) # doctest: +SKIP
object
count 3
unique 3
top a
freq 1
```

Including only categorical columns from a ``DataFrame`` description.

```
>>> df.describe(include=["category"])
category
count 3
unique 3
top d
freq 1
```

Excluding numeric columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[np.number]) # doctest: +SKIP
category object
count 3 3
unique 3 3
top f a
freq 1 1
```

Excluding object columns from a ``DataFrame`` description.

```
>>> df.describe(exclude=[object]) # doctest: +SKIP
category numeric
count 3 3.0
unique 3 NaN
top f NaN
freq 1 NaN
mean NaN 2.0
std NaN 1.0
min NaN 1.0
25% NaN 1.5
50% NaN 2.0
75% NaN 2.5
max NaN 3.0
```

`droplevel(self, level: IndexLabel, axis: Axis = 0) -> Self`  
Return Series/DataFrame with requested index / column level(s) removed.

Parameters

-----

`level` : int, str, or list-like

If a string is given, must be the name of a level

If list-like, elements must be names or positional indexes of levels.

`axis` : {{0 or 'index', 1 or 'columns'}}, default 0

Axis along which the level(s) is removed:

\* 0 or 'index': remove level(s) in column.

\* 1 or 'columns': remove level(s) in row.

For ``Series`` this parameter is unused and defaults to 0.

Returns

-----

Series/DataFrame

Series/DataFrame with requested index / column level(s) removed.

See Also

-----  
DataFrame.replace : Replace values given in `to\_replace` with `value`.  
DataFrame.pivot : Return reshaped DataFrame organized by given  
index / column values.

Examples

-----  
>>> df = (  
... pd.DataFrame([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])  
... .set\_index([0, 1])  
... .rename\_axis(["a", "b"])  
... )

>>> df.columns = pd.MultiIndex.from\_tuples(  
... [("c", "e"), ("d", "f")], names=["level\_1", "level\_2"]  
... )

>>> df  
level\_1 c d  
level\_2 e f  
a b  
1 2 3 4  
5 6 7 8  
9 10 11 12

>>> df.droplevel("a")  
level\_1 c d  
level\_2 e f  
b  
2 3 4  
6 7 8  
10 11 12

>>> df.droplevel("level\_2", axis=1)  
level\_1 c d  
a b  
1 2 3 4  
5 6 7 8  
9 10 11 12

equals(self, other: object) -> bool

Test whether two objects contain the same elements.

This function allows two Series or DataFrames to be compared against each other to see if they have the same shape and elements. NaNs in the same location are considered equal.

The row/column index do not need to have the same type, as long as the values are considered equal. Corresponding columns and index must be of the same dtype.

Parameters

-----  
other : Series or DataFrame

The other Series or DataFrame to be compared with the first.

Returns

-----  
bool

True if all elements are the same in both objects, False otherwise.

See Also

-----  
Series.eq : Compare two Series objects of the same length  
and return a Series where each element is True if the element  
in each Series is equal, False otherwise.  
DataFrame.eq : Compare two DataFrame objects of the same shape and  
return a DataFrame where each element is True if the respective  
element in each DataFrame is equal, False otherwise.  
testing.assert\_series\_equal : Raises an AssertionError if left and  
right are not equal. Provides an easy interface to ignore  
inequality in dtypes, indexes and precision among others.  
testing.assert\_frame\_equal : Like assert\_series\_equal, but targets

DataFrames.  
numpy.array\_equal : Return True if two arrays have the same shape and elements, False otherwise.

#### Examples

```
>>> df = pd.DataFrame({'1': [10], '2': [20]})
>>> df
 1 2
0 10 20
```

DataFrames df and exactly\_equal have the same types and values for their elements and column labels, which will return True.

```
>>> exactly_equal = pd.DataFrame({'1': [10], '2': [20]})
>>> exactly_equal
 1 2
0 10 20
>>> df.equals(exactly_equal)
True
```

DataFrames df and different\_column\_type have the same element types and values, but have different types for the column labels, which will still return True.

```
>>> different_column_type = pd.DataFrame({'1.0': [10], '2.0': [20]})
>>> different_column_type
 1.0 2.0
0 10 20
>>> df.equals(different_column_type)
True
```

DataFrames df and different\_data\_type have different types for the same values for their elements, and will return False even though their column labels are the same values and types.

```
>>> different_data_type = pd.DataFrame({'1': [10.0], '2': [20.0]})
>>> different_data_type
 1 2
0 10.0 20.0
>>> df.equals(different_data_type)
False
```

DataFrames with NaN in the same locations compare equal.

```
>>> df_nan1 = pd.DataFrame({'a': [1, np.nan], 'b': [3, np.nan]})
>>> df_nan2 = pd.DataFrame({'a': [1, np.nan], 'b': [3, np.nan]})
>>> df_nan1.equals(df_nan2)
True
```

If the NaN values are not in the same locations, they compare unequal.

```
>>> df_nan3 = pd.DataFrame({'a': [1, np.nan], 'b': [3, 4]})
>>> df_nan1.equals(df_nan3)
False
```

```
ewm(
 self,
 com: float | None = None,
 span: float | None = None,
 halflife: float | TimedeltaConvertibleTypes | None = None,
 alpha: float | None = None,
 min_periods: int | None = 0,
 adjust: bool = True,
 ignore_na: bool = False,
 times: np.ndarray | DataFrame | Series | None = None,
 method: Literal['single', 'table'] = 'single'
) -> ExponentialMovingWindow
Provide exponentially weighted (EW) calculations.
```

Exactly one of ``com``, ``span``, ``halflife``, or ``alpha`` must be provided if ``times`` is not provided. If ``times`` is provided and ``adjust=True``, ``halflife`` and one of ``com``, ``span`` or ``alpha`` may be provided. If ``times`` is provided and ``adjust=False``, ``halflife`` must be the only provided decay-specification parameter.

#### Parameters



```

com : float, optional
 Specify decay in terms of center of mass

 :math:\alpha = 1 / (1 + com), for :math:com \geq 0.

span : float, optional
 Specify decay in terms of span

 :math:\alpha = 2 / (span + 1), for :math:span \geq 1.

halflife : float, str, timedelta, optional
 Specify decay in terms of half-life

 :math:\alpha = 1 - \exp(-\ln(2) / halflife), for
 :math:halflife > 0.

 If ``times`` is specified, a timedelta convertible unit over which an
 observation decays to half its value. Only applicable to ``mean()``,
 and halflife value will not apply to the other functions.

alpha : float, optional
 Specify smoothing factor :math:\alpha directly

 :math:0 < \alpha \leq 1.

min_periods : int, default 0
 Minimum number of observations in window required to have a value;
 otherwise, result is ``np.nan``.

adjust : bool, default True
 Divide by decaying adjustment factor in beginning periods to account
 for imbalance in relative weightings (viewing EWMA as a moving average).

 - When ``adjust=True`` (default), the EW function is calculated using weights
 :math:w_i = (1 - \alpha)^i. For example, the EW moving average of the series
 [:math:x_0, x_1, \dots, x_t] would be:

 .. math::
 y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots + (1 - \alpha)^t x_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}

 - When ``adjust=False``, the exponentially weighted function is calculated
 recursively:

 .. math::
 \begin{split}
 y_0 &= x_0 \\
 y_t &= (1 - \alpha) y_{t-1} + \alpha x_t,
 \end{split}

ignore_na : bool, default False
 Ignore missing values when calculating weights.

 - When ``ignore_na=False`` (default), weights are based on absolute positions.
 For example, the weights of :math:x_0 and :math:x_2 used in calculating
 the final weighted average of [:math>x_0, \text{None}, x_2] are
 :math:(1 - \alpha)^2 and :math:1 if ``adjust=True``, and
 :math:(1 - \alpha)^2 and :math:\alpha if ``adjust=False``.

 - When ``ignore_na=True``, weights are based
 on relative positions. For example, the weights of :math>x_0 and :math>x_2
 used in calculating the final weighted average of
 [:math>x_0, \text{None}, x_2] are :math:1 - \alpha and :math:1 if
 ``adjust=True``, and :math:1 - \alpha and :math:\alpha if ``adjust=False``.

times : np.ndarray, Series, default None

 Only applicable to ``mean()``.

 Times corresponding to the observations. Must be monotonically increasing and
 ``datetime64[ns]`` dtype.

 If 1-D array like, a sequence with the same shape as the observations.

method : str {'single', 'table'}, default 'single'
 Execute the rolling operation per single column or row (``'single'``)
 or over the entire object (``'table'``).

```

This argument is only implemented when specifying ``engine='numba'`` in the method call.

Only applicable to ``mean()``

#### Returns

-----  
pandas.api.typing.ExponentialMovingWindow

An instance of ExponentialMovingWindow for further exponentially weighted (EW) calculations, e.g. using the ``mean`` method.

#### See Also

-----  
rolling : Provides rolling window calculations.  
expanding : Provides expanding transformations.

#### Notes

-----  
See :ref:`Windowing Operations <window.exponentially\_weighted>` for further usage details and examples.

#### Examples

-----  
>>> df = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]})

>>> df

```
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0
```

>>> df.ewm(com=0.5).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
```

>>> df.ewm(alpha=2 / 3).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
```

**\*\*adjust\*\***

>>> df.ewm(com=0.5, adjust=True).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.670213
```

>>> df.ewm(com=0.5, adjust=False).mean()

```
 B
0 0.000000
1 0.666667
2 1.555556
3 1.555556
4 3.650794
```

**\*\*ignore\_na\*\***

>>> df.ewm(com=0.5, ignore\_na=True).mean()

```
 B
0 0.000000
1 0.750000
2 1.615385
3 1.615385
4 3.225000
```

>>> df.ewm(com=0.5, ignore\_na=False).mean()

```
 B
0 0.000000
```

```

1 0.750000
2 1.615385
3 1.615385
4 3.670213

```

**\*\*times\*\***

Exponentially weighted mean with weights calculated with a `timedelta` `halflife` relative to `times`.

```

>>> times = ['2020-01-01', '2020-01-03', '2020-01-10', '2020-01-15', '2020-01-17']
>>> df.ewm(halflife='4 days', times=pd.DatetimeIndex(times)).mean()
 B
0 0.000000
1 0.585786
2 1.523889
3 1.523889
4 3.233686

```

```

expanding(
 self,
 min_periods: int = 1,
 method: Literal['single', 'table'] = 'single'
) -> Expanding
 Provide expanding window calculations.

```

An expanding window yields the value of an aggregation statistic with all the data available up to that point in time.

Parameters

```

min_periods : int, default 1
 Minimum number of observations in window required to have a value;
 otherwise, result is np.nan.

method : str {'single', 'table'}, default 'single'
 Execute the rolling operation per single column or row ('single')
 or over the entire object ('table').

 This argument is only implemented when specifying engine='numba'
 in the method call.

```

Returns

```

pandas.api.typing.Expanding
 An instance of Expanding for further expanding window calculations,
 e.g. using the sum method.

```

See Also

```

rolling : Provides rolling window calculations.
ewm : Provides exponential weighted functions.

```

Notes

```

See :ref:`Windowing Operations <window.expanding>` for further usage details
and examples.

```

Examples

```

>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
>>> df
 B
0 0.0
1 1.0
2 2.0
3 NaN
4 4.0

```

**\*\*min\_periods\*\***

Expanding sum with 1 vs 3 observations needed to calculate a value.

```

>>> df.expanding(1).sum()
 B
0 0.0
1 1.0

```

```

2 3.0
3 3.0
4 7.0
>>> df.expanding(3).sum()
 B
0 NaN
1 NaN
2 3.0
3 3.0
4 7.0

```

**ffill()**  
Fill NA/NAN values by propagating the last valid observation to next valid.

**Parameters**  
-----

**axis** : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame  
Axis along which to fill missing values. For `Series` this parameter is unused and defaults to 0.

**inplace** : bool, default False  
If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

**limit** : int, default None  
If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

**limit\_area** : {{None, 'inside', 'outside'}}, default None  
If limit is specified, consecutive NaNs will be filled with this restriction.

- \* ``None``: No fill restriction.
- \* 'inside': Only fill NaNs surrounded by valid values (interpolate).
- \* 'outside': Only fill NaNs outside valid values (extrapolate).

.. versionadded:: 2.2.0

**Returns**  
-----

Series/DataFrame  
Object with missing values filled.

**See Also**  
-----

**DataFrame.bfill** : Fill NA/NAN values by using the next valid observation to fill the gap.

**Examples**  
-----

```

>>> df = pd.DataFrame(
... [
... [np.nan, 2, np.nan, 0],
... [3, 4, np.nan, 1],
... [np.nan, np.nan, np.nan, np.nan],
... [np.nan, 3, np.nan, 4],
...],
... columns=list("ABCD"),
...)
>>> df
 A B C D
0 NaN 2.0 NaN 0.0
1 3.0 4.0 NaN 1.0
2 NaN NaN NaN NaN
3 NaN 3.0 NaN 4.0

```

```

>>> df.ffill()

```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	3.0	4.0	NaN	1.0
3	3.0	3.0	NaN	4.0

```
>>> ser = pd.Series([1, np.nan, 2, 3])
>>> ser.fffll()
0 1.0
1 1.0
2 2.0
3 3.0
dtype: float64
```

```
fillna(
 self,
 value: Hashable | Mapping | Series | DataFrame,
 *,
 axis: Axis | None = None,
 inplace: bool = False,
 limit: int | None = None
) -> Self
Fill NA/NaN values with `value`.

Parameters

value : scalar, dict, Series, or DataFrame
 Value to use to fill holes (e.g. 0), alternately a
 dict/Series/DataFrame of values specifying which value to use for
 each index (for a Series) or column (for a DataFrame). Values not
 in the dict/Series/DataFrame will not be filled. This value cannot
 be a list.
axis : {0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame
 Axis along which to fill missing values. For `Series`
 this parameter is unused and defaults to 0.
inplace : bool, default False
 If True, fill in-place. Note: this will modify any
 other views on this object (e.g., a no-copy slice for a column in a
 DataFrame).
limit : int, default None
 This is the maximum number of entries along the entire axis
 where NaNs will be filled. Must be greater than 0 if not None.

Returns

Series/DataFrame
 Object with missing values filled.

See Also

fffll : Fill values by propagating the last valid observation to next valid.
bffll : Fill values by using the next valid observation to fill the gap.
interpolate : Fill NaN values using interpolation.
reindex : Conform object to new index.
asfreq : Convert TimeSeries to specified frequency.

Notes

For non-object dtype, ``value=None`` will use the NA value of the dtype.
See more details in the :ref:`Filling missing data<missing_data.fillna>`
section.
```

#### Examples

```
>>> df = pd.DataFrame(
... [
... [np.nan, 2, np.nan, 0],
... [3, 4, np.nan, 1],
... [np.nan, np.nan, np.nan, np.nan],
... [np.nan, 3, np.nan, 4],
...],
... columns=list("ABCD"),
...)
>>> df
 A B C D
0 NaN 2.0 NaN 0.0
1 3.0 4.0 NaN 1.0
```

```

2 NaN NaN NaN NaN
3 NaN 3.0 NaN 4.0

```

Replace all NaN elements with 0s.

```

>>> df.fillna(0)
 A B C D
0 0.0 2.0 0.0 0.0
1 3.0 4.0 0.0 1.0
2 0.0 0.0 0.0 0.0
3 0.0 3.0 0.0 4.0

```

Replace all NaN elements in column 'A', 'B', 'C', and 'D', with 0, 1, 2, and 3 respectively.

```

>>> values = {"A": 0, "B": 1, "C": 2, "D": 3}
>>> df.fillna(value=values)
 A B C D
0 0.0 2.0 2.0 0.0
1 3.0 4.0 2.0 1.0
2 0.0 1.0 2.0 3.0
3 0.0 3.0 2.0 4.0

```

Only replace the first NaN element.

```

>>> df.fillna(value=values, limit=1)
 A B C D
0 0.0 2.0 2.0 0.0
1 3.0 4.0 NaN 1.0
2 NaN 1.0 NaN 3.0
3 NaN 3.0 NaN 4.0

```

When filling using a DataFrame, replacement happens along the same column names and same indices

```

>>> df2 = pd.DataFrame(np.zeros((4, 4)), columns=list("ABCE"))
>>> df.fillna(df2)
 A B C D
0 0.0 2.0 0.0 0.0
1 3.0 4.0 0.0 1.0
2 0.0 0.0 0.0 NaN
3 0.0 3.0 0.0 4.0

```

Note that column D is not affected since it is not present in df2.

```

filter(
 self,
 items=None,
 like: str | None = None,
 regex: str | None = None,
 axis: Axis | None = None
) -> Self

```

Subset the DataFrame or Series according to the specified index labels.

For DataFrame, filter rows or columns depending on ``axis`` argument.  
Note that this routine does not filter based on content.  
The filter is applied to the labels of the index.

Parameters

```

items : list-like
 Keep labels from axis which are in items.
like : str
 Keep labels from axis for which "like in label == True".
regex : str (regular expression)
 Keep labels from axis for which re.search(regex, label) == True.
axis : {0 or 'index', 1 or 'columns', None}, default None
 The axis to filter on, expressed either as an index (int)
 or axis name (str). By default this is the info axis, 'columns' for
 ``DataFrame``. For ``Series`` this parameter is unused and defaults to
 ``None``.

```

Returns

```

Same type as caller
 The filtered subset of the DataFrame or Series.

```

See Also

-----

`DataFrame.loc` : Access a group of rows and columns  
by label(s) or a boolean array.

Notes

-----

The ```items```, ```like```, and ```regex``` parameters are  
enforced to be mutually exclusive.

```axis``` defaults to the info axis that is used when indexing  
with ```[]```.

Examples

```
>>> df = pd.DataFrame(  
...     np.array([[1, 2, 3], [4, 5, 6]]),  
...     index=["mouse", "rabbit"],  
...     columns=["one", "two", "three"],  
... )  
>>> df
```

```
      one  two  three  
mouse    1    2     3  
rabbit    4    5     6
```

```
>>> # select columns by name  
>>> df.filter(items=["one", "three"])
```

```
      one  three  
mouse    1     3  
rabbit    4     6
```

```
>>> # select columns by regular expression  
>>> df.filter(regex="e$", axis=1)
```

```
      one  three  
mouse    1     3  
rabbit    4     6
```

```
>>> # select rows containing 'bbi'
```

```
>>> df.filter(like="bbi", axis=0)
```

```
      one  two  three  
rabbit    4    5     6
```

`first_valid_index(self)` -> Hashable

Return index for first non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information
on which values are considered missing.

Returns

type of index

Index of first non-missing value.

See Also

`DataFrame.last_valid_index` : Return index for last non-NA value or None, if
no non-NA value is found.

`Series.last_valid_index` : Return index for last non-NA value or None, if no
non-NA value is found.

`DataFrame.isna` : Detect missing values.

Examples

For Series:

```
>>> s = pd.Series([None, 3, 4])
```

```
>>> s.first_valid_index()
```

```
1
```

```
>>> s.last_valid_index()
```

```
2
```

```
>>> s = pd.Series([None, None])
```

```
>>> print(s.first_valid_index())
```

```
None
```

```
>>> print(s.last_valid_index())
```

```
None
```

If all elements in Series are NA/null, returns None.

```
>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None
```

If Series is empty, returns None.

For DataFrame:

```
>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})
>>> df
   A      B
0 NaN    NaN
1 NaN    3.0
2 2.0    4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2
```

```
>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
   A      B
0 None  None
1 None  None
2 None  None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If all elements in DataFrame are NA/null, returns None.

```
>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None
```

If DataFrame is empty, returns None.

`get(self, key, default=None)`
Get item from object for given key (ex: DataFrame column).

Returns ``default`` value if not found.

Parameters

key : object
Key for which item should be returned.
default : object, default None
Default value to return if key is not found.

Returns

same type as items contained in object
Item for given key or ``default`` value, if key is not found.

See Also

DataFrame.get : Get item from object for given key (ex: DataFrame column).
Series.get : Get item from object for given key (ex: DataFrame column).

Examples

```
>>> df = pd.DataFrame(
...     [
...         [24.3, 75.7, "high"],
...         [31, 87.8, "high"],
...         [22, 71.6, "medium"],
...     ]
... )
```



```
...         [35, 95, "medium"],
...     ],
...     columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
...     index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
... )
```

```
>>> df
      temp_celsius  temp_fahrenheit  windspeed
2014-02-12         24.3           75.7      high
2014-02-13         31.0           87.8      high
2014-02-14         22.0           71.6    medium
2014-02-15         35.0           95.0    medium
```

```
>>> df.get(["temp_celsius", "windspeed"])
      temp_celsius  windspeed
2014-02-12         24.3      high
2014-02-13         31.0      high
2014-02-14         22.0    medium
2014-02-15         35.0    medium
```

```
>>> ser = df["windspeed"]
>>> ser.get("2014-02-13")
'high'
```

If the key isn't found, the default value will be used.

```
>>> df.get(["temp_celsius", "temp_kelvin"], default="default_value")
'default_value'
```

```
>>> ser.get("2014-02-10", "[unknown]")
'[unknown]'
```

head(self, n: int = 5) -> Self
Return the first `n` rows.

This function exhibits the same behavior as `df[:n]`, returning the first `n` rows based on position. It is useful for quickly checking if your object has the right type of data in it.

When `n` is positive, it returns the first `n` rows. For `n` equal to 0, it returns an empty object. When `n` is negative, it returns all rows except the last `|n|` rows, mirroring the behavior of `df[:n]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

`n` : int, default 5
Number of rows to select.

Returns

same type as caller
The first `n` rows of the caller object.

See Also

`DataFrame.tail`: Returns the last `n` rows.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "animal": [
...             "alligator",
...             "bee",
...             "falcon",
...             "lion",
...             "monkey",
...             "parrot",
...             "shark",
...             "whale",
...             "zebra",
...         ]
...     }
... )
>>> df
```

```

    animal
0  alligator
1    bee
2   falcon
3    lion
4   monkey
5   parrot
6    shark
7    whale
8    zebra

```

Viewing the first 5 lines

```

>>> df.head()
    animal
0  alligator
1    bee
2   falcon
3    lion
4   monkey

```

Viewing the first `n` lines (three in this case)

```

>>> df.head(3)
    animal
0  alligator
1    bee
2   falcon

```

For negative values of `n`

```

>>> df.head(-3)
    animal
0  alligator
1    bee
2   falcon
3    lion
4   monkey
5   parrot

```

`infer_objects(self, copy: bool | lib.NoDefault = <no_default>) -> Self`
 Attempt to infer better dtypes for object columns.

Attempts soft conversion of object-dtyped columns, leaving non-object and unconvertible columns unchanged. The inference rules are the same as during normal Series/DataFrame construction.

Parameters

`copy` : bool, default False

This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

Returns

same type as input object

Returns an object of the same type as the input object.

See Also

`to_datetime` : Convert argument to datetime.

`to_timedelta` : Convert argument to timedelta.

`to_numeric` : Convert argument to numeric type.

`convert_dtypes` : Convert argument to best possible dtype.

Examples

```

>>> df = pd.DataFrame({"A": ["a", 1, 2, 3]})
>>> df = df.iloc[1:]
>>> df
   A
1  1
2  2
3  3

>>> df.dtypes
A    object
dtype: object

>>> df.infer_objects().dtypes
A    int64
dtype: object

```

interpolate(
 self,
 method: InterpolateOptions = 'linear',
 *,
 axis: Axis = 0,
 limit: int | None = None,
 inplace: bool = False,
 limit_direction: Literal['forward', 'backward', 'both'] | None = None,
 limit_area: Literal['inside', 'outside'] | None = None,
 **kwargs
) -> Self
 Fill NaN values using an interpolation method.

Please note that only ``method='linear'`` is supported for DataFrame/Series with a MultiIndex.

Parameters

method : str, default 'linear'
 Interpolation technique to use. One of:

- * 'linear': Ignore the index and treat the values as equally spaced. This is the only method supported on MultiIndexes.
- * 'time': Works on daily and higher resolution data to interpolate given length of interval. This interpolates values based on time interval between observations.
- * 'index': The interpolation uses the numerical values of the DataFrame's index to linearly calculate missing values.
- * 'values': Interpolation based on the numerical values in the DataFrame, treating them as equally spaced along the index.
- * 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'barycentric', 'polynomial': Passed to ``scipy.interpolate.interp1d``, whereas 'spline' is passed to ``scipy.interpolate.UnivariateSpline``. These methods use the numerical values of the index. Both 'polynomial' and 'spline' require that you also specify an ``order`` (int), e.g. ``df.interpolate(method='polynomial', order=5)``. Note that, 'slinear' method in Pandas refers to the SciPy first order 'spline' instead of Pandas first order 'spline'.
- * 'krogh', 'piecewise_polynomial', 'spline', 'pchip', 'akima', 'cubicspline': Wrappers around the SciPy interpolation methods of similar names. See 'Notes'.
- * 'from_derivatives': Refers to ``scipy.interpolate.BPoly.from\_derivatives``.

axis : {{0 or 'index', 1 or 'columns', None}}, default None
 Axis to interpolate along. For 'Series' this parameter is unused and defaults to 0.

limit : int, optional
 Maximum number of consecutive NaNs to fill. Must be greater than 0.

inplace : bool, default False
 Update the data in place if possible.

limit_direction : {{'forward', 'backward', 'both'}}, optional, default 'forward'
 Consecutive NaNs will be filled in this direction.

limit_area : {{'None', 'inside', 'outside'}}, default None
 If limit is specified, consecutive NaNs will be filled with this restriction.

* ``None``: No fill restriction.

```

    * 'inside': Only fill NaNs surrounded by valid values
    (interpolate).
    * 'outside': Only fill NaNs outside valid values (extrapolate).

**kwargs : optional
    Keyword arguments to pass on to the interpolating function.

Returns
-----
Series or DataFrame
    Returns the same object type as the caller, interpolated at
    some or all ``NaN`` values.

See Also
-----
fillna : Fill missing values using different methods.
scipy.interpolate.Akima1DInterpolator : Piecewise cubic polynomials
    (Akima interpolator).
scipy.interpolate.BPoly.from_derivatives : Piecewise polynomial in the
    Bernstein basis.
scipy.interpolate.interp1d : Interpolate a 1-D function.
scipy.interpolate.KroghInterpolator : Interpolate polynomial (Krogh
    interpolator).
scipy.interpolate.PchipInterpolator : PCHIP 1-d monotonic cubic
    interpolation.
scipy.interpolate.CubicSpline : Cubic spline data interpolator.

Notes
-----
The 'krogh', 'piecewise_polynomial', 'spline', 'pchip' and 'akima'
methods are wrappers around the respective SciPy implementations of
similar names. These use the actual numerical values of the index.
For more information on their behavior, see the
`SciPy documentation
<https://docs.scipy.org/doc/scipy/reference/interpolate.html#univariate-interpolation>`_.

Examples
-----
Filling in ``NaN`` in a :class:`~pandas.Series` via linear
interpolation.

>>> s = pd.Series([0, 1, np.nan, 3])
>>> s
0    0.0
1    1.0
2    NaN
3    3.0
dtype: float64
>>> s.interpolate()
0    0.0
1    1.0
2    2.0
3    3.0
dtype: float64

Filling in ``NaN`` in a Series via polynomial interpolation or splines:
Both 'polynomial' and 'spline' methods require that you also specify
an ``order`` (int).

>>> s = pd.Series([0, 2, np.nan, 8])
>>> s.interpolate(method="polynomial", order=2)
0    0.000000
1    2.000000
2    4.666667
3    8.000000
dtype: float64

Fill the DataFrame forward (that is, going down) along each column
using linear interpolation.

Note how the last entry in column 'a' is interpolated differently,
because there is no entry after it to use for interpolation.
Note how the first entry in column 'b' remains ``NaN``, because there
is no entry before it to use for interpolation.

>>> df = pd.DataFrame(
...     [

```

```

...         (0.0, np.nan, -1.0, 1.0),
...         (np.nan, 2.0, np.nan, np.nan),
...         (2.0, 3.0, np.nan, 9.0),
...         (np.nan, 4.0, -4.0, 16.0),
...     ],
...     columns=list("abcd"),
... )
>>> df
   a    b    c    d
0  0.0 NaN -1.0  1.0
1  NaN  2.0 NaN  NaN
2  2.0  3.0 NaN  9.0
3  NaN  4.0 -4.0 16.0
>>> df.interpolate(method="linear", limit_direction="forward", axis=0)
   a    b    c    d
0  0.0 NaN -1.0  1.0
1  1.0  2.0 -2.0  5.0
2  2.0  3.0 -3.0  9.0
3  2.0  4.0 -4.0 16.0

```

Using polynomial interpolation.

```

>>> df["d"].interpolate(method="polynomial", order=2)
0    1.0
1    4.0
2    9.0
3   16.0
Name: d, dtype: float64

```

`last_valid_index(self)` -> Hashable
Return index for last non-missing value or None, if no value is found.

See the :ref:`User Guide <missing\_data>` for more information on which values are considered missing.

Returns

type of index
Index of last non-missing value.

See Also

`DataFrame.first_valid_index` : Return index for first non-NA value or None, if no non-NA value is found.
`Series.first_valid_index` : Return index for first non-NA value or None, if no non-NA value is found.
`DataFrame.isna` : Detect missing values.

Examples

For Series:

```

>>> s = pd.Series([None, 3, 4])
>>> s.first_valid_index()
1
>>> s.last_valid_index()
2

>>> s = pd.Series([None, None])
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If all elements in Series are NA/null, returns None.

```

>>> s = pd.Series()
>>> print(s.first_valid_index())
None
>>> print(s.last_valid_index())
None

```

If Series is empty, returns None.

For DataFrame:

```

>>> df = pd.DataFrame({"A": [None, None, 2], "B": [None, 3, 4]})

```

```

>>> df
   A      B
0 NaN    NaN
1 NaN    3.0
2 2.0    4.0
>>> df.first_valid_index()
1
>>> df.last_valid_index()
2

>>> df = pd.DataFrame({"A": [None, None, None], "B": [None, None, None]})
>>> df
   A      B
0 None  None
1 None  None
2 None  None
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None

```

If all elements in DataFrame are NA/null, returns None.

```

>>> df = pd.DataFrame()
>>> df
Empty DataFrame
Columns: []
Index: []
>>> print(df.first_valid_index())
None
>>> print(df.last_valid_index())
None

```

If DataFrame is empty, returns None.

```

mask(
    self,
    cond,
    other=<no_default>,
    *,
    inplace: bool = False,
    axis: Axis | None = None,
    level: Level | None = None
) -> Self
Replace values where the condition is True.

```

Parameters

```

cond : bool Series/DataFrame, array-like, or callable
    Where `cond` is False, keep the original value. Where
    True, replace with corresponding value from `other`.
    If `cond` is callable, it is computed on the Series/DataFrame and
    should return boolean Series/DataFrame or array. The callable must
    not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
    Entries where `cond` is True are replaced with
    corresponding value from `other`.
    If other is callable, it is computed on the Series/DataFrame and
    should return scalar or Series/DataFrame. The callable must not
    change input Series/DataFrame (though pandas doesn't check it).
    If not specified, entries will be filled with the corresponding
    NULL value (`np.nan` for numpy dtypes, `pd.NA` for extension
    dtypes).
inplace : bool, default False
    Whether to perform the operation in place on the data.
axis : int, default None
    Alignment axis if needed. For `Series` this parameter is
    unused and defaults to 0.
level : int, default None
    Alignment level if needed.

```

Returns

```

Series or DataFrame
    When applied to a Series, the function will return a Series,
    and when applied to a DataFrame, it will return a DataFrame.

```

See Also

:func:`DataFrame.where` : Return an object of same shape as caller.
:func:`Series.where` : Return an object of same shape as caller.

Notes

The mask method is an application of the if-then idiom. For each element in the caller, if ``cond`` is ``False`` the element is used; otherwise the corresponding element from ``other`` is used. If the axis of ``other`` does not align with axis of ``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions will be filled with True.

The signature for :func:`Series.where` or :func:`DataFrame.where` differs from :func:`numpy.where`. Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

For further details and examples see the ``mask`` documentation in :ref:`indexing <indexing.where\_mask>`.

The dtype of the object takes precedence. The fill value is casted to the object's dtype, if this can be done losslessly.

Examples

>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0 NaN
1 1.0
2 2.0
3 3.0
4 4.0
dtype: float64
>>> s.mask(s > 0)
0 0.0
1 NaN
2 NaN
3 NaN
4 NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0 0
1 99
2 99
3 99
4 99
dtype: int64
>>> s.mask(t, 99)
0 99
1 1
2 99
3 99
4 99
dtype: int64

>>> s.where(s > 1, 10)
0 10
1 10
2 2
3 3
4 4
dtype: int64
>>> s.mask(s > 1, 10)
0 0
1 1
2 10
3 10
4 10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
 A B

```

0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True

```

```

pct_change(
    self,
    periods: int = 1,
    fill_method: None = None,
    freq=None,
    **kwargs
) -> Self
    Fractional change between the current and a prior element.

```

Computes the fractional change from the immediately previous row by default. This is useful in comparing the fraction of change in a time series of elements.

.. note::

Despite the name of this method, it calculates fractional change (also known as per unit change or relative change) and not percentage change. If you need the percentage change, multiply these values by 100.

Parameters

```

-----
periods : int, default 1
    Periods to shift for forming percent change.
fill_method : None
    Must be None. This argument will be removed in a future version of pandas.
freq : DateOffset, timedelta, or str, optional
    Increment to use from time series API (e.g. 'ME' or BDay()).
**kwargs
    Additional keyword arguments are passed into
    `DataFrame.shift` or `Series.shift`.

```

Returns

```

-----
Series or DataFrame
    The same type as the calling object.

```

See Also

```

-----
Series.diff : Compute the difference of two elements in a Series.
DataFrame.diff : Compute the difference of two elements in a DataFrame.
Series.shift : Shift the index by some number of periods.
DataFrame.shift : Shift the index by some number of periods.

```

Examples

```

-----
**Series**

>>> s = pd.Series([90, 91, 85])

```



```
>>> s
0    90
1    91
2    85
dtype: int64
```

```
>>> s.pct_change()
0      NaN
1    0.011111
2   -0.065934
dtype: float64
```

```
>>> s.pct_change(periods=2)
0      NaN
1      NaN
2   -0.055556
dtype: float64
```

See the percentage change in a Series where filling NAs with last valid observation forward to next valid.

```
>>> s = pd.Series([90, 91, None, 85])
>>> s
0    90.0
1    91.0
2      NaN
3    85.0
dtype: float64
```

```
>>> s.ffill().pct_change()
0      NaN
1    0.011111
2    0.000000
3   -0.065934
dtype: float64
```

****DataFrame****

Percentage change in French franc, Deutsche Mark, and Italian lira from 1980-01-01 to 1980-03-01.

```
>>> df = pd.DataFrame(
...     {
...         "FR": [4.0405, 4.0963, 4.3149],
...         "GR": [1.7246, 1.7482, 1.8519],
...         "IT": [804.74, 810.01, 860.13],
...     },
...     index=["1980-01-01", "1980-02-01", "1980-03-01"],
... )
>>> df
```

	FR	GR	IT
1980-01-01	4.0405	1.7246	804.74
1980-02-01	4.0963	1.7482	810.01
1980-03-01	4.3149	1.8519	860.13

```
>>> df.pct_change()
              FR          GR          IT
1980-01-01      NaN          NaN          NaN
1980-02-01  0.013810  0.013684  0.006549
1980-03-01  0.053365  0.059318  0.061876
```

Percentage of change in GOOG and APPL stock volume. Shows computing the percentage change between columns.

```
>>> df = pd.DataFrame(
...     {
...         "2016": [1769950, 30586265],
...         "2015": [1500923, 40912316],
...         "2014": [1371819, 41403351],
...     },
...     index=["GOOG", "APPL"],
... )
>>> df
```

	2016	2015	2014
GOOG	1769950	1500923	1371819
APPL	30586265	40912316	41403351

```
>>> df.pct_change(axis="columns", periods=-1)
           2016      2015  2014
GOOG  0.179241  0.094112   NaN
APPL -0.252395 -0.011860   NaN
```

```
pipe(
    self,
    func: Callable[[Concatenate[Self, P], T] | tuple[Callable[... T], str],
    *args: Any,
    **kwargs: Any
) -> T
Apply chainable functions that expect Series or DataFrames.
```

Parameters

```
func : function
    Function to apply to the Series/DataFrame.
    ``args``, and ``kwargs`` are passed into ``func``.
    Alternatively a ``(callable, data_keyword)`` tuple where
    ``data_keyword`` is a string indicating the keyword of
    ``callable`` that expects the Series/DataFrame.
*args : iterable, optional
    Positional arguments passed into ``func``.
**kwargs : mapping, optional
    A dictionary of keyword arguments passed into ``func``.
```

Returns

```
The return type of ``func``.
The result of applying ``func`` to the Series or DataFrame.
```

See Also

```
DataFrame.apply : Apply a function along input axis of DataFrame.
DataFrame.map : Apply a function elementwise on a whole DataFrame.
Series.map : Apply a mapping correspondence on a
:~pandas.Series`.
```

Notes

```
Use ``.pipe`` when chaining together functions that expect
Series, DataFrames or GroupBy objects.
```

Examples

```
Constructing an income DataFrame from a dictionary.
```

```
>>> data = [[8000, 1000], [9500, np.nan], [5000, 2000]]
>>> df = pd.DataFrame(data, columns=["Salary", "Others"])
>>> df
   Salary  Others
0    8000   1000.0
1    9500     NaN
2    5000   2000.0
```

```
Functions that perform tax reductions on an income DataFrame.
```

```
>>> def subtract_federal_tax(df):
...     return df * 0.9
>>> def subtract_state_tax(df, rate):
...     return df * (1 - rate)
>>> def subtract_national_insurance(df, rate, rate_increase):
...     new_rate = rate + rate_increase
...     return df * (1 - new_rate)
```

```
Instead of writing
```

```
>>> subtract_national_insurance(
...     subtract_state_tax(subtract_federal_tax(df), rate=0.12),
...     rate=0.05,
...     rate_increase=0.02,
... ) # doctest: +SKIP
```

```
You can write
```

```
>>> (
...     df.pipe(subtract_federal_tax)
```

```

...     .pipe(subtract_state_tax, rate=0.12)
...     .pipe(subtract_national_insurance, rate=0.05, rate_increase=0.02)
... )
      Salary  Others
0  5892.48    736.56
1  6997.32      NaN
2  3682.80   1473.12

```

If you have a function that takes the data as (say) the second argument, pass a tuple indicating which keyword expects the data. For example, suppose ``national\_insurance`` takes its data as ``df`` in the second argument:

```

>>> def subtract_national_insurance(rate, df, rate_increase):
...     new_rate = rate + rate_increase
...     return df * (1 - new_rate)
>>> (
...     df.pipe(subtract_federal_tax)
...     .pipe(subtract_state_tax, rate=0.12)
...     .pipe(
...         (subtract_national_insurance, "df"), rate=0.05, rate_increase=0.02
...     )
... )
      Salary  Others
0  5892.48    736.56
1  6997.32      NaN
2  3682.80   1473.12

```

```

rank(
    self,
    axis: Axis = 0,
    method: Literal['average', 'min', 'max', 'first', 'dense'] = 'average',
    numeric_only: bool = False,
    na_option: Literal['keep', 'top', 'bottom'] = 'keep',
    ascending: bool = True,
    pct: bool = False
) -> Self
Compute numerical data ranks (1 through n) along axis.

```

By default, equal values are assigned a rank that is the average of the ranks of those values.

Parameters

```

axis : {0 or 'index', 1 or 'columns'}, default 0
    Index to direct ranking.
    For `Series` this parameter is unused and defaults to 0.
method : {'average', 'min', 'max', 'first', 'dense'}, default 'average'
    How to rank the group of records that have the same value (i.e. ties):
    * average: average rank of the group
    * min: lowest rank in the group
    * max: highest rank in the group
    * first: ranks assigned in order they appear in the array
    * dense: like 'min', but rank always increases by 1 between groups.

```

```

numeric_only : bool, default False
    For DataFrame objects, rank only numeric columns if set to True.

```

```

.. versionchanged:: 2.0.0
    The default value of ``numeric_only`` is now ``False``.

```

```

na_option : {'keep', 'top', 'bottom'}, default 'keep'
    How to rank NaN values:

```

```

    * keep: assign NaN rank to NaN values
    * top: assign lowest rank to NaN values
    * bottom: assign highest rank to NaN values

```

```

ascending : bool, default True
    Whether or not the elements should be ranked in ascending order.
pct : bool, default False
    Whether or not to display the returned rankings in percentile form.

```

Returns

```

-----

```

same type as caller
Return a Series or DataFrame with data ranks as values.

See Also

core.groupby.DataFrameGroupBy.rank : Rank of values within each group.
core.groupby.SeriesGroupBy.rank : Rank of values within each group.

Examples

>>> df = pd.DataFrame(
... data={
... "Animal": ["cat", "penguin", "dog", "spider", "snake"],
... "Number_legs": [4, 2, 4, 8, np.nan],
... }
...)
>>> df

	Animal	Number_legs
0	cat	4.0
1	penguin	2.0
2	dog	4.0
3	spider	8.0
4	snake	NaN

Ties are assigned the mean of the ranks (by default) for the group.

```
>>> s = pd.Series(range(5), index=list("abcde"))  
>>> s["d"] = s["b"]  
>>> s.rank()  
a    1.0  
b    2.5  
c    4.0  
d    2.5  
e    5.0  
dtype: float64
```

The following example shows how the method behaves with the above parameters:

- * default_rank: this is the default behaviour obtained without using any parameter.
- * max_rank: setting ``method = 'max'`` the records that have the same values are ranked using the highest rank (e.g.: since 'cat' and 'dog' are both in the 2nd and 3rd position, rank 3 is assigned.)
- * NA_bottom: choosing ``na\_option = 'bottom'``, if there are records with NaN values they are placed at the bottom of the ranking.
- * pct_rank: when setting ``pct = True``, the ranking is expressed as percentile rank.

```
>>> df["default_rank"] = df["Number_legs"].rank()  
>>> df["max_rank"] = df["Number_legs"].rank(method="max")  
>>> df["NA_bottom"] = df["Number_legs"].rank(na_option="bottom")  
>>> df["pct_rank"] = df["Number_legs"].rank(pct=True)  
>>> df
```

	Animal	Number_legs	default_rank	max_rank	NA_bottom	pct_rank
0	cat	4.0	2.5	3.0	2.5	0.625
1	penguin	2.0	1.0	1.0	1.0	0.250
2	dog	4.0	2.5	3.0	2.5	0.625
3	spider	8.0	4.0	4.0	4.0	1.000
4	snake	NaN	NaN	NaN	5.0	NaN

```
reindex_like(  
    self,  
    other,  
    method: Literal['backfill', 'bfill', 'pad', 'ffill', 'nearest'] | None = None,  
    copy: bool | lib.NoDefault = <no_default>,  
    limit: int | None = None,  
    tolerance=None  
) -> Self  
Return an object with matching indices as other object.
```

Conform the object to the same index on all axes. Optional filling logic, placing NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and copy=False.

Parameters

```

-----
other : Object of the same data type
      Its row and column indices are used to define the new indices
      of this object.
method : {None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}
      Method to use for filling holes in reindexed DataFrame.
      Please note: this is only applicable to DataFrames/Series with a
      monotonically increasing/decreasing index.

      .. deprecated:: 3.0.0

      * None (default): don't fill gaps
      * pad / ffill: propagate last valid observation forward to next
        valid
      * backfill / bfill: use next valid observation to fill gap
      * nearest: use nearest valid observations to fill gap.

copy : bool, default False
      This keyword is now ignored; changing its value will have no
      impact on the method.

      .. deprecated:: 3.0.0

      This keyword is ignored and will be removed in pandas 4.0. Since
      pandas 3.0, this method always returns a new object using a lazy
      copy mechanism that defers copies until necessary
      (Copy-on-Write). See the `user guide on Copy-on-Write
      <https://pandas.pydata.org/docs/dev/user\_guide/copy\_on\_write.html>`__
      for more details.

limit : int, default None
      Maximum number of consecutive labels to fill for inexact matches.
tolerance : optional
      Maximum distance between original and new labels for inexact
      matches. The values of the index at the matching locations must
      satisfy the equation ``abs(index[indexer] - target) <= tolerance``.

      Tolerance may be a scalar value, which applies the same tolerance
      to all values, or list-like, which applies variable tolerance per
      element. List-like includes list, tuple, array, Series, and must be
      the same size as the index and its dtype must exactly match the
      index's type.

Returns
-----
Series or DataFrame
      Same type as caller, but with changed indices on each axis.

See Also
-----
DataFrame.set_index : Set row labels.
DataFrame.reset_index : Remove row labels or move them to new columns.
DataFrame.reindex : Change to new indices or expand indices.

Notes
-----
Same as calling
``.reindex(index=other.index, columns=other.columns,...)``.

Examples
-----
>>> df1 = pd.DataFrame(
...     [
...         [24.3, 75.7, "high"],
...         [31, 87.8, "high"],
...         [22, 71.6, "medium"],
...         [35, 95, "medium"],
...     ],
...     columns=["temp_celsius", "temp_fahrenheit", "windspeed"],
...     index=pd.date_range(start="2014-02-12", end="2014-02-15", freq="D"),
... )

>>> df1
          temp_celsius  temp_fahrenheit  windspeed
2014-02-12          24.3             75.7        high
2014-02-13          31.0             87.8        high
2014-02-14          22.0             71.6        medium

```

```

2014-02-15          35.0          95.0    medium

>>> df2 = pd.DataFrame(
...     [[28, "low"], [30, "low"], [35.1, "medium"]],
...     columns=["temp_celsius", "windspeed"],
...     index=pd.DatetimeIndex(["2014-02-12", "2014-02-13", "2014-02-15"]),
... )

```

```

>>> df2
           temp_celsius windspeed
2014-02-12          28.0        low
2014-02-13          30.0        low
2014-02-15          35.1    medium

```

```

>>> df2.reindex_like(df1)
           temp_celsius  temp_fahrenheit windspeed
2014-02-12          28.0             NaN        low
2014-02-13          30.0             NaN        low
2014-02-14           NaN             NaN         NaN
2014-02-15          35.1             NaN    medium

```

```

replace(
    self,
    to_replace=None,
    value=<no_default>,
    *,
    inplace: bool = False,
    regex: bool = False
) -> Self
Replace values given in `to_replace` with `value`.

```

Values of the Series/DataFrame are replaced with other values dynamically. This differs from updating with ``.loc`` or ``.iloc``, which require you to specify a location to update with some value.

Parameters

`to_replace` : str, regex, list, dict, Series, int, float, or None
How to find the values that will be replaced.

* numeric, str or regex:

- numeric: numeric values equal to `to\_replace` will be replaced with `value`
- str: string exactly matching `to\_replace` will be replaced with `value`
- regex: regexes matching `to\_replace` will be replaced with `value`

* list of str, regex, or numeric:

- First, if `to\_replace` and `value` are both lists, they **must** be the same length.
- Second, if ``regex=True`` then all of the strings in **both** lists will be interpreted as regexes otherwise they will match directly. This doesn't matter much for `value` since there are only a few possible substitution regexes you can use.
- str, regex and numeric rules apply as above.

* dict:

- Dicts can be used to specify different replacement values for different existing values. For example, ``{'a': 'b', 'y': 'z'}`` replaces the value 'a' with 'b' and 'y' with 'z'. To use a dict in this way, the optional `value` parameter should not be given.
- For a DataFrame a dict can specify that different values should be replaced in different columns. For example, ``{'a': 1, 'b': 'z'}`` looks for the value 1 in column 'a' and the value 'z' in column 'b' and replaces these values with whatever is specified in `value`. The `value` parameter should not be ``None`` in this case. You can treat this as a special case of passing two lists except that you are specifying the column to search in.
- For a DataFrame nested dictionaries, e.g., ``{'a': {'b': np.nan}}``, are read as follows: look in column 'a' for the value 'b' and replace it with NaN. The optional `value`

parameter should not be specified to use a nested dict in this way. You can nest regular expressions as well. Note that column names (the top-level dictionary keys in a nested dictionary) **cannot** be regular expressions.

*** None:**

- This means that the ``regex`` argument must be a string, compiled regular expression, or list, dict, ndarray or Series of such elements. If ``value`` is also ``None`` then this **must** be a nested dictionary or Series.

See the examples section for examples of each of these.

`value` : scalar, dict, list, str, regex, default None
Value to replace any values matching ``to_replace`` with.
For a DataFrame a dict of values can be used to specify which value to use for each column (columns not in the dict will not be filled). Regular expressions, strings and lists or dicts of such objects are also allowed.

`inplace` : bool, default False

If True, performs operation inplace.

`regex` : bool or same types as ``to_replace``, default False
Whether to interpret ``to_replace`` and/or ``value`` as regular expressions. Alternatively, this could be a regular expression or a list, dict, or array of regular expressions in which case ``to_replace`` must be ``None``.

Returns

Series/DataFrame
Object after replacement.

Raises

AssertionError
* If ``regex`` is not a ``bool`` and ``to_replace`` is not ``None``.

TypeError

- * If ``to_replace`` is not a scalar, array-like, ``dict``, or ``None``
- * If ``to_replace`` is a ``dict`` and ``value`` is not a ``list``, ``dict``, ``ndarray``, or ``Series``
- * If ``to_replace`` is ``None`` and ``regex`` is not compilable into a regular expression or is a list, dict, ndarray, or Series.
- * When replacing multiple ``bool`` or ``datetime64`` objects and the arguments to ``to_replace`` does not match the type of the value being replaced

ValueError

- * If a ``list`` or an ``ndarray`` is passed to ``to_replace`` and ``value`` but they are not the same length.

See Also

`Series.fillna` : Fill NA values.
`DataFrame.fillna` : Fill NA values.
`Series.where` : Replace values based on boolean condition.
`DataFrame.where` : Replace values based on boolean condition.
`DataFrame.map` : Apply a function to a DataFrame elementwise.
`Series.map` : Map values of Series according to an input mapping or function.
`Series.str.replace` : Simple string replacement.

Notes

-
- * Regex substitution is performed under the hood with ``re.sub``. The rules for substitution for ``re.sub`` are the same.
 - * Regular expressions will only substitute on strings, meaning you cannot provide, for example, a regular expression matching floating point numbers and expect the columns in your frame that have a numeric dtype to be matched. However, if those floating point numbers *are* strings, then you can do this.
 - * This method has *a lot* of options. You are encouraged to experiment and play with this method to gain intuition about how it works.
 - * When dict is used as the ``to_replace`` value, it is like key(s) in the dict are the `to_replace` part and

value(s) in the dict are the value parameter.

Examples

****Scalar `to\_replace` and `value`****

```
>>> s = pd.Series([1, 2, 3, 4, 5])
>>> s.replace(1, 5)
0    5
1    2
2    3
3    4
4    5
dtype: int64
```

```
>>> df = pd.DataFrame(
...     {
...         "A": [0, 1, 2, 3, 4],
...         "B": [5, 6, 7, 8, 9],
...         "C": ["a", "b", "c", "d", "e"],
...     }
... )
>>> df.replace(0, 5)
   A  B  C
0  5  5  a
1  1  6  b
2  2  7  c
3  3  8  d
4  4  9  e
```

****List-like `to\_replace`****

```
>>> df.replace([0, 1, 2, 3], 4)
   A  B  C
0  4  5  a
1  4  6  b
2  4  7  c
3  4  8  d
4  4  9  e
```

```
>>> df.replace([0, 1, 2, 3], [4, 3, 2, 1])
   A  B  C
0  4  5  a
1  3  6  b
2  2  7  c
3  1  8  d
4  4  9  e
```

****dict-like `to\_replace`****

```
>>> df.replace({0: 10, 1: 100})
   A  B  C
0  10  5  a
1 100  6  b
2   2  7  c
3   3  8  d
4   4  9  e
```

```
>>> df.replace({"A": 0, "B": 5}, 100)
   A  B  C
0 100 100  a
1   1   6  b
2   2   7  c
3   3   8  d
4   4   9  e
```

```
>>> df.replace({"A": {0: 100, 4: 400}})
   A  B  C
0 100  5  a
1   1  6  b
2   2  7  c
3   3  8  d
4 400  9  e
```

****Regular expression `to\_replace`****


```
>>> df = pd.DataFrame({"A": ["bat", "foo", "bait"], "B": ["abc", "bar", "xyz"]})
>>> df.replace(to_replace=r"^ba.$", value="new", regex=True)
```

```
   A  B
0  new abc
1  foo new
2  bait xyz
```

```
>>> df.replace({"A": r"^ba.$"}, {"A": "new"}, regex=True)
```

```
   A  B
0  new abc
1  foo bar
2  bait xyz
```

```
>>> df.replace(regex=r"^ba.$", value="new")
```

```
   A  B
0  new abc
1  foo new
2  bait xyz
```

```
>>> df.replace(regex={r"^ba.$": "new", "foo": "xyz"})
```

```
   A  B
0  new abc
1  xyz new
2  bait xyz
```

```
>>> df.replace(regex=[r"^ba.$", "foo"], value="new")
```

```
   A  B
0  new abc
1  new new
2  bait xyz
```

Compare the behavior of `df.replace({'a': None})` and `df.replace('a', None)` to understand the peculiarities of the `to_replace` parameter:

```
>>> s = pd.Series([10, "a", "a", "b", "a"])
```

When one uses a dict as the `to_replace` value, it is like the value(s) in the dict are equal to the `value` parameter. `df.replace({'a': None})` is equivalent to `df.replace(to_replace={'a': None}, value=None)`:

```
>>> s.replace({"a": None})
```

```
0    10
1    None
2    None
3     b
4    None
dtype: object
```

If `None` is explicitly passed for `value`, it will be respected:

```
>>> s.replace("a", None)
```

```
0    10
1    None
2    None
3     b
4    None
dtype: object
```

When `regex=True`, `value` is not `None` and `to_replace` is a string, the replacement will be applied in all columns of the DataFrame.

```
>>> df = pd.DataFrame(
...     {
...         "A": [0, 1, 2, 3, 4],
...         "B": ["a", "b", "c", "d", "e"],
...         "C": ["f", "g", "h", "i", "j"],
...     }
... )
```

```
>>> df.replace(to_replace="^[a-g]", value="e", regex=True)
```

```
   A  B  C
0  0  e  e
1  1  e  e
2  2  e  h
3  3  e  i
```

```
4 4 e j
```

If ``value`` is not ``None`` and ``to\_replace`` is a dictionary, the dictionary keys will be the DataFrame columns that the replacement will be applied.

```
>>> df.replace(to_replace={"B": "[a-c]", "C": "[h-j]"}, value="e", regex=True)
```

```
   A B C
0  0 e f
1  1 e g
2  2 e e
3  3 d e
4  4 e e
```

```
resample(
    self,
    rule,
    closed: Literal['right', 'left'] | None = None,
    label: Literal['right', 'left'] | None = None,
    convention: Literal['start', 'end', 's', 'e'] = 'start',
    on: Level | None = None,
    level: Level | None = None,
    origin: str | TimestampConvertibleTypes = 'start_day',
    offset: TimedeltaConvertibleTypes | None = None,
    group_keys: bool = False
) -> Resampler
Resample time-series data.
```

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (`DatetimeIndex`, `PeriodIndex`, or `TimedeltaIndex`), or the caller must pass the label of a datetime-like series/index to the `on`/`level` keyword parameter.

Parameters

```
rule : DateOffset, Timedelta or str
    The offset string or object representing target conversion.
closed : {'right', 'left'}, default None
    Which side of bin interval is closed. The default is 'left'
    for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
    'BA', 'BQE', and 'W' which all have a default of 'right'.
label : {'right', 'left'}, default None
    Which bin edge label to label bucket with. The default is 'left'
    for all frequency offsets except for 'ME', 'YE', 'QE', 'BME',
    'BA', 'BQE', and 'W' which all have a default of 'right'.
convention : {'start', 'end', 's', 'e'}, default 'start'
    For 'PeriodIndex' only, controls whether to use the start or
    end of 'rule'.
on : str, optional
    For a DataFrame, column to use instead of index for resampling.
    Column must be datetime-like.
level : str or int, optional
    For a MultiIndex, level (name or number) to use for
    resampling. 'level' must be datetime-like.
origin : Timestamp or str, default 'start_day'
    The timestamp on which to adjust the grouping. The timezone of origin
    must match the timezone of the index.
    If string, must be Timestamp convertible or one of the following:

    - 'epoch': 'origin' is 1970-01-01
    - 'start': 'origin' is the first value of the timeseries
    - 'start_day': 'origin' is the first day at midnight of the timeseries

    - 'end': 'origin' is the last value of the timeseries
    - 'end_day': 'origin' is the ceiling midnight of the last day

.. note::

    Only takes effect for Tick-frequencies (i.e. fixed frequencies like
    days, hours, and minutes, rather than months or quarters).
offset : Timedelta or str, default is None
    An offset timedelta added to the origin.

group_keys : bool, default False
    Whether to include the group keys in the result index when using
    ``.apply()`` on the resampled object.

.. versionchanged:: 2.0.0
```

`group_keys` now defaults to `False`.

Returns

`pandas.api.typing.Resampler`
:class: `pandas.core.Resampler` object.

See Also

`Series.resample` : Resample a Series.
`DataFrame.resample` : Resample a DataFrame.
`groupby` : Group Series/DataFrame by mapping, function, label, or list of labels.
`asfreq` : Reindex a Series/DataFrame with the given frequency without grouping.

Notes

See the `user guide`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#resampling> for more.

To learn more about the offset strings, please see `this link`
<https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#dateoffset-objects>

Examples

Start by creating a series with 9 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=9, freq="min")
>>> series = pd.Series(range(9), index=index)
>>> series
2000-01-01 00:00:00    0
2000-01-01 00:01:00    1
2000-01-01 00:02:00    2
2000-01-01 00:03:00    3
2000-01-01 00:04:00    4
2000-01-01 00:05:00    5
2000-01-01 00:06:00    6
2000-01-01 00:07:00    7
2000-01-01 00:08:00    8
Freq: min, dtype: int64
```

Downsample the series into 3 minute bins and sum the values of the timestamps falling into a bin.

```
>>> series.resample("3min").sum()
2000-01-01 00:00:00    3
2000-01-01 00:03:00   12
2000-01-01 00:06:00   21
Freq: 3min, dtype: int64
```

Downsample the series into 3 minute bins as above, but label each bin using the right edge instead of the left. Please note that the value in the bucket used as the label is not included in the bucket, which it labels. For example, in the original series the bucket `2000-01-01 00:03:00` contains the value 3, but the summed value in the resampled bucket with the label `2000-01-01 00:03:00` does not include 3 (if it did, the summed value would be 6, not 3).

```
>>> series.resample("3min", label="right").sum()
2000-01-01 00:03:00    3
2000-01-01 00:06:00   12
2000-01-01 00:09:00   21
Freq: 3min, dtype: int64
```

To include this value close the right side of the bin interval, as shown below.

```
>>> series.resample("3min", label="right", closed="right").sum()
2000-01-01 00:00:00    0
2000-01-01 00:03:00    6
2000-01-01 00:06:00   15
2000-01-01 00:09:00   15
Freq: 3min, dtype: int64
```

Upsample the series into 30 second bins.

```
>>> series.resample("30s").asfreq()[0:5] # Select first 5 rows
2000-01-01 00:00:00    0.0
2000-01-01 00:00:30    NaN
2000-01-01 00:01:00    1.0
2000-01-01 00:01:30    NaN
2000-01-01 00:02:00    2.0
Freq: 30s, dtype: float64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``ffill`` method.

```
>>> series.resample("30s").ffill()[0:5]
2000-01-01 00:00:00    0
2000-01-01 00:00:30    0
2000-01-01 00:01:00    1
2000-01-01 00:01:30    1
2000-01-01 00:02:00    2
Freq: 30s, dtype: int64
```

Upsample the series into 30 second bins and fill the ``NaN`` values using the ``bfill`` method.

```
>>> series.resample("30s").bfill()[0:5]
2000-01-01 00:00:00    0
2000-01-01 00:00:30    1
2000-01-01 00:01:00    1
2000-01-01 00:01:30    2
2000-01-01 00:02:00    2
Freq: 30s, dtype: int64
```

Pass a custom function via ``apply``

```
>>> def custom_resampler(arraylike):
...     return np.sum(arraylike) + 5
>>> series.resample("3min").apply(custom_resampler)
2000-01-01 00:00:00    8
2000-01-01 00:03:00   17
2000-01-01 00:06:00   26
Freq: 3min, dtype: int64
```

For a Series with a PeriodIndex, the keyword ``convention`` can be used to control whether to use the start or end of ``rule``.

Resample a year by quarter using 'start' ``convention``. Values are assigned to the first quarter of the period.

```
>>> s = pd.Series(
...     [1, 2], index=pd.period_range("2012-01-01", freq="Y", periods=2)
... )
>>> s
2012    1
2013    2
Freq: Y-DEC, dtype: int64
>>> s.resample("Q", convention="start").asfreq()
2012Q1    1.0
2012Q2    NaN
2012Q3    NaN
2012Q4    NaN
2013Q1    2.0
2013Q2    NaN
2013Q3    NaN
2013Q4    NaN
Freq: Q-DEC, dtype: float64
```

Resample quarters by month using 'end' ``convention``. Values are assigned to the last month of the period.

```
>>> q = pd.Series(
...     [1, 2, 3, 4], index=pd.period_range("2018-01-01", freq="Q", periods=4)
... )
>>> q
2018Q1    1
2018Q2    2
2018Q3    3
2018Q4    4
Freq: Q-DEC, dtype: int64
>>> q.resample("M", convention="end").asfreq()
```

```

2018-03    1.0
2018-04    NaN
2018-05    NaN
2018-06    2.0
2018-07    NaN
2018-08    NaN
2018-09    3.0
2018-10    NaN
2018-11    NaN
2018-12    4.0
Freq: M, dtype: float64

```

For DataFrame objects, the keyword `on` can be used to specify the column instead of the index for resampling.

```

>>> df = pd.DataFrame([10, 11, 9, 13, 14, 18, 17, 19], columns=["price"])
>>> df["volume"] = [50, 60, 40, 100, 50, 100, 40, 50]
>>> df["week_starting"] = pd.date_range("01/01/2018", periods=8, freq="W")
>>> df
   price  volume week_starting
0     10     50   2018-01-07
1     11     60   2018-01-14
2      9     40   2018-01-21
3     13    100   2018-01-28
4     14     50   2018-02-04
5     18    100   2018-02-11
6     17     40   2018-02-18
7     19     50   2018-02-25
>>> df.resample("ME", on="week_starting").mean()
   price  volume
week_starting
2018-01-31    10.75    62.5
2018-02-28    17.00    60.0

```

For a DataFrame with MultiIndex, the keyword `level` can be used to specify on which level the resampling needs to take place.

```

>>> days = pd.date_range("1/1/2000", periods=4, freq="D")
>>> df2 = pd.DataFrame(
...     [
...         [10, 50],
...         [11, 60],
...         [9, 40],
...         [13, 100],
...         [14, 50],
...         [18, 100],
...         [17, 40],
...         [19, 50],
...     ],
...     columns=["price", "volume"],
...     index=pd.MultiIndex.from_product([days, ["morning", "afternoon"]]),
... )
>>> df2
           price  volume
2000-01-01 morning     10     50
           afternoon    11     60
2000-01-02 morning      9     40
           afternoon    13    100
2000-01-03 morning     14     50
           afternoon    18    100
2000-01-04 morning     17     40
           afternoon    19     50
>>> df2.resample("D", level=0).sum()
           price  volume
2000-01-01     21    110
2000-01-02     22    140
2000-01-03     32    150
2000-01-04     36     90

```

If you want to adjust the start of the bins based on a fixed timestamp:

```

>>> start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
>>> rng = pd.date_range(start, end, freq="7min")
>>> ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
>>> ts
2000-10-01 23:30:00    0
2000-10-01 23:37:00    3

```

```

2000-10-01 23:44:00    6
2000-10-01 23:51:00    9
2000-10-01 23:58:00   12
2000-10-02 00:05:00   15
2000-10-02 00:12:00   18
2000-10-02 00:19:00   21
2000-10-02 00:26:00   24
Freq: 7min, dtype: int64

```

```

>>> ts.resample("17min").sum()
2000-10-01 23:14:00    0
2000-10-01 23:31:00    9
2000-10-01 23:48:00   21
2000-10-02 00:05:00   54
2000-10-02 00:22:00   24
Freq: 17min, dtype: int64

```

```

>>> ts.resample("17min", origin="epoch").sum()
2000-10-01 23:18:00    0
2000-10-01 23:35:00   18
2000-10-01 23:52:00   27
2000-10-02 00:09:00   39
2000-10-02 00:26:00   24
Freq: 17min, dtype: int64

```

```

>>> ts.resample("17min", origin="2000-01-01").sum()
2000-10-01 23:24:00    3
2000-10-01 23:41:00   15
2000-10-01 23:58:00   45
2000-10-02 00:15:00   45
Freq: 17min, dtype: int64

```

If you want to adjust the start of the bins with an `offset` `Timedelta`, the two following lines are equivalent:

```

>>> ts.resample("17min", origin="start").sum()
2000-10-01 23:30:00    9
2000-10-01 23:47:00   21
2000-10-02 00:04:00   54
2000-10-02 00:21:00   24
Freq: 17min, dtype: int64

```

```

>>> ts.resample("17min", offset="23h30min").sum()
2000-10-01 23:30:00    9
2000-10-01 23:47:00   21
2000-10-02 00:04:00   54
2000-10-02 00:21:00   24
Freq: 17min, dtype: int64

```

If you want to take the largest Timestamp as the end of the bins:

```

>>> ts.resample("17min", origin="end").sum()
2000-10-01 23:35:00    0
2000-10-01 23:52:00   18
2000-10-02 00:09:00   27
2000-10-02 00:26:00   63
Freq: 17min, dtype: int64

```

In contrast with the `start_day`, you can use `end_day` to take the ceiling midnight of the largest Timestamp as the end of the bins and drop the bins not containing data:

```

>>> ts.resample("17min", origin="end_day").sum()
2000-10-01 23:38:00    3
2000-10-01 23:55:00   15
2000-10-02 00:12:00   45
2000-10-02 00:29:00   45
Freq: 17min, dtype: int64

```

```

rolling(
    self,
    window: int | dt.timedelta | str | BaseOffset | BaseIndexer,
    min_periods: int | None = None,
    center: bool = False,
    win_type: str | None = None,
    on: str | None = None,
    closed: IntervalClosedType | None = None,

```

```

    step: int | None = None,
    method: str = 'single'
) -> Window | Rolling
    Provide rolling window calculations.

Parameters
-----
window : int, timedelta, str, offset, or BaseIndexer subclass
    Interval of the moving window.

    If an integer, the delta between the start and end of each window.
    The number of points in the window depends on the ``closed`` argument.

    If a timedelta, str, or offset, the time period of each window. Each
    window will be a variable sized based on the observations included in
    the time-period. This is only valid for datetimelike indexes.
    To learn more about the offsets & frequency strings, please see
    :ref:`this link<timeseries.offset_aliases>`.

    If a BaseIndexer subclass, the window boundaries
    based on the defined ``get_window_bounds`` method. Additional rolling
    keyword arguments, namely ``min_periods``, ``center``, ``closed`` and
    ``step`` will be passed to ``get_window_bounds``.

min_periods : int, default None
    Minimum number of observations in window required to have a value;
    otherwise, result is ``np.nan``.

    For a window that is specified by an offset, ``min_periods`` will default
    to 1.

    For a window that is specified by an integer, ``min_periods`` will default
    to the size of the window.

center : bool, default False
    If False, set the window labels as the right edge of the window index.

    If True, set the window labels as the center of the window index.

win_type : str, default None
    If ``None``, all points are evenly weighted.

    If a string, it must be a valid `scipy.signal` window function
    <https://docs.scipy.org/doc/scipy/reference/signal.windows.html#module-scipy.signal>.

    Certain Scipy window types require additional parameters to be passed
    in the aggregation function. The additional parameters must match
    the keywords specified in the Scipy window type method signature.

on : str, optional
    For a DataFrame, a column label or Index level on which
    to calculate the rolling window, rather than the DataFrame's index.

    Provided integer column is ignored and excluded from result since
    an integer index is not used to calculate the rolling window.

closed : str, default None
    Determines the inclusivity of points in the window

    If ``'right'``, uses the window (first, last] meaning the last point
    is included in the calculations.

    If ``'left'``, uses the window [first, last) meaning the first point
    is included in the calculations.

    If ``'both'``, uses the window [first, last] meaning all points in
    the window are included in the calculations.

    If ``'neither'``, uses the window (first, last) meaning the first
    and last points in the window are excluded from calculations.

    () and [] are referencing open and closed set
    notation respectively.

    Default ``None`` (``'right'``).

step : int, default None

```

Evaluate the window at every ``step`` result, equivalent to slicing as ``[:step]``. ``window`` must be an integer. Using a step argument other than None or 1 will produce a result with a different shape than the input.

method : str {'single', 'table'}, default 'single'

Execute the rolling operation per single column or row (``'single'``) or over the entire object (``'table'``).

This argument is only implemented when specifying ``engine='numba'`` in the method call.

Returns

pandas.api.typing.Window or pandas.api.typing.Rolling
An instance of Window is returned if ``win\_type`` is passed. Otherwise, an instance of Rolling is returned.

See Also

expanding : Provides expanding transformations.
ewm : Provides exponential weighted functions.

Notes

See :ref:`Windowing Operations <window.generic>` for further usage details and examples.

Examples

>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
>>> df

```
   B
0  0.0
1  1.0
2  2.0
3  NaN
4  4.0
```

****window****

Rolling sum with a window length of 2 observations.

>>> df.rolling(2).sum()

```
   B
0  NaN
1  1.0
2  3.0
3  NaN
4  NaN
```

Rolling sum with a window span of 2 seconds.

```
>>> df_time = pd.DataFrame(
...     {"B": [0, 1, 2, np.nan, 4]},
...     index=[
...         pd.Timestamp("20130101 09:00:00"),
...         pd.Timestamp("20130101 09:00:02"),
...         pd.Timestamp("20130101 09:00:03"),
...         pd.Timestamp("20130101 09:00:05"),
...         pd.Timestamp("20130101 09:00:06"),
...     ],
... )
```

>>> df_time

```
                B
2013-01-01 09:00:00  0.0
2013-01-01 09:00:02  1.0
2013-01-01 09:00:03  2.0
2013-01-01 09:00:05  NaN
2013-01-01 09:00:06  4.0
```

>>> df_time.rolling("2s").sum()

```
                B
2013-01-01 09:00:00  0.0
2013-01-01 09:00:02  1.0
2013-01-01 09:00:03  3.0
```



```
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0
```

Rolling sum with forward looking windows with 2 observations.

```
>>> indexer = pd.api.indexers.FixedForwardWindowIndexer(window_size=2)
>>> df.rolling(window=indexer, min_periods=1).sum()
```

```
      B
0  1.0
1  3.0
2  2.0
3  4.0
4  4.0
```

****min_periods****

Rolling sum with a window length of 2 observations, but only needs a minimum of 1 observation to calculate a value.

```
>>> df.rolling(2, min_periods=1).sum()
```

```
      B
0  0.0
1  1.0
2  3.0
3  2.0
4  4.0
```

****center****

Rolling sum with the result assigned to the center of the window index.

```
>>> df.rolling(3, min_periods=1, center=True).sum()
```

```
      B
0  1.0
1  3.0
2  3.0
3  6.0
4  4.0
```

```
>>> df.rolling(3, min_periods=1, center=False).sum()
```

```
      B
0  0.0
1  1.0
2  3.0
3  3.0
4  6.0
```

****step****

Rolling sum with a window length of 2 observations, minimum of 1 observation to calculate a value, and a step of 2.

```
>>> df.rolling(2, min_periods=1, step=2).sum()
```

```
      B
0  0.0
2  3.0
4  4.0
```

****win_type****

Rolling sum with a window length of 2, using the Scipy `''gaussian''` window type. `''std''` is required in the aggregation function.

```
>>> df.rolling(2, win_type="gaussian").sum(std=3)
```

```
      B
0    NaN
1  0.986207
2  2.958621
3    NaN
4    NaN
```

****on****

Rolling sum with a window length of 2 days.

```
>>> df = pd.DataFrame(
...     {
```

```

...         "A": [
...             pd.to_datetime("2020-01-01"),
...             pd.to_datetime("2020-01-01"),
...             pd.to_datetime("2020-01-02"),
...         ],
...         "B": [1, 2, 3],
...     },
...     index=pd.date_range("2020", periods=3),
... )

```

```
>>> df
```

```

           A  B
2020-01-01  1  1
2020-01-02  2  2
2020-01-03  3  3

```

```
>>> df.rolling("2D", on="A").sum()
```

```

           A  B
2020-01-01  1.0  1.0
2020-01-02  3.0  3.0
2020-01-03  6.0  6.0

```

```

sample(
    self,
    n: int | None = None,
    frac: float | None = None,
    replace: bool = False,
    weights=None,
    random_state: RandomState | None = None,
    axis: Axis | None = None,
    ignore_index: bool = False
) -> Self

```

Return a random sample of items from an axis of object.

You can use `random\_state` for reproducibility.

Parameters

n : int, optional
Number of items from axis to return. Cannot be used with `frac`.
Default = 1 if `frac` = None.

frac : float, optional
Fraction of axis items to return. Cannot be used with `n`.

replace : bool, default False
Allow or disallow sampling of the same row more than once.

weights : str or ndarray-like, optional
Default ``None`` results in equal probability weighting.
If passed a Series, will align with target object on index. Index values in weights not found in sampled object will be ignored and index values in sampled object not in weights will be assigned weights of zero.
If called on a DataFrame, will accept the name of a column when axis = 0.
Unless weights are a Series, weights must be same length as axis being sampled.
If weights do not sum to 1, they will be normalized to sum to 1.
Missing values in the weights column will be treated as zero.
Infinite values not allowed.
When replace = False will not allow ``(n \* max(weights) / sum(weights)) > 1`` in order to avoid biased results. See the Notes below for more details.

random_state : int, array-like, BitGenerator, np.random.RandomState, np.random.Generator
If int, array-like, or BitGenerator, seed for random number generator.
If np.random.RandomState or np.random.Generator, use as given.
Default ``None`` results in sampling with the current state of np.random.

axis : {0 or 'index', 1 or 'columns', None}, default None
Axis to sample. Accepts axis number or name. Default is stat axis for given data type. For `Series` this parameter is unused and defaults to `None`.

ignore_index : bool, default False
If True, the resulting index will be labeled 0, 1, ..., n - 1.

Returns

Series or DataFrame
A new object of same type as caller containing `n` items randomly sampled from the caller object.

See Also

```

-----
DataFrameGroupBy.sample: Generates random samples from each group of a
    DataFrame object.
SeriesGroupBy.sample: Generates random samples from each group of a
    Series object.
numpy.random.choice: Generates a random sample from a given 1-D numpy
    array.

```

Notes

```

-----
If `frac` > 1, `replacement` should be set to `True`.

```

When `replace = False` will not allow `((n * max(weights) / sum(weights)) > 1)`, since that would cause results to be biased. E.g. sampling 2 items without replacement with weights `[100, 1, 1]` would yield two last items in 1/2 of cases, instead of 1/102. This is similar to specifying `n=4` without replacement on a Series with 3 elements.

Examples

```

-----
>>> df = pd.DataFrame(
...     {
...         "num_legs": [2, 4, 8, 0],
...         "num_wings": [2, 0, 0, 0],
...         "num_specimen_seen": [10, 2, 1, 8],
...     },
...     index=["falcon", "dog", "spider", "fish"],
... )
>>> df

```

	num_legs	num_wings	num_specimen_seen
falcon	2	2	10
dog	4	0	2
spider	8	0	1
fish	0	0	8

Extract 3 random elements from the `Series` `df['num_legs']`:
Note that we use `random_state` to ensure the reproducibility of the examples.

```

>>> df["num_legs"].sample(n=3, random_state=1)
fish      0
spider    8
falcon    2
Name: num_legs, dtype: int64

```

A random 50% sample of the `DataFrame` with replacement:

```

>>> df.sample(frac=0.5, replace=True, random_state=1)

```

	num_legs	num_wings	num_specimen_seen
dog	4	0	2
fish	0	0	8

An upsample sample of the `DataFrame` with replacement:
Note that `replace` parameter has to be `True` for `frac` parameter > 1.

```

>>> df.sample(frac=2, replace=True, random_state=1)

```

	num_legs	num_wings	num_specimen_seen
dog	4	0	2
fish	0	0	8
falcon	2	2	10
falcon	2	2	10
fish	0	0	8
dog	4	0	2
fish	0	0	8
dog	4	0	2

Using a DataFrame column as weights. Rows with larger value in the `num_specimen_seen` column are more likely to be sampled.

```

>>> df.sample(n=2, weights="num_specimen_seen", random_state=1)

```

	num_legs	num_wings	num_specimen_seen
falcon	2	2	10
fish	0	0	8

```

set_flags(
    self,
    *,
    copy: bool | lib.NoDefault = <no_default>,

```

```

    allows_duplicate_labels: bool | None = None
) -> Self
    Return a new object with updated flags.

    This method creates a shallow copy of the original object, preserving its
    underlying data while modifying its global flags. In particular, it allows
    you to update properties such as whether duplicate labels are permitted. This
    behavior is especially useful in method chains, where one wishes to
    adjust DataFrame or Series characteristics without altering the original object.

    Parameters
    -----
    copy : bool, default False
        This keyword is now ignored; changing its value will have no
        impact on the method.

        .. deprecated:: 3.0.0

        This keyword is ignored and will be removed in pandas 4.0. Since
        pandas 3.0, this method always returns a new object using a lazy
        copy mechanism that defers copies until necessary
        (Copy-on-Write). See the `user guide on Copy-on-Write
        <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__
        for more details.

    allows_duplicate_labels : bool, optional
        Whether the returned object allows duplicate labels.

    Returns
    -----
    Series or DataFrame
        The same type as the caller.

    See Also
    -----
    DataFrame.attrs : Global metadata applying to this dataset.
    DataFrame.flags : Global flags applying to this object.

    Notes
    -----
    This method returns a new object that's a view on the same data
    as the input. Mutating the input or the output values will be reflected
    in the other.

    This method is intended to be used in method chains.

    "Flags" differ from "metadata". Flags reflect properties of the
    pandas object (the Series or DataFrame). Metadata refer to properties
    of the dataset, and should be stored in :attr:`DataFrame.attrs`.

    Examples
    -----
    >>> df = pd.DataFrame({"A": [1, 2]})
    >>> df.flags.allows_duplicate_labels
    True
    >>> df2 = df.set_flags(allows_duplicate_labels=False)
    >>> df2.flags.allows_duplicate_labels
    False

    shift(
        self,
        periods: int | Sequence[int] = 1,
        freq=None,
        axis: Axis = 0,
        fill_value: Hashable = <no_default>,
        suffix: str | None = None
    ) -> Self | DataFrame
        Shift index by desired number of periods with an optional time `freq`.

        When `freq` is not passed, shift the index without realigning the data.
        If `freq` is passed (in this case, the index must be date or datetime,
        or it will raise a `NotImplementedError`), the index will be
        increased using the periods and the `freq`. `freq` can be inferred
        when specified as "infer" as long as either freq or inferred_freq
        attribute is set in the index.

    Parameters

```

```

-----
periods : int or Sequence
    Number of periods to shift. Can be positive or negative.
    If an iterable of ints, the data will be shifted once by each int.
    This is equivalent to shifting by one value at a time and
    concatenating all resulting frames. The resulting columns will have
    the shift suffixed to their column names. For multiple periods,
    axis must not be 1.
freq : DateOffset, tseries.offsets, timedelta, or str, optional
    Offset to use from the tseries module or time rule (e.g. 'EOM').
    If `freq` is specified then the index values are shifted but the
    data is not realigned. That is, use `freq` if you would like to
    extend the index when shifting and preserve the original data.
    If `freq` is specified as "infer" then it will be inferred from
    the freq or inferred_freq attributes of the index. If neither of
    those attributes exist, a ValueError is thrown.
axis : {{0 or 'index', 1 or 'columns', None}}, default None
    Shift direction. For `Series` this parameter is unused and defaults to 0.
fill_value : object, optional
    The scalar value to use for newly introduced missing values.
    the default depends on the dtype of `self`.
    For Boolean and numeric NumPy data types, ``np.nan`` is used.
    For datetime, timedelta, or period data, etc. :attr:`NaT` is used.
    For extension dtypes, ``self.dtype.na_value`` is used.
suffix : str, optional
    If str and periods is an iterable, this is added after the column
    name and before the shift value for each shifted column name.
    For `Series` this parameter is unused and defaults to `None`.

```

Returns

```

-----
Series/DataFrame
    Copy of input object, shifted.

```

See Also

```

-----
Index.shift : Shift values of Index.
DatetimeIndex.shift : Shift values of DatetimeIndex.
PeriodIndex.shift : Shift values of PeriodIndex.

```

Examples

```

-----
>>> df = pd.DataFrame(
...     [[10, 13, 17], [20, 23, 27], [15, 18, 22], [30, 33, 37], [45, 48, 52]],
...     columns=["Col1", "Col2", "Col3"],
...     index=pd.date_range("2020-01-01", "2020-01-05"),
... )
>>> df

```

	Col1	Col2	Col3
2020-01-01	10	13	17
2020-01-02	20	23	27
2020-01-03	15	18	22
2020-01-04	30	33	37
2020-01-05	45	48	52

```

>>> df.shift(periods=3)

```

	Col1	Col2	Col3
2020-01-01	NaN	NaN	NaN
2020-01-02	NaN	NaN	NaN
2020-01-03	NaN	NaN	NaN
2020-01-04	10.0	13.0	17.0
2020-01-05	20.0	23.0	27.0

```

>>> df.shift(periods=1, axis="columns")

```

	Col1	Col2	Col3
2020-01-01	NaN	10	13
2020-01-02	NaN	20	23
2020-01-03	NaN	15	18
2020-01-04	NaN	30	33
2020-01-05	NaN	45	48

```

>>> df.shift(periods=3, fill_value=0)

```

	Col1	Col2	Col3
2020-01-01	0	0	0
2020-01-02	0	0	0
2020-01-03	0	0	0
2020-01-04	10	13	17

```
2020-01-05    20    23    27
```

```
>>> df.shift(periods=3, freq="D")
```

```
      Col1  Col2  Col3
2020-01-04    10    13    17
2020-01-05    20    23    27
2020-01-06    15    18    22
2020-01-07    30    33    37
2020-01-08    45    48    52
```

```
>>> df.shift(periods=3, freq="infer")
```

```
      Col1  Col2  Col3
2020-01-04    10    13    17
2020-01-05    20    23    27
2020-01-06    15    18    22
2020-01-07    30    33    37
2020-01-08    45    48    52
```

```
>>> df["Col1"].shift(periods=[0, 1, 2])
```

```
      Col1_0  Col1_1  Col1_2
2020-01-01    10     NaN     NaN
2020-01-02    20    10.0     NaN
2020-01-03    15    20.0    10.0
2020-01-04    30    15.0    20.0
2020-01-05    45    30.0    15.0
```

`squeeze(self, axis: Axis | None = None) -> Scalar | Series | DataFrame`
 Squeeze 1 dimensional axis objects into scalars.

Series or DataFrames with a single element are squeezed to a scalar.
 DataFrames with a single column or a single row are squeezed to a
 Series. Otherwise the object is unchanged.

This method is most useful when you don't know if your
 object is a Series or DataFrame, but you do know it has just a single
 column. In that case you can safely call `'squeeze'` to ensure you have a
 Series.

Parameters

`axis` : {0 or 'index', 1 or 'columns', None}, default None
 A specific axis to squeeze. By default, all length-1 axes are
 squeezed. For `'Series'` this parameter is unused and defaults to `'None'`.

Returns

DataFrame, Series, or scalar
 The projection after squeezing `'axis'` or all the axes.

See Also

`Series.iloc` : Integer-location based indexing for selecting scalars.
`DataFrame.iloc` : Integer-location based indexing for selecting Series.
`Series.to_frame` : Inverse of `DataFrame.squeeze` for a
 single-column DataFrame.

Examples

```
>>> primes = pd.Series([2, 3, 5, 7])
```

Slicing might produce a Series with a single value:

```
>>> even_primes = primes[primes % 2 == 0]
>>> even_primes
0    2
dtype: int64
```

```
>>> even_primes.squeeze()
np.int64(2)
```

Squeezing objects with more than one value in every axis does nothing:

```
>>> odd_primes = primes[primes % 2 == 1]
>>> odd_primes
1    3
2    5
3    7
```

```
dtype: int64
```

```
>>> odd_primes.squeeze()
```

```
1    3
```

```
2    5
```

```
3    7
```

```
dtype: int64
```

Squeezing is even more effective when used with DataFrames.

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=["a", "b"])
```

```
>>> df
```

```
   a  b
```

```
0  1  2
```

```
1  3  4
```

Slicing a single column will produce a DataFrame with the columns having only one value:

```
>>> df_a = df[["a"]]
```

```
>>> df_a
```

```
   a
```

```
0  1
```

```
1  3
```

So the columns can be squeezed down, resulting in a Series:

```
>>> df_a.squeeze("columns")
```

```
0    1
```

```
1    3
```

```
Name: a, dtype: int64
```

Slicing a single row from a single column will produce a single scalar DataFrame:

```
>>> df_0a = df.loc[df.index < 1, ["a"]]
```

```
>>> df_0a
```

```
   a
```

```
0  1
```

Squeezing the rows produces a single scalar Series:

```
>>> df_0a.squeeze("rows")
```

```
a    1
```

```
Name: 0, dtype: int64
```

Squeezing all axes will project directly into a scalar:

```
>>> df_0a.squeeze()
```

```
np.int64(1)
```

```
tail(self, n: int = 5) -> Self
Return the last `n` rows.
```

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to ``df[|n|:]``.

If ``n`` is larger than the number of rows, this function returns all rows.

Parameters

n : int, default 5
Number of rows to select.

Returns

type of caller
The last `n` rows of the caller object.

See Also

DataFrame.head : The first `n` rows of the caller object.

Examples

```
-----
>>> df = pd.DataFrame(
...     {
...         "animal": [
...             "alligator",
...             "bee",
...             "falcon",
...             "lion",
...             "monkey",
...             "parrot",
...             "shark",
...             "whale",
...             "zebra",
...         ]
...     }
... )
>>> df
   animal
0  alligator
1     bee
2   falcon
3     lion
4   monkey
5   parrot
6    shark
7    whale
8    zebra
```

Viewing the last 5 lines

```
>>> df.tail()
   animal
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

Viewing the last `n` lines (three in this case)

```
>>> df.tail(3)
   animal
6  shark
7  whale
8  zebra
```

For negative values of `n`

```
>>> df.tail(-3)
   animal
3    lion
4  monkey
5  parrot
6   shark
7   whale
8   zebra
```

`take(self, indices, axis: Axis = 0, **kwargs) -> Self`
Return the elements in the given *positional* indices along an axis.

This means that we are not indexing according to actual values in the index attribute of the object. We are indexing according to the actual position of the element in the object.

Parameters

`indices` : array-like

An array of ints indicating which positions to take.

`axis` : {0 or 'index', 1 or 'columns'}, default 0

The axis on which to select elements. ``0`` means that we are selecting rows, ``1`` means that we are selecting columns.

For `Series` this parameter is unused and defaults to 0.

`**kwargs`

For compatibility with `:meth:`numpy.take``. Has no effect on the output.

Returns

same type as caller

An array-like containing the elements taken from the object.

See Also

DataFrame.loc : Select a subset of a DataFrame by labels.

DataFrame.iloc : Select a subset of a DataFrame by positions.

numpy.take : Take elements from an array along an axis.

Examples

```
>>> df = pd.DataFrame(
...     [
...         ("falcon", "bird", 389.0),
...         ("parrot", "bird", 24.0),
...         ("lion", "mammal", 80.5),
...         ("monkey", "mammal", np.nan),
...     ],
...     columns=["name", "class", "max_speed"],
...     index=[0, 2, 3, 1],
... )
>>> df
   name  class  max_speed
0  falcon   bird    389.0
2  parrot   bird     24.0
3   lion  mammal     80.5
1  monkey  mammal      NaN
```

Take elements at positions 0 and 3 along the axis 0 (default).

Note how the actual indices selected (0 and 1) do not correspond to our selected indices 0 and 3. That's because we are selecting the 0th and 3rd rows, not rows whose indices equal 0 and 3.

```
>>> df.take([0, 3])
   name  class  max_speed
0  falcon   bird    389.0
1  monkey  mammal      NaN
```

Take elements at indices 1 and 2 along the axis 1 (column selection).

```
>>> df.take([1, 2], axis=1)
   class  max_speed
0   bird    389.0
2   bird     24.0
3  mammal     80.5
1  mammal      NaN
```

We may take elements using negative integers for positive indices, starting from the end of the object, just like with Python lists.

```
>>> df.take([-1, -2])
   name  class  max_speed
1  monkey  mammal      NaN
3   lion  mammal     80.5
```

to_clipboard(self, *, excel: bool = True, sep: str | None = None, **kwargs) -> None
Copy object to the system clipboard.

Write a text representation of object to the system clipboard.
This can be pasted into Excel, for example.

Parameters

excel : bool, default True

Produce output in a csv format for easy pasting into excel.

- True, use the provided separator for csv pasting.

- False, write a string representation of the object to the clipboard.

sep : str, default ``\t``

Field delimiter.

**kwargs

These parameters will be passed to DataFrame.to_csv.

See Also

DataFrame.to_csv : Write a DataFrame to a comma-separated values (csv) file.
read_clipboard : Read text from clipboard and pass to read_csv.

Notes

Requirements for your platform.

- Linux : `xclip`, or `xsel` (with `PyQt4` modules)
- Windows : none
- macOS : none

This method uses the processes developed for the package `pyperclip`. A solution to render any output string format is given in the examples.

Examples

Copy the contents of a DataFrame to the clipboard.

```
>>> df = pd.DataFrame([[1, 2, 3], [4, 5, 6]], columns=["A", "B", "C"])
```

```
>>> df.to_clipboard(sep=",") # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # ,A,B,C
... # 0,1,2,3
... # 1,4,5,6
```

We can omit the index by passing the keyword `index` and setting it to false.

```
>>> df.to_clipboard(sep=",", index=False) # doctest: +SKIP
... # Wrote the following to the system clipboard:
... # A,B,C
... # 1,2,3
... # 4,5,6
```

Using the original `pyperclip` package for any string output format.

```
.. code-block:: python
```

```
import pyperclip

html = df.style.to_html()
pyperclip.copy(html)
```

```
to_csv(
    self,
    path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
    *,
    sep: str = ',',
    na_rep: str = '',
    float_format: str | Callable | None = None,
    columns: Sequence[Hashable] | None = None,
    header: bool | list[str] = True,
    index: bool = True,
    index_label: IndexLabel | None = None,
    mode: str = 'w',
    encoding: str | None = None,
    compression: CompressionOptions = 'infer',
    quoting: int | None = None,
    quotechar: str = '"',
    lineterminator: str | None = None,
    chunksize: int | None = None,
    date_format: str | None = None,
    doublequote: bool = True,
    escapechar: str | None = None,
    decimal: str = '.',
    errors: OpenFileErrors = 'strict',
    storage_options: StorageOptions | None = None
) -> str | None
Write object to a comma-separated values (csv) file.
```

Parameters

path_or_buf : str, path object, file-like object, or None, default None

String, path object (implementing `os.PathLike[str]`), or file-like object implementing a `write()` function. If `None`, the result is returned as a string. If a non-binary file object is passed, it should be opened with `'newline=''`, disabling universal newlines. If a binary file object is passed, `'mode'` might need to contain a `'b'`.

`sep` : str, default `','`
String of length 1. Field delimiter for the output file.

`na_rep` : str, default `''`
Missing data representation.

`float_format` : str, Callable, default `None`
Format string for floating point numbers. If a Callable is given, it takes precedence over other numeric formatting parameters, like decimal.

`columns` : sequence, optional
Columns to write.

`header` : bool or list of str, default `True`
Write out the column names. If a list of strings is given it is assumed to be aliases for the column names.

`index` : bool, default `True`
Write row names (index).

`index_label` : str or sequence, or `False`, default `None`
Column label for index column(s) if desired. If `None` is given, and `'header'` and `'index'` are `True`, then the index names are used. A sequence should be given if the object uses `MultiIndex`. If `False` do not print fields for index names. Use `index_label=False` for easier importing in R.

`mode` : `{'w', 'x', 'a'}`, default `'w'`
Forwarded to either `'open(mode=)'` or `'fsspec.open(mode=)'` to control the file opening. Typical values include:

- `'w'`, truncate the file first.
- `'x'`, exclusive creation, failing if the file already exists.
- `'a'`, append to the end of file if it exists.

`encoding` : str, optional
A string representing the encoding to use in the output file, defaults to `'utf-8'`. `'encoding'` is not supported if `'path_or_buf'` is a non-binary file object.

`compression` : str or dict, default `'infer'`
For on-the-fly compression of the output data. If `'infer'` and `'path_or_buf'` is path-like, then detect compression from the following extensions: `'.gz'`, `'.bz2'`, `'.zip'`, `'.xz'`, `'.zst'`, `'.tar'`, `'.tar.gz'`, `'.tar.xz'` or `'.tar.bz2'` (otherwise no compression). Set to `'None'` for no compression. Can also be a dict with key `'method'` set to one of `{'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'}` and other key-value pairs are forwarded to `'zipfile.ZipFile'`, `'gzip.GzipFile'`, `'bz2.BZ2File'`, `'zstandard.ZstdCompressor'`, `'lzma.LZMAFile'` or `'tarfile.TarFile'`, respectively. As an example, the following could be passed for faster compression and to create a reproducible gzip archive:
`'compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}'`.

May be a dict with key `'method'` as compression mode and other entries as additional compression options if compression mode is `'zip'`.

Passing compression options as keys in dict is supported for compression modes `'gzip'`, `'bz2'`, `'zstd'`, and `'zip'`.

`quoting` : optional constant from csv module
Defaults to `csv.QUOTE_MINIMAL`. If you have set a `'float_format'` then floats are converted to strings and thus `csv.QUOTE_NONNUMERIC` will treat them as non-numeric.

`quotechar` : str, default `'\"'`
String of length 1. Character used to quote fields.

`lineterminator` : str, optional
The newline character or character sequence to use in the output file. Defaults to `'os.linesep'`, which depends on the OS in which this method is called (`'\\n'` for linux, `'\\r\\n'` for Windows, i.e.).

`chunksize` : int or `None`
Rows to write at a time.

`date_format` : str, default `None`
Format string for datetime objects.

`doublequote` : bool, default `True`
Control quoting of `'quotechar'` inside a field.

`escapechar` : str, default None
 String of length 1. Character used to escape ``sep`` and ``quotechar`` when appropriate.
`decimal` : str, default `'.'`
 Character recognized as decimal separator. E.g. use `'.'` for European data.
`errors` : str, default `'strict'`
 Specifies how encoding and decoding errors are to be handled. See the errors argument for `:func:`open`` for a full list of options.

`storage_options` : dict, optional
 Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with `"s3://"`, and `"gcs://"`) the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer ``here`` https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

Returns

None or str

If `path_or_buf` is None, returns the resulting csv format as a string. Otherwise returns None.

See Also

`read_csv` : Load a CSV file into a DataFrame.

`to_excel` : Write DataFrame to an Excel file.

Examples

Create 'out.csv' containing 'df' without indices

```
>>> df = pd.DataFrame(
...     [
...         ["Raphael", "red", "sai"], ["Donatello", "purple", "bo staff"]],
...     columns=["name", "mask", "weapon"],
... )
>>> df.to_csv("out.csv", index=False) # doctest: +SKIP
```

Create 'out.zip' containing 'out.csv'

```
>>> df.to_csv(index=False)
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
>>> compression_opts = dict(
...     method="zip", archive_name="out.csv"
... ) # doctest: +SKIP
>>> df.to_csv(
...     "out.zip", index=False, compression=compression_opts
... ) # doctest: +SKIP
```

To write a csv file to a new folder or nested folder you will first need to create it using either Pathlib or os:

```
>>> from pathlib import Path # doctest: +SKIP
>>> filepath = Path("folder/subfolder/out.csv") # doctest: +SKIP
>>> filepath.parent.mkdir(parents=True, exist_ok=True) # doctest: +SKIP
>>> df.to_csv(filepath) # doctest: +SKIP

>>> import os # doctest: +SKIP
>>> os.makedirs("folder/subfolder", exist_ok=True) # doctest: +SKIP
>>> df.to_csv("folder/subfolder/out.csv") # doctest: +SKIP
```

Format floats to two decimal places:

```
>>> df.to_csv("out1.csv", float_format="%.2f") # doctest: +SKIP
```

Format floats using scientific notation:

```
>>> df.to_csv("out2.csv", float_format="{:.2e}").format) # doctest: +SKIP
```

```
to_excel(
    self,
    excel_writer: FilePath | WriteExcelBuffer | ExcelWriter,
    *,
```

```

sheet_name: str = 'Sheet1',
na_rep: str = '',
float_format: str | None = None,
columns: Sequence[Hashable] | None = None,
header: Sequence[Hashable] | bool = True,
index: bool = True,
index_label: IndexLabel | None = None,
startrow: int = 0,
startcol: int = 0,
engine: Literal['openpyxl', 'xlsxwriter'] | None = None,
merge_cells: bool = True,
inf_rep: str = 'inf',
freeze_panes: tuple[int, int] | None = None,
storage_options: StorageOptions | None = None,
engine_kwargs: dict[str, Any] | None = None,
autofilter: bool = False
) -> None
Write object to an Excel sheet.

```

To write a single object to an Excel .xlsx file it is only necessary to specify a target file name. To write to multiple sheets it is necessary to create an `ExcelWriter` object with a target file name, and specify a sheet in the file to write to.

Multiple sheets may be written to by specifying unique `sheet_name`. With all data written to the file it is necessary to save the changes. Note that creating an `ExcelWriter` object with a file name that already exists will overwrite the existing file because the default mode is write.

Parameters

```

-----
excel_writer : path-like, file-like, or ExcelWriter object
    File path or existing ExcelWriter.
sheet_name : str, default 'Sheet1'
    Name of sheet which will contain DataFrame.
na_rep : str, default ''
    Missing data representation.
float_format : str, optional
    Format string for floating point numbers. For example
    ``float_format="%2f"`` will format 0.1234 to 0.12.
columns : sequence or list of str, optional
    Columns to write.
header : bool or list of str, default True
    Write out the column names. If a list of string is given it is
    assumed to be aliases for the column names.
index : bool, default True
    Write row names (index).
index_label : str or sequence, optional
    Column label for index column(s) if desired. If not specified, and
    `header` and `index` are True, then the index names are used. A
    sequence should be given if the DataFrame uses MultiIndex.
startrow : int, default 0
    Upper left cell row to dump data frame.
startcol : int, default 0
    Upper left cell column to dump data frame.
engine : str, optional
    Write engine to use, 'openpyxl' or 'xlsxwriter'. You can also set this
    via the options ``io.excel.xlsx.writer`` or
    ``io.excel.xlsm.writer``.
merge_cells : bool or 'columns', default False
    If True, write MultiIndex index and columns as merged cells.
    If 'columns', merge MultiIndex column cells only.
inf_rep : str, default 'inf'
    Representation for infinity (there is no native representation for
    infinity in Excel).
freeze_panes : tuple of int (length 2), optional
    Specifies the one-based bottommost row and rightmost column that
    is to be frozen.
storage_options : dict, optional
    Extra options that make sense for a particular storage connection, e.g.
    host, port, username, password, etc. For HTTP(S) URLs the key-value pairs
    are forwarded to ``urllib.request.Request`` as header options. For other
    URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are
    forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more
    details, and for more examples on storage options refer `here
    <https://pandas.pydata.org/docs/user_guide/io.html?
    highlight=storage_options#reading-writing-remote-files>`_.

```

engine_kwarg : dict, optional
Arbitrary keyword arguments passed to excel engine.
autofilter : bool, default False
If True, add automatic filters to all columns.

See Also

to_csv : Write DataFrame to a comma-separated values (csv) file.
ExcelWriter : Class for writing DataFrame objects into excel sheets.
read_excel : Read an Excel file into a pandas DataFrame.
read_csv : Read a comma-separated values (csv) file into DataFrame.
io.formats.style.Styler.to_excel : Add styles to Excel sheet.

Notes

For compatibility with :meth:`DataFrame.to\_csv`,
to_excel serializes lists and dicts to strings before writing.

Once a workbook has been saved it is not possible to write further
data without rewriting the whole workbook.

pandas will check the number of rows, columns,
and cell character count does not exceed Excel's limitations.
All other limitations must be checked by the user.

Examples

Create, write to and save a workbook:

```
>>> df1 = pd.DataFrame(  
...     [ ["a", "b"], ["c", "d"] ],  
...     index=["row 1", "row 2"],  
...     columns=["col 1", "col 2"],  
... )  
>>> df1.to_excel("output.xlsx") # doctest: +SKIP
```

To specify the sheet name:

```
>>> df1.to_excel("output.xlsx", sheet_name="Sheet_name_1") # doctest: +SKIP
```

If you wish to write to more than one sheet in the workbook, it is
necessary to specify an ExcelWriter object:

```
>>> df2 = df1.copy()  
>>> with pd.ExcelWriter("output.xlsx") as writer: # doctest: +SKIP  
...     df1.to_excel(writer, sheet_name="Sheet_name_1")  
...     df2.to_excel(writer, sheet_name="Sheet_name_2")
```

ExcelWriter can also be used to append to an existing Excel file:

```
>>> with pd.ExcelWriter("output.xlsx", mode="a") as writer: # doctest: +SKIP  
...     df1.to_excel(writer, sheet_name="Sheet_name_3")
```

To set the library that is used to write the Excel file,
you can pass the `engine` keyword (the default engine is
automatically chosen depending on the file extension):

```
>>> df1.to_excel("output1.xlsx", engine="xlsxwriter") # doctest: +SKIP
```

```
to_hdf(  
    self,  
    path_or_buf: FilePath | HDFStore,  
    *,  
    key: str,  
    mode: Literal['a', 'w', 'r+'] = 'a',  
    complevel: int | None = None,  
    complib: Literal['zlib', 'lzo', 'bzip2', 'blosc'] | None = None,  
    append: bool = False,  
    format: Literal['fixed', 'table'] | None = None,  
    index: bool = True,  
    min_itemsize: int | dict[str, int] | None = None,  
    nan_rep=None,  
    dropna: bool | None = None,  
    data_columns: Literal[True] | list[str] | None = None,  
    errors: OpenFileErrors = 'strict',
```

```

    encoding: str = 'UTF-8'
) -> None
    Write the contained data to an HDF5 file using HDFStore.

Hierarchical Data Format (HDF) is self-describing, allowing an
application to interpret the structure and contents of a file with
no outside information. One HDF file can hold a mix of related objects
which can be accessed as a group or as individual objects.

In order to add another DataFrame or Series to an existing HDF file
please use append mode and a different a key.

.. warning::

    One can store a subclass of ``DataFrame`` or ``Series`` to HDF5,
    but the type of the subclass is lost upon storing.

For more information see the :ref:`user guide <io.hdf5>`.

Parameters
-----
path_or_buf : str or pandas.HDFStore
    File path or HDFStore object.
key : str
    Identifier for the group in the store.
mode : {'a', 'w', 'r+'}, default 'a'
    Mode to open file:

    - 'w': write, a new file is created (an existing file with
      the same name would be deleted).
    - 'a': append, an existing file is opened for reading and
      writing, and if the file does not exist it is created.
    - 'r+': similar to 'a', but the file must already exist.
complevel : {0-9}, default None
    Specifies a compression level for data.
    A value of 0 or None disables compression.
complib : {'zlib', 'lzo', 'bzip2', 'blosc'}, default 'zlib'
    Specifies the compression library to be used.
    These additional compressors for Blosc are supported
    (default if no compressor specified: 'blosc:blosclz'):
    {'blosc:blosclz', 'blosc:lz4', 'blosc:lz4hc', 'blosc:snappy',
     'blosc:zlib', 'blosc:zstd'}.
    Specifying a compression library which is not available issues
    a ValueError.
append : bool, default False
    For Table formats, append the input data to the existing.
format : {'fixed', 'table', None}, default 'fixed'
    Possible values:

    - 'fixed': Fixed format. Fast writing/reading. Not-appendable,
      nor searchable.
    - 'table': Table format. Write as a PyTables Table structure
      which may perform worse but allow more flexible operations
      like searching / selecting subsets of the data.
    - If None, pd.get_option('io.hdf.default_format') is checked,
      followed by fallback to "fixed".
index : bool, default True
    Write DataFrame index as a column.
min_itemsize : dict or int, optional
    Map column names to minimum string sizes for columns.
nan_rep : Any, optional
    How to represent null values as str.
    Not allowed with append=True.
dropna : bool, default False, optional
    Remove missing values.
data_columns : list of columns or True, optional
    List of columns to create as indexed data columns for on-disk
    queries, or True to use all columns. By default only the axes
    of the object are indexed. See
    :ref:`Query via data columns<io.hdf5-query-data-columns>`. for
    more information.
    Applicable only to format='table'.
errors : str, default 'strict'
    Specifies how encoding and decoding errors are to be handled.
    See the errors argument for :func:`open` for a full list
    of options.
encoding : str, default "UTF-8"

```

Set character encoding.

See Also

```
-----
read_hdf : Read from HDF file.
DataFrame.to_orc : Write a DataFrame to the binary orc format.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.
DataFrame.to_sql : Write to a SQL table.
DataFrame.to_feather : Write out feather-format for DataFrames.
DataFrame.to_csv : Write out to a csv file.
```

Examples

```
-----
>>> df = pd.DataFrame(
...     {"A": [1, 2, 3], "B": [4, 5, 6]}, index=["a", "b", "c"]
... ) # doctest: +SKIP
>>> df.to_hdf("data.h5", key="df", mode="w") # doctest: +SKIP
```

We can add another object to the same file:

```
>>> s = pd.Series([1, 2, 3, 4]) # doctest: +SKIP
>>> s.to_hdf("data.h5", key="s") # doctest: +SKIP
```

Reading from HDF file:

```
>>> pd.read_hdf("data.h5", "df") # doctest: +SKIP
A B
a 1 4
b 2 5
c 3 6
>>> pd.read_hdf("data.h5", "s") # doctest: +SKIP
0    1
1    2
2    3
3    4
dtype: int64
```

```
to_json(
    self,
    path_or_buf: FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None = None,
    *,
    orient: Literal['split', 'records', 'index', 'table', 'columns', 'values'] | None = None,
    date_format: str | None = None,
    double_precision: int = 10,
    force_ascii: bool = True,
    date_unit: TimeUnit = 'ms',
    default_handler: Callable[[Any], JSONSerializable] | None = None,
    lines: bool = False,
    compression: CompressionOptions = 'infer',
    index: bool | None = None,
    indent: int | None = None,
    storage_options: StorageOptions | None = None,
    mode: Literal['a', 'w'] = 'w'
) -> str | None
    Convert the object to a JSON string.
```

Note NaN's and None will be converted to null and datetime objects will be converted to UNIX timestamps.

Parameters

```
-----
path_or_buf : str, path object, file-like object, or None, default None
    String, path object (implementing os.PathLike[str]), or file-like
    object implementing a write() function. If None, the result is
    returned as a string.
orient : str
    Indication of expected JSON string format.
```

* Series:

- default is 'index'
- allowed values are: {'split', 'records', 'index', 'table'}.

* DataFrame:

- default is 'columns'
- allowed values are: {'split', 'records', 'index', 'columns',


```
'values', 'table'}}.
```

* The format of the JSON string:

- 'split' : dict like {{'index' -> [index], 'columns' -> [columns], 'data' -> [values]}}
- 'records' : list like [{column -> value}, ... , {column -> value}]
- 'index' : dict like {{index -> {{column -> value}}}}
- 'columns' : dict like {{column -> {{index -> value}}}}
- 'values' : just the values array
- 'table' : dict like {{'schema': {{schema}}, 'data': {{data}}}}

Describing the data, where data component is like ``orient='records'``.

date_format : {{None, 'epoch', 'iso'}}

Type of date conversion. 'epoch' = epoch milliseconds, 'iso' = ISO8601. The default depends on the 'orient'. For ``orient='table'``, the default is 'iso'. For all other orients, the default is 'epoch'.

.. deprecated:: 3.0.0

'epoch' date format is deprecated and will be removed in a future version, please use 'iso' instead.

double_precision : int, default 10

The number of decimal places to use when encoding floating point values. The possible maximal value is 15. Passing double_precision greater than 15 will raise a ValueError.

force_ascii : bool, default True

Force encoded string to be ASCII.

date_unit : str, default 'ms' (milliseconds)

The time unit to encode to, governs timestamp and ISO8601 precision. One of 's', 'ms', 'us', 'ns' for second, millisecond, microsecond, and nanosecond respectively.

default_handler : callable, default None

Handler to call if object cannot otherwise be converted to a suitable format for JSON. Should receive a single argument which is the object to convert and return a serialisable object.

lines : bool, default False

If 'orient' is 'records' write out line-delimited json format. Will throw ValueError if incorrect 'orient' since others are not list-like.

compression : str or dict, default 'infer'

For on-the-fly compression of the output data. If 'infer' and 'path_or_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).

Set to ``None`` for no compression.

Can also be a dict with key ``'method'`` set to one of

{{'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'}} and other key-value pairs are forwarded to

``zipfile.ZipFile``, ``gzip.GzipFile``, ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or ``tarfile.TarFile``, respectively.

As an example, the following could be passed for faster compression and to create a reproducible gzip archive:

``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}```.

index : bool or None, default None

The index is only used when 'orient' is 'split', 'index', 'column', or 'table'. Of these, 'index' and 'column' do not support ``index=False``. The string 'index' as a column name with empty :class:`Index` or if it is 'index' will raise a ``ValueError``.

indent : int, optional

Length of whitespace used to indent each record.

storage_options : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?)

```

highlight=storage_options#reading-writing-remote-files>`_.
mode : str, default 'w' (writing)
Specify the IO mode for output when supplying a path_or_buf.
Accepted args are 'w' (writing) and 'a' (append) only.
mode='a' is only supported when lines is True and orient is 'records'.

```

Returns

None or str

If path_or_buf is None, returns the resulting json format as a string. Otherwise returns None.

See Also

read_json : Convert a JSON string to pandas object.

Notes

The behavior of ``indent=0`` varies from the stdlib, which does not indent the output but does insert newlines. Currently, ``indent=0`` and the default ``indent=None`` are equivalent in pandas, though this may change in a future release.

``orient='table'`` contains a 'pandas_version' field under 'schema'. This stores the version of 'pandas' used in the latest revision of the schema.

Examples

```

>>> from json import loads, dumps
>>> df = pd.DataFrame(
...     [{"a", "b"}, {"c", "d"}],
...     index=["row 1", "row 2"],
...     columns=["col 1", "col 2"],
... )

>>> result = df.to_json(orient="split")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "columns": [
    "col 1",
    "col 2"
  ],
  "index": [
    "row 1",
    "row 2"
  ],
  "data": [
    [
      "a",
      "b"
    ],
    [
      "c",
      "d"
    ]
  ]
}}

```

Encoding/decoding a Dataframe using ``'records'`` formatted JSON. Note that index labels are not preserved with this encoding.

```

>>> result = df.to_json(orient="records")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
  {{
    "col 1": "a",
    "col 2": "b"
  }},
  {{
    "col 1": "c",
    "col 2": "d"
  }}
]

```

Encoding/decoding a Dataframe using ``'index'`` formatted JSON:

```
>>> result = df.to_json(orient="index")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "row 1": {{
    "col 1": "a",
    "col 2": "b"
  }},
  "row 2": {{
    "col 1": "c",
    "col 2": "d"
  }}
}}
```

Encoding/decoding a Dataframe using ``'columns'`` formatted JSON:

```
>>> result = df.to_json(orient="columns")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "col 1": {{
    "row 1": "a",
    "row 2": "c"
  }},
  "col 2": {{
    "row 1": "b",
    "row 2": "d"
  }}
}}
```

Encoding/decoding a Dataframe using ``'values'`` formatted JSON:

```
>>> result = df.to_json(orient="values")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
[
  [
    "a",
    "b"
  ],
  [
    "c",
    "d"
  ]
]
```

Encoding with Table Schema:

```
>>> result = df.to_json(orient="table")
>>> parsed = loads(result)
>>> dumps(parsed, indent=4) # doctest: +SKIP
{{
  "schema": {{
    "fields": [
      {{
        "name": "index",
        "type": "string"
      }},
      {{
        "name": "col 1",
        "type": "string"
      }},
      {{
        "name": "col 2",
        "type": "string"
      }}
    ],
    "primaryKey": [
      "index"
    ],
    "pandas_version": "1.4.0"
  }},
  "data": [
    {{

```

```

        "index": "row 1",
        "col 1": "a",
        "col 2": "b"
    }},
    {{
        "index": "row 2",
        "col 1": "c",
        "col 2": "d"
    }}
]
}}

to_latex(
    self,
    buf: FilePath | WriteBuffer[str] | None = None,
    *,
    columns: Sequence[Hashable] | None = None,
    header: bool | SequenceNotStr[str] = True,
    index: bool = True,
    na_rep: str = 'NaN',
    formatters: FormattersType | None = None,
    float_format: FloatFormatType | None = None,
    sparsify: bool | None = None,
    index_names: bool = True,
    bold_rows: bool = False,
    column_format: str | None = None,
    longtable: bool | None = None,
    escape: bool | None = None,
    encoding: str | None = None,
    decimal: str = '.',
    multicolumn: bool | None = None,
    multicolumn_format: str | None = None,
    multirow: bool | None = None,
    caption: str | tuple[str, str] | None = None,
    label: str | None = None,
    position: str | None = None
) -> str | None
    Render object to a LaTeX tabular, longtable, or nested table.

Requires ``\usepackage{booktabs}``. The output can be copy/pasted
into a main LaTeX document or read from an external file
with ``\input{table.tex}``.

.. versionchanged:: 2.0.0
    Refactored to use the Styler implementation via jinja2 templating.

Parameters
-----
buf : str, Path or StringIO-like, optional, default None
    Buffer to write to. If None, the output is returned as a string.
columns : list of label, optional
    The subset of columns to write. Writes all columns by default.
header : bool or list of str, default True
    Write out the column names. If a list of strings is given,
    it is assumed to be aliases for the column names. Braces must be escaped.
index : bool, default True
    Write row names (index).
na_rep : str, default 'NaN'
    Missing data representation.
formatters : list of functions or dict of {str: function}, optional
    Formatter functions to apply to columns' elements by position or
    name. The result of each function must be a unicode string.
    List must be of length equal to the number of columns.
float_format : one-parameter function or str, optional, default None
    Formatter for floating point numbers. For example
    ``float_format="%0.2f"`` and ``float_format="{:0.2f}".format`` will
    both result in 0.1234 being formatted as 0.12.
sparsify : bool, optional
    Set to False for a DataFrame with a hierarchical index to print
    every multiindex key at each row. By default, the value will be
    read from the config module.
index_names : bool, default True
    Prints the names of the indexes.
bold_rows : bool, default False
    Make the row labels bold in the output.
column_format : str, optional
    The columns format as specified in `LaTeX table format

```

<<https://en.wikibooks.org/wiki/LaTeX/Tables>>`__ e.g. 'rcl' for 3 columns. By default, 'l' will be used for all columns except columns of numbers, which default to 'r'.

longtable : bool, optional
 Use a longtable environment instead of tabular. Requires adding a `\usepackage{longtable}` to your LaTeX preamble. By default, the value will be read from the pandas config module, and set to ``True`` if the option ```styler.latex.environment``` is ```longtable```.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed.

escape : bool, optional
 By default, the value will be read from the pandas config module and set to ``True`` if the option ```styler.format.escape``` is ```latex```. When set to False prevents from escaping latex special characters in column names.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed, as has the default value to ``False``.

encoding : str, optional
 A string representing the encoding to use in the output file, defaults to 'utf-8'.

decimal : str, default '.'
 Character recognized as decimal separator, e.g. ',' in Europe.

multicolumn : bool, default True
 Use `\multicolumn` to enhance MultiIndex columns. The default will be read from the config module, and is set as the option ```styler.sparse.columns```.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed.

multicolumn_format : str, default 'r'
 The alignment for multicolumns, similar to ``column_format``. The default will be read from the config module, and is set as the option ```styler.latex.multicol_align```.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed, as has the default value to `"r"`.

multirow : bool, default True
 Use `\multirow` to enhance MultiIndex rows. Requires adding a `\usepackage{multirow}` to your LaTeX preamble. Will print centered labels (instead of top-aligned) across the contained rows, separating groups via clines. The default will be read from the pandas config module, and is set as the option ```styler.sparse.index```.

.. versionchanged:: 2.0.0
 The pandas option affecting this argument has changed, as has the default value to ``True``.

caption : str or tuple, optional
 Tuple (full_caption, short_caption), which results in ```\caption[short_caption]{full_caption}```; if a single string is passed, no short caption will be set.

label : str, optional
 The LaTeX label to be placed inside ```\label{}``` in the output. This is used with ```\ref{}``` in the main ```.tex``` file.

position : str, optional
 The LaTeX positional argument for tables, to be placed after ```\begin{}``` in the output.

Returns

 str or None
 If buf is None, returns the result as a string. Otherwise returns None.

See Also

io.formats.style.Styler.to_latex : Render a DataFrame to LaTeX with conditional formatting.
DataFrame.to_string : Render a DataFrame to a console-friendly tabular output.
DataFrame.to_html : Render a DataFrame as an HTML table.

Notes

As of v2.0.0 this method has changed to use the Styler implementation as part of :meth:`Styler.to\_latex` via ``jinja2`` templating. This means that ``jinja2`` is a requirement, and needs to be installed, for this method to function. It is advised that users switch to using Styler, since that implementation is more frequently updated and contains much more flexibility with the output.

Examples

Convert a general DataFrame to LaTeX with formatting:

```
>>> df = pd.DataFrame(dict(name=['Raphael', 'Donatello'],
...                          age=[26, 45],
...                          height=[181.23, 177.65]))
>>> print(df.to_latex(index=False,
...                    formatters={"name": str.upper},
...                    float_format="{:.1f}".format,
...                    )) # doctest: +SKIP
\begin{tabular}{lrr}
\toprule
name & age & height \\
\midrule
RAPHAEL & 26 & 181.2 \\
DONATELLO & 45 & 177.7 \\
\bottomrule
\end{tabular}
```

```
to_pickle(
    self,
    path: FilePath | WriteBuffer[bytes],
    *,
    compression: CompressionOptions = 'infer',
    protocol: int = 5,
    storage_options: StorageOptions | None = None
) -> None
Pickle (serialize) object to file.
```

Parameters

path : str, path object, or file-like object
String, path object (implementing ``os.PathLike[str]``), or file-like object implementing a binary ``write()`` function. File path where the pickled object will be stored.

compression : str or dict, default 'infer'
For on-the-fly compression of the output data. If 'infer' and 'path_or_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).
Set to ``None`` for no compression.
Can also be a dict with key ``method`` set to one of {'zip', 'gzip', 'bz2', 'zstd', 'xz', 'tar'} and other key-value pairs are forwarded to ``zipfile.ZipFile``, ``gzip.GzipFile``, ``bz2.BZ2File``, ``zstandard.ZstdCompressor``, ``lzma.LZMAFile`` or ``tarfile.TarFile``, respectively.
As an example, the following could be passed for faster compression and to create a reproducible gzip archive:
``compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}``.

protocol : int
Int which indicates which protocol should be used by the pickler, default HIGHEST_PROTOCOL (see [1]_ paragraph 12.1.2). The possible values are 0, 1, 2, 3, 4, 5. A negative value for the protocol parameter is equivalent to setting its value to HIGHEST_PROTOCOL.

.. [1] <https://docs.python.org/3/library/pickle.html>.

storage_options : dict, optional
Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more

details, and for more examples on storage options refer ``here``
<[https://pandas.pydata.org/docs/user_guide/io.html?](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files)
`highlight=storage_options#reading-writing-remote-files``>`_.

See Also

`read_pickle` : Load pickled pandas object (or any object) from file.
`DataFrame.to_hdf` : Write DataFrame to an HDF5 file.
`DataFrame.to_sql` : Write DataFrame to a SQL database.
`DataFrame.to_parquet` : Write a DataFrame to the binary parquet format.

Examples

```
>>> original_df = pd.DataFrame(
...     [{"foo": range(5), "bar": range(5, 10)}]
... ) # doctest: +SKIP
>>> original_df # doctest: +SKIP
   foo  bar
0    0    5
1    1    6
2    2    7
3    3    8
4    4    9
>>> original_df.to_pickle("./dummy.pkl") # doctest: +SKIP
```

```
>>> unpickled_df = pd.read_pickle("./dummy.pkl") # doctest: +SKIP
>>> unpickled_df # doctest: +SKIP
   foo  bar
0    0    5
1    1    6
2    2    7
3    3    8
4    4    9
```

```
to_sql(
    self,
    name: str,
    con,
    *,
    schema: str | None = None,
    if_exists: Literal['fail', 'replace', 'append', 'delete_rows'] = 'fail',
    index: bool = True,
    index_label: IndexLabel | None = None,
    chunksize: int | None = None,
    dtype: DtypeArg | None = None,
    method: Literal['multi'] | Callable | None = None
) -> int | None
Write records stored in a DataFrame to a SQL database.
```

Databases supported by SQLAlchemy [1]_ are supported. Tables can be newly created, appended to, or overwritten.

.. warning::

The pandas library does not attempt to sanitize inputs provided via a `to_sql` call. Please refer to the documentation for the underlying database driver to see if it will properly prevent injection, or alternatively be advised of a security risk when executing arbitrary commands in a `to_sql` call.

Parameters

`name` : str

Name of SQL table.

`con` : ADBC connection, `sqlalchemy.engine.(Engine or Connection)` or `sqlite3.Connection`
ADBC provides high performance I/O with native type support, where available. Using SQLAlchemy makes it possible to use any DB supported by that library. Legacy support is provided for `sqlite3.Connection` objects. The user is responsible for engine disposal and connection closure for the SQLAlchemy connectable. See ``here`` <<https://docs.sqlalchemy.org/en/20/core/connections.html>>
If passing a `sqlalchemy.engine.Connection` which is already in a transaction, the transaction will not be committed. If passing a `sqlite3.Connection`, it will not be possible to roll back the record insertion.

`schema` : str, optional

Specify the schema (if database flavor supports this). If None, use default schema.

`if_exists` : {'fail', 'replace', 'append', 'delete_rows'}, default 'fail'
How to behave if the table already exists.

- * fail: Raise a ValueError.
- * replace: Drop the table before inserting new values.
- * append: Insert new values to the existing table.
- * delete_rows: If a table exists, delete all records and insert data.

index : bool, default True
Write DataFrame index as a column. Uses `index\_label` as the column name in the table. Creates a table index for this column.

index_label : str or sequence, default None
Column label for index column(s). If None is given (default) and `index` is True, then the index names are used.
A sequence should be given if the DataFrame uses MultiIndex.

chunksize : int, optional
Specify the number of rows in each batch to be written to the database connection at a time. By default, all rows will be written at once. Also see the method keyword.

dtype : dict or scalar, optional
Specifying the datatype for columns. If a dictionary is used, the keys should be the column names and the values should be the SQLAlchemy types or strings for the sqlite3 legacy mode. If a scalar is provided, it will be applied to all columns.

method : {None, 'multi', callable}, optional
Controls the SQL insertion clause used:

- * None : Uses standard SQL ``INSERT`` clause (one per row).
- * 'multi': Pass multiple values in a single ``INSERT`` clause.
- * callable with signature ``(pd\_table, conn, keys, data\_iter)``.

Details and a sample callable implementation can be found in the section :ref:`insert method <io.sql.method>`.

Returns

None or int

Number of rows affected by to_sql. None is returned if the callable passed into ``method`` does not return an integer number of rows.

The number of returned rows affected is the sum of the ``rowcount`` attribute of ``sqlite3.Cursor`` or SQLAlchemy connectable which may not reflect the exact number of written rows as stipulated in the `sqlite3` <https://docs.python.org/3/library/sqlite3.html#sqlite3.Cursor.rowcount> or SQLAlchemy <https://docs.sqlalchemy.org/en/20/core/connections.html#sqlalchemy.engine>.

Raises

ValueError

When the table already exists and `if\_exists` is 'fail' (the default).

See Also

read_sql : Read a DataFrame from a table.

Notes

Timezone aware datetime columns will be written as ``Timestamp with timezone`` type with SQLAlchemy if supported by the database. Otherwise, the datetimes will be stored as timezone unaware timestamps local to the original timezone.

Not all datastores support ``method="multi"``. Oracle, for example, does not support multi-value insert.

References

.. [1] <https://docs.sqlalchemy.org>
.. [2] <https://www.python.org/dev/peps/pep-0249/>

Examples

Create an in-memory SQLite database.

```
>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite://', echo=False)
```

Create a table from scratch with 3 rows.


```
>>> df = pd.DataFrame({'name' : ['User 1', 'User 2', 'User 3']})
>>> df
   name
0  User 1
1  User 2
2  User 3
```

```
>>> df.to_sql(name='users', con=engine)
3
>>> from sqlalchemy import text
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3')]
```

An `sqlalchemy.engine.Connection` can also be passed to `con`:

```
>>> with engine.begin() as connection:
...     df1 = pd.DataFrame({'name' : ['User 4', 'User 5']})
...     df1.to_sql(name='users', con=connection, if_exists='append')
2
```

This is allowed to support operations that require that the same DBAPI connection is used for the entire operation.

```
>>> df2 = pd.DataFrame({'name' : ['User 6', 'User 7']})
>>> df2.to_sql(name='users', con=engine, if_exists='append')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 1'), (1, 'User 2'), (2, 'User 3'),
 (0, 'User 4'), (1, 'User 5'), (0, 'User 6'),
 (1, 'User 7')]
```

Overwrite the table with just `df2`.

```
>>> df2.to_sql(name='users', con=engine, if_exists='replace',
...           index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 6'), (1, 'User 7')]
```

Delete all rows before inserting new records with `df3`.

```
>>> df3 = pd.DataFrame({"name": ['User 8', 'User 9']})
>>> df3.to_sql(name='users', con=engine, if_exists='delete_rows',
...           index_label='id')
2
>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM users")).fetchall()
[(0, 'User 8'), (1, 'User 9')]
```

Use `method` to define a callable insertion method to do nothing if there's a primary key conflict on a table in a PostgreSQL database.

```
>>> from sqlalchemy.dialects.postgresql import insert
>>> def insert_on_conflict_nothing(table, conn, keys, data_iter):
...     # "a" is the primary key in "conflict_table"
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = insert(table.table).values(data).on_conflict_do_nothing(index_elements=["a"])
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F821
...                   method=insert_on_conflict_nothing) # doctest: +SKIP
0
```

For MySQL, a callable to update columns `b` and `c` if there's a conflict on a primary key.

```
>>> from sqlalchemy.dialects.mysql import insert # noqa: F811
>>> def insert_on_conflict_update(table, conn, keys, data_iter):
...     # update columns "b" and "c" on primary key conflict
...     data = [dict(zip(keys, row)) for row in data_iter]
...     stmt = (
...         insert(table.table)
...         .values(data)
...     )
```

```

...     stmt = stmt.on_duplicate_key_update(b=stmt.inserted.b, c=stmt.inserted.c)
...     result = conn.execute(stmt)
...     return result.rowcount
>>> df_conflict.to_sql(name="conflict_table", con=conn, if_exists="append", # noqa: F82
...                     method=insert_on_conflict_update) # doctest: +SKIP
2

```

Specify the dtype (especially useful for integers with missing values). Notice that while pandas is forced to store the data as floating point, the database supports nullable integers. When fetching the data with Python, we get back integer scalars.

```

>>> df = pd.DataFrame({"A": [1, None, 2]})
>>> df
   A
0  1.0
1  NaN
2  2.0

>>> from sqlalchemy.types import Integer
>>> df.to_sql(name='integers', con=engine, index=False,
...          dtype={"A": Integer()})
3

>>> with engine.connect() as conn:
...     conn.execute(text("SELECT * FROM integers")).fetchall()
[(1,), (None,), (2,)]

.. versionadded:: 2.2.0

```

pandas now supports writing via ADBC drivers

```

>>> df = pd.DataFrame({'name' : ['User 10', 'User 11', 'User 12']})
>>> df
   name
0  User 10
1  User 11
2  User 12

>>> from adbc_driver_sqlite import dbapi # doctest:+SKIP
>>> with dbapi.connect("sqlite://") as conn: # doctest:+SKIP
...     df.to_sql(name="users", con=conn)
3

```

`to_xarray(self)`

Return an xarray object from the pandas object.

Returns

xarray.DataArray or xarray.Dataset
Data in the pandas structure converted to Dataset if the object is a DataFrame, or a DataArray if the object is a Series.

See Also

DataFrame.to_hdf : Write DataFrame to an HDF5 file.
DataFrame.to_parquet : Write a DataFrame to the binary parquet format.

Notes

See the `xarray` docs <<https://xarray.pydata.org/en/stable/>>`__

Examples

```

>>> df = pd.DataFrame(
...     [
...         ("falcon", "bird", 389.0, 2),
...         ("parrot", "bird", 24.0, 2),
...         ("lion", "mammal", 80.5, 4),
...         ("monkey", "mammal", np.nan, 4),
...     ],
...     columns=["name", "class", "max_speed", "num_legs"],
... )
>>> df
   name  class  max_speed  num_legs
0  falcon   bird      389.0         2
1  parrot   bird       24.0         2

```

```

2   lion  mammal      80.5      4
3  monkey  mammal      NaN      4

>>> df.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:      (index: 4)
Coordinates:
  * index        (index) int64 32B 0 1 2 3
Data variables:
  name           (index) object 32B 'falcon' 'parrot' 'lion' 'monkey'
  class          (index) object 32B 'bird' 'bird' 'mammal' 'mammal'
  max_speed      (index) float64 32B 389.0 24.0 80.5 nan
  num_legs       (index) int64 32B 2 2 4 4

>>> df["max_speed"].to_xarray() # doctest: +SKIP
<xarray.DataArray 'max_speed' (index: 4)>
array([389. , 24. , 80.5, nan])
Coordinates:
  * index        (index) int64 0 1 2 3

>>> dates = pd.to_datetime(
...     ["2018-01-01", "2018-01-01", "2018-01-02", "2018-01-02"]
... )
>>> df_multiindex = pd.DataFrame(
...     {
...         "date": dates,
...         "animal": ["falcon", "parrot", "falcon", "parrot"],
...         "speed": [350, 18, 361, 15],
...     }
... )
>>> df_multiindex = df_multiindex.set_index(["date", "animal"])

>>> df_multiindex

```

date	animal	speed
2018-01-01	falcon	350
	parrot	18
2018-01-02	falcon	361
	parrot	15

```

>>> df_multiindex.to_xarray() # doctest: +SKIP
<xarray.Dataset>
Dimensions:      (date: 2, animal: 2)
Coordinates:
  * date         (date) datetime64[s] 2018-01-01 2018-01-02
  * animal       (animal) object 'falcon' 'parrot'
Data variables:
  speed          (date, animal) int64 350 18 361 15

```

```

truncate(
    self,
    before=None,
    after=None,
    axis: Axis | None = None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self

```

Truncate a Series or DataFrame before and after some index value.

This is a useful shorthand for boolean indexing based on index values above or below certain thresholds.

Parameters

```

-----
before : date, str, int
    Truncate all rows before this index value.
after  : date, str, int
    Truncate all rows after this index value.
axis   : {0 or 'index', 1 or 'columns'}, optional
    Axis to truncate. Truncates the index (rows) by default.
    For `Series` this parameter is unused and defaults to 0.
copy   : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.

```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since

pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html>`__ for more details.

Returns

type of caller
The truncated Series or DataFrame.

See Also

DataFrame.loc : Select a subset of a DataFrame by label.
DataFrame.iloc : Select a subset of a DataFrame by position.

Notes

If the index being truncated contains only datetime values, ``before`` and ``after`` may be specified as strings instead of Timestamps.

Examples

>>> df = pd.DataFrame(
... {
... "A": ["a", "b", "c", "d", "e"],
... "B": ["f", "g", "h", "i", "j"],
... "C": ["k", "l", "m", "n", "o"],
... },
... index=[1, 2, 3, 4, 5],
...)
>>> df
 A B C
1 a f k
2 b g l
3 c h m
4 d i n
5 e j o

>>> df.truncate(before=2, after=4)
 A B C
2 b g l
3 c h m
4 d i n

The columns of a DataFrame can be truncated.

>>> df.truncate(before="A", after="B", axis="columns")
 A B
1 a f
2 b g
3 c h
4 d i
5 e j

For Series, only rows can be truncated.

>>> df["A"].truncate(before=2, after=4)
2 b
3 c
4 d
Name: A, dtype: str

The index values in ``truncate`` can be datetimes or string dates.

>>> dates = pd.date_range("2016-01-01", "2016-02-01", freq="s")
>>> df = pd.DataFrame(index=dates, data={"A": 1})
>>> df.tail()
 A
2016-01-31 23:59:56 1
2016-01-31 23:59:57 1
2016-01-31 23:59:58 1
2016-01-31 23:59:59 1
2016-02-01 00:00:00 1

```
>>> df.truncate(
...     before=pd.Timestamp("2016-01-05"), after=pd.Timestamp("2016-01-10")
... ).tail()
      A
2016-01-09 23:59:56 1
2016-01-09 23:59:57 1
2016-01-09 23:59:58 1
2016-01-09 23:59:59 1
2016-01-10 00:00:00 1
```

Because the index is a `DatetimeIndex` containing only dates, we can specify `'before'` and `'after'` as strings. They will be coerced to `Timestamps` before truncation.

```
>>> df.truncate("2016-01-05", "2016-01-10").tail()
      A
2016-01-09 23:59:56 1
2016-01-09 23:59:57 1
2016-01-09 23:59:58 1
2016-01-09 23:59:59 1
2016-01-10 00:00:00 1
```

Note that `truncate` assumes a 0 value for any unspecified time component (midnight). This differs from partial string slicing, which returns any partially matching dates.

```
>>> df.loc["2016-01-05":"2016-01-10", :].tail()
      A
2016-01-10 23:59:55 1
2016-01-10 23:59:56 1
2016-01-10 23:59:57 1
2016-01-10 23:59:58 1
2016-01-10 23:59:59 1
```

```
tz_convert(
    self,
    tz,
    axis: Axis = 0,
    level=None,
    copy: bool | lib.NoDefault = <no_default>
) -> Self
    Convert tz-aware axis to target time zone.
```

Parameters

```
tz : str or tzinfo object or None
    Target time zone. Passing 'None' will convert to
    UTC and remove the timezone information.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
    The axis to convert
level : int, str, default None
    If axis is a MultiIndex, convert a specific level. Otherwise
    must be None.
copy : bool, default False
    This keyword is now ignored; changing its value will have no
    impact on the method.
```

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write` <https://pandas.pydata.org/docs/dev/user_guide/copy_on_write.html> for more details.

Returns

```
Series/DataFrame
    Object with time zone converted axis.
```

Raises

```
TypeError
    If the axis is tz-naive.
```

See Also

DataFrame.tz_localize: Localize tz-naive index of DataFrame to target time zone.
Series.tz_localize: Localize tz-naive index of Series to target time zone.

Examples

Change to another time zone:

```
>>> s = pd.Series(  
...     [1],  
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]),  
... )  
>>> s.tz_convert("Asia/Shanghai")  
2018-09-15 07:30:00+08:00    1  
dtype: int64
```

Pass None to convert to UTC and get a tz-naive index:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))  
>>> s.tz_convert(None)  
2018-09-14 23:30:00    1  
dtype: int64
```

```
tz_localize(  
    self,  
    tz,  
    axis: Axis = 0,  
    level=None,  
    copy: bool | lib.NoDefault = <no_default>,  
    ambiguous: TimeAmbiguous = 'raise',  
    nonexistent: TimeNonexistent = 'raise'  
) -> Self  
    Localize time zone naive index of a Series or DataFrame to target time zone.
```

This operation localizes the Index. To localize the values in a time zone naive Series, use :meth:`Series.dt.tz\_localize`.

Parameters

tz : str or tzinfo or None
 Time zone to localize. Passing ``None`` will remove the time zone information and preserve local time.
axis : {{0 or 'index', 1 or 'columns'}}, default 0
 The axis to localize
level : int, str, default None
 If axis is a MultiIndex, localize a specific level. Otherwise must be None.
copy : bool, default False
 This keyword is now ignored; changing its value will have no impact on the method.

.. deprecated:: 3.0.0

This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the `user guide on Copy-on-Write`_ for more details.

ambiguous : 'infer', bool, bool-ndarray, 'NaT', default 'raise'
 When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the `ambiguous` parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on order
- bool (or bool-ndarray) where True signifies a DST time, False designates a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : str, default 'raise'

A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST. Valid values are:

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

Returns

Series/DataFrame

Same type as the input, with time zone naive or aware index, depending on `tz`.

Raises

TypeError

If the TimeSeries is tz-aware and tz is not None.

See Also

Series.dt.tz_localize: Localize the values in a time zone naive Series.

Timestamp.tz_localize: Localize the Timestamp to a timezone.

Examples

Localize local times:

```
>>> s = pd.Series(
...     [1],
...     index=pd.DatetimeIndex(["2018-09-15 01:30:00"]),
... )
>>> s.tz_localize("CET")
2018-09-15 01:30:00+02:00    1
dtype: int64
```

Pass None to convert to tz-naive index and preserve local time:

```
>>> s = pd.Series([1], index=pd.DatetimeIndex(["2018-09-15 01:30:00+02:00"]))
>>> s.tz_localize(None)
2018-09-15 01:30:00    1
dtype: int64
```

Be careful with DST changes. When there is sequential data, pandas can infer the DST time:

```
>>> s = pd.Series(
...     range(7),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 02:00:00",
...             "2018-10-28 02:30:00",
...             "2018-10-28 03:00:00",
...             "2018-10-28 03:30:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous="infer")
2018-10-28 01:30:00+02:00    0
2018-10-28 02:00:00+02:00    1
2018-10-28 02:30:00+02:00    2
2018-10-28 02:00:00+01:00    3
2018-10-28 02:30:00+01:00    4
2018-10-28 03:00:00+01:00    5
2018-10-28 03:30:00+01:00    6
dtype: int64
```

In some cases, inferring the DST is impossible. In such cases, you can pass an ndarray to the ambiguous parameter to set the DST explicitly

```
>>> s = pd.Series(
...     range(3),
...     index=pd.DatetimeIndex(
...         [
...             "2018-10-28 01:20:00",
...             "2018-10-28 02:36:00",
...             "2018-10-28 03:46:00",
...         ]
...     ),
... )
>>> s.tz_localize("CET", ambiguous=np.array([True, True, False]))
2018-10-28 01:20:00+02:00    0
2018-10-28 02:36:00+02:00    1
2018-10-28 03:46:00+01:00    2
dtype: int64
```

If the DST transition causes nonexistent times, you can shift these dates forward or backward with a `timedelta` object or `'shift_forward'` or `'shift_backward'`.

```
>>> dti = pd.DatetimeIndex(
...     ["2015-03-29 02:30:00", "2015-03-29 03:30:00"], dtype="M8[ns]"
... )
>>> s = pd.Series(range(2), index=dti)
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_forward")
2015-03-29 03:00:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent="shift_backward")
2015-03-29 01:59:59.999999999+01:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
>>> s.tz_localize("Europe/Warsaw", nonexistent=pd.Timedelta("1h"))
2015-03-29 03:30:00+02:00    0
2015-03-29 03:30:00+02:00    1
dtype: int64
```

```
where(
    self,
    cond,
    other=<no_default>,
    *,
    inplace: bool = False,
    axis: Axis | None = None,
    level: Level | None = None
) -> Self
Replace values where the condition is False.
```

This method allows conditional replacement of values. Where the condition evaluates to True, the original values are retained; where it evaluates to False, values are replaced with corresponding entries from `'other'`.

Parameters

```
cond : bool Series/DataFrame, array-like, or callable
    Where 'cond' is True, keep the original value. Where
    False, replace with corresponding value from 'other'.
    If 'cond' is callable, it is computed on the Series/DataFrame and
    should return boolean Series/DataFrame or array. The callable must
    not change input Series/DataFrame (though pandas doesn't check it).
other : scalar, Series/DataFrame, or callable
    Entries where 'cond' is False are replaced with
    corresponding value from 'other'.
    If other is callable, it is computed on the Series/DataFrame and
    should return scalar or Series/DataFrame. The callable must not
    change input Series/DataFrame (though pandas doesn't check it).
    If not specified, entries will be filled with the corresponding
    NULL value ('np.nan' for numpy dtypes, 'pd.NA' for extension
    dtypes).
inplace : bool, default False
    Whether to perform the operation in place on the data.
axis : int, default None
    Alignment axis if needed. For 'Series' this parameter is
    unused and defaults to 0.
level : int, default None
    Alignment level if needed.
```


Returns

Series or DataFrame

When applied to a Series, the function will return a Series, and when applied to a DataFrame, it will return a DataFrame.

See Also

:func:`DataFrame.mask` : Return an object of same shape as caller.
:func:`Series.mask` : Return an object of same shape as caller.

Notes

The where method is an application of the if-then idiom. For each element in the caller, if ``cond`` is ``True`` the element is used; otherwise the corresponding element from ``other`` is used. If the axis of ``other`` does not align with axis of ``cond`` Series/DataFrame, the values of ``cond`` on misaligned index positions will be filled with False.

The signature for :func:`Series.where` or :func:`DataFrame.where` differs from :func:`numpy.where`. Roughly ``df1.where(m, df2)`` is equivalent to ``np.where(m, df1, df2)``.

For further details and examples see the ``where`` documentation in :ref:`indexing <indexing.where\_mask>`.

The dtype of the object takes precedence. The fill value is casted to the object's dtype, if this can be done losslessly.

Examples

```
>>> s = pd.Series(range(5))
>>> s.where(s > 0)
0    NaN
1    1.0
2    2.0
3    3.0
4    4.0
dtype: float64
>>> s.mask(s > 0)
0    0.0
1    NaN
2    NaN
3    NaN
4    NaN
dtype: float64

>>> s = pd.Series(range(5))
>>> t = pd.Series([True, False])
>>> s.where(t, 99)
0     0
1    99
2    99
3    99
4    99
dtype: int64
>>> s.mask(t, 99)
0    99
1     1
2    99
3    99
4    99
dtype: int64

>>> s.where(s > 1, 10)
0    10
1    10
2     2
3     3
4     4
dtype: int64
>>> s.mask(s > 1, 10)
0     0
1     1
2    10
3    10
```

```

3    10
4    10
dtype: int64

>>> df = pd.DataFrame(np.arange(10).reshape(-1, 2), columns=["A", "B"])
>>> df
   A  B
0  0  1
1  2  3
2  4  5
3  6  7
4  8  9
>>> m = df % 3 == 0
>>> df.where(m, -df)
   A  B
0  0 -1
1 -2  3
2 -4 -5
3  6 -7
4 -8  9
>>> df.where(m, -df) == np.where(m, df, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True
>>> df.where(m, -df) == df.mask(~m, -df)
   A  B
0  True  True
1  True  True
2  True  True
3  True  True
4  True  True

```

```

xs(
    self,
    key: IndexLabel,
    axis: Axis = 0,
    level: IndexLabel | None = None,
    drop_level: bool = True
) -> Self
    Return cross-section from the Series/DataFrame.

    This method takes a `key` argument to select data at a particular
    level of a MultiIndex.

    Parameters
    -----
    key : label or tuple of label
        Label contained in the index, or partially in a MultiIndex.
    axis : {0 or 'index', 1 or 'columns'}, default 0
        Axis to retrieve cross-section on.
    level : object, defaults to first n levels (n=1 or len(key))
        In case of a key partially contained in a MultiIndex, indicate
        which levels are used. Levels can be referred by label or position.
    drop_level : bool, default True
        If False, returns object with same levels as self.

    Returns
    -----
    Series or DataFrame
        Cross-section from the original Series or DataFrame
        corresponding to the selected index levels.

    See Also
    -----
    DataFrame.loc : Access a group of rows and columns
        by label(s) or a boolean array.
    DataFrame.iloc : Purely integer-location based indexing
        for selection by position.

    Notes
    -----
    `xs` can not be used to set values.

    MultiIndex Slicers is a generic way to get/set values on

```

any level or levels.
 It is a superset of `xs` functionality, see
[:ref:`MultiIndex Slicers <advanced.mi\\_slicers>`](#).

Examples

```
>>> d = {
...     "num_legs": [4, 4, 2, 2],
...     "num_wings": [0, 0, 2, 2],
...     "class": ["mammal", "mammal", "mammal", "bird"],
...     "animal": ["cat", "dog", "bat", "penguin"],
...     "locomotion": ["walks", "walks", "flies", "walks"],
... }
>>> df = pd.DataFrame(data=d)
>>> df = df.set_index(["class", "animal", "locomotion"])
>>> df
```

			num_legs	num_wings
class	animal	locomotion		
mammal	cat	walks	4	0
	dog	walks	4	0
	bat	flies	2	2
bird	penguin	walks	2	2

Get values at specified index

```
>>> df.xs("mammal")
          num_legs  num_wings
animal locomotion
cat      walks      4         0
dog      walks      4         0
bat      flies      2         2
```

Get values at several indexes

```
>>> df.xs(("mammal", "dog", "walks"))
num_legs      4
num_wings      0
Name: (mammal, dog, walks), dtype: int64
```

Get values at specified index and level

```
>>> df.xs("cat", level=1)
          num_legs  num_wings
class locomotion
mammal walks      4         0
```

Get values at several indexes and levels

```
>>> df.xs(("bird", "walks"), level=[0, "locomotion"])
          num_legs  num_wings
animal
penguin      2         2
```

Get values at specified column and axis

```
>>> df.xs("num_wings", axis=1)
class animal locomotion
mammal cat      walks      0
        dog      walks      0
        bat      flies      2
bird   penguin  walks      2
Name: num_wings, dtype: int64
```

Readonly properties inherited from `pandas.core.generic.NDFrame`:

flags

Get the properties associated with this pandas object.

The available flags are

* :attr:`Flags.allows\_duplicate\_labels`

See Also

`Flags` : Flags that apply to pandas objects.

`DataFrame.attrs` : Global metadata applying to this dataset.

Notes

"Flags" differ from "metadata". Flags reflect properties of the pandas object (the Series or DataFrame). Metadata refer to properties of the dataset, and should be stored in :attr:`DataFrame.attrs`.

Examples

```
>>> df = pd.DataFrame({"A": [1, 2]})
>>> df.flags
<Flags(allows_duplicate_labels=True)>
```

Flags can be get or set using ``.``

```
>>> df.flags.allows_duplicate_labels
True
>>> df.flags.allows_duplicate_labels = False
```

Or by slicing with a key

```
>>> df.flags["allows_duplicate_labels"]
False
>>> df.flags["allows_duplicate_labels"] = True
```

Data descriptors inherited from pandas.core.generic.NDFrame:

attrs

Dictionary of global attributes of this dataset.

.. warning::

attrs is experimental and may change without warning.

See Also

DataFrame.flags : Global flags applying to this object.

Notes

Many operations that create new datasets will copy ``attrs``. Copies are always deep so that changing ``attrs`` will only affect the present dataset. :func:`pandas.concat` and :func:`pandas.merge` will only copy ``attrs`` if all input datasets have the same ``attrs``.

Examples

For Series:

```
>>> ser = pd.Series([1, 2, 3])
>>> ser.attrs = {"A": [10, 20, 30]}
>>> ser.attrs
{'A': [10, 20, 30]}
```

For DataFrame:

```
>>> df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
>>> df.attrs = {"A": [10, 20, 30]}
>>> df.attrs
{'A': [10, 20, 30]}
```

Methods inherited from pandas.core.base.PandasObject:

__sizeof__(self) -> int

Generates the total memory usage for an object that returns either a value or Series of values

Methods inherited from pandas.core.accessor.DirNamesMixin:

__dir__(self) -> list[str]

Provide method name lookup and completion.

Notes

Only provide 'public' methods.

Readonly properties inherited from pandas.core.indexing.IndexingMixin:

at

Access a single value for a row/column label pair.

Similar to ``loc``, in that both provide label-based lookups. Use ``at`` if you only need to get or set a single value in a DataFrame or Series.

Raises

KeyError

If getting a value and 'label' does not exist in a DataFrame or Series.

ValueError

If row/column label pair is not a tuple or if any label from the pair is not a scalar for DataFrame.

If label is list-like (*excluding* NamedTuple) for Series.

See Also

DataFrame.at : Access a single value for a row/column pair by label.

DataFrame.iat : Access a single value for a row/column pair by integer position.

DataFrame.loc : Access a group of rows and columns by label(s).

DataFrame.iloc : Access a group of rows and columns by integer position(s).

Series.at : Access a single value by label.

Series.iat : Access a single value by integer position.

Series.loc : Access a group of rows by label(s).

Series.iloc : Access a group of rows by integer position(s).

Notes

See :ref:`Fast scalar value getting and setting <indexing.basics.get\_value>` for more details.

Examples

```
>>> df = pd.DataFrame(  
...     [[0, 2, 3], [0, 4, 1], [10, 20, 30]],  
...     index=[4, 5, 6],  
...     columns=["A", "B", "C"],  
... )  
>>> df
```

```
   A  B  C  
4  0  2  3  
5  0  4  1  
6 10 20 30
```

Get value at specified row/column pair

```
>>> df.at[4, "B"]  
np.int64(2)
```

Set value at specified row/column pair

```
>>> df.at[4, "B"] = 10  
>>> df.at[4, "B"]  
np.int64(10)
```

Get value within a Series

```
>>> df.loc[5].at["B"]  
np.int64(4)
```

iat

Access a single value for a row/column pair by integer position.

Similar to ``iloc``, in that both provide integer-based lookups. Use ``iat`` if you only need to get or set a single value in a DataFrame or Series.

Raises

IndexError
When integer position is out of bounds.

See Also

DataFrame.at : Access a single value for a row/column label pair.
DataFrame.loc : Access a group of rows and columns by label(s).
DataFrame.iloc : Access a group of rows and columns by integer position(s).

Examples

>>> df = pd.DataFrame(
... [[0, 2, 3], [0, 4, 1], [10, 20, 30]], columns=["A", "B", "C"]
...)
>>> df
 A B C
0 0 2 3
1 0 4 1
2 10 20 30

Get value at specified row/column pair

>>> df.iat[1, 2]
np.int64(1)

Set value at specified row/column pair

>>> df.iat[1, 2] = 10
>>> df.iat[1, 2]
np.int64(10)

Get value within a series

>>> df.loc[0].iat[1]
np.int64(2)

iloc

Purely integer-location based indexing for selection by position.

.. versionchanged:: 3.0

Callables which return a tuple are deprecated as input.

``.iloc[]`` is primarily integer position based (from ``0`` to ``length-1`` of the axis), but may also be used with a boolean array.

Allowed inputs are:

- An integer, e.g. ``5``.
- A list or array of integers, e.g. ``[4, 3, 0]``.
- A slice object with ints, e.g. ``1:7``.
- A boolean array.
- A ``callable`` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above). This is useful in method chains, when you don't have a reference to the calling object, but would like to base your selection on some value.
- A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. ``(0, 1)``.

``.iloc`` will raise ``IndexError`` if a requested indexer is out-of-bounds, except *slice* indexers which allow out-of-bounds indexing (this conforms with python/numpy *slice* semantics).

See more at :ref:`Selection by Position <indexing.integer>`.

See Also

DataFrame.iat : Fast integer location scalar accessor.
DataFrame.loc : Purely label-location based indexer for selection by label.
Series.iloc : Purely integer-location based indexing for selection by position.

Examples

```
>>> mydict = [
...     {"a": 1, "b": 2, "c": 3, "d": 4},
...     {"a": 100, "b": 200, "c": 300, "d": 400},
...     {"a": 1000, "b": 2000, "c": 3000, "d": 4000},
... ]
>>> df = pd.DataFrame(mydict)
>>> df
```

	a	b	c	d
0	1	2	3	4
1	100	200	300	400
2	1000	2000	3000	4000

****Indexing just the rows****

With a scalar integer.

```
>>> type(df.iloc[0])
<class 'pandas.Series'>
>>> df.iloc[0]
```

	a	b	c	d
0	1	2	3	4

Name: 0, dtype: int64

With a list of integers.

```
>>> df.iloc[[0]]
```

	a	b	c	d
0	1	2	3	4

```
>>> type(df.iloc[[0]])
<class 'pandas.DataFrame'>
```

```
>>> df.iloc[[0, 1]]
```

	a	b	c	d
0	1	2	3	4
1	100	200	300	400

With a `slice` object.

```
>>> df.iloc[:3]
```

	a	b	c	d
0	1	2	3	4
1	100	200	300	400
2	1000	2000	3000	4000

With a boolean mask the same length as the index.

```
>>> df.iloc[[True, False, True]]
```

	a	b	c	d
0	1	2	3	4
2	1000	2000	3000	4000

With a callable, useful in method chains. The `x` passed to the ``lambda`` is the DataFrame being sliced. This selects the rows whose index label even.

```
>>> df.iloc[lambda x: x.index % 2 == 0]
```

	a	b	c	d
0	1	2	3	4
2	1000	2000	3000	4000

****Indexing both axes****

You can mix the indexer types for the index and columns. Use ``:``` to select the entire axis.

With scalar integers.

```
>>> df.iloc[0, 1]
np.int64(2)
```

With lists of integers.

```
>>> df.iloc[[0, 2], [1, 3]]
```

	b	d
0	2	4

```
2 2000 4000
```

With ``slice`` objects.

```
>>> df.iloc[1:3, 0:3]
   a    b    c
1  100  200  300
2 1000 2000 3000
```

With a boolean array whose length matches the columns.

```
>>> df.iloc[:, [True, False, True, False]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

With a callable function that expects the Series or DataFrame.

```
>>> df.iloc[:, lambda df: [0, 2]]
   a    c
0   1    3
1  100  300
2 1000 3000
```

`loc`

Access a group of rows and columns by label(s) or a boolean array.

``.loc[]`` is primarily label based, but may also be used with a boolean array.

Allowed inputs are:

- A single label, e.g. ``5`` or ``a``, (note that ``5`` is interpreted as a `*label*` of the index, and `**never**` as an integer position along the index).
- A list or array of labels, e.g. ``['a', 'b', 'c']``.
- A slice object with labels, e.g. ``a':'f``.

.. warning:: Note that contrary to usual python slices, `**both**` the start and the stop are included

- A boolean array of the same length as the axis being sliced, e.g. ``[True, False, True]``.
- An alignable boolean Series. The index of the key will be aligned before masking.
- An alignable Index. The Index of the returned selection will be the input.
- A ``callable`` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above)

See more at :ref:`Selection by Label <indexing.label>`.

Raises

KeyError

If any items are not found.

IndexingError

If an indexed key is passed and its index is unalignable to the frame index.

See Also

DataFrame.at : Access a single value for a row/column label pair.

DataFrame.iloc : Access group of rows and columns by integer position(s).

DataFrame.xs : Returns a cross-section (row(s) or column(s)) from the Series/DataFrame.

Series.loc : Access group of values using labels.

Examples

`**Getting values**`

```
>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=["cobra", "viper", "sidewinder"],
...     columns=["max_speed", "shield"],
... )
>>> df
```


	max_speed	shield
cobra	1	2
viper	4	5
sidewinder	7	8

Single label. Note this returns the row as a Series.

```
>>> df.loc["viper"]
max_speed    4
shield       5
Name: viper, dtype: int64
```

List of labels. Note using ``[]`` returns a DataFrame.

```
>>> df.loc[["viper", "sidewinder"]]
      max_speed  shield
viper         4       5
sidewinder    7       8
```

Single label for row and column

```
>>> df.loc["cobra", "shield"]
np.int64(2)
```

Slice with labels for row and single label for column. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc["cobra":"viper", "max_speed"]
cobra    1
viper    4
Name: max_speed, dtype: int64
```

Boolean list with the same length as the row axis

```
>>> df.loc[[False, False, True]]
      max_speed  shield
sidewinder    7       8
```

Alignable boolean Series:

```
>>> df.loc[
...     pd.Series([False, True, False], index=["viper", "sidewinder", "cobra"])
... ]
      max_speed  shield
sidewinder    7       8
```

Index (same behavior as ``df.reindex``)

```
>>> df.loc[pd.Index(["cobra", "viper"], name="foo")]
      max_speed  shield
foo
cobra         1       2
viper         4       5
```

Conditional that returns a boolean Series

```
>>> df.loc[df["shield"] > 6]
      max_speed  shield
sidewinder    7       8
```

Conditional that returns a boolean Series with column labels specified

```
>>> df.loc[df["shield"] > 6, ["max_speed"]]
      max_speed
sidewinder    7
```

Multiple conditional using ``&`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 1) & (df["shield"] < 8)]
      max_speed  shield
viper         4       5
```

Multiple conditional using ``|`` that returns a boolean Series

```
>>> df.loc[(df["max_speed"] > 4) | (df["shield"] < 5)]
      max_speed  shield
cobra         1       2
```

```
sidewinder          7      8
```

Please ensure that each condition is wrapped in parentheses ``()``.
See the :ref:`user guide<indexing.boolean>`
for more details and explanations of Boolean indexing.

.. note::

If you find yourself using 3 or more conditionals in ``.loc[]``,
consider using :ref:`advanced indexing<advanced.advanced\_hierarchical>`.

See below for using ``.loc[]`` on MultiIndex DataFrames.

Callable that returns a boolean Series

```
>>> df.loc[lambda df: df["shield"] == 8]
      max_speed  shield
sidewinder      7      8
```

****Setting values****

Set value for all items matching the list of labels

```
>>> df.loc[["viper", "sidewinder"], ["shield"]] = 50
>>> df
      max_speed  shield
cobra           1      2
viper           4     50
sidewinder       7     50
```

Set value for an entire row

```
>>> df.loc["cobra"] = 10
>>> df
      max_speed  shield
cobra         10     10
viper          4     50
sidewinder      7     50
```

Set value for an entire column

```
>>> df.loc[:, "max_speed"] = 30
>>> df
      max_speed  shield
cobra         30     10
viper         30     50
sidewinder     30     50
```

Set value for rows matching callable condition

```
>>> df.loc[df["shield"] > 35] = 0
>>> df
      max_speed  shield
cobra         30     10
viper          0      0
sidewinder      0      0
```

Add value matching location

```
>>> df.loc["viper", "shield"] += 5
>>> df
      max_speed  shield
cobra         30     10
viper          0      5
sidewinder      0      0
```

Setting using a ``Series`` or a ``DataFrame`` sets the values matching the
index labels, not the index positions.

```
>>> shuffled_df = df.loc[["viper", "cobra", "sidewinder"]]
>>> df.loc[:] += shuffled_df
>>> df
      max_speed  shield
cobra         60     20
viper          0     10
sidewinder      0      0
```

****Getting values on a DataFrame with an index that has integer labels****

Another example using integers for the index

```
>>> df = pd.DataFrame(
...     [[1, 2], [4, 5], [7, 8]],
...     index=[7, 8, 9],
...     columns=["max_speed", "shield"],
... )
>>> df
   max_speed  shield
7          1       2
8          4       5
9          7       8
```

Slice with integer labels for rows. As mentioned above, note that both the start and stop of the slice are included.

```
>>> df.loc[7:9]
   max_speed  shield
7          1       2
8          4       5
9          7       8
```

****Getting values with a MultiIndex****

A number of examples using a DataFrame with a MultiIndex

```
>>> tuples = [
...     ("cobra", "mark i"),
...     ("cobra", "mark ii"),
...     ("sidewinder", "mark i"),
...     ("sidewinder", "mark ii"),
...     ("viper", "mark ii"),
...     ("viper", "mark iii"),
... ]
>>> index = pd.MultiIndex.from_tuples(tuples)
>>> values = [[12, 2], [0, 4], [10, 20], [1, 4], [7, 1], [16, 36]]
>>> df = pd.DataFrame(values, columns=["max_speed", "shield"], index=index)
>>> df
               max_speed  shield
cobra  mark i          12       2
       mark ii           0       4
sidewinder mark i         10      20
          mark ii          1       4
viper    mark ii          7       1
          mark iii         16      36
```

Single label. Note this returns a DataFrame with a single index.

```
>>> df.loc["cobra"]
   max_speed  shield
mark i       12       2
mark ii       0       4
```

Single index tuple. Note this returns a Series.

```
>>> df.loc[("cobra", "mark ii")]
max_speed    0
shield       4
Name: (cobra, mark ii), dtype: int64
```

Single label for row and column. Similar to passing in a tuple, this returns a Series.

```
>>> df.loc["cobra", "mark i"]
max_speed    12
shield        2
Name: (cobra, mark i), dtype: int64
```

Single tuple. Note using ``[[]]`` returns a DataFrame.

```
>>> df.loc[["cobra", "mark ii"]]
               max_speed  shield
cobra mark ii           0       4
```

Single tuple for the index with a single label for the column

```
>>> df.loc[("cobra", "mark i"), "shield"]
np.int64(2)
```

Slice from index tuple to single label

```
>>> df.loc[("cobra", "mark i") : "viper"]
           max_speed  shield
cobra      mark i      12      2
           mark ii      0      4
sidewinder mark i      10     20
           mark ii      1      4
viper      mark ii      7      1
           mark iii     16     36
```

Slice from index tuple to index tuple

```
>>> df.loc[("cobra", "mark i") : ("viper", "mark ii")]
           max_speed  shield
cobra      mark i      12      2
           mark ii      0      4
sidewinder mark i      10     20
           mark ii      1      4
viper      mark ii      7      1
```

Please see the :ref:`user guide<advanced.advanced\_hierarchical>` for more details and explanations of advanced indexing.

****Assignment with Series****

When assigning a Series to `.loc[row_indexer, col_indexer]`, pandas aligns the Series by index labels, not by order or position.

Series assignment with `.loc` and index alignment:

```
>>> df = pd.DataFrame({"A": [1, 2, 3]}, index=[0, 1, 2])
>>> s = pd.Series([10, 20], index=[1, 0]) # Note reversed order
>>> df.loc[:, "B"] = s # Aligns by index, not order
>>> df
   A  B
0  1  20.0
1  2  10.0
2  3   NaN
```

FUNCTIONS

```
assert_almost_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = 'equiv',
    rtol: float = 1e-05,
    atol: float = 1e-08,
    **kwargs
)
```

-> None
Check that the left and right objects are approximately equal.

By approximately equal, we refer to objects that are numbers or that contain numbers which may be equivalent to specific levels of precision.

Parameters

left : object

right : object

check_dtype : bool or {'equiv'}, default 'equiv'

Check dtype if both a and b are the same type. If 'equiv' is passed in, then `RangeIndex` and `Index` with int64 dtype are also considered equivalent when doing type checking.

rtol : float, default 1e-5

Relative tolerance.

atol : float, default 1e-8

Absolute tolerance.

```
assert_attr_equal(attr: str, left, right, obj: str = 'Attributes') -> None
Check attributes are equal. Both objects must have attribute.
```

Parameters

attr : str

Attribute name being compared.

```

    left : object
    right : object
    obj : str, default 'Attributes'
        Specify object name being compared, internally used to show appropriate
        assertion message

assert_categorical_equal(
    left,
    right,
    check_dtype: bool = True,
    check_category_order: bool = True,
    obj: str = 'Categorical'
) -> None
    Test that Categoricals are equivalent.

Parameters
-----
left : Categorical
right : Categorical
check_dtype : bool, default True
    Check that integer dtype of the codes are the same.
check_category_order : bool, default True
    Whether the order of the categories should be compared, which
    implies identical integer codes. If False, only the resulting
    values are compared. The ordered attribute is
    checked regardless.
obj : str, default 'Categorical'
    Specify object name being compared, internally used to show appropriate
    assertion message.

assert_class_equal(left, right, exact: bool | str = True, obj: str = 'Input') -> None
    Checks classes are equal.

assert_contains_all(iterable, dic) -> None

assert_copy(iter1, iter2, **eq_lkwargs) -> None
    iter1, iter2: iterables that produce elements
    comparable with assert_almost_equal

    Checks that the elements are equal, but not
    the same object. (Does not check that items
    in sequences are also not the same object)

assert_datetime_array_equal(
    left,
    right,
    obj: str = 'DatetimeArray',
    check_freq: bool = True
) -> None

assert_dict_equal(left, right, compare_keys: bool = True) -> None

assert_equal(left, right, **kwargs) -> None
    Wrapper for tm.assert_*_equal to dispatch to the appropriate test function.

Parameters
-----
left, right : Index, Series, DataFrame, ExtensionArray, or np.ndarray
    The two items to be compared.
**kwargs
    All keyword arguments are passed through to the underlying assert method.

assert_extension_array_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = True,
    index_values=None,
    check_exact: bool | lib.NoDefault = <no_default>,
    rtol: float | lib.NoDefault = <no_default>,
    atol: float | lib.NoDefault = <no_default>,
    obj: str = 'ExtensionArray'
) -> None
    Check that left and right ExtensionArrays are equal.

    This method compares two ``ExtensionArray`` instances for equality,
    including checks for missing values, the dtype of the arrays, and
    the exactness of the comparison (or tolerance when comparing floats).

```

Parameters

left, right : ExtensionArray
 The two arrays to compare.
check_dtype : bool, default True
 Whether to check if the ExtensionArray dtypes are identical.
index_values : Index | numpy.ndarray, default None
 Optional index (shared by both left and right), used in output.
check_exact : bool, default False
 Whether to compare number exactly.

.. versionchanged:: 2.2.0

 Defaults to True for integer dtypes if none of
 `check\_exact`, `rtol` and `atol` are specified.
rtol : float, default 1e-5
 Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
 Absolute tolerance. Only used when check_exact is False.
obj : str, default 'ExtensionArray'
 Specify object name being compared, internally used to show appropriate
 assertion message.

.. versionadded:: 2.0.0

See Also

testing.assert_series_equal : Check that left and right ``Series`` are equal.
testing.assert_frame_equal : Check that left and right ``DataFrame`` are equal.
testing.assert_index_equal : Check that left and right ``Index`` are equal.

Notes

Missing values are checked separately from valid values.
A mask of missing values is computed for each and checked to match.
The remaining all-valid values are cast to object dtype and checked.

Examples

>>> from pandas import testing as tm
>>> a = pd.Series([1, 2, 3, 4])
>>> b, c = a.array, a.array
>>> tm.assert_extension_array_equal(b, c)

```
assert_frame_equal(  
    left,  
    right,  
    check_dtype: bool | Literal['equiv'] = True,  
    check_index_type: bool | Literal['equiv'] = 'equiv',  
    check_column_type: bool | Literal['equiv'] = 'equiv',  
    check_frame_type: bool = True,  
    check_names: bool = True,  
    by_blocks: bool = False,  
    check_exact: bool | lib.NoDefault = <no_default>,  
    check_datetimelike_compat: bool = False,  
    check_categorical: bool = True,  
    check_like: bool = False,  
    check_freq: bool = True,  
    check_flags: bool = True,  
    rtol: float | lib.NoDefault = <no_default>,  
    atol: float | lib.NoDefault = <no_default>,  
    obj: str = 'DataFrame'  
) -> None  
Check that left and right DataFrame are equal.
```

This function is intended to compare two DataFrames and output any differences. It is mostly intended for use in unit tests. Additional parameters allow varying the strictness of the equality checks performed.

Parameters

left : DataFrame
 First DataFrame to compare.
right : DataFrame
 Second DataFrame to compare.
check_dtype : bool, default True

```

    Whether to check the DataFrame dtype is identical.
check_index_type : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical.
check_column_type : bool or {'equiv'}, default 'equiv'
    Whether to check the columns class, dtype and inferred_type
    are identical. Is passed as the ``exact`` argument of
    :func:`assert_index_equal`.
check_frame_type : bool, default True
    Whether to check the DataFrame class is identical.
check_names : bool, default True
    Whether to check that the `names` attribute for both the `index`
    and `column` attributes of the DataFrame is identical.
by_blocks : bool, default False
    Specify how to compare internal data. If False, compare by columns.
    If True, compare by blocks.
check_exact : bool, default False
    Whether to compare number exactly. If False, the comparison uses the
    relative tolerance (``rtol``) and absolute tolerance (``atol``)
    parameters to determine if two values are considered close,
    according to the formula:  $|a - b| \leq (atol + rtol * |b|)$ .

.. versionchanged:: 2.2.0

    Defaults to True for integer dtypes if none of
    ``check_exact``, ``rtol`` and ``atol`` are specified.
check_datetimelike_compat : bool, default False
    Compare datetime-like which is comparable ignoring dtype.

.. deprecated:: 3.0

check_categorical : bool, default True
    Whether to compare internal Categorical exactly.
check_like : bool, default False
    If True, ignore the order of index & columns.
    Note: index labels must match their respective rows
    (same as in columns) - same labels must be with the same data.
check_freq : bool, default True
    Whether to check the `freq` attribute on a DatetimeIndex or TimedeltaIndex.
check_flags : bool, default True
    Whether to check the `flags` attribute.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'DataFrame'
    Specify object name being compared, internally used to show appropriate
    assertion message.

```

See Also

```

-----
assert_series_equal : Equivalent method for asserting Series equality.
DataFrame.equals : Check DataFrame equality.

```

Examples

```

-----
This example shows comparing two DataFrames that are equal
but with columns of differing dtypes.

>>> from pandas.testing import assert_frame_equal
>>> df1 = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
>>> df2 = pd.DataFrame({"a": [1, 2], "b": [3.0, 4.0]})

df1 equals itself.

>>> assert_frame_equal(df1, df1)

df1 differs from df2 as column 'b' is of a different type.

>>> assert_frame_equal(df1, df2)
Traceback (most recent call last):
...
AssertionError: Attributes of DataFrame.iloc[:, 1] (column name="b") are different

Attribute "dtype" are different
[left]: int64
[right]: float64

```

Ignore differing dtypes in columns with check_dtype.

```
>>> assert_frame_equal(df1, df2, check_dtype=False)
```

```
assert_index_equal(
    left: Index,
    right: Index,
    exact: bool | str = 'equiv',
    check_names: bool = True,
    check_exact: bool = True,
    check_categorical: bool = True,
    check_order: bool = True,
    rtol: float = 1e-05,
    atol: float = 1e-08,
    obj: str | None = None
) -> None
Check that left and right Index are equal.

Parameters
-----
left : Index
    The first index to compare.
right : Index
    The second index to compare.
exact : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical. If 'equiv', then RangeIndex can be substituted for
    Index with an int64 dtype as well.
check_names : bool, default True
    Whether to check the names attribute.
check_exact : bool, default True
    Whether to compare number exactly.
check_categorical : bool, default True
    Whether to compare internal Categorical exactly.
check_order : bool, default True
    Whether to compare the order of index entries as well as their values.
    If True, both indexes must contain the same elements, in the same order.
    If False, both indexes must contain the same elements, but in any order.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'Index' or 'MultiIndex'
    Specify object name being compared, internally used to show appropriate
    assertion message.
```

See Also

```
-----
testing.assert_series_equal : Check that two Series are equal.
testing.assert_frame_equal : Check that two DataFrames are equal.
```

Examples

```
-----
>>> from pandas import testing as tm
>>> a = pd.Index([1, 2, 3])
>>> b = pd.Index([1, 2, 3])
>>> tm.assert_index_equal(a, b)
```

```
assert_indexing_slices_equivalent(ser: Series, l_slc: slice, i_slc: slice) -> None
Check that ser.iloc[i_slc] matches ser.loc[l_slc] and, if applicable,
ser[l_slc].
```

```
assert_interval_array_equal(
    left,
    right,
    exact: bool | Literal['equiv'] = 'equiv',
    obj: str = 'IntervalArray'
) -> None
Test that two IntervalArrays are equivalent.
```

Parameters

```
-----
left, right : IntervalArray
    The IntervalArrays to compare.
exact : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical. If 'equiv', then RangeIndex can be substituted for
```



```

        Index with an int64 dtype as well.
    obj : str, default 'IntervalArray'
        Specify object name being compared, internally used to show appropriate
        assertion message

assert_is_sorted(seq) -> None
    Assert that the sequence is sorted.

assert_metadata_equivalent(
    left: DataFrame | Series,
    right: DataFrame | Series | None = None
) -> None
    Check that ._metadata attributes are equivalent.

assert_numpy_array_equal(
    left,
    right,
    strict_nan: bool = False,
    check_dtype: bool | Literal['equiv'] = True,
    err_msg=None,
    check_same=None,
    obj: str = 'numpy array',
    index_values=None
) -> None
    Check that 'np.ndarray' is equivalent.

Parameters
-----
left, right : numpy.ndarray or iterable
    The two arrays to be compared.
strict_nan : bool, default False
    If True, consider NaN and None to be different.
check_dtype : bool, default True
    Check dtype if both a and b are np.ndarray.
err_msg : str, default None
    If provided, used as assertion message.
check_same : None|'copy'|'same', default None
    Ensure left and right refer/do not refer to the same memory area.
obj : str, default 'numpy array'
    Specify object name being compared, internally used to show appropriate
    assertion message.
index_values : Index | numpy.ndarray, default None
    optional index (shared by both left and right), used in output.

assert_period_array_equal(left, right, obj: str = 'PeriodArray') -> None

assertProducesWarning(
    expected_warning: type[Warning] | bool | tuple[type[Warning], ...] | None = <class 'Warning'>,
    filter_level: Literal['error', 'ignore', 'always', 'default', 'module', 'once'] = 'always',
    check_stacklevel: bool = True,
    raise_on_extra_warnings: bool = True,
    match: str | tuple[str | None, ...] | None = None,
    must_find_all_warnings: bool = True
) -> Generator[list[warnings.WarningMessage]]
    Context manager for running code expected to either raise a specific warning,
    multiple specific warnings, or not raise any warnings. Verifies that the code
    raises the expected warning(s), and that it does not raise any other unexpected
    warnings. It is basically a wrapper around ``warnings.catch_warnings``.

Parameters
-----
expected_warning : {Warning, False, tuple[Warning, ...], None}, default Warning
    The type of Exception raised. ``exception.Warning`` is the base
    class for all warnings. To raise multiple types of exceptions,
    pass them as a tuple. To check that no warning is returned,
    specify ``False`` or ``None``.
filter_level : str or None, default "always"
    Specifies whether warnings are ignored, displayed, or turned
    into errors.
    Valid values are:

    * "error" - turns matching warnings into exceptions
    * "ignore" - discard the warning
    * "always" - always emit a warning
    * "default" - print the warning the first time it is generated
    from each location
    * "module" - print the warning the first time it is generated

```

```

    from each module
    * "once" - print the warning the first time it is generated

check_stacklevel : bool, default True
    If True, displays the line that called the function containing
    the warning to show where the function is called. Otherwise, the
    line that implements the function is displayed.
raise_on_extra_warnings : bool, default True
    Whether extra warnings not of the type `expected_warning` should
    cause the test to fail.
match : {str, tuple[str, ...]}, optional
    Match warning message. If it's a tuple, it has to be the size of
    `expected_warning`. If additionally `must_find_all_warnings` is
    True, each expected warning's message gets matched with a respective
    match. Otherwise, multiple values get treated as an alternative.
must_find_all_warnings : bool, default True
    If True and `expected_warning` is a tuple, each expected warning
    type must get encountered. Otherwise, even one expected warning
    results in success.

```

Examples

```

-----
>>> import warnings
>>> with assert_produces_warning():
...     warnings.warn(UserWarning())
>>> with assert_produces_warning(False):
...     warnings.warn(RuntimeWarning())
Traceback (most recent call last):
...
AssertionError: Caused unexpected warning(s): ['RuntimeWarning'].
>>> with assert_produces_warning(UserWarning):
...     warnings.warn(RuntimeWarning())
Traceback (most recent call last):
...
AssertionError: Did not see expected warning of class 'UserWarning'.

..warn:: This is *not* thread-safe.

```

```

assert_series_equal(
    left,
    right,
    check_dtype: bool | Literal['equiv'] = True,
    check_index_type: bool | Literal['equiv'] = 'equiv',
    check_series_type: bool = True,
    check_names: bool = True,
    check_exact: bool | lib.NoDefault = <no_default>,
    check_datetimelike_compat: bool = False,
    check_categorical: bool = True,
    check_category_order: bool = True,
    check_freq: bool = True,
    check_flags: bool = True,
    rtol: float | lib.NoDefault = <no_default>,
    atol: float | lib.NoDefault = <no_default>,
    obj: str = 'Series',
    *,
    check_index: bool = True,
    check_like: bool = False
) -> None
    Check that left and right Series are equal.

```

Parameters

```

-----
left : Series
    First Series to compare.
right : Series
    Second Series to compare.
check_dtype : bool, default True
    Whether to check the Series dtype is identical.
check_index_type : bool or {'equiv'}, default 'equiv'
    Whether to check the Index class, dtype and inferred_type
    are identical.
check_series_type : bool, default True
    Whether to check the Series class is identical.
check_names : bool, default True
    Whether to check the Series and Index names attribute.
check_exact : bool, default False
    Whether to compare number exactly. This also applies when checking

```

```

Index equivalence.

.. versionchanged:: 2.2.0

    Defaults to True for integer dtypes if none of
    ``check_exact``, ``rtol`` and ``atol`` are specified.

.. versionchanged:: 3.0.0

    check_exact for comparing the Indexes defaults to True by
    checking if an Index is of integer dtypes.

check_datetimelike_compat : bool, default False
    Compare datetime-like which is comparable ignoring dtype.

.. deprecated:: 3.0

check_categorical : bool, default True
    Whether to compare internal Categorical exactly.
check_category_order : bool, default True
    Whether to compare category order of internal Categoricals.
check_freq : bool, default True
    Whether to check the `freq` attribute on a DatetimeIndex or TimedeltaIndex.
check_flags : bool, default True
    Whether to check the `flags` attribute.
rtol : float, default 1e-5
    Relative tolerance. Only used when check_exact is False.
atol : float, default 1e-8
    Absolute tolerance. Only used when check_exact is False.
obj : str, default 'Series'
    Specify object name being compared, internally used to show appropriate
    assertion message.
check_index : bool, default True
    Whether to check index equivalence. If False, then compare only values.
check_like : bool, default False
    If True, ignore the order of the index. Must be False if check_index is False.
    Note: same labels must be with the same data.

See Also
-----
testing.assert_index_equal : Check that two Indexes are equal.
testing.assert_frame_equal : Check that two DataFrames are equal.

Examples
-----
>>> from pandas import testing as tm
>>> a = pd.Series([1, 2, 3, 4])
>>> b = pd.Series([1, 2, 3, 4])
>>> tm.assert_series_equal(a, b)

assert_sp_array_equal(left, right) -> None
    Check that the left and right SparseArray are equal.

Parameters
-----
left : SparseArray
right : SparseArray

assert_timedelta_array_equal(
    left,
    right,
    obj: str = 'TimedeltaArray',
    check_freq: bool = True
) -> None

at(x)

box_expected(expected, box_cls, transpose: bool = True)
    Helper function to wrap the expected output of a test in a given box_class.

Parameters
-----
expected : np.ndarray, Index, Series
box_cls : {Index, Series, DataFrame}

Returns
-----

```

```

subclass of box_cls

can_set_locale(lc: str, lc_var: int = 0) -> bool
    Check to see if we can set a locale, and subsequently get the locale,
    without raising an Exception.

Parameters
-----
lc : str
    The locale to attempt to set.
lc_var : int, default `locale.LC_ALL`
    The category of the locale being set.

Returns
-----
bool
    Whether the passed locale can be set

convert_rows_list_to_csv_str(rows_list: list[str]) -> str
    Convert list of CSV rows to single CSV-formatted string for current OS.

    This method is used for creating expected value of to_csv() method.

Parameters
-----
rows_list : List[str]
    Each element represents the row of csv.

Returns
-----
str
    Expected output of to_csv() in current OS.

decompress_file(path: FilePath | BaseBuffer, compression: CompressionOptions) -> Generator[IO[bytes]]
    Open a compressed file and return a file object.

Parameters
-----
path : str
    The path where the file is read from.

compression : {'gzip', 'bz2', 'zip', 'xz', 'zstd', None}
    Name of the decompression to use

Returns
-----
file object

external_error_raised(expected_exception: type[Exception]) -> ContextManager
    Helper function to mark pytest.raises that have an external error message.

Parameters
-----
expected_exception : Exception
    Expected error to raise.

Returns
-----
Callable
    Regular `pytest.raises` function with `match` equal to `None`.

get_cython_table_params(ndframe, func_names_and_expected)
    Combine frame, functions from com._cython_table
    keys and expected result.

Parameters
-----
ndframe : DataFrame or Series
func_names_and_expected : Sequence of two items
    The first item is a name of an NDFrame method ('sum', 'prod') etc.
    The second item is the expected return value.

Returns
-----
list
    List of three items (DataFrame, function, expected result)
get_dtype(obj) -> DtypeObj

```

```

get_finetest_unit(left: str, right: str) -> str
    Find the higher of two datetime64 units.

get_locales(prefix: str | None = None, normalize: bool = True) -> list[str]
    Get all the locales that are available on the system.

    Parameters
    -----
    prefix : str
        If not ``None`` then return only those locales with the prefix
        provided. For example to get all English language locales (those that
        start with ``"en"``), pass ``prefix="en"``.
    normalize : bool
        Call ``locale.normalize`` on the resulting list of available locales.
        If ``True``, only locales that can be set without throwing an
        ``Exception`` are returned.

    Returns
    -----
    locales : list of strings
        A list of locale strings that can be set with ``locale.setlocale()``.
        For example::

            locale.setlocale(locale.LC_ALL, locale_string)

    On error will return an empty list (no locale available, e.g. Windows)

get_obj(df: DataFrame, klass)
    For sharing tests using frame_or_series, either return the DataFrame
    unchanged or return it's first column as a Series.

get_op_from_name(op_name: str) -> Callable
    The operator function for a given op name.

    Parameters
    -----
    op_name : str
        The op name, in form of "add" or "__add__".

    Returns
    -----
    function
        A function performing the operation.

getitem(x)

iat(x)

iloc(x)

loc(x)

maybe_produces_warning(warning: type[Warning], condition: bool, **kwargs) -> AbstractContextManager
    Return a context manager that possibly checks a warning based on the condition

raise_assert_detail(
    obj,
    message,
    left,
    right,
    diff=None,
    first_diff=None,
    index_values=None
) -> NoReturn

raises_chained_assignment_error(extra_warnings=(), extra_match=())

round_trip_pathlib(writer, reader, tmp_path: Path)
    Write an object to file specified by a pathlib.Path and read it back

    Parameters
    -----
    writer : callable bound to pandas object
        IO writing function (e.g. DataFrame.to_csv )
    reader : callable
        IO reading function (e.g. pd.read_csv )
    path : str, default None

```

The path where the object is written and then read.

Returns

pandas object

The original object that was serialized and then re-read.

round_trip_pickle(obj: Any, tmp_path: Path) -> DataFrame | Series
Pickle an object and then read it again.

Parameters

obj : any object

The object to pickle and then re-read.

path : str, path object or file-like object, default None

The path where the pickled object is written and then read.

Returns

pandas object

The original object that was pickled and then re-read.

run_multithreaded(closure, max_workers, arguments=None, pass_barrier=False)

set_locale(new_locale: str | tuple[str, str], lc_var: int = 0) -> Generator[str | tuple[str, str]]
Context manager for temporarily setting a locale.

Parameters

new_locale : str or tuple

A string of the form <language_country>.<encoding>. For example to set the current locale to US English with a UTF8 encoding, you would pass "en_US.UTF-8".

lc_var : int, default `locale.LC\_ALL`

The category of the locale being set.

Notes

This is useful when you want to run a particular block of code under a particular locale, without globally setting the locale. This probably isn't thread-safe.

set_timezone(tz: str) -> Generator[None]
Context manager for temporarily setting a timezone.

Parameters

tz : str

A string representing a valid timezone.

Examples

>>> from datetime import datetime

>>> from dateutil.tz import tzlocal

>>> tzlocal().tzname(datetime(2021, 1, 1)) # doctest: +SKIP
'IST'

>>> with set_timezone("US/Eastern"):

... tzlocal().tzname(datetime(2021, 1, 1))

'EST'

setitem(x)

shares_memory(left, right) -> bool
Pandas-compatible for np.shares_memory.

to_array(obj)
Similar to pd.array, but does not cast numpy dtypes to nullable dtypes.

with_csv_dialect(name: str, **kwargs) -> Generator[None]
Context manager to temporarily register a CSV dialect for parsing CSV.

Parameters

name : str

The name of the dialect.

kwargs : mapping

The parameters for the dialect.

Raises

ValueError : the name of the dialect conflicts with a builtin one.

See Also

csv : Python's CSV library.

write_to_compressed(compression, path: str, data, dest: str = 'test') -> None
Write data to a compressed file.

Parameters

compression : {'gzip', 'bz2', 'zip', 'xz', 'zstd'}
The compression type to use.
path : str
The file path to write the data.
data : str
The data to write.
dest : str, default "test"
The destination file (for ZIP only)

Raises

ValueError : An invalid compression value was passed in.

DATA

```
ALL_INT_EA_DTYPES = ['UInt8', 'UInt16', 'UInt32', 'UInt64', 'Int8', 'I...
ALL_INT_NUMPY_DTYPES = ['uint8', 'uint16', 'uint32', 'uint64', '<class ...
ALL_NUMPY_DTYPES = [<class 'float'>, 'float32', 'float64', 'uint8', 'u...
ALL_REAL_NUMPY_DTYPES = [<class 'float'>, 'float32', 'float64', 'uint8...
BOOL_DTYPES = [<class 'bool'>, 'bool']
BYTES_DTYPES = [<class 'bytes'>, 'bytes']
COMPLEX_DTYPES = [<class 'complex'>, 'complex64', 'complex128']
DATETIME64_DTYPES = ['datetime64[ns]', 'M8[ns]']
ENDIAN = '<'
FLOAT_EA_DTYPES = ['Float32', 'Float64']
FLOAT_NUMPY_DTYPES = [<class 'float'>, 'float32', 'float64']
NARROW_NP_DTYPES = [<class 'numpy.float16'>, <class 'numpy.float32'>, ...
NP_NAT_OBJECTS = [np.datetime64('NaT', 'Y'), np.datetime64('NaT', 'M'), ...
NULL_OBJECTS = [None, nan, NaT, nan, <NA>, Decimal('NaN')]
OBJECT_DTYPES = [<class 'object'>, 'object']
SIGNED_INT_EA_DTYPES = ['Int8', 'Int16', 'Int32', 'Int64']
SIGNED_INT_NUMPY_DTYPES = [<class 'int'>, 'int8', 'int16', 'int32', 'i...
STRING_DTYPES = ['U']
TIMEDelta64_DTYPES = ['timedelta64[ns]', 'm8[ns]']
UNSIGNED_INT_EA_DTYPES = ['UInt8', 'UInt16', 'UInt32', 'UInt64']
UNSIGNED_INT_NUMPY_DTYPES = ['uint8', 'uint16', 'uint32', 'uint64']
__all__ = ['ALL_INT_EA_DTYPES', 'ALL_INT_NUMPY_DTYPES', 'ALL_NUMPY_DTY...
__annotations__ = {'ALL_FLOAT_DTYPES': 'list[Dtype]', 'ALL_INT_DTYPES'...
__conditional_annotations__ = set()
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas_testing__init.

Help on module pandas._typing in pandas:

NAME

pandas._typing

CLASSES

builtins.object
builtins.type

```
type_t = class type(object)
| type(object) -> the object's type
| type(name, bases, dict, **kwargs) -> a new type
|
| Methods defined here:
|
| __call__(self, /, *args, **kwargs)
|     Call self as a function.
|
| __delattr__(self, name, /)
```

```

    Implement setattr(self, name).

__dir__(self, /)
    Specialized __dir__ implementation for types.

__getattr__(self, name, /)
    Return getattr(self, name).

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__instancecheck__(self, instance, /)
    Check if an object is an instance.

__or__(self, value, /)
    Return self|value.

__repr__(self, /)
    Return repr(self).

__ror__(self, value, /)
    Return value|self.

__setattr__(self, name, value, /)
    Implement setattr(self, name, value).

__sizeof__(self, /)
    Return memory consumption of the type object.

__subclasscheck__(self, subclass, /)
    Check if a class is a subclass.

__subclasses__(self, /)
    Return a list of immediate subclasses.

mro(self, /)
    Return a type's method resolution order.

-----
Class methods defined here:

__prepare__(name, bases, /, **kwds)
    Create the namespace for the class statement

-----
Static methods defined here:

__new__(*args, **kwargs)
    Create and return a new object. See help(type) for accurate signature.

-----
Data descriptors defined here:

__abstractmethods__

__annotate__

__annotations__

__dict__

__text_signature__

-----
Data and other attributes defined here:

__base__ = <class 'object'>
    The base class of the class hierarchy.

    When called, it accepts no arguments and returns a new featureless
    instance that has no instance attributes and cannot be given any.

__bases__ = (<class 'object'>,)

__basicsize__ = 936

```



```
|  __dictoffset__ = 264
|  __flags__ = 2155896066
|  __itemsized__ = 40
|  __mro__ = (<class 'type'>, <class 'object'>)
|  __type_params__ = ()
|  __weakrefoffset__ = 368
```

DATA

```
__all__ = ['type_t']
__annotations__ = {'AggFuncType': 'TypeAlias', 'AggFuncTypeBase': 'Typ...
__conditional_annotations__ = set()
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\pandas\_typing.py
```